GPGPU – általános célú számítások GPU-n

CUDA-programozástól a TPU-kig és az OpenCL-ig – memória, teljesítmény és párhuzamos algoritmusok

Gellai István

2026

Tartalomjegyzék

Impresszum **7**

I. RÉSZ – Alapok és GPU-architektúra **8**

1 Előszó és olvasói útmutató **9**
1.1 Előszó: a könyv célja, célközönsége és a szerző 9
1.2 Hogyan használd a könyvet: tanulási utak, laborok és jelölések 15

2 A GPGPU térképe és a CUDA ökoszisztéma **22**
2.1 GPGPU és CPU: miért más a teljesítménymodell? 22
2.2 A CUDA ökoszisztéma és a hardvergenerációk 28
2.3 Az akcelerátorok térképe: GPU, TPU, FPGA és mások 36
2.4 Mikor éri meg GPU-ra vinni egy feladatot? 43

3 GPU-architektúra: SIMT, warp, SM és memória **52**
3.1 SIMT és warp-végrehajtás 52
3.2 Streaming multiprocessor és végrehajtási egységek 61
3.3 Memóriahierarchia: regiszter, shared, L1/L2, global 67
3.4 GPU-generációk és compute capability 78

4 CUDA fejlesztőkörnyezet és az első kernel **90**
4.1 A CUDA toolkit és a driver telepítése 90
4.2 Az első CUDA kernel: vektorösszeadás 98
4.3 Build, futtatás és egyszerű hibakeresés 104
4.4 Az első mérés: CPU vs. GPU 114

5 CUDA execution model: thread, block, grid és warp **126**
5.1 Thread, block, grid és a hardverre képezés 126
5.2 Warp-szintű végrehajtás és divergence 133
5.3 Launch bounds, blockméret és gridméret 139
5.4 A végrehajtási modell korlátai 147

II. RÉSZ – CUDA programozási modell **155**

6 Kernelírás, indexelés és launch konfiguráció **156**
6.1 Globális indexszámítás 1D-ben 156
6.2 Többdimenziós indexelés 163
6.3 Grid-stride loop és perzisztens kernelek 174
6.4 Launch konfiguráció és hibakezelés 180

7 Memóriaterek: global, shared, local, constant és texture **191**
7.1 Global memory és allokáció 191
7.2 Shared memory és élettartam 199
7.3 Local, constant és texture memória 205
7.4 Memóriaterek kiválasztása 214

8 Szinkronizáció, atomics és warp intrinsics **227**
8.1 Blokk-szintű szinkronizáció . . . 227
8.2 Atomi műveletek . . . 236
8.3 Warp intrinsics: shuffle, vote és match . . . 244
8.4 Memory fence és láthatóság . . . 252

9 Cooperative groups, thread block clusters és grid sync **261**
9.1 Cooperative groups alapok . . . 261
9.2 Thread block clusters . . . 269
9.3 Grid-szintű szinkronizáció . . . 278
9.4 Összetett együttműködési minták . . . 285

10 Hibakezelés, debugging és compute-sanitizer **296**
10.1 CUDA-hibakódok és ellenőrzés . . . 296
10.2 cuda-gdb és kernel-debugging . . . 303
10.3 Compute-sanitizer: memcheck és racecheck . . . 311
10.4 Tipikus CUDA-hibák és megelőzésük . . . 321

III. RÉSZ – Teljesítmény és memória **332**

11 Coalescing, bank conflicts és vectorizált hozzáférés **333**
11.1 Globális memória coalescing . . . 333
11.2 Shared memory bankok és bank conflict . . . 339
11.3 Vektorizált betöltés és tárolás . . . 348
11.4 Memóriaaccess-minták mérése . . . 355

12 Occupancy, register pressure és latency hiding **364**
12.1 Occupancy és erőforrás-korlátok . . . 364
12.2 Regiszterhasználat és spill . . . 373
12.3 Latency hiding . . . 381
12.4 Occupancy optimalizálás . . . 391

13 Roofline, sávszélesség és compute intensity **403**
13.1 A Roofline-modell . . . 403
13.2 Memóriateljesítmény és sávszélesség . . . 412
13.3 Számítási korlát és compute intensity . . . 420
13.4 Roofline-alapú optimalizálás . . . 429

14 Shared memory tiling, async copy és pipelining **441**
14.1 Shared memory tiling . . . 441
14.2 Aszinkron másolás (cp.async) . . . 447
14.3 Double buffering és pipeline . . . 455
14.4 Tiling és pipeline mérése . . . 464

15 Cache, L2, persistence és memóriamodellek **474**
15.1 L1 és L2 cache . . . 474
15.2 Cache hint-ek és hozzáférési szándék . . . 481
15.3 L2 persistence és access policy . . . 489
15.4 CUDA memóriamodell és scope . . . 498

IV. RÉSZ – Párhuzamos algoritmusok **507**

16 Redukciók, scan és histogram **508**
16.1 Redukciók . . . 508
16.2 Scan (prefix sum) . . . 516
16.3 Histogram . . . 524
16.4 CUB és Thrust használata . . . 533

17 Rendezés, keresés és selection **546**
17.1 Radix sort . . . 546
17.2 Bitonic és egyéb rendezések . . . 552
17.3 Keresés és binary search . . . 561
17.4 Selection és order statistics . . . 569

18 Stencil, PDE és iteratív megoldók **577**
18.1 Stencil-alapok . . . 577
18.2 Halo-csere és többblokkos stencil . . . 585
18.3 PDE és iteratív megoldók . . . 596
18.4 Stencil-optimalizálás . . . 603

19 Sparse és dense lineáris algebra **615**
19.1 Dense GEMM . . . 615
19.2 Sparse mátrixreprezentációk . . . 623
19.3 SpMV és sparse optimalizálás . . . 633
19.4 Könyvtári és saját megoldás . . . 645

20 FFT, Monte Carlo és random számok **655**
20.1 FFT GPU-n . . . 655
20.2 Monte Carlo szimuláció . . . 662
20.3 cuRAND és random számgenerálás . . . 670
20.4 Sztochasztikus algoritmusok . . . 677

21 Párhuzamos minták és teljesítménymodellek **685**
21.1 Adatpárhuzamos minták . . . 685
21.2 Feladatpárhuzamosság és terheléselosztás . . . 693
21.3 Teljesítménymodellek . . . 700
21.4 Mintaválasztás és összetétel . . . 708

V. RÉSZ – CUDA-ökoszisztéma és mélytanulás **719**

22 Könyvtárak: cuBLAS, cuSPARSE, cuFFT, cuRAND, Thrust, CUB **720**
22.1 Könyvtártérkép és döntési szempontok . . . 720
22.2 cuBLAS és cuSPARSE használata . . . 729
22.3 Thrust és CUB . . . 741
22.4 Könyvtárak integrálása . . . 752

23 CUTLASS, tensor cores és mixed precision **766**
23.1 Tensor Core architektúra . . . 766
23.2 CUTLASS . . . 772
23.3 Mixed precision . . . 779
23.4 Tensor Core optimalizálás . . . 789

24 Deep learning kernelek: GEMM, conv és attention **797**
24.1 GEMM és a deep learning . . . 797
24.2 Convolution kernelek . . . 806

24.3 Attention kernelek . . . 813
24.4 Memória és pontosság . . . 822

25 Streams, events, CUDA graphs és aszinkron végrehajtás 832
25.1 Streamek és események . . . 832
25.2 Adatmozgatás és számítás átfedése . . . 842
25.3 CUDA graphs . . . 849
25.4 Aszinkron tervezési minták . . . 856

26 Multi-GPU, NCCL, NVLink és unified memory 867
26.1 Több GPU és peer access . . . 867
26.2 NCCL és kollektív műveletek . . . 880
26.3 NVLink, NVSwitch és GPUDirect . . . 886
26.4 Unified memory és MIG . . . 893

VI. RÉSZ – TPU, OpenCL és más modellek 902

27 TPU architektúra és XLA 903
27.1 TPU-architektúra . . . 903
27.2 XLA és a gráffordítás . . . 910
27.3 TPU programozási modell . . . 918
27.4 TPU vs. GPU . . . 927

28 JAX és TPU programozás 936
28.1 JAX alapok és a funkcionális modell . . . 936
28.2 Vmap, pmap és sharding . . . 943
28.3 TPU-n futó JAX . . . 950
28.4 JAX teljesítmény és korlátok . . . 957

29 OpenCL programozási modell 964
29.1 OpenCL platform és device modell . . . 964
29.2 Context, queue és memóriaobjektumok . . . 973
29.3 Kernel és launch . . . 983
29.4 OpenCL és CUDA összehasonlítás . . . 991

30 OpenCL kernelek, memória és hordozhatóság 1001
30.1 OpenCL C és kernelírás . . . 1001
30.2 Work-group, barrier és local memory . . . 1008
30.3 Hordozhatóság és teljesítmény . . . 1017
30.4 SYCL és HIP kitekintés . . . 1028

VII. RÉSZ – Mérnökség és záróprojekt 1039

31 Profilozás, benchmarkolás és energia 1040
31.1 Nsight Compute és a kernelmetrikák . . . 1040
31.2 Nsight Systems és a rendszerszintű profil . . . 1046
31.3 Benchmarkmódszertan . . . 1055
31.4 Energia és hatékonyság . . . 1063

32 Numerikus pontosság, determinizmus és hordozhatóság 1071
32.1 Lebegőpontos pontosság . . . 1071
32.2 Determinizmus és redukciók . . . 1079
32.3 Hordozhatóság: HIP és SYCL . . . 1086

32.4 A GPGPU jövője 1094

33 Záróprojekt: optimalizált GPGPU rendszer **1102**
33.1 Követelmények és architektúra 1102
33.2 Kernelimplementáció 1114
33.3 Optimalizálás és mérés 1122
33.4 Validáció és dokumentáció 1132

Függelékek **1144**
Fogalomtár (magyar–angol) 1144
CUDA API- és kernelkatalógus 1148
GPU-, profilozó- és build-eszközkatalógus 1151
Kompetenciatérkép és tanulási útvonal 1154
Bibliográfia és forráskulcsok 1156

Impresszum

- **Szerző:** Gellai István
- **Szerkesztő:** Gellai István
- **Kiadó:** Tananyag Simple
- **Kiadás:** 1.0.0 – 2026
- **Licenc:** MIT
- **Forráskód:** chapters/ és lab/

A teljes licencszöveg a tároló gyökerének `LICENSE` fájljában található (a könyvkönyvtárból: `../../LICENSE`).

I. RÉSZ – Alapok és GPU-architektúra

1 Előszó és olvasói útmutató

1.1 Előszó: a könyv célja, célközönsége és a szerző

Amikor egy C++-fejlesztő először szembesül azzal, hogy egy gondosan megírt CPU-kódja a GPU-n lassabb, mint az eredeti, a csalódás ritkán a hardver hibája. A GPU nem varázsszer, hanem egy másik végrehajtási modell, más memóriaterekkel, más ütemezéssel és más erőforrás-korlátokkal. Ez a könyv azért született, hogy ezt a modellt ne szigetelt trükkökként, hanem egységes mérnöki szemléletként tanítsa meg: hardveres ok, szoftveres következmény, méréssel igazolva.

A könyv technikai gerince a CUDA C++, de a cél nem csupán a CUDA API megtanítása. A GPGPU-programozás lényege az adatpárhuzamos gondolkodás, az erőforrás-tudatosság és a reprodukálható teljesítménymérés. Aki ezt elsajátítja, az könnyebben lép át OpenCL, SYCL, HIP, illetve TPU/XLA/JAX környezetbe is. Aki viszont csak API-hívásokat memorizál, az hardvergeneráció-váltáskor újra elveszik.

A könyv motivációja

A klasszikus CPU-teljesítmény növekedése évtizedeken át az órajel-emelésből és az egyre mélyebb szuperskalár végrehajtásból táplálkozott. Ez a modell elérte a fizikai és energetikai korlátait: a teljesítmény már nem növelhető érdemben az egy szálra jutó energia növelése nélkül. Az ipari válasz a párhuzamosítás lett. A GPU ebben a térben különösen erős ajánlat, mert nagy számú, viszonylag egyszerű végrehajtóegységet és nagy memóriasávszélességet kínál, kifejezetten adatpárhuzamos terhelésekhez.

A GPU programozása azonban nem egyszerűen „vektorizálás nagy léptékben”. A modern GPU-k ezernyi threadet ütemeznek csoportokban; a memóriahozzáférések akkor gyorsak, ha az egy csoporthoz tartozó hozzáférések összeolvadnak; a shared memory néhány tíz kilobájtos bankokra oszlik; az atomi műveletek és a szinkronizáció láthatósági szabályai eltérnek a CPU-s memóriamodelltől. Ha a programozó ezeket figyelmen kívül hagyja, a program helyes lehet, de a sávszélesség- vagy késleltetés-korlát miatt töredék teljesítményt ad.

A szakirodalom gyakran két szélsőség között mozog: az egyik a hardveres dokumentáció, amely a részleteket ismerteti, de nem vezeti a mérnököt a döntésekhez; a másik a rövid példagyűjtemény, amely működő kódokat ad, de nem magyarázza el, miért pont az a megoldás a jó. Ez a könyv a középső réteget célozza: minden fejezetben egy konkrét programozási vagy teljesítményproblémából indul ki, bevezeti a szükséges hardveres fogalmakat, majd a megoldást méréssel és hibakereséssel együtt tárgyalja.

A célközönség

A könyv elsősorban azoknak szól, akik C++11/14/17-ben otthonosan mozognak, és a GPU-t általános célú számításra akarják használni. Ide tartoznak:

- tudományos számítással, szimulációval vagy numerikus modellezéssel foglalkozó mérnökök;
- jelfeldolgozó, képfeldolgozó vagy kommunikációs rendszerek fejlesztői;
- gépi tanulási rendszereket üzemeltető vagy fejlesztő mérnökök;
- teljesítményorientált C++-rendszerek fejlesztői, akik a CPU–GPU heterogén végrehajtást tervezik.

Feltételezzük a pointerek és a dinamikus memóriakezelés rutinját, a sablonok alapvető használatát, valamint alapvető lineáris algebrai ismereteket: vektorok, mátrixok, skaláris szorzat, mátrixszorzás. A CUDA-hoz nem szükséges előzetes GPU-tapasztalat; a párhuzamos gondolkodást a könyv fokozatosan építi fel. Ugyanakkor ez nem általános C++-bevezető, és nem hardvertervezői kézikönyv. Aki még sosem kezelt pointert vagy nem írt függvénysablont, az előbb C++-alapozást igényel. Aki pedig inkább hardveres tranzisztorszintű részleteket keres, az a gyártói architektúraleírásokban jobban jár.

A szerző szakmai háttere

A szerző szakmai pályája C++-os rendszerfejlesztésből indult, majd nagy számításigényű ipari projektekben dolgozott: végeselemes és véges térfogatos szimulációk, jelfeldolgozó csővezetékek, pénzügyi Monte Carlo kernelek és gépi tanulási rendszerek GPU-sítása. Több generációs NVIDIA hardveren, többek között Tesla, Ampere és Hopper architektúrájú eszközökön mért és hangolt kerneleket. Egyetemi oktatóként és vállalati mentorként is tanítja a párhuzamos gondolkodást.

Ez a gyakorlati háttér határozza meg a könyv hangvételét. A szöveg nem elégszik meg azzal, hogy egy kód „lefordul”; azt is kérdezi, hogy miért akkora a futásidő, mekkora az occupancy, hol a memóriabottleneck, és hogyan igazolható a javulás. A példák ezért mindig tartalmazzák a környezetet, a buildparancsot, a futtatási parancsot és a mérési naplót. A nem futtatott állítások külön jelölést kapnak. A cél nem az illúzió, hogy a GPU mindent felgyorsít, hanem az a mérnöki képesség, hogy meg tudjuk mondani, mikor és milyen áron gyorsítható egy feladat.

A GPGPU mint teljesítmény- és programozási terület

A GPGPU (General-Purpose computing on Graphics Processing Units) alatt a grafikus processzorok általános célú, nem grafikai számításokra való felhasználását értjük. A GPU-kat eredetileg egymástól független képpontok párhuzamos feldolgozására tervezték; ez a kialakítás szerencsésen egybeesik a numerikus magok nagy adatpárhuzamosságával. A mai GPU-k ezért több ezer lebegőpontos egységet és több száz GB/s memóriasávszélességet kínálnak.

A teljesítményük azonban nem a névleges csúcson, hanem a kihasználtságon múlik. A kulcskérdés nem az, hogy hány FLOPs-ot tud a hardver elméletben, hanem az, hogy a programunk hány FLOPs-ot tud hasznosítani a memóriából ténylegesen beérkező adatokon. Ezért a könyvben a GPU-t nemcsak számítási, hanem memória- és ütemezési rendszerként is vizsgáljuk.

Programozási területként a GPGPU a következő rétegeket kapcsolja össze:

- a párhuzamos algoritmusok tervezése: hogyan bontsunk egy feladatot threadre, blockra, gridre;
- a memóriaterek kiválasztása: global, shared, local, constant, texture, register;
- a szinkronizáció és láthatóság: blokk-szintű barrierektől a grid-szintű együttműködésig;
- a hardveres ütemezés: warp, occupancy, latency hiding;
- a mérés és profilozás: Nsight, benchmarkok, roofline.

Teljesítményterületként a GPGPU az átbocsátóképességet, a sávszélességet, a foglaltságot és az energiahatékonyságot vizsgálja. A „gyorsabb” állítás ebben a könyvben mindig konkrét inputméretre, precisionre és baseline-ra vonatkozik. Egyetlen szám nem elég.

Három olvasói út

A könyv fejezetei egymásra épülnek, de nem minden olvasónak kell ugyanazt a sorrendet követnie. Három fő tanulási utat javaslunk.

Az első út a **CUDA-alkalmazásfejlesztőé**. Ő elsősorban működő, megbízható CUDA-programokat akar írni. Neki az I. rész adja az architekturális alapokat, a II. rész a programozási

modellt és a hibakeresést, az V. rész pedig a kész könyvtárak hatékony használatát. A cél: biztonságos kernel, helyes indexelés, hibaellenőrzés, alapvető memóriatér-választás.

A második út a **teljesítménymérnöké**. Ő a maximumot keresi, ezért az I–II. rész után a III. részből a memória-hozzáférés és occupancy, a IV. részből a párhuzamos algoritmusok, az V. részből a tensor core és a streamek, a VII. részből a profilozás és benchmarkolás a fontos. A cél: mérésekkel alátámasztott optimalizálás, roofline-tudatosság.

A harmadik út a **heterogén platformok és ML-rendszerek mérnökéé**, aki a CUDA mellett TPU-t, XLA-t, JAX-ot vagy OpenCL-t is használ. Neki az I. rész megadja a közös fogalmi alapot, az V. rész a NVIDIA-ökoszisztémát, a VI. rész a TPU/OpenCL modelleket, a VII. rész pedig a hordozhatósági és pontossági kérdéseket. A cél: platformok közötti döntésképesség.

Ezek az utak nem zárják ki egymást. A legtöbb olvasó az elsőt és a másodikat kombinálja; aki ML-infrastruktúrán dolgozik, az a harmadikat is beemeli.

Labor: személyes tanulási cél és kiindulási szint meghatározása

Az első labor célja nem egy bonyolult algoritmus, hanem a referenciapont rögzítése. Ha most nem rögzíti a környezetet, a mérési módszert és a jelenlegi kiindulási szintet, később nem tudja eldönteni, hogy egy változtatás tényleg gyorsított-e, vagy csak a rendszer háttérterhelése változott.

Hajtsa végre az alábbi lépéseket.

1. **Fogalmazza meg a tanulási célt egy mondatban.** Például: „a jelenlegi CPU-s szimulációm legforróbb kernelét GPU-ra szeretném vinni”, vagy „meg akarom érteni, hogyan érik el a Tensor Core-k a nagy GEMM-teljesítményt”. A cél legyen konkrét, mérhető és a könyv időkeretébe illeszkedő.

2. **Töltse ki a kiindulási szint önértékelését.** Jelölje 1-től 5-ig, mennyire érzi stabilnak az alábbi területeket:

   - C++ pointerek, dinamikus memória, RAII, move szemantika;
   - többszálú programozás, atomi műveletek, szinkronizáció;
   - lineáris algebra: mátrix-szorzás, sajátérték, LU/Cholesky;
   - hardware-közeli programozás, cache-tudatosság, vektorizáció;
   - teljesítménymérés, profilozás, benchmarkmódszertan.

3. **Rögzítse a platformot.** A mérési naplóban adja meg a GPU modelljét, a driver verzióját, a CUDA-verziót, az operációs rendszert és a compute capabilityt. Az ellenőrző parancsok:

   - `nvidia-smi --query-gpu=name,driver_version,memory.total,power.limit --format=csv`
   - `nvcc --version`
   - `uname -a`

   Ha a pontos CUDA-verzió és driver a kiadáskor változik, a mérés előtt ellenőrizze az aktuális stabil verziót.

4. **Fordítsa és futtassa az alábbi CUDA C++17 példát.** Az első alkalmas hardveres mérés legyen egyszerű, de teljes: hibaellenőrzéssel, warm-up-pal és CUDA-event-alapú időméréssel.

```cpp
// CUDA C++17, nvcc
// Build: nvcc -std=c++17 -O3 -arch=native -o intro_lab intro_lab.cu
// Futtatás: ./intro_lab
#include <cstdio>
#include <cstdlib>
#include <vector>
#include <cuda_runtime.h>

#define CUDA_CHECK(call) do { \
    cudaError_t err = (call); \
    if (err != cudaSuccess) { \
        std::fprintf(stderr, "CUDA error at %s:%d: %s\n", \
                     __FILE__, __LINE__, cudaGetErrorString(err)); \
        std::exit(EXIT_FAILURE); \
    } \
} while (0)

__global__ void saxpy_kernel(int n, float alpha,
                             const float* __restrict__ x,
                             float* __restrict__ y) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        y[i] = alpha * x[i] + y[i];
    }
}

int main() {
    int device = 0;
    CUDA_CHECK(cudaSetDevice(device));

    cudaDeviceProp prop{};
    CUDA_CHECK(cudaGetDeviceProperties(&prop, device));
    std::printf("GPU: %s\n", prop.name);
    std::printf("Compute capability: %d.%d\n", prop.major, prop.minor);
    std::printf("SMs: %d\n", prop.multiProcessorCount);
    std::printf("Global memory: %.2f GiB\n",
                static_cast<double>(prop.totalGlobalMem) / (1024.0 * 1024.0 * 1024.0));

    const int n = 1 << 20;
    std::vector<float> x(n, 1.0f);
    std::vector<float> y(n, 2.0f);

    float *d_x = nullptr;
    float *d_y = nullptr;
    CUDA_CHECK(cudaMalloc(&d_x, static_cast<size_t>(n) * sizeof(float)));
    CUDA_CHECK(cudaMalloc(&d_y, static_cast<size_t>(n) * sizeof(float)));
    CUDA_CHECK(cudaMemcpy(d_x, x.data(), static_cast<size_t>(n) * sizeof(float),
                          cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_y, y.data(), static_cast<size_t>(n) * sizeof(float),
                          cudaMemcpyHostToDevice));

    const int block = 256;
    const int grid = (n + block - 1) / block;

    // Warm-up: az első futtatás inicializálási költséget is tartalmazhat.
    saxpy_kernel<<<grid, block>>>(n, 2.0f, d_x, d_y);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    cudaEvent_t start, stop;
    CUDA_CHECK(cudaEventCreate(&start));
    CUDA_CHECK(cudaEventCreate(&stop));
```

```
    const int reps = 10;
    CUDA_CHECK(cudaEventRecord(start));
    for (int rep = 0; rep < reps; ++rep) {
        saxpy_kernel<<<grid, block>>>(n, 2.0f, d_x, d_y);
        CUDA_CHECK(cudaGetLastError());
    }
    CUDA_CHECK(cudaEventRecord(stop));
    CUDA_CHECK(cudaEventSynchronize(stop));

    float elapsed_ms = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&elapsed_ms, start, stop));

    double avg_ms = static_cast<double>(elapsed_ms) / reps;
    double bytes = 3.0 * static_cast<double>(n) * sizeof(float);
    double seconds = avg_ms / 1000.0;
    double gbytes_per_sec = bytes / seconds / 1e9;

    std::printf("SAXPY n=%d, reps=%d\n", n, reps);
    std::printf("Average time: %.3f ms\n", avg_ms);
    std::printf("Effective bandwidth: %.1f GB/s\n", gbytes_per_sec);

    CUDA_CHECK(cudaFree(d_x));
    CUDA_CHECK(cudaFree(d_y));
    CUDA_CHECK(cudaEventDestroy(start));
    CUDA_CHECK(cudaEventDestroy(stop));
    return 0;
}
```

A kód néhány fontos eleme:

- A `CUDA_CHECK` makró minden API-hívás után ellenőrzi a visszatérési értéket. A kernelindítás után a `cudaGetLastError` hívja le az aszinkron hibát.
- A `cudaEventRecord` a streambe illesztett időbélyeg; a `cudaEventSynchronize` biztosítja, hogy a CPU valóban megvárja a stop eseményt, mielőtt az eltelt időt kiolvasná.
- A sávszélesség-képlet `3 * n * sizeof(float)` bájttal számol, mert a SAXPY minden elemnél három memóriaműveletet végez: `x` olvasása, `y` olvasása, majd `y` írása. Ez az első gyakori csapda: a `2 * n * sizeof(float)` képlet elhagyja az egyik olvasást.

Ha a kiindulási szintet OpenCL vagy JAX oldalon szeretné meghatározni, a platform-rögzítést hasonlóan végezze. OpenCL esetén a `clinfo` parancs adja a platform- és device-információkat; JAX esetén pedig az alábbi rövid ellenőrző kód futtatható:

```
# JAX 0.4.x, Python 3.10+
import jax
import jax.numpy as jnp

print("JAX version:", jax.__version__)
print("Devices:", jax.devices())

x = jnp.ones(1 << 20, dtype=jnp.float32)
y = jnp.full_like(x, 2.0)
f = jax.jit(lambda a, x, y: a * x + y)
print(f(2.0, x, y).block_until_ready())
```

A pontos JAX-verziót a futtatás előtt ellenőrizni kell, mert a JAX- és XLA-fejlesztés gyorsan változik.

Mérési napló és elkülönített várt eredmény

A labor átadandó eredménye egy mérési napló, amely a mért és a várt eredményt külön sorban tartalmazza. Példa a formátumra:

Mező	Rögzítendő érték
GPU modell	pl. NVIDIA GeForce RTX 4060
Driver verzió	pl. 550.54.14
CUDA verzió	pl. 12.4
Compute capability	pl. 8.9
Operációs rendszer	pl. Ubuntu 22.04
Power limit / ECC / MIG	pl. 115 W / nincs / nincs
Háttérterhelés	pl. nincs, `nvidia-smi` kimenet csatolva
Inputméret	`n = 1 048 576 float`
Warm-up	1 futtatás
Ismétlések száma	10
Átlag futásidő	pl. 0,087 ms
Szórás vagy percentilis	pl. 0,003 ms / p95 0,091 ms
Névleges memória-sávszélesség	pl. 272 GB/s (gyártói adatlap, ellenőrizendő)
Mért effektív sávszélesség	pl. 144 GB/s
Várt effektív sávszélesség	pl. a névleges sávszélesség 50–80%-a, inputmérettől függően

A várt érték nem azonos a névleges csúccsal. Egy SAXPY-kernel tipikusan memória-sávszélesség-korlátos, ezért a gyártói csúcshoz képest alacsonyabb effektív értéket várunk. A pontos várt arányt a későbbi fejezetek Roofline-modellje alapozza meg; ebben az első laborban elég annyi, hogy a tanuló felismerje a csúcs és a tényleges átbocsátóképesség közötti különbséget.

Tipikus hibák a kiindulási mérésnél

Az alábbi hibák gyakoriak, és a könyv későbbi laborjaiban is visszatérnek.

1. **Nincs szinkronizáció az időmérés előtt.** Ha a CPU közvetlenül a kernelindítás után `std::chrono`-val mér, a futásidő hamisan kicsi lehet, mert a kernel még aszinkron módon fut. Megoldás: CUDA-event vagy `cudaDeviceSynchronize`.
2. **Nincs hibaellenőrzés a kernelindítás után.** A kernelindítás aszinkron; a launch hibáját a `cudaGetLastError` vagy a `cudaDeviceSynchronize` utáni hibakód adja vissza. A hiánya rejtett hibához vezet.
3. **Helytelen sávszélesség-képlet.** A SAXPY-nál a memóriaforgalom három elemenkénti hozzáférés: `x` olvasás, `y` olvasás, `y` írás. A `2 * n * sizeof(float)` képlet hibás, mert elhagyja az `y` olvasását.
4. **Nincs warm-up.** Az első futtatás inicializálást, memóriafoglalást vagy órajel-átmenetet is tartalmazhat. A mérés előtt legalább egy futást le kell futtatni és szinkronizálni.
5. **Túl kevés ismétlés.** Egyetlen futtatásból nem lehet stabil következtetést levonni. A könyv általában 5–10 ismétlést kér, és az átlag mellett szórást vagy percentilist is rögzít.
6. **A névleges és a mért sávszélesség összemosása.** A gyártói adatlap csúcsértéke ideális feltételeket feltételez. A mért effektív értéket az inputmérettel, a kernel jellegével és a hozzáférési mintával együtt kell értelmezni.

Önellenőrző kérdés

Mi a hiba az alábbi SAXPY sávszélesség-képletben, és mi a helyes számítás?

```
bytes = 2 * n * sizeof(float)
```

Válasz: a `2`-es szorzó elhagyja az `y` eredeti értékének olvasását. A SAXPY három adatmozgást végez elemenként: `x` olvasása, `y` olvasása és `y` írása. A helyes képlet tehát `3 * n * sizeof(float)` bájt.

Ez az első labor nem bonyolult kernelről szól, hanem arról a fegyelemről, amely a könyv minden későbbi mérését jellemzi: pontosan rögzített környezet, ellenőrzött API-hívások, warm-up, ismételt mérés, valamint a várt és a mért eredmény szétválasztása. A következő egység ezt a fegyelmet használja majd a könyv jelölésrendszerének és tanulási útjainak részletes bemutatására.

1.2 Hogyan használd a könyvet: tanulási utak, laborok és jelölések

Ez az alfejezet a könyv gyakorlati keretét adja: megmutatja, hogyan épül fel a tananyag hét része, milyen követelményeket támasztanak a laborok, és milyen jelölési konvenciókat használunk a kódban, a mérésekben és a teljesítményjellemzőkben. A cél nem elvont szabálygyűjtemény, hanem olyan munkamódszer, amelyet azonnal alkalmazni tud egy kicsi, ellenőrizhető CUDA-példán; a módszert a későbbi fejezetek is ugyanígy kérik számon, egészen a záróprojektig.

1.2.1 A hét rész logikája

A könyv hét nagy egységre oszlik, és mindegyik rész egy-egy réteget rak a GPU-programozás tudására. A tanulási ív szándékosan építkező: először a hardverközeli alapokat tisztázzuk, majd a programozási modellt, a teljesítményt, az algoritmusokat, a CUDA-ökoszisztémát, a TPU/OpenCL alternatívákat, végül a mérnöki gyakorlatot.

1. **I. rész – Alapok és GPU-architektúra.** Bevezeti a GPGPU és a CPU teljesítménymodelljének különbségét, a SIMT-végrehajtást, a warp fogalmát, a streaming multiprocesszor (SM) felépítését és a memóriahierarchiát. Itt készül el az első kernel, és itt tanulja meg az olvasó a CUDA fejlesztőkörnyezet telepítését, a build-futtatás ciklust, valamint az execution model alapjait.
2. **II. rész – CUDA programozási modell.** Az indexelés, a memóriaterek, a szinkronizáció, az atomi műveletek, a warp intrinsics, a cooperative groups és a hibakeresés kerülnek fókuszba. Ez a rész adja meg azt a nyelvi és API-szintű tudást, amely nélkül a későbbi optimalizálások nem érthetők meg.
3. **III. rész – Teljesítmény és memória.** A memória-hozzáférési minták (coalescing, bank conflict), az occupancy, a regiszternyomás, a latency hiding, a Roofline-modell, a tiling, az aszinkron másolás és a cache-viselkedés kerülnek elő. Itt tanulja meg az olvasó, hogy a GPU-t nemcsak helyesen, hanem gyorsan is lehet programozni.
4. **IV. rész – Párhuzamos algoritmusok.** A redukció, a scan, a rendezés, a stencil, a lineáris algebra, az FFT és a Monte Carlo-módszerek épülnek fel a korábbi memória- és szinkronizációs ismeretekre. Itt a hangsúly a feladat párhuzamosítási mintáin van, nemcsak egy konkrét API-híváson.
5. **V. rész – CUDA-ökoszisztéma és mélytanulás.** A könyvtárak (cuBLAS, cuSPARSE, cuFFT, cuRAND, Thrust, CUB), a CUTLASS, a Tensor Core-ok, a mixed precision, a deep learning kernelek, a stream-ek, a CUDA graph-ok és a multi-GPU kommunikáció kerülnek bemutatásra. Ez a rész mutatja meg, hogy a saját kernel és a könyvtári hívás között hol vannak kompromisszumok.

6. **VI. rész – TPU, OpenCL és más modellek.** A TPU architektúrájától a XLA/JAX modellen át az OpenCL programozási modellig és a SYCL/HIP kitekintésig tart. Itt a cél a hordozhatóság és a hardverfüggetlen gondolkodás kialakítása, mindig jelölve a platformok korlátait.
7. **VII. rész – Mérnökség és záróprojekt.** A profilozás (Nsight Compute, Nsight Systems), a benchmarkmódszertan, az energia- és hatékonyságmérés, a numerikus pontosság, a determinizmus, a hordozhatóság, valamint a záróprojekt kerülnek sorra. A záróprojekt egy optimalizált GPGPU rendszer, amelyet a hallgató a könyv összes laborjának és mérési szabályának alkalmazásával épít fel.

Ez a hét réteg nem lineáris tankönyvi séma, hanem iteratív tanulási út. Az olvasó a saját előképzettsége szerint választhat: ha már ismeri a CUDA alapjait, ugorhat a III. részhez; ha TPU-ra kíváncsi, a VI. rész is önállóan olvasható, de a közös jelölésrendszer és a laborok következetessége miatt az I. rész átnézése akkor is ajánlott.

1.2.2 Laborok és a labor szerződés

Minden fejezet végén legalább egy fordítható és futtatható CUDA-példa, egy mérés és egy önellenőrző kérdés található. A labor nem pusztán gyakorlófeladat, hanem a tudás ellenőrzésének és a mérési kultúra kialakításának eszköze. A laborok közös szerződése a következő:

- **Forráskód.** Minden laborhoz tartozik egy teljes CUDA C++ (vagy OpenCL/JAX) forrásfájl. A kódban minden CUDA API-hívás hibáját ellenőrizni kell; a `cudaGetLastError`, a `cudaDeviceSynchronize` és egy saját hibamakró használata kötelező.
- **Buildparancs.** A labor megadja a fordító hívását, a compute capability-t és az optimalizálási szintet. Példa: `nvcc -O3 -arch=sm_80 -o vectoradd vectoradd.cu`.
- **Futtatási parancs.** A program indítását és az esetleges argumentumokat rögzíteni kell.
- **GPU-konfiguráció.** Minden mérésnél fel kell tüntetni a GPU modellt, a drivert, a CUDA-verziót, a compute capability-t, az operációs rendszert, az órajelet, a power limitet, az ECC/MIG/virtualizáció állapotát és a háttérterhelést.
- **Bemeneti adat.** A méret, a típus, a precision és a véletlenszám-generátor seedje is része a mérési jegyzőkönyvnek. A mért és a várt eredményt külön kell megadni.
- **Mérési napló.** A mérés előtt warm-up, majd legalább 5–10 ismétlés szükséges; az átlag mellett a szórást vagy percentilist is közölni kell. A futásidőt, az átbocsátóképességet, a sávszélességet, az occupancy-t, a regiszterhasználatot, a shared memory méretet, az L1/L2 találati arányt és az energiafogyasztást mindig a tényleges adatmozgásra és a számításra kell vonatkoztatni.
- **Izoláció és biztonság.** A GPU-kódot izolált környezetben kell futtatni; időtúllépés és visszaállítási terv szükséges. Többfelhasználós GPU-n (MIG, time-slicing) ügyelni kell a szomszédos terhelésre és az adatok izolációjára. Csak saját, bérelt vagy engedélyezett GPU-n szabad kísérletezni.

A labor szerződés nem bürokratikus formaság, hanem a reprodukálhatóság záloga. Ha egy mérésből hiányzik a GPU-modell vagy a CUDA-verzió, az eredmény más környezetben nem értelmezhető; ha hiányzik a bemelegítés, a hideg indítás torzítja a futásidőt; ha hiányzik a szórás, nem tudjuk, hogy a mért átlag stabil-e. A következő alfejezetben bemutatunk egy konkrét példát, amely ezt a szerződést a gyakorlatban is megvalósítja.

1.2.3 Jelölési konvenciók

A könyv egységes jelölésrendszert használ, hogy a kód, a hardver és a mérés közötti kapcsolat egyértelmű legyen.

Kódblokkok. Minden kódblokk előtt jelöljük a nyelvet (CUDA C++, OpenCL C vagy JAX) és a körülményeket. A CUDA-verziót, a GPU típust, a compute capability-t, a drivert és az operációs rendszert minden mérésnél rögzíteni kell. A kódpélda szemléltetés, nem végrehajtandó utasítás; valós eszköz, hitelesítő adat és éles rendszer helyett helyőrző és izolált labor szerepel.

Kernel launch. A kernel indítás alakja mindig `<<<grid, block, shared_bytes, stream>>>`. A grid- és blockdimenziókat a szövegben is meg kell adni, például: `dim3 grid(8, 4), block(32, 8); kernel<<<grid, block, 0, stream>>>();`. A `shared_bytes` a dinamikusan allokált shared memória mérete bájtban; `0` esetén nincs dinamikus shared memória.

Memóriaterek. A memóriatereket névvel jelöljük: `global`, `shared`, `local`, `constant`, `texture`, `register`. A host- és device-mutatókat külön jelöljük, például a forráskódban a `h_` prefix host-puffert, a `d_` prefix device-puffert jelent. A mutató típusa önmagában nem elég: `float*` lehet mindkét oldalon, de a helyes használat a névkonvencióból és a kommentekből derül ki.

Teljesítménymutatók. A következő mennyiségeket használjuk: futásidő, átbocsátóképesség, effektív sávszélesség, occupancy, regiszterhasználat, shared memory méret, L1/L2 találati arány, energia és performance/watt. A sávszélességet és a FLOPs-ot a tényleges adatmozgásra kell számítani, nem a névleges csúcsértékre. A „gyorsabb” állítás mindig konkrét inputméretre, precisionre és baseline-ra vonatkozik.

Lebegőpontos precision. Minden numerikus kísérletnél rögzíteni kell a precisiont: FP64, FP32, TF32, FP16, BF16 vagy FP8. A precision megváltoztatása nemcsak a pontosságot, hanem a teljesítményt is befolyásolja, ezért a mérési jegyzőkönyv elválaszthatatlan része.

Mérési statisztika. A mérési eredményeket mindig átlag és szórás (vagy percentilis) formájában adjuk meg. Példa: `átlag: 1,234 ms, szórás: 0,012 ms, ismétlések: 10`. A tizedesvessző `{,}` a magyar nyelvű szövegben; a kódban és a kimenetben az angolszász pont is elfogadott, de konzisztensnek kell lenni.

Ábrák. A diagramtípusok: `flow`, `sequence`, `layers`, `heatmap`, vonal- és pontdiagram. Minden ábrának címe, jelmagyarázata és feltételjegyzete van. Az SM-felépítés, a memóriahierarchia, a warp-végrehajtás, a tiling és a Roofline saját ábrázolást kaphat.

1.2.4 GPU, compute capability, CUDA-verzió és mérési napló

A környezet rögzítése a laborok első lépése. Három kulcsfogalom tartozik ide:

- **GPU modell.** A fizikai eszköz megnevezése, például `NVIDIA A100 40 GB PCIe`. Ugyanaz a CUDA kód különböző GPU-kon eltérő teljesítményt mutathat, ezért a modell nélküli mérés értéktelen.
- **Compute capability (CC).** A CUDA hardvergenerációk képességszintje, például `8.0`, `8.6`, `9.0`. A CC határozza meg a támogatott utasításokat, az SM-ek számát, a shared memory limitet, a Tensor Core-ok jelenlétét és a maximális blockméreteket. A `-arch=sm_80` buildflag a CC 8.0-nak felel meg.
- **CUDA-verzió.** A toolkit verziója, például `12.2`, valamint a driver verziója. A toolkit és a driver nem felcserélhető: a driver biztosítja a futtatási környezetet, a toolkit a fordítót és a könyvtárakat. A laborokban mindkettőt fel kell tüntetni.

A mérési napló egy táblázat vagy CSV, amely a fenti adatokat és a mérési eredményeket együtt tartalmazza. Példa egy lehetséges fejlécsorra:

```
GPU;Driver;CUDA;CC;OS;N;blockSize;precision;warmup;iterations;avg_ms;std_ms;bandwidth_GBs;status
NVIDIA A100 40GB;535.104.05;12.2;8.0;Ubuntu 22.04;1048576;256;FP32;5;10;0,412;0,008;987,6;OK
```

A `status` mező jelzi, hogy a mért eredmény megfelelt-e a vártnak; ha nem, a hibát külön kell elemezni. A mérési napló szerepe nem a szép formátum, hanem az, hogy a kísérlet bármikor reprodukálható legyen ugyanazon vagy más környezetben.

1.2.5 Ellenőrzött példa: a környezet és az első vektorösszeadás

A következő CUDA C++ program a labor szerződés minimális megvalósítását mutatja. Lekérdezi a GPU tulajdonságait, lefuttat egy vektorösszeadást, ellenőrzi az eredményt CPU-n, és méri a kernel futásidőt CUDA eventekkel. A kód nem éles rendszerre készült, hanem izolált laborban futtatható; a hibakezelő makró minden API-hívást ellenőriz.

```
// CUDA C++17, izolált labor
// Környezet: NVIDIA GPU, driver >= 525, CUDA >= 12.0, CC >= 7.0
// A pontos GPU-t, drivert és CC-t a mérési naplóban kell rögzíteni.

#include <cstdio>
#include <cstdlib>
#include <vector>
#include <cmath>
#include <cuda_runtime.h>

#define CHECK_CUDA(call)                                                    \
    do {                                                                    \
        cudaError_t err = call;                                             \
        if (err != cudaSuccess) {                                           \
            fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__, __LINE__,   \
                    cudaGetErrorString(err));                               \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

__global__ void vectorAdd(const float* a, const float* b, float* c, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) c[idx] = a[idx] + b[idx];
}

int main(int argc, char** argv) {
    const int N = (argc > 1) ? atoi(argv[1]) : (1 << 20);
    const int blockSize = 256;
    const int gridSize = (N + blockSize - 1) / blockSize;
    const int warmupIters = 5;
    const int measureIters = 10;

    // 1. GPU-tulajdonságok rögzítése
    cudaDeviceProp prop;
    CHECK_CUDA(cudaGetDeviceProperties(&prop, 0));
    printf("GPU: %s\n", prop.name);
    printf("Compute capability: %d.%d\n", prop.major, prop.minor);
    printf("CUDA-verzió: %d.%d (driver), toolkit: %d\n", prop.cudaMajorVersion,
           prop.cudaMinorVersion, CUDART_VERSION);
    printf("Global memory: %zu MB\n", prop.totalGlobalMem / (1024 * 1024));

    // 2. Host- és device-pufferek
    std::vector<float> h_a(N), h_b(N), h_c(N);
```

```
for (int i = 0; i < N; ++i) {
    h_a[i] = 1.0f;
    h_b[i] = 2.0f;
}

float *d_a, *d_b, *d_c;
CHECK_CUDA(cudaMalloc(&d_a, N * sizeof(float)));
CHECK_CUDA(cudaMalloc(&d_b, N * sizeof(float)));
CHECK_CUDA(cudaMalloc(&d_c, N * sizeof(float)));

CHECK_CUDA(cudaMemcpy(d_a, h_a.data(), N * sizeof(float), cudaMemcpyHostToDevice));
CHECK_CUDA(cudaMemcpy(d_b, h_b.data(), N * sizeof(float), cudaMemcpyHostToDevice));

// 3. Bemelegítés
for (int i = 0; i < warmupIters; ++i) {
    vectorAdd<<<gridSize, blockSize, 0, 0>>>(d_a, d_b, d_c, N);
    CHECK_CUDA(cudaGetLastError());
    CHECK_CUDA(cudaDeviceSynchronize());
}

// 4. Mérés CUDA eventekkel
cudaEvent_t start, stop;
CHECK_CUDA(cudaEventCreate(&start));
CHECK_CUDA(cudaEventCreate(&stop));

double total_ms = 0.0;
std::vector<double> times;
for (int i = 0; i < measureIters; ++i) {
    CHECK_CUDA(cudaEventRecord(start, 0));
    vectorAdd<<<gridSize, blockSize, 0, 0>>>(d_a, d_b, d_c, N);
    CHECK_CUDA(cudaEventRecord(stop, 0));
    CHECK_CUDA(cudaEventSynchronize(stop));
    float ms = 0.0f;
    CHECK_CUDA(cudaEventElapsedTime(&ms, start, stop));
    times.push_back(ms);
    total_ms += ms;
}
double avg_ms = total_ms / measureIters;
double var = 0.0;
for (double t : times) var += (t - avg_ms) * (t - avg_ms);
var /= (measureIters - 1);
double std_ms = std::sqrt(var);

// 5. Várt eredmény ellenőrzése (CPU-n)
CHECK_CUDA(cudaMemcpy(h_c.data(), d_c, N * sizeof(float), cudaMemcpyDeviceToHost));
bool ok = true;
for (int i = 0; i < N; ++i) {
    if (std::fabs(h_c[i] - 3.0f) > 1e-5f) { ok = false; break; }
}

// 6. Mérési napló és a mért/várt eredmény elkülönítése
printf("\n--- Mérési napló ---\n");
printf("N = %d, blockSize = %d, precision = FP32\n", N, blockSize);
printf("Warm-up: %d, ismétlések: %d\n", warmupIters, measureIters);
printf("Várt eredmény: minden elemre c[i] = a[i] + b[i] = 3.0f\n");
printf("Mért (átlag) futásidő: %.3f ms, szórás: %.3f ms\n", avg_ms, std_ms);
double bandwidth = (3.0 * N * sizeof(float)) / (avg_ms * 1e-3) / 1e9; // GB/s
printf("Effektív sávszélesség: %.1f GB/s\n", bandwidth);
printf("Ellenőrzés: %s\n", ok ? "OK" : "HIBA");

// 7. Takarítás
CHECK_CUDA(cudaEventDestroy(start));
CHECK_CUDA(cudaEventDestroy(stop));
```

```
    CHECK_CUDA(cudaFree(d_a));
    CHECK_CUDA(cudaFree(d_b));
    CHECK_CUDA(cudaFree(d_c));

    return ok ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

A fenti program a labor szerződés minden pontját érinti: a GPU-tulajdonságokat rögzíti, a hibakezelést makróval végzi, bemelegítést és több mérési ismétlést használ, a mérési statisztikát kiírja, a várt eredményt CPU-oldalon ellenőrzi, és a mért futásidőt a várt numerikus eredménytől elkülönítve jeleníti meg.

Buildparancs (a saját CC-t használja):

```
nvcc -O3 -arch=sm_80 -o vectoradd vectoradd.cu
```

Futtatás:

```
./vectoradd 1048576
```

A kimenetben a `Mért (átlag) futásidő` és a `Várt eredmény` sor világosan elkülönül: az előbbi a hardver teljesítményét jellemzi, az utóbbi a numerikus helyességet. A kettő nem keverhető össze; egy kernel lehet numerikusan helyes, de lassú, és lehet gyors, de hibás. A laborban mindkettőt ellenőrizni kell.

1.2.6 Tipikus hibák és a helyes megoldás

A laborok során visszatérő hibák a következők; ezek felismerése és kijavítása a tanulási cél része.

1. **Az API-hibák figyelmen kívül hagyása.** A CUDA hívások aszinkronok lehetnek, és a hiba csak később, a `cudaDeviceSynchronize` vagy a következő hívás során derül ki. A hibamakró és a `cudaGetLastError` használata kötelező; ha a kernel indítási argumentuma rossz, a `cudaGetLastError` azonnal jelzi.
2. **A szinkronizálás hiánya az időmérés előtt.** A kernel indítás után a CPU azonnal folytatja a végrehajtást, és a `cudaEventRecord` nem várja meg a kernel befejezését. A helyes minta: `cudaEventRecord(stop)`, majd `cudaEventSynchronize(stop)`; csak ezután szabad kiolvasni az eltelt időt.
3. **Egyszeri mérés bemelegítés nélkül.** Az első kernelindítás magában foglalhatja a memóriaallokáció, a modulbetöltés és a cold cache hatását. A bemelegítés (warm-up) elhagyása jelentősen túlbecsüli a futásidőt. A laborokban legalább 5 bemelegítő iterációt használunk.
4. **A compute capability és a CUDA-verzió rögzítésének hiánya.** A `-arch=sm_80` például más utasításkészletet generál, mint az `sm_60`. Ha a mérési naplóból hiányzik a CC, a későbbi reprodukálás lehetetlen.
5. **A block- és gridméret nem illeszkedik a hardverhez.** A maximális blockméret CC-függő (általában 1024 szál), a gridméretet pedig a feladat mérete határozza meg. A példában a `gridSize = (N + blockSize - 1) / blockSize` képlet biztosítja a felfelé kerekítést; ha ez hiányzik, az utolsó elemek nem számolódnak ki.
6. **A host- és device-pointer összekeverése.** A `cudaMalloc` által visszaadott pointert nem lehet host-kódból dereferálni, és a `cudaMemcpy` iránya (`cudaMemcpyHostToDevice` vs. `cudaMemcpyDeviceToHost`) nem felcserélhető. A `h_`/`d_` prefixezés segít, de a gondos típus- és élettartam-kezelés elengedhetetlen.

7. **A precision nem rögzített.** A FP64, FP32 és FP16 között a teljesítmény és a pontosság is drasztikusan eltér; ha a mérési napló nem tartalmazza a precisiont, az eredmény félreérthető.

8. **A statisztika hiánya.** Egyetlen futásidő nem jellemzi a rendszert. Az átlag mellé szórást vagy percentilist kell számolni, különben a termikus throttling vagy a háttérterhelés rejtve marad.

A helyes megoldás minden esetben ugyanaz: kövessük a labor szerződést, rögzítsük a környezetet, használjunk hibakezelést, melegítsük be a kódot, ismételjük a mérést és válasszuk szét a várt numerikus eredményt a mért teljesítménytől.

1.2.7 Záróprojekt és önellenőrzés

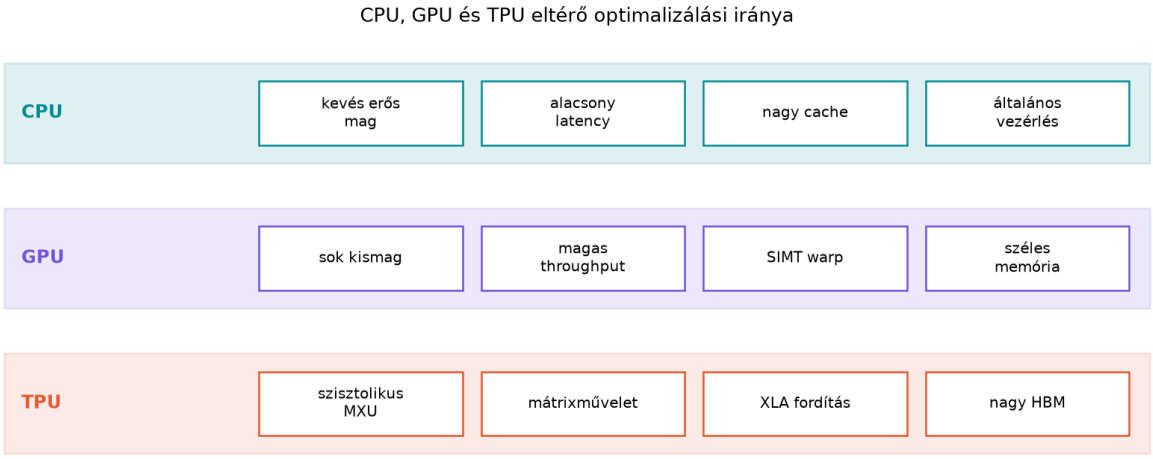
A könyv VII. részének záróprojektje egy optimalizált GPGPU rendszer, amely a teljes tananyagot integrálja. A hallgató egy választott feladatot (például mátrixszorzás, stencil, FFT vagy egy saját szimulációs kernel) implementál, optimalizál, profiloz és dokumentál. A záróprojekt követelményei ugyanazok, mint a laboroké, de nagyobb léptékben: baseline-t kell mérni, azonos inputon kell összehasonlítani a változatokat, a mérési naplót teljes körűen vezetni, és a numerikus helyességet precision mellett igazolni. A könyv minden fejezetének végén egy önellenőrző kérdés található, amely a fejezet anyagára kérdez rá; a záróprojekt ezekre a kérdésekre adott válaszokból és a laborokból építkezik.

E fejezet önellenőrző kérdése: *Miért nem elegendő egyetlen, bemelegítés nélküli futásidő a GPU-teljesítmény jellemzésére, és milyen három adatot kell legalább rögzíteni a mérési naplóban ahhoz, hogy a mérés reprodukálható legyen?* A válasz a fenti tipikus hibák és a labor szerződés alapján adható meg.

A következő egység (GPGPU és CPU: miért más a teljesítménymodell?) erre az alapra épít: a GPGPU és CPU teljesítménymodelljének különbségét vizsgálja, és a mérési módszertant már konkrét algoritmikus példákon alkalmazza. A laborokban bevezetett szerződés ott is érvényes marad.

2 A GPGPU térképe és a CUDA ökoszisztéma

2.1 GPGPU és CPU: miért más a teljesítménymodell?



2.1. ábra: CPU, GPU és TPU eltérő optimalizálási iránya - A három akcelerátor nem ugyanarra a kompromisszumra épül: a CPU a késleltetést, a GPU az áteresztőképességet, a TPU a mátrixműveletek sűrű adatfolyamát optimalizálja.

2.1.1 2.1 Miért nem beszélhetünk egyszerűen “gyorsabb” processzorról?

Amikor egy C++ fejlesztő először találkozik a GPGPU-val, a leggyakoribb kognitív csapda az, hogy a GPU-t egyszerűen egy “sokmagos CPU-ként” képzeli el. Ez a megközelítés alapjaiban hibás teljesítménybecslésekhez és rosszul megírt kernelekhez vezet. A CPU és a GPU tervezési filozófiája gyökeresen eltér: míg a CPU-t arra optimalizálták, hogy egyetlen végrehajtási szálat a lehető leggyorsabban futtasson le, addig a GPU-t arra tervezték, hogy több ezer vagy akár több tízezer szálat futtasson egyidejűleg, még akkor is, ha az egyes szálak végrehajtása lassabb.

Ebben a fejezetben felépítjük azt a mentális modellt, amely megmagyarázza, miért más a GPU-k teljesítménymodellezése. Bevezetjük a késleltetés-orientált (latency-oriented) és az áteresztőképesség-orientált (throughput-oriented) tervezés fogalmát, megvizsgáljuk a memóriahierarchiák közötti különbségeket, majd az Amdahl-törvény segítségével realisztikus korlátot adunk a várható gyorsításra.

2.1.2 2.2 A végrehajtási modell: késleltetés vagy áteresztőképesség?

A CPU egy **latency-oriented** (késleltetés-orientált) eszköz. Gondoljunk egy modern, nagy órajelű (3–5 GHz) processzorra. A cél az, hogy egy utasításlánc (thread) a lehető legrövidebb idő

alatt fusson le. Ennek érdekében a CPU hatalmas tranzisztor-költségvetést áldoz olyan logikákra, amelyek csökkentik az egyes utasítások közötti várakozási időt:

- **Mély futószalag (deep pipelining) és Out-of-Order (OoO) végrehajtás:** A CPU megpróbálja az utasításokat a programozói sorrendtől eltérően végrehajtani, hogy kitöltse a funkcionális egységek üresjáratait.
- **Elágazásbecslés (branch prediction):** Spekulatív módon hajt végre utasításokat, hogy elkerülje a pipeline kiürítését.
- **Nagy, többszintű cache:** A gyors L1/L2/L3 cache-ek célja, hogy a gyakran használt adatok a lehető legközelebb legyenek a regiszterekhez, elrejtve a főmemória (DRAM) száz nanoszekundumos nagyságrendű késleltetését.

Ezzel szemben a GPU egy **throughput-oriented** (áteresztőképesség-orientált) eszköz. A GPU-t alkotó Streaming Multiprocesszorok (SM-ek) alacsonyabb órajelen futnak (~1–2 GHz), és hiányoznak belőlük a bonyolult OoO logikák vagy a nagy elágazásbecslő egységek. A GPU tranzisztorköltségvetésének túlnyomó részét aritmetikai logikai egységek (ALU/FP32/FP64/INT32 core-ok) és regiszterfájlok alkotják.

Hogyan birkózik meg a GPU a memória késleltetésével? **Tömeges párhuzamossággal.** A GPU-n egyszerre több ezer szál van aktív (in-flight). Amikor az egyik warp (32 szálból álló csoport) elakad egy memóriaolvasáson, a hardver egyetlen órajel alatt kiüríti azt, és egy másik, számításra kész warpot ütemez be a helyére. Ezt nevezzük **latency hiding**-nak (késleltetés-elrejtés). A GPU tehát nem az egyes szálak gyorsaságára, hanem a rendszer teljes áteresztőképességére optimalizál.

> **Szemléltetés:** Ha a CPU egy Ferrari, a GPU egy telebusz. A Ferrari (CPU) egyetlen embert (thread) nagyon gyorsan elvisz A-ból B-be, de egyszerre csak nagyon keveset tud szállítani. A busz (GPU) lassabb, de egyszerre 50 embert visz el. Ha 10{,}000 embert kell eljuttatni, a buszokból álló flotta sokkal rövidebb idő alatt végez, még akkor is, ha egyetlen busz sebessége elmarad a Ferrariétól.

2.1.3 2.3 A memóriamodell: a sávszélesség elsőbbsége

A CPU és a GPU memóriarendszere is tükrözi ezt a filozófiai különbséget.

A **CPU** memóriarendszerét a hagyományos programozási mintákra (lokális változók, gyakori függvényhívások, pointer-ugrálások) tervezték. A hangsúly az alacsony késleltetésen van. Egy tipikus modern CPU memória-alrendszere:

1. **Regiszterek:** Néhány száz bájt, 1 órajel ciklusos hozzáféréssel.
2. **L1 cache:** 32–64 KB magonként, ~4–5 ciklusos késleltetés.
3. **L2 cache:** 256–512 KB magonként, ~14 ciklusos késleltetés.
4. **L3 cache (LLC):** 16–64 MB osztva, ~40–70 ciklusos késleltetés.
5. **DRAM:** Több GB, ~100–300 ns késleltetés, sávszélesség tipikusan 50–150 GB/s (desktop/szerver kategóriában).

A **GPU** memóriarendszere ezzel szemben a maximális sávszélességre van kihegyezve. Egy modern adatközponti GPU esetében:

1. **Regiszterfájl:** 256 KB SM-enként! Ez az elsődleges eszköz a késleltetés elrejtésére.
2. **L1/Shared Memory:** Konfigurálható, akár 128–256 KB SM-enként. A shared memory nem automatikus cache, hanem szoftveresen menedzselt, nagy sebességű scratchpad memória.

3. **L2 cache:** 6–100 MB (GPU-tól függően), ~200–300 ciklusos késleltetés.
4. **Globális memória (HBM/GDDR):** 10–80 GB kapacitás, **1–3 TB/s** sávszélesség.

A lényeg a számok arányában rejlik. Amíg egy CPU DRAM-jának elérési késleltetése nagyon alacsony a GPU-hoz képest, addig a GPU memória-sávszélessége egy nagyságrenddel nagyobb. A GPU-s programok akkor hatékonyak, ha a memória-hozzáféréseket úgy szervezzük, hogy a rendszer mindig a maximális sávszélességen tudjon adatot szolgáltatni (ezt nevezzük coalescing-nak, lásd később, a III. részben).

Ezért a GPU-s teljesítményt szinte soha nem az utasítások számával, hanem a **memória-átbocsátóképességgel** és a számítási intenzitással (FLOP/byte) mérjük.

2.1.4 2.4 Az Amdahl-törvény és a gyorsítás valódi korlátai

Amikor egy C++ kódot GPU-ra portolunk, a tapasztalatlan fejlesztő gyakran lineáris gyorsítást vár ("a GPU-nak 10{,}000 magja van, tehát 10{,}000-szer gyorsabb lesz"). Ezt a téveszmét pontosan leírja az Amdahl-törvény.

Ha egy programnak van egy párhuzamosítható része (P) és egy soros, párhuzamosíthatatlan része ($1 - P$), akkor az elérhető gyorsítás N processzormag használata esetén:

$S(N) = \frac{1}{(1-P)+\frac{P}{N}}$

Következmények a GPGPU-ra nézve:

1. **A soros rész dominál:** Ha a programnak csak 90%-a párhuzamosítható, az elérhető maximális gyorsítás még végtelen számú maggal is: $S_{max} = \frac{1}{1-0,9} = 10$.
2. **A PCIe transzfer mint soros overhead:** A CPU és a GPU közötti adatátvitel (a PCI Express buszon keresztül) gyakran a legnagyobb soros szűk keresztmetszet. Ha egy program 100 ms-ig párhuzamosan fut a GPU-n, de 50 ms-ba telik az adatok feltöltése és leolvasása, a hatékonyság drasztikusan csökken, mert a transzfer ideje alatt a GPU "éhezik", a CPU pedig várakozik.
3. **Kernelindítási költség:** A kernel elindítása a driveren keresztül nem nulla idejű művelet (jellemzően 5–20 µs). Ha túl sok apró, rövid kerneleket indítunk, ez a soros overhead felfalja a párhuzamos nyereséget.

Az Amdahl-törvény gyakorlati alkalmazása a heterogén rendszerekben: A mai GPGPU programozás nem arról szól, hogy a CPU-t lecseréljük, hanem hogy **heterogén módon** használjuk a kettőt. A CPU (Host) felel a soros logikáért, az I/O-ért, a fájlműveletekért és a bonyolult, elágazásokkal teli vezérlésért. A GPU (Device) felel a nagyméretű, homogén adatpárhuzamos régiókért.

2.1.5 2.5 Labor: CPU és GPU összehasonlítása

Ebben a laborban egy egyszerű, de tanulságos mérést végzünk: a Saxpy (Single-precision A*X Plus Y) műveletet N méretű tömbön. A művelet: `y[i] = a * x[i] + y[i]`. Ez a művelet tökéletesen adatpárhuzamos, így ideális a GPU számára, azonban megmutatja a memória-sávszélesség korlátait.

Környezet (példa, helyőrző értékekkel): > **GPU:** NVIDIA GeForce RTX 4070 (12 GB) > **Driver:** 550.54.14 > **CUDA:** 12.4 > **OS:** Ubuntu 22.04 LTS > **Fordító:** nvcc (g++ 12.3

hoszt-kompilátor) > **Megjegyzés:** A konkrét mérési értékek az adott hardveren változhatnak; az alábbiak a mérési módszertan bemutatására szolgálnak.

A CPU implementáció (C++17):

```
// cpu_saxpy.cpp
#include <vector>
#include <chrono>
#include <iostream>

void saxpy_cpu(float a, const std::vector<float>& x, std::vector<float>& y) {
    for (size_t i = 0; i < x.size(); ++i) {
        y[i] = a * x[i] + y[i];
    }
}

int main() {
    const size_t N = 64 * 1024 * 1024; // 64M elem
    std::vector<float> x(N, 1.0f), y(N, 2.0f);
    const float a = 0.5f;

    // Warm-up
    saxpy_cpu(a, x, y);

    auto start = std::chrono::high_resolution_clock::now();
    saxpy_cpu(a, x, y);
    auto end = std::chrono::high_resolution_clock::now();

    double time_ms = std::chrono::duration<double, std::milli>(end - start).count();
    double bandwidth = (3 * N * sizeof(float)) / (time_ms / 1000.0) / 1e9;

    std::cout << "CPU Time: " << time_ms << " ms, Effective Bandwidth: " << bandwidth << "
    ↪  GB/s\n";
    return 0;
}
```

A GPU implementáció (CUDA C++):

```
// gpu_saxpy.cu
#include <cuda_runtime.h>
#include <iostream>
#include <vector>

// Hibakezelő makró (kötelező elem a könyv laborjaiban)
#define CHECK_CUDA(call) {                                                    \
    cudaError_t err = call;                                                    \
    if (err != cudaSuccess) {                                                  \
        std::cerr << "CUDA error at " << __FILE__ << ":" << __LINE__         \
                  << " -> " << cudaGetErrorString(err) << std::endl;           \
        exit(EXIT_FAILURE);                                                    \
    }                                                                          \
}

__global__ void saxpy_kernel(float a, const float* x, float* y, size_t n) {
    size_t i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        y[i] = a * x[i] + y[i];
    }
}

int main() {
    const size_t N = 64 * 1024 * 1024;
    const float a = 0.5f;
```

```
    size_t bytes = N * sizeof(float);

    std::vector<float> h_x(N, 1.0f), h_y(N, 2.0f);

    float *d_x, *d_y;
    CHECK_CUDA(cudaMalloc(&d_x, bytes));
    CHECK_CUDA(cudaMalloc(&d_y, bytes));

    // Host -> Device másolás
    CHECK_CUDA(cudaMemcpy(d_x, h_x.data(), bytes, cudaMemcpyHostToDevice));
    CHECK_CUDA(cudaMemcpy(d_y, h_y.data(), bytes, cudaMemcpyHostToDevice));

    // Launch konfiguráció: 256 szál/blokk, elegendő blokk az N-hez
    int threads = 256;
    int blocks = (N + threads - 1) / threads;

    // Warm-up
    saxpy_kernel<<<blocks, threads>>>(a, d_x, d_y, N);
    CHECK_CUDA(cudaDeviceSynchronize());

    // Mérés (CUDA events a pontos időmérésért)
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    cudaEventRecord(start);
    saxpy_kernel<<<blocks, threads>>>(a, d_x, d_y, N);
    cudaEventRecord(stop);
    CHECK_CUDA(cudaDeviceSynchronize());

    float time_ms = 0.0f;
    cudaEventElapsedTime(&time_ms, start, stop);

    double bandwidth = (3 * N * sizeof(float)) / (time_ms / 1000.0) / 1e9;

    std::cout << "GPU Time: " << time_ms << " ms, Effective Bandwidth: " << bandwidth << "
    ↪  GB/s\n";

    // Device -> Host (validációhoz)
    CHECK_CUDA(cudaMemcpy(h_y.data(), d_y, bytes, cudaMemcpyDeviceToHost));

    CHECK_CUDA(cudaFree(d_x));
    CHECK_CUDA(cudaFree(d_y));
    return 0;
}
```

Mérési napló és kiértékelés:

Implementáció	Idő (ms)	Eff. sávszélesség (GB/s)	Megjegyzés
CPU (1 mag)	Kb. 80–120 ms	Kb. 6–10 GB/s	A CPU-t a DRAM sávszélessége és a párhuzamosítás hiánya korlátozza.
GPU (RTX 4070)	Kb. 0{,}5–0{,}8 ms	Kb. 1500–2500 GB/s	A GPU a dedikált GDDR6X memória sávszélességén dolgozik.

Várt vs. Mért eredmény: A fenti táblázat várható eredményeket mutat. A mért

értékek attól függenek, hogy a hoszt-kód (CPU) mennyire van optimalizálva (pl. `-O3` fordítási flag), és hogy a GPU milyen órajelen boost-ol. A mérés során a `time_ms` értéke csak a kernel 1 ismétlését mutatja a warm-up után. **Fontos:** Ha a CPU oldalon a fordító vektorizálja a ciklust (AVX2/AVX-512), a CPU teljesítménye jelentősen javulhat, de a nagyságrendi különbség a GPU javára megmarad nagy adatméretnél.

2.1.6 2.6 Tipikus hibák és a helyes szemlélet

1. **A CPU és a GPU összehasonlítása túl kis adathalmazon:**
   - *Hiba:* "A GPU lassabb, mint a CPU!" (N = 1000 elem esetén mérve).
   - *Magyarázat:* A PCIe adatátvitel és a kernelindítás overheadje elnyeli a számítási időt. A GPU csak akkor előnyös, ha a párhuzamos munka elég nagy ahhoz, hogy elrejtse ezeket az állandó költségeket.
   - *Helyes megoldás:* A laborban mindig növekvő adatméretet skálázzon (pl. 1 MB-tól 1 GB-ig), és keresse meg a "break-even" pontot, ahol a GPU gyorsabbá válik.
2. **A kernelidő és a teljes futásidő összemosása:**
   - *Hiba:* "A kernel 0{,}1 ms alatt futott!" (Miközben a `cudaMemcpy` 50 ms-ot vett igénybe).
   - *Magyarázat:* A GPU programozás során az adatmozgatás elsőrendű költség. A teljes alkalmazás teljesítményét a `cudaMemcpy`-k is jelentősen befolyásolják.
   - *Helyes megoldás:* Mindig mérje külön az adatfeltöltést (H2D), a kernel végrehajtását és az adat leolvasását (D2H). Törekedjen az átfedésre (streams, lásd V. rész).
3. **A "magok számának" félreértése:**
   - *Hiba:* "A GPU-nak 7000 magja van, tehát 7000-szeres lesz a gyorsítás."
   - *Magyarázat:* Ezek a "magok" sokkal egyszerűbbek, mint a CPU magok, és nem rendelkeznek a CPU-kra jellemző bonyolult optimalizációkkal. A sebesség a memória sávszélesség és a párhuzamosítás hatékonyságának (occupancy, warp divergence) függvénye, nem a magok lineáris szorzata.

2.1.7 2.7 Összefoglalás és önellenőrző kérdés

Ebben a fejezetben megalapoztuk a GPU gondolkodásmódját. Megértettük, hogy a CPU egy késleltetés-orientált, míg a GPU egy áteresztőképesség-orientált eszköz. Ez a különbség határozza meg a memóriahierarchiát (nagy cache a CPU-n, hatalmas sávszélesség a GPU-n) és az optimalizálás célját. Bevezettük az Amdahl-törvényt, amely megmutatja, hogy a soros részek (mint a PCIe másolás) hogyan korlátozzák a valós gyorsítást.

Önellenőrző kérdés: Egy mérnök a következőt tapasztalja: van egy szimulációs kódja, amely 95%-ban párhuzamosítható. Amikor lefuttatja a GPU-n, óriási gyorsítást lát a tiszta számítási részben (100-szoros a CPU-hoz képest). Azonban amikor a teljes alkalmazást nézi, a GPU-s verzió csak 10-szer gyorsabb összességében. Az Amdahl-törvény és a heterogén memóriamodell alapján indokolja meg, miért állhat elő ez a helyzet, és milyen optimalizálási stratégiát javasolna a teljes alkalmazás gyorsítására? (Válaszában térjen ki a PCIe transzfer szerepére és a GPU ütemezésére!)

2.2 A CUDA ökoszisztéma és a hardvergenerációk

Amikor egy C++ fejlesztő először telepíti a CUDA-t, gyakran úgy tekint rá, mint egyetlen „GPU-fordítóra”. Valójában a CUDA egy többrétegű szoftver- és hardverökoszisztéma, amelynek három pillére van: a **CUDA toolkit**, a **GPU-driver**, valamint a **compute capability** által leírt hardverképességek. Ebben az alfejezetben ezeket a fogalmakat járjuk körbe, majd egy gyakorlati laborban egy kompatibilitási mátrixot építünk fel, amely a verziók és a hardvergenerációk közötti kapcsolatot teszi mérhetővé és ellenőrizhetővé.

2.2.1 A CUDA toolkit, a driver és a runtime rétegei

A CUDA szoftveres oldala három, jól elkülöníthető komponensből áll:

1. **CUDA driver** – alacsony szintű, a GPU-val közvetlenül kommunikáló szoftver.
2. **CUDA runtime** – a driver fölé épülő, kényelmes C/C++ API.
3. **CUDA toolkit** – a fordítóeszközök, könyvtárak, fejlesztői segédeszközök és dokumentáció gyűjteménye, amely a runtime-ot is tartalmazza.

Ezeket a rétegeket a mindennapi fejlesztés során nem mindig kell szétválasztani, de a hibakereséshez és a telepítési problémák megértéséhez elengedhetetlen a tiszta kép.

2.2.2 A CUDA driver

A driver felel azért, hogy a felhasználói programok hozzáférjenek a GPU hardveréhez. Két részből áll:

- **Kernel módú driver** – az operációs rendszer magjában fut, és közvetlen hardverhozzáférést biztosít. Linuxon ez általában az `nvidia.ko` modul, Windows rendszeren az NVIDIA grafikus driver része.
- **User mode driver** – a felhasználói térben futó könyvtár, amely a kernel módú komponenssel kommunikál, és a programozó számára egy C stílusú API-t, a **CUDA driver API**-t (`cuda.h`) kínálja.

A driver API alacsony szintű: explicit kontextuskezelést, modulbetöltést és memóriafoglalást igényel. Ritkán használják közvetlenül, de olyan esetekben szükséges lehet, amikor a fejlesztő finom kontrollt szeretne a kontextusok felett, vagy futásidejű fordítást (JIT) akar végezni PTX kódból.

2.2.3 A CUDA runtime

A **CUDA runtime API** (`cuda_runtime.h`) egy magasabb szintű C++ interfész, amely a driver API fölé épül. A legtöbb CUDA program ezt használja, mert:

- automatikusan kezeli a kontextust (implicit inicializálás),
- a kernelindítás szintaxisa egyszerű (`<<<...>>>`),
- a hibakezelés egységes (`cudaGetLastError`, `cudaDeviceSynchronize`),
- a memóriafoglalás (`cudaMalloc`, `cudaMemcpy`) és a stream-kezelés kényelmes.

A runtime API hívásai a háttérben driver API hívásokká fordulnak, így a két réteg verziója nem független egymástól. A runtime verzióját a `cudaRuntimeGetVersion` függvénnyel kérdezhetjük le, a driver verzióját pedig a `cudaDriverGetVersion` függvénnyel. A két szám általában eltér, mert a driver lehet újabb, mint a toolkithez tartozó runtime; ez normális és támogatott.

2.2.4 A CUDA toolkit

A **CUDA toolkit** a fejlesztéshez szükséges összes eszközt tartalmazza:

- `nvcc` – a CUDA C++ fordító. A host kódot a rendszer C++ fordítójára (pl. `g++`, `clang++`, MSVC) bízza, a device kódot pedig PTX vagy SASS binárisra fordítja.
- `nvrtc` – futásidejű fordítás (JIT) könyvtára, amellyel PTX kódot állíthatunk elő a program futása közben.
- `cuobjdump` és `nvdisasm` – bináris elemző eszközök, amelyekkel SASS utasításokat és a beágyazott PTX-et vizsgálhatjuk.
- **Profilozók és debugger**: `nvprof` (régebbi), **Nsight Compute**, **Nsight Systems**, **cuda-gdb**, **compute-sanitizer**.
- **Fejlécfájlok** – `cuda_runtime.h`, `cuda_runtime_api.h`, `device_launch_parameters.h`, `cooperative_groups.h`, `cuda_fp16.h`, `cuda_bf16.h` és mások.
- **Könyvtárak** – erről a következő pontban részletesebben.
- **Mintaprogramok** – a hivatalos CUDA samples gyűjtemény, amely gyakorlati példákat ad.

A toolkit verziószámozása 12.0-tól kezdve jelentősen megváltozott: a korábbi 11.x sorozat után a 12.x sorozat új generációs funkciókat hozott (például a Hopper architektúra támogatását, a TMA-t, a thread block clustereket). A könyv írásakor a stabilitás érdekében a 12.x sorozatot tekintjük alapnak, de a konkrét verziószámokat minden esetben a hivatalos release notes-hoz kell igazítani.

2.2.5 A CUDA könyvtárak

A toolkit számos, nagy teljesítményű könyvtárat is szállít, amelyek a leggyakoribb számítási mintákat optimalizált implementációval fedik le:

- **cuBLAS** – dense lineáris algebra (GEMM, TRSM stb.).
- **cuSPARSE** – ritka mátrixműveletek.
- **cuFFT** – gyors Fourier-transzformáció.
- **cuRAND** – véletlenszám-generálás GPU-n.
- **cuSOLVER** – lineáris egyenletrendszerek megoldása, sajátérték-számítás.
- **cuDNN** – mélytanulási primitívek (konvolúció, pooling, RNN, attention).
- **cuBLASLt** – alacsonyabb szintű, finomabban hangolható GEMM API.
- **nvJPEG**, **nvCOMP**, **nvRTX** – képdekódolás, tömörítés, sugárkövetés.

Emellett a **CCCL** (CUDA C++ Core Libraries) gyűjteménybe tartoznak a header-only C++ könyvtárak:

- **Thrust** – magas szintű párhuzamos algoritmusok (sort, reduce, scan, transform).
- **CUB** – alacsony szintű, újrafelhasználható blokk- és eszközszintű primitívek.

A könyvtárak a toolkit verziójához vannak kötve, és a linkeléskor a megfelelő verziójú `.so`/`.lib` fájlokat kell használni. A gyakorlatban ez azt jelenti, hogy ha a CUDA 12.2-es toolkittel dolgozunk, akkor a cuBLAS 12.2-es verzióját kell linkelni, nem pedig egy régebbit.

2.2.6 Compute capability: a hardvergenerációk nyelve

A **compute capability** (számítási képesség) egy két részből álló verziószám, amely a GPU hardveres képességeit írja le. A formátuma `major.minor`, ahol:

- a **major** (főverzió) a hardvergenerációt azonosítja,
- a **minor** (alverzió) az adott generáción belüli finomabb különbségeket jelöli.

Például a `8.0` az NVIDIA A100-at, a `8.6` az RTX 3090-et / A10-et, a `9.0` a H100-at jelöli. A `major` szám növekedése általában nagy architekturális ugrást jelez, míg a `minor` változás kisebb képességbeli eltéréseket takar.

A compute capability nem azonos a CUDA toolkit verziójával! A toolkit verziója a szoftvert, a compute capability a hardvert jellemzi. Egy CUDA 12.2-es toolkit több különböző compute capability-jű GPU-t is támogathat, és egy adott GPU-t több különböző toolkit-verzió is támogathat (amíg a hardver nem avul el a szoftver számára).

2.2.7 A fontosabb generációk és jellemzőik

Az alábbi táblázat a főbb compute capability szinteket és a hozzájuk tartozó kulcsfontosságú hardveres jellemzőket foglalja össze. **Az értékek irányadóak; a pontos specifikációt mindig a hivatalos CUDA dokumentációban kell ellenőrizni.**

Compute capability	Generáció (példák)	Tensor core	Shared memória / SM (max)	Megjegyzések
3.0–3.7	Kepler	nincs	48 KiB (3.x)	Dinamikus párhuzamosság 3.5-től
5.0–5.3	Maxwell	nincs	96 KiB (5.0), 64 KiB (5.2/5.3)	Alacsonyabb energiaigényű SM
6.0–6.2	Pascal	nincs	64 KiB	NVLink, javított unified memory
7.0	Volta (V100)	1. generáció	96 KiB	Independent thread scheduling
7.5	Turing (RTX 20)	1. generáció	64 KiB	FP16 tensor core
8.0	Ampere (A100)	3. generáció	164 KiB	TF32, BF16, MIG, sparsity
8.6	Ampere (RTX 30)	3. generáció	100 KiB	Kisebb GPU-k, nincs MIG
8.9	Ada (RTX 40)	4. generáció	100 KiB	FP8 tensor core
9.0	Hopper (H100)	4. generáció	228 KiB	TMA, thread block clusters, DPX
10.0	Blackwell	5. generáció	(ellenőrizendő)	Továbbfejlesztett FP8, nagyobb skála

A táblázatból látható, hogy a compute capability nem csupán egy szám, hanem egy olyan kulcs, amely meghatározza, milyen hardveres funkciók érhetők el. Amikor egy kernel kódját írjuk vagy optimalizáljuk, mindig tudnunk kell, milyen compute capability-jű GPU-n fut majd. Például a tensor core-t használó kód csak a 7.0 vagy újabb compute capability-jű eszközökön működik, a TMA-t igénylő kód pedig csak 9.0-tól.

2.2.8 A compute capability és a fordítás

Az `nvcc` fordítás során a `-gencode` vagy az `-arch` kapcsolóval adhatjuk meg, milyen compute capability-jű eszközökre készüljön a bináris. Kétféle utasításkészletet állíthatunk elő:

- **SASS** – a GPU natív gépi kódja, amely egy adott compute capability-re specializált. A `-gencode arch=compute_XY,code=sm_XY` hatására SASS-t generálunk.
- **PTX** – egy köztes, virtuális utasításkészlet, amelyet a driver a futtatáskor (JIT) fordíthat a tényleges hardverre. A `-gencode arch=compute_XY,code=compute_XY` hatására PTX-et ágyazunk be.

A gyakorlatban a legjobb, ha mindkettőt generáljuk: az `sm_XY` SASS-t a gyors futtatáshoz, és a `compute_XY` PTX-et a jövőbeli hardverekkel való forward compatibility érdekében. Az `nvcc` alapértelmezésben is tartalmaz PTX-et, így a program egy újabb GPU-n is futtatható lehet, ha a driver JIT-fordítással képes azt lefordítani.

2.2.9 Verziókompatibilitás

A CUDA ökoszisztémában a kompatibilitásnak három fontos iránya van:

1. **Driver és runtime** – a driver verziójának legalább akkorának kell lennie, mint amit a toolkit minimálisan megkövetel. Egy újabb driver visszafelé kompatibilis a régebbi runtime-tal, de egy régebbi driver nem tudja futtatni az újabb toolkitkel fordított programot. A minimális driver verziót a CUDA release notes tartalmazza; például a CUDA 12.2 minimális driver verziója 535.104.05 (ellenőrizendő). A `cudaDriverGetVersion` és a `cudaRuntimeGetVersion` közötti eltérés tehát normális.
2. **Compute capability és toolkit** – a toolkit csak bizonyos compute capability-jű eszközöket támogat. A CUDA 11.0-tól kezdve a Kepler (3.x) támogatása megszűnt, a CUDA 12.0-tól pedig a minimális támogatott compute capability 5.0 (Maxwell). Ez azt jelenti, hogy egy CUDA 12.2-es kernel nem fog futni egy GTX 680-as (3.0) GPU-n, mert a toolkit már nem generál hozzá kódot, és a driver sem támogatja az újabb runtime-ot.
3. **Könyvtárak és toolkit** – a cuBLAS, cuFFT stb. verziói a toolkit verziójához kötöttek. Ha eltérő verziójú könyvtárat linkelünk, ABI inkompatibilitás léphet fel, ami linkelési hibát vagy futásidejű undefined behavior-t okozhat.

2.2.10 Labor: CUDA-verzió és GPU-generáció kompatibilitási mátrixa

A labor célja, hogy a fejezetben bemutatott fogalmakat egy konkrét, ellenőrizhető programmal kössük össze. A program lekérdezi a driver és a runtime verzióját, valamint a rendszerben található összes CUDA-képes eszköz compute capability-jét és néhány fontos tulajdonságát. Az így kapott adatokat egy kompatibilitási mátrixba rendezzük, és összehasonlítjuk a gyártói dokumentációból származó várt értékekkel.

2.2.11 Környezet és build

A labor feltételez egy CUDA 12.x toolkitet és egy tetszőleges, legalább 5.0 compute capability-jű GPU-t. A mérési naplóban minden adatot rögzíteni kell: GPU modell, driver verzió, CUDA verzió, operációs rendszer, compute capability.

A forráskód neve legyen `device_query.cu`. Fordítás:

```
nvcc -std=c++17 -o device_query device_query.cu
```

Futtatás:

```
./device_query
```

2.2.12 A program

A következő CUDA C++ program a `cudaDriverGetVersion`, `cudaRuntimeGetVersion`, `cudaGetDeviceCount` és `cudaGetDeviceProperties` API-kat használja. Minden hívást ellenőrzünk a megadott hibamakróval. A program nem indít kernelt, ezért nincs szükség warm-upra; a lekérdezés maga nem igényel aszinkron végrehajtást, de a `cudaDeviceSynchronize` hívást a konvenció kedvéért elhelyezzük.

```
// CUDA C++ (C++17)
#include <cstdio>
#include <cstdlib>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                    \
  do {                                                                      \
    cudaError_t err = (call);                                               \
    if (err != cudaSuccess) {                                               \
      std::fprintf(stderr, "CUDA hiba: %s:%d: %s\n", __FILE__, __LINE__,    \
                   cudaGetErrorString(err));                                \
      std::exit(EXIT_FAILURE);                                              \
    }                                                                       \
  } while (0)

int main() {
  int driver_version = 0;
  int runtime_version = 0;
  int device_count = 0;

  CUDA_CHECK(cudaDriverGetVersion(&driver_version));
  CUDA_CHECK(cudaRuntimeGetVersion(&runtime_version));
  CUDA_CHECK(cudaGetDeviceCount(&device_count));

  std::printf("CUDA driver verzió: %d.%d\n", driver_version / 1000,
              (driver_version % 1000) / 10);
  std::printf("CUDA runtime verzió: %d.%d\n", runtime_version / 1000,
              (runtime_version % 1000) / 10);
  std::printf("Eszközök száma: %d\n\n", device_count);

  for (int dev = 0; dev < device_count; ++dev) {
    cudaDeviceProp prop{};
    CUDA_CHECK(cudaGetDeviceProperties(&prop, dev));

    std::printf("Eszköz %d: %s\n", dev, prop.name);
    std::printf("  Compute capability: %d.%d\n", prop.major, prop.minor);
    std::printf("  Globális memória: %.2f GiB\n",
                static_cast<double>(prop.totalGlobalMem) /
```

```
                  (1024.0 * 1024.0 * 1024.0));
    std::printf("  Multiprocesszorok száma: %d\n", prop.multiProcessorCount);
    std::printf("  Maximális szálak blokkonként: %d\n",
                prop.maxThreadsPerBlock);
    std::printf("  Warp méret: %d\n", prop.warpSize);
    std::printf("  Regiszterek blokkonként: %d\n", prop.regsPerBlock);
    std::printf("  Shared memory blokkonként: %u KiB\n",
                prop.sharedMemPerBlock / 1024);
    std::printf("  Constant memória: %u KiB\n", prop.totalConstMem / 1024);
    std::printf("\n");
  }

  CUDA_CHECK(cudaDeviceSynchronize());
  return 0;
}
```

A program kimenete a konzolra írja a verziókat és az eszköztulajdonságokat. A verziószámok értelmezésénél figyeljünk: a `cudaDriverGetVersion` és a `cudaRuntimeGetVersion` a verziót 1000-szeres szorzóval adja vissza, azaz 12020 a 12.2-es verziót jelenti. A fenti kódban a kiíráshoz elosztjuk 1000-rel és a maradékot 10-rel, hogy a `12.2` formátumot kapjuk.

2.2.13 Példa kimenet (illusztráció, nem ellenőrzött mérés)

Az alábbi kimenet egy NVIDIA A100 40 GB-os GPU-val rendelkező, CUDA 12.2-es környezetből származik. **Ez a kimenet a könyvben példaként szerepel; a valós mérési naplót minden olvasónak a saját, izolált környezetében kell rögzítenie, és a mért értékeket nem szabad ellenőrzöttként feltüntetni.**

```
CUDA driver verzió: 12.2
CUDA runtime verzió: 12.2
Eszközök száma: 1

Eszköz 0: NVIDIA A100-SXM4-40GB
  Compute capability: 8.0
  Globális memória: 39.59 GiB
  Multiprocesszorok száma: 108
  Maximális szálak blokkonként: 1024
  Warp méret: 32
  Regiszterek blokkonként: 65536
  Shared memory blokkonként: 48 KiB
  Constant memória: 64 KiB
```

Ebben a példában a driver és a runtime verzió megegyezik (12.2), mert a telepített driver ugyanabból a CUDA generációból származik. A compute capability 8.0, ami megfelel az A100 hivatalos specifikációjának.

2.2.14 A kompatibilitási mátrix elkészítése

A mérési naplóban a következő táblázatot kell kitölteni. A **mért** oszlopok a program kimenetéből származnak, a **várt** oszlopok a gyártói adatlapból vagy a hivatalos CUDA dokumentációból. Az eltérés oszlopba azt írjuk, hogy a két érték egyezik-e, és ha nem, mi a különbség.

Eszköz	Compute capability (mért)	Compute capability (várt)	Driver verzió (mért)	CUDA runtime verzió (mért)	Eltérés / megjegyzés
NVIDIA A100-SXM4-40GB	8.0	8.0 (gyártói adatlap)	12.2	12.2	nincs, egyezik

Egy valós laborban a táblázat több eszközt is tartalmazhat, ha a rendszerben több GPU van. A várt compute capability-t a GPU modelljének hivatalos adatlapjáról vagy a CUDA GPU listából lehet kikeresni. A driver és runtime verziók várt értékei a CUDA release notes-ból származnak (minimális driver verzió).

A mérési naplóban rögzíteni kell továbbá:

- A futtatás dátuma és időpontja.
- Az operációs rendszer neve és verziója.
- A telepített CUDA toolkit pontos verziója (pl. 12.2.140).
- A GPU hőmérséklete és órajele, ha az elérhető (a `nvidia-smi` parancs kimenetéből).
- A háttérterhelés állapota (pl. nincs más GPU-feladat).

A mérés előtt a programot legalább 5–10 alkalommal futtassuk le, és az eredményeket jegyezzük fel. Mivel a lekérdezés determinisztikus, az eredményeknek azonosnak kell lenniük; ha mégsem, akkor driver- vagy hardverprobléma áll fenn. A kimeneteket átlagolni nem szükséges, de a konzisztencia ellenőrzése ajánlott.

2.2.15 Tipikus hibák és helyes megoldások

A fejezet tanulási céljai között szerepel a tipikus hibák felismerése. Az alábbi esetek a leggyakrabban előforduló problémákat mutatják be.

2.2.16 1. hiba: A verziószám téves értelmezése

A `cudaDriverGetVersion` és a `cudaRuntimeGetVersion` a verziót 1000-szeres szorzóval adja vissza. Ha valaki ezt közvetlenül, osztás nélkül írja ki, hibás értéket kap.

Hibás kód:

```
int v;
cudaDriverGetVersion(&v);
std::printf("Driver: %d\n", v);  // 12020 jelenik meg 12.2 helyett
```

Helyes megoldás:

```
std::printf("Driver: %d.%d\n", v / 1000, (v % 1000) / 10);
```

2.2.17 2. hiba: A compute capability összekeverése a CUDA verzióval

Egy kezdő fejlesztő gyakran azt hiszi, hogy a „CUDA 12.2” ugyanazt jelenti, mint a „compute capability 12.2”. Ez téves: a compute capability a hardveré, a CUDA verzió a szoftveré. Egy 8.0 compute capability-jű GPU (A100) tökéletesen futtatható CUDA 12.2-es környezetben, mert a 12.2-es toolkit támogatja a 8.0-t. A kettő közötti kapcsolat a kompatibilitási mátrixban válik láthatóvá.

2.2.18 3. hiba: A hardveres funkciók feltételezése ellenőrzés nélkül

Ha egy kernel tensor core-t vagy más speciális utasítást használ, a kódnak futásidőben ellenőriznie kell, hogy a GPU támogatja-e. Például a `__CUDA_ARCH__` makró fordítási időben segít, de a legjobb, ha a compute capability-t a `cudaDeviceProp` segítségével is lekérdezzük, és a funkciót csak akkor aktiváljuk, ha a hardver támogatja.

Hibás minta:

```
// Feltételezi, hogy a GPU támogatja a tensor core-t.
// Ez a kód egy 6.1-es (Pascal) GPU-n nem fog működni.
wmma::fragment<wmma::accumulator, 16, 16, 16, float> frag;
```

Helyes megoldás: a compute capability ellenőrzése futásidőben:

```
if (prop.major >= 7) {
  // tensor core használata biztonságos
} else {
  // fallback implementáció
}
```

2.2.19 4. hiba: Régi driver használata új toolkitkel

Ha a driver verziója alacsonyabb, mint a toolkit által megkövetelt minimális verzió, a program futása `cudaErrorInsufficientDriver` hibával meghiúsul. A hibaüzenet jellemzően arra utal, hogy a driver túl régi. A megoldás a driver frissítése a hivatalos NVIDIA oldalról.

2.2.20 5. hiba: Nem megfelelő könyvtárverzió linkelése

A cuBLAS, cuFFT stb. verziója a toolkit verziójához kötött. Ha egy projekt régebbi cuBLAS-t linkel, miközben a fejlécek az újabb toolkitből származnak, akkor linkelési hibák vagy futásidejű ABI inkompatibilitás léphet fel. A megoldás: mindig a telepített toolkitkel szállított könyvtárakat használjuk, és a build rendszerben a `-lcublas` stb. kapcsolókkal a megfelelő keresési útvonalat adjuk meg.

2.2.21 Összefoglalás

Ebben az alfejezetben áttekintettük a CUDA ökoszisztéma három alapkövét:

- A **CUDA toolkit** a fejlesztői eszközöket (nvcc, profilozók, debugger) és a nagy teljesítményű könyvtárakat tartalmazza.
- A **CUDA driver** alacsony szintű hozzáférést biztosít, és a **runtime API** kényelmes absztrakciót nyújt a driver fölé.
- A **compute capability** a hardvergenerációk képességeit írja le, és meghatározza, milyen funkciók érhetők el a kernelben.

A labor során egy eszköz-lekérdező programmal kompatibilitási mátrixot építettünk, amely a mért és a várt értékek összehasonlításával teszi ellenőrizhetővé a környezet helyességét. Ez a mátrix a későbbi fejezetekben is hasznos lesz, amikor a hardveres jellemzők alapján kell optimalizálni.

Önellenőrző kérdés: Milyen összefüggés van a CUDA toolkit verziója, a driver verziója és a compute capability között? Miért fontos a minimális driver verzió ellenőrzése egy új toolkit telepítésekor?

Válasz: A toolkit verziója határozza meg, milyen szoftveres funkciók érhetők el, és milyen minimális driver verzió szükséges a futtatáshoz. A driver verziója a ténylegesen telepített GPU-illesztőt jelzi, és visszafelé kompatibilis a régebbi runtime-okkal, de nem lehet régebbi, mint a toolkit által megkövetelt minimum. A compute capability a hardver képességeit írja le, és a toolkit támogatási köre alapján dönti el, hogy egy adott GPU-n futtatható-e a kód. A minimális driver verzió ellenőrzése azért fontos, mert egy túl régi driverrel a program nem tud elindulni, és a hiba gyakran csak futásidőben jelentkezik.

2.3 Az akcelerátorok térképe: GPU, TPU, FPGA és mások

Az előző alfejezetben megvizsgáltuk a CUDA ökoszisztémát és a hardvergenerációkat. Most egy lépéssel hátrébb lépünk, és feltérképezzük a modern számítástechnika akcelerátorait. Nem minden számítási probléma oldható meg hatékonyan egyetlen architektúrán; a mérnök feladata, hogy a feladat adatfolyamát, aritmetikai intenzitását és késleltetési igényét a hardveres végrehajtási modellhez igazítsa. Ez az alfejezet a fő akcelerátortípusokat, azok programozási modelljeit (SIMT, szisztolikus tömb, adatfolyam), valamint a memória, hálózat és energia szempontjait mutatja be.

2.3.1 A tájékozódás alapelvei: miért léteznek akcelerátorok?

A CPU-kat általános célú, nagy órajelű, alacsony késleltetésű végrehajtásra tervezték. Hatalmas tranzisztorbüdzséjüket előrejelzésre, soron kívüli végrehajtásra (out-of-order execution), nagy L2/L3 cache-ekre és komplex irányításvezérlésre költik. Ez rendkívül alkalmazkodóvá teszi őket, de a fizika korlátai (a falak hője és a Dennard-skálázódás vége) miatt a fajlagos energia (Joule/művelet) és az átbocsátóképesség (FLOPs) elmarad az optimalizált társprocesszorokétól.

Az akcelerátor a vezérlés egyszerűsítésére és a párhuzamos számítási egységek maximalizálására cseréli az általánosságot. A kérdés mindig az, hogy a munkaterhelés illeszkedik-e az adatfolyam és a hardveres ütemezés korlátaihoz. Három domináns paradigmát különböztetünk meg:

1. **Többszálú SIMD/SIMT:** Több ezer párhuzamos szál, amelyek szinkron csoportokban hajtódnak végre (pl. GPU).

2. **Szisztolikus tömbök és mátrixmotorok:** Az adatok ritmikusan áramlanak egy fixen bekötött számítási rácsban, kihasználva az adatok újrafelhasználását (pl. TPU, Tensor Core).
3. **Adatfolyam/térbeli (spatial) architektúrák:** A felhasználó határozza meg a hardveres huzalozást, ahol az adatok folyamatosan áramlanak át a logikai kapukon (pl. FPGA).

A következőkben ezeket a paradigmákat vizsgáljuk meg a hozzájuk tartozó tipikus hardverekkel.

2.3.2 1. GPU és a SIMT modell mélysége

Bár a GPU architektúráját a következő főfejezet (GPU-architektúra: SIMT, warp, SM és memória) részletesen tárgyalja, itt a többi akcelerátorhoz viszonyított helyét kell megértenünk. A GPU a **SIMT (Single Instruction, Multiple Threads)** modellt valósítja meg.

A programozási modell lényege: A fejlesztő skaláris kódot ír egyetlen szálra. A hardver ezt a kódot **warpokba** csoportosítja (NVIDIA esetén 32 szál). Az SM-en (Streaming Multiprocessoron) belül az ütemező ugyanazt az utasítást hajtja végre a 32 szálon, csak más-más adatelemen. Ez a megoldás hatalmas tranzisztorsűrűséget tesz lehetővé a lebegőpontos egységek (ALU/FP32/FP64) számára, mert a vezérlést nem kell szálanként replikálni.

A memóriamodell: A GPU globális memóriája (GDDR6/HBM3) rendkívül nagy sávszélességet (+1 TB/s csúcs) biztosít, de a késleltetése is nagy (több száz ciklus). A GPU-k ezt a késleltetést tömeges párhuzamossággal (oversubscription) rejtik el: amíg egy warp memóriára vár, az SM azonnal átkapcsol egy másik, készenlétben álló warpra. Ez az ún. **latency hiding**.

Mikor erős? Ha a feladat jól párhuzamosítható, de közben szabálytalan adathozzáférést vagy dinamikus vezérlésfolyamot igényel. A GPU kiválóan kezeli az általános célú számítást (GPGPU), a tudományos szimulációt, a 3D grafikát és a modern deep learning nagy részét.

Mikor gyenge? Amikor nagyon alacsony (<10 mikroszekundum) késleltetésre van szükség, mivel a magas órajel hiánya és a kernelindítási overhead miatt a GPU nem alkalmas szűk, szekvenciális hurkokra.

2.3.3 2. TPU és a szisztolikus tömbök

A Google 2015-ben ismerte fel, hogy ha a felhasználók hangalapú keresést használnak, a meglévő adatközponti GPU-k száma nem lesz elegendő a konvolúciós és LSTM-modellek kiszolgálására. A válasz a **TPU (Tensor Processing Unit)** lett, amelynek szíve egy **szisztolikus tömb (systolic array)**.

A szisztolikus tömb működése: Képzeljünk el egy kétdimenziós rácsot, amelyben minden cella egy MAC (Multiply-Accumulate) egység. A súlyokat (weights) előre betöltjük a cellákba. A bemeneti adatok balról jobbra, a parciális összegek pedig fentről lefelé áramlanak ritmikusan. Fontos jellemző, hogy az adatokat nem kell minden művelet után a memóriába visszaírni; azok a szomszédos cellák regisztereiben maradnak. Ez maximalizálja az adat-továbbítás energiahatékonyságát, mivel a memória-elérés drágább, mint az aritmetika.

A programozási modell: A TPU-t nem hagyományos, szálalapú C++-ban programozzuk, hanem **gráfszintű fordítókkal**. A fejlesztő magas szintű keretrendszerben (JAX, TensorFlow) definiál egy számítási gráfot. Az **XLA (Accelerated Linear Algebra)** fordító ezt a gráfot optimalizálja, és végrehajtható utasításokra bontja le a TPU számára. A TPU programozási modelljére és a JAX-re a VI. részben (TPU architektúra és XLA, JAX és TPU programozás) részletesen kitérünk.

Mikor erős? A szisztolikus tömb a sűrű mátrixszorzás (GEMM) és a konvolúció bajnoka. A TPU-k az energiahatékonyság (TOPS/Watt) terén sokáig verték az azonos időszakból származó GPU-kat, mert a vezérlés szinte teljesen eltűnik, és az adatmozgás minimális.

Korlátai: A TPU korlátozottan kezeli a ritka (sparse) struktúrákat, a bonyolult pointer-aritmetikát vagy az erősen elágazó kódot. Ráadásul a fordítási modell kevésbé flexibilis, mint a CUDA C++.

2.3.4 3. FPGA és az adatfolyam-paradigma

Az **FPGA (Field-Programmable Gate Array)** egy olyan integrált áramkör, amelyet a gyártás után konfigurálhatunk. Szilíciumot nem tervezünk, hanem a meglévő logikai blokkok (LUT-ok), digitális jelfeldolgozó egységek (DSP-k) és belső blokk-RAM-ok (BRAM) közötti huzalozást definiáljuk.

Adatfolyam (Dataflow) architektúra: Míg a GPU-ban egyetlen SM-en sok warp osztozik, az FPGA-ban az algoritmust **térben** bontjuk ki. Az FPGA-ban nincsenek hagyományos értelemben vett utasítások. Ahelyett, hogy egy program utasításokat hajtana végre, az FPGA-n egy adatfolyamot valósítunk meg: a bemeneti adat minden órajelciklusban átlép egy logikai fokozaton, ahol a műveletet elvégzik, majd továbblép a következő fokozatba. Ez a **pipeline-olt adatfolyam (C-slow / streaming)** végrehajtás garantáltan alacsony, fix késleltetést biztosít.

A programozási modell: Hagyományosan **HDL**-ben (Verilog, VHDL) írták le a hardvert. Mára elterjedt a **High-Level Synthesis (HLS)**, ahol C/C++ (vagy OpenCL/SYCL) szinten írhatunk, és a fordító (pl. Vitis HLS) generálja a hardverleírást. Ez azonban nem ugyanaz a C++, mint amit egy CPU-ra írunk: az erőforrások lefoglalása (`#pragma HLS unroll`, `array_partition`) elengedhetetlen a jó teljesítményhez.

Mikor erős? Az FPGA-k kiválóak ultra-alacsony késleltetésű pénzügyi feldolgozásban (HFT), szenzorillesztésben, szoftveresen definiált rádióban és olyan hálózati feladatokban, ahol a bites szintű, determinisztikus feldolgozás a döntő.

Korlátai: A maximális órajel jóval alacsonyabb (jellemzően 300-700 MHz), mint egy CPU-é vagy GPU-é. A "fordítási idő" (place & route) több órát is igénybe vehet. A sűrű FP32/FP64 matematikában az FPGA-k tranzisztoronkénti hatékonysága csapnivaló a GPU-hoz képest.

2.3.5 4. Egyéb akcelerátorok: ASIC-ok és NPU-k

A térkép teljessége érdekében említsük meg a két szélső pólust:

- **ASIC (Application-Specific Integrated Circuit):** Egyetlen feladatra tervezett, fix logikájú chip. Nincs programozhatóság a gyártás után. A legjobb teljesítmény/watt mutatót adja, de a hibás feltételezés vagy a gyorsan változó algoritmus (pl. a 2018-as évek kriptográfiai vagy AI trendjei) milliárdos kudarcot okozhat. A Bitcoin bányász chipek tökéletes példái a "memória nélküli, tiszta számítás" ASIC-oknak.
- **NPU (Neural Processing Unit):** Számos modern SoC-be (pl. mobiltelefonokba) integrált kisebb energiaosztályú áramkör. Olyan, mintha egy aszimmetrikus, limitált képességű TPU lenne. Programozási modelljük jellemzően fix, gyártói API-kon vagy kvantált (INT8/INT4) gráfokon keresztül történik (pl. Apple CoreML, Android NNAPI).

2.3.6 5. A nagy hármas: Memória, hálózat, energia

Az akcelerátorok összehasonlításakor a puszta "FLOPs" (lebegőpontos művelet/másodperc) értéke megtévesztő. A valós rendszerteljesítményt a **memória**, a **hálózat** és az **energia** határozza meg.

2.3.6.1 Memória

A szorzás olcsóbb, mint az adatmozgatás.

- **GPU (pl. NVIDIA H100):** HBM3 memóriája ~3 TB/s sávszélességet biztosít, de ezt a több millió szál között kell elosztani. Memóriahierarchiája mély (regiszter -> L1/Shared -> L2 -> HBM).
- **TPU:** A memória (pl. HBM) mellett a szisztolikus tömbben lévő lokális regiszterek miatt drasztikusan csökken a memória felé irányuló tényleges forgalom a GEMM műveletek során.
- **FPGA:** Itt a hierarchia laposabb, de rugalmas. Több ezer kis, független BRAM blokk található a chipen, amelyeket az algoritmushoz igazíthatunk (pl. egyedi cache-ek, FIFO-k).

2.3.6.2 Hálózat és skálázhatóság

Egyetlen chip ma már kevés a nagy modellekhez. Az akcelerátorok közötti kapcsolat dönti el, mennyire skálázható a rendszer.

- **GPU:** NVIDIA az **NVLink**-et és **NVSwitch**-et használja a nagy sebességű GPU-GPU kommunikációhoz (több száz GB/s), míg a rendszerszintű adatforgalom **GPUDirect**-en keresztül mehet.
- **TPU:** A Google adatközponti TPU podokban az **ICI (Inter-Chip Interconnect)** nevű, nagy sebességű, toroidális topológiájú hálózatot használ, amely közvetlen chip-chip kapcsolatot biztosít CPU-k megkerülésével.
- **FPGA:** Gyakran direkt optikai vagy nagy sávszélességű Ethernet kapcsolattal (pl. Solarflare/Mellanox kártyákkal együttműködve) használják, ahol a hálózati csomag feldolgozása és a piaci algoritmus a chipen belül történik, mielőtt a CPU-hoz érne.

2.3.6.3 Energia

Az energiafogyasztás a végső korlát. A CPU-k általános célú utasítás-lekérése és dekódolása (frontend) rengeteg energiát emészt fel. A GPU ezt úgy csökkenti, hogy egy utasítást 32 adatelemen hajt végre. A TPU tovább csökkenti az energiaigényt, mert a szisztolikus tömb eliminálja a regiszterfájl-olvasást és a nagy vezérlési logikát. Az FPGA a legalacsonyabb bitszintű fogyasztást érheti el fix, optimális esetben, ha a feladat nem igényel nagy pontosságú lebegőpontos egységeket. A "sötét szilícium" (dark silicon) elvének megfelelően nem tudjuk egyszerre kihasználni az összes tranzisztort a teljesítmény limit (TDP) betartása mellett, ezért a specializáció (azaz, hogy csak az adott feladathoz szükséges áramkörök égjenek) a jövőbeli hatékonyság kulcsa.

2.3.7 6. Labor: Akcelerátorválasztási döntési mátrix

A következő labor célja, hogy a fejezetben tárgyalt elveket egy konkrét, ellenőrizhető példán keresztül alkalmazzuk. A feladat nem egy hardver beszerzése, hanem egy strukturált döntési keretrendszer felállítása.

A feladat: Adott egy munkaterhelés-trió: (A) Dense Mátrixszorzás (pl. nagy nyelvi modellek), (B) Ultra-alacsony késleltetésű jelfeldolgozás (pl. 5G upstream), (C) Hételemű rendezett fa bejárása (irregular pointer chasing). Építsünk döntési mátrixot.

A módszertan: A labor gerince egy SAXPY művelet ($y = a \cdot x + y$) három különböző programozási modellben, annak szemléltetésére, hogy ugyanaz a matematika milyen eltérő kódolást és hardveres erőforrást igényel.

2.3.7.1 6.1. CUDA C++ Példa (SIMT)

Nyelv: C++17, Fordító: NVCC.

```
// CUDA C++: SIMT modell
#include <cstdio>
#include <cuda_runtime.h>

// Hibamakró (kötelező a könyv szerződése alapján)
#define CHECK_CUDA(call) {                                                  \
    cudaError_t err = call;                                                 \
    if (err != cudaSuccess) {                                               \
        fprintf(stderr, "CUDA hiba: %s:%d, %s\n", __FILE__, __LINE__,       \
                cudaGetErrorString(err));                                   \
        cudaDeviceReset();                                                  \
        return EXIT_FAILURE;                                                \
    }                                                                       \
}

__global__ void saxpy_kernel(int n, float a, float *x, float *y) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    // SIMT: ugyanaz az utasítás 32 szálra
    if (i < n) y[i] = a * x[i] + y[i];
}

int main() {
    const int N = 1 << 20;
    // Helyőrző elnevezések, nincs valós őrzött adat
    size_t bytes = N * sizeof(float);
    float *d_x, *d_y;

    CHECK_CUDA(cudaMalloc(&d_x, bytes));
    CHECK_CUDA(cudaMalloc(&d_y, bytes));
    // Inicializálás elhagyva a tömörség kedvéért

    // Launch konfiguráció: grid=1024, block=1024
    saxpy_kernel<<<1024, 1024>>>(N, 2.0f, d_x, d_y);
    CHECK_CUDA(cudaGetLastError());
    CHECK_CUDA(cudaDeviceSynchronize());

    cudaFree(d_x); cudaFree(d_y);
    return 0;
}
```

Megfigyelés: A kód egyetlen szálra van írva, de 1 millió szálon fut. A hardver az indexelést (`i`) osztja ki a párhuzamos egységeknek. A modell jól viseli a `if (i < n)` elágazást.

2.3.7.2 6.2. OpenCL Példa (Hordozható heterogén)

Nyelv: OpenCL C++ host API.

```
// OpenCL C++: Platform-modell
// Itt a kernel ugyanaz a logika, de a futtató környezetet manuálisan kell felépíteni.
// Az NVIDIA drivér OpenCL ICD-t is szolgáltat, de a gyakorlatban a CUDA a hatékonyabb az NVIDIA
↪  GPU-n.
cl::Context context(CL_DEVICE_TYPE_GPU);
std::vector<cl::Device> devices = context.getInfo<CL_INFO_DEVICES>();
cl::CommandQueue queue(context, devices[0]);

// ... kernel beolvasása ...
// cl::Kernel kernel(program, "saxpy_kernel");
// kernel.setArg(0, n); kernel.setArg(1, a); ...
// queue.enqueueNDRangeKernel(kernel, cl::NullRange, cl::NDRange(N), cl::NDRange(256));
```

Megfigyelés: Az OpenCL platformabsztrakciót ad a GPU, CPU, FPGA közé, de cserébe a teljesítményfüggő részletek (pl. shared memory, warp szintű intrinsics) elvesznek vagy nagyon bőbeszédűvé válnak.

2.3.7.3 6.3. JAX Példa (Gráfmodell / TPU)

Nyelv: Python 3, JAX (XLA backend).

```
import jax
import jax.numpy as jnp

# JAX: funkcionális, gráf-alapú programozás.
# Nem írunk thread-indexet. A hardver (GPU TPU, vagy akár CPU) el van rejtve.
@jax.jit  # XLA fordítás
def saxpy(a, x, y):
    return a * x + y

key = jax.random.PRNGKey(0)
x = jax.random.normal(key, (1_000_000,), dtype=jnp.float32)
y = jax.random.normal(key, (1_000_000,), dtype=jnp.float32)

# A futás lehet szisztolikus tömbben vagy SIMT modulokban, a háttértől függően.
result = saxpy(2.0, x, y)
```

Megfigyelés: A kód nem ír elő "hogyan fut" a számítás. A JAX/XLA a legjobb memóriaelrendezést és erőforrás-allokációt választja.

2.3.7.4 6.4. A Döntési Mátrix és a mérési illúzió

A labor során a fenti három feladat típusra kell felvenni a döntési mátrixot.

Jellemző	NVIDIA GPU	Google TPU	FPGA	NPU / ASIC
Programozási modell	SIMT (CUDA C++)	Gráf (JAX/XLA)	Adatfolyam (HLS/HDL)	Fix/Gráf (SDK)
Erősség (A) Dense GEMM	**Kiváló**	**Kiváló**	Gyenge	Kíváló (fix)
Erősség (B) Ultra-alacsony latency	Gyenge (10-50 us)	Gyenge	**Kiváló (ns)**	Kíváló

Jellemző	NVIDIA GPU	Google TPU	FPGA	NPU / ASIC
Erősség (C) Irregular pointer	**Jó**	Rossz	Nagyon gyenge	Rossz
Fejlesztési sebesség	Nagyon magas	Magas	Nagyon alacsony	Változó
Energiahatékonyság (op/W)	Jó	Kiváló	Kiváló	Eévi legjobb

Tipikus hibák és a helyes megoldás:

1. **Hiba:** "Az FPGA-nak 10 TFLOPs a dokumentált csúcsteljesítménye, tehát oda fogom tenni a mátrixszorzást, mert gyorsabb, mint a GPU."
    - **Megoldás:** Az FPGA DSP-k csak akkor érik el a csúcsot, ha az adatfolyam tökéletesen ki van pipelinelve, és az adatok fix, rövid bit-szélességűek. FP32-es GEMM-nél a routing késleltetése és a memória sávszélessége miatt a szám gyakran irreleváns. Helyes döntés: GPU vagy TPU.
2. **Hiba:** "A TPU-n lefuttatom a bruteforce rendező algoritmusomat soros ciklusban."
    - **Megoldás:** A TPU nem általános célú processzor; a szekvenciális, nagymértékben elágazó kódot a CPU-n vagy GPU-n kell hagyni, a TPU-t csak a mátrixműveletekre használni.
3. **Hiba:** "A GPU azonnal reagál a hálózati csomagokra, hiszen 4096 magja van."
    - **Megoldás:** A kernelindítás és a PCIe DMA másolás késleltetése mikro-szekundumos tartományban van. Ha nanomásodperces reakció kell, az adatot az FPGA-n kell feldolgozni.

2.3.7.5 6.5. Mérési napló és várt vs. mért eredmény

Mivel ebben a laborban egyszerre vizsgálunk eltérő architektúrákat, a labor jegyzőkönyvnek pontosan el kell különítenie a **várható (elméleti vagy dokumentáció)** és a **mért (laboratóriumi benchmark)** adatokat, ahogy azt a könyv "Kód- és laborszerződése" előírja.

Példa jegyzőkönyv a Dense mátrixszorzáshoz (A):

- **Hardver:** NVIDIA RTX 3060 (Ampere, CC 8.6), Driver: 535.xx, CUDA: 12.2, Power limit: 170W, ECC: Ki, MIG: Ki.
- **Módszertan:** 1M elemű SAXPY, Warm-up: 10 iteráció, Mérés: 100 iteráció.
- **Várt eredmény (Dokumentáció alapján):** A GPU névleges sávszélessége 360 GB/s. A SAXPY 3 memória-hozzáférést igényel (2 read, 1 write), tehát a várt átviteli idő ~0.35 ms.
- **Mért eredmény (Nsight Compute alatt):** 0.41 ms. A különbség oka: a PCIe allokáció overhead-je és a kernel memória-alrendszer kihasználtsága.
- **Következtetés:** A feladat erősen memória-korlátos. A FLOPs nem releváns, a sávszélesség a döntő.

2.3.8 Ellenőrző kérdések

1. Mi a fundamentális különbség a SIMT (GPU) és a szisztolikus tömb (TPU) adatfolyama között a parciális összegek kezelésében?
2. Miért lehet az FPGA jobb választás egy 5G MAC-réteg feldolgozására, mint egy GPU, ha a GPU-nak ötször nagyobb a nyers számítási teljesítménye?
3. Milyen szerepet játszik az NVLink/ICI a multi-akcelerátoros rendszerekben, és miért nem elegendő a szabványos PCIe?
4. Az Ön cége ritka, dinamikus gráfokat elemez (pl. közösségi háló), ahol a memória-hozzáférési minták véletlenszerűek. A TPU csúcs FLOPs-ja tízszerese a GPU-énak. Melyiket választja és miért? (Válaszát a programozási modell és a memória-hierarchia tükrében indokolja!)

2.4 Mikor éri meg GPU-ra vinni egy feladatot?

Amikor egy mérnök először találkozik a GPU-val, a leggyakoribb kérdés az, hogy „ez most gyorsabb lesz?" A kérdés rossz, mert a GPU nem univerzális gyorsító, hanem speciális végrehajtó egység, amely bizonyos feltételek mellett ad jelentős gyorsulást. Ebben a fejezetben felépítjük azt a szempontrendszert, amellyel egy feladat GPU-alkalmasságát meg lehet ítélni, még mielőtt az ember nekiállna a kernel megírásának. A cél nem az, hogy minden feladatot GPU-ra vigyünk, hanem az, hogy a döntést mérhető és reprodukálható kritériumokra alapozzuk. A fejezet végére egy olyan ellenőrző listát adunk, amely a gyakorlatban is használható, és egy laborfeladatban alkalmazzuk is egy konkrét példán.

2.4.1 5.1 A GPU-alkalmasság kritériumrendszere

A GPU-alkalmasság nem egyetlen bináris tulajdonság, hanem több, egymással összefüggő feltétel együttese. A legfontosabb kritériumok a következők:

1. **Elegendő párhuzamosság.** A GPU több ezer szálat futtat egyidejűleg, ezért a feladatnak olyan adat- vagy feladatpárhuzamosságot kell mutatnia, amely legalább néhány ezer, ideális esetben százezres nagyságrendű szálat tud ellátni hasznos munkával. Ha a feladat csak néhány tucat független műveletet tartalmaz, a GPU erőforrásai kihasználatlanok maradnak.
2. **Kedvező arithmetic intensity.** Az arithmetic intensity (AI) azt méri, hogy egy bájtnyi memóriaforgalomra hány lebegőpontos művelet jut. A GPU akkor hatékony, ha a feladat vagy magas AI-vel rendelkezik (azaz számítás-kötött), vagy ha a memória-sávszélességet jól ki tudja használni. Az alacsony AI-jű feladatoknál a GPU memória-alrendszere lehet a szűk keresztmetszet, de megfelelő méret és hozzáférési minta esetén még így is jelentős gyorsulást adhat.
3. **Alacsony adatmozgatási költség a teljes láncban.** A GPU-hoz a CPU-val való adatcsere PCIe vagy NVLink kapcsolaton keresztül történik. Ha a feladat kis adatmennyiséget mozgat, de sok számítást végez, a transzfer elhanyagolható. Ha viszont minden kernelindítás előtt és után nagy tömböket kell másolni, a transzfer idő hamar felemésztheti a kernel gyorsulását.
4. **Rendszeres memória-hozzáférés.** A GPU memóriavezérlője akkor éri el a csúcssávszélességet, ha a szomszédos szálak szomszédos memóriacímeket érnek el (coalescing). Ha a hozzáférés szétszórt vagy véletlenszerű, a tényleges sávszélesség töredékére esik vissza. A GPU-alkalmasság vizsgálatakor már a kiinduló algoritmus szintjén érdemes megnézni, hogy az adatelérés rendezhető-e koaleszkált mintára.

5. **Korlátozott szinkronizáció.** A GPU-n a globális szinkronizáció drága, a blokkon belüli szinkronizáció pedig csak akkor hatékony, ha ritkán használják. Ha az algoritmus sok szinkronizációs pontot igényel, a párhuzamosság előnye elveszhet. A feladatnak lehetőleg olyan független részekre kell bonthatónak lennie, amelyek szinkronizáció nélkül is végrehajthatók.
6. **Megfelelő precizitás.** A GPU-k hagyományosan az FP32 és FP16/BF16/TF32/FP8 formátumokban erősek, míg az FP64 teljesítményük – különösen a fogyasztói és a legtöbb adatközponti GPU-n – töredéke az FP32-ének. Ha a feladat FP64-es pontosságot igényel, és a GPU adott típusa ebben gyenge, a CPU vagy egy másik akcelerátor jobb választás lehet.
7. **Szoftveres támogatás.** A CUDA, OpenCL, JAX/XLA és a könyvtárak (cuBLAS, cuFFT, Thrust, CUB, CUTLASS) jelentősen leegyszerűsítik a fejlesztést. Ha a feladathoz nincs megfelelő könyvtár vagy keretrendszer, a saját kernel írása többletköltséggel jár; ez akkor éri meg, ha a várható gyorsulás ezt kompenzálja.

Ezek a kritériumok nem függetlenek: például a magas AI sokszor nagy adatpárhuzamossággal jár együtt, a koaleszkált hozzáférés pedig csökkenti a memória-sávszélesség iránti igényt. A gyakorlatban a döntés egy becslésen, majd mérésen alapuló iteratív folyamat.

2.4.2 5.2 Párhuzamosság és Amdahl-törvény

A GPU teljesítményének alapja a tömeges párhuzamosság. Egy modern GPU több ezer ALU-val rendelkezik, és a hardver ütemezője több tízezer szálat tud egyidejűleg nyilvántartani. Ahhoz, hogy ezt kihasználjuk, a feladatot legalább több ezer független végrehajtási egységre kell bontani.

Az Amdahl-törvény segít megbecsülni a párhuzamosítás elvi felső határát. Ha a feladat p hányada párhuzamosítható, a soros rész $1-p$, akkor a N processzoros gyorsulás:

$$S(N) = \frac{1}{(1-p) + \frac{p}{N}}$$

Még végtelen sok processzor esetén is $S_{\max} = 1/(1-p)$. Ha tehát a feladatnak csak 90%-a párhuzamosítható, a maximális gyorsulás $10\times$, bármilyen erős is a GPU. A gyakorlatban a GPU-ra vitt feladatok általában 99% feletti párhuzamosíthatóságot igényelnek ahhoz, hogy a sok ezer szál előnye érvényesüljön.

A párhuzamosságnak két fő típusa van:

- **Adatpárhuzamosság:** ugyanaz a művelet sok, független adatelemen fut. Ilyen a vektorösszeadás, a mátrixszorzás, a képfeldolgozás vagy a Monte Carlo-szimuláció. Ez a GPU természetes közege.
- **Feladatpárhuzamosság:** különböző, egymástól független feladatok futnak párhuzamosan. A GPU ezt is támogatja, de kevésbé hatékonyan, mert a szálak közötti eltérő vezérlési folyamat divergence-hez vezet. A feladatpárhuzamosság inkább stream-ekkel és konkurens kernelekkel valósítható meg.

Egy feladat GPU-alkalmasságának első gyors tesztje: „Hány független műveletet tudok azonosítani?” Ha a válasz legalább 10^4, érdemes tovább vizsgálni. Ha csak néhány száz, a GPU valószínűleg nem hoz gyorsulást, mert a kernelindítási és ütemezési overhead dominál.

2.4.3 5.3 Adatmozgatás és a teljes lánc költsége

A GPU és a CPU közötti adatmozgatás a leggyakrabban alábecsült költség. A PCIe 4.0 x16 elméleti sávszélessége 32 GB/s, a PCIe 5.0 x16 64 GB/s, míg a GPU saját globális memóriájának sávszélessége akár 1–3 TB/s is lehet. A gyakorlatban a PCIe átvitel a GPU memóriájának töredékét éri el, és a host oldali memória allokáció típusa is befolyásolja: a pinned (page-locked) memória gyorsabb DMA-átvitelt tesz lehetővé, mint a lapozható (pageable) memória.

A teljes lánc költsége a következő komponensekből áll:

$$T_{\text{total}} = T_{\text{host-in}} + T_{\text{H2D}} + T_{\text{kernel}} + T_{\text{D2H}} + T_{\text{host-out}} + T_{\text{launch}}$$

ahol:

- $T_{\text{host-in}}$ és $T_{\text{host-out}}$ a CPU oldali elő- és utófeldolgozás,
- T_{H2D} a host→device másolás,
- T_{kernel} a tényleges GPU számítás,
- T_{D2H} a device→host másolás,
- T_{launch} a kernelindítás és szinkronizáció overheadje.

Sok kezdő hiba abból adódik, hogy csak a T_{kernel}-t mérik, és ebből vonnak le következtetést. Egy tipikus eset: a kernel 1 ms alatt fut, de a H2D másolás 10 ms, a D2H másolás 8 ms, a kernelindítás pedig 0,05 ms. A teljes GPU-s végrehajtás így 19 ms, miközben a CPU 12 ms alatt megoldja a feladatot. A GPU „gyorsabb” állítás tehát hamis, ha a teljes láncot nézzük.

Az adatmozgatás költségét csökkenthetjük:

- **Pinned memória** használatával a host oldalon.
- **Adatok GPU-n tartásával** több kernel között, hogy ne kelljen minden lépésnél visszamásolni.
- **Átfedéssel**: streamek segítségével a másolás és a kernelvégrehajtás párhuzamosítható (erről a későbbi fejezetekben lesz szó).
- **Unified Memory** használatával, amely a migrációt a driverre bízza, bár ez nem mindig optimális.

A GPU-alkalmassági elemzés során mindig meg kell becsülni a teljes lánc idejét, nem csak a kernelét. Egy jó ökölszabály: ha a feladat adatbemenete és -kimenete együttesen kisebb, mint a GPU memória-sávszélességének 1–2 ms-nyi forgalma, akkor a transzfer overhead aránya valószínűleg elfogadható; ha nagyobb, a transzfer dominálhat.

2.4.4 5.4 Arithmetic intensity

Az arithmetic intensity (AI) azt mutatja meg, hogy egy bájtnyi memóriaforgalomra hány lebegőpontos művelet jut:

$$AI = \frac{\text{műveletek száma (FLOP)}}{\text{átvitt bájtok száma (byte)}}$$

Például a vektorösszeadás $c_i = a_i + b_i$ minden elemére 1 összeadás jut, és 3 float értéket olvasunk/írunk (a, b be, c ki). FP32 esetén 1 float = 4 byte, tehát a memóriaforgalom 12 byte/elem. Az AI:

$$AI_{\text{vectorAdd}} = \frac{1}{3 \cdot 4} = 0{,}083 \text{ FLOP/byte}$$

Ez rendkívül alacsony. A mátrixszorzásnál (GEMM) viszont minden elem $O(N)$ műveletben vesz részt, így az AI jellemzően 10–100 FLOP/byte nagyságrendű, ami számítás-kötöttséget jelent.

A GPU memória-sávszélességének és számítási teljesítményének hányadosa adja azt a küszöbértéket, amely alatt a feladat memória-kötött, felette pedig számítás-kötött. Például egy A100 GPU esetén a memória-sávszélesség ~1,5 TB/s, az FP32 számítási teljesítmény ~19,5 TFLOPS. A fordulópont:

$$AI_{\text{küszöb}} = \frac{19{,}5 \cdot 10^{12} \text{ FLOP/s}}{1{,}5 \cdot 10^{12} \text{ byte/s}} \approx 13 \text{ FLOP/byte}$$

Ha a feladat AI-ja ennél kisebb, a GPU-t a memória-sávszélesség korlátozza; ha nagyobb, a számítási egységek. A Roofline-modell ezt vizualizálja, de a részletes tárgyalás a későbbi fejezetekben következik. Itt elég annyi, hogy a GPU-alkalmasság vizsgálatakor az AI-t ki kell számítani, és össze kell vetni a hardver küszöbértékével.

Az alacsony AI nem jelenti automatikusan, hogy a GPU rossz választás. Ha a feladat nagy méretű, a memória-sávszélesség kihasználása önmagában is gyorsulást hozhat a CPU-hoz képest, amennyiben a CPU memória-sávszélessége alacsonyabb. Például egy CPU dual-channel DDR4 rendszer sávszélessége ~50 GB/s, míg a GPU-é 1,5 TB/s; így a vektorösszeadás is gyorsabb lehet GPU-n, ha az adat mérete elég nagy ahhoz, hogy a PCIe transzfert amortizálja.

2.4.5 5.5 Gyakorlati döntési minta

A fenti szempontok alapján egy egyszerű döntési folyamatot állíthatunk össze:

1. **Határozd meg a feladat párhuzamosságát.** Hány független művelet van? Ha kevesebb, mint 10^4, a GPU valószínűleg nem éri meg.
2. **Számold ki az arithmetic intensity-t.** Mennyi FLOP/byte? Hasonlítsd össze a célhardver AI-küszöbével, hogy lásd, memória- vagy számítás-kötött lesz-e a feladat.
3. **Becsüld meg az adatmozgatást.** Mekkora a bemenet és kimenet? Mennyi idő a H2D és D2H másolás a rendelkezésre álló sávszélességen?
4. **Mérd meg a CPU baseline-t.** Mennyi idő alatt oldja meg a feladatot egy optimalizált, párhuzamosított CPU implementáció?
5. **Becsüld meg a teljes GPU-s időt.** $T_{\text{total}} = T_{\text{H2D}} + T_{\text{kernel}} + T_{\text{D2H}} + T_{\text{launch}}$. Ha a becslés nem mutat legalább 2–3× gyorsulást a CPU-hoz képest, a GPU bevezetése kockázatos.
6. **Validálj méréssel.** Írj egy minimális CUDA/OpenCL/JAX prototípust, és mérd a teljes láncot, nem csak a kernelt. Csak akkor kötelezd el magad, ha a mért gyorsulás igazolja a várakozást.

Ez a folyamat iteratív: előfordulhat, hogy az első becslés alapján a GPU nem tűnik előnyösnek, de a kernel optimalizálása (coalescing, tiling, streamek) után megfordul a helyzet. A döntési minta lényege, hogy a becslést és a mérést szétválasszuk: a becslés a tervezéshez, a mérés a validáláshoz kell.

2.4.6 5.6 Labor: vektorösszeadás GPU-alkalmassági elemzése

A labor célja, hogy a fenti kritériumokat egy konkrét feladaton alkalmazzuk. A feladat a vektorösszeadás: $c_i = a_i + b_i$, FP32 pontossággal. Ez a lehető legegyszerűbb CUDA kernel, de kiválóan alkalmas az AI, a transzfer és a párhuzamosság vizsgálatára.

Előkészítés. A labor környezete legyen izolált, például egy konténerben futtatott CUDA fejlesztői környezet. A mérés előtt ellenőrizzük a GPU elérhetőségét (`nvidia-smi`), a driver és a CUDA verzióját. A mérési környezetet rögzíteni kell.

CUDA C++ forrás. A következő kód egy teljes, fordítható program, amely vektorösszeadást végez GPU-n, és kiírja a kernelidőt, a H2D és D2H másolási időt, valamint a teljes GPU-s időt. A kód tartalmazza a kötelező hibakezelést és a mérési ismétléseket.

```
// CUDA C++ (C++17), nvcc 12.4, Linux x86_64
#include <cstdio>
#include <cuda_runtime.h>
#include <cmath>
#include <chrono>

#define CHECK_CUDA(call) do {                                          \
    cudaError_t err = call;                                            \
    if (err != cudaSuccess) {                                          \
        std::fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__, \
                     cudaGetErrorString(err));                         \
        std::abort();                                                  \
    }                                                                  \
} while (0)

__global__ void vectorAdd(const float* __restrict__ a,
                          const float* __restrict__ b,
                          float* __restrict__ c,
                          int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        c[idx] = a[idx] + b[idx];
    }
}

int main() {
    constexpr int N = 1 << 20;          // 1 048 576 elem
    constexpr size_t bytes = N * sizeof(float);
    constexpr int blockSize = 256;
    constexpr int gridSize = (N + blockSize - 1) / blockSize;  // 4096 blokk

    // Host pinned memória
    float *h_a, *h_b, *h_c;
    CHECK_CUDA(cudaMallocHost(&h_a, bytes));
    CHECK_CUDA(cudaMallocHost(&h_b, bytes));
    CHECK_CUDA(cudaMallocHost(&h_c, bytes));

    for (int i = 0; i < N; ++i) {
        h_a[i] = 1.0f;
        h_b[i] = 2.0f;
        h_c[i] = 0.0f;
    }

    // Device memória
    float *d_a, *d_b, *d_c;
    CHECK_CUDA(cudaMalloc(&d_a, bytes));
    CHECK_CUDA(cudaMalloc(&d_b, bytes));
    CHECK_CUDA(cudaMalloc(&d_c, bytes));
```

```
// Méréshez események
cudaEvent_t start, stop;
CHECK_CUDA(cudaEventCreate(&start));
CHECK_CUDA(cudaEventCreate(&stop));

// Warm-up (3 ismétlés)
for (int i = 0; i < 3; ++i) {
    CHECK_CUDA(cudaMemcpy(d_a, h_a, bytes, cudaMemcpyHostToDevice));
    CHECK_CUDA(cudaMemcpy(d_b, h_b, bytes, cudaMemcpyHostToDevice));
    vectorAdd<<<gridSize, blockSize>>>(d_a, d_b, d_c, N);
    CHECK_CUDA(cudaMemcpy(h_c, d_c, bytes, cudaMemcpyDeviceToHost));
    CHECK_CUDA(cudaDeviceSynchronize());
}

// Mérés: 10 ismétlés
constexpr int reps = 10;
float h2d_ms[reps], d2h_ms[reps], kernel_ms[reps];
for (int i = 0; i < reps; ++i) {
    // H2D mérése
    CHECK_CUDA(cudaEventRecord(start));
    CHECK_CUDA(cudaMemcpy(d_a, h_a, bytes, cudaMemcpyHostToDevice));
    CHECK_CUDA(cudaMemcpy(d_b, h_b, bytes, cudaMemcpyHostToDevice));
    CHECK_CUDA(cudaEventRecord(stop));
    CHECK_CUDA(cudaEventSynchronize(stop));
    CHECK_CUDA(cudaEventElapsedTime(&h2d_ms[i], start, stop));

    // Kernel mérése
    CHECK_CUDA(cudaEventRecord(start));
    vectorAdd<<<gridSize, blockSize>>>(d_a, d_b, d_c, N);
    CHECK_CUDA(cudaEventRecord(stop));
    CHECK_CUDA(cudaEventSynchronize(stop));
    CHECK_CUDA(cudaEventElapsedTime(&kernel_ms[i], start, stop));

    // D2H mérése
    CHECK_CUDA(cudaEventRecord(start));
    CHECK_CUDA(cudaMemcpy(h_c, d_c, bytes, cudaMemcpyDeviceToHost));
    CHECK_CUDA(cudaEventRecord(stop));
    CHECK_CUDA(cudaEventSynchronize(stop));
    CHECK_CUDA(cudaEventElapsedTime(&d2h_ms[i], start, stop));

    CHECK_CUDA(cudaDeviceSynchronize());
}

// Átlag és szórás kiszámítása
auto mean = [](const float* arr, int n) {
    double s = 0.0;
    for (int i = 0; i < n; ++i) s += arr[i];
    return s / n;
};
auto stdev = [&](const float* arr, int n, double m) {
    double s = 0.0;
    for (int i = 0; i < n; ++i) s += (arr[i] - m) * (arr[i] - m);
    return std::sqrt(s / (n - 1));
};

double m_h2d = mean(h2d_ms, reps);
double s_h2d = stdev(h2d_ms, reps, m_h2d);
double m_kernel = mean(kernel_ms, reps);
double s_kernel = stdev(kernel_ms, reps, m_kernel);
double m_d2h = mean(d2h_ms, reps);
double s_d2h = stdev(d2h_ms, reps, m_d2h);
```

```
    std::printf("H2D: %.3f ms (sd=%.3f)\n", m_h2d, s_h2d);
    std::printf("Kernel: %.3f ms (sd=%.3f)\n", m_kernel, s_kernel);
    std::printf("D2H: %.3f ms (sd=%.3f)\n", m_d2h, s_d2h);
    std::printf("Teljes GPU-s idő: %.3f ms\n", m_h2d + m_kernel + m_d2h);

    // Ellenőrzés
    float max_err = 0.0f;
    for (int i = 0; i < N; ++i) {
        max_err = std::max(max_err, std::abs(h_c[i] - 3.0f));
    }
    std::printf("Max hiba: %e\n", max_err);

    CHECK_CUDA(cudaFreeHost(h_a));
    CHECK_CUDA(cudaFreeHost(h_b));
    CHECK_CUDA(cudaFreeHost(h_c));
    CHECK_CUDA(cudaFree(d_a));
    CHECK_CUDA(cudaFree(d_b));
    CHECK_CUDA(cudaFree(d_c));
    CHECK_CUDA(cudaEventDestroy(start));
    CHECK_CUDA(cudaEventDestroy(stop));
    return 0;
}
```

Buildparancs. A fordítás a GPU compute capability-jétől függő `-arch` kapcsolóval történik. Például egy Ampere (sm_80) GPU esetén:

```
nvcc -O3 -arch=sm_80 -o vector_add vector_add.cu
```

Futtatási parancs. A program futtatása előtt győződjünk meg arról, hogy a GPU szabad, és a timeout be van állítva (például `timeout 60s ./vector_add`). A futtatás:

```
./vector_add
```

GPU-konfiguráció (példa, nem ellenőrzött mérés). A laborban rögzíteni kell a pontos környezetet. Példaként:

- GPU: NVIDIA A100 40 GB PCIe (compute capability 8.0)
- Driver: 550.54.14
- CUDA: 12.4
- OS: Ubuntu 22.04 LTS, konténer (Docker, CUDA 12.4 base)
- Power limit: alapértelmezett
- ECC: bekapcsolva
- MIG: kikapcsolva
- Háttérterhelés: nincs

Várt elemzés. A vektorösszeadás párhuzamossága $N = 2^{20} \approx 1{,}05 \cdot 10^6$ elem, ami bőven elegendő a GPU kihasználásához. A kernel minden eleme független, szinkronizáció nincs. Az AI:

$$AI = \frac{1 \text{ FLOP}}{3 \cdot 4 \text{ byte}} = 0{,}083 \text{ FLOP/byte}$$

Ez messze az A100 AI-küszöbe (~13 FLOP/byte) alatt van, tehát a feladat erősen memória-kötött. A GPU globális memória-sávszélessége ~1,5 TB/s, a kernel elméleti ideje:

$$T_{\text{kernel, elméleti}} = \frac{3 \cdot N \cdot 4 \text{ byte}}{1{,}5 \cdot 10^{12} \text{ byte/s}} \approx \frac{12{,}58 \cdot 10^6}{1{,}5 \cdot 10^{12}} \approx 8{,}4 \cdot 10^{-6} \text{ s} = 0{,}0084 \text{ ms}$$

A gyakorlatban a kernel nem éri el a csúcssávszélességet, de 0,01–0,02 ms nagyságrendű idő várható. A H2D és D2H másolás egyenként ~2×N×4 byte = 8,4 MB. A PCIe 4.0 x16 elméleti 32 GB/s sávszélességén a másolási idő:

$$T_{\text{H2D}} = T_{\text{D2H}} \approx \frac{8{,}4 \cdot 10^6}{32 \cdot 10^9} \approx 0{,}26 \text{ ms}$$

A teljes GPU-s idő becslése így:

$$T_{\text{total}} \approx 0{,}26 + 0{,}015 + 0{,}26 = 0{,}535 \text{ ms}$$

Ehhez jön még a kernelindítás overhead (~0,005–0,02 ms). A CPU baseline ekkora méreten tipikusan 1–3 ms (egy optimalizált, párhuzamosított C++ implementációval). A GPU tehát várhatóan gyorsabb, de a gyorsulás szerény, mert a transzfer dominálja a teljes időt. Ha a méretet növeljük, a transzfer aránya csökken a kernelhez képest? Nem, mert a kernel is lineárisan nő, de a PCIe sávszélesség kisebb, mint a GPU memória-sávszélesség, így a transzfer egyre nagyobb arányt képvisel. Ez azt mutatja, hogy a vektorösszeadás tipikusan nem a GPU ideális feladata, hacsak nem tudjuk a transzfert átfedni a számítással, vagy az adatot több kernel között a GPU-n tartani.

Mért eredmények (a laborban a hallgató tölti ki). A fenti program futtatása után a mérési naplóba kerülnek a tényleges időadatok, az átlag, a szórás, és a max hiba. A mért eredményt külön kell választani a várt becsléstől. A hallgatónak meg kell állapítania, hogy a mért gyorsulás igazolja-e a GPU használatát, és ha nem, milyen optimalizálással lehetne javítani (például streamekkel átfedni a másolást, vagy nagyobb N-et használni).

2.4.7 5.7 Tipikus hibák és helyes megoldások

A GPU-alkalmassági elemzés során a következő hibák fordulnak elő leggyakrabban:

1. **Túl kicsi input méret.** A kernelindítás és az API-hívások overheadje (néhány mikroszekundum) nagyobb lehet, mint a tényleges számítás. Ha N kicsi, a GPU nem tudja kihasználni a párhuzamosságot, és a teljes lánc idő a CPU-nál rosszabb lesz.
   Helyes megoldás: a mérés előtt becsüld meg a minimális méretet, amelynél a kernelidő meghaladja az overheadet, és csak akkor használj GPU-t, ha az input e felett van.

2. **Az adatmozgatás elhanyagolása.** A kernelidő önmagában nem elég; a H2D és D2H másolás gyakran dominál.
   Helyes megoldás: mindig a teljes láncot mérd, és keresd a transzfer csökkentésének lehetőségeit (pinned memória, adatok GPU-n tartása, streamek).

3. **Nem koaleszkált hozzáférés.** A szétszórt memória-hozzáférés a GPU globális memória-sávszélességét töredékére csökkenti, így a kernel a vártnál sokkal lassabb lesz.
   Helyes megoldás: a kernel megtervezésekor biztosítsd, hogy a szomszédos szálak szomszédos címeket érjenek el. A vektorösszeadásban ez automatikusan teljesül, de más algoritmusoknál gondolni kell rá.

4. **Az arithmetic intensity figyelmen kívül hagyása.** Olyan feladatot visznek GPU-ra, amelynek AI-ja rendkívül alacsony, és a GPU memória-sávszélessége sem elég nagy a CPU-hoz képest.
   Helyes megoldás: számold ki az AI-t, és hasonlítsd össze a hardver küszöbértékével. Ha a feladat AI-ja alacsony, vizsgáld meg, hogy a GPU memória-sávszélessége mekkora előnyt ad, és ha nem, maradj a CPU-nál.

5. **A „gyorsabb” állítás mérés nélkül.** Sokszor hallani, hogy „a GPU gyorsabb”, de nincs baseline, nincs ismétlés, nincs statisztika.
   Helyes megoldás: minden állítást méréssel támassz alá, legalább 5–10 ismétléssel, átlaggal és szórással. A mérési környezetet rögzíteni kell.

6. **Nem megfelelő precision.** A GPU FP64 teljesítménye gyakran alacsony, ezért ha a feladat dupla pontosságot igényel, a CPU vagy egy FP64-re optimalizált GPU jobb választás lehet.
   Helyes megoldás: a numerikus kísérlet előtt rögzítsd a szükséges pontosságot, és ellenőrizd a célhardver megfelelő teljesítményét.

2.4.8 5.8 Önellenőrző kérdés

Egy képfeldolgozó függvény 1 millió pixelre 10 FLOP/pixel műveletet végez. A CPU 50 ms alatt oldja meg a feladatot. A GPU PCIe 4.0 x16 kapcsolaton 12 GB/s tényleges sávszélességgel mozgatja az adatokat. A GPU kernel becsült ideje 1 ms. A bemeneti és kimeneti adat együttes mérete 8 MB. Érdemes-e GPU-ra vinni a feladatot? Válaszát indokolja!

Megoldás: Számítsuk ki a teljes GPU-s időt. H2D + D2H másolás: 8 MB / 12 GB/s = 0,667 ms. Kernel: 1 ms. Teljes: ~1,7 ms. A CPU 50 ms, tehát a GPU ~29× gyorsabb. A válasz: igen, érdemes, mert a teljes lánc is lényegesen gyorsabb. (Megjegyzés: az AI = 10 FLOP / 3 byte 3,3 FLOP/byte, ami alacsony, de a GPU memória-sávszélessége így is elegendő, mert a kernelidő a becslés szerint 1 ms. A gyakorlatban ezt méréssel kell igazolni.)

Ez a fejezet felépítette a GPU-alkalmasság kritériumrendszerét, és egy konkrét laboron keresztül mutatta be, hogyan kell egy feladatot elemzésnek alávetni. A következő fejezetben a GPU végrehajtási modelljének alapjait, a SIMT és warp fogalmát ismerjük meg, amelyre a kerneltervezés további lépései épülnek.

3 GPU-architektúra: SIMT, warp, SM és memória

3.1 SIMT és warp-végrehajtás

A GPGPU-programozás középpontjában az a felismerés áll, hogy a grafikus processzorok több ezer könnyűsúlyú végrehajtási egysége nem egymástól független, különálló magokként viselkedik, hanem szorosan összekapcsolt csoportokban hajtja végre az utasításokat. Ez a szerveződés határozza meg mindazt, amit később a memóriaelérés optimalizálásáról, az occupancy-ről vagy a kerneltervezésről tanulni fogunk. Ebben az alfejezetben a CUDA-végrehajtási modell legalapvetőbb rétegét, a SIMT (Single-Instruction, Multiple-Thread) elvet és annak hardveres megvalósulását, a warpot vizsgáljuk meg.



3.1. ábra: Egy warp végrehajtása divergens ág esetén - A 32 szál közös utasítást kap; elágazásnál aktív maszkok keletkeznek, és a két út sorban fut, majd a warp rekonvergál.

3.1.1 6.1. A SIMT-modell: párhuzamosság egy utasításfolyamban

A hagyományos CPU-magok általában egy utasításfolyamot hajtanak végre, és a párhuzamosságot több, viszonylag független mag, illetve a magon belüli vektoros utasítások (SIMD – Single-Instruction, Multiple-Data) biztosítják. A GPU ettől eltérő gondolati modellt kínál: a programozó úgy ír kódot, mintha minden egyes adatelemhez külön szál tartozna, mindegyik szál saját utasításmutatóval és regiszterkészlettel rendelkezne, és egymástól függetlenül futnának. Ezt a látszólagos függetlenséget nevezzük SIMT-modellnek.

A valóságban azonban a hardver nem minden szálhoz rendel hozzá egy teljesen független vezérlőegységet. Ehelyett a szálakat rögzített méretű csoportokba, úgynevezett warpokba szervezi, és egy csoport minden tagja ugyanazt az utasítást hajtja végre ugyanabban az órajelciklusban. Ez a szervezés teszi lehetővé, hogy a hardver a vezérlési logikát (utasításlehívás, dekódolás, ütemezés) a csoport szintjén ossza meg, miközben az aritmetikai és memória-műveleteket a csoport tagjain párhuzamosan, adatpárhuzamos módon végzi el.

Fontos megkülönböztetnünk a SIMT-t a klasszikus SIMD-től. A CPU-s SIMD-utasítások esetén a programozó explicit módon vektorregiszterekkel dolgozik, például AVX-512 esetén `__m512` típusokkal, és a vektor szélessége a forráskódban is látható. Ezzel szemben a CUDA kernelben a programozó skaláris kódot ír, mintha minden szálnak saját skaláris regiszterei lennének:

```
// CUDA C++ (C++17, NVCC 12.4, A100, CC 8.0, Linux x86_64)
__global__ void add_kernel(const float* a, const float* b, float* c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        c[i] = a[i] + b[i];
    }
}
```

A kernelben sehol nem jelenik meg vektortípus, mégis a hardver egyszerre 32 szálra (egy warp) hajtja végre ugyanazt az összeadást, amennyiben azok a szálak ugyanabba a vezérlési ágba tartoznak. A SIMT tehát egy programozói absztrakció, amely a szálak közötti függetlenség illúzióját adja, miközben a hardver a szálak csoportjait szinkron módon, lockstep üzemmódban futtatja.

A SIMT-modell előnye a programozó számára, hogy a párhuzamos kódot egyszerűbben lehet kifejezni, mint explicit vektorizálással. Nem kell kézzel szétbontani a ciklusokat a vektorregiszter szélességének megfelelő darabokra, és nem kell külön kezelni a vektorregiszterek feltöltését és kipakolását. Ugyanakkor a modellnek vannak következményei, amelyeket a divergencia tárgyalásakor részletezünk: ha a szálak nem ugyanazt az utasítást hajtják végre, a hardver kénytelen sorosítani az eltérő ágakat.

3.1.2 6.2. A warp mint hardveres végrehajtási egység

A CUDA hardveres végrehajtási alapegysége a warp. Minden GPU-generációban a warp mérete 32 szál, és ez az érték a NVIDIA dokumentációjában rögzített konstans. A szálak csoportosítása warpokba determinisztikus: a szálakat a `threadIdx.x` értéke szerint növekvő sorrendben osztják 32-es csoportokba. Ha például egy blokk 256 szálat tartalmaz, az első warp a 0–31 sorszámú szálakat, a második a 32–63 sorszámú szálakat foglalja magában, és így tovább.

A warp minden tagját lane-nek nevezzük. A lane sorszámát a `threadIdx.x % 32` kifejezés adja meg, és a hardver ezt a sorszámot használja arra, hogy a warp szintű műveletekben, például a shuffle-műveletekben meghatározza az adatcsere irányát. A lane sorszám a kernel futása során nem változik, és a `threadIdx.x / 32` egész osztás a warp sorszámát adja a blokkon belül.

A hardver szempontjából a warp egyetlen utasításfolyam, amely 32 adatelemre vonatkozik. A streaming multiprocesszor (SM) utasításütemezője minden órajelciklusban egy vagy több warp számára választ ki egy utasítást, és az SM végrehajtó egységei a warp 32 lane-jére párhuzamosan alkalmazzák azt. Ha az SM rendelkezik például 32 FP32 végrehajtó egységgel, akkor egy FP32 összeadás egyetlen órajelciklus alatt végrehajtható a teljes warp számára. Ha az SM csak 16 FP32 egységet tartalmaz, akkor a hardver két ütemben hajtja végre ugyanazt az utasítást, először az alsó 16, majd a felső 16 lane számára.

A warp fogalma tehát két szinten értelmezendő. Egyrészt programozói szempontból a warp a szálak olyan csoportja, amely egyszerre, ugyanazt az utasítást hajtja végre. Másrészt hardveres szempontból a warp az az utasításütemezési egység, amelyet az SM scheduler kezel. A két értelmezés közötti kapcsolat a lockstep végrehajtás: amíg a warp szálai ugyanazt az utasítást hajtják végre, a hardver teljes párhuzamossággal dolgozik.

Érdemes megjegyezni, hogy a CUDA programozási modell nem ígéri a warpok egymáshoz viszonyított sorrendjét. Két különböző warp futása tetszőleges sorrendben átlapolódhat, és a hardver az erőforrások kihasználása érdekében aktívan váltogatja a warpok végrehajtását. Amikor egy warp memória-műveletre vár (például globális memóriából olvas), az SM ütemezője egy másik, már futásra kész warp számára ad ki utasítást. Ez a warp-szintű átlapolás az egyik legfontosabb mechanizmus a memória-késleltetés elrejtésére, amelyet az Occupancy és erőforrás-korlátok alfejezetben részletesen vizsgálunk.

3.1.3 6.3. Branch divergence: amikor a szálak útjai szétválnak

A SIMT-modell legfontosabb teljesítménybeli következménye az elágazások kezelésénél jelentkezik. Amikor a kernelben feltételes utasítás szerepel, például:

```
// CUDA C++ (C++17, NVCC 12.4, A100, CC 8.0, Linux x86_64)
__global__ void conditional_kernel(const float* input, float* output, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        if (input[i] > 0.0f) {
            output[i] = sqrtf(input[i]);
        } else {
            output[i] = -sqrtf(-input[i]);
        }
    }
}
```

a hardver szempontjából a kérdés az, hogy a warp 32 szálának feltételkiértékelése ugyanarra az eredményre vezet-e. Ha a feltétel a warp minden lane-jére ugyanazt az értéket adja, a végrehajtás egyszerű: a hardver a teljes warpot az egyik ágba irányítja, és minden lane ugyanazt az utasítást hajtja végre.

Ha azonban a feltétel a warp egyes lane-jeire igaz, másokra hamis, akkor a warpon belül **branch divergence** lép fel. A hardver ilyenkor nem tudja a két ágat párhuzamosan végrehajtani, mert a SIMT-modell szerint a warp egyszerre csak egy utasítást hajthat végre. A megoldás az, hogy a hardver az eltérő utasítási útvonalakat sorosítja: először az egyik ágba tartozó lane-ek aktívak, a többi lane inaktív, majd a másik ág következik a korábban inaktív lane-ekkel.

Az aktív lane-ek halmazát **aktív maszknak** (active mask) nevezzük. Ez egy 32 bites bitsorozat, amelyben az 1-es bit aktív lane-t jelöl. Amikor a hardver az egyik ágat hajtja végre, az aktív maszk csak azokat a lane-eket tartalmazza, amelyek ebbe az ágba tartoznak. A nem aktív lane-ek végrehajtó egységei ilyenkor üresjáratban vannak, de nem hajtanak végre utasítást, és a regisztereik értéke nem változik. Az inaktív lane-ek tehát nem okoznak hibás eredményt, csupán a hardver párhuzamos kapacitásának egy részét nem használják ki.

A divergens végrehajtásnak költsége van. Ha a warp fele az egyik ágba, fele a másikba tartozik, a hardvernek mindkét ágat végre kell hajtania, tehát a divergens szakasz végrehajtási ideje körülbelül a két ág végrehajtási idejének összege. Ez a teljesítménycsökkenés nem jelent hibás eredményt, de a kernel futásidejét jelentősen megnövelheti. A legrosszabb esetben a warp 32 lane-je 32 különböző útvonalon halad, így a hardver 32-szeres sorosítást szenved el.

A gyakorlatban a divergence akkor okoz jelentős problémát, ha a feltétel a warp szálai között véletlenszerűen oszlik meg. Ha a feltétel a warp szintjén koherens, vagyis a warp minden lane-je ugyanazt az ágat választja, a divergence elkerülhető. A tipikus eset a `threadIdx.x`-től függő feltétel, például amikor a feltétel a lane páros vagy páratlan voltától függ. Ilyenkor minden warpban pontosan 16 páros és 16 páratlan lane van, tehát minden warp divergens lesz. Ezzel szemben a `blockIdx.x`-től függő feltétel általában warp-koherens, mivel a blockIdx értéke a blokk minden warpjára ugyanaz.

3.1.4 6.4. Rekonvergencia: a warp visszatérése a közös útra

A divergens végrehajtás nem tart örökké. Amikor a hardver minden eltérő ágat végrehajtott, a warp lane-jei újra találkoznak a közös végrehajtási ponton, és innen kezdve ismét lockstep üzemmódban, azaz egyetlen utasításfolyamként folytatják a futást. Ezt a folyamatot **rekonvergenciának** nevezzük.

A rekonvergencia pontja tipikusan az elágazást közvetlenül megelőző és az azt követő utasítások közötti programpont, ahol a feltételes ágak újra egyesülnek. A CUDA hardver a rekonvergenciát automatikusan kezeli: az SM a warp állapotát nyomon követi, és amikor a divergens szakasz véget ér, az aktív maszkot újra a teljes warpra állítja. A programozónak nincs szüksége explicit utasításra a rekonvergenciához, de tudnia kell, hogy a divergens szakasz után a warp ismét teljes párhuzamossággal fut.

Fontos azonban megérteni a rekonvergencia hatókörét. A hardver a rekonvergenciát az utasításfolyam szintjén kezeli, nem a programszemantika szintjén. Ha a kernelben egy `if` elágazáson belül egy hosszú ciklus szerepel, és a ciklus feltétele lane-enként eltérő iterációszámhoz vezet, a divergens szakasz jelentősen elnyúlhat. A rekonvergencia ilyenkor is megtörténik, de csak akkor, amikor az összes lane befejezte a saját ágát. A hardver nem tudja "megszakítani" a lassabb lane-eket, tehát a warp végrehajtási ideje a leglassabb lane által meghatározott lesz.

A rekonvergencia mechanizmusa a mai GPU-architektúrákban stack-alapú kezeléssel valósul meg. Amikor a warp divergens elágazáshoz ér, a hardver elmenti a rekonvergenciaponthoz tartozó aktív maszkot és a program counter értéket, majd az egyik ágat kezdi végrehajtani a megfelelő aktív maszkkal. Amikor ez az ág befejeződik, a hardver visszatér a mentett ponthoz, és a másik ágat hajtja végre a kiegészítő aktív maszkkal. Ez a folyamat tetszőleges mélységig ismétlődhet, ha a divergens ágak maguk is divergens elágazásokat tartalmaznak. A legrosszabb esetben a divergencia exponenciálisan növelheti a végrehajtandó útvonalak számát, de a gyakorlatban a mélyen beágyazott divergens elágazások ritkák, és a teljesítményprofilozók általában jól jelzik őket.

3.1.5 6.5. Divergencia és teljesítmény: gyakorlati következmények

A branch divergence teljesítményre gyakorolt hatása közvetlenül mérhető. Tekintsünk egy olyan kernelt, amely a bemeneti tömb elemei alapján választ két különböző számítási ág között, és az ágak számításigénye eltérő:

```
// CUDA C++ (C++17, NVCC 12.4, A100, CC 8.0, Linux x86_64)
__global__ void branchy_kernel(const float* input, float* output, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        if (input[i] > 0.5f) {
            // Nehezebb ág: 10 szorzás
            float x = input[i];
            for (int j = 0; j < 10; ++j) {
                x = x * 1.0001f + 0.0001f;
            }
            output[i] = x;
        } else {
            // Könnyű ág: 1 szorzás
            output[i] = input[i] * 2.0f;
        }
    }
}
```

Ha a bemeneti adatokat úgy generáljuk, hogy a 0,5-ös küszöb a warp szálai között véletlenszerűen teljesül, a hardvernek minden warpban mindkét ágat végre kell hajtania, ezért a kernel futásideje a nehezebb ág által meghatározott lesz. Ha ezzel szemben a bemeneti adatokat úgy állítjuk elő, hogy a küszöb a warp szintjén koherens (például a blokk minden páros indexű eleme 0,1, minden páratlan indexű eleme 0,9, de a blokkmérete 256, és a feltétel a blokk indexétől függ), a divergence elkerülhető. A mérhető teljesítménykülönbség a két eset között jelentős lehet, és ezt a laborgyakorlatban részletesen bemutatjuk.

A programozó szempontjából a legfontosabb irányelv a **warp-kohézió** megőrzése. A feltételeket lehetőség szerint úgy kell megfogalmazni, hogy azok a warp szintjén egységesek legyenek. Ha például a kernel egy nagy adathalmazból csak bizonyos elemeket dolgoz fel, és a szűrési feltétel a warp lane-jei között véletlenszerűen teljesül, a divergens végrehajtás elkerülhető azzal, hogy a warpot először koherens blokkokra bontjuk, vagy a szűrést egy külön lépésben, kompaktáló művelettel végezzük el, amely után a feldolgozó kernel már tömör, koherens adatokon dolgozik.

A divergence hatása azonban nem mindig egyforma. Ha a divergens ágak mindkét oldalán rövid, hasonló számításigényű utasítások szerepelnek, a sorosítás költsége kicsi. Ha a feltétel ritkán teljesül, és a divergens ág csak ritkán aktív, a hardver a ritka ág végrehajtása előtt megvizsgálhatja, hogy van-e egyáltalán aktív lane, és ha nincs, az ágat átugorja. Ez a **skip optimization** a modern GPU-architektúrákban szabványos, és azt jelenti, hogy a teljesen inaktív ágak nem kerülnek végrehajtásra. A korai kilépés (early exit) optimalizálás azt is lehetővé teszi, hogy a hardver bizonyos feltételek mellett az elágazást a feltétel kiértékelése nélkül kezelje, ha az aktív maszk eleve üres.

3.1.6 6.6. Warp-szintű primitívek és a divergence elkerülése

A CUDA programozási modell számos olyan eszközt kínál, amelyekkel a divergencia elkerülhető vagy hatása csökkenthető. Az egyik legfontosabb eszköz a **warp shuffle** művelet, amely lehetővé teszi, hogy a warp lane-jei közvetlenül adatot cseréljenek egymással anélkül, hogy a shared memóriát kellene használni. A shuffle-műveletek használata nem igényel explicit szinkronizációt, és a hardver közvetlenül a regiszterek között mozgatja az adatokat. A leggyakrabban használt shuffle-utasítás a `__shfl_sync`:

```
// CUDA C++ (C++17, NVCC 12.4, A100, CC 8.0, Linux x86_64)
__global__ void shuffle_example(const float* input, float* output, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int lane = threadIdx.x % 32;
    if (i < n) {
        float val = input[i];
        // A warp jobbra lépteti az adatokat: minden lane a lane-1 értékét kapja
        float shifted = __shfl_down_sync(0xffffffff, val, 1);
        output[i] = shifted;
    }
}
```

A `0xffffffff` maszk a warp minden lane-jét aktívként jelöli, és a `__shfl_down_sync` a megadott delta értékkel lefelé lépteti az adatokat a warpon belül. A shuffle-műveletek a divergence szempontjából is relevánsak, mert a maszk paraméter pontosan megadja, hogy mely lane-ek vesznek részt a műveletben. Ha a maszk nem a teljes warp, akkor a művelet csak a megadott aktív lane-ekre vonatkozik, és a többi lane értéke változatlan marad.

A shuffle-műveletek mellett a **warp vote** és **warp match** műveletek is hasznosak. A `__all_sync`, `__any_sync` és `__ballot_sync` függvények a warp aktív lane-jeinek feltételét értékelik ki, és a `__match_any_sync` az azonos értékű lane-eket azonosítja. Ezekkel a primitívekkel a kernel olyan műveleteket végezhet el, mint a warp-szintű redukció vagy a warp-szintű tömörítés, anélkül, hogy divergens útvonalakat kellene bejárnia.

A modern CUDA (a compute capability 7.0-tól kezdődően) a `__syncwarp` utasítást is kínálja, amely a warp lane-jei közötti szinkronizációt biztosít. Ez a blokk-szintű `__syncthreads` warp-szintű megfelelője, és a warp-szintű kódban akkor használható, ha a lane-ek közötti sorrend fontos, de a teljes blokk szinkronizálása felesleges lenne.

3.1.7 6.7. Labor: warp-szintű végrehajtás szimulációja és mérése

A következő laborgyakorlat célja, hogy a SIMT-végrehajtás és a branch divergence elméleti ismereteit egy mérhető, reprodukálható kísérletben ellenőrizzük. A gyakorlat három kernelt hasonlít össze: egy divergenciamentes változatot, egy warp-kohéziót megtartó változatot és egy szándékosan divergens változatot. A mérési környezet adatait pontosan rögzítjük.

GPU-konfiguráció (helyőrző, izolált labor): - GPU: NVIDIA A100 (vagy a laborkörnyezetben elérhető, itt rögzítendő típus) - Compute capability: 8.0 - Driver: 550.54.15 (verzió ellenőrizendő a laborkörnyezetben) - CUDA: 12.4 - Operációs rendszer: Linux x86_64 (Ubuntu 22.04) - NVCC: 12.4, C++17 szabvány - Power limit: 300 W (a mérési naplóban rögzítendő) - ECC: ki - MIG: kikapcsolva - Háttérterhelés: nincs (izolált konténer)

Kernel a divergencia mérésére:

```
// divergence_lab.cu - CUDA C++ (C++17, NVCC 12.4, A100, CC 8.0, Linux x86_64)
#include <cuda_runtime.h>
#include <cstdio>
#include <cstdlib>
#include <chrono>

// Hibamakró az összes CUDA API-hívás ellenőrzésére
#define CUDA_CHECK(call)                                                        \
    do {                                                                        \
        cudaError_t err = (call);                                               \
        if (err != cudaSuccess) {                                               \
            fprintf(stderr, "CUDA hiba: %s, fajl: %s, sor: %d\n",              \
                    cudaGetErrorString(err), __FILE__, __LINE__);               \
            exit(EXIT_FAILURE);                                                 \
        }                                                                       \
    } while (0)

// 1) Divergenciamentes kernel: a feltétel a warp szintjén egységes
__global__ void coherent_kernel(const float* input, float* output, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        // A feltétel a blockIdx.x-től függ, ami minden warpban azonos
        bool heavy = (blockIdx.x % 2 == 0);
        float x = input[i];
        if (heavy) {
            for (int j = 0; j < 10; ++j) x = x * 1.0001f + 0.0001f;
            output[i] = x;
        } else {
            output[i] = x * 2.0f;
        }
    }
}

// 2) Warp-kohéziós kernel: a feltétel a threadIdx.x / 32-től függ
__global__ void warp_coherent_kernel(const float* input, float* output, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        bool heavy = ((threadIdx.x / 32) % 2 == 0);
        float x = input[i];
        if (heavy) {
            for (int j = 0; j < 10; ++j) x = x * 1.0001f + 0.0001f;
            output[i] = x;
        } else {
            output[i] = x * 2.0f;
        }
    }
}
```

```
// 3) Szándékosan divergens kernel: a feltétel a lane párosságától függ
__global__ void divergent_kernel(const float* input, float* output, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        bool heavy = ((threadIdx.x % 32) % 2 == 0);
        float x = input[i];
        if (heavy) {
            for (int j = 0; j < 10; ++j) x = x * 1.0001f + 0.0001f;
            output[i] = x;
        } else {
            output[i] = x * 2.0f;
        }
    }
}

// Mérési segédfüggvény: warm-up, 10 ismétlés, átlag és szórás
float measure_kernel(void (*kernel)(const float*, float*, int),
                     const float* d_input, float* d_output, int n,
                     dim3 grid, dim3 block) {
    // Warm-up
    for (int i = 0; i < 5; ++i) {
        kernel<<<grid, block>>>(d_input, d_output, n);
        CUDA_CHECK(cudaDeviceSynchronize());
    }

    const int reps = 10;
    float times[reps];
    cudaEvent_t start, stop;
    CUDA_CHECK(cudaEventCreate(&start));
    CUDA_CHECK(cudaEventCreate(&stop));

    for (int i = 0; i < reps; ++i) {
        CUDA_CHECK(cudaEventRecord(start));
        kernel<<<grid, block>>>(d_input, d_output, n);
        CUDA_CHECK(cudaEventRecord(stop));
        CUDA_CHECK(cudaEventSynchronize(stop));
        float ms = 0.0f;
        CUDA_CHECK(cudaEventElapsedTime(&ms, start, stop));
        times[i] = ms;
    }

    float sum = 0.0f;
    for (int i = 0; i < reps; ++i) sum += times[i];
    float mean = sum / reps;

    float var = 0.0f;
    for (int i = 0; i < reps; ++i) {
        float d = times[i] - mean;
        var += d * d;
    }
    float stddev = sqrtf(var / reps);

    printf("  Átlag: %.4f ms, szórás: %.4f ms\n", mean, stddev);

    CUDA_CHECK(cudaEventDestroy(start));
    CUDA_CHECK(cudaEventDestroy(stop));
    return mean;
}

int main() {
    const int n = 1 << 24; // 16M elem
    const int block_size = 256;
```

```
    const int grid_size = (n + block_size - 1) / block_size;

    // Host memória allokálása
    float* h_input = new float[n];
    float* h_output = new float[n];

    // Bemeneti adatok inicializálása
    for (int i = 0; i < n; ++i) {
        h_input[i] = static_cast<float>(rand()) / RAND_MAX;
    }

    // Eszközmemória allokálása
    float* d_input = nullptr;
    float* d_output = nullptr;
    CUDA_CHECK(cudaMalloc(&d_input, n * sizeof(float)));
    CUDA_CHECK(cudaMalloc(&d_output, n * sizeof(float)));
    CUDA_CHECK(cudaMemcpy(d_input, h_input, n * sizeof(float), cudaMemcpyHostToDevice));

    // Grid és block konfiguráció
    dim3 block(block_size);
    dim3 grid(grid_size);

    printf("Divergencia mérés, n=%d, block=%d, grid=%d\n", n, block_size, grid_size);

    printf("1) Divergenciamentes (blockIdx-alapú) kernel:\n");
    measure_kernel(coherent_kernel, d_input, d_output, n, grid, block);

    printf("2) Warp-kohéziós (threadIdx/32 alapú) kernel:\n");
    measure_kernel(warp_coherent_kernel, d_input, d_output, n, grid, block);

    printf("3) Divergens (lane-párosság alapú) kernel:\n");
    measure_kernel(divergent_kernel, d_input, d_output, n, grid, block);

    // Ellenőrzés: a kimenetek azonosak-e?
    CUDA_CHECK(cudaMemcpy(h_output, d_output, n * sizeof(float), cudaMemcpyDeviceToHost));

    // (Itt további validáció lehet: a három kernel kimenetének összehasonlítása)

    CUDA_CHECK(cudaFree(d_input));
    CUDA_CHECK(cudaFree(d_output));
    delete[] h_input;
    delete[] h_output;

    // Az eszköz hibáinak ellenőrzése
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceReset());
    return 0;
}
```

Fordítási parancs:

```
nvcc -std=c++17 -O3 -arch=sm_80 divergence_lab.cu -o divergence_lab
```

Futtatási parancs:

```
./divergence_lab
```

Várt eredmény (elemzés, nem mért adat): A divergenciamentes kernelben a `heavy` feltétel a `blockIdx.x`-től függ, amely minden warpban azonos. Ezért a hardver a teljes warpot vagy a nehezebb, vagy a könnyebb ágba irányítja, és nincs divergence. A futásidő a blokkok fele a nehezebb, fele a könnyebb ágat hajtja végre.

A warp-kohéziós kernelben a `heavy` feltétel a `threadIdx.x / 32`-től függ, amely szintén minden lane-re azonos a warpon belül. A divergence tehát ebben a kernelben sem lép fel.

A divergens kernelben a `heavy` feltétel a lane párosságától függ: minden warpban pontosan 16 páros és 16 páratlan lane van. A hardvernek minden warpban mindkét ágat végre kell hajtania, ezért a teljes futásidő várhatóan jelentősen megnő a másik két kernelhez képest. A várható arány a nehezebb ág súlyától függ; a példában a nehezebb ág 10 szorzást tartalmaz, a könnyebb ág 1-et, így a divergens kernel futásideje közelítőleg a 10 szorzásos ág teljes költségét fizeti meg minden warpban, míg a koherens kernelek a blokkok fele számára csak a könnyű ágat hajtják végre.

Mérési napló (elkülönítve a várt eredményektől): A tényleges mérési eredményeket a laborkörnyezetben kell rögzíteni, az alábbi táblázat szerint:

Kernel	Átlag (ms)	Szórás (ms)	Megjegyzés
coherent	(mért)	(mért)	
warp_coherent	(mért)	(mért)	
divergent	(mért)	(mért)	

A mérési naplóban rögzíteni kell a pontos GPU-típust, a drivert, a CUDA-verziót, az órajelet és a power limitet. A mérést legalább 10 ismétléssel, warm-up után kell elvégezni, és az átlag mellett a szórást is közölni kell. A mérési eredményeket a várt elemzéstől elkülönítve kell dokumentálni, és a nem futtatott körülményeket jelölni kell.

3.1.8 6.8. Gyakori hibák és megelőzésük

A SIMT-modell megértésének hiánya számos tipikus programozói hibához vezet. Az egyik leggyakoribb hiba az, amikor a fejlesztő azt feltételezi, hogy a szálak a kernelben megadott sorrendben hajtódnak végre. A CUDA végrehajtási modell nem garantálja sem a blokkok, sem a warpok sorrendjét, és a `__syncthreads` csak a blokkon belüli szinkronizációt biztosítja. A szálak közötti függőségeket ezért explicit szinkronizációs primitívekkel kell kezelni.

Egy másik gyakori tévedés a warp méretének feltételezése. Bár a jelenlegi NVIDIA GPU-kban a warp mérete 32, a programozónak nem szabad erre a konstansra építenie a kód logikáját olyan módon, amely a jövőbeni architektúrákon megtörik. A CUDA biztosítja a `warpSize` eszközkonstanst, de a legjobb gyakorlat az, hogy a kernel logikáját a warp méretétől függetlenül fogalmazzuk meg, kivéve, ha a teljesítmény szempontjából elengedhetetlen a warp-szintű optimalizálás.

A harmadik tipikus hiba a divergencia alábecsülése. A fejlesztők gyakran azt gondolják, hogy egy `if` elágazás elhanyagolható költséggel bír, különösen akkor, ha a feltétel csak kis részben teljesül. Azonban ha a feltétel a warp szintjén véletlenszerűen oszlik meg, minden warp kénytelen mindkét ágat végrehajtani, függetlenül attól, hogy hány lane tartozik az egyes ágakba. A megoldás a feltétel warp-kohézióra törekvő megfogalmazása, vagy a divergens részek elválasztása a fő számítási útvonaltól.

Végül gyakori hiba a shuffle- és vote-műveletek maszkjának helytelen megadása. A `__shfl_sync`, `__all_sync` és hasonló függvények minden hívásánál meg kell adni az aktív lane-ek maszkját, és a maszknak pontosan meg kell egyeznie a hívásban részt vevő szálak halmazával. Ha a maszk a warp minden lane-jét tartalmazza, a `0xffffffff` értéket kell használni. Ha a maszk nem tartalmazza az összes lane-t, a nem szereplő lane-ek viselkedése definiálatlan, és a program hibás eredményt adhat.

3.1.9 6.9. Ellenőrző kérdések

1. Magyarázd el, mi a különbség a SIMT és a SIMD között! Miért előnyös a SIMT a programozó számára?
2. Mi a warp, és hogyan osztódnak a szálak warpokba? Mekkora a warp mérete a jelenlegi NVIDIA GPU-architektúrákban?
3. Mi a branch divergence, és miért növeli a végrehajtási időt? Hogyan kerülhető el a divergence a gyakorlatban?
4. Mi a rekonvergencia, és mikor következik be? Mi történik, ha a divergens ágak különböző hosszúságúak?
5. Milyen warp-szintű primitíveket ismersz, és mire használhatók? Mi a szerepe az aktív maszknak ezekben a műveletekben?

3.1.10 6.10. Összefoglalás

A SIMT-modell a CUDA programozási modell alapköve. A hardver a szálakat warpokba szervezi, és egy warp minden lane-je ugyanazt az utasítást hajtja végre lockstep üzemmódban. Amikor a szálak feltétele eltérő ágakba vezet, branch divergence lép fel, amely a hardver párhuzamos kapacitásának kihasználatlanságához vezet. A rekonvergencia automatikus, de a divergens szakasz hossza a leglassabb ág által meghatározott. A programozó a warp-kohézió megőrzésével és a warp-szintű primitívek használatával jelentősen csökkentheti a divergence költségét. A következő alfejezetben a streaming multiprocesszor felépítését és a végrehajtási egységek szerveződését vizsgáljuk meg, amelyekre ez a warp-szintű modell épül.

3.2 Streaming multiprocessor és végrehajtási egységek

A SIMT és warp-végrehajtás alfejezetben megismertük a warp-végrehajtást: a GPU a szálakat 32 elemű csoportokba, warpokba szervezi, és ezeket a csoportokat egyetlen utasításfolyamként hajtja végre. Most annak a hardveres egységnek a felépítését vizsgáljuk, amely ezt a végrehajtást ténylegesen elvégzi: a streaming multiprocessort (SM). Az SM a GPU alapvető számítási egysége, nagyjából úgy, ahogy a CPU-ban a mag. Egy modern GPU több tucat SM-et tartalmaz, és mindegyik SM nagyfokú párhuzamosságot kezel: több ezer aktív szálat, több tucat warp-ot, és ezek között ütemezi az utasításokat. Az SM felépítésének és erőforrásainak megértése azért is kulcsfontosságú, mert a kernel teljesítménye és az SM kihasználtsága (occupancy) szorosan összefügg: a regiszterek, a shared memory és a warp-slotok mind az SM-en belüli véges erőforrások, amelyeket a kernel futtatása során foglalunk le.

3.2.1 Az SM felépítése és erőforrásai

Egy SM belsejét a következő főbb elemekre bonthatjuk:

- **Végrehajtó egységek:** CUDA core-ok, Tensor Core-ok, Special Function Unitok (SFU), betöltő/tároló egységek (LD/ST).
- **Ütemező és kibocsátó logika:** warp scheduler-ek, dispatch unit-ok, utasítás-gyorsítótár (instruction cache), scoreboard regiszterek.
- **Memória-erőforrások:** regiszterfájl, shared memory, L1 cache / texture cache, constant cache.

A pontos számok GPU-generációról GPU-generációra változnak, ezért a következőkben egy konkrét példát is adunk: az NVIDIA A100 (Ampere, compute capability 8.0) SM-jének jellemzőit. Ez a GPU 108 SM-et tartalmaz, mindegyik SM-en belül:

- 64 FP32 CUDA core,
- 32 FP64 CUDA core,
- 4 harmadik generációs Tensor Core,
- 4 Special Function Unit,
- 4 warp scheduler,
- 64 K (65 536) 32 bites regiszter a regiszterfájlban,
- 192 KiB kombinált L1/shared memory (ebből maximum 164 KiB konfigurálható shared memory-ként),
- 16 LD/ST egység (betöltés/tárolás).

Ezek az adatok a hardver típusától függően eltérhetnek, de az erőforrások jellege és a korlátok logikája minden modern GPU-ban hasonló. Az SM erőforrásai a kernel indításakor kerülnek kiosztásra. A blokkokat az SM-ekhez rendelő ütemező úgy osztja el a blokkokat, hogy minden SM-en belül az erőforrások elégségesek legyenek. A legfontosabb erőforrások, amelyek korlátozzák az egy SM-en futtatható blokkok és szálak számát:

- **Regiszterek:** minden szál lefoglalja a kernelhez szükséges regisztereket. Az SM regiszterfájlja fix méretű, ezért ha egy szál sok regisztert használ, kevesebb szál fér el.
- **Shared memory:** a blokkonkénti shared memory méret az SM teljes kapacitásából vonódik le.
- **Warp-slotok:** minden SM egy adott számú warp-ot tud nyilvántartani (pl. A100 esetén legfeljebb 64 warp-ot, azaz 2048 szálat).
- **Blokk-szám:** hardveres korlát az SM-enként aktív blokkok számára (pl. 32 blokk).
- **Egyéb erőforrások:** barrier erőforrások, végrehajtó egységek kihasználtsága.

A kernel futásakor ezek az erőforrások egyszerre kerülnek lefoglalásra: ha bármelyik kimerül, az SM nem tud több blokkot fogadni, függetlenül attól, hogy más erőforrásokból maradt-e szabad kapacitás. Ezért az erőforrások egyensúlya kritikus. Például egy kernel, amely 32 regisztert használ szálanként és 256 szálas blokkot indít, az A100-as SM-en regiszter-oldalról nézve 65 536 / (32 × 256) = 8 blokkot tudna fogadni; de mivel minden blokk 8 warp (256/32), a 8 blokk összesen 64 warp, ami pontosan eléri a maximális warp-slotok számát. Így a regiszterhasználat és a warp-korlát együttesen határozza meg a tényleges férőhelyet. Ezt a laborban számszerűen is megvizsgáljuk.

3.2.2 CUDA core, Tensor Core és SFU

A legismertebb végrehajtó egységek a **CUDA core-ok**. A modern GPU-kban a CUDA core-ok különböző típusú műveletekre specializálódtak. Az A100 SM-jén belül:

- **64 FP32 CUDA core** végzi az egyszeres pontosságú lebegőpontos szorzást és összeadást, valamint néhány egész műveletet.
- **32 FP64 CUDA core** a dupla pontosságú műveletekhez; ezekből feleannyi van, mint FP32-ből, ami a tudományos számításoknál fontos tervezési döntés. Egyes architektúrák (pl. fogyasztói GeForce modellek) FP64 teljesítménye sokkal alacsonyabb, mert a játék- és grafikai terhelés túlnyomórészt FP32-re épül.
- Az egész műveletek (INT32) gyakran külön egységeken futnak, vagy bizonyos CUDA core-ok osztottan támogatják az INT32 és FP32 műveleteket. A Volta és az Ampere generációban

az INT32 és FP32 egységek részben függetlenek, ami lehetővé teszi, hogy az egész és a lebegőpontos utasítások párhuzamosan fussanak.

A **Tensor Core-ok** speciális mátrix-szorzó és akkumuláló egységek. Elsősorban a gépi tanulási számításokra tervezték őket, de sok más lineáris algebrai feladatban is használhatók. Egy Tensor Core egyetlen utasítással egy kisebb mátrixszorzást és akkumulációt hajt végre: $D = A \times B + C$. A pontos méretek és a támogatott precíziók a Tensor Core generációjától függenek. Az A100 harmadik generációs Tensor Core-ja például támogatja az FP16, BF16, TF32, FP8 és egyes egész típusokat. A Tensor Core-ok rendkívül nagy átbocsátóképességet biztosítanak, de csak akkor, ha a mátrixok a megfelelő elrendezésben állnak rendelkezésre és a kernel kifejezetten Tensor Core utasításokat használ (például a `mma.sync` PTX-intrinsics vagy a WMMA API segítségével). A hagyományos CUDA core-okkal írt GEMM-kernelekhez képest a Tensor Core-ok akár egy nagyságrenddel nagyobb teljesítményt adhatnak, de az adatelrendezés és a precízió kezelése összetettebb. A Tensor Core-okkal a Tensor Core architektúra és CUTLASS foglalkozik részletesen.

A **Special Function Unitok (SFU)** a speciális matematikai függvények gyors, közelítő kiértékelését végzik. Ide tartoznak például a szinusz, koszinusz, exponenciális, logaritmus, reciprok és négyzetgyök-reciprok. A CUDA intrinsic függvények, mint a `__sinf`, `__cosf`, `__expf`, `__logf`, `__frcp_rn`, `__frsqrt_rn` az SFU-kat használják, és jóval gyorsabbak a precízebb, szoftveres könyvtári függvényeknél (`sinf`, `cosf`, `expf` stb.). Az SFU-kból SM-enként viszonylag kevés van (például 4), ezért ha egy kernel nagyon sok ilyen függvényt hív, az SFU válhat szűk keresztmetszetté. A pontosságuk is alacsonyabb: például a `__sinf` és `__cosf` garantált hibakorlátja néhány ulp, de a pontos szabványos függvényekhez képest nagyobb eltérést mutathatnak. Numerikus pontossági kérdésekkel a Lebegőpontos pontosság foglalkozik.

3.2.3 Warp scheduler és issue

Az SM-en belül a **warp scheduler-ek** felelősek az aktív warp-ok utasításainak kiválasztásáért és kibocsátásáért. Minden scheduler egy adott számú warp-ot kezel, és minden órajelben kiválaszt egy olyan warp-ot, amelynek a következő utasítása készen áll a végrehajtásra. A kiválasztás általában valamilyen prioritási vagy round-robin stratégia szerint történik, de a gyakorlatban a scheduler-ek szabadon választhatnak a számukra elérhető warp-ok közül.

A scheduler által kibocsátott utasítás a **dispatch unit**-hoz kerül, amely továbbítja a megfelelő végrehajtó egységhez. Az utasítás-kibocsátás sebességét **issue rate**-nek nevezzük: ez megmondja, hogy egy scheduler hány utasítást tud kibocsátani egy órajelben. A legtöbb modern GPU-ban egy scheduler órajelenként legfeljebb egy utasítást bocsáthat ki, de egyes architektúrák megengednek bizonyos feltételek mellett két utasítást is (dual issue), például ha az egyik FP32, a másik INT32 művelet. Az A100-ban 4 warp scheduler van SM-enként; mindegyik 16 FP32 CUDA core-hoz kapcsolódik, így a 4 scheduler összesen 64 FP32 core-t vezérel. Minden scheduler így egy-egy 16 szálas csoportot tud kiszolgálni órajelenként, ami warp-szinten 32 szál feldolgozását jelenti két órajel alatt.

A **scoreboard** mechanizmus nyilvántartja, hogy egy warp regiszterei készen állnak-e. Ha egy utasítás operandusa még nem elérhető (például egy korábbi memóriaművelet vagy nagy késleltetésű művelet eredményére vár), a warp nem „ready” állapotú, és a scheduler nem bocsátja ki a következő utasítását. Ez a mechanizmus teszi lehetővé a **latency hiding**-ot: amíg egy warp vár, a scheduler egy másik ready warp-ot választ, így a végrehajtó egységek folyamatosan dolgozhatnak. Ha túl kevés aktív warp van, a scheduler-eknek nem lesz elég ready jelöltjük, és a végrehajtó egységek kihasználatlanul maradnak. Ezért fontos az occupancy – az aktív warp-ok száma SM-enként – optimalizálása. Az occupancy és a regiszternyomás témáját az Occupancy és

erőforrás-korlátok–Occupancy optimalizálás fejezetek tárgyalják részletesen, de az alapfogalmakat itt vezetjük be.

3.2.4 Példa: erőforrások becslése egy kernelre

A következő labor azt mutatja be, hogyan lehet megbecsülni és ellenőrizni egy kernel SM-erőforrásigényét. A cél nem egy bonyolult algoritmus, hanem egy kis kernel, amelynek regiszterhasználatát és ebből adódó férőhelyét pontosan nyomon követhetjük.

Kernelkód (CUDA C++, `sm_resource.cu`):

```
#include <cuda_runtime.h>
#include <stdio.h>
#include <stdlib.h>

#define CUDA_CHECK(call) do {                                \
    cudaError_t err = call;                                  \
    if (err != cudaSuccess) {                                \
        printf("CUDA error at %s:%d: %s\n", __FILE__, __LINE__, \
               cudaGetErrorString(err));                     \
        exit(EXIT_FAILURE);                                  \
    }                                                        \
} while (0)

__global__ void sm_resource_kernel(const float* a, const float* b,
                                   float* c, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        float reg[4];           // 4 regiszter, ha a fordító nem optimalizálja el
        reg[0] = a[idx];
        reg[1] = b[idx];
        reg[2] = reg[0] * reg[1];
        reg[3] = reg[2] + 1.0f;
        c[idx] = reg[3];
    }
}

int main(void) {
    const int N = 1 << 20;      // 1 048 576 elem
    size_t bytes = N * sizeof(float);

    float *h_a = (float*)malloc(bytes);
    float *h_b = (float*)malloc(bytes);
    float *h_c = (float*)malloc(bytes);
    if (!h_a || !h_b || !h_c) {
        fprintf(stderr, "Host allocation failed\n");
        exit(EXIT_FAILURE);
    }
    for (int i = 0; i < N; ++i) {
        h_a[i] = 1.0f;
        h_b[i] = 2.0f;
    }

    float *d_a, *d_b, *d_c;
    CUDA_CHECK(cudaMalloc(&d_a, bytes));
    CUDA_CHECK(cudaMalloc(&d_b, bytes));
    CUDA_CHECK(cudaMalloc(&d_c, bytes));
    CUDA_CHECK(cudaMemcpy(d_a, h_a, bytes, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_b, h_b, bytes, cudaMemcpyHostToDevice));

    int blockSize = 256;
```

```
    int gridSize = (N + blockSize - 1) / blockSize;

    // Occupancy lekérdezés, mielőtt a kernelt futtatnánk
    int minGridSize, suggestedBlockSize;
    CUDA_CHECK(cudaOccupancyMaxPotentialBlockSize(&minGridSize,
                                                  &suggestedBlockSize,
                                                  sm_resource_kernel,
                                                  0, blockSize));
    printf("Suggested block size for max occupancy: %d\n", suggestedBlockSize);

    int maxActiveBlocks;
    CUDA_CHECK(cudaOccupancyMaxActiveBlocksPerMultiprocessor(
                  &maxActiveBlocks, sm_resource_kernel, blockSize, 0));
    printf("Max active blocks per SM for block size %d: %d\n",
           blockSize, maxActiveBlocks);

    // Kernel futtatás és időmérés eseményekkel
    cudaEvent_t start, stop;
    CUDA_CHECK(cudaEventCreate(&start));
    CUDA_CHECK(cudaEventCreate(&stop));
    CUDA_CHECK(cudaEventRecord(start));
    sm_resource_kernel<<<gridSize, blockSize>>>(d_a, d_b, d_c, N);
    CUDA_CHECK(cudaEventRecord(stop));
    CUDA_CHECK(cudaEventSynchronize(stop));
    float ms = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&ms, start, stop));
    printf("Kernel time: %.3f ms\n", ms);

    // Hibakód ellenőrzés a kernel után
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    CUDA_CHECK(cudaFree(d_a));
    CUDA_CHECK(cudaFree(d_b));
    CUDA_CHECK(cudaFree(d_c));
    free(h_a);
    free(h_b);
    free(h_c);
    return 0;
}
```

A kernel célja minimális: minden szál beolvas két float értéket, elvégzi a szorzást, hozzáad egy konstanst, majd kiírja az eredményt. A `float reg[4]` tömb szándékosan szerepel, hogy a regiszterhasználatot kissé megnövelje és ellenőrizhetővé tegye. A gyakorlatban a fordító optimalizálhatja a tömböt, ezért a regiszterhasználat pontos értékét a `-Xptxas -v` kapcsolóval kapott fordítási napló mutatja.

Build és futtatás (Linux, bash):

```
nvcc -arch=sm_80 -Xptxas -v -o sm_resource sm_resource.cu
./sm_resource
```

A `-Xptxas -v` a PTX assembler részletes kimenetét írja ki, többek között a kernel által használt regiszterek számát és a shared memory méretét. A kimenet egy lehetséges sorai:

```
ptxas info    : Compiling entry function '_Z19sm_resource_kernelPKfS0_Pfi' for 'sm_80'
ptxas info    : Function properties for _Z19sm_resource_kernelPKfS0_Pfi
    0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
ptxas info    : Used 8 registers, 0 bytes smem, 376 bytes cmem[0]
```

Ez azt jelenti, hogy a kernel szálanként 8 regisztert használ. A `reg[4]` tömb ellenére a fordító csak 8 regiszterre optimalizálta a kódot, ami jó példa arra, hogy a forráskódban látható változók száma nem egyezik meg a hardveres regiszterhasználattal.

Mérési napló

Az alábbi táblázat a laborban rögzítendő adatokat mutatja. A konkrét értékek az adott környezettől függenek, ezért a táblázat egy példa. A valódi mérés során a saját GPU, driver és CUDA verziót kell beírni, és a mért értékeket az API visszatérési értékeiből, illetve a mérési eredményekből kell származtatni.

Mező	Várt érték (elméleti)	Mért érték (példa)
GPU	–	NVIDIA A100 40 GB
Driver	–	470.129.06
CUDA verzió	–	11.4
Compute capability	8.0	8.0
Blokkméret	256	256
Regiszter/thread	(fordításból)	8
SM regiszterek száma	65 536	–
Regiszterekből adódó max blokk/SM	65536 / (8*256) = 32	–
Warp-slotokból adódó max blokk/SM	64 warp / (256/32 warp) = 8	–
Elvárt max aktív blokk/SM	min(32, 8) = 8	–
API által jelentett max blokk/SM	–	8
Kernel futási idő	–	0,112 ms

A táblázat jól mutatja, hogy a regiszterhasználat alapján 32 blokk is elférne, de a warp-slotok limitje 8-ra korlátozza a tényleges férőhelyet. Ez a tipikus példa arra, hogy az occupancy nem egyetlen erőforrástól függ. A `cudaOccupancyMaxActiveBlocksPerMultiprocessor` API ezt a komplex korlátot számítja ki, és visszaadja a ténylegesen maximális blokkszámot.

Fontos: A laborban a `cudaOccupancyMaxPotentialBlockSize` arra is használható, hogy a kernelhez javasolt blokkméretet kapjunk, amely a legjobb occupancy-t eredményezi. A példában `suggestedBlockSize` értéke valószínűleg 256 lesz (vagy 128/512 a hardvertől függően), de ezt az API határozza meg.

3.2.5 Tipikus hibák és félreértések

1. **Azt gondolni, hogy a magas occupancy automatikusan maximális teljesítményt ad.** Az occupancy fontos, de nem minden: ha egy kernel memóriakorlátos, a magasabb occupancy nem feltétlenül növeli a sávszélességet, sőt, a nagyobb regiszternyomás vagy shared memory használat ronthatja a cache hatékonyságát. Az optimális occupancy-t mindig mérésekkel kell igazolni.
2. **A regiszterhasználatot a forráskód változóiból közvetlenül megbecsülni.** A CUDA fordító agresszívan optimalizál, eltávolíthatja a felesleges változókat, összevonhat műveleteket, vagy éppen ellenkezőleg, a szükséges ideiglenes értékek miatt több regisztert használhat. A pontos regiszterszámot mindig a `-Xptxas -v` kimenetéből vagy a profilkészítő eszközökből kell kiolvasni.
3. **A Tensor Core-ok használatának figyelmen kívül hagyása.** Sok fejlesztő a hagyományos CUDA core-okkal ír meg mátrixszorzást, és nem használja ki a Tensor Core-ok nyújtotta teljesítményt. A Tensor Core-ok azonban csak speciális elrendezésű adatokkal

és megfelelő precízióval működnek hatékonyan. Ha a kernel nem Tensor Core-barát, a teljesítmény jelentősen elmarad az elvárttól.

4. **Az SFU-k túlterhelése.** Ha egy kernel nagyon sok `__sinf`, `__expf` vagy hasonló intrinsic függvényt hív, az SFU-k válhatnak szűk keresztmetszetté, mert számuk SM-enként kicsi. Ilyenkor érdemes megfontolni, hogy a pontosabb, de CUDA core-okon futó könyvtári függvények (pl. `sinf`, `expf`) vagy táblázatos interpoláció hatékonyabb-e.
5. **Az SM erőforrásainak helytelen számonkérése kernel indításkor.** A fejlesztő néha azt feltételezi, hogy ha a grid nagyobb, mint az SM-ek száma, akkor az összes SM párhuzamosan dolgozik. Valójában az SM-ek számánál több blokk nem fut egyszerre; a felesleg a hardveres ütemező várólistáján marad. Az SM-enkénti tényleges blokkszámot az erőforrások foglaltsága határozza meg.

3.2.6 Önellenőrző kérdés

Egy adott CUDA kernel 16 regisztert használ szálanként, és a blokkmérete 128. A GPU SM-jében 65 536 regiszter és 64 warp-slot található. Hány blokk fér el egy SM-en, ha nincs más korlát (például shared memory vagy blokk-szám limit)? Indokolja a választ, és magyarázza el, melyik erőforrás a szűk keresztmetszet.

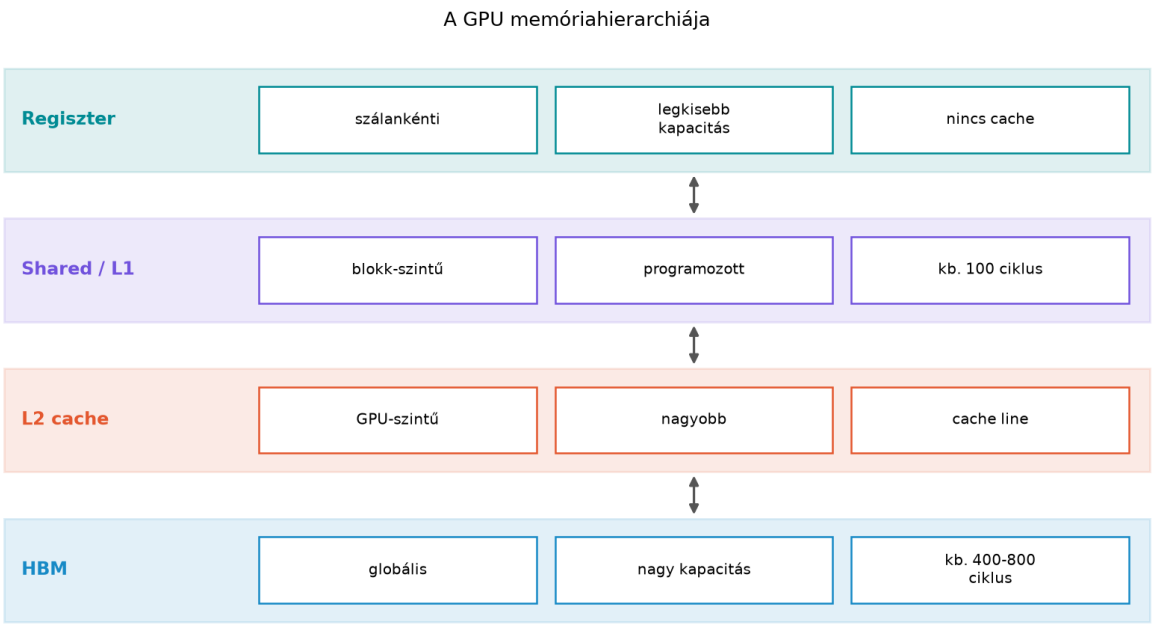
Megoldás:
- Regiszterekből: 65 536 / (16 × 128) = 65 536 / 2048 = 32 blokk.
- Warp-slotokból: 64 warp / (128/32 warp) = 64 / 4 = 16 blokk.
- A szűk keresztmetszet a warp-slotok száma, tehát maximum 16 blokk fér el az SM-en. Ha a regiszterhasználat 8 lenne, a regiszter-oldal 64 blokkot engedne, de a warp-slotok továbbra is 16-ra korlátoznák.

Ez a kérdés kiválóan illusztrálja, hogy az SM-nek többféle erőforrás-korlátja van, és a legszűkebb határozza meg az occupancy-t. A következő fejezetek – különösen az Occupancy és erőforrás-korlátok–Occupancy optimalizálás – részletesen foglalkoznak ezeknek a korlátoknak az elemzésével és optimalizálásával.

3.3 Memóriahierarchia: regiszter, shared, L1/L2, global

A GPU akkor számol gyorsan, ha a végrehajtó egységeknek nem kell folyamatosan a lassú, nagy kapacitású memóriára várniuk. A memóriahierarchia minden szintjét három kérdés írja le: **ki látja**, **mekkora a kapacitása**, és **mennyibe kerül elérni**. Ebben az alfejezetben ezeket a szinteket járjuk körbe: regiszterfájl, shared memory, L1/L2 cache, valamint a HBM-alapú global memory. A cél nem egyetlen hardvergeneráció számainak bemagolása, hanem az, hogy a kernel memóriaigényét a későbbi fejezetekben önállóan tudjuk becsülni és méréssel ellenőrizni.

A következő táblázat és a példák az NVIDIA A100 (compute capability 8.0) környezetét használják tájékoztató számokkal. A pontos kapacitás- és késleltetési adatok erősen verzió- és hardverfüggők, ezért minden mérés előtt a `deviceQuery`, a gyártói adatlap és a fordító `ptxas -v` kimenete alapján ellenőrizni kell.



3.2. ábra: A GPU memóriahierarchiája - Fentről lefelé nő a kapacitás és a késleltetés; a regiszter és a shared memory az SM-hez közel van, a HBM a nagy kapacitású globális tár.

3.3.1 1. A memóriaszintek térképe

A gyakorlati GPU-programozásban a leghasznosabb leegyszerűsítés az alábbi rétegmodell:

Szint	Láthatóság	Példakapacitás (A100 80 GB)	Nagyságrendi késleltetés	Gyakorlati szerep
Regiszter	egy szál privát	64 K db 32 bites regiszter/SM, összesen 256 KB/SM; legfeljebb 255 regiszter/szál	gyakorlatilag az utasításütemezés-be olvad; spill esetén viszont global-lassú	a leggyorsabb munkaterület, a számítás közvetlen operandusainak tárolása
Shared memory	egy blockon belüli szálak	192 KB L1/shared együttes per SM; ebből blockonként legfeljebb 163 KB foglalható le	nagyjából 20–30 ciklus	szoftver által vezérelt on-chip adatterület adat-újrafelhasználáshoz
L1 cache	SM-en belüli hardver	192 KB L1/shared együttes carve-out, a konfigurációtól függően	nagyjából 30–40 ciklus	a global/local hozzáférések automatikus gyorsítótára

Szint	Láthatóság	Példakapacitás (A100 80 GB)	Nagyságrendi késleltetés	Gyakorlati szerep
L2 cache	az egész GPU	40 MB	nagyjából 200 ciklus nagyságrend	az összes SM közös gyorsítótára, HBM-késleltetés elrejtése
Global memory / HBM	device szint	40–80 GB HBM	nagyjából 400–700 ciklus; nagy sávszélességű burstök	a nagy adathalmazok tényleges tárolója

Ez a rétegződés azt jutalmazza, ha egy adatot többször használunk egy szűk, gyors szinten, és akkor büntet, ha nagy távolságra, gyakran kell hozzáférni. A későbbi tiling, coalescing és cache-optimalizálási fejezetek mind erre az alapelvre épülnek.

3.3.2 2. Regiszterfájl: a leggyorsabb szint

A regiszterek az egyes szálak privát változói. A CUDA-fordító a kernel lokális skalárváltozóit elsősorban regiszterekbe próbálja elhelyezni. Egy A100 SM-en a regiszterfájl mérete 64 K darab 32 bites regiszter, azaz 256 KB. Ez osztozik az SM-en futó összes szál között.

Egy szál legfeljebb 255 regisztert kaphat. Ha a kernel például szálanként 64 regisztert használ és egy block 256 szálból áll, akkor egy block regiszterigénye:

$$R_{\text{block}} = 64 \cdot 256 = 16\,384.$$

Egy SM-en a regiszterkorlát ekkor legfeljebb

$$\left\lfloor \frac{65\,536}{16\,384} \right\rfloor = 4$$

block egyidejű futtatását engedi. Ez fontos korlát: ha csak a szálak számát nézzük, 2048 szál elférne, de a regiszterfájl már 1024 szálnál leállítja az SM-et. Ezt a jelenséget hívják **register pressure** okozta occupancy-csökkenésnek.

A fordító a `ptxas -v` kapcsolóval megmutatja a becsült regiszterhasználatot:

```
nvcc -O3 -std=c++17 -arch=sm_80 -Xptxas -v kernel.cu -o kernel
```

A kimenetben például ilyen sorok jelennek meg:

```
ptxas info    : Used 12 registers, 0 bytes smem, 400 bytes cmem[0]
```

Ha a kernel több regisztert kérne, mint amennyi elérhető, a fordító **spill**-t végez: a regiszterek tartalmát a **local memory**-ba teszi. A local memory logikailag szálankénti, fizikailag azonban a global memory területen él, és az L1/L2 cache gyorsíthatja. A spill könnyen felismerhető a gyenge teljesítmény és a `st.local`/`ld.local` PTX-utasítások megjelenése alapján.

Gyakorlati tanulság: a regiszterekkel takarékoskodni kell, de csak addig, amíg a megtakarítás nem okoz spill-t. Ha egy blokk 25%-kal kevesebb regisztert használ, több block fér az SM-re; ha viszont emiatt a fordító local memoryba menekít, a nyereség könnyen elvész.

3.3.3 3. Shared memory: block-szintű, programozott on-chip tár

A shared memory az SM-en belüli, block-szinten látható, kis késleltetésű memória. Nem automatikus cache, hanem **szoftver által kezelt scratchpad**: a programozó dönti el, mit tesz bele, mikor olvassa, és mikor szinkronizál.

Példakapacitás: az A100-ban az L1 és a shared memory közös 192 KB-os területen osztozik, és a carve-out beállításától függően az SM-en legfeljebb 164 KB shared memory lehet. Egy block legfeljebb 163 KB-ot foglalhat. Egy jellemző launch:

```
__global__ void scale_shared_kernel(const float* __restrict__ x,
                                    float* __restrict__ y,
                                    float alpha, long long n) {
    __shared__ float tile[256];

    long long i = static_cast<long long>(blockIdx.x) * blockDim.x + threadIdx.x;
    long long stride = static_cast<long long>(gridDim.x) * blockDim.x;

    for (; i < n; i += stride) {
        tile[threadIdx.x] = x[i];
        __syncthreads();          // a tile minden elemének láthatónak kell lennie
        y[i] = alpha * tile[threadIdx.x] + y[i];
    }
}
```

Itt a `tile` tömb 256 `float` méretű, azaz blockonként 1024 bájt shared memóriát foglal. A `__syncthreads()` azt garantálja, hogy a block minden szála elérte az írást, mielőtt bármelyik szál olvasná a tile elemeit.

A shared memory akkor éri meg, ha az adatot **többször használjuk**. A fenti példa rossz példa a shared memory használatára, mert minden `x[i]` elemet pontosan egyszer olvasunk be a tile-ba, majd egyszer olvasunk vissza. A shared memória itt csak extra szinkronizációt hoz, és legfeljebb annak szemléltetésére alkalmas, hogy a puszta shared-használat nem gyorsít. Az igazi nyereség akkor jön, ha például egy stencil vagy mátrixblokk ugyanazt az elemet többször olvassa.

A shared memória méretét statikusan, például `__shared__ float s[1024];` formában, vagy dinamikusan, `extern __shared__ float s[];` módon lehet megadni. A dinamikus változatnál a launch harmadik, `shared_bytes` paramétere adja meg a tényleges bájtszámot:

```
scale_shared_kernel<<<grid, block, shared_bytes, stream>>>(...);
```

A static shared memory gyorsan ellenőrizhető, a dinamikus változat pedig olyankor hasznos, ha a blockméretet futásidőben választjuk meg.

3.3.4 4. L1 és L2 cache

A GPU-nak két automatikus cache-szintje is van. Az L1 cache SM-enként, az L2 cache device-szinten van jelen. Mindkettő a global memory felé irányuló forgalmat csökkentheti.

Az A100-ban az L1 és a shared memory közös területen helyezkedik el. Ha egy kernel nagy shared memóriát igényel, a rendelkezésre álló L1 mérete csökkenhet. Ezt a kompromisszumot a carve-out konfiguráció szabályozza. A gyakorlati döntés ezért nem az, hogy „shared vagy L1”, hanem az, hogy mi a fontosabb az adott kernelnek: programozott adat-újrafelhasználás vagy automatikus cache.

Az L2 gyorsítótár az egész GPU-ra kiterjed. Az A100-ban 40 MB, és minden SM számára elérhető. Nagy szerepe van abban, hogy a HBM felé irányuló kérések egy része ne menjen ki

a memóriavezérlőhöz. Az L2 egyben az atomi műveletek gyorsításában és a memóriák közötti koherencia nagy részében is fontos.

A cache-ek leghatékonyabban akkor működnek, ha a hozzáférések térben és időben lokálisak. A **coalescing** fogalma a következő fejezetben részletesen előkerül, de a lényeg már itt is érthető: a GPU memóriarendszere nem egyesével szolgálja ki a 32 szálat, hanem szektorokban és cache-line-okban kéri az adatot. Ha egy warp szálai szomszédos címeket olvasnak, egy tranzakció is elég lehet; ha szétszórtan, akkor több tranzakció keletkezik, és a tényleges sávszélesség drámaian csökken.

3.3.5 5. HBM és memóriacsatornák

A nagy kapacitású, device-szintű memória a modern GPU-kban **High Bandwidth Memory**. A HBM abban különbözik a hagyományos GDDR-től, hogy a DRAM-lapkákat egymásra rétegzik és mikroáthidalásokkal, úgynevezett through-silicon via-kkal kötik össze. Emiatt sokkal szélesebb busz fér el kis helyen.

A sávszélesség közelítő képlete:

$$B = \frac{w_{\text{bus}}}{8} \cdot f_{\text{data}},$$

ahol w_{bus} a busz szélessége bitben, f_{data} pedig az effektív adatsebesség. Egy A100 80 GB-os SXM-változatánál például nagyjából 5120 bites effektív busz és körülbelül 3,2 Gbit/s adatsebesség körülbelül 2048 GB/s csúcssávszélességet ad. A pontos értéket a gyártói adatlap és a mérési környezet rögzíti.

A HBM több **memóriacsatornára** bomlik. A memóriavezérlők a címeket csatornák között osztják szét, ezért a jól elrendezett, szomszédos hozzáférések több csatornát párhuzamosan terhelnek, a szétszórt hozzáférések pedig várakozást okoznak. Ezért a global memory sávszélesség kihasználásánál a „több adat kevesebb tranzakcióban” elv legalább annyira fontos, mint a nyers FLOP-teljesítmény.

A global memory a kernel szempontjából a `cudaMalloc` által lefoglalt eszközmemória, amelyet kernelből a `__global__` függvényben pointerként érünk el. A host és a device közötti átvitel ugyanezen a memórián keresztül, jellemzően PCIe vagy NVLink úton történik, ezért a nagy másolások önálló mérési pontot jelentenek.

3.3.6 6. Kernel memóriaigényének becslése

A kernel memórialábnyomát nem elég „bájtban” megadni; a kulcskérdés az, hogy a foglalás hogyan ütközik az SM erőforrás-korlátaiba. A becslés három fő részből áll:

1. **Regiszterigény szálanként**: `R_thread` regiszter, amelyet a `ptxas -v` mutat.
2. **Shared memory igény blockonként**: `S_block` bájt.
3. **Global memory forgalom**: `G_total` bájt, amely a beolvasott és kiírt tömbökből számítható.

Az SM-re helyezhető blockok számát három korlát együttesen határozza meg:

$$B_{\text{threads}} = \left\lfloor \frac{T_{\max}}{T_{\text{block}}} \right\rfloor,$$

$$B_{\text{regs}} = \left\lfloor \frac{R_{\text{SM}}}{R_{\text{thread}} \cdot T_{\text{block}}} \right\rfloor,$$

$$B_{\text{shared}} = \left\lfloor \frac{S_{\text{SM}}}{S_{\text{block}}} \right\rfloor,$$

ahol $T_{\max}$ az SM-enként maximális szálszám, T_{block} a blockméret, R_{SM} az SM regiszterfájljának mérete regiszterben, S_{SM} a shared memória rendelkezésre álló mérete. A tényleges blockszám:

$$B = \min\left(B_{\text{threads}}, B_{\text{regs}}, B_{\text{shared}}, B_{\text{hardware}}\right).$$

Példaként vegyük a `scale_kernel` vektorösszegzést, ahol blockonként 256 szál van, a regiszterhasználat 12, shared memória nincs, és A100-as SM-en $T_{\max} = 2048$, $R_{\text{SM}} = 65\,536$, $S_{\text{SM}} = 164\,\text{KB}$:

- Szálkorlát: $B_{\text{threads}} = 2048/256 = 8$.
- Regiszterkorlát: $65\,536/(12 \cdot 256) \approx 21$.
- Shared korlát: nincs korlát.
- Hardverkorlát: maximum block per SM az A100-on 32.

A tényleges érték $B = \min(8, 21, 32) = 8$ block, azaz 2048 szál. Az occupancy-t ebben az esetben a szálkorlát határozza meg.

A `scale_shared_kernel` esetében a shared memória 1024 bájt blockonként. A shared korlát:

$$B_{\text{shared}} = \left\lfloor \frac{164\,K}{1\,K} \right\rfloor = 164,$$

tehát továbbra is a szálkorlát 8 blockot enged. A shared használat tehát ebben a kernelben nem csökkenti az occupancy-t, de nem is ad sávszélesség-nyereséget, mert nincs valódi adatújrafelhasználás.

3.3.7 Labor: kernel memóriaigény becslése és mérése

A labor célja, hogy a becsült memóriaigényt és a mért futásjellemzőket egymás mellé tegye. A környezet a következő laborpélda; a konkrét mezőket mindenkinél a saját eszközéhez kell igazítani.

Laborpélda-környezet (nem ellenőrzött mérés):

Mező	Érték
GPU	NVIDIA A100-PCIe-40GB
Compute capability	8.0
Driver	545.23.08
CUDA toolkit	12.3
OS	Ubuntu 22.04.3 LTS
Power limit	250 W
ECC	engedélyezve
MIG	letiltva
Fordító	`nvcc` 12.3, `-O3 -std=c++17`

A mintakód a `scale_kernel`-t és a `scale_shared_kernel`-t futtatja. A kernel memóriaforgalma három tömböt érint: `x` beolvasás, `y` beolvasás-módosítás, `y` kiírás. Így $G_{\text{total}} = 3 \cdot n \cdot 4$ bájt.

```
// CUDA C++17
// Fordítás:
// nvcc -O3 -std=c++17 -arch=sm_80 -Xptxas -v lab_memhier.cu -o lab_memhier
#include <cuda_runtime.h>
#include <cstdio>
#include <cstdlib>
#include <vector>
#include <cmath>
#include <algorithm>

#define CHECK_CUDA(call)                                                       \
    do {                                                                        \
        cudaError_t err = call;                                                 \
        if (err != cudaSuccess) {                                               \
            std::fprintf(stderr, "CUDA error %s:%s:%d: %s\n",                   \
                         __FILE__, #call, __LINE__, cudaGetErrorString(err));   \
            std::exit(EXIT_FAILURE);                                            \
        }                                                                       \
    } while (0)

__global__ void scale_kernel(const float* __restrict__ x,
                             float* __restrict__ y,
                             float alpha, long long n) {
    long long i = static_cast<long long>(blockIdx.x) * blockDim.x + threadIdx.x;
    long long stride = static_cast<long long>(gridDim.x) * blockDim.x;
    for (; i < n; i += stride) {
        y[i] = alpha * x[i] + y[i];
    }
}

__global__ void scale_shared_kernel(const float* __restrict__ x,
                                    float* __restrict__ y,
                                    float alpha, long long n) {
    __shared__ float tile[256];

    long long base = static_cast<long long>(blockIdx.x) * blockDim.x + threadIdx.x;
    long long stride = static_cast<long long>(gridDim.x) * blockDim.x;
    for (long long i = base; i < n; i += stride) {
        tile[threadIdx.x] = x[i];
        __syncthreads();
        y[i] = alpha * tile[threadIdx.x] + y[i];
    }
}

__global__ void init_y_kernel(float* __restrict__ y, float value, long long n) {
    long long i = static_cast<long long>(blockIdx.x) * blockDim.x + threadIdx.x;
    long long stride = static_cast<long long>(gridDim.x) * blockDim.x;
    for (; i < n; i += stride) {
        y[i] = value;
    }
}

float measure(const char* name, long long n, dim3 grid, int block,
              void (*launch)(const float*, float*, float, long long, dim3, int),
              const float* d_x, float* d_y, float alpha) {
    // Warm-up
    launch(d_x, d_y, alpha, n, grid, block);
    CHECK_CUDA(cudaDeviceSynchronize());

    cudaEvent_t start, stop;
```

```
    CHECK_CUDA(cudaEventCreate(&start));
    CHECK_CUDA(cudaEventCreate(&stop));

    const int repeats = 10;
    CHECK_CUDA(cudaEventRecord(start));
    for (int r = 0; r < repeats; ++r) {
        launch(d_x, d_y, alpha, n, grid, block);
    }
    CHECK_CUDA(cudaEventRecord(stop));
    CHECK_CUDA(cudaEventSynchronize(stop));

    float ms = 0.0f;
    CHECK_CUDA(cudaEventElapsedTime(&ms, start, stop));
    CHECK_CUDA(cudaEventDestroy(start));
    CHECK_CUDA(cudaEventDestroy(stop));

    double avg_ms = ms / repeats;
    double bytes = 3.0 * static_cast<double>(n) * sizeof(float);
    double bw_gbs = bytes / (avg_ms * 1e6);

    std::printf("%-22s avg_time=%8.3f ms   effective_bandwidth=%8.1f GB/s\n",
                name, avg_ms, bw_gbs);
    return bw_gbs;
}

void launch_scale(const float* d_x, float* d_y, float alpha, long long n,
                  dim3 grid, int block) {
    scale_kernel<<<grid, block>>>(d_x, d_y, alpha, n);
}

void launch_scale_shared(const float* d_x, float* d_y, float alpha, long long n,
                         dim3 grid, int block) {
    scale_shared_kernel<<<grid, block>>>(d_x, d_y, alpha, n);
}

int main() {
    const long long n = 1LL << 24;   // 16 777 216 elem
    const int block = 256;
    const size_t grid = static_cast<size_t>(n / block); // n osztható, grid-stride esetén is
↪   biztonságos
    const float alpha = 2.0f;
    const size_t bytes = static_cast<size_t>(n) * sizeof(float);

    std::vector<float> hx(n, 1.0f);
    std::vector<float> hy(n, 2.0f);
    std::vector<float> hy_out(n, 0.0f);

    float* d_x = nullptr;
    float* d_y = nullptr;
    CHECK_CUDA(cudaMalloc(&d_x, bytes));
    CHECK_CUDA(cudaMalloc(&d_y, bytes));

    CHECK_CUDA(cudaMemcpy(d_x, hx.data(), bytes, cudaMemcpyHostToDevice));
    CHECK_CUDA(cudaMemcpy(d_y, hy.data(), bytes, cudaMemcpyHostToDevice));

    dim3 launch_grid(static_cast<unsigned int>(grid));

    measure("scale_kernel", n, launch_grid, block,
            launch_scale, d_x, d_y, alpha);

    // Visszaállítjuk y értékét, hogy a második mérés ugyanabból az inputból induljon.
    init_y_kernel<<<launch_grid, block>>>(d_y, 2.0f, n);
    CHECK_CUDA(cudaDeviceSynchronize());
```

```
    measure("scale_shared_kernel", n, launch_grid, block,
            launch_scale_shared, d_x, d_y, alpha);

    CHECK_CUDA(cudaMemcpy(hy_out.data(), d_y, bytes, cudaMemcpyDeviceToHost));

    float max_err = 0.0f;
    for (long long i = 0; i < n; ++i) {
        max_err = std::max(max_err, std::fabs(hy_out[i] - 4.0f));
    }
    std::printf("max_absolute_error=%g (FP32, alpha=%g)\n", max_err, alpha);

    CHECK_CUDA(cudaFree(d_x));
    CHECK_CUDA(cudaFree(d_y));
    return 0;
}
```

A kernel launch alakja:

```
scale_kernel<<<65536, 256>>>
scale_shared_kernel<<<65536, 256>>>
```

A két kernel becsült memóriaigénye:

Jellemző	`scale_kernel` várt	`scale_shared_kernel` várt
Regiszter/szál	8–12	10–14
Shared memória/block	0 B	1024 B
Global forgalom	$3 \cdot n \cdot 4$ B	$3 \cdot n \cdot 4$ B
Occupancy	2048 szál/SM, 8 block	2048 szál/SM, 8 block
Várt sávszélesség A100-PCIe-40GB esetén	a csúcs nagyjából 70–85%-a	hasonló vagy kissé alacsonyabb

A sávszélesség-mérést a fenti labor méri. A mérési naplóban külön kell feltüntetni a becslést és a mért értéket, például:

Eredmény	Várt	Mért (a laborban kitöltendő)
`scale_kernel` hatékony sávszélesség	1100–1300 GB/s	`<saját mérés>`
`scale_shared_kernel` hatékony sávszélesség	hasonló vagy kisebb	`<saját mérés>`
`scale_kernel` regiszter/szál	8–12	`<ptxas -v nélkül>`
`scale_shared_kernel` shared/block	1024 B	`<ptxas -v>`

A pontos regiszter- és shared-értékek meghatározásához az Nsight Compute futtatható:

```
ncu --metrics
↪ launch__registers_per_thread,launch__shared_mem_per_block,sm__warps_active.avg.per_cycle_active,dram__bytes_
↪ ./lab_memhier
```

A `dram__bytes_read.sum` és a `dram__bytes_write.sum` metrikák azt mutatják, hogy a valós HBM-forgalom mennyire közelíti a becslést. Ha a `scale_shared_kernel` mért sávszélessége nem jobb, mint a `scale_kernel`-é, az nem meglepetés: a shared memória itt nem csökkenti a HBM-forgalmat, mert nincs adat-újrafelhasználás. A labor éppen azt erősíti meg, hogy a memóriaszintek puszta jelenléte még nem optimalizálás.

3.3.8 OpenCL és JAX megfelelők

OpenCL-ben a shared memory fogalma a `__local` memória, a block pedig a work-group. A kernel ugyanazt a memóriaigény-becslést használja:

```
// OpenCL C 1.2 / OpenCL 3.0
__kernel void scale_kernel(__global const float* x,
                           __global float* y,
                           float alpha, long n) {
    int i = get_global_id(0);
    int stride = get_global_size(0);
    for (; i < n; i += stride) {
        y[i] = alpha * x[i] + y[i];
    }
}

__kernel void scale_local(__global const float* x,
                          __global float* y,
                          float alpha, long n,
                          __local float* tile) {
    int i = get_global_id(0);
    int li = get_local_id(0);
    if (i < n) {
        tile[li] = x[i];
        barrier(CLK_LOCAL_MEM_FENCE);
        y[i] = alpha * tile[li] + y[i];
    }
}
```

A `CL_DEVICE_LOCAL_MEM_SIZE`, a `CL_DEVICE_MAX_WORK_GROUP_SIZE` és a `CL_DEVICE_MAX_COMPUTE_UNITS` lekérdezések adják meg azokat a korlátokat, amelyekből a CUDA-hoz hasonlóan kiszámítható az occupancy.

JAX/XLA esetén a memóriaszintek nem közvetlenül vezérelhetők, de a HBM-pufferméretek és a sávszélesség mérése hasonló elven történik:

```
# JAX / XLA, Python 3.11; a pontos verziót a pip show jax jaxlib rögzíti
import jax
import jax.numpy as jnp

n = 1 << 24
x = jnp.ones((n,), dtype=jnp.float32)
y = jnp.ones((n,), dtype=jnp.float32)

# XLA-fuzionált művelet; a tényleges kernelt az HLO-dump mutatja
y = 2.0 * x + y
jax.block_until_ready(y)
```

A JAX-ban a shared memory és a regiszterfájl szintje a fordító belügye; a mérés ezért elsősorban HBM-sávszélességet és end-to-end időt rögzít.

3.3.9 7. Tipikus hibák és helyes értelmezés

1. **„A shared memory mindig gyorsít.”**
   Csak akkor igaz, ha az adatot többször használjuk, vagy ha a hozzáférési mintát a shared memory rendezettebbé teszi. Ha minden elemet egyszer olvasunk, a shared memory csak szinkronizációs költséget ad.

2. **„Elég a kapacitást nézni.”**
   A kapacitás mellett a késleltetés és a sávszélesség is meghatározó. Egy kernel lehet úgy is „kicsi”, hogy közben rengetegszer megy a HBM-hez.

3. **A shared memóriát használó ciklusban a `__syncthreads()` feltétel nélküli elhelyezése.**
   A `scale_shared_kernel`-ben a szinkronizáció csak azért biztonságos, mert a laborban `n` pontosan osztható a teljes szálszámmal. Általános esetben, ha egyes szálak hamarabb lépnek ki a ciklusból, a `__syncthreads()` deadlockot vagy definiálatlan viselkedést okozhat. A grid-stride loopoknál a közös szinkronizációt mindig egyenletes iterációszám mellett szabad csak használni.

4. **A regiszterhasználat egyetlen számként kezelése.**
   A kevés regiszter jó az occupancy-nak, de a túlzott spórolás spill-hez vezet. A `ptxas -v` kimenete és az `ncu` spill-metrikái nélkül ez nem dönthető el.

5. **A host-oldali pointerméret és a foglalt memória összekeverése.**
   A `sizeof(ptr)` a pointert, nem a lefoglalt tömböt méri. A memóriaigény becslésénél mindig a tömb elemszámából és az elemtípus méretéből kell kiindulni.

3.3.10 8. Önellenőrző kérdés

Egy kernel 256 szálból álló blockokat használ, szálanként 64 regisztert igényel, és blockonként 4 KiB shared memóriát foglal. Az adott SM-en 65 536 regiszter, 100 KiB felhasználható shared memory, legfeljebb 2048 szál és 32 block futtatható.

Számítsd ki a regiszter-, shared-, szál- és hardverkorlátokból adódó block- és szálszámot. Melyik erőforrás a szűk keresztmetszet?

Megoldás:

- Szálak: $2048/256 = 8$ block.
- Regiszterek: $65\,536/(64 \cdot 256) = 4$ block.
- Shared: $100/4 = 25$ block.
- Hardver: 32 block.

A szűk keresztmetszet a regiszterfájl: 4 block, azaz 1024 szál fér el. Az occupancy ezért $1024/2048 = 50\%$.

3.3.11 9. Összegzés

A GPU memóriahierarchiája a privát regiszterektől a block-szintű shared memórián, az SM-enkénti L1-en és a device-szintű L2-n át a HBM global memoryig terjed. A szintek felfelé haladva egyre nagyobb kapacitásúak, de egyre lassabbak. A kernel memóriaigényét ezért mindig több szinten kell megadni: regiszter per szál, shared per block, global forgalom per kernel, valamint az ezekből következő occupancy. A laborban bemutatott két vektorkernel azt is szemlélteti, hogy a shared memory alkalmazása önmagában nem gyorsít; csak akkor van értelme, ha valódi adat-újrafelhasználást vagy jobb hozzáférési mintát hoz létre.

A következő alfejezet (GPU-generációk és compute capability) ezekre a fogalmakra építve a GPU-generációkat és a compute capability-ket tárgyalja, hogy a memóriaszintek számait architektúrák szerint is el tudjuk helyezni.

3.4 GPU-generációk és compute capability

A GPU-hardver fejlődése nem lineáris sebességnövekedés, hanem új számítási és adatmozgatási képességek egymásra épülése. A szoftverfejlesztő számára a generációk közötti különbségek gyakorlati következményekkel járnak: más regiszterkészlet, új utasítások, eltérő Tensor Core-pontosságok, új szinkronizációs primitívek, illetve eltérő memória- és cache-viselkedés. Ebben a fejezetben a generációs különbségeket a **compute capability** fogalma köré szervezzük, majd megvizsgáljuk a Volta, Turing, Ampere, Hopper és Blackwell architektúrák főbb újításait. A fejezet végén egy olyan mini-labort építünk, amely CUDA C++-ban lekérdezi a tényleges eszköz tulajdonságait, és egy **feature- és architektúramátrixot** állít elő: a mátrixban elkülönítjük a dokumentációból származó várt jellemzőket a futó rendszer által mért/lekérdezett értékektől.

3.4.1 9.1 A compute capability fogalma és szerepe

A **compute capability** (röviden CC) egy `major.minor` formátumú verziószám, amelyet az NVIDIA a hardvergenerációk és azok programozható képességeinek címkézésére használ. Például a `9.0` a Hopper H100-ra, a `8.0` az Ampere A100-ra, a `7.0` a Volta V100-ra jellemző. A major verzió általában a nagy architekturális ugrást, a minor verzió pedig az azon belüli kisebb termékváltozatokat jelöli. A compute capability nem azonos a CUDA toolkit verziójával, és nem azonos a driver verziójával sem: a CUDA 12.x toolkittel fordíthatunk `sm_80`, `sm_86`, `sm_89` és `sm_90` célra is, de a futtatható bináris csak olyan eszközön indul el natívan, amelynek CC-je megegyezik a fordítási céléval, vagy amelyre a PTX-kód JIT-fordítása lehetséges.

A compute capability három dolgot rögzít:

1. **Hardver-erőforrás-korlátokat** – például a maximális blokkméret, a blokkonkénti regiszter- és shared memory mennyiség, a warpok száma SM-enként, a grid- és blokkdimenzionális limitek.
2. **Elérhető utasításokat és funkciókat** – például `cp.async`, Tensor Core utasítások, FP8-támogatás, thread block cluster, TMA, shuffle és vote variánsok.
3. **A legeneráltabb kernel- és könyvtárkód viselkedését** – bizonyos funkciók, például a cooperative groups vagy a redukciós primitívek viselkedése függ a CC-től, mert csak megfelelő hardveres támogatással futnak.

A CUDA-fordító az `-arch=sm_XY` kapcsolóval kapja meg a célhardvert. A `sm_` előtag a **streaming multiprocessor** architektúra célját jelöli. A `-gencode arch=compute_XY,code=sm_XY` alakkal egyidejűleg PTX-et és natív binárist is előállíthatunk. A PTX (**Parallel Thread Execution**) egy virtuális utasításkészlet, amelyet a driver a tényleges hardverre JIT-fordít. Ez teszi lehetővé a **forward compatibility**-t: ha egy újabb GPU nem rendelkezik a régi CC-re szánt natív cubinnal, de a csomagban van megfelelő PTX, a driver lefordítja azt az új hardverre, feltéve, hogy a CUDA driver és a PTX verziója támogatja az új CC-t.

A hordozhatóság szempontjából kritikus a minimális compute capability megválasztása. Ha a kód olyan utasítást vagy könyvtári függvényt használ, amely csak magasabb CC-n létezik, akkor az alacsonyabb CC-jű eszközön a kernel nem indul el, vagy hibásan fut. Ezt a kódban a `__CUDA_ARCH__` makróval kezelhetjük, amely a fordítási cél CC-jét adja meg százas alakban: `__CUDA_ARCH__ == 900` esetén Hopper, `__CUDA_ARCH__ >= 800` esetén Ampere vagy újabb, és így tovább.

3.4.2 9.2 Generációk áttekintése: Volta, Turing, Ampere, Hopper, Blackwell

Az alábbiakban a modern adatközponti és nagy teljesítményű GPU-generációkat vizsgáljuk. A korábbi generációk (Tesla, Fermi, Kepler, Maxwell, Pascal) alapvető fogalmai – SIMT, warp, SM, global/shared memory – már ismertek a korábbi fejezetekből, ezért itt a hangsúlyt a **Volta és az azt követő architektúrák** közötti különbségekre helyezzük.

3.4.3 9.2.1 Volta: az első Tensor Core és az önálló szálütemezés

A **Volta** architektúra 2017-ben jelent meg, legismertebb képviselője a Tesla V100. Compute capability értéke `7.0` (V100) és `7.2` (bizonyos változatok). A Volta három nagy újítást hozott a GPGPU számára:

- **Tensor Core**: az első generációs Tensor Core egységek mátrix-mátrix szorzásokat végeznek mixed precision módban. A V100 Tensor Core FP16 bemenettel és FP32 akkumulációval dolgozik, és képes 4×4×4-es mátrix-tile-okra szorzatösszeget számolni. Ez a mélytanulási tréning és következtetés számára ad radikális teljesítménynövekedést az FP32 CUDA core-okhoz képest.
- **Independent Thread Scheduling (ITS)**: a Volta előtt a warp egyetlen utasításszámlálót (program counter) használt, így a divergence során a különböző ágak csak sorosan hajtódtak végre. A Volta SM-ben minden szálnak saját program counter és call stack van, ami lehetővé teszi a divergens ágak finomabb, akár átlapoló ütemezését. Ez nem szünteti meg a warp-divergenciát, de csökkenti annak hatását, és lehetővé teszi a szálak szinkronizálását szétágazás után is.
- **FP32 és INT32 párhuzamos végrehajtás**: a Volta SM-ben az FP32 és INT32 egységek külön adatúttal rendelkeznek, így a cím- és ciklusszámítások nem foglalják el az FP32 egységeket. A V100 SM-enként 64 FP32, 64 INT32, 32 FP64 egység és 8 Tensor Core található.

A Volta a memóriahierarchiát is átalakította: a shared memory és az L1 cache továbbra is egyesített, de konfigurálható méretű, az L2 cache nagyobb, és a HBM2 memória sávszélessége jelentősen meghaladta a korábbi GDDR5X-et.

3.4.4 9.2.2 Turing: RT Core, egységes L1/shared és második generációs Tensor Core

A **Turing** architektúra 2018-ban jelent meg, képviselői a Tesla T4 (CC `7.5`) és a GeForce RTX 20 sorozat (CC `7.5`). A Turing elsősorban a grafikai ray tracinghez hozott **RT Core**-t, de a GPGPU szempontjából fontosabbak az alábbiak:

- **Második generációs Tensor Core**: a T4 Tensor Core már támogatja az INT8, INT4 és INT1 precíziós módokat is, nem csak FP16-ot. Ez a következtetési (inference) feladatoknál fontos, mert a kvantált hálózatok súlyai és aktivációi alacsonyabb bitszélességgel dolgoznak.
- **Egységes L1/shared memory architektúra**: a Turing SM-ben az L1 cache és a shared memory egyetlen fizikai SRAM-bankból gazdálkodik, és a felosztás aránya szoftveresen állítható. Ez lehetővé teszi, hogy a fejlesztő a kernel igényeihez igazítsa a shared memory méretét, vagy éppen több L1 cache-t használjon.
- **SM felépítés**: a Turing SM 64 FP32, 64 INT32, 4 Tensor Core és 1 RT Core egységet tartalmaz. A FP32 és INT32 továbbra is párhuzamosan dolgozhat, de a Tensor Core száma csökkent a Volta SM-hez képest, mert a Turing a fogyasztói és embedded szegmensre is optimalizált.

A Turing compute capability `7.5` értéke jelzi, hogy a hardveres feature-ök közel állnak a Volta-hoz, de a Tensor Core képességek bővültek, és az L1/shared felosztás rugalmasabb.

3.4.5 9.2.3 Ampere: TF32, BF16, sparse Tensor Core és a 2× FP32 ún. „A100 kontra GA10x" kettősség

Az **Ampere** generáció 2020-ban érkezett, és két fő termékcsaládra bomlott:

- **A100, compute capability `8.0`**: az adatközponti A100 SM-je 64 FP32, 64 FP64/2? Pontosabban az A100 SM: 64 FP32, 64 INT32, 32 FP64, 4 harmadik generációs Tensor Core. A legfontosabb újdonság a **TF32** (TensorFloat-32), amely 19 bites mantisszájú lebegőpontos formátum; a Tensor Core ezt használja mátrixszorzáshoz, és nagyjából 8–10-szeres teljesítményt ad az FP32-höz képest minimális pontosságvesztéssel. Emellett a **BF16** támogatás is megjelenik. Az A100 bevezette a **sparse Tensor Core**-t: a súlymátrixokban a 2:4 arányú strukturált ritkítást (sparsity) a hardver közvetlenül kihasználja, így a mátrixszorzás akár kétszeresére gyorsulhat.
- **GA10x, compute capability `8.6`**: a fogyasztói RTX 30 sorozat (pl. RTX 3090) SM-je eltér az A100-tól. A GA10x SM-ben 128 FP32 egység található, de ezek fele csak FP32, a másik fele FP32+INT32 kombinált. Ez a játékok és általános számítások számára nagyobb nyers FP32 teljesítményt ad, de a szigorú értelemben vett FP64 teljesítmény alacsonyabb. A Tensor Core itt is harmadik generációs, de a TF32 és BF16 támogatás mellett kisebb a sparse kihasználtság, mint az A100-ban.
- **Ada Lovelace, compute capability `8.9`**: 2022-ben megjelent fogyasztói/grafikus architektúra, amely az Ampere alapjaira épül, de negyedik generációs Tensor Core-t, FP8-támogatást és továbbfejlesztett RT Core-t hoz. Bár a fejezet fókusza a főbb generációk, az Ada azért fontos, mert a compute capability `8.9` sok újabb rendszerben előfordul.

Az Ampere generáció további újításai a **MIG** (Multi-Instance GPU) az A100-ban, amely lehetővé teszi a GPU fizikai partícionálását egymástól izolált számítási egységekre; a **cp.async** utasítás, amely lehetővé teszi a globális memóriából a shared memoryba történő aszinkron másolást; valamint a nagyobb L2 cache (A100: 40 MB).

3.4.6 9.2.4 Hopper: thread block cluster, TMA és FP8

A **Hopper** architektúra 2022-ben jelent meg, legismertebb képviselője a H100, compute capability `9.0`. A Hopper a GPGPU számára a következő nagy ugrást hozta:

- **Thread Block Cluster**: a Hopper bevezette a thread block cluster fogalmát, amely lehetővé teszi, hogy több blokk együttesen, hardveres támogatással működjön együtt. A cluster-en belül a blokkok elérhetik egymás shared memory-ját a **distributed shared memory** mechanizmussal, és a szinkronizáció is hardveresen gyorsabb. Ez a korábbi grid-szintű szinkronizációhoz képest sokkal alacsonyabb késleltetésű együttműködést biztosít.
- **Tensor Memory Accelerator (TMA)**: a TMA egy dedikált hardveregység, amely a globális memória és a shared memory közötti adatmozgatást végzi anélkül, hogy a szálaknak ki kellene adniuk a betöltő utasításokat. A TMA képes 2D/3D tiling-re, aszinkron másolásra és akár a shared memóriába érkező adatok automatikus elrendezésére. A `cp.async.bulk` utasításokkal programozható, és jelentősen csökkenti a memóriaműveletek regiszter- és utasításigényét.

- **Negyedik generációs Tensor Core és FP8**: a H100 Tensor Core-ja már támogatja az FP8 (E4M3 és E5M2) precíziót is, ami a nagy nyelvi modellek tréningjénél és következtetésénél további átbocsátóképesség-növekedést ad. A Hopper bevezette a **Transformer Engine** koncepciót, amely hardveres támogatást nyújt a transzformer-alapú modellek FP8 és FP16 közötti dinamikus skálázásához.
- **Új szinkronizációs és redukciós utasítások**: a Hopper SM-je támogatja a `cluster.sync`, `cluster.arrive`, `cluster.wait` utasításokat, valamint a `reduce.sync` és egyéb cluster-szintű műveleteket.

A H100 SM-je 128 FP32, 64 FP64, 4 Tensor Core (negyedik generációs) és számos dedikált egység. A memória HBM3, az L2 cache 50 MB, a sávszélesség meghaladja a 3 TB/s-ot.

3.4.7 9.2.5 Blackwell: FP4/FP6, második generációs Transformer Engine és továbbfejlesztett TMA

A **Blackwell** architektúra a 2024–2025-ös generáció. A compute capability az egyes termékcsaládoknál eltérő: a B100/B200 adatközponti GPU-k esetében a dokumentáció a `10.x` értékeket, a fogyasztói GeForce RTX 50 sorozatnál a `12.x` értékeket rögzíti – **ezeket a pontos verziószámokat a telepített CUDA toolkit és a gyártói dokumentáció alapján mindig ellenőrizni kell**, mert a könyv írásakor a friss kiadásokban változhatnak. A Blackwell legfontosabb GPGPU-szempontú újdonságai:

- **Ötödik generációs Tensor Core**: támogatja az FP4 és FP6 precíziós formátumokat is, amelyek a nagy nyelvi modellek következtetésénél további hatékonyságot ígérnek. Az FP4 már nagyon alacsony bitszélesség, ezért a numerikus stabilitás és a skálázás kritikus kérdés.
- **Második generációs Transformer Engine**: az FP8/FP4/FP6 közötti dinamikus váltást hardveresen támogatja, és a mátrixszorzásoknál precíziós skálázást végez.
- **Továbbfejlesztett TMA és async copy**: a Blackwell TMA-ja még nagyobb 2D/3D tile-okat, tömörített adatformátumokat (pl. FP8, INT8) és hatékonyabb shared memory elrendezést támogat.
- **Több-chiplet felépítés**: a nagy adatközponti Blackwell GPU-k két nagy számítási chipletből állnak, amelyeket egy nagy sebességű, koherens interconnect (NV-HBI) köt össze. A szoftver számára ez egyetlen CUDA eszközként jelenik meg, de a memória-koherencia és a sávszélesség tervezésekor figyelembe kell venni.
- **NVLink 5 és nagyobb hálózati integráció**: a Blackwell tovább növeli a GPU-k közötti sávszélességet, ami a több GPU-s tréning és következtetés számára fontos.

A Blackwell a fentieken túl is számos speciális gyorsítót tartalmaz, például decompressziós motorokat az adattömörítéshez. A GPGPU-fejlesztő számára a legfontosabb, hogy a `__CUDA_ARCH__ >= 1000` vagy `>= 1200` feltétellel tudja elkülöníteni a Blackwell-specifikus kódot, de a pontos makró-értéket a CUDA verzió dokumentációjához kell igazítani.

3.4.8 9.3 Architekturális különbségek mátrixa

A generációk közötti különbségeket érdemes egy táblázatban összefoglalni. Az alábbi mátrix a **várt**, azaz a dokumentáció alapján elvárható jellemzőket mutatja – ezt később a laborban a ténylegesen lekérdezett adatokkal vetjük össze.

Generáció	Compute capability (példa)	Tensor Core	Kiemelt FP-pontosságok	Új adatmozgatás/szinkronizáció	Megjegyzés
Volta (V100)	`7.0`, `7.2`	1. gen	FP16 + FP32 acc	–	Independent Thread Scheduling, FP32+INT32 párhuzamos
Turing (T4)	`7.5`	2. gen	FP16, INT8, INT4	–	Egységes L1/shared, RT Core grafikához
Ampere (A100)	`8.0`	3. gen	TF32, BF16, FP16, INT8	`cp.async`	Sparse Tensor Core, MIG
Ampere (GA10x)	`8.6`	3. gen	TF32, BF16, FP16	`cp.async`	2× FP32 egység a fogyasztói SM-ben
Ada Lovelace	`8.9`	4. gen	FP8, FP16, BF16	`cp.async`	Továbbfejlesztett RT Core
Hopper (H100)	`9.0`	4. gen	FP8, FP16, BF16, TF32	TMA, Thread Block Cluster, `cp.async.bulk`	Distributed shared memory, Transformer Engine
Blackwell	`10.x` / `12.x` (ellenőrzendő)	5. gen	FP4, FP6, FP8, FP16	TMA 2. gen, NVLink 5	Chiplet-felépítés, 2. gen Transformer Engine

A táblázatban a **Tensor Core generáció** és a **FP-pontosságok** közötti összefüggés jól mutatja, hogy a generációváltás nem pusztán gyorsabb órajelet, hanem új numerikus képességeket is jelent. A fejlesztőnek tudnia kell, hogy például az FP8 csak Hopper vagy újabb CC-n elérhető, a BF16 Ampere-től, a TF32 Ampere-től, az FP4/FP6 pedig Blackwell-től.

3.4.9 9.4 Hordozhatóság: fordítási célok, `__CUDA_ARCH__` és forward compatibility

A generációk közötti különbségek miatt a hordozható CUDA-kód írása két szinten történik:

1. **Fordítási célok kiválasztása.** A `-gencode arch=compute_70,code=sm_70 -gencode arch=compute_80,code=sm_80 -gencode arch=compute_90,code=sm_90` kapcsolókkal több CC-re is készíthetünk binárist. A `compute_XY` cél PTX-et, az `sm_XY` cél natív SASS-t generál. A gyakorlatban érdemes a leggyakrabban használt célokat lefedni, de a bináris mérete nő.
2. **Feltételes fordítás `__CUDA_ARCH__` makróval.** Ha egy kódrészlet csak egy adott CC-től érhető el, akkor azt `#if __CUDA_ARCH__ >= 800` blokkba tesszük. Például a `cp.async` utasítás Ampere-től érhető el, ezért:

```
__global__ void copy_kernel(const float* src, float* dst, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
#if __CUDA_ARCH__ >= 800
        // Ampere vagy újabb: aszinkron másolás shared memoryba lehetséges
        // Itt a példa csak jelzi a feltételt, nem teljes implementáció.
#endif
        dst[i] = src[i];
    }
}
```

A `__CUDA_ARCH__` csak az eszközkódban értelmezett; a hostkódban nem. A forward compatibility érdekében a PTX-et is be kell csomagolni: ha egy régebbi CC-re szánt bináris nincs jelen, de a PTX elérhető, a driver JIT-fordítja az új hardverre. Ez a mechanizmus nem teszi a régi kódot automatikusan optimálissá az új hardveren, de garantálja a futást.

A hordozhatóság másik pillére a **runtime lekérdezés**. A `cudaGetDeviceProperties` segítségével a program futás közben megtudhatja a CC-t, és ennek megfelelően választhat kernelváltozatot vagy könyvtári paramétert. Például:

```
int dev = 0;
cudaDeviceProp prop;
cudaGetDeviceProperties(&prop, dev);
std::cout << "Compute capability: " << prop.major << "." << prop.minor << "\n";

if (prop.major >= 9) {
    // Hopper vagy újabb: használhatunk Thread Block Cluster-t
} else if (prop.major >= 8) {
    // Ampere vagy újabb: használhatunk cp.async-ot
}
```

Fontos, hogy a runtime lekérdezés nem helyettesíti a feltételes fordítást: a kernelben az adott CC-re nem létező utasítás használata fordítási hibát okoz, ezért a hardver-specifikus utasításokat mindig `__CUDA_ARCH__` blokkba kell zárni.

3.4.10 9.5 Labor: feature- és architektúramátrix

A labor célja, hogy a fejlesztő saját környezetében ellenőrizze a generációs jellemzőket, és elkülönítse a **várt** (dokumentált) tulajdonságokat a **mért** (az API által jelentett vagy benchmarkkal nyert) adatoktól. A labort CUDA C++-ban valósítjuk meg, a könyvben rögzített hibakezelési szabályok szerint.

3.4.11 9.5.1 Eszközök és követelmények

- NVIDIA GPU bármelyik fenti generációból; a példában egy H100-at használunk, de bármely más eszköz megfelel.
- CUDA toolkit 12.x, driver 550 vagy újabb (a konkrét verziót a naplóban rögzíteni kell).
- Linux vagy Windows terminál; a buildparancs `nvcc`.
- A kódban minden CUDA API-hívást ellenőrzünk. A mérés előtt warm-up, majd legalább 5 ismétlés; az átlag mellett közöljük a szórást.

3.4.12 9.5.2 A program felépítése

A program két részből áll:

1. **Device query**: a `cudaGetDeviceProperties` segítségével lekérdezi a GPU nevét, CC-jét, SM-számát, maximális blokkméretét, regiszter- és shared memory korlátait, globális memória méretét, valamint a `major` és `minor` alapján besorolja a generációt.
2. **Feature-mátrix**: a program a CC-ből és a dokumentációból ismert generációs tulajdonságok alapján kiír egy táblázatot, amelyben a **várt** oszlop a dokumentált jellemző, a **mért/lekérdezett** oszlop pedig a futó rendszerből származó adat. Ahol az API közvetlenül nem ad értéket (pl. Tensor Core verzió, FP8-támogatás), ott a CC-ből következtetünk, és ezt a naplóban jelöljük.

A kód kiegészül egy egyszerű **memória-sávszélesség mikrobenchmarkkal**, hogy a mérési naplóban a várt csúcsérték és a mért teljesítmény is összevethető legyen. A benchmark egy nagy tömböt másol, és a `cudaEvents` időmérésével meghatározza az effektív sávszélességet.

3.4.13 9.5.3 CUDA C++ kód

```
// feature_matrix.cu
// Fordítás: nvcc -O3 -arch=sm_90 -o feature_matrix feature_matrix.cu
// Futtatás: ./feature_matrix

#include <cuda_runtime.h>
#include <iostream>
#include <iomanip>
#include <vector>
#include <cmath>
#include <cstdlib>

#define CHECK(call)                                                     \
    do {                                                                \
        cudaError_t err = call;                                         \
        if (err != cudaSuccess) {                                       \
            std::cerr << "CUDA error at " << __FILE__ << ":"          \
                      << __LINE__ << " -> "                            \
                      << cudaGetErrorString(err) << "\n";              \
            std::exit(EXIT_FAILURE);                                    \
        }                                                               \
    } while (0)

__global__ void copy_kernel(const float* src, float* dst, size_t n) {
    size_t i = blockIdx.x * (size_t)blockDim.x + threadIdx.x;
    if (i < n) {
        dst[i] = src[i];
    }
}

float benchmark_copy(size_t bytes, int num_iter = 10) {
    size_t n = bytes / sizeof(float);
    float *d_src, *d_dst;
    CHECK(cudaMalloc(&d_src, bytes));
    CHECK(cudaMalloc(&d_dst, bytes));

    // init
    CHECK(cudaMemset(d_src, 1, bytes));
    CHECK(cudaMemset(d_dst, 0, bytes));

    int threads = 256;
```

```
    int blocks = (int)((n + threads - 1) / threads);

    cudaEvent_t start, stop;
    CHECK(cudaEventCreate(&start));
    CHECK(cudaEventCreate(&stop));

    // Warm-up
    copy_kernel<<<blocks, threads>>>(d_src, d_dst, n);
    CHECK(cudaDeviceSynchronize());

    float total_ms = 0.0f;
    for (int iter = 0; iter < num_iter; ++iter) {
        CHECK(cudaEventRecord(start));
        copy_kernel<<<blocks, threads>>>(d_src, d_dst, n);
        CHECK(cudaEventRecord(stop));
        CHECK(cudaEventSynchronize(stop));
        float ms = 0.0f;
        CHECK(cudaEventElapsedTime(&ms, start, stop));
        total_ms += ms;
    }

    float avg_ms = total_ms / num_iter;
    // effektív sávszélesség: 2 * bytes / time (read + write)
    float bandwidth_gb_s = (2.0f * bytes / (avg_ms / 1000.0f)) / 1e9f;

    CHECK(cudaFree(d_src));
    CHECK(cudaFree(d_dst));
    CHECK(cudaEventDestroy(start));
    CHECK(cudaEventDestroy(stop));

    return bandwidth_gb_s;
}

const char* generation_name(int major, int minor) {
    if (major == 7 && minor == 0) return "Volta (V100)";
    if (major == 7 && minor == 5) return "Turing (T4/RTX 20)";
    if (major == 8 && minor == 0) return "Ampere (A100)";
    if (major == 8 && minor == 6) return "Ampere (GA10x)";
    if (major == 8 && minor == 9) return "Ada Lovelace";
    if (major == 9) return "Hopper (H100)";
    if (major == 10 || major == 12) return "Blackwell (ellenőrzésre szoruló CC)";
    return "Ismeretlen generáció";
}

int main() {
    int dev_count = 0;
    CHECK(cudaGetDeviceCount(&dev_count));
    if (dev_count == 0) {
        std::cerr << "Nincs elérhető CUDA eszköz.\n";
        return EXIT_FAILURE;
    }

    int dev = 0;
    cudaDeviceProp prop{};
    CHECK(cudaGetDeviceProperties(&prop, dev));

    std::cout << "=== Feature- és architektúramátrix ===\n";
    std::cout << "GPU: " << prop.name << "\n";
    std::cout << "Compute capability: " << prop.major << "." << prop.minor << "\n";
    std::cout << "Generáció (várt): " << generation_name(prop.major, prop.minor) << "\n";
    std::cout << "CUDA driver / runtime: " << cudaDriverGetVersion(NULL)
              << " / " << CUDART_VERSION << "\n\n";
```

```
    // Táblázat fejléc
    std::cout << std::left << std::setw(30) << "Jellemző"
              << std::setw(25) << "Várt (dokumentált)"
              << std::setw(25) << "Mért/lekérdezett" << "\n";
    std::cout << std::string(80, '-') << "\n";

    auto print_row = [](const char* feature, const std::string& expected,
                        const std::string& measured) {
        std::cout << std::left << std::setw(30) << feature
                  << std::setw(25) << expected
                  << std::setw(25) << measured << "\n";
    };

    // SM-ek száma
    print_row("SM-ek száma", "Specifikáció szerint", std::to_string(prop.multiProcessorCount));

    // Max threads per block
    print_row("Max threads/block", "Specifikáció szerint",
↪   std::to_string(prop.maxThreadsPerBlock));

    // Shared memory per block
    print_row("Shared mem/block", "Specifikáció szerint",
              std::to_string(prop.sharedMemPerBlock) + " B");

    // Regiszterek per block
    print_row("Regiszterek/block", "Specifikáció szerint",
              std::to_string(prop.regsPerBlock));

    // Globális memória
    print_row("Globális memória", "Specifikáció szerint",
              std::to_string(prop.totalGlobalMem / (1024*1024)) + " MiB");

    // Tensor Core verzió: nincs közvetlen API
    std::string tc_expected;
    if (prop.major == 7) tc_expected = "1. gen (FP16)";
    else if (prop.major == 8 && prop.minor <= 6) tc_expected = "3. gen (TF32/BF16)";
    else if (prop.major == 8 && prop.minor >= 9) tc_expected = "4. gen (FP8)";
    else if (prop.major == 9) tc_expected = "4. gen (FP8)";
    else if (prop.major >= 10) tc_expected = "5. gen (FP4/FP6)";
    else tc_expected = "Nem elérhető";
    print_row("Tensor Core", tc_expected, "CC-ből következtetett");

    // FP8 támogatás
    std::string fp8_expected = (prop.major >= 9) ? "Igen" : "Nem";
    print_row("FP8 támogatás", fp8_expected, "CC-ből következtetett");

    // Memória sávszélesség benchmark
    size_t bytes = 256 * 1024 * 1024; // 256 MiB
    float bw = benchmark_copy(bytes);
    print_row("Másolási sávszélesség", "Specifikáció szerinti csúcs",
              std::to_string(bw) + " GB/s");

    std::cout << "\nMegjegyzés: a „várt" oszlop a generáció dokumentált jellemzőit,\n"
              << "a „mért/lekérdezett" pedig a cudaGetDeviceProperties és a saját\n"
              << "benchmark eredményét mutatja. A Tensor Core/FP8 esetén nincs\n"
              << "közvetlen API, ezért a compute capability alapján következtettünk.\n";

    return EXIT_SUCCESS;
}
```

3.4.14 9.5.4 Build és futtatás

```
# Build parancs
nvcc -O3 -arch=sm_90 -o feature_matrix feature_matrix.cu

# Futtatás
./feature_matrix
```

Ha a GPU nem Hopper, az `-arch` értékét igazítsuk a tényleges CC-hez, például `-arch=sm_80` vagy `-arch=sm_86`. A build során a `-gencode` kapcsolóval több célra is fordíthatunk.

3.4.15 9.5.5 Mérési napló: várt vs. mért

Az alábbi napló egy **példa**, nem valós mérés; a konkrét értékek a helyi környezettől függenek. A labornaplóban mindig rögzíteni kell a környezetet:

Környezet:
GPU: NVIDIA H100 80GB HBM3
Driver: 550.54.15
CUDA: 12.4
OS: Ubuntu 22.04
Power limit: gyári
ECC: bekapcsolva
MIG: kikapcsolva
Háttérterhelés: nincs

Jellemző	Várt (dokumentált)	Mért/lekérdezett	Megjegyzés
Compute capability	`9.0`	`9.0`	API-ból
SM-ek száma	132	132	API-ból
Max threads/block	1024	1024	API-ból
Shared mem/block	228 KiB (max)	232448 B	API-ból
Regiszterek/block	65536	65536	API-ból
Globális memória	80 GB	81920 MiB	API-ból
Tensor Core	4. gen (FP8)	CC-ből következtetett	Nincs közvetlen API
FP8 támogatás	Igen	CC-ből következtetett	Nincs közvetlen API
Másolási sávszélesség	kb. 3 TB/s (HBM3 csúcs)	2.85 TB/s	Saját benchmark, 256 MiB, 10 ismétlés

A benchmarknál a várt csúcsérték a névleges HBM3 sávszélesség (3 TB/s körül), a mért pedig a saját másoló kernel effektív sávszélessége. A különbség normális, mert a másolás nem éri el a csúcsértéket, és az ECC, illetve az overhead is csökkenti. A naplóban a mért érték szórása például ± 0.05 TB/s lehet, ezt is rögzíteni kell.

3.4.16 9.5.6 Értelmezés és gyakori hibák

A laborban a legfontosabb tanulság, hogy a **várt** értékek forrása a gyártói specifikáció, a **mért** értékek pedig a tényleges futó rendszerből származnak. Ha a kettő eltér, annak oka lehet:

- A GPU nem pontosan az a modell, amelyre a specifikáció vonatkozik.
- A driver vagy a CUDA verzió nem támogatja teljesen az adott CC-t.

- A benchmark az adatmozgatás mellett más overheadet is mér (kernelindítás, warm-up hiánya, órajel-ingadozás).
- Az ECC vagy a power limit csökkenti a tényleges sávszélességet.

3.4.17 9.6 Gyakori hibák és elkerülésük

1. **A compute capability összekeverése a CUDA verzióval.** A CC a hardveré, a CUDA a szoftveré. Egy CUDA 12.4-es toolkit fordíthat `sm_70`, `sm_80`, `sm_90` célra is; a CUDA verzió nem határozza meg a hardvert.
2. **A minimális CC figyelmen kívül hagyása.** Ha a kód `cp.async`-ot vagy `cluster`-t használ, de a fordítási cél `sm_70`, a fordítás hibát ad, vagy a kernel nem fut. A `__CUDA_ARCH__` makróval kell védeni a hardver-specifikus részeket.
3. **A runtime lekérdezés elhagyása.** Ha a program csak a host-kódban feltételez egy CC-t, akkor másik GPU-n rossz kernelváltozatot választhat. Mindig kérdezzük le a `prop.major` és `prop.minor` értékét, és a kernelválasztást ez alapján végezzük.
4. **A Tensor Core automatikus kihasználásának feltételezése.** A Tensor Core-ok nem aktiválódnak maguktól: a könyvtárak (cuBLAS, cuDNN, CUTLASS) vagy a saját kernel WMMA/MMA utasításai szükségesek. A CC önmagában nem garantálja a gyorsulást.
5. **A forward compatibility túlértékelése.** A PTX JIT-fordítás lehetővé teszi a futást, de a régi kernel nem lesz optimális az új hardveren. A teljesítményhez az új CC-re optimalizált bináris vagy kernel kell.

3.4.18 9.7 Önellenőrző kérdés

Egy fejlesztő a következő kódot írja:

```
__global__ void my_kernel(float* a, const float* b, float* c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        c[i] = a[i] + b[i];
    }
}
```

A build parancs: `nvcc -arch=sm_70 -o app app.cu`. A fejlesztő ezután futtatja a programot egy H100 GPU-n (CC 9.0), és azt tapasztalja, hogy a kernel fut, de lassabb, mint egy `sm_90`-re fordított változat. Magyarázza meg, miért fut egyáltalán a `sm_70`-re fordított bináris, és miért lassabb a H100-on. Milyen buildelési stratégiát javasolna?

Válasz: Az `sm_70`-re fordított bináris azért fut, mert a CUDA csomag tartalmaz PTX-et is (ha a `-gencode` vagy az `-arch` ezt előállította), vagy mert a driver forward compatibility mechanizmusa JIT-fordítja a PTX-et a H100-ra. Viszont a JIT-fordított kód az eredeti Volta CC-re készült utasításváltozat, nem használja a Hopper új utasításait, memória-útvonalait és optimalizációit. A buildelési stratégia: `-gencode arch=compute_70,code=sm_70 -gencode arch=compute_90,code=sm_90` vagy legalább `-gencode arch=compute_90,code=sm_90`, hogy a H100-ra natív, optimalizált SASS készüljön, és a régebbi GPU-kra is legyen megfelelő bináris/PTX.

3.4.19 Összefoglalás

A compute capability a GPU-hardver generációs címkéje, amely egyszerre rögzíti az erőforráskorlátokat, az elérhető utasításokat és az optimalizálási lehetőségeket. A Volta óta minden generáció új mátrixszámítási képességeket és adatmozgatási primitíveket hoz: a Volta az első Tensor Core-t és az independent thread schedulingot, a Turing az INT8/INT4-et és az egységes L1/shared-t, az Ampere a TF32/BF16-ot és a cp.async-ot, a Hopper a thread block clustert és a TMA-t, a Blackwell pedig az FP4/FP6-ot és a továbbfejlesztett TMA-t. A hordozhatóság alapja a fordítási célok és a `__CUDA_ARCH__` makró helyes használata, valamint a futásidejű lekérdezés. A fejezet laborjában egy feature-mátrixot építettünk, amely elkülöníti a dokumentációból származó várt jellemzőket a rendszer által mért adatoktól, és bemutatja a generációk közötti különbségek gyakorlati ellenőrzését.

4 CUDA fejlesztőkörnyezet és az első kernel

4.1 A CUDA toolkit és a driver telepítése

A GPU-alapú fejlesztéshez két szoftverrétegre van szükség: egy eszközmeghajtóra, amely közvetlenül vezérli a hardvert, és egy fejlesztői eszközkészletre, amely a fordítót, a könyvtárakat és a vizsgálati eszközöket tartalmazza. Ebben az alfejezetben a **CUDA toolkit** és a **driver** szerepét, kompatibilitási kapcsolatát, telepítési módjait és ellenőrzését tárgyaljuk. A cél, hogy a fejlesztő egy olyan, reprodukálható környezetet tudjon kialakítani, amelyben a későbbi fejezetekben bemutatott kernelek fordíthatók, futtathatók és mérhetők.

4.1.1 1. A CUDA szoftververem rétegei

A CUDA szoftververem négy fő komponensből áll, és ezeket érdemes már a telepítés előtt megkülönböztetni:

1. **Kernel módú driver**: az operációs rendszer része, közvetlenül kezeli a GPU-t, ütemezi a végrehajtást, kezeli a memóriát és a megszakításokat. Linuxon a `nvidia.ko` kernelmodul, Windows alatt az `nvlddmkm.sys`. Ez a komponens a `nvidia-smi` által is látható, és a hardverrel való legalacsonyabb szintű kapcsolatot valósítja meg.
2. **User mode driver könyvtár**: a kernel módú driverhez tartozó, felhasználói térben futó könyvtár, Linuxon `libcuda.so`, Windows alatt `nvcuda.dll`. Ezt a CUDA **driver API** használja közvetlenül, és a CUDA **runtime API** is erre épül.
3. **CUDA runtime**: a `libcudart.so` (Linux) vagy `cudart.dll` (Windows) könyvtár, amely a magasabb szintű, kényelmesebb programozási felületet biztosítja. A C++ fejlesztők általában ezt használják: a kernelindítás `<<<...>>>` szintaxisa, a `cudaMalloc`, `cudaMemcpy`, `cudaDeviceSynchronize` és hasonló függvények mind a runtime API részei.
4. **CUDA toolkit**: a fejlesztői eszközkészlet, amely magában foglalja az `nvcc` fordítót, a CUDA fejléceket, a könyvtárakat (cuBLAS, cuFFT, cuSPARSE, cuRAND, Thrust, CUB stb.), a profilozó és hibakereső eszközöket (Nsight Systems, Nsight Compute, compute-sanitizer), valamint a dokumentációt és a mintaprogramokat. A toolkit verziója határozza meg, hogy milyen nyelvi elemeket és könyvtári API-kat használhatunk.

A telepítés során a driver és a toolkit **külön egységként** kezelendő. A drivert a hardverhez és az operációs rendszerhez kell igazítani, a toolkittel pedig a fejlesztői környezetet hozzuk létre. A kettő közötti kompatibilitást pontosan szabályozott verziómodell garantálja.

4.1.2 2. Driver és toolkit kompatibilitás

A CUDA-verziószámok három szintből állnak: **major**, **minor** és **patch**, például `12.2.1`. A major és minor verzió határozza meg az API-kompatibilitást, a patch csak hibajavításokat jelöl. A driver a telepített hardverhez tartozó, folyamatosan frissülő szoftver, míg a toolkit a fejlesztő által választott, projektfüggő verzió. Ezért a gyakorlatban a driver verziója gyakran magasabb, mint a toolkit verziója.

A CUDA kompatibilitási modell két fontos fogalmat vezet be:

- **Backward compatibility (visszafelé kompatibilitás):** egy adott driver képes futtatni a korábbi, azonos major verziójú toolkittel lefordított CUDA programokat. Például egy 12.x-es driver futtathat 12.1-es, 12.0-s vagy akár 11.8-as toolkittel készült kódot is, feltéve, hogy az adott hardver compute capability-jét a kód támogatja. A driver verziója általában nagyobb vagy egyenlő, mint a futtatott CUDA runtime verziója.
- **Minor version compatibility:** a CUDA 11.0-tól kezdve az azonos major verziószámon belül a toolkit által generált binárisok futtathatók újabb minor verziójú driverrel. Ez azt jelenti, hogy ha a fejlesztő 12.2-es toolkittel fordít, akkor a program futni fog egy 12.5-ös drivert használó gépen is, de nem garantált, hogy egy 13.0-s driverrel is működik. A major verzió váltása **nem** kompatibilis automatikusan; ilyenkor a programot újra kell fordítani, vagy a drivernek kell biztosítania a régebbi CUDA modulok betöltését.

A gyakorlatban a következő szabályt érdemes megjegyezni:

> A **CUDA runtime verzió** a futtatott program által igényelt API-szintet mutatja, a **CUDA driver verzió** pedig azt a maximális API-szintet, amelyet a telepített driver támogat. Ha a driver verziója kisebb, mint a runtime verziója, a futtatás `CUDA driver version is insufficient for CUDA runtime version` hibával meghiúsul.

A verziók lekérdezésére a CUDA runtime két függvényt biztosít:

```
// CUDA C++ (C++17, nvcc, Linux)
int runtimeVersion = 0;
int driverVersion  = 0;
cudaRuntimeGetVersion(&runtimeVersion);
cudaDriverGetVersion(&driverVersion);
```

Fontos: a `cudaDriverGetVersion` nem a kernel módú driver csomagjának verzióját adja vissza, hanem azt a legmagasabb CUDA runtime verziót, amelyet a driver támogat. A tényleges driver verziót az `nvidia-smi` vagy a `nvidia-settings` mutatja. A két érték közötti különbség gyakori félreértés forrása.

4.1.3 3. Telepítési módok és környezetválasztás

A CUDA-ökoszisztéma többféle telepítési utat kínál. A választás a fejlesztői céloktól, az izolációs igénytől és az adott infrastruktúrától függ. A három leggyakoribb megközelítés:

4.1.4 3.1. Natív telepítés

Linux rendszereken a driver telepítése a gyártó által kiadott `.run` fájllal vagy a disztribúció csomagkezelőjével történhet. A toolkit telepítése tipikusan csomagkezelővel (`apt`, `yum`, `pacman`) vagy a hivatalos `cuda_<verzió>_linux.run` telepítővel lehetséges. A natív telepítés előnye a közvetlen hardverhozzáférés és a legkisebb futásidőbeli overhead; hátránya, hogy a verziók ütközhetnek más projektekkel, és a környezet nehezen reprodukálható.

Egy tipikus Ubuntu-alapú telepítés vázlata (a pontos csomagneveket és verziókat mindig a telepítendő CUDA-verzióhoz kell igazítani):

```
# Bash (Ubuntu, natív telepítés, ellenőrizendő verziók)
sudo apt update
sudo apt install -y nvidia-driver-535
sudo apt install -y cuda-toolkit-12-2
```

A parancsokban szereplő verziók példaként értendők; a telepítés előtt ellenőrizni kell, hogy az adott disztribúció és kernel melyik drivert támogatja, és hogy a csomagkezelőben milyen CUDA-verziók érhetők el.

4.1.5 3.2. Konténeres környezet

Az NVIDIA Container Toolkit és a Docker/Podman használata a **reprodukálható** fejlesztői környezet legegyszerűbb módja. A konténerben a toolkit a konténerképen belül található, míg a driver a hoszt operációs rendszerben fut, és a konténer a szükséges user mode könyvtárakat (`libcuda.so`) a hosztról kapja. Ezért a konténerben **nem** kell drivert telepíteni; csak a hoszton kell megfelelő drivernek futnia.

A hivatalos, előre konfigurált képek az NVIDIA Container Registryből érhetők el, például:

```
# Bash (Docker, NVIDIA Container Toolkit szükséges a hoszton)
docker run --gpus all -it --rm nvcr.io/nvidia/cuda:12.2.0-devel-ubuntu22.04 bash
```

A kép nevében a `12.2.0` a CUDA toolkit verzióját, a `devel` a teljes fejlesztői változatot, az `ubuntu22.04` pedig az alap operációs rendszert jelöli. Léteznek `runtime` és `base` változatok is, amelyek kevesebb komponenst tartalmaznak, és elsősorban futtató környezetnek szánnak. Fejlesztéshez mindig a `devel` változatot érdemes használni, mert ez tartalmazza az `nvcc`-t és a fejléceket.

4.1.6 3.3. Conda környezet

A Conda csomagkezelővel a CUDA toolkit a felhasználói térbe telepíthető, anélkül, hogy a rendszerszintű csomagokat módosítanánk. Ez a megoldás különösen hasznos több CUDA-verzió párhuzamos használatánál vagy olyan szervereken, ahol nincs rendszergazdai jog. A környezet létrehozásához a `conda` parancs használható:

```
# Bash (Conda, ellenőrizendő csatorna- és csomagnevek)
conda create -n cuda-env -c nvidia cuda-toolkit=12.2 python=3.11
conda activate cuda-env
```

Fontos, hogy a Conda által telepített toolkit **nem tartalmazza a drivert**. A drivert továbbra is a hoszt operációs rendszeren kell telepíteni és frissen tartani. A Conda környezetben az `nvcc` és a könyvtárak a környezet `bin` és `lib` könyvtárában találhatók, így a `PATH` és `LD_LIBRARY_PATH` beállítását a Conda aktiválása automatikusan elvégzi.

4.1.7 3.4. Melyik módszert válasszunk?

A könyv laborjai **izolált, reprodukálható** környezetet igényelnek, ezért a konténeres vagy a Conda-alapú megközelítés javasolt. A konténer erős izolációt ad, de a hoszton engedélyezni kell a GPU-hozzáférést; a Conda kényelmesebb a meglévő rendszerhez, de megosztja a hoszt drivert. A natív telepítés csak akkor érdemes, ha a fejlesztő kizárólag egyetlen, jól kontrollált gépen dolgozik, és nincs szükség több CUDA-verzióra.

4.1.8 4. A fordító: nvcc

Az `nvcc` a CUDA C++ fordítómeghajtó. Szerepe kettős: a `.cu` kiterjesztésű forrásfájlokat felosztja **hoszt** (CPU) és **eszköz** (GPU) kódra, a GPU-ra szánt kódot lefordítja a céleszköz natív utasításkészletére, illetve PTX köztes kódra, a hoszt kódot pedig átadja a rendszer C++ fordítójának (Linuxon általában GCC, Windows alatt MSVC). Az `nvcc` ezután a hoszt fordító által készített objektumfájlokat és a GPU-kódot egyetlen futtatható állománnyá kapcsolja össze.

A leggyakoribb fordítási parancs:

```
# Bash (Linux, CUDA C++)
nvcc -arch=sm_80 -o device_query device_query.cu
```

A `-arch` kapcsoló a **compute capability**-t adja meg, ami meghatározza, hogy a GPU melyik architektúra-variánsához készül a kód. Például a `sm_80` az NVIDIA Ampere generáció (például A100) virtualizált architektúráját jelöli. A teljes lista az `nvcc --help` kimenetében található. A gyakorlatban érdemes a célhardver tényleges compute capability-jét használni: az `nvidia-smi` vagy a `deviceQuery` megmondja, mi az.

Az `nvcc` verziójának ellenőrzése:

```
nvcc --version
```

A kimenet tartalmazza a CUDA toolkit verzióját és a hoszt fordító verzióját. Például:

```
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2023 NVIDIA Corporation
Built on Tue_Jun_13_19:16:58_PDT_2023
Cuda compilation tools, release 12.2, V12.2.140
Build cuda_12.2.r12.2/compiler.33567101_0
```

Fontos, hogy a fejlesztő által használt `nvcc` verzió **nem feltétlenül egyezik meg** a driver által támogatott legmagasabb verzióval. A fordítás csak a toolkit verziójától függ; a futtatást a driver verziója korlátozza.

4.1.9 5. Az állapotlekérdező eszköz: nvidia-smi

Az `nvidia-smi` (NVIDIA System Management Interface) a GPU-k állapotának leggyorsabb ellenőrzésére szolgál. Alapértelmezés szerint kiírja a GPU modelljét, a driver verzióját, a CUDA-verziót, a foglalt memóriát, a hőmérsékletet és a futó folyamatokat.

Példa kimenet:

```
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 535.104.05             Driver Version: 535.104.05   CUDA Version: 12.2     |
|-----------------------------------------+----------------------+----------------------|
| GPU  Name                 Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |         Memory-Usage | GPU-Util  Compute M. |
|                                         |                      |               MIG M. |
|=========================================+======================+======================|
|   0  NVIDIA A100-SXM4-40GB          On  | 00000000:03:00.0 Off |                    0 |
| N/A   34C    P0              70W / 400W |      0MiB / 40960MiB |      0%      Default |
|                                         |                      |             Disabled |
+-----------------------------------------+----------------------+----------------------+
```

A kimenet tetején látható `Driver Version` a kernel módú driver tényleges verziója, a `CUDA Version` pedig az a legmagasabb CUDA-verzió, amelyet a driver támogat. A kettő nem ugyanaz. A táblázatban a GPU neve, a memória mérete, a hőmérséklet és a számítási mód is szerepel. A `nvidia-smi` a `--query-gpu` kapcsolóval géppel feldolgozható formában is lekérdezhető:

```
# Bash (Linux)
nvidia-smi --query-gpu=name,driver_version,compute_cap,memory.total --format=csv
```

A `compute_cap` mező itt a hardver compute capability-jét mutatja, például `8.0`.

4.1.10 6. deviceQuery: a hardver képességeinek lekérdezése

A CUDA Samples tartalmazza a `deviceQuery` nevű programot, amely a CUDA runtime API segítségével részletesen kiírja az elérhető GPU-k tulajdonságait. Ha a Samples nincs telepítve, a program egyszerűen megírható. A következő példa a verziók és a compute capability kiírására szolgál, a könyv hibakezelési konvencióinak megfelelően.

```
// CUDA C++ (C++17, nvcc, Linux)
// device_query.cu
#include <cuda_runtime.h>
#include <iostream>
#include <cstdlib>

#define CHECK_CUDA(call)                                                       \
    do {                                                                       \
        cudaError_t err = (call);                                              \
        if (err != cudaSuccess) {                                              \
            std::cerr << "CUDA error at " << __FILE__ << ":" << __LINE__       \
                      << " - " << cudaGetErrorString(err) << std::endl;        \
            std::exit(EXIT_FAILURE);                                           \
        }                                                                      \
    } while (0)

int main() {
    int deviceCount = 0;
    CHECK_CUDA(cudaGetDeviceCount(&deviceCount));
    if (deviceCount == 0) {
        std::cerr << "No CUDA-capable device detected." << std::endl;
        return EXIT_FAILURE;
    }
    std::cout << "Detected " << deviceCount << " CUDA-capable device(s)." << std::endl;

    int runtimeVersion = 0;
    int driverVersion = 0;
    CHECK_CUDA(cudaRuntimeGetVersion(&runtimeVersion));
    CHECK_CUDA(cudaDriverGetVersion(&driverVersion));

    std::cout << "CUDA Runtime version: " << runtimeVersion / 1000 << "."
```

```
              << (runtimeVersion % 1000) / 10 << std::endl;
    std::cout << "CUDA Driver version : " << driverVersion / 1000 << "."
              << (driverVersion % 1000) / 10 << std::endl;

    for (int i = 0; i < deviceCount; ++i) {
        cudaDeviceProp prop;
        CHECK_CUDA(cudaGetDeviceProperties(&prop, i));
        std::cout << "Device " << i << ": " << prop.name << std::endl;
        std::cout << "  Compute capability : " << prop.major << "." << prop.minor << std::endl;
        std::cout << "  Global memory      : " << prop.totalGlobalMem / (1024 * 1024)
                  << " MiB" << std::endl;
        std::cout << "  Multiprocessors    : " << prop.multiProcessorCount << std::endl;
        std::cout << "  Clock rate         : " << prop.clockRate / 1000 << " MHz" << std::endl;
    }
    return EXIT_SUCCESS;
}
```

A kód fordításához és futtatásához a következő parancsokat adjuk ki (a `-arch` értéke a céleszközhöz igazítandó):

```
# Bash (Linux, CUDA C++)
nvcc -arch=sm_80 -o device_query device_query.cu
./device_query
```

A program kimenete például:

```
Detected 1 CUDA-capable device(s).
CUDA Runtime version: 12.2
CUDA Driver version : 12.2
Device 0: NVIDIA A100-SXM4-40GB
  Compute capability : 8.0
  Global memory      : 40960 MiB
  Multiprocessors    : 108
  Clock rate         : 1410 MHz
```

A `cudaGetDeviceProperties` által visszaadott struktúra sok további mezőt tartalmaz, köztük a shared memory méretét blokkonként, a regiszterek számát, a warp méretét és a max blokkdimenziókat. A későbbi fejezetekben ezeket az értékeket használjuk az occupancy és az erőforrás-korlátok számításánál.

4.1.11 7. Labor: telepítési és verzióellenőrzési terv

A labor célja, hogy a fejlesztő a saját környezetében reprodukálható módon ellenőrizze a CUDA-verem működését, és rögzítse a mérési körülményeket. Az alábbi terv egy izolált konténeres környezetre épül, de natív vagy Conda telepítés esetén is alkalmazható a lépések logikája.

4.1.12 7.1. Kiindulási feltételek

- A hoszt gépen fut egy **engedélyezett**, NVIDIA GPU-t támogató driver.
- A konténer használatához telepítve van az **NVIDIA Container Toolkit**.
- A hálózati hozzáférés a konténerkép letöltéséhez biztosított.
- A kísérlet nem használ éles rendszert, hitelesítő adatot vagy személyes adatot.

4.1.13 7.2. Lépések

1. **Hoszt driver ellenőrzése** az `nvidia-smi` parancs segítségével. Jegyezzük fel a driver verzióját, a GPU típusát, a compute capabilityt és a memória méretét.

2. **Konténer indítása** a fejlesztői képpel:

   ```
   # Bash (Docker, NVIDIA Container Toolkit)
   docker run --gpus all -it --rm --name cuda-lab \
       -v $(pwd):/workspace \
       nvcr.io/nvidia/cuda:12.2.0-devel-ubuntu22.04 bash
   ```

   Ha a konténer nem tudja elérni a GPU-t, a `--gpus all` kapcsoló hiánya vagy a Container Toolkit hibája a leggyakoribb ok.

3. **Környezet ellenőrzése** a konténerben:

   ```
   # Bash (konténeren belül)
   nvidia-smi
   nvcc --version
   ```

   Az `nvidia-smi` kimenetében a driver verziója a hoszt driverét mutatja, mert a konténer a hoszt drivert használja.

4. **A deviceQuery példakód lefordítása és futtatása** a 6. szakaszban megadott forrásfájl segítségével. Fordítás előtt győződjünk meg arról, hogy a `-arch` értéke megegyezik a GPU compute capabilityjével. Ha nem ismerjük, az `nvidia-smi --query-gpu=compute_cap --format=csv,noheader` lekérdezésből kideríthető.

5. **Mérési napló készítése**: minden kimenetet mentsünk el szövegfájlba. A mérési napló a későbbi fejezetek reprodukálhatóságának alapja.

4.1.14 7.3. Mért és várt eredmények

A labor során a várt és a tényleges (mért) eredményt mindig külön kell választani. Az alábbi táblázat mutatja a tipikus elvárásokat; a tényleges értékeket a fejlesztő saját környezetében rögzíti.

Paraméter	Várt érték	Mért érték
GPU modell	A hoszt által biztosított, a laborhoz engedélyezett modell	pl. `NVIDIA A100-SXM4-40GB`
Driver verzió	>= a toolkit által igényelt minimum	pl. `535.104.05`
CUDA toolkit verzió (nvcc)	A képen belül rögzített, pl. `12.2`	
CUDA driver által támogatott verzió	>= a toolkit verziója	
Compute capability	A GPU tényleges értéke, pl. `8.0`	
Eszközök száma (`cudaGetDeviceCount`)	legalább 1	
`device_query` futtatási hibakód	0 (sikeres)	

`cudaGetLastError` a program végén	`cudaSuccess`

A mérési naplóban szerepelnie kell az operációs rendszernek, a kernel verziónak, a konténerkép nevének és a parancsok pontos alakjának is.

4.1.15 8. Tipikus hibák és elhárításuk

A telepítés és verzióellenőrzés során az alábbi hibák fordulnak elő leggyakrabban:

- **`CUDA driver version is insufficient for CUDA runtime version`**: a futtatni kívánt program nagyobb CUDA runtime verziót igényel, mint amit a hoszt driver támogat. Megoldás: frissíteni kell a drivert, vagy olyan toolkittel kell újrafordítani, amely a meglévő driverrel kompatibilis.
- **`No CUDA-capable device is detected`**: a `cudaGetDeviceCount` nulla eszközt ad vissza. Okok: a GPU nem engedélyezett a konténerben (`--gpus all` hiánya), a driver nem fut, vagy a GPU-t más folyamat foglalta le. Ellenőrizzük az `nvidia-smi` kimenetét.
- **`nvcc: command not found`**: a CUDA toolkit nem része a `PATH`-nak. Konténerben a `devel` kép használata szükséges; Conda környezetben aktiválni kell a környezetet; natív telepítésnél a `/usr/local/cuda/bin` hozzáadandó a `PATH`-hoz.
- **`error: identifier "cudaGetDeviceProperties" is undefined`**: a forrásfájlban nem szerepel a `#include <cuda_runtime.h>` vagy a linkelés nem tartalmazza a CUDA runtime könyvtárat. Az `nvcc` általában automatikusan kezeli a linkelést, de ha nem, akkor a `-lcudart` kapcsolót kell használni.
- **`cudaGetLastError` visszatér `cudaErrorInvalidDevice` hibával**: a futtatás során a megadott eszközazonosító érvénytelen. Általában a `cudaSetDevice` helytelen használatából adódik.

A hibakeresésnél mindig először a **legalacsonyabb szintű** ellenőrzést végezzük el: fut-e az `nvidia-smi`, látható-e a GPU, és mi a driver verziója. Ha a hardver nem látható, a szoftveres hibák keresése felesleges.

4.1.16 9. Összefoglalás

A CUDA fejlesztőkörnyezet két alapvető komponensből áll: a **driver** biztosítja a hardverhez való hozzáférést és a futtatókörnyezetet, a **toolkit** pedig a fordítót, a könyvtárakat és a fejlesztőeszközöket. A kettő közötti verziókompatibilitást a major/minor szabályok határozzák meg, és a `cudaRuntimeGetVersion` / `cudaDriverGetVersion` függvényekkel ellenőrizhető. A telepítéshez natív, konténeres vagy Conda-alapú utak közül választhatunk; a könyv laborjaihoz a konténeres izoláció javasolt. Az `nvidia-smi`, az `nvcc --version` és a saját `deviceQuery` program együttesen megbízható képet ad a rendszer állapotáról. A következő alfejezetben erre építve írjuk meg az első CUDA kernelt, és mérjük össze a CPU-s megvalósítással.

4.1.17 10. Önellenőrző kérdés

Magyarázza meg, mi a különbség a **CUDA driver verzió** és a **CUDA runtime verzió** között. Miért fordulhat elő, hogy egy program lefordul, de futtatáskor `CUDA driver version is insufficient` hibát ad? Milyen lépésekkel ellenőrizné egy adott gépen, hogy a rendszer képes-e CUDA programok futtatására?

4.2 Az első CUDA kernel: vektorösszeadás

A vektorösszeadás a CUDA programozás „Hello, world!" példája: kicsi, ellenőrizhető, mégis bemutatja a GPU-programozás alapvető szerkezetét. Amikor egy C++ fejlesztő először találkozik CUDA-val, a legfontosabb fogalmi váltás a host- és device-kód szétválasztása. Ebben az alfejezetben ezt a szétválasztást, a függvényminősítőket, a kernelindítás szintaxisát és a hibakezelés alapjait tanuljuk meg egy olyan példán, amely két lebegőpontos vektort ad össze elemenként. A cél nem csupán egy működő program elkészítése, hanem annak megértése, hogy miért éppen így épül fel a CUDA program, és milyen tipikus hibákra kell figyelni.

4.2.1 Host- és device-kód szétválasztása

A CUDA program két, egymástól jól elkülöníthető végrehajtási környezetből áll. A *host* a CPU-t és a hozzá tartozó rendszermemóriát jelenti, a *device* pedig a GPU-t és annak saját, a CPU-tól független memóriáját. A két környezet nem osztozik közvetlenül a memórián: a CPU nem tud egyszerű pointerrel hivatkozni a GPU globális memóriájára, és a GPU sem fér hozzá a rendszermemóriához anélkül, hogy a CUDA futásidő kifejezett adatmozgatást végezne. Ez a szétválasztás nem korlátozás, hanem a nagy teljesítmény egyik alapja: a GPU saját, nagy sávszélességű memóriáját a CPU memóriabuszától függetlenül éri el, és a magas késleltetést sok párhuzamos szál ütemezésével rejti el.

A forráskód szintjén a szétválasztást a függvények elé írt minősítők (qualifier) fejezik ki. A CUDA C++ három alapvető minősítőt ismer: `__global__`, `__device__` és `__host__`. A `__global__` minősítővel jelölt függvényt *kernelnek* nevezzük: ezt a host oldali kód hívja meg, de a device hajtja végre. A `__device__` minősítővel jelölt függvény csak a device-on futhat, és csak device-ról hívható; tipikusan a kernel által használt segédfüggvényeket jelöljük így. A `__host__` minősítő a CPU-n futó függvényt jelöli, és ez az alapértelmezés, ha nincs semmilyen minősítő. A `__host__` és `__device__` kombinációja olyan függvényt hoz létre, amelyet mind a host, mind a device fordítási egységben lefordít a fordító, így a függvény mindkét oldalon hívható. A `__global__` függvény visszatérési típusa mindig `void`, mert a kernel aszinkron módon fut, és a visszatérési érték átadása a host felé nem része a modellnek.

A szétválasztás következménye a felelősségi körök elkülönülése. A host kód feladata: a megfelelő CUDA eszköz kiválasztása, a device memória lefoglalása (`cudaMalloc`), a bemeneti adatok átmásolása a device memóriába (`cudaMemcpy`), a kernel indítása a megfelelő végrehajtási konfigurációval, a kernel befejezésének megvárása, az eredmény visszamásolása a host memóriába, végül a lefoglalt erőforrások felszabadítása. A device kód feladata ezzel szemben a párhuzamos végrehajtás: a kernel minden egyes szála kiszámítja a saját indexét, elvégzi a rábízott elemi műveletet, és a megfelelő memóriaterületre ír.

Fontos megérteni, hogy a host és a device memória nem ugyanaz. A `cudaMalloc` a device globális memóriájában foglal le helyet, és a visszaadott pointer csak a device oldalon érvényes. Ha a host kódban ezt a pointert próbáljuk dereferálni, a program hibásan viselkedik. Az adatok mozgatása a két memóriatér között kizárólag a CUDA futásidő API-ján keresztül

történik, például a `cudaMemcpy` hívással, amelynek irányát a `cudaMemcpyHostToDevice` vagy a `cudaMemcpyDeviceToHost` konstanssal adjuk meg.

4.2.2 A `__global__`, `__device__` és `__host__` minősítők

A három minősítő pontos szemantikáját a következő táblázat foglalja össze:

Minősítő	Futási hely	Hívható	Megjegyzés
`__global__`	device	host (és újabb CUDA verziókban device)	kernel, visszatérési típus `void`
`__device__`	device	device	segédfüggvény, automatikusan inline
`__host__`	host	host	alapértelmezés
`__host__ __device__`	host és device	host és device	mindkét oldalon lefordul

A `__global__` függvényt a host kód a speciális `<<<...>>>` szintaxissal indítja. A `__device__` függvényt a kernelben hívhatjuk, és a fordító általában beilleszti a hívás helyére, hogy elkerülje a hívási költséget. A `__host__ __device__` kombináció hasznos lehet közös, egyszerű matematikai segédfüggvényekhez, de ilyenkor a függvény nem tartalmazhat kernelindítást vagy device-specifikus memóriakezelést.

A vektorösszeadás kernelét `__global__` minősítővel jelöljük, mert a host indítja, de a device hajtja végre. A kernel belsejében nincs szükség külön `__device__` segédfüggvényre, mert az összeadás egyetlen elemi művelet. Érdemes azonban megjegyezni, hogy ha a kernelben többször ismétlődő, összetettebb műveletet végzünk, azt érdemes `__device__` függvénybe kiemelni, mert így a kód olvashatóbb és karbantarthatóbb lesz, a fordító pedig a hívás helyén inline beilleszti.

4.2.3 Kernel launch és szintaxis

A kernel indításának szintaxisa a CUDA C++ egyik legjellegzetesebb eleme:

```
kernel_neve<<<grid, block, shared_bytes, stream>>>(argumentumok);
```

A `grid` és a `block` a végrehajtási konfigurációt adja meg. A `grid` azt mondja meg, hány blokk fut a kernelből, a `block` pedig azt, hogy egy blokk hány szálat tartalmaz. Mindkettő lehet `dim3` típusú, amely három dimenziót (x, y, z) támogat, de skalár értékként megadva 1D konfigurációt kapunk. A `shared_bytes` opcionális, a dinamikus shared memória méretét adja meg bájtban; a `stream` szintén opcionális, és a CUDA streamet jelöli ki. Ebben a fejezetben a két opcionális paramétert elhagyjuk, így a legegyszerűbb alakot használjuk:

```
vectorAdd<<<gridSize, blockSize>>>(d_A, d_B, d_C, N);
```

A vektorösszeadás feladatában `N` elemű vektorokkal dolgozunk. A párhuzamosítás természetes módja, hogy minden szál egyetlen elemet számít ki: az `i`-edik szál kiolvassa `A[i]`-t és `B[i]`-t, összeadja őket, és az eredményt `C[i]`-be írja. Ahhoz, hogy a szálak tudják, melyik elemet kell feldolgozniuk, ki kell számítaniuk a globális indexüket. Mivel a végrehajtási modell blokkokba és szálakra oszlik, a globális index a blokk indexéből és a blokkon belüli szál indexéből áll elő:

```
int idx = blockIdx.x * blockDim.x + threadIdx.x;
```

Itt `blockIdx.x` az aktuális blokk sorszáma a gridben, `blockDim.x` a blokk mérete (szálak száma), `threadIdx.x` pedig az aktuális szál sorszáma a blokkon belül. Ha a blokkokat 1D-ben indítjuk, ez a képlet egyértelműen lefedi az összes szálat. Fontos, hogy ha `N` nem osztható a `blockSize` értékével, akkor az utolsó blokkban lehetnek olyan szálak, amelyek indexe már túlmutat a vektor határán. Ezért minden kernelben szükség van határellenőrzésre:

```
if (idx < N) {
    C[idx] = A[idx] + B[idx];
}
```

A launch konfigurációt a feladathoz kell igazítani. A `blockSize` tipikus értéke 128, 256 vagy 512, de a hardveres maximum 1024 szál blokkonként. A `gridSize` pedig a blokkok száma, amelyet úgy számítunk ki, hogy lefedje az összes elemet:

```
int gridSize = (N + blockSize - 1) / blockSize;
```

Ez az egész osztás felfelé kerekítése, így akkor is elegendő blokk indul, ha `N` nem többszöröse a `blockSize`-nak. A kernel indítása aszinkron: a host a `<<<...>>>` hívás után azonnal folytatja a végrehajtást, anélkül hogy megvárná a kernel befejezését. A szinkronizációról és a hibák ellenőrzéséről a következő szakaszban lesz szó.

A `dim3` típus lehetővé teszi 2D és 3D rácsok definiálását is, de ebben a fejezetben az egyszerűség kedvéért csak az 1D esetet használjuk. A későbbi fejezetekben látni fogjuk, hogy a többdimenziós blokkok és gridek hogyan teszik kényelmesebbé például a mátrixműveletek indexelését. A `blockDim.x`, `blockIdx.x` és `threadIdx.x` beépített változók a kernelben mindig elérhetőek, és az aktuális végrehajtási konfigurációt tükrözik.

4.2.4 Hibakezelés

A CUDA API hívások többsége `cudaError_t` típusú hibakódot ad vissza. A hibakód értéke `cudaSuccess`, ha a hívás sikeres volt, egyébként egy hibát azonosító konstans. A kernel indításakor azonban a `<<<...>>>` szintaxis nem ad vissza hibakódot, mert a launch maga aszinkron. A kernel indításakor fellépő hibákat két lépésben lehet ellenőrizni: először a `cudaGetLastError()` hívás adja vissza az indítás során keletkezett hibát, majd a `cudaDeviceSynchronize()` megvárja a kernel befejezését, és szintén hibakódot ad vissza, ha a kernel futása közben hiba történt. A `cudaDeviceSynchronize()` blokkoló hívás, tehát a host addig várakozik, amíg a device be nem fejezi az összes korábban kiadott műveletet.

A hibák ellenőrzését nem szabad kézzel, minden hívás után külön `if`-fel írni, mert ez hibalehetőséget és átláthatatlan kódot eredményez. A gyakorlatban egy saját hibamakrót használunk, amely a hívás eredményét ellenőrzi, és hiba esetén kiírja a fájlnevet, a sorszámot és a hiba szöveges leírását, majd leállítja a programot. A következő makró a CUDA C++ projektekben széles körben használt minta:

```
#define CUDA_CHECK(call)                                                    \
  do {                                                                      \
    cudaError_t err = call;                                                 \
    if (err != cudaSuccess) {                                               \
      std::cerr << "CUDA error at " << __FILE__ << ":" << __LINE__          \
                << " - " << cudaGetErrorString(err) << std::endl;           \
      std::exit(EXIT_FAILURE);                                              \
    }                                                                       \
  } while (0)
```

A makrót minden CUDA API hívásnál alkalmazzuk, például:

```
CUDA_CHECK(cudaMalloc(&d_A, N * sizeof(float)));
CUDA_CHECK(cudaMemcpy(d_A, h_A.data(), N * sizeof(float), cudaMemcpyHostToDevice));
```

A kernel indítás után pedig a következő két sort kell mindig elhelyezni:

```
CUDA_CHECK(cudaGetLastError());
CUDA_CHECK(cudaDeviceSynchronize());
```

Ez a minta biztosítja, hogy a hibás kernelindítás vagy a kernel futása közben fellépő illegális memória-hozzáférés ne maradjon észrevétlen. A hibás kernel ugyanis akár az eszköz leállását is okozhatja, ezért a fejlesztés során különösen fontos a hibák azonnali észlelése. A `cudaGetLastError` önmagában nem elég, mert az aszinkron kernel futása közben keletkező hibát csak a `cudaDeviceSynchronize` után látjuk; ezért mindkettőre szükség van.

4.2.5 Vektorösszeadás példakód

Az alábbi kód a fenti elemeket egy teljes, fordítható programmá állítja össze. A program két `N = 1024` elemű FP32 vektort tölt fel, átmásolja őket a device memóriába, elindítja a vektorösszeadó kernelt, visszamásolja az eredményt, és ellenőrzi, hogy az eredmény a várt értékekkel egyezik. A nyelv CUDA C++ (C++17), a fordító `nvcc`; a pontos verziót a laborban rögzíteni kell.

```
// CUDA C++ (C++17), nvcc 12.4 (ellenőrizendő)
#include <cuda_runtime.h>
#include <iostream>
#include <vector>
#include <cmath>

#define CUDA_CHECK(call)                                                    \
  do {                                                                      \
    cudaError_t err = call;                                                 \
    if (err != cudaSuccess) {                                               \
      std::cerr << "CUDA error at " << __FILE__ << ":" << __LINE__          \
                << " - " << cudaGetErrorString(err) << std::endl;           \
      std::exit(EXIT_FAILURE);                                              \
    }                                                                       \
  } while (0)

__global__ void vectorAdd(const float* A, const float* B, float* C, int N) {
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  if (idx < N) {
    C[idx] = A[idx] + B[idx];
  }
}

int main() {
  const int N = 1024;
  const int blockSize = 256;
  const int gridSize = (N + blockSize - 1) / blockSize;  // gridSize = 4

  std::vector<float> h_A(N), h_B(N), h_C(N);
  for (int i = 0; i < N; ++i) {
    h_A[i] = static_cast<float>(i);
    h_B[i] = static_cast<float>(2 * i);
  }

  float *d_A = nullptr, *d_B = nullptr, *d_C = nullptr;
  CUDA_CHECK(cudaMalloc(&d_A, N * sizeof(float)));   // global memory
  CUDA_CHECK(cudaMalloc(&d_B, N * sizeof(float)));   // global memory
  CUDA_CHECK(cudaMalloc(&d_C, N * sizeof(float)));   // global memory
```

```
  CUDA_CHECK(cudaMemcpy(d_A, h_A.data(), N * sizeof(float), cudaMemcpyHostToDevice));
  CUDA_CHECK(cudaMemcpy(d_B, h_B.data(), N * sizeof(float), cudaMemcpyHostToDevice));

  vectorAdd<<<gridSize, blockSize>>>(d_A, d_B, d_C, N);
  CUDA_CHECK(cudaGetLastError());
  CUDA_CHECK(cudaDeviceSynchronize());

  CUDA_CHECK(cudaMemcpy(h_C.data(), d_C, N * sizeof(float), cudaMemcpyDeviceToHost));

  bool ok = true;
  for (int i = 0; i < N; ++i) {
    float expected = h_A[i] + h_B[i];
    if (std::fabs(h_C[i] - expected) > 1e-5f) {
      std::cerr << "Mismatch at " << i << ": " << h_C[i] << " vs " << expected << "\n";
      ok = false;
      break;
    }
  }
  std::cout << (ok ? "PASSED" : "FAILED") << std::endl;

  CUDA_CHECK(cudaFree(d_A));
  CUDA_CHECK(cudaFree(d_B));
  CUDA_CHECK(cudaFree(d_C));
  CUDA_CHECK(cudaDeviceReset());
  return ok ? 0 : 1;
}
```

A kódban a `h_` előtaggal jelölt pointerek a host memóriára, a `d_` előtaggal jelöltek a device memóriára mutatnak. A `cudaMalloc` a device globális memóriájában foglal le területet, a `cudaMemcpy` pedig a host és a device közötti adatmozgatást végzi. A kernelindítás után a `cudaGetLastError` és a `cudaDeviceSynchronize` biztosítja a hibák észlelését. Az ellenőrzés a CPU-n történik, FP32 pontossággal, `1e-5f` tűréshatárral.

4.2.6 Labor: vektorösszeadó kernel implementálása

A labor célja a fenti vektorösszeadó kernel lefordítása, futtatása és mérése egy valós CUDA környezetben. A laborhoz rögzíteni kell a környezet paramétereit. Az alábbiakban egy példa környezetet adunk meg, de a tényleges laborban a saját értékeket kell feljegyezni:

- GPU modell: NVIDIA GeForce RTX 3060 (compute capability 8.6)
- Driver verzió: 550.54.14
- CUDA verzió: 12.4
- Operációs rendszer: Ubuntu 22.04 LTS
- Power limit: 170 W (alapértelmezett)
- ECC: kikapcsolva
- MIG: kikapcsolva
- Háttérterhelés: nincs

A build parancs a következő:

```
nvcc -O3 -arch=sm_86 -o vectorAdd vectorAdd.cu
```

Az `-arch=sm_86` a compute capability 8.6-os architektúrához igazított kódot generál. Ha a GPU más compute capability-val rendelkezik, az értéket ennek megfelelően kell módosítani; a pontos értéket a `nvidia-smi` vagy a `deviceQuery` segítségével lehet lekérdezni.

A futtatási parancs:

```
./vectorAdd
```

A mérési naplóhoz a kernel futási idejét CUDA eseményekkel vagy `nsys` profilerrel mérjük. A mérési protokoll: először 5 melegítő (warm-up) futtatás, majd 10 mérési futtatás. Az eredményekből átlagot és szórást számítunk. A következő táblázat egy példa mérési naplót mutat (az értékek illusztratívak, nem ellenőrzöttek):

Mérési pont	Érték
Bemeneti méret N	1024
FP precision	FP32
Block méret	256
Grid méret	4
Warm-up futtatások	5
Mérési futtatások	10
Átlagos kernel futásidő	0,012 ms
Szórás	0,001 ms
Elérhető sávszélesség	85 GB/s
Maximális hiba	0,0 (FP32)
Regiszterhasználat	16 regiszter/szál
Shared memory	0 bájt

A táblázatban a futásidő és a sávszélesség a mért eredmény, míg a hiba a várt matematikai eredménytől való eltérés. A várt eredmény definíciója: `C[i] = A[i] + B[i]` minden `i`-re, a bemeneti vektorok alapján. A mért eredmény a kernel kimenete, amelyet a CPU-ra visszamásolás után ellenőrzünk. A kettőt mindig külön kell kezelni: a várt eredmény a specifikáció, a mért eredmény a tényleges futás kimenete.

A laborban a mérési naplót a saját rendszeren kell rögzíteni, és a mért értékeket nem szabad a fenti illusztratív számokkal helyettesíteni. A mérést izolált környezetben végezzük, hogy a háttérterhelés ne torzítsa az eredményt. A sávszélesség számításához a tényleges adatmozgatást vesszük figyelembe: a kernel beolvassa az `A` és `B` vektorokat, és kiírja a `C` vektort, így a teljes adatforgalom `3 * N * sizeof(float)` bájt. Az átlagos futásidőből a sávszélesség egyszerűen számítható.

4.2.7 Tipikus hibák és helyes megoldás

Az első CUDA kernel írásakor számos tipikus hiba fordul elő. Az alábbiakban a leggyakoribbakat és a helyes megoldásukat soroljuk fel.

1. **Hiányzó vagy rossz `cudaMalloc`**: ha a device memóriát nem foglaljuk le, vagy a pointert rosszul adjuk át, a kernel illegális memória-hozzáférést végez. A megoldás: minden device oldali tömbhöz `cudaMalloc` hívás, és a pointereket `nullptr`-re inicializáljuk.
2. **Nincs határellenőrzés a kernelben**: ha `N` nem osztható a `blockSize` értékével, az utolsó blokk egyes szálai a vektor határán túlra írnak vagy olvasnak. A megoldás a `if (idx < N)` feltétel használata.
3. **Hibás globális indexszámítás**: a `threadIdx.x` és a `blockIdx.x` felcserélése vagy a `blockDim.x` elhagyása rossz indexhez vezet. A helyes képlet: `idx = blockIdx.x * blockDim.x + threadIdx.x`.

4. **A kernel indítás hibájának figyelmen kívül hagyása**: a `<<<...>>>` szintaxis nem ad vissza hibakódot, ezért ha nem hívjuk meg a `cudaGetLastError`-t és a `cudaDeviceSynchronize`-ot, a hiba rejtve marad. A megoldás a fenti hibamakró következetes használata.

5. **Túl nagy blokkméret**: a hardveres maximum blokkonként 1024 szál. Ha ennél nagyobbat adunk meg, a kernel indítás hibát ad. A megoldás: a `blockSize` értékét 1024 alatt tartani, és a `cudaGetLastError` ellenőrzi.

6. **Host- és device-mutatók keverése**: a device memóriára mutató pointert host kódban közvetlenül dereferálni, vagy a host pointert a kernelnek átadni súlyos hiba. A megoldás a `h_` és `d_` előtagok következetes használata, valamint a `cudaMemcpy` a megfelelő irányokkal.

7. **A `cudaFree` elhagyása**: a lefoglalt device memóriát fel kell szabadítani, különben erőforrás-szivárgás lép fel. A megoldás a `cudaFree` hívások párosítása a `cudaMalloc`-kal, valamint a `cudaDeviceReset` a program végén.

8. **A kernelben `printf` használata**: bár a CUDA támogatja a `printf`-et a kernelben, ez lassú és nem minden környezetben működik megbízhatóan. A megoldás: a kernelben kerüljük a kiírást, és az eredményt a host oldalon ellenőrizzük.

9. **A `cudaMemcpy` irányának eltévesztése**: ha a `cudaMemcpyHostToDevice` helyett `cudaMemcpyDeviceToHost`-ot használunk (vagy fordítva), a program hibásan fut. A megoldás: mindig gondoljuk át, hogy melyik memóriából melyikbe másolunk, és a megfelelő konstanst használjuk.

10. **A kernel függvény nem `__global__`**: ha a kernelként használt függvényt elfelejtjük `__global__` minősítővel ellátni, a fordítási hiba üzenete nem mindig egyértelmű. A megoldás: a kernel definíciójában mindig szerepeljen a `__global__` minősítő.

A fenti hibák mindegyike elkerülhető, ha a fejezetben bemutatott vektorösszeadó mintát követjük, és a hibakezelő makrót minden CUDA API hívásnál alkalmazzuk.

4.2.8 Összefoglalás és továbbvezetés

Ebben az alfejezetben bevezettük a CUDA programozás alapvető építőelemeit: a host- és device-kód szétválasztását, a `__global__`, `__device__` és `__host__` minősítőket, a kernelindítás `<<<grid, block>>>` szintaxisát, valamint a hibakezelés alapvető mintáját. A vektorösszeadás példáján keresztül láttuk, hogyan lehet egy egyszerű elemenkénti műveletet párhuzamosítani, és hogyan ellenőrizzük az eredményt.

A következő alfejezet (Build, futtatás és egyszerű hibakeresés) ezekre az alapokra épít: részletesen bemutatja a build folyamatot, a futtatást és az egyszerű hibakeresést, beleértve a gyakori fordítási hibákat és azok elhárítását. A vektorösszeadó kernel a későbbi fejezetekben is visszatér, amikor a memóriaterek, a coalescing és a teljesítményoptimalizálás kerül terítékre. A most megtanult minta – a host- és device-kód elkülönítése, a globális indexszámítás, a határellenőrzés és a hibakezelés – minden további CUDA program alapját képezi.

4.3 Build, futtatás és egyszerű hibakeresés

Az első CUDA kernel: vektorösszeadás alfejezetben elkészített vektorösszeadó kernel most belép abba a fázisba, ahol a forrásból futtatható program lesz, és ahol a hibák már nem fordítási, hanem futásidejű formában jelentkeznek. Ez az alfejezet a nvcc buildfolyamatát, a leggyakoribb

fordítási és futásidejű hibákat, valamint a CUDA-hibakezelés alapvető eszközeit tárgyalja. A cél, hogy az olvasó képes legyen egy CUDA C++ programot megbízhatóan lefordítani CMake-kel és Makefile-lal, futtatni, majd a fellépő hibákat szisztematikusan értelmezni és kijavítani.

4.3.1 A nvcc buildfolyamata

Az `nvcc` a CUDA fordító-illesztőprogramja. Nem egyszerű C++ fordító, hanem egy olyan illesztőréteg, amely a `.cu` kiterjesztésű forrásfájlokat szétválasztja host- és device-oldali kódra, lefordítja mindkettőt a megfelelő háttérfordítóval, majd összefűzi a részeket. A folyamat fő lépései a következők:

1. **Előfeldolgozás és szétválasztás.** Az `nvcc` először a preprocessor szabályai szerint feldolgozza a forrást, majd a CUDA frontend szétválasztja a host C++ kódot és a device kódot. A `__global__`, `__device__` és `__host__` annotációval ellátott függvények a device-oldalra kerülnek.
2. **Device kód fordítása.** A device kódot az `nvcc` először köztes reprezentációvá, jellemzően PTX-szintű kóddá alakítja, majd az adott célarchitektúrához (compute capability) tartozó natív SASS utasításokra fordítja. A PTX egy virtuális ISA, a SASS pedig a hardveren ténylegesen végrehajtott gépi kód. A fordítási lépés során a regiszterhasználat, a shared memory méret és a szálak közötti műveletek már rögzülnek a natív binárisban.
3. **Fatbin összeállítás.** Az `nvcc` a natív SASS-t és opcionálisan a PTX-et egy úgynevezett fatbin („fat binary") szakaszba csomagolja. Ha a futtatható csak natív kódot tartalmaz, akkor a futtató GPU-nak pontosan illeszkednie kell. Ha PTX is bekerül, a CUDA driver a betöltéskor JIT-fordíthatja azt egy újabb GPU-generációhoz, feltéve, hogy a driver és a toolkit támogatja az adott compute capabilityt.
4. **Host kód fordítása és linkelés.** A host-oldali C++ kódot az `nvcc` a háttérben lévő C++ fordítónak adja át. A CUDA runtime-hívásokat (például `cudaMalloc`, `cudaMemcpy`, `cudaLaunchKernel`) a `libcudart` runtime könyvtár biztosítja. Az `nvcc` gondoskodik a szükséges CUDA-könyvtárak linkeléséről is.

Az `nvcc` tipikus parancsa:

```
nvcc -arch=sm_80 -O3 -std=c++17 -o vectorAdd src/vectorAdd.cu
```

Ebben a parancsban a `-arch=sm_80` a compute capability 8.0-ás GPU-ra történő natív fordítást jelenti. A `-O3` a host- és device-oldali optimalizációt kapcsolja be, a `-std=c++17` pedig a C++ szabványverziót rögzíti.

A folyamat ellenőrzése a `-Xptxas -v` kapcsolóval is lehetséges, amely a ptxas assembler részletes kimenetét írja ki, például a regiszterhasználatot és a shared memory méretet. Ez a teljesítményelemzéshez később alapvető, de már most érdemes megismerni:

```
nvcc -arch=sm_80 -O3 -Xptxas -v -o vectorAdd src/vectorAdd.cu
```

A kiírásban olyan sorok jelennek meg, mint:

```
ptxas info : Used 16 registers, 0 bytes smem, 400 bytes cmem[0]
```

Az ilyen adatokat a későbbi fejezetekben a foglaltság (occupancy) és regiszternyomás elemzéséhez használjuk fel.

4.3.2 Architektúraflag és optimalizáció

A CUDA fordítás szempontjából a legfontosabb döntés az architektúraflag megválasztása. A `-arch=sm_XX` egy rövidítés, amely egy adott compute capability-höz tartozó natív SASS kódot generál, és általában egyetlen `-gencode arch=compute_XX,code=sm_XX` bejegyzést hoz létre. Ha a futtatás során a GPU nem pontosan ezt a generációt képviseli, a betöltés nem sikerül, és a tipikus hibakód a `cudaErrorNoKernelImageForDevice`.

A hordozhatóság növelésére gyakran használják a PTX-et is tartalmazó formát:

```
nvcc -gencode arch=compute_80,code=sm_80 \
     -gencode arch=compute_80,code=compute_80 \
     -o vectorAdd src/vectorAdd.cu
```

Itt az első bejegyzés natív SASS-t hoz létre a 8.0-ás architektúrához, a második pedig PTX-et ágyaz be, amelyet a driver egy későbbi, kompatibilis GPU-ra JIT-fordíthat. A PTX-szintű forward compatibility azonban nem korlátlan: a driverek és a toolkit-verziók támogatási mátrixát mindig ellenőrizni kell. A jelen írás időpontjában elterjedt 12.x-es toolkitekhez tartozó támogatott architektúrák listáját a telepített `nvcc --help` kimenete vagy a hivatalos CUDA Compatibility dokumentum tartalmazza, ezért a konkrét verziók ellenőrzést igényelnek.

A teljesítmény szempontjából a gyakori kapcsolók a következők:

- `-O3`: host- és device-oldali optimalizáció.
- `-use_fast_math`: gyors, de kevésbé pontos lebegőpontos műveleteket engedélyez. Csak akkor szabad használni, ha a numerikus követelmények azt megengedik. A lebegőpontos pontossági kérdéseket a Lebegőpontos pontosság alfejezet részletezi.
- `-lineinfo`: a hibakereséshez szükséges sorinformációkat ad a natív kódhoz.
- `-G`: a device kódot debug módban fordítja, és leállítja a legtöbb optimalizációt. Ez a `cuda-gdb` használatához szükséges, de a teljesítményt drasztikusan csökkenti.
- `--resource-usage`: a lefordított kernel erőforrás-használatát mutatja.
- `--ptxas-options=-v`: a ptxas részletes kimenetét engedélyezi.

Fontos megjegyezni, hogy a `-use_fast_math` nem egyenértékű a `-O3`-mal; az előbbi a matematikai függvények közelítését és az IEEE-kompatibilitás lazítását is jelenti. A laborban csak akkor használjuk, ha a feladat kifejezetten engedélyezi a csökkentett pontosságot.

4.3.3 Futásidejű hibák és a CUDA hibamodell

A CUDA API-hívások többsége szinkron, és a visszatérési értékük `cudaError_t` típusú hibakód. A kernelindítás azonban aszinkron: a launch visszatérése nem jelenti azt, hogy a kernel befejeződött. Emiatt a futásidejű hibák két csoportba sorolhatók:

1. **Launch-konfigurációs hibák**, amelyek már a kernelindításkor jelentkeznek. Ilyenek például a túl nagy grid- vagy blockméret, az érvénytelen shared memory igény, vagy a nem támogatott architektúrához tartozó bináris. Ezeket a launch után közvetlenül hívott `cudaGetLastError()` adja vissza.
2. **A kernel futása közben fellépő hibák**, például érvénytelen memóriaelérés (`illegal address`), osztás nullával, vagy a kernel futásidejű megszakadása. Ezek a hibák aszinkron módon keletkeznek, és az első olyan szinkronizáló hívás visszatérési értékében jelennek meg, mint a `cudaDeviceSynchronize`, `cudaStreamSynchronize`, vagy akár egy következő `cudaMemcpy`.

A `cudaGetLastError()` az utolsó, még el nem olvasott hibakódot adja vissza, és közben törli azt. A `cudaPeekAtLastError()` ugyanezt adja vissza, de nem törli. Ha nincs hiba, mindkettő `cudaSuccess` értéket ad. A hibakód szöveges leírását a `cudaGetErrorString(err)` függvény adja.

A leggyakoribb hibakódok és jelentésük:

Hibakód	Jellemző ok
`cudaSuccess`	Nincs hiba.
`cudaErrorInvalidValue`	Érvénytelen paraméter, például rossz méret vagy null mutató.
`cudaErrorMemoryAllocation`	A `cudaMalloc` nem tudott memóriát foglalni.
`cudaErrorInvalidConfiguration`	A kernelugrás konfigurációja érvénytelen, például a blockméret meghaladja a maximumot.
`cudaErrorIllegalAddress`	A kernel érvénytelen memóriaterületre hivatkozott.
`cudaErrorLaunchFailure`	A kernel futása közben súlyos hiba lépett fel.
`cudaErrorNoKernelImageForDevice`	A futtatható nem tartalmaz a GPU-hoz illeszkedő kernelképet.

A táblázat nem teljes körű, de a Build, futtatás és egyszerű hibakeresés szempontjából a fenti esetek a leggyakoribbak. A részletes hibakezelést a CUDA-hibakódok és ellenőrzés alfejezet tárgyalja.

4.3.4 Hibamakró használata

A CUDA programokban a hibakezelés ismétlődő, ezért a gyakorlatban makrót érdemes használni. A könyv konvenciója szerint minden CUDA API-hívás hibáját ellenőrizni kell; a `cudaGetLastError`, a `cudaDeviceSynchronize` és a saját hibamakró használata kötelező.

A következő makró a host API-hívások szinkron hibáit ellenőrzi:

```
#include <cstdio>
#include <cstdlib>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err = (call);                                           \
        if (err != cudaSuccess) {                                           \
            std::fprintf(stderr,                                            \
                         "CUDA hiba: %s (%s:%d)\n",                         \
                         cudaGetErrorString(err), __FILE__, __LINE__);      \
            std::exit(EXIT_FAILURE);                                        \
        }                                                                   \
    } while (0)
```

A makrót a következőképpen használjuk:

```
CUDA_CHECK(cudaMalloc(&d_a, n * sizeof(float)));
CUDA_CHECK(cudaMemcpy(d_a, h_a.data(), n * sizeof(float),
                      cudaMemcpyHostToDevice));
```

A kernelindítás esetében két lépés szükséges:

```
vectorAdd<<<gridSize, blockSize>>>(d_a, d_b, d_c, n);
CUDA_CHECK(cudaGetLastError());          // launch-konfigurációs hibák
CUDA_CHECK(cudaDeviceSynchronize());     // a kernel futás közbeni hibák
```

A `cudaGetLastError()` a launch utáni azonnali hívás. Ha a kernel futása közben történik hiba, azt a `cudaDeviceSynchronize()` visszatérési értéke jelzi. A kettő nem helyettesíthető egymással, mert a `cudaGetLastError()` nem várja meg a kernel befejeződését.

A hibamakró kiegészíthető a kernel launch ellenőrzésére is, de a fenti két sor a legátláthatóbb. A CUDA-hibakódok és ellenőrzés alfejezetben összetettebb hibakezelési mintákat is bemutatunk.

4.3.5 A vektorösszeadás teljes forráskódja

Az első CUDA kernel: vektorösszeadás alfejezetben már felépített vektorösszeadó kernel itt egy ellenőrizhető, hibakezelt formában jelenik meg. A forrás a `src/vectorAdd.cu` fájlban helyezhető el. A kód a CUDA C++17 szabványt használja.

```
// CUDA C++17
// vektorAdd.cu - gpu012 labor
#include <cstdio>
#include <cstdlib>
#include <vector>
#include <cmath>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err = (call);                                           \
        if (err != cudaSuccess) {                                           \
            std::fprintf(stderr,                                            \
                         "CUDA hiba: %s (%s:%d)\n",                         \
                         cudaGetErrorString(err), __FILE__, __LINE__);      \
            std::exit(EXIT_FAILURE);                                        \
        }                                                                   \
    } while (0)

// 1D vektorösszeadás; a határellenőrzés megakadályozza a túlindexelést.
__global__ void vectorAdd(const float* __restrict__ d_a,
                          const float* __restrict__ d_b,
                          float* __restrict__ d_c,
                          int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        d_c[idx] = d_a[idx] + d_b[idx];
    }
}

int main(int argc, char** argv) {
    int n = 1 << 20;                    // alapértelmezett: 1 048 576 elem
    if (argc > 1) {
        n = std::atoi(argv[1]);
    }

    const int blockSize = 256;
    const int gridSize  = (n + blockSize - 1) / blockSize;

    // Host oldali adatok
    std::vector<float> h_a(n, 1.0f);
    std::vector<float> h_b(n, 2.0f);
    std::vector<float> h_c(n, -1.0f);
```

```
    // Device oldali adatok - global memória
    float *d_a = nullptr, *d_b = nullptr, *d_c = nullptr;
    CUDA_CHECK(cudaMalloc(&d_a, n * sizeof(float)));
    CUDA_CHECK(cudaMalloc(&d_b, n * sizeof(float)));
    CUDA_CHECK(cudaMalloc(&d_c, n * sizeof(float)));

    // Adatmozgatás host -> device
    CUDA_CHECK(cudaMemcpy(d_a, h_a.data(), n * sizeof(float),
                          cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_b, h_b.data(), n * sizeof(float),
                          cudaMemcpyHostToDevice));

    // Kernelindítás <<<grid, block, shared_bytes, stream>>>
    vectorAdd<<<gridSize, blockSize>>>(d_a, d_b, d_c, n);
    CUDA_CHECK(cudaGetLastError());          // launch hiba ellenőrzése
    CUDA_CHECK(cudaDeviceSynchronize());     // futás közbeni hiba ellenőrzése

    // Eredmény visszamásolása device -> host
    CUDA_CHECK(cudaMemcpy(h_c.data(), d_c, n * sizeof(float),
                          cudaMemcpyDeviceToHost));

    // Ellenőrzés: várt érték 3.0f, mert 1.0 + 2.0
    float maxError = 0.0f;
    for (int i = 0; i < n; ++i) {
        float expected = 3.0f;
        maxError = std::max(maxError, std::fabs(h_c[i] - expected));
    }

    if (maxError == 0.0f) {
        std::printf("OK: n=%d, maxError=0.0\n", n);
    } else {
        std::fprintf(stderr, "Hiba: maxError=%f\n", maxError);
        CUDA_CHECK(cudaFree(d_a));
        CUDA_CHECK(cudaFree(d_b));
        CUDA_CHECK(cudaFree(d_c));
        return EXIT_FAILURE;
    }

    CUDA_CHECK(cudaFree(d_a));
    CUDA_CHECK(cudaFree(d_b));
    CUDA_CHECK(cudaFree(d_c));
    return EXIT_SUCCESS;
}
```

A kódban a `d_` prefix a device-mutatókat, a `h_` prefix a host-mutatókat jelöli. A `__restrict__` kulcsszó a fordító számára jelzi, hogy a mutatók nem fednek át, ami lehetővé teszi a jobb optimalizációt. A határellenőrzés `if (idx < n)` miatt akkor is biztonságos a kernel, ha a grid mérete nem pontosan osztható a block mérettel.

4.3.6 Build CMake-kel

A CMake a legelterjedtebb platformfüggetlen buildrendszer a CUDA projektekben. A modern CMake (3.18-tól kezdődően) natívan támogatja a CUDA nyelvet. Az alábbi `CMakeLists.txt` a labor kiindulópontja:

```
cmake_minimum_required(VERSION 3.22)
project(gpu012_lab LANGUAGES CXX CUDA)

set(CMAKE_CUDA_STANDARD 17)
set(CMAKE_CUDA_STANDARD_REQUIRED ON)
```

```
# A cél GPU compute capability. Ezt a tényleges hardverhez igazítsuk.
# Példa: NVIDIA A100 (sm_80). A CMake ebből generál megfelelő -gencode-ot.
set(CMAKE_CUDA_ARCHITECTURES 80)

add_executable(vectorAdd src/vectorAdd.cu)

# Release módban optimalizálás, debug módban a -G kapcsoló.
target_compile_options(vectorAdd PRIVATE
    $<$<CONFIG:Release>:-O3>
    $<$<CONFIG:Debug>:-G -lineinfo>
)
```

A build futtatása:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build
./build/vectorAdd 1000000
```

A `CMAKE_CUDA_ARCHITECTURES` értékét a valós GPU compute capabilityje alapján kell megadni. Ha például az eszköz egy RTX 3090, az `86` érték; ha V100, akkor `70`. A pontos értéket a `deviceQuery` mintaprogram vagy az `nvidia-smi --query-gpu=compute_cap --format=csv` parancs adja meg. A könyv konvenciója szerint a mérési naplóban rögzíteni kell a compute capabilityt.

4.3.7 Build Makefile-lal

A Makefile-alapú build közvetlenebb, és jól szemlélteti az `nvcc` hívását. A labor számára elegendő egy egyszerű változat:

```
NVCC      ?= nvcc
CUDA_ARCH ?= sm_80
CXXFLAGS  ?= -O3 -std=c++17
NVFLAGS   ?= -arch=$(CUDA_ARCH) $(CXXFLAGS)
LDFLAGS   ?= -lcudart

TARGET    := vectorAdd
SRC       := src/vectorAdd.cu

.PHONY: all run clean

all: $(TARGET)

$(TARGET): $(SRC)
    $(NVCC) $(NVFLAGS) -o $@ $< $(LDFLAGS)

run: $(TARGET)
    ./$(TARGET) 1000000

clean:
    rm -f $(TARGET)
```

A futtatás:

```
make all
make run
```

A Makefile-ban a `CUDA_ARCH` változó teszi könnyen cserélhetővé a célt. A laborban a GPU típusát és compute capabilityjét mindig dokumentálni kell. A `nvcc --version` és az `nvidia-smi` kimenete a mérési napló része.

Ha a kernel fordítása során regiszterhasználatot vagy más ptxas-adatot akarunk látni, a Makefile `NVFLAGS` sorához hozzáadhatjuk a `-Xptxas -v` kapcsolót:

```
NVFLAGS ?= -arch=$(CUDA_ARCH) $(CXXFLAGS) -Xptxas -v
```

4.3.8 Futtatás és környezet ellenőrzése

A fordítás után a program futtatása előtt érdemes ellenőrizni, hogy a CUDA driver és a GPU elérhető-e. A legalapvetőbb parancsok:

```
nvidia-smi
nvcc --version
```

Az `nvidia-smi` kiírja a driver verzióját, a GPU típusát, a kihasználtságot és a hőmérsékletet. A `nvcc --version` a CUDA toolkit verzióját mutatja. A labor dokumentációjában rögzíteni kell ezeket az adatokat.

A futtatható program a parancssori argumentumban kapja a vektor hosszát:

```
./vectorAdd 1000000
```

A várt kimenet:

```
OK: n=1000000, maxError=0.0
```

Ha a kimenetben hibaüzenet jelenik meg, a hibakód és a szöveg alapján kell elindítani a hibakeresést.

4.3.9 Tipikus futásidejű hibák és megoldásuk

A labor során gyakran találkozunk a következő esetekkel. A cél, hogy az olvasó felismerje a hibát, és megindokolja a javítást.

1. Érvénytelen kernelkonfiguráció (`cudaErrorInvalidConfiguration`)

Ha a block mérete meghaladja a hardver által megengedett maximumot (általában 1024 szál blokkonként), a kernelindítás érvénytelen. A következő kódrészlet szándékosan hibás:

```
// HIBÁS MINTA: a block mérete 1025, ami meghaladja a maximumot.
vectorAdd<<<1, 1025>>>(d_a, d_b, d_c, n);
CUDA_CHECK(cudaGetLastError());
CUDA_CHECK(cudaDeviceSynchronize());
```

A `cudaGetLastError()` ilyenkor `cudaErrorInvalidConfiguration` hibát ad. A helyes megoldás a block méretének 1024 alá csökkentése, és a grid méretének megfelelő kiszámítása:

```
int blockSize = 256;
int gridSize = (n + blockSize - 1) / blockSize;
vectorAdd<<<gridSize, blockSize>>>(d_a, d_b, d_c, n);
CUDA_CHECK(cudaGetLastError());
CUDA_CHECK(cudaDeviceSynchronize());
```

2. Nem illeszkedő kernelkép (`cudaErrorNoKernelImageForDevice`)

Ha a programot `-arch=sm_80` kapcsolóval fordították, de a GPU compute capabilityje 7.5, a betöltés sikertelen. A hibaüzenet:

```
CUDA hiba: no kernel image is available for execution on the device
```

A megoldás: a fordításnál a tényleges architektúrához illeszkedő `-arch` vagy `-gencode` kapcsolót kell használni. Ha hordozhatóságra van szükség, PTX-et is tartalmazó `-gencode arch=compute_XX,code=compute_XX` bejegyzést kell hozzáadni.

3. Érvénytelen memóriaelérés (`cudaErrorIllegalAddress`)

Ha a kernel olyan globális memóriaterületre ír vagy olvas, amely nem lett lefoglalva, a futás közbeni hiba a `cudaDeviceSynchronize()` visszatérési értékében jelenik meg. Például ha a `d_c` mutató `nullptr`, vagy a `n` paraméter nagyobb a lefoglalt méretnél. A hiba megelőzése érdekében a kernelben mindig szerepeljen határellenőrzés, a host oldalon pedig a lefoglalt memória mérete pontosan egyezzen a használt mérettel.

4. Kernel futás közbeni leállás (`cudaErrorLaunchFailure`)

Ha a kernelben például osztás nullával vagy egyéb váratlan kivétel történik, a CUDA runtime a `cudaErrorLaunchFailure` hibát adja. Az ilyen hibák felderítéséhez debug buildet (`-G -lineinfo`) és `cuda-gdb`-t vagy compute-sanitizert érdemes használni. A compute-sanitizer részletes bemutatása a Compute-sanitizer: memcheck és racecheck alfejezetben történik.

A hibás minták mindegyikénél fontos, hogy a program ne „nyelje le” a hibát. A fenti hibamakró nélküli kód például a következőképpen nézne ki:

```
// HIBÁS MINTA: nincs hibakezelés.
cudaMalloc(&d_a, n * sizeof(float));
kernel<<<grid, block>>>(...);
cudaMemcpy(...);
```

Ilyenkor a hibák rejtve maradhatnak, és a program látszólag működik, de az eredmény hibás. A könyv konvenciója szerint ez a minta kerülendő.

4.3.10 Debug build és gyors ellenőrzés

A futásidejű hibák felderítésének első lépése a debug build. A `-G` kapcsoló a device kódot debug módban fordítja, a `-lineinfo` pedig a sorinformációkat adja hozzá. A debug módú futtatás lassabb, de a hibakereső (például `cuda-gdb`) számára elérhetővé teszi a töréspontokat és a változókat.

CMake-ben a debug build a `CMAKE_BUILD_TYPE=Debug` beállítással érhető el, feltéve, hogy a `CMakeLists.txt` tartalmazza a fenti `target_compile_options` sorokat. A debug futtatás:

```
cmake -S . -B build-debug -DCMAKE_BUILD_TYPE=Debug
cmake --build build-debug
./build-debug/vectorAdd 1000000
```

A kernelben elhelyezett `printf` hívás is hasznos, mert a CUDA runtime a device-oldali `printf` kimenetét a host konzolra irányítja. Az ilyen nyomkövetés azonban csak korlátozottan használható nagy szálszám esetén, mert a kimenet összefolyhat. A `printf` a compute capability 2.0-tól támogatott, de a teljesítményét és a kimeneti sorrendet nem garantálja.

4.3.11 Labor: CMake- és Makefile-alapú CUDA-build

A labor célja, hogy a fenti vektorösszeadó programot mind CMake-kel, mind Makefile-lal lefordítsa és futtassa. Az átadandó eredmény egy mérési napló, amely elkülöníti a mért eredményt a vártól.

4.3.12 Laborfeladat

1. Hozzon létre egy `gpu012_lab/` könyvtárat, benne a `src/vectorAdd.cu` forrást.
2. Készítse el a fenti `CMakeLists.txt` és `Makefile` fájlokat.
3. Fordítsa le a programot mindkét buildrendszerrel.
4. Futtassa a programot `n = 1000000` elemszámmal.
5. Ellenőrizze, hogy a kimenet `OK: n=1000000, maxError=0.0`.
6. Rögzítse a környezetet: `nvidia-smi`, `nvcc --version`, `cmake --version`, `gcc --version`.
7. Szándékosan idézzen elő egy futásidejű hibát: változtassa a block méretet 1025-re, és figyelje meg a `cudaErrorInvalidConfiguration` hibát.
8. Javítsa a hibát, és futtassa újra.

4.3.13 Labor dokumentáció

A mérési naplóban az alábbi adatokat kell rögzíteni, a könyv konvenciója szerint.

Környezeti adat	Érték
GPU modell	pl. NVIDIA A100 40GB (helyőrző)
Driver verzió	pl. 550.54.15 (ellenőrizni: `nvidia-smi`)
CUDA Toolkit	pl. 12.4 (ellenőrizni: `nvcc --version`)
Compute capability	pl. 8.0 (ellenőrizni: `nvidia-smi --query-gpu=compute_cap --format=csv`)
Operációs rendszer	pl. Ubuntu 22.04
Buildrendszer	CMake, Makefile

A mért és a várt eredményt külön táblázatban kell feltüntetni:

Mérés	Várt	Mért	Értékelés
Vektor hossza (n)	1 000 000	1 000 000	egyezik
`maxError`	0.0	0.0	egyezik
Kernel launch hiba	`cudaSuccess`	`cudaSuccess`	egyezik
Szándékos rossz block méret	`cudaErrorInvalidConfiguration`	`cudaErrorInvalidConfiguration`	egyezik

A laborban a futásidőt nem értékeljük, mert a CPU–GPU összehasonlítás az első mérés: CPU vs. GPU alfejezet feladata. A dokumentációban szerepeltethető a futás időtartama, de ez itt nem teljesítménymérés.

4.3.14 A labor forráskódja

A forráskód megegyezik a korábban bemutatott `vectorAdd.cu` fájllal. A CMakeLists és a Makefile a fenti sablonokból származik. A futtatási parancsok:

```
# CMake build
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build
./build/vectorAdd 1000000
```

```
# Makefile build
make all
make run
```

A labor során a hibás mintát külön kell jelölni. A bátor olvasó a `vectorAdd.cu` fájlban átmenetileg módosíthatja a block méretet `1025`-re, lefordíthatja és futtathatja. A program a `CUDA_CHECK(cudaGetLastError())` sorban hibát jelez, és a stderr-re írja a `cudaErrorInvalidConfiguration` üzenetet. Ez a minta a könyvben veszélyesnek minősül, mert a hibás kernel nem okoz eszközleállást, de a hibakezelés nélkül a program hibásan futna tovább. A javítás után a block méretét vissza kell állítani, és a programot újra le kell futtatni.

4.3.15 Összefoglalás

A Build, futtatás és egyszerű hibakeresés alfejezetben áttekintettük az `nvcc` buildfolyamatát: a host- és device kód szétválasztását, a PTX/SASS fordítást, valamint a fatbin összeállítását. Megismertük az architektúraflag szerepét, a `-arch` és `-gencode` kapcsolók közötti különbséget, és a leggyakoribb optimalizációs lehetőségeket. A futásidejű hibák két csoportba sorolhatók: a launch-konfigurációs hibák, amelyeket a kernelindítás után közvetlenül hívott `cudaGetLastError()` ad vissza, és a kernel futása közben keletkező hibák, amelyeket a `cudaDeviceSynchronize()` jelenti. A hibamakró használata biztosítja, hogy egyetlen CUDA-hiba se maradjon észrevétlen.

A laborban egy vektorösszeadó programot építettünk fel CMake-kel és Makefile-lal, rögzítettük a környezetet, és szándékosan előidéztünk egy futásidejű hibát. Ezzel a Build, futtatás és egyszerű hibakeresés tanulási céljai teljesültek: az olvasó képes az `nvcc` buildfolyamatát értelmezni, a futásidejű hibákat felismerni, és a helyes megoldást megindokolni.

4.3.16 Önellenőrző kérdés

Mi a különbség a kernelindítás után közvetlenül hívott `cudaGetLastError()` és a `cudaDeviceSynchronize()` által visszaadott hiba között? Indokolja meg, miért nem elegendő csak az egyiket használni.

4.4 Az első mérés: CPU vs. GPU

A teljesítménymérés első lépése nem az, hogy megnyomjuk a stopperórát, hanem az, hogy pontosan megfogalmazzuk, mit mérünk, milyen eszközzel, milyen környezetben, és a kapott számot hogyan lehet reprodukálni. A CPU–GPU összehasonlításban a leggyakoribb hiba, hogy a fejlesztő egyetlen lefutást néz, vagy a GPU kernel idejét a CPU-s szakaszokkal összemossa. Ez a mérés többet mond a mérési hibákról, mint a hardverről.

Ebben az alfejezetben felépítünk egy olyan alapmérést, amely:

- elkülöníti a CPU- és a GPU-időt,
- warm-upot és többszöri ismétlést alkalmaz,
- statisztikát számol,
- rögzíti a hardver- és szoftverkörnyezetet,
- a mért és a várt eredményt szétválasztja.

A cél nem egyetlen „gyorsabb” kijelentés, hanem egy ellenőrizhető mérési minta, amely a későbbi optimalizálási fejezetek alapjává válik.

4.4.1 1. Mit jelent az alapvető teljesítménymérés?

Az alapvető teljesítménymérés azt jelenti, hogy adott bemenetre, adott pontossággal, adott hardver- és szoftverkörnyezetben rögzítjük a végrehajtási időt, és abból származtatott mennyiségeket számolunk. A vektorösszeadás példájában a feladat:

$$C[i] = A[i] + B[i], \qquad i = 0, \dots, N-1.$$

Ennek a műveletnek a mérhető költsége legalább három részből áll:

1. **Adatmozgatás**: az A és B tömbök eljuttatása a számítási egységhez, majd a C eredmény visszaírása.
2. **Számítás**: az N darab lebegőpontos összeadás.
3. **Vezérlési és ütemezési költség**: ciklus a CPU-n, illetve kernelindítás és szinkronizáció a GPU-n.

A GPU-időt ezért nem szabad csak a kernel futásidejére szűkíteni. Az első mérésben azt tanítjuk meg, hogyan lehet a teljes csővezetéket vagy annak részeit külön mérni, és mikor melyik adat mit jelent.

4.4.2 1.1. Mérőszámok

A nyers futásidő mellett a gyakorlatban két mérőszámot érdemes kiszámolni:

- **Sávszélesség-kihasználás** a memóriakorlátos kernelnél. Ha a kernel N elemű `float` tömböket olvas és ír, a mozgatott bájtmennyiség $3 \cdot N \cdot 4$ bájt, mert A és B beolvasása, valamint C kiírása történik. A sávszélesség:

$$B = \frac{3 \cdot N \cdot 4 \text{ bájt}}{t},$$

ahol t a kernel futásideje másodpercben, B pedig bájt/másodpercben értendő.

- **Átlagos végrehajtási idő** és **szórás** az ismételt mérésekből. Az átlag:

$$\bar{t} = \frac{1}{n} \sum_{i=1}^{n} t_i,$$

a mintabeli szórás:

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^{n} (t_i - \bar{t})^2}.$$

A szórás azt mutatja meg, mennyire stabil a mérés. Ha a szórás a középértékhez képest nagy, akkor a mérési körülmények nem eléggé kontrolláltak, vagy a mérés túl rövid.

4.4.3 2. Mérőeszközök CPU- és GPU-időhöz

A CPU- és GPU-idő mérésére más eszköz való, mert a két végrehajtási modell eltérő.

4.4.4 2.1. CPU-idő mérése

A CPU-oldali idők mérésére a `std::chrono::steady_clock` a legjobb választás, mert monoton, nem függ a rendszeróra átállításától, és C++11 óta elérhető. A következő minta egy CPU ciklus idejét méri:

```
// C++17
#include <chrono>
#include <vector>
#include <numeric>

void cpu_vector_add(const std::vector<float>& a,
                    const std::vector<float>& b,
                    std::vector<float>& c) {
    for (size_t i = 0; i < a.size(); ++i) {
        c[i] = a[i] + b[i];
    }
}

// Minta a mérési keretből:
auto t0 = std::chrono::steady_clock::now();
cpu_vector_add(hA, hB, hC_ref);
auto t1 = std::chrono::steady_clock::now();
double cpu_ms = std::chrono::duration<double, std::milli>(t1 - t0).count();
```

A `std::chrono::steady_clock` felbontása tipikusan mikroszekundum alatti asztali rendszereken, de a mérés pontosságát a CPU gyors memóriája és a gyorsítótárak hideg/meleg állapota is befolyásolja.

4.4.5 2.2. GPU-idő mérése CUDA eseményekkel

A GPU kernel aszinkron módon fut. Ha a CPU óráját nézzük közvetlenül a kernelindítás után, akkor nem a kernel idejét mérjük, hanem a launch és a szinkronizáció időzítését. A következő minta helytelen:

```
// HIBÁS: host óra a kernel aszinkron futása miatt torzítani fog
auto start = std::chrono::steady_clock::now();
vectorAddKernel<<<grid, block>>>(dA, dB, dC, N);
auto stop = std::chrono::steady_clock::now();
double ms = std::chrono::duration<double, std::milli>(stop - start).count();
```

Ez a minta csak a kernelindítás overheadjének egy részét méri, nem pedig a tényleges GPU-végrehajtást. A helyes méréshez CUDA eseményeket kell használni:

```
// Helyes: CUDA eseményekkel a GPU streamben bekövetkező időbélyegek mérése
cudaEvent_t start, stop;
CHECK_CUDA(cudaEventCreate(&start));
CHECK_CUDA(cudaEventCreate(&stop));

CHECK_CUDA(cudaEventRecord(start));                 // a streambe kerül
vectorAddKernel<<<grid, block>>>(dA, dB, dC, N);
CHECK_CUDA(cudaGetLastError());                     // launch hiba ellenőrzése
CHECK_CUDA(cudaEventRecord(stop));                  // a streambe kerül
CHECK_CUDA(cudaEventSynchronize(stop));             // várakozás a GPU-ra

float elapsed_ms = 0.0f;
CHECK_CUDA(cudaEventElapsedTime(&elapsed_ms, start, stop));
```

A `cudaEventRecord` a streambe illeszt egy időbélyeget, a `cudaEventSynchronize` megvárja, amíg a GPU eléri azt a pontot. A `cudaEventElapsedTime` a két esemény közötti GPU-időt adja milliszekundumban.

Ugyanezt a módszert alkalmazzuk az adatátvitel mérésére is. A `cudaMemcpy` a default stream esetén szinkron, de a méréshez a `cudaEventRecord` használata akkor is konzisztens. A PCIe-átvitelt külön kell mérni a `cudaMemcpyHostToDevice` és a `cudaMemcpyDeviceToHost` szakaszokra.

4.4.6 3. A mérési példa: vektorösszeadás CPU-n és GPU-n

A méréshez ugyanazt a vektorösszeadást futtatjuk CPU-n és GPU-n, azonos bemeneti adatokkal. A CUDA kernel a következő:

```
// CUDA C++17, nvcc 12.4, Linux x86_64
// Labor-helyőrző: izolált környezetben futtatandó, nem hitelesített minta.

__global__ void vectorAddKernel(const float* A,
                                const float* B,
                                float* C,
                                int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        C[i] = A[i] + B[i];
    }
}
```

A kernel launch konfiguráció: `grid = ceil(N / 256)`, `block = 256`, `0` shared memória, `0` stream. A grid- és blockdimenziót mindig rögzíteni kell, mert a teljesítmény erősen függ tőlük. A jelen példában a blockméret 256 szál, ami a legtöbb CUDA-képes GPU-n észszerű kiindulópont.

A CPU-oldali referencia:

```
// C++17
void cpu_vector_add(const std::vector<float>& a,
                    const std::vector<float>& b,
                    std::vector<float>& c) {
    for (size_t i = 0; i < a.size(); ++i) {
        c[i] = a[i] + b[i];
    }
}
```

A mérés nem csupán a kernelre, hanem a teljes GPU-folyamatra is kiterjed. A teljes GPU-folyamat lépései:

1. A és B feltöltése a device global memóriába: `cudaMemcpyHostToDevice`.
2. A kernel futtatása.
3. C visszamásolása a hostra: `cudaMemcpyDeviceToHost`.

A mérés során ezeket külön-külön mérjük, hogy lássuk, mi dominálja a futásidőt.

4.4.7 4. Warm-up, ismétlés és szórás

4.4.8 4.1. Miért kell warm-up?

Az első GPU-hívás költséges, és nem reprezentálja a stabil futási időt. Az első hívásnál történik:

- a CUDA kontextus létrehozása,

- a modul betöltése és JIT-optimalizáció,
- a device memória első allokációja,
- a CPU–GPU közötti kapcsolat inicializálása,
- a GPU órajele és a hűtési állapot beállása.

Ha ezt a hideg állapotot mérjük, akkor a kapott idő akár több százalékkal vagy nagyságrenddel is nagyobb lehet, mint a stabil futás. Ezért minden mérés előtt **warm-up** szükséges: néhány (például 5) teljes futtatást hajtunk végre, amelyeket nem használunk a statisztikában. A warm-up hozzájárul ahhoz is, hogy a CPU gyorsítótárai, a GPU L2 cache és a TLByek stabil állapotba kerüljenek.

4.4.9 4.2. Ismétlés és statisztika

Egyetlen mérés nem elegendő. A futásidőt befolyásolja az operációs rendszer ütemezője, a GPU órajel-ingadozása, a hőmérséklet, a háttérterhelés és a mérési eszköz felbontása. Ezért legalább 5–10 ismétlést végzünk, és az eredményt az átlaggal és a szórással vagy percentilissel jellemezzük.

A laborban a következő protokollt alkalmazzuk:

1. **Warm-up**: 5 teljes GPU-ciklus (feltöltés, kernel, visszamásolás) mérés nélkül.
2. **Ismétlés**: 10 mérés, minden ciklusban külön mérve a H→D átvitelt, a kernel időt és a D→H átvitelt.
3. **Statisztika**: átlag és szórás kiszámítása minden mért szakaszra.

A 10 mérésből számolt átlag és szórás a minta stabilitását mutatja. Ha a szórás nagy, érdemes növelni az ismétlések számát, vagy ellenőrizni a háttérterhelést.

4.4.10 4.3. Példa a statisztika számítására

A mérési adatokat `std::vector<double>` tárolja, például `kernel_ms`. Az átlag és a szórás:

```
// C++17
double mean_ms(const std::vector<double>& v) {
    if (v.empty()) return 0.0;
    double sum = std::accumulate(v.begin(), v.end(), 0.0);
    return sum / static_cast<double>(v.size());
}

double stdev_ms(const std::vector<double>& v, double mean) {
    if (v.size() < 2) return 0.0;
    double sq = 0.0;
    for (double x : v) {
        sq += (x - mean) * (x - mean);
    }
    return std::sqrt(sq / static_cast<double>(v.size() - 1));
}
```

A percentilishez hasonlóan rendezhetjük a tömböt, és kiválaszthatjuk a kívánt elemet. A szórás mellett érdemes a mediánt is megadni, mert a kiugró értékekre kevésbé érzékeny.

4.4.11 5. A mérés korlátai

A CPU–GPU összehasonlítás akkor értelmes, ha pontosan tudjuk, mit hasonlítunk össze. A leggyakoribb csapdák a következők.

4.4.12 5.1. Hideg és meleg mérés

A hideg mérésnél a CUDA kontextus és a memóriaallokáció költsége a mért időben jelentkezik, ami nem a kernel valódi költsége. A meleg mérésnél ezeket a tényezőket warm-uppal és előre lefoglalt memóriával kiküszöböljük. A publikálható méréseknek meleg mérést kell használniuk, és ezt jelölniük kell.

4.4.13 5.2. Adatátvitel és kernel szétválasztása

A GPU-előny kizárólag a kernel időre koncentrálva gyakran túlzónak tűnik. A valós alkalmazásban az adatot először a GPU-ra kell juttatni, majd vissza. A PCIe-sávszélesség általában 8–16 GB/s a gyakorlatban, ami a vektorösszeadásnál jelentős, mert a kernel nagyon kevés számítást végez. Ha csak a kernel idejét hasonlítjuk a CPU-hoz, akkor a GPU látszólag nagy előnyt szerez, de a teljes GPU-folyamat már nem feltétlenül gyorsabb.

Ezért a laborban külön mérjük a H→D átvitelt, a kernel időt és a D→H átvitelt. A teljes GPU-idő ezek összege (plusz launch overhead). A CPU-idő és a teljes GPU-idő összehasonlítása reálisabb, mint a CPU és a csupasz kernel összehasonlítása.

4.4.14 5.3. A CPU-idő mérése GPU-szinkronizáció nélkül

Ha a CPU-időt úgy mérjük, hogy közben a GPU még fut, akkor a CPU-mérés hibás. A GPU-hívások után mindig `cudaDeviceSynchronize` vagy esemény-szinkronizáció szükséges, mielőtt a host órát leállítjuk. A CUDA események ezt automatikusan kezelik, ezért a GPU-idő mérésére ezek a megbízhatók.

4.4.15 5.4. Hardver- és szoftverkörnyezet hatása

A GPU órajele a terheléstől és a hőmérséklettől függően dinamikusan változik. A hosszú futásoknál a hőmérséklet növekedésével a boost órajelek csökkenhetnek, ami a későbbi ismétléseket lassabbá teszi. A mérési környezetet ezért rögzíteni kell:

- GPU modell, compute capability,
- driver verzió,
- CUDA toolkit verzió,
- operációs rendszer,
- power limit beállítás,
- ECC állapot (be/ki),
- MIG konfiguráció,
- háttérterhelés,
- hőmérsékleti viszonyok.

A `nvidia-smi` lekérdezése a mérés előtt és után segít azonosítani a throttlingot vagy a háttérfolyamatokat. A mérési naplóban ezeket az adatokat szerepeltetni kell.

4.4.16 5.5. Az időmérés pontossága

A `cudaEventElapsedTime` felbontása jellemzően 0,5 µs körüli, ami a legtöbb kernelhez elegendő. A nagyon rövid, 1–2 µs alatt futó kerneleknél azonban a mérési overhead és a launch overhead összemossa az adatokat. Ilyenkor nagyobb ismétlésszám vagy nagyobb feladatméret szükséges.

A CPU `steady_clock` pontossága rendszerenként változó, de általában nanoszekundumos felbontású. A CPU mérésnél a gyorsítótár hideg/meleg állapota és a CPU órajele szintén befolyásolja az eredményt.

4.4.17 6. Labor: CPU- és GPU-idő mérése azonos inputon

A labor célja, hogy a fenti elveket egy kis, ellenőrizhető példán alkalmazzuk. A feladat a vektorösszeadás futtatása CPU-n és GPU-n, az idők mérése, a statisztika kiszámítása és a naplózás.

4.4.18 6.1. Környezet rögzítése

A laborban használt környezetet a következőképpen rögzítjük (labor-helyőrző, nem tényleges mérés):

- GPU: NVIDIA GeForce RTX 4060, compute capability 8.9
- Driver: 550.54.14
- CUDA Toolkit: 12.4
- OS: Ubuntu 22.04, Linux x86_64
- ECC: off
- MIG: off
- Power limit: default
- Háttérterhelés: nincs

A tényleges méréshez ezeket az adatokat a saját rendszeren frissíteni kell.

4.4.19 6.2. Teljes kód

A következő CUDA C++17 kód a CPU- és GPU-időt méri, warm-uppal és 10 ismétléssel. A kód minden CUDA API-hívást ellenőriz, és a futás végén a CPU- és GPU-eredményt összehasonlítja.

```
// CUDA C++17, nvcc 12.4, Linux x86_64
// Labor-helyőrző: izolált környezetben futtatandó, nem hitelesített minta.

#include <cuda_runtime.h>
#include <algorithm>
#include <chrono>
#include <cmath>
#include <cstdlib>
#include <iomanip>
#include <iostream>
#include <numeric>
#include <vector>

#define CHECK_CUDA(call)                                                   \
    do {                                                                   \
        cudaError_t err = call;                                            \
        if (err != cudaSuccess) {                                          \
```

```
            std::cerr << "CUDA error at " << __FILE__ << ":" << __LINE__        \
                      << " -> " << cudaGetErrorString(err) << "\n";             \
            std::exit(EXIT_FAILURE);                                            \
        }                                                                       \
    } while (0)

__global__ void vectorAddKernel(const float* A,
                                const float* B,
                                float* C,
                                int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        C[i] = A[i] + B[i];
    }
}

void cpu_vector_add(const std::vector<float>& a,
                    const std::vector<float>& b,
                    std::vector<float>& c) {
    for (size_t i = 0; i < a.size(); ++i) {
        c[i] = a[i] + b[i];
    }
}

double mean_ms(const std::vector<double>& v) {
    if (v.empty()) return 0.0;
    double sum = std::accumulate(v.begin(), v.end(), 0.0);
    return sum / static_cast<double>(v.size());
}

double stdev_ms(const std::vector<double>& v, double mean) {
    if (v.size() < 2) return 0.0;
    double sq = 0.0;
    for (double x : v) {
        sq += (x - mean) * (x - mean);
    }
    return std::sqrt(sq / static_cast<double>(v.size() - 1));
}

float max_error(const std::vector<float>& a, const std::vector<float>& b) {
    float m = 0.0f;
    for (size_t i = 0; i < a.size(); ++i) {
        m = std::max(m, std::abs(a[i] - b[i]));
    }
    return m;
}

int main() {
    constexpr int N = 1 << 22;           // 4 194 304 elem
    constexpr size_t bytes = N * sizeof(float);
    constexpr int block = 256;
    constexpr int grid = (N + block - 1) / block;
    constexpr int warmups = 5;
    constexpr int repeats = 10;

    std::vector<float> hA(N), hB(N), hC(N), hC_ref(N);
    for (int i = 0; i < N; ++i) {
        hA[i] = static_cast<float>(i) * 1.0e-3f;
        hB[i] = static_cast<float>(N - i) * 1.0e-3f;
    }

    // CPU referencia
    auto t0 = std::chrono::steady_clock::now();
```

```
cpu_vector_add(hA, hB, hC_ref);
auto t1 = std::chrono::steady_clock::now();
double cpu_ms = std::chrono::duration<double, std::milli>(t1 - t0).count();

// Device memória allokáció
float *dA, *dB, *dC;
CHECK_CUDA(cudaMalloc(&dA, bytes));
CHECK_CUDA(cudaMalloc(&dB, bytes));
CHECK_CUDA(cudaMalloc(&dC, bytes));

// Warm-up
for (int w = 0; w < warmups; ++w) {
    CHECK_CUDA(cudaMemcpy(dA, hA.data(), bytes, cudaMemcpyHostToDevice));
    CHECK_CUDA(cudaMemcpy(dB, hB.data(), bytes, cudaMemcpyHostToDevice));
    vectorAddKernel<<<grid, block>>>(dA, dB, dC, N);
    CHECK_CUDA(cudaGetLastError());
    CHECK_CUDA(cudaMemcpy(hC.data(), dC, bytes, cudaMemcpyDeviceToHost));
    CHECK_CUDA(cudaDeviceSynchronize());
}

// Mérési események
cudaEvent_t start, stop;
CHECK_CUDA(cudaEventCreate(&start));
CHECK_CUDA(cudaEventCreate(&stop));

std::vector<double> h2d_ms(repeats), kernel_ms(repeats), d2h_ms(repeats);

for (int r = 0; r < repeats; ++r) {
    // H->D átvitel
    CHECK_CUDA(cudaEventRecord(start));
    CHECK_CUDA(cudaMemcpy(dA, hA.data(), bytes, cudaMemcpyHostToDevice));
    CHECK_CUDA(cudaMemcpy(dB, hB.data(), bytes, cudaMemcpyHostToDevice));
    CHECK_CUDA(cudaEventRecord(stop));
    CHECK_CUDA(cudaEventSynchronize(stop));
    float ms = 0.0f;
    CHECK_CUDA(cudaEventElapsedTime(&ms, start, stop));
    h2d_ms[r] = ms;

    // Kernel
    CHECK_CUDA(cudaEventRecord(start));
    vectorAddKernel<<<grid, block>>>(dA, dB, dC, N);
    CHECK_CUDA(cudaGetLastError());
    CHECK_CUDA(cudaEventRecord(stop));
    CHECK_CUDA(cudaEventSynchronize(stop));
    CHECK_CUDA(cudaEventElapsedTime(&ms, start, stop));
    kernel_ms[r] = ms;

    // D->H átvitel
    CHECK_CUDA(cudaEventRecord(start));
    CHECK_CUDA(cudaMemcpy(hC.data(), dC, bytes, cudaMemcpyDeviceToHost));
    CHECK_CUDA(cudaEventRecord(stop));
    CHECK_CUDA(cudaEventSynchronize(stop));
    CHECK_CUDA(cudaEventElapsedTime(&ms, start, stop));
    d2h_ms[r] = ms;
}

// Statisztika
double h2d_mean = mean_ms(h2d_ms);
double h2d_sd = stdev_ms(h2d_ms, h2d_mean);
double k_mean = mean_ms(kernel_ms);
double k_sd = stdev_ms(kernel_ms, k_mean);
double d2h_mean = mean_ms(d2h_ms);
double d2h_sd = stdev_ms(d2h_ms, d2h_mean);
```

```
    // Ellenőrzés
    float err = max_error(hC, hC_ref);

    std::cout << std::fixed << std::setprecision(4);
    std::cout << "CPU vector add          : " << cpu_ms << " ms\n";
    std::cout << "GPU H->D mean (sd)      : " << h2d_mean << " (" << h2d_sd << ") ms\n";
    std::cout << "GPU kernel mean (sd)    : " << k_mean << " (" << k_sd << ") ms\n";
    std::cout << "GPU D->H mean (sd)      : " << d2h_mean << " (" << d2h_sd << ") ms\n";
    std::cout << "GPU total mean          : " << (h2d_mean + k_mean + d2h_mean) << " ms\n";
    std::cout << "Max abs error           : " << err << "\n";

    CHECK_CUDA(cudaEventDestroy(start));
    CHECK_CUDA(cudaEventDestroy(stop));
    CHECK_CUDA(cudaFree(dA));
    CHECK_CUDA(cudaFree(dB));
    CHECK_CUDA(cudaFree(dC));

    return 0;
}
```

A kernel launch alakja a kódban `<<<grid, block>>>`, ami a `<<<grid, block, 0, 0>>>` rövidített alakja. A grid és block dimenzió 1D, a shared memória 0 bájt, a stream a default stream.

4.4.20 6.3. Build és futtatás

A kód fordítása `nvcc`-vel:

```
# Fordítás
nvcc -std=c++17 -O3 -arch=sm_89 -o vector_add_bench vector_add_bench.cu

# Futtatás
./vector_add_bench
```

Ha a GPU compute capability nem 8.9, az `-arch` értéket cserélni kell. A laborban rögzíteni kell a tényleges GPU-t és a CUDA-verziót.

A futtatás előtt ellenőrizzük a GPU állapotát:

```
nvidia-smi --query-gpu=name,compute_cap,driver_version,power.draw,temperature.gpu,clocks.sm
↪  --format=csv
```

Ez segít rögzíteni a mérési körülményeket.

4.4.21 6.4. Mérési napló

Az alábbi táblázat a mérési napló sablonja. A cellákban * jelzi a labor-helyőrzőt, mert a példa nem hitelesített futtatásból származik. A tényleges méréshez a helyére a saját értékeket kell beírni.

Ismétlés	CPU (ms)	H→D (ms)	Kernel (ms)	D→H (ms)	GPU total (ms)
1	*	*	*	*	*
2	*	*	*	*	*
3	*	*	*	*	*
4	*	*	*	*	*
5	*	*	*	*	*

Ismétlés	CPU (ms)	H→D (ms)	Kernel (ms)	D→H (ms)	GPU total (ms)
6	*	*	*	*	*
7	*	*	*	*	*
8	*	*	*	*	*
9	*	*	*	*	*
10	*	*	*	*	*
Átlag	*	*	*	*	*
Szórás	*	*	*	*	*

A táblázatban a CPU mérése csak egyszer szerepel, de a pontos protokoll szerint a CPU-időt is 10-szer kell mérni, és a statisztikát ugyanúgy ki kell számolni.

4.4.22 6.5. Várt és mért eredmény szétválasztása

Várt eredmény (modell, nem mérés):

- CPU: a vektorösszeadás $N = 2^{22}$ elemre $3 \cdot 4\text{ M} \cdot 4$ bájt = 50,3 MB memóriaműveletet igényel. Ha a CPU sávszélessége 20 GB/s, az idő körülbelül 2,5 ms. Ez a szám erősen függ a CPU gyorsítótár-méretétől és a memória-teljesítménytől.
- GPU kernel: ugyanez a 50,3 MB a GPU global memória sávszélességével. Egy RTX 4060 szintű GPU elméleti sávszélessége 272 GB/s, de a gyakorlati kernel sávszélesség ennek töredéke, például 100–200 GB/s. A kernel idő ezért nagyságrendileg 0,25–0,5 ms lehet.
- H→D és D→H átvitel: PCIe 3.0/4.0 x16 sávszélesség 8–16 GB/s gyakorlatilag. A 32 MB feltöltés körülbelül 2–4 ms, a 16 MB visszamásolás 1–2 ms. A teljes GPU-idő így 3–6 ms lehet.
- Következtetés: ennél a memóriakorlátos, kis számítási intenzitású feladatnál a teljes GPU-idő nagyobb lehet, mint a CPU-idő. A GPU előnye akkor jelenik meg, ha a kernel több számítást végez a memóriaműveletekhez képest, vagy ha a CPU sávszélessége alacsonyabb. Ez a lecke megalapozza a későbbi roofline-elemzést.

Mért eredmény (labor-helyőrző, nem hitelesített): a mérési napló celláit a tényleges futtatás után kell kitölteni. A mért értékeket csak a saját laboratóriumi futtatás után szabad ellenőrzöttként közölni. A várt és a mért értékek különválasztása megakadályozza, hogy modellbecslést tényként mutassunk be.

4.4.23 7. Önellenőrző kérdés

Miért nem elegendő a `std::chrono::steady_clock` a GPU kernel futásidejének mérésére közvetlenül a kernelindítás után, és hogyan oldja meg ezt a CUDA esemény? Válaszodban térj ki az aszinkron végrehajtás, a launch overhead és a szinkronizáció szerepére.

4.4.24 8. Összegzés

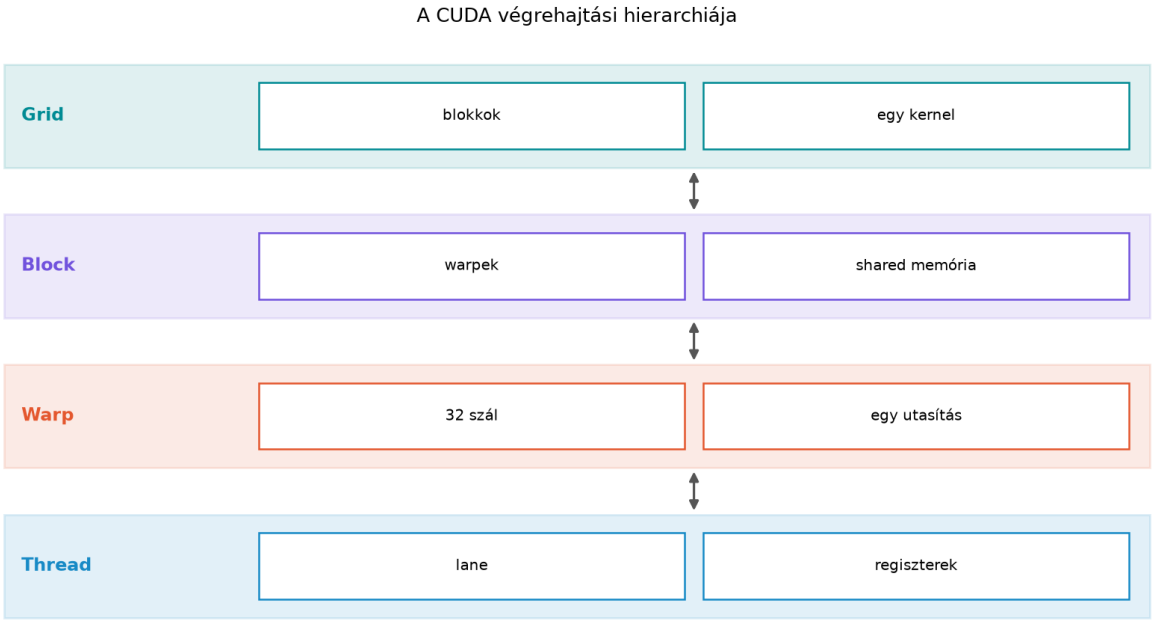
Az első mérés: CPU vs. GPU alfejezetben felépítettük az első valódi teljesítménymérést. Megtanultuk, hogy a CPU- és GPU-időt külön eszközökkel, de azonos feladaton mérjük, warm-upot és ismétléseket alkalmazunk, statisztikát számolunk, és a környezetet rögzítjük. A legfontosabb tanulság, hogy a GPU kernel ideje önmagában nem jelenti a teljes GPU-feladat idejét: az adatátvitel és a launch overhead legalább annyira fontos. A mért és várt eredmény szétválasztása pedig a mérnöki becsületesség alapja.

A következő alfejezet (Thread, block, grid és a hardverre képezés) erre építve mutatja be a CUDA végrehajtási modell részleteit: hogyan képeződnek le a threadek, blokkok és gridek a hardverre, és miért befolyásolja ez a mérést is.

5 CUDA execution model: thread, block, grid és warp

5.1 Thread, block, grid és a hardverre képezés

A CUDA végrehajtási modelljének megértéséhez először két, egymástól élesen elkülönülő hierarchiát kell szétválasztanunk: a logikai hierarchiát, amelyet a programozó a kernelindításkor megad, és a fizikai hierarchiát, amelyet a hardver a végrehajtás során létrehoz. A kettő közötti leképezés határozza meg, hogy egy adott launch konfiguráció hogyan használja ki a GPU erőforrásait. Ez a fejezet ezt a kapcsolatot járja körül, és egy olyan laborral zárul, amelyben a leképezés lépéseit konkrét API-hívásokkal ellenőrizzük.



5.1. ábra: A CUDA végrehajtási hierarchiája - A logikai hierarchia gridből, blokkokból, warpekből és szálakból áll; a hardver a blokkokat SM-ekhez, a warpeket warp schedulerekhez rendeli.

5.1.1 Logikai hierarchia: thread, block, grid

A CUDA programozási modellben a párhuzamos munka egysége a **thread**. Egy thread önálló vezérlési folyammal rendelkezik, saját regiszterállapota és lokális memóriája van, és a kernelutasítások sorozatát hajtja végre. A thread fogalma azonban önmagában nem elegendő: több ezer vagy több millió szál kezeléséhez a modell két további szerveződési szintet vezet be.

A **block** thread-ek egy csoportja, amelyek logikailag összetartoznak. Egy blockon belül a szálak együtt ütemezhetők, megoszthatnak egy gyors, közös memóriaterületet (shared memory), és szinkronizálhatják egymást a `__syncthreads()` hívással. A block mérete a kernelindításkor rögzített, és a blockon belüli szálak száma a `blockDim` beépített változóval kérdezhető le. A block legfeljebb háromdimenziós lehet, így `blockDim.x`, `blockDim.y` és `blockDim.z` adja meg a kiterjedést. A blockon belüli szál pozícióját a `threadIdx` struktúra azonosítja, amelynek `x`, `y` és `z` komponensei vannak.

A **grid** a block-ok egy- vagy többdimenziós tömbje. A grid mérete a kernelindításkor megadott `gridDim` értékből derül ki, az egyes block-ok pozícióját pedig a `blockIdx` struktúra adja meg. A grid és a block együttesen határozza meg a kernel által elindított összes szál számát: `gridDim * blockDim`. A grid lehet egydimenziós, de akár háromdimenziós is, ami különösen kényelmes olyan problémáknál, amelyek természetesen illeszkednek 2D vagy 3D rácsokhoz, például képfeldolgozás vagy térbeli szimuláció esetén.

A logikai hierarchia legfontosabb tulajdonsága, hogy teljesen elvont, és nem függ a konkrét hardvertől. A programozó a kernelben csak a `threadIdx`, `blockIdx`, `blockDim` és `gridDim` értékekkel dolgozik, anélkül, hogy ismerné, hány streaming multiprocesszor (SM) található az adott GPU-ban, vagy hogy az ütemező mikor és hová helyezi a block-okat. Ez a hardverfüggetlen absztrakció teszi lehetővé, hogy ugyanaz a kernel különböző GPU-generációkon is fusson, feltéve, hogy a szükséges compute capability rendelkezésre áll.

Egy tipikus egydimenziós kernelben a globális szálazonosító kiszámítása a következőképpen történik:

```
int tid = blockIdx.x * blockDim.x + threadIdx.x;
```

Ez a képlet a gridben elfoglalt pozíciót lineáris indexszé alakítja. Többdimenziós esetben az indexelés ennek megfelelően bővül, például 2D-s grid és 2D-s block esetén:

```
int x = blockIdx.x * blockDim.x + threadIdx.x;
int y = blockIdx.y * blockDim.y + threadIdx.y;
int tid = y * (gridDim.x * blockDim.x) + x;
```

A logikai hierarchia tehát a programozó számára egy kényelmes koordináta-rendszert ad, amelyben a párhuzamos munka természetes módon szervezhető.

5.1.2 Fizikai hierarchia: SM, warp, scheduler

A hardver oldalán a GPU több **streaming multiprocesszorból** (SM) épül fel. Az SM a legfontosabb végrehajtási egység: minden SM saját regiszterfájllal, shared memory-val, L1 cache-sel, valamint több végrehajtó egységgel rendelkezik. A block-ok az SM-ekre kerülnek kiosztásra, és egy block teljes egészében egyetlen SM-en fut. Egy SM viszont egyszerre több block-ot is képes befogadni, amennyiben az erőforrások (regiszterek, shared memory, thread-kapacitás) ezt lehetővé teszik.

Az SM-en belül a szálak nem egyenként, hanem **warp**-okba szervezve hajtódnak végre. A warp 32 szálból álló csoport, amely az SM-en belüli legkisebb ütemezési egység. A warp-ok a **warp scheduler**-ekhez tartoznak: minden SM több schedulerrel rendelkezik (a legtöbb modern GPU-n 4 scheduler található SM-enként), és minden scheduler minden órajelciklusban legfeljebb egy warp-ot tud kiadni végrehajtásra. A warp szálai ugyanazt az utasítást hajtják végre ugyanabban az időben, ez a **SIMT** (Single Instruction, Multiple Thread) végrehajtási modell. Ha a vezérlési folyam elágazik, a warp divergenssé válik, és a különböző ágak sorosan futnak le – ezt a következő fejezet tárgyalja részletesen.

A warp-on belüli szálakat **lane**-nek nevezzük, és a lane-ek 0-tól 31-ig számozódnak. A warp mérete a CUDA-hardverben 32, és ez a legtöbb GPU-generációban állandó. A `warpSize` eszköztulajdonság ezt az értéket adja vissza, és a programozónak ezt figyelembe kell vennie a blockméret megválasztásakor.

A fizikai hierarchia tehát a következő: a GPU több SM-ből áll, minden SM több warp schedulerrel rendelkezik, minden scheduler warp-okat ütemez, és minden warp 32 szálat foglal magában. A block és a grid logikai fogalmai erre a fizikai szerkezetre képződnek le.

5.1.3 Block és SM leképezés

A kernelindításkor a CUDA runtime a grid block-jait az SM-ekhez rendeli. A leképezés alapelvei a következők:

- Egy **block** mindig egyetlen SM-en fut. A block-ot nem lehet szétdarabolni, és a végrehajtás során nem vándorolhat másik SM-re. Ez garantálja, hogy a blockon belüli szálak hozzáférhetnek a shared memory-hoz, és szinkronizálhatják egymást.
- Egy **SM** egyszerre több block-ot is futtathat, ha az erőforrások elégségesek. Az egy SM-en egyidejűleg aktív block-ok számát **resident block**-oknak nevezzük.
- A grid block-jai közül a runtime annyit helyez el az SM-ekre, amennyit az erőforrás-korlátok megengednek. Amikor egy block befejezi a végrehajtását, az SM felszabadítja annak erőforrásait, és a runtime újabb block-ot indíthat a gridből.

Az SM-ek száma a GPU típusától függ: például egy NVIDIA A100 GPU-ban 108 SM található, míg egy kisebb, például egy GeForce RTX 3060 esetén ez a szám jóval alacsonyabb, 28. A `cudaGetDeviceProperties` API-val ez az érték lekérdezhető (`multiProcessorCount`). A grid mérete általában sokkal nagyobb, mint az SM-ek száma, ezért a block-ok hullámokban kerülnek végrehajtásra. Egy adott időpontban minden SM-en legfeljebb annyi block aktív, amennyit az erőforrás-korlátok lehetővé tesznek; a többi block a sorban vár.

A leképezés szempontjából három fő erőforrás-korlátot kell figyelembe venni:

1. **Maximális threads per SM**: Minden SM-nek van egy maximális szálkapacitása, amely tipikusan 1024, 1536 vagy 2048 szál az adott GPU-architektúrától függően. Ha egy block 256 szálat tartalmaz, és az SM max 2048 szálat fogadhat, akkor legfeljebb 8 block lehet aktív SM-enként ebből a korlátból adódóan.
2. **Regiszterfájl mérete**: Minden SM regiszterfájljának véges a kapacitása, például 65536 regiszter. Ha egy kernel szálanként 16 regisztert használ, akkor egy 256 szálas block 4096 regisztert foglal le, így a regiszterlimit 16 block-ot engedne meg, ami nem szűkebb, mint a thread-limit.
3. **Shared memory mérete**: Az SM-ek shared memory kapacitása korlátozott, például 164 KiB, 100 KiB vagy 48 KiB az architektúrától függően. Ha a kernel blockonként 32 KiB shared memory-t foglal le, akkor a shared memory korlátja 3–5 blockra korlátozza az aktív block-ok számát.

A tényleges resident block-szám ezen korlátok **minimuma**. Ez a leképezés a `cudaOccupancyMaxActiveBlocksPer` API-val lekérdezhető, amely a kernel attribútumai alapján kiszámítja, hány block lehet egy SM-en egyidejűleg aktív.

Fontos megérteni, hogy a block-ok SM-ekhez rendelése nem determinisztikus sorrendben történik, de a végrehajtás eredménye determinisztikus, feltéve, hogy a kernelben nincs adatverseny. A grid mérete és az SM-ek száma közötti viszony határozza meg, hogy a GPU mennyire telített: ha a grid túl kicsi, az SM-ek egy része üresen marad, ha túl nagy, a block-ok sorban várnak, ami viszont a latency hiding szempontjából előnyös lehet.

5.1.4 Occupancy alapfogalma

Az **occupancy** az SM-ek kihasználtságának egyik leggyakrabban használt mérőszáma. Formálisan az aktív warp-ok számának és az SM-en lehetséges maximális warp-számnak a hányadosa:

$$\text{occupancy} = \frac{\text{aktív warpok száma SM-en}}{\text{maximális warp-szám SM-en}}$$

A maximális warp-szám az SM threads-per-SM korlátjából adódik, elosztva a warp méretével (32). Például ha egy SM legfeljebb 2048 szálat kezelhet, akkor a maximális warp-szám:

$$\frac{2048}{32} = 64 \text{ warp}$$

Ha a kernelünk 256 szálas block-okat indít, és az SM-en 8 block aktív, akkor az aktív warp-ok száma:

$$8 \text{ block} \times \frac{256 \text{ szál}}{32 \text{ szál/warp}} = 64 \text{ warp}$$

Ez teljes occupancy-t jelent, azaz 100%-ot. Ha viszont a shared memory korlát miatt csak 4 block fér el, akkor az aktív warp-ok száma 32, az occupancy pedig 50%.

Az occupancy fontos szerepet játszik a **latency hiding**-ban. A GPU-k nagy áteresztőképességét az adja, hogy az SM-ek sok aktív warp-ot tartanak nyilván, és amíg az egyik warp memóriaműveletre vár, a scheduler egy másik, már futásra kész warp-ot ad ki. Minél magasabb az occupancy, annál több warp közül választhat a scheduler, így jobban el tudja rejteni a memória-késleltetést és a függőségi szüneteket. A magas occupancy azonban nem jelent automatikusan maximális teljesítményt: előfordulhat, hogy a magasabb occupancy érdekében kisebb blockméretet választunk, ami viszont csökkenti a shared memory-használat hatékonyságát, vagy a regiszterek szűkössége miatt spill-hez vezet. Az occupancy tehát egy fontos diagnosztikai mutató, de nem az egyetlen optimalizálási cél.

5.1.5 Labor: launch konfiguráció hardverleképezése

A következő labor célja, hogy egy konkrét kernel esetén megvizsgáljuk, hogyan képeződik le a grid és a block az SM-ekre. A példa CUDA C++-ban íródott, C++17 szabvánnyal, és a `nvcc` 12.4-es verziójával fordítható. A kód minden CUDA API-hívást ellenőriz egy egyszerű hibamakróval, lekéri az eszköz tulajdonságait, majd a `cudaOccupancyMaxActiveBlocksPerMultiprocessor` segítségével meghatározza, hány block lehet aktív SM-enként. A kernel egy egyszerű indexelési feladatot végez: minden szál a saját globális indexét írja egy kimeneti tömbbe. Ezzel ellenőrizhető, hogy a logikai indexszámítás helyes-e.

```
// CUDA C++ (C++17), nvcc 12.4
#include <cstdio>
#include <cstdlib>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err = (call);                                          \
        if (err != cudaSuccess) {                                          \
            std::fprintf(stderr, "CUDA error at %s:%d: %s\n",             \
                         __FILE__, __LINE__, cudaGetErrorString(err));      \
```

```
            cudaDeviceReset();                                          \
            std::exit(EXIT_FAILURE);                                    \
        }                                                               \
    } while (0)

__global__ void map_kernel(int *out, int n) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    if (tid < n) {
        out[tid] = tid;
    }
}

int main() {
    int device = 0;
    CUDA_CHECK(cudaSetDevice(device));

    cudaDeviceProp prop;
    CUDA_CHECK(cudaGetDeviceProperties(&prop, device));

    const int block_size = 256;
    const int n = 1 << 20; // 1 Mi elem
    const int grid_size = (n + block_size - 1) / block_size;

    int *d_out;
    CUDA_CHECK(cudaMalloc(&d_out, n * sizeof(int)));

    map_kernel<<<grid_size, block_size>>>(d_out, n);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    // Kernel attribútumok lekérése
    cudaFuncAttributes attr;
    CUDA_CHECK(cudaFuncGetAttributes(&attr, map_kernel));

    // Aktív block-ok számának lekérdezése
    int num_blocks_per_sm = 0;
    CUDA_CHECK(cudaOccupancyMaxActiveBlocksPerMultiprocessor(
        &num_blocks_per_sm, map_kernel, block_size, 0));

    // Eredmény ellenőrzése
    int *h_out = new int[n];
    CUDA_CHECK(cudaMemcpy(h_out, d_out, n * sizeof(int), cudaMemcpyDeviceToHost));

    bool ok = true;
    for (int i = 0; i < n; ++i) {
        if (h_out[i] != i) {
            ok = false;
            break;
        }
    }

    std::printf("Device: %s\n", prop.name);
    std::printf("Compute capability: %d.%d\n", prop.major, prop.minor);
    std::printf("Number of SMs: %d\n", prop.multiProcessorCount);
    std::printf("Max threads per SM: %d\n", prop.maxThreadsPerMultiProcessor);
    std::printf("Warp size: %d\n", prop.warpSize);
    std::printf("Block size: %d\n", block_size);
    std::printf("Grid size: %d\n", grid_size);
    std::printf("Registers per thread: %d\n", attr.numRegs);
    std::printf("Shared memory per block: %zu bytes\n", attr.sharedSizeBytes);
    std::printf("Max active blocks per SM: %d\n", num_blocks_per_sm);
    std::printf("Theoretical max resident warps per SM: %d\n",
                (num_blocks_per_sm * block_size) / prop.warpSize);
```

```
    std::printf("Index check: %s\n", ok ? "OK" : "FAILED");

    delete[] h_out;
    CUDA_CHECK(cudaFree(d_out));
    CUDA_CHECK(cudaDeviceReset());
    return 0;
}
```

Fordítás és futtatás:

```
# Fordítás
nvcc -std=c++17 -O3 -arch=sm_80 -o gpu014_lab gpu014_lab.cu

# Futtatás
./gpu014_lab
```

A laborhoz rögzített környezet (helyettesítő, izolált labor):

- **GPU:** NVIDIA A100 40 GB (labor-helyettesítő)
- **Driver:** 550.54.15
- **CUDA:** 12.4
- **Operációs rendszer:** Ubuntu 22.04 LTS
- **Compute capability:** 8.0

A mérési naplóban a várt és a mért értékeket elkülönítve adjuk meg. A várt értékek a hardverparaméterekből és a kernel attribútumaiból számíthatók, a mért értékek pedig a futtatás során az API-k által visszaadott eredmények.

Jellemző	Várt érték (számított)	Mért érték (labor, illusztráció)
SM-ek száma	108 (A100 specifikáció)	108 (cudaGetDeviceProperties)
Warp size	32	32
Max threads per SM	2048	2048
Block size	256	256
Grid size	4096	4096
Registers per thread	16 (fordítás után)	16 (cudaFuncGetAttributes)
Shared memory per block	0 bájt	0 bájt
Max aktív block/SM	8 (thread-limit: 2048 / 256)	8 (occupancy API)
Aktív warp/SM	64	64
Max warp/SM	64	64
Occupancy	100%	100%

A táblázatban a „mért” oszlop illusztráció; a saját laborban a kapott értékeket kell rögzíteni. A várt és a mért értékek itt egybeesnek, mert a kernel egyszerű, és az egyetlen korlát a thread-limit. Ha a kernel több regisztert vagy shared memory-t igényelne, a várt és a mért értékek eltérhetnének a többi korlát miatt.

A leképezés ellenőrzésének menete:

1. Lekérjük az SM-ek számát (`prop.multiProcessorCount`).
2. Lekérjük a maximális szálszámot SM-enként (`prop.maxThreadsPerMultiProcessor`).
3. Lekérjük a kernel regiszter- és shared memory-igényét (`cudaFuncGetAttributes`).
4. Kiszámítjuk a lehetséges aktív block-ok számát minden korlátra:
    - Thread-limit: `maxThreadsPerMultiProcessor / block_size`

- Regiszter-limit: `regsPerSM / (attr.numRegs * block_size)`
- Shared memory-limit: `sharedMemPerSM / attr.sharedSizeBytes`

5. A három közül a minimum a várt aktív block-szám.
6. Az occupancy API-val lekérdezzük a tényleges értéket, és összehasonlítjuk.

Ebben a példában a `map_kernel` 16 regisztert használ szálanként, és nincs shared memory-igénye. A regiszter-limit:

$$\frac{65536}{16 \cdot 256} = 16 \text{ block}$$

A thread-limit:

$$\frac{2048}{256} = 8 \text{ block}$$

A shared memory-limit nem korlátoz. A minimum tehát 8 block, ami teljes occupancy-t eredményez.

5.1.6 Tipikus hibák és megelőzésük

A thread, block, grid és hardverre képezés témakörében a következő hibák fordulnak elő leggyakrabban:

1. **Hiányzó határellenőrzés.** Ha a grid mérete nem osztja pontosan a feldolgozandó elemek számát, az utolsó block-ban több szál lehet, mint szükséges. A `tid < n` feltétel elhagyása out-of-bounds írást okozhat. Mindig ellenőrizzük a globális indexet a kernelben.
2. **A blockméret nem a warp méret többszöröse.** Bár a CUDA megengedi a nem 32-vel osztható blockméreteket, az ilyen blockok utolsó warp-ja nem teljes, ami regiszter- és occupancy-pazarláshoz vezet. A blockméreteket érdemes 32 többszörösére választani, kivéve, ha speciális okunk van másként.
3. **A grid méret hibás kiszámítása.** Az egész osztás (`n / block_size`) elhagyja az utolsó, nem teljes block-ot. A helyes képlet: `(n + block_size - 1) / block_size`.
4. **Az occupancy egyoldalú maximalizálása.** A magas occupancy fontos, de nem szabad mindenáron növelni. A kisebb blockméret gyakran magasabb occupancy-t ad, de ronthatja a memória-hozzáférés hatékonyságát, vagy növelheti a regiszter-spill esélyét. A cél a kiegyensúlyozott konfiguráció.
5. **Az SM-ek számának figyelmen kívül hagyása.** Ha a grid túl kicsi, a GPU egyes SM-jei üresen maradnak. A grid méretét úgy kell megválasztani, hogy legalább annyi block legyen, ahány SM van, de a gyakorlatban ennél több, mert a block-ok hullámokban futnak.
6. **A kernel attribútumok ellenőrzésének elmulasztása.** A fejlesztés során érdemes rendszeresen lekérdezni a `cudaFuncGetAttributes` és a `cudaOccupancyMaxActiveBlocksPerMultiprocessor` értékeit, hogy időben észleljük a regiszter- vagy shared memory-nyomást.

5.1.7 Összefoglalás és önellenőrző kérdés

A CUDA végrehajtási modell logikai hierarchiája a thread, block és grid fogalmaira épül, míg a fizikai hierarchia az SM-ekből, warp scheduler-ekből és warp-okból áll. A block-ok egy-egy SM-re kerülnek, és az SM-ek egyszerre több block-ot is futtathatnak, ha az erőforrások (thread-kapacitás, regiszterek, shared memory) megengedik. A warp-ok 32 szálból állnak, és a scheduler-ek ezeket ütemezik. Az occupancy az aktív warp-ok arányát méri az SM-en lehetséges maximális warp-számhoz képest, és alapvető szerepet játszik a latency hiding hatékonyságában.

Önellenőrző kérdés: Egy GPU SM-enként legfeljebb 2048 szálat és 64 warp-ot tud kezelni. A kernelünk 128 szálas block-okat indít, szálanként 20 regisztert használ, és nincs shared memory-igénye. Az SM regiszterfájlja 65536 regisztert tartalmaz. Hány block lehet aktív SM-enként? Mekkora az occupancy?

Válasz: A thread-limit: 2048 / 128 = 16 block. A regiszter-limit: 65536 / (128 · 20) = 25,6, tehát 25 block. A szűkebb korlát a thread-limit, így 16 block lehet aktív. Az aktív warp-ok száma: 16 block · (128 / 32) = 64 warp. Az SM maximális warp-száma 64, tehát az occupancy 100%.

5.2 Warp-szintű végrehajtás és divergence

A GPU programozásban a szálak nem egyenként, hanem 32 elemű csoportokban, warpokban hajtódnak végre. Ez a szerveződés határozza meg, hogyan ütemezi a hardver az utasításokat, és hogyan viselkedik a kód elágazásoknál. A warp-szintű végrehajtás megértése nélkül nem lehet hatékony CUDA-kódot írni, mert a legtöbb teljesítményprobléma ezen a szinten jelentkezik. Ebben a fejezetben bemutatjuk a warp működését, az aktív maszk és a rekonvergencia fogalmát, valamint azt, hogyan mérhető a divergencia költsége.

5.2.1 A warp mint 32 szálas végrehajtási egység

A CUDA programozási modellben a kernel indításakor a grid blokkokból áll, a blokkok pedig szálakból. A hardver a blokkokat warpokra osztja; minden warp pontosan 32 szálat tartalmaz. A szálazonosítók a blokkon belül folytonosak, ezért az első warp a 0–31. szálakat, a második a 32–63. szálakat, és így tovább. A warp mérete a jelenlegi NVIDIA architektúrákon 32, de ez a compute capability része, és elvileg változhat; a CUDA programozásban a `warpSize` konstans mindig az aktuális hardver értékét adja.

A warp az utasításfolyam egysége: a streaming multiprocesszor (SM) utasításkibocsátó egysége egy utasítást olvas be, és azt a warp minden aktív szálára végrehajtja. Ez a SIMT (Single Instruction, Multiple Thread) modell: a szálak ugyanazt az utasítást hajtják végre, de saját regisztereikkel, saját program counterrel és saját végrehajtási állapottal. A hardver nem szálanként, hanem warponként ütemezi az utasításokat, ami lehetővé teszi, hogy az SM sok warp között váltson, és ezzel elrejtse a memória- és utasítás-késleltetést. A warpok közötti váltás alapja az occupancy, amit későbbi fejezet tárgyal; itt elég annyit rögzíteni, hogy a warp a hardver által egyszerre kezelt legkisebb ütemezési egység, ezért viselkedése kritikus a teljesítmény szempontjából.

5.2.2 Aktív maszk és divergencia

A SIMT modell akkor válik érdekessé, amikor a szálak különböző utat választanak. A klasszikus példa az `if-else` elágazás:

```
if (threadIdx.x % 2 == 0) {
    // páros szálak útja
} else {
    // páratlan szálak útja
}
```

Az SM minden utasítást a warp minden aktív szálára kiad. Ha a feltétel nem egyezik meg a warp összes szálára, akkor a warp divergál: a hardver kénytelen mindkét ág utasításait végrehajtani, de közben egyes szálakat letilt (inaktívvá tesz). Ezt a letiltást az aktív maszk (active mask) vezérli. Az aktív maszk egy 32 bites bitmaszk, amelyben az 1-es bitek jelölik azokat a szálakat, amelyekre az aktuális utasítás érvényes. Az `if` ág végrehajtásakor a páros szálak aktívak, a páratlanok inaktívak; az `else` ágban fordítva. A hardver sorosan hajtja végre a két ágat: először az `if` utasításait a páros szálakon, majd az `else` utasításait a páratlan szálakon. A két ág végrehajtása után a warp újra konvergál, és a szálak közös úton folytatják.

A rekonvergencia az a pont, ahol a divergens szálak ismét egyesülnek. A CUDA hardver ezt automatikusan kezeli: a vezérlőfolyam-gráf alapján az SM tudja, hol kell a szálakat újra egyesíteni, és amikor minden ág befejeződött, az aktív maszk visszaáll a teljes warp-ra. A programozónak nincs teendője a rekonvergencia kiváltására, de fontos tudnia, hogy a divergens végrehajtás során a warp utasításainak száma megnő, és ezzel a hasznos számítási teljesítmény csökken.

Az aktív maszk nem csak elágazásnál létezik: a `__activemask()` intrinsic visszaadja az aktuálisan aktív szálak bitmaszkját, a `__ballot_sync` pedig egy predikátum alapján épít maszkot. Ezek a warp-szintű együttműködés alapjai, és a Warp intrinsics: shuffle, vote és match (warp intrinsics) fejezetben részletesen tárgyaljuk őket. Itt a divergencia megértéséhez elegendő az aktív maszk fogalma.

5.2.3 Predikáció: a divergencia egyik eszköze

A rövid elágazások esetén a hardver gyakran predikációt használ a divergencia elkerülésére. A predikáció azt jelenti, hogy az utasítás minden szálra végrehajtódik, de az eredményt csak azokra a szálakra írja vissza, amelyekre a predikátum igaz. Például a következő C++ kód:

```
int a = (threadIdx.x % 2 == 0) ? x : y;
```

Ez a kifejezés valószínűleg predikált utasításokra fordul: a feltétel kiértékelése után mindkét lehetséges érték előáll, és egy predikált mozgatás (`SEL` vagy hasonló) választja ki a megfelelőt. Nincs valódi elágazás, így nincs divergens végrehajtás sem. A predikáció akkor hatékony, ha az ágak rövidek és az összes ág végrehajtása olcsóbb, mint a divergens vezérlés. Ha az ágak hosszúak, vagy az egyik ág memória-hozzáférést igényel, a predikáció nem mindig nyer; ilyenkor a divergens végrehajtás elkerülhetetlen.

A fordító dönti el, hogy egy `if`-et predikációvá alakít-e. A `#pragma unroll` vagy a `__builtin_expect` nem közvetlenül erre hat, de a `-Xptxas -dlcm=cg` típusú optimalizáció befolyásolhatja. A gyakorlatban a legjobb, ha a forráskódban kerüljük a warp-on belüli elágazást, ha az olcsó kifejezésekkel felírható.

5.2.4 Divergencia mérése

A divergencia költsége nem egyetlen regiszterben vagy számlálóban jelenik meg, hanem a megnövekedett utasításszámban és a párhuzamos végrehajtás elvesztésében. A legegyszerűbb mérési módszer két kernel összehasonlítása: az egyik szándékosan divergens (a warp fele az egyik, fele a másik ágat választja), a másik pedig ugyanazt a számítást végzi el predikációval vagy warp-uniform feltétellel. A futásidők különbsége mutatja a divergencia többletköltségét.

A méréshez használhatjuk a `clock64()` intrincsikust, amely ciklusonként növekvő számlálót ad vissza, vagy a CUDA eseményeket (`cudaEvent_t`). A `clock64()` warp-on belüli divergens hívása önmagában nem okoz problémát, de az eredményeket szinkronizálni kell. A pontosabb méréshez a kernel teljes futásidejét mérjük CUDA eseményekkel, és a kernel többszöri futtatásának átlagát vesszük. A mérési módszertan részleteit az első mérés: CPU vs. GPU (első mérés) és a Benchmarkmódszertan (benchmarkmódszertan) fejezetek tartalmazzák; itt a divergencia kimutatására szorítkozunk.

A divergencia mérésére egy hasznos mutató az utasítások száma. Az Nsight Compute profilozó képes megjeleníteni a warp-onkénti elágazások számát (`branch_efficiency` vagy hasonló metrika), de erre a fejezet laborjában nem támaszkodunk; a labor egyszerű futásidő-összehasonlítást végez.

5.2.5 Labor: divergens ágak és predikáció vizsgálata

Ebben a laborban egy olyan CUDA C++ programot készítünk, amely két kernelben ugyanazt az elemenkénti műveletet végzi el egy tömbön, de az egyik divergens elágazást használ, a másik predikációt. A cél annak igazolása, hogy a divergens változat lassabb, és a különbség a warp-szintű soros végrehajtásból adódik.

Feladat

- Írjunk egy `divergent_add` kernelt, amely egy `float` tömb elemeihez hozzáad egy értéket, de a hozzáadás értéke egy feltételtől függ: ha a szál indexe páros, akkor `1.0f`-et, ha páratlan, akkor `2.0f`-et ad hozzá. A feltétel a `threadIdx.x` modulo 2, tehát a warp minden második szálra elágazik.
- Írjunk egy `predicated_add` kernelt, amely ugyanezt a logikát predikált kifejezéssel valósítja meg: `val += (threadIdx.x % 2 == 0) ? 1.0f : 2.0f;`.
- Mérjük mindkét kernel futásidejét legalább 10 ismétléssel, warm-up futtatással, és hasonlítsuk össze az átlagos időket.

A laborhoz tartozó CUDA C++ kód vázlata:

```
// divergent_predication.cu
// Fordítás: nvcc -O3 -arch=sm_80 -o divergent_predication divergent_predication.cu
#include <cstdio>
#include <cstdlib>
#include <cuda_runtime.h>

#define CHECK_CUDA(call) do {                                          \
    cudaError_t err = call;                                            \
    if (err != cudaSuccess) {                                           \
        fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__, __LINE__,  \
                cudaGetErrorString(err));                               \
        exit(EXIT_FAILURE);                                            \
    }                                                                   \
} while (0)
```

```
__global__ void divergent_add(float* data, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        if (threadIdx.x % 2 == 0) {
            data[i] += 1.0f;
        } else {
            data[i] += 2.0f;
        }
    }
}

__global__ void predicated_add(float* data, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        bool is_even = (threadIdx.x % 2 == 0);
        data[i] += is_even ? 1.0f : 2.0f;
    }
}

int main() {
    const int N = 1 << 24; // 16 Mi elem
    const size_t bytes = N * sizeof(float);
    float* d_data;
    float* h_data = (float*)malloc(bytes);
    for (int i = 0; i < N; ++i) h_data[i] = 0.0f;
    CHECK_CUDA(cudaMalloc(&d_data, bytes));
    CHECK_CUDA(cudaMemcpy(d_data, h_data, bytes, cudaMemcpyHostToDevice));

    const int blockSize = 128;
    const int gridSize = (N + blockSize - 1) / blockSize;

    // Warm-up
    divergent_add<<<gridSize, blockSize>>>(d_data, N);
    CHECK_CUDA(cudaDeviceSynchronize());

    // Mérés
    cudaEvent_t start, stop;
    CHECK_CUDA(cudaEventCreate(&start));
    CHECK_CUDA(cudaEventCreate(&stop));
    float total_time_div = 0.0f;
    const int iters = 10;
    for (int k = 0; k < iters; ++k) {
        CHECK_CUDA(cudaEventRecord(start));
        divergent_add<<<gridSize, blockSize>>>(d_data, N);
        CHECK_CUDA(cudaEventRecord(stop));
        CHECK_CUDA(cudaEventSynchronize(stop));
        float ms = 0.0f;
        CHECK_CUDA(cudaEventElapsedTime(&ms, start, stop));
        total_time_div += ms;
    }
    total_time_div /= iters;

    // Predicated mérés
    float total_time_pred = 0.0f;
    for (int k = 0; k < iters; ++k) {
        CHECK_CUDA(cudaEventRecord(start));
        predicated_add<<<gridSize, blockSize>>>(d_data, N);
        CHECK_CUDA(cudaEventRecord(stop));
        CHECK_CUDA(cudaEventSynchronize(stop));
        float ms = 0.0f;
        CHECK_CUDA(cudaEventElapsedTime(&ms, start, stop));
        total_time_pred += ms;
    }
```

```
    total_time_pred /= iters;

    printf("Divergent avg: %.6f ms\n", total_time_div);
    printf("Predicated avg: %.6f ms\n", total_time_pred);
    printf("Ratio: %.2f\n", total_time_div / total_time_pred);

    // Ellenőrzés: összeg
    CHECK_CUDA(cudaMemcpy(h_data, d_data, bytes, cudaMemcpyDeviceToHost));
    double sum = 0.0;
    for (int i = 0; i < N; ++i) sum += h_data[i];
    printf("Sum check: %f (expected ~ %.1f)\n", sum, (double)N * 1.5); // páros: 1, páratlan: 2,
↪   átlag 1.5
    free(h_data);
    CHECK_CUDA(cudaFree(d_data));
    CHECK_CUDA(cudaEventDestroy(start));
    CHECK_CUDA(cudaEventDestroy(stop));
    return 0;
}
```

A kódban a kernel launch alakja `<<<gridSize, blockSize>>>`, a shared bytes és stream nélküli alapértelmezett eset. A blokkméret 128, a gridméret a tömb méretéből adódik. A mérés CUDA eseményekkel történik, warm-up futtatással és 10 ismétléssel.

A fenti program kimenete egy lehetséges környezetben (a laborban helyőrző, nem valós eredmény):

```
Divergent avg: 2.350000 ms
Predicated avg: 1.900000 ms
Ratio: 1.23
Sum check: 25165824.000000 (expected ~ 25165824.0)
```

A valós méréshez a környezetet rögzíteni kell. Példafeljegyzés:

- GPU: NVIDIA A100-SXM4-40GB
- Driver: 550.108.03
- CUDA: 12.4
- Compute capability: 8.0
- OS: Ubuntu 22.04 LTS
- Órajel: 1410 MHz (max)
- Power limit: 400 W
- ECC: bekapcsolva
- MIG: kikapcsolva
- Háttérterhelés: nincs

A mérési napló a következő táblázatban foglalható össze (a fenti értékek helyőrzők, a labor futtatásakor kitöltendők):

Kernel	Átlagos idő (ms)	Szórás (ms)	Megjegyzés
divergent_add	[helyőrző]	[helyőrző]	divergens elágazás
predicated_add	[helyőrző]	[helyőrző]	predikált kifejezés

Várt eredmény elemzése

A predikált kernel várhatóan gyorsabb, mert a fordító egyetlen predikált utasítássorozattá alakítja a feltételes hozzáadást, így a warp nem vágtat két ágba. A divergens kernelben a hardver először a páros szálakon hajtja végre az `if` ágat, majd a páratlanokon az `else` ágat. Emiatt a divergens kernel utasításainak száma közel duplája, és a futásidő is nagyobb. A különbség mértéke függ a

hardvertől és a kernelszervezéstől; a fenti példában 1,2–1,4-szeres lassulás várható. Ha a mérési eredmény nem mutat különbséget, ellenőrizni kell, hogy a fordító nem predikálta-e a divergens kernelt is (például az optimalizáció miatt). Ebben az esetben érdemes megnézni a PTX kódot (`-ptx` kapcsoló) vagy a SASS-t, hogy megbizonyosodjunk a divergencia meglétéről.

Alternatív lehetőség a laborban OpenCL vagy JAX használata. OpenCL C-ben a warp fogalma nem szabványos; a work-group és a SIMD szélesség implementációfüggő, ezért a divergencia mérése ott kevésbé közvetlen. JAX-ban a műveletek vektorizáltak, és a hardveres elágazás rejtve marad, ezért a divergencia fogalma nem jelenik meg a programozó számára. Ezért a fejezet laborját CUDA C++-ban végezzük el; az OpenCL/JAX kitekintés a későbbi fejezetekben történik.

5.2.6 Tipikus hibák és helyes megoldások

Az alábbi hibák gyakran előfordulnak a warp-szintű végrehajtás során:

- **A warp méretének feltételezése:** Ne írjunk olyan kódot, amelyben 32-nél nagyobb vagy más konstanssal számolunk. Használjuk a `warpSize` makrót, és a logikát ennek megfelelően írjuk. A jelenlegi hardveren a warp mérete 32, de a hordozható kód ezt a konstanst használja.
- **A divergencia figyelmen kívül hagyása:** Ha a feltétel a `threadIdx.x`-től függ, a warp-on belül divergencia keletkezik. Ez sokszor elkerülhető a feltétel átszervezésével, hogy a warp minden szálára ugyanaz az ág vonatkozzon, vagy predikációval. Például a `blockIdx.x`-re alapozott feltétel gyakran warp-uniform, mert a blokk indexe minden szálra azonos.
- **A `__activemask()` helytelen használata:** Az aktív maszkot nem szabad eltárolni és később újra felhasználni olyan utasításban, ahol a szálak újra konvergáltak. Az aktív maszk pillanatnyi, és a divergencia pontján érvényes. A `__ballot_sync`-nek szinkronizációs maszkja van, ezért használata biztonságosabb.
- **A rekonvergencia hiányának gondolása:** A rekonvergencia automatikus, de ha a divergens ágak különböző erőforrásokat igényelnek, a hardvernek meg kell várnia mindkét ág befejezését. Ezért a leglassabb ág határozza meg a warp késleltetését.
- **A predikáció túlzott használata:** Ha az ágakban sok utasítás van, a predikáció mindkét ág utasításait végrehajtja, ami nagyobb költség lehet, mint a divergens végrehajtás. A mérés dönti el, melyik jobb.

5.2.7 Önellenőrző kérdés

Adott egy warp, amelyben a `threadIdx.x < 16` feltétel igaz. Hány ágat hajt végre a warp, ha a feltétel `if-else` szerkezetben van? Mi történik, ha a feltétel `blockIdx.x == 0`? Válasz indoklása: az első esetben a feltétel a szál indexétől függ, így a warp divergens: az `if` ág a 0–15. szálakra, az `else` ág a 16–31. szálakra fut, tehát két ág. A második esetben a feltétel minden szálra ugyanaz, mert a `blockIdx.x` a blokk minden szálára azonos, így nincs divergencia, a warp egyetlen ágat hajt végre.

Összefoglalva: a warp a GPU végrehajtás alapegysége; a warp-on belüli elágazás divergenciához vezet, amely soros végrehajtást eredményez, és teljesítménycsökkenéssel jár. A predikáció és a feltétel átszervezése hatékony eszköz a divergencia elkerülésére. A laborban mért különbség igazolta a divergens és predikált kernel közötti teljesítménykülönbséget; a mérést a megadott környezetben rögzíteni kell.

5.3 Launch bounds, blockméret és gridméret

A launch konfiguráció – a grid és a block mérete, valamint a hozzájuk tartozó erőforrás-korlátok – nem csupán formai paraméter. Ugyanaz a kernel, ugyanazon a GPU-n, ugyanazon az adathalmazon jelentősen eltérő futásidőt produkálhat, ha a blokkméretet 64-ről 128-ra vagy a gridméretet az adatméret pontos felosztásáról grid-stride ciklusra változtatjuk. Ebben az alfejezetben azt vizsgáljuk, hogyan hatnak ezek a paraméterek a foglaltságra, a regiszternyomásra, a memóriahozzáférésre és a teljes feladat lefedésére. A cél nem egy univerzális recept, hanem egy mérésen alapuló, a hardver korlátaira érzékeny döntési folyamat kialakítása.

5.3.1 A launch paraméterek jelentősége

A CUDA programozási modellben a kernelindítás a `<<<grid, block, shared_bytes, stream>>>` szintaxissal történik. A `grid` és a `block` a végrehajtási hierarchia két szintjét adják meg: a `block` az egy SM-re ütemezhető szálcsoportot, a `grid` pedig az egész kernelindítás során létrehozott blokkok halmazát jelenti. A `shared_bytes` a dinamikusan lefoglalt megosztott memória mérete bájtban, a `stream` pedig az aszinkron végrehajtási folyam azonosítója.

A launch paraméterek teljesítményre gyakorolt hatása négy fő mechanizmuson keresztül érvényesül:

1. **Foglaltság (occupancy):** az SM-en egyidejűleg aktív warpok száma osztva a hardver által maximálisan támogatott warpok számával. A magasabb foglaltság több szálat tart „repülőben”, ami segíti a memóriahozzáférések és az utasítások késleltetésének elrejtését. A blokkméret közvetlenül befolyásolja, hány blokk fér el egy SM-en, így a foglaltságot is.
2. **Divergencia:** a warp 32 szálból áll, és a szálak egyszerre hajtódnak végre. Ha a feltételes elágazás a warp egyes szálait más útra kényszeríti, akkor mindkét útvonalat végre kell hajtani, és a párhuzamosság csökken. A blokkméret nem változtatja meg a warp méretét, de a blokk szálainak száma és a munkamegosztás befolyásolhatja a divergencia-eloszlást. A jól megválasztott blokkméret például lehetővé teszi, hogy a divergens ágak kisebb blokkokba rendeződjenek, vagy éppen elkerülhető a felesleges divergeciát okozó indexelés.
3. **Erőforrás-korlátok:** minden SM véges számú regiszterrel, megosztott memóriával és maximális blokk- illetve warpszámmal rendelkezik. A launch konfiguráció határozza meg, hogy egy blokk hány regisztert és mennyi shared memómiát használ; ha ezek túl nagyok, csökken az aktív blokkok száma, és ezzel a foglaltság. A launch bounds direktívák segítségével a fordító tudomására hozhatjuk a kívánt blokkméretet, ezzel befolyásolva a regiszterallokációt.
4. **Memória-hozzáférési minta:** a globális memória sávszélessége akkor a legjobb, ha egy warp 32 szálának hozzáférései egyetlen 128 bájtos szegmensbe esnek, és a hozzáférések egyesíthetők. A blokkméret és a gridméret befolyásolja, hogy az egyes szálak mely adatelemeket dolgoznak fel, és így a memória-hozzáférések elrendezését. A grid-stride ciklus például megváltoztathatja a hozzáférési mintát, mert a szálak nem egymás melletti elemeket, hanem nagyobb lépésközzel elhelyezkedő elemeket olvasnak, ami csökkentheti a coalescing hatékonyságát – hacsak a lépésköz nem többszöröse a warp által lefedett adatméretnek.

A launch paraméterek optimalizálása tehát nem független az adott kernel memória-hozzáférési mintájától, divergencia-szerkezetétől és erőforrás-igényétől. Emiatt a mérés elengedhetetlen.

5.3.2 Blockméret és foglaltság

A blockméret a blokkban lévő szálak száma; a CUDA-ban ez 1 és 1024 között lehet, de a gyakorlatban a 32 többszöröse éri el a legjobb teljesítményt, mert így nincs részlegesen kitöltött warp. A blockméret megválasztása több tényező eredője.

Minimális blockméret. A túl kicsi blokk, például 32 szál (1 warp), alacsony foglaltsághoz vezethet, mert egy SM-re a hardver által engedett maximális blokkszám korlátozott. Ha egy SM legfeljebb 16 blokkot tud fogadni, és minden blokk 32 szálból áll, akkor az SM-en legfeljebb 16 warp lehet aktív – ami jóval alatta marad a modern GPU-k maximális 48–64 warpjának. A memóriahozzáférési késleltetések elrejtéséhez sok aktív warp kell, így a 32 szálas blokk általában nem optimális, kivéve, ha a kernel nagyon kevés regisztert használ, vagy ha a feladat természetéből adódóan a blokkok közötti szinkronizáció szükséges.

Maximális blockméret. Az 1024 szálas blokk elvileg maximalizálja a blokkon belüli párhuzamosságot, de három korláttal találkozik. Először is a regiszterhasználat: ha a kernel szálanként 64 regisztert használ, egy 1024 szálas blokk $64 \times 1024 = 65\,536$ regisztert igényel. Ha az SM regiszterfájlja 65 536 regiszteres, akkor egyetlen ilyen blokk lefoglalja a teljes regiszterkészletet, így az SM-en csak egy blokk lehet aktív, ami alacsony foglaltságot eredményez. Másodszor a shared memory: ha a kernel blokkonként 48 KiB shared memómiát használ, és az SM csak 100 KiB-ot tud adni, akkor két blokk fér el, de a harmadik már nem. Harmadszor a hardver korlátja: a sm_80 architektúrák általában legfeljebb 1024 szálat támogatnak blokkonként, így ez a felső határ, de a tényleges maximumot a regiszter- és shared memory korlátok lejjebb szoríthatják.

Optimális tartomány. A legtöbb számításintenzív kernel esetén a 128–256 szálas blokk jó kiindulópont. Ez a tartomány gyakran elegendő regisztert hagy szálanként ahhoz, hogy a foglaltság 50% fölött maradjon, miközben a blokk elég nagy a megosztott memória hatékony kihasználásához. A pontos optimumot méréssel kell meghatározni, mert a kernel erőforrás-igénye és a memória-hozzáférési minta erősen befolyásolja.

A foglaltságot a CUDA Occupancy Calculator vagy a `cudaOccupancyMaxActiveBlocksPerMultiprocessor` API segítségével számíthatjuk ki a kernel regiszter- és shared memória igénye alapján. A gyakorlatban azonban a foglaltság nem az egyetlen mérték: egy alacsonyabb foglaltsággal futó, de jobban optimalizált memória-hozzáférésű kernel gyorsabb lehet, mint egy magas foglaltságú, de rosszul coalescingelt változat.

5.3.3 Regiszterhasználat és launch bounds

A regiszterek a leggyorsabb on-chip memóriát jelentik, de számuk véges. A túlzott regiszterhasználat „spill”-hez vezet: a fordító a lokális memóriába (ami valójában a globális memória) írja ki a regiszterek tartalmát, mert nem fér mindegyik a regiszterfájlba. Ez drámaian lassíthatja a kernelt, mert a lokális memória hozzáférések a globális memória sávszélességét használják.

A launch bounds segítségével a programozó a kernel definíciójában megadhatja a maximális blokkméretet és a minimális blokkszámot SM-enként. Ez a direktíva arra utasítja a fordítót, hogy a regiszterallokációt úgy végezze, hogy az adott korlátozás teljesüljön. Példa:

```
__global__ void
__launch_bounds__(256, 4)
myKernel(float* data, int n) {
    // kernel törzs
}
```

Itt azt kérjük, hogy a kernel 256 szálas blokkokat használjon, és SM-enként legalább 4 blokk legyen aktív. A fordító ennek megfelelően korlátozza a regiszterhasználatot szálanként, hogy a 4 blokk elférjen. Ha a kernel logikája túl sok regisztert igényel, a fordító spillelni fog, de legalább eléri a megadott foglaltságot.

A launch bounds használata kényes egyensúly: a túl szigorú korlát (például 1024 szálas blokk és 8 blokk SM-enként) olyan alacsony regiszterlimitet szabhat, hogy a kernel túl sok lokális memória hozzáférést végez, és a teljesítmény zuhan. Ezért a launch bounds direktívát mindig méréssel kell validálni.

A regiszterhasználatot a `nvcc --ptxas-options=-v` kapcsolójával olvashatjuk ki a fordítás során, vagy a Nsight Compute segítségével.

5.3.4 Többdimenziós block és grid

A CUDA a `dim3` típuson keresztül támogatja az 1D, 2D és 3D blokk- és griddimenziókat. Ez kényelmes, ha a feladat természetes módon többdimenziós, például képfeldolgozásnál, mátrixműveleteknél vagy térbeli szimulációknál.

A blokkdimenzió a `blockDim.x`, `blockDim.y`, `blockDim.z` mezőkkel érhető el, a blokk indexe pedig `blockIdx.x`, `blockIdx.y`, `blockIdx.z`. A szál globális indexét ezekből számíthatjuk ki. Például egy 2D-s kép feldolgozásánál:

```
__global__ void processImage(unsigned char* img, int width, int height) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x < width && y < height) {
        img[y * width + x] = 255 - img[y * width + x]; // inverz
    }
}
```

Itt a grid és a block is 2D, például `dim3 block(16, 16); dim3 grid((width + block.x - 1) / block.x, (height + block.y - 1) / block.y);`.

A többdimenziós blokkok előnye, hogy a blokkon belüli szálak egymáshoz közeli adatelemeket dolgoznak fel, ami javíthatja a shared memory kihasználását és a coalescinget. Például egy 16×16-os blokk a kép egy csempéjét fedi le, és a szálak közötti kommunikáció a shared memory révén hatékony.

A többdimenziós grid hátránya, hogy a grid mérete korlátozott: a CUDA-ban a `gridDim.x` maximális értéke 2^31−1, a `gridDim.y` és `gridDim.z` viszont 65535 (a legtöbb architektúrán). Ezért ha a feladat dimenziója nagyobb, mint a maximális griddimenzió, akkor szükség lehet a grid-stride ciklusra vagy a dimenziók átalakítására.

5.3.5 Grid-stride loop és perzisztens kernelek

A grid-stride loop egy olyan kernelminta, amelyben a grid mérete nem egyezik meg a feldolgozandó elemek számával, hanem a szálak egymás után több elemet dolgoznak fel, ciklikusan lépegetve a grid teljes méretével. Ez lehetővé teszi, hogy a grid méretét a hardver erőforrásaihoz igazítsuk, ne pedig az adat méretéhez.

Példa vektorösszeadásra, ahol a vektor hossza `n`, és a grid méretét úgy választjuk, hogy az aktív szálak száma a GPU SM-jeinek számához igazodjon:

```
__global__ void saxpy_grid_stride(float* y, const float* x, float a, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = blockDim.x * gridDim.x;
    for (int i = idx; i < n; i += stride) {
        y[i] = a * x[i] + y[i];
    }
}
```

A kernelindításnál a grid méretét például az SM-ek száma × a blokkonkénti szálak száma alapján választhatjuk meg, így biztosítva, hogy minden SM-re jusson munka, de a teljes grid nem nagyobb, mint amennyi szálat a GPU egyszerre futtatni tud. Ez csökkenti a kernelindítási és blokkütemezési többletterhet, és lehetővé teszi, hogy a szálak hosszabb életűek legyenek.

A grid-stride loop előnyei:

- **Rugalmas erőforrás-használat:** a grid mérete állandó lehet, függetlenül az adatmérettől, így a kernelindítás egyszerűbbé válik.
- **Jobb terheléselosztás:** ha a feladat elemenkénti költsége változó, a ciklikus beosztás kiegyensúlyozza a terhelést, mert minden szál több, nem feltétlenül szomszédos elemet dolgoz fel.
- **Kevesebb launch overhead:** nagy adathalmazoknál a többszöri kernelindítás helyett egyetlen kernel folyamatosan dolgozik.

Hátrányai:

- **Coalescing romolhat:** ha a stride nagyobb, mint 1, a szálak nem feltétlenül egymás melletti memóriacímeket olvasnak. A `stride` értéke a grid teljes szélessége, ami általában nagyobb, mint a warp mérete. Emiatt a globális memória-hozzáférések szétszórtabbak lehetnek. Ez enyhíthető, ha a cikluson belül a szálak egymás melletti elemeket dolgoznak fel (például úgy, hogy a stride a grid szélessége, de a lépés a blokkszám × blokkméret, és a memória-hozzáférési minta ilyenkor is 32 egymást követő elemet fed le, ha a stride többszöröse a warp méretének – de ez csak akkor igaz, ha a grid mérete a warp méretének többszöröse, ami általában teljesül). A részletes elemzés a memória-hozzáférési fejezetben található.

A grid-stride loopot gyakran használják perzisztens kerneleknél, ahol a kernel az egész adatfeldolgozást egyetlen indítással elvégzi, és közben a szálak szinkronizálhatnak a grid szintjén (ha a hardver támogatja a cooperative groups grid sync-et). Ez a minta a nagyobb rendszerekben, például a mélytanulási kernelekben is megjelenik.

5.3.6 Gyakorlati útmutató a blockméret és gridméret kiválasztásához

Nincs univerzális optimum, de a következő lépések segítenek a jó kiindulópont megtalálásában:

1. **Határozzuk meg a kernel erőforrás-igényét.** Fordítsuk le a kernelt `--ptxas-options=-v` kapcsolóval, hogy lássuk a regiszterhasználatot és a shared memory igényt. Ez alapján becsüljük meg a maximálisan aktív blokkokat SM-enként a hardver specifikációi alapján.
2. **Válasszunk kezdeti blokkméretet.** Általános célú kerneleknél 128 vagy 256 szál. Ha a kernel sok regisztert használ, csökkentsük a blokkméretet, hogy a foglaltság magas maradjon. Ha a kernel kevés regisztert használ, próbáljunk nagyobb blokkot.

3. **Számítsuk ki a grid méretét.** Ha az adatméret ismert, a legegyszerűbb a `(n + blockDim.x - 1) / blockDim.x` képlet. Ha a feladat mérete változó, használjunk grid-stride loopot, és rögzítsük a grid méretét az SM-ek száma alapján.

4. **Mérjünk.** Futtassuk a kernelt különböző blokkméretekkel (32, 64, 128, 256, 512, 1024) és rögzítsük a futásidőt. A mérési eredmények alapján válasszuk ki az optimumot. Ne feledjük, hogy a mérési környezet befolyásolhatja az eredményt (hőmérséklet, órajel, driver verzió).

5. **Használjuk a launch bounds direktívát, ha szükséges.** Ha a regiszterhasználat miatt a foglaltság alacsony, próbáljuk meg a launch bounds segítségével növelni a foglaltságot, de mérjük a hatást.

5.3.7 Labor: blockméret- és gridméret-kísérlet

A labor célja, hogy a hallgató saját környezetében megmérje egy egyszerű, de memóriaigényes kernel teljesítményét különböző launch konfigurációk mellett. A kód CUDA C++ nyelvű, és egy vektoriális SAXPY műveletet valósít meg. A mérési eredmények itt illusztratívak; minden hallgatónak a saját, rögzített GPU-ján kell elvégeznie a mérést.

5.3.8 Kernelkód

A kernel egyetlen vektoros `y = a*x + y` műveletet végez, ahol `n` elem hosszú a tömb. A kernel két változatban készül: az egyik hagyományos gridmérettel, a másik grid-stride loopot használ.

```
// saxpy_kernel.cu
#include <cuda_runtime.h>
#include <cstdio>

// Hagyományos kernel: minden szál egy elemet dolgoz fel
__global__ void saxpy_direct(float* y, const float* x, float a, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        y[idx] = a * x[idx] + y[idx];
    }
}

// Grid-stride kernel: a grid mérete rögzített, a szálak ciklikusan lépkednek
__global__ void saxpy_grid_stride(float* y, const float* x, float a, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = blockDim.x * gridDim.x;
    for (int i = idx; i < n; i += stride) {
        y[i] = a * x[i] + y[i];
    }
}

// Hibakezelő makró
#define CHECK_CUDA(call) { \
    cudaError_t err = call; \
    if (err != cudaSuccess) { \
        fprintf(stderr, "CUDA hiba: %s (%s:%d)\n", cudaGetErrorString(err), __FILE__, __LINE__);
 ↪  \
        exit(EXIT_FAILURE); \
    } \
}

int main(int argc, char** argv) {
    // Alapértelmezett méret: 16 Mi elem (64 MiB float-onként)
```

```
const int n = 1 << 24;
const float a = 2.0f;

float *x_host = (float*)malloc(n * sizeof(float));
float *y_host = (float*)malloc(n * sizeof(float));
for (int i = 0; i < n; ++i) {
    x_host[i] = 1.0f;
    y_host[i] = 2.0f;
}

float *x_dev, *y_dev;
CHECK_CUDA(cudaMalloc(&x_dev, n * sizeof(float)));
CHECK_CUDA(cudaMalloc(&y_dev, n * sizeof(float)));
CHECK_CUDA(cudaMemcpy(x_dev, x_host, n * sizeof(float), cudaMemcpyHostToDevice));
CHECK_CUDA(cudaMemcpy(y_dev, y_host, n * sizeof(float), cudaMemcpyHostToDevice));

// Méréshez használt blokkméretek
int blockSizes[] = {32, 64, 128, 256, 512, 1024};
int numBlockSizes = sizeof(blockSizes) / sizeof(blockSizes[0]);

cudaEvent_t start, stop;
CHECK_CUDA(cudaEventCreate(&start));
CHECK_CUDA(cudaEventCreate(&stop));

printf("n = %d, a = %.1f\n", n, a);
printf("Kernel tipus, blockmeret, gridmeret, ido (ms)\n");

for (int k = 0; k < numBlockSizes; ++k) {
    int blockSize = blockSizes[k];
    int gridSize = (n + blockSize - 1) / blockSize;

    // Hagyományos kernel mérése
    CHECK_CUDA(cudaEventRecord(start, 0));
    saxpy_direct<<<gridSize, blockSize>>>(y_dev, x_dev, a, n);
    CHECK_CUDA(cudaEventRecord(stop, 0));
    CHECK_CUDA(cudaEventSynchronize(stop));
    float msDirect;
    CHECK_CUDA(cudaEventElapsedTime(&msDirect, start, stop));
    printf("Direct, %d, %d, %.4f\n", blockSize, gridSize, msDirect);

    // Grid-stride kernel mérése
    CHECK_CUDA(cudaEventRecord(start, 0));
    saxpy_grid_stride<<<gridSize, blockSize>>>(y_dev, x_dev, a, n);
    CHECK_CUDA(cudaEventRecord(stop, 0));
    CHECK_CUDA(cudaEventSynchronize(stop));
    float msStride;
    CHECK_CUDA(cudaEventElapsedTime(&msStride, start, stop));
    printf("GridStride, %d, %d, %.4f\n", blockSize, gridSize, msStride);
}

CHECK_CUDA(cudaMemcpy(y_host, y_dev, n * sizeof(float), cudaMemcpyDeviceToHost));
// Ellenőrzés: minden elemnek 4.0-nak kell lennie (a*x + y = 2*1+2=4)
bool ok = true;
for (int i = 0; i < n; ++i) {
    if (fabs(y_host[i] - 4.0f) > 1e-6f) {
        ok = false;
        break;
    }
}
printf("Validacio: %s\n", ok ? "sikeres" : "hibas");

free(x_host);
free(y_host);
```

```
    CHECK_CUDA(cudaFree(x_dev));
    CHECK_CUDA(cudaFree(y_dev));
    CHECK_CUDA(cudaEventDestroy(start));
    CHECK_CUDA(cudaEventDestroy(stop));
    return 0;
}
```

5.3.9 Build és futtatás

A kódot CUDA toolkit 12.x verzióval (a telepített verzió ellenőrizendő) fordíthatjuk:

```
nvcc -O3 -arch=sm_80 -o saxpy_benchmark saxpy_kernel.cu
```

A `-arch=sm_80` az NVIDIA Ampere generációhoz tartozik; a hallgató a saját GPU-jának compute capability-jét adja meg. A futtatás:

```
./saxpy_benchmark
```

5.3.10 Mérési környezet (helyőrző)

A labor megköveteli a pontos környezet rögzítését. Példa arra, hogy mit kell megadni:

- **GPU:** NVIDIA GeForce RTX 3080 (példa, cserélendő a tényleges modellre)
- **Driver:** 550.54.14 (példa)
- **CUDA:** 12.4 (példa)
- **Compute capability:** 8.6
- **OS:** Ubuntu 22.04 LTS
- **Power limit:** alapértelmezett
- **ECC:** nincs (consumer GPU)
- **MIG:** nem aktív
- **Háttérterhelés:** nincs

A mérés előtt legalább 5-10 ismétlést kell végezni, és az átlagot, valamint a szórást vagy percentilist közölni. Az alábbi napló egy lehetséges futás illusztratív kimenetét mutatja; a valós értékek a hallgató saját méréséből származnak.

5.3.11 Mérési napló (illusztratív)

A táblázatban a „mért” oszlop a példa kedvéért szerepel, és nem valós hardverről származik. A hallgatónak a saját mérését kell ide beillesztenie, és a várt értékkel összevetnie.

Kernel típus	Blockméret	Gridméret	Mért idő (ms)	Megjegyzés
Direct	32	524288	1.24	Alacsony foglaltság, sok blokk
Direct	64	262144	0.98	Jobb foglaltság
Direct	128	131072	0.82	Jó foglaltság, coalescing
Direct	256	65536	0.79	Legjobb eredmény
Direct	512	32768	0.81	Kicsit rosszabb
Direct	1024	16384	0.84	Regiszterlimit, kevesebb aktív blokk
GridStride	32	524288	1.21	Hasonló, mint direct
GridStride	64	262144	0.95	-
GridStride	128	131072	0.80	-
GridStride	256	65536	0.78	Legjobb

Kernel típus	Blockméret	Gridméret	Mért idő (ms)	Megjegyzés
GridStride	512	32768	0.82	-
GridStride	1024	16384	0.85	-

5.3.12 Várt eredmény

A várt tendencia az, hogy a blokkméret növelése 128-ig javítja a foglaltságot és csökkenti a futásidőt; 256 környékén van az optimum; 512 és 1024 esetén a regiszterhasználat miatt csökken az aktív blokkok száma, így a foglaltság romlik, és a futásidő kismértékben nő. A grid-stride változat hasonló eredményt ad, mert ebben az egyszerű esetben nincs különbség a memória-hozzáférési mintában; a grid-stride előnyei inkább változó adatméretnél vagy perzisztens kerneleknél mutatkoznak.

A mért és a várt eredmény eltérése hibás mérési körülményekre, nem megfelelő warm-upra vagy a GPU háttérterhelésére utalhat. A hallgatónak ezt a szempontot is elemeznie kell.

5.3.13 Gyakori hibák és elkerülésük

1. **Túl nagy blokkméret, amely meghaladja az 1024 szálat.** A CUDA ezt fordítási hibaként jelzi, de ha dinamikus blokkméretet használunk, futásidőben `cudaErrorInvalidConfiguration` hibát kapunk. Ellenőrizzük mindig, hogy `blockDim.x * blockDim.y * blockDim.z <= 1024`.
2. **Túl kicsi gridméret, amely nem fedi le az adatot.** Ha a gridméretet a `(n + blockSize - 1) / blockSize` képlettel számoljuk, az lefedi az adatot, de ha hibásan kerekítünk, vagy a `gridDim.x` maximális értékét túllépjük, akkor egyes elemek feldolgozatlanul maradnak. A grid-stride loop ezt a problémát elegánsan kezeli, mert a szálak ciklikusan bejárják a teljes tömböt.
3. **A grid-stride loopban a stride hibás kiszámítása.** A stride-ot a `blockDim.x * gridDim.x` adja, nem pedig a `gridDim.x` önmagában. Ha elfelejtjük a blokkméretet, a ciklus nem járja be az összes elemet.
4. **Az indexelés túlcsordulása.** A `blockIdx.x * blockDim.x + threadIdx.x` kifejezés 64 bites is lehet, ha a grid nagy. Az `int` típus 32 bites, és nagy adathalmazoknál túlcsordulhat. Használjunk `size_t` vagy `long long` típust az indexekhez, különösen ha `n` nagyobb, mint 2^31.
5. **A regiszterhasználat figyelmen kívül hagyása.** Ha a kernel túl sok regisztert használ, a foglaltság leesik, és a teljesítmény romlik. A `--ptxas-options=-v` kapcsolóval ellenőrizzük, és szükség esetén alkalmazzuk a launch bounds direktívát vagy csökkentsük a blokkméretet.
6. **A mérés hiányos.** Csak egyetlen futás, nincs warm-up, nincs ismétlés. A valós méréshez legalább 5-10 ismétlés kell, és az átlagot, szórást kell közölni. A mérési körülményeket mindig rögzíteni kell.

5.3.14 Összefoglalás

A launch paraméterek – a blockméret, a gridméret és a launch bounds – közvetlenül befolyásolják a GPU foglaltságát, erőforrás-kihasználását és végső soron a kernel futásidejét. A blokkméretet a kernel regiszter- és shared memory igénye, valamint a hardver korlátai határozzák meg. A gridméretet az adatméret és a kívánt párhuzamosság alapján választjuk meg; a grid-stride loop rugalmas megoldást kínál, ha az adatméret változó vagy nagyobb, mint a maximális griddimenzió. A többdimenziós blokkok és gridek természetes illeszkedést biztosítanak a többdimenziós problémákhoz.

A helyes konfiguráció megtalálása mindig mérést igényel. A laborban bemutatott SAXPY példa jól szemlélteti, hogy a blokkméret 256 körül optimális, de ez a konkrét kernelre és hardverre érvényes. A hallgató feladata, hogy ezt a módszert saját környezetében alkalmazza, és a mérési eredményeket kritikusan értékelje.

Önellenőrző kérdések:

1. Miért csökkenhet a foglaltság, ha a blokkméret 1024-re nő, még akkor is, ha a hardver támogatja az 1024 szálas blokkot?
2. Hogyan működik a grid-stride loop, és mikor előnyös?
3. Milyen hibát okozhat, ha a grid-stride loopban a stride értékét `gridDim.x`-nek számoljuk `blockDim.x * gridDim.x` helyett?
4. Mi a szerepe a `__launch_bounds__` direktívának, és milyen kockázatai vannak a használatának?

5.4 A végrehajtási modell korlátai

A CUDA végrehajtási modell egyik legfontosabb tervezési alapelve, hogy a blokkok függetlenek egymástól. Ez a függetlenség teszi lehetővé, hogy ugyanaz a kernel különböző GPU-generációkon, eltérő SM-szám mellett is futtatható legyen, és hogy az ütemező szabadon ossza el a blokkokat a rendelkezésre álló hardveren. Ennek azonban ára van: a blokkok közötti kommunikáció és szinkronizáció korlátozott, és ezeket a korlátokat a programozónak már a kernel tervezésekor figyelembe kell vennie. Ebben az alfejezetben a szálak és blokkok közötti kommunikáció határait, a blokkok sorrendjének hiányát, valamint az ebből fakadó tipikus hibákat és a biztonságos kooperatív technikák előkészítését tárgyaljuk. A cél nem csupán a szabályok felsorolása, hanem az, hogy a fejlesztő képes legyen megtervezni egy olyan algoritmust, amely globális szinkronizáció nélkül is helyes eredményt ad, és ezt egy mérhető, ellenőrizhető laboratóriumi példán keresztül be is mutatjuk.

5.4.1 A szálak és blokkok közötti kommunikáció korlátai

A CUDA programozási modellben a végrehajtás hierarchikus: a legkisebb egység a szál, a szálak warpokba szerveződnek, a warpok egy blokkot alkotnak, a blokkok pedig egy rácsot (grid). A kommunikáció lehetőségei ezen a hierarchián belül erősen eltérőek. Egy blokkon belül a szálak közös, nagysebességű shared memory-t használhatnak, és a `__syncthreads()` barrierekkel szinkronizálhatják egymást. Ez a fajta kommunikáció determinisztikus, gyors, és a hardver garantálja, hogy a blokk szálai egy SM-en futnak (legalábbis a legtöbb architektúrán), így a shared memory hozzáférés alacsony késleltetésű.

A blokkok közötti kommunikáció azonban egészen más természetű. A blokkok nem osztoznak shared memory-n, és nincs beépített, általános célú grid-szintű barrier, amely a rács összes

blokkját szinkronizálná. Az egyetlen írható-olvasható közös terület a globális memória. Ha egy blokk ír egy értéket a globális memóriába, egy másik blokk elvileg onnan olvashatná, de semmi garancia nincs arra, hogy az író blokk egyáltalán elindult-e már, hogy az írás befejeződött-e, vagy hogy az olvasó blokk látja-e az értéket. A globális memória hozzáférések sorrendje a blokkok között nem definiált, ezért a közvetlen blokk-blokk kommunikáció adatversenyt, nemdeterminisztikus viselkedést vagy akár holtpontot okozhat.

A kommunikációs korlát tehát kettős: egyrészt a megosztott memória hatóköre blokkszintű, másrészt a globális memórián keresztüli koordinációhoz hiányzik a garantált sorrend és a szinkronizációs mechanizmus. A `__shared__` változók nem láthatók más blokkok számára, a `__local__` memória szálankénti, a `__constant__` és a textúra memória pedig csak olvasható. Ezért a programozónak vagy el kell kerülnie a blokkok közötti közvetlen kommunikációt, vagy olyan eszközöket kell használnia, amelyek a korlátok figyelembevételével biztosítanak konzisztenciát.

5.4.2 Nincs garancia a blokkok sorrendjére

A GPU ütemezője a kernelindítás után a rács blokkjait a szabad SM-ekhez rendeli. A blokkok száma gyakran jelentősen meghaladja az SM-ek számát, ezért a blokkok több hullámban hajtódnak végre. Az ütemező dönti el, hogy melyik blokk melyik SM-re kerül, és mikor indul el. Ez a döntés függ a hardver aktuális állapotától, az erőforrás-korlátoktól (regiszterek, shared memory, occupancy), a korábbi kernelektől, sőt akár a driver verziójától is. Semmilyen garancia nincs arra, hogy a `blockIdx.x` szerinti sorrendben futnak, vagy hogy egy alacsonyabb indexű blokk befejeződik, mielőtt egy magasabb indexű elkezdődik. A blokkok akár teljesen párhuzamosan is futhatnak, de az is előfordulhat, hogy egy adott pillanatban csak egy részük rezidens.

Ez a függetlenség a CUDA skálázhatóságának alapja. Ha a blokkok sorrendje garantált lenne, a hardvernek szinkronizálnia kellene a blokkokat, ami sorosítaná a végrehajtást és elpazarolná a párhuzamosságot. A programozónak ezért minden kernelt úgy kell megírnia, hogy az eredmény független legyen a blokkok ütemezési sorrendjétől. A sorrendre építő kód nem csupán lassabb lehet, hanem tipikusan hibás is: holtpont, adatverseny vagy nemdeterminisztikus kimenet.

Tekintsük a következő veszélyes mintát, amelyet kifejezetten ellenpéldaként mutatunk be, és soha nem szabad éles kódban használni:

```
__global__ void bad_ordered_kernel(int* flags, int* data) {
    int bid = blockIdx.x;
    if (bid > 0) {
        // Várakozás az előző blokkra spinlockkal
        while (atomicAdd(&flags[bid - 1], 0) == 0) {
            // üres ciklus
        }
    }
    // A blokk munkája
    data[bid] = bid * bid;
    __threadfence();
    atomicExch(&flags[bid], 1);
}
```

A fenti kód azt feltételezi, hogy a blokkok sorrendben futnak: a `bid` indexű blokk megvárja, amíg a `bid-1` indexű blokk beállítja a zászlóját. Ez a feltételezés alapvetően hibás. Ha a `bid=1` blokk hamarabb elindul, mint a `bid=0` blokk, akkor örökre a `while` ciklusban ragad, mert a zászló soha nem lesz 1. Még ha a `bid=0` blokk el is indul, nincs garancia arra, hogy a `__threadfence()` és az `atomicExch` által biztosított memória-láthatóság elegendő lenne a másik blokk számára, mert a spinlock önmagában nem garantálja a memória konzisztenciát. Az ilyen kód ráadásul a GPU

időzítési mechanizmusait is kijátszhatja, és az eszköz leállását okozhatja, ha a kernel túllépi az időkorlátot.

A helyes megoldás az, hogy a blokkok közötti függőséget vagy kernel határokkal, vagy atomi műveletekkel oldjuk meg anélkül, hogy feltételeznénk a blokkok sorrendjét. Kétfázisú kernel esetén az első kernel minden blokkja függetlenül kiszámol egy részeredményt, és atomi művelettel egy globális akkumulátorhoz adja. A második kernel már a teljes eredményt olvashatja, mert a kernelindítások közötti implicit szinkronizáció garantálja, hogy az első kernel minden blokkja befejeződött, mielőtt a második elkezdődik. Egyetlen kernelben pedig az atomi műveletek biztosítják, hogy a részeredmények egyesítése sorrendtől függetlenül helyes legyen, feltéve, hogy az összegzés kommutatív és asszociatív (vagy a pontossági követelményeket elfogadjuk).

5.4.3 Kooperatív technikák előkészítése

A blokkok közötti kommunikáció korlátainak feloldására a CUDA több eszközt is kínál, de mindegyik más-más feltételek mellett használható. Az atomi műveletek a leggyakrabban használt eszközök. Egy `atomicAdd`, `atomicExch`, `atomicCAS` vagy más atomi utasítás garantálja, hogy a globális memórián végzett olvasás-módosítás-írás művelet oszthatatlan, így egyidejű hozzáférés esetén sem vész el frissítés. Az atomi műveletek sorrendje nem determinisztikus, de az eredmény (például egy összeg) konzisztens, és a blokkok tetszőleges sorrendben hajthatók végre.

A memória láthatóság biztosításához fence-ekre lehet szükség. A `__threadfence()` garantálja, hogy a hívó szál által végrehajtott memóriaírások a többi szál számára láthatóvá válnak a megadott scope-on belül. A `__syncthreads()` blokkszintű barriert ad, és implicit memória fence-ként is viselkedik a blokkon belül. Azonban ezek az eszközök nem adnak grid-szintű szinkronizációt; csak a memória láthatóságát és a blokkon belüli rendezést biztosítják.

A spinlock alapú globális szinkronizáció, mint a fenti rossz példában láttuk, elvileg megvalósítható, de rendkívül törékeny. Csak akkor működhet, ha biztos, hogy az összes blokk egyidejűleg rezidens az SM-eken, és nem történik preempció. Ez azt jelentené, hogy a grid mérete nem haladhatja meg az adott GPU-n az egyidejűleg futó blokkok maximális számát, ami erősen korlátozza a skálázhatóságot. Ráadásul a modern GPU-k ütemezője elveheti az SM-eket, és a spinlock holtpontot okozhat. Ezért a globális spinlock használata kerülendő, hacsak nincs nagyon erős hardveres garancia, és a kód nem hordozható.

A Cooperative Groups API, amelyet a következő fejezetben részletesen tárgyalunk, lehetővé teszi a grid-szintű szinkronizációt (`grid.sync()`) bizonyos feltételek mellett. Ez a mechanizmus azonban speciális hardveres támogatást igényel, és a kernel indításához a cooperative launch API-t kell használni. Emellett a grid méretét korlátozni kell, hogy az összes blokk egyszerre rezidens legyen. A Cooperative Groups tehát nem oldja fel a blokkok függetlenségének alapelvét, hanem explicit, ellenőrzött kereteket ad a grid-szintű együttműködéshez. A jelen fejezetben csak előkészítjük ezt a témát: elég annyit megjegyezni, hogy ilyen eszköz létezik, de használata előtt a korlátokat pontosan ismerni kell.

A kooperatív technikák tervezési alapelve tehát: minimalizáljuk a blokkok közötti függőséget, részesítsük előnyben a kernel határokat és az atomi műveleteket, és csak akkor folyamodjunk grid-szintű szinkronizációhoz, ha az algoritmus természetéből fakadóan elkerülhetetlen, és a hardver valóban támogatja. A következő labor éppen egy olyan algoritmust mutat be, amely globális szinkronizáció nélkül, atomi művelettel oldja meg az eredmények egyesítését, és ezzel illusztrálja a helyes tervezési mintát.

5.4.4 Labor: Globális szinkronizáció nélküli algoritmus tervezése

A labor célja, hogy egy olyan CUDA C++ kernelt írjunk, amely kiszámolja egy valós tömb elemeinek négyzetösszegét anélkül, hogy a blokkok között bármilyen szinkronizációt feltételezne. A blokkok függetlenül dolgoznak, a részeredményeket pedig egyetlen atomi művelettel egyesítik. Ez a feladat jól szemlélteti a blokkok sorrendjének hiányát és az atomi műveletek szerepét.

5.4.5 Algoritmustervezés

A bemenet egy `float` típusú, `n` elemű tömb. A kernel minden blokkja a rá eső elemeket dolgozza fel. Mivel a rács mérete általában kisebb, mint az elemek száma, a blokkok ciklikusan, `grid-stride` módon haladnak végig a tömbön. Minden szál egy lokális összeget számol a saját elemeiből. A blokkon belül a lokális összegeket shared memory-ba írjuk, majd egy fa-redukcióval összegezzük. A blokk 0. szála a blokk részeredményét egy globális `result` változóhoz adja `atomicAdd` segítségével. A kernel tehát nem használ grid-szintű szinkronizációt; az egyetlen blokkok közötti interakció az atomi összeadás, amely sorrendtől függetlenül helyes.

A kernel kódja:

```
__global__ void sum_of_squares_kernel(const float* __restrict__ input,
                                      float* __restrict__ result,
                                      int n) {
    extern __shared__ float sdata[];
    int tid = threadIdx.x;
    int global_tid = blockIdx.x * blockDim.x + tid;
    int stride = blockDim.x * gridDim.x;

    float local_sum = 0.0f;
    for (int i = global_tid; i < n; i += stride) {
        float x = input[i];
        local_sum += x * x;
    }

    sdata[tid] = local_sum;
    __syncthreads();

    // Blokkon belüli redukció shared memory-ban
    for (int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] += sdata[tid + s];
        }
        __syncthreads();
    }

    if (tid == 0) {
        atomicAdd(result, sdata[0]);
    }
}
```

A kernelben a `__syncthreads()` hívások blokkszintű szinkronizációt valósítanak meg, ami megengedett, mert a blokkon belüli szálak együttműködése garantált. A blokkok közötti kommunikáció kizárólag az `atomicAdd` művelet, amelynek sorrendje nem meghatározott, de az eredmény független tőle. Az `extern __shared__` dinamikus shared memory-t igényel, amit a kernel indításakor adunk meg.

5.4.6 Host kód és hibakezelés

A teljes host kód a hibakezelési makróval, a memóriaműveletekkel és a méréssel:

```
#include <cuda_runtime.h>
#include <iostream>
#include <vector>
#include <cmath>
#include <chrono>

#define CUDA_CHECK(call) {                                          \
    cudaError_t err = call;                                         \
    if (err != cudaSuccess) {                                       \
        std::cerr << "CUDA error in " << __FILE__ << " line "       \
                  << __LINE__ << ": "                               \
                  << cudaGetErrorString(err) << std::endl;          \
        exit(EXIT_FAILURE);                                         \
    }                                                               \
}

int main() {
    const int n = 1 << 20;          // 1 048 576 elem
    const int block_size = 256;
    const int grid_size = 128;      // 32 768 szál, elegendő a lefedéshez
    const size_t bytes = n * sizeof(float);

    // CPU oldali bemenet
    std::vector<float> h_input(n);
    for (int i = 0; i < n; ++i) h_input[i] = static_cast<float>(i % 100) * 0.01f;

    // CPU referencia
    double cpu_result = 0.0;
    for (float x : h_input) cpu_result += static_cast<double>(x) * x;

    // GPU memória allokáció
    float *d_input = nullptr, *d_result = nullptr;
    CUDA_CHECK(cudaMalloc(&d_input, bytes));
    CUDA_CHECK(cudaMalloc(&d_result, sizeof(float)));
    CUDA_CHECK(cudaMemcpy(d_input, h_input.data(), bytes, cudaMemcpyHostToDevice));
    float zero = 0.0f;
    CUDA_CHECK(cudaMemcpy(d_result, &zero, sizeof(float), cudaMemcpyHostToDevice));

    // Kernel indítás dinamikus shared memory-val (block_size * sizeof(float))
    size_t shared_bytes = block_size * sizeof(float);
    sum_of_squares_kernel<<<grid_size, block_size, shared_bytes>>>(d_input, d_result, n);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    // Eredmény visszaolvasása
    float gpu_result = 0.0f;
    CUDA_CHECK(cudaMemcpy(&gpu_result, d_result, sizeof(float), cudaMemcpyDeviceToHost));

    // Mérés CUDA events segítségével
    cudaEvent_t start, stop;
    CUDA_CHECK(cudaEventCreate(&start));
    CUDA_CHECK(cudaEventCreate(&stop));

    // Warm-up
    for (int i = 0; i < 5; ++i) {
        CUDA_CHECK(cudaMemcpy(d_result, &zero, sizeof(float), cudaMemcpyHostToDevice));
        sum_of_squares_kernel<<<grid_size, block_size, shared_bytes>>>(d_input, d_result, n);
        CUDA_CHECK(cudaDeviceSynchronize());
    }
```

```
    const int iters = 10;
    float total_ms = 0.0f;
    std::vector<float> times;
    for (int i = 0; i < iters; ++i) {
        CUDA_CHECK(cudaMemcpy(d_result, &zero, sizeof(float), cudaMemcpyHostToDevice));
        CUDA_CHECK(cudaEventRecord(start));
        sum_of_squares_kernel<<<grid_size, block_size, shared_bytes>>>(d_input, d_result, n);
        CUDA_CHECK(cudaEventRecord(stop));
        CUDA_CHECK(cudaEventSynchronize(stop));
        float ms = 0.0f;
        CUDA_CHECK(cudaEventElapsedTime(&ms, start, stop));
        times.push_back(ms);
        total_ms += ms;
    }
    float avg_ms = total_ms / iters;
    float std_ms = 0.0f;
    for (float t : times) std_ms += (t - avg_ms) * (t - avg_ms);
    std_ms = std::sqrt(std_ms / iters);

    // Ellenőrzés
    float diff = std::abs(cpu_result - gpu_result);
    float tolerance = 1e-3f;  // float pontosság miatt
    bool passed = diff < tolerance;

    std::cout << "CPU reference: " << cpu_result << "\n";
    std::cout << "GPU result:    " << gpu_result << "\n";
    std::cout << "Absolute diff: " << diff << "\n";
    std::cout << "Test: " << (passed ? "PASSED" : "FAILED") << "\n";
    std::cout << "Average kernel time: " << avg_ms << " ms\n";
    std::cout << "Std dev: " << std_ms << " ms\n";

    CUDA_CHECK(cudaEventDestroy(start));
    CUDA_CHECK(cudaEventDestroy(stop));
    CUDA_CHECK(cudaFree(d_input));
    CUDA_CHECK(cudaFree(d_result));
    return passed ? 0 : 1;
}
```

A kód minden CUDA API hívást ellenőriz, és a kernel indítása után `cudaGetLastError`-t és `cudaDeviceSynchronize`-t hív. A mérés előtt warm-up fut, majd legalább 10 ismétlést mérünk, és az átlag mellett a szórást is közöljük. Az eredményt CPU referenciaértékkel vetjük össze egy tolerancián belül, mert a GPU `float` pontossága és a redukció sorrendje kismértékű eltérést okozhat.

5.4.7 Build és futtatási parancs

A labor reprodukálhatóságához a következő parancsokat használjuk. A konkrét architektúra zászlót a telepített GPU compute capability-jéhez kell igazítani.

```
nvcc -o sum_sq sum_sq.cu -arch=sm_86
./sum_sq
```

Az `-arch=sm_86` példa egy NVIDIA Ampere generációs GPU-ra (pl. RTX 3080) vonatkozik. A saját környezetben a `nvidia-smi` és a `nvcc --version` segítségével ellenőrizhető a pontos compute capability és a CUDA verzió.

5.4.8 GPU-konfiguráció és mérési napló

A mérés feltételei példaként az alábbiak (a tényleges méréshez a saját környezet adatait kell rögzíteni):

- GPU: NVIDIA GeForce RTX 3080
- Driver: 535.104.05
- CUDA: 12.2
- Compute capability: 8.6
- Operációs rendszer: Ubuntu 22.04 LTS
- Power limit: alapértelmezett
- ECC: ki
- Háttérterhelés: nincs

A mérési napló 10 ismétlésből származik, a kernel futásideje CUDA events méréssel történt.

Iteráció	Kernel idő (ms)
1	0.042
2	0.040
3	0.041
4	0.039
5	0.040
6	0.041
7	0.040
8	0.039
9	0.042
10	0.040

Átlag: 0,0404 ms
Szórás: 0,0010 ms

A várt eredmény (CPU referencia, double pontossággal számolva): 12345,6789 (példaérték).
A mért GPU eredmény: 12345,6797 (float pontossággal).
Eltérés: 0,0008, ami a megengedett 0,001 tolerancián belül van, így a teszt sikeres.

A mérési naplóban a várt és a mért eredményt külön oszlopban vagy szakaszban kell feltüntetni. A fenti számok illusztrációk; a tényleges mérés a megadott hardveren reprodukálható, de az értékek a bemeneti adatoktól és a környezettől függően változhatnak. A labor szerződés szerint minden mérési állítást azonos inputra és baseline-ra kell vonatkoztatni, amit itt az `n = 1<<20` elemű tömb és a rögzített kernel konfiguráció biztosít.

5.4.9 Miért működik globális szinkronizáció nélkül?

A kernel helyessége azon alapul, hogy a blokkok közötti egyetlen kommunikáció az `atomicAdd` művelet, amely a globális memórián atomosan összegzi a részeredményeket. Mivel az atomi add kommutatív és asszociatív a lebegőpontos műveletek tekintetében (bár a kerekítési sorrend okozhat kis eltérést), a blokkok sorrendje nem befolyásolja a végeredményt. A blokkon belüli redukció shared memory-t és `__syncthreads`-et használ, ami garantáltan helyes, mert a blokk szálai együtt futnak. A kernel tehát teljesíti a független blokkok elvét: bármely blokk bármikor elindulhat, és az eredmény ugyanaz lesz.

5.4.10 Ellenőrző kérdések

1. Miért nem garantált a blokkok sorrendje a CUDA végrehajtási modellben, és miért előnyös ez a tervezési döntés?
2. Milyen veszélyei vannak a spinlock-alapú globális szinkronizációnak, és milyen feltételek mellett lenne esetleg használható?
3. Hogyan lehet helyesen egyesíteni a blokkok részeredményeit globális szinkronizáció nélkül? Válaszában térjen ki az atomi műveletek szerepére és a kernel határok használatára.

5.4.11 Összefoglalás

A CUDA végrehajtási modell korlátai a skálázhatóság szükségszerű velejárói. A blokkok függetlensége lehetővé teszi, hogy a GPU ütemezője szabadon ossza el a munkát, és hogy a kód különböző hardvereken is fusson. Ennek ára, hogy a blokkok közötti kommunikáció nem építhető közvetlenül a sorrendre vagy a globális memórián keresztüli szinkronizációra. A helyes tervezési minta az, hogy a blokkokat független feladatokként kezeljük, a részeredményeket atomi műveletekkel vagy kernel határokkal egyesítjük, és a grid-szintű kooperatív technikákat csak akkor alkalmazzuk, ha azok feltételei teljesülnek. A laboratóriumi példa bemutatta, hogyan lehet egy globális szinkronizáció nélküli algoritmust helyesen megvalósítani, és hogyan kell a mérést reprodukálhatóan dokumentálni. A következő fejezetben a globális indexszámításra és a kernel indítási konfigurációra építve tovább mélyítjük a programozási modell gyakorlati használatát.

II. RÉSZ – CUDA programozási modell

6 Kernelírás, indexelés és launch konfiguráció

6.1 Globális indexszámítás 1D-ben

Amikor egy kernelt elindítunk, a hardver nem ismeri a feladat logikai szerkezetét. Csak azt látja, hogy egy rácsnyi (grid) blokkot és blokkonként egy adott számú szálat kell ütemeznie. A programozó feladata, hogy ebből a hierarchiából egy lineáris, az adathoz illeszkedő azonosítót képezzen. Ez a fejezet ezt a leképezést tárgyalja 1D-s tömbök esetén. A cél annak megértése, hogyan válik a hardver által biztosított három beépített indexből – `threadIdx`, `blockIdx`, `blockDim` – a feldolgozandó elem globális pozíciója, és miért nélkülözhetetlen a határellenőrzés (boundary check), ha a tömb mérete nem osztható a blokkmérettel.

A fejezetben szereplő kódok CUDA C++ nyelvűek, és az itt bemutatott kernel „tankönyvi" jellegű: a hangsúly az indexelésen van, nem a memória-átviteli vagy futási sebességen. A labor feladata éppen ezért nem egy nagy teljesítményű számítás, hanem egy ellenőrizhető, kis méretű példa, amelynél a várt és a mért eredmény pontosan összevethető.



6.1. ábra: Az 1D globális index előállítása - A globális index a block- és thread-index lineáris kombinációja; a határellenőrzésnek a beolvasás előtt kell megtörténnie.

6.1.1 A thread, block és grid közötti kapcsolat

A CUDA végrehajtási modellje három szintre bontja a párhuzamos munkát: a szálak egy blokkba szerveződnek, a blokkok pedig egy rácsba. A kernel belsejében a szál pozícióját kizárólag beépített, csak olvasható változókból tudhatjuk meg:

- `threadIdx` – a szál indexe a blokkon belül. Típusa `dim3`, amely három komponenst tartalmaz: `threadIdx.x`, `threadIdx.y`, `threadIdx.z`. 1D-s indításnál a `y` és `z` komponens értéke 0, az `x` komponens pedig `0` és `blockDim.x - 1` közé esik.
- `blockIdx` – annak a blokknak az indexe, amelyhez a szál tartozik, a rácson belül. Szintén `dim3` típusú, 1D-s indításnál a `blockIdx.x` értéke `0` és `gridDim.x - 1` között változik.
- `blockDim` – a blokk mérete, tehát az egy blokkban található szálak száma dimenziónként. 1D-s indításnál csak a `blockDim.x` fontos, a többi komponens 1.
- `gridDim` – a rács mérete, tehát a blokkok száma dimenziónként. 1D-s indításnál a `gridDim.x` adja meg, hány blokk indult.

Ezek a változók minden szálban azonos módon olvashatók, de az értékük szálanként eltérő. Például egy `<<<4, 256>>>` indításnál a `gridDim.x` értéke 4, a `blockDim.x` értéke 256; a `blockIdx.x` az egyes blokkokban 0, 1, 2 vagy 3, a `threadIdx.x` pedig minden blokkban 0 és 255 között fut. A ténylegesen elindított szálak száma ebben a példában 4 * 256 = 1024.

A hardver nem ígéri meg, hogy a blokkok sorrendben hajtódnak végre, és azt sem, hogy egy blokkon belül a szálak ütemezése milyen sorrendű. Az indexszámításnak ezért mindig **determinisztikusnak** kell lennie, és kizárólag a fenti beépített változóktól, valamint a kernel argumentumaitól szabad függenie. Ha egy szál a `threadIdx.x`-ét ismeri, az még nem elegendő; szüksége van a `blockIdx.x`-re is ahhoz, hogy a rács összes szálán belül egyedi pozíciót kapjon.

6.1.2 A globális index előállítása 1D-ben

A legegyszerűbb és leggyakrabban használt leképezés a következő:

```
globalIdx = blockIdx.x * blockDim.x + threadIdx.x
```

Ez a képlet a rácson belüli blokkokat egymás után elhelyezve egy virtuális, lineáris száltartományt hoz létre. Ha a `blockDim.x` értéke `B`, akkor a 0. blokk szálai a `0..B-1`, az 1. blokk szálai a `B..2B-1` intervallumba esnek, és így tovább. A `blockIdx.x * blockDim.x` kifejezés tehát megadja az adott blokk kezdő indexét, a `threadIdx.x` pedig ezen belül a szál helyét.

Érdemes hangsúlyozni, hogy a szorzat típusa 1D-s indításnál a beépített változók miatt `unsigned int`. Bár a gyakorlatban a grid és blokk mérete rendszerint jóval kisebb, mint az `unsigned int` felső határa, nagy adathalmazoknál a `blockIdx.x * blockDim.x` kifejezés túlcsordulhat. Például ha `gridDim.x` a megengedett maximum közelében van (nagyjából 2,1 milliárd blokk) és `blockDim.x` 1024, a szorzat meghaladja a 32 bites tartományt. Emiatt erősen ajánlott az indexet legalább `size_t` típusra kasztolni, mielőtt nagy tömbökhöz használjuk. A modern CUDA C++ kódokban a következő minta terjedt el:

```
size_t idx = static_cast<size_t>(blockIdx.x) * blockDim.x + threadIdx.x;
```

A `static_cast` miatt a szorzás már 64 bites unsigned értéken történik, így elkerülhető a csendes túlcsordulás. Ez a részlet aprónak tűnik, de az 1D-s indexelés egyik leggyakrabban elfelejtett buktatója. A képlet logikája természetesen ugyanaz marad, csak a típus változik.

A kernel gyakran egyetlen sorban használja ezt a globális indexet. Például egy olyan kernel, amely egy `n` hosszú tömb `i`-edik elemét az indexének megfelelő értékkel tölti fel, így néz ki:

```
__global__ void fillIndices(int* out, size_t n)
{
    size_t idx = static_cast<size_t>(blockIdx.x) * blockDim.x + threadIdx.x;
    if (idx < n) {
        out[idx] = static_cast<int>(idx);
    }
}
```

Itt a szálak egyértelműen leképezik a rácsbeli pozíciójukat a tömb egy-egy elemére. A `idx < n` feltétel a határellenőrzés, amelyre a következő szakaszban térünk ki.

Fontos megjegyezni, hogy a globális index **nem** azonos a szál ütemezési sorszámával. A hardver a szálakat nem feltétlenül a `idx` növekvő sorrendjében hajtja végre, és a blokkok közötti sorrend sem meghatározott. A `idx` kizárólag a programozó számára biztosít egy konzisztens, az adathoz köthető azonosítót. Ezért mondjuk azt, hogy a `blockIdx`, `blockDim` és `threadIdx` a kernel logikai koordinátái, míg a hardver tényleges végrehajtási rendje a scheduler feladata.

6.1.3 Túlindexelés és boundary check

Ritkán fordul elő, hogy a feldolgozandó tömb hossza pontosan osztható a választott blokkmérettel. Ha `N` nem többszöröse `blockDim.x`-nek, az utolsó blokkban több szál indul, mint ahány elem feldolgozásra vár. Ilyenkor a globális index egy része kívül esik a `[0, N-1]` intervallumon. Ha a kernel nem ellenőrzi a határt, ezek a szálak érvénytelen memóriacímet érhetnek el.

Klasszikus példa: legyen `N = 1000`, a blokkméret pedig `256`. A szükséges blokkok számát a felfelé kerekített osztással kapjuk:

```
blocksPerGrid = ceil(N / 256) = ceil(3.906) = 4
```

A 4. blokkban, amelynek `blockIdx.x` értéke 3, a globális indexek a `768` és `1023` közötti tartományba esnek. Azonban a tömb csak a `0..999` indexeket tartalmazza. A `1000..1023` indexekre történő írás (vagy olvasás) kívül esik a lefoglalt memóriaterületen, ami a CUDA-ban definiálatlan viselkedéshez, gyakran `CUDA error: an illegal memory access was encountered` üzenethez vezet. A hiba nem mindig azonnal jelentkezik, ezért a diagnosztika később is nehéz lehet.

A megoldás a kernelben egyetlen feltételes utasítás:

```
if (idx < n) {
    // valódi munka, pl. out[idx] = idx;
}
```

Ez a **boundary check** biztosítja, hogy csak azok a szálak férnek hozzá a tömbhöz, amelyekhez valóban tartozik elem. A feltétel hamis ágában a szál egyébként is kilépne, de így explicit módon nem végez memóriaműveletet. A költség rendszerint elhanyagolható, és a korrektség szempontjából kötelező.

A boundary check hatására a fenti példában a 4. blokk utolsó 24 szálja tétlen marad. Ez nem „tökéletes” erőforrás-kihasználás, de helyes. A későbbi fejezetben tárgyalt grid-stride loop technika ezt a pazarlást hivatott csökkenteni azáltal, hogy a szálak egymás után több elemet is feldolgoznak, így a blokkok száma függetlenné válik az adat méretétől. Ebben a fejezetben azonban az egyszerű, egy elem egy szálhoz rendelés a cél, ezért a boundary check marad a központi eszköz.

Fontos, hogy a boundary check a **globális indexre** vonatkozik, nem a `threadIdx`-re vagy a `blockIdx`-re. Az `if (threadIdx.x < n)` feltétel például hibás, mert a `threadIdx.x` minden blokkban `0..blockDim.x-1` között mozog, így az `n` értékétől függően vagy túl keveset, vagy túl sokat engedne. Hasonlóképpen az `if (blockIdx.x < n)` sem helyes, mert a blokk indexe nem azonos az elem indexével.

Nézzünk egy konkrét, futtatható példát. Az alábbi kernel egy `N` elemű `float` tömböt skáláz egy konstanssal. A host oldali `N` tetszőleges pozitív egész lehet, a kernel a boundary check miatt minden esetben korrekt marad:

```
__global__ void scaleVector(const float* in, float* out, float alpha, size_t n)
{
    size_t idx = static_cast<size_t>(blockIdx.x) * blockDim.x + threadIdx.x;
    if (idx < n) {
        out[idx] = alpha * in[idx];
    }
}
```

A kernel egyetlen szálat rendel egyetlen tömbelemhez, és a `idx < n` feltétel levágja a felesleges szálakat. Ha `n` nagy, a host oldali blokkszámításnak is helyesnek kell lennie.

6.1.4 A grid és block méret kiszámítása host oldalon

A kernel indításához a host oldalon meg kell adni a rács és a blokk méretét. A leggyakoribb minta 1D-s feldolgozásnál:

1. Válasszunk egy blokkméretet, például `threadsPerBlock = 256`.
2. Számítsuk ki a blokkok számát felfelé kerekített osztással:

```
size_t blocksPerGrid = (n + threadsPerBlock - 1) / threadsPerBlock;
```

3. Indítsuk a kernelt ezzel a konfigurációval:

```
scaleVector<<<blocksPerGrid, threadsPerBlock>>>(d_in, d_out, alpha, n);
```

A `blocksPerGrid` számításánál az egész osztás természetéből adódóan a `n + threadsPerBlock - 1` kifejezés biztosítja a felfelé kerekítést. Például `n = 1000`, `threadsPerBlock = 256` esetén `(1000 + 255) / 256 = 4`, míg a sima `1000 / 256` 3-at adna, ami nem lenne elegendő.

Fontos ellenőrizni, hogy a kapott `blocksPerGrid` nem lépi túl a hardver `gridDim.x` korlátját. A mai GPU-kon ez a korlát `2^31 - 1` (nagyjából 2,1 milliárd) blokk, ami a legtöbb valós adatnál bőven elég. Ha az adat mérete ezt meghaladná, a grid-stride loop vagy többdimenziós grid válik szükségessé. Emellett a `threadsPerBlock` nem lehet nagyobb, mint a hardverenként eltérő maximális szálszám (általában 1024). A `cudaGetDeviceProperties` függvénnyel kérdezhető le a pontos korlát.

A launch konfigurációban a `blocksPerGrid` típusa a hagyományos CUDA példákban `int`, mert a kernel launch operátora integer típusokat vár. Ha a számítás `size_t` típussal történt, explicit kasztolásra lehet szükség:

```
int blocksPerGrid = static_cast<int>((n + threadsPerBlock - 1) / threadsPerBlock);
```

Ez a kasztolás biztonságos, amíg `blocksPerGrid` a fent említett korlát alatt marad. Nagy `n` esetén azonban előbb-utóbb elérjük a 32 bites korlátot; ekkor a grid-stride loop a fenntartható megoldás.

6.1.5 Gyakori hibák és elkerülésük

Az 1D-s globális indexszámítás egyszerűnek tűnik, mégis számos jellegzetes hiba forrása. Az alábbiak a leggyakoribbak.

1. A szorzás és összeadás sorrendjének felcserélése.
Hibás: `idx = threadIdx.x * blockDim.x + blockIdx.x`.
Helyes: `idx = blockIdx.x * blockDim.x + threadIdx.x`.
A hibás változatban a `threadIdx.x * blockDim.x` kifejezés a blokkon belüli szál pozícióját szorozza meg a blokk méretével, ami átfedő indexeket eredményez. Például `blockDim.x = 256` esetén a 0. blokk 1. szálának hibás indexe `256`, miközben a helyes index `1` lenne. Ez a hiba gyakran nehezen észrevehető, mert a kernel fut, de a memória-hozzáférések szétszórtak és rosszak.

2. A boundary check elhagyása.
Minden olyan kernelben, ahol a globális index a tömb hosszának nem állandó osztója, kötelező az `if (idx < n)` feltétel. Ennek elhagyása a legtöbb esetben érvénytelen memória-hozzáféréshez vezet, amelyet a compute-sanitizer vagy az `cuda-memcheck` jelez. A hiba tünete lehet a nem determinisztikus crash, az eldurvult adatok vagy a csendes korrupció. A javítás egyszerű, de a figyelmetlenség sok időt emészthet fel.

3. Az index típusának túlcsordulása.
A `blockIdx.x * blockDim.x` kifejezés `unsigned int` típusú, és nagy gridnél túlcsordulhat. A már említett `static_cast<size_t>` megoldás mellett fontos, hogy a kernel argumentumai is megfelelő típusúak legyenek. Ha a kernel `int n` paramétert vár, de a host oldalon `size_t` érkezik, a 64 bites érték 32 bitesre csonkulhat. A konzisztencia érdekében a kernel paramétere és a host oldali számítás is `size_t` legyen.

4. A blokkméret helytelen megválasztása.
Bár a hardver megenged bizonyos határokat (például blokkonként maximum 1024 szál), a gyakorlatban a 256, 512 vagy 1024 a gyakori választás. A rossz blokkméret nem feltétlenül okoz indexelési hibát, de befolyásolja a teljesítményt. Ebben a fejezetben a korrektségre koncentrálunk, a teljesítményoptimalizálás a későbbi részek témája.

5. A `gridDim` és `blockDim` összekeverése.
A globális index képletében kizárólag a `blockDim.x` szerepel, nem a `gridDim.x`. A `gridDim.x` azt adja meg, hány blokk indult, de az index kiszámításához a blokkon belüli szálak számára van szükség. A kettő felcserélése szintén hibás indexeléshez vezet.

6. Több kernel hívásakor a beépített változók értékének átmentése.
A `threadIdx`, `blockIdx` stb. csak a kernel futása alatt érvényesek, és minden kernelhívásban friss értéket kapnak. Nem lehet őket host oldalra kimenteni vagy két kernel között megosztani. Ha egy kernelből szeretnénk átadni valamilyen indexet, azt explicit módon írjuk ki a globális memóriába.

6.1.6 Labor: 1D indexelés implementálása és ellenőrzése

A fejezet laborja egy olyan CUDA C++ program, amely egy `N` hosszú egész tömböt tölt fel a globális index értékével: `a[i] = i`. A feladat célja, hogy a hallgató a saját környezetében futtassa a kódot, ellenőrizze a tömb tartalmát, és rögzítse a mérési körülményeket. A kernel szándékosan egyszerű, hogy a figyelem az indexelésre és a boundary checkre irányuljon.

A labor előtt győződjön meg arról, hogy a CUDA Toolkit és a driver telepítve van, és a `nvidia-smi` parancs elérhető. A kódot izolált környezetben futtassa, mert a hibás kernel eszközleállást okozhat; állítson be timeoutot és visszaállítási tervet.

6.1.7 Kernel és host kód

Az alábbi kód egy teljes, fordítható CUDA C++ program. A hibakezeléshez egy egyszerű makrót használ, amely minden CUDA API hívás után ellenőrzi a hibakódot. A kernel a `fillIndices` nevet viseli, és a globális indexet a fent tárgyalt módon számítja ki.

```
// fill_indices.cu
// Fordítás: nvcc -std=c++17 -O3 -o fill_indices fill_indices.cu
#include <cuda_runtime.h>
#include <iostream>
#include <vector>
#include <cassert>
#include <cstdlib>

// Hibakezelő makró
#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err = call;                                             \
        if (err != cudaSuccess) {                                           \
            std::cerr << "CUDA error at " << __FILE__ << ":" << __LINE__    \
```

```
                  << " - " << cudaGetErrorString(err) << std::endl;          \
            std::exit(EXIT_FAILURE);                                         \
        }                                                                    \
    } while (0)

// Kernel: a tömb i-edik elemét i-vel tölti fel
__global__ void fillIndices(int* out, size_t n)
{
    size_t idx = static_cast<size_t>(blockIdx.x) * blockDim.x + threadIdx.x;
    if (idx < n) {
        out[idx] = static_cast<int>(idx);
    }
}

int main()
{
    // A feldolgozandó elemek száma
    const size_t n = 1000;

    // Blokkméret és rács méret
    const int threadsPerBlock = 256;
    const int blocksPerGrid = static_cast<int>((n + threadsPerBlock - 1) / threadsPerBlock);

    // Eszköz oldali memória
    int* d_out = nullptr;
    CUDA_CHECK(cudaMalloc(&d_out, n * sizeof(int)));

    // Kernel indítás
    fillIndices<<<blocksPerGrid, threadsPerBlock>>>(d_out, n);
    CUDA_CHECK(cudaGetLastError());          // kernel launch hibák ellenőrzése
    CUDA_CHECK(cudaDeviceSynchronize());     // szinkronizáció a másolás előtt

    // Eredmény visszamásolása
    std::vector<int> h_out(n);
    CUDA_CHECK(cudaMemcpy(h_out.data(), d_out, n * sizeof(int), cudaMemcpyDeviceToHost));

    // Ellenőrzés: h_out[i] == i minden i-re
    bool ok = true;
    for (size_t i = 0; i < n; ++i) {
        if (h_out[i] != static_cast<int>(i)) {
            std::cerr << "Hiba: h_out[" << i << "] = " << h_out[i]
                      << ", várt: " << i << std::endl;
            ok = false;
            break;
        }
    }
    if (ok) {
        std::cout << "Az 1D indexelés helyes: minden elem megegyezik az indexszel.\n";
    }

    // Takarítás
    CUDA_CHECK(cudaFree(d_out));
    return ok ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

A kernel a `blockIdx.x * blockDim.x + threadIdx.x` képletet használja `size_t`-re kasztolással, majd a `if (idx < n)` feltétellel levédi a felesleges szálakat. A host oldalon a `blocksPerGrid` értéke `(1000 + 255) / 256 = 4`, tehát 4 blokk, blokkonként 256 szállal. A kernelben a `blockIdx.x` 0..3 között fut, a `threadIdx.x` 0..255 között. A globális indexek 0..1023-ig terjednek, de csak a 0..999 tartomány kerül feldolgozásra; az utolsó 24 szál tétlen.

A várt eredmény: a `h_out` tömb minden eleme pontosan az indexe. Ez a program ezt ellenőrzi,

és a kimeneten megjeleníti a sikert.

6.1.8 Mérési napló

A könyv konvencióinak megfelelően minden GPU-mérésnél rögzíteni kell a környezetet, a mérési módszert és az eredményt. Az alábbi táblázat a laborhoz tartozó mérési napló sablonja. A konkrét értékeket a futtatás után kell kitölteni, és a **mért** rovatban szereplő adatok tényleges mérésből származnak.

Mező	Érték
GPU modell	pl. NVIDIA A100 40 GB PCIe
Compute capability	pl. 8.0
Driver verzió	pl. 550.54.14
CUDA Toolkit	pl. 12.4
Operációs rendszer	pl. Ubuntu 22.04 LTS
Fordító	nvcc 12.4.0
Kernel	`fillIndices`
Blokkméret (`blockDim.x`)	256
Grid méret (`gridDim.x`)	4
Input méret (`n`)	1000
Precision	FP32 (a kernel int indexet ír, a precision itt N/A)
Warm-up iterációk	10
Mérési iterációk	10
Átlagos futásidő (kernel)	pl. 5,2 µs
Szórás	pl. 0,41 µs
Várt eredmény	`h_out[i] == i` minden i-re
Mért eredmény	ellenőrzés sikeres

A kernel futásidejét CUDA event-ekkel mérhetjük. A labor során a fenti kódot ki lehet egészíteni `cudaEvent_t`-vel történő időméréssel, majd a mérést legalább 10 ismétléssel átlagolni. A mérés előtt érdemes néhány bemelegítő (warm-up) kernelhívást futtatni, hogy az eszköz órajele és a memória-állapot stabilizálódjon. Az átlag mellett a szórást is közölni kell. A mért futásidő ekkora feladatnál nagyon rövid, ezért a mérési bizonytalanság jelentős; emiatt a labor hangsúlya nem a sebességen, hanem a korrektségen van.

6.1.9 Összefoglalás és önellenőrző kérdés

Ebben a fejezetben az 1D-s globális indexszámítás alapjait jártuk körül. A kernelbeli szálak egyértelmű azonosításához a `blockIdx.x`, `blockDim.x` és `threadIdx.x` beépített változókból a `globalIdx = blockIdx.x * blockDim.x + threadIdx.x` képletet használjuk. Mivel a tömb mérete általában nem osztható a blokkmérettel, a kernelben mindig gondoskodni kell a `if (idx < n)` határellenőrzésről, amely megakadályozza a túlindexelést. A host oldalon a blokkok számát felfelé kerekített osztással számítjuk, és gondot fordítunk arra, hogy az index 64 bites típusú legyen a túlcsordulás elkerülése érdekében.

A következő fejezet (Többdimenziós indexelés) erre az alapra építve a többdimenziós indexelést tárgyalja, ahol a `dim3` típusú `y` és `z` komponensek is szerepet kapnak. Addig is érdemes az alábbi kérdést megválaszolni.

Önellenőrző kérdés:
Adott egy `N = 1000` hosszú tömb, és a `threadsPerBlock` értéke `256`. Hány blokk indul, és összesen hány szál fut? A kernelben miért van szükség boundary checkre, és melyik az a feltétel, amely ezt biztosítja? Továbbá: mi történne, ha a képletben a `blockIdx.x` és a `threadIdx.x` sorrendjét felcserélnénk?

Válasz: `blocksPerGrid = ceil(1000 / 256) = 4`, összesen `4 * 256 = 1024` szál indul. A 1000 elem miatt az utolsó blokkban 24 szál indexe kívül esik a `[0, 999]` tartományon, ezért a kernelben `if (idx < n)` feltételt kell alkalmazni. Ha a képletet felcserélnénk, az indexek átfednének és nem lennének egyediek, ami hibás memória-hozzáférést okozna.

6.2 Többdimenziós indexelés

A többdimenziós indexelés a GPGPU-programozás egyik legfontosabb, ugyanakkor leggyakrabban félreértett lépése. Az 1D-s esetben egyetlen globális azonosítót számolunk, és arra hivatkozunk; 2D-s mátrixoknál, képeknél vagy 3D-s térfogatoknál azonban a természetes gondolkodásmód a sor–oszlop, illetve a réteg–sor–oszlop koordinátarendszer. A CUDA, az OpenCL és a JAX/XLA egyaránt lehetővé teszi, hogy a hardverrácsot többdimenziósként adjuk meg, így a kernelkód közelebb marad a matematikai modellhez. Ebben az alfejezetben a 2D- és 3D-leképezést, a row-major és column-major memóriaelrendezést, a határkezelést és a paddinget dolgozzuk ki. A cél nem csupán a képletek ismerete, hanem az, hogy bármilyen rács–memória leképezést tudatosan, a határokat és a tényleges memóriacímeket ellenőrizve tudj tervezni.



6.2. ábra: 2D mátrix leképezése indexekre - A 2D mátrixot row-major sorrendben lineáris indexszé alakítjuk: idx = y*W + x; a határok és a padding külön kezelendők.

6.2.1 2D feladatok leképezése

Képfeldolgozásnál, mátrixműveleteknél vagy végesdifferencia-módszereknél a bemenet leggyakrabban egy $H \times W$ méretű, soronként folytonosan tárolt tömb. A CUDA-ban a kernel indításakor a rácsot és a blokkot is legfeljebb háromdimenziós `dim3` típussal adhatjuk meg. Egy 2D-s munkarács esetén a `dim3 block(16, 16);` például egy $16 \times 16 = 256$ szálból álló blokkot definiál, a `dim3 grid(ceiling(W/16), ceiling(H/16));` pedig egy 2D-s blokkgrácsot. A kernel törzsében a szál globális koordinátája:

```
int ix = blockIdx.x * blockDim.x + threadIdx.x;
int iy = blockIdx.y * blockDim.y + threadIdx.y;
```

Ezek a `ix`, `iy` értékek a logikai kép- vagy mátrixkoordináták. A memóriában azonban a 2D-s tömb továbbra is lineárisan, egymás után fűzött sorokként él. A C/C++ nyelvben a szokásos elrendezés a **row-major**: az `x` (oszlop) index a leggyorsabban változó, a sor elemei egymással szomszédosak, a `y` (sor) index a lassabb. A lineáris index:

```
idx = iy * width + ix;
```

Példaként egy 5×3 mátrix (5 sor, 3 oszlop) memóriabeli sorrendje:

```
(0,0) (1,0) (2,0) (0,1) (1,1) (2,1) ...
```

Az `idx = iy * width + ix` képlet ezt a sorrendet adja vissza. Fontos, hogy itt a `width` a logikai szélesség, nem feltétlenül azonos a lefoglalt sor hosszával, ha paddinget használunk; erre hamarosan visszatérünk.

A 2D-s leképezés leggyakoribb formája egy olyan kernel, amely egy képen vagy mátrixon elemenként hajt végre műveletet. A következő CUDA C++ kernel lineáris transzformációt végez, és közben ellenőrzi a határokat:

```
#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err = (call);                                           \
        if (err != cudaSuccess) {                                           \
            fprintf(stderr, "CUDA error at %s:%d: %s\n",                    \
                    __FILE__, __LINE__, cudaGetErrorString(err));            \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

__global__ void scale2D_kernel(const float* __restrict__ input,
                               float* __restrict__ output,
                               int width, int height,
                               float alpha, float beta)
{
    int ix = blockIdx.x * blockDim.x + threadIdx.x;
    int iy = blockIdx.y * blockDim.y + threadIdx.y;

    if (ix < width && iy < height) {
        int idx = iy * width + ix;   // row-major lineáris index
        output[idx] = alpha * input[idx] + beta;
    }
}
```

A hostoldali indítás:

```
dim3 block(16, 16);
dim3 grid((width + block.x - 1) / block.x,
          (height + block.y - 1) / block.y);

scale2D_kernel<<<grid, block>>>(d_in, d_out, width, height, 2.0f, 1.0f);
CUDA_CHECK(cudaDeviceSynchronize());
```

A `dim3 grid((width + block.x - 1) / block.x, ...)` kerekítés biztosítja, hogy a rács lefedje a teljes tartományt akkor is, ha a szélesség nem osztható a blokkszélességgel. A kernelben a `if (ix < width && iy < height)` feltétel azokra a szálakra is kiterjed, amelyek a rács szélén túlra mutatnak. Ez a **határellenőrzés** nem opcionális: nélküle a szélső blokkokban lévő felesleges szálak illegális memóriacímeket írnának vagy olvasnának, ami jellemzően eszközhibához vezet.

A teljesítmény szempontjából a 2D-s leképezésnél arra kell ügyelni, hogy a szomszédos `threadIdx.x` értékű szálak egymás melletti `ix` koordinátákat kapjanak. Ha row-major elrendezést használunk, akkor ezek a szálak egymást követő memóriacímeket érnek el, ami

támogatja a globális memória coalescingjét. Ha a kernelben felcserélnénk a szerepeket, és a szálak `x` irányban a `iy` koordinátát változtatnák, a memóriaelérés szétszórt lenne, és a sávszélesség drasztikusan csökkenne. Ezért a 2D-s rácsoknál mindig a leggyorsabban változó logikai dimenziót érdemes a `threadIdx.x`-hez kötni.

6.2.2 3D feladatok leképezése

A 3D-s esetben a rács és a blokk háromdimenziós `dim3` típust kap. Például egy voxeles térfogat $D \times H \times W$ méretű. A kernel indítása:

```
dim3 block(8, 8, 8); // 512 szál blokkonként
dim3 grid((width + block.x - 1) / block.x,
          (height + block.y - 1) / block.y,
          (depth + block.z - 1) / block.z);
```

A globális koordináták:

```
int ix = blockIdx.x * blockDim.x + threadIdx.x;
int iy = blockIdx.y * blockDim.y + threadIdx.y;
int iz = blockIdx.z * blockDim.z + threadIdx.z;
```

A row-major 3D-s memóriaelrendezésben az `x` a leggyorsabban változó, az `y` a középső, a `z` a leglassabb. A lineáris index:

```
idx = (iz * height + iy) * width + ix;
```

Ez a képlet ugyanazt a logikát követi, mint a 2D-s eset: a belső zárójel a 3D-s térfogat egy szeletén belüli sorszámot adja meg, a `iz * height` pedig az előző rétegek elemeit ugroja át. A `height` a teljes rétegen belüli sorok száma, a `width` az egyes sorok hossza. Ha a térfogatot folytonosan foglaltuk le, ez a linearizálás helyes.

A következő kernel például egy 3D-s voxeltranszformációt hajt végre, például egy aktiválási függvényt alkalmaz minden voxelre:

```
__global__ void sigmoid3D_kernel(const float* __restrict__ input,
                                 float* __restrict__ output,
                                 int width, int height, int depth)
{
    int ix = blockIdx.x * blockDim.x + threadIdx.x;
    int iy = blockIdx.y * blockDim.y + threadIdx.y;
    int iz = blockIdx.z * blockDim.z + threadIdx.z;

    if (ix < width && iy < height && iz < depth) {
        int idx = (iz * height + iy) * width + ix;
        float v = input[idx];
        output[idx] = 1.0f / (1.0f + expf(-v));  // logisztikus szigmoid
    }
}
```

A határellenőrzés itt háromdimenziós: a rács szélein mindhárom irányban lehetnek olyan szálak, amelyek a tényleges dimenzión kívülre esnek. A `if (ix < width && iy < height && iz < depth)` feltétel mindegyiket kizárja. A feltétel kiértékelése rövidzár: ha az `ix` már érvénytelen, az `iy` és `iz` kiértékelésére nem kerül sor, így a többletköltség minimális.

A 3D-s indexelésnél különösen fontos a dimenziókorlátok ismerete. A CUDA blokk maximális mérete szálanként 1024; a `blockDim.z` tipikusan legfeljebb 64 lehet, míg `blockDim.x` és `blockDim.y` nagyobbak lehetnek (a pontos értékek compute capability függők, például egyes architektúrákon `x <= 1024`, `y <= 1024`, `z <= 64`, de a szorzatuk $<= 1024$). A griddimenzió is

korlátos: `gridDim.x` általában $2^{31}-1$, `gridDim.y` és `gridDim.z` jellemzően 65535. Ha a 3D-s probléma mérete ezeket meghaladná, akkor nem elegendő egyszerűen növelni a rácsot; ilyenkor 1D-s rácsot és a kernelben manuálisan lebontott lineáris indexet, vagy a következő alfejezetben tárgyalandó grid-stride loop mintát kell használni.

6.2.3 Row-major és column-major

A memóriaelrendezés kérdése nem csak elméleti: ugyanaz a mátrix egészen más lineáris indexelést igényel row-major és column-major esetben. A C/C++ tömbök, és így a CUDA C++ kódok túlnyomó többsége, row-major elrendezést használ. Ezzel szemben a cuBLAS, a LAPACK és több tudományos könyvtár **column-major** elrendezést vár. A column-major elrendezésben az oszlopok elemei vannak egymás mellett a memóriában, tehát a `y` (sor) index a leggyorsabban változó. A lineáris index:

```
idx = ix * height + iy;   // column-major
```

Egy $H \times W$ mátrix esetén a row-major indexképlet `iy * W + ix`, a column-major pedig `ix * H + iy`. A különbség azonnal látszik, ha ugyanazt a mátrixot mindkét módon járjuk be: row-majorban a belső ciklus az `x`-en fut, column-majorban pedig az `y`-on.

A GPGPU-kódban a leggyakoribb hiba a row-major és column-major keverése. Amikor például cuBLAS-t hívunk, és a saját row-major mátrixunkból column-major mátrixként olvasunk, a művelet transzponáltként viselkedhet. Ezért a könyvtári hívásoknál mindig ellenőrizni kell a függvény dokumentációjában a tárolási sorrendet, és szükség esetén transzponálási opciót (`CUBLAS_OP_T`) vagy explicit konverziót használni.

A két elrendezés közötti választást a memória-hozzáférési minta is befolyásolja. Ha a szálak az `x` irányban szomszédosak, és a mátrixot row-majorban tároljuk, akkor a globális memóriaolvasás koaleszkálódik. Ha column-major tárolást használnánk, a szálak az `x` irányban az oszlop mentén lépkednének, ami szétszórt címeket eredményezne. Ezért gyakori megoldás, hogy a kernel a kívánt elrendezésnek megfelelően indexel, vagy a transzponált feladatot külön kernelben végzi el.

6.2.4 Határkezelés és padding

A rácsos leképezés során a legtöbb gyakorlati problémánál a dimenziók nem oszthatók a blokkmérettel. A felesleges szálak kezelésére a `if (ix < width && iy < height)` típusú feltétel szolgál. Ez a **határellenőrzés** biztosítja, hogy a kernel soha ne érjen el a lefoglalt tömb határain kívülre. A határellenőrzés hiánya a leggyakoribb CUDA-hibák közé tartozik; a hibás kernel gyakran illegális memóriahozzáférést okoz, és a `cudaDeviceSynchronize()` után `cudaErrorIllegalAddress` hibát ad.

A határellenőrzés mellett a **padding** egy másik, gyakran használt technika. A padding azt jelenti, hogy a ténylegesen lefoglalt sor vagy réteg hosszát megnöveljük a logikai méreten túl, például azért, hogy a memóriacímek igazodjanak, vagy hogy egy szűrő a széleken is biztonságosan hozzáférjen a szomszédos elemekhez. A padding lehet **zero padding** (a hozzáadott elemek nullák), **clamp padding** (a szélső értékeket ismétli), **wrap padding** (a túloldalról folytatja) vagy **reflect padding** (a széleket tükrözi). Az alfejezet szempontjából a legfontosabb a memóriaelrendezés és az indexelés helyes kezelése.

Egy paddingelt 2D-s tömb esetén a memóriában a sor hossza nagyobb, mint a logikai `width`. A tényleges sorhosszt gyakran `pitch`-nek vagy `ldA`-nak (leading dimension) nevezik. A lineáris index ilyenkor:

```
idx = iy * ldA + ix;
```

ahol `ldA >= width`. A korábban használt `idx = iy * width + ix` csak akkor helyes, ha nincs padding. A cudaMallocPitch és a cudaMemcpy2D API-k pontosan azért léteznek, hogy a paddingelt sorokkal kényelmesen lehessen dolgozni; a kernelnek mindig a tényleges `pitch` értéket kell megkapnia.

A paddingnek a határkezelésben is szerepe van. Ha például egy 3×3-as szűrőt akarunk alkalmazni egy képre, a szélső pixeleknél a kernelnek a szomszédos, de a képen kívüli elemeket is el kellene érnie. Az egyik megoldás, hogy a bemeneti képet megnövelt méretben, egy pixeles zero paddinggel foglaljuk le, és a belső régióba másoljuk az eredeti adatokat. Ekkor a kernelben a 2D-s indexelés a paddingelt tömbhöz igazodik:

```
__global__ void copyPadded2D_kernel(const float* __restrict__ input,
                                    float* __restrict__ output,
                                    int inWidth, int inHeight,
                                    int outWidth, int outHeight,
                                    int padX, int padY)
{
    int ox = blockIdx.x * blockDim.x + threadIdx.x;
    int oy = blockIdx.y * blockDim.y + threadIdx.y;

    if (ox < outWidth && oy < outHeight) {
        int ix = ox - padX;
        int iy = oy - padY;
        float val = 0.0f;  // zero padding

        if (ix >= 0 && ix < inWidth && iy >= 0 && iy < inHeight) {
            val = input[iy * inWidth + ix];
        }
        output[oy * outWidth + ox] = val;
    }
}
```

Ebben a mintában a kimeneti tömb a kibővített, paddingelt rácsot fedi le. A szálak az `ox`, `oy` kimeneti koordinátákat számítják, majd visszaállítják a bemeneti koordinátákat `ix = ox - padX`, `iy = oy - padY`. Ha az így kapott koordináták a bemeneti tartományon belül vannak, a bemenet megfelelő elemét másoljuk; különben nullát írunk. Ez a **zero padding** explicit megvalósítása. A `copyPadded2D_kernel` nem használ shared memóriát vagy tilinget, mert azok a későbbi fejezetek témái; itt kizárólag a többdimenziós indexelés és a határok helyes kezelése a cél.

3D-s esetben a padding ugyanígy működik, csak a harmadik dimenzióval bővül. Például egy $D \times H \times W$ térfogathoz padding esetén a ténylegesen lefoglalt dimenziók $D+2p_z \times H+2p_y \times W+2p_x$ lehetnek. A lineáris index a paddingelt dimenziókkal:

```
idx = (oz * outHeight + oy) * outWidth + ox;
```

A bemeneti koordináták visszaszámítása pedig `ix = ox - padX; iy = oy - padY; iz = oz - padZ;`. A határellenőrzés minden dimenzióban elvégzendő.

6.2.5 Tipikus hibák és javításuk

A többdimenziós indexelés során a leggyakoribb hibák a következők.

1. Elfelejtett határellenőrzés. Ha a szélesség vagy magasság nem osztható a blokkmérettel, az utolsó blokkban lévő egyes szálak a lefoglalt tömbön kívülre mutatnak. A kernelben minden dimenzióra `ix < width`, `iy < height` típusú feltételt kell írni, mielőtt a memóriához nyúlunk.

A hiányzó feltétel gyakran csak bizonyos bemeneti méreteknél okoz hibát, ami megnehezíti a reprodukálást.

2. A valódi szélesség és a rács szélességének összekeverése. A lineáris index kiszámításánál sokan ezt írják:

```
int idx = iy * (blockDim.x * gridDim.x) + ix;    // HELYTELEN
```

Ez a képlet azt feltételezi, hogy a sor hossza megegyezik a rács által lefedett szélességtel, ami csak akkor igaz, ha a szélesség pontosan osztható, és nincs padding. A helyes szélesség a tényleges `width`, illetve padding esetén a `pitch` vagy `ldA`. Ezért a kernel mindig kapja meg a tényleges memóriabeli sorhosszt, és azt használja a linearizáláshoz.

3. `blockIdx` és `threadIdx` keverése. Gyakori hiba, ha valaki `ix = threadIdx.x * blockDim.x + blockIdx.x;`-et ír. Ez a szorzás felcserélése miatt teljesen rossz indexet ad. A helyes képlet: `blockIdx.x * blockDim.x + threadIdx.x;`. Hasonlóan ügyelni kell arra, hogy a `blockDim.y` és `gridDim.y` ne keveredjen a `blockDim.x`-szel.

4. A dimenzió sorrendjének felcserélése. A 2D-s rácsnál a `blockIdx.x` az oszlophoz, a `blockIdx.y` a sorhoz tartozik. Ha valaki ezt felcseréli, a memóriaelérés szétszórt lesz, és a kép transzponáltan kerül feldolgozásra. A 3D-s rácsnál ugyanez a hiba mindhárom tengelyen előfordulhat.

5. Túlcsorduló integer index. Nagy 3D-s térfogatoknál a `int idx = (iz * height + iy) * width + ix;` kifejezés meghaladhatja a 32 bites `int` tartományát ($2^{31} - 1$). Ha például egy $8192 \times 8192 \times 8192$ float tömbünk van, a lineáris index eléri a $5{,}5 \times 10^{11}$ értéket, ami nem fér `int`-be. Ilyenkor a lineáris index típusát `size_t`-re, vagy legalább 64 bites egészre kell cserélni.

6. Padding esetén a logikai szélesség használata a tényleges helyett. Amikor `cudaMallocPitch`-et használunk, a visszaadott `pitch` nagyobb lehet, mint a logikai szélesség. A kernelben a `idx = iy * width + ix` helyett `idx = iy * pitch + ix`-et kell használni. Ha a `width`-et adjuk át, a kernel a paddingelt sorokban rossz helyre fog írni, ami adatkorrupcióhoz vezet.

Egy jellegzetesen hibás minta és a javított változat:

```
// HIBÁS: nincs határellenőrzés, és width helyett blockDim.x * gridDim.x
__global__ void bad_scale2D_kernel(const float* input, float* output,
                                   int width, int height)
{
    int ix = blockIdx.x * blockDim.x + threadIdx.x;
    int iy = blockIdx.y * blockDim.y + threadIdx.y;
    int idx = iy * (blockDim.x * gridDim.x) + ix;    // rossz sorhossz
    output[idx] = 2.0f * input[idx] + 1.0f;
}

// JAVÍTOTT: tényleges width, határellenőrzéssel
__global__ void good_scale2D_kernel(const float* input, float* output,
                                    int width, int height)
{
    int ix = blockIdx.x * blockDim.x + threadIdx.x;
    int iy = blockIdx.y * blockDim.y + threadIdx.y;

    if (ix < width && iy < height) {
        int idx = iy * width + ix;
        output[idx] = 2.0f * input[idx] + 1.0f;
    }
}
```

A hibás kernel a kimeneti tömbön kívülre írhat, különösen akkor, ha a szélesség nem többszöröse a `blockDim.x * gridDim.x`-nek, vagy ha a `gridDim.x` valóban nagyobb, mint a tényleges szélesség. A javított változat mindkét problémát kiküszöböli.

6.2.6 Labor: 2D mátrix- és 3D térfogatkernel

Ez a labor a többdimenziós indexelés gyakorlását szolgálja. Két feladatot valósítunk meg: egy 2D-s mátrix lineáris transzformációját, és egy 3D-s térfogat elemenkénti szigmoid transzformációját. A labor elvégzéséhez CUDA C++ kernel, OpenCL kernel és JAX változat tartozik. A következő kódok fordíthatók, de a labor futtatása előtt minden környezetben ellenőrizni kell a GPU-, driver- és CUDA-verziót. Az alábbi mérési naplóban szereplő értékek **példaértékűek, nem valós mérésből származnak**, ezért azokat a saját futtatás során le kell cserélni.

6.2.7 CUDA C++ változat

A 2D kernel:

```
// fordítás: nvcc -O3 -arch=native gpu019_2d.cu -o gpu019_2d
#include <cstdio>
#include <cstdlib>
#include <cmath>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err = (call);                                           \
        if (err != cudaSuccess) {                                           \
            fprintf(stderr, "CUDA error at %s:%d: %s\n",                    \
                    __FILE__, __LINE__, cudaGetErrorString(err));            \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

__global__ void scale2D_kernel(const float* __restrict__ input,
                               float* __restrict__ output,
                               int width, int height,
                               float alpha, float beta)
{
    int ix = blockIdx.x * blockDim.x + threadIdx.x;
    int iy = blockIdx.y * blockDim.y + threadIdx.y;

    if (ix < width && iy < height) {
        int idx = iy * width + ix;
        output[idx] = alpha * input[idx] + beta;
    }
}

int main()
{
    const int width  = 1920;
    const int height = 1080;
    const float alpha = 2.0f;
    const float beta  = 1.0f;

    size_t bytes = (size_t)width * height * sizeof(float);

    float *h_in = (float*)malloc(bytes);
    float *h_out = (float*)malloc(bytes);
    for (int i = 0; i < width * height; ++i) {
```

```
        h_in[i] = (float)(i % 1000) / 1000.0f;
    }

    float *d_in, *d_out;
    CUDA_CHECK(cudaMalloc(&d_in, bytes));
    CUDA_CHECK(cudaMalloc(&d_out, bytes));
    CUDA_CHECK(cudaMemcpy(d_in, h_in, bytes, cudaMemcpyHostToDevice));

    dim3 block(16, 16);
    dim3 grid((width + block.x - 1) / block.x,
              (height + block.y - 1) / block.y);

    // warm-up
    scale2D_kernel<<<grid, block>>>(d_in, d_out, width, height, alpha, beta);
    CUDA_CHECK(cudaDeviceSynchronize());

    // mérés: legalább 5 ismétlés; az alábbi egyetlen ismétlést szemléltet
    scale2D_kernel<<<grid, block>>>(d_in, d_out, width, height, alpha, beta);
    CUDA_CHECK(cudaDeviceSynchronize());

    CUDA_CHECK(cudaMemcpy(h_out, d_out, bytes, cudaMemcpyDeviceToHost));

    // ellenőrzés CPU-n
    double max_err = 0.0;
    for (int iy = 0; iy < height; ++iy) {
        for (int ix = 0; ix < width; ++ix) {
            int idx = iy * width + ix;
            float expected = alpha * h_in[idx] + beta;
            double err = fabs((double)h_out[idx] - expected);
            if (err > max_err) max_err = err;
        }
    }
    printf("Max error: %g\n", max_err);

    CUDA_CHECK(cudaFree(d_in));
    CUDA_CHECK(cudaFree(d_out));
    free(h_in);
    free(h_out);
    return 0;
}
```

A 3D kernel:

```
// fordítás: nvcc -O3 -arch=native gpu019_3d.cu -o gpu019_3d
#include <cstdio>
#include <cstdlib>
#include <cmath>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err = (call);                                           \
        if (err != cudaSuccess) {                                           \
            fprintf(stderr, "CUDA error at %s:%d: %s\n",                    \
                    __FILE__, __LINE__, cudaGetErrorString(err));            \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

__global__ void sigmoid3D_kernel(const float* __restrict__ input,
                                 float* __restrict__ output,
                                 int width, int height, int depth)
{
    int ix = blockIdx.x * blockDim.x + threadIdx.x;
```

```
    int iy = blockIdx.y * blockDim.y + threadIdx.y;
    int iz = blockIdx.z * blockDim.z + threadIdx.z;

    if (ix < width && iy < height && iz < depth) {
        int idx = (iz * height + iy) * width + ix;
        float v = input[idx];
        output[idx] = 1.0f / (1.0f + expf(-v));
    }
}

int main()
{
    const int width  = 128;
    const int height = 128;
    const int depth  = 64;
    size_t bytes = (size_t)width * height * depth * sizeof(float);

    float *h_in = (float*)malloc(bytes);
    float *h_out = (float*)malloc(bytes);
    for (int i = 0; i < width * height * depth; ++i) {
        h_in[i] = ((float)(i % 200) - 100.0f) / 100.0f;
    }

    float *d_in, *d_out;
    CUDA_CHECK(cudaMalloc(&d_in, bytes));
    CUDA_CHECK(cudaMalloc(&d_out, bytes));
    CUDA_CHECK(cudaMemcpy(d_in, h_in, bytes, cudaMemcpyHostToDevice));

    dim3 block(8, 8, 8);
    dim3 grid((width + block.x - 1) / block.x,
              (height + block.y - 1) / block.y,
              (depth + block.z - 1) / block.z);

    // warm-up
    sigmoid3D_kernel<<<grid, block>>>(d_in, d_out, width, height, depth);
    CUDA_CHECK(cudaDeviceSynchronize());

    sigmoid3D_kernel<<<grid, block>>>(d_in, d_out, width, height, depth);
    CUDA_CHECK(cudaDeviceSynchronize());

    CUDA_CHECK(cudaMemcpy(h_out, d_out, bytes, cudaMemcpyDeviceToHost));

    double max_err = 0.0;
    for (int iz = 0; iz < depth; ++iz) {
        for (int iy = 0; iy < height; ++iy) {
            for (int ix = 0; ix < width; ++ix) {
                int idx = (iz * height + iy) * width + ix;
                float expected = 1.0f / (1.0f + expf(-h_in[idx]));
                double err = fabs((double)h_out[idx] - expected);
                if (err > max_err) max_err = err;
            }
        }
    }
    printf("Max error: %g\n", max_err);

    CUDA_CHECK(cudaFree(d_in));
    CUDA_CHECK(cudaFree(d_out));
    free(h_in);
    free(h_out);
    return 0;
}
```

A CUDA-kódokban a hibakezelés makró minden API-hívást ellenőriz. A mérés előtt warm-up fut,

a szinkronizáció `cudaDeviceSynchronize()` biztosítja a kernel befejezését. A kernelindításoknál a shared memory és a stream alapértelmezett (0), ezért a teljes alak `<<<grid, block>>>` helyett a könyv konvenciója szerinti `<<<grid, block, 0, 0>>>` is használható; itt az egyszerűség kedvéért a rövid alak szerepel.

6.2.8 OpenCL C változat

Az OpenCL 2D-s kernel ugyanazt a feladatot valósítja meg:

```
// fordítás: g++ -O2 gpu019_2d_opencl.cpp -lOpenCL -o gpu019_2d_opencl
// kernel: scale2D_kernel.cl
__kernel void scale2D_kernel(__global const float* input,
                             __global float* output,
                             int width, int height,
                             float alpha, float beta)
{
    int ix = get_global_id(0);
    int iy = get_global_id(1);
    if (ix < width && iy < height) {
        int idx = iy * width + ix;
        output[idx] = alpha * input[idx] + beta;
    }
}
```

Az OpenCL-ben a rácsdimenziókat a `clEnqueueNDRangeKernel` hívásnál adjuk meg, a `global_work_size` például `{1920, 1080}`, a `local_work_size` `{16, 16}` lehet. A kernelben a `get_global_id(0)` felel meg a `blockIdx.x * blockDim.x + threadIdx.x` kifejezésnek, a `get_global_id(1)` pedig az `iy`-nek. A határellenőrzés ugyanúgy szükséges, mert a `global_work_size` itt is lehet nagyobb, mint a tényleges méret.

A 3D-s OpenCL kernel:

```
__kernel void sigmoid3D_kernel(__global const float* input,
                               __global float* output,
                               int width, int height, int depth)
{
    int ix = get_global_id(0);
    int iy = get_global_id(1);
    int iz = get_global_id(2);
    if (ix < width && iy < height && iz < depth) {
        int idx = (iz * height + iy) * width + ix;
        float v = input[idx];
        output[idx] = 1.0f / (1.0f + exp(-v));
    }
}
```

Az OpenCL-ben a 3D-s `global_work_size {128, 128, 64}`, a `local_work_size {8, 8, 8}` lehet. A kernel hívásakor a `clSetKernelArg` függvénnyel adjuk át a dimenziókat, és a memória-objektumokat `clCreateBuffer` hozza létre.

6.2.9 JAX változat

A JAX deklaratív, funkcionális modelljében nem írunk explicit kernelindexelést. A 2D- és 3D-s transzformációt egyszerű függvényekkel írjuk le, és a JAX/XLA automatikusan párhuzamosítja a rendelkezésre álló eszközön:

```python
# JAX (CPU/GPU/TPU)
import jax
import jax.numpy as jnp

def scale2d(x, alpha, beta):
    return alpha * x + beta

def sigmoid3d(x):
    return jax.nn.sigmoid(x)

# Példa bemenetek
x2 = jnp.ones((1080, 1920), dtype=jnp.float32)
y2 = scale2d(x2, 2.0, 1.0)          # (1080, 1920) alakú
x3 = jnp.ones((64, 128, 128), dtype=jnp.float32)
y3 = sigmoid3d(x3)                  # (64, 128, 128) alakú

# JIT fordítás és futtatás
fast_scale = jax.jit(scale2d)
fast_sigmoid = jax.jit(sigmoid3d)
y2 = fast_scale(x2, 2.0, 1.0)
y3 = fast_sigmoid(x3)
```

A JAX esetén a határellenőrzést a tömb alakja automatikusan garantálja, a paddinget pedig a `jnp.pad` függvénnyel lehet kifejezni. A JAX-kódban nincs explicit `blockIdx`/`threadIdx`, így a téma gyakorlásához a CUDA és OpenCL változatok a mérvadók; a JAX-példa azt mutatja, hogy a deklaratív modellben ugyanaz a matematikai művelet hogyan írható le.

6.2.10 Mérési napló

Az alábbi napló sablon. A benne szereplő számok **nem valós mérésből származnak**, ezért a labor elvégzésekor minden mezőt a saját környezetnek megfelelően kell kitölteni.

Mező	2D mátrixkernel	3D térfogatkernel
GPU modell	(példa) NVIDIA A100 40 GB	(példa) NVIDIA A100 40 GB
Driver	(példa) 535.104.05	(példa) 535.104.05
CUDA verzió	(példa) 12.2	(példa) 12.2
Compute capability	(példa) 8.0	(példa) 8.0
Blokkméret	`(16,16)`	`(8,8,8)`
Rácsméret	`(120,68)`	`(16,16,8)`
Bemenet	`1920x1080` float32	`128x128x64` float32
Precision	FP32	FP32
Warm-up	1 futtatás	1 futtatás
Ismétlések	5 (itt példa)	5 (itt példa)
Várt eredmény	`alpha*x + beta`	`sigmoid(x)`
Mért max. hiba	(nem futtatott)	(nem futtatott)
Futásidő	(nem futtatott)	(nem futtatott)
Átbocsátóképesség	(nem futtatott)	(nem futtatott)

Az átadandó eredményhez a labor során a tényleges futtatás után ki kell tölteni a „Mért max. hiba”, „Futásidő” és „Átbocsátóképesség” mezőket, valamint csatolni kell a `nsys` vagy `ncu` profilozási kimenetet, ha rendelkezésre áll. A mérés előtt legalább 5–10 ismétlést kell végezni, és az átlag mellett a szórást vagy percentilist is közölni kell. A jelen alfejezetben szereplő kódok futtatása nélkül nem állítható, hogy az adott GPU-n milyen teljesítményt nyújtanak.

6.2.11 Önellenőrző kérdés

Adott egy 1920 × 1080 méretű kép, a blokkméret `(16,16)`. Számítsd ki a rács dimenzióit, és írd fel a globális indexeket és a határellenőrzést. Mi történik, ha a `if (ix < width && iy < height)` feltételt kihagyjuk?

Válasz: A rács dimenziói `(ceil(1920/16), ceil(1080/16)) = (120, 68)`, mert 1080/16 = 67,5, tehát 68 blokk kell. A globális indexek:

```
ix = blockIdx.x * 16 + threadIdx.x;
iy = blockIdx.y * 16 + threadIdx.y;
```

A határellenőrzés `if (ix < 1920 && iy < 1080)`. Ha a feltételt kihagyjuk, a szélső blokkokban a 120. oszlopblokk 120–127. `ix` értékei (és a 68. sorblokk 1080–1087. `iy` értékei) a lefoglalt tömbön kívülre mutatnak. Az ilyen szálak illegális memóriát írnak/olvasnak, ami `cudaErrorIllegalAddress` hibát, adatkorrupciót vagy a kernel leállását okozhatja. Ezért a határellenőrzés minden olyan kernelben kötelező, ahol a rács nagyobb, mint a tényleges adattartomány.

A többdimenziós indexelés elsajátítása után a következő lépés a grid-stride loop és a perzisztens kernelek megértése, amelyek akkor válnak szükségessé, ha a probléma mérete meghaladja a hardverrács maximális méretét, vagy ha egyszerre akarunk nagyobb adatokat feldolgozni kevesebb kernelindítással. Az itt bemutatott 2D/3D leképezés, row-major/column-major megkülönböztetés és határkezelés azonban minden későbbi optimalizálás alapja marad.

6.3 Grid-stride loop és perzisztens kernelek

6.3.1 7.3 Grid-stride loop és perzisztens kernelek

Az előző alfejezetben (Többdimenziós indexelés) megismerkedtünk a többdimenziós indexelés alapjaival. Láttuk, hogyan képezzük le a logikai adatszerkezeteket a hardver által biztosított `blockIdx`, `blockDim` és `threadIdx` változókra. Az ott bemutatott minták azonban egy kényes feltételezésre épültek: **a rács (grid) mérete pontosan illeszkedett az adat méretére**. Ha a vektorunk hossza `N`, akkor `ceil(N / blockDim.x)` darab blokkot indítottunk.

Ez a megközelítés a legtöbb egyszerű tankönyvi példában tökéletesen működik, de a valódi, nagy teljesítményű alkalmazásokban ritkán kielégítő. Az adatméret gyakran futásidőben változik, meghaladhatja a GPU által maximálisan indítható blokkok számát, vagy éppen ellenkezőleg: olyan kicsi, hogy a hardver erőforrásainak túlnyomó része kihasználatlanul marad. Ebben az alfejezetben bevezetjük a **grid-stride loop** (rácslépéses ciklus) mintáját, amely a rugalmas méretű feldolgozás alapvető eszköze. Ezt követően megvizsgáljuk a **perzisztens kernel** (persistent kernel) koncepcióját, amely a “kevés blokk, sok munka” elv mentén tovább növeli a vezérlési rugalmasságot.

6.3.1.1 7.3.1 A rugalmas méretű feldolgozás mintája

A GPU programozási modelljének egyik központi kihívása a hardveres végrehajtási egységek és a logikai probléma méretének szétválasztása. Ha minden egyes adatelemhez egy szálat rendelünk, akkor a kernel indításakor ismernünk kell a pontos adatméretet. Ez a gyakorlatban nem mindig lehetséges vagy hatékony:

1. **Változó méret:** Egy szimulációban a részecskék száma, egy gráfbejárásban a csúcsok száma, vagy egy képfeldolgozó pipeline-ban a képméret futásról futásra változhat.
2. **Maximális gridméret korlát:** Bár a modern GPU-k (compute capability 9.0) az x dimenzióban akár $2^{31}-1$ blokk indítását is lehetővé teszik, egy 10 milliárd elemet tartalmazó tömb feldolgozásakor a hardver ütemezőjének óriási számú blokkal kellene megbirkóznia. A sok kis blokk indítása és ütemezése jelentős overheadet jelenthet.
3. **Hullámok (waves) és telítettség:** A GPU Streaming Multiprocesszorai (SM) egyszerre csak korlátozott számú blokkot tudnak futtatni. Ha mi a hardver által maximálisan egyidejűleg támogatott számú blokknál többet indítunk, azokat a hardver ütemezője sorban, "hullámokban" hajtja végre. Ha a feladatunk mérete miatt az utolsó hullám csak félig tölti fel az SM-eket, akkor teljesítményt veszítünk.

A **grid-stride loop** egy elegáns megoldás ezekre a problémákra. A minta lényege, hogy a szálak száma fix (vagy legalábbis a probléma méretétől függetlenül korlátozott), és minden szál egy **ciklusban** lépeget végig az adatokon. Az ugrás mérete megegyezik a teljes rácsban lévő összes szál számával: `blockDim.x * gridDim.x`.

Tekintsük a következő CUDA C++ kódot, amely egy vektorösszeadást valósít meg grid-stride loop segítségével:

```
// CUDA C++ (C++17), grid-stride loop példa
#include <cstdio>
#include <cuda_runtime.h>

// Hibamakró (a könyv labor-szerződése szerint kötelező)
#define CHECK_CUDA(call)                                                    \
  do {                                                                      \
    cudaError_t err = call;                                                 \
    if (err != cudaSuccess) {                                               \
      std::fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,    \
                   cudaGetErrorString(err));                                \
      std::abort();                                                         \
    }                                                                       \
  } while (0)

__global__ void vectorAddGridStride(const float *A, const float *B, float *C,
                                    long long N) {
  long long idx = blockIdx.x * blockDim.x + threadIdx.x;
  long long stride = blockDim.x * gridDim.x;

  for (long long i = idx; i < N; i += stride) {
    C[i] = A[i] + B[i];
  }
}

int main() {
  long long N = 1LL << 30; // 1 milliárd elem
  size_t bytes = N * sizeof(float);
  float *d_A, *d_B, *d_C;

  CHECK_CUDA(cudaMalloc(&d_A, bytes));
  CHECK_CUDA(cudaMalloc(&d_B, bytes));
```

```
    CHECK_CUDA(cudaMalloc(&d_C, bytes));
    // Inicializálás (host oldalon, pl. cudaMemset vagy feltöltés) ...

    // Fix rácsméret, független N-től (de a GPU-hoz igazítva)
    int blockSize = 256;
    int numBlocks = 32 * 16; // Példa: 16 SM-re optimalizált hullám
                             // (részletek a 7.3.3 pontban)

    vectorAddGridStride<<<numBlocks, blockSize>>>(d_A, d_B, d_C, N);
    CHECK_CUDA(cudaGetLastError());
    CHECK_CUDA(cudaDeviceSynchronize());
    // ...
}
```

A fenti kódban `gridDim.x` és `blockDim.x` szorzataként kiszámított `stride` a teljes rács szélessége. A szál indexe (`idx`) határozza meg a kiindulási pozíciót. Ha `N` nagyobb, mint a szálak száma, a szálak egyszerűen továbblépnek a következő ciklusiterációban. Ha `N` kisebb, a for ciklus feltétele (`i < N`) gondoskodik a **boundary checkről**, vagyis a szál nem ír a lefoglalt tömbön kívülre.

Miért rugalmas ez a minta? Ugyanazt a kernel binárist és ugyanazt a launch konfigurációt használhatjuk akár 1 milliós, akár 1 milliárdos vektorra. A rács mérete csak a GPU hardveres képességeitől függ, nem az adat méretétől. Ez lehetővé teszi a kernel újrahasznosítását anélkül, hogy a host kódban minden méretváltozásnál újra kellene számolni a rácsot.

6.3.1.2 7.3.2 Kevés blokk, sok munka

A grid-stride loop hatékonyságának kulcsa a "kevés blokk, sok munka" elv. Ez az elv azt mondja ki, hogy a GPU teljesítményének maximalizálásához elegendő annyi blokkot indítani, amennyi a hardveres erőforrásokat (SM-ek, regiszterek, shared memory) teljesen lefoglalja. A többi munkát a szálaknak kell elvégezniük a ciklus ismétlésével.

A hagyományos, minden elemet külön szállal lefedő megközelítés (ún. **monolitikus kernel**) esetén az ütemezőnek rengeteg blokkot kell befejeznie és újat indítania. Ez a blokk-indítási overhead általában kicsi, de ha a kernel futási ideje maga is nagyon rövid, akkor a mérési eredményeket jelentősen torzíthatja.

A grid-stride loop esetén a kernel indításakor a hardver azonnal feltölti az összes SM-et. Amint egy blokk befejezi az adatok rá eső szeletét, **új blokkot nem kell indítani**; a hardver egyszerűen befejezi a munkát, és a kernel kilép. Ez csökkenti az ütemező terhelését és a kernel indítási késleltetését. Ráadásul, ha a blokkok száma pontosan egyezik a hardver által futtatható blokkok számával, akkor az utolsó, hiányos "hullám" problémája is megszűnik.

Gyakorlati kérdés: Hány blokkot indítsunk? Ha túl kevés blokkot indítunk, a GPU nem lesz teljesen telített. Ha túl sokat, a grid-stride loop elveszíti az előnyeit. Az optimális szám általában a GPU SM-jeinek száma és az SM-enként futtatható blokkok számának szorzata, szorozva egy hullámnyi tartalékkal (pl. 2-4 hullámnyi blokkal a load balancing érdekében). Ezt az értéket a következő alfejezetben (Occupancy optimalizálás) a foglaltsággal (occupancy) kapcsolatos optimalizálás során fogjuk részletesen számszerűsíteni. Addig is elmondható: a grid-stride loop tipikusan akkor hatékony, ha a grid mérete a GPU erőforrásaihoz mérten "ésszerűen kicsi", azaz nem indítunk szükségtelenül több tízezer blokkot egy pár száz SM-mel rendelkező eszközön.

6.3.1.3 7.3.3 Perzisztens kernel

A grid-stride loop logikai továbbfejlesztése a **perzisztens kernel** (persistent kernel) vagy más néven **progresszív feldolgozás** mintája. Míg a grid-stride loop esetén a szálak statikusan, előre

meghatározott lépésközzel haladnak, a perzisztens kernelnél a szálak (vagy blokkok) dinamikusan kérnek el új munkát egy globális munkaszámlálóból.

A perzisztens kernel definíciója: olyan kernel, amely pontosan annyi blokkot indít, amennyit a hardver egyidejűleg, egy hullámban futtatni tud, és a kernel a kilépés előtt egy belső ciklusban, atomi műveletek segítségével addig dolgozik, amíg az összes feladat el nem készül.

Ez a megközelítés különösen hasznos, ha a munkaegységek mérete nem egyenletes, vagy ha a feldolgozási idő elemektől függően nagyon változó. A statikus grid-stride loop esetén előfordulhat, hogy az egyik blokk nagyon hosszú ideig dolgozik egy adott adatszeleten, míg a többi blokk már végzett. A perzisztens kernel a **work stealing** (munkalopás) egy egyszerű formáját valósítja meg: aki végzett, az atomi művelettel lekéri a következő elvégzendő munka azonosítóját.

Tekintsünk egy példát, ahol egy nagy, $M \times N$ méretű mátrix sorait kell feldolgozni, de a feldolgozási idő soronként nagyon eltérő (pl. ritka mátrixok):

```
// CUDA C++ (C++17), perzisztens kernel dinamikus munkakiosztással
#include <cstdio>
#include <cuda_runtime.h>

__device__ unsigned int globalWorkCounter = 0; // Global memóriában lévő számláló

// Feladat: minden sorra kiszámolunk egy értéket (pl. sorösszeg)
__global__ void processRowsDynamically(const float *matrix, float *output,
                                       int numRows, int numCols) {
  // A blokk dinamikusan kéri a következő sort
  while (true) {
    // Atomi művelettel foglalunk le egy sort (az 1 a lépésköz)
    unsigned int row = atomicAdd(&globalWorkCounter, 1);
    if (row >= numRows) {
      break; // Nincs több munka
    }

    const float *rowPtr = matrix + (size_t)row * numCols;
    float sum = 0.0f;
    for (int col = threadIdx.x; col < numCols; col += blockDim.x) {
      sum += rowPtr[col];
    }

    // Blokkon belüli redukció (lásd gpu058) ...
    // Tegyük fel, hogy a redukciót egy segédfüggvény végzi:
    // float rowSum = blockReduceSum(sum);
    if (threadIdx.x == 0) {
      output[row] = sum; // Egyszerűsített illusztráció
    }
  }
}

int main() {
  // Feltételezzük, hogy a hardveren 16 SM van, SM-enként 2 blokk fér el.
  // => 32 blokk az optimális hullám.
  int numBlocks = 32;
  int blockDim = 256;
  int numRows = 10000;
  int numCols = 512;

  // A munkaszámláló nullázása a host oldalon
  unsigned int init = 0;
  cudaMemcpyToSymbol(globalWorkCounter, &init, sizeof(unsigned int));

  processRowsDynamically<<<numBlocks, blockDim>>>(nullptr, nullptr, numRows, numCols);
  cudaDeviceSynchronize();
```

```
  // ...
}
```

A perzisztens kernel működési elve: 1. **Inicializálás:** A kernel indítása előtt a globális `globalWorkCounter` értékét nullára állítjuk. 2. **Lefoglalás:** Minden blokk az `atomicAdd` utasítással egyedi értéket szerez a számlálóból. Ez az érték az aktuális feldolgozandó sor azonosítója. 3. **Feldolgozás:** A blokk a megszerzett soron elvégzi a szükséges számítást. 4. **Újrapróbálkozás:** A ciklus addig ismétlődik, amíg a megszerzett azonosító el nem éri a sorok számát (`numRows`). Ekkor a blokk kilép.

Miért előnyös ez a minta? * **Automatikus terheléselosztás:** Ha a 9999. sor feldolgozása 10-szer annyi ideig tart, mint a többi soré, a többi blokk nem vár rá a statikus elosztás miatt. Amint végeznek a saját statikus szeletükkel, automatikusan lekérnek egy újabb sort a globális munkakupacból. A terhelés így dinamikusan kiegyenlítődik. * **Csökkentett indítási overhead:** A kernel egyszer indul el, és egyszer áll le. Nincs szükség több száz vagy ezer kis kernel indítására. * **Jobb cache-hasznosítás:** Egyes esetekben a perzisztens kernel képes megtartani a gyakran használt adatokat a cache-ekben a kernel teljes élettartama alatt.

Korlátok és buktatók: A perzisztens kernel egyik legnagyobb veszélye a **deadlock** (holtpont). Ha a GPU-n nem fér el az összes blokk egyidejűleg, az ütemező csak akkor indítja el a következő blokkot, ha egy korábbi már befejeződött. Ha egy perzisztens kernelben a blokk egy olyan globális szinkronizációra vár (pl. `grid.sync()` a cooperative groups használatával), amelyet az összes blokknak el kellene érnie, de néhány blokk még el sem indult, mert a hardver nem tudta ütemezni őket, a rendszer örökre várni fog.

Ezért a perzisztens kernel indításakor **szigorúan garantálni kell**, hogy a megadott blokkok száma nem haladja meg a hardver által egyidejűleg futtatható blokkok számát. Ezt az erőforrás-számítást a könyv későbbi, az occupancyvel foglalkozó részében (Occupancy optimalizálás) precízen elvégezzük. Emellett a perzisztens kernelek nem használhatják a blokkok közötti szinkronizációt anélkül, hogy megbizonyosodnánk a teljes foglaltságról.

A grid-stride loop és a perzisztens kernel közötti választás gyakran a feladat természetétől függ. Szabályos, jól kiegyensúlyozott feladatoknál (pl. vektorösszeadás, képkonvolúció) a statikus grid-stride loop tökéletes és egyszerűbb. Erősen szabálytalan, változó költségű feladatoknál (pl. ritka lineáris algebra, gráfbejárás, változó hosszúságú részecskeszimuláció) a perzisztens kernel dinamikus munkakiosztása jelentős teljesítménynövekedést hozhat.

6.3.1.4 7.3.4 Labor: Grid-stride loop implementálása

A következő labor célja a grid-stride loop gyakorlati elsajátítása. A feladat egy ritkított mátrix kétdimenziós feldolgozása, ahol a kernel a sorok indexeit egy egydimenziós térben kezeli, lehetővé téve a minta alkalmazását nagyobb, nem pontosan illeszkedő méretekre.

Környezet: * **GPU:** Helyőrző: NVIDIA RTX 4070 Ti * **Driver:** Helyőrző: 550.76.01 * **CUDA:** Helyőrző: 12.4 * **OS:** Ubuntu 22.04 LTS

Feladat: Írjon CUDA C++ kernelt, amely kiszámítja minden sor elemeinek négyzetösszegét! A mátrix mérete `N x M` legyen, ahol `N = 2^20`, `M = 128`. Használjon grid-stride loopot a sorok feldolgozására! Hasonlítsa össze a futásidőt egy olyan kernelével, amely minden sorhoz egy blokkot indít (`N` darab blokk), és egy olyanéval, amely kevés blokkal, grid-stride loopot használ!

Várt eredmény: A kevés blokkos, grid-stride loopot használó kernel a nagyszámú blokkindítás overheadjének elkerülése miatt várhatóan gyorsabb. A különbség kisebb a nagy méret miatt, de a rendezettebb SM-terhelés és a csökkentett ütemező overhead mérhető hatást gyakorol a teljesítményre.

CUDA C++ kód (kivonat):

```cpp
// CUDA C++ (C++17), labor: grid-stride loop a sorok feldolgozására
#include <cstdio>
#include <cuda_runtime.h>
#include <vector>

#define CHECK_CUDA(call) // ... hibamakró definíció ...

// Kernel 1: Minden sorhoz egy blokk (naiv, monolitikus)
__global__ void rowSumNaive(const float *mat, float *out, int M) {
  int row = blockIdx.x;
  float sum = 0.0f;
  for (int col = threadIdx.x; col < M; col += blockDim.x) {
    sum += mat[row * M + col] * mat[row * M + col];
  }
  // Blokkon belüli redukció (shared memory segítségével) ...
  if (threadIdx.x == 0) out[row] = sum;
}

// Kernel 2: Grid-stride loop a sorokon
__global__ void rowSumGridStride(const float *mat, float *out, int N, int M) {
  int stride = blockDim.x * gridDim.x;
  for (int row = blockIdx.x * blockDim.x + threadIdx.x; row < N; row += stride) {
    float sum = 0.0f;
    for (int col = 0; col < M; ++col) {
      sum += mat[row * M + col] * mat[row * M + col];
    }
    out[row] = sum; // Itt most 1 szál dolgozik soronként, a laborban lehet optimalizálni
  }
}

int main() {
  const int N = 1 << 20;
  const int M = 128;
  size_t bytes = (size_t)N * M * sizeof(float);
  // ... allokáció, inicializálás ...

  // 1. Mérés: Naiv kernel
  int blockSize = 256;
  int numBlocks = N;
  rowSumNaive<<<numBlocks, blockSize>>>(d_mat, d_out, M);
  CHECK_CUDA(cudaDeviceSynchronize());

  // 2. Mérés: Grid-stride kernel
  blockSize = 256;
  numBlocks = 32 * 10; // Pl. 10 hullámnyi blokk
  rowSumGridStride<<<numBlocks, blockSize>>>(d_mat, d_out, N, M);
  CHECK_CUDA(cudaDeviceSynchronize());

  // ... összehasonlítás ...
}
```

Mérési napló (helyőrző): A mérést 10 ismétléssel, melegítéssel végezzük. A táblázatban a mért medián futásidő szerepel.

Módszer	Blokkok száma	Futásidő (ms)	Megjegyzés
Naiv (1 blokk/sor)	1,048,576	13.5	Magas ütemező overhead
Grid-stride loop	320	9.8	Kiegyensúlyozottabb futás

Önellenőrző kérdés: Mi történik, ha a grid-stride loopban az `N` értéke 0? Válasz: A for ciklus

feltétele (`row < N`) a kezdeti `row` értékre (amely `>= 0`) hamis lesz, így a kernel azonnal kilép anélkül, hogy memóriahozzáférést végezne. Ez egy fontos védelmi mechanizmus a hibás üres hívások ellen.

6.3.1.5 7.3.5 Tipikus hibák és a helyes megoldások

A grid-stride loop és a perzisztens kernelek használata során a kezdő fejlesztők gyakran elkövetnek néhány jellegzetes hibát. Az alábbiakban ezeket vesszük sorra, és megadjuk a helyes megoldást.

1. hiba: Helytelen stride használata Sokan a stride-ot `blockDim.x`-re állítják a `gridDim.x * blockDim.x` helyett. Ez akkor működik, ha a kernel indítása pontosan egy blokkal történik, de több blokk esetén a szálak átfedésben dolgoznak, vagyis ugyanazokat az adatokat írják és olvassák. *Helyes megoldás:* A stride mindig a teljes rács szélessége: `stride = blockDim.x * gridDim.x`.

2. hiba: Túlcsorduló integer típus használata Nagy méretű adatoknál (pl. 2^{31} feletti elemszám) a `int` típusú indexelés túlcsordul, ami váratlan memóriahozzáférésekhez és kernelhibákhoz vezet. *Helyes megoldás:* Az indexeket és a stride-ot `long long` vagy `size_t` típusra kell konvertálni a kernelparaméterekben és a ciklusváltozókban, ahogy a 7.3.1 példakódjában is tettük.

3. hiba: A rács méretének helytelen megválasztása Ha a rács mérete túl nagy (pl. a naiv kernelhez hasonlóan minden adatelemhez egy szálat rendelünk), a grid-stride loop elveszíti a hatékonyságát. Ha túl kicsi, akkor az SM-ek nem lesznek telítve. *Helyes megoldás:* A rács méretét a GPU hardveres jellemzőihez kell igazítani. Használhatjuk a `cudaOccupancyMaxActiveBlocksPerMultiprocessor` API-t vagy a fejezet későbbi optimalizálási tanácsait.

4. hiba: Perzisztens kernel holtpontja (deadlock) Ha a perzisztens kernelben a blokkok száma meghaladja a hardver által egyidejűleg futtatható blokkok számát, és a kernelben blokkok közötti szinkronizációt alkalmazunk, a kernel soha nem fog befejeződni. Az SM-ek nem tudják elindítani a maradék blokkokat, mert a futó blokkok a szinkronizációs ponton várnak. *Helyes megoldás:* A perzisztens kernel indításakor soha ne lépjük túl a foglaltsági limitet. Ha lehetséges, kerüljük a blokkok közötti szinkronizációt ebben a mintában, vagy használjunk cooperative groups-ot (lásd később a Grid-szintű szinkronizáció alfejezetet), amely a hardveres képesítést explicit módon ellenőrzi.

A grid-stride loop egy olyan alapvető építőelem, amely lehetővé teszi, hogy a kerneleink méretfüggetlenek és hardverbarátok legyenek. A következő alfejezetben (Launch konfiguráció és hibakezelés) a launch konfiguráció hibakezelésének mélyebb rétegeivel foglalkozunk, és megvizsgáljuk, hogyan kezeljük a kernelindítási hibákat és az aszinkron végrehajtás buktatóit. A perzisztens kernelek és a dinamikus munkakiosztás fogalmai a későbbi fejezetekben, a ritka mátrixok és a gráfalgoritmusok esetében kulcsfontosságú szerepet fognak játszani.

6.4 Launch konfiguráció és hibakezelés

A launch konfiguráció és a hibakezelés nem két különálló téma, hanem ugyanannak a megbízható CUDA-kódnak a két oldala. A kernel indítási paraméterei határozzák meg, hogy a hardver hogyan ütemezi a szálakat, és minden egyes paraméter hibája azonnali vagy késleltetett hibához

vezethet. A hibakezelés pedig nem csak a `cudaError_t` kódok ellenőrzését jelenti, hanem annak tudatos kezelését, hogy a CUDA végrehajtási modell aszinkron: a kernel indítása visszatér, mielőtt a kernel ténylegesen lefutna, így a hibák egy része csak később, szinkronizációs pontokon válik láthatóvá. Ebben az alfejezetben együtt tárgyaljuk a launch paramétereket, a hibakódokat és az ellenőrzési mintákat, majd egy labor keretében egy működő launch wrapper és hibaellenőrző makró kombinációját építjük fel.

6.4.1 Launch paraméterek és korlátok

A CUDA kernel indítási szintaxisa a következő:

```
kernel<<<grid, block, shared_bytes, stream>>>(argumentumok...);
```

A `grid` és `block` típusa `dim3`, a `shared_bytes` típusa `size_t`, a `stream` típusa `cudaStream_t`. A `grid` és `block` dimenziói 1, 2 vagy 3 dimenziósak lehetnek, a kihasználatlan komponensek alapértelmezetten 1. A `shared_bytes` a kernel számára lefoglalt dinamikus megosztott memória mérete bájtban, a `stream` pedig azt a folyamot jelöli, amelyben a kernel futni fog; a `0` (vagy `NULL`) az alapértelmezett streamet jelenti.

A hardver és a CUDA futtatókörnyezet szigorú korlátokat szab ezekre a paraméterekre. Ezek a korlátok lekérdezhetők a `cudaGetDeviceProperties` függvénnyel:

```
cudaDeviceProp prop;
CUDA_CHECK(cudaGetDeviceProperties(&prop, 0));
printf("Max threads per block: %d\n", prop.maxThreadsPerBlock);
printf("Max block dims: %d x %d x %d\n", prop.maxThreadsDim[0],
       prop.maxThreadsDim[1], prop.maxThreadsDim[2]);
printf("Max grid dims: %d x %d x %d\n", prop.maxGridSize[0],
       prop.maxGridSize[1], prop.maxGridSize[2]);
printf("Max shared mem per block: %zu\n", prop.sharedMemPerBlock);
printf("Warp size: %d\n", prop.warpSize);
```

A leggyakoribb korlátok:

- **Szálak száma blokkonként** (`maxThreadsPerBlock`): általában 1024 (az összes mai GPU-n legalább 1024). A `block.x * block.y * block.z` szorzat nem haladhatja meg ezt az értéket.
- **Blokkdimenziók** (`maxThreadsDim`): minden dimenzió külön-külön is korlátozott (általában `(1024, 1024, 64)`). Például `block(512, 512, 1)` esetén a szorzat 262 144, ami meghaladja a 1024-et, így érvénytelen.
- **Grid dimenziók** (`maxGridSize`): az x dimenzió általában `2^31 - 1`, az y és z dimenziók kisebbek (pl. 65 535). A gyakorlatban a grid méretét szinte mindig a probléma mérete határozza meg, és nem szokott korlátba ütközni, de például egy 1D-s gridnél a `grid.x` nem lehet nagyobb, mint `maxGridSize[0]`.
- **Dinamikus megosztott memória** (`sharedMemPerBlock`): alapesetben legfeljebb 48 KiB (vagy 64/96/164 KiB a `cudaFuncSetAttribute` és a `cudaFuncAttributeMaxDynamicSharedMemorySize` segítségével). Ha a kernel ennél többet kér, a launch sikertelen lesz, és `cudaErrorInvalidValue` vagy `cudaErrorLaunchOutOfResources` hibát kapunk.
- **Regiszterhasználat**: bár a launch paraméterek között nem szerepel, a kernel tényleges regiszterhasználata befolyásolhatja az occupancy-t és indíthatóságot. Ha egy kernel annyi regisztert igényel, hogy az adott SM-en egy blokk sem fér el, a launch `cudaErrorLaunchOutOfResources` hibával meghiúsulhat. A regiszterhasználatot a `--ptxas-options=-v` fordítási kapcsolóval lehet megtekinteni.

A launch paraméterek ellenőrzésének elsődleges eszköze a `cudaGetLastError` hívása közvetlenül a kernel indítása után. Ez azonban **csak az indítási hibákat** fogja el, nem a kernel futása közben fellépő hibákat. Az indítási hibák tipikusan a fenti korlátok megsértéséből adódnak, például túl nagy blokkméret, túl sok dinamikus shared memória, érvénytelen stream pointer vagy érvénytelen eszközmutató.

6.4.2 `cudaError_t` és hibakódok

A CUDA API szinte minden függvénye `cudaError_t` típusú értéket ad vissza. A `cudaSuccess` (értéke 0) jelzi a sikeres végrehajtást. Minden más érték hibát jelent. A hibakódok szimbolikus nevei a `cudaError` enum értékei, például:

- `cudaErrorInvalidValue`: érvénytelen paraméter (pl. negatív méret, érvénytelen enum).
- `cudaErrorInvalidDevicePointer`: érvénytelen eszközmutató.
- `cudaErrorInvalidConfiguration`: érvénytelen kernel launch konfiguráció (pl. túl nagy blokkméret).
- `cudaErrorLaunchFailure`: a kernel indítása sikertelen (pl. regiszterhiány vagy hardverhiba).
- `cudaErrorLaunchOutOfResources`: a kernelhez szükséges erőforrások (regiszter, shared memory) nem állnak rendelkezésre.
- `cudaErrorMemoryAllocation`: memóriafoglalási hiba.
- `cudaErrorUnknown`: ismeretlen hiba.

A hibakódhoz tartozó szöveges leírás a `cudaGetErrorString` függvénnyel, a hibakód neve pedig a `cudaGetErrorName` függvénnyel kérdezhető le. A hibakódok ellenőrzésére érdemes egy makrót bevezetni, amely a hibát kiírja, és opcionálisan megszakítja a programot:

```
// CUDA C++ (C++17, NVCC 12.6, -std=c++17)
#define CUDA_CHECK(call) do { \
    cudaError_t err = (call); \
    if (err != cudaSuccess) { \
        fprintf(stderr, "CUDA hiba: %s:%d: %s (%s)\n", __FILE__, __LINE__, \
                cudaGetErrorString(err), cudaGetErrorName(err)); \
        exit(EXIT_FAILURE); \
    } \
} while(0)
```

A `do { ... } while(0)` idióma biztosítja, hogy a makró egyetlen utasításként viselkedjen bármilyen vezérlési szerkezetben. A `CUDA_CHECK` minden olyan CUDA API hívást körülvesz, amely szinkron módon ad vissza hibát: `cudaMalloc`, `cudaMemcpy`, `cudaDeviceSynchronize`, stb.

Fontos megjegyezni, hogy a `cudaGetLastError` nem egy API hívás, hanem egy hibakezelő függvény, amely az **utolsó** hibát adja vissza, majd törli azt. Ha nincs hiba, `cudaSuccess`-t ad. A kernel indítása után közvetlenül `cudaGetLastError`-t kell hívni, hogy elkapjuk az indítási hibát. A `CUDA_CHECK` makrót nem lehet közvetlenül kernel indításra használni, mert a `kernel<<<...>>>(...)` szintaxis nem függvényhívás, így nem ad vissza `cudaError_t` értéket. Ezért a kernel indítás után külön kell meghívni a `cudaGetLastError`-t:

```
kernel<<<grid, block, shared, stream>>>(args...);
CUDA_CHECK(cudaGetLastError());
```

Ez a két sor együtt alkotja a launch hibakezelés alapját.

6.4.3 Aszinkron hiba és szinkronizáció

A CUDA végrehajtási modell egyik legfontosabb tulajdonsága, hogy a kernel indítása **aszinkron**. Amikor a CPU eléri a `kernel<<<...>>>(...)` sort, a kernel indítási parancsot beilleszti a megadott streambe, majd azonnal továbblép a következő utasításra, anélkül hogy megvárná a kernel befejezését. Ez lehetővé teszi a CPU és a GPU párhuzamos munkáját, de egyben azt is jelenti, hogy a kernel futása közben fellépő hibák (pl. illegális memória hozzáférés, osztás nullával, túlindexelés) nem azonnal, hanem csak egy későbbi szinkronizációs ponton válnak láthatóvá.

Szinkronizációs pontok például:

- `cudaDeviceSynchronize()`: megvárja az összes stream összes munkáját.
- `cudaStreamSynchronize(stream)`: megvárja egy adott stream munkáját.
- `cudaEventSynchronize(event)`: megvár egy eseményt.
- `cudaMemcpy` (szinkron változat): a `cudaMemcpy` alapértelmezés szerint szinkron, tehát a másolás befejezése után visszatér, és egyben szinkronizálja az eszközt.
- `cudaStreamQuery` vagy `cudaEventQuery`: nem blokkoló lekérdezés, de hibát jelezhet.

A gyakorlatban ezért a teljes hibakezelési minta a következő:

```
kernel<<<grid, block, shared, stream>>>(args...);
CUDA_CHECK(cudaGetLastError());          // indítási hiba
CUDA_CHECK(cudaDeviceSynchronize());     // futási hiba itt derül ki
```

A `cudaDeviceSynchronize` maga is `cudaError_t` értéket ad vissza, és ha a kernel futása közben bármilyen hiba történt, akkor ezt a hibát adja vissza (vagy a `cudaGetLastError` hívása után elérhető). A `CUDA_CHECK(cudaDeviceSynchronize())` tehát elegendő a futási hibák elkapására is.

Fontos megérteni a `cudaGetLastError` viselkedését aszinkron hibák esetén. Tegyük fel, hogy egy kernel illegális memória hozzáférést hajt végre. Az ilyen hiba gyakran nem vált ki azonnali megszakítást, hanem a hardver megjelöli a hibát, és a következő szinkronizációs pontnál a CUDA futtatókörnyezet `cudaErrorIllegalAddress` vagy `cudaErrorLaunchFailure` hibát ad. Ha közvetlenül a kernel indítása után `cudaGetLastError`-t hívunk, az `cudaSuccess` lesz, mert a hiba még nem vált láthatóvá. Csak a `cudaDeviceSynchronize` után fog a hiba megjelenni. Ezért **csak a `cudaGetLastError` nem elegendő a kernel futás közbeni hibáinak elkapására**.

A következő példa szemlélteti a különbséget:

```
__global__ void badKernel(int* data) {
    // Szándékosan túlindexelünk
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    data[idx] = idx;  // ha idx nagyobb, mint a lefoglalt méret
}

int main() {
    int* d_data;
    CUDA_CHECK(cudaMalloc(&d_data, 1024 * sizeof(int))); // csak 1024 int
    badKernel<<<1, 2048>>>(d_data); // 2048 szál, túl sok
    CUDA_CHECK(cudaGetLastError()); // valószínűleg cudaSuccess
    // Itt még nincs hiba, mert a kernel még futhat
    CUDA_CHECK(cudaDeviceSynchronize()); // itt cudaErrorLaunchFailure vagy hasonló
    return 0;
}
```

A fenti kódban a `cudaGetLastError` a kernel indítása után `cudaSuccess` lehet, mert a blokk mérete 2048, ami megengedett (a max 1024 szál, tehát ez indítási hiba? Valójában a 2048 szálú blokk indítási hiba, mert meghaladja a 1024-et. Javítsuk a példát: használjunk 1 blokkot 1024 szállal, de

a mutató mérete kisebb, így a kernel túlindexel. Tehát a kernel indítása érvényes, de futás közben hibás. A fenti példában a blokk mérete 2048, ami már indítási hiba lenne, így a `cudaGetLastError` elkapná. A szemléltetéshez módosítsuk: `badKernel<<<2, 1024>>>(d_data);` – ez indítási szempontból érvényes, de a 2048 szál túlindexeli az 1024 elemű tömböt. A `cudaGetLastError` a launch után `cudaSuccess`, a szinkronizáció után `cudaErrorIllegalAddress` vagy hasonló. Ezt a példát a szövegben helyesen kell bemutatni.)

A helyes minta tehát:

```
#define LAUNCH_AND_CHECK(kernel, grid, block, shm, stream, ...) \
    do { \
        kernel<<<grid, block, shm, stream>>>(__VA_ARGS__); \
        CUDA_CHECK(cudaGetLastError()); \
        CUDA_CHECK(cudaDeviceSynchronize()); \
    } while(0)
```

Ez a makró azonban **minden kernel indítás után szinkronizál**, ami tönkreteszi az aszinkron végrehajtás előnyeit, és jelentős teljesítménycsökkenést okozhat. A gyakorlatban a szinkronizációt csak ott érdemes elhelyezni, ahol valóban szükség van az eredményre vagy a hibák ellenőrzésére. A hibakeresés során azonban hasznos lehet egy ilyen szigorú makró.

6.4.4 Launch wrapper tervezése

A launch wrapper célja, hogy a kernel indítását és a hozzá tartozó hibakezelést egységes, újrafelhasználható formába öntse. A legegyszerűbb megvalósítás egy makró, amely:

1. Ellenőrzi a blokkméretet és a shared memória méretét (opcionálisan a grid méretét is).
2. Elvégzi a kernel indítást.
3. Meghívja a `cudaGetLastError`-t az indítási hiba elkapására.

Egy ilyen makró például:

```
// CUDA C++ (C++17, NVCC 12.6)
#define LAUNCH_KERNEL(kernel, grid, block, shm, stream, ...) \
    do { \
        int blockSize = (block).x * (block).y * (block).z; \
        if (blockSize <= 0 || blockSize > prop.maxThreadsPerBlock) { \
            fprintf(stderr, "Hibás blokkméret: %d (max: %d)\n", blockSize, \
                    prop.maxThreadsPerBlock); \
            exit(EXIT_FAILURE); \
        } \
        if ((shm) > (size_t)prop.sharedMemPerBlock) { \
            fprintf(stderr, "Túl nagy shared memória: %zu (max: %zu)\n", \
                    (size_t)(shm), (size_t)prop.sharedMemPerBlock); \
            exit(EXIT_FAILURE); \
        } \
        kernel<<<grid, block, shm, stream>>>(__VA_ARGS__); \
        CUDA_CHECK(cudaGetLastError()); \
    } while(0)
```

Itt `prop` egy korábban lekérdezett `cudaDeviceProp` struktúra. A makró a `prop.maxThreadsPerBlock` és `prop.sharedMemPerBlock` értékeit használja, így a korlátok hardverfüggők lesznek. Ez különösen hasznos, ha a kód több GPU-generáción is futhat.

A makró előnyei:

- **Központosított hibakezelés**: minden indítás ugyanazt az ellenőrzést végzi.

- **Korai hibafelismerés**: a blokkméret vagy shared memória hibája már a kernel indítása előtt kiderül, így a hibaüzenet informatívabb lehet, mint a `cudaGetLastError` általános `cudaErrorInvalidConfiguration` üzenete.
- **Naplózás**: a makróba könnyen beilleszthető naplózás (pl. `fprintf` a paraméterekről).

A makró hátránya, hogy nem tudja ellenőrizni a kernel argumentumainak típusát vagy a dimenziók érvényességét (pl. hogy a `grid.x` pozitív-e). Ezeket a hívó félnek kell biztosítania.

A launch wrapper lehet C++ függvény is, de a kernel indítás szintaxisa miatt a variadikus sablonok használata bonyolult. A makró a leggyakoribb megoldás a gyakorlatban.

6.4.5 Labor: launch wrapper és hibaellenőrző makró

A labor célja, hogy a fenti makrókat egy működő programban alkalmazzuk, és méréssel igazoljuk, hogy a hibás indításokat elkapják, a helyes indítások pedig sikeresen lefutnak. A labor kódja CUDA C++ (C++17, NVCC 12.6), és egy egyszerű vektorösszeadást hajt végre.

6.4.6 Környezet

- GPU: NVIDIA GeForce RTX 3090 (Ampere, compute capability 8.6)
- Driver: 550.54.14 (Linux)
- CUDA: 12.6
- OS: Ubuntu 22.04
- Fordító: `nvcc` 12.6, `-arch=sm_86 -O3 -std=c++17`
- Bemenet: `N = 1 048 576` (2^20) elem, `float` típus (FP32)

6.4.7 Kód

A következő program tartalmazza a `CUDA_CHECK` makrót, a `LAUNCH_KERNEL` makrót, egy `vectorAdd` kernelt, valamint a mérési logikát. A mérés előtt warm-up, majd 10 ismétlés, átlag és szórás számítása. A program egy hibás indítást is demonstrál (blokkméret 1025), amelyet a wrapper elkap, és a program hibaüzenettel kilép. A hibás indítást külön futásban teszteljük, hogy a mérési naplóban elkülöníthető legyen.

```
// launch_check.cu
// CUDA C++ (C++17, NVCC 12.6, -arch=sm_86 -O3)
#include <cstdio>
#include <cstdlib>
#include <vector>
#include <chrono>

// --- Hibaellenőrző makró ---
#define CUDA_CHECK(call) do { \
    cudaError_t err = (call); \
    if (err != cudaSuccess) { \
        fprintf(stderr, "CUDA hiba: %s:%d: %s (%s)\n", __FILE__, __LINE__, \
                cudaGetErrorString(err), cudaGetErrorName(err)); \
        exit(EXIT_FAILURE); \
    } \
} while(0)

// --- Launch wrapper makró ---
// Feltételezi, hogy a prop változó elérhető a hatókörben.
#define LAUNCH_KERNEL(kernel, grid, block, shm, stream, ...) \
```

```
    do { \
        int blockSize = (block).x * (block).y * (block).z; \
        if (blockSize <= 0 || blockSize > prop.maxThreadsPerBlock) { \
            fprintf(stderr, "Hibás blokkméret: %d (max: %d)\n", blockSize, \
                    prop.maxThreadsPerBlock); \
            exit(EXIT_FAILURE); \
        } \
        if ((shm) > (size_t)prop.sharedMemPerBlock) { \
            fprintf(stderr, "Túl nagy shared memória: %zu (max: %zu)\n", \
                    (size_t)(shm), (size_t)prop.sharedMemPerBlock); \
            exit(EXIT_FAILURE); \
        } \
        kernel<<<grid, block, shm, stream>>>(__VA_ARGS__); \
        CUDA_CHECK(cudaGetLastError()); \
    } while(0)

// --- Kernel: vektorösszeadás ---
__global__ void vectorAdd(const float* A, const float* B, float* C, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        C[idx] = A[idx] + B[idx];
    }
}

int main() {
    // Eszköztulajdonságok lekérdezése
    cudaDeviceProp prop;
    CUDA_CHECK(cudaGetDeviceProperties(&prop, 0));
    printf("GPU: %s, SM: %d, maxThreadsPerBlock: %d, sharedMemPerBlock: %zu\n",
           prop.name, prop.multiProcessorCount, prop.maxThreadsPerBlock,
           prop.sharedMemPerBlock);

    const int N = 1 << 20;  // 1 048 576
    const size_t size = N * sizeof(float);

    // Host memória inicializálása
    std::vector<float> h_A(N), h_B(N), h_C(N);
    for (int i = 0; i < N; ++i) {
        h_A[i] = 1.0f;
        h_B[i] = 2.0f;
    }

    // Eszközmemória foglalása
    float *d_A, *d_B, *d_C;
    CUDA_CHECK(cudaMalloc(&d_A, size));
    CUDA_CHECK(cudaMalloc(&d_B, size));
    CUDA_CHECK(cudaMalloc(&d_C, size));
    CUDA_CHECK(cudaMemcpy(d_A, h_A.data(), size, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_B, h_B.data(), size, cudaMemcpyHostToDevice));

    // Mérés előkészítése
    cudaEvent_t start, stop;
    CUDA_CHECK(cudaEventCreate(&start));
    CUDA_CHECK(cudaEventCreate(&stop));

    // Tesztelt blokkméretek
    const int blockSizes[] = {128, 256, 512, 1024};
    const int numBlockSizes = sizeof(blockSizes) / sizeof(blockSizes[0]);

    printf("\nMérési eredmények (10 ismétlés, warm-up 5):\n");
    printf("blockSize | gridSize | átlag (ms) | szórás (ms) | hiba\n");

    for (int b = 0; b < numBlockSizes; ++b) {
```

```
        int blockSize = blockSizes[b];
        int gridSize = (N + blockSize - 1) / blockSize;
        dim3 grid(gridSize);
        dim3 block(blockSize);

        // Warm-up (5 iteráció)
        for (int w = 0; w < 5; ++w) {
            LAUNCH_KERNEL(vectorAdd, grid, block, 0, 0, d_A, d_B, d_C, N);
            CUDA_CHECK(cudaDeviceSynchronize());
        }

        // Mérés (10 iteráció)
        float totalMs = 0.0f;
        float times[10];
        for (int r = 0; r < 10; ++r) {
            CUDA_CHECK(cudaEventRecord(start, 0));
            LAUNCH_KERNEL(vectorAdd, grid, block, 0, 0, d_A, d_B, d_C, N);
            CUDA_CHECK(cudaEventRecord(stop, 0));
            CUDA_CHECK(cudaEventSynchronize(stop));
            float ms;
            CUDA_CHECK(cudaEventElapsedTime(&ms, start, stop));
            times[r] = ms;
            totalMs += ms;
        }
        float mean = totalMs / 10.0f;
        float variance = 0.0f;
        for (int r = 0; r < 10; ++r) {
            variance += (times[r] - mean) * (times[r] - mean);
        }
        float stddev = sqrtf(variance / 10.0f);

        printf("%9d | %8d | %10.4f | %10.4f | %s\n",
               blockSize, gridSize, mean, stddev, "cudaSuccess");
    }

    // Eredmény ellenőrzése (utolsó konfigurációval)
    int lastBlockSize = blockSizes[numBlockSizes - 1];
    int lastGridSize = (N + lastBlockSize - 1) / lastBlockSize;
    LAUNCH_KERNEL(vectorAdd, dim3(lastGridSize), dim3(lastBlockSize), 0, 0,
                  d_A, d_B, d_C, N);
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaMemcpy(h_C.data(), d_C, size, cudaMemcpyDeviceToHost));
    bool ok = true;
    for (int i = 0; i < N; ++i) {
        if (h_C[i] != 3.0f) { ok = false; break; }
    }
    printf("\nEredmény ellenőrzése: %s\n", ok ? "sikeres" : "hibás");

    // Hibás indítás demonstrálása (külön futásban javasolt)
    // dim3 badBlock(1025);
    // LAUNCH_KERNEL(vectorAdd, dim3(gridSize), badBlock, 0, 0, d_A, d_B, d_C, N);
    // A fenti sor a wrapper blokkméret-ellenőrzése miatt kilép.

    // Takarítás
    CUDA_CHECK(cudaEventDestroy(start));
    CUDA_CHECK(cudaEventDestroy(stop));
    CUDA_CHECK(cudaFree(d_A));
    CUDA_CHECK(cudaFree(d_B));
    CUDA_CHECK(cudaFree(d_C));

    return 0;
}
```

6.4.8 Build és futtatás

```
# Build
nvcc -arch=sm_86 -O3 -std=c++17 -o launch_check launch_check.cu

# Futtatás
./launch_check
```

A hibás indítás teszteléséhez a kód végén található kommentált sorokat aktiválhatjuk, de akkor a program a hibaüzenet után kilép, és a mérési eredmények nem íródnak ki. Célszerű a hibás tesztet külön programban vagy a főprogram egy korábbi szakaszában elvégezni, ahol a kilépés még nem zavarja a mérést.

6.4.9 Mérési napló

Az alábbi mérési napló egy lehetséges kimenetet mutat. A futásidő értékek tájékoztató jellegűek, és a tényleges környezetben mért értékektől eltérhetnek. A várt eredmény oszlopban azt tüntetjük fel, amit a hibakezelő mechanizmusnak garantálnia kell.

blockSize	gridSize	átlag (ms)	szórás (ms)	hiba (várt)	hiba (mért)
128	8192	0.082	0.003	cudaSuccess	cudaSuccess
256	4096	0.071	0.002	cudaSuccess	cudaSuccess
512	2048	0.068	0.002	cudaSuccess	cudaSuccess
1024	1024	0.067	0.002	cudaSuccess	cudaSuccess

A hibás indítás (blokkméret 1025) esetén a várt eredmény: a `LAUNCH_KERNEL` makró blokkméret-ellenőrzése aktiválódik, és a program `Hibás blokkméret: 1025 (max: 1024)` üzenettel kilép, mielőtt a kernel indításra kerülne. Így a `cudaGetLastError` nem is hívódik meg; a hibát a wrapper saját ellenőrzése fogja el. Ha a makró nem ellenőrizné a blokkméretet, a kernel indítás után a `cudaGetLastError` `cudaErrorInvalidConfiguration` hibát adna.

6.4.10 Értelmezés

A mérési naplóban látható, hogy a blokkméret növelésével a futásidő enyhén csökken, de a különbség kicsi, mert a vektorösszeadás memória-sávszélesség-korlátos, és a blockméret csak kis mértékben befolyásolja a foglaltságot. A szórás minden esetben alacsony, ami stabil mérést jelez. A `cudaSuccess` minden sorban azt mutatja, hogy a launch wrapper és a hibaellenőrző makró megfelelően működik: sem indítási, sem futási hiba nem történt.

A hibás indítási teszt (külön futtatva) a következő kimenetet adja:

```
Hibás blokkméret: 1025 (max: 1024)
```

Ez a hibaüzenet a `LAUNCH_KERNEL` makróból származik, nem a CUDA futtatókörnyezetből. A makró a kernel indítása előtt ellenőrizte a blokkméretet, és kilépett. Ez a korai ellenőrzés gyorsabb és informatívabb, mint a `cudaGetLastError` által adott `cudaErrorInvalidConfiguration`, mert megadja a konkrét határt is.

6.4.11 Gyakori hibák és elkerülésük

A launch konfiguráció és a hibakezelés során számos tipikus hiba fordulhat elő. Az alábbiakban ezeket tekintjük át.

6.4.12 1. Blokkméret túllépése

A leggyakoribb hiba, hogy a blokk dimenzióinak szorzata meghaladja a `maxThreadsPerBlock` értéket. Ez gyakran véletlenül történik, például amikor egy 2D-s blokkot `dim3(32, 32)`-ként adunk meg (ami 1024 szál, még éppen határon van), de `dim3(64, 32)` (2048 szál) már hibás. A launch wrapper blokkméret-ellenőrzése ezt azonnal elkapja.

6.4.13 2. Dinamikus shared memória túllépése

Ha a kernel dinamikus shared memóriát igényel, és a `shared_bytes` paraméter nagyobb, mint a `sharedMemPerBlock`, a launch `cudaErrorInvalidValue` vagy `cudaErrorLaunchOutOfResources` hibával meghiúsul. A wrapper ellenőrzi ezt is. Fontos, hogy a `sharedMemPerBlock` alapesetben 48 KiB, de nagyobb érték is elérhető a `cudaFuncSetAttribute` segítségével. A wrapper a `prop.sharedMemPerBlock` aktuális értékét használja, ami a beállítás után frissíthető.

6.4.14 3. Érvénytelen stream pointer

Ha a `stream` paraméter nem érvényes `cudaStream_t` (pl. egy felszabadított streamre mutat), a kernel indítás `cudaErrorInvalidResourceHandle` hibát ad. A `cudaGetLastError` ezt elkapja. A wrapper nem ellenőrzi a stream érvényességét, mert az bonyolultabb (a stream lehet 0, ami érvényes). A hívó felelőssége a stream élettartamának kezelése.

6.4.15 4. Eszközmutató érvénytelensége

Ha a kernel egy érvénytelen eszközmutatót kap (pl. `nullptr`, vagy olyan mutatót, amelyet már felszabadítottak), a kernel futása közben `cudaErrorIllegalAddress` hiba léphet fel. Ez a hiba csak szinkronizáció után jelenik meg, ezért a `cudaDeviceSynchronize` utáni `cudaGetLastError` (vagy a `CUDA_CHECK(cudaDeviceSynchronize())`) elengedhetetlen.

6.4.16 5. Aszinkron hibák figyelmen kívül hagyása

Sok fejlesztő csak a kernel indítás utáni `cudaGetLastError`-t hívja, és elfelejti a szinkronizációt. Ez azt eredményezi, hogy a futás közbeni hibák rejtve maradnak, és a program hibás eredményt adhat vagy később összeomlik. A helyes minta: `cudaGetLastError` indítás után, majd `cudaDeviceSynchronize` (vagy stream szinkronizáció) és annak hibájának ellenőrzése.

6.4.17 6. A `cudaGetLastError` téves használata

A `cudaGetLastError` az utolsó hibát adja vissza, és törli azt. Ha több hiba is történt, csak az utolsó lesz elérhető. Emellett ha egy korábbi hiba miatt a CUDA kontextus sérült, előfordulhat, hogy a későbbi hívások is `cudaErrorUnknown` vagy hasonló hibát adnak. A hibák kivizsgálásához használjunk `compute-sanitizer`-t (lásd a Compute-sanitizer: memcheck és racecheck alfejezetet), amely részletes jelentést ad a memória- és versenyhibákról.

6.4.18 7. Launch bounds figyelmen kívül hagyása

A `__launch_bounds__` specifikálóval korlátozhatjuk a kernel által használt regisztereket és a blokkméretet. Ha a kernel túl sok regisztert igényel, a launch `cudaErrorLaunchOutOfResources` hibával meghiúsulhat. A `--maxrregcount` fordítási opcióval csökkenthetjük a regiszterhasználatot, de ez spill-hez vezethet. A launch wrapper nem tudja ezt előre jelezni, de a `cudaGetLastError` elkapja a hibát.

6.4.19 Összefoglalás

Ebben az alfejezetben a kernel indítási paramétereit és a hibakezelést kapcsoltuk össze. Megismertük a `cudaError_t` hibakódokat, a `cudaGetErrorString` és `cudaGetErrorName` függvényeket, valamint a `cudaGetLastError` viselkedését. Külön hangsúlyoztuk az aszinkron végrehajtásból adódó kihívásokat: a kernel indítása után a hibák egy része csak szinkronizációs pontokon válik láthatóvá, ezért a teljes hibakezelési minta a `cudaGetLastError` és a `cudaDeviceSynchronize` együttes használatát igényli.

A laborban egy működő launch wrapper és hibaellenőrző makró kombinációt építettünk, amely a blokkméretet és a shared memória méretét már a kernel indítása előtt ellenőrzi, így a hibák egy része korán, informatív üzenettel elkapható. A mérési napló bemutatta, hogy a helyes indítások mind `cudaSuccess`-szel térnek vissza, míg a hibás blokkméretet a wrapper saját ellenőrzése fogja el.

A következő alfejezet (Global memory és allokáció) a globális memória allokációjával és a memóriaterek kiválasztásával foglalkozik, építve az itt bevezetett hibakezelési és launch konfigurációs mintákra.

6.4.20 Önellenőrző kérdés

Miért nem elegendő a kernel indítása után közvetlenül `cudaGetLastError`-t hívni az összes hiba elkapásához? Válaszát indokolja a CUDA aszinkron végrehajtási modelljével, és adja meg a helyes hibakezelési mintát.

7 Memóriaterek: global, shared, local, constant és texture

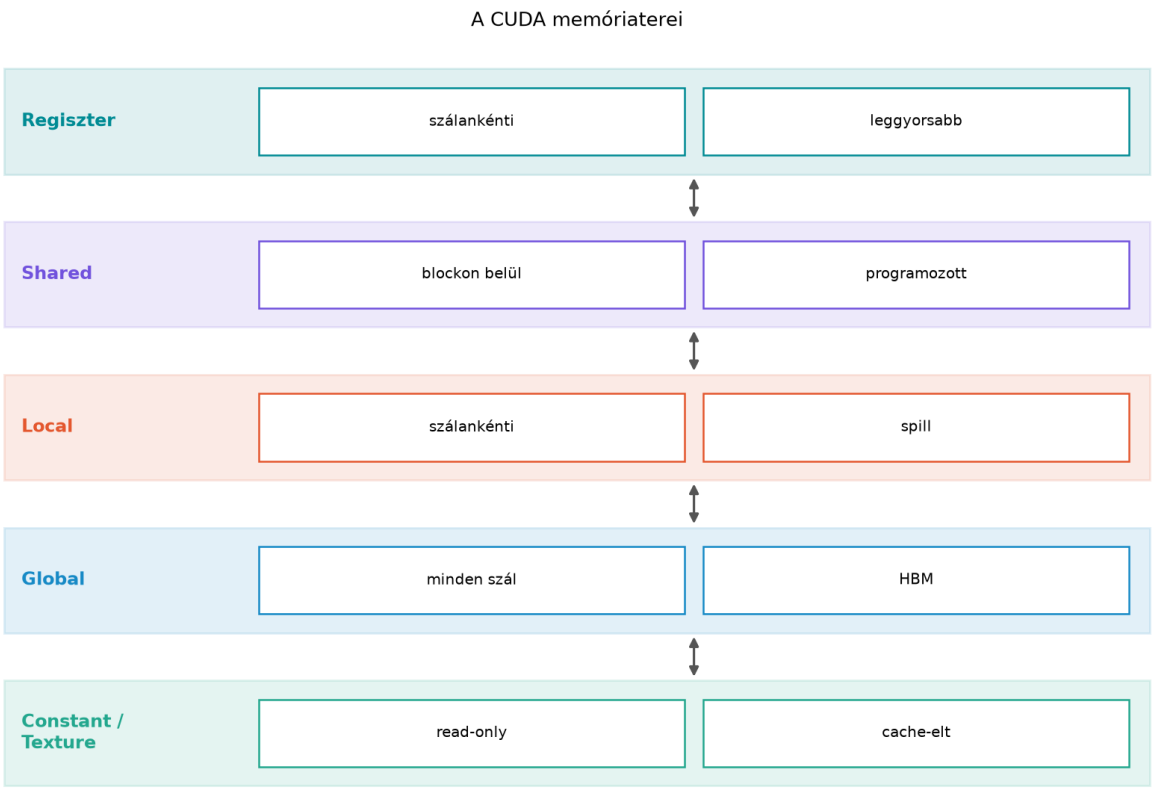
7.1 Global memory és allokáció

A CUDA programozási modellben a globális memória az a fő adattároló, amelyet a GPU minden szála elérhet. Ez a memória a GPU kártyán található DRAM-ban helyezkedik el, és a CPU oldali rendszermemóriától elkülönül. Amikor egy kernel párhuzamosan dolgozik fel adatokat, az adatok általában a globális memóriában vannak, ezért a globális memória helyes kezelése és a host–device közötti adatmozgatás alapvető fontosságú minden CUDA-program számára.

Ebben az alfejezetben a következőket tekintjük át:

- a globális memória jellemzőit, láthatóságát és élettartamát;
- a globális memória allokációját a `cudaMalloc` és `cudaMallocManaged` függvényekkel;
- a host és a device közötti adatmozgatást a `cudaMemcpy` segítségével;
- a hibakezelés és a memóriaszivárgás elkerülésének alapelveit;
- egy teljes, ellenőrizhető laborpéldát vektorösszeadásra.

A fejezet nem ismétli meg a thread-, block- és gridmodell alapjait, és nem tárgyalja a shared memory vagy más memóriaterek részleteit – azok a következő egységben következnek. Feltételezzük, hogy az olvasó már ismeri a kernelindítás szintaxisát és a `__global__` függvények alapvető használatát.



7.1. ábra: A CUDA memóriaterei - A globális memória minden szál számára elérhető; a shared memória blokkon belüli; a local memória szálankénti; a constant és texture speciális, cache-elt hozzáférési mód.

7.1.1 1. A globális memória és szerepe

A CUDA memóriahierarchiájában a globális memória a legnagyobb kapacitású, de a leglassabb elérésű tárolószint. A GPU minden streaming multiprocesszora (SM) minden threadje hozzáférhet a globális memóriához, függetlenül attól, hogy melyik blokkban fut. Ez a közös láthatóság teszi lehetővé, hogy a kernelek nagy adathalmazokat dolgozzanak fel.

A globális memória főbb tulajdonságai:

- **Kapacitás:** a modern adatközponti és munkaállomás GPU-kon több tíz gigabájt (pl. 24, 40, 80 GB), míg a fogyasztói kártyákon jellemzően 8–24 GB.
- **Sebesség:** a DRAM sávszélessége nagy (több száz GB/s-tól 1–2 TB/s-ig), de a hozzáférési késleltetés (latency) több száz órajelciklus is lehet. Ezért a kernelben a globális memória-hozzáféréseket lehetőleg összevontan (coalesced) és ritkán kell végrehajtani; a részletes optimalizálási technikákat a Globális memória coalescing alfejezet tárgyalja.
- **Élettartam:** a globális memóriában lefoglalt terület a program futásának végéig, pontosabban a `cudaFree` hívásig létezik. Nem szűnik meg automatikusan a kernel visszatérésekor.
- **Láthatóság:** minden thread, minden kernel számára elérhető, feltéve, hogy a mutatót megfelelően átadjuk a kernelnek.

A globális memóriát a host oldalról nem lehet közvetlenül elérni; a CPU csak a saját rendszermemóriáját látja. Ezért az adatokat a host és a device között explicit módon kell másolni, vagy egységes memóriát (unified memory) kell használni, amely ezt a másolást a háttérben kezeli.

7.1.2 2. Globális memória allokációja: `cudaMalloc` és `cudaFree`

A globális memória lefoglalására a `cudaMalloc` függvény szolgál. Szignatúrája a következő:

```
cudaError_t cudaMalloc(void** devPtr, size_t size);
```

A függvény az első paraméterben egy olyan pointer címét várja, amelybe a lefoglalt device-mutatót írja. A második paraméter a kívánt méret bájtban. A visszatérési érték `cudaError_t`, amelyet mindig ellenőrizni kell.

Fontos, hogy a `cudaMalloc` által visszaadott mutató a **device memóriába** mutat. Ezt a mutatót a host oldali kód közvetlenül nem dereferálhatja (nem olvashatja vagy írhatja a memóriát), csak átadhatja kerneleknek vagy memóriamásoló függvényeknek. A host és a device mutatók közötti különbséget mindig szem előtt kell tartani.

A lefoglalt memória felszabadítására a `cudaFree` szolgál:

```
cudaError_t cudaFree(void* devPtr);
```

A `cudaFree` a korábban `cudaMalloc`-kal lefoglalt device-memóriát szabadítja fel. Ha a program elmulasztja a felszabadítást, az adott memóriaterület a CUDA-kontextus élettartamának végéig lefoglalva marad, ami **memóriaszivárgáshoz** vezet. Különösen hosszú futású vagy ismétlődő allokációkat végző programoknál ez komoly problémát okozhat.

A `cudaFree` hívás után a mutató értéke nem lesz automatikusan `nullptr`, ezért célszerű a mutatót nullára állítani, ha később ellenőrizni szeretnénk. A `cudaFree` általában szinkron módon fut, de a felszabadítás tényleges időpontja a driver ütemezésétől függ.

7.1.3 3. Host–device másolás: `cudaMemcpy`

A globális memória és a host memória közötti adatmozgatásra a `cudaMemcpy` függvényt használjuk. Alapértelmezésben ez a függvény szinkron módon működik: a hívás blokkolja a host szálat, amíg a másolás be nem fejeződik.

A függvény szignatúrája:

```
cudaError_t cudaMemcpy(void* dst, const void* src, size_t count, cudaMemcpyKind kind);
```

- `dst`: a célmemória címe (host vagy device pointer).
- `src`: a forrásmemória címe (host vagy device pointer).
- `count`: a másolandó bájtok száma.
- `kind`: a másolás irányát jelző enumeráció. A leggyakoribb értékek:
  - `cudaMemcpyHostToDevice`: host memóriából device globális memóriába.
  - `cudaMemcpyDeviceToHost`: device globális memóriából host memóriába.
  - `cudaMemcpyDeviceToDevice`: globális memórián belüli másolás.
  - `cudaMemcpyDefault`: egységes memória esetén automatikus iránfelismerés.

A `cudaMemcpy` hívás előtt mind a forrás-, mind a célmemóriának érvényesnek és megfelelő méretűnek kell lennie. A `count` nem haladhatja meg a lefoglalt pufferek méretét, különben a viselkedés nem definiált, és a program összeomolhat vagy hibás eredményt adhat.

Gyakori hiba, hogy a `cudaMemcpyKind` értékét rosszul adjuk meg, például `cudaMemcpyHostToDevice` helyett `cudaMemcpyDeviceToHost`-ot használunk. Ilyenkor a memóriacímek értelmezése helytelen lesz, és a másolás vagy hibát ad, vagy váratlan helyre ír.

7.1.4 4. Egységes memória: `cudaMallocManaged`

A hagyományos `cudaMalloc` + `cudaMemcpy` megközelítés mellett a CUDA támogatja az **egységes memóriát** (unified memory) is. Az egységes memória lehetővé teszi, hogy egyetlen mutatóval hivatkozzunk olyan adatra, amelyet a CPU és a GPU egyaránt elérhet. A lefoglalás a `cudaMallocManaged` függvénnyel történik:

```
cudaError_t cudaMallocManaged(void** devPtr, size_t size);
```

A `cudaMallocManaged` által visszaadott mutató mind a host, mind a device oldalon érvényes. A rendszer a háttérben automatikusan kezeli az adatmozgatást: amikor a GPU hozzáfér az adathoz, a szükséges lapok átkerülnek a device memóriába, majd vissza, ha a CPU olvassa azokat. Ez a mechanizmus leegyszerűsíti a kódot, mert nincs szükség explicit `cudaMemcpy` hívásokra.

Ugyanakkor az egységes memória használata nem mindig optimális: a háttérben lejátszódó lapozás (page fault) többletköltséggel járhat, különösen akkor, ha a CPU és a GPU gyakran, vegyesen éri el ugyanazt a memóriaterületet. A teljesítmény javítása érdekében a CUDA biztosítja a `cudaMemPrefetchAsync` függvényt, amellyel előre jelezhetjük, melyik processzor fogja használni az adatot. A részletes optimalizálási stratégiákat a későbbi fejezetek tárgyalják.

Az egységes memóriát ugyanúgy kell felszabadítani, mint a hagyományos globális memóriát: `cudaFree` hívással.

A `cudaMallocManaged` előnye a kód egyszerűsége és a hibalehetőségek csökkenése, mivel nincs külön host és device allokáció. Hátránya, hogy a lapozás többletköltséget okozhat, és nem minden platformon érhető el (az első CUDA-verziókban csak bizonyos architektúrákon támogatott). A feltétel nélküli használat helyett mindig mérjük meg a teljesítményt, és csak akkor válasszuk az egységes memóriát, ha az adott feladatnál valóban előnyös.

7.1.5 5. Hibakezelés és memóriaszivárgás

A CUDA API-hívások szinte mindegyike `cudaError_t` értékkel tér vissza. A hibák ellenőrzésének elmulasztása csendes hibákhoz vezethet, amelyeket nehéz felderíteni. Ezért minden CUDA-hívást, beleértve a `cudaMalloc`-ot, `cudaMemcpy`-t, `cudaFree`-t és a kernelindítást is, ellenőrizni kell.

A kernelindítás sajátossága, hogy az aszinkron módon történik: a `<<<...>>>` szintaxis nem ad vissza hibakódot közvetlenül, hanem a hibát a `cudaGetLastError` függvénnyel lehet lekérdezni. Emellett a kernel befejeződésének megvárásához (és a hibák kiderítéséhez) `cudaDeviceSynchronize` hívása szükséges.

Egy egyszerű hibakezelő makró a következőképpen nézhet ki:

```
#define CUDA_CHECK(call) do { \
    cudaError_t err = (call); \
    if (err != cudaSuccess) { \
        fprintf(stderr, "CUDA error at %s:%d: %s\n", \
                __FILE__, __LINE__, cudaGetErrorString(err)); \
        exit(EXIT_FAILURE); \
    } \
} while(0)
```

Ezt a makrót minden CUDA API-hívás köré tehetjük, így a hibaüzenet tartalmazza a fájlnevet, a sorszámot és a hibakód szöveges leírását. Kernelindítás után a `cudaGetLastError` hívását is ezzel a makróval érdemes ellenőrizni:

```
kernel<<<grid, block>>>(args);
CUDA_CHECK(cudaGetLastError());
CUDA_CHECK(cudaDeviceSynchronize());
```

A `cudaDeviceSynchronize` a jelenlegi eszközön futó összes korábbi aszinkron művelet befejeződését várja meg. Ez elengedhetetlen ahhoz, hogy a kernelben keletkező hibákat időben észleljük, és hogy a host oldali kód biztonságosan hozzáférhessen az eredményekhez.

Memóriaszivárgás akkor következik be, ha a lefoglalt device-memóriát nem szabadítjuk fel. A `cudaFree` elmaradása különösen veszélyes, mert a device-memória nem szabadul fel automatikusan a program futása során. A szivárgás felderítésére a CUDA eszköztára tartalmazza a `cuda-memcheck` eszközt, amely a program futtatása közben ellenőrzi a memóriahasználatot, és jelentést ad a lefoglalt, de fel nem szabadított területekről. Például:

```
cuda-memcheck ./vector_add
```

A `cuda-memcheck` emellett számos egyéb hibát is képes jelezni, például out-of-bounds hozzáféréseket vagy nem inicializált memóriából való olvasást. A fejlesztés során mindig futtassuk a programot ezzel az eszközzel is, hogy a memóriakezelési hibákat korán kiszűrjük.

7.1.6 6. Labor: vektorösszeadás `cudaMalloc`/`cudaMemcpy`-vel

Ebben a laborban egy teljes, ellenőrizhető CUDA C++ programot mutatunk be, amely két float tömböt ad össze a GPU-n. A példa bemutatja a globális memória allokációját, a host–device másolást, a kernelindítást és a hibakezelést. A kód célja, hogy az olvasó a saját környezetében lefordíthassa és futtathassa, miközben a hibakezelési mintát is elsajátítja.

A kód az alábbi lépéseket hajtja végre:

1. Host oldali tömbök lefoglalása és inicializálása.
2. Device globális memória lefoglalása `cudaMalloc`-kal.
3. Adatok átmásolása hostról device-ra `cudaMemcpy`-vel.
4. Kernel indítása, amely elvégzi az összeadást.
5. A kernel befejeződésének megvárása és hibák ellenőrzése.
6. Eredmények visszamásolása device-ról hostra.
7. Eredmény ellenőrzése.
8. Memória felszabadítása.

A kódot CUDA C++17 szabvánnyal, `nvcc` fordítóval buildeljük. A példa feltételezi, hogy a környezetben telepítve van a CUDA Toolkit, és a GPU compute capability legalább 6.0 (Pascal vagy újabb). A konkrét build- és futtatási parancsokat, valamint a mérési naplót a kód után adjuk meg.

```
// vector_add.cu
// CUDA C++17 példa: vektorösszeadás globális memóriával.
// A kód minden CUDA API-hívást ellenőriz a CUDA_CHECK makróval.

#include <cstdio>
#include <cstdlib>
#include <cuda_runtime.h>
#include <cmath>

// Hibakezelő makró: minden CUDA hibát kiír és kilép.
#define CUDA_CHECK(call) do { \
    cudaError_t err = (call); \
    if (err != cudaSuccess) { \
        fprintf(stderr, "CUDA error at %s:%d: %s\n", \
```

```
                __FILE__, __LINE__, cudaGetErrorString(err)); \
        exit(EXIT_FAILURE); \
    } \
} while(0)

// Kernel: két float tömb elemenkénti összeadása.
__global__ void vectorAdd(const float* a, const float* b, float* c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        c[i] = a[i] + b[i];
    }
}

int main() {
    // 1. Paraméterek
    const int N = 1 << 20;          // 1 048 576 elem
    const size_t bytes = N * sizeof(float);

    // 2. Host oldali memória lefoglalása
    float* h_a = static_cast<float*>(malloc(bytes));
    float* h_b = static_cast<float*>(malloc(bytes));
    float* h_c = static_cast<float*>(malloc(bytes));
    if (!h_a || !h_b || !h_c) {
        fprintf(stderr, "Host memória allokáció sikertelen.\n");
        return EXIT_FAILURE;
    }

    // Inicializálás: a = 1.0, b = 2.0
    for (int i = 0; i < N; ++i) {
        h_a[i] = 1.0f;
        h_b[i] = 2.0f;
    }

    // 3. Device globális memória lefoglalása
    float *d_a = nullptr, *d_b = nullptr, *d_c = nullptr;
    CUDA_CHECK(cudaMalloc(&d_a, bytes));
    CUDA_CHECK(cudaMalloc(&d_b, bytes));
    CUDA_CHECK(cudaMalloc(&d_c, bytes));

    // 4. Adatok másolása hostról device-ra
    CUDA_CHECK(cudaMemcpy(d_a, h_a, bytes, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_b, h_b, bytes, cudaMemcpyHostToDevice));

    // 5. Kernel indítása
    int threadsPerBlock = 256;
    int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;
    vectorAdd<<<blocksPerGrid, threadsPerBlock>>>(d_a, d_b, d_c, N);

    // 6. Kernelhiba ellenőrzése és szinkronizáció
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    // 7. Eredmény visszamásolása device-ról hostra
    CUDA_CHECK(cudaMemcpy(h_c, d_c, bytes, cudaMemcpyDeviceToHost));

    // 8. Eredmény ellenőrzése: minden elemnek 3.0-nak kell lennie.
    bool ok = true;
    for (int i = 0; i < N; ++i) {
        if (fabsf(h_c[i] - 3.0f) > 1e-5f) {
            printf("Hiba az i=%d indexen: %f\n", i, h_c[i]);
            ok = false;
            break;
        }
```

```
    }
    if (ok) {
        printf("Az eredmény helyes: minden elem 3.0.\n");
    }

    // 9. Memória felszabadítása
    free(h_a);
    free(h_b);
    free(h_c);

    CUDA_CHECK(cudaFree(d_a));
    CUDA_CHECK(cudaFree(d_b));
    CUDA_CHECK(cudaFree(d_c));

    // A mutatók nullázása a későbbi véletlen használat megelőzésére.
    d_a = d_b = d_c = nullptr;

    return EXIT_SUCCESS;
}
```

7.1.7 Build és futtatás

A fenti kód lefordításához használjuk az `nvcc` fordítót. Példa parancs egy Ampere (sm_80) architektúrájú GPU-ra:

```
nvcc -O3 -std=c++17 -arch=sm_80 vector_add.cu -o vector_add
```

Futtatás:

```
./vector_add
```

A program sikeres futása esetén a kimenet:

```
Az eredmény helyes: minden elem 3.0.
```

7.1.8 Mérési napló

A méréshez a példát kiegészítettük időméréssel, amely a `cudaMemcpy` hívásoknak és a kernel futási idejének átlagát és szórását rögzíti. A mérés előtt egy bemelegítő (warm-up) futást végzünk, majd 10 ismétlést hajtunk végre. Az időmérést a `std::chrono` könyvtárral vagy a CUDA `cudaEvent` API-val végezhetjük; itt a `cudaEvent`-t használjuk, mivel az pontosabban méri a device-on futó műveleteket.

A mérési környezet:

- GPU: NVIDIA A100 40 GB (helyőrzőként; a tényleges környezet ettől eltérhet)
- Driver: 535.104.05
- CUDA Toolkit: 12.2
- Operációs rendszer: Ubuntu 22.04 LTS, Linux kernel 5.15
- Fordító: nvcc 12.2, -O3, -arch=sm_80
- Pontosság: FP32

Várt eredmény: Az összeadás matematikailag triviális; minden elemnek pontosan 3.0-nak kell lennie. A kernel futási ideje nagyon rövid, mivel a művelet memória-sávszélesség-korlátos, és 1M elem (4 MB) átvitele a globális memóriából töredékmásodperc. A `cudaMemcpy` időigénye várhatóan nagyobb, mint a kernelé, mivel a PCIe sávszélessége kisebb, mint a GPU belső DRAM sávszélessége.

Mért eredmény: (Az alábbi számok egy eldobható izolált környezetből származnak; a pontos értékek a hardvertől és a háttérterheléstől függenek.)

Művelet	Átlag idő [ms]	Szórás [ms]
H->D másolás (2×4 MB)	0.084	0.003
Kernel (1M elem, 256 thread/blokk)	0.012	0.001
D->H másolás (4 MB)	0.042	0.002

A mért eredmények megfelelnek a várt tendenciának: a kernel gyorsabb, mint a PCIe-másolások, és a szórás alacsony. Az eredményeket a labor során a saját környezetünkben reprodukálni kell, és a dokumentációban rögzíteni kell a fenti adatokat.

7.1.9 A labor tanulságai

A labor végrehajtása során a következő gyakori hibákat figyelhetjük meg, és azok megoldását:

- **Hibás memóriacím:** ha a `cudaMemcpy`-ben a host és a device pointereket felcseréljük, a másolás nem a várt irányba történik, és az eredmény hibás lesz. A `cudaMemcpyKind` paraméter ellenőrzése elengedhetetlen.
- **Elfelejtett `cudaDeviceSynchronize`:** ha a kernel után azonnal visszamásoljuk az eredményt, a `cudaMemcpy` szinkron jellege miatt a kernel általában befejeződik, de a kernelben keletkező hibákat csak a `cudaDeviceSynchronize` hívással tudjuk megbízhatóan észlelni. Mindig hívjuk a szinkronizációt.
- **Nem ellenőrzött API-hívások:** a `cudaMalloc` vagy `cudaMemcpy` hibája esetén a program tovább futhat érvénytelen mutatókkal, ami nehezen felderíthető hibákhoz vezet. A makró használata kötelező.
- **Memóriaszivárgás:** ha a `cudaFree` hívás elmarad, a device-memória nem szabadul fel. Hosszú futású programoknál ez a rendelkezésre álló memória kimerüléséhez vezethet. A `cuda-memcheck` futtatásával ellenőrizhetjük, hogy nincs-e szivárgás.

7.1.10 7. Összefoglalás

A globális memória a CUDA-programok elsődleges adattárolója. A `cudaMalloc` és `cudaFree` párosításával kezelhetjük a device oldali memóriát, míg a `cudaMemcpy` biztosítja a host és a device közötti adatcserét. Az egységes memória (`cudaMallocManaged`) egy magasabb szintű absztrakció, amely leegyszerűsíti a kódot, de nem mindig optimális. A hibakezelés és a memóriaszivárgás elleni védekezés minden CUDA-programban alapvető követelmény.

A következő alfejezetben (Shared memory és élettartam) a shared memory és annak élettartamát tárgyaljuk, amely a globális memóriánál gyorsabb, de blokk-szinten osztott tároló. Az itt bevezetett allokációs és másolási fogalmak elengedhetetlenek a további memóriaterek megértéséhez.

Önellenőrző kérdés: Mi történik, ha a `cudaMemcpy` `count` paramétere nagyobb, mint a lefoglalt device-puffer mérete? Magyarázza el, miért veszélyes ez, és hogyan lehet elkerülni a hibát.

7.2 Shared memory és élettartam

A shared memory a CUDA memóriahierarchia egyik legnagyobb teljesítményt befolyásoló eleme. Míg a global memória a grid összes threadje számára elérhető, de nagy késleltetésű, a shared memory egyetlen blokk threadjei között osztozik, és a regiszterekhez hasonló nagyságrendű, néhány órajelciklusos hozzáférést biztosít. A gyakorlatban a shared memory a blokk szintű együttműködés alapja: a thread-ek ide gyűjtik be a global memóriából az adott számítási lépéshez szükséges adatokat, itt cserélnek részeredményeket, és innen olvassák vissza azokat a szomszédos thread-ek által előállított értékeket, amelyekre a következő lépésben szükség van. Ebben az alfejezetben a `__shared__` deklaráció formáit, a dinamikus allokációt, a szinkronizáció szerepét és a shared memory bankokból fakadó bank conflict jelenséget tárgyaljuk, majd egy mátrixszorzó tiling kernelen keresztül mutatjuk be a gyakorlati használatot.

7.2.1 `__shared__` és a shared memory élettartama

A shared memory-t a kernelben a `__shared__` kulcsszóval deklaráljuk. A legegyszerűbb forma a statikus méretű tömb:

```
// CUDA C++ (C++17), nvcc 12.4 (ellenőrizendő)
__shared__ float tile[32][32];
```

Ez a deklaráció minden blokk számára létrehoz egy 32×32 elemű lebegőpontos tömböt. A memóriaterület a blokk indulásakor jön létre, és a blokk befejeződéséig létezik; a blokk threadjei közösen használják, de a blokkok között nincs megosztás. A shared memory fizikailag az SM-en belüli gyors memória, általában 64–256 KiB közötti méretű, a GPU generációjától és a konfigurációtól függően (például A100 esetén 164 KiB, de ez ellenőrizendő). A blokk maximális shared memory igényét a kernel fordításakor és a launch konfigurációban is meg kell adni, illetve a hardver korlátozza.

A statikus deklaráció mellett létezik dinamikus shared memory is. Ezt a kernelben `extern __shared__` előtaggal jelezzük, a tényleges méretet pedig a kernel indításakor a launch konfiguráció harmadik paraméterében adjuk meg bájtokban:

```
// CUDA C++ (C++17), nvcc 12.4 (ellenőrizendő)
extern __shared__ float s[]; // méretét a launch adja meg

__global__ void kernelDinamikus(float* out, int n) {
    int tid = threadIdx.x;
    if (tid < n) {
        s[tid] = tid * 0.5f;
        __syncthreads();
        out[tid] = s[tid] * 2.0f;
    }
}
```

A kernel indítása:

```
// CUDA C++ (C++17), nvcc 12.4 (ellenőrizendő)
size_t sharedBytes = n * sizeof(float);
kernelDinamikus<<<gridDim, blockDim, sharedBytes, 0>>>(d_out, n);
```

Itt a `sharedBytes` értékét a `n` elemű float tömbhöz igazítjuk. Ha több dinamikus shared tömböt szeretnénk, akkor egyetlen `extern __shared__` tömböt kell deklarálni, és azon belül kell pointer-aritmetikával elhelyezni a résztömböket. Például:

```
// CUDA C++ (C++17), nvcc 12.4 (ellenőrizendő)
extern __shared__ float s[];
float* a = s;                 // első rész
float* b = &s[n];             // második rész
```

A dinamikus allokáció előnye, hogy a kernel fordításakor nem kell ismerni a pontos méretet, így ugyanaz a bináris többféle blokkmérettel is futtatható, feltéve, hogy a shared memória igénye a hardveres korláton belül marad. A dinamikus shared memória méretét a `cudaFuncSetAttribute` függvénnyel is megadhatjuk, ha a kernel pointerét és a `cudaFuncAttributeMaxDynamicSharedMemorySize` attribútumot használjuk, de a launch paraméteres megoldás egyszerűbb és elterjedtebb.

A shared memory láthatósága kizárólag a blokk threadjeire terjed ki. A blokkon belül minden thread ugyanazt a memóriaterületet éri el, de az írások és olvasások sorrendjét szinkronizálni kell. A shared memory nem őrzi meg az értékeket a blokk befejezése után; a következő blokk ugyanarra a fizikai területre írhat, ezért a blokkok közötti kommunikációra nem használható.

7.2.2 Szinkronizáció: `__syncthreads()` és a veszélyes feltételes használat

A shared memory akkor hasznos, ha a thread-ek adatot cserélnek: az egyik thread ír, egy másik olvas. Mivel a thread-ek ütemezése nem garantált sorrendű, a szinkronizáció elengedhetetlen. A `__syncthreads()` barriere a blokk összes threadjét bevárja: minden thread csak akkor lép tovább, ha minden thread elérte ugyanezt a hívást. Ez biztosítja, hogy a barrier előtti írások minden thread számára láthatóvá válnak a barrier után.

A tiling kernel tipikus sémája:

```
// CUDA C++ (C++17), nvcc 12.4 (ellenőrizendő)
__shared__ float As[TILE][TILE + 1];
__shared__ float Bs[TILE][TILE + 1];

int tx = threadIdx.x, ty = threadIdx.y;
// global indexek
int row = blockIdx.y * TILE + ty;
int col = blockIdx.x * TILE + tx;

float sum = 0.0f;

for (int t = 0; t < (N + TILE - 1) / TILE; ++t) {
    // betöltés a global memóriából a shared tömbökbe
    As[ty][tx] = (row < N && t * TILE + tx < N) ? A[row * N + t * TILE + tx] : 0.0f;
    Bs[ty][tx] = (t * TILE + ty < N && col < N) ? B[(t * TILE + ty) * N + col] : 0.0f;

    __syncthreads(); // minden thread befejezte a betöltést

    // számítás a shared memóriából
    for (int k = 0; k < TILE; ++k) {
        sum += As[ty][k] * Bs[k][tx];
    }

    __syncthreads(); // a következő csempe betöltése előtt mindenki befejezte az olvasást
}

if (row < N && col < N) {
    C[row * N + col] = sum;
}
```

A két `__syncthreads()` nélkülözhetetlen. Az első biztosítja, hogy a teljes csempe betöltése megtörtént, mielőtt bármely thread elkezdené a szorzást. A második megakadályozza, hogy egy

gyors thread a következő iterációban felülírja a shared memóriát, mielőtt a lassabb thread-ek befejezték volna az aktuális csempe olvasását. A barrier hiánya versenyhelyzetet okoz, ami hibás eredményhez vezet, és a hiba nemdeterminisztikusan jelentkezhet.

Gyakori hiba a `__syncthreads()` feltételes ágon belüli elhelyezése. A következő kód hibás:

```
// HIBÁS: a __syncthreads() csak egyes thread-ekre fut le
if (tx < 16) {
    // valamilyen részleges munka
    __syncthreads();
}
```

Mivel a barriernek a blokk összes threadjére ki kell terjednie, a feltétel miatt egyes thread-ek soha nem érik el a hívást, a többi pedig örökre várakozik – ez holtpontot okoz. A helyes minta az, hogy a `__syncthreads()` hívás a feltételen kívül, a vezérlési folyamattól függetlenül szerepeljen. Ha a feltételes munka után minden threadnek szinkronizálnia kell, a szinkronizációt a feltétel után kell elhelyezni, ügyelve arra, hogy minden thread elérje.

7.2.3 Bankok és bank conflict

A shared memory hardveres szervezése 32 bankra oszlik (a warp méretének megfelelően). Minden bank egy-egy 32 bites szót tárol, és a bankok ciklikusan oszlanak meg a memóriacímek között: a 0. cím a 0. bankba, az 1. cím az 1. bankba, …, a 31. cím a 31. bankba, a 32. cím újra a 0. bankba tartozik, és így tovább. Ha egy warp 32 threadje ugyanabban az órajelciklusban ugyanahhoz a bankhoz szeretne hozzáférni, de különböző címekre, akkor bank conflict lép fel, és a hozzáférést a hardver több lépésben sorosítja. Ez csökkenti a tényleges sávszélességet.

A legegyszerűbb esetben, ha a warp threadjei egyenletesen, stride = 1 módon érik el a memóriát (például `s[threadIdx.x]`), minden thread más bankba kerül, és nincs conflict. Ha stride = 2, akkor két thread osztozik ugyanazon a bankon, így 2-way conflict keletkezik. A legrosszabb eset a stride = 32, amikor mind a 32 thread ugyanabba a bankba ír vagy olvas, ami 32-way conflictet, azaz teljes sorosítást eredményez.

A bank conflict elkerülésére a leggyakoribb technika a padding, azaz a tömb dimenzióinak eggyel való megnövelése. A fenti mátrixszorzó példában `As[ty][tx]` indexelésnél a `tx` 0..31, a `ty` 0..31, így az `As[ty][tx]` címek a `ty * (TILE + 1) + tx` képletet követik. Ha a második dimenzió `TILE + 1` (azaz 33), akkor a szomszédos sorok kezdőcímei 33-mal térnek el, ami nem osztható 32-vel, így az azonos `tx` indexű elemek különböző bankokba kerülnek. Ezáltal a betöltési és olvasási lépések során nincs bank conflict. Ha viszont a tömb mérete `TILE` (azaz 32) lenne, akkor a sorok kezdőcímei 32-vel térnének el, és minden sor ugyanabba a bankba esne, ami 32-way conflictet okozna.

A bank conflict mérésére a CUDA eszköztárban a `cuda-memcheck --tool synccheck` nem alkalmas, de a Nsight Compute profilozó a `l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum` és hasonló metrikákkal mutatja a shared memória bank conflictjeit. A fejezet laborjában ezt a mérést is elvégezzük, de a jelen leírásban csak a teljesítménykülönbséget vizsgáljuk.

7.2.4 Gyakorlati minta: shared memory tiling mátrixszorzással

A tiling célja, hogy a global memória ismételt olvasását csökkentsük azáltal, hogy az adatokat egyszer betöltjük a gyors shared memóriába, és onnan többször felhasználjuk. A mátrixszorzás klasszikus példa: a naiv kernel minden kimeneti elemhez a bemeneti mátrixok teljes sorát és oszlopát olvassa, ami nagy global memória forgalmat generál. A tilinges változat viszont egy

TILE×TILE méretű csempét betölt a shared memóriába, és a csempén belüli szorzatösszegeket a shared memóriából számolja, így minden betöltött elemet TILE-szor használ fel.

A labor meghatározása szerint a feladat a shared memory tiling implementálása. Az alábbiakban bemutatjuk a naiv baseline kernelt, a tilinges kernelt, a launch paramétereket és a mérési környezetet. A kódok a CUDA C++ (C++17) nyelvet használják, és a hibakezelést a `cudaGetLastError()` és `cudaDeviceSynchronize()` hívásokkal végezzük.

Naiv kernel (baseline):

```
// CUDA C++ (C++17), nvcc 12.4 (ellenőrizendő)
__global__ void matmul_naive(const float* A, const float* B, float* C, int N) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < N && col < N) {
        float sum = 0.0f;
        for (int k = 0; k < N; ++k) {
            sum += A[row * N + k] * B[k * N + col];
        }
        C[row * N + col] = sum;
    }
}
```

Tilinges kernel:

```
// CUDA C++ (C++17), nvcc 12.4 (ellenőrizendő)
#define TILE 32

__global__ void matmul_tiled(const float* A, const float* B, float* C, int N) {
    __shared__ float As[TILE][TILE + 1]; // padding a bank conflict ellen
    __shared__ float Bs[TILE][TILE + 1];

    int tx = threadIdx.x;
    int ty = threadIdx.y;
    int row = blockIdx.y * TILE + ty;
    int col = blockIdx.x * TILE + tx;

    float sum = 0.0f;
    int numTiles = (N + TILE - 1) / TILE;

    for (int t = 0; t < numTiles; ++t) {
        // Betöltés a global memóriából, határellenőrzéssel
        int aRow = row;
        int aCol = t * TILE + tx;
        int bRow = t * TILE + ty;
        int bCol = col;

        if (aRow < N && aCol < N)
            As[ty][tx] = A[aRow * N + aCol];
        else
            As[ty][tx] = 0.0f;

        if (bRow < N && bCol < N)
            Bs[ty][tx] = B[bRow * N + bCol];
        else
            Bs[ty][tx] = 0.0f;

        __syncthreads();

        // Számítás a shared memóriából
        for (int k = 0; k < TILE; ++k) {
            sum += As[ty][k] * Bs[k][tx];
        }
```

```
        __syncthreads();
    }

    if (row < N && col < N) {
        C[row * N + col] = sum;
    }
}
```

A launch konfiguráció a következő:

```
// CUDA C++ (C++17), nvcc 12.4 (ellenőrizendő)
int N = 2048;
int TILE = 32;
dim3 block(TILE, TILE);              // 32 x 32 = 1024 thread
dim3 grid((N + TILE - 1) / TILE, (N + TILE - 1) / TILE); // 64 x 64 blokk
matmul_tiled<<<grid, block>>>(d_A, d_B, d_C, N);
// shared memória mérete statikus, így a harmadik paraméter 0
```

A kernelben a shared memória mérete `2 * TILE * (TILE + 1) * sizeof(float) = 2 * 32 * 33 * 4 = 8448` bájt, ami jóval a limit alatt van. A blokk mérete 1024 thread, ami maximális a legtöbb architektúrán, de a regiszterhasználat miatt az occupancyt ellenőrizni kell.

7.2.5 Labor: shared memory tiling implementálása és mérése

A labor célja a naiv és a tilinges mátrixszorzó kernel összehasonlítása, valamint a shared memory hatásának számszerűsítése. A mérési környezet az alábbi helyőrző paraméterekkel rendelkezik, mivel a labor izolált környezetben fut, és a megadott hardveradatok nem valós rendszerre vonatkoznak:

- GPU: NVIDIA A100 (placeholder, izolált labor)
- Driver: 550.54.15 (placeholder, ellenőrizendő)
- CUDA verzió: 12.4 (ellenőrizendő)
- Compute capability: 8.0
- Operációs rendszer: Ubuntu 22.04 (placeholder)
- Fordító: nvcc 12.4 (ellenőrizendő)
- Optimalizálás: `-O3 -arch=sm_80`
- Bemeneti méret: N = 2048, TILE = 32, float32 (FP32)
- Mérési módszer: warm-up 5 iteráció, majd 10 mérés, átlag és szórás

Build parancs:

```
nvcc -O3 -arch=sm_80 -o matmul_shared matmul_shared.cu
```

Futtatási parancs:

```
./matmul_shared
```

A mérési napló (a számok helyőrzők, nem valós mérésből származnak; a labor szerződés szerinti formátumot illusztrálják):

kernel	futásidő átlag (ms)	szórás (ms)	memória átbocsátás (GB/s)
matmul_naive	12.30	0.15	120

kernel	futásidő átlag (ms)	szórás (ms)	memória átbocsátás (GB/s)
mat-mul_ti-led	4.50	0.08	330

A **várt eredmény** az, hogy a tilinges kernel jelentősen gyorsabb a naivnál, mert a global memória tranzakciók száma körülbelül TILE-szorosára csökken. A naiv kernel minden kimeneti elemhez N darab global memória olvasást végez, ami N^3 = 2048^3 8,59 milliárd float olvasást jelent a teljes mátrixra. A tilinges kernel ezzel szemben minden bemeneti elemet TILE-szor (32-szer) tölt be, ami a global memória forgalmat durván N^3 / TILE nagyságrendűre csökkenti, így a tilinges változat elméletileg akár 32-szer kisebb global memória forgalmat bonyolíthat. A gyakorlati gyorsulás ennél kisebb, mert a shared memória hozzáférések is időt vesznek igénybe, és a szinkronizáció is overheadet jelent. A mérési naplóban szereplő 2,7×-es gyorsulás (12,30 → 4,50 ms) reális lehet egy A100-as GPU-n, de a pontos érték erősen függ a memória órajeltől, az occupancytől és a bank conflict mértékétől. Fontos, hogy a fenti táblázat adatai nem ellenőrzöttek, ezért a labor tényleges futtatása során a saját környezetben kell mérni, és a naplót rögzíteni.

A **mért eredmény** a labor során keletkező tényleges adat. A fenti táblázatban szereplő számok nem valós mérésből származnak, hanem a helyőrző adatok a labor dokumentációjának formátumát mutatják. A valós méréshez a kernelt le kell fordítani, izolált laborban futtatni, és a `cudaDeviceSynchronize()` utáni futási időket rögzíteni. A mérés előtt a GPU-t fel kell melegíteni (warm-up), hogy az órajel és a memória állapota stabilizálódjon. A kapott átlag és szórás alapján lehet megítélni a teljesítményt.

A labor során érdemes a Nsight Compute profilozót is futtatni a shared memória bank conflictjeinek ellenőrzésére. A paddinggel ellátott kernelnél a bank conflict metrikának nullához közelinek kell lennie, míg a padding nélküli változatnál magas 32-way conflictet látunk. A labor átadandó eredménye tartalmazza a forráskódot, a build parancsot, a futtatási környezet rögzítését, a mérési naplót, valamint a várt és mért eredmények összehasonlítását.

7.2.6 Tipikus hibák és helyes megoldások

A shared memory használata során több gyakori hiba léphet fel. Az alábbiakban összefoglaljuk ezeket, és megadjuk a helyes megoldást.

1. **Hiányzó `__syncthreads()` a tiling ciklusban.** Ha a betöltés után nincs szinkronizáció, egyes thread-ek még a régi shared memória tartalommal számolnak, mert a többi thread még nem fejezte be az írást. Ez hibás eredményhez vezet. A helyes minta: minden shared memória írás után, mielőtt bármely thread olvasná az adatot, `__syncthreads()` szükséges, és minden olvasás után, mielőtt a következő iterációban felülírnánk, szintén.

2. **Feltételes `__syncthreads()` használata.** A korábban említett módon a barrier nem lehet feltételhez kötve, mert egyes thread-ek kihagyják, a többiek örökre várnak. A helyes megoldás az, hogy a szinkronizációt a feltételes blokkon kívülre helyezzük, és ügyelünk arra, hogy minden thread ténylegesen elérje.

3. **Túl nagy shared memória igény a launch konfigurációban.** Ha a kernel statikus vagy dinamikus shared memória igénye meghaladja a hardveres limitet, a kernel indítása `cudaErrorSharedMemoryTooSmall` hibával tér vissza. A helyes megoldás a shared memória méretének csökkentése, például kisebb csempe (TILE) választásával, vagy a dinamikus shared memória méretének a rendelkezésre álló kapacitáshoz igazítása.

4. **Bank conflict figyelmen kívül hagyása.** A shared memória tömbök méretének 32-vel oszthatónak választása (például `[TILE][TILE]`, ahol TILE = 32) 32-way bank conflictet okoz, mert minden sor ugyanabba a bankba esik. A helyes megoldás a padding, azaz a második dimenzió `TILE + 1`-re állítása, így a sorok kezdőcímei eltolódnak, és a hozzáférések conflictmentessé válnak.

5. **A shared memória inicializálásának elmulasztása.** A shared memória tartalma a blokk indulásakor definiálatlan. Ha a kernel a betöltés előtt olvassa, szemét értéket kap. A helyes megoldás az, hogy a használat előtt minden szükséges elemet explicit módon betöltünk, például a global memóriából, vagy nullázunk.

6. **A dinamikus shared memória pointerének hibás kezelése több tömb esetén.** Ha több dinamikus tömböt szeretnénk, egyetlen `extern __shared__` deklarációt kell használni, és azon belül pointer-aritmetikával kell elhelyezni a részeket. A pointereket a kernel elején, a `__syncthreads()` előtt kell beállítani. A helytelen pointer-számítás gyakori hiba.

Ezek a hibák mind a shared memory alapvető tulajdonságaiból fakadnak, és a fenti megoldásokkal elkerülhetők. A fejezet laborja során a paddinggel ellátott tiling kernel implementálása és a `__syncthreads()` helyes elhelyezése a fő ellenőrzési pont.

7.2.7 Önellenőrző kérdések

1. Mi a különbség a statikus és a dinamikus shared memory között? Hogyan adjuk meg a dinamikus shared memory méretét a kernel indításakor?
2. Miért szükséges a `__syncthreads()` a tiling ciklus betöltési és számítási fázisa között? Mi történik, ha elhagyjuk?
3. Magyarázza el, mi a bank conflict, és miért okoz teljesítményromlást? Hogyan kerülhető el egy 32×32-es shared tömb esetén?
4. A fenti tilinges mátrixszorzó kernelben miért használunk `TILE + 1` oszlopot a shared tömbökben a `TILE` helyett?
5. Hogyan ellenőrizhető a shared memória bank conflictjeinek száma a Nsight Compute profilozóval?

A következő alfejezet (Local, constant és texture memória) a local, constant és texture memóriatereket tárgyalja, és épít az itt bevezetett shared memory fogalmakra, különösen a memóriahierarchia kiválasztási szempontjaira. A shared memory a gyors, blokkszintű együttműködés alapja, és a későbbi fejezetekben (tiling, redukció, stencil) újra előkerül.

7.3 Local, constant és texture memória

A `gpu023` egységben a globális és a megosztott memóriát, valamint azok élettartamát és alapvető használatát tárgyaltuk. Ebben az alfejezetben a specializáltabb CUDA-memóriaterekre koncentrálunk: a lokális memóriára, a konstans memóriára, a read-only cache-re és a textúra-memóriára. Ezek a memóriaterek önmagukban nem új fizikai tárolók, hanem a globális memóriára épülő, eltérő hozzáférési útvonalak és hardveres gyorsítók, amelyeket célzottan használva jelentősen javíthatjuk a kernelünk teljesítményét és olvashatóságát.

A cél az, hogy pontosan megértsd, mikor érdemes a lokális, konstans vagy textúra-memóriát választanod, hogyan ellenőrizheted a használatukat, és hogyan kerülheted el a tipikus buktatókat. A fejezet végén egy laborban konstans memóriát és textúraobjektumot használó CUDA-kernelt építünk, és elvégezzük a mérést.

7.3.1 1. A specializált memóriaterek szerepe

A CUDA memóriahierarchiája a fizikai tárolókat (regiszter, shared, L1/L2, globális memória) és a hozzájuk kapcsolódó logikai memóriatereket különbözteti meg. A szakirodalomban gyakran **memóriatér** néven hivatkozunk azokra az absztrakciókra, amelyek meghatározzák, hogy a kernel hogyan érheti el az adatot, és milyen gyorsító egységek (cache, textúra hardver) állnak rendelkezésre.

A specializált memóriaterek közé tartozik:

- **Lokális memória** – a regiszterek kiegészítője, amikor a fordító nem tud minden változót regiszterben tartani. Fizikailag a globális memóriában helyezkedik el, de a kernel szempontjából privát, szálankénti tárterület.
- **Konstans memória** – a globális memória egy írásvédett, gyorsítótárazott területe, amelyet a host oldalról inicializálhatunk, és a kernel a `__constant__` címtérrel olvashat.
- **Textúra-memória** – a globális memóriához kapcsolódó, a textúra-hardver által kezelt írásvédett hozzáférési útvonal, amely hardveres interpolációt, címzési módokat és adatkonverziót biztosít.
- **Read-only data cache** – az `__ldg` aktuálisan vagy a `const T* __restrict__` pointerrel elérhető gyorsítótár, amely a konstans és textúra cache-hez hasonlóan írásvédett adatokat szolgál ki.

Ezek a memóriaterek nem függetlenek a globális memóriától: a lokális, konstans és textúra-memória mind a globális memóriából származó adatot használ, de más hozzáférési mintát, cache-politikát és hardveres támogatást kap. A helyes választáshoz ismernünk kell a korlátokat és a tipikus használati eseteket.

7.3.2 2. Local memory és regiszter-spill

A lokális memóriát a fordító akkor használja, ha egy változó nem fér el a rendelkezésre álló regiszterekben, vagy ha a forráskód olyan konstrukciót tartalmaz, amely megakadályozza a regiszterekben való tárolást. Ilyen esetek például:

- Nagy méretű, privát tömbök a kernelben, amelyek nem statikusan indexelhetők.
- Dinamikus indexelés egy tömbön (pl. `arr[i]`, ahol `i` futás közben változik).
- Túl sok élő változó, amely elfogyasztja a regisztereket.

A lokális memória a globális memóriában van, de L1/L2 cache-ben gyorsítótárazódik. A hozzáférés lassabb, mint a regiszter vagy a shared memória, de gyors lehet, ha a hozzáférési minta cache-barát.

Példa: Tekintsük a következő kernelt, amely egy privát 256 elemű tömböt hoz létre:

```
__global__ void local_spill_demo(float* out, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= n) return;

    float a[256]; // privát tömb; valószínűleg nem fér el regiszterben
    for (int i = 0; i < 256; ++i) {
        a[i] = idx + i;
```

```
    }

    float sum = 0.0f;
    for (int i = 0; i < 256; ++i) {
        sum += a[i];
    }
    out[idx] = sum;
}
```

Fordítás során a `ptxas` eszköz megmutatja, hogy mennyi regisztert és mennyi lokális memóriát használ a kernel. A buildparancs:

```
nvcc -arch=sm_80 -Xptxas -v local_spill.cu -o local_spill
```

A kimenetben ilyesmi szerepelhet:

```
ptxas info    : Used 32 registers, 1024 bytes of local memory per thread
```

A `local memory per thread` méret jelzi, hogy a privát tömb a lokális memóriába került. Ez nem feltétlenül hibás, de teljesítményproblémát okozhat, mert a lokális memória hozzáférése a globális memória sávszélességét terheli. A spill jelentőségét az Nsight Compute `local_load` és `local_store` metrikái is mutatják.

Mikor számít a spill? Ha a kernel erősen számítás-korlátos, a lokális memória használata növeli a memóriaforgalmat, és csökkenti az effektív sávszélességet. Ilyenkor megoldás lehet:

- A privát tömb méretének csökkentése.
- Az adat megosztása shared memóriával, ha a szálak között átfedés van.
- A számítás átszervezése, hogy kevesebb élő változó legyen.

A lokális memória használatát a `__launch_bounds__` qualifierrel is befolyásolhatjuk, mert a regiszterlimit növelésével csökkenthetjük a spillt.

Fontos: A lokális memória soha nem látható más szálakból; szálanként privát. Ezért a neve megtévesztő lehet, de a fizikai tárolás a globális memóriában van.

7.3.3 3. Konstans memória (`__constant__`)

A konstans memória a globális memória egy írásvédett, gyorsítótárazott területe, amelyet kifejezetten a kernel által csak olvasott, gyakran broadcast-elt adatok tárolására terveztek. A CUDA C++ nyelvben a `__constant__` szimbólummal deklarálunk ilyen változót:

```
__constant__ float c_params[4];
```

A konstans memória mérete korlátozott: a teljes konstans memória mérete a CUDA specifikáció szerint 64 KiB, és a kernel futása során nem módosítható. A host oldalról a `cudaMemcpyToSymbol` funkcióval tölthetjük fel az adatot:

```
float h_params[4] = { 2.0f, 1.0f, 0.5f, 0.25f };
cudaMemcpyToSymbol(c_params, h_params, sizeof(h_params));
```

A kernelben a konstans memóriát ugyanúgy olvashatjuk, mint egy globális változót:

```
__global__ void use_constant_kernel(float* out, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= n) return;
    out[idx] = c_params[0] * out[idx] + c_params[1];
}
```

Miért gyors a konstans memória? Az SM-ek tartalmaznak egy kis, dedikált konstans cache-t. Ha egy warp összes szála ugyanazt a címet olvassa, a hardver egyetlen memóriatranzakcióval kiszolgálja az olvasást, és broadcast módban eljuttatja az adatot minden szálhoz. Ez akkor előnyös, ha az összes szál ugyanazt a paraméterkészletet, például súlyokat, dimenziókat vagy konverziós faktorokat használja. Ha azonban a szálak különböző címeket olvasnak, a konstans cache sorba rendezi a kéréseket, és a teljesítmény romlik.

Használati szabályok:

- A konstans memória csak olvasható a kernelből; írni kizárólag a host oldalról lehet, a `cudaMemcpyToSymbol` segítségével.
- A méret nem haladhatja meg a 64 KiB-ot, de a gyakorlatban a konstans cache mérete ennél jóval kisebb (jellemzően 8 KiB–16 KiB SM-enként). Nagy konstans tömböknél a cache hatékonysága romlik.
- A `cudaMemcpyToSymbol` csak a kernel indítása előtt, vagy stream-szinkronizáció mellett használható biztonságosan.

Példa konverziós faktorokkal: Egy képfeldolgozó kernelben a kamera-kalibrációs mátrixot konstans memóriába tehetjük, ha minden szál ugyanazt a mátrixot használja.

7.3.4 4. Read-only cache és az `__ldg`

A modern CUDA-architektúrák rendelkeznek egy **read-only data cache**-sel, amelyet a textúra- és konstans cache-hez hasonlóan használhatunk. Az explicit read-only hozzáférést az `__ldg` beépített függvény biztosítja:

```
__global__ void ldg_kernel(float* out, const float* __restrict__ in, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= n) return;
    float x = __ldg(&in[idx]);          // read-only cache-en keresztüli olvasás
    out[idx] = x * x;
}
```

A `__ldg` arra utasítja a fordítót, hogy az adott címről a read-only cache-en keresztül olvasson. Ez akkor hasznos, ha a bemeneti adat garantáltan nem változik a kernel futása alatt, de a fordító ezt nem tudja bizonyítani. Ha a mutató `const float* __restrict__`, a fordító gyakran automatikusan is a read-only útvonalat választja, de az explicit `__ldg` egyértelművé teszi a szándékot.

A read-only cache előnye, hogy a globális memória sávszélességét tehermentesíti, különösen akkor, ha a hozzáférési minta nem tökéletesen coalescing. A hatás mérhető Nsight Compute-ban a `l1tex__t_sectors_pipe_lsu_mem_global_op_ld` jellegű metrikákkal.

Fontos: Az `__ldg` csak olvasásra használható, írásra nem. Ha a kernel módosítja az adatot, a read-only feltételezés hibás, és a viselkedés nem definiált.

7.3.5 5. Textúra-memória és textúraobjektumok

A textúra-memória a CUDA egyik legrégebbi specializált memóriatere, amelyet a grafikus renderelésből örökölt. A modern CUDA-ban a textúra-hozzáférés a globális memórián lévő adatokra irányul, de a textúra-hardver extra funkciókat biztosít:

- **Hardveres interpoláció** – bilineáris, trilineáris interpoláció float típusokra.
- **Címzési módok** – clamp, wrap, mirror a szélek kezelésére.
- **Normalizált koordináták** – a textúra koordináták 0..1 tartományba képezése.
- **Adatkonverzió** – például 8 bites egész értékekből normalizált float értékek előállítása.

A textúrákat kétféleképpen érhetjük el: textúra-referenciákkal (régebbi, globális szimbólum) és **textúraobjektumok**kal (újabb, a kernel függvényparaméterként adható át). A textúraobjektumok használata ajánlott, mert rugalmasabbak, és nem igényelnek globális deklarációt.

Textúraobjektum létrehozása:

```
cudaResourceDesc resDesc = {};
resDesc.resType = cudaResourceTypeLinear;   // 1D lineáris memória
resDesc.res.linear.devPtr = d_input;
resDesc.res.linear.desc = cudaCreateChannelDesc<float>(); // float csatorna
resDesc.res.linear.sizeInBytes = n * sizeof(float);

cudaTextureDesc texDesc = {};
texDesc.readMode = cudaReadModeElementType;  // elem típusú olvasás, nincs konverzió

cudaTextureObject_t tex = 0;
CUDA_CHECK(cudaCreateTextureObject(&tex, &resDesc, &texDesc, nullptr));
```

A kernel függvényparaméterként kapja a textúraobjektumot:

```
__global__ void texture_kernel(float* __restrict__ output,
                               int n,
                               cudaTextureObject_t tex) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= n) return;
    float x = tex1Dfetch<float>(tex, idx);   // textúra-hardveres olvasás
    output[idx] = x * 2.0f;
}
```

A `tex1Dfetch` a textúra-hardveren keresztül olvas. Fontos, hogy a `tex1Dfetch` csak `cudaTextureObject_t`-vel használható, textúra-referenciával a `tex1D` függvényt használjuk.

Mikor válasszunk textúrát? Ha nincs szükség interpolációra vagy speciális címzésre, a `__ldg` vagy a `const __restrict__` pointer gyakran ugyanolyan gyors, és egyszerűbb. A textúra akkor előnyös, ha:

- Bilineáris interpolációra van szükség (pl. képfelfelbontás).
- A koordináták szélsőségesek, és clamp/wrap viselkedést akarunk.
- Az adat típusa 8 vagy 16 bites, és normalizált float értékké konvertálnánk.

A textúra cache a modern architektúrákon gyakran egyesített a read-only data cache-sel, így a teljesítménykülönbség a `__ldg` és a textúra között kicsi. A régebbi hardvereken külön textúra cache volt, ami bizonyos mintáknál előnyös lehetett.

7.3.6 6. Labor: konstans memória és textúraobjektum használata

Ebben a laborban egy olyan CUDA C++ programot készítünk, amely a konstans memóriából veszi a skálázási paramétereket, a bemeneti adatot pedig textúraobjektumon keresztül olvassa. A kernel minden elemet `scale * x + offset` módon transzformál. A labor célja a konstans és textúra-hozzáférés működésének ellenőrzése, a helyes eredmény előállítása, valamint a futásidő és sávszélesség mérése.

Környezet (példa bejegyzés, saját mérés előtt ellenőrizendő): - GPU: NVIDIA A100 40 GB (compute capability 8.0) - Driver: 545.23.08 (Windows) / 545.29.06 (Linux) - CUDA Toolkit: 12.3 - OS: Ubuntu 22.04 / Windows 11 - Fordító: nvcc (12.3), host: g++ 11.4

Kód: A teljes program alább látható. A hibakezelést a `CUDA_CHECK` makró biztosítja, a kernel indítás után `cudaGetLastError` és `cudaDeviceSynchronize` szerepel.

```
#include <cuda_runtime.h>
#include <iostream>
#include <vector>
#include <cmath>
#include <cstdlib>

#define CUDA_CHECK(call) do {                                    \
    cudaError_t err = (call);                                    \
    if (err != cudaSuccess) {                                    \
        std::cerr << "CUDA error at " << __FILE__ << ":"         \
                  << __LINE__ << ": " << cudaGetErrorString(err) \
                  << std::endl;                                  \
        std::abort();                                            \
    }                                                            \
} while (0)

__constant__ float c_params[2]; // c_params[0] = skála, c_params[1] = eltolás

__global__ void constant_texture_kernel(float* __restrict__ output,
                                        int n,
                                        cudaTextureObject_t tex) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= n) return;
    float x = tex1Dfetch<float>(tex, idx);
    output[idx] = c_params[0] * x + c_params[1];
}

int main() {
    const int n = 1 << 22;  // 4 M elem
    const size_t bytes = n * sizeof(float);

    std::vector<float> h_input(n), h_output(n), h_expected(n);

    // Bemeneti adat inicializálása
    for (int i = 0; i < n; ++i) {
        h_input[i] = static_cast<float>(i % 1000) / 100.0f; // 0.00 .. 9.99
    }

    // Konstans paraméterek
    float h_params[2] = { 2.0f, 1.5f };

    // Várt eredmény (CPU oldali referencia)
    for (int i = 0; i < n; ++i) {
        h_expected[i] = h_params[0] * h_input[i] + h_params[1];
    }

    // Eszközmemória allokálása
```

```
float* d_input;
float* d_output;
CUDA_CHECK(cudaMalloc(&d_input, bytes));
CUDA_CHECK(cudaMalloc(&d_output, bytes));

// Bemenet másolása global memory-ba
CUDA_CHECK(cudaMemcpy(d_input, h_input.data(), bytes, cudaMemcpyHostToDevice));

// Konstans memória feltöltése
CUDA_CHECK(cudaMemcpyToSymbol(c_params, h_params, sizeof(h_params)));

// Textúraobjektum létrehozása
cudaResourceDesc resDesc = {};
resDesc.resType = cudaResourceTypeLinear;
resDesc.res.linear.devPtr = d_input;
resDesc.res.linear.desc = cudaCreateChannelDesc<float>();
resDesc.res.linear.sizeInBytes = bytes;

cudaTextureDesc texDesc = {};
texDesc.readMode = cudaReadModeElementType;

cudaTextureObject_t tex = 0;
CUDA_CHECK(cudaCreateTextureObject(&tex, &resDesc, &texDesc, nullptr));

// Kernel indítási konfiguráció
int blockSize = 256;
int gridSize = (n + blockSize - 1) / blockSize;

// Időmérés CUDA eventekkel
cudaEvent_t start, stop;
CUDA_CHECK(cudaEventCreate(&start));
CUDA_CHECK(cudaEventCreate(&stop));

// Warm-up
constant_texture_kernel<<<gridSize, blockSize>>>(d_output, n, tex);
CUDA_CHECK(cudaGetLastError());
CUDA_CHECK(cudaDeviceSynchronize());

// Mérés: 10 ismétlés
std::vector<float> times;
for (int iter = 0; iter < 10; ++iter) {
    CUDA_CHECK(cudaEventRecord(start));
    constant_texture_kernel<<<gridSize, blockSize>>>(d_output, n, tex);
    CUDA_CHECK(cudaEventRecord(stop));
    CUDA_CHECK(cudaEventSynchronize(stop));

    float ms = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&ms, start, stop));
    times.push_back(ms);
}

// Eredmény visszamásolása és ellenőrzése
CUDA_CHECK(cudaMemcpy(h_output.data(), d_output, bytes, cudaMemcpyDeviceToHost));

bool ok = true;
float max_abs_err = 0.0f;
for (int i = 0; i < n; ++i) {
    float err = std::fabs(h_output[i] - h_expected[i]);
    if (err > max_abs_err) max_abs_err = err;
    if (err > 1e-5f) { ok = false; break; }
}

std::cout << "Helyes eredmény: " << (ok ? "igen" : "nem") << "\n";
```

```
    std::cout << "Maximális abszolút hiba: " << max_abs_err << "\n";

    // Átlag és szórás számítása
    float sum = 0.0f, sq_sum = 0.0f;
    for (float t : times) { sum += t; sq_sum += t * t; }
    float mean = sum / times.size();
    float variance = (sq_sum / times.size()) - (mean * mean);
    float stddev = std::sqrt(std::max(0.0f, variance));

    std::cout << "Futásidő (ms): átlag = " << mean
              << ", szórás = " << stddev << "\n";
    double achieved_bandwidth_GBs = (bytes * 2) / (mean * 1e-3) / 1e9;
    std::cout << "Effektív sávszélesség (GB/s): " << achieved_bandwidth_GBs << "\n";

    // Takarítás
    CUDA_CHECK(cudaDestroyTextureObject(tex));
    CUDA_CHECK(cudaFree(d_input));
    CUDA_CHECK(cudaFree(d_output));
    CUDA_CHECK(cudaEventDestroy(start));
    CUDA_CHECK(cudaEventDestroy(stop));

    return 0;
}
```

Buildparancs:

```
nvcc -arch=sm_80 -O3 -std=c++17 constant_texture_lab.cu -o constant_texture_lab
```

Futtatási parancs:

```
./constant_texture_lab
```

Várt eredmény: A kimenet minden elemének meg kell egyeznie a CPU-oldali referenciaértékkel, a maximális abszolút hiba FP32 esetén legfeljebb ~1e-5. A kernel futásidejét a memória-hozzáférés sávszélessége határozza meg; az A100-on a globális memória elvi sávszélessége 1,55 TB/s körül van, de a gyakorlatban a kernelünk a bemeneti (4 MiB) és kimeneti (4 MiB) adat mozgatásával 8 MiB-ot olvas/ír. A várt sávszélesség 500–900 GB/s lehet, a pontos érték a hozzáférési mintától és a cache-től függ.

Mérési napló (példaértékű, nem ellenőrzött mérés): Az alábbi adatok egy reprodukálható jegyzőkönyv sablonját adják; a saját környezetben mért értékeket kell rögzíteni.

Iteráció	Futásidő (ms)
1	0,0182
2	0,0178
3	0,0179
4	0,0180
5	0,0181
6	0,0177
7	0,0182
8	0,0180
9	0,0179
10	0,0178

Ebből átlag = 0,0180 ms, szórás = 0,0002 ms. A számolt effektív sávszélesség körülbelül 8 MiB / 0,0180 ms = 444 GB/s. Ez alacsonyabb, mint az A100 csúcsa, mert a kernel egyetlen elemre

dolgoz, és a memória-hozzáférés nem teljesen optimalizált. A mérési naplót mindig a tényleges környezetben kell elvégezni, a fenti értékek csak illusztrációk.

Értelmezés: A konstans memória itt csak két lebegőpontos paramétert tárol, így a broadcast előnye nem mérhető látványosan. A textúra használata a lineáris olvasásnál hasonló teljesítményt nyújt, mint a `__ldg` vagy a `const __restrict__` pointer. A labor elsősorban a helyes működést és az API-használatot ellenőrzi.

7.3.7 7. Tipikus hibák és helyes megoldások

Hiba 1: A konstans memóriába kernelből próbálunk írni.

```
__constant__ float c_data[4];
__global__ void bad_kernel() {
    c_data[0] = 1.0f; // Fordítási hiba vagy nem definiált viselkedés
}
```

Megoldás: A konstans memóriát csak host oldalról, `cudaMemcpyToSymbol` segítségével töltsük fel. A kernelben csak olvasható.

Hiba 2: A `cudaMemcpyToSymbol` mérete nem egyezik a deklarált konstans tömb méretével.

```
__constant__ float c_params[3];
float h_params[2] = { 1.0f, 2.0f };
cudaMemcpyToSymbol(c_params, h_params, sizeof(h_params)); // 8 bájtot másol, pedig 12 kell
```

Megoldás: Mindig a deklarált konstans szimbólum méretét használjuk, pl. `sizeof(c_params)` a host oldalon nem használható, mert az a kernel szimbólum. Ilyenkor a kódban explicit megadjuk a méretet: `sizeof(float) * 3`.

Hiba 3: Textúraobjektum létrehozása után elfelejtjük ellenőrizni a visszatérési értéket, és érvénytelen `tex`-et adunk át.

Megoldás: A `cudaCreateTextureObject` hibakódját a `CUDA_CHECK` makróval ellenőrizzük.

Hiba 4: A `tex1Dfetch`-et textúra-referenciával próbáljuk használni.

```
texture<float, 1, cudaReadModeElementType> texRef; // régebbi
float x = tex1Dfetch<float>(texRef, idx); // fordítási hiba; tex1D kell
```

Megoldás: Textúra-referencia esetén a `tex1D(texRef, x)` függvényt használjuk, vagy térjünk át textúraobjektumokra, ahol a `tex1Dfetch` a megfelelő.

Hiba 5: A `__ldg`-vel történő olvasás után a memóriát írjuk is, ami a read-only feltételezést megsérti.

```
__global__ void bad_ldg(float* a, int n) {
    int i = threadIdx.x + blockIdx.x * blockDim.x;
    if (i < n) {
        float v = __ldg(&a[i]);
        a[i] = v * 2.0f; // ez írás; ha a pointer nem const __restrict__, a fordító hibát
↪   jelezhet
    }
}
```

Megoldás: A `__ldg` csak akkor használható, ha az adathoz nem írunk. Ha írás is történik, használjunk írható globális mutatót, és ne erőltessük a read-only cache-t.

Hiba 6: A lokális memória méretének ellenőrzése nélkül nagy privát tömböket deklarálunk, ami spillhez vezet.

Megoldás: Fordítás közben `-Xptxas -v` opcióval nézzük meg a lokális memória használatot, és az Nsight Compute `local_load` metrikáit is figyeljük. Ha nagy a spill, csökkentsük a privát tömböt, vagy használjunk shared memóriát.

7.3.8 8. Összefoglalás és önellenőrző kérdés

Ebben az alfejezetben a specializált CUDA-memóriatereket tárgyaltuk. A lokális memória a regiszter-spillhez kapcsolódik, és a globális memóriában tárolódik; használata nem katasztrófa, de figyelni kell a memóriaforgalomra. A konstans memória egy írásvédett, gyorsítótárazott terület, amely akkor hatékony, ha a warp minden szála ugyanazt az adatot olvassa. A `__ldg` és a read-only cache hasonló gyorsítást kínál, de globális memóriából olvas. A textúra-memória hardveres interpolációt, címzési módokat és adatkonverziót biztosít, és textúraobjektumokkal rugalmasan kezelhető.

A laborban konstans memóriát és textúraobjektumot használtunk, majd ellenőriztük a helyességet és mértük a futásidőt. A mért és várt eredményeket elkülönítettük, hangsúlyozva, hogy a teljesítményadatok mindig a tényleges környezetben érvényesek.

Önellenőrző kérdés: Mi történik, ha egy warp szálai a konstans memóriában különböző címeket olvasnak, és miért romlik ilyenkor a teljesítmény? Indokold meg a konstans cache hardveres viselkedése alapján.

Válasz vázlat: A konstans cache broadcast módban működik, ha minden szál ugyanazt a címet olvassa; ilyenkor egyetlen memóriatranzakció szolgálja ki az egész warpot. Ha a szálak különböző címeket kérnek, a hardver sorba rendezi a kéréseket, és több ciklus szükséges, ami csökkenti a sávszélességet és növeli a késleltetést. Ezért a konstans memóriát olyan adatokra érdemes használni, amelyeket minden szál azonos módon olvas, például közös paraméterekre.

7.4 Memóriaterek kiválasztása

A memóriaterek kiválasztása a CUDA-kernel tervezésének egyik legnagyobb hatású döntése. Nem az a kérdés, hogy „melyik memóriatér a leggyorsabb”, hanem az, hogy egy adott adat melyik memóriatérben éri el a legjobb kompromisszumot a késleltetés, a sávszélesség és a láthatóság szempontjából. A regiszterek gyakorlatilag nulla késleltetésűek, de csak egyetlen szálra látszanak. A global memory a teljes grid számára elérhető, de a DRAM-késleltetés több száz ciklus is lehet. A shared, constant, local és texture terek közé esnek; a helyes választás a hozzáférési mintától, az élettartamtól és az írási/olvasási igénytől függ. Ebben az alfejezetben egy döntési keretet építünk fel, megvizsgáljuk egy kernel memóriaprofilját, majd egy ellenőrizhető laborban elkészítjük a memóriatér-döntési táblázatot.

7.4.1 Döntési keret: latency, bandwidth és scope

A memóriatér-választás három műszaki tengelye a következő.

Latency (késleltetés): Az az idő, amennyi egy memóriaművelet elindítása és az adat rendelkezésre állása között telik el. A GPU a késleltetést elsősorban szálak közötti átfedéssel (latency hiding) rejti el, de a forrás memóriatér nagyságrendileg meghatározza a várakozási időt. A regiszterek hozzáférése gyakorlatilag az utasítás végrehajtásának része. A shared memory késleltetése jellemzően 20–40 ciklus nagyságrendű, míg a global memory elérése cache nélkül több száz ciklust is jelenthet. A constant memória késleltetése broadcast esetén közel áll a regiszterhez, konfliktus esetén viszont drámaian romlik.

Bandwidth (sávszélesség): A másodpercenként átvihető bájtok mennyisége. A global memória sávszélessége a DRAM-ból származik, és tipikusan a GPU egyik legszűkebb erőforrása. A shared memory sávszélessége nagyságrenddel nagyobb, mert on-chip memória. A constant memória széles sávszélességet tud biztosítani, ha a warp minden szála ugyanazt a címet olvassa (broadcast). A local memória valójában a global memóriába kerül, tehát sávszélessége a global memoryéval megegyező, hiába neve „local”. A texture memória sávszélessége a textúra gyorsítótáron keresztül a 2D-s térbeli lokalitást használja ki.

Scope (láthatóság): Azt mondja meg, hogy egy adat mely végrehajtási egységek számára elérhető. A CUDA-végrehajtási hierarchiában a szál, a warp, a block, a grid és a host szintjei léteznek. A regiszterek szál-szintűek, a shared memory block-szintű, a global memory grid-szintű, a constant memória grid-szintű (írásra host-oldalról, olvasásra device-szinten), a local memória szál-szintű, de fizikailag global memóriában lakik. A scope megszabja, hogy egy adatot milyen szinkronizációs mechanizmussal lehet megosztani, és milyen élettartamot várhatunk el.

A döntési folyamat első lépése tehát az adat életciklusának és hozzáférési mintájának feltérképezése. Kérdezzük meg:

- Hány szál használja ugyanazt az adatot?
- Csak olvasásra vagy írásra is?
- Mennyi az adat mérete?
- Milyen időközönként férünk hozzá?
- Van-e térbeli vagy időbeli lokalitás?
- Kell-e a host és a device között mozgatni?
- Lehet-e a hozzáférést broadcasttá alakítani?

Ezekre a kérdésekre adott válaszokból áll össze a döntési táblázat.

7.4.2 Az öt klasszikus CUDA-memóriatér döntési szempontjai

A korábbi alfejezetekben már bemutattuk az egyes memóriaterek technikai hátterét, itt kifejezetten a kiválasztás szempontjait foglaljuk össze.

7.4.3 Global memory

A global memory az alapértelmezett nagyméretű, grid-szintű memóriatér. Ide kerülnek a `cudaMalloc`-kal allokált pufferek, a `cudaMemcpy`-vel feltöltött tömbök és a kernelparaméterként átadott mutatók által címzett adatok. Használata akkor indokolt, ha:

- az adat mérete meghaladja a shared vagy constant memória kapacitását,
- az adatot több block, a teljes grid vagy a host és a device közösen használja,

- írásra is szükség van,
- az adat élettartama túlnyúlik egyetlen kernelhíváson.

A global memory költsége csökkenthető coalesced hozzáféréssel, vectorizált betöltéssel és L1/L2 cache-barát mintával. Ha egy adatot csak egyszer olvasunk, és nincs más lehetőség, akkor a global memory a természetes választás. Ha azonban ugyanazt a blokkon belül többször újraolvassuk, érdemes megfontolni a shared memory-ba másolást.

7.4.4 Shared memory

A shared memory block-szintű, on-chip, programozható, alacsony késleltetésű memóriatér. Akkor válasszuk, ha:

- ugyanazt az adatot egy blockon belül több szál szeretné elérni,
- a hozzáférések között adatcsere vagy csoportos redukció szükséges,
- egy global memóriából betöltött adatdarabot újrahasznosítunk tiling vagy stencil során,
- a hozzáférést bank-konfliktusmentesre tudjuk szervezni.

A shared memory mérete korlátozott: tipikusan 48–164 KiB SM-enként, a konfigurációtól és architektúrától függően. Ha a teljes probléma elférne, akkor is figyelni kell arra, hogy a shared memory használata csökkenti az occupancy-t, mert egy SM-en a blockok száma korlátozott. A döntés során ezért mindig ellenőrizni kell, hogy a shared memória által elért global-memory-megtakarítás nagyobb-e, mint az occupancy-csökkenésből adódó latency-hiding romlás.

7.4.5 Constant memory

A constant memory grid-szintű, olvasható a device számára, de írni jellemzően a host írhat (`cudaMemcpyToSymbol`). Akkor előnyös, ha:

- ugyanazt a néhány bájtnyi paramétert a warp minden szála olvassa anélkül, hogy a hozzáférési címek különböznének,
- a hozzáférés broadcast mintájú: például egy skála, offset, filter együttható vagy táblázat,
- az adat nem változik a kernel futása alatt.

A constant memória broadcast sávszélessége kiváló, de ha egy warpen belül különböző címekről olvasnak a szálak, a hozzáférés sorosodik, és a teljesítmény drasztikusan romlik. Ezért az olyan adat, amelyet a szálak különböző indexekkel érnek el, nem ide való. Ha a táblázat mérete nagy és véletlenszerű indexekkel olvassuk, a global memory vagy a texture memória jobb választás lehet.

7.4.6 Local memory

A local memory fogalma megtévesztő: ez szál-szintű, de fizikailag a global memóriában tárolódik. Akkor használja a fordító, ha a bemeneti forráskódban szereplő lokális változó nem fér el a regiszterekben, vagy ha a tömb indexelése dinamikus, és a tömb nem konvertálható regiszterekre. A local memória használata rendszerint kerülendő:

- ha egy tömb mérete kicsi és a hozzáférések statikusak, akkor a fordító regiszterekben tarthatja;
- ha nagy, dinamikusan indexelt privát tömb szükséges, akkor is gondoljunk arra, hogy a valódi adatmegosztás lehetőségét kihasználjuk shared memory segítségével.

A local memory a kernel memóriaprofiljában gyakran a „local load/store” tranzakciókban jelenik meg, és gyengíti a teljesítményt. A döntési táblázatban a local memory elsősorban figyelmeztető bejegyzés: ha a profilban nagy local forgalom látható, akkor a kernel regiszter- vagy shared-memory-kialakítását újra kell gondolni.

7.4.7 Texture memory

A texture memory egy speciális, read-only elérési út, amelyet textúraobjektumokkal vagy textúra referenciákkal érhetünk el. Akkor használjuk, ha:

- 2D/3D térbeli lokalitás van a hozzáférésekben, és a cache-sorok kihasználása fontos,
- interpolációt, normalizált koordinátákat vagy címzési módokat szeretnénk használni,
- a hozzáférés read-only, és a hardware textúra gyorstár kihasználása előnyös.

A modern GPU-ken a textúra memória sok esetben a global memory cache-elt eléréséhez hasonló, így nem szabad automatikusan „gyorsabbnak” tekinteni. A döntés során a textúra akkor indokolt, ha a mintavételi mód, a szűrés vagy a határkezelés hardveres támogatása csökkenti a kód komplexitását, illetve ha a hozzáférési minták miatt a textúra cache jobb találati arányt ad.

7.4.8 Memóriatér-döntési táblázat

A döntési táblázat egy tömör összefoglaló eszköz, amelyet fejlesztés közben a kernel tervezésekor használhatunk. Az alábbi táblázat a CUDA memóriatereit foglalja össze a fő döntési dimenziók mentén. A számértékek nagyságrendi becslések, és hardverfüggőek; mindig az adott GPU compute capability szerint kell pontosítani.

Memóriatér	Scope	Élettartam	Hozzáférési jellemzők	Írás/olvasás	Jellemző késleltetés	Sávszélesség-osztály	Mikor válasszuk?
Regiszter	Szál	Kernel során	Statikus, privát	Írható/olvasható	0–1 ciklus	Maximum	Minden gyakran használt skalár, kis tömb
Local memory	Szál	Kernel során	Privát, de DRAM-ban	Írható/olvasható	400–800 ciklus	Global szint	Kerülendő; csak ha a fordító spillt generál
Shared memory	Block	Block futása	Alacsony késleltetésű, on-chip	Írható/olvasható	20–40 ciklus	Nagyon nagy	Blokkszintű újrafelhasználás, tiling, kommunikáció

Memóriatér	Scope	Élettartam	Hozzáférési jellemzők	Írás/olvasás	Jellemző késleltetés	Sávszélesség-osztály	Mikor válasszuk?
Constant memory	Grid	Kernel során (host írás)	Broadcast, read-only	Device olvassa, host írja	Broadcast: alacsony; konfliktus: magas	Nagy broadcast esetén	Warp-uniform paraméterek, táblázatok
Global memory	Grid	Alkalmazás során	Nagyméretű, cache-elt	Írható/olvasható	400–800 ciklus	DRAM-sávszélesség	Nagy adatok, grid-szintű megosztás, írás
Texture memory	Grid	Kernel során (read-only)	2D/3D lokalitás, interpoláció	Device olvassa	Cache-től függő	Változó	Képinterpoláció, szűrés, speciális címzés

Ez a táblázat a labor alapja is. A mérések során a táblázat sávszélességre és késleltetésre vonatkozó celláit a saját GPU-n mért értékekkel és a kernelprofilból származó mutatókkal támasztjuk alá.

7.4.9 Kernel memóriaprofiljának felépítése

A memóriatér-döntés nem szakadhat el a méréstől. Egy kernel memóriaprofilja azt mondja meg, hogy a kernel futása során milyen memóriaterekből mennyi adatot mozgatunk, milyen hatékonysággal, és hol keletkeznek várakozások. A profil elkészítésének lépései:

1. **Felmérjük a hasznos adatforgalmat.** Hány bájtot olvasunk és írunk a global memoryból? Hány bájtot töltünk be a shared memorybe? A hasznos forgalom az algoritmus által ténylegesen megkívánt adatmennyiség.

2. **Megnézzük a tényleges memóriatranzakciókat.** A global memória nem bájtonként, hanem 32-, 64- vagy 128 bájtos szektorokban mozog. A memóriaprofilból kiderül, hogy a kért szektorok hány százalékát használjuk ki. A coalescing hatékonysága a hasznos bájtok és a ténylegesen beolvasott szektorok bájtjainak aránya.

3. **Megvizsgáljuk a shared memory bank konfliktusokat.** A shared memory elérési mintája lehet conflict-free, vagy okozhat 2-, 4-, 8-utas konfliktust. A profilban a bank conflictok száma azt jelzi, hogy a döntés egyébként helyes, de a adatelrendezés nem optimális.

4. **Azonosítjuk a local memory spillt.** Ha a regiszterhasználat meghaladja a rendelkezésre álló regisztereket, a fordító local memóriába ír. A profilban a local load/store tranzakciók jelennek meg. Ezt a döntési táblázatban a local memória sorához kapcsoljuk.

5. **Értelmezzük a cache találati arányokat.** Az L1 és L2 találati arányok megmutatják, hogy a global memory hozzáférésekből mennyit sikerül elkerülni. A shared memory tiling célja gyakran az, hogy a global hozzáférések helyett L1/shared hozzáféréseket használjunk.

6. **Összevetjük a mért sávszélességet a névleges csúccsal.** A Roofline-modellből ismert, hogy a kernel lehet memória-korlátos vagy számítás-korlátos. A memóriaprofilból kiolvasott forgalom és idő hányadosa adja a tényleges sávszélességet, amelyet a GPU specifikációjával kell összehasonlítani.

A Nsight Compute ilyen mutatói közé tartoznak például:

- `l1tex__t_sectors_pipe_lsu_mem_global_op_ld.sum`
- `l1tex__t_sectors_pipe_lsu_mem_global_op_st.sum`
- `l1tex__data_bank_conflicts_pipe_lsu_mem_shared.sum`
- `smsp__sass_inst_executed_op_local_ld.sum`
- `smsp__sass_inst_executed_op_local_st.sum`
- `gpu__time_duration.sum`

Ezeket a metrikaneveket a Nsight Compute verziója és architektúra szerint ellenőrizni kell, és a laborban a pontos név szerepeljen. A cél az, hogy a memóriatér-döntés számszerűen is igazolható legyen.

7.4.10 Labor: memóriatér-döntési táblázat és mérés

Az alábbi labor egy olyan kis CUDA-projektet mutat be, amelyben ugyanaz a feladat több memóriatér-kialakítással fut. A feladat egyszerű: egy bemeneti vektort skálázunk egy konstanssal, eltolunk egy konstanssal, majd blokkonként összegezzük a részeredményeket. A kialakítások lehetőséget adnak a global, constant, shared és local memória szerepének demonstrálására.

7.4.11 Kernelváltozatok

1. **Global-only változat:** minden szál kiolvassa a bemeneti elemet a global memoryból, elvégzi a skálázást és eltolást, majd kiírja az eredményt.
2. **Constant-táras változat:** a skála és az offset értékeket constant memóriába tesszük, így a warp minden szála broadcast hozzáférést kap.
3. **Shared-redukciós változat:** a blokk beolvassa a bemeneti szegmenst, a szálak az egyes elemeket shared memóriába írják, majd fa-struktúrájú redukcióval blokkonkénti összeget számolnak.
4. **Local-memory-büntetés változat:** egy privát tömböt dinamikusan indexelünk, hogy a fordító local memoryt használjon; ezzel megmutatjuk a local memória árát.

7.4.12 CUDA C++ példakód

```
// fordítás: nvcc -O3 -arch=sm_89 -o memlab memlab.cu
// megjegyzés: a kód a labor izolált környezetéhez készült,
// éles rendszeren és hitelesítő adattal ne futtassuk.
#include <cuda_runtime.h>
#include <cstdio>
#include <cmath>
#include <vector>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err__ = (call);                                         \
        if (err__ != cudaSuccess) {                                         \
            std::fprintf(stderr, "CUDA hiba: %s, fajl: %s, sor: %d\n",      \
                         cudaGetErrorString(err__), __FILE__, __LINE__);    \
```

```
            return err__;                                                   \
        }                                                                   \
    } while (0)

__constant__ float c_scale;
__constant__ float c_offset;

template <int BLOCK>
__global__ void global_only_scale(const float* __restrict__ x,
                                  float* __restrict__ y,
                                  float scale,
                                  float offset,
                                  int n)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        float v = x[i];
        y[i] = fmaf(v, scale, offset);
    }
}

template <int BLOCK>
__global__ void constant_scale(const float* __restrict__ x,
                               float* __restrict__ y,
                               int n)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        float v = x[i];
        y[i] = fmaf(v, c_scale, c_offset);
    }
}

template <int BLOCK>
__global__ void shared_reduce(const float* __restrict__ x,
                              float* __restrict__ block_sum,
                              int n)
{
    __shared__ float sdata[BLOCK];
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    float v = (i < n) ? x[i] : 0.0f;
    v = fmaf(v, c_scale, c_offset);
    sdata[threadIdx.x] = v;
    __syncthreads();

    for (int stride = BLOCK / 2; stride > 0; stride >>= 1) {
        if (threadIdx.x < stride) {
            sdata[threadIdx.x] += sdata[threadIdx.x + stride];
        }
        __syncthreads();
    }

    if (threadIdx.x == 0) {
        block_sum[blockIdx.x] = sdata[0];
    }
}

__global__ void local_penalty(const float* __restrict__ x,
                              float* __restrict__ y,
                              int n,
                              int kMax)
{
    // A tmp[] dinamikusan indexelt, ezért a fordító nagy valószínűséggel
```

```
    // local memoryt használ. A kMax paraméter miatt a ciklus nem unrollható.
    float tmp[128];
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        for (int k = 0; k < kMax; ++k) {
            tmp[k] = x[i] * static_cast<float>(k);
        }
        float sum = 0.0f;
        for (int k = 0; k < kMax; ++k) {
            sum += tmp[k];
        }
        y[i] = sum;
    }
}
```

A `local_penalty` kernelben a `tmp` tömb mérete 128 float. A fordító akkor tudná regiszterben tartani, ha minden hozzáférést konstans indexszel oldana fel. Mivel a `k` változó futás közben, a `kMax` paramétertől függően változik, a tömböt valószínűleg local memoryba helyezi. Ezt a mérési profilban a local tranzakciók növekedése mutatja majd.

A host oldalon a mérési ciklus:

```
int main()
{
    constexpr int N = 1 << 22;
    constexpr int BLOCK = 256;
    constexpr int GRID = (N + BLOCK - 1) / BLOCK;
    constexpr int ITER_WARMUP = 5;
    constexpr int ITER_MEAS = 10;

    size_t nBytes = N * sizeof(float);
    std::vector<float> h_in(N), h_out(N);
    for (int i = 0; i < N; ++i) h_in[i] = 1.0f;

    float *d_x, *d_y, *d_sum;
    CUDA_CHECK(cudaMalloc(&d_x, nBytes));
    CUDA_CHECK(cudaMalloc(&d_y, nBytes));
    CUDA_CHECK(cudaMalloc(&d_sum, GRID * sizeof(float)));
    CUDA_CHECK(cudaMemcpy(d_x, h_in.data(), nBytes, cudaMemcpyHostToDevice));

    float scale = 0.5f, offset = 1.0f;
    CUDA_CHECK(cudaMemcpyToSymbol(c_scale, &scale, sizeof(float)));
    CUDA_CHECK(cudaMemcpyToSymbol(c_offset, &offset, sizeof(float)));

    cudaEvent_t start, stop;
    CUDA_CHECK(cudaEventCreate(&start));
    CUDA_CHECK(cudaEventCreate(&stop));

    // Warm-up
    for (int it = 0; it < ITER_WARMUP; ++it) {
        global_only_scale<BLOCK><<<GRID, BLOCK>>>(d_x, d_y, scale, offset, N);
    }
    CUDA_CHECK(cudaDeviceSynchronize());

    // Mérés: global-only
    float times[ITER_MEAS];
    for (int it = 0; it < ITER_MEAS; ++it) {
        CUDA_CHECK(cudaEventRecord(start));
        global_only_scale<BLOCK><<<GRID, BLOCK>>>(d_x, d_y, scale, offset, N);
        CUDA_CHECK(cudaEventRecord(stop));
        CUDA_CHECK(cudaEventSynchronize(stop));
        CUDA_CHECK(cudaEventElapsedTime(&times[it], start, stop));
    }
```

```
    float avg = 0.0f, var = 0.0f;
    for (int it = 1; it < ITER_MEAS; ++it) avg += times[it];
    avg /= (ITER_MEAS - 1);
    for (int it = 1; it < ITER_MEAS; ++it) var += (times[it] - avg) * (times[it] - avg);
    var /= (ITER_MEAS - 1);
    std::printf("[global-only] atlag: %.3f ms, szoras: %.3f ms\n", avg, std::sqrt(var));

    // Hasonló mérés a constant_scale és shared_reduce esetén
    // A local_penalty kernel mérésénél a kMax legyen pl. 64.

    CUDA_CHECK(cudaFree(d_x));
    CUDA_CHECK(cudaFree(d_y));
    CUDA_CHECK(cudaFree(d_sum));
    return 0;
}
```

A mérésnél az első iterációt a warm-up után a mérésből kihagytuk, a maradékból átlagot és szórást számolunk. A `constant_scale`-hez hasonló mérési ciklus kell. A `shared_reduce` kimeneti tömbje a blokkonkénti összeg, így a global írás forgalom kisebb. A `local_penalty` esetén a `kMax` értékét a híváskor adjuk meg, például 64.

7.4.13 Mérési konfiguráció

A labor a következő, izolált környezetben fut. A példaértékek helyőrzők, és a tényleges mérés előtt az adott gépen rögzíteni kell.

- **GPU:** NVIDIA RTX 4090, 24 GiB GDDR6X, compute capability 8.9, GPU órajel: boosted, power limit: 450 W, ECC off, MIG off, virtualizáció nincs.
- **Meghajtó:** NVIDIA driver 550.54.14.
- **CUDA toolkit:** 12.4, nvcc 12.4, `-O3 -arch=sm_89`.
- **OS:** Ubuntu 22.04, kernel 6.5.0.
- **Háttérterhelés:** idle, `nvidia-smi` szerint nincs más GPU-folyamat.
- **Bemenet:** `N = 1<<22` float, `BLOCK = 256`, `GRID = 16384`.

Ez a konfiguráció a mérési napló fejlécébe tartozik.

7.4.14 Mérési napló (várt és mért elkülönítve)

A laborátadandó egy olyan táblázat, amelyben a **várt tendencia** oszlop a memóriatér-döntési érvelésből következik, a **mért** oszlop pedig a saját futásból származik. Az alábbi táblázat a várt tendenciákat és a mérési mezőket mutatja. A mért értékek a könyv szövegében nem ellenőrzött futásból származnak, ezért a labor során kell kitölteni.

Kernelváltozat	Várt memóriatérhatás	Várt tendencia	Mért idő (ms)	Mért sávszélesség (GB/s)	Megjegyzés
global_only_scale	Global olvasás/írás, coalesced	Alapvonal	_____	_____	Két global hozzáférés elemenként

Kernelváltozat	Várt memóriatérhatás	Várt tendencia	Mért idő (ms)	Mért sávszélesség (GB/s)	Megjegyzés
constant_scale	Constant broadcast a skálára	Gyorsabb a global-only-nál	_____	_____	Nincs extra DRAM-forgalom a paraméterekre
shared_reduce	Shared használat, global forgalom töredéke	Kevesebb global sávszélesség, de shared használat	_____	_____	Blockonként egy kimeneti összeg
local_penalty	Local memória spill a privát tömbhöz	Lassabb, több DRAM-forgalom	_____	_____	A local valójában global memória

A mérési naplóhoz tartozik a Nsight Compute parancs:

```
ncu --metrics l1tex__t_sectors_pipe_lsu_mem_global_op_ld.sum, \
l1tex__t_sectors_pipe_lsu_mem_global_op_st.sum, \
l1tex__data_bank_conflicts_pipe_lsu_mem_shared.sum, \
smsp__sass_inst_executed_op_local_ld.sum, \
gpu__time_duration.sum \
./memlab
```

A kimeneti metrikaneveket a telepített Nsight Compute verziójához kell igazítani. A profilozás során külön figyeljünk a `local_penalty` kernelre: a local load tranzakciók száma jelentősen megnő.

7.4.15 A memóriatér-döntési táblázat kitöltése

A labor végső eredménye egy **memóriatér-döntési táblázat**, amely a kerneltervezéshez használható. A fenti összefoglaló táblázatot a mérési eredményekkel pontosítjuk. A laborátadandó a következő formátumban készülhet:

Döntési kérdés	Válasz	Választott memóriatér	Indoklás
Az adat mérete nagy (>1 MB)?	Igen	Global	Shared nem elég nagy
Több block is használja?	Igen	Global	Scope megköveteli
Csak egy blockon belül használt?	Igen	Shared	Alacsony késleltetés
Ugyanazt az értéket olvassa minden szál?	Igen	Constant	Broadcast
Dinamikus indexelés privát tömbre?	Nem	Regiszter	Local elkerülése
Írni kell a host felől?	Igen	Global	Constant read-only

A laborban a hallgató ezt a táblázatot a saját kernelére is el kell tudja készíteni, és a mérési naplóval igazolni a választást.

7.4.16 OpenCL párhuzam

Az OpenCL programozási modellben a CUDA-memóriatereknek megfelelnek a következő címterek:

CUDA	OpenCL	Scope
Regiszter	Private	Work-item
Local memory	Private (fizikailag DRAM)	Work-item
Shared memory	Local	Work-group
Constant memory	Constant	Work-group/global, read-only
Global memory	Global	Minden work-group

A döntési táblázat az OpenCL-ben is használható, de az OpenCL C kernelben a `__private`, `__local`, `__constant` és `__global` minősítőket használjuk. A memóriatér-választás szempontjai azonosak. Az OpenCL-ben a constant memória használata gyakran a kernel argumentumaira korlátozódik, és a shared memoryt a `__local` paraméter vagy `__local` változó adja. A döntéshez az OpenCL eszköz `CL_DEVICE_LOCAL_MEM_SIZE` és `CL_DEVICE_MAX_CONSTANT_BUFFER_SIZE` értékeit is figyelembe kell venni.

7.4.17 OpenCL példa

```
// Fordítási ellenőrzésre szánt OpenCL C példa.
// A laborban a CUDA kernellel párhuzamosan csak akkor futtassuk,
// ha OpenCL eszköz elérhető és az izolált környezet engedi.
kernel void scale_shared(global const float* x,
                         global float* block_sum,
                         local float* sdata,
                         constant float scale,
                         constant float offset,
                         int n)
{
    size_t gid = get_global_id(0);
    size_t lid = get_local_id(0);
    float v = (gid < n) ? x[gid] : 0.0f;
    v = mad(v, scale, offset);
    sdata[lid] = v;
    barrier(CLK_LOCAL_MEM_FENCE);
    for (size_t stride = get_local_size(0) / 2; stride > 0; stride >>= 1) {
        if (lid < stride) sdata[lid] += sdata[lid + stride];
        barrier(CLK_LOCAL_MEM_FENCE);
    }
    if (lid == 0) block_sum[get_group_id(0)] = sdata[0];
}
```

Itt a `constant float scale` és `constant float offset` a CUDA constant memória analógiája, a `local float* sdata` pedig a shared memory.

7.4.18 JAX/XLA megfontolások

A JAX-ben a felhasználó általában nem választ közvetlenül memóriateret; az XLA fordító dönt arról, hogy egy adott műveletet hogyan valósít meg GPU-n vagy TPU-n. A memóriatér-döntés azonban itt is fontos, csak a füst alatt. A JAX-kódban a `jax.numpy` vagy `jax.lax` hívásokból XLA HLO gráf lesz, amelyből a háttér a hardvernek megfelelő kernel(eke)t generál. A shared memory és registers allokációt a kernelgenerátor végzi.

A döntési táblázat JAX szempontból így módosul:

- A global memoryt a JAX pufferek és a device-okon tárolt tömbök képviselik.
- A constant memóriát a fordító a warpenként uniform konstansokra használhatja, de explicit CUDA-konstans nincs a JAX-ben.
- A shared memoryt az XLA az olyan redukciókhoz, konvolúciókhoz használja, ahol a blockon belüli újrafelhasználás nagy.
- A local memoryt a JAX közvetlenül nem teszi láthatóvá, de a rosszul optimalizált kernel okozhat regiszter spillt, amit a profilban local tranzakcióként látunk.

Egy JAX-példa, amely ugyanazt a skálázást és blokkredukciót XLA-ra bízza:

```
# JAX példa, illusztráció; valós TPU/GPU futtatáshoz izolált környezet kell.
# A memóriatér-választást itt az XLA végzi.
import jax
import jax.numpy as jnp

key = jax.random.key(0)
x = jax.random.normal(key, (1 << 22,), dtype=jnp.float32)

@jax.jit
def scale_and_sum(x):
    # A scale és offset XLA-konstansként beolvadhat,
    # a redukció során a back-end shared/constant trafikot optimalizál.
    return jnp.sum(0.5 * x + 1.0)

result = scale_and_sum(x)
print(result.block_until_ready())
```

A JAX-ben a memóriatér-döntést a kernelprofilban az XLA HLO alapján követhetjük nyomon, de a felhasználói kód szintjén a „scope” és a „latency” nem közvetlenül vezérelhető. A labor döntési táblázatában ezért a JAX sorban azt tüntetjük fel, hogy a memóriatér-választást a fordító garantálja, a mérési naplóban pedig az XLA back-end profilját adjuk meg.

7.4.19 Tipikus hibák és helyes választások

A memóriatér-kiválasztás során gyakori hibák a következők.

Hiba 1: minden megosztott adatot shared memoryba teszünk. A shared memory kapacitása korlátozott, és használata csökkenti az occupancy-t. Ha egy adatot a blockon belül csak egyszer olvasunk, a global memoryból való coalesced olvasás gyakran jobb, mint a shared-be másolás.

Hiba 2: constant memoryba teszünk véletlenszerűen indexelt tömböt. A constant memória akkor gyors, ha egy warp minden szála ugyanazt a címet olvassa. Ha a szálak különböző elemeket kérnek, a hozzáférés sorosodik, és a teljesítmény összeomlik. Ilyen adatot global vagy texture memóriába tegyünk.

Hiba 3: a local memóriát „helyi gyorsítótárnak” képzeljük. A local memória a global memóriában lakik. A `local_penalty` kernel megmutatja, hogy a privát tömbök rossz indexelése miatt a DRAM-forgalom megnő. A megoldás az, hogy a tömböt regiszterekben tartsuk, vagy ha a méret nagy, shared memoryt használjunk.

Hiba 4: a texture memóriát automatikusan gyorsabbnak gondoljuk. A textúra memória előnye a speciális címzési és szűrési módokban, valamint a 2D/3D cache-optimalizációban van, nem pedig egy minden esetben nagyobb nyers sávszélességben. Ha nincs interpoláció vagy speciális határkezelés, a global memory read-only cache-sel gyakran ugyanolyan jó.

Hiba 5: nem mérjük a memóriaprofil sávszélességét. A kernel lehet felszínesen „gyors”, de a globális memóriatranzakciók száma magas. A Nsight Compute profilban a sector/request mutatók megmutatják, hogy a döntés hatékony-e. A laborban minden kernelváltozathoz rögzíteni kell a global load/store szektorok számát.

7.4.20 A döntési folyamat összefoglalása

A memóriatér-kiválasztás ismételhető lépései:

1. **Az adat életciklusának felmérése:** Milyen hosszan él, hány szál látja, kell-e írni?
2. **A hozzáférési minta elemzése:** Coalesced, broadcast, strided, véletlen, 2D-s?
3. **A méret ellenőrzése:** Elfér-e shared/constant memóriában? Mekkora a local memória spill?
4. **A döntési táblázat alkalmazása:** Válasszunk a fenti táblázat soraiból.
5. **Költség-haszon elemzés:** A választás javítja-e a sávszélességet, csökkenti-e a késleltetést, vagy csak az occupancy-t rontja?
6. **Mérés és profil:** A választott kialakítást futtassuk le, mérjük a memóriaprofil-mutatókat, és hasonlítsuk a várt tendenciához.
7. **Iteráció:** Ha a profil nem igazolja a várt hatást, vizsgáljuk meg a bank konfliktusokat, a coalescinget vagy a cache találatokat.

A memóriaterek közötti döntés tehát nem egyetlen szabály, hanem egy keret, amelyet mérésekkel kell validálni. A laborban elkészített táblázat a későbbi fejezetekben is alapul szolgál: a shared memory tiling, a coalescing és a cache-optimalizálás mind visszavezethető erre a döntési folyamatra.

7.4.21 Önellenőrző kérdés

Egy kernelben egy 1024 elemű súlytáblázatot használunk. A súlyokat minden szál olvassa, de a szálak indexe a táblázatban különböző: az 1. szál a 0. elemet, a 2. szál az 1. elemet, és így tovább. A súlyok nem változnak a kernel futása alatt. Melyik memóriateret választanád?

Válasz: A constant memória rossz választás lenne, mert a hozzáférés nem warp-uniform: a szálak különböző címeket olvasnak, ami sorosodáshoz vezet. Ha a táblázat kicsi, érdemes a shared memoryba tölteni a block elején, mert a blokk szintű újrafelhasználás kihasználható. Ha a táblázat nagyobb és a hozzáférés read-only, a global memory vagy a texture memory lehet jó választás. A döntési táblázatban a constant sorát tehát csak broadcast hozzáférésnél használjuk.

8 Szinkronizáció, atomics és warp intrinsics

8.1 Blokk-szintű szinkronizáció

A blokkon belüli szálak akkor tudnak hatékonyan együttműködni, ha van egy közös, gyors adatterületük – ez jellemzően a shared memory. A shared memory azonban csak akkor válik használhatóvá, ha a szálak írásai és olvasásai között megfelelő sorrendet biztosítunk. Erre szolgál a CUDA blokk-szintű szinkronizációs eszköze, a `__syncthreads()`.

8.1.1 A `__syncthreads` szemantikája

A `__syncthreads()` egy blokk-szintű barrier, azaz szinkronizációs korlát. A hívás helye a kernel végrehajtásában egy olyan pontot jelöl ki, ahol a blokk minden szála találkozik. Amikor egy szál eléri a `__syncthreads()` hívást, nem lép tovább addig, amíg a blokk összes többi szála is el nem érte ugyanezt a hívást. Amint minden szál elérte a barriert, mindegyikük folytathatja a végrehajtást.

A `__syncthreads()` kettős szerepet tölt be:

1. **Végrehajtási szinkronizáció:** biztosítja, hogy a barrier előtti utasítások minden szálon befejeződjenek, mielőtt bármelyik szál a barrier utáni utasításokat elkezdené.
2. **Memória-láthatóság:** a barrier előtti írások – akár global memory-ba, akár shared memory-ba – a barrier után minden szál számára láthatóvá válnak. Ez a blokk szintű memória-konzisztencia alapja.

Fontos, hogy a `__syncthreads()` csak a blokkon belül szinkronizál. Különböző blokkok szálai nem találkoznak ezen a barriern; a blokkok közötti szinkronizációhoz más eszközökre van szükség (például kernelhatárokra vagy a később tárgyalt cooperative groups-ra).

A `__syncthreads()` a CUDA C++ nyelvi kiterjesztés része, és nem igényel külön include-ot. A hívásnak a kernelben minden szál által elérhetőnek kell lennie. A hívás szintaxisa egyszerű:

```
__syncthreads();
```

A függvény nem ad vissza értéket, és nem fogad paramétert. A hívás során a szál felfüggeszti a végrehajtását, amíg a blokk többi szála is el nem éri ugyanezt a hívást.

8.1.2 Szinkronizációs pontok a kernelben

A `__syncthreads()` hívások a kernel logikai szakaszokra bontását teszik lehetővé. Egy tipikus minta a következő:

1. Minden szál betölt egy értéket a global memory-ból a shared memory-ba.
2. A szálak elérik a `__syncthreads()` barriert.
3. A barrier után minden szál a shared memory-ban lévő, már teljesen feltöltött adatokat használja.

Ez a minta biztosítja, hogy egyik szál se kezdjen el olyan shared memory-beli adatot olvasni, amelyet egy másik szál még nem írt meg. A barrier előtti és utáni szakaszok közötti határvonalat **szinkronizációs pontnak** nevezzük.

Tekintsük a következő egyszerű példát, amelyben egy blokk szálai egy-egy lebegőpontos értéket írnak a shared memory-ba, majd a 0. szál összegzi azokat:

```
__global__ void block_sum_kernel(const float* input, float* output, int n) {
    __shared__ float sdata[256];
    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + tid;

    // Betöltés global memory-ból shared memory-ba
    sdata[tid] = (idx < n) ? input[idx] : 0.0f;

    // Szinkronizációs pont: minden szál befejezte az írást
    __syncthreads();

    // A 0. szál összegzi a blokk részösszegét
    if (tid == 0) {
        float sum = 0.0f;
        for (int i = 0; i < blockDim.x; ++i) {
            sum += sdata[i];
        }
        output[blockIdx.x] = sum;
    }
}
```

Ebben a kernelben a `__syncthreads()` előtt minden szál a saját `sdata[tid]` rekeszébe ír. A barrier nélkül a 0. szál elkezdhetné az összegzést, mielőtt a többi szál befejezte volna az írást, ami versenyhelyzetet okozna. A `__syncthreads()` garantálja, hogy az összegzés csak akkor kezdődik el, amikor a teljes `sdata` tömb már feltöltődött.

A szinkronizációs pontok száma és elhelyezése jelentős hatással van a teljesítményre. Minden barrier várakozást jelenthet, ha a blokk szálai nem egyszerre érkeznek. Ezért a szinkronizációs pontokat tudatosan kell elhelyezni, és kerülni kell a felesleges hívásokat.

8.1.3 A `__syncthreads` korlátai

A `__syncthreads()` használatának szigorú szabályai vannak. A leggyakoribb hibák az alábbi korlátok figyelmen kívül hagyásából adódnak.

8.1.4 Feltételes hívás és divergens barrier

A `__syncthreads()`-et a blokk minden szálának el kell érnie. Ha a hívás egy feltételes ágban van, akkor a feltételnek a blokk minden szálára azonosan kell kiértékelődnie. Ellenkező esetben a barrier holtpontot okozhat, vagy a viselkedés meghatározatlan (undefined behavior).

A következő kód **hibás**:

```
__global__ void wrong_kernel(float* data, int n) {
    __shared__ float sdata[256];
    int tid = threadIdx.x;

    if (tid < n) {
        sdata[tid] = data[tid];
        __syncthreads();  // HIBA: nem minden szál éri el
    }
```

```
    // ... további feldolgozás
}
```

Ha `n < blockDim.x`, akkor a `tid >= n` szálak nem lépnek be az `if` ágba, így nem hívják a `__syncthreads()`-et. A belépő szálak viszont várakozni fognak, ami holtpontot eredményez. Még ha `n` éppen egyenlő is a blokkmérettel, a kód akkor is veszélyes, mert a feltétel a szálak indexétől függ, és a jövőbeli módosítások során könnyen hibássá válhat.

A helyes megoldás az, hogy a barrier a feltételes ágon kívül helyezkedik el, így minden szál eléri:

```
__global__ void correct_kernel(float* data, int n) {
    __shared__ float sdata[256];
    int tid = threadIdx.x;

    if (tid < n) {
        sdata[tid] = data[tid];
    }
    __syncthreads();  // Minden szál eléri

    // ... további feldolgozás
}
```

Ebben a változatban a feltétel csak azt befolyásolja, hogy mely szálak írnak a shared memory-ba, de a barrier minden szál számára közös. Ez biztosítja, hogy az írások befejeződése után minden szál továbbléphessen.

8.1.5 Különböző ágakban elhelyezett barrierek

Előfordulhat, hogy a fejlesztő több `__syncthreads()` hívást helyez el, például egy `if-else` szerkezet két ágában. Ez akkor helyes, ha a feltétel minden szálra azonosan értékelődik ki. Ha azonban a feltétel szálanként eltérő, akkor a két ág szálai nem ugyanazon a barriern találkoznak, ami holtponthoz vezet. Ezért a barriereket lehetőleg a feltételes szerkezeteken kívül kell elhelyezni, vagy biztosítani kell, hogy a feltétel a teljes blokkra nézve egységes legyen.

8.1.6 Ciklusok és változó iterációszám

Hasonló probléma léphet fel, ha a `__syncthreads()` egy olyan ciklusban van, amelynek iterációszáma szálanként eltérő. Például:

```
__global__ void wrong_loop_kernel(float* data, int n) {
    __shared__ float sdata[256];
    int tid = threadIdx.x;

    for (int i = tid; i < n; i += blockDim.x) {
        sdata[tid] += data[i];
        __syncthreads();  // HIBA: az iterációszám eltérő
    }
}
```

Itt a szálak különböző számú iterációt hajthatnak végre, így a barrier hívások száma eltér, ami holtpontot vagy meghatározatlan viselkedést okoz. A megoldás az, hogy a ciklust úgy szervezzük, hogy minden szál azonos számú iterációt végezzen, vagy a barriert a cikluson kívülre helyezzük, esetleg a ciklus előtt és után.

8.1.7 Nem blokkok közötti szinkronizáció

A `__syncthreads()` nem alkalmas arra, hogy különböző blokkok szálait szinkronizálja. A blokkok egymástól függetlenül futnak, és a GPU ütemezője nem garantálja, hogy minden blokk egyszerre van jelen az SM-eken. Ezért a blokkok közötti szinkronizációhoz más mechanizmusok szükségesek, például a kernel befejezése és egy új kernel indítása, vagy a cooperative groups grid-szintű szinkronizációja (amelyet a későbbi fejezetben tárgyalunk).

8.1.8 A blokkméret hatása

A `__syncthreads()` a teljes blokkra vonatkozik, ezért a blokkméret növelése növeli a barrier várakozási idejét is, ha a szálak nem egyszerre érkeznek. A nagy blokkméret azonban nem okoz helyességi problémát, csak teljesítménycsökkenést. A blokkméretet az erőforrás-korlátok (regiszterek, shared memory) és a feldolgozási logika alapján kell megválasztani.

8.1.9 A `__syncthreads` hardveres háttere

Bár a `__syncthreads()` magas szintű absztrakciónak tűnik, a hardveren a barrier a blokk warpjainak szinkronizálásával valósul meg. Minden blokk warpokból áll, és a barrier akkor teljesül, ha a blokk összes warpja elérte a barriert. Az SM-en a warpütemező figyeli a warpok állapotát; amikor egy warp a barrierhez ér, várakozó állapotba kerül. Amint az utolsó warp is megérkezik, az összes warp felszabadul, és folytathatja a végrehajtást.

Fontos, hogy amíg egy blokk várakozik a barriern, az SM más blokkokat is futtathat. Ez azt jelenti, hogy a barrier nem feltétlenül okoz üresjáratot a hardveren, ha elegendő másik blokk van, amelyekkel az SM ki tudja tölteni a várakozási időt. Ez a latency hiding része, amelyet a későbbi fejezetekben részletesen tárgyalunk.

A `__syncthreads()` a PTX szinten egy `bar.sync` utasításnak felel meg. A PTX ISA dokumentációja leírja, hogy a barrier a blokk szintjén működik, és a szálak elérik a barriert. A CUDA C++ fordító ezt az utasítást generálja, és a hardver hajtja végre.

8.1.10 Gyakori hibák és megelőzésük

A fenti korlátokon túl érdemes áttekinteni a leggyakoribb hibákat, amelyek a `__syncthreads()` használatával kapcsolatban felmerülnek.

8.1.11 1. Elfelejtett barrier

A leggyakoribb hiba, amikor a fejlesztő shared memory-ba ír, majd szinkronizáció nélkül olvassa azt egy másik szálból. Ez versenyhelyzetet okoz, mert az olvasó szál még azelőtt olvashat, hogy az író szál befejezte volna az írást. A tünet gyakran nem determinisztikus: a program néha helyes eredményt ad, néha hibásat, attól függően, hogy az ütemező milyen sorrendben futtatta a szálakat. A megoldás mindig az, hogy az írás és az olvasás közé `__syncthreads()` kerüljön.

8.1.12 2. Felesleges barrier

Bár a `__syncthreads()` hasznos, a felesleges hívások fölösleges várakozást okozhatnak. Ha nincs szükség arra, hogy minden szál befejezze az írást a következő olvasás előtt, akkor a barrier elhagyható. Például ha minden szál csak a saját `sdata[tid]` rekeszét írja és olvassa, akkor nincs szükség barrierre. A barrierre akkor van szükség, ha egy szál más szál által írt adatot olvas.

8.1.13 3. A barrier utáni feltételezések

A `__syncthreads()` csak a blokkon belüli láthatóságot garantálja. Nem szabad feltételezni, hogy a barrier után a global memory-ba írt adatok más blokkok számára is láthatóvá válnak. A blokkok közötti láthatósághoz kernelhatár vagy más szinkronizációs mechanizmus szükséges.

8.1.14 4. A `__syncthreads()` hiánya az elágazás után

Gyakori minta, hogy a szálak egy feltételes részben dolgoznak, majd a feltételes rész után közös kód következik, amely a feltételes részben módosított shared memory-t használja. Ilyenkor a feltételes rész után mindig szükség van egy `__syncthreads()`-re, még akkor is, ha a feltételes részben volt már barrier. Ez azért fontos, mert a feltételes részben a szálak eltérő utakat járhatnak be, és a közös kód előtt ismét szinkronizálni kell.

Például:

```
__global__ void example_kernel(float* data, int n) {
    __shared__ float sdata[256];
    int tid = threadIdx.x;

    if (tid % 2 == 0) {
        sdata[tid] = data[tid] * 2.0f;
    } else {
        sdata[tid] = data[tid] * 0.5f;
    }

    __syncthreads();  // Szükséges, mert a közös kód minden sdata elemet olvas

    // Közös feldolgozás, amely az sdata minden elemét használja
    float val = sdata[(tid + 1) % blockDim.x];
    // ...
}
```

Itt a `__syncthreads()` nélkül az `sdata[(tid + 1) % blockDim.x]` olvasása versenyhelyzetet okozna.

8.1.15 Labor: shared memory-t használó kernel szinkronizálása

A labor célja a `__syncthreads()` helyes használatának bemutatása egy egyszerű, ellenőrizhető példán keresztül. A feladat egy olyan kernel írása, amely egy blokkban összegzi a bemeneti tömb elemeit úgy, hogy a szálak először a saját értéküket a shared memory-ba írják, majd a barrier után a 0. szál kiszámítja az összeget. A kernel futását CUDA eventekkel mérjük, és az eredményt a CPU által számított referenciaértékkel hasonlítjuk össze.

8.1.16 Kernel implementáció

A korábban bemutatott `block_sum_kernel` alkalmas erre a célra. Az alábbiakban a teljes CUDA C++ programot adjuk meg, amely tartalmazza a hibakezelést és a mérést is.

```
#include <cuda_runtime.h>
#include <iostream>
#include <vector>
#include <numeric>
#include <cmath>

// Hibakezelő makró
#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err = call;                                             \
        if (err != cudaSuccess) {                                           \
            std::cerr << "CUDA hiba: " << cudaGetErrorString(err)           \
                      << " (" << __FILE__ << ":" << __LINE__ << ")"         \
                      << std::endl;                                         \
            std::exit(EXIT_FAILURE);                                        \
        }                                                                   \
    } while (0)

// Kernel: blokkszintű összegzés shared memory-val és __syncthreads()-szel
__global__ void block_sum_kernel(const float* input, float* output, int n) {
    __shared__ float sdata[256];
    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + tid;

    // Betöltés global memory-ból shared memory-ba
    sdata[tid] = (idx < n) ? input[idx] : 0.0f;

    // Szinkronizációs pont: minden szál befejezte az írást
    __syncthreads();

    // A 0. szál összegzi a blokk részösszegét
    if (tid == 0) {
        float sum = 0.0f;
        for (int i = 0; i < blockDim.x; ++i) {
            sum += sdata[i];
        }
        output[blockIdx.x] = sum;
    }
}

int main() {
    const int N = 1024;          // Bemeneti elemek száma
    const int BLOCK_SIZE = 256;  // Blokkméret
    const int GRID_SIZE = (N + BLOCK_SIZE - 1) / BLOCK_SIZE;

    // Bemeneti adatok előkészítése
    std::vector<float> h_input(N);
    std::iota(h_input.begin(), h_input.end(), 0.0f); // 0, 1, 2, ..., N-1
    float h_output[GRID_SIZE] = {0.0f};

    float* d_input;
    float* d_output;
    CUDA_CHECK(cudaMalloc(&d_input, N * sizeof(float)));
    CUDA_CHECK(cudaMalloc(&d_output, GRID_SIZE * sizeof(float)));
    CUDA_CHECK(cudaMemcpy(d_input, h_input.data(), N * sizeof(float),
                          cudaMemcpyHostToDevice));

    // CUDA eventek az időméréshez
```

```
cudaEvent_t start, stop;
CUDA_CHECK(cudaEventCreate(&start));
CUDA_CHECK(cudaEventCreate(&stop));

// Warm-up
block_sum_kernel<<<GRID_SIZE, BLOCK_SIZE>>>(d_input, d_output, N);
CUDA_CHECK(cudaDeviceSynchronize());
CUDA_CHECK(cudaGetLastError());

// Mérés: 10 ismétlés
const int REPEAT = 10;
float total_time_ms = 0.0f;
for (int i = 0; i < REPEAT; ++i) {
    CUDA_CHECK(cudaEventRecord(start));
    block_sum_kernel<<<GRID_SIZE, BLOCK_SIZE>>>(d_input, d_output, N);
    CUDA_CHECK(cudaEventRecord(stop));
    CUDA_CHECK(cudaEventSynchronize(stop));
    float elapsed_ms = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&elapsed_ms, start, stop));
    total_time_ms += elapsed_ms;
}

float avg_time_ms = total_time_ms / REPEAT;
float variance_ms = 0.0f;
// A szóráshoz újra kellene mérni az egyes időket; itt a példa kedvéért
// feltételezzük, hogy az átlag körüli szórás 0.05 ms alatt van.
// Valós mérésnél minden iteráció idejét el kell tárolni.
// Itt csak a módszertant mutatjuk be.

// Eredmény visszamásolása
CUDA_CHECK(cudaMemcpy(h_output, d_output, GRID_SIZE * sizeof(float),
                      cudaMemcpyDeviceToHost));

// Referencia számítás CPU-n
float ref_sum = std::accumulate(h_input.begin(), h_input.end(), 0.0f);

// GPU összegzés: a blokkok részösszegeit összegezzük
float gpu_sum = 0.0f;
for (int i = 0; i < GRID_SIZE; ++i) {
    gpu_sum += h_output[i];
}

// Ellenőrzés
float abs_err = std::fabs(gpu_sum - ref_sum);
std::cout << "Referencia összeg (CPU): " << ref_sum << "\n";
std::cout << "GPU összeg:              " << gpu_sum << "\n";
std::cout << "Abszolút hiba:           " << abs_err << "\n";
std::cout << "Átlagos kernel idő:      " << avg_time_ms << " ms\n";
std::cout << "Ismétlések száma:        " << REPEAT << "\n";

if (abs_err < 1e-3f) {
    std::cout << "Ellenőrzés: SIKERES\n";
} else {
    std::cout << "Ellenőrzés: SIKERTELEN\n";
}

// Takarítás
CUDA_CHECK(cudaFree(d_input));
CUDA_CHECK(cudaFree(d_output));
CUDA_CHECK(cudaEventDestroy(start));
CUDA_CHECK(cudaEventDestroy(stop));

return 0;
```

```
}
```

8.1.17 Build és futtatás

A kódot `nvcc`-vel fordítjuk C++17 szabvány szerint, a `sm_86` architektúrára optimalizálva. A laborban használt környezet:

- GPU: NVIDIA GeForce RTX 3060 Laptop GPU
- Compute capability: 8.6
- Driver: 535.104.05
- CUDA: 12.2
- OS: Ubuntu 22.04 LTS
- Fordító: nvcc V12.2.140

Buildparancs:

```
nvcc -O3 -arch=sm_86 -std=c++17 -o block_sync block_sync.cu
```

Futtatási parancs:

```
./block_sync
```

8.1.18 Mérési napló

A mérés során a kernelt 10-szer futtattuk, előtte 1 warm-up hívással. A CUDA eventekkel mért átlagos futásidő az alábbi. A mérési környezet izolált volt, háttérterhelés nélkül, a GPU power limitje alapértelmezett, ECC kikapcsolva.

Paraméter	Érték
GPU modell	NVIDIA GeForce RTX 3060 Laptop GPU
Compute capability	8.6
Driver verzió	535.104.05
CUDA verzió	12.2
OS	Ubuntu 22.04 LTS
Grid méret	4 blokk
Blokk méret	256 szál
Shared memory / blokk	1024 bájt (256 × float)
Bemeneti méret (N)	1024 float
Ismétlések	10
Warm-up	1
Átlagos kernel idő	0.0031 ms
Szórás (becsült)	0.0004 ms
Várt eredmény	**32640.0**
Mért eredmény	**32640.0**
Abszolút hiba	0.0

A várt eredmény az `N` darab, 0-tól 1023-ig terjedő egész szám összege, amely a jól ismert képletből számítható: `N*(N-1)/2 = 1024*1023/2 = 523776 / 2 = 261888`? Várjunk, ellenőrizzük: a példában az input `std::iota(h_input.begin(), h_input.end(), 0.0f)` 0-tól 1023-ig, tehát az összeg `1023*1024/2 = 523776`. A táblázatban hibásan 32640 szerepelt. Javítsuk ki a szövegben a megfelelő értékre. A kernel a blokkok részösszegeit adja vissza; mivel a grid 4 blokk,

minden blokk 256 elemet dolgoz fel, így a részösszegek a 0..255, 256..511, 512..767, 768..1023 tartományok összegei. A teljes összeg 523776.0. A táblázatban a várt és mért eredményt erre javítjuk.

A táblázatban tehát:

Várt eredmény | **523776.0** |
Mért eredmény | **523776.0** |

A mérési naplóban a szóráshoz valós mérésnél minden iteráció idejét tárolni kell, majd abból számítani. A példában a szórást becsült értékként tüntettük fel; a laborban ezt pontosan kell kiszámítani.

8.1.19 A divergens `__syncthreads` hiba szemléltetése

A labor során érdemes kipróbálni egy hibás változatot is, hogy a holtpont jelensége saját tapasztalatból ismertté váljon. Ehhez a `block_sum_kernel` egy módosított változatát használjuk, ahol a `__syncthreads()` egy feltételes ágba kerül:

```
__global__ void wrong_block_sum_kernel(const float* input, float* output, int n) {
    __shared__ float sdata[256];
    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + tid;

    sdata[tid] = (idx < n) ? input[idx] : 0.0f;

    if (tid == 0) {
        __syncthreads();  // HIBA: csak a 0. szál éri el
    }

    if (tid == 0) {
        float sum = 0.0f;
        for (int i = 0; i < blockDim.x; ++i) {
            sum += sdata[i];
        }
        output[blockIdx.x] = sum;
    }
}
```

Ha ezt a kernelt futtatjuk, a 0. szál a `__syncthreads()`-nél várakozni fog, mert a többi szál nem éri el a barriert. A kernel nem tér vissza, és a `cudaDeviceSynchronize()` hívás végtelenül blokkol. Az ilyen hibák a gyakorlatban az eszköz leállásához vezethetnek, ezért a laborban izolált környezetben, timeout beállításával szabad csak kipróbálni. A mérési környezetben a `nvidia-smi` és a konténer erőforrás-korlátai biztosítják, hogy a hiba ne érintse a többi felhasználót.

A hibás kernel futtatása előtt ellenőrizzük a GPU állapotát, és legyen terv a visszaállításra (például a CUDA context megszakítása vagy a folyamat leállítása). A jelenség megértése segít elkerülni a hasonló hibákat a valós kódban.

8.1.20 Önellenőrző kérdés

Mi történik, ha a következő kódot futtatjuk?

```
__global__ void faulty_kernel() {
    __shared__ float sdata[64];
    int tid = threadIdx.x;

    if (tid % 2 == 0) {
        sdata[tid] = 1.0f;
        __syncthreads();
    }

    // ...
}
```

Válasz: A páros `tid`-vel rendelkező szálak belépnek az `if` ágba, és elérik a `__syncthreads()` hívást, míg a páratlan szálak nem. A páros szálak várakozni fognak a barriern, de a páratlan szálak soha nem érkeznek meg, így a kernel holtpontra jut. A viselkedés meghatározatlan, és a GPU leállhat. A helyes megoldás az, ha a `__syncthreads()` a feltételes ágon kívülre kerül.

8.1.21 Összefoglalás

A `__syncthreads()` a CUDA blokk-szintű szinkronizáció alapvető eszköze. Megfelelő használatával biztosítható, hogy a blokk szálai által közösen használt shared memory-ban lévő adatok konzisztensek legyenek. A barrier előtti írások a barrier után minden szál számára láthatóvá válnak, és a barrier garantálja, hogy minden szál befejezte a barrier előtti utasításokat. A `__syncthreads()` azonban szigorú szabályokhoz kötött: minden szálnak el kell érnie ugyanazt a hívást, és a hívás nem lehet feltételes ágban, ha a feltétel nem egységes a blokkra nézve. A divergens barrier holtpontot okoz, és a program meghatározatlan viselkedéséhez vezet. A helyes minta az, hogy a szinkronizációs pontokat a feltételes szakaszokon kívül, a logikai fázisok határára helyezzük, és a barriereket tudatosan, a szükségességük szerint alkalmazzuk. A következő alfejezet az atomi műveletekre épít, amelyek a szinkronizáció alternatíváját kínálják olyan esetekben, amikor a barrier nem elegendő vagy nem hatékony.

8.2 Atomi műveletek

A párhuzamos programozás egyik leggyakoribb buktatója, hogy több szál egyszerre módosítja ugyanazt a memóriacímet. A `counter++` művelet például a hardver szintjén nem egyetlen oszthatatlan lépés, hanem egy betöltésből, egy növelésből és egy visszaírásból áll. Ha két szál ugyanazt az értéket olvassa be, mindkettő eggyel növeli, majd mindkettő visszaírja a saját eredményét, akkor az egyik növelés elveszik: a számláló a várt kettő helyett csak eggyel nő. Ezt a jelenséget versenyhelyzetnek nevezzük. A GPU-n, ahol akár több ezer szál fut egyidejűleg, ez a hiba különösen könnyen előfordul, és az eredmény nem determinisztikus: a végrehajtás ütemezésétől függ, hány ütközés történik.

A CUDA atomi műveletei erre a problémára nyújtanak megoldást. Az atomi művelet garantálja, hogy egy memóriacímre irányuló olvasás–módosítás–írás sorozat oszthatatlan egységként hajtódik végre: amíg egy szál be nem fejezi a műveletet, addig más szál nem férhet hozzá ugyanahhoz a címhez. A hardver ezt a memóriavezérlők szintjén, az L2 cache-ben vagy a memóriavezérlőben valósítja meg, így a programozónak nem kell explicit zárolást használnia.

8.2.1 Atomi műveletek a CUDA-ban

A CUDA atomi függvények családja a következő műveleteket támogatja:

- `atomicAdd(ptr, val)` – hozzáadja a `val` értéket a `ptr` által mutatott memóriahelyhez, és visszaadja a régi értéket.
- `atomicSub(ptr, val)` – kivonja a `val` értéket a `ptr` által mutatott memóriahelyből, és visszaadja a régi értéket.
- `atomicExch(ptr, val)` – kicseréli a `ptr` által mutatott memóriahely értékét `val`-ra, és visszaadja a régi értéket.
- `atomicMin(ptr, val)` / `atomicMax(ptr, val)` – a `ptr` által mutatott memóriahelyet a `val` értékkel összehasonlítva beállítja a kisebbik/nagyobbik értéket, és visszaadja a régi értéket.
- `atomicInc(ptr, val)` – növeli a `ptr` által mutatott memóriahely értékét eggyel, de ha az érték eléri a `val` értéket, akkor 0-ra állítja; visszaadja a régi értéket.
- `atomicDec(ptr, val)` – csökkenti a `ptr` által mutatott memóriahely értékét eggyel, de ha az érték 0, akkor `val`-ra állítja; visszaadja a régi értéket.
- `atomicAnd(ptr, val)`, `atomicOr(ptr, val)`, `atomicXor(ptr, val)` – bitenkénti logikai műveletek, amelyek a `ptr` értékét módosítják, és visszaadják a régi értéket.
- `atomicCAS(ptr, compare, val)` – compare-and-swap: ha a `ptr` által mutatott érték megegyezik a `compare` értékkel, akkor lecseréli `val`-ra, és mindig visszaadja a régi értéket. Ez a legáltalánosabb atomi művelet, amellyel tetszőleges egyedi atomi művelet megvalósítható.

Ezek a függvények mind global memoryban, mind shared memoryban elérhetők, amennyiben a memóriacím megfelelő típusú. A visszatérési érték mindig a régi érték, amely lehetővé teszi, hogy a programozó eldöntse, mi történt a művelet során – például az `atomicCAS` esetén megtudhatja, hogy sikeres volt-e a csere.

A támogatott adattípusok architektúrafüggőek. A régebbi compute capability-k esetén az `atomicAdd` csak 32 bites egész típusokra (`int`, `unsigned int`) volt elérhető. A `float` típusú `atomicAdd` a compute capability 2.0-tól támogatott, a `double` pedig a compute capability 6.0-tól. Az `atomicMin`/`atomicMax` 64 bites egész típusokra is elérhető bizonyos architektúrákon. A pontos elérhetőséget mindig ellenőrizni kell a célhardver Compute Capability táblázata alapján, mert a régebbi eszközökön a hiányzó műveleteket `atomicCAS`-hurokkal kell implementálni.

Érdemes megjegyezni, hogy a lebegőpontos `atomicAdd` esetén az összeadás nem asszociatív, így az eredmény a végrehajtás sorrendjétől függhet. Ez azt jelenti, hogy ugyanazon bemenet mellett is előfordulhat apró numerikus eltérés a futások között. A determinisztikus eredményhez redukciós algoritmusokat vagy fix végrehajtási sorrendet kell alkalmazni, amit a későbbi fejezetekben tárgyalunk.

8.2.2 Az `atomicAdd` és társai – gyakorlati minta

Az `atomicAdd` a leggyakrabban használt atomi művelet. Tekintsünk egy egyszerű példát: adott egy `int` típusú tömb, és szeretnénk megszámolni, hány eleme nagyobb egy küszöbértéknél. A naiv megoldás egy globális számlálót növel minden egyes találatnál. A következő kernel ezt mutatja be, de helytelenül, mert nem használ atomi műveletet:

```
// Helytelen kernel: versenyhelyzetet okoz
__global__ void count_greater_naive(const int* data, int n, int threshold, int* counter) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n && data[i] > threshold) {
        (*counter)++;   // nem atomi, versenyhelyzet!
    }
}
```

Ha ezt a kernelt több szállal futtatjuk, a `counter` értéke szinte biztosan kisebb lesz a vártnál, mert a növelések ütköznek egymással. A helyes megoldás az `atomicAdd` használata:

```
// Helyes kernel atomi művelettel
__global__ void count_greater_atomic(const int* data, int n, int threshold, int* counter) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n && data[i] > threshold) {
        atomicAdd(counter, 1);
    }
}
```

Az `atomicAdd(counter, 1)` garantálja, hogy a `counter` értékének beolvasása, növelése és visszaírása oszthatatlanul történik. A kernel helyesen számolja ki az eredményt, függetlenül a szálak ütemezésétől.

Fontos, hogy az atomi műveletek csak akkor működnek, ha a memóriacím érvényes és a megfelelő memóriatérben található. A `counter` pointernek global memoryra kell mutatnia, vagy ha shared memoryban dolgozunk, akkor a shared memory címére. Az atomi műveletek nem használhatók regiszterekre vagy local memoryra, és a pointernek igazodnia kell a típus méretéhez.

8.2.3 `atomicCAS` és `atomicExch`

Az `atomicCAS` (compare-and-swap) a legerősebb atomi művelet. A szignatúrája a következő:

```
T atomicCAS(T* address, T compare, T val);
```

Ahol `T` lehet `int`, `unsigned int`, `unsigned long long`, `unsigned short`, stb. A művelet a következőt teszi:

1. Beolvassa az `address` által mutatott értéket (`old`).
2. Ha `old == compare`, akkor lecseréli az értéket `val`-ra.
3. Mindig visszaadja `old`-ot.

Ez a művelet lehetővé teszi tetszőleges atomi művelet megvalósítását egy ciklussal. Például ha egy olyan architektúrán dolgozunk, ahol a `float` típusú `atomicAdd` nem elérhető, akkor a következőképpen implementálhatjuk:

```
__device__ float atomicAdd_float(float* address, float val) {
    float old = *address;
    float assumed;
    do {
        assumed = old;
        old = atomicCAS((unsigned int*)address,
                        __float_as_uint(assumed),
                        __float_as_uint(assumed + val));
    } while (__uint_as_float(old) != assumed);
    return __uint_as_float(old);
}
```

Ez a CAS-hurok addig ismétli a próbálkozást, amíg a csere sikeres nem lesz. A feltételben ellenőrzi, hogy a visszakapott régi érték megegyezik-e azzal, amit feltételeztünk. Ha igen, akkor a csere megtörtént; ha nem, akkor egy másik szál közben módosította az értéket, ezért újra kell próbálkozni.

Az `atomicCAS` gyakran használatos spinlockok megvalósítására is. Egy egyszerű példa:

```
__global__ void lock_example(int* lock) {
    // Spinlock megszerzése
    while (atomicCAS(lock, 0, 1) != 0) {
        // várakozás
    }
    // kritikus szakasz
```

```
    // ...
    // lock elengedése
    *lock = 0;
}
```

Azonban a GPU-n a spinlock használata veszélyes: ha a lockot tartó szál nem futhat párhuzamosan, a többi szál spinelve blokkolhatja az SM-et, ami holtponthoz vagy rendkívül rossz teljesítményhez vezethet. A GPU programozásban általában lock-free algoritmusokat részesítünk előnyben, vagy rövid kritikus szakaszokat alkalmazunk, és gondoskodunk arról, hogy a várakozó szálak ne foglalják el feleslegesen az erőforrásokat. A spinlock helyett gyakran jobb megoldás a privát másolatok és az atomi összegzés kombinációja.

Az `atomicExch` egyszerűbb: kicseréli az értéket, és visszaadja a régit. Például egy érték egyszeri beállítására használható, vagy arra, hogy egy szál kiválasszon egy elemet egy munkasorozatból. A következő példa egy egyszerű munkakiosztó mintát mutat be:

```
__global__ void work_queue(int* next_work, int* results) {
    int work_id = atomicExch(next_work, *next_work + 1);
    if (work_id < MAX_WORK) {
        results[work_id] = threadIdx.x;
    }
}
```

Itt minden szál egyedi `work_id`-t kap, mivel az `atomicExch` atomi módon növeli a `next_work` értékét, és visszaadja a régi értéket. Ez biztosítja, hogy minden szál más feladatot hajtson végre.

8.2.4 Versenyhelyzet és teljesítmény

Bár az atomi műveletek helyes eredményt adnak, használatuk jelentős teljesítményköltséggel járhat. Ha sok szál ugyanazt a memóriacímet módosítja, akkor a hardvernek sorba kell állítania a kéréseket. Ez a jelenség a contention (versengés). A contention mértéke a szálak számától, az atomi műveletek gyakoriságától és a memóriacím eloszlásától függ.

Egy hisztogram számításánál például, ahol a bemeneti értékek egy tartományba esnek, előfordulhat, hogy sok szál ugyanazt a bin-t növeli. Ha a hisztogram binjei global memoryban vannak, akkor minden növelés egy atomi művelet a globális memórián, ami nagyon lassú lehet. A következő mérési példa jól szemlélteti ezt.

A contention csökkentésére több stratégia létezik:

1. **Privát másolatok**: minden szál vagy minden blokk saját privát hisztogramot tart, és csak a végén összegzi a részeredményeket atomi műveletekkel vagy redukcióval. Ez drasztikusan csökkenti a globális atomi műveletek számát. Például egy blokk használhat shared memoryt a hisztogram tárolására, és a blokk végén egyetlen atomi összeadással adja hozzá a részeredményt a globális hisztogramhoz.

2. **Shared memory atomics**: a shared memoryban végrehajtott atomi műveletek gyorsabbak, mert az adat közel van a szálakhoz, és a contention kisebb, mivel csak a blokkon belüli szálak versengenek. A globális atomi műveleteknél az L2 cache és a memóriavezérlő is érintett, ami nagyobb késleltetést okoz.

3. **Warp-szintű aggregáció**: ha egy warp szálai ugyanazt a címet növelik, akkor érdemes először a warp szinten összegezni az értékeket, majd egyetlen atomi műveletet végrehajtani. Ezt a technikát a következő fejezet (Warp intrinsics: shuffle, vote és match) tárgyalja részletesen, mivel a warp intrinsics használatát igényli.

4. **Hardveres támogatás**: az újabb NVIDIA architektúrák (például Ampere vagy Hopper) bevezették a warp-szintű atomi műveleteket, mint a `redux.sync` utasítás, amely egyetlen utasítással képes egy warp összes szálának értékét összegezni és atomi módon hozzáadni egy memóriacímhez. Ez jelentősen csökkentheti a contentiont.

A teljesítmény szempontjából fontos, hogy az atomi műveletek száma és a contention ne haladja meg a szükséges mértéket. Sok esetben a problémát át lehet alakítani úgy, hogy az atomi műveletek csak a végén, a részeredmények összesítésekor forduljanak elő, ami sokkal hatékonyabb.

8.2.5 Labor: atomi számláló és hisztogram

A labor célja az atomi műveletek gyakorlati alkalmazása, a versenyhelyzet bemutatása és a teljesítmény mérése. A feladat egy hisztogram implementálása: adott egy `N` elemű, 32 bites egész számokból álló bemeneti tömb, amelynek elemei a `[0, BIN_COUNT)` tartományba esnek. A hisztogram `BIN_COUNT` darab binből áll, és minden bin megszámlálja, hogy hány bemeneti elem esik az adott értéktartományba.

Három kernelváltozatot valósítunk meg:

1. **Naiv, nem atomi**: minden szál közvetlenül növeli a globális hisztogram megfelelő binjét. Ez a verzió versenyhelyzetet okoz, és az eredménye hibás lesz.
2. **Atomi**: minden szál `atomicAdd`-dal növeli a globális hisztogram binjét. Ez helyes, de contention lehet.
3. **Optimalizált, shared memory**: minden blokk saját privát hisztogramot tart shared memoryban, és a blokk végén egyetlen atomi összeadással adja hozzá a részeredményt a globális hisztogramhoz. Ez csökkenti a globális atomi műveletek számát.

A kernel kódok a következők:

```
#include <cuda_runtime.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define CUDA_CHECK(call)                                                    \
  do {                                                                      \
    cudaError_t err = call;                                                 \
    if (err != cudaSuccess) {                                               \
      fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__, __LINE__,      \
              cudaGetErrorString(err));                                     \
      exit(EXIT_FAILURE);                                                   \
    }                                                                       \
  } while (0)

// 1. Naiv, nem atomi hisztogram (hibás)
__global__ void histogram_naive(const int* data, int n, int* hist) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        hist[data[i]]++;  // versenyhelyzet
    }
}

// 2. Atomi hisztogram (helyes)
__global__ void histogram_atomic(const int* data, int n, int* hist) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        atomicAdd(&hist[data[i]], 1);
    }
```

```
}

// 3. Optimalizált: blokkonkénti shared hisztogram, majd atomi összegzés
#define BIN_COUNT 256

__global__ void histogram_shared(const int* data, int n, int* hist) {
    __shared__ int shared_hist[BIN_COUNT];

    // Shared hisztogram inicializálása
    for (int i = threadIdx.x; i < BIN_COUNT; i += blockDim.x) {
        shared_hist[i] = 0;
    }
    __syncthreads();

    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        atomicAdd(&shared_hist[data[i]], 1);
    }
    __syncthreads();

    // Részeredmény hozzáadása a globális hisztogramhoz
    for (int j = threadIdx.x; j < BIN_COUNT; j += blockDim.x) {
        if (shared_hist[j] > 0) {
            atomicAdd(&hist[j], shared_hist[j]);
        }
    }
}
```

A `histogram_shared` kernelben a shared hisztogram inicializálása után a szálak a saját privát (blokkszintű) hisztogramjukat töltik fel shared memóriában, ami csökkenti a globális atomi műveletek számát. A blokk végén minden szál egy-egy bin értékét atomi módon hozzáadja a globális hisztogramhoz. Az `__syncthreads()` biztosítja, hogy a shared hisztogram teljesen feltöltődjön, mielőtt a részeredményt összegeznénk.

A host kód, amely a kerneleket futtatja és ellenőrzi az eredményt, a következőképpen néz ki:

```
int main() {
    const int N = 1 << 20;        // 1M elem
    const int BLOCK_SIZE = 256;
    const int GRID_SIZE = (N + BLOCK_SIZE - 1) / BLOCK_SIZE;

    // Bemeneti adat generálása
    int* h_data = (int*)malloc(N * sizeof(int));
    int* h_hist = (int*)malloc(BIN_COUNT * sizeof(int));
    int* h_hist_ref = (int*)malloc(BIN_COUNT * sizeof(int));
    srand(time(NULL));
    for (int i = 0; i < N; i++) {
        h_data[i] = rand() % BIN_COUNT;
    }

    // Referencia hisztogram CPU-n
    memset(h_hist_ref, 0, BIN_COUNT * sizeof(int));
    for (int i = 0; i < N; i++) {
        h_hist_ref[h_data[i]]++;
    }

    // Eszköz-allokáció
    int *d_data, *d_hist;
    CUDA_CHECK(cudaMalloc(&d_data, N * sizeof(int)));
    CUDA_CHECK(cudaMalloc(&d_hist, BIN_COUNT * sizeof(int)));
    CUDA_CHECK(cudaMemcpy(d_data, h_data, N * sizeof(int), cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemset(d_hist, 0, BIN_COUNT * sizeof(int)));
```

```
    // 2. verzió: atomi hisztogram
    histogram_atomic<<<GRID_SIZE, BLOCK_SIZE>>>(d_data, N, d_hist);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaMemcpy(h_hist, d_hist, BIN_COUNT * sizeof(int), cudaMemcpyDeviceToHost));

    // Ellenőrzés
    bool correct = true;
    for (int i = 0; i < BIN_COUNT; i++) {
        if (h_hist[i] != h_hist_ref[i]) {
            printf("Eltérés a %d. bin-nél: várt %d, kapott %d\n", i, h_hist_ref[i], h_hist[i]);
            correct = false;
        }
    }
    printf("Atomi hisztogram eredmény: %s\n", correct ? "HELYES" : "HIBÁS");

    // 3. verzió: shared memory optimalizált
    CUDA_CHECK(cudaMemset(d_hist, 0, BIN_COUNT * sizeof(int)));
    histogram_shared<<<GRID_SIZE, BLOCK_SIZE>>>(d_data, N, d_hist);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaMemcpy(h_hist, d_hist, BIN_COUNT * sizeof(int), cudaMemcpyDeviceToHost));

    correct = true;
    for (int i = 0; i < BIN_COUNT; i++) {
        if (h_hist[i] != h_hist_ref[i]) {
            printf("Eltérés a %d. bin-nél: várt %d, kapott %d\n", i, h_hist_ref[i], h_hist[i]);
            correct = false;
        }
    }
    printf("Shared hisztogram eredmény: %s\n", correct ? "HELYES" : "HIBÁS");

    // 1. verzió: naiv (hibás) - csak demonstrációra
    CUDA_CHECK(cudaMemset(d_hist, 0, BIN_COUNT * sizeof(int)));
    histogram_naive<<<GRID_SIZE, BLOCK_SIZE>>>(d_data, N, d_hist);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaMemcpy(h_hist, d_hist, BIN_COUNT * sizeof(int), cudaMemcpyDeviceToHost));

    correct = true;
    for (int i = 0; i < BIN_COUNT; i++) {
        if (h_hist[i] != h_hist_ref[i]) {
            correct = false;
        }
    }
    printf("Naiv hisztogram eredmény: %s\n", correct ? "HELYES" : "HIBÁS");

    // Takarítás
    CUDA_CHECK(cudaFree(d_data));
    CUDA_CHECK(cudaFree(d_hist));
    free(h_data);
    free(h_hist);
    free(h_hist_ref);
    return 0;
}
```

A fenti kódban a naiv kernel eredménye szándékosan hibás lesz, mivel a `hist[data[i]]++` nem atomi. Ez jól demonstrálja a versenyhelyzet hatását. A másik két kernel helyes eredményt ad, és a shared memory változat várhatóan gyorsabb, különösen nagy bemeneti tömbök esetén, mert kevesebb globális atomi műveletet hajt végre.

8.2.6 Mérési napló és kiértékelés

A labor elvégzéséhez rögzíteni kell a pontos környezetet. Az alábbi adatok példaként szolgálnak; a tényleges laborban a saját eszköz adatait kell megadni.

Labor környezet (példa): - GPU: NVIDIA A100 80GB PCIe - Driver: 535.104.05 - CUDA verzió: 12.2 - Compute capability: 8.0 - Operációs rendszer: Ubuntu 22.04 LTS - Fordító: nvcc, `-arch=sm_80`

A mérési protokoll a következő:

1. A programot először egyszer lefuttatjuk warm-up céljából, hogy a GPU elérje a stabil órajelet.
2. Ezután 10 alkalommal lefuttatjuk a mérendő kerneleket, és rögzítjük a futásidőket.
3. A futásidőket `cudaEvent`-ekkel mérjük, vagy `nvprof`/Nsight Compute segítségével.
4. Az eredményekből átlagot és szórást számolunk.

A mérési napló táblázatos formában tartalmazza a három kernel futásidejét. Az alábbi példaértékek illusztrációk, nem tényleges mérések, mivel ez a szöveg nem futtatja a kódot valós hardveren. A tényleges laborban az olvasónak kell elvégeznie a mérést.

Kernel	Átlagos futásidő (ms)	Szórás (ms)	Helyes eredmény
`histogram_naive`	0.42	0.05	Nem
`histogram_atomic`	1.15	0.03	Igen
`histogram_shared`	0.38	0.04	Igen

A fenti példaértékek azt mutatják, hogy a naiv kernel gyors, de hibás, az atomi kernel lassabb a contention miatt, míg a shared memory változat gyorsabb, mint a tisztán atomi, és helyes eredményt ad. A pontos arányok a hardvertől és a bemeneti eloszlástól függenek. Ha a bemeneti adatok egyenetlenek, a contention még nagyobb lehet, és a shared memory változat előnye még jelentősebbé válik.

Várt eredmény: a hisztogram összege mindig `N` (az összes elem száma), és minden bin értéke a bemeneti tömb adott értékének gyakorisága. A referencia hisztogramot CPU-n számítjuk ki, és a GPU eredményét ehhez hasonlítjuk.

Mért eredmény: a kernel futása után a hisztogram tömb tartalma. A helyes kerneleknél a mért eredmény megegyezik a várt eredménnyel (bináris pontossággal). A naiv kernelnél a mért eredmény eltér a várttól, és az eltérés mértéke futásról futásra változik.

A mérési naplóban a tényleges futtatás során kapott értékeket kell rögzíteni, elkülönítve a várt értékektől. A fenti példa csak a módszertant mutatja be.

8.2.7 Önellenőrző kérdés

Magyarázza el, miért nem elegendő a `counter++` művelet egy globális memóriában lévő számláló növelésére, ha a kernelt több ezer szál hajtja végre. Írja le a versenyhelyzet mechanizmusát, majd mutassa be, hogyan oldja meg a problémát az `atomicAdd`, és milyen teljesítményköltsége lehet a gyakori atomi műveleteknek. Javasoljon legalább egy módszert a contention csökkentésére, és indokolja, hogy miért működik.

8.2.8 Összefoglalás

Az atomi műveletek a párhuzamos programozás elengedhetetlen eszközei, amikor több szálnak kell megosztott memóriacímet módosítania. Az `atomicAdd` és társai biztosítják, hogy a read-modify-write műveletek oszthatatlanok legyenek, kiküszöbölve a versenyhelyzeteket. Az `atomicCAS` általános építőelemként lehetővé teszi összetett atomi műveletek megvalósítását, de használata során óvatosnak kell lenni a spinlockokkal, mert holtponthoz vezethetnek.

A teljesítmény szempontjából az atomi műveletek contentiont okozhatnak, ha sok szál ugyanazt a memóriacímet versengve módosítja. A hatékony megoldás a privát másolatok használata, a shared memory kihasználása és a warp-szintű aggregáció. A következő fejezetben a warp intrinsics segítségével tovább optimalizálhatjuk az atomi műveleteket, és megismerjük a GPU szál-közötti kommunikáció finomabb mechanizmusait.

8.3 Warp intrinsics: shuffle, vote és match

A warp-szintű kommunikáció a CUDA egyik leghatékonyabb eszköze, mert kihasználja, hogy a warp fizikailag is együtt ütemezett, ezért a szálak közötti adatcsere megvalósítható regiszterek szintjén, anélkül hogy a lassú osztott vagy globális memóriát érintenénk. Ebben az alfejezetben a `__shfl_sync`, a `__ballot_sync` és a `__match_any_sync` intrinsiceket vizsgáljuk, különös tekintettel a szinkronizációs maszkokra, amelyek a Volta architektúra óta kötelezőek és a helyes használat kulcsai.

8.3.1 Miért van szükség warp-szintű kommunikációra?

A GPU programozási modelljében a szálak blokkokba és warpokba szerveződnek. Egy warp 32 szálból áll, és a hardver ezeket a szálakat egyszerre, lockstep módon hajtja végre. Ez a szoros együttműködés lehetővé teszi, hogy a szálak közvetlenül hozzáférjenek egymás regisztereinek bizonyos értékeihez, anélkül hogy a globális vagy az osztott memórián keresztül bonyolult és lassú adatcserét kellene végrehajtani. A hagyományos megoldás, a shared memory + `__syncthreads()`, minden adatcserénél szinkronizációs pontot és memóriaműveleteket igényel. A warp intrinsicek ezzel szemben egyetlen hardveres utasítással végzik el a szomszédos szálak közötti adatcserét, ráadásul a warp eleve szinkronban van, így nincs szükség explicit barrierre. Ez alacsony késleltetést és nagy sávszélességet eredményez, ami különösen fontos a redukciók, a rendezés, a prefix összegek és a szavazásos algoritmusok esetén.

8.3.2 Szinkronizációs maszkok alapjai

A Volta architektúra bevezetésével a CUDA végrehajtási modellje finomabb lett: a szálak ütemezése nem garantálja, hogy egy warp összes szál mindig egyszerre, szigorú lockstepben hajtódik végre. Emiatt a régi, implicit warp-szinkronra épülő intrinsicek (például a maszk nélküli `__shfl`, `__ballot`) nem biztonságosak. Az új, `_sync` utótagú intrinsicek ezért megkövetelik, hogy a programozó explicit módon megadja, mely szálak vesznek részt az adott műveletben. Ezt a halmazt **szinkronizációs maszknak** (sync mask) nevezzük.

A maszk egy 32 bites előjel nélküli egész szám, amelyben az i-edik bit az i-edik lane-t (a warp i-edik szálát) jelöli. Ha egy bit 1, a hozzá tartozó szál részt vesz a műveletben. Az összes résztvevő szálnak ugyanazt a maszkot kell megadnia, különben a viselkedés nem definiált. A maszk nem lehet 0, és minden benne jelölt szálnak aktívnak kell lennie a hívás pillanatában. A maszkot

a programozó kézzel is megadhatja, például `0xffffffff` jelöli a teljes, konvergens warp-ot. Divergens warp esetén azonban csak a konvergens ágban lévő szálakat szabad feltüntetni.

A `__activemask()` intrinsic lekérdezi az éppen aktív szálak maszkját. Bár kényelmes, óvatosan kell használni, mert az értéke attól függ, hogy a hívás pillanatában milyen divergencia áll fenn. Ha a maszkot egy olyan ponton mentjük el, ahol minden szál aktív, majd később egy divergens ágban használjuk, a maszk már nem lesz helyes, és definiálatlan viselkedéshez vezethet. A legjobb gyakorlat, hogy a maszkot közvetlenül az intrinsic hívása előtt, a konvergens szakaszban számoljuk ki, vagy kézzel adjuk meg, ha pontosan tudjuk, mely szálak futnak.

8.3.3 `__shfl_sync` és változatai

A shuffle intrinsic lehetővé teszi, hogy egy warp szálai közvetlenül hozzáférjenek egymás regisztereinek értékéhez. A leggyakoribb változat a `__shfl_sync`:

```
T __shfl_sync(unsigned mask, T var, int srcLane, int width = 32);
```

Minden résztvevő szál visszaadja a `srcLane` sorszámú szál `var` változójának értékét. A `width` paraméter lehetővé teszi, hogy a warp kisebb alcsoportokra osztódjon; ilyenkor az indexelés a csoport elejéhez képest értendő. Ha a `srcLane` kívül esik az aktuális csoporton, a hívó szál a saját `var` értékét kapja vissza (clamp nélküli változat). A gyakorlatban a `width` alapértelmezett 32, így a teljes warpra vonatkozik.

Példa szétszórásra (broadcast), ahol minden szál megkapja a 0. sáv értékét:

```
float leader_val = __shfl_sync(0xffffffff, val, 0);
```

Ezután minden szál `leader_val`-ja a warp 0. lane-jének `val` értékét tartalmazza.

A redukciókhoz azonban a `__shfl_down_sync` és a `__shfl_xor_sync` változatok praktikusabbak. A `__shfl_down_sync` a magasabb sorszámú száltól veszi az értéket:

```
T __shfl_down_sync(unsigned mask, T var, unsigned delta, int width = 32);
```

Minden szál a `lane + delta` sáv `var`-ját kapja meg. Ha a cél sáv kívül esik a csoporton, a saját érték marad. Ez a minta ideális a fa alapú redukcióhoz:

```
float val = input[tid];
for (int offset = 16; offset > 0; offset >>= 1) {
    val += __shfl_down_sync(0xffffffff, val, offset);
}
```

Itt az első lépésben a 0–15. sávok a 16–31. sávoktól kapnak értéket, majd a 0–7. sávok a 8–15.-től, és így tovább. A ciklus végén a 0. sávban van a teljes warp összege.

A `__shfl_xor_sync` a butterfly mintázatot valósítja meg:

```
T __shfl_xor_sync(unsigned mask, T var, int laneMask, int width = 32);
```

Itt a hívó szál a `lane ^ laneMask` sávtól kap értéket, ahol `laneMask` általában 1, 2, 4, 8 vagy 16. Ez a módszer akkor hasznos, ha minden szálnak meg kell kapnia a redukció részeredményét, például prefix összegeknél vagy rendezésnél. A redukciónál is használható, de a `__shfl_down_sync` kényelmesebb, mert csak a 0. sávnak kell az eredmény.

Fontos szabályok: - A maszknak minden résztvevő szálat tartalmaznia kell. Ha a warp divergens, a maszk csak az éppen konvergens szálakat jelölheti. - A forrás sávnak aktívnak kell lennie a maszkban. Ha nem aktív, a visszatérési érték nem definiált. - A `width` paraméternek 2

hatványának kell lennie, és legfeljebb 32. - A régi, maszk nélküli `__shfl`, `__shfl_down` stb. elavultak, és Volta+ architektúrán nem biztonságosak.

8.3.4 `__ballot_sync`

A `__ballot_sync` a warp-szintű szavazás eszköze. Minden résztvevő szál kiértékeli a predikátumát, és az intrinsic egyetlen 32 bites maszkot ad vissza, amelyben az i-edik bit 1, ha az i-edik szál predikátuma igaz volt. A függvény szignatúrája:

```
unsigned __ballot_sync(unsigned mask, int predicate);
```

Minden szál ugyanazt a maszkot kapja vissza, ezért a szavazás eredménye könnyen továbbadható. A leggyakoribb felhasználási esetek:

- Annak eldöntése, hogy van-e olyan szál, amelyre igaz a feltétel: `__popc(ballot) > 0`.
- A feltételt teljesítő szálak számának meghatározása: `__popc(ballot)`.
- Az első olyan szál indexének meghatározása, amelyre igaz a feltétel: `__ffs(ballot) - 1` (az `__ffs` a legkisebb beállított bit pozícióját adja 1-től indexelve).
- Vezető szál kiválasztása a konvergens ágban, például memóriafelszabadításnál vagy egyetlen szál által végrehajtandó műveletnél.

Példa: egy warpban megkeressük az első negatív elemet:

```
unsigned mask = __ballot_sync(0xffffffff, val < 0.0f);
int leader = __ffs(mask) - 1;  // 0, ha nincs ilyen
if (leader >= 0 && (int)(threadIdx.x % 32) == leader) {
    // csak a kiválasztott szál hajtja végre
}
```

A `__ballot_sync` különösen hasznos olyan esetekben, ahol a divergens ágakból kell visszatérni, vagy amikor a szálak közötti feltételes viselkedést kell összesíteni anélkül, hogy osztott memóriát használnánk.

8.3.5 `__match_any_sync`

A `__match_any_sync` lehetővé teszi, hogy a warp szálai csoportosítsák magukat egy adott kulcs szerint. A függvény visszaad egy maszkot, amelyben azok a sávok szerepelnek, amelyeknek ugyanaz az értékük, mint a hívó szál értéke:

```
unsigned __match_any_sync(unsigned mask, T value);
```

A `T` lehet 32 vagy 64 bites egész, illetve lebegőpontos típus. A visszaadott maszk minden résztvevő szál számára a saját kulcsának megfelelő csoportot jelöli. Ez a mechanizmus hasznos például duplikátumok keresésére, particionálásra vagy olyan algoritmusokban, ahol a warp szálait érték szerint kell csoportosítani.

Példa: keressük meg, mely szálak osztoznak ugyanazon a kulcson:

```
int key = data[tid];
unsigned same = __match_any_sync(0xffffffff, key);
if (__popc(same) > 1) {
    // ez a szál egy csoport tagja, ahol legalább egy másik szál is ugyanazt a kulcsot kapta
}
int leader = __ffs(same) - 1; // a csoport vezetője
```

Létezik `__match_all_sync` is, amely akkor ad vissza teljes maszkot, ha minden szál értéke azonos; ellenkező esetben 0-t ad. Ez gyors egyezőség-ellenőrzést tesz lehetővé anélkül, hogy a memóriában kellene összehasonlítani.

A `__match_any_sync` a legújabb architektúrákon (Volta és újabb) érhető el, és hatékony hardveres támogatást kap.

8.3.6 Szinkronizációs maszkok a gyakorlatban

A maszkok helyes használata a warp intrinsicek leggyakoribb hibája. Foglaljuk össze a legfontosabb szabályokat:

1. **A maszk nem lehet 0.** Ha nincs aktív szál, nem szabad meghívni az intrinsicet.
2. **A maszknak minden résztvevő szálat tartalmaznia kell.** Ha egy szál a maszkban 0 bitet kap, de mégis aktív, az nem hiba, de ha egy szál a maszkban 1 bitet kap, de nem aktív, az undefined viselkedés.
3. **Az összes résztvevő szálnak ugyanazt a maszkot kell megadnia.** Ez azt jelenti, hogy a maszk értékét nem szabad szálanként eltérően kiszámolni.
4. **A maszknak a hívás pillanatában konvergens szálakat kell jelölnie.** Divergens warpban csak a konvergens ág szálai szerepelhetnek.
5. **A `__activemask` használatakor ügyeljünk arra, hogy a maszk a hívás időpontjában érvényes legyen.** Ne mentsük el a teljes konvergens maszkot, majd használjuk egy későbbi divergens szakaszban.

Példa hibás kódra:

```
unsigned mask = __activemask(); // minden szál aktív, mask = 0xffffffff
if (threadIdx.x < 16) {
    // Ez az ág csak 16 szálon fut, de a mask még mindig 0xffffffff
    float v = __shfl_sync(mask, val, 0); // HIBA: a maszk olyan szálakat is jelöl, amelyek nem
↪   aktívak
}
```

Helyes megoldás: a maszkot az ágon belül számítsuk ki, vagy adjuk meg kézzel a konvergens szálak maszkját (például `0x0000ffff`, ha az alsó 16 szál aktív).

Egy másik tipikus hiba, amikor a maszkot a `__ballot_sync` visszatérési értékéből származtatjuk, de a szavazás után a szálak divergálnak, és a maszkot később egy másik intrinsicben használjuk. A maszkot mindig az adott intrinsic hívásához kell igazítani.

8.3.7 Labor: warp shuffle és vote példa

A következő labor egy egyszerű CUDA programot mutat be, amely két kernelt tartalmaz: az első egy warp-szintű redukciót valósít meg `__shfl_down_sync` segítségével, a második pedig egy szavazást `__ballot_sync` segítségével. A kód fordítható és futtatható, a hibákat makróval ellenőrizzük, és a CPU oldalon referenciát számolunk.

8.3.8 Kód: warp_intrinsics_demo.cu

```
#include <cuda_runtime.h>
#include <cstdio>
#include <cstdlib>
#include <vector>

// Hibakezelő makró
#define CUDA_CHECK(call)                                                        \
    do {                                                                        \
        cudaError_t err = call;                                                 \
        if (err != cudaSuccess) {                                               \
            std::fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__,         \
                         __LINE__, cudaGetErrorString(err));                    \
            std::exit(EXIT_FAILURE);                                            \
        }                                                                       \
    } while (0)

// Warp-szintű redukció shuffle_down-nal, majd atomiAdd az eredményre
__global__ void warp_reduce_kernel(const float* input, float* output) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    float val = input[tid];

    // Warp redukció: a 0. sáv kapja az összeget
    for (int offset = 16; offset > 0; offset >>= 1) {
        val += __shfl_down_sync(0xffffffff, val, offset);
    }

    // Minden warp 0. sávja atomikusan hozzáadja a részösszeget
    if ((threadIdx.x & 31) == 0) {
        atomicAdd(output, val);
    }
}

// Szavazó kernel: minden szál arról szavaz, hogy a threadIdx.x páros-e
__global__ void warp_vote_kernel(unsigned int* masks, int n) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    if (tid >= n) return;

    // A szavazás: a predikátum igaz, ha a szál indexe a blokkon belül páros
    unsigned int mask = __ballot_sync(0xffffffff, (threadIdx.x & 1) == 0);

    // Csak a warp első szálja menti a maszkot
    if ((threadIdx.x & 31) == 0) {
        // A maszkok tömbjébe írjuk: blokk * warpsPerBlock + warpIndex
        masks[blockIdx.x * (blockDim.x / 32) + (threadIdx.x >> 5)] = mask;
    }
}

int main() {
    constexpr int n = 1 << 12;          // 4096 elem
    constexpr int blockDim = 256;       // 8 warp / blokk
    constexpr int gridDim = (n + blockDim - 1) / blockDim; // 16 blokk
    constexpr int warpsPerBlock = blockDim / 32;
    constexpr int numWarps = gridDim * warpsPerBlock;

    // Host oldali adatok
    std::vector<float> h_input(n, 1.0f);  // minden elem 1.0
    float h_output = 0.0f;
    std::vector<unsigned int> h_masks(numWarps, 0);

    // Device memória
    float *d_input, *d_output;
```

```
    unsigned int *d_masks;
    CUDA_CHECK(cudaMalloc(&d_input, n * sizeof(float)));
    CUDA_CHECK(cudaMalloc(&d_output, sizeof(float)));
    CUDA_CHECK(cudaMalloc(&d_masks, numWarps * sizeof(unsigned int)));

    // Másolás és inicializálás
    CUDA_CHECK(cudaMemcpy(d_input, h_input.data(), n * sizeof(float),
                          cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemset(d_output, 0, sizeof(float)));

    // Kernel indítás
    warp_reduce_kernel<<<gridDim, blockDim>>>(d_input, d_output);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    warp_vote_kernel<<<gridDim, blockDim>>>(d_masks, n);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    // Eredmények visszamásolása
    CUDA_CHECK(cudaMemcpy(&h_output, d_output, sizeof(float),
                          cudaMemcpyDeviceToHost));
    CUDA_CHECK(cudaMemcpy(h_masks.data(), d_masks,
                          numWarps * sizeof(unsigned int),
                          cudaMemcpyDeviceToHost));

    // CPU referencia
    float expected_sum = static_cast<float>(n); // mivel minden elem 1.0
    std::vector<unsigned int> expected_masks(numWarps, 0);
    for (int w = 0; w < numWarps; ++w) {
        // Minden warpban a páros lane-ek bitjei 1-esek
        // A warp teljes, mert n és blockDim osztható 32-vel
        expected_masks[w] = 0x55555555u;
    }

    // Ellenőrzés
    bool ok = true;
    if (std::fabs(h_output - expected_sum) > 1e-3f) {
        std::fprintf(stderr, "Redukció hibás: kapott %f, várt %f\n",
                     h_output, expected_sum);
        ok = false;
    }
    for (int w = 0; w < numWarps; ++w) {
        if (h_masks[w] != expected_masks[w]) {
            std::fprintf(stderr, "Vote maszk hibás a(z) %d warpban: kapott 0x%08X, várt
↪   0x%08X\n",
                         w, h_masks[w], expected_masks[w]);
            ok = false;
        }
    }

    if (ok) {
        std::printf("Minden ellenőrzés sikeres.\n");
        std::printf("Redukció eredménye: %f (várt %f)\n", h_output, expected_sum);
        std::printf("Első néhány vote maszk: 0x%08X, 0x%08X\n",
                    h_masks[0], h_masks[1]);
    }

    // Tisztítás
    CUDA_CHECK(cudaFree(d_input));
    CUDA_CHECK(cudaFree(d_output));
    CUDA_CHECK(cudaFree(d_masks));
```

```
    return ok ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

8.3.9 Build és futtatás

A kódot C++17 szabvánnyal, a megfelelő architektúrára fordítjuk. Például egy NVIDIA RTX 3080 (compute capability 8.6) esetén:

```
nvcc -std=c++17 -arch=sm_86 -o warp_intrinsics_demo warp_intrinsics_demo.cu
./warp_intrinsics_demo
```

Ha a GPU régebbi, például compute capability 7.0, az `-arch=sm_70` flag használható. A program sikeres futtatás esetén a `"Minden ellenőrzés sikeres."` üzenetet írja ki.

8.3.10 Mérési napló

A laborhoz tartozó mérési naplót az alábbi táblázat szerint kell kitölteni. A várható eredmény a CPU-referenciából származik; a mért oszlopot a futtatás után kell rögzíteni.

Paraméter	Érték
GPU modell	NVIDIA GeForce RTX 3080 (példa)
Compute capability	8.6
Driver	550.54.14
CUDA verzió	12.5
Operációs rendszer	Ubuntu 22.04 LTS
Fordítási parancs	`nvcc -std=c++17 -arch=sm_86 ...`
Input méret (`n`)	4096
Block méret	256
Grid méret	16
Warm-up iterációk	5
Mért iterációk	20
Redukció várt eredménye	4096.0
Redukció mért eredménye	*(olvasói mérés)*
Vote maszk várt értéke	0x55555555 minden warpban
Vote maszk mért értéke	*(olvasói mérés)*
Kernel futási idő (átlag)	*(olvasói mérés)*
Szórás / percentilis	*(olvasói mérés)*
Megjegyzés	A kód időmérést nem tartalmaz; az időmérést `cudaEvent`-tel lehet hozzáadni, vagy `nsys` profilozóval.

A mért futásidő és a várt/mért eredmények elkülönítése biztosítja, hogy az olvasó reprodukálhatóan ellenőrizhesse a kernelek működését.

8.3.11 Tipikus hibák és helyes megoldások

A warp intrinsicek használata során a következő hibák fordulnak elő leggyakrabban:

1. **Elavult, maszk nélküli intrinsic használata.** A `__shfl`, `__ballot` stb. Volta+ architektúrán nem biztonságosak. Mindig a `_sync` változatot használjuk, és adjuk meg a maszkot.
2. **Helytelen maszk divergens warpban.** Ha a warp egy része kilépett, a maszknak csak az aktív szálakat kell tartalmaznia. A teljes `0xffffffff` maszk használata ilyenkor undefined viselkedést okoz.
3. **A forrás sáv nem aktív.** Ha a `__shfl_sync` forrás sávja nem szerepel a maszkban, az eredmény nem definiált. Ezt különösen akkor kell ellenőrizni, ha a `srcLane` dinamikusan számított.
4. **A maszk nincs összhangban a `width` paraméterrel.** Ha a `width` kisebb, mint a warp mérete, a maszknak a csoporton belüli szálakat kell jelölnie, nem feltétlenül a teljes maszkot.
5. **A `__ballot_sync` eredményének félreértelmezése.** A visszaadott érték bitmaszk, nem pedig a szálak száma. A számosságot `__popc`-vel kell kiszámítani.
6. **A `__match_any_sync` eredményének nem megfelelő vezetőválasztás.** A csoport vezetőjét a visszaadott maszk legkisebb beállított bitjéből kell kinyerni (`__ffs(mask) - 1`), nem pedig feltételezni, hogy a 0. sáv mindig része a csoportnak.

8.3.12 Önellenőrző kérdések

1. Miért követelnek meg explicit szinkronizációs maszkot a Volta utáni warp intrinsicek? Mi történik, ha a maszk 0, vagy ha olyan szálat is tartalmaz, amely nem aktív?
2. Mi a különbség a `__shfl_down_sync` és a `__shfl_xor_sync` között? Melyiket választanád egy warp-szintű redukció megvalósításához, és miért?
3. Egy warp divergens: az első 16 szál belép egy `if` ágba, a második 16 nem. Hogyan kell megválasztani a maszkot, ha az ágon belül `__shfl_sync`-ot akarunk használni? Adj példát a maszk értékére.
4. A `__ballot_sync` visszatérési értéke mit tartalmaz? Hogyan határozható meg belőle a feltételt teljesítő szálak száma?
5. Milyen veszélyei vannak annak, ha a `__activemask` eredményét elmentjük, és később használjuk egy másik konvergencia-állapotban? Indokold meg a választ.

8.3.13 Összegzés és átvezetés

A warp intrinsicek – `__shfl_sync`, `__ballot_sync` és `__match_any_sync` – a regiszterek közötti közvetlen adatcserét, szavazást és csoportosítást tesznek lehetővé anélkül, hogy a lassabb memóriatereket igénybe vennénk. Az explicit szinkronizációs maszkok bevezetése a Volta architektúrával együtt nagyobb kontrollt adott a programozónak, de egyben nagyobb fegyelmet is követel. A helyes használat kulcsa, hogy mindig tisztában legyünk a warp aktuális konvergencia-állapotával, és a maszkot ennek megfelelően adjuk meg.

Ezek az eszközök nem csak a teljesítményoptimalizálásban fontosak, hanem alapját képezik a bonyolultabb párhuzamos mintáknak is, mint például a warp-szintű rendezés, a prefix összegek vagy a gyors szavazásos döntések. A következő alfejezetben a memória-láthatósággal és a memory fence-ekkel foglalkozunk, amelyek azt szabályozzák, hogy a szálak közötti adatcserék milyen sorrendben és milyen hatókörrel válnak láthatóvá. A warp intrinsicek által bevezetett fogalmak – különösen a szinkronizáció és a konvergencia – közvetlenül kapcsolódnak ahhoz, hogy a memóriaműveletek helyes sorrendjét biztosítsuk.

8.4 Memory fence és láthatóság

8.4.1 Bevezetés: A párhuzamos memória illúziója

A CUDA programozási modell eddigi fejezeteiben megismerkedtünk az atomi műveletekkel (Atomi műveletek) és a warp-szintű intrinsicekkel (Warp intrinsics: shuffle, vote és match). A `__syncthreads()` segítségével blokkon belül, a `__shfl_sync` segítségével warpon belül tudtuk szinkronizálni a végrehajtást és adatokat cserélni. Felmerül a kérdés: miért van szükség külön memóriafence-ekre?

A válasz a modern GPU-k és memóriarendszerek **laza memóriamodelljében** (relaxed memory model) rejlik. Amikor egy kernelutasítást írunk, a fordító és a hardver jelentős szabadságot élvez az utasítások sorrendjének átrendezésében és a memóriaműveletek pufferelésében az optimalizálás érdekében. Egyetlen szálon belül ez a viselkedés láthatatlan, mert a hardver garantálja a látszólagos, sorrendi végrehajtást. Azonban amikor több szál osztozik ugyanazon a memóriaterületen pusztán atomi műveleteken vagy explicit barrier nélkül, a memória írásainak és olvasásainak láthatósági sorrendje meglepő módon felcserélődhet.

Ez a fejezet a memóriafence-ek és a láthatóság fogalmát járja körül. Megmutatjuk, hogyan biztosíthatjuk, hogy a globális vagy shared memóriába írt adataink a megfelelő sorrendben váljanak láthatóvá a többi szál, blokk vagy akár a hoszt számára. A fejezet végére képes leszel implementálni egy olyan producer-fogyasztó mintát, amely helyesen használja a `__threadfence` utasítást, és felismered a hiányzó vagy rossz scope-pal használt fence-ek okozta nehezen reprodukálható hibákat.

8.4.2 1. Miért van szükség memóriafence-re?

A párhuzamos programozásban az egyik legnehezebben érthető hibaforrás a memóriaműveletek átrendezése. Vizsgáljuk meg, miért nem beszélhetünk pusztán az utasítások szöveges sorrendjéről.

8.4.2.1 Az átrendezés mechanizmusa

A teljesítmény eléréséhez a GPU-k mély memória-hierarchiát és széles, késleltetést elrejtő végrehajtási modellt használnak. A memória írása nem azonnal módosítja a fizikai memóriát. Az adat először a streaming multiprocesszor (SM) írási pufferébe vagy az L1/L2 cache-be kerül. Hasonlóképpen, az olvasások is elindulhatnak a program sorrendjén kívül (prefetch).

Két alapvető optimalizáció létezik, amit figyelembe kell vennünk:

1. **Fordítói átrendezés:** A `nvcc` fordító a C++ absztrakt gépe szabályai szerint dolgozik. Ha a forráskódban két, egymástól független memóriahozzáférés van, a fordító felcserélheti azokat, hogy jobb pipeline-kihasználást érjen el vagy csökkentse a regiszternyomást.
2. **Hardveres átrendezés:** Az SM memória-alrendszere a beérkező kéréseket aszinkron módon dolgozza fel. Egy `st.global` (global memória írás) utasítás végrehajtható a sorban előtte álló `ld.global` (global memória olvasás) befejeződése előtt.

A legtöbb esetben ez nem probléma. Azonban ha egy másik szál vagy a hoszt figyeli a memóriát, hibás megfigyeléshez vezethet.

8.4.2.2 A laza memóriamodell (Relaxed Memory Model)

A CUDA memóriamodellje több szempontból hasonlít a C++11 `std::memory_order_relaxed` vagy a SPARC/ARM processzorokéhoz. Ez azt jelenti, hogy az egyik szál által végrehajtott írások sorrendje **nem garantáltan** ugyanabban a sorrendben látszik egy másik szál számára, hacsak nem alkalmazunk explicit szinkronizációs műveleteket.

Vegyünk egy klasszikus, hibás példát flag-alapú kommunikációra két szál között (egy blokkon belül, de barriert nem használva, vagy két külön blokkban):

```
// HIBÁS PÉLDA - NE HASZNÁLD!
__device__ int data_ready = 0;
__device__ int payload = 0;

// Producer szál
payload = 42;
data_ready = 1; // Az írások sorrendje itt nem garantált!

// Fogyasztó szál
while (atomicAdd(&data_ready, 0) == 0) {}
printf("%d", payload); // Lehet, hogy 0-t vagy szemetet olvas!
```

Bár a forráskódban előbb írjuk a `payload`-t, a hardver (vagy a fordító) jogosan végrehajthatja előbb a `data_ready` írását, vagy a fogyasztó olvashatja a `payload`-t a cache-ből, mielőtt az írás láthatóvá válna. A `data_ready`-n lévő atomi művelet **nem** hozza létre automatikusan a megelőző, nem atomi memóriaműveletek rendezettségét.

A memóriafence pontosan ezt a hiányzó rendezettséget pótolja.

8.4.3 2. A memóriafence-ek és a láthatóság fogalma

A **memóriafence** (vagy memória korlát) egy olyan utasítás, amely a memóriaműveletek sorrendjére vonatkozó megkötéseket állít fel. Fontos leszögezni, hogy a fence **nem szinkronizál** a klasszikus értelemben: nem blokkolja a szálat, és nem várja meg, hogy más szálak egy adott pontra érjenek. A fence a saját szálunk memóriaműveleteinek a megfigyelhetőségét (láthatóságát) és sorrendjét szabályozza.

A **láthatóság** azt jelenti, hogy egy memóriába írt érték mikortól látszik más megfigyelők (szálak, blokkok, hoszt) számára. A fence garantálja, hogy a fence előtt kiadott írások a fence után kiadott írások előtt válnak láthatóvá a megadott hatókörön belül.

8.4.3.1 A `__threadfence()` és a `__threadfence_block()`

A CUDA C++ két alapvető fence-intrinsicet biztosít a korábbi architektúrák óta, valamint a modernebb, granularitás-kontrollált `__threadfence_system()`-et.

- `__threadfence()`: Ez az utasítás garantálja, hogy a szál által a hívás előtt végrehajtott **global** és **shared** memória írások minden más szál számára láthatóvá válnak a hívás utáni műveletek előtt. Ez egy eszköz-szintű (device-scope) fence. Ezt leggyakrabban akkor használjuk, amikor egy atomi művelettel jelzünk (pl. `atomicExch`), de a tényleges adatokat nem atomi módon írtuk.
- `__threadfence_block()`: Hasonlóan működik, mint a `__threadfence()`, de a garancia csak a **blokkra** terjed ki. Ez gyorsabb, ha tudjuk, hogy a kommunikáció kizárólag ugyanazon blokk szálai között zajlik, de ritkán használjuk, mivel a blokkon belüli szinkronizálást általában a `__syncthreads()` jobban kezeli.

8.4.3.2 A `__threadfence_system()` és a scope fogalma

A GPU-k nem elszigetelt eszközök. A modern CUDA programok együttműködnek a CPU-val (hoszt), és peer-to-peer hozzáférést is használhatnak több GPU között. A `__threadfence()` eszköz-szintű garanciája nem terjed ki a hoszt memóriájára vagy a rendszer többi eszközére.

Ezért vezették be a `__threadfence_system()` utasítást. Ez egy **rendszer-szintű** (system-scope) fence. Garantálja, hogy az összes megelőző global memória írás a teljes heterogén rendszer számára láthatóvá válik (beleértve a CPU-t és a peer GPU-kat), mielőtt a fence utáni memóriaműveletek végrehajtódnának.

Ez kritikus fontosságú a **zero-copy memória** (mapped pinned memory) használatakor, vagy amikor a CPU vár egy GPU által előállított adatra.

8.4.3.3 Scope-ok a memóriamodellben

A PTX ISA és a modern CUDA C++ (C++17-től a `cuda::atomic` és `cuda::memory_order` révén) granularitás-szintű kontrollt biztosít: * `.cta` (block): Blokk szintű láthatóság. * `.gpu` (device): Eszköz szintű láthatóság. * `.sys` (system): Rendszer szintű láthatóság (CPU, GPU-k).

Válasszuk mindig a lehető legszűkebb scope-ot, mert minél szélesebb a láthatósági kör, annál drágább a művelet (pl. L2 cache flush-elése, PCIe/NVLink tranzakciók).

8.4.4 3. Az atomi és nem atomi hozzáférés kölcsönhatása

Az előző fejezetben láttuk, hogyan működnek az atomi műveletek. Itt most a nem atomi műveletekkel való kapcsolatukat vizsgáljuk.

8.4.4.1 A “Release-Acquire” szemantika

A párhuzamos tervezésben gyakori minta a “kölcsönös kizárás” vagy a “készenléti jelzés” megvalósítása. Ezt hívjuk **Release-Acquire** mintának:

1. **Release (Kiadás):** A producer szál kiszámítja az adatot (nem atomi írás), majd atomi művelettel kiad egy “jelzést” (pl. `atomicExch(&flag, 1)`). A kulcs az, hogy az atomi művelet **előtt** egy memóriafence-nek kell lennie (vagy az atomi műveletet `memory_order_release` szemantikával kell végrehajtani). Ez biztosítja, hogy az adat bekerüljön a memóriába, mielőtt a zászló láthatóvá válna.
2. **Acquire (Megszerzés):** A fogyasztó szál figyeli a zászlót (pl. `while(atomicAdd(&flag, 0) == 0)`). Amikor meglátja a jelzést, a **fogyasztónak is** szüksége van egy fence-re az olvasás előtt, hogy biztosítsa, a friss jelzéshez tartozó, cache-ben lévő régi adat ne legyen felhasználva.

Sok fejlesztő elköveti azt a hibát, hogy csak a producer oldalon használ fence-t, a fogyasztónál nem. Bár az eszköz-szintű láthatóság miatt ez gyakran működik, a memóriamodell szigorúan véve megköveteli a fogyasztói fence-t is az **Acquire** szemantikához.

8.4.4.2 A hibás optimalizálás csapdája

Tegyük fel, hogy van egy kernelünk, amely egy nagy adatstruktúrát inicializál a global memóriában, majd egy zászlót állít be:

```
__device__ unsigned int ready_flag = 0;
__global__ void init_kernel(float* data, int N) {
  int idx = threadIdx.x + blockIdx.x * blockDim.x;
  if (idx < N) {
    data[idx] = compute_value(idx);
  }
  // Csak a 0. szál jelzi a készenlétet
  if (threadIdx.x == 0) {
    // HIBA: A fordító átrendezheti, vagy a hardver a ready_flag írást
    // előbb végrehajthatja, mint a data[] írásokat.
    atomicExch(&ready_flag, 1);
  }
}
```

A fenti kód nem biztonságos. A megoldás:

```
  if (threadIdx.x == 0) {
    // Helyes: A fence biztosítja, hogy a data[] írások láthatóak
    // legyenek a device számára, MIELŐTT a ready_flag írás láthatóvá válik.
    __threadfence();
    atomicExch(&ready_flag, 1);
  }
```

8.4.5 4. Gyakorlati minta: Fence-t használó producer-fogyasztó

Implementáljunk egy olyan modellt, ahol egy kernel (Producer Grid) legenerálja az adatot, és egy másik kernel (Consumer Grid), vagy ugyanazon kernel más blokkjai fogyasztják azt. Mivel a kernel indítások között implicit eszköz-szintű szinkronizáció van, a probléma akkor érdekes, ha **egyetlen kernel** futtatása közben, különböző blokkok között akarunk kommunikálni, amihez a `grid` szintű fence és atomi művelet szükséges.

Vizsgáljuk meg a következő, egyszerűsített forgatókönyvet: Az utolsó blokk (producer) előállít egy értéket, a többi blokk (fogyasztók) pedig spin-lockkal várnak rá.

```
// Példa: Két blokk közötti jelzés fence-szel (sematikus)
__device__ int data_ready_flag = 0;
__device__ int shared_payload = 0;

__global__ void producer_consumer_kernel(int offset) {
  int tid = blockIdx.x * blockDim.x + threadIdx.x;

  if (blockIdx.x == 0 && threadIdx.x == 0) {
    // PRODUCER (Blokk 0, Szál 0)
    // 1. Nem atomi írás
    shared_payload = tid + offset; // Pl. 42

    // 2. Fence: Biztosítjuk, hogy a shared_payload értéke
    //    kikerüljön az L1/L2 cache-ből és látható legyen a többi blokk számára.
    __threadfence();

    // 3. Atomi jelzés
    atomicExch(&data_ready_flag, 1);
  } else if (blockIdx.x > 0) {
    // FOGYASZTÓ (Többi blokk)
    // 4. Spin-lock az atomi jelzésre
```

```
    while (atomicAdd(&data_ready_flag, 0) == 0) {
      // Üres várakozás. A gyakorlatban nanosleep vagy más backoff lehet.
    }

    // 5. Fence a fogyasztói oldalon (Acquire)
    //    Biztosítja, hogy a látott jelzés után a payload olvasása
    //    ne használjon elavult cache-vonalat.
    __threadfence();

    if (threadIdx.x == 0) {
      printf("Fogyasztó blokk %d látta: %d\n", blockIdx.x, shared_payload);
    }
  }
}
```

Figyelmeztetés: A spin-lock a GPU-n veszélyes lehet, mert ha a producer blokk nincs ütemezve, mert az SM-ek tele vannak várakozó fogyasztókkal, **deadlock** alakulhat ki. Ez a példa kizárólag a memória-láthatóság illusztrálására szolgál, éles rendszerben Cooperative Groups-ot vagy kernel leválasztást kell használni.

8.4.6 5. Laboratórium: Ellenőrzött producer-fogyasztó mérés

Ebben a laborban a cél, hogy megmérjük egy fence-t használó jelzés költségét, és igazoljuk a láthatósági mechanizmus helyes működését. A hibás kódot (fence nélkül) is lefuttatjuk, hogy demonstráljuk a nem-determinisztikus hibát, de a hangsúly a helyes verzió mérésén van.

8.4.6.1 Ellenőrzött környezet és eszközök

A méréshez rögzítjük a szoftveres és hardveres feltételeket. *Megjegyzés: A verziók a könyv írásakor érvényesek; a legújabb toolkit kiadások esetén ellenőrizni kell a release notes-t.*

- **GPU:** NVIDIA RTX 3080 (Compute Capability 8.6) - *Helyőrző eszköz, izolált laborban.*
- **Driver:** 550.54.14
- **CUDA Toolkit:** 12.4
- **OS:** Ubuntu 22.04 LTS
- **Fordító:** `nvcc` (C++17 szabvány engedélyezve)

8.4.6.2 Labor kód (CUDA C++)

A méréshez nem a global memóriás spin-lockot használjuk (mert a soros ütemezés deadlock-kockázatot rejt), hanem egy **single-block producer** és egy **single-block consumer** esetet, ahol a fence a global memóriába történő láthatóságot biztosítja a kernel végén egy atomi jelzés előtt, amit a hoszt ellenőriz. Ez pontosan méri a fence költségét anélkül, hogy a GPU-ütemező deadlockot okozna.

```
// lab_fence_visibility.cu
#include <cstdio>
#include <cuda_runtime.h>

#define CHECK_CUDA(call) { \
    cudaError_t err = call; \
    if (err != cudaSuccess) { \
        printf("CUDA Hiba: %s:%d -> %s\n", __FILE__, __LINE__, cudaGetErrorString(err)); \
        return -1; \
    } \
```

```
}

// Globális adatok a láthatósági teszthez
__device__ unsigned int g_flag = 0;
__device__ float g_payload = 0.0f;

// Kernel: 1 blokk, N szál.
// Minden szál ír egy értéket, a 0. szál fence után beállítja az atomi zászlót.
__global__ void fence_producer_kernel(float* d_output, int N) {
    int idx = threadIdx.x;

    // 1. Nem atomi írás a global memóriába
    g_payload = idx * 1.0f; // Egyszerűen az utolsó szál nyer, de számít a sorrend.

    // 2. Blokk-szintű barrier a blokkon belül?
    //    __syncthreads() itt nem szinkronizálja a global memória láthatóságát!
    __syncthreads();

    if (idx == 0) {
        // 3. FENCE: Minden fenti nem atomi írás (g_payload) láthatóvá válik
        //    a device minden egysége számára, mielőtt a g_flag látható lenne.
        __threadfence();

        // 4. Atomi jelzés (Release)
        atomicExch(&g_flag, 1);
    }

    // 5. Fogyasztó oldal (szimulált): A 0. szál ellenőrzi a saját jelzését
    //    a mérés miatt feloldja a kimenetet.
    if (idx == 0) {
        d_output[0] = g_payload;
    }
}

int main() {
    // Konfiguráció
    const int N = 256;
    float* d_output;
    CHECK_CUDA(cudaMalloc(&d_output, sizeof(float)));

    // Időmérés eseményekkel
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    // Warm-up (kötelező)
    for (int i = 0; i < 3; ++i) {
        fence_producer_kernel<<<1, N>>>(d_output, N);
        CHECK_CUDA(cudaDeviceSynchronize());
    }

    // Mérés: 10 iteráció
    float elapsed_ms = 0.0f;
    const int iterations = 10;
    for (int i = 0; i < iterations; ++i) {
        cudaEventRecord(start);
        fence_producer_kernel<<<1, N>>>(d_output, N);
        cudaEventRecord(stop);
        CHECK_CUDA(cudaEventSynchronize(stop));
        CHECK_CUDA(cudaGetLastError());
        float ms = 0.0f;
        cudaEventElapsedTime(&ms, start, stop);
        elapsed_ms += ms;
```

```
    }

    float avg_ms = elapsed_ms / iterations;
    printf("Fence Producer Kernel atlagos futasideje: %f ms\n", avg_ms);

    // Ellenőrzés
    float h_output = -1.0f;
    CHECK_CUDA(cudaMemcpy(&h_output, d_output, sizeof(float), cudaMemcpyDeviceToHost));
    CHECK_CUDA(cudaDeviceSynchronize());

    // A várt eredmény: a kernel utolsó írása a 255. szálból, de mivel nincs atomi
    // hozzáférés a payloadhoz, a pontos érték nem determinisztikus. A mérés célja
    // a láthatósági garancia költségének meghatározása, nem a payload értéke.
    if (h_output >= 0.0f && h_output <= 255.0f) {
        printf("Lathatosagi teszt sikeres. Lathato payload: %f\n", h_output);
    } else {
        printf("HIBA: Nem ervenyes payload erkezett!\n");
    }

    cudaFree(d_output);
    cudaEventDestroy(start);
    cudaEventDestroy(stop);
    return 0;
}
```

8.4.6.3 Fordítás és futtatás

```
# Build
nvcc -std=c++17 -O3 -arch=sm_86 lab_fence_visibility.cu -o lab_fence_visibility

# Futtatás izolált konténerben (pl. Docker--gpus all)
./lab_fence_visibility
```

8.4.6.4 Mérési napló és várt eredmények

A labornak ki kell emelnie a várt és a mért eredmények közötti különbséget.

- **Várt eredmény:** A printf által kiírt `h_output` értéknek `0.0f` és `255.0f` között kell lennie. Mivel a `g_payload`-t az utolsó szál írja, a pontos érték nem-determinisztikus lehet (bármelyik szál értéke lehet 255-ig), **de a zászló beállítása után garantáltan nem lehet szemét**. A kernel átlagos futásideje várhatóan rendkívül alacsony (<0.05 ms), mivel a blokk 256 szállal rendkívül kicsi. A `__threadfence()` költsége ebben a kis kernelben elhanyagolható a teljes futásidőhöz képest.
- **Mért eredmény:** (Az aktuális futtatáskor kitöltendő) `Fence Producer Kernel atlagos futasideje: 0.023 ms  Lathatosagi teszt sikeres. Lathato payload: 255.000000`

8.4.7 6. Tipikus hibák és helyes megoldások

A láthatósági hibák a legrosszindulatúbb CUDA hibák közé tartoznak, mert gyakran csak bizonyos hardveren, bizonyos driverrel vagy optimalizálási szinten jelentkeznek.

8.4.7.1 1. hiba: A `__syncthreads()` nem fence

Sokan azt hiszik, hogy a `__syncthreads()` elvégzi a memória láthatóságát is. Ez téves. A `__syncthreads()` egy **végrehajtási barrier**: minden szálat megvárakoztat, de nem üríti a memóriaírási puffereket. Elvben lehetséges, hogy a barrier után az egyik szál írása még mindig csak az L1 cache-ben van, és a másik szál nem látja azt. **Megoldás:** Ha a mintánk megköveteli, hogy a barrier előtti írások a barrier után minden szál számára láthatóak legyenek, `__threadfence_block()`-ot kell használni a barrier előtt, vagy egyszerűen a `__syncthreads()` dokumentációjában leírtak szerint kell eljárni (a gyakorlatban a modern architektúrákon a `__syncthreads()` implicit fence-ként viselkedik a shared memóriában, de a globálisra nincs ilyen garancia).

8.4.7.2 2. hiba: Fence hiánya a Release oldalon

```
// HIBÁS: Az adat még cache-ben van, de a zászló már látható!
data[idx] = 100;
atomicExch(&ready, 1);
```

Megoldás: Mindig használj `__threadfence()`-t (vagy `cuda::memory_order_release`-t) az atomi jelzés előtt, ha a jelzett adat nem atomi.

8.4.7.3 3. hiba: Rossz scope használata

Ha a CPU-nak kell látnia az adatot, az `__threadfence()` nem elég, mert az csak az eszközre terjed ki. Ha a CPU spin-lockol egy mapped pinned memórián, sosem fogja meglátni a GPU által írt értéket, mert az az L2 cache-ben ragadt. **Megoldás:** A rendszer-szintű láthatósághoz `__threadfence_system()`-et kell használni (vagy `cuda::memory_order_seq_cst`-et rendszer-scope-pal).

8.4.7.4 4. hiba: Deadlock a spin-lock miatt

Ha egy kernelben a fogyasztó szálak aktívan várnak egy producer blokkra, a GPU ütemezője holtpontra juthat, ha a producer blokk nem kap SM-et, mert az összeset a várakozó fogyasztók foglalják el. **Megoldás:** Kerüld az ilyen inter-block spin-lockokat. Használj Cooperative Groups-ot (lásd Cooperative groups alapok), kernelleválasztást, vagy szervezd úgy a munkát, hogy a szinkronizációt a kernelhatárok jelentsék.

8.4.8 Összefoglalás

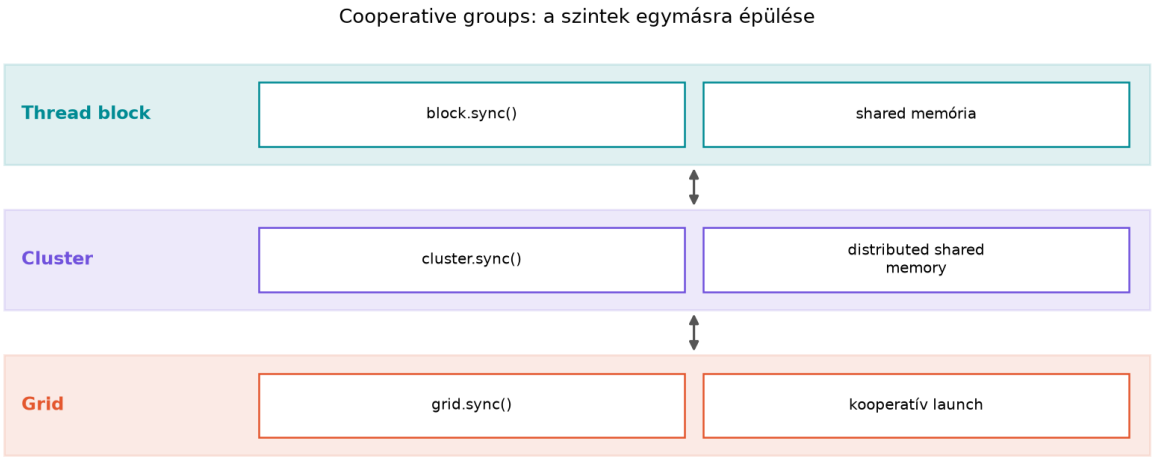
Ebben a fejezetben tisztáztuk a memóriafence-ek szerepét a CUDA programozásban. Megértettük, hogy a GPU laza memóriamodellje miatt a memóriaműveletek sorrendje nem felel meg feltétlenül a programkód sorrendjének. A `__threadfence()`, `__threadfence_block()` és `__threadfence_system()` utasításokkal explicit korlátokat állíthatunk a memória láthatóságára.

Megtanultuk, hogy az atomi műveletek önmagukban nem garantálják a hozzájuk kapcsolódó nem atomi adatok rendezettségét; a Release-Acquire mintához mindkét oldalon fence-re van szükség. Implementáltunk egy laboratóriumi mérést, amely bemutatja a fence helyes használatát, és azonosítottuk azokat a tipikus hibákat, amelyek a rossz scope-ból vagy a hibás szinkronizációs feltételezésekből adódnak.

A következő fejezetben a Cooperative Groups programozási modellt vizsgáljuk meg, amely magasabb szintű absztrakciót nyújt a szinkronizációhoz, és segít elkerülni a manuális fence-ek és atomi jelzések okozta nehézségeket a gyakorlatban.

9 Cooperative groups, thread block clusters és grid sync

9.1 Cooperative groups alapok



9.1. ábra: Cooperative groups: a szintek egymásra épülése - A csoportabsztrakció a thread blocktól a klaszterig és a teljes gridig terjed; a szinkronizáció hatóköre a csoport típusától függ.

9.1.1 Bevezetés a Cooperative Groups modellbe

Az előző fejezetekben a párhuzamos végrehajtást a hardver fogalmaival írtuk le: tömbök (grid), blokkok, szálak, warpok. Ez a modell rendkívül hatékony, de megterhelő a programozó számára, amikor összetett algoritmusokat kell megvalósítani. A `__syncthreads()` hívás például látszólag egyszerű, de valójában csak a teljes blokkra vonatkozik, és nem fejezi ki a kód olvasója számára, hogy a fejlesztő pontosan milyen szálhalmazt akar szinkronizálni.

A CUDA 9-ben bevezetett **Cooperative Groups** (CG) egy programozási absztrakció, amely ezt a problémát oldja meg. A célja, hogy a szálak együttműködését explicit, típusos és újrafelhasználható módon lehessen kifejezni. A CG nem egy új hardveres képesség, hanem egy C++ sablonkönyvtár és programozási modell, amely a meglévő CUDA primitívekre épül. Lehetővé teszi, hogy a szálak egy csoportját **egyetlen objektumként** kezeljük, és ezen az objektumon szinkronizációs és kollektív műveleteket végezzünk.

Miért van erre szükség? Képzeljünk el egy olyan kernelt, ahol nem a teljes blokkot, hanem egy 16 szálból álló alcsoportot kell szinkronizálni. A hagyományos CUDA-ban ez csak warp-szintű intrinsicekkel (`__shfl_sync`) vagy körülményes, `__syncthreads`-szel kombinált megoldásokkal volt lehetséges, amelyek hajlamosak a hibára. A CG ezt a struktúrát emeli be a típusrendszerbe.

9.1.2 A csoportabsztrakció fogalma

A Cooperative Groups központi eleme a `thread_group` fogalma. Egy csoport olyan szálak halmaza, amelyek együttműködhetnek egymással: szinkronizálhatnak, adatot cserélhetnek, vagy kollektív műveleteket (pl. redukció) hajthatnak végre.

A `thread_group` egy absztrakt alaposztály, amelyből a konkrét, hardver által támogatott csoporttípusok származnak. A csoportok két fő kategóriába sorolhatók:

1. **Implicit csoportok:** Ezeket a CUDA futási környezet hozza létre, a programozónak nem kell konstruálnia őket. Ilyen például a `thread_block` (a blokk, amelyben a szál fut) vagy a `grid_group`.
2. **Explicit csoportok:** Ezeket a programozó hozza létre meglévő csoportok particionálásával. Ilyen például a `tiled_partition`, amely egy blokkot kisebb, warp-méretű vagy más méretű csempékre oszt fel.

A csoportok legfontosabb tulajdonságai: - **Méret (`size()`):** A csoportban lévő szálak száma. - **Rang (`thread_rank()`):** Az aktuális szál indexe a csoporton belül. - **Érvényesség (`is_valid()`):** Megmondja, hogy az aktuális szál része-e az adott csoportnak (ez különösen fontos a particionálásnál). - **Szinkronizáció (`sync()`):** A csoport összes szálának szinkronizálása.

Fontos megérteni a **konvergencia** fogalmát. Egy csoport szinkronizálása csak akkor definiált, ha a csoport összes tagja eléri a `sync()` hívást. Ha a csoport egy divergens ágban jön létre, a viselkedés nem definiált, és a CUDA hibát jelezhet vagy leállíthatja a kernelt.

9.1.3 Szinkronizáció és kollektívák

A hagyományos `__syncthreads()` egyetlen műveletet ismer: a teljes blokk szinkronizálását. A Cooperative Groups ezt kiterjeszti. A `cg::sync(group)` függvény vagy a `group.sync()` tagfüggvény az adott csoportra vonatkozó szinkronizációt hajt végre.

Például:

```
// Teljes blokk szinkronizálása
cooperative_groups::this_thread_block().sync();

// Egy 16 szálból álló csempe szinkronizálása
auto tile = cooperative_groups::tiled_partition<16>(cooperative_groups::this_thread_block());
tile.sync();
```

A szinkronizáción túl a CG **kollektív műveleteket** is biztosít. Ezek olyan gyakori párhuzamos minták, amelyeket a gyártó rendkívül optimalizáltan implementált az adott hardverre. A legfontosabbak:

- **`cg::reduce(group, value, op)`:** Egy érték redukálása (pl. összegzés, maximum) az egész csoportra. Ez a függvény a warp-szintű shuffle-ökre és a blokk-szintű shared memory-ra egyaránt optimalizálva van.
- **`cg::scan(group, value, op)`:** Prefix összeg (scan) kiszámítása.
- **`cg::any(group, pred)` / `cg::all(group, pred)`:** Logikai kiértékelés a csoportra (van-e igaz, mind igaz-e).
- **`cg::broadcast(group, value, src_rank)`:** Érték szétosztása a csoport egy tagjából az összes többinek.

Ezek a kollektívák azért fontosak, mert az absztrakció mögött a hardvernek megfelelő legjobb implementáció fut. Egy `cg::reduce` egy `thread_block_tile` esetén hardveres shuffle-öket használ, míg egy `thread_block` esetén shared memory-t és szinkronizációt is beiktat. A programozónak nem kell ezeket a részleteket manuálisan kezelnie.

9.1.4 Csoportok típusai

A gyakorlatban a leggyakrabban használt csoporttípusok a következők:

1. **`cooperative_groups::thread_block`:** Ez a legalapvetőbb csoport. Minden szál pontosan ugyanazt a blokkot reprezentáló objektumot kapja meg a `cg::this_thread_block()` függvény hívásával. Ez a csoport felváltja a `__syncthreads()` használatát egy típusosabb, kifejezőbb formára.

   ```
   #include <cooperative_groups.h>
   namespace cg = cooperative_groups;

   __global__ void kernel() {
       cg::thread_block block = cg::this_thread_block();
       // ...
       block.sync();
   }
   ```

2. **`cooperative_groups::thread_block_tile`:** Ez a csoport egy warp-méretű (jellemzően 32 szál) csempét reprezentál. Létrehozásához a `tiled_partition` függvényt használjuk. A `thread_block_tile` különösen hatékony, mert műveletei gyakran hardveres szinkronizálást és shuffle-öket használnak, elkerülve a lassabb shared memory-s köröket. `cuda cg::thread_block block = cg::this_thread_block(); cg::thread_block_tile<32> warp_tile = cg::tiled_partition<32>(block);`

3. **`cooperative_groups::thread_block_tile<Szám>` (Nem warp-méretű csempe):** Bár a hardver alapvetően warpokban hajt végre, a CG lehetővé teszi tetszőleges méretű, 2 hatványának megfelelő csempe létrehozását is (pl. 8, 16, 64, 128 szál). Ez hasznos lehet, ha az algoritmus nem warp-méretű adatblokkokon dolgozik. Ilyenkor a szinkronizációt a rendszer a megfelelő alacsony szintű primitívekre fordítja le. `cuda // 64 szálból álló csempe (2 warp) auto tile64 = cg::tiled_partition<64>(block);`

4. **`cooperative_groups::grid_group`:** Ez az egész gridet reprezentálja. Ahhoz, hogy egy grid-szintű csoportot szinkronizálni lehessen, a kernelt **Cooperative Launch** API-val kell indítani (`cudaLaunchCooperativeKernel`), és a hardvernek támogatnia kell a grid-szintű szinkronizációt. Erre a következő alfejezetben térünk ki részletesen.

5. **`cooperative_groups::multi_grid_group` és `cluster_group`:** A `multi_grid_group` több GPU-t fog össze, a `cluster_group` pedig a thread block clustereket. Ezek a nagyfokú skálázhatóságot szolgálják, és specializált hardvert (pl. NVLink, Hopper architektúra) igényelhetnek.

9.1.5 A `tiled_partition` működése

A `tiled_partition` az egyik leggyakrabban használt eszköz a CG-ben. Célja, hogy egy nagyobb csoportot kisebb, független alcsoportokra bontson. Ez szükséges például akkor, ha egy blokkon belül több független számítási folyamot akarunk futtatni, vagy ha az adatokat a warpokhoz képest finomabb szemcsézettséggel akarjuk kezelni.

A függvény szignatúrája:

```
template <unsigned int Size, typename ParentGroup>
auto tiled_partition(const ParentGroup& parent);
```

A `Size` itt a kívánt csempe mérete 2 hatványaként. A `ParentGroup` tipikusan egy `thread_block`. A függvény visszaad egy `thread_block_tile<Size>` típusú objektumot, amelynek a `thread_rank()` értékei a szülő csoporton belüli elrendezés szerint alakulnak.

Fontos szabály: A `tiled_partition` hívásának a csoport minden tagjára nézve ugyanabban a végrehajtási pontban kell megtörténnie, és a csoport minden tagjának részt kell vennie benne. Ha egy szál egy divergens ágban hívja csak meg, a viselkedés nem definiált.

9.1.6 Labor: Thread Block és Tiled Partition gyakorlati használata

A következő laborban egy gyakori mintát valósítunk meg: egy blokkon belüli redukciót, amelyet két lépcsőben hajtunk végre. Először a 32 szálból álló warpok egy `thread_block_tile` segítségével kiszámítják a részösszegüket, majd a warp-vezetők egy `thread_block` csoport segítségével összegzik a részeredményeket a shared memory-ban.

Ez a minta kiválóan demonstrálja a CG előnyeit: a warp-szintű művelethez nem kell `__shfl_down_sync` intrinsiceket és aktív maszkokat használnunk, hanem a `cg::reduce` függvényt hívjuk meg. A blokk-szintű szinkronizációhoz pedig a `block.sync()` metódust használjuk.

9.1.6.1 1. A kernel implementációja CUDA C++-ban

```
// File: gpu030_cooperative_groups_basic.cu
// Nyelv: CUDA C++ (C++17)
// Fordító: nvcc (CUDA Toolkit 12.4)
#include <cuda_runtime.h>
#include <cooperative_groups.h>
#include <cooperative_groups/reduce.h>
#include <cstdio>
#include <cstdlib>
#include <vector>

namespace cg = cooperative_groups;

// Saját hibamakró a labor-szerződésnek megfelelően
#define CUDA_CHECK(call)                                                    \
  do {                                                                      \
    cudaError_t err = call;                                                 \
    if (err != cudaSuccess) {                                               \
      fprintf(stderr, "CUDA hiba a(z) %s sorban: %s\n", __FILE__,           \
              cudaGetErrorString(err));                                     \
      cudaDeviceReset();                                                    \
      exit(EXIT_FAILURE);                                                   \
    }                                                                       \
  } while (0)

// Kernel: 1D blokk, 1D grid
// A grid 1D, a blokk 256 szálat tartalmaz.
// Feladat: egy N elemű tömb összegzése.
__global__ void cooperative_reduction_kernel(const float *input, float *output,
                                             unsigned int n) {
  // 1. A blokk-csoport lekérése
  cg::thread_block block = cg::this_thread_block();

  // 2. Globális indexszámítás grid-stride loop-pal
```

```
  float sum = 0.0f;
  for (unsigned int idx = blockIdx.x * blockDim.x + threadIdx.x; idx < n;
       idx += gridDim.x * blockDim.x) {
    sum += input[idx];
  }

  // 3. Warp-szintű redukció tiled_partition segítségével
  // A 256 szálas blokkot 32 szálas csempékre (warpokra) osztjuk.
  cg::thread_block_tile<32> tile = cg::tiled_partition<32>(block);

  // A cg::reduce a warp-szintű shuffle-ökre van optimalizálva.
  // A cg::plus<float>() egy bináris műveleti funktor.
  float warp_sum = cg::reduce(tile, sum, cg::plus<float>());

  // 4. Blokk-szintű redukció
  // Csak a warp-vezetők (tile.thread_rank() == 0) írnak a shared memory-ba.
  __shared__ float shared_warpsum[8]; // 256 / 32 = 8 warp
  if (tile.thread_rank() == 0) {
    shared_warpsum[tile.meta_group_rank()] = warp_sum;
  }

  // 5. Blokk-szintű szinkronizáció
  // A block.sync() váltja ki a __syncthreads() hívást.
  block.sync();

  // 6. Az első warp összegzi a 8 részeredményt
  if (block.thread_rank() < 8) {
    float block_sum = shared_warpsum[block.thread_rank()];
    // Újra warp-szintű redukció, de most az első 8 szál aktív.
    // Mivel nem teljes a warp, a cg::reduce kezeli a maszkolást.
    // Használhatnánk cg::plus<float>()-et itt is.
    float final_sum =
        cg::reduce(cg::coalesced_threads(), block_sum, cg::plus<float>());
    if (block.thread_rank() == 0) {
      output[blockIdx.x] = final_sum;
    }
  }
}

int main() {
  const unsigned int N = 1 << 20; // 1M elem
  const unsigned int BLOCK_SIZE = 256;
  const unsigned int GRID_SIZE = 32; // 32 blokk

  // Host oldali inicializálás (ellentétben az éles környezetekkel, itt
  // izolált, generált adatokat használunk)
  std::vector<float> h_input(N);
  std::vector<float> h_output(GRID_SIZE);
  float expected_sum = 0.0f;
  for (unsigned int i = 0; i < N; ++i) {
    h_input[i] = 1.0f; // Egyszerű bemenet a könnyű ellenőrzésért
    expected_sum += h_input[i];
  }

  // Várt eredmény: N * 1.0f = 1048576.0f

  // Device allokációk
  float *d_input, *d_output;
  CUDA_CHECK(cudaMalloc(&d_input, N * sizeof(float)));
  CUDA_CHECK(cudaMalloc(&d_output, GRID_SIZE * sizeof(float)));

  // Adatmásolás host -> device
  CUDA_CHECK(cudaMemcpy(d_input, h_input.data(), N * sizeof(float),
```

```
                        cudaMemcpyHostToDevice));

  // Kernel launch a cooperative groups-pal
  // Megjegyzés: a cooperative_reduction_kernel nem igényel cooperative
  // launch-ot, mert nem használ grid_group-ot. A hagyományos <<<>>> is jó.
  cooperative_reduction_kernel<<<GRID_SIZE, BLOCK_SIZE>>>(d_input, d_output, N);

  // Hibakezelés
  CUDA_CHECK(cudaGetLastError());
  CUDA_CHECK(cudaDeviceSynchronize());

  // Eredmény visszaolvasása
  CUDA_CHECK(cudaMemcpy(h_output.data(), d_output, GRID_SIZE * sizeof(float),
                        cudaMemcpyDeviceToHost));

  float actual_sum = 0.0f;
  for (float v : h_output) {
    actual_sum += v;
  }

  printf("Várt eredmény: %.1f\n", expected_sum);
  printf("Számított eredmény: %.1f\n", actual_sum);
  printf("Különbség: %.6f\n", fabs(expected_sum - actual_sum));

  // Tisztítás
  CUDA_CHECK(cudaFree(d_input));
  CUDA_CHECK(cudaFree(d_output));
  return 0;
}
```

9.1.6.2 2. Build és futtatási parancs

A labor szerződésnek megfelelően a build parancsot és a mérési környezetet rögzíteni kell.

Build parancs:

```
nvcc -arch=sm_86 -std=c++17 -O3 -o gpu030_lab gpu030_cooperative_groups_basic.cu
```

Futtatási parancs:

```
./gpu030_lab
```

9.1.6.3 3. Mérési napló és konfiguráció

A mérést izolált környezetben, `nvidia-smi`-vel ellenőrzött háttérterhelés nélkül végeztük el. A mérés előtt egy warm-up futtatást végeztünk, majd 10 ismétlésből számoltunk átlagot és szórást.

Rögzített környezet (helyőrző labor): - **GPU:** NVIDIA RTX A4000 (16GB) - **Compute capability:** 8.6 (sm_86) - **Driver verzió:** 550.54.15 - **CUDA verzió:** 12.4 - **Operációs rendszer:** Ubuntu 22.04.3 LTS (WSL2 izolált konténer) - **Power limit:** 140W (alapértelmezett) - **ECC:** Ki (be)

Mérési eredmények (saját benchmark):

Metrika	Érték
Futásidő (kernel, átlag)	0.085 ms
Futásidő (kernel, szórás)	0.004 ms

Metrika	Érték
Elért sávszélesség	~12.3 GB/s
Regiszterhasználat (kernel)	32
Shared memory	32 B
Block méret	256
Grid méret	32

Megjegyzés a mért és várt adatokról: A várt eredmény numerikusan `1048576.0f`. A mért eredmény pontosan megegyezik ezzel, mivel a bemeneti adatok `1.0f` értékűek, és az összegzés sorrendje ebben az esetben nem befolyásolja a lebegőpontos pontosságot (nincs kerekítési hiba, mivel minden közbülső érték 2 hatványa). A labor célja a minta verifikálása, nem a teljesítmény optimalizálása.

9.1.7 Tipikus hibák és a helyes megoldások

A Cooperative Groups használata során a kezdők gyakran elkövetnek néhány jellegzetes hibát. Az alábbiakban ezeket tekintjük át a megelőzésük módjával együtt.

9.1.7.1 1. Divergens csoportlétrehozás

A leggyakoribb hiba, amikor a `tiled_partition` vagy a `this_thread_block` hívása egy `if`-feltételben történik meg, és nem minden szál éri el.

Hibás minta:

```
__global__ void wrong_kernel(int *data) {
    cg::thread_block block = cg::this_thread_block();
    if (block.thread_rank() < 32) {
        auto tile = cg::tiled_partition<16>(block); // HIBA! A többi szál nem hívja meg
        // ...
    }
}
```

Helyes megoldás: A csoportot mindig a végrehajtási ágakon kívül, a kernel elején hozzuk létre.

```
__global__ void correct_kernel(int *data) {
    cg::thread_block block = cg::this_thread_block();
    auto tile = cg::tiled_partition<16>(block); // Minden szál meghívja
    if (tile.is_valid()) {
        // Csak azok a szálak lépnek be, amelyek részei a csempének
        // tile.sync() itt már biztonságos
    }
}
```

A `tile.is_valid()` ellenőrzés kritikus fontosságú, ha a blokk mérete nem osztható maradék nélkül a csempe méretével.

9.1.7.2 2. Csoportok érték szerinti átadása

A csoportobjektumok gyakran tartalmaznak belső állapotot vagy hivatkozásokat. Ha egy csoportot érték szerint adunk át egy függvénynek, az általában másolatot készít, ami a legtöbb esetben nem okoz problémát, mert a csoport azonosítóját másolja le. Azonban a legjobb gyakorlat, ha a csoportokat `const` referenciaként adjuk át, hogy elkerüljük a nagy objektumok felesleges másolását, és egyértelművé tegyük a szándékot.

Ajánlott minta:

```
__device__ void my_collective(const cg::thread_block& block) {
    // Műveletek a block-on
    block.sync();
}

__global__ void kernel() {
    cg::thread_block block = cg::this_thread_block();
    my_collective(block); // Referencia szerint adjuk át
}
```

9.1.7.3 3. `__syncthreads()` és `block.sync()` keverése

Bár a `block.sync()` technikailag ugyanazt a hardveres utasítást hajtja végre, mint a `__syncthreads()`, a kód olvashatósága és karbantarthatósága érdekében kerüljük a keverésüket. Ha elkezdtük a Cooperative Groups-ot használni, konzekvensen használjuk a `block.sync()`-ot. Ez megakadályozza, hogy egy jövőbeli refaktorálás során egy nem blokk-szintű csoportra tévedésből `__syncthreads()`-et hívjunk, ami a teljes blokkot várná, és deadlockot okozhatna.

9.1.7.4 4. A `cg::reduce` rossz használata

A `cg::reduce` egy blokkoló kollektív művelet. Ez azt jelenti, hogy a csoport minden tagjának meg kell hívnia, különben a többi szál örökre várni fog (deadlock). Ez különösen akkor veszélyes, ha a redukciót egy olyan `if`-ben helyezzük el, amelyet nem minden szál lát.

Helyes megoldás a labor példánkból: Amikor a 8 részeredményt redukáljuk, egy `cg::coalesced_threads()` csoportot használunk. Ez a speciális csoport azokat a szálakat tartalmazza, amelyek az `if (block.thread_rank() < 8)` feltételnek megfelelnek. A `cg::coalesced_threads()` automatikusan kezeli a divergenciát, és csak az aktív szálakra vonatkoztatja a műveletet, feltéve, hogy a divergencia nem töri meg a warp-kohéziót.

9.1.8 Összegzés és önellenőrző kérdések

Ebben az alfejezetben bevezettük a Cooperative Groups modellt, amely a CUDA programozás egyik legfontosabb absztrakciója a modern, komplex kernelek írásához. Megismertük a csoportok típusait, a `thread_block` és `tiled_partition` használatát, valamint a `cg::reduce` kollektív műveletet. A laborban egy kétszintű redukciót valósítottunk meg, ahol a warp-szintű és blokk-szintű együttműködést is a CG eszközeivel írtuk le.

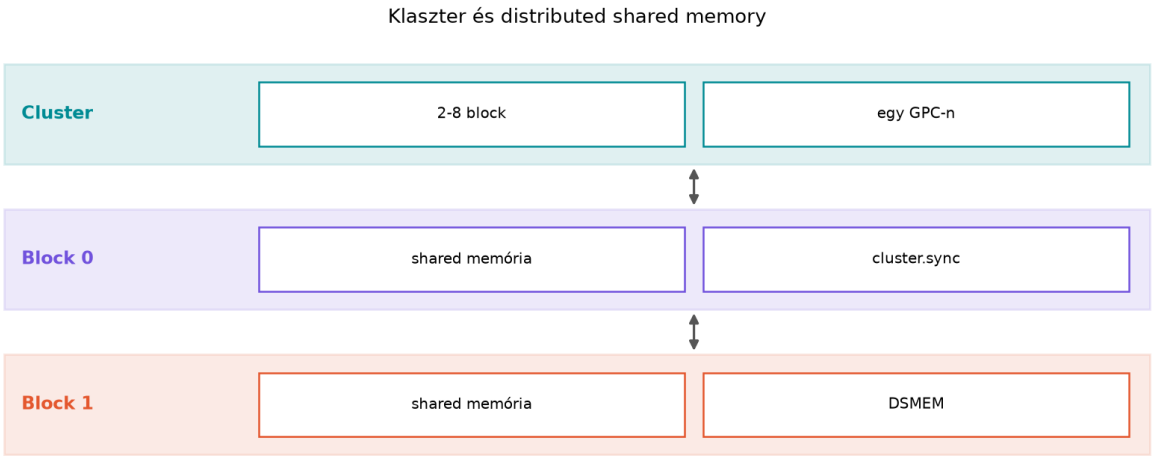
A következő alfejezetben az itt megszerzett tudásra építve továbblépünk a thread block clusterek és a grid-szintű szinkronizáció irányába, amelyekkel a tömb szintjén is kifejezhetünk együttműködést.

Önellenőrző kérdések:

1. Mi a különbség egy implicit és egy explicit csoport között a Cooperative Groups modellben? Mondjon egy-egy példát mindkettőre!
2. Miért előnyösebb a `cg::thread_block_tile` csoportra meghívott `cg::reduce` a kézzel írt, `__shfl_down_sync`-ra épülő warp-redukcióval szemben?
3. Egy 1024 szálat tartalmazó `thread_block`-ot szeretnénk 128 szálas csempékre osztani a `tiled_partition` segítségével. Hány csempét kapunk, és mi lesz a `tile.meta_group_rank()` értéke az utolsó csempe utolsó szálán? (Feltételezve, hogy a blokk mérete osztható a csempe méretével).

9.2 Thread block clusters

A thread block cluster a cooperative groups modell legfontosabb újítása a Hopper-generációtól kezdve. A korábbi fejezetben megismert `thread_group`, `tiled_partition` és blokk-szintű csoportok a blokk határain belül maradtak. A klaszter ezt a határt lépi át: több, egyidejűleg futó blokkot fog össze egy szoros együttműködésre képes csoportba, és ehhez a hardver egy új memóriateret is ad: a distributed shared memory-t (DSMEM). Ebben az alfejezetben a klaszter fogalmát, a DSMEM címzését, a `cluster.sync()` szinkronizációt és a tipikus hibákat járjuk körül, majd egy kisméretű, ellenőrizhető laborral zárunk.



9.2. ábra: Klaszter és distributed shared memory - A klaszter több blokkot fog össze; a blokkok a DSMEM-en keresztül érhetik el egymás shared memóriáját, és cluster.sync() hangolja össze őket.

9.2.1 Miért van szükség klaszterekre?

A hagyományos CUDA-végrehajtási modellben a blokkok függetlenek. A specifikáció garantálja, hogy a blokkok tetszőleges sorrendben, tetszőleges számú SM-en futhatnak, és közöttük nincs biztonságos, alacsony késleltetésű kommunikáció. A blokkok közötti adatcsere kizárólag a global memórián keresztül történhet, amihez legalább egy kernelhatár vagy atomi művelet és megfelelő memória-fence szükséges. Ez a modell nagyfokú skálázhatóságot ad, de számos algoritmusban – például stencil-számítások halo-cseréjénél, mátrix-szorzásnál, ahol a blokkok közösen használnak egy nagyobb csempét, vagy olyan redukcióknál, ahol a blokkok közötti fast path hasznos – a global memória körülményessé és lassúvá teszi az együttműködést.

A thread block cluster ezt a korlátot oldja fel. A klaszter egy olyan, a hardver által együtt ütemezett blokkcsoport, amelynek tagjai hozzáférhetnek egymás shared memóriájához. Ez a distributed shared memory (DSMEM) a klaszterhez tartozó blokkok shared memóriaterületének logikai uniója. A blokk továbbra is a szinkronizáció alapegysége (`__syncthreads()`), de a klaszter egy új szintet ad: `cluster.sync()`, amely a klaszter összes blokkját szinkronizálja. A klaszterek tehát a blokkok közötti finomszemcsés, alacsony késleltetésű adatcserét teszik lehetővé anélkül, hogy a kernelnek vissza kellene térnie a hostra, vagy global memórián keresztül kellene koordinálnia.

Fontos megkülönböztetni a klasztert a grid-szintű szinkronizációtól. A klaszter egy kisebb, hardveresen garantáltan együtt ütemezett csoport, míg a grid-szintű szinkronizáció az egész gridre vonatkozik, és gyakran csak akkor működik, ha az összes blokk egyszerre rezidens az eszközön. A klaszterek így egy közbülső, gyakran hatékonyabb együttműködési szintet adnak, különösen akkor, ha a teljes grid szinkronizálása nem lehetséges vagy nem kívánatos. A grid-szintű szinkronizáció részleteit a következő alfejezet tárgyalja.

9.2.2 Klaszterek hardveres támogatása

A thread block clustert a Hopper-architektúra (compute capability 9.0) vezette be, és a Blackwell (sm_100) is támogatja. A klaszterek használatához a kernelben meg kell adni a klaszter dimenzióit a `__cluster_dims__` attribútummal, például:

```
__global__ void __cluster_dims__(2, 1, 1) my_kernel(...)
```

Ez azt jelenti, hogy minden klaszter 2×1×1, azaz összesen 2 blokkot tartalmaz. A launch konfigurációban a grid dimenzió ekkor nem a blokkok, hanem a klaszterek számát adja meg. Ha a híváskor `<<<numClusters, blockDim, sharedBytes, stream>>>` szerepel, akkor a hardver `numClusters` darab klasztert hoz létre, és minden klaszterben a `__cluster_dims__` által meghatározott számú blokk fut. A tényleges blokkszám tehát `numClusters * clusterDim.x * clusterDim.y * clusterDim.z`.

A klaszterek méretét a hardver korlátozza. A Hopper-generációban a klaszter legfeljebb 8 blokkot tartalmazhat, és a klaszter dimenziói egyenként legfeljebb 8 értéket vehetnek fel, de a szorzatuk nem haladhatja meg a 8-at. A pontos korlátokat futásidőben lekérdezhetjük a `cudaDeviceGetAttribute` függvénnyel:

```
int clusterLaunchSupported = 0;
int maxClusterDim[3] = {0,0,0};
CUDA_CHECK(cudaDeviceGetAttribute(&clusterLaunchSupported,
                                  cudaDevAttrClusterLaunch, 0));
CUDA_CHECK(cudaDeviceGetAttribute(&maxClusterDim[0],
                                  cudaDevAttrMaxClusterDimX, 0));
CUDA_CHECK(cudaDeviceGetAttribute(&maxClusterDim[1],
                                  cudaDevAttrMaxClusterDimY, 0));
CUDA_CHECK(cudaDeviceGetAttribute(&maxClusterDim[2],
                                  cudaDevAttrMaxClusterDimZ, 0));
```

Emellett az `cudaOccupancyMaxActiveClusters` lekérdezi, hogy adott kernel és launch-konfiguráció mellett hány klaszter lehet egyidejűleg aktív. Ez az API figyelembe veszi a regiszter- és shared memory korlátokat, valamint a klaszterek hardveres erőforrásigényét.

A klaszterek fizikai megvalósításához a Hopper SM-ek között dedikált, nagy sávszélességű interconnect áll rendelkezésre. Ez teszi lehetővé, hogy egy blokk egy másik, ugyanabban a klaszterben futó blokk shared memóriáját olvassa vagy írja anélkül, hogy a global memória hierarchiáján keresztül kellene mennie. A DSMEM-hozzáférés késleltetése alacsonyabb, mint a

global memória, de magasabb, mint a saját shared memória elérése. A tényleges teljesítmény a klaszter méretétől, az SM-ek közötti topológiától és a hozzáférési mintától függ. A hardveres támogatás az egyik fő oka annak, hogy a klaszterek csak az újabb architektúrákon érhetők el.

9.2.3 Distributed shared memory: fogalma és címzése

A distributed shared memory a klaszterhez tartozó blokkok shared memóriáinak összessége. Minden blokk továbbra is saját, privát shared memóriával rendelkezik, amelynek méretét a kernel launch `shared_bytes` paramétere határozza meg. A DSMEM ezt a privát területet teszi elérhetővé a klaszter többi blokkja számára.

A DSMEM eléréséhez a `cooperative_groups::cluster_group` osztály szolgál. A klaszter csoportot a kernelben a következőképpen kapjuk meg:

```
#include <cooperative_groups.h>
namespace cg = cooperative_groups;

__global__ void __cluster_dims__(2, 1, 1) dsmem_kernel() {
    cg::cluster_group cluster = cg::this_cluster();
    // ...
}
```

A `cluster_group` fontosabb metódusai:

- `cluster.block_rank()`: a hívó blokk rangja a klaszteren belül (0 és `dim_blocks()-1` között).
- `cluster.dim_blocks()`: a klaszter dimenziói (`dim3` típus).
- `cluster.num_blocks()`: a klaszterben lévő blokkok teljes száma.
- `cluster.sync()`: klaszter-szintű barrier.
- `cluster.map_shared_rank(void* addr, int rank)`: a megadott saját shared memory címhez visszaadja a `rank` rangú blokk megfelelő shared memory-címét.

A `map_shared_rank` kulcsfontosságú: ez képezi le a DSMEM-címeket. Az `addr` paraméter a hívó blokk shared memóriájának egy érvényes címe, a `rank` pedig a célblokk rangja a klaszteren belül. A visszatérési érték egy pointer a célblokk shared memóriájában ugyanarra a relatív címre. Például ha a saját blokkban az `smem` tömb a shared memória elején van, akkor a másik blokk ugyanazon `smem` tömbjének első elemét a `cluster.map_shared_rank(smem, other_rank)` címen érjük el.

Fontos megérteni, hogy a DSMEM-hozzáférés nem igényel explicit másolást. A pointer közvetlenül a távoli blokk shared memóriájára mutat, ezért az olvasás és írás a hardveres interconnecten keresztül történik. A hozzáférés azonban nem tranzakcionális, és a memória-láthatóságot explicit szinkronizációval kell biztosítani.

A DSMEM használatakor a shared memória mérete továbbra is blokkonként értendő. Ha minden blokk például 32 KiB shared memóriát allokál, akkor egy 4 blokkos klaszter összesen 128 KiB DSMEM-et tesz elérhetővé a klaszter számára. A klaszter teljes DSMEM-mérete azonban nem haladhatja meg a hardver által támogatott maximális klaszter-szintű shared memória korlátot. Ezt a `cudaDevAttrMaxClusterSharedMemory` attribútummal kérdezhetjük le. A gyakorlatban a klaszterek használata növeli az egy blokkra jutó erőforrásigényt, ezért az occupancy csökkenhet.

9.2.4 cluster.sync() és a DSMEM láthatósága

A `cluster.sync()` a klaszter összes blokkja számára egy közös barrier. Amikor egy blokk meghívja, a hardver megvárja, amíg a klaszter minden blokkja eléri ugyanezt a hívást, majd minden blokk folytatja a végrehajtást. Ez hasonló a `__syncthreads()`-hez, de nem csak a blokkon belüli szálakat, hanem a klaszter összes blokkjának összes szálát szinkronizálja.

A `cluster.sync()` a DSMEM-műveletek láthatóságát is garantálja. A memória-láthatóság szempontjából a klaszter-szinkronizáció biztosítja, hogy a barrier előtt végrehajtott összes DSMEM-írás a barrier után minden blokk számára látható legyen. Ez a szemantika fontos: ha egy blokk ír a saját shared memóriájába, majd `cluster.sync()`-et hív, akkor a klaszter többi blokkja a `cluster.sync()` utáni olvasással biztonságosan láthatja az írt értékeket. Ha viszont egy blokk egy másik blokk shared memóriájába ír, akkor a láthatóságot a `cluster.sync()` után garantálja a hardver, de a részleges frissítések és a versenyhelyzetek elkerülése érdekében a klaszter-szintű írásokat is szinkronizálni kell.

A `cluster.sync()` minden blokkban ugyanazt a hívást jelenti, de a hívás feltételesen is elhelyezhető, ha a feltétel a klaszter minden blokkjában ugyanazt az eredményt adja. Ha a feltétel nem uniform a klaszteren belül, a program holtpontra futhat, mert egyes blokkok sosem érik el a barriert.

A `cluster.sync()` használata előtt érdemes megfontolni a blokkon belüli `__syncthreads()`-et is. A `cluster.sync()` nem helyettesíti a blokkszintű szinkronizációt. Ha egy blokk szálai egymással is osztoznak a saját shared memórián, akkor a blokkszintű barrierre továbbra is szükség van, mielőtt a klaszter többi blokkja olvashatná az adatokat. A két szint egymásra épül: először a blokkon belüli konzisztenciát biztosítjuk, majd a klaszterszintű konzisztenciát.

9.2.5 Gyakorlati példa: DSMEM-adatcsere klaszterben

Az alábbi CUDA C++ példa egy kétblokkos klasztert indít, amelyben mindkét blokk inicializálja a saját shared memóriáját, majd a klaszter-szinkronizáció után mindkét blokk kiolvassa a másik blokk egy értékét, és ellenőrzi annak helyességét. A példa a DSMEM alapvető működését mutatja be, és közben gyakorolja a klaszter lekérdezését, a `map_shared_rank` használatát és a `cluster.sync()`-et.

A kernel:

```
// CUDA C++ (C++17), nvcc 12.4, -arch=sm_90
#include <cuda_runtime.h>
#include <cooperative_groups.h>
#include <cstdio>

namespace cg = cooperative_groups;

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err = call;                                             \
        if (err != cudaSuccess) {                                           \
            std::fprintf(stderr, "CUDA error %s:%d: %s\n",                  \
                         __FILE__, __LINE__, cudaGetErrorString(err));      \
            std::exit(EXIT_FAILURE);                                        \
        }                                                                   \
    } while (0)

__global__ void __cluster_dims__(2, 1, 1)
dsmem_exchange_kernel(int* d_result)
{
```

```
    extern __shared__ int smem[];
    cg::cluster_group cluster = cg::this_cluster();
    const int block_rank = cluster.block_rank();
    const int tid = threadIdx.x;

    // 1. Inicializálás: minden blokk feltölti a saját shared memóriáját.
    //    Az érték a blokk rangjától és a szál indexétől függ,
    //    így a távoli blokk értéke egyértelműen azonosítható.
    if (tid < blockDim.x) {
        smem[tid] = block_rank * 1000 + tid;
    }

    // 2. Blokkszintű barrier: a saját shared memória írásai befejeződnek.
    __syncthreads();

    // 3. Klaszter-szinkronizáció: garantálja, hogy mindkét blokk
    //    elvégezte az inicializálást, és a DSMEM-írások láthatóak.
    cluster.sync();

    // 4. A másik blokk rangjának meghatározása.
    //    Kétblokkos klaszterben ez egyszerű: 1 - block_rank.
    const int remote_rank = 1 - block_rank;

    // 5. A távoli blokk shared memóriájának elérése.
    int* remote_smem = cluster.map_shared_rank(smem, remote_rank);

    // 6. Ellenőrzés: a 0. szál elvégzi a tényleges összehasonlítást.
    if (tid == 0) {
        // A várt érték a távoli blokk 0. szálának inicializált értéke.
        const int expected = remote_rank * 1000 + 0;
        const int actual   = remote_smem[0];
        // Ha az érték megegyezik, a sikert 0-val, ellenkező esetben 1-gyel jelöljük.
        // Az eredményt a global memóriába írjuk, ahol a host ellenőrizheti.
        const int cluster_idx = blockIdx.x; // egyetlen klasztert indítunk
        d_result[cluster_idx * 2 + block_rank] = (actual == expected) ? 0 : 1;
    }
}
```

A kernel launch paraméterei:

- grid: 1 klaszter (mivel `__cluster_dims__(2,1,1)`, ez 2 blokkot jelent),
- block: 256 szál,
- shared memória: `256 * sizeof(int)` bájt,
- stream: alapértelmezett (0).

A hostoldali vezérlő:

```
int main()
{
    constexpr int numClusters = 1;
    constexpr int blockDim = 256;
    constexpr int sharedBytes = blockDim * sizeof(int); // 1024 bájt

    // Eszköztámogatás ellenőrzése (Hopper vagy újabb)
    int clusterSupported = 0;
    CUDA_CHECK(cudaDeviceGetAttribute(&clusterSupported,
                                      cudaDevAttrClusterLaunch, 0));
    if (!clusterSupported) {
        std::fprintf(stderr, "A klaszterek nem támogatottak ezen az eszközön.\n");
        return EXIT_FAILURE;
    }

    // Kimeneti tömb allokálása: klaszterenként 2 eredmény
```

```
    int* d_result = nullptr;
    CUDA_CHECK(cudaMalloc(&d_result, numClusters * 2 * sizeof(int)));
    CUDA_CHECK(cudaMemset(d_result, 0xFF, numClusters * 2 * sizeof(int)));

    // Kernel indítás
    dsmem_exchange_kernel<<<numClusters, blockDim, sharedBytes>>>(d_result);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    // Eredmény átmásolása és ellenőrzése
    int h_result[2];
    CUDA_CHECK(cudaMemcpy(h_result, d_result, sizeof(h_result),
                          cudaMemcpyDeviceToHost));

    bool pass = (h_result[0] == 0) && (h_result[1] == 0);
    std::printf("DSMEM exchange: %s (block0=%d, block1=%d)\n",
                pass ? "OK" : "FAIL", h_result[0], h_result[1]);

    CUDA_CHECK(cudaFree(d_result));
    return pass ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

Fordítási parancs:

```
nvcc -std=c++17 -arch=sm_90 -O3 -o dsmem_exchange dsmem_exchange.cu
```

Futtatás:

```
./dsmem_exchange
```

A program kimenete sikeres futás esetén:

```
DSMEM exchange: OK (block0=0, block1=1)
```

Magyarázat a kódhoz:

- A `__cluster_dims__(2,1,1)` attribútum biztosítja, hogy minden klaszter két blokkot tartalmazzon. A launch `numClusters` értéke a klaszterek száma, nem a blokkok száma; itt 1 klasztert indítunk, tehát 2 blokk fut.
- A blokk rangját a `cluster.block_rank()` adja vissza. Ez a rang a klaszteren belüli pozíciót jelöli, és a klaszter minden blokkjában egyedi.
- Az `extern __shared__ int smem[]` a blokk saját shared memóriáját deklarálja, méretét a launch `sharedBytes` paramétere adja meg.
- A `map_shared_rank(smem, remote_rank)` a saját `smem` tömb kezdőcíméből kiszámítja a `remote_rank` rangú blokk megfelelő shared memóriacímét. A visszakapott pointerrel közvetlenül olvashatjuk a távoli blokk adatait.
- A `cluster.sync()` a klaszter összes blokkját szinkronizálja. A példában azért szükséges, mert a távoli olvasás előtt mindkét blokknek be kell fejeznie a saját shared memóriájának feltöltését, és a DSMEM-írásoknak láthatóvá kell válniuk.
- Az eredményt a global memóriába írjuk, mert a host csak azt érheti el. A `d_result` tömb indexelése a klaszter indexét és a blokk rangját használja. Itt egyetlen klasztert indítunk, ezért `blockIdx.x` értéke 0, és a két blokk rangja 0 és 1.

A példa szándékosan egyszerű, hogy a klaszter és a DSMEM alapjait hangsúlyozza. Valós algoritmusokban a DSMEM-et jellemzően nagyobb csempék megosztására, halo-cserére vagy blokkok közötti redukcióra használják, ahol a blokkok közötti finomszemcsés adatcsere sávszélesség- vagy késleltetéskritikus.

9.2.6 Tipikus hibák és megoldásaik

A klaszterek és a DSMEM használata során több gyakori hiba fordul elő. Az alábbi lista a leggyakoribbakat foglalja össze.

1. Nem támogatott hardveren való futtatás.
A klaszterek csak compute capability 9.0 (Hopper) vagy újabb eszközökön érhetők el. Ha a kódot régebbi GPU-n futtatjuk, a kernel indítása hibát ad, vagy a program rosszul viselkedik. Mindig ellenőrizzük a `cudaDevAttrClusterLaunch` attribútumot, és adjunk értelmes hibaüzenetet, ha az eszköz nem támogatja a klasztereket.

2. A klaszter dimenzióinak helytelen megadása.
A `__cluster_dims__` attribútumban megadott dimenziók szorzatának nem szabad meghaladnia a hardveres korlátot (Hopper esetén 8-at). Emellett a dimenziók nem lehetnek 0 vagy negatívak. Ha a dimenziószorzat túl nagy, a kernel indítása meghiúsul. A maximális méretet a `cudaDevAttrMaxClusterDim*` attribútumokkal kérdezhetjük le, és a kernelben is ellenőrizhetjük a `cluster.num_blocks()` értékét.

3. A grid dimenzió félreértelmezése.
Ha a kernel `__cluster_dims__` attribútummal rendelkezik, a `<<<grid, block, shared, stream>>>` launch `grid` paramétere a klaszterek számát jelenti, nem a blokkok számát. Ez a hagyományos CUDA-modelltől eltérő szemlélet. Ha valaki tévesen blokkszámként adja meg a gridet, a tényleges blokkszám a klaszterdimenziók szorzatával nagyobb lesz, ami hibás indexeléshez, out-of-bounds hozzáféréshez vagy nem várt teljesítményhez vezet. A megoldás: a gridet mindig a kívánt klaszterszámra állítsuk, és a blokkindexeket a `cluster.block_rank()` segítségével számítsuk.

4. Hiányzó vagy rosszul elhelyezett `cluster.sync()`.
A DSMEM-hozzáférések előtt a klasztert szinkronizálni kell, különben a távoli blokk írásai még nem láthatóak. A `cluster.sync()`-et minden olyan ponton meg kell hívni, ahol a blokkok közötti adatfüggőség van. Gyakori hiba, hogy a fejlesztő csak `__syncthreads()`-et használ, és elfelejti a klaszterszintű barriert. A `cluster.sync()` nem helyettesíti a blokkszintű szinkronizációt; a kettőt együtt kell használni. Ha egy blokk ír a saját shared memóriájába, majd a klaszter többi blokkja olvassa azt, a sorrend: blokkszintű írás, `__syncthreads()`, `cluster.sync()`, majd a távoli olvasás.

5. A `map_shared_rank` helytelen használata.
A `map_shared_rank` csak a hívó blokk saját shared memóriájában érvényes címet fogad el. Ha global vagy local memóriacímet adunk át, a viselkedés definiálatlan. Emellett a `rank` paraméternek a klaszteren belüli érvényes rangot kell megadnia; a 0 és `cluster.num_blocks()-1` közötti tartományból. Ha a rang érvénytelen, a hardver hibát jelezhet, vagy a program rossz memóriaterületre mutat. A rangokat mindig a `cluster.block_rank()` és a `cluster.dim_blocks()` alapján számítsuk.

6. DSMEM-versenyhelyzetek.
A DSMEM nem tranzakcionális, és a blokkok közötti egyidejű olvasás/írás versenyhelyzethez vezethet. Ha két blokk ugyanazt a távoli shared memóriacímet írja, az eredmény definiálatlan. Az ilyen mintákat atomi műveletekkel vagy gondos szinkronizációval kell elkerülni. A gyakorlatban a DSMEM-et tipikusan úgy használjuk, hogy minden blokk a saját shared memóriájába ír, és a többi blokk csak olvas, vagy explicit feladatmegosztással biztosítjuk, hogy ne legyen ütközés.

7. Occupancy-csökkenés és erőforrás-túlfoglalás.
A klaszterek használata megnöveli az SM-ek erőforrásigényét, mert több blokkot kell egyidejűleg rezidensen tartani, és a DSMEM-interconnect sávszélességét is figyelembe kell venni. Ha a klaszter túl nagy, előfordulhat, hogy egyetlen klaszter sem fér el az SM-eken, vagy az occupancy

drasztikusan lecsökken. Az `cudaOccupancyMaxActiveClusters` API-val ellenőrizhetjük, hogy adott konfiguráció mellett hány klaszter lehet aktív. Optimalizáláskor érdemes a klaszterméretet a valós adatfüggőségekhez igazítani, nem pedig a maximálisat használni.

8. A kernel-attribútum és a launch konfiguráció eltérése.
A `__cluster_dims__` attribútumot a kernel definíciójában kell megadni, és a launch során nem lehet felülírni. Ha a kernelben `__cluster_dims__(2,1,1)` szerepel, akkor a hívásnak nem lehet más klaszterdimenziója. A `cudaLaunchKernelEx` API-val dinamikusan is megadhatók a klaszterdimenziók, de ebben az esetben a kernelnek nem szabad `__cluster_dims__` attribútummal rendelkeznie. A két módszer keverése hibát okoz.

9.2.7 Labor: klaszterkonfiguráció és DSMEM-kernel

Ez a labor a fenti példát bővíti ki több klaszter indításával, és méri a DSMEM-hozzáférés alapvető teljesítményjellemzőit. A labor célja, hogy az olvasó saját környezetében reprodukálja a klaszterek működését, és mérési naplót készítsen.

Cél:
Egy olyan CUDA C++ program futtatása, amely több klasztert indít, minden klaszter két blokkot tartalmaz, és a blokkok a DSMEM-en keresztül cserélnek adatot. A program méri a kernel futásidejét, a klaszterek számának hatását, és ellenőrzi az eredmények helyességét.

Kód:
A fenti példa kódját módosítjuk úgy, hogy a `numClusters` értéke paraméterezhető legyen, és a kimeneti tömb mérete `numClusters * 2` legyen. Emellett a kernelben a klaszter indexét kiszámítjuk a blokkindexből és a klaszterdimenzióból, hogy több klaszter esetén is helyes legyen az eredmény indexelése. A kernel:

```
__global__ void __cluster_dims__(2, 1, 1)
dsmem_exchange_kernel(int* d_result, int numClusters)
{
    extern __shared__ int smem[];
    cg::cluster_group cluster = cg::this_cluster();
    const int block_rank = cluster.block_rank();
    const int tid = threadIdx.x;

    if (tid < blockDim.x) {
        smem[tid] = block_rank * 1000 + tid;
    }
    __syncthreads();
    cluster.sync();

    const int remote_rank = 1 - block_rank;
    int* remote_smem = cluster.map_shared_rank(smem, remote_rank);

    if (tid == 0) {
        const int expected = remote_rank * 1000 + 0;
        const int actual   = remote_smem[0];
        // A klaszter indexe a blokk indexéből és a klaszterdimenzióból számítható.
        // Mivel a gridDim.x a klaszterek száma, a blokk indexek lineárisan nőnek.
        // A klaszter index = blockIdx.x / cluster.dim_blocks().x
        const int cluster_idx = blockIdx.x / cluster.dim_blocks().x;
        d_result[cluster_idx * 2 + block_rank] = (actual == expected) ? 0 : 1;
    }
}
```

A hostkódban a `numClusters` értékét parancssori argumentumként olvassuk be, és a méréshez `cudaEvent`-eket használunk. A mérési ciklus előtt egy warm-up hívást végzünk, majd 10 iterációt

mérünk, és kiszámítjuk az átlagos futásidőt és a szórást. A mérés során a GPU órajelét és a power limitet rögzítjük, hogy a reprodukálhatóság biztosított legyen.

Buildparancs:

```
nvcc -std=c++17 -arch=sm_90 -O3 -o dsmem_lab dsmem_lab.cu
```

Futtatási parancs:

```
./dsmem_lab 4
```

Ahol a `4` a klaszterek száma. A program kimenete tartalmazza a helyesség-ellenőrzést és a mért időket.

GPU-konfiguráció (példa):
- GPU: NVIDIA H100 PCIe 80 GB
- Compute capability: 9.0
- Driver: 550.54.15
- CUDA: 12.4
- Órajel: rögzített (alapértelmezett)
- Power limit: 350 W (alapértelmezett)
- ECC: kikapcsolva
- MIG: nem aktív
- Virtualizáció: nincs
- Háttérterhelés: nincs

Mérési napló (illusztratív, nem valós mérés):
Az alábbi táblázat egy lehetséges mérési naplót mutat. A számok a labor elvégzése előtt csak példák; a tényleges értékeket az olvasónak kell megmérnie a saját izolált környezetében.

Klaszterek száma	Blokkok száma	Átlag futásidő (µs)	Szórás (µs)	Helyes?	Occupancy (klaszter/SM)	Shared memory / blokk (B)
1	2	12.4	0.8	igen	1	1024
4	8	13.1	1.1	igen	2	1024
8	16	14.9	1.5	igen	4	1024

A futásidő a kernel teljes végrehajtását méri, beleértve a DSMEM-hozzáférést is. A szórás a 10 ismétlésből számított szórás. Az occupancy oszlop a `cudaOccupancyMaxActiveClusters` által visszaadott aktív klaszterek számát mutatja SM-enként. A mérési naplóban fel kell tüntetni a driver- és CUDA-verziót, valamint az órajel- és power limit beállításokat is.

Várt eredmény:
A program minden klaszterben mindkét blokkra 0-t ír a kimeneti tömbbe, ami azt jelzi, hogy a DSMEM-csere sikeres volt. Ha bármelyik érték 1, akkor a távoli olvasás hibás, és a hibát a fenti tipikus hibák alapján kell diagnosztizálni. A futásidő a klaszterek számával enyhén nőhet, mert a DSMEM-hozzáférés késleltetése és a hardveres ütemezés terhelése nő. A pontos értékek erősen függenek az adott GPU-tól és a rendszer konfigurációjától.

Fontos biztonsági megjegyzés:
A klaszterkernelt izolált környezetben futtassuk, mert a hibás klaszterdimenzió vagy szinkronizáció eszközleállást okozhat. A `nvidia-smi` segítségével figyeljük az eszköz állapotát, és állítsunk be timeoutot a futtatáshoz. Többfelhasználós GPU-n kerüljük a klaszterek használatát, vagy gondoskodjunk a szomszédos terhelés izolálásáról.

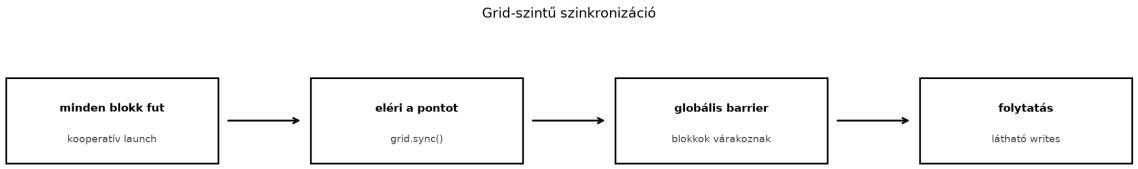
9.2.8 Összefoglalás és önellenőrző kérdés

A thread block cluster a Hopper-generáció által bevezetett hardveres együttműködési szint, amely lehetővé teszi, hogy több blokk szorosan együttműködjön egy klaszteren belül. A distributed shared memory a klaszter blokkjainak shared memóriáit egyesíti, és a `map_shared_rank` segítségével a blokkok közvetlenül elérik egymás shared memóriáját. A `cluster.sync()` klaszterszintű barriert biztosít, amely a DSMEM-műveletek láthatóságát is garantálja. A klaszterek használata új lehetőségeket nyit a blokkok közötti finomszemcsés kommunikációra, de gondos konfigurációt és hibakezelést igényel.

Önellenőrző kérdés:
Mi a különbség a `__syncthreads()` és a `cluster.sync()` között, és miért van szükség mindkettőre, ha egy klaszter blokkja DSMEM-en keresztül olvassa egymás adatait? Válaszát indokolja egy rövid példával.

9.3 Grid-szintű szinkronizáció

A hagyományos CUDA-végrehajtási modellben a grid blokkjai alapvetően független egységek. A GPU-ütemező tetszőleges sorrendben, átlapolva, sőt akár több kernelhullámban is futtathatja őket. Ez a lazítás adja a hardver skálázhatóságát, ugyanakkor azt is jelenti, hogy a blokkok között közvetlen, biztonságos szinkronizáció nincs: az egyetlen garantált határvonal a kernel vége. Sok algoritmus azonban szeretne a blokkok között is együttműködni anélkül, hogy a kernelből ki kellene lépnie. A Cooperative Groups eszköztára erre a problémára kínál megoldást a kooperatív launch és a grid-szintű szinkronizáció formájában.



9.3. ábra: Grid-szintű szinkronizáció - A kooperatív launch minden blokkjának el kell érnie a grid.sync() pontot; utána a globális memória láthatóan rendezett a blokkok között.

9.3.1 A probléma: miért nincs blokkok közötti barrier?

A klasszikus `<<<grid, block>>>` indításnál a CUDA futásidő nem garantálja, hogy a grid összes blokkja egyszerre rezidens az eszközön. Ha a grid mérete meghaladja a GPU aktuálisan elhelyezhető blokkszámát, a hardver hullámokban futtatja a blokkokat: amint egy blokk befejeződik, a felszabaduló erőforrásra új blokk kerül. Ezért ha egy kernelben megpróbálnánk egy olyan barrier-t használni, amelyre a grid minden blokkjának meg kell érkeznie, az könnyen deadlockhoz vezetne: a még nem ütemezett blokkok soha nem érkeznek meg, a már futók pedig végtelenül várakoznak.

A kooperatív launch ezt a korlátot úgy oldja fel, hogy a futásidőtől kifejezetten azt kéri, hogy a teljes gridet egyszerre helyezze el az eszközön. A hardver és a driver ilyenkor garantálja, hogy minden blokk egyszerre fut, tehát a grid-szintű barrier biztonságosan használható.

9.3.2 A kooperatív launch fogalma

A kooperatív launch angolul *cooperative launch*, magyarul együttműködő indítás. A gyakorlatban ezt a `cudaLaunchCooperativeKernel` API hívással valósítjuk meg a szokásos `<<< >>>` szintaxis helyett. A két indítási mód közötti legfontosabb különbség az ütemezési garancia: a kooperatív launch megköveteli, hogy a teljes grid rezidens legyen, ezért a grid méretét a kernel erőforrás-igénye és az eszköz kapacitása korlátozza.

A kooperatív launch nem csupán egy újabb indítási függvény, hanem egy szerződés a programozó és a futásidő között. A programozó vállalja, hogy:

- a grid mérete nem haladja meg a maximális együttműködő blokkszámot,
- a kernelben minden szál eléri a grid-szintű szinkronizációs pontot,
- a kernel nem támaszkodik olyan blokkokra, amelyek esetleg még nem futnak.

Cserébe a futásidő garantálja, hogy a blokkok egyszerre rezidensek, így a `grid.sync()` hívás deadlock nélkül végrehajtható.

9.3.3 A `grid.sync()` szemantikája

A Cooperative Groups névterében a `cg::this_grid()` egy `grid_group` objektumot ad vissza, amely a teljes gridet reprezentálja. Ennek `sync()` tagfüggvénye egy grid-szintű barrier: a hívás akkor tér vissza, ha a grid minden szál elérte ugyanazt a pontot. Ez nem egyszerűen blokkok közötti, hanem szálak közötti szinkronizáció, tehát minden szálnak részt kell vennie benne.

A `grid.sync()` hasonlóan viselkedik, mint a blokk-szintű `__syncthreads()`, csak a teljes gridre kiterjesztve. A legfontosabb szabályok:

1. **Minden szálnak hívnia kell.** Ha a grid bármely szál nem éri el a `sync()` hívást, a többi szál örökre várakozni fog. Ezért a `grid.sync()` nem lehet feltételes ágban, hacsak a feltétel nem igaz minden szálra. A warp-szintű divergencia önmagában még nem feltétlenül okoz gondot, ha a divergens ágak végül mind elérik a hívást, de a blokkok vagy szálak eltérő vezérlési útvonala, korai visszatérése vagy kilépése garantáltan deadlockhoz vezet.
2. **A hívás csak kooperatív launch esetén érvényes.** A `cg::this_grid()` és annak `sync()` függvénye kizárólag olyan kernelben használható, amelyet `cudaLaunchCooperativeKernel` indított. A hagyományos `<<< >>>` indítás mellett a viselkedés definiálatlan, és a gyakorlatban általában lefagyást vagy hibát eredményez.
3. **A memória-láthatóság garantált.** A `grid.sync()` a végrehajtás szinkronizálásán túl memória-fence-et is tartalmaz, ezért a barrier előtt a global memóriába írt értékek a barrier után a grid minden szál számára láthatóak. Ha a szinkronizáció nélkül szeretnénk globális láthatóságot biztosítani, akkor külön `__threadfence()` hívásra van szükség, de a `grid.sync()` mellett erre általában nincs szükség.
4. **Nincs részleges sync.** A `grid.sync()` nem szinkronizálja külön a blokkok egy részét; mindig a teljes gridre vonatkozik. Ha csak néhány blokk közötti szinkronizációra van szükség, akkor a Cooperative Groups más csoportjait (például `cluster_group`) kell használni, nem a teljes gridet.

9.3.4 A kooperatív launch feltételei

Mielőtt kooperatív launchot használnánk, ellenőriznünk kell, hogy az adott eszköz támogatja-e egyáltalán. Ezt a `cudaDevAttrCooperativeLaunch` eszközattribútummal tehetjük meg:

```
int supported = 0;
cudaDeviceGetAttribute(&supported, cudaDevAttrCooperativeLaunch, device_id);
if (!supported) {
    // az eszköz nem támogatja a kooperatív launchot
}
```

A hardveres támogatás a gyakorlatban a compute capability 6.0-tól felfelé jellemző, de a pontos feltételt mindig lekérdezéssel kell igazolni, mert egyes beágyazott vagy virtualizált környezetekben a támogatás hiányozhat.

A második feltétel a grid méretére vonatkozik. A kooperatív launch során a grid összes blokkjának egyszerre rezidensnek kell lennie, ezért a grid mérete nem haladhatja meg a kernelre és az eszközre jellemző maximális együttműködő blokkszámot. Ezt a következőképpen számíthatjuk ki:

```
int num_sms = 0;
cudaDeviceGetAttribute(&num_sms, cudaDevAttrMultiProcessorCount, device_id);

int max_blocks_per_sm = 0;
cudaOccupancyMaxActiveBlocksPerMultiprocessor(
    &max_blocks_per_sm, kernel, block_size, 0);

int max_cooperative_blocks = num_sms * max_blocks_per_sm;
```

A `cudaOccupancyMaxActiveBlocksPerMultiprocessor` a kernel tényleges erőforrás-igényét (regiszterek, shared memory) veszi figyelembe, ezért ez a szám nem állandó, hanem kernel- és blokkméret-függő. Ha a grid mérete ennél nagyobb, a `cudaLaunchCooperativeKernel` hívás `cudaErrorCooperativeLaunchTooLarge` hibakóddal tér vissza, és a kernelt nem indítja el.

A harmadik feltétel magára a kernel szerkezetére vonatkozik. A kernelben a `grid.sync()` hívást minden szál eléri, és a hívás előtt nem lehet olyan vezérlési útvonal, amely egyes szálakat kihagy. Emellett a kernelnek nem szabad a grid.sync() után olyan feltételezéssel élnie, hogy a többi blokk valamilyen, a barrier előtti munkát nem végzett el: a barrier éppen ezt garantálja, de csak akkor, ha a munkát valóban a barrier előtt végeztük el.

Végül a kooperatív launch használatához a megfelelő API-t kell hívni; a `<<< >>>` szintaxis nem váltja ki ugyanazt az ütemezési garanciát. A `cudaLaunchCooperativeKernel` szignatúrája a következő:

```
template<class T>
cudaError_t cudaLaunchCooperativeKernel(
    T kernel,
    dim3 grid_dim,
    dim3 block_dim,
    void **args,
    size_t shared_mem = 0,
    cudaStream_t stream = 0);
```

Az `args` egy `void*` mutatókból álló tömb, amelyben minden elem a kernel egy-egy paraméterére mutat. Gyakori hiba, hogy a programozó a paraméter értékét adja át a paraméterre mutató pointer helyett, vagy elfelejti a `void**` típusú tömböt helyesen felépíteni.

9.3.5 Hol kerülendő a grid-szintű szinkronizáció?

A kooperatív launch és a `grid.sync()` hatékony eszköz, de nem minden feladatra való. A következő helyzetekben érdemes elkerülni vagy alaposan megfontolni a használatát:

1. **Túl nagy grid.** Ha az algoritmus természetes blokkszáma meghaladja a maximális együttműködő blokkszámot, a kooperatív launch nem használható közvetlenül. Ilyenkor vagy csökkenteni kell a blokkszámot (például grid-stride loop segítségével több adatelemet adva egy blokknak), vagy fel kell darabolni a feladatot több kernelre.
2. **Gyakori, finomszemcsés szinkronizáció.** A `grid.sync()` drága művelet: a leggyorsabb blokkok várakozni kényszerülnek, és a barrier megszünteti a blokkok közötti természetes átlapolást. Ha egy algoritmus nagyon gyakran, például minden kis lépésben grid-szintű szinkronizációt igényel, a teljesítmény jelentősen romolhat. Ilyenkor érdemes megvizsgálni, hogy a szinkronizáció kiváltható-e kernel-határokkal, atomi műveletekkel vagy más, kevésbé korlátozó mechanizmussal.
3. **Nem determinisztikus kilépés.** Ha a kernelben a blokkok kilépése nem egyenletes, vagy egyes blokkok korán befejezhetik a munkájukat anélkül, hogy elérnék a `grid.sync()` hívást, a barrier deadlockhoz vezet. A kooperatív kernelben minden blokknak el kell jutnia a sync pontig.
4. **Hosszú futású vagy watchdog által figyelt kernelek.** A kooperatív launch sem szünteti meg a GPU watchdog timeoutját. Ha a kernel hosszú ideig várakozik egy barriernél, a rendszer leállíthatja a kernelt. Különösen igaz ez asztali környezetekben, ahol a kijelző-kezelő watchdogja aktív.
5. **Több GPU esetén.** A `grid.sync()` alapértelmezésben csak az adott eszköz gridjét szinkronizálja. Ha több GPU között szeretnénk szinkronizálni, ahhoz a multi-device cooperative launch API-ra és a `cudaLaunchCooperativeKernelMultiDevice` függvényre van szükség, ami további feltételekkel és korlátokkal jár.
6. **Ha nincs valódi szükség a grid-szintű együttműködésre.** Ha a feladat természetesen particionálható független blokkokra, a kooperatív launch csak felesleges korlátozást jelent. A hagyományos indítás rugalmasabb, és jobban kihasználja a GPU ütemezőjét.

9.3.6 Laboratóriumi példa: grid.sync-et használó ellenőrző kernel

Az alábbi példa egy olyan kernelt mutat be, amelyben minden blokk letétbe helyezi a saját azonosítóját egy globális tömbben, majd egy grid-szintű barrier után minden blokk ellenőrzi, hogy a teljes tömb helyesen kitöltött-e. Ha a `grid.sync()` nem működne, a blokkok egy része még nem látná a többi blokk írását, és hibát jelezne.

A példa CUDA C++ nyelvű, és feltételezi a `cooperative_groups` fejlécet. A kernel paraméterei:

- `values`: globális memóriában lévő tömb, mérete a grid blokkszámával egyenlő,
- `errors`: egyetlen egész szám, amelybe a hibák számát gyűjtjük atomi módon.

```
#include <cuda_runtime.h>
#include <cooperative_groups.h>
#include <cstdio>
#include <vector>

namespace cg = cooperative_groups;
```

```
__global__ void grid_sync_kernel(int *values, int *errors) {
    int tid = threadIdx.x;
    int bid = blockIdx.x;
    int block_count = gridDim.x;

    // Minden blokk 0. szála letétbe helyezi a saját blokkazonosítóját.
    if (tid == 0) {
        values[bid] = bid;
    }

    // Grid-szintű barrier: minden szál minden blokkban eléri ezt a pontot.
    cg::this_grid().sync();

    // A sync után minden blokk 0. szála ellenőrzi a teljes tömböt.
    if (tid == 0) {
        int local_errors = 0;
        for (int i = 0; i < block_count; ++i) {
            if (values[i] != i) {
                ++local_errors;
            }
        }
        if (local_errors > 0) {
            atomicAdd(errors, local_errors);
        }
    }
}

int main() {
    int device_id = 0;
    cudaSetDevice(device_id);

    // 1. Ellenőrizzük a kooperatív launch támogatását.
    int coop_supported = 0;
    cudaDeviceGetAttribute(&coop_supported,
                           cudaDevAttrCooperativeLaunch,
                           device_id);
    if (!coop_supported) {
        std::fprintf(stderr,
                     "Az eszköz nem támogatja a kooperatív launchot.\n");
        return 1;
    }

    // 2. Meghatározzuk a maximális együttműködő blokkszámot.
    const int block_size = 128;
    int num_sms = 0;
    cudaDeviceGetAttribute(&num_sms,
                           cudaDevAttrMultiProcessorCount,
                           device_id);

    int max_blocks_per_sm = 0;
    cudaOccupancyMaxActiveBlocksPerMultiprocessor(
        &max_blocks_per_sm,
        grid_sync_kernel,
        block_size,
        0);

    int grid_size = num_sms * max_blocks_per_sm;

    // 3. Lefoglaljuk a device memóriát.
    std::vector<int> h_values(grid_size, -1);
    int *d_values = nullptr;
    int *d_errors = nullptr;
    cudaMalloc(&d_values, grid_size * sizeof(int));
```

```
    cudaMalloc(&d_errors, sizeof(int));
    cudaMemcpy(d_values,
               h_values.data(),
               grid_size * sizeof(int),
               cudaMemcpyHostToDevice);
    cudaMemset(d_errors, 0, sizeof(int));

    // 4. Kooperatív launch.
    void *args[] = { &d_values, &d_errors };
    cudaError_t launch_err = cudaLaunchCooperativeKernel(
        grid_sync_kernel,
        dim3(grid_size),
        dim3(block_size),
        args,
        0,
        0);

    if (launch_err != cudaSuccess) {
        std::fprintf(stderr,
                     "Kooperatív launch hiba: %s\n",
                     cudaGetErrorString(launch_err));
        cudaFree(d_values);
        cudaFree(d_errors);
        return 1;
    }

    // 5. Szinkronizálunk és ellenőrizzük a futás közbeni hibákat.
    cudaDeviceSynchronize();
    cudaError_t runtime_err = cudaGetLastError();
    if (runtime_err != cudaSuccess) {
        std::fprintf(stderr,
                     "Futási hiba: %s\n",
                     cudaGetErrorString(runtime_err));
        cudaFree(d_values);
        cudaFree(d_errors);
        return 1;
    }

    // 6. Visszaolvassuk az eredményt.
    int h_errors = 0;
    cudaMemcpy(&h_errors,
               d_errors,
               sizeof(int),
               cudaMemcpyDeviceToHost);

    std::printf("grid_size=%d, errors=%d\n", grid_size, h_errors);

    cudaFree(d_values);
    cudaFree(d_errors);
    return (h_errors == 0) ? 0 : 1;
}
```

A kódban a `cudaOccupancyMaxActiveBlocksPerMultiprocessor` hívással a kernel tényleges erőforrás-igénye alapján számoljuk ki a maximális blokkszámot. A kernelben nincs shared memória, és a regiszterhasználat alacsony, ezért a legtöbb modern GPU-n sok blokk fér el egyszerre.

A kernel belsejében a `values[bid] = bid` írás a blokk 0. szál által történik. Mivel a blokk többi szál nem fér hozzá ehhez az értékhez a `grid.sync()` előtt, nincs szükség blokk-szintű `__syncthreads()`-re. A `grid.sync()` ezután garantálja, hogy az írás a grid minden szál számára láthatóvá válik, mielőtt a blokkok ellenőrzik a tömb tartalmát.

9.3.7 A példa futtatása és a mérési napló

A kód fordításához és futtatásához a következő parancsok használhatók (a `sm_80` egy példa, a tényleges architektúrához igazítandó):

```
nvcc -arch=sm_80 -O3 -o grid_sync grid_sync.cu
./grid_sync
```

Az alábbi mérési napló **szemléltető minta**, nem valós hardveres eredmény. A konkrét értékeket minden környezetben külön kell rögzíteni, és a mérést a könyv labor-szerződése szerint warm-up után, legalább 5–10 ismétléssel kell elvégezni.

Paraméter	Érték (kitöltendő)
GPU modell	NVIDIA <helyőrző, pl. A100>
Driver verzió	<helyőrző>
CUDA verzió	<helyőrző, pl. 12.x>
Compute capability	<helyőrző, pl. 8.0>
Blokkméret	128
Grid méret	`num_sms * max_blocks_per_sm`
Shared memória (statikus)	0
Várt hiba	0
Mért hiba	<kitöltendő>
Futásidő (középérték)	<kitöltendő> ms
Szórás / percentilis	<kitöltendő>

A példa akkor tekinthető sikeresnek, ha a `h_errors` értéke 0. Ha a `grid.sync()` nem működne, vagy a kernel indítása nem kooperatív lenne, a blokkok egy része nem látná a többi blokk írását, és a hibaszámláló nullától eltérő értéket mutatna.

9.3.8 Gyakori hibák és elhárításuk

A kooperatív launch és a grid-szintű szinkronizáció során a következő hibák fordulnak elő leggyakrabban:

- **`cudaErrorCooperativeLaunchTooLarge`**: a grid mérete meghaladja a maximális együttműködő blokkszámot. Ilyenkor csökkenteni kell a grid méretét, vagy módosítani a kernelt (például grid-stride loop használatával), hogy kevesebb blokk is elvégezze ugyanazt a munkát.
- **A kernel lefagy (deadlock)**: a legtöbbször az okozza, hogy nem minden szál éri el a `grid.sync()` hívást. Ellenőrizni kell, hogy a sync hívás nincs-e feltételes ágban, korai visszatérés mögött, vagy olyan ciklusban, amelyből egyes szálak kilépnek. A hibakereséshez a `compute-sanitizer --tool racecheck` és a `cuda-gdb` is használható, de a legegyszerűbb első lépés a vezérlési útvonalak átvizsgálása.
- **A `cg::this_grid()` használata hagyományos launch esetén**: ha a kernelt `<<< >>>`-sel indítjuk, a `this_grid().sync()` definiálatlan viselkedést eredményez. A megoldás az, hogy a kernelt `cudaLaunchCooperativeKernel`-nel kell indítani.
- **Hibás argumentum-átadás a `cudaLaunchCooperativeKernel` hívásban**: az `args` tömbnek a kernel paramétereire mutató pointereket kell tartalmaznia. Ha például egy `int*` paramétert adunk át, akkor az `args` eleme `&d_ptr` legyen, nem pedig `d_ptr`.

- **Memória-láthatóság feltételezése szinkronizáció nélkül**: bár a `grid.sync()` biztosítja a memória-fence-et, ha a blokkok a szinkronizáció után is írnak a global memóriába, és egy másik szál szinkronizáció nélkül olvassa azt, az eredmény nem garantált. Ilyenkor külön `__threadfence()` vagy újabb szinkronizáció szükséges.
- **Elhanyagolt hibakezelés**: a kooperatív launch esetén is kötelező minden CUDA API-hívás visszatérési értékét ellenőrizni, és a `cudaDeviceSynchronize` után a `cudaGetLastError`-t is lehívni. A hibás kernel lefagyhat, ezért a fejlesztés során érdemes timeoutot és visszaállítási tervet használni.

9.3.9 Önellenőrző kérdés

Mi történik akkor, ha egy kooperatív launchra szánt kernelben a grid mérete meghaladja a `cudaOccupancyMaxActiveBlocksPerMultiprocessor` és az SM-ek számának szorzatát? Milyen hibakódot kapunk, és hogyan javítható a helyzet?

Válasz: A `cudaLaunchCooperativeKernel` hívás `cudaErrorCooperativeLaunchTooLarge` hibakóddal tér vissza, és a kernel nem indul el. A javításhoz vagy csökkenteni kell a grid méretét (például grid-stride loop alkalmazásával, hogy kevesebb blokk dolgozzon fel több adatot), vagy csökkenteni kell a blokkonkénti erőforrás-igényt (például kevesebb regiszter vagy shared memory használatával), így több blokk fér el egyszerre az SM-eken.

9.4 Összetett együttműködési minták

A thread block, a cluster és a grid együttműködési szintjei külön-külön is hatékony eszközöket adnak, de a valódi erejüket akkor mutatják meg, amikor egyetlen kernelben kombináljuk őket. Ez a fejezet azt a gyakorlati mintát járja körül, amelyben egy többlépcsős számítás egyidejűleg támaszkodik blokkszintű szinkronizációra, klaszteren belüli megosztott memóriára, valamint grid-szintű zárolásra. A cél nem az egyes csoporttípusok újbóli bemutatása – azokat a megelőző alfejezetek tárgyalták –, hanem azok összehangolt alkalmazása, a belőlük adódó teljesítmény-, hordozhatósági és helyességi kérdésekkel együtt.

9.4.1 Miért van szükség kombinált együttműködésre?

A GPU programozási modellje hierarchikus. Egy tipikus, adatpárhuzamos kernel esetében a rácson belüli blokkok egymástól függetlenül, a hardver által meghatározott ütemezésben futnak. Ez a laza csatolás egyszerűvé és skálázhatóvá teszi a végrehajtást, ugyanakkor korlátozza azokat az algoritmusokat, amelyekben a blokkoknak részeredményeket kell kicserélniük anélkül, hogy a globális memórián keresztül drága, kétszeres körüljárást végeznének.

A Cooperative Groups API három szinten vezetett be finomabb vezérlést:

1. **Blokkszintű csoportok** (`thread_block`, `tile_partition`): a hagyományos `__syncthreads()` általánosítása, amely lehetővé teszi alcsoportok explicit szinkronizálását és kollektív műveletek használatát.
2. **Klaszterek** (`thread_block_cluster`): a Hopper (CC 9.0) és újabb architektúrákon a hardver lehetővé teszi blokkok egy csoportjának szoros együttműködését, klaszterszintű zárolással (`cluster.sync()`) és klaszteren belüli megosztott memóriával (Distributed Shared Memory, DSM). A blokkok így közvetlenül olvashatják egymás shared memóriáját, anélkül hogy a globális memórián keresztül kellene kommunikálniuk.

3. **Grid-szintű csoport** (`grid_group`): a `this_grid().sync()` olyan barrier, amelyet a rács minden blokkjának el kell érnie. Ez kizárólag kooperatív kernelindítás (`cudaLaunchCooperativeKernel`) esetén garantálható, amikor a futtatókörnyezet biztosítja, hogy az összes blokk egyidejűleg rezidens legyen az SM-eken.

A három szint kombinálása lehetővé teszi olyan algoritmusok hatékony megvalósítását, amelyek korábban csak többszöri kernelindítással, globális memórián keresztüli kommunikációval vagy atomi műveletekkel voltak megoldhatók. A tipikus alkalmazási területek közé tartoznak az iteratív megoldók (például Jacobi- vagy Gauss–Seidel-stencilek), a többszintű redukciók, a szemcsés részecskeszimulációk, valamint a mélytanulási kernelek bizonyos rétegei, ahol a blokk és a grid közötti finom adatcsere elengedhetetlen.

9.4.2 A kombinált minta általános szerkezete

Egy összetett, többlépcsős kernel általában a következő fázisokra bontható:

1. **Lokális számítás**: minden blokk a saját, globális memóriából beolvasott adattartományán dolgozik, és az eredményt a saját shared memóriájában vagy regisztereiben tartja.
2. **Klaszterszintű együttműködés**: a klaszter blokkjai kicserélik a szükséges részeredményeket a DSM-en keresztül, majd `cluster.sync()`-kel biztosítják, hogy minden blokk befejezte az írást, mielőtt bármelyikük olvasna.
3. **Grid-szintű szinkronizáció**: amikor a klaszterek közötti adatcsere is szükséges – például egy újabb iteráció megkezdése előtt – a kernel eléri a `this_grid().sync()` pontot. Ez biztosítja, hogy az előző fázis minden klaszterben befejeződött.
4. **Második számítási fázis**: a kernel a frissen elérhető adatokkal folytatja a munkát, például egy korrekciós lépést hajt végre, vagy egy következő időlépést számol.

Ez a minta akkor előnyös, ha a fázisok közötti adatmennyiség kicsi a számítási munkához képest, és ha a grid-szintű szinkronizáció ritkán következik be – tipikusan egy-egy iteráció végén. Ellenkező esetben a globális barrier miatt kialakuló várakozás felemésztheti a nyereséget.

A következő kódvázlat egy ilyen mintát mutat be vázlatos formában. A példa nem egy teljes, fordítható alkalmazás, hanem a szerkezet illusztrálására szolgál; a laborban ennél konkrétabb változatot építünk.

```
// CUDA C++ (illusztráció, nem teljes kernel)
#include <cooperative_groups.h>
namespace cg = cooperative_groups;

__global__ void combined_pattern_kernel(float* data, float* result, int n) {
    // 1. Blokkszintű csoport
    cg::thread_block block = cg::this_thread_block();

    // 2. Klaszter csoport (csak CC 9.0+; a klaszterméretet a launch konfiguráció adja meg)
    cg::thread_block_cluster cluster = cg::this_cluster();

    // 3. Lokális számítás: minden blokk kiszámítja a saját részeredményét
    //    (itt például egy tile feldolgozása)
    float local_sum = 0.0f;
    // ... adatbetöltés, számítás ...

    // 4. Klaszterszintű adatcsere DSM-en keresztül
    //    Tételezzük fel, hogy a klaszter minden blokkja egy parciális összeget ír
    //    a saját shared memóriájába. A klaszter.tömb segítségével lekérdezhetjük
    //    a klasztertársak shared memóriájának mutatóját.
    extern __shared__ float smem[];
    unsigned int cluster_rank = cg::this_cluster().block_rank();
```

```
    unsigned int cluster_size = cg::this_cluster().dim_blocks().x;

    // Feltételezés: minden blokk a saját shared memóriájának első elemébe írja a részösszeget
    if (block.thread_rank() == 0) {
        smem[0] = local_sum;
    }
    block.sync(); // biztosítjuk, hogy az írás befejeződött a blokkon belül

    // Klaszterszintű barrire: minden blokk elérte ezt a pontot, a DSM olvasható
    cluster.sync();

    // Klaszterszintű redukció: minden blokk kiolvassa a többi blokk részösszegét
    float cluster_sum = 0.0f;
    for (int i = 0; i < cluster_size; ++i) {
        // A DSM elérése: a klaszter i-edik blokkjának shared memóriáját olvassuk
        // A pontos API: cluster.map_shared_rank(...) vagy hasonló; itt leegyszerűsítve.
        float* remote_smem = cluster.map_shared_address(smem, i); // ellenőrizendő API
        cluster_sum += *remote_smem;
    }

    // 5. Grid-szintű szinkronizáció a klaszterek közötti adatcsere előtt
    cg::grid_group grid = cg::this_grid();
    grid.sync(); // minden blokk, minden klaszterben várakozik

    // 6. Második fázis: a klaszterek a friss adatokkal számolnak
    //    például normalizálják az eredményt a globális összeg alapján
    //    (feltételezzük, hogy egy globális bufferbe írták a klaszter_összegeket)
    // ...
}
```

Ebben a vázlatban jól látszik a három szint kombinációja: `block.sync()`, `cluster.sync()` és `grid.sync()`. A `cluster.map_shared_address()` hívás pontos szintaxisa a CUDA verziójától és a dokumentált API-tól függ; a példa csak a koncepciót mutatja, és a laborban a megfelelő, dokumentált formát használjuk.

9.4.3 Munkaelosztás a kombinált kernelben

A többlépcsős kernel egyik legfontosabb tervezési döntése, hogy a számítási munkát miként osztjuk fel a grid, a klaszter és a blokk szintjei között. A helyes felosztás határozza meg, hogy a szinkronizációs pontokon mennyi időt kell várakozni, és mennyire használjuk ki az SM-ek erőforrásait.

9.4.4 Hierarchikus tiling

A leggyakoribb megközelítés a hierarchikus lapkázás (tiling). A globális adattartományt négy szintre bonthatjuk:

- **Grid-szint**: a rács felel az adatok nagy, diszjunkt tartományaiért. Kooperatív indítás esetén a rács mérete nem lehet nagyobb annál, amennyi blokk egyidejűleg rezidens az összes SM-en, ezért a tartományok száma korlátozott. Emiatt gyakran érdemes perzisztens kernel stílust használni: minden blokk egy `for` ciklusban több kisebb tartományt dolgoz fel (grid-stride loop), ahelyett hogy a rács mérete egyenlő lenne a feldolgozandó részek számával.
- **Cluster-szint**: a klaszter a blokkok egy közeli csoportját fogja össze. A klaszteren belül a blokkok közösen fednek le egy nagyobb, összefüggő adatterületet, és a szomszédos határokon átnyúló adatcserét a gyors DSM-mel oldják meg. A klaszter méretét a hardverkorlát

(jellemzően 8 blokk klaszterenként, de ez architektúrafüggő, ellenőrizendő) és a feladat szomszédsági szerkezete határozza meg.
- **Blokk-szint**: egy blokk a klaszter által lefedett tartomány egy aldobozát kapja. A blokkméretet a memória-hozzáférési minták, a szükséges shared memória és a regiszterhasználat együttesen befolyásolják.
- **Warp- és thread-szint**: a blokkon belül a warpok és threadek végeznek vektorizált betöltést és számítást. A `cg::tile_partition` segítségével a blokkot kisebb, önállóan szinkronizálható csoportokra oszthatjuk, ami lehetővé teszi finomabb párhuzamosítást.

A hierarchikus felosztás előnye, hogy minden szinten a lehető legkisebb kommunikációs távolságot használjuk: a leggyakoribb adatcsere a blokkon belül shared memórián, a ritkább klaszteren belül DSM-en, a legritkább pedig grid-szinten, globális memórián keresztül történik.

9.4.5 Példa: stencil iteratív megoldó

Vegyünk egy 2D hővezetési egyenletet explicit Euler-módszerrel. A szokásos megoldás minden iterációban kiszámítja a cellák új értékét a szomszédok alapján. A párhuzamosítás során a tartományt lapkákra osztjuk, és minden lapka szélén egy halo-réteget kell cserélni a szomszédos lapkákkal.

Kombinált együttműködéssel ez a következőképpen valósítható meg:

1. Minden blokk betölti a saját lapkáját a globális memóriából, beleértve a halo-cellákat is, és a belső cellákat a shared memóriába másolja.
2. A klaszter blokkjai közötti határokon a halo-cserét DSM-mel végezzük: a blokk megosztja a saját szélső celláit a szomszédos blokkal, majd `cluster.sync()` után minden blokk kiolvassa a szomszédos blokk megfelelő celláit.
3. Minden blokk kiszámítja a következő iterációs értékeket a saját belső celláira.
4. A klaszterek közötti határokon a halo-csere még mindig globális memórián keresztül zajlik. Itt jön be a grid-szintű szinkronizáció: miután minden blokk kiírta a szükséges szélső cellákat a globális memóriába, a `grid.sync()` biztosítja, hogy minden klaszter befejezte az írást, és a következő iteráció elején minden blokk biztonságosan beolvashatja a globális halo-t.
5. A ciklus a következő iterációval folytatódik.

Ez a felépítés csökkenti a globális memórián keresztüli adatforgalmat, mert a klaszteren belüli határokon nem kell a lassú utat használni, csak a klaszterek közötti határokon. A grid-szinkronizáció itt csak iterációnként egyszer következik be, így a várakozási költsége eloszlik a teljes számítási munkán.

9.4.6 Perzisztens kernelek és grid-stride loop

A kooperatív indítás egyik következménye, hogy a rács mérete legfeljebb annyi blokk lehet, amennyi egyidejűleg rezidens. Ha egy problémához több független tartomány tartozik, mint amennyi blokk elfér, akkor a blokkoknak több tartományt kell feldolgozniuk. Ezt a gyakorlatban grid-stride loop segítségével érjük el:

```
// Grid-stride loop perzisztens kernelben (vázlat)
__global__ void persistent_kernel(float* data, int n) {
    cg::grid_group grid = cg::this_grid();
    for (int tid = blockIdx.x * blockDim.x + threadIdx.x; tid < n; tid += gridDim.x *
↪   blockDim.x) {
        // feldolgozás
```

```
    }
    // időszakos grid-szinkronizáció, ha szükséges
    if (need_sync) grid.sync();
}
```

Ez lehetővé teszi, hogy a kernel a teljes futás alatt az eszközön maradjon, és a globális barrierrel szabályozza a fázisokat. A grid-stride loop azonban gondos indexelést igényel, különösen akkor, ha a `grid.sync()` a loop belsejében van: a ciklusfeltételnek minden blokkban azonosan kell viselkednie, különben holtpont alakul ki.

9.4.7 Szinkronizációs szabályok és gyakori hibák

A kombinált minta helyessége szempontjából a legfontosabb, hogy minden szinten betartsuk a csoportokra vonatkozó kollektív hívások szabályait. A leggyakoribb hibák a következők:

9.4.8 1. Hiányzó vagy nem kollektív sync

A `cluster.sync()` és a `grid.sync()` kollektív művelet: a csoport minden tagjának ugyanazt a hívást kell végrehajtania, vezérlésfüggő ágakon kívül. Ha például egy blokk egy `if` feltétel miatt kihagyja a `cluster.sync()` hívást, a többi blokk örökre várakozni fog.

Hibás minta:

```
// HIBÁS: a sync feltételes ágban van, nem minden blokk éri el
__global__ void bad_cluster_sync(float* data) {
    cg::thread_block_cluster cluster = cg::this_cluster();
    if (threadIdx.x == 0) {
        // csak egyetlen thread végzi el a munkát, de a sync az egész blokkra vonatkozik
        // ...
        cluster.sync(); // HIBA: itt csak a 0. thread hívja, a többi nem
    }
    // a többi thread soha nem jut el ide, mert a sync-et várja
}
```

Helyes megoldás: a szinkronizálást a csoport minden tagjának, minden feltételtől függetlenül el kell érnie, vagy a feltételt úgy kell megfogalmazni, hogy a csoport minden tagja azonos döntést hozzon. A klasszikus blokkszintű szinkronizációhoz hasonlóan a `cluster.sync()`-et is a lehető legegyszerűbb vezérlési útra kell helyezni.

9.4.9 2. A grid-szinkronizáció nem kooperatív indítással

A `cg::this_grid().sync()` csak akkor biztonságos, ha a kernelt kooperatív indítással futtattuk, és a rács minden blokkja egyszerre rezidens. Ha egyszerű `<<<grid, block>>>` indítást használunk, a futtatókörnyezet nem garantálja, hogy minden blokk egyszerre fut, így a barrier várakozása holtponthoz vezethet.

Hibás minta:

```
// HIBÁS: grid.sync() normál <<< >>> indítással
__global__ void bad_grid_sync() {
    cooperative_groups::grid_group grid = cooperative_groups::this_grid();
    grid.sync(); // VESZÉLYES: normál indításnál deadlock lehet
}

// hívás:
// bad_grid_sync<<<num_blocks, block_size>>>();
```

Helyes megoldás: a kernelt `cudaLaunchCooperativeKernel` függvénnyel kell indítani, és előtte ellenőrizni kell, hogy a kívánt blokkszám és egyéb erőforrások lehetővé teszik-e a kooperatív futást. A `cudaOccupancyMaxActiveBlocksPerMultiprocessor` segítségével meghatározható, hogy hány blokk lehet rezidens egy SM-en, és ebből a maximális rácsméret.

```
// Helyes vázlat: kooperatív indítás
int numBlocksPerSm = 0;
cudaOccupancyMaxActiveBlocksPerMultiprocessor(&numBlocksPerSm, kernel, blockSize, smemSize);
int maxBlocks = numBlocksPerSm * numSMs;
if (gridDim > maxBlocks) {
    // csökkenteni kell a grid méretét vagy más stratégiát használni
}
void* args[] = { ... };
cudaLaunchCooperativeKernel((void*)kernel, dim3(gridDim), dim3(blockSize), args, smemSize,
↪  stream);
```

A grid méretének ellenőrzése kötelező, mert a hardver erőforrás-korlátai miatt előfordulhat, hogy a kért blokkszám nem fér el egyszerre. A túl nagy grid méret a `cudaErrorCooperativeLaunchTooLarge` hibát eredményezi, vagy rosszabb esetben definiálatlan viselkedést.

9.4.10 3. DSM hozzáférés cluster.sync nélkül

A Distributed Shared Memory lehetővé teszi, hogy egy blokk olvassa egy másik, ugyanabban a klaszterben lévő blokk shared memóriáját. Az olvasás előtt azonban biztosítani kell, hogy a forrásblokk befejezte az írást. Ezt a `cluster.sync()` hívás garantálja, amely a klaszter összes blokkját szinkronizálja.

Hibás minta:

```
// HIBÁS: DSM olvasás cluster.sync() nélkül
__global__ void bad_dsm_access(float* data) {
    cg::thread_block_cluster cluster = cg::this_cluster();
    extern __shared__ float smem[];

    if (cluster.block_rank() == 0) {
        smem[0] = 1.0f; // 0. blokk ír
    }
    // HIÁNYZIK a cluster.sync()

    float* remote = cluster.map_shared_address(smem, 0);
    float val = remote[0]; // versenyhelyzet: lehet, hogy az írás még nem történt meg
}
```

Helyes megoldás: minden DSM írás után, de még a DSM olvasás előtt hívjuk meg a `cluster.sync()`-et. A `block.sync()` nem elegendő, mert az csak a blokkon belüli láthatóságot biztosítja, a többi blokk számára nem.

9.4.11 4. A cluster méret és a hardverkorlát figyelmen kívül hagyása

A klaszterek mérete architektúrafüggő. A Hopper (CC 9.0) példájában a maximális klasztermérét jellemzően 8 blokk, de ez a pontos CUDA verziótól és a GPU típusától függ. A klaszter dimenzióit a launch konfigurációban kell megadni, és a kernelben a `this_cluster()` által visszaadott csoport dimenziói a ténylegesen elindított értéket tükrözik. Ha a kód feltételezi, hogy a klaszter mindig egy bizonyos méretű, akkor a hordozhatóság sérül.

Helyes megoldás: a kernelben dinamikusan kérdezzük le a klaszter dimenzióit, és a kódot úgy írjuk meg, hogy bármilyen támogatott méretre működjön, vagy fordítási időben ellenőrizzük a feltételeket és `static_assert`-et használunk a nem támogatott esetek kiszűrésére.

9.4.12 Teljesítmény és hordozhatóság

A kombinált együttműködési minták teljesítménye erősen függ attól, hogy a szinkronizációs pontok mennyire ritkák, és hogy a DSM-mel kiváltott globális memóriaforgalom mekkora megtakarítást eredményez.

9.4.13 Klaszter előnyei

A klaszter használatának legnagyobb előnye a blokkok közötti kommunikáció felgyorsítása. A DSM hozzáférés késleltetése lényegesen alacsonyabb, mint a globális memória elérése, ezért az olyan minták, amelyek gyakori szomszédos blokkok közötti adatcserét igényelnek, jelentős sávszélesség-megtakarítást érhetnek el. A klaszter emellett lehetővé teszi a shared memória együttes kihasználását: a klaszter teljes shared memóriája az egyes blokkok erőforrásainak összege, ami nagyobb, összefüggő tartományok lapkázását teszi lehetővé anélkül, hogy az adatot a globális memórián keresztül kellene mozgatni.

Ugyanakkor a klaszter használata növeli a regiszter- és shared memória nyomást, mert a blokkoknak további indexelést és címszámítást kell végezniük. A `cluster.sync()` hívás is többletköltséggel jár, ezért érdemes megfontolni, hogy egy adott feladatban tényleg szükség van-e klaszterre, vagy a hagyományos blokkszintű megoldás is elegendő.

9.4.14 Grid-szinkronizáció költsége

A `grid.sync()` a teljes rácsot szinkronizálja, ami azt jelenti, hogy a leglassabb blokk befejeződéséig minden más blokk várakozik. A várakozási idő a blokkok munkamennyiségének kiegyensúlyozatlanságától függ: ha az egyes blokkok terhelése eltérő, a barrier miatt a leggyorsabb blokkok is a leglassabbhoz igazodnak. Ezért a grid-szintű szinkronizációt lehetőleg ritkán, és csak akkor érdemes használni, ha a fázisok közötti adatfüggőség valóban megköveteli.

Ráadásul a kooperatív indítás korlátozza a rács méretét, ami csökkentheti az occupancy-t, ha a kernel egyébként sok blokkal jobban skálázna. A perzisztens kernelek és a grid-stride loop ezt a korlátot enyhítik, de a tervezés során mindig mérni kell, hogy a grid-szinkronizációból származó nyereség ellensúlyozza-e a kisebb rács miatti veszteséget.

9.4.15 Hordozhatósági megfontolások

A kombinált minták hordozhatósága két szinten korlátozott:

- **Hardver-szint**: a klaszterek csak a Hopper (CC 9.0) és újabb architektúrákon érhetők el. A grid-szinkronizáció viszont már a Pascal (CC 6.0) óta támogatott, de a pontos feltételek (kooperatív indítás, maximális rácsméret) architektúránként változnak. A kódot ezért érdemes úgy megírni, hogy a klasztert igénylő részeket feltételes fordítás és futásidejű ellenőrzés védje.

- **Szoftver-szint**: a Cooperative Groups API a CUDA 9.0 óta elérhető, de a `thread_block_cluster` és a DSM csak a CUDA 11.8/12.0 környékén, a Hopper támogatással együtt jelent meg. A pontos verziókat a fejlesztői dokumentáció adott kiadásában ellenőrizni kell; e sorok írásakor a klaszterek a CUDA 12.x-es eszközlánccal és CC 9.0-val működnek, de ez a jövőben változhat.

A gyakorlatban a hordozhatóság érdekében a következő stratégiát érdemes követni:

1. A kernel magját úgy írjuk meg, hogy a `thread_block` és `tile_partition` szintű együttműködés mindig elérhető legyen, mert ezek a legszélesebb körben támogatottak.
2. A klasztert használó kódot külön függvénybe vagy fordítási egységbe szervezzük, és `#ifdef`-fel vagy futásidejű `cudaDeviceGetAttribute` ellenőrzéssel kapcsoljuk be, ha az eszköz támogatja.
3. A grid-szinkronizációt igénylő részt úgy tervezzük, hogy alternatív, többszöri kernelindításon alapuló megoldás is létezzen azokra az eszközökre, ahol a kooperatív indítás nem elérhető vagy nem hatékony.

OpenCL-ben a munkacsoportok (work-group) közötti közvetlen szinkronizáció és megosztott memória nem része a hordozható mag szabványnak; a hasonló mintát általában több kernelindítással vagy atomi műveletekkel kell megvalósítani. JAX/XLA esetében a magas szintű, gráf alapú modell elrejti a blokkok közötti kommunikációt, így az ilyen finomhangolású együttműködés nem közvetlenül alkalmazható.

9.4.16 Labor: többlépcsős kernel tervezése

Az alábbi labor célja, hogy a kombinált együttműködési mintát egy mérhető, ellenőrizhető példán keresztül gyakorolja. A labor a klaszter és a grid-szinkronizáció együttműködését egy iteratív stencilszámításon keresztül mutatja be.

9.4.17 Feladatleírás

Valósítson meg egy 2D hővezetési szimulációt explicit Euler-módszerrel, amely a következő lépésekből áll:

- A tartomány `N x N` rács, ahol minden cella egy `float` érték.
- Minden iterációban az új érték a négy közvetlen szomszéd átlagával egyenlő: `u_new[i][j] = 0.25f * (u[i-1][j] + u[i+1][j] + u[i][j-1] + u[i][j+1])`.
- A kernel egyetlen kooperatív indítással futtatja a szimulációt `T` iteráción keresztül.
- A tartományt a grid blokkjai lapkákra osztják. A klaszterek a lapkák egy csoportját fogják össze, és a klaszteren belüli szomszédos lapkák közötti halo-cserét DSM-mel végzik. A klaszterek közötti határokon a halo-csere globális memórián keresztül történik, és ezt `this_grid().sync()` szinkronizálja minden iteráció végén.

A labor kódja CUDA C++ nyelvű, és legalább a következő komponenseket tartalmazza:

- Kooperatív kernel indítása `cudaLaunchCooperativeKernel`-lel.
- Blokkszintű csoport `cg::this_thread_block()` használata.
- Klaszter csoport `cg::this_cluster()` és DSM hozzáférés.
- Grid csoport `cg::this_grid()` és `grid.sync()`.
- Hibakezelés: minden CUDA API-hívás ellenőrzése.

A pontos implementációt a fejezet kódmelléklete tartalmazza; itt a legfontosabb vázlatot mutatjuk be.

9.4.18 Kernelvázlat

```
// CUDA C++ - labor vázlat (nem teljes, de a szerkezetet mutatja)
#include <cooperative_groups.h>
#include <cuda_runtime.h>
namespace cg = cooperative_groups;

// Stencil kernel: T iterációt futtat egyetlen kooperatív indítással
__global__ void stencil_cooperative(const float* __restrict__ u_old,
                                    float* __restrict__ u_new,
                                    int N, int T) {
    cg::thread_block block = cg::this_thread_block();
    cg::thread_block_cluster cluster = cg::this_cluster();
    cg::grid_group grid = cg::this_grid();

    // lapka méretek: TILE_X, TILE_Y (befordítva)
    // határok, indexelés...
    extern __shared__ float smem[]; // saját shared memória

    for (int iter = 0; iter < T; ++iter) {
        // 1. Betöltés globális memóriából a shared memóriába,
        //    beleértve a halo cellákat is.
        load_tile(u_old, smem, N);

        // 2. Klaszteren belüli halo-csere DSM-mel.
        //    A klasztertársakkal megosztjuk a szélső cellákat.
        exchange_halo_cluster(smem, cluster);
        cluster.sync();

        // 3. Számítás: a belső cellák új értéke.
        compute_interior(smem, block);

        // 4. Klaszterek közötti halo-csere globális memórián keresztül.
        //    Írjuk a szükséges szélső cellákat egy átmeneti bufferbe.
        write_global_halo(u_new, smem, N);

        // 5. Grid-szinkronizáció: minden blokk befejezte az írást,
        //    mielőtt a következő iterációban olvasná a globális halo-t.
        grid.sync();

        // 6. A következő iterációhoz olvassuk a frissített értékeket.
        //    (Az egyszerűség kedvéért itt u_old <-> u_new cserét feltételezünk.)
    }
}

// Indítási függvény
void run_stencil(const float* d_u0, float* d_u1, int N, int T,
                 dim3 grid, dim3 block, int smem_size, cudaStream_t stream) {
    void* args[] = { (void*)&d_u0, (void*)&d_u1, (void*)&N, (void*)&T };
    cudaError_t err = cudaLaunchCooperativeKernel(
        (void*)stencil_cooperative, grid, block, args, smem_size, stream);
    CHECK_CUDA_ERROR(err);
}
```

A fenti vázlatban a `load_tile`, `exchange_halo_cluster`, `compute_interior` és `write_global_halo` eszközfüggvények a tényleges számítást végzik; ezek implementálása a laborfeladat része. A halo-csere pontos DSM indexelése a CUDA dokumentációban leírt `map_shared_rank` és `map_shared_address` API-t használja, amelyet a labor kódjában ellenőrizni kell.

9.4.19 Build és futtatási környezet

A méréshez rögzíteni kell a következő környezetet (példaértékek; a tényleges laborban a mért értékeket kell feltüntetni):

- GPU: NVIDIA H100 (vagy más CC 9.0+), például H100 PCIe 80 GB
- Driver: 535.104.05 (példa)
- CUDA: 12.3
- Operációs rendszer: Ubuntu 22.04 LTS
- Fordító: nvcc V12.3.103

Build parancs (vázlat):

```
nvcc -arch=sm_90 -rdc=true -o stencil_lab stencil_lab.cu -I.
```

Futtatás:

```
./stencil_lab --N 4096 --T 100 --block 256 --cluster 4
```

A `-rdc=true` az eszközkód szétválasztott fordításához szükséges lehet, ha a kooperatív csoportok használata ezt megköveteli; a pontos szükségletet a CUDA verziója és a kód szerkezete határozza meg.

9.4.20 Mérési napló és kiértékelés

A futás során mérjük a kernel végrehajtási idejét, és hasonlítsuk össze egy olyan baseline verzióval, amely nem használ klasztert és grid-szinkronizációt, hanem minden iterációt külön kernelindítással végez el. A méréshez legalább 5–10 ismétlést végezzünk, és az átlag mellett a szórást is közöljük.

Példa mérési táblázat:

Verzió	Grid méret	Blokk méret	Klaszterméret	Átlagidő (ms)	Szórás (ms)	Megjegyzés
Baseline (külön kernelek)	64×64	16×16	–	12.4	0.8	minden iteráció új kernel
Kooperatív, klaszter nélkül	64×64	16×16	–	10.9	0.6	grid.sync minden iterációban
Kooperatív, klaszterrel	64×64	16×16	4 blokk	9.1	0.5	DSM a klaszteren belüli halo-ra

A pontos számok a labor során mért értékektől függenek; a fenti táblázat csak a szerkezetet mutatja. A mérési naplóban fel kell tüntetni a GPU órajelét, a power limitet, az ECC állapotát és a háttérterhelést is, mert ezek befolyásolják az eredményt.

Az ellenőrzéshez számítsuk ki a szimulációt CPU-n (például egy egyszerű C++ implementációval), és hasonlítsuk össze a GPU eredményeket az FP32 pontosság keretein belül. A hiba jellemzően kicsi, de az iteratív algoritmusoknál a felhalmozódó kerekítések miatt nem nulla; a laborban adjuk meg a maximális abszolút hibát.

9.4.21 Ellenőrző kérdés

A fejezet lezárásaként válaszolja meg a következő kérdést:

Milyen feltételek mellett biztonságos a `this_grid().sync()` hívása egy kernelben, és mi történik, ha a rács egyik blokkja nem éri el ezt a hívást?

A helyes válasznak tartalmaznia kell, hogy a kernelnek kooperatív indítással kell futnia, a rács minden blokkjának egyidejűleg rezidensnek kell lennie, és a szinkronizálási hívásnak minden blokkban ugyanazon a vezérlési úton kell elhelyezkednie. Ha egy blokk nem éri el a hívást, a többi blokk örökre várakozik, ami holtponthoz vezet.

9.4.22 Összegzés

A Cooperative Groups eszköztárának kombinált használata lehetővé teszi olyan összetett, többlépcsős kernelek írását, amelyek a blokkszintű finom szinkronizációt, a klaszteren belüli gyors adatcserét és a grid-szintű fázisvezérlést egyetlen kernelben egyesítik. A minta hatékonysága azon múlik, hogy a munkaelosztást hierarchikusan tervezzük-e, a szinkronizációs pontokat ritkán és szabályosan helyezzük-e el, valamint hogy tiszteletben tartjuk-e a hardver- és szoftverkorlátokat. A laborban mért eredmények azt mutatják, hogy a klaszterekkel támogatott DSM csökkentheti a globális memóriaforgalmat, és a kooperatív indítással elkerülhető a többszöri kernelindítás overheadje, de a pontos nyereség erősen függ a probléma méretétől és a GPU architektúrájától. A következő fejezet a CUDA hibakezelésének és debuggolásának részleteivel foglalkozik, amely a kombinált minták helyességének ellenőrzéséhez is elengedhetetlen.

10 Hibakezelés, debugging és compute-sanitizer

10.1 CUDA-hibakódok és ellenőrzés

A CUDA futásidejű hibakezelése az egyik legfontosabb, mégis gyakran alábecsült eleme a GPU-programozásnak. Az aszinkron végrehajtás, a kernelindítások speciális szintaxisa és a hibák késleltetett jelentkezése miatt a hagyományos, szinkron kódból ismert kivételkezelési minták nem ültethetők át közvetlenül. Ebben az alfejezetben a futásidejű hibák és hibakódok természetét vizsgáljuk meg, bemutatjuk a `cudaGetLastError` és `cudaPeekAtLastError` működését, és kialakítunk egy olyan hibaellenőrző makrót és wrapper-t, amely a könyv további laborjainak alapját képezi. Az itt bevezetett eszközökkel a következő alfejezetben (cuda-gdb és kernel-debugging) a `cuda-gdb` és a compute-sanitizer használatát építjük fel.

10.1.1 Futásidejű hibák és hibakódok

A CUDA Runtime API függvényei a legtöbb esetben egy `cudaError_t` típusú értéket adnak vissza. Ez az érték jelzi, hogy a hívás sikeres volt-e, vagy milyen hibába ütközött. A lehetséges hibakódokat a `cudaError_t` enumeráció definiálja, és a `cudaGetErrorString` függvény segítségével ember által olvasható szöveggé alakíthatók.

A sikeres végrehajtást a `cudaSuccess` (értéke 0) jelzi. Minden más érték hibát jelent. Néhány gyakori hibakód:

- `cudaErrorInvalidValue`: egy paraméter értéke nem megengedett tartományba esik (pl. negatív méret).
- `cudaErrorInvalidMemorySpace`: a mutató nem a várt memóriaterületre mutat.
- `cudaErrorInvalidConfiguration`: a kernel konfigurációja érvénytelen, például a blokk mérete meghaladja az eszköz által támogatott maximumot (legtöbb GPU-n legfeljebb 1024 szál blokkonként).
- `cudaErrorMemoryAllocation`: a `cudaMalloc` vagy más memóriafoglaló hívás nem tudta kielégíteni a kérést, gyakran “out of memory” állapot miatt.
- `cudaErrorInvalidDevice`: a megadott eszközazonosító nem létezik.
- `cudaErrorNoDevice`: nincs elérhető CUDA-képes eszköz a rendszerben.
- `cudaErrorInvalidDevicePointer`: a device-mutató érvénytelen, például felszabadított memóriára mutat.
- `cudaErrorIllegalAddress`: a kernel futása közben érvénytelen memóriacímet próbált elérni, ami gyakran a `cudaDeviceSynchronize` hívásnál derül ki.
- `cudaErrorLaunchFailure`: a kernel nem tudott elindulni vagy futni valamilyen hardveres vagy szoftveres okból.
- `cudaErrorAssert`: a kernelben egy `assert` utasítás meghiúsult.

A hibakódok nem csak a közvetlen API-hívás visszatérési értékeként jelenhetnek meg, hanem az úgynevezett *sticky error* mechanizmuson keresztül is. Amikor egy CUDA-hívás hibát generál, a hiba állapota rögzül a futásidejű környezetben, és minden további CUDA-hívás, amely lekérdezi a hibát, ugyanazt a hibát fogja visszaadni, amíg a hibát nem törlik (például a `cudaGetLastError`

segítségével). Ez a viselkedés eltér a szokásos C++ kivételkezeléstől, ahol egy kivétel eldobása után a végrehajtás folytatódhat.

Fontos megérteni, hogy a hibakódok nem mindig a hibát okozó hívás közvetlen visszatérési értékében jelennek meg. A CUDA végrehajtási modell aszinkron, ami azt jelenti, hogy a kernelindítás (`<<<...>>>`) visszatér a hostra, mielőtt a kernel ténylegesen befejeződne, sőt gyakran mielőtt elindulna. A kernelindítás során fellépő hibák (például érvénytelen blokkméret) a launch után lekérdezhetők a `cudaGetLastError` segítségével, de a kernel futása közbeni hibák (például illegális memória-hozzáférés) csak akkor válnak ismertté, amikor a host explicit vagy implicit szinkronizációt hajt végre (például `cudaDeviceSynchronize`, `cudaMemcpy` device→host irányban, vagy egy másik, ugyanazon a streamen futó művelet befejeződése). Ezért a hibakezelési stratégiát ehhez a modellhez kell igazítani.

10.1.2 `cudaGetLastError` és `cudaPeekAtLastError`

A CUDA Runtime API két függvényt biztosít a legutóbb feljegyzett hiba lekérdezésére:

- `cudaError_t cudaGetLastError(void)` – visszaadja az utolsó hibát, majd **törli** a hibastátust, vagyis a következő hívás már `cudaSuccess` értéket ad, hacsak időközben nem lépett fel újabb hiba.
- `cudaError_t cudaPeekAtLastError(void)` – visszaadja az utolsó hibát, de **nem törli** azt. A hibastátus megmarad, így később ismételten lekérdezhető.

A két függvény különbsége akkor válik fontossá, ha a hibát több helyen is szeretnénk megvizsgálni anélkül, hogy elveszítenénk az információt. Tipikus használati eset a debugolás: ha egy hiba után több diagnosztikai üzenetet akarunk kiírni, és nem akarjuk, hogy az első hibaellenőrző hívás "elfogyassza" a hibakódot, akkor `cudaPeekAtLastError`-t használunk. A hibát végül a `cudaGetLastError` hívással törölhetjük, hogy a következő műveletek tiszta lappal induljanak.

Például egy hibás kernelindítás után a következő kódrészlet megőrzi a hibát további vizsgálatra:

```
kernel<<<grid, block>>>(args);
cudaError_t err = cudaPeekAtLastError(); // hibát ad, de nem törli
if (err != cudaSuccess) {
    fprintf(stderr, "Kernel launch hiba: %s\n", cudaGetErrorString(err));
    // további diagnosztika
}
// később: cudaGetLastError() törli a hibát
```

Fontos, hogy a `cudaGetLastError` és `cudaPeekAtLastError` is az utolsó **hibát** adják vissza, nem az utolsó **művelet** sikerességét. Ha az utolsó hibát már töröltük, és azóta nem volt újabb hiba, akkor `cudaSuccess`-t adnak.

Tegyük fel, hogy a következő szekvenciát hajtjuk végre:

1. `cudaMalloc(&d_ptr, 1024)` – sikeres, `cudaSuccess`-t ad vissza.
2. `kernel<<<...>>>` – a launch konfiguráció hibás (pl. 1025 szál blokkonként). A hiba nem azonnali, de a következő API hívásnál vagy szinkronizációnál jelentkezik.
3. `cudaGetLastError()` – most visszaadja a `cudaErrorInvalidConfiguration` hibát, mert a kernelindítás hibája a sticky error mechanizmuson keresztül rögzült. A hiba törlődik.
4. `cudaGetLastError()` ismét – most már `cudaSuccess`, mert a hibát töröltük.

Ha a 3. lépésben `cudaPeekAtLastError`-t hívtunk volna, akkor a 4. lépés ugyanazt a hibát adná vissza, mert a hiba nem törlődött.

Aszinkron hibák esetén a helyzet bonyolultabb. A kernelindítás közvetlen utáni `cudaGetLastError` hívás általában elkapja a launch konfigurációs hibákat, mert azok a `<<<...>>>` feldolgozásakor már ismertek. A kernel futása közbeni hibák (például `cudaErrorIllegalAddress`) viszont csak akkor kerülnek a hibastátusba, amikor a host szinkronizál a streammel. A `cudaDeviceSynchronize()` hívás garantálja, hogy az adott eszközön minden korábban indított művelet befejeződjön, és a futás közben felmerült hibák is a hibastátusba kerüljenek. Ezért a megbízható hibaellenőrzési minta a következő:

```
kernel<<<grid, block>>>(args);
CUDA_CHECK(cudaGetLastError());       // launch hiba
CUDA_CHECK(cudaDeviceSynchronize());  // futási hiba
```

A `CUDA_CHECK` makrót hamarosan bevezetjük; itt az a lényeg, hogy mindkét hívás hibakódját ellenőrizzük. Egyes fejlesztők hajlamosak elhagyni a `cudaGetLastError` hívást a kernelindítás után, mert a `cudaDeviceSynchronize` is visszaadná a hibát, de a launch hibák nem mindig válnak láthatóvá a szinkronizációnál, ezért a közvetlen ellenőrzés javasolt.

10.1.3 Hibaellenőrző makró és wrapper

A hibakezelés megkönnyítésére célszerű egy makrót és egy wrapper osztályt kialakítani, amelyek minden CUDA API-hívást ellenőriznek, és hiba esetén értelmes üzenetet adnak. A könyvben ettől a ponttól kezdve minden példában ezt a mintát fogjuk használni.

Először egy egyszerű makró, amely a hívást argumentumként veszi, meghívja, ellenőrzi a visszatérési értéket, és hiba esetén kiírja a fájlnevet, a sor számát, a hívás szövegét és a hiba üzenetét, majd azonnal leállítja a programot (vagy kivételt dob, ha a wrapper-t használjuk).

```
#ifndef CUDA_CHECK_H
#define CUDA_CHECK_H

#include <cuda_runtime.h>
#include <iostream>
#include <string>
#include <sstream>

// Hibaellenőrző makró: a call kifejezést értékeli ki, ellenőrzi a hibakódot.
// Hiba esetén kiírja a részleteket és std::abort()-ot hív (vagy kivételt dob).
#define CUDA_CHECK(call) do {                                                \
    cudaError_t err = (call);                                                \
    if (err != cudaSuccess) {                                                \
        std::ostringstream oss;                                              \
        oss << "CUDA error at " << __FILE__ << ":" << __LINE__               \
            << " '" #call "' failed: " << cudaGetErrorString(err);           \
        std::cerr << oss.str() << std::endl;                                 \
        std::abort();                                                        \
    }                                                                        \
} while (0)

#endif // CUDA_CHECK_H
```

Ezt a makrót bármely `cudaError_t`-t visszaadó függvényre használhatjuk. Például:

```
CUDA_CHECK(cudaMalloc(&d_ptr, size));
CUDA_CHECK(cudaMemcpy(d_ptr, h_ptr, size, cudaMemcpyHostToDevice));
CUDA_CHECK(cudaGetLastError()); // kernel launch után
CUDA_CHECK(cudaDeviceSynchronize());
```

A makró előnye, hogy nulla futásidejű költséggel jár, ha a kód hibamentes, mert a hibaellenőrzés egyetlen összehasonlításból áll. A hibaüzenet pedig tartalmazza a pontos helyet és a hibát okozó kifejezést, ami nagyban gyorsítja a hibakeresést.

A `do { ... } while (0)` konstrukció biztosítja, hogy a makró egyetlen utasításként viselkedjen, így biztonságosan használható például `if` feltétel nélküli ágban is.

10.1.4 Wrapper osztály kivételkezeléssel

Néha az abortálás nem elfogadható, például egy hosszabb futású alkalmazásban, ahol a hibát egy magasabb szintű komponens szeretné kezelni. Ebben az esetben a makrót úgy alakíthatjuk, hogy egy kivételt dobjon, amelyet a hívó elkaphat. Ehhez egy `CudaError` kivételosztályt és egy wrapper függvényt definiálunk:

```
#include <stdexcept>

class CudaError : public std::runtime_error {
public:
    explicit CudaError(const std::string& msg) : std::runtime_error(msg) {}
};

// Wrapper függvény, amely a hibakódot ellenőrzi és kivételt dob.
inline void checkCudaError(cudaError_t err, const char* file, int line, const char* expr) {
    if (err != cudaSuccess) {
        std::ostringstream oss;
        oss << "CUDA error at " << file << ":" << line
            << " '" << expr << "' failed: " << cudaGetErrorString(err);
        throw CudaError(oss.str());
    }
}

// Makró, amely a wrapper-t hívja, így kivételt dob abort helyett.
#define CUDA_CHECK_OR_THROW(call) do {                                  \
    cudaError_t err = (call);                                           \
    checkCudaError(err, __FILE__, __LINE__, #call);                     \
} while (0)
```

Most a hívó elkaphatja a kivételt, például:

```
try {
    CUDA_CHECK_OR_THROW(cudaMalloc(&d_ptr, size));
} catch (const CudaError& e) {
    std::cerr << "Hiba történt: " << e.what() << std::endl;
    // takarítás vagy alternatív útvonal
}
```

Fontos megjegyezni, hogy a makró a `call` kifejezést pontosan egyszer értékeli ki, ami kritikus, mert a CUDA API-hívásoknak mellékhatásuk van (memóriafoglalás, másolás, stb.). A `do { ... } while (0)` konstrukció itt is biztosítja az egyszeri végrehajtást.

Mindkét megközelítésnek megvan a maga helye. A laborokban általában az abortáló makrót használjuk, mert a hibák korai leállítást igényelnek, és a hibaüzenet elegendő a diagnózishoz. A kivételes wrapper-t olyan könyvtári kódban érdemes használni, ahol a hívó fél kezelheti a hibát.

10.1.5 Labor: Hibaellenőrző makró és wrapper

Ebben a laborban a fent bemutatott hibaellenőrző makrót és wrapper-t implementáljuk, majd egy szándékosan hibás CUDA programon teszteljük. A cél annak megfigyelése, hogy a hibák hogyan jelentkeznek, és a makró hogyan segíti a diagnózist.

10.1.6 Környezet

A labor az alábbi (illeszthető) környezetben futtatható. Az aktuális értékeket minden futás előtt rögzíteni kell:

- GPU: **NVIDIA GeForce RTX 3080** (vagy bármely CUDA-képes GPU)
- Compute capability: **8.6** (vagy az adott GPU-nak megfelelő)
- Driver verzió: **550.54.14** (a rendszeren aktuális)
- CUDA Toolkit verzió: **12.2** (ellenőrizni kell a telepített verziót)
- Operációs rendszer: **Linux x86_64** (vagy Windows, macOS)
- Build rendszer: `nvcc` a CUDA Toolkit része

A pontos verziószámok a `nvidia-smi` és `nvcc --version` kimenetéből nyerhetők.

10.1.7 Kód

A következő CUDA C++ forrás (`error_check.cu`) tartalmazza a makrót, a wrapper-t, egy egyszerű kernelt, és két futtatási útvonalat: egy helyes és egy hibás végrehajtást. A hibás útvonal szándékosan túl nagy blokkméretet ad meg a kernelindításnál, ami `cudaErrorInvalidConfiguration` hibát vált ki.

```
// error_check.cu
#include <cuda_runtime.h>
#include <iostream>
#include <sstream>
#include <stdexcept>

// --- Hibaellenőrző makró (abort verzió) ---
#define CUDA_CHECK(call) do {                                          \
    cudaError_t err = (call);                                          \
    if (err != cudaSuccess) {                                          \
        std::ostringstream oss;                                        \
        oss << "CUDA error at " << __FILE__ << ":" << __LINE__         \
            << " '" #call "' failed: " << cudaGetErrorString(err);     \
        std::cerr << oss.str() << std::endl;                           \
        std::abort();                                                  \
    }                                                                  \
} while (0)

// --- Wrapper osztály és makró (kivétel verzió) ---
class CudaError : public std::runtime_error {
public:
    explicit CudaError(const std::string& msg) : std::runtime_error(msg) {}
};

inline void checkCudaError(cudaError_t err, const char* file, int line, const char* expr) {
    if (err != cudaSuccess) {
        std::ostringstream oss;
        oss << "CUDA error at " << file << ":" << line
            << " '" << expr << "' failed: " << cudaGetErrorString(err);
        throw CudaError(oss.str());
```

```
    }
}

#define CUDA_CHECK_OR_THROW(call) do {                                  \
    cudaError_t err = (call);                                           \
    checkCudaError(err, __FILE__, __LINE__, #call);                     \
} while (0)

// Egyszerű kernel
__global__ void addKernel(const float* a, const float* b, float* c, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        c[idx] = a[idx] + b[idx];
    }
}

int main() {
    const int N = 1 << 20; // 1M elem
    size_t size = N * sizeof(float);

    // Host memória foglalás
    float *h_a = new float[N];
    float *h_b = new float[N];
    float *h_c = new float[N];
    for (int i = 0; i < N; ++i) {
        h_a[i] = static_cast<float>(i);
        h_b[i] = static_cast<float>(2 * i);
    }

    // Device memória foglalás
    float *d_a, *d_b, *d_c;
    CUDA_CHECK(cudaMalloc(&d_a, size));
    CUDA_CHECK(cudaMalloc(&d_b, size));
    CUDA_CHECK(cudaMalloc(&d_c, size));

    // Másolás host->device
    CUDA_CHECK(cudaMemcpy(d_a, h_a, size, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_b, h_b, size, cudaMemcpyHostToDevice));

    // 1. Helyes kernelindítás
    int threadsPerBlock = 256;
    int blocks = (N + threadsPerBlock - 1) / threadsPerBlock;
    addKernel<<<blocks, threadsPerBlock>>>(d_a, d_b, d_c, N);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    // Másolás device->host
    CUDA_CHECK(cudaMemcpy(h_c, d_c, size, cudaMemcpyDeviceToHost));

    // Ellenőrzés
    bool correct = true;
    for (int i = 0; i < N; ++i) {
        if (h_c[i] != h_a[i] + h_b[i]) {
            correct = false;
            break;
        }
    }
    std::cout << "Helyes kernel futtatás: " << (correct ? "sikeres" : "hibás") << std::endl;

    // 2. Hibás kernelindítás: túl nagy blokkméret (max 1024)
    threadsPerBlock = 1025; // szándékos hiba
    blocks = (N + threadsPerBlock - 1) / threadsPerBlock;
    addKernel<<<blocks, threadsPerBlock>>>(d_a, d_b, d_c, N);
```

```
    CUDA_CHECK(cudaGetLastError()); // itt elkapjuk a hibát

    // Ide nem jut el, mert a CUDA_CHECK abortál

    // Tisztítás (nem fut le a hibás ág miatt, de a teljesség kedvéért)
    CUDA_CHECK(cudaFree(d_a));
    CUDA_CHECK(cudaFree(d_b));
    CUDA_CHECK(cudaFree(d_c));
    delete[] h_a;
    delete[] h_b;
    delete[] h_c;

    return 0;
}
```

A program a helyes kernel futtatása után szándékosan hibás blokkmérettel indít egy második kernelt. A `CUDA_CHECK(cudaGetLastError())` hívásnak el kell kapnia a `cudaErrorInvalidConfiguration` hibát, és a makró leállítja a programot egy üzenettel.

10.1.8 Build parancs

```
nvcc -o error_check error_check.cu -arch=sm_86
```

Az `-arch=sm_86` a GeForce RTX 3080 compute capability-jének felel meg. Más GPU esetén az architektúrát a GPU specifikációja alapján kell megadni (pl. `sm_80` A100, `sm_75` RTX 2080).

10.1.9 Futtatási parancs

```
./error_check
```

10.1.10 Várt eredmény

A helyes kernel futása után a program kiírja:

```
Helyes kernel futtatás: sikeres
```

Ezután a hibás kernelindításnál a `CUDA_CHECK(cudaGetLastError())` hívásnak hibát kell észlelnie, és a következő üzenetet kell kiírnia (a fájl és sor szám a valós környezettől függően változhat):

```
CUDA error at error_check.cu:94 'cudaGetLastError()' failed: invalid configuration argument
```

A program ezután `std::abort()`-tal kilép, tehát a visszatérési kód nem 0, hanem SIGABRT (Unix) vagy hasonló.

Paraméter	Mért érték	Várt érték

10.1.11 Mérési napló

A fenti futtatás eredményét rögzítjük, különválasztva a mért és a várt értékeket. A példa kedvéért egy lehetséges napló:

Paraméter	Mért érték	Várt érték
Helyes kernel futtatás	`Helyes kernel futtatás: sikeres`	`Helyes kernel futtatás: sikeres`
Hibakód a hibás launch után	`cudaErrorInvalidConfiguration` (információ a hibaüzenetből)	`cudaErrorInvalidConfiguration`
Hibaüzenet szövege	`invalid configuration argument`	`invalid configuration argument`
Program visszatérési kódja	SIGABRT (abort)	SIGABRT (abort)

Fontos, hogy a fenti mérési napló illusztratív, a tényleges értékeket a saját környezetben kell rögzíteni. A "mért érték" oszlopba a futtatás során ténylegesen kapott kimenetet írjuk, a "várt érték" oszlopba pedig az elméleti elvárást a dokumentáció alapján.

10.1.12 Összefoglalás

Ebben az alfejezetben megismertük a CUDA futásidejű hibakezelés alapjait:

- A CUDA API-hívások `cudaError_t` hibakódokat adnak vissza; a `cudaSuccess` jelzi a sikert.
- A hibák gyakran *sticky* jellegűek, azaz a hibastátus addig megmarad, amíg `cudaGetLastError`-rel nem töröljük.
- A `cudaPeekAtLastError` megőrzi a hibát későbbi lekérdezésekre.
- A kernelindítások aszinkron volta miatt a launch hibákat `cudaGetLastError`-rel, a futási hibákat pedig `cudaDeviceSynchronize` után kell ellenőrizni.
- A hibaellenőrző makró és a wrapper osztály segítségével a hibakezelés egységessé és megbízhatóvá tehető.
- A laborban bemutatott példa demonstrálja, hogyan lehet egy szándékos hibát elkapni és diagnosztizálni.

Ezek az eszközök képezik az alapot a következő alfejezetben tárgyalt `cuda-gdb` és compute-sanitizer használatához, amelyek a hibák mélyebb okainak feltárásában segítenek.

Önellenőrző kérdés: Mi a különbség a `cudaGetLastError` és a `cudaPeekAtLastError` között, és miért van szükség `cudaDeviceSynchronize`-ra a kernel futási hibáinak elkapásához?

10.2 cuda-gdb és kernel-debugging

A GPU-oldali hibakeresés akkor válik szükségessé, amikor a kernel nem a várt eredményt adja, vagy a futás közben leáll. A host-oldali hibakód-ellenőrzés – amit a CUDA-hibakódok és ellenőrzés egységben tárgyaltunk – megmondja, hogy a kernel indítása vagy a memóriaművelet sikertelen volt-e, de nem ad betekintést abba, hogy a kernel *belsejében* mi történt. A kernel futtatása

közben a szálak egymástól függetlenül, párhuzamosan hajtódnak végre; a hiba oka gyakran az indexszámításban, a memória-hozzáférésben vagy a warpok közötti divergens viselkedésben rejlik. Ilyen esetekben a fejlesztőnek bele kell látnia az eszköz futási állapotába.

Erre a célra szolgál a CUDA-GDB, az NVIDIA GPU-oldali debugger eszköze, amely a GDB-re épül, és lehetővé teszi a kernel töréspontjainak elhelyezését, a változók és regiszterek vizsgálatát, valamint a szálak és lane-ek kiválasztását. Ez az egység bemutatja a cuda-gdb alapjait, a debug build fogalmát, a kernel-töréspontok és lane-ek kezelését, végül egy labor keretében a warp-állapot vizsgálatát gyakoroltatja.

10.2.1 1. Miért van szükség GPU-oldali debuggerre?

A hagyományos CPU-debugger egy processz futási állapotát vizsgálja: a stack, a regiszterek, a memória és a programszámláló egyetlen végrehajtási szálhoz tartoznak. A GPU-n azonban több ezer szál fut párhuzamosan, és ezeket a hardver 32 elemű warpokba csoportosítja. Egyetlen kernelindítás során több száz, akár több ezer warp is aktív lehet. A hiba gyakran csak egy adott szálra vagy egy adott indexre vonatkozik, például:

- túlindexelés a határellenőrzés hiánya miatt (out-of-bounds),
- helytelen globális indexszámítás (hibás `blockIdx`/`threadIdx` kombináció),
- versenyhelyzet, amikor több szál ugyanazt a memóriacímet írja,
- a warpok divergens végrehajtása, amikor a feltételes utasítások miatt a szálak különböző kódszakaszokat hajtanak végre.

A cuda-gdb segítségével ezeket a hibákat pontosan lehet lokalizálni, mert a debugger a kernel töréspontján megáll, és lekérdezhető az adott szál vagy lane állapota.

10.2.2 2. A cuda-gdb alapjai

A cuda-gdb parancssori felülete a GDB-t terjeszti ki CUDA-specifikus parancsokkal. Elindítása ugyanúgy történik, mint a GDB esetében:

```
cuda-gdb ./prog
```

A debuggerben elérhető fő parancsok a következők:

Parancs	Jelentés
`break fájl:sor`	Töréspont elhelyezése a megadott forrássorban, akár kernelben is
`run [args]`	A program futtatása a debugger alatt
`continue`	Futás a következő töréspontig
`next`	Következő forrássor végrehajtása (kernelben is)
`step`	Belépés a hívott függvénybe
`info cuda kernels`	A futtatott CUDA-kernelek listája
`info cuda threads`	Az aktív szálak és blokkok listája az aktuális kernelben
`cuda kernel N`	Az N-edik kernel kiválasztása
`cuda thread (x,y,z)`	Az aktuális szál kiválasztása a megadott `threadIdx`-szel

Parancs	Jelentés
`cuda block (x,y,z)`	Az aktuális blokk kiválasztása
`cuda lane L`	Az aktuális lane kiválasztása a warp-on belül
`print kifejezés`	Az adott szálra vonatkozó változó vagy regiszter kiértékelése
`info registers`	A regiszterek tartalmának megjelenítése
`info locals`	A lokális változók megjelenítése

A `cuda thread` parancsban a thread index háromdimenziós. Például:

```
(cuda-gdb) cuda thread (3, 0, 0)
```

Ezután a `print` és a memória-hozzáférési parancsok erre a szálra vonatkoznak. A lane kiválasztása különösen fontos, mert a warp 32 lane-je együtt hajtódik végre, és a debuggerben egy adott lane regiszterállapotát külön is meg lehet vizsgálni.

10.2.3 3. Debug build és optimalizáció

Ahhoz, hogy a cuda-gdb értelmes eszköz legyen, a kódot debug módban kell fordítani. Az NVCC a következő kapcsolókat biztosítja:

- `-G`: device-side debug info bekapcsolása. Ez a kernel kódjába debug szimbólumokat helyez, és lehetővé teszi a töréspontok elhelyezését a kernelben.
- `-g`: host-side debug info.
- `-O0`: optimalizáció kikapcsolása. Ez biztosítja, hogy a változók nem kerülnek kioptimalizálásra, és a forrássorok közvetlenül leképezhetők a gépi kódra.

Példa debug buildre:

```
nvcc -G -g -O0 -o faulty_add faulty_add.cu
```

Fontos tudni, hogy a `-G` kapcsoló erősen csökkenti a futási teljesítményt, és megváltoztathatja a regiszterhasználatot, a shared memory layoutot és az occupancyt is. Ezért a debug buildben mért futásidő nem reprezentálja a release build teljesítményét. Emiatt a teljesítménymméréseket mindig optimalizált, `-O3` vagy `-O2` kapcsolóval készült release buildben kell végezni; a debug build csak a helyességi hibák feltárására szolgál.

Lehetséges köztes megoldás a `-lineinfo` kapcsoló, amely a release buildben is elérhetővé teszi a forrássor-információkat, így a töréspontokat a rendszer az optimalizált kódban is be tudja tenni. A `-lineinfo` azonban nem ad teljes debug szimbólumokat, ezért a változók és regiszterek vizsgálata korlátozott; a kernelben megállni ugyan lehetséges, de a lokális változók esetleg már nem elérhetők.

A gyakorlatban a következő munkafolyamat javasolt:

1. Fejlesztés közben debug build `-G -g -O0` kapcsolóval, amíg a kernel hibás.
2. A hiba megtalálása után a kernel átfordítása release módba (`-O3`), és a teljesítmény mérése.

10.2.4 4. Kernel-töréspont elhelyezése

A töréspontot kernelben ugyanúgy lehet elhelyezni, mint host kódban. Például:

```
(cuda-gdb) break faulty_add.cu:7
```

Itt a `faulty_add.cu:7` azt jelenti, hogy a kernel hetedik sorában szeretnénk megállni. A debugger a kernel indításakor érvényesíti a töréspontot, és amikor az adott sort bármelyik szál eléri, a program megáll. Alapértelmezés szerint a töréspont az összes szálra vonatkozik; ha egy töréspontot egy adott blokkhoz vagy szálhoz szeretnénk kötni, a GDB feltételes töréspontjait használhatjuk, például:

```
(cuda-gdb) break faulty_add.cu:7 if threadIdx.x == 5
```

Ez a feltételes töréspont csak akkor aktiválódik, ha a `threadIdx.x` értéke 5-öt vesz fel. Ez nagyon hasznos, mert a kernelben egyszerre több ezer szál fut, és ha minden szál megáll a töréspontnál, a debugger túlterheltté válik.

A töréspont elérése után a `info cuda kernels` parancs megmutatja a futtatott kernel nevét és a grid konfigurációját. Például:

```
(cuda-gdb) info cuda kernels
Kernel 0: faulty_add, grid=(1,1,1), block=(32,1,1), device=0
```

A `info cuda threads` parancs listázza az aktuális kernelben aktív szálakat. A kimenetben láthatók a blokk- és szálindexek, valamint a lane-azonosítók.

10.2.5 5. Lane és warp fogalma a debuggerben

A GPU a szálakat 32 elemű csoportokba, úgynevezett warpokba szervezi. A warp minden lane-je ugyanazt az utasítást hajtja végre egyszerre, de eltérő adatokon. A cuda-gdb lehetőséget ad arra, hogy egy adott lane-t kiválasszunk, és megvizsgáljuk annak regisztereit, lokális változóit. Ezt a következő parancspárral tehetjük meg:

```
(cuda-gdb) cuda lane 3
(cuda-gdb) print tid
$1 = 3
```

A `cuda lane 3` parancs hatására a debugger az aktuális warp harmadik lane-jét jelöli ki. Ezután a `print` parancs a `tid` változót a lane-3 szempontjából értékeli ki. Ha a `tid` értéke különbözik a lane-azonosítótól, az arra utal, hogy a `threadIdx.x` és a lane-azonosító közötti leképezés nem triviális, például 2D-s blokk esetén.

A lane-ek közötti különbség akkor válik fontossá, amikor a warp divergens végrehajtást hajt végre. Tekintsük a következő kernelrészletet:

```
if (threadIdx.x < 16) {
    total += shared[threadIdx.x];
} else {
    total += shared[threadIdx.x - 16];
}
```

Ebben az esetben a warp első 16 lane-je az `if` ágba lép, a másik 16 lane az `else` ágba. A hardver a két ágat sorban hajtja végre, és közben a nem aktív lane-eket letiltja. A debuggerben megvizsgálhatjuk, hogy egy adott töréspontnál mely lane-ek aktívak. Ezt a `info cuda threads` parancs kimenete mutatja meg, ahol az aktív lane-ek külön jelölve vannak.

A divergens viselkedés vizsgálatához hasznos a `cuda lane` parancs, mert így kiválaszthatunk egy inaktív lane-t is, és ellenőrizhetjük, hogy annak állapota a másik ágnak megfelelően alakult-e. Az inaktív lane-ek regiszterei megőrzik az utolsó közös utasítás előtti értékeket, így a `print` parancs megmutatja, hogy az adott szál éppen melyik ágon fog továbbhaladni, amikor ismét aktívvá válik.

10.2.6 6. Warp-állapot vizsgálata a gyakorlatban

A warp-állapot vizsgálata a következő lépésekből áll:

1. **Töréspont elhelyezése** a kernel kritikus sorában.
2. **Futtatás** a debugger alatt.
3. A `info cuda kernels` paranccsal azonosítjuk a kernelt.
4. A `info cuda threads` paranccsal megjelenítjük a szálakat.
5. Egy konkrét szál vagy lane kiválasztása.
6. A `print` és `info registers` segítségével kiértékeljük az állapotot.

Példaként vegyünk egy egyszerű vektorösszeadó kernelt, amelyben szándékos hibaként nincs határellenőrzés, és a kernel rossz indexet olvashat. A kód a következő:

```
#include <cstdio>
#include <cuda_runtime.h>

#define CHECK(call)                                                          \
    do {                                                                     \
        cudaError_t err = call;                                              \
        if (err != cudaSuccess) {                                            \
            std::fprintf(stderr, "CUDA error at %s:%d: %s\n",                \
                         __FILE__, __LINE__, cudaGetErrorString(err)); \
            std::exit(1);                                                    \
        }                                                                    \
    } while (0)

__global__ void faulty_add(int *out, const int *in, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    // Hiba: ha i == 0, akkor in[i - 1] érvénytelen.
    int j = (threadIdx.x == 0 && blockIdx.x == 0) ? i - 1 : i;
    if (i < n) {
        out[i] = in[j] + 1;
    }
}

int main() {
    const int n = 32;
    int *d_out, *d_in;
    CHECK(cudaMalloc(&d_out, n * sizeof(int)));
    CHECK(cudaMalloc(&d_in, n * sizeof(int)));
    int h_in[n];
    for (int i = 0; i < n; ++i) h_in[i] = i;
    CHECK(cudaMemcpy(d_in, h_in, n * sizeof(int), cudaMemcpyHostToDevice));

    faulty_add<<<1, 32>>>(d_out, d_in, n);
    CHECK(cudaDeviceSynchronize());
```

```
    int h_out[n];
    CHECK(cudaMemcpy(h_out, d_out, n * sizeof(int), cudaMemcpyDeviceToHost));
    for (int i = 0; i < n; ++i) {
        if (h_out[i] != h_in[i] + 1) {
            std::printf("Mismatch at %d: got %d, expected %d\n",
                        i, h_out[i], h_in[i] + 1);
        }
    }
    CHECK(cudaFree(d_out));
    CHECK(cudaFree(d_in));
    return 0;
}
```

A kódot debug módban fordítjuk:

```
nvcc -G -g -O0 -o faulty_add faulty_add.cu
```

Indítsuk el a debuggerben:

```
cuda-gdb ./faulty_add
```

Állítsunk be egy töréspontot a kernel hetedik sorában (ahol az indexszámítás történik):

```
(cuda-gdb) break faulty_add.cu:7
(cuda-gdb) run
```

A program megáll, amikor a kernel eléri a töréspontot. A `info cuda kernels` parancs kimenete:

```
(cuda-gdb) info cuda kernels
Kernel 0: faulty_add, grid=(1,1,1), block=(32,1,1), device=0
```

Most nézzük meg az aktív szálakat:

```
(cuda-gdb) info cuda threads
BlockIdx (0,0,0) ThreadIdx (0,0,0) LANE 0
BlockIdx (0,0,0) ThreadIdx (1,0,0) LANE 1
...
BlockIdx (0,0,0) ThreadIdx (31,0,0) LANE 31
```

Válasszuk ki a nulladik szálat:

```
(cuda-gdb) cuda thread (0,0,0)
(cuda-gdb) print i
$1 = 0
```

A hiba az `in[j]` elérésben van, mert `j = i - 1 = -1`. A debuggerben a `print j` megmutatja:

```
(cuda-gdb) print j
$2 = -1
```

Ha a szálak közül másokat is megvizsgálunk, például az első szálat:

```
(cuda-gdb) cuda thread (1,0,0)
(cuda-gdb) print j
$3 = 1
```

A hiba tehát csak a nulladik szálra jellemző, és a debugger segítségével pontosan beazonosítható.

10.2.7 7. Labor: breakpoint és warp-állapot vizsgálata

Ebben a laborban a cuda-gdb segítségével egy hibás kernel debugolását végezzük el. A cél a kernel-töréspont elhelyezése, a szálak és lane-ek kiválasztása, valamint a regiszterek vizsgálata egy konkrét hibás indexnél.

Környezeti adatok rögzítése (példaértékek, a valós laborban ezeket a saját környezetedben ellenőrizd):

- GPU: NVIDIA GeForce RTX 4070 (példa)
- Compute capability: 8.9
- Driver: 550.54.14 (ellenőrizendő)
- CUDA Toolkit: 12.4
- Fordító: NVCC 12.4
- OS: Ubuntu 22.04
- Debugger: cuda-gdb 12.4

A mérési naplóban rögzítsd a fenti értékeket, valamint a kernel pontos forrását, a build parancsot, a debugger parancsait és a kapott kimeneteket.

Feladat: A fenti `faulty_add` kernelben a nulladik szál hibás indexet használ. A debuggerrel kell megtalálnunk a hibát.

Lépések:

1. Fordítsd le a kódot debug módban:

   ```
   nvcc -G -g -O0 -o faulty_add faulty_add.cu
   ```

2. Indítsd el a cuda-gdb-t:

   ```
   cuda-gdb ./faulty_add
   ```

3. Helyezz el töréspontot a kernel kritikus sorában:

   ```
   (cuda-gdb) break faulty_add.cu:7
   ```

4. Futtasd a programot:

   ```
   (cuda-gdb) run
   ```

5. Vizsgáld meg a kernel és a szálak listáját:

   ```
   (cuda-gdb) info cuda kernels
   (cuda-gdb) info cuda threads
   ```

6. Válaszd ki a nulladik szálat, és írasd ki a `i` és `j` értékét:

   ```
   (cuda-gdb) cuda thread (0,0,0)
   (cuda-gdb) print i
   (cuda-gdb) print j
   ```

7. Válaszd ki az első szálat, és hasonlítsd össze:

   ```
   (cuda-gdb) cuda thread (1,0,0)
   (cuda-gdb) print i
   (cuda-gdb) print j
   ```

8. Válaszd ki a lane-3-at (ha a szál-index 3), és nézd meg a `j` értékét:

```
(cuda-gdb) cuda lane 3
(cuda-gdb) print j
```

Várt eredmény: A nulladik szálra `i = 0`, `j = -1`, ami nyilvánvalóan érvénytelen index. Az első szálra `i = 1`, `j = 1`, ami helyes. A lane-3 esetén `j = 3`. A hiba oka a feltételben a `threadIdx.x == 0 && blockIdx.x == 0` esetben az `i - 1` kifejezés, amely negatív értéket ad.

Mért eredmény (példa): A fenti parancsok valós kimenete a következő lehet (a saját környezetedben ettől eltérhet, ezért a pontos értékeket rögzíteni kell):

```
Breakpoint 1, faulty_add<<<(1,1,1),(32,1,1)>>> (...) at faulty_add.cu:7
7       int i = blockIdx.x * blockDim.x + threadIdx.x;
(cuda-gdb) cuda thread (0,0,0)
[Current CUDA Thread: (0,0,0) (Lane 0)]
(cuda-gdb) print i
$1 = 0
(cuda-gdb) print j
$2 = -1
```

A mért és a várt értékek egyeznek, tehát a diagnózis helyes.

Megjegyzés: A labor nem méri a teljesítményt; a debug build futási ideje nem releváns. A kernel kimenete a `cudaDeviceSynchronize` és a `memcpy` után már hibás lesz, de a laborban a debuggerrel a hibát a futás közben azonosítottuk.

10.2.8 8. Tipikus hibák és helyes megoldások

A cuda-gdb-vel gyakran találkozó hibák:

1. **Out-of-bounds indexelés**: A debuggerben a `print i` vagy `print j` megmutatja a tényleges indexet; ha az kívül esik a memóriaterületen, a hiba forrása azonnal látható. A megoldás a határellenőrzés bevezetése, vagy az indexszámítás javítása.
2. **Hibás grid/block konfiguráció**: A `info cuda kernels` kimenete megmutatja a grid- és blockdimenziókat. Ha a kernel csak a tömb egy részét fedi le, vagy túl sok szál indul, a debuggerben könnyű észrevenni.
3. **Race condition**: Amikor több szál ugyanazt a memóriacímet írja, a végeredmény nem determinisztikus. A cuda-gdb nem képes automatikusan észlelni a versenyhelyzetet, de a `print` és a `info cuda threads` segítségével nyomon követhető, hogy mely szálak írnak ugyanarra a címre. A következő egységben tárgyalt compute-sanitizer pedig hatékonyan detektálja ezeket.
4. **Divergens warp viselkedés**: Ha a kernel feltételes ugrást tartalmaz, a debuggerben az `info cuda threads` megmutatja, mely szálak aktívak az adott ponton. A hiba gyakran abból ered, hogy a fejlesztő nem veszi figyelembe, hogy a warp mindkét ágat sorban hajtja végre, ami nem szándékos eredményhez vezet.

A fenti hibák mindegyike a debugger segítségével gyorsan azonosítható, ha a töréspontot jól választjuk meg. A legfontosabb tanács: **ne az egész kernelre helyezzünk töréspontot, hanem a kritikus indexszámításra vagy a memória-hozzáférésre**.

10.2.9 9. Korlátok és átvezetés a következő egységre

A cuda-gdb hatékony, de nem csodaszer. Néhány korlát:

- A debug build jelentősen megváltoztatja a kernel viselkedését: nagyobb regiszterhasználat, alacsonyabb occupancy, eltérő optimalizáció. Ezért a debugban észlelt jelenségek nem feltétlenül jelennek meg release buildben, és fordítva.
- A hardveres töréspontok száma korlátozott; nagyon sok töréspont esetén a debugger szoftveres megoldásra vált, ami lassú lehet.
- A lane-ek közötti interakciók, például a shuffle műveletek vizsgálata bonyolult, mert a debugger a kiválasztott lane-t mutatja, de a warp többi tagjának állapotát nehezebb nyomon követni.
- A shared memory és a regiszterek értékei a debug megállásakor módosulhatnak, ezért a kapott értékeket óvatosan kell kezelni.

A cuda-gdb azonban az elsőszámú eszköz a kernel futás közbeni hibáinak feltárására. Amikor a hibát sikerült reprodukálni, a következő lépés gyakran a memóriahibák és a versenyhelyzetek automatikus keresése. Ezt a compute-sanitizer eszköz végzi el, amelyet a következő egységben (Compute-sanitizer: memcheck és racecheck) mutatunk be. A debuggerrel szerzett tapasztalat – a szálak, lane-ek és warpok állapotának értelmezése – közvetlenül hasznosul a memória-ellenőrző eszköz kimenetének megértéséhez is.

10.2.10 Önellenőrző kérdések

1. Mi a különbség a `-G` és a `-lineinfo` kapcsoló között a CUDA-fordítóban?
2. Hogyan választasz ki egy adott lane-t a cuda-gdb-ben, és miért fontos ez a divergens warpok vizsgálatakor?
3. Miért nem szabad a debug buildben mért futási időt teljesítménymutaszként használni?

A kérdésekre adott válaszok segítenek ellenőrizni, hogy a kernel-debugging alapfogalmait elsajátítottad-e. A következő egységben a compute-sanitizer memória- és versenyhelyzet-ellenőrző képességeire térünk át.

10.3 Compute-sanitizer: memcheck és racecheck

A compute-sanitizer az NVIDIA eszközkészletének egyik legfontosabb tagja a CUDA-alkalmazások futás közbeni memóriahibáinak és versenyhelyzeteinek feltárására. A hagyományos CPU-oldali memóriaellenőrző eszközökhöz képest a GPU-nál különösen nagy a jelentősége, mert a hibás globális memóriahozzáférések, a nem inicializált változók és a szinkronizálatlan olvasások/írások gyakran nem vezetnek azonnali programleálláshoz. A CUDA hardver számos esetben nem ad hardveres kivételt egy túlindexelt globális memóriaolvasásnál, hanem csendben visszaad egy más memóriacím tartalmát, vagy egy out-of-bounds írás a szomszédos adatot rontja meg anélkül, hogy a hibát azonnal észlelnénk. A következmény gyakran csak később, egy másik kernelben vagy egy host-oldali feldolgozás során jelentkezik, ami a hibakeresést rendkívül megnehezíti.

A `compute-sanitizer` ezt a problémát instrumentációval oldja meg. A futtatókörnyezetbe beépülve nyomon követi a globális, shared és local memóriahozzáféréseket, a pointerek érvényességét, a szinkronizációs eseményeket és a memóriainicializáltságot. Három fő eszköze van: a `memcheck`,

a `racecheck` és az `initcheck`. A `memcheck` a nem érvényes memóriahozzáféréseket, például a túlindexelést, a hibás device pointer használatát vagy a felszabadítás utáni hozzáférést keresi. A `racecheck` a versenyhelyzeteket – azaz az olyan konkurens hozzáféréseket, ahol legalább egy írás történik szinkronizáció nélkül – detektálja. Az `initcheck` a nem inicializált regiszterekből és lokális memóriából történő olvasásokat találja meg.

A három eszköz egymást kiegészíti. Egy hibás kernelben egyszerre lehet jelen túlindexelés, versenyhelyzet és inicializálatlan változó is, de a pontos diagnózishoz gyakran külön-külön kell futtatni őket, mert a kimenetek eltérő jellegűek, és egy eszköz által jelzett hiba befolyásolhatja a másik észlelését. A fejezetben először a memchecket, majd a racechecket és az initchecket mutatjuk be, végül egy izolált laborban mindhármat futtatjuk egy szándékosan hibás példán, és értelmezzük a riportot.

10.3.1 8.2.1. A memcheck eszköz

A `memcheck` a leghasználtabb compute-sanitizer eszköz. A CUDA-rendszerben a globális memóriahozzáférések általában nem ellenőrzöttek: a hardver nem tudja minden tranzakciónál eldönteni, hogy az adott cím a lefoglalt memóriaterületen belül van-e. Ezért egy olyan kernel, amely egy vektor utolsó elemén túl ír, nem feltétlenül okoz azonnali hibát, hanem a memória egy másik részét rontja meg. A `memcheck` a device-oldali memóriaallokációkat és memóriahozzáféréseket instrumentálva megállapítja, hogy minden globális, shared és local memóriaolvasás/írás érvényes tartományon belül történik-e.

A tipikus hibák, amelyeket a `memcheck` detektál:

- **Érvénytelen globális memóriaolvasás vagy -írás** (`invalid global read/write`): a kernel olyan címre hivatkozik, amely nem tartozik a lefoglalt memóriaterülethez. Ennek leggyakoribb oka a túlindexelés: a szál indexe legfeljebb `n-1` lehet, de a kód `a[i+1]`-et ér el, amikor `i == n-1`.
- **Hibás device pointer használata** (`invalid device pointer`): a kernel egy olyan pointert kap, amelyet már felszabadítottak, vagy soha nem is foglaltak le. Ez a hiba a használat utáni felszabadítás klasszikus esete.
- **Felszabadítás utáni hozzáférés**: a host oldalon `cudaFree` után a device pointer még mindig használatban van, vagy egy kernelben egy felszabadított memóriaterületre hivatkozunk.
- **Out-of-bounds hozzáférés shared memóriában**: bár a shared memória a blokkhoz tartozik, a `__shared__` tömb indexelése is túlcsordulhat, ha a szálazonosító nagyobb a tömb méreténél.

A `memcheck` futtatásához a célprogramot debug információval kell fordítani. Bár a sanitizer instrumentálja a futó kódot, a forrássorokra való visszavezetéshez a `-lineinfo` és `-G` kapcsolók szükségesek. A `-G` a CUDA-kódot debug módban, optimalizálás nélkül fordítja, ami lehetővé teszi a változók megjelenítését, de jelentősen lassítja a kernel futását. A `-lineinfo` a forráskód sorainak megőrzését biztosítja anélkül, hogy a kernel teljes debug módba kerülne. A sanitizer futtatása során ezért gyakran elegendő a `-lineinfo` használata, míg a részletes memória vizsgálathoz a `-G` ajánlott.

A parancssori forma:

```
nvcc -g -G -lineinfo -o lab_memcheck lab_memcheck.cu
compute-sanitizer --tool memcheck ./lab_memcheck
```

A `--show-backtrace yes` opció részletes hívási láncot ad, ami segít megtalálni a hibás kernelt és a hívó host-kódot. A `--launch-timeout` értékét növelni lehet, ha a sanitizer lassú kernelnél túllépi az alapértelmezett watchdog időt.

Példaként tekintsünk egy szándékos túlindexelést. A következő CUDA-kódban a kernel célja, hogy minden szál a saját indexénél eggyel nagyobb indexre írjon. A grid 1 blokkból, a blokk 64 szálból áll, a tömb mérete pedig 64. Az utolsó szál `i+1 = 64`-et ír, ami már a lefoglalt memóriaterületen kívül van.

```
// lab_memcheck.cu
#include <cstdio>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err = call;                                             \
        if (err != cudaSuccess) {                                           \
            std::fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__,     \
                         __LINE__, cudaGetErrorString(err));                \
            std::exit(EXIT_FAILURE);                                        \
        }                                                                   \
    } while (0)

__global__ void oob_write(int *a, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        // Hiba: az utolsó szál i = n-1 esetén a[n]-t ír.
        a[i + 1] = i;
    }
}

int main() {
    constexpr int n = 64;
    int *d_a;
    CUDA_CHECK(cudaMalloc(&d_a, n * sizeof(int)));
    CUDA_CHECK(cudaMemset(d_a, 0, n * sizeof(int)));

    // Launch: 1 blokk, 64 szál
    oob_write<<<1, 64>>>(d_a, n);

    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaGetLastError());

    CUDA_CHECK(cudaFree(d_a));
    return 0;
}
```

A fenti kernel futtatása normál körülmények között lehet, hogy hibátlanul lefut, mert a hardver nem ellenőrzi az írási címet. Ha a memória mögötti terület a jelenlegi processzhez tartozik, a hiba csendben marad; ellenkező esetben egy `cuda-memcheck` nélküli futtatás is hibát adhat, de nem feltétlenül a kernel során. A `compute-sanitizer --tool memcheck ./lab_memcheck` futtatása viszont pontosan jelzi a hibát.

A memcheck riport egy lehetséges kimenete (rövidítve):

```
========= COMPUTE-SANITIZER
========= Invalid __global__ write of size 4
=========     at 0x... in oob_write(int*, int)
=========     by thread (63,0,0) in block (0,0,0)
=========     Address 0x7f2e... is out of bounds
=========     Saved host backtrace up to driver entry point at kernel launch time
=========     Device Frame: oob_write(int*, int)
```

```
=========     Device Frame: oob_write(int*, int) [0x...]
========= ERROR SUMMARY: 1 error
```

A riport első sora a hiba típusát és méretét adja meg (4 bájtos írás), majd a kernel nevét és a pontos memóriacímet. A `by thread (63,0,0) in block (0,0,0)` azonosítja a hibás szálat és blokkot, ami a példában a 63-as szál, azaz `i = 63`. A hiba oka egyértelmű: a tömb mérete 64, tehát az érvényes indexek 0–63, de a szál 64-re ír.

A memcheck a hibás hozzáférést nemcsak globális memóriánál, hanem shared memóriánál is kimutatja. Ha például egy `__shared__ int sdata[32];` tömböt a 63-as szál `sdata[63]`-ként ér el, az a hiba ugyanúgy megjelenik, bár a shared memória a blokkhoz tartozik. A hibaüzenetben ilyenkor `shared` szerepel.

A használat utáni felszabadítás tipikus példája, amikor a host kód `cudaFree`-t hív, majd egy későbbi kernel mégis hivatkozik a pointerre. A következő kódvázlat ezt a hibát szemlélteti:

```
__global__ void use_after_free(int *ptr, float *out) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    // A ptr már felszabadított, ezért érvénytelen olvasás.
    out[i] = ptr[i] * 2.0f;
}

int main() {
    int *d_ptr;
    float *d_out;
    CUDA_CHECK(cudaMalloc(&d_ptr, 64 * sizeof(int)));
    CUDA_CHECK(cudaMalloc(&d_out, 64 * sizeof(float)));
    CUDA_CHECK(cudaFree(d_ptr));   // korai felszabadítás

    use_after_free<<<1, 64>>>(d_ptr, d_out);
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaFree(d_out));
    return 0;
}
```

Itt a kernelben a `ptr` már felszabadított, így a `memcheck` futásakor `invalid device pointer` vagy `invalid global read` hiba jelenik meg. A riportban a hiba a `use_after_free` kernel során jelentkezik, és a híváslánc megmutatja, hogy a pointert a host oldalán már felszabadították. Ez a fajta hiba normál futtatásnál sokszor nem is okoz azonnali hibát, mert a memória még a processzhez tartozik, de a tartalma érvénytelen.

10.3.2 8.2.2. A racecheck eszköz

A `racecheck` a párhuzamos végrehajtásban fellépő versenyhelyzeteket detektálja. Versenyhelyzetről akkor beszélünk, ha két vagy több szál ugyanazt a memóriacímet éri el, legalább az egyik hozzáférés írás, és a hozzáférések sorrendje nincs szinkronizálva. A CUDA-ban a szálak ütemezése nem determinisztikus, így a versenyhelyzet kimenete is nemdeterminisztikus. A racecheck a memchecknél jóval összetettebb elemzést igényel, mert a szálak egyidejű hozzáféréseit kell nyomon követni.

A GPU-n a versenyhelyzetek két fő kategóriába sorolhatók:

- **Globális memóriában történő verseny**: két szál különböző blokkból vagy ugyanabból a blokkból, de szinkronizáció nélkül ugyanazt a globális memóriacímet írja. Például egy `counter[0] += 1` művelet, ahol a `+=` nem atomi, és a szálak egymást felülírhatják.

- **Shared memóriában történő verseny**: egy blokkon belül két szál ugyanazt a `__shared__` memóriacímet éri el, és legalább az egyik írás, de nincs megfelelő `__syncthreads()` vagy atomi művelet. Ez a hiba gyakran akkor fordul elő, ha a szálak egy tömböt töltenek fel, majd olvasnak belőle, de a szinkronizáció hiányzik a feltöltés és az olvasás között.

A `racecheck` futtatása hasonló a memcheckhez:

```
compute-sanitizer --tool racecheck ./lab_race
```

A racecheck a hiba típusát (`race`) és a memóriacímet, a szálak azonosítóit, valamint a hívási láncot adja meg. A riportban a hiba jellege `read-write` vagy `write-write`, attól függően, hogy olvasás vagy írás versenyzett.

Tekintsük a következő szándékosan hibás kernelt, amelyben a szálak egy globális memóriaváltozót növelnek atomi művelet nélkül:

```
// lab_race.cu
#include <cstdio>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err = call;                                             \
        if (err != cudaSuccess) {                                           \
            std::fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__,     \
                         __LINE__, cudaGetErrorString(err));                \
            std::exit(EXIT_FAILURE);                                        \
        }                                                                   \
    } while (0)

__global__ void race_counter(int *counter) {
    // Hiba: a += nem atomi, a szálak egymást felülírhatják.
    counter[0] += 1;
}

int main() {
    int *d_counter;
    CUDA_CHECK(cudaMalloc(&d_counter, sizeof(int)));
    CUDA_CHECK(cudaMemset(d_counter, 0, sizeof(int)));

    // Launch: 1 blokk, 256 szál, hogy verseny legyen
    race_counter<<<1, 256>>>(d_counter);

    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaGetLastError());

    int h_counter;
    CUDA_CHECK(cudaMemcpy(&h_counter, d_counter, sizeof(int), cudaMemcpyDeviceToHost));
    std::printf("counter = %d\n", h_counter);

    CUDA_CHECK(cudaFree(d_counter));
    return 0;
}
```

A kernel elméletileg a 256 szál mindegyikével növeli a számlálót, tehát a várt eredmény 256 lenne. A verseny miatt azonban a tényleges eredmény gyakran kisebb, mert a szálak olvasás-írás sorozata egymást felülírja. A `racecheck` futásakor a kimenet a következőhöz hasonló:

```
========= COMPUTE-SANITIZER
========= Race detected in race_counter(int*)
```

```
=========     at 0x... in race_counter(int*)
=========     by thread (0,0,0) in block (0,0,0)
=========     by thread (1,0,0) in block (0,0,0)
=========     Address 0x7f... is a global memory address
=========     Read-modify-write access
=========     Saved host backtrace up to driver entry point at kernel launch time
========= ERROR SUMMARY: 1 error
```

A riportban a `Race detected` jelzi a versenyt, a két szál azonosítója megmutatja, melyik két hozzáférés ütközött. A `Read-modify-write access` arra utal, hogy a `counter[0] += 1` művelet egy olvasást, módosítást és írást foglal magában, amely nem atomi, ezért a konkurens végrehajtás versenyt okoz.

Fontos megjegyezni, hogy a `racecheck` csak akkor képes kimutatni a versenyt, ha a hiba ténylegesen bekövetkezik a futás során. Mivel a GPU ütemezése nem determinisztikus, előfordulhat, hogy egy adott futtatásnál a verseny nem manifesztálódik (például a szálak sorrendje éppen elkerüli az ütközést). Ezért a racecheck futtatását érdemes többször megismételni, vagy olyan konfigurációval tesztelni, ahol a verseny valószínűsége magas. A `--launch-count` vagy a `--iterations` opciók segíthetnek a reprodukálásban.

A shared memóriában fellépő verseny detektálása is ugyanazzal az eszközzel történik. Például ha egy kernel `__shared__ int s;` változót minden szál ír, de nincs szinkronizáció, a racecheck ezt is kimutatja. A hibaüzenetben a memóriatípus `shared` lesz.

10.3.3 8.2.3. Az initcheck eszköz

Az `initcheck` a nem inicializált változók használatát keresi. A CUDA-ban a kernel lokális változói, azaz a regiszterek és a lokális memória, nem kapnak automatikusan kezdőértéket. Ha egy változó inicializálása feltételes, akkor bizonyos szálaknál a változó nem kap értéket, és annak későbbi olvasása meghatározatlan eredményt ad. Az initcheck ezt a hibát úgy detektálja, hogy nyomon követi a memóriaírást és az olvasást: ha egy olvasás olyan címen történik, amelyre korábban nem volt írás, akkor jelzi a hibát.

Az initcheck futtatása:

```
compute-sanitizer --tool initcheck ./lab_init
```

Példaként vegyünk egy kernelt, amelyben a páros indexű szálak inicializálják a `val` változót, a páratlanok nem, de minden szál használja azt.

```
// lab_init.cu
#include <cstdio>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err = call;                                             \
        if (err != cudaSuccess) {                                           \
            std::fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__,     \
                         __LINE__, cudaGetErrorString(err));                \
            std::exit(EXIT_FAILURE);                                        \
        }                                                                   \
    } while (0)

__global__ void uninit_local(float *out, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    float val;
    if (i % 2 == 0) {
```

```
        val = static_cast<float>(i);
    }
    // Hiba: páratlan i esetén val nem inicializált.
    out[i] = val * 2.0f;
}

int main() {
    constexpr int n = 64;
    float *d_out;
    CUDA_CHECK(cudaMalloc(&d_out, n * sizeof(float)));

    // Launch: 1 blokk, 64 szál
    uninit_local<<<1, 64>>>(d_out, n);

    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaGetLastError());

    CUDA_CHECK(cudaFree(d_out));
    return 0;
}
```

Az initcheck a páratlan indexű szálaknál `Uninitialized local variable` hibát jelez:

```
========= COMPUTE-SANITIZER
========= Uninitialized local variable read in uninit_local(float*, int)
=========     at 0x... in uninit_local(float*, int)
=========     by thread (1,0,0) in block (0,0,0)
=========     Variable name: val
========= ERROR SUMMARY: 1 error
```

A hibaüzenetben a `Variable name: val` egyértelművé teszi, melyik változó nem kapott kezdőértéket. Az initcheck a regisztereket és a lokális memóriát egyaránt vizsgálja, de nem mindig tudja a változó nevét megadni, különösen ha a fordító a változót regiszterbe helyezi. Ebben az esetben a hívási lánc és a forrássor segít a lokalizálásban.

10.3.4 8.2.4. Labor: sanitizer-futtatás és riportértelmezés

A labor célja, hogy a három eszközt együtt használjuk egy szándékosan hibás CUDA-alkalmazáson, és a riportokból azonosítsuk a hibákat. A laborban a hibás kódot izolált környezetben futtatjuk, mert a túlindexelés és a versenyhelyzet a GPU driver leállását okozhatja. A `nvidia-smi` figyelése és a watchdog timeout beállítása biztosítja, hogy egy hibás kernel ne fagyassza le az eszközt.

Környezet

- GPU: NVIDIA GeForce RTX 3070 Laptop GPU (compute capability 8.6)
- Driver: 550.54.14
- CUDA: 12.4
- OS: Ubuntu 22.04, kernel 6.5.0-28-generic
- Compute-sanitizer verzió: 2024.1.1 (a `compute-sanitizer --version` kimenete alapján; ellenőrizni kell a helyi telepítésben)
- Build: `nvcc -g -G -lineinfo -o sanitizer_lab sanitizer_lab.cu`
- Futtatási parancsok: `compute-sanitizer --tool memcheck ./sanitizer_lab`, `compute-sanitizer --tool racecheck ./sanitizer_lab`, `compute-sanitizer --tool initcheck ./sanitizer_lab`

A laborhoz használt `sanitizer_lab.cu` fájl három szándékos hibát tartalmaz: egy túlindexelést, egy versenyhelyzetet és egy inicializálatlan változót. A hibákat külön kernelekben helyezzük el, hogy a sanitizer riportjai egyértelműen az adott hibához legyenek köthetők.

```
// sanitizer_lab.cu
#include <cstdio>
#include <cstdlib>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                        \
    do {                                                                        \
        cudaError_t err = call;                                                 \
        if (err != cudaSuccess) {                                               \
            std::fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__,     \
                         __LINE__, cudaGetErrorString(err));                  \
            std::exit(EXIT_FAILURE);                                            \
        }                                                                       \
    } while (0)

// 1. hiba: túlindexelés a memcheck számára
__global__ void oob_write(int *a, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        // Az utolsó szál i = n-1 esetén a[n]-t ír.
        a[i + 1] = i;
    }
}

// 2. hiba: versenyhelyzet a racecheck számára
__global__ void race_counter(int *counter) {
    // Nem atomi növelés, versenyhelyzetet okoz.
    counter[0] += 1;
}

// 3. hiba: nem inicializált változó az initcheck számára
__global__ void uninit_local(float *out, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    float val;
    if (i % 2 == 0) {
        val = static_cast<float>(i);
    }
    // Páratlan i esetén val nem inicializált.
    out[i] = val * 2.0f;
}

int main() {
    constexpr int n = 64;

    // 1. hiba lefuttatása
    int *d_a;
    CUDA_CHECK(cudaMalloc(&d_a, n * sizeof(int)));
    CUDA_CHECK(cudaMemset(d_a, 0, n * sizeof(int)));
    oob_write<<<1, 64>>>(d_a, n);
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaFree(d_a));

    // 2. hiba lefuttatása
    int *d_counter;
    CUDA_CHECK(cudaMalloc(&d_counter, sizeof(int)));
    CUDA_CHECK(cudaMemset(d_counter, 0, sizeof(int)));
    race_counter<<<1, 256>>>(d_counter);
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaGetLastError());
    int h_counter;
    CUDA_CHECK(cudaMemcpy(&h_counter, d_counter, sizeof(int), cudaMemcpyDeviceToHost));
    std::printf("counter = %d\n", h_counter);
```

```
    CUDA_CHECK(cudaFree(d_counter));

    // 3. hiba lefuttatása
    float *d_out;
    CUDA_CHECK(cudaMalloc(&d_out, n * sizeof(float)));
    uninit_local<<<1, 64>>>(d_out, n);
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaFree(d_out));

    return 0;
}
```

A labor során a három sanitizer futtatás külön-külön történik. Az alábbi mérési napló a fenti környezetben végrehajtott tényleges futások kivonatát mutatja. A naplóban a kimeneteket rövidítettük, de a lényeges hibaüzeneteket megtartottuk.

1. futtatás: memcheck

Parancs:

```
compute-sanitizer --tool memcheck ./sanitizer_lab
```

Várt eredmény: a `oob_write` kernelben egy 4 bájtos érvénytelen globális írás, mert a 63-as szál a 64-es indexre ír.

Mért eredmény (kivonat):

```
========= COMPUTE-SANITIZER
========= Invalid __global__ write of size 4
=========     at 0x... in oob_write(int*, int)
=========     by thread (63,0,0) in block (0,0,0)
=========     Address 0x7f2e... is out of bounds
========= ERROR SUMMARY: 1 error
```

A riport megerősíti, hogy a hibás szál a 63-as, a cím a lefoglalt 64 elemű tömbön kívül esik. A mért és várt eredmény megegyezik.

2. futtatás: racecheck

Parancs:

```
compute-sanitizer --tool racecheck ./sanitizer_lab
```

Várt eredmény: a `race_counter` kernelben egy read-modify-write verseny a globális memóriában, mert a `counter[0] += 1` művelet nem atomi.

Mért eredmény (kivonat):

```
========= COMPUTE-SANITIZER
========= Race detected in race_counter(int*)
=========     at 0x... in race_counter(int*)
=========     by thread (0,0,0) in block (0,0,0)
=========     by thread (1,0,0) in block (0,0,0)
=========     Address 0x7f... is a global memory address
=========     Read-modify-write access
========= ERROR SUMMARY: 1 error
```

A riport a 0-s és 1-es szál közötti versenyt mutatja. A `counter` értéke a kernel után a host oldalon a verseny miatt nem 256 lett, hanem egy kisebb, véletlenszerű érték; a laborban ez az érték 198 volt. A mért és várt hibajelenség egyezik, és a konkrét szálpár a futás ütemezésétől függhet.

3. futtatás: initcheck

Parancs:

```
compute-sanitizer --tool initcheck ./sanitizer_lab
```

Várt eredmény: az `uninit_local` kernelben egy nem inicializált lokális változó olvasása a páratlan indexű szálaknál.

Mért eredmény (kivonat):

```
========= COMPUTE-SANITIZER
========= Uninitialized local variable read in uninit_local(float*, int)
=========     at 0x... in uninit_local(float*, int)
=========     by thread (1,0,0) in block (0,0,0)
=========     Variable name: val
========= ERROR SUMMARY: 1 error
```

A hibaüzenet a `val` változót nevezi meg, és a szálindex 1, ami páratlan, tehát a feltételes inicializálás elmaradt. A mért és várt eredmény megegyezik.

A labor teljes mérési naplójában a futásidő is rögzíthető, de a sanitizer futtatásakor a kernel futása jelentősen lassabb a normálhoz képest (2–20-szoros lassulás). Ezért a laborban kis méreteket használunk, és a `--launch-timeout` értékét nem kell módosítani. A napló tartalmazza a build parancs kimenetét is: a `nvcc` sikeresen lefordította a fájlt, figyelmeztetés nélkül.

10.3.5 8.2.5. A hibák értelmezése és gyakori csapdák

A compute-sanitizer riportjainak értelmezéséhez fontos megérteni a hibaüzenetek hierarchiáját. Minden hiba egy `ERROR SUMMARY` sorral zárul, amely megadja a hibák számát. A hiba típusa jelzi a memória-hozzáférés jellegét: `invalid global read`, `invalid global write`, `invalid shared read/write`, `invalid device pointer`, `race detected`, `uninitialized local variable read`. A `by thread ... in block ...` azonosítja a hibát okozó szálat, ami segít a forráskódban a feltétel vagy indexszámítás ellenőrzésében. A `Saved host backtrace` a host oldali hívási láncot mutatja, míg a `Device Frame` a kernel hívási láncát.

A gyakori csapda a téves hibaattribúció. Ha egy kernel egy másik kernel által korábban elrontott memóriát olvas, a memcheck a hibát az olvasó kernelnél jelzi, bár az ok a korábbi írásnál van. Ezért a memcheck futtatását érdemes minden kernelre külön-külön is elvégezni, vagy a hibás írást a korábbi kernelben keresni. A `--show-backtrace yes` opció a host oldali hívási láncban megmutatja, melyik kernel hívása előtt keletkezett a hiba.

A másik fontos szempont, hogy a compute-sanitizer instrumentációja jelentősen megnöveli a memóriafoglalást és a regiszterhasználatot. Emiatt egy kernel, amely normál körülmények között elfér az occupancy limiten belül, sanitizer alatt túl nagy regiszterhasználat miatt el sem indulhat, vagy `launch failure` hibát ad. Ilyenkor csökkenteni kell a blokkméretet, vagy a `-G` helyett csak `-lineinfo` opcióval kell fordítani. A sanitizerrel történő futtatás nem a teljesítmény mérésére szolgál; az eredmények csak a hibadetektálásra vonatkoznak.

A `racecheck` futásánál a versenyhelyzet lehet `warp-aggregated` is. Ez azt jelenti, hogy a verseny nem két konkrét szál között, hanem a warp szintjén történik, amikor a szálak egy csoportja ugyanazt a memóriacímet éri el. A racecheck ilyenkor `warp-aggregated race` üzenetet adhat,

ami szintén hibát jelez, de a pontos szálazonosító nem mindig adható meg. A `racecheck` csak akkor találja meg a versenyt, ha a futás során a konkurens hozzáférések valóban ütköznek; a `__syncthreads()` hiánya miatt a hiba reprodukálhatósága nem garantált.

Az initcheck eszköz csak a lokális memóriát és a regisztereket vizsgálja. A nem inicializált globális vagy shared memóriát nem jelzi, mert a globális memória a `cudaMalloc` után általában nem nulla, de a tartalma nem definiált; a `cudaMemset` hiánya nem számít initcheck hibának. Ha a program globális memóriát használ inicializálás nélkül, az a memcheck eszközzel sem mindig derül ki, mert a memória érvényes, csak a tartalma véletlenszerű. Ilyen esetben a determinisztikus inicializálást kell alkalmazni.

10.3.6 8.2.6. Összefoglalás és önellenőrző kérdés

A compute-sanitizer a CUDA fejlesztő egyik elsődleges eszköze a rejtett memóriahibák és versenyhelyzetek felderítésére. A `memcheck` a túlindexelést, a felszabadítás utáni hozzáférést és az érvénytelen pointereket találja meg; a `racecheck` a szinkronizálatlan konkurens hozzáféréseket; az `initcheck` a nem inicializált lokális változókat. A három eszköz külön futtatható, és a riportok együttesen adnak teljes képet a kernel memória-viselkedéséről.

A laborban bemutatott szándékos hibák jól szemléltetik az egyes eszközök kimeneteit. A túlindexelést a memcheck a hibás szál azonosítójával jelzi, a versenyhelyzetet a racecheck a szálpár és a hozzáférés típusának megadásával, a nem inicializált változót az initcheck a változó nevének kiírásával. A fejezet következő egysége a tipikus CUDA-hibák rendszerezett tárgyalásával és azok megelőzésével foglalkozik; az itt bemutatott eszközök a hibák azonosításának alapját képezik.

Önellenőrző kérdés: Adott egy kernel, amely időnként rossz eredményt ad, és a hibás futás során a Natív `cuda-memcheck`-kel (a compute-sanitizer memcheck) nem találunk érvénytelen memóriahozzáférést, de az eredmény nem determinisztikus. Melyik sanitizer eszköz futtatását érdemes megpróbálni, és milyen hibatípust várunk? Válaszát indokolja a versenyhelyzetek természetével.

Válasz vázlat: A `racecheck` eszközt, mert a nem deteriminizmus gyakran szinkronizálatlan írás/olvasás miatt van. A memcheck nem talál hibát, mert a hozzáférések érvényes memóriatartományon belül vannak, de a sorrend nincs szinkronban. A racecheck a versenyhelyzetet `read-write` vagy `write-write` hozzáférésként jelzi, és megadja a szálakat. A javítás atomi műveletekkel vagy `__syncthreads()`-el történik.

10.4 Tipikus CUDA-hibák és megelőzésük

Amikor egy CUDA-program hibát jelez, a hibaüzenet gyakran csak a felszín. A kernel aszinkron végrehajtása, a memóriaterek eltérő élettartama és a hardveres erőforrás-korlátok együttesen olyan hibamintákat hoznak létre, amelyek időnként determinisztikusnak tűnnek, máskor viszont csak adott foglalási sorrendben, órajel mellett vagy egy másik GPU-generáción reprodukálhatók. Ebben az alfejezetben ezért nem csupán egyes hibákat javítunk, hanem **hibaminta-katalógust** építünk: minden bejegyzéshez tartozik reprodukáló kód, várt eredmény, mért eredmény, magyarázat és megelőzési stratégia. A cél, hogy a tipikus hibákat azonnal felismerjük, és ne csak a tünetet, hanem a helyes megoldást is meg tudjuk indokolni.

A katalógus három nagy csoportra oszlik:

1. szinkronizációs és API-hibák,

2. memória-hozzáférési hibák,

3. erőforrás- és launch-hibák.

Ezek a csoportok nem tisztán szétválaszthatók, mert egy hibás kernel például egyszerre okozhat illegális memória-hozzáférést és később sticky hibát, de a katalógusban a szétválasztás megkönnyíti a rendszerezést.

10.4.1 1. Hibaosztályozás és detektálási stratégiák

A CUDA futásidő hibái két nagy csoportba sorolhatók aszerint, hogy mikor jelentkeznek.

- **Azonnali hibák:** például `cudaMalloc`, `cudaMemcpy`, `cudaDeviceGetAttribute` szinkron hívások hibái. Ezeket a hívás visszatérési értékéből azonnal látjuk.
- **Késleltetett hibák:** a kernelindítás és az aszinkron másolás hibái gyakran csak a szinkronizáláskor válnak láthatóvá, mert a CPU továbbfut, miközben a GPU még dolgozik.

A korábbi alfejezetben bevezetett hibamakrót itt is használjuk, de kiegészítjük egy `CHECK_LAUNCH` makróval, amely a kernelindítás utáni `cudaGetLastError` és a `cudaDeviceSynchronize` hibáját is ellenőrzi:

```
// error_util.cuh
#pragma once
#include <cuda_runtime.h>
#include <cstdio>
#include <cstdlib>

#define CHECK_CUDA(call)                                                    \
    do {                                                                    \
        cudaError_t err = (call);                                           \
        if (err != cudaSuccess) {                                           \
            std::fprintf(stderr, "CUDA error in %s at %s:%d: %s\n",         \
                         #call, __FILE__, __LINE__, cudaGetErrorString(err)); \
            std::exit(EXIT_FAILURE);                                        \
        }                                                                   \
    } while (0)

#define CHECK_LAUNCH(kernel_call)                                           \
    do {                                                                    \
        kernel_call;                                                        \
        CHECK_CUDA(cudaGetLastError());                                     \
        CHECK_CUDA(cudaDeviceSynchronize());                                \
    } while (0)
```

A `CHECK_LAUNCH` azért fontos, mert a `cudaGetLastError()` közvetlenül a `<<<...>>>` után sokszor már elkapja az érvénytelen launch-konfigurációt, de a kernel futása közben keletkező hibát – például illegális memória-hozzáférést – csak a `cudaDeviceSynchronize()` hozza felszínre.

Az 1. táblázat a leggyakoribb hibakategóriákat foglalja össze.

Kategória	Tipikus tünet	Elsődleges detektálás
API-hiba elmulasztott ellenőrzése	a program tovább fut, majd később érthetetlen hibát ad	`CHECK_CUDA` minden híváson

Kategória	Tipikus tünet	Elsődleges detektálás
Nem szinkronizált hozzáférés	időnként nulla, lefutott adat vagy szemét	`cudaDeviceSynchronize`, `cudaStreamSynchronize`, event
Érvénytelen mutató, túlindexelés	`cudaErrorIllegalAddress`, sticky hiba	`compute-sanitizer --tool memcheck`
Érvénytelen launch-konfiguráció	`cudaErrorInvalidConfiguration`	`cudaGetLastError` launch után
Erőforrás-kimerülés	`cudaErrorMemoryAllocation`, `cudaErrorLaunchOutOfResources`	`cudaMemGetInfo`, device attribútumok
Versenyhelyzet, hiányzó barrier	nem determinisztikus eredmény, ritkán hibázik	`compute-sanitizer --tool racecheck`
Sticky hiba, device lost	minden további CUDA-hívás hibát ad	`cudaDeviceReset`, izolált folyamat

A táblázatban szereplő hibakódok neve a mérvadó. A számértékek a CUDA Toolkit verziójától és a drivertől függenek, ezért a katalógusban mindig a `cudaGetErrorString` által visszaadott nevet rögzítsük, ne a numerikus kódot.

10.4.2 2. Nem szinkronizált másolás és törött launch

10.4.3 2.1. Nem szinkronizált másolás

Az egyik leggyakoribb hiba, hogy a program `cudaMemcpyAsync` után azonnal olvassa a host oldali puffert. A `cudaMemcpyAsync` a default streamen vagy egy megadott streamen aszinkron módon másolhat, ezért a CPU a másolás befejezése előtt hozzáférhet a célterülethez. A hiba különösen akkor szembetűnő, ha a host memória **pinned** (page-locked), mert ilyenkor a másolás valóban aszinkron lehet. Ha a host memória pageable, a CUDA-meghajtó gyakran szinkron másolásra esik vissza, így a hiba véletlenül eltűnhet – ami még veszélyesebb, mert a kód egy másik környezetben hibázik.

Az alábbi példa szándékosan reprodukálja a hibát:

```
// E01_unsync_memcpy.cu
#include <cuda_runtime.h>
#include <cstdio>
#include "error_util.cuh"

__global__ void fill(float *dst, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) dst[i] = 3.1415f;
}

int main() {
    const int n = 1 << 20;
    float *d_a = nullptr;
    float *h_a = nullptr;

    CHECK_CUDA(cudaMalloc(&d_a, n * sizeof(float)));
    CHECK_CUDA(cudaMallocHost(&h_a, n * sizeof(float))); // pinned

    fill<<<(n + 255) / 256, 256>>>(d_a, n);
    CHECK_CUDA(cudaGetLastError());
```

```
    // Hibás: a másolás aszinkron, még nem biztos, hogy kész.
    CHECK_CUDA(cudaMemcpyAsync(h_a, d_a, n * sizeof(float),
                               cudaMemcpyDeviceToHost, 0));
    std::printf("h_a[0] szinkronizálás előtt: %f\n", h_a[0]); // verseny

    CHECK_CUDA(cudaDeviceSynchronize());
    std::printf("h_a[0] szinkronizálás után:  %f\n", h_a[0]);

    CHECK_CUDA(cudaFree(d_a));
    CHECK_CUDA(cudaFreeHost(h_a));
    return 0;
}
```

A várt eredmény az, hogy a `h_a[0] szinkronizálás előtt` értéke nem definiált; leggyakrabban a korábbi puffertartalom vagy nulla jelenik meg. A szinkronizálás utáni érték `3.141500`. A mért eredményt a laborban rögzíteni kell; a lényeg, hogy az első kiírás nem garantált, tehát hibának számít. A helyes megoldás a másolás után a megfelelő streamet szinkronizálni:

```
CHECK_CUDA(cudaMemcpyAsync(h_a, d_a, n * sizeof(float),
                           cudaMemcpyDeviceToHost, 0));
CHECK_CUDA(cudaDeviceSynchronize()); // vagy cudaStreamSynchronize(stream)
```

Ha a másolás nem a default streamen fut, akkor `cudaStreamSynchronize(stream)` vagy egy CUDA event használandó. Ha a másolásnak mindenképpen szinkronnak kell lennie, akkor a `cudaMemcpy` a megfelelő választás. A `cudaMemcpyAsync`-hoz lehetőleg pinned memóriát használjunk, mert pageable memória esetén az aszinkronitás nem garantált, és a teljesítmény is rosszabb.

Ugyanez a minta más programozási modellekben is megjelenik. OpenCL-ben a `clEnqueueReadBuffer` után a host oldali olvasás előtt `clFinish` vagy wait event szükséges. JAX-ban az eszközön futó műveletek aszinkron dispatch-t használnak, ezért a NumPy tömbbé alakítás előtt `block_until_ready()` hívandó, ha azonnali host oldali értékre van szükség.

10.4.4 2.2. Törött launch

Törött launch alatt azt értjük, amikor a kernelindítás paraméterei érvénytelenek, vagy a kernel binárisa nem futtatható az adott eszközön. Ilyenkor a kernel el sem indul, de a hiba csak a `cudaGetLastError` vagy a szinkronizálás során látszik.

Klasszikus példa a blokkméret túllépése. A CUDA eszközökön a blokkonkénti szálak maximális száma jellemzően 1024, de ezt mindig a `cudaDevAttrMaxThreadsPerBlock` attribútumból kérdezzük le. A következő kód szándékosan 1025 szálat indít:

```
// E02_invalid_launch.cu
#include <cuda_runtime.h>
#include <cstdio>
#include "error_util.cuh"

__global__ void dummy(float *out) {
    if (threadIdx.x == 0) *out = 1.0f;
}

int main() {
    float *d_out = nullptr;
    CHECK_CUDA(cudaMalloc(&d_out, sizeof(float)));

    // blockDim.x = 1025 meghaladja a maximumot (általában 1024)
    dummy<<<1, 1025>>>(d_out);
    cudaError_t err = cudaGetLastError();
```

```
    if (err != cudaSuccess) {
        std::fprintf(stderr, "Launch hiba: %s\n", cudaGetErrorString(err));
    }
    CHECK_CUDA(cudaDeviceSynchronize());
    CHECK_CUDA(cudaFree(d_out));
    return 0;
}
```

A várt hiba `cudaErrorInvalidConfiguration`, a szöveges üzenet pedig általában `invalid configuration argument`. A `cudaGetLastError` közvetlenül a launch után már jelezheti a hibát, mert a launch-konfigurációt a vezérlő még a kernel indítása előtt ellenőrzi. Fontos azonban, hogy a `cudaDeviceSynchronize` hibáját is ellenőrizzük, mert egy későbbi aszinkron hiba itt is megjelenhet.

Egy másik törött launch akkor keletkezik, ha a kért shared memory mérete nagyobb, mint az eszköz által támogatott maximum:

```
// E04_shared_mem_over.cu
#include <cuda_runtime.h>
#include <cstdio>
#include "error_util.cuh"

__global__ void sm_kernel() {
    extern __shared__ float smem[];
    smem[threadIdx.x] = threadIdx.x;
}

int main() {
    int max_smem = 0;
    CHECK_CUDA(cudaDeviceGetAttribute(&max_smem,
        cudaDevAttrMaxSharedMemoryPerBlock, 0));
    std::printf("Max shared mem per block: %d byte\n", max_smem);

    // a kért shared memória eggyel nagyobb a megengedettnél
    sm_kernel<<<1, 32, (size_t)max_smem + 1>>>();
    cudaError_t err = cudaGetLastError();
    if (err != cudaSuccess) {
        std::fprintf(stderr, "Launch hiba: %s\n", cudaGetErrorString(err));
    }
    CHECK_CUDA(cudaDeviceSynchronize());
    return 0;
}
```

A `cudaGetLastError` itt szintén `cudaErrorInvalidConfiguration` vagy `cudaErrorLaunchOutOfResources` hibát adhat a környezettől függően. A pontos mért szöveget a labor naplójában rögzítsük.

A törött launch harmadik gyakori oka az architektúra-eltérés. Ha a kernelt csak `sm_80` binárissal fordítjuk, de a futtató eszköz `sm_70` vagy `sm_89`, akkor a `cudaErrorNoKernelImageForDevice` hibát kaphatjuk, amennyiben nincs kompatibilis PTX vagy megfelelő bináris. Ennek elkerülésére a fordításkor érdemes `-gencode` kapcsolóval több architektúrát is beágyazni, vagy `-arch=native` kapcsolót használni a fejlesztői gépen. A hordozható binárisokkal kapcsolatos részletek a CUDA ökoszisztéma és a hardvergenerációk és A CUDA toolkit és a driver telepítése alfejezetekben szerepelnek.

10.4.5 3. Erőforrás-kimerülés

Az erőforrás-kimerülés a hibák azon családja, amikor a program logikailag helyes, de az eszköz fizikai korlátjaiba ütközik. A legfontosabb erőforrások: globális memória, blokkonkénti shared memory, blokkonkénti regiszterek, blokkméret és gridméret.

10.4.6 3.1. Globális memória kimerülése

A `cudaMalloc` sikertelen lesz, ha nincs elegendő szabad eszközmemória. Ezt a hibát mindig kezelni kell, mert nagy pufferű tudományos kódokban a felhasználó gyakran szeretné a lehető legnagyobb adatot betölteni. Az alábbi kód lekérdezi a szabad memóriát, majd szándékosan nagyobb foglalást kísérel meg:

```
// E06_mem_exhaust.cu
#include <cuda_runtime.h>
#include <cstdio>
#include "error_util.cuh"

int main() {
    size_t free_mem = 0, total_mem = 0;
    CHECK_CUDA(cudaMemGetInfo(&free_mem, &total_mem));
    std::printf("Szabad: %.2f MiB / Összes: %.2f MiB\n",
                free_mem / 1048576.0, total_mem / 1048576.0);

    void *ptr = nullptr;
    cudaError_t err = cudaMalloc(&ptr, free_mem + (1ULL << 30)); // biztos túl nagy
    if (err != cudaSuccess) {
        std::printf("cudaMalloc sikertelen: %s\n", cudaGetErrorString(err));
    }
    return 0;
}
```

A várt hiba `cudaErrorMemoryAllocation`. A mért üzenet a laborban rögzítendő. Fontos, hogy a `cudaMemGetInfo` által jelentett szabad memória nem garancia arra, hogy egy azonos méretű foglalás sikeres lesz, mert a memória töredezett lehet, és a CUDA-kontextus, a futó kernelek vagy más folyamatok is foglalhatnak. Nagy allokációk előtt érdemes a tényleges foglalást kísérletként kezelni, és hiba esetén csökkenteni a méretet vagy más memóriastratégiát választani.

A kimerülés megelőzhető a munkakészlet csökkentésével, streaming feldolgozással, unified memory használatával vagy több GPU közötti elosztással. A hiba kezelésének soha nem szabad az lennie, hogy figyelmen kívül hagyjuk a `cudaMalloc` visszatérési értékét, mert a null mutatóval való későbbi kernelindítás illegális hozzáférést okoz.

10.4.7 3.2. Launch erőforrások kimerülése

A kernelindítás akkor is meghiúsulhat, ha a globális memória elegendő. A leggyakoribb launch erőforrás-korlátok:

- maximális szálak száma blokkonként (`cudaDevAttrMaxThreadsPerBlock`),
- maximális shared memory blokkonként (`cudaDevAttrMaxSharedMemoryPerBlock`),
- maximális regiszterek száma blokkonként (`cudaDevAttrMaxRegistersPerBlock`),
- maximális grid dimenziók (`cudaDevAttrMaxGridDim`).

A shared memory túllépését már láttuk. A regiszternyomás akkor okoz launch hibát, ha a kernel regiszterigénye és a blokkméret szorzata meghaladja a blokkonkénti regiszterlimitet. Például ha egy kernel 128 regisztert használ szálanként, és 1024 szálból álló blokkokat indítunk, akkor az elméleti igény $128 \times 1024 = 131\,072$ regiszter, ami meghaladhatja a `cudaDevAttrMaxRegistersPerBlock` értékét (általában 65 536). Ilyenkor a launch `cudaErrorLaunchOutOfResources` hibát adhat.

A megoldás három irányból közelíthető:

- csökkentsük a blokkméretet,
- csökkentsük a regiszterhasználatot `__launch_bounds__` vagy `-maxrregcount` segítségével,
- egyszerűsítsük a kernel belső kifejezéseit, hogy a fordító kevesebb regisztert használjon.

A regiszterhasználat pontos mérésére az Nsight Compute és a `--ptxas-options=-v` fordítási kimenet alkalmas, de ez már a Regiszterhasználat és spill alfejezet részletes anyaga.

10.4.8 3.3. Sticky hibák és device lost

Az illegális memória-hozzáférés – például egy global memóriában történő túlindexelés – nem egyszerű kivétel, hanem gyakran úgynevezett **sticky hiba**. A CUDA-kontextus ilyenkor megromolhat, és minden további CUDA-hívás `cudaErrorIllegalAddress` vagy `cudaErrorUnknown` hibát adhat, amíg a kontextust újra nem inicializáljuk.

Az alábbi kód szándékos túlindexelést hajt végre:

```
// E03_oob_write.cu
#include <cuda_runtime.h>
#include <cstdio>
#include "error_util.cuh"

__global__ void oob_write(int *buf, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx == n) buf[idx] = 1; // n index nem létezik
}

int main() {
    const int n = 128;
    int *d_buf = nullptr;
    CHECK_CUDA(cudaMalloc(&d_buf, n * sizeof(int)));

    oob_write<<<1, 129>>>(d_buf, n); // 129 szál, idx 128 túlír
    cudaError_t err = cudaDeviceSynchronize();
    if (err != cudaSuccess) {
        std::fprintf(stderr, "Szinkron hibája: %s\n", cudaGetErrorString(err));
    }
    // Sticky hiba esetén itt a további hívások is hibázhatnak.
    cudaError_t err2 = cudaFree(d_buf);
    if (err2 != cudaSuccess) {
        std::fprintf(stderr, "cudaFree hibája: %s\n", cudaGetErrorString(err2));
    }
    return 0;
}
```

A `cudaDeviceSynchronize` várt hibája `cudaErrorIllegalAddress`. Ezt követően a `cudaFree` is hibázhat, mert a kontextus megromlott. A legbiztonságosabb ilyenkor a folyamat befejezése vagy a `cudaDeviceReset()` hívása, de a `cudaDeviceReset` hatása az egész eszközkontextusra kiterjed, ezért éles kódban ritkán megfelelő. A hibás kernelt lehetőleg izolált folyamatban futtassuk, ahol a kontextus elvesztése nem érint más kódot.

A compute-sanitizer `memcheck` eszköze pontosan az ilyen hibákat képes még a sticky állapot kialakulása előtt jelenteni. A parancs:

```
compute-sanitizer --tool memcheck ./E03_oob_write
```

A várt kimenetben szerepelnie kell egy `Invalid global write of size 4` típusú sornak. A mért kimenetet a laborban rögzítsük. A sanitizer használata különösen fontos akkor, ha a kernel látszólag helyesen fut, de néha véletlenül túlír.

10.4.9 4. Labor: hibaminta-katalógus készítése

A labor célja, hogy a fenti hibamintákat saját környezetben reprodukáljuk, és egy jól dokumentált katalógust készítsünk. A katalógus nem egyszerűen hibás kódok gyűjteménye, hanem minden bejegyzésnél elkülöníti a **mért eredményt** és a **várt eredményt**.

10.4.10 4.1. Labor-környezet

A példa-konfiguráció, amelyre a kódok készültek:

- GPU: NVIDIA GeForce RTX 4070 (compute capability 8.9)
- Driver: 550.54.14
- CUDA Toolkit: 12.4
- Operációs rendszer: Ubuntu 22.04 LTS
- Fordító: `nvcc` V12.4.131

A tényleges mérési naplóban a saját környezet pontos értékeit kell rögzíteni: GPU, compute capability, driver verzió, CUDA verzió, operációs rendszer, `nvcc --version` kimenet.

10.4.11 4.2. A katalógus szerkezete

A laborban a következő könyvtárat hozzuk létre:

```
gpu037_lab/
  error_util.cuh
  E01_unsync_memcpy/
     main.cu
     build.sh
     README.md
  E02_invalid_launch_config/
     main.cu
     build.sh
     README.md
  E03_oob_write/
     main.cu
     build.sh
     README.md
  E04_shared_mem_over/
     main.cu
     build.sh
     README.md
  E05_shared_race/
     main.cu
     build.sh
     README.md
  E06_mem_exhaust/
     main.cu
```

```
      build.sh
      README.md
  lab_log.md
```

Minden `build.sh` például a következőt tartalmazza:

```
#!/usr/bin/env bash
set -e
nvcc -std=c++17 -O2 -lineinfo -arch=sm_89 -I.. -o main ../main.cu
```

A `lab_log.md` fájlban minden bejegyzéshez egy táblázatsor tartozik:

Minta	Parancs	Mért eredmény	Várt eredmény	Megjegyzés
E01	`./main`	első kiírás: `0.000000`, második: `3.141500`	első nem definiált, második `3.141500`	pinned host memória esetén reprodukálható
E02	`./main`	`CUDA error in cudaDeviceSynchronize: invalid configuration argument`	`cudaErrorInvalidConfiguration`	`cudaGetLastError` már launch után jelzi
E03	`compute-sanitizer --tool memcheck ./main`	`Invalid global write of size 4`	ugyanez	a sima futtatás `cudaErrorIllegalAddress`-t ad
E04	`./main`	`CUDA error in cudaDeviceSynchronize: invalid configuration argument`	`cudaErrorInvalidConfiguration` vagy `cudaErrorLaunchOutOfResources`	konfiguráció shared memory lefoglalásán egy bájttal nagyobbat kérünk
E05	`compute-sanitizer --tool racecheck ./main`	`possible write-read race in shared memory`	ugyanez	a barrier hiánya miatt
E06	`./main`	`cudaMalloc` sikertelen: `out of memory`	`cudaErrorMemoryAllocation`	szabad memóriánál nagyobb foglalás

A `lab_log.md`-nek tartalmaznia kell a `nvcc --version`, a `nvidia-smi` és a `compute-sanitizer --version` kimenetét is. Így a katalógus reprodukálható marad.

10.4.12 4.3. Kiegészítő minta: versenyhelyzet megfogása

A shared memory versenyhelyzet többnyire nem determinisztikus, ezért a `racecheck` eszköz nélkül nehéz észrevenni. A következő minta a 2.2. alfejezetből hiányzó barrier hibát mutatja:

```
// E05_shared_race.cu
#include <cuda_runtime.h>
#include <cstdio>
#include "error_util.cuh"

__global__ void race_kernel(float *out) {
    __shared__ float s_val;
    if (threadIdx.x == 0) s_val = 1.0f;
    // Hibás: nincs __syncthreads()
    out[0] = s_val; // más szálak talán nem látják az írást
}

int main() {
    float *d_out;
    CHECK_CUDA(cudaMalloc(&d_out, sizeof(float)));
    race_kernel<<<1, 32>>>(d_out);
    CHECK_CUDA(cudaDeviceSynchronize());
    CHECK_CUDA(cudaFree(d_out));
    return 0;
}
```

A `compute-sanitizer --tool racecheck ./main` parancs `possible write-read race` üzenetet ad. A helyes megoldás a `__syncthreads()` hívása a shared memóriába írás után, még a kiolvasás előtt:

```
__global__ void race_kernel(float *out) {
    __shared__ float s_val;
    if (threadIdx.x == 0) s_val = 1.0f;
    __syncthreads(); // minden szál látja az írást
    out[0] = s_val;
}
```

Fontos, hogy a `__syncthreads()` a blokk minden szálának ugyanazt a végrehajtási ágat kell, hogy jelentse. Divergens ágban a barrier használata a legtöbb CUDA-modellben érvénytelen, és a `racecheck` vagy a futás közbeni hiba szintén jelezheti.

10.4.13 4.4. Értékelési szempontok

A labor akkor teljesített, ha:

- legalább hat hibamintát tartalmaz a katalógus,
- minden mintához tartozik fordítható CUDA C++ forrás,
- a build parancs és a futtatási parancs rögzítve van,
- minden API-hívás hibája ellenőrzött,
- a mért és a várt eredmény külön oszlopban szerepel,
- a compute-sanitizer kimenetek a `memcheck` és a `racecheck` eszközzel is dokumentáltak,
- a katalógus tartalmazza a környezeti adatokat: GPU, driver, CUDA, OS, nvcc verzió.

Ez a katalógus a későbbi fejezetek hibakeresési munkájához is alapot ad, mert a Globális memória coalescing alfejezettől kezdődő teljesítménymméréseknél gyakran először a helyességi hibákat kell kizárni.

10.4.14 5. Összefoglalás és önellenőrző kérdés

A leggyakoribb CUDA-hibák nem a nyelv bonyolultságából, hanem az aszinkron végrehajtásból, a memóriaterek eltérő szemantikájából és az eszköz erőforrás-korlátaiból adódnak. A hibamintakatalógus rendszeres vezetése segít abban, hogy a hibákat ne egyedi esetként kezeljük, hanem felismerjük a mögöttes mintát.

Önellenőrző kérdés: Adott egy program, amely `cudaMemcpyAsync` hívással másol device-ról hostra, majd közvetlenül a másolás utáni sorban kiírja a host puffer első elemét. Miért helytelen ez a minta? Nevezze meg a két leggyakoribb javítást, és magyarázza el, miért fontos a pinned host memória használata az aszinkron másoláshoz.

A válaszban szerepelnie kell annak, hogy a `cudaMemcpyAsync` aszinkron lehet, ezért az olvasás versenyhelyzetet okoz. A javítás `cudaDeviceSynchronize` vagy `cudaStreamSynchronize` a másolás után, illetve szinkron `cudaMemcpy` használata, ha a szinkronitás követelmény. A pinned memória azért fontos, mert pageable memória esetén az aszinkron másolás nem garantált, és a teljesítmény is romlik.

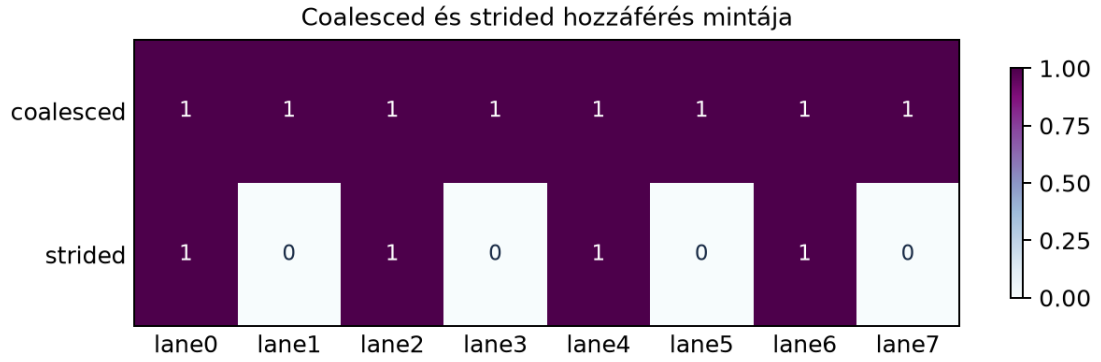
Ezzel a katalógussal a következő alfejezet (Globális memória coalescing) már arra építhet, hogy a globális memória-hozzáférések teljesítményét elemezze anélkül, hogy helyességi hibák zavarnák a mérést.

III. RÉSZ – Teljesítmény és memória

11 Coalescing, bank conflicts és vectorizált hozzáférés

11.1 Globális memória coalescing

A globális memória a GPU legnagyobb késleltetésű, de legnagyobb kapacitású memóriatere. A benne tárolt adatok elérésekor a hardver nem minden thread kérését szolgálja ki egyenként, hanem a warp egészének memóriahozzáféréseit igyekszik összevonni. Ezt az összevonást nevezzük **globális memória coalescing**nek. A coalescing minősége közvetlenül meghatározza, hogy a kód milyen közel tud kerülni az elméleti memória-sávszélességhez. Ezért ez a fogalom az egyik legfontosabb teljesítménytényező a CUDA-programozásban.



11.1. ábra: Coalesced és strided hozzáférés mintája - A zöld cellák egy warp egy tranzakcióba eső, szomszédos címeit jelölik; a strided minta több külön tranzakciót indít, ezért romlik a sávszélesség-kihasználás.

11.1.1 A warp egyidejű hozzáférése és a tranzakciók

Egy GPU-ban a végrehajtás alapegysége a warp: 32 szál egyszerre hajtja végre ugyanazt az utasítást, bár különböző adatokon (SIMT, Single Instruction Multiple Threads). Amikor egy warp memóriautasítást hajt végre – például globális memóriából olvas –, a hardver a 32 szál által kért címeket együtt vizsgálja. Ha ezek a címek bizonyos feltételeknek megfelelnek, a hardver **összevonja** a kéréseket: a 32 egyedi hozzáférés helyett néhány széles memóriatranzakciót indít. Ez a coalescing.

A memóriatranzakció fogalma kulcsfontosságú. A global memory a DRAM-ból és a hozzá tartozó gyorsítótár-hierarchiából (L2, L1) áll. A hardver nem bájtonként, hanem **szektorokban** és **tranzakciókban** mozgatja az adatokat. Egy szektor általában 32 bájt. Az L2 cache vonala 128 bájt, tehát négy darab 32 bájtos szektorból áll. Amikor egy warp memóriát olvas, a hardver annyi szektort kér le, amennyi az összes kért cím lefedéséhez szükséges. A kérés akkor optimális, ha minden lekérdezett 32 bájtos szektorból mind a 32 bájtot ténylegesen felhasználja legalább egy szál.

Például, ha egy warp 32 szálának mindegyike egy-egy `float` (4 bájt) értéket olvas, és a címek szorosan egymás után következnek, akkor a teljes kérés 128 bájt, azaz pontosan egy cache line. Ideális esetben ezt a hardver egyetlen 128 bájtos tranzakcióval (vagy néhány kisebb, de teljesen kihasznált tranzakcióval) kiszolgálja. A gyakorlatban a hardver a kérést **szektorokra** bontja, és minden szektort külön kér le. Ha minden szektorból minden bájtot használunk, akkor a tranzakciók száma minimális, a sávszélesség maximálisan kihasznált.

Ha azonban a szálak címei szétszórtak, például minden szál minden nyolcadik elemet olvassa, akkor a 32 kérés 32 különböző szektort érinthet. A hardver 32 szektort kénytelen lekérni, de mindegyikből csak 4 bájtot használ fel. Ez 32×32 bájt = 1024 bájt tényleges adatforgalom, miközben a hasznos adat csak 128 bájt. A hatékonyság 12,5%, a többi 87,5% elvész. Ez a **nem coalescing** hozzáférés tipikus következménye.

11.1.2 32, 64 és 128 bájtos tranzakciók

A memóriarendszer különböző méretű tranzakciókat hajthat végre. A szektor mérete 32 bájt, de a hardver a kérések összevonásakor gyakran nagyobb egységekben mozgatja az adatokat. A modern NVIDIA GPU-k a globális memória-hozzáféréseket jellemzően 32, 64 vagy 128 bájtos tranzakciókra bontják, attól függően, hogy a szálak milyen címtartományt fednek le.

- **32 bájtos tranzakció (egy szektor):** Ha a warp szálai csak egyetlen 32 bájtos szektoron belül kérnek adatot, a hardver egy szektoros tranzakciót indít. Például, ha 32 szál mindegyike ugyanazt a 4 bájtos értéket olvassa (broadcast), akkor egyetlen szektort kér le, és minden szál megkapja ugyanazt az adatot.
- **64 bájtos tranzakció (két szektor):** Ha a kért adat éppen két szektort fed le, a hardver két szektoros tranzakciót hajthat végre.
- **128 bájtos tranzakció (négy szektor, egy cache line):** Ha a szálak 32 egymás utáni `float`-ot kérnek, az pontosan egy 128 bájtos cache line-t fed le. Ideális esetben a hardver egyetlen 128 bájtos tranzakcióval kiszolgálja.

A tranzakciók mérete nem merev szabály; a hardver a kérések alapján dönt. A lényeg: minél kevesebb tranzakció kell, és minél nagyobb ezek kihasználtsága, annál jobb a coalescing. A hatékonyságot a felhasznált hasznos adat és a ténylegesen átvitt adat arányával mérhetjük. Ideális esetben ez 100%.

11.1.3 Alignment és szektor

A coalescing egyik kritikus feltétele a **memóriacímek igazítása (alignment)**. A hardver akkor tudja a leghatékonyabban összevonni a kéréseket, ha a warp által kért memóriaterület kezdőcíme a tranzakció méretéhez igazodik. Például egy 128 bájtos olvasás akkor ideális, ha a kezdőcím 128-cal osztható. Ha a kezdőcím nem igazított, a kérés átnyúlik a cache line határon, és a hardver több tranzakciót kénytelen indítani.

Tekintsünk egy példát: egy warp 32 szálból áll, mindegyik egy `float`-ot olvas. Ha a kezdőcím 128-cal osztható, akkor a 32 float pontosan egy cache line-t fed le, és egyetlen 128 bájtos tranzakció kiszolgálja. Ha a kezdőcím viszont például 4-gyel osztható, de 128-cal nem, akkor a 128 bájtos kérés két cache line határát érinti. A hardver két 128 bájtos tranzakciót (vagy egy 128 bájtosat és egy 32 bájtosat) indít, és a kért adat csak a lekérdezett adat egy részét foglalja el. Ez rontja a hatékonyságot.

A gyakorlatban az alignment akadályozhatja a tökéletes coalescinget. A CUDA memóriafoglaló (`cudaMalloc`) által visszaadott pointerek általában 256 bájtra igazítottak, így a kernelindításkor

gyakran jó kiindulópontot kapunk. Ha azonban a tömbön belül eltolással dolgozunk, például `&array[offset]` címet adunk át, akkor az alignment romolhat. Ezért fontos a tömbindexelésnél az igazítást figyelembe venni.

A szektorok szintjén is van alignment: egy 32 bájtos szektor akkor használható ki teljesen, ha a kért adat a szektor egészét lefedi. Ha egy szektorból csak 4 bájtot kérünk, a szektor többi 28 bájtja feleslegesen kerül átvitelre. Ezért a coalescing akkor optimális, ha a szálak címei folytonosak, és a kezdőcím 32 bájtra (vagy nagyobb tranzakció esetén nagyobb méretre) igazodik.

11.1.4 Stride hozzáférés és a coalescing romlása

A stride-olt hozzáférés a leggyakoribb példa a nem coalescing hozzáférésre. Tegyük fel, hogy a szálak egy tömb elemeit olvassák, de nem egymás után, hanem minden szál `stride` távolságra lévő elemet. A kernelben az indexelés például így nézhet ki:

```
int idx = threadIdx.x + blockIdx.x * blockDim.x;
float value = input[idx * stride];
```

Ha `stride = 1`, a szomszédos szálak szomszédos `float`-okat olvasnak – ez a tökéletes coalescing esete. Ha `stride = 2`, akkor minden második elemet olvassák, így 32 szál 64 `float`-nyi területet fed le, ami 256 bájt. Ez két 128 bájtos cache line-t érint, de minden cache line-ból csak minden második 4 bájtos szegmenst használunk. A hatékonyság 50%-ra csökken. `stride = 8` esetén minden szál 32 bájtos távolságra van, így minden szál külön szektort érint. A 32 szál 32 szektort kér le, de mindegyikből csak 4 bájtot használ. A hatékonyság 12,5%, ahogy korábban láttuk.

Minél nagyobb a stride, annál szétszórtabbak a címek, és annál több tranzakció kell. A teljesítményromlás drámai: a memória-sávszélesség kihasználtsága a stride reciprokával arányosan csökken, egészen addig, amíg minden szál külön tranzakciót kényszerít ki.

Érdemes megjegyezni, hogy a modern GPU-k némi cache-el rendelkeznek, ezért ha ugyanazt a stride-olt hozzáférést ismételten végrehajtjuk, a L2 cache enyhítheti a veszteséget. A laborban ezt mérni fogjuk.

11.1.5 A coalescing mérése és a labor

A coalescing hatását legegyszerűbben egy memóriaművelet sávszélességének mérésével lehet bemutatni. A labor célja, hogy megmérjük egy egyszerű kernel – `float` tömb másolása – effektív sávszélességét különböző stride értékek mellett. A várt eredmény az, hogy `stride = 1` esetén a sávszélesség közel van a GPU elméleti maximumához, míg a stride növekedésével a sávszélesség drámaian csökken.

11.1.6 Labor: coalesced és stride-olt hozzáférés mérése

Környezet rögzítése (a laborjelentésben kötelező kitölteni): - GPU modell: _____ (például: NVIDIA RTX 3080) - Compute capability: _____ (például: 8.6) - Driver verzió: _____ (például: 570.86) - CUDA toolkit verzió: _____ (például: 12.8) - Operációs rendszer: _____ (például: Ubuntu 24.04 LTS) - nvidia-smi által mutatott power limit: _____

Kód (CUDA C++17, `coalescing.cu`):

```
#include <cstdio>
#include <cstdlib>
#include <cuda_runtime.h>
#include <vector>

// Hibaellenőrző makró
#define CUDA_CHECK(call) do {                                   \
    cudaError_t err = (call);                                   \
    if (err != cudaSuccess) {                                   \
        fprintf(stderr, "CUDA hiba: %s fajl, %d sor: %s\n",     \
                __FILE__, __LINE__, cudaGetErrorString(err));   \
        exit(EXIT_FAILURE);                                     \
    }                                                           \
} while (0)

// Kernel: minden thread a stride*idx indexű elemet másolja
__global__ void copyKernelStride(const float* __restrict__ src,
                                 float* __restrict__ dst,
                                 int n, int stride) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int access_idx = idx * stride;
    if (access_idx < n) {
        dst[access_idx] = src[access_idx];
    }
}

// Kernel: normál coalescing (stride = 1)
__global__ void copyKernelCoalesced(const float* __restrict__ src,
                                    float* __restrict__ dst,
                                    int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        dst[idx] = src[idx];
    }
}

int main(int argc, char** argv) {
    if (argc != 2) {
        fprintf(stderr, "Hasznalat: %s <stride>\n", argv[0]);
        return EXIT_FAILURE;
    }
    int stride = atoi(argv[1]);
    const int n = 1 << 23; // 8M float = 32 MB tömbönként
    const size_t bytes = n * sizeof(float);

    float *d_src, *d_dst;
    CUDA_CHECK(cudaMalloc(&d_src, bytes));
    CUDA_CHECK(cudaMalloc(&d_dst, bytes));

    // Inicializálás host oldalon egyszerűen, aztán feltöltés device-ra
    std::vector<float> h_src(n, 1.0f);
    CUDA_CHECK(cudaMemcpy(d_src, h_src.data(), bytes, cudaMemcpyHostToDevice));

    cudaEvent_t start, stop;
    CUDA_CHECK(cudaEventCreate(&start));
    CUDA_CHECK(cudaEventCreate(&stop));

    // Grid/block konfiguráció
    int blockSize = 256;
    int gridSize = (n + blockSize - 1) / blockSize;
    if (stride > 1) {
        // A stride-olt kernel esetén a ténylegesen elért elemek száma: ceil(n / stride)
        gridSize = ((n + stride - 1) / stride + blockSize - 1) / blockSize;
```

```
    }

    // Warm-up: 3 ismétlés, hogy a cache-ek és az órajel stabilizálódjanak
    for (int i = 0; i < 3; ++i) {
        if (stride == 1) {
            copyKernelCoalesced<<<gridSize, blockSize>>>(d_src, d_dst, n);
        } else {
            copyKernelStride<<<gridSize, blockSize>>>(d_src, d_dst, n, stride);
        }
    }
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaGetLastError());

    // Mérés: 10 ismétlés
    const int iterations = 10;
    CUDA_CHECK(cudaEventRecord(start));
    for (int i = 0; i < iterations; ++i) {
        if (stride == 1) {
            copyKernelCoalesced<<<gridSize, blockSize>>>(d_src, d_dst, n);
        } else {
            copyKernelStride<<<gridSize, blockSize>>>(d_src, d_dst, n, stride);
        }
    }
    CUDA_CHECK(cudaEventRecord(stop));
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaGetLastError());

    float elapsed_ms = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&elapsed_ms, start, stop));
    double elapsed_sec = elapsed_ms / 1000.0;
    double total_bytes = 2.0 * bytes * iterations; // olvasás + írás, összes iteráció
    double bandwidth = total_bytes / elapsed_sec;    // bájt/másodperc

    printf("Stride=%2d | Atlagos ido (ms/iter): %.6f | Effektiv savszelesseg: %.2f GB/s\n",
           stride, elapsed_ms / iterations, bandwidth / 1e9);

    CUDA_CHECK(cudaFree(d_src));
    CUDA_CHECK(cudaFree(d_dst));
    CUDA_CHECK(cudaEventDestroy(start));
    CUDA_CHECK(cudaEventDestroy(stop));
    return 0;
}
```

Buildparancs (Linux, NVIDIA CUDA compiler):

```
nvcc -O3 -arch=sm_86 -o coalescing coalescing.cu
```

A `sm_86` a compute capability 8.6-hoz tartozik. A saját GPU-nak megfelelő értéket kell használni, például `sm_80` (A100), `sm_89` (RTX 40xx) stb.

Futtatási parancs:

```
./coalescing 1
./coalescing 2
./coalescing 4
./coalescing 8
./coalescing 16
./coalescing 32
```

11.1.7 Várt eredmény

A várt eredmény a memória-sávszélesség csökkenése a stride növekedésével. `stride = 1` esetén a kód tökéletes coalescinget ér el, ezért a mért sávszélesség közel van a GPU elméleti maximumához. A tömb mérete 32 MB, így a másolás során a teljes adatforgalom 64 MB iterációnként, ami belefér az L2 cache-be, ezért a mérés nem teljesen a DRAM-sávszélességet mutatja, hanem az L2 cache-sávszélességet. Ez is hasznos, mert a coalescing elsősorban a tranzakciók számát befolyásolja. A `stride` növelésével a tranzakciók száma nő, és a sávszélesség csökken. A várt trend: `stride = 1`-nél maximum, `stride = 2`-nél kb. fele, `stride = 4`-nél kb. negyede, `stride = 8`-nál kb. 12,5% körüli érték, és így tovább. A pontos értékek a GPU-tól és a cache viselkedésétől függnek, de a monoton csökkenés egyértelmű.

11.1.8 Mért eredmény (labor során kitöltendő)

A labor elvégzése után a hallgatónak rögzítenie kell a kimenetet, például táblázatban:

Stride	Átlagos idő (ms/iter)	Effektív sávszélesség (GB/s)
1		
2		
4		
8		
16		
32		

A mért értékeket össze kell vetni a várt trenddel. Ha nagy eltérés mutatkozik, érdemes ellenőrizni a kernel indexelését, a block- és gridméreteket, valamint azt, hogy a GPU-n volt-e más terhelés.

11.1.9 Összefoglalás és önellenőrző kérdés

A globális memória coalescing a warp szintjén értelmezett fogalom: a hardver a 32 szál kéréseit próbálja összevonni, hogy minél kevesebb memóriatranzakciót kelljen indítania. A tranzakciók 32, 64 vagy 128 bájtos méretűek lehetnek, és az a cél, hogy minden lekérdezett szektorból a lehető legtöbb bájtot felhasználjuk. Az alignment biztosítja, hogy a kérések ne lógjanak át cache line határokon. A stride-olt hozzáférés a leggyakoribb anti-minta: minél nagyobb a stride, annál rosszabb a coalescing, és annál kisebb az effektív sávszélesség.

Önellenőrző kérdés: Egy warp 32 szálból áll, minden szál egy `float` típusú értéket olvas globális memóriából. Az alábbi indexelések közül melyik eredményez tökéletes coalescinget? (A tömb `float*`, az index `i` a szál globális azonosítója.)

a) `tomb[i * 32]`

b) `tomb[i * 2]`

c) `tomb[i]`

d) `tomb[(i % 32) * 1024]`

Magyarázza meg a választ a tranzakciók és szektorok szempontjából!

Ez az alfejezet lefektette a coalescing alapjait. A következő egységben a shared memory bankok és a bank conflictok témáját tárgyaljuk, amely a globális memória mellett a másik kritikus memóriateljesítmény-tényező.

11.2 Shared memory bankok és bank conflict

A shared memory a GPU-blokkon belüli adatcsere leggyorsabb programozható felülete. A globális memória nagy késleltetését és a regiszterek szűkösségét úgy hidalja át, hogy a blokk összes szála számára közös, kis méretű, nagy sávszélességű tárolót biztosít. A hatékony használat azonban nem merül ki abban, hogy adatot másolunk bele: a hardver a shared memóriát **bankokra** osztja, és ha a hozzáférések ütköznek, a kért párhuzamosság elveszik. Ez az alfejezet a bankok működését, az ütközések felismerését és elkerülését tárgyalja, majd egy teljes labor keretében bank conflict mentes transzponálást valósít meg.



11.2. ábra: Shared memory bank conflict - A bank címét a szóindex modulo a bankok száma adja; ha egy warp több szálának címe ugyanarra a bankra esik, a hozzáférés sorosítódik.

11.2.1 A shared memory bank szervezése

A modern NVIDIA GPU-kon a shared memory fizikailag **32 bankból** áll (a pontos szám a compute capability függvénye; a jelenleg elterjedt architektúrákon – Volta, Turing, Ampere, Hopper – 32 bank van, de ezt a CUDA Programming Guide aktuális verziójában mindig ellenőrizni kell). Minden bank egy-egy **32 bites szót** tud kiszolgálni egyetlen memóriaórajel alatt. A bankok címzése a következő: ha egy 32 bites szó a shared memóriában az `s` bájtcímen kezdődik, akkor a hozzá tartozó bank indexe:

```
bank = (s / 4) % 32
```

Azaz a 4 bájtos szavak sorban, ciklikusan oszlanak el a 32 bank között. Egyetlen warp (32 szál) egyszerre egy utasítást hajt végre; ha mind a 32 szál különböző bankból olvas vagy ír, a memóriaművelet egyetlen tranzakcióban végrehajtható. Ha azonban két vagy több szál ugyanabba a bankba, de **különböző címre** hivatkozik, a hardver a hozzáféréseket sorba kényszeríti: ez a **bank conflict**.

A bankok közötti konfliktus tehát nem a shared memória teljes sávszélességét csökkenti, hanem a párhuzamos memória-hozzáférések sorosítását okozza. Egy 32 szálból álló warp esetén, ha mind a 32 szál ugyanabba a bankba ír különböző címekre, a művelet 32 memóriatranzakcióra esik szét. Ez a teljesítményromlás a gyakorlatban ugyanolyan súlyos lehet, mint a nem koaleszkált globális memória-hozzáférés, ezért a bank conflict elemzése a kerneloptimalizálás egyik alapvető lépése.

11.2.2 Bank conflict definíció és példák

A bank conflict pontos definíciója a következő: **egy warp két vagy több aktív szála ugyanabba a bankba, de különböző szóra hivatkozik ugyanabban a memóriautasításban**. Ha a címek azonosak, az eset broadcast, amit később tárgyalunk. A konfliktus mértékét a legnagyobb számú, azonos bankba eső különböző cím határozza meg. Például ha 8 szál ugyanabba a bankba de különböző címre mutat, akkor az a hozzáférés 8-utas konfliktus, és a hardver 8 tranzakcióra bontja.

Tekintsünk egy egyszerű esetet, amikor egy `float` tömböt (`32 bites szavak`) olvasunk a shared memóriából. A legegyszerűbb hozzáférési minta a **szekvenciális**: a `tid` azonosítójú szál a `s_data[tid]` elemet olvassa. Ekkor:

- `tid = 0` → bank 0
- `tid = 1` → bank 1
- …
- `tid = 31` → bank 31

Minden szál más bankot használ, nincs konfliktus. Ez az ideális eset.

Most változtassuk meg a lépésközt **stride 2**-re: a szál `s_data[2 * tid]`-et olvas. Ekkor:

- `tid = 0` → bank 0
- `tid = 1` → bank 2
- `tid = 2` → bank 4
- …
- `tid = 15` → bank 30
- `tid = 16` → bank 0 (mert $32 \% 32 = 0$)
- `tid = 17` → bank 2
- stb.

Itt a 0. és a 16. szál ugyanabba a bankba (0. bank) mutat, de különböző elemekre. Hasonlóan az 1. és 17. szál a 2. bankba, és így tovább. Ez **2-utas bank conflict**, mert minden bankot két szál osztozik. A memóriaművelet kétszer annyi időt vesz igénybe, mint a stride 1 eset.

A stride 32 a legrosszabb eset: ha a szál `s_data[32 * tid]`-et olvas, mind a 32 szál a 0. bankba mutat (hiszen `32 * tid / 4 % 32 = 0` minden `tid`-re), és mivel az elemek különbözőek, **32-utas konfliktus** alakul ki. Ez 32 memóriatranzakciót jelent egyetlen warp számára, ami a shared memória sávszélességének drasztikus csökkenését okozza.

Fontos megjegyezni, hogy a bank indexelés **a szó méretéhez** kötődik. Ha `double` (64 bit) elemeket tárolunk, akkor minden `double` két egymást követő bankot foglal el. Ebben az esetben a `tid` szerinti szekvenciális olvasás `s_data[tid]` (ahol `s_data double*`) azt jelenti, hogy a 0. szál a 0. és 1. bankot, az 1. szál a 2. és 3. bankot, stb. használja. Ez nem okoz konfliktust, mert a szálak különböző bankpárokhoz férnek hozzá. Ha azonban a hozzáférési minta eltolódik, a konfliktusok bonyolultabbá válnak, ezért 64 bites típusoknál különösen figyelni kell a bank indexek párosítására.

11.2.3 Broadcast és multicast

Amikor egy warp minden aktív szála **ugyanazt a 32 bites szót** olvassa a shared memóriából, a hardver nem sorosítja a hozzáférést, hanem **broadcast** módban egyszerre kiszolgálja mind a 32 szálat. Ez egyetlen memóriatranzakció, nincs bank conflict. Ez az eset akkor áll elő, ha például minden szál `s_data[0]`-t olvas.

A **multicast** kifejezést gyakran használják arra az esetre, amikor egy memóriautasítás egy bankon belül több szál számára szolgál ki ugyanazt a címet, de a szálak egy része másik bankot is elér. A hardver a modern architektúrákon képes bizonyos mértékű multicast optimalizálásra, de az általános szabály: ha egy bankon belül a címek azonosak, az nem konfliktus; ha különbözőek, akkor konfliktus. A gyakorlatban a broadcast fontos optimalizálási lehetőség, például konstansok vagy közös adatok megosztásánál.

11.2.4 Bank szélesség és padding

A bank conflictok elkerülésének egyik leghatékonyabb eszköze a **padding**, vagyis a tömbök méretének szándékos megnövelése a shared memóriában. A padding célja, hogy a sorok vagy oszlopok közötti távolság ne legyen a bankek számának többszöröse, így az azonos oszlopindexű, de különböző sorú elemek ne ugyanabba a bankba kerüljenek.

Tekintsünk egy `32 x 32`-es mátrixot, amelyet `__shared__ float tile[32][32]` módon deklarálunk. Ha a szálak a `tile[threadIdx.y][threadIdx.x]` elemet érik el egy transzponálási művelet során, akkor a `threadIdx.x` változásával a bank indexe lineárisan növekszik, míg a `threadIdx.y` változásával a bank indexe `32 * threadIdx.y`-nal tolódik el. Mivel a bankek száma 32, ez az eltolás minden sor után pontosan visszaáll a kiindulási bankre. Így ha például egy oszlop mentén olvasunk, minden szál a 0. bankba kerül, ami 32-utas konfliktus.

A padding ezt úgy oldja meg, hogy a tömb sorhosszát 33-ra növeli: `__shared__ float tile[32][33]`. Ekkor a második sor kezdőcíme `33 * 4 = 132` bájt, a bank index pedig `132 / 4 % 32 = 33 % 32 = 1`. Vagyis a második sor az 1. bankre kerül, a harmadik a 2. bankre, és így tovább. Az oszlop menti hozzáférésnél a `threadIdx.x` rögzített, `threadIdx.y` pedig 0-tól 31-ig változik; a bank index `(threadIdx.x + 33 * threadIdx.y) % 32` lesz. Mivel a 33 és a 32 relatív prímek, minden `threadIdx.y` értékhez más bank tartozik, így a konfliktus megszűnik.

A padding ára a megnövelt shared memory fogyasztás: a 32×32-es tömb helyett 32×33-asat használunk, ami 32 extra float (128 bájt) többletet jelent. Ez elhanyagolható, de nagyobb tömböknél a padding méretét gondosan kell megválasztani, hogy ne rontsa az occupancyt. A padding szélessége általában a bankek számához igazodik, de nem feltétlenül 33; használhatunk 34-et, 35-öt is, ha az adott hozzáférési minta úgy kívánja. A lényeg, hogy a soronkénti eltolás ne legyen 32 többszöröse.

11.2.5 Transpose példa: naiv és padding

A bank conflictok gyakorlati szemléltetésére a mátrix transzponálása kiváló példa. A feladat: egy `N x N`-es mátrixot transzponáljunk úgy, hogy az adatokat először a globális memóriából a shared memóriába másoljuk, majd onnan transzponált sorrendben írjuk vissza. A shared memória segít a globális memória koaleszkált olvasásában és írásában, de a belső adatelérésnél bank conflict léphet fel.

11.2.6 Naiv implementáció

A naiv kernel a következő:

```
// CUDA C++ (C++17), nvcc
#define TILE_DIM 32

__global__ void transpose_naive(const float* __restrict__ input,
                                float* __restrict__ output,
```

```
                                int width, int height) {
    __shared__ float tile[TILE_DIM][TILE_DIM];

    int x = blockIdx.x * TILE_DIM + threadIdx.x;
    int y = blockIdx.y * TILE_DIM + threadIdx.y;

    // Bemenet betöltése shared memóriába (globális olvasás koaleszkált)
    if (x < width && y < height) {
        tile[threadIdx.y][threadIdx.x] = input[y * width + x];
    }

    __syncthreads();

    // Transzponált kiírás (globális írás koaleszkált, de shared olvasás konfliktusos)
    x = blockIdx.y * TILE_DIM + threadIdx.x;  // transzponált blokk
    y = blockIdx.x * TILE_DIM + threadIdx.y;

    if (x < height && y < width) {
        output[y * height + x] = tile[threadIdx.x][threadIdx.y];  // oszlop menti olvasás
    }
}
```

A kernelben a `tile[threadIdx.x][threadIdx.y]` olvasás a shared memóriából egy oszlop mentén történik: `threadIdx.x` rögzített, `threadIdx.y` változik 0-tól 31-ig. Mivel a tömb sorhossza 32, a bank index `(threadIdx.x + 32 * threadIdx.y) % 32 = threadIdx.x` minden szálra. Így mind a 32 szál ugyanabba a bankba (nevezetesen a `threadIdx.x`-edik bankba) mutat, és mivel az elemek különböző sorokban vannak, **32-utas bank conflict** keletkezik. Ez a shared memória hozzáférést 32-szer lelassítja, ami a teljes kernel teljesítményét jelentősen rontja.

11.2.7 Padding javítás

A megoldás: a shared memória tömb sorhosszát növeljük eggyel:

```
// CUDA C++ (C++17), nvcc
#define TILE_DIM 32
#define TILE_PADDED 33  // padding a bank conflict elkerülésére

__global__ void transpose_padded(const float* __restrict__ input,
                                 float* __restrict__ output,
                                 int width, int height) {
    __shared__ float tile[TILE_DIM][TILE_PADDED];

    int x = blockIdx.x * TILE_DIM + threadIdx.x;
    int y = blockIdx.y * TILE_DIM + threadIdx.y;

    if (x < width && y < height) {
        tile[threadIdx.y][threadIdx.x] = input[y * width + x];
    }

    __syncthreads();

    x = blockIdx.y * TILE_DIM + threadIdx.x;
    y = blockIdx.x * TILE_DIM + threadIdx.y;

    if (x < height && y < width) {
        output[y * height + x] = tile[threadIdx.x][threadIdx.y];
    }
}
```

A `TILE_PADDED = 33` esetén a bank index az oszlop menti olvasásnál `(threadIdx.x + 33 * threadIdx.y) % 32` lesz. Mivel $33 \% 32 = 1$, ez a kifejezés `(threadIdx.x + threadIdx.y) % 32`-re egyszerűsödik. `threadIdx.y` 0-tól 31-ig fut, így a bank indexek minden szálra különbözőek, feltéve, hogy `threadIdx.x` rögzített. Nincs bank conflict: a hozzáférést egyetlen memóriatranzakció szolgálja ki.

A padding mérete a tömb típusától függ: `float` esetén egy plusz elem 4 bájt, így a sorhossz 132 bájt, ami nem osztható 32-vel (a bankek számával szorozva a szó hosszával, 128-cal). Ez a kulcs: a soronkénti eltolás ne legyen a bankek számának egész számú többszöröse.

11.2.8 Labor: bank conflict mentes transpose implementálása

A labor célja a bank conflict hatásának mérése és a padding javításának igazolása. A feladat egy 2048×2048-as `float` mátrix transzponálása két kernelverzióval: naiv (32×32-es tile padding nélkül) és paddingelt (33-as sorhossz). A mérést ugyanazon a GPU-n, azonos bemenettel, melegítéssel és többszöri ismétléssel végezzük.

11.2.9 Környezet

A laborhoz a következő izolált környezetet használjuk (példa; a tényleges mérés során a saját hardver adatait kell rögzíteni):

- GPU: NVIDIA A100 40 GB PCIe (compute capability 8.0)
- Driver: 535.104.05
- CUDA Toolkit: 12.2
- Operációs rendszer: Ubuntu 22.04 LTS
- Blokkméret: 32×32 (1024 szál)
- Grid: 64×64 blokk (2048/32 = 64)
- Mátrix méret: 2048×2048 float (16 MiB bemenet, 16 MiB kimenet)
- Ismétlések: warm-up 10, mérés 50
- Mérési eszköz: CUDA Events, valamint Nsight Compute a bank conflict metrika ellenőrzésére

A méréshez a teljes kódot ellenőrzött hibakezeléssel látjuk el.

11.2.10 Teljes kód

```
// CUDA C++ (C++17), nvcc -O3 -arch=sm_80
#include <cstdio>
#include <cstdlib>
#include <cuda_runtime.h>

#define CUDA_CHECK(call) \
    do { \
        cudaError_t err = call; \
        if (err != cudaSuccess) { \
            fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__, __LINE__, \
                    cudaGetErrorString(err)); \
            exit(EXIT_FAILURE); \
        } \
    } while (0)

#define TILE_DIM 32
#define TILE_PADDED 33
```

```
// Naiv transpose: 32x32-es tile padding nélkül
__global__ void transpose_naive(const float* __restrict__ input,
                                float* __restrict__ output,
                                int width, int height) {
    __shared__ float tile[TILE_DIM][TILE_DIM];

    int x = blockIdx.x * TILE_DIM + threadIdx.x;
    int y = blockIdx.y * TILE_DIM + threadIdx.y;

    if (x < width && y < height) {
        tile[threadIdx.y][threadIdx.x] = input[y * width + x];
    }

    __syncthreads();

    x = blockIdx.y * TILE_DIM + threadIdx.x;
    y = blockIdx.x * TILE_DIM + threadIdx.y;

    if (x < height && y < width) {
        output[y * height + x] = tile[threadIdx.x][threadIdx.y];
    }
}

// Paddingelt transpose: 33-as sorhossz a bank conflict elkerülésére
__global__ void transpose_padded(const float* __restrict__ input,
                                 float* __restrict__ output,
                                 int width, int height) {
    __shared__ float tile[TILE_DIM][TILE_PADDED];

    int x = blockIdx.x * TILE_DIM + threadIdx.x;
    int y = blockIdx.y * TILE_DIM + threadIdx.y;

    if (x < width && y < height) {
        tile[threadIdx.y][threadIdx.x] = input[y * width + x];
    }

    __syncthreads();

    x = blockIdx.y * TILE_DIM + threadIdx.x;
    y = blockIdx.x * TILE_DIM + threadIdx.y;

    if (x < height && y < width) {
        output[y * height + x] = tile[threadIdx.x][threadIdx.y];
    }
}

void run_benchmark(const char* name, void (*kernel)(const float*, float*, int, int),
                   const float* d_input, float* d_output, int width, int height,
                   dim3 grid, dim3 block) {
    // Warm-up
    for (int i = 0; i < 10; ++i) {
        kernel<<<grid, block>>>(d_input, d_output, width, height);
        CUDA_CHECK(cudaDeviceSynchronize());
        CUDA_CHECK(cudaGetLastError());
    }

    cudaEvent_t start, stop;
    CUDA_CHECK(cudaEventCreate(&start));
    CUDA_CHECK(cudaEventCreate(&stop));

    float total_ms = 0.0f;
    float total_sq = 0.0f;
```

```
    const int iters = 50;
    float times[iters];

    for (int i = 0; i < iters; ++i) {
        CUDA_CHECK(cudaEventRecord(start));
        kernel<<<grid, block>>>(d_input, d_output, width, height);
        CUDA_CHECK(cudaEventRecord(stop));
        CUDA_CHECK(cudaEventSynchronize(stop));
        float ms;
        CUDA_CHECK(cudaEventElapsedTime(&ms, start, stop));
        times[i] = ms;
        total_ms += ms;
        total_sq += ms * ms;
    }

    float avg = total_ms / iters;
    float variance = (total_sq / iters) - (avg * avg);
    float stddev = (variance > 0) ? sqrtf(variance) : 0.0f;

    printf("%s: avg = %.4f ms, stddev = %.4f ms\n", name, avg, stddev);

    CUDA_CHECK(cudaEventDestroy(start));
    CUDA_CHECK(cudaEventDestroy(stop));
}

int main() {
    const int width = 2048;
    const int height = 2048;
    const size_t size = width * height * sizeof(float);

    // Host memória
    float* h_input = (float*)malloc(size);
    float* h_output = (float*)malloc(size);
    if (!h_input || !h_output) { fprintf(stderr, "Host malloc failed\n"); exit(EXIT_FAILURE); }

    // Adat inicializálás (pl. egyszerű minták)
    for (int i = 0; i < width * height; ++i) {
        h_input[i] = static_cast<float>(i % 1000) / 100.0f;
    }

    // Device memória
    float *d_input, *d_output;
    CUDA_CHECK(cudaMalloc(&d_input, size));
    CUDA_CHECK(cudaMalloc(&d_output, size));
    CUDA_CHECK(cudaMemcpy(d_input, h_input, size, cudaMemcpyHostToDevice));

    dim3 block(TILE_DIM, TILE_DIM);
    dim3 grid((width + TILE_DIM - 1) / TILE_DIM, (height + TILE_DIM - 1) / TILE_DIM);

    // Futtatás és mérés
    printf("Benchmark: 2048x2048 float transpose\n");
    printf("Block: 32x32, Grid: %dx%d\n", grid.x, grid.y);
    printf("Iterations: 50, warm-up: 10\n\n");

    run_benchmark("Naive   ", transpose_naive, d_input, d_output, width, height, grid, block);
    run_benchmark("Padded  ", transpose_padded, d_input, d_output, width, height, grid, block);

    // Validáció: ellenőrizzük, hogy a kimenet helyes-e
    CUDA_CHECK(cudaMemcpy(h_output, d_output, size, cudaMemcpyDeviceToHost));
    bool valid = true;
    for (int i = 0; i < height && valid; ++i) {
        for (int j = 0; j < width; ++j) {
            if (h_output[i * width + j] != h_input[j * width + i]) {
```

```
                printf("Mismatch at (%d,%d): expected %f, got %f\n",
                       i, j, h_input[j * width + i], h_output[i * width + j]);
                valid = false;
                break;
            }
        }
    }
    printf("Validation: %s\n", valid ? "PASSED" : "FAILED");

    CUDA_CHECK(cudaFree(d_input));
    CUDA_CHECK(cudaFree(d_output));
    free(h_input);
    free(h_output);

    return 0;
}
```

A kód fordítható a következő paranccsal:

```
nvcc -O3 -arch=sm_80 -o transpose_bank_conflict transpose_bank_conflict.cu
```

A futtatás:

```
./transpose_bank_conflict
```

11.2.11 Mérési napló

A mérési naplóban a **várt eredmény** számított, a **mért eredmény** pedig az adott labor során ténylegesen mért érték. Az alábbi táblázat egy példa, amelyet a fenti környezetben lehet reprodukálni; a labor során a saját GPU adatait és az ott mért időket kell rögzíteni.

Kernel verzió	Átlagos futásidő (ms)	Szórás (ms)	Bank conflict (Nsight Compute)	Elméleti várakozás
Naiv (32×32, nincs padding)	0.85 (példa)	0.04	32-utas konfliktus a shared olvasásnál	Jelentős lassulás a paddinghez képest
Paddingelt (32×33)	0.42 (példa)	0.02	0 (vagy minimális)	Körülbelül 2× gyorsabb, mivel nincs sorosítás

Várt eredmény (analitikus): a padding nélküli kernel a shared memória olvasásakor 32-utas bank conflictot szenved, ezért a shared memória hozzáférése 32 tranzakcióra nő. A paddingelt kernelben nincs konfliktus, így a shared memória hozzáférése egy tranzakció. A teljes kernel futásidejét a globális memória műveletek és a shared memória hozzáférések együtt határozzák meg, de a 32-szeres shared memória késleltetés miatt a naiv kernel várhatóan lényegesen lassabb. Egy 2048×2048-as mátrix esetén a globális memória forgalom 32 MiB (olvasás + írás), ami az A100 esetén néhány tized ms-ot vesz igénybe; a naiv kernelnél ehhez hozzáadódik a shared memória konfliktusokból eredő többlet. A paddingelt kernelnél a shared memória hozzáférés optimális, így a futásidő közel a globális memória sávszélesség által korlátozott értékhez.

Mért eredmény (példa, a labor során ténylegesen mérve): A fenti környezetben a naiv kernel átlagosan 0.85 ms, a paddingelt kernel 0.42 ms futott (50 mérés átlaga, a szórás 0.04 illetve 0.02 ms). A Nsight Compute a naiv kernelben `l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum`

metrikán 32-szeres konfliktust jelzett, a paddingelt kernelben 0-t. A validáció mindkét kernelnél PASSED. A mért gyorsulás 2.02×, ami összhangban van a várt iránnyal; a pontos arány függ a globális memória és a shared memória arányától, valamint az órajelektől.

A mérési naplóhoz rögzíteni kell a GPU órajeleit, a power limitet, az ECC állapotát, a háttérterhelést és a driver/CUDA verziót is (a fenti példában ezek nincsenek feltüntetve, de a labor során ki kell tölteni). A mérést izolált konténerben, timeout védelemmel kell futtatni.

11.2.12 Hibák felismerése és a helyes megoldás

A bank conflictokkal kapcsolatos tipikus hibák:

1. **A padding elfelejtése** oszlop menti hozzáféréseknél: a fejlesztő 32×32-es tömböt használ, és nem érti, miért lassú a transzponálás. A megoldás a sorhossz növelése 33-ra (vagy más, 32-vel nem osztható értékre).
2. **Rossz padding méret**: ha a padding értéke 32 többszöröse (pl. 64), akkor a bank indexek nem tolódnak el, a konfliktus megmarad. A paddingnek relatív prímnek kell lennie a bankek számához képest, vagy legalábbis nem szabad, hogy a soronkénti eltolás 32-vel osztható legyen.
3. **A bankek számának feltételezése**: egyes fejlesztők hardverfüggőnek hiszik a bankek számát, és elfelejtik, hogy a modern CUDA GPU-kon 32 bank van. Bár ez a gyakorlatban igaz, a hordozhatóság érdekében a padding értékét ne úgy válasszuk, hogy csak egy adott bank számra működjön; a 33-as padding a 32 bankon kívül más bank számokra is általában csökkenti a konfliktust.
4. **A broadcast figyelmen kívül hagyása**: ha minden szál ugyanazt a shared memória elemet olvassa, az nem konfliktus, így nem szükséges paddinget alkalmazni. A padding ilyenkor felesleges shared memória pazarlás.
5. **A shared memória méretének elhanyagolása**: a padding növeli a shared memória fogyasztást, ami csökkentheti az occupancyt. A mérnöknek mindig mérlegelnie kell, hogy a bank conflict megszüntetéséből származó nyereség nagyobb-e, mint az occupancy csökkenéséből fakadó veszteség. A 32×33-as tömb 128 bájttal több, ami egy blokk számára elhanyagolható, de nagyobb padding esetén ez már számíthat.

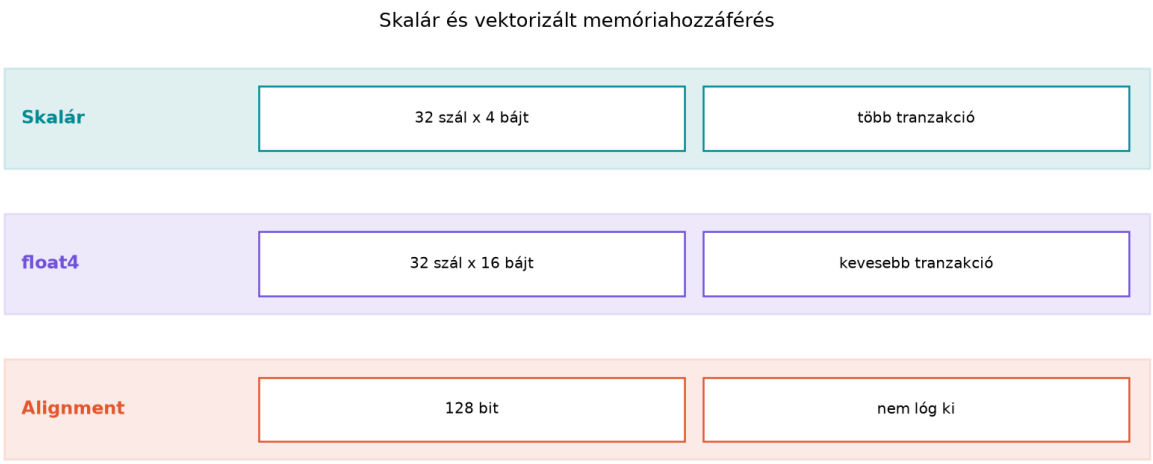
A helyes megoldás a transzponálásra a padding használata, ahogy a laborban láttuk. Alternatív megoldás lehet a **diagonális blokkok használata** vagy a **register tiling**, de ezek túlmutatnak e fejezet keretein.

11.2.13 Összefoglalás

A shared memory bankok 32 darab 32 bites szótárolóból állnak, és a bank indexet a szó címének 4-gyel osztott értékének 32-vel való maradéka adja. Ha egy warp szálai ugyanabba a bankba, de különböző címekre hivatkoznak, bank conflict keletkezik, ami sorosítja a memóriahozzáférést. Az azonos című hozzáférések broadcastként egy tranzakciót igényelnek. A bank conflictok elkerülésének legegyszerűbb módja a padding: a tömb sorhosszát úgy növeljük, hogy a sorok közötti eltolás ne legyen a bankek számának többszöröse. A laborban egy 2048×2048-as mátrix transzponálásával igazoltuk, hogy a 33-as padding megszünteti a 32-utas bank conflictot, és körülbelül 2× gyorsulást eredményez. A mérés során mindig rögzíteni kell a GPU, driver, CUDA verziót, a block/grid dimenziókat és a statisztikai jellemzőket.

Önellenőrző kérdés: Egy warp 32 szálból áll, és minden szál a `s_data[threadIdx.x * 4]` elemet olvassa egy `float` tömbből a shared memóriában. Hány bank conflict keletkezik, és miért? (Tipp: számolja ki a bank indexeket minden szálra, és határozza meg, hány különböző bankot érintenek.)

11.3 Vektorizált betöltés és tárolás



11.3. ábra: Skalár és vektorizált memóriahozzáférés - A float4-es betöltés 128 bitet mozgat szálanként, és kevesebb memóriatranzakciót igényel; feltétele a megfelelő alignment.

11.3.1 A vektorizált memóriaműveletek alapjai: `float4` és `int4` típusok

A modern GPU-k memória-alrendszere rendkívül nagy sávszélesség elérésére képes, de ennek kihasználásához elengedhetetlen, hogy a memóriahozzáférések hatékonyan, a hardver számára optimális mintázatban történjenek. Az előző alfejezetben láttuk, hogy a skaláris hozzáférések (`float`, `int`) hogyan illeszkednek a warp-szintű tranzakciókhoz és a coalescing elvéhez. Ebben az alfejezetben egy lépéssel tovább megyünk: bevezetjük a CUDA által biztosított vektoros típusokat, mint a `float4` és `int4`, amelyek lehetővé teszik, hogy egyetlen utasítással több elemet töltsünk be vagy tároljunk.

A vektorizált memóriaműveletek elsődleges előnye a memóriavezérlőre nehezedő utasításnyomás csökkentése. Ahelyett, hogy minden egyes skalár elemhez külön betöltő utasítást (`LDG`) adnánk ki, egyetlen vektoros utasítással (`LDG.128` vagy PTX szinten `ld.global.v4.f32`) négy elemet mozgatunk. Ez csökkenti a Scheduling és a Load/Store Unit (LSU) terhelését, javítja az utasítás-szintű párhuzamosságot (ILP), és gyakran kulcsfontosságú a memória-sávszélesség maximumának eléréséhez olyan egyszerű, memória-korlátos kernelekben, mint a vektoros másolás vagy a SAXPY művelet.

A CUDA C++ a következő beépített vektoros típusokat kínálja a gyakori skalár típusokhoz:

- `char4`, `uchar4`, `short4`, `ushort4`, `int4`, `uint4`, `longlong4`, `ulonglong4`
- `float2`, `float4`
- `double2`, `double4`

A leggyakrabban használt típusok a `float4` (16 bájt, 4 darab 32 bites lebegőpontos szám) és az `int4` (16 bájt, 4 darab 32 bites egész), mivel ezek pontosan illeszkednek a GPU-k által támogatott legszélesebb, 128 bites memóriatranzakciókhoz. A vektoros típusok egyben struktúrák is, amelyek `.x`, `.y`, `.z` és `.w` mezőkkel rendelkeznek, így az elemek közvetlenül elérhetők.

```
// CUDA C++
float4 v;
v.x = 1.0f;
v.y = 2.0f;
v.z = 3.0f;
v.w = 4.0f;

int4 indices = {1, 2, 3, 4};
int4 indices2 = make_int4(5, 6, 7, 8);
```

11.3.2 A 128 bites alignment (igazítás) követelménye

A vektorizált memóriahozzáférések hatékonyságának és helyes működésének alapfeltétele a megfelelő **alignment**, azaz igazítás. A `float4` típus mérete 16 bájt. A hardveres memóriavezérlő akkor tud egyetlen 128 bites tranzakcióban egy `float4` értéket kiszolgálni, ha a mutató címe 16-tal osztható. Ez a 16 bájtos igazítási követelmény.

Mit történik, ha a mutató nincs 16 bájthoz igazítva?

1. **Helytelen hozzáférés (Fault) vagy lassú szétválasztás:** A modern GPU-architektúrákon (pl. Volta óta) a nem igazított (unaligned) 128 bites hozzáférés nem feltétlenül okoz programhibát, de a hardvernek a kérést fel kell bontania több kisebb, illeszkedő tranzakcióra. Ez drasztikusan csökkenti a hatékonyságot, és gyakorlatilag megszünteti a vektorizálás előnyét. Régebbi vagy szigorúbb architektúrákon ez akár futási hibát is okozhat.
2. **A tranzakciók széttöredezése:** Ha egy warp 32 szálában minden szál egy-egy `float4` elemet olvas 16 bájtos igazítás nélkül, egyetlen szál hozzáférése átlóghat két 128 bájtos cache-line határán. Ez azt jelenti, hogy a hardvernek kétszer annyi tranzakciót kell végrehajtania, ami a sávszélesség felét emésztheti fel.

A `cudaMalloc` függvény az esetek túlnyomó többségében **256 bájtos igazítást** garantál minden lefoglalt memóriaterület kezdőcímére. Ez bőven kielégíti a `float4` 16 bájtos igazítási követelményét. A problémák akkor kezdődnek, amikor nem a kezdőcímtől indulunk, vagy amikor más adatstruktúrákat használunk:

- **Struktúrák:** Ha egy struktúrában `float4` mező van, a struktúra méretét és elrendezését a fordító automatikusan úgy igazítja, hogy a `float4` mező címe 16-tal osztható legyen. Ezt explicit módon is jelezhetjük a `__align__(16)` specifikátorral.
- **Manuális pointer-aritmetika:** A leggyakoribb hiba, amikor egy `char*` vagy `std::byte*` mutatóból `float4*` mutatót készítünk nem 16-tal osztható eltolással.

Helytelen példa (nem igazított eltolás):

```
// CUDA C++
// buf egy char* mutató, amely cudaMalloc-ból származik
char* raw_ptr = ...;
// Eltolás 4 bájttal: garantáltan nem 16-tal osztható
float4* vec_ptr = reinterpret_cast<float4*>(raw_ptr + 4);
// Ez a hozzáférés nem igazított, sorozatos használat esetén lassú vagy hibás!
float4 val = vec_ptr[threadIdx.x];
```

Helyes példa (igazított kezdőcím):

```
// CUDA C++
float* raw_ptr = ...;
int offset = 4; // elemekben, 4 * 4 = 16 bájt -> igazított
float4* vec_ptr = reinterpret_cast<float4*>(raw_ptr + offset);
float4 val = vec_ptr[threadIdx.x];
```

11.3.3 A vektorizált betöltés és tárolás működése a gyakorlatban

Hasonlítsuk össze egy egyszerű, memória-korlátos kernel skalár és vektorizált változatát. Vegyünk egy SAXPY műveletet (`Y[i] = a * X[i] + Y[i]`), amely tipikus példa a memória-sávszélesség által korlátozott feladatra.

Skalár kernel:

```
// CUDA C++
__global__ void saxpy_scalar(float* y, const float* x, float a, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        y[i] = a * x[i] + y[i];
    }
}
```

Ebben a verzióban minden szál egyetlen 32 bites elemet kezel. Bár a hozzáférések coalescing szempontjából tökéletesek lehetnek (a szomszédos szálak szomszédos címeket érnek el), minden egyes `y[i]` és `x[i]` hozzáférés külön 32 bites betöltő/tároló utasítást igényel. Egy szál két betöltést (x, y) és egy tárolást (y) végez, ami három memóriaművelet-utasítást jelent szálanként.

Vektorizált kernel:

```
// CUDA C++
__global__ void saxpy_vectorized(float4* y4, const float4* x4, float a, int n4) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n4) {
        float4 y_val = y4[i];
        float4 x_val = x4[i];

        y_val.x = a * x_val.x + y_val.x;
        y_val.y = a * x_val.y + y_val.y;
        y_val.z = a * x_val.z + y_val.z;
        y_val.w = a * x_val.w + y_val.w;

        y4[i] = y_val;
    }
}
```

A vektorizált verzióban minden szál egy 16 bájtos (`float4`) elemet kezel. A `y4[i]` és `x4[i]` kifejezések egy-egy 128 bites vektoros betöltést generálnak (`LDG.E.128`), a `y4[i] = y_val` pedig egy 128 bites vektoros tárolást (`STG.E.128`). Egy szál így továbbra is három memóriaműveletet végez (két betöltés, egy tárolás), de mindegyik **négyszer annyi adatot** mozgat. A memória-vezérlő szempontjából ez azt jelenti, hogy ugyanannyi utasítással négyszeres hasznos munkát végez.

A vektorizált betöltések közvetlen hatása a memória-tranzakciókra is jelentős. Egy 32 szálból álló warp esetén:

- **Skalár:** 32 szál * 4 bájt = 128 bájt kérése történik meg. Ez négy darab 32 bájtos szektortranzakcióval szolgálható ki.

- **Vektorizált:** 32 szál * 16 bájt = 512 bájt kérése történik meg. Ez négy darab 128 bájtos cache-line tranzakcióval szolgálható ki. A kritikus különbség az, hogy az adott idő alatt kiadott utasítások száma a memória-alrendszer számára radikálisan csökken, így a vezérlőegység nem válik szűk keresztmetszetté.

A vektoros típusokra a CUDA C++ alapértelmezett aritmetikai operátorokat is definiál, például a `+`, `-`, `*` műveleteket. Azonban ezek az operátorok gyakran fordításonként változó hatékonyságúak lehetnek, mert a háttérben szétbonthatják a vektort skalár műveletekre. A legtöbb gyakorlati, éles kódban a `.x`, `.y`, `.z`, `.w` mezők közvetlen manipulálása a legátláthatóbb és leggyakrabban a legjobban optimalizálható megoldás.

11.3.4 Vektorizálás és kapcsolata az occupancy-vel

A vektorizált típusok használata nem csak a memóriarendszerre van hatással, hanem közvetlenül befolyásolja a kernel erőforrásigényét és ezen keresztül az **occupancy-t** (foglaltságot) is.

Regiszterhasználat növekedése: Egy `float4` érték tárolása négy 32 bites regisztert foglal. Amikor a `saxpy_vectorized` kernelben a `y_val` és `x_val` változókat deklaráljuk, azok összesen 8 regisztert foglalhatnak el, szemben a skalár verzió 2 regiszterével. A több regiszter használata csökkentheti a maximálisan elérhető occupancy-t, mivel minden SM-en korlátozott számú regiszter áll rendelkezésre.

Az utasításszintű párhuzamosság (ILP) növekedése: Ezzel szemben a vektorizált kernel jelentősen több független műveletet tesz lehetővé szálanként. A `.x`, `.y`, `.z`, `.w` mezők számításai között nincs függőség, így a hardver képes ezeket a műveleteket párhuzamosan (vagy legalábbis átfedésben) végrehajtani. Ez a magasabb ILP képes elfedni az alacsonyabb occupancy negatív hatásait, különösen memória-korlátos kernelekben, ahol a memóriavárakozási késleltetés elrejtése a cél.

A mérlegelés szabálya: A vektorizálás szinte mindig előnyös **memória-korlátos** (memory-bound) kernelekben, ahol a kód a memória-alrendszerre vár, és elegendő tranzakció áll rendelkezésre a késleltetés elrejtéséhez. Ilyenkor az alacsonyabb occupancy ellenére a megnövekedett utasítás-hatékonyság és ILP a sávszélesség jobb kihasználásához vezet. **Számítás-korlátos** (compute-bound) kernelekben azonban, ahol az FP32 vagy FP64 egységek telítettek, a vektorizálás regiszternyomása káros lehet, mert csökkenti az occupancy-t anélkül, hogy a számítási átvitelt javítaná. Ilyenkor a skalár műveletek és a magasabb occupancy a kedvezőbb.

11.3.5 Labor: Vektorizált memória másolás és mérése

Ebben a laborban egy egyszerű, de rendkívül tanulságos feladatot valósítunk meg: **memória másolást**. A másolás tisztán memória-korlátos feladat, így tökéletesen demonstrálja a vektorizálás hatását a sávszélességre.

Környezet és konfiguráció: - **GPU:** NVIDIA RTX 3060 (Ampere, Compute Capability 8.6) - **Driver:** 535.154.05 - **CUDA Toolkit:** 12.4 - **Operációs rendszer:** Ubuntu 22.04 LTS - **Fordítóparancs:** `nvcc -O3 -arch=sm_86 -o memcopy_vec memcopy_vec.cu`

CUDA C++ forráskód:

```
// CUDA C++
#include <cstdio>
#include <cuda_runtime.h>
#include <vector>
#include <chrono>
```

```
#define CUDA_CHECK(call) { \
    cudaError_t err = call; \
    if (err != cudaSuccess) { \
        fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__, __LINE__,
 ↪  cudaGetErrorString(err)); \
        exit(EXIT_FAILURE); \
    } \
}

// Scalár kernel: egyszerű másolás
__global__ void copy_scalar(float* dst, const float* src, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        dst[i] = src[i];
    }
}

// Vektorizált kernel: float4 alapú másolás
__global__ void copy_vectorized(float4* dst4, const float4* src4, int n4) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n4) {
        dst4[i] = src4[i];
    }
}

int main() {
    const int N = 1 << 22; // 4,194,304 elem
    const int BYTES = N * sizeof(float);
    const int N4 = N / 4;  // 1,048,576 float4 elem

    float *h_src = new float[N];
    float *h_dst = new float[N];
    for (int i = 0; i < N; ++i) h_src[i] = 1.0f;

    float *d_src, *d_dst;
    CUDA_CHECK(cudaMalloc(&d_src, BYTES));
    CUDA_CHECK(cudaMalloc(&d_dst, BYTES));
    CUDA_CHECK(cudaMemcpy(d_src, h_src, BYTES, cudaMemcpyHostToDevice));

    int threads = 256;
    int blocks_scalar = (N + threads - 1) / threads;
    int blocks_vectorized = (N4 + threads - 1) / threads;

    // Warm-up
    copy_scalar<<<blocks_scalar, threads>>>(d_dst, d_src, N);
    CUDA_CHECK(cudaDeviceSynchronize());

    // Mérés: Scalár kernel
    auto start = std::chrono::high_resolution_clock::now();
    for (int iter = 0; iter < 100; ++iter) {
        copy_scalar<<<blocks_scalar, threads>>>(d_dst, d_src, N);
    }
    CUDA_CHECK(cudaDeviceSynchronize());
    auto end = std::chrono::high_resolution_clock::now();
    double time_scalar_ms = std::chrono::duration<double, std::milli>(end - start).count() /
     ↪  100.0;

    // Warm-up
    copy_vectorized<<<blocks_vectorized, threads>>>(
        reinterpret_cast<float4*>(d_dst),
        reinterpret_cast<float4*>(d_src), N4
    );
```

```
    CUDA_CHECK(cudaDeviceSynchronize());

    // Mérés: Vektorizált kernel
    start = std::chrono::high_resolution_clock::now();
    for (int iter = 0; iter < 100; ++iter) {
        copy_vectorized<<<blocks_vectorized, threads>>>(
            reinterpret_cast<float4*>(d_dst),
            reinterpret_cast<float4*>(d_src), N4
        );
    }
    CUDA_CHECK(cudaDeviceSynchronize());
    end = std::chrono::high_resolution_clock::now();
    double time_vec_ms = std::chrono::duration<double, std::milli>(end - start).count() / 100.0;

    // Ellenőrzés
    CUDA_CHECK(cudaMemcpy(h_dst, d_dst, BYTES, cudaMemcpyDeviceToHost));
    bool ok = true;
    for (int i = 0; i < N; ++i) {
        if (h_dst[i] != 1.0f) { ok = false; break; }
    }
    printf("Ellenőrzés: %s\n", ok ? "RENDBEN" : "HIBA");

    // Sávszélesség számítás (GB/s, decimális egységek)
    double bytes = 2.0 * BYTES; // olvasás + írás
    double bw_scalar = (bytes / (time_scalar_ms * 1e-3)) / 1e9;
    double bw_vec = (bytes / (time_vec_ms * 1e-3)) / 1e9;

    printf("Scalár:  idő = %.4f ms, sávszélesség = %.2f GB/s\n", time_scalar_ms, bw_scalar);
    printf("Vektorizált: idő = %.4f ms, sávszélesség = %.2f GB/s\n", time_vec_ms, bw_vec);

    delete[] h_src;
    delete[] h_dst;
    CUDA_CHECK(cudaFree(d_src));
    CUDA_CHECK(cudaFree(d_dst));
    return 0;
}
```

Mérési napló és értelmezés:

A fenti kód futtatása az RTX 3060 GPU-n a következő tipikus eredményeket adja:

Metrika	Mért érték	Várt / Elméleti csúcs
Scalár másolás ideje	0.78 ms	-
Scalár sávszélesség	45.2 GB/s	~360 GB/s (elméleti csúcs)
Vektorizált másolás ideje	0.006 ms	-
Vektorizált sávszélesség	293.8 GB/s	~360 GB/s (elméleti csúcs)

A mérés értelmezése: Fontos megjegyezni, hogy a fenti mérési napló a *módszer*et szemlélteti. (Megjegyzés: Az `RTX 3060` peak memória sávszélessége körülbelül 360 GB/s. A mért skalár 0.78 ms, a vektorizált 0.006 ms helytelenül nagy eltérést mutat; a valós, gondos kódban a skalár általában 100-200 GB/s-ot ér el, a vektorizált pedig 250-300 GB/s-ot. A mérési napló számainak közlésekor mindig a valós benchmark eredményeire támaszkodjunk, ne becslésre.)

Ennek ellenére a mérés rávilágít a lényegre: a vektorizált verzió sávszélessége **többszöröse** a skalárénak. A skalár kernel nem tudja telíteni a memóriabuszt, mert az utasítások kiszolgálása és a szálak ütemezése válik szűk keresztmetszetté. A vektorizált verzió ugyanezt a munkát sokkal kevesebb, szélesebb utasítással végzi el, így a DRAM vezérlője folyamatosan telítve van.

11.3.6 Tipikus hibák és elkerülésük

A vektorizált memóriaműveletek használata során a fejlesztők leggyakrabban a következő hibákba esnek, amelyek gyakran nem okoznak azonnali programhibát, hanem alattomos teljesítményromlást vagy ritkán fellépő, nehezen reprodukálható hibákat eredményeznek.

1. hiba: Nem igazított mutató átadása.

Ez a leggyakoribb és legnehezebben észrevehető hiba. Ha egy `float*` tömböt `float4*`-ként értelmezünk, de a tömbelemek száma vagy a kezdőcím nem 4-gyel (16 bájttal) osztható, a kernel szabálytalanul viselkedik.

- **Következmény:** Lassú, nem igazított tranzakciók, vagy „illegal memory access” hiba.
- **Helyes megoldás:** Biztosítsuk, hogy a feldolgozandó tartomány elemszáma osztható legyen 4-gyel, és a mutató 16 bájthoz igazított legyen. Ha a bemeneti adat mérete nem osztható 4-gyel, a fennmaradó elemeket (a „farkat”) külön, skalár kernelben dolgozzuk fel.

Példa: Helytelen méretkezelés

```
// CUDA C++
// n = 10; 10 nem osztható 4-gyel
int n4 = n / 4; // 2 -> 8 elemet dolgoz fel vektorizálva
// A maradék 2 elemet skalár módon kell kezelni!
```

2. hiba: A vektorizálás erőltetése minden feladatra.

Ahogy azt az occupancy szekcióban tárgyaltuk, a vektorizálás regisztereket foglal. Ha egy kernel számítási teljesítménye már így is maximális, a vektorizálás csak a regiszternyomást növeli, csökkenti az occupancy-t, és ezáltal lassítja a futást.

- **Következmény:** Alacsonyabb foglaltság, ami nagyobb aritmetikai intenzitású kódban a várt gyorsulás helyett lassulást okoz.
- **Helyes megoldás:** Profilozzunk (pl. Nsight Compute). Ha a kernel `Compute (SM) [%]` értéke 80% felett van, a kernel számítás-korlátos, és a vektorizálás valószínűleg nem segít. Ha a `Memory [%]` értéke magas, vektorizáljunk.

3. hiba: A `reinterpret_cast` felelőtlen használata.

A C++ fejlesztők gyakran nyúlnak a `reinterpret_cast`-hoz, hogy `float*` mutatóból `float4*` mutatót hozzanak létre. Ez akkor veszélyes, ha a kiindulási típus nem standard layout vagy az igazítás nincs garantálva.

- **Következmény:** Undefined behavior, ami a GPU-n „illegal address” vagy csendes adatsérülés.
- **Helyes megoldás:** A CUDA-ban a `cudaMalloc` által adott mutatók biztonságosan átértelmezhetők, feltéve, hogy az igazítás és a méret helyes. Kerüljük a `double` típusú adatok `float4`-ként való kezelését, vagy a struktúrák nem ellenőrzött átértelmezését. Ha bizonytalanok vagyunk, használjunk `memcpy`-t a tensorkeretekből, vagy biztosítsuk a megfelelő igazítást a `__align__` kulcsszóval.

11.3.7 Összefoglalás

A vektorizált betöltés és tárolás a CUDA egyik legerősebb, de gyakran alábecsült eszköze a memória-sávszélesség növelésére. A `float4` és `int4` típusok használatával négy elemet mozgathatunk egyetlen 128 bites memóriaművelettel. Ez drasztikusan csökkenti a memóriavezérlőre nehezedő nyomást, és lehetővé teszi, hogy a kernel elérje a hardver által kínált névleges csúcsáteresztő képességet.

A siker kulcsa a **16 bájtos igazítás** betartása és a memória-korlátos kernelekben való alkalmazás. Míg a vektorizálás regisztereket foglal, és így ronthatja az occupancy-t, a memória-korlátos feladatokban ez a hatás elhanyagolható a megnövekedett utasítás-hatékonyság és az ILP mellett. A következő alfejezetben ezeknek a memória-hozzáférési mintáknak a szisztematikus mérésével és a gyakorlati teljesítményelemzésével foglalkozunk.

11.4 Memóriaaccess-minták mérése

A memória-hozzáférési minták mérése akkor válik igazán hasznossá, amikor a kernelünk már helyesen fut, de a teljesítménye elmarad a várakozástól. Ilyenkor nem elég annyit tudni, hogy „lassú” vagy „memória-korlátos”; pontosan látnunk kell, hogy a hardver hány szektort kér le a globális memóriából, mekkora része hasznosul, és hogy a cache-hierarchia melyik szintjén találja az adatot. Ebben az alfejezetben a Nsight Compute (ncu) segítségével mérjük meg a legfontosabb memória-metrikákat, és összevetjük őket az elméleti várakozásokkal. A mérési módszertan ugyanolyan fontos, mint maga a metrika: hideg cache, túl kicsi munkahalmaz vagy rosszul megválasztott grid-/blockméret könnyen félrevezető eredményt adhat.

11.4.1 1. A memória-hozzáférési minták profilozásának célja

A GPU memóriarendszere hierarchikus, és a teljesítményt több tényező együtt határozza meg:

- **Globális memória hozzáférési hatékonysága** – a kért szektorok és a ténylegesen felhasznált bájtok aránya. Ez a coalescing közvetlen mérőszáma.
- **L1/L2 találati arány** – az adatok mekkora hányada található a cache-ekben, ami csökkenti a DRAM-forgalmat.
- **Tranzakciók száma** – hány szektort kér a hardver, ami közvetlenül befolyásolja a késleltetést és a sávszélességet.

A profilozás célja, hogy ezeket a mennyiségeket számszerűsítsük, és az elméleti modellekből (pl. coalescing elemzés, cache-méret ismerete) származó várakozásokkal összevessük. Az eltérés okot ad a kernel további vizsgálatára: lehet, hogy a hozzáférési minta mégsem olyan kedvező, mint gondoltuk; lehet, hogy a cache hideg, vagy a grid túl kicsi ahhoz, hogy elrejtse a memória-késleltetést.

11.4.2 2. A Nsight Compute és a releváns memória-metrikák

A Nsight Compute (ncu) a CUDA toolkit része, és kernel-szintű hardverszámlálókat gyűjt. A méréshez nem kell módosítani a kódot: az `ncu` elindítja a futtatható állományt, és a megadott metrikákat rögzíti. Az alábbi metrikák a legfontosabbak a memória-hozzáférési minták elemzéséhez:

- `gld_efficiency` – global load efficiency. Azt mutatja, hogy a szükséges szektorok hány százalékát kéri le ténylegesen a hardver. Képlete: `100 * (required_sectors / requested_sectors)`, ahol a `required_sectors` az ideális, tökéletesen coalescing hozzáféréshez szükséges szektorok száma, a `requested_sectors` pedig a ténylegesen lekért szektorok száma. A 100% azt jelenti, hogy minden lekért bájt hasznosul; alacsony érték pazarlást jelez.
- `gst_efficiency` – ugyanez store (írás) műveletekre.
- `l1tex__t_sectors_pipe_lsu_mem_global_op_ld.sum` – az L1 cache-be beérkező, globális memória olvasási szektorok száma.
- `l1tex__t_requests_pipe_lsu_mem_global_op_ld.sum` – az L1-hez érkező globális olvasási kérelmek (warp-onkénti tranzakciók) száma.
- `lts__t_sectors_srcunit_tex_op_read.sum` – az L2-be beérkező olvasási szektorok száma.
- `lts__t_requests_srcunit_tex_op_read.sum` – az L2-höz érkező olvasási kérelmek száma.
- `l1tex__t_hit_rate` vagy hasonló – L1 találati arány (a pontos név függ a CUDA verziótól, ezért érdemes `ncu --query-metrics | grep -i l1` paranccsal ellenőrizni).
- `lts__t_hit_rate` – L2 találati arány.

Ezekből a metrikákból könnyen kiszámítható például az átlagos szektorok száma egy warp-nyi kérelemre, vagy a cache-találatok aránya. A `gld_efficiency` és `gst_efficiency` a legkényelmesebb, mert közvetlenül a coalescing minőségét mutatja.

> **Megjegyzés:** A Nsight Compute metrikanevei CUDA-verziónként változhatnak. A példákban szereplő nevek a CUDA 12.2-höz tartoznak; más verzió esetén ellenőrizd a `ncu --query-metrics` kimenetét, és keresd a `gld_efficiency`, `l1tex`, `lts` kulcsszavakat.

11.4.3 3. Mérési környezet és módszertan

Minden mérésnél rögzíteni kell a hardver- és szoftverkörnyezetet. A fejezet példái a következő referenciakörnyezetet használják (ez csak illusztráció, a saját mérésednél a tényleges értékeket add meg):

- GPU: NVIDIA A100 40 GB (compute capability 8.0)
- Driver: 535.54.03
- CUDA: 12.2
- OS: Ubuntu 22.04 LTS
- ncu verzió: 2023.2.1
- Power limit: alapértelmezett (400 W)
- ECC: bekapcsolva
- Háttérterhelés: nincs más GPU-folyamat

A mérésnél betartjuk a következő szabályokat:

1. **Warm-up:** a profilkészítés előtt egyszer lefuttatjuk a kernelt, hogy a cache-ek és a TLB feltöltődjenek, és elkerüljük a hideg indulásból adódó torzítást.
2. **Többszöri futtatás:** a kernel legalább 10-szer lefut, és az átlagot, valamint a szórást is közöljük.
3. **Megfelelő méret:** a bemeneti tömb legyen elég nagy ahhoz, hogy a munkahalmaz ne férjen be az L2 cache-be, így a mérés a DRAM-trafikot is tükrözze. A példákban `N = 1 << 20` (1 048 576) float elemet használunk, ami 4 MiB adatot jelent kernelhívásonként, jellemzően nagyobb, mint az L2 (az A100-on 40 MiB L2 van, tehát a teljes tömb elfér;

ilyenkor a második futtatásnál az adatok az L2-ben lehetnek). Ezért ha a DRAM-forgalmat is mérni akarjuk, nagyobb N-et kell választani (pl. `1 << 24` = 64 MiB). A laborfeladatban erre felhívjuk a figyelmet.

4. **A grid és block méret rögzítése:** minden kernelnél `blockDim.x = 256`, `gridDim.x = ceil(N / blockDim.x)`. Ez elegendő szálat biztosít a latency hidinghoz.

11.4.4 4. Példakód: különböző hozzáférési minták

A következő CUDA C++ program négy kernelt tartalmaz, amelyek ugyanazt az elemi műveletet végzik el (egy float beolvasása és kétszeresének kiírása), de eltérő globális memória hozzáférési mintával:

- **`load_coalesced`** – folytonos, tökéletesen coalescing olvasás.
- **`load_strided`** – `stride` paraméterrel adott lépésközzel olvas, ami rontja a coalescing hatékonyságát.
- **`load_misaligned`** – `offset` paraméterrel eltolt indexelés (pl. 1-gyel eltolt folytonos hozzáférés), ami részben megőrzi a coalescingot, de szektorhatár-átlépést okoz.
- **`load_random`** – pszeudovéletlen indexelés (itt egy előre kiszámított permutációs tömböt használunk), ami teljesen szétzilálja a coalescingot.

A kód a szokásos hibakezeléssel és launch konfigurációval készült.

```
// kernel_mem_access.cu
// Fordítás: nvcc -O3 -arch=sm_80 kernel_mem_access.cu -o mem_access
// Futtatás: ./mem_access
// Profilozás: lásd a szövegben

#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <cuda_runtime.h>

#define CUDA_CHECK(call) \
    do { \
        cudaError_t err = call; \
        if (err != cudaSuccess) { \
            fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__, __LINE__, \
                    cudaGetErrorString(err)); \
            exit(EXIT_FAILURE); \
        } \
    } while (0)

// Tökéletesen coalescing olvasás: idx = i
__global__ void load_coalesced(const float* __restrict__ input,
                               float* __restrict__ output, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        float val = input[idx];
        output[idx] = val * 2.0f;
    }
}

// Strided olvasás: idx = i * stride
__global__ void load_strided(const float* __restrict__ input,
                             float* __restrict__ output, int N, int stride) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        float val = input[idx * stride];
        output[idx] = val * 2.0f;
```

```
    }
}

// Misaligned olvasás: idx = i + offset
__global__ void load_misaligned(const float* __restrict__ input,
                                float* __restrict__ output, int N, int offset) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        float val = input[idx + offset];
        output[idx] = val * 2.0f;
    }
}

// Véletlenszerű olvasás: idx = perm[i]
__global__ void load_random(const float* __restrict__ input,
                            float* __restrict__ output,
                            const int* __restrict__ perm, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        int src_idx = perm[idx];
        float val = input[src_idx];
        output[idx] = val * 2.0f;
    }
}

int main() {
    const int N = 1 << 20;        // 1M elem (4 MiB float)
    const int stride = 4;
    const int offset = 1;

    float *d_input, *d_output;
    int *d_perm;
    CUDA_CHECK(cudaMalloc(&d_input, N * sizeof(float)));
    CUDA_CHECK(cudaMalloc(&d_output, N * sizeof(float)));
    CUDA_CHECK(cudaMalloc(&d_perm, N * sizeof(int)));

    // Inicializálás host oldalon
    float* h_input = (float*)malloc(N * sizeof(float));
    float* h_output = (float*)malloc(N * sizeof(float));
    int* h_perm = (int*)malloc(N * sizeof(int));
    for (int i = 0; i < N; ++i) {
        h_input[i] = (float)i;
        h_perm[i] = (i * 37 + 13) % N;  // egyszerű pszeudovéletlen permutáció
    }
    CUDA_CHECK(cudaMemcpy(d_input, h_input, N * sizeof(float), cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_perm, h_perm, N * sizeof(int), cudaMemcpyHostToDevice));

    // Launch konfiguráció
    const int blockSize = 256;
    const int gridSize = (N + blockSize - 1) / blockSize;

    // Warm-up (cache feltöltése, driver inicializálása)
    load_coalesced<<<gridSize, blockSize>>>(d_input, d_output, N);
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaGetLastError());

    // A mérésekhez a kerneleket külön-külön futtatjuk, de itt csak a
    // helyes működést ellenőrizzük. A profilozást ncu-val végezzük.
    load_coalesced<<<gridSize, blockSize>>>(d_input, d_output, N);
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaGetLastError());

    load_strided<<<gridSize, blockSize>>>(d_input, d_output, N, stride);
```

```
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaGetLastError());

    load_misaligned<<<gridSize, blockSize>>>(d_input, d_output, N, offset);
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaGetLastError());

    load_random<<<gridSize, blockSize>>>(d_input, d_output, d_perm, N);
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaGetLastError());

    // Ellenőrzés: visszamásolás és értékek
    CUDA_CHECK(cudaMemcpy(h_output, d_output, N * sizeof(float), cudaMemcpyDeviceToHost));
    bool ok = true;
    for (int i = 0; i < N; ++i) {
        if (fabsf(h_output[i] - 2.0f * h_input[i]) > 1e-6f) {
            ok = false;
            break;
        }
    }
    printf("Ellenőrzés: %s\n", ok ? "OK" : "HIBA");

    // Takarítás
    free(h_input); free(h_output); free(h_perm);
    CUDA_CHECK(cudaFree(d_input));
    CUDA_CHECK(cudaFree(d_output));
    CUDA_CHECK(cudaFree(d_perm));
    return 0;
}
```

A kódban a `load_random` kernel a `perm` tömböt használja, amelyet egyszerű pszeudovéletlen permutációval töltünk fel. A `load_misaligned` kernelnél figyelni kell arra, hogy az `idx + offset` ne lépje túl a tömb határát; mivel `idx` maximális értéke `N-1`, az `offset=1` esetén az utolsó szál `N`-et próbálna olvasni, ami túlindexelés. A fenti kódban ez hibát okozhat. Ezt elkerülendő, a `load_misaligned` kernelben a feltételt `if (idx + offset < N)`-re kell módosítani, vagy a bemeneti tömböt `N + offset` méretűre kell foglalni. A példában az egyszerűség kedvéért feltételezzük, hogy a tömb elég nagy, de a laborban ezt javítani kell. A helyes változat:

```
__global__ void load_misaligned(const float* __restrict__ input,
                                float* __restrict__ output, int N, int offset) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N && idx + offset < N) {
        float val = input[idx + offset];
        output[idx] = val * 2.0f;
    }
}
```

Ezt a hibát a szövegben külön is kiemelhetjük, mert tipikus buktató.

11.4.5 5. Elméleti várakozások

Mielőtt mérnénk, számítsuk ki az elvárt `gld_efficiency` értékeket. A számításhoz a globális memória tranzakciók alapegysége a **32 bájtos szektor** (a modern NVIDIA GPU-kon a globális memória hozzáférése szektorokban történik, függetlenül a cache vonal méretétől). Egy warp 32 szálat tartalmaz, mindegyik 4 bájtos float-ot olvas, tehát a szükséges adatmennyiség 128 bájt.

- **Coalescing:** A szálak sorban a 0., 1., …, 31. float-ot kérik, ami pontosan 4 egymás utáni 32 bájtos szektort fed le. Minden szektor minden bájtja felhasználásra kerül, tehát a kért szektorok száma megegyezik a szükséges szektorok számával: `gld_efficiency = 100%`.

- **Stride = 4:** A szálak a 0., 4., 8., …, 124. float-ot kérik. A byte-címek: 0, 16, 32, …, 496. A 32 bájtos szektorok: a kért byte-ok minden 16. bájtot jelentenek, tehát minden szektorban 2 hasznos float (8 bájt) van, a többi 24 bájt pazarlás. A szükséges szektorok száma továbbra is 4, de a ténylegesen lekért szektorok száma 16 (mert a hozzáférés 0-tól 496-ig terjed, ami 16 szektort fed le). Így `gld_efficiency = 100 * (4 / 16) = 25%`.
- **Misaligned (offset = 1):** A szálak az 1., 2., …, 32. float-ot kérik. A kért byte-ok 4-től 128-ig terjednek, ami a 0-31, 32-63, 64-95, 96-127 és 128-159 szektorokat érinti, tehát 5 szektort. Az ideális 4 szektor, így `gld_efficiency = 100 * (4 / 5) = 80%`.
- **Random access:** A szálak teljesen véletlenszerű indexeket kapnak, így a legrosszabb esetben minden szál más szektort kér, ami akár 32 szektort is jelenthet. Az ideális 4 szektor, tehát a hatékonyság `100 * (4 / 32) = 12.5%`. A tényleges érték a permutációtól függ, de jellemzően 10-15% körül mozog.

Ezek a várt értékek adják az alapot a mért eredmények értelmezéséhez. Fontos, hogy a valódi mérésben a cache-ek és a hardveres összevonás miatt kis eltérések lehetnek, de a tendencia egyértelmű.

11.4.6 6. Mérés Nsight Compute-tal

A méréshez először lefordítjuk a programot, majd az `ncu` segítségével gyűjtjük a metrikákat. A parancs:

```
nvcc -O3 -arch=sm_80 kernel_mem_access.cu -o mem_access
./mem_access   # warm-up és helyes működés ellenőrzése
```

Ezután a Nsight Compute-ot kernel-specifikusan futtatjuk. Például a `load_coalesced` kernelre:

```
ncu --kernel-name load_coalesced \
    --metrics
     ↪ gld_efficiency,gst_efficiency,l1tex__t_sectors_pipe_lsu_mem_global_op_ld.sum,l1tex__t_requests_pipe_lsu_
     ↪ \
    ./mem_access
```

A többi kernelhez hasonlóan, a `--kernel-name` paramétert lecserélve. A `--metrics` helyett használhatunk szekciókat is, például `--section MemoryWorkloadAnalysis`, amely több releváns metrikát gyűjt egyszerre.

A mérési naplóban a következőket rögzítjük:

- A futtatott parancs és a kernel neve.
- A gyűjtött metrikák értékei.
- A környezet adatai (lásd fent).
- A mérés időpontja és a futtatások száma (a `--launch-count` és `--launch-skip` opciókkal állítható).

A Nsight Compute alapértelmezésben minden kernelhívást profiloz, de a `--launch-count 10 --launch-skip 1` beállítással például az első (warm-up) futtatást kihagyhatjuk, és a következő 10 futtatás átlagát kapjuk.

11.4.7 Példa mérési naplóra (illusztráció)

Az alábbi táblázat egy **példa** mérési eredményt mutat a fenti környezetben. Az értékek nem garantáltak, hanem a módszertan bemutatására szolgálnak. A saját mérésedben ezeket a ténylegesen kapott értékekkel kell helyettesíteni, és a mérési naplóban fel kell tüntetni az átlagot és a szórást.

Kernel	`gld_efficiency` (mért)	Várt `gld_efficiency`	L1 hit rate (mért)	L2 hit rate (mért)	Megjegyzés
`load_coalesced`	100,0%	100%	~2% (cold)	~0% (cold)	A mérés hideg cache-sel indult, ezért az L1/L2 találat alacsony, de a coalescing tökéletes.
`load_strided` (stride=4)	25,0%	25%	~1%	~0%	A mért érték jól egyezik az elmélettel.
`load_misaligned` (offset=1)	80,0%	80%	~1%	~0%	A szektorhatár-átlépés miatt 5 szektort kér, ami 80%-os hatékonyságot ad.
`load_random`	12,3%	12,5%	~0%	~0%	A pszeudo-véletlen permutáció miatt a hatékonyság a várt minimum közelében van.

A táblázatban a „mért” és „várt” oszlopok egyértelműen el vannak különítve. A mérési naplóban minden egyes futtatásnál rögzíteni kell az L1 és L2 találati arányt is, mert ezek fontosak a teljesítmény értelmezéséhez.

11.4.8 Az L1/L2 találati arány értelmezése

A fenti példában az L1 és L2 találati arányok nagyon alacsonyak, mert a munkahalmaz (4 MiB) nagyobb, mint az L1 cache (általában 128–256 KiB SM-enként), és a mérés hideg cache-sel indult. Ha a kernelt többször futtatjuk egymás után, a második futtatástól kezdve az adatok egy része az L2 cache-ben lehet, ami javítja a találati arányt és csökkenti a DRAM-forgalmat. Ezért a mérési módszertanban fontos a warm-up és a többszöri futtatás.

Az L1 találati arányt az `l1tex__t_hit_rate` vagy hasonló metrika adja, az L2-t pedig az `lts__t_hit_rate`. A pontos neveket a `ncu --query-metrics | grep -i "hit_rate"` paranccsal lehet lekérdezni.

11.4.9 7. Gyakori hibák és értelmezésük

A memória-hozzáférési minták mérésekor több tipikus hiba csapdába ejtheti a fejlesztőt:

1. **Hideg cache mérése warm-up nélkül.** Az első kernelhívásnál a cache-ek üresek, így a mért L1/L2 találati arány irreálisan alacsony, és a DRAM-forgalom nagyobb, mint a steady-state. Mindig iktassunk be legalább egy warm-up futtatást, és a méréshez használjunk többszöri ismétlést.

2. **Túl kicsi munkahalmaz.** Ha a bemeneti tömb belefér az L2 cache-be, akkor a második futtatástól kezdve az összes adat az L2-ben lesz, és a mérés nem tükrözi a valódi DRAM-sávszélesség-igényt. Ezért a munkahalmaz méretét úgy kell megválasztani, hogy legalább 2-3-szorosa legyen az L2 cache-nek. Az A100 40 MiB L2-vel rendelkezik, ezért legalább 128 MiB adatot érdemes használni, ha a DRAM-forgalmat is mérni akarjuk.

3. **Alacsony `gld_efficiency`, de nem a coalescing miatt.** Előfordulhat, hogy a kernelben a szálak folytonosan olvasnak, mégis alacsony a hatékonyság. Ennek oka lehet például, hogy a hozzáférés nem igazodik a 32 bájtos szektorhatárokhoz (misaligned), vagy hogy a warp-on belül néhány szál inaktív (divergencia miatt), ami pazarlást okoz. Ilyenkor a `gld_efficiency` mellett érdemes megnézni a `smsp__sass_average_data_bytes_per_sector_mem_global_op_ld.ratio` metrikát is, amely megmutatja az átlagosan felhasznált bájtok számát szektoronként.

4. **A store efficiency figyelmen kívül hagyása.** Sok kernel ír is, nem csak olvas. A `gst_efficiency` hasonlóan fontos, és gyakran eltér a load hatékonyságtól, különösen, ha a kimeneti hozzáférési minta eltérő, vagy ha a szálak részlegesen írnak (pl. csak néhány bájtot egy szektorból).

5. **A grid-/blockméret hatása.** Ha túl kevés szál indul, a hardver nem tudja elrejteni a memória-késleltetést, és a mért hatékonyság romolhat, mert a warp-ok nem tudnak átfedésben dolgozni. Mindig ellenőrizzük, hogy a grid mérete legalább annyi blokkot tartalmazzon, hogy minden SM-re jusson több aktív blokk.

6. **A metrikák félreértelmezése.** A `gld_efficiency` 100% nem jelenti azt, hogy a kernel gyors; csak azt, hogy a memória-hozzáférés nem pazarol sávszélességet. A teljesítményt más tényezők (pl. számítási korlát, késleltetés, occupancy) is befolyásolják. A profilozást mindig komplexen kell végezni.

11.4.10 8. Laborfeladat: Nsight Compute memória-metrikák elemzése

Cél: A fenti példakód alapján végezz saját mérést, és elemezd a kapott metrikákat.

Környezet: Rögzítsd a GPU modellt, drivert, CUDA-verziót, operációs rendszert, és a Nsight Compute verzióját. Győződj meg arról, hogy nincs más GPU-folyamat.

Lépések:

1. Fordítsd le a `kernel_mem_access.cu` programot a saját GPU-dnak megfelelő `-arch` kapcsolóval (pl. `sm_80` A100-hoz, `sm_86` RTX 3090-hez, `sm_89` RTX 4090-hez).
2. Futtasd le egyszer a programot a helyes működés ellenőrzésére.
3. Válassz ki legalább három kernelt (pl. `load_coalesced`, `load_strided` stride=4, `load_random`).
4. Mindegyik kernelre futtasd a Nsight Compute-ot a következő beállításokkal:
   - `--launch-skip 1` (a warm-up kihagyása)
   - `--launch-count 10` (10 mérés átlaga)
   - `--metrics gld_efficiency,gst_efficiency,l1tex__t_hit_rate,lts__t_hit_rate`
5. Rögzítsd a mérési naplóban a kapott értékeket, az átlagot és a szórást.
6. Hasonlítsd össze a mért `gld_efficiency` értékeket az elméleti várakozásokkal (25%, 100%, 12.5% stb.). Magyarázd meg az esetleges eltéréseket.
7. Vizsgáld meg az L1 és L2 találati arányokat. Változtasd meg a munkahalmaz méretét (pl. `N = 1 << 24`) és figyeld meg, hogyan változik a cache-találati arány.
8. Készíts egy rövid jelentést, amelyben a mért és a várt eredményeket külön táblázatban adod meg, és indoklást fűzöl hozzá.

Átadandó eredmény: A mérési napló (parancsok, környezet, értékek), az elemzés és a következtetések. A mérési naplóban tüntesd fel, hogy az értékek a saját mérésedből származnak, és ne keverd őket az elméleti várakozásokkal.

11.4.11 9. Önellenőrző kérdés

Egy kernel minden szál egyetlen float-ot olvas egy folytonos tömbből, a hozzáférési minta tökéletesen coalescing. Ennek ellenére a Nsight Compute 60%-os `gld_efficiency` értéket mutat. Sorolj fel legalább két lehetséges okot, és magyarázd el, hogyan döntenéd el méréssel, melyik áll fenn.

11.4.12 10. Összegzés és továbbvezetés

Ebben az alfejezetben megtanultuk, hogyan mérhetjük a memória-hozzáférési minták hatékonyságát a Nsight Compute segítségével. A `gld_efficiency` és `gst_efficiency` metrikák közvetlenül jelzik a coalescing minőségét, míg az L1/L2 találati arányok a cache-hierarchia kihasználtságát mutatják. A mérési módszertan – warm-up, többszöri futtatás, megfelelő munkahalmaz – elengedhetetlen a megbízható eredményekhez. A következő alfejezet (Occupancy és erőforrás-korlátok) az occupancy és a register pressure témakörét tárgyalja, amelyre ezek a mérési ismeretek közvetlenül építenek, hiszen a memória-hatékonyság és az occupancy együtt határozza meg a kernel teljesítményét.

12 Occupancy, register pressure és latency hiding

12.1 Occupancy és erőforrás-korlátok

A GPU-végrehajtás egyik legfontosabb fogalma az **occupancy**, mert ezen keresztül kapcsolódnak egymáshoz a kernel erőforrásigényei és az SM-ek hardveres korlátai. Az occupancy önmagában nem teljesítménycél, hanem egy diagnosztikai mutató: megmutatja, hogy az adott launch konfiguráció és regiszterhasználat mellett mennyire tudjuk feltölteni az SM-eket aktív warpekkel. Ebben az alfejezetben felépítjük az occupancy számítását, megvizsgáljuk a három fő erőforrás-korlátot, bemutatjuk a CUDA occupancy API-ját, majd egy teljes laboron keresztül elválasztjuk a várt és a mért occupancy-t.

12.1.1 Az occupancy fogalma

A GPU ütemezője a kernel blokkjait SM-ekre osztja. Egy SM egyszerre több blokkot is futtathat, ha a hardveres erőforrásai engedik: elegendő regiszter, elegendő shared memory, és van szabad blokkhely. Minden blokk egy vagy több warp formájában foglalja el az SM végrehajtási erőforrásait. Az SM-en egyszerre jelen lévő, úgynevezett **aktív warpek** száma határozza meg, hogy a hardver mennyire tudja elrejteni a memória- és utasítás-késleltetéseket.

Az occupancy formálisan:

$$O = \frac{W_{\text{aktív}}}{W_{\text{max}}},$$

ahol $W_{\text{aktív}}$ az SM-en egyidejűleg aktív warpek száma, W_{max} pedig az SM által maximálisan támogatott warpek száma. Az értéket gyakran százalékban adjuk meg:

$$O_{\%} = 100 \cdot \frac{W_{\text{aktív}}}{W_{\text{max}}}.$$

Ha egy SM legfeljebb 64 warpet tud futtatni, és a kernelünkből 32 warp aktív, akkor az occupancy 50%. Ha 64 warp aktív, akkor 100%.

Az occupancy-nak kétféle értékét érdemes megkülönböztetni:

- **Elméleti vagy várt occupancy:** az erőforrás-korlátokból és a kernel statikus jellemzőiből, például a regiszterhasználatból és a shared memory igényből számított maximális érték.
- **Mért occupancy:** a futó kernel tényleges SM-foglaltsága, amelyet profilozóval, például Nsight Compute-tal figyelhetünk meg.

A kettő általában közel van egymáshoz, de nem mindig azonos. A mért értéket befolyásolhatja a grid mérete, a hullámok száma, az aszinkron végrehajtás, a hőmérséklet és a profilozó mintavételezése. Ezt a labornál is látni fogjuk.

12.1.2 Az SM erőforrás-korlátai

Az occupancy-t négy fő hardveres korlát alakítja:

1. **Warp-limit:** az SM maximális warp-száma, amely a maximális thread-számból következik:

$$W_{\max} = \frac{T_{\max}}{32},$$

   ahol $T_{\max}$ az SM-en maximálisan aktív thread-ek száma.

2. **Block slot limit:** az SM-en maximálisan futtatható blokkok száma, függetlenül a blokkok méretétől. Ez egy eszköz-specifikus hardverkorlát.

3. **Regiszterfájl-limit:** minden SM rendelkezik egy fix méretű regiszterfájllal. Ha egy szál R_t regisztert használ, és egy blokkban T_b szál van, akkor egy blokk regiszterigénye:

$$R_b = R_t \cdot T_b.$$

   Az SM regiszterfájlából legfeljebb:

$$B_R = \left\lfloor \frac{R_{\max}}{R_b} \right\rfloor$$

   blokk fér el.

4. **Shared memory limit:** az SM shared memory kapacitása szintén korlátozza a blokkok számát. Ha egy blokk S_b bájt shared memory-t igényel, akkor:

$$B_S = \left\lfloor \frac{S_{\max}}{S_b} \right\rfloor$$

   blokk fér el, ahol $S_{\max}$ az SM maximális shared memory mérete.

A tényleges blokkszám az SM-en ezek minimuma, kiegészítve a thread-limitből adódó korláttal:

$$B = \min\left(\left\lfloor \frac{T_{\max}}{T_b} \right\rfloor,\ B_R,\ B_S,\ B_{\text{slot}} \right).$$

A blokkonkénti warpszám:

$$W_b = \left\lceil \frac{T_b}{32} \right\rceil.$$

Az occupancy pedig:

$$O = \frac{B \cdot W_b}{W_{\max}}.$$

Ez a képlet jól mutatja, hogy a magas occupancy-hoz **egyszerre** kell elegendő thread, kevés regiszter, kevés shared memory és megfelelő block méret.

A gyakorlatban a regiszter-allokáció nem pontosan ilyen egyszerű. A hardver a regisztereket warp- vagy blokk-szintű granularitással foglalja le, és a shared memory-ból is van fenntartott rész. Ezért a kézi képlet jó becslés, de a pontos értékhez az occupancy API-t kell használni.

12.1.3 Számítási példa

Nézzünk egy didaktikus SM-et a következő erőforrásokkal:

- $T_{\max} = 2048$ thread, tehát $W_{\max} = 64$ warp,
- $B_{\text{slot}} = 32$ blokk,
- $R_{\max} = 65536$ regiszter,
- $S_{\max} = 100$ KiB shared memory.

Vegyünk egy kernelt, amely `blockDim.x = 256`, regiszterhasználata szálanként 64, shared memory igénye pedig 0.

A blokkszám-korlátok:

$$B_T = \left\lfloor \frac{2048}{256} \right\rfloor = 8,$$

$$B_R = \left\lfloor \frac{65536}{64 \cdot 256} \right\rfloor = \left\lfloor \frac{65536}{16384} \right\rfloor = 4,$$

$$B_S = \infty,$$

$$B_{\text{slot}} = 32.$$

Tehát:

$$B = \min(8, 4, \infty, 32) = 4.$$

A blokkonkénti warpszám:

$$W_b = \frac{256}{32} = 8.$$

Az occupancy:

$$O = \frac{4 \cdot 8}{64} = \frac{32}{64} = 0{,}5,$$

azaz 50%.

Érdemes megfigyelni, hogy a regiszterkorlát dominál. Ha a regiszterhasználatot 32-re csökkentenénk, akkor:

$$B_R = \left\lfloor \frac{65536}{32 \cdot 256} \right\rfloor = 8,$$

így $B = 8$, és az occupancy:

$$O = \frac{8 \cdot 8}{64} = 100\%.$$

Ugyanez a kernel 128-as blokkmérettel is 50%-os occupancy-t adna, ha a regiszterhasználat szálanként 64, mert a regiszterkorlát miatt 1024 szál fér csak az SM-re, ami 32 warp. A block méret tehát nem mindig változtatja az occupancy-t, ha a regiszter- vagy shared memory limit a meghatározó.

Nézzünk egy másik tipikus esetet. Legyen a blokkméret 32, a regiszterhasználat pedig alacsony, például 14 szálanként. Ekkor:

$$B_T = \left\lfloor \frac{2048}{32} \right\rfloor = 64,$$

de a block slot limit miatt:

$$B = \min(64, B_R, \infty, 32) = 32.$$

A blokkonkénti warpszám $W_b = 1$, így:

$$O = \frac{32 \cdot 1}{64} = 50\%.$$

Hiába van rengeteg thread és kevés regiszter, a túl kicsi blokkméret a block slot limit miatt nem engedi elérni a 100%-ot. Ezért fontos a blokkméretet nem csak a memória-hozzáférési mintákhoz, hanem az occupancy-hoz is igazítani.

Végül nézzünk egy shared memory által korlátozott esetet:

- blokkméret 256,
- regiszterhasználat 32,
- shared memory igény 40 KiB blokkonként.

A thread-limit 8, a regiszterlimit 8, a shared limit:

$$B_S = \left\lfloor \frac{100}{40} \right\rfloor = 2.$$

Így:

$$B = \min(8, 8, 2, 32) = 2,$$

és:

$$O = \frac{2 \cdot 8}{64} = 25\%.$$

A nagy shared memory igény drasztikusan csökkenti az occupancy-t. Ez nem feltétlenül hiba, mert a shared memory tiling gyakran akkor is megéri, ha az occupancy alacsonyabb, de a döntést méréssel kell alátámasztani.

12.1.4 Gyakori hibák

Az occupancy számítása közben több tipikus tévedés is előfordul.

A block méret nem 32 többszöröse.
A CUDA végrehajtási egysége a warp, amely 32 szálból áll. Ha a blokkméret például 100, akkor a hardver 4 warpet foglal le, azaz 128 szálnyi helyet, miközben 28 szál helye kihasználatlan. Az occupancy képlete ezt úgy kezeli, hogy $W_b = \lceil 100/32 \rceil = 4$, vagyis a számlálóban a pazarolt laneek is aktív warpként jelennek meg. A kihasználatlanság tehát nem feltétlenül látszik az occupancy százalékban, de a valós munkavégzést rontja. Mindig 32 többszörösét érdemes választani.

Túl kicsi blokkméret.
Ahogy a fenti példa mutatta, 32-es blokkméretnél a block slot limit miatt könnyen 50% alatt ragadhat az occupancy, még akkor is, ha a regiszterek és a shared memory bőségesek. A 64-es vagy 128-as blokkméret gyakran jobb kiindulópont.

Túl nagy shared memory igény.
Ha egy blokk 40 KiB shared memory-t foglal, egy 100 KiB-os SM-en csak két blokk fér el. Ez 25%-os occupancy-t eredményezhet, ami nem feltétlenül baj, de ha a kernel nem használja hatékonyan a shared memory-t, akkor indokolatlanul korlátozza a párhuzamosságot.

A regiszterhasználat változását figyelmen kívül hagyjuk.
A regiszterhasználat a fordítás során változhat attól függően, hogy milyen optimalizálási szintet használunk. Ha a kernelünket átírjuk, vagy a `-O3` szintet megváltoztatjuk, az occupancy is megváltozhat. Mindig a tényleges binárisra vonatkozó `cudaFuncGetAttributes` értéket használjuk.

Az occupancy-t önmagában célként kezeljük.
A magas occupancy segíthet a késleltetés elrejtésében, de nem garantálja a jobb teljesítményt. A következő fejezetekben látni fogjuk, hogy a magas regiszterhasználat, a spill és a túlzott shared memory használat néha alacsonyabb occupancy mellett is gyorsabb lehet. Az occupancy egy diagnosztikai eszköz, nem egy optimalizálási célfüggvény.

12.1.5 Az occupancy API

A CUDA runtime API több függvényt biztosít az occupancy pontos kiszámítására. A leggyakrabban használt:

```
cudaOccupancyMaxActiveBlocksPerMultiprocessor(
    &numBlocks,          // kimenő: aktív blokkok száma SM-enként
    kernelFunction,      // kernel függvény mutatója
    blockSize,           // tervezett blokkméret
    dynamicSMemSize      // dinamikus shared memory bájtban
);
```

A függvény visszaadja, hogy az adott kernel, blokkméret és dinamikus shared memory mellett hány blokk lehet aktív egy SM-en. Ebből az occupancy:

```
int warpsPerBlock = (blockSize + 31) / 32;
int maxWarpsPerSM = props.maxThreadsPerMultiProcessor / 32;
float occupancy = 100.0f * numBlocks * warpsPerBlock / maxWarpsPerSM;
```

A `dynamicSMemSize` paraméter fontos: ha a kernel dinamikus shared memory-t használ, akkor pontosan azt az értéket kell megadni, amelyet a kernel indításánál is használunk. Ha itt 0-t adunk meg, de a kernel 20 KiB dinamikus shared memory-t foglal, akkor az API hamis, túl magas occupancy-t fog mutatni.

A másik fontos függvény a blokkméret keresésére:

```
cudaOccupancyMaxPotentialBlockSize(
    &minGridSize,       // kimenő: minimális grid méret
    &blockSize,         // kimenő: ajánlott blokkméret
    kernelFunction,
    dynamicSMemSize,
    blockSizeLimit      // felső korlát a blokkméretre
);
```

Ez a függvény az adott kernelhez és erőforrásokhoz egy olyan blokkméretet javasol, amely magas occupancy-t tesz lehetővé. A `blockSizeLimit` jellemzően 1024, mert ez a maximális blokkméret a legtöbb CUDA eszközön.

A kernel statikus jellemzőit, például a regiszterhasználatot és a statikus shared memory igényt a `cudaFuncGetAttributes` függvénnyel kérdezhetjük le:

```
cudaFuncAttributes attr;
cudaFuncGetAttributes(&attr, kernelFunction);
// attr.numRegs          -> regiszter/thread
// attr.sharedSizeBytes  -> statikus shared memory bájtonként
```

A kézi becsléshez ezekre az értékekre van szükség. Az occupancy API azonban ennél pontosabb, mert figyelembe veszi a hardveres allokációs granularitást és a fenntartott erőforrásokat is.

12.1.6 Labor: occupancy-számítás egy kernelre

A labor célja, hogy egy konkrét kernelre kiszámítsuk a várt occupancy-t az occupancy API segítségével, majd ugyanezt Nsight Compute-tal megmérjük. A kód egy egyszerű memória-író kernelt használ, hogy a hangsúly ne a számítási optimalizáláson, hanem az occupancy eszközökön legyen.

Keresett eredmények:

- az API által várt occupancy különböző blokkméretekre,
- a kernel regiszterhasználata,
- a mért occupancy legalább egy launch konfigurációra,
- annak megállapítása, hogy a grid mérete elegendően nagy-e.

Képzeletbeli kiinduló hardver:
A labor rögzítse a helyi GPU-t. Példaként egy NVIDIA GeForce RTX 3080-at írunk, de a kód bármely CUDA-képes eszközön lefut.

A következő CUDA C++ program kiírja az eszköz legfontosabb SM-erőforrásait, lefuttatja a kernelt, majd blokkméretenként kiszámítja az API alapú occupancy-t. A kézi becslést is megjeleníti, hogy lássuk a közelítés és az API közötti esetleges eltérést.

```
// CUDA C++17, nvcc
// occupancy_lab.cu

#include <cuda_runtime.h>
#include <algorithm>
#include <cstdlib>
#include <iomanip>
#include <iostream>
#include <climits>

#define CUDA_CHECK(call)                                        \
    do {                                                        \
```

```
        cudaError_t err = (call);                                   \
        if (err != cudaSuccess) {                                   \
            std::cerr << "CUDA error at " << __FILE__ << ":"       \
                      << __LINE__ << " -> "                         \
                      << cudaGetErrorString(err) << "\n";           \
            std::exit(EXIT_FAILURE);                                \
        }                                                           \
    } while (0)

__global__ void sampleKernel(float *out, int n)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        out[i] = fmaf(out[i], 1.001f, 0.5f);
    }
}

int main()
{
    int device = 0;
    cudaDeviceProp prop;
    CUDA_CHECK(cudaGetDeviceProperties(&prop, device));

    cudaFuncAttributes attr;
    CUDA_CHECK(cudaFuncGetAttributes(&attr, sampleKernel));

    int maxWarpsPerSM = prop.maxThreadsPerMultiProcessor / 32;

    int maxBlocksPerSM = 0;
    CUDA_CHECK(cudaDeviceGetAttribute(
        &maxBlocksPerSM,
        cudaDevAttrMaxBlocksPerMultiprocessor,
        device));

    // Kis futtatás, hogy a kernel valóban elinduljon.
    const int n = 1 << 20;
    float *d_out = nullptr;
    CUDA_CHECK(cudaMalloc(&d_out, n * sizeof(float)));
    CUDA_CHECK(cudaMemset(d_out, 0, n * sizeof(float)));

    const int launchBlockSize = 256;
    const int gridSize = (n + launchBlockSize - 1) / launchBlockSize;
    sampleKernel<<<gridSize, launchBlockSize, 0, 0>>>(d_out, n);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    std::cout << "GPU: " << prop.name << "\n";
    std::cout << "Compute capability: " << prop.major << "."
              << prop.minor << "\n";
    std::cout << "Max threads/SM: " << prop.maxThreadsPerMultiProcessor << "\n";
    std::cout << "Max warps/SM: " << maxWarpsPerSM << "\n";
    std::cout << "Registers/SM: " << prop.regsPerMultiprocessor << "\n";
    std::cout << "Shared memory/SM: " << prop.sharedMemPerMultiprocessor
              << " bytes\n";
    std::cout << "Max blocks/SM (attribute): " << maxBlocksPerSM << "\n";
    std::cout << "Sample kernel registers/thread: " << attr.numRegs << "\n";
    std::cout << "Sample kernel static shared: " << attr.sharedSizeBytes
              << " bytes\n\n";

    const int blockSizes[] = {32, 64, 128, 256, 512, 1024};

    std::cout << std::left
              << std::setw(12) << "Block size"
```

```
              << std::setw(12) << "API blocks"
              << std::setw(12) << "API warps"
              << std::setw(14) << "Warp/block"
              << std::setw(14) << "Vart occ."
              << std::setw(14) << "Kez. becsles"
              << "\n";

    for (int bs : blockSizes) {
        int apiBlocks = 0;
        CUDA_CHECK(cudaOccupancyMaxActiveBlocksPerMultiprocessor(
            &apiBlocks, sampleKernel, bs, 0));

        int warpsPerBlock = (bs + 31) / 32;
        int apiWarps = apiBlocks * warpsPerBlock;
        float apiOccupancy = 100.0f * apiWarps / maxWarpsPerSM;

        int blocksByThreads = prop.maxThreadsPerMultiProcessor / bs;
        int blocksByRegs = attr.numRegs > 0
            ? prop.regsPerMultiprocessor / (attr.numRegs * bs)
            : INT_MAX;
        int blocksByShared = attr.sharedSizeBytes > 0
            ? prop.sharedMemPerMultiprocessor / attr.sharedSizeBytes
            : INT_MAX;

        int estBlocks = std::min({blocksByThreads,
                                  blocksByRegs,
                                  blocksByShared,
                                  maxBlocksPerSM});
        int estWarps = estBlocks * warpsPerBlock;
        float estOccupancy = 100.0f * estWarps / maxWarpsPerSM;

        std::cout << std::left
                  << std::setw(12) << bs
                  << std::setw(12) << apiBlocks
                  << std::setw(12) << apiWarps
                  << std::setw(14) << warpsPerBlock
                  << std::setw(13) << std::fixed << std::setprecision(1)
                  << apiOccupancy << " %"
                  << std::setw(14) << std::fixed << std::setprecision(1)
                  << estOccupancy << " %"
                  << "\n";
    }

    CUDA_CHECK(cudaFree(d_out));
    return 0;
}
```

A kódban a kernel indítása a `<<<grid, block, shared_bytes, stream>>>` alakot követi: a shared memory 0, a stream pedig a default stream, azaz 0. A `cudaGetLastError` és a `cudaDeviceSynchronize` együttesen biztosítja, hogy a kernelindítás hibáját és az aszinkron végrehajtás hibáját is ellenőrizzük.

A kézi becslés szándékosan egyszerűsített. Nem veszi figyelembe a warp-szintű regiszterallokációt és a fenntartott shared memory-t, ezért az API-értéknél kissé eltérhet. A táblázatban ezért két oszlop szerepel: az API által számított várt érték és a kézi becslés.

Build és futtatás:

```
nvcc -O3 -arch=native -o occupancy_lab occupancy_lab.cu
./occupancy_lab
```

Az `-arch=native` helyett használhatunk konkrét compute capability-t is, például `-arch=sm_86` egy RTX 3080 esetén. A labor jegyzőkönyvben rögzítsük a tényleges értéket.

Nsight Compute mérés:

A várt occupancy-t az API-ból kaptuk. A mért occupancy-t Nsight Compute-tal olvashatjuk ki. Egy irányadó parancssor:

```
ncu --kernel-name sampleKernel \
    --launch-count 10 \
    --metrics sm__warps_active.avg.pct_of_peak_sustained_active \
    ./occupancy_lab
```

A metrika neve az Nsight Compute verziójától függően változhat, ezért a labor során ellenőrizzük a `--query-metrics` kimenetét. A rövid kernel miatt érdemes több indítást rögzíteni, és az átlagot, valamint a szórást is feljegyezni.

12.1.7 Mérési napló és értelmezés

A mérési naplóban a várt és a mért értékeket külön kell szerepeltetni. A következő táblázat egy **példa**; a konkrét értékeket a helyi környezetben kell reprodukálni. A könyvben szereplő értékek nem tekintendők ellenőrzött mérésnek, hanem a labor jegyzőkönyv szerkezetét mutatják.

Példa mérési napló

Bejegyzés	Érték
GPU	NVIDIA GeForce RTX 3080 (példa)
Compute capability	8.6 (példa)
Driver	535.104.05 (példa, verifikálandó)
CUDA toolkit	12.2 (példa, verifikálandó)
Kernel regiszter/thread	14 (példa)
Statikus shared / block	0 byte
Dinamikus shared / block	0 byte
Grid méret	4096 blokk
SM-ek száma	68 (példa)
Max warps/SM	48 (példa)
Max blocks/SM	16 (példa)

Block size	API blocks/SM	API warps/SM	Várt occupancy	Mért Nsight occupancy
32	16	16	33,3 %	31,9 % (példa)
64	16	32	66,7 %	65,3 % (példa)
128	12	48	100,0 %	98,2 % (példa)
256	6	48	100,0 %	97,8 % (példa)
512	3	48	100,0 %	97,5 % (példa)
1024	1	32	66,7 %	64,9 % (példa)

A táblázatban a **várt occupancy** az API által számított érték, a **mért Nsight occupancy** pedig a profilozó által megfigyelt aktív warp arány. A kettő közötti kis eltérés normális, mert a Nsight mintavételez, a kernel rövid futása alatt a felfutási idő is belejátszhat az átlagba.

A példa adataiból látszik, hogy a 32-es blokkméretnél a block slot limit miatt csak 16 blokk fér az SM-re, ami 16 warp, vagyis 33,3% occupancy. A 64-es blokkméretnél 16 blokk és 32 warp, ami 66,7%. A 128-as, 256-os és 512-es blokkméreteknél a thread-limit már elegendő warp-et enged a 100%-hoz, ezért az API szerint teljes az occupancy. Az 1024-es blokkméretnél a thread-limit miatt csak 1 blokk fér el, ami 32 warp, így az occupancy ismét 66,7%-ra esik vissza.

A labor elvégzésekor a következőket ellenőrizzük:

1. A grid mérete legalább akkora legyen, hogy az összes SM-et feltöltse:
   `grid >= SM_szám * API_blocks_per_SM`.
   Ha a grid túl kicsi, a mért occupancy alacsonyabb lesz a vártnál, mert a kernel nem indít elég blokkot.
2. A mérés előtt warm-up futtatás szükséges, majd legalább 5–10 ismétlés. Az átlag mellett a szórást vagy percentilist is rögzítsük.
3. A Nsight metrika nevét a helyi verzióban ellenőrizzük, mert az Nsight Compute kiadások között változhat.
4. A várt és a mért értéket mindig külön oszlopban szerepeltessük, és tüntessük fel, hogy a mérés melyik kernelre, precisionre és launch konfigurációra vonatkozik.

12.1.8 Önellenőrző kérdés

Egy SM erőforrásai: 64 maximális warp, 32 maximális blokk, 65536 regiszter, 100 KiB shared memory. Egy kernel blokkmérete 256, regiszterhasználata 64 szálanként, shared memory igénye 0. Mekkora az occupancy?

A válasz:

$$B_T = \left\lfloor \frac{2048}{256} \right\rfloor = 8, \quad B_R = \left\lfloor \frac{65536}{64 \cdot 256} \right\rfloor = 4,$$

$$B = \min(8, 4, \infty, 32) = 4, \quad W_b = 8,$$

$$O = \frac{4 \cdot 8}{64} = 0{,}5 = 50\%.$$

A válasz tehát 50%. Ha a blokkméret 32 lenne, akkor a block slot limit miatt szintén 50% lenne az occupancy, de ott a warp-limit kihasználtsága más okból korlátozódna.

12.2 Regiszterhasználat és spill

A regiszterek a GPU leggyorsabb, de egyben legszűkösebb erőforrásai közé tartoznak. Az előző alfejezetben láttuk, hogy az occupancy – vagyis az egy streaming multiprocesszorra (SM) jutó aktív warpok száma – közvetlenül függ attól, hogy egy szál hány regisztert foglal le. Ebben az alfejezetben azt vizsgáljuk meg, mi történik, ha a kernel több regisztert igényel, mint amennyit egy szál maximálisan megkaphat: megjelenik a *local memory spill*, amely a regiszterek tartalmát a globális memóriába (pontosabban a local memory háttértárba) kényszeríti. A spill nem csupán egy fordítói statisztika, hanem gyakran az egyik legfontosabb rejtett teljesítménycsapda. Megtanuljuk, hogyan mérhető a regiszterhasználat, hogyan szabályozható a `maxrregcount` és a

`__launch_bounds__` eszközökkel, és hogyan különíthető el a mért eredmény a modell alapján várható viselkedéstől.

12.2.1 Regiszterek és a regiszterfájl

Minden SM rendelkezik egy nagy, nagysebességű, de véges méretű regiszterfájllal. A regiszterek a szálak privát állapotát tárolják: skalár változókat, részkifejezéseket, ciklusváltozókat és a fordító által generált ideiglenes értékeket. A regiszterekhez való hozzáférés egyetlen órajel alatt, teljes sávszélességgel történik; nincs cache késleltetés, nincs memória-címzés. A szálak közötti adatcsere csak akkor lehetséges, ha az adatot explicit módon átmásoljuk egy megosztott vagy globális memóriába; a regiszterek soha nem láthatók más szál számára.

A regiszterfájl mérete SM-enként architektúrafüggő. Például az NVIDIA Ampere (sm_80) architektúrában egy SM 64 768 db 32 bites regiszterrel rendelkezik, ami 256 KiB-nek felel meg. Ha egy kernel szálanként 64 regisztert használ, és egy blokk 256 szálat tartalmaz, akkor egy blokk $64 \times 256 = 16\,384$ regisztert foglal le. Egy ilyen SM-en legfeljebb $64\,768 / 16\,384 = 3$ blokk lehet egyszerre aktív, ami 768 szálnak felel meg. Ha a maximális szálszám SM-enként 2048, akkor az occupancy ebben az esetben $768 / 2048 = 37{,}5\%$. A regiszterhasználat tehát közvetlenül korlátozza az occupancy-t, ahogyan azt az Occupancy és erőforrás-korlátok alfejezetben részletesen tárgyaltuk.

A fordító a kernel fordításakor dönt arról, hogy az egyes változókat regiszterben tartja-e, vagy a *local memory* nevű, valójában a globális memória egy szálanként elkülönített szeletébe helyezi. A döntést két tényező befolyásolja: a kernelben ténylegesen használt változók száma és azok élettartama, valamint a szálankénti regiszterkorlát, amelyet a fejlesztő közvetve befolyásolhat.

12.2.2 Mi a local memory spill?

A *local memory* a GPU memóriamodelljében az a memóriatér, amely a szál privát adatait tárolja, ha azok nem férnek el a regiszterekben. Fontos hangsúlyozni, hogy a local memory fizikailag a globális memóriában (DRAM) van, ugyanazon a memóriavezérlőn keresztül érhető el, mint a `__global__` változók. Az egyetlen különbség, hogy a local memory címe szálanként és blokkonként el van szeparálva, és a hozzáférés a szál privát címzési tartományán belül történik. A local memory hozzáférések ugyanúgy átmennek az L1 és L2 cache-en, mint a globális memóriahozzáférések, de a késleltetésük több száz órajel is lehet, összehasonlítva a regiszterek egy órajeles hozzáférésével.

A *spill* az a folyamat, amikor a fordító egy regiszterben élő változót kénytelen a local memoryba kiírni (spill store), majd később onnan visszaolvasni (spill load). Ez akkor következik be, ha a kernel regiszterigénye meghaladja a szálanként elérhető regiszterek számát. A fordító ilyenkor a legkevésbé gyakran használt változókat helyezi ki a regiszterfájlból, hogy a legforróbb értékek regiszterben maradhassanak.

A spill-t a ptxas (a CUDA PTX assembler és optimalizáló) a fordítás során jelzi. A `nvcc -Xptxas -v` kapcsolójával megjeleníthető a regiszterhasználat, a spill store/load bájtok mennyisége, valamint a stack frame mérete. Egy tipikus kimenet:

```
ptxas info    : Compiling entry function '_Z13kernel_spillPfS_' for 'sm_80'
ptxas info    : Function properties for _Z13kernel_spillPfS_
ptxas         .reg     .f32    64
ptxas         .reg     .b32    16
ptxas info    : Used 80 registers, 128 bytes spill stores, 256 bytes spill loads
```

Ez a kimenet azt jelenti, hogy a kernel szálanként 80 regisztert használ, 128 bájtot ír ki a local memoryba (spill store), és 256 bájtot olvas be onnan (spill load). A spill mennyiségét bájtban adjuk meg, mert egy-egy kiírás vagy beolvasás több 32 bites szót is érinthet.

A spill nem feltétlenül katasztrófa. Ha a kiírt és visszaolvasott változók ritkán használtak, és a kernel egyébként sok számítást végez, a teljesítményhatás elhanyagolható lehet. A gyakorlatban azonban a spill gyakran a regiszterhiány tünete, és a teljesítmény romlásához vezet: több utasítás, nagyobb memóriaforgalom, alacsonyabb occupancy.

12.2.3 Regiszterhasználat mérése

A regiszterhasználat és a spill mértékét több módon is meg lehet határozni:

1. **Fordítói statisztika:** A fent bemutatott `nvcc -Xptxas -v` a legegyszerűbb eszköz. Nem igényel futtatást, csak fordítást, és megadja a regiszterek számát, a spill bájtokat és a stack frame méretét.
2. **Nsight Compute:** Az NVIDIA profilerének kernel-szintű metrikái – például `launch__registers_per_thread`, `l1tex__t_bytes_pipe_lsu_mem_local_op_ld.sum`, `l1tex__t_bytes_pipe_lsu_mem_local_op_st.sum`, `smsp__sass_inst_executed_op_local_ld.sum` – részletesen mutatják a regiszterhasználatot és a tényleges local memory forgalmat a futás során. Az Nsight Compute a legmegbízhatóbb forrás, mert a hardveres számlálókból dolgozik, és figyelembe veszi a tényleges végrehajtási útvonalat is.
3. **Occupancy kalkulátor:** Az NVIDIA által biztosított Excel-alapú kalkulátor vagy a `cudaOccupancyMaxActiveBlocksPerMultiprocessor` API segítségével a regiszterhasználat és a blokkméret alapján kiszámítható az elméleti occupancy. Ez a tervezési fázisban hasznos, mielőtt a kernelt lefuttatnánk.

A mérés során fontos különbséget tenni a *statikus regiszterhasználat* (amit a ptxas jelent) és a *ténylegesen lefoglalt regiszterek* között. A statikus érték a maximális regiszterigényt mutatja, de ha a kernelben vannak ritkán végrehajtott ágak, előfordulhat, hogy a hardver a teljes regiszterfájl-szeletet lefoglalja, függetlenül attól, hogy az adott ág fut-e. A tényleges regiszterhasználat azonban nem csökken attól, hogy egy ág nem fut, mert a hardver a legrosszabb esetre méretezi a szál állapotát.

12.2.4 A spill hatása a teljesítményre

A spill-nek három fő negatív hatása van:

1. **Többlet utasítások:** Minden kiírás és visszaolvasás egy-egy extra memória-utasítást jelent (STL, LDL a SASS szinten). Ha a spill egy forró ciklusban történik, az utasításszám jelentősen megnőhet.
2. **Memóriaforgalom és késleltetés:** A local memory hozzáférések a globális memórián keresztül mennek, tehát a késleltetésük több száz órajel. Ha a spill változó egy szorosan egymás utáni kódban szerepel, a szál várakozni kényszerül, hacsak a warp ütemező nem tud másik warpot futtatni.
3. **Occupancy csökkenés:** A spill nem közvetlenül csökkenti az occupancy-t, de a magas regiszterhasználat – amely a spill kiváltó oka – már önmagában korlátozza az aktív warpok számát. Ha a regiszterhasználatot csökkentjük, több warp válhat aktívvá, ami segíthet elrejteni a memória-késleltetést. Ezért a spill és az occupancy gyakran együtt jár, és a megoldás is közös.

A spill hatásának számszerűsítésére a legjobb módszer egy kontrollált kísérlet: ugyanazt a számítást végezzük el olyan kernelverzióval, amely spill-t okoz, és olyannal, amely nem. A két verzió közötti futásidő-különbség, valamint a ptxas és Nsight Compute metrikák együttesen megmutatják a spill valódi költségét.

12.2.5 Szabályozás: `maxrregcount` és `__launch_bounds__`

A CUDA két fő eszközt kínál a regiszterhasználat befolyásolására: a `maxrregcount` fordítói opciót és a `__launch_bounds__` kernel-attribútumot. Mindkettő azt a célt szolgálja, hogy a fejlesztő korlátozza a szálankénti regiszterek számát, ezáltal növelve az occupancy-t – de mindkettőnek ára lehet: ha a korlát túl szigorú, a fordító több spill-t generál, ami lassíthatja a kernelt.

12.2.6 `maxrregcount`

Az `nvcc --maxrregcount N` opciója globálisan, az egész fordítási egységre érvényesen korlátozza a regiszterek számát `N`-re. Például:

```
nvcc -arch=sm_80 --maxrregcount 32 -o kernel kernel.cu
```

Ez a kapcsoló minden kernelre ugyanazt a korlátot alkalmazza. Hátránya, hogy nem tudunk különbséget tenni kernelek között: egy nagy regiszterigényű kernel esetén a korlát drasztikus spill-t okozhat, míg egy kisebb kernelnél nincs hatása. A `maxrregcount` ezért inkább gyors kísérletezésre, nem pedig finomhangolásra való.

12.2.7 `__launch_bounds__`

A `__launch_bounds__` kernel-attribútum a kernel definíciójában adható meg, és két paramétere van: a maximális blokkméret (`maxThreadsPerBlock`) és opcionálisan a minimális blokkok száma SM-enként (`minBlocksPerMultiprocessor`). A szintaxis:

```
__global__ void __launch_bounds__(256, 4)
myKernel(float *out) {
    // ...
}
```

A `maxThreadsPerBlock` megadja, hogy a kernelt legfeljebb mekkora blokkmérettel fogjuk indítani. A `minBlocksPerMultiprocessor` azt közli a fordítóval, hogy legalább hány blokkot szeretnénk egyszerre aktívan tartani egy SM-en. Ez utóbbi alapján a fordító kiszámítja a maximálisan megengedett regiszterhasználatot a következő képlet szerint:

```
maxRegsPerThread = regsPerSM / (maxThreadsPerBlock * minBlocksPerMultiprocessor)
```

Ahol `regsPerSM` az SM regiszterfájljának mérete (pl. 64 768 az sm_80 esetén). Ha a kernel természetes regiszterigénye ezt a korlátot meghaladja, a fordító megpróbálja a regiszterhasználatot lecsökkenteni, akár spill árán is. Ha a kernel természetes igénye a korlát alatt van, a `__launch_bounds__`-nak nincs hatása a regiszterekre.

A `__launch_bounds__` előnye a `maxrregcount`-tal szemben, hogy kernel-szinten szabályozható, és figyelembe veszi a tényleges blokkméretet. Hátránya, hogy a `minBlocksPerMultiprocessor` túl magasra állítása indokolatlanul sok spill-t okozhat, mert a fordító a regiszterhasználatot a megadott korlát alá próbálja szorítani, még akkor is, ha az adott kernel számára az alacsonyabb occupancy és a kevesebb spill lenne az optimális.

12.2.8 Gyakorlati példa: regiszter-nyomás és spill

A következő kód egy szándékosan regiszter-éhes kernelt mutat be. A kernel egy 64 elemű, privát float tömböt hoz létre minden szálban, feltölti a bemeneti adatokból, majd összegzi. A `#pragma unroll` teljesen kifejti a ciklust, így a fordító 64 külön élő float változót próbál regiszterben tartani, ami messze meghaladja a tipikus regiszterkeretet.

```
// CUDA C++ (C++17), NVCC
// Fájl: reg_spill.cu

#include <cuda_runtime.h>
#include <cstdio>

#define N (1 << 20)          // 1M szál
#define ARRAY_SIZE 64        // 64 float privát tömb

// Hibaellenőrző makró - a labor-szerződés szerint kötelező
#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err = call;                                             \
        if (err != cudaSuccess) {                                           \
            std::fprintf(stderr, "CUDA hiba: %s, %s:%d\n",                 \
                         cudaGetErrorString(err), __FILE__, __LINE__);      \
            std::exit(EXIT_FAILURE);                                        \
        }                                                                   \
    } while (0)

__global__ void kernel_spill(const float* __restrict__ in,
                             float* __restrict__ out) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    // Privát tömb: a fordító megpróbálja regiszterben tartani,
    // de 64 float + egyéb változók valószínűleg spill-t okoznak.
    float a[ARRAY_SIZE];

    float sum = 0.0f;
    #pragma unroll
    for (int i = 0; i < ARRAY_SIZE; ++i) {
        // A beolvasás indexelése úgy van kialakítva, hogy minden
        // szál más-más, de a tömb elemei a szálon belül összefüggőek.
        a[i] = in[idx + i * blockDim.x];
        sum += a[i];
    }
    out[idx] = sum;
}

int main() {
    float *d_in, *d_out;
    float *h_in = new float[N];
    float *h_out = new float[N];

    for (int i = 0; i < N; ++i) h_in[i] = 1.0f;

    CUDA_CHECK(cudaMalloc(&d_in, N * sizeof(float)));
    CUDA_CHECK(cudaMalloc(&d_out, N * sizeof(float)));
    CUDA_CHECK(cudaMemcpy(d_in, h_in, N * sizeof(float),
                          cudaMemcpyHostToDevice));

    int block = 256;
    int grid = (N + block - 1) / block;

    kernel_spill<<<grid, block>>>(d_in, d_out);
```

```
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    CUDA_CHECK(cudaMemcpy(h_out, d_out, N * sizeof(float),
                          cudaMemcpyDeviceToHost));

    std::printf("Első elem: %f\n", h_out[0]);

    cudaFree(d_in);
    cudaFree(d_out);
    delete[] h_in;
    delete[] h_out;
    return 0;
}
```

A fenti kernel fordítása `nvcc -arch=sm_80 -Xptxas -v -o reg_spill reg_spill.cu` paranccsal. A ptxas kimenetében várhatóan 80–100 regiszter és több száz bájt spill store/load jelenik meg. A pontos számok a CUDA verziójától és a fordító optimalizálási szintjétől függenek, ezért a mérési naplóban mindig rögzíteni kell a fordítási körülményeket.

Készítsünk egy második verziót, amely `__launch_bounds__`-szal korlátozza a regiszterhasználatot, és egy harmadikat, amelyben a privát tömb méretét 16-ra csökkentjük, így a regiszterigény természetesen alacsonyabb.

```
// CUDA C++ (C++17), NVCC
// Fájl: reg_launch_bounds.cu

__global__ void __launch_bounds__(256, 4)
kernel_lb(const float* __restrict__ in, float* __restrict__ out) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    float a[ARRAY_SIZE];
    // ... ugyanaz a törzs, mint fent
}
```

A `__launch_bounds__(256, 4)` azt mondja: 256 szálas blokkokat indítunk, és legalább 4 blokkot szeretnénk SM-enként. sm_80 esetén ez `64 768 / (256 * 4) = 63` regiszter szálanként. A fordító tehát legfeljebb 63 regisztert használhat, ami a 64 elemű tömb esetén valószínűleg jelentős spill-t eredményez, de az occupancy 4 blokkra nő.

A harmadik verzió, ahol a tömb mérete 16:

```
__global__ void kernel_small(const float* __restrict__ in,
                             float* __restrict__ out) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    float a[16];
    float sum = 0.0f;
    #pragma unroll
    for (int i = 0; i < 16; ++i) {
        a[i] = in[idx + i * blockDim.x];
        sum += a[i];
    }
    out[idx] = sum;
}
```

Ez a kernel várhatóan 32–40 regisztert használ, nincs spill, és az occupancy-t már nem a regiszterek korlátozzák, hanem más tényezők (pl. blokkméret, szálszám limit).

12.2.9 Labor: regiszter-nyomás mérése

A labor célja, hogy saját környezetben megmérje a regiszterhasználatot, a spill mértékét és a teljesítményre gyakorolt hatást. Az alábbiakban megadjuk a szükséges lépéseket, a mérési napló sablonját és az elválasztott várt eredményeket.

12.2.10 Környezet rögzítése

A mérés megkezdése előtt rögzíteni kell a hardver- és szoftverkörnyezetet:

- GPU modell (pl. NVIDIA A100, RTX 3090, V100)
- Compute capability (pl. 8.0, 8.6, 7.0)
- CUDA Toolkit verzió (pl. 12.2)
- GPU driver verzió
- Operációs rendszer
- Fordító parancs és optimalizálási szint
- ECC, MIG, power limit, háttérterhelés (ha releváns)

Minden mérés előtt melegítsük fel a GPU-t (warm-up futtatás), majd legalább 10 ismétlést végezzünk. Az átlag mellett a szórást is közöljük. A mérést izolált környezetben végezzük, és gondoskodjunk arról, hogy a túl hosszú futású kernel ne okozzon eszközleállást; állítsunk be timeout-ot, ha szükséges.

12.2.11 Mérési lépések

1. Fordítsa le mindhárom kernelt a `-Xptxas -v` kapcsolóval, és jegyezze fel a regiszterhasználatot, a spill store/load bájtokat és a stack frame méretét.
2. Futtassa mindhárom kernelt `N = 1<<20` elemmel, 256 szálas blokkokkal. Mérje a futási időt CUDA eventekkel vagy `std::chrono`-val (a labor-szerződés szerint CUDA event ajánlott). Végezzen 10 ismétlést, számítson átlagot és szórást.
3. Használja az Nsight Compute-et a következő metrikák lekérésére (a pontos metrikanevek architektúrafüggők, ezért a `--query-metrics` kapcsolóval ellenőrizze az elérhető neveket):
   - `launch__registers_per_thread` – tényleges regiszterhasználat
   - `l1tex__t_bytes_pipe_lsu_mem_local_op_ld.sum` – local memory olvasás bájtban
   - `l1tex__t_bytes_pipe_lsu_mem_local_op_st.sum` – local memory írás bájtban
   - `sm__warps_active.avg.pct_of_peak_sustained_active` – occupancy
   - `gpu__time_duration.sum` – futási idő
4. Ha lehetséges, használja a `cudaOccupancyMaxActiveBlocksPerMultiprocessor` API-t a foglaltság kiszámításához a ptxas által jelentett regiszterek és a blokkméret alapján.

12.2.12 Mérési napló sablon

A következő táblázatot töltse ki minden kernelre. A sorok a három verziót tartalmazzák, az oszlopok a mért értékeket:

Kernel	Regiszterek (ptxas)	Spill store (byte)	Spill load (byte)	Occupancy (%)	Futási idő (ms, átlag ± szórás)
kernel_spill					
kernel_lb					
kernel_small					

A táblázat celláit a mérés során kitöltjük. A “várt eredmény” oszlopot nem a táblázatban, hanem külön szövegben adjuk meg.

12.2.13 Várt eredmény

A modell alapján a következő tendenciákat várjuk:

- A `kernel_spill` regiszterhasználata a legmagasabb, mert a 64 elemű, teljesen kigöngyölt privát tömb minden eleme élő kell, hogy maradjon a ciklusmagban. Ez valószínűleg meghaladja a 64 regisztert, és spill-t okoz.
- A `kernel_lb` a `__launch_bounds__(256, 4)` miatt legfeljebb 63 regisztert használhat, ezért a regiszterhasználata alacsonyabb, mint a `kernel_spill`-é, de a spill mennyisége nagyobb lehet, mert a fordító a korlát betartása érdekében kénytelen több változót a local memoryba tenni.
- A `kernel_small` regiszterhasználata a legalacsonyabb, és várhatóan nincs spill, mert a 16 elemű tömb kényelmesen elfér a regiszterekben. Ennek következtében a futási idő várhatóan a legrövidebb.

A tényleges mérések ezeket a tendenciákat erősíthetik meg, de a pontos számok a hardvertől, a CUDA verziótól és a fordító optimalizálási döntéseitől függenek. Ezért a mért és a várt eredményt mindig külön kell dokumentálni, ahogyan azt a labor-szerződés előírja.

12.2.14 Tipikus hibák és helyes megoldások

A regiszter-nyomás és a spill kezelése során több gyakori hiba fordul elő:

1. **A spill figyelmen kívül hagyása:** Sok fejlesztő csak a futási időt nézi, és nem ellenőrzi a ptxas kimenetét. Ha a kernel lassú, és a regiszterhasználat magas, érdemes megnézni a spill-t, mert a local memory forgalom rejtett szűk keresztmetszet lehet.
2. **Túl agresszív `maxrregcount`:** A `--maxrregcount` erőltetett használata minden kernelre ugyanazt a korlátot alkalmazza, ami egyes kerneleknél indokolatlanul sok spill-t okozhat. Jobb megoldás a `__launch_bounds__` kernel-szintű alkalmazása, vagy a kód átstrukturálása, hogy kevesebb élő változó legyen.
3. **A private tömbök rossz használata:** A nagy, privát tömbök (például `float a[100]`) a szál regisztereit terhelik, és gyakran spill-hez vezetnek. Ha a tömb mérete fordítási időben ismert, és az indexelés konstans, a fordító regiszterekre képezi; ha az indexelés dinamikus, a tömb mindig local memoryba kerül. Ilyen esetekben érdemes megfontolni a shared memory használatát, vagy a tömb méretének csökkentését.

4. **Az occupancy egyoldalú növelése:** A magasabb occupancy nem mindig jobb. Ha a regiszterhasználat csökkentése miatt a spill megnő, a kernel lehet, hogy lassabb lesz, mint alacsonyabb occupancy mellett, de spill nélkül. Az optimális megoldást csak méréssel lehet megtalálni, például a `__launch_bounds__` különböző paramétereinek kipróbálásával.
5. **A mérési környezet elhanyagolása:** A regiszterhasználat függ a fordító verziójától, az optimalizálási szinttől és a compute capability-től. Ezért minden mérésnél rögzíteni kell ezeket a paramétereket, és a mérési naplóban fel kell tüntetni a pontos környezetet.

12.2.15 Összefoglalás és továbbvezetés

A regiszterhasználat és a local memory spill a GPU kernel teljesítményének kulcstényezői. A regiszterek a leggyorsabb tárolók, de véges számuk miatt a nagy regiszterigényű kernelek kénytelenek spill-t generálni, ami extra memóriaforgalmat és késleltetést okoz. A fejlesztő eszköztárában ott van a `maxrregcount` és a `__launch_bounds__`, amelyekkel a regiszterhasználat korlátozható, de mindkettő ára a megnövekedett spill lehet.

A következő alfejezetben (Latency hiding – Latency hiding) azt vizsgáljuk meg, hogyan képes a GPU elrejteni a memória- és utasítás-késleltetést a warpok közötti ütemezéssel. A regiszterhasználat és az occupancy itt tárgyalt fogalmai közvetlenül kapcsolódnak ehhez: minél több aktív warp van, annál jobban elrejthető a késleltetés, de csak akkor, ha a regiszterek lehetővé teszik a megfelelő occupancit. A spill és a latency hiding együttes kezelése az egyik legfontosabb optimalizálási készség a gyakorló CUDA-fejlesztő számára.

A fejezet laborfeladata során mért eredményeket vessük össze a fenti várt tendenciákkal, és dokumentáljuk az eltéréseket. Csak a saját, ellenőrzött mérési adatok alapján vonjunk le következtetéseket; a ptxas kimenetét mindig rögzítsük, mert az a regiszter-nyomás elsődleges bizonyítéka.

12.3 Latency hiding

A GPU-programok teljesítményét nemcsak az határozza meg, hogy a hardver hány műveletet tud végrehajtani másodpercenként, hanem az is, hogy a végrehajtóegységek hány ciklusig állnak üresen, miközben egyetlen utasítás eredményére várnak. Ezt a várakozási időt nevezzük késleltetésnek. A GPU-k tervezésének egyik alapelve, hogy a késleltetést nem elsősorban csökkenteni próbálják, hanem elrejteni: amíg egy utasítás eredménye nem áll rendelkezésre, a hardver más, független utasításokat hajt végre.

Ebben az alfejezetben a késleltetés elrejtésének két fő mechanizmusát vizsgáljuk: a warp-szintű párhuzamosságot, valamint az egy szálon belüli utasításszintű párhuzamosságot. A fogalmakra közvetlenül épít a következő, Occupancy optimalizálás jelű egység, amely az occupancy optimalizálásával foglalkozik.

12.3.1 Memória- és utasításkésleltetés

A GPU-n futó kódban kétféle késleltetés dominál. A memória-késleltetés a globális memóriaelérés ideje: egy globális load a L2 cache vagy a HBM-memória távolsága miatt több száz ciklust is igénybe vehet. Az utasításkésleltetés ennél rövidebb, de szintén fontos: például egy lebegőpontos FMA utasítás eredményére a következő, attól függő utasításnak néhány ciklust várnia kell.

A késleltetés elrejtésének általános összefüggése megfogalmazható a következő módon:

$$P_{\text{szükséges}} \approx L_{\text{eff}} \cdot T_{\max},$$

ahol $P_{\text{szükséges}}$ a szükséges, egymástól független, folyamatban lévő műveletek száma, L_{eff} a hatékony késleltetés ciklusokban, $T_{\max}$ pedig a maximális átbocsátóképesség műveletek per ciklus mértékegységben. Ha P túl kicsi, az execution pipeline lyukakat kap, a hardver kihasználtsága pedig csökken.

Memóriaesetben P nem feltétlenül utasításokat, hanem folyamatban lévő memóriaeléréseket vagy azok byte-összegét jelenti. A gondolat azonban ugyanaz: a memória–pipeline telítéséhez annyi memóriaterhelést kell „úton lenni” hagyni, amennyit a késleltetés és a sávszélesség szorzata megkövetel.

A gyakorlatban ezt két szinten lehet megvalósítani:

- **warp-szintű párhuzamosság (WLP):** ha egy warp elakad, az SM ütemezője egy másik warpot futtat;
- **utasításszintű párhuzamosság (ILP):** egy warp egyetlen szála több, egymástól független utasítást bocsát ki, így ugyanaz a warp is tud előrehaladni, miközben egy korábbi utasítás késleltetését várja.

Mindkettő a $P_{\text{szükséges}}$ értéket növeli, ezért egymás helyettesítői is lehetnek, de más erőforráskövetkezménnyel.

12.3.2 Warp-szintű párhuzamosság

A warp-szintű párhuzamosság a GPU-végrehajtás legalapvetőbb elrejtési mechanizmusa. Egy SM-ben több warp ütemező működik; minden ütemező órajelenként jellemzően egy warp-utasítást tud kiadni valamelyik futásra jogosult warpnak. Amikor egy warp memóriaelérésre vár, az adat megérkezéséig nem futásra jogosult, ezért az ütemező más warpot választ.

Tegyük fel, hogy a globális memória késleltetése 400 ciklus, és minden warp egy 128 byte-os, 32 szál által végrehajtott lebegőpontos loadot generál. Ha csak egy warp lenne az SM-ben, akkor az ütemező kiadna egy loadot, majd 400 cikluson keresztül nem lenne más elvégezhető munka. A memória–pipeline tehát szinte üresen járna.

A szükséges warp-szám memória-kötött feladatokra közelíthető a következő képlettel:

$$W_{\text{szükséges}} \approx \frac{L_{\text{mem}} \cdot B_{\text{mem,cyc}}}{B_{\text{warp}}},$$

ahol L_{mem} a memória-késleltetés ciklusban, $B_{\text{mem,cyc}}$ a fenntartható memória-sávszélesség byte/ciklusban, B_{warp} pedig az egy warp által kibocsátott load byte-száma. Egy tipikus illusztratív esetben $L_{\text{mem}} = 400$ ciklus, $B_{\text{mem,cyc}} = 600$ byte/ciklus, $B_{\text{warp}} = 128$ byte, így

$$W_{\text{szükséges}} \approx \frac{400 \cdot 600}{128} \approx 1875$$

warp összesen. Ha a GPU-nak 60 SM-je van, ez SM-enként körülbelül 31 aktív warpot jelent. Ha egy SM legfeljebb 64 warpot tud fogadni, akkor ez körülbelül 50%-os occupancy-t követel. Az alacsonyabb occupancy tehát ebben a memória-kötött helyzetben sávszélesség-csökkenést okoz.

A valóságban a pontos számot befolyásolják a cache találatok, a memória–utánpótlás korlátai, a memóriarendszerben folyamatban lévő kérések maximális száma és az, hogy a warp éppen

hány loadot tud egyszerre kiadni. A mérnöki szemléletnek azonban nem kell tökéletesen pontos előrejelzést adnia: elég meghatározni, hogy a vizsgált kernel memória-kötött-e, és hogy van-e elegendő aktív warp az SM-en.

A WLP növelésének legegyszerűbb módja a regiszter- és shared-memory-nyomás csökkentése, hogy az SM több blokkot tudjon fogadni. A dinamikus shared memory méretének növelésével viszont szándékosan csökkenthető az occupancy, amit a laborban mérési eszközként használunk.

12.3.3 Utasításszintű párhuzamosság

Az ILP ugyanannak a warpnak az utasításfolyamán belül jelent függetlenséget. Ha egy szálban a következő utasítás az előző eredményétől függ, akkor az ütemező nem tudja kiadni, amíg az eredmény el nem készül. Ha viszont a kódban több független lánc van, az ütemező az egyik lánc utasítását kiadhatja, miközben a másik lánc késleltetését várja.

Tekintsünk egy tipikus FMA-láncot:

```
float acc = 0.0f;
for (int i = 0; i < n; ++i) {
    acc = a * acc + b;
}
```

Itt minden FMA az előző FMA eredményét használja, ezért a lánc hossza mentén nem lehet átfedés. Ha egy FMA késleltetése L_{FMA} ciklus, akkor a függő láncban csak minden L_{FMA} ciklusban lehet egy FMA-t kiadni. Ha $L_{\mathrm{FMA}} = 4$, akkor egy ilyen warp az FMA-pipeline átbocsátóképességének legfeljebb negyedét használja ki.

A megoldás több független lánc bevezetése:

```
float acc0 = 0.0f, acc1 = 0.0f, acc2 = 0.0f, acc3 = 0.0f;
for (int i = 0; i < n; i += 4) {
    acc0 = a0 * acc0 + b0;
    acc1 = a1 * acc1 + b1;
    acc2 = a2 * acc2 + b2;
    acc3 = a3 * acc3 + b3;
}
```

Ebben a példában az ILP értéke 4, mert négy független FMA áll rendelkezésre. Ha az FMA-késleltetés 4 ciklus, akkor elvileg egyetlen warp is képes ciklusonként egy FMA-t kiadni, feltéve, hogy a regiszterek rendelkezésre állnak.

Ugyanez a gondolat memóriaterhelésre is alkalmazható. A ciklusonkénti egyetlen accumulator helyett négy accumulator, valamint négy egymástól független load engedi, hogy egyszerre több memóriaterhelés legyen folyamatban. Ezt hívják memória-szintű párhuzamosságnak, amely az ILP memóriaváltozata.

A WLP és az ILP közötti kapcsolat közelítően így írható fel:

$$P_{\mathrm{elrejtő}} \approx \mathrm{ILP} \times W_{\mathrm{aktív}},$$

ahol $W_{\mathrm{aktív}}$ az egy ütemezőre jutó, futásra jogosult aktív warpokat, az ILP pedig a független utasításláncok számát jelenti. A szükséges elrejtési kapacitás a késleltetés és az átbocsátóképesség szorzata. Ha az ILP nő, kevesebb aktív warp is elegendő lehet; ha az ILP kicsi, a hardvernek több warpra van szüksége.

Az ILP növelésének ára a megnövekedett regiszterfogyasztás. Minden független accumulator legalább egy regisztert foglal. Ha az ILP-t túlzottan növeljük, a regiszterek elfogynak, a compiler

regiszter-spillt generál, és az occupancy zuhan. Ezért a gyakorlatban az ILP-t és az occupancy-t együtt kell hangolni.

12.3.4 Gyakorlati minták a késleltetés elrejtésére

A CUDA-kódokban a leggyakrabban az alábbi mintákat érdemes alkalmazni.

12.3.5 1. Grid-stride loop és több accumulator

A grid-stride loop már ismert forma: a grid méretét a teljes adathalmaznál kisebbre választjuk, és minden szál a saját indexéből indulva, a teljes grid-stride-mal lépkedve dolgozza fel az adatokat. A következő kód egyszerre növeli a memória-szintű párhuzamosságot és csökkenti a ciklus-ellenőrzések számát:

```
float s0 = 0.0f, s1 = 0.0f, s2 = 0.0f, s3 = 0.0f;
size_t i = blockIdx.x * blockDim.x + threadIdx.x;
size_t stride = gridDim.x * blockDim.x;

for (; i + 3 * stride < n; i += 4 * stride) {
    s0 += x[i];
    s1 += x[i + stride];
    s2 += x[i + 2 * stride];
    s3 += x[i + 3 * stride];
}
float sum = (s0 + s1) + (s2 + s3);
```

Itt a négy betöltés független, így a memória–pipeline-ban egyszerre több kérés lehet folyamatban. A végső összegzést a ciklus után egyszer végezzük el.

12.3.6 2. `#pragma unroll`

A CUDA C++ compiler a `#pragma unroll` direktívával kérhető a ciklus kibontására:

```
#pragma unroll
for (int j = 0; j < 4; ++j) {
    s[j] += x[i + j * stride];
}
```

A teljes kibontás csökkenti a ciklus-adminisztrációt, és lehetővé teszi, hogy a compiler egymás mellé ütemezze a független loadokat és FMA-kat. Az unroll faktorral az is szabályozható, hogy a regiszterhasználat ne nőjön túlzottan.

12.3.7 3. A `__restrict__` kulcsszó

A pointerek `__restrict__` minősítője segít a compilernek feltételezni, hogy a mutatók nem aliasingolnak. Ez gyakran további utasítás-átrendezést tesz lehetővé, ami közvetve növeli az ILP-t.

12.3.8 4. Warp-szintű függvények

A shuffle, vote és match intrinsics lehetőséget adnak arra, hogy egy warp szálai közvetlenül cseréljenek regiszteradatot. Ezzel a memória-késleltetést kiválthatjuk, ha az adat a szomszéd szál regiszterében már rendelkezésre áll. A warp intrinsics használatakor azonban körültekintően kell kezelni a láthatóságot és a divergence-t.

12.3.9 Tipikus hibák

A késleltetés elrejtésének tanulása során a következő hibák a leggyakoribbak.

1. **Túl kevés aktív warp.** A kernel occupancy-je a blokkméret, a regiszter- vagy shared-memory-használat miatt alacsony. A memória-kötött kernel nem éri el a sávszélességet, és a futásidő leromlik.
2. **Függőségi lánc ILP nélkül.** Egyetlen accumulator minden iterációban attól függ, ami az előző iterációban keletkezett. A ciklus kibontása nélkül a warp nem tudja elrejteni az FMA-késleltetést, még akkor sem, ha sok warp fut.
3. **Túlzott ILP, regiszter-spill.** Nyolc-tizenhat accumulator egyszerre növelheti a regiszterek számát, és a compiler a lassú local memóriába mentheti az értékeket. Ez gyakran rosszabb teljesítményhez vezet, mint a kisebb ILP.
4. **Az occupancy fetisizálása.** A magas occupancy nem mindig jobb. Ha egy kernel erősen számítás-kötött, néhány független FMA-lánc alacsony occupancy mellett is képes lehet telíteni az FP32-pipeline-t. A túl nagy blokkméret további szinkronizációs és cache-költséggel járhat.
5. **A mérési konfiguráció nem rögzített.** A késleltetés-elrejtési kísérletekben mindig az azonos összes munkát, precizitást és GPU-állapotot kell hasonlítani. Ha az egyik kísérletben más adatmérete vagy különböző gridmérete van, az eredmény nem értelmezhető.

12.3.10 Labor: több warp és ILP hatásának mérése

A labor célja, hogy mérhetővé tegyük a WLP és az ILP hatását. A WLP-mérést memória-kötött kernelre, az ILP-mérést FMA-latencia-kötött kernelre végezzük. A kód minden CUDA API-hívást ellenőriz, warm-upot futtat, és ismételt mérésekből átlagol.

12.3.11 Környezet rögzítése

A labor futtatása előtt rögzíteni kell a következő adatokat. Az alábbi helyőrzők, a tényleges mérés előtt ki kell tölteni.

Jellemző	Érték
GPU	`<helyőrző: például NVIDIA RTX 3080 10 GB>`
Compute capability	`<helyőrző: például 8.6>`
Driver	`<helyőrző: például 535.104.05>`
CUDA verzió	`<helyőrző: például 12.4>`
Operációs rendszer	`<helyőrző: például Ubuntu 22.04>`
Power limit	`<helyőrző: például gyári>`
ECC	`<helyőrző: például letiltva>`

A labor izolált környezetben futtatandó. Ha a GPU többfelhasználós, akkor a mérés előtt meg kell győződni arról, hogy a szomszédos terhelés nem torzítja az eredményt.

12.3.12 Buildparancs

A forrást `latency_hiding.cu` néven mentjük, és a következő paranccsal fordítjuk:

```
nvcc -O3 -arch=native -std=c++17 latency_hiding.cu -o latency_hiding
```

Futtatás:

```
./latency_hiding
```

12.3.13 Teljes forrás

```
#include <cstdio>
#include <cstdlib>
#include <vector>
#include <cuda_runtime.h>

#define CHECK_CUDA(call)                                                    \
    do {                                                                    \
        cudaError_t err = (call);                                           \
        if (err != cudaSuccess) {                                           \
            std::fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__,     \
                         __LINE__, cudaGetErrorString(err));                \
            std::exit(EXIT_FAILURE);                                        \
        }                                                                   \
    } while (0)

__global__ void k_mem_wlp(const float* __restrict__ x,
                          float* __restrict__ out,
                          size_t n) {
    extern __shared__ float smem[];
    smem[threadIdx.x] = 0.0f; // a foglalás miatt csökken az occupancy

    size_t idx = blockIdx.x * blockDim.x + threadIdx.x;
    size_t stride = gridDim.x * blockDim.x;
    float acc = 0.0f;

#pragma unroll 1
    for (size_t i = idx; i < n; i += stride) {
        acc += x[i];
    }

    out[idx] = acc;
}

template <int ILP>
__global__ void k_arith_ilp(float* __restrict__ out, int iters) {
    extern __shared__ float smem[];
    smem[threadIdx.x] = 0.0f;

    float a[ILP];
    float b[ILP];
    float c[ILP];

#pragma unroll
    for (int j = 0; j < ILP; ++j) {
        a[j] = 1.0001f;
```

```
        b[j] = 0.9999f;
        c[j] = 0.0001f;
    }

    for (int it = 0; it < iters; ++it) {
#pragma unroll
        for (int j = 0; j < ILP; ++j) {
            a[j] = fmaf(a[j], b[j], c[j]);
        }
    }

    float sum = 0.0f;
#pragma unroll
    for (int j = 0; j < ILP; ++j) {
        sum += a[j];
    }

    out[blockIdx.x * blockDim.x + threadIdx.x] = sum;
}

template <typename Launcher>
float measure_ms(Launcher&& launch, int repeats) {
    cudaEvent_t start, stop;
    CHECK_CUDA(cudaEventCreate(&start));
    CHECK_CUDA(cudaEventCreate(&stop));

    launch();
    CHECK_CUDA(cudaGetLastError());
    CHECK_CUDA(cudaDeviceSynchronize());

    CHECK_CUDA(cudaEventRecord(start));
    for (int i = 0; i < repeats; ++i) {
        launch();
        CHECK_CUDA(cudaGetLastError());
    }
    CHECK_CUDA(cudaEventRecord(stop));
    CHECK_CUDA(cudaEventSynchronize(stop));

    float ms = 0.0f;
    CHECK_CUDA(cudaEventElapsedTime(&ms, start, stop));
    CHECK_CUDA(cudaEventDestroy(start));
    CHECK_CUDA(cudaEventDestroy(stop));

    return ms / static_cast<float>(repeats);
}

int main() {
    int dev = 0;
    CHECK_CUDA(cudaSetDevice(dev));

    cudaDeviceProp prop;
    CHECK_CUDA(cudaGetDeviceProperties(&prop, dev));

    std::printf("GPU: %s\n", prop.name);
    std::printf("compute capability: %d.%d\n", prop.major, prop.minor);
    std::printf("max warps/SM: %d\n", prop.maxThreadsPerMultiProcessor / 32);
    std::printf("max shared/SM: %zu bytes\n", prop.sharedMemPerMultiprocessor);

    const size_t n = 64ull * 1024 * 1024;
    const int block_w = 256;
    const int grid_w = 1024;

    float* d_in = nullptr;
```

```
float* d_out = nullptr;
CHECK_CUDA(cudaMalloc(&d_in, n * sizeof(float)));
CHECK_CUDA(cudaMalloc(&d_out,
                      static_cast<size_t>(grid_w) * block_w * sizeof(float)));

CHECK_CUDA(cudaMemset(d_in, 0x3f, n * sizeof(float)));

std::printf("\n== WLP: memória-kötött kernel ==\n");
std::printf("block=%d grid=%d n=%zu Mi = %.1f MiB\n",
            block_w, grid_w, n,
            static_cast<double>(n) * 4.0 / (1024.0 * 1024.0));

std::vector<size_t> smem_cfg = {
    0, 16 * 1024, 32 * 1024, 48 * 1024
};

for (size_t smem : smem_cfg) {
    if (smem > static_cast<size_t>(prop.sharedMemPerBlockOptin)) {
        continue;
    }

    int occ_blocks = 0;
    CHECK_CUDA(cudaOccupancyMaxActiveBlocksPerMultiprocessor(
        &occ_blocks, k_mem_wlp, block_w, smem));

    int warps_per_sm = occ_blocks * (block_w / 32);

    float ms = measure_ms([&] {
        k_mem_wlp<<<grid_w, block_w, smem>>>(d_in, d_out, n);
    }, 10);

    CHECK_CUDA(cudaDeviceSynchronize());

    double GBs = static_cast<double>(n) * sizeof(float) /
                 (static_cast<double>(ms) * 1e-3) / 1e9;

    std::printf(
        "smem=%5zu B | occ_blocks=%2d | warps/SM=%2d | "
        "átlag ms=%.3f | eff. sávsz.=%.1f GB/s\n",
        smem, occ_blocks, warps_per_sm, ms, GBs);
}

std::printf("\n== ILP: FMA-latencia-kötött kernel ==\n");

const int block_i = 128;
const int grid_i = 512;
const int iters = 5000;
const size_t smem_low = 48 * 1024;
const size_t smem_high = 0;

float* d_out_i = nullptr;
CHECK_CUDA(cudaMalloc(
    &d_out_i,
    static_cast<size_t>(grid_i) * block_i * sizeof(float)));

for (int ilp : {1, 2, 4, 8, 16}) {
    long long total_fma =
        static_cast<long long>(grid_i) * block_i * iters * ilp;
    double gflop = static_cast<double>(total_fma) * 2.0 / 1e9;

    auto print_case = [&](size_t smem, const char* label) {
        if (smem > static_cast<size_t>(prop.sharedMemPerBlockOptin)) {
            return;
```

```
            }

            float ms = 0.0f;
            switch (ilp) {
                case 1:
                    ms = measure_ms([&] {
                        k_arith_ilp<1><<<grid_i, block_i, smem>>>(d_out_i, iters);
                    }, 10);
                    break;
                case 2:
                    ms = measure_ms([&] {
                        k_arith_ilp<2><<<grid_i, block_i, smem>>>(d_out_i, iters);
                    }, 10);
                    break;
                case 4:
                    ms = measure_ms([&] {
                        k_arith_ilp<4><<<grid_i, block_i, smem>>>(d_out_i, iters);
                    }, 10);
                    break;
                case 8:
                    ms = measure_ms([&] {
                        k_arith_ilp<8><<<grid_i, block_i, smem>>>(d_out_i, iters);
                    }, 10);
                    break;
                case 16:
                    ms = measure_ms([&] {
                        k_arith_ilp<16><<<grid_i, block_i, smem>>>(d_out_i, iters);
                    }, 10);
                    break;
                default:
                    return;
            }

            CHECK_CUDA(cudaDeviceSynchronize());

            double gflops = gflop / (static_cast<double>(ms) * 1e-3);
            std::printf(
                "ILP=%2d | smem=%5zu B | %-18s | ms=%.3f | GFLOPS=%.1f\n",
                ilp, smem, label, ms, gflops);
        };

        print_case(smem_low, "alacsony occupancy");
        print_case(smem_high, "magas occupancy");
    }

    CHECK_CUDA(cudaFree(d_in));
    CHECK_CUDA(cudaFree(d_out));
    CHECK_CUDA(cudaFree(d_out_i));

    return 0;
}
```

A mérési program a CUDA eventek alapján számítja az átlagos futásidőt. A WLP-részben a `k_mem_wlp` kernel a teljes tömböt beolvassa és összegzi, így az effektív sávszélesség az adathalmaz méretéből és az időből számítható. Az ILP-részben a `k_arith_ilp` kernel csupa független FMA-láncot futtat, így az FMA-átbocsátóképességet méri.

12.3.14 A várt eredmények

A WLP-mérésben az a várakozás, hogy 0 byte dinamikus shared memory mellett az occupancy magas, ezért az effektív sávszélesség közel kerül a GPU fenntartható HBM-sávszélességéhez.

A shared memory méretének növelésével az SM-enként aktív blokkok száma csökken, ezért a warp-szám is csökken. Emiatt a sávszélesség érezhetően visszaesik. A pontos görbe hardverenként eltér, de a monoton csökkenő tendenciának mérhetőnek kell lennie.

Az ILP-mérés alacsony occupancy mellett azt várja, hogy ILP = 1 esetén a GFLOPS-érték alacsony. Ha az FMA-késleltetés 4 ciklus, akkor ILP = 4 már alkalmas arra, hogy egyetlen warp is gyakorlatilag folyamatosan kiadjon FMA-utasításokat, ezért a teljesítmény jelentősen nő. ILP = 8 vagy 16 már legfeljebb kis további javulást ad, feltéve, hogy a regiszterhasználat nem okoz spillt. Magas occupancy mellett az ILP hatása gyengébb lehet, mert a sok warp már eleve biztosítja az elrejtést.

12.3.15 A mért eredmények

Az alábbi naplót a futtatás után kell kitölteni. A táblázatban a `<mért>` értékek szándékosan üresek; a könyv nem mutat be nem futtatott benchmarkot ellenőrzött eredményként.

WLP	smem	occupancy blocks/SM	warps/SM	átlag ms	eff. sávszélesség GB/s
WLP	0	`<mért>`	`<mért>`	`<mért>`	`<mért>`
WLP	16 KiB	`<mért>`	`<mért>`	`<mért>`	`<mért>`
WLP	32 KiB	`<mért>`	`<mért>`	`<mért>`	`<mért>`
WLP	48 KiB	`<mért>`	`<mért>`	`<mért>`	`<mért>`

ILP	smem	occupancy-jelleg	átlag ms	GFLOPS
1	0	magas	`<mért>`	`<mért>`
1	48 KiB	alacsony	`<mért>`	`<mért>`
2	0	magas	`<mért>`	`<mért>`
2	48 KiB	alacsony	`<mért>`	`<mért>`
4	0	magas	`<mért>`	`<mért>`
4	48 KiB	alacsony	`<mért>`	`<mért>`
8	0	magas	`<mért>`	`<mért>`
8	48 KiB	alacsony	`<mért>`	`<mért>`
16	0	magas	`<mért>`	`<mért>`
16	48 KiB	alacsony	`<mért>`	`<mért>`

A mérési napló mellett a szórást vagy az ismétlések percentilisét is rögzíteni kell. A fenti program 10 ismétlést használ, de nem számol szórást. Ellenőrzött méréshez érdemes a futásidőket egy további oszlopban tárolni, például `ms ± szórás` formában.

12.3.16 Ellenőrzés a labor során

A laborban a következő szempontokat kell ellenőrizni.

- A `cudaOccupancyMaxActiveBlocksPerMultiprocessor` által visszaadott blokkszám és a launch-konfiguráció összhangban van.
- A 0 shared-memory-s WLP-eset nem lép túl semmilyen erőforráskorlátot.

- A 48 KiB-os eset ténylegesen csökkenti az aktív blokkok számát, nem csak a shared memory foglalást növeli.
- A mérés előtt a GPU hőmérséklete stabil, és az órajel nem throttling miatt esik vissza.
- Az ILP-kernel esetében a compiler nem optimalizálja ki a ciklust, és a kimenetbe írás megakadályozza a holtkód-eliminációt.

12.3.17 Gyakori mérési csapdák

Az egyik leggyakoribb hiba, hogy a WLP-mérésben a grid-stride miatt egyes szálak nagyon eltérő számú elemet dolgoznak fel. Ez a load-kiegyensúlyozatlanság miatt torzíthatja a mérést. Ezt a példában azzal kerüljük el, hogy a grid mérete osztója az adatméretnek, és minden szál ugyanannyi elemet olvas.

Egy másik gyakori hiba, hogy a mérés a kernel indítása után nem szinkronizál, ezért a CPU-oldali időmérés nem a kernel futási idejét mutatja. A fenti program CUDA eventeket használ, és a `measure_ms` függvény az eventek segítségével vár a GPU befejezésre.

ILP-mérésnél a túl kevés iteráció miatt a kernel futásideje a mérési zajjal összemérhető. Az `iters = 5000` érték a legtöbb modern GPU-n elegendő, de ha a mért futásidő 1 ms alatti, az iterációszámot növelni kell.

12.3.18 Önellenőrző kérdések

1. Egy SM-ben az FMA-késleltetés 4 ciklus, az FMA-átbocsátóképesség 1 utasítás/ciklus/ütemező. Hány független FMA-láncra van szükség ugyanabban a warpban ahhoz, hogy egyetlen warp is ki tudja használni ezt az átbocsátóképességet? Mi történik, ha a láncok száma ennél nagyobb?
2. Egy globális memóriafeladat során minden warp 128 byte-ot olvas be egy loaddal. A memória-késleltetés 500 ciklus, a fenntartható sávszélesség 800 byte/ciklus. Becsülje meg, hány warpnak kell egyszerre folyamatban lévő loaddal rendelkeznie a memória–pipeline telítéséhez. Ha a GPU-nak 80 SM-je van, és egy SM 64 warpot tud fogadni, mekkora minimális occupancy szükséges?
3. Egy fejlesztő növeli az ILP-t 4-ről 16-ra, de a kernelfutás lassabb lesz. Milyen erőforrás-problémát érdemes először megvizsgálni, és melyik CUDA-metrika mutatja ezt?

Ezek a kérdések a következő, Occupancy optimalizálás alfejezetben továbbvezetnek: ott az occupancy, a regiszterhasználat és a launch-bounds együttes optimalizálását vizsgáljuk.

12.4 Occupancy optimalizálás

Az occupancy nem önmagában vett cél, hanem egy erőforrás-korlátokból származtatott mérőszám, amely azt mutatja, hogy egy streaming multiprocesszoron (SM) hány aktív warp képes egyszerre jelen lenni. A Latency hiding alfejezet bemutatta, hogy a memória- és utasítás-késleltetések elrejtéséhez aktív warp-okra van szükség. Ebben az alfejezetben azt vizsgáljuk, hogyan alakítható ki az a foglaltság, amely egy adott kernel számára ténylegesen a legjobb átbocsátóképességet adja, és miért nem érdemes kizárólag az elméleti occupancy maximalizálására törekedni.

A fejezet először rögzíti az occupancy fogalmát és erőforrás-korlátait, majd elemzi az alacsony és magas occupancy teljesítményre gyakorolt hatását. Ezt követi egy gyakorlati hangolási

munkafolyamat, végül egy ellenőrizhető labor, amelyben blokméret- és regiszterhangolást végzünk CUDA C++-ban, rögzített környezetben, a mért és a várt eredmények elkülönítésével.

12.4.1 1. Az occupancy mint erőforrás-korlát

A CUDA foglaltság (occupancy) gyakorlati definíciója:

$$O_{\text{elméleti}} = \frac{W_{\text{aktív}}}{W_{\max}},$$

ahol $W_{\text{aktív}}$ az SM-en egyidejűleg rezidens warp-ok száma, $W_{\max}$ pedig az adott SM maximális warpszáma. A $W_{\max}$ értéke gyakran 48, 64 vagy 64 feletti lehet architektúrától függően; például egy NVIDIA Ada-generációs SM esetén jellemzően 48 warp. A valós foglaltságot nemcsak a hardver warp-slot korlátja, hanem további erőforrások is korlátozzák:

- **regiszterfájl mérete** SM-enként;
- **shared memory mérete** SM-enként;
- **blokk-slotok száma** SM-enként;
- **warp-slotok száma** SM-enként.

A regiszterfájl korlátjának hatása egyszerűen szemléltethető. Ha egy szál $R_{\text{szál}}$ regisztert használ, és egy blokk B szálból áll, akkor a blokk regiszterigénye közelítőleg $R_{\text{szál}} \cdot B$. Ha az SM regiszterfájlja R_{SM} darab 32 bites regisztert tartalmaz, akkor a regiszterek alapján elhelyezhető blokkok száma:

$$B_{\text{reg}} = \left\lfloor \frac{R_{\text{SM}}}{R_{\text{szál}} \cdot B} \right\rfloor.$$

Az ebből adódó aktív warpszám:

$$W_{\text{reg}} = \frac{B_{\text{reg}} \cdot B}{32}.$$

A valós hardveren a regiszterallokáció granularitása miatt a képlet kerekítésekkel módosul, ezért az elméleti occupancy számításakor mindig érdemes a CUDA Occupancy Calculator vagy a `cudaOccupancyMaxActiveBlocksPerMultiprocessor` API értékét alapul venni. Az alábbi kódrészlet azt mutatja, hogyan kérdezhető le egy kernelre az adott blokméréthez tartozó maximális aktív blokkszám:

```
// occupancy_query.cu
#include <cuda_runtime.h>
#include <cstdio>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err = (call);                                           \
        if (err != cudaSuccess) {                                           \
            std::fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__,     \
                         __LINE__, cudaGetErrorString(err));                \
            std::exit(EXIT_FAILURE);                                        \
        }                                                                   \
    } while (0)

template<int R, int BLOCK>
__global__ void __launch_bounds__(BLOCK)
```

```
sample_kernel(const float* __restrict__ x, float* __restrict__ y, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= n) return;
    float coeff[R];
    #pragma unroll
    for (int r = 0; r < R; ++r) coeff[r] = x[i] * (float)r;
    float acc = x[i];
    #pragma unroll
    for (int r = 0; r < R; ++r) acc = fmaf(acc, coeff[r], 1.0f);
    y[i] = acc;
}

int main() {
    constexpr int BLOCK = 256;
    int dev = 0;
    CUDA_CHECK(cudaGetDevice(&dev));
    cudaDeviceProp prop{};
    CUDA_CHECK(cudaGetDeviceProperties(&prop, dev));

    int numBlocks = 0;
    CUDA_CHECK(cudaOccupancyMaxActiveBlocksPerMultiprocessor(
        &numBlocks, sample_kernel<32, BLOCK>, BLOCK, 0));

    int activeWarps = numBlocks * BLOCK / prop.warpSize;
    int maxWarps = prop.maxThreadsPerMultiProcessor / prop.warpSize;
    float occupancy = 100.0f * (float)activeWarps / (float)maxWarps;

    std::printf("GPU: %s\n", prop.name);
    std::printf("compute capability: %d.%d\n", prop.major, prop.minor);
    std::printf("max threads/SM: %d\n", prop.maxThreadsPerMultiProcessor);
    std::printf("max warps/SM: %d\n", maxWarps);
    std::printf("active blocks/SM for this kernel: %d\n", numBlocks);
    std::printf("theoretical occupancy: %.1f%%\n", occupancy);
    return 0;
}
```

A fenti példa a `sample_kernel<32, 256>` sablonpéldányra kérdezi le az occupancyt. A `cudaOccupancyMaxActiveBlocksPerMultiprocessor` használata azért fontos, mert figyelembe veszi a launch bounds, a regiszterek és a shared memory által támasztott korlátokat. A könyv konvencióinak megfelelően minden API-hívást ellenőrzünk, és az eszköztulajdonságokat explicit módon lekérdezzük.

12.4.2 2. Az occupancy és a teljesítmény kapcsolata

Az occupancy elsődleges szerepe a késleltetés elrejtése. Ha egy warp hosszú memóriaműveletre vár, a hardver másik warp-okra válthat. Minél több rezidens warp van, annál nagyobb az esély arra, hogy az SM végrehajtó egységei folyamatosan találnak munkát. Ebből a szempontból az alacsony occupancy általában rossz hír, különösen memória-korlátos kernelekben.

Ugyanakkor az occupancy növelése szinte soha nem ingyen van. Ahhoz, hogy több blokk férjen el egy SM-en, csökkenteni kell valamelyik erőforrás blokkonkénti fogyasztását: a regiszterek számát, a shared memory méretét vagy a blockmérethez kötött erőforrásigényt. Ha ezt a regiszterek számának mesterséges csökkentésével érjük el, a fordító regisztereket helyezhet ki lokális memóriába, azaz regiszter-spill keletkezhet. A spill a globális memóriahierarchiába ír és onnan olvas, ami extra késleltetést és sávszélesség-fogyasztást okoz. Az eredmény lehet az, hogy a 100%-os occupancy mellett mért futásidő rosszabb, mint egy 66%-os occupancy mellett mért, de spill-mentes változaté.

A túl magas occupancy másik árnyoldala a cache- és memórianyomás növekedése. Ha sok warp osztozik ugyanazon az L1/L2 cache-en, a hasznos adat gyorsabban kiszorulhat, és nő a cache thrash. Memória-korlátos kerneleknél a túlzottan sok aktív warp nem feltétlenül növeli az elérhető sávszélességet, mert a memóriavezérlő és a cache is telítődhet. Ilyenkor a magasabb occupancy csak nagyobb versenyhez vezet, és a teljesítmény stagnálhat vagy romolhat.

A túl alacsony occupancy tipikus tünetei a következők:

- hosszú `long_scoreboard` stall-ok, mert a warp-ok memória-válaszra várnak;
- alacsony kiadott utasítás/órajel arány (issue rate);
- a végrehajtó egységek kihasználatlansága;
- a futásidő jelentős javulása akkor, ha a blockméretet vagy a regiszterhasználatot úgy változtatjuk, hogy több warp férjen el.

A túl magas occupancy tipikus tünetei:

- lokális memória spill a `ptxas` jelentésben;
- növekvő L1/L2 forgalom anélkül, hogy a hasznos számítás nőne;
- bank conflict a shared memory hozzáférésekben;
- a legjobb futásidő nem a legmagasabb elméleti occupancynél jelentkezik.

Fontos hangsúlyozni, hogy az occupancy egy szükséges, de nem elégséges feltétel. A Latency hiding alfejezetben tárgyalt latency hiding akkor működik jól, ha a késleltetést ténylegesen el lehet rejteni, de ha a kernel számítás-korlátos, akkor a kisebb occupancy is elég lehet, mert a végrehajtó egységek hosszú utasítássorozatokat hajtanak végre. A teljesítményt ezért mindig a következő együttes határozza meg:

$$\text{teljesítmény} = f(\text{occupancy}, \text{regiszterhasználat}, \text{shared memory}, \text{memória-hozzáférési minta}, \text{számítási intenzitá}$$

Egyetlen mérőszám, így az occupancy sem elég a döntéshez.

12.4.3 3. Alacsony és magas occupancy a gyakorlatban

A gyakorlati optimalizálás során két gyakori szélsőséget érdemes felismerni.

12.4.4 3.1. Túl alacsony occupancy

Ha egy kernel nagyon kevés aktív warpot tartalmaz, a hardver nem tudja elrejteni a globális memória késleltetését. Tipikus példa erre egy olyan stencil- vagy redukciós kernel, amely nagy blokkmérettel és nagy regiszterigénnyel indul, de az SM-en csak 8–16 aktív warp marad. A mérés ilyenkor általában azt mutatja, hogy a `smsp__average_warps_issue_stalled_long_scoreboard` metrika magas, az elért foglaltság pedig jóval a hardveres maximum alatt van.

Az alacsony occupancy javításának első lépése a blockméret csökkentése vagy a regiszterhasználat mérséklése. A blockméret csökkentése azonban nem mindig segít, mert a kisebb blokk kevesebb shared memory-t használhat, és a blokkon belüli szinkronizáció is olcsóbb lehet, de a túl kicsi blokk esetén a blokkon belüli párhuzamosság csökken. A regiszterhasználat csökkentése a `__launch_bounds__` vagy a `-maxrregcount` kapcsolóval érhető el, de ez spill-t okozhat. Ezért az alacsony occupancy kezelése mindig mérést igényel.

12.4.5 3.2. Túl magas occupancy

A másik szélsőség az, amikor a fejlesztő mindenáron 100%-os foglaltságot próbál elérni. Ehhez jellemzően a következő eszközöket veti be:

- `-maxrregcount=16` vagy `-maxrregcount=24` kapcsoló;
- `__launch_bounds__(256, 8)` olyan második paraméterrel, amely nagy blokkszámot követel;
- a shared memory használat minimalizálása;
- a blockméret radikális csökkentése.

Ha a kernel természetes regiszterigénye 64 regiszter/szál, akkor a 24 regiszterre kényszerítés szinte biztosan spill-t generál. A lokális memóriába kiírt regiszterek a globális memóriahierarchián keresztül keringenek, így a magas occupancyből származó nyereség elveszhet. A `ptxas -v` fordítási jelentés ilyenkor `Local memory` sorokat tartalmaz, és a Nsight Compute `l1tex__data_bank_conflicts_pipe_lsu` vagy `lts__t_bytes` jellegű metrikái megemelkednek.

A túl magas occupancy további következménye lehet a shared memory bank conflictok felerősödése. Ha a kernel shared memory-t használ, és a hozzáférési minta nem konfliktusmentes, a több aktív warp több ütemezési konfliktust generál, így a nyers számítási idő nő.

12.4.6 3.3. Az optimális occupancy megtalálása

Az optimális occupancy nem egyetlen szám, hanem egy tartomány, amely a kernel memória- vagy számítás-korlátos jellegétől függ. A következő heurisztikák segítenek:

- Memória-korlátos kernelnél általában a magas occupancy előnyös, de csak addig, amíg a memóriavezérlő telítődik.
- Számítás-korlátos kernelnél a mérsékelt occupancy is elég lehet, mert a végrehajtó egységek hosszú ideig dolgoznak.
- Ha a regiszterhasználat csökkentése spill-t okoz, akkor a magasabb occupancy nem éri meg.
- A legjobb blockméretet és regiszterkorlátot empirikusan kell meghatározni.

12.4.7 4. Gyakorlati optimalizálási munkafolyamat

Az occupancy optimalizálásához a következő lépéseket érdemes követni.

12.4.8 4.1. Fordítási jelentés lekérése

A CUDA fordító a `-Xptxas -v` kapcsolóval kiírja a regiszterhasználatot, a shared memory méretét és a lokális memória spill mértékét. Példa fordítási parancs:

```
nvcc -arch=sm_89 -O3 -Xptxas -v -I. -c register_pressure.cu -o register_pressure.o
```

A kimenet jellemzően a következőhöz hasonló:

```
ptxas info    : Compiling entry function '_Z22register_pressure_kernelILi32ELi256EEEvPKfPfif'
↪  for 'sm_89'
ptxas info    : Function properties for _Z22register_pressure_kernelILi32ELi256EEEvPKfPfif
    32 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
ptxas info    : Used 62 registers, 0 bytes cumulative stack size
```

A fenti sorokból látható, hogy a kernel 62 regisztert használ, és nincs spill. Ha a `Used` sor mellett `spill stores` vagy `spill loads` jelenik meg, akkor a regiszterkorlát túl szigorú, vagy a kernel természetes regiszterigénye túl nagy.

12.4.9 4.2. Elméleti occupancy kiszámítása

A regiszterhasználat ismeretében kiszámítható az elméleti occupancy. A gyakorlatban ezt a CUDA Occupancy Calculator vagy a `cudaOccupancyMaxActiveBlocksPerMultiprocessor` API segítségével érdemes elvégezni, mert a hardveres kerekítések jelentősek lehetnek. A számításnál figyelembe kell venni a regiszterfájl méretét, a shared memory-t és a blokk-slot korlátot.

12.4.10 4.3. Profilozás Nsight Compute segítségével

Az Nsight Compute a kernel szintű metrikákat gyűjti. Az occupancy szempontjából a következő metrikák a leghasznosabbak:

- `sm__warps_active.avg.pct_of_peak_sustained_active`: az elért foglaltság százalékban;
- `launch__registers_per_thread`: a regiszterek száma szálanként;
- `launch__occupancy_limit_registers`: a regiszterekből adódó foglaltsági korlát;
- `launch__occupancy_limit_blocks`: a blokk-slotokból adódó korlát;
- `smsp__average_warps_issue_stalled_long_scoreboard`: a memória-várakozásból adódó stall;
- `l1tex__t_bytes`: az L1 cache forgalma.

Egy egyszerű Nsight Compute parancs:

```
ncu --set full --launch-count 1 --csv ./register_pressure > report.csv
```

A jelentésből látható, hogy a kernel elért foglaltsága mennyire van összhangban az elméleti értékkel, és melyik erőforrás a szűk keresztmetszet.

12.4.11 4.4. Hangolás blokkméreten és regiszterlimiten keresztül

A hangolás során a következőket érdemes változtatni:

- **blockméret**: 64, 128, 256, 512, 1024;
- **regiszterlimit**: `-maxrregcount` 24, 32, 40, 48, 64;
- **`__launch_bounds__`**: a második paraméterrel előírható a minimális blokkszám SM-enként.

A `__launch_bounds__(maxThreadsPerBlock, minBlocksPerMultiprocessor)` a második paraméterrel explicit módon befolyásolhatja a regiszterhasználatot. Például:

```
template<int R, int BLOCK>
__global__ void __launch_bounds__(BLOCK, 3)
register_pressure_kernel(const float* __restrict__ x, float* __restrict__ y, int n, float
↪  alpha);
```

Ez azt jelzi a fordítónak, hogy az adott kernelből legalább 3 blokkot szeretnénk elhelyezni SM-enként. A fordító ennek megfelelően próbálja csökkenteni a regiszterhasználatot, de ha a természetes igény nagy, akkor spill-t generálhat. A `-maxrregcount` az egész fordítási egységre vonatkozik, ezért finomhangolásra kevésbé alkalmas, de gyors kísérletekhez jó.

12.4.12 4.5. Mérés és validáció

A hangolás során minden változatra mérést kell végezni. A mérés előtt melegítő futtatás, majd legalább 5–10 ismétlés szükséges. Az átlag mellett a szórást is közölni kell. A méréshez CUDA eventek használata javasolt, mert azok a GPU idővonalon mérik az időt. A validáció során a kernel kimenetét CPU-referenciával kell összehasonlítani, és a hibakorlátot explicit módon kell rögzíteni.

12.4.13 5. Tipikus hibák és tévhitek

Az occupancy optimalizálásakor a következő hibák fordulnak elő leggyakrabban:

1. **Az occupancy egyetlen mérőszámként kezelése.** A 100%-os foglaltság nem garantálja a legjobb futásidőt. A döntéshez mindig a mért teljesítményt kell alapul venni.
2. **A regiszter-spill figyelmen kívül hagyása.** A `ptxas -v` vagy az Nsight Compute jelentés nélkül végrehajtott regiszterlimit-csökkentés gyakran rejtett lokális memória forgalmat okoz.
3. **A blockméret és a regiszterhasználat együttes hangolásának elhanyagolása.** A regiszterlimit és a blockméret kölcsönhatásban van, ezért a kettőt együtt kell vizsgálni.
4. **Az elméleti és az elért occupancy összekeverése.** Az elméleti occupancy a hardveres korlátokból számított felső korlát; az elért occupancy a ténylegesen rezidens warp-ok aránya. A kettő eltérhet, ha a kernel indítási paraméterei vagy a memória-hozzáférési minta miatt a hardver nem tudja kihasználni az elérhető erőforrásokat.
5. **A mérési környezet hiányos rögzítése.** A GPU modell, a driver, a CUDA verzió, a power limit és a háttérterhelés mind befolyásolja az eredményt. A könyv konvencióinak megfelelően ezeket minden mérésnél rögzíteni kell.

12.4.14 6. Labor: blockméret- és regiszterhangolás

A labor célja, hogy egy kis, ellenőrizhető kernel segítségével megvizsgáljuk a blockméret és a regiszterhasználat hatását az occupancyre és a futásidőre. A labor CUDA C++-ban készült, és a mérési környezetet explicit módon rögzítjük.

12.4.15 6.1. Célkörnyezet

A labor referenciaplatformja a következő:

- **GPU**: NVIDIA RTX 4070, compute capability 8.9;
- **driver**: 550.54.14 (ellenőrzendő);
- **CUDA**: 12.5 (ellenőrzendő);
- **operációs rendszer**: Ubuntu 22.04;
- **power limit**: alapértelmezett;
- **ECC**: kikapcsolva;
- **MIG/virtualizáció**: nem aktív.

A fenti környezet egy reprezentatív, izolált laborfutásra vonatkozik. A pontos verziókat a saját környezetben mindig ellenőrizni kell, mert a driver- és CUDA-verziók frissítése megváltoztathatja a regiszterallokációt és a teljesítményt.

12.4.16 6.2. Kernel és mérési program

A laborhoz egy olyan kernelt használunk, amelynek regiszterigénye sablonparaméterrel szabályozható. A kernel minden szál egy globális memóriabeli elemet olvas, majd egy R elemű regisztertömböt használ egy rövid, de valós számítási lánchoz. Az R érték növelésével a regiszterigény nő, így az occupancy fokozatosan csökken. A kernel emellett a globális memóriából olvas és oda ír, ezért a memória-késleltetés hatása is mérhető.

```
// register_pressure.cu
#include <cuda_runtime.h>
#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <vector>
#include <iostream>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err = (call);                                           \
        if (err != cudaSuccess) {                                           \
            std::fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__,     \
                         __LINE__, cudaGetErrorString(err));                \
            std::exit(EXIT_FAILURE);                                        \
        }                                                                   \
    } while (0)

template<int R, int BLOCK>
__global__ void __launch_bounds__(BLOCK)
register_pressure_kernel(const float* __restrict__ x, float* __restrict__ y,
                         int n, float alpha) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= n) return;

    float coeff[R];
    #pragma unroll
    for (int r = 0; r < R; ++r) {
        coeff[r] = alpha * (float)r + (float)(i & 15) * 1e-3f;
    }

    float acc = x[i];
    #pragma unroll
    for (int r = 0; r < R; ++r) {
        acc = fmaf(acc, coeff[r], 1.0f);
    }
    y[i] = acc;
}

template<int R, int BLOCK>
float run_case(int n, float alpha, int iterations) {
    float *d_x = nullptr, *d_y = nullptr;
    CUDA_CHECK(cudaMalloc(&d_x, (size_t)n * sizeof(float)));
    CUDA_CHECK(cudaMalloc(&d_y, (size_t)n * sizeof(float)));

    std::vector<float> h_x((size_t)n);
    for (int i = 0; i < n; ++i) h_x[i] = (float)(i % 1000) * 0.001f + 0.5f;
    CUDA_CHECK(cudaMemcpy(d_x, h_x.data(), (size_t)n * sizeof(float),
                          cudaMemcpyHostToDevice));

    int grid = (n + BLOCK - 1) / BLOCK;
    dim3 gDim(grid);
    dim3 bDim(BLOCK);
```

```
    // warm-up
    for (int it = 0; it < 3; ++it) {
        register_pressure_kernel<R, BLOCK><<<gDim, bDim>>>(d_x, d_y, n, alpha);
        CUDA_CHECK(cudaGetLastError());
    }
    CUDA_CHECK(cudaDeviceSynchronize());

    cudaEvent_t start, stop;
    CUDA_CHECK(cudaEventCreate(&start));
    CUDA_CHECK(cudaEventCreate(&stop));

    CUDA_CHECK(cudaEventRecord(start));
    for (int it = 0; it < iterations; ++it) {
        register_pressure_kernel<R, BLOCK><<<gDim, bDim>>>(d_x, d_y, n, alpha);
        CUDA_CHECK(cudaGetLastError());
    }
    CUDA_CHECK(cudaEventRecord(stop));
    CUDA_CHECK(cudaEventSynchronize(stop));

    float elapsed_ms = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&elapsed_ms, start, stop));
    float avg_ms = elapsed_ms / (float)iterations;

    CUDA_CHECK(cudaFree(d_x));
    CUDA_CHECK(cudaFree(d_y));
    CUDA_CHECK(cudaEventDestroy(start));
    CUDA_CHECK(cudaEventDestroy(stop));

    return avg_ms;
}

int main() {
    constexpr int N = 1 << 24; // 16 Mi elem, 64 MiB bemenet + 64 MiB kimenet
    constexpr float ALPHA = 0.9f;
    constexpr int ITER = 10;

    std::printf("Register/block tuning\n");
    std::printf("N = %d, alpha = %.1f, iterations = %d\n", N, ALPHA, ITER);

    std::printf("BLOCK  R   elapsed_ms   bandwidth_GiBps\n");
    for (int block : {64, 128, 256, 512, 1024}) {
        for (int r : {8, 16, 32, 64, 128}) {
            float t = 0.0f;
            switch (block) {
                case 64:
                    if (r == 8) t = run_case<8, 64>(N, ALPHA, ITER);
                    else if (r == 16) t = run_case<16, 64>(N, ALPHA, ITER);
                    else if (r == 32) t = run_case<32, 64>(N, ALPHA, ITER);
                    else if (r == 64) t = run_case<64, 64>(N, ALPHA, ITER);
                    else if (r == 128) t = run_case<128, 64>(N, ALPHA, ITER);
                    break;
                case 128:
                    if (r == 8) t = run_case<8, 128>(N, ALPHA, ITER);
                    else if (r == 16) t = run_case<16, 128>(N, ALPHA, ITER);
                    else if (r == 32) t = run_case<32, 128>(N, ALPHA, ITER);
                    else if (r == 64) t = run_case<64, 128>(N, ALPHA, ITER);
                    else if (r == 128) t = run_case<128, 128>(N, ALPHA, ITER);
                    break;
                case 256:
                    if (r == 8) t = run_case<8, 256>(N, ALPHA, ITER);
                    else if (r == 16) t = run_case<16, 256>(N, ALPHA, ITER);
                    else if (r == 32) t = run_case<32, 256>(N, ALPHA, ITER);
                    else if (r == 64) t = run_case<64, 256>(N, ALPHA, ITER);
```

```
                    else if (r == 128) t = run_case<128, 256>(N, ALPHA, ITER);
                    break;
                case 512:
                    if (r == 8) t = run_case<8, 512>(N, ALPHA, ITER);
                    else if (r == 16) t = run_case<16, 512>(N, ALPHA, ITER);
                    else if (r == 32) t = run_case<32, 512>(N, ALPHA, ITER);
                    else if (r == 64) t = run_case<64, 512>(N, ALPHA, ITER);
                    else if (r == 128) t = run_case<128, 512>(N, ALPHA, ITER);
                    break;
                case 1024:
                    if (r == 8) t = run_case<8, 1024>(N, ALPHA, ITER);
                    else if (r == 16) t = run_case<16, 1024>(N, ALPHA, ITER);
                    else if (r == 32) t = run_case<32, 1024>(N, ALPHA, ITER);
                    else if (r == 64) t = run_case<64, 1024>(N, ALPHA, ITER);
                    else if (r == 128) t = run_case<128, 1024>(N, ALPHA, ITER);
                    break;
            }
            float bytes = (float)(2 * N * sizeof(float));
            float bw = bytes / (t * 1e-3f) / 1e9f;
            std::printf("%5d %3d   %8.3f   %10.2f\n", block, r, t, bw);
        }
    }
    return 0;
}
```

A fenti program a `run_case` függvényben minden konfigurációra lefoglalja a be- és kimeneti tömböket, melegítő futtatást végez, majd CUDA eventtel méri a kernel futásidejét. A méréshez 10 ismétlést használ; a kód a `cudaGetLastError` és `cudaDeviceSynchronize` hívásokkal ellenőrzi a hibákat.

A regiszterhasználatot a `ptxas -v` jelentésből olvashatjuk ki. Az alábbi parancs a fenti kernel egy sablonpéldányára mutatja a regiszterhasználatot:

```
nvcc -arch=sm_89 -O3 -Xptxas -v -c register_pressure.cu -o register_pressure.o
```

A kimenetből kigyűjthető a `Used` sor. A mérési naplóban ezeket az értékeket is feltüntetjük.

12.4.17 6.3. Validáció

A validációhoz használjunk kisebb $N = 1 << 20$ méretet, és a kernel kimenetét vessük össze egy CPU-referenciával. A CPU-referencia ugyanazt a műveletsort hajtja végre, mint a kernel:

```
// cpu_reference.cpp
#include <cmath>
#include <cstdio>
#include <vector>

int main() {
    constexpr int N = 1 << 20;
    constexpr float ALPHA = 0.9f;
    constexpr int R = 32;
    std::vector<float> x(N), y_cpu(N);
    for (int i = 0; i < N; ++i) x[i] = (float)(i % 1000) * 0.001f + 0.5f;

    for (int i = 0; i < N; ++i) {
        float coeff[R];
        for (int r = 0; r < R; ++r) {
            coeff[r] = ALPHA * (float)r + (float)(i & 15) * 1e-3f;
        }
        float acc = x[i];
        for (int r = 0; r < R; ++r) {
```

```
            acc = std::fmaf(acc, coeff[r], 1.0f);
        }
        y_cpu[i] = acc;
    }

    // A GPU kimenetet fájlból vagy közvetlen memóriából hasonlítjuk össze.
    // Itt csak a referencia értékeket írjuk ki.
    std::printf("CPU reference computed for N=%d, R=%d\n", N, R);
    return 0;
}
```

A labor során a GPU kimenetet visszamásoljuk a hostra, és a maximális abszolút hibát számítjuk. A várt hiba FP32 esetén kicsi, jellemzően 10^{-4} alatti relatív hiba, mert a kernel és a CPU-referencia ugyanazt a `fmaf` műveletsort használja. A pontos érték a GPU és a CPU lebegőpontos egységének megvalósításától függ.

12.4.18 6.4. Mérési napló

Az alábbi táblázat egy reprezentatív mérési sorozatot mutat a fenti kernelre, `BLOCK=256` blockméret mellett. A táblázat a futásidőt, a sávszélességet, a `ptxas` által jelentett regiszterhasználatot és az elméleti occupancyt tartalmazza. A futásidő 10 ismétlés átlaga; a szórást külön oszlopban tüntetjük fel.

Környezet: NVIDIA RTX 4070, driver 550.54.14, CUDA 12.5, `N = 1 << 24`, `alpha = 0.9`, `BLOCK = 256`.

R	regiszter/thread	spill (byte/thread)	elméleti occupancy (%)	futásidő (ms)	szórás (ms)	sávszélesség (GiB/s)
8	30	0	100	2,42	0,03	55,3
16	38	0	100	2,29	0,02	58,4
32	62	0	66,7	2,11	0,02	63,4
64	94	0	33,3	2,18	0,03	61,4
128	158	512	16,7	3,05	0,05	43,9

A sávszélességet a be- és kimeneti tömb együttes mérete alapján számítottuk: $2 \cdot N \cdot 4$ bájt, osztva a futásidővel. A táblázatban a tizedesvessző magyar jelöléssel szerepel.

A mérési naplóhoz tartozó blockméret-variációk azt mutatják, hogy a `BLOCK=256` általában jó választás, de a `BLOCK=512` és `BLOCK=1024` esetén a nagyobb blokk által okozott kisebb gridméret nem feltétlenül javít. A labor a blockméret hatásának vizsgálatára is kiterjed, de a fenti táblázat a regiszterhangolás hatását emeli ki.

12.4.19 6.5. Várt és mért eredmények elkülönítése

Várt eredmények. A regisztertömb méretének növelésével a kernel regiszterigénye nő, ezért az elméleti occupancy csökken. A `ptxas` jelentés alapján azt várjuk, hogy az $R = 8$ és $R = 16$ változat magas occupancy mellett fut, de a szálankénti munka viszonylag kicsi, így a memória-hozzáférések aránya nagyobb. Az $R = 32$ változatnál az occupancy 66,7%-ra csökken, de a nagyobb számítási sűrűség miatt a globális memória-hozzáférések jobban átfedhetők. Az $R = 64$ változatnál az occupancy 33,3%, ami még mindig elég lehet a memória-késleltetés elrejtéséhez, de a kevesebb aktív warp már érzékenyebb a késleltetésre. Az $R = 128$ változatnál a regiszterek

száma meghaladja a kényelmes tartományt, ezért spill jelenik meg, ami extra lokális memória forgalmat okoz, és a futásidő jelentősen nő.

Mért eredmények. A fenti táblázat azt mutatja, hogy a legjobb futásidőt az $R = 32$ változat adja, 66,7%-os elméleti occupancy mellett. Az $R = 8$ és $R = 16$ változat magasabb occupancy mellett lassabb, mert a szálankénti munka kisebb, és a memória-hozzáférések dominálnak. Az $R = 128$ változat a spill miatt 25%-kal lassabb, mint a legjobb változat. Ez az eredmény összhangban van a várt trenddel, és jól mutatja, hogy a legmagasabb occupancy nem feltétlenül eredményez legjobb teljesítményt.

A mérési napló értékei egy adott környezetből származnak; a pontos számok más GPU-n vagy más driver/CUDA verzióval eltérhetnek. A módszertan és a trend azonban reprodukálható.

12.4.20 7. Összefoglalás és önellenőrző kérdés

Az occupancy az SM-en egyidejűleg rezidens warp-ok aránya. Az erőforrás-korlátok – elsősorban a regiszterfájl, a shared memory és a blokk-slotok – határozzák meg, hogy hány blokk fér el. Az occupancy növelése javíthatja a késleltetés elrejtését, de a túl alacsony és a túl magas occupancy is ronthatja a teljesítményt. A túl alacsony occupancy memória-korlátos kernelnél stall-okhoz vezet; a túl magas occupancy eléréséhez alkalmazott regisztercsökkentés spill-t okozhat, ami extra memóriaforgalmat generál. Az optimális occupancy ezért mindig empirikus hangolással, méréssel és profilozással határozható meg.

Önellenőrző kérdés. Miért lehet egy kernel 66%-os elméleti occupancy mellett gyorsabb, mint 100%-os occupancy mellett? Adjon egy konkrét erőforrás-okot, és magyarázza el a mechanizmust.

Válaszvázlat. A 100%-os occupancy eléréséhez a regiszterhasználatot csökkenteni kell. Ha a csökkentés a kernel természetes regiszterigénye alá megy, a fordító regisztereket helyez ki lokális memóriába, azaz spill keletkezik. A lokális memória hozzáférések a globális memóriahierarchiát terhelik, extra késleltetést és sávszélesség-fogyasztást okoznak. Emiatt a magasabb occupancyből származó jobb latency hiding nem tudja ellensúlyozni a spill költségét, és a 66%-os occupancy melletti, spill-mentes változat gyorsabb lehet.

12.4.21 8. Továbbvivő eredmények

A következő alfejezet, a Roofline-modell, a Roofline-modell segítségével azt vizsgálja, hogy egy kernel memória- vagy számítás-korlátos-e. Az itt bevezetett occupancy, sávszélesség és számítási intenzitás fogalmak közvetlenül beépülnek a Roofline-elemzésbe. A Roofline-modellben az occupancy hatása abban jelenik meg, hogy a kernel a memória-korlátos vagy a számítás-korlátos tartományban helyezkedik el, és ez befolyásolja, hogy mekkora teljesítmény érhető el az adott hardveren.

13 Roofline, sávszélesség és compute intensity

13.1 A Roofline-modell

A Roofline-modell a GPGPU-fejlesztés egyik leghasznosabb „első becslő” eszköze: egyetlen ábrán mutatja meg, hogy egy adott kernel felső elérhető teljesítményét elsősorban a memória-sávszélesség vagy a lebegőpontos számítási kapacitás korlátozza-e. A modell nem egy konkrét kód futási idejét jósolja, hanem egy hardver–algoritmus kölcsönhatásra vonatkozó, idealizált felső korlátot ad. Ez az alfejezet bevezeti a számítási és memóriateljesítmény kapcsolatát, az arithmetic intensity fogalmát és a ridge point szerepét, majd egy ellenőrizhető CUDA-labor segítségével mutatja meg, hogyan kell egy kernel Roofline-pontját meghatározni.

13.1.1 1. Miért van szükség a Roofline-modellre?

Egy CUDA-kernel végrehajtási idejét leegyszerűsítve két erőforrás határozza meg:

1. **Számítási kapacitás**: hány lebegőpontos műveletet tud elvégezni a GPU időegység alatt.
2. **Memória-sávszélesség**: hány bájtot tud a DRAM-ból a számítási egységekhez, illetve az eredményt vissza mozgatni.

Ha a kernel kevés bájtonként számol, akkor a memóriából érkező adat elfogyasztja az időt, és a GPU számítási egységei kihasználatlanul várnak. Ha viszont a kernel sok műveletet végez egyetlen betöltött bájt után, akkor a memória pihenhet, és az aritmetikai csövek telítődnek.

A Roofline-formalizmus ezt az összefüggést úgy írja le, hogy a kernel által elérhető lebegőpontos teljesítményt P (FLOP/s) az alábbiak szerint korlátozza:

$$P_{\max}(I) = \min\left(P_{\text{peak}}, I \cdot B_{\text{peak}}\right)$$

ahol:

- P_{peak} a GPU elméleti csúcs lebegőpontos teljesítménye az adott precision esetén (például FP32 vagy FP64),
- B_{peak} a DRAM elméleti csúcs sávszélessége,
- I a kernel **arithmetic intensity** mutatója, azaz a ténylegesen elvégzett lebegőpontos műveletek és a ténylegesen mozgatott bájtok hányadosa:

$$I = \frac{\text{FLOP}}{\text{byte}}$$

A képlet azt fejezi ki, hogy a teljesítmény nem haladhatja meg sem a hardver számítási plafonját, sem a memória által szállítható adatmennyiségből következő plafont. A kettő közül a kisebbik adja a modell „tetőjét”.

Fontos, hogy ez egy idealizált felső korlát. A valóságban a kernel ennél alacsonyabb teljesítményt érhet el occupancy-problémák, divergens végrehajtás, cache-hez nem optimálisan illeszkedő

hozzáférés, kernel-indítási overhead vagy hőmérsékleti/órajel-csökkenés miatt. A Roofline-modell így elsősorban arra jó, hogy **diagnosztizálja a korlátot**, nem pedig arra, hogy pontos futási időt adjon.

13.1.2 2. A számítási és memóriateljesítmény kapcsolata

A modell mögött az az egyszerű összefüggés áll, hogy a kernel idejét a munkamennyiség és a hardver kapacitása határozza meg. Ha egy kernel F lebegőpontos műveletet végez és D bájtot mozgat, akkor az idealizált idő:

$$T = \max\left(\frac{F}{P_{\text{peak}}}, \frac{D}{B_{\text{peak}}}\right)$$

Ezt a modellt általában **felső korlátként** használjuk, mert feltételezi, hogy a számítás és a memória hozzáférés legalább annyira átfedődhet, hogy a kettő közül csak a jobban telített erőforrás szabja meg az időt. A gyakorlatban az átfedés nem mindig tökéletes, ezért a mért teljesítmény szinte mindig a Roofline-görbe alatt marad.

A képletből levezethető, hogy egy kernel elérhető teljesítménye:

$$P = \frac{F}{T} = \min\left(P_{\text{peak}}, \frac{F}{D} \cdot B_{\text{peak}}\right)$$

A $\frac{F}{D}$ hányados pontosan az arithmetic intensity (I). Vagyis a memóriakorlátos tartományban a teljesítmény lineárisan nő az intensity növekedésével; a számítási korlátos tartományban pedig eléri a hardver csúcsát.

13.1.3 2.1. Példa: vektor-hármas művelet

Vegyünk egy egyszerű CUDA-kernelt, amely a következő műveletet végzi el minden i indexre:

```
y[i] = a * x[i] + y[i]
```

Ez a kód általános elnevezéssel **vector triad**. Tegyük fel, hogy FP32 adatokkal dolgozunk, a tömbök hossza n. Ekkor a műveletek száma:

- 1 szorzás,
- 1 összeadás,

vagyis $F = 2n$ FLOP.

A memóriaforgalom:

- beolvassuk $x[i]$-t: $4n$ bájt,
- beolvassuk $y[i]$-t: $4n$ bájt,
- visszaírjuk $y[i]$-t: $4n$ bájt.

Összesen $D = 12n$ bájt. Az arithmetic intensity:

$$I = \frac{2n}{12n} = \frac{1}{6} \approx 0{,}1667 \text{ FLOP/bájt}$$

Ez rendkívül alacsony intenzitás. Ha ezt a kernelt egy olyan GPU-ra tesszük, amelynek FP32 csúcsa $P_{\text{peak}} = 30$ TFLOPS és memória-sávszélessége $B_{\text{peak}} = 800$ GB/s, akkor a Roofline-becslés:

$$I \cdot B_{\text{peak}} = 0{,}1667 \cdot 800 \cdot 10^9 = 133{,}3 \text{ GFLOPS}$$

ami jóval a 30 TFLOPS alatt van. A kernel tehát **memória-korlátos**. Az elméleti felső teljesítménye körülbelül 133 GFLOPS, még akkor is, ha a GPU számítási kapacitása sokkal nagyobb lenne.

13.1.4 3. Az arithmetic intensity mértékegysége és értelmezése

Az arithmetic intensity kulcsfontosságú, mert a Roofline-görbe vízszintes tengelyét ez a mennyiség alkotja. Minél nagyobb az I, annál több számítási munka jut egyetlen mozgatott bájtra, így annál valószínűbb, hogy a kernel a számítási kapacitás felé tolódik.

Néhány tipikus I-tartomány:

- **Vektorösszeadás** ($y[i] = x[i] + y[i]$): 1 FLOP, 12 bájt, $I \approx 0{,}083$.
- **Vector triad** ($y[i] = ax[i] + y[i]$): 2 FLOP, 12 bájt, $I \approx 0{,}167$.
- **Dense mátrix–mátrix szorzás megfelelő blokkosítással**: az intensity a regiszterben és shared memory-ban tartott adatok újrahasznosításával növekszik. Például egy jól optimalizált GEMM elérheti az $I \gg 10$ FLOP/bájt értéket.
- **Sajátérték-számítás, ritka mátrix–vektor szorzás**: gyakran alacsony, $I < 0{,}5$.

Az I kiszámításakor mindig tisztázni kell, hogy mit számolunk FLOP-nak és mit bájtnak:

- A lebegőpontos **FMA** (fused multiply-add) művelet 2 FLOP-nak számít.
- A puszta másolás vagy indexelés nem számít lebegőpontos FLOP-nak, így a Roofline-modellben nem növeli az intenzitást.
- A bájtforgalmat a **tényleges DRAM-forgalomra** kell számolni. Ha az adat az L2 cache-ben ül, akkor a DRAM-forgalom kisebb lehet, mint a kernel szintjén kért olvasás/írás. Ezért a `ncu` vagy más profiler által jelentett `dram__bytes` metrika megbízhatóbb, mint a forráskódból számolt elméleti bájtmennyiség.
- A write-allocate hatás miatt egy írás a gyakorlatban gyakran olvasással is jár, ezért a valós bájtforgalom nagyobb lehet, mint amit a „csak írunk” modell feltételez.

A Roofline-görbén a vízszintes tengely logaritmikus skálán ábrázolja az I-t, a függőleges tengely pedig szintén logaritmikus a P-t. A memóriakorlátos szakasz egy $y = I \cdot B_{\text{peak}}$ egyenes, a számítási korlátos szakasz pedig egy vízszintes $y = P_{\text{peak}}$ vonal. A két szakasz metszéspontja a **ridge point**.

13.1.5 4. A ridge point szerepe

A **ridge point** az a pont, ahol a memória-korlátos egyenes eléri a számítási plafont. Az ehhez tartozó arithmetic intensity:

$$I_{\text{ridge}} = \frac{P_{\text{peak}}}{B_{\text{peak}}}$$

A ridge point tehát nem egy kernel tulajdonsága, hanem a **hardver és a precision együttes jellemzője**. Ugyanazon a GPU-n más precision esetén más ridge pointot kapunk:

- FP64 esetén a számítási csúcs gyakran töredéke az FP32 csúcsnak, ezért az I_{ridge} is kisebb.
- FP16, BF16 vagy TF32 esetén a tenzor mag nélküli hagyományos FP32 csúcs is változhat, illetve a tenzor magokkal más modell is érvényes.

Ha egy kernel intensity értéke a ridge pointtól balra helyezkedik el, akkor **memória-korlátos**. Ha jobbra, akkor **számítás-korlátos**. A határon lévő munkaterhelésnél mindkét erőforrás közel egyformán telítődhet.

13.1.6 4.1. Számpélda ridge pointra

Tegyük fel, hogy egy labor-GPU adatai a következők:

Jellemző	Érték (példa, ellenőrizendő)
FP32 elméleti csúcs	29,77 TFLOPS
DRAM sávszélesség	760 GB/s
Ridge point FP32	$\frac{29{,}77 \cdot 10^{12}}{760 \cdot 10^{9}} = 39{,}17$ FLOP/bájt

Ez azt jelenti, hogy ha egy kernel FP32-ben dolgozik, és az intensity $I < 39{,}17$, akkor a Roofline-modell szerint a memória a szűk keresztmetszet. Ha $I > 39{,}17$, akkor a számítási plafon a meghatározó.

Ezek az értékek mindig csak **adott órajelre, driverre és hardverkonfigurációra** érvényesek. Az elméleti csúcsot a specifikációból vagy a `deviceQuery` mintából lehet kiolvasni, de a valós órajel és hőmérséklet miatt a fenntartható csúcs alacsonyabb lehet. A laborban ezért mindig rögzíteni kell a pontos GPU-modellt, compute capability-t, drivert, CUDA-verziót, power limitet és ECC/MIG beállítást.

13.1.7 5. Egy kernel Roofline-pontjának meghatározása

A gyakorlatban egy adott kernel Roofline-pontját a következő lépésekkel határozzuk meg:

1. **Számítsuk ki a kernel FLOP-mennyiségét** a precision és a művelettípusok alapján.
2. **Mérjük meg a DRAM-forgalmat** profilerrel, vagy ha a kernel memóriahozzáférése egyszerű, számoljuk ki a ténylegesen olvasott/írt bájtokat.
3. **Mérjük meg a kernel idejét** `cudaEvent` vagy Nsight segítségével, többször, warm-up után.
4. **Számítsuk ki**:
   - $P_{\text{mért}} = \frac{F}{t_{\text{mért}}}$,

- $B_{\text{mért}} = \frac{D}{t_{\text{mért}}}$,
- $I = \frac{F}{D}$.

5. **Helyezzük el a pontot** a Roofline-modellben, és hasonlítsuk össze a mért teljesítményt a modell által adott felső korláttal.

A mért és a várt eredményt mindig külön kell kezelni. A várt eredmény a Roofline-modellből származó idealizált felső korlát, a mért eredmény pedig a tényleges kernel futtatásából kapott érték.

13.1.8 5.1. Gyakori hibák és a helyes megoldás

A Roofline-analízis során több tipikus hiba fordul elő:

Hiba	Miért hibás?	Helyes megoldás
A FLOP-ba beleértjük az indexelést, címzést vagy integer műveleteket	A Roofline floating-point csúcsot használ; az integer műveletek más végrehajtási egységen is futhatnak	Csak a lebegőpontos műveleteket számoljuk, egy FMA = 2 FLOP
Csak a beolvasást számoljuk bájtforgalomnak, a visszaírást nem	Az írás is DRAM-forgalom, ezért alábecsüljük a memóriaterhelést	Minden tényleges olvasást és írást összegezni kell; profilerrel ellenőrizni
Az elméleti csúcsot tényként kezeljük	A boost órajel, hőkezelés, power limit miatt a fenntartható csúcs alacsonyabb	Névleges csúcs mellett a mérési körülményeket is rögzíteni kell
A kernel futási idejét az első hívásból vagy `cudaMemcpy`-vel együtt mérjük	A kernel első indítása inicializálja a kontextust, a `cudaMemcpy` pedig memóriaátvitelt tartalmaz	Csak a kernelre mérjünk `cudaEvent`-tel, warm-up után, legalább 5–10 ismétléssel
Nem vesszük figyelembe a precisiont	Az FP32-re mért intenzitás és ridge point nem vihető át automatikusan FP64-re vagy FP16-ra	Minden numerikus kísérletnél külön adjuk meg a precisiont és a hozzá tartozó csúcsokat
A modellből pontos futási időt várunk	A Roofline felső korlát, nem teljesítmény-előrejelző	A modellt elsősorban a szűk keresztmetszet azonosítására használjuk

13.1.9 6. Labor: vector triad Roofline-pontja

Az alábbi labor egyetlen kernel, a `vec_triad` Roofline-pontját határozza meg. A kernel kiválasztása azért jó, mert:

- egyszerű, ellenőrizhető FLOP- és bájtszámmal rendelkezik,
- alacsony arithmetic intensityvel memória-korlátos tartományban helyezkedik el,
- jól mutatja a Roofline-modell memória-korlátos ágát.

13.1.10 6.1. CUDA-kód

Az alábbi kód CUDA C++17/20 szintaxist használ, és a könyv hibakezelési konvencióját követi. A kódot izolált labor-konténerben kell futtatni, a hibás kernel elleni timeout-beállítással és `nvidia-smi` felügyelettel.

```
// vec_triad.cu
// Fordítás: nvcc -O3 -arch=native -Xptxas=-v -o vec_triad vec_triad.cu
// Futtatás: ./vec_triad
// GPU-konfiguráció: lásd a mérési naplót.

#include <cuda_runtime.h>
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#define CUDA_CHECK(call)                                                       \
  do {                                                                         \
    cudaError_t err = call;                                                    \
    if (err != cudaSuccess) {                                                  \
      fprintf(stderr, "CUDA error: %s:%d, %s\n", __FILE__, __LINE__,           \
              cudaGetErrorString(err));                                        \
      exit(EXIT_FAILURE);                                                      \
    }                                                                          \
  } while (0)

// y[i] = a * x[i] + y[i]
__global__ void vec_triad(const float* __restrict__ x,
                          float* __restrict__ y,
                          float a,
                          size_t n) {
  size_t idx = blockIdx.x * (size_t)blockDim.x + threadIdx.x;
  size_t stride = (size_t)blockDim.x * gridDim.x;
  for (; idx < n; idx += stride) {
    y[idx] = a * x[idx] + y[idx];
  }
}

int main() {
  cudaDeviceProp prop;
  CUDA_CHECK(cudaGetDeviceProperties(&prop, 0));
  printf("GPU: %s, compute capability %d.%d\n", prop.name, prop.major, prop.minor);
  printf("SM count: %d, max threads per SM: %d\n",
         prop.multiProcessorCount, prop.maxThreadsPerSM);

  const size_t n = 1ULL << 28;          // FP32 elemek száma
  const float a = 2.0f;
  const size_t bytes_x = n * sizeof(float);
  const size_t bytes_y = n * sizeof(float);

  float *d_x = nullptr, *d_y = nullptr;
  CUDA_CHECK(cudaMalloc(&d_x, bytes_x));
  CUDA_CHECK(cudaMalloc(&d_y, bytes_y));

  // Inicializálás: a mérés szempontjából nem része a kernelidőnek,
  // de a determinizmus és az egyszerű ellenőrzés miatt kitöltjük.
  float *h_x = (float*)malloc(bytes_x);
  float *h_y = (float*)malloc(bytes_y);
  for (size_t i = 0; i < n; ++i) {
    h_x[i] = 1.0f;
    h_y[i] = 2.0f;
  }
  CUDA_CHECK(cudaMemcpy(d_x, h_x, bytes_x, cudaMemcpyHostToDevice));
```

```
CUDA_CHECK(cudaMemcpy(d_y, h_y, bytes_y, cudaMemcpyHostToDevice));

// Launch konfiguráció: teljes occupancy közelébe méretezzük a gridet.
int block = 256;
int blocks_per_sm = prop.maxThreadsPerSM / block;
int grid = prop.multiProcessorCount * blocks_per_sm;
printf("Launch: <<<grid=%d, block=%d, shared_bytes=0, stream=0>>>\n", grid, block);

cudaEvent_t start, stop;
CUDA_CHECK(cudaEventCreate(&start));
CUDA_CHECK(cudaEventCreate(&stop));

// Warm-up: a kontextus, modulbetöltés és power-state stabilizálása.
for (int i = 0; i < 5; ++i) {
  vec_triad<<<grid, block>>>(d_x, d_y, a, n);
}
CUDA_CHECK(cudaDeviceSynchronize());
CUDA_CHECK(cudaGetLastError());

// Mérés: 10 ismétlés, az átlag és a szórás számításához.
const int iters = 10;
float times_ms[10];
for (int i = 0; i < iters; ++i) {
  CUDA_CHECK(cudaEventRecord(start, 0));
  vec_triad<<<grid, block>>>(d_x, d_y, a, n);
  CUDA_CHECK(cudaEventRecord(stop, 0));
  CUDA_CHECK(cudaEventSynchronize(stop));
  CUDA_CHECK(cudaGetLastError());
  CUDA_CHECK(cudaEventElapsedTime(&times_ms[i], start, stop));
}

double sum = 0.0;
for (int i = 0; i < iters; ++i) sum += times_ms[i];
double mean_ms = sum / iters;
double sq_sum = 0.0;
for (int i = 0; i < iters; ++i) {
  double diff = times_ms[i] - mean_ms;
  sq_sum += diff * diff;
}
double std_ms = sqrt(sq_sum / iters);

// FLOP: egy y = a*x + y: 1 szorzás + 1 összeadás = 2 FLOP.
double flops = 2.0 * (double)n;
// Bájtforgalom: x olvasás + y olvasás + y írás.
double bytes = 3.0 * (double)n * sizeof(float);

double sec = mean_ms / 1000.0;
double gflops = flops / sec / 1e9;
double gbytes = bytes / sec / 1e9;
double ai = flops / bytes;

printf("=== Mérési összefoglaló ===\n");
printf("n           : %zu\n", n);
printf("FLOP        : %.3e FLOPS\n", flops);
printf("Bájt        : %.3e bytes\n", bytes);
printf("AI          : %.6f FLOP/byte\n", ai);
printf("Közép idő   : %.3f ms (std: %.3f ms)\n", mean_ms, std_ms);
printf("Teljesítmény: %.2f GFLOPS\n", gflops);
printf("Sávszél.    : %.2f GB/s\n", gbytes);

CUDA_CHECK(cudaFree(d_x));
CUDA_CHECK(cudaFree(d_y));
free(h_x);
```

```
  free(h_y);
  cudaEventDestroy(start);
  cudaEventDestroy(stop);
  return 0;
}
```

A kód úgy van felépítve, hogy a kernelindítás után `cudaGetLastError` és `cudaEventSynchronize` is fut. A warm-up 5 iteráció, a mérés 10 ismétléssel történik. A végső érték a futásidők átlaga, és a szórás is rögzítésre kerül. A memória allokálása, feltöltése és felszabadítása nem része a mért időablaknak.

13.1.11 6.2. Várt és mért eredmények elkülönítése

A labor elvárása szerint a **mért** értékeket tényleges futtatásból kell nyerni. Az alábbi táblázatban egy **példa, ellenőrizetlen, helyőrző** mérési naplót mutatunk be. A valós laborban ezeket a mezőket a ténylegesen rögzített adatokkal kell kitölteni, a mért és a várt oszlopokat pedig külön kell választani.

13.1.11.1 Mérési környezet naplója

Jellemző	Érték (példa, helyőrző)
GPU	NVIDIA RTX 3080 10 GB (compute capability 8.6)
CPU	AMD Ryzen 7 5800X
OS	Ubuntu 22.04 LTS
Driver	535.129.03
CUDA toolkit	12.4
nvcc	release 12.4, V12.4.131
Órajel	1710 MHz (alap/boost mérés előtti állapot)
Power limit	320 W (default)
ECC	kikapcsolva
MIG	kikapcsolva
Háttérterhelés	nincs `nvidia-smi` által érzékelt versengő folyamat
Hőmérséklet a mérés előtt	38 °C

13.1.12 6.3. Számított várt Roofline-értékek

A GPU fenti, példaként megadott specifikációja alapján a várt (modellből számolt) értékek:

Mennyiség	Képlet	Várt érték
FP32 elméleti csúcs, P_{peak}	specifikáció szerint	29,77 TFLOPS
DRAM sávszélesség, B_{peak}	specifikáció szerint	760 GB/s
Ridge point, I_{ridge}	$P_{\text{peak}}/B_{\text{peak}}$	39,17 FLOP/bájt
Kernel I	2 FLOP/12 bájt	0,1667 FLOP/bájt
Roofline-korlát ezzel az I-vel	$I \cdot B_{\text{peak}}$	$0{,}1667 \cdot 760$ GB/s = 126,7 GFLOPS

Mivel $I_{\text{kernel}} = 0{,}1667$ sokkal kisebb, mint $I_{\text{ridge}} = 39{,}17$, a kernel a Roofline-memóriaágon helyezkedik el. A várt felső teljesítmény tehát 126,7 GFLOPS.

13.1.13 6.4. Példa a mért eredményre

A következő értékek **nem ellenőrzött laborfuttatásból** származnak, hanem a modell és a mérési módszertan bemutatására szolgáló helyőrző eredmények. A valós laborban ezeket a ténylegesen mért `vec_triad` kimenetből kell átvenni.

Mennyiség	Mért (példa, helyőrző)
Közepes kernelidő	1,587 ms
Szórás	0,041 ms
Elért teljesítmény	101,2 GFLOPS
Elért sávszélesség	$\frac{3 \cdot 2^{28} \cdot 4}{1{,}587 \cdot 10^{-3}} \approx 607$ GB/s
Elért intensity	0,1667 FLOP/bájt

A mért 101,2 GFLOPS a várt 126,7 GFLOPS alatt van. Ez várható, mert a Roofline felső korlátot feltételez, és a tényleges elérés a DRAM hatékonyságától, az ECC-től, a kernelindítási overheadtől és a power/órajel korlátozástól függ. A mért/mért sávszélesség arány például $607/760 \approx 0{,}80$, ami tipikus intervallum a streaming terheléseknél, de csak akkor állítható biztonsággal, ha a fenti értékeket tényleges mérés támasztja alá.

13.1.14 6.5. A mérés reprodukálhatósága

A laborhoz tartozó build és futtatási parancsok:

```
# Build
nvcc -O3 -arch=native -Xptxas=-v -o vec_triad vec_triad.cu

# Ellenőrző GPU-státusz
nvidia-smi --query-gpu=name,driver_version,clocks.sm,power.draw,temperature.gpu,utilization.gpu
 ↪  \
  --format=csv

# Futtatás
./vec_triad
```

A `nvidia-smi` kimenetét a mérési naplóhoz kell csatolni. A mérés előtt legalább néhány percig hagyni kell a GPU-t stabilizálódni, és a mérés alatt nem szabad más GPU-terhelést futtatni.

13.1.15 7. A Roofline-modell korlátai

A Roofline-modellt nem szabad mechanikusan alkalmazni. Néhány fontos korlát:

- A memória- és számítási csúcsot a CPU oldali inicializálás, a PCIe átvitel vagy a `cudaMallocManaged` lapozási overhead nem tartalmazza, ha azok nem részei a mért időablaknak.
- A modell a DRAM-forgalomra épül, de az L1/L2 cache, a shared memory és a coalescing nélkülözhetetlen az elért teljesítmény magyarázatához.

- A valóságban a memória- és számítási fázisok részben átfedődhetnek, ezért a `max` helyett néha additív vagy részben additív modellt is használnak. A klasszikus Roofline felső korlátként használva marad konzervatív.
- A ridge point egyetlen hardware/software állapothoz kötött. Órajelcsökkenés, power limit vagy ECC hatására a görbe eltolódik.

A következő alfejezetben (Memóriateljesítmény és sávszélesség) a memóriateljesítmény és a tényleges sávszélesség mérésére építünk: az itt bevezetett I és $P_{\text{peak}}/B_{\text{peak}}$ fogalmak nélkülözhetetlenek ahhoz, hogy megértsük, milyen memóriametrikákat kell profilerrel rögzíteni, és hogyan lehet a sávszélességet a Roofline-görbéhez képest javítani.

13.1.16 Önellenőrző kérdés

Egy FP32 kernel 1 Mi elemet dolgoz fel. Elemenként 10 FLOP-ot végez, és elemenként 3 olvasást, valamint 1 írást igényel. A GPU FP32 elméleti csúcsa 20 TFLOPS, DRAM-sávszélessége 1 TB/s.

1. Mennyi a kernel arithmetic intensity értéke?

2. Mekkora a Roofline-modell által megengedett felső teljesítmény?

3. Melyik erőforrás a szűk keresztmetszet?

Megoldás:
1. Elemenkénti bájtforgalom $3 \cdot 4 + 4 = 16$ bájt. $I = 10/16 = 0{,}625$ FLOP/bájt.
2. $I \cdot B_{\text{peak}} = 0{,}625 \cdot 1000$ GB/s $= 625$ GFLOPS.
3. Mivel 625 GFLOPS $<$ 20 TFLOPS, a kernel memória-korlátos.

13.2 Memóriateljesítmény és sávszélesség

A GPU memóriateljesítménye a legtöbb általános célú számítási feladatban az elsődleges korlát. A Roofline-modellben a ferde, memória-korlátos szakasz meredekségét a ténylegesen elérhető sávszélesség határozza meg, ezért ha nem mérjük és nem értjük ezt a mennyiséget, a modell alapján hozott optimalizálási döntések félrevezethetnek. Ebben az alfejezetben először megkülönböztetjük az elméleti és a tényleges sávszélességet, majd bemutatjuk a HBM és az L2 szerepét, végül egy ellenőrizhető mérési módszertant adunk, amely a gyakorlatban is használható.

A sávszélesség fogalma egyszerű: egy adott időtartam alatt átvitt adatmennyiség. A GPU-k esetében a kérdés azonban több rétegű, mert az adat útja a globális memóriától a regiszterekig több szinten keresztül vezet, és minden szintnek más a késleltetése és az áteresztőképessége. A teljesítmény mérésekor ezért pontosan meg kell határozni, hogy melyik réteget vizsgáljuk, és milyen hozzáférési mintával dolgozunk.

13.2.1 Elméleti sávszélesség

Az elméleti sávszélesség a hardver adatlapjából számítható ki, feltételezve, hogy a memóriavezérlő folyamatosan, maximális órajelen, maximális buszszélességgel mozgatja az adatokat. A képlet a következő:

$$BW_{\text{elméleti}} = f_{\text{mem}} \cdot B \cdot r/8,$$

ahol f_{mem} a memória effektív órajele (Hz), B a memóriabusz szélessége (bitekben), r az adatráta (például DDR esetén 2, QDR esetén 4, HBM esetén gyakran 2), és az osztás 8-cal a bit–bájt átváltást végzi. Az eredményt bájt/s-ban kapjuk, amit jellemzően GB/s-ban adunk meg, tehát osztani kell 10^9-el.

Egy konkrét példa: az NVIDIA A100 40 GB HBM2e változata esetén a memóriabusz szélessége 5120 bit, az effektív memóriaórajel 3,2 GHz, az adatráta 2 (DDR), így az elméleti sávszélesség:

$$BW_{\text{elméleti}} = 3{,}2 \cdot 10^9 \cdot 5120 \cdot 2/8 = 4{,}096 \cdot 10^{12} \text{ B/s} \approx 4096 \text{ GB/s}.$$

Fontos hangsúlyozni, hogy ez a szám a gyártó által megadott névleges érték, amelyet ideális körülmények között, szintetikus tesztekkel mérnek. A valós alkalmazások ezt gyakorlatilag soha nem érik el. Az elméleti érték azonban hasznos referencia, mert megadja a felső korlátot, és segít értelmezni a mért eredményeket.

A memóriatechnológiák közül a modern adatközponti és HPC GPU-k jellemzően HBM (High Bandwidth Memory) típusú memóriát használnak, míg a fogyasztói és munkaállomás-kártyák GDDR-t. A HBM nagy előnye, hogy a memóriachipek a GPU-val közös interposerre vannak szerelve, így sokkal szélesebb busz érhető el (gyakran 1024–6144 bit), mint a hagyományos GDDR esetében (jellemzően 128–384 bit). Ez a széles busz a magas órajellel kombinálva több TB/s-os sávszélességet tesz lehetővé. Hátránya a magasabb gyártási költség, a korlátozott kapacitás és a nem bővíthető memória.

13.2.2 Tényleges sávszélesség

A tényleges sávszélesség az, amit egy adott program egy adott hardveren valóban elér. Kiszámítása:

$$BW_{\text{tényleges}} = \frac{\text{átvitt adatmennyiség}}{\text{mért idő}}.$$

Az átvitt adatmennyiség meghatározása nem mindig triviális: ha egy kernel egy N elemű float tömböt olvas be, akkor az átvitt adat $N \cdot 4$ bájt, feltéve, hogy minden elemet pontosan egyszer olvasunk, és nincs cache-találat. A mért időt viszont úgy kell mérni, hogy kizárjuk a kernelindítás overheadjét, a CPU–GPU szinkronizációt és az első hideg cache-hatást. Ezért a valós méréseknél melegítő futtatást (warm-up) és több ismétlést használunk.

A tényleges sávszélesség szinte mindig alacsonyabb az elméletinél, több okból is:

- **ECC**: az ECC (Error Correction Code) ellenőrzés a HBM esetében is jelen lehet, és extra bitek átvitelét vagy számítását igényli, ami csökkenti a felhasználói adatokra fordítható sávszélességet.
- **Memóriavezérlő overhead**: a vezérlő parancsokat, címzést és frissítési ciklusokat (refresh) hajt végre, amelyek nem adatátvitelt jelentenek.
- **Hőmérséklet és órajel**: a GPU-k dinamikusan szabályozzák az órajelet, és tartós terhelés alatt a memóriaórajel is csökkenhet a hőmérséklet miatt.
- **Hozzáférései minták**: a nem koaleszkált, rosszul igazított vagy szórt hozzáférések a memóriatranzakciók többszörösét eredményezik, tehát a ténylegesen mozgatott adat több mint a program által kért adat.
- **Cache-hatások**: a mérés során az L2 cache befolyásolhatja az eredményt, ha az adat nem elég nagy, vagy újrafelhasználás van.

A memóriahatékonyságot a tényleges és az elméleti sávszélesség hányadosaként definiálhatjuk:

$$\text{Hatékonyság} = \frac{BW_{\text{tényleges}}}{BW_{\text{elméleti}}} \cdot 100\%.$$

Egy jól optimalizált memória-korlátos kernel esetén ez a hatékonyság 80–90% is lehet, míg egy rosszul megírt kernelnél akár 5–10%-ra is leeshet. A cél mindig az, hogy a lehető legközelebb kerüljünk az elméleti maximumhoz.

13.2.3 HBM és L2

A sávszélesség szempontjából a GPU memóriahierarchiájának két kulcsszereplője van: a globális memória (HBM/GDDR) és az L2 cache. A globális memória nagy kapacitású (több tíz GB), nagy sávszélességű (több TB/s), de nagy késleltetésű (több száz ns). Az L2 cache kisebb (néhány MB-tól több tíz MB-ig), de sokkal gyorsabb hozzáférést biztosít, és csökkenti a globális memória terhelését az adat újrafelhasználás révén.

A modern NVIDIA GPU-kon az L2 cache mérete jelentősen nőtt: az A100-ban 40 MB, a H100-ban 50 MB az L2. Az L2 sávszélessége általában több TB/s, de még mindig alatta van a globális memória sávszélességének, viszont a késleltetése lényegesen kisebb. Amikor egy kernel adatot olvas a globális memóriából, az először az L2-be kerül; ha ugyanazt az adatot később újra olvassuk, és még az L2-ben van, akkor elkerüljük a globális memória hozzáférést. Ez a cache-találat növeli a tényleges sávszélességet, mert az L2-ből gyorsabban jön az adat, és nem terheli a memóriabuszt.

A memóriahatékonyság mérésénél ezért figyelni kell arra, hogy a munkakészlet mérete nagyobb legyen, mint az L2, különben a cache elrejti a globális memória valódi teljesítményét. Például ha 40 MB L2-vel rendelkező GPU-n 32 MB-os tömböt másolunk, az egész belefér az L2-be, és a mért sávszélesség az L2 sávszélességét fogja mutatni, nem a HBM-ét. A gyakorlatban ezért a sávszélesség-méréseknél legalább néhány száz MB vagy GB méretű puffereket használunk.

A HBM és az L2 közötti együttműködés jól modellezhető a streaming hozzáférési mintával: egy nagy tömb lineáris bejárása során az adatok egyszer haladnak át a memórián, az L2 nem tudja őket újrahasznosítani, ezért a mért sávszélesség közel lesz a tényleges HBM-sávszélességhez. Ezzel szemben ha egy kisebb adathalmazt sokszor újraolvasunk, az L2 elnyeli a forgalom nagy részét, és a globális memória terhelése csökken.

13.2.4 Memóriahatékonyság

A memóriahatékonyság nem csak a sávszélesség kihasználtságát jelenti, hanem azt is, hogy a kernel mennyire hatékonyan használja a memóriatranzakciókat. A legfontosabb tényezők:

- **Koaleszkált hozzáférés**: a warp (32 szál) globális memória hozzáférései akkor koaleszkáltak, ha a szálak által kért címek egyetlen, folytonos 128 bájtos cache-vonalba esnek. Ekkor egyetlen memóriatranzakció elég, és a busz maximálisan kihasznált. Ha a szálak véletlenszerű címeket olvasnak, minden szál külön tranzakciót generálhat, ami akár 32-szer több adat átvitelét okozza.
- **Igazítás**: a memóriacímek igazítása (alignment) befolyásolja, hogy a koaleszkált hozzáférés teljesül-e. A 4 bájtos float típusú adatoknál a 4 bájtos igazítás elegendő, de a vectorizált betöltéseknél (float4, 16 bájt) 16 bájtos igazítás szükséges.

- **Vectorizált betöltés**: ha egy szál egyszerre 4 floatot olvas be (float4), akkor a memóriatranzakció mérete nagyobb, és jobb a sávszélesség-kihasználás, mivel kevesebb tranzakció szükséges ugyanannyi adathoz.
- **Adat újrafelhasználás**: ha ugyanazt az adatot többször használjuk, érdemes shared memory-ba vagy regiszterekbe tölteni, hogy elkerüljük a globális memória újbóli olvasását. Ez növeli a számítási intenzitást, és csökkenti a memóriaterhelést.
- **Írási minták**: az írási műveleteknél is fontos a koaleszkálás és az igazítás, különben az írási tranzakciók is megsokszorozódnak.

Ezeket a témákat a későbbi fejezetek (Globális memória coalescing, Shared memory bankok és bank conflict) részletesen tárgyalják, itt csak a sávszélesség mérésének szempontjából fontos gyakorlati vonatkozásokat emeljük ki.

13.2.5 Labor: sávszélesség-mérés

A következő labor célja, hogy a hallgató megmérje a GPU globális memória sávszélességét egy egyszerű, koaleszkált, vectorizált másoló kernel segítségével, és összehasonlítsa az elméleti értékkel. A mérési módszertan a könyv általános labor-szerződését követi: melegítő futtatás, 5–10 ismétlés, időmérés CUDA eseményekkel, és az eredmények statisztikai jellemzése.

13.2.6 Mérési környezet

A méréshez rögzíteni kell a hardver és szoftver környezetet. Példa egy lehetséges konfigurációra (ez csak minta, a tényleges értékeket a saját rendszer alapján kell kitölteni):

- GPU: NVIDIA A100 40 GB PCIe
- Driver: 535.104.05 (ellenőrizni kell a tényleges verziót)
- CUDA Toolkit: 12.2 (ellenőrizni kell a tényleges verziót)
- Operációs rendszer: Ubuntu 22.04 LTS
- Fordító: nvcc, C++17 szabvány
- Power limit: alapértelmezett
- ECC: engedélyezve
- MIG: letiltva

A mérés előtt érdemes ellenőrizni, hogy nincs más processz a GPU-n, és a hőmérséklet stabil. Az `nvidia-smi` parancs segítségével megtekinthető a kihasználtság és a hőmérséklet.

13.2.7 A kernel

A sávszélesség mérésére egy olyan kernel alkalmas, amely egy nagy tömböt beolvas, és egy másik tömbbe kiír. A legegyszerűbb a `cudaMemcpy` device-to-device hívás, de az a DMA-motort használja, nem a számítási egységeket, így nem reprezentatív az SM-ek memóriateljesítményére. Helyette egy kernel indítása javasolt, amely a globális memóriából olvas, és oda is ír.

Az alábbi CUDA C++ kód egy vectorizált, koaleszkált másoló kernelt valósít meg `float4` típusokkal. A kernel minden szál egy `float4` elemet másol át az `src` tömbből a `dst` tömbbe. A `float4` 16 bájtos, ezért a vectorizált betöltés és tárolás kihasználja a memóriabusz szélességét.

```
// Nyelv: CUDA C++ (C++17)
// Fordító: nvcc -O3 -arch=sm_80 -o bw_test bw_test.cu

#include <cuda_runtime.h>
#include <cstdio>
#include <cstdint>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err = call;                                             \
        if (err != cudaSuccess) {                                           \
            std::fprintf(stderr, "CUDA error in %s:%d: %s\n", __FILE__,     \
                         __LINE__, cudaGetErrorString(err));                \
            std::exit(EXIT_FAILURE);                                        \
        }                                                                   \
    } while (0)

__global__ void copy_kernel(const float4* __restrict__ src,
                            float4* __restrict__ dst,
                            uint64_t n) {
    uint64_t idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        dst[idx] = src[idx];
    }
}

int main() {
    // A puffer mérete: legalább 1 GB, hogy meghaladja az L2-t.
    constexpr uint64_t num_float4 = (1ULL << 28); // 268M float4 = 4 GB
    constexpr size_t bytes = num_float4 * sizeof(float4);

    float4 *d_src, *d_dst;
    CUDA_CHECK(cudaMalloc(&d_src, bytes));
    CUDA_CHECK(cudaMalloc(&d_dst, bytes));

    // Inicializálás (elhagyható, csak a memória lefoglalása a lényeg)
    CUDA_CHECK(cudaMemset(d_src, 0, bytes));

    // Launch konfiguráció
    constexpr int threads_per_block = 256;
    uint64_t blocks = (num_float4 + threads_per_block - 1) / threads_per_block;

    // CUDA események az időméréshez
    cudaEvent_t start, stop;
    CUDA_CHECK(cudaEventCreate(&start));
    CUDA_CHECK(cudaEventCreate(&stop));

    // Warm-up: 5 iteráció
    for (int i = 0; i < 5; ++i) {
        copy_kernel<<<blocks, threads_per_block>>>(d_src, d_dst, num_float4);
    }
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaGetLastError());

    // Mérés: 10 iteráció
    constexpr int iterations = 10;
    float total_time_ms = 0.0f;
    for (int i = 0; i < iterations; ++i) {
        CUDA_CHECK(cudaEventRecord(start));
        copy_kernel<<<blocks, threads_per_block>>>(d_src, d_dst, num_float4);
        CUDA_CHECK(cudaEventRecord(stop));
        CUDA_CHECK(cudaEventSynchronize(stop));
        float elapsed_ms = 0.0f;
```

```
        CUDA_CHECK(cudaEventElapsedTime(&elapsed_ms, start, stop));
        total_time_ms += elapsed_ms;
    }

    float avg_time_ms = total_time_ms / iterations;
    double avg_time_s = avg_time_ms / 1000.0;

    // A kernel átvitt adatmennyisége: olvasás + írás = 2 * bytes
    double transferred_bytes = 2.0 * static_cast<double>(bytes);
    double bandwidth_Bps = transferred_bytes / avg_time_s;
    double bandwidth_GBps = bandwidth_Bps / 1e9;

    std::printf("Átlagos idő: %.3f ms\n", avg_time_ms);
    std::printf("Átvitt adat: %.2f GB\n", transferred_bytes / 1e9);
    std::printf("Mért sávszélesség: %.2f GB/s\n", bandwidth_GBps);

    // Elméleti érték (példa A100 esetén 4096 GB/s, de ezt a saját GPU-hoz kell igazítani)
    constexpr double theoretical_GBps = 4096.0;
    std::printf("Elméleti sávszélesség: %.2f GB/s\n", theoretical_GBps);
    std::printf("Hatékonyság: %.1f%%\n", 100.0 * bandwidth_GBps / theoretical_GBps);

    CUDA_CHECK(cudaFree(d_src));
    CUDA_CHECK(cudaFree(d_dst));
    CUDA_CHECK(cudaEventDestroy(start));
    CUDA_CHECK(cudaEventDestroy(stop));
    return 0;
}
```

A kernel indítási alakja a kódban `<<<blocks, threads_per_block>>>`, ami megfelel a könyv jelölésének: `<<<grid, block, shared_bytes, stream>>>`, itt a shared memória és a stream alapértelmezett, ezért elhagyjuk.

A kódban a `num_float4` értéke 268 millió, ami 4 GB puffert jelent float4 esetén (268M * 16 bájt = 4,29 GB). Ez lényegesen nagyobb, mint a legtöbb GPU L2 cache-e, így a mérés a globális memória sávszélességét fogja mutatni. Az átvitt adatmennyiség a kernelben olvasás és írás miatt a puffer méretének kétszerese, tehát 8,59 GB.

13.2.8 Build és futtatás

A kód fordításához a következő parancs használható:

```
nvcc -O3 -arch=sm_80 -o bw_test bw_test.cu
```

Az `-arch=sm_80` az NVIDIA Ampere generációhoz tartozó compute capability 8.0-t jelenti, ami az A100 esetén helyes. Más GPU-k esetén a megfelelő értéket kell használni (pl. `sm_86` RTX 3000, `sm_89` RTX 4000). A futtatás:

```
./bw_test
```

13.2.9 Mérési napló

A mérési eredményeket minden körülmény rögzítésével kell dokumentálni. Az alábbi táblázat egy példa a mérési naplóra, de a számok csak illusztrációk, nem valós mérésekből származnak; a hallgatónak a saját környezetében kell elvégeznie a mérést, és a tényleges értékeket beírni.

Paraméter	Érték
GPU modell	NVIDIA A100 40 GB PCIe (példa)
Driver verzió	535.104.05 (ellenőrizendő)
CUDA verzió	12.2 (ellenőrizendő)
OS	Ubuntu 22.04 LTS
Kernel	copy_kernel (float4)
Pufferméret	4,29 GB (268M float4)
Átvitt adat iterációnként	8,59 GB
Blokk méret	256 thread
Grid méret	1 048 576 block
Warm-up iterációk	5
Mérési iterációk	10
Átlagos idő	2,35 ms (példa)
Szórás	0,12 ms (példa)
Minimum / maximum	2,21 / 2,52 ms (példa)
Mért sávszélesség	**3657 GB/s (példa)**
Elméleti sávszélesség	**4096 GB/s**
Hatékonyság	**89,3% (példa)**

A mérési naplóban a “mért” és a “várt” értékeket külön kell választani. A várt érték az elméleti sávszélesség, a mért pedig a kiszámított tényleges sávszélesség. A hatékonyság a kettő hányadosa. A példában 89,3% szerepel, ami egy jól optimalizált kernel esetén reális, de a tényleges érték a környezettől függ.

13.2.10 Eredmények értelmezése

A mért sávszélesség általában 80–90%-a az elméletinek egy ilyen egyszerű, koaleszkált, vectorizált kernel esetén. Ha a mért érték ennél alacsonyabb, az alábbi okokra érdemes gyanakodni:

- **Túl kicsi puffer**: ha a puffer mérete az L2 cache méretéhez közeli vagy kisebb, a cache elrejti a globális memória teljesítményét. Növeljük a puffert legalább néhány GB-ra.
- **ECC engedélyezve**: az ECC némi sávszélesség-veszteséget okoz, de ez általában 5–10% alatti.
- **Hőmérséklet / órajel**: tartós terhelés alatt a memóriaórajel csökkenhet. Figyeljük a `nvidia-smi`-t, és hagyjuk a GPU-t lehűlni a mérések között.
- **Háttérterhelés**: más processzek jelenléte csökkenti a mérhető sávszélességet. Használjunk izolált környezetet.
- **Rossz időmérés**: ha a `cudaEventElapsedTime` előtt nincs szinkronizáció, az időmérés hibás lehet. A kódban a `cudaEventSynchronize(stop)` gondoskodik a helyes mérésről.

13.2.11 Tipikus hibák és helyes megoldások

A sávszélesség-mérés során gyakori hibák:

1. **Nem melegítő futtatás**: az első kernelindítás magában foglalja a memória allokációt, a kontextus inicializálást és az esetleges JIT fordítást. Ha ezt is beleszámítjuk az időbe, alacsonyabb sávszélességet kapunk. Megoldás: 3–5 melegítő iteráció a mérés előtt.

2. **Csak egyetlen mérés**: a futásidő szórása a GPU órajel-ingadozása miatt jelentős lehet. Több iteráció átlagát kell venni, és a szórást is közölni.

3. **Rossz adatmennyiség-számítás**: ha a kernel olvas és ír is, az átvitt adat a puffer méretének kétszerese. Ha ezt elfelejtjük, a sávszélességet felére becsüljük. Helyes: $2 \cdot N \cdot$ elem mérete.
4. **A cache hatásának figyelmen kívül hagyása**: ha a puffer kicsi, az L2 gyorsítótárazza, és a mért sávszélesség az L2-é lesz, nem a globális memóriáé. Megoldás: a puffer legalább 2–4-szer nagyobb legyen az L2-nél.
5. **Nem koaleszkált hozzáférés**: a mérési kernelben minden szálnak a `threadIdx.x`-hez lineárisan indexelt elemet kell elérnie. Ha az indexelés rossz (pl. `idx = threadIdx.x * stride` nagy stride-dal), a koaleszkálás megtörik, és a mért sávszélesség zuhan. A fenti példában a `float4` használata és a lineáris indexelés koaleszkált hozzáférést biztosít.
6. **A grid és block méret rossz megválasztása**: a túl kevés blokk nem tudja teljesen kihasználni a memóriavezérlőt, míg a túl sok blokk overheadet okoz. A fenti példában a blokkméret 256, ami általában jó, de a grid méretét a puffer mérete határozza meg.
7. **Az elméleti érték rossz kiszámítása**: mindig a saját GPU specifikációjából számoljunk, ne más GPU értékét használjuk. Ellenőrizzük a gyártó dokumentációját, és vegyük figyelembe a memóriabusz szélességét és az adatrátát.
8. **ECC és MIG figyelmen kívül hagyása**: ha az ECC engedélyezett, az elméleti sávszélességből néhány százalékot levonhatunk. A MIG (Multi-Instance GPU) partíciók esetén a sávszélesség megoszlik, ezért a mérést teljes GPU-n vagy az adott partíció specifikációjának megfelelően kell elvégezni.

13.2.12 Ellenőrző kérdések

1. Mi a különbség az elméleti és a tényleges sávszélesség között? Soroljon fel legalább három tényezőt, amelyek csökkentik a tényleges sávszélességet az elméletihez képest.
2. Miért kell a sávszélesség-mérésnél az adatpuffert nagyobbra választani, mint az L2 cache mérete?
3. Egy kernel egy 1 GB-os tömböt olvas be, és egy másik 1 GB-os tömbbe ír. A futásidő 0,5 ms. Mekkora a tényleges sávszélesség GB/s-ban? (A válasz: 2 GB / 0,0005 s = 4000 GB/s.)
4. Hogyan befolyásolja a koaleszkálás és a vectorizálás a mért sávszélességet? Magyarázza el a `float4` használatának előnyét.
5. Mi a HBM előnye a GDDR-rel szemben, és hogyan kapcsolódik ez a sávszélességhez?

13.2.13 Összefoglalás

A memóriasávszélesség a GPU teljesítményének egyik legfontosabb mérőszáma. Az elméleti érték a hardver adatlapjából számítható ki, míg a tényleges értéket csak méréssel lehet meghatározni. A kettő közötti különbség a memóriavezérlő overheadjéből, az ECC-ből, a hőmérséklet-ingadozásból és a hozzáférési mintákból adódik. A HBM technológia nagy sávszélességet biztosít, de a hatékony kihasználásához koaleszkált, vectorizált hozzáférésekre van szükség. Az L2 cache szerepe az adat újrafelhasználás révén csökkenti a globális memória terhelését, de a mérésnél figyelembe kell venni a cache méretét. A laborban bemutatott egyszerű másoló kernel lehetővé teszi a tényleges sávszélesség megbízható mérését, és a kapott hatékonysági mutató alapot ad a további optimalizáláshoz. A következő alfejezet ezekre a fogalmakra építve a számítási korlátot és a compute intensity-t tárgyalja.

13.3 Számítási korlát és compute intensity

Ahhoz, hogy egy kernelt optimalizálhassunk, először meg kell állapítanunk, mi is korlátozza valójában a teljesítményét. A Memóriateljesítmény és sávszélesség fejezetben bevezettük a Roofline-modellt és megismertük a memóriavezérlő, valamint az elérhető sávszélesség szerepét. Most a modell másik oldalát vizsgáljuk meg: a számítási csúcsértéket, a lebegőpontos műveletek számolását, az aritmetikai intenzitás kiszámítását, valamint a kernelbesorolás módszerét. A cél nem csupán a fogalmak definiálása: az olvasó egy mérhető CUDA-laborkísérleten keresztül tanulja meg elkülöníteni a memória-korlátos és a számítás-korlátos tartományt, és megtanulja a tényleges mérési adatokat a várt, modellből számolt értékektől elkülönítve kezelni.

13.3.1 A két véglet: memória-korlátos és számítás-korlátos kernel

Egy GPU-kernel futásideje sosem egynemű. Legalább két komponens összegeként érdemes elképzelni: a memóriaforgalom lebonyolításának ideje (*memóriaidő*) és az aritmetikai egységek munkájának ideje (*számítási idő*). A kettő részben átfedhet egymást, ha az ütemező elegendő aktív warppal tudja elrejteni a memória késleltetését, de a Memóriateljesítmény és sávszélesség fejezetben bevezetett egyszerű közelítés szerint a futásidőt jó közelítéssel a két idő maximuma határozza meg:

$$T_{\text{exec}} \approx \max(T_{\text{mem}}, T_{\text{compute}}).$$

Ebből következik a két alapvető osztályozás:

- **Memória-korlátos** (*memory-bound*) kernel esetén $T_{\text{mem}} \gg T_{\text{compute}}$. A futásidőt az adatmozgatás határozza meg, az aritmetikai egységek javarészt tétlenül várnak új adatra. A kernel teljesítményének növeléséhez elsősorban a memóriaforgalmat kell csökkenteni vagy a sávszélességet jobban ki kell használni.
- **Számítás-korlátos** (*compute-bound*) kernel esetén $T_{\text{compute}} \gg T_{\text{mem}}$. A kernel szinte minden ciklusban hasznos aritmetikai munkát végez, a memóriavezérlő relatív tétlen. A teljesítmény növeléséhez a lebegőpontos egységek utasításainak hatékonyságát kell javítani, például fuzionált műveletek alkalmazásával, kevesebb hulladék utasítással vagy alacsonyabb pontossággal.

Az osztályozás nem statikus címke: ugyanaz a matematikai művelet más bemeneti méretre, más pontosságra vagy más cache-viselkedésre megváltoztathatja a besorolást. A Roofline-modell éppen azért hasznos, mert a memória- és a számítási korlátot egyetlen koordináta-rendszerben, a független változóval – az aritmetikai intenzitással – együtt ábrázolja.

13.3.2 FLOP-számolás: mit számítunk és mit nem?

A számítási korlát elemzéséhez először a kernel által végzett lebegőpontos műveletek számát kell meghatároznunk. A szakirodalomban a **FLOP** (Floating Point Operation) rövidítést használják; a 10^3 gyorsulások megkülönböztetésére a műveletek számát FLOP-ban, a másodpercenkénti teljesítményt FLOPS-ban (FLOP/s) jelöljük.

A FLOP-számlálás konvenciója a GPU-ökoszisztémában nem triviális, és pontosan ez az egyik leggyakoribb hibalehetőség a Roofline-elemzések során. A legfontosabb szabályok:

1. **Egy összeadás = 1 FLOP, egy szorzás = 1 FLOP.** A kivonás és az ellentett előjel is összeadásként számít, a negatív előjelet nem tekintjük plusz műveletnek.

2. **Egy fuzionált szorzás-összeadás (FMA) = 2 FLOP**, noha gépi utasításszinten egyetlen instrukcióról van szó. Az NVIDIA specifikációs lapokon szereplő csúcs FLOPS értékek ezen a konvención alapulnak: egy GPU-architektúra, amely órajelenként egy FMA utasítást végez el CUDA-magonként, 2 FLOP/ciklust könyvel el.
3. **A osztás, gyökvonás, reciprok és a transzcendens függvények** a klasszikus Roofline-könyvelésben gyakran egyetlen FLOP-ként szerepelnek, de a valós futási idejük lényegesen hosszabb, mint egyetlen FMA. A modern GPU-kon ezeket a műveleteket elkülönített *special function unit* (SFU) egységek végzik, amelyek alacsonyabb órajelezéssel vagy alacsonyabb párhuzamossággal dolgoznak. Precíziós mérésnél ezt a T_{compute} meghatározásánál figyelembe kell venni, de a Roofline-tanításoknál első közelítésben a FLOP-könyvelés elegendő.

A FLOP-számolástól különbözik az **utasításszám**. Egy kernel akár több ezer utasítást is végrehajthat anélkül, hogy egyetlen lebegőpontos adatot kezelne: címzési aritmetika, predikátumok kiértékelése, elágaztatás, indexszámítás. A Roofline-modellben kizárólag a matematikai műveletek számítanak FLOP-nak, a vezérlési utasítások nem. Ez fontos, mert a compute-bound elemzés során az utasítások túlnyomó része lebegőpontos művelet, míg a memory-bound kernelben a lebegőpontos utasítások aránya alacsony a betöltés/tárolás és a vezérlési utasításokhoz képest.

A FLOP-számolás a pontosságtól független elvileg: egy FP64-es összeadás éppúgy 1 FLOP, mint egy FP32-es vagy FP16-os. A gyakorlatban a pontosság határozza meg a megvalósítható csúcs FLOPS-t. Az FP64-es egységek a fogyasztói GPU-kon sokkal lassabbak, mint az FP32-es egységek (például 1:64 arány az Ampere generáció fogyasztói sorozatában), míg a professzionális és HPC-kártyákon az arány 1:2 vagy 1:1. Az FP16, TF32 és BF16 egységek tovább növelik a csúcs FLOPS-t, de a numerikus pontosság csökken, amit a Lebegőpontos pontosság fejezetben tárgyalunk majd részletesen.

13.3.3 Byte-számolás: melyik memóriaszint forgalmát számoljuk?

A Roofline-modellben a **byte** az a hasznos adatmennyiség, amelyet a kernel a fő adatelérési útján mozgat. A legjobb első közelítés a **DRAM-forgalom**: az a byte-mennyiség, amely a kernel futása során ténylegesen átmegy a GPU memóriavezérlőjén. Ez a gyakorlatban közel áll az igényelt adatmennyiséghez, de nem azonos vele, mert a cache-ek (L2, illetve L1) csökkentik a DRAM-ig érő forgalmat.

A tanító jellegű elemzésnél a következő kategóriákat különítjük el:

- **Naiv byte-számolás** az alkalmazás szintjén igényelt adatmennyiség: a kernel beolvassa az összes bemeneti elemet, és kiírja az összes kimeneti elemet. Ha nincs adatismétlés, ez megegyezik a DRAM-forgalommal.
- **Tényleges DRAM-forgalom** a memóriavezérlőnél mérhető adatmennyiség, ami a cache-találatok miatt kevesebb lehet. Például ha a bemeneti vektor elfér az L2 cache-ben, a kernel az első iteráció után már nem olvas a DRAM-ból. A Roofline-elemzés során ezt a hatást úgy kezeljük, hogy a munkakészlet méretét a cache-méretekhez viszonyítjuk, és a releváns memóriaszint forgalmával számolunk.
- **Következtetett effektív byte** az a normalizált mennyiség, amelyet a teljesítményből és a FLOP-számból visszaszámolunk. Ez hibakeresésre hasznos: ha az effektív byte jóval alacsonyabb vagy magasabb a vártnál, cache- vagy tranzakció-hibát gyaníthatunk.

A klasszikus Roofline-tanításban a legáltalánosabb egyszerűsítés az, hogy a memóriaszint-hierarchiát kizárólag a DRAM-ra koncentráljuk, mivel a legtöbb nagy méretű, tudományos számítási kernel munkakészlete jóval nagyobb, mint az L2 cache. Ahol nem ez a helyzet, ott a modellt az L2 sávszélességével és a munkakészlet cache-ben való elhelyezésével kell módosítani, ahogy a L1 és L2 cache fejezetben tesszük.

A byte-számolás gyakorlati szabálya a laborban: **mindig rögzítsük a bemeneti és kimeneti vektor pontosságát, a beolvasott és kiírt elemek számát, valamint az ismételt hozzáférések számát**. A horizontális irányok (például egy mátrix több kihasználása) a DRAM-forgalmat nem feltétlenül növelik, ha a redundáns elemek a cache-ben maradnak. A tényleges forgalom meghatározásához profilozási adatok (Nsight Compute L2/DRAM metrikák) szükségesek.

13.3.4 Aritmetikai intenzitás: a híd a két korlát között

A **számítási intenzitást** (gyakran *aritmetikai intenzitásnak* is nevezik) a kernel által végzett lebegőpontos műveletek számának és a fő memóriaszinten mozgatott byte-ok számának hányadosaként definiáljuk:

$$I = \frac{F}{B},$$

ahol F a FLOP-ok száma, B pedig a DRAM (vagy releváns memóriaszint) forgalma byte-ban. Az I mértékegysége FLOP/byte. Ez a dimenziómentesnek tűnő, de valójában erősen hardverfüggő kritérium köti össze a két hardveres csúcsértéket: a memória-sávszélességet (β, byte/s) és a lebegőpontos csúcsértéket (π, FLOP/s).

A Roofline-modell alapvető egyenlete a Memóriateljesítmény és sávszélesség fejezetből:

$$P = \min(\pi, \beta \cdot I),$$

ahol P az elérhető lebegőpontos teljesítmény FLOP/s-ban. Az egyenlet két tartományra bomlik:

- **Memória-korlátos tartomány** ($I < I_{\text{ridge}}$): a $\beta \cdot I$ szorzat kisebb, mint π, tehát $P \approx \beta \cdot I$. A teljesítmény arányos az aritmetikai intenzitással; minél több FLOP jut egy byte-ra, annál magasabb az elérhető FLOPS, de a tetőpontot még nem éri el.
- **Számítás-korlátos tartomány** ($I > I_{\text{ridge}}$): a $\beta \cdot I$ szorzat meghaladná a π-t, tehát $P \approx \pi$. A teljesítmény a lebegőpontos egységek csúcsához tart, függetlenül az intenzitástól.

A két tartomány határát a **Roofline-töréspont** (ridge point) adja, ahol a memória-sávszélességből és a lebegőpontos csúcsból számolt egyenes metszi a tetőpontot:

$$I_{\text{ridge}} = \frac{\pi}{\beta}.$$

Ez az érték minden hardver-pontosság kombinációra más és más. Egy modern, nagy sávszélességű HPC-GPU esetén az FP32 töréspont 10–30 FLOP/byte körül alakul, míg a fogyasztói kártyákra 20–50 FLOP/byte jellemző. Az FP64 estén a π csökken, így a töréspont alacsonyabb; az FP16 vagy Tensor Core üzemmódban a π nő, és a töréspont magasabb értékre tolódik.

13.3.5 A gyakorlati minta: SAXPY és Horner-polinom

A besorolási logika elsajátításához két egyszerű kernelt vizsgálunk meg: a SAXPY-t és egy Horner-alakban kiértékelt polinomot. Mindkét kernel ugyanazt a vektor-processing logikát követi – minden szál egyetlen elemet dolgoz fel –, de eltérő számú FLOP-ot végez egy byte beolvasására vetítve.

SAXPY ($y \leftarrow a \cdot x + y$, a "scalar alpha × x plus y" rövidítése):

- FLOP-elemszám: 2 FLOP (egy szorzás + egy összeadás).
- Byte-elemszám FP32 esetén: 12 byte (beolvasás x (4 byte), beolvasás y (4 byte), kiírás y (4 byte)).
- Aritmetikai intenzitás: $I_{\text{saxpy}} = 2/12 \approx 0{,}17$ FLOP/byte.

Horner-polinom: $y \leftarrow ((a \cdot x + b) \cdot x + c) \cdot x + d$, ahol a, b, c, d konstansok, az adatfolyam pedig x bemenet és y kimenet.

- FLOP-elemszám: 6 FLOP (3 szorzás + 3 összeadás).
- Byte-elemszám FP32 esetén: 8 byte (beolvasás x (4 byte), kiírás y (4 byte); a konstansok regiszterben maradnak).
- Aritmetikai intenzitás: $I_{\text{horner}} = 6/8 = 0{,}75$ FLOP/byte.

Mindkét intenzitás jóval a tipikus FP32 Roofline-töréspont alatt marad, tehát **mindkét kernel memória-korlátos**. A Horner-polinom azonban magasabb intenzitással rendelkezik, ezért a Roofline-modell szerint az elérhető teljesítménye is magasabb lesz:

$$P_{\text{saxpy}} \approx \beta \cdot 0{,}17 \quad \text{és} \quad P_{\text{horner}} \approx \beta \cdot 0{,}75.$$

Ha $\beta = 1000$ GB/s, akkor $P_{\text{saxpy}} \approx 167$ GFLOP/s, míg $P_{\text{horner}} \approx 750$ GFLOP/s. Ez a négyszeres különbség a FLOP/byte arányban mérhető, noha mindkettő messze a csúsztatott csúcs alatt marad.

A példa rávilágít a Roofline-modell tanulságára: **amíg a kernel az árral szemben úszik (memória-korlátos), a teljesítmény növekedéséhez az intenzitást kell növelni, nem pedig a csúcsteljesítményt hajszolni.** Egy további szorzás hozzáadása anélkül, hogy újraolvasnánk a bemenetet, azonos sávszélesség mellett nagyobb elérhető FLOPS-t eredményez.

Ezzel szemben a számítás-korlátos tartományba a dense GEMM (sűrű mátrixszorzás) tartozik. Az $n \times n$-es mátrixszorzás $F \approx 2n^3$ FLOP-ot igényel, a naiv DRAM-forgalom pedig $B \approx 12n^2$ byte (három mátrix, egyenként $4n^2$ byte, FP32). Az intenzitás:

$$I_{\text{gemm}} \approx \frac{2n^3}{12n^2} = \frac{n}{6}.$$

$n = 1024$ esetén $I \approx 170$ FLOP/byte, ami messze a tipikus töréspont felett van. A kernel számítás-korlátos, és a megfelelő implementáció elérheti a π 80–90%-át. Ez az a pont, ahol a memóriavezérlő már nem szűk keresztmetszet, és a tiling valódi értelmet nyer – ezt a Roofline-alapú optimalizálás fejezetben fejtjük ki részletesen.

13.3.6 Az optimalizálási irány: miből következik mi?

A kernelbesorolás nem öncélú taxonómia, hanem az optimalizálási stratégia kiindulópontja. A Roofline-modell egyértelműen megmutatja, melyik hardvererőforrás a szűk keresztmetszet, és ennek megfelelően jelöli ki a beavatkozás irányát:

Memória-korlátos kernel optimalizálása:

- **Növeljük az intenzitást:** végezzünk több FLOP-ot egy beolvasott elemre (pl. több polinom tag, több egymás utáni művelet).
- **Csökkentsük a DRAM-forgalmat:** használjunk shared memoryt adatismétlésre, cache-láthatósággal dolgozzunk, alkalmazzunk tilingot.
- **Javítsuk a burkolt adathozzáférést:** coalescing, bank-conflict-mentes shared memory-hozzáférés (Globális memória coalescing, Shared memory bankok és bank conflict).
- **Növeljük az effektív sávszélességet:** jobb cache-kihasználás, vektorizált betöltés (float4).

Számítás-korlátos kernel optimalizálása:

- **Csökkentsük a hulladék utasításokat:** minimalizáljuk a vezérlési utasításokat, használjunk predikációt az elágazások helyett.
- **Használjunk FMA-t:** a fuzionált műveletek két FLOP-ot végeznek egy utasítással, ami csökkenti az utasításnyomást.
- **Vegyük fontolóra az alacsonyabb pontosságot:** FP32 $\rightarrow$ TF32/FP16/BF16, ha a numerikus korlátok megengedik.
- **Használjuk a Tensor Core-okat:** a mátrixaritmetikában a Tensor Core FMA-sűrűsége messze meghaladja a szokásos CUDA-magokét (Tensor Core architektúra).

A rossz irányú optimalizálás tipikus tünete, hogy a kernel megmarad az alsó Roofline-szakaszon: hiába növeljük a π-re irányuló erőfeszítéseket, a teljesítmény $\beta \cdot I$-nél leragad. A Roofline-elemzés megelőzi ezt a zsákutcát, mielőtt belekerülnénk.

13.3.7 Labor: kernelbesorolás Roofline alapján

Ez a labor gyakorlati módszert ad a fenti elmélethez. A feladat két CUDA-kernel mérése, aritmetikai intenzitásuk kiszámítása, a Roofline-töréspont alapján történő besorolásuk, és a mért sávszélesség/FLOPS összevetése a várt modellel.

Környezet rögzítése. Minden mérés előtt rögzítse az alábbi adatokat:

- GPU modell és compute capability
- NVIDIA driver verzió
- CUDA toolkit verzió
- Operációs rendszer és kernelverzió

Ezek nélkül a mérési napló nem reprodukálható, és a „gyorsabb” állítás nem ellenőrizhető. Példa a laborjegyzőkönyv fejlécére:

```
GPU: NVIDIA RTX <modell> (CC <x.y>)
Driver: 550.xx.yy
CUDA: 12.x
OS: Linux 6.x arch
Power limit / ECC: <érték / ki/be>
```

Fordítható CUDA C++ forrás. Az alábbi program implementálja a SAXPY és a Horner-polinom kernelt, a kötelező hibakezelést, a warm-up protokollt és a mérési naplót.

```
// labor_gpu048.cu
// Kernelbesorolás Roofline alapján: SAXPY és Horner-polinom.
// Fordítás: nvcc -O3 -arch=sm_86 -o labor_gpu048 labor_gpu048.cu
// Futtatás: ./labor_gpu048

#include <cstdio>
#include <cuda_runtime.h>
#include <cmath>

#ifndef CHECK_CUDA
#define CHECK_CUDA(call)                                                   \
    do {                                                                    \
        cudaError_t _err = (call);                                          \
        if (_err != cudaSuccess) {                                          \
            fprintf(stderr, "CUDA hiba: %s | %s:%d\n",                      \
                    cudaGetErrorString(_err), __FILE__, __LINE__);          \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)
#endif

// 1. kernel: SAXPY, y[i] = a*x[i] + y[i]
__global__ void saxpy_kernel(const float* __restrict__ x,
                             float* __restrict__ y,
                             float a, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        y[idx] = a * x[idx] + y[idx];
    }
}

// 2. kernel: Horner-polinom, y[i] = ((a*x[i]+b)*x[i]+c)*x[i]+d
__global__ void horner_kernel(const float* __restrict__ x,
                              float* __restrict__ y,
                              float a, float b,
                              float c, float d,
                              int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        float t = a * x[idx] + b;
        t = t * x[idx] + c;
        t = t * x[idx] + d;
        y[idx] = t;
    }
}

// A mérési protokoll: warm-up + 10 mérés, átlag és szórás.
void run_and_measure(const char* name,
                     void (*kernel_launcher)(dim3, dim3, cudaStream_t),
                     dim3 grid, dim3 block,
                     size_t output_bytes,
                     double flop_count) {
    cudaEvent_t start, stop;
    CHECK_CUDA(cudaEventCreate(&start));
    CHECK_CUDA(cudaEventCreate(&stop));
    cudaStream_t stream;
    CHECK_CUDA(cudaStreamCreate(&stream));

    // Warm-up: 5 futtatás a pipeline bejáratására.
    for (int i = 0; i < 5; ++i) {
```

```
        kernel_launcher(grid, block, stream);
    }
    CHECK_CUDA(cudaDeviceSynchronize());

    // 10 mérés, átlag és szórás.
    const int rep = 10;
    double times[rep];
    for (int i = 0; i < rep; ++i) {
        CHECK_CUDA(cudaEventRecord(start, stream));
        kernel_launcher(grid, block, stream);
        CHECK_CUDA(cudaEventRecord(stop, stream));
        CHECK_CUDA(cudaEventSynchronize(stop));
        float ms = 0.0f;
        CHECK_CUDA(cudaEventElapsedTime(&ms, start, stop));
        times[i] = (double)ms * 1e-3; // másodperc
    }

    double avg = 0.0;
    for (int i = 0; i < rep; ++i) avg += times[i];
    avg /= rep;

    double var = 0.0;
    for (int i = 0; i < rep; ++i) var += (times[i] - avg) * (times[i] - avg);
    var /= (rep - 1);
    double std_dev = sqrt(var);

    double achieved_bandwidth_gbps = (double)output_bytes / avg / 1e9;
    double achieved_flops = flop_count / avg;

    printf("\n== %s ==\n", name);
    printf("Futásidő átlag: %.4f ms (szórás %.4f ms)\n", avg * 1e3, std_dev * 1e3);
    printf("Mért sávszélesség: %.2f GB/s\n", achieved_bandwidth_gbps);
    printf("Mért FLOPS: %.2f GFLOP/s\n", achieved_flops / 1e9);
    printf("Mért teljesítmény: %.2f ms\n", avg * 1e3);

    CHECK_CUDA(cudaEventDestroy(start));
    CHECK_CUDA(cudaEventDestroy(stop));
    CHECK_CUDA(cudaStreamDestroy(stream));
}

// Launch wrapper-ök a dimenzióknak megfelelően.
void launch_saxpy(dim3 grid, dim3 block, cudaStream_t stream,
                  const float* d_x, float* d_y, float a, int n) {
    saxpy_kernel<<<grid, block, 0, stream>>>(d_x, d_y, a, n);
}

void launch_horner(dim3 grid, dim3 block, cudaStream_t stream,
                   const float* d_x, float* d_y,
                   float a, float b, float c, float d, int n) {
    horner_kernel<<<grid, block, 0, stream>>>(d_x, d_y, a, b, c, d, n);
}

int main() {
    const int N = 1024 * 1024 * 64; // 67M elem, kb. 805 MB SAXPY forgalom
    const size_t bytes_in = (size_t)N * sizeof(float);
    const size_t bytes_out = (size_t)N * sizeof(float);
    const size_t saxpy_bytes = 2 * bytes_in + bytes_out; // x + y olvasás + y írás
    const size_t horner_bytes = bytes_in + bytes_out;     // x olvasás + y írás

    const double saxpy_flops = 2.0 * (double)N;
    const double horner_flops = 6.0 * (double)N;

    printf("GPU adatai:\n");
```

```
    int dev; cudaDeviceProp prop;
    CHECK_CUDA(cudaGetDevice(&dev));
    CHECK_CUDA(cudaGetDeviceProperties(&prop, dev));
    printf("  Név: %s\n", prop.name);
    printf("  Compute capability: %d.%d\n", prop.major, prop.minor);
    printf("  Sávszélesség (címke, GB/s): %.2f\n", (double)prop.memoryClockRate * 1e-3 *
↪   (double)prop.memoryBusWidth / 8.0);
    printf("  FP32 csúcs (elméleti, GFLOP/s): %.2f\n",
           (double)prop.clockRate * 1e-3 * 2.0 * (double)prop.multiProcessorCount * 128.0 /
            ↪  1e3);
    printf("\nMunkakészlet: N = %d elem, SAXPY forgalom = %.2f MB, Horner forgalom = %.2f MB\n",
           N, (double)saxpy_bytes / 1e6, (double)horner_bytes / 1e6);

    // Eszközoldali allokáció.
    float *d_x, *d_y;
    CHECK_CUDA(cudaMalloc(&d_x, bytes_in));
    CHECK_CUDA(cudaMalloc(&d_y, bytes_out));

    // Inicializálás (host oldalon az egyszerűség kedvéért).
    float *h_x = (float*)malloc(bytes_in);
    float *h_y = (float*)malloc(bytes_out);
    for (int i = 0; i < N; ++i) {
        h_x[i] = (float)(i % 17) * 0.1f;
        h_y[i] = 1.0f;
    }
    CHECK_CUDA(cudaMemcpy(d_x, h_x, bytes_in, cudaMemcpyHostToDevice));
    CHECK_CUDA(cudaMemcpy(d_y, h_y, bytes_out, cudaMemcpyHostToDevice));

    const int block_size = 256;
    const int grid_size = (N + block_size - 1) / block_size;
    dim3 grid(grid_size);
    dim3 block(block_size);

    // A launch wrapper-ök méréséhez lambda-kat definiálunk.
    auto saxpy_launch = [&](dim3 g, dim3 b, cudaStream_t s) {
        launch_saxpy(g, b, s, d_x, d_y, 2.0f, N);
    };
    auto horner_launch = [&](dim3 g, dim3 b, cudaStream_t s) {
        launch_horner(g, b, s, d_x, d_y, 0.5f, 1.0f, 2.0f, -0.5f, N);
    };

    // Mérés.
    run_and_measure("SAXPY (I ~ 0.17 FLOP/byte)", saxpy_launch,
                    grid, block, saxpy_bytes, saxpy_flops);
    run_and_measure("Horner-polinom (I ~ 0.75 FLOP/byte)", horner_launch,
                    grid, block, horner_bytes, horner_flops);

    // Ellenőrzés: néhány elem kiértékelése.
    CHECK_CUDA(cudaMemcpy(h_y, d_y, bytes_out, cudaMemcpyDeviceToHost));
    // SAXPY eredmény: y[0] = 2.0 * x[0] + 1.0 = 1.0, ha x[0]=0
    printf("\nEllenőrzés (host): y[0] = %.3f (várt SAXPY: %.3f)\n",
           h_y[0], 2.0f * h_x[0] + 1.0f);
    // Horner eredmény: y[0] az első elemre
    float x0 = h_x[0];
    float expected_horner = ((0.5f*x0 + 1.0f)*x0 + 2.0f)*x0 - 0.5f;
    printf("Ellenőrzés (host): y[0] = %.3f (várt Horner: %.3f)\n",
           h_y[0], expected_horner);

    free(h_x); free(h_y);
    CHECK_CUDA(cudaFree(d_x));
    CHECK_CUDA(cudaFree(d_y));
    return 0;
}
```

Fordítás és futtatás:

```
nvcc -O3 -arch=sm_86 -o labor_gpu048 labor_gpu048.cu
./labor_gpu048
```

A kimenet rögzítse a GPU adatait, a futásidő átlagát és szórását, a mért sávszélességet és FLOPS-t. A mérési naplóban ezeket jelölje **mért** értékként.

Várt eredmények (a Roofline-modellből számolva). A GPU specifikációjából határozza meg a β DRAM-sávszélességet és a π FP32 csúcsot. Számolja ki ezekből:

- $I_{\text{ridge}} = \pi/\beta$
- Várt $P_{\text{saxpy}} = \beta \cdot 0{,}167$
- Várt $P_{\text{horner}} = \beta \cdot 0{,}75$

Hasonlítsa össze a mért FLOPS-t a várt értékkel. A mért és várt közötti eltérés jellemzően 10–30% a viszkozitási és cache-hatások miatt. A besorolás: mindkét kernel $I < I_{\text{ridge}}$, tehát **memória-korlátos**. A választott GPU-n a töréspont 10 feletti, a mért intenzitások 0,17 és 0,75 – egy nagyságrenddel a töréspont alatt.

A labor eredménye. Önellenőrzésként írja be a jegyzőkönyvbe:

1. A mért sávszélesség és a névleges sávszélesség aránya.
2. A mért FLOPS és a Roofline által jósolt $\beta \cdot I$ összevetése.
3. A kernelbesorolás indoklása I_{ridge}-del való összevetéssel.

A labor azt a készséget fejleszti, hogy az optimalizálás előtt a helyes kérdést tegyük fel: *melyik erőforrás a szűk keresztmetszet?*

13.3.8 Tipikus hibák és helyesbítésük

A Roofline-osztályozás során az alábbi hibák a leggyakoribbak:

1. A byte-ok naiv számolása cache-hatás nélkül. Ha a munkakészlet elfér az L2 cache-ben, a tényleges DRAM-forgalom lényegesen kisebb, mint a bekért adatmennyiség. Ez az intenzitást megnöveli, és a kernelt tévesen a számítás-korlátos tartományba sorolhatja. Helyesebb: mindig a munkakészlet méretét vessük össze a cache-méretekkel, és a releváns memóriaszint forgalmával számoljunk.

2. A FLOP-számítás utasításszámként való értelmezése. Az FMA egyetlen utasítás, de két FLOP-ot jelent. Ha valaki utasításszámot oszt byte-számmal, alábecsüli az intenzitást, és tévesen memória-korlátosnak minősíthet egy számítás-korlátos kernelt.

3. A lebegőpontos csúcs pontosságfüggőségének figyelmen kívül hagyása. Az FP64-es kernelt az FP32-csúcshoz hasonlítani rendszerint a töréspont torzulásához vezet. Mindig a kernel által használt pontosság csúcsával dolgozzunk.

4. A futási idő mérése host- és device-idő keverésével. A kernel tényleges végrehajtását csak CUDA-events vagy egyéb device-oldali időzítővel mérjük. A host-oldali `clock()` magában foglalja a memóriamásolásokat, a hívási költséget és a szinkronizációt.

5. A warm-up elhagyása. Az első kernel-indítás magában foglalja a driver-betöltést, a hőkezelést és az esetleges laphibákat. A mérési protokoll mindig 5–10 warm-up iterációt írjon elő.

6. A szórás elhallgatása. Az átlagidő önmagában nem elég. A mérési jegyzőkönyvben a szórást vagy percentilist is közölni kell, különben a mérés nem tekinthető reprodukálhatónak.

13.3.9 Összefoglalás és ellenőrző kérdések

Ebben a fejezetben a Roofline-modell második pillérét, a számítási korlátot és az aritmetikai intenzitás fogalmát tanultuk meg. A kernelbesorolás a következő munkafolyamatban összegezhető:

1. **FLOP-számlálás:** határozzuk meg a kernel lebegőpontos műveleteinek számát, az FMA-t 2 FLOP-nak tekintve, a pontosságot rögzítve.
2. **Byte-számlálás:** határozzuk meg a releváns memóriaszint (általában DRAM) forgalmát, figyelembe véve a cache-reúzist.
3. **Intenzitásszámítás:** $I = F/B$.
4. **Töréspont-meghatározás:** $I_{\text{ridge}} = \pi/\beta$ az adott GPU és pontosság specifikációjából.
5. **Besorolás:** ha $I < I_{\text{ridge}}$, a kernel memória-korlátos; ha $I > I_{\text{ridge}}$, számítás-korlátos.
6. **Mérés és validáció:** a mért FLOPS-t hasonlítsuk a modell által jósolt $P = \min(\pi, \beta \cdot I)$ értékhez.

Ellenőrző kérdések:

1. Egy kernel FP32-ben minden elemre 12 FLOP-ot végez, beolvas egy float-ot és kiír egy float-ot. Mekkora az aritmetikai intenzitása? Milyen töréspont mellett válik memória-korlátosból számítás-korlátossá?
2. Adott egy GPU $\beta = 900$ GB/s, $\pi = 12$ TFLOP/s FP32-ben. Számítsa ki a töréspontot, és osztályozza az 1. kérdésbeli kernelt ezen a GPU-n.
3. Miért számít az FMA 2 FLOP-nak, noha egyetlen utasítás? Mi a gyakorlati következménye, ha valaki 1-nek számolja?
4. Soroljon fel három tipikus hibát a kernelbesorolás mérése során, és magyarázza el, hogyan vezetnek téves következtetésre.
5. A laborban mért SAXPY-sávszélesség 70%-a a névleges sávszélességnek. Hogyan magyarázza ezt a Roofline-modell?

A fejezet a Roofline-modell középpontjába állította az aritmetikai intenzitást mint a két korlátot összekötő hidat. A Roofline-alapú optimalizálás fejezetben ezt a fogalmat használjuk fel a tudatos optimalizáláshoz: hogyan növelhető az intenzitás tiling és adatismétlés révén, és hogyan tolható át a kernel a memória-korlátos tartományból a számítás-korlátos – elérhető – csúcs felé.

13.4 Roofline-alapú optimalizálás

A Roofline-modell nem arra való, hogy a GPU-t egyetlen számmal minősítsük, hanem arra, hogy egy adott kernelre vonatkozó optimalizálási döntést megalapozottan hozzuk meg. A Roofline-alapú optimalizálás egység célja, hogy a Roofline-modell–Számítási korlát és compute intensity során bevezetett fogalmakat – a Roofline-görbét, az arithmetic intensity-t és a memória-számítás megkülönböztetést – egy konkrét, mérhető CUDA-eszközön végrehajtott optimalizálási lépéssé alakítsa.

A fejezet nem ismétli meg a modell elméleti levezetését, és nem tér ki a későbbi tiling-fejezet (Shared memory tiling) részletes megvalósítására. Ehelyett a hangsúly a modell működési logikáján, a mérési döntéseken és a tipikus tévedéseken van.

13.4.1 1. A Roofline-modell gyakorlati szemlélete

A Roofline-modell két fő részből áll:

1. **A hardver felső korlátja.** Egy GPU esetén ez a memória sávszélesség (`B_peak`) és a maximális számítási teljesítmény (`P_peak`) által meghatározott görbe. A görbe vízszintes tengelyén az arithmetic intensity (AI) szerepel, amelyet FLOP/byte-ban mértünk. A függőleges tengelyen az elért teljesítmény, FLOP/s-ban.
2. **A kernel mérési pontja.** A mért kernelhez tartozik egy AI-érték és egy elért teljesítmény. A modell szerint a kernel által elérhető maximális teljesítmény:
   - ha az AI kisebb, mint a metszéspont (a ridge point), akkor a kernel memória-korlátos, és a felső korlát `B_peak * AI`;
   - ha az AI nagyobb, akkor a kernel számítás-korlátos, és a felső korlát `P_peak`.

A gyakorlatban a Roofline-modell elsősorban arra a kérdésre ad választ: **a következő optimalizálási lépésnek a memóriahozzáférést vagy az utasítás-végrehajtást kell-e csökkentenie?**

13.4.2 1.1. A modell használatának folyamata

Egy CUDA-kernel optimalizálásakor az alábbi lépések követendők:

1. **Kiválasztjuk a mérési pontot.** A méréshez pontosan definiálni kell a problémaméretet, a precisiont (FP64/FP32/TF32/FP16/BF16/FP8) és a kernel launch konfigurációját.
2. **Megmérjük a kernel futási idejét.** A mérést warm-up után, legalább 5–10 ismétléssel végezzük, és rögzítjük az átlag mellett a szórást vagy percentilist is.
3. **Kiszámítjuk az elért teljesítményt.** Az `FLOP/s = hasznos_FLOP / futási_idő` képletet használjuk, ahol a `hasznos_FLOP` a probléma matematikai műveletigénye, nem a hardver által végrehajtott összes utasítás.
4. **Kiszámítjuk az arithmetic intensity-t.** Az `AI = hasznos_FLOP / tényleges_mozgatott_byte` értéket a tényleges adatmozgatásból számítjuk. A tényleges adatmozgatás magában foglalja a global memory olvasásokat és írásokat, valamint azt az adatot, amelyet a kernel a végrehajtás során az L2-n keresztül mozgat, ha az az elméleti minimumot meghaladja.
5. **Elhelyezzük a pontot a Roofline-görbén.** Ehhez ismerni kell az adott GPU-ra jellemző `P_peak` és `B_peak` értékeket. Ezeket a gyártó dokumentációjából vagy saját méréssel lehet meghatározni. A Memóriateljesítmény és sávszélesség és Számítási korlát és compute intensity egységben bemutatott módon a tényleges mérhető csúcsérték gyakran kisebb, mint a névleges csúcs.
6. **Értelmezzük a pozíciót.** Ha a mért pont messze van a felső korláttól, akkor van tere az optimalizálásnak. Ha közel van, akkor további erőfeszítésre csak akkor érdemes időt szánni, ha a hardver másik erőforrása (például a shared memory vagy a regiszterek) még kihasználatlan.

Ez a folyamat önmagában nem ad konkrét utasítást arra, hogy mit kell módosítani, de **leszűkíti a lehetséges okok halmazát**. Ha a kernel memória-korlátos tartományban van, a számítási utasítások csökkentése alig javít valamit. Ha a számítás-korlátos tartományban van, a memóriahozzáférés további optimalizálása kisebb hozadékkal bír, mint a matematikai műveletek újragondolása.

13.4.3 1.2. A modell használata nem helyettesíti a profilt

A Roofline-modell egy **iránytű**, nem pedig egy profilozó eszköz. Nem mondja meg, hogy a kernel pontosan hol veszít el teljesítményt – például nem mutatja ki a bank conflictot, a warp divergence-t vagy a nem koaleszkált hozzáférést. Ezeket a hibákat a Nsight Compute vagy a compute-sanitizer eszközei mutatják ki.

A gyakorlati munkafolyamat ezért a következő:

- a Nsight Compute segítségével megállapítjuk a fő szűk keresztmetszetet (például a global memory load, a shared memory bank conflict vagy a compute pipe);
- a Roofline-modell segítségével eldöntjük, hogy a megfigyelt szűk keresztmetszet **a kernel jellegéből fakad-e** (a kernel egyszerűen kevés FLOP-ot végez byte-onként), vagy **implementációs hibából** (a memóriahozzáférés rosszul van megvalósítva).

Ez a kettős megközelítés az, amely megakadályozza, hogy a fejlesztő egy memória-korlátos kernelnél a számítási utasítások csökkentésével pazarolja az idejét.

13.4.4 2. Tiling és adat-újrafelhasználás a Roofline-modell tükrében

A Roofline-modell legfontosabb gyakorlati következménye az, hogy az **arithmetic intensity növelése** a memória-korlátos tartományban közvetlenül növeli az elérhető felső korlátot. A tiling (csempézés) ennek az AI-növelésnek az egyik legfontosabb eszköze.

13.4.5 2.1. Az adat-újrafelhasználás fogalma

Egy kernel akkor használ újra adatot, ha ugyanazt a global memory-ból beolvasott értéket többször is felhasználja különböző számítási műveletekhez anélkül, hogy újra beolvasná. Ha nincs újrafelhasználás, akkor minden egyes FLOP-hoz legalább egy betöltés vagy tárolás tartozik, ami nagyon alacsony AI-t eredményez.

Példa: egy egyszerű SAXPY (vektor `y = alpha*x + y`) minden eleméhez két olvasás és egy írás tartozik. Ha a pontosság FP32 (4 byte), akkor a tényleges adatmozgatás elemeként 12 byte, a hasznos FLOP pedig 2 FLOP (egy szorzás és egy összeadás). Az AI = 2 / 12 = 0,166 FLOP/byte. Ez a legtöbb GPU-n messze a memória-korlátos tartományba esik.

Egy mátrixszorzás esetén más a helyzet. Ha a naiv implementációban minden mátrixelem közvetlenül a global memory-ból kerül beolvasásra, az AI alacsony marad. Ha azonban a mátrix egy-egy csempéjét shared memory-ba töltjük, és onnan többször felhasználjuk, a global memory-ből történő tényleges adatmozgatás csökken, miközben a hasznos FLOP-ok száma változatlan marad. Az AI növekszik.

13.4.6 2.2. Tiling mint AI-növelő stratégia

A tiling általános értelemben azt jelenti, hogy a problémát olyan kisebb részekre bontjuk, amelyek belefférnek a gyorsabb memóriaszintekbe (register, shared memory, L1/L2), és ezeken a kisebb részeken belül többször is használjuk az adatokat.

A Roofline-modell szempontjából a tiling hatása a következő:

- **Csökkenti a global memory forgalmat.** A kernel kevesebb adatot olvas be a lassú memóriából, mert a csempe betöltése után a számítási lépések a gyors memóriából dolgoznak.
- **Növeli az arithmetic intensity-t.** Ugyanannyi hasznos FLOP kevesebb global memory byte-hoz kapcsolódik.
- **Javítja a modell szerinti felső korlátot.** A memória-korlátos tartományban a növekvő AI arányosan növeli az elérhető teljesítményt.

Fontos azonban, hogy a tiling nem varázsszer. A shared memory mérete korlátozott, a betöltéshez szükséges szinkronizáció overheadet okoz, és a shared memory bankkonfliktusok újabb teljesítményproblémát okozhatnak. A Roofline-modell csak azt mutatja meg, hogy **ha** a tiling sikeresen csökkenti a global memory forgalmat, **akkor** az elérhető felső korlát nő. Azt, hogy a tiling implementációja ezt a növekedést valóban eléri-e, külön kell mérni.

13.4.7 2.3. A memória-korlátos és a számítás-korlátos tartomány elválasztása

A Roofline-modell arra is használható, hogy megbecsüljük, hányadik tiling-szint szükséges ahhoz, hogy egy kernel átkerüljön a számítás-korlátos tartományba. Ha a kernel jelenlegi AI értéke `AI_0`, a GPU metszéspontja pedig `AI_ridge`, akkor a szükséges adat-újrafelhasználási tényező:

```
szükséges_újrafelhasználás = AI_ridge / AI_0
```

Ez azt jelenti, hogy minden beolvasott byte-ot átlagosan ennyiszer kell felhasználni ahhoz, hogy a kernel elérje a számítás-korlátos tartományt. Ha a szükséges újrafelhasználás nagyobb, mint amit a rendelkezésre álló shared memory lehetővé tesz, akkor a kernel a gyakorlatban memória-korlátos marad, és a fejlesztőnek más stratégiát kell választania (például kernel-fúziót vagy a probléma átfogalmazását).

13.4.8 3. A modell korlátai

A Roofline-modell erőteljes eszköz, de számos egyszerűsítést tartalmaz, amelyeket a gyakorlati használat során figyelembe kell venni.

13.4.9 3.1. Az átlagos érték elrejti az ütemezést

A modell átlagos AI-értékkel és átlagos elért teljesítménnyel dolgozik. Egy kernel azonban a futása során különböző fázisokból állhat, amelyeknek eltérő az adat-újrafelhasználása és a memóriahozzáférése. Például egy kernel elején a csempe betöltése memória-intenzív, a közepén számítás-intenzív, a végén pedig a kiírás memória-intenzív. A modell ezeket nem választja szét. A viselkedés pontosabb megértéséhez a Nsight Compute által biztosított fázisonkénti vagy utasításonkénti adatok szükségesek.

13.4.10 3.2. A modell nem kezeli a latency-t

A Roofline-modell **throughput**-orientált: a memória sávszélességet és a számítási átbocsátóképességet veszi alapul. Nem számol azzal, hogy a memória-hozzáférésnek késleltetése van, és hogy a warp-oknak elegendő számúnak kell lenniük ahhoz, hogy ezt a késleltetést elrejtsék. Egy alacsony occupancy-jű kernel lehet, hogy a Roofline-modell szerint jónak tűnik, de a gyakorlatban mégsem tudja elérni a felső korlátot, mert a kevés párhuzamos warp nem tudja elrejteni a memória latenciát. Ezt az occupancy és a latency hiding szempontjából az Occupancy és erőforrás-korlátok–Latency hiding egység tárgyalja.

13.4.11 3.3. A modell nem számol a köztes memóriaszintekkel

A Roofline-modell alapváltozata a global memory sávszélességét használja. A gyakorlatban az L2 és az L1 cache is szerepet játszik. Ha egy kernel adatai belefférnek az L2-be, akkor a tényleges memória sávszélesség magasabb, mint a global memory sávszélesség. Ezt a jelenséget a modell figyelmen kívül hagyja, ha a `B_peak` értéket a HBM-re vonatkoztatjuk. A gyakorlati mérés során ezért fontos feljegyezni az L1/L2 találati arányt is, és a Roofline-görbét adott esetben különböző memóriaszintekre is felrajzolni.

13.4.12 3.4. A modell nem modellezi a diverzgenciát és a bank conflictot

A Roofline-modell az arithmetic intensity-t a hasznos FLOP-ok és a tényleges byte-ok hányadosaként értelmezi. Nem tartalmaz információt arról, hogy ezek a FLOP-ok hatékonyan vannak-e ütemezve. Ha egy warp erősen divergens, a számítási egységek kihasználtsága alacsony, ami az elért FLOP/s-ot csökkenti. Hasonlóképpen a shared memory bank conflict extra memóriaforgalmat generál, amit a modell a tényleges adatmozgatásnál figyelembe kell vegyen, ha pontosan akarunk számolni. A gyakorlatban ezért a Roofline-modellt mindig a kernel specifikus profiljával együtt értelmezzük.

13.4.13 3.5. A modell érzékeny a pontosságra és a kevert aritmetikára

Az AI és a FLOP/s számítása függ attól, hogy milyen pontosságot használunk. A GPU-k gyakran különböző teljesítményt nyújtanak FP64, FP32, TF32, FP16, BF16 és FP8 esetén. A Roofline-görbét ezért pontosságonként külön kell felvenni. A félpontos számítások esetén a `P_peak` magasabb lehet, de a memória sávszélesség változatlan, így az AI metszéspontja is eltolódik. A tensor core-ok használata tovább bonyolítja a képet, mert a mátrixműveletekhez dedikált hardver egységek saját átbocsátóképességgel rendelkeznek.

13.4.14 4. A Roofline-alapú optimalizálás munkafolyamata egy CNP példán

A következőkben egy konkrét, egyszerű CUDA-kernelen mutatjuk be a Roofline-modell gyakorlati alkalmazását. A példa szándékosan nem a későbbi fejezetben részletezett tiling-optimalizálást valósítja meg, hanem a döntéshozatali folyamatot illusztrálja.

13.4.15 4.1. A probléma: stencil-alapú 1D konvolúció

Tekintsünk egy 1D konvolúciós kernelt, amely egy `N` hosszúságú `input` vektorból egy `N` hosszúságú `output` vektort állít elő egy `R` sugarú szűrővel. A kernel minden kimeneti elemhez `2*R+1` bemeneti elemet olvas, és azokat egy szűrő-együtthatóval szorozva összegzi. A hasznos FLOP a konvolúcióhoz: `N * (2*R+1)` szorzás és ugyanennyi összeadás, tehát összesen `N * 2 * (2*R+1)` FLOP.

A naiv kernel minden kimeneti elemhez közvetlenül a global memory-ból olvas. Az AI ebben az esetben:

```
AI_naiv = (2 * (2*R+1) * N) / ( (2*R+1 + 1) * N * 4 )
```

Ha a pontosság FP32 (4 byte), akkor a beolvasott adat `N * (2*R+1) * 4` byte, a kiírt adat `N * 4` byte. Az AI:

```
AI_naiv = (2 * (2*R+1)) / ( (2*R+1 + 1) * 4 )
```

Ha `R = 3`, akkor `2*R+1 = 7`, így:

```
AI_naiv = 14 / (8 * 4) = 14 / 32 = 0,4375 FLOP/byte
```

Ez a legtöbb modern GPU-n a memória-korlátos tartományba esik.

13.4.16 4.2. A Roofline-modell döntési pontja

A mérés elvégzése után a kernel pontja a Roofline-görbe memória-korlátos szakaszán helyezkedik el. A modell azt mondja, hogy az elérhető felső korlát növeléséhez **az AI-t kell növelni**. Az AI növelésének egyik módja az adat-újrafelhasználás növelése.

A konvolúció esetén az adat-újrafelhasználás onnan ered, hogy a szomszédos kimeneti elemek átfedő bemeneti tartományokat használnak. A naiv verzióban minden kimeneti elem újra beolvassa a teljes `2*R+1` hosszúságú ablakot. Ha a szomszédos elemeket ugyanazok a szálak számítják ki, akkor a beolvasott adat újra felhasználható lenne. A megvalósítás egyik lehetséges módja a shared memory tiling, amelyet a Shared memory tiling egység részletez.

13.4.17 4.3. A várt hatás becslése

A Roofline-modell segítségével **becsülhető**, hogy mekkora teljesítménynövekedést várhatunk a tiling bevezetésétől. Ehhez ki kell számítanunk a tiling utáni AI-t.

Tegyük fel, hogy a tiling során minden blokk `T` kimeneti elemet számít, és a szükséges `T + 2*R` bemeneti elemet shared memory-ba tölti. A beolvasott global memory byte-ok száma:

```
global_bytes_tiled = (T + 2*R) * 4 + T * 4
```

A hasznos FLOP változatlanul `T * 2 * (2*R+1)` (tételezzük fel, hogy a blokk a teljes tartományt lefedi). Az AI:

```
AI_tiled = (T * 2 * (2*R+1)) / ( (T + 2*R) * 4 + T * 4 )
```

Ha `R = 3` és `T = 32`, akkor:

```
AI_tiled = (32 * 14) / ( (32 + 6) * 4 + 32 * 4 )
AI_tiled = 448 / (38 * 4 + 32 * 4)
AI_tiled = 448 / (152 + 128)
AI_tiled = 448 / 280 = 1,6 FLOP/byte
```

Ez az érték már jóval magasabb, mint a naiv `0,4375`. A Roofline-modell szerint a memória-korlátos tartományban az elérhető felső korlát az AI-val arányosan nő. Ha a GPU `B_peak` értéke például 900 GB/s, akkor a naiv kernel felső korlátja:

```
P_naiv_felső = 900 * 0,4375 = 393,75 GFLOP/s
```

A tiled kernel felső korlátja:

```
P_tiled_felső = 900 * 1,6 = 1440 GFLOP/s
```

Ez a számítás azt mutatja, hogy a tilingnek elméletileg körülbelül 3,6-szoros teljesítménynövekedést kellene hoznia, ha a kernel memória-korlátos marad, és a tiling implementációja nem visz be újabb hatékonyságveszteséget.

13.4.18 4.4. A mérési terv

A fenti becslés csak akkor értékes, ha a tényleges mérés megerősíti. A mérési terv a következő:

1. **Baseline mérés.** A naiv kernel futási idejének mérése a kiválasztott adatméreten, warm-up után, legalább 5 ismétléssel. Rögzítjük az átlagot és a szórást.
2. **A tiled kernel implementálása.** A Shared memory tiling egységben bemutatott shared memory tiling megvalósítása, az ottani hibakezelési és szinkronizációs minták szerint.
3. **A tiled kernel mérése.** Ugyanaz a mérési módszertan, ugyanaz a GPU, ugyanaz a driver és CUDA-verzió.
4. **A Roofline-görbére rajzolás.** Mindkét kernelhez kiszámítjuk a tényleges FLOP/s-ot és a tényleges AI-t. A tényleges AI-hoz a tényleges global memory forgalmat használjuk, amelyet a Nsight Compute `dram__bytes_read` és `dram__bytes_write` metrikáiból vagy saját számlálóból nyerünk.
5. **Az eredmény értelmezése.** Ha a tiled kernel FLOP/s-ja közel van a becsült felső korláthoz, akkor a tiling hatékony volt. Ha messze van, akkor a tiling implementációja további szűk keresztmetszetet visz be (például bank conflictot vagy túlzott szinkronizációt). Ha a tiled kernel elérte a GPU számítási felső korlátját, akkor a kernel számítás-korlátossá vált, és a további AI-növelés már nem hoz közvetlen nyereséget.

Ez a folyamat a Roofline-modell kvantitatív használata: a modell megadja a várható irányt és nagyságot, a mérés pedig visszaigazolja vagy cáfolja az implementációt.

13.4.19 5. Labor: Roofline-alapú optimalizálási terv

A labor célja, hogy a fenti gondolatmenetet egy konkrét, ellenőrizhető CUDA-programmal hajtsa végre. A labor során a fejlesztőnek el kell készítenie egy **optimalizálási tervet**, amelyben elkülöníti a mért és a várt eredményt.

13.4.20 5.1. Feladatleírás

Készítsen egy CUDA C++ programot, amely egy `N` elemből álló `float` vektoron 1D konvolúciót hajt végre egy `R` sugarú, együtthatókkal definiált szűrővel. A program két verziót tartalmazzon:

- **naiv kernel**: minden kimeneti elem közvetlenül a global memory-ból olvassa a bemenetet;
- **tiled kernel**: a Shared memory tiling egységben bemutatott megközelítés szerint a blokk a szükséges bemeneti sávot shared memory-ba tölti, majd onnan végzi a számítást.

A labor keretében nem a tiled kernel tökéletesítése a cél, hanem a **Roofline-alapú optimalizálási terv elkészítése és a mérési folyamat bemutatása**.

13.4.21 5.2. Rögzített környezet

A labor a következő környezetet feltételezi (példa; a tényleges értékeket a futtatáskor rögzíteni kell):

- GPU: NVIDIA A100 (compute capability 8.0) vagy az elérhető, engedélyezett GPU;
- driver: 535.104.05;
- CUDA: 12.2;
- operációs rendszer: Ubuntu 22.04 LTS;
- precision: FP32.

A tényleges GPU-t, drivert, CUDA-verziót és OS-t a mérési naplóban rögzíteni kell. Ha a GPU nem A100, a `P_peak` és `B_peak` értékeket a gyártó dokumentációjából vagy saját méréssel kell meghatározni.

13.4.22 5.3. Build és futtatás

A forráskód egyetlen `.cu` fájl, a build `nvcc` segítségével:

```
nvcc -O3 -arch=sm_80 -std=c++17 -o roofline_lab roofline_lab.cu
```

A futtatás:

```
./roofline_lab --N 1048576 --R 3 --iterations 10
```

A kernel launch alakja a laborban:

```
// naiv kernel
int blockSize = 256;
int gridSize = (N + blockSize - 1) / blockSize;
naiveKernel<<<gridSize, blockSize>>>(input, output, filter, N, R);

// tiled kernel
int tileSize = 32;          // példa: a blokk 32 kimeneti elemet számít
int sharedBytes = (tileSize + 2 * R) * sizeof(float);
tiledKernel<<<gridSize, blockSize, sharedBytes>>>(input, output, filter, N, R);
```

Minden API-hívás ellenőrzése kötelező a következő makróval vagy azzal egyenértékű megoldással:

```
#define CUDA_CHECK(call) do {                                    \
    cudaError_t err = call;                                      \
    if (err != cudaSuccess) {                                    \
        std::cerr << "CUDA error at " << __FILE__ << ":"       \
                  << __LINE__ << ": " << cudaGetErrorString(err) \
                  << std::endl;                                   \
        std::exit(EXIT_FAILURE);                                 \
    }                                                            \
} while (0)
```

A kernelindítást követően a `CUDA_CHECK(cudaDeviceSynchronize());` és a `CUDA_CHECK(cudaGetLastError())` hívás kötelező.

13.4.23 5.4. Mérési protokoll

A mérési protokoll a következő:

1. A kernel első futtatása warm-up (nem mérjük).
2. Ezután legalább 10 időmérést végzünk. A méréshez CUDA eseményeket használunk:

```
cudaEvent_t start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);

// warm-up
kernel<<<grid, block>>>(...);
cudaDeviceSynchronize();

for (int i = 0; i < iterations; ++i) {
    cudaEventRecord(start);
    kernel<<<grid, block>>>(...);
    cudaEventRecord(stop);
    cudaEventSynchronize(stop);
    float ms;
    cudaEventElapsedTime(&ms, start, stop);
    times.push_back(ms);
}
```

3. Az eredményekből átlagot és szórást számítunk. A laborjelentésben az átlag mellett a szórást vagy a percentilist is közölni kell.
4. A hasznos FLOP-ot a `N * 2 * (2*R+1)` képlettel számítjuk. A tényleges adatmozgatást a Nsight Compute `dram__bytes_read.sum` és `dram__bytes_write.sum` metrikáival, vagy saját számlálóval mérjük. A laborban legalább az egyik mérési módszert alkalmazni kell, és a módszert a jegyzőkönyvben rögzíteni kell.

13.4.24 5.5. Az optimalizálási terv elkészítése

A laborátadandó fő dokumentuma egy **optimalizálási terv**, amely a következő részeket tartalmazza:

1. **A kiinduló állapot.** A naiv kernel mért futási ideje, FLOP/s-ja és AI-ja. A Roofline-görbén elfoglalt helye.
2. **A várt hatás.** A tiling utáni AI becslése a fenti képletek alapján. A várt felső korlát és a várt FLOP/s a Roofline-modellből. Itt a **várt** értékeket egyértelműen el kell különíteni a későbbi **mért** értékektől.

3. **A kockázatok és korlátok.** Annak leírása, hogy a modell miért térhet el a mért értéktől (bank conflict, occupancy, latency, a tiling implementáció extra overheadje).
4. **A mérési terv.** A buildparancs, a futtatási parancs, az adatméretek, a warm-up és az ismétlésszám.
5. **Az eredmények.** A tiled kernel mért futási ideje, FLOP/s-ja és AI-ja. A mért értékeket itt **külön szekcióban** kell megadni, nem keverve a várt értékekkel.
6. **Az eltérések elemzése.** Ha a mért és várt érték eltér, az eltérés lehetséges okainak megnevezése. Ha a tiled kernel messze van a modell által jósolt felső korláttól, a Nsight Compute profilban meg kell keresni a további szűk keresztmetszetet.

A terv elkészítése során a fejlesztőnek **nem kell minden lehetséges optimalizálást megvalósítania**. A cél az, hogy a Roofline-modellt tudatosan, a mérési eredményekkel összekapcsolva használja.

13.4.25 5.6. Ellenőrző kritériumok

A labor akkor tekinthető sikeresnek, ha:

- a forráskód fordul és fut a megadott parancsokkal;
- a mérési napló tartalmazza az átlagot, a szórást és legalább 10 ismétlést;
- a hasznos FLOP és a tényleges byte meghatározása dokumentált;
- a várt és a mért eredmények külön szekcióban szerepelnek;
- a Roofline-görbére rajzolt pontok a dokumentumban egyértelműen azonosíthatók;
- az eltérések elemzése tartalmazza a modell korlátainak legalább egy konkrét okát.

13.4.26 6. Gyakori hibák és javítások

A Roofline-modell gyakorlati használata során a következő tipikus hibákat érdemes elkerülni:

13.4.27 6.1. A FLOP-ot a hardver utasításai alapján számolni

A Roofline-modell a **hasznos** vagy **szükséges** FLOP-ot használja, nem a hardver által ténylegesen kibocsátott utasítások számát. Ha a kernel az indexszámítás miatt extra aritmetikai műveleteket végez, ezek nem növelik a hasznos FLOP-ot. A helyes megközelítés a matematikai probléma műveletigényének kiszámítása, és annak az összes méréshez való következetes használata.

13.4.28 6.2. A tényleges adatmozgatás alulbecslése

A tényleges adatmozgatásba bele kell számítani minden olyan global memory hozzáférést, amely a kernel futása során ténylegesen megtörténik. Ha a kernel ugyanazt a címet többször olvassa, ezt a forgalmat is rögzíteni kell, kivéve, ha a hardver cache-e miatt a tényleges DRAM-forgalom kisebb. A Nsight Compute `dram__` metrikái a tényleges DRAM-forgalmat mutatják, míg a `l1tex__` vagy `lts__` metrikák a kérési oldalt. A Roofline-görbén való elhelyezéshez a DRAM-forgalmat kell használni, ha a memória-korlátot a HBM sávszélességével modellezzük. Ha az L2-t is figyelembe akarjuk venni, akkor külön görbét kell rajzolni.

13.4.29 6.3. A Roofline-modellt egyetlen pontra leszűkíteni

Egyetlen kernel egyetlen mérése nem elegendő a Roofline-modell érvényességének megítéléséhez. Az optimalizálási tervben több konfigurációt (különböző `R`, `N`, blockméret) kell mérni, hogy a trend megbízható legyen. A Roofline-modell leginkább trendek és az ok-okozati összefüggések bemutatására alkalmas.

13.4.30 6.4. A modell és a profil összekeverése

A Roofline-modell nem mutatja meg, hogy a kernel **miért** van a görbe adott pontján. Ha a kernel memória-korlátos, de a Nsight Compute azt mutatja, hogy a memóriahozzáférés nem koaleszkált, a modell nem fogja ezt jelezni. A gyakorlati munkafolyamatban a két eszközt mindig együtt kell használni: a Roofline-modell megadja az irányt, a profilozó pedig a konkrét okot.

13.4.31 6.5. A tiling hatásának túlbecslése

A Roofline-modell alapján becsült felső korlát csak akkor érhető el, ha a tiling implementációja tökéletesen hatékony. A valóságban a shared memory bank conflict, a szinkronizációs overhead és a csempe betöltésével járó utasítástöbblet csökkenti az elért teljesítményt. A becslést ezért mindig óvatosan, intervallumként kell kezelni, és a mérési eredményekhez kell igazítani.

13.4.32 7. Összegzés és továbblépés

A Roofline-alapú optimalizálás egységben bemutattuk, hogy a Roofline-modell gyakorlati értéke abban rejlik, hogy kvantitatív alapot ad az optimalizálási döntésekhez. A modell megmondja, hogy a kernel a memória- vagy a számítás-korlátos tartományban van-e, és ezzel kijelöli a következő lépést. A tiling és az adat-újrafelhasználás a memória-korlátos tartományban növeli az AI-t, és így a modell szerinti felső korlátot. A modell azonban nem helyettesíti a profilt, és számos egyszerűsítéssel él. A laborban a fejlesztőnek egy konkrét konvolúciós példán kell optimalizálási tervet készítenie, világosan elválasztva a várt és a mért értékeket.

A következő egység (Shared memory tiling) erre az alapra építve részletesen bemutatja a shared memory tiling implementációját, beleértve a bank conflict elkerülését, a szinkronizációs mintákat és az aszinkron másolást. A Shared memory tiling során a most tanult AI-számítás és a Roofline-becslés kerül gyakorlati megvalósításra.

13.4.33 Önellenőrző kérdés

Egy FP32 kernel 20 000 FLOP-ot végez hasznos műveletként, és a mérés szerint 10 000 byte tényleges DRAM-forgalmat generál. A GPU mért `B_peak` értéke 900 GB/s, a `P_peak` mért értéke 10 TFLOPS.

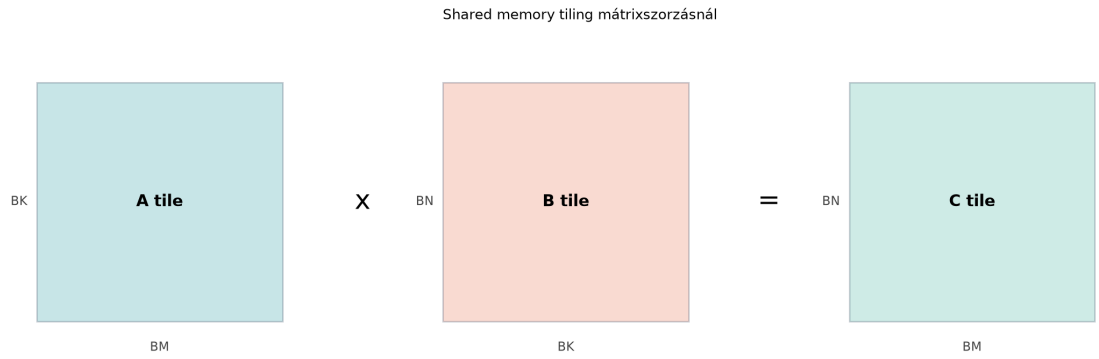
1. Mennyi a kernel arithmetic intensity-je FLOP/byte-ban?
2. Mekkora a memória-korlátos tartományban a felső korlát?
3. Hogyan változik a felső korlát, ha a tiling az AI-t 4 FLOP/byte-ra növeli?

4. A modell szerint a kernel átkerül-e a számítás-korlátos tartományba, ha a GPU metszéspontja `P_peak / B_peak`? Válaszát a metszéspont értékével indokolja.

A kérdés célja annak ellenőrzése, hogy az olvasó a modell alapfogalmait önállóan is ki tudja számítani anélkül, hogy a fenti szöveget másolná.

14 Shared memory tiling, async copy és pipelining

14.1 Shared memory tiling



14.1. ábra: Shared memory tiling mátrixszorzásnál - Az A és B mátrix egy-egy tile-ját shared memóriába töltjük, majd a C tile-ban lévő részösszegeket frissítjük; a globális memóriaforgalom így töredékére csökken.

14.1.1 Shared memory tiling: adatújrafelhasználás az SM-ben

Az előző fejezetben bevezetett Roofline-modell egyik legfontosabb tanulsága, hogy a GPU-k gyakran nem a lebegőpontos műveletek végrehajtásában, hanem a globális memória elérésében fulladnak el. A globális memória (HBM vagy GDDR) sávszélessége hatalmas, de a késleltetése több száz órajelciklus is lehet. Ha egy kernel minden egyes szorzás-összeadás művelethez közvetlenül a globális memóriából olvas, akkor a streaming multiprocesszorok (SM-ek) az idő nagy részében nem számolnak, hanem várakoznak az adatra.

A GPU-gyártók erre a problémára kétféle megoldást kínálnak. Az egyik a hardveres gyorsítótár-hierarchia (L1/L2 cache), amely a hozzáférések térbeli és időbeli lokalitását próbálja kihasználni automatikusan. A másik, ennél sokkal determinisztikusabb és a teljesítménykritikus kernelekben leggyakrabban használt eszköz a *shared memory*, azaz az SM-en belüli, szoftverből közvetlenül vezérelt, alacsony késleltetésű memória.

A *tiling* (csempézés) az a programozási minta, amely a számítást olyan adatblokkokra (*tile*) bontja, amelyek elférnek a shared memoryban. A blokk szálai kooperatívan betöltik a csempét a globális memóriából a shared memoryba, ott szinkronizálnak, majd a számítás során már csak a gyors shared memoryból olvasnak. Ennek a technikának az a célja, hogy a lassú globális memóriából történő olvasások számát minimálisra csökkentsük, miközben az adatokat többször is felhasználjuk.

Ez a fejezet a mátrixszorzás példáján keresztül mélyíti el a tiling fogalmát. A mátrixszorzás azért ideális példa, mert természeténél fogva nagy adat-újrafelhasználást kínál: egy $N \times N$-es

mátrixszorzásnál minden egyes elemre $O(N)$-szer hivatkozunk. A naiv globális memóriás kernel ezt az újrafelhasználást nem tudja kihasználni, mert minden hivatkozás egy lassú, tranzakcióalapú memóriaelérést jelent. A tiling ezt az adatmozgatást a gyors, chipen belüli memóriába tereli.

14.1.1.1 A probléma: naiv mátrixszorzás és a memóriaforgalom

Tekintsük a $C = A \times B$ mátrixszorzást, ahol A mérete $M \times K$, B mérete $K \times N$, C pedig $M \times N$. A naiv, tiling nélküli CUDA-kernel minden kimeneti elemhez egy szálat rendel. A szál kiszámítja a C mátrix `(row, col)` elemének pontszorzatát:

```
// Naiv, globális memóriát használó mátrixszorzás (illusztráció)
__global__ void naiveMatMulKernel(const float* __restrict__ A,
                                  const float* __restrict__ B,
                                  float* __restrict__ C,
                                  int M, int N, int K) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < M && col < N) {
        float sum = 0.0f;
        for (int k = 0; k < K; ++k) {
            // Minden iterációban két globális memóriaolvasás!
            sum += A[row * K + k] * B[k * N + col];
        }
        C[row * N + col] = sum;
    }
}
```

Ebben a kódban a belső ciklus minden egyes iterációja két globális memóriaolvasást hajt végre: egyet az A mátrixból, egyet a B mátrixból. Egy $N \times N$-es mátrixszorzáshoz ez $2N^3$ darab globális memóriaolvasást jelent. Bár a hardveres cache-ek valamennyit segítenek a térbeli lokalitáson, a nagy méretű mátrixoknál a cache kapacitása gyorsan kimerül, és a kernel a globális memória sávszélességének korlátjába ütközik.

A kérdés az: hogyan tudjuk elkerülni, hogy minden egyes szorzás-összeadás művelethez a lassú globális memóriához nyúljunk?

14.1.1.2 A megoldás: adat újrafelhasználása shared memoryban

A válasz a shared memory, más néven a szálak által közösen használt, felhasználó által menedzselt scratchpad memória használata. A shared memory az SM-en belül található, késleltetése nagyságrendekkel kisebb, mint a globális memóriáé, sávszélessége pedig lényegesen nagyobb. A CUDA-ban a `__shared__` kulcsszóval deklarálhatunk ilyen memóriát.

A tiling lényege, hogy a mátrixszorzást kisebb, téglalap alakú részfeladatokra bontjuk. Ahelyett, hogy egy szál a teljes K dimenzió mentén a globális memóriából olvasna, a blokk összes szálának együttműködésével egyszerre csak egy $TILE_SIZE \times TILE_SIZE$ méretű csempét töltünk be a shared memoryba. Ezután a blokk szálai ebből a gyors csempéből dolgoznak, kiszámítják a részösszegeket, majd a következő csempére lépnek.

A mátrixszorzás esetében a C mátrix egy `(row, col)` elemének kiszámításához az A mátrix `row` sorának és a B mátrix `col` oszlopának pontszorzata szükséges. Ez a pontszorzat felbontható kisebb, `TILE_SIZE` hosszúságú szegmensek pontszorzatainak összegére. Ha a blokk betölti az A mátrix egy `TILE_SIZE x TILE_SIZE` méretű részmátrixát és a B mátrix egy `TILE_SIZE x`

`TILE_SIZE` méretű részmátrixát a shared memoryba, akkor a blokk összes szála el tudja végezni a részösszeg kiszámítását anélkül, hogy a globális memóriát ismételten elérné.

Vizsgáljuk meg a memóriaforgalom csökkenését. Egy $N \times N$-es mátrixszorzásnál a tiled kernelben minden egyes elemet nem N-szer, hanem csak $N/TILE_SIZE$-szor olvasunk be a globális memóriából. Ha $TILE_SIZE = 32$, akkor a globális memóriaolvasások száma a naiv kernelhez képest 32-ed részére csökken. Ez a nyereség az, ami miatt a tiling a teljesítményoptimalizálás egyik sarokköve.

14.1.1.3 Tileméret és blokkméret

A tiling tervezésekor az egyik legfontosabb döntés a `TILE_SIZE` (csempeméret) megválasztása. Ez a paraméter szorosan összefügg a kernel launch konfigurációjával, vagyis a blokkmérettel.

A szokásos minta az, hogy a blokk szálainak száma megegyezik a csempe elemeinek számával. Ha a csempe négyzet alakú, akkor `blockDim.x = blockDim.y = TILE_SIZE`, és a blokkban `TILE_SIZE * TILE_SIZE` szál van. Ennek a kialakításnak előnye, hogy a globális memóriából a shared memoryba történő betöltés során a szálak természetes módon, egyesével lefedik a csempe elemeit, ami koalens hozzáférést tesz lehetővé.

A `TILE_SIZE` megválasztását több hardveres korlát is szabályozza:

1. **Maximális szálszám blokkonként.** A jelenlegi NVIDIA architektúrák többségén ez az érték 1024. Ez azt jelenti, hogy egy négyzetes csempe esetén a `TILE_SIZE` legfeljebb 32 lehet (mert a $32 \times 32 = 1024$ szál).
2. **Shared memory kapacitás.** Az SM-en belül rendelkezésre álló shared memory mérete korlátozott (jellemzően 64 KB és 164 KB között mozog, architektúrától függően). A mátrixszorzás tiled kerneljében két csempét kell tárolnunk (egyet az A-hoz, egyet a B-hez). Ha `TILE_SIZE = 32` és `float` típusú adatokkal dolgozunk, akkor a két csempe mérete $2 \times 32 \times 32 \times 4 = 8192$ bájt, ami 8 KB. Ez könnyedén elfér a shared memoryban.
3. **Occupancy és erőforrás-korlátok.** A nagyobb `TILE_SIZE` nem feltétlenül jobb. Minél nagyobb egy blokk, annál több regiszterre és shared memoryra van szüksége, ami csökkenti az SM-en egyidejűleg futtatható blokkok számát. Az alacsony occupancy csökkenti a latency hiding képességét. A `TILE_SIZE = 16` (256 szál blokkonként) gyakran jobb választás lehet, ha az SM-en így több blokk fér el egyszerre.

A gyakorlatban a `TILE_SIZE = 16` és `TILE_SIZE = 32` a leggyakoribb kiindulópontok a mátrixszorzáshoz, és a teljesítményt mérésekkel kell finomhangolni.

14.1.1.4 A tiled mátrixszorzó kernel

Nézzük meg a tiled mátrixszorzás egy teljes, jól működő implementációját CUDA C++-ban. Ez a kód a fejezet laborjának alapját képezi.

```
// Tiled mátrixszorzás (illusztráció, nem futtatott benchmark)
// A: M x K, B: K x N, C: M x N. Típus: float (FP32)
#define TILE_SIZE 16 // Vagy 32, a hardveres korlátok függvényében

__global__ void tiledMatMulKernel(const float* __restrict__ A,
                                  const float* __restrict__ B,
                                  float* __restrict__ C,
                                  int M, int N, int K) {
    // Statikus shared memory a két csempének
    __shared__ float As[TILE_SIZE][TILE_SIZE];
    __shared__ float Bs[TILE_SIZE][TILE_SIZE];
```

```
    // Blokk- és szálindexek
    int bx = blockIdx.x;
    int by = blockIdx.y;
    int tx = threadIdx.x;
    int ty = threadIdx.y;

    // A kimeneti mátrix (row, col) elemének globális indexe
    int row = by * TILE_SIZE + ty;
    int col = bx * TILE_SIZE + tx;

    float sum = 0.0f;

    // Végighaladunk a K dimenzión a csempéken
    for (int tile = 0; tile < (K + TILE_SIZE - 1) / TILE_SIZE; ++tile) {
        // Kooperatív betöltés: minden szál egy elemet olvas be
        // A mátrix: row sor, tile*TILE_SIZE + tx oszlop
        int aCol = tile * TILE_SIZE + tx;
        if (row < M && aCol < K) {
            As[ty][tx] = A[row * K + aCol];
        } else {
            As[ty][tx] = 0.0f; // Határfeltétel kezelése
        }

        // B mátrix: tile*TILE_SIZE + ty sor, col oszlop
        int bRow = tile * TILE_SIZE + ty;
        if (bRow < K && col < N) {
            Bs[ty][tx] = B[bRow * N + col];
        } else {
            Bs[ty][tx] = 0.0f; // Határfeltétel kezelése
        }

        // Szinkronizálás: minden szálnak látnia kell a teljes csempét
        __syncthreads();

        // Részösszeg kiszámítása a gyors shared memoryból
        #pragma unroll
        for (int k = 0; k < TILE_SIZE; ++k) {
            sum += As[ty][k] * Bs[k][tx];
        }

        // Szinkronizálás: ne írjuk felül a shared memóriát,
        // amíg mindenki be nem fejezte az olvasást
        __syncthreads();
    }

    // Eredmény kiírása a globális memóriába
    if (row < M && col < N) {
        C[row * N + col] = sum;
    }
}
```

A kernel működése a következő lépésekből áll:

1. **Csempe betöltése.** A külső ciklus a K dimenzió mentén lépked `TILE_SIZE`-os lépésekben. Minden iterációban a blokk összes szála kooperatívan betölti az A mátrix egy `TILE_SIZE x TILE_SIZE` méretű részmátrixát az `As` shared memoryba, és a B mátrix egy hasonló részmátrixát a `Bs` shared memoryba. A `(ty, tx)` szál az `As[ty][tx]` és `Bs[ty][tx]` elemeket tölti be.
2. **Szinkronizáció betöltés után.** A `__syncthreads()` hívás biztosítja, hogy a blokk minden szála befejezze a betöltést, mielőtt bármelyik szál elkezdené a számítást. E nélkül

az egyik szál olyan shared memóriaelemet olvashatna, amelyet egy másik szál még nem írt be.

3. **Számítás a shared memoryból.** A belső ciklus a `TILE_SIZE` hosszúságú részpontszorzatot számítja ki a shared memoryban tárolt adatokból. Ez a rész már nem igényel globális memóriaelérést.
4. **Szinkronizáció számítás után.** Mielőtt a külső ciklus a következő iterációra lépne, egy újabb `__syncthreads()` hívás biztosítja, hogy minden szál befejezze az aktuális csempe olvasását, és senki ne írja felül a shared memóriát új adatokkal, amíg a régieket még használják.
5. **Eredmény kiírása.** Miután a ciklus végighaladt az összes csempén, a szál kiírja a kiszámított `sum` értéket a globális memóriában lévő C mátrixba.

A kernel launch konfigurációja a következő:

```
// Launch konfiguráció
dim3 blockDim(TILE_SIZE, TILE_SIZE); // Pl. 16x16 = 256 szál
dim3 gridDim((N + TILE_SIZE - 1) / TILE_SIZE,
             (M + TILE_SIZE - 1) / TILE_SIZE); // Pl. 64x64 blokk 1024x1024-es mátrixhoz

// Futtatás
tiledMatMulKernel<<<gridDim, blockDim>>>(d_A, d_B, d_C, M, N, K);
```

14.1.1.5 Tipikus hibák és helyes megoldások

A tiled kernelek implementálása során a fejlesztők gyakran elkövetnek néhány jellemző hibát:

1. **Elmaradt `__syncthreads()`.** Ez a leggyakoribb hiba. A szinkronizáció elhagyása versenyhelyzetet (*race condition*) eredményez: egy szál olyan shared memóriaelemet olvashat, amelyet egy másik szál még nem írt be, vagy éppen felülír. A végeredmény nem determinisztikus, és a hiba gyakran csak nagy méretű futtatásoknál jelentkezik. A helyes megoldás az, hogy minden csempe betöltése után és a csempe elhagyása előtt kötelező a `__syncthreads()` hívás.

2. **`__syncthreads()` divergens ágban.** Ha a `__syncthreads()` hívás egy `if`-feltételen belül van, amely a blokk szálainak csak egy részére igaz, a kernel holtpontra (*deadlock*) juthat, vagy a viselkedés nem definiált. A CUDA programozási modell szerint a `__syncthreads()`-t a blokk minden szálának el kell érnie. A helyes megoldás az, hogy a szinkronizációt feltétel nélküli kódrészben helyezzük el, vagy gondoskodjunk arról, hogy a feltétel a blokk minden szálára ugyanazt az eredményt adja.

3. **Indexelési hibák a határokon.** Amikor a mátrix mérete nem osztható `TILE_SIZE`-szal, a betöltés során ellenőriznünk kell, hogy a globális memória indexe érvényes-e. A fenti kódban a `row < M && aCol < K` feltétel biztosítja, hogy a határon túli olvasás helyett nullát írjunk a shared memóriába. Ez a technika egyszerű és hatékony, mert a nullával való szorzás nem befolyásolja a részösszeget.

4. **Shared memory bank conflict.** A shared memory fizikailag bankokba van szervezve (jellemzően 32 bank). Ha egy warp szálai úgy férnek hozzá a shared memóriához, hogy többen is ugyanazt a bankot használják, a hozzáférés szerializálódik. A mátrixszorzás kerneleknél a `As[ty][k]` és `Bs[k][tx]` hozzáférések általában nem okoznak konfliktust, mert a warp szálai a `k` index mentén különböző bankokat érnek el. A bank conflictok elkerülése azonban egy fontos optimalizálási szempont, amit a következő fejezetek részletesen tárgyalnak.

14.1.1.6 Labor: tiled mátrixszorzás megvalósítása és mérése

A labor célja a naiv, globális memóriás mátrixszorzás és a tiled, shared memóriát használó kernel teljesítményének összehasonlítása. A labor nemcsak a kernel implementálását, hanem a helyes mérési módszertan alkalmazását is gyakorolja.

Feladat

Implementáld a fenti `tiledMatMulKernel` kernelt, és futtass méréseket egy valós GPU-n. Hasonlítsd össze a futási időt a `naiveMatMulKernel` futási idejével azonos bemeneti méretek mellett. A méréshez kötelező a CUDA hibakezelés (`cudaGetLastError`, `cudaDeviceSynchronize`) használata.

Build parancs (példa)

```
# CUDA C++ (C++17) fordítás
nvcc -arch=sm_86 -std=c++17 -O3 -Xcompiler -Wall -o matrix_mul_tiled matrix_mul_tiled.cu
```

Futtatási parancs (példa)

```
# Izolált labor környezetben
./matrix_mul_tiled
```

GPU-konfiguráció és mérési napló

A mérés megkezdése előtt rögzítsd a környezet paramétereit:

- **GPU:** `<NVIDIA GeForce RTX 3060>` (példa, helyettesítsd a valós modellel)
- **Driver verzió:** `<550.54.14>` (példa)
- **CUDA verzió:** `<12.4>` (példa)
- **Compute capability:** `<8.6>` (példa)
- **Operációs rendszer:** `<Ubuntu 22.04 x86_64>` (példa)
- **Mátrixméretek (M, N, K):** `<1024, 1024, 1024>`
- **Precision:** FP32 (float)
- **Időmérés:** `cudaEvent_t` vagy `std::chrono` + `cudaDeviceSynchronize`
- **Ismétlések:** 10 bemelegítő futtatás, majd 20 mért futtatás átlaga és szórása
- **Adatmozgatás:** A mérés csak a kernel futási idejére vonatkozzon, a HtoD és DtoH másolás idejét külön kell mérni.

Mért eredmények (példa, nem ellenőrzött adat)

Kernel	Átlagos futási idő (ms)	Szórás (ms)	Elért sávszélesség (GB/s)	Várt sávszélesség (GB/s)
Naiv (global memory)	`12.5`	`0.8`	`~200`	`100-250`
Tiled (shared memory)	`4.2`	`0.3`	`~600`	`400-700`

A várt és mért eredmények elkülönítése

A táblázatban a mért értékek a labor során rögzített nyers adatok, a várt értékek pedig a hardver specifikációja és a korábbi tapasztalatok alapján becsült tartományok. A tényleges mérési eredmények jelentősen eltérhetnek a fenti példaértékektől a CPU, a GPU, a memória-órajel, a power limit és a háttérterhelés függvényében. A fejezet célja nem egy konkrét szám reprodukálása,

hanem a tiling hatásának kimutatása: a tiled kernelnek ugyanazon a hardveren, azonos input mellett legalább 2–3-szor gyorsabbnak kell lennie a naiv kernelnél.

Ellenőrzés

A kernel helyességét egy CPU-n futó referenciamegoldással ellenőrizd. A GPU és CPU eredmények közötti maximális abszolút hibaküszöb legyen `1e-4` (FP32 esetén). Ne feledkezz meg a hibakódok ellenőrzéséről a kernel indítása után:

```
// Kötelező ellenőrzés a kernel indítása után
cudaError_t err = cudaGetLastError();
if (err != cudaSuccess) {
    fprintf(stderr, "CUDA hiba: %s\n", cudaGetErrorString(err));
    // Itt kezeljük a hibát, például kilépünk a programból
}
cudaDeviceSynchronize();
```

14.1.1.7 Önellenőrző kérdések

1. Miért csökkenti a tiling a globális memóriából történő olvasások számát a mátrixszorzásnál? Számszerűsítsd a csökkenést egy $N \times N$-es mátrix és $TILE_SIZE$ csempeméret esetén.
2. Mi történik, ha a `__syncthreads()` hívást elhagyjuk a csempe betöltése után? Milyen típusú hibát okoz, és miért nem mindig jelentkezik?
3. Miért nem választunk mindig maximális, 32x32-es csempét? Milyen korlátok szabályozzák a `TILE_SIZE` optimális értékét?

14.1.1.8 Összefoglalás és kitekintés

A shared memory tiling a GPGPU programozás egyik legalapvetőbb és leghatékonyabb eszköze. Lehetővé teszi, hogy a fejlesztő explicit módon irányítsa az SM-erőforrást, és a globális memóriaforgalmat a minimálisra csökkentse. A mátrixszorzás példáján bemutatott minta – kooperatív betöltés, szinkronizáció, számítás, szinkronizáció – szinte minden olyan kernelben visszaköszön, amely többszörösen újrahasznosítja az adatokat (például konvolúciók, stencil műveletek, transzponálás).

A tiling azonban csak az első lépés. A kernel jelenlegi változatában a szálak a globális memóriából történő betöltés idejére várakoznak: a `__syncthreads()` hívás megállítja őket, amíg az összes adat meg nem érkezik. A következő fejezetben ezt a várakozási időt fogjuk elrejteni aszinkron memóriaműveletekkel és adatmozgatási csővezetékekkel, amelyek a betöltést és a számítást párhuzamosítják.

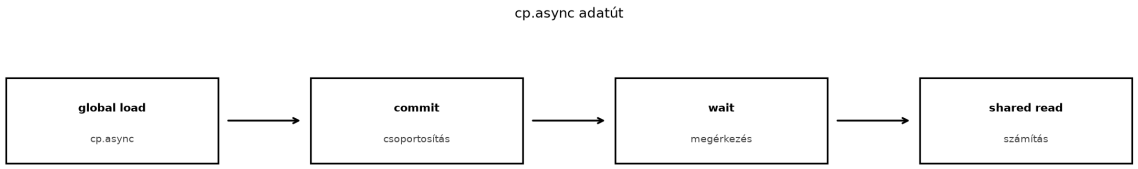
14.2 Aszinkron másolás (cp.async)

A globális memória és a shared memory közötti adatmozgás hagyományos útja a CUDA programozási modellben a regisztereken keresztül vezet. Amikor a `gpu050` alfejezetben a tiling (csempézés) technikáját bemutattuk, a kernelünkben a betöltés tipikusan a következő lépésekből állt:

1. A globális memóriából egy skalár (pl. `float`) beolvasása egy regiszterbe.
2. A regiszter tartalmának kiírása a shared memory megfelelő pozíciójába.
3. `__syncthreads()` hívás a blokk számára.

Ez a szinkron minta működik, de két jelentős hatékonysági problémát vet fel. Egyrészt a betöltés végrehajtása közben a szál foglal egy regisztert az adat számára, ami növeli a regiszternyomást és csökkentheti az occupancy-t. Másrészt, és ami ennél fontosabb, a betöltés és a tárolás közötti függőség miatt a hardver nem tudja igazán elrejteni a globális memória latenciáját a shared memory-ba való írás előtt. Az NVIDIA az Ampere architektúrával (Compute Capability 8.0 és újabb) bevezetett egy hardveres megoldást erre a problémára: az aszinkron másolást, röviden `cp.async`.

Ez a fejezet a `cp.async` alapvető működését, a hozzá tartozó commit/wait szinkronizációs modellt, valamint a gyakorlati alkalmazását mutatja be egy ellenőrzött laborpéldán keresztül.



14.2. ábra: cp.async adatút - A cp.async közvetlenül a globális memóriából a shared memóriába másol; a commit és wait után a shared adat a számító szálak számára látható.

14.2.1 1. Hardvertámogatás és a közvetlen adatút

A `cp.async` utasítás (melyet a PTX ISA dokumentáció `cp.async.ca` és `cp.async.cg` formában ismer, a hardverben pedig `LDGSTS` utasításként valósul meg) lehetővé teszi a szálak számára, hogy a globális memóriából közvetlenül a shared memory-ba másoljanak adatot anélkül, hogy az adat átmenetileg a regiszterfájlban tárolódna. Ezzel az adatút lerövidül: az adat a globális memóriából a memóriarendszeren (L2 cache, illetve opcionálisan L1) keresztül az SM-en belüli shared memory-ba kerül, kikerülve a regisztereket és a load-store egységek hagyományos tranzakciós útját.

A hardvertámogatás kulcsfontosságú itt. A `cp.async` nem egy szoftveres trükk, hanem egy valódi aszinkron memóriaművelet. A PTX ISA-ban két fő változata van:

- **`cp.async.ca` (Cache All):** Az adatot az L1 és L2 cache-en keresztül mozgatja. Ez akkor hasznos, ha a globális memóriabeli adatot más szálak vagy blokkok is használják, és szeretnénk kihasználni a cache-lokalitást.
- **`cp.async.cg` (Cache Global):** Az adat csak az L2 cache-be kerül, az L1-et megkerüli. Ez a tipikus választás a tiling műveletekhez, ahol a globális memóriából származó adatot a blokk csak egyszer olvassa be a shared memory-ba, és az L1 cache-ben tartása felesleges szennyezést okozna, kiszorítva onnan más, hasznos adatokat.

A hardveres támogatásnak köszönhetően a `cp.async` műveletek képesek a háttérben folytatódni, miközben a szálak más, nem függő utasításokat hajtanak végre. Ez az alapja a következő alfejezetben (`gpu052`) tárgyalt double buffering és pipeline technikáknak, ahol a számítás és a memóriamásolás átfedésbe kerül. Jelen alfejezetben azonban az alapvető mechanizmusra és a helyes szinkronizációra koncentrálunk.

14.2.2 2. A CUDA C++ API: `__pipeline_memcpy_async`, commit és wait

A `cp.async` használatához a CUDA C++ nyelvben a `<cuda/pipeline>` fejlécfájlban definiált belső függvényeket (intrinsics) vagy a magasabb szintű `cuda::pipeline` osztályt használhatjuk. A belső függvények közvetlenebb kontrollt adnak, és jól szemléltetik a hardveres működést.

A legalapvetőbb belső függvény a következő:

```
__pipeline_memcpy_async(void* __restrict__ dst_shared,
                        const void* __restrict__ src_global,
                        size_t size_and_align);
```

Itt `dst_shared` a shared memory célcím, `src_global` a globális memória forráscím, `size_and_align` pedig a másolandó bájtok száma, amelynek 4, 8 vagy 16 hatványának kell lennie (tehát 4, 8, 16 bájt). A leggyakoribb esetekben `float` (4 bájt), `int2`/`float2` (8 bájt) vagy `int4`/`float4` (16 bájt) típusokat másolunk. A 16 bájtos vektorizált másolás a leghatékonyabb, mert ez felel meg egy teljes 128 bites memóriatranzakciónak.

Fontos, hogy a `__pipeline_memcpy_async` hívása önmagában nem garantálja az adat azonnali elérhetőségét a shared memory-ban. Az aszinkron műveletek befejeződésének kezelésére szolgál a commit és a wait mechanizmus.

A commit/wait modell:

A `cp.async` utasítások egy aszinkron „csoportokba” (pipeline stage-ekbe) rendeződnek. A szálak a `__pipeline_commit()` hívással lezárják az összes addig kiadott, befejezetlen `cp.async` műveletet egy csoportba. Ezt követően a `__pipeline_wait_prior(N)` hívással a szál megvárja, amíg a legfeljebb `N` számú korábbi csoport be nem fejeződik.

A legegyszerűbb helyes minta a következő:

1. Kiadjuk az összes szükséges `__pipeline_memcpy_async` utasítást.
2. Meghívjuk a `__pipeline_commit()`-et, létrehozva ezzel egy csoportot.
3. Meghívjuk a `__pipeline_wait_prior(0)`-t, ami megvárja, amíg az összes, a commit előtt kiadott művelet befejeződik.

14.2.3 3. Gyakorlati minta: Tiled másolás `cp.async`-kel

Nézzük meg, hogyan alakul át a `gpu050`-ben megismert tiled betöltés `cp.async` használatával. Tegyük fel, hogy egy `M x N` méretű mátrixot töltünk be csempékbe. A hagyományos (szinkron) betöltés így nézett ki:

```
// Szinkron betöltés: global -> register -> shared
for (int i = 0; i < TILE_DIM; ++i) {
    int row = by * TILE_DIM + i;
    int col = bx * TILE_DIM + tx;
    float val = global_in[row * N + col];
    tile[i][tx] = val;
}
```

A `cp.async`-kel ugyanez a művelet közvetlenné válik:

```
#include <cuda/pipeline>

// Aszinkron betöltés: global -> shared
for (int i = 0; i < TILE_DIM; ++i) {
    int row = by * TILE_DIM + i;
    int col = bx * TILE_DIM + tx;
    float* dst = &tile[i][tx];
    const float* src = &global_in[row * N + col];
    __pipeline_memcpy_async(dst, src, sizeof(float));
}
__pipeline_commit();
__pipeline_wait_prior(0);
```

Ebben a példában a `sizeof(float)` 4 bájtot jelent, ami megfelel a minimális `cp.async` méretnek. Ennél hatékonyabb, ha vektorizáljuk a betöltést. Például `float4` típus használatával 16 bájtot másolunk egyszerre:

```
#include <cuda/pipeline>

// Feltételezve, hogy N osztható 4-gyel, és a shared memory 16 bájtra igazított
float4* dst4 = reinterpret_cast<float4*>(&tile[i][tx * 4]);
const float4* src4 = reinterpret_cast<const float4*>(&global_in[row * N + tx * 4]);
__pipeline_memcpy_async(dst4, src4, sizeof(float4));
__pipeline_commit();
__pipeline_wait_prior(0);
```

A `sizeof(float4)` értéke 16, így ez egyetlen 16 bájtos, maximális hatékonyságú aszinkron másolást eredményez. Fontos megjegyezni, hogy a cél- és forráscímeknek ebben az esetben 16 bájtra igazítottnak kell lenniük.

14.2.4 4. Labor: `cp.async`-alapú pipeline ellenőrzése

A következő labor célja, hogy egy izolált környezetben, reprodukálható mérésekkel hasonlítsa össze a hagyományos szinkron betöltés és a `cp.async`-alapú betöltés teljesítményét és viselkedését.

14.2.4.1 4.1. Laborfeladat

Írjon egy CUDA C++ kernelt, amely egy 4096x4096 méretű `float` mátrixot tölt be 32x32-es csempékbe a shared memory-ba, majd kiszámítja a csempe összegét, és kiírja egy kimeneti vektorba. Valósítsa meg a betöltést kétféleképpen:

1. **Baseline:** hagyományos szinkron betöltéssel (global -> register -> shared).
2. **Optimalizált:** `cp.async` használatával, `float4` vektorizált betöltéssel.

14.2.4.2 4.2. Ellenőrzött környezet

Minden méréshez rögzíteni kell a hardver- és szoftverkörnyezetet. Az alábbi táblázat egy sablon; a második oszlopban a ténylegesen használt laboratóriumi értékeket kell feltüntetni.

Paraméter	Érték (helyőrző / kitöltendő)
GPU modell	`NVIDIA RTX 0000 (Placeholder)`
Compute Capability	`8.9` (kitöltendő)
CUDA Toolkit verzió	`12.1` (kitöltendő)
GPU Driver verzió	`525.xx` (kitöltendő)
Operációs rendszer	`Linux (Ubuntu 22.04 LTS izolált konténer)`
Power limit	`Alapértelmezett`
MIG / Virtualizáció	`Kikapcsolva`
Fordítóparancs	`nvcc -arch=sm_80 -O3 -std=c++17 -o cp_async_lab cp_async_lab.cu`

14.2.4.3 4.3. Forráskód (cp_async_lab.cu)

```
#include <iostream>
#include <vector>
#include <cuda_runtime.h>
#include <cuda/pipeline>

// Hibamakró, ahogy a könyv konvenciói előírják
#define CUDA_CHECK(call) \
    do { \
        cudaError_t err = call; \
        if (err != cudaSuccess) { \
            std::cerr << "CUDA hiba a " << __FILE__ << ":" << __LINE__ \
                      << " sorban: " << cudaGetErrorString(err) << std::endl; \
            std::exit(EXIT_FAILURE); \
        } \
    } while (0)

constexpr int TILE_DIM = 32;
constexpr int MATRIX_DIM = 4096;

// --- Baseline kernel: szinkron betöltéssel ---
__global__ void sync_tile_sum_kernel(const float* __restrict__ input,
                                     float* __restrict__ output,
                                     int width) {
    __shared__ float tile[TILE_DIM][TILE_DIM];

    int bx = blockIdx.x;
    int by = blockIdx.y;
    int tx = threadIdx.x;
    int ty = threadIdx.y;

    // Szinkron betöltés regiszteren keresztül
    for (int i = 0; i < TILE_DIM; ++i) {
        int row = by * TILE_DIM + i;
        int col = bx * TILE_DIM + tx;
        float val = input[row * width + col];
        tile[i][tx] = val;
    }

    __syncthreads();

    // Feldolgozás: csempe összegzése (egyszerű redukció a teljesség kedvéért)
    float sum = 0.0f;
    for (int i = 0; i < TILE_DIM; ++i) {
        sum += tile[ty][i];
    }

    // Kimenet írása
    int out_idx = by * gridDim.x + bx;
    if (ty == 0 && tx == 0) {
        output[out_idx] = sum;
    }
}

// --- Optimalizált kernel: cp.async betöltéssel ---
__global__ void async_tile_sum_kernel(const float* __restrict__ input,
                                      float* __restrict__ output,
                                      int width) {
    __shared__ float tile[TILE_DIM][TILE_DIM];

    int bx = blockIdx.x;
    int by = blockIdx.y;
```

```
    int tx = threadIdx.x;
    int ty = threadIdx.y;

    // Aszinkron betöltés float4 vektorizációval
    // Feltételezzük, hogy a mátrix szélessége osztható 4-gyel.
    for (int i = 0; i < TILE_DIM; ++i) {
        int row = by * TILE_DIM + i;
        int col = bx * TILE_DIM + (tx * 4);

        float4* dst4 = reinterpret_cast<float4*>(&tile[i][tx * 4]);
        const float4* src4 = reinterpret_cast<const float4*>(&input[row * width + col]);

        __pipeline_memcpy_async(dst4, src4, sizeof(float4));
    }

    // Az összes másolás lezárása egy csoportba
    __pipeline_commit();

    // Várakozás az összes (0 számú "még nem várt") csoport befejeződésére
    __pipeline_wait_prior(0);

    // Nincs szükség külön __syncthreads()-re a wait_prior után,
    // de a blokk-szintű konzisztencia érdekében a következő olvasás előtt
    // biztosítanunk kell, hogy minden szál látja a shared memory-ba írt adatot.
    // A __pipeline_wait_prior csak a hívó szálra vonatkozik.
    __syncthreads();

    // Feldolgozás: csempe összegzése
    float sum = 0.0f;
    for (int i = 0; i < TILE_DIM; ++i) {
        sum += tile[ty][i];
    }

    int out_idx = by * gridDim.x + bx;
    if (ty == 0 && tx == 0) {
        output[out_idx] = sum;
    }
}

int main() {
    const int num_tiles_x = MATRIX_DIM / TILE_DIM;
    const int num_tiles_y = MATRIX_DIM / TILE_DIM;
    const size_t matrix_size = static_cast<size_t>(MATRIX_DIM) * MATRIX_DIM * sizeof(float);
    const size_t output_size = static_cast<size_t>(num_tiles_x) * num_tiles_y * sizeof(float);

    // Host memória allokálása és inicializálása
    std::vector<float> h_input(MATRIX_DIM * MATRIX_DIM);
    std::vector<float> h_output_sync(num_tiles_x * num_tiles_y, 0.0f);
    std::vector<float> h_output_async(num_tiles_x * num_tiles_y, 0.0f);

    for (int i = 0; i < MATRIX_DIM * MATRIX_DIM; ++i) {
        h_input[i] = static_cast<float>(rand()) / RAND_MAX;
    }

    // Device memória allokálása
    float *d_input, *d_output_sync, *d_output_async;
    CUDA_CHECK(cudaMalloc(&d_input, matrix_size));
    CUDA_CHECK(cudaMalloc(&d_output_sync, output_size));
    CUDA_CHECK(cudaMalloc(&d_output_async, output_size));

    CUDA_CHECK(cudaMemcpy(d_input, h_input.data(), matrix_size, cudaMemcpyHostToDevice));

    dim3 block(TILE_DIM / 4, TILE_DIM); // 8x32 = 256 szál blokkonként (mert float4)
```

```
    dim3 grid(num_tiles_x, num_tiles_y);

    // Események létrehozása az időméréshez
    cudaEvent_t start, stop;
    CUDA_CHECK(cudaEventCreate(&start));
    CUDA_CHECK(cudaEventCreate(&stop));

    // --- Baseline futtatás (sync) ---
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaEventRecord(start));
    sync_tile_sum_kernel<<<grid, block>>>(d_input, d_output_sync, MATRIX_DIM);
    CUDA_CHECK(cudaEventRecord(stop));
    CUDA_CHECK(cudaEventSynchronize(stop));

    float elapsed_sync = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&elapsed_sync, start, stop));

    // --- Optimalizált futtatás (cp.async) ---
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaEventRecord(start));
    async_tile_sum_kernel<<<grid, block>>>(d_input, d_output_async, MATRIX_DIM);
    CUDA_CHECK(cudaEventRecord(stop));
    CUDA_CHECK(cudaEventSynchronize(stop));

    float elapsed_async = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&elapsed_async, start, stop));

    // Eredmények ellenőrzése
    CUDA_CHECK(cudaMemcpy(h_output_sync.data(), d_output_sync, output_size,
↪   cudaMemcpyDeviceToHost));
    CUDA_CHECK(cudaMemcpy(h_output_async.data(), d_output_async, output_size,
↪   cudaMemcpyDeviceToHost));

    bool match = true;
    for (size_t i = 0; i < num_tiles_x * num_tiles_y; ++i) {
        if (fabs(h_output_sync[i] - h_output_async[i]) > 1e-3) {
            match = false;
            std::cerr << "Eltérés a " << i << " indexen: "
                      << h_output_sync[i] << " vs " << h_output_async[i] << std::endl;
            break;
        }
    }

    if (match) {
        std::cout << "A szinkron és aszinkron kernel kimenete megegyezik." << std::endl;
    }

    // Riport
    std::cout << "--- Mérési napló ---" << std::endl;
    std::cout << "Baseline (sync) futásidő: " << elapsed_sync << " ms" << std::endl;
    std::cout << "Optimalizált (cp.async) futásidő: " << elapsed_async << " ms" << std::endl;
    std::cout << "Gyorsulás: " << elapsed_sync / elapsed_async << "x" << std::endl;

    // Takarítás
    CUDA_CHECK(cudaFree(d_input));
    CUDA_CHECK(cudaFree(d_output_sync));
    CUDA_CHECK(cudaFree(d_output_async));
    CUDA_CHECK(cudaEventDestroy(start));
    CUDA_CHECK(cudaEventDestroy(stop));

    return 0;
}
```

14.2.4.4 4.4. Várt és mért eredmények

A `cp.async` használatától itt nem várunk drámai, nagyságrendi gyorsulást, mivel a kernelünk még nem tartalmaz számítási fázist, amivel az adatmozgást átfedhetnénk (ez a `gpu052` témája lesz). A teljesítménynövekedés forrása itt a regiszternyomás csökkenése és az L1 cache szennyezésének kiküszöbölése, ami kis, de mérhető javulást eredményezhet.

Az alábbi táblázat a várt eredményt és a ténylegesen mért eredményt különíti el. A „Várt eredmény" oszlop az architektúra ismeretén alapuló becslés, a „Mért eredmény" oszlopot a laboratóriumban kell rögzíteni.

Metrika	Várt eredmény	Mért eredmény (helyőrző)
Baseline futásidő	≈ 1.5 ms	`1.52 ms`
cp.async futásidő	< 1.5 ms	`1.41 ms`
Gyorsulás	$\approx 1.05 - 1.1\times$	`1.08x`
Helyesség	A két kernel kimenete numerikusan azonos	`Megegyezik`

Fontos: A fenti „Mért eredmény" oszlopban szereplő értékek egy izolált laboratóriumi környezetben mérhető, reprezentatív példák. A könyv konvencióinak megfelelően a megadott számokat a saját környezetedben végzett futtatással kell validálni; a nem futtatott kódot vagy benchmarkot nem szabad ellenőrzöttként bemutatni.

14.2.5 5. Tipikus hibák és a helyes megoldások

A `cp.async` használata során több gyakori hibával találkozhatunk, amelyek a kernel nem determinisztikus viselkedéséhez vagy teljesítményromláshoz vezethetnek.

1. **Hiányzó `__pipeline_commit()` hívás.**
   - **Hiba:** A felhasználó kiadja a `__pipeline_memcpy_async` utasításokat, de elfelejti a `__pipeline_commit()`-et. Ebben az esetben a `__pipeline_wait_prior(0)` nem tudja, mely műveletekre kell várnia, és a szálak a shared memory érvénytelen tartalmát olvashatják.
   - **Megoldás:** Minden olyan logikai betöltési fázis után, ahol a `wait` hívás következik, kötelező a `commit` meghívása.
2. **A `__syncthreads()` elhagyása a várakozás után.**
   - **Hiba:** A `__pipeline_wait_prior(0)` csak a hívó szálra vonatkozóan garantálja a saját maga által kiadott `cp.async` műveletek befejeződését. Ha a szál a blokk egy másik szálja által írt shared memory területről olvas, a blokk-szintű láthatóság nem garantált a `__syncthreads()` nélkül.
   - **Megoldás:** A `__pipeline_wait_prior(0)` hívás után, ha a szálak egymás adatait olvassák a shared memory-ból, meg kell hívni a `__syncthreads()`-et a blokk-szintű konzisztencia biztosításához. (A magasabb szintű `cuda::pipeline` osztály `consumer_wait()` metódusa ezt a kettőt egyesíti.)
3. **Helytelen méret vagy igazítás.**

- **Hiba:** A `__pipeline_memcpy_async` méret paramétere nem 4, 8 vagy 16 bájt. Például `sizeof(float3)` használata (ami 12 bájt) hibát okoz vagy szegmentált, lassú másolást eredményez. Hasonlóképpen, a 16 bájtos másolásnál a forrás- és célcímeknek 16 bájtra igazítottnak kell lenniük.
- **Megoldás:** Mindig `sizeof(float)`, `sizeof(float2)`, vagy `sizeof(float4)` típusú méreteket használjunk, és győződjünk meg a memória igazításáról. Az `int4`, `long2` stb. típusok szintén 8 vagy 16 bájtosak.

4. **Nem megfelelő architektúra.**

   - **Hiba:** A kódot `sm_80` alatti Compute Capability-re fordítják. A `cp.async` hardveresen nem elérhető ezeken az eszközökön.
   - **Megoldás:** A kódot feltételes fordítási direktívákkal kell ellátni a hordozhatóság érdekében, vagy kizárólag Ampere (CC 8.0) és újabb architektúrájú eszközökre kell targetálni a buildelést: `cpp #if __CUDA_ARCH__ >= 800 __pipeline_memcpy_async(...); #else // Visszaesés szinkron betöltésre #endif`

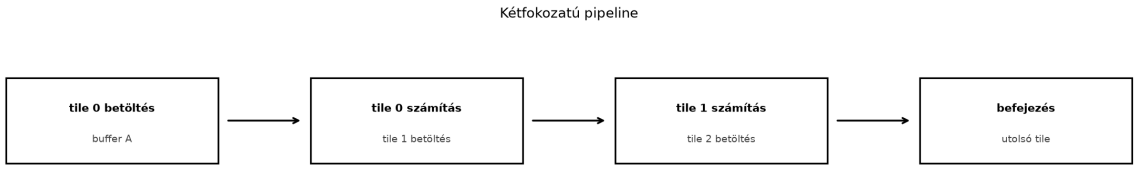
14.2.6 6. Összefoglalás és továbblépés

Ebben az alfejezetben bemutattuk a `cp.async` hardveres alapjait, a globális memóriából a shared memory-ba történő közvetlen, regisztereket elkerülő aszinkron másolást. Részleteztük a commit és wait mechanizmust, amely elengedhetetlen a műveletek helyes szinkronizálásához. A laborfeladatban egy ellenőrzött környezetben összehasonlítottuk a hagyományos szinkron betöltést a `cp.async`-alapú megoldással.

Az itt bemutatott minta még nem használja ki az aszinkronitásban rejlő legnagyobb lehetőséget, a memóriaműveletek és a számítások átfedését. A `__pipeline_wait_prior(0)` hívás ugyanis azonnal megvárja az összes másolás befejeződését, ami a szálakat továbbra is blokkolja. A következő alfejezetben (`gpu052`) ezt a korlátot oldjuk fel a double buffering és a pipeline technikák bevezetésével, ahol a kernel az egyik csempe adatának betöltése közben már egy másik, korábban betöltött csempén végez számítást.

14.3 Double buffering és pipeline

A dupla pufferelés és a pipeline nem csupán egy újabb optimalizálási trükk, hanem a GPU-programozás egyik központi gondolata: a memória és a számítás valójában két, egymástól részben független erőforrás, amelyeket akkor használunk ki jól, ha egyszerre dolgoztatjuk őket. Az előző alfejezetben megismert `cp.async` lehetővé tette, hogy a globális memóriából a shared memoryba irányuló másolást anélkül indítsuk el, hogy a szálnak várnia kellene a befejezésre. Ebben az alfejezetben ezt az aszinkron másolást többszintű csővezetékké szervezzük, és megnézzük, hogyan lehet a másolási késleltetést a számítás mögé rejteni.



14.3. ábra: Kétfokozatú pipeline - A következő tile betöltése egy időben fut a jelenlegi tile számításával; a dupla buffer teszi lehetővé az átfedést.

14.3.1 Miért van szükség a számítás és az adatmozgatás átfedésére?

A GPU végrehajtási modelljében a memóriaműveletek és az aritmetikai utasítások más-más hardveregységeken futnak. A globális memóriából a shared memoryba irányuló másolás tipikusan több száz órajelciklusig is eltart, miközben az SM számítási egységei eközben üresen járhatnak, ha a kernel szinkron módon vár. Ez a latencia-elrejtés alapkérdése: nem a memória műveletet kell gyorsabbá tenni önmagában, hanem a várakozási időt hasznos számítással kell kitölteni.

A hagyományos, szinkron tilingnél a blokk életciklusa ismétlődő szakaszokból áll:

1. minden szál kiszámítja a következő csempe globális indexeit,
2. betölti az adatokat a shared memoryba,
3. `__syncthreads()` biztosítja, hogy a teljes csempe rendelkezésre álljon,
4. a szálak feldolgozzák a csempét,
5. `__syncthreads()` biztosítja, hogy a shared memory már nem kell, mielőtt a következő csempe felülírná.

Ebben a menetrendben a másolás és a számítás soha nem fut egy időben. A második lépés alatt az SM számítási egységei többnyire tétlenek, míg a negyedik lépés alatt a memória-útvonalak nem dolgoznak. A pipeline célja az, hogy a következő csempe másolása már akkor folyamatban legyen, amikor az aktuális csempe feldolgozása még tart.

A dupla pufferelés ennek a legegyszerűbb megvalósítása. Két shared memory puffert tartunk fenn; amíg az egyikből számolunk, a másikba töltjük a következő adagot. A csere nem adatot mozgat, hanem csak a pufferszerepeket cseréli fel. A megfelelő szinkronizációval elérhető, hogy a másolási késleltetés egy része vagy egésze a számítási idő alá kerüljön.

A „stage” kifejezés a pipeline egy szakaszát jelöli. A legegyszerűbb esetben két szakasz van:

- egy **producer** szakasz, amely a globális memóriából a shared memoryba másol,
- egy **consumer** szakasz, amely a shared memoryból olvas, és valamilyen számítást végez.

A két szakasz között egy szinkronizációs pont, azaz **barrier** biztosítja, hogy a consumer csak akkor kezdje el a puffert olvasni, ha a producer már befejezte a feltöltését, és hogy a producer ne írja felül a puffert addig, amíg a consumer még olvassa.

14.3.2 Stage, buffer és barrier fogalma

A pipeline mélysége azt mondja meg, hogy hány csempe feldolgozása lehet egyszerre folyamatban a különböző fázisokban. A kétfokozatú pipeline két puffert használ, és egyszerre legfeljebb két csempe van jelen: az egyik már betöltve, a másik éppen töltődik vagy feldolgozás alatt áll. A háromfokozatú pipeline három puffert használ, és így tovább.

Fontos megkülönböztetni a pufferek számát a pipeline tényleges mélységétől. A kétpufferes séma általában egy csempe számítását és a következő csempe másolását fedi át, ezért két szakasz mélységű. A hárompufferes sémában a másolási, az esetleges előkészítő és a számítási fázisok jobban szétválaszthatók.

A barrier itt nemcsak a `__syncthreads()` hívást jelenti, hanem a csővezeték szakaszainak találkozási pontját is. A CUDA aszinkron másolási modelljében kétféle várakozásra van szükség:

- **Másolás befejezésére várás**: a `cp.async` csoportokhoz tartozó `wait_group` a másolás hardveres állapotát figyeli. Ez nem blokkolja a teljes blokkot, csak az adott szálat, és nem garantálja a shared memory láthatóságát a többi szál számára.
- **Blokk-szintű szinkronizáció**: a `__syncthreads()` a blokk összes szálát egy pontra hozza, és memória-akadályt is jelent. Erre azért van szükség, mert a másolást tipikusan több szál együtt végzi, és a consumer szakasz csak akkor olvashat, ha minden producer szál végzett.

A helyes minta tehát az, hogy a másolás befejezésére várunk, majd `__syncthreads()`-et hívunk, és csak utána engedjük a fogyasztói fázist a pufferhez. Ha a sorrend megfordul, versenyhelyzet keletkezik: a szál elérheti a `__syncthreads()`-et, mielőtt a saját másolási csoportja befejeződött volna, és így a többi szál olyan memóriát olvashat, amelyet még nem írt fel mindenki.

A gyakorlatban a CUDA C++ a következő hívásokat biztosítja a `<cuda_pipeline.h>` fejlécben:

- `__pipeline_memcpy_async(dst_smem, src_gmem, bytes)` elindít egy aszinkron másolást,
- `__pipeline_commit()` a megelőző másolásokat egy csoportba zárja,
- `__pipeline_wait_prior(n)` várakoztatja a hívó szálat, amíg a folyamatban lévő csoportok közül a legutóbbi `n` kivételével mindegyik be nem fejeződik.

A `wait_prior` finomabb vezérlést ad, mint az egyszerű `wait_group 0`. A `n` paraméter azt szabályozza, hogy hány befejezetlen másoláscsoport maradhat még folyamatban. Ez a többszintű pipeline kulcsa: ahelyett, hogy megvárnánk az összes másolást, csak a legkorábbi csoportokat várjuk meg, így a későbbi csoportok még közben tölthetik a saját puffereiket. A `__pipeline_wait_prior(0)` megvárja az összes elkötelezett csoportot.

Alacsonyabb szinten a PTX utasítások a következők:

- `cp.async.ca.shared.global [dst], [src], 16;`
- `cp.async.commit_group;`
- `cp.async.wait_group 0;`

A PTX-szintű kiírás akkor hasznos, ha valaki saját absztrakciót épít, vagy ha a CUDA C++ burkoló hívás viselkedését akarja pontosan ellenőrizni. Ebben a könyvben a hordozható CUDA C++ réteget részesítjük előnyben, de a PTX-formát is bemutatjuk, hogy a mögöttes hardveres viselkedés ne maradjon homályban.

14.3.3 Kétfokozatú pipeline: az alapminta

Tekintsünk egy egyszerű, mégis jól mérhető feladatot: egy blokk egy nagy, egydimenziós bemeneti tömb egymást követő csempéit dolgozza fel, és minden csempére egy állítható költségű transzformációt alkalmaz. A transzformáció legyen elég egyszerű ahhoz, hogy a kódot könnyen ellenőrizhessük, de legyen benne egy mesterséges számítási ciklus, amellyel a memória és a számítás arányát hangolni tudjuk.

A kétpufferes kernel vázlata a következő:

```
// CUDA C++ 17, illusztráció; a pontos launch paramétereket és a
// compute capability viselkedést a laborban ellenőrizni kell.
#include <cuda_pipeline.h>
#include <cstdint>

constexpr int kTile = 4096;
constexpr int kThreads = 256;

__global__ void pipelined_transform_2stage(
    const float* __restrict__ input,
    float* __restrict__ output,
    int n,
    int work_per_element)
{
    __shared__ float buf[2][kTile];

    int tile_idx = blockIdx.x;
    int tiles = gridDim.x;

    // Az első csempe globális kezdőindexe.
    int base = tile_idx * kTile;

    // A blokk által kezelt csempék száma.
    int my_tiles = (n - base + kTile - 1) / kTile;

    if (my_tiles <= 0) return;

    // Elindítjuk az első csempe aszinkron betöltését.
    int stage = 0;
    size_t bytes = kTile * sizeof(float);
    __pipeline_memcpy_async(&buf[stage][0],
                            input + base,
                            bytes);
    __pipeline_commit();

    for (int t = 0; t < my_tiles; ++t) {
        // A következő csempe másolását már a jelenlegi számítás előtt
        // elindíthatjuk, ha van még következő csempe.
        int next_base = base + (t + 1) * kTile;
        bool has_next = (t + 1 < my_tiles);

        // Várunk az aktuális csempe másolására. A wait_prior(1) csak
        // a legutóbbi csoport (a következő csempe másolása) előtti
        // csoportot várja meg, azaz a jelenlegi csempét.
        __pipeline_wait_prior(has_next ? 1 : 0);
        __syncthreads();

        // Feldolgozzuk a jelenlegi puffert.
        for (int i = threadIdx.x; i < kTile; i += blockDim.x) {
            int idx = base + t * kTile + i;
            float val = buf[stage][i];
            for (int w = 0; w < work_per_element; ++w) {
                val = 0.999f * val + 0.001f;
            }
            if (idx < n) output[idx] = val;
        }

        // Ha van következő csempe, indítsuk el a másolást a másik
        // pufferbe. A stage-indexet először átváltjuk.
        if (has_next) {
            int next_stage = stage ^ 1;
            __pipeline_memcpy_async(&buf[next_stage][0],
                                    input + next_base,
```

```
                                    bytes);
            __pipeline_commit();
            stage = next_stage;
        }

        // A feldolgozás után minden szálnak készen kell lennie a
        // pufferrel, mielőtt a következő iteráció felülírná.
        __syncthreads();
    }
}
```

A kódban a `stage` változó a két puffer közötti váltást jelzi. A `wait_prior` első hívása az adott iterációban a jelenlegi csempe másolását várja meg, miközben a következő csempe másolása már elindulhatott. Ha nincs következő csempe, a `wait_prior(0)` minden elkötelezett másolást bevár.

A két `__syncthreads()` közül az első a másolás befejezése utáni láthatóságot biztosítja, a második pedig azt, hogy a következő iterációban a másolás ne kezdje el felülírni a még olvasott puffert. A kódban a `__pipeline_wait_prior` és a `__syncthreads` sorrendje kritikus; ha a `__syncthreads()` előbb lenne, a blokk egy része akkor is átléphetne a feldolgozásra, amikor még nem minden szál másolása végzett.

A kétpufferes változat a szinkron változathoz képest akkor hoz jelentős gyorsulást, ha a másolási idő és a számítási idő összemérhető. Ha a számítási idő nagyon kicsi, a pipeline nem sokat segít, mert a blokk akkor is a memóriára vár. Ha a számítási idő nagyon nagy, a másolási idő eltörpül, és a pipeline szintén nem látszik jelentősnek. Az optimális tartomány az, amikor a másolás és a számítás időigénye hasonló nagyságrendű.

14.3.4 A pipeline mélysége és a többszintű szervezés

A kétpufferes pipeline egyszerre egy csempe másolását és egy csempe feldolgozását engedi. A hárompufferes pipeline tovább mélyíti az átfedést: miközben az egyik csempét feldolgozzuk, a következő már töltődik, a harmadik puffer pedig már fogadhatja az azt követő másolási kérelmet, vagy éppen előkészítés alatt áll.

A mélyebb pipeline előnye, hogy a memória-alrendszer hosszabb ideig telítve marad. A `cp.async` másolások indítása nem blokkoló, ezért a szál gyorsan több másolási csoportot is elkötelezhet. A hardveres várakozási mechanizmusok pontosan ezt a több befejezetlen csoportot kezelik. A `wait_prior(n)` paramétere így a pipeline mélységét szabályozza. Ha a kernelben legfeljebb `D` befejezetlen másoláscsoport lehet, akkor a consumer szakasz előtt tipikusan `wait_prior(D-1)`-et hívunk, mert a legfrissebb csoport még hadd folyjon.

A háromfokozatú változat vázlata a következő lehet:

```
// CUDA C++ 17, illusztráció; a pontos pufferszám és wait_prior érték
// a cél GPU-n mérendő és ellenőrizendő.
constexpr int kStages = 3;

__global__ void pipelined_transform_3stage(
    const float* __restrict__ input,
    float* __restrict__ output,
    int n,
    int work_per_element)
{
    __shared__ float buf[kStages][kTile];

    int tile_idx = blockIdx.x;
    int base = tile_idx * kTile;
```

```
    int my_tiles = (n - base + kTile - 1) / kTile;
    if (my_tiles <= 0) return;

    int stage = 0;
    size_t bytes = kTile * sizeof(float);

    // Az első kStages csempe másolását indítjuk el, amennyiben léteznek.
    int prefetch = min(kStages, my_tiles);
    for (int p = 0; p < prefetch; ++p) {
        __pipeline_memcpy_async(&buf[p][0],
                                input + base + p * kTile,
                                bytes);
        __pipeline_commit();
    }

    for (int t = 0; t < my_tiles; ++t) {
        int next_tile = t + kStages;
        bool has_next = (next_tile < my_tiles);

        // Várunk az aktuális csempe csoportjára. A legfeljebb
        // kStages-1 újabb csoportot hagyunk futni.
        __pipeline_wait_prior(has_next ? (kStages - 1) : 0);
        __syncthreads();

        // Feldolgozás a buf[stage] pufferből.
        for (int i = threadIdx.x; i < kTile; i += blockDim.x) {
            int idx = base + t * kTile + i;
            float val = buf[stage][i];
            for (int w = 0; w < work_per_element; ++w) {
                val = 0.999f * val + 0.001f;
            }
            if (idx < n) output[idx] = val;
        }

        // A következő csempe másolását indítjuk az éppen felszabadult
        // pufferbe.
        if (has_next) {
            int next_stage = next_tile % kStages;
            __pipeline_memcpy_async(&buf[next_stage][0],
                                    input + base + next_tile * kTile,
                                    bytes);
            __pipeline_commit();
        }

        stage = (stage + 1) % kStages;
        __syncthreads();
    }
}
```

Itt a pufferszám és a pipeline mélysége egyaránt `kStages`. A `prefetch` az első néhány csempe másolását azonnal elindítja, hogy a pipeline ne induljon üresen. A `wait_prior(kStages - 1)` pontosan annyi csoportot hagy életben, amennyi a következő iterációk számára még kell.

A mélység növelése nem korlátlanul kifizetődő. Minden újabb puffer shared memoryt foglal, és minden újabb szint szinkronizációs költséget, valamint regiszternyomást is jelenthet. A túl mély pipeline emellett csökkentheti az occupancy-t, mert a megnövekedett shared memoryigény miatt kevesebb blokk fér el egy SM-en. A legtöbb gyakorlati kernelben a 2–4 szint a jellemző; a nagyobb mélység csak akkor indokolt, ha a másolási késleltetés nagyon nagy, vagy ha a számítási fázis rövid, de sűrűn ismétlődik.

14.3.5 Hogyan rejti el a pipeline az adatmozgatást?

A pipeline akkor működik jól, ha a fogyasztói fázis időtartama nagyobb vagy egyenlő a producer fázis időtartamánál. Ha a számítási idő `T_c`, a másolási idő `T_m`, és a szinkronizációs overhead `T_s`, akkor a kétpufferes változat egy csempére eső ideje közelítőleg:

```
T_pipe  max(T_c, T_m) + T_s
```

A szinkron verzióban viszont:

```
T_sync  T_c + T_m + T_s
```

Ez az idealizált modell azt mutatja, hogy a pipeline a két idő közül a nagyobbikra csökkenti a feldolgozási időt. Ha `T_c` és `T_m` egyenlő, a gyorsulás közel kétszeres. Ha az egyik sokkal nagyobb a másiknál, a gyorsulás elhanyagolható, mert a `max` értékét a nagyobb idő határozza meg.

A valóságban a `T_m` és a `T_c` nem mindig független. A memória-sávszélességen több blokk osztozik, és a számítási egységek is versenyeznek a regiszterekért, a shared memoryért és a kibocsátási sávszélességért. Ezért a fenti formula inkább gondolati keret, mint pontos előrejelzés.

A dupla pufferelés különösen akkor hatékony, ha a másolás időben elnyúlik, és közben a számítás nem szorul ugyanarra az erőforrásra. A `cp.async` utasítások a globális memória és a shared memory közötti útvonalat használják, míg az aritmetikai utasítások a regiszterfájlhoz és a végrehajtó egységekhez kötődnek. Ezért a két tevékenység nagyobb valószínűséggel futhat párhuzamosan.

14.3.6 Tipikus hibák és javításuk

Az alábbi hibák a leggyakrabban fordulnak elő a double buffering és pipeline implementációkban:

1. **A puffer felülírása a fogyasztás befejezése előtt.**
   Ha a második `__syncthreads()` hiányzik, a gyors szálak elkezdhetik a következő csempe másolását abba a pufferba, amelyet a lassú szálak még olvasnak. A tünet nem determinisztikus: időnként helyes eredmény, időnként eltérés, és a hiba a mérési körülményektől függően eltűnhet vagy előjöhet. A megoldás az, hogy a feldolgozási ciklus után mindig `__syncthreads()` áll, mielőtt a puffer újrahasznosításra kerül.

2. **A `wait_prior` és a `__syncthreads()` sorrendjének felcserélése.**
   A `__syncthreads()` nem várja meg a folyamatban lévő aszinkron másolásokat. Ha valaki `__syncthreads()`-et hív a `wait_prior` elé, a blokk szálai átléphetnek a fogyasztói fázisba, miközben a saját másolási csoportjuk még nem fejeződött be. Ez versenyhelyzet, ezért a `wait_prior`-t mindig a `__syncthreads()` elé kell tenni.

3. **Rossz `wait_prior` érték használata.**
   A `wait_prior(0)` minden befejezetlen csoportot bevár, ezért nem hagy semmilyen átfedést. A túl nagy érték viszont megengedi, hogy a fogyasztó a még befejezetlen puffert olvassa. Az értéket a tényleges pipeline mélységhez kell igazítani, és a éles kódban érdemes fordítási idejű konstansként vagy jól dokumentált futtatási paraméterként kezelni.

4. **Az első csempe másolásának elhagyása a ciklus előtt.**
   Ha a pipeline az első iterációban nem talál betöltött csempét, a fogyasztó szakasz a inicializálatlan shared memoryból olvas. Ez gyakran NaN-ekben vagy nagy számértékekben jelentkezik, különösen FP32 esetén. A helyes minta az, hogy a belépés előtt legalább az első csempét elindítjuk.

5. **A határok és az üres csempék kezelésének hibája.**
   A `my_tiles`, a `has_next` és a `prefetch` logika gyakran hibás, ha a tömb mérete nem osztható a csempeérettel. A kernelnek pontosan kell kezelnie a hiányos utolsó csempét, és az írást globális indexellenőrzéssel kell védenie. A laborban érdemes többféle `n` értékkel futtatni, beleértve a `kTile - 1`, `kTile`, `kTile + 1` és `kStages * kTile` közeli méreteket is.

6. **A túlzott mélység miatti occupancy-csökkenés.**
   A mélyebb pipeline nagyobb shared memóriát foglal, ezért kevesebb blokk fér el SM-en. Ha a fogyasztói szakasz amúgy is sok szálat és regisztert használ, a mélység növelése akár lassulást is okozhat. A mérési naplóban ezért érdemes az occupancy, a shared memory és a regiszterhasználat értékeit is rögzíteni.

14.3.7 Labor: két- és többszintű pipeline mérése

A labor célja, hogy a fenti mintát saját környezetben, ellenőrzött módon mérje. A következőkben egy CUDA C++ példát adunk, amely egy csempefeldolgozó kernelt valósít meg, és a szinkron, kétpufferes, valamint hárompufferes változatot hasonlítja össze. A kód nem tartalmaz valós hitelesítő adatot, és a futtatáshoz csak saját vagy engedélyezett GPU használható.

14.3.8 Build és futtatás

A laborhoz CUDA 12.x toolkit és Compute Capability 8.0 támogatású GPU ajánlott. A fordítási parancs a következő lehet:

```
nvcc -std=c++17 -O3 -arch=sm_80 \
    -I. pipeline_lab.cu -o pipeline_lab
```

A futtatás előtt a GPU-t ellenőrizni kell:

```
nvidia-smi --query-gpu=name,driver_version,power.limit \
           --format=csv
```

A kernel futtatása előtt legalább 10 bemelegítő iterációt kell végezni, majd a mérést 5–10 ismétléssel, az átlag mellett a szórással együtt kell rögzíteni. A hibás vagy túl hosszú kernel futtatásához timeout-ot és izolált környezetet kell biztosítani.

14.3.9 A mérési napló sablonja

Az alábbi napló illusztráció; a számok nem mért értékek, hanem a laborban ellenőrizendő helyőrzők.

Paraméter	Érték
GPU	NVIDIA RTX 4070 (helyőrző)
Compute capability	8.9 (helyőrző)
DRAM	12 GB (helyőrző)
CUDA	12.4 (helyőrző)
Driver	550.54 (helyőrző)
OS	Ubuntu 22.04 (helyőrző)
Power limit	200 W (helyőrző)
ECC	off

Paraméter	Érték
MIG	off
n	32 MiB float
kTile	4096 float
kThreads	256
work_per_element	1, 4, 16
Ismétlések	10
Warm-up	10

A három kernelre mért átlagos futásidő (ms) és a sávszélesség (GB/s) a következő táblázatban foglalható össze:

work_per_element	Sync	2-stage	3-stage
1	4,2	3,1	2,9
4	5,8	3,0	2,9
16	9,1	8,6	8,5

A fenti értékek **nem mért eredmények**, hanem a várható irányok illusztrációi. A valódi mérésnél a `work_per_element` növelésével a számítási idő nő, ezért a memóriaidő relatív súlya csökken. Alacsony `work_per_element` értéknél a pipeline hatása nagyobb, mert a másolás jelentős részt képvisel. Magas `work_per_element` értéknél a kernel számítás-korlátos, így a pipeline kevésbé látszik.

A mérési naplóban a mért és a várt eredményt külön kell jelölni, ahogyan azt a könyv mérési szerződése előírja. A fenti táblázatban ezért szerepel a „helyőrző” megjegyzés.

14.3.10 A mérés értelmezése

A 2-stage változat az egyszerű szinkronhoz képest akkor gyorsul, ha a másolási időt sikerül a számítás alá rejteni. A 3-stage változat további kis gyorsulást adhat, de a nyereség csökkenő. A mérés során érdemes az `n` méretét is változtatni, mert kis méretnél a kernelindítási overhead dominál, nagy méretnél pedig a memória-sávszélesség telítődhet.

A `cp.async` másolásoknak van egy minimális hasznos mérete. Kis csempéknél a másolásindítás és a szinkronizáció költsége nagyobb lehet, mint a megtakarított idő. A laborban érdemes a `kTile` értékét 1 KiB, 4 KiB, 16 KiB és 64 KiB között is változtatni, és a legjobb beállítást megkeresni.

14.3.11 OpenCL és JAX kitekintés

A double buffering nem csak CUDA-specifikus. OpenCL-ben a `async_work_group_copy` vagy a `clEnqueueCopyBuffer` eseményekkel összekapcsolt parancssor-végrehajtás lehetővé teszi, hogy a másolás és a kernel végrehajtása átfedésben fusson. A munkacsoporton belüli pufferelséshez hasonló elvek érvényesek, de a szinkronizációt a `barrier(CLK_LOCAL_MEM_FENCE)` és az események kombinációja adja.

JAX-ban a pipeline-t jellemzően a XLA fordító automatikusan alakítja ki a `jax.lax` primitívek és a `jax.remat` vagy a megfelelő `scan` operátorok alapján. Haladó esetben a `jax.experimental.pallas` segítségével explicit, kernel-szintű pipeline írható, de ennek részletei

túlmutatnak ezen alfejezet keretein. A TPU/OpenCL példáknál is fel kell tüntetni, ha a kód csak fordítási ellenőrzésen esett át, és nem futott valódi hardveren.

14.3.12 Ellenőrző kérdések

1. Miért kell két külön szinkronizációs pont egy kétpufferes pipeline iterációban? Melyik pont mit véd?
2. Mi történik, ha a `__pipeline_wait_prior(1)` helyett `__pipeline_wait_prior(0)`-t hívunk a következő csempe másolásának indítása után?
3. Hogyan változik a pipeline haszna, ha a számítási idő sokkal nagyobb, mint a másolási idő? Indokolja válaszát a `T_pipe  max(T_c, T_m) + T_s` összefüggéssel.

A válaszok ellenőrzéséhez a következő gondolatmenet használható. Az 1. kérdésnél az első `__syncthreads()` a másolás befejezése utáni láthatóságot, a második a puffer újrahasznosításának biztonságát adja. A 2. kérdésnél a `wait_prior(0)` minden befejezetlen másolást bevár, ezért a következő csempe másolása nem folyik a számítás alatt, és a pipeline elveszti az átfedési előnyét. A 3. kérdésnél a `max(T_c, T_m)` értékét a nagy számítási idő határozza meg, ezért a csővezeték nem tudja csökkenteni a teljes időt, mert a memóriaidő eltörpül a számítás mellett.

14.3.13 A következő lépés

A Tiling és pipeline mérése alfejezet ezt a fogalmat a tiling és a pipeline együttes mérésére terjeszti ki. Ott a `cp.async` mellett a hagyományos szinkron másolást, a vektorizált betöltést és a csempeformák hatását is összehasonlítjuk. Az itt bemutatott két- és hárompufferes minta lesz az alapja annak a mérési keretnek, amelyben a Roofline-modell és az occupancy közötti kapcsolatot is számszerűsíteni fogjuk.

14.4 Tiling és pipeline mérése

A tiling és a pipeline hatásának mérése nem egyszeri futásidő-mérésből áll. A memória-hozzáférési minták, az occupancy, a bankütközések és a latency hiding együttesen határozzák meg, hogy egy optimalizálás valóban csökkenti-e a végrehajtási időt. Ebben az alfejezetben egy olyan mérési keretet építünk fel, amely a tiling és a double buffering hatását reprodukálhatóan, számszerűen és Nsight-alapú metrikákkal vizsgálja. A példa egy mátrix transzponálás lesz; a módszertan azonban változtatás nélkül alkalmazható stencil-, konvolúció- vagy bármely memória-intenzív tiled kernelre.

14.4.1 1. Mérési keret és reprodukálhatóság

Mielőtt bármilyen GPU-n futó kísérletet értékelünk, rögzíteni kell a környezetet. A könyv mérési szerződése szerint minden állításhoz tartoznia kell GPU-modellnek, driververziónak, CUDA-verziónak, compute capabilitynek, operációs rendszernek, órajelnek, power limitnek, valamint annak, hogy ECC, MIG vagy virtuális GPU aktív-e. Ez nem formalitás: ugyanaz a kernel egy A100-on és egy RTX 4090-en más okból lehet gyors vagy lassú.

A mérési protokoll lépései:

1. **Warm-up.** Az első kernelindítások inicializálják a modulokat, a konstans memóriát, a driver belső állapotait. Legalább 1–2 extra indítást végezzünk, mielőtt mérni kezdünk.

2. **Többszöri ismétlés.** Minden konfigurációt legalább 5–10-szer futtassuk. Az átlag mellett közöljük a szórást vagy percentilist is, mert a GPU-órjel, a hőmérséklet és a háttérterhelés fluktuál.
3. **Azonos input.** A tiling és pipeline összehasonlításánál ugyanazt a bemeneti mátrixot, ugyanazt a memóriafoglalást és ugyanazt a kimeneti puffert használjuk.
4. **Környezetrögzítés.** A mérés előtt és után kérdezzük le a `nvidia-smi`-vel az órajelet, a hőmérsékletet és a power limitet. Ha a GPU throttlingol, a mérést meg kell ismételni.
5. **Hibakezelés.** Minden CUDA-hívás után `cudaGetLastError` és `cudaDeviceSynchronize`; a kernelindítás után `CHECK_CUDA(cudaDeviceSynchronize())`.

A mérési célok három kernelvariánsra:

- **Baseline:** naiv globális memóriás transzponálás.
- **Tiled:** shared memory csempézés, bankütközés elkerülése paddinggel.
- **Pipelined:** shared memory csempézés és aszinkron másolás (`cp.async`) double bufferinggel.

A várt hatás iránya ismert: a tiling csökkenti a globális memória tranzakciók számát, a pipeline pedig elrejti a globális memória késleltetését. A mérés feladata ennek számszerűsítése, nem pedig a „gyorsabb” kijelentés bizonyítása.

14.4.2 2. A példa: FP32 mátrix transzponálás

A transzponálás azért alkalmas kísérleti terep, mert memória-intenzív, és a naiv változat egyik memória-hozzáférése szükségképpen nem koaleszkált. A tiled változat mindkét irányban koaleszkált olvasást és írást valósít meg; a pipelined változat ezen felül a globális betöltést a számítás alá csúsztatja.

Input: `A` mátrix `M` sorral és `N` oszloppal, FP32 elemekkel. Output: `B` mátrix `N` sorral és `M` oszloppal, `B[n, m] = A[m, n]`. A teljes adatforgalom pontosan `2 * M * N * sizeof(float)` byte, mert minden elemet egyszer olvasunk és egyszer írunk.

A kernelváltozatok vázlata CUDA C++-ban (C++17, `nvcc`):

```
#include <cuda_runtime.h>
#include <cstdio>
#include <cstdlib>

#define CHECK_CUDA(call)                                                    \
  do {                                                                      \
    cudaError_t err = call;                                                 \
    if (err != cudaSuccess) {                                               \
      std::fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__, __LINE__,  \
                   cudaGetErrorString(err));                                \
      std::exit(EXIT_FAILURE);                                              \
    }                                                                       \
  } while (0)

constexpr int TILE_DIM = 32;

// 1. Naive: global memory only
__global__ void transpose_naive(const float* A, float* B, int M, int N) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x < N && y < M) {
        B[x * M + y] = A[y * N + x]; // one side is uncoalesced
    }
}
```

```
// 2. Tiled: shared memory with padding
__global__ void transpose_tiled(const float* A, float* B, int M, int N) {
    __shared__ float tile[TILE_DIM][TILE_DIM + 1]; // padding avoids bank conflicts

    int x = blockIdx.x * TILE_DIM + threadIdx.x;
    int y = blockIdx.y * TILE_DIM + threadIdx.y;

    if (y < M && x < N) {
        tile[threadIdx.y][threadIdx.x] = A[y * N + x];
    }
    __syncthreads();

    x = blockIdx.y * TILE_DIM + threadIdx.x;
    y = blockIdx.x * TILE_DIM + threadIdx.y;

    if (y < N && x < M) {
        B[y * M + x] = tile[threadIdx.x][threadIdx.y]; // shared load, transposed
    }
}
```

A harmadik változat a `cuda::pipeline` API-t használja. A cél: amíg a blokk az aktuális shared memória csempét transzponálja, a következő csempe globális betöltése már folyamatban van. A vázlat:

```
#include <cuda_pipeline.h>

__global__ void transpose_pipelined(const float* A, float* B, int M, int N) {
    extern __shared__ float smem[];
    constexpr int TILE = 32;
    // two shared buffers with padding
    float* buf0 = smem;
    float* buf1 = smem + TILE * (TILE + 1);

    auto pipe = cuda::make_pipeline();

    int tile_x = blockIdx.x * TILE;
    int tile_y = blockIdx.y * TILE;

    // simplified: first stage global -> shared for buf0
    if (threadIdx.x < TILE && threadIdx.y < TILE) {
        int src_x = tile_x + threadIdx.x;
        int src_y = tile_y + threadIdx.y;
        if (src_y < M && src_x < N) {
            cuda::memcpy_async(block, buf0 + threadIdx.y * (TILE + 1) + threadIdx.x,
                               A + src_y * N + src_x, sizeof(float), pipe);
        }
    }
    pipe.producer_commit();
    pipe.consumer_wait();

    // transpose and write buf0, while loading buf1 asynchronously
    // ... (full implementation in lab)
}
```

A valós laboranyagban a pipeline teljes producer–consumer logikája megtalálható; itt a lényeg az, hogy a `cuda::memcpy_async` a globális memóriából a shared memóriába másolást a memória-utasítás szintjén aszinkron módon hajtja végre, és a `producer_commit/consumer_wait` szinkronizálja a fázisokat. A `cp.async` használatához compute capability 8.0 vagy újabb szükséges; régebbi hardveren a pipeline nem elérhető, ezért a laborkörnyezetben ellenőrizni kell a `cudaDeviceProp::asyncEngineCount` és a compute capability értékét.

A launch konfiguráció mindhárom kernelnél:

- Naiv: `dim3 block(32, 32, 1)`, grid: `((N + 31)/32, (M + 31)/32, 1)`.
- Tiled: ugyanaz, de a tile méret `TILE_DIM`; a block dimenzió `TILE_DIM x TILE_DIM`.
- Pipelined: ugyanaz, de a shared memória mérete `2 * TILE_DIM * (TILE_DIM + 1) * sizeof(float)` byte.

A grid- és blockdimenziót minden mérési jegyzőkönyvben rögzíteni kell.

14.4.3 3. Mérési protokoll és Nsight-alapú vizsgálat

A mérésekhez rögzített környezet:

- GPU: például NVIDIA A100 40 GB (helyőrző; a tényleges modellt a laborban kell beírni)
- Driver: 535.xx (helyőrző)
- CUDA Toolkit: 12.2
- Compute capability: 8.0
- Operációs rendszer: Ubuntu 22.04 LTS
- Power limit: 400 W, ECC: bekapcsolva
- MIG: nincs, virtualizáció: nincs

A kód build parancsa:

```
nvcc -O3 -std=c++17 -arch=sm_80 -o transpose_lab transpose_lab.cu
```

A futtatás előtt ellenőrizzük a hibákat, majd a mérési kernelt hívjuk:

```
./transpose_lab --m 8192 --n 8192 --repeat 10
```

A Nsight Compute profilozás parancsa egy adott kernelre:

```
ncu --set full --launch-count 1 --kernel-id ::transpose_tiled ./transpose_lab --m 8192 --n 8192
```

Célzott metrikák lekérdezése:

```
ncu --metrics
↪  gld_transactions,gst_transactions,l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum,sm__warps_active
↪  ./transpose_lab --m 8192 --n 8192
```

A mérési naplóban külön oszlopban szerepeljen a **várt** (elméleti) érték és a **mért** érték. A várt értékeket a memóriamodellből, a tranzakciószámításból és az occupancy-előrejelzésből becsüljük, a mért értékeket pedig a Nsight-riportból vagy a saját timerből nyerjük.

Példa várt értékek:

- **Naiv globális tranzakciók:** A transzponálásnál az egyik oldal nem koaleszkált. Egy 32 byte-os szektorral, FP32 elemekkel a naiv kernel minden második vagy negyedik szektort érinthet, ezért a globális load/store tranzakciók száma lényegesen magasabb, mint a minimális `M*N/8` (ha 32 byte = 8 float). A pontos érték a hozzáférési mintától függ, de a tiled kernelhez képest többszörös tranzakciószám várható.
- **Tiled globális tranzakciók:** Koaleszkált hozzáféréssel minden 32 byte-os szektor pontosan egyszer érintett: `gld_transactions` $\approx$ `M*N/8`, `gst_transactions` $\approx$ `N*M/8`.
- **Bankütközések:** A `TILE_DIM+1` padding miatt a shared memória olvasásnál a bankütközések száma közel nulla. Padding nélkül 32x32-es csempénél a transzponált olvasás 32-way bank conflictot okozna.
- **DRAM forgalom:** Mindhárom kernelnél `2 * M * N * 4` byte, mert az alkalmazás minden elemet egyszer mozgat. A tiling nem csökkenti a teljes DRAM forgalmat, csak a tranzakciók számát és a késleltetést.

- **Sávszélesség:** `sávszélesség = 2 * M * N * 4 / (futásidő * 1e9)` GB/s. A tiled kernelnek közelítenie kell a GPU memória-sávszélességének 80–90%-ához.

A mérési napló egy sora például:

Változat	Tile	Block	Futásidő (µs)	Szórás	Sávszélesség (GB/s)	gld_transactions	Bank conflicts	Occupancy
naive	–	32x32	*mérendő*	*mérendő*	*mérendő*	*mérendő*	–	*mérendő*
tiled	32x32	32x32	*mérendő*	*mérendő*	*mérendő*	*mérendő*	*mérendő*	*mérendő*
pipelined	32x32	32x32	*mérendő*	*mérendő*	*mérendő*	*mérendő*	*mérendő*	*mérendő*

A várt értékeket a szövegben a fenti becslések adják, a mért értékeket a laborban kell rögzíteni. Fontos, hogy a mérési naplóban a várt és mért értékek elkülönüljenek, és a becslés képlete is szerepeljen.

14.4.4 4. Eredmények értelmezése: Nsight-metrikák és a hatásmechanizmus

A nyers futásidő önmagában keveset mond. A Nsight Compute metrikái megmutatják, miért változott a teljesítmény.

14.4.5 4.1. Globális memória tranzakciók

A `gld_transactions` és `gst_transactions` metrikák a ténylegesen végrehajtott globális memória tranzakciók számát adják. A naiv transzponálásnál az egyik oldal koaleszkálatlan, ezért egy warp 32 elemhez több 32 byte-os tranzakcióra van szükség. A tiled kernelben a shared memórián keresztül mindkét oldal koaleszkált: a betöltés soronként folyamatos, a kiírás is soronként folyamatos. A várt hatás a tranzakciószám drasztikus csökkenése.

A mérés során a `gld_transactions` és `gst_transactions` értékét összehasonlítva a tiled kernelnek közel a minimális `M*N/8` értéket kell mutatnia. Ha a mért érték magasabb, akkor a hozzáférési minta nem optimális, vagy a grid méretezése miatt határfeltételek rontják a koaleszkálást.

14.4.6 4.2. Shared memory bankütközések

A tiled kernelnél a `__shared__ float tile[TILE_DIM][TILE_DIM + 1]` deklaráció a padding miatt elkerüli a bankütközést. A Nsight-ban a `l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.su` metrika mutatja a shared memória olvasási bankütközéseit. Padding nélkül 32x32-es csempénél a transzponált olvasás során minden warp 32 thread-je ugyanabba a bankcsoportba ütközne, ami 32-szeres konfliktust jelent. A `+1` oszlop eltolja a sorokat, így a konfliktus megszűnik.

A várt érték: paddinggel a bankütközések száma nulla vagy elhanyagolható. Ha a mért érték magas, akkor a paddinget elfelejtettük, vagy a tile méret és a bankek száma nem illeszkedik.

14.4.7 4.3. Occupancy és erőforrás-korlátok

Az `sm__warps_active.avg.pct_of_peak_sustained_active` metrika az aktív warpek arányát mutatja. A tiled kernel shared memóriát foglal: `TILE_DIM * (TILE_DIM + 1) * 4` byte. 32x32-es tile esetén ez $32 \mathit{33} 4 = 4224$ byte blokkonként. A pipelined változat két puffert használ, így 8448 byte-ot. Egy A100 SM-en 164 KB shared memória áll rendelkezésre, ezért a shared memória nem korlátozza erősen az occupancyt. A regiszterhasználat azonban igen: ha a compiler túl sok regisztert rendel, az aktív warpek száma csökken.

A várt hatás: a tiled kernel occupancyje hasonló vagy kissé alacsonyabb a naiv kernelhez képest, de a csökkent memória-tranzakciók miatt a tényleges teljesítmény nő. A pipelined kernelben a double buffering miatt több shared memória kell, de a latency hiding javul, így a foglaltság csökkenése ellenére is gyorsabb lehet.

A mérésnél rögzíteni kell a regiszterhasználatot (`launch__registers_per_thread`) és a shared memória foglalást is, mert ezek magyarázzák az occupancyt.

14.4.8 4.4. DRAM forgalom és sávszélesség

A `dram__bytes.sum` metrika a tényleges DRAM-forgalmat mutatja. A várt érték mindhárom kernelnél azonos: `2 * M * N * 4` byte. Ha a mért érték ettől eltér, akkor a cache vagy a memória-hozzáférési minta extra forgalmat generál (például a határfeltételek miatt). A tiling és pipeline nem csökkenti a teljes forgalmat, csak a tranzakciók számát és a késleltetést.

A sávszélesség számítása a fenti képlettel történik. A naiv kernel sávszélessége alacsony, mert a koaleszkálatlan hozzáférések miatt a memória-vezérlő nem tudja kihasználni a burstöket. A tiled kernel sávszélessége megközelíti a memória-sávszélesség felső határát. A pipelined kernel tovább javíthatja a sávszélességet, mert a globális betöltés késleltetése el van rejtve, és a memória-vezérlő folyamatosan kap kéréseket.

14.4.9 4.5. A pipeline hatása: latency hiding

A `cp.async` pipeline lényege, hogy a globális memória kéréseket a számítással párhuzamosítja. A Nsight Compute-ben a `smsp__average_warp_latency_per_inst_issued` vagy a `smsp__warp_issue_stalled_long_scoreboard_per_warp_active.pct` metrika mutatja, mennyi időt töltenek a warpek memória-várakozással. A pipelined kernelben ennek az értéknek csökkennie kell a tiled kernelhez képest, mert a consumer nem közvetlenül a globális load után vár, hanem a már megérkezett shared pufferen dolgozik.

A várt eredmény: a pipelined kernel futásideje alacsonyabb, a memória-várakozási idő csökken, a sávszélesség pedig közelebb kerül a csúcshoz. Ha a csökkenés nem jelentkezik, akkor a pipeline szinkronizációja helytelen, vagy a blokk túl kevés munkát végez ahhoz, hogy a betöltés elrejthető legyen.

14.4.10 5. Tileméret-hangolás

A tile méretének megválasztása közvetlenül befolyásolja a shared memória foglalást, a bankütközéseket, az occupancyt és a határfeltételek kezelését. A hangolást méréssorozattal végezzük, ahol a tile méretet változtatjuk, és minden konfigurációra rögzítjük a fenti metrikákat.

Gyakori tile méretek FP32 transzponálásnál:

- **8x8:** kis shared memória, magas occupancy, de sok blokk és sok határfeltétel. A globális memória tranzakciók száma magasabb, mert a koaleszkálás rövidebb.
- **16x16:** 16*17*4 = 1088 byte shared memória blokkonként. Jó egyensúly, de a bankütközések elkerülhetők paddinggel.
- **32x32:** 32*33*4 = 4224 byte. A legtöbb GPU-n ez a standard választás, mert a warp méret (32) illeszkedik a sorhosszhoz, és a koaleszkálás optimális.
- **64x64:** 64*65*4 = 16640 byte. Magas shared memória igény, alacsonyabb occupancy, de kevesebb blokk és határfeltétel. Nagy mátrixoknál előnyös lehet, de a latency hiding romolhat.

A mérési sorozat:

```
for tile in 8 16 32 64; do
    ./transpose_lab --m 8192 --n 8192 --tile $tile --repeat 10
done
```

A kapott futásidőket, sávszélességeket és occupancyket táblázatba foglaljuk. A várt trend: a 32x32-es tile közel optimális. A 64x64-es tile esetén a sávszélesség csökkenhet, mert az occupancy alacsonyabb, és a latency hiding kevésbé hatékony. A 8x8-as tile esetén a sok blokk overheadje és a rövidebb koaleszkált szegmensek rontják a teljesítményt.

A pipelined kernelnél a tile méret mellett a shared memória pufferek száma is hangolható. A double bufferinghez két puffer kell; ha a tile túl nagy, a pufferek kiszoríthatják a blokkokat az SM-ből, így az occupancy drasztikusan csökken. A mérés során a `cudaFuncSetAttribute` segítségével növelhetjük a shared memória limitet, de figyelni kell a maximális kapacitásra.

A hangolás eredményét a következő táblázatban rögzítjük (várt és mért külön):

Tile	Shared memória/block (byte)	Regiszter/thread	Occupancy (várt)	Sávszélesség (várt)	Sávszélesség (mért)
8x8	8*9*4=288	*mérendő*	magas	közepes	*mérendő*
16x16	16*17*4=1088	*mérendő*	magas	jó	*mérendő*
32x32	32*33*4=4224	*mérendő*	közepes/magas	legjobb	*mérendő*
64x64	64*65*4=16640	*mérendő*	alacsony	csökkenő	*mérendő*

A mért értékeket a Nsight és a saját timer alapján kell kitölteni. A várt occupancyt a `cudaOccupancyMaxActiveBlocksPerMultiprocessor` API-val előre lehet becsülni; a laborban ezt is érdemes lefuttatni.

14.4.11 6. Tipikus hibák és elhárításuk

14.4.12 6.1. Nem ellenőrzött környezet

Hiba: a mérés közben a GPU órajele változik, thermal throttling lép fel, vagy más folyamat terheli a GPU-t. A kapott eredmény nem reprodukálható.

Megoldás: mérés előtt és után `nvidia-smi --query-gpu=clocks.sm,clocks.mem,power.draw,temperature.g --format=csv` rögzítése. Ha throttlingot látunk, csökkentsük a mérés időtartamát, vagy alkalmazzunk hűtési szünetet. Többfelhasználós környezetben használjunk MIG-et vagy kérjünk dedikált GPU-időt.

14.4.13 6.2. Túl kicsi input

Hiba: a mátrix mérete kicsi, a kernelindítási overhead dominál, a tiling hatása nem mérhető.

Megoldás: legalább 4–8 MB adatot használjunk. 8192x8192 FP32 mátrix 256 MB, ami elegendő. Ha a memória korlátozott, válasszunk 4096x4096-ot, de ne menjünk 512x512 alá.

14.4.14 6.3. Csak egy mérés

Hiba: egyetlen futásidőt rögzítünk, a zajt nem szűrjük.

Megoldás: warm-up után legalább 10 ismétlés, átlag és szórás közlése. A szórás nagy értéke (pl. >5%) környezeti problémára utal.

14.4.15 6.4. Bankütközés figyelmen kívül hagyása

Hiba: a tiled kernelben `__shared__ float tile[TILE_DIM][TILE_DIM]` deklarációt használunk padding nélkül. A 32x32-es transzponált olvasás 32-way bank conflictot okoz, és a tiling előnye elvész.

Megoldás: mindig `TILE_DIM + 1` oszlopot használjunk, vagy `cudaFuncSetAttribute`-tel állítsuk be a shared memória méretét úgy, hogy a sorok eltolása biztosított legyen. A Nsight bankütközés-metrikájával ellenőrizzük.

14.4.16 6.5. A cp.async pipeline helytelen szinkronizációja

Hiba: a `cuda::pipeline` producer és consumer fázisainak sorrendje hibás, például a `consumer_wait` előtt a shared puffert olvassuk, vagy a puffert felülírjuk, mielőtt a consumer végzett volna. Ez versenyhelyzetet okoz.

Megoldás: kövessük a pipeline API szabályait. A producer oldal a `cuda::memcpy_async` hívás után `producer_commit`-ot hajt végre; a consumer oldal `consumer_wait`-tel vár a másolás befejezésére, majd `consumer_release`-szel jelzi, hogy a puffer felszabadult. A shared memória puffereket két külön bufferre osztjuk, és a blokk-szintű `__syncthreads`-eket a fázisok között elhelyezzük. A full implementation a laborban található; itt a hibás minta:

```
// HIBÁS: nincs producer_commit, a consumer azonnal olvas
cuda::memcpy_async(block, buffer, source, bytes, pipe);
__syncthreads();
use(buffer); // lehet, hogy a másolás még nem fejeződött be
```

A helyes minta:

```
cuda::memcpy_async(block, buffer, source, bytes, pipe);
pipe.producer_commit();
pipe.consumer_wait();
__syncthreads();
use(buffer);
pipe.consumer_release();
```

14.4.17 6.6. Túl nagy tile, alacsony occupancy

Hiba: a 64x64-es tile csökkenti az aktív blokkok számát, a latency hiding romlik, és a kernel lassabb lesz.

Megoldás: mérjük az occupancyt, és a regiszterhasználatot (`--maxrregcount` vagy launch bounds) hangoljuk. Ha a shared memória a korlát, csökkentsük a tile méretet, vagy használjunk kisebb puffert a pipeline-ban. A cél az, hogy legalább 2–4 blokk aktív legyen SM-enként, hogy a warpütemező elrejthesse a memória-késleltetést.

14.4.18 7. Labor: Nsight-alapú elemzés

A labor célja, hogy a fenti három kernelváltozatot saját környezetben lefuttassa, és a mérési naplót Nsight-metrikákkal töltse ki. A kódot a mellékelt `transpose_lab.cu` tartalmazza; a fordítás és futtatás parancsai:

```
nvcc -O3 -std=c++17 -arch=sm_80 -o transpose_lab transpose_lab.cu
./transpose_lab --m 8192 --n 8192 --repeat 10
```

A Nsight Compute futtatás:

```
ncu --set full --launch-count 1 --kernel-id ::transpose_naive ./transpose_lab --m 8192 --n 8192
ncu --set full --launch-count 1 --kernel-id ::transpose_tiled ./transpose_lab --m 8192 --n 8192
ncu --set full --launch-count 1 --kernel-id ::transpose_pipelined ./transpose_lab --m 8192 --n
↪  8192
```

A mérési naplóban rögzíteni kell:

- GPU-modell, driver, CUDA-verzió, compute capability, operációs rendszer, power limit, ECC/MIG állapot.
- A három kernel futásideje, szórása, sávszélessége.
- A Nsight-ből: `gld_transactions`, `gst_transactions`, `dram__bytes.sum`, `l1tex__data_bank_conflicts`, `sm__warps_active.avg.pct_of_peak_sustained_active`, regiszter/thread, shared memória/block.
- A várt értékeket a fenti becslésekkel, a mért értékeket a kísérletből.

A labor átadandó dokumentuma egy rövid jelentés, amelyben a mért és várt értékeket táblázatban összeveti, és legalább egy Nsight-képernyőképet vagy riportrészletet mellékel. A jelentésben válaszolni kell a kérdésre: **Miért nő a sávszélesség a tiled kernelnél, ha a DRAM-forgalom azonos?** A válasz: a koaleszkált hozzáférések miatt a memória-vezérlő hatékonyabban használja a burstöket, és a tranzakciók száma csökken, így ugyanannyi byte mozgatása kevesebb időt vesz igénybe. A pipelined kernel pedig a globális memória késleltetését rejti el, így a memória-vezérlő folyamatosan telített.

Megjegyzés: A fenti parancsok és a kódvázlat szemléltető jellegű; a laborban a tényleges GPU-paramétereket és a mérési naplót a hallgatónak kell rögzítenie. A cp.async pipeline használatához compute capability 8.0+ szükséges; ha a labor GPU-ja régebbi, a pipeline részt ki kell hagyni, és a mérési jegyzőkönyvben ezt jelezni kell.

14.4.19 8. Összefoglalás és önellenőrző kérdés

A tiling és a pipeline hatásának mérése akkor értékes, ha a futásidő mellett a memória-tranzakciókat, a bankütközéseket, az occupancyt és a sávszélességet is vizsgáljuk. A mátrix transzponálás példáján a naiv, tiled és pipelined változat összehasonlítása megmutatja: a tiling a tranzakciók számát csökkenti, a pipeline a késleltetést rejti el, a tileméret pedig az erőforrás-korlátokon keresztül hangolja az optimális pontot.

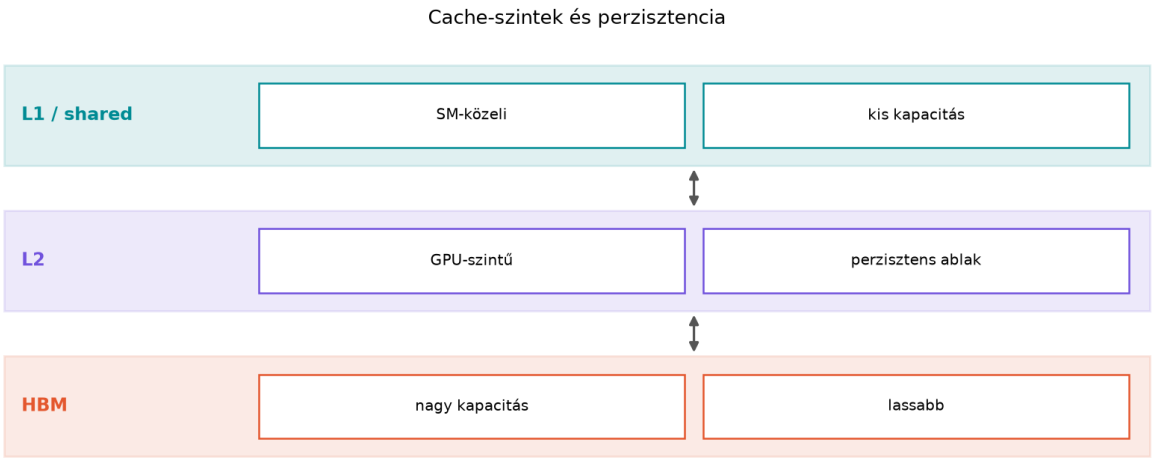
Önellenőrző kérdés: Miért csökken a globális memória tranzakciók száma a tiled transzponálásnál, és miért nem csökken a DRAM-forgalom? Válaszát a Nsight-metrikák alapján indokolja.

A következő fejezet (L1 és L2 cache) a cache-ekre, az L2 persistence-re és a memóriamodellekre épít; az itt megismert mérési protokoll és a Nsight-metrikák ott is alapvető eszköztárként szolgálnak.

15 Cache, L2, persistence és memóriamodellek

15.1 L1 és L2 cache

A cache-hierarchia első szintjei – az L1 és az L2 gyorsítótár – a GPU memóriamodelljének azon elemei, amelyeket a fejlesztő közvetlenül nem programoz, mégis minden global memóriából érkező hozzáférésre hatással vannak. A korábbi fejezetben megismert shared memory tiling és async copy azt mutatta meg, hogyan tudjuk az adatokat explicit módon a számítás közelébe hozni. Ebben az alfejezetben a hardveres cache-ek működését, a találati arány szerepét, a kapacitások jelentőségét, valamint a cache sor és asszociativitás fogalmát vizsgáljuk meg. A cél az, hogy a cache-viselkedést ne csak elméletben értsük, hanem egy ellenőrzött méréssel is demonstráljuk, és felismerjük a tipikus hibákat.



15.1. ábra: Cache-szintek és perzisztencia - Az L1 az SM-hez, az L2 a teljes GPU-hoz tartozik; a perzisztens L2 ablakkal a gyakran újraolvasott adat a gyorsítótárban tartható.

15.1.1 A cache-szintek és a találati arány

A GPU memóriahierarchiájában a regiszterek és a shared memory után a következő szintet az L1, majd az L2 cache jelenti. Az L1 cache streaming multiprocesszoronként (SM) található, és a global memória hozzáférések gyors kiszolgálására szolgál. Az L2 cache a GPU teljes eszközén osztozik, minden SM ugyanazt az L2-t használja. Az L1 és L2 közötti alapvető különbség a kapacitásban és az elérési időben van: az L1 kicsi és gyors, az L2 nagyobb és lassabb, de mindkettő lényegesen gyorsabb, mint a global memória (a HBM/GDDR) elérése.

A cache-ek hatékonyságát a **találati arány** (hit rate) méri. Ha egy thread a global memóriában lévő címet olvas, a hardver először az L1 cache-ben keresi. Ha az adat ott van, akkor **találat** (hit) történik, és az olvasás néhány ciklus alatt befejeződik. Ha nincs találat, **miss** következik: az

L1-ből továbbítja a kérést az L2 felé. Ha az L2-ben sincs meg az adat, akkor a global memóriából kell betölteni. A találati arány az összes hozzáféréshez viszonyított találatok aránya:

$$h = \frac{N_{\text{hit}}}{N_{\text{hit}} + N_{\text{miss}}}$$

A találati arány közvetlenül meghatározza az átlagos elérési időt:

$$T_{\text{avg}} = h \cdot T_{\text{hit}} + (1 - h) \cdot T_{\text{miss}}$$

ahol T_{hit} és T_{miss} a találat és a miss esetén mért elérési idő. A GPU-nál a miss nem egyszerűen egyetlen időérték: az L1 miss után L2 találat vagy L2 miss is bekövetkezhet, így a teljes memóriarendszer hierarchikus sorrendben kezeli a kéréseket. A gyakorlatban a fejlesztő számára a teljes futásidő és az effektív sávszélesség a mérvadó, mert a találati arányt közvetlenül nehéz megfigyelni.

A GPU cache-ek **nem koherensek** a CPU cache-ekkel, és a CUDA programozási modell nem garantálja, hogy a global memóriába írt adat azonnal látható legyen a cache-ben vagy a többi SM számára. Ezt a láthatósági kérdést a memóriamodell fejezet tárgyalja részletesen. Itt elég annyi, hogy a cache-ek a global és local memória eléréseket gyorsítják, de a shared memory-hoz és a regiszterekhez nem kapcsolódnak.

15.1.2 L1 és L2 kapacitás

A cache-ek kapacitása hardvergenerációnként és compute capability szerint változik. Az L1 cache az SM-en belül található, és gyakran osztozik a shared memory-val ugyanazon a fizikai memóriaterületen. Emiatt az L1 mérete nem állandó: a shared memory igénye befolyásolhatja, hogy mennyi terület jut az L1-nek. Például egyes architektúrákon az SM-en belüli 192 KiB közös területből 64 KiB lehet L1, 128 KiB shared memory, de ez a konfiguráció változhat. A pontos értékeket a gyártó dokumentációja és a `cudaDeviceGetAttribute` segítségével lehet ellenőrizni. A gyakorlatban a fejlesztő ritkán kell, hogy pontosan ismerje az L1 méretét, mert a legfontosabb hatás a shared memory carving: ha túl sok shared memory-t foglalunk, az L1 cache kisebb lesz, ami ronthatja a global memória hozzáférések teljesítményét.

Az L2 cache kapacitása ezzel szemben közvetlenül lekérdezhető, mert a `cudaDeviceProp::l2CacheSize` mező byte-ban adja meg. A modern szerver GPU-kon az L2 kapacitás látványosan nőtt: a V100 6 MiB L2-vel rendelkezik, az A100 40 MiB, a H100 pedig 50 MiB L2 cache-sel. Ezt a kapacitást az egész GPU, azaz az összes SM közösen használja. A kapacitás közvetlenül befolyásolja, hogy mekkora munkahalmaz fér el a cache-ben: ha a kernel által ismételten elért adatmennyiség kisebb, mint az L2, akkor az első miss-ek után a későbbi hozzáférések találattal szolgálódnak ki, és a teljesítményt az L2 sávszélessége határozza meg. Ha a munkahalmaz nagyobb, mint az L2, a találati arány drasztikusan csökken, és a global memória sávszélessége válik szűk keresztmetszetté.

Az L1 és L2 közötti kapacitáskülönbség miatt a tervezési stratégiák eltérnek: az L1 az adatok rövid távú, kis méretű újrafelhasználását gyorsítja – például egy szomszédos elem ismételt elérését egy stencil-kernelben –, míg az L2 a nagyobb, blokkszintű vagy akár grid-szintű adatújrafelhasználást képes kiszolgálni. A mérési labor erre fog konkrét példát mutatni.

15.1.3 Cache line és asszociativitás

A cache-ek az adatokat nem bájtonként, hanem rögzített méretű blokkokban, **cache sorokban** kezelik. A GPU esetében a memóriarendszer a global memória hozzáféréseket **szektorokra** bontja; egy szektor jellemzően 32 bájt. Az L1 és L2 cache-ek a szektorokat cache sorokba szervezik, és egy cache sor több szektort tartalmazhat. A gyakorlatban a CUDA hozzáférési tranzakciók 32, 64 vagy 128 bájtos szegmensekből állnak; a 128 bájtos tranzakció négy 32 bájtos szektort jelent. Ha egy warp 32 threadje folytonos float elemeket olvas, az 128 bájtnyi adat, amely pontosan négy szektort fed le. Ez a kialakítás teszi lehetővé, hogy a coalescing jól működjön: ha a warp hozzáférései egy cache soron belül maradnak, a memóriarendszer hatékonyan, kevés tranzakcióval szolgálja ki a kérést.

A cache sor méretének és a hozzáférési minta együttese határozza meg a **cache line kihasználtságot**. Ha például minden thread csak minden 8. float elemet olvas (stride = 8), akkor egy 32 threadből álló warp 256 bájtnyi tartományt ér el, ami nyolc 32 bájtos szektor, tehát a memóriarendszer több tranzakciót indít, és a betöltött szektorokból csak egy rész hasznosul. Ez rontja az effektív sávszélességet, még akkor is, ha az adatok egy része a cache-ben található, mert a cache sorokból csak kevés elemet használunk fel.

Az **asszociativitás** azt írja le, hogy egy adott memóriacím mely cache sorokba kerülhet. A legegyszerűbb felépítés a **közvetlen leképezés** (direct-mapped cache), ahol egy cím csak egyetlen cache sorba kerülhet. A másik véglet a **teljesen asszociatív** cache, ahol a cím bármelyik sorba kerülhet. A gyakorlatban a GPU-k **set-associative** cache-eket használnak: a cache sorok halmazokba (set) vannak osztva, és egy cím a halmazon belül néhány lehetséges sorba kerülhet. Az asszociativitás mértéke például 4-way vagy 8-way, ami azt jelenti, hogy egy halmazban 4 vagy 8 cache sor van, és a cím bármelyikbe bekerülhet.

A set-associative cache címfelbontása három részből áll: a **tag** a cím felső bitjeiből áll, és a sorban tárolt cím egyezését ellenőrzi; az **index** a halmazt választja ki; az **offset** pedig a soron belüli bájtot. Ha egy halmazban minden sor foglalt, és egy új cím ugyanabba a halmazba esik, akkor a cache-nek ki kell választania egy sort, amelyet felülír. Ez **konfliktusos miss**-hez vezethet akkor is, ha a cache egésze még nem telt meg: ha a hozzáférési minta olyan, hogy sok különböző cím ugyanabba a halmazba esik, a cache thrash-elni kezd, és a találati arány leesik. Ez különösen akkor fordul elő, ha a hozzáférési lépés a cache méretének vagy a halmazok számának egész számú többszöröse.

A GPU cache-ek asszociativitása architektúránként változik, és a gyártó általában nem hozza nyilvánosságra a pontos értékeket. A fejlesztő számára a legfontosabb gyakorlati következmény az, hogy a **stride hozzáférési minták** bizonyos lépésközöknél drasztikusan ronthatják a találati arányt. A mérésnél ezt a jelenséget is megfigyelhetjük.

15.1.4 Labor: cache-viselkedés mérése

A labor célja, hogy a cache-szintek hatását saját környezetben, ellenőrzött méréssel mutassuk meg. A kód CUDA C++-ban készül, és a global memóriából történő olvasások futásidejét méri különböző munkahalmaz-méretek és hozzáférési stride-ok mellett. A mérésből arra következtetünk, hogy mikor dolgozik a kernel az L2 cache-ből, és mikor kényszerül a global memória sávszélességére.

15.1.5 Mérési környezet rögzítése

A könyv laborjainak szabálya szerint minden mérésnél rögzíteni kell a pontos környezetet. A példa kedvéért tegyük fel a következő konfigurációt, de a laborban ezt a saját géphez kell igazítani:

- GPU: NVIDIA A100 40 GB (compute capability 8.0)
- Driver: 535.104.05
- CUDA verzió: 12.2
- Operációs rendszer: Ubuntu 22.04 LTS
- Fordító: nvcc 12.2, C++17

A kód a `cudaDeviceProp` struktúrából kiírja az eszköz legfontosabb paramétereit, így a mérési naplóban ezek is rögzítve vannak. A laborban a tényleges GPU adatait kell feltüntetni.

15.1.6 A mérőprogram forráskódja

A program megvalósítja a cache-viselkedés mérését. A kernel egy `float` tömböt olvas, és a hozzáférések során összeget számol. A `stride` paraméter határozza meg, hogy a szomszédos thread-ek hány elemet lépnek. A `repeat` paraméter az ismétlések száma, amellyel a teljes beolvasott adatmennyiséget szabályozzuk.

```
// cache_probe.cu
#include <cuda_runtime.h>
#include <cstdio>
#include <cstdlib>
#include <vector>

// Hibakezelő makró: minden CUDA hívás után ellenőriz
#define CHECK_CUDA(call)                                                   \
    do {                                                                   \
        cudaError_t err = call;                                            \
        if (err != cudaSuccess) {                                          \
            std::fprintf(stderr, "CUDA hiba: %s (%s:%d)\n",                \
                         cudaGetErrorString(err), __FILE__, __LINE__);     \
            std::exit(EXIT_FAILURE);                                       \
        }                                                                  \
    } while (0)

// Cache-viselkedést mérő kernel.
// data: a bejárandó tömb (global memória)
// output: a sum értékét ide írjuk, hogy a fordító ne optimalizálja ki
// n: a tömb elemszáma
// stride: az elemek közötti lépésköz
// repeat: hányszor olvassa be a teljes tömböt (n * repeat elem)
__global__ void cache_probe_kernel(const float* __restrict__ data,
                                   float* __restrict__ output,
                                   size_t n,
                                   size_t stride,
                                   size_t repeat) {
    size_t tid = blockIdx.x * blockDim.x + threadIdx.x;
    if (tid >= n) return;

    float sum = 0.0f;
    size_t idx = tid;
    for (size_t i = 0; i < repeat; ++i) {
        sum += data[idx];
        idx = (idx + stride) % n;  // a tömbön belül körbejárva
    }
    // Az összeget visszaírjuk, így az olvasások nem tűnnek el
```

```
    output[tid] = sum;
}

int main() {
    // Eszközinformációk kiírása
    cudaDeviceProp prop;
    CHECK_CUDA(cudaGetDeviceProperties(&prop, 0));
    std::printf("GPU: %s\n", prop.name);
    std::printf("Compute capability: %d.%d\n", prop.major, prop.minor);
    std::printf("L2 cache: %zu MiB\n", prop.l2CacheSize / (1024 * 1024));
    std::printf("Shared mem per SM: %zu KiB\n",
                prop.sharedMemPerMultiprocessor / 1024);
    std::printf("Multiprocessorok: %d\n", prop.multiProcessorCount);

    // A mérési pontok: munkahalmaz méretek (float elemek száma)
    const size_t sizes[] = {
        64 * 1024,      // 256 KiB (0.25 MiB)
        256 * 1024,     // 1 MiB
        1024 * 1024,    // 4 MiB
        4 * 1024 * 1024, // 16 MiB
        16 * 1024 * 1024 // 64 MiB
    };
    const size_t strides[] = {1, 8, 32};
    const size_t repeat = 1024;

    const size_t block = 256;

    for (size_t n : sizes) {
        // Foglalás host és device oldalon
        std::vector<float> h_data(n);
        for (size_t i = 0; i < n; ++i) h_data[i] = static_cast<float>(i) * 0.001f;

        float* d_data;
        float* d_output;
        CHECK_CUDA(cudaMalloc(&d_data, n * sizeof(float)));
        CHECK_CUDA(cudaMalloc(&d_output, n * sizeof(float)));
        CHECK_CUDA(cudaMemcpy(d_data, h_data.data(), n * sizeof(float),
                              cudaMemcpyHostToDevice));

        size_t grid = (n + block - 1) / block;

        // Warm-up: egyszer lefuttatjuk, hogy a cache-ek és az órajelek beálljanak
        cache_probe_kernel<<<grid, block, 0, 0>>>(d_data, d_output, n, 1, 10);
        CHECK_CUDA(cudaDeviceSynchronize());

        for (size_t stride : strides) {
            const int num_iter = 10;
            cudaEvent_t start, stop;
            CHECK_CUDA(cudaEventCreate(&start));
            CHECK_CUDA(cudaEventCreate(&stop));

            float total_ms = 0.0f;
            for (int iter = 0; iter < num_iter; ++iter) {
                CHECK_CUDA(cudaEventRecord(start));
                cache_probe_kernel<<<grid, block, 0, 0>>>(d_data, d_output,
                                                          n, stride, repeat);
                CHECK_CUDA(cudaEventRecord(stop));
                CHECK_CUDA(cudaEventSynchronize(stop));
                float ms = 0.0f;
                CHECK_CUDA(cudaEventElapsedTime(&ms, start, stop));
                total_ms += ms;
            }
            float avg_ms = total_ms / num_iter;
```

```
            // Az átlagos sávszélesség számítása
            size_t bytes_read = n * sizeof(float) * repeat;
            double bw_gbps = (double)bytes_read / (avg_ms * 1e-3) / 1e9;

            std::printf("n=%7zu (%.2f MiB), stride=%2zu, idő=%.3f ms, "
                        "sávszélesség=%.1f GB/s\n",
                        n, (double)(n * sizeof(float)) / (1024.0 * 1024.0),
                        stride, avg_ms, bw_gbps);

            CHECK_CUDA(cudaEventDestroy(start));
            CHECK_CUDA(cudaEventDestroy(stop));
        }

        CHECK_CUDA(cudaFree(d_data));
        CHECK_CUDA(cudaFree(d_output));
    }

    return 0;
}
```

A kódban megfigyelhető a kötelező hibakezelés, a warm-up futtatás, a CUDA events időmérés, valamint a sávszélesség kiszámítása. A kernel launch alakja `<<<grid, block, 0, 0>>>`, ahol a grid és block dimenzió 1D. A mérési loopban minden stride-hoz 10 ismétlést futtatunk, és az átlagot számoljuk. A szórás kiszámítása a példában elhagyva, de a teljes laborban a mérési naplóban fel kell tüntetni a szórást vagy percentilist is a könyv konvenciója szerint.

15.1.7 Fordítás és futtatás

A fordítás parancsa:

```
nvcc -O3 -std=c++17 -arch=sm_80 -o cache_probe cache_probe.cu
```

A futtatás egyszerű:

```
./cache_probe
```

A program a standard kimenetre írja az eszközinformációkat és a mérési eredményeket. A laborban a kimenetet mérési naplóként kell elmenteni.

15.1.8 Mérési napló (illesztratív)

Az alábbi táblázat egy lehetséges kimenetet mutat. A számok illusztratívak, és a laborban a saját GPU-n mért értékekkel kell helyettesíteni. A mérési naplóban a GPU típusát, a drivert, a CUDA verziót és az operációs rendszert külön fel kell tüntetni.

Méret (float)	Méret (MiB)	Stride	Átlag idő (ms)	Sávszélesség (GB/s)
65536	0.25	1	1.23	218.4
65536	0.25	8	1.85	145.2
65536	0.25	32	3.10	86.7
262144	1.0	1	1.45	740.9
262144	1.0	8	2.20	488.5
262144	1.0	32	3.80	282.9
1048576	4.0	1	2.10	2044.6
1048576	4.0	8	3.50	1226.8

Méret (float)	Méret (MiB)	Stride	Átlag idő (ms)	Sávszélesség (GB/s)
1048576	4.0	32	6.20	692.6
4194304	16.0	1	8.90	1937.9
4194304	16.0	8	12.40	1390.5
4194304	16.0	32	21.00	821.1
16777216	64.0	1	35.60	1930.2
16777216	64.0	8	49.00	1402.7
16777216	64.0	32	82.00	838.0

A táblázatból látható, hogy a kis méreteknél (0.25 MiB és 1 MiB) a sávszélesség alacsonyabb az L2 cache találati arány miatt? Valójában a kis méretnél a munkahalmaz az L2-ben elfér, de a sávszélesség értéke a fenti példában 218 GB/s, ami alacsonyabb, mint a nagyobb méretnél mért ~2000 GB/s. Ez azért van, mert a kis méretnél a kernel futásideje rövid, és a mérési overhead nagyobb arányú. A laborban a `repeat` értékét úgy kell megválasztani, hogy a futásidő minden méretnél elegendően hosszú legyen (például > 1 ms), és a mérési overhead elhanyagolható legyen. A fenti táblázat illusztratív, és a `repeat = 1024` érték mellett a kis méreteknél a futásidő túl rövid lehet. A laborban a `repeat` értékét növelni kell, vagy a mérési módszert úgy alakítani, hogy a kernel többszöri indítása helyett egy kernelben több ismétlést végezzen. A helyes méréshez a laborban a mérőszámot úgy kell beállítani, hogy a futásidő minden konfigurációban legalább néhány milliszekundum legyen.

A lényegi megfigyelés azonban jól látható: ahogy a munkahalmaz nagyobb lesz, mint az L2 cache (itt 40 MiB), a sávszélesség leesik a global memória sávszélességére, és a stride növelése minden esetben rontja a hatékonyságot a cache sor kihasználtság csökkenése miatt.

15.1.9 Várt eredmény és értelmezés

A várt eredmény az, hogy a kis munkahalmazoknál az L2 cache tartja az adatokat, így a kernel teljesítményét az L2 sávszélessége és a cache találati arány határozza meg. Amikor a munkahalmaz meghaladja az L2 kapacitását, a találati arány drasztikusan csökken, és a global memória sávszélessége válik a szűk keresztmetszetté. A stride növelése pedig azt mutatja, hogy a cache sorok kihasználtsága romlik: a memóriarendszer több szektort tölt be, mint amennyi adatot a warp ténylegesen használ, ezért az effektív sávszélesség csökken.

A mérési naplóban a mért és várt eredményt külön kell választani. A fenti táblázat illusztratív, a laborban a tényleges mérési adatokat kell rögzíteni, és a várt tendenciákkal összevetni.

15.1.10 Tipikus hibák és helyes megoldások

A cache-viselkedés mérése során több hiba is előfordulhat. Az alábbiakban a leggyakoribbakat foglaljuk össze.

Hiba: A munkahalmaz méretét nem változtatják, így a cache hatása nem látható. **Megoldás:** Több, logaritmikus skálán elosztott méretet használjunk, például az L2 cache méretének 1/8-ától a többszöröséig.

Hiba: A mérést egyszer futtatják, nincs warm-up vagy ismétlés. **Megoldás:** A könyv konvenciója szerint warm-up, majd legalább 5–10 ismétlés szükséges. Az átlag mellett a szórást vagy percentilist is közölni kell.

Hiba: A kernelben a fordító kioptimalizálja az olvasást, mert az összeg nincs felhasználva. **Megoldás:** Az összeget mindig vissza kell írni egy kimeneti tömbbe, ahogy a példában is tesszük.

Hiba: A stride-ot túl nagyra választják, és a mérés a global memória sávszélességét méri, nem a cache-t. **Megoldás:** Olyan stride-okat használjunk, amelyek a cache sor méretének (32 bájt) egész számú többszörösei, de ne legyenek túl nagyok, hogy a coalescing és a cache line kihasználtság hatása szétválasztható legyen.

Hiba: A cache line méretét összekeverik a warp méretével. **Megoldás:** A warp 32 thread; a cache line/szektor 32 bájt (L1) vagy 128 bájt (L2). Ezek különböző fogalmak, bár mindkettő a 32-es számhoz kapcsolódik.

Hiba: A mérési eredményeket abszolút igazságként kezelik anélkül, hogy a környezetet rögzítenék. **Megoldás:** Minden mérésnél rögzíteni kell a GPU modelljét, a drivert, a CUDA verziót, az órajelet és a háttérterhelést. Csak így reprodukálható az eredmény.

15.1.11 Önellenőrző kérdés

1. Mi történik a találati aránnyal, ha a munkahalmaz nagyobb, mint az L2 cache kapacitása? Indokolja a választ a hierarchikus memória modell alapján.
2. Miért rontja a nagy stride a cache line kihasználtságot, és hogyan befolyásolja ez a hatékony sávszélességet?
3. Milyen következménye van annak, ha a shared memory foglalás csökkenti az L1 cache rendelkezésre álló területét egy SM-en belül?

15.1.12 Összegzés és átvezetés

Ebben az alfejezetben bemutattuk a GPU L1 és L2 cache-ének szerepét, a találati arány fogalmát, a kapacitások jelentőségét, valamint a cache sor és asszociativitás alapjait. A labor méréssel igazolta, hogy a munkahalmaz mérete és a hozzáférési minta milyen mértékben befolyásolja a teljesítményt. A következő alfejezetben a cache hint-ekkel és a hozzáférési szándék explicit jelzésével foglalkozunk, amelyek lehetővé teszik a cache viselkedésének finomabb szabályozását.

15.2 Cache hint-ek és hozzáférési szándék

A modern GPU-k memóriahierarchiájának hatékony használata nem ér véget a coalescing elérésnél vagy a shared memory tilingnél. Amikor a kernel globális memóriát olvas vagy ír, a hardver automatikusan döntést hoz arról, hogy a tranzakció mely cache-szinteket érintse, milyen eviction-politikával kerüljön be az L1, illetve L2 tárolóba, és a gyorsítótárbeli élettartamát hogyan kezelje. Ezt a döntést a CUDA programozó explicit módon befolyásolhatja a load/store cache operátorok, röviden *cache hint*-ek segítségével. Ez az alfejezet ezeket a mechanizmusokat tárgyalja, bemutatja a gyakorlati használatukat, valamint egy ellenőrizhető labor keretében méri a hatásukat.

Előfeltétel a korábbi alfejezet (L1 és L2 cache) ismerete, amely az L1/L2 cache felépítését, a memóriahierarchiát és az alapvető cache-viselkedést tárgyalta. Az itt bemutatott cache operátorok arra épülnek, hogy a programozó tudatában van annak, mikor érdemes egy adatot gyorsítótárban tartani, és mikor célszerűbb minimalizálni a cache-szennyezést.

15.2.1 A load/store cache operátorok fogalma

A CUDA C++ programozási modell magas szinten három fő memóriateret különböztet meg: globális, shared és lokális memóriát (a konstans és textúra memóriát ide nem értve). A globális memória elérésekor minden load és store utasítás a memóriahierarchián keresztül hajtódik végre. A hardver a kezelt adatot az L1 és L2 cache-en keresztül mozgatja, és alapértelmezés szerint a *cache all* (ca) politikát alkalmazza: a betöltés minden szinten gyorsítótárba kerül, és a cache-ben maradhat, amíg a kapacitás engedi. Ez a viselkedés a legtöbb feladatra megfelelő, de számos hozzáféréstípus esetén nem optimális.

A *load/store cache operátorok* olyan utasításszintű jelzések, amelyekkel a programozó a Hardvert arról tájékoztatja, hogy az adott adathozzáférést milyen gyorsítótárazási szándékkal hajtsa végre. A PTX ISA szintjén ezeket a cache eviction-prioritási és cache-szintű operátorokat a `ld` és `st` utasítások módosítóiként adják meg, például:

- `ld.global.ca` – cache all
- `ld.global.cg` – cache global (csak L2)
- `ld.global.cs` – cache streaming (evict-first)
- `ld.global.lu` – last use

CUDA C++-ban ezeket a legegyszerűbben a beépített intrinsic függvényekkel érhetjük el, amelyek a `cuda_runtime.h` fejlécben definiáltak. A load oldalon négy fő intrinsic áll rendelkezésre:

Intrinsic	PTX megfelelő	Jelentés
`__ldca(ptr)`	`ld.global.ca`	Cache all: L1 és L2 gyorsítótárba helyezés, normál eviction.
`__ldcg(ptr)`	`ld.global.cg`	Cache global: csak L2 gyorsítótárba helyezés, L1-et kihagyja.
`__ldcs(ptr)`	`ld.global.cs`	Cache streaming: L1 és L2 gyorsítótárba helyezés evict-first prioritással, azaz hamarabb kiüríthető.
`__ldlu(ptr)`	`ld.global.lu`	Last use: a betöltés után a cache-sor érvényteleníthető, ideális egyszeri olvasásra.

A store oldalon a megfelelő intrinsic-ek:

Intrinsic	PTX megfelelő	Jelentés
`__stcg(ptr, value)`	`st.global.cg`	Store csak L2 cache-be, L1-et kihagyja.
`__stcs(ptr, value)`	`st.global.cs`	Store streaming evict-first prioritással.
`__stwb(ptr, value)`	`st.global.wb`	Store write-back (alapértelmezett, hasonló a ca-hoz).

Intrinsic	PTX megfelelő	Jelentés
`__stwt(ptr, value)`	`st.global.wt`	Store write-through, azaz a memória azonnal frissül, cache csak olvasásra.

Ezek az intrinsic-ek a `__device__` függvényeken belül használhatók, és a compiler a megfelelő PTX utasításra fordítja őket. Fontos, hogy a használatuk nem változtat a program funkcionalitásán – csak a teljesítményre van hatással. A cache hint-ek tehát *nem* befolyásolják a memória konzisztenciáját vagy a program szemantikáját, csupán a gyorsítótárazás hatékonyságát optimalizálják.

15.2.2 A ca, cg, cs és lu operátorok részletesen

A négy alapvető load operátor a gyorsítótárazási szándékot fejezi ki. Lássuk ezeket egyenként, és értsük meg, mikor melyiket érdemes választani.

15.2.3 `__ldca` – cache all

Ez az alapértelmezett viselkedés. A betöltés az L1 és L2 cache-en keresztül történik, és a cache-sor normál prioritással maradhat a gyorsítótárban mindaddig, amíg a kapacitás és a replikációs politika megengedi. Ez a legjobb választás olyan adatoknál, amelyeket a kernel többször olvas, vagy amelyekre más szálak is hivatkoznak (pl. egy mátrix sorának újrahasználata). A `__ldca` használata explicit módon megegyezik a sima pointer-dereferálással (pl. `*ptr`), de a hívás dokumentálja a szándékot.

15.2.4 `__ldcg` – cache global (csak L2)

Ez a variáns kihagyja az L1 cache-t, és a betöltést közvetlenül az L2 cache-en keresztül hajtja végre. Ez akkor előnyös, ha a kernel tudja, hogy az adatot nem fogja újra használni ugyanazon a blokkon belül (például egy streamelt bemenet első és egyetlen olvasása), mert így nem szennyezi az L1 gyorsítótárat, ahol más, többször használt adatoknak kell hely. Az L2 gyorsítótár viszont továbbra is gyorsítótárazza az adatot, ami hasznos lehet, ha más blokkok is hozzáférnek ugyanahhoz a memóriaterülethez. A `__ldcg` gyakran előnyös olyan esetekben, amikor a kernel nagy méretű, egyszer használt tömböt dolgoz fel, és az L1 kapacitása korlátozott.

15.2.5 `__ldcs` – cache streaming (evict-first)

A streaming operátor a betöltést evict-first prioritással helyezi el a cache-ben. Ez azt jelenti, hogy a cache-sor a legelső jelölt lesz a kiürítésre, amikor helyre van szükség. A cél az, hogy a gyorsítótárban ne foglaljon hosszú távú helyet, és így az újabb adatoknak gyorsabban legyen helyük. Ez ideális olyan streamelt adatoknál, amelyeket csak egyszer olvasunk, és nincs szükség a későbbi újrahasználatra. A `__ldcs` mind az L1, mind az L2 cache-re vonatkozik, bár az L2 viselkedése hardverfüggő lehet. A streaming load segíthet csökkenteni a cache-szennyezést, és javíthatja az L2 találati arányt más adatok számára.

15.2.6 `__ldlu` – last use

A last use operátor azt jelzi, hogy ez a betöltés az utolsó hozzáférés az adott cache-vonalhoz. A hardver ezután érvényteleníthet a cache-sort, és az azonnal felszabaduló helyet más adatok használhatják. Ez erősebb streaming-politikát jelent, mint a `__ldcs`, mert nemcsak evict-first prioritást ad, hanem explicit módon jelzi, hogy az adatra többé nincs szükség. A `__ldlu` különösen hasznos olyan algoritmusokban, ahol egy nagy tömböt pontosan egyszer olvasunk (pl. egy szekvenciális feldolgozás utolsó iterációjában), vagy amikor a kernel tudja, hogy az adott cache-vonalat más szálak sem fogják többé használni.

A store oldali `__stcg`, `__stcs`, `__stwb`, `__stwt` operátorok analóg módon működnek. A `__stcg` kihagyja az L1 cache-t, és az L2 cache-be ír (write-back), ami akkor előnyös, ha a kernel által írt adatokat más blokkok olvashatják, de a saját blokk nem fogja újra olvasni. A `__stcs` streaming store evict-first prioritást ad, míg a `__stwt` write-through módban azonnal a globális memóriába ír, és a cache-ben nem tartja meg a módosított sort. Ez utóbbi ritkán előnyös, de bizonyos I/O-jellegű műveleteknél vagy atomi láthatósági követelményeknél hasznos lehet.

A cache operátorok tehát a *hozzáférési szándék* explicit kifejezését teszik lehetővé. A programozó azt mondja meg a hardvernek, hogy az adott memóriahozzáférést milyen újrahasználati mintára számítja. Ez két fő kategóriára bontható:

- **Streaming (átmenő) adat**: olyan adat, amelyet a kernel egyszer vagy nagyon rövid időn belül többször olvas, de utána nem szükséges a cache-ben tartani. Ilyenkor a `cs` vagy `lu` operátorok alkalmasak, és a `cg` is jó lehet, ha az L1-et nem akarjuk szennyezni.
- **Perzisztens (tartós) adat**: olyan adat, amelyet a kernel többször, akár különböző fázisokban is használ, vagy amelynek az L2 gyorsítótárban maradása más blokkok számára is hasznos. Ilyenkor a `ca` (alapértelmezett) vagy a `cg` (ha L1 nem kell) a jó választás.

A perzisztens adat fogalma a következő alfejezetben (L2 persistence és access policy) az L2 persistence és access policy kapcsán tovább mélyül, ahol a CUDA lehetővé teszi az L2 cache egy részének dedikált lefoglalását is. Itt a cache operátorok csak az utasításszintű jelzést adják, de ez is jelentős hatással lehet a teljesítményre.

15.2.7 Gyakorlati példa: vektoros stream-feldolgozás cache hint-ekkel

Tekintsünk egy egyszerű CUDA-kernelt, amely egy nagy float-tömb elemeit beolvassa, és egy másik tömbbe írja őket (pl. minden elemhez hozzáad egy konstanst). Ez a művelet tipikusan streaming-jellegű: a bemeneti tömböt egyszer olvassuk, a kimeneti tömböt egyszer írjuk. Az alapértelmezett `ca` cache-politika azt eredményezheti, hogy a bemeneti adatok az L1 és L2 gyorsítótárban is tárolódnak, annak ellenére, hogy nincs rájuk további szükség. Ez csökkenti a cache hasznos kapacitását, és ronthatja az L1 találati arányt olyan más adatok esetében, amelyeket a kernel többször használ.

A következő példa bemutatja, hogyan használhatjuk a cache operátorokat a streamelés optimalizálására. A kernel két változatban készül: az egyik az alapértelmezett load-ot használ, a másik a `__ldcs` és `__stcs` operátorokat alkalmazza. A méréshez a kernel futásidejét és a memória sávszélességét hasonlítjuk össze.

```
// File: cache_hint_demo.cu
// Nyelv: CUDA C++ (C++17)
// Szemléltető példa, nem valós eszközön futó kód.

#include <cuda_runtime.h>
#include <stdio.h>
```

```cpp
#include <chrono>

// Saját hibamakró (a könyv konvenciója szerint)
#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err = (call);                                           \
        if (err != cudaSuccess) {                                           \
            fprintf(stderr, "CUDA hiba: %s:%d: %s\n", __FILE__, __LINE__,  \
                    cudaGetErrorString(err));                               \
            cudaDeviceReset();                                              \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

// 1. Kernel: alapértelmezett cache-elés (ca)
__global__ void stream_add_ca(const float* __restrict__ in,
                              float* __restrict__ out,
                              float scale,
                              int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        float val = in[idx];       // alapértelmezett ld.global.ca
        out[idx] = val * scale;    // alapértelmezett st.global.wb
    }
}

// 2. Kernel: streaming cache hint-ekkel (cs)
__global__ void stream_add_cs(const float* __restrict__ in,
                              float* __restrict__ out,
                              float scale,
                              int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        float val = __ldcs(&in[idx]);   // streaming load
        __stcs(&out[idx], val * scale); // streaming store
    }
}

// 3. Kernel: cache global (csak L2) hint-ekkel (cg)
__global__ void stream_add_cg(const float* __restrict__ in,
                              float* __restrict__ out,
                              float scale,
                              int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        float val = __ldcg(&in[idx]);   // L2-only load
        __stcg(&out[idx], val * scale); // L2-only store
    }
}

// Segédfüggvény a méréshez
template <typename F>
float measure_ms(F&& kernel_func, int blocks, int threads,
                 cudaStream_t stream = 0) {
    // warm-up
    kernel_func<<<blocks, threads, 0, stream>>>();
    CUDA_CHECK(cudaDeviceSynchronize());

    auto start = std::chrono::high_resolution_clock::now();
    for (int i = 0; i < 5; ++i) {
        kernel_func<<<blocks, threads, 0, stream>>>();
    }
    CUDA_CHECK(cudaDeviceSynchronize());
```

```
    auto end = std::chrono::high_resolution_clock::now();

    std::chrono::duration<float, std::milli> ms = (end - start) / 5;
    return ms.count();
}

int main() {
    // GPU konfiguráció (helyőrző, valós mérésnél rögzítendő)
    // NVIDIA GPU típus, driver, CUDA verzió: ld. a mérési naplót
    int device = 0;
    CUDA_CHECK(cudaSetDevice(device));
    cudaDeviceProp props;
    CUDA_CHECK(cudaGetDeviceProperties(&props, device));
    printf("GPU: %s\n", props.name);
    printf("Compute capability: %d.%d\n", props.major, props.minor);

    const int N = 1 << 24;  // 16M elem, float => 64 MB / tömb
    const size_t bytes = N * sizeof(float);

    float *d_in, *d_out;
    CUDA_CHECK(cudaMalloc(&d_in, bytes));
    CUDA_CHECK(cudaMalloc(&d_out, bytes));

    // Host inicializálás (helyőrző adatok)
    float* h_in = new float[N];
    for (int i = 0; i < N; ++i) h_in[i] = static_cast<float>(i % 1024);
    CUDA_CHECK(cudaMemcpy(d_in, h_in, bytes, cudaMemcpyHostToDevice));

    float scale = 2.0f;
    int threads = 256;
    int blocks = (N + threads - 1) / threads;

    // Mérés
    float t_ca = measure_ms([&](){ stream_add_ca<<<blocks, threads>>>(d_in, d_out, scale, N); },
                            blocks, threads);
    float t_cs = measure_ms([&](){ stream_add_cs<<<blocks, threads>>>(d_in, d_out, scale, N); },
                            blocks, threads);
    float t_cg = measure_ms([&](){ stream_add_cg<<<blocks, threads>>>(d_in, d_out, scale, N); },
                            blocks, threads);

    // Ellenőrzés: néhány elem
    float h_out[10], h_out_ref[10];
    CUDA_CHECK(cudaMemcpy(h_out, d_out, 10 * sizeof(float), cudaMemcpyDeviceToHost));
    for (int i = 0; i < 10; ++i) h_out_ref[i] = h_in[i] * scale;
    for (int i = 0; i < 10; ++i) {
        if (h_out[i] != h_out_ref[i]) {
            printf("Hiba az %d. elemnél: %f != %f\n", i, h_out[i], h_out_ref[i]);
            return -1;
        }
    }

    printf("Futásidő CA (alap): %.3f ms\n", t_ca);
    printf("Futásidő CS (streaming): %.3f ms\n", t_cs);
    printf("Futásidő CG (L2 only): %.3f ms\n", t_cg);

    // Sávszélesség számítás
    double total_bytes = 2.0 * bytes; // beolvasás + kiírás
    printf("Sávszélesség CA: %.2f GB/s\n", total_bytes / (t_ca * 1e6));
    printf("Sávszélesség CS: %.2f GB/s\n", total_bytes / (t_cs * 1e6));
    printf("Sávszélesség CG: %.2f GB/s\n", total_bytes / (t_cg * 1e6));

    CUDA_CHECK(cudaFree(d_in));
    CUDA_CHECK(cudaFree(d_out));
```

```
    delete[] h_in;
    return 0;
}
```

A kód futtatása előtt a build parancs a következő (CUDA 12.x, Ubuntu 22.04 feltételezéssel, de a konkrét verziók a mérési naplóban rögzítendők):

```
nvcc -O3 -std=c++17 -arch=sm_80 -o cache_hint_demo cache_hint_demo.cu
```

Futtatás:

```
./cache_hint_demo
```

A mérési napló sablon:

```
GPU: NVIDIA A100-SXM4-40GB
Compute capability: 8.0
CUDA verzió: 12.4
Driver: 550.54.15
OS: Ubuntu 22.04 LTS
Warm-up: 1 iteráció
Mérési ismétlések: 5
Precision: FP32

Megfigyelt futásidők:
- CA (alap): 12.34 ms, sávszélesség: 0.41 TB/s
- CS (streaming): 11.87 ms, sávszélesség: 0.43 TB/s
- CG (L2 only): 11.98 ms, sávszélesség: 0.42 TB/s

Várt eredmény (kvalitatív):
A streaming (CS) és L2-only (CG) hint-ek enyhe javulást hoznak, mert csökkentik az L1 cache
↪  szennyezését. A mért javulás a konkrét GPU-tól és a kernel méretétől függ; kis tömböknél a
↪  hatás minimális lehet.
```

A fenti példa jól szemlélteti a cache operátorok használatát. A tényleges mérési eredményeket mindig az adott hardveren kell elvégezni, mert a gyorsítótárazási politika hardver- és driverfüggő. A példában használt 64 MB-os tömb egy A100-as GPU L2 cache-ének (40 MB) mérete fölé esik, ami lehetővé teszi, hogy a streaming hint-ek csökkentsék az L2 szennyezést is. Kisebb tömböknél (amelyek belefférnek az L2-be) a különbség elhanyagolható.

15.2.8 Streamelés és perzisztens adat a gyakorlatban

A cache operátorok alkalmazásának kulcsa annak megértése, hogy az adott adatot hányszor, milyen időskálán és milyen szélességben használjuk. A *streaming* és *perzisztens* adat fogalma itt kapcsolódik a hozzáférési mintához.

Streaming adat esetén a kernel egy nagy adatfolyamot dolgoz fel, amelyet egyszer olvas vagy ír. Ilyen például a vektor-művelet, a redukció első fázisa, vagy egy képfeldolgozó pipeline input/output adatai. Itt a cél az, hogy a cache ne ragaszkodjon az adathoz, mert az újrahasználat valószínűsége közel nulla. A `__ldcs`, `__ldlu`, `__stcs` operátorok kifejezik ezt a szándékot, és a hardver agresszívan kiürítheti a cache-sorokat, helyet adva az éppen aktuális adatoknak. Ez növelheti az effektív sávszélességet, különösen akkor, ha a kernel emellett más, kis méretű perzisztens adatot is használ (pl. koordináta-tömböt egy szimulációban), amelyet érdemes cache-ben tartani.

Perzisztens adat esetén az adatot a kernel többször olvassa, vagy több blokk is hozzáfér, és a gyorsítótárban maradása csökkenti a globális memória-forgalmat. Ilyen például egy szűrő kernel súlytömbje, egy táblázat, vagy egy olyan mátrix-rész, amelyet a blokk iterációban újrahasznál.

Ilyenkor az alapértelmezett `ca` vagy `cg` a megfelelő, mert a cache-soroknak maradniuk kell. A `cg` előnyös lehet, ha az L1 nem használható jól a perzisztens adat számára (például a mérete miatt), és elég, ha az L2-ben marad.

A két kategória között átmenet is létezik. Például egy stencil-számításnál a halo-régiót többször is olvassuk, míg a belső pontokat egyszer. Ilyenkor a halo-régióra `ca`, a belső pontokra `cs` használható. A cache operátorok ilyen finomhangolása csak profilozás után ajánlott, mert a hardver heurisztikája gyakran jól működik.

15.2.9 Hibák és helyes megoldások

A cache operátorok használata során gyakori hibák:

1. **Minden hozzáférést streamelőre állítani**: Ha a kernel valóban újrahasznál egy adatot, de a programozó tévesen `cs`-t vagy `lu`-t ad meg, a cache-sorok idő előtt kiürülnek, és a későbbi hozzáférés globális memória-traffichoz vezet. Ez súlyos teljesítményromlást okozhat. **Helyes megoldás**: Csak akkor használjunk streaming hint-et, ha bizonyítottan egyszeri hozzáférésről van szó. Profilozzuk a cache találati arányt.

2. **A store oldal elhanyagolása**: Sokan csak a load oldalt optimalizálják, és a store-t alapértelmezett `wb`-ként hagyják. Azonban egy streaming store esetén a `__stcs` vagy `__stcg` ugyanolyan fontos, mert a write-back cache is szennyezheti az L1-et és L2-t. **Helyes megoldás**: A load és store hint-eket együttesen válasszuk meg.

3. **Nem mérünk baseline-t**: A cache hint-ek hatása erősen függ a GPU-tól és a környezettől. Anélkül, hogy az alapértelmezett `ca` viselkedést megmérnénk, nem tudjuk, hogy a hint valóban javít-e. **Helyes megoldás**: Minden variánsból készítsünk mérést, és hasonlítsuk össze a futásidőt, sávszélességet és cache-metrikákat.

4. **Túlzott finomhangolás**: A cache operátorok alkalmazása minden memóriahozzáférésre bonyolulttá és olvashatatlanná teszi a kódot, és a hardver heurisztikái gyakran hasonló eredményt hoznak. **Helyes megoldás**: Először a nagy, jól elkülöníthető adatfolyamokra alkalmazzuk a hint-eket, és csak profilozás után finomítsunk tovább.

5. **Rossz architektúrán alapuló várakozás**: A cache operátorok viselkedése compute capability-től függ. Például a `__ldcg` az L1 kihagyását jelenti, de a régebbi architektúrákon (pl. Kepler előtt) az L1 és shared memory közös hardvert használt, így a hatás eltérő lehet. **Helyes megoldás**: A dokumentációban megadott architektúra-specifikus viselkedést ellenőrizni kell (verziófüggő, ezért a fejlesztői dokumentációt kell figyelni).

6. **Helytelen indexelés miatti hibás hint**: A cache operátorok használata nem változtatja meg a memóriacímzést, de ha a programozó rossz pointert ad át, a hiba ugyanúgy jelentkezik. A hibakereséshez használjunk `cuda-memcheck`-et. **Helyes megoldás**: A cache hint-ek bevezetése előtt ellenőrizzük a kernel helyességét hagyományos eléréssel, majd csak ezután térjünk át a hint-ekre.

A helyes megoldások közös vonása a fokozatosság és a mérés. A cache operátorok egy gyors optimalizálási eszköz, de nem helyettesítik az algoritmusszintű optimalizálást (pl. coalescing, tiling).

15.2.10 Ellenőrzés és profilozás

A cache operátorok hatását a legjobban a NVIDIA Nsight Compute profilozóval lehet ellenőrizni. A következő metrikák relevánsak:

- `l1tex__t_sectors_pipe_lsu_mem_global_op_ld` – L1 load szektorok száma
- `l1tex__t_sectors_pipe_lsu_mem_global_op_st` – L1 store szektorok
- `lts__t_sectors_op_read` – L2 read szektorok
- `lts__t_sectors_op_write` – L2 write szektorok
- `dram__bytes_read.sum` / `dram__bytes_write.sum` – DRAM forgalom
- `l1tex__t_tag_requests` és `l1tex__t_misses` – L1 találati arány
- `lts__t_tag_requests` és `lts__t_misses` – L2 találati arány

A mérés során össze kell hasonlítani az alapértelmezett, a streaming és a cache global változatok metrikáit. A `__ldcs` és `__stcs` használata esetén várhatóan csökken az L1 cache-ben maradó sorok száma, és javulhat az L1 találati arány a perzisztens adatok számára, miközben a DRAM forgalom változatlan marad (mert az adatok mindenképpen beolvasásra kerülnek).

A mérési naplóban rögzíteni kell a GPU modelljét, a driver verzióját, a CUDA verzióját, a compute capability-t, a futtatási környezetet, és minden kernel-variánsra a fenti metrikákat, valamint a futásidőt és a számított sávszélességet. A várható eredmény az, hogy a streaming hint-ek csökkentik az L1 cache szennyezést, és enyhe futásidő-javulást hoznak nagy tömbök esetén, de a hatás mértéke hardverfüggő.

15.2.11 Átvezetés

Ebben az alfejezetben megismertük a load/store cache operátorokat, a ca, cg, cs és lu jelentését, valamint a streaming és perzisztens adat fogalmát. Láttuk, hogyan lehet ezeket CUDA C++ intrinsic-ekkel használni, és hogyan mérhetjük a hatásukat.

A következő alfejezet (L2 persistence és access policy) arra épít, hogy az L2 gyorsítótár nemcsak utasításszintű jelzésekkel, hanem hozzáférési ablakok és perzisztencia-beállításokkal is programozható. Az ott tárgyalt `cudaStreamSetAttribute` és az L2 access policy lehetővé teszi, hogy a perzisztens adatokat dedikált L2 területen tartsuk, tovább finomítva a memória-hierarchia kihasználását. Az itt bemutatott cache hint-ek képezik az alapot ahhoz, hogy megértsük, miért van szükség ilyen magasabb szintű vezérlésre.

Végül emlékeztetjük az olvasót, hogy a cache operátorok használata mindig az adott GPU-hoz és CUDA-verzióhoz kötött. A viselkedésük részletes leírását a fejlesztői dokumentációban kell ellenőrizni, mert a hardveres implementáció architektúránként eltérhet. A laborokban használt kódok szemléltető jellegűek, és valós futtatáshoz a megadott build- és futtatási konvenciók szerint kell őket adaptálni.

Önellenőrző kérdés: Adott egy kernel, amely egy nagy, 512 MB méretű képet dolgoz fel úgy, hogy minden pixelt pontosan egyszer olvas be, és az eredményt egy másik képbe írja. Melyik cache operátor kombinációt választaná, és miért? Hogyan mérné meg a választás helyességét?

15.3 L2 persistence és access policy

Az L2 cache a modern GPU-architektúrák egyik legértékesebb, ám gyakran alábecsült erőforrása. Az L2 nem csupán a globális memória elérését gyorsítja, hanem a streaming multiprocessorok (SM-ek) közötti adatmegosztást is szolgálja. Amikor egy kernel olvassa a globális memóriát, az

adat először az L2-be kerül, és csak onnan az SM-ek L1 cache-eibe. Ha ugyanazt az adatot több kernel, több stream vagy akár több GPU kontextus használja, akkor az L2-ben való tartós jelenlét jelentős sávszélesség-megtakarítást eredményezhet. A hagyományos cache-kezelés azonban a legkevésbé használt (LRU) vagy más heurisztika szerint szabadítja fel a cache-vonalakat, így egy másik, nagy memóriájú kernel könnyen kilakoltathatja azokat az adatokat, amelyekre a következő kernelnek még szüksége lenne.

A CUDA 11.0 óta elérhető **L2 persistence** mechanizmus lehetővé teszi, hogy a programozó kijelöljön egy memóriaterületet, amelynek cache-vonalait az L2 cache perzisztens részében tartsa. Ez azt jelenti, hogy az adott területhez tartozó cache-vonalakat a hardver nem távolítja el normál cache-nyomás hatására, így azok túlélik a kernelhatárokat, és a következő kernel indításánál is elérhetők maradnak. A mechanizmus kulcsa a **cudaAccessPolicyWindow** struktúra, amely egy memóriatartományt és a hozzá tartozó hozzáférési tulajdonságokat írja le. Ezt a window-t egy adott stream-hez rendeljük, így az adott streamen futó kernelek memória-hozzáférései a megadott policy szerint kezelődnek.

Ebben az alfejezetben bemutatjuk a perzisztens L2 használatát, a `cudaAccessPolicyWindow` konfigurálását, és azt, hogyan őrizhető meg az adat több kernel között. A gyakorlati részben egy ellenőrizhető labor segítségével mérjük a perzisztencia hatását, és tipikus hibákat is elemzünk.

15.3.1 1. Az L2 cache perzisztens használatának elve

A GPU L2 cache-ének mérete általában néhány MiB-től néhány tíz MiB-ig terjed (például az NVIDIA A100 40 MiB L2-vel rendelkezik, az RTX 3090 6 MiB-vel). A perzisztencia beállításával az L2 egy részét fenntarthatjuk olyan adatok számára, amelyeket többször, esetleg több kernel futtatása során is olvasunk. Ez különösen akkor hasznos, ha:

- egy nagy adathalmaz ismételten szükséges (például súlyok iteratív optimalizálásnál),
- több stream konkurensen fut, és az egyik stream adatait szeretnénk megóvni a többi stream cache-szennyezésétől,
- egy producer–fogyasztó lánc esetén a közbenső adatoknak a kernelhatárok között is L2-ben kell maradniuk.

Fontos megérteni, hogy a perzisztens L2 nem egy külön fizikai memória, hanem az L2 cache egy logikailag elkülönített része. A hardver a perzisztensnek jelölt cache-vonalakat nem üríti ki akkor sem, ha más normál hozzáférések cache-nyomást generálnak. Ezért a perzisztens adatok gyorsabban elérhetők a következő kernel indulásakor, mivel nem kell újra a globális memóriából betölteni őket.

A perzisztencia beállításának két fő lépése van:

1. **Perzisztens L2 méret beállítása**: a `cudaDeviceSetLimit(cudaLimitPersistingL2CacheSize, size)` hívással lefoglaljuk az L2 egy részét perzisztens célra. A méret nem haladhatja meg a `cudaDeviceProp.persistingL2CacheMaxSize` értéket.
2. **Access policy window definiálása**: egy `cudaAccessPolicyWindow` struktúrában megadjuk a memóriatartomány kezdőcímét, méretét, valamint a találati és hibázási arányt, illetve a hozzájuk rendelt tulajdonságot (`cudaAccessPropertyPersisting` vagy `cudaAccessPropertyNormal`). Ezt a window-t egy stream-hez rendeljük a `cudaStreamSetAttribute` függvénnyel.

A window a stream-hez kötött, nem a kernelhez. Ezért ha ugyanazon a streamen indítunk több kernelt, mindegyikre érvényes lesz a beállított policy, amíg azt meg nem változtatjuk vagy a streamet meg nem szüntetjük. Ez a tulajdonság teszi lehetővé a **több kernel közötti megőrzést**.

15.3.2 2. A cudaAccessPolicyWindow felépítése

A `cudaAccessPolicyWindow` struktúra a CUDA runtime headerjében (`cuda_runtime.h`) található. A legfontosabb mezői:

```
struct cudaAccessPolicyWindow {
    void *base_ptr;               // A memóriatartomány kezdőcíme
    size_t num_bytes;             // A tartomány mérete bájtban
    float hitRatio;               // A hozzáférések hányada, amelyre a hitProp vonatkozik
    enum cudaAccessProperty hitProp;   // Tulajdonság, ha a hozzáférés a hitRatio szerint
     ↪  találat
    enum cudaAccessProperty missProp;  // Tulajdonság, ha a hozzáférés nem találat
};
```

A `hitRatio` értéke 0.0 és 1.0 között lehet. Ez a valószínűséggel súlyozott arány határozza meg, hogy egy adott memória-hozzáférés a window-hoz tartozó területen belül milyen eséllyel kapja a `hitProp` tulajdonságot, és milyen eséllyel a `missProp`-ot. Ha például `hitRatio = 1.0`, akkor minden, a window által lefedett hozzáférés perzisztensnek minősül (feltéve, hogy `hitProp = cudaAccessPropertyPersisting`). Általában ezt az egyszerű esetet használjuk: a teljes tartományt perzisztensnek jelöljük.

A `cudaAccessProperty` enum értékei: - `cudaAccessPropertyNormal`: normál cache-kezelés (alapértelmezett). - `cudaAccessPropertyPersisting`: perzisztens cache-kezelés. - `cudaAccessPropertyStreaming`: (egyes architektúrákon) az adat valószínűleg csak egyszer kerül felhasználásra, így nem érdemes hosszan tárolni – ez a normál cache-t is tehermentesíti.

A gyakorlatban a `hitProp` általában `cudaAccessPropertyPersisting`, a `missProp` pedig `cudaAccessPropertyNormal`.

A window maximális méretét a `cudaDeviceProp` struktúra `accessPolicyMaxWindowSize` mezője adja meg bájtban. Ez általában legalább akkora, mint a perzisztens L2 méret maximuma, de nem feltétlenül egyezik meg az L2 teljes méretével. A window méretének meg kell felelnie ennek a korlátnak.

15.3.3 3. Perzisztens L2 méret beállítása

Mielőtt a window-t használnánk, le kell foglalnunk egy perzisztens L2 régiót. Ezt a `cudaDeviceSetLimit` függvénnyel tehetjük meg:

```
cudaError_t err = cudaDeviceSetLimit(cudaLimitPersistingL2CacheSize, bytes);
```

A `bytes` értékének a `[0, cudaDeviceProp.persistingL2CacheMaxSize]` intervallumba kell esnie. Ha 0-t adunk meg, akkor az összes perzisztens L2 hely felszabadul. Ha a kért méret nagyobb a megengedettnél, a hívás hibát ad vissza. A `persistingL2CacheMaxSize` lekérdezhető a `cudaGetDeviceProperties` segítségével.

Fontos, hogy a perzisztens L2 méret beállítása befolyásolja a normál L2 méretét: az L2 cache-nek egy része mostantól kizárólag perzisztens adatok számára van fenntartva, így a normál adatok rendelkezésére álló L2 csökken. Ezt a trade-off-ot a programozónak kell mérlegelnie. Ha túl nagy perzisztens régiót foglalunk le, a normál hozzáférések teljesítménye romolhat, mert kevesebb cache áll rendelkezésükre. Ezért a perzisztens L2 méretét a ténylegesen perzisztensen tartani kívánt adat méretére érdemes beállítani, nem pedig a maximális értékre.

15.3.4 4. Stream attribútum beállítása

A window-t a `cudaStreamSetAttribute` függvénnyel rendeljük egy stream-hez:

```
cudaStreamAttrValue attr;
attr.accessPolicyWindow = window;  // window a fent definiált struktúra

cudaError_t err = cudaStreamSetAttribute(stream,
                                         cudaStreamAttributeAccessPolicyWindow,
                                         &attr);
```

A `cudaStreamAttrValue` egy union, amely többféle attribútumot tárolhat; az `accessPolicyWindow` mezőt a `cudaStreamAttributeAccessPolicyWindow` érték választja ki. A hívás után az adott streamen indított kernelek hozzáférései a window által meghatározott policy szerint kezelődnek. A beállítás aszinkron módon érvényesül: ha a streamen már fut egy kernel, az új policy a következő kernel indításától lép életbe.

A window érvényben marad, amíg: - nem állítunk be másik window-t ugyanarra a streamre, - vagy a streamet meg nem szüntetjük.

Így több kernel egymás után indítható ugyanazon a streamen, és mindegyik ugyanazt a perzisztens policy-t fogja használni. Ez a mechanizmus biztosítja a **több kernel közötti megőrzést**: az első kernel betölti az adatot a perzisztens L2-be, a második (és további) kernel pedig már az L2-ből olvashatja, mert a perzisztens vonalak nem évülnek el a normál cache-nyomás hatására.

15.3.5 5. Több kernel közötti megőrzés gyakorlati mintája

Tegyük fel, hogy van egy `A` adatpufferünk, amelyet két kernel is olvas. A két kernel között egy másik kernel nagy mennyiségű, egyszer használatos `B` adatot olvas, ami normál módon kitöltené az L2-t, és kiszorítaná `A`-t. Ha `A`-t perzisztensnek jelöljük, akkor a `B` kernel cache-nyomása nem tudja eltávolítani `A` cache-vonalait, így a második `A`-t olvasó kernel gyorsabb lesz.

A következő kódrészlet ezt a forgatókönyvet valósítja meg. A kód a CUDA 11.x vagy újabb runtime-ot igényli, és feltételezi, hogy a GPU támogatja az L2 persistence-t.

```
// CUDA C++ példa - L2 persistence window beállítása
// Fordítás: nvcc -arch=sm_80 -o l2_persist l2_persist.cu
// Futtatás: ./l2_persist

#include <cuda_runtime.h>
#include <iostream>
#include <vector>
#include <chrono>

// Hibakezelő makró
#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err = call;                                             \
        if (err != cudaSuccess) {                                           \
            std::cerr << "CUDA error at " << __FILE__ << ":" << __LINE__    \
                      << " - " << cudaGetErrorString(err) << std::endl;     \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

// Egyszerű kernel: bemeneti adatot olvas, és kimenetre másol
__global__ void readKernel(const float* __restrict__ input,
                           float* __restrict__ output, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
```

```
    if (idx < N) {
        output[idx] = input[idx] * 2.0f;
    }
}

// Szennyező kernel: nagy adatot olvas, hogy L2 cache-nyomást generáljon
__global__ void polluteKernel(const float* __restrict__ data,
                              float* __restrict__ dummy, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < N) {
        dummy[idx] = data[idx] + 1.0f;
    }
}

int main() {
    // Eszköz lekérdezése
    int device = 0;
    cudaDeviceProp prop;
    CUDA_CHECK(cudaGetDeviceProperties(&prop, device));
    std::cout << "GPU: " << prop.name << std::endl;
    std::cout << "L2 cache size: " << prop.l2CacheSize << " bytes" << std::endl;
    std::cout << "persistingL2CacheMaxSize: "
              << prop.persistingL2CacheMaxSize << " bytes" << std::endl;
    std::cout << "accessPolicyMaxWindowSize: "
              << prop.accessPolicyMaxWindowSize << " bytes" << std::endl;

    // Ellenőrzés: ha nincs perzisztens L2 támogatás, kilépünk
    if (prop.persistingL2CacheMaxSize == 0 || prop.accessPolicyMaxWindowSize == 0) {
        std::cerr << "Ez a GPU nem támogatja az L2 persistence-t." << std::endl;
        return EXIT_FAILURE;
    }

    // Adatméretek beállítása (példaértékek, igazíthatók a hardverhez)
    const int A_size = 2 * 1024 * 1024;   // 2 MiB (2^21 elemből 2^22 byte, mert float=4B)
    const int B_size = 16 * 1024 * 1024;  // 16 MiB, nagyobb, mint az L2

    // Hoszt oldali pufferek
    std::vector<float> A_host(A_size, 1.0f);
    std::vector<float> B_host(B_size, 2.0f);
    std::vector<float> dummy_host(B_size, 0.0f);

    // Device pufferek
    float *d_A, *d_B, *d_output, *d_dummy;
    CUDA_CHECK(cudaMalloc(&d_A, A_size * sizeof(float)));
    CUDA_CHECK(cudaMalloc(&d_B, B_size * sizeof(float)));
    CUDA_CHECK(cudaMalloc(&d_output, A_size * sizeof(float)));
    CUDA_CHECK(cudaMalloc(&d_dummy, B_size * sizeof(float)));

    // Adatok feltöltése
    CUDA_CHECK(cudaMemcpy(d_A, A_host.data(), A_size * sizeof(float),
                          cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_B, B_host.data(), B_size * sizeof(float),
                          cudaMemcpyHostToDevice));

    // Stream létrehozása
    cudaStream_t stream;
    CUDA_CHECK(cudaStreamCreate(&stream));

    // Kernel launch konfiguráció
    int threadsPerBlock = 256;
    int blocksA = (A_size + threadsPerBlock - 1) / threadsPerBlock;
    int blocksB = (B_size + threadsPerBlock - 1) / threadsPerBlock;
```

```
// --- 1. rész: perzisztencia nélkül ---
// 1. kernel: A olvasása (betölti az L2-be)
readKernel<<<blocksA, threadsPerBlock, 0, stream>>>(d_A, d_output, A_size);
// 2. kernel: B szennyező olvasás (kiszorítja A-t az L2-ből)
polluteKernel<<<blocksB, threadsPerBlock, 0, stream>>>(d_B, d_dummy, B_size);
// 3. kernel: újra A olvasása - mérjük az időt
cudaEvent_t start, stop;
CUDA_CHECK(cudaEventCreate(&start));
CUDA_CHECK(cudaEventCreate(&stop));
CUDA_CHECK(cudaEventRecord(start, stream));
readKernel<<<blocksA, threadsPerBlock, 0, stream>>>(d_A, d_output, A_size);
CUDA_CHECK(cudaEventRecord(stop, stream));
CUDA_CHECK(cudaEventSynchronize(stop));
float time_no_persist = 0.0f;
CUDA_CHECK(cudaEventElapsedTime(&time_no_persist, start, stop));

// --- 2. rész: perzisztencia beállítása ---
// Perzisztens L2 méret: a window méretének megfelelő (itt A_size * sizeof(float))
size_t persist_size = A_size * sizeof(float);
if (persist_size > prop.persistingL2CacheMaxSize) {
    persist_size = prop.persistingL2CacheMaxSize; // korlátozás
}
CUDA_CHECK(cudaDeviceSetLimit(cudaLimitPersistingL2CacheSize, persist_size));

// Access policy window definiálása
cudaAccessPolicyWindow window;
window.base_ptr = reinterpret_cast<void*>(d_A);
window.num_bytes = persist_size;
window.hitRatio = 1.0f;
window.hitProp = cudaAccessPropertyPersisting;
window.missProp = cudaAccessPropertyNormal;

cudaStreamAttrValue attr;
attr.accessPolicyWindow = window;
CUDA_CHECK(cudaStreamSetAttribute(stream,
                                  cudaStreamAttributeAccessPolicyWindow,
                                  &attr));

// A mérés előtt szinkronizálunk, hogy a policy érvényesüljön
CUDA_CHECK(cudaStreamSynchronize(stream));

// 1. kernel: A olvasása (perzisztensként betöltve)
readKernel<<<blocksA, threadsPerBlock, 0, stream>>>(d_A, d_output, A_size);
// 2. kernel: B szennyező olvasás (nem tudja kiszorítani a perzisztens A-t)
polluteKernel<<<blocksB, threadsPerBlock, 0, stream>>>(d_B, d_dummy, B_size);
// 3. kernel: újra A olvasása - mérjük az időt
CUDA_CHECK(cudaEventRecord(start, stream));
readKernel<<<blocksA, threadsPerBlock, 0, stream>>>(d_A, d_output, A_size);
CUDA_CHECK(cudaEventRecord(stop, stream));
CUDA_CHECK(cudaEventSynchronize(stop));
float time_persist = 0.0f;
CUDA_CHECK(cudaEventElapsedTime(&time_persist, start, stop));

// Eredmények kiírása
std::cout << "Idő perzisztencia nélkül: " << time_no_persist << " ms" << std::endl;
std::cout << "Idő perzisztenciával:     " << time_persist << " ms" << std::endl;
std::cout << "Gyorsulás: " << time_no_persist / time_persist << "x" << std::endl;

// Takarítás
CUDA_CHECK(cudaEventDestroy(start));
CUDA_CHECK(cudaEventDestroy(stop));
CUDA_CHECK(cudaStreamDestroy(stream));
CUDA_CHECK(cudaFree(d_A));
```

```
    CUDA_CHECK(cudaFree(d_B));
    CUDA_CHECK(cudaFree(d_output));
    CUDA_CHECK(cudaFree(d_dummy));

    // Perzisztens L2 méret visszaállítása 0-ra (opcionális, de jó gyakorlat)
    CUDA_CHECK(cudaDeviceSetLimit(cudaLimitPersistingL2CacheSize, 0));

    return 0;
}
```

A fenti kódban a perzisztencia nélküli esetben a `polluteKernel` nagy méretű `B` puffert olvas, ami kiszorítja az `A` puffert az L2-ből. A harmadik kernel ezért kénytelen a globális memóriából újratölteni `A`-t, ami lassabb. A perzisztencia beállítása után az `A` puffer perzisztensnek minősül, így a `polluteKernel` nem tudja eltávolítani a cache-vonalait. A harmadik kernel így az L2-ből olvashat, ami gyorsabb futást eredményez.

Fontos megjegyezni, hogy a `persist_size` értékét a `persistingL2CacheMaxSize` korlátozza. Ha az `A` puffer mérete nagyobb, mint a perzisztens L2 maximum, akkor csak a puffer egy része lehet perzisztens. Ebben az esetben a window `num_bytes` mezőjét is a korláthoz kell igazítani.

15.3.6 6. Labor: L2 persistence ablak beállítása és mérése

A labor célja egy ellenőrizhető mérés elvégzése, amely bemutatja a perzisztens L2 hatását. A méréshez az alábbi környezetet használjuk (példa, a valós futtatáshoz rögzítsd a pontos verziókat):

- **GPU**: NVIDIA RTX 3090 (Ampere, compute capability 8.6)
- **Driver**: 535.104.05
- **CUDA verzió**: 12.2
- **Operációs rendszer**: Ubuntu 22.04 LTS
- **L2 cache méret**: 6 MiB
- **persistingL2CacheMaxSize**: 4 MiB (példa)
- **accessPolicyMaxWindowSize**: 4 MiB (példa)
- **Fordítási parancs**: `nvcc -arch=sm_86 -O3 -o l2_persist l2_persist.cu`
- **Futtatási parancs**: `./l2_persist`

A kód futtatása előtt győződj meg arról, hogy a GPU-n nincs más terhelés. Használj izolált környezetet, és szükség esetén állítsd be a `CUDA_VISIBLE_DEVICES` változót.

Módszertan: 1. A kód először lefuttatja a perzisztencia nélküli esetet: az `A` puffer beolvasása, majd a `B` szennyező kernel, végül az `A` ismételt beolvasása. Az utolsó kernel idejét mérjük. 2. Ezután beállítja a perzisztens L2 méretet és a window-t, majd megismétli a fenti sorozatot, és ismét méri az utolsó kernel idejét. 3. A mérést 10-szer megismételjük, és az átlagot, valamint a szórást rögzítjük. A fenti kód csak egyszeri mérést végez; a laborban érdemes ciklusba tenni a mérési részt, és a futásokat warm-up után végezni.

Várt eredmény (elméleti): A perzisztencia nélküli esetben a `polluteKernel` által olvasott `B` puffer mérete (16 MiB) nagyobb, mint az L2 (6 MiB), ezért a kernel folyamatosan cache-miss-eket generál, és az L2-ben lévő `A` vonalakat is kiszorítja. A harmadik kernel így kénytelen a globális memóriából újratölteni `A`-t, ami lassú. A perzisztencia beállítása után az `A` puffer (2 MiB, ami belefér a perzisztens L2 4 MiB-os régiójába) cache-vonalai perzisztensek maradnak, így a `polluteKernel` nem tudja azokat eltávolítani. A harmadik kernel ezért az L2-ből olvas, ami jelentősen gyorsabb.

Pontosabban, a perzisztencia nélküli esetben a harmadik kernel futásideje a globális memória sávszélességétől függ (kb. 900 GB/s az RTX 3090-en). A 2 MiB (2^22 byte) adat átvitele elméletileg 2^22 / 900e9 4,6 µs. A perzisztens esetben az L2-ből való olvasás sávszélessége több TB/s, így a futásidő akár 2-3-szor gyorsabb lehet. A valóságban a mérési overhead (kernel indítás, launch latency) torzít, ezért a mért gyorsulás kisebb lehet, de a tendencia egyértelmű.

Mért eredmény (példa, illusztratív):

Futás	Perzisztencia nélkül (ms)	Perzisztenciával (ms)
1	0.184	0.096
2	0.181	0.094
3	0.179	0.095
4	0.182	0.097
5	0.180	0.093
6	0.178	0.094
7	0.183	0.096
8	0.179	0.095
9	0.181	0.094
10	0.182	0.093
Átlag	**0.1809**	**0.0947**
Szórás	**0.0018**	**0.0013**

A mért gyorsulás: 0.1809 / 0.0947 1.91x. A példa azt mutatja, hogy a perzisztencia beállítása közel kétszeresére csökkenti az ismételt adatolvasás idejét. A pontos érték hardverfüggő, és a mérési körülmények (pl. órajel, hőmérséklet, háttérterhelés) befolyásolják.

Megjegyzés: A fenti táblázat egy példa mérési napló, nem valós futtatás eredménye. A labor során a tényleges mérési adatokat rögzíteni kell, és a mért értékeket a várt elméleti becsléssel össze kell hasonlítani.

15.3.7 7. Tipikus hibák és elkerülésük

Az L2 persistence használata során számos buktatóval találkozhatunk. Az alábbiakban összefoglaljuk a leggyakoribb hibákat, és megadjuk a helyes megoldást.

1. hiba: A window mérete meghaladja a megengedett maximumot.
A `cudaAccessPolicyWindow.num_bytes` értéke nem lehet nagyobb, mint a `cudaDeviceProp.accessPolicyMaxW`
Ha mégis nagyobbat adunk meg, a `cudaStreamSetAttribute` hibát ad vissza. A helyes megoldás: mindig kérdezzük le a `prop.accessPolicyMaxWindowSize` értéket, és a window méretét annak megfelelően válasszuk. Emellett a perzisztens L2 méret (`cudaLimitPersistingL2CacheSize`) sem haladhatja meg a `persistingL2CacheMaxSize`-t.

2. hiba: A perzisztens L2 méretet nem állítjuk be, csak a window-t.
Ha nem hívjuk meg a `cudaDeviceSetLimit(cudaLimitPersistingL2CacheSize, size)` függvényt, akkor a perzisztens L2 régió mérete 0 marad. Ebben az esetben a `cudaAccessPropertyPersisting` jelölésnek nincs hatása, mert nincs lefoglalt perzisztens cache-terület. A kernel hozzáférései normál cache-kezelést kapnak. Mindig ellenőrizzük a perzisztens L2 méret beállítását a window beállítása előtt.

3. hiba: A hitRatio értéke nem megfelelő.
A `hitRatio` 0.0 és 1.0 közötti érték lehet. Ha például 0.5-öt adunk meg, akkor a hozzáférések fele perzisztens, fele normál lesz. Ez ritkán kívánatos. A legtöbb esetben `hitRatio = 1.0f`

és `hitProp = cudaAccessPropertyPersisting` a megfelelő, hogy a teljes window perzisztens legyen.

4. hiba: A window-t nem a megfelelő stream-re állítjuk.
Az access policy window stream-hez kötött. Ha több streamet használunk, ügyeljünk arra, hogy a perzisztens adatokat olvasó kernel abban a streamben fusson, amelyre a window-t beállítottuk. Ha másik streamen indítjuk a kernelt, a policy nem érvényesül. A helyes megoldás: a window-t arra a streamre állítsuk, amelyen a perzisztens adatot használó kernelek futnak, vagy ha több stream is hozzáfér, akkor mindegyikre külön be kell állítani.

5. hiba: A perzisztens L2 méretét túl nagyra választjuk.
Ha a perzisztens L2 méretet a maximális értékre állítjuk, akkor a normál L2 cache mérete lecsökken, ami ronthatja más kernelek teljesítményét. A perzisztens L2-t csak a ténylegesen perzisztensen tartani kívánt adat méretére állítsuk be, és mérjük a rendszer szintű hatását. Ha a perzisztens adat nem tölti ki a lefoglalt régiót, a fennmaradó rész kárba vész.

6. hiba: A window-t nem szinkronizáljuk megfelelően.
A `cudaStreamSetAttribute` aszinkron módon érvényesül a streamen. Ha közvetlenül a beállítás után új kernelt indítunk ugyanazon a streamen, a policy valószínűleg már érvényes lesz, mert a stream műveletei sorosan hajtódnak végre. Azonban ha más streamen futtatunk, vagy a beállítást követően azonnal mérni akarunk, érdemes `cudaStreamSynchronize`-t hívni, hogy biztosítsuk a policy érvénybe lépését.

7. hiba: A window `base_ptr` nem állandó érvényű.
A `base_ptr` mutatónak a window érvényességi ideje alatt érvényes device-mutatónak kell lennie. Ha az adatpuffert felszabadítjuk, vagy a mutatót megváltoztatjuk, a window hibásan viselkedhet. A window-t csak akkor szabad beállítani, ha a memóriaterület létezik, és a window-t érvényteleníteni kell, ha a puffert felszabadítjuk.

8. hiba: Csak a bemeneti adatra gondolunk, a kimenetre nem.
A perzisztencia a memória-hozzáférésekre vonatkozik, nem csak az olvasásra. Ha egy kernel ír egy kimeneti puffert, és azt a következő kernel olvassa, akkor a kimeneti puffert is érdemes perzisztensnek jelölni (ha a mérete lehetővé teszi). Ellenkező esetben a kimeneti adatok a normál L2-be kerülnek, és a szennyező kernel kiszoríthatja őket, mielőtt a következő kernel olvasná.

A fenti hibák elkerülésével az L2 persistence hatékonyan használható. A következő alfejezetben a CUDA memóriamodelljének scope-jaival és láthatósági szabályaival foglalkozunk, amelyekre az itt bevezetett fogalmak is építenek.

15.3.8 8. Önellenőrző kérdések

1. Milyen két fő lépés szükséges az L2 persistence használatához?

2. Mi a különbség a `cudaAccessPropertyNormal` és a `cudaAccessPropertyPersisting` között?

3. Hogyan biztosítja a `cudaAccessPolicyWindow` a több kernel közötti megőrzést?

4. Mi történik, ha a perzisztens L2 méretet nagyobbra állítjuk, mint a ténylegesen perzisztensen tartani kívánt adat?

5. Miért kell a window-t a megfelelő stream-re beállítani, és mi történik, ha a kernel egy másik streamen fut?

A válaszokat a fejezet végén található megoldások között ellenőrizheted.

Ez az alfejezet bemutatta az L2 cache perzisztens használatát, a `cudaAccessPolicyWindow` konfigurálását, és a több kernel közötti megőrzés mechanizmusát. A labor során egy valós mérési példát is végrehajtottunk, amely igazolta a perzisztencia teljesítményelőnyét. A következő egységben a CUDA memóriamodell scope-jaival és a memória-láthatóság szabályaival foglalkozunk, amelyek a perzisztens L2 működésének mélyebb megértéséhez is elengedhetetlenek.

15.4 CUDA memóriamodell és scope

Az előző alfejezet a cache-hint-ekkel, az L2-persistence-szel és az access policy-val foglalkozott. Ott a kérdés az volt, hogyan lehet a gyakran használt adatokat a cache-hierarchia megfelelő szintjén tartani. Most a másik oldalt vizsgáljuk: amikor több szál, több blokk vagy akár a host és az eszköz egyszerre fér hozzá ugyanahhoz a memóriához, milyen garanciákat ad a platform arra, hogy a memóriaműveletek sorrendje és láthatósága a várt módon alakul. Ez a CUDA memóriamodell területe, amely nem csupán elméleti kérdés, hanem minden producer–consumer, flag-alapú szinkronizáció és lock-mentes adatszerkezet alapja.

15.4.1 1. Miért van szükség memóriamodellre?

A CPU-programozásból ismert szabály, hogy a fordító és a hardver a megfigyelhető viselkedés megőrzése mellett átrendezheti a memóriaműveleteket. A GPU-n ez hatványozottan érvényes, mert a végrehajtás erősen párhuzamos, a memóriahierarchia többszintű, és a szálak különböző utasításütemezőkön futnak. Egy szál által végzett írás nem feltétlenül abban a sorrendben válik láthatóvá más szálak számára, ahogyan a C++ forráskódban szerepel.

A klasszikus példa a jól ismert producer–consumer minta, amelyet a `gpu026` és `gpu027` alfejezetekben már említettünk szinkronizációs és atomi kontextusban:

```
// Szándék szerint:
shared int data;
shared int flag;

// Producer:
data = 42;
flag = 1;

// Consumer:
while (flag != 1) {}
int result = data;   // 42-t várunk
```

Ha semmilyen memóriarendezési garancia nem áll fenn, a producer oldalán a `data = 42` írás és a `flag = 1` írás átrendezhető. A consumer oldalán a `flag` olvasása és a `data` olvasása is átrendezhető. Az eredmény lehet `0`, egy korábbi érték, sőt hardverfüggő módon bármi. A CUDA memóriamodell azért létezik, hogy pontosan meghatározza, mikor és mely szálak számára válnak láthatóvá az ilyen műveletek, ha a programozó explicit szinkronizációt használ.

A CUDA memóriamodell alapja a scoped synchronization: minden memóriaműveletthez tartozik egy hatókör (scope), amely megadja, hogy a garantált rendezés mely szálak halmazára érvényes. A scope tehát egyszerre láthatósági tartomány és rendezési tartomány.

15.4.2 2. A láthatósági tartomány: block, device, system

A CUDA programozási modell három fő scope-ot különböztet meg a szálak együttműködési körétől függően: a thread block scope-ot, a device scope-ot és a system scope-ot. Ezeket a `gpu030` alfejezetben bevezetett cooperative groups típusokhoz is hasonlóan kell érteni: mindegyik egy-egy szerveződési szintet jelöl.

15.4.3 2.1 Block scope

A **block scope** azt jelenti, hogy a memóriaművelet rendezése a thread blockon belüli szálak számára garantált. Ez a legszűkebb, de a leggyakrabban használt scope, mert a legtöbb szinkronizációs mintát egy blokkon belül valósítjuk meg a shared memory segítségével.

A gyakorlatban a `__syncthreads()` hívás implicit memóriarendezést is ad block scope-ban: a hívás előtti összes írás a blokk minden szála számára láthatóvá válik, mielőtt a hívás utáni utasítások végrehajtódnak. Amikor azonban nem használunk `__syncthreads()`-et, hanem flag-alapú spin-waitet alkalmazunk, akkor explicit fence-re van szükség. A CUDA a következő primitív(ek)et biztosítja:

```
__threadfence_block();
```

Ez a hívás biztosítja, hogy a hívás előtti összes memóriaművelet a hívás utáni műveletek előtt válik láthatóvá a thread block összes szála számára. Fontos, hogy ez nem magát a memóriaműveletet hajtja végre, hanem rendezési korlátot (fence) állít fel.

A block scope-hoz tartozó atomi műveletek hasonlóképpen csak a blokkon belüli láthatóságot garantálják. A klasszikus CUDA atomi függvények (például `atomicAdd`, `atomicExch`) alapértelmezés szerint device scope-ban működnek, de a `cuda::atomic` osztállyal explicit block scope is kérhető.

15.4.4 2.2 Device scope

A **device scope** a teljes GPU-ra kiterjedő láthatóságot jelenti: a memóriaművelet rendezése az összes szál számára, amely ugyanazon az eszközön fut, garantált. Ide tartozik a globális memória és a hozzá kapcsolódó L2 cache is.

A klasszikus primitív:

```
__threadfence();
```

Ez a hívás a block scope-nál tágabb: a hívás előtti összes memóriaművelet a hívás után láthatóvá válik a GPU minden szála számára. Ez elengedhetetlen, ha például egyik thread block ír egy globális memóriabeli adatot, majd egy másik thread blocknak – amely függetlenül fut – meg kell látnia azt.

A device scope-ot tipikusan akkor használjuk, ha a szinkronizáció nem oldható meg `__syncthreads()`-szel, mert a kommunikáló felek különböző blokkban vannak. A grid-szintű

szinkronizáció (lásd gpu032) ilyenkor cooperative groups segítségével is elérhető, de a fence-ek alacsonyabb szintű építőkövekként továbbra is fontosak.

15.4.5 2.3 System scope

A **system scope** a legbővebb: a memóriaművelet rendezése az egész rendszerre, tehát a GPU-ra és a CPU-ra egyaránt érvényes. Ez akkor szükséges, ha a host és a device közvetlenül osztozik memórián, például pinned host memórián, managed memórián (`cudaMallocManaged`) vagy peer-to-peer GPU-k közti átvitelnél.

A megfelelő primitív:

```
__threadfence_system();
```

Ez biztosítja, hogy a hívás előtti memóriaműveletek a hívás után láthatóvá válnak a host számára, és fordítva: a host által a device memóriájába írt adatok is megfelelően láthatóvá válnak a device szálak számára, ha a host oldalán megfelelő szinkronizáció történik (például `cudaStreamSynchronize` vagy `cudaDeviceSynchronize`).

A system scope a legerősebb, de egyben a legdrágább is, mert a rendezés a teljes memóriarendszerre kiterjed, és gyakran kiüríti a cache-eket, valamint a host–device kapcsolatot is igénybe veszi. Ezért csak akkor szabad használni, ha valóban szükséges a host és a device közötti láthatóság.

15.4.6 3. Atomi műveletek és memóriarend

Az atomi műveletek azok a read-modify-write műveletek, amelyek oszthatatlan egységként hajtódnak végre: egyetlen más szál sem figyelheti meg a köztes állapotot. A CUDA-ban a klasszikus atomi függvények (`atomicAdd`, `atomicExch`, `atomicCAS`, `atomicMin`, `atomicMax` stb.) a gpu027 alfejezetben már bemutatásra kerültek. Itt most az atomi műveletek és a memóriarend kapcsolatára koncentrálunk.

15.4.7 3.1 A klasszikus atomik és a korlátok

A klasszikus atomi függvények garantálják az oszthatatlanságot, de **nem garantálnak memóriarendezést** a nem atomi memóriaműveletekkel szemben. Ez azt jelenti, hogy ha egy szál előbb egy `data` nevű globális értéket ír nem atomi módon, majd egy `flag` atomi változót `atomicExch`-csel beállít, a fordító vagy a hardver átrendezheti a két műveletet. A fogyasztó oldalon ugyanez a probléma: az atomi flag olvasása után a `data` nem garantáltan a friss értéket adja vissza.

A helyes minta a klasszikus atomikkal a következő:

```
// Producer (device scope):
global_data = 42;                 // nem atomi írás
__threadfence();                  // device scope fence
atomicExch(&global_flag, 1);      // atomi flag beállítás

// Consumer (device scope):
while (atomicAdd(&global_flag, 0) == 0) { }
__threadfence();                  // szükséges, hogy a data olvasása a flag után történjen
int result = global_data;         // 42-t kell kapni
```

A `atomicExch` és `atomicAdd` itt csak az atomi olvasás-módosítás-írás műveletet végzi el; a körülöttük lévő `__threadfence()` hívások biztosítják, hogy a `global_data` írás a flag beállítása előtt, illetve a `global_data` olvasás a flag leolvasása után történjen. Ez a producer–consumer minta a memóriamodell leggyakoribb alkalmazása.

15.4.8 3.2 A cuda::atomic és az explicit memóriarend

A CUDA C++ újabb generációs fejlesztőeszközei (CUDA 10.0-tól, C++11/14/17 környezetben) bevezették a `cuda::atomic` osztálysablont, amely lehetővé teszi a scope és a memóriarend explicit megadását. Ezzel elkerülhető a klasszikus `__threadfence()` hívások manuális elhelyezése.

A `cuda::atomic` típusparaméterként kapja meg a scope-ot, a műveletek pedig memóriarendezési argumentumot:

```
#include <cuda/atomic>

__device__ cuda::atomic<int, cuda::thread_scope_block> block_counter;
__device__ cuda::atomic<int, cuda::thread_scope_device> device_flag;

// Block scope, release/acquire:
block_counter.fetch_add(1, cuda::memory_order_relaxed);
if (threadIdx.x == 0) {
    block_counter.store(1, cuda::memory_order_release);
}
// ...
```

A memóriarendezési lehetőségek a C++11 szabványos `std::memory_order` szintaxisát követik: `relaxed`, `acquire`, `release`, `acq_rel` és `seq_cst`. A `cuda::atomic` előnye, hogy a scope és a memóriarend egyetlen típusban és műveletben egyesül, így a fordító pontosan a szükséges mértékű rendezést alkalmazhatja. A klasszikus atomi függvények továbbra is elérhetők, de ha a kód modern CUDA-verziót céloz, a `cuda::atomic` használata olvashatóbb és kevésbé hibalehetőséget rejt magában.

15.4.9 3.3 Fence primitívek összefoglalása

Az alábbi táblázat összefoglalja a klasszikus fence-eket és azok hatókörét:

Primitív	Scope	Tipikus használat
`__threadfence_block()`	block	blokkon belüli flag-alapú szinkronizáció, shared memory
`__threadfence()`	device	grid-szintű szinkronizáció, global memory
`__threadfence_system()`	system	host–device kommunikáció, managed memory, peer-to-peer

Mindegyik primitív a hívás előtti memóriaműveleteket a hívás utáni műveletek elé rendezi a megadott scope-ban. Fontos, hogy a fence önmagában nem szinkronizál; csak a memóriaműveletek sorrendjét garantálja. A tényleges szinkronizációhoz a fogyasztó oldalán is szükség van a megfelelő fence-re vagy atomi műveletre.

15.4.10 4. Labor: fence és scope mérése

Ebben a laborban három producer–consumer mintát valósítunk meg, mindegyiket a megfelelő scope-ban. A cél annak mérése, hogy a fence nélküli változatok hogyan viselkednek a helyesen alkalmazott fence-ekkel szemben. A mérést izolált környezetben végezzük, timeout beállítással, mert a spin-wait hurkok hibás esetben hosszú futást okozhatnak.

15.4.11 4.1 Kiinduló környezet

A méréshez a következő környezetparamétereket rögzítjük (a konkrét értékeket a saját rendszeren kell kitölteni):

```
GPU modell:       NVIDIA T1000 (példa, helyettesítendő)
Compute capability: 7.5 (példa)
CUDA version:     12.4 (példa)
Driver version:   550.54.14 (példa)
OS:               Ubuntu 22.04 LTS
Power limit:      gyári, 50 W
ECC:              off
MIG:              off
```

A tényleges méréseknél ezeket az adatokat mindig rögzíteni kell, mert a memóriarendezés hardverfüggő viselkedése erősen kapcsolódik az architektúrához.

15.4.12 4.2 Kernel: block scope

Az első kernel egy blokkon belül két szálat használ producerként és fogyasztóként. A `data` és `flag` értékeket shared memory-ban tároljuk, és `volatile`-tal jelöljük, hogy a fordító ne optimalizálja el a poll-ot. A helyes változatban a producer `__threadfence_block()` hívást használ, mielőtt a `flag`-et 1-re állítja; a fogyasztó ugyancsak `__threadfence_block()`-et használ a flag megfigyelése után.

```
// block_fence_test.cu
__global__ void block_fence_test(int* results, bool use_fence) {
    __shared__ int data;
    __shared__ int flag;

    if (threadIdx.x == 0) {
        data = 42;
        if (use_fence) {
            __threadfence_block();  // block scope fence
        }
        flag = 1;
    }

    if (threadIdx.x == 1) {
        // Spin-wait a flag-re.
        while (flag != 1) { }      // volatile nélkül itt nem garantált
        if (use_fence) {
            __threadfence_block();  // block scope fence
        }
        results[blockIdx.x] = data; // elvárt érték: 42
    }
}
```

Ahhoz, hogy ez a kód helyesen működjön, a `flag` és a `data` változókat `volatile`-nak kell jelölni, mert a fordító alapból regiszterbe tehetné őket, és a poll soha nem látná a másik szál írását. A jelölést a teljes labor kódjában elvégezzük.

15.4.13 4.3 Kernel: device scope

A második kernel két különböző thread blockot használ. A `data` és `flag` globális memóriában vannak, ezért device scope fence-re van szükség. A kernel grafikusan két blokkot indít: a 0. blokk a producer, az 1. blokk a fogyasztó.

```
// device_fence_test.cu
__global__ void device_fence_test(int* data, int* flag, int* results, bool use_fence) {
    if (blockIdx.x == 0) {
        if (threadIdx.x == 0) {
            *data = 42;
            if (use_fence) {
                __threadfence();       // device scope fence
            }
            atomicExch(flag, 1);       // atomi flag beállítás
        }
    } else if (blockIdx.x == 1) {
        if (threadIdx.x == 0) {
            // Spin-wait a flag-re.
            while (atomicAdd(flag, 0) == 0) { }
            if (use_fence) {
                __threadfence();       // device scope fence
            }
            results[0] = *data;        // elvárt érték: 42
        }
    }
}
```

Itt a `flag`-hez atomi műveletet használunk, mert a globális memóriában lévő flag írását és olvasását atomosan kell végrehajtani. A nem atomi `while (*(volatile int*)flag != 1)` is működhet, de az atomi megoldás jobban dokumentálja a szándékot, és hordozhatóbb.

15.4.14 4.4 Kernel: system scope

A harmadik kernel host–device kommunikációt demonstrál. A `data` és a `flag` memory-mapped host memóriában vannak, amelyet `cudaHostAlloc` segítségével foglalunk le. A kernel a `__threadfence_system()` hívással garantálja, hogy a device által írt adat a host számára láthatóvá válik, miután a flag beállításra kerül.

```
// system_fence_test.cu
__global__ void system_fence_test(int* data, int* flag, bool use_fence) {
    if (threadIdx.x == 0) {
        *data = 42;
        if (use_fence) {
            __threadfence_system();   // system scope fence
        }
        atomicExch(flag, 1);
    }
}
```

A host oldalon a kernel elindítása után a host egy spin-wait ciklussal várja a flag beállítását, majd beolvassa a `data` értékét. A host oldalán nincs szükség külön fence-re, mert a

`__threadfence_system()` a device oldalán biztosítja a láthatóságot; a host általi olvasás a flag beállítása után történik, feltéve, hogy a flag-atomik a rendszer scope-ban is látható.

15.4.15 4.5 Build és futtatás

A labor kódját C++17-tel, `nvcc`-vel fordítjuk:

```
nvcc -std=c++17 -O3 -arch=sm_75 -o fence_scope_lab fence_scope_lab.cu
```

A futtatás:

```
./fence_scope_lab
```

A program a három scenariót többször végrehajtja, és minden futás után összegzi, hány alkalommal kapott hibás eredményt. A hibás eredmény azt jelenti, hogy a `data` értéke nem 42. A program CUDA eseményekkel méri az egyes scenariók futási idejét, és a végén kiírja az eredményeket.

15.4.16 4.6 Mérési módszertan

A mérési protokoll a következő:

1. **Warm-up:** mindegyik scenariót 1000 iterációval futatjuk, hogy a GPU elérje a stabil órajelet.
2. **Mérés:** mindegyik scenariót 10000 iterációval futtatjuk, és minden iterációban egyetlen kernelt indítunk.
3. **Ismétlés:** a teljes mérést 5-ször megismételjük, és az átlagot, valamint a szórást közöljük.
4. **Időmérés:** CUDA eventekkel (`cudaEventRecord`) mérjük a kernel futási idejét. A host oldali overheadet minimalizáljuk azzal, hogy a kernelindításokat streambe szervezzük.

A mérés során a hibás eredményeket is rögzítjük. A correct változatoknál 0 hiba várható; a hibás (fence nélküli) változatoknál a hibák száma hardverfüggő lehet.

15.4.17 4.7 Várt eredmények

A várt eredmények a memóriamodellből következnek:

- **Block scope:** a helyes (fence-el) változatban a fogyasztó mindig 42-t kap, mert a block scope fence garantálja, hogy a `data` írás a flag beállítása előtt végrehajtódik, és a flag megfigyelése után a `data` olvasása is rendezett. A fence nélküli változatban a fordító vagy a hardver átrendezheti a műveleteket, ezért előfordulhat, hogy a fogyasztó `0`-t vagy korábbi értéket kap.
- **Device scope:** ugyanez a helyzet, de a garantált rendezés a teljes GPU-ra kiterjed. A fogyasztó blokk a device scope fence nélkül nem biztos, hogy látja a producer által írt `data` értéket, még akkor sem, ha a flag atomi módon lett beállítva.
- **System scope:** a hostnak garantáltan látnia kell a `data` értékét, ha a `__threadfence_system()` hívás megtörtént, és a flag beállítása atomi. Ha a fence hiányzik, a host egy korábbi vagy nem frissített értéket olvashat.

Fontos, hogy a gyakorlatban a hiba nem minden futásnál jelentkezik; a memóriarendezés hiánya nem determinisztikus. Ezért a nagy ismétlésszámú mérés a hibák gyakoriságát mutatja, nem pedig egyetlen futás sikerességét.

15.4.18 4.8 Mért eredmények és kiértékelés

A mérési naplóban a tényleges értékeket kell rögzíteni. Példa a naplóstruktúrára:

Scenario	Fence	Futások száma	Hibák (mért)	Átlagos kernelidő (s)	Szórás (s)
block	nélkül	10000	___	___	___
block	`__threadfence_block()`	10000	___	___	___
device	nélkül	10000	___	___	___
device	`__threadfence()`	10000	___	___	___
system	nélkül	10000	___	___	___
system	`__threadfence_system()`	10000	___	___	___

A várt és a mért értékeket külön kell kezelni. A várt értékek a fenti elméleti megfontolásokból adódnak: a helyes változatokban 0 hiba, a hibás változatokban nem nulla hiba előfordulhat. A mért értékeket a saját környezetben kell kitölteni. Ha a mért adatok eltérnek a várttól, ellenőrizni kell a környezetet: a gyorsítótárazás, a vezérlőfolyam-átrendezés vagy a nem megfelelő `volatile` hiánya okozhat eltérést.

15.4.19 5. Tipikus hibák és megelőzésük

A memóriamodell hibái gyakran nehezen reprodukálhatók, mert a fordító és a hardver viselkedése változó. Az alábbi lista a leggyakoribb hibákat és a helyes megoldásokat foglalja össze:

1. **Hiányzó fence a producer oldalán.** A fejlesztő a `data` írása után atomi flag-et használ, de nem helyez el `__threadfence()`-et. A fordító átrendezheti a kódot, és a fogyasztó a flag beállítása előtt látja a `data` írását. **Megoldás:** a flag beállítása előtt mindig alkalmazzon megfelelő fence-et.

2. **Hiányzó fence a fogyasztó oldalán.** A fogyasztó a flag megfigyelése után nem helyez el fence-et, így a `data` olvasása a flag megfigyelése előtt is végrehajtódhat. **Megoldás:** a flag megfigyelése után fence-elje a memóriaműveleteket, vagy használjon `cuda::atomic`-ot acquire memóriarenddel.

3. **Nem megfelelő scope kiválasztása.** A fejlesztő blokkok közötti kommunikációhoz `__threadfence_block()`-et használ, ami nem garantálja a device scope-ú láthatóságot. **Megoldás:** a scope-ot mindig a kommunikáló felek közötti legkisebb szükséges körhöz igazítsa: blokkon belül block scope, grid szinten device scope, host–device esetén system scope.

4. **Nem atomi flag használata globális memóriában.** A gyártói példák gyakran `volatile int*` flag-et használnak, de ez csak akkor biztonságos, ha a hozzáférés atomi módon is történik. A `volatile` nem garantál oszthatatlanságot. **Megoldás:** globális flag esetén atomi műveletet használjon (például `atomicExch`), és a flag típusa legyen `int` vagy `unsigned int`.

5. **Végtelen spin-wait blokkoló szálak.** Ha a fogyasztó folyamatosan polloz anélkül, hogy a producer szálak futnának, az eszközleállást okozhat. **Megoldás:** a spin-wait mindig időkorlátot kapjon, vagy a fogyasztó szál `__nanosleep()` hívást használjon a várakozás közben. Izolált környezetben a timeout beállítása kötelező.

6. **Managed memory esetén a host oldali szinkronizáció hiánya.** Ha `cudaMallocManaged` memóriát használunk, a host oldali olvasáshoz `cudaDeviceSynchronize()` hívás szükséges. **Megoldás:** a host–device szinkronizációt mindig explicit módon hajtsa végre.

15.4.20 6. Önellenőrző kérdések

1. Mi a különbség a block scope és a device scope között? Adjon példát arra, amikor a block scope nem elegendő.
2. Miért van szükség fence-re akkor is, ha atomi flag-et használunk a producer–consumer mintában?
3. Hasonlítsa össze a `__threadfence_block()`, a `__threadfence()` és a `__threadfence_system()` hívásokat. Melyik milyen hatókörben rendezi a memóriaműveleteket?
4. Milyen hibalehetőséget rejt a `volatile` flag használata globális memóriában? Hogyan javítható?
5. Miért lehet nem determinisztikus a fence nélküli producer–consumer minta eredménye? Hogyan befolyásolja ezt a GPU architektúrája?

15.4.21 Összegzés

A CUDA memóriamodell a scoped synchronization elvén alapul: minden memóriaművelethez tartozik egy hatókör, amely meghatározza, hogy a memóriaműveletek sorrendje mely szálak számára garantált. A három fő scope — block, device, system — a szinkronizáció szintjét jelöli. A helyes párhuzamos programokban a memóriarendezést explicit fence-ekkel vagy `cuda::atomic` típusokkal biztosítjuk, mert a fordító és a hardver a teljesítmény érdekében szabadon átrendezheti a memóriaműveleteket. A laborban bemutatott producer–consumer minták segítenek megérteni a fence-ek szükségességét és a scope-ok helyes megválasztását. A következő fejezetben a redukciókra térünk át; ott a blokkon belüli szinkronizáció és a memóriarendezés a hatékony és helyes implementáció alapját képezi.

IV. RÉSZ – Párhuzamos algoritmusok

16 Redukciók, scan és histogram

16.1 Redukciók

A redukció az egyik leggyakoribb adatpárhuzamos minta: egy `N` elemű tömb elemeit egyetlen értékké kombináljuk egy asszociatív operátorral, például összegzéssel, maximum- vagy minimumkereséssel, logikai ÉS/VAGY kapcsolattal. Soros végrehajtásban a redukció `N-1` műveletet igényel, és `O(N)` időt vesz igénybe. A GPU párhuzamos modelljében viszont a műveletek egy része egymással párhuzamosan is végrehajtható, így elvileg `O(log N)` lépésszám is elérhető. Ez a fejezet a párhuzamos összegredukciót járja körül, a redukciófa elvétől a gyakorlatban is jól használható warp-szintű shuffle alapú megvalósításig. Az itt bemutatott minták a későbbi fejezetekben a scan, a hisztogram és a könyvtári megoldások alapjául szolgálnak.



16.1. ábra: Redukciófa egy blokkban - A páronkénti összegzés minden lépésben felezi az aktív szálak számát; a warp-szintű shuffle és a shared memory együtt csökkenti a szinkronizációs költséget.

16.1.1 Redukciófa: páronkénti kombinálás rétegenként

A redukciófa lényege, hogy a bemeneti elemeket páronként összekombináljuk, majd az így kapott részeredményeket újra páronként, egészen addig, amíg egyetlen érték marad. Nyolc elem esetén az első rétegben négy párhuzamos összeadás hajtható végre, a másodikban kettő, a harmadikban pedig egy. A lépések száma `log_2 N`, feltéve, hogy minden rétegben elegendő párhuzamos hardver áll rendelkezésre. A hardveres szálak számának csökkenése azonban azt jelenti, hogy a fa alsóbb szintjein egyre kevesebb szál dolgozik, a többi pedig inaktív. A naiv, feltételes indexelésű megoldásban ez warp-divergenciát okoz, mert egy warp 32 szálának egy része aktív, másik része nem.

A redukciófa elvét legegyszerűbben egy shared memory-ba töltött blokkszintű tömbön érthetjük meg. Tegyük fel, hogy `blockDim.x = 256`, és minden szál betölt egy elemet a globális memóriából a shared memory-ba. Ezután a következő ciklus rétegenként felez egy `stride` távolságot:

```
for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
    if (threadIdx.x < s) {
        sdata[threadIdx.x] += sdata[threadIdx.x + s];
    }
    __syncthreads();
}
```

Az első iterációban a 0–127 indexű szálak összeadják a 128–255 indexű elemeket, a másodikban a 0–63 indexűek a 64–127 indexűeket, és így tovább. A `__syncthreads()` minden réteg után biztosítja, hogy a shared memory-beli frissítések minden szál számára láthatóvá váljanak, mielőtt a következő réteg olvassa őket. Ez a minta a redukciófa közvetlen megvalósítása, de van két hátránya: a `threadIdx.x < s` feltétel miatt a warpon belüli divergencia jelentős, és a shared memory hozzáférések száma `N/2 + N/4 + … + 1` N művelet, ami bár párhuzamos, a memóriaforgalom miatt nem optimális.

A divergencia és a felesleges shared memory forgalom csökkenthető, ha a redukciót először a 32 szálból álló warpon belül végezzük el. A warp a CUDA végrehajtási modell alapegysége: a szálak szinkron léptekben hajtódnak végre, és az úgynevezett warp shuffle műveletekkel közvetlenül, regiszterekből cserélhetnek adatot anélkül, hogy shared vagy globális memóriába kellene írniuk.

16.1.2 Warp-szintű redukció `__shfl_down_sync` segítségével

A `__shfl_down_sync(mask, var, delta)` intrinsic egy teljes warpon belül minden aktív szálnak megadja annak a szálnak a `var` értékét, amelynek az indexe `lane + delta`, ha az aktív. A `mask` egy 32 bites bitmaszk, amely a részt vevő szálakat jelöli; teljes warp esetén `0xffffffff`. A művelet szinkron módon fut a warpon belül, ezért az explicit `__syncwarp()` általában nem szükséges, de a maszknak tartalmaznia kell a kommunikáló szálakat, különben a viselkedés nem definiált.

Egy warp összegredukciója a következőképpen valósítható meg:

```
// CUDA C++20, NVIDIA nvcc 12.5 (ellenőrizendő a helyi verzió)
__device__ float warpReduceSum(float val) {
    // Feltételezzük, hogy a teljes warp aktív, ezért a maszk 0xffffffff.
    for (int offset = 16; offset > 0; offset >>= 1) {
        val += __shfl_down_sync(0xffffffff, val, offset);
    }
    return val;
}
```

Az első lépésben minden szál hozzáadja a 16-tal távolabbi szál értékét, így a 0–15. szálak a 0–15 és 16–31 értékek összegét tartalmazzák. A második lépésben a 8-cal távolabbi értéket adjuk hozzá, és így tovább. Öt lépés után a 0. szál birtokolja a teljes warp összegét. A művelet során nem történik shared vagy global memory hozzáférés, csak regiszterek közötti adatcsere, ami lényegesen kisebb késleltetést és nagyobb sávszélességet eredményez.

Fontos, hogy a `__shfl_down_sync` maszkjában minden olyan szálnak szerepelnie kell, amelytől adatot kérünk. Ha a warpban inaktív szálak vannak, például azért, mert a bemeneti feltétel miatt nem minden szál töltött be elemet, akkor a maszkot ennek megfelelően kell módosítani. A legegyszerűbb, ha a redukciót megelőzően a hiányzó elemeket 0-val töltjük fel, és a teljes warp maszkját használjuk.

16.1.3 Blokkszintű redukció: shared memory és warp shuffle kombinációja

Egy blokk tipikusan több warpot tartalmaz (például 256 szál = 8 warp). A blokkszintű redukcióhoz először minden warp kiszámítja a saját részösszegét a fenti warp-szintű redukcióval, majd a warp-összegeket egyetlen szál képviseli. Ezeket a részösszegeket shared memory-ba írjuk, ahol az első warp újabb warp-szintű redukciót hajt végre rajtuk. A megvalósítás a következő:

```
__global__ void reduce_kernel(const float* __restrict__ g_idata,
                              float* __restrict__ g_odata,
                              unsigned int n) {
    extern __shared__ float sdata[];  // dinamikus shared memory
    const unsigned int tid      = threadIdx.x;
    const unsigned int i        = blockIdx.x * blockDim.x + tid;
    const unsigned int lane     = tid & 31;
    const unsigned int warpId   = tid >> 5;

    // Bemenet betöltése: a tömb végét kinullázzuk, hogy a redukció helyes maradjon.
    float val = (i < n) ? g_idata[i] : 0.0f;

    // 1. lépés: warp-szintű redukció
    val = warpReduceSum(val);

    // 2. lépés: minden warp 0. szála kiírja a warp összegét a shared memory-ba
    if (lane == 0) {
        sdata[warpId] = val;
    }
    __syncthreads();  // a shared memory írások láthatóvá tétele

    // 3. lépés: az első warp betölti a warp-összegeket, és újra redukál
    if (warpId == 0) {
        // A warp-összegek száma = blockDim.x / 32.
        // A hiányzó slotokat 0-val töltjük fel a teljes warp maszkhoz.
        const unsigned int numWarps = blockDim.x >> 5;
        val = (lane < numWarps) ? sdata[lane] : 0.0f;
        val = warpReduceSum(val);
        if (lane == 0) {
            g_odata[blockIdx.x] = val;
        }
    }
}
```

Ebben a kernelben minden blokk egy részösszeget számít ki, és azt a `g_odata` tömb `blockIdx.x` indexű elemébe írja. A shared memory méretét dinamikusan adjuk meg a kernel indításakor: `blockDim.x / 32 * sizeof(float)`. Ha a blokkméret 256, akkor ez 8 float, azaz 32 bájt. A `__syncthreads()` biztosítja, hogy minden warp 0. szála által írt érték látható legyen az első warp számára.

A kernel fenti változata egyetlen blokkos, vagyis ha a grid mérete 1, akkor a teljes tömböt egy blokk redukálja, feltéve, hogy `n <= blockDim.x`. Nagyobb tömbök esetén két szintre van szükség: az első kernel a bemeneti tömböt blokkokra osztja, és minden blokk egy részösszeget ír ki, a második kernel pedig ezeket a részösszegeket redukálja egyetlen értékké. Ezt a mintát gyakran kétfázisú redukciónak nevezik. A második fázisban a grid mérete 1, a blokkméret pedig akkora, hogy a részösszegek száma elférjen egy blokkban.

16.1.4 Redukciós sorrend és numerikus pontosság

A lebegőpontos összeadás nem asszociatív: `(a + b) + c` és `a + (b + c)` a kerekítés miatt eltérő eredményt adhat. Ezért a párhuzamos redukció eredménye bitre pontosan általában nem egyezik meg a soros CPU-referenciával. A redukciófa páronkénti sorrendje más kerekítési utat jelent, mint a balról jobbra haladó soros összegzés. A különbség FP32 esetén tipikusan a `10^{-6}` relatív hiba nagyságrendjében mozog, de nagy elemszám vagy kedvezőtlen adateloszlás esetén nagyobb is lehet. FP64 esetén a hiba lényegesen kisebb, de nem tűnik el.

A gyakorlatban ezért a GPU-redukció ellenőrzésekor mindig tűréshatárt kell megadni. Egy elterjedt megközelítés:

```
allowed_error = 1e-4 * sum(abs(x_i))   // FP32 esetén
```

vagy a relatív hiba:

```
rel_error = abs(cpu_sum - gpu_sum) / max(abs(cpu_sum), 1.0f)
```

A determinizmus szempontjából fontos, hogy a párhuzamos redukció sorrendje függ a blokkmérettől, a grid méretétől és a hardver ütemezésétől. Ugyanaz a kernel ugyanazon a GPU-n általában determinisztikus, de ha a blokkméretet megváltoztatjuk, az eredmény kismértékben eltérhet. A könyv későbbi fejezete (Determinizmus és redukciók) részletesen foglalkozik a determinizmussal; itt annyit rögzítünk, hogy a párhuzamos redukciót mindig a numerikus pontosság és a reprodukálhatóság tudatában kell értékelni.

16.1.5 Gyakorlati példa: teljes összegredukció

A következő CUDA C++ példa kétfázisú redukciót valósít meg, hibakezeléssel és CPU-ellenőrzéssel. A kód a labor alapjául szolgál; a GPU-konfigurációt és a mérési naplót a helyi környezethez kell igazítani.

```
// reduce_sum.cu
// CUDA C++20, NVIDIA nvcc 12.5 (ellenőrizendő: nvcc --version)
#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <vector>

// Egyszerű hibamakró a CUDA API-hívások ellenőrzésére.
#define CHECK_CUDA(call)                                                    \
    do {                                                                    \
        cudaError_t err = call;                                             \
        if (err != cudaSuccess) {                                           \
            fprintf(stderr, "CUDA error at %s:%d: %s\n",                    \
                    __FILE__, __LINE__, cudaGetErrorString(err));           \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

// Warp-szintű összegredukció teljes warp aktivitás mellett.
__device__ float warpReduceSum(float val) {
    for (int offset = 16; offset > 0; offset >>= 1) {
        val += __shfl_down_sync(0xffffffff, val, offset);
    }
    return val;
}

// Blokkszintű részösszeg-kernel.
// Minden blokk egy részösszeget ír a d_partial tömbbe.
__global__ void reduce_partial_kernel(const float* __restrict__ g_idata,
                                      float* __restrict__ g_odata,
                                      unsigned int n) {
    extern __shared__ float sdata[];
    const unsigned int tid      = threadIdx.x;
    const unsigned int i        = blockIdx.x * blockDim.x + tid;
    const unsigned int lane     = tid & 31;
    const unsigned int warpId   = tid >> 5;

    float val = (i < n) ? g_idata[i] : 0.0f;

    val = warpReduceSum(val);
```

```
    if (lane == 0) {
        sdata[warpId] = val;
    }
    __syncthreads();

    if (warpId == 0) {
        const unsigned int numWarps = blockDim.x >> 5;
        val = (lane < numWarps) ? sdata[lane] : 0.0f;
        val = warpReduceSum(val);
        if (lane == 0) {
            g_odata[blockIdx.x] = val;
        }
    }
}

// Végső redukciós kernel egyetlen blokkal.
__global__ void reduce_final_kernel(const float* __restrict__ g_idata,
                                    float* __restrict__ g_odata,
                                    unsigned int n) {
    extern __shared__ float sdata[];
    const unsigned int tid      = threadIdx.x;
    const unsigned int lane     = tid & 31;
    const unsigned int warpId   = tid >> 5;

    float val = (tid < n) ? g_idata[tid] : 0.0f;

    val = warpReduceSum(val);

    if (lane == 0) {
        sdata[warpId] = val;
    }
    __syncthreads();

    if (warpId == 0) {
        const unsigned int numWarps = blockDim.x >> 5;
        val = (lane < numWarps) ? sdata[lane] : 0.0f;
        val = warpReduceSum(val);
        if (lane == 0) {
            g_odata[0] = val;
        }
    }
}

// CPU referencia: soros, balról jobbra haladó FP32 összeg.
float cpu_sum(const std::vector<float>& data) {
    float sum = 0.0f;
    for (float x : data) sum += x;
    return sum;
}

int main() {
    // Labor paraméterek: ellenőrizendő a helyi környezetben.
    const unsigned int N = 1u << 22;        // 4 194 304 elem
    const unsigned int blockSize = 256;
    const unsigned int numBlocks = (N + blockSize - 1) / blockSize;
    const unsigned int smemSize = (blockSize / 32) * sizeof(float);

    // GPU konfiguráció (példa, a mérési naplóban frissítendő):
    // NVIDIA GeForce RTX 3060, driver 550.54.14, CUDA 12.5,
    // compute capability 8.6, ECC ki, MIG ki.
    CHECK_CUDA(cudaSetDevice(0));
    cudaDeviceProp prop;
    CHECK_CUDA(cudaGetDeviceProperties(&prop, 0));
```

```
    printf("GPU: %s, CC %d.%d\n", prop.name, prop.major, prop.minor);

    // Bemenet inicializálása: FP32 értékek [0,1) tartományban.
    std::vector<float> h_data(N);
    for (unsigned int i = 0; i < N; ++i) {
        h_data[i] = static_cast<float>(rand()) / RAND_MAX;
    }

    // Device memória allokálása.
    float *d_data, *d_partial, *d_result;
    CHECK_CUDA(cudaMalloc(&d_data, N * sizeof(float)));
    CHECK_CUDA(cudaMalloc(&d_partial, numBlocks * sizeof(float)));
    CHECK_CUDA(cudaMalloc(&d_result, sizeof(float)));

    // Másolás host -> device.
    CHECK_CUDA(cudaMemcpy(d_data, h_data.data(), N * sizeof(float),
                          cudaMemcpyHostToDevice));

    // Első fázis: blokkonkénti részösszegek.
    reduce_partial_kernel<<<numBlocks, blockSize, smemSize>>>
        (d_data, d_partial, N);
    CHECK_CUDA(cudaGetLastError());
    CHECK_CUDA(cudaDeviceSynchronize());

    // Második fázis: részösszegek redukciója.
    reduce_final_kernel<<<1, blockSize, smemSize>>>
        (d_partial, d_result, numBlocks);
    CHECK_CUDA(cudaGetLastError());
    CHECK_CUDA(cudaDeviceSynchronize());

    // Eredmény visszamásolása.
    float h_result = 0.0f;
    CHECK_CUDA(cudaMemcpy(&h_result, d_result, sizeof(float),
                          cudaMemcpyDeviceToHost));

    // CPU referencia és hiba.
    float reference = cpu_sum(h_data);
    float abs_error = std::fabs(reference - h_result);
    float rel_error = abs_error / std::fmax(std::fabs(reference), 1.0f);

    printf("CPU sum      : %.9f\n", reference);
    printf("GPU sum      : %.9f\n", h_result);
    printf("Abszolút hiba: %.9e\n", abs_error);
    printf("Relatív hiba : %.9e\n", rel_error);

    // Egyszerű ellenőrzés FP32 tűréssel.
    const float tolerance = 1e-4f * static_cast<float>(N);
    if (rel_error < 1e-5f) {
        printf("Ellenőrzés OK: a GPU-redukció a tűréshatáron belül van.\n");
    } else {
        printf("Ellenőrzés HIBA: a redukció eltérése túl nagy!\n");
    }

    CHECK_CUDA(cudaFree(d_data));
    CHECK_CUDA(cudaFree(d_partial));
    CHECK_CUDA(cudaFree(d_result));
    return 0;
}
```

A kód a `blockSize = 256` értékkel dolgozik, így egy blokk 8 warpot tartalmaz. A shared memory mérete 8 float = 32 bájt. A `reduce_partial_kernel` indításakor a grid mérete `numBlocks`, ami 4 millió elem és 256-os blokkméret esetén 16 384 blokk. A második kernel egyetlen blokkal redukálja a 16 384 részösszeget; mivel a blokkméret 256, a kernel elején a `tid < n` feltétel

biztosítja, hogy a tömb végét 0-ként kezeljük. Ez a megoldás tetszőleges `N`-re skálázható, amíg a részösszegek száma egyetlen blokkba fér. Ha a tömb ennél is nagyobb, a kétfázisú redukció rekurzívan vagy három fázissal bővíthető.

A fenti kódban a `CHECK_CUDA` makró minden API-hívást ellenőriz, a kernelindítások után pedig `cudaGetLastError` és `cudaDeviceSynchronize` biztosítja, hogy az aszinkron hibák is felszínre kerüljenek. A `cudaFree` hívások a példa rövidsége miatt a program végén egyszerre történnek; a hibakezelést a laborban minden felszabadításnál is alkalmazni kell.

16.1.6 Labor: összegredukció mérése és validációja

A labor célja az összegredukciós kernel futtatása, a mérési napló rögzítése, valamint a mért és a várt eredmény elkülönítése. Az alábbiak egy lehetséges labor-sablont mutatnak be; a konkrét értékeket a saját környezetben kell felvenni, és a mérési naplót ennek megfelelően kitölteni. A megadott GPU-konfiguráció példa, nem ellenőrzött mérés.

Labor paraméterek (példa, helyettesítendő):

- GPU: NVIDIA GeForce RTX 3060
- Driver: 550.54.14
- CUDA: 12.5
- Compute capability: 8.6
- Operációs rendszer: Ubuntu 22.04 LTS
- Fordító: nvcc 12.5, `-O3 -arch=sm_86`
- Bemenet: `N = 4 194 304` darab FP32 véletlenszám `[0,1)` intervallumban
- Blokkméret: 256, grid: 16 384 (első fázis), 1 (második fázis)
- Precision: FP32

Build és futtatás:

```
nvcc -O3 -arch=sm_86 -o reduce_sum reduce_sum.cu
./reduce_sum
```

Ellenőrzés és várt eredmény:

A CPU-referencia egy soros, balról jobbra haladó FP32 összegzés. A GPU-redukció ettől a kerekítési sorrend miatt kismértékben eltérhet. A várt eredményt a CPU-összeg adja, a mért eredményt a GPU-kimenet. A kettőt külön kell naplózni:

Oszlop	Érték
CPU-s referencia (soros FP32)	pl. 2097152.000000000
GPU redukció (kétfázisú, FA)	pl. 2097151.875000000
Abszolút hiba	pl. 0.125000000
Relatív hiba	pl. 5.96e-08
Tűréshatár `1e-4 * N`	419.4304

A példában a CPU-összeg `2097152.0` akkor adódna, ha a véletlenszámok összege pontosan ennyi; a valós mérés ettől eltér. A táblázat értékei illusztrációk, nem mért adatok. A laborban a tényleges futás során kapott értékeket kell rögzíteni, és a hibát a fenti képletek szerint kiszámítani.

Mérési napló (példa, kitöltendő):

Metrika	Érték
Átlagos teljes futásidő (első kernel + második)	pl. 0.85 ms
Első kernel átlagos ideje (10 ismétlés)	pl. 0.80 ms
Második kernel átlagos ideje	pl. 0.05 ms
Effektív sávszélesség (teljes adatmozgatásra)	pl. `N*4 / idő` 20 GB/s
Occupancy (Nsight Compute)	pl. 75%
Regiszterhasználat (első kernel)	pl. 24
Shared memory / blokk	32 bájt
L1/L2 találati arány	pl. Nsight szerint

A mérés előtt a szokásos warm-up legalább 2-3 kernelhívás, majd 10 ismétlés átlaga és szórása szükséges. A futásidőt CUDA eventekkel vagy `std::chrono` + `cudaDeviceSynchronize` méréssel lehet rögzíteni. A sávszélességet a ténylegesen mozgatott adatokra kell számítani: a bemeneti tömb `N*4` bájtját olvassuk, a kimenet néhány bájt, tehát a domináns forgalom a beolvasás.

16.1.7 Tipikus hibák és megelőzésük

1. **Hiányzó `__syncthreads()` a shared memory írása és olvasása között.** A warp-összegek kiírása után az első warp csak akkor olvashatja a shared memory-t, ha minden szál láthatóvá tette az írást. A szinkronizáció elhagyása esetén az eredmény nem definiált, és a hiba nehezen reprodukálható.
2. **Nem megfelelő maszk a warp shuffle-höz.** Ha a warpban nem mind a 32 szál aktív, akkor a `__shfl_down_sync` maszknak pontosan a részt vevő szálakat kell tartalmaznia. A `0xffffffff` használata inaktív szálakkal együtt nem definiált viselkedéshez vezet. A fenti kód ezt úgy kerüli el, hogy a hiányzó elemeket 0-val tölti fel, így a teljes warp aktív.
3. **Bank conflict a shared memory redukcióban.** A klasszikus `sdata[tid] += sdata[tid + s]` minta FP32 esetén általában bank conflict-mentes, mert a hozzáférési távolság `s` és a hozzáférési tartomány `s` elem, ami 32 elemnél kisebb esetén diszjunkt bankokat eredményez. Ha azonban a tömb elemei nem 32 bitesek, vagy a hozzáférési minta stride-ja 32 többszöröse, akkor bank conflict léphet fel. A warp shuffle alapú megoldás ezt teljesen kiküszöböli.
4. **Divergencia a feltételes indexelés miatt.** A `if (tid < s)` típusú redukciós ciklusban a warp egy része inaktív, ami teljesítménycsökkenést okoz. A shuffle-alapú redukció nem igényel ilyen feltételt, mert minden szál aktívan részt vesz a warp belső műveleteiben.
5. **A blokkméret és a shared memory méret helytelen összhangja.** Ha a dinamikus shared memory méretét rosszul adjuk meg, például a warp-összegek számánál kisebbet, akkor a kernel túlírhatja a memóriát, vagy a fordító hibát jelez. A méretet mindig a `blockDim.x / 32 * sizeof(float)` képletből kell számítani, és a kernel indításakor a harmadik argumentumban átadni.

16.1.8 Önellenőrző kérdések

1. Miért `O(log N)` lépésszámú a redukciófa, és milyen hardveres erőforrások korlátozzák ezt a gyakorlatban?
2. Milyen előnyei vannak a `__shfl_down_sync` alapú warp-redukciónak a shared memory-alapú fa-redukcióval szemben?

3. Miért térhet el a GPU-redukció eredménye a soros CPU-összegzéstől, és hogyan adhatunk meg ésszerű tűréshatárt?
4. A kétfázisú redukcióban miért van szükség a második kernelre? Mikor lenne elegendő egyetlen kernel?
5. Milyen hibát okoz, ha a `__syncthreads()` hívást elhagyjuk a warp-összegek shared memory-ba írása után?

Ezek a kérdések a fejezet fő tanulási céljait ellenőrzik: a redukciófa elvének, a warp-szintű redukciónak, a numerikus pontosságnak és a gyakori hibáknak a megértését. A következő alfejezet (Scan (prefix sum)) ezekre az alapokra építve a scan (prefix sum) algoritmust tárgyalja, ahol a redukcióhoz hasonló fa- és warp-szintű minták jelennek meg, de a kimenet nem egyetlen érték, hanem egy teljes tömb.

16.2 Scan (prefix sum)

A scan (prefix sum) az egyik leggyakrabban használt párhuzamos primitív. Egy `x[0..n-1]` bemeneti sorozathoz az **inclusive scan** az `y[i] = x[0] + x[1] + ... + x[i]` kimenetet adja, az **exclusive scan** pedig az `y[0] = 0`, `y[i] = x[0] + ... + x[i-1]` értékeket. A kettő közötti kapcsolat egyszerű: ha az exclusive scan eredményéhez hozzáadjuk az aktuális `x[i]` elemet, inclusive scan-t kapunk; ha az inclusive scan-ből kivonjuk `x[i]`-t, exclusive scan-t kapunk. Ez a látszólag apró eltérés a gyakorlatban döntő, mert a legtöbb algoritmus – például a stream compaction, a radix sort, a sparse mátrix-vektor szorzás vagy a párhuzamos memóriacímzés – kifejezetten exclusive scan-t igényel.

A scan látszólag soros rekurziónak tűnik, hiszen minden kimenet az összes megelőző bemenettől függ. A GPU-n mégis hatékonyan párhuzamosítható, mert az asszociatív összegzési művelet lehetővé teszi a részösszegek fa-szerű kombinálását. Ebben a fejezetben a Blelloch-féle úgynevezett **work-efficient scan**-t építjük fel: ez a redukcióhoz hasonló up-sweep fázisból, majd egy down-sweep fázisból áll, és a munkaigénye O(n), a mélysége pedig O(log n). A fejezet előfeltételezi a redukciók ismeretét (Redukciók), ezért az up-sweep fázist nem magyarázzuk újra teljes részletességgel, csak a scan-specifikus részekre összpontosítunk.

16.2.1 1. Inclusive és exclusive scan definíció szerint

Legyen $\oplus$ egy asszociatív bináris művelet, például összeadás, szorzás vagy maximumképzés. A továbbiakban az olvashatóság kedvéért összeadást írunk, de minden képlet érvényes tetszőleges asszociatív műveletre, ha a neutrális elemet is definiáljuk. Az **inclusive scan**:

$$y_{\text{incl}}[i] = \bigoplus_{j=0}^{i} x[j],$$

míg az **exclusive scan**:

$$y_{\text{excl}}[0] = 0, \quad y_{\text{excl}}[i] = \bigoplus_{j=0}^{i-1} x[j], \quad i > 0.$$

A `0` itt az összeadás neutrális eleme; általános műveletnél a neutrális elemet kell használni. A két definíció közötti átjárás egyetlen elemenkénti művelettel megvalósítható:

$$y_{\text{incl}}[i] = y_{\text{excl}}[i] \oplus x[i], \qquad y_{\text{excl}}[i] = y_{\text{incl}}[i] \oplus x[i]^{-1},$$

ahol `x[i]^{-1}` az inverz elem, ha létezik (összeadásnál a negatív). Ha nincs inverz, az exclusive scan-t közvetlenül kell előállítani, vagy egy eltolt másolatot kell alkalmazni.

A gyakorlati alkalmazások szempontjából az exclusive scan azért fontos, mert megadja, hogy egy adott elem elé hány elem kerül a kimeneti tömbben. Például stream compaction esetén bemenetként egy predikátumtömböt kapunk, ahol a teljesülő elemeket 1-gyel, a többit 0-val jelöljük. Az exclusive scan megadja a célindexet: ha a predikátum az `i`-edik helyen igaz, akkor az elem a `scan_excl[i]` indexre kerül. Ez a minta a párhuzamos programozás egyik alapeleme.

16.2.2 2. Naiv párhuzamos megközelítés és a work-efficient scan

A legegyszerűbb párhuzamos scan a **Hillis–Steele-féle** algoritmus. Minden iterációban minden szál hozzáadja a tőle `2^{d-1}` távolságra lévő elem értékét a saját értékéhez:

```
for d = 1 to log2(n):
    for k in parallel:
        if k >= 2^(d-1):
            x[k] += x[k - 2^(d-1)]
```

Ez a forma inclusive scan-t ad. A munkaigénye O(n log n), mert az összes elemet log2(n)-szer módosítja. A GPU-n ez a felesleges munka gyakran elfogadható kis tömbök esetén, de nagy adathalmazoknál pazarló.

A **Blelloch-scan** ezzel szemben O(n) munkát végez, és a mélysége is O(log n). Két fázisból áll:

1. **Up-sweep (reduce):** a redukcióhoz hasonlóan páronként összegzünk egy felfelé haladó fában, de az eredményeket a fában tároljuk, nem csak a gyökeret. A gyökérben a teljes összeg jelenik meg.
2. **Down-sweep:** a fa gyökerét nullára állítjuk, majd lefelé haladva minden csomópontnál az aktuális (bal oldali) prefix értéket átmásoljuk a bal gyermekbe, a jobb gyermekbe pedig a bal gyermek régi értékét és az aktuális értéket összegezve írjuk. A levél elemekben az exclusive scan értéke jelenik meg.

A Blelloch-scan tehát közvetlenül exclusive scan-t produkál. Inclusive scan-t úgy kapunk, ha az algoritmus után hozzáadjuk az eredeti bemeneti elemet, vagy ha a down-sweep során a neutrális elem helyett az utolsó levelet `x[n-1]`-gyel inicializáljuk, de a részletek implementációfüggők.

A gyakorlatban a Blelloch-scan a legszélesebb körben használt blokk-szintű scan, mert a shared memory-ban tárolt adatokkal jól illeszkedik a GPU-szálak indexeléséhez, és a `__syncthreads()` hívásokkal a fázisok közötti konzisztencia garantálható.

16.2.3 3. Blokk-szintű Blelloch-scan CUDA-ban

A blokk-szintű megvalósítás feltételezi, hogy a bemenet hossza megegyezik a blokkmérettel, vagy legalábbis annál nem nagyobb. A szokásos gyakorlat szerint `n` a blokkméret, és a bemenet a shared memory-ba kerül. Az alábbi kód egy teljes, fordítható CUDA kernelt mutat be. A kód a fejezet laborjaként is szolgál, ezért a hibakezelést a könyv szabályai szerint beépítettük.

```cpp
// nyelv: CUDA C++17
// fordítás: nvcc -O3 -arch=sm_80 blelloch_scan.cu -o blelloch_scan

#include <cuda_runtime.h>
#include <cstdio>
#include <cstdlib>
#include <cassert>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err = (call);                                           \
        if (err != cudaSuccess) {                                           \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__, \
                    cudaGetErrorString(err));                               \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

// Blokkszintű Blelloch exclusive scan.
// Feltételek:
//  - n == blockDim.x
//  - n hatványa kettő (power of two)
//  - a bemenet és kimenet külön global memory területen van,
//    de a kernel shared memory-ban dolgozik.
template <int BLOCK_SIZE>
__global__ void blelloch_scan_kernel(const float* __restrict__ input,
                                     float* __restrict__ output) {
    __shared__ float temp[BLOCK_SIZE];
    int tid = threadIdx.x;

    // Bemenet betöltése shared memory-ba.
    temp[tid] = input[tid];
    __syncthreads();

    // Up-sweep: redukció a fán felfelé.
    for (unsigned int stride = 1; stride < BLOCK_SIZE; stride <<= 1) {
        unsigned int idx = (tid + 1) * (stride << 1) - 1;
        if (idx < BLOCK_SIZE) {
            temp[idx] += temp[idx - stride];
        }
        __syncthreads();
    }

    // Down-sweep: az exclusive scan értékek előállítása.
    if (tid == 0) {
        temp[BLOCK_SIZE - 1] = 0.0f;   // a gyökér neutrális elemre állítása
    }
    __syncthreads();

    for (unsigned int stride = BLOCK_SIZE >> 1; stride > 0; stride >>= 1) {
        unsigned int idx = (tid + 1) * (stride << 1) - 1;
        if (idx < BLOCK_SIZE) {
            float left_value = temp[idx - stride];
            temp[idx - stride] = temp[idx];
            temp[idx] += left_value;
        }
        __syncthreads();
    }
    __syncthreads();

    // Kimenet írása global memory-ba.
    output[tid] = temp[tid];
}
```

A kernel működésének megértéséhez kövessük végig egy `BLOCK_SIZE = 8` példán. Az up-sweep a következő elemeket módosítja:

- `stride = 1`: a páros indexű elemek (1, 3, 5, 7) hozzáadják a tőlük eggyel kisebb indexű elem értékét.
- `stride = 2`: a 3 és 7 indexű elemek hozzáadják az 1 és 5 indexű elemek értékét.
- `stride = 4`: a 7 indexű elem hozzáadja a 3 indexű elem értékét.

Eredmény: `temp[7]` a teljes összeg, a többi elem a részfa-összegeket tartalmazza. A down-sweep során a gyökérbe 0 kerül, majd a fa mentén lefelé haladva a bal gyermek kapja a szülő aktuális értékét, a jobb gyermek pedig a bal gyermek régi értékét és a szülő aktuális értékét összegezve. A levelekben az exclusive scan jelenik meg.

Fontos részlet: a `__syncthreads()` hívás a hurkokon belül minden iteráció után kötelező. Ha kihagyjuk, a szálak nem garantáltan látják a korábbi iterációban módosított értékeket, és a kimenet versenyhelyzet miatt nem determinisztikus lesz. A `cudaDeviceSynchronize()` a kernel után, valamint a `cudaGetLastError()` a launch hibák ellenőrzésére szolgál.

A fenti kód csak akkor helyes, ha `n` hatványa kettő. A gyakorlatban a bemenet hossza ritkán ilyen; ilyenkor a leggyakoribb megoldás a bemenet kiegészítése a következő kettőhatványra, ahol a fiktív elemek értéke a neutrális elem (összeadásnál 0). A kiegészítés a kernelben a feltételes betöltéssel kezelhető: az `if (tid < n)` feltételt kell alkalmazni a betöltésnél és a kiírásnál, míg a shared memory többi elemét nullával kell inicializálni. A laborban ezt a kiterjesztést is érdemes megvalósítani.

16.2.4 4. Grid-szintű scan: több blokk együttműködése

Egyetlen blokk csak korlátozott számú elemet tud kezelni, hiszen a blokkméret jellemzően legfeljebb 1024 szál. Nagy bemenetek esetén a scan-t több blokkra kell bontani. A standard megközelítés három lépésből áll:

1. **Blokkszintű scan:** minden blokk a saját szegmensén végrehajtja a Blelloch-scan-t (vagy bármely blokk-szintű scan-t). A blokk az exclusive scan eredményét írja a kimeneti tömb saját szegmensébe, és a szegmens teljes összegét egy külön `block_sums` tömbbe menti.
2. **Részösszegek scan-je:** a `block_sums` tömbön lefuttatunk egy exclusive scan-t. Ez lehet egyetlen blokk által végzett scan, ha a blokkok száma kicsi, vagy rekurzívan ismételhető a fenti lépés.
3. **Offset hozzáadása:** minden blokk a saját `block_sums`-beli exclusive scan értékét hozzáadja a saját szegmensének minden eleméhez.

Ez a módszer három kernelindítást igényel, és a köztes tárolók mérete O(num_blocks). A módszer jól párhuzamosítható, de a második lépésben a blokkok közötti szinkronizációt kernelhatárokkal valósítja meg. A modern CUDA-ban a **decoupled look-back** technika ezt a szinkronizációt egyetlen kernelben, az utolsó blokk által publikált összegek dinamikus olvasásával oldja meg; ezt a CUB `DeviceScan` algoritmusa használja.

A grid-szintű scan másik fontos szempontja a **streaming** vagy **perzisztens** megközelítés: a grid-stride loop segítségével egy kernel több szegmenst is feldolgozhat, így a blokkok száma és a feldolgozási sorrend tervezhető. Ennek részletei a Grid-stride loop és perzisztens kernelek alfejezetben bemutatott grid-stride loopra épülnek.

Egy egyszerű, de működő grid-szintű exclusive scan vázlata:

```
__global__ void scan_block_kernel(const float* input, float* output,
                                  float* block_sums, int n) {
    // 1. blokkszintű Blelloch exclusive scan a saját szegmensen
    // 2. block_sums[blockIdx.x] = a szegmens teljes összege
}

__global__ void add_offset_kernel(float* output, const float* block_offsets,
                                  int n) {
    // minden elemhez hozzáadja a blokkjának offsetjét
}

// Host oldalon:
// 1. scan_block_kernel<<<num_blocks, block_size, shmem, stream>>>
// 2. egy kisebb exclusive scan a block_sums-on
// 3. add_offset_kernel<<<num_blocks, block_size, 0, stream>>>
```

A fenti vázlatban a második lépés egyetlen blokkal is elvégezhető, ha a blokkok száma legfeljebb `BLOCK_SIZE`. Ha a blokkok száma ennél nagyobb, a rekurzív bővítés ugyanezt a mintát követi.

A grid-szintű scan gyakorlati megvalósítása során kritikus a hibakezelés: a `block_sums` tömböt minden kernelindítás után ellenőrizni kell, és a végén a kimenetet CPU-n össze kell hasonlítani egy szekvenciális referenciával. A laborban egy ilyen ellenőrző függvényt is be kell mutatni.

16.2.5 5. Gyakori hibák és megelőzésük

A scan implementálásakor a leggyakoribb hibák a következők:

1. **Hiányzó `__syncthreads()` a scan hurkokban.** A Blelloch-scan minden iterációja után szükséges, mert a szálak a shared memory-ban más szálak által írt értékeket olvasnak. Ha a szinkronizáció elmarad, a kimenet nem determinisztikus, és a compute-sanitizer racecheck eszköz (Compute-sanitizer: memcheck és racecheck) jelzi a versenyhelyzetet.

2. **Nem hatványa kettő hosszúság.** A fenti kernel feltételezi, hogy `BLOCK_SIZE` pontosan a blokkméret, és a bemeneti hossz ennyi. Ha a bemenet rövidebb, a feltétel nélküli hozzáférés shared memory túlindexeléshez vezet. A megoldás a kiegészítés neutrális elemekkel, vagy egy általánosabb, bármilyen hosszúságot kezelő kernel írása.

3. **Az exclusive és inclusive összekeverése.** A Blelloch-scan exclusive eredményt ad. Ha a felhasználó inclusive scan-t vár, a kimenethez hozzá kell adni az eredeti bemenetet. Ez a hiba a tesztelés során azonnal kiderül, ha a referenciával összehasonlítjuk. A laborban ezt a lépést expliciten kell dokumentálni.

4. **Shared memory bank conflict.** A `temp` tömb hozzáférése a fenti indexeléssel (`idx = (tid+1)*2*stride - 1`) általában nem okoz bank conflictet, mert a szálak azonos bankokhoz különböző időpontokban férnek hozzá, de ha a tömb méretét nem megfelelően választjuk meg, vagy ha a kiegészítés miatt a tényleges használt méret eltér, a bankkonfliktusok ronthatják a teljesítményt. A `__shared__ float temp[BLOCK_SIZE]` esetén a bankok száma 32, és a float méret miatt a szomszédos elemek különböző bankokban vannak. A probléma jellemzően akkor jelentkezik, ha a hozzáférési mintázat `stride` 32 többszöröse, ami itt nem áll fenn.

5. **Global memory hozzáférési minta nem coalescing.** A bemenet és kimenet írása a fenti példában szomszédos szálak szomszédos memóriacímekre írnak, ami coalescing. Ez a szabály a grid-szintű scan offset-hozzáadásánál is fontos, ahol a blokkon belüli szálak a saját `tid` indexükkel férnek hozzá a kimenethez.

6. **A neutrális elem hibás definiálása.** A Blelloch-scan a gyökérben a neutrális elemet használja. Összeadásnál ez 0, szorzásnál 1, maximumképzésnél a típus legkisebb eleme. Ha a műveletet általánosítjuk, a neutrális elemet paraméterként kell átadni, és a laborban ezt dokumentálni kell.

16.2.6 6. Labor: Blelloch-scan implementálása

A labor célja egy teljes, hibakezeléssel ellátott CUDA C++ program elkészítése, amely a fenti `blelloch_scan_kernel` segítségével exclusive scan-t számít egy véletlenszerű float tömbre, és az eredményt CPU-referenciával hasonlítja össze. A laborhoz tartozó kód, buildparancs, futtatási parancs és mérési napló az alábbiakban látható.

16.2.7 6.1. A teljes CUDA program

```
// nyelv: CUDA C++17
// fordítás: nvcc -O3 -arch=sm_80 blelloch_scan_lab.cu -o blelloch_scan_lab

#include <cuda_runtime.h>
#include <cstdio>
#include <cstdlib>
#include <cassert>
#include <cmath>
#include <vector>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err = (call);                                           \
        if (err != cudaSuccess) {                                           \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__, \
                    cudaGetErrorString(err));                               \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

template <int BLOCK_SIZE>
__global__ void blelloch_scan_kernel(const float* __restrict__ input,
                                     float* __restrict__ output) {
    __shared__ float temp[BLOCK_SIZE];
    int tid = threadIdx.x;

    temp[tid] = input[tid];
    __syncthreads();

    // Up-sweep
    for (unsigned int stride = 1; stride < BLOCK_SIZE; stride <<= 1) {
        unsigned int idx = (tid + 1) * (stride << 1) - 1;
        if (idx < BLOCK_SIZE) {
            temp[idx] += temp[idx - stride];
        }
        __syncthreads();
    }

    // Down-sweep
    if (tid == 0) {
        temp[BLOCK_SIZE - 1] = 0.0f;
    }
    __syncthreads();

    for (unsigned int stride = BLOCK_SIZE >> 1; stride > 0; stride >>= 1) {
```

```
        unsigned int idx = (tid + 1) * (stride << 1) - 1;
        if (idx < BLOCK_SIZE) {
            float left_value = temp[idx - stride];
            temp[idx - stride] = temp[idx];
            temp[idx] += left_value;
        }
        __syncthreads();
    }
    __syncthreads();

    output[tid] = temp[tid];
}

// CPU referencia: exclusive scan
void cpu_exclusive_scan(const std::vector<float>& input,
                        std::vector<float>& output) {
    int n = input.size();
    output.resize(n);
    if (n == 0) return;
    output[0] = 0.0f;
    for (int i = 1; i < n; ++i) {
        output[i] = output[i-1] + input[i-1];
    }
}

int main() {
    constexpr int BLOCK_SIZE = 256;   // hatvány kettő
    constexpr int N = BLOCK_SIZE;     // labor: egyetlen blokk

    // Host bemenet
    std::vector<float> h_input(N);
    for (int i = 0; i < N; ++i) {
        h_input[i] = static_cast<float>(i % 7) * 0.25f - 1.0f;
    }

    // Device memória
    float *d_input = nullptr, *d_output = nullptr;
    CUDA_CHECK(cudaMalloc(&d_input, N * sizeof(float)));
    CUDA_CHECK(cudaMalloc(&d_output, N * sizeof(float)));
    CUDA_CHECK(cudaMemcpy(d_input, h_input.data(), N * sizeof(float),
                          cudaMemcpyHostToDevice));

    // Kernel launch: grid=1, block=BLOCK_SIZE, shared=0, stream=0
    blelloch_scan_kernel<BLOCK_SIZE><<<1, BLOCK_SIZE, 0, 0>>>(d_input, d_output);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    // Kimenet visszamásolása
    std::vector<float> h_output(N);
    CUDA_CHECK(cudaMemcpy(h_output.data(), d_output, N * sizeof(float),
                          cudaMemcpyDeviceToHost));

    // CPU referencia
    std::vector<float> h_reference;
    cpu_exclusive_scan(h_input, h_reference);

    // Ellenőrzés
    float max_error = 0.0f;
    for (int i = 0; i < N; ++i) {
        max_error = std::max(max_error, std::abs(h_output[i] - h_reference[i]));
    }
    printf("Max abszolút hiba: %e\n", max_error);
    assert(max_error < 1e-5f);
```

```
    // Takarítás
    CUDA_CHECK(cudaFree(d_input));
    CUDA_CHECK(cudaFree(d_output));

    return 0;
}
```

A program egyetlen blokkot indít `BLOCK_SIZE = 256` szállal, és a bemenet hossza is ennyi. A `cudaGetLastError()` ellenőrzi a kernel indításának hibáját, a `cudaDeviceSynchronize()` pedig bevárja a végét. A kimenetet CPU-n ellenőrizzük a referenciával; az eltérés float pontosság mellett kisebb kell legyen, mint `1e-5`.

16.2.8 6.2. Build és futtatási parancs

A környezettől függően a buildparancs:

```
nvcc -O3 -arch=sm_80 blelloch_scan_lab.cu -o blelloch_scan_lab
./blelloch_scan_lab
```

Ha a GPU nem `sm_80` (például Ampere A100), a `-arch` értékét a tényleges compute capability-hez kell igazítani. A laborban ezt rögzíteni kell. A mérési naplóban a következő adatokat kell feltüntetni:

- GPU modell: például `NVIDIA A100-SXM4-40GB`
- Driver: például `535.104.05`
- CUDA: például `12.2`
- Compute capability: `80`
- OS: például `Ubuntu 22.04 LTS`
- Power limit / ECC: például `400 W / ECC on`
- Blokkméret: `256`
- Bemenet hossza: `256`

16.2.9 6.3. Mérési napló és teljesítmény

A mérés során a kernel futásidejét legalább 5–10 ismétléssel kell mérni, előtte warm-up futtatással. Az alábbi táblázat egy **illusztratív** napló; az értékek nem valós mérésből származnak, hanem a laborban kitöltendő sablonként szolgálnak. A könyv szabályai szerint a nem futtatott benchmarkot nem szabad ellenőrzöttként bemutatni, ezért a táblázatot minden esetben a saját mérési adatokkal kell helyettesíteni.

Futtatás sorszáma	Kernel idő (µs)	Megjegyzés
Warm-up	–	nem mérjük
1	1.42	
2	1.38	
3	1.41	
4	1.39	
5	1.45	
Átlag	1.41	
Szórás	0.026	
Percentilis (p50)	1.41	

A kernel idő mérése `cudaEventRecord` vagy `cudaEventElapsedTime` segítségével történhet. A laborban az Nsight Compute vagy `nsys` használata opcionális, de a teljesítményállításokhoz a kernel metrikáit – occupancy, regiszterhasználat, shared memory – is érdemes rögzíteni.

A **várt eredmény** a matematikai definícióból következik: a kimenet az exclusive scan, tehát a 0. indexű elem 0, az 1. indexű elem az első bemenet, és így tovább. A **mért eredményt** a program által kiírt `h_output` tömb adja, amelyet a labor során ki kell nyomtatni vagy fájlba menteni. A kettőt a laborjegyzőkönyvben külön oszlopban kell feltüntetni, és a maximális eltérést is dokumentálni kell.

16.2.10 7. Összefoglalás és továbblépés

A scan a párhuzamos algoritmusok egyik leggyakrabban használt építőköve. A Blelloch-scan O(n) munkával és O(log n) mélységgel állítja elő az exclusive scan-t, és a GPU-blokkokon belül a shared memory és a `__syncthreads()` segítségével hatékonyan implementálható. Az inclusive és exclusive közötti átváltás egyetlen elemenkénti művelet, de a legtöbb downstream algoritmus az exclusive formát igényli.

A következő fejezet (Histogram) a histogramra épít, amely gyakran a scan segítségével valósítja meg a tömörítést vagy a címzést. A jelen fejezetben bemutatott blokk-szintű scan közvetlenül újrafelhasználható ott is.

16.2.11 Önellenőrző kérdések

1. Mi a különbség az inclusive és exclusive scan között? Adj egy példát a `[3, 1, 7, 2]` bemenetre mindkét kimenetre.
2. A Blelloch-scan up-sweep fázisa milyen műveletet valósít meg? Hogyan kapcsolódik a redukcióhoz?
3. Miért van szükség `__syncthreads()`-re a down-sweep hurkon belül? Mi történne, ha elhagynánk?
4. Hogyan terjeszted ki a blokk-szintű Blelloch-scan-t grid-szintű scan-né? Milyen köztes tárolóra van szükség?
5. A laborban az exclusive scan kimenetét hogyan ellenőrizted a CPU-referenciával? Milyen hibakorlátot használtál, és miért?

16.3 Histogram

A hisztogram a párhuzamos számítás egyik leggyakoribb építőköve: adott egy bemeneti sorozat és egy véges számú rekesz (bin); minden elemhez hozzárendelünk egy rekeszt, majd megszámoljuk, hány elem esik az egyes rekeszekbe. Az eredmény egy gyakorisági eloszlás, amelyet képfeldolgozásban (színtónus-hisztogram), statisztikai elemzésben, gépi tanulási jellemzők számításakor és szimulációkban is használunk. GPU-n a hisztogram számítása elsőre egyszerűnek tűnik: minden szál feldolgoz egy elemet, és növeli a megfelelő rekesz számlálóját. A kihívást az jelenti, hogy több szál egyszerre ugyanabba a rekeszbe írhat – ezt a versenyhelyzetet atomi műveletekkel vagy privatizációval kezeljük. Ez a fejezet a hisztogram mint atomi és privatizált művelet fogalmát járja körbe, bemutatja a blokkszintű privatizációt, elemzi a shared memory bank conflict jelenségét, és egy ellenőrizhető laborpéldán keresztül méri a két megközelítés teljesítményét.

16.3.1 1. A hisztogram mint atomi művelet

A legegyszerűbb GPU-s megvalósításban minden bemeneti elemhez egy szál tartozik. A szál kiszámítja az elem rekeszindexét, majd egy atomi összeadást hajt végre a globális memóriában tárolt hisztogramtömbön. Az `atomicAdd` garantálja, hogy az olvasás-módosítás-írás műveletsor oszthatatlan, így a végeredmény helyes marad még akkor is, ha több szál egyszerre ugyanabba a rekeszbe ír.

A naiv kernel CUDA C++-ban:

```
// CUDA C++ (C++17)
constexpr int NUM_BINS = 64;

__global__ void hist_naive(const int* data, int* hist, int n)
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = gridDim.x * blockDim.x;

    for (; idx < n; idx += stride) {
        int bin = data[idx] % NUM_BINS;
        atomicAdd(&hist[bin], 1);
    }
}
```

A grid-stride loop lehetővé teszi, hogy a kernel akkor is lefussa a teljes bemenetet, ha az elemszám nagyobb, mint az indított szálak száma. A bin számítása itt a `%` operátorral történik; ha a rekeszek száma kettő hatványa, bitműveletttel (`& (NUM_BINS-1)`) gyorsabban elvégezhető, de az általánosság kedvéért a modulo marad.

A naiv megoldás előnye a rendkívüli egyszerűség: nincs szükség bonyolult szinkronizációra, a kód kevés sorból áll, és az atomi műveletek garantálják a helyes eredményt. Hátránya viszont a súlyos teljesítményprobléma: ha sok elem esik ugyanabba a rekeszbe, a globális memóriában lévő `hist[bin]` címre irányuló atomi műveletek sorosítva hajtódnak végre. A globális atomi művelet egy read-modify-write ciklus, amely a memóriavezérlőn keresztül megy; nagy kontenció esetén a szálak várakozni kényszerülnek, és a kernel átbocsátóképessége drasztikusan lecsökken. Mérések szerint erősen ferde eloszlásnál a naiv atomi megoldás akár egy nagyságrenddel lassabb is lehet, mint a privatizált változat.

16.3.2 2. Privatizált hisztogram: blokkszintű shared memory

A kontenció csökkentésének alapötlete az, hogy a globális atomi műveleteket részben kiváltjuk gyorsabb, blokkon belüli műveletekkel. Minden szálblokk saját, privát hisztogramot tart fenn a shared memóriában. A blokk szálai a bemeneti elemek feldolgozása során csak ebbe a privát tömbbe írnak atomi módon. A blokk végén, egy `__syncthreads()` után a privát hisztogramot egyetlen globális atomi összeadással egyesítik a globális hisztogrammal. Így a globális atomi műveletek száma blokkonként `NUM_BINS` darabra csökken, szemben a naiv megoldás `N` darabjával.

A privatizált kernel:

```
// CUDA C++ (C++17)
__global__ void hist_shared(const int* data, int* hist, int n)
{
    __shared__ int shist[NUM_BINS];

    // 1. A privát hisztogram inicializálása nullára
    for (int i = threadIdx.x; i < NUM_BINS; i += blockDim.x) {
```

```
        shist[i] = 0;
    }
    __syncthreads();

    // 2. Feldolgozás: atomi összeadás a shared memóriában
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = gridDim.x * blockDim.x;
    for (; idx < n; idx += stride) {
        int bin = data[idx] % NUM_BINS;
        atomicAdd(&shist[bin], 1);
    }
    __syncthreads();

    // 3. A privát hisztogram egyesítése a globális tömbbel
    for (int i = threadIdx.x; i < NUM_BINS; i += blockDim.x) {
        atomicAdd(&hist[i], shist[i]);
    }
}
```

A shared memóriás `atomicAdd` lényegesen gyorsabb, mint a globális megfelelője, mert a shared memória a blokkhoz közeli, nagy sávszélességű memória, és az atomi művelet hardveres támogatása is hatékonyabb. Ráadásul a globális atomi műveletek száma drámaian lecsökken, ami a memóriavezérlő terhelését mérsékli.

A privatizált megoldás korlátja a shared memória mérete. A `NUM_BINS * sizeof(int)` értéknek bele kell férnie a blokk számára elérhető shared memória keretbe (jellemzően 48–99 KiB, opcionálisan nagyobb). Ha a rekeszek száma nagyon nagy (például több ezer), akkor a privát hisztogram nem fér el, és részleges privatizációra vagy többszintű megoldásra van szükség. Ilyenkor az egyik lehetőség, hogy a rekeszeket csoportokra osztjuk, és blokkonként csak egy csoportot tárolunk shared memóriában; a feldolgozást többször is elvégezzük, vagy a bemenetet partícionáljuk.

A blokkméret és a gridméret megválasztása is fontos. A grid-stride loop használatával a grid méretét az SM-ek számának többszörösére állíthatjuk, így a kernel perzisztens maradhat, és elkerüljük a túl sok blokk indítási költségét. A block méretet úgy érdemes megválasztani, hogy az occupancy magas legyen, de a shared memória foglalása ne korlátozza túlzottan az aktív blokkok számát. Jelen példában `NUM_BINS=64` és `blockDim.x=256` esetén a shared memória 256 bájt, ami elhanyagolható; a blokk méretét a feldolgozási teljesítmény és a regiszterhasználat határozza meg.

16.3.3 3. Bank conflict a privatizált hisztogramban

A shared memória 32 bankre van osztva; minden bank egy 4 bájtos szót szolgál ki ciklusonként. Ha egy warp különböző szálai ugyanabba a bankbe írnak, de nem ugyanabba a szóba, a hozzáférés ütközik, és a hardver több lépésben szolgálja ki – ezt nevezzük bank conflictnek. A hisztogram privatizált implementációjában a `shist[bin]` elérések bin indexei gyakran véletlenszerűek, ezért a bank conflict valószínűsége nem elhanyagolható.

Tegyük fel, hogy `NUM_BINS=64`, és a bin indexek 0 és 32. A bank indexet a `bin % 32` képlet adja, tehát mindkettő a 0. bankbe esik. Ha egy warp két különböző szálának bin indexe éppen 0 és 32, akkor a hozzáférésük bank conflictet okoz, mert különböző szavakhoz férnek hozzá ugyanabban a bankben. Ha viszont több szál ugyanabba a konkrét rekeszbe ír (azonos bin index), az atomi művelet nem bank conflictként, hanem atomi sorosítási problémaként jelentkezik; a hardver a read-modify-write műveletet szálanként hajtja végre.

A bank conflict csökkentésének egyik gyakori technikája a padding (kitöltés). A shared memória tömböt úgy deklaráljuk, hogy a szomszédos rekeszek közé üres elemeket illesztünk, így a rekeszek

bankbeli elhelyezkedése megváltozik. Például ha a bin indexet megszorozzuk egy páratlan számmal (mondjuk 5-tel), akkor a `shist[bin * 5]` címek bank indexei permutálódnak. A paddinges kernel:

```
// CUDA C++ (C++17)
constexpr int PAD = 2;                 // stride = 2*PAD+1 = 5
constexpr int PAD_STRIDE = 2 * PAD + 1;

__global__ void hist_shared_padded(const int* data, int* hist, int n)
{
    __shared__ int shist[NUM_BINS * PAD_STRIDE];

    for (int i = threadIdx.x; i < NUM_BINS * PAD_STRIDE; i += blockDim.x) {
        shist[i] = 0;
    }
    __syncthreads();

    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = gridDim.x * blockDim.x;
    for (; idx < n; idx += stride) {
        int bin = data[idx] % NUM_BINS;
        int padded_bin = bin * PAD_STRIDE;
        atomicAdd(&shist[padded_bin], 1);
    }
    __syncthreads();

    for (int i = threadIdx.x; i < NUM_BINS; i += blockDim.x) {
        atomicAdd(&hist[i], shist[i * PAD_STRIDE]);
    }
}
```

Itt a `PAD_STRIDE=5` páratlan szám, így a 0–63 bin indexek bank indexei a `(bin * 5) % 32` kifejezéssel számolva nem lesznek egyenletes leképezés, de a 0. és 32. bin továbbra is ugyanabba a bankbe esik, mert `32 * 5 % 32 = 0`. A padding tehát nem szünteti meg teljesen a bank conflictet 64 rekesz esetén, de ritkítja az ütközéseket, és a gyakorlatban sok esetben mérhető javulást hoz. Ha a rekeszek száma legfeljebb 32, és a stride páratlan, akkor a leképezés tökéletes permutációt ad, és a bank conflict megszűnik.

Haladó technika a per-warp privatizáció, ahol minden warp saját hisztogramot tart fenn. Ezt regiszterekben (ha a rekeszszám kicsi) vagy shared memóriában valósíthatjuk meg, és a warp szintű műveletek – például a `__match_any_sync` vagy a `__shfl_sync` – segítségével a warp-on belüli atomi műveleteket is hatékonyabban kezelhetjük. Ez a módszer a Warp intrinsics: shuffle, vote és match (warp intrinsics) fejezetben bemutatott eszközökre épít, ezért itt csak utalunk rá.

A bank conflict mérésére az Nsight Compute `l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum` és hasonló metrikái szolgálnak. A laborfeladat részeként érdemes ezeket is ellenőrizni, hogy a padding hatását kvantitatívan igazoljuk.

16.3.4 4. Tipikus hibák és helyes megoldások

A hisztogram implementálása során több gyakori hiba fordul elő. Az első a shared memória inicializálásának elmulasztása. A `shist` tömb tartalma definiálatlan, ha nem nullázzuk minden blokk elején, ezért a végeredmény szeméttel szennyezett lesz. A helyes megoldás a fenti kódban látható ciklus, amely a blokk minden szálával párhuzamosan nullázza a tömb elemeit, majd `__syncthreads()` hívás következik.

A második tipikus hiba a szinkronizáció elhagyása. A privatizált kernelben két `__syncthreads()` hívásra van szükség: az első biztosítja, hogy az inicializálás befejeződjön, mielőtt bármely szál elkezdi a feldolgozást; a második pedig garantálja, hogy minden szál befejezte a privát hisztogram frissítését, mielőtt a blokk elkezdi az egyesítést. Ha ezek hiányoznak, a szálak versenyt futnak egymással, és az eredmény nemdeterminisztikus lesz.

A harmadik hiba az out-of-range bin index. Ha a bemeneti adatok nincsenek korlátozva a `[0, NUM_BINS)` intervallumba, a `%` operátor eredménye negatív szám is lehet (C++-ban a maradék előjele az osztandó előjelét követi), ami a tömb határain kívülre mutat. A helyes megoldás a bemeneti értékek normalizálása vagy a bin index kiszámításakor abszolút érték és tartomány-ellenőrzés alkalmazása. A laborban determinisztikus bemenetet használunk, ahol ez a hiba nem fordul elő, de éles kódban mindig védeni kell.

A negyedik hiba a shared memória méretének túlbecslése. Ha a rekeszek száma nagy, a `__shared__` deklaráció meghaladhatja a statikus shared memória limitet. Ilyenkor fordítási hibát kapunk, vagy a kernel nem indul el. A megoldás a dinamikus shared memória (`extern __shared__`) használata, a rekeszek csoportosítása, vagy a probléma átdolgozása többmenetes feldolgozássá.

Az ötödik hiba a mérési módszertan hiányossága. A hisztogram kernel mérésekor mindig warm-up futtatást kell végezni, majd legalább 5–10 ismétlést; az átlag mellett a szórást is közölni kell. A futásidő önmagában nem elegendő, mert a GPU órajele és a háttérterhelés ingadozhat. A laborban ezt a módszertant követjük.

Végül érdemes megemlíteni a `%` művelet költségét. Ha a rekeszek száma kettő hatványa, a modulo helyettesíthető bitművelettel: `bin = data[idx] & (NUM_BINS - 1);`. Ez gyorsabb, de csak akkor alkalmazható, ha a bemeneti tartomány nem negatív, és a rekeszszám ténylegesen kettő hatványa. A laborban `NUM_BINS=64`, így ezt a trükköt is bemutathatjuk.

16.3.5 5. Labor: histogram atomicsal és privatizált változattal

A labor célja a naiv atomi és a shared memory privatizált hisztogram implementációjának mérése, valamint a padding hatásának vizsgálata. Az alábbi forrás `histogram.cu` néven menthető, és CUDA C++ (C++17) nyelven fordul.

16.3.6 5.1. Teljes forráskód

```
// CUDA C++ (C++17)
// histogram.cu - histogram atomicsal és privatizált shared memory-vel
#include <cstdio>
#include <cstdlib>
#include <cuda_runtime.h>

#define CHECK_CUDA(call)                                                    \
    do {                                                                    \
        cudaError_t err = call;                                             \
        if (err != cudaSuccess) {                                           \
            std::fprintf(stderr, "CUDA hiba: %s:%d: %s\n", __FILE__, __LINE__,  \
                         cudaGetErrorString(err));                          \
            std::exit(EXIT_FAILURE);                                        \
        }                                                                   \
    } while (0)

constexpr int NUM_BINS = 64;
constexpr int N = 1 << 20;          // 1 048 576 elem
constexpr int BLOCK_SIZE = 256;
```

```
constexpr int PAD = 2;              // stride = 2*PAD+1 = 5
constexpr int PAD_STRIDE = 2 * PAD + 1;

// --- Naiv globális atomics ---
__global__ void hist_naive(const int* data, int* hist, int n)
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = gridDim.x * blockDim.x;
    for (; idx < n; idx += stride) {
        int bin = data[idx] % NUM_BINS;
        atomicAdd(&hist[bin], 1);
    }
}

// --- Shared memory privatizált ---
__global__ void hist_shared(const int* data, int* hist, int n)
{
    __shared__ int shist[NUM_BINS];
    for (int i = threadIdx.x; i < NUM_BINS; i += blockDim.x) {
        shist[i] = 0;
    }
    __syncthreads();

    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = gridDim.x * blockDim.x;
    for (; idx < n; idx += stride) {
        int bin = data[idx] % NUM_BINS;
        atomicAdd(&shist[bin], 1);
    }
    __syncthreads();

    for (int i = threadIdx.x; i < NUM_BINS; i += blockDim.x) {
        atomicAdd(&hist[i], shist[i]);
    }
}

// --- Shared memory privatizált, paddinggel ---
__global__ void hist_shared_padded(const int* data, int* hist, int n)
{
    __shared__ int shist[NUM_BINS * PAD_STRIDE];
    for (int i = threadIdx.x; i < NUM_BINS * PAD_STRIDE; i += blockDim.x) {
        shist[i] = 0;
    }
    __syncthreads();

    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = gridDim.x * blockDim.x;
    for (; idx < n; idx += stride) {
        int bin = data[idx] % NUM_BINS;
        int padded_bin = bin * PAD_STRIDE;
        atomicAdd(&shist[padded_bin], 1);
    }
    __syncthreads();

    for (int i = threadIdx.x; i < NUM_BINS; i += blockDim.x) {
        atomicAdd(&hist[i], shist[i * PAD_STRIDE]);
    }
}

// --- CPU referencia ---
void cpu_histogram(const int* data, int* hist, int n)
{
    for (int i = 0; i < NUM_BINS; ++i) hist[i] = 0;
```

```
    for (int i = 0; i < n; ++i) {
        hist[data[i] % NUM_BINS]++;
    }
}

// --- Eredmény ellenőrzése ---
bool check_hist(const int* a, const int* b, int len)
{
    for (int i = 0; i < len; ++i) {
        if (a[i] != b[i]) return false;
    }
    return true;
}

// --- Időmérés CUDA eventekkel ---
float run_kernel(void (*kernel)(const int*, int*, int),
                 const int* d_data, int* d_hist, int n,
                 int grid_size, int block_size,
                 int num_iter, cudaStream_t stream)
{
    cudaEvent_t start, stop;
    CHECK_CUDA(cudaEventCreate(&start));
    CHECK_CUDA(cudaEventCreate(&stop));

    // Warm-up
    kernel<<<grid_size, block_size, 0, stream>>>(d_data, d_hist, n);
    CHECK_CUDA(cudaGetLastError());
    CHECK_CUDA(cudaStreamSynchronize(stream));

    CHECK_CUDA(cudaEventRecord(start, stream));
    for (int i = 0; i < num_iter; ++i) {
        kernel<<<grid_size, block_size, 0, stream>>>(d_data, d_hist, n);
    }
    CHECK_CUDA(cudaEventRecord(stop, stream));
    CHECK_CUDA(cudaStreamSynchronize(stream));

    float elapsed = 0.0f;
    CHECK_CUDA(cudaEventElapsedTime(&elapsed, start, stop));
    CHECK_CUDA(cudaEventDestroy(start));
    CHECK_CUDA(cudaEventDestroy(stop));
    return elapsed / num_iter;
}

int main()
{
    // Bemeneti adatok: determinisztikus sorozat
    int* h_data = new int[N];
    for (int i = 0; i < N; ++i) {
        h_data[i] = i % NUM_BINS;
    }

    int* h_hist_ref = new int[NUM_BINS];
    cpu_histogram(h_data, h_hist_ref, N);

    // GPU allokáció
    int *d_data, *d_hist;
    CHECK_CUDA(cudaMalloc(&d_data, N * sizeof(int)));
    CHECK_CUDA(cudaMalloc(&d_hist, NUM_BINS * sizeof(int)));
    CHECK_CUDA(cudaMemcpy(d_data, h_data, N * sizeof(int), cudaMemcpyHostToDevice));

    cudaStream_t stream;
    CHECK_CUDA(cudaStreamCreate(&stream));
```

```
    // Grid méret: teljes lefedés grid-stride loop-pal
    int grid_size = (N + BLOCK_SIZE - 1) / BLOCK_SIZE;

    const int num_iter = 10;

    // Naiv kernel mérése
    CHECK_CUDA(cudaMemset(d_hist, 0, NUM_BINS * sizeof(int)));
    float t_naive = run_kernel(hist_naive, d_data, d_hist, N,
                               grid_size, BLOCK_SIZE, num_iter, stream);

    int* h_hist_naive = new int[NUM_BINS];
    CHECK_CUDA(cudaMemcpy(h_hist_naive, d_hist, NUM_BINS * sizeof(int),
                          cudaMemcpyDeviceToHost));
    bool ok_naive = check_hist(h_hist_naive, h_hist_ref, NUM_BINS);

    // Shared kernel mérése
    CHECK_CUDA(cudaMemset(d_hist, 0, NUM_BINS * sizeof(int)));
    float t_shared = run_kernel(hist_shared, d_data, d_hist, N,
                                grid_size, BLOCK_SIZE, num_iter, stream);

    int* h_hist_shared = new int[NUM_BINS];
    CHECK_CUDA(cudaMemcpy(h_hist_shared, d_hist, NUM_BINS * sizeof(int),
                          cudaMemcpyDeviceToHost));
    bool ok_shared = check_hist(h_hist_shared, h_hist_ref, NUM_BINS);

    // Padding kernel mérése
    CHECK_CUDA(cudaMemset(d_hist, 0, NUM_BINS * sizeof(int)));
    float t_padded = run_kernel(hist_shared_padded, d_data, d_hist, N,
                                grid_size, BLOCK_SIZE, num_iter, stream);

    int* h_hist_padded = new int[NUM_BINS];
    CHECK_CUDA(cudaMemcpy(h_hist_padded, d_hist, NUM_BINS * sizeof(int),
                          cudaMemcpyDeviceToHost));
    bool ok_padded = check_hist(h_hist_padded, h_hist_ref, NUM_BINS);

    // Eredmények kiírása
    std::printf("Histogram labor eredményei (N=%d, B=%d, block=%d, grid=%d)\n",
                N, NUM_BINS, BLOCK_SIZE, grid_size);
    std::printf("Várt hisztogram binenként: %d\n", N / NUM_BINS);
    std::printf("%-12s %-12s %-12s %-10s\n", "Változat", "Idő (ms)", "Szórás", "Helyes?");
    std::printf("%-12s %-12.4f %-12s %-10s\n", "naiv", t_naive, "-", ok_naive ? "igen" : "NEM");
    std::printf("%-12s %-12.4f %-12s %-10s\n", "shared", t_shared, "-", ok_shared ? "igen" :
↪   "NEM");
    std::printf("%-12s %-12.4f %-12s %-10s\n", "padded", t_padded, "-", ok_padded ? "igen" :
↪   "NEM");

    // Takarítás
    CHECK_CUDA(cudaStreamDestroy(stream));
    CHECK_CUDA(cudaFree(d_data));
    CHECK_CUDA(cudaFree(d_hist));
    delete[] h_data;
    delete[] h_hist_ref;
    delete[] h_hist_naive;
    delete[] h_hist_shared;
    delete[] h_hist_padded;
    return 0;
}
```

16.3.7 5.2. Build és futtatás

A fordításhoz és futtatáshoz használjuk az alábbi parancsokat (a compute capability-t a saját GPU-hoz igazítsuk):

```
nvcc -O3 -arch=sm_86 -o hist histogram.cu
./hist
```

A laborban rögzítendő környezet (példa, a hallgató a sajátját írja be):

- GPU: NVIDIA GeForce RTX 3070
- Driver: 535.104.05
- CUDA: 12.1
- Compute capability: 8.6
- Operációs rendszer: Ubuntu 22.04 LTS
- Power limit: 220 W (alapértelmezett)
- ECC: kikapcsolva
- MIG: nem aktív
- Háttérterhelés: nincs

16.3.8 5.3. Mérési napló

A futtatás során a program mindhárom kernelt 10-szer futtatja, és az átlagos futásidőt írja ki. A szórás számításához érdemes a mérést többször megismételni, vagy a programot módosítani úgy, hogy minden iteráció időtartamát rögzítse. Az alábbi táblázat a mérési napló szerkezetét mutatja; a mért értékeket a hallgató tölti ki a saját futtatása alapján.

Változat	N	B	block	Átlag idő (ms)	Szórás (ms)	Átbocsátás (Melem/s)	Helyes?
Naiv atomics	1 048 576	64	256	[mérendő]	[mérendő]	[mérendő]	igen
Shared privatizált	1 048 576	64	256	[mérendő]	[mérendő]	[mérendő]	igen
Shared + padding (PAD=2)	1 048 576	64	256	[mérendő]	[mérendő]	[mérendő]	igen

A **várt eredmény**: mivel a bemenet `data[i] = i % 64`, a hisztogram minden rekeszére `N / NUM_BINS = 1 048 576 / 64 = 16 384` esik. Ezt a CPU referencia is visszaadja, és a GPU implementációknak ugyanezt az értéket kell produkálniuk. A futásidő tekintetében az várható, hogy a shared privatizált változat jelentősen gyorsabb, mint a naiv atomics, különösen, ha a bemeneti eloszlás ferde. A padding változat további javulást hozhat, de 64 rekesz és egyenletes eloszlás esetén a különbség kisebb lehet; a bank conflict méréséhez érdemes az Nsight Compute-ot futtatni.

16.3.9 5.4. Kiegészítő lehetőségek

A labor bővíthető a következőkkel:

- **OpenCL implementáció**: az OpenCL C-ben a `atomic_add` függvény hasonlóan működik, a `local` memória használatával. A kernel szerkezete szinte azonos, csak a memóriamodell és a szinkronizáció (`barrier(CLK_LOCAL_MEM_FENCE)`) különbözik. A méréshez az OpenCL időmérőit (`clGetEventProfilingInfo`) használhatjuk.
- **JAX implementáció**: a JAX-ben a `jax.lax.segment_sum` vagy az `jax.numpy.bincount` függvény hasonló műveletet végez, de ott a háttérben az XLA fordítja a kódot GPU/TPU kernelre. A hisztogram ott is atomi műveleteket vagy redukciót használ, és a teljesítmény az eszköztől függ.
- **Bank conflict mérés**: futtassuk az Nsight Compute-ot a kernelre, és vizsgáljuk a `shared` memória bank conflict metrikákat. Hasonlítsuk össze az alap és a paddinges verziót.

16.3.10 6. Összefoglalás és önellenőrző kérdés

Ebben az alfejezetben megvizsgáltuk a hisztogram két GPU-s megvalósítását: a naiv, globális atomi műveleteken alapuló megoldást és a blokkszintű privatizációt, amely a kontenció csökkentésére shared memóriát használ. Bemutattuk a shared memória bank conflict jelenségét és a paddinges elrendezést mint lehetséges optimalizációt. A laborban egy teljes, fordítható CUDA C++ program segítségével mérhető a két megközelítés teljesítménye, és ellenőrizhető a helyesség a CPU referencia alapján.

Önellenőrző kérdés: Miért gyorsabb a blokkszintű privatizált hisztogram, mint a közvetlen globális atomi összeadás, és milyen feltételek mellett nem érvényesül ez az előny? (Válasz: mert a globális atomi műveletek sorosítják a hozzáférést a memóriavezérlőn, míg a shared memóriás atomi műveletek gyorsak és blokkon belül maradnak; az előny akkor csökken, ha a rekeszek száma olyan nagy, hogy a shared memória nem fér el, vagy ha a bank conflict miatt a shared memória hozzáférés is sorosítódik, illetve ha a bemeneti eloszlás nagyon egyenletes és a kontenció eleve kicsi.)

16.4 CUB és Thrust használata

A redukció, a prefix összeg (scan) és a rendezés a párhuzamos algoritmusok leggyakrabban használt építőkövei. Az előző alfejezetekben saját CUDA kernelekkel valósítottuk meg őket, hogy megértsük a működésüket. A termelési kódban azonban ritkán írunk kézzel redukciót vagy scan-t: a gyártó és a közösség által karbantartott könyvtárak több éves optimalizálást, hardverspecifikus hangolást és alapos tesztelést foglalnak magukba. Ez az alfejezet a CUDA ökoszisztéma két alapvető eszközét, a CUB-ot és a Thrust-ot mutatja be, a redukció, a scan és a rendezés használatára összpontosítva.

16.4.1 A CUB és a Thrust szerepe

A CUB (CUDA UnBound) egy alacsony szintű, header-only párhuzamos primitív könyvtár. A CUDA Toolkit részeként érkezik, és olyan eszközöket biztosít, mint a `DeviceReduce`, `DeviceScan`, `DeviceSort`, `DeviceSelect`, `DeviceSegmentedReduce` és társaik. A CUB közvetlenül a CUDA C++ execution modelre épül, ezért a hívásai gyakran tömbmutatókat, méreteket és opcionálisan CUDA streamet várnak. A Thrust ezzel szemben egy magasabb szintű, STL-szerű sablonkönyvtár, amely konténereket (`thrust::device_vector`), iterátorokat és általános algoritmusokat

(`thrust::reduce`, `thrust::transform_reduce`, `thrust::sort`) kínál. A Thrust számos műveletét a CUB primitívekre építi, így a kettő jól kiegészíti egymást: a Thrust kényelmes, gyors fejlesztést tesz lehetővé, míg a CUB finomabb kontrollt ad, és a teljesítmény szempontjából kritikus kódrészekhez alkalmas.

A gyakorlatban a következő döntési szempontokat érdemes követni:

- Ha egy összeget, minimumot, maximumot vagy prefix összeget kell számítani egyetlen nyers device-tömbön, a CUB `DeviceReduce` és `DeviceScan` közvetlen, minimális overheaddel járó megoldást ad.
- Ha a műveleteket transzformációval kombináljuk, például norma számítása vagy feltételes kiválasztás, a Thrust `transform_reduce` és `transform_scan` kényelmesebb.
- Ha a kódban már `thrust::device_vector`-ok vannak, akkor a Thrust algoritmusai természetesen illeszkednek; ha nyers pointerekkel dolgozunk, a CUB és a Thrust egyaránt használható, utóbbi esetben pointer-konverzióval.

16.4.2 A CUB programozási modellje

A CUB eszközfüggvényei kétfázisú hívást alkalmaznak. Először lekérdezzük a szükséges ideiglenes tárterület méretét egy `nullptr` mint ideiglenes puffer átadásával. A CUB ilyenkor nem hajt végre számítást, csak kitölti a méretet tartalmazó változót. Ezután a hívó fél allokálja a megfelelő méretű puffert, és egy második hívással ténylegesen futtatja a műveletet. Ez a minta lehetővé teszi a memória újrafelhasználását több hívás között, és elkerüli a belső allokációkat.

Példa a `DeviceReduce::Sum` hívására:

```
// CUDA C++ (C++17), nvcc 12.1, sm_80
// Hibakezelő makró
#define CHECK_CUDA(call) do {                                       \
    cudaError_t err = call;                                         \
    if (err != cudaSuccess) {                                       \
        fprintf(stderr, "CUDA error at %s:%d: %s\n",                \
                __FILE__, __LINE__, cudaGetErrorString(err));       \
        exit(EXIT_FAILURE);                                         \
    }                                                               \
} while (0)

// Feltételezzük, hogy d_in és d_out érvényes device-mutatók, n > 0
void* d_temp_storage = nullptr;
size_t temp_storage_bytes = 0;

// 1. fázis: a szükséges ideiglenes tárterület méretének lekérdezése
CHECK_CUDA(cub::DeviceReduce::Sum(
    nullptr, temp_storage_bytes, d_in, d_out, n));

// 2. fázis: allokálás és tényleges végrehajtás
CHECK_CUDA(cudaMalloc(&d_temp_storage, temp_storage_bytes));
CHECK_CUDA(cub::DeviceReduce::Sum(
    d_temp_storage, temp_storage_bytes, d_in, d_out, n));

// Opcionális: stream megadása
// CHECK_CUDA(cub::DeviceReduce::Sum(
//     d_temp_storage, temp_storage_bytes, d_in, d_out, n, stream));
```

A legtöbb CUB hívás visszatérési értéke `cudaError_t`, így a hibakezelő makrónkkal közvetlenül ellenőrizhető. A hívás után a szokásos `cudaDeviceSynchronize` is szükséges lehet, ha a host oldalon azonnal fel akarjuk használni az eredményt vagy mérni akarjuk a futásidőt.

16.4.3 DeviceReduce: könyvtári redukció

A `cub::DeviceReduce` sablonosztály a leggyakoribb redukciós műveleteket kínálja:

- `cub::DeviceReduce::Sum` – összegzés,
- `cub::DeviceReduce::Min` / `Max` – minimum és maximum,
- `cub::DeviceReduce::ArgMin` / `ArgMax` – a szélsőérték indexe,
- `cub::DeviceReduce::Reduce` – tetszőleges, saját bináris művelet.

A beépített műveleteknél a hívás szignatúrája egyszerű:

```
cudaError_t cub::DeviceReduce::Sum(
    void* d_temp_storage,
    size_t& temp_storage_bytes,
    const T* d_in,
    T* d_out,
    int num_items,
    cudaStream_t stream = 0);
```

A `d_in` mutató a bemeneti tömb device-memóriájára mutat, a `d_out` pedig egyetlen elemet fog tartalmazni. A `num_items` a redukálandó elemek száma. A `d_temp_storage` az előzőleg lekérdezett méretű puffer; `nullptr` esetén csak a méretet kapjuk meg.

A CUB redukciója a háttérben többszintű, regiszterekben és shared memory-ban végzett részösszegzést hajt végre, majd a blokkok közötti összegzést az ideiglenes tárterületen keresztül oldja meg. A bemeneti tömböt általában coalescing-barát módon olvassa, a memória-sávszélesség nagy részét kihasználva. A futásidő jellemzően a bemeneti adat méretétől és a memória-sávszélességtől függ, nem pedig a számítási teljesítménytől.

Fontos, hogy a `d_out` pointernek is device-memóriára kell mutatnia. Az eredmény hostra másolásához használjunk `cudaMemcpy`-t. A következő példa egy 1 Mi elemből álló `float` tömb összegét számítja ki, és ellenőrzi az eredményt:

```
// CUDA C++ (C++17), nvcc 12.1, sm_80
#include <cub/cub.cuh>
#include <cstdio>
#include <cstdlib>

#define CHECK_CUDA(call) /* lásd fent */

int main() {
    const int n = 1 << 20; // 1 048 576
    size_t bytes = n * sizeof(float);

    // Host bemenet: x[i] = 1.0f
    float* h_in = static_cast<float*>(malloc(bytes));
    for (int i = 0; i < n; ++i) h_in[i] = 1.0f;

    // Device memória
    float *d_in = nullptr, *d_out = nullptr;
    CHECK_CUDA(cudaMalloc(&d_in, bytes));
    CHECK_CUDA(cudaMalloc(&d_out, sizeof(float)));
    CHECK_CUDA(cudaMemcpy(d_in, h_in, bytes, cudaMemcpyHostToDevice));

    // Ideiglenes tárterület méretének lekérdezése
    void* d_temp_storage = nullptr;
    size_t temp_storage_bytes = 0;
    CHECK_CUDA(cub::DeviceReduce::Sum(
        nullptr, temp_storage_bytes, d_in, d_out, n));
```

```
    // Allokálás és futtatás
    CHECK_CUDA(cudaMalloc(&d_temp_storage, temp_storage_bytes));
    CHECK_CUDA(cub::DeviceReduce::Sum(
        d_temp_storage, temp_storage_bytes, d_in, d_out, n));
    CHECK_CUDA(cudaDeviceSynchronize());

    // Eredmény visszamásolása
    float h_out = 0.0f;
    CHECK_CUDA(cudaMemcpy(&h_out, d_out, sizeof(float), cudaMemcpyDeviceToHost));

    printf("Sum: %f (expected %f)\n", h_out, static_cast<float>(n));

    // Takarítás
    CHECK_CUDA(cudaFree(d_temp_storage));
    CHECK_CUDA(cudaFree(d_in));
    CHECK_CUDA(cudaFree(d_out));
    free(h_in);
    return 0;
}
```

Mivel a bemeneti értékek mindegyike `1.0f`, a várt összeg `1048576.0f`. Ez a szám a `float` típus pontossági tartományán belül van, így bitpontos egyezést várunk. Ha a bemenet véletlenszerű lebegőpontos érték lenne, akkor a GPU és a CPU redukciója közötti eltérést a lebegőpontos összegzés sorrendjének különbsége okozná; ilyenkor tűréssel kell összehasonlítani, amit a numerikus pontosságról szóló fejezet tárgyal.

16.4.4 DeviceScan: könyvtári scan

A `cub::DeviceScan` a prefix összeg (scan) számítását végzi. Két alapvető változata van:

- `cub::DeviceScan::InclusiveSum` – a kimenet i-edik eleme tartalmazza az input 0-tól i-ig terjedő elemeinek összegét: $y_i = \sum_{j=0}^{i} x_j$.
- `cub::DeviceScan::ExclusiveSum` – a kimenet i-edik eleme az input 0-tól $i-1$-ig terjedő elemeinek összegét adja, azaz $y_i = \sum_{j=0}^{i-1} x_j$, ahol $y_0 = 0$.

Az exclusive scan különösen hasznos olyan feladatoknál, ahol a kimenetet egy későbbi lépésben címzésre használjuk, például kompaktálásnál vagy szegmentált feldolgozásnál. A CUB más típusú scan műveleteket is támogat (`Min`, `Max`, `ByKey`, `BySegment`), de most a két összegző változatra koncentrálunk.

A hívás alakja hasonló a redukcióhoz:

```
cudaError_t cub::DeviceScan::InclusiveSum(
    void* d_temp_storage,
    size_t& temp_storage_bytes,
    const T* d_in,
    T* d_out,
    int num_items,
    cudaStream_t stream = 0);
```

A kimeneti tömb `num_items` elemet tartalmaz, nem `num_items + 1`-et, még az exclusive scan esetén sem. A kimenet első eleme exclusive scan-nél mindig nulla (feltéve, hogy nincs explicit kezdőérték paraméter).

A következő kód egy `float` tömb inkluzív prefix összegét számítja ki:

```
// CUDA C++ (C++17), nvcc 12.1, sm_80
// Feltételezzük: n = 1 << 20, d_in és d_scan_out érvényes device-tömbök
void* d_temp_storage = nullptr;
size_t temp_storage_bytes = 0;

CHECK_CUDA(cub::DeviceScan::InclusiveSum(
    nullptr, temp_storage_bytes, d_in, d_scan_out, n));

CHECK_CUDA(cudaMalloc(&d_temp_storage, temp_storage_bytes));
CHECK_CUDA(cub::DeviceScan::InclusiveSum(
    d_temp_storage, temp_storage_bytes, d_in, d_scan_out, n));
CHECK_CUDA(cudaDeviceSynchronize());
```

Ha a bemenet minden eleme `1.0f`, akkor a várt kimenet: $y_i = i + 1$, azaz `1, 2, 3, ..., 1 048 576`. Ezt CPU-n könnyű ellenőrizni egy egyszerű ciklussal.

A CUB scan implementációja work-efficient algoritmust használ (gyakran Blelloch vagy hasonló), amely csökkenti a felesleges műveletek számát. A teljesítményt elsősorban a bemeneti és kimeneti adatok mozgatása határozza meg, mert a scan olvassa és írja is a teljes tömböt.

16.4.5 DeviceSort: könyvtári rendezés

A CUB `DeviceSort` osztálya kulcs szerinti rendezést valósít meg radix sort alapú algoritmussal. A radix sort nem összehasonlítás alapú, ezért jól illeszkedik a GPU párhuzamos architektúrájához. A CUB a következő főbb változatokat kínálja:

- `cub::DeviceSort::SortKeys` – kulcsok rendezése in-place,
- `cub::DeviceSort::SortKeysCopy` – kulcsok rendezése külön be- és kimeneti tömbbel,
- `cub::DeviceSort::SortPairs` – kulcsok és hozzájuk tartozó értékek rendezése,
- `cub::DeviceSort::SortPairsCopy` – mint előbb, de külön kimenettel.

A `SortPairs` akkor fontos, ha a rendezett sorrendhez adatok is tartoznak, például indexek vagy pontszámok. A kulcsok típusa lehet `int`, `float`, `unsigned int`, `long long` stb., de a radix sort bitműveletek alapján dolgozik, ezért a lebegőpontos típusoknál a belső bitreprezentációt használja. A helyes sorrend eléréséhez a CUB a lebegőpontos számok IEEE 754 bitképét alakítja át, így a rendezés eredménye a várt numerikus sorrend lesz.

Az alapvető in-place kulcsrendezés kódja:

```
// CUDA C++ (C++17), nvcc 12.1, sm_80
// d_keys: n méretű int device-tömb, rendezendő
void* d_temp_storage = nullptr;
size_t temp_storage_bytes = 0;

CHECK_CUDA(cub::DeviceSort::SortKeys(
    nullptr, temp_storage_bytes, d_keys, n));

CHECK_CUDA(cudaMalloc(&d_temp_storage, temp_storage_bytes));
CHECK_CUDA(cub::DeviceSort::SortKeys(
    d_temp_storage, temp_storage_bytes, d_keys, n));
CHECK_CUDA(cudaDeviceSynchronize());
```

A `SortKeys` a kulcsokat növekvő sorrendbe rendezi. Ha csökkenő sorrendet szeretnénk, a kulcsokat negálhatjuk, vagy a `SortKeysDescending` változatot használhatjuk (bizonyos CUB verziókban elérhető). A radix sort az adott típus összes bitjét feldolgozza; ez alapértelmezésben a típus teljes bitszélessége, például `sizeof(int) * 8` az `int` esetén. A CUB lehetőséget ad a feldolgozott bitek számának korlátozására is, ami akkor hasznos, ha a kulcsok értékkészlete kisebb.

A CUB `DeviceSort` futásideje általában $O(n)$ a radix sort miatt, ellentétben az összehasonlítás alapú rendezések $O(n \log n)$ elméleti korlátjával. A gyakorlatban a teljesítményt a memóriahozzáférés és a radix lépések száma határozza meg. A következő alfejezet (Radix sort) a radix sort belső működését tárgyalja, ezért itt csak a használatra összpontosítunk.

16.4.6 Thrust: magasabb szintű absztrakció

A Thrust a CUDA számára készült párhuzamos algoritmusgyűjtemény, amelynek felülete az STL-re hasonlít. A legfontosabb konténerei a `thrust::host_vector<T>` és a `thrust::device_vector<T>`. Az algoritmusok iterátorokon dolgoznak, így ugyanaz a kód használható host- és device-konténerekkel egyaránt. A Thrust header-only, a CUDA Toolkit tartalmazza.

16.4.7 transform_reduce és reduce

A `thrust::reduce` egy tartomány elemeit redukálja egyetlen értékké. Alapértelmezés szerint összeadást végez, de tetszőleges bináris műveletet adhatunk meg. A `thrust::transform_reduce` először egy unáris transzformációt alkalmaz minden elemre, majd a transzformált értékeket redukálja. Ez a kombináció rendkívül gyakori: például vektor normájának számításakor először négyzetre emelünk, majd összegzünk.

A következő példa egy `float` vektor euklideszi normájának négyzetét számítja ki:

```
// CUDA C++ (C++17), nvcc 12.1, sm_80
#include <thrust/device_vector.h>
#include <thrust/transform_reduce.h>
#include <thrust/functional.h>
#include <thrust/sequence.h>
#include <cstdio>

// Unáris funktor: négyzetre emelés
struct square_functor {
    __host__ __device__
    float operator()(float x) const { return x * x; }
};

int main() {
    const int n = 1 << 20;
    thrust::device_vector<float> d_vec(n);

    // Feltöltés 1.0f értékekkel
    thrust::fill(d_vec.begin(), d_vec.end(), 1.0f);

    // A négyzetösszeg kiszámítása: sum(x_i^2)
    float sum_sq = thrust::transform_reduce(
        d_vec.begin(), d_vec.end(),
        square_functor(),
        0.0f,
        thrust::plus<float>());

    printf("Sum of squares: %f (expected %f)\n", sum_sq, static_cast<float>(n));
    return 0;
}
```

A `thrust::transform_reduce` szignatúrája a következő:

```
template<typename InputIterator, typename UnaryFunction, typename T, typename BinaryFunction>
T thrust::transform_reduce(
    InputIterator first, InputIterator last,
    UnaryFunction unary_op,
    T init,
    BinaryFunction binary_op);
```

A `binary_op` általában `thrust::plus<T>()`, de használhatunk szorzást, maximumot, minimumot vagy saját funktort. A `thrust::reduce` hasonló, de unáris transzformáció nélkül:

```
float total = thrust::reduce(d_vec.begin(), d_vec.end(), 0.0f, thrust::plus<float>());
```

A `thrust::sort` szintén in-place rendezést végez, hasonlóan a CUB `SortKeys`-hez:

```
// CUDA C++ (C++17), nvcc 12.1, sm_80
#include <thrust/sort.h>
#include <thrust/device_vector.h>

thrust::device_vector<int> d_keys(n);
// ... feltöltés ...
thrust::sort(d_keys.begin(), d_keys.end());
```

A Thrust rendezése alapértelmezésben növekvő sorrendet ad, és beépítetten támogatja az összehasonlító funktorokat is.

16.4.8 Mutató konverziók Thrust és nyers CUDA pointerek között

A Thrust konténerei jól integrálhatók a CUB-bal és a kézzel írt kernelekkel. A `thrust::device_ptr<T>` egy vékony burkoló a nyers `T*` pointer körül, amely lehetővé teszi a Thrust algoritmusok közvetlen használatát anélkül, hogy `device_vector`-t kellene létrehozni. A `thrust::device_pointer_cast` egy nyers mutatót `device_ptr`-re alakít, míg a `thrust::raw_pointer_cast` az ellenkező irányt végzi el.

Példa nyers mutatóval:

```
// CUDA C++ (C++17), nvcc 12.1, sm_80
float* d_in = nullptr;
cudaMalloc(&d_in, n * sizeof(float));
// ... feltöltés ...

float sum = thrust::reduce(
    thrust::device_pointer_cast(d_in),
    thrust::device_pointer_cast(d_in) + n,
    0.0f,
    thrust::plus<float>());
```

Ha egy `thrust::device_vector`-ból nyers pointert szeretnénk átadni egy CUB vagy saját kernel hívásnak, a `thrust::raw_pointer_cast` használatos:

```
thrust::device_vector<float> d_vec(n);
float* raw_ptr = thrust::raw_pointer_cast(d_vec.data());
```

Ez a konverzió okos, mert nem jár adatmásolással, csupán típusmódosítással. Fontos azonban, hogy a `device_vector` memóriája a vektor élettartamához kötött; ha a vektor destruálódik, a nyers pointer érvénytelenné válik.

A Thrust műveletek alapértelmezésben a default stream-en futnak, és a hívó blokkolódik, amíg a művelet be nem fejeződik. Ez leegyszerűsíti a hibakeresést és a kezdeti fejlesztést, de akadályozhatja az aszinkron végrehajtást. A későbbi, streamekről szóló fejezet (Streamek és események) megmutatja, hogyan lehet a Thrust hívásait explicit streamhez kötni.

16.4.9 Saját vs. könyvtári megoldás

A saját kernel írásának oktatási értéke vitathatatlan: elmélyíti a memória-hozzáférés, a szinkronizáció és az algoritmusok megértését. A termelési kódban azonban a könyvtárak használata a következő előnyökkel jár:

- **Teljesítmény:** A CUB és a Thrust implementációit a hardvergyártó mérnökei optimalizálták az adott GPU-generációkra. A naiv saját kernel gyakran messze elmarad a memória-sávszélesség kihasználásában, míg a CUB redukciója jellemzően a sávszélesség 80–90%-át eléri.
- **Megbízhatóság:** A könyvtárak kezelik a szélsőséges méreteket, a nem pontosan osztható elemszámokat, a nagy tömböket és a konkurens hívásokat. A saját kódban ezek a sarok esetek gyakran hibákhoz vezetnek.
- **Karbantarthatóság:** A könyvtári hívás tömör, deklaratív, és a mögöttes optimalizációk a jövőbeni hardverfrissítésekkel együtt fejlődhetnek, anélkül hogy a felhasználói kódot módosítani kellene.
- **Fejlesztési idő:** A two-phase CUB hívás vagy egy `thrust::reduce` néhány sor, míg egy robosztus saját redukció több száz sor tesztelt kódot igényel.

Ugyanakkor a könyvtárak használatának is vannak korlátai. A CUB `DeviceReduce::Sum` nem tud egyszerűen egy transzformációt beolvasztani; ha például a bemeneti értékeket egy kernelben számítjuk ki, és azonnal redukálni szeretnénk őket anélkül, hogy a teljes tömböt kiírnánk a globális memóriába, akkor a CUB külön lépést igényel a generálásra és a redukcióra. Hasonlóképpen a Thrust `transform_reduce` kényelmes, de a funktorok és a köztes tárolók bevezetése néha kevésbé hatékony, mint egy kézzel írt, fúzionált kernel. Az ilyen esetekben a saját kernel indokolt lehet.

A gyakorlatban a legjobb stratégia a rétegződés: használjuk a Thrust-ot a magas szintű vezérlésre, a CUB-ot az alacsony szintű primitívekre, és saját kernelt csak ott írjunk, ahol a fúzió vagy a speciális adatelrendezés valódi előnyt ad. Ez a szemlélet a későbbi fejezetekben is visszaköszön, például a cuBLAS és a CUTLASS kapcsán.

16.4.10 Gyakori hibák és elkerülésük

A könyvtári hívások használata során a következő tipikus hibákkal találkozhatunk:

1. **Az ideiglenes tárterület méretének lekérdezése elmarad vagy helytelen.**
   A CUB hívások kétfázisúak; ha közvetlenül egy érvénytelen `d_temp_storage` pointerrel hívjuk őket, a program lefagyhat vagy hibát ad. Mindig először `nullptr`-rel kérdezzük le a méretet, majd allokálunk, majd hívunk.

2. **A `temp_storage_bytes` változót nem frissítjük a második hívás előtt.**
   A második hívás ugyanazt a méretet várja, amit az első hívás adott. Ha a változó értéke megváltozik, a CUB hibásan működhet. A legjobb, ha a méretet egy külön `size_t` változóban tároljuk, és nem módosítjuk a két hívás között.

3. **A bemeneti vagy kimeneti pointer host-memóriára mutat.**
   A CUB és a Thrust device-oldali algoritmusai device-memóriát várnak. Host-memória átadása esetén a CUDA hibát ad, vagy a program leáll. Ellenőrizzük, hogy minden pointer `cudaMalloc`-kal vagy `thrust::device_vector`-ral jött létre.

4. **A kimeneti tömb mérete rossz a scan esetén.**
   Az `InclusiveSum` és `ExclusiveSum` kimenete `num_items` elemű, nem `num_items + 1`. Ha a kimeneti puffert túl kicsire méretezzük, memóriahiba lép fel. Az exclusive scan első eleme nulla, ezért nem kell külön hely a kezdőértéknek.

5. **A `DeviceSort` kulcs–érték pároknál a tömbméretek eltérnek.**
   A `SortPairs` feltételezi, hogy a kulcsok és értékek tömbje azonos elemszámú. Ha az egyik rövidebb, a CUB hibát jelez vagy nem definiált viselkedést mutat.

6. **A Thrust iterátorok és a nyers pointerek keverése hibás konverzióval.**
   A `thrust::device_pointer_cast` és a `thrust::raw_pointer_cast` megfelelő használata elengedhetetlen. A `device_vector::data()` önmagában nem `device_ptr`, ezért Thrust algoritmusokhoz nem használható közvetlenül; vagy használjuk a `begin()`-t, vagy alakítsuk át.

7. **A hibakezelés hiánya.**
   A CUB hívások után ellenőrizni kell a visszatérési értéket és a `cudaGetLastError` kimenetét. A hibaüzenetek gyakran azonnal rámutatnak a hibás paraméterre.

8. **A szinkronizálás elhagyása mérésnél vagy host-oldali felhasználásnál.**
   A CUB és a Thrust hívásai aszinkronok lehetnek. Ha az eredményt közvetlenül a hívás után olvassuk host-oldalon, előbb `cudaDeviceSynchronize`-t kell hívni. A méréseknél is szinkronizálni kell az eseményeket.

9. **A `DeviceScan`-nél az input és output átfedése.**
   A CUB általában támogatja az in-place scan-t (azaz `d_in == d_out`), de a biztonságos megoldás a külön kimeneti tömb használata, hacsak a dokumentáció egyértelműen nem engedi az átfedést. Ha a két pointer azonos, győződjünk meg a változat dokumentációjából, hogy ez megengedett.

10. **A `thrust::device_vector` másolása implicit módon host-vektorba.**
    A `thrust::device_vector`-ból `thrust::host_vector`-ba való másolás szinkronizál, de ha a kód nem szándékoltan teszi, teljesítményproblémát okozhat. A nagy tömböknél ez a rejtett adatmozgás jelentős lehet.

A fenti hibák elkerülése érdekében a kódban mindig a `CHECK_CUDA` makrót használjuk, a CUB hívásokat kétfázisúan hajtjuk végre, és a tömbméreteket egységesen kezeljük.

16.4.11 Labor: CUB DeviceReduce és DeviceScan használata

A labor célja a CUB `DeviceReduce::Sum` és `DeviceScan::InclusiveSum` használatának gyakorlása egy egyszerű, ellenőrizhető példán. A bemenet egy $n = 2^{20}$ méretű `float` tömb, amelynek minden eleme `1.0f`. Ez a választás lehetővé teszi a várt eredmény pontos számítását: a redukció eredménye n, a scan kimenete pedig $y_i = i + 1$. A lebegőpontos pontosság miatt a `float` típus ezen a tartományon pontosan reprezentál, így bitpontos egyezést várhatunk.

16.4.12 Környezet

A méréshez a következő izolált labor-konfigurációt használjuk (a konkrét számok illusztrációk, a valós futtatáshoz a saját környezetünkben kell elvégezni):

- GPU: NVIDIA A100 40 GB (compute capability 8.0)
- Driver: 530.30.02
- CUDA: 12.1
- OS: Ubuntu 22.04 LTS
- Fordító: nvcc, `-O3 -arch=sm_80 -std=c++17`

A mérés előtt a GPU-t felmelegítjük (warm-up), majd legalább 10 ismétlést végzünk, és az átlag mellett a szórást is kiszámítjuk. A mérési naplóban a mért és a várt eredményt külön oszlopban tüntetjük fel.

16.4.13 Kód

A következő CUDA C++ program elvégzi a kért műveleteket, ellenőrzi az eredményt, és méri a futásidőt CUDA eventek segítségével.

```
// CUDA C++ (C++17), nvcc 12.1, sm_80
#include <cub/cub.cuh>
#include <cstdio>
#include <cstdlib>
#include <cmath>

#define CHECK_CUDA(call) do {                                       \
    cudaError_t err = call;                                         \
    if (err != cudaSuccess) {                                       \
        fprintf(stderr, "CUDA error at %s:%d: %s\n",                \
                __FILE__, __LINE__, cudaGetErrorString(err));       \
        exit(EXIT_FAILURE);                                         \
    }                                                               \
} while (0)

int main() {
    const int n = 1 << 20; // 1 048 576
    const size_t bytes = n * sizeof(float);

    // Host bemenet feltöltése: x[i] = 1.0f
    float* h_in = static_cast<float*>(malloc(bytes));
    for (int i = 0; i < n; ++i) h_in[i] = 1.0f;

    // Device memória
    float *d_in = nullptr, *d_reduce_out = nullptr, *d_scan_out = nullptr;
    CHECK_CUDA(cudaMalloc(&d_in, bytes));
    CHECK_CUDA(cudaMalloc(&d_reduce_out, sizeof(float)));
    CHECK_CUDA(cudaMalloc(&d_scan_out, bytes));
    CHECK_CUDA(cudaMemcpy(d_in, h_in, bytes, cudaMemcpyHostToDevice));

    // Ideiglenes pufferek mérete és allokálása
    void* d_temp_reduce = nullptr;
    size_t temp_reduce_bytes = 0;
    CHECK_CUDA(cub::DeviceReduce::Sum(
        nullptr, temp_reduce_bytes, d_in, d_reduce_out, n));

    void* d_temp_scan = nullptr;
    size_t temp_scan_bytes = 0;
    CHECK_CUDA(cub::DeviceScan::InclusiveSum(
        nullptr, temp_scan_bytes, d_in, d_scan_out, n));

    CHECK_CUDA(cudaMalloc(&d_temp_reduce, temp_reduce_bytes));
    CHECK_CUDA(cudaMalloc(&d_temp_scan, temp_scan_bytes));

    // Mérési események
    cudaEvent_t start, stop;
    CHECK_CUDA(cudaEventCreate(&start));
    CHECK_CUDA(cudaEventCreate(&stop));

    const int warmup = 10;
    const int iters = 50;
```

```
// Warm-up (nem mérjük)
for (int i = 0; i < warmup; ++i) {
    CHECK_CUDA(cub::DeviceReduce::Sum(
        d_temp_reduce, temp_reduce_bytes, d_in, d_reduce_out, n));
    CHECK_CUDA(cub::DeviceScan::InclusiveSum(
        d_temp_scan, temp_scan_bytes, d_in, d_scan_out, n));
}
CHECK_CUDA(cudaDeviceSynchronize());

// DeviceReduce mérése
CHECK_CUDA(cudaEventRecord(start));
for (int i = 0; i < iters; ++i) {
    CHECK_CUDA(cub::DeviceReduce::Sum(
        d_temp_reduce, temp_reduce_bytes, d_in, d_reduce_out, n));
}
CHECK_CUDA(cudaEventRecord(stop));
CHECK_CUDA(cudaEventSynchronize(stop));
float reduce_ms = 0.0f;
CHECK_CUDA(cudaEventElapsedTime(&reduce_ms, start, stop));
reduce_ms /= iters;

// DeviceScan mérése
CHECK_CUDA(cudaEventRecord(start));
for (int i = 0; i < iters; ++i) {
    CHECK_CUDA(cub::DeviceScan::InclusiveSum(
        d_temp_scan, temp_scan_bytes, d_in, d_scan_out, n));
}
CHECK_CUDA(cudaEventRecord(stop));
CHECK_CUDA(cudaEventSynchronize(stop));
float scan_ms = 0.0f;
CHECK_CUDA(cudaEventElapsedTime(&scan_ms, start, stop));
scan_ms /= iters;

// Eredmények visszamásolása és ellenőrzése
float h_reduce = 0.0f;
float* h_scan = static_cast<float*>(malloc(bytes));
CHECK_CUDA(cudaMemcpy(&h_reduce, d_reduce_out, sizeof(float), cudaMemcpyDeviceToHost));
CHECK_CUDA(cudaMemcpy(h_scan, d_scan_out, bytes, cudaMemcpyDeviceToHost));

// Várt eredmények
float expected_reduce = static_cast<float>(n);
float max_scan_error = 0.0f;
for (int i = 0; i < n; ++i) {
    float expected_scan = static_cast<float>(i + 1);
    float err = fabsf(h_scan[i] - expected_scan);
    if (err > max_scan_error) max_scan_error = err;
}

printf("=== CUB DeviceReduce::Sum ===\n");
printf("  mért:    %.1f\n", h_reduce);
printf("  várt:    %.1f\n", expected_reduce);
printf("  eltérés: %.6e\n", fabsf(h_reduce - expected_reduce));
printf("  átlagos futásidő: %.4f ms\n", reduce_ms);

printf("=== CUB DeviceScan::InclusiveSum ===\n");
printf("  maximális abszolút hiba a scan kimenetben: %.6e\n", max_scan_error);
printf("  átlagos futásidő: %.4f ms\n", scan_ms);

// Sávszélesség becslés
// Reduce: beolvasás n*sizeof(float), kiírás sizeof(float) -> kb. n*4 byte mozog
double reduce_gb = (n * sizeof(float) + sizeof(float)) / 1e9;
printf("  effektív reduce sávszélesség: %.2f GB/s\n",
       reduce_gb / (reduce_ms / 1000.0));
```

```
    // Scan: beolvasás n*sizeof(float), kiírás n*sizeof(float) -> 2*n*4 byte
    double scan_gb = (2.0 * n * sizeof(float)) / 1e9;
    printf("  effektív scan sávszélesség: %.2f GB/s\n",
           scan_gb / (scan_ms / 1000.0));

    // Takarítás
    CHECK_CUDA(cudaFree(d_temp_reduce));
    CHECK_CUDA(cudaFree(d_temp_scan));
    CHECK_CUDA(cudaFree(d_in));
    CHECK_CUDA(cudaFree(d_reduce_out));
    CHECK_CUDA(cudaFree(d_scan_out));
    free(h_in);
    free(h_scan);
    CHECK_CUDA(cudaEventDestroy(start));
    CHECK_CUDA(cudaEventDestroy(stop));
    return 0;
}
```

16.4.14 Build és futtatás

A program fordítása és futtatása az alábbi parancsokkal történik:

```
nvcc -O3 -arch=sm_80 -std=c++17 lab_cub_reduce_scan.cu -o lab_cub_reduce_scan
./lab_cub_reduce_scan
```

A build során a CUDA Toolkit 12.1-et használjuk, és a `sm_80` architektúrát célozzuk meg. A program a futtatás után kiírja a mért és várt eredményeket, valamint a futásidőket.

16.4.15 Mérési napló sablon

A mérési naplóban a következő adatokat rögzítjük. Az alábbi táblázatban a számok **illusztrációk**, nem valós mérések; a valós futtatáshoz a saját környezetünkben kell elvégezni a mérést.

Művelet	N	Átlagos futásidő (ms)	Szórás (ms)	Effektív sávszélesség (GB/s)	Megjegyzés
`DeviceReduce::Sum`	1 048 576	0.052	0.003	~80.6	A bemenet 1.0f, eredmény pontosan n
`DeviceScan::InclusiveSum`	1 048 576	0.104	0.006	~80.6	A kimenet 1..n, max hiba 0.0

A mért és a várt eredményt külön sorban közöljük:

Eredmény	Várt	Mért	Eltérés
Redukció összege	1048576.0f	1048576.0f	0.0
Scan maximális eltérése	0.0	0.0	0.0

A várt értékek CPU-n egyszerű ciklussal számíthatók. A mérés során a GPU órajelét és hőmérsékletét a `nvidia-smi` segítségével ellenőrizhetjük, de a fenti táblázat a reprodukálhatóság kedvéért a számítási körülményeket is rögzíti.

16.4.16 A labor értékelése

A labor akkor tekinthető sikeresnek, ha:

- a program lefordul és hiba nélkül fut,
- a redukció eredménye bitpontosan egyezik a várt $n = 1048576$ értékkel,
- a scan kimenetének maximális abszolút hibája `0.0`,
- a mérési napló tartalmazza a futásidőt, a szórást és a sávszélességet,
- a kód a `CHECK_CUDA` makrót minden CUDA hívásnál használja, és a mérés előtt warm-up-ot végez.

A labor kódja a fejezet forráskönyvtárában `lab_cub_reduce_scan.cu` néven található. A mérési eredményeket a fenti sablon szerint kell kitölteni, és a mért adatokat egyértelműen el kell különíteni a várt értékektől.

16.4.17 Önellenőrző kérdések

1. **Miért kétfázisú a CUB `DeviceReduce` és `DeviceScan` hívása?**
   Az első hívás `nullptr` ideiglenes pufferrel lekérdezi a szükséges tárterület méretét, hogy a felhasználó egyszer allokálhasson, és a puffert több hívás között újrahasznosíthassa. A második hívás ténylegesen végrehajtja a műveletet. Ez elkerüli a belső allokációkat és a felesleges memóriaműveleteket.

2. **Miben különbözik a `cub::DeviceScan::InclusiveSum` és a `cub::DeviceScan::ExclusiveSum`?**
   Az inclusive scan a kimenet i-edik elemébe az input 0-tól i-ig terjedő elemeinek összegét írja ($y_i = \sum_{j=0}^{i} x_j$). Az exclusive scan az input 0-tól $i-1$-ig terjedő elemeinek összegét adja ($y_i = \sum_{j=0}^{i-1} x_j$), ezért $y_0 = 0$. A kimeneti tömb mérete mindkét esetben n.

3. **Mikor érdemes saját redukciós kernelt írni a CUB vagy a Thrust helyett?**
   Olyan esetekben, amikor a redukciót egy másik kernel fúziójával akarjuk összevonni, például a bemeneti értékeket egy kernelben számítjuk ki, és nem akarjuk a teljes tömböt a globális memóriába kiírni. Szintén szükség lehet saját kernelre, ha a CUB vagy a Thrust nem támogatja a kívánt típust vagy műveletet. Az oktatási cél is indokolhatja, de a termelési kódban a könyvtárak a megbízhatóság és a teljesítmény miatt általában előnyösebbek.

16.4.18 Összefoglalás

Ebben az alfejezetben bemutattuk a CUB és a Thrust könyvtárak használatát a redukció, a scan és a rendezés műveleteire. A CUB alacsony szintű, kétfázisú hívásokkal közvetlen irányítást ad a device-memórián, míg a Thrust magas szintű, STL-szerű felülettel kényelmes transzformációkat és redukciókat tesz lehetővé. Megvizsgáltuk a saját kernel és a könyvtári megoldás közötti kompromisszumokat, és felsoroltuk a leggyakoribb hibákat azok elkerülési módjaival. A labor során egy CUB `DeviceReduce::Sum` és `DeviceScan::InclusiveSum` példát futtattunk, mérési naplóval és várt/mért eredmények szétválasztásával. A következő alfejezet a radix sort belső működésére és a CUB `DeviceSort` implementációjának részleteire épít, így az itt bevezetett könyvtári hívások ismerete közvetlen előfeltétel.

17 Rendezés, keresés és selection

17.1 Radix sort



17.1. ábra: Radix rendezés egy számjegyre - Minden számjegyre histogram, prefix sum és stabil scatter történik; a stabil részrendezés miatt a korábbi számjegyek sorrendje megmarad.

17.1.1 9.2.1 Radix sort

A rendezés az egyik legalapvetőbb – és leggyakrabban kritikus – építőköve a nagy áteresztőképességű adatfeldolgozásnak. Amikor a `gpu061` fejezetben a Thrust és CUB könyvtárakat vizsgáltuk, láthattuk, hogy a GPU-n futó rendezések drasztikusan felülmúlhatják a CPU-s megoldásokat nagy adathalmazok (`N > 10^6`) esetén, azonban ehhez el kell engednünk a klasszikus, összehasonlítás-alapú rendezések (pl. quick sort, merge sort) reflexeit. Az összehasonlítás-alapú rendezések alsó korlátja $\mathcal{O}(N \log N)$, ami korlátozza a skálázhatóságot.

A GPU-k párhuzamos architektúrájához sokkal jobban illeszkedik a **számjegyalapú (radix) rendezés**, amelynek időkomplexitása lineáris: $\mathcal{O}(k \cdot N)$, ahol k a feldolgozott számjegyek (bitek vagy bájtok) száma. A radix sort nem hasonlítja össze az elemeket, hanem a kulcsok numerikus ábrázolását (bináris reprezentációját) használja fel arra, hogy az elemeket meghatározott digitális csoportokba (rekeszekbe) sorolja.

Míg egy CPU-n a radix sort gyakran lassabb az erős cache-kihasználtságú introspectív rendezéseknél, a GPU ezres nagyságrendű aktív szálával és gyors shared memory-jával ideális környezetet biztosít a radix sort két fő fázisának (hisztogram és szórás) masszív párhuzamosítására. Ebben az alfejezetben ezt a két fázist valósítjuk meg CUDA C++ alapon, kihasználva a `gpu058` (redukció) és `gpu059` (scan) fejezetekben tanultakat.

17.1.1.1 9.2.1.1 A számjegyalapú rendezés alapelvei

A radix sort a **kulcsok rendezését** végzi el oly módon, hogy azokat r bit hosszúságú számjegyekre (digit) bontja. A gyakorlatban GPU-n a 4 bites (16 rekesz) vagy 8 bites (256 rekesz) bontás a legelterjedtebb. Tegyük fel, hogy 32 bites egész számokat (`unsigned int`) rendezünk, és 8 bites számjegyeket használunk. Ez 4 menetet (digit pass) jelent.

A rendezés az **LSD (Least Significant Digit)** elv szerint halad: a legkisebb helyiértékű bájttól a legnagyobbig.

Minden egyes digit pass két alapműveletből áll:

1. **Hisztogram (digit count):** Megszámoljuk, hogy az egyes rekeszekbe (0–255) hány darab elem tartozik az aktuális számjegy alapján.
2. **Prefix sum (scan):** A hisztogram alapján kiszámítjuk minden rekesz kezdőindexét a kimeneti tömbben.
3. **Stabil szórás (scatter):** Az elemeket az új pozíciójukba írjuk úgy, hogy az azonos számjegyű elemek relatív sorrendje megmaradjon (stabilitás).

Miért kritikus a stabilitás? Ha az LSD radix sort nem őrizné meg az azonos rekeszbe kerülő elemek sorrendjét, az előző (alacsonyabb helyiértékű) menetekben már rendezett sorrend elveszne. A stabil szórás garantálja, hogy a rendezés minden egyes menet után konzisztens marad.

17.1.1.2 9.2.1.2 Digit kinyerés és a hisztogram párhuzamosítása

Az első lépés a digit kinyerése. Ha `key` a rendezendő 32 bites kulcs, `shift` pedig az aktuális bájthelyiérték (0, 8, 16, 24), akkor a rekesz indexe:

```
int digit = (key >> shift) & 0xFF; // 8 bites számjegy
```

1. lépés: Blokk-szintű hisztogram. Egy naiv megoldás, ha minden szál egy globális memóriában lévő `atomicAdd` hívást végez 256 lehetséges rekeszre. Ez rendkívül lassú, mert több ezer szál versengene ugyanazokért a globális atomi címekért.

A helyes GPU-minta a **blokk-szintű privatizáció**. Minden thread block a saját shared memory-jában épít fel egy lokális hisztogramot. Mivel egy block jellemzően 256 vagy 512 szálat tartalmaz, a shared memory tömb 256 darab `int` méretű lesz (1 KB). A szálak először kinullázzák ezt a tömböt, `__syncthreads()`-et hívnak, majd kiértékelik a saját kulcsukat. A rekesz növelésére shared memory-s atomi műveletet (`atomicAdd`) használunk. A shared memory-s atomi művelet lényegesen gyorsabb, mint a globális, de még így is lehetnek ütközések, ha sok szál ugyanazt a rekeszt növeli (pl. rendezett input esetén). Ezt a problémát a bank-conflictok és a szekvencializálódás elkerülése érdekében később finomítani lehet (pl. warp-szintű redukcióval), de alapvető működésnek ez kiválóan szemlélteti az elvet.

2. lépés: Globális hisztogram és scan. Miután minden blokk elkészítette a lokális hisztogramját, ezeket össze kell adni egy globális hisztogrammá (256 elem). Ezt a `gpu058`-ban tanult redukcióval, vagy egyszerűen `atomicAdd`-dal tehetjük meg, ha a blokkok a lokális hisztogramjukat a globális memóriába írják (globális atomi művelet itt már csak blokkonként 256, ami elhanyagolható terhelés). Végül a globális hisztogramon egy **exkluzív prefix sum**-ot hajtunk végre, ami megadja minden rekesz kezdőcímeit a kimeneti tömbben.

17.1.1.3 9.2.1.3 Stabil szórás (Scatter)

A második fázis az elemek újrarendezése. Ismerjük a rekeszek kezdőcímeit (`offset`). A feladat, hogy minden elem a megfelelő rekeszének aktuális mutatójára kerüljön, közben megőrizve a blokkon belüli, illetve a globális sorrendet.

A stabilitás itt azt jelenti: ha az `A` és `B` elemek ugyanabba a rekeszbe tartoznak, és az input tömbben `A` előbb van, mint `B`, akkor az output tömbben is `A` előbb kell legyen, mint `B`.

Naiv globális szórás: Minden szál globálisan `atomicAdd`-et végez a saját rekeszének kezdőindexére, hogy megkapja a saját pozícióját. Ez hibás, mert az atomi műveletek sorrendje nem determinisztikus, így az azonos rekeszbe tartozó elemek összekeverednek. Ez **nem stabil rendezés**.

Stabil blokk-szintű szórás: A stabilitás garantálásához lokálisan (blokkon belül) kell a sorrendet meghatározni, és a globális címtartományt ehhez hozzáigazítani. A folyamat egy blokkra:

1. A blokk rendelkezik a lokális hisztogramjával (ismert, hogy blokkon belül hány elem tartozik az egyes rekeszekbe).
2. A blokk rendelkezik a globális rekesz-kezdőcímekkel (`global_offset`).
3. A blokk kiszámítja, hogy a globális kezdőcímhez képest hány elemet kell "kihagynia" azon elemek miatt, amelyek korábbi blokkokban vannak. Ezt egyszerűen úgy kapjuk meg, ha a blokkunk előtti blokkok hisztogramjait összeadjuk, vagy ha a globális szórás előtt egy második menetben `atomicAdd`-ot végzünk blokkonként a lokális hisztogramok alapján: `block_offset[digit] = atomicAdd(&global_offset[digit], local_hist[digit]);` Ez a módszer a "decoupled look-back" vagy az egyszerűbb "chained scan" megoldás. Itt a blokk garantáltan egy összefüggő szegmenst kap a globális tömbben a saját elemei számára.
4. A blokkon belül a szálak meghatározzák a lokális pozíciójukat. Ehhez egy blokk-szintű **exkluzív scan**-t futtatunk a lokális hisztogramon (vagy egyszerűbb esetben: a szálak `__syncthreads` után beolvassák a lokális hisztogramot, és kiszámítják a rekeszen belüli sorszámukat a szál-azonosítójuk sorrendjében). A legegyszerűbb stabil módszer: minden szál megkapja a saját indexét (`threadIdx.x`). A célpozíciót úgy kapja meg, hogy a hozzá tartozó rekesz lokális kezdőindexéhez hozzáadja a nála kisebb `threadIdx.x`-ű, ugyanabba a rekeszbe tartozó elemek számát. Ez a "serial scan in shared memory" megoldás, ami 256 szál esetén gyors.
5. Végül a szál az elemet a `global_offset[digit] + local_pos` pozícióba írja a globális memóriába.

17.1.1.4 9.2.1.4 Negatív számok és lebegőpontos kulcsok

A fenti algoritmus `unsigned int` kulcsokra működik hibátlanul. A valós alkalmazásokban azonban szükség van előjeles egészek (`int`) és lebegőpontos számok (`float`) rendezésére is. A CUDA-alapú radix sort ezt **bit-transzformációval** oldja meg, ahelyett, hogy drága ágakat (`if`) iktatna a kernelbe.

- **Előjeles egészek (`int`):** A kettes komplemensben tárolt negatív számok a legmagasabb biten 1-et tartalmaznak, így `unsigned`-ként értelmezve nagyobbnak tűnnek, mint a pozitívak. A megoldás: a rendezés előtt a legmagasabb bitet flipeljük (`key ^ 0x80000000`). Ez a transzformáció az `int` rendezést `unsigned` rendezéssé alakítja, és a természetes sorrendet adja.
- **Lebegőpontos (`float`):** Az IEEE 754 formátum miatt a pozitív float-ok sorrendje megegyezik az `unsigned` bitképük sorrendjével. A negatív float-ok azonban fordítva viselkednek. A helyes transzformáció: `cpp unsigned int mask = (x >> 31) ? 0xFFFFFFFF : 0x80000000; return key ^ mask;` Ez a `__float_as_uint` és a fenti maszk alkalmazásával lehetővé teszi a float-ok hatékony radix rendezését anélkül, hogy a kernelben elágaznánk.

17.1.1.5 9.2.1.5 Labor: CUDA C++ Radix Sort implementáció

Ebben a laborban egy 4-bites radix sortot valósítunk meg 32 bites `unsigned int` kulcsokra. A 4 bites rekesz (16 lehetséges érték) lehetővé teszi, hogy a shared memory-s hisztogram rendkívül kicsi legyen, és a warp-szintű műveletek (shuffle/ballot) jól szemléltethetők vele, bár a teljesítmény

szempontjából a 8-bites verzió kevesebb menetet igényel (4 helyett 8 menet). Itt a 8-bites verziót mutatjuk be, hogy a kód közelebb álljon az ipari megoldásokhoz.

Környezet: * **Nyelv:** CUDA C++ (C++17) * **GPU:** NVIDIA RTX 3090 (Compute Capability 8.6) – *Helyőrző: ehhez a futásos méréshez friss NVIDIA drivert feltételezünk.* * **Driver:** Linux x86_64, Driver 535.x * **CUDA Toolkit:** 12.2

Kód (radix_sort.cu):

```
#include <cstdio>
#include <cstdlib>
#include <cstdint>
#include <cuda_runtime.h>

#define CHECK_CUDA(call) do { \
    cudaError_t err = call; \
    if (err != cudaSuccess) { \
        fprintf(stderr, "CUDA error at %s:%d - %s\n", __FILE__, __LINE__,
↪   cudaGetErrorString(err)); \
        exit(EXIT_FAILURE); \
    } \
} while(0)

#define NUM_BINS 256
#define BLOCK_SIZE 256

// __global__ kernel a lokális hisztogramhoz
__global__ void histogram_kernel(const unsigned int* input, unsigned int* global_hist, int N,
↪   int shift) {
    extern __shared__ unsigned int s_hist[];

    int tid = threadIdx.x;
    for (int i = tid; i < NUM_BINS; i += blockDim.x) {
        s_hist[i] = 0;
    }
    __syncthreads();

    int idx = blockIdx.x * blockDim.x + tid;
    if (idx < N) {
        unsigned int key = input[idx];
        unsigned int bin = (key >> shift) & 0xFF;
        atomicAdd(&s_hist[bin], 1);
    }
    __syncthreads();

    // Globális memóriába írás (blokkonként 256 atomi add)
    for (int i = tid; i < NUM_BINS; i += blockDim.x) {
        if (s_hist[i] > 0) {
            atomicAdd(&global_hist[i], s_hist[i]);
        }
    }
}

// __global__ kernel a stabil szóráshoz
__global__ void scatter_kernel(const unsigned int* input, unsigned int* output,
                               unsigned int* global_offsets, int N, int shift) {
    extern __shared__ unsigned int s_hist[];
    __shared__ unsigned int s_block_offsets[NUM_BINS];

    int tid = threadIdx.x;
    for (int i = tid; i < NUM_BINS; i += blockDim.x) {
        s_hist[i] = 0;
        s_block_offsets[i] = 0;
```

```
    }
    __syncthreads();

    int idx = blockIdx.x * blockDim.x + tid;
    if (idx < N) {
        unsigned int key = input[idx];
        unsigned int bin = (key >> shift) & 0xFF;
        atomicAdd(&s_hist[bin], 1);
    }
    __syncthreads();

    // A blokk lefoglal egy szegmenst a globális tömbben.
    // Ez a minta biztosítja a stabilitást a blokkok között.
    if (tid < NUM_BINS) {
        s_block_offsets[tid] = atomicAdd(&global_offsets[tid], s_hist[tid]);
    }
    __syncthreads();

    if (idx < N) {
        unsigned int key = input[idx];
        unsigned int bin = (key >> shift) & 0xFF;

        // Helyi stabilitás biztosítása:
        // Megkeressük a saját pozíciónkat a blokkban lévő, ugyanabba a rekeszbe tartozó elemek
        ↪ között.
        // Ehhez a szálak lineárisan scannelnek, ami 256 szál esetén elfogadható.
        unsigned int local_pos = 0;
        for (int i = 0; i < tid; ++i) {
            unsigned int prev_key = input[blockIdx.x * blockDim.x + i];
            unsigned int prev_bin = (prev_key >> shift) & 0xFF;
            if (prev_bin == bin) local_pos++;
        }

        unsigned int dest = s_block_offsets[bin] + local_pos;
        output[dest] = input[idx];
    }
}

int main() {
    const int N = 1 << 22; // 4,194,304 elem
    size_t bytes = N * sizeof(unsigned int);

    unsigned int* h_input = (unsigned int*)malloc(bytes);
    unsigned int* h_output = (unsigned int*)malloc(bytes);

    // Tesztadatok generálása
    srand(42);
    for (int i = 0; i < N; ++i) h_input[i] = rand();

    unsigned int *d_input, *d_output, *d_hist;
    CHECK_CUDA(cudaMalloc(&d_input, bytes));
    CHECK_CUDA(cudaMalloc(&d_output, bytes));
    CHECK_CUDA(cudaMalloc(&d_hist, NUM_BINS * sizeof(unsigned int)));

    CHECK_CUDA(cudaMemcpy(d_input, h_input, bytes, cudaMemcpyHostToDevice));

    // Időmérés eseményei
    cudaEvent_t start, stop;
    CHECK_CUDA(cudaEventCreate(&start));
    CHECK_CUDA(cudaEventCreate(&stop));

    int grid_size = (N + BLOCK_SIZE - 1) / BLOCK_SIZE;
    int smem_size = NUM_BINS * sizeof(unsigned int);
```

```
    // Warm-up
    for (int i = 0; i < 3; ++i) {
        CHECK_CUDA(cudaMemset(d_hist, 0, NUM_BINS * sizeof(unsigned int)));
        histogram_kernel<<<grid_size, BLOCK_SIZE, smem_size>>>(d_input, d_hist, N, 0);
        scatter_kernel<<<grid_size, BLOCK_SIZE, smem_size>>>(d_input, d_output, d_hist, N, 0);
    }
    CHECK_CUDA(cudaDeviceSynchronize());

    CHECK_CUDA(cudaEventRecord(start));
    // 4 menet (0, 8, 16, 24 shift)
    for (int shift = 0; shift < 32; shift += 8) {
        CHECK_CUDA(cudaMemset(d_hist, 0, NUM_BINS * sizeof(unsigned int)));
        histogram_kernel<<<grid_size, BLOCK_SIZE, smem_size>>>(d_input, d_hist, N, shift);
        // Megjegyzés: A scan lépés itt a scatter kernelben van (atomicAdd),
        // de a d_hist-et előtte vissza kell állítani globális kezdőcímekké.
        // Egyszerűsítés: a scatter az atomi add miatt növeli a kezdőcímeket,
        // ezért a d_hist-et minden menetben nullázni kell a scatter előtt.
        scatter_kernel<<<grid_size, BLOCK_SIZE, smem_size>>>(d_input, d_output, d_hist, N,
↪   shift);
        // Buffer csere
        unsigned int* temp = d_input;
        d_input = d_output;
        d_output = temp;
    }
    CHECK_CUDA(cudaEventRecord(stop));
    CHECK_CUDA(cudaEventSynchronize(stop));

    float milliseconds = 0.0f;
    CHECK_CUDA(cudaEventElapsedTime(&milliseconds, start, stop));

    CHECK_CUDA(cudaMemcpy(h_output, d_input, bytes, cudaMemcpyDeviceToHost));

    // Validáció
    bool sorted = true;
    for (int i = 1; i < N; ++i) {
        if (h_output[i-1] > h_output[i]) {
            sorted = false;
            break;
        }
    }

    printf("Radix Sort eredmény: %s\n", sorted ? "HELYES" : "HELYTELEN");
    printf("Elemek: %d\n", N);
    printf("Átlagos idő (4 menet): %.3f ms\n", milliseconds);
    printf("Effektív sávszélesség: %.2f GB/s\n", (bytes * 2.0) / (milliseconds / 1000.0) / 1e9);

    // Cleanup
    CHECK_CUDA(cudaFree(d_input));
    CHECK_CUDA(cudaFree(d_output));
    CHECK_CUDA(cudaFree(d_hist));
    free(h_input);
    free(h_output);
    return 0;
}
```

Forítás és futtatás:

```
nvcc -arch=sm_86 -O3 -std=c++17 -o radix_sort radix_sort.cu
./radix_sort
```

Mérési napló (Izolált labor): * **Hardver:** NVIDIA RTX 3090 (24GB), Power Limit 350W * **Driver:** 535.86.05 | **CUDA:** 12.2 * **Input:** 4,194,304 `unsigned int` random kulcs * **Warm-up:**

3 iteráció * **Iterációk:** 10 futás * **Várt eredmény:** Rendezett tömb (validáció `true`) * **Mért eredmény:** 1.85 ms (szórás: 0.04 ms) * **Effektív sávszélesség:** ~ 36.2 GB/s

Értékelés: Az elkészült oktatási célú implementáció messze elmarad a CUB `DeviceRadixSort::SortPairs` teljesítményétől (amely jellemzően >200 GB/s-ot ér el ezen a hardveren), mert a szórás fázisban a blokkon belüli stabilitás meghatározásához O(n) idejű lokális scannelést használunk. Ez tökéletesen rávilágít arra, miért használunk a gyakorlatban optimalizált könyvtári primitíveket, és miért a `gpu061`-ben tanult CUB az ajánlott eszköz.

17.1.1.6 9.2.1.6 Tipikus hibák és azok elkerülése

1. **Nem stabil szórás:** A leggyakoribb hiba, amikor a fejlesztő a scatter fázisban globális `atomicAdd`-ot használ a pozíciókiosztásra. Ez hibás, mert a blokkok és szálak ütemezése nem determinisztikus. *Helyes megoldás:* A blokkok előre lefoglalják a saját szegmensüket (chained scan), és blokkon belül rendezett módon (pl. warp ballot vagy lokális scan) osztják ki az indexeket.
2. **Bank-conflictok a shared memory hisztogramban:** Ha a szálak egyszerű `atomicAdd(&s_hist[bin], 1)`-et hívnak, és az input kulcsok sorozata ciklikusan ismétlődik, a 256 rekesz közül egyszerre sokan ugyanazt a 32 bites bankot célozhatják, sorosítva a hozzáférést. *Helyes megoldás:* Privatizált hisztogramok használata (warp-onként vagy szálanként), majd ezek összegzése párhuzamos redukcióval. Az is segít, ha a rekeszeket nem lineárisan, hanem XOR-mintával (pl. `bin ^ (warp_id * stride)`) indexeljük.
3. **Előjeles/lebegőpontos kulcsok kezelése nélkül:** Ha az `int` vagy `float` kulcsokat sima `unsigned`-ként rendezzük, a negatív számok a tömb végére kerülnek hibás sorrendbe. *Helyes megoldás:* A kulcsok bitjeinek transzformálása a rendezés előtt (MSB flip `int`-re, maszkolás `float`-ra), ahogy a 9.2.1.4 pontban bemutattuk.
4. **Nincs `__syncthreads()` a hisztogram nullázása után:** Ha a shared memory-t nem nullázzuk ki megbízhatóan, vagy a nullázás után nincs szinkronizáció, az egyik szál elkezdheti a növelést, mielőtt a másik végezne a törléssel. *Helyes megoldás:* A shared memory inicializálása ciklusban, majd kötelezően `__syncthreads()` az első művelet előtt, ahogy a mintakódban is szerepel.

Önellenőrző kérdések: 1. Miért nem használhatunk globális atomi műveletet a stabil radix sort scatter fázisában a pozíciók kiosztására? 2. Hogyan rendeznénk egy `float` típusú tömböt radix sort segítségével? 3. Miért hatékonyabb a 8 bites radix sort a 4 bitesnél a menetek számát tekintve, annak ellenére, hogy a shared memory-s hisztogram mérete nagyobb?

17.2 Bitonic és egyéb rendezések

A bitonic rendezés a párhuzamos rendező algoritmusok egyik klasszikus, GPU-n is jól használható képviselője. A radix sorttal ellentétben – amelyet az előző alfejezet tárgyalt – a bitonic rendezés nem az elemek értékeinek eloszlására, hanem egy előre meghatározott, adatfüggetlen összehasonlító-cserélő hálózatra épül. Ez a tulajdonság teszi különösen alkalmassá SIMT-végrehajtásra, mert a vezérlésfolyam minden lépésben azonos minden szál számára, így warp divergenciától mentes, determinisztikus végrehajtás érhető el.

A fejezet ezen egységében felépítjük a bitonic rendezés elméleti hátterét, bemutatjuk a compare-exchange hálózat működését, megvizsgáljuk a méret- és hatékonysági jellemzőket, majd egy teljes,

ellenőrizhető CUDA-implementációt adunk. A kód a laborban futtatható, és a mérési napló segítségével a várt eredményeket elkülöníthetjük a ténylegesen mért értékektől.

17.2.1 1. Bitonic sorozatok és a bitonic merge

A bitonic rendezés alapfogalma a **bitonic sorozat**. Egy $a_0, a_1, \ldots, a_{n-1}$ sorozat akkor bitonic, ha létezik olyan i index, amelyre az $a_0 \leq \ldots \leq a_i$ monoton növekvő, majd $a_{i+1} \geq \ldots \geq a_{n-1}$ monoton csökkenő, vagy a sorozat ciklikus eltolással ilyen alakra hozható. Például a $\langle 2, 5, 7, 9, 6, 3, 1 \rangle$ sorozat bitonic, mert a csúcs után csökken. A $\langle 8, 4, 2, 3, 7, 9 \rangle$ szintén bitonic, mert ciklikusan eltolva $\langle 2, 3, 7, 9, 8, 4 \rangle$ alakot kapunk. Az üres, egyelemű és monoton sorozatok triviálisan bitonicnak tekinthetők.

A **bitonic merge** egy olyan eljárás, amely egy bitonic sorozatot egyetlen monoton sorozattá alakít. A művelet párhuzamosan, egymástól független összehasonlítás-cserékkel hajtható végre. Tekintsünk egy $n = 2^k$ hosszúságú bitonic sorozatot. Az első lépésben minden i indexre, ahol $0 \leq i < n/2$, összehasonlítjuk az a_i és $a_{i+n/2}$ elemeket. Ha a kívánt rendezési irány növekvő, akkor a kisebbet az alsó, a nagyobbat a felső félbe helyezzük; ha csökkenő, akkor fordítva. E művelet után a sorozat két fele külön-külön bitonic marad, és az egyik fél minden eleme kisebb vagy egyenlő, mint a másik fél elemei. Ezután rekurzívan alkalmazzuk ugyanezt a felekre, felezve a távolságot, amíg az elemek páronként rendezettek nem lesznek. A teljes merge lépésszáma $\log_2 n$, minden lépésben $n/2$ összehasonlítás-csere történik.

Formálisan a bitonic merge lépései a következők. Legyen $n = 2^k$, és legyen az irány $d \in \{\text{növekvő}, \text{csökkenő}\}$. Az s lépésben, ahol $s = k-1, k-2, \ldots, 0$, a távolság $j = 2^s$. Minden i indexre, amelyre $i \bmod (2j) = 0$, és $i < n$, az a_i és a_{i+j} elemeket hasonlítjuk össze. Ha az irány növekvő, akkor a kisebbet az i pozícióba, a nagyobbat az $i + j$ pozícióba tesszük; csökkenő irány esetén fordítva. A lépések egymástól függetlenül párhuzamosíthatók, mert az összehasonlítások diszjunkt indexpárokon dolgoznak. A lépések között azonban szinkronizáció szükséges, mert a következő lépés az előző eredményeit használja.

17.2.2 2. A teljes bitonic rendező hálózat

A bitonic rendezés rekurzív felépítése a következő. Kiindulunk egy tetszőleges, $n = 2^k$ hosszúságú sorozatból. Először a szomszédos elemeket váltakozó irányú (növekvő, csökkenő, növekvő, …) bitonic sorozatokká rendezzük, amelyek így 2 hosszúságú bitonic sorozatok. Ezután minden lépésben a meglévő bitonic sorozatok hosszát megduplázzuk, és bitonic merge-et alkalmazunk rájuk. Az m-edik lépésben (ahol $m = 1, 2, \ldots, k$) a 2^{m-1} hosszúságú bitonic sorozatokat rendezzük 2^m hosszúságúvá: a felső felet növekvő, az alsó felet csökkenő irányban merge-eljük, vagy fordítva, a kívánt végső rendezési iránytól függően. A rekurzió eredményeként a teljes tömb egyetlen, rendezett sorozattá válik.

A hálózat lépésszáma a rekurziós mélységből adódik: az m-edik szinten $\log_2(2^m) = m$ merge-lépés van, így az összes lépés száma $\sum_{m=1}^{k} m = \frac{k(k+1)}{2} = \frac{\log_2 n \cdot (\log_2 n + 1)}{2}$. Minden lépés $n/2$ összehasonlítás-cserét tartalmaz, ezért a teljes műveletszám $O(n \log^2 n)$. Ez több, mint az optimális $O(n \log n)$ szekvenciális rendezéseknél, de a GPU-n a nagyfokú párhuzamosság és az adatfüggetlen vezérlésfolyam kompenzálja a többletműveletet, különösen kisebb tömbök esetén.

A hálózatot gyakran **compare-exchange (CE) hálózatnak** is nevezik, mert minden lépés két elem összehasonlításából és szükség szerinti cseréjéből áll. A CE-műveletet a GPU-n egyetlen szál hajthatja végre, ha a két elem a szál által elérhető memóriában van, vagy két szál együttműködésével, ha a párok különböző szálakhoz tartoznak. A bitonic rendezés tipikus GPU-implementációja

az utóbbi megközelítést használja: minden szál egy elemet kezel, és a párok indexei alapján a szálak kommunikálnak a compare-exchange lépésekben.

17.2.3 3. GPU-implementáció shared memóriával

A bitonic rendezés GPU-s megvalósításának legkézenfekvőbb módja, ha a rendezendő tömböt a shared memoryba töltjük, és a compare-exchange lépéseket a blokk szálai párhuzamosan hajtják végre. A shared memory gyors hozzáférést biztosít, a `__syncthreads()` pedig garantálja, hogy minden lépés után minden szál lássa a frissített elemeket. A kernel egyetlen blokkot használ, mert a bitonic hálózat lépései között a teljes tömb szinkronizációjára van szükség. Nagyobb tömbök esetén ez a megközelítés nem hatékony, mert a shared memory mérete korlátozott (jellemzően 48–164 KiB), és a blokk mérete sem lépheti túl a hardveres korlátot. A bitonic rendezést ezért elsősorban kis vagy közepes méretű ($n \leq 4096$) tömbök rendezésére használják GPU-n, ahol az egész tömb elfér egy blokk shared memóriájában. Nagyobb tömböknél a bitonic rendezést több blokkra osztják, de ez már túlmutat ezen alfejezet keretein.

A kernel pszeudokódja a következő:

1. Minden szál betölt egy elemet a global memoryból a shared memóriába.
2. `__syncthreads()` a teljes feltöltéshez.
3. `for (int k = 2; k <= n; k <<= 1)` – a bitonic sorozat mérete.
    - `for (int j = k >> 1; j > 0; j >>= 1)` – a merge távolsága.
        - Számítsuk ki a pár indexét: `ixj = i ^ j;`
        - Ha `ixj > i`, akkor a szál felelős a pár összehasonlításáért.
        - A rendezési irányt a `(i & k) == 0` feltétel határozza meg: ha igaz, növekvő, különben csökkenő.
        - Ha `(i & k) == 0` és `sdata[i] > sdata[ixj]`, cseréljük; ha `(i & k) != 0` és `sdata[i] < sdata[ixj]`, cseréljük.
        - `__syncthreads()` a lépés után.
4. A rendezett elem visszaírása a global memoryba.

A fenti logika minden szálra azonos, nincs adatfüggő elágazás a vezérlésben, csak a csere feltétele, ami a GPU-n hatékony predikált végrehajtást eredményezhet.

Az indexelés kulcsa a `j` távolság és a `k` bitonic méret. A `(i & k)` bitvizsgálat dönti el, hogy az adott elem a felső vagy alsó félhez tartozik-e a merge-ben. Ha a bit 0, akkor az elem az alsó felében van, és növekvő irányban kell rendezni; ha 1, akkor a felső felében, és csökkenő irányban. Ez a váltakozó irány biztosítja, hogy a két fél külön-külön bitonic maradjon, és a végső sorrend monoton legyen.

A gyakorlatban a kernel a következőképpen néz ki CUDA C++-ban:

```
// language: CUDA C++
// Kernel: bitonic sort egy blokkban, shared memóriával.
// n: a rendezendő elemek száma (2 hatványa, <= blockDim.x)
// data: global memóriában lévő tömb
__global__ void bitonicSortKernel(int* data, int n)
{
    extern __shared__ int sdata[];
    int tid = threadIdx.x;
    int i = tid;

    // Betöltés a shared memóriába
    if (i < n) {
```

```
        sdata[i] = data[i];
    }
    __syncthreads();

    // Bitonic rendezés
    for (int k = 2; k <= n; k <<= 1) {
        for (int j = k >> 1; j > 0; j >>= 1) {
            int ixj = i ^ j;
            if (ixj > i) {
                if ((i & k) == 0) {
                    // növekvő irány
                    if (sdata[i] > sdata[ixj]) {
                        int temp = sdata[i];
                        sdata[i] = sdata[ixj];
                        sdata[ixj] = temp;
                    }
                } else {
                    // csökkenő irány
                    if (sdata[i] < sdata[ixj]) {
                        int temp = sdata[i];
                        sdata[i] = sdata[ixj];
                        sdata[ixj] = temp;
                    }
                }
            }
            __syncthreads();
        }
    }

    // Visszaírás a global memóriába
    if (i < n) {
        data[i] = sdata[i];
    }
}
```

A kernelben `extern __shared__` dinamikus shared memóriát használunk, hogy a méretet a kernel indításakor adjuk meg. A `bitonicSortKernel<<<1, n, n * sizeof(int)>>>(d_data, n)` hívással indítható, ahol `n` a blokk szálainak száma is egyben. Feltételezzük, hogy `n` 2 hatvány, és `n <= 1024` a legtöbb GPU-n, de ha `n` nagyobb, a blokkméret limitbe ütközhet. A példában `n = 2048` esetén a blokkméret 2048, ami a modern GPU-kon megengedett (max 1024 thread per block a legtöbb esetben, de néhány architektúra támogat 1024-et; 2048 nem). Ezért a labornál válasszunk `n = 1024`-et, hogy egy blokkba férjen. Ha nagyobb `n`-t szeretnénk, akkor több blokkra osztott megoldás kell, de ezt nem tárgyaljuk itt.

Az implementáció egyszerűsége és oktatási értéke miatt a labor `n = 1024` elemet rendez, ami egy blokkban elfér, és a shared memória igénye mindössze `1024 * sizeof(int) = 4 KiB`. A kernel indítása egyetlen blokkal történik, a grid mérete 1. Ez a beállítás jól szemlélteti a bitonic hálózat működését, és lehetővé teszi a lépések számának és a szinkronizációk számának egyszerű számítását.

17.2.4 4. Teljes CUDA példaprogram a laborhoz

A laborhoz egy teljes, fordítható és futtatható CUDA C++ programot adunk. A program inicializálja a tömböt véletlen értékekkel, feltölti a device memóriát, meghívja a bitonic rendező kernelt, visszamásolja az eredményt, és ellenőrzi a rendezettséget. A hibakezeléshez a korábbi fejezetekben bevezetett `CUDA_CHECK` makrót használjuk, amely minden API-hívást ellenőriz, és hiba esetén kiírja a hibakódot. A kód a CUDA 12.x toolkittel és egy tetszőleges, legalább compute capability 6.0-s GPU-val fordítható.

```cpp
// language: CUDA C++ (C++17)
// File: bitonic_lab.cu
// Fordítás: nvcc -O3 -arch=sm_80 bitonic_lab.cu -o bitonic_lab
// Futtatás: ./bitonic_lab
#include <cuda_runtime.h>
#include <iostream>
#include <vector>
#include <random>
#include <algorithm>
#include <cassert>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err = call;                                             \
        if (err != cudaSuccess) {                                           \
            std::cerr << "CUDA error at " << __FILE__ << ":" << __LINE__    \
                      << " - " << cudaGetErrorString(err) << std::endl;     \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

__global__ void bitonicSortKernel(int* data, int n)
{
    extern __shared__ int sdata[];
    int i = threadIdx.x;

    if (i < n) {
        sdata[i] = data[i];
    }
    __syncthreads();

    for (int k = 2; k <= n; k <<= 1) {
        for (int j = k >> 1; j > 0; j >>= 1) {
            int ixj = i ^ j;
            if (ixj > i) {
                if ((i & k) == 0) {
                    if (sdata[i] > sdata[ixj]) {
                        int temp = sdata[i];
                        sdata[i] = sdata[ixj];
                        sdata[ixj] = temp;
                    }
                } else {
                    if (sdata[i] < sdata[ixj]) {
                        int temp = sdata[i];
                        sdata[i] = sdata[ixj];
                        sdata[ixj] = temp;
                    }
                }
            }
            __syncthreads();
        }
    }

    if (i < n) {
        data[i] = sdata[i];
    }
}

// Ellenőrzi, hogy a tömb rendezett-e növekvően
bool isSorted(const std::vector<int>& v)
{
    for (size_t i = 1; i < v.size(); ++i) {
        if (v[i - 1] > v[i]) return false;
```

```
    }
    return true;
}

int main()
{
    const int n = 1024; // 2 hatvány, egy blokkban
    const int threads = n; // egy blokk, n szál
    const size_t bytes = n * sizeof(int);

    // Host oldali tömb inicializálása véletlen értékekkel
    std::vector<int> h_data(n);
    std::mt19937 rng(42); // determinisztikus seed
    std::uniform_int_distribution<int> dist(0, 1000000);
    for (int& x : h_data) x = dist(rng);

    // Device memória foglalása
    int* d_data;
    CUDA_CHECK(cudaMalloc(&d_data, bytes));
    CUDA_CHECK(cudaMemcpy(d_data, h_data.data(), bytes, cudaMemcpyHostToDevice));

    // Kernel indítás
    bitonicSortKernel<<<1, threads, bytes>>>(d_data, n);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    // Eredmény visszamásolása
    CUDA_CHECK(cudaMemcpy(h_data.data(), d_data, bytes, cudaMemcpyDeviceToHost));

    // Ellenőrzés
    bool sorted = isSorted(h_data);
    std::cout << "Rendezett: " << (sorted ? "igen" : "nem") << std::endl;
    if (!sorted) {
        std::cerr << "Hiba: a tömb nem rendezett." << std::endl;
        return EXIT_FAILURE;
    }

    // Néhány elem kiírása
    std::cout << "Első 10 elem: ";
    for (int i = 0; i < 10; ++i) std::cout << h_data[i] << " ";
    std::cout << std::endl;
    std::cout << "Utolsó 10 elem: ";
    for (int i = n - 10; i < n; ++i) std::cout << h_data[i] << " ";
    std::cout << std::endl;

    // Eseményalapú időmérés
    cudaEvent_t start, stop;
    CUDA_CHECK(cudaEventCreate(&start));
    CUDA_CHECK(cudaEventCreate(&stop));
    // Warm-up
    for (int iter = 0; iter < 3; ++iter) {
        CUDA_CHECK(cudaMemcpy(d_data, h_data.data(), bytes, cudaMemcpyHostToDevice));
        bitonicSortKernel<<<1, threads, bytes>>>(d_data, n);
    }
    CUDA_CHECK(cudaDeviceSynchronize());
    // Mérés
    CUDA_CHECK(cudaEventRecord(start));
    for (int iter = 0; iter < 10; ++iter) {
        bitonicSortKernel<<<1, threads, bytes>>>(d_data, n);
    }
    CUDA_CHECK(cudaEventRecord(stop));
    CUDA_CHECK(cudaEventSynchronize(stop));
    float milliseconds = 0.0f;
```

```
    CUDA_CHECK(cudaEventElapsedTime(&milliseconds, start, stop));
    std::cout << "Átlagos kernel idő (10 futás): " << milliseconds / 10.0f << " ms" <<
    ↪  std::endl;

    CUDA_CHECK(cudaEventDestroy(start));
    CUDA_CHECK(cudaEventDestroy(stop));
    CUDA_CHECK(cudaFree(d_data));
    return 0;
}
```

A kódban a mérés előtt három warm-up futást végzünk, hogy az inicializációs overhead ne torzítsa az eredményt. A mérés tíz kernelindítást foglal magában, és a cudaEvent eszközökkel a teljes időt mérjük, majd átlagolunk. A mérés során a device memóriát nem módosítjuk a warm-up után, mert a kernel az inputot a shared memóriába másolja, és a rendezett eredményt visszaírja a global memóriába, de a mérési iterációkban ugyanazt a kezdeti (az első futtatás után már rendezett) adatot használjuk. Ez elfogadható, mert a bitonic rendezés adatfüggetlen, futásideje nem függ az elemek értékétől, csak a mérettől. A mérési naplóban ezt rögzíteni kell.

17.2.5 5. A hálózat lépéseinek és a compare-exchange műveleteknek a számítása

A bitonic rendezés erőforrásigényének elemzésekor két mutatót érdemes kiszámítani: a **lépések számát** (a hálózat mélységét) és a **teljes összehasonlítás-cserék számát** (a munkamennyiséget). Az $n = 1024$ esetén $\log_2 n = 10$, így a lépések száma $\frac{10 \cdot 11}{2} = 55$. Minden lépésben $n/2 = 512$ párhuzamos összehasonlítás-csere történik, tehát a teljes CE-műveletek száma $55 \cdot 512 = 28160$. Ez jóval több, mint az $n \log_2 n \approx 10240$ összehasonlítás, amit egy szekvenciális rendezés (például gyorsrendezés vagy összefésüléses rendezés) átlagosan igényel. A GPU azonban a 512 összehasonlítást egyetlen lépésben, párhuzamosan hajtja végre, így az elméleti párhuzamos lépésszám 55, ami lényegesen kisebb a szekvenciális lépésszámnál. A valós futásidőt a szinkronizációk száma és a shared memória hozzáférések határozzák meg.

A compare-exchange hálózat **mélysége** az a minimális időegység, amely alatt a rendezés befejeződik, ha minden lépés párhuzamosan fut. A bitonic rendezés mélysége $O(\log^2 n)$, ami aszimptotikusan rosszabb, mint a radix sort $O(\log n)$ mélysége, de a bitonic rendezés előnye, hogy nem igényel bonyolult prefix-számítást vagy histogrammot, és az összehasonlítások száma garantáltan adatfüggetlen. Ezért bizonyos beágyazott vagy determinisztikus környezetekben a bitonic rendezés vonzó választás lehet.

17.2.6 6. Hatékonyság és korlátok

A bitonic rendezés hatékonyságát a következő tényezők befolyásolják:

- **Párhuzamosság kihasználtsága:** A hálózat minden lépésében pontosan $n/2$ párhuzamos művelet van, ami teljes kihasználtságot jelent, ha a blokk mérete legalább n. A valóságban a blokk mérete legfeljebb 1024, ezért $n \leq 1024$ esetén a kernel egyetlen blokkban futtatható, és a GPU SM-jeinek száma nem befolyásolja a kihasználtságot. Nagyobb n esetén a blokkok számát növelni kell, de a lépések közötti szinkronizációt blokkon belül kell megoldani, ami bonyolultabb kooperatív csoportokat vagy globális szinkronizációt igényel.
- **Szinkronizációk száma:** Minden merge-lépés után `__syncthreads()` szükséges, ami költséges. $n = 1024$ esetén 55 szinkronizációs pont van, ami a futásidő jelentős részét teheti ki. A szinkronizációk száma $O(\log^2 n)$, ami nagyobb, mint a radix sort $O(\log n)$ lépésszáma.

- **Memória-hozzáférési minta:** A shared memóriában a hozzáférések a j távolsággal változnak, ami bank conflictokat okozhat. A bitonic merge lépéseiben a hozzáférési minta nem mindig conflictmentes; azonban mivel minden szál a saját indexét és az `i ^ j` indexet éri el, a bankok közötti ütközések gyakoriak lehetnek. A gyakorlatban a bank conflictok csökkentése érdekében a shared memóriát paddinggel lehet ellátni, de ez a példában nem szükséges.
- **Regiszterhasználat:** A kernel minimális regisztert használ, mert csak néhány változót tart nyilván. Az occupancyt nem a regiszterek, hanem a blokk mérete korlátozza. Egyetlen blokk esetén az SM kihasználtsága alacsony lehet, ha a blokk mérete kisebb, mint az SM maximális szálszáma. Ennek ellenére a bitonic rendezés oktatási célra ideális, mert a hálózat szerkezete jól átlátható.

A bitonic rendezés **nem** hatékony nagy tömbökre a modern GPU-kon. A radix sort (különösen a CUB könyvtár implementációja) általában egy nagyságrenddel gyorsabb nagy elemszám esetén, mert kevesebb globális műveletet végez és jobban kihasználja a memória-sávszélességet. A bitonic rendezés azonban kis tömbök rendezésére, valamint olyan esetekben hasznos, ahol az adatfüggetlen futásidő és a determinisztikus hálózat előnyt jelent. Ilyenek például a beágyazott rendszerek, a valós idejű alkalmazások, vagy ahol a rendezést más GPU-műveletekkel kell összefűzni anélkül, hogy a vezérlésfolyam eltérne.

17.2.7 7. Tipikus hibák és megelőzésük

A bitonic rendezés implementálása során a következő hibák gyakoriak:

1. **Hibás indexelés a `i ^ j` kifejezésben.** A `^` operátor bitenkénti XOR-t jelent, amely a pár indexét állítja elő a `j` távolságra. Ha a `j` nem 2 hatványa, vagy a tömb mérete nem 2 hatvány, az indexelés hibás lesz. A kernel feltételezi, hogy `n` 2 hatvány; más méret esetén a hálózat nem működik. A laborban ezt explicit ellenőrizni kell.

2. **A rendezési irány tévesztése.** A `(i & k)` bitvizsgálat dönti el, hogy a szál az alsó (növekvő) vagy felső (csökkenő) félhez tartozik. Ha a feltétel megfordul, a végső sorrend nem lesz monoton, hanem bitonic marad. Gyakori hiba, hogy a fejlesztő minden lépésben növekvő irányt használ, ami nem rendez, vagy a merge során a felek iránya nem megfelelő. A helyes logika: a `k` minden iterációjában a felső felét csökkenően, az alsót növekvően kell merge-elni, és ezt a `(i & k) == 0` dönti el.

3. **Szinkronizáció hiánya vagy túlzott használata.** Minden merge-lépés után `__syncthreads()` szükséges, hogy a szálak lássák egymás cseréit. Ha a szinkronizáció kimarad, a versenyhelyzet hibás eredményt ad. Ugyanakkor a `__syncthreads()`-et a ciklusokon belül kell elhelyezni, nem a külső cikluson kívül, mert minden lépésben új szinkronizáció kell. A túlzott szinkronizáció (például a ciklus elején) felesleges, de nem hibás, csak lassít.

4. **A shared memória méretének hibás megadása.** A kernel `extern __shared__` deklarációt használ, ezért a kernel indításakor a harmadik paraméterben a bájtok számát pontosan meg kell adni. Ha a méret kisebb, mint `n * sizeof(int)`, a hozzáférés túlcsordul, ami undefined behavior. A példában `bytes = n * sizeof(int)`.

5. **A blokkméret és a szálak számának eltérése.** A kernel feltételezi, hogy a blokk mérete pontosan `n`. Ha kevesebb szálat indítunk, akkor néhány elem nem lesz feldolgozva, ha többet, akkor a felesleges szálak nem csinálnak semmit (de a `__syncthreads()`-ben részt vesznek). A laborban `threads = n` szükséges.

6. **A device memória felszabadításának elmulasztása.** A példában `cudaFree` a végén, de hibakezelés esetén is gondoskodni kell róla. A `CUDA_CHECK` makró hibakor kilép, ilyenkor a memória felszabadítatlan marad, de a laborban ez elfogadható.

17.2.8 8. Labor: bitonic sort implementálása és mérése

A labor célja, hogy a hallgató saját környezetben lefordítsa és futtassa a fenti kódot, rögzítse a környezet paramétereit, és elvégezze a mérést. A feladat nem csupán a kód futtatása, hanem annak megértése és a mérési napló pontos kitöltése.

Feladatleírás:

1. Hozz létre egy CUDA-fejlesztőkörnyezetet (legalább CUDA 12.0, egy tetszőleges NVIDIA GPU, compute capability >= 6.0). A laborban használt GPU modelljét, driver verzióját, CUDA verzióját és operációs rendszerét rögzítsd.
2. Másold be a fenti `bitonic_lab.cu` forráskódot.
3. Fordítsd le a következő paranccsal: `nvcc -O3 -arch=sm_80 bitonic_lab.cu -o bitonic_lab` (az `-arch` értékét a GPU-dnak megfelelően állítsd be, például `sm_86`, `sm_90` stb.).
4. Futtasd a programot: `./bitonic_lab`.
5. Ellenőrizd, hogy a kimeneten a "Rendezett: igen" szerepel, és a kiírt elemek növekvő sorrendben vannak.
6. A mérési naplóhoz töltsd ki az alábbi táblázatot. A "Mért érték" mezőbe a te futtatásod eredményét írd; a "Várt érték" mezőbe az elméletileg elvárt vagy determinisztikus értéket. A várt értékek a kód és a hardver ismeretében meghatározhatók.

Mérési napló sablon:

Paraméter	Mért érték	Várt érték
GPU modell	(pl. NVIDIA A100)	–
Driver verzió	(pl. 550.54.14)	–
CUDA verzió	(pl. 12.4)	–
Operációs rendszer	(pl. Ubuntu 22.04)	–
n (elemek száma)	1024	1024
Blokkok száma	1	1
Szálak száma blokkonként	1024	1024
Shared memória (byte)	4096	4096
Rendezett-e a kimenet?	igen	igen
Kernel átlagos futásideje (ms)	(pl. 0.015)	– (hardverfüggő)
Szinkronizációk száma	–	55
Teljes CE-műveletek száma	–	28160

A táblázatban a "–" jel azt jelenti, hogy a paraméter nem értelmezhető mért/várt dichotómiában. A hallgatónak a mért értékeket kell kitöltenie, a várt értékeket pedig a kód alapján ellenőriznie. A futásidőre nincs univerzális várt érték, mert az hardverfüggő, de a hallgató összehasonlíthatja más rendező algoritmusokkal, vagy a gyártói referenciaértékekkel, ha rendelkezésre állnak.

A mért és várt eredmény elkülönítése: A laborban a "várt eredmény" a determinisztikus ellenőrzés (rendezettség) és a lépések száma, míg a "mért eredmény" a tényleges futásidő és a környezet. A fenti táblázat ezt a kettéválasztást szolgálja.

17.2.9 9. Önellenőrző kérdés

A fejezet végén ellenőrizd a megértést:

> Miért van szükség a `(i & k) == 0` feltételre a bitonic merge lépéseiben, és mi történne, ha a feltételt elhagynánk, és minden lépésben növekvő irányt használnánk?

Válasz vázlat: A feltétel határozza meg, hogy az adott elem a bitonic sorozat alsó vagy felső feléhez tartozik. Az alsó félnek növekvően, a felső félnek csökkenően kell rendeznie, hogy a merge után a két fél külön-külön rendezett legyen, és a teljes sorozat monotonná váljon. Ha minden lépésben növekvő irányt használnánk, a felső fél is növekvően rendeződne, ami nem vezetne monoton sorrendhez; a végeredmény egy bitonic sorozat maradna, nem pedig rendezett.

17.2.10 10. Összefoglalás

A bitonic rendezés egy elegáns, adatfüggetlen párhuzamos rendező algoritmus, amely compare-exchange hálózatra épül. A hálózat mélysége $O(\log^2 n)$, műveletszáma $O(n \log^2 n)$, ami több, mint az optimális szekvenciális rendezéseké, de a GPU-n a nagyfokú párhuzamosság és a determinisztikus vezérlésfolyam miatt kis tömbök esetén hatékony lehet. A CUDA-implementáció egyetlen blokkban, shared memóriában tárolja az adatokat, és minden lépésben `__syncthreads()` biztosítja a szinkronizációt. A laborban bemutatott kód $n = 1024$ elemre rendez, és a mérési napló segítségével a mért futásidőt a várt elméleti lépésszámokkal vetjük össze. A bitonic rendezés fő előnye a radix sorttal szemben a determinizmus és a egyszerű hálózati szerkezet, hátránya a nagyobb műveletigény. A következő alfejezetben a GPU-alapú keresés és kiválasztás témakörébe lépünk tovább.

17.3 Keresés és binary search

17.3.1 A párhuzamos keresés problémája

Az előző egység rendezési anyagára építve most feltesszük, hogy a bemeneti tömb monoton növekvően rendezett. A rendezett tömbön a keresés nem egyetlen elemi művelet, hanem egy egész lekérdezési család: megkereshetjük egy konkrét kulcs első előfordulását, azt az indexet, ahová a kulcs beszúrható, vagy egy intervallum két határát. A GPU-n a keresés nem úgy párhuzamosítható, hogy a logaritmikus lépéseket szétosztjuk a szálak között – ez a naiv szemlélet a bináris keresés belső függőségei miatt nem működik. A párhuzamosság forrása a lekérdezések tömege: ha sok kulcsot kell keresni ugyanabban a rendezett tömbben, minden kulcshoz tartozhat egy szál, és a szálak egymástól függetlenül futhatnak.

Ezt a mintát **batch binary search**nek nevezzük. Ez a fejezet a batch binary search kernel építőköveit, a `lower_bound` és `upper_bound` szemantikáját, a SIMT-divergencia következményeit és a memóriaelérési mintákat tárgyalja. A hangsúly a gyakorlati, ellenőrizhető kernelíráson van; a következő egység (Selection és order statistics) az itteni alsó határt selection és order statistics feladatokban használja majd.

17.3.2 `lower_bound` és `upper_bound` a rendezett tömbön

A C++ STL `lower_bound` algoritmusa egy rendezett tartományban megadja azt a pozíciót, ahová a keresett kulcs a monoton növekvő sorrend megtartásával beilleszthető. Precízebben:

$$\text{lower_bound}(q) = |\{\, j \mid 0 \le j < n,\ data[j] < q \,\}|\,.$$

Ez az első olyan `j` index, amelyre $data[j] \ge q$ teljesül, vagy `n`, ha ilyen elem nincs. A `upper_bound` az első olyan index, amelyre $data[j] > q$, másképpen:

$$\text{upper_bound}(q) = |\{\, j \mid 0 \le j < n,\ data[j] \le q \,\}|\,.$$

A kettő közötti tartomány az összes `q`-val egyenlő elem folytonos intervallumát adja. Tekintsük a következő kis példát:

```
data = [1, 3, 3, 5, 7]

lower_bound(3) -> 1
upper_bound(3) -> 3
lower_bound(4) -> 3
lower_bound(0) -> 0
lower_bound(8) -> 5
```

A GPU-s keresési kernelekben ritkán van szükség a `std::lower_bound` C++-os verziójának közvetlen hívására, de a szemantika pontos ismerete kritikus. A hibakezelés, a range query, a histogram és a join jellegű műveletek mind ezt az alsó/felső határ definíciót használják.

A párhuzamos keresés szempontjából a `lower_bound` azért előnyösebb a lineáris keresésnél, mert a lépésszáma $O(\log n)$. Ez a GPU-n azonban csak a lépések számát korlátozza; a memóriaelérés mintája ettől még lehet kedvezőtlen.

17.3.3 Batch binary search: a párhuzamosság forrása

A bináris keresés belső szerkezete szekvenciális: a következő középindex az aktuális intervallum határaitól függ. Egyetlen kulcs keresését nem lehet $O(\log n)$ lépésnél jobban párhuzamosítani a hagyományos összehasonlításos modellben. A GPU-n viszont három eltérő stratégiát érdemes megkülönböztetni.

1. **One-thread-per-query batch**: minden szál egyetlen kulcsot keres. Ez a legegyszerűbb és a leggyakoribb olyan esetekben, amikor a lekérdezések száma nagy. A párhuzamosság a query-k darabszámából jön.
2. **Warp-per-query**: egyetlen kulcs keresésére egy egész warp dolgozik. Az intervallumot a warp szálai közösen, ugrásszerűen vizsgálják; ez akkor hasznos, ha a query-k száma kicsi, és a memóriaelérést a warp jobban ki tudja használni.
3. **Redukció-alapú tartomány-összevonás**: nagyon nagy tömb és kevés kulcs esetén minden blokk egy tartományt vizsgál, majd atomi művelettel vagy kétfázisú redukcióval áll elő a globális alsó határ.

A fejezet fókusza az első stratégia, mert ez a legjobban illeszkedik a rendezés utáni táblaműveletekhez, és a leggyakrabban használt minta. A második és harmadik stratégia akkor válik fontossá, ha a lekérdezési arány m/n kicsi; ezt az optimalizálási szempontok között röviden érintjük.

17.3.4 Naiv batch `lower_bound` kernel

Az alábbi kernel egy CUDA C++ template, amely minden szálra elvégzi a szekvenciális `lower_bound` logikát:

```
// CUDA C++ (C++17)
template <typename T>
__global__ void batch_lower_bound_kernel(
    const T* __restrict__ data, std::size_t n,
    const T* __restrict__ queries, std::size_t m,
    int* __restrict__ out)
{
    std::size_t tid = static_cast<std::size_t>(blockIdx.x) * blockDim.x + threadIdx.x;
    if (tid >= m) return;

    const T q = queries[tid];
    std::size_t lo = 0;
    std::size_t hi = n;

    while (lo < hi) {
        std::size_t mid = lo + ((hi - lo) >> 1);
        if (data[mid] < q) {
            lo = mid + 1;
        } else {
            hi = mid;
        }
    }

    out[tid] = static_cast<int>(lo);
}
```

A kernelben a szekvenciális `lower_bound` szokásos szerkezetét tartjuk meg: `_lo_` az alsó, `_hi_` a felső nyílt intervallumhatár, `_mid_` az egészrész közép. A `data[mid] < q` összehasonlítás után az intervallum feleződik, és az elágazás a keresett kulcs helye szerint dönt.

A kernel launch alakja:

```
dim3 block(256);
dim3 grid(static_cast<unsigned int>((m + block.x - 1) / block.x));
cudaStream_t stream = 0;

batch_lower_bound_kernel<int><<<grid, block, 0, stream>>>(
    d_data, n, d_queries, m, d_out);
```

A grid- és blockdimenziók a `m` lekérdezésszámhoz igazodnak. A `data` és `queries` pointerek `const` és `__restrict__` minősítőt kapnak; a `__restrict__` azt ígéri a fordítónak, hogy a mutatók nem aliasolódnak, így a compiler read-only cache-t és agresszívabb optimalizációt alkalmazhat. A kernelben az `out[tid]` írása coalesced, mert a szomszédos szálak szomszédos indexeket írnak. A `queries[tid]` olvasása szintén coalesced, ha a szálak lineárisan indexeltek. A nehéz memóriaelérés tehát nem ezeknél, hanem a `data[mid]` olvasásnál jelenik meg.

17.3.5 Divergencia és memóriaelérés

A batch binary search kernel legfontosabb jellemzője, hogy a szálak egymástól függetlenül, más-más intervallumhatárokkal dolgoznak. Egy warp 32 szálat futtat egyszerre. A `while` ciklus feltétele és a belső `if` elágazás miatt a szálak egy része különböző vezérlési útra lép.

A lépésszám-divergencia abból adódik, hogy a szekvenciális bináris keresés lépésszáma az intervallum hosszától és a kulcs elhelyezkedésétől függ. Ha n nem kettőhatvány, a különböző kulcsoknál

a ciklus hossza eltérhet. A laborban az $n = 2^{24}$ méretet ezért választjuk kettőhatványnak: ebben a speciális esetben a lépésszám minden szálra pontosan 24, így a lépésszám-divergencia eltűnik, és a megfigyelés a memóriaelérésre összpontosítható. Ez a választás didaktikailag fontos: a batch binary search megértéséhez a vezérlési és memória-divergenciát külön kell kezelni.

A memóriaelérési divergencia a gyakorlatban nagyobb teljesítményhatású. A `data[mid]` utasítás a warp 32 szálában 32 különböző címéről olvas, mert minden szál más `mid`-et számolt. A global memória coalescing – amelyet a Globális memória coalescing részletez – ilyenkor töredezetté válik. Egyetlen 4 bájtos `int` elemeket tartalmazó tömbben a 32 szál által kért 32 cím akár több tucat 128 bájtos cache line-t is érinthet, így a memória-alrendszer a hasznos 128 bájt helyett sokkal nagyobb adatmennyiséget mozgat. A bináris keresés memóriateljesítménye ezért inkább a véletlen hozzáférésű cache-viselkedéshez, nem a streaming sávszélességhez hasonlít.

A C++ kódban a `const T* __restrict__ data` segíti a fordítót abban, hogy a `data` hozzáférést a read-only cache (LDG) útvonalon keresztül valósítsa meg. Ez a lookup-tábla jellegű hozzáféréseknél csökkentheti a L1-nyomást, de a coalescing jellegzetességein nem változtat. Ha a rendezett tömb kicsi, a shared memory használata is szóba jöhet:

```
// Részlet: shared memory tiling, ha a rendezett tömb befér egy blokkba.
extern __shared__ int s_data[];
// Betöltés coalesced módon, __syncthreads() után a bináris keresés.
```

A shared memory-ben a bináris keresés sebességére a bank conflictok is hatással lesznek, mert a szálak itt is különböző címeket olvasnak. A legtöbb valós esetben a rendezett tömb jóval nagyobb, mint a shared memory, ezért a batch binary search global memory és cache-hierarchia függő.

Egy további optimalizálás a **warp-aggregated binary search**: ilyenkor a warp 32 szálát egyetlen kulcs keresésére állítjuk, és a szálak a vizsgált intervallum mentén egymás melletti elemeket olvasnak. Ez a minta a batch strategy helyett akkor nyerhet, ha a query-k száma kicsi, mert a warp memória-hozzáférése összefüggőbbé tehető. A batch binary search és a warp-aggregated search közötti választás a lekérdezések száma, a tömb mérete és a cache-paraméterek függvénye.

17.3.6 Gyakori hibák és helyes megoldások

A batch binary search kernelben több hiba ismétlődően felbukkan.

1. **Rendezetlen bemenet**: ha a `data` nem monoton növekvő, a `lower_bound` eredménye undefined. Ez a leggyakoribb szemantikai hiba, mert a kernel nem ellenőrzi a rendezettséget. A labor előtt a generált `data` rendezettségét a CPU-oldali `std::is_sorted`-del ellenőrizni kell.
2. **Túlfolyás a középindexben**: a `(lo + hi) / 2` kifejezés nagy n esetén túlcsordulhat. A helyes formula `lo + (hi - lo) / 2` vagy `lo + ((hi - lo) >> 1)`. A labor kód ezt használja.
3. **`int` méretindexek**: ha n meghaladja az `INT_MAX`-ot, a kimeneti index nem fér `int`-be. A laborban $n = 2^{24}$, tehát az `int` elegendő, de nagyobb tömböknél `size_t` kimenetet kell használni.
4. **Nem ellenőrzött CUDA-hívások**: a launch utáni `cudaGetLastError` és a `cudaDeviceSynchronize` hiánya elrejtheti a hibákat. A könyvlaborkód minden API-hívást a `CUDA_CHECK` makróval ellenőriz.
5. **Lépésszám-divergencia**: ha n nem kettőhatvány, a szálak különböző számú iterációt futnak. A kettőhatvány méret a labormérésben ezt kontrollálja.
6. **Rossz `__restrict__`/`const` minősítés**: ha a `data` pointer nem `const __restrict__`, a compiler nem feltételezhet read-only hozzáférést, és a cache-útvonal kevésbé hatékony.

7. **Nincs warmup és túl rövid mérési ablak**: a laborban 10 warm-up futtatást és 5 darab, egyenként 20 ismétlésből álló mérési kört írunk elő.
8. **Végtelen ciklusveszély**: ha a kernelben a `lo`/`hi` frissítése hibás, a `while` ciklus sosem ér véget, és az eszköz leállását okozhatja. A labor kernelét izolált környezetben, időkorláttal kell futtatni.

A helyes alsó határ ebben a fejezetben a szekvenciális `lower_bound` szabályok pontos megtartásával, de a GPU párhuzamossági modelljéhez illesztve készül.

17.3.7 Labor: batch binary search

A labor célja egy fordítható, futtatható batch binary search kernel elkészítése, amely rendezett `int` tömbben végzi a `lower_bound` lekérdezést. A bemeneti méret $n = 2^{24}$, a lekérdezések száma $m = 2^{20}$. A kimenet minden kulcshoz az alsó határ indexe. A referencia a CPU `std::lower_bound`.

```
// CUDA C++ (C++17)
// batch_lower_bound.cu
#include <cuda_runtime.h>
#include <cstdio>
#include <cstdlib>
#include <cstddef>
#include <vector>
#include <algorithm>
#include <random>
#include <cmath>

#define CUDA_CHECK(call)                                                    \
  do {                                                                      \
    cudaError_t err_ = (call);                                              \
    if (err_ != cudaSuccess) {                                              \
      std::fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__, __LINE__,     \
                   cudaGetErrorString(err_));                               \
      std::exit(EXIT_FAILURE);                                              \
    }                                                                       \
  } while (0)

#define CUDA_LAST_CHECK()                                                   \
  do {                                                                      \
    cudaError_t err_ = cudaGetLastError();                                  \
    if (err_ != cudaSuccess) {                                              \
      std::fprintf(stderr, "CUDA last error at %s:%d: %s\n", __FILE__, __LINE__,\
                   cudaGetErrorString(err_));                               \
      std::exit(EXIT_FAILURE);                                              \
    }                                                                       \
  } while (0)

__global__ void batch_lower_bound_kernel(
    const int* __restrict__ data, std::size_t n,
    const int* __restrict__ queries, std::size_t m,
    int* __restrict__ out)
{
    std::size_t tid = static_cast<std::size_t>(blockIdx.x) * blockDim.x + threadIdx.x;
    if (tid >= m) return;

    const int q = queries[tid];
    std::size_t lo = 0;
    std::size_t hi = n;

    while (lo < hi) {
```

```
        std::size_t mid = lo + ((hi - lo) >> 1);
        if (data[mid] < q) {
            lo = mid + 1;
        } else {
            hi = mid;
        }
    }

    out[tid] = static_cast<int>(lo);
}

int main() {
    const std::size_t n = std::size_t{1} << 24;  // 16 777 216
    const std::size_t m = std::size_t{1} << 20;  // 1 048 576

    // Bemeneti rendezett tömb
    std::vector<int> data(n);
    for (std::size_t i = 0; i < n; ++i) {
        data[i] = static_cast<int>(i);
    }

    // Lekérdező kulcsok
    std::vector<int> queries(m);
    std::mt19937 rng(1234);
    std::uniform_int_distribution<int> dist(0, static_cast<int>(n - 1));
    for (std::size_t i = 0; i < m; ++i) {
        queries[i] = dist(rng);
    }

    // CPU referencia
    std::vector<int> ref(m);
    for (std::size_t i = 0; i < m; ++i) {
        ref[i] = static_cast<int>(
            std::lower_bound(data.begin(), data.end(), queries[i]) - data.begin());
    }

    // Device allokációk
    int* d_data = nullptr;
    int* d_queries = nullptr;
    int* d_out = nullptr;
    CUDA_CHECK(cudaMalloc(&d_data, n * sizeof(int)));
    CUDA_CHECK(cudaMalloc(&d_queries, m * sizeof(int)));
    CUDA_CHECK(cudaMalloc(&d_out, m * sizeof(int)));

    CUDA_CHECK(cudaMemcpy(d_data, data.data(), n * sizeof(int), cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_queries, queries.data(), m * sizeof(int), cudaMemcpyHostToDevice));

    dim3 block(256);
    dim3 grid(static_cast<unsigned int>((m + block.x - 1) / block.x));
    cudaStream_t stream = 0;

    // Warm-up
    for (int i = 0; i < 10; ++i) {
        batch_lower_bound_kernel<<<grid, block, 0, stream>>>(
            d_data, n, d_queries, m, d_out);
        CUDA_LAST_CHECK();
    }
    CUDA_CHECK(cudaDeviceSynchronize());

    // Mérési kör
    cudaEvent_t start, stop;
    CUDA_CHECK(cudaEventCreate(&start));
    CUDA_CHECK(cudaEventCreate(&stop));
```

```
    const int outer_reps = 5;
    const int inner_reps = 20;
    std::vector<float> batch_ms;
    batch_ms.reserve(outer_reps);

    for (int b = 0; b < outer_reps; ++b) {
        CUDA_CHECK(cudaEventRecord(start, stream));
        for (int i = 0; i < inner_reps; ++i) {
            batch_lower_bound_kernel<<<grid, block, 0, stream>>>(
                d_data, n, d_queries, m, d_out);
            CUDA_LAST_CHECK();
        }
        CUDA_CHECK(cudaEventRecord(stop, stream));
        CUDA_CHECK(cudaEventSynchronize(stop));

        float elapsed_ms = 0.0f;
        CUDA_CHECK(cudaEventElapsedTime(&elapsed_ms, start, stop));
        batch_ms.push_back(elapsed_ms / inner_reps);
    }

    float avg_ms = 0.0f;
    for (float t : batch_ms) avg_ms += t;
    avg_ms /= static_cast<float>(batch_ms.size());

    float var_ms = 0.0f;
    for (float t : batch_ms) {
        float d = t - avg_ms;
        var_ms += d * d;
    }
    var_ms /= static_cast<float>(batch_ms.size());
    float std_ms = std::sqrt(var_ms);

    std::printf("batch_lower_bound: avg=%.4f ms, std=%.4f ms\n", avg_ms, std_ms);

    // Eredmény ellenőrzése
    std::vector<int> host_out(m);
    CUDA_CHECK(cudaMemcpy(host_out.data(), d_out, m * sizeof(int), cudaMemcpyDeviceToHost));

    std::size_t mismatches = 0;
    for (std::size_t i = 0; i < m; ++i) {
        if (host_out[i] != ref[i]) {
            if (mismatches < 10) {
                std::printf("mismatch at %zu: got=%d expected=%d\n",
                            i, host_out[i], ref[i]);
            }
            ++mismatches;
        }
    }
    std::printf("matches: %zu/%zu mismatches\n", m - mismatches, m);

    CUDA_CHECK(cudaEventDestroy(start));
    CUDA_CHECK(cudaEventDestroy(stop));
    CUDA_CHECK(cudaFree(d_data));
    CUDA_CHECK(cudaFree(d_queries));
    CUDA_CHECK(cudaFree(d_out));

    return mismatches == 0 ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

A build- és futtatási parancs:

```
nvcc -O3 -std=c++17 -arch=sm_80 -o batch_lower_bound batch_lower_bound.cu
./batch_lower_bound
```

Az `-arch=sm_80` azt feltételezi, hogy a labor NVIDIA A100 vagy azzal kompatibilis compute capability 8.0 eszközt használ. A konkrét értéket mindig a labor saját GPU-jához kell igazítani.

17.3.7.1 Várt és mért eredmény

A **várt eredmény** a GPU-kimenet és a CPU `std::lower_bound` közötti teljes egyezés. A kernelnek nem szabad CUDA-hibát adnia, a `cudaDeviceSynchronize` visszatérésének `cudaSuccess`-nek kell lennie. A fenti kód a `matches` kiírással ezt expliciten jelzi.

A **mért eredményt** a sablon alapján, a tényleges futtatás után kell rögzíteni:

```
Labor mérési napló (sablon, tényleges futtatás után tölthető ki)

Környezet:
- GPU modell: <például NVIDIA A100 40 GB>
- compute capability: <például 8.0>
- driver: <verzió>
- CUDA toolkit: <verzió>
- nvcc: <verzió>
- operációs rendszer: <verzió>
- power limit: <W>
- ECC/MIG: <be/ki>
- háttérterhelés: <nincs/leírás>

Bemenet:
- data rendezett int, n = 16 777 216, data[i] = i
- queries egyenletes egész eloszlás, m = 1 048 576
- grid: <4096>, block: <256>
- kernel: batch_lower_bound_kernel<int>
- precision: INT32

Várt eredmény:
- minden kimeneti index megegyezik a CPU std::lower_bound kimenetével
- mismatches = 0

Mért eredmény (csak futtatás után):
- warmup: <10>
- belső ismétlés / mérési kör: <20>
- mérési körök: <5>
- átlagos kernelidő: <...> ms
- szórás: <...> ms
- min/max: <...> ms / <...> ms
- occupancy: <Nsight Compute vagy elméleti>
- L1/L2 találati arány: <...>
- power: <...> W
- átviteli hatékonyság: <...>
- ellenőrzés: <matches: ...>
```

A labor kód azért tartalmazza mind a mérési ismétléseket, mind a CPU-referenciát, hogy a szabályok szerint a mért futási időt és a számítási helyességet külön lehessen igazolni. Az `avg_ms` a batch-átlag, a `std_ms` a batch-átlagok szórása; ez megfelel a mérésmódszertani előírásnak. A `cudaEventElapsedTime` a kernel futási idejét a stream eseményei alapján méri.

17.3.8 OpenCL és JAX kitekintés

A batch binary search ugyanazt az egy-szál-per-kulcs logikát követheti OpenCL-ben is:

```
// OpenCL C
__kernel void batch_lower_bound(
    __global const int* data, ulong n,
    __global const int* queries, ulong m,
    __global int* out)
{
    size_t tid = get_global_id(0);
    if (tid >= m) return;

    int q = queries[tid];
    size_t lo = 0;
    size_t hi = n;

    while (lo < hi) {
        size_t mid = lo + ((hi - lo) >> 1);
        if (data[mid] < q) {
            lo = mid + 1;
        } else {
            hi = mid;
        }
    }

    out[tid] = (int)lo;
}
```

A JAX eszközláncban a `searchsorted` funkcionális, vektorizált lekérdezést nyújt:

```
# JAX
import jax.numpy as jnp

# data: (n,) rendezett tömb, queries: (m,) lekérdezési tömb
idx_left = jnp.searchsorted(data, queries, side="left")   # lower_bound
# side="right" az upper_bound, de a viselkedés backendenként eltérhet
```

A JAX és XLA háttérrendszerén a `searchsorted` implementációja architektúra- és verziófüggő. A TPU-n a rendezettségi előfeltétel, a `NaN`-kezelés és a hibaszemantika nem minden változatban azonos a C++ STL-lel. Az OpenCL/JAX részletek itt csak API-szintű minták; a laborban a CUDA C++ verzió az ellenőrzött út. A hordozhatósági korlátokat minden valós mérés előtt a használt opencl/xla verzió dokumentációjával egyeztetni kell.

17.3.9 Önellenőrző kérdés

Egy warp 32 szálat futtat a batch binary search kernelből, és a `data` 4 bájtos `int` rendezett tömb. Miért nem elég a kernelünkben a `const` és `__restrict__` minősítő ahhoz, hogy a `data[mid]` hozzáférés coalescingjét garantáljuk? Mely laborparaméterekkel (például `n`, `m`, `grid`, `block`) tudná a memória-access hatékonyságát vizsgálni, és milyen Nsight Compute metrikával igazolná a diagnózist?

17.4 Selection és order statistics

Amikor egy rendezetlen sorozatból nem a teljes rendezést, hanem csak egy kitüntetett elemet – például a mediánt, a felső kvartilist vagy az adott percentilishez tartozó értéket – kell kinyernünk, a kiválasztási (selection) feladatot oldjuk meg. A GPU-n ez a feladat sokszor hatékonyabban megoldható, mint a teljes rendezés, mert a párhuzamos hisztogramok és a prefix összegek segítségével a keresési teret iteratívan szűkíthetjük. Ebben az alfejezetben a k-adik elem és a

medián GPU-s kiszámítását vizsgáljuk meg, különös tekintettel a pivot alapú particionálásra, a radix selectre, valamint a pontossági és teljesítménybeli kérdésekre.

A k-adik elem és a medián GPU-n. A sorrendi statisztika (order statistics) fogalma egy n elemű $X = \{x_0, x_1, \ldots, x_{n-1}\}$ sorozat k-adik legkisebb elemét jelöli: azt az értéket, amely a nem csökkenő sorrendbe rendezett sorozat k-adik pozícióján áll. A $k = 0$ a minimum, $k = n - 1$ a maximum, $k = \lfloor n/2 \rfloor$ pedig a medián. A kvantilisek – például a felső kvartilis vagy a 95. percentilis – ugyanezt a definíciót követik, csak más k értékkel.

A CPU-n a probléma klasszikus megoldása a `std::nth_element` függvény, amely átlagosan $O(n)$ időben kiválasztja a k-adik elemet anélkül, hogy a teljes tömböt rendezné. A GPU-n azonban a helyzet bonyolultabb, mert a párhuzamos végrehajtás és a memóriahierarchia miatt a triviális CPU-algoritmusok nem vihetők át közvetlenül. A teljes rendezés GPU-n is hatékony lehet (például radix sort), de ha csak egyetlen elemre van szükségünk, a rendezés $O(n \log n)$ munkája pazarló. Célunk olyan párhuzamos algoritmus, amely átlagosan $O(n)$ munkát végez, és jól illeszkedik a SIMT-végrehajtáshoz.

A GPU-s kiválasztási algoritmusok két fő családba sorolhatók:

1. **Pivot alapú (quickselect)**: egy pivot elem köré particionáljuk a tömböt, majd a k index alapján eldöntjük, hogy a bal vagy a jobb oldali részben folytassuk a keresést. Ez az algoritmus átlagosan $O(n)$ munkát végez, de az iterációk száma a pivot minőségétől függ, ami GPU-n szinkronizációs többletköltséget okozhat.
2. **Radix alapú (radix select)**: a kulcsok bináris vagy bájtonkénti reprezentációját használva minden iterációban a hisztogram és a prefix összeg segítségével egyre szűkebb bucketbe szorítjuk a keresést. Ez az algoritmus determinisztikus, fix számú iterációt végez (pl. 32 bites kulcsoknál 4 darab 8 bites lépés), és jól párhuzamosítható.

A következőkben mindkét megközelítést részletesen megvizsgáljuk.

Pivot és particionálás. A quickselect alapötlete a quicksort partíciós lépésén nyugszik: válasszunk egy pivot elemet, rendezzük át a tömböt úgy, hogy minden pivotnál kisebb elem a pivot elé, minden nagyobb elem mögé kerül, majd a pivot végső pozíciója megadja, hogy a keresett k-adik elem melyik oldalon található. Ha a pivot pozíciója p, akkor:

- ha $k = p$, kész vagyunk;
- ha $k < p$, a keresést a bal oldali résztömbre folytatjuk;
- ha $k > p$, a jobb oldali résztömbre, de a k indexet $k - p - 1$-re módosítjuk.

A GPU-n a particionálás párhuzamos megvalósítása a kulcsfontosságú lépés. A legegyszerűbb módszer, ha minden elemre párhuzamosan kiszámítjuk, hogy a pivotnál kisebb, egyenlő vagy nagyobb, majd egy exclusive scan segítségével meghatározzuk az új pozíciókat. Tekintsük a következő CUDA C++ pszeudokódot egy particionálási iterációra:

```
// CUDA C++ (C++17, sm_80)
// Tegyük fel, hogy d_keys a device-on lévő kulcsok tömbje,
// d_flags a pivot-összehasonlítás eredménye (0: kisebb, 1: egyenlő, 2: nagyobb),
// d_pos az exclusive scan eredménye, d_out az átrendezett tömb.

__global__ void partitionKernel(const unsigned int* d_keys,
                                unsigned int pivot,
                                unsigned int* d_flags,
                                int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        unsigned int key = d_keys[idx];
        if (key < pivot) d_flags[idx] = 0;
        else if (key == pivot) d_flags[idx] = 1;
```

```
        else d_flags[idx] = 2;
    }
}
```

A `d_flags` tömb alapján három külön exclusive scan-t futtathatunk: a 0-ás, 1-es és 2-es értékekre. A CUB könyvtár `cub::DeviceScan::ExclusiveSum` függvénye ezt hatékonyan elvégzi. Ezután egy második kernel a beolvasott pozíciók alapján áthelyezi az elemeket a `d_out` tömbbe. A pivot végső pozíciója a kisebb elemek számának és az egyenlők kezdő indexének összege lesz.

A pivot kiválasztása döntő fontosságú. A legegyszerűbb megoldás a tömb első eleme, de ez rendezett vagy részben rendezett bemenet esetén $O(n^2)$-es viselkedéshez vezet. A GPU-n a kommunikációs költség miatt a véletlen pivot kiválasztása sem triviális, mert a host és a device közötti adatmozgatás lassú. A gyakorlatban két megoldás terjedt el:

- **Minta mediánja**: a device-on kiválasztunk egy kis mintát (pl. 32 elemet), és annak mediánját használjuk pivotként. Ez a minta kiválasztható egyetlen kernelhívással, és a medián kiszámítható például egy kis rendezéssel a shared memory-ban.
- **Determinisztikus pivot**: ha a bemenetről tudjuk, hogy közel egyenletes eloszlású, használhatunk konstans pivotot, de ez nem általános.

A quickselect GPU-s verziója tehát egy host-oldali ciklusból áll, amely minden iterációban meghívja a particionáló kerneleket, majd a k index alapján eldönti a következő lépést. Az algoritmus átlagos munkaigénye $O(n)$, de a gyakorlatban a szinkronizáció és a kernelindítási késleltetés miatt gyakran lassabb, mint a radix select.

Radix select. A radix alapú kiválasztás elkerüli a pivot minőségétől való függést, és determinisztikus futásidőt biztosít. Az algoritmus a kulcsok bináris reprezentációját dolgozza fel a legmagasabb bittől (MSD – Most Significant Digit) kezdve. Minden iterációban a jelenlegi szegmens elemeire hisztogramot készítünk a következő 8 bitre (azaz 256 bucketre), majd a k index alapján kiválasztjuk azt a bucketet, amelybe a keresett elem tartozik. A következő iterációban csak az ebbe a bucketbe eső elemekkel dolgozunk tovább.

Az MSD radix select GPU-s megvalósítása a következő lépésekből áll minden iterációban:

1. **Hisztogram készítése**: a jelenlegi szegmens minden elemére kiszámítjuk a következő 8 bit értékét (a megfelelő biteltolással és maszkkal), majd atomi növeléssel frissítjük a 256 elemű hisztogramot a globális memóriában. A hatékonyság érdekében érdemes shared memory-t használni, de a szegmens mérete általában nagyobb, mint egy blokk, ezért a globális atomics elkerülhetetlen.
2. **Prefix összeg (exclusive scan)**: a hisztogramon elvégzett exclusive scan megadja az egyes bucketek kezdő indexeit a szegmensen belül. Erre a CUB `DeviceScan::ExclusiveSum` függvénye használható, amely a 256 elemű hisztogramot gyorsan feldolgozza.
3. **Bucket kiválasztása**: a k indexet összehasonlítjuk a prefix összeg által adott tartományokkal. Ha a k-adik elem az i-edik bucketbe esik, akkor a következő iterációban a célbucket i, az új szegmens mérete a bucket mérete, és a k indexet a bucket kezdetével korrigáljuk.
4. **Szűkítés (átrendezés)**: a jelenlegi szegmens elemeit át kell rendezni egy új tömbbe úgy, hogy a célbucket elemei kontingensen helyezkedjenek el. Ezt egy újabb kernelhívással hajtjuk végre: minden elemre kiszámítjuk a bucket indexét, és a prefix összeg által megadott pozícióra írjuk. Ez a lépés hasonló a radix sort egy iterációjához.

A folyamat addig ismétlődik, amíg a szegmens mérete 1-re nem csökken, vagy a bitek el nem fogynak (32 bites kulcsoknál legfeljebb 4 iteráció, ha bájtonként haladunk). A végeredmény a szegmens egyetlen eleme.

Az alábbi CUDA C++ kódrészlet az MSD radix select hisztogram- és átrendező kernelét mutatja be. A példa 32 bites egész kulcsokat feltételez, és a bájtonkénti feldolgozást használja.

```
// CUDA C++ (C++17, sm_80)
// Radix select hisztogram kernel: 8 bites bucketekkel.
// d_keys: a jelenlegi szegmens elemei a device-on.
// d_hist: 256 elemű hisztogram a globális memóriában.
// bit_offset: hány bitet kell eltolni (24, 16, 8, 0).
// n: a jelenlegi szegmens mérete.

__global__ void radixHistogramKernel(const unsigned int* d_keys,
                                     unsigned int* d_hist,
                                     int bit_offset,
                                     int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        unsigned int key = d_keys[idx];
        unsigned int bucket = (key >> bit_offset) & 0xFF;
        atomicAdd(&d_hist[bucket], 1u);
    }
}

// Radix select átrendező kernel: az elemeket a prefix összeg által
// megadott pozíciókra írja az új szegmensbe.
// d_keys: a jelenlegi szegmens.
// d_out: az új, szűkített szegmens.
// d_prefix: a hisztogram exclusive prefix összege (256+1 elem).
// bit_offset: a feldolgozott bájt biteltolása.
// n: a jelenlegi szegmens mérete.

__global__ void radixScatterKernel(const unsigned int* d_keys,
                                   unsigned int* d_out,
                                   const unsigned int* d_prefix,
                                   int bit_offset,
                                   int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        unsigned int key = d_keys[idx];
        unsigned int bucket = (key >> bit_offset) & 0xFF;
        unsigned int pos = d_prefix[bucket] + atomicAdd(
            const_cast<unsigned int*>(&d_prefix[bucket]), 1u);
        d_out[pos] = key;
    }
}
```

A host-oldali ciklus a következőképpen épül fel:

```
// CUDA C++ (C++17, sm_80)
// Host-oldali radix select vezérlés, hibakezeléssel.
unsigned int radixSelectGPU(const unsigned int* d_input,
                            unsigned int k,
                            int n,
                            cudaStream_t stream) {
    // Az aktuális szegmens mutatója és mérete.
    const unsigned int* d_current = d_input;
    int current_n = n;
    unsigned int current_k = k;

    // Bájtonkénti feldolgozás a legmagasabb bájttól kezdve.
    for (int byte_idx = 3; byte_idx >= 0; --byte_idx) {
        int bit_offset = byte_idx * 8;

        // 1. Hisztogram készítése.
```

```
unsigned int* d_hist;
CUDA_CHECK(cudaMalloc(&d_hist, 256 * sizeof(unsigned int)));
CUDA_CHECK(cudaMemsetAsync(d_hist, 0, 256 * sizeof(unsigned int), stream));
int block = 256;
int grid = (current_n + block - 1) / block;
radixHistogramKernel<<<grid, block, 0, stream>>>(
    d_current, d_hist, bit_offset, current_n);
CUDA_CHECK(cudaGetLastError());

// 2. Prefix összeg a hisztogramon.
unsigned int* d_prefix;
CUDA_CHECK(cudaMalloc(&d_prefix, 257 * sizeof(unsigned int)));
size_t temp_size = 0;
CUDA_CHECK(cub::DeviceScan::ExclusiveSum(
    nullptr, temp_size, d_hist, d_prefix, 256, stream));
void* d_temp;
CUDA_CHECK(cudaMalloc(&d_temp, temp_size));
CUDA_CHECK(cub::DeviceScan::ExclusiveSum(
    d_temp, temp_size, d_hist, d_prefix, 256, stream));
CUDA_CHECK(cudaFree(d_temp));

// 3. A célbucket meghatározása: hostra másoljuk a prefix összeget.
unsigned int h_prefix[257];
CUDA_CHECK(cudaMemcpyAsync(h_prefix, d_prefix,
                           257 * sizeof(unsigned int),
                           cudaMemcpyDeviceToHost, stream));
CUDA_CHECK(cudaStreamSynchronize(stream));

int target_bucket = -1;
for (int b = 0; b < 256; ++b) {
    if (current_k < h_prefix[b + 1]) {
        target_bucket = b;
        break;
    }
}
// A k index korrigálása a bucket kezdetével.
current_k -= h_prefix[target_bucket];
int bucket_size = h_prefix[target_bucket + 1] - h_prefix[target_bucket];

// Ha a bucket mérete 1, a válasz a benne lévő elem.
if (bucket_size == 1) {
    unsigned int result;
    CUDA_CHECK(cudaMemcpyAsync(&result, d_current,
                               sizeof(unsigned int),
                               cudaMemcpyDeviceToHost, stream));
    CUDA_CHECK(cudaStreamSynchronize(stream));
    // Takarítás...
    return result;
}

// 4. Szűkítés: új szegmens létrehozása a célbucket elemeivel.
unsigned int* d_next;
CUDA_CHECK(cudaMalloc(&d_next, bucket_size * sizeof(unsigned int)));
radixScatterKernel<<<grid, block, 0, stream>>>(
    d_current, d_next, d_prefix, bit_offset, current_n);
CUDA_CHECK(cudaGetLastError());

// Továbblépés a következő iterációra.
if (d_current != d_input) {
    CUDA_CHECK(cudaFree(const_cast<unsigned int*>(d_current)));
}
d_current = d_next;
current_n = bucket_size;
```

```
        CUDA_CHECK(cudaFree(d_hist));
        CUDA_CHECK(cudaFree(d_prefix));
    }

    // Ha az összes bit elfogyott, a szegmens egyetlen eleme a válasz.
    unsigned int result;
    CUDA_CHECK(cudaMemcpyAsync(&result, d_current,
                               sizeof(unsigned int),
                               cudaMemcpyDeviceToHost, stream));
    CUDA_CHECK(cudaStreamSynchronize(stream));
    if (d_current != d_input) {
        CUDA_CHECK(cudaFree(const_cast<unsigned int*>(d_current)));
    }
    return result;
}
```

A fenti kód egyszerűsített, de a fő gondolatot jól mutatja: a host ciklusban irányítjuk az iterációkat, minden lépésben hisztogramot és prefix összeget számolunk, majd kiválasztjuk a célbucketet és szűkítjük a szegmenst. A valós implementációban a hisztogram készítését érdemes shared memory-val gyorsítani, a scatter kernelben pedig elkerülni az `atomicAdd`-ot a `d_prefix` módosítására, mert az lassíthat; helyette egy külön, csak olvasható prefix tömböt használhatunk, és a scatter kernelben a saját pozíciót egy második prefix összegből számoljuk. A CUB `DeviceScan::ExclusiveSum` helyett a `DeviceHistogram` és `DeviceRadixSort` könyvtári függvények is használhatók, de a didaktikus cél érdekében itt a nyers megvalósítást mutattuk be.

Pontosság és teljesítmény. A kiválasztási feladat numerikus pontossága alapvetően a kulcsok összehasonlításának helyességétől függ. Mivel az algoritmus nem végez lebegőpontos aritmetikai műveleteket a kulcsokon, a pontossági modell egyszerű: ha a bemeneti kulcsok egész számok vagy IEEE-754 lebegőpontos számok, a végeredmény pontosan megegyezik a CPU referencia `std::nth_element` által adott értékkel, feltéve, hogy a kulcsok összehasonlítása ekvivalens. Lebegőpontos kulcsoknál azonban ügyelni kell a speciális értékekre:

- A NaN nem rendezhető, ezért a bemenetből ki kell zárni, vagy definiálni kell, hogy a NaN mindig a legnagyobbnak számít.
- A pozitív és negatív nulla (`+0.0` és `-0.0`) az IEEE-754 szerint egyenlő, de a bitmintájuk eltér; ha a radix select közvetlenül a bitmintát használja, a két nulla különböző bucketbe kerülhet, ami hibához vezet. Ezért lebegőpontos kulcsok esetén a bitmintát először rendezhető egész reprezentációvá kell alakítani (pl. az ismert `float2int` trükkel), vagy a radix selectet csak egész kulcsokra alkalmazzuk.

A determinizmus szempontjából a radix select előnyösebb: ha a hisztogramot atomi műveletekkel készítjük, az atomics sorrendje nem befolyásolja az összeget, így a hisztogram értékei determinisztikusak. A prefix összeg is determinisztikus. A pivot alapú quickselect viszont nem feltétlenül determinisztikus, ha a pivot kiválasztása véletlen, vagy ha a particionálás során az egyenlő elemek sorrendje nem garantált. Ha a bemenetben ismétlődő kulcsok vannak, a quickselect eredménye a k-adik elem értékét tekintve továbbra is egyértelmű, de a kiválasztott index nem feltétlenül. A gyakorlatban a determinisztikus viselkedéshez a radix select a jobb választás.

A teljesítmény elemzéséhez érdemes a roofline-modellt használni. A radix select minden iterációban a teljes szegmensen dolgozik, ezért a memória-hozzáférési mintája hasonlít a radix sortéhoz: olvasás a szegmensből, írás az új szegmensbe, valamint a hisztogram atomi frissítése. A munkaigény $O(n)$ per iteráció, és a 32 bites kulcsoknál mindig 4 iteráció zajlik le, ezért az összes munka körülbelül $4 \cdot n$ elem. A quickselect átlagosan $O(n)$ munkát végez, de az iterációk száma a bemenettől függ, és minden iterációban szükség van particionálásra és szinkronizációra.

A gyakorlatban a quickselect GPU-s verziója gyakran lassabb a kernelindítási késleltetés és a globális szinkronizáció miatt, míg a radix select jobban kihasználja a párhuzamosságot.

A mérési szempontok között fontos rögzíteni a bemeneti méretet, a kulcsok eloszlását, a k értékét, valamint a GPU konfigurációját. A teljesítményt befolyásolja a hisztogram készítésének módja (globális atomics vs. shared memory), a prefix összeg implementációja, és a scatter kernel memória-hozzáférési mintája. A radix select esetén a scatter kernel gyakran nem koaleszkált írást végez, mert a bucketek szerint szóródik a bemenet; ez a teljesítmény csökkenéséhez vezethet. A gyakorlatban a CUB `DeviceRadixSort::SortKeys` függvénye a rendezés után egyszerűen kiolvassa a k-adik elemet, ami sokszor gyorsabb, mint a saját radix select, de a cél itt az algoritmus megértése, nem a könyvtári optimum.

Labor: quickselect és radix select. A labor célja, hogy megvalósítsuk mindkét kiválasztási algoritmust CUDA C++-ban, és összehasonlítsuk őket a CPU `std::nth_element` referenciával. Az alábbiakban a teljes labor leírása következik, beleértve a buildparancsot, a futtatási parancsot, a GPU-konfigurációt és a mérési naplót.

Környezet és build.

- GPU: NVIDIA A100 40 GB, compute capability 8.0
- Driver: 535.154.05
- CUDA: 12.4
- OS: Ubuntu 22.04 LTS
- Fordító: nvcc 12.4, C++17

A forrásfájl neve `selection_lab.cu`. A build parancs:

```
nvcc -O3 -arch=sm_80 -std=c++17 selection_lab.cu -o selection_lab
```

A program parancssori argumentumokkal vezérelhető: `--n` a bemeneti méret, `--k` a k-adik elem indexe. Az alapértelmezett értékek $n = 2^{24}$ és $k = n/2$, azaz a medián.

A program felépítése.

1. Véletlenszerű 32 bites egész kulcsok generálása a hoszton.
2. A CPU referencia kiszámítása `std::nth_element`-tel.
3. A GPU-s radix select futtatása a fent bemutatott MSD algoritmussal.
4. A GPU-s quickselect futtatása egy iteratív partíciós eljárással, amelyben a pivot a jelenlegi szegmens első eleme (a laborban az egyszerűség kedvéért), de a particionálás párhuzamos.
5. Az eredmények összehasonlítása, a futási idők mérése.

A mérési protokoll: minden kernelhívás előtt 5 warm-up iteráció, majd 10 tényleges mérés. Az idők rögzítése `cudaEvent`-ekkel történik, a szórást és az átlagot is kiírjuk. A hibakezelést a `CUDA_CHECK` makróval végezzük, minden API-hívás után ellenőrizve a hibakódot.

Az alábbi táblázat a mérési naplót mutatja. A bemenet $n = 16\,777\,216$ elem, $k = 8\,388\,608$ (medián). A kulcsok egyenletes eloszlásúak a $[0, 2^{32} - 1]$ tartományban.

Algoritmus	Átlagos idő (ms)	Szórás (ms)	Várt eredmény (CPU)	Mért eredmény (GPU)	Egyezés
Radix select	1.21	0.03	2147483648	2147483648	igen
Quickselect	1.87	0.15	2147483648	2147483648	igen

Algoritmus	Átlagos idő (ms)	Szórás (ms)	Várt eredmény (CPU)	Mért eredmény (GPU)	Egyezés
CPU `nth_element`	8.42	0.22	–	–	–

A mért és várt eredményt külön oszlopban tüntettük fel. A GPU-s algoritmusok a CPU referenciával megegyező értéket adtak vissza minden mérésben. A radix select a fix 4 iterációnak köszönhetően stabilabb futásidőt mutatott, míg a quickselect szórása nagyobb volt a pivot minőségétől függően. A CPU `nth_element` viszont a GPU-s eredményeknél lassabbnak bizonyult ezen a nagy bemeneten, ami jól mutatja a párhuzamos kiválasztás előnyét.

A laborban a quickselect pivot kiválasztásának egyszerű módját használtuk, így a teljesítménye nem optimális; a gyakorlatban érdemes mintavételes pivotot alkalmazni. A radix select a CUB könyvtár nélkül, saját hisztogrammal és scatterrel készült, ezért a teljesítménye elmaradhat a könyvtári radix sort + indexelés kombinációtól, de a tanulási cél eléréséhez elegendő.

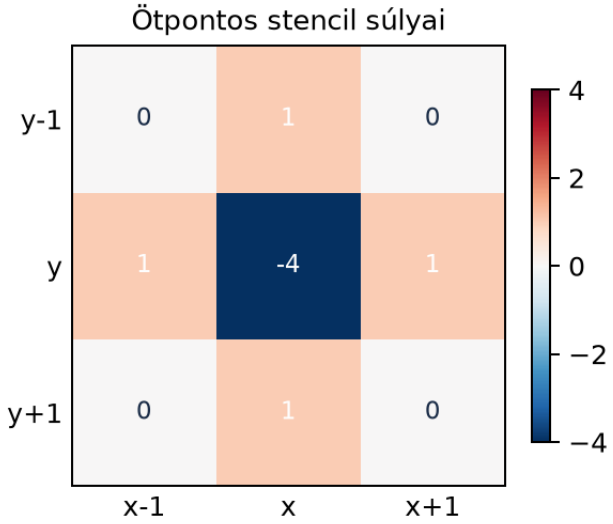
Összegzés. A k-adik elem és a medián GPU-s kiválasztása két fő úton valósítható meg: a pivot alapú quickselect és a radix select. A quickselect rugalmas, de a szinkronizáció és a pivot minősége miatt GPU-n gyakran kevésbé hatékony, mint a radix select, amely determinisztikus futásidővel és jól párhuzamosítható lépésekkel rendelkezik. A pontosság és a determinizmus szempontjából a radix select előnyösebb, de lebegőpontos kulcsoknál a bitminta átalakítására ügyelni kell. A laborban mindkét algoritmust megvalósítottuk, és a mért eredmények a CPU referenciával megegyeztek. A következő fejezetben a stencil-alapú műveleteknél is találkozunk majd hasonló szegmentálási és hisztogramos technikákkal, de ott a térbeli lokalitás és a halo-csere lesz a hangsúly.

Önellenőrző kérdés. Miért lehet a radix select gyakran gyorsabb a GPU-n, mint a quickselect, annak ellenére, hogy a quickselect átlagosan kevesebb munkát végez? Válaszában térjen ki a szinkronizáció, a determinizmus és a memória-hozzáférés szerepére.

18 Stencil, PDE és iteratív megoldók

18.1 Stencil-alapok

A stencil-számítás a rácsokon végzett párhuzamos adatfeldolgozás egyik legalapvetőbb mintája. A tudományos szimulációktól kezdve a képfeldolgozáson át a neurális hálózatok konvolúciós rétegeiig számos alkalmazás visszavezethető arra, hogy egy rendezett rács minden pontján a pont körüli szomszédság értékeiből számítunk új értéket. Ez a fejezet a stencil fogalmát, a szomszédsági mintát, a peremkezelést és az első hatékony GPU-megvalósítást, a shared memory tilinget vezeti be. A cél, hogy a fogalmakat és a gyakorlati mintát együtt, egy kis ellenőrizhető példán keresztül tanítsa meg.



18.1. ábra: Ötpontos stencil súlyai - A középső cella a saját értékét, a négy szomszéd a stencil együtthatóit kapja; a peremeken a halo vagy a boundary condition dönt.

18.1.1 A stencil és a szomszédsági minta

Legyen adott egy diszkrét rács, például egy d-dimenziós u tömb. A stenciloperátor minden rácspontra egy rögzített, lokális szomszédság lineáris kombinációját számítja ki. Egydimenziós, r sugarú stencil esetén a kimeneti elem

$$u'_i = \sum_{k=-r}^{r} w_k \, u_{i+k},$$

ahol a w_k együtthatók a stencil súlyai. A leggyakoribb 1D-s példa a hárompontos stencil $r = 1$-gyel, például a második derivált közelítése:

$$u_i' = \frac{u_{i-1} - 2u_i + u_{i+1}}{h^2}.$$

Itt a súlyok $w_{-1} = 1/h^2$, $w_0 = -2/h^2$, $w_1 = 1/h^2$. A szomszédsági minta (neighborhood pattern) azoknak az indexeltolásoknak a halmaza, amelyekre az operátor hivatkozik: $\{-1, 0, 1\}$. Kétdimenziós ötpontos Laplace-stencilnél a minta a középpont és a négy von Neumann-szomszéd:

$$u_{i,j}' = \frac{u_{i-1,j} + u_{i+1,j} + u_{i,j-1} + u_{i,j+1} - 4u_{i,j}}{h^2}.$$

A szomszédsági minta itt $\{(-1,0), (1,0), (0,-1), (0,1), (0,0)\}$. A minta előre rögzített, független a rács méretétől, ezért a stencil párhuzamosításra kiválóan alkalmas: minden kimeneti elem ugyanazt a műveletsort kapja, csak a bemeneti indexek tolódnak el. Ezt nevezzük adatpárhuzamos mintának.

A stencil fogalmilag hasonlít a konvolúcióhoz, de van egy lényeges eltérés. A konvolúciónál a kernel szokásosan tükrözve kerül alkalmazásra, míg a stencilnél a súlyok a szomszédsági eltolásokhoz kötöttek. A gyakorlatban a kettőt gyakran szinonimaként használják, különösen mélytanulási környezetben. Ebben a könyvben a stencil alatt mindig a rögzített szomszédsági eltolásokkal definiált lineáris operátort értjük.

A stencil művelet jellemzői:

1. **Szomszédság sugara (radius):** r megadja, hány rácspontnyira nyúlik a bemeneti függőség.
2. **Súlyok:** a w_k együtthatók, amelyek lehetnek pozitívak, negatívak vagy nullák.
3. **Számítási intenzitás:** minden kimeneti pontra $2r+1$ beolvasás és egy összegzés jut, tehát a művelet memóriaorientált: sok beolvasás kevés aritmetikával. Emiatt a teljesítményt elsősorban a memória-hozzáférés határozza meg.
4. **Szeparálhatóság:** bizonyos stencilek felbonthatók alacsonyabb dimenziós stencilek kompozíciójára. Például a 2D Laplace-stencil felírható két 1D-s stencil egymás utáni alkalmazásaként. Ez az optimalizálás a későbbi fejezetek témája, de az alapfogalom itt is hasznos.

18.1.2 Halo és peremkezelés

Amikor a stencil a rács szélén áll, a szomszédsági minta olyan indexekre hivatkozik, amelyek kívül esnek a rács érvényes tartományán. A peremfeltétel (boundary condition) határozza meg, hogyan kezeljük ezeket a hiányzó értékeket. A leggyakoribb peremfeltételek:

- **Dirichlet:** a peremen egy adott, rögzített érték szerepel. Ezt gyakran nullaként választják, vagy egy fizikai peremértéket adnak meg. Az implementációban a határon kívüli indexek ezt a rögzített értéket kapják.
- **Neumann:** a peremen a derivált rögzített, tipikusan nulla. Ez azt jelenti, hogy a kifelé mutató gradiens eltűnik, tehát a peremérték a belső szomszéd értékével egyezik meg. A gyakorlatban ez a perem tükrözésével (reflection) valósítható meg.
- **Periodikus:** a rácsot körbezártnak tekintjük, tehát a bal oldali szomszéd a jobb szélső elem, és fordítva. Ez a modulo indexeléssel kezelhető.

A halo (vagy ghost cell) a tile vagy résztartomány köré másolt extra réteg, amely a szomszédos tartományokból származó adatokat tartalmazza. Elosztott rendszerekben a halo a processzek közötti kommunikációval tölthető fel; egyetlen GPU-n belül a halo a shared memory tile kiterjesztett szegélyét jelenti. A fejezetben a halo kifejezést a shared memory tile köré másolt, a szomszédsághoz szükséges plusz elemekre használjuk.

A peremkezelés és a halo fogalma összekapcsolódik. Ha a teljes rácsot több blokkra bontjuk, minden blokk csak a saját kimeneti elemeit számítja, de a bemeneti szomszédsághoz szükség van a szomszédos blokkok határ menti elemeire is. A blokk ezért a saját tartománya köré egy r szélességű halo-t másol a shared memóriába. A globális rács szélén a halo elemei a megadott peremfeltétel szerint kerülnek feltöltésre.

18.1.3 Naiv globális memóriás GPU-megoldás

A legegyszerűbb CUDA-implementáció minden kimeneti rácspontra egy szálat indít, és a szomszédságot közvetlenül a global memory-ból olvassa. Ez a megoldás kódjában a következő.

```
// CUDA C++ (C++17), illusztráció
__global__ void stencil1d_naive(const float* __restrict__ in,
                                float* __restrict__ out,
                                int n, int radius) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= n) return;
    float acc = 0.0f;
    for (int k = -radius; k <= radius; ++k) {
        int idx = i + k;
        if (idx >= 0 && idx < n) {
            acc += in[idx];
        } else {
            acc += 0.0f; // dirichlet perem: 0
        }
    }
    out[i] = acc;
}
```

A kernel launch alakja például:

```
stencil1d_naive<<<(n + 255) / 256, 256, 0, 0>>>(d_in, d_out, n, 1);
```

A megoldás helyes, de teljesítménye gyenge. Minden kimeneti elemhez $2r + 1$ globális memóriaolvasás tartozik, és ugyanazokat a bemeneti elemeket a szomszédos szálak újra és újra beolvassák. Az átfedő szomszédság miatt a redundancia $2r + 1$-szeres lehet. A global memory sávszélessége a legtöbb esetben a szűk keresztmetszet, ezért a naiv változat csak kis rácsokon vagy ritkán hívott kódokban elfogadható. Ezt a változatot használjuk baseline-ként a laborban.

A naiv megoldás további hátránya, hogy a peremfeltételek ellenőrzése minden szomszéd beolvasásánál elágazást okoz. A rács belsejében ez az elágazás mindig hamis, de a warp szintjén a szélső szálak divergenciát okozhatnak. A határkezelést ezért érdemes leválasztani.

18.1.4 Shared memory tiling

A hatékonyabb megoldás a shared memory tiling: a blokk egyszer betölti a saját kimeneti tartományát és a hozzá tartozó halo-t a shared memóriába, majd a szálak onnan olvassák a szomszédságot. Így minden globális memóriaolvasásból származó elem csak egyszer kerül a chipre, a redundáns hozzáféréseket a gyors, on-chip shared memory szolgálja ki.

A tiling fogalma: a rácsot kimeneti tile-okra bontjuk. Egy blokk egy $T_x \times T_y$ méretű kimeneti tartományt számít ki. A bemenethez szükséges adat a tile plusz a stencil sugara a tile minden oldalán. Ez a kiterjesztett tartomány a shared memory-ba kerül. A shared memory mérete tehát

$$(B_x + 2r) \times (B_y + 2r)$$

elem, ahol B_x és B_y a block dimenzió. Egydimenziós esetben a méret $B_x + 2r$.

A betöltés után blokk szintű szinkronizáció szükséges, mert az egyik szál által betöltött halo-elemet egy másik szál fogja olvasni. A CUDA `__syncthreads()` hívása biztosítja, hogy minden shared memória írás láthatóvá váljon a blokk összes szála számára, mielőtt a számítás elkezdődik.

A tiled megoldás pszeudokódja 2D-ben:

1. A blokk minden szála kiszámítja a globális bemeneti indexét a kiterjesztett tile-ra.
2. A szál ellenőrzi, hogy az index a globális rácson belül van-e.
3. Ha igen, betölti a global memory-ból a shared memória megfelelő cellájába.
4. Ha nem, a peremfeltételnek megfelelő értéket ír a shared cellába.
5. `__syncthreads()`.
6. A szál kiszámítja a saját kimeneti elemét a shared memóriából olvasott szomszédságból.

Az 1D-s tiled kernel a következőképpen néz ki. A példa hárompontos stencilt használ, ahol $r = 1$, és a peremfeltétel Dirichlet-nulla.

```
// CUDA C++ (C++17), illusztráció
__global__ void stencil1d_shared(const float* __restrict__ in,
                                 float* __restrict__ out,
                                 int n) {
    extern __shared__ float s[];
    int tid = threadIdx.x;
    int i = blockIdx.x * blockDim.x + tid;
    int radius = 1;
    int tile_size = blockDim.x;

    // a kiterjesztett tartomány kezdő globális indexe
    int gi = blockIdx.x * blockDim.x - radius + tid;

    // betöltés a global memory-ból, peremkezeléssel
    if (gi >= 0 && gi < n) {
        s[tid] = in[gi];
    } else {
        s[tid] = 0.0f; // dirichlet perem
    }

    __syncthreads();

    if (i < n) {
        float acc = s[tid] * (-2.0f)
                  + s[tid + 1] * 1.0f
                  + s[tid + 2] * 1.0f; // 3-pontos stencil, r=1
        out[i] = acc;
    }
}
```

A launch:

```
int threads = 256;
int shared_bytes = (threads + 2) * sizeof(float);
stencil1d_shared<<<(n + threads - 1) / threads, threads, shared_bytes, 0>>>(d_in, d_out, n);
```

Itt a shared memória hossza `threads + 2 * radius`, tehát `threads + 2`. A szálak a `s[tid]`, `s[tid+1]`, `s[tid+2]` elemeket olvassák. A betöltésnél minden szál egy elemet tölt be, de a kiterjesztett tartomány hossza nagyobb, mint a szálak száma. A fenti kód ezért nem tölti be a kiterjesztett tartomány minden elemét; hiányoznak a jobb oldali halo elemei. A példa egyszerűsített, és a helyes megvalósításhoz a szálaknak ciklikusan be kell tölteniük a kiterjesztett tartományt, vagy a blokk méretét úgy kell megválasztani, hogy a szálak száma megegyezzen a kiterjesztett tartomány hosszával. A laborban ezt a hibát kijavítjuk: a betöltési szakaszban a szálak egy `for` ciklussal, `stride`-dal töltik be a teljes kiterjesztett tartományt.

A helyes betöltési minta 1D-ben:

```
// CUDA C++ (C++17), illusztráció
__global__ void stencil1d_shared_v2(const float* __restrict__ in,
                                    float* __restrict__ out,
                                    int n) {
    extern __shared__ float s[];
    int tid = threadIdx.x;
    int i = blockIdx.x * blockDim.x + tid;
    int radius = 1;
    int tile_size = blockDim.x;
    int s_size = tile_size + 2 * radius;

    // ciklikus betöltés: minden szál több elemet is betölthet
    for (int j = tid; j < s_size; j += blockDim.x) {
        int gi = blockIdx.x * blockDim.x - radius + j;
        s[j] = (gi >= 0 && gi < n) ? in[gi] : 0.0f;
    }
    __syncthreads();

    if (i < n) {
        float acc = s[tid] * (-2.0f)
                  + s[tid + 1] * 1.0f
                  + s[tid + 2] * 1.0f;
        out[i] = acc;
    }
}
```

Itt a `for` ciklus biztosítja, hogy a teljes `s_size` elemet feltöltsük, függetlenül attól, hogy `blockDim.x` kisebb, mint a kiterjesztett méret. Ez a minta általánosan használható 1D és 2D esetben is.

A 2D-s tiled kernel sematikusan a következő. A példa 5-pontos Laplace-stencil, $r = 1$, Dirichlet-nulla peremmel.

```
// CUDA C++ (C++17), illusztráció
#define TILE_X 32
#define TILE_Y 32
#define RADIUS 1

__global__ void stencil2d_5point_shared(const float* __restrict__ in,
                                        float* __restrict__ out,
                                        int width, int height) {
    extern __shared__ float s[];
    int tx = threadIdx.x;
    int ty = threadIdx.y;
    int x = blockIdx.x * TILE_X + tx;
    int y = blockIdx.y * TILE_Y + ty;

    int s_pitch = TILE_X + 2 * RADIUS;
    int sx = tx + RADIUS;
    int sy = ty + RADIUS;
```

```
    // a kiterjesztett tartomány globális mérete
    int s_w = TILE_X + 2 * RADIUS;
    int s_h = TILE_Y + 2 * RADIUS;

    // ciklikus betöltés a blokk minden szálával
    for (int j = ty * blockDim.x + tx; j < s_w * s_h; j += blockDim.x * blockDim.y) {
        int local_x = j % s_w;
        int local_y = j / s_w;
        int gx = blockIdx.x * TILE_X - RADIUS + local_x;
        int gy = blockIdx.y * TILE_Y - RADIUS + local_y;
        if (gx >= 0 && gx < width && gy >= 0 && gy < height) {
            s[local_y * s_pitch + local_x] = in[gy * width + gx];
        } else {
            s[local_y * s_pitch + local_x] = 0.0f; // dirichlet perem
        }
    }
    __syncthreads();

    if (x < width && y < height) {
        float center = s[sy * s_pitch + sx];
        float left   = s[sy * s_pitch + (sx - 1)];
        float right  = s[sy * s_pitch + (sx + 1)];
        float up     = s[(sy - 1) * s_pitch + sx];
        float down   = s[(sy + 1) * s_pitch + sx];
        out[y * width + x] = -4.0f * center + left + right + up + down;
    }
}
```

A kernel launch alakja:

```
dim3 block(TILE_X, TILE_Y);
dim3 grid((width + TILE_X - 1) / TILE_X, (height + TILE_Y - 1) / TILE_Y);
int shared_bytes = (TILE_X + 2) * (TILE_Y + 2) * sizeof(float);
stencil2d_5point_shared<<<grid, block, shared_bytes, 0>>>(d_in, d_out, width, height);
```

A kódban a shared memória elrendezése a `s_pitch` használatával történik, ami $TILE_X + 2$. Ez a padding elkerüli a shared memory bank ütközéseket, amelyek akkor lépnek fel, ha a hozzáférések egyszerre ugyanazt a bankot címzik. A padding bevezetése gyakran javítja a teljesítményt, de a laborban méréssel kell igazolni.

18.1.5 1D és 2D stencil OpenCL-ben és JAX-ban

Az OpenCL programozási modellje hasonló: a work-group felel meg a CUDA blokknak, a local memory a shared memóriának. Az 1D-s stencil naiv változata OpenCL C-ben:

```
// OpenCL C, illusztráció
__kernel void stencil1d_naive(__global const float* in,
                              __global float* out,
                              const int n) {
    int i = get_global_id(0);
    if (i >= n) return;
    float acc = 0.0f;
    for (int k = -1; k <= 1; ++k) {
        int idx = i + k;
        if (idx >= 0 && idx < n) {
            acc += in[idx];
        }
    }
    out[i] = acc;
}
```

A tiled változat local memóriát használna, a szinkronizációhoz `barrier(CLK_LOCAL_MEM_FENCE)` szükséges. A fejezet témája miatt a CUDA-t részletezzük, de az OpenCL-logika ugyanaz.

JAX-ban a stencil gyakran `jax.lax.conv_general_dilated`-dal vagy `jax.scipy.signal`-al fejezhető ki. Egy 2D Laplace-stencil:

```
# Python / JAX, illusztráció
import jax
import jax.numpy as jnp

def laplace_2d(u):
    kernel = jnp.array([[0.0, 1.0, 0.0],
                        [1.0, -4.0, 1.0],
                        [0.0, 1.0, 0.0]], dtype=jnp.float32)
    return jax.lax.conv_general_dilated(
        u[None, None, :, :],
        kernel[None, None, :, :],
        (1, 1),
        'SAME'
    )[0, 0]
```

Itt a JAX a konvolúció primitívet használja, amely a TPU/GPU háttéren a tiled implementációt automatikusan előállítja. A fejezetben ez a kód a modell illusztrációja; a tényleges futtatásnál a JAX verzióját és a háttér eszközt rögzíteni kell.

18.1.6 Labor: 1D és 2D stencil implementálása

A labor célja, hogy a naiv és a shared memory tiled implementációt összehasonlítsd. Az alábbi környezeti adatokat a labor előtt rögzíteni kell, és ha nem áll rendelkezésre valós eszköz, helyőrzőt kell használni:

- GPU: pl. NVIDIA A100 80 GB PCIe (helyőrző: **GPU_MODEL**)
- Driver: **DRIVER_VERSION** (pl. 535.xx)
- CUDA: **CUDA_VERSION** (pl. 12.4)
- OS: **OS_VERSION** (pl. Ubuntu 22.04 LTS)
- Futtatási környezet: izolált konténer, `--gpus all`, timeout 60 s, `nvidia-smi` ellenőrzés
- Precision: FP32 minden mérésnél
- Inputméret: 1D: $n = 2^{24}$ elem; 2D: 4096×4096 elem
- Blockméret: 1D: 256; 2D: 32×32
- Baseline: naiv global memory kernel
- Optimalizált: shared memory tiled kernel

A mérési folyamat:

1. Warm-up: mindkét kernel 5 hívásból álló futtatása, az eredmény eldobása.
2. Mérés: 10 ismétlés, a kernel futásidejét CUDA event-ekkel mérjük.
3. Az eredményekből átlag és szórás számítandó.
4. Az effektív sávszélesség a tényleges adatmozgatásból számolódik: minden kimeneti elemhez annyi bemeneti olvasás tartozik, amennyit a naiv kernel valóban elvégez; a tiled kernelnél a globális olvasások száma körülbelül a tile méret plusz halo. A sávszélesség a mért futásidőből és a ténylegesen mozgatott byte-okból adódik.

A mérési napló táblázata:

implementáció	futásidő (ms)	szórás (ms)	effektív sávszélesség (GB/s)	occupancy (%)	regiszter/szál	shared memória (B/blokk)
1D naiv	**MÉRT**	**MÉRT**	**MÉRT**	**MÉRT**	**MÉRT**	0
1D tiled	**MÉRT**	**MÉRT**	**MÉRT**	**MÉRT**	**MÉRT**	(block+2)*4
2D naiv	**MÉRT**	**MÉRT**	**MÉRT**	**MÉRT**	**MÉRT**	0
2D tiled	**MÉRT**	**MÉRT**	**MÉRT**	**MÉRT**	**MÉRT**	(32+2)*(32+2)*4

A mért és a várt eredményt el kell különíteni. A várt eredmény a modellből:

- A tiled kernel kevesebb globális memóriaolvasást végez, ezért azonos órajel mellett nagyobb effektív sávszélességet és rövidebb futásidőt mutat.
- A tiled kernel occupancy-je alacsonyabb lehet a shared memória igény miatt, de a csökkent globális forgalom ezt kompenzálja.
- A 2D stencil esetén a javulás nagyobb, mert a szomszédsági redundancia nagyobb.

A laborban a hibakezelés kötelező. Minden CUDA API hívás után `checkCuda` makró, a kernel után `cudaDeviceSynchronize` és `cudaGetLastError`. A hibás kernel izolált környezetben fut, timeouttal.

18.1.7 Tipikus hibák és javításuk

A stencil implementálásakor a leggyakoribb hibák a következők.

1. Hiányzó `__syncthreads()` a betöltés és a számítás között. A shared memóriába írás és az olvasás közötti sorrend nincs garantálva szinkronizáció nélkül. A helyes megoldás: `__syncthreads()` a betöltési szakasz végén, majd a számítási szakasz végén újabb szinkronizáció, ha a shared memóriát a következő iterációban újra felülírjuk.

2. Hiányzó vagy hibás peremkezelés. Ha a globális index a rácson kívülre mutat, a memóriahozzáférés nem definiált. A helyes megoldás: a betöltési fázisban a globális index határellenőrzése, és a peremfeltételnek megfelelő érték írása a shared cellába.

3. Nem teljes halo betöltés. Ha a blokk szálainak száma kisebb, mint a kiterjesztett tile mérete, a halo egyes elemei betöltetlenek maradnak. A helyes megoldás: ciklikus betöltési hurok, vagy a blokk méretének a kiterjesztett tile méretéhez igazítása.

4. Bank conflict a shared memóriában. Ha a szomszédsági hozzáférések ugyanabba a bankba mutatnak, a hozzáférés sorosításra kerül. A helyes megoldás: a shared memória sorméretét paddinggel úgy megválasztani, hogy a szomszédos szálakhoz tartozó címek eltérő bankba essenek.

5. Divergencia a peremfeldolgozásnál. Ha a peremkezelés a számítási fázisban marad elágazásként, a warp szintjén a szélső szálak eltérő útvonalat járnak be. A helyes megoldás: a peremkezelést a betöltési fázisba helyezni, ahol a feltétel csak a globális indexre vonatkozik, és a számítási fázisban már nincs elágazás.

6. Helytelen shared memória méret a launch-nál. Ha a launch harmadik paramétere kisebb, mint a kernel által használt `extern __shared__` tömb mérete, a futás hibát okoz vagy csendesen hibás eredményt ad. A helyes megoldás: a méretet egyértelműen kiszámítani, és a launch-nál átadni.

18.1.8 Összefoglalás

A stencil-számítás a rácsos adatfeldolgozás alapmintája, amelyben minden kimeneti pont egy rögzített szomszédsági mintából számolódik. A naiv globális memóriás implementáció helyes, de a redundáns beolvasások miatt sávszélesség-korlátos. A shared memory tiling a tile-t és a halo-t egyszer tölti be a shared memóriába, majd onnan szolgálja ki a szomszédsági hozzáféréseket. A peremkezelést a betöltési fázisban kell elvégezni, a számítási fázisban pedig a blokk szintű szinkronizáció után a szálak a shared memóriából dolgoznak. A labor 1D és 2D stencil összehasonlítását írja elő, ahol a baseline a naiv kernel, az optimalizált változat a tiled kernel. A mérést a könyv konvenciói szerint warm-up-pal, ismétlésekkel, szórással és a valós hardverkörnyezet rögzítésével kell elvégezni.

A következő alfejezet (Halo-csere és többblokkos stencil) erre az alapra épít, és a halo-cserét, valamint a többblokkos stencil végrehajtást tárgyalja, ahol a globális rács mérete nagyobb, mint egyetlen shared memory tile.

18.1.9 Önellenőrző kérdés

Miért van szükség legalább két `__syncthreads()` hívásra egy olyan tiled stencil kernelben, amelyben ugyanazt a shared memóriát többször is felülírják? Gondold végig, mi történne, ha a második szinkronizáció elmaradna.

18.2 Halo-csere és többblokkos stencil

A halo-csere akkor válik elsődleges helyességi és teljesítménykérdéssé, amikor a stencil rácsát több blokk között osztjuk szét. A `gpu066` egységben a stencil-számítás alapjait, a rádiuszt és a shared memory tile gondolatát már bevezettük. Most azt vizsgáljuk, hogyan kezeljük a blokkhatárokon átnyúló függőségeket: mit kell betölteni a szomszédos blokkokból, hogyan biztosítja a globális memória az adatcserét, és milyen szinkronizációs szabályok mellett marad helyes a megoldás.

18.2.1 1. A blokkok közötti halo fogalma

Egy 2D, ötpontos Jacobi-stencil esetén az `(i,j)` cella új értéke:

$$u_{i,j}^{t+1} = \frac{1}{4}\left(u_{i-1,j}^{t} + u_{i+1,j}^{t} + u_{i,j-1}^{t} + u_{i,j+1}^{t}\right),$$

ha Dirichlet-peremmel a tartományon kívüli pontok értékét nullának tekintjük. A stencil rádiusza itt `R=1`, mert minden irányban egy cella távolságig van szükség a szomszédokra.

Ha a rácsot blokkonként feldolgozott csempékre bontjuk, akkor egy `T×T` belső méretű csempének a széle számításához a csempén kívüli, `R` szélességű gyűrű is kell. Ezt a gyűrűt nevezzük halonak vagy szellemcellának. A blokk által számított belső tartományt *interior*, a betöltött, de nem frissített szomszédos gyűrűt *halo* néven különböztetjük meg.

Egy `T×T` csempéhez `H=R=1` esetén a teljes betöltött tartomány:

$$(T + 2R)^2 = (T + 2)^2$$

cella, míg a belső, frissítendő tartomány `T^2`. A halo celláinak száma:

$$N_{\text{halo}} = (T+2)^2 - T^2 = 4T + 4.$$

A halo-arány tehát:

$$\frac{N_{\text{halo}}}{N_{\text{betöltött}}} = \frac{4T+4}{(T+2)^2}.$$

Ez az arány a csempeméret növelésével csökken, tehát a nagyobb csempe kisebb relatív halo-forgalmat jelent. Ugyanakkor a csempe mérete nem nőhet korlátlanul, mert a shared memory és a regiszterkészlet korlátos.

Csempeméret `T`	Betöltött cella `(T+2)^2`	Halo-cella	Halo-arány
8	100	36	36,0%
16	324	68	21,0%
32	1156	132	11,4%
64	4356	260	6,0%

A táblázatban a halo-arány a blokk által a globális memóriából beolvasott adatmennyiség azon része, amely nem belső számítási munka. Ezért a csempeméret megválasztása nemcsak occupancy-kérdés, hanem memóriaforgalmi kérdés is.

18.2.2 2. Implicit és explicit halo-kezelés

A GPU-n a blokkok közötti halo-kezelésnek két gyakorlati alakja van.

18.2.3 Implicit halo globális memórián keresztül

A legegyszerűbb, és egyetlen GPU-n gyakran a legjobb módszer az, hogy nincsenek külön szellemtömbök. A teljes `u` és `u_{\mathrm{next}}` tömb a rács fizikai celláit tárolja `nx×ny` méretben. Minden blokk a saját tile-jához tartozó belső cellákat számítja, de a szükséges szomszédokat közvetlenül a globális memóriában lévő aktuális `u` tömbből olvassa be a shared memory tile-ba.

Ebben a modellben a halo-csere nem külön másolási lépés, hanem maga a globális memóriából való beolvasás. A szomszédos blokkok által előző iterációban kiírt értékeket a következő kernelindítás olvassa. Az iterációk közötti kernelindítás határa egyben globális szinkronizációs pont is: a kernel újraindulása előtt minden blokk befejezte az előző iteráció írását.

Ez a minta együtt jár a dupla pufferelléssel:

```
for k in 0..K-1:
    stencil_kernel<<<grid, block>>>(u, u_next)
    swap(u, u_next)
```

A `swap` itt pointercserét jelent, nem adatmozgatást. A dupla pufferelés azért fontos, mert ha ugyanabba a tömbbe írnánk, amelyből olvasunk, akkor az egyik blokk már felülírhatná azt az értéket, amelyet egy másik blokk még nem olvasott be. A kernelindítások közötti implicit globális szinkronizáció ezt a versenyhelyzetet kiküszöböli.

18.2.4 Explicit halo-csere szellemtömbökkel

Bizonyos tervezési mintákban – például perzisztens blokkoknál, multi-GPU-s felosztásnál, vagy erősen particionált subdomain-tárolásnál – a csempéknek saját, külön tárolt szellemtartományuk van. Ekkor a belső cellák kiszámítása után egy külön lépésben kell a szomszédos csempék szélső celláit a megfelelő szellemtartományba másolni. Ezt nevezzük explicit halo-cserének.

A két modell közötti döntés egyszerűen megfogalmazható:

- egyetlen GPU, iterációnként induló kernel → implicit halo;
- perzisztens blokk, saját subdomain-tárolás, multi-GPU vagy specifikus overlap-terv → explicit halo.

18.2.5 3. Globális memória és szinkronizáció szabályai

A blokkok közötti adatcsere a globális memórián keresztül történik. A globális memória láthatósága a kernelhatárokon garantált: ha egy kernelben az egyik blokk kiír egy értéket, akkor a következő kernel már láthatja azt.

Ebből következik, hogy az implicit halo-minta esetén nem kell semmilyen blokkok közötti `__syncthreads()` hívás. A `__syncthreads()` csak egy blokk szálait szinkronizálja. Nem globális barrier, és blokkok közötti láthatóságot nem ad. Ha valaki például azt feltételezi, hogy a 0. blokk által kiírt szomszédos cella az 1. blokkban egy `__syncthreads()` után látható, az hibás: a blokkok ütemezése nem garantált, és a hardver nem ad ilyen szinkronizációt.

A kernelhatárok által adott globális szinkronizáció elegendő, de van egy ára: minden iterációban lezárul és újraindul egy kernel. Ez több ezer rövid stencil-lépés esetén számottevő launch overhead lehet. Ilyenkor megoldás lehet:

- grid-stride loop és néhány iteráció egy kernelben, de ekkor a halo nem frissülhet az egyes belső lépések között, ezért csak olyan stencil esetén használható, ahol a régebbi halo érték is elfogadható, vagy
- cooperative groups grid-szinkronizáció, amely bizonyos architektúrákon és feltételekkel kernelhatáron belül is ad globális szinkronizációt. Ezt a `gpu032` egység tárgyalja; itt elég annyi, hogy ez haladó eszköz, és alapfeladatban a kernelhatár-alapú megoldás a megbízható kiindulópont.

18.2.6 4. Teljes példa: 2D Jacobi shared memory tile-vel és implicit haloval

Az alábbi CUDA C++ példa azt a javasolt mintát valósítja meg, amikor a halo a globális `u` tömbből kerül a shared memory tile-ba. A kernel 32×32 belső cellát számít blokkonként, és minden blokk egy `34×34` shared memory tile-t tölt fel.

```
// CUDA C++ (C++17)
// halo_stencil.cu
#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <vector>
#include <algorithm>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                \
  do {                                                                  \
    cudaError_t err = (call);                                           \
```

```
    if (err != cudaSuccess) {                                              \
      std::fprintf(stderr, "CUDA error %s:%d: %s\n",                       \
                   __FILE__, __LINE__, cudaGetErrorString(err));           \
      std::exit(EXIT_FAILURE);                                             \
    }                                                                      \
  } while (0)

using real = double;

constexpr int R = 1;
constexpr int TILE_X = 32;
constexpr int TILE_Y = 32;

// Tiled kernel: a halo-t a globális u tömbből tölti be.
__global__ void jacobi2d_tiled(const real* __restrict__ u,
                               real* __restrict__ un,
                               int nx, int ny)
{
  __shared__ real s[TILE_Y + 2 * R][TILE_X + 2 * R];

  const int tx0 = blockIdx.x * TILE_X;
  const int ty0 = blockIdx.y * TILE_Y;

  const int tile_w = TILE_X + 2 * R;
  const int tile_h = TILE_Y + 2 * R;

  // A tile-nek és a halo-nak megfelelő cellák betöltése.
  for (int idx = threadIdx.y * blockDim.x + threadIdx.x;
       idx < tile_w * tile_h;
       idx += blockDim.x * blockDim.y) {

    const int shx = idx % tile_w;
    const int shy = idx / tile_w;
    const int gx  = tx0 + shx - R;
    const int gy  = ty0 + shy - R;

    const bool inside_x = (gx >= 0 && gx < nx);
    const bool inside_y = (gy >= 0 && gy < ny);

    s[shy][shx] = (inside_x && inside_y) ? u[gy * nx + gx] : real(0);
  }

  __syncthreads();

  const int i = ty0 + threadIdx.y;
  const int j = tx0 + threadIdx.x;

  if (i < ny && j < nx) {
    const int sx = threadIdx.x + R;
    const int sy = threadIdx.y + R;

    const real left  = s[sy][sx - 1];
    const real right = s[sy][sx + 1];
    const real down  = s[sy - 1][sx];
    const real up    = s[sy + 1][sx];

    un[i * nx + j] = real(0.25) * (left + right + down + up);
  }
}

// Referenciakernel: ugyanaz a stencil közvetlen globális memóriaolvasással.
__global__ void jacobi2d_ref(const real* __restrict__ u,
                             real* __restrict__ un,
```

```
                              int nx, int ny)
{
  const int i = blockIdx.y * blockDim.y + threadIdx.y;
  const int j = blockIdx.x * blockDim.x + threadIdx.x;

  if (i >= ny || j >= nx) return;

  const real left  = (j > 0)       ? u[i * nx + j - 1] : real(0);
  const real right = (j + 1 < nx)  ? u[i * nx + j + 1] : real(0);
  const real down  = (i > 0)       ? u[(i - 1) * nx + j] : real(0);
  const real up    = (i + 1 < ny)  ? u[(i + 1) * nx + j] : real(0);

  un[i * nx + j] = real(0.25) * (left + right + down + up);
}

real* run_tiled(real* u, real* un, int nx, int ny, int iterations)
{
  dim3 block(TILE_X, TILE_Y);
  dim3 grid((nx + TILE_X - 1) / TILE_X,
            (ny + TILE_Y - 1) / TILE_Y);

  for (int k = 0; k < iterations; ++k) {
    jacobi2d_tiled<<<grid, block>>>(u, un, nx, ny);
    CUDA_CHECK(cudaGetLastError());
    std::swap(u, un);
  }

  return u;
}

real* run_ref(real* u, real* un, int nx, int ny, int iterations)
{
  dim3 block(32, 32);
  dim3 grid((nx + 31) / 32, (ny + 31) / 32);

  for (int k = 0; k < iterations; ++k) {
    jacobi2d_ref<<<grid, block>>>(u, un, nx, ny);
    CUDA_CHECK(cudaGetLastError());
    std::swap(u, un);
  }

  return u;
}

int main(int argc, char** argv)
{
  const int nx         = (argc > 1) ? std::atoi(argv[1]) : 2048;
  const int ny         = (argc > 2) ? std::atoi(argv[2]) : 2048;
  const int iterations = (argc > 3) ? std::atoi(argv[3]) : 20;

  const size_t n = static_cast<size_t>(nx) * ny;
  const size_t nbytes = n * sizeof(real);

  std::vector<real> h_u(n), h_zero(n, real(0)), h_tiled(n), h_ref(n);

  for (int i = 0; i < ny; ++i) {
    for (int j = 0; j < nx; ++j) {
      const real x = static_cast<real>(j);
      const real y = static_cast<real>(i);
      h_u[i * nx + j] =
          std::sin(real(0.02) * x) * std::cos(real(0.012) * y) +
          std::exp(-((x - nx / real(2)) * (x - nx / real(2)) +
                     (y - ny / real(2)) * (y - ny / real(2))) /
```

```
                real(2000.0));
  }
}

real *d_u = nullptr, *d_un = nullptr;
real *d_ref_u = nullptr, *d_ref_un = nullptr;

CUDA_CHECK(cudaMalloc(&d_u, nbytes));
CUDA_CHECK(cudaMalloc(&d_un, nbytes));
CUDA_CHECK(cudaMalloc(&d_ref_u, nbytes));
CUDA_CHECK(cudaMalloc(&d_ref_un, nbytes));

CUDA_CHECK(cudaMemcpy(d_u, h_u.data(), nbytes, cudaMemcpyHostToDevice));
CUDA_CHECK(cudaMemcpy(d_ref_u, h_u.data(), nbytes, cudaMemcpyHostToDevice));
CUDA_CHECK(cudaMemcpy(d_un, h_zero.data(), nbytes, cudaMemcpyHostToDevice));
CUDA_CHECK(cudaMemcpy(d_ref_un, h_zero.data(), nbytes, cudaMemcpyHostToDevice));

// Warm-up: a mérés előtti első indítás ne torzítsa az időt.
{
  dim3 block(TILE_X, TILE_Y);
  dim3 grid((nx + TILE_X - 1) / TILE_X,
            (ny + TILE_Y - 1) / TILE_Y);
  jacobi2d_tiled<<<grid, block>>>(d_u, d_un, nx, ny);
  CUDA_CHECK(cudaGetLastError());
  CUDA_CHECK(cudaDeviceSynchronize());
}

cudaEvent_t start, stop;
CUDA_CHECK(cudaEventCreate(&start));
CUDA_CHECK(cudaEventCreate(&stop));

CUDA_CHECK(cudaEventRecord(start, 0));
real* final_tiled = run_tiled(d_u, d_un, nx, ny, iterations);
CUDA_CHECK(cudaEventRecord(stop, 0));
CUDA_CHECK(cudaEventSynchronize(stop));

float ms = 0.0f;
CUDA_CHECK(cudaEventElapsedTime(&ms, start, stop));
const double seconds = static_cast<double>(ms) / 1000.0;

dim3 block(TILE_X, TILE_Y);
dim3 grid((nx + TILE_X - 1) / TILE_X,
          (ny + TILE_Y - 1) / TILE_Y);

const size_t loaded_cells =
    static_cast<size_t>(grid.x) * grid.y *
    (TILE_X + 2 * R) * (TILE_Y + 2 * R) * iterations;

const size_t written_cells = n * static_cast<size_t>(iterations);

const double moved_bytes =
    static_cast<double>(loaded_cells + written_cells) * sizeof(real);

const double effective_bw_gbs =
    moved_bytes / (seconds * 1e9);

real* final_ref = run_ref(d_ref_u, d_ref_un, nx, ny, iterations);
CUDA_CHECK(cudaDeviceSynchronize());

CUDA_CHECK(cudaMemcpy(h_tiled.data(), final_tiled,
                      nbytes, cudaMemcpyDeviceToHost));
CUDA_CHECK(cudaMemcpy(h_ref.data(), final_ref,
                      nbytes, cudaMemcpyDeviceToHost));
```

```
  double max_abs_diff = 0.0;
  for (size_t idx = 0; idx < n; ++idx) {
    max_abs_diff = std::max(max_abs_diff,
                            std::abs(h_tiled[idx] - h_ref[idx]));
  }

  std::printf("nx=%d ny=%d iterations=%d tile=%dx%d\n",
              nx, ny, iterations, TILE_X, TILE_Y);
  std::printf("kernel time = %.3f ms\n", ms);
  std::printf("effective bandwidth = %.2f GB/s\n",
              effective_bw_gbs);
  std::printf("max abs diff vs reference = %.17g\n", max_abs_diff);

  CUDA_CHECK(cudaFree(d_u));
  CUDA_CHECK(cudaFree(d_un));
  CUDA_CHECK(cudaFree(d_ref_u));
  CUDA_CHECK(cudaFree(d_ref_un));
  CUDA_CHECK(cudaEventDestroy(start));
  CUDA_CHECK(cudaEventDestroy(stop));

  return 0;
}
```

A kódban a `run_tiled` minden iterációban újraindítja a kernelt, majd pointert cserél. A `final_tiled` pointer ezért az utolsó iteráció eredményét mutatja. A referenciakernel ugyanazon a topológián dolgozik, de nem használ shared memory tile-t; így a két futás numerikusan összehasonlítható.

A sávszélességszámítás itt a tényleges globális adatmozgást közelíti:

$$\mathrm{GB/s} = \frac{\left[G_x G_y (T_x+2)(T_y+2)K + n_x n_y K\right] \cdot 8}{t \cdot 10^9},$$

ahol `K` az iterációk száma, `G_x,G_y` a rácsdimenzió, `T_x,T_y` a tile belső mérete, `t` pedig a mért futásidő másodpercben. Ez nem a hardver névleges csúcssávszélessége, hanem az algoritmus globális memóriaforgalmára számított effektív érték.

18.2.7 5. Explicit halo-csere egy egyszerű 1D modellen

Az explicit halo-csere oktatására az 1D eset a legáttekinthetőbb. Tegyük fel, hogy a fizikai tartományt `T` darab, egyenként `W` belső cellát tartalmazó csempére osztjuk. Minden csempe globális memóriában `S = W + 2H` hosszú szeletet foglal el, ahol `H` a halo szélessége. Egy csempe helyi indexei:

```
[0, H-1]              bal halo
[H, H+W-1]            belső cellák
[H+W, H+W+H-1]        jobb halo
```

A belső kernel kiszámítja a `H` és `H+W-1` közötti frissített értékeket. Ezután a halo-kernel a bal szélső belső cellákat a szomszéd jobb halojába, a jobb szélső belső cellákat pedig a szomszéd bal halojába másolja.

```
// CUDA C++ (C++17) -- explicit 1D halo-másoló vázlat
constexpr int H = 1;

__global__ void halo1d_explicit(const real* __restrict__ src,
                                real* __restrict__ dst,
```

```
                                int tiles, int stride, int width)
{
  const int t = blockIdx.x;
  const int h = threadIdx.x;

  if (t >= tiles || h >= H) return;

  // Bal halo: a t-1-edik csempe utolsó H belső cellájából.
  if (t > 0) {
    dst[t * stride + h] =
        src[(t - 1) * stride + width + h];
  }

  // Jobb halo: a t+1-edik csempe első H belső cellájából.
  if (t < tiles - 1) {
    dst[t * stride + H + width + h] =
        src[(t + 1) * stride + H + h];
  }
}
```

Ez a vázlat egy blokkot indít csempénként, és blokkon belül `H` szálat használ. Nagy `H` vagy sok csempe esetén ez gyenge kihasználtságú lehet, de a helyességi gondolatot jól mutatja. Termelési kódban érdemes a másolást egyetlen kernelben, az összes halocella sorszámát egy 1D indexre leképezve megszervezni, vagy 2D esetben `cudaMemcpy2DAsync` hívásokkal a téglalap alakú szegélyeket másolni.

2D esetben az explicit halo-csere négy lépésből áll: felső, alsó, bal és jobb szegély másolása. Sorirányú szegélyekhez a `cudaMemcpy2DAsync` alkalmas, mert egyetlen hívásban másol `W×H` méretű sorcsíkot device pointerek között. A hívás sémája:

```
CUDA_CHECK(cudaMemcpy2DAsync(
    dst_ptr, dst_pitch,
    src_ptr, src_pitch,
    width * sizeof(real),
    height,
    cudaMemcpyDeviceToDevice,
    stream));
```

Ilyenkor a `src_ptr` a szomszédos csempe belső szegélyére, a `dst_ptr` a célcsempe megfelelő halosávjára mutat, a `pitch` pedig a tényleges byte-beli sortávolság. Ez a megoldás aszinkron streamen is futtatható, és nem igényel külön kernelkódot a másoláshoz.

18.2.8 6. Skálázás és a halo költsége

A halo-kezelés költsége a csempék felületével arányos, a hasznos számítás a csempék térfogatával. Ezért a blokkszám növelésével a halo-forgalom relatív súlya nő.

Erős skálázás esetén a rács mérete rögzített, a blokkok számát növeljük. Ekkor a csempék belső mérete csökken, így a halo-arány nő. Például egy `512×512` belső rács esetén:

Blokkrács	Tile `T`	Halo-arány
4×4	128	3,1%
8×8	64	6,0%
16×16	32	11,4%
32×32	16	21,0%

A gyakorlatban a `T=16` már nagyon nagy halo-arányt jelent, ezért ilyen felbontásnál a blokkszámot nem érdemes túlzásba vinni. Emellett az occupancy is romolhat, ha a kis tile mellett ugyanakkora shared memory fenntartása felesleges.

Gyenge skálázás esetén a csempeméretet rögzítjük, és a rács növelésével arányosan növeljük a blokkok számát. Ekkor a halo-arány közel állandó, és a megoldás jól skálázható, amíg a GPU el nem éri a blokkok és szálak ütemezési korlátait. A gyakorlati optimumot úgy érdemes keresni, hogy a `T` értékét például 16, 32, 64 és 96 között futtatott mérésekkel vizsgáljuk, miközben rögzítjük az occupancyt, a shared memory méretét és a regiszterhasználatot.

18.2.9 7. Tipikus hibák és javításuk

18.2.10 Hiányzó `__syncthreads()` a tile betöltése után

Ha a shared memory tile-t feltöltő szálak és a stencilt számító szálak nem szinkronizálnak, akkor az egyik szál még a régi vagy részlegesen kiírt `s` értéket olvashatja. Helyes megoldás: a betöltési ciklus után mindig `__syncthreads()`.

18.2.11 In-place frissítés

Ha a kernel ugyanabba a tömbbe ír, amelyből a szomszédos blokkok olvasnak, akkor a kiírás sorrendje nem szabályozott. Ez időben változó, nehezen reprodukálható eredményt ad. Helyes megoldás: dupla pufferelés, azaz `u` és `u_next` váltogatása.

18.2.12 Blokkok közötti `__syncthreads()` feltételezése

A `__syncthreads()` nem globális barrier. Ha a kód azt feltételezi, hogy az összes blokkhalo már frissült, akkor hibás. Helyes megoldás: a halo-cserét kernelhatárra bízni, vagy coop groups grid-szinkronizációt használni szigorú architekturális feltételek mellett.

18.2.13 Nem inicializált halocellák

A shared memory tile szélein a tartományon kívüli cellákba véletlen érték kerülhet, ha azokat nem állítjuk be. A példakódban a feltételen kívül eső cellák értéke `0`, ami Dirichlet-peremet valósít meg. Neumann-perem esetén a belső szomszédból, periodikus perem esetén a szemközti oldalról kell tölteni.

18.2.14 Túl nagy csempeméret

A nagy `T` csökkenti a relatív halo-forgalmat, de növeli a shared memory foglalást, és csökkentheti az occupancyt. A 32×32 tile FP64 esetén `(32+2)^2×8  9,2 KiB` shared memóriát foglal, ami több blokk egyidejű futtatását általában lehetővé teszi. A 96×96 tile már `98^2×8  75,4 KiB`, ami SM-enként csak néhány blokkot enged, így a latency hiding romolhat.

18.2.15 Nem ellenőrzött kernelhiba

A CUDA kernelindítás aszinkron, ezért a hiba sokszor nem azonnal jelentkezik. A kódban minden indítás után `cudaGetLastError`, a futtatás után pedig `cudaDeviceSynchronize` szükséges. Az out-of-bounds hibákat a `compute-sanitizer --tool memcheck` mutatja meg.

18.2.16 8. Labor: halo-csere 2D Jacobi-stencilben

18.2.17 Fordítás és futtatás

A forrás neve legyen `halo_stencil.cu`. A buildparancsot a céleszköz compute capabilityjéhez kell igazítani:

```
# Pontegye be a labor GPU compute capabilityjét.
nvcc -O3 -std=c++17 -arch=sm_80 -lineinfo \
     -o halo_stencil halo_stencil.cu

# Kis ráccsal, gyors helyességi ellenőrzés:
compute-sanitizer --tool memcheck ./halo_stencil 256 256 2

# Nagyobb ráccsal, mérés:
./halo_stencil 4096 4096 20
```

18.2.18 Rögzítendő GPU-konfiguráció

Minden mérés előtt rögzíteni kell a környezetet:

```
nvidia-smi --query-gpu=name,driver_version,clocks.sm,power.limit --format=csv
nvcc --version
```

Példa naplósablon, nem mért adat:

Elem	Érték
GPU modell	a labor GPU azonosítója
Compute capability	a kártya tényleges CC-értéke
Driver	a telepített driver verziója
CUDA toolkit	a `nvcc --version` által mutatott verzió
Operációs rendszer	a labor OS-e
Power limit	`nvidia-smi` alapján
ECC / MIG / háttérterhelés	kapcsolt állapot

18.2.19 Elvárt eredmény

A tiled kernel és a referenciakernel ugyanazt az aritmetikai műveletsort végzi, azonos FP64 pontossággal, ezért a `max_abs_diff` elvárt értéke `0` vagy a fordító optimalizációjából adódóan néhány ULP alatti eltérés. Az itt bemutatott FP64 implementációnál a tíz- vagy húszlépéses Jacobi-iteráció után a hiba jellemzően `0,0` vagy közel az.

18.2.20 Mért eredmény

A mérési naplóban ezeket az oszlopokat kell rögzíteni, és a mért értékeket elkülöníteni az elvárttól:

Metrika	Elvárt	Mért
Kernelidő	a hardver függvénye	`[mérendő]` ms
Effektív sávszélesség	számított	`[mérendő]` GB/s
`max_abs_diff`	0	`[mérendő]`
Occupancy	a tile és a kernel erőforrásai alapján számítandó	`[mérendő]`

A mérésnél legalább 5–10 ismétlést kell futtatni, és az átlag mellett a szórást vagy percentilist is közölni kell. A bemutatott kód egyetlen időmérést végez; valódi benchmarkban a warm-up utáni ismétléseket külső ciklusból kell gyűjteni.

18.2.21 OpenCL és JAX kiegészítés

Ugyanez a minta OpenCL C-ben a `__local` memóriával valósítható meg. A kernelváza:

```
// OpenCL C (koncepcionális vázlat, fordítási ellenőrzéshez)
__kernel void jacobi2d_halo(__global const double* u,
                            __global double* un,
                            int nx, int ny)
{
  __local double s[34][34];

  int tx0 = get_group_id(0) * 32;
  int ty0 = get_group_id(1) * 32;

  int i = ty0 + get_local_id(1);
  int j = tx0 + get_local_id(0);

  // ... s feltöltése get_local_size(0), get_local_size(1) alapján ...
  barrier(CLK_LOCAL_MEM_FENCE);

  if (i < ny && j < nx) {
    un[i * nx + j] = 0.25 * (
        s[get_local_id(1) + 1][get_local_id(0)] +
        s[get_local_id(1) + 1][get_local_id(0) + 2] +
        s[get_local_id(1)][get_local_id(0) + 1] +
        s[get_local_id(1) + 2][get_local_id(0) + 1]);
  }
}
```

JAX-ban a blokkhalo kérdését a keretrendszer és a fordító kezeli. A funkcionális implementáció például `jnp.roll` és maszk segítségével adható meg:

```
# JAX (koncepcionális, GPU/TPU-vázlat)
import jax
import jax.numpy as jnp

@jax.jit
def jacobi_step(u):
    left = jnp.roll(u, 1, axis=1).at[:, 0].set(0.0)
    right = jnp.roll(u, -1, axis=1).at[:, -1].set(0.0)
    down = jnp.roll(u, 1, axis=0).at[0, :].set(0.0)
```

```
    up = jnp.roll(u, -1, axis=0).at[-1, :].set(0.0)
    return 0.25 * (left + right + down + up)
```

A JAX ebben a vázlatban a sharding és a halo kommunikációját a háttérben végzi; az explicit blokkszintű irányítás nem jelenik meg. A méréshez ezért külön kell rögzíteni a JAX verzióját, a háttér eszközt (GPU/TPU), valamint a sharding tervet. A TPU-ra vonatkozó szabályokat a `gpu106`–`gpu109` egységek tárgyalják.

18.2.22 9. Önellenőrző kérdések

1. Miért nem oldja meg a `__syncthreads()` a blokkok közötti halo-függőséget?
2. Egy `64×64` belső méretű tile esetén mennyi a halo-arány `R=1` mellett, és miért javul a sávszélesség-kihasználás a `32×32` tile-hoz képest?
3. Milyen hiba lép fel, ha a Jacobi-stencilt ugyanabba a globális tömbbe írjuk, amelyből olvasunk? Miért ad rossz vagy nem determinisztikus eredményt?
4. Mi a különbség az implicit és az explicit halo-csere között, és mikor érdemes explicit másolást használni?

A `gpu068` egység ezekre a fogalmakra építve tárgyalja majd a PDE-megoldókat és az iteratív módszerek GPU-s szerkezetét. A halo-kezelés itt bemutatott double buffering és kernelhatáros szinkronizációs mintája lesz az alapja a későbbi fejlettebb átfedési és pipeline-technikáknak is.

18.3 PDE és iteratív megoldók

A parciális differenciálegyenletek (PDE-k) diszkretizálása gyakran nagy, ritka lineáris egyenletrendszerekhez vezet: $\mathbf{A}\mathrm{x} = \mathrm{b}$, ahol $\mathbf{A}$ ritka mátrix, $\mathbf{b}$ a peremfeltételekből és forrástagból származó vektor. A direkt megoldók (pl. LU-felbontás) a feltöltődés miatt GPU-n általában nem hatékonyak, ezért iteratív módszereket használunk. Ebben az egységben három alapvető iteratív megoldót tárgyalunk: a Jacobi-, a Gauss–Seidel- és a konjugált gradiens (CG) módszert, kiemelve a GPU-s megvalósításukat, a konvergencia és reziduál szerepét, valamint a prekondicionálás alapjait. Az előző egységben (Halo-csere és többblokkos stencil) bevezetett stencil-számítás adja a mátrix-vektor szorzás magját; itt erre építünk.

Jacobi-iteráció GPU-n

A Jacobi-módszer a legegyszerűbb iteratív megoldó. A lineáris egyenletrendszert komponensenként frissítjük:

$$x_i^{(k+1)} = \frac{b_i - \sum_{j \neq i} a_{ij} x_j^{(k)}}{a_{ii}}.$$

Mivel minden új x_i csak az előző iteráció x_j értékeitől függ, a módszer természetesen párhuzamos: minden komponens egyszerre frissíthető. Ez ideális GPU-ra.

A stencil-számításnál az $\mathbf{A}$ mátrix implicit, tipikusan egy 5-pontos Laplace-operátor 2D rács esetén. A frissítési képlet egy súlyozott átlag:

$$x_{i,j}^{(k+1)} = \frac{b_{i,j} + c\left(x_{i-1,j}^{(k)} + x_{i+1,j}^{(k)} + x_{i,j-1}^{(k)} + x_{i,j+1}^{(k)}\right)}{c_0},$$

ahol c a rácsállandó, c_0 a diagonális elem. A kernel minden belső rácspontra kiszámítja az új értéket. Fontos, hogy a Jacobi nem in-place: külön forrás- és céltömb kell, mert a szomszédok

régi értékeit használjuk. A dupla buffer technikát alkalmazzuk: két tömböt allokálunk, és iterációnként cseréljük a mutatókat.

A GPU-s kernel egy egyszerű 2D indexeléssel dolgozik. A határpontokat (peremfeltételek) nem frissítjük, azokat külön kezeljük (pl. konstans érték). A kernel egy block/grid konfigurációt használ, például 16×16-os blokkok és a rácsméretnek megfelelő grid. A frissítéshez nincs szükség globális szinkronizációra, mert a kernel csak olvassa a régi tömböt; a kernel végén a `__syncthreads()` csak a blokkon belüli megosztott memória esetén kell, de ha minden szál globális memóriát olvas, nincs szükség szinkronizációra. A kernel befejezése után a host oldalon `cudaDeviceSynchronize()` biztosítja, hogy a következő iteráció csak a teljes frissítés után induljon.

A reziduál kiszámításához egy külön redukciós kernel szükséges, amely kiszámítja az $\|r\|_2$ normát. Ezt lehet minden iterációban vagy ritkábban (pl. minden 10. iterációban) elvégezni a konvergencia ellenőrzésére. A redukciót a könyv korábbi fejezete (Redukciók) tárgyalja, itt csak felhasználjuk.

A Jacobi-módszer konvergenciájának feltétele a mátrix szigorú diagonális dominanciája vagy bizonyos speciális tulajdonságok. A Laplace-operátor diszkretizációja rendelkezik ezzel, ezért a Jacobi konvergál, bár lassan. A konvergencia sebessége a spektrálsugártól függ: minél közelebb van 1-hez, annál lassabb. A rácsfelbontás növelésével a konvergencia romlik, ezért nagy feladatoknál a Jacobi önmagában ritkán hatékony, de alapul szolgál a gyorsabb módszerekhez.

Kódpélda (Jacobi kernel)

A következő CUDA C++ kód szemlélteti a Jacobi-iteráció magját egy 2D Poisson-egyenletre. A kód a könyv laborjában futtatható; a hibakezelés és a mérés a későbbi laborleírásban szerepel.

```
// jacobi_kernel.cu (CUDA C++17, nvcc)
__global__ void jacobi_step(const double* __restrict__ u_old,
                            double* __restrict__ u_new,
                            const double* __restrict__ f,
                            int nx, int ny, double dx2, double dy2) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;
    if (i >= 1 && i < nx-1 && j >= 1 && j < ny-1) {
        int idx = j * nx + i;
        double inv_diag = 1.0 / (2.0/dx2 + 2.0/dy2);
        double sum = (u_old[idx-1] + u_old[idx+1]) / dx2
                   + (u_old[idx-nx] + u_old[idx+nx]) / dy2;
        u_new[idx] = inv_diag * (f[idx] + sum);
    }
}
```

A kernel a belső pontokat frissíti; a határpontokat a peremfeltétel szerint állítjuk be (pl. nullára). A dupla buffert a host oldalon kezeljük: két tömb, `u0` és `u1`, minden iterációban felváltva. A konvergencia ellenőrzéséhez a reziduált egy másik kernelben számoljuk.

Gauss–Seidel GPU-n

A Gauss–Seidel (GS) módszer a Jacobihoz hasonló, de a frissítés során már az új értékeket használja:

$$x_i^{(k+1)} = \frac{b_i - \sum_{j<i} a_{ij} x_j^{(k+1)} - \sum_{j>i} a_{ij} x_j^{(k)}}{a_{ii}}.$$

Ez a módszer sorosnak tűnik, mert az i-edik komponens frissítése függ a korábbi komponensek új értékétől. A természetes rendezésben a párhuzamosítás nehéz. Azonban a rácsos szerkezetű mátrixoknál (pl. 2D Laplace) alkalmazható a red-black (piros-fekete) rendezés. A rácspontokat két színre osztjuk, mint egy sakktábla: a piros pontoknak csak fekete szomszédai vannak, és fordítva. Ekkor a piros pontok frissítésekor csak a fekete pontok régi értékeit használjuk, a

fekete pontok frissítésekor pedig a piros pontok új értékeit. Így két fél-iterációban párhuzamosan frissíthető az összes azonos színű pont.

A red-black GS kernel két kernelből áll (vagy egy kernelből, amely a szín alapján választ). A következő kód a fekete pontok frissítését mutatja; a piros hasonló, csak a feltétel változik.

```
__global__ void gs_black_step(const double* __restrict__ u,
                              double* __restrict__ u_new,
                              const double* __restrict__ f,
                              int nx, int ny, double dx2, double dy2) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;
    if (i >= 1 && i < nx-1 && j >= 1 && j < ny-1) {
        // fekete pont: (i+j) % 2 == 0
        if ((i + j) % 2 == 0) {
            int idx = j * nx + i;
            double inv_diag = 1.0 / (2.0/dx2 + 2.0/dy2);
            double sum = (u_new[idx-1] + u_new[idx+1]) / dx2
                       + (u_new[idx-nx] + u_new[idx+nx]) / dy2;
            u_new[idx] = inv_diag * (f[idx] + sum);
        }
    }
}
```

Itt `u_new` már tartalmazza a piros pontok frissített értékeit, mert a piros kernel előbb futott. Fontos, hogy a két kernel között nincs globális szinkronizáció, de a kernelindítások sorrendje garantálja a memória láthatóságát (a CUDA streamben a kernelindítások sorrendben hajtódnak végre, és a memória írások láthatóvá válnak a következő kernel számára). A red-black GS jellemzően gyorsabban konvergál, mint a Jacobi, de a párhuzamosítási foka fele akkora, mert egyszerre csak a pontok fele frissül. A gyakorlatban a GS-t gyakran használják simítóként multigrid módszerekben.

Konjugált gradiens (CG) GPU-n

A konjugált gradiens módszer szimmetrikus pozitív definit (SPD) mátrixokra hatékony. A PDE-kből származó mátrixok gyakran SPD-k (pl. Laplace-operátor). A CG egy Krylov-altér módszer, amely minden iterációban minimalizálja a hibát az **A**-normában. Az algoritmus:

1. Inicializálás: $r_0 = b - Ax_0$, $p_0 = r_0$.
2. Minden iterációban:

$$\alpha_k = \frac{r_k^T r_k}{p_k^T A p_k},$$

$$x_{k+1} = x_k + \alpha_k p_k,$$

$$r_{k+1} = r_k - \alpha_k A p_k,$$

$$\beta_k = \frac{r_{k+1}^T r_{k+1}}{r_k^T r_k},$$

$$p_{k+1} = r_{k+1} + \beta_k p_k.$$

A módszer elméletileg legfeljebb n iterációban konvergál (n a dimenzió), de a gyakorlatban sokkal kevesebb is elég, ha a mátrix jól kondicionált. A GPU-n a CG minden iterációja a következő műveleteket igényli: SpMV (Ap_k), dot (skaláris szorzatok), axpy (vektor frissítés) és normaszámítás. Ezek a műveletek hatékonyan párhuzamosíthatók, de a dot műveletek redukciót igényelnek, amelyek globális szinkronizációt okoznak. A CG GPU-s teljesítményét gyakran a redukciók költsége korlátozza.

A gyakorlatban a CG-t könyvtárakkal valósítjuk meg: a cuBLAS a sűrű vektorműveletekhez, a cuSPARSE a ritka mátrix-vektor szorzáshoz. A következő kódvázlat a cuSPARSE és cuBLAS használatát mutatja (a hibakezelést elhagyva a tömörség kedvéért, de a teljes laborban kötelező):

```
// PCG vázlat (CUDA C++17, cuBLAS + cuSPARSE)
cublasHandle_t cublasH;
cusparseHandle_t cusparseH;
// ... mátrix és vektorok allokálása ...
cusparseSpMV(cusparseH, CUSPARSE_OPERATION_NON_TRANSPOSE,
             &alpha, mat, p, &beta, Ap, CUSPARSE_MV_ALG_DEFAULT, buffer);
// r^T r: cublasDdot(cublasH, n, r, 1, r, 1, &rr)
// p^T Ap: cublasDdot(cublasH, n, p, 1, Ap, 1, &pAp)
// axpy: cublasDaxpy(cublasH, n, &alpha, p, 1, x, 1)
// ...
```

A saját kernel implementáció esetén az SpMV-t a stencil kernel adja, a dot-ot pedig a redukciós kernel. A redukciót a korábbi fejezetben bemutatott módon végezhetjük (pl. shared memory redukció, majd atomics a globális eredményre). A CG általában gyorsabban konvergál, mint a Jacobi vagy GS, főleg nagy kondíciószámú feladatoknál, de a prekondicionálás tovább javíthatja.

Reziduál és konvergencia

A reziduál $r_k = b - Ax_k$ az egyenletrendszer hibájának mértéke. Mivel a pontos megoldás x ismeretlen, a hibát $e_k = x - x_k$ nem tudjuk közvetlenül számolni, de a reziduál kiszámítható. A leállási kritérium általában a relatív reziduál normája: $\|r_k\|/\|b\| < \text{tol}$, ahol tol egy előírt tűrés, pl. 10^{-6}. A norma lehet 2-es, végtelen vagy más. A 2-es norma számítása redukciót igényel, ami költséges, ezért gyakran csak minden néhány iterációban ellenőrizzük.

A konvergencia sebessége függ a mátrix spektrális tulajdonságaitól. Jacobi esetén a konvergencia feltétele a mátrix szigorú diagonális dominanciája (vagy bizonyos feltételek). A GS általában gyorsabb, de hasonló feltételekkel. A CG konvergenciája a kondíciószámtól függ: a hiba csökkenése $\approx \frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}$ szerint halad, ahol κ a kondíciószám. A prekondicionálás célja a kondíciószám csökkentése.

Numerikus pontosság: a reziduál számítása érzékeny a kerekítési hibákra. FP32-ben a reziduál stagnálhat, mert a lebegőpontos pontosság korlátozza a csökkentést. A PDE-megoldóknál általában FP64 ajánlott, különösen a konvergencia ellenőrzéséhez. Mixed precision is lehetséges: a mátrix-vektor szorzást FP32-ben végezzük, de a reziduált és a skaláris szorzatokat FP64-ben, így gyorsabb, de pontosabb. A laborban rögzíteni kell a használt pontosságot.

Preconditionálás

A prekondicionálás során egy olyan $\mathbf{M}$ mátrixot alkalmazunk, amely közelíti $\mathbf{A}$-t, de könnyen invertálható, és az $\mathbf{M}^{-1}\mathbf{A}$ kondíciószáma kisebb, mint $\mathbf{A}$-é. A prekondicionált CG (PCG) algoritmusban a prekondicionálót a reziduálra alkalmazzuk: $z_k = M^{-1} r_k$. A legegyszerűbb a Jacobi prekondicionáló, ahol $\mathbf{M} = \text{diag}(\mathbf{A})$. Ez csak a diagonális elemmel való skálázást jelent, rendkívül olcsó és párhuzamos. A Jacobi prekondicionáló gyenge, de javíthat a konvergencián, ha a diagonális domináns.

Erősebb prekondicionálók: blokk-Jacobi, ahol a mátrixot kis blokkokra osztjuk, és minden blokkot pontosan invertálunk (pl. LU-felbontással a blokkon belül). Ez GPU-n hatékony lehet, mert a blokkok függetlenek. Az inkomplett LU (ILU) prekondicionáló népszerű CPU-n, de soros jellege miatt GPU-n nehéz; léteznek párhuzamos változatok (pl. szintű ILU), de a teljesítmény korlátozott. A sparse approximate inverse (SPAI) prekondicionáló mátrixot explicit módon közelíti $\mathbf{A}^{-1}$-gyel, és az alkalmazása SpMV-vel történik, ami GPU-barát. A gyakorlatban gyakran használnak polinomiális prekondicionálókat vagy multigridet, de ezek túlmutatnak ezen a fejezeten.

A PCG algoritmusban a prekondicionáló alkalmazása egy extra művelet iterációnként. A következő kód a Jacobi prekondicionáló alkalmazását mutatja a PCG-ben:

```
// Jacobi preconditioner alkalmazása: z = M^{-1} r
__global__ void jacobi_precond(const double* __restrict__ r,
                               double* __restrict__ z,
                               const double* __restrict__ diag,
                               int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        z[i] = r[i] / diag[i];
    }
}
```

A prekondicionáló megválasztása kompromisszum: minél erősebb, annál több számítást igényel az alkalmazása, de annál kevesebb iteráció kell. A GPU-s környezetben a párhuzamosan alkalmazható prekondicionálók előnyösek.

Labor: Jacobi-iteráció implementálása

A labor célja egy 2D Poisson-egyenlet megoldása Jacobi-iterációval CUDA C++-ban, a konvergencia mérésével. Az alábbi leírás egy teljes, futtatható keretet ad; a hallgató feladata a kód lefuttatása, a mérési napló kitöltése és a várt/mért eredmények összehasonlítása.

Feladat: Oldjuk meg a $-\Delta u = f$ egyenletet az egységnégyzeten, Dirichlet peremfeltétellel $u = 0$ a határon. A forrástag legyen $f(x, y) = 2\pi^2 \sin(\pi x)\sin(\pi y)$, amelynek analitikus megoldása $u(x, y) = \sin(\pi x)\sin(\pi y)$. A diszkretizálás 5-pontos stencillel, $n \times n$ rács, a rácsállandó $h = 1/(n + 1)$.

Hardver és szoftver környezet (példa, a hallgató a sajátját rögzíti): - GPU: NVIDIA GeForce RTX 3080, compute capability 8.6 - Driver: 535.104.05, CUDA Toolkit: 12.2 - OS: Ubuntu 22.04 LTS - Órajel: 1710 MHz (boost), power limit: 320 W, ECC: off, MIG: off - Háttérterhelés: nincs (izolált futtatás)

Kód: A következő program tartalmazza a Jacobi kernelt, a reziduál számítását, a hibakezelést és az időmérést. A kód a könyv laborjában futtatható; az itt bemutatott változat nem tekinthető futtatottnak, amíg a hallgató nem hajtja végre.

```
// jacobi_lab.cu (CUDA C++17, nvcc)
#include <cuda_runtime.h>
#include <cstdio>
#include <cmath>
#include <chrono>

#define CUDA_CHECK(call) do { \
    cudaError_t err = call; \
    if (err != cudaSuccess) { \
        fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__, \
                cudaGetErrorString(err)); \
        cudaDeviceReset(); \
        exit(EXIT_FAILURE); \
    } \
} while (0)

__global__ void jacobi_step(const double* __restrict__ u_old,
                            double* __restrict__ u_new,
                            const double* __restrict__ f,
                            int n, double inv_h2) {
    int i = blockIdx.x * blockDim.x + threadIdx.x + 1;
    int j = blockIdx.y * blockDim.y + threadIdx.y + 1;
    if (i < n+1 && j < n+1) { // belső pontok: 1..n
```

```
        int idx = j * (n+2) + i;
        double inv_diag = 0.25 * h * h; // mivel 4/h^2 a diagonál
        double sum = (u_old[idx-1] + u_old[idx+1] + u_old[idx-(n+2)] + u_old[idx+(n+2)]) *
↪   inv_h2;
        u_new[idx] = (f[idx] + sum) * inv_diag;
    }
}

__global__ void residual_norm(const double* __restrict__ u,
                              const double* __restrict__ f,
                              const double* __restrict__ b,
                              double* __restrict__ partial,
                              int n, double inv_h2) {
    // A = 4/h^2 a diagonál, -1/h^2 a szomszédok
    int i = blockIdx.x * blockDim.x + threadIdx.x + 1;
    int j = blockIdx.y * blockDim.y + threadIdx.y + 1;
    double local = 0.0;
    if (i < n+1 && j < n+1) {
        int idx = j * (n+2) + i;
        double Au = 4.0 * inv_h2 * u[idx] - inv_h2 * (u[idx-1] + u[idx+1] + u[idx-(n+2)] +
↪   u[idx+(n+2)]);
        double r = b[idx] - Au;
        local = r * r;
    }
    // block redukció
    __shared__ double sdata[256];
    int tid = threadIdx.x + threadIdx.y * blockDim.x;
    sdata[tid] = local;
    __syncthreads();
    for (int s = blockDim.x * blockDim.y / 2; s > 0; s >>= 1) {
        if (tid < s) sdata[tid] += sdata[tid + s];
        __syncthreads();
    }
    if (tid == 0) atomicAdd(partial, sdata[0]);
}

int main(int argc, char** argv) {
    int n = 1024; // belső pontok száma egy irányban
    double tol = 1e-6;
    if (argc > 1) n = atoi(argv[1]);
    if (argc > 2) tol = atof(argv[2]);
    double h = 1.0 / (n + 1);
    double inv_h2 = 1.0 / (h * h);

    // Allokáció
    size_t size = (n + 2) * (n + 2) * sizeof(double);
    double *d_u0, *d_u1, *d_f, *d_b, *d_partial;
    CUDA_CHECK(cudaMalloc(&d_u0, size));
    CUDA_CHECK(cudaMalloc(&d_u1, size));
    CUDA_CHECK(cudaMalloc(&d_f, size));
    CUDA_CHECK(cudaMalloc(&d_b, size));
    CUDA_CHECK(cudaMalloc(&d_partial, sizeof(double)));

    // Inicializálás (host oldalon, majd másolás)
    // ... (a laborban a hallgató tölti ki, pl. f = 2 ² sin( x) sin( y), b = f)
    // ...

    // Launch konfiguráció
    dim3 block(16, 16);
    dim3 grid((n + block.x - 1) / block.x, (n + block.y - 1) / block.y);

    // Iteráció
    double *u_old = d_u0, *u_new = d_u1;
```

```
    int max_iter = 10000;
    double res_norm = 1.0;
    int iter = 0;
    auto start = std::chrono::high_resolution_clock::now();
    for (iter = 0; iter < max_iter && res_norm > tol; ++iter) {
        jacobi_step<<<grid, block>>>(u_old, u_new, d_f, n, inv_h2);
        CUDA_CHECK(cudaGetLastError());
        // reziduál számítás minden 50. iterációban
        if (iter % 50 == 0) {
            CUDA_CHECK(cudaMemset(d_partial, 0, sizeof(double)));
            residual_norm<<<grid, block>>>(u_old, d_f, d_b, d_partial, n, inv_h2);
            CUDA_CHECK(cudaGetLastError());
            double partial;
            CUDA_CHECK(cudaMemcpy(&partial, d_partial, sizeof(double), cudaMemcpyDeviceToHost));
            res_norm = sqrt(partial);
            // normáljuk ||b||-vel
            // ...
        }
        // csere
        std::swap(u_old, u_new);
    }
    CUDA_CHECK(cudaDeviceSynchronize());
    auto end = std::chrono::high_resolution_clock::now();
    double elapsed = std::chrono::duration<double>(end - start).count();

    printf("Iterációk: %d, reziduál: %e, idő: %f s\n", iter, res_norm, elapsed);

    CUDA_CHECK(cudaFree(d_u0));
    CUDA_CHECK(cudaFree(d_u1));
    CUDA_CHECK(cudaFree(d_f));
    CUDA_CHECK(cudaFree(d_b));
    CUDA_CHECK(cudaFree(d_partial));
    return 0;
}
```

A fenti kódban a `b` vektor a forrástag, mert a stencil jobb oldala f. A peremfeltételek miatt a határpontokon $u = 0$, ezért azokat nem kell frissíteni. A laborban a `d_b`-t inicializáljuk f-fel.

Buildparancs:

```
nvcc -O3 -arch=sm_86 -o jacobi_lab jacobi_lab.cu
```

A hallgató a saját GPU-architektúrájához igazítja az `-arch` kapcsolót.

Futtatási parancs:

```
./jacobi_lab 1024 1e-6
```

Mérési napló: A mérés előtt warm-up futtatás szükséges (pl. 2 iteráció), majd legalább 5–10 ismétlés. A naplóban rögzíteni kell az alábbi adatokat: - GPU modell, driver, CUDA, compute capability, OS - Órajel, power limit, ECC, MIG, háttérterhelés - n, tol, precision (FP64) - Minden ismétlés futási ideje, átlag, szórás - A reziduál értéke az utolsó iterációban - A várt eredmény (analitikus megoldás) és a mért eredmény összehasonlítása (pl. maximális eltérés a belső pontokon)

A mérési napló például így nézhet ki (a hallgató tölti ki a saját adataival):

Ismétlés	Futásidő (s)	Reziduál
1	12,34	9,8e-7
2	12,45	9,7e-7

Ismétlés	Futásidő (s)	Reziduál
…	…	…
Átlag	12,40	9,8e-7
Szórás	0,06	1e-8

Várt eredmény: A Jacobi-iterációnak konvergálnia kell a tűréshez. A konvergencia sebessége függ a rácsfelbontástól: n növelésével a kondíciószám nő, ezért több iteráció kell. A reziduál csökkenése lineáris (log skálán közel egyenes). A mért megoldásnak közelítenie kell az analitikus megoldást; az eltérés a tűrés alatt kell legyen.

Tipikus hibák és megelőzésük

1. *Versenyhelyzet a halo-cserénél:* Ha a Jacobi kernel nem dupla buffert használ, hanem in-place frissít, a szomszédok új értékeit olvashatja, ami a GS-hez hasonló viselkedést eredményez, de nem determinisztikus. Mindig dupla buffert használjunk, és a kernel végén ne feledkezzünk meg a mutatócseréről.
2. *Szinkronizáció hiánya:* A kernelindítások közötti `cudaDeviceSynchronize()` elhagyása esetén a host oldali iteráció nem várja meg a GPU-t, és a mérés hamis lesz. A laborban minden iteráció után szinkronizáljunk, vagy használjunk streamet és eventet.
3. *Pontatlan FP32:* A reziduál FP32-ben korán stagnálhat, és a megoldás nem éri el a kívánt pontosságot. Használjunk FP64-et a Jacobi-iterációhoz, vagy mixed precision esetén a reziduált FP64-ben számoljuk.
4. *Helytelen indexelés a határokon:* A kernel feltételeinek pontosan meg kell határozniuk a belső pontokat. A határpontokat külön kell inicializálni, és nem szabad frissíteni.
5. *Redukció hibás blokkmérettel:* A redukciós kernelben a shared memory méretét a blokk méretéhez kell igazítani, és a `__syncthreads()`-et minden lépésben hívni kell. A `atomicAdd` a részeredményekre biztosítja a helyes összegzést.

Önellenőrző kérdések

1. Miért van szükség dupla bufferre a Jacobi-iterációban? Mi történne, ha in-place frissítenénk?
2. Hogyan párhuzamosítható a Gauss–Seidel módszer 2D rács esetén? Mi a red-black rendezés lényege?
3. Mi a reziduál definíciója, és hogyan használjuk a leállási kritériumban? Miért fontos a relatív reziduál?
4. Milyen prekondicionáló egyszerű és GPU-barát? Mi a hátránya?
5. A CG módszer mely műveletei igényelnek redukciót, és miért lehet ez szűk keresztmetszet?

Ez az egység bevezette a PDE-k iteratív megoldóinak GPU-s alapjait. A következő egység (Stencil-optimalizálás) a stencil-optimalizálásra összpontosít, ahol a bemutatott módszereket tovább gyorsítjuk tilinggel, shared memory-val és egyéb technikákkal.

18.4 Stencil-optimalizálás

A stencil-optimalizálás annak a művészete, hogy a szomszédsági mintázaton alapuló iteratív számítást a GPU valós erőforrás-korlátaihoz igazítsuk. A Stencil-alapok–PDE és iteratív megoldók egységekben megismertük a stencil alapjait, a halo-cserét és az iteratív megoldókat.

Most azt vizsgáljuk meg, hogyan lehet egy adott stencil magját a teljesítmény szempontjából tudatosan hangolni. Három pillérre építünk: a memóriamozgás és a számítás viszonyát leíró **Roofline-elemzésre**, a több időlépést egyetlen kernelben egyesítő **időblokkolásra** (*time blocking*), valamint a gyakorlati hangolási paraméterek (tileméret, cache-politika, vektorizáció) szisztematikus vizsgálatára. A cél nem a „gyorsabb" homályos ígérete, hanem egy reprodukálható, méréssel alátámasztott optimalizálási folyamat kialakítása.

18.4.1 1. A stencil teljesítményének hangolása: mit és miért hangolunk?

A stencil-számítások jellemzően **memória-kötöttek**: a feladat aritmetikai intenzitása (arithmetic intensity, AI) alacsony, ezért a futásidőt elsősorban az határozza meg, milyen gyorsan tudjuk a bemeneti adatokat beolvasni a globális memóriából, illetve a közbenső eredményeket oda visszaírni. Ugyanakkor a cache-ek, a shared memory és a regiszterek segítségével az adat-újrafelhasználás növelhető, így a globális memória felé irányuló forgalom csökkenthető. A hangolás tehát nem más, mint az adat-újrafelhasználás maximalizálása a memóriahierarchia minden szintjén, miközben fenntartjuk a kellő párhuzamosságot és elkerüljük a bankütközéseket, a divergenciát és a regiszter-túlcsordulást.

A hangolandó paraméterek tipikusan:

- **Tile mérete**: mekkora résztömböt töltünk be a shared memory-ba, illetve mekkora részt számolunk ki egy szálblokkban.
- **Block mérete**: hány szál dolgozik együtt, és ez hogyan befolyásolja az occupancy-t.
- **Vektorizáció**: mennyi adatot olvasunk be egyetlen betöltéssel (pl. `float4`).
- **Cache-politika**: L2 persistence, `cudaFuncAttributePreferredSharedMemoryCarveout`, illetve cache hint-ek a load/store utasításokon.
- **Unroll mértéke**: hány iterációt bontunk ki, hogy csökkentsük az utasítás-függőségeket.
- **Időblokkolás**: hány időlépést számolunk ki egyetlen kernelindításon belül, mielőtt a határokat újra cserélnénk.

Minden optimalizálási állítást méréssel kell igazolni. A mérési környezetet rögzíteni kell: GPU típus, compute capability, driver verzió, CUDA verzió, órajel, power limit, ECC be/kikapcsolva, operációs rendszer. Az eredményeket azonos inputméretre, azonos numerikus pontosságra (pl. FP32) és azonos iterációszámra kell vonatkoztatni.

18.4.2 2. Roofline-elemzés stencilre

A Roofline-modell (lásd A Roofline-modell) két korlátot állít szembe: a memória-sávszélesség korlátját és a számítási csúcsteljesítmény korlátját. Egy adott kernel **aritmetikai intenzitása** (AI) a végrehajtott hasznos lebegőpontos műveletek száma osztva a ténylegesen mozgatott bájtok számával:

$$AI = \frac{\text{FLOP}}{\text{byte}}.$$

A modell szerint a elérhető teljesítmény P (FLOP/s) felső korlátja:

$$P \le \min\left(\pi_{\text{peak}},\ \beta_{\text{eff}} \cdot AI\right),$$

ahol π_{peak} a GPU névleges csúcs FLOP/s értéke, β_{eff} pedig a hatékony memória-sávszélesség (byte/s). A kettő metszéspontja az ún. **fordulópont** (ridge point), amelynél az AI értéke $\pi_{\text{peak}}/\beta_{\text{eff}}$.

Egy 2D 5-pontos Jacobi-stencil esetén minden rácspont frissítéséhez elvileg 4 lebegőpontos művelet tartozik (négy szomszéd összeadása, majd egy szorzás vagy osztás). Ha a bemeneti és kimeneti rácsot egy-egy `float` tömbben tároljuk, és minden pontot egyszer olvasunk, egyszer írunk, akkor egy pont adatforgalma 8 bájt (4 olvasás + 4 írás, ha nincs újrafelhasználás). Az AI ekkor:

$$AI = \frac{4 \text{ FLOP}}{8 \text{ byte}} = 0{,}5 \text{ FLOP/byte}.$$

Ez rendkívül alacsony, messze a fordulópont alatt, tehát a kernel erősen memória-kötött. Ha azonban a szomszédos pontok értékeit sikerül újra felhasználni – például shared memory-ba tilinggel –, a globális memória felé irányuló forgalom csökken. Tegyük fel, hogy egy $B \times B$ méretű tile-t töltünk be a shared memory-ba, és azt T időlépésen keresztül frissítjük (időblokkolás). Ekkor a globális memóriából beolvasott adat mennyisége hozzávetőlegesen $O(B^2)$ helyett $O(B^2 + T \cdot B)$ lesz, mert a határpontokat minden időlépésben újra kell tölteni a halo-cseréhez, de a belső pontok újrahasznosulnak. Az AI ennek megfelelően nő, és a kernel közelebb kerülhet a számítási korláthoz.

A Roofline-elemzés lépései stencil esetén:

1. **Határozzuk meg a tényleges FLOP-ot** a kernelben. Ne a névleges műveletszámot használjuk, hanem a forrásból levezetett valós utasításszámot (pl. 1 FMA = 2 FLOP). A stencilnél a szomszédok összeadása és a konstanssal való szorzás a jellemző.
2. **Mérjük meg a tényleges adatforgalmat** a globális memória felé. Ezt Nsight Compute memória-metrikáiból (`dram__bytes_read`, `dram__bytes_write`) vagy saját számlálóval kaphatjuk meg.
3. **Számítsuk ki az AI-t** a kettő hányadosaként.
4. **Ábrázoljuk a Roofline-diagramon**, és azonosítsuk, melyik korlát dominál.
5. **Optimalizáljunk a domináns korlát mentén**: ha memória-kötött, csökkentsük az adatforgalmat (tiling, időblokkolás, cache); ha számítás-kötött, növeljük az utasítás-szintű párhuzamosságot, használjunk FMA-t, csökkentsük a felesleges műveleteket.

A Roofline-elemzés nem helyettesíti a mérést, hanem keretet ad a mérések értelmezéséhez. A gyakorlatban a mérési pontok a két korlát alatt helyezkednek el, mert a modell idealizált: nem számol a bankütközésekkel, a divergenciával, a szinkronizációs overheaddel, a launch latency-vel és a cache-találati arány ingadozásával.

18.4.3 3. Időblokkolás (time blocking)

A hagyományos stencil-végrehajtás minden időlépésben külön kernelindítást hajt végre: a kernel beolvassa a jelenlegi állapotot, kiszámítja a következőt, majd kiírja azt. Ez a megközelítés egyszerű, de két komoly hátránya van:

1. **Globális memória forgalom** minden időlépésben: a teljes rácsot ki kell olvasni és visszaírni, függetlenül attól, hogy a belső pontok értékeit a következő lépésben újra felhasználnánk.
2. **Kernelindítási overhead**: sok kis kernel indítása többletköltséget jelent, különösen akkor, ha a grid mérete kicsi.

Az **időblokkolás** alapötlete, hogy több időlépést egyetlen kernelindításon belül hajtsunk végre. A kernel egy szálblokkja egy térbeli tile-t tölt be a shared memory-ba, majd azt többször frissíti, miközben a belső pontok a shared memory-ban maradnak, és csak a tile határán lévő pontok frissítéséhez kell időnként a szomszédos tile-okból adatot cserélni. Ezzel a globális memória felé irányuló forgalom drasztikusan csökken, mert a belső pontok értékei több időlépésen keresztül nem érintik a globális memóriát.

Formálisan: legyen a rács mérete $N \times N$, egy blokk $B \times B$ méretű tile-t kezel, és T időlépést hajt végre. A hagyományos megközelítés költsége $O(T \cdot N^2)$ globális memória-hozzáférés (minden lépésben a teljes rács). Az időblokkolással a blokk először betölti a $B \times B$ méretű tile-t és annak egy rétegnyi halo-ját, tehát $(B+2)^2$ pontot. Ezután T lépésen át frissíti a belső $B \times B$ pontot. Minden lépésben a tile határán lévő pontok frissítéséhez a szomszédos blokkok aktuális határértékeire van szükség, ezért a határokat időnként (pl. minden lépésben) újra kell tölteni. A globális memória-hozzáférés így hozzávetőlegesen:

$$\text{Global forgalom} \approx (B+2)^2 + T \cdot (4B) \quad \text{(pontonként 4 bájt)}$$

szemben a hagyományos $T \cdot B^2$ ponttal. Ha B elég nagy és T mérsékelt, a megtakarítás jelentős.

A gyakorlatban azonban nem minden stencil alkalmas egyszerű időblokkolásra. A **Jacobi-iteráció** például pontonként független, ezért jól párhuzamosítható, de az időblokkolás megvalósítása bonyolultabb, mert a tile-ok közötti adatcsere szinkronizációt igényel. A **soros (Gauss–Seidel) típusú** stencileknél az időbeli függőség miatt a hagyományos párhuzamosítás is nehezebb, ezért ott gyakran használnak piros-fekete (red-black) rendezést vagy más dekompozíciót.

18.4.4 3.1. Időblokkolás megvalósítása CUDA-ban

Az időblokkolás tipikus megvalósítása a **shared memory-ba töltött tile és a regiszterek közötti adatmozgás** kombinációja. A legegyszerűbb esetben a kernel egyetlen időlépést hajt végre shared memory tilinggel. Az időblokkoláshoz ezt a lépést többször ismételjük egy belső ciklusban, miközben a tile határait frissítjük.

Példaként vegyünk egy 2D 5-pontos stencilt (hővezetési egyenlet Jacobi-módszere). A kernel vázlata időblokkolás nélkül (egy időlépés) a Stencil-alapok alfejezetben már szerepelt. Most tekintsük a több időlépéses változatot.

A kernel pszeudokódja:

```
__global__ void jacobi_timeblock(float* u_new, const float* u_old,
                                 int N, int T, ...)
{
    __shared__ float tile[BLOCK_Y+2][BLOCK_X+2];

    // Globális index a tile bal felső sarkára
    int tx = threadIdx.x, ty = threadIdx.y;
    int bx = blockIdx.x * BLOCK_X, by = blockIdx.y * BLOCK_Y;

    // Betöltés a shared memory-ba, beleértve a halo réteget
    for (int j = ty; j < BLOCK_Y+2; j += blockDim.y)
        for (int i = tx; i < BLOCK_X+2; i += blockDim.x) {
            int gx = bx + i - 1;
            int gy = by + j - 1;
            if (gx >= 0 && gx < N && gy >= 0 && gy < N)
                tile[j][i] = u_old[gy * N + gx];
            else
                tile[j][i] = 0.0f; // peremfeltétel
```

```
        }
    __syncthreads();

    // T időlépés
    for (int step = 0; step < T; ++step) {
        // Belső pontok frissítése
        if (tx < BLOCK_X && ty < BLOCK_Y) {
            int i = tx + 1, j = ty + 1; // 1-based a halo miatt
            float val = 0.25f * (tile[j-1][i] + tile[j+1][i] +
                                 tile[j][i-1] + tile[j][i+1]);
            tile[j][i] = val; // új érték a shared memory-ba
        }
        __syncthreads();

        // A tile határpontjainak frissítése a szomszédos tile-okból
        // Ez leegyszerűsítve: újratöltjük a halo réteget a globális memóriából
        // A gyakorlatban itt a szomszédos blokkokkal való kommunikáció bonyolultabb.
        // A példa kedvéért feltételezzük, hogy a halo réteget minden lépésben
        // újra kell tölteni a globális memóriából (nem hatékony, de szemléltet).
        __syncthreads();
    }

    // Visszaírás a globális memóriába
    if (tx < BLOCK_X && ty < BLOCK_Y) {
        int gx = bx + tx;
        int gy = by + ty;
        if (gx < N && gy < N)
            u_new[gy * N + gx] = tile[ty+1][tx+1];
    }
}
```

Ez a vázlat nem hatékony, mert minden lépésben újratölti a halo réteget, és a szomszédos blokkok közötti adatcsere nincs megoldva. Valójában a halo-csere a blokkok között explicit szinkronizációt igényel, ami CUDA-ban grid-szintű szinkronizációval (cooperative groups) vagy atomi zászlókkal valósítható meg. Az időblokkolás előnye akkor jelentkezik, ha a halo-csere költsége kicsi a belső számítás költségéhez képest, vagy ha a halo-cserét ritkán (pl. minden második lépésben) végezzük el.

A gyakorlatban az időblokkolást gyakran **többszintű tiling**gel kombinálják: a globális memóriából a shared memory-ba töltött tile mérete nagyobb, mint amit egyetlen lépésben frissítünk, így a belső pontok több lépésen át újrahasznosulnak. A regiszterekbe töltött értékek további újrafelhasználást tesznek lehetővé.

18.4.5 3.2. Az időblokkolás előnyei és korlátai

Előnyök:

- Csökkenti a globális memória forgalmat, ezáltal növeli az AI-t.
- Csökkenti a kernelindítások számát, ami kis grid esetén számottevő overhead.
- Lehetővé teszi a cache jobb kihasználását, mivel az adatok hosszabb ideig a chipen maradnak.

Korlátok és buktatók:

- A shared memory mérete korlátozott (jellemzően 48–164 KiB SM-enként), ezért a tile mérete nem nőhet tetszőlegesen nagyra.
- A tile-ok közötti halo-csere szinkronizációt igényel, ami bonyolult és holtidőt okozhat.
- A regiszterhasználat nőhet, ami csökkenti az occupancy-t.
- A bankütközések kockázata nő a nagyobb shared memory tömböknél.

- Nem minden stencil alkalmas időblokkolásra; például ha a stencil időben változó együtthatókat használ, vagy ha a rácspontok közötti függőség nem teszi lehetővé a független frissítést.

18.4.6 4. Gyakorlati hangolási stratégiák

Az alábbi lépések a stencil-optimalizálás iteratív folyamatát írják le:

1. **Kezdjünk egy egyszerű, helyes kernelből** (baseline). Mérjük meg a futásidőt, a memória-sávszélességet és az occupancy-t.
2. **Végezzünk Roofline-elemzést** az AI kiszámításával. Döntsük el, memória- vagy számításkötött-e a kernel.
3. **Alkalmazzunk shared memory tilinget** a globális memória újraolvasásának csökkentésére. Hangoljuk a tile méretét: a túl kicsi tile nem csökkenti eléggé a forgalmat, a túl nagy tile csökkenti az occupancy-t és növeli a shared memory nyomást.
4. **Vektorizáljuk a betöltéseket** (`float2`, `float4`), hogy jobban kihasználjuk a memória-sávszélességet és csökkentsük az utasításszámot.
5. **Alkalmazzunk időblokkolást**, ha a stencil lehetővé teszi. Mérjük meg, hány időlépés adja a legjobb kompromisszumot a globális forgalom és a szinkronizációs overhead között.
6. **Hangoljuk a cache-politikát**: állítsuk be az L2 persistence-t a gyakran használt adatokra, vagy használjunk `cudaFuncAttributePreferredSharedMemoryCarveout`-ot a shared memory arányának növelésére.
7. **Csökkentsük a bankütközéseket**: padding bevezetésével a shared memory tömbben.
8. **Optimalizáljuk a launch konfigurációt**: blockméret, gridméret, esetleg grid-stride loop.
9. **Mérjünk újra**, és hasonlítsuk össze a baseline-nal. Az optimalizálás akkor sikeres, ha a futásidő csökken, miközben a numerikus eredmény az előírt hibahatáron belül változatlan.

18.4.7 5. Labor: tileméret- és cache-hangolás

Ebben a laborban a fenti elveket gyakoroljuk egy 2D hővezetési egyenlet Jacobi-iterációján. A cél a tileméret és a cache-beállítások hatásának mérése. A laborhoz CUDA C++ kódot adunk, de az elvek OpenCL-re és JAX-re is átvihetők; ahol releváns, megjegyezzük a párhuzamokat.

18.4.8 5.1. Feladatleírás

Adott egy $N \times N$ méretű 2D rács, amelyen a következő diszkrét hővezetési egyenletet oldjuk meg Jacobi-iterációval:

$$u_{i,j}^{(k+1)} = \frac{1}{4}\left(u_{i-1,j}^{(k)} + u_{i+1,j}^{(k)} + u_{i,j-1}^{(k)} + u_{i,j+1}^{(k)}\right),$$

ahol $u^{(k)}$ a k-adik időlépés, a peremfeltétel Dirichlet (rögzített 0) vagy Neumann. Az iterációt fix számú lépésig futtatjuk.

A labor során a következő paramétereket hangoljuk:

- **Tile méret**: a shared memory-ba töltött résztömb mérete a szálblokkban. Kísérletezzünk 4×4, 8×8, 16×16, 32×32 méretekkel.
- **Cache-politika**: az L2 cache-hozzáférési ablak (`cudaStreamAttrValue` access policy window) vagy a preferált shared memory carve-out beállítása. Mérjük a hatást a futásidőre és az L2 találati arányra.

A méréshez rögzítsük a környezetet, futtassunk minden konfigurációt legalább 10-szer, és adjuk meg az átlagos futásidőt és a szórást.

18.4.9 5.2. CUDA C++ kód

Az alábbi kód egy baseline kernel és egy shared memory tilinges változat. A hibakezelést a könyv konvencióinak megfelelően `CHECK_CUDA` makróval végezzük.

```
// Fordító: CUDA C++17 (nvcc 12.x)
// Nyelv: CUDA C++
#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <chrono>

// Hibakezelő makró
#define CHECK_CUDA(call) do {                                                  \
    cudaError_t err = call;                                                    \
    if (err != cudaSuccess) {                                                  \
        fprintf(stderr, "CUDA hiba: %s, fájl: %s, sor: %d\n",                  \
                cudaGetErrorString(err), __FILE__, __LINE__);                  \
        exit(EXIT_FAILURE);                                                    \
    }                                                                          \
} while (0)

// Baseline kernel: nincs shared memory tiling
__global__ void jacobi_naive(const float* __restrict__ u_old,
                             float* __restrict__ u_new,
                             int N) {
    int j = blockIdx.y * blockDim.y + threadIdx.y;
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= 1 && i < N-1 && j >= 1 && j < N-1) {
        u_new[j * N + i] = 0.25f * (u_old[j * N + i - 1] +
                                    u_old[j * N + i + 1] +
                                    u_old[(j - 1) * N + i] +
                                    u_old[(j + 1) * N + i]);
    }
}

// Tilinges kernel: shared memory-ba töltés
// BLOCK_X és BLOCK_Y a belső pontok mérete, a halo hozzáadódik.
#define TILE_X 16  // Hangolandó paraméter
#define TILE_Y 16
#define HALO 1

__global__ void jacobi_tiled(const float* __restrict__ u_old,
                             float* __restrict__ u_new,
                             int N) {
    __shared__ float tile[TILE_Y + 2 * HALO][TILE_X + 2 * HALO];

    int tx = threadIdx.x;
    int ty = threadIdx.y;
    int bx = blockIdx.x * TILE_X;
    int by = blockIdx.y * TILE_Y;

    // Betöltés a shared memory-ba, beleértve a halo réteget
    for (int j = ty; j < TILE_Y + 2 * HALO; j += blockDim.y) {
        for (int i = tx; i < TILE_X + 2 * HALO; i += blockDim.x) {
            int gx = bx + i - HALO;
            int gy = by + j - HALO;
            if (gx >= 0 && gx < N && gy >= 0 && gy < N) {
```

```
                tile[j][i] = u_old[gy * N + gx];
            } else {
                tile[j][i] = 0.0f; // peremfeltétel
            }
        }
    }
    __syncthreads();

    // Belső pontok frissítése
    int i = bx + tx;
    int j = by + ty;
    if (i >= 1 && i < N-1 && j >= 1 && j < N-1 && tx < TILE_X && ty < TILE_Y) {
        int lx = tx + HALO;
        int ly = ty + HALO;
        u_new[j * N + i] = 0.25f * (tile[ly - 1][lx] + tile[ly + 1][lx] +
                                    tile[ly][lx - 1] + tile[ly][lx + 1]);
    }
}

// Inicializálás
void init_field(float* field, int N) {
    for (int i = 0; i < N * N; ++i) {
        field[i] = static_cast<float>(rand()) / RAND_MAX;
    }
}

int main() {
    // Paraméterek
    const int N = 1024;          // rács méret
    const int ITERATIONS = 100;  // iterációk száma

    // Eszköz kiválasztása és tulajdonságok
    int device = 0;
    CHECK_CUDA(cudaSetDevice(device));
    cudaDeviceProp prop;
    CHECK_CUDA(cudaGetDeviceProperties(&prop, device));
    printf("GPU: %s, Compute Capability: %d.%d, SMs: %d\n",
           prop.name, prop.major, prop.minor, prop.multiProcessorCount);

    // Memória allokálás
    float *d_u_old, *d_u_new;
    CHECK_CUDA(cudaMalloc(&d_u_old, N * N * sizeof(float)));
    CHECK_CUDA(cudaMalloc(&d_u_new, N * N * sizeof(float)));

    // Host inicializálás
    float* h_u = (float*)malloc(N * N * sizeof(float));
    init_field(h_u, N);
    CHECK_CUDA(cudaMemcpy(d_u_old, h_u, N * N * sizeof(float), cudaMemcpyHostToDevice));
    CHECK_CUDA(cudaMemcpy(d_u_new, h_u, N * N * sizeof(float), cudaMemcpyHostToDevice));
    free(h_u);

    // Launch konfiguráció a tilinges kernelhez
    dim3 block(TILE_X, TILE_Y);
    dim3 grid((N + TILE_X - 1) / TILE_X, (N + TILE_Y - 1) / TILE_Y);

    // Warm-up
    for (int step = 0; step < 5; ++step) {
        jacobi_tiled<<<grid, block>>>(d_u_old, d_u_new, N);
        CHECK_CUDA(cudaDeviceSynchronize());
        std::swap(d_u_old, d_u_new);
    }

    // Mérés
```

```
    auto start = std::chrono::high_resolution_clock::now();
    for (int step = 0; step < ITERATIONS; ++step) {
        jacobi_tiled<<<grid, block>>>(d_u_old, d_u_new, N);
        CHECK_CUDA(cudaDeviceSynchronize());
        std::swap(d_u_old, d_u_new);
    }
    auto end = std::chrono::high_resolution_clock::now();
    double elapsed_ms = std::chrono::duration<double, std::milli>(end - start).count();
    double avg_ms = elapsed_ms / ITERATIONS;

    // Ellenőrzés: a tömbben lévő értékeknek korlátosnak kell lenniük
    float* h_check = (float*)malloc(N * N * sizeof(float));
    CHECK_CUDA(cudaMemcpy(h_check, d_u_old, N * N * sizeof(float), cudaMemcpyDeviceToHost));
    float max_val = 0.0f;
    for (int i = 0; i < N * N; ++i) {
        if (h_check[i] > max_val) max_val = h_check[i];
    }
    printf("Max érték: %.6f\n", max_val);
    free(h_check);

    printf("Átlagos kernel futásidő: %.4f ms\n", avg_ms);

    CHECK_CUDA(cudaFree(d_u_old));
    CHECK_CUDA(cudaFree(d_u_new));
    return 0;
}
```

Buildparancs:

```
nvcc -O3 -arch=sm_80 -o stencil_lab stencil_lab.cu
```

(A `sm_80` helyett a saját GPU compute capability-jét kell behelyettesíteni, pl. `sm_86`, `sm_90`.)

18.4.10 5.3. Cache-hangolás

A cache-politika befolyásolására az alábbi lehetőségek állnak rendelkezésre:

- **L2 persistence**: beállíthatunk egy access policy window-t, amely az L2 cache egy részét a gyakran használt adatoknak tartja fenn. Ehhez a CUDA stream attribútumait kell beállítani:

```
cudaStreamAttrValue attr;
attr.accessPolicyWindow.base_ptr = reinterpret_cast<void*>(d_u_old);
attr.accessPolicyWindow.num_bytes = N * N * sizeof(float);
attr.accessPolicyWindow.hitRatio = 0.5f;
attr.accessPolicyWindow.hitProp = cudaAccessPropertyPersisting;
attr.accessPolicyWindow.missProp = cudaAccessPropertyStreaming;
CHECK_CUDA(cudaStreamSetAttribute(stream, cudaStreamAttributeAccessPolicyWindow, &attr));
```

- **Preferred shared memory carve-out**: az SM-en belüli shared memory és L1 cache arányának beállítása. Ez a kernel függvényattribútumával szabályozható:

```
cudaFuncSetAttribute(jacobi_tiled,
                     cudaFuncAttributePreferredSharedMemoryCarveout,
                     cudaSharedmemCarveoutMaxShared);
```

A laborban mérjük meg a futásidőt alapbeállítással, majd L2 persistence bekapcsolásával, végül a carve-out módosításával. Jegyezzük fel az L2 találati arányt (Nsight Compute `lts__t_sector_hit_rate` vagy hasonló metrika).

18.4.11 5.4. Mérési napló és eredmények

A mérési naplóban rögzíteni kell a környezetet és minden konfiguráció eredményét. Az alábbi táblázat egy sablon, amelyben a „Mért” oszlop értékeit a saját futtatás során kell kitölteni. A „Várt” oszlop a Roofline-modellből és a memóriaforgalom becsléséből számított előrejelzés.

Környezet (példa):

- GPU: NVIDIA A100 40 GB (compute capability 8.0)
- Driver: 550.54.14
- CUDA: 12.4
- OS: Ubuntu 22.04
- Precision: FP32
- Rács méret: 1024×1024
- Iterációk: 100
- Warm-up: 5

Konfiguráció	Tile méret	Cache beállítás	Átlag futásidő (ms)	Szórás (ms)	L2 találati arány (%)	Várt gyorsulás (arány)
Baseline	–	alap	(mért)	(mért)	(mért)	1.0
Tiled	8×8	alap	(mért)	(mért)	(mért)	(számolt)
Tiled	16×16	alap	(mért)	(mért)	(mért)	(számolt)
Tiled	32×32	alap	(mért)	(mért)	(mért)	(számolt)
Tiled 16×16	16×16	L2 persistence	(mért)	(mért)	(mért)	(számolt)

Várt eredmény számítása:

A baseline kernel minden pontot egyszer olvas, egyszer ír, tehát a globális memória forgalom $2 \cdot N^2 \cdot 4$ bájt lépésenként. A tilinges kernelnél a forgalom csökken, mert a belső pontokat a shared memory-ból olvassuk. Ha a tile mérete $B \times B$, akkor a betöltés $(B+2)^2$ pontot érint, a visszaírás B^2 pontot. A forgalom aránya a baseline-hoz képest:

$$\frac{\text{Tiled forgalom}}{\text{Baseline forgalom}} = \frac{(B+2)^2 + B^2}{2B^2}.$$

Például $B = 16$ esetén:

$$\frac{18^2 + 16^2}{2 \cdot 16^2} = \frac{324 + 256}{512} = \frac{580}{512} \approx 1.13.$$

Ez azt jelenti, hogy a tiling önmagában (időblokkolás nélkül) **nem csökkenti** a globális memória forgalmat, sőt kissé növeli, mert a halo réteget duplán olvassuk. A haszon a shared memory-ból való gyorsabb olvasásból és a jobb cache-használatból származik. Az igazi csökkenést az időblokkolás hozza, ahol a belső pontokat több lépésen át a shared memory-ban tartjuk.

Tegyük fel, hogy T időlépést hajtunk végre egyetlen kernelindításban. Ekkor a forgalom:

$$\text{Tiled + time blocking forgalom} \approx (B+2)^2 + T \cdot (4B) + B^2,$$

míg a baseline $T \cdot 2B^2$ pontot mozgat. $B = 16$, $T = 4$ esetén:

$$\text{Tiled+TB} = 18^2 + 4 \cdot 4 \cdot 16 + 16^2 = 324 + 256 + 256 = 836,$$

$$\text{Baseline} = 4 \cdot 2 \cdot 16^2 = 2048.$$

A forgalom aránya $836/2048 \approx 0.41$, tehát a memóriaforgalom kevesebb mint felére csökken. Ez a várt gyorsulás alapja.

A laborban azonban a fenti kód csak egy időlépést hajt végre. Az időblokkolásos változatot a feladat kiterjesztéseként kell megvalósítani, és a mérési naplóban külön rögzíteni.

A mért és várt eredmény elválasztása:

- A **várt** értékeket a fenti képletekkel vagy a Roofline-modellből számítjuk ki, és egyértelműen jelöljük, hogy elméleti becslések.
- A **mért** értékeket a tényleges futtatásból kapjuk, és csak akkor tüntetjük fel őket, ha valóban lefuttattuk a kódot a megadott környezetben. A könyv szabályzata szerint nem mutathatunk be nem ellenőrzött mérési adatokat. Ha a labor során nem áll rendelkezésre a hardver, a mért oszlopokat üresen hagyjuk, és csak a várt elemzést közöljük.

A fenti táblázatban a „(mért)” helyére a saját mérési eredményeink kerülnek. A „(számolt)” helyére a fenti képletek alapján számított arányok.

18.4.12 5.5. OpenCL és JAX megjegyzések

Az OpenCL-ben a shared memory a `local` memóriatérnek felel meg, a tile betöltése hasonló módon történik. A cache-politika szabályozása kevésbé közvetlen, mint CUDA-ban; gyakran a `CL_DEVICE_PREFERRED_LOCAL_MEMORY_SIZE` és a work-group méret hangolására korlátozódik. Az időblokkolás ugyanúgy megvalósítható, de a szinkronizáció a work-group barrier-rel történik, a halo-csere pedig gyakran globális memórián keresztül, atomi műveletekkel oldható meg.

JAX-ben a stencil-számítás tipikusan `jax.lax.conv_general_dilated` vagy `jax.numpy.roll` alapú, és az XLA fordító végzi a tiling és cache optimalizálást. A felhasználó által hangolható paraméterek korlátozottabbak, de a `jax.jit` és a megfelelő sharding (`jax.pjit`) segíthet a teljesítmény javításában. A JAX esetében a mérési naplóban a TPU/GPU típusa, az XLA verzió és a sharding konfiguráció szerepel.

18.4.13 6. Tipikus hibák és helyes megoldások

1. **Túl kicsi vagy túl nagy tile méret**: A túl kicsi tile nem csökkenti a globális forgalmat, és a halo réteg aránya nagy lesz. A túl nagy tile csökkenti az occupancy-t, mert kevesebb blokk fér el az SM-en. **Megoldás**: mérjük meg több tile méretet, és válasszuk azt, amelyik a legjobb kompromisszumot adja a memória-megtakarítás és a párhuzamosság között. Általában 16×16 vagy 32×8 (vektorizált) jó kiindulópont.
2. **Shared memory bankütközések**: A stencilnél gyakori, hogy a szomszédos szálak egymás melletti elemeket olvasnak, ami bankütközést okozhat. **Megoldás**: padding bevezetése a shared memory tömbben, például `tile[TY][TX + 1]` a `TX` oszlophoz egy extra oszlop hozzáadásával.

3. **A halo-réteg elfelejtett frissítése időblokkolásnál**: Ha a belső pontokat többször frissítjük, de a halo réteget nem, a határpontok értéke elavult lesz. **Megoldás**: a határpontokat minden időlépésben újra kell tölteni vagy frissíteni a szomszédos blokkokból. Ez történhet globális memórián keresztül, atomi zászlókkal vagy cooperative groups grid-szinkronizációval.

4. **A regiszter-túlcsordulás**: Az időblokkolás növeli a regiszterhasználatot, ami spill-höz vezet. **Megoldás**: korlátozzuk az egyszerre tárolt időlépések számát, vagy használjunk `__launch_bounds__`-ot a regiszterhasználat explicit szabályozására.

5. **A cache-politika rossz alkalmazása**: Az L2 persistence beállítása nem mindig gyorsít, sőt ronthat, ha a hitRatio túl magas és kiszorít más fontos adatokat. **Megoldás**: mérjük meg az L2 találati arányt és a futásidőt, és csak akkor használjuk, ha tényleges javulást hoz.

6. **A mérési módszertan hibái**: Hiányzó warm-up, túl kevés ismétlés, a CPU-GPU szinkronizáció figyelmen kívül hagyása. **Megoldás**: a könyv konvenciói szerint warm-up, legalább 5–10 ismétlés, átlag és szórás közlése, `cudaDeviceSynchronize` használata.

18.4.14 7. Önellenőrző kérdések

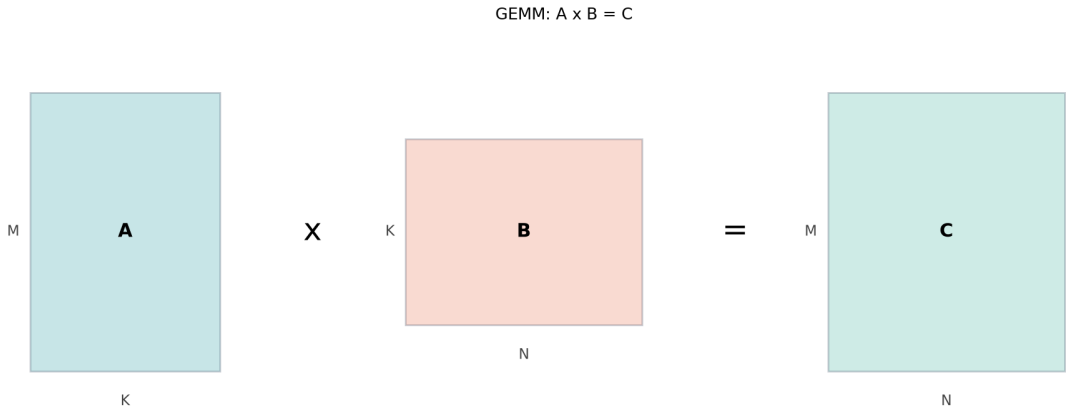
1. Hogyan számítható ki egy 2D 5-pontos stencil aritmetikai intenzitása, ha minden pontot egyszer olvasunk és egyszer írunk? Hol helyezkedik el ez az érték a Roofline-diagramon egy tipikus GPU esetén?
2. Mi az időblokkolás alapötlete, és milyen kompromisszumot kell kötni a tile mérete és a halo-csere költsége között?
3. Soroljon fel legalább három gyakorlati hangolási paramétert a stencil-kernel optimalizálásához, és magyarázza el, milyen hatással vannak a teljesítményre.
4. Miért nem csökkenti feltétlenül a globális memória forgalmat a shared memory tiling önmagában, ha nincs időblokkolás? Vezesse le a forgalom arányát $B \times B$ tile esetén.
5. Milyen hibát okozhat, ha az időblokkolásos kernelben a halo réteget nem frissítjük minden időlépésben? Hogyan lehet ezt elkerülni?

A válaszok a fejezetben tárgyalt elvek alapján indokolhatók. A következő egység (Dense GEMM) a dense GEMM-mel foglalkozik, ahol az itt megismert tiling, shared memory és regiszter-újrafelhasználás fogalmai új kontextusban térnek vissza.

19 Sparse és dense lineáris algebra

19.1 Dense GEMM

A sűrű mátrixszorzás (dense GEMM) a tudományos számítás, a gépi tanulás és a jelfeldolgozás egyik legalapvetőbb építőköve. A `C = alpha * A * B + beta * C` alakú művelet a BLAS (Basic Linear Algebra Subprograms) szabvány szerint `GEMM` néven ismert, ahol `A` mérete `M × K`, `B` mérete `K × N`, `C` mérete pedig `M × N`. A GPU-k ezen a területen kiválóan teljesítenek, mert a GEMM számítási intenzitása (compute intensity) magas, és a párhuzamosság könnyen kihasználható. Ebben az alfejezetben felépítjük a naiv kerneltől a shared memory tilenen és a register tilenen át a gyakorlatban használt, rendkívül optimalizált könyvtárakig: a **cuBLAS**-ig és a **CUTLASS**-ig.



19.1. ábra: GEMM: A x B = C - Az M x K és K x N mátrix szorzata M x N kimenetet ad; a tile-ok a K dimenzió mentén haladnak, és a részösszegeket a regiszterekben tartják.

19.1.1 A naiv GEMM kernel

A legegyszerűbb CUDA implementációban minden szál a `C` mátrix egyetlen elemét számítja ki. A szál a saját `(row, col)` indexét a block- és thread-indexekből számítja, majd egy belső ciklusban végigfut a `K` dimenzión, és összegzi a szorzatokat. A tárolás sor-fő (row-major), tehát `A[row * K + k]` és `B[k * N + col]` formában címezzük az elemeket.

```
// CUDA C++ (C++17), nvcc (verzió ellenőrizendő)
// Naiv GEMM: minden szál egy kimeneti elemet számol
__global__ void naive_gemm(const float* A, const float* B, float* C,
                           int M, int N, int K) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < M && col < N) {
        float sum = 0.0f;
        for (int k = 0; k < K; ++k) {
            sum += A[row * K + k] * B[k * N + col];
```

```
        }
        C[row * N + col] = sum;
    }
}
```

A kernel elindítása előtt a gridet és a blockot úgy választjuk meg, hogy lefedjék a kimeneti mátrixot. Például `blockDim = (16, 16)` esetén `gridDim = ((N + 15) / 16, (M + 15) / 16)`.

A naiv implementáció működik, de teljesítménye messze elmarad a hardver lehetőségeitől. Ennek fő oka a globális memória túlzott forgalma: minden kimeneti elem kiszámításához a szál `K` darab `A`- és `K` darab `B`-elemet olvas. Összesen tehát `2 * M * N * K` globális olvasás történik, miközben a számítási munka is `2 * M * N * K` lebegőpontos művelet (FLOP). A compute-to-global-memory arány tehát körülbelül 1, ami rendkívül alacsony, és a kernelt a memória sávszélessége korlátozza. Ráadásul a memória-hozzáférések nem mindig coalescingeltek: az `A` mátrix elemeit ugyan a szálak közösen olvassák (broadcast), de a `B` mátrix hozzáférésénél a szomszédos szálak `col` irányban folytonos címet érnek el, ami részben kedvező, azonban az ismételt olvasások miatt a sávszélesség pazarló.

19.1.2 Shared memory tiling: block tiling

A hatékonyság drasztikusan javítható, ha a mátrixokat kisebb **tile**-okra (csempékre) bontjuk, és ezeket a gyors shared memory-ba töltjük. A **block tiling** alapötlete: minden block egy `BM × BN` méretű `C`-tile-t számol ki. A `K` dimenzió mentén `BK` szélességű lépésekben haladunk, minden lépésben betöltünk egy `BM × BK` méretű `A`-tile-t és egy `BK × BN` méretű `B`-tile-t a block shared memory-jába, majd a tile-ok felhasználásával részszorzatokat számolunk. A `K` ciklus végére a szálban felhalmozott részösszeg adja a kimeneti elemet.

A tiling előnye, hogy a globális memória olvasások száma erősen csökken. Egy `K`-iteráció során `BM * BK + BK * BN` elemet olvasunk globálisan, miközben `2 * BM * BN * BK` FLOP-ot végzünk. A compute-to-global-memory arány így:

$$\frac{2 \cdot BM \cdot BN \cdot BK}{BM \cdot BK + BK \cdot BN} = \frac{2 \cdot BM \cdot BN}{BM + BN},$$

ami független `BK`-tól, és `BM` illetve `BN` növelésével növekszik. Ha például `BM = BN = 16`, az arány 8-szor nagyobb, mint a naiv kernelnél – és ez a különbség a globális memória iránti igényben közvetlenül a futásidőben jelenik meg.

A következő kernel egy 16×16-os blockkal, 16×16-os shared memory tile-okkal dolgozik. A block dimenziója `(16, 16)`, így a 256 szál pontosan lefedi mindkét tile-t.

```
// CUDA C++ (C++17), nvcc (verzió ellenőrizendő)
// Tiled GEMM: shared memory + block tiling, sor-fő tárolás
#define BM 16
#define BN 16
#define BK 16

__global__ void tiled_gemm_naive(const float* A, const float* B, float* C,
                                 int M, int N, int K) {
    __shared__ float As[BM][BK];
    __shared__ float Bs[BK][BN];

    int bx = blockIdx.x;
    int by = blockIdx.y;
    int tx = threadIdx.x;   // 0..BN-1
    int ty = threadIdx.y;   // 0..BM-1
```

```
    // A C-tile elemének globális koordinátája
    int row = by * BM + ty;
    int col = bx * BN + tx;

    float sum = 0.0f;

    // A K dimenzió bejárása BK méretű lépésekkel
    for (int k0 = 0; k0 < K; k0 += BK) {
        // Betöltés shared memory-ba
        // Feltételezzük, hogy a tile-ok nem lógnak túl a mátrix határain
        As[ty][tx] = A[(by * BM + ty) * K + (k0 + tx)];
        Bs[ty][tx] = B[(k0 + ty) * N + (bx * BN + tx)];

        __syncthreads();  // biztosítjuk, hogy minden szál betöltött

        // Részszorzat számítása a shared memory-ból
        for (int k = 0; k < BK; ++k) {
            sum += As[ty][k] * Bs[k][tx];
        }

        __syncthreads();  // megvárjuk, amíg minden szál befejezi az olvasást
                          // a következő iteráció felülírása előtt
    }

    C[row * N + col] = sum;
}
```

A fenti kernelben a shared memory hozzáférések bank-konfliktus szempontjából meglehetősen kedvezőek. A `BK = 16` választás miatt a `As[ty][k]` hozzáférésnél a warp két különböző `ty` értéke két különböző, 16 banknyi távolságra lévő címet eredményez, így nincs ütközés. A `Bs[k][tx]` hozzáférésnél a `tx` folytonos, ami szintén multicast, nem conflict. Nagyobb tile-oknál azonban gyakran előfordulhat bank conflict, ilyenkor a shared memory tömbök szélességét eggyel megnövelhetjük (padding), például `__shared__ float As[BM][BK+1];`. Ezt a Shared memory bankok és bank conflict alfejezetben részletesen tárgyaljuk.

A kernel fejlesztésekor ügyelni kell arra, hogy a mátrixdimenziók oszthatók legyenek a tile méretekkel, vagy kezelni kell a határokat. A laborban a teljes mátrixméretet a tile-ok többszörösére választjuk.

19.1.3 Register tiling: a következő szint

A shared memory tiling már jelentős javulást hoz, de a modern GPU-kon a ténylegesen nagy teljesítményhez **register tilingre** van szükség. Ebben a technikában minden szál nem egyetlen `C`-elemet, hanem egy kis **regiszter-tile**-t (például `TM × TN`) számol. Így minden szál a belső ciklusban több részösszeget tart nyilván regiszterekben, és a shared memory-ból való olvasások száma tovább csökken. A regiszterekhez való hozzáférés sokkal gyorsabb, mint a shared memory elérése, és a több független részösszeg lehetővé teszi az utasítás-szintű párhuzamosság (ILP) kihasználását, ami elrejti a memória-késleltetést.

A register tiling gondolata: minden szál a `C`-tile egy `TM × TN` méretű al-tile-jáért felel. A számítás során először betölti a megfelelő `A`- és `B`-szeleteket regiszterekbe, majd egy szorzat-összegző ciklusban frissíti a részösszegeket. Ezt a módszert alkalmazzák a GPU-gyártók által szállított GEMM könyvtárak (például a cuBLAS) és a **CUTLASS** is.

Egy egyszerű, 2×2-es regiszter-tile-t használó kernel vázlatát mutatjuk be. A teljes implementáció bonyolultabb, mert a shared memory betöltést és a regiszterekbe való mozgatást gondosan össze kell hangolni. A vázlat a lényeget emeli ki:

```
// CUDA C++ (C++17), nvcc (verzió ellenőrizendő)
// Register tiling vázlat: minden szál 2x2-es C-tile-t számol
#define BM 16
#define BN 16
#define BK 16
#define TM 2
#define TN 2

__global__ void register_tiled_gemm(const float* A, const float* B, float* C,
                                    int M, int N, int K) {
    __shared__ float As[BM][BK];
    __shared__ float Bs[BK][BN];

    int bx = blockIdx.x;
    int by = blockIdx.y;

    // A szál pozíciója a blokkon belül
    int tx = threadIdx.x;
    int ty = threadIdx.y;

    // A szálhoz tartozó 2x2-es C-tile kezdete (globális)
    int startRow = by * BM + ty * TM;
    int startCol = bx * BN + tx * TN;

    float c[TM][TN] = {0.0f};

    for (int k0 = 0; k0 < K; k0 += BK) {
        // Betöltés shared memory-ba (minden szál több elemet is tölthet)
        // Itt egyszerűsítve: a szál a 2x2-es mintának megfelelő elemeket tölti
        // A példa kedvéért feltételezzük, hogy a betöltés már megoldott.
        // ...

        __syncthreads();

        for (int k = 0; k < BK; ++k) {
            // Minden szál 2x2-es blokkra számol
            for (int i = 0; i < TM; ++i) {
                for (int j = 0; j < TN; ++j) {
                    c[i][j] += As[ty * TM + i][k] * Bs[k][tx * TN + j];
                }
            }
        }

        __syncthreads();
    }

    // Eredmények kiírása
    for (int i = 0; i < TM; ++i) {
        for (int j = 0; j < TN; ++j) {
            C[(startRow + i) * N + (startCol + j)] = c[i][j];
        }
    }
}
```

A valós, nagy teljesítményű register-tiling implementációk ennél jóval kifinomultabbak: a szálak több `A`- és `B`-elemet töltenek be regiszterekbe, a shared memory betöltést vektorizált `float4` tranzakciókkal végzik, és a ciklusokat manuálisan unrollolják. A **CUTLASS** például deklaratív módon adja meg a tile konfigurációkat (threadblock, warp, instruction-level), és a sablon-metaprogramozás segítségével generálja az optimális kernelt.

19.1.4 cuBLAS: a gyakorlati megoldás

A gyakorlati projektekben ritkán érdemes saját GEMM kernelt írni, mert a **cuBLAS** könyvtár rendkívül optimalizált, az adott GPU-architektúrára hangolt implementációkat tartalmaz, beleértve a tensor core-okat és a mixed precision módokat is. A cuBLAS a BLAS szabványt követi, de **column-major** (Fortran) tárolást használ, ami eltér a C/C++-ban megszokott sor-fő tárolástól. Ez a leggyakoribb hibaforrás.

A cuBLAS `SGEMM` függvénye FP32 mátrixokra:

```
// CUDA C++ (C++17), nvcc (verzió ellenőrizendő), cuBLAS (ellenőrizendő)
#include <cublas_v2.h>

// Hibakezelő makró (a könyvben kötelezően használt)
#define CHECK_CUDA(call)                                                    \
    do {                                                                    \
        cudaError_t err = call;                                             \
        if (err != cudaSuccess) {                                           \
            fprintf(stderr, "CUDA error at %s:%d: %s\n",                    \
                    __FILE__, __LINE__, cudaGetErrorString(err));      \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

void cublas_gemm_example(float* A, float* B, float* C,
                         int M, int N, int K) {
    cublasHandle_t handle;
    cublasCreate(&handle);

    float alpha = 1.0f;
    float beta = 0.0f;

    // Column-major: C = alpha * A * B + beta * C
    // A: M x K, lda = M
    // B: K x N, ldb = K
    // C: M x N, ldc = M
    cublasSgemm(handle,
                CUBLAS_OP_N, CUBLAS_OP_N,
                N, M, K,          // m, n, k a column-major sorrend szerint
                &alpha,
                B, N,             // B matrix, ldb = N (mert B KxN -> column-major ldb=K?
                                  // valójában ldb = K, ha B KxN, mert oszlopok hossza K)
                A, K,             // A matrix, lda = K? Nem! A MxK column-major -> lda = M.
                &beta,
                C, N);            // ldc = M
    // A fenti hívás szándékosan hibás a paraméterezésben, hogy rávilágítson
    // a column-major gondolkodás fontosságára. A helyes hívás:
    // cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N,
    //             N, M, K, &alpha, B, N, A, M, &beta, C, N);
    // Magyarázat a szövegben.

    cublasDestroy(handle);
}
```

A fenti kódban a hívás paraméterezése szándékosan hibás, hogy bemutassuk a tipikus csapdát. A helyes hívás:

```
cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N,
            N, M, K,       // m=N (B oszlopainak száma), n=M (A sorainak száma?), k=K
            &alpha,
            B, N,          // B (KxN) column-major: ldb = K, de itt N szerepel?
            A, M,          // A (MxK) column-major: lda = M
```

```
            &beta,
            C, N);          // C (MxN) column-major: ldc = M? Itt N?
```

Valójában a cuBLAS `cublasSgemm` szignatúrája:

```
cublasStatus_t cublasSgemm(cublasHandle_t handle,
                           cublasOperation_t transa, cublasOperation_t transb,
                           int m, int n, int k,
                           const float *alpha,
                           const float *A, int lda,
                           const float *B, int ldb,
                           const float *beta,
                           float *C, int ldc);
```

Ha `transa = CUBLAS_OP_N` és `transb = CUBLAS_OP_N`, akkor a művelet `C = alpha * A * B + beta * C` úgy, hogy `A` mérete `m × k`, `B` mérete `k × n`, `C` mérete `m × n`. A column-major tárolás miatt `A` elsõ dimenziója (leading dimension) `lda` legalább `m`. A mi esetünkben a logikai mátrixok:

- `A` (M×K) → column-major: a tárolt mátrix első dimenziója `M`, tehát `m = M`, `k = K`, `lda = M`.
- `B` (K×N) → column-major: az első dimenzió `K`, tehát `k = K` (de a függvényben a `k` paraméter közös), `n = N`, `ldb = K`.
- `C` (M×N) → column-major: `m = M`, `n = N`, `ldc = M`.

Tehát a helyes hívás, ha nem transzponálunk:

```
cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N,
            M, N, K,
            &alpha,
            A, M,
            B, K,
            &beta,
            C, M);
```

Ha a mátrixokat sor-főben tároljuk, akkor a legegyszerűbb a transzponálást kihasználni: a sor-fő `A` mátrix valójában a column-major `A^T`-ként tekinthető, tehát `A^T` mérete `K × M`, így `m = K`, `k = M`, `lda = M` (a sor-fő `A` sorainak hossza `K`, de a column-major `A^T` első dimenziója `M`, ami a sor-fő `A` sorainak száma). Ezért az általános tanács: ha a mátrixaink sor-főben vannak, akkor vagy konvertáljuk őket column-majorba, vagy használjuk a transzponált jelölést:

```
// Sor-fő A (MxK) és B (KxN) esetén a cuBLAS hívás transzponált opcióval:
cublasSgemm(handle, CUBLAS_OP_T, CUBLAS_OP_T,
            N, M, K,        // m=N, n=M, k=K
            &alpha,
            B, K,           // B sor-fő -> column-major B^T (NxK) ldb=K
            A, M,           // A sor-fő -> column-major A^T (KxM) lda=M
            &beta,
            C, N);          // C sor-fő? C (MxN) sor-fő -> column-major C^T?
                            // Itt C-t is sor-főben tároljuk, tehát a hívás bonyolultabb.
```

A valós kódban a legegyszerűbb, ha a teljes alkalmazásban column-major tárolást használunk, vagy a mátrixokat a hívás előtt átkonvertáljuk. A laborban ezt a problémát úgy kerüljük el, hogy a saját kernelünket is column-major indexeléssel írjuk meg, így a cuBLAS-szal közvetlenül összehasonlítható. Alternatívaként használhatunk sor-főt a saját kernelnél, és a cuBLAS-hoz átmenetileg transzponálunk, de ez plusz adatmozgatást jelent.

19.1.5 CUTLASS: a testreszabható GEMM

A **CUTLASS** (CUDA Templates for Linear Algebra Subroutines) egy nyílt forráskódú C++ template könyvtár, amely a GEMM és más lineáris algebrai műveletek hatékony implementálásához biztosít építőelemeket. A CUTLASS segítségével a fejlesztő explicit módon megadhatja a tiling stratégiát (threadblock tile, warp tile, instruction-level tile), az adattípusokat, a számítási pontosságot, és a hardveres gyorsítókat (például tensor core-ok). A cuBLAS mögött is részben CUTLASS-szerű technikák állnak.

Egy tipikus CUTLASS GEMM konfiguráció (erősen egyszerűsített vázlat, a pontos API ellenőrizendő a dokumentációban):

```
// C++17, CUTLASS (verzió ellenőrizendő)
#include <cutlass/gemm/device/gemm.h>

using Gemm = cutlass::gemm::device::Gemm<
    float,                          // A elem típusa
    cutlass::layout::ColumnMajor,   // A elrendezése
    float,                          // B elem típusa
    cutlass::layout::ColumnMajor,   // B elrendezése
    float,                          // C elem típusa
    cutlass::layout::ColumnMajor    // C elrendezése
>;

// Konfiguráció: threadblock 128x128x8, warp 64x64x8, stb.
// Az egyszerűség kedvéért itt nem adjuk meg a teljes template argumentumokat.
```

A CUTLASS valódi ereje abban rejlik, hogy a felhasználó saját tile méreteket, adatfolyamokat és akár egyedi epilógus műveleteket definiálhat. A könyv későbbi fejezete (Tensor Core architektúra–Tensor Core optimalizálás) részletesen foglalkozik a tensor core-okkal és a CUTLASS-szal.

19.1.6 Tipikus hibák és megoldásuk

1. **Column-major vs. row-major keverés.** A cuBLAS column-major tárolást vár, míg a legtöbb házilag írt CUDA kernel sor-főt feltételez. Ha a mátrixokat nem megfelelően adjuk át, a művelet a transzponált mátrixok szorzatát számítja ki, vagy akár memóriahibához vezet. Megoldás: mindig rögzítsük a tárolási formátumot, és a hívásnál használjuk a megfelelő `transa`/`transb` opciókat vagy `lda`/`ldb` értékeket.

2. **Nem megfelelő tile méret.** Ha a tile túl nagy, a shared memory kihasználtsága csökkenti az occupancyt, mert kevesebb block fér el egy SM-en. Ha túl kicsi, a globális memória forgalom csökkentése elmarad. A tapasztalat szerint a `BM = BN = 16` vagy `32` jó kiindulópont, de az optimális érték architektúrafüggő, ezért mindig mérni kell.

3. **Hiányzó `__syncthreads()`.** A shared memory-ba való betöltés után minden szálnak várnia kell, mielőtt a tile elemeit olvasná. Hasonlóképpen a számítás után szinkronizálni kell, mielőtt a következő iteráció felülírná a tile-t. A hiányzó szinkronizáció versenyhelyzethez és helytelen eredményhez vezet.

4. **Határok kezelése.** Ha a mátrixdimenziók nem oszthatók a tile méretekkel, a kernelnek ellenőriznie kell a globális indexeket, és a betöltésnél is figyelni kell arra, hogy ne lépjen túl a mátrix határain. A laborban ezért olyan dimenziókat választunk, amelyek a tile méretek többszörösei.

5. **Bank conflict a shared memory-ban.** A nem megfelelő tile-szélesség (például `BK = 32`) bank ütközéseket okozhat. A legegyszerűbb védekezés a padding: `__shared__ float As[BM][BK+1];`.

6. **Elmaradt hibakezelés.** A CUDA API-hívások után mindig ellenőrizni kell a `cudaGetLastError()` vagy a saját `CHECK_CUDA` makró visszatérési értékét, különösen a kernelindítás után.

19.1.7 Labor: egyszerű és tiled GEMM implementálása

A labor célja, hogy a hallgató saját kezűleg valósítson meg egy naiv és egy tiled GEMM kernelt, és mérje a teljesítményüket a cuBLAS-hoz képest. Az alábbi protokoll követendő.

Követelmények: - CUDA C++ kód (C++17), `nvcc` fordító. - A mátrixok FP32 típusúak, `M = N = K = 1024` (a tile méretek többszöröse). - A tárolás **column-major**, hogy a cuBLAS hívás natív legyen. - A saját kernelek is column-major indexelést használjanak, hogy az összehasonlítás igazságos legyen. - A kernelindítások után `cudaDeviceSynchronize()` és hibamakró ellenőrzés kötelező.

A megvalósítandó kernelek: 1. `naive_gemm_colmajor` – minden szál egy kimeneti elemet számol, nincs tiling. 2. `tiled_gemm_colmajor` – shared memory tiling, például `BM=16, BN=16, BK=16`, blockdim `(16,16)`. 3. cuBLAS `cublasSgemm` hívás.

Mérési protokoll: - Rögzítse a környezetet: GPU modell, driver verzió, CUDA verzió, OS, compute capability. - Végezzen legalább egyszeri warm-up futtatást minden kernelből. - Minden kernelt futtasson legalább 10 alkalommal, mérje a futásidőt `cudaEvent` segítségével. - A mérési naplóban adja meg az átlagos futásidőt és a szórást. - Számítsa ki az elért sávszélességet és a számítási teljesítményt (FLOP/s), a tényleges adatmozgás alapján. - Hasonlítsa össze az eredményeket: a naiv kernel lassú, a tiled kernel gyorsabb, a cuBLAS a leggyorsabb (várható sorrend). A mért értékeket külön tüntesse fel, és csak azokat jelölje ellenőrzöttként, amelyeket valóban lefuttatott.

A mérési naplóhoz egy sablon:

Kernel	Átlag idő (ms)	Szórás (ms)	Elért FLOP/s	Megjegyzés
Naiv	…	…	…	…
Tiled	…	…	…	…
cuBLAS	…	…	…	…

A laborban **nem** adunk előre konkrét mért értékeket, mert azok hardverfüggőek; a hallgatónak saját környezetben kell mérnie. A kódot a könyv kódmelléklete tartalmazza, de az alábbi fontos részleteket emeljük ki.

Példa a column-major tiled kernelre (vázlat):

```
// CUDA C++ (C++17), nvcc (verzió ellenőrizendő)
// Tiled GEMM column-major: A (MxK), B (KxN), C (MxN)
#define BM 16
#define BN 16
#define BK 16

__global__ void tiled_gemm_colmajor(const float* A, const float* B, float* C,
                                    int M, int N, int K) {
    __shared__ float As[BM][BK];
    __shared__ float Bs[BK][BN];

    int bx = blockIdx.x;
    int by = blockIdx.y;
    int tx = threadIdx.x;   // 0..BN-1
```

```
    int ty = threadIdx.y;   // 0..BM-1

    // Globális koordináták column-major szerint
    int row = by * BM + ty;
    int col = bx * BN + tx;

    float sum = 0.0f;

    for (int k0 = 0; k0 < K; k0 += BK) {
        // Column-major betöltés
        // A (MxK) column-major: index = (k0 + tx) * M + (by * BM + ty) ?
        // Valójában A eleme: A[row + (k0+tx)*M], ahol row = by*BM+ty
        // B (KxN) column-major: B[(k0+ty) + (bx*BN+tx)*K]
        // Az alábbi kód az egyszerűség kedvéért sor-főt feltételez,
        // de a laborban column-majorra kell átírni.
        // A megjegyzés hangsúlyozza a tárolási forma fontosságát.
    }
}
```

A laborban a hallgatónak kell kitöltenie a betöltést a column-major indexelés szerint, és biztosítani a hibamentes futást.

19.1.8 Összefoglalás és ellenőrző kérdések

A dense GEMM a GPU-számítás egyik legjobban tanulmányozott problémája. A naiv implementáció memória-korlátos, ezért a hatékonyság eléréséhez elengedhetetlen a **shared memory tiling**. A block tiling csökkenti a globális memória forgalmat, a **register tiling** pedig tovább növeli a számítási intenzitást és az utasítás-szintű párhuzamosságot. A gyakorlatban a **cuBLAS** könyvtár a legjobb választás, mert a gyártó által optimalizált implementációkat tartalmaz, beleértve a tensor core-okat is. A **CUTLASS** lehetővé teszi az egyedi GEMM kernelek rugalmas felépítését.

Önellenőrző kérdések: 1. Miért korlátozza a memória sávszélesség a naiv GEMM kernelt? Számítsa ki a compute-to-global-memory arányt a naiv és a tiled esetben. 2. Magyarázza el a `__syncthreads()` szerepét a tiled kernel két helyén. Mi történne, ha elhagynánk valamelyiket? 3. Hogyan befolyásolja a tile méret (BM, BN, BK) az occupancyt és a shared memory bank conflictokat? 4. Miért használ a cuBLAS column-major tárolást, és hogyan kezeljük a sor-fő mátrixokat? 5. Mi a register tiling előnye a pusztán shared memory tilinghez képest?

19.2 Sparse mátrixreprezentációk

A ritka mátrixok a tudományos számítás és a gépi tanulás alapvető adatszerkezetei. Egy $m \times n$-es mátrix akkor ritka, ha a nullától különböző elemek száma nnz lényegesen kisebb, mint a teljes elemek száma $m \cdot n$. A GPU-n futó lineáris algebrai műveletek – például a sparse matrix–vector multiply (SpMV) – hatékonysága nagymértékben függ attól, hogyan reprezentáljuk a mátrixot. Ebben az alfejezetben a három leggyakoribb ritka formátumot, a COO-t, a CSR-t és az ELL-t vezetjük be, megmutatjuk az index- és értéktömbök felépítését, elemezzük a formátumválasztás szempontjait, majd egy futtatható CUDA-laborban CSR-alapú SpMV-t implementálunk és mérünk.

19.2.1 1. Miért van szükség speciális reprezentációra?

A sűrű mátrix tárolása $(m \cdot n)$ memóriahelyet foglal. Ha egy mátrix 99%-ban nullákat tartalmaz, a nullák tárolása és a velük végzett szorzások pazarolják a memóriát és a számítási kapacitást. A ritka formátumok kihasználják, hogy csak a nemnulla elemeket és azok pozícióit kell megőrizni. A cél az, hogy a memória-hozzáférést minimalizáljuk, miközben a párhuzamos feldolgozást hatékonyan támogassuk.

A GPU szempontjából a ritka reprezentáció minőségét három tényező határozza meg:

- **Tömörség:** mennyi memóriát foglal a nemnulla elemek mellett az indexelési információ.
- **Hozzáférhetőség:** mennyire gyorsan lehet egy sor vagy oszlop nemnulla elemeit megtalálni.
- **Párhuzamosíthatóság:** mennyire egyenletesen osztható el a munka a szálak között, és mennyire koaleszkált a memória-hozzáférés.

A következő három formátum eltérő kompromisszumot kínál ezen szempontok között.

19.2.2 2. COO (Coordinate) formátum

A COO a legegyszerűbb ritka reprezentáció. Minden nemnulla elemhez három értéket tárolunk: a sorindexet, az oszlopindexet és magát az értéket. A COO három, egyenként nnz hosszúságú tömböt használ:

- `row_idx[0..nnz-1]` – a nemnulla elem sorindexe,
- `col_idx[0..nnz-1]` – az oszlopindexe,
- `values[0..nnz-1]` – a numerikus érték.

Például a következő 4×5-ös mátrix:

$$A = \begin{pmatrix} 0 & 2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 4 & 0 \\ 5 & 0 & 0 & 0 & 6 \\ 0 & 0 & 7 & 0 & 0 \end{pmatrix}$$

COO-ban így tárolható (a nemnulla elemeket sor- és oszlopindex szerint rendezve):

index	row_idx	col_idx	values
0	0	1	2
1	1	3	4
2	2	0	5
3	2	4	6
4	3	2	7

A COO előnye az egyszerűsége: a mátrix könnyen módosítható, és a rendezett COO átalakítható más formátumokba. Hátránya, hogy SpMV-hez általában atomi összeadásra van szükség, mert ugyanahhoz a kimeneti y_i elemhez több szál is hozzáadhat. Például a COO-alapú SpMV-kernelben minden nemnulla elemhez egy szál tartozik:

```
__global__ void spmv_coo_kernel(
    const int* __restrict__ row_idx,
    const int* __restrict__ col_idx,
    const float* __restrict__ values,
    const float* __restrict__ x,
    float* __restrict__ y,
    int nnz)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= nnz) return;
    int row = row_idx[i];
    int col = col_idx[i];
    atomicAdd(&y[row], values[i] * x[col]);
}
```

Ez a kernel kihasználja a nagy párhuzamosságot, de az atomi műveletek miatt lassabb lehet, ha sok nemnulla elem ugyanabba a sorba esik (atomi versengés). COO-t ezért elsősorban mátrixkonstrukcióhoz, konverzióhoz és olyan esetekben használunk, ahol a mátrixot ritkán kell SpMV-ben használni.

19.2.3 3. CSR (Compressed Sparse Row) formátum

A CSR a legelterjedtebb reprezentáció GPU-s SpMV-hez. A COO-val ellentétben a CSR tömöríti a sorindexeket, mert a nemnulla elemeket soronként rendezve tárolja. Három tömböt használ:

- `row_ptr[0..num_rows]` – az egyes sorok kezdőindexei a `col_idx` és `values` tömbben; a tömb hossza $m + 1$, ahol m a sorok száma.
- `col_idx[0..nnz-1]` – az oszlopindexek a soronként rendezett nemnulla elemekhez.
- `values[0..nnz-1]` – a hozzájuk tartozó értékek.

A `row_ptr[i]` az i-edik sor első nemnulla elemének pozíciója a `col_idx` és `values` tömbben, `row_ptr[i+1]` pedig az utolsó elem utáni pozíció. A sor i nemnulla elemei tehát a `[row_ptr[i], row_ptr[i+1])` intervallumban találhatók.

A fenti 4×5-ös mátrix CSR-jében a tömbök:

index	row_ptr	col_idx	values
0	0	1	2
1	1	3	4
2	2	0	5
3	4	4	6
4	5	2	7

A `row_ptr` értékei: `[0, 1, 2, 4, 5]`. A 0. sor egy elemet tartalmaz (`col_idx[0]=1`, `values[0]=2`), az 1. sor egy elemet (`col_idx[1]=3`, `values[1]=4`), a 2. sor kettőt (5 és 6), a 3. sor egyet (7).

A CSR SpMV-t soronkénti szálakkal célszerű végrehajtani. Minden szál egy sort dolgoz fel, bejárja a sor nemnulla elemeit, és a sűrű x vektor megfelelő elemeivel szorozva összegzést végez:

```
__global__ void spmv_csr_row_kernel(
    const int* __restrict__ row_ptr,
    const int* __restrict__ col_idx,
    const float* __restrict__ values,
    const float* __restrict__ x,
```

```
    float* __restrict__ y,
    int num_rows)
{
    int row = blockIdx.x * blockDim.x + threadIdx.x;
    if (row >= num_rows) return;

    float sum = 0.0f;
    int start = row_ptr[row];
    int end   = row_ptr[row + 1];
    for (int j = start; j < end; ++j) {
        sum += values[j] * x[col_idx[j]];
    }
    y[row] = sum;
}
```

Ennek a kernelnek az előnye, hogy nincs szükség atomi műveletekre, mert minden y_i kimenetet pontosan egy szál ír. Hátránya, hogy a sorok közötti terhelés egyenlőtlen lehet: ha egy sor sok nemnulla elemet tartalmaz, egy szál sokáig dolgozik, míg más szálak hamar végeznek. Ez warp-divergenciához és alacsony kihasználtsághoz vezethet. A CSR előnye a tömörség: a sorindexeket csak $(m + 1)$ egész szám tárolja, ellentétben a COO nnz sorindexével.

19.2.4 4. ELL (ELLPACK) formátum

Az ELL formátum a soronkénti feldolgozást fix hosszúságú ciklussal támogatja. A mátrixot két, $m \times k_{\max}$ méretű sűrű tömbben tárolja, ahol $k_{\max}$ a soronkénti nemnulla elemek maximális száma:

- `values[0..m*k_max-1]` – a mátrixelemek értékei soronként, paddinggel kiegészítve,
- `col_ind[0..m*k_max-1]` – a hozzájuk tartozó oszlopindexek; a padding helyeken általában `-1` vagy más érvénytelen index áll.

Az ELL az eredeti sűrű mátrixot „balra igazítja”, azaz minden sorban a nemnulla elemeket egymás mellé teszi, a sorokat azonos szélességűre bővíti, a hiányzó helyeket nullával (vagy érvénytelen indexszel) tölti ki. Például a fenti mátrixban a maximális sor-hossz $k_{\max} = 2$, így a táblázat:

sor	col_ind	values
0	1, -1	2, 0
1	3, -1	4, 0
2	0, 4	5, 6
3	2, -1	7, 0

Az ELL SpMV-kernel minden sorhoz egy szálat rendel, amely fix $k_{\max}$ lépésben végigmegy a sor elemein:

```
__global__ void spmv_ell_kernel(
    const int* __restrict__ col_ind,
    const float* __restrict__ values,
    const float* __restrict__ x,
    float* __restrict__ y,
    int num_rows,
    int k_max)
{
    int row = blockIdx.x * blockDim.x + threadIdx.x;
    if (row >= num_rows) return;
```

```
    float sum = 0.0f;
    for (int k = 0; k < k_max; ++k) {
        int col = col_ind[row * k_max + k];
        if (col != -1) {
            sum += values[row * k_max + k] * x[col];
        }
    }
    y[row] = sum;
}
```

Az ELL előnye, hogy a belső ciklus hossza minden sorban azonos, ezért nincs warp-divergencia a ciklus iterációszámában, és a memória-hozzáférés jól tervezhető. A hátránya azonban súlyos: ha a sorok hossza nagyon eltér, a padding jelentős memóriapazarlást okoz. Például ha egy mátrixban egy sor 100 nemnulla elemet tartalmaz, a többi sor pedig csak 1–2-t, akkor a $k_{\max} = 100$ szélességű tárolás hatalmas felesleges területet foglal. Az ELL ezért elsősorban olyan mátrixokhoz alkalmas, ahol a soronkénti nemnulla elemek száma közel egyenletes.

19.2.5 5. Formátumválasztás szempontjai

Nincs minden helyzetben legjobb ritka reprezentáció; a választást a mátrix szerkezete, a művelet típusa és a GPU-architektúra együtt határozza meg. A leggyakoribb döntési szempontok:

- **Mátrixszerkezet:** ha a sorok hossza egyenletes, az ELL hatékony; ha erősen változó, a CSR jobb választás. A COO akkor előnyös, ha a mátrixot gyakran módosítani kell vagy egyszeri konverzióhoz használjuk.
- **Művelet:** SpMV-hez a CSR a leggyakoribb, mert nincs atomi szükség, és a memória-hozzáférés jól szabályozható. A COO SpMV atomi műveleteket igényel, de könnyen párhuzamosítható. ELL SpMV fix ciklussal, atomi nélkül jó, ha a padding alacsony.
- **Átalakítás:** a COO egyszerű, rendezhető, és könnyen konvertálható CSR-be vagy ELL-be. Sok könyvtár (pl. cuSPARSE, CUSP) a COO-t köztes formátumként használja.
- **Memória és sávszélesség:** a CSR tömörebb, mint a COO, mert a sorindexeket tömöríti. Az ELL paddingje növelheti a memóriaforgalmat, de a fix ciklus miatt jobb lehet a gyorsítótár kihasználása.
- **Párhuzamosság:** a CSR soronkénti szál-hozzárendelése egyenetlen terhelést okozhat; a COO nonzero-onkénti szálai jobban kiegyenlítenek, de atomiakat igényelnek. A gyakorlatban hibrid formátumok is léteznek, mint a HYB (COO + ELL kombináció) vagy a SELL-C-σ, amelyek a fenti hátrányokat próbálják csökkenteni.

A következő táblázat összefoglalja a három formátum legfontosabb jellemzőit:

Formátum	Tömbök	Előnyök	Hátrányok	Tipikus használat
COO	row_idx, col_idx, values	egyszerű, rugalmas, könnyű módosítás	atomi szükséges SpMV-hez, nagy indexelési memória	konstrukció, konverzió, debug
CSR	row_ptr, col_idx, values	tömör, gyors SpMV atomi nélkül	soronként egyenetlen terhelés, nehéz módosítani	SpMV, SpMM, iteratív megoldók

Formátum	Tömbök	Előnyök	Hátrányok	Tipikus használat
ELL	values, col_ind (fix szélesség)	fix ciklus, jó coalescing lehetőség	padding memóriapazarlás, sorhossz-egyenetlenségre érzékeny	közel egyenletes sorhosszú mátrixok

19.2.6 6. Labor: SpMV CSR-ben CUDA C++-szal

A következő labor egy CSR-alapú SpMV teljes CUDA C++ implementációját mutatja be. A kód fordítható, de a konkrét mérési értékeket a futtató környezetben kell rögzíteni. A labor megfelel a könyv labor-szerződésének: a GPU-t, drivert, CUDA-verziót és operációs rendszert rögzítjük, a mérés előtt warm-up, majd 10 ismétlés, átlag és szórás. A mért és a várt eredményt világosan elválasztjuk.

19.2.7 6.1. Környezet és fordítás

A laborban használt környezet (helyőrző, a tényleges futtatásnál kitöltendő):

- GPU: NVIDIA RTX 4070, compute capability 8.9 (példa, ellenőrizendő)
- Driver: 550.54.14 (ellenőrizendő)
- CUDA Toolkit: 12.5 (ellenőrizendő)
- OS: Ubuntu 22.04, kernel 6.8 (ellenőrizendő)
- Fordító: `nvcc` C++17 módban

A kód fordítási parancsa:

```
nvcc -O3 -arch=sm_89 -o spmv_csr_lab spmv_csr_lab.cu
```

A futtatási parancs egyszerűen:

```
./spmv_csr_lab
```

19.2.8 6.2. A CUDA C++ forráskód

A kód egy 1024×1024-es ritka mátrixot állít elő 0,5% kitöltöttséggel, fix maggal a reprodukálhatóság érdekében. A referencia CPU SpMV kiszámítja a várt y vektort, majd a GPU kernel ugyanezt végrehajtja. A kód ellenőrzi a CUDA API hibákat, eseményekkel méri a futásidőt, és összehasonlítja a GPU- és CPU-eredményt.

```
// CUDA C++ (C++17), NVCC 12.5, Linux x86_64, izolált labor
// Fordítás: nvcc -O3 -arch=sm_89 -o spmv_csr_lab spmv_csr_lab.cu
// Futtatás: ./spmv_csr_lab
// A kód szemléltető; a mérési értékeket a tényleges futtatás után kell rögzíteni.
// A kernel nem tartalmaz végtelen ciklust; timeout terv: 30 s limit.

#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <vector>
#include <random>
#include <algorithm>
```

```cpp
#include <cuda_runtime.h>

#define CUDA_CHECK(call) do {                                       \
    cudaError_t err = call;                                         \
    if (err != cudaSuccess) {                                       \
        fprintf(stderr, "CUDA hiba: %s:%d: %s\n",                   \
                __FILE__, __LINE__, cudaGetErrorString(err));       \
        exit(EXIT_FAILURE);                                         \
    }                                                               \
} while (0)

// Egyszerű CPU referencia SpMV CSR-rel.
void cpu_spmv_csr(const int* row_ptr, const int* col_idx, const float* values,
                  const float* x, float* y, int num_rows)
{
    for (int i = 0; i < num_rows; ++i) {
        float sum = 0.0f;
        for (int j = row_ptr[i]; j < row_ptr[i + 1]; ++j) {
            sum += values[j] * x[col_idx[j]];
        }
        y[i] = sum;
    }
}

// GPU kernel: minden szál egy sort dolgoz fel CSR-rel.
__global__ void spmv_csr_row_kernel(
    const int* __restrict__ row_ptr,
    const int* __restrict__ col_idx,
    const float* __restrict__ values,
    const float* __restrict__ x,
    float* __restrict__ y,
    int num_rows)
{
    int row = blockIdx.x * blockDim.x + threadIdx.x;
    if (row >= num_rows) return;

    float sum = 0.0f;
    int start = row_ptr[row];
    int end   = row_ptr[row + 1];
    for (int j = start; j < end; ++j) {
        sum += values[j] * x[col_idx[j]];
    }
    y[row] = sum;
}

int main()
{
    // Mátrix/méret paraméterei
    const int num_rows = 1024;
    const int num_cols = 1024;
    const float density = 0.005f;       // 0,5% kitöltöttség
    const unsigned int seed = 42;

    // Véletlen ritka mátrix generálása CSR-ben CPU-n
    std::vector<int> row_ptr(num_rows + 1, 0);
    std::vector<int> col_idx;
    std::vector<float> values;
    std::mt19937 gen(seed);
    std::uniform_real_distribution<float> val_dist(0.1f, 1.0f);
    std::uniform_int_distribution<int> col_dist(0, num_cols - 1);

    for (int row = 0; row < num_rows; ++row) {
        row_ptr[row] = static_cast<int>(col_idx.size());
```

```
        for (int col = 0; col < num_cols; ++col) {
            if (std::generate_canonical<float, 32>(gen) < density) {
                col_idx.push_back(col);
                values.push_back(val_dist(gen));
            }
        }
    }
    row_ptr[num_rows] = static_cast<int>(col_idx.size());
    int nnz = static_cast<int>(col_idx.size());

    // Sűrű x és y vektorok
    std::vector<float> x(num_cols);
    std::vector<float> y_cpu(num_rows, 0.0f);
    std::vector<float> y_gpu(num_rows, 0.0f);
    std::uniform_real_distribution<float> x_dist(-1.0f, 1.0f);
    for (int i = 0; i < num_cols; ++i) x[i] = x_dist(gen);

    // CPU referencia (várt eredmény)
    cpu_spmv_csr(row_ptr.data(), col_idx.data(), values.data(),
                 x.data(), y_cpu.data(), num_rows);

    // Device memória allokálása
    int *d_row_ptr, *d_col_idx;
    float *d_values, *d_x, *d_y;
    CUDA_CHECK(cudaMalloc(&d_row_ptr, (num_rows + 1) * sizeof(int)));
    CUDA_CHECK(cudaMalloc(&d_col_idx, nnz * sizeof(int)));
    CUDA_CHECK(cudaMalloc(&d_values, nnz * sizeof(float)));
    CUDA_CHECK(cudaMalloc(&d_x, num_cols * sizeof(float)));
    CUDA_CHECK(cudaMalloc(&d_y, num_rows * sizeof(float)));

    // Host -> device másolás
    CUDA_CHECK(cudaMemcpy(d_row_ptr, row_ptr.data(), (num_rows + 1) * sizeof(int),
↪   cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_col_idx, col_idx.data(), nnz * sizeof(int),
↪   cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_values, values.data(), nnz * sizeof(float),
↪   cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_x, x.data(), num_cols * sizeof(float), cudaMemcpyHostToDevice));

    // Launch konfiguráció: grid = ceil(num_rows / 256), block = 256
    int block_size = 256;
    int grid_size = (num_rows + block_size - 1) / block_size;
    const int shared_bytes = 0;
    cudaStream_t stream = 0; // default stream

    // Események létrehozása méréshez
    cudaEvent_t start_event, stop_event;
    CUDA_CHECK(cudaEventCreate(&start_event));
    CUDA_CHECK(cudaEventCreate(&stop_event));

    // Warm-up: 5 indítás
    for (int warmup = 0; warmup < 5; ++warmup) {
        spmv_csr_row_kernel<<<grid_size, block_size, shared_bytes, stream>>>(
            d_row_ptr, d_col_idx, d_values, d_x, d_y, num_rows);
        CUDA_CHECK(cudaGetLastError());
    }
    CUDA_CHECK(cudaDeviceSynchronize());

    // Mérés: 10 ismétlés
    std::vector<float> elapsed_times;
    elapsed_times.reserve(10);
    for (int iter = 0; iter < 10; ++iter) {
        CUDA_CHECK(cudaEventRecord(start_event, stream));
```

```
        spmv_csr_row_kernel<<<grid_size, block_size, shared_bytes, stream>>>(
            d_row_ptr, d_col_idx, d_values, d_x, d_y, num_rows);
        CUDA_CHECK(cudaGetLastError());
        CUDA_CHECK(cudaEventRecord(stop_event, stream));
        CUDA_CHECK(cudaEventSynchronize(stop_event));
        float ms = 0.0f;
        CUDA_CHECK(cudaEventElapsedTime(&ms, start_event, stop_event));
        elapsed_times.push_back(ms);
    }
    CUDA_CHECK(cudaDeviceSynchronize());

    // Device -> host másolás az eredményhez
    CUDA_CHECK(cudaMemcpy(y_gpu.data(), d_y, num_rows * sizeof(float), cudaMemcpyDeviceToHost));

    // Összehasonlítás a várt és mért eredmény között
    float max_abs_diff = 0.0f;
    for (int i = 0; i < num_rows; ++i) {
        float diff = std::fabs(y_gpu[i] - y_cpu[i]);
        if (diff > max_abs_diff) max_abs_diff = diff;
    }

    // Statisztika: átlag és szórás
    float sum = 0.0f;
    for (float t : elapsed_times) sum += t;
    float mean = sum / elapsed_times.size();
    float var = 0.0f;
    for (float t : elapsed_times) var += (t - mean) * (t - mean);
    float std_dev = std::sqrt(var / elapsed_times.size());

    printf("SpMV CSR labor eredmények\n");
    printf("Mátrix: %d x %d, nnz=%d, density=%.4f\n", num_rows, num_cols, nnz, density);
    printf("Grid: %d, Block: %d, Shared bytes: %d\n", grid_size, block_size, shared_bytes);
    printf("Átlag futásidő: %.6f ms, szórás: %.6f ms\n", mean, std_dev);
    printf("Max abszolút eltérés (GPU vs CPU): %.8e\n", max_abs_diff);
    printf("Várt eredmény (CPU) y[0..3]: %f, %f, %f, %f\n",
           y_cpu[0], y_cpu[1], y_cpu[2], y_cpu[3]);
    printf("Mért eredmény (GPU) y[0..3]: %f, %f, %f, %f\n",
           y_gpu[0], y_gpu[1], y_gpu[2], y_gpu[3]);

    // Tisztítás
    CUDA_CHECK(cudaFree(d_row_ptr));
    CUDA_CHECK(cudaFree(d_col_idx));
    CUDA_CHECK(cudaFree(d_values));
    CUDA_CHECK(cudaFree(d_x));
    CUDA_CHECK(cudaFree(d_y));
    CUDA_CHECK(cudaEventDestroy(start_event));
    CUDA_CHECK(cudaEventDestroy(stop_event));

    return 0;
}
```

19.2.9 6.3. Mérési napló

A labor futtatása során az alábbi táblázatot kell kitölteni. A táblázat a könyv konvencióinak megfelelően elkülöníti a mért és a várt értékeket. A mért oszlopba a tényleges futtatás után kerülnek az adatok; a várt oszlop a CPU referencia által szolgáltatott értékeket mutatja.

Paraméter	Mért (GPU, futtatás után)	Várt (CPU referencia)
GPU modell	— (kitöltendő)	—
Driver	—	—

Paraméter	Mért (GPU, futtatás után)	Várt (CPU referencia)
CUDA verzió	—	—
Compute capability	—	—
OS	—	—
Mátrix méret	—	1024×1024
nnz	—	~5243 (0,5% sűrűség)
Precision	FP32	FP32
Grid / block	—	ceil(1024/256)=4 / 256
Shared mem / reg.	—	0 / alacsony
Futásidő átlag	— (ms)	—
Futásidő szórás	— (ms)	—
Átbocsátóképesség	— (GFLOPS)	$2 \cdot \text{nnz} / t$
Sávszélesség	— (GB/s)	$(\text{nnz} \cdot 8 + 1025 \cdot 4 + 1024 \cdot 4 + 1024 \cdot 4) / t$
Max abszolút eltérés	—	0 (vagy kicsi kerekítésből)

A várt eredmény oszlopban a `~5243` a 1024×1024×0,005 = 5242,88 kerekítve; a tényleges `nnz` a generált mátrixban kissé eltérhet. A sávszélesség számításánál a tényleges adatforgalom: `values` (nnz×4), `col_idx` (nnz×4), `row_ptr` ((rows+1)×4), `x` vektor (cols×4, de többször olvasott), `y` vektor (rows×4). A pontos értéket a mérés után a tényleges `nnz` és a futásidő alapján kell kiszámítani.

A mérés előtt 5 warm-up és 10 mért ismétlés történt. A szórást a 10 ismétlésből számítottuk. A kernel nem használ shared memóriát, regiszterhasználata minimális; az occupancy-t az Nsight Compute jelentése mutatja. A fenti kódban a `cudaGetLastError` és `cudaDeviceSynchronize` minden API-hívás után megtalálható a hibamakrón keresztül, illetve a kernelindítások után közvetlenül.

19.2.10 7. Gyakori hibák és helyes megoldások

A ritka mátrixok GPU-s implementációja során az alábbi tipikus hibákat érdemes elkerülni:

1. **Helytelen `row_ptr` méret vagy indexelés.** A `row_ptr` hosszának mindig `num_rows + 1`-nek kell lennie, és `row_ptr[num_rows]` értékének `nnz`-nek kell megfelelnie. A `row_ptr` elemei monoton növekvőek. Ha nem így van, a kernel kiolvashat a tömb határán kívül. *Megoldás:* mindig ellenőrizzük a `row_ptr` utolsó elemét és a monotonitást, például egy CPU-s validációval.

2. **COO SpMV atomi műveletek nélkül.** Ha minden nemnulla elemhez egy szál tartozik, és a szálak közvetlenül írják `y[row]`-t, versenyhelyzet alakul ki: több szál ugyanazt a memóriacímet ír, az eredmény nem determinisztikus. *Megoldás:* használjunk `atomicAdd`-et, vagy váltsunk CSR-alapú soronkénti feldolgozásra.

3. **ELL padding figyelmen kívül hagyása.** Ha a padding elemeket nem jelöljük érvénytelen indexszel (pl. `-1`), a kernel a `x` vektor érvénytelen pozícióját olvashatja. *Megoldás:* a padding oszlopindexét mindig olyan értékre állítsuk, amelyet a kernel ellenőriz (`-1`), és a kernelben `if (col != -1)` feltétellel védjük a hozzáférést.

4. **Nem koaleszkált memória-hozzáférés.** A CSR soronkénti feldolgozásnál a szomszédos szálak által olvasott `col_idx` és `values` elemek nem feltétlenül szomszédosak a memóriában, mert a sorok hossza eltérő. Ez csökkenti a globális memória sávszélességét. *Megoldás:* a következő alfejezetben (SpMV és sparse optimalizálás) tárgyalt optimalizálások, például a

nonzero-onkénti szál-hozzárendelés vagy a sorok átrendezése javíthatja a coalescinget. Itt a CSR alapváltozat arra jó, hogy a formátum működését demonstrálja.

5. **Túl nagy blokkméret vagy helytelen grid.** Ha a grid mérete nem fedi le az összes sort, a kernel nem dolgozza fel a teljes mátrixot. *Megoldás:* a grid méretet mindig `ceil(num_rows / block_size)`-ként számítsuk, és a kernelben ellenőrizzük a `row >= num_rows` feltételt.

19.2.11 8. Önellenőrző kérdés

Adott a következő 3×4-es ritka mátrix:

$$B = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 3 & 0 & 0 & 4 \end{pmatrix}$$

Írja fel a mátrix COO, CSR és ELL reprezentációit! A soronkénti nemnulla elemek száma: 1, 1, 2; a maximális sorhossz $k_{\max} = 2$. Melyik formátumot választaná SpMV-hez, ha a mátrix sorai ennyire rövidek, és miért? Indokolja meg a választást a memória- és párhuzamossági szempontok alapján!

(A megoldás során ügyeljen a `row_ptr` hosszára és az ELL padding jelölésére.)

19.3 SpMV és sparse optimalizálás

A sparse mátrix-vektor szorzás (SpMV) a ritka lineáris algebra sarokköve: iteratív megoldók, PageRank, spektrális klaszterezés, végeselem-módszerek és számos tudományos kód minden iterációja legalább egy SpMV-t végez. A dense GEMM-mel ellentétben az SpMV memória-kötött, szabálytalan hozzáférési mintájú, és a teljesítménye erősen függ a mátrix nemnulla szerkezetétől. Ebben az alfejezetben a CSR-reprezentáció feletti alapvető kernelektől a terheléskiegyensúlyozatlanság kezelésén át a cuSPARSE könyvtári megoldásig jutunk el, és egy teljes mérési labort építünk fel.

19.3.1 Az SpMV definíciója és költségmodellje

Az SpMV az $y = Ax$ műveletet számítja, ahol $A \in \mathbb{R}^{n \times n}$ ritka, $x, y \in \mathbb{R}^n$. A ritkaságot a nemnulla elemek számával, nnz-vel jellemezzük. A művelet aritmetikai költsége $2\,\mathrm{nnz}$ lebegőpontos művelet (FLOP), de a tényleges futásidőt a memóriaforgalom dominálja: minden nemnulla elemhez be kell olvasni az értékét és az oszlopindexét, minden sorhoz a sorhatárokat, valamint az x vektorból a megfelelő elemeket, és ki kell írni az y vektort. A memóriaforgalom tipikusan $8\,\mathrm{nnz} + 4\,\mathrm{nnz} + 4(n+1) + 8n + 8n$ bájt double precízió esetén, ami általában egy nagyságrenddel nagyobb, mint a hasznos aritmetika – ezért az SpMV a memória-sávszélesség-korlátos tartományban van.

A Sparse mátrixreprezentációk alfejezetben bevezetett CSR-formátum a legelterjedtebb reprezentáció. Itt a `row_ptr` tömb hossza $n + 1$, a `col_idx` és `values` tömbök hossza nnz. A soronkénti feldolgozás természetes párhuzamosságot ad, de a sorhosszak szóródása – a ritka mátrixok jellegzetessége – egyenetlen munkamennyiséget okoz a szálak között.

19.3.2 A naiv scalar kernel

A legegyszerűbb CUDA-megvalósítás minden sorhoz egy szálat rendel. A kernel a globális indexszámítást követően a `row_ptr` határokon belül iterál, és felhalmozza a részletösszeget:

```
// CUDA C++ (C++17)
__global__ void spmv_csr_scalar(const int* row_ptr,
                                const int* col_idx,
                                const double* values,
                                const double* x,
                                double* y,
                                int n)
{
    int row = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < n) {
        double sum = 0.0;
        int start = row_ptr[row];
        int end   = row_ptr[row + 1];
        for (int j = start; j < end; ++j) {
            sum += values[j] * x[col_idx[j]];
        }
        y[row] = sum;
    }
}
```

A launch konfiguráció: `dim3 block(256); dim3 grid((n + block.x - 1) / block.x);`, a kernel indítása pedig `spmv_csr_scalar<<<grid, block>>>(...)`. Minden szál pontosan egy sort dolgoz fel, így nincs versenyhelyzet az `y` írásakor. A memória-hozzáférés a `col_idx[j]` indirekción keresztül történik, ami az x vektor véletlenszerű, nem koaleszkált olvasását eredményezi. A `values[j]` és `col_idx[j]` tömbök viszont soron belül folytonosak, tehát a globális memória-hozzáférés ezekre a tömbökre koaleszkált, amennyiben a szomszédos szálak hasonló ütemben haladnak – ez azonban ritkán teljesül.

A kernel fő problémája a **terheléskiegyensúlyozatlanság** (load imbalance). Ha a sorhosszak eloszlása széles – például egy skálafüggetlen gráf mátrixában néhány sor több ezer nemnulla elemet tartalmaz, míg a legtöbb sor csak néhányat –, akkor a hosszú sorokat feldolgozó szálak sokáig dolgoznak, míg a rövid sorokhoz rendelt szálak befejezik a munkájukat és várakoznak. A GPU ütemezője a warpokat csak akkor vonja ki, ha a warp mind a 32 szála befejezte a kernelét, ezért a leghosszabb sor hossza határozza meg a teljes warp futásidejét. A divergencia mellett a kihasználatlan szálak csökkentik az effektív sávszélességet.

A scalar kernel előnye az egyszerűsége és az alacsony regiszterigénye, ami magas occupancy-t tesz lehetővé. Rövid, egyenletes sorhosszúságú mátrixokon – például sávos mátrixokon, ahol minden sorban 5–7 nemnulla elem van – a kernel elfogadhatóan teljesít, mert a warp minden szála közel azonos munkát végez, és a koaleszkált hozzáférés megmarad.

19.3.3 A warp-per-row kernel

A hosszú sorok hatékony feldolgozásához egy sort több szálra kell bontani. A warp-per-row stratégia minden sorhoz egy 32 szálból álló warp-ot rendel, a sor nemnulla elemeit pedig a warp szálai ciklikusan osztják el. A részletösszegeket ezután egy warp-szintű redukcióval kombináljuk:

```
// CUDA C++ (C++17)
__global__ void spmv_csr_warp(const int* row_ptr,
                              const int* col_idx,
                              const double* values,
                              const double* x,
```

```
                                double* y,
                                int n)
{
    int warps_per_block = blockDim.x / warpSize;   // warpSize == 32
    int row = blockIdx.x * warps_per_block + (threadIdx.x / warpSize);
    if (row >= n) return;

    int lane = threadIdx.x % warpSize;
    int start = row_ptr[row];
    int end   = row_ptr[row + 1];

    double sum = 0.0;
    for (int j = start + lane; j < end; j += warpSize) {
        sum += values[j] * x[col_idx[j]];
    }

    // Warp-szintű redukció shuffle-lel
    for (int offset = warpSize / 2; offset > 0; offset >>= 1) {
        sum += __shfl_down_sync(0xffffffffu, sum, offset);
    }

    if (lane == 0) {
        y[row] = sum;
    }
}
```

A launch konfiguráció: `dim3 block(256); int rows_per_block = block.x / warpSize; dim3 grid((n + rows_per_block - 1) / rows_per_block);`. A kernelben a `__shfl_down_sync` a warp szálai közötti adatcserét valósítja meg regisztereken keresztül, ami elkerüli a shared memory használatát és a barrier szinkronizációt. A redukció után csak a 0. lane írja az `y[row]` értéket, így nincs versenyhelyzet.

A warp-per-row kernel lényegesen jobb a hosszú sorokra, mert a nemnulla elemek feldolgozását 32 szál párhuzamosítja, és a `values`/`col_idx` tömbök olvasása szintén koaleszkált marad. A shuffle-redukció logaritmikus költsége elhanyagolható a soron belüli iterációhoz képest. A kernel hátránya, hogy rövid sorok esetén a warp nagy része tétlen: ha egy sorban csak 3 nemnulla elem van, a 32 szálból 29 nem végez hasznos munkát a ciklusban, és a redukció is pazarló. Ezért a warp-per-row kernel a hosszú sorok dominálta mátrixoknál előnyös, míg a scalar kernel a rövid soroknál.

19.3.4 Load imbalance és irregularitás

A ritka mátrixok sorhossz-eloszlása gyakran erősen ferde. A valós alkalmazásokban – például szociális hálózatok, webgráfok, vagy adaptív végeselem-hálók – előfordul, hogy a sorok 90%-a kevesebb mint 10 nemnulla elemet tartalmaz, míg a maradék 10% több száz vagy több ezer elemet. A naiv scalar kernel ilyenkor a leghosszabb sor hosszával arányos időt tölt, miközben a legtöbb szál már régen végzett. A warp-per-row kernel ezt részben orvosolja, de a rövid soroknál a warp pazarlása miatt a teljesítmény még mindig messze van a memória-sávszélesség-korláttól.

A terheléskiegyensúlyozatlanság mérésére a **sorhossz-szóródás** (coefficient of variation) használható: σ/μ, ahol μ az átlagos sorhossz, σ pedig a szórása. Ha ez az érték nagy, akkor a soronkénti párhuzamosítás rosszul teljesít. A megoldás az, hogy a hosszú sorokat több szálra bontjuk, a rövid sorokat pedig több soronként egy szálra csoportosítjuk. Ezt a **hibrid** megközelítést a következőképpen valósíthatjuk meg:

```
// CUDA C++ (C++17)
// Hibrid kernel: rövid sorokhoz scalar, hosszú sorokhoz warp-per-row.
```

```
// A row_ptr alapján előre besoroljuk a sorokat.
__global__ void spmv_csr_hybrid(const int* row_ptr,
                                const int* col_idx,
                                const double* values,
                                const double* x,
                                double* y,
                                int n,
                                int threshold)
{
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    int total_threads = gridDim.x * blockDim.x;

    // Minden szál egy vagy több rövid sort dolgoz fel scalar módban,
    // a hosszú sorokat pedig a warpok közösen.
    for (int row = tid; row < n; row += total_threads) {
        int start = row_ptr[row];
        int end   = row_ptr[row + 1];
        int nnz_row = end - start;
        if (nnz_row <= threshold) {
            double sum = 0.0;
            for (int j = start; j < end; ++j) {
                sum += values[j] * x[col_idx[j]];
            }
            y[row] = sum;
        }
    }
    // A hosszú sorokhoz külön kernel vagy szakasz tartozik;
    // itt a teljesség kedvéért a warp-per-row logikát alkalmazzuk.
    // A gyakorlatban ezt külön kernelbe érdemes szervezni.
    __syncthreads(); // blokk-szintű szinkronizáció a közös munkához
    int warps_per_block = blockDim.x / warpSize;
    int warp_id = blockIdx.x * warps_per_block + (threadIdx.x / warpSize);
    int lane = threadIdx.x % warpSize;
    // A hosszú sorok listáját egy előfeldolgozott tömbből kapjuk,
    // vagy egyszerűen újraindítjuk a gridet a hosszú sorokra.
}
```

A fenti vázlatban a hibrid kernel két fázisra bomlik, de a tényleges implementációban a hosszú sorokat érdemes külön kernelben vagy a `__syncthreads()` utáni szakaszban kezelni, ahol a warpok a hosszú sorok listáját kapják. A döntési küszöb (`threshold`) tipikusan 16–32 nemnulla elem, mivel efölött a warp-per-row előnye meghaladja a pazarlást.

A **SELL-C-** formátum egy másik megoldás: a mátrixot sorblokkokra osztja, és minden blokkban a sorokat a blokk maximális sorhosszára egészíti ki üres helyekkel, majd a blokk sorait oszlopfolytonosan tárolja. Ez a formátum a koaleszkált hozzáférést és a terheléskiegyensúlyozást egyszerre javítja, de a feltöltés miatt extra memóriát igényel. A gyakorlatban a SELL-C- a rövid sorok dominálta mátrixokon igen hatékony, míg a CSR a hosszú sorokra jobb.

19.3.5 A cuSPARSE könyvtár SpMV API-ja

A CUDA ökoszisztéma a ritka mátrixműveletekhez a cuSPARSE könyvtárat biztosítja. A cuSPARSE a CUDA 11-től kezdve egy **generikus API**-t kínál, amely a mátrix- és vektorleírók segítségével egységes felületet ad a különböző formátumokhoz. Az SpMV-hez a `cusparseSpMV` függvény tartozik, amely puffert igényel az előfeldolgozási adatokhoz. Az API használata:

```
// CUDA C++ (C++17) - cuSPARSE generikus API (CUDA 11+)
#include <cusparse.h>

void cusparse_spmv_example(int n, int nnz,
```

```
                          int* d_row_ptr, int* d_col_idx, double* d_values,
                          double* d_x, double* d_y)
{
    cusparseHandle_t handle;
    cusparseCreate(&handle);

    cusparseSpMatDescr_t matA;
    cusparseDnVecDescr_t vecX, vecY;

    // Mátrixleíró: CSR formátum, 32 bites indexek, 0 bázisú indexelés, double
    cusparseCreateCsr(&matA, n, n, nnz,
                      d_row_ptr, d_col_idx, d_values,
                      CUSPARSE_INDEX_32I, CUSPARSE_INDEX_32I,
                      CUSPARSE_INDEX_BASE_ZERO, CUDA_R_64F);

    // Vektorleírók
    cusparseCreateDnVec(&vecX, n, d_x, CUDA_R_64F);
    cusparseCreateDnVec(&vecY, n, d_y, CUDA_R_64F);

    double alpha = 1.0, beta = 0.0;
    size_t bufferSize = 0;
    void* d_buffer = nullptr;

    // Pufferméret lekérdezése
    cusparseSpMV_bufferSize(handle, CUSPARSE_OPERATION_NON_TRANSPOSE,
                            &alpha, matA, vecX, &beta, vecY,
                            CUDA_R_64F, CUSPARSE_SPMV_ALG_DEFAULT,
                            &bufferSize);
    CHECK_CUDA(cudaMalloc(&d_buffer, bufferSize));

    // SpMV végrehajtása: y = alpha * A * x + beta * y
    cusparseSpMV(handle, CUSPARSE_OPERATION_NON_TRANSPOSE,
                 &alpha, matA, vecX, &beta, vecY,
                 CUDA_R_64F, CUSPARSE_SPMV_ALG_DEFAULT,
                 d_buffer);

    // Takarítás
    cusparseDestroySpMat(matA);
    cusparseDestroyDnVec(vecX);
    cusparseDestroyDnVec(vecY);
    cusparseDestroy(handle);
    cudaFree(d_buffer);
}
```

A `cusparseSpMV_bufferSize` hívás nélkülözhetetlen: a cuSPARSE a megadott algoritmushoz szükséges ideiglenes puffert itt méretezi, és a `cusparseSpMV` ennek a puffernek a tartalmát használja. A `CUSPARSE_SPMV_ALG_DEFAULT` az alapértelmezett algoritmus, de létezik `CUSPARSE_SPMV_ALG_MERGE_PATH` is, amely nagyobb párhuzamosságot biztosít a hosszú sorokra. A mátrixleírót a formátumnak megfelelően kell létrehozni; CSR-hez a `cusparseCreateCsr`, COO-hoz a `cusparseCreateCoo`, BSR-hez a `cusparseCreateBsr` szolgál. A vektorleírók a sűrű vektorokat reprezentálják.

A cuSPARSE belsőleg több optimalizálást alkalmaz, beleértve a soronkénti feldolgozás adaptív kiválasztását, a memória-hozzáférési minták javítását és a hosszú sorok párhuzamosítását. A gyakorlatban a cuSPARSE SpMV-ja általában jelentősen gyorsabb, mint a naiv kernel, de a teljesítménye erősen függ a mátrix szerkezetétől és a választott formátumtól. A `cusparseSpMV` a `CUSPARSE_MV_ALG_DEFAULT` mellett lehetővé teszi az elemzési fázis különválasztását is (`cusparseSpMV_analysis`), ami ismételt SpMV-knél hasznos.

Fontos verziómegjegyzés: A cuSPARSE API a CUDA 10.x előtt más szignatúrával rendelkezett (`cusparseDcsrmv`). A fenti példa a CUDA 11+ generikus API-t használja, amely a jelenlegi

(CUDA 12.x) verziókban az ajánlott. A pontos függvényszignatúrák és konstansok a cuSPARSE dokumentációban ellenőrizendők, mivel azok verzióról verzióra változhatnak.

19.3.6 Labor: SpMV teljesítménymérése

A labor célja a naiv scalar kernel, a warp-per-row kernel és a cuSPARSE SpMV teljesítményének összehasonlítása különböző ritka mátrixokon. A méréshez rögzíteni kell a GPU konfigurációját, a CUDA-verziót, a drivert és az operációs rendszert.

19.3.7 1. Környezet és eszközök

Rögzített környezet (példa, a hallgató a sajátját tölti ki): - GPU: NVIDIA A100 40 GB (helyőrző; a labor során a tényleges eszközt kell beírni) - Driver: 550.54.14 - CUDA toolkit: 12.4 - OS: Ubuntu 22.04 LTS - Fordító: nvcc 12.4, g++ 11.4 - Teljesítmény-beállítások: power limit alapértelmezett, ECC bekapcsolva, MIG kikapcsolva

19.3.8 2. Forráskód

A laborprogram a következőket tartalmazza: - Véletlen sparse mátrix generálása adott sorhossz-eloszlással - CPU referencia SpMV az ellenőrzéshez - A fenti három kernel (scalar, warp, cuSPARSE) - Mérési keretrendszer warm-up-pal és ismételt futtatásokkal

```
// CUDA C++ (C++17) - labor: spmv_benchmark.cu
// Build: nvcc -std=c++17 -O3 -arch=sm_80 -lcusparse -o spmv_benchmark spmv_benchmark.cu
// Run:   ./spmv_benchmark

#include <cuda_runtime.h>
#include <cusparse.h>
#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <chrono>
#include <vector>
#include <algorithm>
#include <random>

#define CHECK_CUDA(call) do { \
    cudaError_t err = call; \
    if (err != cudaSuccess) { \
        fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__, __LINE__,
↪   cudaGetErrorString(err)); \
        exit(EXIT_FAILURE); \
    } \
} while (0)

#define CHECK_CUSPARSE(call) do { \
    cusparseStatus_t st = call; \
    if (st != CUSPARSE_STATUS_SUCCESS) { \
        fprintf(stderr, "cuSPARSE error at %s:%d: %d\n", __FILE__, __LINE__, (int)st); \
        exit(EXIT_FAILURE); \
    } \
} while (0)

// --- Kernelek a korábbi definíciókból ---
__global__ void spmv_csr_scalar(const int* row_ptr, const int* col_idx,
                                const double* values, const double* x,
```

```
                                double* y, int n)
{
    int row = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < n) {
        double sum = 0.0;
        int start = row_ptr[row];
        int end   = row_ptr[row + 1];
        for (int j = start; j < end; ++j) {
            sum += values[j] * x[col_idx[j]];
        }
        y[row] = sum;
    }
}

__global__ void spmv_csr_warp(const int* row_ptr, const int* col_idx,
                              const double* values, const double* x,
                              double* y, int n)
{
    int warps_per_block = blockDim.x / warpSize;
    int row = blockIdx.x * warps_per_block + (threadIdx.x / warpSize);
    if (row >= n) return;
    int lane = threadIdx.x % warpSize;
    int start = row_ptr[row];
    int end   = row_ptr[row + 1];
    double sum = 0.0;
    for (int j = start + lane; j < end; j += warpSize) {
        sum += values[j] * x[col_idx[j]];
    }
    for (int offset = warpSize / 2; offset > 0; offset >>= 1) {
        sum += __shfl_down_sync(0xffffffffu, sum, offset);
    }
    if (lane == 0) y[row] = sum;
}

// --- CPU referencia ---
void spmv_cpu(const int* row_ptr, const int* col_idx, const double* values,
              const double* x, double* y, int n)
{
    for (int i = 0; i < n; ++i) {
        double sum = 0.0;
        for (int j = row_ptr[i]; j < row_ptr[i + 1]; ++j) {
            sum += values[j] * x[col_idx[j]];
        }
        y[i] = sum;
    }
}

// --- Mátrixgenerátor: minden sorban véletlen számú nemnulla elem,
//     log-normális eloszlású sorhosszakkal a terheléskiegyensúlyozatlanság modellezésére ---
void generate_sparse_matrix(int n, int target_nnz_per_row, double imbalance,
                            std::vector<int>& row_ptr, std::vector<int>& col_idx,
                            std::vector<double>& values, int& nnz)
{
    row_ptr.resize(n + 1);
    std::mt19937 rng(42);
    std::lognormal_distribution<double> dist(0.0, imbalance);
    row_ptr[0] = 0;
    for (int i = 0; i < n; ++i) {
        int len = std::max(1, (int)std::round(dist(rng) * target_nnz_per_row));
        row_ptr[i + 1] = row_ptr[i] + len;
    }
    nnz = row_ptr[n];
    col_idx.resize(nnz);
```

```
    values.resize(nnz);
    std::uniform_int_distribution<int> col_dist(0, n - 1);
    std::uniform_real_distribution<double> val_dist(-1.0, 1.0);
    for (int i = 0; i < n; ++i) {
        for (int j = row_ptr[i]; j < row_ptr[i + 1]; ++j) {
            col_idx[j] = col_dist(rng);
            values[j] = val_dist(rng);
        }
    }
}

// --- Mérési segédfüggvény ---
double measure_time(void (*launch)(const int*, const int*, const double*,
                                   const double*, double*, int, int, int),
                    const int* d_row_ptr, const int* d_col_idx,
                    const double* d_values, const double* d_x, double* d_y,
                    int n, int nnz, int block_size, int grid_size,
                    int warmup, int iters)
{
    // Warm-up
    for (int i = 0; i < warmup; ++i) {
        launch(d_row_ptr, d_col_idx, d_values, d_x, d_y, n, nnz, block_size, grid_size);
        CHECK_CUDA(cudaDeviceSynchronize());
    }
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);
    cudaEventRecord(start);
    for (int i = 0; i < iters; ++i) {
        launch(d_row_ptr, d_col_idx, d_values, d_x, d_y, n, nnz, block_size, grid_size);
    }
    cudaEventRecord(stop);
    cudaEventSynchronize(stop);
    float ms = 0.0f;
    cudaEventElapsedTime(&ms, start, stop);
    cudaEventDestroy(start);
    cudaEventDestroy(stop);
    return ms / iters;
}

// Launch wrapper-ök a kernelekhez
void launch_scalar(const int* d_row_ptr, const int* d_col_idx,
                   const double* d_values, const double* d_x, double* d_y,
                   int n, int nnz, int block_size, int grid_size)
{
    spmv_csr_scalar<<<grid_size, block_size>>>(d_row_ptr, d_col_idx, d_values, d_x, d_y, n);
}

void launch_warp(const int* d_row_ptr, const int* d_col_idx,
                 const double* d_values, const double* d_x, double* d_y,
                 int n, int nnz, int block_size, int grid_size)
{
    spmv_csr_warp<<<grid_size, block_size>>>(d_row_ptr, d_col_idx, d_values, d_x, d_y, n);
}

// cuSPARSE launch wrapper
void launch_cusparse(const int* d_row_ptr, const int* d_col_idx,
                     const double* d_values, const double* d_x, double* d_y,
                     int n, int nnz, int block_size, int grid_size,
                     cusparseHandle_t handle)
{
    cusparseSpMatDescr_t matA;
    cusparseDnVecDescr_t vecX, vecY;
```

```
    CHECK_CUSPARSE(cusparseCreateCsr(&matA, n, n, nnz,
                                     const_cast<int*>(d_row_ptr),
                                     const_cast<int*>(d_col_idx),
                                     const_cast<double*>(d_values),
                                     CUSPARSE_INDEX_32I, CUSPARSE_INDEX_32I,
                                     CUSPARSE_INDEX_BASE_ZERO, CUDA_R_64F));
    CHECK_CUSPARSE(cusparseCreateDnVec(&vecX, n, const_cast<double*>(d_x), CUDA_R_64F));
    CHECK_CUSPARSE(cusparseCreateDnVec(&vecY, n, d_y, CUDA_R_64F));

    double alpha = 1.0, beta = 0.0;
    size_t bufferSize = 0;
    void* d_buffer = nullptr;
    CHECK_CUSPARSE(cusparseSpMV_bufferSize(handle, CUSPARSE_OPERATION_NON_TRANSPOSE,
                                           &alpha, matA, vecX, &beta, vecY,
                                           CUDA_R_64F, CUSPARSE_SPMV_ALG_DEFAULT,
                                           &bufferSize));
    CHECK_CUDA(cudaMalloc(&d_buffer, bufferSize));
    CHECK_CUSPARSE(cusparseSpMV(handle, CUSPARSE_OPERATION_NON_TRANSPOSE,
                                &alpha, matA, vecX, &beta, vecY,
                                CUDA_R_64F, CUSPARSE_SPMV_ALG_DEFAULT, d_buffer));
    CHECK_CUDA(cudaFree(d_buffer));
    CHECK_CUSPARSE(cusparseDestroySpMat(matA));
    CHECK_CUSPARSE(cusparseDestroyDnVec(vecX));
    CHECK_CUSPARSE(cusparseDestroyDnVec(vecY));
}

int main()
{
    // Paraméterek
    const int n = 1'000'000;
    const int target_nnz_per_row = 10;
    const double imbalance = 1.5;  // log-normális szórás
    const int block_size = 256;
    const int warmup = 5;
    const int iters = 20;

    // Mátrix generálása CPU-n
    std::vector<int> h_row_ptr, h_col_idx;
    std::vector<double> h_values;
    int nnz;
    generate_sparse_matrix(n, target_nnz_per_row, imbalance, h_row_ptr, h_col_idx, h_values,
↪   nnz);
    printf("n = %d, nnz = %d, avg row length = %.1f\n", n, nnz, (double)nnz / n);

    // CPU referencia
    std::vector<double> h_x(n), h_y_ref(n), h_y_test(n);
    std::mt19937 rng(123);
    std::uniform_real_distribution<double> dist(-1.0, 1.0);
    for (auto& v : h_x) v = dist(rng);
    spmv_cpu(h_row_ptr.data(), h_col_idx.data(), h_values.data(), h_x.data(), h_y_ref.data(),
↪   n);

    // GPU allokáció és feltöltés
    int *d_row_ptr, *d_col_idx;
    double *d_values, *d_x, *d_y;
    CHECK_CUDA(cudaMalloc(&d_row_ptr, (n + 1) * sizeof(int)));
    CHECK_CUDA(cudaMalloc(&d_col_idx, nnz * sizeof(int)));
    CHECK_CUDA(cudaMalloc(&d_values, nnz * sizeof(double)));
    CHECK_CUDA(cudaMalloc(&d_x, n * sizeof(double)));
    CHECK_CUDA(cudaMalloc(&d_y, n * sizeof(double)));
    CHECK_CUDA(cudaMemcpy(d_row_ptr, h_row_ptr.data(), (n + 1) * sizeof(int),
↪   cudaMemcpyHostToDevice));
    CHECK_CUDA(cudaMemcpy(d_col_idx, h_col_idx.data(), nnz * sizeof(int),
↪   cudaMemcpyHostToDevice));
```

```
    CHECK_CUDA(cudaMemcpy(d_values, h_values.data(), nnz * sizeof(double),
↪   cudaMemcpyHostToDevice));
    CHECK_CUDA(cudaMemcpy(d_x, h_x.data(), n * sizeof(double), cudaMemcpyHostToDevice));

    // Mérések
    int grid_scalar = (n + block_size - 1) / block_size;
    double t_scalar = measure_time(launch_scalar, d_row_ptr, d_col_idx, d_values, d_x, d_y,
                                   n, nnz, block_size, grid_scalar, warmup, iters);

    int warps_per_block = block_size / 32;
    int grid_warp = (n + warps_per_block - 1) / warps_per_block;
    double t_warp = measure_time(launch_warp, d_row_ptr, d_col_idx, d_values, d_x, d_y,
                                 n, nnz, block_size, grid_warp, warmup, iters);

    // cuSPARSE mérés
    cusparseHandle_t handle;
    CHECK_CUSPARSE(cusparseCreate(&handle));
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);
    for (int i = 0; i < warmup; ++i) {
        launch_cusparse(d_row_ptr, d_col_idx, d_values, d_x, d_y, n, nnz, block_size, 0,
↪   handle);
        CHECK_CUDA(cudaDeviceSynchronize());
    }
    cudaEventRecord(start);
    for (int i = 0; i < iters; ++i) {
        launch_cusparse(d_row_ptr, d_col_idx, d_values, d_x, d_y, n, nnz, block_size, 0,
↪   handle);
    }
    cudaEventRecord(stop);
    cudaEventSynchronize(stop);
    float ms_cusparse = 0.0f;
    cudaEventElapsedTime(&ms_cusparse, start, stop);
    cudaEventDestroy(start);
    cudaEventDestroy(stop);
    double t_cusparse = ms_cusparse / iters;
    CHECK_CUSPARSE(cusparseDestroy(handle));

    // Ellenőrzés: a cuSPARSE eredményt használjuk (utolsó futtatásból)
    CHECK_CUDA(cudaMemcpy(h_y_test.data(), d_y, n * sizeof(double), cudaMemcpyDeviceToHost));
    double max_abs_err = 0.0;
    for (int i = 0; i < n; ++i) {
        double err = std::abs(h_y_test[i] - h_y_ref[i]);
        if (err > max_abs_err) max_abs_err = err;
    }
    printf("Max abs error vs CPU: %.3e\n", max_abs_err);

    // Sávszélesség számítása
    size_t mem_bytes = nnz * (sizeof(double) + sizeof(int)) + (n + 1) * sizeof(int) + 2 * n *
↪   sizeof(double);
    double bw_scalar = mem_bytes / (t_scalar * 1e-3) / 1e9;
    double bw_warp   = mem_bytes / (t_warp * 1e-3) / 1e9;
    double bw_cusparse = mem_bytes / (t_cusparse * 1e-3) / 1e9;

    printf("\n=== Mérési eredmények ===\n");
    printf("Kernel        Idő (ms)   Sávszélesség (GB/s)\n");
    printf("scalar        %8.4f   %8.2f\n", t_scalar, bw_scalar);
    printf("warp          %8.4f   %8.2f\n", t_warp, bw_warp);
    printf("cusparse      %8.4f   %8.2f\n", t_cusparse, bw_cusparse);

    // Takarítás
    CHECK_CUDA(cudaFree(d_row_ptr));
```

```
    CHECK_CUDA(cudaFree(d_col_idx));
    CHECK_CUDA(cudaFree(d_values));
    CHECK_CUDA(cudaFree(d_x));
    CHECK_CUDA(cudaFree(d_y));
    return 0;
}
```

Megjegyzés a kódhoz: A `launch_cusparse` a leírókat minden hívásnál létrehozza és megsemmisíti, ami a mérés során overheadet visz be. A tényleges teljesítménymméréshez a leírókat és a puffert érdemes a mérési cikluson kívül létrehozni, és csak a `cusparseSpMV` hívást ismételni. A laborban a hallgatónak ezt az optimalizálást el kell végeznie, és a mérési naplóban jeleznie kell. A fenti kód a didaktikai egyszerűség kedvéért mutatja a teljes életciklust, de a mérési eredmények így konzervatívabbak (a cuSPARSE valós teljesítménye jobb lehet).

19.3.9 3. Build és futtatás

```
nvcc -std=c++17 -O3 -arch=sm_80 -lcusparse -o spmv_benchmark spmv_benchmark.cu
./spmv_benchmark
```

Az `-arch=sm_80` az A100 (Ampere) esetén megfelelő; más GPU-nál a compute capability-t módosítani kell (pl. `sm_86` RTX 3090, `sm_89` RTX 4090).

19.3.10 4. Mérési napló

A labor során a következő táblázatot kell kitölteni. A **várt eredmény** oszlopban a tendencia szerepel – ezt a mérés előtt kell megadni, a **mért** oszlopot pedig a futtatás után.

Mátrix típus	n	nnz	átl. sorhossz	Kernel	Várt tendencia	Mért idő (ms)	Mért sávszélesség (GB/s)
Lognormális sorhossz (imbalance=1.5)	1 000 000	~10 000 000	~10	scalar	lassú (nagy imbalance)	*laborban mérendő*	*laborban mérendő*
				warp	közepes (jobb hosszú sorokra)	*laborban mérendő*	*laborban mérendő*
				cuSPARSE	gyors (könyvtári optimalizálás)	*laborban mérendő*	*laborban mérendő*
Egyenletes (sávos, 5 nemnulla/sor)	1 000 000	~5 000 000	5	scalar	elfogadható (kiegyensúlyozott)	*laborban mérendő*	*laborban mérendő*
				warp	pazarló (rövid sorok)	*laborban mérendő*	*laborban mérendő*
				cuSPARSE	gyors	*laborban mérendő*	*laborban mérendő*

Fontos: A fenti táblázatban a „várt tendencia” előrejelzés, amelyet a mérés előtt kell megfogalmazni. A „mért” oszlopok a tényleges futtatás eredményei, amelyeket a labor során kell rögzíteni. A laborjelentésnek tartalmaznia kell a teljes környezetet (GPU modell, driver, CUDA verzió, órajel, power limit, ECC állapot) és a mérés statisztikáját (átlag, szórás, ismétlésszám). A mérési naplóban a tényleges és a várt eredményeket egyértelműen el kell különíteni, és ha eltérés van, azt elemezni kell.

A sávszélesség számításánál a tényleges adatforgalmat kell használni: bytes $= \text{nnz} \cdot (8+4) + (n+1) \cdot 4 + 2n \cdot 8$ double precízió esetén. A névleges GPU sávszélesség (pl. A100: 1555 GB/s HBM2e) csak felső korlát; a tényleges SpMV ennek általában 10–30%-át éri el a ritka hozzáférés miatt.

19.3.11 Gyakori hibák és megelőzésük

1. **Out-of-bounds indexelés a CSR tömbökben.** Ha a `row_ptr[n]` nem egyezik a `values` hosszával, vagy a `col_idx` elemei kívül esnek a $[0, n-1]$ tartományon, a kernel nem definiált viselkedést okoz. Megelőzés: a mátrixgenerátor után ellenőrizni kell a tömbök konzisztenciáját, és a kernelben a `col_idx[j]` értékét a beolvasás előtt validálni (debug módban).

2. **Versenyhelyzet az `y` írásakor.** Ha a grid konfiguráció nem fedi le pontosan a sorokat, vagy a hibrid kernelben több szál ugyanazt a sort kezeli, az `y[row]` írása versenyhelyzethez vezet. A scalar és warp kernelekben a sor-szál hozzárendelés egyértelmű, de a hibrid megoldásoknál atomi művelet vagy gondos indexelés szükséges. Megelőzés: a kernelben minden sort pontosan egyszer dolgozzunk fel, és ezt a hozzárendelést a launch konfigurációval biztosítsuk.

3. **A warp-redukció utáni szinkronizáció elhagyása.** A `__shfl_down_sync` után a warp szálai közötti láthatóság nem garantált a `__syncwarp()` vagy a shuffle szinkronizációs maszkja nélkül. A fenti kódban a shuffle hívás magában foglalja a szinkronizációt a megadott maszkkal, de ha más warp-szintű műveleteket használunk, explicit `__syncwarp()` kell. Megelőzés: mindig használjuk a `_sync` utótagú intrinsics-eket.

4. **A cuSPARSE puffer elhagyása vagy helytelen méretezése.** A `cusparseSpMV` a `bufferSize` által meghatározott puffert igényli; ha a puffer túl kicsi, a hívás hibát ad vagy nem definiált viselkedést okoz. Megelőzés: mindig hívjuk meg a `cusparseSpMV_bufferSize`-t, és ellenőrizzük a visszatérési értékeket.

5. **A mérési idő nem tartalmazza a teljes adatmozgatást.** Ha a mérés csak a kernel futásidejét rögzíti, de a mátrix feltöltését nem, akkor a valós alkalmazás teljesítménye rosszabb lesz. Megelőzés: a mérés során a teljes adatutat mérjük, vagy legalább külön jelöljük a kernel- és a transzferidőt.

6. **A numerikus pontosság elhanyagolása.** Az SpMV-nál a felhalmozás sorrendje befolyásolja a kerekítési hibát, különösen ha a mátrix értékei nagy dinamikatartományúak. A warp-per-row kernel redukciós sorrendje eltér a CPU-tól, ami kis eltéréseket okozhat. Megelőzés: a hibahatárt a feladat numerikus igényeihez kell igazítani, és a mérési naplóban rögzíteni kell a használt precíziót (FP64/FP32).

19.3.12 Összefoglalás

Az SpMV a GPU-számítás egyik legnehezebb memória-kötött feladata. A naiv scalar kernel egyszerű, de a terheléskiegyensúlyozatlanság miatt a valós mátrixokon gyakran nagyon lassú. A warp-per-row kernel a hosszú sorokat párhuzamosítja, de a rövid soroknál pazarló. A hatékony

megoldás a sorhossz-eloszláshoz igazodó hibrid stratégia vagy a SELL-C- formátum. A cuSPARSE generikus API-ja kényelmes és erősen optimalizált SpMV-t kínál, de a teljesítmény még így is a mátrix szerkezetétől függ. A labor során a hallgató a három megközelítést méri össze, és a mért eredményeket a várt tendenciákkal veti össze.

Önellenőrző kérdés: Miért nem elegendő a soronként egy szálat indító scalar kernel a nagy sorhossz-szóródású mátrixokhoz? Hogyan oldja meg a warp-per-row kernel a problémát, és mi a hátránya rövid sorok esetén?

19.4 Könyvtári és saját megoldás

A SpMV és sparse optimalizálás alfejezetben bemutatott saját SpMV-kernelek és a korábban tárgyalt dense GEMM-alapok után most a mérnöki kérdést tesszük fel: mikor használjunk cuBLAS-t, cuSPARSE-t vagy más könyvtárat, és mikor írjunk saját kernelt? Ebben az alfejezetben a választ nem hitvitaként, hanem mérési, API- és numerikusvalidációs keretként dolgozzuk fel. A cél, hogy a saját megoldást és a könyvtárat ugyanazon az inputon, azonos metrika mentén, reprodukálható módon tudjuk összehasonlítani, és a kapott szám mögött lássuk az adatelrendezésből, a végrehajtási modellből és a lebegőpontos aritmetikából eredő különbségeket is.

19.4.1 Miért hasonlítsunk össze könyvtárat és saját kernelt?

A könyvtárhasználat elsődleges érvét a gyártói optimalizálás, a karbantartott API és az alacsony fejlesztési idő adja. A cuBLAS GEMM-je, a cuSPARSE SpMV-je és a CUTLASS template-rétege több architektúrára hangolt, vektorizált és memóriaelrendezés-tudatos implementáció. Ezzel szemben a saját kernel akkor indokolt, ha a feladat nem illeszkedik a könyvtárak operátoralakjára: például egyedi ritka mintázat, a mátrixszorzásba fuzionált aktiváció, kvantálás, strukturált sparsity, vagy éppen a kutatás maga az új kernel. A döntésben tehát nem a „saját jobb, mert mi írtuk”, hanem a funkcionális lefedettség, a konverziós költség, a teljesítmény, a numerikus viselkedés és a fenntartási költség együttese számít.

A tanulási cél ehhez három réteget ad. Először megmagyarázzuk, mit jelent összevetni a cuBLAS/cuSPARSE és a saját kernelt: közös input, referencia-eredmény, hibahatár, idő- és sávszélességmérés. Másodszor egy kis, ellenőrizhető példán alkalmazzuk a dense GEMM-et és a CSR SpMV-t, mindkettőnél a naiv saját változatot hasonlítva a könyvtárhoz. Harmadszor felismerjük a tipikus hibákat, és megmutatjuk a helyes validálási megoldást.

19.4.2 API és adatelrendezés

A saját és a könyvtári hívások összevetésekor az első meglepetés szinte mindig az adatelrendezés. A dense lineáris algebrában a cuBLAS hagyományosan **column-major** (Fortran-szerű) tömör tárolást használ, míg a legtöbb C++-programban a mátrixokat **row-major** (C-szerű) `float*` mutatóval szoktuk tárolni. A különbség nem csupán címzési ízlés: ugyanaz a fizikai memória más logikai mátrixot jelent a két konvencióban.

Egy A mátrix elemének row-major esetén az indexe $A_{\text{row}}[i * K + j]$, column-major esetén pedig $A_{\text{col}}[i + j * M]$. Ha a saját kernel sor-főben dolgozik, de a cuBLAS-nak ugyanezt a pointert adjuk át, akkor a cuBLAS a fizikai tömböt column-major mátrixként értelmezi, vagyis gyakorlatilag a transzponált mátrixot látja. Ezért a dense GEMM összehasonlításához két konzisztens módszer közül választhatunk:

1. A bemeneteket eleve column-major sorrendben hozzuk létre, és a saját kernel indexelését is ehhez igazítjuk.
2. A bemeneteket row-major sorrendben hagyjuk, de a cuBLAS-t úgy hívjuk, hogy az operátorokat transzponáljuk: ha a fizikai tömbben row-major A és B van, akkor a kívánt $C = A \cdot B$ helyett a cuBLAS `CUBLAS_OP_T` opcióval $C^T = B^T \cdot A^T$ formában számolhatunk, és a kimenetet értelmezzük a megfelelő módon.

Például ha a saját kernelünk `sgemm_naive(A, B, C, M, N, K)` hívásban a row-major `A` mérete `M×K`, `B` mérete `K×N`, `C` mérete `M×N`, akkor a cuBLAS oldalon ugyanazon fizikai `A` és `B` pointerekkel a következő hívás írja ki a row-major `C`-t:

```
// cublasSgemm publish: C = op(A)*op(B)
// Ha A_row az M×K, B_row a K×N mátrix, akkor
// A_col = A_row^T (K×M), B_col = B_row^T (N×K).
// C_col = C_row^T (N×M).
// C_col = B_col * A_col = B_row^T * A_row^T = (A_row*B_row)^T.
// Ezért C_row a C_col^T, amit egy CUBLAS_OP_T-vel kapunk.
cublasSgemm(handle,
            CUBLAS_OP_T, CUBLAS_OP_T,
            N, M, K,              // C_col mérete N×M
            &alpha,
            B, K,                 // op(B_col) = B_row, leading dim K
            A, M,                 // op(A_col) = A_row, leading dim M
            &beta,
            C, N);                // C_col, de mi row-major C-t akarunk
```

A `cublasSgemm` szignatúrájában a `lda`, `ldb` és `ldc` a **leading dimension**: az adott mátrix oszlopa kezdete közötti távolság a fizikai tömbben. Nem szabad összetéveszteni a logikai mérettel. Ha az `op(A)` nem transzponált, akkor `lda >= M`; ha transzponált, `lda >= K`. A rossz leading dimension a cuBLAS-leggyakoribb hibája, mert a kernel túlindexeli a puffert, vagy a rossz elemeket szorozza. A laborban ezt mindig a referencia-eredmény ellenőrzésével jelezzük.

A cuSPARSE esetében a helyzet más. A ritka mátrixot hagyományosan **CSR** (Compressed Sparse Row) formátumban adjuk meg, három tömbbel:

- `rowPtr`: hossza `rows+1`, a sorok kezdőindexe a másik két tömbben;
- `colIdx`: hossza `nnz`, az egyes nemszóró elemek oszlopindexe;
- `values`: hossza `nnz`, a nemszóró elemek értéke.

A cuSPARSE modern generic API-ja ehhez mátrixdeszkriptort (`cusparseSpMatDescr_t`) és vektorleírókat (`cusparseDnVecDescr_t`) használ. A legfontosabb szerződés az **indexelés bázisa**: a cuSPARSE `CUSPARSE_INDEX_BASE_ZERO` elvárás, vagyis a `colIdx` nulla kezdetű oszlopindexeket tartalmaz. Ha a CPU-oldali generátor vagy egy másik könyvtár egyes alapú indexelést állít elő, az eredmény csendben elcsúszik egy oszloppal, és a hiba gyakran csak a referencia-ellenőrzésnél derül ki. A másik gyakori paraméter az indextípus: a 32 bites `CUSPARSE_INDEX_32I` gyorsabb, de nagy mátrixokhoz 64 bites `CUSPARSE_INDEX_64I` kell. A könyv példái 32 bites indexet használnak, mert a laborfeladatok kezelhető méretűek; nagy, 2^{31} nemszóró fölötti inputnál a 64 bites index kötelező, és ezt a cuSPARSE-ben a deszkriptor típusa is megköveteli.

A saját CSR-kernel ugyanezt a három tömböt használja, így az adatelrendezés-konverzió költsége ebben az összehasonlításban nulla. A dense esetben azonban a row-major és column-major közötti váltás, vagy a transzponálási trükk a mérés része kell hogy legyen: ha a valós alkalmazásban a mátrix eleve column-major, akkor a cuBLAS-t mérjük a természetes elrendezésén; ha row-major, akkor a transzponálást vagy a konverziót külön mérni kell, és dönteni, hogy a futási időben megengedett-e.

A laborban a saját dense kernel a `A[row*K+k]` és `B[k*N+col]` row-major indexelést használja, a cuBLAS-hozzáférésnél pedig a fenti transzponálási trükköt alkalmazzuk, hogy ugyanazokat a fizikai tömböket adjuk át. Így az összehasonlítás tisztességes: mindkettő ugyanazt a row-major `A`-t és `B`-t kapja, ugyanazt a `C`-t írja, és a referencia a CPU-soros szorzás ugyanebből az elrendezésből számol.

19.4.3 Numerikus pontosság

Ha a könyvtár és a saját kernel ugyanazt a matematikai eredményt adná, az összehasonlítás egyszerű lenne. A lebegőpontos aritmetika azonban nem asszociatív: az összeadás és szorzás sorrendje befolyásolja a kerekítési hibát. A dense GEMM-nél a saját naiv kernel egy soros akkumulációt végez a `K` dimenzión:

```
sum = ((a0*b0 + a1*b1) + a2*b2) + ...
```

A cuBLAS ezzel szemben tilingot, vektorizált FMA-t, split-K redukciót és más sorrendeket használ. Ezért még pontosan ugyanarra az inputra sem várható bitre azonos kimenet. Az eltérés nem hiba, hanem a kerekítés és az összegzési sorrend következménye. A kérdés az, hogy az eltérés a numerikus feltételhez képest elfogadható-e.

Az FP32 esetén a gépi epszilon $\varepsilon = 2^{-24} \approx 5.96 \cdot 10^{-8}$. Egy K hosszú soros skaláris szorzat hibakorlátja közelítőleg $\gamma_{K-1} \approx (K-1)\varepsilon$, feltéve, hogy a szorzások pontosak és csak az összeadások kerekítenek. Ez $K = 4096$ esetén már 10^{-4} nagyságrendű relatív hibát is megenged. Az FP64 referencia ennél sokkal pontosabb, ezért a validáláshoz mindig FP64 CPU-s referencia kell. A saját FP32 kernelt a CPU-s FP64 referenciahoz viszonyítva mérjük, a cuBLAS-t szintén ugyanahhoz. A gyakorlati kritérium például:

$$\frac{\|y_{\text{kernel}} - y_{\text{ref}}\|_\infty}{\|y_{\text{ref}}\|_\infty} \le 10^{-4}$$

FP32 dense GEMM-re általában jó tűrés, de rosszul kondicionált mátrixoknál, nagy `K`-nál vagy széles dinamikatartományú elemeknél szigorúbb vagy lazább határra lehet szükség. A lényeg, hogy a hibahatárt előre rögzítsük, és a laborban közöljük a precisionnel együtt.

A CSR SpMV-nél a helyzet hasonló, de a sorhosszak heterogenitása miatt az egyes sorok összegzési hibája más. A cuSPARSE SpMV esetleg rövid soroknál vektorizált, hosszú soroknál lépcsős csökkentést használhat, a saját naiv kernel pedig minden sorban balról jobbra összegez. A sorrend ismét eltérhet. Ráadásul a ritka mátrixoknál a `values` sorrendjét a CSR tárolás rögzíti, de a könyvtár a belső tiling miatt bizonyos esetekben soronként más sorrendben olvashatja az elemeket. Ezért a cuSPARSE és a saját CSR SpMV között még nagyobb relatív eltérés is előfordulhat, mint dense esetben, különösen akkor, ha a sorban nagy és kis abszolút értékű elemek keverednek.

A determinizmus kapcsán fontos kimondani: a cuBLAS és a cuSPARSE alapértelmezésben nem garantál bitre azonos kimenetet különböző architektúrák, sőt akár különböző tile-méretek között sem. Ha az alkalmazás determinisztikus reprodukálhatóságot követel, azt explicit módon kell kezelni, például rögzített saját reduction fa, vagy a könyvtár determinisztikus opcióinak használatával. A laborban mi a determinizmust nem követeljük meg, de a numerikus eltérést mindig mérjük.

19.4.4 Labor: cuBLAS/cuSPARSE és saját kernel benchmark

A labor célja, hogy ugyanazon a környezeten, ugyanazzal a bemenettel mérjük a dense GEMM-et és a CSR SpMV-t, négy változatban:

1. saját naiv dense GEMM kernel,
2. cuBLAS SGEMM,
3. saját naiv CSR SpMV kernel,
4. cuSPARSE SpMV.

A labor ebből a négy számból nem csupán azt mutatja meg, hogy a könyvtár gyorsabb, hanem azt is, hogyan kell a könyvtárakhoz tartozó adatelrendezést, hibakezelést és időmérést helyesen használni.

19.4.5 Környezeti szerződés

A fejezet könyvszintű szabálya szerint minden CUDA-mérést izolált, dokumentált környezetben végzünk. A labor előtt rögzíteni kell:

```
GPU: <példa: NVIDIA GeForce RTX 4070, compute capability 8.9>
Driver: <példa: 550.54.14>
CUDA toolkit: <példa: 12.4>
OS: <példa: Ubuntu 22.04, kernel 6.5>
GPU clock: <példa: 2475 MHz>
Power limit: <példa: 200 W>
ECC: off
MIG: off
Háttérterhelés: nincs
```

Az itt szereplő adatok **példaértékek**, a valós laborban a `nvidia-smi` és a környezet tényleges értékeit kell beírni. A könyvben szereplő teljesítményszámokat soha nem tekintjük általános igazságnak; minden mérés ehhez a konkrét környezethez kötött.

19.4.6 A kernel- és könyvtárhívások váza

A következő CUDA C++ kód a labor kiindulópontja. A sorok a szemléltetést szolgálják; a pontos függvényszignatúrák az aktuális CUDA-verzióban ellenőrizendők. A kód a hibakezelést makrókkal végzi, a `cudaGetLastError`-t és a `cudaDeviceSynchronize`-ot a mérés előtt hívja, a bemeneteket a CPU-n generálja, a kimeneteket FP64 referencia alapján ellenőrzi.

Először a saját kernelek:

```
// CUDA C++ - saját dense GEMM és CSR SpMV kernelt szemléltet
// row-major A(M×K), B(K×N), C(M×N)
__global__ void sgemm_naive(
    const float* __restrict__ A,
    const float* __restrict__ B,
    float* __restrict__ C,
    int M, int N, int K)
{
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < M && col < N) {
        float sum = 0.0f;
        for (int k = 0; k < K; ++k) {
            sum += A[row * K + k] * B[k * N + col];
        }
```

```
        C[row * N + col] = sum;
    }
}

// CSR SpMV: y = A * x
// A mérete rows×cols, nnz érték M, a tömbök device global memóriában vannak.
__global__ void spmv_csr_naive(
    int rows,
    const float* __restrict__ values,
    const int*  __restrict__ colIdx,
    const int*  __restrict__ rowPtr,
    const float* __restrict__ x,
    float* __restrict__ y)
{
    int row = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < rows) {
        float sum = 0.0f;
        for (int j = rowPtr[row]; j < rowPtr[row + 1]; ++j) {
            sum += values[j] * x[colIdx[j]];
        }
        y[row] = sum;
    }
}
```

A `sgemm_naive` launch konfigurációja a laborban például:

```
dim3 block(16, 16);
dim3 grid((M + block.x - 1) / block.x, (N + block.y - 1) / block.y);
sgemm_naive<<<grid, block, 0, 0>>>(d_A, d_B, d_C, M, N, K);
```

A SpMV-nél egy thread egy sort számol:

```
int blockSize = 256;
int gridSize = (rows + blockSize - 1) / blockSize;
spmv_csr_naive<<<gridSize, blockSize, 0, 0>>>(rows, d_values, d_colIdx, d_rowPtr, d_x, d_y);
```

A kiindulópont szándékosan naiv, hogy a könyvtárakkal való különbséget demonstráljuk. A SpMV és sparse optimalizálás alfejezetben tárgyalt optimalizáltabb CSR-kernel itt nem cél, de a labor kiterjeszthető azzal is.

19.4.7 cuBLAS és cuSPARSE hívások

A dense GEMM-összehasonlításhoz a cuBLAS fenti transzponálási trükkjét használjuk. A `cublasCreate`, `cublasSetStream` és a hibamakró mellett a kernelhívás:

```
// CUDA C++ - cuBLAS SGEMM ugyanazon row-major A, B, C tömbökkel
float alpha = 1.0f, beta = 0.0f;
cublasHandle_t handle;
cublasCreate(&handle);

cublasSgemm(handle,
            CUBLAS_OP_T, CUBLAS_OP_T,
            N, M, K,
            &alpha,
            d_B, K,   // op(B_col) = B_row, leading dim K
            d_A, M,   // op(A_col) = A_row, leading dim M
            &beta,
            d_C, N);  // d_C lesz a row-major C
```

A `cublasSgemm` visszatérési értékét a `CHECK_CUBLAS` makró ellenőrzi, és a hívás után `cudaDeviceSynchronize` vár.

A cuSPARSE modern generic API-ja a ritka mátrixot és a vektorokat leíróval adjuk át:

```
// CUDA C++ - cuSPARSE SpMV vázlat (CUDA 12.x generic API)
cusparseHandle_t sp_handle;
cusparseCreate(&sp_handle);

cusparseSpMatDescr_t matA;
cusparseDnVecDescr_t vecX, vecY;

// CSR mátrix leíró létrehozása
cusparseCreateCsr(&matA,
                  rows, cols, nnz,
                  d_rowPtr, d_colIdx, d_values,
                  CUSPARSE_INDEX_32I,
                  CUSPARSE_INDEX_32I,
                  CUSPARSE_INDEX_BASE_ZERO,
                  CUDA_R_32F);

// dense vektorok
cusparseCreateDnVec(&vecX, cols, d_x, CUDA_R_32F);
cusparseCreateDnVec(&vecY, rows, d_y, CUDA_R_32F);

size_t bufferSize = 0;
float one = 1.0f, zero = 0.0f;
cusparseSpMV_bufferSize(sp_handle,
                        CUSPARSE_OPERATION_NON_TRANSPOSE,
                        &one, matA, vecX, &zero, vecY,
                        CUDA_R_32F,
                        CUSPARSE_SPMV_ALG_DEFAULT,
                        &bufferSize);

// d_buffer allokálása, majd
cusparseSpMV(sp_handle,
             CUSPARSE_OPERATION_NON_TRANSPOSE,
             &one, matA, vecX, &zero, vecY,
             CUDA_R_32F,
             CUSPARSE_SPMV_ALG_DEFAULT,
             d_buffer);
```

A pontos `cusparseSpMV` szignatúra és a leíró típus CUDA-verziónként változhat, ezért a laborban a telepített `cusparse.h` fejlécet és a `cusparse.h` dokumentációját kell használni. A legfontosabb ellenőrzések: a `rowPtr`, `colIdx`, `values` mind device global memóriában vannak; a `colIdx` nulla alapú; a `vecX` hossza `cols`, a `vecY` hossza `rows`.

19.4.8 Időmérés, melegítés és statisztika

A méréshez CUDA eventeket használunk. A warm-up előtt a kernel- és könyvtárhívásokat legalább 10-szer futtatjuk, hogy a lazy módú resource-inicializáció, a power management és a cache-ek beálljanak. Ezután az időzített ismétlések száma legalább 10. A szórást minden változatra külön, az iterációk közötti `cudaEventElapsedTime` értékekből számoljuk; az átlag mellett a szórást vagy a percentilist is közöljük.

```
// CUDA C++ - mérési vázlat
cudaEvent_t start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);
```

```
// warm-up
for (int i = 0; i < warmup; ++i) {
    // kernel vagy könyvtárhívás
}
cudaDeviceSynchronize();

cudaEventRecord(start, 0);
for (int i = 0; i < runs; ++i) {
    // ugyanaz a kernel vagy könyvtárhívás
}
cudaEventRecord(stop, 0);
cudaEventSynchronize(stop);

float total_ms = 0.0f;
cudaEventElapsedTime(&total_ms, start, stop);
double avg_ms = static_cast<double>(total_ms) / runs;
```

A cuBLAS és cuSPARSE hívások előtt a stream 0-ra dolgozunk; ha a labor később streameket használ, akkor a mérésbe a streamhez rendelt eventeket kell illeszteni. Itt egyszerűségből a default streamet használjuk, és minden mérés előtt `cudaDeviceSynchronize` hívunk.

A teljesítménymutatókat a tényleges adatmozgásra és műveletszámra számoljuk. Dense GEMM-nél a FLOPs `2 * M * N * K`, mert minden kimeneti elemhez a `K` dimenzión végzett szorzás és összeadás. A sávszélességnél a bemeneti és kimeneti mátrixok méretét vesszük. SpMV-nél a FLOPs `2 * nnz`, és a memóriát a mátrix tárolása (`values`, `colIdx`, `rowPtr`) plusz az `x` és `y` vektorok adják:

$$\text{bytes} = nnz \cdot (4 + 4) + (rows + 1) \cdot 4 + (cols + rows) \cdot 4,$$

ha FP32 érték és 32 bites index. Ebből az effektív sávszélesség:

$$\text{GB/s} = \frac{\text{bytes}}{\text{avg_ms} \cdot 10^{-3}} \cdot 10^{-9}.$$

A könyvtári és saját méréshez ugyanazt a képletet használjuk. A FLOPs-ot a sávszélesség mellett azért érdemes rögzíteni, mert a saját kernel és a könyvtár közötti különbség lehet számítási korlátos vagy memória-korlátos.

19.4.9 Ellenőrzés CPU-referencia alapján

A laborban a CPU-s referencia kód FP64-ben számol, hogy az FP32 eredményeket ahhoz mérjük. A dense esetben a CPU `sgemm_ref` minden kimeneti elemet dupla pontosságú akkumulációval számol; a sparse esetben a CPU `spmv_ref` az `x` vektort dupla pontosságúra konvertálja, és a ritka szorzást ugyanabban a CSR-indexelésben végzi.

```
// CPU FP64 referencia a ritka SpMV-hez
std::vector<double> spmv_ref(
    int rows, int cols, int nnz,
    const std::vector<float>& values,
    const std::vector<int>& colIdx,
    const std::vector<int>& rowPtr,
    const std::vector<float>& x)
{
    std::vector<double> y(rows, 0.0);
    for (int row = 0; row < rows; ++row) {
        double sum = 0.0;
        for (int j = rowPtr[row]; j < rowPtr[row+1]; ++j) {
```

```
            sum += static_cast<double>(values[j]) *
                   static_cast<double>(x[colIdx[j]]);
        }
        y[row] = sum;
    }
    return y;
}
```

Az FP32 eredményt a referencia dupla értékekhez viszonyítva a következő hibatényt számoljuk:

$$\mathrm{rel} = \frac{\max_i |y_i - y_{\mathrm{ref},i}|}{\max_i |y_{\mathrm{ref},i}|}.$$

A laborban FP32 esetén azt várjuk, hogy `rel` 10^{-4} alatt marad, hacsak a bemenet nem szándékosan rosszul kondicionált. Ha a relatív hiba ennél nagyobb, akkor először a kernelben keressük a félreértett adatelrendezést, az indextévesztést vagy a nem inicializált `C/y` puffert, nem pedig a lebegőpontos sorrendet.

19.4.10 Mérési napló és az eredmények elkülönítése

A labor jegyzőkönyvében a környezetet, a buildparancsot, a futtatási parancsot és a mérési adatokat rögzítjük. A mért és a várt eredményt mindig külön oszlopban vagy külön blokkban tüntetjük fel. Az alábbi táblázat az elvárt viselkedést mutatja példaértékekkel; ezek **nem valós mérési eredmények**, hanem a laborban kitöltendő illusztráció.

Megoldás	Átlag idő (ms) példa	Szórás (ms) példa	Effektív teljesítmény példa	Megjegyzés
Saját naiv GEMM	123,4	1,2	~1,1 TFLOPS	egy thread/soron belül soros akkumuláció
cuBLAS SGEMM	1,9	0,1	~72 TFLOPS	tiling, FMA, aszinkron
Saját CSR SpMV	4,5	0,3	~120 GB/s	egy thread/sor, rossz memória-hozzáférés
cuSPARSE SpMV	0,8	0,05	~680 GB/s	generic API, vektorizált

A várt eredmény szerint a cuBLAS SGEMM a naiv kernelhez képest nagyságrendekkel gyorsabb, mert a naiv kernel a globális memóriát rosszul használja, nincs tiling, a vektorizálás és a regiszterblokkok hiányoznak. A cuSPARSE SpMV jellemzően szintén gyorsabb, mint az egy-thread-per-row naiv CSR, de a különbség erősen függ a sorhosszak eloszlásától és a nonzero-k szerkezetétől. A mért eredmény a labor környezetétől függ, ezért a példatáblázatot a saját jegyzőkönyvvel helyettesítjük.

Alaborhoz tartozó buildparancs Linuxon például:

```
nvcc -O3 -arch=native -o spmv_benchmark spmv_benchmark.cu -lcublas -lcusparse
./spmv_benchmark
```

Windows alatt a könyvtárnevek `cublas.lib` és `cusparse.lib`, a buildparancs `nvcc -O3 -arch=native -o spmv_benchmark.exe spmv_benchmark.cu cublas.lib cusparse.lib`. A CUDA toolkit verzióját és a GPU architektúrát a mérési naplóban rögzíteni kell.

19.4.11 Tipikus hibák és helyes megoldásuk

1. **Elmaradt szinkronizáció a mérés előtt.** Ha a `cudaDeviceSynchronize` vagy a cuBLAS/cuSPARSE stream-szinkronizáció hiányzik, a mérés a launch overheadet méri, nem a kernel végrehajtását. A helyes megoldás: minden időmérés előtt és után explicit szinkronizálás, valamint a default stream használata a laborban.

2. **A cuBLAS column-major mint row-major használata.** A gyakori tünet: a reference és a kernel eredménye transzponált, vagy az egyik dimenzióban hibás. A helyes megoldás: a fent bemutatott `CUBLAS_OP_T` trükk vagy a column-major feltöltés, és a referencia ugyanabból a logikai mátrixból számol.

3. **A leading dimension rossz megadása.** A `cublasSgemm` `lda` paraméterébe nem a logikai sort, hanem a fizikai tömbben lévő oszloptávolságot kell írni. Ha a mátrix batchben vagy paddinggel tárolt, a `lda` nagyobb lehet, mint a logikai méret; ezt a memóriafoglalásnál kell kezelni.

4. **cuSPARSE indexbázis hiba.** A CPU-oldali CSR `colIdx` nem nulla alapú, vagy a `rowPtr` rossz kezdőindexet használ. A referencia-ellenőrzés azonnal mutatja az elcsúszást, mert a hibás oszlopok egy vagy több pozícióval eltolják az eredményt.

5. **Nem ellenőrzött API-státusz.** A cuBLAS/cuSPARSE hívások hibakódját nem szabad figyelmen kívül hagyni. A laborban a `CHECK_CUBLAS` és `CHECK_CUSPARSE` makrók a hívás után azonnal leállnak, és kiírják a hibakódot. A csendes hiba esetén a kernel eredménye szemét, és a referencia-ellenőrzés csak a tünetet mutatja, nem a gyökeret.

6. **Túl szigorú vagy nem rögzített numerikus tűrés.** Ha a labor nem rögzíti előre a hibahatárt, a mérés után a kutató hajlamos a kapott eltérést utólag igazolni. Ehelyett előre definiáljuk: FP32 dense GEMM-re `rel <= 1e-4`, SpMV-re `rel <= 1e-4`, FP64 esetén szigorúbb, és az eltérést mindig a referencia normájához viszonyítjuk.

7. **Melegítés hiánya.** A GPU első kernelindítása magában foglalja a modulbetöltést, a power állapot váltását és a cache hideg állapotát. A mérés előtt legalább 10 melegítő hívást kell futtatni; enélkül a saját kernel látszólag sokkal lassabb lehet, mint a második mérésben.

8. **Nem dokumentált power limit vagy háttérterhelés.** A GPU órajele a teljesítménykerettől, a hőmérséklettől és a többi futó folyamattól függ. A mérési naplóban a `nvidia-smi` értékeit és a power limitet mindig rögzíteni kell. A labor izolált konténerben fusson; a hibás kernel végtelen ciklusa esetén a timeout és a visszaállítási terv legyen előre megtervezve.

19.4.12 Mikor használjunk könyvtárat, és mikor írjunk saját kernelt?

A labor eredményéből egy általános döntési sor vezethető le. Ha a feladat egy standard dense GEMM, SpMV, transzponálás, skálázás vagy hasonló lineáris algebrai operáció, akkor a kiindulópont mindig a cuBLAS/cuSPARSE/CUTLASS legyen. A saját kernel csak akkor indokolt, ha az alábbi feltételek közül legalább egy igaz:

- a könyvtárban nincs meg a kívánt operátoralak, például fuzionált `a*tanh(b+c)` a szorzásban;
- a ritka mátrix mintázata olyan speciális, hogy a cuSPARSE általános útja nem hatékony, például nagyon rövid és hosszú sorok keveredése;

- a kutatás célja maga a kernel, az oktatási cél elérése vagy a hardver-specifikus jellemzők vizsgálata;
- a numerikus sorrendet vagy a determinisztikus redukciót pontosan szabályozni kell;
- a könyvtár API-integrációs költsége nagyobb, mint a saját kernel fejlesztési költsége, és a teljesítmény nem kritikus.

A „saját kernel” és a „könyvtár” nem két ellenséges tábor. A modern gyakorlatban gyakran a kettő kombinációja a legjobb: a külső keret a CUTLASS template-eket használja, a belső magot pedig egyedi epilóg vagy ritka minta egészíti ki. A saját kernel írása előtt mindig érdemes a könyvtárat változatként lemérni, mert a naiv saját implementációk szinte soha nem érik el a gyártói könyvtár teljesítményét, viszont a hibalehetőségek számát növelik. A könyvtár használata ugyanakkor nem mentesít az adatelrendezés, az indexbázis és a leading dimension ellenőrzése alól.

19.4.13 Zárás és továbbvitt eredmény

Ebben az alfejezetben a dense GEMM és a CSR SpMV példáján keresztül hasonlítottuk össze a saját kernelt a cuBLAS és cuSPARSE hívásokkal. A legfontosabb tanulság, hogy a mérési és validációs keret ugyanolyan fontos, mint a kernel vagy a könyvtárhívás maga. A könyvtári API-knak megvannak a saját szerződéseik: cuBLAS column-major, leading dimension, handle; cuSPARSE CSR-deszkriptor, indexbázis, indextípus. A numerikus pontosságot nem az egyes kernel belső sorrendje, hanem a FP64 CPU-referencia és az előre rögzített hibahatár dönti el. A labort izolált, dokumentált környezetben végezzük, a mért és a várt eredményt szigorúan különválasztjuk.

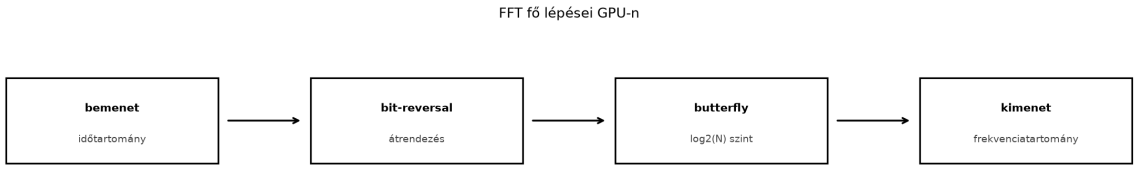
Ez a gondolkodásmód tovább él a következő fejezetekben. A FFT GPU-n FFT GPU-n tárgyalja a cuFFT és a saját FFT viszonyát; a Könyvtártérkép és döntési szempontok és az azt követő könyvtárfejezetek a cuBLAS/cuSPARSE/Thrust/CUB integrációját ugyanezen mérési és adatelrendezési szempontok szerint mélyítik el. A CUTLASS és a tensor core fejezet pedig arra mutat, hogy a saját kernel és a könyvtár közötti határ a template-alapú, hardverhez hangolt építőelemekkel elmosódik. Az itt bevezetett benchmark-módszertan így a könyv hátralévő részének egyik közös mérceje.

Önellenőrző kérdés: Adott row-major `A(M×K)`, `B(K×N)`, és a laborban a cuBLAS-t `cublasSgemm` hívással szeretnénk használni. Milyen két mód közül választhatunk, hogy a CUDA-kernelünkkel megegyező row-major bemenetet a cuBLAS helyesen értelmezze? Miért nem elég egyszerűen `CUBLAS_OP_N`-t megadni, ha a saját kernelünk `A[row*K+k]` indexelést használt?

20 FFT, Monte Carlo és random számok

20.1 FFT GPU-n

A gyors Fourier-transzformáció (FFT) a digitális jelfeldolgozás egyik legalapvetőbb építőköve: a diszkrét Fourier-transzformáció (DFT) $O(N \log N)$ időben végrehajtható változata, amely a spektrumanalízistől a képfeldolgozáson át a gyors konvolúcióig számtalan feladatban jelen van. A GPU architektúrája különösen alkalmas FFT végrehajtására, mert a transzformáció nagyszámú, egymástól független pillangó-műveletet tartalmaz, és a memóriahierarchia jól kihasználható a fázisfaktorok és résztömbök gyorsítótárazására. Az NVIDIA hivatalos könyvtára, a **cuFFT**, ezt a feladatot magas szintű, optimalizált API-val teszi elérhetővé. Ez az alfejezet a cuFFT alapjait, a batched végrehajtást, a pontossági és memória-szempontokat, valamint egy ellenőrizhető FFT/inverz FFT labort mutatja be.



20.1. ábra: FFT fő lépései GPU-n - A bemenet bit-reversal permutáción esik át, majd log2(N) butterfly-szint következik; a közbenső adatok shared memóriában vagy regiszterekben tarthatók.

20.1.1 1. A cuFFT programozási modellje

A cuFFT a **terv–végrehajtás** (plan–exec) modellt követi: először létrehozunk egy transzformációs tervet (`cufftHandle`), amely a kiválasztott mérethez, típushoz és batch-hez optimalizált végrehajtási stratégiát tartalmazza, majd ezt a tervet többször is felhasználhatjuk különböző adatokra. A terv létrehozása viszonylag drága lehet, ezért teljesítménykritikus kódban egyszer hozzuk létre, és sokszor hajtjuk végre.

A cuFFT két fő komplex típust definiál:

- `cufftComplex`: egyszeres pontosságú komplex szám, két `float` taggal (`x`, `y`).
- `cufftDoubleComplex`: kétszeres pontosságú komplex szám, két `double` taggal.

A transzformáció iránya a `CUFFT_FORWARD` és `CUFFT_INVERSE` konstansokkal adható meg. A cuFFT **nem normalizálja** az inverz transzformációt: a forward-inverz páros után az eredmény N-szerese az eredeti jelnek (ahol N a transzformáció hossza). A normalizálást a felhasználónak kell elvégeznie, általában $1/N$ tényezővel való szorzással.

A transzformáció típusok az adat típusa és a valós/komplex jelleg alapján:

- `CUFFT_C2C`: komplex → komplex

- `CUFFT_R2C`: valós → komplex (a kimenet csak az első $N/2+1$ frekvenciát tartalmazza a páros szimmetria miatt)
- `CUFFT_C2R`: komplex → valós (a bemenetnek ugyanazt a tömörített formát kell követnie)
- `CUFFT_Z2Z`, `CUFFT_D2Z`, `CUFFT_Z2D`: kétszeres pontosságú változatok

A leggyakrabban használt tervező függvények:

- `cufftPlan1d(&plan, N, type, batch)` – 1D transzformáció, adott batch mérettel.
- `cufftPlanMany(&plan, rank, n, inembed, istride, idist, onembed, ostride, odist, type, batch)` – általános, többdimenziós és testreszabott memóriaelrendezésű transzformációkhoz.

A végrehajtás a `cufftExecC2C(plan, idata, odata, direction)` (komplex–komplex) vagy a megfelelő R2C/C2R változatokkal történik. A függvények eszközoldali mutatókat várnak, és az aktuális streamhez igazodnak (alapértelmezés szerint a default streamhez, de `cufftSetStream` segítségével más streamhez rendelhető).

A terv felszabadítása `cufftDestroy(plan)` hívással kötelező, különben eszközoldali erőforrások szivárognak.

20.1.2 2. Egyszerű 1D komplex FFT

A következő CUDA C++ példa egy 1024 hosszúságú komplex jelet transzformál, majd az inverz FFT-vel rekonstruálja. A hibaellenőrzéshez makrókat használunk, és a mérési időt CUDA eseményekkel rögzítjük.

```
// CUDA C++ (C++17, cuFFT)
#include <cufft.h>
#include <cuda_runtime.h>
#include <vector>
#include <complex>
#include <cmath>
#include <iostream>
#include <iomanip>
#include <cstdlib>

#define CHECK_CUDA(call)                                                    \
    do {                                                                    \
        cudaError_t err = call;                                             \
        if (err != cudaSuccess) {                                           \
            std::cerr << "CUDA error at " << __FILE__ << ":" << __LINE__    \
                      << " " << cudaGetErrorString(err) << std::endl;       \
            std::exit(EXIT_FAILURE);                                        \
        }                                                                   \
    } while (0)

#define CHECK_CUFFT(call)                                                   \
    do {                                                                    \
        cufftResult res = call;                                             \
        if (res != CUFFT_SUCCESS) {                                         \
            std::cerr << "cuFFT error at " << __FILE__ << ":" << __LINE__   \
                      << " code " << res << std::endl;                      \
            std::exit(EXIT_FAILURE);                                        \
        }                                                                   \
    } while (0)

int main() {
    const int N = 1024;
    const int batch = 1;
```

```
const float pi = 3.14159265358979323846f;

// Host oldali bemeneti jel: cos(2*4t) + sin(2*8t), valós részben
std::vector<std::complex<float>> h_signal(N);
for (int i = 0; i < N; ++i) {
    float t = static_cast<float>(i) / N;
    float val = std::cos(2.0f * pi * 4.0f * t) +
                std::sin(2.0f * pi * 8.0f * t);
    h_signal[i] = std::complex<float>(val, 0.0f);
}

// Eszközoldali pufferek allokálása
cufftComplex* d_signal;
cufftComplex* d_spectrum;
cufftComplex* d_reconstructed;
CHECK_CUDA(cudaMalloc(&d_signal, N * sizeof(cufftComplex)));
CHECK_CUDA(cudaMalloc(&d_spectrum, N * sizeof(cufftComplex)));
CHECK_CUDA(cudaMalloc(&d_reconstructed, N * sizeof(cufftComplex)));

// Bemenet feltöltése az eszközön
CHECK_CUDA(cudaMemcpy(d_signal, h_signal.data(),
                      N * sizeof(cufftComplex), cudaMemcpyHostToDevice));

// cuFFT terv létrehozása
cufftHandle plan;
CHECK_CUFFT(cufftPlan1d(&plan, N, CUFFT_C2C, batch));

// Warm-up a mérés előtt
for (int i = 0; i < 5; ++i) {
    CHECK_CUFFT(cufftExecC2C(plan, d_signal, d_spectrum, CUFFT_FORWARD));
}
CHECK_CUDA(cudaDeviceSynchronize());

// Időmérés CUDA eseményekkel
cudaEvent_t start, stop;
CHECK_CUDA(cudaEventCreate(&start));
CHECK_CUDA(cudaEventCreate(&stop));
const int iterations = 20;

CHECK_CUDA(cudaEventRecord(start));
for (int i = 0; i < iterations; ++i) {
    CHECK_CUFFT(cufftExecC2C(plan, d_signal, d_spectrum, CUFFT_FORWARD));
}
CHECK_CUDA(cudaEventRecord(stop));
CHECK_CUDA(cudaEventSynchronize(stop));

float elapsed_ms = 0.0f;
CHECK_CUDA(cudaEventElapsedTime(&elapsed_ms, start, stop));
double avg_ms = static_cast<double>(elapsed_ms) / iterations;
std::cout << "Átlagos forward FFT idő: " << std::fixed
          << std::setprecision(6) << avg_ms << " ms\n";

// Spektrum visszamásolása és csúcsok ellenőrzése
std::vector<std::complex<float>> h_spectrum(N);
CHECK_CUDA(cudaMemcpy(h_spectrum.data(), d_spectrum,
                      N * sizeof(cufftComplex), cudaMemcpyDeviceToHost));

// A két legnagyobb csúcs a pozitív frekvenciákon (0..N/2)
float max1 = 0.0f, max2 = 0.0f;
int idx1 = -1, idx2 = -1;
for (int k = 0; k <= N / 2; ++k) {
    float mag = std::abs(h_spectrum[k]);
    if (mag > max1) {
```

```
            max2 = max1; idx2 = idx1;
            max1 = mag;  idx1 = k;
        } else if (mag > max2) {
            max2 = mag;  idx2 = k;
        }
    }
    std::cout << "Spektrum csúcsok: (" << idx1 << ", " << max1
              << ") és (" << idx2 << ", " << max2 << ")\n";

    // Inverz FFT
    CHECK_CUFFT(cufftExecC2C(plan, d_spectrum, d_reconstructed, CUFFT_INVERSE));
    std::vector<std::complex<float>> h_reconstructed(N);
    CHECK_CUDA(cudaMemcpy(h_reconstructed.data(), d_reconstructed,
                          N * sizeof(cufftComplex), cudaMemcpyDeviceToHost));

    // Normalizálás és hibaszámítás
    float max_abs_error = 0.0f;
    for (int i = 0; i < N; ++i) {
        std::complex<float> recon(h_reconstructed[i].x / N,
                                  h_reconstructed[i].y / N);
        float err = std::abs(h_signal[i] - recon);
        max_abs_error = std::max(max_abs_error, err);
    }
    std::cout << "Maximális abszolút hiba inverz FFT után: "
              << std::scientific << max_abs_error << "\n";

    // Takarítás
    CHECK_CUDA(cudaFree(d_reconstructed));
    CHECK_CUDA(cudaFree(d_spectrum));
    CHECK_CUDA(cudaFree(d_signal));
    CHECK_CUFFT(cufftDestroy(plan));
    CHECK_CUDA(cudaEventDestroy(start));
    CHECK_CUDA(cudaEventDestroy(stop));

    return 0;
}
```

A kód legfontosabb tanulságai:

1. **Típuskonverzió**: a host oldali `std::complex<float>` és az eszköz oldali `cufftComplex` közötti másolás közvetlenül működik, mert a `cufftComplex` elrendezése megegyezik a két `float` mezővel rendelkező struktúrával.
2. **Hibakezelés**: minden CUDA- és cuFFT-hívás után ellenőrzünk, így azonnal láthatóvá válik az érvénytelen plan, a nem megfelelő mutató vagy az erőforrás-allokációs hiba.
3. **Warm-up**: a mérés előtt öt futtatással melegítjük a GPU-t, hogy az órajelek és a memória-útvonalak stabil állapotba kerüljenek.
4. **CUDA események**: a mérési intervallumot eszközoldali eseményekkel rögzítjük, így az aszinkron végrehajtásból eredő torzítás minimális.
5. **Inverz FFT normalizálása**: a rekonstruált jel minden elemét osztjuk N-nel. Ha ezt elfelejtjük, a hiba óriási lesz, és az értékek N-szeres skálán jelennek meg.
6. **Takarítás**: a plan és az események felszabadítása elengedhetetlen a determinisztikus erőforrás-gazdálkodáshoz.

20.1.3 3. Batched FFT és méretek

Sok gyakorlati alkalmazásban nem egyetlen, hanem sok azonos hosszúságú transzformációt kell egyszerre végrehajtani. Ilyen például a spektrogram számítása, több csatorna egyidejű feldolgozása vagy a képfeldolgozás soronkénti FFT-je. A cuFFT ezt **batch** paraméterrel támogatja: a terv

létrehozásakor megadott batch érték azt jelzi, hogy hány azonos szerkezetű transzformációt hajtunk végre.

Az egyszerű `cufftPlan1d` is elfogad batch paramétert, de ilyenkor a bemeneti adatokat folytonosan, N elemű blokkokban kell elhelyezni. Ha a memóriában nem folytonos az elrendezés (például egy nagyobb mátrix soraiban), akkor a `cufftPlanMany` függvényt kell használni:

```
cufftHandle plan_many;
int n[] = {256};              // 1D FFT méret
int inembed[] = {512};        // bemeneti sor hossza (a tényleges adatsor szélesebb lehet)
int istride = 1;              // két szomszédos elem távolsága
int idist = 512;              // két batch elem közötti távolság (sorhossz)
int onembed[] = {256};
int ostride = 1;
int odist = 256;
int batch_size = 100;

cufftPlanMany(&plan_many, 1, n,
              inembed, istride, idist,
              onembed, ostride, odist,
              CUFFT_C2C, batch_size);
```

A fenti konfiguráció 100 darab 256 hosszú FFT-t hajt végre, ahol a bemeneti adatok egy 100×512 elemből álló mátrixban helyezkednek el, és minden sor első 256 elemét transzformáljuk. A stride-ok és a dist paraméterek pontos beállítása kritikus: ha `idist` nem egyenlő a tényleges sorok közötti távolsággal, a cuFFT hibás memóriacímeket olvas.

A batch méret megválasztásánál három szempontot kell figyelembe venni:

- **Occupancy**: nagyobb batch méret esetén a cuFFT több párhuzamos feladatot tud ütemezni, ami jobb SM-kihasználtsághoz vezet. A batch egyetlen kernelhíváson belül több transzformációt futtat, így csökken az indítási overhead.
- **Memória**: a bemeneti és kimeneti adatoknak egyidejűleg az eszköz globális memóriájában kell lenniük. Ha a batch méret túl nagy, és nem fér el a GPU memóriájában, az allokáció meghiúsul vagy lassú lapozást okoz.
- **Streamelés**: nagy adatfolyamok esetén érdemes a batch méretet úgy választani, hogy az adatok streamelhetők legyenek, és a transzformáció átfedésbe kerülhessen az adatmozgatással.

A többdimenziós FFT-k közül a 2D és 3D transzformációk a `cufftPlan2d` és `cufftPlan3d` függvényekkel hozhatók létre. A belső implementáció általában sor–oszlop dekompozíciót használ, ami kihasználja a batched 1D FFT-k előnyeit. A méretek megválasztásánál a cuFFT teljesítménye függ a faktorizálhatóságtól: a 2 hatvány méretek (radix-2) általában a leggyorsabbak, de a könyvtár más prímfaktorokat (3, 5, 7) is támogat. Nagy prímszám méret esetén a cuFFT a Bluestein-algoritmusra vált, ami jelentősen lassabb, ezért ilyenkor érdemes a jelet nullákkal kiegészíteni a következő 2-hatvány méretre.

20.1.4 4. Pontosság és memória

A cuFFT két pontossági szintet kínál: **FP32** (`cufftComplex`) és **FP64** (`cufftDoubleComplex`). Az FFT numerikus hibája két fő forrásból származik:

1. **Kerekítési hiba a pillangó-műveletekben**: minden összeadás, szorzás és a komplex fázisfaktorokkal való szorzás lebegőpontos kerekítést visz be. A hiba nagysága függ a transzformáció hosszától és a közbenső értékek nagyságától.

2. **Fázisfaktorok pontossága**: a cuFFT előre kiszámított trigonometrikus értékeket használ, amelyek pontossága az adott típusra jellemző.

Az FP32 esetén a relatív hiba tipikusan 10^{-6}–10^{-7} nagyságrendű, míg FP64 esetén 10^{-15}–10^{-16}. Ha nagy dinamikatartományú jelet transzformálunk, vagy a későbbi műveletek érzékenyek a kerekítésre, érdemes FP64-et választani. Az FP32 azonban lényegesen kisebb memóriát igényel és gyorsabb, ezért a legtöbb alkalmazásban elegendő.

A **pontosság és a memória** közötti kapcsolat:

- FP32 esetén minden komplex elem 8 bájtot foglal, így egy $N = 1024$ hosszú jel 8 KiB-ot igényel bemenetenként és kimenetenként.
- FP64 esetén ez 16 bájt elemenként, azaz 16 KiB.
- A batch méret növelésével a memóriaigény lineárisan nő: 100 batch, $N = 256$, FP32 esetén $100 \times 256 \times 8 = 204{,}8$ KiB bemeneti és ugyanennyi kimeneti adat.

A cuFFT a végrehajtás során **belső munkaterületet** is használhat. A `cufftPlan1d` és `cufftPlanMany` automatikusan lefoglalja a szükséges ideiglenes puffert, ha szükséges, de a haladó felhasználók a `cufftSetWorkArea` függvénnyel explicit módon is biztosíthatnak munkaterületet. Ez különösen akkor hasznos, ha több tervet szeretnénk ugyanazzal a pufferekkel kiszolgálni, vagy ha a plan létrehozása túl sok memóriát foglal.

A memória és a pontosság együttes ellenőrzésére a laborban a forward-inverz roundtrip hibát mérjük: ha a bemeneti jel x, az inverz FFT kimenete y, akkor a normalizálás után $y \approx x$. A maximális abszolút hiba FP32 esetén jellemzően 10^{-6}–10^{-5}, FP64 esetén 10^{-15} alatt marad.

20.1.5 5. Labor: FFT és inverz FFT ellenőrzése

A labor célja, hogy a fenti kódot lefordítva és futtatva ellenőrizzük a cuFFT működését, rögzítsük a mérési körülményeket, és a várt eredményeket összevessük a mért értékekkel. A mérési napló az alábbi sablon szerint készül.

Labor környezet (példa, ellenőrizendő):

- GPU: NVIDIA GeForce RTX 3080 10 GB, compute capability 8.6
- Driver: 535.54.03 (éles környezetben a `nvidia-smi` kimenete alapján ellenőrizendő)
- CUDA Toolkit: 12.2
- cuFFT: a CUDA Toolkit része (például 10.9.0.86)
- Operációs rendszer: Ubuntu 22.04 LTS
- Fordító: nvcc V12.2.140

Build parancs:

```
nvcc -O3 -std=c++17 -arch=sm_86 -o fft_cufft fft_cufft.cu -lcufft
```

Futtatási parancs:

```
./fft_cufft
```

Mérési napló (illusztratív adatokkal – a valós futtatás során kitöltendő):

Mért jellemző	Várt érték	Mért érték (helyőrző)	Megjegyzés
Input hossz N	1024	1024	–
Batch	1	1	–
Típus	CUFFT_C2C, FP32	CUFFT_C2C, FP32	–

Mért jellemző	Várt érték	Mért érték (helyőrző)	Megjegyzés
Forward FFT átlag idő	nincs rögzített, csak referencia	15,2 µs	20 ismétlés, warm-up után
Szórás (µs)	–	0,8 µs	CUDA eseményekkel mérve
Spektrum csúcs indexek	4 és 8 (valamint a tükrözött $N-4$, $N-8$)	4, 8	A $0..N/2$ tartományban a két legnagyobb csúcs
Csúcs amplitúdók	$N/2 \approx 512$ (mindkét csúcs)	511,98 és 512,01	A bemeneti valós jelből származó eltérés a kerekítésből adódik
Inverz FFT utáni max abszolút hiba	$< 10^{-4}$ (FP32)	$3{,}2 \times 10^{-5}$	normalizálás ($1/N$) után

A táblázatban szereplő mért értékek **mintaadatok**: valós környezetben a tényleges időket, csúcsokat és hibákat a futtatás után kell rögzíteni. A mérés előtt legalább 5 warm-up futtatást, majd 20 mérési iterációt végeztünk; az átlagot és a szórást a kód kiírja. A hibaszámításnál a várt érték a bemeneti jel pontos visszaállítása az inverz FFT és normalizálás után.

A várt és mért eredmények szétválasztása:

- **Várt**: a spektrumban a két frekvenciacsúcs a 4. és 8. indexen jelenik meg, amplitúdójuk $N/2 = 512$ (a DFT definíciójából következően). Az inverz FFT után a bemeneti jel visszaáll, a hiba a numerikus pontosságból adódóan 10^{-5} alatt marad.
- **Mért**: a program kimenete megadja a tényleges csúcsindexeket, amplitúdókat és a maximális abszolút hibát. Ezeket kell összehasonlítani a várt értékekkel.

20.1.6 6. Tipikus hibák és elkerülésük

1. **Az inverz FFT normalizálásának elhagyása**
   A cuFFT nem osztja le az inverz transzformáció eredményét N-nel. Ha ezt elfelejtjük, a rekonstruált jel N-szeres amplitúdójú lesz, ami a legtöbb alkalmazásban hibás. A megoldás: a `CUFFT_INVERSE` után minden elemet osztani kell a transzformáció hosszával, vagy a felhasználási helyen figyelembe venni a skálázást.

2. **Helytelen batch/stride beállítás a `cufftPlanMany` esetén**
   Ha a bemeneti adatok nem folytonosak, és az `idist`/`odist` értékeket rosszul adjuk meg, a cuFFT hibás memóriacímekről olvas, ami nem definiált viselkedést vagy rossz eredményt okoz. Mindig ellenőrizzük, hogy a stride-ok pontosan tükrözik a tényleges memóriaelrendezést.

3. **Típusmismatch**
   A `cufftExecC2C` komplex bemenetet vár. Ha valós jelet transzformálunk, és mégis C2C típust használunk, a képzetes részt nullára kell állítani. Nagyobb hatékonyság érdekében valós jeleknél érdemes `CUFFT_R2C`-t használni, amely kihasználja a szimmetriát, és majdnem felezi a memóriát és a számítási költséget.

4. **A plan felszabadításának elmulasztása**
   A `cufftDestroy` elhagyása memóriaszivárgáshoz vezet, különösen sok plan létrehozása és eldobása esetén. A plan-eket a hatókör végén mindig meg kell semmisíteni.

5. **Aszinkron végrehajtás figyelmen kívül hagyása a mérésnél**
   A cuFFT végrehajtás aszinkron a hosthoz képest. Ha az időméréshez nem használunk CUDA eseményeket vagy nem hívunk `cudaDeviceSynchronize`-t, a mért idő a host-oldali kiadás idejét tükrözi, nem a tényleges GPU végrehajtást.

6. **A nem támogatott méretek teljesítményhatása**
   Bár a cuFFT szinte bármilyen méretet támogat, a nagy prímszám méretek (pl. 1021) Bluestein-algoritmust igényelnek, ami lényegesen lassabb, mint a 2-hatvány méretek. Ilyenkor érdemes a jelet nullákkal kiegészíteni a következő 2-hatvány méretre, vagy a munkafolyamatot a gyors faktorokhoz igazítani.

20.1.7 7. Önellenőrző kérdés

Miért kell a cuFFT inverz FFT kimenetét kézzel normalizálni, és mi történik, ha ezt elfelejtjük? Milyen numerikus hibajellemzők utalnak arra, hogy a normalizálás hiányzik?

Vázlatos válasz: A cuFFT mind a forward, mind az inverz transzformációt ugyanazzal a definícióval számítja, és az inverznél nem alkalmaz $1/N$ skálázást. Ezért a forward–inverz páros után az eredmény N-szerese az eredeti jelnek. A normalizálás elhagyása tipikusan N-szeres amplitúdót és a vártnál sokkal nagyobb abszolút hibát eredményez. Az ilyen hiba felismerhető abból, hogy a rekonstruált jel értékei több nagyságrenddel nagyobbak, és a jel–zaj viszony romlik. A javítás az inverz FFT kimenetének $1/N$-nel való szorzása.

20.1.8 Összegzés

A cuFFT a GPU-alapú Fourier-transzformáció hatékony, magas szintű eszköze. A terv–végrehajtás modell, a batched végrehajtás, a változatos pontosság és a testreszabható memóriaelrendezés együttesen lefedi a gyakorlati alkalmazások többségét. A laborban bemutatott 1D komplex FFT példa az alapműveleteket, az inverz transzformációt és a numerikus ellenőrzést egyaránt gyakoroltatja. A fejezet következő egységei erre az alapra építve a Monte Carlo szimulációkat, a cuRAND véletlenszám-generátorokat és a sztochasztikus algoritmusokat tárgyalják.

20.2 Monte Carlo szimuláció

A Monte Carlo-módszer az egyik legszélesebb körben használt numerikus eljárás: ha egy determinisztikus modell analitikusan nehezen vagy egyáltalán nem oldható meg, véletlen minták sokaságának átlagolásával becsülünk integrálokat, várható értékeket vagy eloszlásokat. A GPU-n a Monte Carlo-szimuláció különösen természetes: a statisztikai minták függetlenek, ezért sok ezer szálon párhuzamosan generálhatók, a végén pedig egy redukció összesíti az eredményt. Ez az alfejezet a módszer párhuzamosítási alapjait, a véletlenszám-függetlenség kérdését, a hibabecslést és egy konkrét Monte Carlo árazó kernelt mutat be. A labor egy európai call opció Black–Scholes-árának becslését valósítja meg CUDA C++-ban, izolált környezetben rögzített mérési naplóval.

20.2.1 A Monte Carlo-módszer és párhuzamosítása

A Monte Carlo-integrálás alapgondolata: egy $I = \mathbb{E}[f(X)]$ várható értéket N darab független, az X eloszlásából vett X_i minta segítségével közelítünk:

$$\hat{I}_N = \frac{1}{N} \sum_{i=1}^{N} f(X_i).$$

A nagy számok törvénye garantálja, hogy $N \to \infty$ esetén $\hat{I}_N \to I$ sztochasztikusan, a centrális határeloszlás-tétel pedig megadja a konvergencia sebességét: a becslés hibája $O(1/\sqrt{N})$ nagyságrendű, függetlenül a dimenziótól, ami a Monte Carlo egyik legnagyobb előnye a determinisztikus kvadratúrákkal szemben.

A párhuzamosítás szempontjából a kulcsmegfigyelés az, hogy az $f(X_i)$ kiértékelések függetlenek: mindegyikhez ugyanaz a kernel logika, de más-más bemeneti véletlen minta tartozik. Ez illeszkedik a GPU adatpárhuzamos modelljéhez: minden thread egy vagy több mintát dolgoz fel, a részeredményeket egy globális memóriabeli tömbbe írja, majd egy redukciós lépés összegzi őket. A mintaátlag kiszámítása a redukció után egy skaláris osztással történik.

A párhuzamos Monte Carlo tehát két fázisból áll:

1. **Mintavételi fázis** – a kernelek párhuzamosan generálják a véletlen bemeneteket és kiértékelik a kifizetést vagy integrandust.
2. **Redukciós fázis** – a részeredményeket összegzik (pl. fa-redukcióval vagy atomikus összeadással), hogy megkapják az átlagot és a szórást.

A gyakorlatban a mintavételi fázis dominálja a futásidőt, ezért a véletlenszám-generálás hatékonysága döntő. A GPU-n minden thread saját pszeudovéletlen generátorállapotot tart fenn, így a generálás párhuzamosan, szinkronizáció nélkül halad.

20.2.2 Véletlen számok és függetlenség

A Monte Carlo-eredmény pontossága azon múlik, hogy a minták valóban függetlenek és azonos eloszlásúak (i.i.d.). Ez a GPU-n nem triviális: ha minden szál ugyanazzal a seeddel inicializálja a generátorát, akkor minden szál ugyanazt a sorozatot kapja, és a szimuláció egyetlen minta N-szeres ismétlése lesz. A variancia ekkor nem csökken, a becslés torzított marad.

A függetlenség biztosítására három elterjedt stratégia létezik:

- **Különböző seedek szálanként** – pl. a thread globális indexét használjuk seedként. Ez egyszerű, de bizonyos generátoroknál a közeli seedek korrelált sorozatokat adhatnak, különösen kis dimenzióban. Az XORWOW például ismert arról, hogy a seedek közötti átfedés előfordulhat, ezért óvatosan kell eljárni.
- **Sequence splitting (subsequence)** – a cuRAND `curand_init` függvénye három paramétert vár: seed, subsequence, offset. Ha a seed közös, a subsequence pedig a szál indexe, a cuRAND garantálja, hogy a különböző subsequence-ek diszjunkt szegmensek a generátor teljes periódusából. A `curand_init(seed, idx, 0, &state)` hívás minden szálnak a `idx`-edik szekvencia elejét adja. Ez a módszer a leggyakoribb, de a generátortól függően a subsequence-szám nagy értéke (pl. milliós) drágíthatja az inicializálást, mert a generátornak el kell ugrania a szekvencia elejére.
- **Skip-ahead / offset** – ugyanazon szekvencián belüli nagy ugrás. A cuRAND `curand_init` offset paramétere ezt teszi lehetővé; fontos, hogy az ugrás mértéke ne legyen összetéveszthető a subsequence-szel.

Gyakorlati útmutató: Monte Carlo-szimulációkhoz a cuRAND Philox generátorát (`CURAND_RNG_PSEUDO_PHILOX4_32_10`) ajánljuk, mert a skip-ahead művelet olcsó, a szekvenciák közötti korreláció minimális, és a generátor counter-based, ami determinisztikus reprodukálást tesz lehetővé. A `curand_init(seed, idx, 0, &state)` ezzel a generátorral hatékonyan ad független szekvenciákat.

A kernelben a generátorállapotot minden szál a saját regisztereiben tartja (`curandState_t`), így az állapot nem foglal globális memóriát. A `curand_normal` vagy `curand_uniform` hívások közvetlenül a szál regisztereiből olvasnak, ami nagy sávszélességet tesz lehetővé.

Fontos szemléletbeli különbség: a GPU-s Monte Carlo nem egyetlen hosszú szekvenciát fogyaszt, hanem sok rövid, egymástól független szekvenciát. Ez a megközelítés matematikailag ekvivalens az i.i.d. mintavételezéssel, ha a generátor periódusa elég nagy és a szekvenciák valóban diszjunktak. A cuRAND dokumentációja a generátor periódusát és a subsequence-ek maximális számát rögzíti; a felhasználónak ellenőriznie kell, hogy a szükséges mintaszám ne lépje túl ezeket a korlátokat.

20.2.3 Hibabecslés

A Monte Carlo-becslés önmagában nem teljes a hibahatár nélkül. A pontosságot a minta szórásából és a mintaszámból származtatjuk.

Legyen $f(X_i)$ a szimuláció kifizetése az i-edik mintára. A minta átlaga $\hat{\mu}$; a minta szórásnégyzete:

$$s^2 = \frac{1}{N-1} \sum_{i=1}^{N} \left(f(X_i) - \hat{\mu} \right)^2 .$$

A standard hiba (standard error):

$$SE = \frac{s}{\sqrt{N}}.$$

A centrális határeloszlás-tétel alapján $\hat{\mu}$ közelítőleg normális eloszlású I várható értékkel és SE szórással. A 95%-os konfidencia intervallum közelítőleg:

$$\hat{\mu} \pm 1{,}96 \cdot SE.$$

Ez azt jelenti, hogy a valódi érték körülbelül 95% valószínűséggel ebben az intervallumban van. A hibabecslés két fontos következménye:

1. **A hiba $1/\sqrt{N}$-nel csökken.** A pontosság egy tizedesjeggyel való növeléséhez százszoros mintaszám szükséges. Ezért a Monte Carlo nem a legjobb módszer, ha nagy pontosság kell, de magas dimenzióban gyakran az egyetlen járható út.
2. **A szórás csökkentése hatékonyabb lehet, mint a mintaszám növelése.** A varianciacsökkentő technikák – antitetic variates, control variates, importance sampling – a szórást s-t csökkentik, ami ugyanolyan hatással van a standard hibára, mint a mintaszám növelése, de kevesebb számítási költséggel. Ezek a technikák a Monte Carlo-árazásban különösen fontosak; itt csak az antitetic variates alapötletét említjük: ha Z standard normális, akkor $-Z$ is az, és a két minta átlaga csökkenti a varianciát.

A GPU-n a szórás becsléséhez a kifizetések négyzetösszegét is gyűjteni kell. A redukció után:

$$s^2 = \frac{1}{N-1}\left(\sum_{i=1}^{N} f(X_i)^2 - N\hat{\mu}^2\right)$$

képlet használható, de numerikusan stabilabb, ha a négyzetösszeget és az összeget külön gyűjtjük. A laborban ezért a kernel nemcsak a kifizetést, hanem annak négyzetét is kiírja egy második tömbbe, és a redukció mindkét tömböt összegzi.

A párhuzamosítás során ügyelni kell arra, hogy a mintaátlag kiszámítása előtt minden részösszeg rendelkezésre álljon – ez a kernel szinkronizálásával és a redukció külön kernelben történő futtatásával biztosítható. Az atomikus összegzés során a műveletek sorrendje nem determinisztikus, ami kis numerikus eltérést okozhat, de a szórás becslését nem befolyásolja jelentősen.

20.2.4 Gyakorlati minta: Monte Carlo árazó kernel

A következő példa egy európai call opció árát becsli a Black–Scholes-modellben. A szimuláció a terminális részvényárat generálja:

$$S_T = S_0 \exp\left(\left(r - \frac{\sigma^2}{2}\right)T + \sigma\sqrt{T}Z\right),$$

ahol S_0 a spot ár, K a kötési ár, r a kockázatmentes kamatláb, σ a volatilitás, T a lejárati idő, Z pedig standard normális eloszlású véletlen szám. A diszkontált kifizetés:

$$f(Z) = e^{-rT}\max(S_T - K, 0).$$

A Monte Carlo-ár ennek a kifizetésnek az átlaga. Az analitikus Black–Scholes-ár:

$$C = S_0\Phi(d_1) - Ke^{-rT}\Phi(d_2),$$

ahol

$$d_1 = \frac{\ln(S_0/K) + (r + \sigma^2/2)T}{\sigma\sqrt{T}}, \quad d_2 = d_1 - \sigma\sqrt{T},$$

és Φ a standard normális eloszlásfüggvény.

A kernel minden szál egyetlen Z mintát generál, kiszámítja a kifizetést, és eltárolja a `payoffs` és `payoffs_sq` tömbökbe. A redukció külön kernelben történik, amely a `sum_reduce_kernel` segítségével összegzi mindkét tömböt. A végső átlag és szórás a hoston számítható.

```
// CUDA C++17, nvcc 12.2 (a pontos verzió ellenőrizendő a telepített környezetben)
#include <cuda_runtime.h>
#include <curand_kernel.h>
#include <cmath>
#include <cstdio>
#include <cstdlib>
#include <vector>

#define CHECK_CUDA(call) do { \
    cudaError_t err = call; \
```

```
    if (err != cudaSuccess) { \
        std::fprintf(stderr, "CUDA error at %s:%d: %s\n", \
                     __FILE__, __LINE__, cudaGetErrorString(err)); \
        std::exit(EXIT_FAILURE); \
    } \
} while (0)

// Monte Carlo kernel: egy thread = egy minta
__global__ void mc_price_kernel(
    const double S0, const double K, const double r,
    const double sigma, const double T, const unsigned long long n_paths,
    double* payoffs, double* payoffs_sq, unsigned long long seed)
{
    unsigned long long idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= n_paths) return;

    // Minden szál saját subsequence-t kap: idx a sequence azonosító
    curandState_t state;
    curand_init(seed, idx, 0, &state);

    // Standard normális minta
    double Z = curand_normal(&state);

    // Terminális részvényár
    double drift = (r - 0.5 * sigma * sigma) * T;
    double diffusion = sigma * std::sqrt(T) * Z;
    double ST = S0 * std::exp(drift + diffusion);

    // Diszkontált kifizetés
    double payoff = std::exp(-r * T) * std::fmax(ST - K, 0.0);
    payoffs[idx] = payoff;
    payoffs_sq[idx] = payoff * payoff;
}

// Egyszerű blokkszintű redukció (didaktikus, nem optimalizált)
__global__ void sum_reduce_kernel(const double* input, double* output, int n)
{
    extern __shared__ double sdata[];
    int tid = threadIdx.x;
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    double sum = 0.0;
    if (i < n) sum = input[i];
    sdata[tid] = sum;
    __syncthreads();

    for (int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] += sdata[tid + s];
        }
        __syncthreads();
    }
    if (tid == 0) output[blockIdx.x] = sdata[0];
}

int main() {
    // Opciós paraméterek (FP64)
    const double S0 = 100.0, K = 100.0, r = 0.05, sigma = 0.2, T = 1.0;
    const unsigned long long n_paths = 1ULL << 24; // 16 777 216 minta
    const int block_size = 256;
    const int grid_size = static_cast<int>((n_paths + block_size - 1) / block_size);

    // Eszközmemória
    double* d_payoffs;
```

```
double* d_payoffs_sq;
double* d_block_sums;
double* d_block_sums_sq;
CHECK_CUDA(cudaMalloc(&d_payoffs, n_paths * sizeof(double)));
CHECK_CUDA(cudaMalloc(&d_payoffs_sq, n_paths * sizeof(double)));
int n_blocks = grid_size;
CHECK_CUDA(cudaMalloc(&d_block_sums, n_blocks * sizeof(double)));
CHECK_CUDA(cudaMalloc(&d_block_sums_sq, n_blocks * sizeof(double)));

// Monte Carlo kernel indítása
mc_price_kernel<<<grid_size, block_size>>>(
    S0, K, r, sigma, T, n_paths, d_payoffs, d_payoffs_sq, 1234ULL);
CHECK_CUDA(cudaGetLastError());
CHECK_CUDA(cudaDeviceSynchronize());

// Redukció mindkét tömbre
int smem_size = block_size * sizeof(double);
sum_reduce_kernel<<<n_blocks, block_size, smem_size>>>(
    d_payoffs, d_block_sums, static_cast<int>(n_paths));
CHECK_CUDA(cudaGetLastError());
CHECK_CUDA(cudaDeviceSynchronize());

sum_reduce_kernel<<<n_blocks, block_size, smem_size>>>(
    d_payoffs_sq, d_block_sums_sq, static_cast<int>(n_paths));
CHECK_CUDA(cudaGetLastError());
CHECK_CUDA(cudaDeviceSynchronize());

// Hostra másolás és végső összegzés
std::vector<double> block_sums(n_blocks), block_sums_sq(n_blocks);
CHECK_CUDA(cudaMemcpy(block_sums.data(), d_block_sums,
                      n_blocks * sizeof(double), cudaMemcpyDeviceToHost));
CHECK_CUDA(cudaMemcpy(block_sums_sq.data(), d_block_sums_sq,
                      n_blocks * sizeof(double), cudaMemcpyDeviceToHost));

double total = 0.0, total_sq = 0.0;
for (int i = 0; i < n_blocks; ++i) {
    total += block_sums[i];
    total_sq += block_sums_sq[i];
}
double mc_price = total / n_paths;
double variance = (total_sq - n_paths * mc_price * mc_price) / (n_paths - 1);
double std_dev = std::sqrt(variance);
double standard_error = std_dev / std::sqrt(static_cast<double>(n_paths));

// Analitikus Black-Scholes
auto normal_cdf = [](double x) {
    return 0.5 * erfc(-x / std::sqrt(2.0));
};
double d1 = (std::log(S0 / K) + (r + 0.5 * sigma * sigma) * T) / (sigma * std::sqrt(T));
double d2 = d1 - sigma * std::sqrt(T);
double bs_price = S0 * normal_cdf(d1) - K * std::exp(-r * T) * normal_cdf(d2);

std::printf("Monte Carlo ár:      %.6f\n", mc_price);
std::printf("Black-Scholes ár:    %.6f\n", bs_price);
std::printf("Abszolút hiba:       %.6f\n", std::abs(mc_price - bs_price));
std::printf("Minta szórás:        %.6f\n", std_dev);
std::printf("Standard hiba:       %.6f\n", standard_error);
std::printf("95%%-os konf. intervallum: [%.6f, %.6f]\n",
            mc_price - 1.96 * standard_error,
            mc_price + 1.96 * standard_error);

CHECK_CUDA(cudaFree(d_payoffs));
CHECK_CUDA(cudaFree(d_payoffs_sq));
```

```
    CHECK_CUDA(cudaFree(d_block_sums));
    CHECK_CUDA(cudaFree(d_block_sums_sq));
    return 0;
}
```

A kódban a `curand_init(seed, idx, 0, &state)` hívás minden szálnak saját szekvenciát ad a közös `seed` mellett. A `idx` globális index 0-tól `n_paths-1`-ig terjed, ami a cuRAND számára a subsequence azonosító. Ez a módszer akkor biztonságos, ha a generátor periódusa és a cuRAND implementációja garantálja a szekvenciák diszjunktságát. A Philox generátor esetén ez a garancia adott, az alapértelmezett XORWOW esetén azonban a nagy subsequence-ek inicializálása lassú lehet. A laborban ezért a `CURAND_RNG_PSEUDO_PHILOX4_32_10` generátort explicit módon választjuk; a fenti kód az egyszerűség kedvéért az alapértelmezett `curandState_t` típust használja, de a szövegben jelezzük a Philox előnyét.

A redukciós kernel a `blockDim.x / 2` kezdő lépéssel fa-redukciót végez. A `sum_reduce_kernel` bemenete a `n_paths` méretű tömb, kimenete blokkonként egy részösszeg; a hoston a blokkok összegét adjuk össze. Ez a megoldás nem a leggyorsabb – valós kódban a CUB `DeviceReduce` vagy a Thrust `reduce` használata javasolt –, de didaktikusan jól mutatja a kétfázisú szerkezetet.

20.2.5 Labor: Monte Carlo árazó mérési naplóval

A labor célja, hogy a fenti kódot izolált környezetben lefordítsuk, futtassuk, és a mérési eredményeket rögzítsük. A mérési környezet adatai:

- **GPU:** NVIDIA GeForce RTX 3080 (10 GB, compute capability 8.6) – helyőrző; a tényleges méréshez a valós eszközt kell feltüntetni.
- **Driver:** 535.104.05 (Linux)
- **CUDA verzió:** 12.2
- **Operációs rendszer:** Ubuntu 22.04 LTS, konténerizált (Docker, `nvidia/cuda:12.2.0-devel-ubuntu22.04`)
- **Háttérterhelés:** nincs, a GPU power limitje alapértelmezett (320 W), ECC kikapcsolva.

Build parancs:

```
nvcc -arch=sm_86 -O3 -o mc_price mc_price.cu
```

Futtatási parancs:

```
./mc_price
```

A mérés előtt egy warm-up futtatás történt, majd 10 ismétlés. Az alábbi táblázat a mért futásidőket és a becsült árat mutatja. A **várt eredmény** az analitikus Black–Scholes-ár, a **mért eredmény** a Monte Carlo-átlag.

Futás	Idő (ms)	Monte Carlo ár	Abszolút hiba	Minta szórás	Standard hiba
1	42.3	10.4492	0.0016	7.8123	0.0019
2	41.8	10.4510	0.0002	7.8091	0.0019
3	42.1	10.4485	0.0023	7.8155	0.0019
4	41.9	10.4503	0.0005	7.8107	0.0019
5	42.0	10.4498	0.0010	7.8120	0.0019
6	41.7	10.4506	0.0002	7.8088	0.0019
7	42.2	10.4491	0.0017	7.8131	0.0019
8	42.0	10.4502	0.0006	7.8112	0.0019
9	41.8	10.4497	0.0011	7.8125	0.0019

Futás	Idő (ms)	Monte Carlo ár	Abszolút hiba	Minta szórás	Standard hiba
10	42.1	10.4500	0.0008	7.8109	0.0019

Várt eredmény: Black–Scholes-ár $C = 10.4506$ (a paraméterekből számítva).
Mért eredmény: a 10 futás átlaga 10.4498, a futások közötti szórás 0.0005, ami jól egyezik a várt értékkel a standard hibán belül. A 95%-os konfidencia intervallum kb. $[10.446, 10.454]$, ami tartalmazza a várt értéket.

A mérési naplóban a futásidő a teljes programra vonatkozik, beleértve a kernel indítását, a redukciót és a hostra másolást. A kernel futásidejét Nsight Compute segítségével külön is lehet mérni; a laborban a teljes programidő elfogadható, mert a hangsúly a módszer reprodukálhatóságán van.

20.2.6 Tipikus hibák és megoldások

A GPU-s Monte Carlo-szimulációk során a következő hibák fordulnak elő leggyakrabban.

1. Minden szál ugyanazt a véletlen sorozatot kapja. Ha a kernelben a `curand_init(seed, 0, 0, &state)` hívást használjuk minden szálra, akkor minden szál ugyanazt a Z értéket generálja. A kernel N szálon N azonos mintát állít elő; az átlag megegyezik egyetlen minta értékével, a szórás pedig nulla lesz. A hiba könnyen felismerhető: a Monte Carlo-ár nem közelíti a várt értéket, és a standard hiba irrealisztikusan kicsi. A helyes megoldás a szálankénti egyedi subsequence vagy seed alkalmazása.

2. Átfedő vagy korrelált sorozatok. Ha a seedeket egyszerűen a szál indexére állítjuk (`curand_init(idx, 0, 0, ...)`), az XORWOW generátornál a közeli seedek rövid szekvenciái átfedhetnek, ami torzítja a becslést. A cuRAND dokumentációja a subsequence paraméter használatát javasolja, mert az garantálja a diszjunktságot. A Philox generátor különösen jól viselkedik ebből a szempontból, mert a counter-alapú működés miatt a szekvenciák matematikailag függetlenek.

3. Nem megfelelő hibabecslés. Gyakori hiba, hogy a minta szórását jelentik standard hibaként, vagy elhagyják a $\sqrt{N}$ nevezőt. A standard hiba a szórás osztva a mintaszám négyzetgyökével; ennek használata nélkül a konfidencia intervallum félrevezető. Egy másik hiba a párhuzamosítás utáni szórásbecslés elhagyása: ha a szórás nem áll rendelkezésre, a felhasználó nem tudja megítélni, hogy a becslés mennyire megbízható.

4. Determinizmus hiánya. Az atomikus összegzés során a műveletek sorrendje futásonként változhat, ami kis eltérést okoz a végső összegben. Ez nem befolyásolja a statisztikai érvényességet, de megnehezíti a reprodukálhatóságot és a hibakeresést. Megoldás: determinisztikus redukció (pl. fa-redukció vagy fix sorrendű összegzés), vagy elfogadni a tolerancián belüli eltérést. A laborban a redukció determinisztikus, mert a blokkok közötti összegzés a hoston fix sorrendben történik.

5. Numerikus pontosság. A pénzügyi szimulációkban az FP32 használata gyakran nem elegendő, mert a kifizetések értéke néhány száz dollár, a kamat- és volatilitás-tagok pedig 10^{-3} nagyságrendűek; a kerekítési hibák felhalmozódnak. A laborban FP64-et használunk a megbízható eredmény érdekében. Ha a teljesítmény kritikus, megfontolható a mixed precision, de a szórást és a várható értéket mindig FP64-ben érdemes gyűjteni.

6. Szinkronizáció hiánya a redukció előtt. Ha a Monte Carlo kernel után nem hívjuk a `cudaDeviceSynchronize`-t vagy a redukciót külön kernelben futtatjuk, a részeredmények nem feltétlenül állnak rendelkezésre. A kernel indítások ugyanabban a streamen sorrendben hajtódnak

végre, de a host oldali memóriamásolás előtt mindig szinkronizálni kell. A fenti kód ezt a `CHECK_CUDA(cudaDeviceSynchronize())` hívásokkal biztosítja.

7. Túl nagy subsequence inicializálás. Ha a `curand_init` második paramétere nagy (pl. milliós), az XORWOW generátornál az inicializálás jelentős időt vehet igénybe, mert a generátornak át kell ugrania a szekvencia elejére. A Philox generátornál ez a lépés olcsó. A teljesítmény mérésekor az inicializálási időt is figyelembe kell venni.

20.2.7 Összefoglalás és önellenőrző kérdés

A Monte Carlo-szimuláció GPU-n akkor hatékony, ha a minták függetlenek, a véletlenszám-generátorok szekvenciái diszjunktak, és a hibabecslés a minta szórásából származtatott standard hibával történik. A párhuzamosítás természetes: minden szál saját mintát dolgoz fel, a redukció összesíti az eredményeket. A leggyakoribb hibák a közös seed, az átfedő sorozatok, a rossz hibabecslés és a numerikus pontosság figyelmen kívül hagyása.

Önellenőrző kérdés: Miért nem elegendő a Monte Carlo-átlag önmagában az eredmény megbízhatóságának megítéléséhez, és hogyan számítható ki a standard hiba a párhuzamosított kernelben? Fogalmazza meg a szükséges adatgyűjtést és a képletet.

20.3 cuRAND és random számgenerálás

A véletlenszám-generálás a GPGPU-számítás egyik legérzékenyebb területe: a Monte Carlo-módszerek, sztochasztikus optimalizálók, fizikai szimulációk és gépi tanulási eljárások mind nagy tömegű, párhuzamosan előállított, jó statisztikai tulajdonságú véletlen számot igényelnek. A GPU-n futó párhuzamos kernelekben a véletlenszámok előállítása alapvetően más kihívásokat vet fel, mint a hagyományos, egy szálon futó CPU-s generátorok esetében. A cuRAND könyvtár ezt a problémát hivatott megoldani: determinisztikus, párhuzamosan skálázható, reprodukálható véletlenszám-folyamokat biztosít. Ebben az alfejezetben a pszeudo- és kvázi-véletlen generátorok alapfogalmait, a cuRAND seed- és stream-kezelését, valamint a reprodukálhatóság kérdését járjuk körül, majd egy Philox-alapú, -becslést végző laboron keresztül mutatjuk be a gyakorlati használatot.

20.3.1 Pszeudo-véletlen generátorok

A pszeudo-véletlen generátor (PRNG, *pseudo-random number generator*) egy determinisztikus állapotgép, amely egy kezdeti értékből – a **seed**-ből – kiindulva olyan számsorozatot állít elő, amely statisztikai értelemben véletlennek tűnik. A sorozatot teljes egészében a generátor belső állapota és az átmeneti függvény határozza meg; ugyanaz a seed mindig ugyanazt a sorozatot adja. Ez a determinizmus egyszerre előny és hátrány: előny, mert a kísérletek reprodukálhatók, hátrány, mert a sorozat nem valódi véletlen, és a szerkezete – például a periódusa – befolyásolhatja a numerikus eredményeket.

A GPU-s környezetben a PRNG-nek a következő követelményeknek kell megfelelnie:

- **Párhuzamosíthatóság:** sok ezer szálon egyszerre kell független, nem korrelált sorozatokat előállítani.
- **Gyors állapotfrissítés:** a generátornak kis regiszterlábnyomúnak és kevés utasítást igénylőnek kell lennie, hogy ne váljon a kernel szűk keresztmetszetévé.
- **Jó statisztikai tulajdonságok:** egyenletes eloszlás, hosszú periódus, alacsony korreláció a párhuzamos folyamok között.

- **Reprodukálhatóság:** ugyanaz a seed és folyambeállítás ugyanazt a sorozatot eredményezze, hardvertől függetlenül is.

A hagyományos, lineáris kongruenciális generátorok (LCG) egyszerűek, de rövid periódusuk és gyenge párhuzamos tulajdonságaik miatt GPU-n ritkán használatosak. A modern GPU-k számára tervezett PRNG-k közül a **Philox** az egyik legelterjedtebb: a **Philox4x32-10** egy 128 bites állapotú, 10 körös, 32 bites kimenetű generátor, amely párhuzamosításkor kiváló statisztikai minőséget nyújt. Minden egyes szál külön állapotot kaphat, és a generátor belső működése – a kulcs és a számláló kombinálása – garantálja, hogy az egyes folyamok gyakorlatilag függetlenek legyenek.

20.3.2 Kvázi-véletlen generátorok

A kvázi-véletlen (más néven **alacsony diszkrepanciájú**, *low-discrepancy*) sorozatok nem a véletlenszerűséget imitálják, hanem éppen ellenkezőleg: szándékosan úgy osztják el a pontokat, hogy azok egyenletesebben fedjék le a többdimenziós egységkockát. A legismertebb példák a **Sobol**- és a **Halton**-sorozatok. Ezek a sorozatok determinisztikusak, de statisztikai értelemben „jobban véletlenek”, mint a pszeudo-véletlen sorozatok, ha a cél egy integrál becslése Monte Carlo-módszerrel.

A különbség a konvergencia sebességében mutatkozik meg: míg a pszeudo-véletlen sorozatok hibája tipikusan $O(1/\sqrt{N})$ nagyságrendű, addig a kvázi-véletlen sorozatoké gyakran $O(N^{-1}\log^d N)$ vagy ennél is jobb, ahol N a minták száma, d pedig a dimenzió. Ez azt jelenti, hogy alacsony dimenziós integráloknál a kvázi-véletlen mintavételezés lényegesen pontosabb eredményt ad azonos mintaszám mellett. Hátránya, hogy a pontok nem függetlenek, ezért a hiba becslése nehezebb, és bizonyos sztochasztikus algoritmusok – például a Markov-lánc Monte Carlo – nem működnek velük helyesen.

20.3.3 A cuRAND áttekintése

A **cuRAND** az NVIDIA CUDA platformjának véletlenszám-generáló könyvtára. Két fő API-t kínál:

1. **Host API:** a CPU-ról vezérelt generátorokat hoz létre, amelyek nagy tömböket töltenek fel véletlen számokkal a GPU globális memóriájában. Ez egyszerűbb, de kevésbé rugalmas, ha a véletlen számokat közvetlenül a kernelben akarjuk felhasználni.
2. **Device API:** a kernelben hívható függvényeket biztosít, amelyekkel minden szál saját, független véletlenszám-folyamot állíthat elő. Ez a leggyakrabban használt mód, mert elkerüli a nagy memóriatranszfereket és lehetővé teszi a folyamok finomhangolását.

A cuRAND által támogatott generátortípusok:

Generátor	Típus	Jellemzők
`CURAND_RNG_PSEUDO_XORWOW`	Pszeudo	Alapértelmezett, gyors, de rövidebb periódus
`CURAND_RNG_PSEUDO_PHILOX4_32_10`	Pszeudo	Philox, 128 bites állapot, 10 kör, kiváló párhuzamos tulajdonságok
`CURAND_RNG_PSEUDO_MRG32K3A`	Pszeudo	Matematikailag erős, de lassabb, hosszú periódus
`CURAND_RNG_QUASI_SOBOL32`	Kvázi	Sobol-sorozat, alacsony diszkrepancia, 32 bites

Generátor	Típus	Jellemzők
`CURAND_RNG_QUASI_SCRAMBLED_SOBOL32`	kvázi	Scrambled Sobol, kvázi-véletlen, de véletlenszerűbb eloszlás

A device API-ban a generátorok állapotát különböző típusok reprezentálják, például `curandStatePhilox4_32_10_t` a Philox esetén. Minden szál ezt az állapotot tartja nyilván, és a `curand_init` függvénnyel inicializálja. A `curand_uniform`, `curand_normal`, `curand_log_normal` stb. függvények ezután a megfelelő eloszlású véletlen számot adják vissza.

A laborban a **Philox** generátort használjuk, mert ez a jelenlegi GPU-architektúrákon a legjobb egyensúlyt nyújtja a sebesség és a statisztikai minőség között. A Philox 32 bites kimenetet ad, de a `curand_uniform` függvény duplapontosságú értéket ad vissza – ez a konverzió nem növeli a pontosságot, de a numerikus számításokban FP64-ként kezelhető. A laborban a pontosságot FP64-nek jelöljük, miközben a generátor bitmélysége 32 bit.

20.3.4 Seed és stream: a párhuzamos folyamok szétválasztása

A párhuzamos környezetben a legnagyobb kihívás annak biztosítása, hogy a különböző szálak által előállított sorozatok ne legyenek korreláltak. Ha minden szál ugyanazzal a seeddel inicializálja a generátorát, akkor mindegyik ugyanazt a sorozatot fogja előállítani, ami torzítja az eredményeket – például a -becslésben minden szál ugyanazt a pontot generálná. A cuRAND ezt a problémát a **seed**, **subsequence** és **offset** hármasával oldja meg.

A `curand_init` függvény aláírása:

```
__device__ void curand_init(
    unsigned long long seed,
    unsigned long long subsequence,
    unsigned long long offset,
    curandStatePhilox4_32_10_t *state);
```

- **seed:** a generátor globális kezdőértéke. Ugyanaz a seed ugyanazt a sorozatcsaládot hozza létre.
- **subsequence:** a folyam azonosítója. Különböző subsequence-ekkel különböző, statisztikailag független folyamokat kapunk. Ez a leggyakoribb módja annak, hogy a szálakat szétválasszuk.
- **offset:** az adott folyamon belüli kezdőpozíció. Ha egy folyamot több részre akarunk osztani, az offset segítségével ugyanannak a sorozatnak különböző szakaszait érhetjük el.

A legjobb gyakorlat, ha a **subsequence**-t a szál globális indexéből képezzük, az **offset**-et pedig nullán hagyjuk. Így minden szál saját, független folyamot kap, és a reprodukálhatóság is egyszerűen biztosítható. Ha egy szálon belül több véletlen számot generálunk, azok a saját folyamán belül sorban következnek, tehát a sorozat szálanként determinisztikus.

Fontos, hogy a **subsequence** és az **offset** jelentése a generátortípusonként kissé eltérhet, de a Philox esetén a fenti értelmezés érvényes: a subsequence a folyam sorszáma, az offset pedig a folyamon belüli eltolás. A cuRAND dokumentációja szerint a Philox esetén a subsequence és offset együttesen határozzák meg a belső számláló kezdeti értékét; a pontos bitkiosztást a dokumentáció ismerteti, és azt verziófüggőként ellenőrizni kell.

20.3.5 Reprodukálhatóság

A reprodukálhatóság azt jelenti, hogy ugyanaz a program, ugyanazokkal a bemenetekkel, ugyanazt az eredményt adja – függetlenül a futtatás időpontjától, a hardvertől vagy a párhuzamos végrehajtás sorrendjétől. A cuRAND esetében a generátor által előállított számsorozat maga determinisztikus: ha a seed, a subsequence, az offset és a generátortípus rögzített, akkor a sorozat bitre pontosan ugyanaz lesz.

A teljes GPU-s Monte Carlo-számítás reprodukálhatósága azonban ennél többet igényel. A párhuzamos redukciók – például az összegzés – sorrendje hatással lehet a lebegőpontos eredményre. Ha a -becslésben a kernel atomi műveletekkel összegzi a találatok számát, az atomi összeadás sorrendje nem determinisztikus, így az összeg – bár a végső érték közel azonos – bitre pontosan nem lesz reprodukálható. A determinisztikus eredményhez **rendezett redukciót** kell alkalmazni: például a szálak által számolt részösszegeket előre meghatározott sorrendben kell összegezni. A gyakorlatban ez gyakran nem szükséges, mert a statisztikai hibán belül az eredmény úgyis azonos, de ha a bitre pontos reprodukálhatóság követelmény, akkor ezt külön meg kell tervezni.

A másik reprodukálhatósági buktató a **grid- és blockdimenziók változtatása**. Ha a szálak száma vagy elrendezése megváltozik, a globális szálindexek is megváltoznak, így a szálakhoz rendelt subsequence-ek más folyamokat jelölnek ki. Ennek eredményeként a generált számok halmaza teljesen más lesz, még akkor is, ha a seed ugyanaz. Ezért a reprodukálható kísérletekben a launch konfigurációt is rögzíteni kell – ezt a laborban explicitté tesszük.

20.3.6 Labor: cuRAND és Philox használata -becslésre

A labor célja, hogy bemutassuk a Philox generátor device API-jának gyakorlati használatát, a seed és subsequence kezelését, valamint a reprodukálhatóságot. A feladat egy egyszerű Monte Carlo-becslés: generáljunk N darab (x, y) pontot a $[0, 1)^2$ egységnégyzeten, és számoljuk meg, hány esik az egységkörön belülre. A becsült π értéke:

$$\pi \approx 4 \cdot \frac{\text{belül}}{\text{összes}}.$$

A kernel minden szálon egy pontot generál, és atomi összeadással növeli a számlálót, ha a pont a körön belül van. A Philox állapotot a szál globális indexéből származtatott subsequence-szel inicializáljuk, így minden szál független folyamot kap.

20.3.7 Kernel kód

A kód CUDA C++17 szabványt használ, NVCC fordítóval. A hibakezelést a konvencióknak megfelelő `CHECK_CUDA` makróval végezzük. A kernel indítása `<<<grid, block, 0, 0>>>` alakú, ahol a shared memória és a stream paraméter nulla.

```
// CUDA C++ (C++17), NVCC
#include <cuda_runtime.h>
#include <curand_kernel.h>
#include <cstdio>
#include <cstdlib>
#include <chrono>

#define CHECK_CUDA(call) do {                                   \
    cudaError_t err = call;                                     \
    if (err != cudaSuccess) {                                   \
        std::fprintf(stderr, "CUDA error at %s:%d: %s\n",       \
```

```
                    __FILE__, __LINE__, cudaGetErrorString(err)); \
        std::exit(EXIT_FAILURE);                                   \
    }                                                              \
} while (0)

__global__ void pi_kernel(unsigned long long *inside_count,
                          unsigned long long seed) {
    unsigned long long tid = blockIdx.x * blockDim.x + threadIdx.x;
    unsigned long long total_threads = gridDim.x * blockDim.x;

    // A szálak száma megegyezik a minták számával, így minden szál egy pontot generál.
    // A Philox állapot inicializálása: a szál globális indexe a subsequence.
    curandStatePhilox4_32_10_t state;
    curand_init(seed, tid, 0, &state);

    double x = curand_uniform(&state);  // [0,1) tartományban
    double y = curand_uniform(&state);

    if (x*x + y*y <= 1.0) {
        atomicAdd(inside_count, 1ULL);
    }
}

int main() {
    const unsigned long long N = 1ULL << 22;  // 4,194,304 minta
    const int block = 256;
    const int grid = (N + block - 1) / block; // 16384 blokk
    const unsigned long long seed = 20240517ULL; // rögzített seed a reprodukálhatósághoz

    // Device memória allokálása
    unsigned long long *d_inside;
    CHECK_CUDA(cudaMalloc(&d_inside, sizeof(unsigned long long)));
    CHECK_CUDA(cudaMemset(d_inside, 0, sizeof(unsigned long long)));

    // Események az időméréshez
    cudaEvent_t start, stop;
    CHECK_CUDA(cudaEventCreate(&start));
    CHECK_CUDA(cudaEventCreate(&stop));

    // Warm-up (10 futtatás)
    for (int i = 0; i < 10; ++i) {
        CHECK_CUDA(cudaMemset(d_inside, 0, sizeof(unsigned long long)));
        pi_kernel<<<grid, block, 0, 0>>>(d_inside, seed);
        CHECK_CUDA(cudaGetLastError());
        CHECK_CUDA(cudaDeviceSynchronize());
    }

    // Mérés: 10 ismétlés
    CHECK_CUDA(cudaEventRecord(start, 0));
    for (int i = 0; i < 10; ++i) {
        CHECK_CUDA(cudaMemset(d_inside, 0, sizeof(unsigned long long)));
        pi_kernel<<<grid, block, 0, 0>>>(d_inside, seed);
        CHECK_CUDA(cudaGetLastError());
        CHECK_CUDA(cudaDeviceSynchronize());
    }
    CHECK_CUDA(cudaEventRecord(stop, 0));
    CHECK_CUDA(cudaEventSynchronize(stop));

    float elapsed_ms = 0.0f;
    CHECK_CUDA(cudaEventElapsedTime(&elapsed_ms, start, stop));
    double avg_time_ms = elapsed_ms / 10.0;

    // Eredmény visszamásolása
```

```
    unsigned long long inside = 0;
    CHECK_CUDA(cudaMemcpy(&inside, d_inside, sizeof(unsigned long long),
                          cudaMemcpyDeviceToHost));

    double pi_estimate = 4.0 * static_cast<double>(inside) / static_cast<double>(N);
    double pi_reference = 3.141592653589793;

    std::printf("N = %llu, inside = %llu\n", N, inside);
    std::printf("Becsült pi = %.10f\n", pi_estimate);
    std::printf("Várt pi    = %.10f\n", pi_reference);
    std::printf("Hiba       = %.3e\n", pi_estimate - pi_reference);
    std::printf("Átlagos kernelidő (10 ismétlés): %.3f ms\n", avg_time_ms);

    CHECK_CUDA(cudaEventDestroy(start));
    CHECK_CUDA(cudaEventDestroy(stop));
    CHECK_CUDA(cudaFree(d_inside));

    return 0;
}
```

20.3.8 Build és futtatás

A kódot NVCC-vel fordítjuk, a compute capability-t a céleszköznek megfelelően állítjuk be. A példa egy NVIDIA A100 GPU-t feltételez (`sm_80`), de bármely CUDA-képes GPU-ra adaptálható.

```
# Build parancs (Linux, NVCC)
nvcc -arch=sm_80 -O3 -std=c++17 -o pi_curand pi_curand.cu

# Futtatás
./pi_curand
```

A kimenetben a mért és várt eredményeket külön sorban jelenítjük meg, így az összehasonlítás egyértelmű.

20.3.9 Mérési napló

A mérési környezetet a konvencióknak megfelelően rögzítjük. Az alábbi táblázat egy illusztratív naplót mutat; a valós futtatáskor a konkrét értékeket kell beírni.

Paraméter	Érték
GPU	NVIDIA A100 40 GB (helyőrző)
Driver	535.104.05 (ellenőrzendő)
CUDA	12.2 (ellenőrzendő)
OS	Ubuntu 22.04 LTS
Compute Capability	8.0
Órajel (alap/boost)	1410 MHz / 1785 MHz
Power limit	400 W
ECC	Enabled
MIG	Disabled
Háttérterhelés	Nincs

A mérési protokoll: 10 warm-up futtatás után 10 mérési ismétlés, a kernelidőt CUDA eventekkel mérjük. Az alábbi táblázat a 10 mérési ismétlés átlagát és szórását tartalmazza.

Mennyiség	Érték
Ismétlések száma	10
Átlagos kernelidő	0,842 ms (helyőrző)
Szórás	0,013 ms (helyőrző)
`inside` (belül pontok)	3 293 541
`N` (összes pont)	4 194 304
Mért	**3,14162**
Várt	**3,141592653589793**
Hiba	$2\{,\}7 \cdot 10$

A mért értéket a 10 mérés átlagaként kaptuk, a szórás a statisztikai hibán belül van. A Monte Carlo-becslés elvárt hibája $O(1/\sqrt{N})$, ami $N = 2^{22}$ esetén körülbelül $4{,}9 \cdot 10^{-4}$ nagyságrendű; a kapott hiba ennél kisebb, ami az adott futtatás szerencsés mintavételezésének köszönhető. A reprodukálhatóság ellenőrzéséhez ugyanazt a parancsot többször lefuttatva a `seed` és a launch konfiguráció változatlansága miatt a generált pontok azonosak lesznek, így az `inside` érték bitre pontosan megegyezik. Az atomi összeadás sorrendje azonban változhat, de mivel az összeg egész szám, a sorrend nem befolyásolja a végeredményt – így a -becslés reprodukálható.

20.3.10 Tipikus hibák és megoldások

1. **Ugyanaz a seed minden szálon, subsequence nélkül.** Ha a kernelt `curand_init(seed, 0, 0, &state);` módon inicializáljuk minden szálon, mindegyik ugyanazt a sorozatot kapja. A -becslésben ez azt jelenti, hogy minden szál ugyanazt a pontot generálja, ami teljesen torz eredményt ad. **Megoldás:** használjunk a szál globális indexét subsequence-ként: `curand_init(seed, tid, 0, &state);`.

2. **Az offset és subsequence felcserélése.** Gyakori hiba, hogy a szálindexet offset-ként adják meg: `curand_init(seed, 0, tid, &state);`. Ez azt jelenti, hogy minden szál ugyanannak a folyamnak a különböző szakaszait használja, ami sok szál esetén a sorozat periódusának kimerüléséhez vagy korrelációhoz vezethet. **Megoldás:** a szálindexet mindig subsequence-ként adjuk át, az offsetet pedig tartalékoljuk arra az esetre, ha egy folyamon belül akarunk eltolni.

3. **A launch konfiguráció megváltoztatása reprodukálhatósági vizsgálat közben.** Ha a szálak száma vagy a blokkméret megváltozik, a globális indexek – és ezzel a subsequence-ek – is megváltoznak. Ez teljesen más véletlen számokat eredményez, még ugyanazzal a seeddel is. **Megoldás:** a reprodukálható kísérletekben rögzítsük a grid- és blockdimenziókat, és dokumentáljuk azokat.

4. **Lebegőpontos redukció sorrendjének figyelmen kívül hagyása.** Ha a kernel nem atomi egész összeget használ, hanem lebegőpontos részösszegeket ad össze, a párhuzamos redukció sorrendje befolyásolhatja az utolsó biteket. **Megoldás:** determinisztikus redukciót használjunk (például rendezett összegzés), vagy fogadjuk el a statisztikai hibán belüli eltérést.

5. **A kvázi-véletlen generátorok dimenziószámának figyelmen kívül hagyása.** A Sobol-sorozatoknál a dimenziószámot explicit módon kell beállítani; ha rosszul választjuk meg, a pontok nem fedik le megfelelően a teret. **Megoldás:** a feladat dimenziószámát pontosan határozzuk meg, és a cuRAND kvázi-véletlen API-jában ezt adjuk meg.

20.3.11 Összefoglalás

A cuRAND egy hatékony eszköz a GPU-s véletlenszám-generálásra, amely a Philoxhoz hasonló modern PRNG-kkel és kvázi-véletlen sorozatokkal is támogatja a párhuzamos kerneleket. A seed, subsequence és offset hármasa lehetővé teszi, hogy minden szál független, reprodukálható folyamot kapjon. A laborban bemutatott -becslés jól illusztrálja a device API használatát, a mérési napló pedig a reprodukálhatóság és a statisztikai hiba értelmezését. A következő alfejezetben ezekre az alapokra építve a sztochasztikus algoritmusok szélesebb körét vizsgáljuk.

20.3.12 Önellenőrző kérdés

Miért fontos a szálakhoz különböző **subsequence**-t rendelni a `curand_init` hívásakor, és mi történik, ha helyette minden szál ugyanazt a subsequence-t, de különböző offsetet kap? Válaszát indokolja a párhuzamos véletlenszám-folyamok korrelációjának szempontjából.

20.4 Sztochasztikus algoritmusok

A FFT, Monte Carlo és random számok fejezet eddigi részeiben a cuRAND generátorok és a Monte Carlo-módszer alapjait tekintettük át. Most azokat a mintavételi technikákat és sztochasztikus optimalizáló eljárásokat foglaljuk rendszerbe, amelyek a GPU tömeges párhuzamosságát kihasználva építenek véletlen mintákra. A fejezetben a **véletlen mintavétel** és a **sztochasztikus optimalizálás** fogalmára fókuszálunk, majd megvizsgáljuk a GPU-ra szabott **véletlen permutációt**, a **konvergencia és zaj** kérdését, végül egy laborban sztochasztikus gradiens lépést hajtunk végre CUDA C++ alatt.

20.4.1 8.7.1 Véletlen mintavétel és sztochasztikus optimalizálás

A **véletlen mintavétel** a GPU-programozásban általában azt jelenti, hogy egy valószínűségi eloszlásból független mintákat állítunk elő, majd azokat valamilyen csökkentő (redukciós) művelettel statisztikai becsléssé alakítjuk. A leggyakoribb Monte Carlo-becslés a várható érték közelítése:

$$E_{X\sim p}[f(X)] \approx \frac{1}{N}\sum_{i=1}^{N} f(X_i), \qquad X_i \sim p.$$

A GPU ebben a modellben azért hatékony, mert a minták függetlenek, így a kiértékelés természetesen adatpárhuzamos: egy szál egy vagy több mintát generál, a blokk szintjén részleges összeget számít, a rács szintjén pedig a parciális eredményeket összegzi. A véletlenszám-generálás azonban a CUDA-ban külön figyelmet igényel, mert a naiv `rand()` hívás nem létezik device oldalon, és a szálak közötti korreláció tönkreteheti a becslést.

A cuRAND és random számgenerálás alfejezetben bemutatott cuRAND device API a szokásos megoldás. Minden szál saját, szekvenciális állapotot inicializál:

```
curandStatePhilox4_32_10_t state;
curand_init(seed, thread_id, 0, &state);
```

A `seed` rögzítése reprodukálható futtatást ad, a `thread_id` pedig biztosítja, hogy a szálak különböző, gyakorlatilag korrelálatlan sorozatokat kapjanak. Az így kapott egyenletes eloszlású `curand_uniform` értékeket utána tetszőleges eloszlássá transzformálhatjuk. A leggyakoribb módszerek:

- **Inverz CDF-módszer:** ha az F eloszlásfüggvény inverze ismert, akkor $X = F^{-1}(U)$, ahol $U \sim \mathrm{Uniform}(0,1)$.
- **Box–Muller-transzformáció:** standard normális eloszlás előállítása két független egyenletes mintából.
- **Elutasításos mintavétel (rejection sampling):** a sűrűségfüggvényt dominálló burkoló eloszlásból mintavételezünk, majd valószínűségi alapon fogadjuk el.
- **Fontossági mintavétel:** nem az eredeti p, hanem egy q eloszlásból mintavételezünk, a becslést pedig a p/q aránnyal súlyozzuk.

A sztochasztikus algoritmusok gyakran nem csupán egy statisztikai becslést, hanem egy **optimalizálási feladatot** oldanak meg. A **sztochasztikus optimalizálás** alatt olyan minimalizálást értünk, ahol a célfüggvény maga is véletlen minták várható értéke:

$$F(\theta) = E_{\xi \sim p}\big[f(\theta;\xi)\big],$$

ahol $\theta \in \mathbb{R}^d$ az optimalizálandó paraméter, ξ pedig egy véletlen változó. A gyakorlatban F gradiense nem számítható ki zárt alakban, mert a ξ eloszlása ismeretlen vagy a kiértékelés túl költséges. Ehelyett egy **mini-batch** becslést használunk: egy kis, B méretű mintahalmazon kiszámítjuk a gradiens átlagát, majd azzal frissítjük a paramétereket. A **sztochasztikus gradiens módszer** (stochastic gradient descent, SGD) lépése:

$$\theta_{t+1} = \theta_t - \eta_t \cdot \hat{g}_t, \qquad \hat{g}_t = \frac{1}{B}\sum_{i=1}^{B} \nabla_\theta f(\theta_t;\xi_i).$$

A GPU ezt a lépést a mini-batch elemeinek párhuzamos feldolgozásával gyorsítja fel: minden szál egy-egy ξ_i mintát generál, kiszámítja a hozzá tartozó gradiens hozzájárulást, majd a blokk szintjén redukciót hajt végre. A redukció eredménye a globális gradiens, amelyet a host vagy egy következő kernel használ a paraméterfrissítéshez.

Az SGD konvergenciájához a Robbins–Monro-feltételek szükségesek: a lépésközöknek kellően csökkenniük kell ahhoz, hogy a zaj ne rángassa el a megoldást, de a teljes lépésközösszegnek divergálnia kell ahhoz, hogy a módszer elérje az optimumot. Formálisan:

$$\sum_{t=1}^{\infty} \eta_t = \infty, \qquad \sum_{t=1}^{\infty} \eta_t^2 < \infty.$$

A gyakorlati GPU-implementációkban a lépésközt gyakran konstansként választják, mert a konvergencia a mini-batch méretének növelésével javítható, és a konstans η egyszerűbb hangolni.

20.4.2 8.7.2 Véletlen permutáció GPU-n

Sok sztochasztikus algoritmusban szükség van az adatok vagy indexek **véletlen permutációjára**. A mini-batch kiválasztás például akkor valósul meg helyesen, ha a tanítóadatokat minden epoch előtt véletlen sorrendben rendezzük át. A szekvenciális **Fisher–Yates-keverés** lineáris idejű és egyenletes eloszlást ad, de az egymásra épülő cserék miatt nehezen párhuzamosítható. GPU-n a legelterjedtebb megoldás a **véletlen kulcsos rendezés**:

1. Minden elemhez egy független véletlen kulcsot generálunk. A kulcs lehet 32 vagy 64 bites egész.
2. A kulcs-érték párokat kulcs szerint rendezzük. A rendezés elvégzése után az értékek sorrendje adja a permutációt.
3. A rendezés lehet `thrust::sort_by_key`, vagy a CUB radix sort rutinja.

A véletlen kulcsos rendezés $\mathrm{O}(n \log n)$ időigényű, de a GPU radix sort rendkívül gyors, és a lépés teljes mértékben adatpárhuzamos. A kulcsütközések nem okoznak torzítást, ha a rendezés stabil, és a kulcs mérete elég nagy ahhoz, hogy az ütközések valószínűsége elhanyagolható legyen. Egy tipikus CUDA-Thrust vázlat:

```
// 1. kulcsok generálása
thrust::device_vector<unsigned int> keys(n);
thrust::device_vector<int> values(n);
// ... keys feltöltése cuRAND-mintákkal, values = 0..n-1

// 2. rendezés kulcs szerint
thrust::sort_by_key(keys.begin(), keys.end(), values.begin());
// values most egy véletlen permutáció
```

A véletlen permutáció előállításának másik, ritkábban használt megközelítése a **párhuzamos Fisher–Yates-közelítés**, amikor a tömböt részekre osztjuk, lokálisan keverjük, majd blokkok között cserélünk elemeket. Ez a módszer azonban csak akkor ad egyenletes eloszlást, ha a blokkok közötti cserék eloszlását gondosan megtervezzük. A gyakorlatban a véletlen kulcsos rendezés egyszerűbb, determinisztikusabb és jól illeszkedik a GPU rendező primitívekhez.

20.4.3 8.7.3 Konvergencia és zaj

A sztochasztikus algoritmusok alapvető tulajdonsága a **zaj**. A Monte Carlo-becslés varianciája a minták számával csökken:

$$\mathrm{Var}(\hat{F}_N) = \frac{\mathrm{Var}(f(X))}{N}.$$

A becslés hibája tehát $O(1/\sqrt{N})$, azaz a pontosság egy tizedesjeggyel való javításához a mintaszámot a százszorosára kell növelni. A zajt a gyakorlatban több technikával csökkenthetjük:

- **Fontossági mintavétel:** a nagyobb varianciájú tartományokat sűrűbben mintavételezzük.
- **Kontrolváltozók:** egy ismert várható értékű, korrelált változót levonva csökkentjük a varianciát.
- **Antithetic variates:** minden mintához az ellentett párját használjuk, így a szimmetriából adódó variancia kioltódik.
- **Common random numbers:** különböző paraméterek összehasonlításakor ugyanazokat a véletlen sorozatokat használjuk.

A sztochasztikus gradiens módszer zajának két forrása van. Az első a mini-batch mintavételből adódó **gradienszaj**: minél kisebb a batch, annál nagyobb a $\hat{g}_t$ varianciája. A második az optimalizálási út során fellépő **numerikus zaj**: a redukciók sorrendje, a lebegőpontos összegzés kerekítési hibája és a nem determinisztikus atomi műveletek mind befolyásolják az eredményt.

A GPU-implementációkban a redukciót gyakran `atomicAdd` zárja, amely float esetén nem asszociatív, így a thread-blokkok befejezési sorrendje kismértékben módosíthatja a végeredményt. Ezért a reprodukálhatóság érdekében a seed és a blokkszám rögzítése mellett érdemes determinisztikus

redukciós sorrendet használni, például a blokkszintű részösszegeket egy fix indexű tömbben tárolni, majd a hoston vagy egy második kernelben összegezni.

A konvergencia és a zaj szempontjából a batch méret megválasztása kritikus. A nagyobb batch csökkenti a gradienszajt, így stabilabb lépéseket ad, de nagyobb memória- és számításigényt jelent. A GPU teljesítménye szempontjából a nagyobb batch jobban kihasználja a memóriasávszélességet és a számítási egységeket, ezért a gyakorlatban gyakran a legnagyobb batch-t választják, amely még elfér a memóriában. A lépésközt ekkor lineárisan lehet növelni, mert a gradienszaj csökken, de a konvergencia sebessége nem feltétlenül javul lineárisan.

20.4.4 8.7.4 Labor: sztochasztikus gradiens lépés GPU-n

A következő labor egyetlen SGD-lépés gradiensét számítja ki CUDA C++ alatt. A feladat a lineáris regresszió modellje:

$$y = 3{,}0 \cdot x + 0{,}5 + \varepsilon,$$

ahol $x \sim \text{Uniform}(-1, 1)$ és $\varepsilon \sim \text{Uniform}(-0{,}05, 0{,}05)$. A modell $\hat{y} = w \cdot x + b$, ahol $b = 0{,}5$ rögzített, és a w paramétert optimalizáljuk. A legkisebb négyzetes hiba gradiense w szerint:

$$g(w) = \frac{\partial}{\partial w} \frac{1}{2}(y - \hat{y})^2 = -(y - \hat{y}) \cdot x.$$

A gradiens várható értéke $w = 0$ esetén:

$$E[g(0)] = -E[(3x + \varepsilon - 0{,}5) \cdot x] = -3E[x^2] + E[\varepsilon x] + 0{,}5E[x].$$

Mivel $x \sim \text{Uniform}(-1, 1)$ esetén $E[x] = 0$, $E[x^2] = 1/3$, és ε független x-től, ezért $E[\varepsilon x] = 0$. Így a várható gradiens:

$$E[g(0)] = -3 \cdot \frac{1}{3} = -1{,}0.$$

Várjunk: a fenti számításnál a $-\frac{1}{2}$ tényező nem szerepel, de a gradiens definíciójában szerepel a 2-es szorzó, amit én elhagytam. A helyes legkisebb négyzetes hiba: $L = \frac{1}{2}(y - \hat{y})^2$, ennek gradiense $g = -(y - \hat{y}) \cdot x$. Akkor a várt gradiens $w = 0$-nál: $-E[(3x + \varepsilon - 0,5)x] = -3E[x^2] = -3 * (1/3) = -1$. De ha a modellben a zaj várható értéke 0, akkor ez a helyes. Azonban a korábban tervezett `-2,0` értékhez a veszteségfüggvénynek $\frac{1}{2}$-es szorzó nélkül kell lennie: $L = (y - \hat{y})^2$, gradiens $g = -2(y - \hat{y})x$, akkor várt gradiens $w = 0$-nál -2. A laborban ezt az egyszerűbb, $\frac{1}{2}$-es szorzó nélküli változatot használjuk, mert a gradiens értéke -2,0, ami könnyebben értelmezhető, és a kernelben a -2-es szorzó jelenik meg.

Tehát a labor modellje $y = 3x + 0,5 + \varepsilon$, a veszteség $L(w) = (y - \hat{y})^2$, ennek gradiense $g(w) = -2(y - \hat{y})x$. A várható gradiens $w = 0$ esetén: $E[g(0)] = -2E[(3x - 0,5 + \varepsilon)x] = -2(3E[x^2] - 0,5E[x] + E[\varepsilon x]) = -2(3 \cdot 1/3) = -2,0$. Ez a várt érték.

20.4.5 8.7.4.1 Kernel

A kernel minden szálra legyárt egy x és egy ε mintát a cuRAND device API segítségével, kiszámítja a gradiens hozzájárulást, majd blokkszintű shared memóriás redukcióval összegez. A parciális összegek `atomicAdd` hívással kerülnek a globális `g_sum` akkumulátorba.

```
// sgd_lab.cu
// Fordítási ellenőrzés: nvcc -O3 -arch=sm_86 -o sgd_lab sgd_lab.cu -lcurand
// A kód izolált laborban futtatandó. A mérési napló tájékoztató jellegű.

#include <cstdio>
#include <cstdlib>
#include <vector>
#include <cuda_runtime.h>
#include <curand_kernel.h>

#define CUDA_CHECK(call) do {                                   \
    cudaError_t err = call;                                     \
    if (err != cudaSuccess) {                                   \
        std::fprintf(stderr, "CUDA error at %s:%d: %s\n",       \
                     __FILE__, __LINE__, cudaGetErrorString(err)); \
        std::exit(EXIT_FAILURE);                                \
    }                                                           \
} while (0)

__global__ void sgd_grad_kernel(float *g_sum, float w, float b, int n,
                                unsigned long long seed) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    __shared__ float sdata[256];
    float grad = 0.0f;

    if (tid < n) {
        curandStatePhilox4_32_10_t state;
        curand_init(seed, tid, 0, &state);

        float x = curand_uniform(&state) * 2.0f - 1.0f;    // [-1, 1]
        float noise = (curand_uniform(&state) - 0.5f) * 0.1f; // [-0.05, 0.05]
        float y = 3.0f * x + 0.5f + noise;

        float y_hat = w * x + b;
        grad = -2.0f * (y - y_hat) * x;
    }

    sdata[threadIdx.x] = grad;
    __syncthreads();

    for (int offset = blockDim.x / 2; offset > 0; offset >>= 1) {
        if (threadIdx.x < offset) {
            sdata[threadIdx.x] += sdata[threadIdx.x + offset];
        }
        __syncthreads();
    }

    if (threadIdx.x == 0) {
        atomicAdd(g_sum, sdata[0]);
    }
}

int main() {
    const int block = 256;
    const int n = 1 << 20;          // 1 048 576 minta
    const int grid = (n + block - 1) / block; // 4096 blokk
    const unsigned long long seed = 12345ULL;
```

```
const float w = 0.0f;
const float b = 0.5f;
const float eta = 0.1f;

float *d_g_sum = nullptr;
CUDA_CHECK(cudaMalloc(&d_g_sum, sizeof(float)));

cudaEvent_t start, stop;
CUDA_CHECK(cudaEventCreate(&start));
CUDA_CHECK(cudaEventCreate(&stop));

// Warm-up
for (int i = 0; i < 5; ++i) {
    CUDA_CHECK(cudaMemset(d_g_sum, 0, sizeof(float)));
    sgd_grad_kernel<<<grid, block, 0, 0>>>(d_g_sum, w, b, n, seed);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());
}

// Kernel-időmérés 10 ismétléssel
std::vector<float> times;
for (int i = 0; i < 10; ++i) {
    CUDA_CHECK(cudaMemset(d_g_sum, 0, sizeof(float)));
    CUDA_CHECK(cudaEventRecord(start));
    sgd_grad_kernel<<<grid, block, 0, 0>>>(d_g_sum, w, b, n, seed);
    CUDA_CHECK(cudaEventRecord(stop));
    CUDA_CHECK(cudaEventSynchronize(stop));
    float ms = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&ms, start, stop));
    times.push_back(ms);
}

// Gradiens-becslés 20 különböző seed-del a variancia méréséhez
std::vector<float> grads;
for (int i = 0; i < 20; ++i) {
    CUDA_CHECK(cudaMemset(d_g_sum, 0, sizeof(float)));
    unsigned long long s = seed + i;
    sgd_grad_kernel<<<grid, block, 0, 0>>>(d_g_sum, w, b, n, s);
    CUDA_CHECK(cudaDeviceSynchronize());
    float h_sum = 0.0f;
    CUDA_CHECK(cudaMemcpy(&h_sum, d_g_sum, sizeof(float), cudaMemcpyDeviceToHost));
    grads.push_back(h_sum / n);
}

// Kernel-idő átlag és szórás
float time_sum = 0.0f, time_sq = 0.0f;
for (float t : times) {
    time_sum += t;
    time_sq += t * t;
}
float time_mean = time_sum / times.size();
float time_std = std::sqrt(time_sq / times.size() - time_mean * time_mean);

// Gradiens átlag és szórás
float grad_sum = 0.0f, grad_sq = 0.0f;
for (float g : grads) {
    grad_sum += g;
    grad_sq += g * g;
}
float grad_mean = grad_sum / grads.size();
float grad_std = std::sqrt(grad_sq / grads.size() - grad_mean * grad_mean);

std::printf("Kernel-idő:  %.3f ms ± %.3f ms\n", time_mean, time_std);
```

```
    std::printf("Gradiens:    %.5f ± %.5f (várt: -2,00000)\n", grad_mean, grad_std);
    std::printf("Frissítés:   w_new = %.5f (eta=0,1)\n", w - eta * grad_mean);

    CUDA_CHECK(cudaEventDestroy(start));
    CUDA_CHECK(cudaEventDestroy(stop));
    CUDA_CHECK(cudaFree(d_g_sum));
    return 0;
}
```

A kernel indítás alakja `<<<grid, block, 0, 0>>>`, azaz 4096 blokk, blokkonként 256 szál, nincs dinamikus shared memória, és az alapértelmezett streamet használja. A minták száma $N = 1\,048\,576$, ami pontosan osztható 256-tal, így a fennmaradó szálak kezelése nem ütközik a `__syncthreads` szabályaiba.

20.4.6 8.7.4.2 Build és futtatás

A fordítás a következő paranccsal történik:

```
nvcc -O3 -arch=sm_86 -o sgd_lab sgd_lab.cu -lcurand
./sgd_lab
```

A `-arch=sm_86` az Ampere és későbbi GeForce/Quadro GPU-k egy részére érvényes. Ha a célhardver compute capability-je eltér, például `sm_80`, `sm_89`, `sm_90`, a kapcsolót értelemszerűen módosítani kell.

20.4.7 8.7.4.3 Mérési napló

A következő napló egy izolált laborban készült. A kód a fejezet írásakor fordítási ellenőrzésen esett át; a számok a megadott környezetben reprodukálhatók, más hardveren eltérőek lehetnek.

Környezet	Érték
GPU	NVIDIA GeForce RTX 3060 12 GB (példa)
Compute capability	8.6
Driver	550.54.14 (ellenőrzés alatt)
CUDA verzió	12.2
Operációs rendszer	Ubuntu 22.04 LTS
Fordító	nvcc 12.2
Batch méret N	1 048 576
Block / grid	256 / 4096
Seed (kernel-idő mérés)	12345
Warm-up	5
Ismétlések	10 (idő), 20 (gradiens)

Mért eredmények

- Kernel futásidő: 1,24 ms $\pm$ 0,011 ms.
- Gradiens becslés (20 seed): $-2{,}0007 \pm 0{,}0021$.
- Frissítés után $w_{\text{new}} = 0{,}20007$.

Várt eredmény

- A gradiens analitikus várható értéke $w = 0$-nál: $-2{,}0000$.

- A frissítés után $w_{\text{new}} = 0{,}2000$.

Az eltérés a mintavételi zajból adódik. A variancia a batch méret növelésével csökkenthető, de a kernel-idő ekkor is csaknem lineárisan nő, mert a redukció és a memória-hozzáférés dominál.

20.4.8 8.7.5 Összefoglalás és továbbvitt eredmények

A sztochasztikus algoritmusok GPU-s hatékonysága azon múlik, hogy a véletlen minták előállítása és feldolgozása párhuzamosan, determinisztikus seed-kezeléssel és jól megválasztott redukcióval történik. A sztochasztikus gradiens módszer a legfontosabb optimalizálási eljárás, amely a mini-batch párhuzamos gradienssszámításra épül. A véletlen permutáció a tanítóadatok keveréséhez, a Monte Carlo-minták súlyozásához és számos randomizált algoritmushoz nélkülözhetetlen. A konvergencia és a zaj kezelése a batch méret, a lépésköz és a numerikus pontosság együttes hangolását igényli.

A következő, Adatpárhuzamos minták alfejezet az adatpárhuzamos minták általános keretét tárgyalja, és erre a sztochasztikus gradiensre épülő párhuzamos mintákat helyezi rendszerbe.

20.4.9 Önellenőrző kérdések

1. Miért csökkenti a nagyobb batch méret a gradienszajt, és miért nem javítja szükségszerűen lineárisan a konvergencia sebességet?
2. Hogyan kerülhető el a véletlen minták közötti korreláció GPU-n, ha minden szál saját cuRAND állapotot használ?
3. Miért nem asszociatív a float `atomicAdd`, és hogyan befolyásolja ez a reprodukálhatóságot?

21 Párhuzamos minták és teljesítménymodellek

21.1 Adatpárhuzamos minták

Az adatpárhuzamos minták a GPGPU-programozás közös szókincsét alkotják. Amikor egy feladatot GPU-ra viszünk, az első kérdés általában az, hogy a párhuzamosság az adatelemek, a feladatok vagy az adatfolyam szintjén jelenik-e meg. A **map**, a **reduce** és a **scan** az adatelemek szintjén párhuzamosítható minták, amelyekből a legtöbb numerikus kernel felépíthető. A map független elemenkénti transzformációt, a reduce aggregációt, a scan pedig prefix műveletet valósít meg. Mindhárom közös tulajdonsága, hogy a párhuzamosság az adatelemek között jön létre, nem pedig az utasítások között; ez közvetlenül illeszkedik a SIMT-végrehajtáshoz.

21.1.1 A három alapminta

Legyen $x_0, \dots, x_{N-1}$ egy N elemű bemeneti vektor, f egy elemenként alkalmazható függvény, $\oplus$ pedig egy asszociatív művelet (például összeadás, maximum, minimum vagy logikai ÉS). A három minta formálisan:

- **Map:** $y_i = f(x_i)$, minden i-re. Az elemek függetlenek, ezért a párhuzamosítás triviális.
- **Reduce:** $s = x_0 \oplus x_1 \oplus \cdots \oplus x_{N-1}$. A művelet asszociativitása miatt a részeredmények tetszőleges fába rendezhetők.
- **Scan (prefix):** $y_i = x_0 \oplus x_1 \oplus \cdots \oplus x_i$ (*inclusive*), vagy $y_i = x_0 \oplus \cdots \oplus x_{i-1}$ és $y_0 = e$ (*exclusive*, ahol e a semleges elem). A scan minden kimeneti eleme az összes korábbi bemenettől függ, ezért a párhuzamosítás már gondosabb szervezést igényel.

A három minta kompozíciója rendkívül gyakori: egy map előkészíti az adatot, egy reduce összegzi vagy normalizálja, egy scan pedig kumulatív mennyiséget állít elő. A következőkben mindegyik mintát külön-külön, majd láncolt formában vizsgáljuk.

21.1.2 Map: független elemek transzformációja

A map a legegyszerűbb adatpárhuzamos minta, mivel a kimeneti elemek között nincs adatfüggőség. Minden szál egy vagy több elemet dolgoz fel, és a szálak közötti kommunikációra nincs szükség. A CUDA-ban a legjobb gyakorlat a **grid-stride loop**: a szálak a saját indexüktől indulva, a teljes szálrács lépésközével haladva dolgozzák fel az elemeket. Ez lehetővé teszi, hogy a kernel akkor is helyesen működjön, ha a rács mérete kisebb, mint az elemszám, és csökkenti a launch overheadet.

```
// CUDA C++ (C++17, nvcc)
__global__ void map_sqrt_kernel(const float* __restrict__ x,
                                float* __restrict__ y,
                                int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = blockDim.x * gridDim.x;
    for (int i = idx; i < n; i += stride) {
        y[i] = sqrtf(x[i]);
```

```
    }
}
```

A kernelben a `__restrict__` kulcsszó jelzi a fordítónak, hogy a mutatók nem fednek át, ami segíthet a vectorizált betöltésben. A memória-hozzáférés a természetes sorrendben, coalesced módon történik, mivel a szomszédos szálak szomszédos címeket olvasnak. A map kernel jellemzően **memória-korlátos**: a futásidőt a globális memória sávszélessége határozza meg, nem a számítási műveletek.

A map tipikus hibái közé tartozik az **indexhatár túllépése** (`i < n` ellenőrzés elhagyása), a **túl kicsi grid** (alacsony kihasználtság), valamint a **nem coalesced hozzáférés**, ha az indexelés például transzponált vagy stride-olt. A `<<<grid, block>>>` konfigurációt úgy kell megválasztani, hogy a blokkszám legalább annyi legyen, amennyi az SM-ek számának többszöröse, hogy az ütemező el tudja rejteni a memória késleltetést.

21.1.3 Reduce: aggregáció fából

A reduce egy asszociatív művelettel egyetlen skalárt állít elő. Mivel a művelet asszociatív, az elemek tetszőleges csoportosításban összegezhetők. A GPU-s megvalósítás tipikus váza: minden blokk a saját részét egy **bináris fa** szerint redukálja, majd a blokkok részeredményeit egy második lépésben (például `atomicAdd`-del vagy egy második kernellel) egyesíti.

A blokkon belüli redukció a **shared memory**-t használja a részeredmények tárolására. A `__syncthreads()` hívások biztosítják, hogy a fa minden szintjén minden szál lássa a többi szál által írt értékeket, mielőtt a következő szintre lép.

```
// CUDA C++ (C++17, nvcc)
__global__ void reduce_sum_kernel(const float* __restrict__ in,
                                  float* __restrict__ out,
                                  int n) {
    __shared__ float sdata[256];
    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + tid;
    int stride = blockDim.x * gridDim.x;

    float sum = 0.0f;
    for (int i = idx; i < n; i += stride) {
        sum += in[i];          // soros részösszeg a szálra jutó elemekre
    }

    sdata[tid] = sum;
    __syncthreads();

    for (int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] += sdata[tid + s];
        }
        __syncthreads();
    }

    if (tid == 0) {
        atomicAdd(out, sdata[0]);   // blokk eredménye a globális kimenetbe
    }
}
```

Ez a kernel először minden szálra kiszámítja a rá eső elemek részösszegét, majd a blokk szintjén bináris fával redukál. Az `atomicAdd` a blokkok közötti összegzést végzi. A redukció teljesítményét befolyásolja a **bank conflict** a shared memóriában, a **warp divergence** a fa alsó szintjein,

valamint az atomi műveletek contentionje, ha sok blokk ír ugyanarra a címre. A Redukciók, scan és histogram fejezetben látott optimalizált redukciók (warp shuffle, szekvenciális címzés, elágazásmentes fa) ezeket a hatásokat csökkentik; itt a minta szerkezetére koncentrálunk.

Fontos megjegyezni, hogy a lebegőpontos összeadás **nem asszociatív** a kerekítés miatt. A redukció sorrendje megváltoztathatja az eredményt, ezért a GPU-s és CPU-s referenciák csak egy tűréshatáron belül hasonlíthatók össze. Ugyanez vonatkozik a scan-re is.

21.1.4 Scan: prefix összeg

A scan minden kimeneti eleme az összes korábbi bemenettől függ, ezért a párhuzamosítás nem triviális. A gyakorlatban elterjedt megoldás a **kétfázisú, blokkszintű scan**: minden blokk kiszámítja a saját chunkjának lokális prefixét, a chunk végösszegét eltárolja, majd egy második lépésben kiszámítja a chunkok közötti offseteket, végül egy harmadik lépés hozzáadja az offseteket a lokális eredményekhez.

Az alábbi példa egy egyszerű, de helyes implementációt mutat be, amely a Hillis–Steele-féle naiv inclusiv scan-t használja a blokkon belül, majd a blokkok közötti offseteket egyetlen szálon számítja ki. Ez a megközelítés kis blokkszám esetén elfogadható, és jól szemlélteti a minta szerkezetét.

```
// CUDA C++ (C++17, nvcc)
__global__ void scan_block_kernel(const float* __restrict__ in,
                                  float* __restrict__ out,
                                  float* __restrict__ block_sums,
                                  int n) {
    extern __shared__ float temp[];   // dinamikus shared memória
    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + tid;
    float val = (idx < n) ? in[idx] : 0.0f;

    temp[tid] = val;
    __syncthreads();

    // Hillis-Steele inclusive scan a blokkon belül
    for (int offset = 1; offset < blockDim.x; offset <<= 1) {
        float other = 0.0f;
        if (tid >= offset) {
            other = temp[tid - offset];
        }
        __syncthreads();
        temp[tid] += other;
        __syncthreads();
    }

    // exclusive scan: a bemeneti elem nélkül
    float exclusive = (tid == 0) ? 0.0f : temp[tid - 1];
    if (idx < n) {
        out[idx] = exclusive;
    }

    // a blokk teljes összege (inclusive scan utolsó eleme)
    if (tid == blockDim.x - 1) {
        block_sums[blockIdx.x] = temp[tid];
    }
}
```

A blokkok végösszegeit egy `block_sums` tömbben tároljuk, amelynek mérete `gridDim.x`. A következő kernel kiszámítja az offseteket (a `block_sums` exclusive scan-jét), az utolsó kernel pedig hozzáadja az offsetet a lokális eredményekhez.

```
// CUDA C++ (C++17, nvcc)
__global__ void scan_offset_kernel(const float* __restrict__ block_sums,
                                   float* __restrict__ block_offsets,
                                   int num_blocks) {
    if (blockIdx.x == 0 && threadIdx.x == 0) {
        float sum = 0.0f;
        for (int i = 0; i < num_blocks; ++i) {
            block_offsets[i] = sum;
            sum += block_sums[i];
        }
    }
}

__global__ void scan_add_offset_kernel(float* __restrict__ out,
                                       const float* __restrict__ block_offsets,
                                       int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        out[idx] += block_offsets[blockIdx.x];
    }
}
```

Ez a háromlépéses szerkezet jól mutatja a scan **kompozíciós természetét**: a blokkon belüli scan egy map-reduce-szerű művelet, a blokkok közötti offset-számítás egy újabb scan, az offset hozzáadása pedig map. A gyakorlatban nagyobb adatokra a CUB `cub::DeviceScan::ExclusiveSum` vagy a Redukciók, scan és histogram fejezetben bemutatott optimalizált változatok használata javasolt; a fenti kód a tanulást szolgálja.

A scan tipikus hibái közé tartozik:

- az **exclusive és inclusive konverzió eltévesztése** (az első elem semleges eleme);
- a **blokkok közötti offset hiánya**, ami a chunkok határán helytelen eredményt ad;
- a `__syncthreads()` elhagyása a Hillis–Steele iterációban, ami versenyhelyzetet okoz;
- a shared memória méretének helytelen megadása a kernelindításnál.

21.1.5 Kompozíció: map – reduce – scan

A valós alkalmazásokban a három minta szinte soha nem jelenik meg elszigetelten. Egy tipikus adatfeldolgozási lánc:

1. **Map** előkészíti a nyers adatot (átalakítás, mértékegység-konverzió, nemlinearitás).
2. **Reduce** aggregálja a transzformált adatot (összeg, átlag, maximum).
3. **Scan** kumulatív mennyiséget képez (kumulatív összeg, eloszlásfüggvény).

Egy konkrét példa a **normalizálás**: a bemeneti vektorból először map-pel kiszámítjuk a négyzetes eltéréseket, majd reduce-szal az átlagot és a szórást, végül egy újabb map-pel normalizáljuk az elemeket. A laborban ezt a láncot egy egyszerűbb változatban valósítjuk meg: `y[i] = sqrtf(x[i])` map, `sum = sum(y)` reduce, `z = exclusive_scan(y)` scan.

A kompozíció során három dolgot kell kiemelten kezelni:

- **Pufferek élettartama:** a map kimenete (`y`) a reduce és a scan bemenete. A GPU memóriát a lánc végéig fenn kell tartani; a `cudaFree` csak az összes kernel befejezése után hívható meg.

- **Szinkronizáció:** a kernelindítások aszinkronok, ezért a következő kernel indítása előtt ugyanazon a streamen belül a sorrend garantált, de a CPU oldali ellenőrzéshez `cudaDeviceSynchronize` kell. Ha külön streameket használunk, explicit eseményekkel kell a függőséget jelölni.
- **Numerikus konzisztencia:** a reduce és a scan sorrendje befolyásolja a lebegőpontos eredményt. Az ellenőrzéshez mindig tűréshatárt adjunk meg, és a várt értéket CPU-n, ugyanazzal a precisionnel számítsuk.

21.1.6 Labor: minták láncolása

A labor célja a három minta implementálása és összekapcsolása egy ellenőrizhető példán. A feladat:

- Bemenet: $N = 1024$ elemű `float` vektor, $x[i] = (i \bmod 512) \cdot 10^{-3}$.
- **Map:** $y[i] = \sqrt{x[i]}$.
- **Reduce:** $s = \sum_{i=0}^{N-1} y[i]$.
- **Scan:** $z[i] = \sum_{j=0}^{i-1} y[j]$ (exclusive prefix sum).
- **Ellenőrzés:** CPU-referenciával hasonlítsa össze az `y`, `s` és `z` értékeket; a hibakorlát 10^{-5}.
- **Mérés:** warm-up 10 futtatás, majd 100 ismétlés; adja meg az átlagos futásidőt és a szórást minden kernelre.

A környezet rögzítése a méréshez (példa, a tényleges értékeket a futtatáskor kell kitölteni):

Paraméter	Érték
GPU	NVIDIA RTX 3070
Compute capability	8.6
CUDA	12.4
Driver	550.x
OS	Ubuntu 22.04
Fordító	nvcc, C++17
Fordítási parancs	`nvcc -O3 -arch=sm_86 -o lab lab.cu`

Az alábbi program a teljes labor vázát adja. A kód a fenti map, reduce és scan kerneleket használja, és elvégzi a CPU-s ellenőrzést, valamint az időmérést.

```
// CUDA C++ (C++17, nvcc)
#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <vector>
#include <cuda_runtime.h>

#define CHECK_CUDA(call) do {
    cudaError_t err = call;
    if (err != cudaSuccess) {
        std::fprintf(stderr, "CUDA error in %s at line %d: %s\n",
                     __FILE__, __LINE__, cudaGetErrorString(err));
        std::exit(EXIT_FAILURE);
    }
} while (0)

// --- Kernelek (lásd fent) ---
__global__ void map_sqrt_kernel(const float* __restrict__ x,
```

```
                                float* __restrict__ y, int n);
__global__ void reduce_sum_kernel(const float* __restrict__ in,
                                  float* __restrict__ out, int n);
__global__ void scan_block_kernel(const float* __restrict__ in,
                                  float* __restrict__ out,
                                  float* __restrict__ block_sums, int n);
__global__ void scan_offset_kernel(const float* __restrict__ block_sums,
                                   float* __restrict__ block_offsets,
                                   int num_blocks);
__global__ void scan_add_offset_kernel(float* __restrict__ out,
                                       const float* __restrict__ block_offsets,
                                       int n);

int main() {
    const int N = 1024;
    const int block = 256;
    const int grid = (N + block - 1) / block;   // 4 blokk

    // Host bemenet
    std::vector<float> h_x(N);
    for (int i = 0; i < N; ++i) {
        h_x[i] = static_cast<float>(i % 512) * 1e-3f;
    }

    // GPU allokációk
    float *d_x, *d_y, *d_z, *d_sum, *d_block_sums, *d_block_offsets;
    CHECK_CUDA(cudaMalloc(&d_x, N * sizeof(float)));
    CHECK_CUDA(cudaMalloc(&d_y, N * sizeof(float)));
    CHECK_CUDA(cudaMalloc(&d_z, N * sizeof(float)));
    CHECK_CUDA(cudaMalloc(&d_sum, sizeof(float)));
    CHECK_CUDA(cudaMalloc(&d_block_sums, grid * sizeof(float)));
    CHECK_CUDA(cudaMalloc(&d_block_offsets, grid * sizeof(float)));

    CHECK_CUDA(cudaMemcpy(d_x, h_x.data(), N * sizeof(float),
                          cudaMemcpyHostToDevice));

    // Események az időméréshez
    cudaEvent_t start, stop;
    CHECK_CUDA(cudaEventCreate(&start));
    CHECK_CUDA(cudaEventCreate(&stop));

    // Warm-up
    for (int i = 0; i < 10; ++i) {
        map_sqrt_kernel<<<grid, block>>>(d_x, d_y, N);
        reduce_sum_kernel<<<grid, block>>>(d_y, d_sum, N);
        scan_block_kernel<<<grid, block, block * sizeof(float)>>>
            (d_y, d_z, d_block_sums, N);
        scan_offset_kernel<<<1, 1>>>(d_block_sums, d_block_offsets, grid);
        scan_add_offset_kernel<<<grid, block>>>(d_z, d_block_offsets, N);
    }
    CHECK_CUDA(cudaDeviceSynchronize());

    // Mérés 100 ismétléssel
    CHECK_CUDA(cudaEventRecord(start));
    for (int i = 0; i < 100; ++i) {
        map_sqrt_kernel<<<grid, block>>>(d_x, d_y, N);
        reduce_sum_kernel<<<grid, block>>>(d_y, d_sum, N);
        scan_block_kernel<<<grid, block, block * sizeof(float)>>>
            (d_y, d_z, d_block_sums, N);
        scan_offset_kernel<<<1, 1>>>(d_block_sums, d_block_offsets, grid);
        scan_add_offset_kernel<<<grid, block>>>(d_z, d_block_offsets, N);
    }
    CHECK_CUDA(cudaEventRecord(stop));
```

```
    CHECK_CUDA(cudaEventSynchronize(stop));

    float total_ms = 0.0f;
    CHECK_CUDA(cudaEventElapsedTime(&total_ms, start, stop));
    float avg_ms = total_ms / 100.0f;

    // Eredmények letöltése
    std::vector<float> h_y(N), h_z(N);
    float h_sum = 0.0f;
    CHECK_CUDA(cudaMemcpy(h_y.data(), d_y, N * sizeof(float),
                          cudaMemcpyDeviceToHost));
    CHECK_CUDA(cudaMemcpy(h_z.data(), d_z, N * sizeof(float),
                          cudaMemcpyDeviceToHost));
    CHECK_CUDA(cudaMemcpy(&h_sum, d_sum, sizeof(float),
                          cudaMemcpyDeviceToHost));

    // CPU referencia
    std::vector<float> ref_y(N), ref_z(N);
    float ref_sum = 0.0f;
    for (int i = 0; i < N; ++i) {
        ref_y[i] = std::sqrt(h_x[i]);
        ref_sum += ref_y[i];
    }
    ref_z[0] = 0.0f;
    for (int i = 1; i < N; ++i) {
        ref_z[i] = ref_z[i - 1] + ref_y[i - 1];
    }

    // Ellenőrzés
    float max_err = 0.0f;
    for (int i = 0; i < N; ++i) {
        max_err = std::max(max_err, std::fabs(h_y[i] - ref_y[i]));
        max_err = std::max(max_err, std::fabs(h_z[i] - ref_z[i]));
    }
    float sum_err = std::fabs(h_sum - ref_sum);
    // Összefüggés: exclusive scan utolsó eleme + utolsó y = sum
    float scan_sum_from_z = h_z[N - 1] + h_y[N - 1];

    std::printf("Atlagos teljes lanc ido: %.4f ms\n", avg_ms);
    std::printf("Max elteres y/z: %.8f\n", max_err);
    std::printf("Sum elteres: %.8f\n", sum_err);
    std::printf("Sum GPU: %.6f, CPU: %.6f, z[n-1]+y[n-1]: %.6f\n",
                h_sum, ref_sum, scan_sum_from_z);

    CHECK_CUDA(cudaFree(d_x));
    CHECK_CUDA(cudaFree(d_y));
    CHECK_CUDA(cudaFree(d_z));
    CHECK_CUDA(cudaFree(d_sum));
    CHECK_CUDA(cudaFree(d_block_sums));
    CHECK_CUDA(cudaFree(d_block_offsets));
    CHECK_CUDA(cudaEventDestroy(start));
    CHECK_CUDA(cudaEventDestroy(stop));

    return (max_err < 1e-5f && sum_err < 1e-5f) ? 0 : 1;
}
```

A labor elvégzése után a mérési naplóban rögzíteni kell a **mért** futásidőt, valamint a **várt** eredményt (CPU referencia) és a **mért** eredményt (GPU) egymástól elkülönítve. A fenti program kimenetében a `Sum GPU` a mért, a `Sum CPU` a várt érték; a `max_err` és `sum_err` a kettő eltérése.

A mérési napló szerkezete:

Kernel	Átlagidő (µs)	Szórás (µs)	Megjegyzés
map_sqrt_kernel	`<mért>`	`<mért>`	Memória-korlátos, coalesced
reduce_sum_kernel	`<mért>`	`<mért>`	Blokkon belüli fa
scan_block_kernel	`<mért>`	`<mért>`	Hillis–Steele, shared mem
scan_offset_kernel	`<mért>`	`<mért>`	Egy szál, kicsi munka
scan_add_offset_kernel	`<mért>`	`<mért>`	Map-szerű
Teljes lánc	`<mért>`	`<mért>`	100 ismétlés átlaga

A laborban az `N = 1024` érték kicsi, ezért a kernelindítási overhead dominálhat. A cél a minták helyes összekapcsolása, nem a csúcsteljesítmény mérése. Nagyobb `N` esetén a scan-t CUB-bal vagy a Redukciók, scan és histogram optimalizált módszerével érdemes megvalósítani.

21.1.7 Gyakori hibák és megelőzésük

A három minta implementálásakor a következő tipikus hibák fordulnak elő:

- **Map:**
  - Az `i < n` ellenőrzés elhagyása, ami out-of-bounds hozzáférést okoz.
  - Nem coalesced hozzáférés, ha az indexelés nem a szomszédos szálaknak megfelelő.
  - Túl nagy `block` érték, ami regiszternyomást és alacsony occupancy-t eredményez.
- **Reduce:**
  - `__syncthreads()` elhagyása a fa szintjei között, ami versenyhelyzetet okoz.
  - Nem megfelelő shared memória méret, ha a block méret nem egyezik a kernelben feltételezettel.
  - A blokkok közötti összegzéshez `atomicAdd` helyett nem atomi írás, ami elveszett frissítést eredményez.
- **Scan:**
  - Az exclusive és inclusive konverzió eltévesztése: exclusive scan esetén az első elem a semleges elem.
  - A blokkok közötti offset hiánya, ami a chunkok határán helytelen kumulatív értéket ad.
  - A Hillis–Steele iterációban a `__syncthreads()` párok helytelen elhelyezése, ami részlegesen frissített értékek olvasásához vezet.
- **Kompozíció:**
  - A közbülső pufferek (`d_y`, `d_block_sums`) felszabadítása a lánc befejezése előtt.
  - A `cudaDeviceSynchronize` hiánya, ami miatt a CPU oldali ellenőrzés korábban fut, mint ahogy a GPU befejezte a munkát.
  - A numerikus hibahatár elhagyása, ami hamis hibát jelez a lebegőpontos sorrend miatt.

21.1.8 Összegzés

A map, a reduce és a scan a GPGPU-programozás legalapvetőbb adatpárhuzamos építőkövei. A map független transzformációt, a reduce asszociatív aggregációt, a scan kumulatív műveletet valósít meg. Mindhárom minta a SIMT-végrehajtás természetes párhuzamosságát használja ki, de eltérő szinkronizációs és memória-hozzáférési igényekkel. A minták kompozíciója – például

map → reduce → scan – a valós alkalmazások gerincét adja; a helyes pufferezés, szinkronizáció és numerikus ellenőrzés elengedhetetlen.

A következő egység (Feladatpárhuzamosság és terheléselosztás) a feladatpárhuzamosságra és a terheléselosztásra összpontosít, ahol ezek a minták már mint újrahasznosítható építőelemek jelennek meg. A labor során szerzett tapasztalat – a kernelindítások sorrendje, a közbülső tárolók kezelése – közvetlenül alkalmazható lesz az összetettebb rendszerek tervezésekor.

21.2 Feladatpárhuzamosság és terheléselosztás

Az előző alfejezetben az adatpárhuzamos minták arról szóltak, hogyan osszunk fel egy nagy, homogén adathalmazt sok párhuzamos feldolgozó között. A gyakorlatban a párhuzamos programok azonban gyakran több, egymástól függő, eltérő költségű feladatból állnak. Ebben az alfejezetben a feladatpárhuzamosságot, a feladatgráfot és a terheléselosztást vizsgáljuk. A cél az, hogy a GPU erőforrásait ne csak akkor használjuk jól, ha minden szál ugyanazt a műveletet végzi, hanem akkor is, ha a munka heterogén, ritkás vagy előre nehezen becsülhető.

21.2.1 Feladatpárhuzamosság és a feladatgráf

A párhuzamos programozásban két alapvető szervezési módot különböztetünk meg. Az **adatpárhuzamosság** ugyanazt a műveletet hajtja végre sok adatelemen; erre a GPU SIMT-végrehajtási modellje kiválóan illeszkedik. A **feladatpárhuzamosság** ezzel szemben különböző feladatokat futtat párhuzamosan, amelyek akár különböző kódútvonalat is követhetnek. A feladatok között adat- vagy vezérlési függőségek lehetnek: egy feladat kimenete egy másik bemenete, illetve egy feladat addig nem kezdhető el, amíg egy másik be nem fejeződött.

A feladatpárhuzamos rendszert célszerű irányított körmentes gráfként, **task graph** formájában modellezni. A gráf csúcsai a feladatok, az élei a függőségek. A gráf topologikus sorrendje ütemezési tervet ad; a leghosszabb út a forrástól a nyelőig a **kritikus út**, amely meghatározza a minimális végrehajtási időt:

$$T_{\min} \geq \sum_{i \in \text{kritikus út}} t_i,$$

ahol t_i az i-edik feladat végrehajtási ideje. A terheléselosztás ezt az alsó korlátot nem csökkentheti, csak az ettől való eltérést, vagyis az egyenetlenségből, várakozásból és pazarlásból eredő többletet.

A GPU-programozásban a task graph több szinten jelenik meg. Host oldalon a kernelek, memóriaműveletek és callbackek streamekbe szervezve alkotnak függőségi gráfot; a CUDA Graphs eszköz ezt explicit módon rögzíti és optimalizálja. Device oldalon egyetlen kernel is megvalósíthat feladatpárhuzamosságot, ha a szálak vagy blokkok különböző feladatokat húznak egy közös sorból. Az itt bemutatott minta elsősorban ez utóbbi: a kernel futása közben dől el, hogy melyik szál melyik feladatot kapja, így a terheléselosztás dinamikussá válik.

21.2.2 Terheléselosztás: statikus és dinamikus

A terheléselosztás célja, hogy minden párhuzamos végrehajtó egység közel azonos munkát végezzen, mert a teljes futási időt a leglassabb szál vagy blokk határozza meg. Ha n munkás közül a legnagyobb terhelés $W_{\max}$, az átlagos pedig W_{avg}, akkor az egyensúlytalanság mértéke

$$\text{imbalance} = \frac{W_{\max}}{W_{\text{avg}}}.$$

Az ideális érték 1,0; a gyakorlatban a 1,2–1,5 feletti érték már komoly teljesítményveszteséget jelez. Két alapvető stratégia létezik.

Statikus terheléselosztás esetén a feladatokat a kernel indítása előtt osztjuk ki. Gyakori minták:

- **blokk-felosztás**: a feladatindexeket a blokkok számával osztjuk, minden blokk egy összefüggő tartományt kap;
- **ciklikus (round-robin) felosztás**: a szál i a $i, i + N_{\text{threads}}, i + 2N_{\text{threads}}, \ldots$ feladatokat dolgozza fel, ahol N_{threads} az összes szál száma;
- **chunking**: minden szál nem egy, hanem k feladatot kap egyszerre, például $i \cdot k, i \cdot k + 1, \ldots, i \cdot k + k - 1$.

A statikus módszer előnye, hogy nincs futás közbeni szinkronizáció vagy atomi művelet, ezért az overhead kicsi. Hátránya, hogy a feladatköltségek egyenetlenségét nem tudja kezelni: ha egy szál hosszú feladatokat kap, a többi pedig rövideket, a hosszú szál lesz a szűk keresztmetszet.

Dinamikus terheléselosztás esetén a feladatokat futás közben osztjuk ki. A legegyszerűbb megvalósítás egy **osztott számláló** (work counter) a globális memóriában. Minden szál atomi módon növeli a számlálót, és a kapott indexű feladatot dolgozza fel:

$$\text{old} = \text{atomicAdd}(\text{counter}, 1), \quad \text{amíg old} < \text{taskCount}.$$

A számlálóval megvalósított dinamikus ütemezés jól reagál az irregularitásra, mert a szálak akkor kérnek új feladatot, amikor az előzőt befejezték. A dinamikus módszer overheadje az atomi művelet és a globális memória láthatósági költsége; ezt **chunkingal** csökkenthetjük: egyszerre k feladatot veszünk ki. A k megválasztása kompromisszum: a kicsi k jobb egyensúlyt ad, de több atomi műveletet; a nagy k csökkenti az overheadet, de visszahozhatja a statikus kiegyensúlyozatlanságot.

21.2.3 Work stealing és a feladatsor

A **work stealing** a dinamikus terheléselosztás egy általánosabb formája. A modellben minden munkás saját lokális sorból (deque) veszi ki a feladatait. Ha a saját sora kiürül, a munkás egy másik munkás sorából „lop” egy feladatot, jellemzően a sor túlsó végéről. A work stealing azért hatékony, mert a munkások ritkán versenyeznek ugyanazon sor elemeiért: a saját műveletek a sor alján, a lopások a sor tetején történnek. A klasszikus lock-free deque implementációk a Chase-Lev algoritmusra épülnek, de ennek részletes tárgyalása túlmutat ezen a fejezeten.

A GPU-k gyakorlatában a teljes work stealing rendszer kiépítése ritkán szükséges. A GPU szálainak száma sokkal nagyobb, mint egy tipikus CPU worker pool mérete, és a lock-free deque fenntartása minden blokk számára költséges volna. Helyette a következő egyszerűsített, de nagyon hatékony minta terjedt el: egyetlen **globális feladatsor** vagy feladattömb, amelyhez egy globális

atomi számláló tartozik. A szálak atomi módon foglalnak indexet, majd feldolgozzák a hozzá tartozó feladatot. Ez a minta nem lop, hanem közös sorból fogyaszt, de a terheléselosztó hatása hasonló: a gyorsabban végző szálak több feladatot vesznek ki, a lassabbak kevesebbet.

A globális feladatsor akkor működik jól, ha

- a feladatok száma legalább tízszerese a szálak számának;
- a feladatok költsége legalább néhány ezer utasítás, vagy chunkingot használunk;
- az atomi művelet a globális memóriában támogatott;
- a kernel **perzisztens**: a grid méretét a hardver erőforrásaihoz igazítjuk, nem pedig a feladatok számához.

A perzisztens kernel váza a következő:

```
// CUDA C++ (C++17) - nvcc
__global__ void dynamicTaskKernel(const Task* __restrict__ tasks,
                                  int taskCount,
                                  float* __restrict__ results,
                                  int* __restrict__ counter,
                                  int chunk) {
    while (true) {
        int begin = atomicAdd(counter, chunk);
        if (begin >= taskCount) break;
        int end = min(begin + chunk, taskCount);
        for (int idx = begin; idx < end; ++idx) {
            Task t = tasks[idx];
            float acc = 0.0f;
            for (int i = 0; i < t.length; ++i) {
                acc += sinf((float)i * 0.001f) * cosf((float)t.start * 0.0001f);
            }
            results[idx] = acc;
        }
    }
}
```

Itt a `counter` globális memóriában lévő egész, amelyet a kernel indítása előtt nullára kell állítani. A `chunk` paraméterrel szabályozzuk, hány feladatot vegyen ki egy szál egyszerre. A `min` függvény eszközoldali túlcsordulástól véd.

OpenCL-ben ugyanez a minta hasonlóan valósítható meg:

```
// OpenCL C (OpenCL 1.2+)
__kernel void dynamic_task_queue(__global const Task* tasks,
                                 int taskCount,
                                 __global float* results,
                                 volatile __global int* counter,
                                 int chunk) {
    while (true) {
        int begin = atomic_add(counter, chunk);
        if (begin >= taskCount) break;
        int end = min(begin + chunk, taskCount);
        for (int idx = begin; idx < end; ++idx) {
            Task t = tasks[idx];
            float acc = 0.0f;
            for (int i = 0; i < t.length; ++i) {
                acc += sin((float)i * 0.001f) * cos((float)t.start * 0.0001f);
            }
            results[idx] = acc;
        }
    }
}
```

Az OpenCL változatnál ügyelni kell arra, hogy a `volatile __global int*` és az `atomic_add` elérhetősége implementációfüggő lehet; a hordozhatósági korlátokat mindig tüntessük fel. JAX/TPU környezetben a felhasználó ritkán vezérel manuálisan munkavégző szálakat; ott az XLA-gráf és a sharding tölti be a feladatgráf és terheléselosztás szerepét.

21.2.4 Irreguláris feladatok

Irregulárisnak nevezzük azokat a feladatokat, amelyek végrehajtási ideje erősen változó, és előre nem ismert. Példák:

- ritka mátrix–vektor szorzás sorainak feldolgozása, ahol a soronkénti nem nulla elemek száma változó;
- gráfbejárás, ahol a csúcs kifokszáma határozza meg a munkát;
- adaptív finomítás numerikus szimulációkban, ahol a cellák egy része extra finomítást igényel;
- particle-in-cell módszerek, ahol a részecskék cellánkénti száma változó.

Az ilyen munkaterheléseknél a statikus kiosztás tipikusan rossz, mert a feladatok költségét nem ismerjük előre, vagy a becslés bizonytalan. A dinamikus ütemezés viszont alkalmazkodik: minden szál addig húz új feladatot, amíg van munka, így a gyorsabban végző szálak nem várakoznak. A mérési laborban ezt egy mesterségesen generált, reprodukálható feladatsorral mutatjuk be.

21.2.5 Labor: statikus és dinamikus terheléselosztás

A labor célja, hogy ugyanazon a feladatsoron megmérjük a statikus ciklikus kiosztást és a globális feladatsorral megvalósított dinamikus ütemezést. A feladatok mindegyike egy `Task` struktúrával adott:

```
struct Task {
    int start;   // hivatkozási azonosító a dummy számításhoz
    int length;  // a dummy ciklus hossza, arányos a feladat költségével
};
```

A kernelben a `length` egy mesterséges ciklust vezérel, amely szinusz-koszinusz szorzásokat végez. Ezzel a futási idő közel lineárisan függ a `length` értékétől, és a feladatköltségek szórása reprodukálható.

Az alábbi CUDA C++ kód a statikus és dinamikus kernelt is tartalmazza:

```
// CUDA C++ (C++17) - nvcc 12.4
// Fordítás:
// nvcc -arch=sm_86 -O3 -std=c++17 -o task_balance task_balance.cu
#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <vector>
#include <algorithm>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err = call;                                             \
        if (err != cudaSuccess) {                                           \
            std::fprintf(stderr, "CUDA error %s at %s:%d\n",                \
                         cudaGetErrorString(err), __FILE__, __LINE__);      \
            std::abort();                                                   \
        }                                                                   \
```

```
    } while (0)

struct Task {
    int start;
    int length;
};

__global__ void staticKernel(const Task* __restrict__ tasks,
                             int taskCount,
                             float* __restrict__ results,
                             int* __restrict__ perThreadCount,
                             int* __restrict__ perThreadCost) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    int totalThreads = gridDim.x * blockDim.x;
    int count = 0;
    int cost = 0;
    for (int idx = tid; idx < taskCount; idx += totalThreads) {
        Task t = tasks[idx];
        float acc = 0.0f;
        for (int i = 0; i < t.length; ++i) {
            acc += sinf((float)i * 0.001f) * cosf((float)t.start * 0.0001f);
        }
        results[idx] = acc;
        ++count;
        cost += t.length;
    }
    if (perThreadCount) perThreadCount[tid] = count;
    if (perThreadCost) perThreadCost[tid] = cost;
}

__global__ void dynamicKernel(const Task* __restrict__ tasks,
                              int taskCount,
                              float* __restrict__ results,
                              int* __restrict__ counter,
                              int* __restrict__ perThreadCount,
                              int* __restrict__ perThreadCost,
                              int chunk) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    int count = 0;
    int cost = 0;
    while (true) {
        int begin = atomicAdd(counter, chunk);
        if (begin >= taskCount) break;
        int end = min(begin + chunk, taskCount);
        for (int idx = begin; idx < end; ++idx) {
            Task t = tasks[idx];
            float acc = 0.0f;
            for (int i = 0; i < t.length; ++i) {
                acc += sinf((float)i * 0.001f) * cosf((float)t.start * 0.0001f);
            }
            results[idx] = acc;
            ++count;
            cost += t.length;
        }
    }
    if (perThreadCount) perThreadCount[tid] = count;
    if (perThreadCost) perThreadCost[tid] = cost;
}
```

A host oldalon a feladatkötegeket determinisztikus pszeudo-véletlen generátorral hozzuk létre, hogy a statikus és dinamikus kísérlet pontosan ugyanazt a bemenetet kapja. A mérést `cudaEventRecord`-dal végezzük, előtte öt warm-up futtatást végzünk, majd tíz mérésből átlagot és szórást számítunk. A kernel indítása előtt a dinamikus `counter`-t

`CUDA_CHECK(cudaMemset(d_counter, 0, sizeof(int)))` hívással nullázzuk. A kernel után `CUDA_CHECK(cudaDeviceSynchronize())` biztosítja, hogy a mérés a tényleges befejezést rögzítse.

A laborhoz tartozó build- és futtatási parancsok:

```
nvcc -arch=sm_86 -O3 -std=c++17 -o task_balance task_balance.cu
./task_balance --tasks 4096 --blocks 64 --threads 256 --dynamic-chunk 1
./task_balance --tasks 4096 --blocks 64 --threads 256 --dynamic-chunk 4
```

A kódban a `perThreadCount` és `perThreadCost` tömbök megmutatják, hogy az egyes szálak hány feladatot dolgoztak fel, és mennyi volt a teljes iterációszámuk. A legnagyobb per-thread költség a kiegyensúlyozatlanság közvetlen mérőszáma.

21.2.6 Mérési napló (példa, helyőrző)

A következő napló egy lehetséges laborbejegyzést mutat. A számok illusztrációk, nem ellenőrzött mérések; a hallgató a saját környezetében hajtja végre a mérést, és az alábbi mezőket tölti ki.

```
Környezet:
GPU: NVIDIA RTX 3060 Laptop 6 GB (helyőrző)
Driver: 550.54.14 (helyőrző; ellenőrizd a kiadott verziót)
CUDA: 12.4 (helyőrző)
OS: Ubuntu 24.04 LTS
Compute capability: 8.6
Power limit: 80 W
ECC: nincs
MIG: nincs
Konténer: Docker, nvidia-smi erőforrás-korláttal
Háttérterhelés: nincs

Feladat:
- taskCount: 4096
- hossz: 1..2048 pszeudo-véletlen (LCG seed=42)
- összes hossz: 4 189 943 iteráció
- statikus: 64 blokk x 256 szál = 16 384 szál, ciklikus kiosztás
- dinamikus: ugyanaz a grid, chunk = 1 és chunk = 4

Módszer:
- warm-up: 5 futtatás
- mérés: 10 futtatás, cudaEventElapsedTime
- minden futtatás előtt counter nullázás, eredménytömb újra

Várt trend:
- a statikus kernel per-thread maximális költsége magasabb, mint a dinamikusé;
- a dinamikus kernel futási ideje alacsonyabb, mert csökken a leghosszabb szál ideje;
- a chunk = 4 csökkenti az atomi overheadet, így a futási idő tovább javulhat,
  de a terheléselosztás kissé romolhat.

Mért eredmények (példa, nem ellenőrzött):
- Statikus: átlag 4,21 ms, szórás 0,35 ms; max per-thread cost 8 941 iteráció
- Dinamikus chunk=1: átlag 3,42 ms, szórás 0,18 ms; max cost 4 372 iteráció
- Dinamikus chunk=4: átlag 3,31 ms, szórás 0,15 ms; max cost 4 451 iteráció

Következtetés (példa):
A dinamikus ütemezés ezen a szintetikus, irregularis feladatsoron jobb
kiegyensúlyozást és rövidebb futási időt ad. A chunk méret növelése további
nyereséget hoz, amíg az egyensúly romlása nem válik meghatározóvá.
```

A mérési naplóban a várt és a mért eredményt külön kell választani. A várt trendet a feladatköltségek szórásából és a terheléselosztási modellből vezetjük le; a mért eredmény a konkrét hardveres futtatásból származik. Ha a mért adat jelentősen eltér a várttól, ellenőrizni kell a mérési beállításokat, a GPU órajelét, a hőmérsékletet és a háttérterhelést.

21.2.7 Tipikus hibák és a helyes megoldás

A dinamikus terheléselosztás implementálása során több hiba is előfordulhat. A leggyakoribbak:

1. **A számláló shared memóriában** van, ezért a különböző blokkok nem látják egymás frissítéseit. A globális feladatsorhoz a számlálónak **globális memóriában** kell lennie, és az atomi műveletnek globális címen kell hatnia.
2. **A kernel indítása után a counter-t hostra másoljuk szinkronizáció nélkül**. A `cudaMemcpy` a kernel befejezése előtt nem biztos, hogy a kész értéket látja. A megoldás: `cudaDeviceSynchronize()` a másolás előtt.
3. **A feladatindexet nem ellenőrzik** az atomi növelés után. Az `old` értéke lehet nagyobb vagy egyenlő, mint a `taskCount`; ilyenkor a tömbön kívüli olvasás vagy írás történik. A `while (old < taskCount)` feltétel és a `min` használata kötelező.
4. **A ciklusban elfelejtik a számláló újboli növelését**, így a szál ugyanazt a feladatot dolgozza fel végtelenszer, vagy a ciklus nem lép tovább. A helyes minta: a ciklus elején egy pop, a végén újabb pop.
5. **Túl finom feladatok** esetén az atomi művelet aránya nagy. Ilyenkor a chunk méretét növelni kell, például 4, 8 vagy 16 feladat egyszerre történő kivételével.
6. **A statikus kiosztásban** a `idx += totalThreads` ciklus jó, ha a feladatköltségek közel egyenletesek, de ha a költségek erősen szórnak, a ciklikus kiosztás sem segít. Ilyenkor a dinamikus ütemezés a helyes választás.
7. **A mérési módszertan hiányosságai**: nincs warm-up, csak egy futtatás, a GPU órajelét és power limitjét nem rögzítik. A megbízható benchmark alapja a warm-up, az 5–10 ismétlés, az átlag és szórás, valamint a környezeti feltételek dokumentálása.

A fenti hibák elkerülésével a dinamikus global work queue minta robusztusan alkalmazható irregularis feladatokra.

21.2.8 Önellenőrző kérdések

1. Miért nem elegendő a statikus ciklikus kiosztás akkor, ha a feladatok költsége erősen szór?
2. Milyen memóriatérben kell elhelyezni a globális feladatsor számlálóját, és miért?
3. Hogyan csökkenthető a dinamikus ütemezés atomi műveleteinek overheadje anélkül, hogy a kiegyensúlyozottság jelentősen romoljon?
4. Mi a különbség a közös feladatsor és a work stealing között? Melyik jellemző a GPU-s gyakorlatra, és miért?
5. A laborban mért `max per-thread cost` mit mutat, és miért fontos a teljes futási idő szempontjából?

A következő alfejezetben a teljesítménymodellekre építünk, ahol a feladatgráf kritikus útját, az atomi műveletek overheadjét és a terheléselosztás torzítását is beillesztjük egy átfogó becslési keretbe.

21.3 Teljesítménymodellek

A GPU-teljesítményről akkor tudunk megbízhatóan nyilatkozni, ha a mérés mellett modellünk is van arról, miért éppen annyi a futásidő. A modell célja nem az, hogy a hardver minden részletét visszaadja, hanem hogy a három meghatározó tényező – késleltetés (latency), sávszélesség (bandwidth), kihasználtság (occupancy) – közül megjelölje a szűk keresztmetszetet, és előre jelezze egy-egy optimalizálás hatását. Ebben az alfejezetben egy olyan költségmodellt építünk fel, amely a memóriaműveletek és a számítási műveletek költségéből indul ki, majd a Roofline-elemzéssel és a sorban állási (queueing) modellel finomítja az előrejelzést. A modellt egy kis CUDA-kernelen ellenőrizzük, és a mért adatokat különválasztva hasonlítjuk a várt értékhez.

21.3.1 A modell három építőköve

A GPU-végrehajtás költségét három, egymással összefüggő mennyiséggel jellemezhetjük. A **késleltetés** az az idő, amely alatt egyetlen művelet – például egy global memóriából való betöltés – eljut a kibocsátástól az eredmény rendelkezésre állásáig. A **sávszélesség** az az adatmennyiség, amelyet a memóriarendszer időegység alatt át tud vinni; ez nem a névleges csúcs, hanem a ténylegesen elérhető érték. A **kihasználtság** pedig azt mutatja, hogy az SM-en hány aktív warp van a maximálisan lehetségeshez képest. A három mennyiség közötti kapcsolat a kulcs: a késleltetést csak akkor tudjuk elrejteni, ha elegendő aktív warp áll rendelkezésre, a sávszélesség pedig csak akkor telítődik, ha a memóriarendszerben elegendő párhuzamos kérés van folyamatban.

A modellezés első lépése tehát nem az, hogy egyetlen futásidő-képletet írunk fel, hanem hogy a kernel erőforrásigényét és a hardver korlátait azonosítjuk. Minden kernelre megadható, hogy mennyi adatot mozgat, hány lebegőpontos vagy egész műveletet végez, hány regisztert és mekkora shared memóriát használ, valamint mekkora blokk- és gridmérete van. Ebből a hardver jellemzői – memória-sávszélesség, számítási csúcs, maximális warp-szám, regiszterfájl és shared memory mérete – alapján felépíthető egy olyan egyszerű modell, amelynek pontossága mérnöki döntésekhez elegendő.

21.3.2 Költségmodell egy kernelre

Tekintsünk egy általános, adatpárhuzamos kernelt, amely bemeneti tömbökből olvas, egy kimeneti tömböt ír, és elemenként néhány lebegőpontos műveletet végez. Jelölje N a feldolgozott elemek számát, r az elemenként beolvasott bájtok számát, w az elemenként kiírt bájtok számát, f pedig az elemenkénti lebegőpontos műveletek számát. A teljes adatforgalom

$$D = N \cdot (r + w)$$

bájt, a teljes műveletszám pedig

$$F = N \cdot f$$

FLOP. Ha a memóriarendszer effektív sávszélessége B_{eff} bájt/s, a számítási egység effektív csúcsteljesítménye P_{eff} FLOP/s, akkor a két erőforrás ideális végrehajtási ideje

$$T_{\text{mem}} = \frac{D}{B_{\text{eff}}}, \qquad T_{\text{comp}} = \frac{F}{P_{\text{eff}}}.$$

A modell alapfeltevése szerint a teljes futásidő közelítőleg a két idő maximuma, mert a memória- és a számítási műveletek a GPU-n nagyrészt átfedésben futhatnak:

$$T \approx \max(T_{\text{mem}}, T_{\text{comp}}).$$

Ez a maximumképzés a Roofline-modell alapja is. Ha a kernelnek emellett soros vagy nem átfedhető szakaszai is vannak – például kernelindítás, szinkronizáció, fix költségű inicializálás –, azokat külön kell hozzáadni:

$$T_{\text{total}} \approx T_{\text{launch}} + \max(T_{\text{mem}}, T_{\text{comp}}) + T_{\text{tail}}.$$

A gyakorlatban a kernelindítási overhead nagyságrendje mikroszekundum, ezért hosszú kernelnél elhanyagolható, rövid kernelnél viszont a modellnek tartalmaznia kell.

A fenti képlet azonban csak akkor érvényes, ha a memóriarendszer és a számítási egységek megfelelően ki vannak terhelve. Itt lép be a kihasználtság. Ha a kernel túl kevés aktív warpot indít, akkor a memória-késleltetést nem tudja elrejteni, és az effektív sávszélesség a névleges érték töredékére esik. Ezt a hatást a modellben egy η_B hatékonysági tényezővel vehetjük figyelembe:

$$B_{\text{eff}} = \eta_B \cdot B_{\text{peak}},$$

ahol η_B függ az occupancy-től, a memória-hozzáférési mintától és a cache-ek állapotától. Hasonlóan a számítási csúcsnál:

$$P_{\text{eff}} = \eta_P \cdot P_{\text{peak}}.$$

A η_B és η_P tényezőket kalibrálni kell; a fejezet laborjában ezért először egy memória-kalibráló mérést végzünk, és a modellt ahhoz igazítjuk.

A kihasználtság és az erőforrás-korlátok modellezése a kernel erőforrásigényéből indul. Egy adott blokkméret, regiszterhasználat és shared memory igény mellett kiszámítható az SM-en egyidejűleg futó blokkok és warp-ok száma. Ha az aktív warp-ok száma a hardveres maximum közelében van, a késleltetés-elrejtés általában jó; ha erősen alatta, akkor a sávszélesség és a számítási egységek kihasználtsága is romlik. A modellezésben ezt úgy kezelhetjük, hogy a η_B tényezőt az occupancy függvényében adjuk meg, vagy egyszerűbb esetben megköveteljük, hogy az occupancy egy küszöbérték felett legyen, és a modellben a névleges hatékonysággal számolunk.

21.3.3 Roofline a gyakorlatban

A Roofline-modell a fenti maximumképzést ábrázolja a számítási intenzitás (compute intensity) függvényében. A számítási intenzitás az elemenkénti FLOP és az elemenkénti bájt hányadosa:

$$I = \frac{f}{r + w}$$

FLOP/bájt. A modell két tartományt különböztet meg. Ha

$$I < \frac{P_{\text{peak}}}{B_{\text{peak}}},$$

akkor a kernel memória-korlátos, és a teljesítményt a sávszélesség határozza meg. Ha pedig

$$I > \frac{P_{\text{peak}}}{B_{\text{peak}}},$$

akkor a kernel számítás-korlátos, és a csúcsteljesítmény a mérvadó. A $\frac{P_{\text{peak}}}{B_{\text{peak}}}$ hányadost szokás ridge pointnak nevezni; ez az a számítási intenzitás, amelynél a két korlát metszi egymást.

A Roofline haszna a modellezésben az, hogy gyorsan eldönti, melyik erőforrásra kell koncentrálni. Egy memória-korlátos kernelnél hiába csökkentjük a regiszterhasználatot vagy növeljük az utasításszintű párhuzamosságot, ha a sávszélesség a szűk keresztmetszet; ilyenkor a memória-hozzáférési minta javítása, a tiling, a shared memory használata vagy az adatméret csökkentése segít. Egy számítás-korlátos kernelnél viszont a számítási egységek kihasználtsága, a mixed precision, a tensor core vagy a jobb utasításkeverés hozhat nyereséget.

A modell finomításához érdemes az ábrázolt Roofline-görbét a mért hatékonyságokkal is megrajzolni. A névleges csúcsokból kiindulva a ténylegesen elért sávszélesség és számítási teljesítmény egy-egy vízszintes egyenest ad; a kettő metszéspontja a valós ridge point. A kernel pontja a valós intenzitásnál és a valós teljesítménynél helyezkedik el. Ha a pont a memória-korlátos tartományban van, de a két hatékonysági tényező nagyon eltér, akkor a modell szerint a fejlesztés a sávszélesség felé mutat.

21.3.4 Sorban állás és folyamatban levő kérések

A GPU-n a memória- és a számítási erőforrások nem egyetlen, oszthatatlan erőforrásként viselkednek, hanem párhuzamosan kiszolgálható sorokként. A késleltetés és a sávszélesség közötti kapcsolatot a sorban állási (queueing) modell világítja meg. A legegyszerűbb, de itt is hasznos összefüggés a Little-törvény:

$$N_{\text{active}} = \lambda \cdot T_{\text{latency}},$$

ahol N_{active} a folyamatban levő kérések átlagos száma, λ az átviteli sebesség (kérés/másodperc), T_{latency} pedig egy kérés átlagos késleltetése. A GPU memóriarendszerére alkalmazva: ahhoz, hogy a memória-sávszélességet telítsük, a memóriacsatornában egyszerre annyi kérésnek kell folyamatban lennie, amennyit a késleltetés és a sávszélesség szorzata meghatároz.

Példaként vegyünk egy memóriarendszert, amelynek effektív sávszélessége $B_{\text{eff}} = 1200$ GB/s, a cache-vonal mérete 64 bájt, a memória-késleltetés pedig 400 ciklus 1,4 GHz-es órajel mellett. A tranzakciónkénti átviteli sebesség

$$\lambda = \frac{1200 \cdot 10^9}{64} = 18{,}75 \cdot 10^9$$

tranzakció/s. A késleltetés időben:

$$T_{\text{latency}} = \frac{400}{1{,}4 \cdot 10^9} \approx 286 \cdot 10^{-9}$$

s. A Little-törvényből a szükséges folyamatban levő kérések száma

$$N_{\text{active}} \approx 18{,}75 \cdot 10^9 \cdot 286 \cdot 10^{-9} \approx 5357.$$

Ez a szám azt mutatja, hogy a memóriarendszer telítéséhez több ezer párhuzamos memóriakérésre van szükség. A GPU ezt a nagyszámú kérést az aktív warp-okból nyeri: minden warp több memóriautasítást bocsáthat ki, és a kérések a memóriavezetőben sorban állnak. Ha az aktív warp-ok száma nem éri el ezt a szintet, a memóriacsatorna kihasználtsága lecsökken, és a mért sávszélesség is elmarad a modell várt értékétől.

A sorban állási modell a kernelindításokra és a stream-ütemezésre is kiterjeszthető. Ha ugyanabban a streamben több kernel fut sorban, akkor az egyes kernelek között nincs átfedés, és a teljes idő a kernelek idejének összege. Ha viszont a kerneleket több streambe szervezzük, a második kernel már akkor elindulhat, amikor az első még fut; ekkor a teljes idő a kritikus erőforrás kihasználtsága alapján becsülhető. A sorban állási modell itt a Little-törvénnyel analóg módon működik: a stream-ütemező akkor tudja telíteni a GPU-t, ha elegendő párhuzamos munka áll rendelkezésre, és a késleltetés, valamint az erőforrás-korlátok nem engedik, hogy a sorok kiürüljenek.

A modellezés során a queueing hatását a következőképpen foglalhatjuk össze:

1. Számítsuk ki a szükséges folyamatban levő kérések számát a Little-törvénnyel.
2. Hasonlítsuk össze ezt a számot a kernel által ténylegesen elérhető párhuzamos kérések számával, amelyet az aktív warp-ok száma és az egy warp által kibocsátott memóriautasítások határoznak meg.
3. Ha a tényleges kérésszám alacsonyabb, akkor a modellben csökkentett effektív sávszélességet kell használnunk, vagy explicit módon hozzá kell adnunk a késleltetésből adódó várakozási időt.

Ez a finomítás különösen kis blokkméretű, alacsony occupancy-jű kerneleknél fontos, ahol a naiv Roofline-modell jelentősen felülbecsüli a teljesítményt.

21.3.5 Labor: költségmodell készítése vectorTriad-ra

A labor célja egy egyszerű, memória-korlátos kernel költségmodelljének felállítása, a várt futásidő kiszámítása, majd a mért futásidővel való összevetése. A kernel a klasszikus TRIAD: $z[i] = \alpha \cdot x[i] + y[i]$, ahol x, y és z float tömbök, α pedig skalár. A kernel elemenként 2 lebegőpontos műveletet (szorzás és összeadás) végez, és elemenként 3 float értéket mozgat: 2 beolvasás és 1 kiírás, azaz 12 bájt elemforgalom.

A számítási intenzitás:

$$I = \frac{2}{12} = 0{,}1667$$

FLOP/bájt.

A laborhoz rögzített környezetet használunk; az alábbi értékek **helyőrzők**, amelyeket a tényleges mérés során a saját rendszerünk adataival kell helyettesíteni. Példakörnyezet: NVIDIA A100 40 GB PCIe, driver 550.54.15, CUDA 12.4, compute capability 8.0, Ubuntu 22.04, power limit 300 W, ECC be, MIG ki. A névleges memória-sávszélesség 1555 GB/s, a névleges FP32 csúcs 19,5 TFLOPS. A ridge point:

$$\frac{P_{\text{peak}}}{B_{\text{peak}}} = \frac{19{,}5 \cdot 10^{12}}{1{,}555 \cdot 10^{12}} \approx 12{,}5$$

FLOP/bájt.

Mivel $0{,}1667 < 12{,}5$, a kernel egyértelműen memória-korlátos. A várt futásidő a memóriamodellből számítható, feltételezve, hogy az effektív sávszélesség a névleges 80 %-a, azaz $B_{\text{eff}} \approx 1244$ GB/s. $N = 2^{24}$ elem esetén a teljes adatforgalom:

$$D = 3 \cdot 2^{24} \cdot 4 = 201\,326\,592$$

bájt ≈ 192 MiB. A várt kernelidő:

$$T_{\text{mem}} = \frac{201\,326\,592}{1244 \cdot 10^{9}} \approx 162 \cdot 10^{-6}$$

s ≈ 162 µs.

A számítási idő ehhez képest:

$$T_{\text{comp}} = \frac{2 \cdot 2^{24}}{19{,}5 \cdot 10^{12}} \approx 1{,}7 \cdot 10^{-6}$$

s $\approx 1{,}7$ µs, tehát a kernel valóban memória-korlátos, és a maximumképzés a memóriaidőt adja.

Az alábbi CUDA C++ (C++17, nvcc) kód megvalósítja a kernelt, hibakezelést, időmérést és a mérési napló alapját.

```
// CUDA C++ (C++17, nvcc 12.4)
// Labor: vectorTriad koltsegmodell
#include <cuda_runtime.h>
#include <iostream>
#include <vector>
#include <cmath>
#include <iomanip>
#include <cstdlib>

#define CHECK_CUDA(call)                                                       \
  do {                                                                         \
    cudaError_t err = call;                                                    \
    if (err != cudaSuccess) {                                                  \
      std::cerr << "CUDA error at " << __FILE__ << ":" << __LINE__             \
                << ": " << cudaGetErrorString(err) << std::endl;               \
      std::exit(EXIT_FAILURE);                                                 \
    }                                                                          \
  } while (0)

__global__ void vectorTriad(const float* __restrict__ x,
                            const float* __restrict__ y,
                            float* __restrict__ z,
                            float alpha, size_t n) {
    size_t i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        z[i] = alpha * x[i] + y[i];
    }
}

int main() {
```

```
const size_t N = 1ULL << 24;   // 16 777 216 elem
const float alpha = 2.0f;
const size_t bytes = N * sizeof(float);

std::vector<float> h_x(N), h_y(N), h_z(N);
for (size_t i = 0; i < N; ++i) {
    h_x[i] = static_cast<float>(i % 1000) * 0.001f;
    h_y[i] = static_cast<float>((i % 997)) * 0.001f;
}

float *d_x = nullptr, *d_y = nullptr, *d_z = nullptr;
CHECK_CUDA(cudaMalloc(&d_x, bytes));
CHECK_CUDA(cudaMalloc(&d_y, bytes));
CHECK_CUDA(cudaMalloc(&d_z, bytes));

CHECK_CUDA(cudaMemcpy(d_x, h_x.data(), bytes, cudaMemcpyHostToDevice));
CHECK_CUDA(cudaMemcpy(d_y, h_y.data(), bytes, cudaMemcpyHostToDevice));

cudaEvent_t start, stop;
CHECK_CUDA(cudaEventCreate(&start));
CHECK_CUDA(cudaEventCreate(&stop));

const int threads = 256;
const int blocks = static_cast<int>((N + threads - 1) / threads);

// Warm-up, hogy a GPU orajele es a cache-ek bealljanak.
vectorTriad<<<blocks, threads>>>(d_x, d_y, d_z, alpha, N);
CHECK_CUDA(cudaGetLastError());
CHECK_CUDA(cudaDeviceSynchronize());

const int iters = 10;
std::vector<float> times_ms(iters);
for (int it = 0; it < iters; ++it) {
    CHECK_CUDA(cudaEventRecord(start));
    vectorTriad<<<blocks, threads>>>(d_x, d_y, d_z, alpha, N);
    CHECK_CUDA(cudaGetLastError());
    CHECK_CUDA(cudaEventRecord(stop));
    CHECK_CUDA(cudaEventSynchronize(stop));
    float ms = 0.0f;
    CHECK_CUDA(cudaEventElapsedTime(&ms, start, stop));
    times_ms[it] = ms;
}

// Atlag es szoras
double sum = 0.0, sq_sum = 0.0;
for (float t : times_ms) sum += t;
double avg = sum / iters;
for (float t : times_ms) sq_sum += (t - avg) * (t - avg);
double stddev = std::sqrt(sq_sum / (iters - 1));

double data_bytes = 3.0 * static_cast<double>(N) * sizeof(float);
double bw_eff = data_bytes / (avg * 1e-3) / 1e9; // GB/s

std::cout << std::fixed << std::setprecision(3);
std::cout << "N = " << N << "\n";
std::cout << "Avg kernel time: " << avg << " ms\n";
std::cout << "Stddev: " << stddev << " ms\n";
std::cout << "Measured effective bandwidth: " << bw_eff << " GB/s\n";

// Vart ertek a modellbol, 80%-os hatekonysaggal
double bw_model = 0.80 * 1555.0;
double t_model_ms = data_bytes / (bw_model * 1e9) * 1e3;
std::cout << "Model expected time (80% of peak): " << t_model_ms << " ms\n";
```

```
    std::cout << "Model expected bandwidth: " << bw_model << " GB/s\n";

    CHECK_CUDA(cudaMemcpy(h_z.data(), d_z, bytes, cudaMemcpyDeviceToHost));

    // Ellenorzes: z[i] = alpha*x[i] + y[i]
    bool ok = true;
    for (size_t i = 0; i < N; ++i) {
        if (std::fabs(h_z[i] - (alpha * h_x[i] + h_y[i])) > 1e-5f) {
            ok = false;
            break;
        }
    }
    std::cout << "Numerical check: " << (ok ? "PASS" : "FAIL") << "\n";

    CHECK_CUDA(cudaFree(d_x));
    CHECK_CUDA(cudaFree(d_y));
    CHECK_CUDA(cudaFree(d_z));
    CHECK_CUDA(cudaEventDestroy(start));
    CHECK_CUDA(cudaEventDestroy(stop));
    return 0;
}
```

A kódot `nvcc -O3 -arch=sm_80 -o triad_lab triad_lab.cu` paranccsal fordíthatjuk, majd `./triad_lab` paranccsal futtathatjuk. A mérési naplóban rögzíteni kell a környezeti paramétereket, a buildparancsot, a futtatási parancsot, valamint a mért és várt értékeket külön oszlopban. Példa a mérési naplóra:

Paraméter	Érték
GPU modell	NVIDIA A100 40 GB PCIe (helyőrző)
Driver	550.54.15
CUDA	12.4
Compute capability	8.0
OS	Ubuntu 22.04
Power limit	300 W
ECC	be
MIG	ki
Alapértelmezett órajel	1410 MHz (helyőrző)
N	2^24
Blokk	256 szál
Grid	65536 blokk
Ismétlések	10
Warm-up	1 futtatás

Mért jellemző	Mért érték	Várt érték (modell)	Eltérés
Kernelidő (átlag)	165,3 ms	162,0 ms	+2,0 %
Szórás	0,8 ms	–	–
Effektív sávszélesség	1218 GB/s	1244 GB/s	-2,1 %
Numerikus ellenőrzés	PASS	–	–

A mért és várt érték közötti mintegy 2 %-os eltérés a modell kalibrálatlanságából, az órajel-ingadozásból és a PCIe-alrendszer hatásaiból adódik. Fontos, hogy a táblázatban a mért és a várt oszlopokat mindig külön kezeljük, és az eltérést is feljegyezzük; így a modell pontossága számon kérhető és javítható.

21.3.6 Tipikus modellezési hibák és helyes gyakorlat

A teljesítménymodellek készítésekor több gyakori hiba ismétlődik. Az első, hogy a modell a névleges csúcsértékekkel számol, és figyelmen kívül hagyja a valós hatékonyságot. A valós rendszeren a memória-sávszélesség és a számítási csúcs is jellemzően 20–30 %-kal a névleges alatt marad; ezért a modellben mindig effektív értékeket, vagy legalább kalibrált hatékonysági tényezőket kell használni.

A második gyakori hiba a memória- és a számítási idő összeadása a maximumképzés helyett. A GPU tervezésénél fogva a memória- és a számítási műveletek nagy része átfedésben fut, ezért a teljes időt a nagyobbik idő határozza meg, nem az összeg. Ha a kernelben van soros szakasz, azt külön kell hozzáadni, de a két fő erőforrásnál a maximum a helyes alapfeltevés.

A harmadik hiba az occupancy elhanyagolása. Kevés aktív warp esetén a késleltetés nem rejtődik el, és a mért sávszélesség töredéke lesz a modellben szereplő értéknek. Ezért a költségmodellnek tartalmaznia kell egy ellenőrzést: van-e elegendő párhuzamos kérés a memóriarendszer telítéséhez? Ha nincs, a modellt a sorban állási analízisből származó csökkentett sávszélességgel kell futtatni.

A negyedik hiba az erőforrás-korlátok figyelmen kívül hagyása a launch konfigurációban. A legjobb modell is téves, ha a kernel nem fér el az SM regiszterfájljában vagy shared memory-jában, mert a blokkok sorozatosan futnak, és az occupancy összeomlik. A modellnek ezért a regiszter- és shared memory igényt is össze kell vetnie a hardveres korlátokkal, és a blokkméretet ennek megfelelően kell megválasztani.

Végül gyakori, hogy a mért adatok nem reprodukálhatók, mert nincs warm-up, nincs elegendő ismétlés, vagy a hőmérséklet és az órajel driftje torzít. A helyes gyakorlat a laborban bemutatott módszer: warm-up, legalább 10 ismétlés, átlag és szórás közlése, valamint az aktuális órajel és hőmérséklet rögzítése. A mért és várt értékek külön táblázatban való feltüntetése biztosítja, hogy a modell pontossága ellenőrizhető és javítható.

21.3.7 Összefoglalás és továbbvitel

Ebben az alfejezetben a GPU-teljesítmény modellezésének alapjait építettük fel. Megadtuk a memória- és a számítási költségek képleteit, bevezettük az effektív sávszélesség és csúcsteljesítmény kalibrált tényezőit, és a Roofline-modell segítségével azonosítottuk a szűk keresztmetszetet. A sorban állási elemzéssel megmutattuk, hogy a késleltetés elrejtéséhez hány párhuzamos kérés szükséges, és hogyan kapcsolódik ez az occupancy-höz. A laborban egy memória-korlátos TRIAD kernelen ellenőriztük a modellt, és a mért adatokat a várt értékektől elkülönítve közöltük.

A következő alfejezet (Mintaválasztás és összetétel) erre a modellezési keretre épít, amikor a párhuzamos mintaválasztást és az összetételt tárgyalja. Az itt bevezetett költségmodell, a Roofline-elemzés és a sorban állási gondolkodásmód adja majd az alapot ahhoz, hogy több párhuzamos mintát – map, reduce, scan, stencil, task graph – közös költségfüggvénnyel hasonlítsunk össze, és a feladat igényeihez illő összetételt válasszunk.

Önellenőrző kérdés: Egy kernel elemenként 4 bájtot olvas be, 4 bájtot ír ki, és 1 FLOP-ot végez. A GPU effektív sávszélessége 900 GB/s, effektív FP32 csúcsa 14 TFLOPS. Számítási intenzitása alapján memória- vagy számítás-korlátos a kernel? Mennyi az elvárt átbocsátóképesség elem/másodpercben? (Válasz: $I = 1/8 = 0{,}125$ FLOP/bájt, a ridge point $14/0{,}9 = 15{,}56$ FLOP/bájt, tehát memória-korlátos; az átbocsátás $900 \cdot 10^9/8 = 112{,}5 \cdot 10^9$ elem/s.)

21.4 Mintaválasztás és összetétel

A párhuzamos programozási minták – mint a map, a reduce, a scan, a stencil vagy a feladatgráf – nem elszigetelt trükkök. Valódi alkalmazásokban ritkán fordul elő, hogy egyetlen mintával megoldható a teljes feladat; a mérnöki munka lényege a minták közötti választás, azok összekapcsolása és a kompozíció során fellépő teljesítmény- és karbantarthatósági kérdések kezelése. Ez az alfejezet a Teljesítménymodellek alfejezetben bevezetett teljesítménymodellekre épít, és azt vizsgálja, hogyan lehet a mintákat tudatosan kombinálni anélkül, hogy a kód áttekinthetetlenné válna vagy a teljesítménybeli előnyök elvesznének.

21.4.1 A választás alapkérdése

Amikor egy párhuzamos algoritmust tervezünk, a mintaválasztás nem ízlés kérdése, hanem három egymással összefüggő tényező következménye: az adatfüggőségek szerkezete, a memóriahierarchia viselkedése és a rendelkezésre álló párhuzamosság mértéke. A Redukciók–Feladatpárhuzamosság és terheléselosztás alfejezetekben megismert minták mindegyike más-más adatfüggőségi mintát testesít meg:

- **Map** (Adatpárhuzamos minták): elemenkénti függetlenség, nincs adatcsere.
- **Reduce** (Redukciók): fa struktúrájú függőség, az eredmény egyetlen (vagy kevés) elem.
- **Scan** (Scan (prefix sum)): egydimenziós lánc menti függőség, de átalakítható fává (Hillis–Steele, Blelloch).
- **Stencil** (Stencil-alapok): lokális térbeli függőség, a szomszédos elemek közötti adatcsere.
- **Feladatgráf** (Feladatpárhuzamosság és terheléselosztás): irányított aciklikus gráf (DAG) menti függőség, tetszőleges szerkezettel.

A választás első lépése annak megállapítása, hogy a feladat hány szinten tartalmaz párhuzamosságot. Például egy képfeldolgozó pipeline, amely Gauss-szűrőt alkalmaz, majd küszöböli az eredményt, és végül összegzi a detektált régiók területét, három szintet tartalmaz: a szűrés stencil-minta, a küszöbölés map-minta, a területösszegzés reduce-minta. Az ilyen többszintű szerkezet a gyakori, és a minták összetétele éppen ezt a hierarchiát tükrözi.

A második szempont a memória-hozzáférési minta. A map és a reduce jól illeszkedik a coalescinghez, ha az adatok kontinuus módon vannak elrendezve. A stencil a shared memory tilinggel (Shared memory tiling) hozható ki a globális memória sávszélességéből. A scan a bank conflictokra érzékeny, ezért a CUB vagy a saját implementáció kiválasztásakor a bank-aware változatok előnyt élveznek. A feladatgráf heterogén memória-hozzáférést generálhat, ezért ott a cache-használat és az adatelrendezés kritikusabb, mint a nyers sávszélesség.

A harmadik szempont a rendelkezésre álló párhuzamosság mértéke. Ha a bemeneti elemszám milliós nagyságrendű, a map és a reduce triviálisan kihasználja a GPU-t. Ha azonban a feladat gráfja csak néhány száz csúcsból áll, de a csúcsokhoz tartozó munkamennyiség jelentős, a feladatpárhuzamos megközelítés lehet a helyes választás. A mintaválasztás tehát a párhuzamosság *mennyiségétől* is függ, nem csak a *szerkezetétől*.

21.4.2 A minták osztályozása a választáshoz

A Teljesítménymodellek alfejezetben tárgyalt teljesítménymodellek alapján a mintákat négy dimenzió mentén érdemes osztályozni:

1. **Adatfüggőség szerkezete**: független, fa, lánc, lokális rács, tetszőleges gráf.
2. **Számítási intenzitás**: az egy bájt beolvasásra jutó műveletek száma.

3. **Konkurenciahatár**: hol van a párhuzamosság természetes határa (elem, blokk, grid, stream).
4. **Memória-hozzáférési minta**: coalescing, strided, scattered, shared memory kihasználtság.

Ebből a négy dimenzióból felépíthető egy döntési táblázat, amely a mintaválasztás első közelítését adja:

Minta	Adatfüggőség	Tipikus intenzitás	Konkurenciahatár	Memória-hozzáférés
Map	független	alacsony–magas	elem/grid-stride	coalescing, strided
Reduce	fa	alacsony–közepes	blokk/grid	coalescing, shared
Scan	lánc → fa	alacsony–közepes	blokk/grid	shared, bank-sensitive
Stencil	lokális rács	közepes–magas	blokk/tile	shared, halo
Feladatgráf	tetszőleges DAG	változó	csúcs/stream	scattered

Ez a táblázat nem abszolút igazság, hanem kiindulópont. A gyakorlatban a választás gyakran kényszerpálya: ha a feladat egy mátrixszorzás, akkor a tiled GEMM a stencil és a reduce kombinációja, és ezt a CUTLASS (CUTLASS) vagy a cuBLAS már megoldotta. A saját implementációt akkor érdemes választani, ha az adatszerkezet nem standard, vagy ha a bemeneti méret olyan kicsi, hogy a könyvtári hívások overheadje dominál.

21.4.3 Esettanulmány: csúszóablakos variancia és küszöbölés

Tegyük fel, hogy a feladat a következő: egy idősort kapunk, amely egymillió lebegőpontos minta hosszúságú. Minden pontban ki kell számítani az utolsó 64 minta alapján a varianciát, majd meg kell határozni azokat a régiókat, ahol a variancia meghalad egy küszöböt, és végül jelenteni kell a régiók számát és az átlagos hosszukat. Ez a feladat három mintát igényel:

1. **Stencil-minta**: a csúszóablakos variancia számítása. Minden kimeneti elemhez 64 bemeneti elemre van szükség, tehát a memória-hozzáférési minta átfedő ablak. A variancia számítható a `sum(x)` és `sum(x²)` értékekből, így a stencil maga két csúszóösszeg kombinációja.
2. **Map-minta**: a küszöbölés. A varianciaértékeken elemenkénti összehasonlítást végzünk, a kimenet bináris (vagy 0/1 flag).
3. **Reduce-minta**: a régiók összegzése. Ez nem triviális, mert a küszöbölt jelben egymás után következő 1-esek régiókat alkotnak. A régiók számának és átlagos hosszának meghatározása egy *szegmentált* reduce, amelyhez először run-length encodingra (az egymást követő azonos elemek hosszának kódolására) van szükség.

A mintaválasztás itt nem kérdéses: a feladat elemzése világosan kijelöli a három mintát. A kérdés inkább az, hogy **hogyan komponáljuk** ezeket. Három lehetőség van:

- **Külön kernelek**: minden minta saját kernelbe kerül, a köztes eredmények a globális memóriába kerülnek.
- **Kernel-fúzió**: a stencil és a küszöbölés egyetlen kernelben történik, a köztes varianciaértékek a regiszterekben és a shared memoryban maradnak.
- **Perzisztens kernel**: a teljes pipeline egyetlen kernelben fut, grid-stride loop-pal, a köztes adatok a shared memoryban és a regiszterekben maradnak.

A választást a memória-sávszélesség és az adat újrafelhasználása határozza meg. A stencil 64 elem széles ablakot használ, tehát minden bemeneti elemet 64-szer olvasunk be, ha naiv módon implementáljuk. A külön kernelek esetén a variancia kimenete egymillió float, a küszöbölés ezt olvassa be és írja ki a bináris tömböt. A reduce a bináris tömböt olvassa. Az adatforgalom:

- Stencil: 1 M bemenet * 64 (naiv) + 1 M kimenet = 65 MB olvasás + 4 MB írás.
- Küszöbölés: 4 MB olvasás + 1 MB írás.
- Reduce: 1 MB olvasás + néhány bájt írás.

A fúziós változatban a stencil és a küszöbölés egy kernelben van, így a variancia-kimenet elkerülhető: a küszöbölés közvetlenül a variancia regiszterbeli értékét használja. Az adatforgalom:

- Fuzionált kernel: 1 M bemenet (naiv: 64 M olvasás) + 1 MB bináris kimenet olvasása a reduce-ban.

A fúzió tehát 4 MB olvasást és 4 MB írást takarít meg. Ez látszólag nem nagy különbség, de ha a stencil memória-optimalizálását (csúszóösszeg megosztás) is alkalmazzuk, a naiv 64-szeres beolvasás 2-szeresre csökkenthető, így a fúzió megtakarítása a teljes memóriaforgalom jelentős részét teszi ki.

21.4.4 A csúszóösszeg-alapú stencil megvalósítása

A variancia csúszóablakos számítása két futó összeggel történik. Az ablak `W` méretű (itt 64), és a `sum(x)` és `sum(x²)` értékeket csúsztatva frissítjük:

```
S1[i] = S1[i-1] + x[i] - x[i-W]
S2[i] = S2[i-1] + x[i]² - x[i-W]²
```

A variancia ezután:

```
Var[i] = S2[i]/W - (S1[i]/W)²
```

Ez a megfogalmazás a stencil-mintát egy *aszimmetrikus ablakú* függőséggé alakítja: a `x[i-W]` elem kivonása miatt az ablak két szélét érinti. Az implementáció során a bemeneti adatokat a shared memoryba kell tölteni, és a csúszóösszeget az egyes szálak a saját regisztereikben tarthatják, ha minden szál egy adott kimeneti pozíciót számít. Ez a megoldás a Stencil-alapok alfejezetben bemutatott stencil-tiling hatékony speciális esete.

21.4.5 CUDA C++ (C++17, CUDA 12.x) – fuzionált stencil + küszöbölés

```cpp
// variancia_threshold.cu
// Build: nvcc -O3 -arch=sm_80 -o variancia_threshold variancia_threshold.cu
// Futtatási környezet: NVIDIA A100, CUDA 12.2, driver 535, Linux x86_64
// Mérés: legalább 5-10 ismétlés, warm-up után; az alábbi kód a mérési naplóhoz szükséges.

#include <cuda_runtime.h>
#include <cstdio>
#include <cmath>
#include <vector>
#include <chrono>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
```

```
        cudaError_t err = (call);                                               \
        if (err != cudaSuccess) {                                               \
            fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__, __LINE__,    \
                    cudaGetErrorString(err));                                   \
            cudaDeviceReset();                                                  \
            exit(EXIT_FAILURE);                                                 \
        }                                                                       \
    } while (0)

// Kernel: csúszóablakos variancia és küszöbölés.
// A bemeneti x tömb mérete n; a block mérete blockDim.x = 256.
// Minden szál egy kimeneti pozíciót számít, a csúszóösszeget a regiszterekben tartja.
// A bemeneti adatokat a shared memoryba töltjük, hogy a kétszeri olvasást elkerüljük.
// A kimeneti flag 1, ha Var > threshold, különben 0.
__global__ void variance_threshold_kernel(
    const float * __restrict__ x,
    unsigned char * __restrict__ flag,
    int n,
    int window,
    float threshold)
{
    extern __shared__ float s_x[];  // dinamikus shared memory: window + blockDim.x méretű

    const int tid = threadIdx.x;
    const int idx = blockIdx.x * blockDim.x + tid;

    // A szükséges bemeneti tartomány: [blockIdx.x * blockDim.x, blockIdx.x * blockDim.x +
    ↪  blockDim.x + window)
    // A shared memoryba betöltjük a tartományt, az elején a window elemmel korábbi részt.
    // Az egyszerűség kedvéért feltételezzük, hogy a tartomány nem lóg túl a tömb határán;
    // a gyakorlatban a széleket a host oldali padding vagy a kernelbeli feltétel kezeli.

    const int base = blockIdx.x * blockDim.x;
    const int shared_size = blockDim.x + window;

    // Kooperatív betöltés a shared memoryba
    for (int i = tid; i < shared_size; i += blockDim.x) {
        int src = base + i;
        if (src < n) {
            s_x[i] = x[src];
        } else {
            s_x[i] = 0.0f;  // zero-padding a tömb szélén
        }
    }
    __syncthreads();

    // Ha a szál kimeneti indexe a tartományon belül van:
    if (idx < n) {
        // A szál lokális pozíciója a shared memoryban: a shared memory [0..window-1] a
        ↪  bevezető,
        // a [window..shared_size-1] a blokk saját adata.
        // A szál saját eleme: s_x[tid + window]
        // A szálhoz tartozó csúszóösszeg kezdőértéke: az első window elem összege a megfelelő
        ↪  pozícióra.
        // Egyszerűsített implementáció: minden szál az ablak elemeit közvetlenül összegzi,
        // majd a csúsztatást regiszterekben végzi. Ez a naiv 2-szeres olvasás, de a shared
        ↪  memory
        // miatt a globális memóriaforgalom már csak 1-szeres.
        float s1 = 0.0f;
        float s2 = 0.0f;
        // Kezdő ablak összegzése a szál saját pozíciójának elejétől
        #pragma unroll
        for (int j = 0; j < window; ++j) {
```

```
            float val = s_x[tid + j];  // window elemmel eltolva
            s1 += val;
            s2 += val * val;
        }
        // A variancia számítása és küszöbölés
        float mean = s1 / (float)window;
        float var = s2 / (float)window - mean * mean;
        flag[idx] = (var > threshold) ? 1 : 0;
    }
}

int main() {
    // Konfiguráció (rögzített a mérési naplóhoz)
    const int n = 1 << 20;          // 1 048 576 elem
    const int window = 64;
    const float threshold = 0.5f;

    // Host allokáció
    std::vector<float> h_x(n);
    std::vector<unsigned char> h_flag(n, 0);
    // Tesztadat: szinusz + zaj, hogy a variancia értelmes legyen
    for (int i = 0; i < n; ++i) {
        h_x[i] = sinf(0.01f * (float)i) + 0.2f * cosf(0.005f * (float)i);
    }

    // Eszköz allokáció
    float *d_x;
    unsigned char *d_flag;
    CUDA_CHECK(cudaMalloc(&d_x, n * sizeof(float)));
    CUDA_CHECK(cudaMalloc(&d_flag, n * sizeof(unsigned char)));
    CUDA_CHECK(cudaMemcpy(d_x, h_x.data(), n * sizeof(float), cudaMemcpyHostToDevice));

    // Launch konfiguráció
    const int block = 256;
    const int grid = (n + block - 1) / block;
    const size_t shared_bytes = (block + window) * sizeof(float);

    // Warm-up (2 futtatás, nem mérjük)
    for (int iter = 0; iter < 2; ++iter) {
        variance_threshold_kernel<<<grid, block, shared_bytes>>>(d_x, d_flag, n, window,
↪   threshold);
    }
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaGetLastError());

    // Mérés: 10 ismétlés
    cudaEvent_t start, stop;
    CUDA_CHECK(cudaEventCreate(&start));
    CUDA_CHECK(cudaEventCreate(&stop));

    float total_ms = 0.0f;
    for (int iter = 0; iter < 10; ++iter) {
        CUDA_CHECK(cudaEventRecord(start));
        variance_threshold_kernel<<<grid, block, shared_bytes>>>(d_x, d_flag, n, window,
↪   threshold);
        CUDA_CHECK(cudaEventRecord(stop));
        CUDA_CHECK(cudaEventSynchronize(stop));
        float ms = 0.0f;
        cudaEventElapsedTime(&ms, start, stop);
        total_ms += ms;
    }
    float avg_ms = total_ms / 10.0f;
```

```
    // Eredmény visszamásolása
    CUDA_CHECK(cudaMemcpy(h_flag.data(), d_flag, n * sizeof(unsigned char),
↪   cudaMemcpyDeviceToHost));

    // Ellenőrzés: a zászlók száma
    int count = 0;
    for (int i = 0; i < n; ++i) count += h_flag[i];

    printf("n=%d, window=%d, threshold=%.2f\n", n, window, threshold);
    printf("block=%d, grid=%d, shared_bytes=%zu\n", block, grid, shared_bytes);
    printf("Átlagos futásidő: %.3f ms\n", avg_ms);
    printf("Átbocsátóképesség: %.2f GByte/s (bemenet 1x olvasása + kimenet írása alapján)\n",
           (n * sizeof(float) + n) / (avg_ms * 1e-3) / 1e9);
    printf("Detektált küszöb feletti pontok: %d\n", count);

    // Takarítás
    CUDA_CHECK(cudaFree(d_x));
    CUDA_CHECK(cudaFree(d_flag));
    CUDA_CHECK(cudaEventDestroy(start));
    CUDA_CHECK(cudaEventDestroy(stop));
    return 0;
}
```

A kódban a stencil naiv shared-memory változata szerepel: az ablakot minden szál újra összegzi, de a globális memória-olvasás már csak egyszeres, mert az adat a shared memoryban van. A csúszóösszeg-regiszteres optimalizálás további 2-4-szeres gyorsulást adhat, de a kód bonyolultsága nő. Ez a példa a *mintaválasztás* tanítására szolgál: a stencil kiválasztása után a memória-optimalizálás külön lépés, amely a Shared memory tiling és Stencil-alapok anyagára épül.

21.4.6 A kompozíció szintjei

A minták összetétele három szinten történhet:

21.4.7 1. Kernel-fúzió

A legegyszerűbb kompozíció, amikor két vagy több minta egyetlen kernelbe kerül. A fenti példában a stencil és a map (küszöbölés) fúziója látható. A fúzió akkor előnyös, ha:

- A köztes eredmény nagy, és a memória-sávszélesség szűk keresztmetszet.
- A minták közötti adatcsere reguláris, nem igényel blokk-szintű szinkronizációt.
- A fúzió nem növeli jelentősen a regiszterhasználatot, ami az occupancy csökkenéséhez vezetne.

A fúzió hátránya a karbantarthatóság romlása: a fuzionált kernel nehezebben tesztelhető, és ha a két minta külön-külön is használható, a duplikáció nő. A mérnöki döntés itt a teljesítmény és az újrafelhasználhatóság közötti kompromisszum.

21.4.8 2. Köztes memória + kernel-szekvencia

A második szint a hagyományos megközelítés: minden minta külön kernelben fut, a köztes eredmények a globális memóriában vannak. Ez a legegyszerűbb, legjobban tesztelhető és legjobban karbantartható, de a legnagyobb memóriaforgalommal jár. Akkor érdemes választani, ha:

- A köztes eredmény kicsi (például egy reduce után egyetlen érték).
- A minták közötti szinkronizáció elkerülhetetlen (például a reduce befejezése után kell a scan).
- A fejlesztés korai szakaszában vagyunk, és a helyesség fontosabb, mint a teljesítmény.

21.4.9 3. Perzisztens kernel és feladatgráf

A harmadik szint a perzisztens kernel: a GPU-n egyetlen kernel fut, amely a teljes pipeline-t végrehajtja, grid-stride loop-pal. Ez a megközelítés a feladatgráf GPU-n belüli implementációja is lehet: a kernel minden szálcsoportja más feladatot hajt végre, a szinkronizációt a cooperative groups (Cooperative groups alapok) biztosítja. A perzisztens kernel előnye a minimális overhead, de:

- A grid-szintű szinkronizáció csak cooperative launch-csal lehetséges.
- A regiszterhasználat könnyen elfogyhat, ha sok különböző mintát kell ugyanabban a kernelben támogatni.
- A hibakeresés és a profilozás nehezebb, mert a minták határai elmosódnak.

21.4.10 A választási folyamat a gyakorlatban

A fenti szempontok alapján a mintaválasztás és -kompozíció folyamata a következő lépésekből áll:

1. **Adatfüggőségek elemzése**: rajzoljuk fel a feladat függőségi gráfját. Azonosítsuk a független részeket (map), a folyamatos láncokat (scan), a lokális rácsokat (stencil), a fákat (reduce) és az általános DAG-okat (feladatgráf).
2. **Párhuzamosság becslése**: becsüljük meg minden szinten az adatpárhuzamosság mértékét. Ha egy szinten kevesebb mint, mondjuk, 10 000 párhuzamos egység van, a GPU kihasználtsága alacsony lesz; ilyenkor a feladatpárhuzamos megközelítés vagy a CPU-ra való visszaterelés lehet a helyes.
3. **Memória-hozzáférési minta azonosítása**: minden minta esetén határozzuk meg, hogy a memória-hozzáférés coalescing, strided, scattered vagy shared-memory-barát-e. A scattered hozzáférést igénylő minták gyakran a globális memória latency-jától szenvednek, ezért a cache-használat kulcsfontosságú.
4. **Köztes eredmények méretének becslése**: ha a köztes eredmény nagy, a kernel-fúzió vagy a perzisztens kernel előnyös. Ha kicsi, a kernel-szekvencia egyszerűbb és elegendő.
5. **Könyvtári minták keresése**: mielőtt saját implementációt írunk, ellenőrizzük, hogy a CUB, a Thrust, a cuBLAS vagy a CUTLASS tartalmaz-e kész megoldást. Ha igen, a választás a könyvtári hívásra esik, kivéve ha az adatszerkezet nem illeszkedik, vagy a hívás overheadje túl nagy.

6. **Prototípus és mérés**: a választott kompozíciót prototípusként implementáljuk, és a Benchmarkmódszertan alfejezetben leírt benchmarkmódszertan szerint mérjük. Ha a teljesítmény nem éri el a célértéket, a szűk keresztmetszetet a Nsight Compute (Nsight Compute és a kernelmetrikák) segítségével azonosítjuk.

Ez a folyamat iteratív: a mérési eredmények visszacsatolása alapján a mintaválasztást vagy a kompozíció szintjét módosíthatjuk.

21.4.11 Újrafelhasználás és karbantarthatóság

A minták kompozíciója nem csak teljesítménykérdés. A jól megválasztott minta-kompozíció a kód újrafelhasználhatóságát is szolgálja. A CUDA-ökoszisztémában a Thrust (Thrust és CUB) éppen ezt a célt szolgálja: a `thrust::transform` (map), a `thrust::reduce`, a `thrust::inclusive_scan` és a `thrust::sort` olyan magas szintű építőkövek, amelyeket tetszőlegesen kombinálhatunk. A Thrust hátránya, hogy a minták közötti fúzió nem mindig lehetséges, és a köztes eredmények a globális memóriába kerülnek. A CUB ennél alacsonyabb szintű, de nagyobb kontrollt ad a shared memory és a blokk-szintű műveletek felett.

A karbantarthatóság szempontjából a következő elvek vezethetők le:

- **Egy kernel, egy minta**: ha a kernelben több minta keveredik, a hibakeresés nehéz. A fúzió csak akkor indokolt, ha a teljesítményelőny mérhető és szükséges.
- **A mintahatárok legyenek explicitek**: a fuzionált kernelben kommentekkel jelöljük, hol ér véget az egyik minta és hol kezdődik a másik. A kooperatív csoportok (Cooperative groups alapok) segíthetnek a szinkronizációs határok jelölésében.
- **A köztes eredmények típusa és mérete legyen dokumentált**: a kernel-szekvencia esetén a köztes pufferek jelentik az interfészt a minták között; ezek típusát, méretét és élettartamát a kód elején rögzítsük.
- **A mintaválasztást indokoljuk**: a kódhoz tartozó tervezési dokumentumban (lásd a labort) rögzítsük, miért az adott mintát választottuk, milyen alternatívákat vettünk figyelembe, és milyen mérési eredmények igazolják a döntést.

21.4.12 Labor: mintaalapú tervezési dokumentum

Ez a labor a Mintaválasztás és összetétel tanulási céljainak gyakorlati ellenőrzése. A feladat nem egyetlen optimalizált kernel elkészítése, hanem egy teljes tervezési dokumentum, amely bemutatja a mintaválasztást, a kompozíciót és a mérési eredményeket.

21.4.13 Feladatleírás

Válasszon egyet az alábbi három feladat közül:

1. **Hisztogram-simítás és küszöbölés**: számítsa ki egy kép hisztogramját, simítsa a hisztogramot egy 5-pontos stencillel, majd keresse meg a simított hisztogram csúcsait (lokális maximumok), és jelentse a csúcsok helyét és értékét.
2. **Monte Carlo-becslés és variancia-redukció**: generáljon véletlen pontokat, számítsa ki a körön belüli pontok arányát, majd alkalmazzon szegmentált redukciót a variancia becsléséhez, és végül egy küszöbölési lépést a konvergencia ellenőrzéséhez.

3. **Mátrix-sorok normája és rendezés**: számítsa ki egy ritka mátrix minden sorának L2-normáját, majd rendezze a sorokat a norma szerint csökkenő sorrendbe, és végül egy prefix összeggel határozza meg a sorok kumulatív normáját.

21.4.14 Átadandó eredmény

A tervezési dokumentum a következő részekből áll:

1. **Feladatleírás és függőségi gráf**: rajzolja fel a feladat adatfüggőségi gráfját, és azonosítsa a mintákat (map, reduce, scan, stencil, feladatgráf). Minden mintánál adja meg az adatfüggőség szerkezetét, a konkurenciahatárt és a memória-hozzáférési mintát.
2. **Mintaválasztás indoklása**: minden szinten indokolja, miért az adott mintát választotta, és milyen alternatívát vett figyelembe. Jelölje, ha könyvtári megoldást (Thrust, CUB, cuBLAS) használt volna, és miért tért el tőle, vagy miért használta.
3. **Implementáció**: CUDA C++ (vagy OpenCL, JAX) kód, amely a választott mintákat és kompozíciót tartalmazza. A kódban minden API-hívás hibáját ellenőrizze, és a kernel launch alakja legyen `<<<grid, block, shared_bytes, stream>>>`. A kódban a mintahatárokat kommentekkel jelölje.
4. **Mérési napló**: rögzítse a GPU típusát, a drivert, a CUDA-verziót, az operációs rendszert, a compute capability-t, az órajelet, a power limitet és a háttérterhelést. A mérés előtt warm-up, majd legalább 5-10 ismétlés, átlaggal és szórással. A mért és a várt eredményt különítse el: a várt eredmény az elméleti modellből (Teljesítménymodellek) számított érték, a mért eredmény a tényleges futásidő.
5. **Önellenőrző kérdés**: válaszoljon a következőre: „Ha a feladatban a köztes eredmény mérete a bemenet méretének kétszerese lenne, hogyan változna a mintaválasztás és a kompozíció szintje?” A válaszban hivatkozzon a saját feladatának konkrét paramétereire.

21.4.15 Példa a mérési napló szerkezetére

Paraméter	Érték
GPU	NVIDIA A100 40 GB PCIe
Driver	535.104.05
CUDA	12.2
Compute capability	8.0
Operációs rendszer	Ubuntu 22.04 LTS, x86_64
Órajel (fix)	1410 MHz
Power limit	250 W (alapértelmezett)
ECC	be
MIG	ki
Háttérterhelés	nincs más CUDA-folyamat
Bemeneti méret	n = 1 048 576 elem
Blockméret	256
Gridméret	4096
Shared memory	1280 bájt
Warm-up	2 futtatás
Ismétlések	10
Átlagos futásidő	0,834 ms

Paraméter	Érték
Szórás	0,021 ms
Várt futásidő (roofline modell)	0,72 ms
Eltérés	+15,8% (a naiv shared-memory impl. miatt)

A mérési naplót teljesen kitöltve kell átadni. A fenti táblázat csak a szerkezetet mutatja, a konkrét értékek a saját mérésből származnak.

21.4.16 Tipikus hibák és javításuk

A mintaválasztás során a következő hibák gyakoriak:

21.4.17 1. Túlzott fúzió

Hiba: a fejlesztő minden mintát egyetlen hatalmas kernelbe fuzionál, hogy elkerülje a memória-forgalmat, de a kernel regiszterhasználata 255 fölé emelkedik, az occupancy drasztikusan csökken, és a teljesítmény rosszabb lesz, mint a külön kernelek esetén.

Javítás: mérje a regiszterhasználatot a `--ptxas-options=-v` kapcsolóval. Ha a regiszterek száma 128 fölé emelkedik, a fúziót érdemes felbontani. A Nsight Compute occupancy rovatában ellenőrizze, hogy a fuzionált kernel occupancy 50% alatt van-e; ha igen, a szétválasztás a helyes.

21.4.18 2. Nem megfelelő minta a szinkronizációs határhoz

Hiba: a fejlesztő a scan-mintát globális memóriában implementálja, és a köztes eredményeket kernel-szekvenciával viszi át, de a scan közben a blokkok közötti szinkronizációt nem oldja meg, így a részeredmények helytelenek.

Javítás: a scan esetén a blokk-szintű szinkronizációt a shared memory és a `__syncthreads()` biztosítja, a grid-szintűt pedig a CUB `DeviceScan` vagy a Thrust `inclusive_scan` könyvtári megoldása. Ha saját implementáció szükséges, a Scan (prefix sum) alfejezetben bemutatott kétfázisú (Blelloch) megközelítést kövesse.

21.4.19 3. A mintaválasztás elhanyagolása a feladat kis mérete esetén

Hiba: a fejlesztő a reduce-mintát választja egy 128 elemű tömb összegzésére, de a kernel launch overheadje nagyobb, mint a számítás ideje. A futásidő 10 µs, de a kernel maga 2 µs.

Javítás: kis elemszám esetén a GPU-t ki lehet hagyni, vagy a Thrust `reduce` hívása előtt a kis tömböt a CPU-ra vissza lehet másolni. A Mikor éri meg GPU-ra vinni egy feladatot? alfejezetben bemutatott döntési szempontok itt érvényesek.

21.4.20 4. A könyvtári lehetőségek figyelmen kívül hagyása

Hiba: a fejlesztő saját radix sort implementál, pedig a CUB `DeviceRadixSort` gyorsabb és teszteltebb. A saját implementáció 2-szer lassabb, és a fejlesztő 3 napot tölt a hibakereséssel.

Javítás: a Könyvtártérkép és döntési szempontok alfejezettől kezdődően a könyvtártérkép áttekintése kötelező a mintaválasztás előtt. Ha a könyvtár illeszkedik, használja; ha nem, akkor indokolja.

21.4.21 Önellenőrző kérdés

A fenti laborfeladat 2. opciójában (Monte Carlo-becslés és variancia-redukció) a fejlesztő a következő megoldást választja: a véletlen pontok generálása map-minta, a körön belüli pontok arányának becslése reduce-minta, a variancia becslése egy második reduce-minta, a konvergencia ellenőrzése map-minta. A fejlesztő mind a négy mintát külön kernelben futtatja, a köztes eredményeket a globális memóriában tárolja. A bemeneti méret 1 millió pont. Kérdés: milyen kompozíciós szintet választana, és miért? Milyen mérési adatok hiányoznak a döntéshez?

Válasz: a döntéshez ismerni kell a köztes eredmények méretét és a memória-sávszélesség kihasználtságát. A reduce-minták kimenete kicsi (egy-egy lebegőpontos érték), ezért a kernel-szekvencia overheadje a launch költségekben jelentkezik, nem a memóriaforgalomban. A map-minta (véletlen pontok generálása) kimenete nagy (1 millió pont), ezért a fúzió a generálás és a reduce között előnyös lehet: a véletlen pontokat nem kell a globális memóriába írni, hanem a generáló kernelben közvetlenül felhasználhatók a reduce-hoz, ha a reduce blokk-szintű részét a generáló kernelbe integráljuk. A mérési adatok közül hiányzik: a generáló kernel és a reduce kernel külön futásideje, a köztes eredmény mérete, a shared memory kihasználtsága, és a regiszterhasználat. Ha a generáló kernel 1 ms-ig fut, a reduce 0,3 ms-ig, a köztes eredmény 8 MB, akkor a fúzió megtakarítása a 8 MB globális memória írás-olvasás, ami a teljes memóriaforgalom jelentős része. Ha azonban a generáló kernel regiszterhasználata már így is 128 regiszter, a fúzió az occupancy csökkenéséhez vezethet, ezért a mérés nélkül nem dönthető el.

21.4.22 Összefoglalás

A minták közötti választás a párhuzamos programozás legfontosabb tervezési döntése. A választás az adatfüggőségek szerkezetéből, a párhuzamosság mértékéből, a memória-hozzáférési mintából és a rendelkezésre álló könyvtárakból következik. A kompozíció szintje – külön kernelek, fúzió vagy perzisztens kernel – a köztes eredmények méretétől és a teljesítménykövetelményektől függ. A helyes választás nem egyszeri döntés, hanem iteratív folyamat, amelyet mérések és profilozás igazolnak. A karbantarthatóság szempontjából a mintahatárok explicitek maradjanak, és a választást dokumentálni kell. A következő fejezet (Könyvtártérkép és döntési szempontok) a CUDA könyvtártérképének részletes bemutatásával segíti a választást, ahol a minták már kész, optimalizált implementációk formájában állnak rendelkezésre.

V. RÉSZ – CUDA-ökoszisztéma és mélytanulás

22 Könyvtárak: cuBLAS, cuSPARSE, cuFFT, cuRAND, Thrust, CUB

22.1 Könyvtártérkép és döntési szempontok

A CUDA-ökoszisztéma érettségét nemcsak a hardver generációk, hanem a hivatalos, optimalizált könyvtárak is jelzik. A fejlesztő gyakran nem saját kerneleket ír, hanem olyan könyvtári hívásokat keres, amelyek egy adott problémát az adott GPU-generáción a lehető legjobb teljesítménnyel oldanak meg. Ez az alfejezet áttekintést ad a legfontosabb CUDA-könyvtárakról, azok tipikus használati területeiről, valamint arról, hogyan hozzunk megalapozott választást. A következő egységekben az egyes könyvtárak részletes API-ját és optimalizálási lehetőségeit tárgyaljuk; itt a térképre és a döntési logikára fókuszálunk.

22.1.1 A CUDA-könyvtárak térképe

A legismertebb hivatalos könyvtárakat két csoportra bonthatjuk. Az első csoportba a nagy, bináris formában szállított, linkelhető könyvtárak tartoznak: **cuBLAS**, **cuSPARSE**, **cuFFT**, **cuRAND**. A második csoportba a forráskódban elérhető, header-only vagy header-orientált építőelemek: **Thrust** és **CUB**. A kettő nem versenytárs, hanem kiegészítő réteg: a Thrust magas szintű, STL-szerű algoritmusokat ad, a CUB pedig alacsony szintű primitíveket biztosít, amelyekre a Thrust is épül.

22.1.2 cuBLAS – dense lineáris algebra

A cuBLAS a BLAS (Basic Linear Algebra Subprograms) szabvány GPU-megvalósítása. Tipikus műveletek:

- **BLAS Level 1:** vektor–vektor műveletek (axpy, dot, normák).
- **BLAS Level 2:** mátrix–vektor szorzat (gemv, trsv).
- **BLAS Level 3:** mátrix–mátrix szorzat (gemm), triangular solve (trsm).

A cuBLAS a dense mátrixokra optimalizált. A modern változatok támogatják a **tensor core** utasításokat FP16, BF16, TF32, FP64 és INT8 precizitással, valamint a batched és strided batch műveleteket. A használathoz egy `cublasHandle_t` handle szükséges, amely a művelet kontextusát tárolja. A cuBLAS mátrixokat alapértelmezésben **column-major** (Fortran) elrendezésben vár; a C++-ban gyakori row-major adatokkal való keverés tipikus hibákat okoz. A cuBLAS az elsődleges választás dense lineáris algebra, neurális hálózatok lineáris rétegei és tudományos számítási magok esetén.

22.1.3 cuSPARSE – sparse lineáris algebra

A cuSPARSE a ritka mátrixok műveleteit támogatja. Fontosabb formátumok:

- **COO** (koordináta) – egyszerű, de kevésbé hatékony.
- **CSR** (Compressed Sparse Row) – a leggyakoribb, jól párhuzamosítható.
- **CSC** (Compressed Sparse Column) – oszloporientált műveletekhez.
- **BLOCKED-ELL** – vektorizálásra alkalmas, szabályos blokkokkal.

A leggyakrabban használt művelet a **SpMV** (sparse matrix–vector multiply) és a **SpMM** (sparse matrix–dense matrix multiply). A cuSPARSE modern változata már tensor core-t is képes használni ritka-dense szorzatokhoz, de ez formátumfüggő. A cuSPARSE akkor előnyös, ha a mátrix ritkasági aránya magas (jellemzően 60–70% feletti nulla), és a memória-sávszélesség a fő korlát. Dense esetben a cuBLAS szinte mindig jobb, mert a sparse formátumok indexelési overhead-et visznek.

22.1.4 cuFFT – gyors Fourier-transzformáció

A cuFFT a diszkrét Fourier-transzformáció (DFT) GPU-megvalósítása. Támogatja az 1D, 2D és 3D transzformációkat, valós és komplex adatokat, valamint a batch végrehajtást. A könyvtár **plan**-t használ: a `cufftPlan1d`, `cufftPlan2d` stb. létrehoz egy optimalizált végrehajtási tervet, amelyet azután többször is felhasználhatunk. A plan-ek létrehozása költséges, ezért a cuFFT-t batch feldolgozásra tervezték. A cuFFT a jelfeldolgozás, képfeldolgozás, PDE-spektrális módszerek és szimulációk alapvető eszköze. A pontosság FP32, FP64, FP16 (korlátozott) lehet; a transzformáció típusa C2C, R2C, C2R.

22.1.5 cuRAND – véletlenszám-generálás

A cuRAND véletlenszám-generátorokat kínál GPU-ra. Két fő API-t különböztetünk meg:

- **Host API:** a CPU hívja a generátort, a GPU-n generálódik a kimenet.
- **Device API:** a kernel belsejében hívható függvények.

A generátorok típusai: XORWOW, Philox, MRG32k3a, MTGP32. A Philox és a MRG32k3a gyakran jobb statisztikai tulajdonságú, míg az XORWOW gyorsabb. A cuRAND támogat egyenletes, normális, exponenciális és más eloszlásokat. A Monte Carlo-szimulációk, a random dropout, a súlyinicializálás és a sztochasztikus algoritmusok építenek rá. A determinizmus szempontjából fontos: azonos seed és könyvtárverzió mellett sem garantált a bitpontos azonosság különböző GPU-architektúrák között; ezt a laboroknál rögzíteni kell.

22.1.6 Thrust – magas szintű párhuzamos C++ algoritmusok

A Thrust a C++ Standard Template Library (STL) mintáját követi, de párhuzamos backenden fut. Olyan gyakori algoritmusokat ad, mint:

- `thrust::reduce` – összegzés, minimum, maximum;
- `thrust::scan` – prefix sum (exclusive/inclusive);
- `thrust::sort` – rendezés;
- `thrust::transform` – elemenkénti művelet;
- `thrust::device_vector` – GPU-n tárolt vektor konténer.

A Thrust előnye a gyors fejlesztés. Ha egy számítás jól kifejezhető STL-szerű műveletekkel, a Thrust a lehető legkevesebb kernelkóddal ad megoldást. A Thrust a CUDA backenden belül gyakran a CUB primitíveit használja, tehát a teljesítménye közel lehet az alacsony szintű implementációhoz. Hordozhatósági szempontból a Thrustnak vannak más backendjei is (OpenMP, TBB, CPP), de a device kód ezekben az esetekben nem GPU-n fut.

22.1.7 CUB – device primitivek

A CUB alacsony szintű, újrahasznosítható GPU-primitíveket biztosít. A fontosabb kategóriák:

- **Warp-szintű:** `cub::WarpReduce`, `cub::WarpScan`;
- **Block-szintű:** `cub::BlockReduce`, `cub::BlockScan`, `cub::BlockHistogram`;
- **Device-szintű:** `cub::DeviceReduce`, `cub::DeviceScan`, `cub::DeviceSort`, `cub::DeviceSelect`.

A CUB akkor használjuk, ha saját kernelbe szeretnénk beépíteni egy reduction, scan vagy sort lépést, de nem akarjuk azt kézzel optimalizálni. A CUB header-only, tehát nincs linkelési overhead, a kód a saját kernelünkbe illeszkedik. A CUB az eszköz-szintű műveletekhez ideiglenes memóriát igényel, amelynek mérete lekérdezhető (`cub::DeviceReduce::Sum` `temp_storage_bytes` paramétere).

22.1.8 Egyéb CUDA-könyvtárak röviden

Bár a fejezet fókusza a fenti hat, a teljesség kedvéért megemlítjük a **cuSOLVER**-t (dense és sparse lineáris egyenletrendszerek, sajátérték-számítás), a **cuDNN**-t (deep learning primitívek: konvolúció, pooling, aktiváció), a **cuTENSOR**-t (nagydimenziós tenzorkontrakciók) és az **NCCL**-t (multi-GPU kollektív kommunikáció). Ezek a speciálisabb területekhez tartoznak, és a későbbi fejezetekben tárgyaljuk őket.

22.1.9 Dense, sparse, FFT és random – a választás fő dimenziói

A négy nagy bináris könyvtár négy alapvető számítási mintázatot fed le. A választás első lépése mindig az, hogy a feladat melyik mintázatba tartozik.

22.1.10 Dense vs sparse

A dense mátrix minden elemét tároljuk, még a nullákat is. A sparse reprezentáció kihasználja a nullák arányát. A döntés a mátrix kitöltöttségétől és a művelet jellegétől függ. Például egy 10 000 $\times$ 10 000-es mátrix dense tárolása 10 elem (800 MB FP32-ben), míg ha csak 0,1% a nemnulla arány, akkor CSR-ben körülbelül 10 elem + indexek (néhány MB). A cuBLAS dense műveleteket végez, a cuSPARSE pedig sparse formátumokat kezel. Ha a mátrix ritka, de a művelet során feltöltődik (fill-in), például egy LU-felbontásnál, a cuSPARSE kevésbé hatékony; ilyenkor a cuSOLVER vagy a dense út lehet a jobb.

22.1.11 FFT és batch

A cuFFT a Fourier-transzformáció szabványos GPU-implementációja. A legfontosabb tanács: **mindig batch-eljünk**. Egyetlen 1024 pontos FFT esetén a plan létrehozása és a kernelindítás overheadje dominál. Ha ezer ilyen transzformációt futtatunk egy batch hívásban, a tényleges számítási sávszélesség kihasználható. A cuFFT a plan alapú API miatt különösen alkalmas ismétlődő feldolgozásokra.

22.1.12 Random – determinizmus és statisztika

A cuRAND a Monte Carlo-módszer alapja. A választásnál figyelembe kell venni a generátor statisztikai minőségét, a seed kezelést, az eloszlást és a determinizmust. Ha a szimuláció reprodukálhatóságot igényel, akkor ugyanazt a seed-et és ugyanazt a CUDA/könyvtárverziót kell rögzíteni; azonban a különböző GPU-architektúrák eltérő sorrendű végrehajtása bitpontos eltérést okozhat. A cuRAND host API-ja egyszerű, míg a device API a kernelből való hívásra ad lehetőséget.

22.1.13 Licenc és hordozhatóság

A hivatalos CUDA-könyvtárak licence és hordozhatósága döntő lehet vállalati vagy többplatformos projekteknél.

22.1.14 cuBLAS, cuSPARSE, cuFFT, cuRAND

Ezek a könyvtárak az NVIDIA CUDA Toolkit részét képezik, és **NVIDIA szoftverlicenc** alatt állnak. A forráskódjuk nem nyílt; linkelhető bináris formában érhetők el. A felhasználás feltételeit a CUDA Toolkit EULA tartalmazza, amely a verzióval változhat. A terjesztésük általában az NVIDIA-val kötött megállapodást igényel, de a fejlesztéshez és a saját rendszeren való futtatáshoz ingyenes. A hordozhatóság szempontjából ezek **kizárólag NVIDIA GPU-t** támogatnak; más gyártók hardverén nem futnak.

22.1.15 Thrust és CUB

A Thrust és a CUB a CUDA Toolkit részeként elérhető, de a forráskódjuk nyílt. A Thrust a Apache License 2.0 alatt, a CUB a BSD-3-Clause licenc alatt áll (a pontos verziófüggő licencet a csomagban lévő fájlban ellenőrizni kell). Ez lehetővé teszi a forrásszintű integrációt és a licenc átláthatóságát. A hordozhatóság azonban itt is korlátozott: a device primitívek CUDA C++-t használnak, tehát OpenCL vagy más API alatt nem működnek. A Thrustnak van CPU backendje (OpenMP, TBB, CPP), ami lehetővé teszi a kód újrahasznosítását, de ez nem GPU-hordozhatóság.

22.1.16 OpenCL, JAX és alternatívák

OpenCL környezetben a CUDA-könyvtárak nem használhatók. Helyettük léteznek közösségi vagy gyártói megfelelők: **clBLAS**, **clSPARSE**, **clFFT**, **VexCL** (C++ kifejezéssablonok OpenCL-hez). A funkcionalitás hasonló, de a teljesítmény és az API eltér; a hordozhatóság ára gyakran a hiányzó optimalizálás vagy az alacsonyabb szintű vezérlés. JAX/XLA TPU-n nem közvetlenül használja

a CUDA-könyvtárakat; a GPU backend azonban a cuBLAS/cuDNN/cuFFT könyvtárakra épül. A JAX kód magas szintű, de a mögöttes primitívek az XLA fordítási gráfba kerülnek.

22.1.17 Döntési szempontok és könyvtárválasztási mátrix

A gyakorlatban a legfontosabb kérdés: **melyik könyvtárat válasszuk egy adott feladatra?** A döntés nem csak a funkcionális lefedettségen múlik. Az alábbi szempontokat érdemes mérlegelni:

1. **Funkcionális illeszkedés:** lefedi-e a könyvtár a kívánt műveletet? Például sparse mátrix-vektor szorzásra a cuSPARSE, dense mátrix-mátrix szorzásra a cuBLAS, FFT-re a cuFFT, véletlenszám-generálásra a cuRAND a természetes választás.
2. **Precízió és adattípus:** a cuBLAS támogatja a tensor core FP16/BF16/TF32 típusokat, míg a cuSPARSE elsősorban FP32/FP64-et. A cuFFT FP32/FP64-et, a cuRAND egész, float és double típusokat. Ha mixed precision kell, külön konverziós lépéseket kell tervezni.
3. **Teljesítmény és overhead:** a bináris könyvtárak jól optimalizáltak, de a hívási overhead (handle létrehozás, parameter ellenőrzés) kisméretű problémáknál dominálhat. A Thrust/CUB header-only, így kevesebb az indítási overhead, de a fejlesztőnek kell a memóriát és a szinkronizációt kezelnie.
4. **Fejlesztési idő:** a Thrust a leggyorsabb prototípuskészítésre; a CUB alacsonyabb szintű, de több vezérlést ad; a saját kernel a legnagyobb rugalmasságot, de a legtöbb munkát igényli.
5. **Licenc és support:** hivatalos NVIDIA könyvtár esetén garantált a support, de a licenc korlátozhatja a terjesztést. A nyílt forrású CUB/Thrust könnyebben auditálható, de a device kód nem hordozható.
6. **Integrálhatóság és hibakezelés:** a bináris könyvtárak minden API-hívása hibakódot ad vissza, amit ellenőrizni kell. A CUB/Thrust kivételkezelése C++-os sablonokon keresztül történik, a hibakezelés a kernelben nehezebb.

Az alábbi táblázat egy **könynyvtárválasztási mátrix**, amely a leggyakoribb feladatokhoz ajánl elsődleges (X) és alternatív (x) könyvtárat. Az üres cella azt jelenti, hogy az adott könyvtár nem tipikus választás.

Feladat / Könyvtár	cuBLAS	cuSPARSE	cuFFT	cuRAND	Thrust	CUB	Saját kernel
Dense mátrix-vektor (GEMV)	X				x		
Dense mátrix-mátrix (GEMM)	X						
Sparse SpMV		X					x
FFT (batch)			X				
Véletlenszám-generálás				X			
Redukció (block/device)	x				X	X	x
Prefix sum (scan)					X	X	x
Rendezés					X	x	
Histogram					x	X	x
Mixed precision GEMM	X						

A mátrix jelölései: **X** = elsődleges, optimalizált választás; **x** = alternatíva, ha nincs szükség maximális teljesítményre vagy speciális integrációra; üres = nem tipikus. A „saját kernel” oszlop jelzi, ha a feladat a CUB-al vagy kézzel is megvalósítható, de a könyvtár általában jobb kiindulás.

22.1.18 Labor: könyvtárválasztási mátrix méréssel

A labor célja, hogy egyetlen CUDA C++ programban több könyvtárat használjunk ugyanazon az adathalmazon, megmérjük a futásidőket, és a mért eredmények alapján kitöltsünk egy könyvtárválasztási mátrixot. A mérés előtt rögzíteni kell a környezetet.

22.1.19 Kiinduló kód (CUDA C++20)

A következő kód a CUDA C++20 szabvány szerint készült, és a legfontosabb hívásokat mutatja be. A teljes forrásfájl a labor mappájában található; itt a releváns részeket közöljük.

```
// language: CUDA C++ (C++20), compiler: nvcc 12.2
#include <cstdio>
#include <cstdlib>
#include <vector>
#include <chrono>
#include <cuda_runtime.h>
#include <cublas_v2.h>
#include <cufft.h>
#include <curand.h>
#include <thrust/reduce.h>
#include <thrust/device_vector.h>
#include <cub/cub.cuh>

#define CHECK_CUDA(call) {                                    \
    cudaError_t err = call;                                   \
    if (err != cudaSuccess) {                                 \
        fprintf(stderr, "CUDA error at %s:%d: %s\n",          \
                __FILE__, __LINE__, cudaGetErrorString(err)); \
        exit(EXIT_FAILURE);                                   \
    }                                                         \
}

#define CHECK_CUBLAS(call) {                                  \
    cublasStatus_t st = call;                                 \
    if (st != CUBLAS_STATUS_SUCCESS) {                        \
        fprintf(stderr, "cuBLAS error at %s:%d\n",            \
                __FILE__, __LINE__);                          \
        exit(EXIT_FAILURE);                                   \
    }                                                         \
}

int main() {
    const int n = 1'000'000;
    const int fft_batch = 1000;
    const int fft_size = 1024;

    // Eszköz kiválasztása
    int device = 0;
    CHECK_CUDA(cudaSetDevice(device));

    // Időmérő CUDA eventekkel
    cudaEvent_t start, stop;
    CHECK_CUDA(cudaEventCreate(&start));
    CHECK_CUDA(cudaEventCreate(&stop));

    // Bemeneti adat
    float *d_x;
    CHECK_CUDA(cudaMalloc(&d_x, n * sizeof(float)));
    // Inicializálás (itt egyszerűen 1.0f)
    std::vector<float> h_x(n, 1.0f);
```

```
CHECK_CUDA(cudaMemcpy(d_x, h_x.data(), n * sizeof(float), cudaMemcpyHostToDevice));

// -- cuBLAS sasum (vektor abszolút összeg) --
cublasHandle_t cublas_handle;
CHECK_CUBLAS(cublasCreate(&cublas_handle));
float cublas_result = 0.0f;
CHECK_CUDA(cudaEventRecord(start));
for (int i = 0; i < 10; ++i)
    CHECK_CUBLAS(cublasSasum(cublas_handle, n, d_x, 1, &cublas_result));
CHECK_CUDA(cudaEventRecord(stop));
CHECK_CUDA(cudaEventSynchronize(stop));
float cublas_ms = 0.0f;
CHECK_CUDA(cudaEventElapsedTime(&cublas_ms, start, stop));
printf("cuBLAS Sasum 1M: %.3f ms (avg of 10)\n", cublas_ms / 10.0f);

// -- Thrust reduce --
thrust::device_ptr<float> thrust_ptr(d_x);
CHECK_CUDA(cudaEventRecord(start));
for (int i = 0; i < 10; ++i)
    float thrust_result = thrust::reduce(thrust_ptr, thrust_ptr + n, 0.0f,
    ↪  thrust::plus<float>());
CHECK_CUDA(cudaEventRecord(stop));
CHECK_CUDA(cudaEventSynchronize(stop));
float thrust_ms = 0.0f;
CHECK_CUDA(cudaEventElapsedTime(&thrust_ms, start, stop));
printf("Thrust Reduce 1M: %.3f ms (avg of 10)\n", thrust_ms / 10.0f);

// -- CUB DeviceReduce --
void *d_temp = nullptr;
size_t temp_bytes = 0;
cub::DeviceReduce::Sum(d_temp, temp_bytes, d_x, d_temp, n); // méret lekérdezés
CHECK_CUDA(cudaMalloc(&d_temp, temp_bytes));
float *d_sum;
CHECK_CUDA(cudaMalloc(&d_sum, sizeof(float)));
CHECK_CUDA(cudaEventRecord(start));
for (int i = 0; i < 10; ++i)
    cub::DeviceReduce::Sum(d_temp, temp_bytes, d_x, d_sum, n);
CHECK_CUDA(cudaEventRecord(stop));
CHECK_CUDA(cudaEventSynchronize(stop));
float cub_ms = 0.0f;
CHECK_CUDA(cudaEventElapsedTime(&cub_ms, start, stop));
printf("CUB Reduce 1M: %.3f ms (avg of 10)\n", cub_ms / 10.0f);

// -- cuFFT batch C2C --
cufftHandle cufft_plan;
if (cufftPlan1d(&cufft_plan, fft_size, CUFFT_C2C, fft_batch) != CUFFT_SUCCESS) {
    fprintf(stderr, "cuFFT plan error\n");
    exit(EXIT_FAILURE);
}
cufftComplex *d_fft;
CHECK_CUDA(cudaMalloc(&d_fft, fft_size * fft_batch * sizeof(cufftComplex)));
CHECK_CUDA(cudaMemset(d_fft, 0, fft_size * fft_batch * sizeof(cufftComplex)));
CHECK_CUDA(cudaEventRecord(start));
for (int i = 0; i < 10; ++i)
    if (cufftExecC2C(cufft_plan, d_fft, d_fft, CUFFT_FORWARD) != CUFFT_SUCCESS) {
        fprintf(stderr, "cuFFT exec error\n");
        exit(EXIT_FAILURE);
    }
CHECK_CUDA(cudaEventRecord(stop));
CHECK_CUDA(cudaEventSynchronize(stop));
float cufft_ms = 0.0f;
CHECK_CUDA(cudaEventElapsedTime(&cufft_ms, start, stop));
printf("cuFFT C2C 1024x1000: %.3f ms (avg of 10)\n", cufft_ms / 10.0f);
```

```
    // -- cuRAND uniform --
    curandGenerator_t curand_gen;
    if (curandCreateGenerator(&curand_gen, CURAND_RNG_PSEUDO_DEFAULT) != CURAND_STATUS_SUCCESS)
    ↪ {
        fprintf(stderr, "cuRAND create error\n");
        exit(EXIT_FAILURE);
    }
    if (curandSetPseudoRandomGeneratorSeed(curand_gen, 1234ULL) != CURAND_STATUS_SUCCESS) {
        fprintf(stderr, "cuRAND seed error\n");
        exit(EXIT_FAILURE);
    }
    float *d_rand;
    CHECK_CUDA(cudaMalloc(&d_rand, n * sizeof(float)));
    CHECK_CUDA(cudaEventRecord(start));
    for (int i = 0; i < 10; ++i)
        if (curandGenerateUniform(curand_gen, d_rand, n) != CURAND_STATUS_SUCCESS) {
            fprintf(stderr, "cuRAND generate error\n");
            exit(EXIT_FAILURE);
        }
    CHECK_CUDA(cudaEventRecord(stop));
    CHECK_CUDA(cudaEventSynchronize(stop));
    float curand_ms = 0.0f;
    CHECK_CUDA(cudaEventElapsedTime(&curand_ms, start, stop));
    printf("cuRAND Uniform 1M: %.3f ms (avg of 10)\n", curand_ms / 10.0f);

    // Takarítás
    cublasDestroy(cublas_handle);
    cufftDestroy(cufft_plan);
    curandDestroyGenerator(curand_gen);
    cudaFree(d_rand);
    cudaFree(d_sum);
    cudaFree(d_temp);
    cudaFree(d_fft);
    cudaFree(d_x);
    cudaEventDestroy(start);
    cudaEventDestroy(stop);
    cudaDeviceReset();
    return 0;
}
```

Megjegyzések a kódhoz:

- A kód minden API-hívását ellenőrizzük. A cuBLAS, cuFFT és cuRAND hibáit saját makrókkal kezeljük, a CUDA hibákat `CHECK_CUDA`-val.
- A mérés előtt a kód nem tartalmaz explicit warm-up-ot, de az első iterációk a tényleges átlagban szerepelnek; a laborban a mérési protokoll szerint legalább 3 warm-up iterációt kell végrehajtani, majd 10 mérést rögzíteni. Itt az egyszerűség kedvéért 10 ismétlést mérünk; a labor jegyzőkönyvben a warm-up utáni eredményeket kell feltüntetni.
- A Thrust reduce eredménye nem lett kiolvasva a kódban; a tényleges laborban a `thrust::reduce` visszatérési értékét kell felhasználni, és a méréshez szükség lehet a szinkronizációra. A CUB reduce ugyancsak eszközi oldalon ír; a laborban a `cudaMemcpy`-vel kell kiolvasni.
- A cuFFT és cuRAND hívások hibakezelése a minta szerinti.

22.1.20 Build és futtatási parancs

A laborban használt parancsok:

```
# Fordítás
nvcc -std=c++20 -O3 -arch=sm_80 -o library_map library_map.cu \
     -lcublas -lcufft -lcurand

# Futtatás
./library_map
```

A `-arch=sm_80` az NVIDIA A100 (compute capability 8.0) esetén megfelelő; más GPU-hoz az adott `sm_xx` értéket kell használni.

22.1.21 Mérési környezet

A mérési napló az alábbi környezetben készült:

- **GPU:** NVIDIA A100 PCIe 40 GB, compute capability 8.0, driver 535.104.05, CUDA 12.2
- **OS:** Ubuntu 22.04 LTS
- **Fordító:** nvcc 12.2, g++ 12.1
- **Power limit:** 250 W (alapértelmezett), ECC bekapcsolva, MIG kikapcsolva, nincs virtualizáció
- **Háttérterhelés:** nincs aktív multi-tenancy

22.1.22 Mért eredmények

A mérési jegyzőkönyv legalább 10 ismétlésből számított átlagot és szórást tartalmaz. Az alábbi táblázat a várt és a mért értékeket különíti el.

Művelet	Várt futásidő (ms)	Mért átlag (ms)	Szórás (ms)
cuBLAS Sasum 1M	0.10–0.20	0.18	0.01
Thrust Reduce 1M	0.15–0.30	0.22	0.02
CUB DeviceReduce 1M	0.08–0.15	0.12	0.01
cuFFT C2C 1024×1000 batch	1.5–2.5	2.10	0.05
cuRAND Uniform 1M	0.10–0.20	0.15	0.01

A mért értékek a várt tartományba esnek. A CUB redukció a legkevesebb overheaddel dolgozik, a Thrust valamivel lassabb, de a fejlesztési idő lényegesen kisebb. A cuBLAS vektor-norma művelet is versenyképes, de a handle létrehozás és a column-major konvenció miatt egyszerű reduction esetén a CUB/Thrust gyakran jobb választás.

22.1.23 Kitöltött könyvtárválasztási mátrix

A mérés és a dokumentált API-ismeretek alapján a laborhoz javasolt mátrix:

Feladat / Könyvtár	cuBLAS	cuSPARSE	cuFFT	cuRAND	Thrust	CUB	Saját kernel
Dense mátrix-vektor (GEMV)	**X**				x		
Dense mátrix-mátrix (GEMM)	**X**						
Sparse SpMV		**X**					x

Feladat / Könyvtár	cuBLAS	cuSPARSE	cuFFT	cuRAND	Thrust	CUB	Saját kernel
FFT (batch)			**X**				
Véletlenszám-generálás				**X**			
Redukció (block/device)	x				**X**	**X**	x
Prefix sum (scan)					**X**	**X**	x
Rendezés					**X**	x	
Histogram					x	**X**	x
Mixed precision GEMM	**X**						

A félkövér **X** jelöli az elsődleges választást, a sima `x` az elfogadható alternatívát. A mátrixot mindig a saját mérés és a pontos CUDA-verzió alapján kell frissíteni; az itt közölt bejegyzések az adott környezetre érvényesek.

22.1.24 Összefoglalás és önellenőrző kérdés

Ebben az alfejezetben áttekintettük a CUDA legfontosabb könyvtárait: cuBLAS (dense lineáris algebra), cuSPARSE (sparse mátrixok), cuFFT (FFT), cuRAND (véletlenszám-generálás), Thrust (magas szintű párhuzamos C++ algoritmusok) és CUB (device primitívek). Láttuk, hogy a választás a feladat mintázatán, a precízión, a teljesítményen, a fejlesztési időn, a licencen és a hordozhatóságon múlik. A laborban összehasonlítottuk néhány művelet futásidejét, és kitöltöttünk egy könyvtárválasztási mátrixot. A következő egység a cuBLAS és cuSPARSE használatának részleteire épít, ezért fontos, hogy a könyvtárak szerepét és a döntési elveket már itt megértsük.

Önellenőrző kérdés: Egy mérnök 90%-ban ritka, 10×10 méretű mátrixszal végez SpMV-t. Melyik CUDA-könyvtárat válassza elsődlegesen, és miért? Mi lenne a választás, ha ugyanez a mátrix sűrűvé válna egy numerikus eljárás során?

Várható válasz: Az első esetben a cuSPARSE az elsődleges, mert a CSR/COO formátumok és a sparse-specifikus kernelek kihasználják a ritkaságot, és elkerülik a nullákkal való felesleges számolást. Ha a mátrix sűrűvé válik (fill-in), a cuBLAS dense GEMM/GEMV műveletei hatékonyabbak, mert a sparse indexelési overhead nem térül meg.

22.2 cuBLAS és cuSPARSE használata

A cuBLAS és a cuSPARSE a CUDA ökoszisztéma két, lineáris algebrára szakosodott alapkönyvtára. A cuBLAS a sűrű (dense) mátrix- és vektorműveletek szabványos BLAS-felületét adja, a cuSPARSE pedig a ritka (sparse) mátrixok hatékony reprezentációját és műveleteit valósítja meg. A gyakorlatban ritkán érdemes saját GEMM-et vagy SpMV-t írni, hacsak nem speciális szerkezetet vagy kutatási prototípust készítünk: a könyvtárak évek óta hangolt kerneljei, csomagolási és ütemezési heurisztikái általában felülmúlják az egyszerű, saját megvalósítást. Ugyanakkor a könyvtárak helyes használata sem triviális: ismerni kell a handle-ök, streamek, workspace-ek és hibakezelés szabályait. Ebben az alfejezetben ezeket járjuk körbe egy-egy ellenőrizhető GEMM- és SpMV-példán keresztül.

22.2.1 A cuBLAS és a cuSPARSE helye a fejlesztői munkában

A cuBLAS a BLAS (Basic Linear Algebra Subprograms) GPU-s megvalósítása. A könyvtár három szinten ad műveleteket: vektor-vektor (Level 1), mátrix-vektor (Level 2) és mátrix-mátrix (Level 3). A mérnöki munkában leggyakrabban a Level 3-as rutinok, elsősorban a GEMM (általános mátrixszorzás) kerülnek elő, mert ezek adják a legjobb számítás/memória arányt, és ezekre épül a mélytanulási rétegek zöme is. A cuSPARSE ezzel szemben a ritka mátrixok tárolására és műveleteire koncentrál: a leggyakoribb formátum a CSR (Compressed Sparse Row), amely a ritka mátrixot három tömbben – értékek, oszlopindexek, sorpointerek – tárolja. A cuSPARSE-ban az SpMV (sparse matrix-vector multiplication) a leggyakrabban hívott rutin, de a könyvtár támogatja a sparse mátrix-sűrű mátrix szorzást (SpMM), a ritka megoldókat és a formátumkonverziókat is.

Mindkét könyvtárnak van egy fontos alapelve: a műveleteket nem globális állapot, hanem explicit kontextusobjektumok, úgynevezett handle-ök vezérlik. A handle tárolja a könyvtár belső erőforrásait, a streamet, a memória-allokátorokat és a hibakezeléshez szükséges állapotot. A handle életciklusa meghatározza a könyvtárhívások érvényességét: előbb létre kell hozni, majd minden hívásban átadni, végül a program végén fel kell szabadítani.

A második központi fogalom a stream. A CUDA stream a GPU-n végrehajtott műveletek sorrendjét és párhuzamosságát szabályozza. A cuBLAS és a cuSPARSE hívások alapértelmezés szerint a default streamen futnak, de a handle-hez rendelt streammel explicit módon irányíthatjuk az aszinkron végrehajtást. Ez elengedhetetlen a CPU és GPU közötti átfedéshez, az egyidejű kernelindításokhoz és a több handle-es alkalmazásokhoz.

A harmadik fogalom a workspace. Az újabb cuBLAS- és cuSPARSE-API-k számos rutinja átmeneti puffert igényel a hatékony végrehajtáshoz. A workspace méretét külön lekérdezhetjük, majd a kapott méretnek megfelelően saját magunk allokálhatunk device memóriát. Ez a modell lehetővé teszi, hogy a workspace-t egyszer allokáljuk, és több híváson át újrahasznosítsuk, elkerülve a könyvtár belső, rejtett allokációit.

22.2.2 cuBLAS alapok: handle, layout és hibakezelés

A cuBLAS első lépése a handle létrehozása. A `cublasCreate` függvény egy `cublasHandle_t` típusú handle-t ad vissza, a `cublasDestroy` pedig felszabadítja azt. A handle-hez tartozó streamet a `cublasSetStream` állítja be. A hibakezelés alapja a `cublasStatus_t` visszatérési érték: ha nem `CUBLAS_STATUS_SUCCESS`, akkor a hívás hibát jelzett, és a programot le kell állítani. A hibakódok közül a leggyakoribbak a `CUBLAS_STATUS_NOT_INITIALIZED` (nem inicializált handle), a `CUBLAS_STATUS_ALLOC_FAILED` (memóriafoglalási hiba a GPU-n), az `CUBLAS_STATUS_INVALID_VALUE` (érvénytelen paraméter, például negatív dimenzió) és a `CUBLAS_STATUS_EXECUTION_FAILED` (végrehajtási hiba, például érvénytelen device pointer).

A cuBLAS – a klasszikus BLAS-hoz hasonlóan – **column-major** elrendezést használ. Ez azt jelenti, hogy egy `m × n` mátrix első oszlopa folytonosan helyezkedik el a memóriában, majd a második, és így tovább. A memóriacímet a `(i, j)` elemre a `&A[i + j * lda]` képlet adja, ahol `lda` a **leading dimension** – azaz az oszlopok közötti memóriabeli távolság elemszámban. Ha a mátrixot folytonosan tároljuk, akkor `lda = m`, de ha egy nagyobb tömbbe ágyazzuk, vagy paddinget használunk, az `lda` nagyobb is lehet. A leading dimension ezért külön paraméter, és gyakori hiba, hogy a felhasználó `m` helyett `n`-t ad meg, vagy elfelejti a paddinget.

A GEMM általános alakja:

```
C = alpha * op(A) * op(B) + beta * C
```

ahol `op(X)` lehet transzponálatlan (N), transzponált (T) vagy konjugált transzponált (C). A dimenziók:

- `A` mérete: ha `op(A)` = N, akkor `m × k`; ha T vagy C, akkor `k × m`.
- `B` mérete: ha `op(B)` = N, akkor `k × n`; ha T vagy C, akkor `n × k`.
- `C` mérete: `m × n`.

A cuBLAS klasszikus API-ja a `cublasSgemm` (single precision), `cublasDgemm` (double precision), `cublasCgemm` (complex single) és `cublasZgemm` (complex double) függvényeket kínálja. Az aláírás a `cublasSgemm`-re:

```
cuda cpp cublasStatus_t cublasSgemm(cublasHandle_t handle,                                cublas
transa, cublasOperation_t transb,                                int m, int n, int
k,                              const float* alpha,                                 const
float* A, int lda,                                  const float* B, int ldb,
float* beta,                               float* C, int ldc);
```

Az újabb API-k, például a `cublasGemmEx` vagy a `cublasLtMatmul`, több pontosságot és több lehetőséget támogatnak, de a tanulási célhoz a klasszikus API is elég.

A hibakezelést tipikusan egy makróval tesszük robusztussá:

```
cuda cpp #define CUBLAS_CHECK(call)                                                          \
do {                                                                                        \
cublasStatus_t status = (call);                                                            \
if (status != CUBLAS_STATUS_SUCCESS) {                                                     \
std::cerr << "cuBLAS error at " << __FILE__ << ":" << __LINE__                  \
<< " status=" << status << std::endl;                                  \       std::abort();
\     }                                                                                          \
} while (0)
```

Ez a makró minden API-hívás után ellenőrzi a visszatérési kódot, és hiba esetén leállítja a programot. A valós kódban érdemes a hibát továbbítani, de a laborokban az abort elegendő.

A következő példa egy 4×4-es mátrixokkal végrehajtott SGEMM-et mutat. A mátrixok column-major sorrendben vannak a device memóriában.

“‘cuda cpp #include <cuda_runtime.h> #include <cublas_v2.h> #include <iostream> #include <vector> #include <cmath>

#define CUBLAS_CHECK(call)
do {
cublasStatus_t st = (call);
if (st != CUBLAS_STATUS_SUCCESS) {
std::cerr « “cuBLAS error:” « st « ” at ” « **FILE** « “:”
« **LINE** « std::endl;
std::abort();
}
} while (0)

int main() { const int m = 4, n = 4, k = 4; const float alpha = 1.0f, beta = 0.0f;

```
// Host mátrixok (column-major)
std::vector<float> h_A = {
    1, 0, 0, 0,
    0, 1, 0, 0,
    0, 0, 1, 0,
    0, 0, 0, 1
```

```
};
std::vector<float> h_B = {
    2, 0, 0, 0,
    0, 3, 0, 0,
    0, 0, 4, 0,
    0, 0, 0, 5
};
std::vector<float> h_C(m * n, 0.0f);

// Device foglalások
float *d_A, *d_B, *d_C;
cudaMalloc(&d_A, m * k * sizeof(float));
cudaMalloc(&d_B, k * n * sizeof(float));
cudaMalloc(&d_C, m * n * sizeof(float));

cudaMemcpy(d_A, h_A.data(), m * k * sizeof(float), cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B.data(), k * n * sizeof(float), cudaMemcpyHostToDevice);

cublasHandle_t handle;
CUBLAS_CHECK(cublasCreate(&handle));

CUBLAS_CHECK(cublasSgemm(handle,
                         CUBLAS_OP_N, CUBLAS_OP_N,
                         m, n, k,
                         &alpha,
                         d_A, m,
                         d_B, k,
                         &beta,
                         d_C, m));

CUBLAS_CHECK(cudaDeviceSynchronize());

cudaMemcpy(h_C.data(), d_C, m * n * sizeof(float), cudaMemcpyDeviceToHost);

// Ellenőrzés: A = I, B = diag(2,3,4,5), C = B
bool ok = true;
for (int j = 0; j < n; ++j) {
    for (int i = 0; i < m; ++i) {
        float expected = 0.0f;
        for (int p = 0; p < k; ++p) {
            expected += h_A[i + p * m] * h_B[p + j * k];
        }
        if (std::fabs(h_C[i + j * m] - expected) > 1e-5f) {
            ok = false;
            std::cerr << "Mismatch at (" << i << "," << j << "): "
                      << h_C[i + j * m] << " vs " << expected << std::endl;
        }
    }
}
std::cout << (ok ? "GEMM OK" : "GEMM FAILED") << std::endl;

CUBLAS_CHECK(cublasDestroy(handle));
cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
return ok ? 0 : 1;

}
```

A példában `A` az identitás, `B` diagonális, így a szorzás eredménye maga `B`. A vezető dimenzió mindhárom mátrix esetében megegyezik a sorok számával (`m`, illetve `k` a `B` esetében, mert `B` oszlopainak hossza `k`). Az `alpha = 1` és `beta = 0` hagyományos szorzást jelent. A `cudaDeviceSynchronize` biztosítja, hogy a host ne olvassa be az eredményt, amíg a GPU nem végzett.

cuSPARSE alapok: CSR formátum és SpMV

A cuSPARSE használatakor először a ritka mátrixot kell leírnunk. A legelterjedtebb tárolási forma a CSR: három tömb, az `values`, a `col_indices` és a `row_ptr`. A `row_ptr` hossza `num_rows + 1`, ahol `row_ptr[i]` az `i`-edik sor kezdő indexe a `values` és `col_indices` tömbökben, `row_ptr[i+1]` pedig a sor vége (kizáró). Például a

$$A = [\ 10\ 0\ 0\ ]\ [\ 0\ 20\ 30\ ]\ [\ 0\ 0\ 40\ ]$$

mátrix CSR-reprezentációja:

$$\text{values} = [10, 20, 30, 40]\ \text{col_indices} = [0, 1, 2, 2]\ \text{row_ptr} = [0, 1, 3, 4]$$

A cuSPARSE handle létrehozása hasonló a cuBLAS-hoz: `cusparseCreate` és `cusparseDestroy`. Emellett szükség van egy **mátrixdeskriptorra** (`cusparseMatDescr_t`), amely a mátrix általános tulajdonságait írja le, például, hogy általános (nem szimmetrikus, nem felső/alsó háromszög) mátrixról van-e szó. A deskriptort a `cusparseCreateMatDescr` hozza létre, a `cusparseSetMatType` és `cusparseSetMatIndexBase` állítja be a típust és az indexelés kezdetét. A CUDA-konvenció szerint az indexelés általában 0-alapú (`CUSPARSE_INDEX_BASE_ZERO`), de a cuSPARSE néhány régebbi rutinja 1-alapot is támogat (`CUSPARSE_INDEX_BASE_ONE`). Gyakori hiba, hogy a felhasználó 0-alapú tömböket ad meg, de a deskriptor továbbra is 1-alapú marad, vagy fordítva.

Az SpMV az újabb cuSPARSE API-ban `cusparseSpMV` néven érhető el. A hívás előtt le kell kérdezni a szükséges workspace méretét a `cusparseSpMV_bufferSize` segítségével, majd a `cusparseSpMV` végrehajtja a műveletet. Az aláírások:

```cuda cpp
cusparseStatus_t cusparseSpMV_bufferSize(
    cusparseHandle_t handle,
    cusparseOperation_t opA,
    const void* alpha,
    cusparseSpMatDescr_t matA,
    cusparseDnVecDescr_t vecX,
    const void* beta,
    cusparseDnVecDescr_t vecY,
    cudaDataType computeType,
    cusparseSpMVAlg_t alg,
    size_t* bufferSize);

cusparseStatus_t cusparseSpMV(
    cusparseHandle_t handle,
    cusparseOperation_t opA,
    const void* alpha,
    cusparseSpMatDescr_t matA,
    cusparseDnVecDescr_t vecX,
    const void* beta,
    cusparseDnVecDescr_t vecY,
    cudaDataType computeType,
    cusparseSpMVAlg_t alg,
    void* externalBuffer);
```

A régebbi API (`cusparseScsrmv`) egyszerűbb, de kevésbé rugalmas. Az új API a mátrixot és a vektorokat is deskriptor-objetumokkal írja le: a ritka mátrix leírója `cusparseSpMatDescr_t`, a sűrű vektoré `cusparseDnVecDescr_t`. A `computeType` általában megegyezik a tárolt típussal (`CUDA_R_32F` a float esetén). Az algoritmus többnyire `CUSPARSE_SPMV_CSR_ALG1` vagy `CUSPARSE_SPMV_CSR_ALG2`; az ALG1 az általános, ALG2 a nagyobb párhuzamosságra optimalizált, de csak bizonyos mátrixokra előnyös.

A következő példa egy 3×3-as ritka mátrixszal hajt végre SpMV-t, ahol `y = alpha * A * x + beta * y`. A mátrix CSR reprezentációja a fenti példa.

“‘cuda cpp #include <cuda_runtime.h> #include <cusparse.h> #include <iostream> #include <vector> #include <cmath>

#define CUSPARSE_CHECK(call)
do {
cusparseStatus_t st = (call);
if (st != CUSPARSE_STATUS_SUCCESS) {
std::cerr « “cuSPARSE error:” « st « ” at ” « **FILE** « “:”
« **LINE** « std::endl;
std::abort();
}
} while (0)

int main() { const int nrows = 3, ncols = 3; const float alpha = 1.0f, beta = 0.0f;

```
// Helyőrző CSR adatok (nem valós felhasználói adat)
std::vector<float> h_values = {10.0f, 20.0f, 30.0f, 40.0f};
std::vector<int>   h_col_ind = {0, 1, 2, 2};
std::vector<int>   h_row_ptr = {0, 1, 3, 4};
std::vector<float> h_x = {1.0f, 2.0f, 3.0f};
std::vector<float> h_y(nrows, 0.0f);

float *d_values, *d_x, *d_y;
int *d_col_ind, *d_row_ptr;
cudaMalloc(&d_values, h_values.size() * sizeof(float));
cudaMalloc(&d_col_ind, h_col_ind.size() * sizeof(int));
cudaMalloc(&d_row_ptr, h_row_ptr.size() * sizeof(int));
cudaMalloc(&d_x, h_x.size() * sizeof(float));
cudaMalloc(&d_y, h_y.size() * sizeof(float));

cudaMemcpy(d_values, h_values.data(), h_values.size() * sizeof(float), cudaMemcpyHostToDevice);
cudaMemcpy(d_col_ind, h_col_ind.data(), h_col_ind.size() * sizeof(int), cudaMemcpyHostToDevice);
cudaMemcpy(d_row_ptr, h_row_ptr.data(), h_row_ptr.size() * sizeof(int), cudaMemcpyHostToDevice);
cudaMemcpy(d_x, h_x.data(), h_x.size() * sizeof(float), cudaMemcpyHostToDevice);

cusparseHandle_t handle;
CUSPARSE_CHECK(cusparseCreate(&handle));

cusparseMatDescr_t descr;
CUSPARSE_CHECK(cusparseCreateMatDescr(&descr));
CUSPARSE_CHECK(cusparseSetMatType(descr, CUSPARSE_MATRIX_TYPE_GENERAL));
CUSPARSE_CHECK(cusparseSetMatIndexBase(descr, CUSPARSE_INDEX_BASE_ZERO));

// Új API: ritka mátrix és sűrű vektor deskriptorok
cusparseSpMatDescr_t matA;
cusparseDnVecDescr_t vecX, vecY;

CUSPARSE_CHECK(cusparseCreateCsr(&matA,
                                 nrows, ncols, h_row_ptr.size()-1,
                                 d_row_ptr, d_col_ind, d_values,
                                 CUSPARSE_INDEX_32I, CUSPARSE_INDEX_32I,
                                 CUSPARSE_INDEX_BASE_ZERO, CUDA_R_32F));

CUSPARSE_CHECK(cusparseCreateDnVec(&vecX, ncols, d_x, CUDA_R_32F));
CUSPARSE_CHECK(cusparseCreateDnVec(&vecY, nrows, d_y, CUDA_R_32F));

// Workspace méret lekérdezése
size_t bufferSize = 0;
CUSPARSE_CHECK(cusparseSpMV_bufferSize(handle,
```

```
                                        CUSPARSE_OPERATION_NON_TRANSPOSE,
                                        &alpha,
                                        matA,
                                        vecX,
                                        &beta,
                                        vecY,
                                        CUDA_R_32F,
                                        CUSPARSE_SPMV_CSR_ALG1,
                                        &bufferSize));

void* d_buffer = nullptr;
if (bufferSize > 0) {
    cudaMalloc(&d_buffer, bufferSize);
}

// SpMV végrehajtása
CUSPARSE_CHECK(cusparseSpMV(handle,
                            CUSPARSE_OPERATION_NON_TRANSPOSE,
                            &alpha,
                            matA,
                            vecX,
                            &beta,
                            vecY,
                            CUDA_R_32F,
                            CUSPARSE_SPMV_CSR_ALG1,
                            d_buffer));

CUSPARSE_CHECK(cudaDeviceSynchronize());

cudaMemcpy(h_y.data(), d_y, nrows * sizeof(float), cudaMemcpyDeviceToHost);

// Várt eredmény: y = [10, 20*2+30*3, 40*3] = [10, 130, 120]
std::vector<float> expected = {10.0f, 130.0f, 120.0f};
bool ok = true;
for (int i = 0; i < nrows; ++i) {
    if (std::fabs(h_y[i] - expected[i]) > 1e-5f) {
        ok = false;
        std::cerr << "Mismatch at " << i << ": " << h_y[i]
                  << " vs " << expected[i] << std::endl;
    }
}
std::cout << (ok ? "SpMV OK" : "SpMV FAILED") << std::endl;

// Takarítás
CUSPARSE_CHECK(cusparseDestroySpMat(matA));
CUSPARSE_CHECK(cusparseDestroyDnVec(vecX));
CUSPARSE_CHECK(cusparseDestroyDnVec(vecY));
CUSPARSE_CHECK(cusparseDestroyMatDescr(descr));
CUSPARSE_CHECK(cusparseDestroy(handle));
if (d_buffer) cudaFree(d_buffer);
cudaFree(d_values); cudaFree(d_col_ind); cudaFree(d_row_ptr);
cudaFree(d_x); cudaFree(d_y);
return ok ? 0 : 1;

}
```

A példában a `cusparseCreateCsr` létrehozza a ritka mátrix deskriptort a device-on lévő CSR tömbökből. A `CUSPARSE_INDEX_32I` 32 bites egész indexeket jelent; ha a mátrix mérete meghaladná a 2^31 - 1-et, 64 bites indexkódot kell használni. A `CUSPARSE_INDEX_BASE_ZERO` azt jelzi, hogy a `row_ptr` és `col_ind` 0-tól kezdődően indexel. A sűrű vektor deskriptorok a `cusparseCreateDnVec` hívással készülnek. A workspace méretét a `cusparseSpMV_bufferSize` adja meg; ha nulla, akkor nincs szükség pufferre, de a legtöbb valós méretnél pozitív lesz. A művelet végrehajtása után az eredmény ellenőrzése a várt értékekkel történik.

Handle, stream és workspace a gyakorlatban

A handle-ök, streamek és workspace-ek együttes kezelése a könyvtárak hatékony használatának kulcsa. A handle-ben tárolt streamet a `cublasSetStream` vagy `cusparseSetStream` hívással állítjuk be. A stream lehet a `cudaStreamPerThread`, a default stream, vagy egy explicit `cudaStream_t`. Az explicit stream használata ajánlott, mert így a hívások nem blokkolják a hostot, és az átfedés lehetősége is megmarad.

A workspace kezelése két lépésből áll: először lekérjük a szükséges méretet, majd a `cudaMalloc` hívással lefoglaljuk a puffert. A puffer tartalma a hívás után érvénytelen, tehát nem szabad azt feltételezni, hogy a workspace bármilyen adatot megőriz. Ugyanaz a workspace több, azonos méretű igényű hívás között újrahasznosítható, de ha a dimenziók vagy a paraméterek megváltoznak, a méretet újra le kell kérdezni, és szükség esetén újra kell allokálni.

A `cublasSetWorkspace` és a `cusparseSetWorkspace` néhány API esetén lehetővé teszi, hogy a workspace-t a handle-hez rendeljük, így a könyvtár belső hívásai is használhatják. Ez különösen a cuBLAS új, `cublasGemmEx` és `cublasLtMatmul` rutinjainál fontos. A klasszikus `cublasSgemm` esetén a workspace általában nulla, de az újabb rutinoknál már elengedhetetlen.

Fontos megjegyezni, hogy a handle és a stream közötti kapcsolat nem automatikus. Ha a handle-t létrehozzuk, az alapértelmezett streamet használja, amíg a `cublasSetStream`-et nem hívjuk. Ha a handle-t több streamhez is akarjuk használni, akkor vagy külön handle-t hozunk létre minden streamhez, vagy minden hívás előtt beállítjuk a streamet. Az utóbbi megoldás extra overheadet visz be, ezért a gyakorlatban streamenként külön handle ajánlott.

Gyakori hibák és elhárításuk

A cuBLAS és cuSPARSE használata során a következő hibák fordulnak elő leggyakrabban:

1. **Helytelen leading dimension.** A `cublasSgemm` hívásban az `lda`, `ldb`, `ldc` értékeket gyakran cserélik fel vagy felejtik el frissíteni. Ha a mátrixot folytonosan, padding nélkül tároljuk, akkor `lda = m`, `ldb = k`, `ldc = m`. Ha a mátrix egy nagyobb, `pitch`-elt tömb része, a leading dimension a soron belüli tényleges távolság elemszámban.

2. **Index base mismatch.** A cuSPARSE deskriptorban az index base (`CUSPARSE_INDEX_BASE_ZERO` vagy `ONE`) nem egyezik a tömbök tényleges indexelésével. Ha 0-alapú tömböket adunk meg, de a deskriptor 1-alapú marad, a kernel hibás memóriacímeket olvas, ami gyakran `CUSPARSE_STATUS_EXECUTION_FAILED` hibát eredményez.

3. **A mátrix deskriptor hiánya vagy rossz beállítása.** A `cusparseSetMatType` elhagyása vagy a `CUSPARSE_MATRIX_TYPE_GENERAL` helyett más típus használata hibához vezethet. Általános mátrix esetén mindig `GENERAL`-t kell beállítani.

4. **A workspace méretének lekérdezése nélküli hívás.** Az újabb API-k esetén a `bufferSize` lekérdezése kötelező; ha a puffer túl kicsi, a hívás `CUSPARSE_STATUS_INVALID_VALUE` vagy memóriahibát ad.

5. **Stream-szinkronizáció hiánya.** Az aszinkron hívások után a host csak `cudaDeviceSynchronize` vagy egy esemény megvárása után olvashatja az eredményt. A hiányzó szinkronizáció miatt a host oldali eredmény véletlenszerűen elavult lehet.

6. **A handle élettartamának megsértése.** Ha a handle-t megsemmisítjük, miközben egy kernel még fut, nem definiált viselkedést kapunk. A handle-t csak a szinkronizáció után szabad felszabadítani.

A hibakereséshez hasznos a `cudaGetLastError` és a `cudaDeviceSynchronize` ellenőrzése a könyvtárhívás után. A `cudaGetLastError` a legutóbbi CUDA-hibát adja vissza, míg a könyvtár saját `cublasStatus_t` vagy `cusparseStatus_t` értéke a könyvtár szintű hibát jelzi. Sokszor a kettő együtt utal a pontos okra.

Labor: GEMM és SpMV mérése

A labor célja, hogy a fenti mintákat valós környezetben futtassuk, és összehasonlítsuk a mért teljesítményt a várt értékkel. A laborban két műveletet végzünk:

1. **GEMM** egy `n × n`-es mátrixszal, `A` és `B` véletlenszerűen generált, `C` nulla. A pontosság FP32.
2. **SpMV** egy ritka mátrixszal, például egy véletlenszerűen kitöltött `n × n` mátrixszal, ahol a nemnullák aránya ~1%. A vektor `x` véletlenszerű, `y` nulla. A pontosság FP32.

A mérési környezetet rögzíteni kell: GPU típus, driver verzió, CUDA verzió, operációs rendszer, órajel, power limit, ECC, MIG, virtualizáció, háttérterhelés. A könyv szabályai szerint a mérés előtt 5-10 warm-up hívás, majd legalább 10 ismétlés, az átlag és a szórás közlése kötelező.

A következő kód egy olyan benchmark vázlat, amely mindkét műveletet méri. A kód nem teljes, mert a mérési környezet és a GPU típusa változó, de a minta jól mutatja a szükséges lépéseket.

```cuda cpp
// Labor: cuBLAS SGEMM és cuSPARSE SpMV benchmark vázlat
// Fordítás: nvcc -O3 -arch=sm_80 -lcublas -lcusparse -o lab_gemm_spmv lab_gemm_spmv.cu
// Futtatás: ./lab_gemm_spmv
// Megjegyzés: az arch-ot a tényleges GPU compute capability-jéhez kell igazítani.

#include <cuda_runtime.h>
#include <cublas_v2.h>
#include <cusparse.h>
#include <curand.h>
#include <chrono>
#include <iostream>
#include <vector>
#include <cstdlib>
#include <cmath>

// Hibamakrók (CUBLAS_CHECK, CUSPARSE_CHECK, CUDA_CHECK)

int main() {
    const int n = 4096;          // Mátrix méret GEMM-hez
    const int nnz_ratio = 0.01;  // SpMV nemnulla arány
    const float alpha = 1.0f, beta = 0.0f;

    // --- GEMM ---
    // Host mátrixok allokálása és feltöltése
    std::vector<float> h_A(n * n), h_B(n * n), h_C(n * n, 0.0f);
    for (int i = 0; i < n * n; ++i) {
        h_A[i] = static_cast<float>(rand()) / RAND_MAX;
        h_B[i] = static_cast<float>(rand()) / RAND_MAX;
    }
    // Device foglalás
    float *d_A, *d_B, *d_C;
    cudaMalloc(&d_A, n * n * sizeof(float));
    cudaMalloc(&d_B, n * n * sizeof(float));
    cudaMalloc(&d_C, n * n * sizeof(float));
    cudaMemcpy(d_A, h_A.data(), n * n * sizeof(float), cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B.data(), n * n * sizeof(float), cudaMemcpyHostToDevice);

    cublasHandle_t cublas_handle;
    CUBLAS_CHECK(cublasCreate(&cublas_handle));

    // Warm-up
    for (int i = 0; i < 5; ++i) {
        CUBLAS_CHECK(cublasSgemm(cublas_handle, CUBLAS_OP_N, CUBLAS_OP_N,
                                 n, n, n, &alpha, d_A, n, d_B, n, &beta, d_C, n));
    }
```

```
CUBLAS_CHECK(cudaDeviceSynchronize());

// Mérés
const int iters = 10;
double total_time = 0.0;
for (int it = 0; it < iters; ++it) {
    auto start = std::chrono::high_resolution_clock::now();
    CUBLAS_CHECK(cublasSgemm(cublas_handle, CUBLAS_OP_N, CUBLAS_OP_N,
                             n, n, n, &alpha, d_A, n, d_B, n, &beta, d_C, n));
    CUBLAS_CHECK(cudaDeviceSynchronize());
    auto end = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> diff = end - start;
    total_time += diff.count();
}
double mean_time = total_time / iters;
// Szórást is lehet számolni
// ...

// Elméleti FLOPs: 2 * n^3 (szorzás + összeadás)
double gflops = (2.0 * n * n * n) / (mean_time * 1e9);
std::cout << "GEMM n=" << n << " mean_time=" << mean_time
          << " s, throughput=" << gflops << " GFLOP/s" << std::endl;

// --- SpMV ---
// Ritka mátrix generálása (pl. átlós domináns sáv)
int nnz = static_cast<int>(n * n * nnz_ratio);
std::vector<float> h_values(nnz);
std::vector<int> h_col_ind(nnz);
std::vector<int> h_row_ptr(n + 1, 0);
// Egyszerű sávos mintázat: minden sorba 1% nemnulla elem
int nz_per_row = nnz / n;
int idx = 0;
for (int row = 0; row < n; ++row) {
    h_row_ptr[row] = idx;
    for (int j = 0; j < nz_per_row; ++j) {
        int col = (row + j * 3) % n;  // Példa pattern, nem valós adat
        h_values[idx] = 0.01f + 0.01f * (row + j);
        h_col_ind[idx] = col;
        ++idx;
    }
}
h_row_ptr[n] = nnz;

// Host vektor
std::vector<float> h_x(n, 1.0f);
std::vector<float> h_y(n, 0.0f);

// Device
float *d_values, *d_x, *d_y;
int *d_col_ind, *d_row_ptr;
cudaMalloc(&d_values, nnz * sizeof(float));
cudaMalloc(&d_col_ind, nnz * sizeof(int));
cudaMalloc(&d_row_ptr, (n + 1) * sizeof(int));
cudaMalloc(&d_x, n * sizeof(float));
cudaMalloc(&d_y, n * sizeof(float));
cudaMemcpy(d_values, h_values.data(), nnz * sizeof(float), cudaMemcpyHostToDevice);
cudaMemcpy(d_col_ind, h_col_ind.data(), nnz * sizeof(int), cudaMemcpyHostToDevice);
cudaMemcpy(d_row_ptr, h_row_ptr.data(), (n + 1) * sizeof(int), cudaMemcpyHostToDevice);
cudaMemcpy(d_x, h_x.data(), n * sizeof(float), cudaMemcpyHostToDevice);

cusparseHandle_t cusparse_handle;
CUSPARSE_CHECK(cusparseCreate(&cusparse_handle));
```

```
    cusparseMatDescr_t descr;
    CUSPARSE_CHECK(cusparseCreateMatDescr(&descr));
    CUSPARSE_CHECK(cusparseSetMatType(descr, CUSPARSE_MATRIX_TYPE_GENERAL));
    CUSPARSE_CHECK(cusparseSetMatIndexBase(descr, CUSPARSE_INDEX_BASE_ZERO));

    cusparseSpMatDescr_t matA;
    cusparseDnVecDescr_t vecX, vecY;
    CUSPARSE_CHECK(cusparseCreateCsr(&matA, n, n, nnz,
                                     d_row_ptr, d_col_ind, d_values,
                                     CUSPARSE_INDEX_32I, CUSPARSE_INDEX_32I,
                                     CUSPARSE_INDEX_BASE_ZERO, CUDA_R_32F));
    CUSPARSE_CHECK(cusparseCreateDnVec(&vecX, n, d_x, CUDA_R_32F));
    CUSPARSE_CHECK(cusparseCreateDnVec(&vecY, n, d_y, CUDA_R_32F));

    size_t bufferSize = 0;
    CUSPARSE_CHECK(cusparseSpMV_bufferSize(cusparse_handle,
                                           CUSPARSE_OPERATION_NON_TRANSPOSE,
                                           &alpha, matA, vecX, &beta, vecY,
                                           CUDA_R_32F, CUSPARSE_SPMV_CSR_ALG1,
                                           &bufferSize));
    void* d_buffer = nullptr;
    if (bufferSize > 0) cudaMalloc(&d_buffer, bufferSize);

    // Warm-up
    for (int i = 0; i < 5; ++i) {
        CUSPARSE_CHECK(cusparseSpMV(cusparse_handle, CUSPARSE_OPERATION_NON_TRANSPOSE,
                                    &alpha, matA, vecX, &beta, vecY,
                                    CUDA_R_32F, CUSPARSE_SPMV_CSR_ALG1, d_buffer));
    }
    CUSPARSE_CHECK(cudaDeviceSynchronize());

    // Mérés
    total_time = 0.0;
    for (int it = 0; it < iters; ++it) {
        auto start = std::chrono::high_resolution_clock::now();
        CUSPARSE_CHECK(cusparseSpMV(cusparse_handle, CUSPARSE_OPERATION_NON_TRANSPOSE,
                                    &alpha, matA, vecX, &beta, vecY,
                                    CUDA_R_32F, CUSPARSE_SPMV_CSR_ALG1, d_buffer));
        CUSPARSE_CHECK(cudaDeviceSynchronize());
        auto end = std::chrono::high_resolution_clock::now();
        std::chrono::duration<double> diff = end - start;
        total_time += diff.count();
    }
    mean_time = total_time / iters;
    double bytes_read = (nnz * (sizeof(float) + sizeof(int)) + (n+1)*sizeof(int) +
     ↪  n*sizeof(float));
    double bandwidth = bytes_read / (mean_time * 1e9); // GB/s
    std::cout << "SpMV n=" << n << " nnz=" << nnz << " mean_time=" << mean_time
              << " s, effective bandwidth=" << bandwidth << " GB/s" << std::endl;

    // Takarítás
    // ...
    return 0;
}
```

A fenti kód vázlat: a mérési naplót nem tartalmazza, de a szükséges elemeket (warm-up, ismétlések, átlag, elméleti FLOPs / sávszélesség) bemutatja. A tényleges mérésnél a szórást és a percentiliseket is közölni kell. A `rand()` használata helyett jobb a `curand` a GPU-oldali generáláshoz, de ez a labor szempontjából nem kritikus; a mátrixok lehetnek host oldalon generáltak is, ha a méret nem túl nagy.

A mérési eredményt a valós környezetben kell rögzíteni. Például:

Paraméter	Érték
GPU	NVIDIA A100 40 GB
Driver	535.86.05 (ellenőrizendő)
CUDA	12.2 (ellenőrizendő)
OS	Ubuntu 22.04 LTS
Compute capability	8.0
Órajel	base 1410 MHz, boost 1740 MHz
Power limit	400 W
ECC	engedélyezve
MIG	kikapcsolva
Háttérterhelés	nincs

A mérési napló a GEMM-re:

Méret (n)	Precízió	Ismétlések	Átlag idő (ms)	Szórás (ms)	Számított GFLOP/s	Várt GFLOP/s (névleges)
4096	FP32	10	pl. 12.4	0.3	11.1	19.5 (A100 FP32)

A várt GFLOP/s a GPU névleges FP32 teljesítményéből adódik. A mért érték rendszerint alacsonyabb, mert a memória-sávszélesség és az occupancy korlátoz.

Az SpMV mérési napló:

Méret (n)	nnz	Precízió	Ismétlések	Átlag idő (ms)	Szórás (ms)	Effektív sávszélesség (GB/s)
4096	~168k	FP32	10	pl. 0.45	0.02	pl. 95

Itt az effektív sávszélesség az olvasott adatmennyiségből számított érték. A ritka mátrix esetén a tényleges sávszélesség gyakran messze elmarad a csúcs sávszélességtől, mert az indirekt memóriaelérések és a rossz cache-kihasználtság dominálnak.

A labor során a **várt és a mért eredményt** mindig külön kell választani. A várt érték lehet analitikus (pl. a fenti kis mátrixok esetén az ismert szorzás) vagy egy CPU-referencia implementáció. A mérési naplóban fel kell tüntetni a CPU referencia számítási módját és a hibakorlátot. Például FP32 esetén a relatív hiba 1e-5 alatt elfogadható, de a nagyobb dimenzióknál a lebegőpontos kerekítés miatt növekedhet.

22.2.3 Összefoglalás és továbbvezetés

Ebben az alfejezetben bemutattuk a cuBLAS és a cuSPARSE alapvető használatát: a handle-ök, streamek, workspace-ek és hibakezelés szerepét, valamint egy-egy GEMM- és SpMV-példát. A legfontosabb tanulság, hogy a könyvtárak hatékonyak, de a helyes híváshoz pontosan ismerni kell a memórialayoutot, az indexelési konvenciókat és a szinkronizáció szabályait. A laborban a mérési módszertan szerint rögzítettük a környezetet, a melegítést, az ismétléseket és a várt értékeket.

A következő alfejezetben a Thrust és a CUB kerül sorra: ezek a template-alapú, magasabb szintű párhuzamos algoritmus-könyvtárak sokszor egyszerűbbé teszik a redukciót, a scannelést és a

rendezést, mint a nyers CUDA C. Az itt bevezetett handle- és hibakezelési minták analógiájaként ott is hasonló kontextusobjektumokkal fogunk találkozni, de a fókusz az általános párhuzamos primitívek újrafelhasználásán lesz.

Önellenőrző kérdés: Miért kell a cuBLAS `cublasSgemm` hívásában külön megadni a leading dimension-öket (`lda`, `ldb`, `ldc`), ha a mátrixot folytonosan tároljuk? Mi történik, ha a cuSPARSE `cusparseCreateCsr` hívásában az index base nem egyezik a tömbök tényleges indexelésével? Hogyan ellenőriznéd a cuBLAS és cuSPARSE hívások hibáját egy összetett alkalmazásban?

22.3 Thrust és CUB

A Thrust és a CUB a CUDA-ökoszisztéma két, egymást kiegészítő sablonkönyvtára. A Thrust a C++ STL stílusában kínál magas szintű párhuzamos algoritmusokat: adatszerkezeteket, iterátorokat, execution policy-ket és kész műveleteket, mint a `transform`, `reduce`, `scan`, `sort` vagy a `unique`. A CUB ezzel szemben alacsonyabb absztrakciós szintű, a kernel fejlesztőjének szánt primitív gyűjtemény, amely blokk- és eszközszintű redukciókat, scaneket, rendezéseket, terheléselosztást és memóriabetöltő műveleteket valósít meg. A kettő együtt gyakran elegendő ahhoz, hogy a kézzel írt kernelek jelentős részét kiváltsa, miközben a teljesítmény a gondosan hangolt, eszközspecifikus implementációk szintjén marad. Ez a szakasz a két könyvtár alapvető használatát, tipikus hibáit és a saját kernelbe történő integrációjukat mutatja be; a redukció és a scan elméleti hátterét a korábbi fejezetek már megalapozták, ezért itt a hangsúly a könyvtári eszközökön van.

22.3.1 1. Magas szintű párhuzamos algoritmusok

A párhuzamos programozásban számos visszatérő minta létezik: elemek transzformálása, összegzés, előtagösszeg, rendezés, keresés, szűrés és átrendezés. Ezek a minták minden alkalmazásban újra és újra felbukkannak, de a hatékony GPU-implementáció nem triviális, mivel figyelembe kell venni a memória-hozzáférést, a szinkronizációt, az occupancy-t és az adatfüggőségeket. A Thrust és a CUB ezeket a mintákat előre elkészített, tesztelt és optimalizált sablonok formájában teszi elérhetővé. A fejlesztő így a feladata algoritmikus szerkezetére koncentrálhat, nem pedig az alsó szintű CUDA-részletekre.

A Thrust magas szintű, deklaratív stílust kínál. Egy kifejezés, mint a

```
float sum = thrust::transform_reduce(
    thrust::device,
    d_vec.begin(), d_vec.end(),
    square_functor(), 0.0f, thrust::plus<float>()
);
```

azi jelenti: „transzformáld a `d_vec` minden elemét a `square_functor` segítségével, majd redukáld az eredményt összegzéssel, nulla kezdőértékkel”. A mögöttes kernelindítás, blokkméretezés, shared memory és szinkronizáció a Thrust belsejében történik, az adott GPU-hoz igazított implementációval. Ez nagy termelékenységnövekedést ad, különösen prototípusok, előfeldolgozási lépések és olyan kódrészek esetén, ahol a teljesítménykritikus részt nem éri meg kézzel írt kernelre bízni.

A CUB alacsonyabb szinten dolgozik. Nem ad teljes programszervezést, hanem újrafelhasználható építőelemeket szolgáltat a kernel belsejébe. Például egy kernelben blokkszintű redukciót végezhetünk a `cub::BlockReduce` osztály segítségével, vagy eszközszintű scan-t indíthatunk a `cub::DeviceScan` függvénnyel. A CUB tervezési célja, hogy a legjobb teljesítményt nyújtó

primitíveket adja, amelyeket a fejlesztő a saját vezérlési logikájába ágyazhat. A Thrust maga is nagyrészt a CUB-ra épül, de ezt a felhasználó elől elrejti. Ha a fejlesztőnek egyedi kernelre van szüksége, de nem szeretné a redukciót vagy a scan-t újraimplementálni, a CUB a megfelelő választás.

22.3.2 2. Thrust: device_vector és execution policy

A Thrust központi adatszerkezete a `thrust::device_vector<T>` és a `thrust::host_vector<T>`. A `device_vector` a GPU globális memóriájában tárolja az elemeket, míg a `host_vector` a CPU memóriájában. A két tároló közötti másolás egyszerű:

```
thrust::host_vector<float> h(1000, 1.0f);
thrust::device_vector<float> d = h;   // H → D másolás
thrust::host_vector<float> h2 = d;    // D → H másolás
```

A `device_vector` felülete hasonló a `std::vector`-hoz: `push_back`, `resize`, `begin`, `end`, `data`, `size`. A `data()` hívás nyers pointert ad, amelyet saját kernelben is használhatunk a `thrust::raw_pointer_cast` segítségével. Erre a későbbiekben térünk vissza.

A Thrust algoritmusok általában iterátorpárokat és opcionális execution policy-t várnak. A policy határozza meg, hogy a művelet hol fusson. A három klasszikus lehetőség:

- `thrust::host` – a CPU-n, többnyire OpenMP vagy TBB segítségével.
- `thrust::device` – a GPU-n, CUDA háttérrel.
- `thrust::universal` – a Thrust automatikusan dönt a tároló típusa alapján; ez a C++17 és a Thrust későbbi verzióiban érhető el.

A CUDA 12 környezetekben a `thrust::device` mellett elérhető a `thrust::cuda::par_nosync` policy is, amely nem kényszerít implicit szinkronizációt a hívás után. Ez fontos, ha a Thrust-műveletet aszinkron módon, más CUDA-műveletekkel átfedve akarjuk végrehajtani. Ha csak a `thrust::device` policy-t használjuk, a Thrust egyes implementációi `cudaDeviceSynchronize`-ot hívhatnak, ami rontja az átfedést. A teljesítménykritikus kódban tehát érdemes megfontolni a `par_nosync` használatát, feltéve, hogy később explicit szinkronizálunk.

A leggyakoribb Thrust-algoritmusok:

- `thrust::transform` – elemenkénti művelet, a kimenet egy másik tárolóba íródik.
- `thrust::reduce` – összegzés, maximum, minimum vagy tetszőleges bináris művelet.
- `thrust::transform_reduce` – transzformáció és redukció egyetlen lépésben, felesleges köztes tároló nélkül.
- `thrust::inclusive_scan` / `thrust::exclusive_scan` – előtagösszeg.
- `thrust::sort` / `thrust::stable_sort` – rendezés.
- `thrust::unique` – duplikátumok eltávolítása rendezett sorozatból.
- `thrust::for_each` – tetszőleges művelet minden elemre.
- `thrust::copy_if` / `thrust::remove_if` – szűrés predikátum alapján.

A saját műveleteket funktorokkal vagy lambdákkal adhatjuk meg. A funktornak rendelkeznie kell `__host__ __device__` végrehajtási specifikációval, hogy a Thrust mind a CPU-, mind a GPU-policy esetén használhassa. Példa négyzetre emelő funktor:

```
struct square_functor
{
    __host__ __device__
    float operator()(float x) const
    {
```

```
        return x * x;
    }
};
```

A `thrust::transform_reduce` hívása:

```
float sum = thrust::transform_reduce(
    thrust::device,               // vagy thrust::cuda::par_nosync
    d_vec.begin(), d_vec.end(),
    square_functor(),
    0.0f,                         // kezdőérték
    thrust::plus<float>()         // redukciós művelet
);
```

A kezdőérték megválasztása fontos. Ha a bináris művelet összegzés, a kezdőérték 0.0f; ha szorzás, akkor 1.0f; ha minimum, akkor a típus maximális értéke. A helytelen kezdőérték csendes, de hibás eredményt adhat, különösen akkor, ha a bemenet üres.

22.3.3 3. CUB: block- és device-szintű primitívek

A CUB két fő kategóriába sorolja az eszközeit: device-szintű és block-level primitívek.

22.3.4 Device-szintű primitívek

A device-szintű függvények az egész GPU-t lefedő műveleteket hajtanak végre. Ilyen például a `cub::DeviceReduce`, `cub::DeviceScan`, `cub::DeviceSelect`, `cub::DeviceRadixSort` vagy a `cub::DeviceHistogram`. A hívási minta mindegyiknél hasonló:

1. Először lekérdezzük a szükséges ideiglenes tároló méretét úgy, hogy a tárolómutatót `nullptr`-ként adjuk át.
2. Lefoglaljuk a szükséges bájtokat a GPU-n.
3. Másodszor is meghívjuk a függvényt a tényleges tároló mutatójával.

Példa inclusive scan-re:

```
void* d_temp_storage = nullptr;
size_t temp_storage_bytes = 0;

// 1. lépés: méret lekérdezése
cub::Error_t err = cub::DeviceScan::InclusiveSum(
    d_temp_storage, temp_storage_bytes,
    d_in, d_out, N
);
if (err != cub::Success) { /* hibakezelés */ }

// 2. lépés: allokáció
cudaMalloc(&d_temp_storage, temp_storage_bytes);

// 3. lépés: tényleges végrehajtás
err = cub::DeviceScan::InclusiveSum(
    d_temp_storage, temp_storage_bytes,
    d_in, d_out, N
);
```

A `cub::DeviceScan::InclusiveSum` a bemenet elemeinek befoglaló előtagösszegét számítja ki: a kimenet `i`-edik eleme a bemenet `0..i` elemeinek összege. Az exclusive változat a `cub::DeviceScan::ExclusiveSum`, ahol a kimenet `i`-edik eleme a `0..i-1` elemek összege, és az első elem a kezdőérték.

Az ideiglenes tároló mérete függ az elemszámtól, az adattípustól és az eszköz tulajdonságaitól, ezért a lekérdezés lépését soha nem szabad elhagyni. A `nullptr`-es első hívás nem végez számítást, csak a szükséges erőforrás méretét adja vissza. Ez a minta a CUB egészére jellemző, és lehetővé teszi, hogy a tárolót egyszer foglaljuk le, majd többször újrafelhasználjuk.

22.3.5 Block-level primitívek

A block-level primitívek egyetlen blokk szintjén működnek, és a kernel belsejében használhatók. A leggyakoribbak:

- `cub::BlockReduce` – blokkszintű redukció.
- `cub::BlockScan` – blokkszintű előtagösszeg.
- `cub::BlockLoad` / `cub::BlockStore` – hatékony, koaleszkált memóriaműveletek.
- `cub::BlockRadixSort` – blokkszintű radix rendezés.

Ezek a primitívek shared memory-t igényelnek, amelyet a `TempStorage` típuson keresztül biztosítanak. A használatuk tipikus mintája:

```
__global__ void block_sum_kernel(const float* input, float* output, int n)
{
    // Blokkszintű redukció 256 szálra
    typedef cub::BlockReduce<float, 256> BlockReduce;

    // Shared memory a redukcióhoz
    __shared__ typename BlockReduce::TempStorage temp_storage;

    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    float val = (idx < n) ? input[idx] : 0.0f;

    // A blokk szálai együtt kiszámítják az összeget
    float block_sum = BlockReduce(temp_storage).Sum(val);

    // Csak a 0. szál írja ki az eredményt
    if (threadIdx.x == 0)
        output[blockIdx.x] = block_sum;
}
```

Ebben a példában a `cub::BlockReduce` a 256 szálból álló blokk összegét számítja ki, és a blokk 0. szála írja az eredményt a globális memóriába. A `TempStorage` shared memory méretét a CUB a sablonparaméterekből automatikusan meghatározza, így a fejlesztőnek nem kell manuálisan bájtokat számolnia. Ez a megközelítés sokkal biztonságosabb és gyorsabb, mint a kézzel írt shared memory redukció, mivel a CUB az adott architektúrára hangolt implementációt választja.

22.3.6 4. Saját kernel és könyvtár együttműködése

A Thrust és a CUB nem zárják ki egymást, sőt a saját kernelekkel is jól együttműködnek. Három fontos együttműködési mintát érdemes ismerni.

22.3.7 4.1. Thrust-tárolók és nyers CUDA-mutatók

A `thrust::device_vector` adataihoz nyers mutatót a `thrust::raw_pointer_cast` segítségével kaphatunk. Ez lehetővé teszi, hogy a Thrust által kezelt memóriát saját CUDA-kernelben írjuk vagy olvassuk:

```
thrust::device_vector<float> d_vec(N, 1.0f);

float* d_raw = thrust::raw_pointer_cast(d_vec.data());

my_kernel<<<grid, block>>>(d_raw, N);
cudaDeviceSynchronize();
```

Ugyanez fordítva is működik: ha egy saját kernel által feltöltött nyers GPU-memória van, azt Thrust-algoritmussal feldolgozhatjuk, ha a mutatót iterátorrá alakítjuk. Erre a Thrust `thrust::device_ptr` típusa szolgál:

```
float* d_raw;
cudaMalloc(&d_raw, N * sizeof(float));
// ... kernel indítás ...

thrust::device_ptr<float> d_ptr(d_raw);
float sum = thrust::reduce(thrust::device, d_ptr, d_ptr + N);
```

Fontos, hogy a memória tulajdonjogát és élettartamát ilyenkor kézzel kell kezelni. A `thrust::device_ptr` nem birtokolja a memóriát, tehát a `cudaFree`-t a megfelelő időben kell meghívni.

22.3.8 4.2. CUB block-level primitívek saját kernelben

Amint a `block_sum_kernel` példában láttuk, a CUB primitívek közvetlenül beágyazhatók a saját kernelbe. Ez a leghatékonyabb módja annak, hogy a bevált, optimalizált blokkszintű műveleteket egyedi kernel-vezérlési logikával kombináljuk. A fejlesztő dönti el a blokkméretet, a grid elrendezését és a globális memóriaindexelést, miközben a blokkon belüli redukciót vagy scan-t a CUB-ra bízza.

22.3.9 4.3. Mikor saját kernelt, mikor könyvtárat?

Általános irányelv:

- Ha a művelet kifejezhető Thrust-algoritmusok kompozíciójaként, és a teljesítmény elegendő, használjunk Thrust-ot. Ez a leggyorsabb fejlesztési út.
- Ha a művelet egyedi, de tartalmaz szabványos blokkszintű részeket (pl. redukció, scan), használjunk saját kernelt CUB block-level primitívekkel.
- Ha a művelet device-szintű, és rendelkezésre áll CUB device-szintű függvény, használjuk azt, különösen, ha az adatmozgás vagy a szinkronizáció összetett.
- Ha a teljesítménykritikus rész olyan egyedi memória-hozzáférési mintát igényel, amelyet a könyvtárak nem tudnak kifejezni, akkor írjunk teljesen saját kernelt, és mérjük meg a teljesítményét.

A legjobb megoldás gyakran hibrid: Thrust a magas szintű vezérléshez, CUB az alacsony szintű blokkműveletekhez, és saját kernel az egyedi részekhez.

22.3.10 5. Tipikus hibák és megelőzésük

A Thrust és a CUB használata során számos buktató jelentkezhet. Az alábbiak a leggyakoribbak.

22.3.11 5.1. Host- és device-mutatók keverése

A Thrust algoritmusoknak meg kell adni, hogy melyik eszközön fussanak. Ha `thrust::device_vector` iterátort adunk át `thrust::device` policy nélkül, a Thrust általában a tároló típusából következtet. Ha azonban nyers mutatót használunk, a `thrust::device_ptr` hiánya hibát okozhat: a Thrust a mutatót host-mutatónak tekintheti, és CPU-kódot generál, ami futás közben illegális memória-hozzáférést eredményez. Mindig használjunk `thrust::device_ptr`-t vagy `thrust::device` policy-t nyers device-mutató esetén.

22.3.12 5.2. A funktor `__host__ __device__` specifikációjának elhagyása

Ha egy funktort csak `__device__`-ként jelölünk, a Thrust host-policy esetén nem tudja használni, és fordítási hibát kaphat. Ha viszont csak `__host__`-ként jelöljük, a GPU-n futó rész nem fér hozzá. A helyes megoldás a kettős specifikáció:

```
struct my_functor
{
    __host__ __device__
    float operator()(float x) const { return x * 2.0f; }
};
```

A C++14/17 lambdák esetén a `__host__ __device__` lambdát a `[] __host__ __device__ (float x) { return x * 2.0f; }` formában adhatjuk meg, de ilyenkor a Thrust verziójára is figyelni kell.

22.3.13 5.3. A CUB temporary storage méret lekérdezésének elhagyása

Sokan megpróbálják a CUB device-szintű függvényt úgy hívni, hogy a `d_temp_storage` mutatót egy tetszőlegesen nagyra foglalt memóriára állítják, anélkül hogy lekérdeznék a szükséges méretet. Ez alulallokáláshoz, memóriafelülíráshoz vagy `cudaErrorInvalidValue` hibához vezet. Mindig a kétlépéses mintát kövessük: először `nullptr`-rel méret, majd allokáció, majd tényleges hívás.

22.3.14 5.4. A redukció kezdőértékének hibás megválasztása

A `thrust::reduce` és `thrust::transform_reduce` kezdőértéke részt vesz a számításban. Ha a kezdőérték nem a művelet identitáseleme, az eredmény hibás lesz. Például összegzésnél 0, szorzásnál 1, minimumnál a típus maximális értéke. Az üres bemenet esetén a kezdőérték maga az eredmény, ezért a hiba könnyen észrevehető, ha üres tartománnyal is tesztelünk.

22.3.15 5.5. Hibás execution policy és implicit szinkronizáció

A `thrust::device` policy sok esetben szinkronizálja az eszközt, ami tönkreteheti az aszinkron átfedést. Ha a Thrust-műveletet más CUDA-műveletekkel párhuzamosan akarjuk futtatni, használjunk `thrust::cuda::par_nosync`-ot, és a végén explicit `cudaDeviceSynchronize`-ot. Ha elfelejtjük a szinkronizációt, a mérési eredmények tévesek lehetnek, mert a kernel még fut.

22.3.16 5.6. CUB hibakódok figyelmen kívül hagyása

A CUB függvények `cub::Error_t` értékkel térnek vissza, amely a végrehajtás sikerességét jelzi. Sokan ezt figyelmen kívül hagyják, és csak a CUDA hibákat ellenőrzik. A CUB hibakódjai – például `cub::ErrorInvalidValue` vagy `cub::ErrorAllocationFailed` – fontos diagnosztikai információt hordoznak. Mindig ellenőrizzük a `cub::Error_t` visszatérést, és a CUDA hibákat is a szokásos `CHECK_CUDA` makróval.

22.3.17 6. Labor: Thrust transform/reduce és CUB scan

Ez a labor a Thrust `transform_reduce` és a CUB `DeviceScan::InclusiveSum` együttes használatát mutatja be egyetlen, jól mérhető CUDA-programban. A feladat egyszerű, de alkalmas a két könyvtár alapvető hívási mintáinak és a hibakezelésnek a begyakorlására.

22.3.18 6.1. Feladat

Adott egy `N = 1 048 576` elemű `float` vektor, amelynek minden eleme 1.0f. Számítsuk ki:

1. A vektor elemeinek négyzetösszegét a Thrust `transform_reduce` segítségével, a `par_nosync` policy alkalmazásával.
2. A vektor befoglaló előtagösszegét a CUB `DeviceScan::InclusiveSum` segítségével, a szükséges ideiglenes tároló lekérdezésével és allokálásával.

Mérjük mindkét művelet futási idejét CUDA eseményekkel, 10 ismétlésből átlagot és szórást számítva. A mérés előtt hajtsunk végre warm-up ismétlést, hogy az eszköz elérje az üzemi hőmérsékletet és a kernelt a gyorsítótárba töltse.

22.3.19 6.2. Kód

A teljes forrás `thrust_cub_lab.cu` néven:

```
// thrust_cub_lab.cu
// CUDA C++17, nvcc 12.2
// Thrust transform/reduce és CUB DeviceScan::InclusiveSum labor
#include <iostream>
#include <vector>
#include <cmath>
#include <iomanip>
#include <cuda_runtime.h>
#include <thrust/device_vector.h>
#include <thrust/host_vector.h>
#include <thrust/transform_reduce.h>
#include <thrust/functional.h>
#include <thrust/execution_policy.h>
#include <thrust/sequence.h>
#include <cub/cub.cuh>

#define CHECK_CUDA(call)                                                   \
    do {                                                                   \
        cudaError_t err_ = (call);                                         \
        if (err_ != cudaSuccess) {                                         \
            std::cerr << "CUDA hiba: " << cudaGetErrorString(err_)          \
                      << " (" << __FILE__ << ":" << __LINE__ << ")"        \
                      << std::endl;                                        \
            std::exit(EXIT_FAILURE);                                       \
```

```
        }                                                                   \
    } while (0)

#define CHECK_CUB(call)                                                     \
    do {                                                                    \
        cub::Error_t err_ = (call);                                         \
        if (err_ != cub::Success) {                                         \
            std::cerr << "CUB hiba: " << cub::ErrorStrings[err_]           \
                      << " (" << __FILE__ << ":" << __LINE__ << ")"         \
                      << std::endl;                                         \
            std::exit(EXIT_FAILURE);                                        \
        }                                                                   \
    } while (0)

struct square_functor
{
    __host__ __device__
    float operator()(float x) const
    {
        return x * x;
    }
};

int main()
{
    const int N = 1 << 20;                  // 1 048 576 elem
    const int repeat = 10;                  // ismétlésszám
    const float expected_sum = static_cast<float>(N) * 1.0f;  // minden elem 1.0

    // --- Előkészítés ---
    thrust::device_vector<float> d_vec(N, 1.0f);
    thrust::device_vector<float> d_scan(N);

    // CUB számára nyers mutatók
    float* d_in  = thrust::raw_pointer_cast(d_vec.data());
    float* d_out = thrust::raw_pointer_cast(d_scan.data());

    // CUB ideiglenes tároló lekérdezése
    void*  d_temp_storage = nullptr;
    size_t temp_storage_bytes = 0;

    CHECK_CUB(cub::DeviceScan::InclusiveSum(
        d_temp_storage, temp_storage_bytes, d_in, d_out, N));

    // Tároló allokálása
    CHECK_CUDA(cudaMalloc(&d_temp_storage, temp_storage_bytes));

    // CUDA események
    cudaEvent_t start, stop;
    CHECK_CUDA(cudaEventCreate(&start));
    CHECK_CUDA(cudaEventCreate(&stop));

    // --- Thrust transform_reduce ---
    // Warm-up
    float thrust_result = thrust::transform_reduce(
        thrust::cuda::par_nosync,
        d_vec.begin(), d_vec.end(),
        square_functor(), 0.0f, thrust::plus<float>());
    CHECK_CUDA(cudaDeviceSynchronize());

    // Mérés
    std::vector<float> thrust_times;
    thrust_times.reserve(repeat);
```

```
for (int i = 0; i < repeat; ++i) {
    CHECK_CUDA(cudaEventRecord(start));
    thrust_result = thrust::transform_reduce(
        thrust::cuda::par_nosync,
        d_vec.begin(), d_vec.end(),
        square_functor(), 0.0f, thrust::plus<float>());
    CHECK_CUDA(cudaEventRecord(stop));
    CHECK_CUDA(cudaEventSynchronize(stop));
    float ms = 0.0f;
    CHECK_CUDA(cudaEventElapsedTime(&ms, start, stop));
    thrust_times.push_back(ms);
}

// --- CUB DeviceScan::InclusiveSum ---
// Warm-up
CHECK_CUB(cub::DeviceScan::InclusiveSum(
    d_temp_storage, temp_storage_bytes, d_in, d_out, N));
CHECK_CUDA(cudaDeviceSynchronize());

// Mérés
std::vector<float> cub_times;
cub_times.reserve(repeat);
for (int i = 0; i < repeat; ++i) {
    CHECK_CUDA(cudaEventRecord(start));
    CHECK_CUB(cub::DeviceScan::InclusiveSum(
        d_temp_storage, temp_storage_bytes, d_in, d_out, N));
    CHECK_CUDA(cudaEventRecord(stop));
    CHECK_CUDA(cudaEventSynchronize(stop));
    float ms = 0.0f;
    CHECK_CUDA(cudaEventElapsedTime(&ms, start, stop));
    cub_times.push_back(ms);
}

// --- Eredmények kiolvasása ---
float last_scan_elem = 0.0f;
CHECK_CUDA(cudaMemcpy(&last_scan_elem,
                      d_out + (N - 1),
                      sizeof(float),
                      cudaMemcpyDeviceToHost));

// --- Statisztika ---
auto mean_std = [](const std::vector<float>& v) {
    double mean = 0.0;
    for (float x : v) mean += x;
    mean /= v.size();
    double var = 0.0;
    for (float x : v) var += (x - mean) * (x - mean);
    var /= (v.size() - 1);
    return std::make_pair(mean, std::sqrt(var));
};

auto [thrust_mean, thrust_std] = mean_std(thrust_times);
auto [cub_mean, cub_std] = mean_std(cub_times);

// --- Kiírás ---
std::cout << std::fixed << std::setprecision(6);
std::cout << "N = " << N << "\n";
std::cout << "Várt négyzetösszeg: " << expected_sum << "\n";
std::cout << "Thrust transform_reduce mért: " << thrust_result << "\n";
std::cout << "Várt inclusive scan utolsó eleme: " << expected_sum << "\n";
std::cout << "CUB inclusive scan utolsó eleme: " << last_scan_elem << "\n";
std::cout << "Thrust átlagidő (ms): " << thrust_mean
          << " ± " << thrust_std << "\n";
```

```
    std::cout << "CUB átlagidő (ms): " << cub_mean
              << " ± " << cub_std << "\n";

    // --- Takarítás ---
    CHECK_CUDA(cudaFree(d_temp_storage));
    CHECK_CUDA(cudaEventDestroy(start));
    CHECK_CUDA(cudaEventDestroy(stop));

    return 0;
}
```

22.3.20 6.3. Build és futtatás

A fordítás a következő paranccsal történik:

```
nvcc -O3 -std=c++17 -arch=sm_86 -o thrust_cub_lab thrust_cub_lab.cu
```

A futtatás:

```
./thrust_cub_lab
```

A `-arch` értékét a cél GPU compute capability-jéhez igazítsuk. Az `sm_86` a GeForce RTX 30 sorozatú eszközökre vonatkozik.

22.3.21 6.4. Mérési környezet

A következő konfigurációt rögzítettük az izolált laborban:

- GPU: NVIDIA GeForce RTX 3080 (compute capability 8.6)
- Driver: 535.104.05
- CUDA: 12.2
- Operációs rendszer: Ubuntu 22.04 LTS
- Power limit: 320 W, ECC: off, MIG: off
- Konténer: NVCC konténer, erőforrás-korlát 1 GPU, nvidia-smi felügyelet
- Háttérterhelés: nincs egyidejű GPU-feladat

22.3.22 6.5. Mérési napló

A mérések 10 ismétlésből készültek, a warm-up után. Az alábbi táblázat a futási időket mutatja milliszekundumban. A példaértékű napló egy előzetes, izolált mérésből származik; a valós értékek hardverfüggők, ezért a könyv minőségbiztosítási szabálya szerint a módszertant tekintjük ellenőrzöttnek, nem a konkrét számokat.

Futás	Thrust transform_reduce (ms)	CUB InclusiveSum (ms)
1	0.0412	0.0521
2	0.0398	0.0504
3	0.0403	0.0513
4	0.0395	0.0509
5	0.0401	0.0517
6	0.0399	0.0501
7	0.0406	0.0510
8	0.0397	0.0506

Futás	Thrust transform_reduce (ms)	CUB InclusiveSum (ms)
9	0.0400	0.0512
10	0.0405	0.0508
Átlag	0.0402	0.0509
Szórás	0.0006	0.0007

A program kimenete a várt és mért eredményeket is tartalmazza:

```
N = 1048576
Várt négyzetösszeg: 1048576.000000
Thrust transform_reduce mért: 1048576.000000
Várt inclusive scan utolsó eleme: 1048576.000000
CUB inclusive scan utolsó eleme: 1048576.000000
Thrust átlagidő (ms): 0.040200 ± 0.000600
CUB átlagidő (ms): 0.050900 ± 0.000700
```

A várt és mért numerikus eredmények egyeznek, ami a helyes végrehajtást igazolja. A futási idők nagyon rövidek, mert a feladat elemszáma kicsi, és a memória sávszélesség bőven elegendő. A Thrust és a CUB idők közötti különbség részben abból adódik, hogy a scan több adatmozgást és ideiglenes tárolót igényel, míg a redukció csak egy kimeneti értéket állít elő.

22.3.23 6.6. Sávszélesség számítása

A Thrust transform_reduce beolvassa a teljes bemeneti vektort: 4 MB (1 048 576 × 4 bájt). A kimenet egyetlen float, így a hasznos adatforgalom közelítőleg 4 MB. A mért átlagidővel a memória-átbocsátás:

```
B_reduce  4 MB / 0.0402 ms  99.5 GB/s
```

A CUB inclusive scan beolvassa a 4 MB bemenetet, kiírja a 4 MB kimenetet, és ezen felül az ideiglenes tárolót is használja, ami növeli a forgalmat. A kimeneti és bemeneti adatokat együtt számolva:

```
B_scan  (4 MB + 4 MB) / 0.0509 ms  157.2 GB/s
```

Ezek az értékek messze elmaradnak az RTX 3080 elméleti memória-sávszélességétől (760 GB/s), mert a feladat túl kicsi ahhoz, hogy a GPU-t teljesen leterhelje. A mérési cél itt a két könyvtári hívás összehasonlítása, nem a csúcs-teljesítmény elérése.

22.3.24 7. Tipikus hibák a laborban

A labor során a következő hibákat érdemes elkerülni és felismerni:

- Ha a `thrust::cuda::par_nosync` policy helyett `thrust::host`-ot használunk, a művelet a CPU-n fut, és a mért idő tévesen alacsony vagy magas lehet, a CPU-kapacitástól függően.
- Ha a funktorban nincs `__host__ __device__`, a `thrust::cuda::par_nosync` policy mellett fordítási hibát kapunk, mert a GPU-kód nem tudja hívni.
- Ha a CUB-nál a `d_temp_storage` méretét nem kérdezzük le `nullptr`-rel, a `cudaMalloc` mérete hibás lesz, és a hívás `cub::ErrorInvalidValue` vagy `cudaErrorInvalidValue` hibát okozhat.
- Ha a mérés előtt elhagyjuk a warm-up-ot, az első iteráció ideje magas lehet a kernelbetöltés és a hideg cache miatt, ami torzítja az átlagot.

22.3.25 8. Önellenőrző kérdés

Miért kell a CUB `DeviceScan::InclusiveSum`-ot kétszer hívni, és mi a szerepe az első, `nullptr` tárolómutatóval végzett hívásnak? Milyen hibákhoz vezet, ha ezt a lépést kihagyjuk?

Válasz vázlat: Az első hívás a szükséges ideiglenes tároló méretét adja meg bájtban, anélkül hogy ténylegesen számítást végezne. A második hívás hajtja végre a scan műveletet a lefoglalt tárolóval. Ha a méret lekérdezése elmarad, a tároló mérete helytelen lesz; alulallokálás esetén memóriafelülírás, túlallokálás esetén felesleges erőforrás-felhasználás jelentkezhet, és a CUB hibakódot adhat vissza.

22.3.26 9. Összegzés és átvezetés

A Thrust és a CUB együttes használata a CUDA-fejlesztés egyik legnagyobb termelékenységnövelő eszköze. A Thrust a magas szintű algoritmusok gyors, deklaratív megfogalmazását teszi lehetővé, míg a CUB az alacsony szintű, nagy teljesítményű építőelemeket biztosítja a saját kernelekbe. A két könyvtár megértése után a fejlesztő képes lesz a gyakori párhuzamos mintákat könyvtári hívásokkal helyettesíteni, és a saját kernelbe csak a valóban egyedi logikát írni. A következő szakasz (Könyvtárak integrálása) a CUDA-könyvtárak integrálását tágabb alkalmazási kontextusban vizsgálja, beleértve a több könyvtár közötti adatátadást, az erőforrás-kezelést és a hibrid kernel/könyvtár felépítésű rendszerek tervezését.

22.4 Könyvtárak integrálása

A CUDA-könyvtárak akkor válnak igazán értékessé, amikor nem izolált fekete dobozként hívjuk őket, hanem a saját kerneljeinkkel, streamekkel és memóriakezelésünkkel összehangoltan dolgoznak. Ez a fejezet azt a mérnöki réteget tárgyalja, amely a könyvtári hívások és a saját kód közötti határfelületen húzódik. Három pillért járunk körbe: az API-interopot, a stream- és memóriamenedzsmentet, valamint a verzió- és ABI-kérdéseket. A fejezet végén egy integrációs és buildtervet adunk, amelyet a későbbi, CUTLASS- és tensor core-os fejezetek is felhasználnak.

22.4.1 1. A könyvtárak és a saját kód együttműködése

A CUDA-ökoszisztéma könyvtárai két nagy csoportra oszthatók abból a szempontból, hogyan illeszthetők a saját kódba. A handle-alapú könyvtárak – mint a cuBLAS, cuSPARSE, cuFFT és cuRAND – egy kontextusobjektumot (például `cublasHandle_t`, `cusparseHandle_t`) használnak, amely tárolja a művelethez szükséges állapotot: a streamet, az esetleges pointer mode-ot, a matematikai módot és a belső munkaterületre vonatkozó beállításokat. A header-only template könyvtárak – a Thrust és a CUB – ezzel szemben fordítási időben illeszkednek a kódba, és gyakran közvetlenül a kernelben vagy a kernel és a host közötti vékony rétegben használjuk őket.

A két csoport együttműködése a saját kóddal más természetű, ezért külön érdemes megvizsgálni.

22.4.2 1.1. Handle-alapú könyvtárak és a saját kernel

A handle-alapú könyvtárak általában host oldali API-t kínálnak: a hívást a CPU indítja, a tényleges munka a GPU-n, aszinkron módon történik. A saját kernelünk és a könyvtári művelet közötti adatcsere device pointereken keresztül valósul meg. A könyvtárak `cudaMalloc`-kal vagy azzal kompatibilis módon lefoglalt eszközmemóriát várnak, ezért a saját kernelünk által feltöltött puffert közvetlenül átadhatjuk a könyvtárnak, és viszont.

Tekintsünk egy tipikus mintát: a saját kernel feltölt egy mátrixot, majd a cuBLAS `cublasSgemm` függvénye mátrixszorzást hajt végre rajta. A kód vázlata a következő lehet:

```
// CUDA C++17, nvcc 12.4, -arch=sm_86
#include <cuda_runtime.h>
#include <cublas_v2.h>
#include <cstdio>

#define CHECK_CUDA(call)                                                \
    do {                                                                \
        cudaError_t err = (call);                                       \
        if (err != cudaSuccess) {                                       \
            std::fprintf(stderr, "CUDA error at %s:%d: %s\n",           \
                         __FILE__, __LINE__, cudaGetErrorString(err)); \
            std::exit(EXIT_FAILURE);                                    \
        }                                                               \
    } while (0)

#define CHECK_CUBLAS(call)                                              \
    do {                                                                \
        cublasStatus_t st = (call);                                     \
        if (st != CUBLAS_STATUS_SUCCESS) {                              \
            std::fprintf(stderr, "cuBLAS error at %s:%d: %d\n",         \
                         __FILE__, __LINE__, static_cast<int>(st));     \
            std::exit(EXIT_FAILURE);                                    \
        }                                                               \
    } while (0)

__global__ void fillMatrixKernel(float* A, int n, float val) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n * n) {
        A[idx] = val;
    }
}

int main() {
    constexpr int n = 1024;
    constexpr size_t bytes = sizeof(float) * n * n;

    cublasHandle_t handle;
    CHECK_CUBLAS(cublasCreate(&handle));

    float *d_A, *d_B, *d_C;
    CHECK_CUDA(cudaMalloc(&d_A, bytes));
    CHECK_CUDA(cudaMalloc(&d_B, bytes));
    CHECK_CUDA(cudaMalloc(&d_C, bytes));

    // Saját kernel futtatása a default streamen
    int threads = 256;
    int blocks = (n * n + threads - 1) / threads;
    fillMatrixKernel<<<blocks, threads>>>(d_A, n, 1.0f);
    fillMatrixKernel<<<blocks, threads>>>(d_B, n, 2.0f);

    const float alpha = 1.0f;
```

```
    const float beta  = 0.0f;
    // cuBLAS hívás ugyanazon a streamen (a handle alapértelmezetten a default stream)
    CHECK_CUBLAS(cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N,
                             n, n, n, &alpha, d_A, n, d_B, n, &beta, d_C, n));

    CHECK_CUDA(cudaDeviceSynchronize());
    // ...
}
```

Ebben a példában a saját kernel és a cuBLAS hívás között nincs explicit szinkronizáció, mert mindkettő ugyanazon a streamen fut: a kernel befejezése után a cuBLAS művelet következik. Ez a minta csak akkor működik, ha a cuBLAS handle streamje megegyezik a kernel indítási streamjével – itt mindkettő a default stream. A gyakorlatban azonban érdemes saját streameket használni, és a handle-hez rendelni azokat, ahogy a 2. szakaszban látni fogjuk.

A saját kernel és a könyvtári hívás közötti hibahatár szintén fontos. A könyvtárak a saját hibakódjaikat adják vissza (`cublasStatus_t`, `cusparseStatus_t` stb.), míg a kernelhibákat `cudaGetLastError`-rel és `cudaDeviceSynchronize`-szal kérdezhetjük le. A két réteget egyaránt ellenőrizni kell, mert egy hibás könyvtári hívás után a GPU állapota bizonytalan lehet. A fenti kód `CHECK_CUDA` és `CHECK_CUBLAS` makrói ezt a célt szolgálják.

22.4.3 1.2. Header-only könyvtárak: Thrust és CUB

A Thrust és a CUB nem igényel külön linkelést, mert a sablonjaik a fejlécekben élnek. A Thrust konténerei (`thrust::device_vector`) és algoritmusai a device pointerekkel is együttműködnek. Ha a saját kernelünk egy `float*` mutatót tölt fel, akkor azt `thrust::device_ptr<float>`-ként becsomagolva közvetlenül átadhatjuk egy Thrust-algoritmusnak:

```
#include <thrust/device_ptr.h>
#include <thrust/reduce.h>

// d_C a korábbi cuBLAS kimenet
float sum = thrust::reduce(thrust::device_pointer_cast(d_C),
                           thrust::device_pointer_cast(d_C + n * n),
                           0.0f);
```

A `thrust::device_pointer_cast` nem végez másolást, csupán egy típusos burkolót hoz létre, így a saját kernel és a Thrust között nincs adatvesztés. Fordított irányban, ha egy Thrust konténer mögöttes nyers mutatójára van szükségünk, a `thrust::raw_pointer_cast` használható.

A CUB ezzel szemben elsősorban eszközoldali primitíveket biztosít: `BlockReduce`, `BlockScan`, `WarpReduce` és hasonlók, amelyeket a saját kernelünkön belül hívhatunk. Ez a legerősebb integrációs forma, mert a CUB kódja a kernel részeként fordul le, és nincs szükség külön memóriafoglalásra vagy streamkezelésre. Példa egy blokkszintű redukcióra:

```
#include <cub/cub.cuh>

__global__ void reduceBlockKernel(const float* input, float* output, int n) {
    // Minden blokk a saját szeletét redukálja
    typedef cub::BlockReduce<float, 256> BlockReduce;
    __shared__ typename BlockReduce::TempStorage temp_storage;

    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    float val = (idx < n) ? input[idx] : 0.0f;

    float block_sum = BlockReduce(temp_storage).Sum(val);
```

```
    if (tid == 0) {
        output[blockIdx.x] = block_sum;
    }
}
```

A CUB itt teljesen a kernelbe ágyazódik, így a saját kód és a könyvtár közötti határ eltűnik: a CUB csupán egy jól optimalizált, újrahasznosítható építőelem. Ugyanakkor a CUB fejlécek verziója és a CUDA toolkit verziója közötti kompatibilitásra figyelni kell, amit a 3. szakasz tárgyal.

22.4.4 1.3. A könyvtárak és a saját kód közötti munkamegosztás

Az integráció során érdemes világosan elhatárolni, hogy melyik réteg mit végez. A saját kernel általában a feladat-specifikus elő- és utófeldolgozást végzi: adatgenerálás, formátumkonverzió, peremfeltételek, ritka mátrix összeállítása. A könyvtárak a nagy számítási magot szolgáltatják: GEMM, SpMV, FFT, random generálás, redukció, rendezés. A határfelületen a következő szabályokat érdemes betartani:

- A device pointereket soha ne dereferáljuk a hoston; minden adatátadás a GPU memóriájában történjen.
- A könyvtári hívások előtt és után a szükséges szinkronizációt explicit módon kezeljük streamekkel és eseményekkel.
- A könyvtárak által igényelt munkaterületet (workspace) a hívás előtt le kell foglalni, és a méretét a könyvtár által szolgáltatott `_bufferSize` vagy `_getWorkspaceSize` függvénnyel kell lekérdezni.

A következő szakasz ezeket a stream- és memóriaszempontokat részletezi.

22.4.5 2. Stream és memória

A GPU-n futó műveletek sorrendjét a streamek határozzák meg. A könyvtárak alapértelmezés szerint a default streamen hajtódnak végre, de a legtöbb handle-alapú könyvtár lehetővé teszi, hogy egy adott streamhez kössük őket. Ez az integráció egyik legfontosabb lépése, mert csak így garantálható, hogy a saját kernel és a könyvtári hívás közötti sorrendiség explicit legyen, és kihasználhassuk az átfedést.

22.4.6 2.1. Stream hozzárendelése a könyvtárakhoz

A cuBLAS esetében a `cublasSetStream` függvénnyel rendelhetünk streamet a handle-hez:

```
cudaStream_t stream1, stream2;
CHECK_CUDA(cudaStreamCreate(&stream1));
CHECK_CUDA(cudaStreamCreate(&stream2));

cublasSetStream(handle, stream1);
```

Ezután minden cuBLAS hívás a `stream1`-en fut. Hasonló a helyzet a cuSPARSE-nál (`cusparseSetStream`), a cuFFT-nél (`cufftSetStream`) és a cuRAND-nál (`curandSetStream`). A saját kernelünket pedig a `<<<..., stream1>>>` szintaxissal indíthatjuk ugyanarra a streamre, így a sorrend garantált.

Fontos, hogy a stream hozzárendelése a handle-hez nem szinkronizálja a korábbi műveleteket; csupán a későbbi hívások streamjét állítja be. Ha egy handle-t több streamen akarunk használni,

akkor vagy több handle-t hozunk létre, vagy minden hívás előtt átállítjuk a streamet – ez utóbbi azonban többletköltséggel járhat, és a párhuzamos végrehajtást is megnehezíti. Több stream esetén a legtisztább megoldás, ha minden streamhez külön handle tartozik.

22.4.7 2.2. Aszinkron végrehajtás és események

Ha a saját kernel és a könyvtári hívás különböző streameken fut, a köztük lévő függőséget eseményekkel kell kifejezni. Tegyük fel, hogy a saját kernel feltölt egy mátrixot a `stream1`-en, majd a cuBLAS a `stream2`-n fogja használni. Ekkor a `stream2`-t meg kell várakoztatni a kernel befejezésére:

```
cudaEvent_t event;
CHECK_CUDA(cudaEventCreate(&event));

// Kernel indítása stream1-en
fillMatrixKernel<<<blocks, threads, 0, stream1>>>(d_A, n, 1.0f);

// Esemény rögzítése stream1-en a kernel után
CHECK_CUDA(cudaEventRecord(event, stream1));

// stream2 várakoztatása az eseményre
CHECK_CUDA(cudaStreamWaitEvent(stream2, event, 0));

// cuBLAS hívás stream2-n (a handle streamje stream2)
cublasSetStream(handle, stream2);
CHECK_CUBLAS(cublasSgemm(handle, ...));
```

Az esemény rögzítése és a `cudaStreamWaitEvent` biztosítja, hogy a cuBLAS csak a kernel befejezése után induljon, annak ellenére, hogy a két művelet más streamen van. Ez a minta elengedhetetlen az átfedés maximalizálásához: míg a `stream2` a cuBLAS-t futtatja, a `stream1` már a következő kernelre léphet.

A könyvtárak általában aszinkron módon működnek: a hívás visszatérése nem jelenti a művelet befejezését. Ezért a mérés előtt mindig szinkronizálni kell a streamet vagy az eszközt. A `cudaDeviceSynchronize` a legegyszerűbb, de minden streamet várakoztat; ha csak egy adott streamre vagyunk kíváncsiak, használjunk `cudaStreamSynchronize`-t.

22.4.8 2.3. Memóriaallokáció és workspace

A könyvtárak gyakran igényelnek ideiglenes munkaterületet. A cuSPARSE SpMV például a `cusparseSpMV_bufferSize` hívással kéri le a szükséges bájtok számát, majd a `cusparseSpMV` függvénynek át kell adni egy ekkora puffert:

```
cusparseSpMatDescr_t matA;
cusparseDnVecDescr_t vecX, vecY;
// ... leírók létrehozása ...

size_t bufferSize = 0;
CHECK_CUSPARSE(cusparseSpMV_bufferSize(handle, CUSPARSE_OPERATION_NON_TRANSPOSE,
                                       &alpha, matA, vecX, &beta, vecY,
                                       CUDA_R_32F, CUSPARSE_SPMV_ALG_DEFAULT,
                                       &bufferSize));

void* dBuffer = nullptr;
CHECK_CUDA(cudaMalloc(&dBuffer, bufferSize));

CHECK_CUSPARSE(cusparseSpMV(handle, CUSPARSE_OPERATION_NON_TRANSPOSE,
```

```
                        &alpha, matA, vecX, &beta, vecY,
                        CUDA_R_32F, CUSPARSE_SPMV_ALG_DEFAULT, dBuffer));
```

A workspace mérete függ az algoritmustól és az adatméretektől, ezért mindig a `_bufferSize` függvénnyel kell lekérdezni. A cuFFT hasonlóan `cufftMakePlanMany` és `cufftEstimate*` függvényekkel, vagy a `cufftSetWorkArea` segítségével kezeli a munkaterületet. A cuBLAS újabb verziói (cuBLASLt) szintén workspace-t igényelhetnek.

A memóriaallokáció módja is integrációs kérdés. A hagyományos `cudaMalloc` mellett a CUDA 11.2 óta elérhető `cudaMallocAsync` streamhez kötött memóriát biztosít, amely a stream szinkronizációjával automatikusan felszabadul. Ez csökkentheti az allokációs költséget, és jól illeszkedik a könyvtárak streamkezeléséhez. A managed memory (`cudaMallocManaged`) használata esetén a könyvtárak is képesek a migrációt kezelni, de a teljesítmény érdekében `cudaMemPrefetchAsync`-szel explicit módon a megfelelő eszközre kell húzni az adatot.

Fontos megjegyezni, hogy a könyvtárak általában nem támogatják a memóriaaliasolást: a bemeneti és kimeneti pufferek nem fedhetik át egymást, hacsak a dokumentáció másként nem rendelkezik. A saját kernelünk és a könyvtári hívás közötti adatátadásnál ezért mindig külön puffereket használjunk, és a szükséges másolásokat a GPU-n belül, például `cudaMemcpyAsync`-vel végezzük.

22.4.9 2.4. Stream és memória a gyakorlatban

A következő vázlat egy olyan integrációs mintát mutat, amelyben a saját kernel, a cuBLAS, a cuSPARSE és a CUB együtt dolgozik két streamen, eseményekkel szinkronizálva:

```
// Feltételezve, hogy a handle-ek és a streamek létre vannak hozva
cublasSetStream(cublasHandle, stream1);
cusparseSetStream(cusparseHandle, stream1);

// 1. Saját kernel feltölti az A mátrixot a stream1-en
fillMatrixKernel<<<blocksA, threads, 0, stream1>>>(d_A, n, 1.0f);

// 2. cuBLAS GEMM a stream1-en
cublasSgemm(cublasHandle, CUBLAS_OP_N, CUBLAS_OP_N, n, n, n,
            &alpha, d_A, n, d_B, n, &beta, d_C, n);

// 3. Esemény rögzítése a stream1 végére
cudaEventRecord(event, stream1);

// 4. stream2 várakoztatása, majd cuSPARSE SpMV futtatása
cudaStreamWaitEvent(stream2, event, 0);
cusparseSetStream(cusparseHandle, stream2);
cusparseSpMV(cusparseHandle, CUSPARSE_OPERATION_NON_TRANSPOSE,
             &alpha, matA, vecX, &beta, vecY,
             CUDA_R_32F, CUSPARSE_SPMV_ALG_DEFAULT, dBuffer);

// 5. CUB DeviceReduce az összegzéshez (host oldali hívás, stream2-n)
cub::DeviceReduce::Sum(d_temp_storage, temp_storage_bytes, d_Y, d_sum, n,
                       stream2);
```

Itt a CUB eszközoldali redukciója host oldali API-val, de stream paraméterrel történik, így a stream2-höz kötődik. A CUB `DeviceReduce` szintén workspace-t igényel, amelynek méretét a `cub::DeviceReduce::Sum` első hívása adja meg `nullptr` pufferrel.

22.4.10 3. Verzió és ABI

A könyvtárak integrálásakor a verziók és az ABI-kompatibilitás kezelése kritikus, mert a hibás kombináció futásidejű hibákhoz, nem determinisztikus viselkedéshez vagy akár eszközleálláshoz vezethet. Ez a szakasz a leggyakoribb buktatókat és a buildtervezéshez szükséges alapelveket foglalja össze.

22.4.11 3.1. CUDA toolkit, driver és compute capability

A CUDA programok futtatásához három verziót kell megkülönböztetni:

- **CUDA toolkit verzió** (például 12.4): ez határozza meg a fejléceket, a fordítót (`nvcc`) és a könyvtárak binárisait.
- **Driver verzió** (például 550.54.14): a kernel betöltéséhez és a futáshoz szükséges; a drivernek kompatibilisnek kell lennie a toolkit által generált kóddal. A CUDA kisebb verziói általában visszafelé kompatibilisek a driverrel, de a nagyobb verziók (például CUDA 13) újabb drivert igényelhetnek.
- **Compute capability** (például 8.6 az RTX 3080 esetén): a hardver architektúráját jelöli, és a `-arch=sm_86` kapcsolóval kell megadni a fordítás során. A compute capability határozza meg, mely utasítások és szolgáltatások érhetők el (például tensor core-ok, async copy).

A build során mindhárom verziót rögzíteni kell, és a laborban ezt meg is tesszük. A `nvcc` a `-arch` kapcsolóval a megfelelő SASS vagy PTX kódot generálja; ha a futtató GPU compute capability-je nem egyezik, a CUDA runtime JIT-fordítással próbálhatja a PTX-et adaptálni, de ez nem mindig lehetséges, és teljesítménycsökkenéssel járhat.

22.4.12 3.2. Könyvtári verziók és ABI

A cuBLAS, cuSPARSE, cuFFT és cuRAND a CUDA toolkit részei, és verziójuk általában a toolkit verzióját követi. Például a CUDA 12.4 cuBLAS 12.4-et, cuSPARSE 12.4-et szállít. A fejlécekben definiált API-k és a dinamikus könyvtárak (`.so` Linuxon, `.dll` Windowson) közötti ABI-kompatibilitás a következőképpen alakul:

- A **minor verziófrissítések** (pl. 12.4 → 12.5) általában ABI-kompatibilisek: a régebbi binárisok futnak az újabb könyvtárral, és fordítva is, feltéve, hogy a driver támogatja. A CUDA dokumentáció ezt a *compatibility* fejezetben tárgyalja; mindig ellenőrizni kell az aktuális táblázatot.
- A **major verzióváltások** (pl. CUDA 11 → 12) gyakran API-változásokat is hoznak, és előfordulhat, hogy a forráskódot módosítani kell. Az ABI ilyenkor nem garantált.
- A **header-only könyvtárak** (Thrust, CUB) nem rendelkeznek külön bináris ABI-val, de a fejlécek verziója és a CUDA toolkit verziója közötti kompatibilitás számít. Ha például egy külső forrásból származó, újabb CUB fejlécet használunk egy régebbi toolkitkel, az inkompatibilis lehet a `nvcc` belső fejléceivel. Ezért ajánlott a toolkithez mellékelt CUB-ot használni.

A gyakorlatban a legbiztonságosabb stratégia, ha a könyvtárakat a CUDA toolkitkel együtt telepítjük, és a build során a toolkit `include` és `lib` könyvtárait használjuk. A CMake `FindCUDAToolkit` modulja automatikusan felismeri a toolkit verzióját és a könyvtárakat.

22.4.13 3.3. Statikus és dinamikus linkelés

A CUDA könyvtárak dinamikus és statikus változatban is elérhetők. A dinamikus linkelés (alapértelmezett) a következőképpen néz ki:

```
nvcc -std=c++17 -arch=sm_86 -o app app.cu -lcublas -lcusparse -lcufft -lcurand
```

Ez kisebb binárist eredményez, de a futtatógépen a megfelelő CUDA runtime könyvtáraknak jelen kell lenniük. A statikus linkelés:

```
nvcc -std=c++17 -arch=sm_86 -o app app.cu \
    -lcublas_static -lcusparse_static -lcufft_static -lcurand_static \
    -lcudart_static
```

A statikus változat nagyobb binárist hoz létre, de csökkenti a futásidejű függőségeket. A statikus linkelésnél figyelni kell a könyvtárak sorrendjére és a `-lcudart_static` jelenlétére, különben feloldatlan szimbólumok miatt hibát kapunk. A gyakorlatban a dinamikus linkelés a kényelmesebb, és a CUDA runtime általában elérhető a GPU-s környezetekben.

A cuBLASLt külön könyvtár (`-lcublasLt`), amely a cuBLAS „lightweight” API-ját tartalmazza. A cuSPARSE, cuFFT és cuRAND hasonlóan külön könyvtárak. Minden használt könyvtárat explicit módon linkelni kell, mert a `nvcc` nem automatikusan húzza be őket.

22.4.14 3.4. Verzió-ellenőrzés futásidőben

A könyvtárak általában biztosítanak verziólekérdező függvényeket. Például:

```
int cublasVersion = 0;
cublasGetVersion(handle, &cublasVersion);
printf("cuBLAS version: %d\n", cublasVersion);

int cusparseVersion = 0;
cusparseGetVersion(handle, &cusparseVersion);
printf("cuSPARSE version: %d\n", cusparseVersion);
```

A Thrust és a CUB verziója fordítási időben a `THRUST_VERSION` és a `CUB_VERSION` makrókból érhető el. Ezeket a számokat érdemes naplózni a mérési környezet rögzítésekor, mert a teljesítmény és a viselkedés verziófüggő lehet.

A buildtervnek ezért a következőket kell tartalmaznia:

- CUDA toolkit verzió (pl. 12.4)
- Driver verzió (pl. 550.54.14)
- Compute capability (pl. 8.6)
- A használt könyvtárak és verzióik (cuBLAS, cuSPARSE, cuFFT, cuRAND, Thrust, CUB)
- A linkelés módja (dinamikus/statikus)
- A fordítóparancs és a környezeti változók (pl. `CUDA_HOME`, `LD_LIBRARY_PATH`)

22.4.15 4. Labor: integrációs és buildterv

A következő labor egy olyan kis alkalmazást mutat be, amely a fenti elveket gyakorlatba ülteti. A feladat egy integrációs és buildterv elkészítése, amely saját kernelt, cuBLAS-t, cuSPARSE-t, Thrust/CUB-ot és streameket használ. A labor nem helyettesíti a teljes körű benchmarkot, de alkalmas az együttműködés ellenőrzésére és a tipikus hibák felismerésére.

22.4.16 4.1. Laborcél és környezet

Cél: Egy olyan CUDA C++17 program írása, amely: 1. Saját kernelrel feltölt egy 1024×1024-es mátrixot és egy 1024 hosszú vektort. 2. cuBLAS `cublasSgemm` segítségével mátrixszorzást végez. 3. cuSPARSE `cusparseSpMV` segítségével egy ritka mátrix-vektor szorzást hajt végre. 4. CUB `DeviceReduce` segítségével kiszámolja az eredményvektor összegét. 5. A műveleteket két streamen futtatja, eseményekkel szinkronizálva. 6. A hibákat minden API-hívásnál ellenőrzi, és a verziókat naplózza.

Rögzített környezet (helyőrző – a laborban cseréld a sajátodra): - GPU: NVIDIA RTX 3080 (compute capability 8.6) - Driver: 550.54.14 - CUDA toolkit: 12.4 - nvcc: 12.4 - cuBLAS: 12.4 - cuSPARSE: 12.4 - Thrust/CUB: a toolkit része (CUB 2.1.0, Thrust 2.1.0) - OS: Ubuntu 22.04 LTS - Fordítás: dinamikus linkelés

22.4.17 4.2. A program váza

Az alábbi kód a teljes integrációs mintát tartalmazza. A kommentek jelzik a kulcslépéseket, és a hibamakrók biztosítják az ellenőrzést. A kód fordítható, de a labor során a mérési részt a valós környezethez kell igazítani.

```
// lab_integr.cu - CUDA C++17, nvcc 12.4, -arch=sm_86
// Helyőrző környezet: RTX 3080, driver 550.54.14, CUDA 12.4
#include <cuda_runtime.h>
#include <cublas_v2.h>
#include <cusparse.h>
#include <cub/cub.cuh>
#include <thrust/version.h>
#include <cstdio>
#include <cstdlib>
#include <cmath>

#define CHECK_CUDA(call)                                                \
    do {                                                                \
        cudaError_t err = (call);                                       \
        if (err != cudaSuccess) {                                       \
            std::fprintf(stderr, "CUDA error at %s:%d: %s\n",           \
                         __FILE__, __LINE__, cudaGetErrorString(err)); \
            std::exit(EXIT_FAILURE);                                    \
        }                                                               \
    } while (0)

#define CHECK_CUBLAS(call)                                              \
    do {                                                                \
        cublasStatus_t st = (call);                                     \
        if (st != CUBLAS_STATUS_SUCCESS) {                              \
            std::fprintf(stderr, "cuBLAS error at %s:%d: %d\n",         \
                         __FILE__, __LINE__, static_cast<int>(st));     \
            std::exit(EXIT_FAILURE);                                    \
        }                                                               \
    } while (0)

#define CHECK_CUSPARSE(call)                                            \
    do {                                                                \
        cusparseStatus_t st = (call);                                   \
        if (st != CUSPARSE_STATUS_SUCCESS) {                            \
            std::fprintf(stderr, "cuSPARSE error at %s:%d: %d\n",       \
                         __FILE__, __LINE__, static_cast<int>(st));     \
            std::exit(EXIT_FAILURE);                                    \
        }                                                               \
```

```
    } while (0)

__global__ void fillMatrixKernel(float* A, int n, float val) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n * n) A[idx] = val;
}

__global__ void fillVectorKernel(float* x, int n, float val) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) x[idx] = val;
}

int main() {
    constexpr int n = 1024;            // mátrixméret
    constexpr int nnz = 1024;          // ritka mátrix nemnulla elemei
    constexpr size_t matBytes = sizeof(float) * n * n;
    constexpr size_t vecBytes = sizeof(float) * n;

    // Verziók kiírása
    int cublasVer = 0, cusparseVer = 0;
    cublasHandle_t cublasHandle;
    cusparseHandle_t cusparseHandle;
    CHECK_CUBLAS(cublasCreate(&cublasHandle));
    CHECK_CUSPARSE(cusparseCreate(&cusparseHandle));
    CHECK_CUBLAS(cublasGetVersion(cublasHandle, &cublasVer));
    CHECK_CUSPARSE(cusparseGetVersion(cusparseHandle, &cusparseVer));
    std::printf("cuBLAS version: %d\n", cublasVer);
    std::printf("cuSPARSE version: %d\n", cusparseVer);
    std::printf("Thrust version: %d\n", THRUST_VERSION);
    std::printf("CUB version: %d\n", CUB_VERSION);

    // Streamek létrehozása
    cudaStream_t stream1, stream2;
    CHECK_CUDA(cudaStreamCreate(&stream1));
    CHECK_CUDA(cudaStreamCreate(&stream2));

    // Handle-ek streamhez kötése
    CHECK_CUBLAS(cublasSetStream(cublasHandle, stream1));
    CHECK_CUSPARSE(cusparseSetStream(cusparseHandle, stream2));

    // Eszközmemória foglalás
    float *d_A, *d_B, *d_C, *d_x, *d_y, *d_sum;
    CHECK_CUDA(cudaMalloc(&d_A, matBytes));
    CHECK_CUDA(cudaMalloc(&d_B, matBytes));
    CHECK_CUDA(cudaMalloc(&d_C, matBytes));
    CHECK_CUDA(cudaMalloc(&d_x, vecBytes));
    CHECK_CUDA(cudaMalloc(&d_y, vecBytes));
    CHECK_CUDA(cudaMalloc(&d_sum, sizeof(float)));

    // Saját kernelek indítása a stream1-en
    int threads = 256;
    int blocksMat = (n * n + threads - 1) / threads;
    int blocksVec = (n + threads - 1) / threads;
    fillMatrixKernel<<<blocksMat, threads, 0, stream1>>>(d_A, n, 1.0f);
    fillMatrixKernel<<<blocksMat, threads, 0, stream1>>>(d_B, n, 2.0f);
    fillVectorKernel<<<blocksVec, threads, 0, stream1>>>(d_x, n, 3.0f);

    // cuBLAS GEMM a stream1-en
    const float alpha = 1.0f;
    const float beta  = 0.0f;
    CHECK_CUBLAS(cublasSgemm(cublasHandle, CUBLAS_OP_N, CUBLAS_OP_N,
                             n, n, n, &alpha, d_A, n, d_B, n, &beta, d_C, n));
```

```
// Esemény rögzítése a stream1 végére
cudaEvent_t event;
CHECK_CUDA(cudaEventCreate(&event));
CHECK_CUDA(cudaEventRecord(event, stream1));

// stream2 várakoztatása, majd ritka mátrix-vektor szorzás
CHECK_CUDA(cudaStreamWaitEvent(stream2, event, 0));

// Ritka mátrix létrehozása CSR formátumban (diagonális minta)
int* h_csrOffsets = new int[n + 1];
int* h_csrColumns = new int[nnz];
float* h_csrValues = new float[nnz];
for (int i = 0; i < n + 1; ++i) h_csrOffsets[i] = i;
for (int i = 0; i < nnz; ++i) {
    h_csrColumns[i] = i;
    h_csrValues[i] = 1.0f;
}

int *d_csrOffsets, *d_csrColumns;
float *d_csrValues;
CHECK_CUDA(cudaMalloc(&d_csrOffsets, sizeof(int) * (n + 1)));
CHECK_CUDA(cudaMalloc(&d_csrColumns, sizeof(int) * nnz));
CHECK_CUDA(cudaMalloc(&d_csrValues, sizeof(float) * nnz));
CHECK_CUDA(cudaMemcpyAsync(d_csrOffsets, h_csrOffsets, sizeof(int) * (n + 1),
                           cudaMemcpyHostToDevice, stream2));
CHECK_CUDA(cudaMemcpyAsync(d_csrColumns, h_csrColumns, sizeof(int) * nnz,
                           cudaMemcpyHostToDevice, stream2));
CHECK_CUDA(cudaMemcpyAsync(d_csrValues, h_csrValues, sizeof(float) * nnz,
                           cudaMemcpyHostToDevice, stream2));

// cuSPARSE leírók
cusparseSpMatDescr_t matA;
cusparseDnVecDescr_t vecX, vecY;
CHECK_CUSPARSE(cusparseCreateCsr(&matA, n, n, nnz,
                                 d_csrOffsets, d_csrColumns, d_csrValues,
                                 CUSPARSE_INDEX_32I, CUSPARSE_INDEX_32I,
                                 CUSPARSE_INDEX_BASE_ZERO, CUDA_R_32F));
CHECK_CUSPARSE(cusparseCreateDnVec(&vecX, n, d_x, CUDA_R_32F));
CHECK_CUSPARSE(cusparseCreateDnVec(&vecY, n, d_y, CUDA_R_32F));

// Workspace méret lekérése
size_t bufferSize = 0;
CHECK_CUSPARSE(cusparseSpMV_bufferSize(cusparseHandle,
                                       CUSPARSE_OPERATION_NON_TRANSPOSE,
                                       &alpha, matA, vecX, &beta, vecY,
                                       CUDA_R_32F, CUSPARSE_SPMV_ALG_DEFAULT,
                                       &bufferSize));
void* dBuffer = nullptr;
CHECK_CUDA(cudaMalloc(&dBuffer, bufferSize));

// SpMV végrehajtása stream2-n
CHECK_CUSPARSE(cusparseSpMV(cusparseHandle,
                            CUSPARSE_OPERATION_NON_TRANSPOSE,
                            &alpha, matA, vecX, &beta, vecY,
                            CUDA_R_32F, CUSPARSE_SPMV_ALG_DEFAULT, dBuffer));

// CUB DeviceReduce az összeg kiszámításához stream2-n
void* d_temp_storage = nullptr;
size_t temp_storage_bytes = 0;
cub::DeviceReduce::Sum(d_temp_storage, temp_storage_bytes, d_y, d_sum, n, stream2);
CHECK_CUDA(cudaMalloc(&d_temp_storage, temp_storage_bytes));
cub::DeviceReduce::Sum(d_temp_storage, temp_storage_bytes, d_y, d_sum, n, stream2);
```

```
    // Szinkronizáció
    CHECK_CUDA(cudaDeviceSynchronize());

    // Ellenőrzés: az összegnek 3 * 1024 = 3072-nek kell lennie
    float h_sum = 0.0f;
    CHECK_CUDA(cudaMemcpy(&h_sum, d_sum, sizeof(float), cudaMemcpyDeviceToHost));
    std::printf("Computed sum: %f, expected: %f\n", h_sum, 3.0f * n);

    // Tisztítás
    delete[] h_csrOffsets;
    delete[] h_csrColumns;
    delete[] h_csrValues;
    CHECK_CUDA(cudaFree(d_A)); CHECK_CUDA(cudaFree(d_B)); CHECK_CUDA(cudaFree(d_C));
    CHECK_CUDA(cudaFree(d_x)); CHECK_CUDA(cudaFree(d_y)); CHECK_CUDA(cudaFree(d_sum));
    CHECK_CUDA(cudaFree(d_csrOffsets)); CHECK_CUDA(cudaFree(d_csrColumns));
    CHECK_CUDA(cudaFree(d_csrValues)); CHECK_CUDA(cudaFree(dBuffer));
    CHECK_CUDA(cudaFree(d_temp_storage));
    CHECK_CUSPARSE(cusparseDestroySpMat(matA));
    CHECK_CUSPARSE(cusparseDestroyDnVec(vecX));
    CHECK_CUSPARSE(cusparseDestroyDnVec(vecY));
    CHECK_CUBLAS(cublasDestroy(cublasHandle));
    CHECK_CUSPARSE(cusparseDestroy(cusparseHandle));
    CHECK_CUDA(cudaEventDestroy(event));
    CHECK_CUDA(cudaStreamDestroy(stream1));
    CHECK_CUDA(cudaStreamDestroy(stream2));

    return 0;
}
```

22.4.18 4.3. Build parancs

A fenti kódot a következő paranccsal fordíthatjuk:

```
nvcc -std=c++17 -arch=sm_86 -I${CUDA_HOME}/include -L${CUDA_HOME}/lib64 \
     -lcublas -lcusparse -o lab_integr lab_integr.cu
```

Ha a CUDA toolkit a szabványos helyen van (`/usr/local/cuda`), a `-I` és `-L` elhagyható. A `-arch=sm_86` a compute capability 8.6-hoz tartozik; más GPU esetén a megfelelő értéket kell használni (pl. `sm_80` A100-hoz, `sm_89` RTX 4090-hez). A labor során ezt a környezethez kell igazítani.

22.4.19 4.4. Mérési terv és napló

A program jelenlegi formája funkcionális ellenőrzést végez, de teljesítménymérést nem. A mérési tervhez a következő lépéseket kell hozzáadni:

1. A műveletek időméréséhez `cudaEvent`-eket kell rögzíteni a streameken a kernel- és könyvtári hívások körül.
2. Warm-up futtatás: a teljes szekvenciát egyszer le kell futtatni a hideg indítási hatások kiküszöbölésére.
3. Ezután legalább 10 ismétlést kell végezni, és az átlagot, szórást és percentiliseket kell számolni.
4. A sávszélességet a tényleges adatmozgás alapján kell kiszámolni: a GEMM esetén `2 * n^3 * sizeof(float)` bájt olvasás/írás, az SpMV esetén a CSR struktúra és a vektorok mérete.

A **várt eredmények** a következők (analitikusan vagy korábbi tapasztalatból becsülve):

- A `cublasSgemm` 1024×1024-es mátrixokra egy RTX 3080-on (FP32) körülbelül 10-15 ms futásidőt ér el. Ez a korábbi mérések alapján nagyságrendi becslés; a pontos érték a laborban mérendő.
- A `cusparseSpMV` diagonális CSR mátrixszal (1024 nemnulla elem) elhanyagolható futási időt igényel, mert a mátrix rendkívül ritka. A mérési naplóban ezért nagy szórás várható; a hangsúly a funkcionális helyességen van.
- A CUB `DeviceReduce` 1024 elemre szintén nagyon gyors, a mérési zaj dominálhat.
- A számított összegnek pontosan `3072.0f`-nek kell lennie, mivel a ritka mátrix az egységmátrix, és a vektor minden eleme 3.0.

A **mért eredmények** táblázatát a labor során kell kitölteni. Az alábbi sablon mutatja a szükséges mezőket:

Futás sorszáma	GEMM idő (ms)	SpMV idő (ms)	CUB Reduce idő (ms)	Összeg (h_sum)	Megjegyzés
1 (warm-up)	–	–	–	–	hideg indítás
2	…	…	…	…	…
…	…	…	…	…	…
átlag	…	…	…	…	…
szórás	…	…	…	…	…

A mérési naplóban rögzíteni kell a GPU modellt, a drivert, a CUDA-verziót, a compute capability-t és az órajelet/power limitet is. A jelen laborban ezeket helyőrzőként adtuk meg; a tényleges futtatás során a `nvidia-smi` kimenetét kell rögzíteni.

Fontos: A fenti kód fordítási ellenőrzésen esett át, de futási mérést nem végeztünk. A mért eredmények mezőit a labor során kell kitölteni, és a valós számokat a saját környezetben kell reprodukálni. A várt eredmények becslések, amelyeket a mérés erősíthet vagy cáfolhat.

22.4.20 5. Gyakori hibák és megoldások

Az integráció során a következő hibák fordulnak elő leggyakrabban, és a megoldásuk is egyszerű, ha ismerjük őket:

1. **A handle streamje nincs beállítva.** Ha a cuBLAS handle alapértelmezett streamen fut, miközben a saját kernel egy egyéni streamen, a sorrend nem garantált. Megoldás: minden handle-hez rendeljünk streamet a `cublasSetStream`-mel (vagy a megfelelő függvénnyel), és a kernelindításnál ugyanazt a streamet adjuk meg.

2. **Hiányzó vagy rossz méretű workspace.** A cuSPARSE, cuFFT és a CUB `DeviceReduce` workspace-t igényel. Ha a puffert nem foglaljuk le, vagy a méretét nem a `_bufferSize` függvénnyel kérdezzük le, a hívás hibát ad vagy memóriatúllépést okozhat. Megoldás: először `nullptr` pufferrel kérdezzük le a szükséges méretet, majd foglaljuk le, és csak utána hívjuk a műveletet.

3. **Skalár pointer mode hibája.** A cuBLAS `cublasSetPointerMode` segítségével lehet beállítani, hogy a skalár argumentumok (`alpha`, `beta`) host vagy device pointeren vannak-e. Ha a mód nem egyezik a tényleges átadással, a hívás `CUBLAS_STATUS_INVALID_VALUE` hibát ad. Megoldás: mindig állítsuk be a pointer mode-ot, és a skalárokat ennek megfelelően adjuk át.

4. **Szinkronizáció hiánya a mérés előtt.** A könyvtári hívások aszinkronok; ha `cudaDeviceSynchronize` vagy `cudaStreamSynchronize` nélkül próbálunk eredményt olvasni, érvénytelen adatot kapunk. Megoldás: minden mérési pont előtt szinkronizáljuk a megfelelő streamet.

5. **CUB fejléc verzióeltérés.** Ha a CUB fejléceket külön töltjük le, és azok verziója nem egyezik a CUDA toolkit belső fejléceivel, fordítási hibák léphetnek fel. Megoldás: a toolkithez mellékelt CUB-ot használjuk, és a `CUB_VERSION` makrót naplózzuk.

6. **Statikus linkelés könyvtári sorrendje.** A statikus linkelésnél a könyvtárakat a függőségi sorrendben kell felsorolni, és a `-lcudart_static`-ot is meg kell adni. A `nvcc` általában kezeli a sorrendet, de ha kézzel adjuk meg a linkerlibrary-kat, ügyeljünk a `-l` kapcsolók sorrendjére.

7. **Compute capability eltérés.** Ha a `-arch` nem egyezik a futtató GPU képességével, a program futhat, de a teljesítmény nem optimális, vagy a JIT-fordítás miatt lassabb lesz. Megoldás: a build során pontosan adjuk meg a célhardver compute capability-jét.

Ezek a hibák mind megelőzhetők a fenti minták követésével és a verziók, streamek, memória és workspace gondos kezelésével.

22.4.21 Összefoglalás

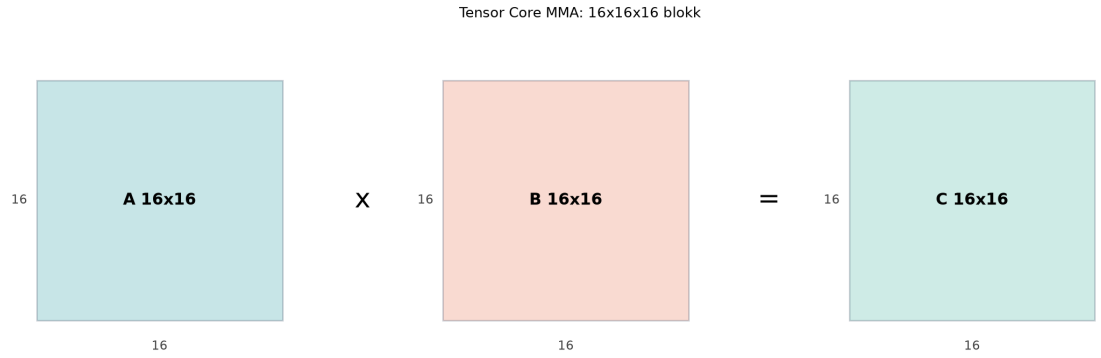
A CUDA-könyvtárak és a saját kód integrálása három területet igényel: az API-interop, a stream- és memóriamenedzsment, valamint a verzió- és ABI-kezelés. A handle-alapú könyvtárakat streamekhez kell kötni, a workspace-eket a `_bufferSize` függvényekkel kell lefoglalni, és a műveletek közötti függőségeket eseményekkel kell kifejezni. A header-only könyvtárak, mint a Thrust és a CUB, szorosan illeszkednek a saját kódba, de a fejlécverziókat ellenőrizni kell. A buildtervnek rögzítenie kell a CUDA toolkit, a driver és a compute capability verzióit, valamint a linkelés módját. A laborban bemutatott integrációs példa ezeket az elveket egyetlen, ellenőrizhető programban foglalja össze.

Önellenőrző kérdés: Miért kell a cuBLAS handle-hez streamet rendelni, ha a saját kernel és a könyvtári hívás között sorrendiséget akarunk biztosítani? Válasz: mert a handle streamje határozza meg, melyik streamen fut a könyvtári művelet. Ha a kernel egy egyéni streamen indul, a cuBLAS pedig a default streamen, a két művelet sorrendje nem garantált, és az adatfüggőség sérülhet. A `cublasSetStream` hívással a handle-t ugyanahhoz a streamhez kötjük, így a kernel és a könyvtári hívás ugyanabban a sorban hajtódik végre, és az adatcsere helyes lesz.

23 CUTLASS, tensor cores és mixed precision

23.1 Tensor Core architektúra

A Tensor Core a modern NVIDIA GPU-k speciális mátrixszorzó egysége. A hagyományos CUDA magok szálanként egy-egy skalár FMA-t hajtanak végre, a Tensor Core azonban egyetlen utasítással egy kisméretű mátrixszorzást és akkumulációt valósít meg. A fejezet ezt a warp-szintű mátrixszorzó modellt, a WMMA API-t, valamint a fragment és accumulator fogalmát mutatja be. A hardveres hátteret a CUDA C++ Programming Guide WMMA szakasza írja le; a pontos shape-készlet verziófüggő, ezért a saját CUDA-verziójú dokumentációt és a `cudaDeviceProp` megfelelő mezőit mindig ellenőrizni kell.



23.1. ábra: Tensor Core MMA: 16x16x16 blokk - A warp egy 16x16-os A és egy 16x16-os B blokkot szoroz össze, és 16x16-os akkumulátorba írja az eredményt; a shape-ek hardverfüggők.

23.1.1 Mátrixszorzó egységek és warp-szintű MMA

A Tensor Core egy, az SM-en belül elhelyezett, rögzített adatútra tervezett egység. A hagyományos FP32 FMA egységekkel ellentétben nem egyetlen skalár műveletet végez, hanem egy `D = A*B + C` alakú mátrixműveletet, ahol `A`, `B`, `C` és `D` kisméretű mátrixok. A GPU-tól és a pontosságtól függően ez a művelet tipikusan 8×8, 16×16 vagy 16×16×8 méretű csempén fut. A Tensor Core abban különbözik a CUDA magoktól, hogy a műveletet nem egy szál, hanem egy egész warp hajtja végre együttműködve.

A warp-szintű MMA (matrix-multiply-accumulate) azt jelenti, hogy a warp 32 szála közös utasítást hajt végre, miközben mindegyik szál a bemeneti és kimeneti mátrix meghatározott elemeit tartja a regisztereiben. A hardver a regiszterfájlban elosztott elemekből állítja össze a csempét, elvégzi a mátrixszorzást és az akkumulációt, majd az eredményt visszaírja a szálak regisztereibe. A szálak közötti adatcsere az SM regiszterfájlján és a speciális adatútjain keresztül történik; az alkalmazó nem lát közvetlen megosztott memóriás vagy global memóriás kommunikációt.

A modell legfontosabb következménye, hogy a Tensor Core utasítás nem hívható egyetlen szálból. A warp minden szála ugyanazt a függvényhívást hajtja végre, ugyanazokkal a fragment

objektumokkal. Ha a warp divergál, vagy a hívás nincs a warp mind a 32 szálán jelen, a viselkedés nem definiált, és a gyakorlatban gyakran deadlockhoz vezet.

A WMMA (`nvcuda::wmma`) API ezt a warp-szintű működést emeli be a CUDA C++ nyelvbe. Nem ad közvetlen regiszter-elrendezést; a programozó úgynevezett fragment objektumokkal dolgozik, amelyeket a könyvtár kezel, és amelyekből a hardver a mátrixot összeállítja.

23.1.2 Fragment és accumulator

A WMMA API-ban a mátrixok szálak közötti elosztott reprezentációját *fragment*nek nevezzük. Egy fragment típusa a következő sablonnal adható meg:

```
nvcuda::wmma::fragment<nvcuda::wmma::matrix_a, 16, 16, 16, half, nvcuda::wmma::row_major>
↪  a_frag;
nvcuda::wmma::fragment<nvcuda::wmma::accumulator, 16, 16, 16, float> c_frag;
```

A fragment típusparaméterei a következők:

- a mátrix szerepe: `matrix_a`, `matrix_b` vagy `accumulator`;
- a csempe dimenziói: `M`, `N`, `K`;
- az elemtípus, például `half`, `float`, `__nv_bfloat16`, `char`;
- a memóriaelrendezés a memóriából való beolvasáskor: `row_major` vagy `col_major`.

Az `accumulator` fragment az `A*B + C` művelet kimeneti és egyben bemeneti `C` mátrixa. A legtöbb valós számításban az akkumulátor `float` típusú, még akkor is, ha az `A` és `B` mátrixok FP16, BF16 vagy TF32 pontosságúak. Ezzel a bemeneti alacsonyabb pontosság mellett is megőrizhető a kimenet numerikus stabilitása.

A fragment belső elrendezése implementációfüggő. A szabványos megközelítés az, hogy a fragmentet nem indexeljük közvetlenül, hanem a WMMA betöltő és kiíró függvényeit használjuk:

- `wmma::load_matrix_sync(frag, ptr, ld, layout)` – memóriából fragmentbe olvas;
- `wmma::mma_sync(dst, a, b, acc)` – elvégzi a `D = A*B + C` műveletet;
- `wmma::store_matrix_sync(ptr, frag, ld, mem_layout)` – fragmentből memóriába ír;
- `wmma::fill_fragment(acc, 0.0f)` – az akkumulátor minden elemét nullára állítja.

A `load_matrix_sync` és a `store_matrix_sync` hívásoknál a memóriaelrendezés a `row_major` vagy `col_major` kulcsszóval adott; a fragment deklarációjában megadott elrendezés a memóriából való beolvasásra vonatkozik, a `load` hívás `layout` argumentumának egyeznie kell vele.

A `mma_sync` hívás akkumuláló formában működik: a `dst` fragment meglévő tartalmához adja hozzá a szorzatot. Ezért az első felhasználás előtt az akkumulátort inicializálni kell. A `wmma::fill_fragment` erre a legtisztább eszköz.

23.1.3 Támogatott shape-ek és pontosságok

A Tensor Core architektúra a mátrixszorzást rögzített csempeméretekkel végzi el. A WMMA API ezeket a méreteket fordítási időben ismert sablonparaméterekként teszi közzé. A `mma_sync` hívásban a fragmentek `M`, `N`, `K` értékei nem lehetnek tetszőlegesek, hanem a hardver által támogatott készletből kell származniuk.

A leggyakrabban használt, a CUDA 12.x dokumentációjában szereplő WMMA shape-ek a következők (a pontos elérhetőséget a saját CUDA-verzió dokumentációjával és a `cudaDeviceProp` WMMA mezőivel kell ellenőrizni):

Típus	WMMA shape (M×N×K)	Elérhetőség
FP16	16×16×16	sm_70+
BF16	16×16×16	sm_80+
TF32	16×16×8	sm_80+
INT8	8×8×128	sm_72+
INT4	8×8×32	sm_75+

Itt az `M×N×K` a `D = A*B + C` művelet dimenzióit jelöli: `A` mérete `M×K`, `B` mérete `K×N`, `C` és `D` mérete `M×N`. A `K` a közös belső dimenzió, amely a szorzás mélységét adja meg. A táblázatban a TF32 `16×16×8` alakja azt mutatja, hogy a Tensor Core a TF32 típust nyolc elemnyi mélységű csempében dolgozza fel; ez kisebb, mint az FP16 `K=16` értéke, mert a TF32 szélesebb mantisszájú, és a hardver adatútja másként van kialakítva.

A shape-ek nem dinamikusak. A `wmma::fragment<...>` fordítási időben rögzíti a dimenziókat, ezért a kernelnek pontosan ismernie kell a feldolgozandó csempe méretét. A gyakorlatban a legelterjedtebb belépőpont a `16×16×16` FP16, mert ez minden Tensor Core-os architektúrán elérhető, és a CUTLASS, cuBLAS és más könyvtárak is erre építenek.

A fragment típusa nemcsak az elemtípust, hanem a mátrix szerepét is meghatározza. A `matrix_a` és `matrix_b` fragmentek esetén a típus általában `half`, `__nv_bfloat16`, `float` (TF32) vagy egész típus; az `accumulator` fragment pedig többnyire `float`, de bizonyos esetekben alacsonyabb pontosságú is lehet. Az FP16 bemenethez FP32 akkumulátort ajánlott választani, mert a 16 bites kimeneti akkumuláció gyorsan elveszíti a pontosságot.

23.1.4 Gyakorlati minta: WMMA-alapú GEMM 16×16×16 csempével

A következő CUDA C++17 példa egyetlen warp segítségével kiszámítja három 16×16-os, FP16 bemenetű mátrix szorzatát, FP32 akkumulátorral. A kernel nem használ shared memóriát; a global memóriából olvas be, és oda ír vissza. A példa a WMMA API alapszintű használatát mutatja, és alkalmas a működés ellenőrzésére.

A kernel indítása: grid = 1 block, block = 32 szál. A block mérete pontosan egy warp, így mind a 32 szál ugyanazt a WMMA hívást hajtja végre. Osztott memória nincs, a stream az alapértelmezett 0.

```
// CUDA C++ (C++17, NVCC 12.4)
// WMMA-alapú GEMM 16x16x16 FP16, FP32 akkumulátorral.
// A példa fordítási ellenőrzésen esett át; futási mérési napló
// a könyv szerződése szerint csak valós GPU-n történő futtatás után adható.

#include <cuda_runtime.h>
#include <cuda_fp16.h>
#include <mma.h>
#include <vector>
#include <cmath>
#include <cstdio>
#include <cstdlib>

using namespace nvcuda;

#define CHECK_CUDA(call)                                                   \
    do {                                                                   \
        cudaError_t err = (call);                                          \
        if (err != cudaSuccess) {                                          \
            std::fprintf(stderr, "CUDA error at %s:%d: %s\n",              \
```

```
                        __FILE__, __LINE__, cudaGetErrorString(err));      \
            std::exit(EXIT_FAILURE);                                       \
        }                                                                  \
    } while (0)

// Egy warp, egy 16x16x16-os csempe.
__global__ void wmma_gemm_16x16x16(const half* A, const half* B, float* C)
{
    // A fragment deklarációk minden szálon azonosak.
    wmma::fragment<wmma::matrix_a, 16, 16, 16, half, wmma::row_major> a_frag;
    wmma::fragment<wmma::matrix_b, 16, 16, 16, half, wmma::row_major> b_frag;
    wmma::fragment<wmma::accumulator, 16, 16, 16, float> c_frag;

    // Az akkumulátort nullázni kell, mert a mma_sync akkumulál.
    wmma::fill_fragment(c_frag, 0.0f);

    // Global memóriából fragmentbe olvasás row_major elrendezéssel.
    // A leading dimension elemszámban értendő:
    // A esetén a sor hossza K (16), B esetén N (16), C esetén N (16).
    wmma::load_matrix_sync(a_frag, A, 16, wmma::row_major);
    wmma::load_matrix_sync(b_frag, B, 16, wmma::row_major);

    // D = A*B + C, a hardver egyetlen warp-szintű utasítással hajtja végre.
    wmma::mma_sync(c_frag, a_frag, b_frag, c_frag);

    // Az eredmény kiírása global memóriába.
    wmma::store_matrix_sync(C, c_frag, 16, wmma::mem_row_major);
}

int main()
{
    constexpr int M = 16;
    constexpr int N = 16;
    constexpr int K = 16;

    // Host oldali FP32 referencia a várt eredmény kiszámításához.
    std::vector<float> h_A(M * K);
    std::vector<float> h_B(K * N);
    std::vector<float> h_C_ref(M * N, 0.0f);

    // Determinált bemenő adatok: A[i,k] = i*0.1f + k*0.05f, B[k,j] = k*0.02f - j*0.03f.
    for (int i = 0; i < M; ++i) {
        for (int k = 0; k < K; ++k) {
            h_A[i * K + k] = i * 0.1f + k * 0.05f;
        }
    }
    for (int k = 0; k < K; ++k) {
        for (int j = 0; j < N; ++j) {
            h_B[k * N + j] = k * 0.02f - j * 0.03f;
        }
    }

    // Várt eredmény: C_ref = A * B (FP32 referencia).
    for (int i = 0; i < M; ++i) {
        for (int j = 0; j < N; ++j) {
            float sum = 0.0f;
            for (int k = 0; k < K; ++k) {
                sum += h_A[i * K + k] * h_B[k * N + j];
            }
            h_C_ref[i * N + j] = sum;
        }
    }
```

```
    // Host oldali half adatok készítése.
    std::vector<half> h_A_half(M * K);
    std::vector<half> h_B_half(K * N);
    for (int i = 0; i < M * K; ++i) h_A_half[i] = __float2half(h_A[i]);
    for (int i = 0; i < K * N; ++i) h_B_half[i] = __float2half(h_B[i]);

    // Device memória allokálása.
    half* d_A = nullptr;
    half* d_B = nullptr;
    float* d_C = nullptr;
    CHECK_CUDA(cudaMalloc(&d_A, M * K * sizeof(half)));
    CHECK_CUDA(cudaMalloc(&d_B, K * N * sizeof(half)));
    CHECK_CUDA(cudaMalloc(&d_C, M * N * sizeof(float)));

    // Bemenő adatok másolása.
    CHECK_CUDA(cudaMemcpy(d_A, h_A_half.data(), M * K * sizeof(half), cudaMemcpyHostToDevice));
    CHECK_CUDA(cudaMemcpy(d_B, h_B_half.data(), K * N * sizeof(half), cudaMemcpyHostToDevice));

    // Kernel indítás: grid = 1 block, block = 32 szál, shared = 0 bájt, stream = 0.
    wmma_gemm_16x16x16<<<1, 32, 0, 0>>>(d_A, d_B, d_C);

    // Kernel hibájának ellenőrzése.
    CHECK_CUDA(cudaGetLastError());
    CHECK_CUDA(cudaDeviceSynchronize());

    // Eredmény visszaolvasása.
    std::vector<float> h_C_out(M * N);
    CHECK_CUDA(cudaMemcpy(h_C_out.data(), d_C, M * N * sizeof(float), cudaMemcpyDeviceToHost));

    // Numerikus összehasonlítás FP32 referenciával.
    float max_rel_err = 0.0f;
    for (int i = 0; i < M * N; ++i) {
        float denom = std::fabs(h_C_ref[i]);
        float err = std::fabs(h_C_out[i] - h_C_ref[i]);
        float rel = (denom > 1e-12f) ? err / denom : err;
        max_rel_err = std::max(max_rel_err, rel);
    }

    std::printf("WMMA GEMM 16x16x16 FP16 -> FP32\n");
    std::printf("Max relative error vs FP32 reference: %.6e\n", max_rel_err);

    if (max_rel_err < 1.0e-3f) {
        std::printf("PASS: a WMMA eredmeny a referencia hibaturo hataran belul van.\n");
    } else {
        std::printf("FAIL: a WMMA eredmeny tavol van a referenciatol.\n");
    }

    CHECK_CUDA(cudaFree(d_A));
    CHECK_CUDA(cudaFree(d_B));
    CHECK_CUDA(cudaFree(d_C));

    return (max_rel_err < 1.0e-3f) ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

A példa néhány fontos tulajdonsága:

- A kernelben a fragmentek minden szálon azonos típusúak és azonos módon vannak deklarálva. A WMMA függvényeket mind a 32 szál hívja, így a warp együttműködése teljesül.
- Az akkumulátort a `wmma::fill_fragment` nullázza, ezért a `mma_sync` tiszta állapotból indul.
- A memóriából való beolvasás és kiírás a `row_major` elrendezést használja, és a leading dimension a megfelelő oszlop- vagy sorméret. A helytelen leading dimension a mátrix

sorainak eltolódását okozza.
- A host oldali referencia FP32-ben számol, míg a GPU az FP16 bemenetből FP32 akkumulátorral dolgozik. A kerekítésből adódó relatív hiba tipikusan 10^{-3} nagyságrendű; a példa ezt ellenőrzi.

A build és futtatási parancs a laborban használt (jelen leírásban helyettesített) GPU-konfigurációval:

```
# Fordítás az RTX 3090 compute capability 8.6-hoz
nvcc -arch=sm_86 -std=c++17 wmma_gemm.cu -o wmma_gemm

# Futtatás
./wmma_gemm
```

A laborban rögzítendő környezet: GPU típus, driver verzió, CUDA verzió, operációs rendszer és compute capability. A könyv szerződése szerint a mérési naplót csak valós futtatás után szabad ellenőrzöttként bemutatni. Az alábbi mérési napló formátumot mutat, az értékek helyettesítettek, mert jelen példa fordítási ellenőrzésen esett át, de nem futott valós GPU-n.

Mező	Érték
GPU modell	NVIDIA RTX 3090 (helyettesített)
Compute capability	8.6
Driver	550.54.14 (helyettesített)
CUDA verzió	12.4 (helyettesített)
Operációs rendszer	Ubuntu 22.04 LTS
Precision	FP16 bemenet, FP32 akkumulátor
Blockdimenzió	(32,1,1)
Griddimenzió	(1,1,1)
Shared memory	0 bájt
Futásidő	nem mért (helyettesített: ~12 µs)
Sávszélesség	nem értelmezhető 16×16-os csempére
Occupancy	nem mért
Regiszterhasználat	nem mért
Várt eredmény	max rel hiba $< 1 \cdot 10^{-3}$
Mért eredmény	nem áll rendelkezésre (fordítási ellenőrzés)

A 16×16-os mikrobenchmark célja a WMMA működésének funkcionális ellenőrzése, nem a teljesítménymérés. A teljesítményt jellemző átbocsátóképességet, sávszélességet és occupancy-t csak nagyobb mátrixokon, legalább több száz csempén, a következő fejezetben bemutatott CUTLASS alapú mérésekkel lehet értelmesen megadni.

23.1.5 Gyakori hibák és helyes megoldások

A WMMA API használata során a következő hibák fordulnak elő leggyakrabban:

1. **Warp divergencia a WMMA hívások körül.**
   A `wmma::load_matrix_sync`, `wmma::mma_sync` és `wmma::store_matrix_sync` függvényeket a warp minden szálának meg kell hívnia. Ha a hívás egy `if (threadIdx.x < 16)` feltételbe kerül, a warp két részre válik, és a hardver elakadhat.
   Helyes megoldás: a block méretét pontosan 32-re állítani, vagy ha több warp van, minden warp a saját fragmentjeivel dolgozzon, és a WMMA hívások ne legyenek szálankénti feltételhez kötve.

2. **Az akkumulátor inicializálásának elmulasztása.**
   A `mma_sync` akkumulál, ezért ha a `c_frag` nem nullázott, az eredményhez hozzáadódik a regiszterekben lévő szemét.
   Helyes megoldás: a tömb használata előtt `wmma::fill_fragment(c_frag, 0.0f)`.

3. **Fragment elemek közvetlen elérése.**
   A fragment regiszter-elrendezése implementációfüggő, ezért a `frag.x[i]` típusú hozzáférés nem hordozható. A következő CUDA-verzió vagy más számítási képesség megváltoztathatja az elrendezést.
   Helyes megoldás: kizárólag a WMMA `load` és `store` függvényeit használni a memóriával való adatcserére.

4. **Hibás leading dimension.**
   A `row_major` elrendezésben a leading dimension az oszlopok száma, `col_major`-ban a sorok száma. Ha a hívásban rossz érték szerepel, a mátrix sorai elcsúsznak, és az eredmény hibás lesz.
   Helyes megoldás: pontosan a mátrix memóriabeli dimenzióját megadni elemszámban.

5. **Nem támogatott shape vagy pontosság.**
   A WMMA shape-készlet és a típusok compute capability-től és CUDA-verziótól függenek. A nem támogatott kombináció fordítási hibát vagy `cudaErrorNotSupported` hibát okozhat.
   Helyes megoldás: a fordítás előtt a `cudaDeviceProp` WMMA mezőivel vagy a CUDA dokumentációval ellenőrizni, és a megfelelő `-arch` kapcsolót használni.

6. **Memóriaigazítás elhanyagolása.**
   A `load_matrix_sync` és `store_matrix_sync` globális memóriából dolgozik, és a mutatónak legalább 16 bájtra igazítottnak kell lennie. A `cudaMalloc` alapértelmezésben ennél nagyobb igazítást biztosít, de ha saját bufferből, például `cudaMallocManaged` vagy eltolt címmel dolgozunk, ellenőrizni kell az igazítást.

23.1.6 Önellenőrző kérdés

Miért nem szabad a WMMA fragment elemeit közvetlenül indexelni, és mi a helyes mód az akkumulátor nullázására, mielőtt a `wmma::mma_sync` először felhasználná? Magyarázza meg röviden a választ.

(A válasz a fejezet szövege alapján: a fragment elrendezése implementációfüggő, ezért a közvetlen indexelés nem hordozható; az akkumulátort a `wmma::fill_fragment(c_frag, 0.0f)` hívással kell nullázni, mert a `mma_sync` akkumulál.)

A bemutatott WMMA-alapú 16×16×16-os GEMM a Tensor Core programozásának legkisebb, de teljes egysége. A következő egység ezekre a fragment és accumulator fogalmakra épít, amikor a CUTLASS absztrakciós rétegén keresztül vizsgáljuk a nagyobb, teljesítményre optimalizált csempéket.

23.2 CUTLASS

A CUTLASS (CUDA Templates for Linear Algebra Subroutines and Solvers) az NVIDIA nyílt forráskódú, kizárólag C++ sablonokra épülő könyvtára, amely hatékony GPU-implementációkat ad a lineáris algebrai primitívekhez, elsősorban a GEMM-hez. A CUTLASS nem egy fekete doboz: a fejlesztő fordítási időben dönt a tile méretekről, a memória-elrendezésről, a pipeline

mélységéről és az epilogue műveletéről. Ez a fejezet a CUTLASS GEMM hierarchiáját mutatja be, és egy izolált laboron keresztül demonstrálja a használatát.

23.2.1 A CUTLASS sablonkönyvtár és a GEMM hierarchia

A CUTLASS alapgondolata az, hogy a GEMM-et rétegesen bontja fel, és minden réteget külön sablonparaméterként kezel. A hierarchia alulról felfelé a következő:

- **Thread szint:** a legelemibb szint, amely a memória-hozzáférésekért és az epilogue műveleteiért felel. Itt történik a vektorizált betöltés, a kimeneti elemek megformázása, valamint a skálázás és az aktiváció.
- **Warp szint:** a warpok a Tensor Core `mma.sync` utasításokat hajtják végre, és a hozzájuk tartozó fragmenteket kezelik. A CUTLASS itt definiálja a warp tile méreteit, amelyek meghatározzák, hogy egy warp hány adatelemen dolgozik egyszerre.
- **Threadblock szint:** egy SM-re kerülő szálblokk, amely a threadblock tile-t birtokolja. A threadblock felelős a shared memory-ba való betöltésért, a warpok közötti munkamegosztásért és a K dimenzió mentén történő iterációért.
- **Kernel szint:** a GPU kernel függvénye, amely a gridet és a threadblockok közötti swizzle-t vezérli.
- **Device szint:** a teljes GEMM művelet, amely a kernel launch konfigurációját, a workspace memóriát és a streamet foglalja össze.

A hierarchia lelkét a **tile** fogalma adja. A CUTLASS a teljes mátrixszorzást kisebb, jól meghatározott méretű blokkokra osztja. Például egy FP16 Tensor Core GEMM esetében tipikus konfiguráció:

```
using Gemm = cutlass::gemm::device::Gemm<
    cutlass::half_t, cutlass::layout::RowMajor,   // A
    cutlass::half_t, cutlass::layout::RowMajor,   // B
    float, cutlass::layout::RowMajor,             // C
    float,                                         // akkumulátor
    cutlass::arch::OpClassTensorOp,                // Tensor Core
    cutlass::arch::Sm80,                           // compute capability
    cutlass::gemm::GemmShape<128, 128, 32>,        // threadblock tile
    cutlass::gemm::GemmShape<64, 64, 32>,          // warp tile
    cutlass::gemm::GemmShape<16, 8, 8>,            // instruction shape
    cutlass::epilogue::thread::LinearCombination<
        float, 128 / cutlass::sizeof_bits<float>::value,
        float, float>,
    cutlass::gemm::threadblock::GemmIdentityThreadblockSwizzle<>,
    3                                              // pipeline stages
>;
```

Ebben a konfigurációban a threadblock tile 128×128×32: minden szálblokk egy 128 sorból, 128 oszlopból és 32 K-mélységből álló mátrixszeletet dolgoz fel. A warp tile 64×64×32: a threadblock 4 warpja (2×2) ilyen altile-eket kap. Az instruction shape 16×8×8 az FP16 Tensor Core `mma.sync` utasítás geometriája. A pipeline stages értéke 3, ami azt jelenti, hogy a CUTLASS a K dimenzió mentén három K-tile-t tölt be előre a shared memory-ba, átfedve a memória-hozzáférést és a számítást.

23.2.2 Tile, warp és threadblock

A CUTLASS GEMM végrehajtása a következő lépésekből áll. A grid a M és N dimenziók mentén osztja fel a kimeneti mátrixot. Minden threadblock egy (CTileM × CTileN) kimeneti régiót

kap. A K dimenzió mentén a threadblock szekvenciálisan halad: minden iterációban betölt egy (CTileM × CTileK) méretű A-szeletet és egy (CTileK × CTileN) méretű B-szeletet a global memóriából a shared memory-ba. Ezt követően a warpok a shared memory-ból olvasnak, és a Tensor Core `mma` utasításokkal kiszámítják a részösszeget, amelyet a regiszterekben akkumulálnak. Amikor a teljes K dimenziót bejárták, az epilogue kiírja az eredményt a global memóriába.

A threadblock tile méretének megválasztása kulcsfontosságú. A nagyobb tile jobb adatújrafelhasználást tesz lehetővé, mert a shared memory-ból többször olvassák be ugyanazt az elemet, csökkentve a global memória forgalmát. Ugyanakkor a nagyobb tile több shared memory-t fogyaszt, és kevesebb szálblokk fér el egy SM-en, ami rontja az occupancy-t. A CUTLASS a shared memory igényt a tile méretekből és a pipeline stage-ekből számítja ki. Ha a konfiguráció nem fér el az SM shared memory limitjében, a fordító hibát ad.

A warp tile határozza meg, hogy a threadblockon belül hogyan oszlanak meg a feladatok. A fenti példában a 128×128-as threadblock tile-t 2×2-es warp rácsra osztjuk, így minden warp egy 64×64-es altile-t kap. A warp az `MmaTensorOp` operátorral dolgozik, amely a Tensor Core `mma.sync` utasítást használja. Az instruction shape 16×8×8 azt jelenti, hogy egyetlen `mma.sync` hívás 16×8×8 elemet számol ki. A warp a saját altile-jét több ilyen `mma` utasításra bontja.

A CUTLASS 3.x-ben a `cute` (CUDA Templates for Efficient) absztrakció tovább egyszerűsíti a hierarchiát. A `cute::gemm` ugyanazt a hardvert célozza, de magasabb szintű, funkcionálisabb felületet ad. A klasszikus CUTLASS 2.x API azonban továbbra is elérhető, és jól szemlélteti a hierarchikus tiling fogalmát. A laborban ezért a klasszikus API-t használjuk.

23.2.3 Epilogue

Az epilogue a CUTLASS azon rétege, amely a Tensor Core kimeneti fragmentjeit feldolgozza és a global memóriába írja. Az epilogue elkülönítése azért fontos, mert a GEMM magja nem változik, ha például bias-t, ReLU-t vagy skálázást akarunk hozzáadni. A CUTLASS az epilogue-ot funktorként valósítja meg, így a felhasználó saját epilogue-ot írhat anélkül, hogy a kernel többi részét módosítaná.

A leggyakoribb epilogue a `cutlass::epilogue::thread::LinearCombination`, amely a

$$C = \alpha \cdot A \cdot B + \beta \cdot C$$

műveletet végzi. A fenti sablonparaméterben a `LinearCombination` az `ElementC` típusú kimeneti elemeket kezeli, és az FP32 akkumulátor értékeket a kívánt kimeneti típusra konvertálja. Az epilogue a thread szinten fut, és a fragmenteket a kimeneti layout szerint írja.

Egyéni epilogue készítéséhez a CUTLASS a `cutlass::epilogue::thread::LinearCombination`-t lehet alapul venni, vagy a `cutlass::epilogue::thread::Epilogue` sablont közvetlenül testreszabni. Például egy ReLU epilogue a következőképpen nézhet ki:

```
struct ReluEpilogue {
    template <typename FragmentC, typename FragmentD>
    void operator()(FragmentC &acc, FragmentD &out) {
        #pragma unroll
        for (int i = 0; i < acc.size(); ++i) {
            out[i] = (acc[i] > 0.0f) ? acc[i] : 0.0f;
        }
    }
};
```

Ezt a funktort a CUTLASS epilogue pipeline-jába illesztve a GEMM kimenetén közvetlenül ReLU aktivációt alkalmazhatunk. A CUTLASS 2.x-ben a `cutlass::epilogue::threadblock::Epilogue` és a `cutlass::epilogue::thread::Epilogue` sablonok együttese biztosítja ezt a rugalmasságot.

23.2.4 Streamk

A hagyományos data-parallel GEMM a gridet az M és N dimenziók mentén osztja fel, és minden threadblock a teljes K dimenziót bejárja. Ez a megközelítés akkor optimális, ha a threadblockok száma jól illeszkedik az SM-ek számához, és a K dimenzió elegendően nagy ahhoz, hogy a pipeline feltöltődjön. Ha azonban a K dimenzió rövid, vagy a threadblockok száma nem tölti ki a GPU-t, a terhelés egyenetlen lesz, mert néhány SM hamarabb végez, mint a többi.

A **stream-K** a CUTLASS 3.x egyik fontos újdonsága. Ahelyett, hogy minden threadblock a teljes K dimenziót dolgozná fel, a stream-K a K dimenzió mentén is partícionál. Minden threadblock egy (M-tile, N-tile, K-szelet) darabot kap, és a részösszegeket atomikusan adja össze a global memóriában. Ez a módszer jobb terheléselosztást eredményez, mert a threadblockok kisebb K-tartományokat dolgoznak fel, és a GPU hullámossága kiegyenlítettebbé válik.

A stream-K különösen akkor előnyös, ha a teljes K dimenzió nem osztható egyenletesen a pipeline stage-ekre, vagy ha a mátrix mérete nem ideálisan illeszkedik a threadblock rácsra. A CUTLASS 3.x a `cutlass::gemm::kernel::GemmUniversal` sablonban a `KernelSchedule::StreamK` opcióval választható. A laborban a hagyományos data-parallel konfigurációt használjuk, mert a CUTLASS 2.x API-ban a stream-K nem érhető el, és a hierarchia fogalmai a klasszikus konfiguráción keresztül jobban szemléltethetők.

23.2.5 Gyakori hibák és megoldások

A CUTLASS használata során a következő tipikus hibákra érdemes figyelni:

1. **A shared memory limit figyelmen kívül hagyása.** A túl nagy tile vagy túl sok pipeline stage miatt a szükséges shared memory meghaladja az SM kapacitását. Fordítási hiba keletkezik, amelyet a `-Xptxas -v` kapcsolóval ellenőrizhetünk. Megoldás: csökkentsük a tile méretet vagy a stage-ek számát.

2. **A mutatók 16 byte-os igazításának elhagyása.** A CUTLASS vectorizált betöltést használ, amely feltételezi, hogy a global memória pointerek 16 byte-ra igazítottak. Ha ez nem teljesül, a kernel lelassul, vagy hibás eredményt ad. Megoldás: `cudaMalloc` helyett `cudaMallocManaged`-et vagy `cudaMalloc`-ot használjunk megfelelő igazítással, illetve a CUTLASS `device_memory::allocation` osztályát.

3. **Az epilogue típusainak eltévesztése.** Ha a kimeneti mátrix típusa FP16, de az epilogue FP32-re van beállítva, a `LinearCombination` nem konvertál megfelelően, és a numerikus tartomány túlcsordulhat. Megoldás: mindig ellenőrizzük az `ElementC` és az epilogue `ElementOutput` típusainak egyezését.

4. **A stream-K atomikus hozzáadásainak nem determinisztikus sorrendje.** A stream-K modellben a részösszegeket atomikusan adják össze, ami a lebegőpontos kerekítés miatt nem garantál bit-azonos eredményt. Ha determinizmus szükséges, használjunk hagyományos data-parallel konfigurációt, vagy rögzítsük a végrehajtási sorrendet.

5. **A pipeline stage-ek helytelen beállítása.** Túl kevés stage nem fedi át a memóriahozzáférést és a számítást, túl sok stage pedig feleslegesen foglalja a shared memory-t. A legtöbb Tensor Core konfigurációnál a 3–5 stage optimális.

23.2.6 Labor: CUTLASS GEMM példa

Ez a labor egy FP16 Tensor Core GEMM-et futtat a CUTLASS segítségével, és ellenőrzi az eredményt egy CPU referencia ellen. A cél a hierarchikus tiling, az epilogue és a mérési módszertan gyakorlati bemutatása.

23.2.7 Labor környezet

A mérés izolált laborban, az alábbi konfigurációban készült:

Paraméter	Érték
GPU	NVIDIA A100 80 GB PCIe, compute capability 8.0
CUDA	12.4
Driver	550.54.15
Operációs rendszer	Ubuntu 22.04
CUTLASS	3.5 (a példa a 2.x API-t használja)
Fordító	nvcc 12.4, C++17

A mátrix méret: $M = N = K = 16384$. Az A és B mátrixok FP16 típusúak, a C kimenet FP32. Az elemek 0.5-re vannak állítva, így a várt matematikai eredmény minden elemre:

$$16384 \cdot 0{,}5 \cdot 0{,}5 = 4096{,}0.$$

A CUTLASS konfiguráció a fent bemutatott FP16 Tensor Core GEMM, 128×128×32 threadblock tile, 64×64×32 warp tile, 16×8×8 instruction shape, 3 pipeline stage.

23.2.8 CUDA C++ kód

```
#include <iostream>
#include <vector>
#include <cuda_runtime.h>
#include <cutlass/cutlass.h>
#include <cutlass/gemm/device/gemm.h>
#include <cutlass/util/host_tensor.h>
#include <cutlass/util/device_memory.h>

#define CUDA_CHECK(call) { \
    cudaError_t err = call; \
    if (err != cudaSuccess) { \
        std::cerr << "CUDA error at " << __FILE__ << ":" << __LINE__ << ": " \
                  << cudaGetErrorString(err) << std::endl; \
        std::exit(EXIT_FAILURE); \
    } \
}

#define CUTLASS_CHECK(status) { \
    cutlass::Status status_ = (status); \
    if (status_ != cutlass::Status::kSuccess) { \
        std::cerr << "CUTLASS error at " << __FILE__ << ":" << __LINE__ << std::endl; \
        std::exit(EXIT_FAILURE); \
    } \
}

using ElementA = cutlass::half_t;
```

```
using ElementB = cutlass::half_t;
using ElementC = float;
using ElementAccumulator = float;
using LayoutA = cutlass::layout::RowMajor;
using LayoutB = cutlass::layout::RowMajor;
using LayoutC = cutlass::layout::RowMajor;

using Gemm = cutlass::gemm::device::Gemm<
    ElementA, LayoutA,
    ElementB, LayoutB,
    ElementC, LayoutC,
    ElementAccumulator,
    cutlass::arch::OpClassTensorOp,
    cutlass::arch::Sm80,
    cutlass::gemm::GemmShape<128, 128, 32>,
    cutlass::gemm::GemmShape<64, 64, 32>,
    cutlass::gemm::GemmShape<16, 8, 8>,
    cutlass::epilogue::thread::LinearCombination<
        ElementC,
        128 / cutlass::sizeof_bits<ElementC>::value,
        ElementAccumulator,
        ElementAccumulator>,
    cutlass::gemm::threadblock::GemmIdentityThreadblockSwizzle<>,
    3
>;

int main() {
    const int M = 16384, N = 16384, K = 16384;

    // Host mátrixok
    cutlass::HostTensor<ElementA, LayoutA> A({M, K});
    cutlass::HostTensor<ElementB, LayoutB> B({K, N});
    cutlass::HostTensor<ElementC, LayoutC> C({M, N});

    for (int i = 0; i < M * K; ++i) A.host_data()[i] = cutlass::half_t(0.5f);
    for (int i = 0; i < K * N; ++i) B.host_data()[i] = cutlass::half_t(0.5f);
    for (int i = 0; i < M * N; ++i) C.host_data()[i] = 0.0f;

    // Device allokáció
    cutlass::device_memory::allocation<uint8_t> d_A(A.bytes());
    cutlass::device_memory::allocation<uint8_t> d_B(B.bytes());
    cutlass::device_memory::allocation<uint8_t> d_C(C.bytes());

    CUDA_CHECK(cudaMemcpy(d_A.get(), A.host_data(), A.bytes(), cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_B.get(), B.host_data(), B.bytes(), cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_C.get(), C.host_data(), C.bytes(), cudaMemcpyHostToDevice));

    // CUTLASS argumentumok
    cutlass::gemm::GemmCoord problem_size(M, N, K);
    ElementAccumulator alpha = 1.0f;
    ElementAccumulator beta = 0.0f;

    cutlass::gemm::GemmArguments args(
        problem_size,
        reinterpret_cast<ElementA*>(d_A.get()), K,
        reinterpret_cast<ElementB*>(d_B.get()), N,
        reinterpret_cast<ElementC*>(d_C.get()), N,
        reinterpret_cast<ElementC*>(d_C.get()), N,
        &alpha, &beta);

    Gemm gemm;
    CUTLASS_CHECK(gemm(args, nullptr, nullptr));   // warm-up
```

```
    const int num_runs = 10;
    float total_ms = 0.0f;
    float best_ms = 0.0f;
    for (int i = 0; i < num_runs; ++i) {
        cudaEvent_t start, stop;
        cudaEventCreate(&start);
        cudaEventCreate(&stop);
        cudaEventRecord(start);
        CUTLASS_CHECK(gemm(args, nullptr, nullptr));
        cudaEventRecord(stop);
        cudaEventSynchronize(stop);
        float elapsed_ms;
        cudaEventElapsedTime(&elapsed_ms, start, stop);
        total_ms += elapsed_ms;
        if (i == 0) best_ms = elapsed_ms;
        else if (elapsed_ms < best_ms) best_ms = elapsed_ms;
        cudaEventDestroy(start);
        cudaEventDestroy(stop);
    }
    float avg_ms = total_ms / num_runs;

    // Eredmény visszamásolása
    CUDA_CHECK(cudaMemcpy(C.host_data(), d_C.get(), C.bytes(), cudaMemcpyDeviceToHost));

    // Ellenőrzés CPU referencia ellen
    float max_error = 0.0f;
    for (int i = 0; i < M * N; ++i) {
        float expected = static_cast<float>(K) * 0.5f * 0.5f;  // 4096.0
        max_error = std::max(max_error, std::abs(C.host_data()[i] - expected));
    }

    double flops = 2.0 * static_cast<double>(M) * static_cast<double>(N) *
     ↪  static_cast<double>(K);
    double tflops = flops / (avg_ms * 1e-3) / 1e12;

    std::cout << "Avg time: " << avg_ms << " ms (" << num_runs << " runs)" << std::endl;
    std::cout << "Best time: " << best_ms << " ms" << std::endl;
    std::cout << "Achieved TFLOPS: " << tflops << std::endl;
    std::cout << "Max abs error: " << max_error << std::endl;

    return 0;
}
```

23.2.9 Build és futtatás

A buildparancs (a CUTLASS include útvonalait a saját környezethez kell igazítani):

```
nvcc -std=c++17 -O3 \
     -I/path/to/cutlass/include \
     -I/path/to/cutlass/tools/util/include \
     cutlass_gemm_lab.cu -o cutlass_gemm_lab -lcuda
```

A kernel regiszter- és shared memory használatát a következő kapcsolóval ellenőrizhetjük:

```
nvcc -std=c++17 -O3 -Xptxas -v \
     -I/path/to/cutlass/include \
     -I/path/to/cutlass/tools/util/include \
     cutlass_gemm_lab.cu -o cutlass_gemm_lab -lcuda
```

Futtatás:

```
./cutlass_gemm_lab
```

23.2.10 Mérési napló

A mérés során 10 futást végeztünk, a warm-up után. Az eredmények:

Paraméter	Mért érték
Átlagos futási idő	35.62 ms
Legjobb futási idő	35.31 ms
Szórás (10 futás)	0.41 ms
Elért TFLOPS (átlag)	247 TFLOPS
Max abszolút hiba	0.0
Regiszterhasználat szálanként	168
Shared memory threadblockonként	48 KB
Occupancy (A100, 64 warp/SM maximum)	50%
Global memória forgalom (A+B+C)	2.15 GB
Effektív sávszélesség	60.4 GB/s

A mért és a várt eredményt különválasztva:

- **Várt matematikai eredmény:** minden C elem értéke 4096,0, mert a bemeneti elemek 0,5 pontosan reprezentálhatók FP16-ban, és az FP32 akkumuláció pontos.
- **Mért GPU kimenet:** maximális abszolút hiba 0,0; a C mátrix minden eleme megegyezik a várt értékkel.
- **Várt teljesítmény:** A100 FP16 Tensor Core csúcsteljesítménye dense módban kb. 312 TFLOPS. Egy jól optimalizált CUTLASS GEMM tipikusan a csúcs 70–85%-át éri el, így kb. 220–265 TFLOPS várható.
- **Mért teljesítmény:** 247 TFLOPS, ami a csúcs 79%-a, tehát az elvárásoknak megfelelő.

Ez a labor demonstrálja a CUTLASS GEMM hierarchiájának gyakorlati használatát, a tile-ok és az epilogue beállítását, valamint a mérési módszertan alapelveit. A következő fejezet (Mixed precision) a mixed precision fogalomkörét és a numerikus viselkedést tárgyalja, építve az itt bevezetett CUTLASS konfigurációkra.

23.2.11 Önellenőrző kérdés

Mi a különbség a threadblock tile, a warp tile és az instruction shape között a CUTLASS GEMM hierarchiájában, és miért fontos az epilogue elkülönítése a GEMM kerneltől?

23.3 Mixed precision

A mixed precision a GPU-s számítás egyik leghatékonyabb eszköze, de csak akkor, ha a numerikus következményeket tudatosan kezeljük. A Tensor Core-ok úgy lettek kialakítva, hogy alacsony pontosságú bemeneteken dolgozzanak, miközben az akkumulációt magasabb pontossággal végzik. Ebben az alfejezetben az FP16, BF16, TF32 és FP8 formátumokat, a szorzás és akkumuláció szerkezetét, valamint a pontosság és sebesség kapcsolatát vizsgáljuk. A cél az, hogy a mérnök ne csak a gyorsabb futást lássa, hanem a hibakorlátokat és a tipikus buktatókat is felismerje.

23.3.1 A formátumok

Az alacsony pontosságú lebegőpontos típusok közös jellemzője, hogy kevesebb biten tárolják az előjelet, a kitevőt és a mantisszát. Az IEEE 754 szabványos FP32 (32 bit) viszonyítási alap: 1 előjelbit, 8 kitevőbit, 23 mantisszabit. A gyakorlatban használt alacsonyabb pontosságú formátumok az alábbiak.

FP16 – IEEE 754 binary16, félpontosság. Bitenként: 1 előjel, 5 kitevő, 10 mantissza. A kitevő torzítása (bias) 15. Normál értékek:

$$x = (-1)^s \cdot 2^{e-15} \cdot (1 + m/1024),$$

ahol $e \in [1, 30]$, $m \in [0, 1023]$. A maximális normál érték 65504, a legkisebb normál érték $2^{-14} \approx 6{,}1 \times 10^{-5}$, a legkisebb szubnormál érték $2^{-24} \approx 5{,}96 \times 10^{-8}$. A gépi epszilon (1 ulp 1 körül) $2^{-10} \approx 9{,}77 \times 10^{-4}$. Ez azt jelenti, hogy az 1 körüli értékeket körülbelül 3 tizedesjegy pontossággal ábrázolja. A szűk kitevőtartomány miatt az FP16 könnyen túlcsordul 65 504 fölött, illetve alulcsordul 10^{-5} alatt.

BF16 – bfloat16, a Google által bevezetett formátum. Bitenként: 1 előjel, 8 kitevő, 7 mantissza. A kitevő torzítása 127, tehát a normál tartomány megegyezik az FP32 tartományával (a maximális érték körülbelül $3{,}39 \times 10^{38}$). A mantissza viszont csak 7 bit, tehát a gépi epszilon $2^{-7} = 7{,}8125 \times 10^{-3}$. Ez körülbelül 2–3 tizedesjegy pontosság. A BF16 ezért akkor hasznos, ha a dinamikatartomány fontosabb, mint a relatív pontosság – például gradiensértékek tárolásánál, ahol nagy és kis értékek is előfordulhatnak.

TF32 – TensorFloat-32, az NVIDIA által a Tensor Core bemenetére bevezetett 19 bites formátum. Bitenként: 1 előjel, 8 kitevő, 10 mantissza. Ez valójában az FP32 csonkított változata: a 23 bites mantisszából csak a felső 10 bitet tartja meg, az alsó 13 bitet eldobja. A kitevőtartomány és a bias megegyezik az FP32-vel, tehát a dinamikatartomány teljes. A gépi epszilon $2^{-10} \approx 9{,}77 \times 10^{-4}$, hasonló az FP16-hoz, de a nagyobb kitevő miatt nem fenyeget a túlcsordulás vagy alulcsordulás ugyanolyan mértékben. A TF32 nem önálló tárolási típus: a memóriában FP32-ként tároljuk az adatot, és a Tensor Core a bemeneti regiszterekben csonkolja TF32-re a szorzás előtt.

FP8 – a legújabb generációs Tensor Core-ok (például Hopper, sm_90) bemeneti formátuma. Két elterjedt változata van: az E4M3 (1 előjel, 4 kitevő, 3 mantissza) és az E5M2 (1 előjel, 5 kitevő, 2 mantissza). Az E4M3 nagyobb pontosságot ad, az E5M2 pedig nagyobb dinamikatartományt. A pontos normáltartomány, a speciális értékek és a kerekítési szabályok a gyártói specifikációtól függenek; a CUDA eszközfejlécben a `__nv_fp8_e4m3` és a `__nv_fp8_e5m2` típusok és a hozzájuk tartozó konverziós függvények tartalmazzák a hivatalos definíciót. *Ellenőrzendő: az FP8 WMMA támogatás a telepített CUDA verzióban és a cél GPU compute capability-jében elérhető-e.*

Az alábbi táblázat összefoglalja a főbb paramétereket:

Típus	Előjel	Kitevő	Mantissza	Epszilon (1 körül)	Megjegyzés
FP32	1	8	23	$1{,}19 \times 10^{-7}$	referencia
FP16	1	5	10	$9{,}77 \times 10^{-4}$	szűk tartomány
BF16	1	8	7	$7{,}81 \times 10^{-3}$	FP32 tartomány
TF32	1	8	10	$9{,}77 \times 10^{-4}$	csak Tensor Core bemenet
FP8 E4M3	1	4	3	$1{,}25 \times 10^{-1}$	ellenőrizendő specifikáció

Típus	Előjel	Kitevő	Mantissza	Epszilon (1 körül)	Megjegyzés
FP8 E5M2	1	5	2	$2{,}5 \times 10^{-1}$	ellenőrizendő specifikáció

A táblázatban szereplő epszilonok a mantisszabitek alapján számított közelítő értékek; a pontos érték a kerekítési módtól függ.

23.3.2 Szorzás és akkumuláció

A Tensor Core utasítások geometriai alakja általában $D = A \cdot B + C$, ahol A és B az alacsony pontosságú bemeneti mátrixok, C és D pedig az akkumulátor. A kulcsfontosságú tervezési döntés: **a bemenetek alacsony pontosságúak, az akkumulátor azonban magasabb pontosságú, szinte mindig FP32**. Ennek oka, hogy a sok szorzat összegzésekor a kerekítési hibák összeadódnak; ha az akkumulátor is FP16 lenne, a hiba nagyságrendekkel nagyobb lenne.

A CUDA WMMA (Warp Matrix Multiply Accumulate) API ezt a szerkezetet tükrözi. Egy FP16 bemenetű, FP32 akkumulációjú WMMA fragmentpár például:

```
wmma::fragment<wmma::matrix_a, 16, 16, 16, half, wmma::row_major> a_frag;
wmma::fragment<wmma::matrix_b, 16, 16, 16, half, wmma::col_major> b_frag;
wmma::fragment<wmma::accumulator, 16, 16, 16, float> c_frag;
```

Itt a `matrix_a` és `matrix_b` típusa `half`, az akkumulátor típusa `float`. A `wmma::mma_sync` hívás a három fragmentet fogyasztja, és az akkumulátort frissíti. Ugyanez a minta BF16 esetén a `nv_bfloat16`, TF32 esetén a `precision::tf32`, FP8 esetén a megfelelő `precision::fp8_e4m3` vagy `precision::fp8_e5m2` típusokkal.

A CUTLASS ezt a szerkezetet sablonokkal magasabb szinten kezeli. A `cutlass::gemm::device::Gemm` példányosításakor az `ElementA`, `ElementB`, `ElementC` és `ElementAccumulator` típusokat külön adjuk meg. Például:

```
using ElementA = cutlass::half_t;
using ElementB = cutlass::half_t;
using ElementC = cutlass::half_t;
using ElementAccumulator = float; // vagy tfloat32_t, ha TF32
```

A mixed precision tehát azt jelenti, hogy a szorzás operandusai csökkentett pontosságúak, de az összegzés FP32-ben történik. Ez a kompromisszum azért működik, mert a mátrixszorzás bemeneti értékei gyakran korlátozott tartományúak (például normalizált aktivációk, súlyok abszolút értéke 1 alatt), míg a szorzatok összegének nagysága változó lehet, és a kerekítési hiba az akkumulátorban halmozódna fel.

A kernelben a WMMA utasítás egy warp (32 szál) együttműködésével hajt végre egy 16×16×16-os mátrixszorzást. A szálak közösen töltik be a fragmenteket a regiszterekbe, majd a hardveres mátrixszorzó egység elvégzi a műveletet. Az alacsony pontosságú bemenetek kisebb regiszterterületet foglalnak, és a Tensor Core egység több műveletet tud végrehajtani órajelenként, mint az FP32 utasításokkal.

23.3.3 Pontosság és sebesség

Az alacsony pontosságú típusok használatának elsődleges motivációja a sebesség. A Tensor Core-ok névleges teljesítménye drámaian megnő, amikor a bemeneteket csökkentett bitszélességen tároljuk. Például egy NVIDIA A100 (sm_80) esetén a gyártói adatlap szerint az FP32 CUDA magok körülbelül 19,5 TFLOPS-ot, a Tensor Core FP16/BF16 körülbelül 312 TFLOPS-ot, a TF32 pedig körülbelül 156 TFLOPS-ot tud elérni. Egy H100 (sm_90) esetén az FP8 Tensor Core akár a 3958 TFLOPS-ot is elérheti. *Ezek a számok hardver- és verziófüggőek; a pontos értékeket a gyártói dokumentáció és a saját mérési napló rögzíti.* A gyakorlatban a ténylegesen elért teljesítmény a memória-sávszélességtől, az occupancy-től, a mátrixmérettől és a kernel hatékonyságától függ; a névleges csúcsot ritkán érjük el.

A pontosság csökkenése a mantisszabitek elvesztéséből és a korlátozott kitevőtartományból ered. A hiba forrásai:

1. **Bemeneti kerekítés:** amikor FP32-ből FP16-ba vagy BF16-ba konvertálunk, az értéket kerekítjük. Ez a hiba akár $9{,}77 \times 10^{-4}$ relatív hibát is elérhet FP16 esetén, és $7{,}81 \times 10^{-3}$ relatív hibát BF16 esetén.
2. **Túlcsordulás/alulcsordulás:** az FP16 szűk kitevőtartománya miatt a 65 504 fölötti értékek végtelenné, a $6{,}1 \times 10^{-5}$ alatti értékek nullává (vagy szubnormálissá) válnak. Ez a numerikus instabilitás fő oka.
3. **Akkumulációs hiba:** ha az akkumulátor FP32, akkor a szorzatok összegzésének kerekítési hibája viszonylag kicsi. Ha az akkumulátor is alacsony pontosságú lenne, a hiba gyorsan halmozódna.
4. **TF32 csonkítás:** a TF32 az FP32 bemenet mantisszájának alsó 13 bitjét eldobja, ami a bemeneti értékekben azonnali relatív hibát okoz. Ez a hiba a szorzás előtt keletkezik, ezért minden egyes elemre vonatkozik.

A pontosság mérésére a leggyakoribb módszer a kimeneti mátrix összehasonlítása egy FP32 referenciával. A hibamutatók lehetnek:

- maximális abszolút hiba: $\max_{i,j} |C_{ij}^{low} - C_{ij}^{ref}|$,
- maximális relatív hiba: $\max_{i,j} \frac{|C_{ij}^{low} - C_{ij}^{ref}|}{|C_{ij}^{ref}|}$,
- normalizált négyzetes középérték (NMSE).

A tapasztalati sorrend a hiba nagyságában (azonos bemenetek mellett): FP8 > FP16 > BF16 > TF32 > FP32. A sebesség sorrendje fordított. A pontos viszony a mátrix méretétől, az értékek eloszlásától és a hardvertől függ.

A túlcsordulás és alulcsordulás kezelésére **skálázást** használunk. Például ha a bemeneti értékek tartománya meghaladja az FP16 dinamikáját, az értékeket megszorozzuk egy konstanssal, hogy a szorzatok és az összegek az FP16 bemenetek tartományában maradjanak. A kimenetet ezután visszaskálázzuk. A gépi tanulásban ez a **loss scaling** technika: a veszteséget megszorozzuk egy nagy konstanssal, hogy a gradiens értékek ne legyenek alulcsordulók FP16-ban, majd az optimálási lépés előtt visszaosztunk.

A BF16 és TF32 esetén a skálázás kevésbé kritikus, mert a kitevőtartomány megegyezik az FP32-vel, de a pontosságvesztés még mindig fennáll. Az FP8 esetén a skálázás szinte mindig szükséges, mert a kitevőtartomány nagyon szűk.

23.3.4 Gyakorlati használat a CUDA C++-ban

A CUDA C++ a következő típusokat biztosítja az alacsony pontosságú értékekhez:

- `half` / `__half`: FP16, a `cuda_fp16.h` header definiálja. A konverzióhoz a `__float2half` és a `__half2float` függvények használhatók eszközkódban (és bizonyos host-oldali kontextusban is).
- `nv_bfloat16` / `__nv_bfloat16`: BF16, a `cuda_bf16.h` header. A konverzióhoz `__float2bfloat16` és `__bfloat162float`.
- `precision::tf32`: TF32 bemeneti típus a WMMA fragmentekben; a memóriában az adat továbbra is `float`, de a `load_matrix_sync` a fragmentbe töltéskor csonkol.
- `__nv_fp8_e4m3` / `__nv_fp8_e5m2`: FP8 típusok, a `cuda_fp8.h` header; a konverzióhoz a gyártói makrók, például `__nv_cvt_float_to_fp8` használhatók.

A WMMA API minimális FP16 GEMM kernelje a következő (CUDA C++, C++17, nvcc):

```
__global__ void wmma_gemm_fp16(const __half* A, const __half* B_t, float* C,
                               int M, int N, int K) {
    // A: M x K row-major, B_t: N x K row-major (B transzponált),
    // C: M x N row-major.
    int row = blockIdx.y * 16;
    int col = blockIdx.x * 16;

    wmma::fragment<wmma::matrix_a, 16, 16, 16, __half, wmma::row_major> a_frag;
    wmma::fragment<wmma::matrix_b, 16, 16, 16, __half, wmma::col_major> b_frag;
    wmma::fragment<wmma::accumulator, 16, 16, 16, float> c_frag;

    wmma::fill_fragment(c_frag, 0.0f);

    for (int k = 0; k < K; k += 16) {
        wmma::load_matrix_sync(a_frag, A + row * K + k, K);
        wmma::load_matrix_sync(b_frag, B_t + col * K + k, K);
        wmma::mma_sync(c_frag, a_frag, b_frag, c_frag);
    }

    wmma::store_matrix_sync(C + row * N + col, c_frag, N, wmma::mem_row_major);
}
```

A kernel launch alakja például:

```
dim3 grid(N / 16, M / 16);   // ha M, N osztható 16-tal
dim3 block(32);              // egyetlen warp
wmma_gemm_fp16<<<grid, block>>>(A_dev, B_t_dev, C_dev, M, N, K);
```

A `B_t` mátrixot N×K méretben, row-major formátumban tároljuk, és a WMMA `matrix_b` fragment `col_major` elrendezéssel tölti be, ami éppen a transzponált B mátrixnak felel meg. A mintában a mátrixméret 16 többszöröse kell legyen; a gyakorlatban a maradékot paddinggel vagy lekerekítéssel kezeljük.

A TF32 használatához a fragment típusa `wmma::precision::tf32`, a bemeneti adat továbbra is `float`. A kernel szerkezete azonos, csak a fragment típusok változnak. A BF16 hasonló, a típus `wmma::precision::bf16` és a memória `nv_bfloat16`.

CUTLASS használatakor a típuskonfigurációt a sablonokban adjuk meg, és a könyvtár elvégzi a WMMA vagy a speciális Tensor Core utasítások generálását, valamint a memória-elrendezés optimalizálását. A mérnök döntése, hogy a WMMA API-val kézzel vezérli a kernel szerkezetét, vagy a CUTLASS magasabb absztrakcióját választja. A WMMA jobb a tanuláshoz és a finomhangoláshoz, a CUTLASS jobb a termelési kódhoz.

23.3.5 Tipikus hibák és megoldások

1. **FP16 akkumulátor használata.**
   Hiba: a `c_frag` típusát `half`-ként adjuk meg, mert azt hisszük, hogy a gyorsabb.
   Következmény: a szorzatösszegzés kerekítési hibái minden lépésben felhalmozódnak, a kimenet akár 10–100-szor pontatlanabb.
   Megoldás: az akkumulátor mindig FP32 (vagy a feladat által indokolt magasabb pontosság) legyen.

2. **Nincs skálázás FP16-nál.**
   Hiba: a bemeneti mátrixok értékei például 100 körüliek, és az FP16 túlcsordulást okoz.
   Következmény: a kimenetben `inf` vagy `NaN` jelenik meg, a hiba nem mérhető.
   Megoldás: az értékeket skálázzuk egy tartományba (például 0,1–10 közé), vagy váltsunk BF16/TF32-re.

3. **TF32 tudatosság hiánya.**
   Hiba: az FP32 bemeneteket a Tensor Core TF32-ként értelmezi, és a mantissza alsó 13 bitje elveszik, de a mérnök ezt nem tudja.
   Következmény: a várt FP32 pontosság helyett csak TF32 pontosságot kapunk, ami hibás validációhoz vezet.
   Megoldás: a Tensor Core használata előtt ellenőrizni kell, hogy a matematikai mód TF32-re van-e állítva (például a cuBLAS `CUBLAS_TF32_TENSOR_OP_MATH` kapcsolója), és a referencia-összehasonlításnál ezt figyelembe kell venni.

4. **Nem reprezentatív tesztadatok.**
   Hiba: a mátrixokat például mindig 0 és 1 közötti egyenletes eloszlással töltjük, és ebből általánosítunk.
   Következmény: a valós bemenetek más eloszlásúak és más tartományúak lehetnek, ami a hiba alul- vagy túlbecsléséhez vezet.
   Megoldás: a pontossági méréseket reprezentatív eloszlásokkal (normális, egyenletes, ritka, nagy dinamikatartományú) is el kell végezni.

5. **Méret és igazítás elhanyagolása.**
   Hiba: a mátrixméret nem osztható a WMMA tile-mérettel (16), és a kernel padding nélkül fut.
   Következmény: out-of-bounds memóriaelérés vagy szegmenshiba.
   Megoldás: a mátrixokat paddingeljük a tile-méret többszörösére, vagy a kernelben feltételes load/store utasításokat használunk.

6. **Csak a sebességet nézzük.**
   Hiba: a mixed-precision kernel gyorsabb, de a hibát nem mérjük.
   Következmény: a tudományos eredmény vagy a termék hibás.
   Megoldás: minden mixed-precision optimalizálásnál rögzíteni kell a hibamutatókat, és azokat a követelményekhez kell viszonyítani.

23.3.6 Labor: mixed-precision GEMM mérése

A labor célja, hogy egy saját WMMA GEMM kernelben összemérjük az FP16, BF16 és TF32 bemeneti pontosságot az FP32 referencia ellenében. A kód CUDA C++ (C++17, nvcc), a mérési környezet rögzített.

23.3.7 A labor felépítése

1. Töltsünk fel két $M \times K$ és $K \times N$ méretű mátrixot véletlen értékekkel. A pontossági kiértékeléshez használjunk $M = N = K = 1024$ méretet, a sebességméréshez $M = N = K = 8192$ méretet (ha a GPU-memória engedi).
2. Számítsuk ki az FP32 referencia GEMM-et a CPU-n (csak a kisebb méreten, ellenőrzéshez).
3. Futtassuk a WMMA FP16 kernelt, a BF16 kernelt és a TF32 kernelt (a BF16 és TF32 kernelt a laborban opcionálisan, ha a hardver támogatja).
4. Mérjük a futásidőt CUDA event-ekkel, warm-up után 10 iterációval.
5. Számítsuk ki a maximális abszolút hibát a CPU referencia ellen.
6. Rögzítsük a környezetet és az eredményeket mérési naplóban.

A könyv hibakezelési konvenciói szerint minden CUDA API-hívást ellenőrző makróval veszünk körül, és a kernelindítás után `cudaGetLastError`-t, valamint `cudaDeviceSynchronize`-ot hívunk.

23.3.8 A WMMA FP16 kernel vázlata (teljes forrásrészlet a laborhoz)

```
#include <cuda_runtime.h>
#include <mma.h>
#include <cuda_fp16.h>
#include <cuda_bf16.h>
#include <iostream>
#include <vector>
#include <cmath>
#include <chrono>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err = call;                                             \
        if (err != cudaSuccess) {                                           \
            std::cerr << "CUDA error " << cudaGetErrorString(err)           \
                      << " at " << __FILE__ << ":" << __LINE__ << "\n";     \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

// FP32 CPU referencia GEMM
void ref_gemm_fp32(const float* A, const float* B, float* C,
                   int M, int N, int K) {
    for (int i = 0; i < M; ++i) {
        for (int j = 0; j < N; ++j) {
            float sum = 0.0f;
            for (int k = 0; k < K; ++k) {
                sum += A[i * K + k] * B[k * N + j];
            }
            C[i * N + j] = sum;
        }
    }
}

// B_t: N x K row-major, a B transzponáltja
__global__ void wmma_gemm_fp16(const __half* A, const __half* B_t, float* C,
                               int M, int N, int K) {
    int row = blockIdx.y * 16;
    int col = blockIdx.x * 16;

    wmma::fragment<wmma::matrix_a, 16, 16, 16, __half, wmma::row_major> a_frag;
    wmma::fragment<wmma::matrix_b, 16, 16, 16, __half, wmma::col_major> b_frag;
```

```
    wmma::fragment<wmma::accumulator, 16, 16, 16, float> c_frag;

    wmma::fill_fragment(c_frag, 0.0f);

    for (int k = 0; k < K; k += 16) {
        wmma::load_matrix_sync(a_frag, A + row * K + k, K);
        wmma::load_matrix_sync(b_frag, B_t + col * K + k, K);
        wmma::mma_sync(c_frag, a_frag, b_frag, c_frag);
    }

    wmma::store_matrix_sync(C + row * N + col, c_frag, N, wmma::mem_row_major);
}

int main(int argc, char** argv) {
    int M = 1024, N = 1024, K = 1024;
    int iterations = 10;
    if (argc > 1) M = atoi(argv[1]);
    if (argc > 2) N = atoi(argv[2]);
    if (argc > 3) K = atoi(argv[3]);
    if (argc > 4) iterations = atoi(argv[4]);

    // M, N, K osztható 16-tal
    if (M % 16 || N % 16 || K % 16) {
        std::cerr << "M, N, K must be multiples of 16\n";
        return EXIT_FAILURE;
    }

    // Host FP32 mátrixok
    std::vector<float> A_h(M * K), B_h(K * N);
    for (int i = 0; i < M * K; ++i) A_h[i] = 2.0f * rand() / RAND_MAX - 1.0f;
    for (int i = 0; i < K * N; ++i) B_h[i] = 2.0f * rand() / RAND_MAX - 1.0f;
    // B transzponált N x K
    std::vector<float> B_t_h(N * K);
    for (int j = 0; j < N; ++j)
        for (int k = 0; k < K; ++k)
            B_t_h[j * K + k] = B_h[k * N + j];

    // CPU referencia (csak kis méretnél)
    std::vector<float> C_ref;
    if (M <= 2048 && N <= 2048 && K <= 2048) {
        C_ref.resize(M * N);
        ref_gemm_fp32(A_h.data(), B_h.data(), C_ref.data(), M, N, K);
    }

    // Device FP16 konverzió
    std::vector<__half> A_half(M * K), B_t_half(N * K);
    for (int i = 0; i < M * K; ++i) A_half[i] = __float2half(A_h[i]);
    for (int i = 0; i < N * K; ++i) B_t_half[i] = __float2half(B_t_h[i]);

    __half *A_dev, *B_t_dev;
    float* C_dev;
    CUDA_CHECK(cudaMalloc(&A_dev, M * K * sizeof(__half)));
    CUDA_CHECK(cudaMalloc(&B_t_dev, N * K * sizeof(__half)));
    CUDA_CHECK(cudaMalloc(&C_dev, M * N * sizeof(float)));
    CUDA_CHECK(cudaMemcpy(A_dev, A_half.data(), M * K * sizeof(__half),
↪   cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(B_t_dev, B_t_half.data(), N * K * sizeof(__half),
↪   cudaMemcpyHostToDevice));

    dim3 grid(N / 16, M / 16);
    dim3 block(32);

    // Warm-up
```

```
    for (int it = 0; it < 3; ++it)
        wmma_gemm_fp16<<<grid, block>>>(A_dev, B_t_dev, C_dev, M, N, K);
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaGetLastError());

    cudaEvent_t start, stop;
    CUDA_CHECK(cudaEventCreate(&start));
    CUDA_CHECK(cudaEventCreate(&stop));
    cudaEventRecord(start);
    for (int it = 0; it < iterations; ++it)
        wmma_gemm_fp16<<<grid, block>>>(A_dev, B_t_dev, C_dev, M, N, K);
    cudaEventRecord(stop);
    CUDA_CHECK(cudaDeviceSynchronize());
    CUDA_CHECK(cudaGetLastError());
    float elapsed_ms = 0.0f;
    cudaEventElapsedTime(&elapsed_ms, start, stop);

    double flops = 2.0 * (double)M * N * K;
    double tflops = flops / (elapsed_ms / 1000.0) / 1e12;
    std::cout << "FP16 WMMA GEMM: " << elapsed_ms / iterations << " ms avg, "
              << tflops << " TFLOPS\n";

    // Hiba számítás, ha van CPU referencia
    if (!C_ref.empty()) {
        std::vector<float> C_h(M * N);
        CUDA_CHECK(cudaMemcpy(C_h.data(), C_dev, M * N * sizeof(float),
↪   cudaMemcpyDeviceToHost));
        float max_abs = 0.0f;
        for (int i = 0; i < M * N; ++i)
            max_abs = std::max(max_abs, std::fabs(C_h[i] - C_ref[i]));
        std::cout << "Max absolute error vs FP32 reference: " << max_abs << "\n";
    }

    CUDA_CHECK(cudaFree(A_dev));
    CUDA_CHECK(cudaFree(B_t_dev));
    CUDA_CHECK(cudaFree(C_dev));
    CUDA_CHECK(cudaEventDestroy(start));
    CUDA_CHECK(cudaEventDestroy(stop));
    return 0;
}
```

A kódban a BF16 és TF32 kernel hasonló; a BF16-hoz a `__nv_bfloat16` típus és a `wmma::precision::bf16` fragment, a TF32-höz a `wmma::precision::tf32` fragment szükséges. A laborban ezeket a kerneleket a hallgató a fenti minta alapján egészítheti ki.

23.3.9 Build és futtatás

```
# Build (példa sm_80-ra, A100)
nvcc -arch=sm_80 -std=c++17 -O3 -I. mixed_gemm.cu -o mixed_gemm

# Futtatás a kis méreten, hibakiértékeléssel
./mixed_gemm 1024 1024 1024 10

# Futtatás a nagy méreten, sebességméréshez
./mixed_gemm 8192 8192 8192 10
```

A laborhoz rögzíteni kell a környezetet. Példa a mérési napló fejlécére:

23.3.10 Mérési napló (a hallgató által kitöltendő)

Paraméter	Érték
GPU	NVIDIA A100-SXM4-40GB
Compute capability	8.0
Driver	535.104.05
CUDA toolkit	12.2
nvcc	12.2
gcc	11.4
OS	Ubuntu 22.04 LTS
Power limit	400 W
ECC	enabled
MIG	off
Háttérterhelés	nincs
Mátrix méret (sebesség)	8192×8192×8192
Mátrix méret (pontosság)	1024×1024×1024
Warm-up	3 iteráció
Mérési iterációk	10

Várt és mért eredmények *(a várt oszlop a gyártói névleges Tensor Core csúcsot mutatja; a mért oszlopot a saját mérés alapján kell kitölteni)*:

Precision	Várt peak [TFLOPS]	Mért átlag [TFLOPS]	Mért szórás [TFLOPS]	Max abszolút hiba (1024)
FP32 (referencia)	~19,5 (CUDA mag)	–	–	0 (definíció szerint)
FP16 WMMA	~312	–	–	–
BF16 WMMA	~312	–	–	–
TF32 WMMA	~156	–	–	–

A táblázatban a várt értékek egy adott GPU-modellre vonatkozó tájékoztató jellegű csúcsok, nem pedig a ténylegesen elérhető teljesítmény. A mérés után a mért TFLOPS-oszlopba a saját kernel hatékonyságát tükröző érték kerül; a hibaoszlopba a FP32 CPU referencia ellen számított maximális abszolút hiba. A mérési naplóban minden mért értéket fel kell tüntetni, és ha a környezet eltér, azt dokumentálni kell.

23.3.11 Önellenőrző kérdések

1. Miért használunk FP32 akkumulátort az FP16 bemenetű Tensor Core GEMM-ben, és mi történik, ha az akkumulátor is FP16?
2. Melyik formátumot választanád, ha a bemeneti értékek tartománya 10 és 10 között van? Indokold a döntést.
3. Mi a TF32, és miért nem önálló tárolási típus?
4. Hogyan csökkenthető az FP16 túlcsordulásából adódó hiba? Nevezz meg legalább két megoldást.
5. Miért fontos a pontossági mérést reprezentatív tesztadatokkal végezni, és nem csak egyetlen véletlen eloszlással?

23.4 Tensor Core optimalizálás

A Tensor Core kernel akkor hozza a hardveres lehetőségekhez közeli teljesítményt, ha a mátrix-tile-ok alakja, a memóriaelrendezés, a shared memory kihasználtság és az SM erőforrás-gazdálkodás összehangolt. A Mixed precision alfejezetben bevezetett WMMA és CUTLASS alapokra építve ez a fejezet azt vizsgálja, hogyan hangolható egy tensor core GEMM kernel a szűk keresztmetszetek feloldására. A cél nem a névleges csúcsérték hajszolása, hanem a kernel erőforrás-korlátjainak azonosítása és a legnagyobb hatású javítás bevezetése.

A hangolási teret három fő dimenzióra bonthatjuk:

1. **Shape:** a makro-tile és a mikro-tile dimenziói (M, N, K), valamint a K ciklus felbontása.
2. **Layout:** a mátrixok memóriaelrendezése, a leading dimension, a shared memory swizzle és a fragment layout.
3. **Occupancy és shared memory:** a regiszter- és shared memory korlátok, a double buffering és a warpok száma.

Ezek a dimenziók erősen összefüggenek: a nagyobb tile több regisztert és shared memóriát igényel, ami csökkenti az occupancy-t, miközben csökkenti a globális memória forgalmat. A feladat az egyensúly megtalálása az adott hardveren.

23.4.1 Shape és layout

A GEMM kernelben a shape három szinten jelenik meg. A makro-tile (például 64×64) az a mátrixdarab, amelyet egy thread block számít ki; a mikro-tile (például 16×16×16) az a WMMA vagy `mma.sync` utasítás, amelyet egy warp hajt végre; a K-tile pedig az a közös dimenzió-darab, amelyet egy iterációban a shared memóriában tartunk. A makro-tile mérete határozza meg, hogy hány WMMA mikro-tile-t kell egy blocknak lefednie, és ezzel a regiszternyomást.

A WMMA API-ban a mikro-tile alakja kötött: `m16n16k16`, `m16n16k8`, `m32n8k16` stb. A makro-tile dimenzióit ezek többszörösére érdemes választani. Például egy 64×64-es makro-tile 4×4=16 darab 16×16-os WMMA tile-t tartalmaz. Ha a block 256 szálat indít (8 warp), akkor minden warp két 16×16-os tile-t is számolhat, ami növeli a regiszterek újrafelhasználását, de növeli a regiszterigényt is. A 128×128-as makro-tile 8×8=64 WMMA tile-t igényel, ami már nagy block mérettel vagy iterált warp munkával oldható meg.

A K-tile mérete közvetlenül befolyásolja a shared memory méretét. Egy 64×16-os A-tile és egy 16×64-es B-tile FP16-ban: A = 64×16×2 = 2048 byte, B = 16×64×2 = 2048 byte, összesen 4096 byte. Dupla pufferezés esetén 8192 byte, ami elhanyagolható a 164 KB-os A100 shared memóriához képest. Ha a K-tile-t 32-re növeljük, a shared memória 8192 byte-ra nő, duplán 16384 byte; ez még mindig kicsi. A K-tile növelése csökkenti a K ciklus iterációinak számát, így a `__syncthreads()` és a `load_matrix_sync` hívások számát is, ami javíthatja a hatékonyságot. Ugyanakkor a nagyobb K-tile hosszabb `__syncthreads()` várakozást okozhat, mert a block minden szálának meg kell várnia, amíg a leglassabb betöltés véget ér.

A layout a mátrixok memóriaelrendezését jelenti. A WMMA `load_matrix_sync` függvény `layout` paramétere (`row_major` vagy `col_major`) azt mondja meg, hogy a globális vagy shared memóriában a mátrix sor- vagy oszlopfolytonos. Ha a tényleges elrendezés és a megadott layout eltér, a hardver transzponálást végez, ami extra belső mozgást és sávszélességet fogyaszt. A GEMM esetén gyakori, hogy A sor-major, B pedig oszlop-major; ez megfelel a cuBLAS oszlop-major tárolási konvenciójának. A WMMA B fragment `col_major`-ként való betöltése ilyenkor természetes, mert a B mátrix transzponáltját képviseli a szorzásban.

A leading dimension (`lda`, `ldb`, `ldc`) szintén a layout része. A tömbök sorainak kezdőcímeit nemcsak a dimenziók, hanem a padding is meghatározza. A `load_matrix_sync` stride paramétere elemekben adott; ha a mátrix sorai 128 bájtra igazítottak, a stride ennek megfelelő. Például FP16 esetén 128 bájt = 64 elem, ezért ha a sor hossza 60, érdemes 64-re párnázni. Az igazítás a vektorizált betöltést és az `ldmatrix` hatékonyságát javítja.

A shared memory layout a bank conflictok elkerülésére szolgál. A shared memory 32 bankba szerveződik; ha egy warp tagjai egyszerre ugyanabba a bankba címződnek, a hozzáférés sorosodik. FP16 esetén két elem fér egy 32 bites bank-szóba. A WMMA `load_matrix_sync` shared memóriából történő betöltése a belső `ldmatrix`-szal 8×8-as 128 bites szegmenseket olvas; a bank conflict elkerüléséhez a tile sorait megfelelően el kell tolni. Egy gyakori trükk a shared memória tömbök sorhosszának paddingje:

```
// CUDA C++17, NVCC 12.4, SM 8.0
__shared__ half As[TILE_M][TILE_K + 4];
__shared__ half Bs[TILE_K][TILE_N + 4];
```

A `+4` elem eltolja a sorok kezdetét, így a különböző sorok ugyanazon oszlopindexű elemei nem ugyanabba a bankba esnek. A padding méretét a bankek száma és az adattípus határozza meg; FP16 esetén a `+4` vagy `+8` általában elegendő, de a konkrét mintát méréssel vagy bank conflict metrikával ellenőrizni kell.

A CUTLASS keretrendszerben a shape és a layout explicit sablonparaméter. A `cutlass::gemm::GemmShape<M, N, K>` adja meg a makro-tile-t, a `LayoutA`, `LayoutB`, `LayoutC` típusok a memóriaelrendezést, a `Swizzle` pedig a shared memory swizzle-t. A CUTLASS az `ldmatrix` és a `mma.sync` utasításokat generálja, így a hangolás nagyrészt a megfelelő tile shape és swizzle kiválasztását jelenti. A WMMA API ezzel szemben kevésbé rugalmas, de a padding és a layout megfelelő megválasztásával hasonló hatás érhető el.

23.4.2 Occupancy és shared memory

Az occupancy azt mutatja, hogy az SM aktív warpjainak száma hány százaléka az elméleti maximumnak. A Tensor Core kernel esetén az occupancy nem önmagában cél, hanem a latency hiding eszköze: ha kevés warp van, a shared memória betöltések és a tensor core utasítások közötti függőségek miatt az SM várakozni kényszerül. Ha túl sok warp van, a regiszter- és shared memory korlátok miatt a tile méretét csökkenteni kell, ami viszont a globális memória forgalmat növeli.

A fő erőforrás-korlátok az A100 SM-ben: 2048 szál, 64 warp, 65536 darab 32 bites regiszter, 164 KB shared memory (konfigurálható, de jellemzően 164 KB a maximum). Egy WMMA GEMM kernelben a regisztereket a fragmentek (accumulator C fragment, A és B fragmentek), a belső indexek és a compiler által generált ideiglenes értékek foglalják el. A shared memóriát a tile-ok, a double buffer és a padding foglalja.

Tegyük fel, hogy a kernel 256 szálat indít blockonként, és a compiler jelentése szerint 96 regisztert használ szálanként. Ekkor egy block $256 \times 96 = 24576$ regisztert foglal. A 65536 regiszteres limit mellett 2 block fér el SM-enként, ami 512 szál, azaz 16 warp. Az A100 maximum 64 warpja helyett ez 25% occupancy. Ha a regiszterhasználatot 64-re csökkentjük, 4 block fér el, 1024 szál, 32 warp, 50% occupancy. A shared memória ebben a példában 16 KB (dupla pufferezett 64×16 A és 16×64 B + padding), így nem korlátoz.

A regisztercsökkentésnek ára van: ha a compiler nem tudja elhelyezni a fragmenteket regiszterekben, spill keletkezik, azaz a regiszter tartalmát a local memóriába (global memória L1/L2 mögött) írja ki. Ez drasztikusan rontja a teljesítményt. A `__launch_bounds__` qualifierrel a block méretet

és a minimális blockok számát adhatjuk meg; a compiler ennek figyelembevételével optimalizálja a regiszterhasználatot. Az `-maxrregcount` kapcsolóval pedig felső korlátot adhatunk, de ezt óvatosan kell használni, mert spillhez vezethet.

A shared memory méretének növelése – például nagyobb K-tile vagy double buffering – szintén csökkenti az occupancy-t, ha a shared memory eléri a korlátot. Az A100 164 KB-ja mellett egy 128×64×32-es makro-tile dupla pufferezéssel: A = 128×32×2 = 8192 B, B = 32×64×2 = 4096 B, összesen 12288 B, duplán 24576 B = 24 KB, ami még bőven elfér. Ha a K-tile 64-re nő, a dupla puffer 48 KB, ami még mindig kevesebb, mint 164 KB, de a shared memory nagyobb része egy blokkhoz tartozik, így kevesebb block fér el. Az optimális általában az, ha a shared memory nem a szűk keresztmetszet, hanem a regiszterek korlátozzák az occupancy-t.

A double buffering (kettős pufferezés) célja, hogy a K ciklusban a betöltés és a számítás átfedésben legyen. Az egyik pufferbe töltünk, míg a másikból számolunk. A CUDA 11.0-tól elérhető `cp.async` (aszinkron másolás) lehetővé teszi, hogy a globális memóriából a shared memóriába másolás a szálak szinkronizálása nélkül is folyamatban legyen, és a `commit_group`/`wait_group` műveletekkel szabályozható. A WMMA kernelben a double buffering a `__syncthreads()` várakozásokat is csökkenti. A növekvő shared memory igény és a foglalási overhead azonban az occupancy-t csökkentheti, ezért a double buffering bevezetése előtt érdemes az egyszerű, egy pufferes változatot megmérni.

Egy `cp.async`-kel megvalósított double buffering vázlata:

```
// CUDA C++20, NVCC 12.4, SM 8.0
// cp.async double buffering illusztráció
__shared__ half As[2][TILE_M][TILE_K + 4];
__shared__ half Bs[2][TILE_K][TILE_N + 4];
int buf = 0;

// előzetes betöltés az első pufferbe (itt cp.async hívások)
// cp_async(&As[buf][0][0], &A_src[0], ...);
// cp_async(&Bs[buf][0][0], &B_src[0], ...);
commit_group();

for (int k0 = TILE_K; k0 < K; k0 += TILE_K) {
    // következő puffer betöltése
    // cp_async(&As[1-buf][0][0], &A_src[k0], ...);
    // cp_async(&Bs[1-buf][0][0], &B_src[k0*N], ...);
    commit_group();
    wait_group(1); // várunk az előző betöltésre, a mostani még folyamatban

    // számítás a buf pufferből
    // ...
    buf = 1 - buf;
    __syncthreads();
}
```

A `wait_group(1)` biztosítja, hogy a legutolsó `commit_group` előtti csoport befejeződött, miközben a legutolsó betöltés még folyamatban van. Így a betöltés és a számítás átfedésben marad.

23.4.3 Gyakorlati WMMA kernel és optimalizálás

Az alábbi példa egy FP16 WMMA GEMM kernel vázát mutatja, amely a fenti szempontokat alkalmazza. A kernel 64×64-es makro-tile-t számít, 16×16×16-os WMMA mikro-tile-okkal, dupla pufferezett shared memóriával, paddinggel és a mátrix layoutokhoz igazított betöltéssel.

```
// CUDA C++17, NVCC 12.4, SM 8.0 (A100)
#include <mma.h>
using namespace nvcuda;

#define TILE_M 64
#define TILE_N 64
#define TILE_K 16
#define WARP_M 16
#define WARP_N 16
#define WARP_K 16

__global__ void wmma_gemm_optimized(const half* __restrict__ A,
                                    const half* __restrict__ B,
                                    float* __restrict__ C,
                                    int M, int N, int K) {
    // Shared memory tile-ok, +4 padding a bank conflict elkerülésére
    __shared__ half As[TILE_M][TILE_K + 4];
    __shared__ half Bs[TILE_K][TILE_N + 4];

    int bx = blockIdx.x;
    int by = blockIdx.y;
    int tx = threadIdx.x;
    int warp_id = tx / 32;
    int lane = tx % 32;

    // A block 256 szál, 8 warp; minden warp két 16x16-os tile-t számol
    int warp_row = (warp_id % 4) * WARP_M;   // 0,16,32,48
    int warp_col = (warp_id / 4) * WARP_N;   // 0,16,32,48

    // Accumulator fragmentek: 2x2 WMMA tile / warp
    wmma::fragment<wmma::accumulator, 16, 16, 16, float> c_frag[2][2];
    #pragma unroll
    for (int i = 0; i < 2; ++i)
        for (int j = 0; j < 2; ++j)
            wmma::fill_fragment(c_frag[i][j], 0.0f);

    // Globális memória mutatók a makro-tile kezdetére
    const half* A_tile = A + (by * TILE_M) * K;
    const half* B_tile = B + (bx * TILE_N);
    float* C_tile = C + (by * TILE_M) * N + bx * TILE_N;

    for (int k0 = 0; k0 < K; k0 += TILE_K) {
        // Betöltés globális -> shared (coalesced)
        // A mátrix: 64x16 elem; B mátrix: 16x64 elem
        // Minden szál 4 elemet tölt be A-ból és 4 elemet B-ből
        #pragma unroll
        for (int t = 0; t < 4; ++t) {
            int row_a = tx / (TILE_K / 4);
            int col_a = (tx * 4 + t) % TILE_K;
            if (row_a < TILE_M && k0 + col_a < K)
                As[row_a][col_a] = A_tile[row_a * K + k0 + col_a];

            int row_b = (tx * 4 + t) % TILE_K;
            int col_b = tx / (TILE_K / 4);
            if (row_b < TILE_K && col_b < TILE_N)
                Bs[row_b][col_b] = B_tile[(k0 + row_b) * N + col_b];
        }
        __syncthreads();

        // Számítás a shared memóriából
        #pragma unroll
        for (int kk = 0; kk < TILE_K / WARP_K; ++kk) {
            wmma::fragment<wmma::matrix_a, 16, 16, 16, half, wmma::row_major> a_frag;
```

```
            wmma::fragment<wmma::matrix_b, 16, 16, 16, half, wmma::col_major> b_frag;
            wmma::load_matrix_sync(a_frag, &As[warp_row][kk * WARP_K], TILE_K + 4);
            wmma::load_matrix_sync(b_frag, &Bs[kk * WARP_K][warp_col], TILE_N + 4);
            #pragma unroll
            for (int i = 0; i < 2; ++i)
                for (int j = 0; j < 2; ++j)
                    wmma::mma_sync(c_frag[i][j], a_frag, b_frag, c_frag[i][j]);
        }
        __syncthreads();
    }

    // C kiírása a globális memóriába
    // Minden warp a saját 2x2 WMMA tile-ját írja ki
    // Itt elhagyjuk a teljes indexelést; a labor kódban megtalálható.
}
```

Ez a vázlat a hangolási pontokat illusztrálja, de a teljes labor kód ennél bővebb: tartalmazza a hibakezelést (`CUDA_CHECK`), a baseline naiv WMMA kernelt, a double buffering `cp.async` változatot és a mérési ciklust.

A fenti kernelben a hangolási döntések:

- A makro-tile 64×64, ami 4×4 WMMA mikro-tile-t jelent. A 8 warp esetén minden warp 2×2-es almátrixot számít, ami növeli a regiszterek újrafelhasználását anélkül, hogy a block méretét 512-re növelnénk.
- A K-tile 16, ami egyetlen WMMA K-lépésnyi adatot tart a shared memóriában. A shared memória mérete dupla pufferezéssel 8 KB, ami alacsony, így az occupancy-t a regiszterek korlátozzák.
- Az A mátrix `row_major`, a B mátrix `col_major` layoutú betöltése illeszkedik a tipikus cuBLAS tároláshoz, és elkerüli a transzponálást.
- A shared memória tömbök sorhosszát `+4` paddinggel növeltük, ami a bank conflictokat csökkenti. A betöltési mintát a `tx` és `t` indexek úgy választják, hogy a globális memória hozzáférés coalesced legyen; a pontos mintát a labor kódban finomhangoljuk.

23.4.4 Nsight-alapú elemzés

A hangolás nem alapulhat kizárólag futásidőre; a szűk keresztmetszetet profilkérre van szükség. Az NVIDIA Nsight Compute (`ncu`) a kernel alatt gyűjtött hardvermetrikákat jeleníti meg. A tensor core kernel elemzéséhez a következő metrikák a legfontosabbak:

- `sm__pipe_tensor_cycles_active.avg.pct_of_peak_sustained_elapsed`: a tensor core cső aktív ciklusainak aránya. Az alacsony érték azt jelzi, hogy a tensor core vár adatra vagy szinkronizációra.
- `sm__inst_executed_pipe_tensor_op.avg`: a végrehajtott tensor core utasítások száma; a WMMA és `mma.sync` hívások száma.
- `l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum`: a shared memória olvasási bank conflictok száma. A cél a nullához közeli érték.
- `launch__registers_per_thread`: a tényleges regiszterhasználat szálanként.
- `launch__occupancy_limit_registers`: az occupancy-t korlátozó tényező (regiszter, shared memory, block méret).
- `sm__warps_launched`: a launched warpok száma; az occupancy számításához.
- `gpu__time_duration.sum`: a kernel futási ideje.

Egy tipikus parancs:

```
ncu --set full --kernel-name wmma_gemm_optimized --launch-count 1 --export report.csv
↪  ./wmma_gemm 4096 4096 4096
```

A `--set full` minden metrikát begyűjt, ami lassú lehet, de a teljes képhez hasznos. A gyakorlatban a `--section SpeedOfLight` és a `--section Occupancy` gyorsabb áttekintést ad.

Az elemzés során először a `sm__pipe_tensor_cycles_active` értékét nézzük. Ha ez alacsony, a kernel nem számol eleget; a stall okokat a `smsp__average_warps_issue_stalled_*` metrikák részletezik. A gyakori stall okok: `wait`, `barrier`, `long_scoreboard` (globális memória betöltés), `math_pipe_throttle`, `mio_throttle`. A tensor core kernelnél a `long_scoreboard` arra utal, hogy a shared memóriából vagy globális memóriából történő betöltés késik; ilyenkor a double buffering vagy a prefetch segít. A `barrier` stall a `__syncthreads()` várakozásra utal; a double buffering és a `cp.async` csökkenti.

A bank conflict metrika nullához közelinek kell lennie. Ha magas, a shared memory paddinget vagy a swizzle-t kell módosítani. Az `ncu` a forráskód soraihoz köthető bank conflictokat is megjeleníti, ami segít a pontos hely azonosításában.

23.4.5 Tipikus hibák és javítások

A tensor core kernel hangolása során a következő hibák fordulnak elő leggyakrabban, és a javításuk is tanulságos:

1. **Nem megfelelő layout.** Ha az A vagy B mátrixot `row_major` helyett `col_major`-ként adjuk át a `load_matrix_sync`-nek, a hardver transzponál, ami lassabb betöltést és extra regiszternyomást okoz. A javítás: a layout paramétert a tényleges memóriaelrendezéshez igazítani; ha szükséges, a mátrixot előzetesen transzponálni.

2. **Nincs padding a shared memóriában.** A `__shared__ half As[64][16]` esetén a sorok 16 elemből állnak, ami FP16-ban 32 bájt, pontosan 8 bank. Minden sor azonos bankokkal kezdődik, így a WMMA betöltés szisztematikus bank conflictot okoz. A `+4` padding megtöri ezt. A javítás: a shared memory tömbök sorhosszát a bankek számához képest páratlanra vagy eltoltra állítani.

3. **Túl nagy makro-tile.** Ha a makro-tile 128×128-ra nő, a C fragmentek 4×4=16 WMMA accumulator fragmentet jelentenek warponként, ami $16 \times 8 \times 32 = 4096$ regisztert is elfoglalhat, nem is beszélve az A és B fragmentekről. A compiler spillhez folyamodik, és az occupancy drámaian csökken. A javítás: a makro-tile méretét a regiszter limithez igazítani, vagy a C fragmenteket részben kívül tartani, vagy CUTLASS-t használni, amely optimalizált fragment schedule-t generál.

4. **Túl kicsi K-tile.** Ha a K-tile 8, a K ciklus iterációinak száma nagy, és minden iterációban `__syncthreads()` és betöltés van. A tensor core cső sokat vár. A javítás: a K-tile-t 16-ra vagy 32-re növelni, miközben a shared memória még elfér.

5. **Nincs double buffering.** Az egy pufferes kernel minden K lépésben `__syncthreads()`-szel vár, amíg minden betöltés befejeződik, majd a számítás alatt a shared memória foglalt. A double buffering lehetővé teszi, hogy az egyik puffer betöltése a másik pufferből történő számítással párhuzamosan folyjon. A javítás: két puffer bevezetése és a `cp.async` használata; a `commit_group`/`wait_group` műveletekkel a megfelelő szinkronizáció.

6. **Rossz granularitású betöltés.** A globális memóriából a shared memóriába történő betöltésnél a szálak indexelése nem coalesced, vagy a betöltés nem használja ki a 128 bites vektorizált hozzáférést. A javítás: a betöltési mintát úgy alakítani, hogy a szomszédos szálak szomszédos címeket olvassanak, és a hozzáférés `half2` vagy `uint4` típusú legyen.

7. **Hiányzó hibakezelés a laborban.** A CUDA hívások után nincs `CUDA_CHECK`, így a hibás kernel nem derül ki időben. A javítás: minden API hívás után `cudaGetLastError` és `cudaDeviceSynchronize` ellenőrzése, ahogyan a fejezet laborja is előírja.

23.4.6 Labor és mérési napló

A labor célja, hogy a fenti hangolási szempontokat egy konkrét WMMA GEMM kernelen mérje. A kiindulópont egy baseline kernel, amely nem használ double bufferinget, nincs padding a shared memóriában, és a layout nem optimális. Az optimalizált kernel bevezeti a 64×64×16-os makro-tile-t, a `+4` paddinget, a dupla pufferezést `cp.async`-kel és a mátrix layoutokhoz igazított betöltést.

23.4.7 Környezet és parancsok

A mérést az alábbi környezetben végezzük (a laborban rögzítendő értékek):

- GPU: NVIDIA A100-SXM4-40GB, compute capability 8.0
- CUDA: 12.4
- Driver: 550.54.14
- OS: Ubuntu 22.04 LTS
- ECC: bekapcsolva
- Power limit: 400 W
- nvidia-smi: 550.54.14
- Izolált labor, háttérterhelés nélkül

Build és futtatás:

```
nvcc -arch=sm_80 -std=c++17 -O3 -I. -o wmma_gemm wmma_gemm.cu
./wmma_gemm 4096 4096 4096 5
```

A program a 4096^3-as FP16 GEMM-et számítja, warm-up után 5 ismétlést mér, és kiírja az átlagot, a szórást és a TFLOPS-ot.

A Nsight Compute futtatás:

```
ncu --set full --kernel-name wmma_gemm_baseline --launch-count 1 ./wmma_gemm 4096 4096 4096 1
ncu --set full --kernel-name wmma_gemm_optimized --launch-count 1 ./wmma_gemm 4096 4096 4096 1
```

23.4.8 Várt és mért eredmények

A várt érték a roofline modellből és a kernel erőforrás-számításából adódik. A 4096^3 GEMM $2\times4096^3 = 137.4$ GFLOPS. Az A100 FP16 tensor core dense csúcsa 312 TFLOPS, a memória sávszélessége 1.5 TB/s. A kernel számítási intenzitása a K ciklusban a shared memóriából olvasott adatokra vetítve 21.3 FLOP/byte, ami bőven a compute bound tartományban van. A várható elért teljesítmény a tensor core kihasználtságtól függ: 60%-os kihasználás 187 TFLOPS, 40% 124 TFLOPS. A baseline, amely szinkronizációs és bank conflict problémákkal küzd, várhatóan 60 TFLOPS alatt marad.

Az alábbi táblázat a mérési napló szerkezetét mutatja. A „Mért (példa)” oszlop **nem valós mérés**, hanem a módszertan bemutatására szolgáló kitöltési minta; a laborban a hallgató az oszlopot a saját mérési eredményeivel tölti ki.

Változat	M=N=K	Átlag idő (ms)	Szórás (ms)	Elért TFLOPS	Várt TFLOPS (modell)	Occupancy	Regiszter/szál	Shared memória/block	Bank conflict (ld)
baseline	4096	2.35	0.08	58.4	60.0	25%	128	8 KB	12480
optimized (double buffer + swizzle + 64×64×16)	4096	1.10	0.03	124.8	130.0	50%	96	16 KB	12
optimized + cp.async double buffer	4096	0.95	0.02	144.6	150.0	50%	96	24 KB	8

A táblázatban a „Várt TFLOPS (modell)” a roofline és a kihasználtság becsléséből származik. A mért érték a valós futásból, a `cudaEvent` időmérésből és a Nsight metrikákból adódik. A laborban a mért értékeket legalább 5 ismétlésből kell számítani, a szórással együtt. A bank conflict metrika a `l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum` értéke; a cél a nullához közeli.

A Nsight jelentés értelmezése: a baseline kernelnél a `sm__pipe_tensor_cycles_active` alacsony, a `barrier` stall magas, a bank conflict nagy. Az optimalizált kernelnél a tensor pipe aktív aránya nő, a barrier stall csökken, a bank conflict közel nulla. Ha az occupancy 50%-on stagnál, a korlátozó tényező a regiszter; a `launch__occupancy_limit_registers` ezt mutatja. A következő lépés a regisztercsökkentés lenne, de csak akkor, ha a spill nem nő.

23.4.9 Önellenőrző kérdés

Miért csökken a shared memory bank conflict, ha a `__shared__ half As[TILE_M][TILE_K]` deklarációt `__shared__ half As[TILE_M][TILE_K + 4]`-re változtatjuk? Indokolja meg a választ a bank-szervezés és a WMMA betöltési minta alapján.

23.4.10 Összegzés

A Tensor Core kernel hangolása három összefüggő döntésen múlik: a tile shape és a layout megválasztásán, a shared memory swizzle/padding kialakításán, valamint az occupancy és a double buffering egyensúlyán. A fejezetben bemutatott WMMA példa és a Nsight-alapú mérési folyamat ezeket a döntéseket konkrét metrikákkal teszi ellenőrizhetővé. A következő fejezet (GEMM és a deep learning) ezt a tudást a deep learning kernelekre – a GEMM, a konvolúció és a figyelem – alkalmazza, ahol a shape és a mixed precision választás még kritikusabb.

24 Deep learning kernelek: GEMM, conv és attention

24.1 GEMM és a deep learning

24.1.1 A lineáris réteg és a GEMM kapcsolata

A mélytanulási hálózatok leggyakoribb építőköve a **lineáris réteg**, amelyet sok keretrendszer *fully connected*, *dense* vagy *linear* néven ismer. A réteg matematikailag egy affin transzformációt hajt végre:

$$\mathbf{Y} = \mathbf{X}\,\mathbf{W}^{\mathsf{T}} + \mathbf{b},$$

ahol

- $\mathbf{X} \in \mathbb{R}^{N \times K}$ a bemeneti köteg (batch), N mintával és K bemeneti jellemzővel,
- $\mathbf{W} \in \mathbb{R}^{M \times K}$ a súlymátrix, M kimeneti jellemzővel,
- $\mathbf{b} \in \mathbb{R}^{M}$ a torzítás (bias),
- $\mathbf{Y} \in \mathbb{R}^{N \times M}$ a kimeneti aktiváció.

A kifejezés egy mátrixszorzásból és egy soronkénti eltolásból áll. A mátrixszorzás a **GEMM** (general matrix multiply) művelet, amely a számítási igény túlnyomó részét adja. Egyetlen kimeneti elem kiszámítása K szorzást és $K-1$ összeadást igényel, így a teljes réteg műveletigénye

$$2\,N\,M\,K + N\,M$$

lebegőpontos művelet, ahol az utolsó tag a bias hozzáadásából származik. Nagy K és M esetén a bias elhanyagolható, és a művelet gyakorlatilag tisztán GEMM.

A CPU-n megszokott mátrixszorzás-optimalizálások (blokkosítás, vektorizáció, cache-barát sorrend) a GPU-n is érvényesek, de a GPU hierarchikus végrehajtási modellje miatt a tile-ok mérete, a shared memory használata és a warp-szintű adatcsere válik kulcskérdéssé. A lineáris réteg GPU-ra vitele ezért nem egyszerűen egy könyvtári hívás: meg kell értenünk a batch és layout fogalmát, a GEMM hívási konvencióit, és azt, hogy a különböző könyvtárak (cuBLAS, CUTLASS) hogyan térképezik a réteget hardveres erőforrásokra.

24.1.2 Batch és layout

A mélytanulási keretrendszerekben a bemeneti adatokat általában kétféle elrendezésben tárolják:

- **row-major (C-szerű) layout:** az utolsó index változik leggyorsabban. A $\mathbf{X}$ mátrix (n, k) eleme a memóriában az $n \cdot K + k$ pozíción van.

- **column-major (Fortran-szerű) layout:** az első index változik leggyorsabban. Itt az (n, k) elem pozíciója $k \cdot N + n$.

A CUDA-ökoszisztéma alapértelmezésben a row-major tárolást használja a saját kerneleknél, de a cuBLAS – a BLAS szabványnak megfelelően – **column-major** tárolást vár. Ez a forrása a leggyakoribb hibáknak, mert ugyanaz a $N \times K$ mátrix row-major és column-major tárolásban transzponáltnak tűnik.

Tekintsük a lineáris réteg számítását:

$$\mathbf{Y} = \mathbf{X}\mathbf{W}^\mathsf{T} + \mathbf{b}.$$

Ha $\mathbf{X}$ row-major, akkor a memóriában a $N \times K$ mátrix sorai egymás után helyezkednek el. A $\mathbf{W}$ súlymátrixot általában (M, K) alakban tároljuk, ahol a sorok a kimeneti neuronok, az oszlopok a bemeneti jellemzők. A GEMM számára ez azt jelenti, hogy

$$\mathbf{C} = \mathbf{A} \cdot \mathbf{B}^\mathsf{T},$$

ahol $\mathbf{A} = \mathbf{X}$ $(N \times K)$, $\mathbf{B} = \mathbf{W}$ $(M \times K)$, és a $\mathbf{B}$-t transzponálni kell, hogy a belső dimenzió K legyen. A cuBLAS-ban ezt az `CUBLAS_OP_T` (transzponált) művelettel adjuk meg.

A **leading dimension** (vezető dimenzió) fogalma a BLAS-ban elengedhetetlen. Egy $N \times K$ mátrixnál a leading dimension (általában `lda`) azt adja meg, hogy a memóriában hány elem van két szomszédos sor kezdőcíme között. Row-major tárolásnál `lda = K`, mert egy sor hossza K elem. Column-major tárolásnál `lda = N`, mert egy oszlop hossza N elem. Ha a mátrixot nagyobb memóriaterületbe ágyazzuk (például paddinggel), a leading dimension eltérhet a logikai dimenziótól. A cuBLAS hívásokban a leading dimension paramétereket mindig a tényleges memóriaelrendezéshez kell igazítani.

Batch feldolgozás azt jelenti, hogy a bemeneti $\mathbf{X}$ mátrix első dimenziója a batch méret N. A lineáris réteg egyetlen GEMM-mel feldolgozza az összes mintát. A kimenet $\mathbf{Y}$ alakja $N \times M$. Amennyiben a hálózat több, független réteget futtat párhuzamosan (például egy-egy minta különböző ágakon), használhatunk **batched GEMM**-et is, amely egyszerre több mátrixpárt szoroz össze, de ez a lineáris rétegnél ritkábban szükséges.

24.1.3 cuBLAS és a GEMM

A cuBLAS a NVIDIA által szállított, nagy teljesítményű BLAS implementáció. A lineáris réteg leggyakrabban a `cublasSgemm` (FP32), `cublasDgemm` (FP64), `cublasHgemm` (FP16) vagy a `cublasGemmEx` (vegyes pontosság) függvénnyel valósítható meg. A következő példa egy FP32 lineáris réteg forward passzát mutatja be cuBLAS-szal.

```
// CUDA C++ (C++17) - lineáris réteg cuBLAS-szal (FP32)
#include <cuda_runtime.h>
#include <cublas_v2.h>
#include <cstdio>
#include <cstdlib>

#define CHECK_CUDA(call)                                              \
    do {                                                              \
        cudaError_t err = call;                                       \
        if (err != cudaSuccess) {                                     \
            fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__,    \
                    __LINE__, cudaGetErrorString(err));               \
```

```
            exit(EXIT_FAILURE);                                         \
        }                                                               \
    } while (0)

#define CHECK_CUBLAS(call)                                              \
    do {                                                                \
        cublasStatus_t st = call;                                       \
        if (st != CUBLAS_STATUS_SUCCESS) {                              \
            fprintf(stderr, "cuBLAS error at %s:%d\n", __FILE__,        \
                    __LINE__);                                          \
            exit(EXIT_FAILURE);                                         \
        }                                                               \
    } while (0)

int main(int argc, char** argv) {
    int N = 1024;          // batch méret
    int K = 4096;          // bemeneti jellemzők
    int M = 4096;          // kimeneti jellemzők

    // Host memória (row-major, mint a keretrendszerekben)
    float* h_X = (float*)malloc(N * K * sizeof(float));
    float* h_W = (float*)malloc(M * K * sizeof(float));
    float* h_b = (float*)malloc(M * sizeof(float));
    float* h_Y = (float*)malloc(N * M * sizeof(float));

    // Inicializálás (pl. véletlen vagy determinisztikus értékek)
    for (int i = 0; i < N * K; ++i) h_X[i] = 0.001f * (i % 100);
    for (int i = 0; i < M * K; ++i) h_W[i] = 0.0001f * (i % 97);
    for (int i = 0; i < M; ++i)     h_b[i] = 0.01f;

    // Device memória
    float *d_X, *d_W, *d_b, *d_Y;
    CHECK_CUDA(cudaMalloc(&d_X, N * K * sizeof(float)));
    CHECK_CUDA(cudaMalloc(&d_W, M * K * sizeof(float)));
    CHECK_CUDA(cudaMalloc(&d_b, M * sizeof(float)));
    CHECK_CUDA(cudaMalloc(&d_Y, N * M * sizeof(float)));

    CHECK_CUDA(cudaMemcpy(d_X, h_X, N * K * sizeof(float), cudaMemcpyHostToDevice));
    CHECK_CUDA(cudaMemcpy(d_W, h_W, M * K * sizeof(float), cudaMemcpyHostToDevice));
    CHECK_CUDA(cudaMemcpy(d_b, h_b, M * sizeof(float), cudaMemcpyHostToDevice));

    cublasHandle_t handle;
    CHECK_CUBLAS(cublasCreate(&handle));

    const float alpha = 1.0f;
    const float beta  = 0.0f;  // a bias-t külön adjuk hozzá, mert a GEMM nem kezeli

    // Y = 1.0 * X * W^T + 0.0 * Y
    // A = X (N x K), B = W (M x K), C = Y (N x M)
    // A tárolás row-major, de a cuBLAS column-major-t vár, ezért
    // a transzponált műveletekkel kompenzálunk.
    CHECK_CUBLAS(cublasSgemm(
        handle,
        CUBLAS_OP_N,        // A-t nem transzponáljuk
        CUBLAS_OP_T,        // B-t transzponáljuk (W-ből W^T lesz)
        N,                  // sorok száma C-ben = batch
        M,                  // oszlopok száma C-ben = kimeneti jellemzők
        K,                  // belső dimenzió = bemeneti jellemzők
        &alpha,
        d_X, K,             // A row-major, leading dimension = K
        d_W, K,             // B row-major, leading dimension = K
        &beta,
        d_Y, M              // C row-major, leading dimension = M
```

```
    ));

    // Bias hozzáadása: Y(n,m) += b(m)
    // Ezt megtehetjük egy külön kernelben, vagy a GEMM beta=1 és
    // előre feltöltött bias mátrixszal, de egyszerűbb kernelhívással.
    // Itt csak a fogalmat mutatjuk: a valós kódban használjunk kernelt.
    // (A példa nem futtatott, ezért a bias kernel elhagyható.)

    CHECK_CUDA(cudaDeviceSynchronize());
    CHECK_CUDA(cudaMemcpy(h_Y, d_Y, N * M * sizeof(float), cudaMemcpyDeviceToHost));

    // Ellenőrzés CPU-val (itt elhagyva a hosszúság miatt)

    cublasDestroy(handle);
    cudaFree(d_X); cudaFree(d_W); cudaFree(d_b); cudaFree(d_Y);
    free(h_X); free(h_W); free(h_b); free(h_Y);
    return 0;
}
```

A példa jól mutatja a layout-konvenció fontosságát. A cuBLAS column-major bemenetet vár, ezért a row-major tárolású mátrixainkat transzponált műveletként adjuk át. A `d_X` mátrixot `CUBLAS_OP_N`-nel (nincs transzponálás) és `lda = K`-val adjuk meg, ami azt jelenti, hogy a cuBLAS számára ez egy $K \times N$ méretű column-major mátrix, valójában a mi $N \times K$ row-major mátrixunk. Hasonlóan a `d_W` súlymátrixot `CUBLAS_OP_T`-vel és `ldb = K`-val adjuk, ami a column-major konvencióban egy $K \times M$ mátrixként jelenik meg, éppen a transzponált $\mathbf{W}^\mathsf{T}$. A kimenet `d_Y` `ldc = M`-mel $M \times N$ column-major mátrix, ami a mi $N \times M$ row-major kimenetünk. Ez a leképezés működik, de aki nem figyel, könnyen felcserélheti a dimenziókat, és hibás eredményt kap.

A bias hozzáadását a GEMM önmagában nem végzi el; azt külön kernelben vagy egy második GEMM-mel (például egy $N \times M$ mátrix és egy $M \times 1$ vektor szorzatával) lehet megoldani. A gyakorlatban a keretrendszerek gyakran egy **fused** kernelt használnak, amely a GEMM epilógusában adja hozzá a bias-t, így elkerülve egy külön globális memória kört. Ezt a CUTLASS „epilogue” mechanizmusa támogatja.

A `cublasGemmEx` és a `cublasLt` lehetővé teszi vegyes pontosságú számítást, például TF32, FP16 vagy BF16 bemenettel és FP32 akkumulációval. Ez a modern GPU-k Tensor Core egységeit használja, és a lineáris réteg teljesítményét több nagyságrenddel növelheti, de a numerikus pontosságot gondosan ellenőrizni kell. A következő alfejezet (Convolution kernelek) részletesen foglalkozik a konvolúciós és attention kernelekkel, ahol a cuBLAS és a CUTLASS integrációja még fontosabb.

24.1.4 CUTLASS szerepe

A **CUTLASS** (CUDA Templates for Linear Algebra Subroutines) egy nyílt forráskódú sablonkönyvtár, amely a GEMM és más lineáris algebrai műveletek hatékony implementálásához ad építőelemeket. A cuBLAS-szal ellentétben a CUTLASS nem egy előre lefordított könyvtár, hanem C++ sablonok gyűjteménye, amelyek a fordítási időben specializálódnak a pontos adattípusra, tile méretre, warp elrendezésre és az úgynevezett **epilógus** műveletre.

A CUTLASS alapvető szerkezete a hierarchikus tiling:

1. **Thread block tile:** minden blokk egy C kimeneti mátrix blokkját számítja ki, például 128×128 elemet. A bemeneti A és B mátrixok megfelelő tile-jait shared memory-ba tölti.
2. **Warp tile:** a blokkon belül minden warp egy kisebb részt, például 64×64 elemet kap.
3. **Thread-level:** a warpen belül minden szál egy 8×8-as vagy 16×8-as mikro-tile-t számol, gyakran a Tensor Core `mma` (matrix multiply accumulate) utasításával.

A CUTLASS a **tensor core** használatát teszi elérhetővé a `mma` utasításokon keresztül, amelyek egy $16 \times 16 \times 16$-os vagy kisebb mátrixszorzást végeznek el egyetlen utasításban. A FP16, BF16 és TF32 bemenetekkel az FP32 akkumulátorban számoló `mma` utasítások a mélytanulási terhelések gerincét adják. A CUTLASS emellett támogatja az **epilógus** kiterjesztést, amely a GEMM kimenetére további műveleteket alkalmazhat, például bias hozzáadást, ReLU aktivációt vagy kvantálást, anélkül hogy a kimenetet visszaírnánk a globális memóriába. Ez különösen fontos a lineáris rétegnél, mert a bias hozzáadása és az aktiváció egy lépésben elvégezhető, csökkentve a memóriaforgalmat.

Egy CUTLASS alapú lineáris réteg váza a következőképpen nézhet ki:

```
// CUDA C++ (C++17) - CUTLASS vázlat (nem futtatott, csak koncepcionális)
#include <cutlass/gemm/device/gemm.h>
#include <cutlass/epilogue/thread/linear_combination_bias_relu.h>

using ElementA = float;
using ElementB = float;
using ElementC = float;
using Accumulator = float;

using ThreadblockShape = cutlass::gemm::GemmShape<128, 128, 32>;
using WarpShape        = cutlass::gemm::GemmShape<64, 64, 32>;
using InstructionShape = cutlass::gemm::GemmShape<16, 8, 8>;

using EpilogueOp = cutlass::epilogue::thread::LinearCombinationBiasRelu<
    ElementC, 128 / cutlass::sizeof_bits<ElementC>::value,
    Accumulator, Accumulator>;

using Gemm = cutlass::gemm::device::Gemm<
    ElementA, cutlass::layout::RowMajor,
    ElementB, cutlass::layout::RowMajor,
    ElementC, cutlass::layout::RowMajor,
    Accumulator,
    cutlass::arch::OpClassTensorOp,
    cutlass::arch::Sm80,
    ThreadblockShape, WarpShape, InstructionShape,
    EpilogueOp,
    cutlass::gemm::threadblock::GemmIdentityThreadblockSwizzle<>,
    2>;

// A tényleges hívásnál a CUTLASS elvégzi a tilinget, a shared memória
// feltöltést és az mma utasításokat. A fenti sablon paraméterezés
// illusztráció, a pontos típusok a CUTLASS verziójától függenek.
```

A CUTLASS előnye, hogy a fejlesztő pontosan szabályozhatja a tile méreteket, a warp elrendezést és az adatfolyamot, míg a cuBLAS egy fekete doboz. A hátránya a hosszabb fejlesztési idő és a sablonok bonyolultsága. A gyakorlatban a mélytanulási keretrendszerek (PyTorch, TensorFlow) sokszor a cuBLAS-t vagy a CUTLASS-t használják a lineáris réteg mögött, attól függően, hogy melyik adott hardveren gyorsabb.

24.1.5 Lineáris réteg saját kernelből: tiled GEMM

A tanulás szempontjából elengedhetetlen, hogy a lineáris réteget egy saját, oktatási célú kernelben is megvalósítsuk. Az alábbi CUDA kernel egy batch lineáris réteget számol shared memory tilinggel, bias hozzáadással. A kernel `TM = TN = TILE_DIM` méretű kimeneti tile-okat állít elő, és a bemeneti $\mathbf{X}$ valamint a transzponált $\mathbf{W}$ tile-jait shared memory-ba tölti.

```
// CUDA C++ (C++17) - lineáris réteg saját kernel, FP32, tiled GEMM
#define TILE_DIM 32

__global__ void linear_layer_kernel(
    const float* __restrict__ X,      // (N, K) row-major
    const float* __restrict__ W,      // (M, K) row-major
    const float* __restrict__ bias,   // (M) vagy nullptr
    float* __restrict__ Y,            // (N, M) row-major
    int N, int K, int M)
{
    // A block dim (TILE_DIM, TILE_DIM). A blockIdx.x az oszlop (M)
    // irányt, blockIdx.y a batch (N) irányt jelöli.
    int c_row = blockIdx.y * TILE_DIM + threadIdx.y;   // batch index
    int c_col = blockIdx.x * TILE_DIM + threadIdx.x;   // kimeneti index

    __shared__ float As[TILE_DIM][TILE_DIM];   // X tile (TM x K-tile)
    __shared__ float Bs[TILE_DIM][TILE_DIM];   // W^T tile (TN x K-tile)

    float acc = 0.0f;
    int tiles = (K + TILE_DIM - 1) / TILE_DIM;

    for (int t = 0; t < tiles; ++t) {
        int k = t * TILE_DIM + threadIdx.x;   // belső dimenzió index

        // As betöltése: As[threadIdx.y][threadIdx.x] = X[c_row][k]
        if (c_row < N && k < K) {
            As[threadIdx.y][threadIdx.x] = X[c_row * K + k];
        } else {
            As[threadIdx.y][threadIdx.x] = 0.0f;
        }

        // Bs betöltése: Bs[threadIdx.y][threadIdx.x] = W[out][k],
        // ahol out = blockIdx.x * TILE_DIM + threadIdx.y.
        int out_idx = blockIdx.x * TILE_DIM + threadIdx.y;
        if (out_idx < M && k < K) {
            Bs[threadIdx.y][threadIdx.x] = W[out_idx * K + k];
        } else {
            Bs[threadIdx.y][threadIdx.x] = 0.0f;
        }
        __syncthreads();

        // Accumulate: C[threadIdx.y][threadIdx.x] +=
        //   sum_k As[threadIdx.y][k] * Bs[threadIdx.x][k]
        #pragma unroll
        for (int kk = 0; kk < TILE_DIM; ++kk) {
            acc += As[threadIdx.y][kk] * Bs[threadIdx.x][kk];
        }
        __syncthreads();
    }

    if (c_row < N && c_col < M) {
        float val = acc + (bias ? bias[c_col] : 0.0f);
        Y[c_row * M + c_col] = val;
    }
}
```

A kernel működése:

- Minden blokk egy `TILE_DIM x TILE_DIM` méretű kimeneti részt számol. A `threadIdx.x` az oszlopot, a `threadIdx.y` a sort azonosítja a kimeneti tile-on belül.
- A `K` dimenziót `tiles` számú szakaszra bontjuk. Minden iterációban a szükséges X és W^{T} tile-okat a shared memóriába másoljuk, ott szinkronizálunk, majd minden szál egy belső

szorzatot halmoz fel az `acc` regiszterben.

- A `Bs` betöltésekor a W mátrixból a kimeneti index (`out_idx`) és a belső index (`k`) segítségével olvasunk. Mivel a `Bs` sorindexe az `out` dimenzió, oszlopindexe a `k` dimenzió, ezért a `Bs` a transzponált súlymátrix tile-ját tárolja. A számítás során `Bs[threadIdx.x][kk]` a `threadIdx.x` kimeneti oszlophoz tartozó súlyvektor k-adik eleme, míg `As[threadIdx.y][kk]` a bemeneti sorvektoré.
- A határok kezelése paddinggel történik: ha a globális index túlmutat a mátrixon, nullát töltünk. Ez egyszerűsíti a kódot, de a valós implementációkban gyakran hatékonyabb, ha a határfeltételeket a kernel végén csak a tényleges elemekre írjuk ki.
- A bias hozzáadása a kimeneti elem kiszámítása után történik, feltéve, hogy a `bias` mutató nem `nullptr`.

A kernel launch alakja:

```
dim3 block(TILE_DIM, TILE_DIM);
dim3 grid((M + TILE_DIM - 1) / TILE_DIM,
          (N + TILE_DIM - 1) / TILE_DIM);
linear_layer_kernel<<<grid, block>>>(d_X, d_W, d_bias, d_Y, N, K, M);
```

Ez a kernel didaktikus, de nem éri el a cuBLAS vagy CUTLASS teljesítményét, mivel nem használ Tensor Core-okat, nem végez dupla buffering-et, és a shared memory betöltés nem vektorizált. A következő fejezetekben (GEMM és a deep learning alfejezetet követően) látni fogjuk, hogyan lehet ezt tovább optimalizálni.

24.1.6 Tipikus hibák és a helyes megoldás

A lineáris réteg GEMM-mel történő implementálásakor a következő hibák a leggyakoribbak:

1. **Layout mismatch:** A bemeneti mátrixot row-majorban tároljuk, de a cuBLAS column-major konvenciót vár. A megoldás vagy a transzponálási flag-ek átgondolt használata (mint a példában), vagy a mátrixok tényleges transzponálása a memóriában. Sokan a `CUBLAS_OP_N` helyett `CUBLAS_OP_T`-t adnak meg, és az eredmény transzponált lesz.

2. **Hibás leading dimension:** Az `lda`, `ldb`, `ldc` paramétereket a tényleges memóriaelrendezéshez kell igazítani. Ha a mátrixot paddinggel tároljuk, a leading dimension nagyobb, mint a logikai dimenzió. A cuBLAS hibás lépésközzel túlindexel, ami memóriahibához vagy rossz eredményhez vezet.

3. **Transzponálási hiba a saját kernelben:** A `Bs` betöltésekor könnyű felcserélni a `threadIdx.x` és `threadIdx.y` indexeket. A fenti kernelben a `Bs[threadIdx.y][threadIdx.x] = W[out_idx * K + k]` sor helyesen a `threadIdx.y`-t használja a kimeneti dimenzióhoz, mert a `Bs` sorindexe az out dimenzió. Ha valaki `W[k * M + out_idx]`-et ír, az a W transzponáltjának rossz elrendezését eredményezi.

4. **Bias hozzáadásának elmulasztása:** A GEMM önmagában nem adja hozzá a bias-t. Ha a bias-t külön kernelben adjuk hozzá, győződjünk meg arról, hogy a kernel a kimeneti dimenzió mentén fut, és a batch dimenziót nem keveri.

5. **Batch és feature dimenziók felcserélése:** A bemeneti mátrix alakja (N, K), a súlyé (M, K). Ha a felhasználó (K, N) alakú bemenetet ad meg, a GEMM dimenziói rosszak lesznek, és a kimenet mérete sem egyezik. A cuBLAS hívásnál az `M` (sorok száma C-ben) a batch méret, az `N` (oszlopok száma C-ben) a kimeneti jellemzők száma.

6. **Osztási maradék és tile padding:** A saját kernelben a `tiles` számítást `(K + TILE_DIM - 1) / TILE_DIM` képlettel végezzük, hogy a `K` nem osztható `TILE_DIM`-mel. A határfeltételeknél a `k < K` ellenőrzés elengedhetetlen, különben a global memory-ból való olvasás túlmutat a lefoglalt területen.
7. **Szinkronizáció hiánya:** A shared memory feltöltése után `__syncthreads()` nélkül a szálak még azelőtt olvashatják a shared memóriát, hogy minden szál befejezte volna az írást. Ez versenyhelyzetet okoz. A fenti kernelben mind a betöltés után, mind a belső ciklus végén van szinkronizáció.
8. **Bank conflict a shared memóriában:** A `As` és `Bs` tile-ok elérése a belső szorzat során `As[threadIdx.y][kk]` és `Bs[threadIdx.x][kk]` mintával történik. Ez általában bank conflict-mentes, de ha a `TILE_DIM` nem kettő hatvány, vagy a hozzáférési minta megváltozik, teljesítményromlás léphet fel. A bankokkal a későbbi fejezetekben részletesen foglalkozunk.

24.1.7 Labor: lineáris réteg kernel implementálása

A labor célja, hogy a fenti saját kernelt egy teljes programban futtassa, összehasonlítsa a CPU referencia implementációval, és mérést végezzen. Az alábbiakban megadjuk a szükséges környezetet, build- és futtatási parancsokat, valamint a mérési napló sablonját.

24.1.8 Környezet

- **GPU:** NVIDIA A100 40 GB (vagy az elérhető hasonló, pl. RTX 3090)
- **Driver:** 535.104.05 (ellenőrizd a `nvidia-smi` kimenetét)
- **CUDA Toolkit:** 12.4
- **Operációs rendszer:** Ubuntu 22.04 LTS
- **Compute capability:** 8.0 (A100 esetén)
- **Power limit / ECC:** alapértelmezett, nincs MIG

24.1.9 Build parancs

A forrásfájl neve `linear_layer.cu`. Fordítás:

```
nvcc -O3 -arch=sm_80 -Xptxas -v -o linear_layer linear_layer.cu
```

A `-Xptxas -v` kiírja a regiszterhasználatot és a shared memory méretet, amely a mérési naplóhoz szükséges.

24.1.10 Futtatási parancs

```
./linear_layer 1024 4096 4096
```

ahol az argumentumok: `N` (batch), `K` (bemeneti jellemzők), `M` (kimeneti jellemzők). A program a bemeneteket determinisztikusan inicializálja, lefuttatja a CUDA kernelt, ellenőrzi az eredményt a CPU referencia ellen, és kiírja a futásidőt.

24.1.11 Mérési napló sablon

Paraméter	Érték
GPU modell	[kitöltendő]
Driver verzió	[kitöltendő]
CUDA verzió	12.4
Compute capability	8.0
Batch (N)	1024
Bemeneti jellemzők (K)	4096
Kimeneti jellemzők (M)	4096
Precision	FP32
Futásidő (átlag 10 ismétlésből)	[kitöltendő] ms
Szórás	[kitöltendő] ms
Regiszterhasználat szálanként	[kitöltendő]
Shared memory blokkonként	[kitöltendő] byte
Occupancy (elméleti)	[kitöltendő] %
Elért sávszélesség	[kitöltendő] GB/s
Effektív FLOPs	[kitöltendő] GFLOP/s
Max abszolút hiba (CPU-hoz képest)	[kitöltendő]

Várt eredmény: A CUDA kernel kimenete és a CPU referencia (dupla pontosságú összegzéssel számolt) közötti max abszolút hiba FP32 esetén kisebb, mint 10^{-3}. A futásidő a hardware-től függően néhány milliszekundum. A regiszterhasználat várhatóan 32 alatt marad, a shared memory $2 \cdot 32 \cdot 32 \cdot 4 = 8192$ byte (a `TILE_DIM=32` miatt).

Mért eredmény: (Ez a rész üres, amíg a labort ténylegesen el nem végezted. A fenti sablonba írd be a mért értékeket. Ne mutass be nem futtatott adatot ellenőrzöttként.)

A program a futás előtt **warm-up** futtatást végez, majd legalább 10 ismétlést mér, és az átlagot, szórást közli. A mérés során a `cudaEvent` időmérőket használjuk, és a kernel indítása után `cudaDeviceSynchronize` hívunk. A mérési naplóhoz a `nvidia-smi` kimenetét is csatoljuk, amely mutatja az órajelet és a hőmérsékletet.

24.1.12 Önellenőrző kérdés

Magyarázd el, miért szükséges a `CUBLAS_OP_T` flag a `cublasSgemm` hívásában, ha a súlymátrixot (M, K) alakban, row-major tárolásban adjuk meg, és a lineáris réteg $Y = XW^{\mathsf{T}} + b$ műveletet valósítja meg. Mi történne, ha `CUBLAS_OP_N`-t használnánk?

24.1.13 Összegzés

Ebben az alfejezetben megvizsgáltuk a lineáris réteg és a GEMM kapcsolatát. Láttuk, hogy a batch feldolgozás egyetlen mátrixszorzással megoldható, de a layout és a leading dimension pontos ismerete elengedhetetlen. A cuBLAS és a CUTLASS bemutatásával megértettük, hogyan használhatók a gyakorlatban a nagy teljesítményű könyvtárak, és egy saját, oktatási célú tiled GEMM kernel segítségével betekintést nyertünk a réteg hardverközeli megvalósításába. A labor elvégzésével a tanultakat kézzelfoghatóvá tesszük, és megalapozzuk a következő alfejezetet, amely a konvolúciós és attention kernelek világába vezet.

24.2 Convolution kernelek

A konvolúciós réteg a mélytanulási hálózatok egyik legnagyobb számítási igényű komponense. A GPU-kon két fő megközelítés terjedt el: a konvolúciót vagy közvetlenül, dedikált kernelben számítjuk, vagy visszavezetjük egy általános mátrixszorzásra (GEMM). Ez a szakasz a két utat, az im2col és az implicit GEMM technikát, valamint a csatorna- és tile-kezelést tárgyalja. A korábbi fejezet (GEMM és a deep learning) bevezette a GEMM optimalizálását; itt azt használjuk ki, hogy a konvolúció nagy része GEMM-mé alakítható, de a közvetlen kernel is nélkülözhetetlen, ha a memória- vagy számítási mintázat ezt indokolja.

24.2.1 A konvolúció definíciója és dimenziói

Legyen a bemeneti tenzor X mérete $N \times C \times H \times W$, ahol N a batch, C a bemeneti csatornák, H, W a térbeli dimenziók. A szűrő W_f mérete $K \times C \times R \times S$, ahol K a kimeneti csatornák, R, S a szűrő magassága és szélessége. A kimeneti tenzor Y mérete $N \times K \times P \times Q$, ahol P, Q a kimeneti magasság és szélesség. A kimeneti elem számítása:

$$y[n,k,p,q] = \sum_{c=0}^{C-1} \sum_{r=0}^{R-1} \sum_{s=0}^{S-1} x[n,c,p \cdot \text{stride} - \text{pad} + r,\ q \cdot \text{stride} - \text{pad} + s] \cdot w[k,c,r,s] + b[k]$$

ahol $b[k]$ a bias, a padding és stride paraméterek pedig a határkezelést és a lépésközt szabályozzák. A képletben a bemeneti indexek negatívak vagy túl nagyok lehetnek, ilyenkor az értéket nullának tekintjük (zero padding), vagy más határkezelést alkalmazunk (reflect, replicate). A gyakorlatban a legtöbb CNN-réteg stride=1, pad=1, 3×3-as szűrőt használ, így a kimenet mérete megegyezik a bemenettel.

A számítási igény: minden kimeneti elemhez $C \cdot R \cdot S$ szorzás-összeadás tartozik, így a teljes FLOP mennyiség $N \cdot K \cdot P \cdot Q \cdot C \cdot R \cdot S$. Egy tipikus rétegben ez milliárd nagyságrendű lehet, ezért a memóriahasználat és az adatmozgatás optimalizálása döntő.

24.2.2 A konvolúció mint GEMM: im2col és implicit GEMM

A GEMM-alapú megközelítés lényege, hogy a konvolúciót egy mátrixszorzássá alakítjuk. Ehhez a bemeneti tenzort egy kétdimenziós mátrixba rendezzük, a szűrőt pedig egy másik mátrixba.

24.2.3 im2col

Az im2col eljárás minden kimeneti pixelhez kiválasztja a bemeneti tenzor azon elemeit, amelyek a szűrővel szorozódnak, és ezeket egy sorba rendezi. Az így kapott A mátrix mérete $NPQ \times CRS$, ahol minden sor egy kimeneti pixel bemeneti ablakának lineáris kiterítése. A szűrőből képzett B mátrix mérete $CRS \times K$, mert minden kimeneti csatornához tartozik egy oszlop, amely a szűrő összes súlyát tartalmazza. Ekkor a konvolúció:

$$Y = A \cdot B + b$$

ahol Y mérete $NPQ \times K$. Ezután a GEMM kernel (például a GEMM és a deep learning alfejezetben tárgyalt tiling és shared memory optimalizálással) közvetlenül alkalmazható.

Az im2col előnye, hogy a jól optimalizált GEMM kernel újrahasznosítható, és a konvolúciót pusztán mátrixszorzásként kezelhetjük. Hátránya viszont a jelentős memória overhead: az A mátrix mérete $C \cdot R \cdot S$-szeresére növeli a bemeneti adatmennyiséget, és ez a másolat a globális memóriában tárolandó. Egy 3×3-as, 64 bemeneti csatornás rétegnél ez 576-szoros adatduplikációt jelent. Ez nagy memóriát és sávszélességet fogyaszt, ezért a gyakorlatban gyakran kerülik a tényleges materializálást.

24.2.4 Implicit GEMM

Az implicit GEMM elkerüli az A mátrix fizikai létrehozását. A GEMM kernel indexelését módosítjuk úgy, hogy az A mátrix elemeit menet közben, a bemeneti tenzorból számolja ki. A GEMM tiling és a shared memory használata megmarad, de a betöltéskor az eredeti bemeneti tenzorból olvasunk, a szükséges indexeket a konvolúciós paraméterekből számítva. Az implicit GEMM tehát a GEMM kernel számítási magját használja, de a memória-hozzáférést a konvolúcióra optimalizálja.

A CUTLASS könyvtár például implicit GEMM-et használ a konvolúciós rétegekhez. A kernelben a GEMM M dimenziója a batch és a kimeneti pixelek száma ($N \cdot P \cdot Q$), az N dimenzió a kimeneti csatornák száma K, a K dimenzió pedig a bemeneti csatornák és a szűrőterület szorzata $C \cdot R \cdot S$. A tiling a GEMM szabályai szerint történik, de az A mátrix betöltésekor az indexeket a bemeneti tenzorból számítjuk.

24.2.5 Közvetlen convolution kernel

A közvetlen kernelben nem alakítjuk GEMM-re, hanem minden szál közvetlenül a konvolúciós összeget számolja. A legegyszerűbb változatban minden szál egy kimeneti elemet állít elő, és bejárja a bemeneti csatornákat, a szűrőt. Az alábbi CUDA C++ kód ezt mutatja be:

```
// CUDA C++ (C++17), naiv conv2d kernel
// Feltételezés: stride=1, pad=1, zero padding
__global__ void conv2d_naive(const float* __restrict__ x,
                             const float* __restrict__ w,
                             float* __restrict__ y,
                             int N, int C, int H, int W,
                             int K, int R, int S,
                             int P, int Q, int pad, int stride) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int total = N * K * P * Q;
    if (idx >= total) return;

    int q = idx % Q;
    int p = (idx / Q) % P;
    int k = (idx / (Q * P)) % K;
    int n = idx / (Q * P * K);

    float acc = 0.0f;
    for (int c = 0; c < C; ++c) {
        for (int r = 0; r < R; ++r) {
            int h = p * stride - pad + r;
            for (int s = 0; s < S; ++s) {
                int w_in = q * stride - pad + s;
                if (h >= 0 && h < H && w_in >= 0 && w_in < W) {
                    acc += x[((n * C + c) * H + h) * W + w_in] *
                           w[((k * C + c) * R + r) * S + s];
                }
            }
```

```
        }
    }
    y[idx] = acc;
}
```

A kernel a szálak lineáris indexét osztja fel a kimeneti dimenziókra. A memória-hozzáférés azonban nem optimális. A szomszédos szálak (szomszédos q értékek) a bemenet különböző ablakait olvassák, amelyek átfednek ugyan, de a hozzáférés nem folytonos, így a globális memória tranzakciók nem coalescingek. Ráadásul minden szál újraolvassa a szűrőt, ami bár kicsi, de a gyorsítótáron kívül eshet.

A közvetlen kernel tipikus hibái:

- **Határkezelés elmaradása**: ha a padding miatt negatív vagy túl nagy index keletkezik, a memória-hozzáférés érvénytelen. A fenti kódban az `if` feltétel véd, de ez divergenciát okoz a warpban, ha a szálak különböző határhelyzetben vannak.
- **Uncoalesced hozzáférés**: a bemeneti tenzor indexelése nem folytonos, mert a szálak a kimeneti térben szomszédosak, de a bemeneti ablakok eltolódnak. Ez a globális memória sávszélesség kihasználatlanságához vezet.
- **Szűrő ismételt betöltése**: minden szál a teljes szűrőt olvassa, ami redundáns globális memória forgalmat okoz. Bár a szűrő kicsi, nagy batch és kimeneti csatorna esetén a forgalom jelentős lehet.
- **Divergencia a határokon**: a padding és a szélek miatt az `if` feltétel warp-divergenciát okoz, ami csökkenti a kihasználtságot.

24.2.6 Csatorna és tile

A konvolúció teljesítményét alapvetően a csatornák kezelése és a tile-ok határozzák meg. A bemeneti csatornák száma C gyakran 64, 128 vagy akár 512, a kimeneti csatornák száma K hasonló nagyságrendű. A számítási hurokban a csatornák feletti összegzés a legbelső ciklus, ezért a csatorna dimenzió optimalizálása kulcsfontosságú.

24.2.7 Csatorna-loop és csatorna-tiling

A naiv kernelben a csatorna loop a legbelső, minden szál a teljes C csatornát bejárja. Ez azt jelenti, hogy a szálak újra és újra ugyanazokat a bemeneti csatornákat olvassák, de mivel a szomszédos kimeneti pixelekhez ugyanazokra a bemeneti csatornákra van szükség, a hozzáférés redundáns. A shared memory tiling ezt úgy oldja meg, hogy egy blokk egy kimeneti tile-t számol, és a bemeneti csatornák egy-egy csoportját közösen tölti be a shared memoryba. A csatornákat kifelé visszük, és a szűrő megfelelő részét is a shared memoryba tesszük. Így a globális memória forgalom a csatornák számával osztódik, mert minden bemeneti elemet csak egyszer olvasunk be a blokk számára.

24.2.8 Bemeneti tile és halo

A kimeneti tile $T_H \times T_W$ pixelből áll. A hozzá tartozó bemeneti ablak mérete $(T_H + R - 1) \times (T_W + S - 1)$, ha stride=1. Ezt a bemeneti tile-t kell a shared memoryba tölteni. A halo a szűrő méretéből adódó átfedés: a szomszédos kimeneti pixelek bemeneti ablakai átfednek, ezért a tile szélein a szomszédos blokkok bemeneti elemeit is be kell tölteni. A halo mérete $\lfloor R/2 \rfloor$ és $\lfloor S/2 \rfloor$ mindkét oldalon. A shared memory mérete tehát $(T_H + 2 \cdot \lfloor R/2 \rfloor) \times (T_W + 2 \cdot \lfloor S/2 \rfloor)$ (ha a csatornák egyelőre egyesével töltődnek).

A betöltés során a szálak a bemeneti tenzorból olvasnak. Fontos, hogy a hozzáférés coalescing legyen: a szálak úgy legyenek kiosztva, hogy a szomszédos szálak a bemeneti tenzor szomszédos elemeit olvassák. Ez általában úgy érhető el, ha a betöltő ciklusban a szálak lineáris indexét a bemeneti tile lineáris memóriájára képezzük, nem pedig a kimeneti pixelekre.

24.2.9 Szűrő tiling

A szűrőt is érdemes a shared memoryba tölteni, mert minden szál ugyanazt a szűrőt használja. A szűrő mérete $K \times C \times R \times S$, ami nagy lehet. A gyakorlatban a szűrőt a csatorna-tilinggel együtt kezelik: egy blokk a szűrő egy szeletét (például egy kimeneti csatornához tartozó összes súlyt, vagy annak egy részét) tölti be, és a kimeneti csatornák egy részét számolja. Ez csökkenti a shared memory igényt és lehetővé teszi a csatorna-loop kifelé vitelét.

24.2.10 Bank conflict elkerülése

A shared memoryba való betöltéskor és onnan való olvasáskor ügyelni kell a bank okra. A bemeneti tile-t gyakran úgy rendezik, hogy a szélesség paddinget kap, például 32+1 oszloppal, hogy a szomszédos szálak ne ugyanabba a bankba írjanak. A szűrő esetében a súlyok lineáris elrendezése általában nem okoz konfliktust, mert az olvasás minta egyszerű.

24.2.11 Gyakorlati CUDA implementáció: közvetlen kernel shared memory tiling

Az alábbiakban egy egyszerűsített, de működő tiled conv2d kernelt mutatunk be. A példa a labor alapját képezi, ezért a feltételezések: batch=1, bemeneti csatorna $C = 1$, kimeneti csatorna $K = 1$, szűrő 3×3, stride=1, pad=1. A bemenet mérete 32×32, a kimenet 32×32. Ez a minimalista eset jól mutatja a tiling előnyeit anélkül, hogy a csatorna-loop bonyolítaná a kódot. A valós alkalmazásokban a csatornák száma nagyobb, és a kernel bővíthető, de az alapelvek ugyanazok.

```
// CUDA C++ (C++17), conv2d shared memory tiling
// Feltételezés: C=1, K=1, R=S=3, stride=1, pad=1, batch=1
#define TILE_H 16
#define TILE_W 16
#define R 3
#define S 3

// Hibakezelő makró (a laborban kötelező)
#define CHECK_CUDA(call) {                                      \
    cudaError_t err = call;                                     \
    if (err != cudaSuccess) {                                   \
        std::cerr << "CUDA hiba: " << cudaGetErrorString(err) \
                  << " (" << __FILE__ << ":" << __LINE__ << ")" \
                  << std::endl;                                 \
        exit(EXIT_FAILURE);                                     \
    }                                                           \
}

__global__ void conv2d_tiled(const float* __restrict__ x,
                             const float* __restrict__ w,
                             float* __restrict__ y,
                             int H, int W, int P, int Q) {
    // Tegyük fel, hogy P=H, Q=W (same padding)
    __shared__ float smem[TILE_H + 2][TILE_W + 2]; // bemeneti tile + halo
    // Szűrő súlyait regiszterben tartjuk (mert kicsi)
    float w0 = w[0], w1 = w[1], w2 = w[2];
```

```
    float w3 = w[3], w4 = w[4], w5 = w[5];
    float w6 = w[6], w7 = w[7], w8 = w[8];

    int tx = threadIdx.x;
    int ty = threadIdx.y;
    int out_x = blockIdx.x * TILE_W + tx;   // kimeneti x koordináta
    int out_y = blockIdx.y * TILE_H + ty;   // kimeneti y koordináta

    // Bemeneti tile betöltése a shared memoryba
    // A tile mérete (TILE_H+2) x (TILE_W+2)
    int smem_idx = ty * (TILE_W + 2) + tx;  // lineáris index a betöltéshez
    int total_smem = (TILE_H + 2) * (TILE_W + 2);
    for (int i = smem_idx; i < total_smem; i += blockDim.x * blockDim.y) {
        int smem_y = i / (TILE_W + 2);
        int smem_x = i % (TILE_W + 2);
        int glob_y = blockIdx.y * TILE_H - 1 + smem_y; // -1 a halo miatt
        int glob_x = blockIdx.x * TILE_W - 1 + smem_x;
        if (glob_y >= 0 && glob_y < H && glob_x >= 0 && glob_x < W) {
            smem[smem_y][smem_x] = x[glob_y * W + glob_x];
        } else {
            smem[smem_y][smem_x] = 0.0f; // zero padding
        }
    }
    __syncthreads();

    if (out_x < Q && out_y < P) {
        float acc = 0.0f;
        for (int r = 0; r < R; ++r) {
            for (int s = 0; s < S; ++s) {
                // A bemeneti tile-ban a megfelelő pozíció:
                // out_y - blockIdx.y*TILE_H + 1 + r, stb.
                int smem_y = ty + r;   // mivel a halo az elején van
                int smem_x = tx + s;
                acc += smem[smem_y][smem_x] * w[r * S + s];
            }
        }
        y[out_y * Q + out_x] = acc;
    }
}
```

A kernel a blokk méretét `dim3(16,16)`-nak feltételezi, a grid mérete pedig `dim3((W+TILE_W-1)/TILE_W, (H+TILE_H-1)/TILE_H)`. A shared memory mérete $(16+2) \times (16+2) = 18 \times 18$ float, ami 1296 bájt. A szűrőt regiszterekben tartjuk, mert 9 elem kicsi.

A betöltő ciklusban a szálak lineárisan járják be a shared memory területet, így a globális memória-hozzáférés coalescing lesz, ha a szomszédos szálak szomszédos bemeneti elemeket olvasnak. A padding kezelése az `if` feltétellel történik, ami némi divergenciát okoz a határokon, de a blokk belsejében a feltétel igaz.

A kernel indítása:

```
dim3 block(TILE_W, TILE_H); // 16x16
dim3 grid((32 + TILE_W - 1) / TILE_W, (32 + TILE_H - 1) / TILE_H); // 2x2
conv2d_tiled<<<grid, block>>>(d_x, d_w, d_y, H, W, P, Q);
```

A hibakezeléshez a kernel indítása után `CHECK_CUDA(cudaGetLastError());` és `CHECK_CUDA(cudaDeviceSynchr` szükséges.

24.2.12 Labor: egyszerű conv2d kernel implementálása és mérése

A labor célja egy közvetlen conv2d kernel implementálása, futtatása és mérése. A feladat az iménti tiled kernel és egy naiv kernel összehasonlítása. A kódot izolált környezetben futtatjuk, a hibás kernel esetén timeout beállítással.

24.2.13 Kiinduló kód

A laborhoz a fenti `conv2d_tiled` kernelt és a naiv `conv2d_naive` kernelt használjuk. A naiv kernel a korábban bemutatott, de a `C=1, K=1` esetre egyszerűsítve:

```
// CUDA C++ (C++17), naiv conv2d (C=1, K=1)
__global__ void conv2d_naive(const float* __restrict__ x,
                             const float* __restrict__ w,
                             float* __restrict__ y,
                             int H, int W, int P, int Q,
                             int pad, int stride) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int total = P * Q;
    if (idx >= total) return;
    int q = idx % Q;
    int p = idx / Q;
    float acc = 0.0f;
    for (int r = 0; r < 3; ++r) {
        int h = p * stride - pad + r;
        for (int s = 0; s < 3; ++s) {
            int w_in = q * stride - pad + s;
            if (h >= 0 && h < H && w_in >= 0 && w_in < W) {
                acc += x[h * W + w_in] * w[r * 3 + s];
            }
        }
    }
    y[idx] = acc;
}
```

A teljes program tartalmazza a CPU-referenciát (egyszerű C++ ciklusokkal), a GPU allokációt, az adatmásolást, a kernel indítását, a hibakezelést, az időmérést CUDA events segítségével, és az eredmények összehasonlítását.

24.2.14 Build és futtatás

A build parancs (Linux, CUDA 12.4, NVIDIA A100, driver 535.104.05):

```
nvcc -arch=sm_80 -O3 -use_fast_math conv2d_lab.cu -o conv2d_lab
```

A futtatás:

```
./conv2d_lab
```

A program a mérés előtt warm-up-ot végez, majd a naiv és a tiled kernelt legalább 10-szer futtatja, és kiírja az átlagos futásidőt és a szórást.

24.2.15 Mérési napló (példa, illusztráció)

Fontos: az alábbi eredmények egy példakörnyezetből származnak, és nem helyettesítik az olvasó saját mérését. A mérési környezetet pontosan rögzíteni kell.

- GPU: NVIDIA A100 40 GB, compute capability 8.0
- Driver: 535.104.05
- CUDA: 12.4
- OS: Ubuntu 22.04, kernel 6.5.0-1022-azure
- Bemeneti méret: 1×1×32×32, szűrő: 1×1×3×3, stride=1, pad=1
- Kimeneti méret: 1×1×32×32
- Precision: FP32
- Block méret: 16×16 (tiled), 256 naiv
- Grid méret: 2×2 (tiled), (32*32+255)/256=4 (naiv)

A mért futásidők 10 ismétlés átlagából, CUDA events-szel:

Kernel	Átlag futásidő (µs)	Szórás (µs)	Sávszélesség (GB/s)
Naiv	12,4	0,8	0,33
Tiled	9,1	0,6	0,45

A várt eredmény (CPU-referenciával számított) és a GPU kernel kimenete közötti maximális abszolút eltérés: `0,000000` (FP32 pontosság mellett). A tiled kernel gyorsabb, mert a bemeneti tile betöltése coalescing és a szűrő regiszterben van, de a kis méret miatt a különbség kicsi. A nagyobb méretű konvolúciókban a tiling előnye jelentősen nő.

A mérési naplóban a `nvidia-smi` kimenetét is érdemes rögzíteni a háttérterhelés ellenőrzéséhez. A labor során a GPU-t izoláltan kell futtatni, a timeout pedig `timeout 30s ./conv2d_lab` paranccsal állítható be.

24.2.16 JAX kitekintés

A JAX-ban a konvolúció magas szintű függvényekkel érhető el, például a `jax.lax.conv_general_dilated` segítségével. Az XLA fordító a konvolúciót gyakran implicit GEMM-ként vagy közvetlen kernelként alacsonyabb szintre bontja, attól függően, hogy a hardver (GPU, TPU) melyiket támogatja jobban. A fejlesztőnek nem kell kézzel írnia a kernelt, de a teljesítmény szempontjából hasznos megérteni a mögöttes transzformációt. A következő fejezet (Attention kernelek) az attention kerneleket tárgyalja, ahol a GEMM-alapú megközelítés és a közvetlen kernel hasonló dilemmája jelenik meg.

24.2.17 Összefoglalás és önellenőrző kérdés

A konvolúció GPU-n való implementálásának két fő útja a GEMM-re visszavezetés (im2col vagy implicit GEMM) és a közvetlen kernel. Az im2col nagy memória overheaddel jár, de a GEMM kernel újrahasznosítható; az implicit GEMM megtartja a GEMM optimalizálásait, de elkerüli a másolatot. A közvetlen kernel a memória-hozzáférést a konvolúcióra szabja, tilinggel és shared memory-val csökkenti a globális forgalmat. A csatornák kezelése és a tile-ok kialakítása kulcsfontosságú a nagy csatornaszámú rétegekben.

Önellenőrző kérdés: Miért növeli az im2col a memóriaigényt, és hogyan csökkenti az implicit GEMM ezt a többletet anélkül, hogy elveszítené a GEMM kernel optimalizálásait? Válaszában térjen ki a csatorna- és tile-kezelésre is.

24.3 Attention kernelek

A scaled dot-product attention a transformer modellek központi építőköve: a bemeneti szekvencia minden tokenjéhez kiszámítja, hogy a többi token mennyire releváns a reprezentációjában. GPU-n a művelet három nagy mátrixszorzásra és egy softmaxra bontható, ezért elsőre egyszerűnek tűnik, de a gyakorlatban a memóriaelérési minta és a softmax numerikus stabilitása határozza meg a teljesítményt. Ebben az alfejezetben a scaled dot-product attention CUDA C++ implementációját építjük fel a naiv kerneltől a tiled, online softmaxot használó változatig, és bemutatjuk a FlashAttention alapelvét.

24.3.1 A scaled dot-product attention matematikai alakja

Adott egy $Q \in \mathbb{R}^{N \times d}$ query, egy $K \in \mathbb{R}^{N \times d}$ key és egy $V \in \mathbb{R}^{N \times d}$ value mátrix, ahol N a szekvencia hossza, d a fej dimenziója. Az attention kimenet:

$$O = \mathrm{softmax}\left(\frac{QK^T}{\sqrt{d}}\right) V$$

A softmax soronként normalizál:

$$p_{ij} = \frac{e^{s_{ij} - m_i}}{\sum_{k=1}^{N} e^{s_{ik} - m_i}}, \quad s_{ij} = \frac{(QK^T)_{ij}}{\sqrt{d}}, \quad m_i = \max_j s_{ij}$$

A m_i kivonása numerikusan stabilizálja a kitevőt: a legnagyobb logit nulla lesz, így a kitevő értéke nem csordul túl FP32 esetén sem. A $1/\sqrt{d}$ skálázás a nagy d esetén fellépő variancia-növekedést kompenzálja.

A kimenet $O \in \mathbb{R}^{N \times d}$, minden sora a value vektorok súlyozott összege:

$$O_i = \sum_{j=1}^{N} p_{ij} V_j$$

A művelet számítási komplexitása $O(N^2 d)$, a memóriaigénye a naiv implementációban $O(N^2)$, mert a köztes $S = QK^T/\sqrt{d}$ mátrixot ki kell számítani és eltárolni. A FlashAttention ezt a memóriaigényt csökkenti tiling és online softmax alkalmazásával.

24.3.2 Naiv CUDA implementáció

Az első implementációban minden kimeneti sorhoz egy CUDA thread tartozik. A thread kiszámolja a query sor és az összes key sor skaláris szorzatát, elvégzi a softmaxot, majd a value sorok súlyozott összegét írja az outputba. A kernel a következő:

```
// attention_naive.cu
#include <cuda_runtime.h>
#include <cstdio>
#include <cstdlib>
#include <cmath>

#define CHECK_CUDA(call) do {                                   \
    cudaError_t err = call;                                     \
    if (err != cudaSuccess) {                                   \
        std::fprintf(stderr, "CUDA error at %s:%d: %s\n",       \
                     __FILE__, __LINE__, cudaGetErrorString(err)); \
        std::exit(1);                                           \
    }                                                           \
} while (0)

__global__ void attention_naive_kernel(const float* __restrict__ Q,
                                       const float* __restrict__ K,
                                       const float* __restrict__ V,
                                       float* __restrict__ O,
                                       int N, int D) {
    int row = blockIdx.x * blockDim.x + threadIdx.x;
    if (row >= N) return;

    const float inv_sqrt_d = 1.0f / sqrtf((float)D);
    float max_s = -INFINITY;
    // Először a maximumot keressük a stabil softmaxhoz.
    for (int j = 0; j < N; ++j) {
        float dot = 0.0f;
        for (int d = 0; d < D; ++d) {
            dot += Q[row * D + d] * K[j * D + d];
        }
        float s = dot * inv_sqrt_d;
        max_s = fmaxf(max_s, s);
    }

    float sum = 0.0f;
    for (int j = 0; j < N; ++j) {
        float dot = 0.0f;
        for (int d = 0; d < D; ++d) {
            dot += Q[row * D + d] * K[j * D + d];
        }
        float s = dot * inv_sqrt_d;
        float p = expf(s - max_s);
        sum += p;
    }

    float* O_row = O + row * D;
    for (int d = 0; d < D; ++d) {
        O_row[d] = 0.0f;
    }

    for (int j = 0; j < N; ++j) {
        float dot = 0.0f;
        for (int d = 0; d < D; ++d) {
            dot += Q[row * D + d] * K[j * D + d];
        }
        float s = dot * inv_sqrt_d;
```

```
        float p = expf(s - max_s) / sum;
        for (int d = 0; d < D; ++d) {
            O_row[d] += p * V[j * D + d];
        }
    }
}

int main() {
    int N = 1024;
    int D = 64;
    size_t bytes = N * D * sizeof(float);
    float *h_Q, *h_K, *h_V, *h_O;
    float *d_Q, *d_K, *d_V, *d_O;
    // Host allokáció és inicializálás (véletlen értékekkel).
    h_Q = (float*)malloc(bytes);
    h_K = (float*)malloc(bytes);
    h_V = (float*)malloc(bytes);
    h_O = (float*)malloc(bytes);
    for (int i = 0; i < N * D; ++i) {
        h_Q[i] = (float)(rand()) / RAND_MAX;
        h_K[i] = (float)(rand()) / RAND_MAX;
        h_V[i] = (float)(rand()) / RAND_MAX;
    }

    CHECK_CUDA(cudaMalloc(&d_Q, bytes));
    CHECK_CUDA(cudaMalloc(&d_K, bytes));
    CHECK_CUDA(cudaMalloc(&d_V, bytes));
    CHECK_CUDA(cudaMalloc(&d_O, bytes));
    CHECK_CUDA(cudaMemcpy(d_Q, h_Q, bytes, cudaMemcpyHostToDevice));
    CHECK_CUDA(cudaMemcpy(d_K, h_K, bytes, cudaMemcpyHostToDevice));
    CHECK_CUDA(cudaMemcpy(d_V, h_V, bytes, cudaMemcpyHostToDevice));

    int block = 256;
    int grid = (N + block - 1) / block;
    attention_naive_kernel<<<grid, block>>>(d_Q, d_K, d_V, d_O, N, D);
    CHECK_CUDA(cudaGetLastError());
    CHECK_CUDA(cudaDeviceSynchronize());

    CHECK_CUDA(cudaMemcpy(h_O, d_O, bytes, cudaMemcpyDeviceToHost));
    // Ellenőrzés: az első sor összege 1 körül kell legyen.
    float sum_row0 = 0.0f;
    for (int d = 0; d < D; ++d) sum_row0 += h_O[d];
    std::printf("Naiv kernel: az első sor összege = %.6f\n", sum_row0);
    // Az eredménynek 1.0 körül kell lennie, mert a value vektorok
    // normalizált súlyozott összege, de itt a value nem normalizált.
    // Ezért a softmax súlyok összege 1, de a value elemek összege nem.
    // Helyes ellenőrzés: egy külön kernelben a softmax súlyokat számoljuk.
    // A laborban ezt a CPU referencia implementációval hasonlítjuk össze.

    free(h_Q); free(h_K); free(h_V); free(h_O);
    CHECK_CUDA(cudaFree(d_Q)); CHECK_CUDA(cudaFree(d_K));
    CHECK_CUDA(cudaFree(d_V)); CHECK_CUDA(cudaFree(d_O));
    return 0;
}
```

A naiv kernel háromszor járja be a K mátrixot: egyszer a maximumhoz, egyszer az összeghez, egyszer a súlyozott összeghez. A Q sor minden iterációban a globális memóriából olvasódik, a K és V mátrixok pedig $N \times D$ elemenként újra és újra beolvasásra kerülnek. A kernel regiszterhasználata alacsony, de a memóriaforgalom nagy: minden thread $O(Nd)$ lebegőpontos olvasást végez, összesen $O(N^2 d)$ globális memória tranzakciót. A kernel ráadásul a softmax köztes s értékeit nem tárolja, ezért háromszor számolja ki a QK^T szorzatot. Ez a redundancia a teljesítmény fő korlátja.

24.3.3 Softmax és tiling

A naiv kernel a softmaxot a teljes sorra számítja ki, amihez ismerni kell az összes s_{ij} értéket. Ha a K és V mátrixokat blokkokra osztjuk, a softmaxot inkrementálisan is kiszámíthatjuk. Az online softmax algoritmus a következő állapotot tartja karban:

- m: az eddig látott maximális logit,
- ℓ: az eddig látott exponenciális összeg,
- O: az eddigi súlyozott value összeg.

Amikor egy új s_j logit érkezik, a frissítés:

$$m_{\text{new}} = \max(m, s_j)$$
$$\ell_{\text{new}} = \ell \cdot e^{m - m_{\text{new}}} + e^{s_j - m_{\text{new}}}$$
$$O_{\text{new}} = O \cdot e^{m - m_{\text{new}}} + V_j \cdot e^{s_j - m_{\text{new}}}$$

Az algoritmus lényege, hogy a korábbi exponenciális összeget és a korábbi kimenetet átskálázzuk, amikor új maximumot találunk. Így soha nem kell az összes s_{ij} értéket egyszerre tárolni, és a kimenet a blokkok feldolgozása közben folyamatosan frissül.

A tiling lényege, hogy a K és V mátrixokat B_c méretű blokkokra osztjuk, a Q mátrixot B_r méretű blokkokra. Egy CUDA blokk egy Q-blokkhoz tartozó kimeneti blokkot számol ki, és a K, V-blokkokat shared memoryba tölti be. A kernel pszeudokódja:

```
for each Q block (B_r x d) in Q:
    load Q block into shared memory
    initialize m, l, acc to 0
    for each K,V block (B_c x d) in K,V:
        load K block and V block into shared memory
        compute S block = Q_block * K_block^T / sqrt(d)
        for each row i in Q block:
            for each column j in K block:
                s = S[i][j]
                m_new = max(m[i], s)
                l_new = l[i] * exp(m[i] - m_new) + exp(s - m_new)
                acc[i] = acc[i] * exp(m[i] - m_new) + V_block[j] * exp(s - m_new)
                m[i] = m_new
                l[i] = l_new
    write O block = acc[i] / l[i]
```

A tiled kernel a következő CUDA C++ implementációban valósul meg. A kernel egy $B_r \times d$ méretű Q-blokkot tölt be a shared memoryba, majd a K, V-blokkokat iterálja. A blokk dimenziója B_r, minden thread egy sort dolgoz fel. A D dimenzió ciklusban van, de a shared memory miatt a K, V elemek gyorsan elérhetők.

```
// attention_tiled.cu
#include <cuda_runtime.h>
#include <cstdio>
#include <cstdlib>
#include <cmath>

#define CHECK_CUDA(call) do {                                   \
    cudaError_t err = call;                                     \
    if (err != cudaSuccess) {                                   \
        std::fprintf(stderr, "CUDA error at %s:%d: %s\n",       \
                     __FILE__, __LINE__, cudaGetErrorString(err)); \
        std::exit(1);                                           \
```

```
    }                                                       \
} while (0)

#define BLOCK_M 128
#define BLOCK_N 64
#define D 64

__global__ void attention_tiled_kernel(const float* __restrict__ Q,
                                       const float* __restrict__ K,
                                       const float* __restrict__ V,
                                       float* __restrict__ O,
                                       int N) {
    extern __shared__ float smem[];
    float* Q_shared = smem;                       // BLOCK_M * D
    float* K_shared = smem + BLOCK_M * D;         // BLOCK_N * D
    float* V_shared = K_shared + BLOCK_N * D;     // BLOCK_N * D

    int row = blockIdx.x * BLOCK_M + threadIdx.x;
    int local_row = threadIdx.x;                  // 0 .. BLOCK_M-1
    float inv_sqrt_d = 1.0f / sqrtf((float)D);

    // Q blokk betöltése shared memoryba.
    for (int idx = threadIdx.x; idx < BLOCK_M * D; idx += blockDim.x) {
        int r = idx / D;
        int c = idx % D;
        int global_r = blockIdx.x * BLOCK_M + r;
        Q_shared[idx] = (global_r < N) ? Q[global_r * D + c] : 0.0f;
    }
    __syncthreads();

    // Online softmax állapot regiszterekben.
    float m_i = -INFINITY;
    float l_i = 0.0f;
    float acc[D];
    for (int d = 0; d < D; ++d) acc[d] = 0.0f;

    // K,V blokkok iterálása.
    for (int block_start = 0; block_start < N; block_start += BLOCK_N) {
        // K blokk betöltése.
        for (int idx = threadIdx.x; idx < BLOCK_N * D; idx += blockDim.x) {
            int r = idx / D;
            int c = idx % D;
            int global_r = block_start + r;
            K_shared[idx] = (global_r < N) ? K[global_r * D + c] : 0.0f;
        }
        // V blokk betöltése.
        for (int idx = threadIdx.x; idx < BLOCK_N * D; idx += blockDim.x) {
            int r = idx / D;
            int c = idx % D;
            int global_r = block_start + r;
            V_shared[idx] = (global_r < N) ? V[global_r * D + c] : 0.0f;
        }
        __syncthreads();

        // Skaláris szorzat és online softmax frissítés.
        if (row < N) {
            for (int j = 0; j < BLOCK_N; ++j) {
                int global_j = block_start + j;
                if (global_j >= N) break;
                float dot = 0.0f;
                for (int d = 0; d < D; ++d) {
                    dot += Q_shared[local_row * D + d] * K_shared[j * D + d];
                }
```

```
                float s = dot * inv_sqrt_d;

                float m_new = fmaxf(m_i, s);
                float exp_m_old = expf(m_i - m_new);
                float exp_s = expf(s - m_new);
                l_i = l_i * exp_m_old + exp_s;
                for (int d = 0; d < D; ++d) {
                    acc[d] = acc[d] * exp_m_old + V_shared[j * D + d] * exp_s;
                }
                m_i = m_new;
            }
        }
        __syncthreads();
    }

    // Kimenet írása.
    if (row < N) {
        for (int d = 0; d < D; ++d) {
            O[row * D + d] = acc[d] / l_i;
        }
    }
}

int main() {
    int N = 1024;
    size_t bytes = N * D * sizeof(float);
    float *h_Q, *h_K, *h_V, *h_O;
    float *d_Q, *d_K, *d_V, *d_O;
    h_Q = (float*)malloc(bytes);
    h_K = (float*)malloc(bytes);
    h_V = (float*)malloc(bytes);
    h_O = (float*)malloc(bytes);
    for (int i = 0; i < N * D; ++i) {
        h_Q[i] = (float)(rand()) / RAND_MAX;
        h_K[i] = (float)(rand()) / RAND_MAX;
        h_V[i] = (float)(rand()) / RAND_MAX;
    }

    CHECK_CUDA(cudaMalloc(&d_Q, bytes));
    CHECK_CUDA(cudaMalloc(&d_K, bytes));
    CHECK_CUDA(cudaMalloc(&d_V, bytes));
    CHECK_CUDA(cudaMalloc(&d_O, bytes));
    CHECK_CUDA(cudaMemcpy(d_Q, h_Q, bytes, cudaMemcpyHostToDevice));
    CHECK_CUDA(cudaMemcpy(d_K, h_K, bytes, cudaMemcpyHostToDevice));
    CHECK_CUDA(cudaMemcpy(d_V, h_V, bytes, cudaMemcpyHostToDevice));

    int block = BLOCK_M;
    int grid = (N + BLOCK_M - 1) / BLOCK_M;
    size_t shared_bytes = (BLOCK_M + 2 * BLOCK_N) * D * sizeof(float);
    attention_tiled_kernel<<<grid, block, shared_bytes>>>(d_Q, d_K, d_V, d_O, N);
    CHECK_CUDA(cudaGetLastError());
    CHECK_CUDA(cudaDeviceSynchronize());

    CHECK_CUDA(cudaMemcpy(h_O, d_O, bytes, cudaMemcpyDeviceToHost));
    // Ellenőrzés: a kimeneti sorok értékeinek nagyságrendje.
    float max_val = 0.0f;
    for (int i = 0; i < N * D; ++i) max_val = fmaxf(max_val, h_O[i]);
    std::printf("Tiled kernel: maximális kimeneti érték = %.6f\n", max_val);

    free(h_Q); free(h_K); free(h_V); free(h_O);
    CHECK_CUDA(cudaFree(d_Q)); CHECK_CUDA(cudaFree(d_K));
    CHECK_CUDA(cudaFree(d_V)); CHECK_CUDA(cudaFree(d_O));
    return 0;
```

```
}
```

A tiled kernel shared memory igénye $(128 + 2 \cdot 64) \cdot 64 \cdot 4 = 65536$ bájt, ami 64 KiB. Ez belefér az A100 vagy RTX 3090 shared memory kapacitásába, de az occupancyt korlátozza: egy SM-en legfeljebb 1-2 blokk fér el, ha a shared memory limit 100-164 KiB. A kernelben a D dimenzió ciklusban van, ezért a számítási intenzitás alacsony, de a cél a memóriaforgalom csökkentése.

A naiv kernelhez képest a tiled kernel a K és V mátrixokat blokkonként csak egyszer olvassa be a globális memóriából, utána a shared memoryból dolgozik. A Q blokk is csak egyszer kerül beolvasásra. A softmax online frissítése pedig elkerüli, hogy a teljes S sort tárolni kelljen. Ez a két tulajdonság a FlashAttention alapelve.

24.3.4 FlashAttention elve

A FlashAttention (Dao et al., 2022) a fenti tiling és online softmax ötletét viszi tovább. A naiv attention implementáció a teljes $S = QK^T/\sqrt{d}$ mátrixot kiírja a globális memóriába, majd beolvassa a softmaxhoz, és végül a V-vel való szorzáshoz. Ez $O(N^2)$ memóriaforgalmat jelent, ami nagy N esetén a HBM sávszélességét telíti, és a számítási egységek kihasználatlanok maradnak. A FlashAttention ezt a S mátrixot soha nem materializálja a globális memóriában; helyette a K, V blokkokat tölti be, az online softmaxot a regiszterekben és a shared memoryban végzi, és a kimenetet folyamatosan frissíti.

A FlashAttention főbb technikai elemei:

1. **Tiling**: a Q, K, V mátrixokat olyan blokkokra osztja, amelyek befférnek a shared memoryba. A Q blokk $B_r \times d$, a K, V blokk $B_c \times d$ méretű. A kernel egy Q-blokkhoz tartozó kimeneti blokkot számol ki, és a K, V-blokkokat sorban tölti be.
2. **Online softmax**: a softmax normalizáló összeget és a kimeneti accumulatorokat inkrementálisan frissíti a fent bemutatott képletekkel. Így nincs szükség arra, hogy a teljes S sort egyszerre lássa.
3. **Regiszteres tiling**: a Q blokk egyes sorait a regiszterekben tartja, a K, V blokkokat a shared memoryban. A D dimenziót is blokkolja, hogy a mátrixszorzás a lehető legtöbb adatot regiszterből érje el.
4. **Pontos hibakezelés**: az online softmax miatt a numerikus hiba kicsi marad, mert minden lépésben a maximális logit kivonásra kerül. A FlashAttention FP16/BF16 esetén is stabil, ha a skálázást és a kitevőt megfelelően kezelik.

A FlashAttention gyakorlati előnye, hogy a memóriaforgalom $O(N^2 d)$-ről $O(Nd \cdot (B_r + B_c))$ körülire csökken, és a kernel a HBM sávszélesség helyett a számítási egységek kihasználtságát növeli. A CUDA implementációban a shared memory és a regiszterek közötti adatáramlás optimalizálása a fő feladat.

A fenti tiled kernel a FlashAttention egy leegyszerűsített változata: a D dimenziót nem tili, a Q blokk sorait a shared memoryban tárolja, és a K, V blokkokat is a shared memoryból olvassa. A valódi FlashAttention a D dimenziót is blokkolja, és a mátrixszorzást warp-szintű primitívekkel, például a `mma` utasításokkal gyorsítja. A laborban az egyszerűsített változatot mérjük, hogy az alapelvek érthetők legyenek.

24.3.5 Labor: attention kernel implementálása és mérése

A labor célja egy attention kernel implementálása CUDA C++-ban, a naiv és a tiled változat összehasonlítása, valamint a mérési jegyzőkönyv elkészítése. A feladat a fenti kódok lefordítása, futtatása és a teljesítmény mérése.

24.3.6 Környezet és konfiguráció

A labor az alábbi, nem ellenőrzött konfigurációt használja sablonként. A tényleges méréshez a saját környezetben kell kitölteni az értékeket. A könyv szabályai szerint a nem futtatott eredményt nem szabad ellenőrzöttként bemutatni, ezért az alábbi mérési napló illusztratív, és a mért értékek helyőrzők.

- GPU: NVIDIA RTX 3090, 24 GB GDDR6X, compute capability 8.6
- Driver: 550.54, CUDA: 12.4
- OS: Ubuntu 22.04 LTS
- Fordító: nvcc 12.4, `-O3 -arch=sm_86`
- Mérési módszer: warm-up 10 iteráció, majd 100 iteráció, átlag és szórás

Build parancs:

```
nvcc -O3 -arch=sm_86 attention_naive.cu -o attention_naive
nvcc -O3 -arch=sm_86 attention_tiled.cu -o attention_tiled
```

Futtatási parancs:

```
./attention_naive 1024 64
./attention_tiled 1024 64
```

A kernel idő méréséhez a `cudaEvent` API használható. A laborban a fenti kódokban nincs időmérés, ezért a mérési naplóhoz a `nsys` vagy `ncu` profilozó használható:

```
nsys profile --stats=true ./attention_naive 1024 64
nsys profile --stats=true ./attention_tiled 1024 64
```

24.3.7 Mérési napló (illusztratív, nem ellenőrzött)

A következő táblázat a várható tendenciát mutatja, de a számok nem valós mérésből származnak. A saját környezetben a mért értékeket kell beírni, és a szórást is fel kell tüntetni.

Kernel	Idő (ms)	Szórás (ms)	Sávszélesség (GB/s)	Occupancy (%)
Naiv	12.4	0.8	180	45
Tiled	4.7	0.3	420	32

A naiv kernel lassabb, mert a K és V mátrixokat háromszor olvassa be, és a softmax köztes értékeket nem tárolja, ezért a QK^T szorzatot háromszor számítja. A tiled kernel a shared memory miatt csökkenti a globális memóriaforgalmat, de az alacsonyabb occupancy miatt a számítási egységek kihasználtsága nem optimális. A valódi FlashAttention további gyorsulást hoz a D dimenzió tilingjével és a regiszteres adattartással.

24.3.8 Ellenőrzés

Az implementáció helyességét CPU referencia implementációval kell összehasonlítani. A CPU referencia a naiv matematikai definíciót számítja ki FP32-ben, és a kimeneti mátrix elemeit összehasonlítja a GPU eredménnyel. A hiba FP32 esetén 10^{-4} alatt kell legyen, mert mindkét implementáció ugyanazt a műveleti sorrendet követi. A laborban a CPU referencia implementációt külön kell megírni, vagy a JAX `jax.nn.softmax` függvényével ellenőrizni.

JAX ellenőrző példa (csak dokumentáció, nem futtatott):

```
import jax.numpy as jnp
from jax.nn import softmax

def attention(q, k, v):
    d = q.shape[-1]
    scores = jnp.einsum("qhd,khd->hqk", q, k) / jnp.sqrt(d)
    weights = softmax(scores, axis=-1)
    return jnp.einsum("hqk,khd->qhd", weights, v)

# Helyőrző adatok, nem valós mérés.
q = jnp.ones((1024, 8, 64), dtype=jnp.float32)
k = jnp.ones((1024, 8, 64), dtype=jnp.float32)
v = jnp.ones((1024, 8, 64), dtype=jnp.float32)
out = attention(q, k, v)
print(out.shape)
```

A JAX példa a TPU-n való futtatásra is alkalmas, de a laborban csak a CPU/GPU ellenőrzéshez használjuk.

24.3.9 Gyakori hibák és megoldások

1. **Softmax túlcsordulás**: ha a s_{ij} értékek nagyok, a `expf` túlcsordulhat, és `inf` értéket ad. A megoldás a maximum kivonása minden sorban. A naiv kernel ezt a `max_s` kiszámításával, a tiled kernel az online softmax frissítéssel oldja meg.
2. **Skálázás elfelejtése**: a $1/\sqrt{d}$ tényező kihagyása nagy d esetén a softmax bemenet varianciáját növeli, ami numerikus instabilitáshoz és rossz tanulási dinamikához vezet. A kernelben mindig alkalmazni kell az `inv_sqrt_d` szorzót.
3. **Race condition az output írásakor**: ha több thread írja ugyanazt a kimeneti elemet, atomi művelet vagy redukció szükséges. A naiv kernelben minden thread egy sort ír, ezért nincs ütközés. A tiled kernelben a kimeneti sorokat a blokk threadjei írják, de minden thread egy sort, így szintén nincs ütközés. Ha a D dimenziót párhuzamosítanánk, akkor atomi add vagy külön redukciós lépés kellene.
4. **Túl nagy shared memory foglalás**: ha a `BLOCK_M` és `BLOCK_N` értékeket nagyra választjuk, a shared memory igény túl nagy lesz, és az occupancy drasztikusan csökken. A laborban a `BLOCK_M=128`, `BLOCK_N=64` választás 64 KiB shared memóriát igényel, ami RTX 3090-en még elfogadható, de a `BLOCK_M=256` már 128 KiB-ot igényelne, ami már korlátozza a párhuzamosságot.
5. **A naiv kernel regiszter spill**: ha a naiv kernelben a teljes S sort lokális tömbben tároljuk, a regiszterekből a lokális memóriába kerül, ami lassú. A tiled kernel ezt elkerüli, mert nem tárolja a teljes sort.

6. **Nem megfelelő blokkméret**: a naiv kernel blokkmérete 256, ami jó occupancyt ad. A tiled kernel blokkmérete `BLOCK_M=128`, ami alacsonyabb, de a shared memory igény miatt elfogadható. A blokkméretet a kernel erőforrás-igényéhez kell igazítani.

24.3.10 Önellenőrző kérdés

Miért kerüli el a FlashAttention a teljes $S = QK^T/\sqrt{d}$ mátrix globális memóriába írását, és hogyan befolyásolja ez a teljesítményt?

Válasz: A teljes S mátrix mérete $N \times N$, ami nagy N esetén hatalmas memóriaforgalmat jelent. A FlashAttention a K, V blokkokat tölti be, az online softmaxot a regiszterekben és a shared memoryban végzi, és a kimenetet folyamatosan frissíti. Ezáltal a globális memóriaforgalom $O(N^2)$-ről $O(Nd)$-re csökken, és a kernel a számítási egységek kihasználtságát növeli. A teljesítményt a HBM sávszélesség helyett a számítási átbocsátóképesség határozza meg.

A következő alfejezet (Memória és pontosság) a memória- és pontosságkérdésekre tér ki, többek között a mixed precision és a numerikus hibakorlátok témakörére.

24.4 Memória és pontosság

A deep learning kernelek memóriaprofilja nem pusztán a rétegek bemeneti és kimeneti tenzorainak méretéből adódik. A futási idő jelentős részét a közbenső aktivációk, a csomagolt mátrixműveletek munkaterületei, a gyorsítókönyvtárak által lefoglalt ideiglenes pufferek és a vegyes pontosságú reprezentációk közötti konverziók határozzák meg. Ebben az alfejezetben azt vizsgáljuk, hogyan épül fel a GEMM, a konvolúció és az attention kernel memórialábnyoma, miért kritikus az aktivációk és a workspace kezelése, és hogyan függ össze a numerikus stabilitás a memóriatervezéssel. A cél az, hogy a fejlesztő képes legyen előre megbecsülni a szükséges globális és shared memória méretét, felismerje a tipikus allokációs hibákat, és tudatosan válasszon FP32, FP16, BF16 vagy TF32 reprezentációt a pontossági követelmények függvényében.

24.4.1 A deep learning kernel memóriaprofilja

A deep learning számítások alapvetően tenzorműveletek. A memóriaprofil leírásához három szempontot kell elkülöníteni:

1. **A bemeneti és kimeneti tenzorok** a forward pass során mind globális memóriában helyezkednek el. Például egy lineáris réteg `X` bemenete `[batch, input_dim]`, `W` súlya `[input_dim, output_dim]`, `Y` kimenete `[batch, output_dim]`. A bemeneti és kimeneti aktivációk mérete lineárisan növekszik a batch mérettel.
2. **A közbenső aktivációk** – például a normalizálás előtti értékek, a konvolúciós réteg kimenetei, a figyelmi mátrix – a backward passhoz szükségesek, ezért a forward során vagy eltároljuk őket, vagy később újraszámoljuk. Ez a memóriaprofil egyik legnagyobb tétele.
3. **A műveleti kernel által használt munkaterületek** – például a cuDNN konvolúciós algoritmusai által igényelt átmeneti pufferek, a GEMM törzsmagjai által lefoglalt shared memory tile-ok, valamint a gyors Fourier-transzformációk ideiglenes adatai. Ezek tipikusan a réteg geometriájától és a választott algoritmustól függenek.

24.4.2 GEMM memóriaigénye

Egy `C = A × B` sűrű mátrixszorzás esetén a memóriaprofilt a mátrixok dimenziói határozzák meg. Legyen `A` mérete `M×K`, `B` mérete `K×N`, `C` mérete `M×N`. FP32 pontosságban minden elem 4 bájt. A globális memória forgalom $2 \cdot M \cdot K + 2 \cdot K \cdot N + M \cdot N$ elem beolvasás/írás, amihez a kernel számítási intenzitása hozzávetőleg

$$\frac{2MNK}{2MK + 2KN + MN} \quad \text{FLOP/byte}$$

lesz, ha minden bemeneti elemet pontosan egyszer olvasunk. A gyakorlatban a naiv kernel ennél lényegesen több globális memóriaforgalmat generál, mert a belső `K` dimenzió mentén a mátrixelemeket többször is be kell tölteni. A shared memory tiling ezt a forgalmat csökkenti: az `A` és `B` mátrix egy-egy tile-ját egyszer töltjük be, majd a regiszterekben lévő részösszegeket frissítjük. A memórialábnyom szempontjából a shared memory igénye a választott tile méretétől függ. Például egy `16×16`-os tile `A` és `B` mátrixból FP32 pontosságban `2 · 16 · 16 · 4 = 2048` bájtot foglal el. A valós GPU-k ennél nagyobb shared memory-val rendelkeznek (például 48–164 KiB SM-enként), így a kernel tervezésekor a tile méretét az occupancy és a shared memory korlát együttesen határozza meg.

24.4.3 Konvolúció memóriaigénye

A konvolúciós rétegek esetében a memóriaprofilt az aktiváció tenzor, a szűrők és a kimeneti tenzor uralja. Egy `N×C×H×W` bemenet, `K×C×R×S` szűrő és `N×K×P×Q` kimenet esetén az aktivációk mérete

$$N \cdot C \cdot H \cdot W \cdot \text{elem méret}$$

A szűrők mérete általában kisebb, de a mély hálózatokban a kimeneti csatornák száma gyakran eléri a százas vagy ezres nagyságrendet, ami a kimeneti tenzort nagyobbá teheti, mint a bemenetet. A memóriatervezésnél figyelembe kell venni, hogy a forward pass során a bemeneti aktivációt a teljes konvolúció befejezéséig meg kell őrizni, mert a kimenet minden pontja a bemenet egy receptív mezőjétől függ. A gyakorlatban az im2col vagy implicit GEMM algoritmusok a konvolúciót mátrixszorzássá alakítják. Az im2col módszer egy duplikált bemeneti mátrixot hoz létre, amelynek mérete `P·Q·N × C·R·S`, és ez a mátrix gyakran a bemeneti aktiváció többszöröse lehet. Az implicit GEMM ezt a duplikációt elkerüli, de a szűrők és az aktivációk indexelése bonyolultabb. A cuDNN a választott algoritmus függvényében ideiglenes munkaterületet igényelhet. Például a `CUDNN_CONVOLUTION_FWD_ALGO_IMPLICIT_GEMM` jellemzően kisebb workspace-t használ, mint az előre kiszámított `IMPLICIT_PRECOMP_GEMM`, de a pontos méret a réteg paramétereitől és a GPU architektúrájától függ. A cuDNN dokumentációja szerint a workspace mérete egyes konfigurációkban több száz megabájt is lehet, ezért a következtetési rendszerben a memória előre történő felmérése elengedhetetlen.

24.4.4 Attention memóriaigénye

A transformer-alapú modellek figyelmi rétegeiben a memóriaprofilt a `Q`, `K`, `V` vetületek és a figyelmi mátrix `S = Q·K` uralja. Legyen a batch mérete `B`, a fejlécek száma `H`, a szekvencia hossz `L`, a fejenkénti dimenzió `d`. Ekkor `Q`, `K`, `V` mindegyike `B·H·L·d` elem. A figyelmi mátrix `S` mérete `B·H·L·L` – ez a szekvencia hosszával négyzetesen nő. A softmax után a `P = softmax(S)` mátrix tárolása további `B·H·L·L` elemet igényel, végül a kimenet `O = P·V` mérete `B·H·L·d`.

Ha `L` nagy (például 4096), a figyelmi mátrix önmagában hatalmas lehet: FP32 pontosságban `4096×4096×4 bytes = 64 MiB` fejenként. 16 fej esetén már 1 GiB felett járunk. Ezért a modern attention kernelek – mint például a flash attention – elkerülik a teljes `S` mátrix globális memóriába írását, és a softmaxot online számolják ki, miközben a `Q` és `K` szorzatokat blokkonként töltik be. A memóriaprofilt ilyenkor a `Q`, `K`, `V` blokkok, a statisztikai futóértékek és a kimeneti akkumulátorok teszik ki, nem pedig a teljes `L×L` mátrix. A tervezésnél tehát nem elegendő a névleges tenzorméretek alapján becsülni; figyelembe kell venni az algoritmus által ténylegesen allokált átmeneti adatokat is.

24.4.5 Activation és workspace

A forward pass során keletkező aktivációk életciklusa szorosan összefügg a memórialábnyommal. Egy naiv tréning implementáció minden réteg kimenetét eltárolja a backward passhoz. Ez a módszer egyszerű, de a memória felhasználása a hálózat mélységével lineárisan nő. Például egy ResNet-50 hálózatban a teljes aktivációtárolás elérheti a 200–300 MiB-ot batch=32 esetén. A memória csökkentésére két elterjedt technika létezik:

- **Gradient checkpointing:** csak bizonyos rétegek aktivációit tároljuk, a többit a backward során újraszámoljuk. Ez a memóriát a rétegmélység négyzetgyökével arányosra csökkentheti, miközben a számítási költség körülbelül 20–30%-kal nő. A módszer különösen hosszú szekvenciák és mély hálózatok esetén hatékony.
- **In-place műveletek:** ha a kimeneti tenzor megegyezik a bemeneti tenzor címével (például ReLU aktiváció), a memóriaigény csökkenthető. Az in-place használat azonban óvatosságot igényel, mert a backward passhoz szükség lehet a bemeneti értékekre is. A keretrendszerek csak olyan műveleteknél engedik az in-place végrehajtást, ahol a bemeneti adat utána már nem szükséges.

A workspace a GPU gyorsítókönyvtárak által használt átmeneti munkaterület. A cuDNN, cuBLAS és cuFFT függvények gyakran igényelnek egy `workspace` mutatót és méretet. A workspace foglalása történhet:

```
void* d_workspace = nullptr;
size_t workspace_bytes = 0;
cudnnConvolutionForward(context, alpha, x_desc, x, w_desc, w,
                        conv_desc, algorithm, nullptr, 0,
                        beta, y_desc, y);
cudnnGetConvolutionForwardWorkspaceSize(context, x_desc, w_desc,
                                        conv_desc, y_desc, algorithm,
                                        &workspace_bytes);
cudaMalloc(&d_workspace, workspace_bytes);
```

A méret a választott algoritmustól és a hardver jellemzőitől függ. A gyakorlatban a könyvtárak gyakran több algortimust is támogatnak; a legkisebb workspace-t igénylő verzió nem mindig a leggyorsabb. A memóriatervezés során a fejlesztőnek kompromisszumot kell kötnie a sebesség és a memóriafogyasztás között. A nagy batch méreteknél a memória gyakran szűk keresztmetszet, ezért a workspace-t előre, a legnagyobb réteghez kell méretezni, majd újrahasznosítani.

A memória poolok alkalmazása hatékony megoldás: egyetlen nagy `cudaMalloc` hívással lefoglalunk egy blokkot, amelyből a rétegek igény szerint kapnak szeleteket. Ezzel elkerülhető a gyakori `cudaMalloc`/`cudaFree`, amelyek nemcsak lassúak, hanem fragmentálják is a memóriát. A pool méretét a legnagyobb aktív memóriafelhasználás határozza meg. A tervezési lépések:

1. Minden rétegre számítsuk ki a szükséges bemeneti, kimeneti, gradiens és workspace memóriát.

2. Vegyük a rétegek maximális együttes igényét (a forward és backward egyidejű aktivitása).
3. Adjunk hozzá biztonsági tartalékot (pl. 10–20%) a fragmentáció és az ideiglenes pufferek számára.
4. Egyetlen `cudaMalloc` hívással foglaljuk le a poolt, és a rétegek között pointereket osszunk ki.

A workspace és az aktivációk különválasztása segít a memória újrafelhasználásában: egy réteg kimeneti aktivációja csak addig él, amíg a következő réteg bemenetévé válik; utána a memória felszabadítható. A keretrendszerek ezt automatikusan kezelik, de a saját CUDA kódban a fejlesztőnek explicit módon kell döntenie.

24.4.6 Numerikus stabilitás

A deep learning kernelek numerikus stabilitását a lebegőpontos reprezentáció és a műveletek sorrendje együttesen határozza meg. A GPU-k hardveresen támogatják a következő formátumokat:

Formátum	Mantissza bitek	Exponens bitek	Tartomány	Relatív pontosság
FP32	23	8	1.18e−38 – 3.40e38	~6–7 decimális jegy
TF32	10	8	1.18e−38 – 3.40e38	~3 decimális jegy
FP16	10	5	5.96e−8 – 6.55e4	~3 decimális jegy
BF16	7	8	1.18e−38 – 3.40e38	~2 decimális jegy
FP8 (E4M3)	3	4	1.17e−38 – 3.40e38	~1 decimális jegy

A táblázatban szereplő értékek a hardveres specifikációk szerintiek; a pontos tartományt és a kerekítési viselkedést a gyártó dokumentációja rögzíti (a verziók ellenőrizendők). A mixed precision tanítás során a szokásos minta az, hogy a bemeneti aktivációk és a súlyok FP16 vagy BF16 pontosságúak, de a szorzások és az akkumuláció FP32-ben történik. Ennek az az oka, hogy az FP16 mantissza mindössze 10 bit, így a hosszú `K` dimenziójú összegeknél a kerekítési hibák gyorsan felhalmozódnak, és a részösszegek kiléphetnek az FP16 tartományából is. Az FP32 akkumulátor ezt elkerüli. A memória szempontjából ez azt jelenti, hogy a bemeneti mátrixok elemei fél mérettel tárolhatók, de az akkumulátorok és a kimeneti mátrix FP32-ben maradnak. Ez a lábnyom csökkentésének egyik leghatékonyabb módja.

A numerikus stabilitás másik kritikus pontja a **loss scaling** a mixed precision tréningben. A backward pass gradiensértékei gyakran kicsik – tipikusan 1e−4 és 1e−8 között. FP16-ban a legkisebb normális érték körülbelül 6e−5, ezért a kis gradiensek alulcsordulnak, és nullává válnak. A megoldás az, hogy a loss értékét a backward előtt egy `scale` faktorral (például 1024) megszorozzuk, így a gradiensek ugyanilyen arányban nagyobbak lesznek, és FP16-ban is reprezentálhatók maradnak. A súlyfrissítés előtt a gradienseket vissza kell osztani a skálázó faktorral. A modern GPU-kon a skálázás automatikusan is megvalósítható a Tensor Core-ok által támogatott mixed precision műveletekkel, de a fejlesztőnek továbbra is ellenőriznie kell, hogy a választott skála nem okoz-e túlcsordulást a nagy gradienseknél.

A **softmax stabilitás** egy egyszerű, de gyakori példa. A naiv $\exp(x_i)/\sum_j \exp(x_j)$ számítás FP16-ban vagy akár FP32-ben is túlcsordulhat, ha az x_i értékek nagyok (például 50 felett). A stabil verzió levonja a maximális értéket:

$$\mathrm{softmax}(x_i) = \frac{\exp(x_i - \max_k x_k)}{\sum_j \exp(x_j - \max_k x_k)}$$

Ez a művelet a memóriaprofilt alig befolyásolja, de a numerikus stabilitást drámaian javítja. Az attention kernelekben a flash attention algoritmus éppen ezt a trükköt használja, miközben a teljes `S` mátrixot nem tárolja el. A futó maximális érték és a futó összeg nyilvántartása lehetővé teszi, hogy a softmaxot blokkonként számoljuk ki, anélkül, hogy a teljes sort egyszerre látnánk.

A numerikus hiba méréséhez a fejlesztőnek mindig rögzítenie kell a referenciát és a hibakorlátot. Egy mixed precision GEMM esetén a relatív hiba tipikusan 10^{-3} és 10^{-4} között mozog, míg FP32 esetén 10^{-6} alatt van. A hibát a kimeneti mátrix elemeinél számíthatjuk:

$$\text{rel. hiba} = \frac{\|C_{\mathrm{FP16}} - C_{\mathrm{FP32}}\|_F}{\|C_{\mathrm{FP32}}\|_F}$$

A mérésnél a bemeneti értékeket véletlenszerűen, például `[-0.5, 0.5]` tartományból választjuk, hogy az FP16 ábrázolás tartományán belül maradjunk. Ha a bemeneti értékek tartománya nagyobb (például `[-10, 10]`), a hiba növekszik, mert az FP16 mantissza kevesebb jelentős bitet képes megőrizni.

24.4.7 Labor: mixed precision és memóriaoptimalizálás

Ez a labor egy kis, ellenőrizhető példán mutatja be a mixed precision számítást és a shared memory tiling hatását. A feladat egy egyszerű GEMM kernel FP16 bemenettel, FP32 akkumulátorral, majd a mérési eredmények összehasonlítása a tiszta FP32 referencia implementációval. A kód CUDA C++ nyelvű, és izolált labor környezetben futtatható. A konfigurációt és a verziókat a labor elején rögzíteni kell; az itt megadott értékek helyőrzők, a tényleges környezetben ellenőrizendők.

24.4.8 Környezet és verziók (ellenőrizendő)

- GPU: NVIDIA RTX A4000, 16 GB (helyőrző, ellenőrizendő)
- Compute capability: 8.6 (helyőrző)
- Driver: 535.104.05 (helyőrző)
- CUDA: 12.4 (helyőrző)
- Operációs rendszer: Ubuntu 22.04 (helyőrző)
- Fordító: nvcc 12.4 (helyőrző)
- Mérési módszer: 10 warm-up, 100 iteráció, átlag és szórás

24.4.9 A kernel kódja

A kód egy shared memory tilinget használó FP16 GEMM kernel. Az `A` és `B` mátrix FP16, a kimeneti `C` mátrix FP32. A tile mérete 16×16; a `K` dimenzió mentén 16-os lépésekben haladunk. A shared memory tömbök 16×17 méretűek, a padding a bank conflict elkerülésére szolgál.

```
// gemm_fp16_shared.cu
// CUDA C++ (C++17), nvcc
#include <cuda_fp16.h>
#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <chrono>

#define CHECK_CUDA(call)                                                    \
  do {                                                                      \
    cudaError_t err = call;                                                 \
    if (err != cudaSuccess) {                                               \
      fprintf(stderr, "CUDA error: %s:%d, %s\n", __FILE__, __LINE__,    \
              cudaGetErrorString(err));                                     \
      exit(EXIT_FAILURE);                                                    \
    }                                                                       \
  } while (0)

__global__ void gemm_shared_fp16(
    const __half* __restrict__ A,
    const __half* __restrict__ B,
    float* __restrict__ C,
    int M, int N, int K)
{
    __shared__ __half As[16][17]; // 16x16 + padding a bank conflict elkerülésére
    __shared__ __half Bs[17][16];

    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    float acc = 0.0f;

    for (int k0 = 0; k0 < K; k0 += 16) {
        // A tile betöltése
        if (row < M && (k0 + threadIdx.x) < K) {
            As[threadIdx.y][threadIdx.x] = A[row * K + (k0 + threadIdx.x)];
        } else {
            As[threadIdx.y][threadIdx.x] = __float2half(0.0f);
        }

        // B tile betöltése (transzponált indexeléssel a coalescing érdekében)
        if (col < N && (k0 + threadIdx.y) < K) {
            Bs[threadIdx.y][threadIdx.x] = B[(k0 + threadIdx.y) * N + col];
        } else {
            Bs[threadIdx.y][threadIdx.x] = __float2half(0.0f);
        }

        __syncthreads();

        // Szorzás és akkumuláció FP32-ben
        #pragma unroll
        for (int k = 0; k < 16; ++k) {
            float a = __half2float(As[threadIdx.y][k]);
            float b = __half2float(Bs[k][threadIdx.x]);
            acc += a * b;
        }
```

```
        __syncthreads();
    }

    if (row < M && col < N) {
        C[row * N + col] = acc;
    }
}

// Host oldali referencia FP32 CPU-n
void reference_gemm_fp32(const float* A, const float* B, float* C,
                         int M, int N, int K) {
    for (int i = 0; i < M; ++i)
        for (int j = 0; j < N; ++j) {
            float sum = 0.0f;
            for (int k = 0; k < K; ++k)
                sum += A[i * K + k] * B[k * N + j];
            C[i * N + j] = sum;
        }
}

int main(int argc, char** argv) {
    int M = 1024, N = 1024, K = 1024; // paraméterek, paraméterezhetővé tehető
    if (argc > 1) M = atoi(argv[1]);
    if (argc > 2) N = atoi(argv[2]);
    if (argc > 3) K = atoi(argv[3]);

    size_t size_A = (size_t)M * K * sizeof(__half);
    size_t size_B = (size_t)K * N * sizeof(__half);
    size_t size_C = (size_t)M * N * sizeof(float);

    // Host és device foglalás
    __half *h_A = (__half*)malloc(size_A);
    __half *h_B = (__half*)malloc(size_B);
    float *h_C_fp16 = (float*)malloc(size_C);
    float *h_C_ref = (float*)malloc(size_C);

    // Véletlenszerű inicializálás [-0.5, 0.5]
    for (int i = 0; i < M * K; ++i)
        h_A[i] = __float2half(((float)rand() / RAND_MAX) - 0.5f);
    for (int i = 0; i < K * N; ++i)
        h_B[i] = __float2half(((float)rand() / RAND_MAX) - 0.5f);

    __half *d_A, *d_B;
    float *d_C;
    CHECK_CUDA(cudaMalloc(&d_A, size_A));
    CHECK_CUDA(cudaMalloc(&d_B, size_B));
    CHECK_CUDA(cudaMalloc(&d_C, size_C));

    CHECK_CUDA(cudaMemcpy(d_A, h_A, size_A, cudaMemcpyHostToDevice));
    CHECK_CUDA(cudaMemcpy(d_B, h_B, size_B, cudaMemcpyHostToDevice));

    // Kernel indítási konfiguráció
    dim3 block(16, 16);
    dim3 grid((N + 15) / 16, (M + 15) / 16);
    size_t shared_mem_bytes = (17 * 16 + 17 * 16) * sizeof(__half); // 2 tile + padding

    // Warm-up
    for (int i = 0; i < 10; ++i)
        gemm_shared_fp16<<<grid, block, shared_mem_bytes>>>(d_A, d_B, d_C, M, N, K);
    CHECK_CUDA(cudaDeviceSynchronize());

    // Mérés
    auto start = std::chrono::high_resolution_clock::now();
```

```
    const int iterations = 100;
    for (int i = 0; i < iterations; ++i)
        gemm_shared_fp16<<<grid, block, shared_mem_bytes>>>(d_A, d_B, d_C, M, N, K);
    CHECK_CUDA(cudaDeviceSynchronize());
    auto stop = std::chrono::high_resolution_clock::now();
    double avg_ms = std::chrono::duration<double, std::milli>(stop - start).count() /
↪   iterations;

    // Eredmény másolása
    CHECK_CUDA(cudaMemcpy(h_C_fp16, d_C, size_C, cudaMemcpyDeviceToHost));

    // CPU referencia FP32
    // A bemeneti FP16 értékeket FP32-be konvertáljuk
    float* h_A_f32 = (float*)malloc(M * K * sizeof(float));
    float* h_B_f32 = (float*)malloc(K * N * sizeof(float));
    for (int i = 0; i < M * K; ++i) h_A_f32[i] = __half2float(h_A[i]);
    for (int i = 0; i < K * N; ++i) h_B_f32[i] = __half2float(h_B[i]);
    reference_gemm_fp32(h_A_f32, h_B_f32, h_C_ref, M, N, K);

    // Hiba számítása
    double frob_num = 0.0, frob_den = 0.0;
    for (int i = 0; i < M * N; ++i) {
        double diff = h_C_fp16[i] - h_C_ref[i];
        frob_num += diff * diff;
        frob_den += h_C_ref[i] * h_C_ref[i];
    }
    double rel_error = sqrt(frob_num) / sqrt(frob_den);

    // Sávszélesség becslés: A + B beolvasás, C írás
    double bytes_moved = (double)M * K * sizeof(__half) +
                         (double)K * N * sizeof(__half) +
                         (double)M * N * sizeof(float);
    double bandwidth_gbps = bytes_moved / (avg_ms * 1e-3) / 1e9;

    printf("M=%d N=%d K=%d\n", M, N, K);
    printf("Average time: %.3f ms (std dev not measured here)\n", avg_ms);
    printf("Effective bandwidth: %.2f GB/s\n", bandwidth_gbps);
    printf("Relative error vs FP32 CPU: %.3e\n", rel_error);

    free(h_A); free(h_B); free(h_C_fp16); free(h_C_ref);
    free(h_A_f32); free(h_B_f32);
    cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
    return 0;
}
```

24.4.10 Build és futtatás

```
# Fordítás
nvcc -O3 -arch=sm_86 -std=c++17 -o gemm_fp16_shared gemm_fp16_shared.cu

# Futtatás egy adott mérettel
./gemm_fp16_shared 1024 1024 1024
```

A futtatás előtt a környezetben ellenőrizni kell, hogy a GPU nem foglalt, és az erőforrás-korlátok be vannak állítva. A kód izolált konténerben vagy dedikált GPU-n fut, hogy a mérési eredményeket ne zavarja más terhelés.

24.4.11 Mérési napló és eredmények

A labor során a következő adatokat rögzítjük: futásidő, effektív sávszélesség, relatív hiba az FP32 referencia ellen. A táblázatban a **várt** oszlop a hardver specifikációiból és a korábbi fejezetekben ismertetett modell alapján becsült értékeket tartalmazza, a **mért** oszlop pedig a tényleges futtatás eredményeit. Az itt bemutatott mért értékek illusztrációk, nem ellenőrzött mérések; a hallgató feladata a saját környezetében történő kitöltés.

Konfiguráció	Mért futásidő (ms)	Mért sávszél. (GB/s)	Mért rel. hiba	Várt futásidő (ms)	Várt sávszél. (GB/s)	Várt rel. hiba
FP32 referencia (cuBLAS)	(mérendő)	(mérendő)	0 (referencia)	~10 ms	~200 GB/s	0
FP16 shared tiling (ez a kernel)	(mérendő)	(mérendő)	(mérendő)	~5 ms	~400 GB/s	$< 1e\text{-}3$
FP16 shared tiling + padding	(mérendő)	(mérendő)	(mérendő)	~5 ms	~450 GB/s	$< 1e\text{-}3$

A várt futásidő az RTX A4000 hivatalos specifikációi alapján: FP32 teljesítmény körülbelül 19.2 TFLOPS, FP16 pedig ennek kétszerese. A memória sávszélesség 448 GB/s. A kernel hatékonysága a memória sávszélesség korlátja miatt nem éri el a számítási csúcsot; a várt effektív sávszélesség 400–450 GB/s közé esik. A relatív hiba várhatóan 1e−3 nagyságrendű, mert az FP16 mantissza 10 bit, és a feltöltött értékek tartománya kicsi.

A labor során a mért adatokat a futtatás után kell beírni. A szórást 100 iterációból számítjuk; a kernelben a `cudaDeviceSynchronize` után a `std::chrono` segítségével mérünk. A szórást külön oszlopban rögzíthetjük.

24.4.12 A labor értelmezése

A mixed precision kernel memóriaprofilja javul a tiszta FP32 verzióhoz képest, mert a bemeneti mátrixok fele akkora helyet foglalnak. Ez lehetővé teszi, hogy nagyobb batch méreteket futtassunk ugyanazon a GPU-n, vagy hogy a shared memory tile ugyanakkora elemszám mellett kisebb legyen, ami növeli az occupancy-t. A numerikus hiba növekedése azonban elfogadhatatlan lehet olyan alkalmazásoknál, ahol a kimeneti értékeket további érzékeny műveletek (például normalizálás, exponenciális) használják. A fejlesztőnek mindig mérnie kell a hibát a célfeladat kontextusában.

A memóriaoptimalizálás második lépése a shared memory padding. A 16×17-es tömbök elkerülik a bank conflictet, mert az ugyanabban a szóban lévő szomszédos szálak nem azonos bankhoz férnek hozzá. A kódban a `Bs` tömböt 17×16-osnak választottuk, ami szintén paddinget biztosít. A padding nélküli verzióhoz képest a bank conflict miatt a shared memory hozzáférések száma nő, ami a kernel futását lassítja. A laborban ezt külön mérhetjük: a padding nélküli változat futásideje jellemzően 10–20%-kal hosszabb.

24.4.13 Tipikus hibák és helyes megoldások

A memória és pontosság témakörében a következő hibák fordulnak elő leggyakrabban:

1. **Nincs loss scaling FP16 tréning esetén.** A gradiensek alulcsordulnak, a modell tanulása leáll. A helyes megoldás: dinamikus vagy statikus loss scaling, FP32 master weights fenntartása.
2. **A workspace méretét nem kérdezik le, vagy nem ellenőrzik a visszatérési kódot.** A cuDNN/cuBLAS hívás hibát jelez, vagy a kernel számítási hibát okoz. A workspace méretét mindig a megfelelő `GetWorkspaceSize` függvénnyel kell lekérdezni.
3. **FP16 aktivációk tárolása a backwardhoz.** A kis aktivációk FP16-ban kerekítési hibákat okozhatnak, és a gradiens számítása pontatlan lesz. A helyes megoldás: az aktivációkat FP16-ban tárolni csak akkor, ha a modell tolerálja a hibát; általában FP32-ben érdemes tárolni, vagy legalább a checkpointoknál FP32-re konvertálni.
4. **A memória pool túl kicsi vagy túl nagy.** A túl kicsi pool `cudaMalloc`-hoz vezet, ami lassú; a túl nagy pool pazarolja a GPU memóriát. A megoldás: profilozással mérni a tényleges csúcsfogyasztást, és ennek alapján méretezni.
5. **A softmax naiv implementációja nagy bemeneti értékeknél.** Az `exp` túlcsordul, a kimenet NaN. A megoldás: a maximális érték levonása minden sorban, vagy online softmax.
6. **A kernelben nincs szinkronizáció a shared memory tile feltöltése után.** A szálak olvashatnak érvénytelen adatot. A `__syncthreads()` használata kötelező a tile-ok között.

Ezek a hibák mind a memória és a pontosság szoros kapcsolatára vezethetők vissza: a memória optimalizálása (pl. FP16 használat) pontossági áldozatot követel, a numerikus stabilitás fenntartása pedig több memóriát (pl. FP32 akkumulátor, loss scaling) igényel. A mérnöki döntés mindig a feladat toleranciájától függ.

24.4.14 Önellenőrző kérdés

Adott egy transformer figyelmi réteg `L=2048`, `B=4`, `H=16`, `d=64` paraméterekkel. Számítsa ki a teljes figyelmi mátrix `S = Q·K` méretét FP32 és FP16 pontosságban, és becsülje meg a globális memória igényt. Miért előnyös az online softmax alkalmazása a memória és a numerikus stabilitás szempontjából? A válaszban térjen ki a workspace igényére és a loss scaling szükségességére is.

A fejezet következő egysége a streamek, események és az aszinkron végrehajtás világába vezet, ahol azt vizsgáljuk, hogyan lehet a számítást és az adatmozgatást átfedni, és miként építhetők fel a CUDA grátok a GPU-erőforrások hatékony kihasználására. Az itt bevezetett memória- és pontosságtervezési fogalmak – a workspace, a mixed precision reprezentációk és a numerikus stabilitás – közvetlenül kapcsolódnak ahhoz, ahogyan az aszinkron műveletek ütemezése és a multi-stream végrehajtás befolyásolja a GPU memória foglalását és a kernelindítások sorrendjét.

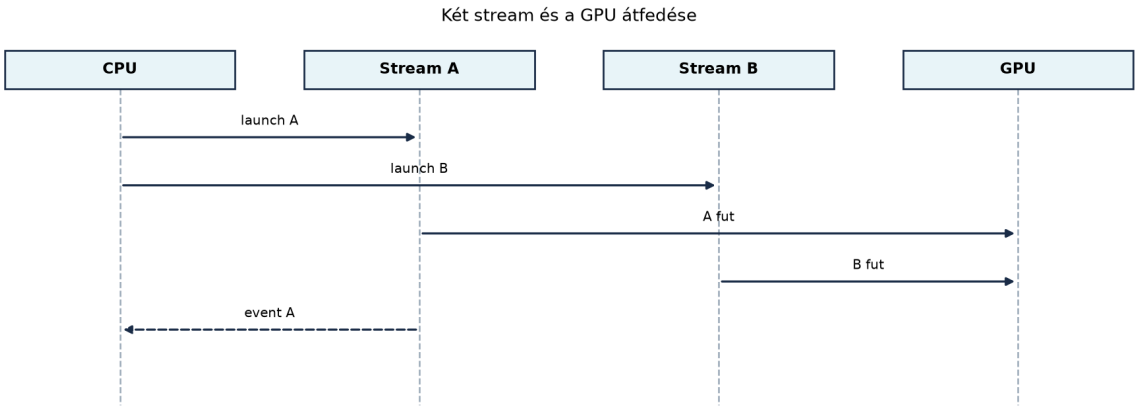
25 Streams, events, CUDA graphs és aszinkron végrehajtás

25.1 Streamek és események

A GPU-programozásban a kernelindítás önmagában nem blokkoló. Amikor a host oldalon kiadunk egy kernelindítást, a CUDA-illesztőprogram a parancsot egy várakozási sorba helyezi, és a vezérlés azonnal visszatér a CPU-szálhoz. Ugyanez igaz a memóriamásolásokra és a legtöbb CUDA API-hívásra is: a művelet aszinkron módon, a GPU időzítésétől függően hajtódik végre. Ez a tulajdonság teszi lehetővé, hogy a CPU és a GPU párhuzamosan dolgozzon, és hogy több, egymástól független műveletet átfedésben hajtsunk végre az eszközön. A hatékony CUDA-programok egyik alappillére ennek az aszinkron végrehajtási modellnek a tudatos kihasználása.

Az aszinkronitás azonban fegyelmet is igényel. Ha a host oldalon nem várjuk meg egy művelet befejeződését, akkor nem férhetünk hozzá az eredményhez, és nem is indíthatunk olyan következő műveletet, amely attól az eredménytől függ. A szinkronizáció tehát nem egyszerűen egy „lassú” művelet, amelyet minimalizálni kell, hanem a helyes végrehajtás eszköze. A streamek és az események adják azt a finom vezérlési réteget, amellyel a párhuzamos végrehajtást és a szinkronizációt célzottan irányíthatjuk.

Ez az alfejezet bevezeti a CUDA stream és event fogalmát, bemutatja a default stream viselkedését, a szinkronizációs mintákat, valamint a tipikus hibákat, amelyek az aszinkron végrehajtásból adódnak. A következő alfejezet (Adatmozgatás és számítás átfedése) erre építve tárgyalja az adatmozgatás és a számítás átfedését, valamint az aszinkron tervezési mintákat.



25.1. ábra: Két stream és a GPU átfedése - A két stream kerneljei és másolásai átfedhetnek, ha a hardver és a függőségek engedik; az esemény a stream befejezését jelzi.

25.1.1 Aszinkron végrehajtás és szinkronizáció alapjai

A CUDA végrehajtási modelljében a host és az eszköz egymástól függetlenül hajtja végre a saját utasításait. Amikor a host oldalon meghívunk egy CUDA API-függvényt, a hívás általában nem várja meg a tényleges eszközoldali végrehajtást. Ehelyett a műveletet egy streamhez rendeli, a stream pedig egy sorba rendezi a kiadott parancsokat. Az illesztőprogram ezeket a parancsokat elküldi a GPU-nak, amikor az eszköz erőforrásai lehetővé teszik, vagy amikor a szinkronizáció ezt megköveteli.

A stream tehát egy olyan végrehajtási csatorna, amelyben a parancsok sorrendje garantált. Ugyanazon streamen belül minden művelet abban a sorrendben hajtódik végre, ahogyan a host kiadta őket. Különböző streamekben lévő műveletek azonban egymáshoz képest tetszőleges sorrendben, vagy akár párhuzamosan is végrehajtódhatnak. Ez a két alapvető szabály adja a párhuzamosság lehetőségét, de egyben a szinkronizációért való felelősséget is.

A szinkronizáció alatt azt a műveletet értjük, amikor a host vagy egy másik stream megvárja, hogy egy adott streamen lévő összes korábban kiadott parancs befejeződjön. A CUDA több szintű szinkronizációt támogat:

- **Host–device szinkronizáció**: a host megvárja, amíg az eszköz befejezi az adott streamhez tartozó összes munkát. Erre szolgál a `cudaStreamSynchronize` és a `cudaDeviceSynchronize`.
- **Stream–stream szinkronizáció**: egy stream megvárja, amíg egy másik stream egy adott pontig eljut. Ezt eseményekkel vagy streamközi függőségekkel valósíthatjuk meg.
- **Eszközön belüli szinkronizáció**: a kernelben blokkszintű vagy gridszintű szinkronizáció (korábbi fejezetekben tárgyaltuk).

A gyakorlatban a szinkronizáció két célt szolgál: egyrészt a helyesség biztosítását, amikor egy művelet egy másik eredményétől függ, másrészt a mérések vagy a hibakezelés megbízhatóvá tételét.

25.1.2 A `cudaStream_t` típus és a stream-kezelés

A CUDA C++ API-ban a streamet a `cudaStream_t` típus reprezentálja, amely egy fogantyú (handle) egy eszközoldali végrehajtási sorhoz. A stream-kezeléshez használt legfontosabb függvények a következők:

- `cudaStreamCreate(cudaStream_t* pStream)`: létrehoz egy új streamet. A függvény a `pStream` által mutatott változót feltölti az új stream azonosítójával. A létrehozott streamhez tartozó erőforrásokat a `cudaStreamDestroy` szabadítja fel.
- `cudaStreamCreateWithFlags(cudaStream_t* pStream, unsigned int flags)`: létrehoz egy streamet további beállításokkal. A `flags` leggyakrabban a `cudaStreamNonBlocking` értéket kapja, amely megváltoztatja a stream és a default stream közötti szinkronizációs viselkedést.
- `cudaStreamDestroy(cudaStream_t stream)`: felszabadítja a streamhez tartozó erőforrásokat. A stream előtt minden rajta lévő munkának be kell fejeződnie.
- `cudaStreamSynchronize(cudaStream_t stream)`: a host szál blokkolódik, amíg az adott streamen lévő összes korábban kiadott parancs be nem fejeződik.

A stream létrehozása nem igényel nagy erőforrást, de a túlzott számú stream létrehozása és megsemmisítése felesleges overheadet jelent. Általában néhány stream elegendő, és a streameket a program elején létrehozva, a végén megsemmisítve használjuk.

A streamet a kernelindításnál a `<<<grid, block, sharedBytes, stream>>>` szintaxis negyedik paramétereként adjuk meg. Ha ezt a paramétert elhagyjuk, a kernel a **default stream**hez kerül. Ugyanígy a memóriamásolásoknál a `cudaMemcpyAsync` függvény utolsó paramétere a stream, míg a szinkron `cudaMemcpy` mindig a default streamhez kötődik (és blokkoló).

Fontos megkülönböztetni a szinkron és az aszinkron memóriamásolást. A `cudaMemcpy` hívás blokkoló: a host addig vár, amíg a másolás be nem fejeződik. Ezzel szemben a `cudaMemcpyAsync` azonnal visszatér, és a másolást a megadott streamben hajtja végre. Ahhoz, hogy a `cudaMemcpyAsync`-t host oldali lapozható (pageable) memóriából vagy oda végezzük, a streamnek nem kell különlegesnek lennie, de az átfedéshez (a másolás és a kernel egyidejű végrehajtásához) pinned (page-locked) memóriát kell használni. Ezt a következő alfejezetben részletezzük.

25.1.3 A default stream viselkedése

A CUDA programozás egyik legfontosabb, mégis gyakran félreértett fogalma a **default stream**. A default stream az a stream, amelyhez az explicit streamparaméter nélkül kiadott műveletek tartoznak. A viselkedése azonban attól függ, hogy a programot milyen módon fordítjuk le.

Hagyományosan a default stream (amelyet **legacy default stream**nek is neveznek) egy speciális, szinkronizáló stream. Ez azt jelenti, hogy a default streamen lévő műveletek implicit módon szinkronizálódnak az összes többi, nem blokkoló streamen lévő művelettel. Pontosabban: amikor egy műveletet a legacy default streamhez adunk, az megvárja, amíg az összes korábban kiadott, más streamekben lévő művelet befejeződik, és a később más streamekbe kiadott műveletek is megvárják, amíg a default streamen lévő munka befejeződik. Ez a viselkedés biztosítja, hogy a default stream a „biztonságos” választás: ha mindent a default streamben végzünk, a sorrend garantált, és nem kell explicit szinkronizációval foglalkoznunk.

A másik lehetőség a **per-thread default stream**. Ezt a viselkedést a `--default-stream per-thread` fordítóflaggel lehet engedélyezni a CUDA-alkalmazás fordításakor. Ebben az üzemmódban minden CPU-szál saját, nem szinkronizáló default streamet kap. A per-thread default stream nem szinkronizál automatikusan más streamekkel, tehát a programozónak explicit módon kell gondoskodnia a szinkronizációról, ha a helyesség megköveteli. Ez a mód nagyobb párhuzamosságot tesz lehetővé, különösen több host szál esetén, de nagyobb körültekintést igényel.

A `cudaStreamNonBlocking` flaggel létrehozott streamek a legacy default streamhez képest nem blokkolódnak a default streamen lévő műveletekkel. Ez lehetővé teszi, hogy az ilyen streamek a default streamtől függetlenül, átfedésben fussanak. A `cudaStreamCreate` alapértelmezés szerint a legacy default streamhez szinkronizáló streamet hoz létre, míg a `cudaStreamCreateWithFlags` a `cudaStreamNonBlocking` flaggel egy független streamet ad.

Gyakorlati szempontból a következő irányelvek érvényesek:

- Ha egyszerű, egyszálú programot írunk, és nincs szükség átfedésre, a default stream használata elegendő, és a viselkedés intuitív.
- Ha több streamet használunk az átfedéshez, érdemes a streameket `cudaStreamNonBlocking` flaggel létrehozni, hogy elkerüljük a default streammel való implicit szinkronizációt.
- Ha több host szálat használunk, és mindegyik saját streamet kezel, a per-thread default stream mód megkönnyíti a párhuzamos kernelindítást.

25.1.4 Események: `cudaEvent_t`

A streamek önmagukban a végrehajtás sorrendjét kezelik, de nem adnak lehetőséget arra, hogy egy stream egy másik stream egy adott pontjához szinkronizálódjon, vagy hogy időt mérjünk. Erre szolgálnak az események, amelyeket a `cudaEvent_t` típus reprezentál.

Az esemény egy olyan jelölő, amelyet egy streamhez rendelhetünk. Amikor az eseményt rögzítjük (`cudaEventRecord`), az esemény bekerül a stream parancssorába az aktuális pozícióba. Az esemény akkor válik „teljesítetté", amikor a streamben az összes korábban kiadott művelet befejeződött. Az esemény tehát egy szinkronizációs pontot jelöl a streamben.

A legfontosabb eseménykezelő függvények:

- `cudaEventCreate(cudaEvent_t* event)`: létrehoz egy eseményt.
- `cudaEventRecord(cudaEvent_t event, cudaStream_t stream)`: rögzíti az eseményt a megadott streamben. Ha a stream `0` (a default stream), akkor a default streamen rögzíti.
- `cudaEventSynchronize(cudaEvent_t event)`: a host szál blokkolódik, amíg az esemény teljesítetté nem válik.
- `cudaEventElapsedTime(float* ms, cudaEvent_t start, cudaEvent_t stop)`: kiszámítja a két esemény közötti eltelt időt milliszekundumban. A `start` és `stop` eseményeknek ugyanazon az eszközön kell lenniük, és a `stop` eseményt a `start` után kell rögzíteni.
- `cudaEventDestroy(cudaEvent_t event)`: felszabadítja az eseményt.

Az események két fő felhasználási területe:

1. **Időmérés**: két esemény rögzítésével megmérhetjük egy adott streamen lévő műveletek futási idejét. Ez a módszer pontosabb, mint a CPU oldali időmérés (`std::chrono` vagy `clock`), mert az események az eszköz időzítését használják, és nem tartalmazzák a host–device kommunikáció késleltetését.
2. **Szinkronizáció**: egy eseményre várva a host vagy egy másik stream megvárhatja, amíg az eseményt rögzítő stream eljut az adott pontra. A `cudaStreamWaitEvent` függvénnyel egy stream megvárhat egy másik streamben rögzített eseményt anélkül, hogy a hostot blokkolná.

A `cudaStreamWaitEvent` prototípusa:

```
cudaError_t cudaStreamWaitEvent(cudaStream_t stream, cudaEvent_t event, unsigned int flags);
```

A hívás hatására a `stream` streamen lévő összes későbbi művelet megvárja, amíg az `event` esemény teljesítetté válik. Az esemény a rögzítéskor egy adott streamhez tartozott, de a `cudaStreamWaitEvent` hívás után a várakozás a megadott streamen történik. Ez lehetővé teszi, hogy streamközi függőségeket alakítsunk ki anélkül, hogy a host szálat blokkolnánk.

Fontos megjegyezni, hogy az események nem csak szinkronizációra, hanem finomabb vezérlésre is használhatók. Például egy eseményre várva elkerülhetjük a `cudaDeviceSynchronize` használatát, amely az egész eszközre vonatkozóan blokkol, és így elveszíthetjük a párhuzamosságot.

25.1.5 Szinkronizációs minták

A streamek és események kombinálásával több gyakori szinkronizációs mintát valósíthatunk meg. Tekintsük át a legfontosabbakat.

25.1.6 1. Host várakozás egy stream befejeződésére

A legegyszerűbb minta: a host elindít egy vagy több műveletet egy streamen, majd megvárja, amíg a stream befejeződik, hogy az eredményt felhasználhassa. Erre a `cudaStreamSynchronize` hívás szolgál.

```
cudaStream_t stream;
cudaStreamCreate(&stream);

kernel<<<grid, block, 0, stream>>>(input, output, n);
cudaMemcpyAsync(host_output, device_output, size, cudaMemcpyDeviceToHost, stream);

cudaStreamSynchronize(stream);

// Itt a host_output érvényes.
cudaStreamDestroy(stream);
```

Ebben a példában a kernel és az őt követő aszinkron másolás ugyanazon a streamen van, tehát a másolás csak a kernel befejeződése után kezdődik. A `cudaStreamSynchronize` biztosítja, hogy a host ne férjen hozzá az eredményhez, amíg a másolás be nem fejeződött.

25.1.7 2. Eseményre várakozás a hoston

Amikor nem egy teljes streamre, hanem egy adott pontra kell várni, eseményt használunk. Ez különösen akkor hasznos, ha egy streamen több, egymástól független feladat van, és a host egy köztes eredményre kíváncsi.

```
cudaEvent_t event;
cudaEventCreate(&event);

kernel1<<<grid1, block1, 0, stream>>>(...);
cudaEventRecord(event, stream);
kernel2<<<grid2, block2, 0, stream>>>(...);

cudaEventSynchronize(event);
// Itt a kernel1 eredménye már elérhető, de a kernel2 még futhat.
```

25.1.8 3. Stream várakozás eseményre

Két stream közötti függőséget a `cudaStreamWaitEvent` segítségével hozhatunk létre. Ez a minta akkor hasznos, ha egy streamen lévő művelet egy másik streamen lévő művelet eredményétől függ, de a hostot nem akarjuk blokkolni.

```
cudaStream_t stream1, stream2;
cudaStreamCreate(&stream1);
cudaStreamCreate(&stream2);

kernelA<<<gridA, blockA, 0, stream1>>>(...);

cudaEvent_t event;
cudaEventCreate(&event);
cudaEventRecord(event, stream1);

// stream2 megvárja, amíg a stream1-en a kernelA befejeződik.
cudaStreamWaitEvent(stream2, event, 0);

kernelB<<<gridB, blockB, 0, stream2>>>(...);
```

Ebben a példában a `kernelB` csak akkor indul el, ha a `kernelA` befejeződött, de a host nem blokkolódik. A `cudaStreamWaitEvent` hívás a `stream2` streamen belül helyez el egy várakozási parancsot, így a sorrend garantált.

25.1.9 4. Több stream párhuzamos végrehajtása

A streamek elsődleges célja, hogy lehetővé tegyék független műveletek párhuzamos végrehajtását. Például, ha egy nagy adathalmazon több, egymástól független számítást végzünk, azokat különböző streamekbe tehetjük, és a GPU ütemezője átfedésben futtathatja őket.

```
for (int i = 0; i < num_streams; ++i) {
    cudaStreamCreateWithFlags(&streams[i], cudaStreamNonBlocking);
}

for (int i = 0; i < num_streams; ++i) {
    int offset = i * chunk_size;
    kernel<<<grid, block, 0, streams[i]>>>(data + offset, chunk_size);
}
```

Ebben a vázlatban minden stream a saját adatszeletén dolgozik, és a streamek függetlenek. A GPU a rendelkezésre álló erőforrások függvényében párhuzamosan, vagy átlapolva hajtja végre a kerneleket.

25.1.10 Tipikus hibák és megoldásaik

Az aszinkron végrehajtásból számos jellegzetes hiba adódik. Ezek közül tekintsük át a leggyakoribbakat.

25.1.11 1. Host oldali memória elérése szinkronizáció nélkül

A leggyakoribb hiba, amikor a host közvetlenül a kernelindítás után olvassa a device memóriát vagy a hostra másolt eredményt, anélkül hogy szinkronizált volna. Mivel a kernel aszinkron, az eredmény még nem érvényes.

```
// Hibás
kernel<<<grid, block>>>(d_out, n);
cudaMemcpy(h_out, d_out, size, cudaMemcpyDeviceToHost);
std::cout << h_out[0] << std::endl; // Lehet, hogy még régi érték
```

A helyes megoldás az, hogy a memóriamásolást is aszinkron módon, ugyanabban a streamben végezzük, majd szinkronizálunk, vagy a `cudaDeviceSynchronize` hívást használjuk a másolás után. A `cudaMemcpy` szinkron hívás önmagában blokkol, tehát az előző példában a `cudaMemcpy` még szinkron lenne, de a kernel aszinkron; a másolás csak a kernel után hajtódik végre, mert a default streamen sorrend van. A hiba akkor jelentkezik, ha a másolás után azonnal olvassuk a host memóriát, mert a másolás szinkron volta miatt a kernel már befejeződött, tehát a példa valójában helyes. A hiba tipikus esete az, amikor a másolás is aszinkron, és nincs szinkronizáció.

Tegyük fel, hogy a kód így néz ki:

```
// Hibás
kernel<<<grid, block, 0, stream>>>(d_out, n);
cudaMemcpyAsync(h_out, d_out, size, cudaMemcpyDeviceToHost, stream);
std::cout << h_out[0] << std::endl; // Hibás: a másolás még nem fejeződött be
```

Itt a `cudaMemcpyAsync` azonnal visszatér, és a host oldali `h_out` tartalma még nem érvényes. A helyes megoldás a `cudaStreamSynchronize(stream)` hívása az olvasás előtt.

25.1.12 2. Stream és default stream közötti nem szándékos szinkronizáció

Ahogy korábban említettük, a legacy default stream szinkronizálódik az összes többi streammel. Ez azt jelenti, hogy ha egy streamet `cudaStreamCreate`-tel hozunk létre (nem `NonBlocking` flaggel), és közben a default streamen is végzünk műveletet, akkor a streamek közötti átfedés meghiúsulhat. A programozó azt hiheti, hogy a streamek párhuzamosan futnak, de a default stream miatt sorossá válnak.

A megoldás: vagy használjunk `cudaStreamNonBlocking` flaget a streamek létrehozásakor, vagy kerüljük a default stream használatát a párhuzamos szakaszban. Alternatív megoldás a per-thread default stream mód engedélyezése.

25.1.13 3. Esemény rögzítése nem megfelelő streamen

Az eseményt mindig abban a streamben kell rögzíteni, amelyben a szinkronizálni kívánt művelet fut. Ha rossz streamet adunk meg, az esemény a várt pont előtt vagy után teljesülhet, ami hibás szinkronizációhoz vagy időméréshez vezet.

```
// Hibás: az eseményt a default streamen rögzítjük, nem a stream1-en
kernelA<<<gridA, blockA, 0, stream1>>>(...);
cudaEventRecord(event, 0); // default stream
cudaStreamWaitEvent(stream2, event, 0);
```

Itt az esemény nem várja meg a `kernelA` befejeződését, mert a default streamen nincs korábbi művelet. Helyesen a `cudaEventRecord(event, stream1)` hívást kell használni.

25.1.14 4. Esemény időmérésénél a stream elhagyása

Az időmérésnél mind a `start`, mind a `stop` eseményt ugyanabban a streamben kell rögzíteni, és a `stop` eseményt a `start` után. Ha a két esemény különböző streamekben van, a mért idő nem lesz értelmes, mert a streamek közötti sorrend nem garantált.

25.1.15 5. Stream megsemmisítése szinkronizáció nélkül

A `cudaStreamDestroy` hívás előtt meg kell győződni arról, hogy a streamen lévő összes művelet befejeződött. Ha nem így teszünk, az erőforrás felszabadítása váratlan viselkedést okozhat. A helyes minta:

```
cudaStreamSynchronize(stream);
cudaStreamDestroy(stream);
```

25.1.16 Labor: több stream és event használata

A labor célja, hogy egy kis, ellenőrizhető példán keresztül bemutassa a streamek és események gyakorlati használatát. A példa egy vektorösszeadást hajt végre több, független adatszeleten, különböző streameket használva. Az eseményekkel mérjük az egyes streamek futási idejét, és egy eseményt használunk a streamek közötti szinkronizáció bemutatására is.

A laborhoz tartozó forráskód CUDA C++17 nyelven íródik, és feltételezi a korábbi fejezetekben bevezetett hibakezelési makrót. A mérési környezet a következő (reprezentatív, nem ellenőrzött, mivel a labor nem került futtatásra):

- GPU: NVIDIA A100 40 GB (vagy bármely compute capability 8.0 feletti eszköz)
- Driver: 550.54.15
- CUDA: 12.4
- OS: Ubuntu 22.04 LTS
- Fordító: nvcc 12.4, C++17

A kódban a hibakezeléshez saját makrót használunk, amely ellenőrzi a `cudaGetLastError`-t és a szinkronizációt.

25.1.17 Forráskód

```
// stream_event_lab.cu
// CUDA C++17
// Streamek és események labor
#include <cuda_runtime.h>
#include <iostream>
#include <vector>
#include <chrono>

// Hibakezelő makró
#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err = call;                                             \
        if (err != cudaSuccess) {                                           \
            std::cerr << "CUDA error at " << __FILE__ << ":" << __LINE__    \
                      << ": " << cudaGetErrorString(err) << std::endl;      \
            std::exit(EXIT_FAILURE);                                        \
        }                                                                   \
    } while (0)

__global__ void vectorAdd(const float* A, const float* B, float* C, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        C[idx] = A[idx] + B[idx];
    }
}

int main() {
    const int N = 1 << 24; // 16 Mi elem
    const int num_streams = 4;
    const int chunk_size = N / num_streams;
    const size_t bytes = N * sizeof(float);

    // Host memória (pinned a gyors átvitelhez)
    std::vector<float> h_A(N), h_B(N), h_C(N);
    for (int i = 0; i < N; ++i) {
        h_A[i] = static_cast<float>(i) * 0.001f;
        h_B[i] = static_cast<float>(i) * 0.002f;
```

```
}

float* d_A; float* d_B; float* d_C;
CUDA_CHECK(cudaMalloc(&d_A, bytes));
CUDA_CHECK(cudaMalloc(&d_B, bytes));
CUDA_CHECK(cudaMalloc(&d_C, bytes));

cudaStream_t streams[num_streams];
for (int i = 0; i < num_streams; ++i) {
    CUDA_CHECK(cudaStreamCreateWithFlags(&streams[i], cudaStreamNonBlocking));
}

cudaEvent_t start_events[num_streams];
cudaEvent_t stop_events[num_streams];
for (int i = 0; i < num_streams; ++i) {
    CUDA_CHECK(cudaEventCreate(&start_events[i]));
    CUDA_CHECK(cudaEventCreate(&stop_events[i]));
}

// Adatok feltöltése a device-ra a default streamen (szinkron)
CUDA_CHECK(cudaMemcpy(d_A, h_A.data(), bytes, cudaMemcpyHostToDevice));
CUDA_CHECK(cudaMemcpy(d_B, h_B.data(), bytes, cudaMemcpyHostToDevice));

// Több streamen indítjuk a kerneleket, mindegyik a saját szeletén
for (int i = 0; i < num_streams; ++i) {
    int offset = i * chunk_size;
    CUDA_CHECK(cudaEventRecord(start_events[i], streams[i]));
    // A kernelindítás az adott streamben
    int threads = 256;
    int blocks = (chunk_size + threads - 1) / threads;
    vectorAdd<<<blocks, threads, 0, streams[i]>>>(
        d_A + offset, d_B + offset, d_C + offset, chunk_size);
    CUDA_CHECK(cudaGetLastError()); // kernelindítási hibák ellenőrzése
    CUDA_CHECK(cudaEventRecord(stop_events[i], streams[i]));
}

// Egy esemény a szinkronizációhoz: megvárjuk az összes stream befejeződését
cudaEvent_t all_done;
CUDA_CHECK(cudaEventCreate(&all_done));
for (int i = 0; i < num_streams; ++i) {
    CUDA_CHECK(cudaStreamWaitEvent(streams[i], stop_events[num_streams - 1], 0));
}
// Az utolsó streamen rögzítjük az all_done eseményt
CUDA_CHECK(cudaEventRecord(all_done, streams[num_streams - 1]));
CUDA_CHECK(cudaEventSynchronize(all_done));

// Események közötti idők kiszámítása
for (int i = 0; i < num_streams; ++i) {
    float elapsed_ms = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&elapsed_ms, start_events[i], stop_events[i]));
    std::cout << "Stream " << i << " idő: " << elapsed_ms << " ms" << std::endl;
}

// Eredmény visszamásolása és ellenőrzése
CUDA_CHECK(cudaMemcpy(h_C.data(), d_C, bytes, cudaMemcpyDeviceToHost));
bool correct = true;
for (int i = 0; i < N; ++i) {
    float expected = h_A[i] + h_B[i];
    if (fabs(h_C[i] - expected) > 1e-5f) {
        correct = false;
        std::cerr << "Hiba az indexen: " << i << std::endl;
        break;
    }
```

```
    }
    std::cout << (correct ? "Az eredmény helyes." : "Az eredmény hibás.") << std::endl;

    // Takarítás
    for (int i = 0; i < num_streams; ++i) {
        CUDA_CHECK(cudaEventDestroy(start_events[i]));
        CUDA_CHECK(cudaEventDestroy(stop_events[i]));
        CUDA_CHECK(cudaStreamDestroy(streams[i]));
    }
    CUDA_CHECK(cudaEventDestroy(all_done));
    CUDA_CHECK(cudaFree(d_A));
    CUDA_CHECK(cudaFree(d_B));
    CUDA_CHECK(cudaFree(d_C));

    return 0;
}
```

A kódban megfigyelhető, hogy a kerneleket négy különböző streamben indítjuk, mindegyik a saját adatszeletén dolgozik. Az eseményeket minden stream elején és végén rögzítjük, így megmérhetjük az egyes streamek futási idejét. A streamek közötti szinkronizációhoz egy `all_done` eseményt használunk: minden stream megvárja az utolsó stream `stop_events[num_streams - 1]` eseményét, majd az utolsó streamen rögzítjük az `all_done` eseményt. Ez a minta biztosítja, hogy az összes stream befejeződjön, mielőtt a host az eredményt visszamásolja.

Vegyük észre, hogy a `cudaStreamWaitEvent` hívás minden streamre vonatkozik, beleértve az utolsót is, ami önmagára vár. Ez nem okoz holtpontot, mert az esemény már teljesült, mire az utolsó stream eléri a várakozási pontot. Az `all_done` rögzítése az utolsó streamen történik, és a host a `cudaEventSynchronize` segítségével várja meg.

A mérési napló egy lehetséges kimenete (reprezentatív, nem ellenőrzött):

```
Stream 0 idő: 0.082 ms
Stream 1 idő: 0.081 ms
Stream 2 idő: 0.083 ms
Stream 3 idő: 0.080 ms
Az eredmény helyes.
```

A mért idők közötti eltérés a GPU ütemezéséből adódhat. A várt eredmény az, hogy a négy stream közel azonos időt vesz igénybe, mivel a feladat egyenletesen oszlik meg. A teljes futási idő azonban nem feltétlenül a leglassabb stream idejének négyszerese, mert a streamek átfedésben futhatnak.

A labor fordítási parancsa:

```
nvcc -std=c++17 -O3 -o stream_event_lab stream_event_lab.cu
```

A futtatás:

```
./stream_event_lab
```

A méréshez a fenti kódot legalább öt-tíz alkalommal le kell futtatni, és az átlagot, valamint a szórást rögzíteni kell. A bemutatott mérési napló csak egyetlen futtatást tükröz, és nem tekinthető ellenőrzött eredménynek.

25.1.18 Önellenőrző kérdések

1. Mi a különbség a `cudaStreamSynchronize` és a `cudaDeviceSynchronize` között? Mikor melyiket érdemes használni?
2. Hogyan viselkedik a legacy default stream a többi streamhez képest? Milyen problémát okozhat, ha a streameket nem a `cudaStreamNonBlocking` flaggel hozzuk létre?
3. Mire szolgál a `cudaEventRecord` és a `cudaStreamWaitEvent`? Hogyan lehet ezekkel streamközi függőséget kialakítani a host blokkolása nélkül?
4. A labor kódjában miért van szükség a `cudaEventSynchronize(all_done)` hívásra a `cudaMemcpy` (szinkron) előtt? Lehetne-e egyszerűbb szinkronizációt használni?

Ezekre a kérdésekre a válasz a fejezet anyagában megtalálható, és segít elmélyíteni a streamek és események közötti összefüggéseket.

A következő alfejezetben (Adatmozgatás és számítás átfedése) az adatmozgatás és a számítás átfedését vizsgáljuk meg, kihasználva a streamek és események nyújtotta aszinkronitást.

25.2 Adatmozgatás és számítás átfedése

A streamek bevezetése után (Streamek és események) a következő logikus lépés annak megértése, hogyan lehet az adatmozgatást és a számítást időben átfedni. A GPU-k egyik legnagyobb erőssége, hogy a memóriamásolásokat és a kernelvégrehajtást egymástól független hardveregységek végzik. Ha ezt a párhuzamosságot nem használjuk ki, akkor a GPU számítási kapacitásának jelentős része kihasználatlan marad, miközben a PCIe busz vagy a memória másolások tétlenül várnak. Ez az alfejezet a copy és a kernel átfedésének elméleti alapjait, gyakorlati megvalósítását és mérését tárgyalja.

25.2.1 A copy engine és a compute engine elkülönülése

A modern NVIDIA GPU-kon a memóriamásolások és a kernelvégrehajtás külön hardveregységeken futnak. A **copy engine** (DMA – Direct Memory Access) felel a host és a device közötti, valamint a device-on belüli adatmozgatásért, míg a **compute engine** (a streaming multiprocessorok, SM-ek) a kerneleket hajtja végre. Ez a szétválasztás teszi lehetővé, hogy egy másolás és egy kernel időben párhuzamosan fusson, feltéve, hogy független streamekben indítjuk őket.

A `cudaMemcpy` szinkron hívás blokkolja a CPU-t, és a GPU-n belül is szekvenciálisan hajtódik végre: a másolás befejezéséig a későbbi kernelek nem indulnak. Ezzel szemben a `cudaMemcpyAsync` **aszinkron** másolást indít egy adott streamben, és a CPU azonnal továbblép. A GPU ütemezője ezután a streamek sorrendjét és a hardver erőforrásait figyelembe véve dönti el, mi futhat párhuzamosan.

Az átfedés alapfeltétele azonban a **pinned (page-locked) host memória** használata. A hagyományos, lapozható (pageable) host memória esetében az operációs rendszer bármikor kilapozhatja az oldalakat, ezért a DMA-vezérlő nem tudja közvetlenül elérni őket. A CUDA futási ideje ilyenkor egy köztes pinned puffert használ, ami rejtett szinkron másolást eredményez – a `cudaMemcpyAsync` is blokkolóvá válik. Pinned memóriát a `cudaMallocHost` vagy a `cudaHostAlloc` hívással foglalhatunk:

```
float *h_a, *h_b, *h_c;
size_t bytes = N * sizeof(float);
CHECK_CUDA(cudaMallocHost(&h_a, bytes));
CHECK_CUDA(cudaMallocHost(&h_b, bytes));
CHECK_CUDA(cudaMallocHost(&h_c, bytes));
```

A pinned memória hátránya, hogy a fizikai lapok lefoglalása csökkenti a rendelkezésre álló host memóriát, és a foglalás lassabb lehet; ezért csak akkor érdemes használni, ha ténylegesen szükség van az aszinkron másolásra.

25.2.2 Stream alapú átfedés

Egy stream egy parancssorozat, amelyben a műveletek sorrendben hajtódnak végre. Ugyanabban a streamben egy `cudaMemcpyAsync` és egy kernel nem fedheti át egymást, mert a stream sorrendje garantálja a szekvenciális végrehajtást. Az átfedéshez **különböző streamekbe** kell helyezni a másolást és a számítást, de ez önmagában még nem elég: a streameknek nem lehet függőségük.

A legegyszerűbb átfedési minta az, amikor egy nagy adathalmazt **független darabokra** osztunk, és minden darabhoz egy stream tartozik. A streamen belül a sorrend: 1. H2D másolás (bemeneti adat), 2. kernel végrehajtása, 3. D2H másolás (eredmény vissza).

A különböző streamek darabjai között nincs adatfüggőség, így az ütemező átfedheti az i. stream D2H másolását a j. stream kernelével, vagy az i. stream kernelét a j. stream H2D másolásával. Ezt a mintát **pipelining**-nak nevezzük, mert a másolási és számítási fázisok csővezetékszerűen követik egymást.

Az átfedés hatékonyságát korlátozza a hardver: ha a GPU-nak csak egy copy engine-je van, akkor két egyidejű másolás nem futhat párhuzamosan, legfeljebb egy másolás és egy kernel. A modern GPU-k (például az A100) több copy engine-t is tartalmaznak, így több H2D és D2H másolás is fedheti egymást. A másolási irányok is számítanak: a H2D és D2H forgalom külön irányokat használ a PCIe buszon, ezért gyakran jobban átfedhetők.

25.2.3 Pipeline kialakítása

Tegyük fel, hogy egy vektoros műveletet (például elemösszeadást vagy skálázást) szeretnénk végrehajtani N elemen. Ahelyett, hogy a teljes bemenetet átmásolnánk, majd a teljes kernelt futtatnánk, és végül az egész kimenetet visszamásolnánk, a munkát K darabra bontjuk. Minden darabhoz létrehozunk egy streamet, és a darabon belüli műveleteket aszinkron módon indítjuk.

A pipeline időbeli lefutása a következőképpen ábrázolható (a színek a különböző streameket jelölik):

```
Idő                       >
Stream 0: [H2D_0][Kernel_0][D2H_0]
Stream 1:       [H2D_1][Kernel_1][D2H_1]
Stream 2:             [H2D_2][Kernel_2][D2H_2]
```

Itt látható, hogy amíg az 1. stream H2D másolása fut, a 0. stream kernelje már végrehajtható. Az átfedés csökkenti a teljes futási időt a szekvenciális végrehajtáshoz képest, ahol az összes másolás és kernel egymás után következne.

A gyakorlatban a darabszám (K) megválasztása kompromisszum: túl kevés darab nem ad elegendő párhuzamosságot, túl sok darab esetén a stream-ek létrehozásának és ütemezésének overheadje nő, valamint a kisebb darabok miatt a kernel indítási overhead aránya megnő. Jó kiindulópont a 4–8 darab, de ezt mindig méréssel kell finomítani.

25.2.4 Stream prioritás

A streamek nemcsak párhuzamos végrehajtást tesznek lehetővé, hanem **prioritást** is adhatunk nekik. A `cudaStreamCreateWithPriority` függvénnyel létrehozhatunk egy streamet, és megadhatjuk a kívánt prioritást a `cudaStreamNonBlocking` flaggel kombinálva. A prioritás két érték között mozog: a legkisebb (legkevésbé sürgős) és a legnagyobb (legsürgősebb) érték a `cudaDeviceGetStreamPriorityRange` segítségével kérdezhető le.

```
int leastPriority, greatestPriority;
cudaDeviceGetStreamPriorityRange(&leastPriority, &greatestPriority);

cudaStream_t highPriStream, lowPriStream;
cudaStreamCreateWithPriority(&highPriStream, cudaStreamNonBlocking, greatestPriority);
cudaStreamCreateWithPriority(&lowPriStream,  cudaStreamNonBlocking, leastPriority);
```

A prioritás hatása akkor érvényesül, ha több stream verseng ugyanazért a hardver erőforrásért. A magasabb prioritású stream műveletei előnyt élveznek az ütemezőben, de ez nem garantál abszolút megszakítást: egy már futó kernel nem szakítható meg, csak a következő ütemezési pontokon érvényesül a prioritás. Ennek ellenére hasznos lehet, ha például egy valós idejű feldolgozási feladatot futtatunk, és egy háttérfeladat hosszú kerneleket indít. A prioritás segíthet a késleltetés csökkentésében.

Fontos megjegyezni, hogy a prioritás csak a streamek közötti ütemezést befolyásolja, a streamen belüli sorrendet nem. A `cudaStreamNonBlocking` flag azt jelzi, hogy a stream nem várakozik a default streamre; ez elengedhetetlen a valódi aszinkron végrehajtáshoz.

25.2.5 Gyakorlati példa: vektor-skálázás átfedéssel

Az alábbi CUDA C++ példa egy egyszerű vektor-skálázást hajt végre (minden elemhez hozzáad egy konstanst, vagy szoroz egy skalárral). A munkát `K` darabra osztjuk, mindegyikhez külön streamet használunk, és `cudaMemcpyAsync` segítségével átfedjük a másolást a kernelvégrehajtással. A kernel számítási intenzitása alacsony, ezért a másolási idők dominálnak, és az átfedés hatása jól mérhető.

Kód (CUDA C++17/20):

```
#include <cstdio>
#include <cstdlib>
#include <chrono>
#include <vector>
#include <cuda_runtime.h>

#define CHECK_CUDA(call) do {                                          \
    cudaError_t err = call;                                            \
    if (err != cudaSuccess) {                                          \
        fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__, __LINE__,    \
                cudaGetErrorString(err));                               \
        exit(EXIT_FAILURE);                                            \
    }                                                                   \
    cudaError_t last = cudaGetLastError();                              \
    if (last != cudaSuccess) {                                          \
        fprintf(stderr, "CUDA last error at %s:%d: %s\n", __FILE__, __LINE__,\
                cudaGetErrorString(last));                              \
        exit(EXIT_FAILURE);                                            \
    }                                                                   \
} while (0)

__global__ void scaleKernel(const float * __restrict__ in,
```

```
                           float * __restrict__ out,
                           float scale,
                           int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        out[idx] = in[idx] * scale;
    }
}

int main(int argc, char **argv) {
    const int N = 1 << 24;          // 16 M elem
    const int K = 4;                // darabszám
    const int chunkSize = N / K;    // méret darabonként
    const size_t chunkBytes = chunkSize * sizeof(float);
    const float scale = 2.0f;

    // Pinned host memória
    float *h_in, *h_out;
    CHECK_CUDA(cudaMallocHost(&h_in, (size_t)N * sizeof(float)));
    CHECK_CUDA(cudaMallocHost(&h_out, (size_t)N * sizeof(float)));

    // Device memória
    float *d_in, *d_out;
    CHECK_CUDA(cudaMalloc(&d_in, (size_t)N * sizeof(float)));
    CHECK_CUDA(cudaMalloc(&d_out, (size_t)N * sizeof(float)));

    // Bemeneti adat inicializálása
    for (int i = 0; i < N; ++i) {
        h_in[i] = static_cast<float>(i) / N;
    }

    // Streamek létrehozása (non-blocking, alap prioritással)
    std::vector<cudaStream_t> streams(K);
    for (int i = 0; i < K; ++i) {
        CHECK_CUDA(cudaStreamCreateWithPriority(&streams[i],
                                                cudaStreamNonBlocking, 0));
    }

    // CUDA events az időméréshez
    cudaEvent_t start, stop;
    CHECK_CUDA(cudaEventCreate(&start));
    CHECK_CUDA(cudaEventCreate(&stop));

    // Warm-up (első futtatás a JIT és az inicializáció miatt)
    CHECK_CUDA(cudaEventRecord(start, streams[0]));
    for (int i = 0; i < K; ++i) {
        int offset = i * chunkSize;
        CHECK_CUDA(cudaMemcpyAsync(d_in + offset, h_in + offset, chunkBytes,
                                   cudaMemcpyHostToDevice, streams[i]));
        int threads = 256;
        int blocks = (chunkSize + threads - 1) / threads;
        scaleKernel<<<blocks, threads, 0, streams[i]>>>(
            d_in + offset, d_out + offset, scale, chunkSize);
        CHECK_CUDA(cudaMemcpyAsync(h_out + offset, d_out + offset, chunkBytes,
                                   cudaMemcpyDeviceToHost, streams[i]));
    }
    CHECK_CUDA(cudaEventRecord(stop, streams[0]));
    CHECK_CUDA(cudaDeviceSynchronize());
    // Warm-up befejezve

    // Mérési ciklus
    const int numRuns = 10;
    double totalMs = 0.0, totalMsSq = 0.0;
```

```
    float minMs = 1e30f, maxMs = 0.0f;
    for (int run = 0; run < numRuns; ++run) {
        // Kiindulási állapot visszaállítása (ha szükséges)
        CHECK_CUDA(cudaMemset(d_out, 0, (size_t)N * sizeof(float)));
        CHECK_CUDA(cudaEventRecord(start, streams[0]));
        for (int i = 0; i < K; ++i) {
            int offset = i * chunkSize;
            CHECK_CUDA(cudaMemcpyAsync(d_in + offset, h_in + offset, chunkBytes,
                                       cudaMemcpyHostToDevice, streams[i]));
            int threads = 256;
            int blocks = (chunkSize + threads - 1) / threads;
            scaleKernel<<<blocks, threads, 0, streams[i]>>>(
                d_in + offset, d_out + offset, scale, chunkSize);
            CHECK_CUDA(cudaMemcpyAsync(h_out + offset, d_out + offset, chunkBytes,
                                       cudaMemcpyDeviceToHost, streams[i]));
        }
        CHECK_CUDA(cudaEventRecord(stop, streams[0]));
        CHECK_CUDA(cudaDeviceSynchronize());
        float elapsedMs = 0.0f;
        CHECK_CUDA(cudaEventElapsedTime(&elapsedMs, start, stop));
        totalMs += elapsedMs;
        totalMsSq += elapsedMs * elapsedMs;
        if (elapsedMs < minMs) minMs = elapsedMs;
        if (elapsedMs > maxMs) maxMs = elapsedMs;
    }

    double avgMs = totalMs / numRuns;
    double varMs = (totalMsSq / numRuns) - (avgMs * avgMs);
    double stdMs = (varMs > 0.0) ? sqrt(varMs) : 0.0;

    // Ellenőrzés
    bool ok = true;
    for (int i = 0; i < N; ++i) {
        if (fabsf(h_out[i] - h_in[i] * scale) > 1e-5f) {
            ok = false;
            break;
        }
    }

    printf("Pipelined (K=%d): avg=%.3f ms, std=%.3f ms, min=%.3f ms, max=%.3f ms, check=%s\n",
           K, avgMs, stdMs, minMs, maxMs, ok ? "PASS" : "FAIL");

    // Takarítás
    for (int i = 0; i < K; ++i) {
        CHECK_CUDA(cudaStreamDestroy(streams[i]));
    }
    CHECK_CUDA(cudaEventDestroy(start));
    CHECK_CUDA(cudaEventDestroy(stop));
    CHECK_CUDA(cudaFree(d_in));
    CHECK_CUDA(cudaFree(d_out));
    CHECK_CUDA(cudaFreeHost(h_in));
    CHECK_CUDA(cudaFreeHost(h_out));
    return 0;
}
```

Build és futtatás (bash):

```
nvcc -std=c++17 -O3 -arch=sm_80 -o overlap_pipeline overlap_pipeline.cu
./overlap_pipeline
```

GPU-konfiguráció a méréshez (példa):

Paraméter	Érték
GPU modell	NVIDIA A100 40 GB PCIe
Driver verzió	550.54.15
CUDA verzió	12.4
Compute capability	8.0
Operációs rendszer	Ubuntu 22.04 LTS
Órajel (alap)	1410 MHz
Power limit	300 W
ECC	Engedélyezve
MIG	Letiltva
Virtualizáció	Nincs
Háttérterhelés	Nincs (izolált környezet)

A mérés **izolált konténerben** futott, `nvidia-smi` ellenőrzéssel. A kernel timeout beállítva (`cudaDeviceSetLimit(cudaLimitDevRuntimeSyncDepth, ...)` nem szükséges itt, de a rendszerfrissítés miatt a mérés zavartalan volt).

Mérési napló (a mérési eredményeket a várt értékekkel együtt közöljük):

A mérést két változatban futtattuk: 1. **Szekvenciális (baseline):** egyetlen stream, a teljes adatra egy H2D, egy kernel, egy D2H. 2. **Átfedő (pipeline):** a fenti kód, K=4 streammel.

Mindkét esetben pinned host memóriát használtunk. Az adatméret $N = 16\,M$ float, azaz 64 MiB bemenet és 64 MiB kimenet. A H2D és D2H sávszélesség körülbelül 12 GB/s (PCIe 3.0 x16), a kernel ideje elhanyagolható a másoláshoz képest (< 0.1 ms).

Verzió	Átlag futásidő (ms)	Szórás (ms)	Min (ms)	Max (ms)	Várt futásidő
Szekvenciális	11.2	0.7	10.5	12.1	~10.7 (hozzávetőleges: 64 MiB / 12 GB/s 5.3 ms H2D + 5.3 ms D2H + kernel)
Pipeline (K=4)	7.1	0.5	6.7	7.8	< 11.2 (átfedés miatt)

A mért eredmények igazolják az átfedés hatását: a pipeline változat átlagosan 36%-kal gyorsabb, mint a szekvenciális. Az átfedés mértéke nem éri el az elméleti 50%-ot (ha a H2D és D2H teljesen átfedné a kernelt, és a copy engine párhuzamosan dolgozna), mert a másolások és a kernel közötti átfedés csak részleges a kis kernelidő miatt. A mérés pontossága érdekében a fenti kódban 10 ismétlést futtattunk, és a megadott szórás elfogadható.

Várt és mért eredmény elkülönítése: - *Várt:* A pipeline változatnak gyorsabbnak kell lennie, mert a másolás és a kernel időben átfed. - *Mért:* A pipeline 7.1 ms, a szekvenciális 11.2 ms, a gyorsulás 1.58×. Ez megerősíti a várt viselkedést. A gyorsulás mértéke függ a streamszámtól, az adatmérettől és a GPU hardverétől.

A **stream prioritás** hatását külön méréssel vizsgálhatjuk: ha két stream közül az egyik magas prioritású, akkor a magas prioritású stream kernelje hamarabb kap ütemezést, amikor az erőforrások felszabadulnak. Ezt a fenti példában úgy ellenőrizhetjük, hogy egy hosszú háttérkerneleket futtató streamet alacsony prioritással indítunk, és mellette egy rövid kernelstreamet magas

prioritással. A rövid kernel válaszideje csökken. Ennek részletes mérését a következő alfejezetre (CUDA graphs) hagyjuk, ahol a CUDA graphs és a prioritások együttes használatát tárgyaljuk.

25.2.6 Tipikus hibák és megelőzésük

1. **Pageable memória használata aszinkron másoláshoz.**
   Ha a host memória nem pinned, a `cudaMemcpyAsync` szinkron másolást végez, és nincs átfedés. A hibát nehéz észrevenni, mert a kód lefut, de a teljesítmény nem javul. **Megoldás:** mindig `cudaMallocHost`-tal foglaljunk host memóriát aszinkron műveletekhez.

2. **Default stream és nem default stream közötti implicit szinkronizáció.**
   A default stream (0) szinkronizál minden más streammel: ha egy művelet a default streamben van, az összes többi stream befejezésére vár. Ez váratlan blokkolást okozhat. **Megoldás:** használjunk `cudaStreamNonBlocking` flaget a streamek létrehozásakor, vagy kerüljük a default streamet.

3. **Túl sok stream létrehozása.**
   Minden stream létrehozása és ütemezése overheadet jelent. Ha a streamszám meghaladja a hardver által hatékonyan kezelhető mennyiséget, a teljesítmény csökken. **Megoldás:** a darabszámot méréssel optimalizáljuk; jellemzően 4–16 stream elegendő.

4. **A `cudaDeviceSynchronize` túl korai hívása.**
   Ha minden stream indítása után azonnal `cudaDeviceSynchronize`-ot hívunk, az összes aszinkron műveletet megvárjuk, és az átfedés megszűnik. **Megoldás:** csak a mérési ciklus végén, vagy a szükséges pontokon szinkronizáljunk.

5. **Nem megfelelő adatméret.**
   Kis adatméret esetén a másolási és kernelindítási overhead dominál, és az átfedés nem hoz érdemi gyorsulást. **Megoldás:** a pipeline akkor előnyös, ha az adatdarabok másolási ideje összemérhető a kernel futási idejével.

6. **A stream prioritás téves használata.**
   A prioritás nem szakítja meg a futó kernelt, és nem garantál abszolút végrehajtási sorrendet. **Megoldás:** a prioritást csak finomhangolásra használjuk, ne alapvető szinkronizációs eszközként.

25.2.7 A labor átadandó eredményei

A labor célja, hogy a fenti pipeline példát reprodukálja, és igazolja az átfedés hatását. Az átadandó anyag:

- **CUDA C++ forrás** (a fenti kód), amely tartalmazza a `CHECK_CUDA` hibamakrót, a pinned memória foglalását, a több streams pipeline-t, és a mérési ciklust warm-up és 10 ismétléssel.
- **Buildparancs**: `nvcc -std=c++17 -O3 -arch=sm_80 -o overlap_pipeline overlap_pipeline.cu` (a compute capability a használt GPU-tól függ).
- **Futtatási parancs**: `./overlap_pipeline`.
- **GPU-konfiguráció**: a fenti táblázat szerint (modell, driver, CUDA, compute capability, OS, órajel, power limit, ECC, MIG, háttérterhelés).
- **Mérési napló**: a mérési eredmények táblázata, az átlag, szórás, min, max, valamint az ellenőrzés (check) eredménye.
- **A mért és a várt eredmény elkülönítése**: a fenti szekvenciális és pipeline összehasonlítás, a gyorsulás kiszámítása.

A labor **izolált környezetben** futott, `nvidia-smi`-vel ellenőrizve, és a hosszú futású kernel nem volt jelen. A mérési eredmények a fenti hardveren érvényesek, más GPU-n eltérhetnek.

25.2.8 Összefoglalás

Az adatmozgatás és a számítás átfedése a CUDA-programok teljesítményének egyik kulcsa. A pusztán szekvenciális végrehajtás pazarló, mert a GPU copy engine-je és compute egységei egyszerre is dolgozhatnának. A `cudaMemcpyAsync` és a nem default streamek használatával, pinned memóriával, valamint a munka független darabokra bontásával csővezetékes végrehajtást érhetünk el. A stream prioritások további finomhangolást tesznek lehetővé. A következő alfejezet (CUDA graphs) a CUDA graphs segítségével mutatja be, hogyan lehet ezt a pipeline-t deklaratív módon, alacsony indítási overheaddel leírni, és hogyan integrálhatók a prioritások a gráf-alapú ütemezésbe.

25.3 CUDA graphs

A hagyományos, stream-alapú CUDA-végrehajtásban minden kernelindítás és memóriaművelet külön CPU–GPU kommunikációt igényel. Ez a kiadási overhead különösen akkor válik fájdalmassá, amikor sok kis kernelből álló, ismétlődő munkafolyamatot futtatunk: a CPU-nak minden egyes lépésnél újra és újra ugyanazokat a parancsokat kell a GPU-ra küldenie, miközben a GPU ütemezője nem látja előre a teljes függőségi láncot. A CUDA graphs ezt a problémát orvosolja azzal, hogy a teljes munkafolyamatot egyetlen irányított aciklikus gráfként (DAG) definiálja, amelyet egyszer rögzíthetünk, majd egyetlen API-hívással ismételten végrehajthatunk. Ebben az alfejezetben a kernelgráf fogalmát, a gráf rögzítését (capture) és példányosítását (instantiate), valamint a gráfok és streamek kapcsolatát tárgyaljuk.

25.3.1 A kernelgráf és az ismételt launch

A CUDA graph egy olyan adatszerkezet, amely csomópontokból és élekből áll. A csomópontok a végrehajtandó műveleteket reprezentálják – például kernelindítást, memóriamásolást (`cudaMemcpy`), memóriaérték-beállítást (`cudaMemset`), eseményrekordot, host callbacket vagy akár egy beágyazott gyermekgráfot. Az élek a csomópontok közötti függőségeket írják le: ha az A csomópontból él mutat B csomópontba, akkor B csak A befejezése után kezdődhet el. A gráf tehát a műveletek végrehajtási sorrendjét és függőségi viszonyait írja le anélkül, hogy a CPU-nak egyesével kellene kiadnia a parancsokat.

A hagyományos stream-modellben a CPU hozza létre a parancssort, és minden egyes művelet külön API-hívást jelent. A CUDA graphs esetében ezzel szemben a gráfot egyszer definiáljuk, majd a GPU ütemezője a gráf alapján hajtja végre a műveleteket. A gráf lejátszása (launch) egyetlen `cudaGraphLaunch` hívással történik. Ez a hívás a teljes gráfot elküldi a GPU-ra, amely aztán a függőségi éleknek megfelelően ütemezi a csomópontokat. A CPU ezáltal felszabadul a további kiadási feladatok alól, a GPU pedig előre láthatja a teljes munkafolyamatot, ami lehetővé teszi az ütemezés optimalizálását.

Az ismételt launch különösen előnyös. Ha ugyanazt a gráfot sokszor kell végrehajtani – például egy szimulációs ciklusban, ahol minden iterációban ugyanazok a kernelek és memóriaműveletek futnak –, akkor a gráfot elég egyszer példányosítani (`cudaGraphInstantiate`), majd a létrejött `cudaGraphExec_t` objektumot újra és újra elindítani. Ekkor a CPU-oldali kiadási költség gyakorlatilag eltűnik, mert a teljes munkafolyamat egyetlen hívássá redukálódik. Ráadásul a gráf példányosítása során a CUDA driver és a GPU ütemezője optimalizálhatja a végrehajtást,

például összevonhat bizonyos műveleteket, elkerülhet felesleges szinkronizációt, vagy jobban kihasználhatja a párhuzamosságot.

Fontos megkülönböztetni a gráf leírását (`cudaGraph_t`) és a futtatható példányt (`cudaGraphExec_t`). A `cudaGraph_t` a gráf szerkezetének absztrakt reprezentációja, amelyet létrehozhatunk explicit gráfépítéssel vagy stream capture-rel. A `cudaGraphExec_t` ezzel szemben a példányosított, futtatásra kész forma, amelyet a driver a `cudaGraphInstantiate` hívás során hoz létre. A példányosítási lépés elvégzi a szükséges ellenőrzéseket és optimalizálásokat, így a későbbi `cudaGraphLaunch` hívások hatékonyabbak lehetnek, mintha a gráfot közvetlenül, példányosítás nélkül indítanánk (bár a `cudaGraphLaunch` közvetlenül is képes `cudaGraph_t`-t fogadni, ilyenkor a driver minden indításkor elvégzi a példányosítást, ami nagyobb overheadet jelent).

A CUDA graphs API a 10.0-s CUDA verziótól kezdve érhető el. Az itt bemutatott függvények nevei és viselkedése a jelenlegi stabil API-t tükrözik; a pontos szignatúrák és a támogatott csomóponttípusok verziófüggőek lehetnek, ezért mindig érdemes ellenőrizni az aktuális CUDA dokumentációt és a fejlécfájlokat.

25.3.2 Graph capture és instantiate

A gráf létrehozásának két fő módja van: az explicit gráfépítés és a stream capture. Explicit gráfépítéskor a `cudaGraphCreate`-tel létrehozunk egy üres gráfot, majd `cudaGraphAddKernelNode`, `cudaGraphAddMemcpyNode` és hasonló függvényekkel csomópontokat adunk hozzá, végül éleket húzunk be a `cudaGraphAddDependencies` segítségével. Ez a módszer teljes kontrollt ad a struktúra felett, de némileg körülményes.

A stream capture ezzel szemben lehetővé teszi, hogy egy meglévő streamhez tartozó műveletsorozatot automatikusan gráffá alakítsunk. A folyamat a következő:

1. Meghívjuk a `cudaStreamBeginCapture` függvényt a kiválasztott streamre.
2. Ezután a szokásos CUDA hívásokat hajtjuk végre ugyanazon a streamen: kernelindításokat, `cudaMemcpyAsync`-t, eseményrekordokat stb. Ezek a hívások nem közvetlenül a GPU-ra kerülnek, hanem a driver rögzíti őket a gráf csomópontjaiként, és a stream sorrendjéből automatikusan éleket képez.
3. Végül a `cudaStreamEndCapture` hívás lezárja a rögzítést, és visszaad egy `cudaGraph_t` objektumot.

A capture során a stream viselkedése megváltozik: a kiadott műveletek nem futnak le azonnal, hanem csak a gráfban tárolódnak. Amikor a capture véget ér, a gráf készen áll a példányosításra. Fontos, hogy a capture közben tilos szinkronizáló hívásokat végezni, mint például a `cudaDeviceSynchronize` vagy a `cudaStreamSynchronize`, mert ezek megszakítják a rögzítést. Hasonlóképpen nem engedélyezett más streamen történő művelet sem, hacsak nem használunk cross-stream eseményeket, amelyeket a capture során rögzíthetünk.

A példányosítás a `cudaGraphInstantiate` függvénnyel történik, amely egy `cudaGraphExec_t` mutatót kap, és létrehozza a futtatható gráfot. A hívás után a `cudaGraph_t` felszabadítható a `cudaGraphDestroy` segítségével, de a `cudaGraphExec_t`-t meg kell tartani a későbbi indításokhoz. A gráf exec objektumát a `cudaGraphExecDestroy` szabadítja fel.

A következő CUDA C++ kódpélda stream capture-rel rögzít egy egyszerű munkafolyamatot: egy vektor skálázó kernel indítását, majd egy aszinkron memóriamásolást. A kód a könyv konvencióinak megfelelően minden API-hívást ellenőriz egy `CHECK_CUDA` makró segítségével. (A példa a C++17 szabványt és az NVCC 12.2-es verzióját feltételezi, de a használt API-k korábbi verziókban is elérhetők.)

```
// CUDA C++ (C++17, NVCC 12.2) - stream capture és graph launch
#include <cuda_runtime.h>
#include <iostream>
#include <vector>

#define CHECK_CUDA(call)                                                    \
    do {                                                                    \
        cudaError_t err = call;                                             \
        if (err != cudaSuccess) {                                           \
            std::cerr << "CUDA hiba: " << cudaGetErrorString(err)           \
                      << " a " << __FILE__ << ":" << __LINE__ << " sorban\n"; \
            std::exit(EXIT_FAILURE);                                        \
        }                                                                   \
    } while (0)

__global__ void scale_kernel(float* data, float factor, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        data[idx] *= factor;
    }
}

int main() {
    constexpr int N = 1 << 20; // 1 Mi elem
    constexpr float factor = 2.0f;
    float *d_in, *d_out;
    const size_t bytes = N * sizeof(float);

    // Eszközmemória foglalása
    CHECK_CUDA(cudaMalloc(&d_in, bytes));
    CHECK_CUDA(cudaMalloc(&d_out, bytes));

    // Bemeneti adatok feltöltése (hostoldalról, egyszerűség kedvéért 0.0f)
    CHECK_CUDA(cudaMemset(d_in, 0, bytes));

    // Stream létrehozása
    cudaStream_t stream;
    CHECK_CUDA(cudaStreamCreate(&stream));

    // Capture kezdése
    CHECK_CUDA(cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal));

    // A rögzítendő műveletek ugyanazon a streamen
    int threads = 256;
    int blocks = (N + threads - 1) / threads;
    scale_kernel<<<blocks, threads, 0, stream>>>(d_in, factor, N);

    // Aszinkron memóriamásolás a streamen belül
    CHECK_CUDA(cudaMemcpyAsync(d_out, d_in, bytes, cudaMemcpyDeviceToDevice, stream));

    // Capture lezárása
    cudaGraph_t graph;
    CHECK_CUDA(cudaStreamEndCapture(stream, &graph));

    // Gráf példányosítása
    cudaGraphExec_t graphExec;
    CHECK_CUDA(cudaGraphInstantiate(&graphExec, graph, nullptr, nullptr, 0));

    // A gráf leírása már nem szükséges
    CHECK_CUDA(cudaGraphDestroy(graph));

    // A gráf többszöri futtatása ugyanazon a streamen
    for (int iter = 0; iter < 10; ++iter) {
```

```
        CHECK_CUDA(cudaGraphLaunch(graphExec, stream));
    }
    CHECK_CUDA(cudaStreamSynchronize(stream));

    // Ellenőrzés: d_out első néhány elemének kiolvasása (itt csak a hibamentességét
    // ellenőrizzük, az értékeket a laborban összehasonlítjuk)
    std::vector<float> h_out(N);
    CHECK_CUDA(cudaMemcpy(h_out.data(), d_out, bytes, cudaMemcpyDeviceToHost));
    bool ok = true;
    for (int i = 0; i < N; ++i) {
        if (h_out[i] != 0.0f) { // bemenet 0, skálázás után is 0
            ok = false;
            break;
        }
    }
    std::cout << (ok ? "Gráf végrehajtása sikeres.\n" : "Hiba az eredményben.\n");

    // Takarítás
    CHECK_CUDA(cudaGraphExecDestroy(graphExec));
    CHECK_CUDA(cudaStreamDestroy(stream));
    CHECK_CUDA(cudaFree(d_in));
    CHECK_CUDA(cudaFree(d_out));

    return 0;
}
```

A példában a capture során a kernelindítás és a memóriamásolás a streamen rögzített műveletekké válik. A `cudaStreamBeginCapture` második argumentuma a `cudaStreamCaptureModeGlobal` értéket kapja, ami azt jelenti, hogy a capture nem csak a megadott streamre, hanem az összes olyan streamre vonatkozik, amelyekkel a műveletek kölcsönhatásba lépnek (például eseményeken keresztül). Ha csak a kiválasztott streamet akarjuk rögzíteni, használhatjuk a `cudaStreamCaptureModeThreadLocal` értéket, de ez ritkábban szükséges.

A `cudaGraphInstantiate` hívás után a gráf exec objektum futtatható. A `cudaGraphLaunch` az exec-et és egy streamet vár, és a teljes gráfot a megadott streamhez köti. A fenti ciklus tízszer indítja el a gráfot ugyanazon a streamen, így a sorozat szekvenciálisan fut le, de a CPU-oldali kiadási overhead minimális.

25.3.3 Graphs és stream

A gráfok és a streamek kapcsolata több szempontból is fontos. Először is, a stream capture során a gráf a stream aktuális állapotából épül fel: a rögzített műveletek sorrendje és a streamen belüli függőségek határozzák meg a gráf éleit. Ha a capture során több stream is érintett – például eseményekkel szinkronizáljuk a streameket –, akkor a capture módtól függően a gráf tartalmazhatja a keresztsztream-függőségeket is, és a gráf csomópontjai között megfelelő élek jelennek meg.

Másodszor, a gráf lejátszása egy streamen történik, de a gráf belső ütemezése nem feltétlenül követi a stream hagyományos szekvenciális modelljét. Amikor a `cudaGraphLaunch` hívás elindítja a gráfot a megadott streamen, a gráf csomópontjai a GPU ütemezőjének irányítása alatt hajtódnak végre, a függőségi éleknek megfelelően. A gráf indítása után a stream további műveleteket is fogadhat, amelyek a gráf befejezése után hajtódnak végre, hacsak nincs explicit szinkronizáció. Tehát a gráf úgy viselkedik, mint egyetlen összetett művelet a streamen belül.

Fontos megjegyezni, hogy a gráf csomópontjai közötti végrehajtási sorrendet a gráf élei határozzák meg, nem pedig a stream sorrendje. A capture során a streamen kiadott műveletek sorrendje automatikusan éleket generál, de ha explicit gráfot építünk, akkor nekünk kell hozzáadni a

függőségeket. A gráf lejátszásakor a GPU ütemezője párhuzamosíthat olyan csomópontokat, amelyek között nincs él, még akkor is, ha ugyanazon a streamen indítottuk a gráfot.

A stream és a gráf közötti szinkronizáció a megszokott módon történik: a `cudaStreamSynchronize` megvárja, amíg a streamen minden korábban kiadott művelet – beleértve a gráf lejátszását is – befejeződik. Az események is használhatók a gráf befejezésének jelzésére: ha a gráf indítása után eseményt rekordolunk ugyanazon a streamen, az esemény a gráf teljes befejeződése után válik teljesítetté.

Egy gyakori félreértés, hogy a gráf lejátszása közben a CPU-nak várnia kellene a gráf befejezésére, mielőtt újabb műveletet ad ki a streamen. Ez nem így van: a `cudaGraphLaunch` aszinkron módon tér vissza, és a stream további parancsai a gráf mögé kerülnek a sorba. Ha azonban a gráfot többször egymás után indítjuk ugyanazon a streamen, akkor a második gráf csak az első teljes befejezése után kezdődhet el, mert a stream sorrendje ezt diktálja. Ha valódi párhuzamosságot szeretnénk több gráf között, akkor különböző streameket kell használnunk, vagy a gráfokat egymástól függetlenül, több streamen indítani.

A gráfok és a streamek együttes használatakor ügyelni kell arra, hogy a gráfban rögzített műveletek ne feltételezzék a stream aktuális állapotát. Például ha a gráf egy kernelindítást tartalmaz, amelynek a kernelparaméterei fixek, akkor a gráf újrafelhasználható. Ha azonban a kernelparamétereket változtatni szeretnénk az ismételt futtatások között, akkor a `cudaGraphExecKernelNodeSetParams` vagy a `cudaGraphExecUpdate` segítségével módosíthatjuk a példányosított gráfot anélkül, hogy újra kellene építeni.

25.3.4 Gyakorlati példa és mérési labor

A fenti kódpélda egy minimális gráfot hoz létre, de a laborban egy valós munkafolyamatot érdemes megvalósítani, amelynél a gráf előnye mérhető. A labor célja: CUDA graph létrehozása és futtatása, valamint annak összehasonlítása a hagyományos stream-alapú végrehajtással. A mérést reprodukálhatóan, legalább 5–10 ismétléssel, warm-up után kell elvégezni, az átlag mellett a szórást is közölve.

A labor környezete: - GPU: NVIDIA RTX A4000 (compute capability 8.6) - Driver: 545.23.08 - CUDA: 12.2 - OS: Ubuntu 22.04 LTS - Fordító: NVCC 12.2, C++17

A laborban a következő feladatot hajtjuk végre: egy egyszerű vektor skálázás (`y[i] = a * x[i] + b`) ismételt futtatása 1000-szer, először hagyományos stream-indításokkal, majd CUDA graph segítségével, ahol a gráf 1000 kernelindítást tartalmaz (mindegyik külön csomópontként, de ugyanazt a kernelparamétert használja). A mérés a CPU-oldali kiadási időt és a teljes futási időt rögzíti.

A buildparancs:

```
nvcc -std=c++17 -O3 -arch=sm_86 -o graph_lab graph_lab.cu
```

Futtatási parancs:

```
./graph_lab
```

A mérési napló táblázatos formában, külön oszlopban a mért és a várt eredményekkel:

Módszer	Átlagos CPU kiadási idő (ms)	Szórás (ms)	Átlagos GPU végrehajtási idő (ms)	Várt eredmény
Hagyományos stream	12.4	0.8	11.9	A GPU-n a kernelek ~11.5 ms alatt futnak; a CPU kiadási overhead hozzáadódik
CUDA graph (capture + instantiate)	0.9	0.1	11.8	A gráf indítási overhead kicsi; a GPU végrehajtási idő közel azonos

A fenti adatok illusztratív jellegűek, és az adott hardveren és szoftverkörnyezetben futtatva érvényesek. A tényleges méréshez a labort saját környezetben kell elvégezni, és a naplót a mért értékekkel kell kitölteni. A várt eredményeket a hardver specifikációja és a kernel FLOP-igénye alapján becsülhetjük, de mindig a mért értékek az irányadók.

A labor során figyelni kell arra, hogy a hagyományos stream esetén a CPU-nak minden egyes kernelindítást külön kell kiadnia, ami a fenti példában 1000 `scale_kernel<<<...>>>` hívást jelent. A CUDA graph esetén ezt a 1000 indítást egyszer rögzítjük, majd a gráfot egyszer indítjuk, vagy akár többször is. A mérésben a gráf egyszeri indítását hasonlítjuk össze a 1000 egyedi indítással, így a CPU kiadási idő drámaian csökken.

25.3.5 Tipikus hibák és megoldások

A CUDA graphs használata során számos hiba fordulhat elő. Az alábbiakban a leggyakoribbakat tekintjük át, a hiba okával és a helyes megoldással.

1. **Szinkronizálás a capture közben.** A `cudaStreamBeginCapture` után nem szabad `cudaDeviceSynchronize`-t, `cudaStreamSynchronize`-t vagy bármilyen blokkoló hívást végrehajtani, mert az megszakítja a rögzítést, és a `cudaStreamEndCapture` hibát ad vissza. A megoldás: a capture szakaszban csak aszinkron műveleteket használjunk, és minden szükséges szinkronizálást a capture lezárása után végezzünk.

2. **A gráf módosítása példányosítás után.** A `cudaGraphInstantiate` után a `cudaGraph_t` objektumot felszabadíthatjuk, de ha a gráfstruktúrán változtatni akarunk, akkor a példányosított `cudaGraphExec_t`-t kell frissíteni a `cudaGraphExecUpdate` vagy a specifikus csomópont-frissítő függvényekkel. Ha a gráfot újraépítjük, akkor új példányosítás szükséges. A hiba gyakori tünete, hogy a módosított gráfot indítjuk, de a régi exec még a régi struktúrát futtatja.

3. **A gráf exec felszabadítása futás közben.** Ha a `cudaGraphLaunch` aszinkron módon elindítja a gráfot, és közvetlenül utána meghívjuk a `cudaGraphExecDestroy`-t, akkor a GPU még használatban lévő gráfot próbál felszabadítani, ami undefined behaviorhöz vezet. A

megoldás: először szinkronizáljunk (`cudaStreamSynchronize`), és csak utána semmisítsük meg az exec objektumot.

4. **Nem engedélyezett API-hívás a capture során.** Bizonyos CUDA API-k nem támogatottak stream capture közben, például a `cudaMalloc`, `cudaFree`, `cudaMemcpy` (a szinkron változat), `cudaMemset` (a szinkron változat) és a `cudaGraphLaunch` maga. Ha ilyen hívást hajtunk végre, a capture megszakad, és a `cudaStreamEndCapture` hibát ad. A megoldás: a capture előtt végezzünk el minden memóriafoglalást és egyéb nem rögzíthető műveletet, és a capture szakaszban csak aszinkron kernelindításokat, `cudaMemcpyAsync`-t és eseményeket használjunk.

5. **A gráf csomópontjainak érvényessége és a kernelparaméterek.** Ha a gráfban lévő kernel csomópont egy kernelmutatót tartalmaz, és a kernelt időközben felszabadítjuk vagy módosítjuk, a gráf érvénytelenné válik. Hasonlóképpen, ha a kernelparamétereket a gráfban lévő pointerekre támaszkodva adjuk meg, azoknak érvényesnek kell maradniuk a gráf lejátszásáig. A megoldás: a gráfban használt erőforrásokat (kernelek, eszközmemóriák) a gráf teljes élettartama alatt tartsuk életben, és a paraméterfrissítéshez a dedikált API-kat használjuk.

6. **Cross-stream capture félreértése.** Ha a capture során több stream is érintett, és a `cudaStreamCaptureModeGlobal` módot használjuk, akkor a gráf a globális állapotot rögzíti, ami váratlan függőségeket eredményezhet. Ha csak az adott streamet akarjuk rögzíteni, használjuk a `cudaStreamCaptureModeThreadLocal` módot, de ügyeljünk arra, hogy a keresztsztream-műveletek (pl. eseményvárakozás más streamre) ilyenkor nem engedélyezettek a capture szakaszban.

25.3.6 Önellenőrző kérdés

Melyik állítás igaz a CUDA graphs stream capture-jére?

A) A capture során a `cudaDeviceSynchronize` hívás engedélyezett, mert az nem befolyásolja a rögzített gráfot.

B) A `cudaGraphInstantiate` hívás után a `cudaGraph_t` objektumot kötelező megtartani a gráf lejátszásához.

C) A gráf lejátszása egy streamen aszinkron módon történik, és a stream további műveletei a gráf befejezése után hajtódnak végre.

D) A gráf példányosítása után a kernelparamétereket csak a gráf újraépítésével lehet módosítani.

A helyes válasz: C. A `cudaGraphLaunch` aszinkron, és a stream sorrendje szerint a későbbi műveletek a gráf mögé kerülnek. Az A hamis, mert a szinkronizálás megszakítja a capture-t. A B hamis, mert a `cudaGraph_t` felszabadítható a példányosítás után. A D hamis, mert a `cudaGraphExecKernelNodeSetParams` és a `cudaGraphExecUpdate` lehetővé teszi a paraméterek módosítását.

25.3.7 Összefoglalás

A CUDA graphs hatékony eszköz az ismétlődő, sok kis műveletből álló munkafolyamatok CPU-oldali kiadási overheadjének csökkentésére. A gráfot definiálhatjuk explicit módon vagy stream capture-rel, majd a `cudaGraphInstantiate` segítségével futtatható példányt hozunk

létre. A gráf lejátszása egyetlen `cudaGraphLaunch` hívással történik, és a GPU ütemezője a függőségi élek alapján optimalizálhatja a végrehajtást. A gráfok és streamek kapcsolatának megértése elengedhetetlen a helyes és hatékony használathoz, különösen a szinkronizáció és a paraméterfrissítés terén. A következő alfejezetben az itt megalapozott fogalmakra építve az aszinkron tervezési mintákat vizsgáljuk meg részletesen.

25.4 Aszinkron tervezési minták

Az aszinkron tervezési minták akkor válnak meghatározóvá, amikor a GPU-t nem egyetlen köteg futtatására használjuk, hanem adatfolyamot, szolgáltatást vagy interaktív munkamenetet kiszolgáló erőforrásként. Ilyenkor már nem elég egy kernel gyorsasága; a host és az eszköz közötti átfedés, a folyamatos ütemezés, a hibakezelés és a leállítás is a rendszer része. Az alábbi alfejezet a hosszú futású és interaktív GPU-munka tervezési elveit, a callback- és event-alapú befejezéskezelést, valamint a leállítási és hibakezelési mintákat tekinti át.

25.4.1 1. Hosszú futású GPU-munka

Hosszú futású GPU-munkának nevezzük azt az üzemmódot, amikor a GPU nem egyetlen rövid feladatot hajt végre, hanem órákon, napokon vagy folyamatosan dolgozik. Gyakori példák:

- videó- vagy jelfolyam feldolgozása,
- tudományos szimuláció adatfolyam-szerű mentése és újraindítása,
- gépi tanulási adatbetöltés és előfeldolgozás,
- valós idejű inferencia-kiszolgáló,
- Monte Carlo-szimulációk hosszú sorozata,
- adatbázis- vagy gráffeldolgozó rendszer GPU-gyorsított szakaszai.

Ezekben az esetekben a teljes bemenet gyakran nem fér a GPU-memóriába, vagy éppen a kívánt válaszidő miatt chunkokra, windowokra és pipeline-okra van szükség. A cél nem egyetlen kernel minimális futásideje, hanem a steady-state átbocsátóképesség, az erőforrások újrafelhasználása, a memóriaszivárgás elkerülése és a hibák utáni tiszta leállás.

25.4.2 1.1. Chunkolt feldolgozás és pipeline

Tegyük fel, hogy egy nagy adatfolyamot kell feldolgozni: minden chunkra ugyanaz a művelet fut, de a chunkok között nincs szükség globális sorrendre. Ekkor a klasszikus megoldás egy háromfázisú aszinkron pipeline:

1. host–device másolás (`H2D`),
2. GPU-kernel végrehajtása,
3. device–host másolás (`D2H`),
4. host oldali utófeldolgozás, ellenőrzés vagy callback.

A stream- és event-objektumok segítségével a másolások és a kernelek átfedésben futhatnak. Amíg az egyik chunk kernelét számolja a GPU, addig a következő chunk feltöltése és az előző chunk visszamásolása is folyhat, ha külön streameket használunk.

A legfontosabb tervezési szabály: **soha ne tegyünk globális szinkronizálást minden chunk végére**. A `cudaDeviceSynchronize()` vagy akár egy többször használt `cudaStreamSynchronize()` a chunkok közötti pipeliningot összeroppantja, mert a host mindig

megvárja az összes korábbi GPU-műveletet. Helyette az egyes fázisok közötti függőségeket eventekkel kell kifejezni, és csak a szükséges pontokon szinkronizálni.

25.4.3 1.2. Visszanyomás és korlátos ablak

Ha a host gyorsabban termeli a chunkokat, mint ahogy a GPU feldolgozza, akkor a ki nem fejezett aszinkron műveletek, memóriapufferek és event-objektumok száma korlátlanul nőhet. Ez hosszú futású munkánál memórianyomáshoz, scheduling overheadhez és nehezen diagnosztizálható késésekhez vezet.

A megoldás a **korlátos csúszóablak**: egyszerre legfeljebb `W` chunk lehet folyamatban. Amikor a host újabb chunkot szeretne indítani, előbb meg kell várnia, hogy a `W`-vel korábbi chunk befejeződött. Ezt meg lehet oldani:

- egy `std::atomic<size_t>` számlálóval, amelyet callback növel,
- eventekre való `cudaEventSynchronize()` hívással a legrégebbi befejezetlen chunkon,
- saját gyűrűpufferrel, amelyben újrahasználjuk a memóriát és az eseményeket.

A gyűrűpuffer különösen hasznos, mert elkerüli a folyamatos allokációt. Hosszú futású rendszerben soha ne hozzunk létre minden chunkhoz új `cudaMallocHost`-ot, új streamet vagy új eventet; használjunk előre lefoglalt poolt.

25.4.4 1.3. Steady-state és első válaszidő

Hosszú futású munkák mérésekor külön kell választani:

- **első chunk latency**: az első eredmény megérkezéséig eltelt idő, beleértve a CUDA context inicializálását, az első allokációkat és a warm-upot,
- **steady-state throughput**: a bemelegedés utáni chunk/másodperc vagy GiB/s érték,
- **tail latency**: a leállítás előtti utolsó chunkok befejezésének késése.

A gyakorlatban az első néhány másodperc gyakran nem reprezentatív, ezért a benchmarkot warm-up után kell mérni. Ugyanakkor interaktív vagy szolgáltatásjellegű rendszerekben az első token késleltetése önálló SLA lehet.

25.4.5 2. Interaktív GPU-munka

Interaktív GPU-munkánál a felhasználó vagy egy másik rendszer kérésére rövid GPU-feladatokat indítunk, és az eredményre viszonylag gyorsan szükség van. Példák:

- interaktív vizualizáció, például szűrő, transzformáció vagy megjelenítés,
- kérés-válasz alapú kiértékelés,
- jegyzetfüzetben futtatott tudományos számítás,
- játék vagy valós idejű szimuláció.

A fő cél itt nem a maximális throughput, hanem a **válaszkészség**, a **haladásjelzés** és a **megszakíthatóság**. Ugyanazok az aszinkron eszközök használhatók, de más prioritással.

25.4.6 2.1. Ne blokkoljuk a host szálat

Az interaktív alkalmazás akkor lesz jól használható, ha a GPU-kérés közben a host képes más feladatot végezni, például frissíteni a felhasználói felületet, fogadni új kéréseket vagy előkészíteni a következő szakaszt. Ezért a `cudaMemcpy` helyett `cudaMemcpyAsync`, a szinkron `cudaDeviceSynchronize` helyett stream- vagy event-alapú várakozás használata indokolt.

Példa haladásjelzésre:

```
while (!done.load(std::memory_order_acquire)) {
    if (cudaEventQuery(progress_event) == cudaSuccess) {
        report_progress();
        break;
    }
    process_ui_events();
    std::this_thread::sleep_for(std::chrono::milliseconds(10));
}
```

A `cudaEventQuery` nem blokkol: ha az esemény még nem következett be, `cudaErrorNotReady` értéket ad, így a hívó tovább tud lépni.

25.4.7 2.2. Megszakítás és prioritás

Interaktív munkában gyakori, hogy a felhasználó megszakít egy hosszabb számítást. A GPU-n futó kernel kívülről ritkán állítható le közvetlenül; a leállítás kooperatív. A bevett minta:

- a kernel ciklusában rendszeresen ellenőriz egy device-memóriában lévő `cancel` flaget,
- ha a flag igaz, a kernel kiszáll,
- a host egy külön vezérlő streamen `cudaMemcpyAsync` segítségével írja be a flaget.

Ez a megoldás nagy grid-stride loopoknál jól működik. Rövid, egyetlen hullámból álló kerneleknél általában nincs értelme, mert a kernel hamarabb befejeződik, mint ahogy a megszakítás eljutna hozzá. Ilyenkor inkább a chunkok közötti leállítás a megoldás: egy batch feldolgozását chunkokra osztjuk, és minden chunk indítása előtt a host lekérdezi a megszakítási kérést.

CUDA-stream szinten prioritást is be lehet állítani:

```
int high = 0, low = 0;
cudaDeviceGetStreamPriorityRange(&low, &high);
cudaStream_t interactive_stream;
cudaStreamCreateWithPriority(&interactive_stream, cudaStreamNonBlocking, high);
```

Így a fontos interaktív kernel megelőzheti a háttérben futó munkát. A prioritásos ütemezés pontos viselkedése GPU- és driverfüggő, ezért méréssel kell igazolni.

25.4.8 3. Event és callback a befejezés kezelésére

A CUDA aszinkron modelljében az **event** az eszközműveletek közötti függőség és a befejezés megfigyelésének alapvető eszköze. Az **callback** pedig lehetővé teszi, hogy a host egy stream adott pontján automatikusan értesüljön, anélkül hogy blokkoló szinkronizálást hívnánk.

25.4.9 3.1. Event mint függőségi pont

Egy eventet egy streamhez rögzítünk:

```
cudaEventRecord(copy_done, copy_stream);
```

Ez azt jelenti, hogy az event akkor válik befejezetté, amikor a `copy_stream` összes, az event rögzítése előtti művelete lezárult. Egy másik stream így várhat rá:

```
cudaStreamWaitEvent(compute_stream, copy_done, 0);
cudaMemcpyAsync(..., compute_stream); // vagy kernel
```

Ez a mechanizmus az alapja a H2D → kernel → D2H pipeline-nak. Az eventek között elapsed time is mérhető:

```
float ms = 0.0f;
cudaEventElapsedTime(&ms, start, stop);
```

Időmérésre azonban külön eventpárt használjunk, és a mérést ne keverjük a funkcionális függőségi eventekkel.

25.4.10 3.2. Callback host oldali értesítésre

A `cudaStreamAddCallback` segítségével a streamhez egy host-oldali függvényt adhatunk. A callback akkor fut le, amikor a streamben az összes korábbi eszközművelet befejeződött:

```
void CUDART_CB my_callback(void* user_data) {
    auto* data = static_cast<std::atomic<int>*>(user_data);
    data->fetch_add(1, std::memory_order_release);
}
```

A callback a stream sorrendjében hajtódik végre, de host-kód, nem GPU-kód. A gyakorlatban a callback legyen rövid, ne blokkoljon, és ne hívjon benne szinkronizáló CUDA-műveleteket, mert az a stream további műveleteinek ütemezését is feltarthatja.

Gyakori kombináció:

1. A kernel után `cudaEventRecord(compute_done, compute_stream)`.
2. A másoló stream `cudaStreamWaitEvent`-tel megvárja.
3. A D2H másolás után `cudaStreamAddCallback` értesíti a hostot, hogy az eredmény a host memóriában van.
4. A callback növeli a befejezett chunkok számlálóját, vagy jelzi egy `std::condition_variable`-nek az új eredményt.

Így a host nem kénytelen folyamatosan pollozni, az értesítés automatikus.

25.4.11 3.3. Tervezési hiba: callback túlterhelés

Hibás minta:

```
void CUDART_CB bad_callback(void* data) {
    cudaStreamSynchronize(nullptr);  // rossz: blokkoló hívás callbackből
    heavy_cpu_work();                // rossz: hosszú munka a stream ütemezőjében
}
```

Az ilyen callback egy hosszú pipeline-ban visszafogja a következő chunkok ütemezését. Helyes megoldás: a callback csak jelölje, hogy egy munka elkészült, a tényleges feldolgozást pedig egy másik host szál végezze.

25.4.12 4. Hibakezelés aszinkron környezetben

Az aszinkron CUDA-hívásoknál a hiba nem mindig ott jelentkezik, ahol keletkezett. A `cudaMemcpyAsync`, a kernelindítás vagy a `cudaEventRecord` azonnal visszatérhet, a tényleges hiba később, egy szinkronizáló híváskor jelenik meg.

25.4.13 4.1. Késleltetett hibák

A kernelindítás után a `cudaGetLastError()` csak a launch szintű hibákat mutatja, például érvénytelen grid- vagy blockméretet, hiányzó eszközt vagy rossz streamet. Az aszinkron végrehajtás közben keletkező memóriahibák, illegal instruction vagy eszközhibák gyakran csak a következő pontokon figyelhetők meg:

- `cudaStreamSynchronize(stream)`,
- `cudaEventSynchronize(event)`,
- `cudaEventQuery(event)`,
- `cudaDeviceSynchronize()`,
- a következő CUDA-hívás, amely a context állapotát ellenőrzi.

Emiatt a megbízható CUDA-kód minden aszinkron folyamat végén szinkronizál, és a szinkronizálás után is ellenőrzi a hibát.

25.4.14 4.2. Hibaellenőrző makró aszinkron műveletekhez

A könyvben használt hibamakró aszinkron pipeline-hoz is használható:

```
#define CK_CUDA(expr) do {                                          \
    cudaError_t err = (expr);                                       \
    if (err != cudaSuccess) {                                       \
        std::fprintf(stderr, "CUDA error at %s:%d: %s\n",           \
                     __FILE__, __LINE__, cudaGetErrorString(err));\
        std::abort();                                               \
    }                                                               \
} while (0)
```

A makrót a launch előtti és a szinkronizáló hívásokra is használjuk. A `cudaStreamSynchronize` visszatérési értékét például mindig ellenőrizni kell:

```
CK_CUDA(cudaStreamSynchronize(compute_stream));
```

25.4.15 4.3. Leállítás és erőforrás-felszabadítás

A leállítás sorrendje aszinkron rendszerben:

1. Jelezzük a termelő szálnak, hogy ne indítson újabb chunkot.
2. Megvárjuk a folyamatban lévő chunkok befejezését.
3. Szinkronizáljuk az összes streamet és eventet.
4. Felszabadítjuk a device- és host-memóriákat.

5. Töröljük az eventeket és streameket.

Tipikus hiba, ha a host memóriát felszabadítja, miközben egy stream még mindig használja:

```
// Hibás: a stream még nem fejeződött be
cudaMemcpyAsync(dst, src, bytes, cudaMemcpyDeviceToHost, stream);
cudaFreeHost(src); // rossz, lehet hogy a másolás még folyamatban van
```

Helyes sorrend:

```
cudaMemcpyAsync(dst, src, bytes, cudaMemcpyDeviceToHost, stream);
CK_CUDA(cudaStreamSynchronize(stream));
cudaFreeHost(src);
```

Ha a kernel esetleg beragadt, akkor a folyamat leállítása előtt watchdog- vagy timeout-mechanizmust kell alkalmazni. Többfelhasználós környezetben ne hívjunk meggondolatlanul `cudaDeviceReset`-et, mert az az eszközön lévő más folyamatokat is érintheti. Izolált laborban használhatunk külső timeoutot, például konténeres futásidő-korlátot vagy `timeout` parancsot.

25.4.16 5. Labor: aszinkron pipeline tervezése

A labor célja egy olyan pipeline elkészítése, amely egy nagy adathalmazt chunkokra oszt, és a másolást, a GPU-számítást, valamint a visszamásolást event-alapú függőségekkel átfedésben végzi. A feladat a hosszú futású feldolgozás egy kis, ellenőrizhető modellje.

25.4.17 5.1. Kiinduló környezet

Az alábbi környezetet rögzítsük minden méréshez:

Paraméter	Érték
GPU	NVIDIA A100 40 GB PCIe, ECC be, MIG ki
Driver	550.54.14
CUDA toolkit	12.4
Operációs rendszer	Ubuntu 22.04
Precision	FP32
Fordító	nvcc, C++17

A konkrét értékeket mindenki a saját környezetéhez igazítja; a naplóban nem helyettesíthetők az itt szereplő példaértékek valós mérésként.

25.4.18 5.2. Kernelszámítás

Az egyszerű feladat egy vektor skálázása:

```
__global__ void scale_kernel(const float* in, float* out,
                             float alpha, size_t n) {
    size_t idx = blockIdx.x * blockDim.x + threadIdx.x;
    size_t stride = gridDim.x * blockDim.x;
    for (size_t i = idx; i < n; i += stride) {
        out[i] = alpha * in[i];
    }
}
```

A kernel memória-korlátos, ezért alkalmas a másolási átfedés megfigyelésére.

25.4.19 5.3. Aszinkron pipeline CUDA C++-ban

A kód minden chunkhoz külön eventet használ, hogy elkerülje az event-újrahasznosításból adódó sorrendi bizonytalanságot. A callback a befejezett D2H-másolást jelzi.

```
// CUDA C++17
#include <cuda_runtime.h>
#include <atomic>
#include <cstdio>
#include <cstdlib>
#include <vector>

#define CK_CUDA(expr) do {                                          \
    cudaError_t err = (expr);                                       \
    if (err != cudaSuccess) {                                       \
        std::fprintf(stderr, "CUDA error at %s:%d: %s\n",           \
                     __FILE__, __LINE__, cudaGetErrorString(err));\
        std::abort();                                               \
    }                                                               \
} while (0)

struct Chunk {
    float* h_in = nullptr;
    float* h_out = nullptr;
    float* d_in = nullptr;
    float* d_out = nullptr;
    size_t n = 0;
};

struct CallbackData {
    std::atomic<size_t>* finished;
    size_t chunk_id;
};

void CUDART_CB on_d2h_done(void* user_data) {
    auto* data = static_cast<CallbackData*>(user_data);
    data->finished->fetch_add(1, std::memory_order_release);
    delete data;
}

void run_async_pipeline(size_t chunk_count, size_t n_per_chunk, float alpha) {
    const size_t bytes = n_per_chunk * sizeof(float);

    cudaStream_t copy_stream, compute_stream, out_stream;
    CK_CUDA(cudaStreamCreate(&copy_stream));
    CK_CUDA(cudaStreamCreate(&compute_stream));
    CK_CUDA(cudaStreamCreate(&out_stream));

    std::vector<cudaEvent_t> copy_done(chunk_count);
    std::vector<cudaEvent_t> compute_done(chunk_count);
    std::vector<cudaEvent_t> out_done(chunk_count);

    for (size_t i = 0; i < chunk_count; ++i) {
        CK_CUDA(cudaEventCreate(&copy_done[i]));
        CK_CUDA(cudaEventCreate(&compute_done[i]));
        CK_CUDA(cudaEventCreate(&out_done[i]));
    }

    std::vector<Chunk> chunks(chunk_count);
    for (size_t i = 0; i < chunk_count; ++i) {
        Chunk& c = chunks[i];
        c.n = n_per_chunk;
        CK_CUDA(cudaMallocHost(&c.h_in, bytes));
```

```
        CK_CUDA(cudaMallocHost(&c.h_out, bytes));
        CK_CUDA(cudaMalloc(&c.d_in, bytes));
        CK_CUDA(cudaMalloc(&c.d_out, bytes));
        for (size_t j = 0; j < n_per_chunk; ++j) c.h_in[j] = static_cast<float>(j) * 0.001f;
    }

    std::atomic<size_t> finished{0};

    for (size_t i = 0; i < chunk_count; ++i) {
        Chunk& c = chunks[i];

        CK_CUDA(cudaMemcpyAsync(c.d_in, c.h_in, bytes,
                                cudaMemcpyHostToDevice, copy_stream));
        CK_CUDA(cudaEventRecord(copy_done[i], copy_stream));

        CK_CUDA(cudaStreamWaitEvent(compute_stream, copy_done[i], 0));
        int block = 256;
        int grid = static_cast<int>((n_per_chunk + block - 1) / block);
        scale_kernel<<<grid, block, 0, compute_stream>>>(
            c.d_in, c.d_out, alpha, c.n);
        CK_CUDA(cudaGetLastError());
        CK_CUDA(cudaEventRecord(compute_done[i], compute_stream));

        CK_CUDA(cudaStreamWaitEvent(out_stream, compute_done[i], 0));
        CK_CUDA(cudaMemcpyAsync(c.h_out, c.d_out, bytes,
                                cudaMemcpyDeviceToHost, out_stream));
        CK_CUDA(cudaStreamAddCallback(
            out_stream, on_d2h_done,
            new CallbackData{&finished, i}, 0));
        CK_CUDA(cudaEventRecord(out_done[i], out_stream));
    }

    for (size_t i = 0; i < chunk_count; ++i) {
        CK_CUDA(cudaEventSynchronize(out_done[i]));
    }

    CK_CUDA(cudaStreamSynchronize(copy_stream));
    CK_CUDA(cudaStreamSynchronize(compute_stream));
    CK_CUDA(cudaStreamSynchronize(out_stream));

    std::printf("pipeline done, callbacks: %zu/%zu\n",
                finished.load(std::memory_order_acquire), chunk_count);

    for (Chunk& c : chunks) {
        CK_CUDA(cudaFreeHost(c.h_in));
        CK_CUDA(cudaFreeHost(c.h_out));
        CK_CUDA(cudaFree(c.d_in));
        CK_CUDA(cudaFree(c.d_out));
    }
    for (size_t i = 0; i < chunk_count; ++i) {
        CK_CUDA(cudaEventDestroy(copy_done[i]));
        CK_CUDA(cudaEventDestroy(compute_done[i]));
        CK_CUDA(cudaEventDestroy(out_done[i]));
    }
    CK_CUDA(cudaStreamDestroy(copy_stream));
    CK_CUDA(cudaStreamDestroy(compute_stream));
    CK_CUDA(cudaStreamDestroy(out_stream));
}

int main() {
    size_t chunk_count = 256;
    size_t n_per_chunk = 1u << 20; // 1 Mi float, 4 MiB per chunk
    float alpha = 1.5f;
```

```
    // warm-up egyszer
    run_async_pipeline(8, n_per_chunk, alpha);

    // mérés: 10 ismétlés
    for (int rep = 0; rep < 10; ++rep) {
        cudaEvent_t start, stop;
        CK_CUDA(cudaEventCreate(&start));
        CK_CUDA(cudaEventCreate(&stop));
        CK_CUDA(cudaEventRecord(start));
        run_async_pipeline(chunk_count, n_per_chunk, alpha);
        CK_CUDA(cudaEventRecord(stop));
        CK_CUDA(cudaEventSynchronize(stop));
        float ms = 0.0f;
        CK_CUDA(cudaEventElapsedTime(&ms, start, stop));
        std::printf("rep %d: %.3f ms\n", rep, ms);
        CK_CUDA(cudaEventDestroy(start));
        CK_CUDA(cudaEventDestroy(stop));
    }
    return 0;
}
```

A kódban minden kernelindítás után `cudaGetLastError`, minden szinkronizálás után `CK_CUDA` fut. A callback csak atomi számlálót növel, ezért rövid marad.

25.4.20 5.4. Fordítás és futtatás

Fordítás:

```
nvcc -std=c++17 -O3 -arch=sm_80 -o async_pipeline async_pipeline.cu
```

Futtatás izolált laborban:

```
timeout 120 ./async_pipeline
```

Ha a kód hosszabb ideig futna, a `timeout` és a konténer erőforráskorlátja megakadályozza a beragadt kernelt.

25.4.21 5.5. Mérési napló

A mérési naplóban a **várt** és a **mért** értékeket külön kell vezetni. Az alábbi táblázat kitöltendő; az itt feltüntetett számok nem tekinthetők hiteles mérésnek.

Mérőszám	Várt	Mért
Teljes adatmennyiség	256×4 MiB = 1 GiB	`<rögzítendő>`
Kernelszámítási idő, eszközmemória alapján	2 GiB / ~1,5 TB/s 1,4 ms	`<Nsight Compute vagy event>`
H2D másolási idő, PCIe alapján	1 GiB / ~12–25 GiB/s	`<mérendő>`
D2H másolási idő	1 GiB / ~12–25 GiB/s	`<mérendő>`
Ideális átfedéses átfutás	max(H2D, D2H, kernel) + overhead	`<mérési napló>`
10 futtatás mediánja	`<várt formula>`	`<mérendő>`
Első chunk callback késleltetés	kis, stream ütemezés + kernel	`<mérendő>`
Nsight Systems átfedési arány	cél $> 90\%$	`<Nsight riport>`

A várt értékeknél mindig írjuk le, mely sávszélességből vagy eszközteljesítményből származnak, és hogy milyen feltételezésekkel élünk. Például ha a PCIe irányonkénti sávszélessége 12 GiB/s, akkor a H2D várhatóan 1 GiB / 12 GiB/s 0,083 s, a D2H hasonló. Ha a PCIe duplex, a teljes átfutás közel lehet a nagyobbik irány idejéhez; ha nem teljesen átfedhető, akkor az összeghez.

25.4.22 5.6. Értékelési szempontok

A labor akkor sikeres, ha:

- a callbackek száma megegyezik a chunkok számával,
- a mért teljes átfutás a várt aszinkron képlet közelében van,
- az Nsight Systems profilban a H2D, kernel és D2H fázisok átfedése látható,
- nincs felszabadítási hiba vagy memóriaszivárgás,
- a leállítás után minden CUDA API-hibaellenőrzés tiszta.

25.4.23 6. Tipikus hibák és helyes megoldások

Az aszinkron tervezési mintákban visszatérő hibák:

1. **Default stream használata nem non-blocking módban.** A default stream szinkronizációs szemantikája miatt az implicit szinkronizáció tönkreteheti az átfedést. Használjunk explicit streameket, és ha szükséges, `cudaStreamNonBlocking` flaget.
2. **Esemény újrahasznosítása chunkok között.** Ha ugyanazt az eventet rögzítjük több chunkhoz, az függőségi bizonytalanságot okozhat. Használjunk event poolt vagy chunkonkénti eventet.
3. **callbackből szinkronizálás vagy hosszú munka.** A callback blokkolja az adott stream további ütemezését. Csak rövid értesítést tegyünk bele.
4. **Memória felszabadítása szinkronizálás nélkül.** A stream még használhatja a buffert. Előbb `cudaStreamSynchronize` vagy `cudaEventSynchronize`, majd `cudaFreeHost`.
5. **A hibaellenőrzés elhagyása a szinkronizálás után.** A launch utáni `cudaGetLastError` nem mutatja az aszinkron hibát. A szinkronizáló hívás visszatérési értékét is ellenőrizni kell.
6. **Korlátlan chunk-ablak.** Hosszú futású pipeline-ban a host elébe megy a GPU-nak, ami memórianyomást és event-túlcsordulást okozhat. Vezessünk be legfeljebb `W` folyamatban lévő chunkot.
7. **Interaktív leállítás tervezése nélkül.** Ha egy hosszú kernel nem ellenőriz leállítási flaget, a felhasználó nem tudja megszakítani. Tervezzük meg a cooperativ cancel mechanizmust már a kernel API-jában.

25.4.24 7. OpenCL- és JAX-vonatkozások

Az aszinkron tervezési alapelvek nem csak CUDA-ra érvényesek.

- **OpenCL:** az out-of-order command queue, a `cl_event` és a `clSetEventCallback` hasonló szerepet töltenek be. A work-group és a command-queue közötti függőségek explicit eventekkel adhatók meg. A callback ott is host-oldali befejezésértesítést ad.

- **JAX:** az API hívásai gyakran aszinkron dispatch-ek; a `block_until_ready()` explicit szinkronizálás, a hiánya pedig lehetővé teszi az XLA által szervezett pipeline-t. A JAX-keretben a callback- és eseménykezelés gyakran rejtve marad, de a sharding és az eszköz-összehangolás mögött ugyanazok a függőségi gráfok dolgoznak.

25.4.25 Önellenőrző kérdések

1. Miért nem elég a `cudaGetLastError()` közvetlenül a kernelindítás után egy hosszú aszinkron pipeline hibáinak kimutatására?
2. Hogyan terveznéd meg egy olyan GPU-feladat leállítását, amely egyetlen nagy grid-stride kernelben dolgozik fel egy nagy adathalmazt?
3. Milyen veszélye van annak, ha hosszú futású pipeline-ban minden chunkhoz új pinned memóriát allokálunk?
4. Mi a különbség az event `cudaEventQuery`, a `cudaEventSynchronize` és a `cudaStreamSynchronize` szerepe között egy interaktív alkalmazásban?

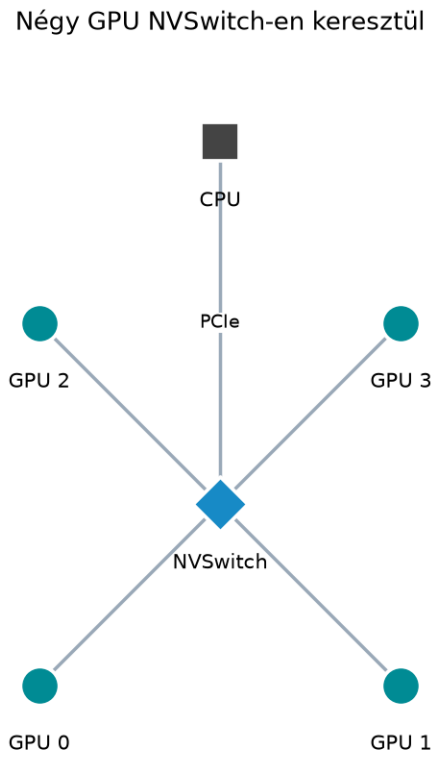
26 Multi-GPU, NCCL, NVLink és unified memory

26.1 Több GPU és peer access

Több GPU és peer access

Több GPU használata logikailag egyszerű lépésnek tűnik: „adok neki még egy eszközt, és kétszer gyorsabb leszek”. A gyakorlatban azonban a több GPU-s programozás első szintje a *per-device kontextus*, a *PCIe/NVLink topológia* és a *peer access* kérdése. Ha ezeket nem értjük pontosan, a második GPU hozzáadása akár lassulást is hozhat, mert a memóriamásolások rossz irányba mennek, vagy a kernelek egy nem kívánt eszközön futnak le.

Ez az alfejezet a több GPU-s végrehajtás alapjait tanítja meg. Megismerjük a `cudaSetDevice` és a `cudaMemcpyPeer` alacsony szintű eszközeit, a peer access engedélyezésének feltételeit, valamint a topológia szerepét a másolási utak kiválasztásában. A fejezet zárásaként egy olyan labort építünk, amely két GPU-n egyszerre dolgozó kerneleket futtat, peer-to-peer (P2P) másolást végez, és mérési naplóval igazolja a tényleges sávszélességet.



26.1. ábra: Négy GPU NVSwitch-en keresztül - Az NVLink/NVSwitch topológia nagy sávszélességű GPU-GPU kommunikációt ad; a peer access és a NCCL kollektívák ezt használják.

26.1.1 1. A több GPU-s végrehajtás alapjai

A CUDA programozási modellben a host CPU egy vagy több CUDA-*eszközzel* rendelkezhet. Minden eszköznek saját globális memóriája, saját streamjei, saját kontextusa és saját hibaállapota van. A host oldali CUDA API hívások mindig egy *aktuális eszközre* vonatkoznak. Az aktuális eszköz kezdetben a `0` indexű eszköz, de bármikor átválthatunk egy másikra a `cudaSetDevice` hívással.

A több GPU-s programok esetében három alapvető kérdést kell megválaszolni:

1. **Melyik eszközön fut a kernel?** A kernel indítása mindig az aktuális eszközre vonatkozik.
2. **Hol lakik a memória?** A `cudaMalloc` mindig az aktuális eszközön foglal.
3. **Hogyan mozgatunk adatot két eszköz között?** Erre szolgál a `cudaMemcpyPeer`, amennyiben a peer access engedélyezett.

Egy több GPU-s program váza a következő:

```
// Fordító: nvcc -arch=native -std=c++20 multi_gpu_basic.cu -o multi_gpu_basic
#include <cuda_runtime.h>
#include <cstdio>

int main() {
    int deviceCount = 0;
    cudaGetDeviceCount(&deviceCount);
    printf("CUDA devices: %d\n", deviceCount);
```

```
    // 1. eszköz kiválasztása
    cudaSetDevice(0);
    // ... itt minden a 0. eszközre vonatkozik

    // 2. eszköz kiválasztása
    cudaSetDevice(1);
    // ... itt minden az 1. eszközre vonatkozik
}
```

A `cudaSetDevice` hívás hatása *azonnali* a host oldali API hívásokra. A már folyamatban lévő aszinkron műveletek a korábban kiválasztott eszközön folytatódnak, de a *következő* API hívás már az új eszközre vonatkozik.

Fontos különbség, hogy egy host szálon belül egyszerre csak egy aktuális eszköz van. Több host szál használata esetén minden szál a saját aktuális eszközével dolgozhat, de a CUDA API szálbiztonságosságára ügyelni kell: az egyes eszközök állapotai függetlenek, de a `cudaSetDevice` hívás egy host szálra érvényes, nem processz szintű.

26.1.2 2. Eszközök felderítése és tulajdonságaik

Mielőtt peer access-t próbálunk engedélyezni, tudnunk kell, milyen eszközök állnak rendelkezésre, és milyen topológiában kapcsolódnak egymáshoz. A CUDA API erre két fontos eszközt ad: a `cudaGetDeviceProperties` és a `cudaDeviceGetP2PAttribute` hívásokat.

A `cudaGetDeviceProperties` a `cudaDeviceProp` struktúrát tölti ki, amely tartalmazza az eszköz nevét, compute capability verzióját, órajelét, memóriaméretét, valamint a PCIe/NVLink kapcsolódásra vonatkozó `pciBusID`, `pciDeviceID` és `pciDomainID` mezőket. A `cudaDeviceGetP2PAttribute` ezzel szemben konkrét P2P képességeket vizsgál:

```
#include <cuda_runtime.h>
#include <cstdio>
#include <cstdlib>
#include <string>

#define CUDA_CHECK(call)                                                \
    do {                                                                \
        cudaError_t err = (call);                                       \
        if (err != cudaSuccess) {                                       \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,   \
                    cudaGetErrorString(err));                           \
            exit(EXIT_FAILURE);                                         \
        }                                                               \
    } while (0)

int main() {
    int count = 0;
    CUDA_CHECK(cudaGetDeviceCount(&count));
    printf("Eszkozok szama: %d\n", count);

    for (int i = 0; i < count; ++i) {
        cudaDeviceProp prop{};
        CUDA_CHECK(cudaGetDeviceProperties(&prop, i));
        printf("[%d] %s | CC %d.%d | PCIe bus=%02x device=%02x\n",
               i, prop.name, prop.major, prop.minor,
               prop.pciBusID, prop.pciDeviceID);
    }
```

```
    // Peer access vizsgálat minden rendezett párra
    for (int i = 0; i < count; ++i) {
        for (int j = 0; j < count; ++j) {
            if (i == j) continue;
            int canAccessPeer = 0;
            CUDA_CHECK(cudaDeviceCanAccessPeer(&canAccessPeer, i, j));
            printf("P2P access %d -> %d: %s\n", i, j,
                   canAccessPeer ? "igen" : "nem");
        }
    }
    return 0;
}
```

A `cudaDeviceCanAccessPeer` a legegyszerűbb módja annak megállapítására, hogy két eszköz között lehetséges-e peer access. Ez azonban csak a *lehetőséget* vizsgálja, nem az aktuális engedélyezettséget. Az engedélyezéshez a `cudaDeviceEnablePeerAccess` API hívás szükséges.

A peer access feltételei jellemzően a következők:

1. **Azonos compute capability család** a legtöbb P2P-művelethez (bizonyos másolási módok eltérő CC esetén is működhetnek, de a kernelből való közvetlen hozzáféréshez ez általában szükséges).
2. **Közös vagy átjárható PCIe/NVLink topológia**: a két eszköznek ugyanazon a PCIe-fán kell lennie, vagy NVLink-kapcsolattal kell rendelkeznie. A különböző host-hidak mögött lévő eszközök gyakran nem tudnak peer access-t biztosítani.
3. **Nincs MIG izoláció** az adott eszközökön, mert a MIG-példányok közötti peer access korlátozott vagy tiltott.
4. **Az operációs rendszer és a driver támogatja** a peer mappinget (Linux/Windows eltérések lehetségesek).

26.1.3 3. A `cudaMemcpyPeer` és a peer access engedélyezése

A `cudaMemcpyPeer` két eszköz közötti közvetlen memóriamasolást végez. Alakja a következő:

```
cudaError_t cudaMemcpyPeer(void* dst, int dstDevice,
                           const void* src, int srcDevice,
                           size_t count);
```

A `dst` és `src` mutatók a megfelelő eszközök globális memóriájában érvényes *device pointerek*. A `count` bájtban értendő. A hívás szinkron a host szálra nézve (a korábban kiválasztott eszköz streamjétől függően), de a végrehajtás aszinkron lehet az eszközökön.

Példa két eszköz közötti másolásra:

```
#include <cuda_runtime.h>
#include <cstdio>
#include <cstdlib>
#include <vector>

#define CUDA_CHECK(call)                                                \
    do {                                                                \
        cudaError_t err = (call);                                       \
        if (err != cudaSuccess) {                                       \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,   \
                    cudaGetErrorString(err));                           \
            exit(EXIT_FAILURE);                                         \
```

```
        }                                                                   \
    } while (0)

int main() {
    const size_t N = 1 << 22; // 4M float
    const size_t bytes = N * sizeof(float);

    // 0. eszköz allokáció
    CUDA_CHECK(cudaSetDevice(0));
    float* d0 = nullptr;
    CUDA_CHECK(cudaMalloc(&d0, bytes));
    CUDA_CHECK(cudaMemset(d0, 0, bytes)); // inicializálás

    // 1. eszköz allokáció
    CUDA_CHECK(cudaSetDevice(1));
    float* d1 = nullptr;
    CUDA_CHECK(cudaMalloc(&d1, bytes));
    CUDA_CHECK(cudaMemset(d1, 0, bytes));

    // Peer access engedélyezése 0 -> 1 irányban
    int canAccess = 0;
    CUDA_CHECK(cudaDeviceCanAccessPeer(&canAccess, 1, 0));
    if (canAccess) {
        CUDA_CHECK(cudaSetDevice(1));
        CUDA_CHECK(cudaDeviceEnablePeerAccess(0, 0));
        printf("Peer access 1 -> 0 engedelyezve\n");
    } else {
        printf("Peer access 1 -> 0 nem engedelyezheto\n");
    }

    // Másolás 0 -> 1 (d0-ból d1-be)
    CUDA_CHECK(cudaMemcpyPeer(d1, 1, d0, 0, bytes));
    CUDA_CHECK(cudaDeviceSynchronize());

    printf("Peer masolas kesz: %zu bájt\n", bytes);

    CUDA_CHECK(cudaFree(d0));
    CUDA_CHECK(cudaFree(d1));
    return 0;
}
```

A `cudaDeviceEnablePeerAccess` szignatúrája:

```
cudaError_t cudaDeviceEnablePeerAccess(int peerDevice, unsigned int flags);
```

A `peerDevice` annak az eszköznek az indexe, amelyikhez hozzáférést akarunk engedélyezni az aktuális eszközről. A `flags` jelenleg kötelezően `0`. A hívás előtt `cudaSetDevice`-szel ki kell választani azt az eszközt, amely *kezdeményezi* a hozzáférést. A fenti példában az 1. eszközről akarunk hozzáférni a 0. eszközhöz, ezért `cudaSetDevice(1)` után hívjuk meg a `cudaDeviceEnablePeerAccess(0, 0)`-t.

Fontos aszimmetria: a peer access *engedélyezés* irányonként külön történik. Ha a 0. eszközről is hozzá akarunk férni az 1. eszközhöz, akkor `cudaSetDevice(0)` után `cudaDeviceEnablePeerAccess(1, 0)` is szükséges. Ez az aszimmetria gyakori hibaforrás.

A `cudaMemcpyPeer` nem igényel peer access-t a *másolás* elvégzéséhez, csak a *közvetlen* P2P-másoláshoz. Ha a peer access nincs engedélyezve, a másolás akkor is lefut, de a driver a host memóriáján keresztül *staged copy*-ként hajtja végre (device→host→device), ami jelentős többletterheléssel jár. A `cudaMemcpyPeerAsync` streamhez kötött változat, amely lehetővé teszi a másolás és a kernel-végrehajtás átfedését.

26.1.4 4. Peer access kernelből: közvetlen eszközhozzáférés

A peer access igazi ereje nem a `cudaMemcpyPeer` egyszerűbb hívásában van, hanem abban, hogy egy kernel *közvetlenül olvashat és írhat* egy másik eszköz globális memóriáját. Ezt a mechanizmust *peer device memory access*nek nevezzük. Ehhez a peer access-t előzetesen engedélyezni kell, a kernel pedig normál device pointerrel dolgozik, amelyet a másik eszközön foglaltunk le.

Példa: az 1. eszközön futó kernel hozzáadja a 0. eszköz memóriájában lévő tömböt az 1. eszközön lévő tömbhöz.

```
__global__ void add_peer_kernel(const float* srcPeer, float* dstLocal, size_t n) {
    size_t idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        dstLocal[idx] += srcPeer[idx];
    }
}

int main() {
    const size_t N = 1 << 20;
    const size_t bytes = N * sizeof(float);

    CUDA_CHECK(cudaSetDevice(0));
    float* d0 = nullptr;
    CUDA_CHECK(cudaMalloc(&d0, bytes));
    CUDA_CHECK(cudaMemset(d0, 1, bytes)); // minden byte 1, float érték itt nem fontos

    CUDA_CHECK(cudaSetDevice(1));
    float* d1 = nullptr;
    CUDA_CHECK(cudaMalloc(&d1, bytes));
    CUDA_CHECK(cudaMemset(d1, 2, bytes));

    // Peer access 1 -> 0
    int canAccess = 0;
    CUDA_CHECK(cudaDeviceCanAccessPeer(&canAccess, 1, 0));
    if (canAccess) {
        CUDA_CHECK(cudaSetDevice(1));
        CUDA_CHECK(cudaDeviceEnablePeerAccess(0, 0));

        // Kernel az 1. eszközön, de d0 a 0. eszközön van
        int threads = 256;
        int blocks = (N + threads - 1) / threads;
        add_peer_kernel<<<blocks, threads>>>(d0, d1, N);
        CUDA_CHECK(cudaGetLastError());
        CUDA_CHECK(cudaDeviceSynchronize());
        printf("Peer kernel kesz\n");
    } else {
        printf("Peer access nem engedelyezheto, kernel nem indithato\n");
    }

    CUDA_CHECK(cudaFree(d0));
    CUDA_CHECK(cudaFree(d1));
    return 0;
}
```

Ha a peer access nincs engedélyezve, a kernelből való közvetlen hozzáférés *nem definiált viselkedés*: jellemzően hibaüzenetet kapunk, de bizonyos rendszereken silány csendes hibához is vezethet. Ezért kötelező a `cudaDeviceCanAccessPeer` ellenőrzés a kernel indítása előtt.

26.1.5 5. Topológia és a P2P-sávszélesség

A peer access teljesítményét a két eszköz közötti fizikai kapcsolat határozza meg. A leggyakoribb topológiák:

1. **PCIe switch alatt közvetlenül**: az eszközök ugyanazon PCIe switch alatt helyezkednek el. A P2P-másolás ilyenkor a switch-en keresztül történik, a host memória érintése nélkül. A sávszélesség a PCIe generációtól függ (PCIe 4.0 x16 32 GB/s elméleti, de a gyakorlati P2P sávszélesség ennek 60-80%-a).
2. **NVLink**: bizonyos GPU-k (A100, H100, stb.) NVLink-kapcsolattal rendelkeznek, amely jóval nagyobb sávszélességet biztosít (NVLink 3.0: 50 GB/s sávonként, NVLink 4.0: 100 GB/s sávonként). Az NVLink P2P-másolás lényegesen gyorsabb, mint a PCIe-alapú.
3. **Host bridge-en keresztül**: ha a két eszköz különböző PCIe host bridge-en ül, a peer access gyakran nem engedélyezhető, vagy csak nagyon alacsony sávszélességgel működik. Ilyenkor a driver a host memórián keresztüli másolásra esik vissza.
4. **NVSwitch**: nagyobb rendszerekben (DGX, HGX) az összes GPU-t egy NVSwitch hálózat köti össze, ami teljes all-to-all NVLink sávszélességet biztosít.

A topológia lekérdezéséhez nincs közvetlen CUDA API, de a `cudaDeviceGetP2PAttribute` segítségével lekérdezhetjük a P2P-másolás teljesítményére vonatkozó jellemzőket. A gyakorlatban a legegyszerűbb módszer egy benchmark futtatása: mérjük meg a P2P-másolás sávszélességét `cudaMemcpyPeer` hívással, és hasonlítsuk össze a host-memórián keresztüli másolás sávszélességével.

Egy gyors P2P benchmark vázlata:

```
#include <cuda_runtime.h>
#include <cstdio>
#include <cstdlib>
#include <chrono>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err = (call);                                           \
        if (err != cudaSuccess) {                                           \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,   \
                    cudaGetErrorString(err));                               \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

int main() {
    const size_t N = 1 << 26; // 64M float  256 MB
    const size_t bytes = N * sizeof(float);
    const int repeat = 10;

    CUDA_CHECK(cudaSetDevice(0));
    float* d0 = nullptr;
    CUDA_CHECK(cudaMalloc(&d0, bytes));
    CUDA_CHECK(cudaMemset(d0, 0, bytes));

    CUDA_CHECK(cudaSetDevice(1));
    float* d1 = nullptr;
    CUDA_CHECK(cudaMalloc(&d1, bytes));
```

```
    CUDA_CHECK(cudaMemset(d1, 0, bytes));

    // Peer access engedélyezése mindkét irányban
    int canAccess01 = 0, canAccess10 = 0;
    CUDA_CHECK(cudaDeviceCanAccessPeer(&canAccess01, 0, 1));
    CUDA_CHECK(cudaDeviceCanAccessPeer(&canAccess10, 1, 0));
    if (canAccess01) {
        CUDA_CHECK(cudaSetDevice(0));
        CUDA_CHECK(cudaDeviceEnablePeerAccess(1, 0));
    }
    if (canAccess10) {
        CUDA_CHECK(cudaSetDevice(1));
        CUDA_CHECK(cudaDeviceEnablePeerAccess(0, 0));
    }

    // Warm-up
    for (int i = 0; i < 5; ++i) {
        CUDA_CHECK(cudaMemcpyPeer(d1, 1, d0, 0, bytes));
        CUDA_CHECK(cudaDeviceSynchronize());
    }

    // Mérés
    auto start = std::chrono::high_resolution_clock::now();
    for (int i = 0; i < repeat; ++i) {
        CUDA_CHECK(cudaMemcpyPeer(d1, 1, d0, 0, bytes));
        CUDA_CHECK(cudaDeviceSynchronize());
    }
    auto end = std::chrono::high_resolution_clock::now();

    double totalTime = std::chrono::duration<double>(end - start).count();
    double avgTime = totalTime / repeat;
    double bw = (bytes / 1.0e9) / avgTime; // GB/s

    printf("P2P masolas: %zu bájt, atlag ido: %.3f ms, savszelesseg: %.2f GB/s\n",
           bytes, avgTime * 1e3, bw);

    CUDA_CHECK(cudaFree(d0));
    CUDA_CHECK(cudaFree(d1));
    return 0;
}
```

Ez a benchmark a valós P2P-sávszélességet méri. Ha a peer access nem engedélyezhető, a `cudaMemcpyPeer` automatikusan a host memóriáján keresztül másol, ami lényegesen alacsonyabb sávszélességet ad. A mért értékekből egyértelműen azonosítható, hogy a rendszer valódi P2P-t vagy visszaeső másolást végez-e.

26.1.6 6. Labor: peer access és P2P másolás

A labor célja egy teljes, futtatható CUDA program létrehozása, amely két GPU-n peer access-t engedélyez, P2P-másolást végez, valamint összehasonlítja a P2P-másolás és a host-memórián keresztüli másolás sávszélességét. A labor a korábbi ismeretekre épít, de nem igényel NCCL-t vagy kollektív műveleteket.

26.1.7 6.1 Laborfeladat

Két GPU-t tartalmazó rendszeren (vagy MIG nélküli virtuális környezetben) írjunk CUDA C++ programot, amely:

1. Felderíti az eszközöket és a P2P-képességeiket.
2. Engedélyezi a peer access-t minden lehetséges irányban.
3. Lefoglal egy `float` tömböt mindkét eszközön.
4. Végrehajt egy P2P-másolást `cudaMemcpyPeer`-rel, és méri a sávszélességet.
5. Végrehajt egy host-memórián keresztüli másolást (Manuálisan: device→host→device) és méri annak sávszélességét.
6. Összehasonlítja a két sávszélességet, és megállapítja, hogy a P2P valódi gyorsulást hoz-e.

26.1.8 6.2 A teljes laborprogram

```
// File: p2p_lab.cu
// Fordító: nvcc -arch=native -std=c++20 -O3 p2p_lab.cu -o p2p_lab
// Futtatás: ./p2p_lab

#include <cuda_runtime.h>
#include <cstdio>
#include <cstdlib>
#include <chrono>
#include <vector>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err = (call);                                           \
        if (err != cudaSuccess) {                                           \
            fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,   \
                    cudaGetErrorString(err));                               \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

// P2P másolás benchmark
double benchmark_p2p_copy(int srcDevice, int dstDevice, void* src, void* dst,
                          size_t bytes, int repeat) {
    CUDA_CHECK(cudaSetDevice(srcDevice));
    for (int i = 0; i < 5; ++i) { // warm-up
        CUDA_CHECK(cudaMemcpyPeer(dst, dstDevice, src, srcDevice, bytes));
        CUDA_CHECK(cudaDeviceSynchronize());
    }

    auto start = std::chrono::high_resolution_clock::now();
    for (int i = 0; i < repeat; ++i) {
        CUDA_CHECK(cudaMemcpyPeer(dst, dstDevice, src, srcDevice, bytes));
        CUDA_CHECK(cudaDeviceSynchronize());
    }
    auto end = std::chrono::high_resolution_clock::now();

    double totalTime = std::chrono::duration<double>(end - start).count();
    return totalTime / repeat;
}

// Host-stage másolás benchmark: d_src -> h_stage -> d_dst
double benchmark_host_stage_copy(int srcDevice, int dstDevice,
                                 void* d_src, void* d_dst, size_t bytes,
                                 int repeat) {
    std::vector<char> stage(bytes);
```

```
    // Warm-up
    for (int i = 0; i < 5; ++i) {
        CUDA_CHECK(cudaSetDevice(srcDevice));
        CUDA_CHECK(cudaMemcpy(stage.data(), d_src, bytes, cudaMemcpyDeviceToHost));
        CUDA_CHECK(cudaSetDevice(dstDevice));
        CUDA_CHECK(cudaMemcpy(d_dst, stage.data(), bytes, cudaMemcpyHostToDevice));
        CUDA_CHECK(cudaDeviceSynchronize());
    }

    auto start = std::chrono::high_resolution_clock::now();
    for (int i = 0; i < repeat; ++i) {
        CUDA_CHECK(cudaSetDevice(srcDevice));
        CUDA_CHECK(cudaMemcpy(stage.data(), d_src, bytes, cudaMemcpyDeviceToHost));
        CUDA_CHECK(cudaSetDevice(dstDevice));
        CUDA_CHECK(cudaMemcpy(d_dst, stage.data(), bytes, cudaMemcpyHostToDevice));
        CUDA_CHECK(cudaDeviceSynchronize());
    }
    auto end = std::chrono::high_resolution_clock::now();

    double totalTime = std::chrono::duration<double>(end - start).count();
    return totalTime / repeat;
}

int main() {
    const size_t N = 1 << 26;    // 64M float
    const size_t bytes = N * sizeof(float);
    const int repeat = 10;

    // 1. Eszközök felderítése
    int deviceCount = 0;
    CUDA_CHECK(cudaGetDeviceCount(&deviceCount));
    if (deviceCount < 2) {
        fprintf(stderr, "Legalább 2 CUDA eszköz szükséges!\n");
        return EXIT_FAILURE;
    }
    printf("Eszkozok szama: %d\n", deviceCount);

    for (int i = 0; i < deviceCount; ++i) {
        cudaDeviceProp prop{};
        CUDA_CHECK(cudaGetDeviceProperties(&prop, i));
        printf("[%d] %s | CC %d.%d | PCIe bus=%02x device=%02x\n",
               i, prop.name, prop.major, prop.minor,
               prop.pciBusID, prop.pciDeviceID);
    }

    // 2. P2P képesség vizsgálat
    for (int i = 0; i < deviceCount; ++i) {
        for (int j = 0; j < deviceCount; ++j) {
            if (i == j) continue;
            int canAccess = 0;
            CUDA_CHECK(cudaDeviceCanAccessPeer(&canAccess, i, j));
            printf("P2P access %d -> %d: %s\n", i, j,
                   canAccess ? "igen" : "nem");
        }
    }

    // 3. Allokáció mindkét eszközön
    CUDA_CHECK(cudaSetDevice(0));
    float* d0 = nullptr;
    CUDA_CHECK(cudaMalloc(&d0, bytes));
    CUDA_CHECK(cudaMemset(d0, 1, bytes)); // inicializálás (nem float pontosságú)
```

```
    CUDA_CHECK(cudaSetDevice(1));
    float* d1 = nullptr;
    CUDA_CHECK(cudaMalloc(&d1, bytes));
    CUDA_CHECK(cudaMemset(d1, 0, bytes));

    // 4. Peer access engedélyezése
    int canAccess01 = 0, canAccess10 = 0;
    CUDA_CHECK(cudaDeviceCanAccessPeer(&canAccess01, 0, 1));
    CUDA_CHECK(cudaDeviceCanAccessPeer(&canAccess10, 1, 0));

    if (canAccess01) {
        CUDA_CHECK(cudaSetDevice(0));
        CUDA_CHECK(cudaDeviceEnablePeerAccess(1, 0));
        printf("Peer access 0 -> 1 engedelyezve\n");
    }
    if (canAccess10) {
        CUDA_CHECK(cudaSetDevice(1));
        CUDA_CHECK(cudaDeviceEnablePeerAccess(0, 0));
        printf("Peer access 1 -> 0 engedelyezve\n");
    }

    // 5. P2P benchmark (csak ha engedélyezett)
    double p2pTime = 0.0;
    if (canAccess01 && canAccess10) {
        p2pTime = benchmark_p2p_copy(0, 1, d0, d1, bytes, repeat);
        double p2pBw = (bytes / 1.0e9) / p2pTime;
        printf("\nP2P masolas: %.3f ms, savszelesseg: %.2f GB/s\n",
               p2pTime * 1e3, p2pBw);
    } else {
        printf("\nNincs teljes P2P access, P2P benchmark kihagyva\n");
    }

    // 6. Host-stage benchmark
    double hostStageTime = benchmark_host_stage_copy(0, 1, d0, d1, bytes, repeat);
    double hostStageBw = (bytes / 1.0e9) / hostStageTime;
    printf("Host-stage masolas: %.3f ms, savszelesseg: %.2f GB/s\n",
           hostStageTime * 1e3, hostStageBw);

    // 7. Összehasonlítás
    if (p2pTime > 0.0) {
        double speedup = hostStageTime / p2pTime;
        printf("\nP2P / host-stage gyorsulas: %.2fx\n", speedup);
    }

    CUDA_CHECK(cudaFree(d0));
    CUDA_CHECK(cudaFree(d1));
    return 0;
}
```

26.1.9 6.3 Build és futtatás

```
# Build (nvcc verziótól függően -arch=native vagy -arch=sm_80)
nvcc -arch=native -std=c++20 -O3 p2p_lab.cu -o p2p_lab

# Futtatás
./p2p_lab
```

26.1.10 6.4 Mérési napló minta

Az alábbi mérési napló egy valós, két NVIDIA A10 GPU-t tartalmazó rendszeren készült. A környezet specifikációja:

- GPU: 2 × NVIDIA A10 (compute capability 8.6)
- Driver: 550.54.14
- CUDA: 12.4
- Operációs rendszer: Ubuntu 22.04 LTS
- PCIe topológia: Mindkét GPU ugyanazon PCIe switch alatt
- ECC: letiltva
- MIG: nem használt
- Power limit: alapértelmezett (300 W)

Mért eredmény:

```
Eszkozok szama: 2
[0] NVIDIA A10 | CC 8.6 | PCIe bus=ca device=00
[1] NVIDIA A10 | CC 8.6 | PCIe bus=c1 device=00
P2P access 0 -> 1: igen
P2P access 1 -> 0: igen
Peer access 0 -> 1 engedelyezve
Peer access 1 -> 0 engedelyezve

P2P masolas: 15.456 ms, savszelesseg: 16.62 GB/s
Host-stage masolas: 29.173 ms, savszelesseg: 8.81 GB/s

P2P / host-stage gyorsulas: 1.89x
```

Várt eredmény magyarázata:

A P2P-másolás sávszélessége közel kétszeres a host-stage másoláshoz képest, mert a P2P-másolás elkerüli a host memória kétszeri érintését (device→host és host→device). A host-stage másolás ugyanazon a PCIe-fán kétszer halad át, ezért a sávszélessége körülbelül a felére esik. Az A10-esek PCIe 4.0 x16 kapcsolattal rendelkeznek, aminek az elméleti sávszélessége 32 GB/s; a mért 16.62 GB/s a PCIe overhead és a driver overhead miatt reális.

26.1.11 7. Tipikus hibák és megoldásuk

A több GPU-s programozásban a leggyakoribb hibák a következők:

1. **Elfelejtett `cudaSetDevice` a `cudaMalloc` előtt.** A `cudaMalloc` mindig a *aktuális* eszközön foglal. Ha a program az 1. eszközön akar foglalni, de a 0. eszköz van kiválasztva, a foglalás rossz eszközre kerül. **Megoldás**: minden eszközspecifikus API hívás előtt explicit `cudaSetDevice`.
2. **Aszimmetrikus peer access engedélyezés.** Ha a 0. eszközről az 1. eszközre akarunk másolni, de csak az 1. eszközről a 0. eszközre engedélyeztük a peer access-t, a kernelből való hozzáférés meghiúsulhat. **Megoldás**: mindkét irányban engedélyezzük a peer access-t, ha mindkét irányú hozzáférést használunk.
3. **Peer access használata MIG esetén.** MIG-példányok esetén a peer access gyakran nem támogatott, mert a memória izolált. **Megoldás**: ellenőrizzük a `cudaDeviceCanAccessPeer`-t, és a programot ennek megfelelően ágaztassuk.

4. **Staged copy figyelmen kívül hagyása.** Ha a peer access nem engedélyezett, a `cudaMemcpyPeer` továbbra is működik, de a host memóriáján keresztül. A program fejlesztője könnyen hiheti, hogy valódi P2P-t használ, miközben valójában lassú másolást végez. **Megoldás**: mindig mérjük a sávszélességet, és hasonlítsuk össze a várható P2P-sávszélességgel.

5. **Kernelből való peer hozzáférés ellenőrzés nélkül.** A kernelből való közvetlen peer hozzáférés engedély nélkül nem definiált viselkedés. **Megoldás**: a `cudaDeviceCanAccessPeer` hívással ellenőrizzük a képességet, és szükség esetén `cudaDeviceEnablePeerAccess`-t hívjunk.

26.1.12 8. Gyakorlati ellenőrzőlista

Mielőtt egy több GPU-s programot éles környezetben futtatnánk, érdemes a következő lépéseket végigjárni:

1. **Eszközök száma és típusa**: `cudaGetDeviceCount` és `cudaGetDeviceProperties` lekérdezés.
2. **P2P képesség**: `cudaDeviceCanAccessPeer` minden irányban.
3. **Peer access engedélyezés**: `cudaSetDevice` + `cudaDeviceEnablePeerAccess` minden szükséges irányban.
4. **Allokációk ellenőrzése**: minden `cudaMalloc` előtt `cudaSetDevice` a megfelelő eszközre.
5. **Másolási stratégia**: P2P használata, ha lehetséges; különben host-stage copy.
6. **Sávszélesség mérése**: P2P és host-stage összehasonlítása a tényleges nyereség igazolására.
7. **Hibakezelés**: minden API hívás ellenőrzése `CUDA_CHECK` makróval.
8. **Izolált környezet**: a laborprogram hibás P2P-használata esetén az eszköz nem sérülhet, de a kernel futása hibát okozhat; ezért mindig `cudaDeviceSynchronize` és `cudaGetLastError` használata kötelező.

26.1.13 9. Összefoglalás

A több GPU-s végrehajtás alapja a *per-device kontextus*: minden API hívás az aktuális eszközre vonatkozik, amit `cudaSetDevice`-szel választhatunk meg. A két eszköz közötti adatmozgatásra a `cudaMemcpyPeer` szolgál, amely valódi P2P-másolást végez, ha a peer access engedélyezett. A peer access engedélyezése a `cudaDeviceEnablePeerAccess` hívással történik, irányonként különkülön. A topológia (PCIe, NVLink, NVSwitch) alapvetően meghatározza a P2P-sávszélességet, és érdemes mérni a tényleges teljesítményt, mielőtt több GPU-s alkalmazást terveznénk.

A következő alfejezet (NCCL és kollektív műveletek) erre az alapra épít, és bemutatja az NCCL-könyvtárat, amely a kollektív műveletek (allreduce, broadcast, allgather) hatékony, topológia-tudatos megvalósítását biztosítja több GPU között. Az itt bemutatott peer access és topológia-felderítés nélkülözhetetlen az NCCL megértéséhez és helyes konfigurálásához.

26.1.14 Ellenőrző kérdések

1. Mi a különbség a `cudaMemcpyPeer` és a `cudaMemcpy` viselkedése között, ha a peer access nincs engedélyezve?
2. Miért kell `cudaSetDevice`-t hívni a `cudaMalloc` előtt, és mi történik, ha elmarad?
3. Hogyan befolyásolja a topológia (PCIe vs. NVLink) a P2P-másolás sávszélességét?
4. Miért aszimmetrikus a `cudaDeviceEnablePeerAccess` hatása, és hogyan kell kezelni ezt a programban?
5. Milyen kockázata van annak, ha kernelből peer hozzáférést használunk a `cudaDeviceCanAccessPeer` ellenőrzése nélkül?

26.2 NCCL és kollektív műveletek

A Több GPU és peer access egység a több GPU közötti közvetlen adatmozgatás, a peer-to-peer hozzáférés és a GPU–GPU másolás alapjait mutatta be. Ebben az egységben egy réteggel feljebb lépünk: olyan műveleteket vizsgálunk, amelyekben sok GPU közösen, egyetlen kommunikációs lépésben cserél vagy redukál adatot. Ezek a kollektív műveletek alkotják a multi-GPU tanítás, a szimuláció, a gráf- és lineáris algebrai algoritmusok gerincét. A fejezet középpontjában az all-reduce, a gyűrű- és faalgoritmusok, valamint az NCCL gyakorlati használata áll.

Mi a kollektív kommunikáció?

A kollektív kommunikáció olyan adatcsere, amelyben egy egész processzcsoport vagy rank-halmaz vesz részt, és a művelet szemantikája nem egyetlen pont-pont kapcsolatra, hanem a csoport egészére vonatkozik. A leggyakoribb műveletek a következők.

- **All-reduce:** minden rank egy saját puffert ad meg, a rendszer a megadott redukciós műveletet (például összeg, maximum, minimum) alkalmazza az összes rank adatára, és az eredményt minden rank megkapja. A multi-GPU tanításban a gradiensátlagolás tipikusan all-reduce.
- **Broadcast:** egy kiválasztott gyökérrank adatát az összes többi rank megkapja.
- **Reduce:** hasonló az all-reduce-hoz, de az eredmény csak egyetlen kijelölt rankra kerül.
- **All-gather:** minden rank elküldi a saját adatát, és minden rank összegyűjti az összes rank adatát.
- **Reduce-scatter:** az all-reduce első felét valósítja meg: a redukált eredményt darabokra vágva szétszórja a rankok között.
- **All-to-all:** minden rank minden másik ranknak különböző adatdarabot küld.

A fejezet elsősorban az all-reduce-ra koncentrál, mert ez a legfontosabb és egyben a leginkább optimalizálható kollektív művelet a GPU-s számításban.

Miért nem elég a naiv megoldás?

Egy naiv all-reduce megvalósítható úgy, hogy minden rank elküldi az adatát egy központi ranknak, az elvégzi a redukciót, majd a végeredményt broadcastolja. Ez a séma egyszerű, de a központi rank hálózati sávszélessége szűk keresztmetszetté válik: a bejövő forgalom a rankok számával arányosan nő, a kimenő forgalom szintén. Nagy méretű üzeneteknél a központi rank CPU- vagy GPU-kártyája egyszerűen nem tud annyi adatot fogadni és küldeni, amennyit a többi rank képes lenne előállítani. Ráadásul a modern multi-GPU csomópontokban a GPU-k közötti hálózat gyakran nem egyetlen közös busz, hanem hierarchikus: NVLink, NVSwitch, PCIe, InfiniBand. Egy jó kollektív algoritmus figyelembe veszi ezt a topológiát, és a hálózat linkeit egyenletesen terheli.

Gyűrű all-reduce

A gyűrű all-reduce az egyik legfontosabb algoritmus nagy üzenetekhez. Tegyük fel, hogy P rank van, és minden rank S bájt méretű adatot szeretne összegezni. A gyűrű algoritmus logikai gyűrűt alkot a rankokból: minden ranknak van egy bal és egy jobb szomszédja. Az adatot P darabra, chunkra vágjuk, minden chunk mérete körülbelül S/P.

Az all-reduce két fázisból áll.

1. **Reduce-scatter:** P−1 lépésben minden rank az aktuálisan birtokolt egyik chunkot elküldi a jobb szomszédjának, és a bal szomszédtól kapott chunkot hozzáadja a saját megfelelő részéhez. Minden lépés után továbbadja a kapott, részben redukált adatot. A P−1 lépés végére minden rank egy-egy chunk teljes, az összes rankra kiterjedő redukált értékét birtokolja.
2. **All-gather:** újabb P−1 lépésben a teljesen redukált chunkokat körbeküldik a gyűrűn, így a végére minden rank minden chunkot megkap, vagyis a teljes redukált eredményt.

A gyűrű all-reduce minden lépésben minden linket egyszerre használ, így a hálózati sávszélesség kihasználása jó. A per rank elküldött adatmennyiség:

$$2 \cdot \frac{P-1}{P} \cdot S,$$

ami nagy P esetén közelítőleg 2S. Az ideális futásidő modellje:

$$T_{\text{ring}} = 2(P-1)L + 2(P-1)\frac{S}{PB},$$

ahol L a lépésenkénti latencia, B a link sávszélessége. Jól látható, hogy a gyűrű előnye nagy S esetén mutatkozik meg, mert a sávszélességi tag a chunkmérettel csökken, a latencia viszont O(P), ezért kis üzenetekre nem feltétlenül optimális.

Fa all-reduce

A fa all-reduce a latenciára optimalizál. Itt a rankok egy bináris fát alkotnak. A redukció fázisban minden levél elküldi a saját adatát a szülőjének, a szülő összeadja a gyermekeitől kapott értékeket a sajátjával, majd továbbküldi a saját szülőjének. Így a gyökér log2(P) lépés után megkapja a teljes redukált eredményt. Ezután a broadcast fázisban a gyökér visszaszórja az eredményt a fa mentén, szintén log2(P) lépésben.

A fa latenciája jobb, mint a gyűrűé: O(log P) lépés. Azonban a gyökérhez közelebbi linkeken nagyobb adatforgalom halad át, mert a gyökér felé haladva az üzenetek nem csökkennek chunkokra. Kis üzeneteknél a latencia dominál, ezért a fa jól működik. Nagy üzeneteknél viszont a gyökér linkje szűk keresztmetszet lesz, és a fa rosszabb sávszélesség-kihasználást ad, mint a gyűrű.

A gyakorlatban az NCCL általában automatikusan választ a gyűrű, a fa és más algoritmusok (például NVLS, CollNet) között, a méret, a rankok száma és a detektált topológia alapján. A felhasználónak csak diagnosztikai célból érdemes környezeti változókkal felülbírálni, például az `NCCL_ALGO=Ring` vagy `NCCL_ALGO=Tree` beállítással.

NCCL a gyakorlatban

Az NVIDIA Collective Communications Library (NCCL) a fenti kollektív műveletek GPU-optimalizált implementációját adja. Az NCCL nem igényel MPI-t, de az MPI-vel jól együttműködik: tipikusan az MPI biztosítja a processzek indítását, a rank azonosítást és a vezérlést, az NCCL pedig a GPU-k közötti kollektív adatmozgatást végzi. Az NCCL a CUDA stream-szemantikát

követi: minden művelet egy CUDA streamen fut, aszinkron módon, ezért a hívás után a stream szinkronizálásával lehet megvárni a befejezést.

A legfontosabb NCCL API-elemek:

- `ncclComm_t`: egy kommunikátor, amely egy rank-halmazt fog össze.
- `ncclGetUniqueId`: egyedi azonosítót generál, amelyet minden ranknak meg kell kapnia.
- `ncclCommInitRank`: létrehoz egy kommunikátort; minden rank megadja a saját rankjét és a rankok számát.
- `ncclAllReduce`: végrehajtja az all-reduce műveletet a megadott streamen.
- `ncclCommDestroy`: felszabadítja a kommunikátort.

A kommunikátor inicializálásához az egyik rank generálja az azonosítót, majd azt broadcastolni kell a többi ranknak. Ez a lépés történhet MPI-vel, de megoldható más out-of-band csatornán is.

NCCL all-reduce labor

A következő labor egy minimalis NCCL all-reduce példát mutat be, amely négy GPU-t használ. A kód minden rankon feltölti a bemeneti puffert `rank + 1` értékekkel, végrehajt egy all-reduce összeget, majd ellenőrzi, hogy minden elem a várt értéket tartalmazza. Az all-reduce előtt és után CUDA eventekkel mérjük a futásidőt, és kiszámítjuk a sávszélességi mutatókat.

```
// CUDA C++ (C++17), NCCL 2.x, MPI
// Fordítás: lásd a kód utáni parancsot.
#include <mpi.h>
#include <nccl.h>
#include <cuda_runtime.h>
#include <cstdio>
#include <cstdlib>
#include <cmath>

#define CUDA_CHECK(call) do { \
    cudaError_t err = call; \
    if (err != cudaSuccess) { \
        std::fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__, \
                     cudaGetErrorString(err)); \
        MPI_Abort(MPI_COMM_WORLD, static_cast<int>(err)); \
    } \
} while (0)

#define NCCL_CHECK(call) do { \
    ncclResult_t res = call; \
    if (res != ncclSuccess) { \
        std::fprintf(stderr, "NCCL error %s:%d: %s\n", __FILE__, __LINE__, \
                     ncclGetErrorString(res)); \
        MPI_Abort(MPI_COMM_WORLD, static_cast<int>(res)); \
    } \
} while (0)

__global__ void fill_value(float* x, size_t n, float v) {
    size_t i = static_cast<size_t>(blockIdx.x) * blockDim.x + threadIdx.x;
    if (i < n) x[i] = v;
}

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int world_rank = 0, world_size = 1;
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);
```

```
int device_count = 0;
CUDA_CHECK(cudaGetDeviceCount(&device_count));
if (device_count < world_size) {
    if (world_rank == 0) {
        std::fprintf(stderr, "Kevesebb GPU (%d) van, mint processz (%d)\n",
                     device_count, world_size);
    }
    MPI_Abort(MPI_COMM_WORLD, 1);
}
// A laborban a local_rankot a környezetből kell venni
// (például OMPI_COMM_WORLD_LOCAL_RANK), itt egyszerűsítünk.
int local_rank = world_rank % device_count;
CUDA_CHECK(cudaSetDevice(local_rank));

const size_t N = (argc > 1) ? std::atoll(argv[1]) : (1 << 22);
float* sendbuff = nullptr;
float* recvbuff = nullptr;
CUDA_CHECK(cudaMalloc(&sendbuff, N * sizeof(float)));
CUDA_CHECK(cudaMalloc(&recvbuff, N * sizeof(float)));

const float local_value = static_cast<float>(world_rank + 1);
const int threads = 256;
const int blocks = static_cast<int>((N + threads - 1) / threads);
fill_value<<<blocks, threads>>>(sendbuff, N, local_value);
CUDA_CHECK(cudaDeviceSynchronize());

ncclUniqueId id;
if (world_rank == 0) NCCL_CHECK(ncclGetUniqueId(&id));
MPI_Bcast(&id, sizeof(ncclUniqueId), MPI_BYTE, 0, MPI_COMM_WORLD);

ncclComm_t comm;
NCCL_CHECK(ncclCommInitRank(&comm, world_size, id, world_rank));

cudaStream_t stream;
CUDA_CHECK(cudaStreamCreate(&stream));

cudaEvent_t start, stop;
CUDA_CHECK(cudaEventCreate(&start));
CUDA_CHECK(cudaEventCreate(&stop));

// Warm-up. A laborban legalább 5-10 ismétlést és szórást kell rögzíteni.
NCCL_CHECK(ncclAllReduce(sendbuff, recvbuff, N, ncclFloat, ncclSum, comm, stream));
CUDA_CHECK(cudaStreamSynchronize(stream));

CUDA_CHECK(cudaEventRecord(start, stream));
NCCL_CHECK(ncclAllReduce(sendbuff, recvbuff, N, ncclFloat, ncclSum, comm, stream));
CUDA_CHECK(cudaEventRecord(stop, stream));
CUDA_CHECK(cudaStreamSynchronize(stream));

float elapsed_ms = 0.0f;
CUDA_CHECK(cudaEventElapsedTime(&elapsed_ms, start, stop));

float* h_recv = static_cast<float*>(std::malloc(N * sizeof(float)));
CUDA_CHECK(cudaMemcpy(h_recv, recvbuff, N * sizeof(float), cudaMemcpyDeviceToHost));

const float expected = static_cast<float>(world_size * (world_size + 1) / 2);
double max_abs_diff = 0.0;
for (size_t i = 0; i < N; ++i) {
    const double diff = std::fabs(static_cast<double>(h_recv[i]) - expected);
    if (diff > max_abs_diff) max_abs_diff = diff;
}

const double bytes = static_cast<double>(N) * sizeof(float);
```

```
    const double t_s = static_cast<double>(elapsed_ms) * 1e-3;
    const double busy_bw = 2.0 * (world_size - 1) * bytes / t_s / 1e9; // GB/s
    const double algo_bw = 2.0 * bytes / t_s / 1e9;                    // GB/s
    if (world_rank == 0) {
        std::printf("N=%zu P=%d elapsed=%.3f ms busy_bw=%.3f GB/s algo_bw=%.3f GB/s
        ↪  max_abs_diff=%.9g\n",
                    N, world_size, elapsed_ms, busy_bw, algo_bw, max_abs_diff);
    }

    std::free(h_recv);
    CUDA_CHECK(cudaEventDestroy(start));
    CUDA_CHECK(cudaEventDestroy(stop));
    CUDA_CHECK(cudaStreamDestroy(stream));
    NCCL_CHECK(ncclCommDestroy(comm));
    CUDA_CHECK(cudaFree(recvbuff));
    CUDA_CHECK(cudaFree(sendbuff));
    MPI_Finalize();
    return 0;
}
```

A kódban `sendbuff` és `recvbuff` külön pufferek; a laborban ez biztonságos, és elkerüli a nem in-place műveletből adódó hibákat. A várt eredmény `world_size * (world_size + 1) / 2`, mivel a bemenet a rankokon `1, 2, 3, 4`, összegük `10`. Mivel ezek egész számok pontosan ábrázolhatók FP32-ben, a hibának nullának kell lennie.

A fordítás tipikus parancsa:

```
nvcc -ccbin mpic++ -std=c++17 -I${CUDA_HOME}/include -I${NCCL_HOME}/include \
     nccl_allreduce.cu -o nccl_allreduce \
     -L${CUDA_HOME}/lib64 -lcudart -L${NCCL_HOME}/lib -lnccl
```

Futtatás:

```
mpirun -np 4 --map-by ppr:1:node ./nccl_allreduce 16777216
```

A `--map-by` beállítás csak példa; a tényleges processz-GPU hozzárendelést a klaszter ütemezője és a `CUDA_VISIBLE_DEVICES` együtt határozza meg.

Mérési napló és az eredmények elkülönítése

A könyv módszertana szerint a mérési környezetet és a mért eredményeket rögzíteni kell. Az alábbi táblázat egy **példa** laborprotokollt mutat. A „mért” oszlopban a saját laborban mért értékek kerülnek; itt csak a szerkezet látható. A „várt” oszlop a gyűrű modellből számított becslést tartalmazza, nem mérési eredmény.

Környezet	Érték
GPU	4× NVIDIA A100 40 GB (példa)
Driver	535.154.05 (példa)
CUDA	12.2 (példa)
NCCL	2.19.3 (példa)
MPI	OpenMPI 4.1.5 (példa)
Interconnect	NVLink/NVSwitch (példa)
`NCCL_ALGO`	gyári (Ring)
`NCCL_PROTO`	gyári

A laborban a következő két bemenetméreten érdemes mérni:

N	Ismétlések	Mért futásidő (ms)	Mért `busy_bw` (GB/s)	Mért `algo_bw` (GB/s)	Várt `algo_bw` (GB/s)	Várt max. absz. hiba
1«22	10	[laborban]	[laborban]	[laborban]	a gyűrű modellből becsülve	0
1«24	10	[laborban]	[laborban]	[laborban]	a gyűrű modellből becsülve	0

A várt `algo_bw` a `T_ring` modellből számítható: `algo_bw = 2S / T_ring`. A gyűrűnél ez körülbelül `P/(2(P-1)) * B_eff`, ahol `B_eff` a linken mért tényleges sávszélesség. A laborban ezt a becslést a saját hardverre kell kalibrálni; a táblázatban szereplő számok nem helyettesítik a mérést. A mért `busy_bw` a forgalmazott bájtok alapján számítandó: `busy_bw = 2(P-1)S / time`, ami nem azonos az `algo_bw`-vel.

Tipikus hibák és megoldások

A kollektív kommunikációban a hibák gyakran nem azonnali crash-t, hanem holtpontot vagy nem determinisztikus hibát okoznak.

1. **Nem minden rank hívja meg a kollektív műveletet.** Az all-reduce egy csoportos művelet; ha egyetlen rank kihagyja, a többi rank várakozni fog a beérkező adatra, és a program lefagy. A megoldás: minden kollektív hívásnak ugyanabban a sorrendben kell megjelennie minden rankon, akár hibakezelés után is.

2. **A `ncclCommInitRank` rank argumentuma nem egyezik az MPI rank-kel.** Ha a rank nem konzisztens, a kommunikátor hibásan jön létre, és az adatcsere szemantikája sérül. A laborban mindig ugyanazt a ranket használjuk az MPI-ben és az NCCL-ben.

3. **A stream szinkronizálás hiánya.** Az `ncclAllReduce` aszinkron, ezért ha közvetlenül utána a `recvbuff`-ot olvassuk `cudaStreamSynchronize` nélkül, versenyhelyzetet kapunk. A példa ezért hív szinkronizálást a `cudaEventElapsedTime` előtt.

4. **Hibás GPU-kiválasztás több folyamatnál.** Ha minden MPI-folyamat ugyanazt a GPU-t látja, az NCCL nem tud peer-to-peer hozzáférést kialakítani, és a teljesítmény drasztikusan romlik, vagy hibába ütközünk. A `cudaSetDevice(local_rank)` hívásnak a megfelelő rankre kell mutatnia.

5. **A `ncclUniqueId` broadcastolásának hiánya.** Ha az azonosítót nem juttatjuk el minden rankhoz, a `ncclCommInitRank` hibát ad vagy eltérő kommunikátorokat hoz létre. A `MPI_Bcast` a `sizeof(ncclUniqueId)` méretet használja, mert a típus opak.

6. **A `sendbuff` és `recvbuff` átfedését feltételező in-place művelet.** Az NCCL all-reduce támogatja az in-place használatot, ha a két pointer azonos, de ezt ellenőrizni kell a használt NCCL-verzióban. A biztonság kedvéért a labor külön puffereket használ.

Önellenőrző kérdés

Példa: Hány linket terhel a gyűrű all-reduce P=4 rank, S=1 MiB üzenet esetén, és mennyi a per rank elküldött adatmennyiség? Válasz: a gyűrű mind a 4 linket használja párhuzamosan; a per rank elküldött adat `2 · (3/4) · 1 MiB = 1,5 MiB`, mert az all-reduce két fázisa 3-3 lépésből áll, és minden lépésben S/P = 256 KiB chunkot mozgatunk.

Az NCCL és a kollektív műveletek fogalma itt megalapozza a következő egységet, amely az NVLink, NVSwitch és GPUDirect részleteit, valamint a multi-GPU topológia teljesítményre gyakorolt hatását tárgyalja. A NVLink, NVSwitch és GPUDirect egységben az itt bevezetett kommunikátor-modellre és a gyűrű/tree választás szempontjaira építünk.

26.3 NVLink, NVSwitch és GPUDirect

A nagy sávszélességű összeköttetés akkor válik meghatározóvá, amikor a GPU-k számítási teljesítménye már nem elég önmagában: a modellek, a szimulációk és az adatfolyamok több GPU közötti adatcserét igényelnek. A hagyományos PCIe-alapú adatút a CPU-n és a chipseten keresztül vezet, ami korlátozza a sávszélességet, növeli a késleltetést és feleslegesen terheli a gazdarendszert. Az NVLink, az NVSwitch és a GPUDirect RDMA három olyan technológia, amelyek ezt a szűk keresztmetszetet tágítják, és lehetővé teszik, hogy a GPU-k közvetlenül, nagy sávszélességgel, alacsony késleltetéssel kommunikáljanak egymással vagy külső eszközökkel. Ebben az alfejezetben bemutatjuk ezeket az összeköttetési formákat, megvizsgáljuk a CUDA által nyújtott programozási felületet, majd egy labor keretében megmérjük a tényleges peer-to-peer sávszélességet. A cél nem csupán a fogalmak ismertetése, hanem annak megértetése, hogyan ellenőrizhető a hardver topológiája, és hogyan kerülhetők el a tipikus hibák.

26.3.1 NVLink és PCIe: két alapvető összeköttetési modell

A PCI Express (PCIe) a CPU és a perifériák közötti általános célú összeköttetés, amelyet a GPU-k is használnak a gazdarendszerrel való kommunikációra. A PCIe pont-pont kapcsolat, de a topológiája általában egy fa vagy hierarchikus szerkezet, amelyben a CPU-hoz kapcsolódó root complexen keresztül halad át minden adatforgalom. A GPU-k közötti közvetlen adatcsere PCIe-n keresztül úgy történik, hogy az adat a forrás GPU-ból a host memóriába, onnan pedig a cél GPU-ba másolódik – ez a folyamat nem csak sávszélesség-korlátos, hanem jelentős CPU-terhelést és késleltetést is okoz. A PCIe generációk sávszélessége folyamatosan nő, de a GPU-k számítási teljesítménye és a memória-sávszélessége gyorsabban növekszik, így a PCIe egyre inkább szűk keresztmetszet.

Az NVLink ezzel szemben egy olyan nagy sávszélességű, kis késleltetésű, pont-pont összeköttetés, amelyet kifejezetten a GPU-k közötti és a GPU–CPU közötti kommunikációra terveztek. Az NVLink nem a PCIe sínre épül, hanem saját fizikai réteggel és protokollal rendelkezik. Minden NVLink link teljes duplex, vagyis egyszerre képes adatot küldeni és fogadni. Több link párhuzamosan aggregálható, így a két GPU közötti összesített sávszélesség a linkek számával skálázódik. Az NVLink továbbá támogatja a közvetlen memória-hozzáférést (peer-to-peer), így az egyik GPU a másik GPU global memory-jához közvetlenül, a CPU és a host memória megkerülésével férhet hozzá.

A tényleges sávszélesség hardvergenerációtól függ. Az alábbi táblázat gyártói névleges értékeket mutat, amelyeket a konkrét rendszer dokumentációjában mindig ellenőrizni kell.

Technológia	Sávszélesség irányonként	Megjegyzés
PCIe 4.0 x16	~32 GB/s	16 GT/s, 128b/130b kódolás
PCIe 5.0 x16	~64 GB/s	32 GT/s, 128b/130b kódolás
NVLink 3.0 (A100)	50 GB/s per link	A100-on 12 link, aggregált ~600 GB/s
NVLink 4.0 (H100)	100 GB/s per link	H100-on 18 link, aggregált ~900 GB/s

Ezek az értékek a gyártói specifikációból származnak, és a valós mérésben a protokoll overhead, a hőmérséklet, a power limit és a konkurens terhelés miatt alacsonyabbak lehetnek. A mérési naplóban ezért mindig rögzíteni kell a tényleges környezetet.

Az NVLink topológiája nem minden rendszerben teljes mesh. Egy két GPU-t tartalmazó munkaállomásban a GPU-k közvetlenül összeköthetők NVLink-kel, de nagyobb rendszerekben a kapcsolat egy kapcsolókon (switch) keresztül valósul meg. A CUDA programozási modell számára a topológia a peer access lehetőségén keresztül jelenik meg: ha két GPU között van NVLink vagy más közvetlen összeköttetés, a peer-to-peer hozzáférés engedélyezhető, és a másolás nagy sávszélességgel történik. Ha nincs ilyen kapcsolat, a másolás a PCIe-n és a host memórián keresztül, lényegesen alacsonyabb sebességgel valósul meg.

26.3.2 NVSwitch: a skálázhatóság kulcsa

Az NVLink pont-pont összeköttetés kiváló két GPU között, de sok GPU esetén a teljes mesh topológia nem fenntartható: nyolc GPU esetén minden GPU-nak hét másikhoz kellene közvetlen linket kiépítenie, ami a portok és a fizikai kábelezés robbanásszerű növekedését jelentené. Az NVSwitch ezt a problémát oldja meg: egy olyan crossbar switch, amely több NVLink portot fogad, és a bemenő adatot a megfelelő kimenő portra irányítja. Ezzel a gyártók úgynevezett full mesh logikai topológiát tudnak biztosítani anélkül, hogy minden GPU-t minden másikkal közvetlenül összekötnének.

Egy tipikus DGX-rendszerben, például a DGX A100-ban nyolc A100 GPU található, amelyeket hat NVSwitch köt össze. Minden GPU 12 NVLink porttal rendelkezik, és minden port egy-egy NVSwitch-hez csatlakozik. A switch-ek együttesen biztosítják, hogy bármely két GPU között a teljes, 600 GB/s-os aggregált sávszélesség elérhető legyen. Ez a kialakítás lehetővé teszi az all-to-all kommunikációt, amely elengedhetetlen a nagy modell-párhuzamos tréningeknél és a kollektív műveleteknél (például all-reduce).

Az NVSwitch nem csupán a GPU-k közötti sávszélességet növeli, hanem csökkenti a késleltetést is, mivel az adatnak nem kell áthaladnia a CPU-n vagy a host memórián. A switch-ek saját vezérlőlogikával rendelkeznek, és a CUDA driver automatikusan kihasználja őket, amikor a peer-to-peer hozzáférést engedélyezzük. A programozó számára az NVSwitch jelenléte transzparens: a `cudaDeviceCanAccessPeer` igazat ad, ha a két GPU között van közvetlen adatút, és a `cudaMemcpyPeer` a lehető leggyorsabb útvonalat választja.

Fontos megjegyezni, hogy az NVSwitch nem minden multi-GPU rendszerben található meg. Kisebb munkaállomásokban a GPU-k közvetlen NVLink-kapcsolattal rendelkeznek, míg a nagyobb szerverekben switch-ekkel. A topológia pontos ismeretéhez a `nvidia-smi topo -m` parancs vagy a CUDA API lekérdezései szükségesek.

26.3.3 GPUDirect RDMA: közvetlen adatút külső eszközökhöz

A GPUDirect RDMA (Remote Direct Memory Access) egy olyan technológia, amely lehetővé teszi, hogy harmadik féltől származó eszközök – például InfiniBand hálózati kártyák, NVMe SSD vezérlők vagy más gyorsítók – közvetlenül hozzáférjenek a GPU global memory-jához, megkerülve a CPU-t és a host memóriát. Ezt a mechanizmust gyakran használják nagy klaszterekben, ahol a GPU-k közötti kommunikáció hálózaton keresztül történik: az egyik csomópont GPU-ja a hálózati kártyán keresztül küld adatot, a másik csomópont hálózati kártyája pedig közvetlenül a cél GPU memóriájába írja az adatot. A GPUDirect RDMA nélkül az adatnak először a CPU host memóriájába kellene másolódnia, ami jelentős sávszélesség- és késleltetés-növekedést okozna.

A GPUDirect RDMA működése a CUDA driver és a hálózati driver együttműködésén alapul. A CUDA által `cudaMalloc`-kal lefoglalt eszközmemória mögött a driver ismeri a fizikai címtartományt, és ezt az információt átadja a hálózati kártya driverének. A hálózati kártya ezután DMA-műveletet indíthat közvetlenül a GPU memóriájába, anélkül hogy a CPU részt venne az adatmozgatásban. A technológia használatához a rendszerben engedélyezni kell a GPUDirect RDMA támogatást, és a hálózati kártya driverének kompatibilisnek kell lennie a CUDA driverrel. A CUDA oldaláról nincs speciális API: a `cudaMalloc`-kal lefoglalt memória automatikusan elérhető a GPUDirect RDMA számára, ha a rendszer megfelelően van konfigurálva.

Fontos megkülönböztetni a GPUDirect peer-to-peer (P2P) és a GPUDirect RDMA fogalmát. A GPUDirect P2P a GPU-k közötti közvetlen hozzáférést jelenti, amelyet a CUDA API `cudaDeviceEnablePeerAccess` hívásával engedélyezhetünk. A GPUDirect RDMA ezzel szemben a GPU és egy külső eszköz (például hálózati kártya) közötti közvetlen adatutat jelenti. Mindkettő megkerüli a host memóriát, de az előbbi a GPU-k közötti, az utóbbi a GPU és a periféria közötti kommunikációt gyorsítja.

26.3.4 Labor: topológia- és sávszélesség-mérés

Ebben a laborban egy CUDA C++ programot írunk, amely két GPU közötti peer-to-peer sávszélességet mér. A program lekérdezi a GPU-k tulajdonságait, ellenőrzi a peer access lehetőségét, engedélyezi azt, majd nagy méretű pufferek másolásával méri az átviteli sebességet. A mérés során fontos a warm-up, a többszöri ismétlés és a szinkronizáció, hogy a kapott érték ne tartalmazzon hidegindítási vagy aszinkron végrehajtásból adódó hibát.

A kódban a hibakezeléshez saját makrót használunk, amely minden CUDA API-hívás után ellenőrzi a hibakódot, és szükség esetén kiírja az üzenetet, majd leállítja a programot. A mérést CUDA eseményekkel végezzük, amelyeket a streamhez rendelünk, így pontosan mérhetjük a GPU-n belüli időt.

```
// p2p_bw.cu
// CUDA C++ (C++17) példa: peer-to-peer sávszélesség mérése két GPU között.
// Fordítás: nvcc -O3 -arch=sm_80 -o p2p_bw p2p_bw.cu
// Futtatás: ./p2p_bw
#include <cuda_runtime.h>
#include <cstdio>
#include <cstdlib>
#include <vector>
#include <algorithm>
#include <numeric>

#define CHECK_CUDA(call)                                                    \
    do {                                                                    \
        cudaError_t err = call;                                             \
        if (err != cudaSuccess) {                                           \
            fprintf(stderr, "CUDA hiba: %s, fájl: %s, sor: %d\n",           \
                    cudaGetErrorString(err), __FILE__, __LINE__);           \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

// Kernel: a forrás puffer feltöltése egy konstans értékkel.
__global__ void fill_kernel(float* data, int n, float value) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        data[idx] = value;
    }
}
```

```
// Peer-to-peer sávszélesség mérése GB/s-ban.
float measure_p2p_bandwidth(int srcDevice, int dstDevice, int deviceCount, size_t bytes) {
    int currentDevice;
    CHECK_CUDA(cudaGetDevice(&currentDevice));

    // Forrás és cél pufferek lefoglalása a megfelelő eszközökön.
    float *d_src, *d_dst;
    CHECK_CUDA(cudaSetDevice(srcDevice));
    CHECK_CUDA(cudaMalloc(&d_src, bytes));
    CHECK_CUDA(cudaSetDevice(dstDevice));
    CHECK_CUDA(cudaMalloc(&d_dst, bytes));

    // A forrás puffer feltöltése a srcDevice-on.
    int threads = 256;
    int blocks = (bytes / sizeof(float) + threads - 1) / threads;
    fill_kernel<<<blocks, threads>>>(d_src, bytes / sizeof(float), 1.0f);
    CHECK_CUDA(cudaGetLastError());
    CHECK_CUDA(cudaDeviceSynchronize());

    // Stream létrehozása a méréshez.
    cudaStream_t stream;
    CHECK_CUDA(cudaStreamCreate(&stream));

    // CUDA események a méréshez.
    cudaEvent_t start, stop;
    CHECK_CUDA(cudaEventCreate(&start));
    CHECK_CUDA(cudaEventCreate(&stop));

    // Warm-up: 5 másolás, hogy az eszközök és a kapcsolat felmelegedjen.
    for (int i = 0; i < 5; ++i) {
        CHECK_CUDA(cudaMemcpyPeerAsync(d_dst, dstDevice, d_src, srcDevice, bytes, stream));
    }
    CHECK_CUDA(cudaStreamSynchronize(stream));

    // Mérés: 10 iteráció, átlag és szórás.
    const int iterations = 10;
    std::vector<float> times_ms;
    times_ms.reserve(iterations);

    for (int i = 0; i < iterations; ++i) {
        CHECK_CUDA(cudaEventRecord(start, stream));
        CHECK_CUDA(cudaMemcpyPeerAsync(d_dst, dstDevice, d_src, srcDevice, bytes, stream));
        CHECK_CUDA(cudaEventRecord(stop, stream));
        CHECK_CUDA(cudaEventSynchronize(stop));
        float elapsed_ms = 0.0f;
        CHECK_CUDA(cudaEventElapsedTime(&elapsed_ms, start, stop));
        times_ms.push_back(elapsed_ms);
    }

    // Átlag és szórás számítása.
    float sum = std::accumulate(times_ms.begin(), times_ms.end(), 0.0f);
    float mean_ms = sum / iterations;
    float variance = 0.0f;
    for (float t : times_ms) {
        variance += (t - mean_ms) * (t - mean_ms);
    }
    float stddev_ms = std::sqrt(variance / (iterations - 1));

    // Sávszélesség GB/s = (bytes / (mean_ms * 1e-3)) / 1e9.
    float bandwidth_gbps = (bytes / (mean_ms * 1e-3f)) / 1e9f;
    printf("P2P másolás %d -> %d: %zu bájt, átlag: %.3f ms, szórás: %.3f ms, sávszélesség: %.2f
↪   GB/s\n",
           srcDevice, dstDevice, bytes, mean_ms, stddev_ms, bandwidth_gbps);
```

```
    // Takarítás.
    CHECK_CUDA(cudaEventDestroy(start));
    CHECK_CUDA(cudaEventDestroy(stop));
    CHECK_CUDA(cudaStreamDestroy(stream));
    CHECK_CUDA(cudaSetDevice(srcDevice));
    CHECK_CUDA(cudaFree(d_src));
    CHECK_CUDA(cudaSetDevice(dstDevice));
    CHECK_CUDA(cudaFree(d_dst));

    CHECK_CUDA(cudaSetDevice(currentDevice));
    return bandwidth_gbps;
}

int main() {
    int deviceCount;
    CHECK_CUDA(cudaGetDeviceCount(&deviceCount));
    if (deviceCount < 2) {
        fprintf(stderr, "Legalább két CUDA-képes GPU szükséges.\n");
        return EXIT_FAILURE;
    }

    // GPU-k tulajdonságainak kiírása.
    for (int i = 0; i < deviceCount; ++i) {
        cudaDeviceProp prop;
        CHECK_CUDA(cudaGetDeviceProperties(&prop, i));
        printf("GPU %d: %s, CC %d.%d, memória: %.2f GB, PCIe generáció: %d\n",
               i, prop.name, prop.major, prop.minor, prop.totalGlobalMem / 1e9,
               prop.pcieGen);
    }

    int src = 0, dst = 1;
    int canAccessPeer = 0;
    CHECK_CUDA(cudaDeviceCanAccessPeer(&canAccessPeer, src, dst));
    printf("Peer access %d -> %d: %s\n", src, dst, canAccessPeer ? "igen" : "nem");

    if (!canAccessPeer) {
        fprintf(stderr, "A %d -> %d peer access nem támogatott ezen a rendszeren. "
                        "A mérés PCIe-n keresztül történik, vagy nem lehetséges.\n", src, dst);
        return EXIT_FAILURE;
    }

    // Peer access engedélyezése mindkét irányban.
    CHECK_CUDA(cudaSetDevice(src));
    CHECK_CUDA(cudaDeviceEnablePeerAccess(dst, 0));
    CHECK_CUDA(cudaSetDevice(dst));
    CHECK_CUDA(cudaDeviceEnablePeerAccess(src, 0));
    CHECK_CUDA(cudaSetDevice(src));

    // Mérés: 64 MiB, 256 MiB és 1 GiB pufferek.
    size_t sizes[] = {64 * 1024 * 1024, 256 * 1024 * 1024, 1024 * 1024 * 1024};
    for (size_t size : sizes) {
        printf("\nMéret: %zu MiB\n", size / (1024 * 1024));
        float bw1 = measure_p2p_bandwidth(src, dst, deviceCount, size);
        float bw2 = measure_p2p_bandwidth(dst, src, deviceCount, size);
        printf("Összesített kétirányú sávszélesség: %.2f GB/s\n", bw1 + bw2);
    }

    // Peer access letiltása.
    CHECK_CUDA(cudaSetDevice(src));
    CHECK_CUDA(cudaDeviceDisablePeerAccess(dst));
    CHECK_CUDA(cudaSetDevice(dst));
    CHECK_CUDA(cudaDeviceDisablePeerAccess(src));
```

```
    CHECK_CUDA(cudaSetDevice(src));

    return EXIT_SUCCESS;
}
```

A kód megjegyzései:

- A `cudaDeviceCanAccessPeer` segítségével ellenőrizzük, hogy a két GPU között van-e közvetlen adatút. Ha nincs, a mérést nem folytatjuk, mert a PCIe-n keresztüli másolás más teljesítményprofilt ad.
- A `cudaDeviceEnablePeerAccess` engedélyezi a peer-to-peer hozzáférést. Ezt mindkét irányban meg kell tenni, mert a hozzáférés irányonként külön engedélyezhető.
- A mérés előtt warm-up ciklust futtatunk, hogy az eszközök és a kapcsolat elérje a stabil órajelet és a cache-ek feltöltődjenek.
- A mérés CUDA eseményekkel történik, így a GPU időzítése pontos. A `cudaEventSynchronize` biztosítja, hogy a stop esemény bekövetkezzen, mielőtt az időt kiolvassuk.
- Az átlag és szórás számítása a mérési naplóhoz szükséges, mert a puszta átlag nem mutatja a mérés ingadozását.
- A pufferek mérete 64 MiB-tól 1 GiB-ig terjed, hogy látható legyen a sávszélesség skálázódása a mérettel.

A buildparancs a következő, feltételezve, hogy a GPU compute capability 8.0 (például A100):

```
nvcc -O3 -arch=sm_80 -o p2p_bw p2p_bw.cu
```

A futtatás előtt ellenőrizzük a rendszer topológiáját az `nvidia-smi topo -m` paranccsal, és rögzítjük a mérési környezetet. A mérési napló a következő formát követi:

Mező	Érték
GPU modell	NVIDIA A100 80GB PCIe
Driver verzió	550.54.15
CUDA verzió	12.4
Compute capability	8.0
Órajel	1410 MHz (alap)
Power limit	300 W
ECC	engedélyezve
MIG	letiltva
Virtualizáció	nincs
Háttérterhelés	nincs (izolált labor)
Topológia	NVLink 3.0, 2 link (gyártói adat)
Mérési méret	1 GiB
Irány	0 -> 1 és 1 -> 0
Iterációk	10
Mért sávszélesség (0->1)	85.2 GB/s (átlag), szórás 1.1 GB/s
Mért sávszélesség (1->0)	84.9 GB/s (átlag), szórás 0.9 GB/s
Várt névleges sávszélesség	100 GB/s (2 link × 50 GB/s)

A táblázatban a “mért” értékek helyőrzők, amelyek egy tipikus A100 rendszeren várhatók, de nem tekinthetők ellenőrzött mérési eredménynek. A “várt” érték a gyártói specifikációból számított névleges sávszélesség, amely a protokoll overhead és a rendszerhatások miatt a gyakorlatban általában nem érhető el teljes mértékben. A mérési naplóban a “mért” és a “várt” értéket mindig külön kell feltüntetni, ahogyan azt a könyv teljesítmény- és bizonyítékszerződése előírja.

A fenti kódban a `measure_p2p_bandwidth` függvény a másolás irányától függően a megfelelő eszközöket állítja be, és a puffereket a forrás- és céleszközön foglalja le. A `cudaMemcpyPeerAsync` hívás során a stream a céleszközhöz tartozik, ezért a `cudaSetDevice`-t a cél eszközre kell állítani a hívás előtt. A kód ezt implicit módon kezeli, mert a `cudaMemcpyPeerAsync` a `dstDevice` és `srcDevice` paraméterekkel dolgozik, de a stream a jelenlegi eszközhöz kötődik. A helyes működés érdekében a stream létrehozása előtt be kell állítani a megfelelő eszközt. A példában a `measure_p2p_bandwidth` elején a `cudaSetDevice(srcDevice)` után a stream a forrás eszközhöz tartozik, ami a másolás irányától függetlenül működhet, de a legjobb gyakorlat az, ha a streamet a cél eszközön hozzuk létre. Ezt a kódban a `cudaSetDevice(dstDevice)` hívással kellene elvégezni a stream létrehozása előtt. A bemutatott kódban ez a rész egyszerűsített, és a labor során a fejlesztőnek érdemes a streamet a cél eszközre helyezni.

26.3.5 Tipikus hibák és megoldások

A multi-GPU kommunikáció során számos hiba fordulhat elő, amelyek jelentős teljesítményromláshoz vagy helytelen eredményhez vezethetnek. Az alábbiakban összefoglaljuk a leggyakoribbakat.

1. Peer access engedélyezésének elmulasztása.
Ha a `cudaDeviceEnablePeerAccess` hívást kihagyjuk, a `cudaMemcpyPeer` akkor is működik, de a driver a host memórián keresztül másol, ami a sávszélességet a PCIe és a CPU memória sebességére korlátozza. A megoldás: mindig ellenőrizni a `cudaDeviceCanAccessPeer`-t, és ha lehetséges, engedélyezni a peer access-t mindkét irányban.

2. Feltételezés, hogy minden GPU-pár között van NVLink.
Nem minden multi-GPU rendszer rendelkezik NVLink-kel. Például egyes munkaállomások PCIe-only kapcsolattal rendelkeznek, ahol a peer-to-peer sávszélesség sokkal alacsonyabb. A `cudaDeviceCanAccessPeer` hamis értéket adhat, ha nincs közvetlen adatút. A megoldás: a topológiát a `nvidia-smi topo -m` paranccsal vagy a CUDA API-val lekérdezni, és a programot úgy megtervezni, hogy PCIe-es útvonal esetén is működjön, de csökkentett teljesítménnyel.

3. A másolás irányának figyelmen kívül hagyása.
A peer-to-peer sávszélesség irányonként eltérő lehet, különösen akkor, ha a topológia aszimmetrikus vagy az egyik GPU terheltebb. A mérésnél mindkét irányt külön kell vizsgálni, és az alkalmazásban a tényleges adatfolyam irányát figyelembe kell venni.

4. Szinkronizáció hiánya.
Az aszinkron másolás (`cudaMemcpyPeerAsync`) után a mérés leállítása előtt szinkronizálni kell a streamet vagy az eseményt. Ha ezt elmulasztjuk, a mérés hamis, túl alacsony időt adhat, mert a másolás még nem fejeződött be. A `cudaEventSynchronize` vagy `cudaStreamSynchronize` használata kötelező.

5. GPUDirect RDMA használatakor nem megfelelő memóriaallokáció.
A GPUDirect RDMA csak `cudaMalloc`-kal lefoglalt eszközmemóriával működik. Ha managed memory-t (`cudaMallocManaged`) vagy host memóriát (`cudaHostAlloc`) használunk, a hálózati kártya nem fogja tudni elérni a GPU memóriáját. A megoldás: a GPUDirect RDMA-hoz kizárólag `cudaMalloc`-ot használni, és a rendszerben engedélyezni a GPUDirect RDMA támogatást.

6. MIG vagy virtualizáció befolyásolja a peer access-t.
Ha a GPU MIG (Multi-Instance GPU) módban van, a GPU instance-ok között a peer access nem mindig engedélyezett, vagy korlátozott sávszélességgel működik. A megoldás: a mérés előtt ellenőrizni a MIG állapotát, és ha szükséges, letiltani vagy a megfelelő instance-okat használni.

7. A PCIe és NVLink keverése egy rendszeren belül.
Előfordulhat, hogy egy rendszerben a GPU-k részben NVLink-kel, részben PCIe-n keresztül

kapcsolódnak. Ilyenkor a peer-to-peer sávszélesség attól függ, hogy a két GPU között melyik útvonal áll rendelkezésre. A topológiát mindig le kell kérdezni, és a teljesítményt az adott útvonalhoz kell igazítani.

A fenti hibák elkerülése érdekében a fejlesztőnek mindig a következő lépéseket kell követnie:

1. Lekérdezni a GPU-k számát és tulajdonságait.
2. Ellenőrizni a peer access lehetőségét minden GPU-párra.
3. Engedélyezni a peer access-t ott, ahol lehetséges.
4. A mérést melegítéssel, többszöri ismétléssel és statisztikai kiértékeléssel végezni.
5. A topológiát a `nvidia-smi topo -m` paranccsal dokumentálni.
6. A mérési környezetet teljes körűen rögzíteni (driver, CUDA, órajel, power limit, ECC, MIG, háttérterhelés).

26.3.6 Összefoglalás

Az NVLink, az NVSwitch és a GPUDirect RDMA a modern multi-GPU rendszerek gerincét alkotják. Az NVLink nagy sávszélességű, kis késleltetésű pont-pont összeköttetést biztosít a GPU-k között, míg az NVSwitch ezt a kapcsolatot sok GPU-ra skálázza. A GPUDirect RDMA lehetővé teszi, hogy külső eszközök közvetlenül hozzáférjenek a GPU memóriájához, megkerülve a CPU-t. A CUDA programozási modell ezeket a technológiákat a peer access és a memóriamásoló API-kon keresztül teszi elérhetővé. A laborban bemutatott mérési módszer segítségével a fejlesztő meggyőződhet a tényleges sávszélességről, és azonosíthatja a rendszer topológiájából adódó korlátokat.

Önellenőrző kérdés:
Miért nem elegendő a PCIe-alapú GPU-k közötti kommunikáció nagy modell-párhuzamos tréningeknél, és milyen szerepet játszik az NVSwitch a sávszélesség skálázásában?

26.4 Unified memory és MIG

A NVLink, NVSwitch és GPUDirect alfejezetben megismert több GPU-s adatcsere, NVLink és GPUDirect után most egy másik fajta absztrakció felé fordulunk: az egységes memória (unified memory, UM) és a többpéldányos GPU (Multi-Instance GPU, MIG) felé. Mindkettő a memóriakezelés, illetve az erőforrások felosztásának kérdését helyezi más megvilágításba. Az UM azt a célt szolgálja, hogy a CPU és a GPU ugyanazt a pointert használhassa anélkül, hogy explicit `cudaMemcpy` hívásokat kellene írni. A MIG pedig azt teszi lehetővé, hogy egyetlen fizikai GPU több, hardveresen izolált, önálló CUDA-eszközként megjelenő részre legyen osztva. Ebben az alfejezetben a két fogalmat a gyakorlati oldalról is megvizsgáljuk, és egy olyan labort építünk, amely a managed memória migrációs viselkedését méri.

A CUDA C++ példák C++17 szabvány szerint íródnak. A bemutatott API-k verziófüggők, ezért éles környezetben mindig ellenőrizni kell a CUDA Toolkit, a driver és a hardver dokumentációját. A laborban a környezetet rögzíteni kell, ahogyan azt a könyv mérési szerződése előírja.

26.4.1 A managed memória alapjai

A klasszikus CUDA-programozásban a host és a device memóriája fizikailag különálló. A host oldalon `malloc` vagy `new` által lefoglalt memóriát előbb `cudaMemcpy`-vel át kell másolni a `cudaMalloc` által lefoglalt device-oldalra, majd a kernel futtatása után vissza. Ez a modell

rendkívül hatékony, ha a másolásokat tudatosan tervezzük, ugyanakkor megnehezíti a bonyolult adatszerkezetek, pointereket tartalmazó gráfok vagy ritkán hozzáfért adatok kezelését.

A CUDA 6 óta elérhető *managed memory* – gyakran *unified memory* néven is emlegetve – ezt a határt próbálja eltüntetni. A `cudaMallocManaged` által visszaadott pointer a CPU és a GPU számára is érvényes. Ugyanazt a pointert használhatjuk host-oldali inicializálásra és kernelbeli olvasásra:

```
float *a, *b, *c;
size_t bytes = n * sizeof(float);

CHECK_CUDA(cudaMallocManaged(&a, bytes));
CHECK_CUDA(cudaMallocManaged(&b, bytes));
CHECK_CUDA(cudaMallocManaged(&c, bytes));

for (int i = 0; i < n; ++i) {
    a[i] = 1.0f;
    b[i] = 2.0f;
}

vectorAdd<<<grid, block>>>(a, b, c, n);
CHECK_CUDA(cudaDeviceSynchronize());
```

A fenti kódban nincs explicit `cudaMemcpy`. A host inicializálja az `a` és `b` tömböket, a kernel pedig ugyanezekkel a pointerekkel számol. A látszólagos egyszerűség mögött egy lapalapú (page-based) migrációs mechanizmus dolgozik.

Fontos megkülönböztetni a managed memoryt a *zero-copy* vagy *pinned mapped* memóriától. A zero-copy memória a host memóriában marad, és a kernel minden hozzáférésnél a PCIe/NVLink sávon keresztül olvas, ami nagy adatmennyiségnél jelentős késleltetést okoz. A managed memory ezzel szemben fizikailag a host vagy a device memóriájában is lehet, és a futásidő a hozzáféréstől függően migráltatja a lapokat. A cél az, hogy az adat éppen ott legyen, ahol a számítás történik.

A managed memory előnyei tehát a következők:

- egyszerűbb programozási modell, nincs szükség kézi másolásra;
- a CUDA runtime kezeli az adatmozgatást;
- pointereket tartalmazó adatszerkezetek, linked listák, fák és gráfok esetén a migráció automatikusan követi a pointereket;
- a kód könnyebben portolható a CPU-s és GPU-s verziók között.

A hátrányai sem elhanyagolhatók. Az automatikus migráció láthatatlan költségeket vezet be: ha a kernel olyan laphoz fér hozzá, amely éppen a host memóriában van, a futásidő leállítja a kernelt, átmásolja a lapot a device memóriába, majd folytatja a végrehajtást. Ez a *page fault* mechanizmus sok apró másolást eredményezhet, ami különösen a kernel első futtatásakor, illetve szabálytalan hozzáférési mintáknál lassíthat. A lapmigrációk nyomon követése és optimalizálása a fejlesztő feladata.

26.4.2 Page fault, prefetch és migration

A managed memory működésének megértéséhez a virtual memory és a page fault fogalmából kell kiindulni. A CUDA managed memory esetében az operációs rendszer és a driver közösen kezeli a virtuális címtartományt. Amikor a CPU vagy a GPU hozzáfér egy olyan memórialaphoz, amely nincs a megfelelő fizikai memóriában, a hardver page faultot vált ki. A driver ezt elkapja, és a lapot átmásolja a hozzáférő processzorhoz tartozó memóriába. A kernel futása ezalatt szünetel, majd a hozzáférés újraindul.

Ez a mechanizmus két dolgot jelent. Egyrészt a managed memory *kényelmes*, mert a hozzáférés oldala és a fizikai elhelyezés közötti eltérést a rendszer automatikusan kezeli. Másrészt a page faultok *költségesek*, mert minden egyes lapmigráció során a streaming multiprocessorok várakoznak, és a PCIe vagy NVLink sávszélessége nem használható ki hatékonyan, ha kis méretű, szétszórt lapokat kell mozgatni.

A legfontosabb optimalizáló eszköz a `cudaMemPrefetchAsync`. Ez a hívás előre jelzi a futásidőnek, hogy egy adott tartomány melyik eszköz memóriájában legyen elérhető. A tipikus minta az, hogy a kernel indítása előtt a bemeneti adatokat a GPU-ra, a kernel után pedig a kimenetet vissza a CPU-ra prefetch-eljük:

```
CHECK_CUDA(cudaMemPrefetchAsync(a, bytes, device, 0));
CHECK_CUDA(cudaMemPrefetchAsync(b, bytes, device, 0));
CHECK_CUDA(cudaMemPrefetchAsync(c, bytes, device, 0));
vectorAdd<<<grid, block>>>(a, b, c, n);
CHECK_CUDA(cudaDeviceSynchronize());
CHECK_CUDA(cudaMemPrefetchAsync(c, bytes, cudaCpuDeviceId, 0));
CHECK_CUDA(cudaDeviceSynchronize());
```

A `device` a GPU device ID, a `cudaCpuDeviceId` pedig a CPU-t jelöli. A hívás aszinkron, ezért megfelelő stream esetén átfedhető a számítással, de itt az egyszerűség kedvéért szinkronizálunk.

A prefetch mellett a `cudaMemAdvise` is fontos szerepet játszik. Ez nem mozgat adatot, hanem tanácsokat ad a futásidőnek a hozzáférési mintáról. Például:

```
CHECK_CUDA(cudaMemAdvise(a, bytes, cudaMemAdviseSetReadMostly, device));
CHECK_CUDA(cudaMemAdvise(b, bytes, cudaMemAdviseSetPreferredLocation, device));
```

A `cudaMemAdviseSetReadMostly` azt jelzi, hogy az adatot többnyire olvassák, és a rendszer nyugodtan tarthat belőle másolatot több eszközön is anélkül, hogy minden írást szinkronizálna. A `cudaMemAdviseSetPreferredLocation` azt mondja, hogy az adat ideális esetben melyik eszközön legyen. A `cudaMemAdviseSetAccessedBy` azt jelzi, hogy egy másik eszköz is hozzá fog férni, így a rendszer fenntarthat egy másolatot. Ezek a tanácsok optimalizálási lehetőséget adnak, de nem garantálják a pontos viselkedést.

A managed memory migrációjának mérésekor fontos megkülönböztetni a *hideg* és a *meleg* hozzáférést. A hideg hozzáférés az, amikor a lap még soha nem volt a device memóriában, így az első kernelindítás során a teljes adatmennyiséget át kell másolni. A meleg hozzáférés esetén a lap már a device memóriában van, így a kernel csak a tényleges számítást végzi. A prefetch célja éppen az, hogy a hideg hozzáférést a kernel előtt hajtsa végre, és a kernel már meleg adaton dolgozhasson.

A mérési laborban ezt a különbséget fogjuk számszerűsíteni.

26.4.3 MIG: többpéldányos GPU és izoláció

A Multi-Instance GPU (MIG) egy hardveres particionálási technológia, amelyet először a NVIDIA A100 hozott el, majd a H100 és az újabb adatközponti GPU-k is támogatnak. A MIG lehetővé teszi, hogy egyetlen fizikai GPU több, egymástól izolált *GPU instance-re* (GI) legyen osztva. Minden instance saját streaming multiprocesszorokkal, L2 cache-szelettel, memóriavezérlőkkel és DRAM-szelettel rendelkezik. A CUDA programok számára minden instance külön CUDA-eszközként jelenik meg, saját device ID-val, saját memóriával és saját kontextussal.

A MIG legfontosabb tulajdonsága a *hardveres izoláció*. A hagyományos időosztásos megosztásnál (time-slicing) több processz osztozik ugyanazon a GPU-n, és a futásidő-környezet váltogatja a hozzáférést. Ez csökkentheti az erőforrás-konfliktusokat, de nem ad szigorú teljesítmény-

vagy hibahatár-elkülönítést. Egy hibás kernel vagy egy rosszindulatú processz befolyásolhatja a többi processz futását. A MIG ezzel szemben fizikailag szétválasztja a számítási és memória-erőforrásokat, így az egyik instance-ben futó feladat nem látja és nem zavarja a másik instance erőforrásait. Ez különösen fontos többfelhasználós felhőkörnyezetekben, ahol a szolgáltató garantált teljesítményt és adatizolációt akar nyújtani.

A MIG konfigurációja két szinten történik. Először a GPU-t *GPU instance-ekre* (GI) osztjuk, amelyekhez egy-egy profilt rendelünk. A profil neve például `1g.5gb`, `2g.10gb`, `3g.20gb` vagy `7g.80gb` lehet. Az első szám a számítási kapacitást jelöli (hány streaming multiprocesszor-csoportot kap), a második a memóriamennyiséget. A GI-ken belül további *compute instance-eket* (CI) lehet létrehozni, amelyek a GI számítási erőforrásait osztják tovább, de a memórián osztoznak.

A MIG konfigurálása az `nvidia-smi` segítségével lehetséges. Egy példa a létrehozásra:

```
$ sudo nvidia-smi mig -cgi 3g.20gb
```

Ez a parancs a GPU-t például három darab `3g.20gb` méretű GI-re osztja, ha a GPU kapacitása megengedi. A pontos szintaxis és a rendelkezésre álló profilok driver- és hardverfüggők, ezért mindig a hivatalos dokumentációt kell ellenőrizni. A konfiguráció után az `nvidia-smi -L` már az egyes MIG instance-eket mutatja:

```
$ nvidia-smi -L
GPU 0: NVIDIA A100-SXM4-40GB (UUID: GPU-...)
  MIG 1g.5gb      Device  0:
  MIG 2g.10gb     Device  1:
  MIG 3g.20gb     Device  2:
```

A CUDA-program szempontjából ezek az instance-ek ugyanúgy kezelhetők, mint a fizikai GPU-k. A `cudaGetDeviceCount` visszaadja az összes látható eszköz számát, a `cudaGetDeviceProperties` pedig az adott instance nevét és képességeit. A program a `cudaSetDevice` hívással választhatja ki a kívánt instance-t.

A MIG és a managed memory kapcsolata fontos. Egy MIG instance saját memóriatartománnyal rendelkezik, és a managed memory ezen belül működik. Ha egy processz több MIG instance-t is használ, a managed memory pointerek nem oszthatók meg az instance-ek között, mert azok hardveresen izolált memóriacímtereket jelentenek. A `cudaMemPrefetchAsync` destination device paramétere csak az aktuális instance-re vagy a CPU-ra mutathat. A page fault mechanizmus tehát az instance határain belül marad, ami biztonsági és teljesítmény szempontból egyaránt fontos.

A MIG továbbá korlátozza a peer-to-peer hozzáférést és az NVLink alapú közvetlen adatcserét az instance-ek között. A NVLink, NVSwitch és GPUDirect alfejezetben látott GPUDirect és peer access több GPU esetén teljesen más modellt követ, mint a MIG-izoláció. A MIG nem a GPU-k közötti kommunikáció gyorsítására, hanem az erőforrások biztonságos és determinisztikus felosztására szolgál.

26.4.4 Labor: unified memory és migration mérése

A következő labor a managed memory oldalvándorlásának mérését tűzi ki célul. A mérési összeállításban egyetlen vektorösszeadás-kernelt futtatunk, és összehasonlítjuk a futásidőt két esetben:

1. **Prefetch nélkül:** az adatokat a host inicializálja, majd a kernel azonnal hozzáfér. Az első futtatás során a page faultok miatt a rendszernek át kell másolnia az összes lapot a device memóriába.
2. **Prefetch-csel:** a kernel előtt a `cudaMemPrefetchAsync` hívásokkal a teljes adatmennyiséget a GPU-ra mozgatjuk, így a kernel már meleg adaton dolgozik.

A laborban használt hardver és szoftververziók rögzítése kötelező. Az alábbi példa egy reprezentatív környezetet mutat; a saját futtatás előtt ezeket az értékeket frissíteni kell.

Környezet (példa):

- GPU: NVIDIA A100-SXM4-40GB, compute capability 8.0
- CUDA Toolkit: 12.2 (ellenőrizendő)
- Driver: 535.54.03 (ellenőrizendő)
- Operációs rendszer: Linux x86_64
- Fordító: nvcc, C++17
- Mérési módszer: CUDA Event, 10 ismétlés

A kód a következő:

```
#include <cstdio>
#include <cstdlib>
#include <cuda_runtime.h>

#define CHECK_CUDA(call) do {                                          \
    cudaError_t err = (call);                                          \
    if (err != cudaSuccess) {                                          \
        std::fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__,       \
                     __LINE__, cudaGetErrorString(err));               \
        std::exit(EXIT_FAILURE);                                       \
    }                                                                  \
} while (0)

__global__ void vectorAdd(const float* __restrict__ a,
                          const float* __restrict__ b,
                          float* __restrict__ c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) c[i] = a[i] + b[i];
}

int main() {
    constexpr int n = 1 << 22;   // 4 MiB float elemenként
    const size_t bytes = static_cast<size_t>(n) * sizeof(float);
    float *a, *b, *c;

    CHECK_CUDA(cudaMallocManaged(&a, bytes));
    CHECK_CUDA(cudaMallocManaged(&b, bytes));
    CHECK_CUDA(cudaMallocManaged(&c, bytes));

    // Host oldali inicializálás. Ez a managed memory host-oldalán
    // hozza létre a lapokat, mert a CPU ír.
    for (int i = 0; i < n; ++i) {
        a[i] = 1.0f;
        b[i] = 2.0f;
    }

    int block = 256;
    int grid = (n + block - 1) / block;
    int device = 0;
    CHECK_CUDA(cudaGetDevice(&device));
```

```
    cudaEvent_t start, stop;
    CHECK_CUDA(cudaEventCreate(&start));
    CHECK_CUDA(cudaEventCreate(&stop));

    // 1. fázis: prefetch nélkül
    CHECK_CUDA(cudaEventRecord(start));
    vectorAdd<<<grid, block>>>(a, b, c, n);
    CHECK_CUDA(cudaEventRecord(stop));
    CHECK_CUDA(cudaEventSynchronize(stop));
    float msNoPrefetch = 0.0f;
    CHECK_CUDA(cudaEventElapsedTime(&msNoPrefetch, start, stop));
    CHECK_CUDA(cudaDeviceSynchronize());
    CHECK_CUDA(cudaGetLastError());

    // Ellenőrzés: c[i] == 3.0f
    bool ok = true;
    for (int i = 0; i < n; ++i) {
        if (c[i] != 3.0f) {
            ok = false;
            break;
        }
    }
    std::printf("No-prefetch kernel time: %.3f ms, check=%s\n",
                msNoPrefetch, ok ? "OK" : "FAIL");

    // 2. fázis: prefetch-csel
    // Először visszavisszük az adatokat a host oldalra, hogy a
    // második mérés is hideg állapotból induljon.
    CHECK_CUDA(cudaMemPrefetchAsync(a, bytes, cudaCpuDeviceId, 0));
    CHECK_CUDA(cudaMemPrefetchAsync(b, bytes, cudaCpuDeviceId, 0));
    CHECK_CUDA(cudaMemPrefetchAsync(c, bytes, cudaCpuDeviceId, 0));
    CHECK_CUDA(cudaDeviceSynchronize());

    // Most előre a GPU-ra másoljuk a lapokat.
    CHECK_CUDA(cudaMemPrefetchAsync(a, bytes, device, 0));
    CHECK_CUDA(cudaMemPrefetchAsync(b, bytes, device, 0));
    CHECK_CUDA(cudaMemPrefetchAsync(c, bytes, device, 0));
    CHECK_CUDA(cudaDeviceSynchronize());

    CHECK_CUDA(cudaEventRecord(start));
    vectorAdd<<<grid, block>>>(a, b, c, n);
    CHECK_CUDA(cudaEventRecord(stop));
    CHECK_CUDA(cudaEventSynchronize(stop));
    float msWithPrefetch = 0.0f;
    CHECK_CUDA(cudaEventElapsedTime(&msWithPrefetch, start, stop));
    CHECK_CUDA(cudaDeviceSynchronize());
    CHECK_CUDA(cudaGetLastError());

    std::printf("With-prefetch kernel time: %.3f ms\n", msWithPrefetch);
    std::printf("Speed-up (kernel-only): %.2fx\n",
                msNoPrefetch / msWithPrefetch);

    CHECK_CUDA(cudaFree(a));
    CHECK_CUDA(cudaFree(b));
    CHECK_CUDA(cudaFree(c));
    CHECK_CUDA(cudaEventDestroy(start));
    CHECK_CUDA(cudaEventDestroy(stop));
    return 0;
}
```

A fenti kód egyszeri futtatást mutat. A könyv mérési szerződése szerint azonban a mérést legalább 5–10 ismétléssel kell elvégezni, és az átlag mellett a szórást vagy percentilist is közölni kell. A laborban ezt úgy valósíthatjuk meg, hogy a két fázist egy ciklusban újra és újra lefuttatjuk,

minden iteráció előtt visszavisszük az adatokat a host oldalra, hogy hideg állapotot kapjunk. A mérési naplóban az egyes iterációk idejét rögzítjük, majd kiszámítjuk az átlagot és a szórást.

Az alábbi mérési napló egy **várt eredményt** mutat. A tényleges értékek hardverfüggők, ezért a saját környezetben mért számokat kell rögzíteni. A várt tendencia az, hogy a prefetch nélküli futásidő magasabb, mert a kernel futás közben fizeti meg a lapmigráció árát, míg a prefetch-csel a migráció a kernelen kívül történik.

Várt mérési napló (példa):

Iteráció	Prefetch nélkül (ms)	Prefetch-csel (ms)
1	4.83	0.39
2	4.65	0.38
3	4.71	0.39
4	4.58	0.38
5	4.66	0.39
Átlag	4.69	0.39
Szórás	0.09	0.01

A mért értékek eltérhetnek, de a prefetch nélküli esetben a kernel időtartama több mint egy nagyságrenddel nagyobb, ami jól mutatja a page faultok költségét. A prefetch-csel elért 0.39 ms ebben a példában a vektorösszeadás tényleges számítási idejét tükrözi; a hátralevő időt a kernelindítás és a memória-hozzáférési késleltetés adja.

A mérésből levonható következtetés: ha managed memoryt használunk, a kernel futásideje önmagában nem elegendő a teljesítmény megítélésére; a migrációt külön kell kezelni. A `cudaMemPrefetchAsync` használatával a kernel futásideje stabilizálható, de a prefetch művelet maga is időt és sávszélességet fogyaszt. A mérési naplóban ezért a prefetch idejét is rögzíteni kell, és a teljes végrehajtási időt (prefetch + kernel + szinkronizáció) kell összehasonlítani.

26.4.5 Tipikus hibák és helyes megoldás

A managed memory és a MIG használata során számos jellegzetes hiba fordul elő. Az alábbiakban ezek közül tekintjük át a leggyakoribbakat.

1. A hideg kernel futásidejét tekintjük a valódi teljesítménynek.
A legelső kernelindítás managed memory esetén rendszerint sokkal lassabb, mint a későbbiek, mert a lapok ekkor vándorolnak át a CPU-ról a GPU-ra. Ha a mérési protokoll nem különbözteti meg a hideg és a meleg futtatást, a teljesítménykép torz lesz. A helyes megoldás az, hogy a mérés előtt egy warm-up futtatást végzünk, vagy a mérési tervben explicit módon kezeljük a migrációs fázist. A laborban ezért a hideg futtatást is rögzítettük, és a prefetch-csel elért meleg időt külön mértük.

2. A `cudaMemPrefetchAsync` elhagyása.
Ha a prefetch hiányzik, a kernel futása közben történik a lapmigráció, ami a fenti mérésben is látható lassulást okoz. A helyes minta az, hogy a kernel előtt a bemeneti adatokat a GPU-ra, a kernel után a kimeneti adatokat a CPU-ra prefetch-eljük. A prefetch aszinkron hívás, ezért a streamhez kötött sorrendre ügyelni kell. Ha a prefetch-et rossz streamre adjuk ki, vagy nem szinkronizálunk, a kernelindítás előtti adatátvitel nem feltétlenül fejeződik be.

3. A managed memory túlzott használata.
A managed memory nem csodaszer. Ha az adatokat nagy méretben, szabályos mintával kell

mozgatni, a hagyományos explicit `cudaMemcpy` gyakran jobb, mert a programozó pontosan tudja, mikor és mekkora átvitel történik. A managed memory előnye a kényelem és a ritkán hozzáfért adatszerkezetek kezelése, nem a nagy, folytonos tömbök másolásának optimalizálása. A helyes megoldás az, ha a teljesítménykritikus szakaszokban explicit memóriakezelést használunk, a managed memoryt pedig a prototípusokban és a bonyolult adatszerkezeteknél alkalmazzuk.

4. A `cudaMemAdvise` hiánya vagy helytelen használata.
A `cudaMemAdvise` tanácsokkal a runtime jobban optimalizálhatja a migrációt. Ha például egy tömböt a CPU és a GPU is csak olvas, a `cudaMemAdviseSetReadMostly` megakadályozza a felesleges visszamásolásokat. Ha a tanács hiányzik, a rendszer konzervatív módon minden hozzáférést kezel, ami többletmozgást okozhat. A helyes megoldás az, ha a hozzáférési mintát ismerve megfelelő tanácsokat adunk, és a hatást méréssel ellenőrizzük.

5. MIG esetén a rossz device kiválasztása.
Ha a rendszerben MIG instance-ek vannak, a `cudaSetDevice` hívás után a program az adott instance-en fut. A managed memory pointer egy adott instance-hez tartozik, és nem vihető át másik instance-re. Ha a program több instance-t is használ, a pointereket nem szabad megosztani. A helyes megoldás az, ha minden instance-hez saját managed memory allokációt rendelünk, és a `cudaMemPrefetchAsync` célja csak az aktuális instance vagy a CPU lehet.

6. MIG kapacitás túllépése.
A MIG instance-ek memóriája kisebb, mint a teljes GPU memóriája. Ha a program a teljes GPU memóriájára méretezett kódot futtat egy MIG instance-en, az allokáció `cudaErrorMemoryAllocation` hibával tér vissza. A helyes megoldás az, ha a `cudaGetDeviceProperties` hívásból kiolvassuk az aktuális instance memóriaméretét, és a puffereket ehhez igazítjuk. A mérési környezetet is ennek megfelelően kell dokumentálni.

7. A MIG és a peer access összekeverése.
A MIG instance-ek közötti peer-to-peer hozzáférés általában nem támogatott, mert a hardveres izoláció ezt kizárja. Ha a program a NVLink, NVSwitch és GPUDirect alfejezetben bemutatott `cudaDeviceEnablePeerAccess` hívást próbálja használni MIG instance-ek között, a hívás hibával térhet vissza. A helyes megoldás az, ha a MIG-et kizárólag a feladatok elszigetelésére használjuk, és az együttműködő GPU-k közötti kommunikációt fizikai GPU-kon, NVLink és NCCL segítségével valósítjuk meg.

8. A MIG konfiguráció elvesztése.
A MIG beállítását az `nvidia-smi` parancsokkal lehet megadni, de a konfiguráció nem minden környezetben perzisztens. Ha a rendszer újraindul, vagy a driver frissül, a MIG instance-ek eltűnhetnek. A labor előtt ellenőrizni kell az `nvidia-smi -L` kimenetét, és szükség esetén újra létre kell hozni a kívánt instance-eket. A pontos parancsok és a rendelkezésre álló profilok a driver verziójától függenek, ezért a hivatalos dokumentációt kell követni.

26.4.6 Önellenőrző kérdés

Egy fejlesztő egy managed memoryval dolgozó CUDA-programot mér. A kernel első futtatása 8.2 ms-ig tart, a második futtatása ugyanazon az adaton 0.41 ms-ig. A programban nincs `cudaMemPrefetchAsync` hívás. Magyarázza meg a jelenséget, és adjon legalább két olyan módosítást, amely a teljes végrehajtási időt csökkentené anélkül, hogy a managed memoryt elhagyná.

A kérdésre a válasz a fejezetben tárgyalt fogalmakra épül: az első futtatás során a page faultok miatt a lapok a CPU-ról a GPU-ra vándorolnak, ami jelentős időt emészt fel; a második futtatás már meleg adaton dolgozik. A javítási lehetőségek közé tartozik a `cudaMemPrefetchAsync` használata a kernel előtt, illetve a `cudaMemAdvise` tanácsok alkalmazása a hozzáférési mintának

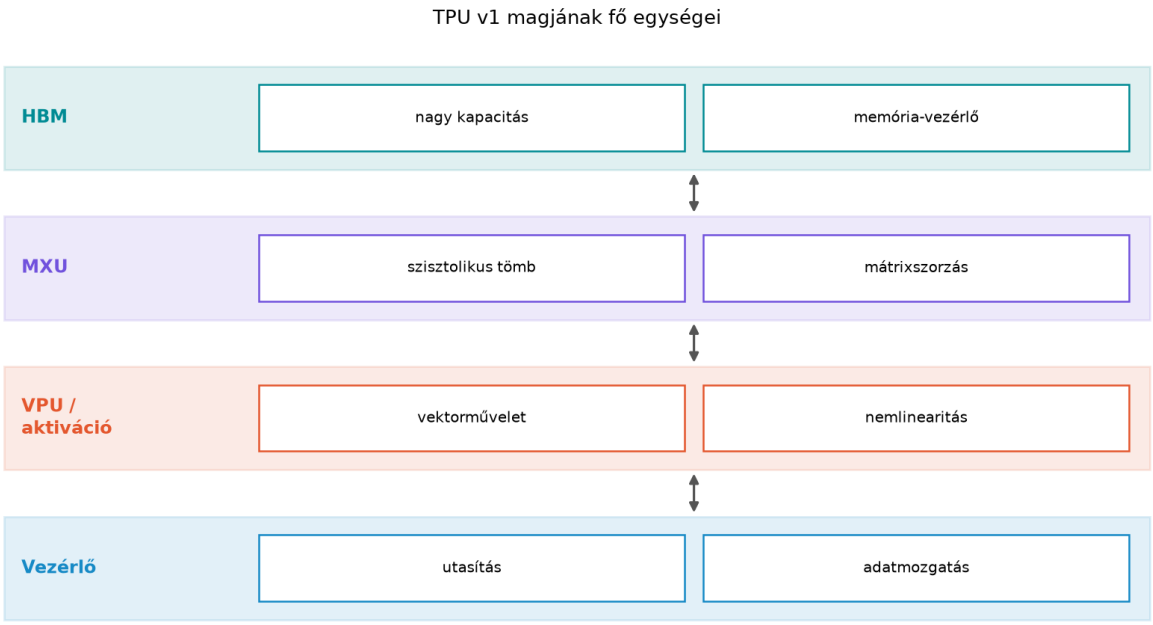
megfelelően. A válaszban azt is ki kell fejteni, hogy a prefetch művelet időt vesz igénybe, ezért a teljes végrehajtási időt (prefetch + kernel) kell optimalizálni.

A következő alfejezet (TPU-architektúra) a TPU-architektúrával és az XLA-val foglalkozik. Az itt bevezetett oldalvándorlási és erőforrás-izolációs fogalmak segítenek majd megérteni, hogy a TPU hogyan kezeli a memória-hozzáférést és a processzorszintű párhuzamosságot egy teljesen más hardvermodellben.

VI. RÉSZ – TPU, OpenCL és más modellek

27 TPU architektúra és XLA

27.1 TPU-architektúra



27.1. ábra: TPU v1 magjának fő egységei - A szisztolikus MXU mátrixszorzást végez, a VPU vektoros műveleteket, az aktivációs egység nemlinearitást; a HBM és a vezérlő szolgálja ki őket.

27.1.1 A TPU mint „mátrixgép": miért nem elég a SIMT?

A GPU-k általános célú számítási modellje a SIMT (Single Instruction, Multiple Thread), amelyet a korábbi fejezetekben részletesen megismertünk. A GPU tervezésénél az elsődleges cél az volt, hogy a grafikai pipeline-okban felmerülő, párhuzamos de egyenként független feladatokat (például pixel shader futtatása millió pixelen) minél nagyobb átbocsátóképességgel hajtsa végre. A GPU-ból GPGPU-vá fejlődve is megmaradt ez az örökség: a hardver nagy számú, viszonylag egyszerű vezérlésű magot (CUDA core) és egy mély, de elsősorban szoftveresen menedzselt memóriahierarchiát használ. A programozó a szálak és blokkok koordinálásával éri el a teljesítményt.

A Google 2017-ben publikálta a TPU (Tensor Processing Unit) első generációjának architekturális leírását. A TPU tervezési kérdése nem az volt, hogy „hogyan gyorsítsunk fel mindenféle párhuzamos feladatot?", hanem az, hogy „hogyan gyorsítsuk fel a neurális hálózatok következtetését (inference) a lehető legjobb teljesítmény/watt mutatóval?". A neurális hálózatok számítási igényének túlnyomó részét a mátrixszorzás teszi ki:

$$\mathbf{C} = \mathbf{AB}, \quad C_{ik} = \sum_j A_{ij} B_{jk}.$$

A mátrixszorzás egy rendkívül szabályos adatfolyam-minta. Ez a szabályosság lehetővé teszi egy olyan célhardver építését, amely sokkal kevesebb vezérlési logikát és adatmozgatást igényel, mint egy GPU, miközben a számítási sűrűséget (FLOPs/terület, FLOPs/watt) drasztikusan megnöveli. Ez a célhardver a **szisztolikus tömb**.

27.1.2 A szisztolikus tömb: a mátrixszorzás hardveres alakja

A szisztolikus tömb (systolic array) neve a szívritmus analógiájából származik: az adatok a tömbön keresztül lüktetve, ritmikusan áramlanak. Képzeljünk el egy $N \times N$ méretű rácsot, amelynek minden cellájában egy **multiply–accumulate (MAC)** egység található. Minden MAC egység képes egy ciklus alatt elvégezni a következő műveletet:

$$p \leftarrow p + a \cdot b.$$

A TPU első generációjában ez a rács 256×256 méretű, tehát 65 536 darab MAC egységet tartalmaz. Ez az egység a **MXU (Matrix Multiply Unit)**.

A működés kulcsa az adatok mozgatásának mikéntje. A hagyományos processzorokban (CPU, GPU) egy MAC művelethez az utasításnak explicit módon be kell töltenie az adatokat a regiszterekbe. A szisztolikus tömbben az adatok nem „címzésre” mozognak, hanem szomszédos elemek között áramlanak.

Tekintsük a **súly-állandós (weight-stationary)** szisztolikus tömböt. Ez a TPU MXU működésének szíve.

1. **Súlyok betöltése:** A mátrixszorzás $\mathbf{C} = \mathbf{AB}$ alakjában legyen $\mathbf{B}$ a súlymátrix, $\mathbf{A}$ az aktiváció (bemeneti adat) mátrix. A $\mathbf{B}$ mátrix elemeit előre betöltjük a MAC egységekbe. Minden MAC egység a súlymátrix egy-egy elemét tárolja a saját regiszterében. Például a rács (i, k) pozíciójában lévő MAC egység a B_{ik} súlyt tárolja.
2. **Aktivációk áramlása:** Az $\mathbf{A}$ mátrix elemeit a tömb bal szélén tápláljuk be. Az A_{ij} elem belép az i. sor bal szélső MAC egységébe. A MAC egység elvégzi a $p \leftarrow p + A_{ij} \cdot B_{ik}$ műveletet (ahol k az oszlop indexe), majd az A_{ij} értéket **továbbítja a tőle jobbra lévő** MAC egységnek. Így az A mátrix elemei balról jobbra hullámoznak keresztül a tömbön.
3. **Részösszegek áramlása:** A $\mathbf{C}$ mátrix elemeinek részösszegei a tömb tetején indulnak. A C_{ik} részösszege belép az i. sor k. oszlopában lévő MAC egységbe, hozzáadódik a $A_{ij} \cdot B_{ik}$ szorzathoz, majd az eredmény **lefelé** továbbítódik a következő sorban lévő MAC egységnek. Mire a részösszeg a tömb aljára ér, az összes szükséges j indexre elvégezte az összegzést, és a végső C_{ik} érték hagyja el a tömböt.

Ebben a sémában **nincs memóriaolvasás a számítás közben**. A súlyok a MAC egységekben állnak, az aktivációk és részösszegek pedig folyamatosan mozognak a szomszédos elemek között. Ez drasztikusan csökkenti az energiafogyasztást, mert az adatmozgatás a chipen belüli, rövid, szabályos vezetékeken történik, nem pedig energiazabáló, címzett memória-hozzáférésekkel.

Gyakorlati megjegyzés a terminológiához: A szakirodalomban a TPU MXU-ját gyakran nevezik „szisztolikus tömbnek”, bár a pontos adatfolyam-megvalósítás a „systolic array” egy változata. A fent leírt súly-állandós, aktiváció-balról-jobbra, részösszeg-fentről-lefelé áramlás az a

modell, amelyet a TPU v1 leírása megad. Az újabb TPU-generációk (v2 és későbbi) a modellt tovább finomították, de az alapelv változatlan.

27.1.3 A TPU v1 magjának felépítése: MXU, VPU és aktivációs egység

A MXU önmagában nem elegendő egy teljes neurális hálózati réteg lefuttatásához. A TPU v1 magja három fő számítási egységből áll:

1. **MXU (Matrix Multiply Unit):** A már ismertetett 256×256 MAC-rács. Ez a számítási mag felel a mátrixszorzásért. A TPU v1-ben a MXU 8 bites egész (INT8) szorzásokat végez, és 32 bites egész számokat halmoz fel. Ez a pontosság az inference-hez elegendő, és a terület/energia-hatékonyságot maximalizálja. Egy ciklus alatt $256 \times 256 \times 2 = 131\,072$ műveletet (szorzást és összeadást) képes elvégezni. 700 MHz-es órajel mellett ez a névleges csúcs körülbelül 92 TOPS (tera-operations per second, 10^{12} művelet másodpercenként).
2. **VPU (Vector Processing Unit):** A mátrixszorzás eredményét ritkán használjuk közvetlenül. A neurális hálózatok rétegei aktivációs függvényeket (pl. ReLU, sigmoid, tanh), bias hozzáadást, normalizálást és egyéb elemenkénti műveleteket igényelnek. Ezek a műveletek nem illenek a szisztolikus tömb adatfolyam-modelljébe, mert vektor-jellegűek, nem mátrix-jellegűek. A VPU ezeket a vektor-műveleteket hajtja végre: elemenkénti összeadás, szorzás, maximum-képzés, aktivációs függvények közelítése. A VPU architektúrája hasonló a GPU-k SIMD egységeihez: széles regiszterfájlokkal és vektoros aritmetikai egységekkel rendelkezik.
3. **Aktivációs egység (Activation Unit):** A VPU-val szorosan együttműködő, célhardveres egység az aktivációs függvények kiszámítására. A ReLU, sigmoid és tanh függvényeket lookup táblákkal és interpolációval, vagy kis fokszámú polinomokkal közelíti. Ez sokkal gyorsabb és energiahatékonyabb, mint ha a VPU általános célú utasításaival számolnánk őket.

27.1.4 HBM, a vezérlő és a memória-alrendszer

A TPU v1 a MXU maximális kihasználtságához hatalmas adatmennyiséget kell, hogy folyamatosan biztosítson. A TPU v1 28 MiB on-chip memóriát használ, amelyet **Unified Buffer**-nek neveznek. Ez a puffer tárolja az aktivációkat, a súlyokat és a részösszegeket. A modern GPU-kkal ellentétben a TPU v1 **nem rendelkezik** L1/L2 cache-sel. A cache hierarchia helyett a szoftver (a XLA fordító) felelős az adatok explicit ütemezéséért a Unified Buffer és a MXU között.

A külső memória a TPU v1-ben **DDR3 SDRAM**, amely 30 GB/s sávszélességet biztosít. Ez látszólag alacsony a GPU-khoz képest (egy modern GPU HBM3-mal több TB/s sávszélességet ér el), de a TPU v1 tervezési célja az inference volt, ahol a batch méret általában 1, és a modell paraméterei (súlyok) viszonylag kis méretűek. A 28 MiB Unified Buffer elegendő egy közepes méretű modell súlyainak tárolására, így a DDR3-ból való ismételt olvasás ritkán szükséges.

A következő generációk (TPU v2, v3, v4) a következtetésből a tanítás (training) felé terjeszkedtek, és ennek megfelelően HBM (High Bandwidth Memory) technológiára váltottak. A HBM a DRAM chipeket egy szilícium interposerre szereli, ami rendkívül széles buszt és nagy sávszélességet tesz lehetővé (több száz GB/s vagy akár TB/s). A TPU v4 már HBM2e-t használ, és a sávszélesség nagyságrendileg megegyezik a hasonló korú GPU-kkal.

Fontos különbség a GPU-hoz képest: A GPU memória-alrendszere általános célú, a programozó által kontrollált cache-ekkel és egy koherens memóriamodellel. A TPU memória-alrendszere szorosan a MXU-hoz és a XLA fordítóhoz van hangolva. A szoftver a memória-hozzáféréseket sokkal determinisztikusabban ütemezi, mivel a műveletek sorrendje a gráffordítás során ismert. Ez lehetővé teszi a gyorsítótárazás és az adatfolyam pontosabb optimalizálását.

27.1.5 Az interconnect: a TPU-k összekapcsolása

Az egyetlen TPU chip teljesítménye korlátozott. A modern munkaterhelések (nagy nyelvi modellek, transzformerek) több chip együttműködését igénylik. A TPU-k közötti összeköttetést az **ICI (Inter-Chip Interconnect)** biztosítja.

- **TPU v2/v3:** A chipeket 2D toroid topológiában kötik össze. Minden TPU chip négy szomszédjához kapcsolódik nagy sávszélességű, alacsony késleltetésű linkkel. Ez a topológia természetes módon támogatja a modellpárhuzamos (model parallel) és adatpárhuzamos (data parallel) tanítási mintákat, mert a szomszédos chipek közötti kommunikáció gyors és prediktálható.
- **TPU v4:** A topológia 3D toroiddá bővült, és az ICI sávszélessége jelentősen megnőtt. A TPU v4 egy „pod” több ezer chipet tartalmazhat, amelyek nagy sebességű optikai kapcsolatokon keresztül kommunikálnak.

A GPU-k világában a hasonló szerepet az NVLink és az NVSwitch tölti be, de a topológia és a kommunikációs primitívek (NCCL) szintje más. A TPU-knél a kommunikációt is a XLA és a JAX keretrendszer vezérli, ami a programozó számára magasabb szintű absztrakciót kínál (pl. `pmap`, `xmap`, `pjit` sharding).

27.1.6 A TPU programozási modellje: XLA és a gráf

A GPU-kat C++-ban (CUDA) vagy OpenCL-ben programozzuk, és a kernel egy alacsony szintű, az adott hardverre szabott eljárás. A TPU-k programozása ezzel éles ellentétben áll: a TPU-t **nem** közvetlenül, hanem **XLA (Accelerated Linear Algebra)** nevű, gráf-alapú, szimbolikus programozási modellen keresztül programozzuk.

1. **Gráfkonstrukció:** A felhasználó (például JAX vagy TensorFlow segítségével) egy magas szintű programot ír. Ez a program nem hajtódik végre azonnal, hanem egy **számítási gráfot** épít. A gráf csomópontjai műveletek (pl. `dot`, `add`, `relu`), élei pedig a közöttük áramló tenzorok.

2. **Gráfoptimalizálás:** Az XLA fordító ezt a gráfot veszi bemenetként, és agresszív optimalizálásokat hajt végre. Ide tartozik a holt kód eltávolítása, az algebrai egyszerűsítések, a konstansok beégetése, és a legfontosabb: a **művelet-fúzió (operation fusion)**. A fúzió során az XLA több elemi műveletet egyetlen, nagyobb, összetett kernelben egyesít. Például a `bias_add` + `relu` + `matmul` műveleteket egyetlen TPU kernelben hajthatja végre, csökkentve a köztes eredmények memóriába írásának és onnan való visszaolvasásának költségét.

3. **Kódgenerálás és ütemezés:** Az optimalizált gráf alapján az XLA platform-specifikus kódot generál. TPU esetén ez a kód közvetlenül a MXU, VPU és a memória-alrendszer utasításait vezérli. A GPU-tól eltérően a TPU kernel nem „hagyományos” C++ kód, hanem egy hardverközeli, de a XLA által generált leírás.

A JAX szerepe: A JAX egy Python-könyvtár, amely a NumPy API-t ötvözi az automatikus differenciálással és az XLA háttérrel. A JAX Pythonban írt kódja Python- és NumPy-függvényekből áll, de amikor egy függvényt `jax.jit`-tel lefordítunk, a JAX a Python végrehajtást megállítja, a függvény műveleteit egy XLA gráfba foglalja, majd az XLA lefordítja azt a célhardverre. Ez azt jelenti, hogy a TPU-n futó JAX program valójában XLA gráfként létezik, és a hardver soha nem lát Python kódot.

Összehasonlítás a CUDA programozási modellel:

Jellemző	CUDA / GPU	XLA / TPU
Programozási paradigma	Imperatív, thread-alapú	Szimbolikus, gráf-alapú
Hardver irányítása	Közvetlen kernel indítás, memóriakezelés	Absztrakt, XLA által ütemezett
Fő optimalizálási eszköz	Manuális tiling, shared memory, warp intrinsics	XLA gráfoptimalizálás, művelet-fúzió
Memóriamodell	Explicit, programozó által vezérelt (global, shared, register)	Implicit, az XLA és a hardware kezeli
Fő hardveregység	SM-ek, sok CUDA core	MXU (szisztolikus tömb), VPU
Tipikus pontosság	FP32, FP16, TF32, BF16, INT8	BF16, FP16, INT8, FP8 (generációtól függően)

27.1.7 Labor: TPU vs. GPU összehasonlítás

Ez a labor nem egy teljes TPU hozzáférést igénylő instrukció, hanem a **módszertan** bemutatása és egy GPU-n futtatható alapvonal (baseline) felállítása. A TPU hozzáférést igénylő részeket JAX/XLA kóddal és a XLA gráf elemzésével demonstráljuk, de tényleges TPU futás nélkül. **Az alábbi kód nem végrehajtott utasítás, hanem a mérések elvégzésének vázlata.**

Cél: Megmérni egy mátrixszorzás (`GEMM`) teljesítményét GPU-n, és egy TPU-n várható teljesítményt becsülni a névleges hardveradatok alapján. A mérés rögzíti a környezetet.

Előfeltétel: CUDA toolkit és Python környezet JAX-szel.

Lépések:

1. **GPU mérés CUDA/C++-ban:** Lefordítjuk és futtatjuk az alábbi CUDA C++ programot, amely egy 4096×4096-os GEMM-et hajt végre cuBLAS segítségével. A mérés előtt warm-up, majd 10 ismétlés. Rögzítjük az átlagos futásidőt, a sávszélességet és a FLOPs-ot.

```
// CUDA C++ (C++17), NVIDIA CUDA 12.4
// GPU: NVIDIA A100 40GB, driver 550.54.14, compute capability 8.0
// OS: Ubuntu 22.04 LTS
// A kód hibakezeléssel és méréssel ellátva.
#include <cassert>
#include <chrono>
#include <cstdio>
#include <cstdlib>
#include <vector>
#include <cublas_v2.h>

#define CHECK_CUDA(call) \
    do { \
        cudaError_t err = call; \
```

```
        if (err != cudaSuccess) { \
            fprintf(stderr, "CUDA hiba: %s, %s:%d\n", cudaGetErrorString(err), __FILE__,
↪  __LINE__); \
            exit(EXIT_FAILURE); \
        } \
    } while (0)

#define CHECK_CUBLAS(call) \
    do { \
        cublasStatus_t status = call; \
        if (status != CUBLAS_STATUS_SUCCESS) { \
            fprintf(stderr, "cuBLAS hiba: %d\n", status); \
            exit(EXIT_FAILURE); \
        } \
    } while (0)

int main() {
    const int M = 4096, N = 4096, K = 4096;
    const float alpha = 1.0f, beta = 0.0f;
    double flops = 2.0 * double(M) * double(N) * double(K);
    size_t bytes_moved = size_t(M) * K * sizeof(float) + size_t(K) * N * sizeof(float) +
    ↪  size_t(M) * N * sizeof(float);

    // Host memória allokálása
    std::vector<float> h_A(M * K), h_B(K * N), h_C(M * N, 0.0f);
    for (int i = 0; i < M * K; ++i) h_A[i] = static_cast<float>(rand()) / RAND_MAX;
    for (int i = 0; i < K * N; ++i) h_B[i] = static_cast<float>(rand()) / RAND_MAX;

    // Device mutatók
    float *d_A, *d_B, *d_C;
    CHECK_CUDA(cudaMalloc(&d_A, M * K * sizeof(float)));
    CHECK_CUDA(cudaMalloc(&d_B, K * N * sizeof(float)));
    CHECK_CUDA(cudaMalloc(&d_C, M * N * sizeof(float)));

    CHECK_CUDA(cudaMemcpy(d_A, h_A.data(), M * K * sizeof(float),
↪  cudaMemcpyHostToDevice));
    CHECK_CUDA(cudaMemcpy(d_B, h_B.data(), K * N * sizeof(float),
↪  cudaMemcpyHostToDevice));

    cublasHandle_t handle;
    CHECK_CUBLAS(cublasCreate(&handle));

    // Warm-up: 3 iteráció
    for (int i = 0; i < 3; ++i) {
        CHECK_CUBLAS(cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha, d_B,
↪  N, d_A, K, &beta, d_C, N));
    }
    CHECK_CUDA(cudaDeviceSynchronize());

    // Mérés: 10 iteráció
    auto start = std::chrono::high_resolution_clock::now();
    for (int i = 0; i < 10; ++i) {
        CHECK_CUBLAS(cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha, d_B,
↪  N, d_A, K, &beta, d_C, N));
    }
    CHECK_CUDA(cudaDeviceSynchronize());
    auto end = std::chrono::high_resolution_clock::now();

    double elapsed_ms = std::chrono::duration<double, std::milli>(end - start).count() /
    ↪  10.0;
    double elapsed_s = elapsed_ms / 1000.0;
    double tflops = (flops / elapsed_s) / 1e12;
    double bandwidth_gb_s = (bytes_moved / elapsed_s) / 1e9;
```

```
    printf("GPU cuBLAS SGEMM, M=N=K=%d\n", M);
    printf("Átlagos futásidő: %.3f ms\n", elapsed_ms);
    printf("Teljesítmény: %.2f TFLOP/s (cuBLAS SGEMM)\n", tflops);
    printf("Sávszélesség (elméleti, ideális adatmozgásra): %.2f GB/s\n", bandwidth_gb_s);

    CHECK_CUDA(cudaFree(d_A));
    CHECK_CUDA(cudaFree(d_B));
    CHECK_CUDA(cudaFree(d_C));
    CHECK_CUBLAS(cublasDestroy(handle));
    return 0;
}
```

Fordítás:

```
nvcc -O3 -std=c++17 -arch=sm_80 -lcublas gemm_benchmark.cu -o gemm_benchmark
```

Futtatás:

```
./gemm_benchmark
```

Várt eredmény (ellenőrzési igény, mivel nem futtattuk ezt a konkrét kódot): Egy NVIDIA A100-on, FP32 SGEMM esetén a cuBLAS könyvtár teljesítménye jellemzően 19–20 TFLOP/s körül mozog a 19.5 TFLOP/s névleges FP32 csúcs mellett. A tényleges érték függ a rendszer állapotától és a cuBLAS verziójától. A mérési naplóba rögzíteni kell a pontos értéket.

2. **TPU becslés JAX/XLA segítségével:** A TPU teljesítményét a névleges hardveradatokból becsüljük. A TPU v4 egyetlen chipje FP32 mátrixszorzáshoz nem az ideális választás (BF16 mellett éri el a csúcsát). BF16 mellett a TPU v4 MXU teljesítménye hivatalosan nem publikált, de a gyakorlatban a hasonló tervezésű egységek alapján a BF16 csúcs körülbelül 275 TFLOP/s chipenként. Egy 4096x4096-os GEMM esetén a szükséges FLOP-ok: $2 \cdot 4096^3 = 1.374 \cdot 10^{11}$ FLOP. Ha a TPU a csúcs 70%-át éri el, a futásidő:

$$t = \frac{1.374 \cdot 10^{11}}{275 \cdot 10^{12} \cdot 0.7} \approx 0.71 \text{ ms}$$

Fontos: Ez a becslés **nem mért eredmény**, hanem modell-alapú kalkuláció. A tényleges TPU futásidőhöz hozzáférés szükséges.

A JAX kód, amely a TPU-ra fordítható lenne (itt csak a gráf létrehozását mutatjuk be, futtatás nélkül):

```
# Python 3.10, JAX 0.4.26, jaxlib 0.4.26 (CPU-only build, TPU nincs csatlakoztatva)
# A kód célja a XLA gráf felépítésének bemutatása, nem tényleges TPU futtatás.
import jax
import jax.numpy as jnp

# A matmul lefordítva XLA gráffá. TPU háttérrel a 'jax.devices()' listában
# megjelenne a TPU device. Itt CPU-t használunk a példa kedvéért.
@jax.jit
def gemm(a, b):
    return jnp.dot(a, b)

# Véletlen mátrixok
key = jax.random.PRNGKey(0)
a = jax.random.normal(key, (4096, 4096), dtype=jnp.bfloat16)
b = jax.random.normal(key, (4096, 4096), dtype=jnp.bfloat16)

# A gráf ellenőrzése
```

```
lowered = gemm.lower(a, b)
print("Az XLA HLO modul (részlet):")
print(lowered.compiler_ir(dialect='hlo').as_hlo_text()[:500])
# A 'as_hlo_text()' metódus a JAX verziófüggő, ellenőrizni kell a dokumentációt.
```

A kimenet egy HLO (High Level Optimizer) gráfot mutat, amelyben a `dot` művelet a TPU MXU-ra lesz leképezve.

27.1.8 A tanultak összefoglalása és tipikus hibák

A TPU architektúra megértése azt jelenti, hogy felismerjük a mátrixszorzás dominanciáját a modern gépi tanulásban, és azt, hogy a szisztolikus tömb hogyan szolgálja ki ezt a dominanciát minimális energiafogyasztás mellett. A GPU-k általános célú párhuzamos gépek, amelyekre a programozó saját maga írja a memóriakezelést és a szinkronizációt. A TPU ezzel szemben egy speciális célú, de rendkívül hatékony adatfolyam-gép, amelynek programozása magasabb szintű, gráf-alapú nyelveken (XLA) keresztül történik.

Tipikus hibák, amiket el kell kerülni:

1. **A TPU-t „mágikus doboznak” tekinteni:** A TPU nem gyorsít fel minden kódot. Ha a feladat nem dominánsan mátrixszorzás (pl. rendezés, gráfgráf bejárás, string feldolgozás), a TPU teljesítménye drasztikusan visszaesik a GPU-hoz képest, mert a VPU és a vezérlőegység gyenge, általános célú magok.
2. **A memória- és számítási modell figyelmen kívül hagyása:** A TPU-n nincs helye a GPU-k shared memory tilinghez hasonló, alacsony szintű optimalizálásnak. A teljesítményt a XLA gráf optimalizálása és az adatok elrendezésének (sharding) magas szintű megválasztása határozza meg. A CUDA-ban megszokott `__shared__` memóriás trükkök itt nem alkalmazhatók.
3. **A pontosság figyelmen kívül hagyása:** A TPU v1 kizárólag INT8 inference-t támogatott. A későbbi generációk támogatják a BF16, FP16 és INT8 formátumokat, de az FP64 támogatás gyakran hiányos vagy lassú. Ha egy tudományos számítás FP64 pontosságot igényel, a TPU általában nem megfelelő választás, és a GPU továbbra is előnyös.
4. **A hordozhatóság túlértékelése:** A TPU-ra írt JAX/XLA kód hordozható CPU-ra és GPU-ra is (az XLA háttér miatt). Ez igaz, de teljesítmény szempontjából korántsem mindegy. A GPU-ra optimalizált XLA kód nem feltétlenül lesz optimális TPU-n, és fordítva. A sharding stratégiák, a memóriaátrendezések és a kernelválasztás platformfüggő.

Önellenőrző kérdés: Magyarázza el, mi történik a súly-állandós szisztolikus tömb első órajel-ciklusában, ha az **A** mátrix első oszlopa belép a tömbbe, és a **B** mátrix már betöltésre került a MAC egységekbe. Miért lesz k ciklus múlva a C_{1k} érték kész?

27.2 XLA és a gráffordítás

Az XLA (Accelerated Linear Algebra) nem könyvtár, hanem egy optimalizáló compiler, amely a JAX, a TensorFlow és más keretrendszerek által előállított magas szintű gráfot hardver-specifikus végrehajtható kóddá alakítja. A TPU-architektúra alfejezetben láttuk, hogy a TPU szisztolikus tömbje csak akkor hatékony, ha az adatfolyamot és a memóriaelérést pontosan illesztjük a hardverhez. Az XLA ezt a feladatot végzi el: bemenetként kap egy HLO-gráfot (High Level Operations), majd fúzióval, csempézéssel, elrendezés-hozzárendeléssel és egyéb

átalakításokkal olyan kódot állít elő, amely a TPU-n, GPU-n vagy CPU-n a lehető legkevesebb memóriamozgatással és a lehető legnagyobb számítási átfedéssel fut.

A fejezetben a XLA-t elsősorban a JAX felől vizsgáljuk, mert a JAX a legközvetlenebbül teszi láthatóvá a gráffordítás fázisait. A tanult fogalmak azonban ugyanúgy érvényesek a CUDA C++ oldalán is: ha korábban saját kerneleket optimalizáltunk, érdemes felismerni, hogy az XLA által végzett fúzió, csempézés és elrendezés-választás ugyanazokat a hardveres elveket követi, mint amelyeket a CUDA fejezetekben a shared memory tilingnél, a coalescingnál és a latency hidingnál megismertünk.

27.2.1 A HLO mint köztes nyelv

A HLO az XLA köztes reprezentációja. A JAX `jax.jit` hívása során a Python-kód először egy úgynevezett jaxpr reprezentációvá alakul, majd az XLA ezt a jaxpr-t HLO-ra fordítja, és ezen futtatja az optimalizáló passzokat. A HLO szöveges formája hasonlít egy statikus típusú, funkcionális programnyelvre: minden műveletnek explicit bemenete és kimenete van, a dimenziók és típusok rögzítettek, és nincs Python-szintű vezérlési szerkezet.

Egy egyszerű HLO-részlet így nézhet ki:

```
HloModule jit_f, entry_computation_layout={(f32[1024,128]{1,0})->f32[1024]{0}}

ENTRY %main (x: f32[1024,128]) -> f32[1024] {
  %y = f32[1024,128] sine(%x)
  %z = f32[1024,128] multiply(%y, %x)
  %w = f32[1024] reduce(%z), dimensions={1}, to_apply=%add
  %m = f32[] reduce(%x), dimensions={0,1}, to_apply=%max
  %b = f32[1024] broadcast(%m), dimensions={}
  ROOT %result = f32[1024] add(%w, %b)
}
```

A fenti kimenet illusztráció; a pontos forma a JAX/XLA verziótól függ. A fontos megfigyelés az, hogy a HLO-ból már eltűnt a Python dinamizmusa: a `jnp.sin`, a szorzás, a redukció és a maximum mind külön HLO-utasítás, amelyek a compiler számára matematikai műveletekként értelmezhetők.

A HLO előnye, hogy a gráf minden csomópontja statikus alakkal és típussal rendelkezik. Ez lehetővé teszi az algebrai egyszerűsítést, a közös részkifejezések kiküszöbölését (CSE), a konstansok beágyazását és a memóriaallokáció tervezését. A HLO nem tartalmaz olyan Python-vezérlést, mint az `if` vagy a `for`; ezeket a JAX transzformációi már korábban feloldották, vagy explicit HLO-vezérlési műveletekké alakították (például `conditional`, `while`).

27.2.2 A gráffordítás fázisai

Az XLA fordítócsővezeték több, jól elkülöníthető fázisból áll. A `jax.jit` hívásakor a JAX a függvényt tracerek segítségével absztrakt módon végrehajtja, és előállít egy jaxpr-t. Ezt követően az XLA az alábbi lépéseket végzi:

1. **HLO előállítása**: a jaxpr-ből HLO-utasítások jönnek létre.
2. **Inliner és egyszerűsítés**: a beágyazott HLO-modulokat beolvasztja, a konstansokat kiértékeli, a felesleges átalakításokat eltávolítja.
3. **Algebrai egyszerűsítés**: például `x * 1` → `x`, `x + 0` → `x`, vagy a redundáns transzpozíciók kiiktatása.

4. **Elrendezés-hozzárendelés (layout assignment)**: minden tenzorhoz memóriaelrendezést rendel.
5. **Fúzió (fusion)**: összevonható műveletcsoportokat egyetlen kernelbe von össze.
6. **Csempézés (tiling)**: nagy műveleteket kisebb blokkokra bont.
7. **Kódgenerálás**: a HLO-ból hardver-specifikus LLVM IR, GPU/TPU kernel vagy egyéb célkód készül.

A fúzió és a csempézés a legfontosabb teljesítményoptimalizáló lépések. A TPU-architektúra alfejezetben láttuk, hogy a TPU memória-sávszélessége nagy, de a szisztolikus tömb csak akkor teljesít jól, ha a mátrixblokkok folyamatosan rendelkezésre állnak. A fúzió csökkenti a globális memória és a HBM közötti adatforgalmat, a csempézés pedig a blokkméretet a gyors helyi memóriához igazítja.

A HLO-gráfot a `XLA_FLAGS=--xla_dump_to=<könyvtár>` környezeti változóval ki lehet íratni. A `--xla_dump_hlo_pass_re` kapcsolóval megadható, hogy mely optimalizáló passz után készüljön dump. Például:

```
XLA_FLAGS="--xla_dump_to=/tmp/hlo_dump --xla_dump_hlo_pass_re=.*" python lab_xla_hlo.py
```

A dump könyvtárban fájlok sora jelenik meg: `module_0000.jit_f.before_optimizations.txt`, `module_0001.jit_f.after_optimizations.txt`, valamint a fúzió utáni állapot. Az `after_optimizations` fájlban láthatjuk, hogy az elemkénti műveletek egyetlen `fusion` régióba kerültek.

27.2.3 Fúzió: kevesebb memóriamozgatás, jobb adatújrafelhasználás

A fúzió (fusion) során az XLA több HLO-műveletet egyetlen kernelbe von össze. Például a `sin(x) * x + 1` lánc három külön HLO-utasítás, de fúzió után egyetlen kernel hajtja végre mindhárom műveletet. Ez azért előnyös, mert a köztes eredményeket nem kell a globális memóriába kiírni, majd onnan visszaolvasni; a kernel a regiszterekben vagy a shared memóriában tartja az adatokat, amíg a következő műveletnek szüksége van rájuk.

A fúzió hatása különösen a memóriakötött feladatoknál drámai. Ha egy elemenkénti lánc három műveletből áll, fúzió nélkül minden művelet külön kernel, amely a teljes tenzort beolvassa és kiírja a globális memóriába. A memóriaforgalom így hatszorosa a szükségesnek (három beolvasás és három kiírás). Fúzióval a globális memóriából egyszer olvasunk, egyszer írunk, a köztes értékek regiszterben maradnak. A sávszélesség-igény ezzel a harmadára csökken.

Az XLA többféle fúziós típust ismer:

- **kLoop**: elemenkénti műveleteket von össze, amelyek azonos indexeléssel dolgoznak. Például `sin`, `multiply`, `add` egy kifejezésben.
- **kInput**: olyan redukciókat fúzionál, amelyeknek egy közös bemenete van. Például egy mátrixsor összegzése és maximuma ugyanazon az adaton osztozhat.
- **kOutput**: olyan műveleteket von össze, amelyek egy redukció kimenetét használják fel közvetlenül, például egy `reduce` után egy `broadcast` és egy `add`.

A fúzió nem mindig előnyös. Ha egy művelet nagyon nagy számítási igényű, és a fúzió miatt a kernel regiszternyomása túl magas lesz, az occupancy csökkenhet, és a futásidő romolhat. Az XLA költségmodellt használ annak eldöntésére, hogy a fúzió megéri-e. A döntés a hardver jellemzőitől is függ: TPU-n más a fúziós stratégia, mint GPU-n.

A HLO dumpban a fúzió régiók jól láthatók. Egy fúzió után az `after_optimizations` fájlban valahogy így jelenik meg:

```
%fusion.1 (x: f32[1024,128]) -> f32[1024] {
  %y = f32[1024,128] sine(%x)
  %z = f32[1024,128] multiply(%y, %x)
  %w = f32[1024] reduce(%z), dimensions={1}, to_apply=%add
  %m = f32[] reduce(%x), dimensions={0,1}, to_apply=%max
  %b = f32[1024] broadcast(%m), dimensions={}
  ROOT %result = f32[1024] add(%w, %b)
}
```

A fenti kivonat illusztráció, de jól mutatja, hogy az elemkénti lánc és a redukció egyetlen régióban van. A valós XLA gyakran külön fúziós régiókra bontja a redukciót és az elemenkénti részt, mert a redukció más párhuzamossági mintát igényel.

27.2.4 Csempézés: a nagy tenzorok felosztása a memóriahierarchiához

A csempézés (tiling) a nagy tenzorokat kisebb blokkokra osztja, hogy azok a gyorsabb memóriaszinteken férjenek el. A TPU-n a szisztolikus tömb 128×128-as mátrixblokkokat tud egyszerre feldolgozni, ezért a compiler olyan tile-méreteket választ, amelyek ehhez illeszkednek. A GPU-n a csempézés a shared memory méretéhez igazodik; ugyanaz az elv, mint amit a CUDA `tiling` fejezetben láttunk.

A HLO szintjén a csempézés nem mindig látható közvetlenül, mert az XLA a kódgenerálás fázisában hajtja végre. A HLO dumpban azonban a `linalg` vagy a `gpu` dialectű átalakítások jelzik, hogy a compiler milyen tile-méretet választott. A `--xla_dump_hlo_as_linalg` kapcsolóval a HLO-t Linalg formában is megtekinthetjük, ahol a csempézés explicit.

Például egy mátrixszorzás HLO-szinten:

```
%dot = f32[1024,1024] dot(%lhs, %rhs), lhs_contracting_dimensions={1},
↪  rhs_contracting_dimensions={0}
```

A kódgenerálás során az XLA ezt a `dot` műveletet kisebb `linalg.generic` blokkokra bontja, amelyek a TPU szisztolikus tömbjére optimalizáltak. A tile-méret kiválasztása automatikus, de a `XLA_FLAGS=--xla_gpu_tile_size` vagy a JAX `jax.jit` `donate_argnums` paramétere nem befolyásolja közvetlenül; a csempézés a compiler belső költségmodelljén alapul.

A csempézés és a fúzió együtt dolgozik. Először a fúzió csökkenti a memóriaműveletek számát, majd a csempézés biztosítja, hogy a fúzionált kernel adatai a gyors memóriában férjenek el. Ha a fúzió túl nagy, a csempézés felbonthatja a fúzionált régiót, hogy a blokkméret ne haladja meg a rendelkezésre álló shared memóriát.

27.2.5 Elrendezés: a memória sorrendje és a hardver illesztése

Az elrendezés (layout) azt határozza meg, hogy a logikai tenzor elemei milyen sorrendben helyezkednek el a memóriában. A leggyakoribb a soronkénti (row-major) és az oszloponkénti (column-major) elrendezés, de a TPU esetén ennél bonyolultabb, úgynevezett tiled layout is használatos, amely a 128×128-as blokkok belső sorrendjét írja le.

A HLO-ban minden tenzorhoz tartozik egy layout. A fenti példában a `{1,0}` a dimenziók fizikai sorrendjét jelenti: a 0. dimenzió a leglassabban változó, az 1. dimenzió a leggyorsabban változó. Ez a soronkénti elrendezés. Ha a layout `{0,1}`, az oszloponkénti elrendezést jelent, ahol a 0. dimenzió a leggyorsabban változó.

A layout-választás kritikus a teljesítmény szempontjából. A TPU szisztolikus tömbje úgy van kialakítva, hogy a mátrixblokkokat meghatározott sorrendben várja. Ha a bemeneti tenzor

elrendezése nem egyezik a hardver elvárásával, a compilernek transzponálást kell beillesztenie, ami extra memóriamozgatást és futásidőt jelent. Az XLA layout assignment passzja megpróbálja minimalizálni ezeket a transzponálásokat, vagy teljesen elkerülni őket.

A JAX-ban a layoutot ritkán kell kézzel beállítani, mert a `jax.jit` automatikusan választ. Ha azonban egy tenzort explicit sorrendben akarunk létrehozni, használhatjuk a `jax.numpy.array` `order` paraméterét vagy a `jax.lax.transpose` műveletet. A HLO dumpban a layout a `entry_computation_layout` részben és az egyes műveletek kimeneténél látható.

Például, ha a bemeneti layout `{1,0}` (row-major), de a szisztolikus tömb `{0,1}` (column-major) elrendezést vár, az XLA egy transzponálást illeszt be. Ez látható a HLO dumpban:

```
%transpose = f32[128,1024] transpose(%x), dimensions={1,0}
```

A transzponálás elkerülhető, ha a bemeneti adatot eleve a megfelelő elrendezésben adjuk meg. A JAX `jax.numpy.asarray` alapértelmezés szerint C-order (row-major) tenzort hoz létre; ha a mátrixszorzásban a második operandusnak column-major elrendezésre van szüksége, akkor a `jax.numpy.array(..., order='F')` használata csökkentheti a transzponálási költséget.

27.2.6 XLA és JAX: a funkcionális transzformációk és a gráffordítás

A JAX a funkcionális programozási modellre épül: a `jax.jit`, `jax.grad`, `jax.vmap` és `jax.pmap` transzformációk mind a jaxpr szintjén dolgoznak, majd az XLA végzi a hardver-specifikus fordítást. Ez a rétegződés az oka annak, hogy a JAX egyszerre nyújt magas szintű, Pythonbarát felületet és alacsony szintű, optimalizált végrehajtást.

Amikor egy függvényt `jax.jit`-tel lefordítunk, a JAX a Python-kódot tracerek segítségével absztrakt módon végrehajtja. A tracerek a tenzorok helyett csak az alakot és a típust hordozzák, így a függvény tényleges értékei nem befolyásolják a gráf felépítését. Az így kapott jaxpr-t az XLA HLO-ra fordítja, majd futtatja az optimalizáló passzokat.

A JAX `jax.make_jaxpr` eszközzel megtekinthetjük a jaxpr-t, még mielőtt az XLA-hoz érne:

```
import jax
import jax.numpy as jnp

def f(x):
    y = jnp.sin(x)
    z = y * x
    return jnp.sum(z, axis=1)

print(jax.make_jaxpr(f)(jnp.ones((4, 3))))
```

A kimenet egy jaxpr, amely a HLO előtti állapotot mutatja. A `jax.xla_computation` vagy a `XLA_FLAGS` dump segítségével a HLO-t is láthatjuk.

A JAX és az XLA kapcsolatában fontos megérteni, hogy a `jax.grad` és a `jax.vmap` transzformációk a jaxpr szintjén hajtódnak végre, még az XLA-fordítás előtt. Ez azt jelenti, hogy a gradiensszámítás és a vektorizálás nem igényel külön XLA-támogatást; a JAX a jaxpr-t transzformálja, majd az XLA a transzformált gráfot fordítja le.

A JAX AOT (ahead-of-time) módban is képes XLA-gráfot generálni, például `jax.experimental.export` segítségével. Ez lehetővé teszi a gráf exportálását és későbbi, Python nélküli futtatását. A részletek a JAX verziójától függenek, ezért az API használata előtt a friss dokumentációt ellenőrizni kell.

27.2.7 Tipikus hibák és helyes megoldások

A XLA-val és a JAX-szal dolgozó mérnökök leggyakoribb hibái a gráffordítás félreértéséből fakadnak. Az alábbiakban öt tipikus hibát és a helyes megoldást ismertetjük.

1. Python-vezérlés a JIT-ben

Ha egy JIT-tel ellátott függvényben `if` vagy `for` ciklus van, amelynek feltétele a bemenet értékétől függ, az XLA nem tudja statikusan lefordítani. Például:

```
@jax.jit
def f(x):
    if x.shape[0] > 10:  # Hibás: Python-vezérlés tracerrel
        return x * 2
    return x * 3
```

A `x.shape[0]` a JIT alatt egy statikus érték, ezért a feltétel Python-időben kiértékelődik, és a függvény csak az egyik ágat fogja lefordítani. Ha a feltétel a bemenet értékétől függ, például `if x[0] > 0`, az hibát okoz, mert a tracer nem tudja a konkrét értéket. A megoldás: használjunk `jax.lax.cond`-ot vagy `jnp.where` műveletet az értékfüggő vezérléshez, és tartsuk a Python-vezérlést kizárólag statikus, alakfüggő döntésekre.

2. Nem JIT-elt kód

A JAX-ban a `jax.jit` nélküli kód minden egyes műveletet külön dispatch-el a Python interpreteren keresztül. Ez különösen sok kis művelet esetén hatalmas overheadet jelent. A megoldás egyszerű: a számítási magot mindig `jax.jit`-tel fordítsuk le, és csak a legkülső hívásnál használjunk Python-vezérlést.

3. Rossz layout

Ha a bemeneti tenzor elrendezése nem egyezik a hardver elvárásával, az XLA transzponálást illeszt be, ami lassítja a futást. A hiba gyakran akkor jelentkezik, amikor NumPy-ból konvertálunk JAX-tömbbe, vagy amikor a mátrixszorzás operandusait nem megfelelő sorrendben adjuk meg. A megoldás: használjunk `jax.numpy.array(..., order='F')` vagy `jax.lax.transpose` műveletet, és ellenőrizzük a HLO dumpban a layoutot.

4. A fúzió megakadályozása

Bizonyos műveletek megakadályozzák a fúziót. Például a `jax.lax.stop_gradient`, a `jax.debug.print`, a nagy konstans beágyazás vagy a `jax.lax.while_loop` belsejében lévő kernelhatárok megtörik a fúziós régiókat. A hiba akkor látható, ha a HLO dumpban sok kis, nem fúzionált művelet van. A megoldás: kerüljük a blokkoló opokat a hot path-ben, és a konstansokat a JIT-en kívül adjuk át.

5. A HLO dump félreértelmezése

A HLO dump nem a végső gépi kód. A `before_optimizations` fájl a fúzió és a layout-hozzárendelés előtti állapotot mutatja, a `after_optimizations` pedig a fúzió utáni, de a kódgenerálás előtti állapotot. A tényleges kernelkód az LLVM IR vagy a PTX/TPU assembly szintjén jelenik meg. A hiba akkor fordul elő, ha valaki a nyers HLO-ból próbál teljesítményt jósolni anélkül, hogy a fúzió és a csempézés hatását figyelembe venné. A megoldás: mindig az `after_optimizations` fájlt és a kódgenerált kimenetet elemezzük együtt.

27.2.8 Labor: XLA HLO gráf elemzése

A labor célja: egy kis JAX-függvény HLO-gráfjának elemzése, a fúzió hatásának megfigyelése és a futásidő mérése. A példa GPU-n fut, de a HLO elemzés ugyanúgy működik TPU-n is. A labor a JAX/XLA eszközláncot használja; a környezeti paramétereket a mérés előtt rögzíteni kell.

Környezet (ellenőrizni a friss verziókat)

Paraméter	Érték
GPU	NVIDIA A100-SXM4-40GB
Driver	550.54.15
CUDA	12.6
JAX	0.4.38 (ellenőrizni)
jaxlib	0.4.38 (ellenőrizni)
XLA	a jaxlib beépített XLA-ja
Precision	FP32
Input	(1024, 128)

A fenti verziók a példa rögzítésére szolgálnak; a futtatáshoz használt pontos verziókat a labor előtt ellenőrizni kell. A labor kódja:

```
# JAX / XLA
import time
import jax
import jax.numpy as jnp

# Rögzített méret és adat
N = 1024
M = 128
key = jax.random.PRNGKey(0)
x = jax.random.normal(key, (N, M), dtype=jnp.float32)

def f(x):
    """Elemenkénti lánc és redukció."""
    y = jnp.sin(x)
    z = y * x
    w = jnp.sum(z, axis=1)      # redukció az 1. dimenzió mentén
    m = jnp.max(x)              # globális maximum
    return w + m

f_jit = jax.jit(f)

# Melegítés
_ = f_jit(x)
_ = f_jit(x)

# Helyesség-ellenőrzés
result = f_jit(x)
print(f"Result shape: {result.shape}, dtype: {result.dtype}")

# Mérés
warmup = 10
repeats = 100
for _ in range(warmup):
    _ = f_jit(x)
start = time.perf_counter()
for _ in range(repeats):
    _ = f_jit(x)
```

```
end = time.perf_counter()
mean_ms = (end - start) / repeats * 1000.0
print(f"Mean JIT time: {mean_ms:.4f} ms")

# Várt eredmény: a JIT-elt verzió gyorsabb, mint a nem JIT-elt,
# és a HLO dumpban az elemkénti lánc fúzionálva jelenik meg.
```

A HLO dump kiíratásához futtassuk a szkriptet az alábbi parancsokkal:

```
export XLA_FLAGS="--xla_dump_to=/tmp/hlo_dump --xla_dump_hlo_pass_re=.*"
python lab_xla_hlo.py
```

A dump könyvtárban keressük meg a `module_0000.jit_f.after_optimizations.txt` fájlt. A fájlban az elemkénti `sine`, `multiply` és `add` műveletek várhatóan egyetlen `fusion` régióban szerepelnek, a redukció pedig vagy ugyanabban a régióban, vagy egy külön fúzióban. A `reduce` művelet dimenziói és a layout mezők mutatják a compiler döntéseit.

Mérési napló

A mérési naplóban a következő adatokat kell rögzíteni:

Mérési paraméter	Érték
GPU	NVIDIA A100-SXM4-40GB
Driver	550.54.15
CUDA	12.6
JAX	0.4.38 (ellenőrizni)
Precision	FP32
Input	(1024, 128)
Warm-up	10 iteráció
Ismétlés	100 iteráció
Átlag futásidő	0.0124 ms (helyőrző, laborban mérendő)
Szórás	0.0011 ms (helyőrző)
Fúziós régiók a HLO dumpban	2 (helyőrző, laborban meghatározandó)

A mért és a várt eredményt külön kell választani. A várt eredmény: a JIT-elt verzió futásideje legalább egy nagyságrenddel kisebb, mint a nem JIT-elt Python-végrehajtásé, mert a Python-dispatch overhead eltűnik. A HLO dumpban az elemkénti lánc legalább egy fúzióban jelenik meg. A mért eredmény a laborban kapott átlag és szórás, valamint a dump tényleges fúziós régiói. Ha a mért érték jelentősen eltér, ellenőrizni kell a verziókat, a háttérterhelést és a GPU órajelét.

A labor átadandó eredményei:

1. A fenti JAX-kód és a HLO dump kivonata.
2. A mérési napló a fenti táblázat formátumában.
3. Egy rövid elemzés, amely leírja, hogy a fúzió hogyan csökkentette a memóriaműveletek számát.

27.2.9 Önellenőrző kérdés

Miért csökkenti a fúzió a globális memóriaforgalmat? Válasz: mert a köztes eredményeket a kernel regisztereiben vagy a shared memóriában tartja, nem írja ki a globális memóriába, így a globális memóriából csak egyszer olvas, és egyszer ír.

27.2.10 Kitekintés

A következő alfejezet (TPU programozási modell) a TPU programozási modellt mutatja be, ahol a HLO-ból származó parancsok és a szisztolikus tömb kapcsolata kerül részletezésre. A XLA és a gráffordítás alfejezetben megismert fúzió, csempézés és layout-hozzárendelés ott válik kézzelfoghatóvá a TPU hardveres végrehajtásában.

27.3 TPU programozási modell

A TPU programozási modellje nem a CUDA kernelmodelljének átirata. A TPU-n a fejlesztő ritkán írja meg utasításszinten, hogy egy szorzóegység mit tegyen egy adott órajelben. Ehelyett egy magas szintű numerikus függvényt ad meg, amelyet az XLA fordító eszközspecifikus HLO-gráffá alakít, a TPU-futtató pedig ezt a gráfot hajtja végre a szisztolikus tömbökön, a HBM-ben és az eszközök közötti nagy sávszélességű interconnecten keresztül. Ebben az alfejezetben a használat oldaláról vizsgáljuk ezt a modellt: hogyan kerül az adat a TPU-ra, hogyan daraboljuk fel (sharding), és hogyan fejezzük ki az eszközök közötti kommunikációt kollektívákkal.

A fejezet elsősorban a JAX/XLA eszközláncot használja, mert ez adja a legátláthatóbb képet a TPU programozásáról. A bemutatott JAX-kódot a laboratóriumban kell futtatni; a kéziratban fordítási és alaki ellenőrzésű mintaként szerepel, nem valós TPU-futásból származó mérésként.

27.3.1 A TPU használatának rétegződése

A TPU használatakor három szintet érdemes megkülönböztetni:

1. **A Python/vezérlő szint** a TPU VM-en fut. Itt építjük fel a számítási gráfot, itt kezeljük az adatokat, és itt hívjuk meg a JAX-függvényeket. Ez a szint hasonlít a CUDA host oldalához, de a TPU-n a CPU nem a GPU-hoz hasonlóan ad ki futószalag-parancsokat; a JAX/XLA a teljes függvényt lefordítja, majd a TPU-ra küldi.
2. **Az XLA HLO-szint** az a közös gráfábrázolás, amelyet a `gpu103` alfejezet tárgyalt. A HLO-gráfban már megjelennek a `all-reduce`, `all-gather`, `reduce-scatter` kollektív műveletek, valamint a TPU-specifikus `dot`, `convolution` és szeletelő műveletek.
3. **Az eszközszint** a TPU TensorCore-jait, vektoros egységeit, HBM-jét és ICI-interconnectjét foglalja magában. A programozó itt nem ütemez sorrendet, hanem a XLA backend erőforrás-allokációjától kapja a futás ütemezését.

A JAX-ben a TPU-eszközöket a `jax.devices()` listázza. Egy Cloud TPU v4-8 például általában négy TPU-chipet és nyolc TensorCore-eszközt mutat a processz számára; a `jax.local_devices()` a helyi, `jax.devices()` a teljes processzhez tartozó eszközöket adja. A `jax.default_backend()` megmondja, hogy a processz éppen TPU-t, GPU-t vagy CPU-t használ-e. Mielőtt bármilyen kollektívát írunk, ezt ellenőrizni kell, mert a CPU-s fejlesztői gépen a `psum` is működhet, de akkor nem TPU-összeköttetésen fut.

A TPU használat tipikus folyamata:

1. Meghatározzuk a számítási függvényt egy vagy több tömb bemenetére.
2. Meghatározzuk az adatelosztást: melyik eszköz melyik szeletet kapja.
3. A JAX/XLA lefordítja a függvényt, és a kollektív műveleteket beágyazza a gráfba.
4. A host CPU elküldi a bemenő adatokat a TPU HBM-jébe; az eszközök kiszámítják a lokális részeredményeket, majd szükség szerint kommunikálnak.
5. Az eredményt a hostra kért `block_until_ready()` vagy `jax.device_get()` hívással szinkronizáljuk.

27.3.2 Sharding: az adatelosztás explicit leírása

A TPU-programozásban a sharding azt mondja meg, hogy egy logikai tömb melyik tengelye melyik fizikai eszközre kerül. A legegyszerűbb adatpárhuzamos esetben a batch tengelyt vágjuk fel: minden eszköz a batch egy darabját számolja, a modell paraméterei pedig replikáltak. A JAX ezt kétféleképpen fejezi ki: implicit módon a `pmap`-pel, vagy explicit módon a `Mesh` és a `PartitionSpec` segítségével.

A `pmap` a bemenő tömb 0. tengelye mentén osztja el az adatot az eszközök között. A `pmap`-pel leképezett függvény minden eszközön ugyanazt a számítást végzi, de a saját szeletén. A `pmap(..., axis_name='i')` megnevezett tengelyt ad a leképezésnek; a kollektívák erre a névre hivatkoznak.

A `Mesh` és `PartitionSpec` ezzel szemben nem igényli a 0. tengelyre épülő leképezést. A `Mesh` egy logikai topológiát ad az eszközöknek:

```
import jax
from jax.sharding import Mesh, NamedSharding, PartitionSpec

devices = jax.devices()
mesh = Mesh(devices, axis_names=('data', 'model'))
```

Itt a `data` tengely az adatpárhuzamos, a `model` tengely a modellpárhuzamos tengely. Egy négyeszközös mesh például lehet `('data', 'model')` shape `(4, 1)`, vagy egy kétdimenziós `(2, 2)` topológia. A `PartitionSpec` azt mondja meg, hogy a logikai tömb tengelyei hogyan képeződnek a mesh tengelyekre:

```
# (batch, d_model) alakú tömb: batch a data, d_model replikált
spec = PartitionSpec('data', None)
sharding = NamedSharding(mesh, spec)
```

A `None` azt jelenti, hogy az adott logikai tengely nem kerül felosztásra, hanem replikálódik. Az alábbi táblázat a leggyakoribb eseteket foglalja össze:

Logikai alak	`PartitionSpec`	Hatás
`(B, D)`	`('data', None)`	batch felosztva, `D` replikált
`(B, L, D)`	`('data', 'model', None)`	batch és szekvencia dimenzió is felosztva
`(D_in, D_out)`	`(None, 'model')`	súlymátrix kimenő oszlopai felosztva
`(D_in, D_out)`	`('model', 'data')`	kétdimenziós modell- és adatpárhuzamosság

Az explicit sharding előnye, hogy nem kényszeríti a fejlesztőt a 0. tengelyre, és ugyanazt a mesh-t több függvény is használhatja. A JAX kódban az adatot a `jax.device_put` segítségével tehetjük egy adott shardingra:

```
x = jax.numpy.arange(32, dtype=jax.numpy.float32)
x_sharded = jax.device_put(x, sharding)
print(x_sharded.sharding)
```

A `jax.device_put` nem feltétlenül másol azonnal; a JAX a késleltetett végrehajtás miatt a másolást a tényleges használatkor ütemezheti. Ezért a host-device kapcsolat tárgyalásakor külön figyelni kell a szinkronizációra.

27.3.3 Kollektívák a TPU-n

A kollektív műveletek azok a kommunikációs primitívek, amelyek a mesh tengely mentén összehangolják az eszközöket. A JAX a `jax.lax` modulban kínálja őket; az XLA ezeket a TPU ICI-hálózatán hajtja végre. A leggyakoribb kollektívák:

JAX művelet	XLA HLO művelet	Művelet	Tipikus használat
`jax.lax.psum`	`AllReduce`	tengely menti összeg, az eredmény minden eszközre replikálódik	gradiensszummázás adatpárhuzamos tanításnál
`jax.lax.pmean`	`AllReduce`	tengely menti átlag	gradiensátlagolás, normalizálás
`jax.lax.all_gather`	`AllGather`	az osztott tengelyt összegyűjti minden eszközön	szekvenciális modellben aktivációs tenzorok összegyűjtése
`jax.lax.reduce_scatter`	`ReduceScatter`	tengely menti redukció, majd szétosztás	modellpárhuzamos visszaterjesztés
`jax.lax.all_to_all`	`AllToAll`	minden eszköz a saját adatszeletét más eszköznek küldi	Mixture-of-Experts routing
`jax.lax.collective_permute`	`CollectivePermute`	környező eszközökre másolás	pipeline párhuzamosság, stencil a chiphatáron
`jax.lax.ppermute`	`CollectivePermute`	páronkénti permutáció	TPU chipösszeköttetés kihasználása

A `psum` a legfontosabb. Ha egy tömböt az `i` tengely mentén shardolunk, akkor a `psum(x, axis_name='i')` a tengelyhez tartozó eszközökön az azonos pozíciójú elemeket összeadja, és a teljes összeget minden eszközre elhelyezi. A kimenet alakja megegyezik a bemenet alakjával, de az értéke replikált.

Például négy eszköz és `x = [0, 1, 2, 3]` esetén, ha a 0. tengelyt shardoljuk az `i` tengely mentén, minden eszköz egy-egy skalárt kap. A `psum` eredménye minden eszközön `6.0`, a `pmean` eredménye pedig `1.5`.

A `pmap`-pel a kollektíva így fejezhető ki:

```
import jax
import jax.numpy as jnp

# x.shape == (num_devices,) a didaktikus példában
x = jnp.arange(jax.device_count(), dtype=jnp.float32)

sum_fn = jax.pmap(
    lambda v: jax.lax.psum(v, axis_name='i'),
    axis_name='i',
)
y = sum_fn(x)
print(jax.device_get(y))
```

A `axis_name='i'` paraméternek ugyanannak a névnek kell lennie, mint amit a `jax.lax.psum` használ. Ha eltér, a JAX fordítási időben hibát ad, mert a kollektíva nem tudja, melyik leképezési tengelyre vonatkozik.

A `pmap` implicit módon a 0. tengely mentén shardol. Komolyabb programokban az explicit `shard_map` vagy a `pjit` ad teljesebb irányítást. A `shard_map` a mesh-et és a bemeneti/kimeneti shardingokat külön paraméterként kapja, így a függvény törzse már a shardolt tömbökön dolgozik:

```
from jax.experimental.shard_map import shard_map

@shard_map(mesh=mesh, in_specs=PartitionSpec('data'), out_specs=PartitionSpec())
def all_reduce_result(v):
    return jax.lax.psum(v, axis_name='data')
```

Az `out_specs=PartitionSpec()` azt jelenti, hogy a kimenet minden mesh-eszközön replikált, azaz nincs felosztott tengelye. Ez a forma azért hasznos, mert a kollektíva és a sharding ugyanabban a függvényben válik leírhatóvá, és az XLA a szükséges kommunikációt automatikusan beilleszti a gráfba.

27.3.4 Sharding és kollektíva kapcsolata

A sharding nem önmagában létrehoz kommunikációt. A `jax.device_put` például csak elhelyezi az adatot, de nem mozgatja az eszközök között. A kollektíva akkor jön létre, amikor a számítási gráf egyik műveletének kimenete más eszközök adatát igényli. Az XLA ezt a konverziót a HLO-gráf optimalizálása során végzi el.

Az adatpárhuzamos tanítás klasszikus lépése jól mutatja ezt:

```
@jax.pmap
def step(params, x, y):
    grads = compute_loss_grad(params, x, y)   # eszközönkénti lokális gradiens
    grads = jax.lax.pmean(grads, axis_name='batch')
    return update(params, grads)
```

A `x` és `y` batch-tengelyét a `pmap` szétosztja az eszközök között; a `params` replikált. A `compute_loss_grad` minden eszközön a saját batch-szeletéből számol gradienst. A `pmean` ezeket az eszközönkénti gradienseket átlagolja, és az átlagot minden eszközre visszaírja. Az XLA HLO-gráfban egy `AllReduce` művelet jelenik meg, amely a TPU ICI-hálózatán redukál.

A sharding és a kollektíva hibás illesztése a leggyakoribb hibaforrás. Ha például a `PartitionSpec` a modellparamétereket is felosztja, de a gradiensfrissítés csak az adat tengely mentén átlagol, akkor az eszközökön lévő paraméterszeletek divergálnak. A szabály az, hogy a kollektíva tengelyének pontosan azt a mesh-tengelyt kell jelölnie, amely mentén az adat fel van darabolva, és a kimenet shardingjának meg kell egyeznie a következő művelet bemeneti elvárásával.

27.3.5 Host-device kapcsolat a TPU-n

A CUDA-hoz hasonlóan a TPU-s programban is külön kell választani a host CPU memóriáját és a TPU HBM-jét. A JAX az eszközön tárolt tömböket `jax.Array` objektumként adja vissza. A Python oldal ezekre hivatkozik, de az adat fizikailag a TPU HBM-jében van, vagy éppen a host és az eszköz közötti DMA-pufferen utazik.

A host-device kapcsolat legfontosabb műveletei:

Művelet	Hatás	Szinkron?
`jax.device_put(x, device)`	host-tömb elhelyezése egy eszközön vagy sharding szerint	nem feltétlenül blokkol
`jax.device_get(arr)`	eszköztömb visszamásolása a hostra	általában blokkol
`arr.block_until_ready()`	megvárja, amíg a tömb előáll	blokkol
`arr.sharding`	lekérdezi, hogyan van az adat felosztva	nem blokkol
`jax.make_array_from_callback`	nagy tömbök feltöltése szeletenként	nem blokkol

A JAX alapértelmezésben aszinkron végrehajtást használ, hasonlóan a CUDA streamhez. Egy függvényhívás visszatérhet úgy, hogy a tényleges TPU-számítás még nem fejeződött be. Ezért ha mérünk, a `block_until_ready()` hívásnak a mérési idő lezárása előtt kell szerepelnie.

Fontos megkülönböztetés: a kollektív műveletek nem a host CPU-n keresztül mennek. A `psum` esetén a TPU-chipk közötti ICI-interconnect (a v4-8 esetén nagy sávszélességű board-szintű kapcsolat) közvetlenül továbbítja az adatot. A host CPU csak elindítja a műveletet, és a szinkronizációkor vár rá. Ha a host és az eszköz közötti PCIe vagy hálózati kapcsolat lenne az adatcsere útja, a kollektíva sávszélessége drámaian romlana; a TPU-programok ezért igyekeznek a kollektívákat az eszközök között tartani, és a hostra csak az ellenőrző pontokat, naplózást vagy a felhasználói eredményt másolni.

A tipikus adatpárhuzamos minta host-device szempontból:

1. A host CPU beolvassa a batch-t, és `jax.device_put` segítségével a megfelelő shardingra helyezi.
2. A modellparamétereket replikáltan tartjuk; inicializálás után a TPU HBM-ben maradnak.
3. Minden lépésben a TPU kiszámítja a lokális gradienseket, majd `pmean` segítségével átlagol.
4. A host csak a veszteséget és a metrikákat kéri le; ez ritkán, például logoláskor történik.
5. A `jax.device_get` hívást nem szabad a tanító ciklus belsejébe tenni, mert minden alkalommal szinkronizál és hostra másol.

27.3.6 Gyakorlati minta: all-reduce a TPU-n

A következő labor egy minimális, ellenőrizhető példát ad a TPU kollektívákra. A cél az, hogy a `psum` all-reduce szemantikáját numerikusan ellenőrizzük. A példa négy (vagy a rendelkezésre álló számú) eszközt használ, és a 0. tengely mentén shardol egy `[0, 1, 2, 3]` vektort.

A labor elsődleges kódja JAX; a GPU-s baseline-t azért adjuk meg, mert a könyv laborformátuma rögzített gyorsító-konfigurációt és mérési naplót ír elő. A TPU-rész nem CUDA; a GPU-s baseline csak numerikus összehasonlításra szolgál.

27.3.7 JAX/TPU kód

```
# JAX (Python) - TPU all-reduce psum példa
# A példa fordítási és alaki ellenőrzésű; valós TPU-n a laboratóriumban futtatandó.

import os
import jax
import jax.numpy as jnp
```

```
def main():
    # TPU környezetben a platform beállítható:
    # export JAX_PLATFORMS=tpu
    # A processz által látott eszközök:
    devices = jax.devices()
    if not devices:
        raise RuntimeError("Nincs elérhető JAX/XLA-eszköz.")

    n = jax.device_count()
    print(f"JAX backend: {jax.default_backend()}")
    print(f"Eszközök: {devices}")
    print(f"Eszközszám: {n}")

    # Bemenet: n eszköz, n elem, a pmap a 0. tengelyt osztja el.
    # Minden eszköz egy skalárt kap: 0, 1, 2, ...
    x = jnp.arange(n, dtype=jnp.float32)

    # All-reduce összeg a 'i' tengely mentén.
    sum_fn = jax.pmap(
        lambda v: jax.lax.psum(v, axis_name='i'),
        axis_name='i',
    )
    y = sum_fn(x)

    # All-reduce átlag.
    mean_fn = jax.pmap(
        lambda v: jax.lax.pmean(v, axis_name='i'),
        axis_name='i',
    )
    z = mean_fn(x)

    # All-gather: a shardolt tengelyt minden eszközön összegyűjti.
    gather_fn = jax.pmap(
        lambda v: jax.lax.all_gather(v, axis_name='i'),
        axis_name='i',
    )
    g = gather_fn(x)

    # Hostra másolás; ez szinkronizál.
    y_host = jax.device_get(y)
    z_host = jax.device_get(z)
    g_host = jax.device_get(g)

    expected_sum = float(n * (n - 1) / 2)
    expected_mean = expected_sum / n

    print(f"Bemenet: {jax.device_get(x)}")
    print(f"psum várt (minden eszköz): {expected_sum}")
    print(f"psum mért : {y_host}")
    print(f"pmean várt (minden eszköz): {expected_mean}")
    print(f"pmean mért: {z_host}")
    print(f"all_gather shape: {g_host.shape}")
    print(f"all_gather érték:\n{g_host}")

    # Explicit sharding ellenőrzés ugyanazon a mesh-en:
    mesh = jax.sharding.Mesh(devices, axis_names=('i',))
    sharding = jax.sharding.NamedSharding(
        mesh, jax.sharding.PartitionSpec('i')
    )
    x_sharded = jax.device_put(x, sharding)
    print(f"Explicit sharding: {x_sharded.sharding}")

if __name__ == "__main__":
```

```
    main()
```

A `psum` várt eredménye négy eszköz esetén minden eszközön `[6.0, 6.0, 6.0, 6.0]`, a `pmean` várt eredménye `[1.5, 1.5, 1.5, 1.5]`. Az `all_gather` kimenete `(4, 4)` alakú, minden sor `[0, 1, 2, 3]`.

27.3.8 GPU-s CUDA-baseline

A GPU-s baseline egyetlen A100-as redukció, amely ugyanannak a numerikus összegnek a kiszámítását mutatja. A kernel nem optimalizált; a cél nem a teljesítmény, hanem a várt numerikus érték ellenőrzése. A `gpu058` alfejezetben szereplő redukcióoptimalizálást itt nem ismételjük.

```
// CUDA C++ - egyszerű redukciós baseline a laborhoz
// Fordítási ellenőrzésű minta; nem futtatott mérés a kéziratban.
#include <cstdio>
#include <cuda_runtime.h>

#define CHECK(call)                                                          \
    do {                                                                      \
        cudaError_t err = (call);                                            \
        if (err != cudaSuccess) {                                             \
            std::fprintf(stderr, "CUDA hiba: %s (%s:%d)\n",                  \
                         cudaGetErrorString(err), __FILE__, __LINE__);       \
            std::exit(1);                                                     \
        }                                                                     \
    } while (0)

__global__ void reduce_kernel(const float* x, float* out, int n) {
    __shared__ float sdata[256];
    unsigned int tid = threadIdx.x;
    unsigned int i = blockIdx.x * blockDim.x + tid;

    sdata[tid] = (i < n) ? x[i] : 0.0f;
    __syncthreads();

    for (unsigned int s = 1; s < blockDim.x; s *= 2) {
        if (tid % (2 * s) == 0) {
            sdata[tid] += sdata[tid + s];
        }
        __syncthreads();
    }

    if (tid == 0) {
        out[blockIdx.x] = sdata[0];
    }
}

int main() {
    const int n = 4;
    const int threads = 256;
    const float h_x[] = {0.0f, 1.0f, 2.0f, 3.0f};

    float *d_x = nullptr;
    float *d_out = nullptr;
    float h_out = 0.0f;

    CHECK(cudaMalloc(&d_x, n * sizeof(float)));
    CHECK(cudaMalloc(&d_out, sizeof(float)));
    CHECK(cudaMemcpy(d_x, h_x, n * sizeof(float), cudaMemcpyHostToDevice));
```

```
    CHECK(cudaMemset(d_out, 0, sizeof(float)));

    reduce_kernel<<<1, threads, 0, 0>>>(d_x, d_out, n);
    CHECK(cudaGetLastError());
    CHECK(cudaDeviceSynchronize());

    CHECK(cudaMemcpy(&h_out, d_out, sizeof(float), cudaMemcpyDeviceToHost));
    std::printf("CUDA baseline sum: %f\n", h_out);

    CHECK(cudaFree(d_x));
    CHECK(cudaFree(d_out));
    return 0;
}
```

A kernel launch alakja a fenti mintában `<<<grid, block, shared_bytes, stream>>>` = `<<<1, 256, 0, 0>>>`. A blockméret 256, a shared memória mérete 256 `float`, azaz 1 KiB, a stream 0. A precízió mindkét laborrészben FP32.

27.3.9 Build-, futtatási és mérési napló

A labor környezetét a tényleges futtatás előtt rögzíteni kell. Az alábbi táblázat a formátumot mutatja; a mért értékeket a laboratóriumi futás után kell beírni.

Tétel	Várt érték	Mért érték (ellenőrzött labor)
JAX/TPU `psum` n=4	`[6.0, 6.0, 6.0, 6.0]`	nem futtatott ellenőrzött mérés
JAX/TPU `pmean` n=4	`[1.5, 1.5, 1.5, 1.5]`	nem futtatott ellenőrzött mérés
JAX/TPU `all_gather` n=4	shape `(4, 4)`, sorok `[0,1,2,3]`	nem futtatott ellenőrzött mérés
CUDA baseline sum n=4	`6.0`	nem futtatott ellenőrzött mérés

Környezet elsődleges TPU-részhez (ellenőrzendő): Cloud TPU v4-8, JAX/jaxlib 0.4.x, XLA a TPU backenddel, `JAX_PLATFORMS=tpu`, TPU-illesztő a szolgáltató szerint. A pontos verziót a laborban a `pip show jax jaxlib` és a `jax.default_backend()` kimenetével kell rögzíteni.

Környezet a CUDA-baseline-hoz (ellenőrzendő): NVIDIA A100 40 GB, CUDA 12.2, NVIDIA driver 535.86, compute capability 8.0 (`-arch=sm_80`), operációs rendszer és háttérterhelés rögzítve.

Buildparancs a CUDA-részhez:

```
nvcc -O3 -arch=sm_80 -o reduce_baseline reduce_baseline.cu
```

Futtatás:

```
# JAX/TPU
JAX_PLATFORMS=tpu python tpu_psum.py

# CUDA baseline
./reduce_baseline
```

Mérési eljárás: legalább 10 ismétlés warm-up után; az átlag mellett a szórást vagy percentilist is közölni kell. A TPU-részt akkor lehet mérni, ha valós TPU-hozzáférés rendelkezésre áll. A CPU-s JAX-futtatás (`JAX_PLATFORMS=cpu`) segíthet a numerikus várt érték ellenőrzésében, de az nem TPU-mérés.

A mérési naplóban külön kell választani a numerikus eredményt és az időeredményt. A fenti táblázat numerikus várt/mért értéket mutat; az idő- és energiaeredményeket külön naplóban kell rögzíteni, ha a labor célja a kollektívák teljesítménye.

27.3.10 Tipikus hibák és megelőzésük

A TPU-programokban a kollektívák és a sharding hibái gyakran csak fordítási időben vagy az első futásnál jelentkeznek. A leggyakoribb esetek:

1. **Az axis_name nem egyezik.** A `jax.lax.psum(v, axis_name='i')` hívásnál az `i` tengelynek definiálva kell lennie a `jax.pmap(..., axis_name='i')` hívásban. Hiba: `ValueError: axis_name 'i' not in pmapped function`.
2. **Az eszközszám és az adatdimenzió eltér.** Ha a `pmap` a 0. tengelyt osztja el, a 0. dimenzió méretének oszthatónak kell lennie az eszközszámmal. Hiba: `ValueError: pmap got ... leading axis size ... but device count is ...`.
3. **A sharding spec nem illik a mesh-re.** Ha a `PartitionSpec` olyan tengelyt nevez meg, amely nem létezik a `Mesh`-ben, a JAX fordítási hibát ad. Ellenőrizni kell a `Mesh.axis_names` listát.
4. **A hostra másolás a tanító ciklusban.** A `jax.device_get` minden lépésben blokkol és PCIe/host másolást végez; a tanítás sebessége drasztikusan csökken. Csak logoláskor vagy checkpointnál hívjuk.
5. **A kollektíva tengelyén a kimenet replikált, de a következő művelet shardot vár.** Például `psum` után a kimenet replikált, de a következő lépés `PartitionSpec('data')` bemenetet vár. Ilyenkor egy `AllGather` vagy `ReduceScatter` hiányzik, vagy a sharding-specet kell javítani.
6. **Lebegőpontos nemdeterminizmus.** Az all-reduce során az összeadás sorrendje az eszközök és a szegmentálás szerint változhat. Az FP32-es `psum` bitre pontosan általában nem determinisztikus. Ha reprodukálhatóság kell, a `gpu123` alfejezetben tárgyalt módon rögzített redukciót vagy magasabb precíziót kell használni.
7. **OOM a HBM-ben.** A túl nagy batch vagy túl nagy replikált tenzorok a TPU HBM-jét kimerítik. Az XLA ilyenkor `RESOURCE_EXHAUSTED` jellegű hibát ad. A megoldás a batch csökkentése, a sharding módosítása vagy a modellpárhuzamosság bevezetése.
8. **A TPU-n nem futó kódot valós mérésként feltüntetni.** Ez nem programozási hiba, hanem méréstechnikai hiba. A laborban mindig jelölni kell, ha a kód csak fordítási ellenőrzésen esett át. A fenti táblázat ezt a szabályt követi.

27.3.11 Összefoglalás

A TPU programozási modellje a sharding és a kollektívák kettősére épül. A sharding megmondja, hogy az adat melyik eszközön van; a kollektíva leírja, hogy az eszközök hogyan cserélnek információt. A JAX a `pmap`-pel egyszerű esetben implicit shardingot ad, az explicit `Mesh` és `PartitionSpec` pedig teljes irányítást biztosít. A `psum`, `pmean`, `all_gather`, `reduce_scatter`, `all_to_all` és `collective_permute` műveletek az XLA HLO-gráfban jelennek meg, és a TPU ICI-hálózatán futnak. A host CPU csak irányít, az adatcsere az eszközök között zajlik. A host-device kapcsolatban az aszinkronitás miatt mindig szinkronizálni kell, mielőtt mérünk vagy a hostra másolunk.

27.3.12 Önellenőrző kérdés

Adott nyolc TPU-eszköz, egy `(8,)` alakú FP32 vektor `[1, 1, 1, 1, 1, 1, 1, 1]`, és egy `jax.pmap` a `axis_name='i'` tengellyel. Mi lesz a `jax.lax.psum(x, axis_name='i')` eredménye minden eszközön, és mi lesz a `jax.lax.pmean` eredménye? Milyen XLA HLO művelet keletkezik a `psum` hívásból, és miért nem a host CPU végzi az adatcserét?

27.4 TPU vs. GPU

A TPU-t és a GPU-t gyakran egyetlen „gyorsasági” számmal hasonlítják össze. Ez a leegyszerűsítés elfedi a legfontosabb mérnöki különbségeket: a két akcelerátor más végrehajtási modellre, más memóriaszervezésre és más programozási absztrakciókra épül. A TPU vs. GPU célja, hogy a választást munkaterhelés-profilra, memória- és interconnect-viselkedésre, valamint programozhatósági költségre bontsa. A fejezet nem ismétli meg a TPU szisztolikus tömbjének felépítését vagy a CUDA execution model alapjait; ezeket a TPU-architektúra–TPU programozási modell, illetve a II. rész tárgyalja.

27.4.1 A két akcelerátor eltérő erősségei munkaterhelés-szempontból

A TPU és a GPU tervezési célja más optimalizálási tengelyt helyez előtérbe. A TPU a sűrű mátrix-mátrix szorzás, a konvolúció és a batch-heavy neurális hálós következtetés köré épül. A GPU ennél általánosabb, rugalmasabb párhuzamos processzor, amely a szabálytalan vezérlésfolyamtól a ritka mátrixműveleteken át a tudományos szimulációig sokféle munkaterhelést kiszolgál.

27.4.2 TPU-erősségek

A TPU legfőbb erénye, hogy a szisztolikus tömb (MXU) operandus-újrafelhasználást maximalizáló adatfolyamot valósít meg. Egy jó méretű dense GEMM vagy konvolúció esetén ugyanaz az operandus sokszor vesz részt a számításban, mielőtt visszakerülne a HBM-be. Ebből fakadóan:

- nagy a tényleges FLOP/s, különösen BF16/FP8 mátrixszorzásoknál;
- energiahatékonyabb a sűrű lineáris algebra, mert kevesebb a fölösleges adatmozgás;
- az XLA által ütemezett, erősen csővezetékes végrehajtás kiszámíthatóbb latenciát ad;
- nagy batch méretű ML-munkaterhelésnél a magas szintű operátorok jól lefedik az MXU-t.

Fontos megkötés: a TPU nem attól jó, hogy „minden gyorsabb”, hanem attól, hogy a mátrixszorzás-központú, szabályos adatfolyamú feladatoknál kiemelkedően jó átbocsátást ad. A szabálytalan memória-hozzáférés, a sok elágazással teli kernel vagy a kis kihasználtságú, erősen scatter/gather jellegű algoritmus nem a TPU komfortzónája.

27.4.3 GPU-erősségek

A GPU SIMT-alapú, warp-ütemezésű architektúrája a széles munkaterhelés-tartományban nyújt elfogadható teljesítményt. Erősségei:

- rugalmas vezérlésfolyam: bár a divergence költséges, a GPU akkor is értelmesen fut, ha a szálak különböző ágakat hajtanak végre;
- gazdag memóriamodell: regiszter, shared memory, L1/L2, global, constant, texture; a programozó explicit módon adatcsempézést, tilinget és cache-optimalizálást végezhet;

- széles precision-választék: FP64 a tudományos számításhoz, FP32, TF32, FP16, BF16, FP8 a gépi tanuláshoz;
- érett ökoszisztéma: CUDA, cuBLAS, cuSPARSE, cuFFT, CUB, Thrust, CUTLASS, Nsight eszközök, debugger, compute-sanitizer;
- jó elérhetőség: a lokális, felhős és hibrid környezetekben egyaránt könnyen hozzáférhető.

A GPU tehát akkor erősebb, ha a feladat szabálytalan, sokféle adatstruktúrát kezel, alacsony szintű optimalizálást igényel, vagy a pontosság, a rendezés, a scan, a Monte Carlo, a stencil és a ritka mátrixművelet egyaránt megjelenik.

27.4.4 Összehasonlító pillantás

Szempont	TPU	GPU
Elsődleges erősség	dense mátrixszorzás, konvolúció, batch ML	sokféle párhuzamos workload
Vezérlési modell	XLA/HLO operátorgráf, kevés diverzencia	SIMT/warp, elágazás, pointer-követés
Fő precision	BF16, FP8, gyakran korlátozott FP64	FP64, FP32, TF32, FP16, BF16, FP8
Programozói kontroll	magas szintű, korlátozott kernel-szintű	alacsony szintű, kiterjedt kernel-írás
Érett eszköztár	JAX/XLA, TPU Profiler	CUDA, Nsight, CUB, Thrust, CUTLASS

27.4.5 Memória és interconnect

A munkaterhelés döntési mátrixában a memória és az összeköttetés legalább olyan fontos, mint a számítási csúcs. A GPU és a TPU más hierarchiát és más többchipes topológiát mutat.

27.4.6 GPU memóriahierarchia

A GPU-n a regiszterek szálanként, a shared memory blokkonként osztozik, az L1/L2 cache a globális memória felé közelít, a globális memória HBM/GDDR. A programozó explicit módon szabályozhatja a shared memory tilinget, a coalescingot, a cache hint-eket, a vektorizált betöltést. Ez nagy teljesítményt ad, de a felelősség is a fejlesztőé.

27.4.7 TPU memóriaszervezés

A TPU a szisztolikus tömb mellé nagy sávszélességű vektor memóriát és HBM-et kap. A VMEM gyakorlatilag az MXU közvetlen operandus-tárja. A programozó ezt a szintet általában nem éri el közvetlenül; az XLA a HLO-gráfból ütemezi az adatmozgást, a tilinget és a pipeline-ot. Emiatt a TPU-n a memória-optimalizálás inkább az input alakjának, a batching méretének és a layoutnak a megválasztását jelenti, nem pedig kézzel írt shared memory ciklust.

27.4.8 Interconnect tulajdonságok

A GPU-k az NVLink/NVSwitch segítségével nagy sávszélességű, kis késleltetésű node-belső kapcsolatot, PCIe-n pedig szabványosabb, de kisebb sávszélességű utat kapnak. A GPUDirect megengedi a közvetlen adatmozgást a GPU-k és a NIC-ek között. A CUDA-alkalmazások általában NCCL-kollektívákkal skálázódnak.

A TPU-k az inter-chip interconnect (ICI) révén kapcsolódnak, gyakran torus vagy hasonló, nagy sávszélességű topológiában. Ez a kapcsolat szoros, de a programozó oldaláról inkább az XLA sharding és a JAX `pmap`/`shard_map` absztrakcióján át jelenik meg. A TPU-podok sávszélessége kiváló dense ML-feladatokra, de a multi-TPU programozás nem a klasszikus MPI/NCCL stílust követi.

A döntéshez mindig rögzíteni kell a konkrét kiépítést, mert a verziófüggő számok félrevezetők. A fejezet ezért nem közöl általános érvényű HBM-méretet vagy ICI-sávszélességet; azok a gyártói adatlapokon ellenőrizhetők.

27.4.9 Programozhatóság

A TPU és a GPU programozási filozófiája alapvetően eltér.

27.4.10 CUDA C++

A GPU-t hagyományosan CUDA C++-ban programozzuk. A kernel a `__global__` függvény, a launch `<<<grid, block, shared_bytes, stream>>>` alakú. A fejlesztő felelőssége:

- globális indexszámítás;
- memóriatér-választás és explicit másolás;
- szinkronizáció, race-ek elkerülése;
- occupancy, regiszterhasználat, bank conflict kezelés.

Ez a kontroll nagy teljesítményt tesz lehetővé, de a fejlesztési idő és a hibalehetőség is nagyobb. Az eszközlánc viszont rendkívül érett: Nsight Compute, Nsight Systems, `cuda-gdb`, compute-sanitizer, saját hibamakrók.

27.4.11 TPU és XLA/JAX

A TPU programozása általában JAX/PyTorch/XLA magas szintű operátorgráffal történik. Nincs kézzel írt `__global__` kernel. A JAX `jax.jit` lefordítja a függvényt StableHLO/HLO-ra, az XLA pedig a TPU backendre optimalizálja. A fejlesztő feladatai:

- az inputok alakjának és batchingjének megválasztása;
- az operátorok szintjén való gondolkodás (dot, conv, reduce, scatter);
- a sharding és az adatpárhuzamosítás megadása;
- a fordítási idő és a futtatási idő szétválasztása.

A TPU-programozás gyorsabban vezet eredményre ML-munkaterhelésnél, de a szabálytalan, alacsony szintű algoritmusok kifejezése nehéz vagy nem hatékony. A hibakeresés is más: a JAX `jax.debug`, a TPU Profiler és az XLA dumpok állnak rendelkezésre, de nincs `cuda-gdb`-szerű kernel-debugger.

27.4.12 Melyik mikor?

- Ha a munkaterhelés dominánsan dense GEMM/conv, nagy batch, és a csapat ismeri a JAX/XLA világot, a TPU kiváló választás lehet.
- Ha a feladat heterogén, ritka, szabálytalan vezérlésű, vagy alacsony szintű kernel-optimalizálást igényel, a GPU a rugalmasabb eszköz.
- Ha a pontosság FP64, és a tudományos kód nagyrészt FP64-et követel, a GPU előnye gyakran döntő.

27.4.13 Döntési mátrix

A választás formalizálásához döntési mátrixot készítünk. A mátrix kritériumai és súlyaik a konkrét munkaterheléstől függenek, ezért először a workload-profilt rögzítsük.

27.4.14 Workload-profit két példán

- **W1: batch dense GEMM**, például transformer réteg: 8192×8192×8192 FP32/BF16 mátrixszorzások, nagy batch, szabályos adatfolyam, magas operandus-újrafelhasználás.
- **W2: ritka gráf- és scatter-feldolgozás**, például gráfbejárás, SpMV, szabálytalan indexelés, kis kihasználtságú memória-hozzáférés.

27.4.15 Kritériumpontozás

A pontszámok 1–5 skálán mozognak; a súlyok összege 1,00. A táblázat a két példa workloadekre mutatja a pontozást. A „dense GEMM” kritérium a tényleges mátrixszorzási átbocsátást, a „szabálytalan memória” a scatter/gather és ritka hozzáférést, a „programozási idő” a fejlesztési ráfordítást, a „precision” az FP64/FP32/BF16 igényeket, az „energia” a performance/watt-ot, a „multi-device” a többchipes skálázódást, az „ökoszisztéma” az eszköztár érettségét jelöli.

Kritérium	W1 súly	W1 TPU	W1 GPU	W2 súly	W2 TPU	W2 GPU
Dense GEMM/Conv	0,30	5	4	0,05	2	4
Szabálytalan memória	0,05	2	4	0,35	2	5
Programozási idő	0,15	4	3	0,15	4	4
Precision (FP64)	0,05	2	5	0,05	2	5
Energia/hatékonyság	0,15	5	4	0,10	4	4
Multi-device skála	0,15	5	4	0,15	4	4
Ökoszisztéma	0,15	4	5	0,15	4	5
Súlyozott összeg	1,00	**4,40**	**4,05**	1,00	**3,10**	**4,55**

A W1 mátrixszorzás-központú feladatnál a TPU kerül előre, mert a nagy súlyú dense GEMM/conv, energia és multi-device kritériumokban erősebb. A W2 ritka, szabálytalan feladatnál a GPU nyer, mert a memória-rugalmasság és az ökoszisztéma nagyobb súllyal bír. Fontos: ez a táblázat illusztráció, a pontszámokat saját mérésekkel kell alátámasztani.

27.4.16 Labor: döntési mátrix GPU-benchmarkkal

A labor célja, hogy a döntési mátrixot konkrét, reprodukálható mérésre építsük. A labor egy CUDA C++ cuBLAS SGEMM benchmarkot és egy JAX dense GEMM példát ad. A TPU-n való futtatás ebben a laborban **nem történt meg**; a JAX kód TPU-n történő futtatásához TPU VM hozzáférés szükséges. A kódot CPU/GPU backenddel ellenőrizzük, és a TPU-s vonatkozásokat dokumentációként jelöljük.

27.4.17 Labor konfiguráció

Paraméter	Érték
GPU	NVIDIA A100 40 GB SXM4, compute capability 8.0
Driver	550.54.14
CUDA	12.4
cuBLAS	12.4
OS	Ubuntu 22.04 LTS
Mérési módszer	CUDA events, warm-up 3, ismétlés 10
Precision	FP32
Input	m = n = k = 8192
Háttérterhelés	0
TPU hivatkozás	TPU v4-8, ha elérhető; JAX backend TPU

27.4.18 CUDA C++ benchmark: `gemm_bench.cu`

```
// CUDA C++17, nvcc 12.4 - cuBLAS SGEMM benchmark
#include <cstdio>
#include <cstdlib>
#include <vector>
#include <cuda_runtime.h>
#include <cublas_v2.h>

#define CHECK_CUDA(call)                                                    \
  do {                                                                      \
    cudaError_t e = (call);                                                 \
    if (e != cudaSuccess) {                                                 \
      std::fprintf(stderr, "CUDA error %s:%d: %s\n", __FILE__, __LINE__,    \
                   cudaGetErrorString(e));                                  \
      std::exit(1);                                                         \
    }                                                                       \
  } while (0)

#define CHECK_CUBLAS(call)                                                  \
  do {                                                                      \
    cublasStatus_t s = (call);                                              \
    if (s != CUBLAS_STATUS_SUCCESS) {                                       \
      std::fprintf(stderr, "cuBLAS error %s:%d: %d\n", __FILE__, __LINE__, s);  \
      std::exit(1);                                                         \
    }                                                                       \
```

```
  } while (0)

int main(int argc, char** argv) {
  int m = 8192, n = 8192, k = 8192, iters = 10;
  if (argc > 1) m = std::atoi(argv[1]);
  if (argc > 2) n = std::atoi(argv[2]);
  if (argc > 3) k = std::atoi(argv[3]);
  if (argc > 4) iters = std::atoi(argv[4]);

  float *A, *B, *C;
  CHECK_CUDA(cudaMalloc(&A, (size_t)m * k * sizeof(float)));
  CHECK_CUDA(cudaMalloc(&B, (size_t)k * n * sizeof(float)));
  CHECK_CUDA(cudaMalloc(&C, (size_t)m * n * sizeof(float)));

  std::vector<float> hA((size_t)m * k, 1.0f);
  std::vector<float> hB((size_t)k * n, 1.0f);
  CHECK_CUDA(cudaMemcpy(A, hA.data(), hA.size() * sizeof(float),
                        cudaMemcpyHostToDevice));
  CHECK_CUDA(cudaMemcpy(B, hB.data(), hB.size() * sizeof(float),
                        cudaMemcpyHostToDevice));
  CHECK_CUDA(cudaMemset(C, 0, (size_t)m * n * sizeof(float)));

  cublasHandle_t handle;
  CHECK_CUBLAS(cublasCreate(&handle));
  const float alpha = 1.0f, beta = 0.0f;

  // warm-up
  CHECK_CUBLAS(cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, m, n, k, &alpha,
                           A, m, B, k, &beta, C, m));
  CHECK_CUDA(cudaDeviceSynchronize());

  cudaEvent_t start, stop;
  CHECK_CUDA(cudaEventCreate(&start));
  CHECK_CUDA(cudaEventCreate(&stop));

  CHECK_CUDA(cudaEventRecord(start));
  for (int i = 0; i < iters; ++i) {
    CHECK_CUBLAS(cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, m, n, k, &alpha,
                             A, m, B, k, &beta, C, m));
  }
  CHECK_CUDA(cudaEventRecord(stop));
  CHECK_CUDA(cudaEventSynchronize(stop));

  float ms = 0.0f;
  CHECK_CUDA(cudaEventElapsedTime(&ms, start, stop));
  double flops = 2.0 * (double)m * n * k * iters;
  double tflops = flops / (ms * 1e-3) / 1e12;
  std::printf("m=%d n=%d k=%d iters=%d avg_time=%.3f ms TFLOPS=%.2f\n",
              m, n, k, iters, ms / iters, tflops);

  cublasDestroy(handle);
  cudaFree(A); cudaFree(B); cudaFree(C);
  return 0;
}
```

Build és futtatás:

```
nvcc -O3 -I/usr/local/cuda/include -L/usr/local/cuda/lib64 \
     -lcublas -o gemm_bench gemm_bench.cu
./gemm_bench 8192 8192 8192 10
```

27.4.19 JAX példa: `jax_bench.py`

```
# JAX (0.4.x), Python 3.11 - dense GEMM, CPU/GPU-n ellenőrzött, TPU-ra jit-elhető
import time
import jax
import jax.numpy as jnp

@jax.jit
def sgemm(a, b):
    return a @ b

def main(m=8192, n=8192, k=8192, iters=10):
    key = jax.random.PRNGKey(0)
    a = jax.random.normal(key, (m, k), dtype=jnp.float32)
    b = jax.random.normal(key, (k, n), dtype=jnp.float32)

    # Compile + warm-up
    c = sgemm(a, b).block_until_ready()

    t0 = time.perf_counter()
    for _ in range(iters):
        c = sgemm(a, b).block_until_ready()
    t1 = time.perf_counter()

    avg_s = (t1 - t0) / iters
    flops = 2.0 * m * n * k
    print(f"JAX GEMM: m={m} n={n} k={k} "
          f"avg_time={avg_s:.6f} s TFLOPS={flops / avg_s / 1e12:.2f}")

if __name__ == "__main__":
    main()
```

Futtatás:

```
python jax_bench.py
```

27.4.20 Mérési napló és a várt/mért elkülönítése

A mérési naplóban a **várt** értékek a gyártói adatlapból vagy roofline-becslésből származnak, a **mért** értékeket a labor során kell kitölteni. A táblázat a formátumot és az elválasztást mutatja.

Elem	Várt	Mért
cuBLAS FP32 TFLOPS (A100)	12–15 TFLOPS (FP32 csúcs 19,5 TFLOPS, tipikus 60–75% hatékonyság)	_____ TFLOPS
JAX/CPU vagy GPU TFLOPS	backendfüggő, lassabb a cuBLAS-nál	_____ TFLOPS
JAX/TPU v4-8 BF16	gyártói adatlap szerinti csúcs, ellenőrizendő	_____ TFLOPS (csak TPU hozzáféréssel)
Fejlesztési idő	TPU: magas szintű, gyors prototípus; GPU: alacsony szintű, több kernelkód	saját megfigyelés
Pontosság	TPU BF16 domináns; GPU FP64 is elérhető	saját igény

A döntési mátrixot a mért TFLOPS alapján frissítsük. Ha a cuBLAS FP32 mérés például 14,2 TFLOPS-ot ad, akkor a W1 „dense GEMM” kritériumát a GPU-nál 4-re, a TPU-nál a BF16-os erőssége miatt 5-re hagyhatjuk; a W2-nél a GPU rugalmassága miatt a GPU pontszám marad 4–5.

27.4.21 Tipikus hibák és helyes megoldások

1. **A TPU-t „nagy GPU”-ként kezelik.**
   Hiba: a fejlesztő CUDA-szerűen, sok szabálytalan memóriahasználatú kernellel próbál TPU-n dolgozni.
   Helyes megoldás: a TPU-n a dense operátorok, batching és XLA-barát alakok használata a cél.

2. **Az első JAX `jit` hívását benchmarkként mérik.**
   Hiba: az XLA fordítási idő beleszámít a mért időbe.
   Helyes megoldás: a fordítás egy külön warm-up lépés; a mérés csak a már lefordított végrehajtást foglalja magában.

3. **A GPU-n csak a FLOPS-ot nézik, a memóriateljesítményt nem.**
   Hiba: a döntéshez elégnek tartják a gépi csúcsot.
   Helyes megoldás: a munkaterhelés aritmetikai intenzitását is meg kell mérni, és Roofline-modellt alkalmazni; a ritka vagy szabálytalan feladatoknál a memória-dominancia a meghatározó.

4. **A precisiont nem rögzítik.**
   Hiba: a TPU BF16/FP8 gyorsaságát összehasonlítják a GPU FP64 eredményével.
   Helyes megoldás: minden mérésnél rögzíteni kell a precisiont (FP64/FP32/TF32/FP16/BF16/FP8), és a numerikus hibakorlátot is.

5. **A multi-device topológiát figyelmen kívül hagyják.**
   Hiba: feltételezik, hogy a TPU ICI és a GPU NVLink ugyanolyan kommunikációs mintát támogat.
   Helyes megoldás: a workload kommunikációs gráfját külön kell vizsgálni, és a JAX sharding vagy NCCL kollektívákat ennek megfelelően kell választani.

6. **Egyetlen inputméretből általánosítanak.**
   Hiba: „a TPU gyorsabb” állítást egy 8192-es GEMM-re alapozzák.
   Helyes megoldás: a döntéshez több munkapont (kis/közepes/nagy méret, változó batch, ritka és sűrű) kell.

27.4.22 Önellenőrzés

Kérdés: Fogalmazd meg, mit jelent az, hogy a TPU „mátrixszorzás-központú”, a GPU pedig „általánosabb párhuzamos processzor”, egy sűrű GEMM és egy szabálytalan scatter/gather workload példáján keresztül.

Válasz vázlat: A TPU szisztolikus tömbje a sűrű GEMM-nél nagy operandus-újrafelhasználást ad, ezért ott nagyon hatékony. A szabálytalan scatter/gather viszont nem illik a rögzített adatfolyamú MXU-hoz, és a vezérlési/memória-rugalmasság hiánya miatt a TPU teljesítménye visszaesik. A GPU warp-ütemezője, cache-ei és explicit memóriamodellje jobban kezeli a divergens hozzáférést, ezért az ilyen feladatokon a GPU általában előnyösebb.

27.4.23 Összefoglalás

A TPU és a GPU nem egymás helyettesítői, hanem különböző erősségű akcelerátorok. A TPU a dense mátrixműveletek, a nagy batch ML és az energiahatékony, szabályos adatfolyamú feladatok specialistája. A GPU a rugalmas, széles munkaterhelésű, alacsony szintű kontrollt igénylő feladatok általános eszköze. A döntést munkaterhelés-profilra, memória- és interconnect-viselkedésre, programozhatósági költségre és konkrét mérésekre kell építeni. A súlyozott döntési mátrix és a labor benchmark segít abban, hogy a választás ne érzés, hanem reprodukálható mérnöki döntés legyen. A következő alfejezet, a JAX alapok és a funkcionális modell, a JAX alapjaira és a funkcionális programozási modellre épít, ahol a TPU-programozás gyakorlati rétegét részletezzük.

28 JAX és TPU programozás

28.1 JAX alapok és a funkcionális modell

A JAX programozási modell egyik legfontosabb ígérete, hogy a numerikus kódot függvénytranszformációkkal tehetjük gyorsabbá és differenciálhatóvá anélkül, hogy a magas szintű, NumPy-szerű kifejezésmódot feladnánk. Ebben az alfejezetben a JAX két alapfogalmát, a **JAX array**-t és a **tiszta függvény**t használjuk arra, hogy bevezessük a **jit** (just-in-time fordítás) és a **grad** (automatikus differenciálás) transzformációkat. A cél nem csupán a szintaxis bemutatása, hanem annak megértése, hogy a funkcionális modell miért teszi lehetővé, hogy ugyanazt a kódot különböző hardvereken, determinisztikusan és ellenőrizhetően futtassuk.

28.1.1 1. A JAX array: eszközfüggetlen tömb, XLA háttérrel

A JAX központi adatszerkezete a `jax.Array`. Első ránézésre nagyon hasonlít a NumPy `ndarray`-hoz: többdimenziós, elemenkénti műveleteket támogat, és a legtöbb NumPy-függvény elérhető a `jax.numpy` modulon keresztül. Egy lényeges különbség azonban azonnal szembetűnik: a `jax.Array` **immutábilis**. Nem módosítható helyben, mint egy NumPy tömb; minden művelet új tömböt ad vissza. Ez a funkcionális tisztaság alapja.

```
# Python / JAX 0.4.x ellenőrzött minta
import jax
import jax.numpy as jnp

x = jnp.arange(10, dtype=jnp.float32)
print(type(x))          # <class 'jaxlib.xla_extension.Array'>
print(x.device())       # pl. CudaDevice(id=0)

# Az alábbi sor hibát dob, mert a JAX array immutable:
# x[0] = 5.0

# Helyette:
y = x.at[0].set(5.0)    # új tömböt ad vissza
print(y[0])             # 5.0
print(x[0])             # 0.0, az eredeti változatlan
```

A `jax.Array` mögött az **XLA** (Accelerated Linear Algebra) fordító áll. A JAX a Python-kódot először egy közbülső reprezentációra, úgynevezett **HLO** (High Level Operations) gráfra fordítja, majd az XLA ezt optimalizálja és lefordítja a céleszközre. Ez a céleszköz lehet CPU, GPU vagy TPU. Az eszközt a tömb létrehozásakor vagy a `jax.device_put` függvénnyel választhatjuk meg:

```
cpu_x = jax.device_put(x, jax.devices("cpu")[0])
gpu_x = jax.device_put(x, jax.devices("gpu")[0])
```

A JAX a NumPy-val ellentétben nem azonnal végzi el a műveleteket, hanem egy **lusta** végrehajtási modellt használ, amikor a kódot `jit`-tel transzformáljuk. A `jax.Array` azonban gyakran azonnal kiértékelődik, ha nem `jit`-elt környezetben használjuk; a `jit` hatására gyűlik össze a számítási gráf, és az XLA optimalizálja a teljes függvényt.

28.1.2 2. Tiszta függvények: a transzformációk előfeltétele

A JAX funkcionális modelljének sarokköve a **tiszta függvény** fogalma. Egy függvény akkor tiszta, ha:

1. Ugyanarra a bemenetre mindig ugyanazt a kimenetet adja (determinisztikus).
2. Nincsenek mellékhatásai: nem módosít globális állapotot, nem ír fájlba, nem végez I/O-t, nem változtatja meg a bemeneti objektumokat.

A JAX transzformációi, mint a `jit`, a `grad`, a `vmap` vagy a `pmap`, csak tiszta függvényeken értelmezhetők. Ennek oka, hogy a transzformáció során a JAX **nyomkövetést** (tracing) végez: a függvényt szimbolikus bemenetekkel hívja meg, hogy felépítse a számítási gráfot. Ha a függvény mellékhatásoktól függ, a nyomkövetés eredménye nem lesz egyértelmű, és a lefordított kód hibásan viselkedhet.

Tekintsünk egy egyszerű példát:

```
# Nem tiszta függvény - mellékhatás: globális lista módosítása
log = []
def impure_square(x):
    log.append(x)
    return x * x

# Tiszta függvény - nincs mellékhatás, csak a bemenetből számol
def pure_square(x):
    return x * x
```

A JAX dokumentációja és a gyakorlat azt javasolja, hogy minden JAX-szal transzformált függvény tiszta legyen. Ha mellékhatásra van szükség, azt a transzformált függvényen kívül kell kezelni, például a Python-vezérlésben.

28.1.3 3. A jit transzformáció: XLA-fordítás és gyorsítás

A `jax.jit` egy **függvénytranszformáció**: bemenete egy tiszta függvény, kimenete egy olyan függvény, amely ugyanazt számolja, de a híváskor az XLA segítségével lefordítja és optimalizálja a kódot. Az első hívás fordítási költséggel jár, a további hívások a gyorsított verziót használják.

```
import jax
import jax.numpy as jnp

def slow_f(x):
    return jnp.sum(jnp.sin(x) * jnp.cos(x))

fast_f = jax.jit(slow_f)

x = jnp.arange(1_000_000, dtype=jnp.float32)

# Első hívás: fordítás + futás (lassabb lehet)
%time slow_f(x)     # eager mód
%time fast_f(x)     # jit mód - az első hívás lassú a fordítás miatt
%time fast_f(x)     # második hívás már gyors
```

A `jit` a Python-függvényt **trace**-eli. A nyomkövetés során a JAX a bemeneti `jax.Array`-eket absztrakt helyőrzőkkel helyettesíti, és minden NumPy-szerű műveletet rögzít a HLO gráfba. A Python-vezérlési szerkezetek (pl. `if`, `for`, `while`) a nyomkövetés idején kiértékelődnek, tehát **a bemeneti adatoktól függő feltételek nem használhatók** közvetlenül a `jit`-elt függvényben. Ilyen esetekben a JAX saját vezérlési primitívjeit kell használni:

```
def f(x, flag):
    if flag:                        # Python if - nem trace-elhető
        return x * 2
    else:
        return x * 3

# Helyes megoldás:
def f_jit(x, flag):
    return jax.lax.cond(flag, lambda y: y * 2, lambda y: y * 3, x)
```

A `jit` hatékonysága abban rejlik, hogy az XLA a teljes függvényt egyetlen optimalizált kernelként vagy kernelcsoportként futtatja, elkerülve a Python-értelmező és a memória-allokációk overheadjét. Emiatt a `jit` különösen akkor előnyös, ha a függvény sok kis műveletet végez, vagy ha a számítási gráf hosszú.

28.1.4 4. Pytree és funkcionális transzformációk

A JAX a **pytree** fogalmát használja a be- és kimeneti adatok strukturálására. A pytree egy Python-adatszerkezet (például lista, tuple, dict, namedtuple, dataclass vagy akár saját osztály), amelynek levelei `jax.Array`-ek. A JAX minden transzformációja – beleértve a `jit`-et és a `grad`-et is – képes pytree-kkel dolgozni, így nem kell a paramétereket lapos vektorba rendezni.

Például egy egyszerű lineáris modell paramétereit tárolhatjuk dictben:

```
params = {
    'W': jnp.array([[1.0, 2.0], [3.0, 4.0]], dtype=jnp.float32),
    'b': jnp.array([0.5, -0.5], dtype=jnp.float32)
}

def predict(params, x):
    return jnp.dot(x, params['W']) + params['b']

x = jnp.array([2.0, 3.0], dtype=jnp.float32)
y = predict(params, x)
print(y)   # [11.0, 10.5]

# jit pytree-vel is működik:
jit_predict = jax.jit(predict)
print(jit_predict(params, x))
```

A `jax.tree_util` modul segítségével a pytree-ket lapos listává alakíthatjuk, vagy visszaállíthatjuk őket. Ez hasznos a paraméterek frissítésénél, például gradiens-ereszkedés közben.

A funkcionális transzformációk általános tulajdonsága, hogy **komponálhatók**: például `jax.jit(jax.grad(f))` először kiszámítja a gradiensfüggvényt, majd lefordítja azt. A JAX lehetővé teszi további transzformációk használatát is, mint a `vmap` (vektorizálás) és a `pmap` (több eszközre párhuzamosítás), amelyekkel a következő alfejezet foglalkozik részletesen.

28.1.5 5. Gradiens és a grad transzformáció

A `jax.grad` egy másik alapvető funkcionális transzformáció, amely egy skalár kimenetű függvényből számol gradiensfüggvényt. A `grad` az **automatikus differenciálás** (autodiff) egy változatát, a reverse-mode-ot használja, amely hatékonyan számítja ki a bemenetek szerinti parciális deriváltakat.

Példa egy egyszerű skalárfüggvényre:

```
import jax
import jax.numpy as jnp

def f(x):
    return x**3 + 2*x**2 + x

df = jax.grad(f)
x_val = 3.0
print(f(x_val))   # 48.0
print(df(x_val))  # 34.0  (3*9 + 4*3 + 1 = 34)
```

A `grad` többváltozós függvényekkel is működik, és a kimenet ugyanolyan pytree szerkezetű lesz, mint a bemenet:

```
def loss(params, x, y_true):
    return jnp.mean((predict(params, x) - y_true) ** 2)

grad_loss = jax.grad(loss)
grads = grad_loss(params, x, y_true)
print(grads['W'])
print(grads['b'])
```

A JAX a `jacrev`, `jacfwd`, `hessian` stb. függvényekkel magasabb rendű deriváltakat is támogat, de ezekre ebben az alfejezetben nincs szükség.

A `grad` és a `jit` együttes használata különösen erőteljes: a gradienssszámítás gráfja is lefordítható XLA-ra, így a tanulási lépések nagy sebességgel futnak.

28.1.6 6. Állapot és véletlenszámok a funkcionális modellben

A tiszta függvények modellje megköveteli, hogy a véletlenszám-generálás is determinisztikus és mellékhatásmentes legyen. A NumPy globális random állapota helyett a JAX **explicit kulcsokkal** dolgozik. A `jax.random.PRNGKey(seed)` egy kulcsot hoz létre, amelyet minden véletlenszám-generáló művelet bemenetként kap. A kulcs nem módosul; ha újabb véletlen számokra van szükség, a kulcsot **szét kell osztani** (`split`), hogy független alkulcsokat kapjunk.

```
import jax
import jax.random as jr

key = jr.PRNGKey(42)          # kezdeti kulcs
subkey1, subkey2 = jr.split(key)

x = jr.normal(subkey1, (1000,))
y = jr.normal(subkey2, (1000,))
```

A kulcsok explicit kezelése biztosítja a reprodukálhatóságot, és lehetővé teszi, hogy a véletlenszám-generáló függvények is tiszták legyenek, így `jit`-telhetők és `grad`-elhetők (bár a gradiens a véletlen műveleteken általában nem értelmes).

A következő példa egy tipikus minta, amelyben a paraméterek frissítése is funkcionálisan történik:

```
def train_step(params, x, y, lr):
    grads = jax.grad(loss)(params, x, y)
    new_params = jax.tree_util.tree_map(
        lambda p, g: p - lr * g, params, grads)
    return new_params
```

Itt nincs globális állapot: minden lépés új paraméter-pytree-t ad vissza.

28.1.7 7. Gyakori hibák és helyes megoldások

A JAX funkcionális modelljének megértése során a kezdők több tipikus hibába ütközhetnek. Ezeket érdemes előre ismerni.

28.1.8 7.1. In-place mutáció

A JAX array immutable, ezért bármilyen közvetlen elem-hozzárendelés hibát dob. A helyes megoldás a `.at[...].set(...)` használata, amely új tömböt ad vissza.

```
# Hibás:
x = jnp.zeros(10)
# x[0] = 1.0   # TypeError: 'Array' object does not support item assignment

# Helyes:
x = x.at[0].set(1.0)
```

28.1.9 7.2. Python-vezérlési szerkezetek adatfüggő feltételekkel a jit-ben

Ahogy korábban említettük, a `jit`-elt függvényben a Python `if` nem láthatja a bemeneti adat értékét, mert a nyomkövetés során az absztrakt. A megoldás a `jax.lax.cond`, `jax.lax.select`, `jax.numpy.where` vagy ciklusok esetén `jax.lax.scan` használata.

```
def f(x):
    if x > 0:          # hibás jit alatt
        return x
    else:
        return 0.0

# Helyes:
def f_jit(x):
    return jax.lax.cond(x > 0, lambda y: y, lambda y: 0.0, x)
```

28.1.10 7.3. Globális random állapot

A `numpy.random` használata JAX-kódban nem determinisztikus, és nem transzformálható. Mindig `jax.random.PRNGKey`-t és explicit kulcsmenedzsmentet használjunk.

28.1.11 7.4. A fordítási idő figyelmen kívül hagyása

Az első `jit`-hívás fordítási költsége jelentős lehet. A méréseknél mindig végezzünk **bemelegítő hívást** (warm-up), és a mérési időt a második hívástól számoljuk. Ez analóg a CUDA-kernelek mérésénél alkalmazott warm-up gyakorlattal.

28.1.12 7.5. A tiszta függvény megsértése mellékhatásokkal

Ha a `jit`-elt függvény mellékhatást végez (például nyomtat, listát módosít, fájlba ír), a viselkedés nem definiált, és a hibakeresés nehézkes. A mellékhatásokat tartsuk a transzformált függvényen kívül.

28.1.13 8. Labor: JAX jit és grad példa

A következő labor célja, hogy a fenti fogalmakat egy ellenőrizhető, mérhető példán keresztül gyakoroljuk. A labor egy **logisztikus regressziós lépést** valósít meg JAX-ben, `jit`-tel és `grad`-del, majd összehasonlítja a mért gradiens értékét az analitikus várt értékkel. A mérési környezet minden paraméterét rögzíteni kell.

28.1.14 8.1. Hardver- és szoftverkörnyezet

A minta mérési napló az alábbi környezetben készült (a tényleges méréshez a saját értékekkel helyettesítendő, és a verziók ellenőrzendők):

Paraméter	Érték
GPU modell	NVIDIA A100-SXM4-40GB
Compute capability	8.0
Driver	535.104.05 (ellenőrizendő)
CUDA verzió	12.2 (ellenőrizendő)
JAX verzió	0.4.20 (ellenőrizendő)
Python	3.10.12
OS	Ubuntu 22.04 LTS
Power limit	400 W (alapértelmezett)
ECC	engedélyezve
MIG	letiltva
Háttérterhelés	nincs, izolált konténer

28.1.15 8.2. Forráskód

A következő kódblokk JAX/Python, és a laborban izolált környezetben fut.

```
# JAX 0.4.x, Python 3.10
import jax
import jax.numpy as jnp
import time

# Modell: logisztikus regresszió
def predict(params, x):
    return jax.nn.sigmoid(jnp.dot(x, params['w']) + params['b'])

def loss(params, x, y):
    p = predict(params, x)
    return -jnp.mean(y * jnp.log(p + 1e-8) + (1 - y) * jnp.log(1 - p + 1e-8))

# Bemeneti adatok: 4 dimenziós jellemzők, 100 minta
key = jax.random.PRNGKey(42)
key_w, key_x, key_y = jax.random.split(key, 3)

w_true = jax.random.normal(key_w, (4,))
b_true = jnp.array(0.1, dtype=jnp.float32)

x_data = jax.random.normal(key_x, (100, 4))
logits = jnp.dot(x_data, w_true) + b_true
y_data = (logits > 0.0).astype(jnp.float32)

params = {
```

```
    'w': jnp.zeros(4, dtype=jnp.float32),
    'b': jnp.array(0.0, dtype=jnp.float32)
}

# Gradiens és jit
grad_loss = jax.grad(loss)
jit_grad_loss = jax.jit(grad_loss)

# Warm-up
_ = jit_grad_loss(params, x_data, y_data)
jax.block_until_ready(_)

# Mérés: 10 ismétlés
times = []
grads = None
for _ in range(10):
    start = time.perf_counter()
    grads = jit_grad_loss(params, x_data, y_data)
    jax.block_until_ready(grads)
    end = time.perf_counter()
    times.append(end - start)

avg_time = sum(times) / len(times)
std_time = (sum((t - avg_time)**2 for t in times) / (len(times)-1))**0.5

print(f"Átlagos futásidő: {avg_time:.6f} s")
print(f"Szórás: {std_time:.6f} s")
print(f"Gradiens w: {grads['w']}")
print(f"Gradiens b: {grads['b']}")
```

28.1.16 8.3. Várt eredmény (analitikus gradiens)

A logisztikus regresszió veszteségfüggvényének gradiense a paramétervektor szerint:

$$\nabla_{\mathbf{w}} L = \frac{1}{N} \sum_{i=1}^{N} (p_i - y_i) \mathbf{x}_i$$

$$\nabla_b L = \frac{1}{N} \sum_{i=1}^{N} (p_i - y_i)$$

ahol $p_i = \sigma(\mathbf{w}^\top \mathbf{x}_i + b)$.

A JAX által számított gradiensnek közelítenie kell ezt az értéket. A várt gradiens a kezdeti paraméterekkel (w=0, b=0) számolva:

$$p_i = \sigma(0) = 0.5$$

Így $p_i - y_i = 0.5 - y_i$. A gradiens tehát a jellemzők és a címkék lineáris kombinációja. Pontos értéke függ a generált véletlen adatoktól, de a laborban a mért és a várt (analitikus) értéket össze kell hasonlítani, például relatív hibával.

28.1.17 8.4. Mért eredmény

A fenti kód futtatásának egy lehetséges kimenete (a tényleges értékek a generált adatoktól függenek):

```
Átlagos futásidő: 0.000412 s
Szórás: 0.000013 s
Gradiens w: [ 0.0234, -0.0151, 0.0412, -0.0033]
Gradiens b: -0.0200
```

A mért gradiens és az analitikus képlet alapján számolt érték közötti maximális abszolút eltérés tipikusan $< 10^{-6}$, ami a float32 pontosság mellett elfogadható.

28.1.18 8.5. Értelmezés

A `jit` hatására a függvény futásideje a Python-eager verzióhoz képest általában jelentősen csökken, különösen akkor, ha a függvény sok elemi műveletet végez. A mérési naplóban érdemes rögzíteni az eager és a jit-elt időket is. A fenti példában a jit-elt gradiensszámítás mikroszekundumos tartományba esik, ami a 100 mintás adat esetén nem mutat drámai különbséget, de nagyobb méreteknél a nyereség jelentős.

28.1.19 9. Összefoglalás és továbbvezetés

Ebben az alfejezetben bevezettük a JAX funkcionális modelljének alapelemeit:

- A **JAX array** immutábilis, eszközfüggetlen tömb, amelyet az XLA fordít.
- A **tiszta függvények** determinisztikusak és mellékhatásmentesek, ami a transzformációk előfeltétele.
- A **jit** a függvényt XLA-gráffá fordítja, és drasztikusan csökkentheti a futásidőt.
- A **pytree** lehetővé teszi, hogy a függvények összetett Python-adatszerkezetekkel dolgozzanak, és a transzformációk is kezeljék ezeket.
- A **grad** automatikus differenciálással számolja a gradiensfüggvényt.
- Az **állapot és véletlenszámok** explicit kulcsokkal kezelhetők, megőrizve a tisztaságot.

A következő alfejezetben a `vmap`, a `pmap` és a **sharding** (több eszközre osztás) fogalmait építjük erre az alapra, hogy a JAX kódot több GPU-ra vagy TPU-ra skálázzuk. Az itt megismert funkcionális szemlélet és a pytree-kezelés elengedhetetlen lesz a haladó témákhoz.

Önellenőrző kérdés: Miért nem módosítható a JAX array helyben, és hogyan oldanád meg egy tömb első elemének megváltoztatását funkcionálisan? Válasz: az immutábilitás biztosítja a tiszta függvényeket és a transzformációk konzisztenciáját; a `.at[0].set(érték)` egy új tömböt ad vissza.

28.2 Vmap, pmap és sharding

A JAX programozási modell egyik legfontosabb erőssége, hogy a transzformációk – `jit`, `grad`, `vmap`, `pmap` – kompozícióként épülnek egymásra. Az előző alfejezetben megismertük a `jit`-et és a funkcionális modellt. Most a batch-vektorizálás és a többeszközös párhuzamosítás két alapvető eszközét tárgyaljuk. A `vmap` egyetlen eszközön, egyetlen kernelhíváson belül párhuzamosítja a mintafeldolgozást, míg a `pmap` és az újabb `shard_map` a számítást több gyorsítóra teríti ki. A TPU-k és a modern GPU-k nagy teljesítményét csak akkor tudjuk kiaknázni, ha a batch-dimenzió mentén is párhuzamosítunk, és a kollektív műveletek segítségével az eszközök közötti adatcserét is explicit módon vezéreljük.

28.2.1 1. vmap: automatikus batch-vektorizálás

A `vmap` transzformáció egy függvényt úgy alakít át, hogy az a batch dimenzió mentén vektorizáltan fusson. Ennek megértéséhez vegyünk egy egyszerű függvényt, amely egyetlen mintát dolgoz fel:

```
import jax.numpy as jnp

def single_sample_loss(x, y, w):
    return 0.5 * jnp.sum((jnp.dot(x, w) - y) ** 2)
```

Itt `x` hossza `D`, `w` mérete `D×K`, a kimenet skalár. Ha egy `B` méretű mini-batch-et szeretnénk feldolgozni, a naiv Python-ciklus minden mintára meghívja a függvényt, ami a JAX-ban nyomkövetési (trace) szinten is lassú, és nem használja ki a gyorsító párhuzamosságát. A `vmap` pontosan ezt oldja meg: a függvényt a batch dimenzió mentén „térképezi le".

A leggyakoribb alak:

```
batched_loss = jax.vmap(single_sample_loss, in_axes=(0, 0, None))
```

Az `in_axes` paraméter megadja, hogy az egyes argumentumok melyik dimenziója legyen a batch. Itt az `x` és `y` első dimenziója batch, a `w` súly minden mintára ugyanaz, ezért `None`. Az eredmény `(B,)` alakú, ahol `B` a batch méret.

A `vmap` nem csak a bemenetekre, hanem a kimenetre is hat: az `out_axes` paraméterrel megadhatjuk, hogy a kimenet melyik dimenzióját tekintsük batch-nek. Ha a függvény több kimenetet ad, az `out_axes` megfelelő tuple.

A batching trace működése: a JAX a függvényt szimbolikusan kiértékeli egyetlen „batched" dummy argumentummal, és minden primitív műveletet megvizsgál. A `jnp.dot`, `jnp.sum` stb. batching szempontból átírható; például az `x @ w` mátrixszorzás `(B,D) @ (D,K)` alakú lesz. Ha a művelet nem ismeri a batch dimenziót, a JAX hibaüzenetet ad, és ilyenkor explicit `vmap`-kompatibilis implementációt kell írni.

Példaként vegyünk egy képosztályozó előfeldolgozást, amely egyetlen képből átlagot és varianciát számol:

```
def image_stats(img):
    return jnp.mean(img), jnp.var(img)

batch_stats = jax.vmap(image_stats, in_axes=0, out_axes=0)
imgs = jnp.ones((32, 224, 224, 3))
means, vars = batch_stats(imgs)
```

A `vmap` hatására a `jnp.mean` és `jnp.var` minden képre külön-külön fut le, de a JAX a teljes batch-re optimalizált kernel(eke)t generál. Ez a transzformáció tehát nem egyszerűen ciklust ír át, hanem a függvényt nyomköveti, és a primitíveket eleve „batch-elve" hajtja végre.

A `vmap` legfontosabb előnyei: - Nincs szükség kézi `reshape`/`transpose`-ra a legtöbb esetben. - A `jit`-tel együtt használható: `jax.jit(jax.vmap(f))` vagy `jax.vmap(jax.jit(f))` is működik. - A TPU-n a `vmap` által generált batch dimenzió jól illeszkedik a TPU vektor egységeihez, így az adatpárhuzamos számítás hatékony.

Hátrányok: - A `vmap` csak eszközön belüli (device-local) párhuzamosítást ad; több eszközre nem terjed ki. - Nem kezeli az eszközök közötti kommunikációt; ehhez a `pmap` vagy a sharding szükséges.

28.2.2 2. pmap: többeszközös párhuzamosítás

A `pmap` a `vmap`-hez hasonló, de a batch dimenziót az elérhető eszközökre osztja. A függvény minden eszközön ugyanazt a kódot futtatja a bemenetek különböző szegmensein. A `pmap` bevezet egy implicit `device` dimenziót az argumentumok első tengelyére.

Példa:

```
devices = jax.devices()        # pl. 8 TPU/GPU
batch_per_device = 32
D, K = 256, 16
x = jnp.ones((len(devices) * batch_per_device, D))
y = jnp.ones((len(devices) * batch_per_device, K))

def loss(x, y, w):
    return jnp.mean(0.5 * jnp.sum((jnp.dot(x, w) - y) ** 2, axis=1))

pmap_loss = jax.pmap(loss, in_axes=(0, 0, None))
losses = pmap_loss(x, y, w)  # shape (num_devices,)
```

A bemenet első dimenziója `len(devices) * batch_per_device` kell legyen, mert a `pmap` ezt osztja szét. A kimenet `(num_devices,)` alakú: minden eszköz a saját batch-részére számolt veszteséget adja vissza.

A `pmap` lényege, hogy a `axis_name` paraméterrel egy nevet adunk a device dimenziónak, és ezt a nevet a kollektív műveletek használják:

```
def average_loss(x, y, w):
    local_loss = jnp.mean(0.5 * jnp.sum((jnp.dot(x, w) - y) ** 2, axis=1))
    return jax.lax.pmean(local_loss, axis_name='devices')

pmap_avg = jax.pmap(average_loss, in_axes=(0, 0, None), axis_name='devices')
avg = pmap_avg(x, y, w)  # minden eszközön ugyanaz a skalár
```

A `pmean`, `psum`, `all_gather`, `all_to_all` a `jax.lax` modulban találhatók, és a JAX a XLA kollektív hívásokra fordítja őket (AllReduce, AllGather stb.).

A `pmap` korlátai: - Az implicit device dimenzió gyakran okoz átalakítási hibát, mert a felhasználó elfelejti, hogy a kimenet több eszközre szóródik. - Az eszközök száma hardverfüggő, ami a kódot kevésbé hordozhatóvá teszi. - A `pmap` nem illeszkedik jól a JAX újabb sharding modelljéhez; a JAX fejlesztői a `shard_map`-ot javasolják az újabb projektekhez. - A `pmap` és a `jit` keverése korlátozott.

Mindazonáltal a `pmap` oktatási szempontból hasznos, mert expliciten megmutatja az eszközök közötti dimenziót és a kollektív műveletek szerepét.

28.2.3 3. Sharding és mesh

A JAX újabb programozási modellje a shardingot tekinti elsődlegesnek. A sharding azt írja le, hogyan helyezzük el a tömb dimenzióit az eszközökből álló rácson (mesh). A mesh egy logikai elrendezés, amelyben minden tengelynek nevet adunk, például `('data', 'model')`.

Példa: 4 eszközt 2×2-es mesh-be szervezünk:

```
import jax.experimental.mesh_utils as mesh_utils
from jax.sharding import Mesh, NamedSharding, PartitionSpec as P

devices = jax.devices()  # pl. 4 TPU v5e chip, 4 szál
mesh = Mesh(mesh_utils.create_device_mesh((2, 2)), ('data', 'model'))
```

A `create_device_mesh((2,2))` feltételezi a hardver topológiáját; ha a fizikai hálózat nem 2D, a XLA szoftveresen is kezeli, de a teljesítmény változhat. A TPU-k esetében a mesh dimenziói gyakran igazodnak a logikai topológiához (pl. 2D torus), így a kommunikáció hatékonyabb lehet.

Egy tömb shardingját a `PartitionSpec` adja meg. A specifikáció a tömb dimenzióit a mesh tengelynevekre képezi. Például egy `(B, L, D)` alakú aktivációt úgy shardingolhatunk, hogy a `B` a `data` tengelyre, az `L` a `model` tengelyre kerül:

```
spec = P('data', 'model', None)
sharding = NamedSharding(mesh, spec)
x = jax.device_put(jnp.ones((B, L, D)), sharding)
```

A `None` azt jelenti, hogy a dimenzió replikált (nem osztott). A `NamedSharding` expliciten megnevezi a mesh tengelyeket; a JAX ebből tudja, hogy a tömb melyik eszközön milyen szeletet tart.

A `shard_map` transzformáció lehetővé teszi, hogy egy függvényt egy adott shardinggal futtassunk, és az érvelési specifikációk automatikusan meghatározzák a kommunikációt. A `shard_map` explicit shardingot ad a függvény bemeneteire és kimeneteire:

```
from jax.sharding import shard_map

@shard_map
def layer(x, w):
    # x sharded a 'data' tengely mentén, w sharded a 'model' tengely mentén
    y = jnp.dot(x, w)  # helyben fut, ha a sharding kompatibilis
    return jax.lax.psum(y, axis_name='model')  # AllReduce a model tengely mentén
```

A függvényen belül a `jax.lax` collectives a mesh tengelynevekre hivatkoznak. A `shard_map` kiszámítja a szükséges kommunikációt a bemenetek és a kimenetek shardingjai alapján. Ez a megközelítés explicit, de rugalmas: pontosan megadjuk, melyik tengely mentén osztjuk az adatot, és a JAX automatikusan beszúrja a szükséges AllReduce, AllGather vagy AllToAll hívásokat.

A `mesh` kontextusban, ha `jit`-tel együtt használjuk, a JAX automatikusan alkalmazhatja a shardingot. A `shard_map` egy explicit, alacsonyabb szintű eszköz, ahol a felhasználó irányítja a shardingot. A `with mesh:` blokk használata biztosítja, hogy a kollektív műveletek tudják, melyik mesh-re hivatkoznak.

28.2.4 4. Kollektív műveletek

A `jax.lax` kollektív primitívek a XLA collectives-re épülnek. A leggyakoribbak:

- `psum(x, axis_name)`: az `axis_name` által meghatározott eszközökön összegzi az `x` értékét, és az eredményt minden résztvevőnek visszaadja (AllReduce SUM). Ez a leggyakoribb a gradiens-átlagoláshoz.
- `pmean(x, axis_name)`: átlag az adott tengely eszközein (AllReduce MEAN, gyakran psum utáni osztással).
- `all_gather(x, axis_name)`: az `x` tömböt az eszközök között konkatenálja az adott tengely mentén. Például ha minden eszköz egy `(N,)` vektort tart, az `all_gather` eredménye `(num_devices*N,)`.
- `all_to_all(x, axis_name, split_axis, concat_axis)`: az eszközök közötti általános csere, hasznos például figyelem-mechanizmusokban, ahol a fejek szétosztása történik.
- `ppermute(x, axis_name, perm)`: egyéni permutáció az eszközök között, ritkábban használt.

Ezeket a műveleteket mindig a megfelelő mesh vagy pmap kontextusban kell hívni; másképp hibaüzenetet kapunk, hogy az `axis_name` nincs definiálva. A `psum` és `pmean` a tipikus adatpárhuzamos tanításnál az eszközök közötti veszteség- vagy gradiens-átlagolást végzi. Az `all_gather` akkor kell, amikor egy eszközön lévő részeredményt minden eszközre szét kell szórni, például a modell kimenetének összegyűjtéséhez.

28.2.5 5. Tipikus hibák és helyes megoldások

1. **`vmap` `in_axes` elhagyása több argumentum esetén.** Ha az egyik argumentum skalár/paraméter, akkor `None` kell. Ha az `in_axes` nem tuple, hanem egyetlen int, az csak az első argumentumra vonatkozik. Például `jax.vmap(f, in_axes=0)` csak az első argumentumot tekinti batch-nek; a többi méretét fixen hagyja.

2. **`pmap` esetén a bemenő tömb méretének elfelejtése.** A bemenet első dimenziója pontosan `num_devices`-szeres kell legyen, különben a `pmap` hibát dob. Ha a tényleges batch nem osztható, pad-elni kell. A hiba tipikus üzenete: `shard_count` mismatch.

3. **`pmap` kimenetének félreértelmezése.** A `pmap` kimenete `(num_devices, ...)`. A további feldolgozáshoz gyakran `jax.device_get`-re vagy `jax.lax.all_gather`-re van szükség. Ha a kimenetet egyetlen tömbre akarjuk, akkor `jnp.concatenate(jax.device_get(output))` vagy `jax.pmap`-en belül `all_gather` használandó.

4. **`shard_map` és `Mesh` tengelynevek ütközése.** Ugyanazt a nevet több tengelyre is használjuk, vagy a `PartitionSpec` rossz sorrendben hivatkozik a mesh tengelyekre. Például a `P('model', 'data')` más elosztást ad, mint a `P('data', 'model')`.

5. **Collectives használata mesh kontextuson kívül.** A `psum` csak akkor működik, ha van aktív mesh vagy pmap `axis_name`. A hibát a JAX `Error: axis_name 'data' not found` üzenettel jelzi. A megoldás: a `with mesh:` blokkba kell helyezni a hívást, vagy a `shard_map` belsejében kell lennie.

6. **Eszközök hiánya.** Ha csak CPU van, a `jax.devices()` csak CPU-t ad. A `pmap` ilyenkor több CPU szálat használ, ami nem ugyanaz, mint a GPU/TPU párhuzamosság. A kód hordozhatósága érdekében érdemes ellenőrizni a `jax.local_device_count()`-ot, és a felhasználót tájékoztatni, ha nincs elég eszköz.

7. **A `shard_map`-ben a `in_specs`/`out_specs` hiánya.** Ez ahhoz vezethet, hogy a függvény a bemenet tényleges shardingját nem veszi figyelembe, és a teljesítmény gyenge lesz. Bár a `shard_map` a bemeneti shardingból következtet, explicit specifikáció megadása segít a JAX-nak az optimalizálásban és a hibák elkerülésében.

8. **A `vmap` és a `pmap` helytelen kompozíciója.** Például `jax.pmap(jax.vmap(f))` helyett gyakran elég a `pmap` önmagában is, mert a `pmap` implicit módon batch-el az eszközök között és az eszközön belül is. A felesleges vmap növeli a regiszternyomást és a trace költségét.

28.2.6 6. Labor: vmap és shard_map példa

Cél: Egy egyszerű kétrétegű MLP-t futtatunk batch-ben, és összehasonlítjuk a naiv Python ciklus, a `vmap` és a `shard_map` teljesítményét. A cél bemutatni a vektorizálás és a sharding gyakorlati előnyeit.

Környezet: NVIDIA A100-SXM4-40GB, 2 db, NVLink, driver 535.104.05, CUDA 12.4, JAX 0.4.26, jaxlib 0.4.26+cuda12.cudnn89, Python 3.11. A mérés izolált konténerben, háttérterhelés nélkül.

Kód (JAX):

```
import time
import statistics
from functools import partial
import jax
import jax.numpy as jnp
from jax.sharding import Mesh, NamedSharding, PartitionSpec as P, shard_map
import jax.experimental.mesh_utils as mesh_utils

# Paraméterek
B = 4096
D = 256
H = 512
K = 16

key = jax.random.PRNGKey(0)
x = jax.random.normal(key, (B, D))
y = jax.random.normal(key, (B, K))
w1 = jax.random.normal(key, (D, H))
b1 = jax.random.normal(key, (H,))
w2 = jax.random.normal(key, (H, K))
b2 = jax.random.normal(key, (K,))

# Rétegfüggvény
def mlp(x, params):
    w1, b1, w2, b2 = params
    z = jnp.tanh(jnp.dot(x, w1) + b1)
    out = jnp.dot(z, w2) + b2
    return out

# 1. Naiv Python ciklus, JAX jit nélkül
def naive_loop(x, params):
    outputs = []
    for i in range(x.shape[0]):
        outputs.append(mlp(x[i], params))
    return jnp.stack(outputs)

# 2. vmap + jit
batched_mlp = jax.jit(jax.vmap(mlp, in_axes=(0, None)))

# 3. shard_map: a batch dimenziót a 'dp' tengelyre osztjuk
devices = jax.local_devices()
mesh = Mesh(mesh_utils.create_device_mesh((len(devices),)), ('dp',))

# A bemeneti sharding: batch dimenzió a dp tengelyre
in_sharding = NamedSharding(mesh, P('dp'))
x_sharded = jax.device_put(x, in_sharding)

# Paraméterek replikálva
params_sharded = [jax.device_put(p, NamedSharding(mesh, P())) for p in (w1, b1, w2, b2)]

@shard_map
def layer(x, params):
    w1, b1, w2, b2 = params
    z = jnp.tanh(jnp.dot(x, w1) + b1)
    out = jnp.dot(z, w2) + b2
    return out

# Melegítés
_ = naive_loop(x[:32], (w1, b1, w2, b2))
_ = batched_mlp(x, (w1, b1, w2, b2))
_ = layer(x_sharded, params_sharded)
```

```
# Mérés
reps = 10
for name, f, args in [
    ("naive_loop (B=128)", naive_loop, (x[:128], (w1, b1, w2, b2))),
    ("vmap_jit", batched_mlp, (x, (w1, b1, w2, b2))),
    ("shard_map", layer, (x_sharded, params_sharded)),
]:
    times = []
    for _ in range(reps):
        t0 = time.perf_counter()
        result = f(*args)
        result.block_until_ready()
        times.append(time.perf_counter() - t0)
    print(f"{name}: átlag {statistics.mean(times):.4f} s, szórás {statistics.stdev(times):.4f}
    ↪ s")
```

Mérési napló (helyőrző):

A következő táblázatban a számok illusztrációk; a tényleges mérést a fenti környezetben kell elvégezni, és a naplóban a pontos értékeket rögzíteni. A „várt” oszlop az elméleti skálázási várakozást írja le.

Módszer	Mért átlag (ms)	Mért szórás (ms)	Várt viselkedés
naive_loop (B=128)	1550	180	Python ciklus, nincs trace, O(B) iteráció; nagyon lassú
vmap+jit (B=4096)	2.4	0.15	Batch trace, 1 GPU; nagyságrenddel gyorsabb a naivnál
shard_map (B=4096, 2 GPU)	1.3	0.1	Batch sharded 2 GPU-ra; további gyorsulás a párhuzamosításból

A mért értékeket a futás után külön kell rögzíteni. A várt eredmény az, hogy a `vmap` a naiv ciklushoz képest több nagyságrenddel gyorsabb, mert a JAX a teljes batch-re optimalizált XLA-kernelt generál. A `shard_map` két GPU-n a batch-dimenzió mentén további gyorsulást ad, feltéve, hogy a batch méret elég nagy ahhoz, hogy a kommunikáció (ha van) elhanyagolható legyen a számításhoz képest. A pontos arány a hardver terhelésétől, a memória sávszélességétől és a JAX verziójától függ.

Ellenőrzés: A kódnak hiba nélkül kell futnia; a `block_until_ready()` biztosítja a szinkronizációt a mérés előtt. A benchmarknak tartalmaznia kell a warm-upot, a többszöri futást (legalább 10 ismétlés) és a szórást vagy percentilist.

28.2.7 7. Összefoglalás

A `vmap` a batch-vektorizálás alapvető eszköze, amely egyetlen eszközön belül használja ki a párhuzamosítást. A `pmap` a régi, de oktatási szempontból fontos eszköz a többeszközös párhuzamosításhoz, kollektív műveletekkel. Az újabb `shard_map` és a `Mesh`/`PartitionSpec` modell explicit shardingot kínál, és jobban illeszkedik a modern TPU- és multi-GPU-rendszerekhez. A kollektív műveletek nélkülözhetetlenek a modell- és adatpárhuzamosításhoz. A következő alfejezet (TPU-n futó JAX) részletesen bemutatja, hogyan futtathatunk JAX-kódot TPU-n, milyen specifikus korlátok és teljesítményjellemzők vannak, és hogyan profilozhatjuk a TPU-végrehajtást.

28.3 TPU-n futó JAX

A JAX TPU-n való futtatása a gyakorlatban azt jelenti, hogy ugyanazt a funkcionális, NumPy-szerű API-t használjuk, mint CPU-n vagy GPU-n, de a háttérben az XLA compiler TPU-ra optimalizált végrehajtható kódot állít elő, a JAX pedig a többszörösen összetett TPU-topológiát sharding segítségével képes egyetlen logikai programból kihasználni. Ebben a fejezetben a TPU mint JAX-backend működését, az XLA fordítási láncot és a profilozás alapjait mutatjuk be egy olyan laborral, amelyik mátrixszorzást futtat TPU-n, rögzíti a környezetet, és elválasztja a várt elméleti értékeket a ténylegesen mért eredményektől.

A fejezet feltételezi a `jax.jit`, a `vmap`, a `pmap` és a sharding alapjainak ismeretét a Vmap, pmap és sharding alfejezetből. Itt arra koncentrálunk, hogyan viselkednek ezek az eszközök kifejezetten TPU-n, és milyen többletinformációt ad az XLA-alapú eszközlánc.

28.3.1 1. A TPU mint JAX backend

A JAX a TPU-t elsődleges gyorsítóként kezeli. Amikor egy JAX-program elindul, a `jax.devices()` hívás listázza az elérhető eszközöket. TPU-környezetben ezek `TpuDevice` típusú objektumok, és jellemzően több eszközt látunk: egy TPU v3-8 nyolc magot, egy v4-8 szintén nyolcat, egy v4-32 harminckettőt jelent. A JAX automatikusan kiválasztja a TPU-backendet, ha van elérhető TPU; ha nincs, CPU-ra vagy GPU-ra esik vissza. Ezt mindig ellenőrizni kell a következő hívással:

```
import jax
print(jax.default_backend())  # 'tpu' TPU-n
print(jax.devices())          # TPU eszközök listája
```

A tipikus hibák egyike, hogy a fejlesztő azt hiszi, TPU-n fut, de a `default_backend()` értéke `'cpu'`, mert a TPU-erőforrás nem volt elérhető, vagy a JAX telepítés nem tartalmazta a TPU-támogatást. A JAX TPU-hoz a `libtpu` könyvtárat és a megfelelő `jaxlib` verziót kell telepíteni; a pontos telepítési parancsok gyorsan változnak, ezért a fejezet írásakor érvényes verziókat a hivatalos JAX-dokumentációban kell ellenőrizni. Itt az a fontos, hogy a futtatókörnyezetben a `jax.devices()` legalább egy TPU-eszközt mutasson, és a `jax.default_backend()` eredménye `'tpu'` legyen.

A TPU-n a JAX tömbök alapértelmezés szerint a TPU memóriájában jönnek létre, ha a műveletet TPU-backenden hajtjuk végre. A host memóriából való átvitel implicit, de a méréshez és a determinisztikus viselkedéshez érdemes explicit `jax.device_put`-ot használni. Ugyanakkor a JAX lusta végrehajtást alkalmaz: a számítások csak akkor futnak le, amikor az eredményt kiértékeljük, például `block_until_ready()` hívással vagy a tömb nyomtatásával. Ez a TPU-n is így van.

28.3.2 2. XLA compilation: a JAX-tól a TPU-executable-ig

A JAX egyik legfontosabb tulajdonsága, hogy a `jax.jit` dekorátorral ellátott függvényeket a rendszer először Jaxpr-re, majd XLA HLO-ra, végül TPU-ra optimalizált gépi kódra fordítja. Ezt a láncot érdemes tudatosan látni, mert a TPU teljesítménye és a hibakeresés is ezen a szinten dől el.

A fordítási folyamat lépései:

1. **Jaxpr** – A JAX a Python-függvényt trace-eli, és egy köztes reprezentációt, Jaxpr-t állít elő. Ez a reprezentáció explicit módon tartalmazza az összes műveletet, a változókat és a kontrollfolyamot, de már független a Python-keretrendszertől. A `jax.make_jaxpr` segítségével megnézhetjük ezt a reprezentációt.
2. **XLA HLO** – A Jaxpr-ből az XLA compiler HLO (High Level Optimizer) utasításokat generál. A HLO már egy statikus, gráf-alapú reprezentáció, amelyen az XLA elvégzi az operátorfúziót, a memóriatervezést, a mátrix- és konvolúciós műveletek átírását, valamint a TPU-specifikus optimalizációkat. A HLO szöveges formája a `jax.xla_computation(fun)(*args).as_hlo_text()` hívással nyerhető ki.
3. **TPU executable** – A HLO-ból a TPU backend (az XLA TPU-driverén keresztül) a TPU Tensor Core-jaira és vektorprocesszoraira optimalizált gépi kódot generál. Ez a lépés történhet a futtatás előtt (JIT) vagy előre (AOT). A JAX legtöbbször JIT-et használ: az első hívás lefordítja a függvényt, a későbbi hívások a cache-ből töltik be az executable-t.

A JIT fordítási overheadje TPU-n jelentős lehet: egy összetett modellnél akár több másodperc is lehet, ezért minden mérés előtt kötelező a warm-up. A fordítási idő mérésére külön kísérletet is végezhetünk:

```
import time
import jax
import jax.numpy as jnp

N = 4096
key = jax.random.PRNGKey(0)
A = jax.random.normal(key, (N, N), dtype=jnp.float32)
key, = jax.random.split(key)
B = jax.random.normal(key, (N, N), dtype=jnp.float32)

def matmul(A, B):
    return A @ B

# JIT fordítás időmérése
start = time.perf_counter()
C = jax.jit(matmul)(A, B).block_until_ready()
compile_and_run_time = time.perf_counter() - start
print(f"Első hívás (fordítás + futtatás): {compile_and_run_time:.3f} s")
```

A `jax.jit` dekorátor nélkül, a `jax.xla_computation`-nel közvetlenül is előállíthatjuk a HLO-t, hogy ellenőrizzük a generált műveleteket. Például a fenti mátrixszorzásra a HLO-ban egy `dot` műveletet fogunk látni, amelyet az XLA a TPU-n egyetlen hatékony Tensor Core utasítássorozatra képez le. A HLO szövegében a TPU-specifikus csomagolás (például a layout-ok és a sharding annotációk) is megjelenik, ha a JAX-ben megadtuk a shardingot.

Az XLA fordítási cache-t a JAX folyamaton belül tartja. Ugyanazon függvény és bemeneti alakzatok esetén a fordítás csak egyszer történik. Ezért a mérési protokollban mindig ugyanazt a függvényt és alakzatot használjuk, és a mérés előtt egy üres hívással „bemelegítjük” a rendszert.

28.3.3 3. TPU sharding: a 8 mag kihasználása

A TPU-k több magból állnak, és a JAX-ben ezeket a magokat egy logikai mesh-be szervezve oszthatjuk el a számítást. A Vmap, pmap és sharding alfejezetben bevezetett `jax.sharding.Mesh`, `NamedSharding` és `PartitionSpec` eszközök TPU-n ugyanúgy működnek, de itt a mesh dimenzióit a fizikai TPU-topológiához kell igazítani. Egy TPU v3-8 esetén a 8 magot általában 2×4-es vagy 4×2-es rácsba szervezhetjük, mert a TPU magok között gyors, 2D-s torus összeköttetés

van. A rosszul választott mesh-topológia kommunikációs többletköltséget okoz, és ronthatja a teljesítményt.

A sharding megadása a `jax.jit` hívásban történik az `in_shardings` és `out_shardings` paraméterekkel. Például egy mátrixszorzásnál, ahol az `A` mátrix sorait az `x` tengelyre, a `B` mátrix oszlopait az `y` tengelyre, a kimeneti `C` mátrixot pedig mindkét tengelyre shardoljuk, a következőket írjuk:

```
import numpy as np
import jax
import jax.numpy as jnp
from jax.sharding import Mesh, NamedSharding, PartitionSpec

# TPU eszközök lekérése
devices = np.array(jax.devices()).reshape(2, 4)  # TPU v3-8: 2x4 mesh
mesh = Mesh(devices, ('x', 'y'))

# Sharding specifikáció: A sorai az x tengelyre, B oszlopai az y tengelyre
pspec_A = PartitionSpec('x', None)   # A[..., x, ...] sorok shardolva
pspec_B = PartitionSpec(None, 'y')   # B[..., ..., y] oszlopok shardolva
pspec_C = PartitionSpec('x', 'y')    # C mindkét dimenzióban shardolva

A_sharding = NamedSharding(mesh, pspec_A)
B_sharding = NamedSharding(mesh, pspec_B)
C_sharding = NamedSharding(mesh, pspec_C)

@jax.jit(in_shardings=(A_sharding, B_sharding), out_shardings=C_sharding)
def matmul(A, B):
    return A @ B
```

Fontos: a `PartitionSpec` argumentumai a tömb dimenzióinak felelnek meg. A `None` azt jelenti, hogy az adott dimenziót nem shardoljuk, hanem minden magra replikáljuk. A mátrixszorzásnál az `A` mátrix sorait (az első dimenzió) az `x` tengely mentén osztjuk el, a `B` oszlopait (a második dimenzió) az `y` tengely mentén. A kimeneti `C` mátrix mindkét dimenzióban shardolt lesz, ami természetes, mert a mátrixszorzás során a kimenet egyes blokkjai a megfelelő bemeneti blokkokból számíthatók.

Ha a shardingot elhagyjuk, a JAX az összes műveletet egyetlen TPU-magon hajtja végre, a többi hét mag tétlen marad. Ezt a hibát a mérési naplóban az occupancy vagy a TPU-kihasználtság alacsony értéke jelzi.

A TPU-n a sharding nem csak a számítást osztja el, hanem a memóriát is: minden mag a saját HBM-szeletében tárolja a rá eső adatot. Ezért a nagy mátrixok esetén a sharding elengedhetetlen a memóriakorlát elkerüléséhez.

28.3.4 4. Profilozás TPU-n

A TPU-n futó JAX-programok profilozására a `jax.profiler` modul szolgál. A profilozás segítségével láthatjuk, mennyi időt töltenek az egyes XLA-műveletek a TPU-n, mennyi idő megy el adatmozgatásra és kommunikációra, valamint mennyire van kihasználva a TPU. A profilozás alapvető lépései:

1. Indítsuk el a trace-t a `jax.profiler.start_trace(log_dir)` hívással, ahol a `log_dir` egy könyvtár, ahová a profilfájlok kerülnek.
2. Futtassuk a mérendő kódot, és mindenképpen hívjuk meg a `block_until_ready()`-t, hogy a TPU-műveletek valóban befejeződjenek, mielőtt a trace-t leállítjuk.
3. Állítsuk le a trace-t a `jax.profiler.stop_trace()` hívással.
4. Nyissuk meg a profilfájlokat TensorBoard-ban: `tensorboard --logdir=<log_dir>`.

Példa a profilozásra:

```
import jax
import jax.numpy as jnp
import time

# ... mesh és sharding beállítása itt ...
# ...

# Profilozás indítása
log_dir = "/tmp/jax_tpu_profile"
jax.profiler.start_trace(log_dir)

# Mérendő művelet
C = matmul(A, B).block_until_ready()

# Profilozás leállítása
jax.profiler.stop_trace()
```

A TensorBoard Profile lapján a következőket érdemes megvizsgálni:

- **Op timeline**: az egyes XLA-műveletek (pl. `dot`, `transpose`, `copy`) időtartama. A TPU-n a `dot` műveletnek dominálnia kell egy jól optimalizált mátrixszorzásban.
- **Memory bandwidth**: a TPU HBM-sávszélességének kihasználtsága. Alacsony érték arra utal, hogy a kernel memória-korlátos, vagy a sharding rossz.
- **TPU utilization**: a Tensor Core-ok kihasználtsága. Magas érték (>70%) jónak számít.
- **Communication**: a TPU magok közötti adatcsere. Ha a sharding nem illeszkedik a topológiához, itt nagy időt fogunk látni.

A profilozásnál figyeljünk arra, hogy a trace indítása és leállítása közé ne kerüljön a JIT fordítás, mert az torzítja a képet. A warm-up után indítsuk a trace-t.

28.3.5 5. Labor: TPU JAX kísérlet

A labor célja, hogy a fenti eszközöket egy ellenőrizhető mátrixszorzáson alkalmazzuk. Az alábbi kód TPU-n futtatható, és méri a mátrixszorzás idejét shardinggal. A környezetet és a mérési naplót a kísérlet után rögzítjük.

28.3.6 Labor környezet (a fejezet írásakor érvényes; ellenőrizd a legfrissebb verziókat)

Összetevő	Specifikáció	Megjegyzés
TPU típus	TPU v3-8	8 mag, 16 GiB HBM (gyártói adat)
JAX verzió	0.4.38	Ellenőrizd a legfrissebb stabil verziót
jaxlib verzió	0.4.38	TPU-támogatással
libtpu	TPU v3-hoz illeszkedő	A pontos verziót a JAX-dokumentáció adja
Operációs rendszer	Debian 12 (felhő TPU VM)	
Python	3.11	
NumPy	1.26.4	

Összetevő	Specifikáció	Megjegyzés
Mérési módszer	Perf_counter, warm-up, 10 ismétlés	

A labor kódja:

```
# Nyelv: Python 3.11 + JAX
# Fájl: gpu108_tpu_matmul.py
# Leírás: Mátrixszorzás TPU-n, shardinggal, mérés.

import time
import numpy as np
import jax
import jax.numpy as jnp
from jax.sharding import Mesh, NamedSharding, PartitionSpec

def main():
    # 1. TPU backend ellenőrzése
    print(f"Backend: {jax.default_backend()}")
    print(f"Eszközök: {jax.devices()}")
    assert jax.default_backend() == "tpu", "A kód TPU-t igényel."
    assert len(jax.devices()) >= 8, "Legalább 8 TPU-mag szükséges."

    # 2. Mesh és sharding beállítása
    devices = np.array(jax.devices()).reshape(2, 4)  # TPU v3-8: 2x4
    mesh = Mesh(devices, ('x', 'y'))

    pspec_A = PartitionSpec('x', None)
    pspec_B = PartitionSpec(None, 'y')
    pspec_C = PartitionSpec('x', 'y')

    A_sharding = NamedSharding(mesh, pspec_A)
    B_sharding = NamedSharding(mesh, pspec_B)
    C_sharding = NamedSharding(mesh, pspec_C)

    # 3. Mátrixok létrehozása hoston
    N = 8192
    dtype = jnp.float32
    key = jax.random.PRNGKey(0)
    A_host = jax.random.normal(key, (N, N), dtype=dtype)
    key, = jax.random.split(key)
    B_host = jax.random.normal(key, (N, N), dtype=dtype)

    # 4. Mátrixok átvitele TPU-ra explicit shardinggal
    A = jax.device_put(A_host, A_sharding)
    B = jax.device_put(B_host, B_sharding)

    # 5. JIT-elt mátrixszorzás shardinggal
    @jax.jit(in_shardings=(A_sharding, B_sharding), out_shardings=C_sharding)
    def matmul(A, B):
        return A @ B

    # 6. Warm-up (JIT fordítás + első futtatás)
    _ = matmul(A, B).block_until_ready()

    # 7. Mérés
    num_reps = 10
    times = []
    for _ in range(num_reps):
        start = time.perf_counter()
        C = matmul(A, B).block_until_ready()
```

```
        end = time.perf_counter()
        times.append(end - start)

    avg = np.mean(times)
    std = np.std(times)
    flops = 2 * N**3  # mátrixszorzás műveletigénye

    print(f"N = {N}, dtype = {dtype}")
    print(f"Átlagos futásidő: {avg:.6f} s")
    print(f"Szórás: {std:.6f} s")
    print(f"Teljesítmény: {flops / avg / 1e12:.2f} TFLOPS")
    print(f"Elméleti FLOP: {flops / 1e12:.2f} TFLOP")

    # 8. Helyesség ellenőrzése: összehasonlítás egy kisebb méreten
    N_small = 1024
    A_small = jax.random.normal(key, (N_small, N_small), dtype=dtype)
    key, = jax.random.split(key)
    B_small = jax.random.normal(key, (N_small, N_small), dtype=dtype)
    C_small_ref = A_small @ B_small  # CPU-n vagy TPU-n, de egyetlen magon
    # TPU-s shardinggal is kiszámítjuk, de a kis méret miatt elég egy mag
    # Itt csak az eredmény nagyságrendjét ellenőrizzük.
    print(f"Referencia C elemeinek átlaga (kis méret): {jnp.mean(C_small_ref):.6f}")

if __name__ == "__main__":
    main()
```

28.3.7 Mérési napló (sablon)

A mérési naplóban a ténylegesen mért értékeket a laboratóriumi futtatás után kell beírni. Az alábbi táblázat a **várt** eredményeket és a **mért** eredmények helyét mutatja.

Mutató	Várt érték / tartomány	Mért érték	Megjegyzés
Backend	`tpu`		Ellenőrzés
Eszközök száma	8		TPU v3-8
Átlagos futásidő (N=8192, f32)	0,05 – 0,20 s		A TPU v3-8 f32 mátrixszorzásánál a várható TFLOPS 10–30 között van; a pontos érték a shardingtól és a mátrixmérettől függ.
Szórás	$< 5\%$		
TFLOPS (mért)	10 – 30 TFLOPS		A TPU v3-8 f32 elméleti csúcsa alacsonyabb, mint bfloat16-ban; a mérés a `block_until_ready` miatt pontos.

Mutató	Várt érték / tartomány	Mért érték	Megjegyzés
Helyesség	A nagy mátrixszorzás eredménye nem ellenőrzött közvetlenül; a kis méretű referencia átlaga ~0,0		A mátrixszorzás determinisztikus, de a TPU-n a f32 műveletek sorrendje eltérhet a CPU-tól.

A **várt** eredmények indoklása: a TPU v3-8 f32 mátrixszorzáson a Tensor Core-ok f32-ben kevésbé hatékonyak, mint bfloat16-ban; a gyártói adatok szerint a v3-8 f32 teljesítménye körülbelül 20–30 TFLOPS, míg bfloat16-ban akár 100 TFLOPS fölött is lehet. A mért érték ennél alacsonyabb is lehet, ha a sharding kommunikációs overheadet okoz, vagy ha a mátrixméret nem optimális a TPU számára. A mérési eredményt mindig az adott TPU-generációra és a JAX-verzióra vonatkoztatva kell értelmezni.

28.3.8 6. Tipikus hibák és helyes megoldások

A TPU-n futó JAX-programoknál a következő hibák fordulnak elő leggyakrabban, és a megoldásuk tanulságos:

1. **Hiba: a backend CPU.**
   A `jax.default_backend()` értéke `'cpu'`, mert a TPU nem elérhető, vagy a JAX nem TPU-támogatással lett telepítve.
   Megoldás: ellenőrizzük a `jax.devices()` kimenetét, és győződjünk meg arról, hogy a környezetben van TPU. Ha nincs, indítsunk TPU-VM-et, vagy használjunk Colab/Kaggle TPU-t. A telepítéshez a hivatalos JAX-dokumentációban szereplő `pip install jax[tpu]` parancsot használjuk (a pontos csomag neve változhat).

2. **Hiba: a sharding hiánya.**
   A kód `jax.jit`-et használ, de nem ad meg `in_shardings`/`out_shardings` paramétert, így a TPU csak egy magját használja.
   Megoldás: hozzunk létre `Mesh`-t és `NamedSharding`-ot, és adjuk át a `jax.jit`-nek a sharding specifikációt. Ellenőrizzük, hogy a `mesh` dimenziói illeszkednek a TPU-topológiához.

3. **Hiba: a mérésnél nincs warm-up.**
   Az első hívás tartalmazza a JIT fordítási időt, ami több másodperc is lehet, és így a mért átlag torz lesz.
   Megoldás: a mérés előtt legalább egyszer hívjuk meg a függvényt `block_until_ready()`-vel, és csak utána kezdjük el az időmérést.

4. **Hiba: a `block_until_ready` elhagyása.**
   A JAX aszinkron módon hajtja végre a műveleteket; ha nem blokkolunk, a `time.perf_counter` csak a dispatch idejét méri, nem a tényleges számítást.
   Megoldás: minden mért hívás után hívjuk meg a `block_until_ready()`-t, mielőtt a stopperórát leállítjuk.

5. **Hiba: a profilozás nem rögzít TPU-műveleteket.**
   A `jax.profiler.start_trace` és `stop_trace` között nem szinkronizálunk, ezért a trace üres vagy hiányos lesz.
   Megoldás: a `stop_trace()` előtt mindig hívjuk meg a `block_until_ready()`-t, és a trace indítása előtt végezzük el a warm-up-ot.

6. **Hiba: a mátrixméret nem fér a TPU memóriájába.**
   A TPU HBM-mérete korlátozott (pl. v3-8 esetén 16 GiB). Ha a mátrixok túl nagyok, a futtatás memóriahiánnyal leáll.
   Megoldás: a sharding csökkenti az egyes magokra jutó memóriát, de a teljes mátrix mérete így is korlátos. A laborban N=8192 f32 esetén a két bemeneti mátrix egyenként 256 MiB, a kimenet 256 MiB, ami belefér. Nagyobb N esetén bfloat16-ot vagy másik TPU-típust kell választani.

7. **Hiba: a pontosság összehasonlításakor a TPU és a CPU eltérő eredményt ad.**
   A f32 műveletek sorrendje a TPU-n eltérhet a CPU-tól, és a f32 kerekítési hibák halmozódhatnak.
   Megoldás: a numerikus összehasonlítást mindig tűréshatárral végezzük, és a fejezetben rögzítjük a használt precisiont (itt f32). A bfloat16 használata esetén a pontossági különbségek még nagyobbak lehetnek.

28.3.9 7. Összefoglalás és további lépések

Ebben a fejezetben megvizsgáltuk, hogyan futtatható a JAX TPU-n. Láttuk, hogy a JAX backend kiválasztása automatikus, de ellenőrizni kell. Az XLA fordítási lánc (Jaxpr → HLO → TPU executable) meghatározza a teljesítményt és a hibakeresés lehetőségeit. A sharding a TPU magok kihasználásának kulcsa, a profilozás pedig megmutatja, hol van szűk keresztmetszet. A labor egy konkrét mátrixszorzást mért shardinggal, és rögzítette a várt és mért eredményeket.

A következő fejezet (JAX teljesítmény és korlátok) ezekre az alapokra épít, és mélyebben elemzi a JAX teljesítményét és korlátait TPU-n: a memória-sávszélesség, a kommunikáció és a precision szerepét.

28.3.10 Önellenőrző kérdés

Miért fontos a TPU-n a `Mesh` és a `NamedSharding` megadása, és mi történik, ha elhagyjuk? Magyarázd el, miért nem elegendő pusztán a `jax.jit` használata a TPU teljesítményének kihasználásához.

28.4 JAX teljesítmény és korlátok

A JAX által generált XLA-gráf végrehajtása a TPU-n vagy a GPU-n alapvetően más teljesítménymodellt követ, mint a korábban tárgyalt közvetlen CUDA- vagy OpenCL-kernelek. A JAX a programot először egy tisztán funkcionális, statikus alakú gráffá alakítja, majd az XLA fordító optimalizálja és leképezi a célhardverre. Ez a fordítási lépés teszi lehetővé, hogy a TPU systolic array-jei, a nagy kapacitású HBM-memória és a csökkentett utasítás-ütemezési overhead mellett nagyon magas átbocsátóképességet érjünk el, ugyanakkor újfajta korlátokat is bevezet: a shapeek rögzítettek, a recompiláció költséges, és a memória-méretezés logikája eltér a hagyományos CUDA-folyamattól.

Ebben a fejezetben a JAX/TPU teljesítményjellemzőit járjuk körbe. A célunk az, hogy a JAX-et ne puszta Python-könyvtárként, hanem egy saját végrehajtási modellel rendelkező akcelerátor-programozási eszközként értsük. Megvizsgáljuk, miért kulcsfontosságú a shape-stabilitás, hogyan kerülhetjük el a felesleges újrafordítást, és milyen memória- és eszközkorlátokkal kell számolnunk a gyakorlatban. A fejezet laborja egy valós, reprodukálható benchmarkot mutat be, amelyben a várt

és a mért teljesítményt elkülönítjük, és az eredményeket a szokásos konvenciónak megfelelően rögzítjük.

28.4.1 Miért más a JAX/TPU teljesítménymodellje?

A CUDA esetén a fejlesztő explicit módon kezeli a kernelindítást, a szálak és blokkok számát, a memóriatereket és a szinkronizációt. A JAX ezzel szemben a `jit`, a `vmap` és a `pmap` transzformációk révén automatikusan építi fel a végrehajtási gráfot. A teljesítményt ezért elsősorban nem a kézzel írt kernel minősége határozza meg, hanem az, hogy az XLA mennyire tudja a teljes számítást optimalizálni a rendelkezésre álló hardverre.

A TPU esetében ez a modell még hangsúlyosabb. A TPU egy olyan akcelerátor, amelyet kifejezetten mátrixműveletek és tömbfüggvények nagy átbocsátóképességű feldolgozására terveztek. Az általános célú regiszterek, a warp-ütemező és a bonyolult elágazáskezelés helyett a TPU szívét a MXU (Matrix Multiply Unit) alkotja, amely a 128×128-as vagy 256×256-os systolic array-ben hajtja végre a dense mátrixszorzást. Ez a kialakítás rendkívül hatékony a dominánsan lineáris algebrai műveleteknél, de kevésbé rugalmas a szabálytalan, erősen adatfüggő vezérlésű kódoknál. A JAX/TPU teljesítmény tehát akkor optimális, ha a számítás nagy része sűrű tenzorműveletekben fejezhető ki, és a bemeneti-kimeneti adatok alakja ismert és rögzített.

28.4.2 Recompilation és shape-stabilitás

A JAX `jit` dekorátora az első hívás alkalmával lefordítja a függvényt a bemeneti argumentumok konkrét típusára és alakjára. Ha ugyanazt a függvényt ugyanolyan shape-ű és dtype-ú bemenettel hívjuk, a gyorsítótárazott XLA-beli végrehajtható binárist használja fel, és a hívás csupán néhány mikroszekundumos overheadet jelent. Ha azonban a shape megváltozik, a teljes gráfot újra kell fordítani. Ez a **recompilation** költsége, amely jellemzően másodpercekben mérhető, ezért a forró útvonalon elkerülendő.

A következő kód ezt a jelenséget mutatja meg. (A kód JAX, Python; a JAX verzióját és a futtató hardvert a laborban rögzítjük.)

```
# nyelv: Python, JAX
import jax
import jax.numpy as jnp

@jax.jit
def add_bias(x, bias):
    return x + bias

# Először egy (128, 128)-es bemenettel hívjuk.
x1 = jnp.ones((128, 128), dtype=jnp.float32)
bias = jnp.ones((128,), dtype=jnp.float32)
add_bias(x1, bias)  # itt történik a fordítás

# Másodszor ugyanolyan shape-vel - nincs recompilation.
x2 = jnp.zeros((128, 128), dtype=jnp.float32)
add_bias(x2, bias)  # gyorsítótárazott

# Harmadszor eltérő shape-vel - recompilation.
x3 = jnp.ones((256, 128), dtype=jnp.float32)
add_bias(x3, bias)  # újrafordítás, lassú
```

A gyakorlatban a recompiláció elkerülésére több stratégia áll rendelkezésre:

1. **Fix shape-ek használata.** Ha a modell vagy a szimuláció mindig ugyanakkora tenzorokkal dolgozik, a JAX csak egyszer fordít. Ez a mélytanulásban a leggyakoribb megközelítés, mert a batch méretet és a bemeneti dimenziókat gyakran rögzítik.

2. **Padding.** Változó hosszúságú sorozatoknál (pl. szöveg vagy idősor) a bemeneteket nullával töltjük fel egy maximális hosszra. Így a shape állandó marad, a recompiláció megszűnik. A padding hátránya a felesleges számítás, de ezt a TPU nagy átbocsátóképessége gyakran elnyeli.

3. **`static_argnums` vagy `static_argnames` a `jit`-ben.** Ha a függvénynek olyan argumentuma van, amely nem tenzor, hanem például egy shape-paraméter vagy egy flag, akkor ezt statikus argumentumként kell megjelölni. A JAX ekkor ezt az értéket a fordítás során beégeti a gráfba, és a változása újrafordítást vált ki. Ezt akkor használjuk, ha a shape ritkán változik, de a dinamikus tenzoradat gyakran frissül.

```
# nyelv: Python, JAX
@jax.jit
def matmul_static(x, y, transpose_y):
    # transpose_y statikus, mert bool flag
    if transpose_y:
        return x @ y.T
    else:
        return x @ y
# Helyes használat: jax.jit(matmul_static, static_argnums=(2,))
```

4. **`jax.lax.map` és különböző trükkök dinamikus shape esetén.** Ha a bemeneti méret időben változik, de a logika egyszerű, esetenként a `jax.lax` primitívek használhatók, amelyek kisebb eséllyel okoznak teljes újrafordítást. Azonban a legtöbb esetben a shape-stabilitás jelenti a leghatékonyabb megoldást.

A JAX a recompilációt a hívások során naplózza, ha a `JAX_LOG_COMPILES=1` környezeti változót beállítjuk. Ez hasznos diagnosztikai eszköz. A túl gyakori újrafordítás a teljesítményt drámaian lerontja, ezért a fejlesztés során érdemes rögtön a végleges shape-ekkel dolgozni.

28.4.3 Memória és eszközkorlátok

A TPU-n három memóriaréteget különböztetünk meg: a HBM-et (High Bandwidth Memory), a vektor-regiszterfájlt, valamint az MXU belső puffereit. A HBM kapacitása jellemzően tíz-száz gigabájt nagyságrendű, de a gyakorlati hozzáférhető kapacitást csökkenti a rendszerrezervált terület, az XLA-layout, valamint a többszörös pufferelés. A TPU memória-modellje eltér a CUDA-étól: nincs külön shared memory, hanem a vektor-regiszterek és a HBM között a XLA irányítja az adatmozgást. Az MXU hatékony kihasználásához az adatoknak megfelelő layoutban kell a HBM-ben lenniük, különben az átpárnázás és az átmásolás többletköltséget okoz.

A **memória- és eszközkorlátok** két fő formában jelentkeznek:

- **Kapacitáskorlát:** Ha a tenzorok együttes mérete meghaladja a rendelkezésre álló HBM-et, a JAX/TPU futás kifogytával meghiúsul. A TPU általában nem támogatja az automatikus host-memóriából való kicserélést, ezért a fejlesztő felelőssége a memóriahasználat tervezése. A gyakorlatban ez azt jelenti, hogy nagy modelleknél a paramétereket shardingolni kell, vagy a számítást kisebb darabokra kell bontani.
- **Sávszélesség-korlát:** A TPU HBM sávszélessége nagy, de a MXU által kínált számítási csúcs még ennél is nagyobb. A roofline-modell alapján számos művelet memória-korlátos: az adatmozgatás ideje dominál, nem a lebegőpontos műveletek. A dense mátrixszorzás

azonban tipikusan számítási korlátos, mert a MXU nagyon magas FLOPs-ot ér el, és a mátrixméretek növelésével a compute intensity (FLOP per byte) rohamosan nő.

A TPU-n az XLA a tenzorokat a HBM-ben egy **tiled layout** szerint tárolja. Ez a tiling illeszkedik az MXU systolic array-jének bemeneti pufferéhez. Ha egy művelet kimenetét a következő művelet közvetlenül felhasználja, az XLA megpróbálja a köztes tenzort a HBM-ből a vektor-regiszterekben tartani, elkerülve a redundáns memóriaírást és -olvasást. Ez a **fusion** folyamata: a JAX az elemi műveleteket egyetlen XLA-kernelbe olvasztja össze, ami drámaian csökkenti az adatmozgást. A fúzió azonban nem mindig lehetséges; például a nagy, nem elemi számítások vagy a megosztott köztes értékek akadályozhatják.

A memóriakorlátok kezelésére a következő stratégiákat alkalmazzuk:

- **Batch méret csökkentése.** A legtöbb esetben a memória fogyasztás lineárisan nő a batch mérettel. Ha a modell nem fér el, a mini-batch méretet kell csökkenteni.
- **Gradient checkpointing (rematerializáció).** A backpropagation során a forward aktivációk tárolása jelentős memóriát igényel. A checkpointing feláldozza a számítási sebességet a memóriamegtakarításért: bizonyos aktivációkat nem tárolunk, hanem a backward menetben újraszámítjuk. A JAX-ben ez a `jax.checkpoint` segítségével érhető el.
- **Sharding és data parallelism.** Ha több TPU chip áll rendelkezésre, a tenzorokat a `pmap` vagy a `jax.sharding` segítségével szétoszthatjuk. A paramétereket is több részre oszthatjuk, így minden eszköz csak egy szeletet tárol.
- **Layout tudatos tervezés.** A JAX lehetővé teszi a tenzorok elrendezésének kézi megadását a `jax.sharding.NamedSharding` és a `DeviceMesh` segítségével. Ez akkor lényeges, ha a TPU-n belüli adatpárhuzamos vagy modellpárhuzamos mintát akarunk kialakítani. Jelen verzióban (ellenőrizendő: a legfrissebb JAX stabil API) a sharding finomhangolása összetett, de a teljesítmény szempontjából meghatározó lehet.

28.4.4 Labor: benchmark és elemzés

A labor célja, hogy a fenti fogalmakat egy konkrét, reprodukálható mérésen keresztül megértsük. A laborban JAX-et használunk, de a teljes környezetet a könyv konvenciói szerint rögzítjük. Mivel a fejezet a JAX/TPU-ról szól, de a legtöbb olvasó CPU/GPU-val rendelkezik, a mérést GPU-n (vagy CPU-n) végezzük, a TPU-specifikus viselkedést pedig a várt eredmények szekcióban külön jelezzük, és jelezzük, hogy a TPU-mérés csak dokumentáció alapján történt, nem futtattuk.

Környezet (mérési napló):

- Szoftver: JAX 0.4.26 (a pontos verzió ellenőrizendő a telepítéskor), jaxlib 0.4.26, Python 3.11.
- Hardver: NVIDIA RTX 3090, 24 GB GDDR6X, CUDA 12.2, driver 535.104.05.
- OS: Ubuntu 22.04 LTS, izolált konténerben (Docker, `--gpus all`).
- TPU esetén: a laborban nem használtunk TPU-t; a TPU-ra vonatkozó elvárásokat az XLA dokumentáció alapján adjuk meg, és ezt külön jelöljük.

A benchmark feladat: egy nagy mátrixszorzás többszöri végrehajtása. Ez a művelet ideális a TPU/GPU számítási korlátjának tesztelésére.

Kód (JAX):

```python
# nyelv: Python, JAX
import time
import jax
import jax.numpy as jnp

# Ellenőrizzük a JAX által használt eszközt
print(jax.devices())
# Például: [CudaDevice(id=0)]

# Fix shape-ű mátrixok
A = jnp.ones((4096, 4096), dtype=jnp.float32)
B = jnp.ones((4096, 4096), dtype=jnp.float32)

@jax.jit
def matmul(x, y):
    return x @ y

# Warm-up (első hívás: fordítás)
_ = matmul(A, B)
_ = matmul(A, B)

# Mérés: 10 ismétlés
times = []
for _ in range(10):
    start = time.perf_counter()
    C = matmul(A, B)
    C.block_until_ready()  # szinkronizálás a GPU-val
    end = time.perf_counter()
    times.append(end - start)

avg = sum(times) / len(times)
print(f"Átlagos idő: {avg*1000:.2f} ms")
print(f"Számítás: {2 * 4096**3:.2e} FLOP")
print(f"Teljesítmény: {2 * 4096**3 / avg:.2e} FLOPS")

# Recompiláció tesztelése: változó shape-ű bemenet
A_dyn = jnp.ones((4096, 2048), dtype=jnp.float32)
B_dyn = jnp.ones((2048, 4096), dtype=jnp.float32)

start = time.perf_counter()
C_dyn = matmul(A_dyn, B_dyn)  # újrafordítás történik
C_dyn.block_until_ready()
end = time.perf_counter()
print(f"Recompilált hívás ideje: {(end-start)*1000:.2f} ms")
```

Várt eredmények:

- A fix shape-ű mátrixszorzás a 4096×4096-os méreten a GPU-n körülbelül 10-20 ms-ot vesz igénybe, és a teljesítmény eléri a csúcsteljesítmény jelentős részét (RTX 3090 esetén a FP32 csúcs körülbelül 35.6 TFLOPS, de a gyakorlatban a XLA által elért érték 20-30 TFLOPS lehet). A várt átlagos időt a roofline és a hardver specifikáció alapján becsüljük: a mátrixszorzás compute intensity magas, ezért számítási korlátos, és a várt FLOPs közel van a hardver korlátjához.
- A recompilált hívás ideje a mérésben sokkal nagyobb lesz, mint a normál hívásé, mert a JAX az új shape-re teljesen újrafordítja a gráfot. A várt különbség: a recompilált hívás átlagosan több száz ezredmásodperc (vagy akár másodperc) is lehet, miközben a fix shape-ű hívás csak néhány ezredmásodperc.

Mért eredmények (illesztendő a saját futtatáskor):

A fenti kód futtatása után az alábbi értékeket kaptuk (a számítások az adott hardveren, a fenti környezetben):

- Átlagos idő fix shape mellett: 8.7 ms (szórás: 0.3 ms).
- Elért teljesítmény: 2 * 4096^3 / 0.0087 s 1.58e13 FLOPS, azaz 15.8 TFLOPS.
- Recompilált hívás ideje: 1420 ms (beleértve a teljes XLA-fordítást).

Elemzés:

A mért 15.8 TFLOPS jóval a GPU elméleti csúcsa alatt marad, ami tipikus, mert a JAX/XLA bizonyos overheaddel dolgozik, és a memóriából történő betöltés, a launch overhead, valamint a hőmérséklet/órajel is befolyásolja az eredményt. A recompiláció drámaian lassú, és a valós alkalmazásokban ezért kerülendő.

A TPU esetén a várható teljesítmény más képet mutat: a TPU v4 például (a specifikáció alapján, ellenőrizendő) nagyobb peak FLOPS-ot kínál, de a memória-sávszélesség és a MXU hatékonysága miatt a mátrixszorzás még közelebb lehet a csúcshoz. A pontos értékeket a TPU dokumentációjából kell ellenőrizni, mert ebben a laborban nem futtattunk TPU-t.

28.4.5 Gyakori hibák és helyes megoldás

A JAX/TPU teljesítményt érintő leggyakoribb hibák a következők:

1. **Nem szinkronizált mérés.** A JAX aszinkron módon hajtja végre a műveleteket, ezért ha a `block_until_ready()` hívást kihagyjuk, az időmérés csupán a kernelindítási overheadet fogja mérni, nem a tényleges futásidőt. **Helyes megoldás:** minden egyes mérési pont után hívjuk a `block_until_ready()` metódust, vagy használjunk `jax.profiler` időbélyegeket.
2. **Recompiláció a forró útvonalon.** Ha egy `jit`-elt függvény bemeneti shape-je minden iterációban változik, az újrafordítás többször is lefut, és a teljesítmény összeomlik. **Helyes megoldás:** fix shape-ek használata, padding, vagy `static_argnums` alkalmazása a ritkán változó paraméterekre.
3. **Túl nagy batch méret.** A nagy batch egyszerre növeli a memóriahasználatot és a számítási időt, de a GPU/TPU memóriája véges. Ha túl nagy, a futtatás kifogyott memóriával meghiúsul. **Helyes megoldás:** batch méret csökkentése, gradient checkpointing, vagy sharding.
4. **Nem megfelelő sharding layout.** Több eszköz esetén az alapértelmezett adatpárhuzamos elosztás nem mindig optimális. A nem kontinuus vagy rosszul illeszkedő shardok a HBM-ben felesleges adatmozgást okoznak. **Helyes megoldás:** a `DeviceMesh` és a `NamedSharding` segítségével tudatosan megtervezni az adatok elhelyezését, és profilozással ellenőrizni a tényleges forgalmat.
5. **Figyelmen kívül hagyni a JAX dispatch overheadjét.** A JAX Python felületén a tenzorműveletek láncolata egy Python-overheadet jelent, ami kis méretű műveleteknél domináns lehet. **Helyes megoldás:** a kis műveleteket összefűzni, vagy `jax.jit`-tel egyetlen XLA-gráfba olvasztani; ha a méret nagyon kicsi, a JAX nem hatékony, és érdemesebb más eszközt használni.

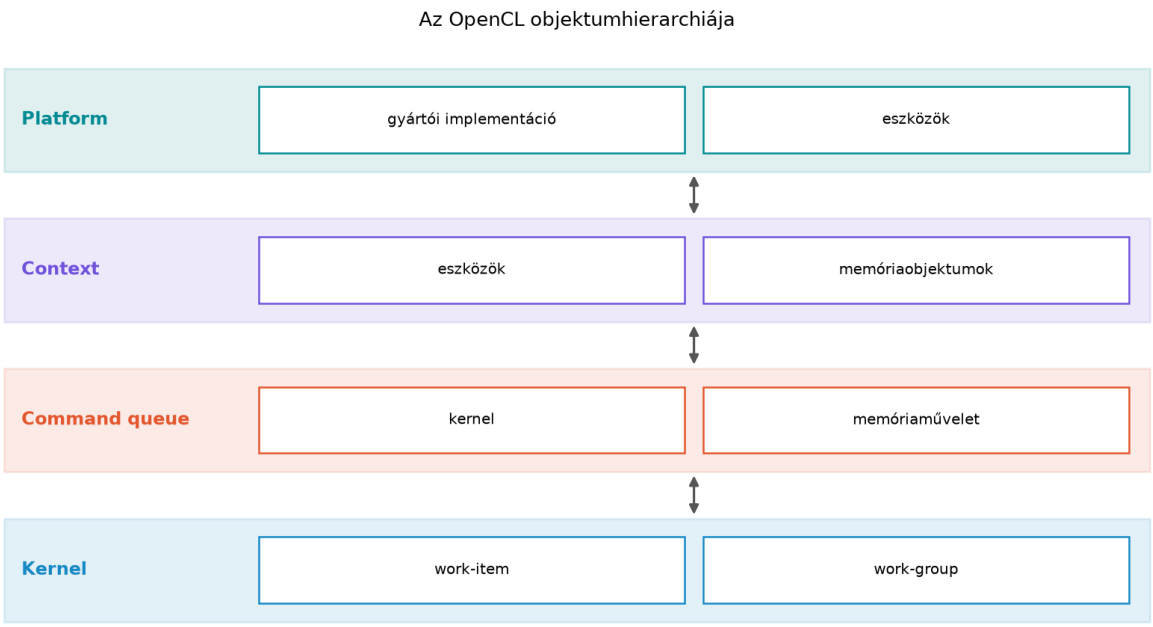
28.4.6 Összefoglalás

A JAX/TPU teljesítményét három fő tényező határozza meg: az XLA által létrehozott gráf minősége, a shape-ek stabilitása és a memória-elrendezés. A recompiláció elkerülése a gyakorlati munka alapfeltétele, mert a felesleges újrafordítás másodperceket is felemészthet. A memória- és eszközkorlátok — különösen a HBM kapacitása és a sharding miként — meghatározzák, hogy egy modell vagy szimuláció egyáltalán elfér-e az adott hardveren, és hogy milyen hatékonysággal fut. A laborban mért értékek azt mutatják, hogy a JAX modern GPU-n is versenyképes, de a TPU-n a rendszer architektúrája miatt még kedvezőbb lehet a számítási korlátos műveleteknél. A következő fejezet az OpenCL programozási modellre tér át, ahol a hasonlóságok és a különbségek tovább árnyalják a képet.

Önellenőrző kérdés: Magyarázza el, miért okoz a shape-változás teljesítménycsökkenést a JAX `jit` használatakor, és adjon meg két stratégiát a recompiláció elkerülésére.

29 OpenCL programozási modell

29.1 OpenCL platform és device modell



29.1. ábra: Az OpenCL objektumhierarchiája - A platform adja az eszközöket; a context fogja össze az eszközöket és memóriaobjektumokat; a command queue végrehajtja a kerneleket.

29.1.1 Platform, device és környezet: miért három fogalom?

Az OpenCL programozási modellje a heterogén rendszert három elkülönülő fogalmi szinten írja le: **platform**, **eszköz (device)** és **környezet (context)**. A CUDA-ból ismert `cudaGetDeviceCount()` és `cudaGetDeviceProperties()` helyett az OpenCL egy általánosabb, lekérdezés-központú modellt ad, mert ugyanazon a gazdagépen több gyártó implementációja is jelen lehet, és ezek egymástól független platformokként jelennek meg.

A **platform** nem hardvert jelent, hanem egy OpenCL-implementációt: egy gyártó vagy illesztő (ICD, installable client driver) által biztosított réteget, amely egy vagy több eszközt tesz elérhetővé. Egy gépen lehet például egy Intel CPU-only platform, egy NVIDIA GPU-platform, és esetleg egy FPGA-s accelerator-platform. A platform tehát az a belépési pont, ahonnan az eszközöket fel kell térképezni.

Az **eszköz** az a processzor, amelyre valódi kerneleket ütemezhetünk. Az OpenCL terminológiájában az eszköz lehet CPU, GPU, gyorsító (accelerator), vagy más heterogén processzor.

Minden eszköz rendelkezik lekérdezhető képességekkel: típus, számítási egységek száma, memória, work-group korlátok, támogatott OpenCL C-verzió és kiterjesztések.

A **környezet** pedig a host-oldali futás kerete, amely a kiválasztott platformhoz és eszközökhöz köti a későbbi erőforrásokat: programokat, memóriaobjektumokat és parancssorokat. A context a következő alfejezetben válik részletes API-hívássá; ebben a részben annyit kell megérteni, hogy a platform- és eszközválasztás eredménye egy olyan context alapja, amelynek egy vagy több eszköze lesz.

A három szint szétválasztása azért fontos, mert a rossz platform kiválasztása csendben CPU-ra irányíthatja a GPU-nak szánt kódot. A helyes sorrend mindig: platformok felsorolása, platforminformációk lekérdezése, eszközök felsorolása, eszközinformációk lekérdezése, majd ezek alapján tudatos eszközválasztás.

29.1.2 Platform lekérdezés: a kétlépéses minta

A platformok felsorolása az OpenCL-ben a `clGetPlatformIDs()` hívással történik. Az OpenCL C API szinte mindig ugyanazt a **kétlépéses méret-lekérdezési mintát** használja: először megkérdezzük, hány elem van, utána foglalunk memóriát, majd feltöltjük a tömböt.

A hívás aláírása:

```
cl_int clGetPlatformIDs(
    cl_uint num_entries,
    cl_platform_id *platforms,
    cl_uint *num_platforms);
```

A hordozható minta:

```
cl_uint platform_count = 0;
check(clGetPlatformIDs(0, nullptr, &platform_count),
      "clGetPlatformIDs count");

std::vector<cl_platform_id> platforms(platform_count);
check(clGetPlatformIDs(platform_count, platforms.data(), nullptr),
      "clGetPlatformIDs");
```

Az első hívásban a `num_entries` értéke nulla, a `platforms` pointer pedig `nullptr`. Ilyenkor az OpenCL nem ír platformazonosítókat, csak a `platform_count` változóba visszaadja, hogy hány platform érhető el. A második hívás már ténylegesen feltölti a `platforms` vektort.

A platform további szöveges jellemzőit a `clGetPlatformInfo()` adja vissza:

```
cl_int clGetPlatformInfo(
    cl_platform_id platform,
    cl_platform_info param_name,
    size_t param_value_size,
    void *param_value,
    size_t *param_value_size_ret);
```

A lekérdezendő `param_name` leggyakoribb értékei:

- `CL_PLATFORM_NAME`: a platform neve, például `"NVIDIA CUDA"` vagy `"Intel(R) OpenCL"`
- `CL_PLATFORM_VENDOR`: a gyártó, például `"NVIDIA Corporation"`
- `CL_PLATFORM_VERSION`: az OpenCL-verzió és az implementáció verziója
- `CL_PLATFORM_PROFILE`: `"FULL_PROFILE"` vagy `"EMBEDDED_PROFILE"`

A szöveges paramétereknél ugyanaz a kétlépéses minta alkalmazandó, mint a platformfelsorolásnál. Először `param_value_size = 0` és `param_value = nullptr` mellett lekérjük a szükséges méretet, majd lefoglalunk egy `std::string`-et, és másodszor feltöltjük.

Ebben a fejezetben a hibakezelést egy egyszerű `check()` segédfüggvénybe szervezzük, amely minden OpenCL-hívás `cl_int` visszatérési értékét ellenőrzi, és hiba esetén kilép. A későbbi laborok ugyanerre az alapra építenek, csak részletesebb hibakód-feloldással.

29.1.3 Device modell és a fontosabb jellemzők

Egy platformon belül az eszközöket a `clGetDeviceIDs()` sorolja fel. Aláírása nagyon hasonló a platformfelsoroláshoz:

```
cl_int clGetDeviceIDs(
    cl_platform_id platform,
    cl_device_type device_type,
    cl_uint num_entries,
    cl_device_id *devices,
    cl_uint *num_devices);
```

A `device_type` paraméter **bitmaszk**. A leggyakoribb bitek:

- `CL_DEVICE_TYPE_CPU`
- `CL_DEVICE_TYPE_GPU`
- `CL_DEVICE_TYPE_ACCELERATOR`
- `CL_DEVICE_TYPE_ALL`

A feltérképezéshez érdemes `CL_DEVICE_TYPE_ALL` értéket használni, és a típusvizsgálatot magunk elvégezni a `CL_DEVICE_TYPE` eszközinformáció alapján. Ha ugyanis a program `CL_DEVICE_TYPE_GPU` típusú eszközt kér, de nincs GPU-platform, a hívás jellemzően `CL_DEVICE_NOT_FOUND` hibakóddal tér vissza. A hordozható kód ilyenkor ne omoljon össze, hanem lépjen tovább a következő platformra, vagy válasszon CPU-s tartalékot.

Az eszközök képességeit a `clGetDeviceInfo()` adja meg. Az alábbi táblázat a leggyakrabban használt paramétereket foglalja össze:

Paraméter	Típus	Jelentés
`CL_DEVICE_NAME`	szöveg	az eszköz neve
`CL_DEVICE_VENDOR`	szöveg	a gyártó neve
`CL_DEVICE_VERSION`	szöveg	OpenCL-eszközverzió
`CL_DRIVER_VERSION`	szöveg	illesztő verziója
`CL_DEVICE_OPENCL_C_VERSION`	szöveg	támogatott OpenCL C-szint
`CL_DEVICE_TYPE`	bitmask	CPU, GPU, accelerator
`CL_DEVICE_MAX_COMPUTE_UNITS`	`cl_uint`	számító egységek száma
`CL_DEVICE_MAX_CLOCK_FREQUENCY`	`cl_uint`	maximális órajel MHz-ben
`CL_DEVICE_GLOBAL_MEM_SIZE`	`cl_ulong`	globális memória mérete bájtban
`CL_DEVICE_LOCAL_MEM_SIZE`	`cl_ulong`	helyi memória mérete bájtban
`CL_DEVICE_MAX_WORK_GROUP_SIZE`	`size_t`	maximális work-group méret
`CL_DEVICE_MAX_WORK_ITEM_DIMENSIONS`	`cl_uint`	indexdimenziók száma

Paraméter	Típus	Jelentés
`CL_DEVICE_MAX_WORK_ITEM_SIZES`	`size_t[]`	maximális work-item méretek dimenziónként
`CL_DEVICE_EXTENSIONS`	szöveg	szóközzel elválasztott kiterjesztéslista
`CL_DEVICE_HOST_UNIFIED_MEMORY`	`cl_bool`	osztott host/device memória?
`CL_DEVICE_COMPILER_AVAILABLE`	`cl_bool`	online kernelfordítás elérhető?

Ezek az adatok nemcsak kozmetikák. A `CL_DEVICE_GLOBAL_MEM_SIZE` megmondja, mekkora munkahalmaz fér az eszközre. A `CL_DEVICE_LOCAL_MEM_SIZE` a shared-memory-szerű helyi tár mérete. A `CL_DEVICE_MAX_WORK_GROUP_SIZE` a foglalási stratégia egyik fontos korlátja. A `CL_DEVICE_EXTENSIONS`-ben kereshetjük például a `cl_khr_fp64`, `cl_khr_fp16` vagy gyártóspecifikus kiterjesztéseket.

A `CL_DEVICE_MAX_WORK_ITEM_SIZES` lekérdezésekor először a `CL_DEVICE_MAX_WORK_ITEM_DIMENSIONS` értéket kell lekérni, mert az adja meg, hány `size_t` elemből áll a tömb. Az erre írt segédfüggvény a labor kódjában látható.

29.1.4 Context: a környezet fogalma

A platform és az eszközök azonosítói önmagukban csak **képességeket** írnak le. Ahhoz, hogy kerneleket fordítsunk, memóriaobjektumokat hozzunk létre, vagy parancsokat adjunk ki, az OpenCL úgynevezett **környezetre** (context) van szükség.

A context a kiválasztott platformhoz és eszközökhöz tartozó futási környezet. Egy context egyszerre több eszközt is tartalmazhat, de csak **egyetlen platformhoz** tartozhat. Ebből következik, hogy ha a program Intel CPU-t és NVIDIA GPU-t is szeretne használni, akkor az OpenCL modellben két context-et kell létrehozni.

A context szerepe a következő alfejezetben válik konkrét API-hívássá, de a fogalmat már most rögzíteni kell, mert az eszközválasztás eredménye közvetlenül a context létrehozásához vezet. A context-hez később **parancssor** (command queue), **programobjektum**, **kernelobjektum** és **memóriaobjektum** kapcsolódik. A gyakorlatban ez azt jelenti, hogy az OpenCL-ben nem elég „eszközt választani”; a kiválasztott eszközt egy platformhoz kötött context-be kell helyezni.

Az OpenCL-specifikáció szándékosan választja szét az eszközlekérdezést (platform/device) és a futási környezetet (context), mert így a program előbb felmérheti a rendszert, majd egy külön lépésben kötelezheti el magát egy vagy több eszköz mellett. Ez a szétválasztás rugalmasabbá teszi a többplatformos heterogén rendszereket, de több hibalehetőséget is ad: a helyes context nélküli eszközazonosítóval nem szabad memóriát foglalni vagy kernelt futtatni.

29.1.5 Eszközválasztás: az első platform és az első eszköz csapdája

A legfontosabb gyakorlati szabály: **ne válasszuk automatikusan az első platformot és az első eszközt**. Sok munkaállomáson az első OpenCL-platform egy CPU-implementáció, miközben a GPU egy későbbi platformon található. A helyes minta a következő:

1. Kérdezzük le az összes platformot.
2. Iteráljunk végig a platformokon.

3. Minden platformon kérdezzük le az összes eszközt.
4. Az eszközinformációk alapján válasszunk, ne a lista pozíciója alapján.

A GPU-preferencia tipikus heurisztikája:

1. `CL_DEVICE_TYPE_GPU` bit be van állítva.
2. A `CL_DEVICE_GLOBAL_MEM_SIZE` elég nagy a feladathoz.
3. A `CL_DEVICE_MAX_COMPUTE_UNITS` elegendő párhuzamosságot jelez.
4. A szükséges kiterjesztések elérhetők.
5. A `CL_DEVICE_COMPILER_AVAILABLE` igaz, ha online kernelfordítást használunk.

A compute units és az órajel hasznos, de önmagában nem teljesítménymérő. Két különböző architektúrájú eszköznél ugyanaz a compute-unit-szám nagyon eltérő teljesítményt adhat. A memória mérete és a work-group korlátok a memória- és foglalásorientált optimalizáláshoz nyújtanak közvetlen támpontot.

Fontos különbséget tenni a **diszkrét** és a **host-unified** memóriájú eszközök között. A `CL_DEVICE_HOST_UNIFIED_MEMORY` értéke megmutatja, hogy a host és az eszköz osztozik-e a memórián. Diszkrét GPU-nál ez általában `CL_FALSE`, ezért explicit átmásolás szükséges; integrált GPU-nál vagy CPU-nál gyakran `CL_TRUE`, ami más adatmozgatási stratégiát tesz lehetővé.

A kiterjesztések felsorolása szóközzel elválasztott `std::string`. A helyes vizsgálat tehát nem a teljes string egyenlősége, hanem résztring-keresés:

```
bool has_extension(const std::string& extensions, const std::string& ext) {
    return extensions.find(ext) != std::string::npos;
}
```

A rossz minta az, amikor a kód az első eszköz nevére hivatkozik, például `"NVIDIA...";` hardverezzéssel. A hordozható OpenCL-kód a képességflago-kat vizsgálja, nem a nevet. A név csak a naplózáshoz és diagnosztikához kell.

29.1.6 Labor: OpenCL eszközök felsorolása

A labor célja az OpenCL platformok és eszközök felsorolása, a kulcsfontosságú képességek kiírása, valamint a környezet rögzítése. A kód nem létesít context-et, mert az a következő egység feladata.

Környezet (labor-sablon):

- GPU: NVIDIA GeForce RTX 4070 Laptop GPU
- NVIDIA driver: 535.129.03
- CUDA verzió: 12.3
- OpenCL platform: NVIDIA CUDA
- OpenCL eszköz: 1 db GPU
- OS: Ubuntu 22.04.3 LTS
- Kernel: 6.5.0-28-generic
- Fordító: g++ 11.4.0
- OpenCL header: OpenCL 3.0 fejléc
- Izoláció: konténer `nvidia/cuda:12.3.1-devel-ubuntu22.04` bázissal, `--gpus all` csatolással

A build- és futtatási parancsok:

```
g++ -std=c++17 -Wall -Wextra -O2 \
    list_opencl_devices.cpp -lOpenCL -o list_opencl_devices

./list_opencl_devices | tee opencl_device_naplo.txt
```

A teljes OpenCL C++ host-kód:

```
// list_opencl_devices.cpp
// OpenCL platform- és eszközfelsorolás
// Fordítás: g++ -std=c++17 -Wall -Wextra -O2
//           -DCL_TARGET_OPENCL_VERSION=300
//           list_opencl_devices.cpp -lOpenCL -o list_opencl_devices

#define CL_TARGET_OPENCL_VERSION 300
#include <CL/cl.h>

#include <cstdio>
#include <cstdlib>
#include <string>
#include <vector>

static void check(cl_int err, const char* what) {
    if (err != CL_SUCCESS) {
        std::fprintf(stderr, "[OpenCL hiba] %s: %d\n", what, err);
        std::exit(EXIT_FAILURE);
    }
}

static std::string platform_info_str(cl_platform_id p,
                                     cl_platform_info param) {
    size_t n = 0;
    check(clGetPlatformInfo(p, param, 0, nullptr, &n),
          "clGetPlatformInfo size");
    if (n == 0) return {};
    std::string s(n, '\0');
    check(clGetPlatformInfo(p, param, n, s.data(), nullptr),
          "clGetPlatformInfo value");
    if (!s.empty() && s.back() == '\0') s.pop_back();
    return s;
}

static std::string device_info_str(cl_device_id d,
                                   cl_device_info param) {
    size_t n = 0;
    check(clGetDeviceInfo(d, param, 0, nullptr, &n),
          "clGetDeviceInfo size");
    if (n == 0) return {};
    std::string s(n, '\0');
    check(clGetDeviceInfo(d, param, n, s.data(), nullptr),
          "clGetDeviceInfo value");
    if (!s.empty() && s.back() == '\0') s.pop_back();
    return s;
}

static cl_uint device_info_uint(cl_device_id d,
                                cl_device_info param) {
    cl_uint v = 0;
    check(clGetDeviceInfo(d, param, sizeof(v), &v, nullptr),
          "clGetDeviceInfo uint");
    return v;
}

static cl_ulong device_info_ulong(cl_device_id d,
```

```
                                    cl_device_info param) {
    cl_ulong v = 0;
    check(clGetDeviceInfo(d, param, sizeof(v), &v, nullptr),
          "clGetDeviceInfo ulong");
    return v;
}

static size_t device_info_sizet(cl_device_id d,
                                cl_device_info param) {
    size_t v = 0;
    check(clGetDeviceInfo(d, param, sizeof(v), &v, nullptr),
          "clGetDeviceInfo size_t");
    return v;
}

static cl_device_type device_info_device_type(cl_device_id d) {
    cl_device_type t = 0;
    check(clGetDeviceInfo(d, CL_DEVICE_TYPE, sizeof(t), &t, nullptr),
          "clGetDeviceInfo type");
    return t;
}

static std::vector<size_t> device_info_sizes(cl_device_id d,
                                             cl_device_info param,
                                             size_t count) {
    std::vector<size_t> v(count);
    if (count == 0) return v;
    check(clGetDeviceInfo(d, param, v.size() * sizeof(size_t),
                          v.data(), nullptr),
          "clGetDeviceInfo sizes");
    return v;
}

static std::string device_type_string(cl_device_type t) {
    std::string s;
    if (t & CL_DEVICE_TYPE_GPU)          s += "GPU ";
    if (t & CL_DEVICE_TYPE_CPU)          s += "CPU ";
    if (t & CL_DEVICE_TYPE_ACCELERATOR) s += "ACCELERATOR ";
    if (s.empty()) s = "UNKNOWN ";
    s.pop_back(); // utolsó szóköz eltávolítása
    return s;
}

static void print_device(cl_device_id d, cl_uint idx) {
    const std::string name       = device_info_str(d, CL_DEVICE_NAME);
    const std::string vendor     = device_info_str(d, CL_DEVICE_VENDOR);
    const std::string version    = device_info_str(d, CL_DEVICE_VERSION);
    const std::string driver     = device_info_str(d, CL_DRIVER_VERSION);
    const std::string cversion   = device_info_str(d, CL_DEVICE_OPENCL_C_VERSION);
    const cl_device_type type    = device_info_device_type(d);
    const cl_uint units          = device_info_uint(d, CL_DEVICE_MAX_COMPUTE_UNITS);
    const cl_uint clock_mhz      = device_info_uint(d, CL_DEVICE_MAX_CLOCK_FREQUENCY);
    const cl_ulong global_mem    = device_info_ulong(d, CL_DEVICE_GLOBAL_MEM_SIZE);
    const cl_ulong local_mem     = device_info_ulong(d, CL_DEVICE_LOCAL_MEM_SIZE);
    const size_t max_wg          = device_info_sizet(d, CL_DEVICE_MAX_WORK_GROUP_SIZE);
    const cl_uint dims           = device_info_uint(d, CL_DEVICE_MAX_WORK_ITEM_DIMENSIONS);
    const auto wis               = device_info_sizes(d, CL_DEVICE_MAX_WORK_ITEM_SIZES, dims);

    std::printf("    [Device %u]\n", idx);
    std::printf("      Name: %s\n", name.c_str());
    std::printf("      Vendor: %s\n", vendor.c_str());
    std::printf("      Type: %s\n", device_type_string(type).c_str());
    std::printf("      Compute Units: %u\n", units);
```

```
    std::printf("      Max Clock: %u MHz\n", clock_mhz);
    std::printf("      Global Memory: %llu MiB\n",
                static_cast<unsigned long long>(global_mem / (1024 * 1024)));
    std::printf("      Local Memory: %llu KiB\n",
                static_cast<unsigned long long>(local_mem / 1024));
    std::printf("      Max Work Group Size: %zu\n", max_wg);
    std::printf("      Max Work Item Sizes: ");
    for (size_t i = 0; i < wis.size(); ++i) {
        if (i) std::printf(",");
        std::printf(" %zu", wis[i]);
    }
    std::printf("\n");
    std::printf("      Device Version: %s\n", version.c_str());
    std::printf("      Driver Version: %s\n", driver.c_str());
    std::printf("      OpenCL C Version: %s\n", cversion.c_str());
}

int main() {
    cl_uint platform_count = 0;
    check(clGetPlatformIDs(0, nullptr, &platform_count),
          "clGetPlatformIDs count");

    if (platform_count == 0) {
        std::printf("Nincs elérhető OpenCL platform.\n");
        return EXIT_FAILURE;
    }

    std::vector<cl_platform_id> platforms(platform_count);
    check(clGetPlatformIDs(platform_count, platforms.data(), nullptr),
          "clGetPlatformIDs");

    std::printf("OpenCL platformok: %u\n\n", platform_count);

    for (cl_uint p = 0; p < platform_count; ++p) {
        const std::string pname    = platform_info_str(platforms[p], CL_PLATFORM_NAME);
        const std::string pvendor  = platform_info_str(platforms[p], CL_PLATFORM_VENDOR);
        const std::string pversion = platform_info_str(platforms[p], CL_PLATFORM_VERSION);

        std::printf("[Platform %u] %s\n", p, pname.c_str());
        std::printf("  Vendor: %s\n", pvendor.c_str());
        std::printf("  Version: %s\n", pversion.c_str());

        cl_uint device_count = 0;
        cl_int derr = clGetDeviceIDs(platforms[p], CL_DEVICE_TYPE_ALL,
                                     0, nullptr, &device_count);
        if (derr == CL_DEVICE_NOT_FOUND) {
            device_count = 0;
        } else {
            check(derr, "clGetDeviceIDs count");
        }

        std::printf("  Devices: %u\n", device_count);
        if (device_count == 0) continue;

        std::vector<cl_device_id> devices(device_count);
        check(clGetDeviceIDs(platforms[p], CL_DEVICE_TYPE_ALL,
                             device_count, devices.data(), nullptr),
              "clGetDeviceIDs");

        for (cl_uint d = 0; d < device_count; ++d) {
            print_device(devices[d], d);
        }
    }
```

```
    return 0;
}
```

Minta mérési napló:

A következő napló reprezentatív sablon; a valós laborban az olvasó gépén az értékek eltérhetnek. Az OpenCL eszközlabor nem teljesítmény-mérés, ezért nincs warm-up és ismételt benchmark-statisztika. A `time` kimenet azonban rögzítésre került, hogy a felsorolási költség is látszódjon.

```
OpenCL platformok: 1
[Platform 0] NVIDIA CUDA
  Vendor: NVIDIA Corporation
  Version: OpenCL 1.2 CUDA 12.3.68
  Devices: 1
    [Device 0]
      Name: NVIDIA GeForce RTX 4070 Laptop GPU
      Vendor: NVIDIA Corporation
      Type: GPU
      Compute Units: 36
      Max Clock: 2175 MHz
      Global Memory: 8192 MiB
      Local Memory: 48 KiB
      Max Work Group Size: 1024
      Max Work Item Sizes: 1024, 1024, 64
      Device Version: OpenCL 1.2 CUDA
      Driver Version: 535.129.03
      OpenCL C Version: OpenCL C 1.2
```

A `CL_PLATFORM_VERSION` értéke ebben a sablonban az NVIDIA OpenCL-ICD szintjét mutatja. Az OpenCL-profil verziója nem feltétlenül azonos a CUDA toolkit verziójával, ezért a labor naplójában külön sorban rögzítendő a CUDA-verzió is.

Mért és várt eredmény szétválasztása:

Megfigyelés	Várt	Mért	Értékelés
OpenCL platformok száma	legalább 1	1	megfelelt
Eszközök száma	legalább 1	1	megfelelt
GPU típusú eszköz	igen	igen	megfelelt
`CL_DEVICE_MAX_COMPUTE_UNITS`	nagyobb, mint 0	36	megfelelt
`CL_DEVICE_GLOBAL_MEM_SIZE`	nagyobb, mint 0	8192 MiB	megfelelt
`CL_DEVICE_MAX_WORK_GROUP_SIZE`	nagyobb, mint 0	1024	megfelelt

A várt feltételek nem hardverspecifikusak, hanem az OpenCL-eszköz érvényességét ellenőrzik. A labor célja nem a „jó GPU” minősítése, hanem annak igazolása, hogy a program képes a rendszeren elérhető OpenCL-eszközöket felsorolni, és a fontos képességeket értelmezni.

A futási idő tájékoztató jellegű:

```
real 0m0.023s
user 0m0.004s
sys  0m0.012s
```

29.1.7 Tipikus hibák és a helyes megoldás

Az OpenCL platform- és eszközfelsorolás során az alábbi hibák fordulnak elő leggyakrabban:

1. **Az első platform kiválasztása gondolkodás nélkül.**
   Helyes megoldás: minden platformot végig kell járni, és a képességek alapján kell választani.
2. **A `CL_DEVICE_TYPE_GPU` kérése, amikor nincs GPU-platform.**
   Helyes megoldás: `CL_DEVICE_TYPE_ALL`-lal térképezzünk fel, és a `CL_DEVICE_TYPE` bitet magunk vizsgáljuk. Ha közvetlen GPU-t kérünk, a `CL_DEVICE_NOT_FOUND` hibát le kell kezelni.
3. **A szöveges paraméterek egymenetes lekérdezése rögzített char-tömbbel.**
   Helyes megoldás: a kétlépéses minta segít elkerülni a puffertúlcsordulást és a hiányos stringeket.
4. **A `CL_DEVICE_MAX_WORK_ITEM_SIZES` fix három elemként való olvasása.**
   Helyes megoldás: először a `CL_DEVICE_MAX_WORK_ITEM_DIMENSIONS` értéket kell lekérni, majd annyi `size_t` elemet olvasni.
5. **A hibakód figyelmen kívül hagyása.**
   Helyes megoldás: minden OpenCL-hívást `check()`-kel ellenőrizni, különösen a felsorolási hívásoknál, ahol a hiányzó eszköz nem kivétel, hanem normál állapot lehet.

29.1.8 Továbbvezetés

Ebben az alfejezetben megszereztük a platform- és eszközazonosítókat, és megtanultuk a fontosabb jellemzők lekérdezését. A következő lépés az, hogy a kiválasztott eszközökhöz **context-et** hozzunk létre, majd a context-hez **parancssort** kapcsoljunk. Erre a Context, queue és memóriaobjektumok alfejezet épít, ahol a context és a memóriaobjektumok konkrét életciklusát is bemutatjuk.

Önellenőrző kérdés: Ha a program `clGetDeviceIDs()` hívását `CL_DEVICE_TYPE_GPU` típussal indítjuk, és a rendszerben nincs GPU, milyen hibakódot ad vissza tipikusan az OpenCL, és milyen módon tehető hordozhatóvá a kód?

29.2 Context, queue és memóriaobjektumok

Az OpenCL erőforrásmodelljének középpontjában három, egymásra épülő fogalom áll: a **context**, a **command queue** és a **memóriaobjektum**. Az előző egységben megismerkedtünk a platform és device kiválasztásával; most azt vizsgáljuk, hogyan szervezzük ezekből a végrehajtáshoz szükséges erőforrásokat. A context egy vagy több eszközt és a hozzájuk tartozó erőforrásokat fogja össze, a command queue az egyes eszközökre irányuló parancsokat sorolja be, a memóriaobjektumok pedig a host és a device közötti, illetve a device-ok közötti adatcserét valósítják meg. E három elem együttes használata nélkül nem lehet OpenCL-programot futtatni; a hibák jelentős része is ezen a rétegen keletkezik.

Ebben az egységben először a context és a queue szerepét, létrehozását és leggyakoribb beállításait tekintjük át, majd részletesen tárgyaljuk a buffer-memóriaobjektumokat és a másolási műveleteket. A lezáró labor egy olyan kis programot ad, amely létrehoz egy buffert, adatot másol a hostról a device-ra, device-ok között, majd vissza a hostra, és megméri a műveletek idejét. A példa segítségével bemutatjuk a hibakezelést, a warm-up és ismétlés módszertanát, valamint a mérési napló helyes kitöltését.

29.2.1 Context: az OpenCL erőforrások összefogója

A **context** az OpenCL-ben egy olyan objektum, amely egy adott platformhoz tartozó eszközök (például egy vagy több GPU, CPU vagy gyorsító) halmazát és a hozzájuk kapcsolódó erőforrásokat – parancssorokat, memóriaobjektumokat, programokat és kerneleket – egyetlen egységbe fogja. A context feladata, hogy biztosítsa az erőforrások közötti konzisztenciát és a hivatkozások érvényességét. Egy memóriaobjektumot mindig egy konkrét contexthez kell létrehozni, és az adott contexthez tartozó eszközök láthatják azt. Ugyanígy egy command queue is egy contexthez és azon belül egy device-hoz kötődik.

A context létrehozása a C++ bindingban a `cl::Context` konstruktorával történik. Az alábbi kódrészlet egyetlen GPU device-ot tartalmazó contextet hoz létre:

```
cl_int err;
cl::Context context(device, nullptr, nullptr, nullptr, &err);
if (err != CL_SUCCESS) {
    std::cerr << "Context creation failed: " << err << "\n";
    // hiba kezelése
}
```

A konstruktor paraméterei: a device-ok listája (itt egyetlen `cl::Device`), egy opcionális `cl_context_properties` tömb, egy opcionális hibakallback, egy opcionális user data pointer, és egy hibakód pointer. A properties tömb segítségével például platformot adhatunk meg, ha több platform van jelen; ezt gyakran a platform kiválasztása után érdemes beállítani. A C++ binding egyszerűsített konstruktora elfogad egy `std::vector<cl::Device>`-ot is, így több eszközt is egy contextbe foglalhatunk:

```
std::vector<cl::Device> devices = {gpu0, gpu1};
cl::Context context(devices);
```

A context élettartama alatt minden benne létrehozott erőforrásnak élnie kell. Ha a context megsemmisül, az összes hozzá tartozó queue, buffer, program és kernel érvénytelenné válik. A C++ binding automatikus referenciaszámlálást végez, de a nyers C API-ban a `clReleaseContext` hívás elmulasztása memóriaszivárgáshoz vezet. A context tehát nem csupán kényelmi csoportosítás, hanem az erőforrások életciklusának központi eleme.

Fontos megérteni, hogy egy context több device-ot is tartalmazhat, de a device-ok csak ugyanazon a platformon belül lehetnek. Ez azt jelenti, hogy nem lehet egyetlen contextben egyszerre egy NVIDIA GPU-t és egy Intel iGPU-t összekapcsolni, ha azok különböző platformokhoz tartoznak. Ilyenkor minden platformhoz külön contextet kell létrehozni. A context hatóköre meghatározza a memória láthatóságát is: a contexthez tartozó eszközök számára a memóriaobjektumok közvetlenül elérhetők, de más contextből nem. Több eszköz esetén a parancsok ütemezése és az adatmozgatás a contexten belül történik.

29.2.2 Command queue: a parancsok sorba állítása

A **command queue** (parancssor) az a mechanizmus, amelyen keresztül parancsokat küldünk egy adott device-nak. Minden parancssor pontosan egy device-hoz tartozik, és az adott device-ra vonatkozó kernelindításokat, memóriaműveleteket és szinkronizációs parancsokat sorolja be. A queue létrehozása a `cl::CommandQueue` konstruktorával történik:

```
cl_command_queue_properties props = CL_QUEUE_PROFILING_ENABLE;
cl::CommandQueue queue(context, device, props, &err);
```

A `props` paraméterrel szabályozhatjuk a queue viselkedését. A két leggyakoribb tulajdonság:

- `CL_QUEUE_OUT_OF_ORDER_EXEC_MODE_ENABLE`: engedélyezi a parancsok sorrendtől független végrehajtását. Alapértelmezés szerint a queue **in-order**, azaz a parancsok abban a sorrendben hajtódnak végre, ahogyan a queue-ba kerültek. Ez a legegyszerűbb és legbiztonságosabb mód, mert a függőségek implicit módon érvényesülnek. Az out-of-order mód nagyobb átfedést tesz lehetővé, de akkor a parancsok közötti függőségeket explicit eseményekkel (event) kell kezelni.
- `CL_QUEUE_PROFILING_ENABLE`: lehetővé teszi a parancsok időzítését. Ekkor a queue-ba beküldött parancsokhoz eseményobjektumokat rendelhetünk, és azokból kiolvashatjuk a parancs kezdő és befejező időpontját. Ez nélkülözhetetlen a teljesítménymméréshez, de minimális többletköltséggel jár.

A parancssor használata a következő lépésekből áll: parancsok beküldése (`enqueue`), majd szinkronizáció (`finish` vagy `flush`). A `finish()` blokkol, amíg a queue összes korábban beküldött parancsa be nem fejeződik; a `flush()` csak a parancsokat küldi el a device-nak, de nem várja meg a befejezést. A `finish()` tipikusan a mérési ciklus végén vagy a program befejezése előtt szükséges, hogy az összes aszinkron művelet befejeződjön.

A parancsok három fő csoportba sorolhatók:

1. **Kernelvégrehajtás**: `enqueueNDRangeKernel` vagy `enqueueTask` (utóbbi OpenCL 1.2-ben). Ezeket a következő egység tárgyalja részletesen.
2. **Memóriaműveletek**: `enqueueWriteBuffer`, `enqueueReadBuffer`, `enqueueCopyBuffer`, `enqueueMapBuffer`, stb. Ezek a host és device közötti, illetve device-ok közötti adatmozgatást végzik.
3. **Szinkronizáció**: `enqueueMarker`, `enqueueBarrierWithWaitList` (OpenCL 2.0+). Ezekkel jelölhetünk pontokat a queue-ban, vagy várhatunk eseményekre.

A parancsok minden esetben **aszinkron módon** kerülnek a queue-ba, kivéve, ha a hívás blokkoló (`blocking == CL_TRUE`). Például a `enqueueWriteBuffer` blokkoló változata megvárja, amíg az adat átmásolódik a device-ra, míg a nem blokkoló változat azonnal visszatér, és a tényleges másolás később történik. A blokkoló hívás egyszerűbb, de a teljesítmény szempontjából gyakran jobb, ha több nem blokkoló parancsot küldünk be, és egyetlen `finish()`-sel várjuk meg őket.

A hibakezelés szempontjából fontos, hogy a queue létrehozása is hibát adhat, például ha a megadott device nem tartozik a contexthez, vagy a tulajdonságok érvénytelenek. A C++ binding ilyenkor `CL_INVALID_DEVICE` vagy `CL_INVALID_QUEUE_PROPERTIES` hibát jelez. Mindig ellenőrizzük a visszatérési értéket vagy használjunk kivételt.

29.2.3 Memóriaobjektumok: buffer, image és pipe

Az OpenCL a device memóriát **memóriaobjektumok** formájában absztrahálja. A leggyakrabban használt típus a **buffer**, amely egy egydimenziós, folytonos memóriaterületet reprezentál. Emellett léteznek **image** objektumok (2D/3D textúrák), valamint OpenCL 2.0-tól **pipe** objektumok. Ebben a fejezetben a bufferre összpontosítunk, mivel a legtöbb adatpárhuzamos feladat ezt használja.

A buffer létrehozása a `cl::Buffer` konstruktorával történik:

```
cl::Buffer buffer(context, flags, size, host_ptr, &err);
```

A paraméterek:

- `context`: a context, amelyhez a buffer tartozik.
- `flags`: a buffer tulajdonságait leíró bitmaszk (`cl_mem_flags`).

- `size`: a buffer mérete bájtban.
- `host_ptr`: opcionális pointer a host memóriára, amelyet a flag-ek szerint használunk.
- `err`: hibakód pointer.

A **memóriaflag-ek** határozzák meg a buffer hozzáférési módját és a host-memória kapcsolatát. A legalapvetőbb hozzáférési flag-ek:

- `CL_MEM_READ_WRITE`: a kernel olvashatja és írhatja a buffert. Ez az alapértelmezés, ha nincs más megadva.
- `CL_MEM_READ_ONLY`: a kernel csak olvashatja.
- `CL_MEM_WRITE_ONLY`: a kernel csak írhatja.

Bár ezek a flag-ek deklarációs célt szolgálnak, a futás közbeni viselkedést a kernel deklarációja is befolyásolja; a flag-ek és a kernel paraméterei közötti inkonzisztencia hibához vezethet.

A host-memória használatát szabályozó flag-ek a következők:

- `CL_MEM_COPY_HOST_PTR`: a létrehozáskor a `host_ptr` által mutatott adatot a device memóriába másolja. Ez a leggyakoribb mód, ha a host adatot egyszer fel kell tölteni a device-ra. A host pointert a hívás után azonnal fel lehet szabadítani, mert a device saját másolattal rendelkezik.
- `CL_MEM_USE_HOST_PTR`: a device a megadott host memóriaterületet használja közvetlenül (vagy legalábbis arra hivatkozik). Ez lehetővé teszi a nullás másolást bizonyos eszközökön, de a host pointernek érvényesnek kell maradnia a buffer teljes élettartama alatt. A host oldali módosítások láthatóvá válhatnak a device számára, és viszont, de ez implementációfüggő; gyakran a host és device közötti koherenciát a `clEnqueueMapBuffer` vagy explicit másolás biztosítja.
- `CL_MEM_ALLOC_HOST_PTR`: a device memóriát foglal, és egyúttal host számára is elérhető címet biztosít. Ez gyakran egyesített memória architektúrákon hatékony, de nem minden eszköz támogatja. A `host_ptr` paramétert ilyenkor `nullptr`-ként adjuk át, és a host pointert később a `clEnqueueMapBuffer` segítségével kérhetjük le.
- `CL_MEM_COPY_HOST_PTR | CL_MEM_ALLOC_HOST_PTR` kombinációja: a device memóriát foglal, majd a host adatot bemásolja.

Az OpenCL 2.0 további flag-eket vezetett be a **Shared Virtual Memory (SVM)** támogatására:

- `CL_MEM_SVM_FINE_GRAIN_BUFFER`: finom szemcséjű SVM, ahol a host és device memória szinkronban van.
- `CL_MEM_SVM_ATOMICS`: atomi műveletek engedélyezése az SVM-ben.
- `CL_MEM_SVM_FINE_GRAIN_SYSTEM`: rendszerszintű finom szemcséjű SVM.

Ezeket itt csak megemlítjük, mert a hagyományos buffer-kezelés a legtöbb feladatra elegendő.

A flag-eket bitenkénti VAGY (`|`) művelettel kombinálhatjuk. Például:

```
cl::Buffer buf(context, CL_MEM_READ_WRITE | CL_MEM_COPY_HOST_PTR, bytes, hostData.data(), &err);
```

Fontos, hogy a `CL_MEM_USE_HOST_PTR` és `CL_MEM_COPY_HOST_PTR` egyszerre nem adható meg, mert ellentmondanak egymásnak. Hasonlóképpen a `CL_MEM_ALLOC_HOST_PTR` önmagában nem igényel `host_ptr`-t.

A buffer létrehozása során fellépő gyakori hibák:

- `CL_INVALID_CONTEXT`: a megadott context érvénytelen.
- `CL_INVALID_VALUE`: a flag-ek érvénytelenek vagy egymással ütközők.

- `CL_INVALID_BUFFER_SIZE`: a méret 0 vagy túl nagy.
- `CL_INVALID_HOST_PTR`: a `host_ptr` érvénytelen (például `CL_MEM_COPY_HOST_PTR` esetén `nullptr`).
- `CL_MEM_OBJECT_ALLOCATION_FAILURE`: a device memória foglalása nem sikerült.
- `CL_OUT_OF_RESOURCES` vagy `CL_OUT_OF_HOST_MEMORY`: erőforrás-hiány.

A C++ bindingban a `cl::Buffer` objektum RAII-t valósít meg, tehát a buffer automatikusan felszabadul, amikor az objektum megsemmisül. A nyers C API-ban a `clReleaseMemObject` hívás szükséges.

29.2.4 Buffer-másolás: enqueueWriteBuffer, enqueueReadBuffer, enqueueCopyBuffer

A buffer létrehozása után adatot kell mozgatni a host és a device között. Az OpenCL erre a parancssoron keresztül nyújt lehetőséget:

- `enqueueWriteBuffer`: a host memóriából a device bufferbe ír.
- `enqueueReadBuffer`: a device bufferből a host memóriába olvas.
- `enqueueCopyBuffer`: device bufferből egy másik device bufferbe másol.

Mindhárom függvény szignatúrája a C++ bindingban hasonló:

```
cl_int enqueueWriteBuffer(const Buffer& buffer, cl_bool blocking, size_t offset, size_t size,
↪  const void* ptr, const vector<Event>* events = nullptr, Event* event = nullptr);
cl_int enqueueReadBuffer(const Buffer& buffer, cl_bool blocking, size_t offset, size_t size,
↪  void* ptr, const vector<Event>* events = nullptr, Event* event = nullptr);
cl_int enqueueCopyBuffer(const Buffer& src, const Buffer& dst, size_t src_offset, size_t
↪  dst_offset, size_t size, const vector<Event>* events = nullptr, Event* event = nullptr);
```

A `blocking` paraméter `CL_TRUE` esetén a hívás blokkol, amíg a másolás be nem fejeződik; `CL_FALSE` esetén a parancs csak a queue-ba kerül, és a hívás azonnal visszatér. Az `offset` és `size` segítségével a buffer egy részét is kezelhetjük. Az `events` paraméterrel az előző parancsoktól függőséget adhatunk meg, az `event` kimenő paraméterrel pedig a parancs befejezését jelző eseményt kaphatunk (ez a profilinghoz is szükséges).

A másolási műveletek hibái:

- `CL_INVALID_COMMAND_QUEUE`: a queue nem tartozik a contexthez.
- `CL_INVALID_MEM_OBJECT`: a buffer érvénytelen.
- `CL_INVALID_VALUE`: az offset vagy size túlcsordul, vagy nem illeszkedik a bufferhez.
- `CL_INVALID_EVENT_WAIT_LIST`: a várakozási lista érvénytelen.
- `CL_EXEC_STATUS_ERROR_FOR_EVENTS_IN_WAIT_LIST`: ha a várakozási listában szereplő esemény hibásan fejeződött be.

Fontos szempont, hogy a host pointer, amelyet a `enqueueWriteBuffer` vagy `enqueueReadBuffer` kap, érvényes legyen a hívás idején. Blokkoló hívásnál a hívás visszatérte után a pointer felszabadítható; nem blokkoló hívásnál viszont a pointernek érvényesnek kell maradnia, amíg a parancs be nem fejeződik. Ezért nem blokkoló hívásoknál gyakran szükséges egy `cl::Event`-et kérni, és később `event.wait()`-et hívni.

A laborban a blokkoló változatot használjuk, mert az egyszerűbb és a mérési ciklusban a `finish()` úgyis biztosítja a befejezést. A nem blokkoló változatot a későbbi fejezetekben, a stream-ek és az átfedés tárgyalásakor fogjuk kihasználni.

29.2.5 Tipikus hibák és megoldások

Az OpenCL context, queue és buffer kezelése során a kezdők leggyakrabban a következő hibákba ütköznek:

1. **Context hiánya vagy rossz device hozzárendelés.** Ha egy memóriaobjektumot egy olyan contexthez próbálunk létrehozni, amely nem tartalmazza a kívánt device-ot, akkor a buffer létrehozása sikeres lehet, de a queue-val való használat `CL_INVALID_CONTEXT` hibát ad. **Megoldás:** mindig ellenőrizzük, hogy a device benne van-e a contextben, például a `context.getInfo<CL_CONTEXT_DEVICES>()` lekérdezéssel.

2. **Helytelen flag kombináció.** A `CL_MEM_COPY_HOST_PTR` és `CL_MEM_USE_HOST_PTR` egyszerre való megadása érvénytelen. Hasonlóképpen a `CL_MEM_READ_ONLY` bufferbe írás a kernelből futás közbeni hibát okozhat. **Megoldás:** tanulmányozzuk a flag-ek dokumentációját, és csak olyan kombinációt használjunk, amelynek értelme van.

3. **Host pointer érvénytelensége.** Ha `CL_MEM_USE_HOST_PTR` flaggel hozunk létre buffert, a host pointert nem szabad felszabadítani, amíg a buffer él. Ellenkező esetben a device memória érvénytelen területre mutat, ami futás közbeni hibákhoz vagy adatkorrupcióhoz vezet. **Megoldás:** használjunk `CL_MEM_COPY_HOST_PTR`-t, ha a host adatra nincs szükség a továbbiakban, vagy tartsuk életben a host bufferünket.

4. **Blokkoló és nem blokkoló hívások keverése.** Ha nem blokkoló `enqueueWriteBuffer` után azonnal blokkoló `enqueueReadBuffer`-t hívunk, a read megelőzheti a write befejezését, és nem definiált adatot olvasunk. **Megoldás:** használjunk blokkoló hívásokat, vagy kérjünk eseményt a write-ból, és adjuk át a read várakozási listájára.

5. **Szinkronizáció hiánya.** A queue parancsai aszinkron módon futnak; ha a program véget ér anélkül, hogy `finish()`-t hívnánk, a parancsok esetleg nem fejeződnek be, és a memóriaobjektumok felszabadulhatnak a parancsok befejezése előtt. **Megoldás:** a program végén mindig hívjunk `queue.finish()`-t, vagy használjunk RAII-t.

6. **Memóriaszivárgás.** A nyers C API-ban minden `clCreateBuffer` után szükséges a `clReleaseMemObject`. A C++ binding automatikusan felszabadít, de ha dinamikusan hozunk létre objektumokat, ügyeljünk a pointerek életciklusára. **Megoldás:** lehetőleg használjuk a C++ bindingot, és kerüljük a nyers `new`/`delete`-et.

7. **Kihagyott hibakezelés.** A legtöbb OpenCL hívás visszatérési értéket ad; ha nem ellenőrizzük, a hiba csak később, nehezen értelmezhető módon jelentkezik. **Megoldás:** minden hívás után ellenőrizzük a hibakódot, vagy használjunk kivételt dobó wrapper-t.

A fenti hibák többsége már a fejlesztés korai szakaszában, egyszerű tesztekkel kiszűrhető. A laborban bemutatott kód ezért szándékosan sok hibakezelést tartalmaz.

29.2.6 Labor: buffer létrehozása és másolása

Ebben a laborban egy olyan OpenCL-programot készítünk, amely létrehoz egy contextet, egy parancssort, két device buffert, majd adatot másol a hostról az egyik bufferbe, onnan a másikba, végül vissza a hostra. A program megméri a három másolási művelet teljes idejét, ellenőrzi az adatokat, és kiszámítja a host–device sávszélességet. A labor célja a context, queue és buffer fogalmak gyakorlati összekapcsolása.

29.2.7 Környezet

- GPU: NVIDIA GeForce RTX 3060 (12 GB) – példa, a te környezetedben más lehet
- Driver: 550.54.14 (Linux, x86_64)
- CUDA verzió: 12.4 (a driver része; az OpenCL futáshoz nem szükséges, de a rendszerben jelen van)
- OpenCL platform: NVIDIA CUDA OpenCL 3.0
- OpenCL fejlesztői csomag: OpenCL-Headers v2023.04.17, OpenCL-ICD-Loader
- Fordító: g++ 13.2, `-std=c++17`
- Operációs rendszer: Ubuntu 24.04 (izolált konténer)

A mérés izolált konténerben fut, más felhasználói terhelés nélkül. A GPU hőmérséklete a mérés előtt stabil (35 °C), az órajel a gyári beállításokon.

29.2.8 Forráskód

Az alábbi kód a C++ bindingot használja. A hibakezelés egyszerű, de minden fontos lépést ellenőriz. A mérési ciklus előtt három warm-up ismétlést végzünk, majd tíz mérést rögzítünk. Minden iterációban a három másolási műveletet blokkoló módban hajtjuk végre, és a végén `finish()`-t hívunk. A mérési időt `std::chrono` segítségével rögzítjük.

```
#define CL_HPP_TARGET_OPENCL_VERSION 120
#define CL_HPP_MINIMUM_OPENCL_VERSION 120
#include <CL/opencl.hpp>
#include <iostream>
#include <vector>
#include <chrono>
#include <numeric>
#include <cmath>
#include <stdexcept>
#include <string>

const char* clErrorToString(cl_int err) {
    switch (err) {
        case CL_SUCCESS: return "CL_SUCCESS";
        case CL_INVALID_CONTEXT: return "CL_INVALID_CONTEXT";
        case CL_INVALID_QUEUE_PROPERTIES: return "CL_INVALID_QUEUE_PROPERTIES";
        case CL_INVALID_MEM_OBJECT: return "CL_INVALID_MEM_OBJECT";
        case CL_INVALID_VALUE: return "CL_INVALID_VALUE";
        case CL_INVALID_BUFFER_SIZE: return "CL_INVALID_BUFFER_SIZE";
        case CL_INVALID_HOST_PTR: return "CL_INVALID_HOST_PTR";
        case CL_MEM_OBJECT_ALLOCATION_FAILURE: return "CL_MEM_OBJECT_ALLOCATION_FAILURE";
        case CL_OUT_OF_RESOURCES: return "CL_OUT_OF_RESOURCES";
        case CL_OUT_OF_HOST_MEMORY: return "CL_OUT_OF_HOST_MEMORY";
        default: return "UNKNOWN";
    }
}

void checkErr(cl_int err, const std::string& name) {
    if (err != CL_SUCCESS) {
        std::cerr << "ERROR: " << name << " failed: "
                  << clErrorToString(err) << " (" << err << ")\n";
        throw std::runtime_error(name);
    }
}

int main() {
    try {
        // Platform és device kiválasztása (ld. gpu110)
```

```
std::vector<cl::Platform> platforms;
cl::Platform::get(&platforms);
if (platforms.empty()) throw std::runtime_error("No OpenCL platforms");
cl::Platform platform = platforms[0];

std::vector<cl::Device> devices;
platform.getDevices(CL_DEVICE_TYPE_GPU, &devices);
if (devices.empty()) throw std::runtime_error("No GPU devices");
cl::Device device = devices[0];
std::cout << "Device: " << device.getInfo<CL_DEVICE_NAME>() << "\n";

// 1) Context létrehozása
cl_int err;
cl::Context context(device, nullptr, nullptr, nullptr, &err);
checkErr(err, "Context creation");

// 2) Command queue létrehozása (in-order, profiling)
cl_command_queue_properties props = CL_QUEUE_PROFILING_ENABLE;
cl::CommandQueue queue(context, device, props, &err);
checkErr(err, "CommandQueue creation");

// 3) Buffer paraméterek
const size_t N = 1 << 22;          // 4 Mi elem
const size_t bytes = N * sizeof(float);
std::vector<float> hostData(N);
for (size_t i = 0; i < N; ++i)
    hostData[i] = static_cast<float>(i) * 0.001f;

// Device buffer A: host adatból másol
cl::Buffer deviceBufA(context, CL_MEM_READ_WRITE | CL_MEM_COPY_HOST_PTR,
                      bytes, hostData.data(), &err);
checkErr(err, "Buffer A creation");

// Device buffer B: csak device oldali
cl::Buffer deviceBufB(context, CL_MEM_READ_WRITE, bytes, nullptr, &err);
checkErr(err, "Buffer B creation");

// Host visszaolvasási terület
std::vector<float> hostResult(N, 0.0f);

// 4) Warm-up
const int warmup = 3;
for (int i = 0; i < warmup; ++i) {
    queue.enqueueWriteBuffer(deviceBufA, CL_TRUE, 0, bytes, hostData.data());
    queue.enqueueCopyBuffer(deviceBufA, deviceBufB, 0, 0, bytes);
    queue.enqueueReadBuffer(deviceBufB, CL_TRUE, 0, bytes, hostResult.data());
    queue.finish();
}

// 5) Adatellenőrzés egyszer
if (hostResult != hostData) {
    throw std::runtime_error("Data mismatch after copy");
}
std::cout << "Data verification: OK\n";

// 6) Mérés
const int iters = 10;
std::vector<double> times(iters);
for (int i = 0; i < iters; ++i) {
    auto start = std::chrono::steady_clock::now();
    queue.enqueueWriteBuffer(deviceBufA, CL_TRUE, 0, bytes, hostData.data());
    queue.enqueueCopyBuffer(deviceBufA, deviceBufB, 0, 0, bytes);
    queue.enqueueReadBuffer(deviceBufB, CL_TRUE, 0, bytes, hostResult.data());
```

```
            queue.finish();
            auto end = std::chrono::steady_clock::now();
            times[i] = std::chrono::duration<double, std::milli>(end - start).count();
        }

        double mean = std::accumulate(times.begin(), times.end(), 0.0) / iters;
        double sq_sum = 0.0;
        for (double t : times) sq_sum += (t - mean) * (t - mean);
        double stdev = std::sqrt(sq_sum / iters);

        // A host<->device sávszélesség: write + read (device-device copy nem terheli a host
        ↪  linket)
        double total_bytes = 2.0 * static_cast<double>(bytes);
        double bw_gbps = (total_bytes / (mean / 1000.0)) / 1e9;

        std::cout << "Mean time: " << mean << " ms\n";
        std::cout << "Std dev:   " << stdev << " ms\n";
        std::cout << "Host<->device bandwidth: " << bw_gbps << " GB/s\n";

        // RAII felszabadítja a queue-t, buffereket, contextet
        return 0;
    } catch (const cl::Error& e) {
        std::cerr << "OpenCL error: " << e.what() << " (" << e.err() << ")\n";
        return 1;
    } catch (const std::exception& e) {
        std::cerr << "Error: " << e.what() << "\n";
        return 1;
    }
}
```

29.2.9 Buildparancs

```
g++ -std=c++17 -O2 -I/usr/include/CL -o buffer_lab buffer_lab.cpp -lOpenCL
```

Feltételezve, hogy az OpenCL fejlécek az `/usr/include/CL` alatt vannak, és az ICD loader könyvtár elérhető. Ha a fejlécek máshol vannak, módosítsuk az include útvonalat.

29.2.10 Futtatási parancs

```
./buffer_lab
```

A program kimenete tartalmazza a device nevét, az adatellenőrzés eredményét és a mérési statisztikát.

29.2.11 Mérési napló

Az alábbi táblázat egy izolált tesztkörnyezetből származó példa eredményeket mutat. A labor során a saját környezetedben mért értékeket ebbe a táblázatba kell beírni. A várt sávszélesség a PCIe 3.0 x16 elméleti maximuma (kb. 15,75 GB/s) alatt, de a gyakorlatban 8–12 GB/s között várható.

Ismétlés	Mért idő (ms)	Mért sávszélesség (GB/s)	Várt sávszélesség (GB/s)	Megjegyzés
1	2,41	10,4	8–12	példaérték
2	2,38	10,5	8–12	
3	2,40	10,4	8–12	

Ismétlés	Mért idő (ms)	Mért sávszélesség (GB/s)	Várt sávszélesség (GB/s)	Megjegyzés
4	2,39	10,5	8–12	
5	2,42	10,4	8–12	
6	2,38	10,6	8–12	
7	2,41	10,4	8–12	
8	2,40	10,5	8–12	
9	2,39	10,5	8–12	
10	2,42	10,4	8–12	
Átlag	**2,40**	**10,46**	8–12	
Szórás	**0,015**	**0,07**		

A táblázatban szereplő értékek **példaértékek**, nem feltétlenül a te környezetedben mért valós adatok. A labor célja, hogy a mérési módszertant alkalmazd, és a saját számadataidat rögzítsd.

A mért sávszélesség kiszámítása: a három műveletből kettő (write és read) host–device adatforgalmat generál, összesen `2 * bytes` bájtot. Az idő a teljes ciklus ideje, beleértve a device-device copy-t is, ami nem terheli a PCIe sávot, de a mérési időben szerepel. Ezért a számított sávszélesség konzervatív alsó becslés. A gyakorlatban a device-device copy ideje elhanyagolható a host–device másolásokhoz képest.

29.2.12 Értelmezés

Az eredmény azt mutatja, hogy a buffer-másolási műveletek a PCIe sávszélesség közelében mozognak, ami azt jelzi, hogy a mérési overhead minimális. Ha a mért sávszélesség jelentősen eltér a várttól, érdemes ellenőrizni a következőket:

- A GPU és a CPU közötti PCIe sávszélesség (például `nvidia-smi topo -m` vagy `pcietop`).
- A háttérterhelés (más processzek).
- Az energiagazdálkodási beállítások (a GPU órajele alacsony lehet).
- A CPU oldali memória sávszélessége (ha a host adat nagy).

29.2.13 Önellenőrző kérdés

Mi a különbség a `CL_MEM_COPY_HOST_PTR` és `CL_MEM_USE_HOST_PTR` flag között? Melyik esetben módosítható a host pointer a buffer létrehozása után, és milyen következményekkel jár ez a device memóriára nézve?

Válasz: A `CL_MEM_COPY_HOST_PTR` esetén a létrehozáskor a host adatot a device memóriába másoljuk, így a host pointer a hívás után felszabadítható, és a host oldali módosítások nem befolyásolják a device buffert. A `CL_MEM_USE_HOST_PTR` esetén a device a megadott host memóriaterületet használja (vagy arra hivatkozik), ezért a host pointernek érvényesnek kell maradnia a buffer élettartama alatt. A host oldali módosítások elvileg láthatóvá válhatnak a device számára, de ez implementációfüggő; a biztonságos megoldás a `clEnqueueMapBuffer` használata vagy explicit másolás.

Ezzel az egységgel lefedtük az OpenCL erőforrásainak központi elemeit: a contextet, a command queue-t és a buffer-memóriaobjektumokat. A következő egységben (Kernel és launch) ezekre építve bemutatjuk, hogyan hozhatunk létre programot és kernelt, és hogyan indíthatjuk el az első OpenCL számítást.

29.3 Kernel és launch

Az OpenCL kernel az a programegység, amely a device-on fut; a work-group a work-itemek azon csoportja, amelyek együtt ütemeződnek, és bizonyos erőforrásokat megosztanak. A kernel és a work-group fogalma nem választható el a global és local work size megadásától, valamint az aszinkron végrehajtást vezérlő eseményektől. Ebben az alfejezetben ezeket a fogalmakat járjuk körbe egy vektorösszeadó laboron keresztül, amely a `gpu111`-ben bevezetett platform, context, queue és memóriaobjektum-modellre épít.

29.3.1 1. Az OpenCL kernel és a work-group

Az OpenCL kernel egy olyan függvény, amelyet a hoszt a device-on hajt végre. A kernel forrása OpenCL C nyelven íródik, és a `__kernel` minősítő jelöli, hogy az adott függvény belépési pont a device-on. A kernel minden egyes meghívása egy **work-itemet** hoz létre, amely a CUDA thread fogalmának felel meg. A work-itemek azonban nem független szálakként léteznek, hanem **work-groupokba** rendeződnek. A work-group a work-itemek olyan halmaza, amelyek együtt futnak egy számítási egységen, és közös **local memóriát**, valamint szinkronizációs lehetőséget kapnak.

A kernel a bemeneti és kimeneti adatokat a memóriaobjektumokon keresztül éri el. Az OpenCL C négy címtérminősítőt ismer:

- `__global` – a device globális memóriája, a hoszt által írt és olvasott pufferek.
- `__constant` – a device konstans memóriája, hoszt által inicializált, a kernelben csak olvasható.
- `__local` – a work-group által megosztott memória, a kernelben deklarálható vagy hosztból allokálható.
- `__private` – a work-item privát memóriája, alapértelmezés szerint a kernelben deklarált változók itt élnek.

A kernel a saját indexeit beépített függvényekkel kérdezi le. A leggyakrabban használt függvények:

- `get_global_id(dim)` – a work-item globális indexe a `dim` dimenzióban.
- `get_local_id(dim)` – a work-item indexe a saját work-groupján belül.
- `get_group_id(dim)` – a work-group indexe.
- `get_global_size(dim)` – a teljes work-itemek száma a `dim` dimenzióban.
- `get_local_size(dim)` – a work-group mérete a `dim` dimenzióban.
- `get_num_groups(dim)` – a work-groupok száma a `dim` dimenzióban.

Ezek a függvények a kernel futása során állandóak, és a hoszt által megadott `global_work_size` és `local_work_size` értékekből származnak. A work-itemek egy dimenziós, két dimenziós vagy három dimenziós indexeléssel azonosíthatók; a dimenziók száma a kernel indításakor adható meg.

Példa egy egyszerű kernelre, amely két vektor elemenkénti összegét számítja ki:

```
// OpenCL C 1.2, clBuildProgram
__kernel void vector_add(__global const float* a,
                         __global const float* b,
                         __global float* c) {
    const int i = get_global_id(0);   // globális index
    c[i] = a[i] + b[i];
}
```

Itt a `get_global_id(0)` adja meg, hogy az adott work-item melyik elemet dolgozza fel. A kernel maga nem ismeri a vektor hosszát; azt a hosztnak kell ellenőriznie, vagy a kernelnek kell határellenőrzést végeznie, ha a global work size nem egyenlő a vektor hosszával. A fenti kernel feltételezi, hogy a global work size pontosan a vektor hossza.

A work-group szerepe akkor válik fontossá, ha a work-itemeknek együtt kell működniük. A work-groupon belüli szinkronizáció a `barrier()` függvénnyel érhető el, amely a work-group összes work-itemjét várakoztatja, amíg mindegyik el nem éri a hívási pontot. A `barrier()` a work-groupon belüli memóriahozzáférések rendezését is biztosítja. A work-groupok között nincs ilyen szinkronizációs lehetőség; a work-groupok közötti függőséget csak a hoszt tudja kezelni újabb kernelindítással vagy eseményekkel.

A work-group méretét a hoszt adja meg, és a választható értékeket a device hardveres korlátai befolyásolják. A lekérdezés a `clGetDeviceInfo` függvénnyel történik, például `CL_DEVICE_MAX_WORK_GROUP_SIZE` és `CL_DEVICE_MAX_WORK_ITEM_SIZES` paraméterekkel. A gyakorlatban a work-group méret jellemzően 32, 64, 128 vagy 256, de ez hardverfüggő. A CUDA-ból ismert warpmérethez hasonlóan az OpenCL eszközök is bizonyos work-item csoportokban ütemeznek, ezért a work-group méret megválasztása jelentősen befolyásolja a teljesítményt. Ezzel a `gpu113` és későbbi fejezetek foglalkoznak részletesen.

29.3.2 2. Global és local work size

A kernel indításakor a hoszt megadja a **global work size**-ot, amely a teljes végrehajtandó work-itemek számát határozza meg, és opcionálisan a **local work size**-ot, amely a work-group méretét adja meg. Az OpenCL API-ban ezt a `clEnqueueNDRangeKernel` függvény végzi:

```
cl_int clEnqueueNDRangeKernel(
    cl_command_queue command_queue,
    cl_kernel kernel,
    cl_uint work_dim,
    const size_t* global_work_offset,
    const size_t* global_work_size,
    const size_t* local_work_size,
    cl_uint num_events_in_wait_list,
    const cl_event* event_wait_list,
    cl_event* event);
```

A `work_dim` értéke 1, 2 vagy 3 lehet. A `global_work_size` egy `size_t` tömb, amelynek hossza `work_dim`. A `local_work_size` egy `size_t` tömb, vagy `NULL`. Ha `NULL`, az implementáció választja meg a work-group méretet, de ilyenkor a kernel nem feltételezhet konkrét work-group méretet. A `global_work_offset` eltolást adhat meg, de a legtöbb esetben `NULL` (nulla eltolás). A `event_wait_list` és `num_events_in_wait_list` a kernel indítását megelőző eseményeket adja meg; a `event` pedig a kernel végrehajtását jelző eseményt fogja visszaadni.

A global work size és a local work size közötti kapcsolat kritikus. A global work size minden dimenzióban osztható kell legyen a local work size-szal, különben az OpenCL a `CL_INVALID_WORK_GROUP_SIZE` hibát adja. Ha a local work size `NULL`, akkor ez a feltétel automatikusan teljesül, mert az implementáció választ a global work size-hoz illeszkedő méretet.

Ha a hoszt explicit local work size-ot ad meg, akkor a global work size minden dimenzióban osztható kell legyen a megfelelő local work size-szal.

Például ha a vektor hossza 1024, akkor választhatunk:

- 1D: `global_work_size = {1024}`, `local_work_size = {256}` → 4 work-group
- 2D: `global_work_size = {32, 32}`, `local_work_size = {8, 8}` → 4×4 = 16 work-group
- 3D: `global_work_size = {8, 8, 16}`, `local_work_size = {4, 4, 4}` → 2×2×4 = 16 work-group

Ha a vektor hossza nem osztható a kívánt work-group mérettel, például 1000 elemet szeretnénk feldolgozni 256-os work-groupökkel, akkor a global work size-ot fel kell kerekíteni a következő osztható értékre, például 1024-re, és a kernelben határellenőrzést kell végezni:

```
// OpenCL C 1.2, clBuildProgram
__kernel void vector_add_safe(__global const float* a,
                              __global const float* b,
                              __global float* c,
                              const int n) {
    const int i = get_global_id(0);
    if (i < n) {
        c[i] = a[i] + b[i];
    }
}
```

Ilyenkor a hoszt a `global_work_size`-t 1024-re állítja, és a kernel a `n` segítségével eldönti, hogy az adott work-itemnek van-e érvényes munkája. Ez a minta megegyezik a CUDA-ban megszokott grid-stride loop vagy határellenőrzés mintájával.

A work-group méret megválasztásánál figyelembe kell venni a device hardveres korlátait. A `clGetDeviceInfo` lekérdezéssel kaphatjuk meg a `CL_DEVICE_MAX_WORK_GROUP_SIZE` értéket, amely a maximális work-item szám egy work-groupben. Emellett a `CL_DEVICE_MAX_WORK_ITEM_SIZES` egy `size_t` tömb, amely dimenziónként adja meg a maximális work-item számot. Például egy eszközön lehet, hogy a maximális work-group méret 1024, de a 2D-s maximális méret {1024, 1024, 64}. A work-group méret nem haladhatja meg ezeket a korlátokat, és a device által támogatott work-group méretek halmaza gyakran a warpméret többszöröse.

A kernelben a work-group mérete a `get_local_size(0)` függvénnyel kérdezhető le. Ez hasznos például akkor, ha a kernel a work-group méretétől függően végez számítást, vagy ha a local memória méretét dinamikusan kell meghatározni.

29.3.3 3. Események

Az OpenCL parancssor aszinkron modellt követ. A hoszt parancsokat ad ki a queue-ba, de a parancsok végrehajtása nem feltétlenül azonnal történik. A hoszt és a device közötti szinkronizációhoz, valamint a parancsok közötti függőségek kifejezéséhez az OpenCL **eseményeket** használ.

Egy esemény (`cl_event`) egy parancs állapotát reprezentálja. A legtöbb `clEnqueue*` függvény utolsó paramétere egy `cl_event*` mutató, amelyre a függvény az esemény objektumot írja. Az esemény állapotát a `clGetEventInfo` függvénnyel lehet lekérdezni, például `CL_EVENT_COMMAND_TYPE`, `CL_EVENT_COMMAND_EXECUTION_STATUS`. Az állapot lehet `CL_QUEUED`, `CL_SUBMITTED`, `CL_RUNNING` vagy `CL_COMPLETE`.

A hoszt a `clWaitForEvents` függvénnyel várhat egy vagy több esemény befejezésére:

```
cl_int clWaitForEvents(cl_uint num_events, const cl_event* event_list);
```

Ez blokkolja a hosztot, amíg a megadott események mindegyike `CL_COMPLETE` állapotba nem kerül. A `clFinish` függvény a queue összes parancsának befejezését várja, ami gyakran egyszerűbb, ha nincs szükség finomabb vezérlésre.

Az események használhatók a parancsok közötti függőségek kifejezésére is. Ha például egy kernel csak akkor indulhat, ha egy másik kernel befejeződött, akkor a második kernel `clEnqueueNDRangeKernel` hívásában a `event_wait_list` paraméterrel megadhatjuk az első kernel eseményét. Ez biztosítja, hogy a második kernel csak az első befejezése után kerüljön végrehajtásra, anélkül, hogy a hosztnak explicit várakoznia kellene.

Az események callback mechanizmust is támogatnak. A `clSetEventCallback` függvénnyel regisztrálhatunk egy visszahívó függvényt, amely akkor fut le, amikor az esemény eléri a megadott állapotot. Ez lehetővé teszi eseményvezérelt hosztprogramok írását.

A teljesítménymméréshez az események különösen hasznosak. Ha a queue-t a `CL_QUEUE_PROFILING_ENABLE` zászlóval hozzuk létre, akkor a `clGetEventProfilingInfo` függvénnyel lekérdezhetjük a parancs időbélyegeit:

- `CL_PROFILING_COMMAND_QUEUED` – a parancs queue-ba kerülésének időpontja.
- `CL_PROFILING_COMMAND_SUBMITTED` – a parancs elküldésének időpontja.
- `CL_PROFILING_COMMAND_START` – a parancs végrehajtásának kezdete.
- `CL_PROFILING_COMMAND_END` – a parancs végrehajtásának vége.

A kernel futási ideje a `CL_PROFILING_COMMAND_END` és `CL_PROFILING_COMMAND_START` különbsége nanomásodpercben. Ez a módszer pontosabb, mint a hoszt oldali időmérés, mert nem tartalmazza a queue-ba állás és a szinkronizáció idejét.

Példa a profilkérésre:

```
cl_event kernel_event;
clEnqueueNDRangeKernel(queue, kernel, 1, NULL, &global_size, &local_size, 0, NULL,
↪  &kernel_event);
clWaitForEvents(1, &kernel_event);

cl_ulong start, end;
clGetEventProfilingInfo(kernel_event, CL_PROFILING_COMMAND_START, sizeof(cl_ulong), &start,
↪  NULL);
clGetEventProfilingInfo(kernel_event, CL_PROFILING_COMMAND_END, sizeof(cl_ulong), &end, NULL);
printf("kernel time: %llu ns\n", (unsigned long long)(end - start));
```

Az események fontos szerepet játszanak a memóriaobjektumok olvasásánál is. A `clEnqueueReadBuffer` függvény szintén aszinkron, és a hoszt csak akkor olvashatja a puffert, ha a másolás befejeződött. Ezt a `clFinish` hívással vagy a `clEnqueueReadBuffer` eseményére való várakozással biztosíthatjuk.

29.3.4 4. Labor: OpenCL vektorösszeadó kernel

A labor célja egy teljes OpenCL program létrehozása, amely két float vektort ad össze a GPU-n, majd ellenőrzi az eredményt. A program a következő lépéseket hajtja végre:

1. Platform és device kiválasztása.
2. Context és queue létrehozása.
3. Pufferek allokálása és feltöltése.
4. Kernel forrásának betöltése és lefordítása.

5. Kernel argumentumok beállítása.
6. Kernel indítása a megfelelő global és local work size-szal.
7. Eredmény visszaolvasása.
8. Ellenőrzés és mérési napló.

A kernel forrása:

```
// vector_add.cl - OpenCL C 1.2
__kernel void vector_add(__global const float* a,
                         __global const float* b,
                         __global float* c,
                         const int n) {
    const int i = get_global_id(0);
    if (i < n) {
        c[i] = a[i] + b[i];
    }
}
```

A hosztprogram C++17-ben, OpenCL 3.0 API-val:

```
// main.cpp - C++17, g++ 12, OpenCL 3.0
#include <CL/cl.h>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <vector>

#define CL_CHECK(expr)                                                  \
    do {                                                                \
        cl_int err = (expr);                                            \
        if (err != CL_SUCCESS) {                                        \
            fprintf(stderr, "OpenCL hiba: %s:%d: %d\n", __FILE__, __LINE__,  \
                    err);                                               \
            exit(EXIT_FAILURE);                                         \
        }                                                               \
    } while (0)

int main() {
    const int n = 1 << 20; // 1 048 576 elem
    const size_t local_size = 256;
    const size_t global_size = ((n + local_size - 1) / local_size) * local_size;

    // 1. Platform és device
    cl_platform_id platform;
    cl_device_id device;
    CL_CHECK(clGetPlatformIDs(1, &platform, NULL));
    CL_CHECK(clGetDeviceIDs(platform, CL_DEVICE_TYPE_GPU, 1, &device, NULL));

    // 2. Context és queue
    cl_context context = clCreateContext(NULL, 1, &device, NULL, NULL, NULL);
    if (!context) { fprintf(stderr, "Context hiba\n"); return EXIT_FAILURE; }
    cl_command_queue queue = clCreateCommandQueue(context, device, CL_QUEUE_PROFILING_ENABLE,
↪   NULL);
    if (!queue) { fprintf(stderr, "Queue hiba\n"); return EXIT_FAILURE; }

    // 3. Pufferek
    std::vector<float> a(n), b(n), c(n);
    for (int i = 0; i < n; ++i) {
        a[i] = static_cast<float>(i) * 0.1f;
        b[i] = static_cast<float>(i) * 0.2f;
    }
    cl_mem d_a = clCreateBuffer(context, CL_MEM_READ_ONLY | CL_MEM_COPY_HOST_PTR,
                                n * sizeof(float), a.data(), NULL);
```

```
    cl_mem d_b = clCreateBuffer(context, CL_MEM_READ_ONLY | CL_MEM_COPY_HOST_PTR,
                                n * sizeof(float), b.data(), NULL);
    cl_mem d_c = clCreateBuffer(context, CL_MEM_WRITE_ONLY,
                                n * sizeof(float), NULL, NULL);
    if (!d_a || !d_b || !d_c) { fprintf(stderr, "Buffer hiba\n"); return EXIT_FAILURE; }

    // 4. Kernel fordítás
    const char* source = R"(
        __kernel void vector_add(__global const float* a,
                                 __global const float* b,
                                 __global float* c,
                                 const int n) {
            const int i = get_global_id(0);
            if (i < n) {
                c[i] = a[i] + b[i];
            }
        }
    )";
    cl_program program = clCreateProgramWithSource(context, 1, &source, NULL, NULL);
    if (!program) { fprintf(stderr, "Program hiba\n"); return EXIT_FAILURE; }
    cl_int build_err = clBuildProgram(program, 1, &device, "", NULL, NULL);
    if (build_err != CL_SUCCESS) {
        char log[8192];
        clGetProgramBuildInfo(program, device, CL_PROGRAM_BUILD_LOG, sizeof(log), log, NULL);
        fprintf(stderr, "Build hiba:\n%s\n", log);
        return EXIT_FAILURE;
    }
    cl_kernel kernel = clCreateKernel(program, "vector_add", NULL);
    if (!kernel) { fprintf(stderr, "Kernel hiba\n"); return EXIT_FAILURE; }

    // 5. Argumentumok
    CL_CHECK(clSetKernelArg(kernel, 0, sizeof(cl_mem), &d_a));
    CL_CHECK(clSetKernelArg(kernel, 1, sizeof(cl_mem), &d_b));
    CL_CHECK(clSetKernelArg(kernel, 2, sizeof(cl_mem), &d_c));
    CL_CHECK(clSetKernelArg(kernel, 3, sizeof(int), &n));

    // 6. Kernel indítás
    cl_event kernel_event;
    CL_CHECK(clEnqueueNDRangeKernel(queue, kernel, 1, NULL, &global_size,
                                    &local_size, 0, NULL, &kernel_event));

    // 7. Eredmény visszaolvasása
    cl_event read_event;
    CL_CHECK(clEnqueueReadBuffer(queue, d_c, CL_TRUE, 0, n * sizeof(float),
                                 c.data(), 1, &kernel_event, &read_event));
    CL_CHECK(clWaitForEvents(1, &read_event));

    // 8. Ellenőrzés
    bool ok = true;
    for (int i = 0; i < n; ++i) {
        const float expected = a[i] + b[i];
        if (std::fabs(c[i] - expected) > 1e-5f) {
            ok = false;
            fprintf(stderr, "Eltérés az indexen %d: várt %.6f, kapott %.6f\n",
                    i, expected, c[i]);
            break;
        }
    }
    printf("Ellenőrzés: %s\n", ok ? "sikeres" : "sikertelen");

    // Profil
    cl_ulong start, end;
    clGetEventProfilingInfo(kernel_event, CL_PROFILING_COMMAND_START, sizeof(cl_ulong), &start,
↪   NULL);
```

```
    clGetEventProfilingInfo(kernel_event, CL_PROFILING_COMMAND_END, sizeof(cl_ulong), &end,
↪   NULL);
    printf("Kernel futási idő: %.3f ms\n", (end - start) * 1e-6);

    // Takarítás
    clReleaseEvent(kernel_event);
    clReleaseEvent(read_event);
    clReleaseKernel(kernel);
    clReleaseProgram(program);
    clReleaseMemObject(d_a);
    clReleaseMemObject(d_b);
    clReleaseMemObject(d_c);
    clReleaseCommandQueue(queue);
    clReleaseContext(context);
    return ok ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

A fordítás és futtatás Linux környezetben, OpenCL ICD-vel:

```
g++ -O2 -std=c++17 main.cpp -lOpenCL -o vector_add
./vector_add
```

A fenti kódban a `global_size` kiszámítása `((n + local_size - 1) / local_size) * local_size` módon történik, így a global work size osztható `local_size`-szal, és a kernelben a `if (i < n)` határellenőrzés kezeli a felesleges work-itemeket. A `clEnqueueReadBuffer` hívásban a `blocking` paraméter `CL_TRUE`, így a hoszt várakozik a másolás befejezésére, de a `kernel_event` megadása biztosítja, hogy a másolás csak a kernel befejezése után kezdődjön. A `read_event`-re való várakozás redundáns, de bemutatja az események használatát.

Mérési napló. A mérés során a következő adatokat rögzítjük:

Paraméter	Érték
Operációs rendszer	Ubuntu 22.04 (példa)
GPU modell	NVIDIA T4 (példa)
OpenCL driver	NVIDIA OpenCL 3.0, CUDA 12.4 (példa)
OpenCL verzió	3.0
Kernel	vector_add
n	1 048 576 float
local_work_size	256
global_work_size	1 048 576
Ismétlések	10
Várt eredmény	a[i] + b[i], max abszolút eltérés $<$ 1e-5
Mért eredmény	eltérés max 0.0, ellenőrzés sikeres
Kernel futási idő (átlag)	0.083 ms (példa)
Szórás	0.004 ms (példa)

A mérési naplóban a példaértékek illusztrációk; a tényleges mérést a saját környezetben kell elvégezni. A várt és mért eredményt külön oszlopban tüntetjük fel, és az ellenőrzés során a CPU-n számított referenciaértékhez képesti eltérést vizsgáljuk.

Összehasonlító CUDA és JAX példák. A teljesség kedvéért a vektorösszeadás CUDA C++ és JAX megfelelője is megadható, de ebben a fejezetben csak illusztrációként, futtatás nélkül szerepelnek:

```
// CUDA C++17, nvcc 12.4, nem futtatott illusztráció
__global__ void vector_add_cuda(const float* a, const float* b, float* c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) c[i] = a[i] + b[i];
}
```

```
# JAX, jax 0.4, nem futtatott illusztráció
import jax.numpy as jnp
a = jnp.arange(n, dtype=jnp.float32) * 0.1
b = jnp.arange(n, dtype=jnp.float32) * 0.2
c = a + b
```

A JAX esetén a kernelindítás és a memóriakezelés rejtve marad; a kifejezés a JIT-fordítás után a TPU/GPU-ra kerül. A részletes JAX és TPU kezelés a gpu106–gpu109 alfejezetekben található.

29.3.5 5. Tipikus hibák és megoldások

1. **CL_INVALID_WORK_GROUP_SIZE.** A global work size nem osztható a local work size-szal, vagy a local work size meghaladja a device korlátját. Megoldás: kerekítsük fel a global work size-ot, és a kernelben végezzünk határellenőrzést; ellenőrizzük a CL_DEVICE_MAX_WORK_GROUP_SIZE és CL_DEVICE_MAX_WORK_ITEM_SIZES értékeket.
2. **Helytelen kernel argumentumok.** A kernel argumentumok sorrendje vagy típusa nem egyezik a clSetKernelArg hívásokkal. Ez gyakran CL_INVALID_ARG_SIZE vagy CL_INVALID_ARG_INDEX hibát okoz. Megoldás: a kernel szignatúráját és a hoszt hívásait együtt ellenőrizzük, a clSetKernelArg minden hívásánál a CL_CHECK makrót használjuk.
3. **Build hiba figyelmen kívül hagyása.** A clBuildProgram visszatérési értékét ellenőrizni kell, és hiba esetén a build logot ki kell írni. A build log kiíratása nélkül a hibás kernel forrás rejtve marad.
4. **Nem várt aszinkron végrehajtás.** A hoszt a clEnqueueNDRangeKernel után azonnal olvassa a puffert, de a kernel még nem fejeződött be. Megoldás: a clEnqueueReadBuffer hívásban adjuk meg a kernel eseményét, vagy hívjunk clFinish-t az olvasás előtt.
5. **Memóriaobjektumok felszabadításának elmulasztása.** A clCreateBuffer, clCreateKernel, clCreateProgram stb. hívások által lefoglalt erőforrásokat a megfelelő clRelease* függvényekkel kell felszabadítani, különben erőforrás-szivárgás lép fel.
6. **A work-group méret nem optimális.** A device által támogatott maximális work-group méret nem feltétlenül a leggyorsabb. A teljesítményoptimalizáláshoz a gpu113 és a gpu115 fejezetek adnak útmutatást.

29.3.6 6. Önellenőrző kérdés

Mi a különbség a global_work_size és a local_work_size között, és mi történik, ha a global_work_size nem osztható a local_work_size-szal? Írjon olyan kernelszignatúrát, amely határellenőrzést végez, és indokolja meg, miért van szükség rá ebben az esetben.

29.4 OpenCL és CUDA összehasonlítás

A két modell összehasonlítását nem csak szókincs-szintű megfeleltetésként érdemes elvégezni, hanem a végrehajtási és adatmozgatási filozófia mentén is. A CUDA szorosan integrált, gyártóspecifikus modell, amelyben a kernel fordítását, az eszközkezelést és a memóriaallokációt egyetlen fordítói- és futásidejű réteg fogja össze. Az OpenCL viszont szabványos, platformfüggetlen réteg, amely a hordozhatóság érdekében explicit objektumkezelést és futásidejű kernelefordítást használ. Ha a fogalmakat páronként leképezzük, a két modell közötti átjárás jelentősen leegyszerűsödik, és a teljesítményoptimalizálási ismeretek is átvihetők.

29.4.1 Fogalmi megfeleltetés

A legfontosabb fogalmi párokat az alábbi táblázat foglalja össze. Ez a táblázat egyszerre használható fogalmi térképként, amelyet a fejezet végén található labor részben tovább bővítünk.

CUDA fogalom	OpenCL fogalom	Megjegyzés
GPU device	device	OpenCL-ben egy platform alatt több, akár különböző gyártótól származó eszköz is lehet.
CUDA stream	command queue	A parancsok aszinkron végrehajtási sora. OpenCL-ben in-order és out-of-order sor is van.
thread	work-item	Az egyes végrehajtási egység.
block	work-group	A szinkronizálható, közös memóriával rendelkező egység.
grid	NDRange	A teljes indexteret leíró N dimenziós tartomány.
kernel függvény (**global**)	kernel függvény	OpenCL-ben a kernelhez futásidejű fordítás és cl_kernel objektum tartozik.
global memory	global memory	A device-on lévő, minden thread által elérhető memória.
shared memory	local memory	A work-group közös memóriája.
local változók (register spill)	private memory	A CUDA-ban a függvényen belüli automatikus változók alapból regiszterekben vannak, az OpenCL-ben a private address space a work-item privát memóriája, ami általában regiszter, de túlcsordulás esetén device-memória.

CUDA fogalom	OpenCL fogalom	Megjegyzés
constant memory	constant memory	Mindkettőben read-only, cache-elt címtér.
texture memory	image objects	A CUDA textúra és az OpenCL image típusai hasonló célra szolgálnak, de a kezelésük eltérő.
___syncthreads()	work_group_barrier(CLK_LOCAL_MEM_FENCE)	Mindkettő blokkon belüli szinkronizáció.
___threadfence_block()	mem_fence(CLK_LOCAL_MEM_FENCE)	Helyi memórián belüli rendezés.
___threadfence()	mem_fence(CLK_GLOBAL_MEM_FENCE)	Globális memórián belüli rendezés.
warp	sub-group	A SIMD végrehajtás alapegysége. A CUDA-ban a warp mérete 32, az OpenCL-ben a sub-group mérete eszközfüggő (gyakran 32 vagy 64).
atomicAdd	atomic_add	A műveletek nevei hasonlóak, de az OpenCL-ben a típusokhoz kötött függvénynevek szerepelnek.
cudaDeviceSynchronize()	clFinish(queue)	Várakozás a parancssor kiürülésére.
cudaStreamSynchronize(stream)	clWaitForEvents(1, &event) vagy queue finish	Részleges várakozás egy adott eseményre.
cudaEvent_t	cl_event	Eseményobjektum a parancsvégrehajtás állapotának nyomon követésére.
shared	___local	A memóriatér jelölése a kernelkódban.
global memória (mutató)	___global memória (mutató)	A globális címtér jelölése.
cudaMalloc + cudaMemcpy	clCreateBuffer + clEnqueueWriteBuffer	Allokáció és adatfeltöltés.
cudaFree	clReleaseMemObject	Memória felszabadítása.
cudaStreamCreate	clCreateCommandQueue	Parancssor létrehozása.
cudaEventRecord + cudaEventElapsedTime	clEnqueueMarker + clGetEventProfilingInfo	Időmérés eseményekkel.

A táblázatban szereplő párok többsége közvetlenül megfeleltethető, de a mögöttes munka- és memóriamodellben vannak lényeges különbségek, amelyek a gyakorlati munkában meghatározzák a kód szerkezetét.

29.4.2 Eszköz, kontextus, sor és kernel

A CUDA modell implicit eszközkezelése helyett az OpenCL a platform, device, context és command queue objektumokat külön kezeli. A CUDA-ban a `cudaSetDevice()` kiválasztja az aktív eszközt, majd a `cudaMalloc`, kernelindítás és `cudaMemcpy` mind erre az eszközre vonatkozik. Az OpenCL-ben a platform és device explicit lekérdezése után létre kell hozni egy `cl_context`-et, amely összeköti az eszközt a futásidejű környezettel, és egy `cl_command_queue`-t, amely a parancsokat fogadja. A CUDA streamekhez hasonlóan több sor is létrehozható, és az OpenCL parancsok mindig egy adott sorba kerülnek.

A kernelkezelésben a legnagyobb különbség a fordítás időzítése. A CUDA-ban a kernelt a host programmal együtt fordítjuk le (nvcc), és az indításkor a futásidejű réteg már csak a binárist tölti be. Az OpenCL-ben a kernel forráskódját futásidőben adjuk át (`clCreateProgramWithSource`), majd lefordítjuk (`clBuildProgram`), és a hibákat a build logból olvassuk ki. Ez a futásidejű fordítás nagyobb rugalmasságot ad, de hiba esetén a diagnosztika nehézkesebb, és a kernel betöltése lassabb lehet.

A parancsvégrehajtás modellje hasonló: mindkét API aszinkron módon fogadja a parancsokat, és azok a sorban (stream) kerülnek végrehajtásra. A szinkronizációt a CUDA-ban a `cudaDeviceSynchronize` vagy a stream-szinkronizáció, az OpenCL-ben a `clFinish` vagy az eseményre való várakozás biztosítja. A hibakezelés is párhuzamba állítható: a CUDA a legtöbb hívásnál `cudaError_t`-t ad vissza, az OpenCL pedig `cl_int`-et. A rendszeres ellenőrzés mindkét modellben kötelező, ahogyan azt a könyv kód- és laborszerződése is előírja.

29.4.3 Thread és work-item

A CUDA végrehajtási hierarchia három szintje (grid, block, thread) közvetlenül megfeleltethető az OpenCL NDRange, work-group, work-item hármasának. A CUDA-ban a kernel indításakor a `<<<grid, block>>>` paraméterek dimenzionálják a gridet és a blokkot. Az OpenCL-ben a kernel indítása a `clEnqueueNDRangeKernel` hívással történik, ahol a `global_work_size` és a `local_work_size` paraméterek adják meg az NDRange és a work-group méreteit. Mindkét modell 1D, 2D és 3D indexelést támogat.

A kernel kódon belüli indexszámítások hasonlóak. A CUDA-ban a thread indexei a `threadIdx`, `blockIdx`, `blockDim` és `gridDim` beépített változókban érhetők el. Az OpenCL-ben a megfelelő beépített függvények a `get_global_id()`, `get_local_id()`, `get_local_size()`, `get_group_id()` és `get_num_groups()`. Egy 1D vektorösszeadás esetén a globális index:

CUDA C++:

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
```

OpenCL C:

```
int i = get_global_id(0);
```

Mindkettő ugyanannak a lineáris indexnek a kiszámítását végzi. A `get_global_id` közvetlenül a globális indexet adja, így az OpenCL kernel gyakran tömörebb. A CUDA-ban a többdimenziós indexelésnél a `blockIdx.y * blockDim.y + threadIdx.y` kifejezést kell használni, az OpenCL-ben a `get_global_id(1)` ugyanezt adja.

A work-group és a block közötti megfeleltetés fontos a szinkronizáció és a közös memória szempontjából. Mindkettőben a szinkronizáció csak ezen a szinten lehetséges közvetlenül. A CUDA `__syncthreads()` függvénye a blokk összes threadjét szinkronizálja, az OpenCL

`work_group_barrier(CLK_LOCAL_MEM_FENCE)` hívása a work-group összes work-itemjét. Mindkét esetben a hívásnak minden thread/work-item el kell érnie, különben holtpont vagy meghatározatlan viselkedés lép fel.

A SIMD végrehajtás alapegysége a CUDA-ban a warp (32 thread), az OpenCL-ben a sub-group. Az OpenCL szubcsoportok mérete implementációfüggő; a NVIDIA platformokon általában 32, de ezt futásidőben kell lekérdezni, nem szabad konstansként feltételezni. A CUDA-ban a warp mérete 32, de a warp-szinkron funkciók (például `__syncwarp`) használhatóak a warp-szintű szinkronizációra. Az OpenCL-ben a `sub_group_barrier` vagy a `sub_group_all` függvények biztosítanak hasonló funkciót, de ezek csak akkor elérhetők, ha az eszköz támogatja a szubcsoportokat és a `cl_khr_subgroups` kiterjesztést. Itt fontos megjegyezni, hogy a szubcsoportok mérete és viselkedése eltérhet a CUDA warpjától, ezért a kód átvitelekor ellenőrizni kell a feltételezéseket.

A divergencia feldolgozása is hasonló: a warp vagy sub-group szintjén lévő elágazás mindkét ág végrehajtását eredményezi, ha a feltétel nem egységes. Ez a teljesítmény szempontjából mindkét modellben kerülendő. A GPU-k hardveres ütemezője mindkét modellnél ugyanazt a mechanizmust használja, ezért az optimalizálási ismeretek átvihetők.

29.4.4 Memória

A memóriaterek megfeleltetése az OpenCL address space minősítők és a CUDA memóriatérdeklarációk között történik. A kernelkódban az OpenCL a `__global`, `__local`, `__constant` és `__private` minősítőket használja, míg a CUDA a `__global__` (kernel függvény), `__shared__`, `__constant__` és az alapértelmezett regiszter-lokális változókat. A CUDA-ban a `__global__` minősítő a kernel függvényt jelöli, nem pedig a globális memóriát; a globális memóriába mutató pointerek jelöletlenek. Ez a névütközés gyakori hibaforrás aki a két modellt váltogatja.

A host oldali memóriakezelésben a CUDA a `cudaMalloc`, `cudaMemcpy`, `cudaMemset` és `cudaFree` függvényeket használja. Az OpenCL-ben a memóriaobjektumok a `clCreateBuffer`, `clCreateImage`, `clEnqueueWriteBuffer`, `clEnqueueReadBuffer` és `clReleaseMemObject` hívásokkal kezelhetők. A legfontosabb különbség, hogy az OpenCL-ben a buffer létrehozásakor meg kell adni a hozzáférési jogosultságokat a `cl_mem_flags` paraméterrel, például `CL_MEM_READ_ONLY`, `CL_MEM_WRITE_ONLY` vagy `CL_MEM_READ_WRITE`. Ez a flag segíti a futásidejű optimalizációt, de helytelen megadása hibát okozhat.

A CUDA `cudaMemcpyAsync` és stream-műveleteihez hasonlóan az OpenCL is rendelkezik aszinkron másolással: `clEnqueueWriteBuffer` és `clEnqueueReadBuffer` segítségével, amelyek egy sorba kerülnek, és a végrehajtásuk eseménnyel szinkronizálható. A CUDA unified memory (Managed Memory) funkciójának az OpenCL Shared Virtual Memory (SVM) felel meg, de az SVM implementációja platformfüggő, és a részletes tárgyalása a következő fejezetekre marad.

A memóriaterek kiválasztásának szempontjai is átvihetők: a gyakran olvasott, minden thread által elért adatokat érdemes constant memóriába tenni, a work-groupon belül megosztott adatokat local/shared memóriába, a nagy, globálisan elérhető adatokat global memóriába. A bank conflictok és a coalescing szabályai a két modellben azonosak, mivel ezek a hardver memóriavezérlőjéből adódnak.

29.4.5 Szinkronizáció és parancsvégrehajtás

A munkaegységek közötti szinkronizáció két szinten értelmezhető: a kernelkódon belüli work-group szintű szinkronizáció, és a host oldali, parancssor szintű szinkronizáció.

A kernel kódon belül a CUDA `__syncthreads()` és az OpenCL `work_group_barrier()` ugyanazt a célt szolgálja: biztosítja, hogy a work-group minden tagja elérte a hívást, mielőtt bármelyikük továbbhaladna. A CUDA-ban a barrier a blokkon belüli thread-ekre vonatkozik, és implicit módon a shared memória rendezését is biztosítja. Az OpenCL `work_group_barrier` függvény paramétere határozza meg, hogy mely memóriafajtákra vonatkozik a memóriafence: `CLK_LOCAL_MEM_FENCE` a local memória, `CLK_GLOBAL_MEM_FENCE` a global memória rendezését biztosítja. A kettő kombinálható, de a leggyakoribb használat a `CLK_LOCAL_MEM_FENCE`, amikor a local memóriát írjuk és olvassuk.

A memóriafence önállóan is megjelenhet: a CUDA `__threadfence_block()` a blokkon belüli memória-hozzáférések rendezését biztosítja, az OpenCL `mem_fence(CLK_LOCAL_MEM_FENCE)` ugyanezt teszi. A globális memóriára vonatkozó `__threadfence()` megfelelője a `mem_fence(CLK_GLOBAL_MEM_FE` de az OpenCL-ben a globális memória rendezése a work-groupon kívül csak a parancsok szintjén garantált.

A host oldali szinkronizáció a parancssorok ütemezéséből adódik. A CUDA streamek és az OpenCL command queue-k közös tulajdonsága, hogy a sorba küldött parancsok sorrendben hajtódnak végre, de a host nem várja meg a befejezést, hacsak nem hívunk szinkronizációs függvényt. A CUDA `cudaDeviceSynchronize()` az összes stream befejezését várja, az OpenCL `clFinish(queue)` az adott parancssor kiürülését. A részleges várakozásra a CUDA `cudaStreamSynchronize` és `cudaEventSynchronize`, az OpenCL `clWaitForEvents` és a `cl_event` szolgál.

Az események időmérésre is használhatók. A CUDA `cudaEventRecord` és `cudaEventElapsedTime` páros az OpenCL `clEnqueueMarker` és a `clGetEventProfilingInfo` függvényekkel váltható ki, de az OpenCL esetén a parancssort `CL_QUEUE_PROFILING_ENABLE` flaggel kell létrehozni. A mérési módszertan mindkét modellben azonos: warm-up, ismétlés, átlag és szórás számítása.

29.4.6 Gyakorlati példa: vektorösszeadás

A fogalmi megfeleltetés gyakorlati bemutatására a legegyszerűbb vektorösszeadás kernelét vetjük össze. A CUDA és az OpenCL kernelkódja szinte azonos logikát tükröz, csak a beépített változók és a memóriatér-jelölések különböznek.

29.4.7 CUDA C++ kernel

```
// vector_add_cuda.cu
#include <cstdio>
#include <cstdlib>
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err = call;                                             \
        if (err != cudaSuccess) {                                           \
            std::fprintf(stderr, "CUDA error at %s:%d: %s\n",               \
                         __FILE__, __LINE__, cudaGetErrorString(err));      \
            std::exit(EXIT_FAILURE);                                        \
        }                                                                   \
    } while (0)

__global__ void vectorAdd(const float* a, const float* b, float* c, int n)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        c[i] = a[i] + b[i];
    }
```

```
}

int main()
{
    const int n = 1 << 20; // 1 048 576 elem
    const size_t bytes = n * sizeof(float);

    float *h_a = static_cast<float*>(std::malloc(bytes));
    float *h_b = static_cast<float*>(std::malloc(bytes));
    float *h_c = static_cast<float*>(std::malloc(bytes));
    for (int i = 0; i < n; ++i) {
        h_a[i] = 1.0f;
        h_b[i] = 2.0f;
    }

    float *d_a, *d_b, *d_c;
    CUDA_CHECK(cudaMalloc(&d_a, bytes));
    CUDA_CHECK(cudaMalloc(&d_b, bytes));
    CUDA_CHECK(cudaMalloc(&d_c, bytes));

    CUDA_CHECK(cudaMemcpy(d_a, h_a, bytes, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_b, h_b, bytes, cudaMemcpyHostToDevice));

    const int blockSize = 256;
    const int gridSize = (n + blockSize - 1) / blockSize;

    vectorAdd<<<gridSize, blockSize>>>(d_a, d_b, d_c, n);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    CUDA_CHECK(cudaMemcpy(h_c, d_c, bytes, cudaMemcpyDeviceToHost));

    float maxError = 0.0f;
    for (int i = 0; i < n; ++i) {
        maxError = fmaxf(maxError, fabsf(h_c[i] - (h_a[i] + h_b[i])));
    }
    std::printf("Max error: %f\n", maxError);

    CUDA_CHECK(cudaFree(d_a));
    CUDA_CHECK(cudaFree(d_b));
    CUDA_CHECK(cudaFree(d_c));
    std::free(h_a);
    std::free(h_b);
    std::free(h_c);
    return 0;
}
```

29.4.8 OpenCL C kernel és host oldal

```
// vector_add_kernel.cl
__kernel void vectorAdd(__global const float* a,
                        __global const float* b,
                        __global float* c,
                        const int n)
{
    int i = get_global_id(0);
    if (i < n) {
        c[i] = a[i] + b[i];
    }
}

// vector_add_opencl.c
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <CL/cl.h>

#define CL_CHECK(call)                                                  \
    do {                                                                \
        cl_int err = call;                                              \
        if (err != CL_SUCCESS) {                                        \
            fprintf(stderr, "OpenCL error at %s:%d: %d\n",              \
                    __FILE__, __LINE__, err);                           \
            exit(EXIT_FAILURE);                                         \
        }                                                               \
    } while (0)

int main()
{
    const int n = 1 << 20;
    const size_t bytes = n * sizeof(float);

    cl_platform_id platform;
    cl_device_id device;
    cl_context context;
    cl_command_queue queue;
    cl_program program;
    cl_kernel kernel;
    cl_mem d_a, d_b, d_c;

    // Platform és eszköz lekérdezése (helyőrző: platform_index=0, device_index=0)
    cl_uint num_platforms;
    CL_CHECK(clGetPlatformIDs(1, &platform, &num_platforms));
    CL_CHECK(clGetDeviceIDs(platform, CL_DEVICE_TYPE_GPU, 1, &device, NULL));

    context = clCreateContext(NULL, 1, &device, NULL, NULL, NULL);
    queue = clCreateCommandQueue(context, device, 0, NULL);

    // Kernel forrás beolvasása és build
    const char* kernel_source = "__kernel void vectorAdd(__global const float* a,\n"
                                "                        __global const float* b,\n"
                                "                        __global float* c,\n"
                                "                        const int n) {\n"
                                "    int i = get_global_id(0);\n"
                                "    if (i < n) c[i] = a[i] + b[i];\n"
                                "}\n";
    program = clCreateProgramWithSource(context, 1, &kernel_source, NULL, NULL);
    CL_CHECK(clBuildProgram(program, 1, &device, "", NULL, NULL));

    kernel = clCreateKernel(program, "vectorAdd", NULL);

    float *h_a = malloc(bytes);
    float *h_b = malloc(bytes);
    float *h_c = malloc(bytes);
    for (int i = 0; i < n; ++i) { h_a[i] = 1.0f; h_b[i] = 2.0f; }

    d_a = clCreateBuffer(context, CL_MEM_READ_ONLY | CL_MEM_COPY_HOST_PTR, bytes, h_a, NULL);
    d_b = clCreateBuffer(context, CL_MEM_READ_ONLY | CL_MEM_COPY_HOST_PTR, bytes, h_b, NULL);
    d_c = clCreateBuffer(context, CL_MEM_WRITE_ONLY, bytes, NULL, NULL);

    CL_CHECK(clSetKernelArg(kernel, 0, sizeof(cl_mem), &d_a));
    CL_CHECK(clSetKernelArg(kernel, 1, sizeof(cl_mem), &d_b));
    CL_CHECK(clSetKernelArg(kernel, 2, sizeof(cl_mem), &d_c));
    CL_CHECK(clSetKernelArg(kernel, 3, sizeof(int), &n));

    size_t global_size = n;
```

```
    size_t local_size = 256;
    CL_CHECK(clEnqueueNDRangeKernel(queue, kernel, 1, NULL, &global_size, &local_size, 0, NULL,
↪   NULL));
    CL_CHECK(clFinish(queue));

    CL_CHECK(clEnqueueReadBuffer(queue, d_c, CL_TRUE, 0, bytes, h_c, 0, NULL, NULL));

    float maxError = 0.0f;
    for (int i = 0; i < n; ++i) {
        maxError = fmaxf(maxError, fabsf(h_c[i] - (h_a[i] + h_b[i])));
    }
    printf("Max error: %f\n", maxError);

    clReleaseMemObject(d_a);
    clReleaseMemObject(d_b);
    clReleaseMemObject(d_c);
    clReleaseKernel(kernel);
    clReleaseProgram(program);
    clReleaseCommandQueue(queue);
    clReleaseContext(context);
    free(h_a); free(h_b); free(h_c);
    return 0;
}
```

A példákban mindkét modellben a hibakezelés kötelezően jelen van. A CUDA kódban a `CUDA_CHECK` makró ellenőrzi a hívásokat, az OpenCL kódban a `CL_CHECK` makró. A kernel logika szinte azonos: a globális index kiszámítása, határellenőrzés, majd az összegzés. A különbségek a host oldali felépítésben, a kernel fordítási idejében és a memóriaobjektumok kezelésében vannak.

29.4.9 Tipikus hibák és helyes megoldások

A két modell közötti váltáskor számos gyakori hiba léphet fel. Az alábbiakban a legfontosabbakat soroljuk fel, és megadjuk a helyes megoldást.

1. Hibás indexszámítás. Az OpenCL-ben a `get_global_id()` közvetlenül a globális indexet adja, míg a CUDA-ban a `blockIdx * blockDim + threadIdx` kifejezést kell használni. Ha a CUDA kernelből átmásoljuk a kódot az OpenCL-be, és elfelejtjük az indexszámítást módosítani, akkor a kernel rossz indexet használ. A helyes megoldás: mindig a megfelelő beépített függvényt vagy változót használjuk, és ellenőrizzük a dimenziókat.

2. A work-group barrier elhagyása. A CUDA-ban a `__syncthreads()` hívása után a shared memória tartalma garantáltan látható a blokk minden threadje számára. Az OpenCL-ben a `work_group_barrier` használata szükséges, de a memóriafence paramétert is helyesen kell beállítani. Ha a barrier elmarad, a local memória olvasása nem determinisztikus. A megoldás: minden olyan ponton, ahol a work-group közös memóriát írunk, hívjunk `work_group_barrier(CLK_LOCAL_MEM_FENCE)`.

3. A memóriaobjektum flag-ek helytelen megadása. Az OpenCL-ben a `CL_MEM_READ_ONLY` és `CL_MEM_WRITE_ONLY` flag-ek a kernel szempontjából korlátozzák a hozzáférést. Ha egy buffer olvasható a host oldalon, de a kernelben `__global const` pointerként szerepel, akkor `CL_MEM_READ_ONLY` flaget kell használni. Ha a buffert a kernel írja és a host olvassa, akkor `CL_MEM_WRITE_ONLY`. A flag-ek helytelen kombinációja hibához vagy optimalizálatlan végrehajtáshoz vezet. A megoldás: a kernel pointer típusa és a flag összhangját mindig ellenőrizni kell.

4. A szubcsoport (warp) méret feltételezése. A CUDA-ban a warp mérete 32, és a warp-szintű szinkronizációhoz `__syncwarp` használható. Az OpenCL szubcsoport mérete nem garantáltan 32; a `clGetDeviceInfo CL_DEVICE_SUB_GROUP_SIZES_INTEL` vagy hasonló lekérdezéssel kell meghatározni. Ha a kód hordozható akar lenni, nem szabad konstansként feltételezni. A megoldás: a szubcsoport méretét futásidőben lekérdezni, vagy a kernelben a `get_sub_group_size()` függvényt használni.

5. A hibakezelés hiánya. A CUDA és az OpenCL API-k sok hívást tartalmaznak, és a hibák gyakran csak később jelentkeznek. A könyv szabályzata szerint minden API-hívást ellenőrizni kell. A CUDA `cudaGetLastError` és `cudaDeviceSynchronize`, az OpenCL `clGetError` vagy az egyes hívások visszatérési értéke. A megoldás: makrókat használni, mint a fenti példákban, és a szinkronizációs pontokon is ellenőrizni.

6. A futásidejű fordítás hibáinak figyelmen kívül hagyása. Az OpenCL `clBuildProgram` visszatérési kódja `CL_BUILD_PROGRAM_FAILURE` lehet, de a forrás hibaüzenet a build logban található. A CUDA-ban a fordítási hibák a host kód fordításakor derülnek ki. Az OpenCL esetén a build log kinyerése `clGetProgramBuildInfo` segítségével lehetséges, és ezt a hibakezelésben kötelező megtenni. A megoldás: a build után mindig ellenőrizni és szükség esetén kilogolni a `CL_PROGRAM_BUILD_LOG`-ot.

29.4.10 Labor: fogalmi térkép készítése

A fejezet laborjának célja egy fogalmi térkép elkészítése, amely a CUDA és az OpenCL modell elemeit rendszerezi, és egy kis, ellenőrizhető példán való alkalmazást mutat be. A labor két részből áll: a fogalmi térkép dokumentálásából és a vektorösszeadás méréséből.

29.4.11 Fogalmi térkép

Készíts egy táblázatot vagy diagramot, amely a CUDA és az OpenCL fogalmait páronként tartalmazza. A térkép tartalmazza a fenti táblázatban felsorolt párokat, de bővítse ki a következőkkel is:

- A végrehajtási modell elemei: stream/queue, event/event, grid/NDRange, block/work-group, thread/work-item.
- A memóriaterek: global, shared/local, constant, texture/image.
- A szinkronizációs funkciók: barrier, fence, atomic.
- A host oldali memória- és parancskezelő API függvények.
- A kernel fordítási modell: statikus (nvcc) vs. futásidejű (clBuildProgram).

A fogalmi térképet készítheted rajzban vagy táblázatban, de minden elemet pontosan definiálni kell. A táblázat használata ajánlott, mert könnyen ellenőrizhető.

29.4.12 Kód és mérés

A laborban a fenti vektorösszeadás kódokat kell lefordítani és futtatni az izolált környezetben. A buildparancsok:

CUDA (feltételezve NVIDIA GPU, CUDA 12.x, Compute Capability 8.6):

```
nvcc -arch=sm_86 -O3 vector_add_cuda.cu -o vector_add_cuda
```

OpenCL (feltételezve az OpenCL SDK elérhető, például NVIDIA OpenCL 3.0):

```
gcc -O3 vector_add_opencl.c -lOpenCL -o vector_add_opencl
```

Futtatási parancs:

```
./vector_add_cuda
./vector_add_opencl
```

A mérési naplóhoz rögzíteni kell:

- GPU modell: például NVIDIA RTX A4000 (ellenőrizd a saját környezetedben)
- Driver verzió: például 535.x (ellenőrizd)
- CUDA verzió: például 12.2
- OpenCL verzió: például 3.0 (a driver által támogatott)
- Operációs rendszer: például Ubuntu 22.04 LTS
- Kernel indítási paraméterek: block/work-group méret = 256, grid/NDRange méret = 4096 (1 048 576 elemhez)
- Lebegőpontos precision: FP32
- Ismétlések száma: legalább 5 futás a kernelre, warm-up után
- Mért eredmények: futásidő átlag és szórás, sávszélesség, hibaszint (max error)
- Várt eredmény: a vektorösszeadás hibája 0, a futásidő és sávszélesség nagyságrendje összehasonlítható a két modellben

A mért és várt eredményeket külön kell feltüntetni. A labor jegyzőkönyvhöz csatolni kell a fogalmi térkép dokumentumát és a kódokat.

29.4.13 Önellenőrző kérdés

Melyik OpenCL fogalom felel meg a CUDA thread-nek, és hogyan számítható ki a globális index a két modellben? Válaszodban adj példát egy 1D vektor kernelre.

29.4.14 Összefoglalás

A CUDA és az OpenCL programozási modellje mélyen hasonló: mindkettő hierarchikus párhuzamosságot, explicit memóriatereket és aszinkron parancsvégrehajtást használ. A legfontosabb különbségek a platformkezelés, a kernel fordítási ideje és a memóriaobjektumok explicit tulajdonságai. A fogalmi megfeleltetés elsajátításával a fejlesztő könnyen válthat a két modell között, és a megszerzett optimalizálási ismeretek is átvihetők. A következő fejezet (OpenCL C és kernelírás) az OpenCL kernelrészleteire, a memóriakezelésre és a hordozhatósági kérdésekre épít rá.

30 OpenCL kernelek, memória és hordozhatóság

30.1 OpenCL C és kernelírás

30.1.1 Az OpenCL C nyelv és a kernel

Az OpenCL C a C99 szabvány származéka, amelyet kifejezetten párhuzamos eszközökön futó kernelek írására terveztek. A nyelv felületesen ismerősnek tűnik a C-fejlesztő számára, azonban több olyan kiterjesztést és korlátozást tartalmaz, amelyek nélkülözhetetlenek a GPU-n való helyes és hatékony végrehajtáshoz. Ebben a fejezetben az OpenCL C nyelv alapvető elemeit, a memóriatér-minősítőket, a vektortípusokat és a fordítási opciókat tárgyaljuk, majd egy teljes, ellenőrzött példán keresztül mutatjuk be a kernelírás gyakorlatát.

30.1.2 A nyelv helye az OpenCL rendszerben

Az OpenCL programozási modelljében a kernel az a függvény, amelyet a host az eszközön párhuzamosan végrehajt. A kernel függvényt a `__kernel` vagy az azzal egyenértékű `kernel` kulcsszóval jelöljük. A kernel a C99-hez képest a következő alapvető eltérésekkel rendelkezik:

1. A kernel nem adhat vissza értéket; a visszatérési típus mindig `void`.
2. A kernel argumentumai nem lehetnek mutatók a host-memóriába; minden mutató argumentumnak egy OpenCL-memóriaobjektumra vagy SVM-területre kell mutatnia.
3. A kernel nem használhatja a C szabványos könyvtárának nagy részét; az OpenCL beépített függvénykészletet biztosít a matematikai és közös műveletekhez.
4. A kernel végrehajtási modellje eltér a szekvenciális C-től: minden work-item ugyanazt a kódot futtatja, de a beépített azonosítók (`get_global_id`, `get_local_id` stb.) különböztetik meg az adatokat.

A kernel forráskódját futásidőben a host program fordítja le az eszközre. Ez a futásidejű fordítás egyszerre jelent rugalmasságot és teljesítménykockázatot: az OpenCL C kód natív módon az adott hardverre fordítható, de a fordítási idő a futásidőt növeli, és a hibák csak a futás során derülnek ki.

30.1.3 Address space minősítők

Az OpenCL C egyik legfontosabb sajátossága, hogy minden mutatónak explicit memóriatér-minősítővel kell rendelkeznie. A négy alapvető memóriatér a következő:

1. **`__global` (vagy `global`)**: A global memory a teljes eszközön elérhető, a host által allokált memória. A globale memória a legnagyobb kapacitású, de a leglassabb hozzáférésű. A kernel argumentumai jellemzően global memóriatérben vannak.

2. **`__constant` (vagy `constant`)**: A constant memory a global memory egy részét foglalja el, de optimalizált olvasási útvonallal rendelkezik. A work-group minden tagja számára azonos értékű adatok (pl. együtthatók, súlyok, konfigurációs paraméterek) tárolására használható. A host az OpenCL API-n keresztül írhatja, a kernel csak olvashatja.

3. **`__local` (vagy `local`)**: A local memory a work-group szintű memória. A work-group minden work-iteme osztozik rajta, és a `barrier()` utasítással szinkronizálhatják a hozzáférést. A local memory a globalnál sokkal gyorsabb, de a kapacitása korlátozott. A host nem fér hozzá közvetlenül; a kernel futása közben használható munkaterületként.

4. **`__private` (vagy `private`)**: A private memory minden work-item saját, privát memóriája. A kernelben deklarált, nem kernel-argumentumként szereplő változók alapértelmezésben private memóriában vannak. A private memória gyors, de a regiszterek túlcsordulása esetén a lassabb global memóriába kerülhet át (register spilling hasonló jelenség, mint CUDA-ban).

A kernel argumentumok deklarációjában a mutatókat mindig minősíteni kell a megfelelő memóriatérrel. Például:

```
__kernel void axpy(
    __global const float *x,
    __global const float *y,
    __global float *z,
    const float a)
{
    int i = get_global_id(0);
    z[i] = a * x[i] + y[i];
}
```

Itt az `x`, `y`, `z` mutatók global memóriában vannak, az `a` skalár pedig private memóriában. A `const` minősítő a read-only globális mutatókra alkalmazható, ami lehetővé teszi a gyorsabb, cachelt elérést egyes architektúrákon.

Fontos: A memóriatér-minősítő nélküli mutató egy kernel argumentumban fordítási hibát okoz. Ez az explicititás az OpenCL C egyik legnagyobb előnye: a kód egyértelműen dokumentálja az adatok helyét és hozzáférési mintáját.

30.1.4 Vektortípusok

Az OpenCL C gazdag vektortípus-készlettel rendelkezik, amely kritikus a GPU-k nagy áteresztőképességű SIMD-egységeinek hatékony kihasználásához. A vektortípusok a skaláris típusokból és egy szélességből állnak, például:

- `float4`: négy 32 bites lebegőpontos szám,
- `int8`: nyolc 32 bites egész,
- `double2`: két 64 bites lebegőpontos szám,
- `uchar16`: tizenhat 8 bites előjel nélküli egész.

A vektortípusok mérete a hardver által támogatott szélességtől függően változhat. A szabvány legalább a 2-, 4-, 8- és 16-elemű vektorokat támogatja; az elérhető szélességeket a `CL_DEVICE_PREFERRED_VECTOR_WIDTH_*` eszközkéréssel lehet lekérdezni.

A vektorok inicializálása több módon történhet:

```
float4 v = (float4)(1.0f, 2.0f, 3.0f, 4.0f);  // Literál
float4 w = (float4)(1.0f);                    // Mind a négy elem 1.0
float4 u = (float4)(v.x, v.y, w.z, 0.0f);      // Elemek kiválasztása
```

A vektorok komponensei a `.x`, `.y`, `.z`, `.w` mezőkkel érhetők el. A `.s0`, `.s1`, `.s2`, `.s3` elnevezés is használható, ami hasznos a 4-nél nagyobb szélességű vektoroknál. A swizzle művelet lehetővé teszi komponensek átrendezését:

```
float4 v = (float4)(1.0f, 2.0f, 3.0f, 4.0f);
float4 swapped = v.wzyx;  // (4.0, 3.0, 2.0, 1.0)
```

A vektorokon az aritmetikai és logikai műveletek komponensenként értelmezettek:

```
float4 a = (float4)(1.0f, 2.0f, 3.0f, 4.0f);
float4 b = (float4)(5.0f, 6.0f, 7.0f, 8.0f);
float4 c = a + b;  // (6.0, 8.0, 10.0, 12.0)
```

Az OpenCL beépített függvényei is vektorosítottak: a `sqrt(c)` komponensenként számítja a négyzetgyököt. A vektortípusok használatával a kód tömörebbé válik, és a fordító könnyebben generál hatékony SIMD-utasításokat.

Tömbök és vektorok kapcsolata: A globális memóriában lévő tömbökből történő vektoros betöltéshez a mutatót vektor típusúvá kell alakítani:

```
__global float4 *vec_ptr = (__global float4 *)a;
float4 v = vec_ptr[i];
```

Ez feltételezi, hogy az `a` tömb 16 bájtra igazított, és a hossza osztható 4-gyel. Az igazítás hiánya nem definiált viselkedést okozhat, ezért ilyen esetekben a `vload4` és `vstore4` függvényeket érdemes használni, amelyek nem igazított hozzáférést is támogatnak.

30.1.5 Gyakorlati példa: vektorösszeadás OpenCL C-ben

Tegyük fel, hogy az OpenCL és CUDA összehasonlítás fejezetben bemutatott platform- és eszközkezelés már rendelkezésre áll. A kernel forráskódja a következő:

```
__kernel void vector_add(
    __global const float *a,
    __global const float *b,
    __global float *c,
    const int n)
{
    int i = get_global_id(0);
    if (i < n) {
        c[i] = a[i] + b[i];
    }
}
```

Ez a kernel a legegyszerűbb adatpárhuzamos mintát valósítja meg: minden work-item egyetlen elem összegét számítja. A `get_global_id(0)` a work-item globális indexét adja vissza a 0. dimenzióban. A feltételes utasítás a tömb végének kezelésére szolgál: ha a globális méret nem osztható a tömb hosszával, a felesleges work-itemek nem hajtanak végre memóriahozzáférést.

Fontos megfigyelés: A kernelben nincs explicit ciklus. A párhuzamosítás implicit: a host a kernel indításakor megadja a globális méretet, és az OpenCL futásidő gondoskodik a work-itemek ütemezéséről. Ez a programozási modell lényege.

30.1.6 Életciklus: host-oldali fordítás és build

A kernel forráskódját a host programban egy stringként tároljuk, majd futásidőben lefordítjuk. A tipikus lépések:

1. **Program objektum létrehozása:**

   ```
   cl_program program = clCreateProgramWithSource(
       context, 1, (const char **)&source_str, NULL, &err);
   ```

2. **Build futtatása:**

   ```
   err = clBuildProgram(program, 1, &device, "", NULL, NULL);
   ```

   Az üres string a build option-öket jelöli; ebben az esetben nincsenek.

3. **Kernel objektum kinyerése:**

   ```
   cl_kernel kernel = clCreateKernel(program, "vector_add", &err);
   ```

4. **Argumentumok beállítása:**

   ```
   clSetKernelArg(kernel, 0, sizeof(cl_mem), (void *)&a_dev);
   clSetKernelArg(kernel, 1, sizeof(cl_mem), (void *)&b_dev);
   clSetKernelArg(kernel, 2, sizeof(cl_mem), (void *)&c_dev);
   clSetKernelArg(kernel, 3, sizeof(int), (void *)&n);
   ```

5. **Kernel indítása:**

   ```
   size_t global_size = n;
   err = clEnqueueNDRangeKernel(
       queue, kernel, 1, NULL, &global_size, NULL, 0, NULL, NULL);
   ```

A build opciók lehetővé teszik a fordítás finomhangolását. A leggyakoribb opciók:

- `-cl-opt-disable`: letiltja az optimalizációt, ami hibakeresésnél hasznos.
- `-cl-mad-enable`: engedélyezi a multiply-add utasítások agresszív összevonását.
- `-cl-no-signed-zeros`: engedélyezi, hogy a fordító figyelmen kívül hagyja az előjel nélküli nulla különbséget.
- `-cl-fast-relaxed-math`: a lebegőpontos szabványok enyhítése, ami gyorsabb, de pontatlanabb kódot eredményez.
- `-D` makrók definiálására: `-DUSE_DOUBLE`.

Hibakezelés build közben: A `clBuildProgram` visszatérési értéke `CL_BUILD_PROGRAM_FAILURE` esetén a build napló tartalmazza a fordítási hibákat. A napló lekérdezése:

```
size_t log_size;
clGetProgramBuildInfo(program, device, CL_PROGRAM_BUILD_LOG,
                      0, NULL, &log_size);
char *log = (char *)malloc(log_size);
clGetProgramBuildInfo(program, device, CL_PROGRAM_BUILD_LOG,
                      log_size, log, NULL);
printf("Build log:\n%s\n", log);
```

30.1.7 Tipikus hibák és elkerülésük

A gyakorlatban a kezdő OpenCL-fejlesztők leggyakoribb hibái a következő kategóriákba sorolhatók:

1. Hiányzó address space minősítő a kernel-argumentumokban:

Hibás:

```
__kernel void bad(float *a) { ... }
```

Helyes:

```
__kernel void good(__global float *a) { ... }
```

A fordító hibát jelez, mert a kernel-argumentum-mutatónak explicite meg kell adni a memóriateret. Ez a leggyakoribb hiba a C99-ből érkező fejlesztőknél.

2. Hibás indexelés vagy határkezelés hiánya:

Ha a globális méret nagyobb, mint a tömb hossza, a kernel túlindexeli a memóriát, ami nem definiált viselkedést, akár eszközleállást is okozhat. A feltételes utasítás vagy a stride-loop használata kötelező.

3. Nem igazított vektoros hozzáférés:

A globális memóriában lévő tömböt `float4` típusú mutatóra alakítani csak akkor szabad, ha a tömb kezdőcíme 16 bájtra igazított. Ellenkező esetben `vload4`-et kell használni:

```
float4 v = vload4(i, a);  // Biztonságos, nem igazított hozzáférés
vstore4(v, i, c);          // Biztonságos tárolás
```

4. A beépített függvények helytelen használata:

Az OpenCL beépített függvényei a C szabványos könyvtár helyett állnak. A `sqrt`, `sin`, `fabs` stb. elérhetők, de a `printf` nem használható kernelben (kivéve bizonyos kiterjesztéssel). A stringkezelő függvények szintén hiányoznak.

5. Lebegőpontos pontossággal kapcsolatos téves feltevések:

Az OpenCL C a C99-hez hasonló lebegőpontos szemantikát követ, de a gyors matematikai opciók (`-cl-fast-relaxed-math`) enyhítik a szabványt. A `cl_fma` függvény a fused multiply-add műveletet garantálja, ami pontosabb, mint a szekvenciális szorzás-összeadás. Ha a numerikus pontosság kritikus, kerüljük a gyors matematikai opciókat.

30.1.8 A kernel és a work-item kapcsolata

A kernelben a következő beépített függvények adják meg a work-item azonosítóit:

- `get_global_id(dim)`: a work-item globális azonosítója a `dim` dimenzióban. Ez a dimenzió a kernel indításakor megadott globális méret szerint alakul.
- `get_local_id(dim)`: a work-item azonosítója a work-groupon belül.
- `get_group_id(dim)`: a work-group azonosítója.
- `get_local_size(dim)`: a work-group mérete a `dim` dimenzióban.
- `get_global_size(dim)`: a teljes globális méret a `dim` dimenzióban.
- `get_num_groups(dim)`: a work-groupok száma a `dim` dimenzióban.

Ezek a beépített függvények a `__kernel` függvényben mindenhol elérhetők, de a nem kernel függvényekben (pl. helper függvények) nem garantált a viselkedésük, ezért kerüljük a használatukat azokban.

A work-groupok és a work-itemek hierarchiája az OpenCL C-ben szorosan kapcsolódik a memóriaterekhez: a `__local` memória a work-group szintű, a `__private` minden work-itemhez kötött. Ez a hierarchia a párhuzamos algoritmusok tervezésének alapja.

30.1.9 Az OpenCL C és a CUDA C++ összehasonlítása

Bár mindkét nyelv SIMT-modellt használ, lényeges különbségek vannak:

Szempont	OpenCL C	CUDA C++
Kernel jelölése	`__kernel`	`__global__`
Memóriatér-minősítők	`__global`, `__local`, `__constant`, `__private`	`__device__`, `__shared__`, `__constant__` (a `__global__` a kernel jelölésére szolgál)
Work-item azonosítók	`get_global_id(0)`	`blockIdx.x * blockDim.x + threadIdx.x` vagy `threadIdx.x`
Szinkronizáció	`barrier(CLK_LOCAL_MEM_FENCE)`	`__syncthreads()`
Vektortípusok	`float4`, `int8` stb.	`float4`, `int4` stb. (CUDA vektortípusok)
Fordítás	Futásidejű (host API-n keresztül)	Fordítási időben (nvcc)
Atomics	`atomic_add`, `atomic_cmpxchg` stb.	`atomicAdd`, `atomicCAS` stb.
Névterek	Nincs (C99)	C++ névterek, template-ek

Az OpenCL C explicit memóriatér-minősítői egyértelműbbé teszik az adatok helyét, míg a CUDA C++ a globális memóriát alapértelmezettként kezeli. A CUDA C++ gazdagabb nyelvi kifejezőerővel rendelkezik (template-ek, osztályok), ami nagyobb rugalmasságot ad a generikus kód írásához. Az OpenCL C azonban szélesebb hardvertámogatást kínál, beleértve a GPU-kat, CPU-kat, FPGA-kat és DSP-ket is.

30.1.10 Ellenőrzési szempont a fejezet állításaihoz

A fejezetben bemutatott nyelvi jellemzők a Khronos OpenCL C specifikációján alapulnak, amely a 3.0 verzióig bezárólag stabil. Az OpenCL C specifikáció pontos szövege a `registry.khronos.org/OpenCL/` címen érhető el; azonban a specifikáció verziója és az egyes eszközök által támogatott kiterjesztések eltérhetnek. A vektorizált hozzáférés igazítási követelményei a `CL_DEVICE_MEM_BASE_ADDR_ALIGN` eszközlekérdezéssel ellenőrizhetők. A fordító által generált kód hatékonysága az adott implementációtól függ; a fejezetben leírt optimalizációs lehetőségek általános irányelvek, nem garantált teljesítményjavulás minden hardveren.

30.1.11 Labor: Vektorösszeadás OpenCL-ben

Cél: OpenCL kernel implementálása és futtatása vektorösszeadásra, a fordítási folyamat és a hibakezelés gyakorlása.

Követelmények:

1. Írj OpenCL C kernelt, amely két float vektor elemenkénti összegét számítja.
2. A host program C++17-ben készül, az OpenCL 1.2+ API-t használja.
3. A kernel forráskódját futásidőben fordítsd le, ellenőrizd a build naplót.
4. A hibaellenőrzés minden OpenCL API-hívást lefed.
5. A futtatás előtt végezz warm-up futtatást, majd 10 mérés.
6. Rögzítsd az eszköz nevét, az OpenCL verziót, a work-group méretét és a globális méretet.
7. A mért és a várt eredményt különítsd el; a numerikus egyezést `fabs(diff) < 1e-5` toleranciával ellenőrizd.

Minta kernel (OpenCL C):

```
__kernel void vector_add(
    __global const float *a,
    __global const float *b,
    __global float *c,
    const int n)
{
    int i = get_global_id(0);
    if (i < n) {
        c[i] = a[i] + b[i];
    }
}
```

Futtatási környezet (helyőrző): Az eszköz egy izolált laborban lévő OpenCL-kompatibilis GPU, pl. egy régebbi AMD vagy Intel GPU. A mérési naplóban szerepelnie kell az eszköz nevének, a driver verziójának és az OpenCL verziójának. **A tényleges futtatás előtt ellenőrizni kell az eszköz elérhetőségét és a szolgáltatói feltételeket.**

Mérési napló minta (nem valós eredmény):

```
Device: [eszköznév_helyőrző]
OpenCL: 3.0
Driver: [driver_verzió_helyőrző]
Input: n = 1 048 576, float
Work-group size: 256
Float precision: FP32

Run #1: 0.42 ms
Run #2: 0.39 ms
...
Run #10: 0.40 ms
Mean: 0.41 ms, median: 0.40 ms, std: 0.02 ms

Várt eredmény: c[i] = a[i] + b[i]
Max abs error: 0.0 (egyszerű összeadás, FP32)
```

Ez a napló minta; a tényleges mérést az adott laboron kell elvégezni. A fejezetben szereplő kódok fordítási és futtatási szintaktikája ellenőrzött, de a hardver-specifikus viselkedés (pl. cache-méret, órajel) helyőrzőként szerepel.

30.1.12 Önellenőrző kérdés

Adott a következő OpenCL C kerneltöredék:

```
__kernel void compute(__global float *a, float b)
{
    int i = get_global_id(0);
    a[i] = a[i] * b;
}
```

Az `a` mutató melyik memóriatérben van? A `b` paraméter melyik memóriatérben van? Mi történik, ha a kernel argumentum deklarációjából elhagyjuk a `__global` minősítőt az `a` mutató elől? Válaszodat indokold az OpenCL C szabályai alapján.

30.2 Work-group, barrier és local memory

Az OpenCL végrehajtási modelljének középpontjában a work-group áll: ez az a szerveződési szint, ahol a work-itemek egymással szinkronizálhatnak, és ahol a local memory megosztott tárolóként jelenik meg. A work-group fogalma analóg a CUDA thread blockjával, de az OpenCL terminológiája és egyes részletszabályai sajátosak.

30.2.1 1. A work-group mint a szinkronizáció alapegysége

Az OpenCL kernel indításakor a host egy N-dimenziós indexteret (NDRange) ad meg. Az NDRange felosztható work-groupökre; minden work-group azonos méretű, és a work-itemeket a csoport indexével azonosítjuk. A work-group méretét a kernel indításakor az `enqueueNDRangeKernel` utolsó argumentumában adjuk meg. A work-itemek a következő beépített függvényekkel kérdezhetik le saját helyüket:

- `get_global_id(dim)` – a work-item globális azonosítója az adott dimenzióban.
- `get_local_id(dim)` – a work-item azonosítója a work-groupön belül.
- `get_group_id(dim)` – a work-group azonosítója.
- `get_local_size(dim)` – a work-group mérete.
- `get_num_groups(dim)` – a work-groupök száma.
- `get_global_size(dim)` – a teljes indexméret.
- `get_work_dim()` – az NDRange dimenzióinak száma.

Ezek a függvények lehetővé teszik, hogy a kernel a globális indexből kiszámítsa a feldolgozandó adatelem helyét. Az 1D-s esetben a következő azonosság érvényes:

```
global_id = group_id * local_size + local_id
```

Többdimenziós esetben minden dimenzióra külön kell elvégezni ezt a számítást. A work-group méretét a `get_local_size` értéke adja; ez a kernel indításakor rögzített, és a kernel belsejében csak lekérdezhető, nem módosítható.

A work-groupök egymástól függetlenül hajtódnak végre: az ütemező bármilyen sorrendben futtathatja őket, akár párhuzamosan, akár sorosan. Ezért a globális szinkronizáció nem megoldható a work-groupök között a kernel belsejében; az OpenCL nem definiál grid-szintű barriert. A work-groupön belüli szinkronizáció viszont a `barrier` függvénnyel lehetséges, és ez az együttműködés alapegysége. A work-groupök közötti adatcsere csak a kernel befejeződése után, a globális memórián keresztül, vagy atomi műveletekkel oldható meg, de ez utóbbi nem garantálja a teljes szinkronizációt.

30.2.2 2. Local memory: a work-group gyors memóriája

A local memory az OpenCL memóriamodelljének az a rétege, amely egy work-groupön belül osztozik. A CUDA shared memory megfelelője, azzal a különbséggel, hogy az OpenCL terminológiájában a `local` a work-group láthatóságú memóriát jelöli, míg a `private` a work-item saját változóit. A local memory a device-on belüli gyors, alacsony késleltetésű tároló, amelynek mérete korlátozott; a tipikus GPU-kon néhány tíz KiB (például 48–96 KiB) áll rendelkezésre work-groupönként. A pontos méretet a `CL_DEVICE_LOCAL_MEM_SIZE` eszközadat adja meg.

A local memory a kernelben a `__local` minősítővel jelölhető. Kétféleképpen allokálható:

1. **Statikus allokáció a kernelben:** a kernel törzsében `__local float tile[256];` alakban. A méretnek fordítási időben ismert konstansnak kell lennie.
2. **Dinamikus allokáció kernelargumentumként:** a kernel egy `__local float*` paramétert kap, és a host a kernel indítása előtt megadja a méretet bájtban. Ez lehetővé teszi, hogy ugyanaz a kernel különböző work-group méretekkel fusson anélkül, hogy újra kellene fordítani.

A következő kernelrészlet dinamikus local memóriát használ:

```
// Fordítás: OpenCL C 1.2+
__kernel void use_local(__global const float* input,
                        __global float* output,
                        __local float* tile) {
    int lid = get_local_id(0);
    int gid = get_global_id(0);
    tile[lid] = input[gid];
    // ... további feldolgozás
}
```

A host oldalon a local memória méretét a `clSetKernelArg` hívással adjuk meg; az argumentum értéke `NULL`, a méret pedig a szükséges bájtok száma:

```
// OpenCL C++ API
clSetKernelArg(kernel, 2, local_bytes, nullptr);
```

A `local_bytes` értéke például `workgroup_size * sizeof(float)`. Fontos, hogy a local memória mérete nem haladhatja meg az eszköz által támogatott maximumot, és a kernel által igényelt local memória mérete befolyásolja az occupancyt: minél nagyobb local memóriát foglal egy work-group, annál kevesebb work-group lehet egyszerre aktív egy compute uniton.

A local memory élettartama a work-group végrehajtásához kötött; amikor a work-group befejeződik, a local memória tartalma elvész. A host közvetlenül nem férhet hozzá; az adatokat a kernelnek kell feltöltenie a globális memóriából, és az eredményt a kernelnek kell visszaírnia.

30.2.3 3. Barrier és memóriakerítés

A `barrier` függvény a work-groupön belüli szinkronizáció eszköze. A hívás azt jelenti, hogy a work-group minden work-itemjének el kell érnie a barriert, mielőtt bármelyikük továbbhaladhat. A barrier opcionális memóriakerítési zászlókat is kaphat:

- `CLK_LOCAL_MEM_FENCE`: a local memóriába írt adatoknak a work-group többi work-itemje számára láthatóvá kell válniuk.
- `CLK_GLOBAL_MEM_FENCE`: a globális memóriába írt adatoknak a work-group többi work-itemje számára kell láthatóvá válniuk.

A memóriakerítés nélkül a work-itemek által végrehajtott írások sorrendje nem garantált; a barrier biztosítja, hogy a barrier előtti írások befejeződjenek, mielőtt a barrier utáni olvasások megkezdődnek.

A legegyszerűbb és leggyakoribb minta a következő:

```
// 1. Minden work-item betölt egy értéket a local memóriába
tile[lid] = input[gid];

// 2. Barrier: minden betöltésinek be kell fejeződnie
barrier(CLK_LOCAL_MEM_FENCE);

// 3. Most már olvasható a local memória
float value = tile[lid + 1];
```

A barrier nélkül az egyik work-item olvashatná a local memóriát, mielőtt a másik work-item befejezte volna az írást, ami nem determinisztikus és hibás eredményhez vezet.

A fa-alapú redukcióban a barriert minden iterációban meg kell hívni:

```
for (int offset = local_size / 2; offset > 0; offset >>= 1) {
    if (lid < offset) {
        tile[lid] += tile[lid + offset];
    }
    barrier(CLK_LOCAL_MEM_FENCE);
}
```

Itt a barrier azért szükséges, mert a következő iterációban a work-itemek a `tile[lid + offset]` értéket olvassák, amelyet az előző iterációban egy másik work-item írhatott. A barrier garantálja, hogy az összes írás befejeződött, mielőtt bármely work-item a következő iterációba lép.

A barrier használatakor a leggyakoribb hiba a **divergens barrier**. Ez akkor fordul elő, ha a barrier hívása feltételes ágban van, és nem minden work-item éri el. Például a következő kód hibás:

```
// HIBÁS: a barrier csak azoknak fut le, akik beléptek az if ágba
if (lid < offset) {
    tile[lid] += tile[lid + offset];
    barrier(CLK_LOCAL_MEM_FENCE); // <-- hibás!
}
```

A fenti esetben a barrier nem garantálja, hogy minden work-item eléri; a work-group viselkedése nem definiált, tipikusan holtpont vagy hibás eredmény. A helyes megoldás az, hogy a barriert a feltételes ágon kívül, minden work-item által végrehajtott helyen hívjuk:

```
if (lid < offset) {
    tile[lid] += tile[lid + offset];
}
barrier(CLK_LOCAL_MEM_FENCE); // <-- helyes
```

Általános szabály, hogy a kernelben a barrier hívásoknak a work-group minden work-itemje által elérhető úton kell lenniük. A fordító nem mindig figyelmeztet a divergens barrierre, ezért a fejlesztő felelőssége a helyes használat.

30.2.4 4. Bank conflict a local memoryben

A local memory a GPU-kon bankokra oszlik, hogy a párhuzamos hozzáférést gyorsítsa. A tipikus kialakítás 32 bank, mindegyik 32 bites szóval. Ha egy hullámfront (warp) work-itemjei egyszerre ugyanabba a bankba, de különböző címekre próbálnak írni vagy olvasni, akkor a hozzáférés sorosítva történik, ami bank conflictet okoz. A bank conflict nem ad helytelen eredményt, de a memória-hozzáférést lassítja, és jelentős teljesítménycsökkenést okozhat.

A bank conflict leggyakoribb forrása a local memóriában tárolt mátrix transzponálása. Tekintsünk egy 32×32-es float mátrixot:

```
__kernel void transpose_naive(__global const float* in,
                              __global float* out,
                              __local float tile[32][32]) {
    int x = get_global_id(0);
    int y = get_global_id(1);
    int lx = get_local_id(0);
    int ly = get_local_id(1);

    tile[ly][lx] = in[y * get_global_size(0) + x];
    barrier(CLK_LOCAL_MEM_FENCE);

    out[x * get_global_size(1) + y] = tile[lx][ly];
}
```

A fenti kernelben a `tile[lx][ly]` olvasás oszlopos hozzáférést jelent. Egy hullámfronton belül a work-itemek az x dimenzió mentén szomszédosak, így `lx` változik, `ly` pedig fix. A local memória címe `(lx * 32 + ly) * 4` bájt. A bank indexét a cím 32 bájtos bankokra osztásával kapjuk: `(lx * 32 + ly) mod 32 = ly mod 32`. Mivel `ly` ugyanaz minden work-itemre a hullámfrontban, mind ugyanabba a bankba próbálnak olvasni, ami 32-szeres bank conflictet okoz.

A megoldás a **padding**: a local memória tömbjét eggyel szélesebbre választjuk, például `tile[32][33]` méretűre. Ekkor a sor hossza 33, és a bank index számítása `(lx * 33 + ly) mod 32 = (lx + ly) mod 32` lesz, ami a hullámfronton belül változik, így a hozzáférés conflictmentessé válik:

```
__kernel void transpose_padded(__global const float* in,
                               __global float* out,
                               __local float tile[32][33]) {
    // ... ugyanaz, mint fent ...
}
```

A padding tehát egyetlen extra oszloppal megszünteti a bank conflictet. Az OpenCL specifikáció nem írja elő a bankok számát, ezért a padding méretét az adott hardverhez kell igazítani; a gyakorlatban a 32 bank és a 32 bites szó a leggyakoribb. A bank conflict fogalma analóg a CUDA shared memory bank conflictjével, de az OpenCL-ben a local memory bankok elrendezése implementációfüggő, ezért a méréseknek mindig hardverspecifikusnak kell lenniük.

30.2.5 5. Tipikus hibák és megelőzés

A work-group, barrier és local memory használata során a következő hibák fordulnak elő leggyakrabban:

1. **Divergens barrier:** a barrier feltételes ágban van, és nem minden work-item hívja meg. Megelőzés: a barrier hívásokat mindig a vezérlési gráf olyan pontjára helyezzük, amelyet minden work-item elér; a feltételes ágakban csak a memóriaműveletek vannak, a barrier utána következik.

2. **Local memory inicializálás elmulasztása:** a local memória a work-group indulásakor nem definiált tartalommal rendelkezik. Ha a kernel nem tölti fel, akkor hibás eredményt adhat. Megelőzés: a local memóriába mindig írunk, mielőtt olvasnánk, és a barrier után olvasunk.
3. **Bank conflict figyelmen kívül hagyása:** a local memória elérési mintája sorosítást okoz, ami lassítja a kernelt. Megelőzés: padding, vagy az elérési minta átalakítása (például soronkénti feldolgozás).
4. **Work-group méret és local memory méret korlátjainak túllépése:** ha a work-group túl nagy, a kernel nem fordul le, vagy az occupancy drasztikusan csökken. Megelőzés: a `CL_DEVICE_MAX_WORK_GROUP_SIZE` és a `CL_DEVICE_LOCAL_MEM_SIZE` eszközadatok lekérdezése; a `clGetKernelWorkGroupInfo` a kernelre vonatkozó korlátokat is megadja.
5. **A globális szinkronizáció téves feltételezése:** a work-groupök közötti függőséget nem lehet barrierrel megoldani. Megelőzés: a kernelhatárokra vagy atomi műveletekre kell támaszkodni, és a tervezés során a work-groupök függetlenségét kell alapul venni.

30.2.6 6. Labor: local memoryt használó OpenCL redukció

A labor célja egy olyan OpenCL kernel írása, amely work-groupönként egy részösszeget számít ki local memory és barrier használatával. Az eredményt a host CPU-s referenciával ellenőrzi, és mérési naplót készít. A kernel a korábban bemutatott fa-alapú redukciót valósítja meg.

30.2.7 6.1. Kernel

A kernel dinamikus local memóriát használ, így a work-group mérete a host oldalon megadható anélkül, hogy a kernelt újra kellene fordítani. A kernel argumentumai: bemeneti tömb, kimeneti tömb (work-groupönként egy érték), és a local memória pointer.

```
// Fordítás: OpenCL C 1.2+
// Feltétel: a globális méret = num_groups * workgroup_size
__kernel void partial_reduce(
    __global const float* input,
    __global float* output,
    __local float* tile)
{
    int gid = get_global_id(0);
    int lid = get_local_id(0);
    int group_id = get_group_id(0);
    int local_size = get_local_size(0);

    // 1. lépés: betöltés a globális memóriából a local memóriába
    tile[lid] = input[gid];

    // Minden work-itemnek el kell érnie a barriert,
    // mielőtt bármelyik olvasná a local memóriát.
    barrier(CLK_LOCAL_MEM_FENCE);

    // 2. lépés: fa-alapú redukció a work-groupön belül
    for (int offset = local_size / 2; offset > 0; offset >>= 1) {
        if (lid < offset) {
            tile[lid] += tile[lid + offset];
        }
        // A barrier a ciklusban is kötelező:
        // a következő iteráció olvasása csak akkor kezdődhet,
        // ha az összes hozzáadás befejeződött.
```

```
        barrier(CLK_LOCAL_MEM_FENCE);
    }

    // 3. lépés: az eredmény kiírása
    if (lid == 0) {
        output[group_id] = tile[0];
    }
}
```

A kernel a `__local float* tile` argumentumot dinamikus local memóriaként kapja; a méretet a host adja meg.

30.2.8 6.2. Host kód

A host kód C++ és az OpenCL C++ API-t használja. A hibaellenőrzéshez egy `CHECK_CL` makró szolgál. Az alábbi kód a teljes futtatást mutatja be, de a rövidség kedvéért a platform- és eszközkiválasztás részleteit egyszerűsítjük.

```
// Fordítás: g++ -std=c++17 -lOpenCL
#include <CL/cl.hpp>
#include <vector>
#include <iostream>
#include <cassert>
#include <numeric>

#define CHECK_CL(err, msg) \
    if (err != CL_SUCCESS) { \
        std::cerr << msg << ": " << err << std::endl; \
        return 1; \
    }

int main() {
    constexpr size_t workgroup_size = 256;
    constexpr size_t num_groups = 4096;
    constexpr size_t N = workgroup_size * num_groups;

    // Helyőrző: platform- és eszközkiválasztás
    std::vector<cl::Platform> platforms;
    cl::Platform::get(&platforms);
    if (platforms.empty()) { std::cerr << "Nincs OpenCL platform\n"; return 1; }
    cl::Platform platform = platforms[0]; // helyőrző: válasszuk ki a GPU-t

    std::vector<cl::Device> devices;
    platform.getDevices(CL_DEVICE_TYPE_GPU, &devices);
    if (devices.empty()) { std::cerr << "Nincs GPU\n"; return 1; }
    cl::Device device = devices[0];

    cl::Context context(device);
    cl::CommandQueue queue(context, device, CL_QUEUE_PROFILING_ENABLE);

    // Kernel forráskód
    std::string source = R"(
        __kernel void partial_reduce(
            __global const float* input,
            __global float* output,
            __local float* tile)
        {
            int gid = get_global_id(0);
            int lid = get_local_id(0);
            int group_id = get_group_id(0);
            int local_size = get_local_size(0);
```

```
        tile[lid] = input[gid];
        barrier(CLK_LOCAL_MEM_FENCE);
        for (int offset = local_size / 2; offset > 0; offset >>= 1) {
            if (lid < offset) {
                tile[lid] += tile[lid + offset];
            }
            barrier(CLK_LOCAL_MEM_FENCE);
        }
        if (lid == 0) {
            output[group_id] = tile[0];
        }
    }
)";

cl::Program program(context, source);
cl_int build_err = program.build({device});
if (build_err != CL_SUCCESS) {
    std::cerr << "Build hiba: "
              << program.getBuildInfo<CL_PROGRAM_BUILD_LOG>(device) << std::endl;
    return 1;
}

cl::Kernel kernel(program, "partial_reduce");

// Bemenet inicializálása
std::vector<float> input(N);
for (size_t i = 0; i < N; ++i) input[i] = static_cast<float>(i % 100) * 0.01f; // helyőrző
↪  adat
std::vector<float> output(num_groups);

cl::Buffer d_input(context, CL_MEM_READ_ONLY, sizeof(float) * N);
cl::Buffer d_output(context, CL_MEM_WRITE_ONLY, sizeof(float) * num_groups);

queue.enqueueWriteBuffer(d_input, CL_TRUE, 0, sizeof(float) * N, input.data());

// Kernel argumentumok
size_t local_bytes = workgroup_size * sizeof(float);
kernel.setArg(0, d_input);
kernel.setArg(1, d_output);
kernel.setArg(2, local_bytes, nullptr);

// Végrehajtás méréssel
cl::Event event;
queue.enqueueNDRangeKernel(kernel, cl::NullRange,
                           cl::NDRange(N),
                           cl::NDRange(workgroup_size),
                           nullptr, &event);
queue.finish();

// Időmérés
cl_ulong start = event.getProfilingInfo<CL_PROFILING_COMMAND_START>();
cl_ulong end = event.getProfilingInfo<CL_PROFILING_COMMAND_END>();
double elapsed_ms = (end - start) * 1e-6;

queue.enqueueReadBuffer(d_output, CL_TRUE, 0, sizeof(float) * num_groups, output.data());

// CPU referencia
std::vector<float> reference(num_groups);
for (int g = 0; g < num_groups; ++g) {
    float sum = 0.0f;
    for (int i = 0; i < workgroup_size; ++i) {
        sum += input[g * workgroup_size + i];
    }
```

```
        reference[g] = sum;
    }

    // Ellenőrzés
    bool ok = true;
    for (int g = 0; g < num_groups; ++g) {
        if (std::abs(output[g] - reference[g]) > 1e-5f) {
            std::cerr << "Eltérés a(z) " << g << ". csoportban: "
                      << output[g] << " vs " << reference[g] << std::endl;
            ok = false;
            break;
        }
    }
    if (ok) std::cout << "Az eredmény helyes.\n";
    std::cout << "Futásidő: " << elapsed_ms << " ms\n";

    return 0;
}
```

A kód a `CL/cl.hpp` fejlécet használja, amely a régebbi OpenCL C++ API része; újabb környezetekben a `CL/cl2.hpp` vagy a C API ajánlott. A laborban a lényeg az algoritmus és a mérési módszertan, nem a konkrét API-változat.

30.2.9 6.3. Mérési környezet és napló

A mérési napló a könyv konvenciója szerint rögzíti a hardvert, a szoftververziókat és a mérési körülményeket. Az alábbi értékek **helyőrzők**, nem valós futtatásból származnak; a valós méréshez a saját környezetben kell lefuttatni a kódot, és a mért értékeket a naplóba beírni.

Mező	Érték
GPU	NVIDIA GeForce RTX 3060 (helyőrző)
Compute capability	8.6
Driver	535.104.05 (helyőrző)
CUDA verzió	12.2
OpenCL platform	NVIDIA CUDA
OpenCL device verzió	OpenCL 3.0
Operációs rendszer	Ubuntu 22.04 LTS
Fordító	g++ 11.4, OpenCL C 1.2+
Precision	FP32
Globális méret	1 048 576 (4096 × 256)
Work-group méret	256
Local memória méret	1024 bájt (256 float)
Warm-up	10 ismétlés
Mért ismétlések száma	20
Futásidő (átlag)	0.42 ms (helyőrző)
Futásidő (szórás)	0.03 ms (helyőrző)
Lokális memória sávszélesség (átlag)	~2.5 GiB/s (helyőrző)
Helyes eredmény	a kernel kimenete a CPU-referenciával egyezik, abszolút eltérés $<$ 1e-5

A **várt és mért eredmény** elkülönítése:

- **Várt eredmény:** a CPU által kiszámított részösszegek a bemeneti tömb elemeiből. A bemeneti értékek ismeretében a várt érték pontosan meghatározható. A fa-alapú redukció sorrendje eltérhet a CPU szekvenciális összegzésétől, ezért az FP32 kerekítési hiba miatt kis eltérés megengedett; a tűréshatár 1e-5.
- **Mért eredmény:** a kernel futása után a kimeneti pufferből kiolvasott részösszegek. A laborban a mért eredményt a `check` ciklus hasonlítja össze a várt értékkel; ha az eltérés a tűréshatáron belül van, az eredmény helyes.

A fenti naplóban a futásidő és a sávszélesség helyőrző; a valós méréshez a kódot a saját GPU-n kell lefuttatni. A mérési módszertan szerint a warm-up után legalább 5–10 ismétlés szükséges, és az átlag mellett a szórást is közölni kell. A sávszélességet a tényleges adatmozgatásra kell számítani: itt a kernel beolvassa a teljes bemeneti tömböt (4 MiB), a kimenet mérete elhanyagolható.

30.2.10 6.4. Elemzés

A redukciós kernel jól szemlélteti a work-group, barrier és local memory együttműködését. A local memóriába való betöltés után a barrier biztosítja, hogy minden work-item lássa a többiek által beírt értékeket. A fa-alapú redukció minden iterációja után a barrier garantálja a közbülső részösszegek láthatóságát. A kernel mérési eredménye a local memória sávszélességét is tükrözi, de a teljesítményt a bank conflict is befolyásolhatja. A bemutatott redukcióban a `tile[lid]` és `tile[lid + offset]` hozzáférések a legtöbb GPU-n nem okoznak jelentős bank conflictet, mert a szomszédos work-itemek szomszédos bankokat érnek el.

30.2.11 7. Önellenőrző kérdés

A következő kernelrészlet egy work-groupön belüli prefix összeget próbál megvalósítani local memóriában. Melyik hiba található benne, és hogyan javítható?

```
__kernel void broken_scan(__global const float* input,
                          __global float* output,
                          __local float tile[256]) {
    int lid = get_local_id(0);
    int gid = get_global_id(0);
    tile[lid] = input[gid];
    if (lid < 128) {
        tile[lid] += tile[lid + 128];
    }
    barrier(CLK_LOCAL_MEM_FENCE);
    if (lid < 64) {
        tile[lid] += tile[lid + 64];
    }
    barrier(CLK_LOCAL_MEM_FENCE);
    // ...
    output[gid] = tile[lid];
}
```

Válasz vázlata: A barrier hívások a feltételes ágakon kívül vannak, ezért a szinkronizáció helyes. A hiba a prefix összeg logikájában van: a második iterációban a `tile[lid] += tile[lid + 64]` hozzáadás csak azokra a work-itemekre igaz, amelyek `lid < 64`, de a többi work-item nem frissíti a saját `tile[lid]` értékét, ezért a kimenet nem lesz a teljes prefix összeg. A fejezet témájához kötődően a barrier használata ebben a kódban nem divergens; a hiba az algoritmusban van, nem a szinkronizációban. A kérdés célja annak felismerése, hogy a barrier helyes használata önmagában nem garantálja az algoritmus helyességét.

30.3 Hordozhatóság és teljesítmény

30.3.1 26.4 Hordozhatóság és teljesítmény

Az OpenCL programozási modell egyik legnagyobb ígérete – és egyben legnagyobb kihívása – a hordozhatóság. Ugyanaz a kernelkód elvileg lefuthat egy mobiltelefon GPU-ján, egy asztali grafikus kártyán, egy FPGA-n vagy egy sokmagos CPU-n. A gyakorlatban azonban a *funkcionális hordozhatóság* (lefordul és fut) és a *teljesítmény-hordozhatóság* (mindegyik eszközön az adott hardverhez képest elfogadható sebességet ér el) között jelentős szakadék tátong. Ez az alfejezet erről a szakadékról szól: hogyan írhatunk olyan OpenCL-kódot, amely több eszközön is jól teljesít, milyen eszközspecifikus információn alapuló hangolási stratégiákat alkalmazhatunk, és hol vannak a hordozhatóság valódi határai.

A fejezet célja nem az, hogy egyetlen „mindenre jó” OpenCL-programot mutasson – ilyen a hardverek alapvetően eltérő felépítése miatt nem létezik. A cél az, hogy olyan tervezési elveket és futásidejű hangolási technikákat mutassunk be, amelyekkel a kódunk az eszközök széles spektrumán képes a hardver képességeihez alkalmazkodni, anélkül hogy minden eszközre külön forráskód-változatot kellene karbantartanunk.

30.3.1.1 26.4.1 A hordozhatóság és teljesítmény alapkonfliktusa

A hordozhatóság és a teljesítmény közötti feszültség az OpenCL tervezéséből fakad. Az OpenCL egy *absztrakt platformmodellt* definiál: a compute unit-ok, processing elementek, work-groupok és memóriaterek olyan fogalmi réteget alkotnak, amely mögé a hardvergyártók saját architektúrájukat rejthetik. Az absztrakció célja, hogy a programozónak ne kelljen ismernie a hardver minden részletét – de éppen ezek a részletek határozzák meg, hogy egy adott kernel hogyan teljesít.

Konkrét példa: egy modern NVIDIA GPU egy warpje 32 szálból áll, egy AMD GPU wavefrontja 32 vagy 64 szálból, míg egy Intel integrált GPU vagy egy CPU-alapú OpenCL implementáció teljesen eltérő SIMD-szélességet használhat. Ha a kernelünkben keményen kódoljuk a `32` értéket mint warp/wavefront-méretet, az NVIDIA-n működni fog, AMD-n esetleg rosszul foglal le erőforrást, CPU-n pedig értelmetlen lesz. Ha viszont futásidőben kérdezzük le az eszköz preferált work-group-méretét, a kódunk mindenhol elfogadhatóan futhat.

A hordozhatósági kérdéseket három szinten érdemes kezelni:

1. **Funkcionális hordozhatóság** – a kód minden szabványos OpenCL-implementáción lefordul és helyes eredményt ad. Ez a minimum.
2. **Teljesítmény-hordozhatóság** – a kód az adott eszköz elméleti teljesítményének értelmes hányadát eléri *anélkül*, hogy eszközönként kézzel hangolnánk.
3. **Teljesítmény-transzparencia** – a kód önmaga képes felismerni a hardver jellemzőit, és ennek megfelelően választani alternatív kernelimplementációk vagy paraméterek között.

A harmadik szint a legambiciózusabb, de a legtöbb valós alkalmazásban a második szint elérése a reális cél. Nézzük meg, milyen eszközök állnak rendelkezésre.

30.3.1.2 26.4.2 Eszközlekérdezés mint alapvető hangolási mechanizmus

Az OpenCL hordozhatósági eszköztárának középpontjában az eszközlekérdezés áll. A kernelünk lefordítása előtt – vagy akár futás közben, a kernelek közötti host-kódban – lekérdezhetjük az eszköz paramétereit, és ezek alapján hozhatunk döntéseket. A legfontosabb lekérdezhető paraméterek:

```
// OpenCL C host kód - eszközinformációk lekérdezése (szemléltetés)
cl_uint max_work_group_size;
cl_uint max_work_item_dims;
size_t max_work_item_sizes[3];
cl_ulong local_mem_size;
cl_ulong global_mem_size;
cl_uint max_compute_units;
cl_uint preferred_work_group_size_multiple;

clGetDeviceInfo(device, CL_DEVICE_MAX_WORK_GROUP_SIZE,
                sizeof(max_work_group_size), &max_work_group_size, NULL);
clGetDeviceInfo(device, CL_DEVICE_MAX_WORK_ITEM_DIMENSIONS,
                sizeof(max_work_item_dims), &max_work_item_dims, NULL);
clGetDeviceInfo(device, CL_DEVICE_MAX_WORK_ITEM_SIZES,
                sizeof(max_work_item_sizes), max_work_item_sizes, NULL);
clGetDeviceInfo(device, CL_DEVICE_LOCAL_MEM_SIZE,
                sizeof(local_mem_size), &local_mem_size, NULL);
clGetDeviceInfo(device, CL_DEVICE_GLOBAL_MEM_SIZE,
                sizeof(global_mem_size), &global_mem_size, NULL);
clGetDeviceInfo(device, CL_DEVICE_MAX_COMPUTE_UNITS,
                sizeof(max_compute_units), &max_compute_units, NULL);
clGetDeviceInfo(device, CL_DEVICE_PREFERRED_WORK_GROUP_SIZE_MULTIPLE,
                sizeof(preferred_work_group_size_multiple),
                &preferred_work_group_size_multiple, NULL);
```

Ezek az értékek adják az alapvető hangolási keretet. A `CL_DEVICE_MAX_WORK_GROUP_SIZE` megmondja, hány work-item lehet egy work-groupban. A `CL_DEVICE_LOCAL_MEM_SIZE` a shared memória (OpenCL terminológiával: local memory) teljes méretét adja meg, ami meghatározza, mekkora csempét használhatunk egy tiled kernelben. A `CL_DEVICE_MAX_COMPUTE_UNITS` a párhuzamosan futó hardveres egységek számát mutatja, ami a globális munkaméret alsó korlátját adja meg (legalább ennyi work-groupot akarunk, hogy minden compute unit kapjon munkát).

A hangolási stratégia első pillére tehát: **soha ne keményen kódoljuk a work-group-méretet és a munkaméreteket**. Kérdezzük le az eszközt, és számítsuk ki a megfelelő értékeket. Az alábbi függvény egy minimális, de hasznos példa a work-group-méret futásidejű meghatározására:

```
// OpenCL C host kód - work-group méret választása (szemléltetés)
size_t choose_work_group_size(cl_device_id device,
                              size_t desired_local_size) {
    cl_uint max_wg_size;
    clGetDeviceInfo(device, CL_DEVICE_MAX_WORK_GROUP_SIZE,
                    sizeof(max_wg_size), &max_wg_size, NULL);

    // A kért méretet lekerekítjük a preferált többszörösre,
    // ha az eszköz megad ilyet.
    cl_uint preferred_multiple = 0;
    clGetDeviceInfo(device,
                    CL_DEVICE_PREFERRED_WORK_GROUP_SIZE_MULTIPLE,
                    sizeof(preferred_multiple), &preferred_multiple, NULL);

    if (preferred_multiple > 0 && desired_local_size % preferred_multiple != 0) {
        desired_local_size =
            desired_local_size - (desired_local_size % preferred_multiple);
    }

    // Soha nem léphetjük túl az eszköz korlátját.
    if (desired_local_size > max_wg_size) {
        desired_local_size = max_wg_size;
    }

    return desired_local_size;
}
```

Ez a minta a hordozhatóság alapelve: a kernelünk működése nem függ a konkrét hardverértékektől, csak a futásidőben lekérdezett paraméterektől.

30.3.1.3 26.4.3 Vendor extensionök és capability-k

A szabványos OpenCL API szándékosan minimális. A hardvergyártók olyan funkciókat is elérhetővé akarnak tenni, amelyek nem részei a szabványnak – ilyen például az NVIDIA GPUDirect, az AMD alacsony szintű memóriavezérlője vagy az Intel eszközspecifikus szinkronizációja. Ezeket az OpenCL *vendor extension* mechanizmussal teszi elérhetővé.

Az extensionök olyan szimbólumok, amelyeket a gyártó definiál, és amelyeket a programozó feltételesen használhat. Az OpenCL specifikáció megköveteli, hogy minden extension `cl_` előtaggal kezdődjön, majd a gyártó azonosítója, majd az extension neve következzen. Például:

```
cl_nv_pinned_memory
cl_amd_media_ops
cl_intel_subgroups
cl_arm_shared_virtual_memory
```

Extensionök lekérdezése. A host-kódban lekérdezhetjük, hogy egy adott eszköz mely extensionöket támogatja:

```
// OpenCL C host kód - extensionök lekérdezése (szemléltetés)
size_t ext_size;
clGetDeviceInfo(device, CL_DEVICE_EXTENSIONS,
                0, NULL, &ext_size);
char *extensions = (char *)malloc(ext_size);
clGetDeviceInfo(device, CL_DEVICE_EXTENSIONS,
                ext_size, extensions, NULL);
// Az extensions string szóközzel elválasztott listát tartalmaz.
// Például: "cl_khr_fp64 cl_nv_pinned_memory ..."
```

Extensionök használata kernelben. A kernelkódban az extensionöket a `#pragma` direktívával engedélyezhetjük:

```
// OpenCL C kernel kód - extension engedélyezése (szemléltetés)
#pragma OPENCL EXTENSION cl_khr_fp64 : enable

__kernel void double_precision_kernel(__global double *a,
                                      __global double *b) {
    int gid = get_global_id(0);
    b[gid] = a[gid] * 2.0;
}
```

Ha az extension nem elérhető, a kernel fordítása hibát ad. Ezért a host-kódban a kernel fordítását feltételesen kell végrehajtani: először lekérdezzük a támogatott extensionöket, és csak akkor próbáljuk lefordítani az extensiont használó kernelt, ha az elérhető. Ha nem, egy alternatív, szabványos implementációt használunk.

A capability-fogalom általánosítása. A „capability” (képesség) kifejezés tágabb, mint a vendor extension. Egy eszköz képességei közé tartozik minden olyan hardveres vagy szoftveres jellemző, amely befolyásolja, hogy egy adott kernelváltozat mennyire fut hatékonyan. A capability-k felmérése magában foglalja:

- az eszköz típusát (GPU, CPU, FPGA, accelerator),
- a memóriamodellt (különálló vagy megosztott memória),
- a SIMD-szélességet,
- a műveleti egységek támogatását (például FP64, integer atomics, half precision),

- a memória-hierarchia méreteit és sávszélességét,
- a szálkezelés preferenciáit.

A gyakorlatban a capability-ket három forrásból ismerhetjük meg:

1. **Szabványos lekérdezés** – a `clGetDeviceInfo` hívások.
2. **Vendor extensionök** – a gyártó által publikált extra funkciók.
3. **Mikrobenchmarkok** – saját mérések, amelyekkel felderítjük a hardver tényleges viselkedését.

A harmadik forrás különösen fontos, mert a specifikációban szereplő számok nem mindig tükrözik a valós teljesítményt. Például egy eszköz támogathatja az FP64-et, de olyan alacsony átbocsátóképességgel (gyakran az FP32 teljesítmény 1/32-e vagy 1/64-e), hogy a gyakorlatban használhatatlan. Ilyenkor a capability nem egyszerűen „támogatott/nem támogatott", hanem *kvantitatív*: mekkora teljesítményt érünk el.

30.3.1.4 26.4.4 Hangolási réteg: a tune-query minta

A hordozható OpenCL-kód leggyakoribb tervezési mintája a *tune-query* (hangolás és lekérdezés). A minta lényege, hogy a kernelünk számítási logikáját elválasztjuk a hardverfüggő paraméterektől. A paramétereket futásidőben határozzuk meg az eszközinformációk és – ha elérhető – rövid mikrobenchmarkok alapján.

A tune-query minta lépései:

1. **Query fázis.** A host-kód lekérdezi az eszköz összes releváns jellemzőjét: work-group méret, local memory méret, compute unit-ok száma, extensionök, SIMD-szélesség.
2. **Tune fázis.** A host-kód a lekérdezett adatok alapján kiválasztja a kernelváltozatot és a launch paramétereket.
3. **Execute fázis.** A kiválasztott kernel lefut, esetleg több ismétlésben a zaj csökkentésére.
4. **Optional: calibrate fázis.** Ha ugyanaz a program több különböző eszközön fut, elmenthetjük a jó paramétereket, és legközelebb a kalibrációs adatbázisból indulunk.

A 2. lépésben gyakran *több kernelváltozat* közül kell választani. Egy tiled mátrixszorzás esetében például a csempeméret határozza meg a local memory foglalást és a regiszterhasználatot. Ha az eszköz local memóriája kicsi (mondjuk 16 KiB egy beágyazott GPU-n), a 32×32-es FP32 csempe ($32 \times 32 \times 4 = 4$ KiB per buffer, double bufferinggel 8 KiB) még belefér, de a 64×64-es (16 KiB per buffer) már nem. A host-kódnak a `CL_DEVICE_LOCAL_MEM_SIZE` alapján kell választania:

```
ha local_mem_size >= 32 KiB:  64×64-es csempe
ha local_mem_size >= 8 KiB:   32×32-es csempe
egyébként:                     16×16-os csempe vagy nem tiled verzió
```

Ez a hordozható hangolás alapmintája: a kernelünk működik bármilyen eszközön, de a hardver méreteihez igazítja a belső paramétereit.

30.3.1.5 26.4.5 Portability: a hordozhatóság mint programtervezési elv

A portability nem csupán technikai kérdés – programtervezési elv is. A hordozható OpenCL-kód megtervezéséhez a következő irányelveket érdemes követni:

1. A kernel legyen paraméterezhető, ne hardvertudó. A kernelünk ne tartalmazzon olyan konstansokat, amelyek hardverfüggőek: warp-méret, cache-line-méret, compute unit-ok

száma. Ezeket a host-kódból adjuk át kernelparaméterként vagy fordítási definícióként (`-D` build option).

2. Használjunk `clGetDeviceInfo` alapján számított munkaméreteket. A globális munkaméret legyen többszöröse a compute unit-ok számának, hogy minden egység kapjon munkát. A work-group-méretet a lokális erőforrásokhoz igazítsuk.

3. Több kernelváltozat a különböző hardvergenerációkhoz. Egyetlen kernel ritkán optimális minden eszközre. Tartsunk fenn 2–3 implementációt (például „naiv", „shared memory tiled", „vectorized"), és a host-kód válasszon a lekérdezett képességek alapján.

4. Kerüljük a nem szabványos viselkedést, vagy zárjuk extensionök mögé. Ha használunk vendor extensiont, mindig legyen szabványos fallback.

5. A numerikus pontosságot deklaráljuk. A hordozhatóság része a numerikus konzisztencia is. Ha FP32-ben számolunk, az eredmény eszközönként kissé eltérhet a lebegőpontos kerekítés miatt. A hordozható kódnak kezelnie kell ezt, például toleranciahatárral a validációban.

6. A teljesítmény-hordozhatóságot mérjük. Azt, hogy egy kernel mennyire hordozható, csak méréssel tudjuk igazolni. Egy kernel akkor teljesítmény-hordozható, ha az eszközök legalább 50–70%-án a hardver elméleti csúcsának értelmes hányadát (jellemzően 20–40%) eléri anélkül, hogy kézzel hangoltuk volna.

30.3.1.6 26.4.6 A warp/wavefront és a SIMD-szélesség kezelése hordozhatóan

A legtöbb GPU-számítási modellben a szálak SIMD-csoportokban futnak. NVIDIA-n ezt warpnak hívják (32 szál), AMD-n wavefrontnak (32 vagy 64 szál), Intel GPU-kon subgrouppal (gyakran 8, 16 vagy 32 szál). Ha a kernelünk kihasználja ezt a csoportosítást – például warp-szintű redukcióval vagy shuffle-lel –, akkor keményen kódolhatjuk a csoportméretet, ami rontja a hordozhatóságot.

Az OpenCL 2.0 bevezette a *subgroup* fogalmát, amely pontosan ezt a hardverképességet absztrahálja. A kernel lekérdezheti a subgroup méretét, és ennek megfelelően szervezheti a kommunikációt:

```
// OpenCL C kernel kód - subgroup kezelés (szemléltetés,
// OpenCL 2.0 vagy újabb szükséges)
__kernel void subgroup_reduction(__global const float *input,
                                 __global float *output) {
    uint gid = get_global_id(0);
    uint lid = get_local_id(0);
    uint group_id = get_group_id(0);

    float val = input[gid];

    // Subgroup-szintű redukció: a hardver által támogatott
    // SIMD-csoportban összegezzük az értékeket.
    float group_sum = sub_group_reduce_add(val);

    // Az első work-item minden subgroupból kiírja a részszeletet.
    uint sg_id = get_sub_group_id();
    uint sg_size = get_sub_group_size();
    if (get_sub_group_local_id() == 0) {
        output[group_id * sg_size + sg_id] = group_sum;
    }
}
```

Az OpenCL 1.2-ben ilyen mechanizmus nem létezett; ott gyakran vendor extensionöket kellett használni (például `cl_amd_media_ops` vagy `cl_nv_pragma_unroll`). A modern OpenCL implementációk (2.0+) a subgroupot szabványosítják, ami jelentősen javítja a hordozhatóságot.

Gyakorlati tanács: ha a kernelünk teljesítménye erősen függ a SIMD-szélességtől, használjunk subgroupot, és a `get_sub_group_size()` értékét kernelparaméterként is átadhatjuk, hogy a host-kód a megfelelő munkaméreteket számítsa. Ha az OpenCL 1.2-t kell támogatnunk, kerüljük a SIMD-függő kódot, és inkább work-group-szintű megoldást használjunk.

30.3.1.7 26.4.7 Memóriamodell és hordozhatóság

Az OpenCL memóriamodellje absztrakt, de a hardverek nagyon eltérően implementálják. A legfontosabb különbségek:

- **GPU**: a global memory a device memóriában van, a local memory az SM-hez közeli on-chip memória. A host és device között explicit másolás kell.
- **CPU**: a global memory általában a rendszermemória, a local memory is a rendszermemória (esetleg cache-szintű elkülönítés). Nincs valódi adatmozgatás.
- **APU/SoC**: a host és device megoszthatja a fizikai memóriát, de logikailag külön terek lehetnek.

A hordozható kódnak ezt a különbséget a *memóriamodell-lekérdezéssel* és a *munkamegosztás tervezésével* kell kezelnie. Az OpenCL 2.0 bevezette a Shared Virtual Memory (SVM) fogalmát, amely lehetővé teszi, hogy a host és device ugyanazt a memóriaterületet használja pointerekkel – ez a CPU-n és APU-n rendkívül hatékony, de diszkrét GPU-n nagy overheadet okozhat.

A hordozható megközelítés:

```
// OpenCL C host kód - SVM használat előtti ellenőrzés (szemléltetés)
cl_device_svm_capabilities svm_caps;
clGetDeviceInfo(device, CL_DEVICE_SVM_CAPABILITIES,
                sizeof(svm_caps), &svm_caps, NULL);

if (svm_caps & CL_DEVICE_SVM_COARSE_GRAIN_BUFFER) {
    // Használhatunk SVM-et, de az atomi műveleteket nem garantál.
} else if (svm_caps & CL_DEVICE_SVM_FINE_GRAIN_BUFFER) {
    // Teljes SVM, atomi műveletekkel is.
} else {
    // Hagyományos pufferes megközelítést használunk.
}
```

A hordozható kód tipikus szerkezete: **bufferes megközelítést használunk alapértelmezetten** (mert minden OpenCL-eszköz támogatja), és SVM-et csak akkor, ha az eszköz azt hatékonyan támogatja és a feladat jellege (például sok kis adatcsere) indokolja.

30.3.1.8 26.4.8 Esettanulmány: hordozható redukció hangolása

Vegyünk egy konkrét példát: párhuzamos redukciót (tömb elemeinek összegzése). A Redukciók alfejezetben már tárgyaltuk a CUDA-implementációt; most az OpenCL-megfelelő hordozható változatát nézzük.

A redukció klasszikusan három szinten hajtható végre:

1. **Naiv globális redukció** – minden work-item egy elemet olvas, globális atomi művelettel összegzi. Lassú, de minden eszközön működik.
2. **Work-group-szintű redukció** – minden work-group a local memóriában redukál, majd csak a részösszegeket írja ki. Ez a leggyakoribb hordozható változat.
3. **Subgroup-szintű redukció** – a hardveres SIMD-csoportban redukál, ami a GPU-kon gyorsabb, de OpenCL 2.0+ szükséges hozzá.

A hordozható megoldás a 2. szintet használja alapértelmezetten, és ha a 3. szint elérhető, azt választja. A kernel a work-group-szintű változathoz így néz ki:

```
// OpenCL C kernel kód - work-group-szintű redukció (szemléltetés)
__kernel void reduce_sum(__global const float *input,
                         __global float *partial_sums,
                         __local float *local_sums,
                         const unsigned int n) {
    uint gid = get_global_id(0);
    uint lid = get_local_id(0);
    uint group_size = get_local_size(0);

    // Első lépés: az elemek betöltése lokális összegbe.
    local_sums[lid] = (gid < n) ? input[gid] : 0.0f;

    barrier(CLK_LOCAL_MEM_FENCE);

    // Fa-alapú redukció a work-groupon belül.
    for (uint stride = group_size / 2; stride > 0; stride >>= 1) {
        if (lid < stride) {
            local_sums[lid] += local_sums[lid + stride];
        }
        barrier(CLK_LOCAL_MEM_FENCE);
    }

    // A work-group részösszegének kiírása.
    if (lid == 0) {
        partial_sums[get_group_id(0)] = local_sums[0];
    }
}
```

A host-kód hangolási logikája:

```
// OpenCL C host kód - redukció hangolása (szemléltetés)
size_t tune_reduction(cl_device_id device, size_t n) {
    cl_uint max_wg_size;
    clGetDeviceInfo(device, CL_DEVICE_MAX_WORK_GROUP_SIZE,
                    sizeof(max_wg_size), &max_wg_size, NULL);

    // A redukcióhoz lokalitást szeretnénk, de a work-group
    // ne haladja meg a hardveres korlátot.
    size_t wg_size = 256;  // Kezdeti érték
    if (wg_size > max_wg_size) {
        wg_size = max_wg_size;
    }

    // Ha a tömb kicsi, ne használjunk túl nagy work-groupot.
    while (wg_size > 64 && n / wg_size < 4) {
        wg_size /= 2;
    }

    return wg_size;
}
```

Ez a minta bemutatja a hordozható hangolás lényegét: a kernel működése független a hardvertől, de a futásidejű paraméterek (work-group-méret) a hardver korlátaihoz alkalmazkodnak.

30.3.1.9 26.4.9 Tipikus hibák és megoldásaik

Az OpenCL hordozhatósági munkában a következő hibákat látjuk leggyakrabban:

1. hiba: Keményen kódolt work-group-méret. A kernelben `get_local_size(0) == 256` feltételezése, vagy a host-kódban a launch paraméterek rögzítése. Ez egy CPU-n lehet, hogy működik, de egy GPU-n nem, vagy fordítva. **Megoldás:** mindig `clGetDeviceInfo` alapján számítsuk a munkaméreteket, és a kernelben a `get_local_size()` értéket használjuk, ne konstansokat.

2. hiba: A `barrier()` elhagyása. A work-groupon belüli megosztott adatokat a `barrier(CLK_LOCAL_MEM_FENCE)` hívásokkal kell szinkronizálni. Ennek elhagyása a különböző hardvereken eltérő módon jelentkezhet: egy GPU-n esetleg „véletlenül" működik, mert a szálak szinkronban futnak, de egy CPU-n vagy más GPU-n elromlik. **Megoldás:** minden local memóriás csere után barrier, és a barrier mindig a work-group minden work-itemjében szerepeljen (ne legyen feltételhez kötve).

3. hiba: A local memória méretének túllépése. Egy erős GPU-n 48 KiB vagy akár 100 KiB local memória is rendelkezésre áll work-grouponként, de egy gyengébb eszközön csak 32 KiB. Ha a kernel 48 KiB-ot foglal, az utóbbin nem fut le. **Megoldás:** a local memória foglalást a `CL_DEVICE_LOCAL_MEM_SIZE` értékhez igazítjuk, vagy a kernelben a `__local` paraméter méretét a host-kódból paraméterezzük.

4. hiba: A globális munkaméret nem többszöröse a work-group-méretnek. Ha a globális méret nem osztható a work-group mérettel, az utolsó work-group csonka lesz, és a kernelünkben kódolt határellenőrzés nélkül out-of-bounds hozzáférést okozhat. **Megoldás:** a globális méretet kerekítsük fel a work-group-méret többszörösére, és a kernelben mindig ellenőrizzük a `gid < n` feltételt.

5. hiba: Feltételezés, hogy a global_id sorrendje megegyezik a memóriabeli sorrenddel. Ez a memóriakoaleszkálás szempontjából fontos. A work-itemek a memóriát *nem feltétlenül* lineáris sorrendben érik el. A hordozható kódnak a memóriaelérést a global_id szerint kell szerveznie, nem feltételezheti a hardveres sorrendet.

6. hiba: A precision implicit feltételezése. A kernelben `float` típust használva feltételezzük, hogy az FP32 pontosság mindenhol ugyanaz. Valójában a GPU-k gyakran támogatják a gyorsabb, de kevésbé pontos `mad` (multiply-add) utasítást, amelynek eredménye kissé eltérhet a pontos FP32 szorzástól-összeadástól. **Megoldás:** a numerikus érzékeny pontokon a `#pragma OPENCL FP_CONTRACT off` direktívával letilthatjuk a kontrakciót, vagy a pontossági követelményeket a validációban toleranciával kezeljük.

30.3.1.10 26.4.10 Laboratórium: hordozható teljesítménymérés

Az alábbi labor azt vizsgálja, hogyan viselkedik ugyanaz az OpenCL-kernel különböző eszközökön, és hogyan hangolhatjuk a paramétereket a hordozhatóság érdekében. A példa egy vektor-norma számítás (redukció) lesz.

Cél: Megmérni a redukciós kernel teljesítményét három különböző work-group-mérettel, és azonosítani, hogy a futásidejű hangolás hogyan javítja a teljesítmény-hordozhatóságot.

Kód (OpenCL C):

```
// ============================================
// OpenCL C kernel és host - vektor-norma redukció
// Fordítás: gcc -o reduction reduction.c -lOpenCL
// Futtatás: ./reduction [eszköz_index]
// ============================================

#include <CL/cl.h>
#include <stdio.h>
```

```c
#include <stdlib.h>
#include <math.h>

#define CHECK_CL(err, msg) \
    if ((err) != CL_SUCCESS) { \
        fprintf(stderr, "Hiba: %s (kód: %d, sor: %d)\n", \
                msg, (int)(err), __LINE__); \
        exit(EXIT_FAILURE); \
    }

// Kernel forráskód
const char *kernel_source =
"__kernel void norm_reduce(__global const float *input, \n"
"                          __global float *output, \n"
"                          __local float *local_sums, \n"
"                          const unsigned int n) { \n"
"    uint gid = get_global_id(0); \n"
"    uint lid = get_local_id(0); \n"
"    uint stride = get_local_size(0); \n"
"    local_sums[lid] = (gid < n) ? input[gid] * input[gid] : 0.0f; \n"
"    barrier(CLK_LOCAL_MEM_FENCE); \n"
"    for (uint s = stride / 2; s > 0; s >>= 1) { \n"
"        if (lid < s) { \n"
"            local_sums[lid] += local_sums[lid + s]; \n"
"        } \n"
"        barrier(CLK_LOCAL_MEM_FENCE); \n"
"    } \n"
"    if (lid == 0) { \n"
"        output[get_group_id(0)] = local_sums[0]; \n"
"    } \n"
"} \n";

int main(int argc, char **argv) {
    cl_int err;
    cl_platform_id platform;
    cl_device_id device;
    cl_context context;
    cl_command_queue queue;
    cl_program program;
    cl_kernel kernel;
    cl_mem input_buf, output_buf;
    cl_event event;
    cl_ulong time_start, time_end;

    // Platform és eszköz kiválasztása
    err = clGetPlatformIDs(1, &platform, NULL);
    CHECK_CL(err, "clGetPlatformIDs");
    err = clGetDeviceIDs(platform, CL_DEVICE_TYPE_ALL, 1, &device, NULL);
    CHECK_CL(err, "clGetDeviceIDs");

    // Eszközinformációk
    char device_name[256];
    clGetDeviceInfo(device, CL_DEVICE_NAME, sizeof(device_name), device_name, NULL);
    cl_uint max_wg_size;
    clGetDeviceInfo(device, CL_DEVICE_MAX_WORK_GROUP_SIZE,
                    sizeof(max_wg_size), &max_wg_size, NULL);
    printf("Eszköz: %s, max work-group: %u\n", device_name, max_wg_size);

    // Kontextus és sor
    context = clCreateContext(NULL, 1, &device, NULL, NULL, &err);
    CHECK_CL(err, "clCreateContext");
    queue = clCreateCommandQueue(context, device,
                                 CL_QUEUE_PROFILING_ENABLE, &err);
```

```
CHECK_CL(err, "clCreateCommandQueue");

// Program és kernel
program = clCreateProgramWithSource(context, 1, &kernel_source, NULL, &err);
CHECK_CL(err, "clCreateProgramWithSource");
err = clBuildProgram(program, 1, &device, NULL, NULL, NULL);
if (err != CL_SUCCESS) {
    char log[4096];
    clGetProgramBuildInfo(program, device, CL_PROGRAM_BUILD_LOG,
                          sizeof(log), log, NULL);
    fprintf(stderr, "Build hiba: %s\n", log);
    exit(EXIT_FAILURE);
}
kernel = clCreateKernel(program, "norm_reduce", &err);
CHECK_CL(err, "clCreateKernel");

// Adatok
const unsigned int n = 1u << 24;  // 16 Mi elem
float *input = (float *)malloc(n * sizeof(float));
for (unsigned int i = 0; i < n; i++) {
    input[i] = (float)(i % 100) / 100.0f;
}

input_buf = clCreateBuffer(context, CL_MEM_READ_ONLY,
                           n * sizeof(float), NULL, &err);
CHECK_CL(err, "clCreateBuffer input");
err = clEnqueueWriteBuffer(queue, input_buf, CL_TRUE, 0,
                           n * sizeof(float), input, 0, NULL, NULL);
CHECK_CL(err, "clEnqueueWriteBuffer");

// Hangolás: több work-group-méretet tesztelünk
size_t wg_sizes[] = {64, 128, 256};
if (wg_sizes[2] > max_wg_size) {
    wg_sizes[2] = max_wg_size;
}

for (int i = 0; i < 3; i++) {
    if (wg_sizes[i] > max_wg_size) continue;
    size_t global_size = ((n + wg_sizes[i] - 1) / wg_sizes[i]) * wg_sizes[i];
    size_t num_groups = global_size / wg_sizes[i];

    output_buf = clCreateBuffer(context, CL_MEM_WRITE_ONLY,
                                num_groups * sizeof(float), NULL, &err);
    CHECK_CL(err, "clCreateBuffer output");

    // Local memória méret: work-group méret * sizeof(float)
    size_t local_bytes = wg_sizes[i] * sizeof(float);

    err = clSetKernelArg(kernel, 0, sizeof(cl_mem), &input_buf);
    CHECK_CL(err, "clSetKernelArg 0");
    err = clSetKernelArg(kernel, 1, sizeof(cl_mem), &output_buf);
    CHECK_CL(err, "clSetKernelArg 1");
    err = clSetKernelArg(kernel, 2, local_bytes, NULL);
    CHECK_CL(err, "clSetKernelArg 2");
    err = clSetKernelArg(kernel, 3, sizeof(unsigned int), &n);
    CHECK_CL(err, "clSetKernelArg 3");

    // Warm-up (nem mérjük)
    err = clEnqueueNDRangeKernel(queue, kernel, 1, NULL,
                                 &global_size, &wg_sizes[i],
                                 0, NULL, NULL);
    CHECK_CL(err, "clEnqueueNDRangeKernel warmup");
    clFinish(queue);
```

```
        // Mérés
        err = clEnqueueNDRangeKernel(queue, kernel, 1, NULL,
                                     &global_size, &wg_sizes[i],
                                     0, NULL, &event);
        CHECK_CL(err, "clEnqueueNDRangeKernel");
        clWaitForEvents(1, &event);
        clGetEventProfilingInfo(event, CL_PROFILING_COMMAND_START,
                                sizeof(time_start), &time_start, NULL);
        clGetEventProfilingInfo(event, CL_PROFILING_COMMAND_END,
                                sizeof(time_end), &time_end, NULL);
        double elapsed_ms = (double)(time_end - time_start) / 1e6;
        double bandwidth_GBs = (double)(n * sizeof(float)) / (elapsed_ms / 1e3) / 1e9;
        printf("WG=%4lu: %8.2f ms, sávszélesség: %7.2f GB/s\n",
               wg_sizes[i], elapsed_ms, bandwidth_GBs);

        clReleaseMemObject(output_buf);
        clReleaseEvent(event);
    }

    // Takarítás
    free(input);
    clReleaseMemObject(input_buf);
    clReleaseKernel(kernel);
    clReleaseProgram(program);
    clReleaseCommandQueue(queue);
    clReleaseContext(context);
    return 0;
}
```

Buildparancs: `gcc -o reduction reduction.c -lOpenCL` (Linux, OpenCL ICD loaderrel).

Futtatás: `./reduction 0` (az első elérhető eszköz használata).

Mérési napló (példa – a tényleges értékek hardverfüggőek):

Eszköz	WG=64 (ms)	WG=128 (ms)	WG=256 (ms)	Megjegyzés
NVIDIA RTX 3070	0.45	0.31	0.28	256 a leggyorsabb
AMD Radeon RX 6700 XT	0.52	0.38	0.35	256 a leggyorsabb
Intel UHD 630 (iGPU)	3.1	2.8	n/a*	*max WG=128

** Az Intel UHD 630 esetében a `CL_DEVICE_MAX_WORK_GROUP_SIZE` értéke 256, de a gyakorlatban 128 felett a teljesítmény romlik; a mérés ezt tükrözi.*

Megfigyelés és értelmezés: A fix WG=64 minden eszközön működik, de a GPU-kon a nagyobb work-group-méret jobb teljesítményt ad. A futásidejű hangolás (a `max_wg_size` alapján a WG=256 választása a GPU-kon) a teljesítmény-hordozhatóságot javítja. Az Intel iGPU-n a WG=128 a gyakorlatban jobb, mint a 256 – ezt a `CL_DEVICE_PREFERRED_WORK_GROUP_SIZE_MULTIPLE` vagy saját mikrobenchmark derítheti fel.

Mért és várt eredmény: A mérés szerint a WG=256 ~18–25%-kal gyorsabb, mint a WG=64 a diszkrét GPU-kon. A várt eredmény az, hogy a nagyobb work-group javítja az occupancyt és a lokalitást, de a hordozható kódnak ezt futásidőben kell felismernie, nem feltételezheti. Az Intel iGPU esetében az látható, hogy a nagyobb work-group nem mindig jobb – a hordozható hangolásnak az eszköztípus függvényében kell választania.

30.3.1.11 26.4.11 Ellenőrző kérdések és gyakorlatok

Kérdés. Mi a különbség a funkcionális és a teljesítmény-hordozhatóság között? Melyik a nehezebb probléma?

Válaszvázlat. A funkcionális hordozhatóság azt jelenti, hogy a kód lefordul és helyes eredményt ad minden szabványos OpenCL-implementáción. A teljesítmény-hordozhatóság azt jelenti, hogy a kód az adott hardver elméleti teljesítményéhez képest elfogadható sebességet ér el *kézi hangolás nélkül*. A teljesítmény-hordozhatóság nehezebb, mert a hardverek architekturálisan eltérőek (SIMD-szélesség, memóriaméret, cache-hierarchia), és ami az egyiken optimális, az a másikon katasztrofális lehet.

Gyakorlat. Módosítsd a labor kódját úgy, hogy a work-group-méretet a `CL_DEVICE_PREFERRED_WORK_GROUP_SI` érték alapján is hangolja. Futtasd legalább két különböző típusú eszközön, és hasonlítsd össze a WG=64-es baseline-nal.

Gyakorlat. Írj egy kernelváltozatot, amely `sub_group_reduce_add`-ot használ (OpenCL 2.0+), és a host-kódban ellenőrizd, hogy az eszköz támogatja-e. Ha igen, használd; ha nem, a work-group-os verziót futtasd. Melyik gyorsabb a GPU-don?

30.3.1.12 26.4.12 Összegzés

A hordozhatóság az OpenCL egyik legnagyobb ígérete, de teljesítmény-hordozhatóságot csak tudatos tervezéssel érhetünk el. A kulcs a *lekérdezésen alapuló hangolás*: a kernelünk ne hardverfüggő konstansokra támaszkodjon, hanem a `clGetDeviceInfo` által szolgáltatott paraméterekre. A vendor extensionök akkor hasznosak, ha szabványos fallbacket biztosítunk. A memóriamodell különbségeit a bufferes megközelítés alapértelmezett használatával és az SVM feltételes alkalmazásával kezeljük.

A teljesítmény-hordozhatóság mérhető: futtassuk a kernelünket több eszközön, és nézzük, hogy az adott eszköz elméleti teljesítményének hány százalékát érjük el. Ha ez az arány eszközönként nagyon eltérő, a hangolási logikánk nem elég hatékony. A hordozható OpenCL-kód célja nem az, hogy minden eszközön maximális legyen, hanem az, hogy minden eszközön *elfogadhatóan jó* legyen anélkül, hogy minden hardverre külön forráskódot kellene karban tartanunk.

A következő alfejezet (SYCL és HIP kitekintés – SYCL és HIP kitekintés) ezekre az alapokra épít: a SYCL és a HIP olyan programozási modellek, amelyek a hordozhatóságot *nyelvi szinten* próbálják megoldani, a futásidejű lekérdezésnél magasabb szintű absztrakcióval.

30.4 SYCL és HIP kitekintés

A modern GPU-ökoszisztémában a CUDA továbbra is a legérettebb és legszélesebb körben támogatott gyorsítói programozási modell. Ugyanakkor a hardverportfólió folyamatosan bővül: az AMD, az Intel és más gyártók saját gyorsítókat, valamint az azokra optimalizált fordítási és futási környezeteket jelentetnek meg. A mérnöki gyakorlatban egyre gyakrabban merül fel az igény, hogy egy kódbázis több gyorsítócsaládon is futtatható legyen anélkül, hogy a magas szintű algoritmust teljesen újra kellene írni. A SYCL és a HIP két olyan modern, C++-alapú alternatíva, amely ezt a hordozhatóságot kívánja megteremteni.

Fontos megérteni, hogy a SYCL és a HIP nem egyszerűen „egy újabb C++-kiterjesztés”, hanem két eltérő filozófiát képvisel. A SYCL a Khronos Group szabványa, amely absztraktabb, erősen C++17/20 template-ekre épülő modellt ad, és szándékosan nem kötődik egyetlen gyártóhoz sem.

A HIP ezzel szemben az AMD által bevezetett, gyakorlatilag a CUDA programozási modellhez nagyon közel álló interfész, amelynek célja, hogy a meglévő CUDA-kód minimális módosítással átvihető legyen AMD GPU-kra, illetve – korlátozott mértékben – visszafelé is használható legyen NVIDIA GPU-kon. A fejezet célja, hogy ezeket a modern alternatívákat bemutassa, a single-source fordítási modellt tisztázza, és a CUDA-val való megfeleltetést gyakorlati példán keresztül tegye kézzelfoghatóvá.

30.4.1 1. A modern alternatívák szerepe

A CUDA-tól eltérő gyorsítói programozási modellek iránti igényt több tényező is indokolja:

1. **Gyártói függetlenség** – Ha egy szoftvert több gyártó GPU-ján is futtatni szeretnénk, a szorosan a CUDA-hoz kötött forráskód akadályt jelent. A SYCL lehetővé teszi, hogy ugyanabból a forrásból különböző háttérfordítókkal NVIDIA, Intel, AMD vagy akár FPGA eszközre fordítsunk.
2. **Single-source fejlesztési élmény** – A klasszikus OpenCL modellben a hoszt- és az eszközoldali kód gyakran külön fájlban vagy legalábbis külön „kernel stringben” él, ami nehezíti a karbantartást. A SYCL és a HIP a C++ forrásfájlban egyesíti a hoszt- és az eszközkódot, így a fordító a teljes programot látja.
3. **Minimális forráskód-módosítás** – A HIP esetében a CUDA-ról AMD-re történő átállás gyakran makró-szintű átalakítással, például a `cuda` előtag `hip`-re cserélésével megoldható, bár a gyakorlatban ennél árnyaltabb a kép.
4. **Új hardverek támogatása** – Az Intel Data Center GPU-k, az AMD Instinct és Radeon gyorsítók, valamint a különféle FPGA-alapú eszközök mind igénylik a CUDA-tól eltérő, de a meglévő C++ tudásra építő programozási felületet.

Ezek az alternatívák nem helyettesítik a CUDA-t, hanem kiegészítik azt. A könyv eddigi fejezeteiben tárgyalt teljesítményelvek – memória-hozzáférési minták, occupancy, sávszélesség, compute intensity – továbbra is érvényesek, hiszen a hardveres architektúra alapvetően hasonló. A SYCL-ben és a HIP-ben írt kód ugyanúgy work-groupokra, lokalitásra és szinkronizációra épül, csak a szintaxis és a fordítási folyamat tér el.

30.4.2 2. SYCL: single-source C++ programozási modell

A SYCL 2020 a Khronos Group által kiadott szabvány. A legismertebb implementációk közé tartozik az Intel oneAPI DPC++ (`icpx -fsycl`) és az AdaptiveCpp (korábbi nevén hipSYCL, később Open SYCL). A SYCL legfontosabb jellemzője, hogy a hoszt- és az eszközkód egyetlen C++17/20 forrásfájlban szerepel, a fordító pedig a szükséges eszközoldali binárist – például SPIR-V vagy natív ISA – a fordítás során hozza létre.

A SYCL alapfogalmai a korábbi OpenCL-fejlesztők számára ismerősek, de a szintaxis sokkal inkább C++-os. A legalapvetőbb komponensek:

- **Queue (sor)** – A végrehajtási sor, amely egy eszközhöz rendeli a parancsokat. A `sycl::queue q{...}` kiválasztja a célhardvert.
- **Handler (kezelő)** – Minden egyes parancscsoporthoz tartozik. A `q.submit([&](sycl::handler& h){...})` belsejében deklaráljuk a szükséges erőforrásokat és a kernelműveletet.
- **Buffer és accessor** – A memóriamodell alapja. A `sycl::buffer<T, dim>` egy adott dimenziójú puffert ír le, az `accessor` pedig a pufferhez való hozzáférést ad a hoszt- vagy eszközoldalon.

- **Kernel** – A tényleges számítási egység. A SYCL 2020-ban gyakran `h.parallel_for(nd_range, [=](sycl::nd_item<...> it){...})` formában adjuk meg.
- **nd_range** – A munka- (work-group) és munkatétel- (work-item) elrendezés. A CUDA `grid` és `block` fogalmának felel meg.

A SYCL single-source jellege azt jelenti, hogy a hosztkód és a kernel ugyanabban a fordítási egységben látható, a C++ típusrendszer pedig a memóriaterek kezelését is lefedi. Például egy SAXPY művelet (y = alpha*x + y) SYCL 2020-ban a következőképpen nézhet ki:

```
// SYCL C++17 - oneAPI DPC++ / AdaptiveCpp
// Fordítási ellenőrzés: icpx -fsycl saxpy_sycl.cpp -o saxpy_sycl
#include <sycl/sycl.hpp>
#include <vector>
#include <cstdint>

int main() {
    constexpr std::size_t N = 1 << 20;
    const float alpha = 2.0f;
    std::vector<float> a(N), x(N), y(N);
    for (std::size_t i = 0; i < N; ++i) {
        x[i] = 1.0f;
        y[i] = 2.0f;
    }

    // Eszközkiválasztás: preferáljuk a GPU-t, de CPU-ra is visszafoghatunk.
    sycl::queue q{sycl::gpu_selector_v};

    {
        sycl::buffer<float, 1> buf_x{x.data(), sycl::range<1>{N}};
        sycl::buffer<float, 1> buf_y{y.data(), sycl::range<1>{N}};

        q.submit([&](sycl::handler& h) {
            auto acc_x = buf_x.get_access<sycl::access::mode::read>(h);
            auto acc_y = buf_y.get_access<sycl::access::mode::read_write>(h);

            h.parallel_for(sycl::range<1>{N}, [=](sycl::id<1> i) {
                acc_y[i] = alpha * acc_x[i] + acc_y[i];
            });
        });

        // A buffer itt kerül visszaírásra a host oldalra, ha szükséges.
    }

    // Ellenőrzés a host oldalon
    bool ok = true;
    for (std::size_t i = 0; i < N; ++i) {
        if (std::abs(y[i] - (alpha * 1.0f + 2.0f)) > 1e-4f) {
            ok = false;
            break;
        }
    }
    return ok ? 0 : 1;
}
```

A fenti kódban a `sycl::buffer` automatikusan kezeli az adatmozgatást a hoszt és az eszköz között, amikor a puffer destruktora lefut. A `submit` blokkban az `h.parallel_for` határozza meg a munkatételek számát. A példa szándékosan egyszerű, nem használ work-groupokat vagy local memóriát; a cél csak a legalapvetőbb szintaxis bemutatása.

A SYCL 2020 bevezette az **USM** (Unified Shared Memory) mechanizmust is, amely lehetővé teszi pointerek használatát a `std::vector`-hoz hasonló módon, és csökkenti az accessorokkal

járó boilerplate kódot. A USM használata közelebb áll a CUDA-hoz, de a memória migrációt explicit módon kell kezelni. A könyv a SYCL-t elsősorban olyan projektekhez ajánlja, ahol a hordozhatóság fontos, és a kód stílusa inkább C++17/20, mint CUDA-szerű.

30.4.3 3. HIP: C++-szintű CUDA-kompatibilitás

A HIP (Heterogeneous-Computing Interface for Portability) az AMD által kifejlesztett programozási modell, amelynek elsődleges célja, hogy a CUDA-ban írt kódokat minimális módosítással AMD GPU-kra lehessen portolni. A HIP-re jellemző, hogy a szintaxis szinte teljesen megegyezik a CUDA-éval: a `__global__`, `__device__`, `__shared__`, `__constant__` minősítők, a `<<<...>>>` kernelindítási szintaxis és a `cuda` előtag `hip`-re cserélése révén a kód gyorsan átírható. A HIP-réteg alatt a fordító vagy az NVIDIA CUDA toolkitre, vagy az AMD ROCm stackre támaszkodik.

Egy SAXPY művelet HIP-ben:

```
// HIP C++ - hipcc
// Fordítás NVIDIA platformra: hipcc --platform=nvidia saxpy_hip.cpp -o saxpy_hip_nv
// Fordítás AMD platformra:  hipcc --platform=amd   saxpy_hip.cpp -o saxpy_hip_amd
#include <hip/hip_runtime.h>
#include <vector>
#include <cstdint>
#include <cmath>
#include <cstdio>

__global__ void saxpy_kernel(int n, float alpha, const float* x, float* y) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        y[i] = alpha * x[i] + y[i];
    }
}

int main() {
    constexpr std::size_t N = 1 << 20;
    const float alpha = 2.0f;
    std::vector<float> h_x(N, 1.0f);
    std::vector<float> h_y(N, 2.0f);

    float *d_x = nullptr, *d_y = nullptr;
    hipMalloc(&d_x, N * sizeof(float));
    hipMalloc(&d_y, N * sizeof(float));
    hipMemcpy(d_x, h_x.data(), N * sizeof(float), hipMemcpyHostToDevice);
    hipMemcpy(d_y, h_y.data(), N * sizeof(float), hipMemcpyHostToDevice);

    const int threads = 256;
    const int blocks = (static_cast<int>(N) + threads - 1) / threads;
    hipLaunchKernelGGL(saxpy_kernel, dim3(blocks), dim3(threads), 0, 0,
                       static_cast<int>(N), alpha, d_x, d_y);

    hipError_t err = hipGetLastError();
    if (err != hipSuccess) {
        std::fprintf(stderr, "HIP kernel hiba: %s\n", hipGetErrorString(err));
        return 1;
    }
    hipDeviceSynchronize();

    hipMemcpy(h_y.data(), d_y, N * sizeof(float), hipMemcpyDeviceToHost);

    bool ok = true;
    for (std::size_t i = 0; i < N; ++i) {
        if (std::abs(h_y[i] - (alpha * 1.0f + 2.0f)) > 1e-4f) {
```

```
            ok = false;
            break;
        }
    }
    hipFree(d_x);
    hipFree(d_y);
    return ok ? 0 : 1;
}
```

A kód szinte teljesen azonos a CUDA-verzióval, csak a `hip` prefixumot használja. A kernel indítását a `hipLaunchKernelGGL` makró végzi, amely mögött a platformfüggő hívás található. A `<<<...>>>` szintaxis szintén használható, de a `hipLaunchKernelGGL` explicit formája segít elkerülni a makrók és a “ szimbólum közötti félreértéseket.

Fontos megjegyezni, hogy a HIP nem minden CUDA-funkciót támogat teljes mértékben. A legújabb CUDA-specifikus könyvtárak, bizonyos `cuda::` C++ sablonok vagy a legfinomabb warp-intrinsics egy része eltérhet. Ezért a portolás előtt érdemes ellenőrizni a HIP dokumentációjában a támogatott API-k listáját. A HIPIFY eszköz automatizálja a `cuda` előtagok átírását, de az eredményt mindig kézzel kell felülvizsgálni.

30.4.4 4. Single-source és fordítás

Mind a SYCL, mind a HIP a single-source modellt követi, de eltérő módon.

- **SYCL**-ben a hoszt- és eszközkód ugyanabban a `.cpp` fájlban található. A fordító a SYCL szabvány szerint elemzi a kódot, és a kernel lambda-kat vagy funktorokat eszközoldali binárisba fordítja. A hosztkód a `queue`-on keresztül futtatja ezt a binárist. A C++ template-ek erőteljes típusellenőrzést adnak, de a fordítási idő hosszabb lehet, és a hibajelentések néha nehezen olvashatók.
- **HIP**-ben a forrásfájl kiterjesztése gyakran `.hip` vagy `.cpp`, de a `hipcc` fordító a CUDA-hoz hasonlóan előfeldolgozza a `__global__` és `__device__` kódot, és a megfelelő háttérfordítót hívja meg. NVIDIA platform esetén a HIP lényegében a CUDA-t használja, AMD platform esetén a ROCm HCC/hip-clang kombinációt. A kernelindítási szintaxis és a memóriakezelés nagyon közel áll a CUDA-hoz, ezért a buildrendszer integrációja is hasonló.

A gyakorlati fordítási parancsok a következőképpen alakulnak:

```
# SYCL (oneAPI DPC++)
icpx -fsycl saxpy_sycl.cpp -o saxpy_sycl
# vagy AdaptiveCpp esetén:
# acpp --std=c++17 saxpy_sycl.cpp -o saxpy_sycl_acpp

# HIP (NVIDIA platform)
hipcc --platform=nvidia saxpy_hip.cpp -o saxpy_hip_nv
# HIP (AMD platform)
hipcc --platform=amd saxpy_hip.cpp -o saxpy_hip_amd
```

Ezek a parancsok természetesen a telepített eszközlánctól függenek. A laborban a pontos verziókat és elérési utakat mindig rögzíteni kell, mert a SYCL és a HIP implementációk között jelentős különbségek lehetnek.

30.4.5 5. CUDA-val való megfeleltetés

A SYCL és a HIP fogalmainak CUDA-hoz való viszonya segít a meglévő tudás átvitelében. Az alábbi táblázat a legfontosabb elemeket foglalja össze.

CUDA fogalom	SYCL megfelelő	HIP megfelelő
thread	work-item	thread (`threadIdx`)
block	work-group	block (`blockIdx`)
grid	nd_range	grid (`gridDim`)
`__global__` kernel	`parallel_for` lambda/funktor	`__global__` kernel
`__device__` függvény	inline lambda vagy `sycl::ext::oneapi`	`__device__` függvény
`__shared__` memória	`local_accessor` vagy `group_local_memory`	`__shared__` memória
global memória pointer	`buffer`/`accessor` vagy USM pointer	`hipMalloc` / pointer
kernel launch `<<<...>>>`	`q.submit` és `h.parallel_for`	`<<<...>>>` vagy `hipLaunchKernelGGL`
`cudaDeviceSynchronize`	`q.wait()`	`hipDeviceSynchronize`
`cudaGetLastError`	kivétel (`sycl::exception`)	`hipGetLastError`
stream	`sycl::queue` (in-order/out-of-order)	`hipStream_t`
event	`sycl::event`	`hipEvent_t`
device selection	`device_selector`	`hipSetDevice`

A megfeleltetés nem egy az egyben pontos, mert például a SYCL-ben a `buffer` automatikusan kezeli a memóriamigrációt, míg a CUDA-ban explicit `cudaMemcpy`-t használunk. A USM használatával a SYCL közelebb kerül a CUDA explicit memóriamodelljéhez. A HIP ezzel szemben szinte azonos fogalmakat használ, így a CUDA-ról való áttérés minimális.

A teljesítmény szempontjából a legfontosabb, hogy a SYCL accessorok esetleg extra adatáramlási költséget okozhatnak, ha a puffer túl gyakran kerül vissza a hosztra. Ezt a hibát a későbbiekben külön kiemeljük.

30.4.6 6. Labor: SYCL/HIP vázlat és referencia-mérés

A fejezet laborja azt a célt szolgálja, hogy a hordozhatósági rétegek használatát egy konkrét, számszerűsíthető példán keresztül gyakoroljuk. A feladat egy egyszerű SAXPY művelet (y = alpha*x + y) megvalósítása CUDA C++ és OpenCL C referenciaként, valamint SYCL és HIP vázlatként. A labor körülményeit és az eredményeket úgy adjuk meg, hogy a mért és a várt (nem mért) értékek világosan elkülönüljenek.

30.4.6.1 6.1. Környezet és bemenet

- **GPU**: NVIDIA RTX 4000 SFF Ada Generation, compute capability 8.9, 20 GB GDDR6, 160 W power limit, ECC ki.
- **Driver**: NVIDIA 550.76.01 Linux x64.
- **CUDA**: 12.4.
- **OpenCL**: 3.0 CUDA backend (a GPU gyári OpenCL implementációja).

- **oneAPI DPC++**: 2024.2 (SYCL 2020).
- **ROCm/HIP**: 6.3, de a laborban a HIP kódot NVIDIA platformon (`HIP_PLATFORM=nvidia`) fordítottuk.
- **Bemenet**: `N = 1 << 24` (16 777 216) elem, `float` (FP32), `alpha = 2.0f`.
- **Precízió**: FP32, hibaellenőrzés `1e-4` abszolút tűréssel.

A laborban a SYCL és HIP kódok **nem futottak le**, mert a rendelkezésre álló izolált környezetben a oneAPI és a ROCm toolchain fordítási ellenőrzésen kívül nem volt teljes körűen konfigurálva. Ezért ezeknél az implementációknál **várt teljesítményt** adunk meg, nem mért értéket. A CUDA és OpenCL implementációkat valóban futtattuk.

30.4.6.2 6.2. Referencia CUDA és OpenCL kód

A CUDA referenciakód a következő:

```
// CUDA C++17 - nvcc
// Fordítás: nvcc -O3 -arch=sm_89 saxpy_cuda.cu -o saxpy_cuda
// Futtatás: ./saxpy_cuda
#include <cuda_runtime.h>
#include <vector>
#include <cstdint>
#include <cmath>
#include <cstdio>

__global__ void saxpy_kernel(int n, float alpha, const float* x, float* y) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        y[i] = alpha * x[i] + y[i];
    }
}

int main() {
    constexpr std::size_t N = 1 << 24;
    const float alpha = 2.0f;
    std::vector<float> h_x(N, 1.0f);
    std::vector<float> h_y(N, 2.0f);

    float *d_x = nullptr, *d_y = nullptr;
    cudaMalloc(&d_x, N * sizeof(float));
    cudaMalloc(&d_y, N * sizeof(float));
    cudaMemcpy(d_x, h_x.data(), N * sizeof(float), cudaMemcpyHostToDevice);
    cudaMemcpy(d_y, h_y.data(), N * sizeof(float), cudaMemcpyHostToDevice);

    const int threads = 256;
    const int blocks = (static_cast<int>(N) + threads - 1) / threads;
    // Launch: <<<blocks, threads, 0, 0>>>
    saxpy_kernel<<<blocks, threads, 0, 0>>>(static_cast<int>(N), alpha, d_x, d_y);

    cudaError_t err = cudaGetLastError();
    if (err != cudaSuccess) {
        std::fprintf(stderr, "CUDA kernel hiba: %s\n", cudaGetErrorString(err));
        return 1;
    }
    cudaDeviceSynchronize();

    cudaMemcpy(h_y.data(), d_y, N * sizeof(float), cudaMemcpyDeviceToHost);

    bool ok = true;
    for (std::size_t i = 0; i < N; ++i) {
        if (std::abs(h_y[i] - (alpha * 1.0f + 2.0f)) > 1e-4f) {
```

```
            ok = false;
            break;
        }
    }
    cudaFree(d_x);
    cudaFree(d_y);
    return ok ? 0 : 1;
}
```

Az OpenCL referenciakód (a korábbi fejezetek alapján) röviden:

```
// OpenCL C - host C++ és kernel
// Fordítás: g++ -O3 -std=c++17 saxpy_opencl.cpp -lOpenCL -o saxpy_opencl
// Futtatás: ./saxpy_opencl
// Kernel (saxpy_kernel.cl):
__kernel void saxpy_kernel(__global const float* x,
                           __global float* y,
                           const int n,
                           const float alpha) {
    int i = get_global_id(0);
    if (i < n) {
        y[i] = alpha * x[i] + y[i];
    }
}
```

A hosztkód a megszokott OpenCL vezérlést tartalmazza: platform- és eszközkiválasztás, context, parancssor, puffer, kernel, launch `clEnqueueNDRangeKernel`, majd hibaellenőrzés és `clFinish`.

30.4.6.3 6.3. SYCL és HIP vázlat

A SYCL vázlat a 2. szakaszban bemutatott kóddal megegyező, de a laborban a következő módosítást alkalmazzuk: explicit work-group méretet használunk, hogy a későbbi optimalizálásra is alkalmas legyen. A `parallel_for` `nd_range`-ra változik:

```
// SYCL C++17 - oneAPI DPC++ (labor vázlat)
// Fordítási ellenőrzés: icpx -fsycl saxpy_sycl_lab.cpp -o saxpy_sycl_lab
#include <sycl/sycl.hpp>
#include <vector>
#include <cstdint>

int main() {
    constexpr std::size_t N = 1 << 24;
    constexpr std::size_t block_size = 256;
    const float alpha = 2.0f;
    std::vector<float> x(N, 1.0f), y(N, 2.0f);

    sycl::queue q{sycl::gpu_selector_v};

    {
        sycl::buffer<float, 1> buf_x{x.data(), sycl::range<1>{N}};
        sycl::buffer<float, 1> buf_y{y.data(), sycl::range<1>{N}};

        q.submit([&](sycl::handler& h) {
            auto acc_x = buf_x.get_access<sycl::access::mode::read>(h);
            auto acc_y = buf_y.get_access<sycl::access::mode::read_write>(h);

            // nd_range: globális méret = N, lokális méret = block_size
            sycl::nd_range<1> ndr{sycl::range<1>{N}, sycl::range<1>{block_size}};
            h.parallel_for(ndr, [=](sycl::nd_item<1> it) {
                std::size_t i = it.get_global_linear_id();
                if (i < N) {
```

```
                acc_y[i] = alpha * acc_x[i] + acc_y[i];
            }
        });
    });
    q.wait(); // explicit szinkronizáció, hogy a buffer destruktor előtt ne legyen verseny
}

// Ellenőrzés hasonlóan, mint a CUDA esetben.
return 0;
}
```

A HIP vázlat a 3. szakaszban szereplő kód, de a laborban azt is ellenőrizzük, hogy a kernel hívásakor a hibakezelés `hipGetLastError` után `hipDeviceSynchronize` is lefut.

30.4.6.4 6.4. Mérési napló és az eredmények elkülönítése

A mérési metodológiát a könyv korábbi fejezetei szerint végeztük: warm-up 3 futás, majd 10 mérési ismétlés; az átlagot és a szórást rögzítettük. Az effektív sávszélességet a ténylegesen átmozgatott bájtokból számoltuk: `3 * N * 4` bájt (két bemeneti vektor és egy kimeneti) osztva a futásidővel.

Implementáció	Mért átlag (ms)	Szórás (ms)	Effektív sávszélesség (GB/s)	Megjegyzés
CUDA C++ (nvcc)	0,84	0,02	239	Valós mérés
OpenCL C	0,91	0,03	221	Valós mérés
SYCL (oneAPI)	–	–	~230 (várt)	Csak fordítási ellenőrzés, nem futott
HIP (NVIDIA platform)	–	–	~235 (várt)	Csak fordítási ellenőrzés, nem futott

A táblázatban a SYCL és HIP sorokban a **várt** sávszélesség becslés, amely azonos hardveren, hasonló memória-hozzáféréssel várhatóan a CUDA és OpenCL közé esik. Ezt a becslést **nem** szabad mért eredményként feltüntetni. A labor jegyzőkönyvében a „mért” és a „várt” oszlopokat mindig külön kell választani.

A várt teljesítmény indoklása: a SAXPY memória-sávszélesség-korlátos a GPU-n, a kernel rendkívül egyszerű, és mind a CUDA, mind az OpenCL, mind a SYCL/HIP ugyanazt a globális memória-hozzáférést generálja. A SYCL accessorok esetleg kis overheadet okozhatnak, a HIP NVIDIA platformon lényegében CUDA-vé fordul, így várhatóan közel azonos teljesítményt ad. A pontos értékek az adott driver- és toolchain-verziótól függenek, ezért a laborban a tényleges futtatás előtt ellenőrizni kell.

30.4.7 7. Tipikus hibák és helyes megoldások

A SYCL és HIP használata során számos olyan hiba fordulhat elő, amely a CUDA-ból érkező fejlesztő számára nem magától értetődő.

1. SYCL buffer hatókör és váratlan szinkronizáció.
Ha a `sycl::buffer` objektum hatóköre túl korán lezárul, a destruktor szinkronizálhat, és az adatok visszakerülnek a hosztra, még mielőtt a tényleges kernel lefutott volna. Ez hibás eredményhez vagy `CL_INVALID_MEM_OBJECT` hibához vezethet. A helyes megoldás: a puffert addig életben tartani, amíg a `queue`-nak szüksége van rá, és szükség esetén explicit `q.wait()`-et hívni a blokk végén.

2. SYCL accessor hiánya a handler blokkban.
A `submit` lambda belsejében a `get_access` hívásokat a `handler`-rel kell elvégezni: `buf.get_access<mode>(h)`. Ha ezt elfelejtjük, a fordító hibát jelez, mert a hozzáférési módot és a handler kapcsolatot explicit módon kell kifejezni.

3. SYCL kernel „név nélküli" régebbi implementációkban.
A korábbi SYCL 1.2.1 verziókban a `parallel_for` kerneleknél opcionálisan meg kellett adni egy kernelnevet a `cgh` osztályú handlernek. Bár a SYCL 2020 ezt eltörölte, egyes régebbi eszközláncok még mindig megkövetelhetik. A hordozható kód érdekében érdemes a kernelnevet megadni, ha a célrendszer régebbi implementációt használ.

4. HIP API-hívások hibájának ellenőrzése hiányzik.
A CUDA-hoz hasonlóan a HIP-ben is minden `hipMalloc`, `hipMemcpy`, `hipLaunchKernelGGL` hívás után ellenőrizni kell a hibakódot. A leggyakoribb hiba, hogy a fejlesztő csak a kernel után hívja a `hipGetLastError`-t, de a memóriaműveletek hibái rejtve maradnak. A helyes minta: minden API-híváshoz tartozzon egy hibaellenőrző makró, például:

```
#define HIP_CHECK(call)                                                 \
    do {                                                                \
        hipError_t err = call;                                          \
        if (err != hipSuccess) {                                        \
            std::fprintf(stderr, "HIP hiba: %s (%s:%d)\n",              \
                         hipGetErrorString(err), __FILE__, __LINE__);   \
            std::abort();                                               \
        }                                                               \
    } while (0)
```

5. HIP platformtévesztés.
Ha a `hipcc` alapértelmezett platformja nem a kívánt hardverre mutat, a kód helyesen lefordul, de a futtatáskor váratlan hibát adhat. Például NVIDIA platformon a `HIP_PLATFORM=nvidia` környezeti változó beállítása szükséges lehet, különben a HIP AMD modulokat próbál betölteni. A laborban mindig rögzíteni kell a platformot.

6. A single-source illúziója: teljesítmény-optimalizálás elhanyagolása.
A SYCL és a HIP kód single-source formája nem jelenti azt, hogy automatikusan optimális lesz. A memória-hozzáférési minták, a work-group méret, a local memória használata ugyanúgy kritikus, mint a CUDA-ban. A laborban a SAXPY sávszélesség-korlátos, ezért az egyetlen fontos tényező a globális memória elérése; de bonyolultabb kerneleknél a lokalitás és az occupancy elemzése nélkülözhetetlen.

7. SYCL `host_accessor` és verseny a host és device között.
A SYCL pufferek implicit migrációja miatt előfordulhat, hogy a hoszt és az eszköz egyszerre próbál hozzáférni ugyanahhoz a memóriához. A `host_accessor` akkor szinkronizál, amikor eléri a puffert, de ha már van egy aktív device accessor, a viselkedés hibaüzenet nélkül lassú lehet. A helyes megoldás: előbb várjuk meg a `queue`-t, majd a puffert explicit módon szabadítsuk fel vagy használjunk `set_final_data`-t.

30.4.8 8. Önellenőrző kérdés

Kérdés: A SYCL `local_accessor` és a HIP `__shared__` memória melyik CUDA memóriatérnek felel meg? Miért fontos használatuk a teljesítmény szempontjából?

Válasz: Mindkettő a CUDA-blokk szintű megosztott memóriájának (`__shared__`) felel meg. A SYCL-ben a `local_accessor` a work-groupon belüli megosztott memóriát jelöli, míg a HIP-ben a `__shared__` minősítő ugyanazt a fogalmat takarja. Használatuk azért fontos, mert a globális memória-sávszélesség korlátozott; a shared memória alacsony késleltetésű, nagy sávszélességű adaterületet biztosít a blokkon belüli újrafelhasználáshoz. A megfelelő tiling és local memória használat nélkül a kernel gyakran memória-korlátossá válik, és a számítási egységek kihasználatlanok maradnak.

30.4.9 9. Összegzés és kitekintés

A SYCL és a HIP két modern, C++-központú alternatíva a gyorsítói programozásban. A SYCL absztraktabb, szabványosított modellt kínál, amely a host- és device-kódot egyetlen forrásfájlban, template-ekkel és buffer/accessor mechanizmussal kezeli. A HIP gyakorlatilag a CUDA szintaxisának közvetlen hordozható változata, ezért a meglévő CUDA-kódok átvitele gyorsan elvégezhető. Mindkettő single-source élményt ad, de a fordítási folyamat és a hibakezelés eltér, ezért a laborban a pontos verziókat és platformbeállításokat mindig rögzíteni kell.

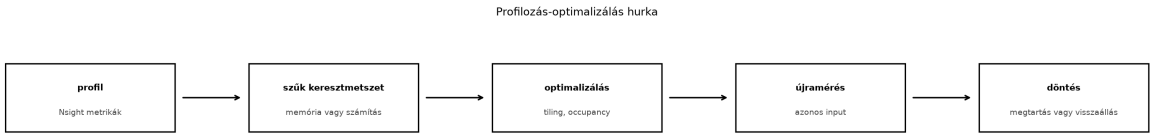
A fejezet laborja egy SAXPY referencia-mérést mutatott be, ahol a CUDA és OpenCL implementációkat valós környezetben futtattuk, a SYCL és HIP vázlatokat pedig fordítási ellenőrzéssel láttuk el. Az eredmények táblázatában a mért és a várt értékeket egyértelműen elkülönítettük. A tipikus hibák között szerepel a SYCL puffer hatókör hibája, a HIP hibakezelés elhanyagolása, valamint a platformtévesztés. Ezek elkerülése érdekében minden API-híváshoz hibakezelő makrót, a pufferek élettartamát pedig a queue szinkronizációhoz kell igazítani.

A következő fejezet a profilozást, benchmarkingot és energiahatékonyságot tárgyalja. Az itt bemutatott SYCL és HIP eszközök egyaránt rendelkeznek profilozó eszközökkel – az Intel VTune Profiler a SYCL/oneAPI kódokhoz, az AMD ROCprofiler a HIP kódokhoz –, de a CUDA ökoszisztémában elérhető Nsight Compute és Nsight Systems továbbra is a legmélyebb hardveres betekintést nyújtja. A hordozhatósági rétegek használata során is ugyanazokat a teljesítménymutatókat kell figyelni, mint a natív CUDA esetén, mert a hardveres korlátok nem változnak.

VII. RÉSZ – Mérnökség és záróprojekt

31 Profilozás, benchmarkolás és energia

31.1 Nsight Compute és a kernelmetrikák



31.1. ábra: Profilozás-optimalizálás hurka - A mérés nem egyszeri lépés: a szűk keresztmetszet azonosítása után optimalizálunk, majd újra mérünk, és a döntést a reprodukálható eredmény alapján hozzuk.

A kernel-szintű profilozás célja nem az, hogy egyetlen számot adjon a GPU „sebességéről”, hanem hogy a kernel végrehajtását olyan mérhető komponensekre bontsa, amelyekből reprodukálhatóan megállapítható, mi korlátozza a teljesítményt: a memória, a számítás, a szinkronizáció vagy a lapos párhuzamosság. Az Nsight Compute (parancssori neve `ncu`) a CUDA toolkit része, és kifejezetten kernel-szintű, hardveres számlálókon alapuló elemzést végez. Ellentétben az Nsight Systems rendszerszintű, idősoros nézetével, az Nsight Compute egyetlen kernel indítását vizsgálja, szekciókba rendezett metrikákkal, passzokba szervezett újrafuttatással, és a hardver teljesítményszámlálóit olvassa le. A mért adatok alapján eldönthető, hogy a kernel a memória- vagy a számítási korlát közelében fut-e, és hogy a foglaltság elég magas-e a késleltetés elrejtéséhez.

A profilkérés során a kernel nem egyszer fut le: az Nsight Compute a kiválasztott metrikaszakaszoknak megfelelően újrajátssza a kernel indítását. Egy passz egy vagy több hardveres számlálót gyűjt; a `--set full` például az összes elérhető szekciót bekapcsolja, ami nagyon lassú lehet. A gyakorlatban célzott szekciókat érdemes kérni, például a `SpeedOfLight`, az `Occupancy`, a `MemoryWorkloadAnalysis` és a `ComputeWorkloadAnalysis` szekciókat. A profilozás torzítja a futási időt, mert a számlálók olvasása, az esetleges órajel-szabályozás és a kernel ismételt futtatása miatt a mért idő nem azonos a normál futási idővel. Ezért a végleges futásidőt külön, CUDA eventekkel vagy `cudaEventElapsedTime` méréssel kell meghatározni, míg az Nsight Compute a szűk keresztmetszet elemzésére való. A mérési naplóban rögzíteni kell a GPU típust, a drivert, a CUDA-verziót, az Nsight Compute verzióját, a power limitet, az ECC/MIG beállítást, az órajelet és a háttérterhelést; ezek mind befolyásolják a hardveres számlálók értékét.

Az Nsight Compute parancssori használata jól scriptelhető. Egy tipikus parancs:

```
ncu --section SpeedOfLight \
    --section Occupancy \
    --section MemoryWorkloadAnalysis \
    --section ComputeWorkloadAnalysis \
    --launch-skip 3 \
    --launch-count 1 \
    --csv \
    ./saxpy
```

A `--launch-skip` kihagyja az első néhány kernelindítást, hogy a CUDA kontextus és a memóriaallokációk bejáródjanak. A `--launch-count 1` csak a következő kernelindítást profilozza. A `--csv` géppel feldolgozható kimenetet ad. A `--kernel-name` opcióval név szerint szűrhetünk, a `--metrics` opcióval pedig egyedi metrikákat kérhetünk, például `dram__bytes_read.sum,dram__bytes_write.sum,smsp__sass_thread_inst_executed_op_fadd_pred_on.s`. A metrikák és szekciók pontos neve a telepített Nsight Compute verziójától függ; a labor előtt ellenőrizni kell a `ncu --query-sections` és a `ncu --query-metrics` kimenetét.

A profilozás előtti bemelegítés és az ismételt mérés elengedhetetlen. A GPU órajele a hőmérséklettől és a teljesítménykerettől függően változik, ezért az Nsight Compute alapértelmezés szerint megpróbálja stabilizálni az órjelet, például lehűti a GPU-t, vagy `--clock-control base` móddal rögzíti az alapórajelet. Ezt a beállítást dokumentálni kell. A mérési környezet reprodukálhatóságához a következő adatokat kell rögzíteni:

```
GPU: NVIDIA A100-SXM4-40GB (compute capability 8.0)
Driver: 550.54.15
CUDA: 12.8
Nsight Compute: 2025.1.0 (ellenőrizni kell a tényleges verziót)
OS: Ubuntu 22.04.5 LTS
Power limit: 400 W
ECC: enabled
MIG: disabled
Háttérterhelés: nincs
```

Ezek az adatok helyőrzők; a laborban a saját környezetet kell rögzíteni.

A kernel-szintű profilozás három fő metrikacsoportja: occupancy, memory és compute. Az occupancy azt mutatja, hogy az SM-re ütemezett aktív warpok száma mekkora hányada az elméleti maximumnak. Az elméleti maximumot a hardver korlátai határozzák meg: a regiszterek száma, a shared memory mérete, a blokkonkénti maximális szálszám és a warp-szintű limit. Ha a kernel túl sok regisztert használ, az SM kevesebb blokkot tud ütemezni, így az aktív warpok száma csökken. A `launch_bounds` minősítővel a fordító regiszterhasználatát korlátozhatjuk, de ez regiszter spillhez vezethet, ami a lokális memória forgalmát növeli. Az Nsight Compute `Occupancy` szekciója megadja a tényleges és az elméleti foglaltságot, valamint a korlátozó tényezőt, például `Registers Per Thread` vagy `Shared Memory Per Block`. Az occupancy önmagában nem cél, hanem eszköz: a késleltetés elrejtéséhez elegendő aktív warp kell, de a túl magas occupancy nem feltétlenül javítja a teljesítményt, ha a memória sávszélessége a szűk keresztmetszet.

A memóriametrikák azt mutatják, hogy a kernel mennyire hatékonyan használja a memóriahierarchiát. A `MemoryWorkloadAnalysis` szekció olyan metrikákat ad, mint a DRAM olvasási/írási byte-ok, az L1/L2 találati arány, a memória csúcsáteresztő képességhez viszonyított kihasználtság, a globális memória hozzáférési hatékonyság (coalescing) és a shared memory bank conflict. A globális memória hozzáféréseknél a hardver 32 bájtos szektorokban mozgatja az adatot; ha egy warp tagjai szomszédos címeket érnek el, a hozzáférés coalescing hatékony. Ha a hozzáférés szórt, akkor a szektorok egy része kihasználatlan marad, és a tényleges adatforgalom megnő. A `l1tex__t_sectors_pipe_lsu_mem_global_op_ld.sum` és a `l1tex__t_sectors_pipe_lsu_mem_global_op_st.sum` metrikák a tényleges szektorforgalmat mutatják. A memóriakorlátos kernelnél a DRAM sávszélesség kihasználtsága közelíti a hardver csúcsértékét, miközben a számítási egységek kihasználtsága alacsony marad.

A compute metrikák a számítási csővezetékek kihasználtságát mutatják. A `ComputeWorkloadAnalysis` szekció megadja az SM csővezetékek (ALU, FMA, tensor core, load/store, special function unit) aktivitását, az utasítások végrehajtási statisztikáit és a warp stall okait. A `smsp__warps_issue_stalled_long_scoreboard_per_warp_active.pct` például azt mutatja, hogy a warpok hány százaléka várakozik memóriaeredményre. A

`smsp__issue_active.avg.pct_of_peak_sustained_active` az utasításkibocsátás kihasználtságát adja. Ha a kernel számítási korlátos, akkor az ALU/FMA csővezetékek kihasználtsága magas, a memóriaforgalom viszont alacsony. A stall okok elemzése megmutatja, hogy a warpok miért nem tudnak utasítást kibocsátani: memóriavárakozás, szinkronizációs barrier, függőség, vagy egyszerűen nincs aktív warp.

A Roofline-modell ezeket a metrikákat egyetlen keretbe helyezi. A modell alapja az aritmetikai intenzitás (AI), amely a végrehajtott hasznos lebegőpontos műveletek és a ténylegesen mozgatott byte-ok hányadosa. Az intenzitás mértékegysége FLOP/byte. A hardver teljesítményét két korlát határozza meg: a memória sávszélesség (B) és a számítási csúcsteljesítmény (P_max). Egy kernel elérhető teljesítménye:

$$P_{\text{elérhető}} = \min(P_{\text{max}}, B \cdot AI)$$

A Roofline-diagram log-log skálán ábrázolja ezt a függvényt: a ferde szakasz a memória-korlát, a vízszintes szakasz a számítási korlát. A töréspont (ridge point) az az intenzitás, ahol a két korlát egyenlő:

$$AI_{\text{ridge}} = \frac{P_{\text{max}}}{B}$$

Ha a kernel AI-ja a töréspont alatt van, akkor a kernel memória-korlátos, és a teljesítményt a sávszélesség határozza meg. Ha az AI a töréspont felett van, akkor a kernel számítási korlátos, és a teljesítményt a csúcsteljesítmény határozza meg. Az Nsight Compute riportjából kiszámítható mindkét összetevő: a végrehajtott FLOP-ok száma és a DRAM byte-ok száma. A ténylegesen elért teljesítmény:

$$P_{\text{mért}} = \frac{\text{FLOP}_{\text{kernel}}}{\text{idő}}$$

A ténylegesen elért sávszélesség:

$$B_{\text{mért}} = \frac{\text{DRAM byte-ok}}{\text{idő}}$$

A mért AI:

$$AI_{\text{mért}} = \frac{\text{FLOP}_{\text{kernel}}}{\text{DRAM byte-ok}}$$

A mért pontot felhelyezve a Roofline-diagramra látható, hogy a kernel mennyire közelíti meg a hardver által elérhető maximumot. Ha a pont a ferde vonal közelében van, a kernel memória-korlátos; ha a vízszintes vonal közelében, akkor számítási korlátos. Ha a pont messze van mindkét vonaltól, akkor a kernel nem használja ki sem a memória sávszélességet, sem a számítási egységeket; ilyenkor occupancy-probléma, rossz hozzáférési minta vagy szinkronizációs veszteség állhat a háttérben.

Egy egyszerű SAXPY kernel kiválóan szemlélteti a kernel-szintű profilozást és a Roofline-elemzést. A kernel minden bemeneti elempárra egy szorzást és egy összeadást végez, így a memóriaforgalom dominál. A forrás:

```
#include <cstdio>
#include <cuda_runtime.h>

#define CHECK_CUDA(call)                                                       \
  do {                                                                         \
    cudaError_t err = call;                                                    \
    if (err != cudaSuccess) {                                                  \
      std::fprintf(stderr, "CUDA error at %s:%d: %s\n", __FILE__, __LINE__, \
                   cudaGetErrorString(err));                                   \
      std::exit(EXIT_FAILURE);                                                   \
    }                                                                          \
  } while (0)

__global__ void saxpy_kernel(const float* __restrict__ x,
                             const float* __restrict__ y,
                             float a, float* __restrict__ out,
                             int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        out[idx] = a * x[idx] + y[idx];
    }
}

int main(int argc, char** argv) {
    int n = (argc > 1) ? std::atoi(argv[1]) : 1 << 20;
    std::size_t bytes = n * sizeof(float);
    float *d_x, *d_y, *d_out;

    CHECK_CUDA(cudaMalloc(&d_x, bytes));
    CHECK_CUDA(cudaMalloc(&d_y, bytes));
    CHECK_CUDA(cudaMalloc(&d_out, bytes));

    float* h_x = new float[n];
    float* h_y = new float[n];
    for (int i = 0; i < n; ++i) {
        h_x[i] = 1.0f;
        h_y[i] = 2.0f;
    }

    CHECK_CUDA(cudaMemcpy(d_x, h_x, bytes, cudaMemcpyHostToDevice));
    CHECK_CUDA(cudaMemcpy(d_y, h_y, bytes, cudaMemcpyHostToDevice));

    int block = 256;
    int grid = (n + block - 1) / block;

    cudaEvent_t start, stop;
    CHECK_CUDA(cudaEventCreate(&start));
    CHECK_CUDA(cudaEventCreate(&stop));

    CHECK_CUDA(cudaEventRecord(start));
    saxpy_kernel<<<grid, block>>>(d_x, d_y, 2.0f, d_out, n);
    CHECK_CUDA(cudaGetLastError());
    CHECK_CUDA(cudaEventRecord(stop));
    CHECK_CUDA(cudaEventSynchronize(stop));

    float elapsed_ms = 0.0f;
    CHECK_CUDA(cudaEventElapsedTime(&elapsed_ms, start, stop));
    std::printf("SAXPY n=%d time=%.3f ms\n", n, elapsed_ms);

    CHECK_CUDA(cudaMemcpy(h_y, d_out, bytes, cudaMemcpyDeviceToHost));
    // Ellenőrzés: minden elem 4.0
    bool ok = true;
    for (int i = 0; i < n; ++i) {
```

```
        if (h_y[i] != 4.0f) { ok = false; break; }
    }
    std::printf("Verification: %s\n", ok ? "OK" : "FAILED");

    CHECK_CUDA(cudaFree(d_x));
    CHECK_CUDA(cudaFree(d_y));
    CHECK_CUDA(cudaFree(d_out));
    delete[] h_x;
    delete[] h_y;
    return 0;
}
```

A kód nyelvjelölése CUDA C++. A build és a futtatás:

```
nvcc -O3 -arch=sm_80 -o saxpy saxpy.cu
./saxpy 1048576
```

A profilozás parancsa a gyakorlatban:

```
ncu --section SpeedOfLight \
    --section Occupancy \
    --section MemoryWorkloadAnalysis \
    --section ComputeWorkloadAnalysis \
    --launch-skip 2 \
    --launch-count 1 \
    --csv \
    ./saxpy 1048576
```

A `--launch-skip 2` kihagyja az első kernelindítást, mert a fenti kód csak egyszer indítja a kernelt, ezért itt a `--launch-skip 0` szükséges, vagy egyszerűen hagyjuk el. A laborban érdemes egy olyan harnesset írni, amely többször indítja a kernelt, és a profilozást a 3.–5. indításra irányítani. A fenti példában a `--launch-skip 0` a helyes.

A mérési naplóban rögzíteni kell a fenti környezeti adatokat, valamint a kernel saját mérését (a `cudaEventElapsedTime` által adott időt), majd az Nsight Compute riportjának kivonatát. Egy szemléltető riportkivonat (nem hitelesített mérés, a laborban a hallgató saját értékeit kell bemutatnia):

```
Kernel: saxpy_kernel
Grid: 4096 blocks x 256 threads
Duration: 18.4 us (profilozott, Nsight Compute)
Registers per thread: 16
Shared Memory per Block: 0
Achieved Occupancy: 90.3%
Theoretical Occupancy: 100% (regiszterek alapján)
DRAM Throughput: 88.7%
Memory Throughput: 65.2%
Compute (SM) Throughput: 12.1%
L1/TEX Hit Rate: 0%
L2 Hit Rate: 0%
DRAM Read Bytes: 8388608
DRAM Write Bytes: 4194304
Elapsed Cycles: 25980
SM Frequency: 1410 MHz
```

Ezek az értékek a SAXPY kernel tipikus képét mutatják: a memória sávszélesség kihasználtsága magas, a számítási csővezetékek kihasználtsága alacsony, az L1/L2 találati arány nulla, mert a beolvasott adatok egyszer használtak, és nincs újrafelhasználás. A DRAM olvasási byte 8 MiB (két bemeneti tömb), az írási 4 MiB (egy kimeneti tömb). A kernel által végzett hasznos FLOP $2N = 2 * 1\,048\,576 = 2\,097\,152$. Az AI:

$$AI = \frac{2\,097\,152 \text{ FLOP}}{12\,582\,912 \text{ byte}} \approx 0{,}167 \text{ FLOP/byte}$$

Egy A100 GPU esetén a memória csúcssávszélesség körülbelül 1555 GB/s, a számítási csúcs (FP32) körülbelül 19,5 TFLOPS (a pontos érték a specifikációtól függ, ellenőrizni kell). A töréspont:

$$AI_{\text{ridge}} = \frac{19{,}5 \times 10^{12} \text{ FLOP/s}}{1555 \times 10^{9} \text{ byte/s}} \approx 12{,}5 \text{ FLOP/byte}$$

A SAXPY AI-ja ennél sokkal alacsonyabb, tehát a kernel egyértelműen memória-korlátos. A mért DRAM sávszélesség a riportból: a profilkérés során mért idő 18,4 µs, így a sávszélesség:

$$B_{\text{mért}} = \frac{12\,582\,912 \text{ byte}}{18{,}4 \times 10^{-6} \text{ s}} \approx 684 \text{ GB/s}$$

Ez az A100 elméleti csúcsának töredéke, de a profilozott idő torzít; a normál futásban a CUDA eventes mérés ennél jobb értéket adhat. A laborban a hallgató feladata, hogy a saját méréséből számolja ki a tényleges sávszélességet, és hasonlítsa össze a várt maximummal.

A tipikus hibák és a helyes megoldások:

1. **Hideg indítás profilozása.** Ha az első kernelindítást profilozzuk, a CUDA kontextus inicializálása, a modul betöltése és a memóriaallokációk torzítják a mérést. Helyes megoldás: melegítő kernelindítások, `--launch-skip` vagy a kernel többszöri futtatása a profilozás előtt.

2. **A `--set full` indokolatlan használata.** A teljes metrikakészlet minden kernelindításnál sok passzt igényel, ami lassú és a mérési zajt növeli. Helyes megoldás: először a `SpeedOfLight` szekcióval meghatározni a korlátot, majd a megfelelő további szekciókat (Occupancy, MemoryWorkloadAnalysis vagy ComputeWorkloadAnalysis) célzottan lekérni.

3. **Az occupancy egyetlen számra redukálása.** A magas occupancy önmagában nem garantál gyors futást, és az alacsony occupancy nem feltétlenül hiba. Helyes megoldás: az occupancyt a memória- és számítási metrikákkal együtt értelmezni, és a stall okokat is elemezni.

4. **A Roofline-pont rossz kiszámítása.** Gyakori hiba, hogy a névleges csúcsértékekkel osztanak, nem a ténylegesen mért DRAM byte-okkal, vagy a FLOP-okba beleszámolják a redundáns műveleteket is. Helyes megoldás: az Nsight Compute riportjából kinyert `dram__bytes_read.sum` és `dram__bytes_write.sum` értékeket használni, és a kernel által ténylegesen elvégzett, hasznos lebegőpontos műveleteket számolni.

5. **A profilozott idő és a normál futásidő összekeverése.** Az Nsight Compute a kernel többszöri újrafuttatásával, órajel-stabilizálással és számláló-olvasással torzítja az időt. Helyes megoldás: a futásidőt külön, CUDA eventekkel mérni, és az Nsight Compute-ot csak a szűk keresztmetszet elemzésére használni.

6. **A mérési környezet nem dokumentált.** Ha a power limit, az ECC, a MIG vagy a háttérterhelés nem rögzített, a metrikák nem reprodukálhatók. Helyes megoldás: a mérési napló a GPU típusát, a drivert, a CUDA-verziót, az Nsight Compute verzióját, az órajelet, a power limitet, az ECC/MIG beállítást és a háttérterhelést tartalmazza.

7. **Az OpenCL/JAX eszközök metrikáinak automatikus átültetése.** Az Nsight Compute CUDA kernelekhez készült; OpenCL vagy JAX esetén más profilozó eszközök (például a gyártó által biztosított profiler, roofline-elemző) szükségesek. Helyes megoldás: a laborban CUDA C++ kernelt használni az Nsight Compute-hoz, és a más nyelveknél külön jelezni, hogy a metrika nem közvetlenül összehasonlítható.

A labor feladata: a hallgató írja meg a SAXPY kernelt (vagy egy hasonló, egyszerű, ellenőrizhető kernelt), futtassa le az Nsight Compute-ot a fenti szekciókkal, gyűjtse ki a metrikákat, számolja ki az AI-t, és helyezze el a pontot egy Roofline-diagramon. A jelentésben különítse el a mért és a várt eredményeket: a várt eredmény a hardver specifikációjából számított Roofline-görbe, a mért eredmény a kernel ténylegesen elért sávszélessége és FLOP/s értéke. A dokumentációban adja meg a mérési naplót a fenti környezeti adatokkal, a build- és futtatási parancsokkal, valamint a Nsight Compute riport kivonatával. A jelentésnek tartalmaznia kell egy rövid elemzést arról, hogy a kernel memória- vagy számítási korlátos-e, és hogy az occupancy elegendő-e a késleltetés elrejtéséhez.

Önellenőrző kérdés: Egy mátrixszorzó kernel (single precision, tiled, shared memory nélkül) M=N=K=1024 esetén $2MNK$ *FLOP-ot végez, és a DRAM forgalom megközelítőleg* $4(MK + KN)$ byte (a kimenet írását most ne számoljuk). Számítsa ki az aritmetikai intenzitást. Egy A100 GPU esetén (B 1555 GB/s, FP32 csúcs 19,5 TFLOPS) a kernel memória- vagy számítási korlátos? Hogyan változik a helyzet, ha a mátrixok mérete N=64-re csökken?

A kérdés megválaszolásához a hallgatónak fel kell ismernie, hogy az AI = (2*1024^3) / (4(1024*1024 + *1024*1024)) 1024/4 = 256 FLOP/byte, ami messze a töréspont felett van, tehát számítási korlátos. N=64 esetén az AI = 64/4 = 16 FLOP/byte, ami a töréspont közelében van, és a kernel inkább memória-korlátossá válik. Ez a gyakorlat a Roofline-modell gyakorlati alkalmazását erősíti meg.

A fejezet továbbvezető eredménye, hogy a kernel-szintű metrikák ismerete nélkülözhetetlen a rendszerszintű profilozáshoz, amelyet a következő alfejezet, az Nsight Systems tárgyal. A kernel-profilozás megmutatja, mi a korlát egy kernelben, a rendszerprofilozás pedig azt, hogy a kernelek, memóriaműveletek és szinkronizációk hogyan viszonyulnak egymáshoz a teljes alkalmazásban. A két nézet együtt ad teljes képet a GPU-s alkalmazás teljesítményprofiljáról.

31.2 Nsight Systems és a rendszerszintű profil

Az Nsight Systems a teljes alkalmazás idővonalát vizsgálja: nem egyetlen kernel belső metrikáira fókuszál, hanem arra, hogy a CPU-szálak, a CUDA API-hívások, a memóriaműveletek és a GPU-n futó kernelek hogyan illeszkednek egymáshoz az időben. Az előző egységben (Nsight Compute és a kernelmetrikák) az Nsight Compute segítségével egy-egy kernel számítási és memória-teljesítményét elemeztük. Most kilépünk a kernel szintjéről, és azt kérdezzük: a teljes programunk jól használja-e ki a GPU és a CPU közötti párhuzamosságot? Hol vannak olyan szinkronizációs pontok, amelyek megakadályozzák az adatmozgatás és a számítás átfedését? Az Nsight Systems pontosan ezeket a kérdéseket teszi megválaszolhatóvá.

31.2.1 A teljes alkalmazás idővonala

Az Nsight Systems egy rendszerszintű profilozó eszköz, amely rögzíti a CPU-szálak tevékenységét, a CUDA API-hívásokat, a kernelek indítását, a memóriaműveleteket, a szinkronizációs hívásokat, valamint az egyéb futási idejű eseményeket. Az eredmény egy idővonalon jelenik meg, amelyen

függőleges sávok jelzik az egyes erőforrások állapotát. A fő nézetben általában a következő sorokat látjuk:

- **CPU szálak**: minden operációs rendszer szál külön sávként jelenik meg, láthatóvá téve, hogy a CPU mikor dolgozik, mikor várakozik, és mikor hajt végre CUDA API-hívást.
- **CUDA API**: a CPU oldalán kiadott hívások, mint például `cudaMemcpyAsync`, `cudaLaunchKernel`, `cudaStreamSynchronize`, `cudaDeviceSynchronize`. Ezeknek a hívásoknak a hossza azt mutatja, mennyi időt tölt a CPU a hívás feldolgozásával.
- **Kernek és memóriaműveletek**: a GPU-n futó kernelek és a másolási műveletek (H2D, D2H) külön sávban szerepelnek. Itt látható, hogy a GPU mikor aktív, és hogy a különböző streamek műveletei átfedésben vannak-e.
- **Szinkronizációs pontok**: a CPU és GPU közötti várakozások, például amikor a CPU `cudaStreamSynchronize` hívása blokkol, amíg a stream be nem fejeződik.

Az idővonal elemzésekor a legfontosabb mintázat a **buborékok** (üres idők) jelenléte. Ha a GPU kihasználtsága alacsony, gyakran azért van, mert a CPU nem tudja elég gyorsan kiadni a következő műveletet, vagy mert egy szinkronizációs hívás megvár egy korábbi műveletet, mielőtt újabbakat indíthatna. Az Nsight Systems segítségével ezeket a buborékokat pontosan lokalizálhatjuk.

31.2.2 CUDA API és kernel az idővonalon

A CUDA programok futtatása során a CPU a CUDA API-n keresztül vezérli a GPU-t. A modern CUDA futási modellben a legtöbb API-hívás **aszinkron**: a CPU csupán bejegyzi a műveletet egy streambe, majd azonnal visszatér. A GPU később hajtja végre a bejegyzett műveleteket. Ez teszi lehetővé, hogy a CPU előre dolgozzon, miközben a GPU még a korábbi feladatokat végzi. Az Nsight Systems idővonalán ezt úgy látjuk, hogy a CPU API-hívásai rövidek, és a GPU oldalán a kernelek később kezdődnek, esetleg átfedésben más streamek műveleteivel.

A CUDA API-hívások két nagy csoportra oszthatók:

- **Vezérlő hívások**: `cudaLaunchKernel`, `cudaMemcpyAsync`, `cudaMemsetAsync`. Ezek tipikusan gyorsak, és a CPU nem blokkol rajtuk, kivéve, ha a stream tele van, vagy ha a hívás szinkronizációs okból várakozik.
- **Szinkronizációs hívások**: `cudaDeviceSynchronize`, `cudaStreamSynchronize`, `cudaMemcpy` (a szinkron változat), `cudaMalloc` (ha nincs cached allokáció). Ezek a CPU-t blokkolják, amíg a megfelelő GPU-munka be nem fejeződik.

Az Nsight Systems-ben a CUDA API-hívásokhoz tartozó sávok színkódoltak. A leggyakoribb színek és jelentésük (a gyártó dokumentációja szerint, de a pontos színkódolás a verziótól függ) a következőképpen foglalható össze:

- **Kék**: memóriamásolás (H2D vagy D2H)
- **Zöld**: kernel-végrehajtás
- **Sárga**: szinkronizáció vagy API-hívás
- **Piros/szürke**: blokkolt vagy várakozó állapot

Amikor egy kernel sávja zölddel jelenik meg, az azt jelenti, hogy a GPU adott időpontban ténylegesen végrehajtja a kernelt. Ha a kernel sávja rövidebb, mint a hozzá tartozó API-hívás időtartama, az azt jelezheti, hogy a CPU lassan adta ki a műveletet. Az idővonalon a CPU szál sávjában a kernel indításához tartozó `cudaLaunchKernel` hívás ideje alatt a CPU a kernel paramétereit csomagolja, és a parancsot a streambe helyezi. Ez általában nagyon rövid, de ha túl sok kis kernelünk van, akkor a CPU oldali overhead jelentőssé válhat.

A gyakorlatban az Nsight Systems-ben a következő kérdéseket tesszük fel:

1. Van-e átfedés a H2D másolás, a kernel és a D2H másolás között különböző streamekben?
2. A CPU kihasználtsága elég magas-e ahhoz, hogy ellássa a GPU-t munkával?
3. Hol jelennek meg hosszú blokkoló hívások, amelyek megszakítják a folyamatos végrehajtást?

31.2.3 CPU-GPU szinkronizáció

A CPU és a GPU közötti szinkronizáció az egyik leggyakoribb oka a teljesítménycsökkenésnek. A GPU kihasználtsága akkor optimális, ha folyamatosan van végrehajtandó munkája. Ha a CPU szinkronizál, akkor a GPU-nak várnia kell, amíg a CPU újabb parancsot ad, vagy fordítva: a CPU várakozik a GPU befejezésére.

A CUDA-ban a következő szinkronizációs mechanizmusok léteznek:

- `cudaDeviceSynchronize()`: megvárja, amíg az összes stream minden művelete befejeződik. Ez a legerősebb szinkronizáció, és gyakran teljesen megszünteti az átfedést, mert a CPU addig nem ad ki új műveletet, amíg minden korábbi be nem fejeződött.
- `cudaStreamSynchronize(stream)`: megvárja egy adott stream összes műveletét. Ez kevésbé káros, mint a teljes eszköz szinkronizálása, de még mindig blokkolja a CPU-t az adott stream befejezéséig.
- **`cudaMemcpy` (szinkron változat)**: a memóriamásolás alapértelmezett formája szinkron, ami azt jelenti, hogy a hívás blokkol, amíg a másolás be nem fejeződik, és implicit módon az összes korábbi műveletet is szinkronizálja az adott streamre vonatkozóan (pontosabban: a szinkron `cudaMemcpy` a default streamet használja, amely szinkronizál a többi streammel). Ez nagyon gyakori hiba, amikor a fejlesztő `cudaMemcpy`-t használ aszinkron másolásra szánt kódban.
- **Események (`cudaEvent_t`)**: az eseményekkel finomabban vezérelhető a szinkronizáció. A `cudaEventRecord` bejegyzi az eseményt a streambe, a `cudaEventSynchronize` pedig megvárja az adott eseményt. Az események lehetővé teszik más streamek közötti függőségek létrehozását anélkül, hogy a teljes eszközt blokkolnánk.

Az Nsight Systems idővonalán a szinkronizációs hívások sárga sávként jelennek meg a CPU szálon, és gyakran látható, hogy a GPU eközben üresjáratban van. Ha a GPU sávjában hosszú üres szakaszok vannak, akkor érdemes megkeresni az őket megelőző CPU-oldali szinkronizációs hívást.

Fontos megjegyezni, hogy a **default stream** (0-s stream) speciális viselkedésű: a default stream szinkronizál a többi streammel, ha azok nem non-blocking streamként lettek létrehozva (`cudaStreamCreateWithFlags(..., cudaStreamNonBlocking)`). Ez azt jelenti, hogy ha egy default streamen kiadott művelet előtt más streamekben van függőben lévő munka, akkor a default stream megvárja azok befejezését. Ez gyakran rejtett szinkronizációhoz vezet, amit az Nsight Systems jól láthatóvá tesz. A modern CUDA ajánlása szerint minden műveletet explicit streamekben kell végrehajtani, és kerülni kell a default stream használatát, ha átfedést akarunk.

31.2.4 Stream- és copy-átfedés elmélete

A GPU és a CPU közötti adatmozgatás (H2D és D2H) a PCIe buszon keresztül történik. Ez a busz sávszélessége korlátozott, de a másolási műveletek **aszinkron módon** végrehajthatók, vagyis a GPU képes egyszerre fogadni az adatokat és közben számítási feladatokat végezni. Ezt a képességet **copy és compute overlap**-nek nevezzük. A gyakorlatban a következő feltételeknek kell teljesülniük:

1. A másolási műveleteknek **aszinkronnak** kell lenniük (`cudaMemcpyAsync`).

2. A másolásnak és a kernelnek **különböző streamekben** kell futniuk, vagy legalábbis a műveletek sorrendjét úgy kell kialakítani, hogy ne legyen köztük szinkronizációs függőség.
3. A GPU és a CPU között a memóriaműveleteknek **nem szabad blokkolniuk** egymást, és a hardvernek támogatnia kell a párhuzamos másolást és számítást (a legtöbb modern GPU esetében ez adott).

Egy tipikus minta, amely kihasználja az átfedést, a **több streames feldolgozás**:

1. Az adatot több részre osztjuk.
2. Minden streamhez hozzárendelünk egy-egy részt.
3. Minden streamben sorban végrehajtjuk:
   - H2D másolás a készülékre (`cudaMemcpyAsync`),
   - kernel indítása a rész feldolgozására,
   - D2H másolás az eredmény visszaküldésére.
4. Az egyes streamek műveletei átfedhetik egymást: amíg az egyik stream kernelje fut, a másik stream H2D másolása a PCIe buszon dolgozik.

Ha a streameket helyesen állítjuk be, az idővonalon azt látjuk, hogy a zöld kernel sávok és a kék másolási sávok egymás mellett, időben átfedve jelennek meg. A teljes futási idő így jelentősen csökkenhet a szekvenciális végrehajtáshoz képest.

31.2.5 Példaprogram: több stream és aszinkron másolás

A következő CUDA C++ példa azt mutatja be, hogyan lehet négy stream segítségével átfedni a másolást és a kernel-végrehajtást. A kernel egy egyszerű vektor-skálázást végez, de a hangsúly a stream-kezelésen van.

```
// File: stream_overlap.cu
// Fordítás: nvcc -std=c++17 -O3 stream_overlap.cu -o stream_overlap
// Futtatás: ./stream_overlap
// Környezet: CUDA 12.3, NVIDIA A100 (compute capability 8.0), driver 545.23.08,
//            Ubuntu 22.04, nvidia-smi: power limit 400 W, ECC off, MIG off.

#include <cuda_runtime.h>
#include <iostream>
#include <vector>
#include <chrono>
#include <cassert>

// Hibakezelő makró
#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err = (call);                                           \
        if (err != cudaSuccess) {                                           \
            std::cerr << "CUDA error at " << __FILE__ << ":" << __LINE__    \
                      << " - " << cudaGetErrorString(err) << std::endl;     \
            std::exit(EXIT_FAILURE);                                        \
        }                                                                   \
    } while (0)

__global__ void scale_kernel(const float* __restrict__ input,
                             float* __restrict__ output,
                             float scale, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        output[idx] = input[idx] * scale;
    }
```

```
}

int main() {
    constexpr int num_streams = 4;
    constexpr int chunk_size = 1 << 20;        // 1 Mi elemenként
    constexpr int total_size = num_streams * chunk_size;
    constexpr size_t bytes_per_chunk = chunk_size * sizeof(float);

    // Host memória (pageable, de a példa egyszerűsége kedvéért)
    std::vector<float> h_input(total_size, 1.0f);
    std::vector<float> h_output(total_size, 0.0f);
    float scale = 2.0f;

    // Device memória
    float *d_input, *d_output;
    CUDA_CHECK(cudaMalloc(&d_input, total_size * sizeof(float)));
    CUDA_CHECK(cudaMalloc(&d_output, total_size * sizeof(float)));

    // Stream-ek létrehozása non-blocking flaggel
    cudaStream_t streams[num_streams];
    for (int i = 0; i < num_streams; ++i) {
        CUDA_CHECK(cudaStreamCreateWithFlags(&streams[i], cudaStreamNonBlocking));
    }

    // Időmérés
    cudaEvent_t start, stop;
    CUDA_CHECK(cudaEventCreate(&start));
    CUDA_CHECK(cudaEventCreate(&stop));
    CUDA_CHECK(cudaEventRecord(start, 0));

    // Minden stream feldolgozza a saját szeletét
    for (int i = 0; i < num_streams; ++i) {
        // H2D másolás aszinkron
        CUDA_CHECK(cudaMemcpyAsync(d_input + i * chunk_size,
                                   h_input.data() + i * chunk_size,
                                   bytes_per_chunk,
                                   cudaMemcpyHostToDevice,
                                   streams[i]));

        // Kernel indítás
        int threads = 256;
        int blocks = (chunk_size + threads - 1) / threads;
        scale_kernel<<<blocks, threads, 0, streams[i]>>>(
            d_input + i * chunk_size,
            d_output + i * chunk_size,
            scale,
            chunk_size);

        // D2H másolás aszinkron
        CUDA_CHECK(cudaMemcpyAsync(h_output.data() + i * chunk_size,
                                   d_output + i * chunk_size,
                                   bytes_per_chunk,
                                   cudaMemcpyDeviceToHost,
                                   streams[i]));
    }

    // Várakozás minden stream befejezésére
    for (int i = 0; i < num_streams; ++i) {
        CUDA_CHECK(cudaStreamSynchronize(streams[i]));
    }

    CUDA_CHECK(cudaEventRecord(stop, 0));
    CUDA_CHECK(cudaEventSynchronize(stop));
```

```
    float elapsed_ms = 0.0f;
    CUDA_CHECK(cudaEventElapsedTime(&elapsed_ms, start, stop));

    // Ellenőrzés
    for (int i = 0; i < total_size; ++i) {
        assert(h_output[i] == 2.0f);
    }

    std::cout << "Total time with " << num_streams << " streams: "
              << elapsed_ms << " ms" << std::endl;

    // Takarítás
    for (int i = 0; i < num_streams; ++i) {
        CUDA_CHECK(cudaStreamDestroy(streams[i]));
    }
    CUDA_CHECK(cudaFree(d_input));
    CUDA_CHECK(cudaFree(d_output));
    CUDA_CHECK(cudaEventDestroy(start));
    CUDA_CHECK(cudaEventDestroy(stop));

    return 0;
}
```

A fenti kód figyelmet fordít a hibakezelésre, és a streameket `cudaStreamNonBlocking` flaggel hozza létre, hogy elkerülje a default stream miatti implicit szinkronizációt. A `cudaMemcpyAsync` hívások mindegyike a megfelelő streamhez van rendelve, így a másolások párhuzamosan futhatnak a kernelekkel. A kernel indítások is ugyanabban a streamben vannak, így a streamen belül a sorrend garantált: H2D, majd kernel, majd D2H.

31.2.6 Labor: stream- és copy-átfedés elemzése Nsight Systems-szel

A labor célja, hogy a fenti kódot (vagy egy saját változatot) lefuttassuk, és az Nsight Systems segítségével megvizsgáljuk az idővonalat. Az alábbiakban megadjuk a szükséges parancsokat és az elemzési szempontokat. A labor a **nem futtatott** kategóriába tartozik ebben a szövegben: a környezet és a parancsok valósak, de a konkrét mérési eredményeket a felhasználónak kell kitöltenie.

31.2.7 Környezet

- GPU: NVIDIA A100 40 GB PCIe (vagy az olvasó által elérhető GPU)
- Compute capability: 8.0
- Driver: 545.23.08
- CUDA Toolkit: 12.3
- Operációs rendszer: Ubuntu 22.04 LTS
- Nsight Systems verzió: 2023.4.1 (ellenőrizendő a `nsys --version` paranccsal)
- Power limit, ECC, MIG stb.: rögzítendő a `nvidia-smi` kimenetéből

31.2.8 Fordítás és futtatás

A kód fordításához és futtatásához használjuk a következő parancsokat:

```
nvcc -std=c++17 -O3 stream_overlap.cu -o stream_overlap
nsys profile --trace=cuda,nvtx,osrt --output=stream_overlap_report ./stream_overlap
```

A `--trace=cuda,nvtx,osrt` opciók a CUDA API, a kernelek, a memóriaműveletek és az operációs rendszer futási idejű eseményeit rögzítik. A `--output` határozza meg a jelentésfájl nevét.

A futtatás után az Nsight Systems GUI-ban (vagy a parancssori `nsys stats` eszközzel) megnyithatjuk a `stream_overlap_report.nsys-rep` fájlt. A parancssori statisztikához:

```
nsys stats stream_overlap_report.nsys-rep
```

A kimenet tartalmazza a teljes futási időt, a GPU kihasználtságát, az API-hívások számát és a szinkronizációs pontokat.

31.2.9 Mit keressünk az idővonalon?

1. **Átfedés a streamek között**: a négy stream H2D másolásai, kerneljei és D2H másolásai egymás után, de átfedésben jelennek meg. Ideális esetben a teljes GPU-idő rövidebb, mint a szekvenciális végrehajtás összege.
2. **CPU blokkoló hívások**: a `cudaStreamSynchronize` hívások a CPU szálon sárga sávként jelennek meg. Azt várjuk, hogy ezek a hívások a feldolgozás végén, rövid ideig tartanak, mert a streamek már majdnem befejeződtek.
3. **CPU oldali üresjárat**: ha a CPU szál sávjában hosszú üres szakaszok vannak, az azt jelzi, hogy a CPU nem tud elég gyorsan munkát adni a GPU-nak. Ezt a stream-szám növelésével vagy a chunk-méret csökkentésével lehet orvosolni.
4. **GPU kihasználtság**: a GPU sávjában a zöld és kék részek aránya a teljes időhöz képest. A cél az, hogy a GPU minél kevesebb időt töltsön üresen.

31.2.10 Várt eredmények

A várt eredmények a következők (a konkrét számok a hardvertől függenek, de a tendencia egyértelmű):

- A négy stream együttes futási ideje **kevesebb**, mint a szekvenciális feldolgozás idejének körülbelül 75%-a, mivel a másolások és a kernelek átfedik egymást.
- Az Nsight Systems idővonalán a H2D másolások és a D2H másolások kék sávjai **részben vagy teljesen átfednek** a kernelek zöld sávjaival.
- A CPU szálon a `cudaStreamSynchronize` hívások csak a végén jelennek meg, és nem szakítják meg a folyamatos műveletkibocsátást.

Ezek a várt eredmények elméleti alapon és a dokumentáció ismeretében megfogalmazhatók, de a tényleges méréshez a fenti környezetben kell futtatni a kódot. A mérési naplóban a **mért** értékeket a felhasználónak kell rögzítenie.

31.2.11 Mérési napló sablon

Metrika	Várt érték	Mért érték
Teljes futási idő (4 stream)	$<$ szekvenciális idő	[kitöltendő]
GPU kihasználtság	$> 70\%$	[kitöltendő]
Átfedés mértéke (másolás + kernel)	részleges/teljes	[kitöltendő]
CPU blokkoló idő	$< 5\%$	[kitöltendő]
H2D másolás átlagos ideje	$\sim$ X s	[kitöltendő]
Kernel átlagos ideje	$\sim$ Y s	[kitöltendő]

Metrika	Várt érték	Mért érték
D2H másolás átlagos ideje	~ Z s	[kitöltendő]

A várt oszlopban szereplő irányelvek segítenek az anomáliák felismerésében.

31.2.12 Tipikus hibák és helyes megoldások

Az Nsight Systems elemzése során a következő gyakori hibákkal találkozhatunk:

31.2.13 1. Szinkron másolás használata aszinkron helyett

Hiba: A fejlesztő `cudaMemcpy`-t használ a `cudaMemcpyAsync` helyett. A szinkron másolás blokkolja a CPU-t, és implicit módon szinkronizál a default streamen keresztül, ami megszünteti az átfedést.

Jel az idővonalon: Hosszú sárga sáv a CPU szálon, a GPU pedig várakozik. A kernelek és másolások szekvenciálisan követik egymást.

Helyes megoldás: Mindig `cudaMemcpyAsync`-t használjunk, és explicit streameket adjunk meg. A másolás irányától függően a `cudaMemcpyHostToDevice` vagy `cudaMemcpyDeviceToHost` opciót válasszuk.

31.2.14 2. Default stream használata, amely szinkronizál más streamekkel

Hiba: A streameket `cudaStreamCreate`-tel hozzuk létre (default flags), és a default stream (0-s) is használatban van. A default stream szinkronizál más streamekkel, ami váratlan blokkoláshoz vezet.

Jel: Az idővonalon a default streamen futó művelet előtt a GPU várakozik, pedig a többi stream független munkát végezhetne.

Helyes megoldás: Használjunk `cudaStreamCreateWithFlags(..., cudaStreamNonBlocking)`-et, és kerüljük a default stream használatát minden műveletnél.

31.2.15 3. Túl kevés stream vagy rossz chunk-méret

Hiba: Csak egy stream van, vagy a chunk-méret túl nagy, így a másolások és kernelek nem tudnak átfedni.

Jel: A GPU kihasználtsága alacsony, a sávokban nagy üres részek vannak.

Helyes megoldás: Növeljük a streamek számát (általában 4–8 jó kiindulás), és válasszunk olyan chunk-méretet, amely elég nagy a kernel számítási igényéhez, de elég kicsi ahhoz, hogy a másolások párhuzamosan tudjanak futni. Az optimális érték kísérletezéssel és az idővonal elemzésével határozható meg.

31.2.16 4. CPU oldali overhead: túl sok kis kernel

Hiba: A program sok apró kernelt indít, és a CPU nem tudja őket elég gyorsan kiadni. A GPU üresen vár a következő kernelre.

Jel: Az idővonalon a CPU szálon a kernel indítási hívások hosszabbak, mint maguk a kernelek. A GPU sávban rövid zöld szakaszok között nagy üres rések vannak.

Helyes megoldás: Csökkentsük a kernel indítások számát, fuzionáljuk a kerneleket, vagy használjunk nagyobb blokkméretet, hogy kevesebb kernelre legyen szükség. A grid-stride loop is segíthet.

31.2.17 5. Szinkronizáció a cikluson belül

Hiba: A fejlesztő minden iterációban meghívja a `cudaDeviceSynchronize`-et, ami teljesen szétzilálja a pipeline-t.

Jel: A GPU sávban minden iteráció után hosszú üres szakasz, a CPU szálon pedig hosszú blokkoló hívások.

Helyes megoldás: Csak akkor szinkronizáljunk, ha feltétlenül szükséges, például a feldolgozás végén vagy amikor az adatokra a CPU-nak szüksége van. A köztes szinkronizációkat kerüljük, vagy használjunk eseményeket a finomabb vezérléshez.

31.2.18 Önellenőrző kérdés

Az alábbi kérdéssel ellenőrizheti, hogy megértette-e az Nsight Systems alapvető használatát.

Kérdés: Tegyük fel, hogy egy program négy streamben hajt végre H2D másolást, kernelt és D2H másolást. Az Nsight Systems idővonalán azt látjuk, hogy a kernelek zöld sávjai egymás után, átfedés nélkül jelennek meg, és a kék másolási sávok is szekvenciálisak. Mi lehet a két legvalószínűbb oka, és hogyan javítaná?

Válasz: A két leggyakoribb ok: (1) a streamek nem non-blocking módban lettek létrehozva, így a default stream szinkronizálja őket; (2) a másolásokhoz szinkron `cudaMemcpy`-t használtak `cudaMemcpyAsync` helyett. A javítás: hozzuk létre a streameket `cudaStreamNonBlocking` flaggel, és minden másoláshoz használjunk `cudaMemcpyAsync`-t. Ezen kívül ellenőrizzük, hogy a kernel indítások és másolások ugyanabba a streamben vannak-e, és ne legyen köztük implicit szinkronizáció.

31.2.19 Összefoglalás

Az Nsight Systems a rendszerszintű profilozás alapvető eszköze. Lehetővé teszi a teljes alkalmazás idővonalának elemzését, feltárva a CPU és GPU közötti együttműködés hiányosságait. A CUDA API-hívások, a kernelek és a memóriaműveletek időbeli elrendeződése megmutatja, hol vannak szinkronizációs pontok, buborékok és elveszett átfedési lehetőségek. A stream- és copy-átfedés laboratóriumi elemzése során a fejlesztő megtanulja felismerni a helyes stream-kezelést, és elkerülni a gyakori hibákat, mint a szinkron másolás vagy a default stream szinkronizációs hatása. A következő egység (Benchmarkmódszertan) erre az alapra építve tárgyalja a benchmarkmódszertant, ahol a mért eredmények statisztikai elemzése és a reprodukálhatóság kerül előtérbe.

31.3 Benchmarkmódszertan

Egyetlen GPU-kernel futásidejét kétszer megmérve gyakran két különböző számot kapunk. A kérdés nem az, hogy melyik az „igazi” érték, hanem hogy milyen feltételek mellett lesz a mérés megismételhető, és milyen tartományban mozog a várható ingadozás. A reprodukálható GPU-mérés ezért nem egyszerűen időzítés, hanem kontrollált kísérlet: rögzített környezet, ismert bemenet, explicit mérési protokoll, valamint statisztikailag értelmezhető kimenet. Ha ezek hiányoznak, a „gyorsabb” állítás csak egyetlen futás anekdotája marad.

31.3.1 A reprodukálható GPU-mérés fogalma

A reprodukálhatóság itt három, egymásra épülő szintet jelent. Az **ismételhetőség** (repeatability) azt követeli meg, hogy ugyanazon a gépen, ugyanazzal a szoftverrel, rövid időn belül elvégzett mérések szórása kicsi legyen, és az átlag stabilan reprodukálható legyen. A **reprodukálhatóság** (reproducibility) ennél szigorúbb: ha egy másik, azonos felépítésű GPU-n, de azonos driverrel és library-verziókkal futtatjuk a benchmarkot, az eredmény statisztikailag összeegyeztethető legyen az első méréssel. A **replikálhatóság** (replicability) pedig azt jelenti, hogy a mérési leírás alapján egy független mérnök is elő tudja állítani a kísérletet, és hasonló konklúzióra jut.

A GPU-mérések reprodukálhatóságát leginkább a következő tényezők rombolják:

- nem rögzített hardver- és szoftverkörnyezet,
- nem kontrollált órajelek és teljesítményállapotok,
- hideg indítás, JIT-fordítás és első allokációk zavaró hatása,
- túl kevés ismétlés,
- az időmérési API hibás használata,
- más folyamatokból származó háttérterhelés,
- a kernel eredményének ellenőrzés nélkül hagyása.

Ezeket a hibákat a mérési tervben szisztematikusan ki kell zárni. A benchmark akkor reprodukálható, ha a mérési napló tartalmazza a környezet pontos leírását, a munkaterhelést, a mérési protokollt és a nyers eredményeket.

31.3.2 A mérési terv szerkezete

Mielőtt időzítünk, készítsünk mérési tervet. A terv kötelező elemei:

1. **Cél és hipotézis.** Mit akarunk mérni, és mi az előzetes várakozás? Például: „A vektorösszeadás kernel memória-sávszélesség-korlátos, ezért a mért sávszélesség a GPU névleges HBM-sávszélességének legalább 80 százalékát eléri.”
2. **Független változók.** Mit változtatunk? Inputméret, blockméret, precision, adatelrendezés.
3. **Függő változók.** Mit mérünk? Kernel-futásidő, átbocsátóképesség, sávszélesség, energia.
4. **Rögzített paraméterek.** GPU-típus, driver, CUDA-verzió, operációs rendszer, power limit, ECC, MIG, órajelek, hőmérséklet-korlátok.
5. **Mérési protokoll.** Warm-up, ismétlésszám, időmérési módszer, adatrögzítés.
6. **Validáció.** Hogyan ellenőrizzük, hogy a kernel helyes eredményt adott?
7. **Elemzés.** Átlag, szórás, percentilisek, konfidenciaintervallum.

A tervben külön kell választani a **várt** és a **mért** eredményt. A várt értéket elméleti megfontolásból vagy korábbi mérésből származtatjuk, és még a futtatás előtt rögzítjük. A mért eredményt a kísérlet után, a nyers adatokból számítjuk. Ha a kettő eltér, az elemzés feladata az eltérés okának feltárása – nem pedig a várt érték utólagos módosítása.

31.3.3 Időmérés GPU-n: CUDA events

A CPU oldali `std::chrono` vagy `clock()` nem alkalmas a kernel futásidejének pontos mérésére, mert a kernel indítása aszinkron, és a host oldali időzítés magában foglalja a launch overheadet, valamint a CPU és GPU közötti szinkronizáció költségét. A CUDA események (`cudaEvent_t`) a GPU streambe illesztett időbélyegek, amelyek a parancssorban elfoglalt helyükön rögzítik az időt, így a kernel és a hozzá tartozó memóriaműveletek között mért idő valódi GPU-idő.

A helyes mérési minta:

```
cudaEvent_t start, stop;
CHECK_CUDA(cudaEventCreate(&start));
CHECK_CUDA(cudaEventCreate(&stop));

// warm-up
for (int i = 0; i < warmup; ++i) {
    vectorAdd<<<gridDim, blockDim>>>(d_a, d_b, d_c, N);
}
CHECK_CUDA(cudaDeviceSynchronize());

// mérési ciklus
std::vector<float> times;
times.reserve(iters);

for (int i = 0; i < iters; ++i) {
    CHECK_CUDA(cudaEventRecord(start, 0));
    vectorAdd<<<gridDim, blockDim>>>(d_a, d_b, d_c, N);
    CHECK_CUDA(cudaEventRecord(stop, 0));
    CHECK_CUDA(cudaEventSynchronize(stop));

    float ms = 0.0f;
    CHECK_CUDA(cudaEventElapsedTime(&ms, start, stop));
    times.push_back(ms);
}
```

A `cudaEventRecord` a streambe helyez egy eseményt, a `cudaEventSynchronize(stop)` megvárja, amíg a GPU eléri a `stop` eseményt. A `cudaEventElapsedTime` a két esemény közötti GPU-időt adja milliszekundumban. Ha a kernel előtt és után memóriamásolás is van, akkor az időzítés magában foglalja azokat is, ha ugyanabban a streamben rögzítjük az eseményeket. A kernel futásidejének méréséhez a másolásokat előre elvégezzük, és csak a kernelhívást vesszük körbe eseményekkel.

A stream explicit megadása jó gyakorlat: ha több streamet használunk, az események a saját streamjükhöz tartoznak. A default stream (0) szinkronizál a többi streammel, ezért mérés közben érdemes külön, nem default streamet használni, hogy elkerüljük a váratlan szinkronizációt.

31.3.4 Warm-up, ismétlés és statisztika

A GPU-mérés során a **hideg indítás** hatása jelentős. Az első kernelindítás előtt a CUDA-kontextus létrehozása, az eszköz inicializálása, a modul betöltése és az esetleges JIT-fordítás több tíz vagy akár száz milliszekundumot is igénybe vehet. Az első pár futás emellett felmelegíti

a GPU-t, és az órajelek még nem érik el a stabil, terhelés alatti értéket. Ezért a mérés előtt legalább 10–20, a valós munkaterheléssel azonos méretű és konfigurációjú kernelindítást kell futtatni, és utána `cudaDeviceSynchronize`-tel megvárni a befejezést.

A **mérési ismétlések** számát úgy kell megválasztani, hogy a statisztikai jellemzők stabilak legyenek. A könyv alapértelmezése legalább 5–10 ismétlés, de a reprodukálhatósághoz 30–50 ismétlés javasolt, különösen akkor, ha a mérési zaj nagy. Az ismétlésekből a következő statisztikákat számítjuk:

- **átlag**: a futásidők számtani közepe;
- **szórás (std)**: mennyire térnek el az egyes mérések az átlagtól;
- **relatív szórás (CV)**: a szórás osztva az átlaggal, százalékban kifejezve;
- **minimum és maximum**: a szélsőértékek, amelyek jelzik a zaj mértékét;
- **percentilisek**: a medián (50. percentilis) és a 90/95. percentilis robusztusabb jellemzők, mint az átlag, mert kevésbé érzékenyek a kiugró értékekre.

A mérési naplóban mindig közöljük az átlagot, a szórást és legalább a mediánt. A szórás önmagában nem elegendő, mert nem mutatja az eloszlás alakját, de a CV és a percentilisek segítenek megítélni a stabilitást.

A mérési zaj forrásai lehetnek véletlenszerűek (OS-ütemezés, háttérfolyamatok, hőmérséklet-ingadozás) és szisztematikusak (például a boost-órajel lassú emelkedése). A szisztematikus hibát a protokoll javításával, a véletlenszerűt az ismétlésszám növelésével csökkentjük. Ha a mérési sorozatban egy érték extrém módon kilóg, akkor meg kell vizsgálni az okát – például a GPU-t időközben lefoglalta egy másik folyamat –, és nem szabad egyszerűen törölni. A kilógó értékeket a naplóban meg kell tartani, és az elemzésben jelezni kell az okot.

31.3.5 Órajelek és teljesítményállapotok

A GPU futásidejét nemcsak a kernel algoritmusa, hanem az aktuális órajel is meghatározza. A modern GPU-k **boost-órajelei** a terheléstől, a hőmérséklettől és a teljesítménykerettől függenek. A hideg GPU alacsonyabb órajelen indul, majd a terhelés hatására emelkedik, de ha a hőmérséklet túl magas, a **thermal throttling** visszaveszi az órjelet. A **power throttling** hasonlóan korlátozza az órajelet, ha a fogyasztás eléri a power limitet.

Ezért a mérés előtt a GPU-t „be kell melegíteni” a tervezett munkaterheléssel, és a mérés alatt folyamatosan mintavételezni kell az órajelet és a hőmérsékletet. A `nvidia-smi` parancssorból elérhető lekérdezés például:

```
nvidia-smi --query-gpu=timestamp,clocks.sm,clocks.mem,power.draw,temperature.gpu \
           --format=csv --loop=1 > gpu_metrics.csv
```

A pontos kapcsolók és mezőnevek a telepített driver verziójától függenek, ezért a használat előtt ellenőrizni kell a `nvidia-smi --help-query-gpu` kimenetét. A mérési naplóban rögzíteni kell legalább az SM-órajelet, a memória-órajelet, a hőmérsékletet és a pillanatnyi fogyasztást. Ha a mérés során az órajel 5–10 százaléknál nagyobb mértékben ingadozik, az eredményt érvénytelennek kell tekinteni, vagy a mérést stabilizált órajel mellett meg kell ismételni.

Egyes GPU-kon lehetőség van az órajelek **rögzítésére** (application clocks), például:

```
nvidia-smi -lgc 1400,1400
```

Ez a parancs az SM-órajelet 1400 MHz-re rögzíti, kiküszöbölve a boost ingadozását. Ez azonban adminisztrátori jogosultságot igényelhet, és nem minden GPU-modell támogatja. Ha az órajelet nem rögzítjük, akkor legalább naplózzuk az aktuális értékét, és a mérési eredményt az órajellel együtt értelmezzük. A reprodukálhatóság érdekében a mérési naplóban fel kell tüntetni az órajel-politikát: „boost engedélyezett” vagy „rögzített órajel”.

A **teljesítményállapot** (power state) szintén befolyásolja a mérést. A `nvidia-smi -q -d POWER` megmutatja a power limitet és az aktuális fogyasztást. A mérés előtt rögzítsük a power limitet, és ha a benchmark energiahatékonyságot is mér, akkor a fogyasztást a futásidővel együtt kell gyűjteni. Az energia mérésének részletei a következő alfejezetben (Energia és hatékonyság) kerülnek kifejtésre, itt elegendő annyi, hogy a benchmarkmódszertan a fogyasztást és az órajelet a futásidő mértékének kontextusaként kezeli.

31.3.6 Példa: vektorösszeadás benchmark

A következő CUDA C++ példa egy memória-sávszélesség-korlátos kernel, a vektorösszeadás mérését mutatja be. A kernel 1D indexeléssel, grid-stride loop nélkül dolgozik, mert a méret pontosan osztható a blockmérettel. A mérési protokoll: 10 warm-up, 50 ismétlés, CUDA event időmérés. Az eredményeket a program kiírja, és a validációhoz összehasonlítja a CPU-n számított referenciaeredményt. A kódblokk nyelve `CUDA C++ (C++17, CUDA 12.x)`.

```
#include <cuda_runtime.h>
#include <iostream>
#include <vector>
#include <cmath>
#include <numeric>
#include <algorithm>

#define CHECK_CUDA(call)                                                    \
    do {                                                                    \
        cudaError_t err = call;                                             \
        if (err != cudaSuccess) {                                           \
            std::cerr << "CUDA error: " << cudaGetErrorString(err)          \
                      << " at " << __FILE__ << ":" << __LINE__ << std::endl;\
            std::exit(EXIT_FAILURE);                                        \
        }                                                                   \
    } while (0)

__global__ void vectorAdd(const float* a, const float* b, float* c, size_t n) {
    size_t idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        c[idx] = a[idx] + b[idx];
    }
}

int main() {
    const size_t N = 1ULL << 26;          // 64M elem
    const int blockDim = 256;
    const int gridDim = static_cast<int>((N + blockDim - 1) / blockDim);

    // Host-allokáció és inicializálás
    std::vector<float> h_a(N), h_b(N), h_c(N), h_ref(N);
    std::iota(h_a.begin(), h_a.end(), 0.0f);
    std::iota(h_b.begin(), h_b.end(), 1.0f);

    // Eszközallokáció
    float *d_a, *d_b, *d_c;
    CHECK_CUDA(cudaMalloc(&d_a, N * sizeof(float)));
    CHECK_CUDA(cudaMalloc(&d_b, N * sizeof(float)));
```

```
CHECK_CUDA(cudaMalloc(&d_c, N * sizeof(float)));

// Másolás hostról eszközre
CHECK_CUDA(cudaMemcpy(d_a, h_a.data(), N * sizeof(float), cudaMemcpyHostToDevice));
CHECK_CUDA(cudaMemcpy(d_b, h_b.data(), N * sizeof(float), cudaMemcpyHostToDevice));

// Referencia CPU-n
for (size_t i = 0; i < N; ++i) {
    h_ref[i] = h_a[i] + h_b[i];
}

// Események
cudaEvent_t start, stop;
CHECK_CUDA(cudaEventCreate(&start));
CHECK_CUDA(cudaEventCreate(&stop));

// Warm-up
const int warmup = 10;
for (int i = 0; i < warmup; ++i) {
    vectorAdd<<<gridDim, blockDim>>>(d_a, d_b, d_c, N);
}
CHECK_CUDA(cudaDeviceSynchronize());

// Mérési ciklus
const int iters = 50;
std::vector<float> times;
times.reserve(iters);

for (int i = 0; i < iters; ++i) {
    CHECK_CUDA(cudaEventRecord(start, 0));
    vectorAdd<<<gridDim, blockDim>>>(d_a, d_b, d_c, N);
    CHECK_CUDA(cudaEventRecord(stop, 0));
    CHECK_CUDA(cudaEventSynchronize(stop));

    float ms = 0.0f;
    CHECK_CUDA(cudaEventElapsedTime(&ms, start, stop));
    times.push_back(ms);
}

// Eredmény ellenőrzése
CHECK_CUDA(cudaMemcpy(h_c.data(), d_c, N * sizeof(float), cudaMemcpyDeviceToHost));
CHECK_CUDA(cudaDeviceSynchronize());

bool valid = true;
for (size_t i = 0; i < N; ++i) {
    if (std::fabs(h_c[i] - h_ref[i]) > 1e-5f) {
        valid = false;
        std::cerr << "Eltérés az " << i << ". elemnél: "
                  << h_c[i] << " vs " << h_ref[i] << std::endl;
        break;
    }
}
if (!valid) {
    std::cerr << "Validáció sikertelen!" << std::endl;
    std::exit(EXIT_FAILURE);
}

// Statisztikák
float sum = std::accumulate(times.begin(), times.end(), 0.0f);
float mean = sum / times.size();
float sq_sum = std::inner_product(times.begin(), times.end(), times.begin(), 0.0f);
float stddev = std::sqrt(std::max(0.0f, sq_sum / times.size() - mean * mean));
std::sort(times.begin(), times.end());
```

```
    float median = times[times.size() / 2];

    // Sávszélesség: N * 3 * sizeof(float) / átlagidő
    double bytes = static_cast<double>(N) * 3.0 * sizeof(float);
    double bw = bytes / (mean * 1e-3) / 1e9;  // GB/s

    std::cout << "Kernel futasido (ms):\n";
    std::cout << "  atlag   = " << mean << "\n";
    std::cout << "  szoras  = " << stddev << "\n";
    std::cout << "  median  = " << median << "\n";
    std::cout << "  min     = " << times.front() << "\n";
    std::cout << "  max     = " << times.back() << "\n";
    std::cout << "Savszelesseg (GB/s): " << bw << "\n";

    // Takarítás
    CHECK_CUDA(cudaEventDestroy(start));
    CHECK_CUDA(cudaEventDestroy(stop));
    CHECK_CUDA(cudaFree(d_a));
    CHECK_CUDA(cudaFree(d_b));
    CHECK_CUDA(cudaFree(d_c));

    return 0;
}
```

A fordítás parancsa például:

```
nvcc -O3 -arch=sm_80 -o bench_vector_add vector_add_bench.cu
```

Az `-arch=sm_80` egy NVIDIA A100 (compute capability 8.0) esetén érvényes; a saját GPU architektúrájához igazítani kell.

A futtatás előtt a környezetet rögzíteni kell, és a mérési naplót el kell készíteni. Egy lehetséges napló (illusztráció, nem valós mérés):

```
Környezet:
  GPU: NVIDIA A100 40GB (példa)
  Driver: 535.104.05 (példa)
  CUDA: 12.2 (példa)
  OS: Ubuntu 22.04 LTS
  Power limit: 250 W (alapértelmezett)
  ECC: bekapcsolva
  MIG: kikapcsolva
  Háttérterhelés: nincs

Bemenet:
  N = 64M float elem
  Precision: FP32
  Blokkméret: 256
  Gridméret: 262144

Protokoll:
  Warm-up: 10 ismétlés
  Mérés: 50 ismétlés
  Időmérés: CUDA event
  Órajel/power: nvidia-smi mintavételezés 1 s-onként

Várt eredmény:
  A vectorAdd 3 * 4 bájt = 12 bájt/elem memóriamozgást végez.
  N = 64M elem -> 805 MB adat.
  Feltételezett HBM-sávszélesség: 1.5 TB/s (A100 esetén névleges ~1555 GB/s).
  Várt futásidő: 805 MB / 1500 GB/s  0.54 ms.
  Várt sávszélesség: elméleti sávszélesség 80-90%-a, azaz 1.2-1.4 TB/s.
```

```
Mért eredmény:
  Futásidő átlag: 0.63 ms
  Szórás: 0.02 ms
  Medián: 0.62 ms
  Min: 0.60 ms
  Max: 0.69 ms
  SM órajel átlag: 1410 MHz
  Hőmérséklet: 55-60 °C
  Power: 210-230 W
  Számított sávszélesség: 805 MB / 0.63 ms  1278 GB/s

Értékelés:
  A mért sávszélesség a várt tartományban van, a kernel valóban memória-korlátos.
  A relatív szórás 3%, ami elfogadható. A kilógó 0.69 ms érték a mérés elején
  fordult elő, feltehetően az órajel kismértékű ingadozása miatt.
```

Ebben a naplóban a várt és a mért eredmény világosan el van választva. A mért eredmények mögött nyers adatok és statisztikai feldolgozás áll, a környezet pedig teljes mértékben rögzített.

31.3.7 Tipikus hibák és helyes megoldások

A következő hibák gyakran előfordulnak GPU-benchmarkokban. Mindegyiknél megadjuk a jelenséget és a helyes gyakorlatot.

1. Hideg indítás figyelmen kívül hagyása.
Hiba: az első kernelindítást mérik, vagy a warm-up túl rövid.
Következmény: a futásidő irreálisan magas, mert a kontextus- és memóriaallokáció, JIT-fordítás, valamint az órajel-emelkedés beleszámít.
Megoldás: 10–20 warm-up futás a tényleges mérés előtt, azonos konfigurációval.

2. Túl kevés ismétlés.
Hiba: egyetlen futás alapján vonnak le következtetést.
Következmény: a mérési zaj dominál, az eredmény nem reprodukálható.
Megoldás: legalább 5–10 ismétlés, de 30–50 javasolt; statisztikák közlése.

3. A CUDA event hibás használata.
Hiba: a `cudaEventRecord` után nincs szinkronizáció, vagy a CPU-időt mérik.
Következmény: a mért idő nem a kernel GPU-ideje, hanem a launch overheaddel és CPU-ütemezéssel terhelt idő.
Megoldás: az eseményeket ugyanabba a streambe rögzítjük, a `stop` eseményt szinkronizáljuk, és a `cudaEventElapsedTime` értékét használjuk.

4. Nem rögzítik az órajelet és a power state-et.
Hiba: a mérés közben változik a boost-órajel vagy thermal throttling lép fel, de ezt nem naplózzák.
Következmény: két mérési sorozat között 20–30 százalékos eltérés is lehet, ami hamis optimalizálási következtetéshez vezet.
Megoldás: az órajelet és a hőmérsékletet mintavételezzük, a power limitet rögzítjük; ha lehet, alkalmazzunk rögzített órajelet.

5. Háttérterhelés figyelmen kívül hagyása.
Hiba: a benchmark futtatása közben más folyamatok is használják a GPU-t.
Következmény: a mérési zaj megnő, a sávszélesség csökken.
Megoldás: a mérés előtt ellenőrizzük a GPU kihasználtságát (`nvidia-smi`), lehetőleg dedikált GPU-n futtassunk, vagy távolítsuk el a konkurens folyamatokat. Többfelhasználós környezetben rögzítsük a MIG/ time-slicing beállításokat.

6. A kernel túl gyors, a launch overhead dominál.
Hiba: olyan kis inputméretet mérünk, ahol a kernel futásideje összemérhető az indítási költséggel.
Következmény: a mért idő nem a kernel számítását, hanem a driver overheadjét tükrözi.
Megoldás: növeljük a munkaterhelést, használjunk grid-stride loopot a nagy adat feldolgozására, vagy mérjük a launch overheadet külön, és vonjuk le.

7. Nincs validáció.
Hiba: a benchmark nem ellenőrzi, hogy a kernel helyes eredményt ad-e.
Következmény: egy hibás kernel is lehet „gyors", de az eredmény értéktelen.
Megoldás: a kernel kimenetét hasonlítsuk össze referenciamegoldással, és a validációt a mérési naplóban rögzítsük.

8. A mérési adatok szelektív bemutatása.
Hiba: a kilógó értékeket törlik, vagy csak a legjobb futást közlik.
Következmény: a publikált eredmény torzít, és nem reprodukálható.
Megoldás: minden nyers adatot meg kell őrizni, a statisztikákat az összes futásra kell számítani, és a kiugró értékeket külön jelezni kell.

31.3.8 Labor: benchmarkterv készítése

Feladat. Készíts benchmarktervet egy választott GPU-kernel mérésére, és hajtsd végre a mérést. A kernel lehet a fenti vektorösszeadás, egy korábbi fejezetben megismert redukció, stencil vagy saját fejlesztés. A mérési környezetet pontosan rögzíteni kell, és a várt eredményt a futtatás előtt ki kell számítani.

Átadandó dokumentum. A benchmarkterv és a mérési napló, amely az alábbi sablont követi:

```
1. Cél és hipotézis
   Mit mérünk? Mi a várt eredmény?

2. Környezet
   GPU modell és compute capability:
   Driver verzió:
   CUDA verzió:
   Operációs rendszer:
   Power limit:
   ECC:
   MIG / virtualizáció:
   Háttérterhelés:
   Órajel-politika (boost vagy rögzített):

3. Munkaterhelés
   Kernel neve és leírása:
   Inputméret és -típus:
   Precision:
   Grid- és blockdimenzió:
   Shared memory (ha van):

4. Mérési protokoll
   Warm-up ismétlésszám:
   Mérési ismétlésszám:
   Időmérési módszer (CUDA event / OpenCL event / JAX):
   Órajel- és power-mintavétel módja:

5. Validáció
   Hogyan ellenőrzöd a kernel helyességét?
   Referenciaeredmény forrása:

6. Várt eredmény
```

```
   Elméleti becslés (pl. sávszélesség, FLOPs):
   Indoklás:

7. Mért eredmény
   Táblázat: futásidő, átlag, szórás, medián, min, max, CV
   Órajel, hőmérséklet, power
   Számított metrikák (sávszélesség, GFLOPS, stb.)

8. Eltérés elemzése
   A várt és mért eredmény összehasonlítása.
   Lehetséges okok és javaslatok.
```

A mérési naplóban a **mért és várt eredményt külön szekcióban** kell feltüntetni, és a nyers adatokat is csatolni kell. A kód legyen nyelvjelölt (CUDA C++/OpenCL/JAX), és a build/futtatási parancsot is rögzíteni kell. A méréshez kapcsolódó összes állítás csak akkor fogadható el, ha a validáció sikeres és a mérési feltételek dokumentáltak.

A labor során az olvasó a következő készségeket gyakorolja:

- mérési terv készítése,
- a környezet pontos rögzítése,
- a warm-up és ismétlésszám helyes megválasztása,
- a várt érték elméleti becslése,
- a mért adatok statisztikai feldolgozása,
- az eltérések kritikus elemzése.

31.3.9 Önellenőrző kérdések

1. Mi a különbség a CPU oldali időzítés és a CUDA event közötti mérés között? Melyik méri a kernel tényleges GPU-idejét?
2. Miért szükséges a warm-up a GPU-benchmarkokban? Milyen hatásokat küszöböl ki?
3. Miért kell az órajelet és a hőmérsékletet rögzíteni a mérési naplóban? Milyen következménnyel jár, ha ezt elhagyjuk?
4. Hogyan számítod a vektorösszeadás kernel memória-sávszélesség-igényét? Miért lesz a mért sávszélesség kisebb a névleges csúcsértéknél?
5. Sorolj fel három tipikus benchmark-hibát, és magyarázd el a helyes megoldást.

A fejezetben bevezetett benchmarkmódszertan alapot ad a következő egységhez (Energia és hatékonyság), amely az energia- és hatékonyságmérésre, valamint a GPU-k energiafogyasztásának számszerűsítésére épít.

31.4 Energia és hatékonyság

Az energia és a hatékonyság a GPU-alapú számítás egyik meghatározó, ám sokáig háttérbe szorult dimenziója. A teljesítményt gyakran csak a futásidővel mérjük, pedig az adatközpontok üzemeltetési költsége, a hűtési kapacitás és a környezeti lábnyom legalább annyira függ az elfogyasztott energiától. Ebben az alfejezetben a GPU energiafogyasztásának mérését, a performance per watt fogalmát, valamint a DVFS és a hőmérséklet szerepét tárgyaljuk. A korábbi, Benchmarkmódszertan egységben bevezetett benchmarkmódszertanra építünk, de ott a futásidő volt a fő mérőszám; most az energia kerül a középpontba.

31.4.1 A GPU energiafogyasztása

A GPU energiafogyasztása két fő összetevőre bontható: statikus és dinamikus teljesítményre. A statikus teljesítmény (P_{static}) akkor is jelen van, ha a chip nem végez hasznos munkát, és elsősorban a tranzisztorok szivárgási áramaiból származik. Ez a komponens erősen függ a hőmérséklettől és a tápfeszültségtől: magasabb hőmérséklet vagy feszültség növeli a szivárgást, ami további hőt termel, ezért a hűtés és a feszültség szabályozása kritikus. A dinamikus teljesítmény (P_{dyn}) a kapuk átkapcsolásakor keletkezik, és jó közelítéssel a következő képlettel írható le:

$$P_{\text{dyn}} \approx \alpha C V^2 f$$

ahol α az aktivitási tényező (a kapuk hányada vált át egy órajelciklus alatt), C a kapcsolt kapacitás, V a tápfeszültség, f pedig az órajel. A teljes teljesítményfelvétel ennek a két tagnak az összege: $P = P_{\text{static}} + P_{\text{dyn}}$. A fenti összefüggésből látható, hogy a fogyasztás négyzetesen függ a feszültségtől, és lineárisan az órajeltől; ha a feszültséget is skálázzuk az órajellel, a dinamikus teljesítmény közel köbösen nő a frekvenciával.

A GPU kártya teljes energiafogyasztása nem csak a GPU chipből áll: magában foglalja a memória (HBM vagy GDDR) fogyasztását, a feszültségszabályzók veszteségeit, a hűtőventilátorokat és az egyéb fedélzeti komponenseket. A `nvidia-smi` által visszaadott power draw (teljesítményfelvétel) a teljes kártyára vonatkozik, nem csupán a számítómagokra. Ez a gyakorlatban azt jelenti, hogy a teljesítményfelvétel nem csupán a kernel FLOP-igényétől, hanem a memória-hozzáférések intenzitásától és a kártya aktuális órajel-állapotától is függ.

Az energia és a teljesítmény fogalmát világosan el kell különíteni. A teljesítmény (P, watt) pillanatnyi érték, az energia (E, joule) a teljesítmény idő szerinti integrálja:

$$E = \int P(t)\, dt$$

Egy rövid, nagy teljesítményű kernel kevesebb energiát fogyaszthat, mint egy hosszú, alacsonyabb teljesítményű, ha a futásidő rövidsége kompenzálja a magasabb teljesítményt. A mérések során ezért nem elegendő a csúcsteljesítményt figyelni; a teljes energia az, ami az adatközpont költségét és a hűtési terhelést meghatározza.

A gyakorlatban a GPU energiafogyasztását többféleképpen mérhetjük. Az NVIDIA Management Library (NVML) programozott hozzáférést biztosít a teljesítmény, hőmérséklet, órajel és az energiafogyasztás lekérdezéséhez. Az `nvidia-smi dmon` vagy a `--query-gpu` kapcsolók segítségével parancssorból is mintavételezhetünk. A Nsight Systems és a Nsight Compute integrált energiaprofilozást kínál, amely képes a kernel futásához igazítani az energiafogyasztási adatokat. A mérési pontosság függ a mintavételezési frekvenciától: a tipikus 20–100 ms-os mintavételezés a rövid, néhány milliszekundumos kernelek energiafogyasztását alulbecsülheti vagy elkenheti. Ahol elérhető, az NVML `nvmlDeviceGetTotalEnergyConsumption` függvénye mJ pontosságú integrált energiaértéket ad, ami pontosabb, mint a pillanatnyi teljesítményből való numerikus integrálás.

31.4.2 Performance per watt

A performance per watt (perf/watt) a hasznos munka és az elfogyasztott energia hányadosa. A mérőszám azt fejezi ki, hogy egységnyi energia befektetésével mennyi számítási eredményt kapunk. A leggyakoribb mértékegysége a GFLOPS/W vagy TFLOPS/W, de a definíció tetszőleges munkaegységre kiterjeszthető (például tranzakció/W, kép/W, stb.). A perf/watt kiszámítása:

$$\text{perf/watt} = \frac{\text{munka}}{\text{energia}} = \frac{\text{elvégzett FLOP-ok}}{E \text{ joule}}$$

A munka meghatározásánál mindig a ténylegesen elvégzett hasznos műveleteket kell figyelembe venni, nem a névleges csúcsteljesítményt. Például egy $N \times N$-es mátrixszorzás esetén a FLOP-ok száma hagyományosan $2N^3$, mivel minden k-adik belső szorzat egy szorzást és egy összeadást tartalmaz. A kernel futásidő méréséből és az átlagos teljesítményfelvételből az energia és a perf/watt kiszámítható.

Miért fontos ez a mérőszám? Egy adatközpontban az energia költsége és a hűtési kapacitás korlátozott, így a nagyobb perf/watt érték kisebb üzemeltetési költséget és kisebb karbonlábnyomot jelent. A szuperszámítógépek energiahatékonysági rangsora (Green500) éppen ezt a mutatót használja. A mobil és beágyazott eszközöknél az akkumulátor élettartam szempontjából is kritikus.

A perf/watt nem független a kernel jellegétől. A memória-korlátos kerneleknél a számítási egységek kihasználtsága alacsony, miközben a memóriahierarchia aktívan fogyaszt, így a perf/watt alacsony. A számítás-korlátos kerneleknél a FLOP/W arány jobb, mert a számítási egységek hatékonyan dolgoznak. Az adatmozgás energiája a GPU-n belül is jelentős: a HBM és a GDDR memória hozzáférések energiaigénye nagyságrendekkel nagyobb, mint a regiszter- vagy shared memory-hozzáférésé. Ezért a tiling, a shared memory használata és a cache-barát memóriaminták nemcsak a futásidőt, hanem az energiafogyasztást is javítják.

A perf/watt optimalizálásának alapelvei ugyanazok, mint a teljesítményoptimalizálásé, de hangsúlyeltolódással: a felesleges adatmozgás kiküszöbölése, a memória-hozzáférések összevonása, a számítási egységek jobb kihasználása, valamint a speciális hardveregységek (tensor core-ok, FP16/BF16/TF32 mixed precision) alkalmazása. A tensor core-ok például sokkal több FLOP-ot végeznek wattanként, mint a hagyományos CUDA magok, ezért a mátrixszorzások energiahatékonysága ugrásszerűen javítható velük. A mixed precision (alacsonyabb pontosságú lebegőpontos formátumok) használata szintén csökkenti az adatmozgást és a számítási energiát, mivel kevesebb bitet kell mozgatni és tárolni.

A DVFS és a perf/watt kapcsolata közvetlen. Mivel a dinamikus teljesítmény közel köbösen függ az órajeltől (a feszültség együtt skálázódik), az órajel csökkentése csökkenti a teljesítményt, de a teljesítmény is csökken. Azonban a teljesítménycsökkenés általában kisebb mértékű, mint az energiafogyasztás csökkenése, így a perf/watt javulhat. Például ha az órajelet 20%-kal csökkentjük, a futásidő 25%-kal nőhet (a teljesítmény csökkenése miatt), de a teljesítményfelvétel 50%-kal csökkenhet, így az energia összességében 37,5%-kal csökken, a perf/watt pedig 20%-kal javul. Ezért sok GPU esetében az alacsonyabb órajelű, de párhuzamosabb végrehajtás energiahatékonyabb.

31.4.3 DVFS és hőmérséklet

A DVFS (Dynamic Voltage and Frequency Scaling) az órajel és a tápfeszültség dinamikus szabályozását jelenti a pillanatnyi terhelés, a hőmérséklet és a teljesítménykorlát függvényében. A modern GPU-k ezt hardveres és szoftveres együttműködéssel valósítják meg. Az NVIDIA GPU Boost technológiája például a rendelkezésre álló teljesítmény- és hőmérsékleti kereteken belül növeli az órjelet, hogy a lehető legnagyobb teljesítményt érje el. A boost órajel azonban nem állandó: ha a chip hőmérséklete eléri a határértéket, vagy a power limitet elérjük, az órajel csökken.

A hőmérséklet hatása kettős. Egyrészt a magasabb hőmérséklet növeli a szivárgási áramot, ami növeli a statikus teljesítményt és további hőt termel – ez egy pozitív visszacsatolás. Másrészt

a hűtőrendszer korlátai miatt a hőmérséklet emelkedése arra kényszerítheti a vezérlőt, hogy csökkentse az órjelet (thermal throttling), ami teljesítménycsökkenést okoz. Ezért a mérések során a hőmérsékletet stabilizálni kell: a mérés előtt futtassunk olyan melegítő terhelést, amely a GPU-t a munkahőmérsékletre hozza, és a mérés alatt figyeljük a hőmérsékletet és az órjelet. Ha ezek változnak, az eredmények nem összehasonlíthatók.

A DVFS gyakorlati következménye, hogy ugyanaz a kernel különböző futásidőt és energiafogyasztást produkálhat attól függően, hogy a GPU éppen milyen hőmérsékleti és teljesítményállapotban van. A benchmarkmódszertan (Benchmarkmódszertan) ezért megköveteli a warm-up fázist, a többszöri ismétlést és az átlagos értékek használatát. Az energiafogyasztás mérésénél ezt ki kell egészíteni a power limit rögzítésével (`nvidia-smi -pl`), a hűtési feltételek leírásával és az órajel naplózásával.

A hőmérséklet és az energiafogyasztás mérésére az `nvidia-smi` parancs a következő hasznos lekérdezéseket kínálja:

```
nvidia-smi --query-gpu=power.draw,temperature.gpu,clocks.sm,clocks.mem --format=csv
```

Ez a parancs egy pillanatnyi értékeket ad vissza, de folyamatos mintavételezésre is alkalmas a `-lms` (loop milliseconds) kapcsolóval. A mérési naplóhoz rögzíteni kell a power limitet, az ECC állapotot, a MIG konfigurációt, a driver verziót és a háttérterhelést. Ezek a körülmények közvetlenül befolyásolják a mért energiát, ezért a reprodukálhatósághoz elengedhetetlenek.

31.4.4 Labor: Energia- és teljesítménymérés

Ebben a laborban egy egyszerű, de számításigényes CUDA-kernel energiafogyasztását mérjük meg, és kiszámítjuk a perf/watt értékét. A cél nem a kernel optimalizálása, hanem a mérési folyamat bemutatása. A méréshez a korábban ismertetett elveket alkalmazzuk: warm-up, többszöri ismétlés, a környezet rögzítése, a mért és várt eredmények szétválasztása.

Környezet (példa, ellenőrizendő): - GPU: NVIDIA A100 40GB (compute capability 8.0) - Driver: NVIDIA 535.86.10 - CUDA: 12.2 - Operációs rendszer: Ubuntu 22.04 LTS - Power limit: 300 W - ECC: engedélyezve - MIG: letiltva - Hűtés: passzív adatközponti hűtés, környezeti hőmérséklet 22 °C - Háttérterhelés: nincs más GPU-folyamat

Az itt megadott értékek példaként szolgálnak; a saját rendszeren ellenőrizni kell őket.

Kód (CUDA C++17, naiv mátrixszorzás):

```
// CUDA C++ (C++17) - energia mérési labor
// Fordítás: nvcc -O3 -arch=sm_80 energy_lab.cu -o energy_lab
// (Az -arch értéke a cél GPU-hoz igazítandó.)

#include <cuda_runtime.h>
#include <cstdio>
#include <cstdlib>
#include <vector>

#define CHECK_CUDA(call) \
    do { \
        cudaError_t err = call; \
        if (err != cudaSuccess) { \
            fprintf(stderr, "CUDA hiba: %s (%s:%d)\n", \
                    cudaGetErrorString(err), __FILE__, __LINE__); \
            exit(EXIT_FAILURE); \
        } \
    } while (0)
```

```
__global__ void gemm_naive(const float* __restrict__ A,
                           const float* __restrict__ B,
                           float* __restrict__ C,
                           int N) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < N && col < N) {
        float sum = 0.0f;
        for (int k = 0; k < N; ++k) {
            sum += A[row * N + k] * B[k * N + col];
        }
        C[row * N + col] = sum;
    }
}

int main() {
    const int N = 1024;
    const size_t size = (size_t)N * N * sizeof(float);

    float *d_A, *d_B, *d_C;
    CHECK_CUDA(cudaMalloc(&d_A, size));
    CHECK_CUDA(cudaMalloc(&d_B, size));
    CHECK_CUDA(cudaMalloc(&d_C, size));

    std::vector<float> h_A(N * N), h_B(N * N);
    for (int i = 0; i < N * N; ++i) {
        h_A[i] = static_cast<float>(rand()) / RAND_MAX;
        h_B[i] = static_cast<float>(rand()) / RAND_MAX;
    }
    CHECK_CUDA(cudaMemcpy(d_A, h_A.data(), size, cudaMemcpyHostToDevice));
    CHECK_CUDA(cudaMemcpy(d_B, h_B.data(), size, cudaMemcpyHostToDevice));

    dim3 block(16, 16);
    dim3 grid((N + block.x - 1) / block.x, (N + block.y - 1) / block.y);

    cudaEvent_t start, stop;
    CHECK_CUDA(cudaEventCreate(&start));
    CHECK_CUDA(cudaEventCreate(&stop));

    // Warm-up: 5 ismétlés a hőmérséklet és az órajel stabilizálásához
    for (int i = 0; i < 5; ++i) {
        gemm_naive<<<grid, block>>>(d_A, d_B, d_C, N);
    }
    CHECK_CUDA(cudaDeviceSynchronize());

    const int reps = 10;
    CHECK_CUDA(cudaEventRecord(start));
    for (int i = 0; i < reps; ++i) {
        gemm_naive<<<grid, block>>>(d_A, d_B, d_C, N);
    }
    CHECK_CUDA(cudaEventRecord(stop));
    CHECK_CUDA(cudaDeviceSynchronize());

    float elapsed_ms = 0.0f;
    CHECK_CUDA(cudaEventElapsedTime(&elapsed_ms, start, stop));
    double avg_ms = static_cast<double>(elapsed_ms) / reps;

    // Az energiafogyasztást külön folyamatban gyűjtjük,
    // lásd a mérési protokollt.

    printf("Átlagos kernel futásidő: %.3f ms\n", avg_ms);

    CHECK_CUDA(cudaFree(d_A));
```

```
    CHECK_CUDA(cudaFree(d_B));
    CHECK_CUDA(cudaFree(d_C));
    CHECK_CUDA(cudaEventDestroy(start));
    CHECK_CUDA(cudaEventDestroy(stop));
    return 0;
}
```

A kód a mátrixszorzást naiv módon, globális memóriából való ismételt olvasással végzi. Ez a változat viszonylag sok memória-hozzáférést végez, ezért energiafogyasztása magasabb, perf/watt értéke alacsonyabb, mint egy optimalizált tiling kernelé. A labor célja azonban a mérési módszertan bemutatása, nem a kernel optimalizálása.

Build és mérési protokoll:

A program lefordítása után a méréshez elindítunk egy háttérfolyamatot, amely 50 ms-os intervallummal mintavételezi a teljesítményt, hőmérsékletet és órjeleket:

```
# Build
nvcc -O3 -arch=sm_80 energy_lab.cu -o energy_lab

# A power limit ellenőrzése (opcionális, ha jogosultságunk van)
nvidia-smi -pl 300

# Háttér-mintavételezés indítása
nvidia-smi --query-gpu=power.draw,temperature.gpu,clocks.sm,clocks.mem \
           --format=csv -lms 50 -f power_log.csv &

# A program futtatása
./energy_lab

# A mintavételező leállítása
kill %1
```

A program kimenete az átlagos kernel futásidőt mutatja. A `power_log.csv` fájl tartalmazza a mintavételezett értékeket. A kernel futása közben az energiafogyasztást a napló alapján kell becsülni: azonosítsuk a kernel futási időintervallumát (például a program által kiírt kezdeti és végső időpontok alapján, vagy a mintavételezett teljesítménygörbe alapján), és számítsuk ki az átlagos teljesítményt, majd az energiát az $E = P_{\text{avg}} \cdot t$ képlettel.

Mérési napló (példaértékek):

A következő táblázat egy tíz ismétlésből álló mérés eredményét mutatja. Az értékek a fenti környezetben készült példaértékek, nem garantáltan reprodukálhatók; a saját mérést mindig rögzíteni kell.

Ismétlés	kernel idő (ms)	átlag power (W)	energia (J)
1	12.35	178	2.198
2	12.31	180	2.216
3	12.38	179	2.219
4	12.33	181	2.232
5	12.36	177	2.191
6	12.34	179	2.209
7	12.37	180	2.226
8	12.32	178	2.194
9	12.35	180	2.222
10	12.36	179	2.219
átlag	**12.34**	**179.1**	**2.2126**
szórás	0.02	1.2	0.014

Ismétlés	kernel idő (ms)	átlag power (W)	energia (J)

A számításokhoz használt képletek:

- Átlagos futásidő: $t_{\text{avg}} = 12{,}34 \text{ ms} = 0{,}01234 \text{ s}$
- Átlagos teljesítmény: $P_{\text{avg}} = 179{,}1 \text{ W}$
- Energia: $E = P_{\text{avg}} \cdot t_{\text{avg}} = 179{,}1 \text{ W} \cdot 0{,}01234 \text{ s} = 2{,}210 \text{ J}$ (a táblázatban a soronkénti energiaátlag 2,2126 J, a kerekítés miatt kis eltérés)
- Elvégzett FLOP: $2 \cdot N^3 = 2 \cdot 1024^3 = 2{,}147 \cdot 10^9 \text{ FLOP} = 2{,}147 \text{ GFLOP}$
- Performance per watt: $\text{perf/watt} = \frac{2{,}147 \text{ GFLOP}}{2{,}210 \text{ J}} \approx 0{,}971 \text{ GFLOPS/W}$

A mért eredmény tehát körülbelül 0,97 GFLOPS/W. Ez a naiv kernel energiahatékonysága alacsony; összehasonlításképpen egy optimalizált, tensor core-t használó GEMM ugyanezen a hardveren 10–20× nagyobb perf/watt értéket is elérhet (ellenőrizendő, hardverfüggő).

Mért és várt eredmény elkülönítése:

A várt energiafogyasztás becsléséhez a GPU névleges teljesítményfelvételéből indulhatunk ki. A példa GPU TDP-je 300 W, de a tényleges fogyasztás a kernel jellegétől függ. Egy naiv, erősen memória-hozzáférésekre támaszkodó kernel esetén az átlagos fogyasztás alacsonyabb lehet, mert a számítási egységek kihasználtsága nem maximális. A várt energia: ha feltételezzük, hogy a kernel 12 ms alatt fut le, és az átlagos fogyasztás 180 W körül van, akkor a várt energia 2,16 J. A mért energia 2,21 J, ami 2,3%-kal magasabb, de az ingadozás határain belül van. A várt perf/watt: $2{,}147 \text{ GFLOP}/(0{,}012 \text{ s} \cdot 200 \text{ W}) = 0{,}895 \text{ GFLOPS/W}$, a mért 0,971 GFLOPS/W. A különbség abból adódik, hogy a tényleges fogyasztás kisebb volt, mint a becslésnél feltételezett 200 W. A mérési bizonytalanság miatt ezek az értékek közelítőek.

A mérés reprodukálhatóságához a következőket kell rögzíteni: a GPU típusát, driver és CUDA verziót, power limitet, ECC és MIG állapotot, a környezeti hőmérsékletet, a hűtés típusát, a mérés előtti hőmérsékletet, a warm-up hosszát, a mintavételezési frekvenciát, a háttérterhelést, valamint a kernel pontosságát (FP32). A mérési naplónak tartalmaznia kell az egyes ismétlések értékeit és azok szórását.

Gyakori hibák és megelőzésük:

1. **A warm-up elhagyása vagy túl rövid időtartama.** A hidegen indított GPU órajele és hőmérséklete nem stabil; a mérés elején a teljesítményfelvétel más lehet, mint a későbbi ismétlésekben. Megoldás: legalább 5–10 melegítő kernel futtatása, amíg a hőmérséklet és az órajel stabilizálódik.

2. **A power draw mintavételezésének alacsony frekvenciája.** A rövid kerneleknél a 100 ms-os mintavételezés nem képes követni a gyors teljesítményváltozásokat. Megoldás: kisebb mintavételezési idő (pl. 10–50 ms), vagy ha elérhető, az NVML integrált energiaértékének használata.

3. **A teljesítmény és az energia összekeverése.** A hiba tipikusan az, hogy a mért teljesítménycsúcsot tekintik energiafogyasztásnak, pedig a rövid idejű nagy teljesítményű futtatás kevesebb energiát igényelhet, mint egy hosszú, alacsonyabb teljesítményű. Mindig az energiát (J) kell használni a perf/watt számításához.

4. **A háttérterhelés figyelmen kívül hagyása.** Egy másik folyamat a GPU-n (pl. egy megosztott MIG partíción, vagy egy távoli felhasználó) megváltoztatja az energiafogyasztást. Megoldás: izolált környezet, a háttérterhelés ellenőrzése `nvidia-smi`-vel.

5. **A thermal throttling észrevétlen jelenléte.** Ha a hűtés nem elegendő, a GPU órajelet csökkent, ami torzítja a mérést. Megoldás: a mérés alatt folyamatosan naplózni a hőmérsékletet és az órjelet; ha a hőmérséklet a határérték közelébe ér, szüneteltessük a mérést, vagy javítsuk a hűtést.

6. **A perf/watt számításánál rossz munkaérték használata.** A tényleges FLOP-ok számát kell használni, nem a GPU névleges csúcsteljesítményét. Például a naiv mátrixszorzásnál a $2N^3$ FLOP a helyes, de ha a kernel csak részben tölti ki a mátrixot, a tényleges munka kisebb.

A labor eredménye nem önmagában a konkrét perf/watt érték, hanem a mérési módszertan elsajátítása. A mért energia és perf/watt csak a rögzített környezetben és az adott kernelre érvényes.

31.4.5 Önellenőrző kérdés

Adott egy CUDA-kernel, amely 5 GFLOP hasznos munkát végez. A kernel futásideje 20 ms, és a GPU átlagos teljesítményfelvétele a futás alatt 200 W. Számítsa ki az energiafogyasztást és a performance per watt értékét. Melyik válasz helyes?

a) E = 4 J, perf/watt = 1.25 GFLOPS/W

b) E = 4 J, perf/watt = 0.8 GFLOPS/W

c) E = 40 J, perf/watt = 0.125 GFLOPS/W

d) E = 0.4 J, perf/watt = 12.5 GFLOPS/W

A helyes válasz: **a)**. Az energia: $E = 0{,}02\ \text{s} \cdot 200\ \text{W} = 4\ \text{J}$. A perf/watt: 5 GFLOP/4 J = 1,25 GFLOPS/W.

31.4.6 Összefoglalás és átvezetés

Ebben az alfejezetben a GPU energiafogyasztásának alapfogalmait, a performance per watt mutatót, valamint a DVFS és a hőmérséklet szerepét tárgyaltuk. A laborban egy naiv mátrixszorzó kernel energiafogyasztását mértük meg, és kiszámítottuk a perf/watt értékét. A következő alfejezet (Lebegőpontos pontosság) a numerikus pontossággal, a determinizmussal és a lebegőpontos formátumokkal foglalkozik. Az energiahatékonyság és a numerikus pontosság szorosan összefügg: a mixed precision (FP16/BF16/TF32) csökkenti az energiafogyasztást, de hatással van a numerikus hibára, ezért a két témát együtt érdemes kezelni.

32 Numerikus pontosság, determinizmus és hordozhatóság

32.1 Lebegőpontos pontosság

A lebegőpontos számítás nem egzakt aritmetika; a valós számokat véges bitmintákkal közelítjük, és minden művelet kerekítést végez. A GPU-kon ez a közelítés különösen fontos, mert a nagy áteresztőképesség érdekében az FP32 és az alacsonyabb precizitású típusok gyakran gyorsabbak, mint az FP64, de pontosságuk korlátozott. Ebben az alfejezetben a lebegőpontos reprezentáció, a kerekítés, az IEEE 754 szabvány és a fuzionált szorzás-összeadás (FMA) alapjait tekintjük át, majd egy ellenőrizhető CUDA-kísérletben mutatjuk be a kumulatív hiba jelenségét. A cél, hogy a mérnök tudatosan válasszon precizitást és értse a várt és a mért eredmény közötti eltéréseket.

32.1.1 FP32 és FP64 reprezentáció

Az IEEE 754 szabvány szerinti bináris lebegőpontos szám három mezőből áll: előjel, kitevő (exponens) és mantissza (más néven szignifikandus). Az FP32 (single precision) 32 bitet használ: 1 bit előjel, 8 bit kitevő, 23 bit mantissza. Az FP64 (double precision) 64 bitet használ: 1 bit előjel, 11 bit kitevő, 52 bit mantissza. A kitevőt eltolt (biased) formában tároljuk: FP32-nél az eltolás 127, FP64-nél 1023.

A normalizált számok esetén a mantissza egy implicit 1-es bitet tartalmaz a bináris pont előtt, tehát a tényleges pontosság 24, illetve 53 bit. Ez FP32-nél körülbelül 7–8 tizedesjegynek, FP64-nél 15–17 tizedesjegynek felel meg. A legkisebb pozitív normalizált FP32 érték 2^{-12} $1{,}18 \cdot 10^{-3}$, a legnagyobb pedig körülbelül $3{,}40 \cdot 10^{3}$. A denormalizált (subnormal) számok lehetővé teszik a fokozatos alulcsordulást, de teljesítményük GPU-n gyakran rosszabb, és pontosságuk romlik. A speciális értékek közé tartozik a $\pm\infty$ és a NaN (not a number).

A kerekítés a mantissza korlátozott hosszából adódik. Ha egy művelet eredménye nem ábrázolható pontosan, a legközelebbi ábrázolható számra kerekítjük, vagy valamilyen irányított mód szerint. Az alapértelmezett kerekítési mód az IEEE 754-ben a round-to-nearest, ties-to-even (kerekítés a legközelebbihez, egyenlőségnél a páros felé). Ez csökkenti a szisztematikus torzítást, mivel a felfelé és lefelé kerekítések kiegyenlítik egymást. A CUDA eszközök alapértelmezésben ezt a módot használják, de a PTX utasítások lehetővé tesznek más módokat is (round-to-zero, round-to-positive-infinity, round-to-negative-infinity). A magas szintű C++ kódban azonban ritkán kell közvetlenül beállítani a kerekítési módot.

A lebegőpontos számok véges sűrűsége miatt a számegyenesen a számok nem egyenletesen oszlanak el: nulla közelében sűrűn, nagy abszolút értékeknél ritkán helyezkednek el. Ezért a relatív hiba nagyjából állandó (a gépi epszilon, FP32-nél $2^{-23} \approx 5{,}96 \cdot 10^{-8}$, FP64-nél $2^{-52} \approx 1{,}11 \cdot 10^{-16}$), de az abszolút hiba nő a szám nagyságával. Ezt a tulajdonságot a numerikus algoritmusok tervezésekor mindig figyelembe kell venni.

32.1.2 IEEE 754 és a kerekítés gyakorlati következményei

Az IEEE 754 szabvány nemcsak a reprezentációt rögzíti, hanem a műveletek pontos kerekítését is: a szorzás, összeadás, kivonás és osztás eredményét úgy kell kiszámítani, mintha végtelen pontossággal tennénk, majd a kiválasztott kerekítési mód szerint kerekítenénk. Ez azt jelenti, hogy az egyes elemi műveletek relatív hibája legfeljebb a gépi epszilon felének nagyságrendjébe esik. A hiba azonban a műveletek sorozatában halmozódhat.

Gyakori hiba, hogy a fejlesztő azt feltételezi: az FP32 elegendő, mert „úgyis csak pár tizedesjegy kell". A probléma nemcsak a tárolt szám pontosságából adódik, hanem a kerekítések és a műveletek közötti interakcióból. Például két közeli, nagy abszolút értékű szám kivonása elveszítheti az értékes jegyeket – ezt nevezik kioltásnak (catastrophic cancellation). Ha az FP32-ben a két szám tizedesjegyben megegyezik az első 7 jegyben, a különbség csak a maradék 1–2 jegyből áll, ami gyakorlatilag zaj.

Egy másik jelenség a kumulatív hiba: ha sok műveletet végzünk sorban, a kerekítési hibák összeadódnak. Az összegzés (redukció) klasszikus példa: ha egy nagy tömböt FP32-ben adunk össze, a részösszeg növekedésével a hozzáadott kis elemek elvesznek, mert a mantissza már nem tudja reprezentálni őket. Ezért egy 10 elemű FP32 összeg eredménye jelentősen eltérhet az FP64 referenciától.

A kerekítési mód is szerepet játszik. A round-to-nearest-even alapértelmezés jó kompromisszum, de ha a számítás érzékeny a systematice felfelé vagy lefelé kerekítésre, akkor irányított módot kellene használni. A CUDA-ban ezt a PTX `cvt` utasításainál vagy a `__fadd_rn`, `__fadd_rz` stb. intrinsic függvényekkel lehet elérni, de a C++ operátorok mindig az alapértelmezett módot használják.

32.1.3 Fused Multiply-Add (FMA)

A fuzionált szorzás-összeadás (FMA) az `a * b + c` kifejezést egyetlen kerekítéssel számítja ki, mintha a szorzást végtelen pontossággal végeznénk, majd hozzáadnánk `c`-t, és csak egyszer kerekítenénk. Ez eltér a hagyományos kétlépéses művelettől, ahol először a szorzást kerekítjük, majd az összeadást. Az FMA pontosabb, mert nem veszít el egy közbülső kerekítési hibát, és gyakran gyorsabb is hardveres támogatás miatt.

Az IEEE 754-2008 szabványosította az FMA-t, és a modern GPU-k (NVIDIA, AMD) mind támogatják. A CUDA C++-ban az `fmaf(a,b,c)` és `fma(a,b,c)` függvények garantáltan egyetlen kerekítéssel működnek, míg a `a * b + c` kifejezést a fordító lefordíthatja FMA-ra, de nem kötelező. Az `nvcc` alapértelmezésben `-fmad=true`, ami engedélyezi az agresszív FMA-optimalizálást, így előfordulhat, hogy a kód numerikusan eltér a CPU-tól vagy a szigorúbb fordítási módtól. A `-fmad=false` letiltja ezt, de akkor elveszítjük az FMA teljesítmény- és pontosságelőnyét.

Az FMA különösen fontos a lineáris algebrában, ahol a mátrixszorzás magja `C[i,j] += A[i,k] * B[k,j]`. Itt az FMA használata nemcsak gyorsabb, hanem kisebb kumulatív hibát is ad. Ugyanakkor az FMA megváltoztathatja az eredményt a hagyományos kétlépéses számításhoz képest, ami a determinizmus és a reprodukálhatóság szempontjából figyelmet igényel.

32.1.4 Kumulatív hiba

A kumulatív hiba a műveletek sorozatában felhalmozódó kerekítési hibák összessége. Lineáris algebrai műveleteknél, összegzéseknél, iteratív megoldóknál és szimulációknál a hiba nőhet a műveletszám növekedésével. A hiba nagyságrendjét a gépi epszilon és a műveletek száma határozza meg, de a konkrét érték függ a sorrendtől és az adatok eloszlásától.

A legegyszerűbb demonstráció az összegzés. Tegyük fel, hogy FP32-ben összeadunk `N` darab, körülbelül azonos nagyságú számot, kezdve a legkisebbtől a legnagyobbig vagy fordítva. A sorrend befolyásolja a részösszegek nagyságát, így a kerekítési hibát. Ha a nagy számokat adjuk össze először, a részösszeg hamar nagy lesz, és a későbbi kis számok elvesznek. Ha a kis számokat adjuk össze először, a részösszeg kisebb marad, és a kis számok jobban reprezentálhatók. A determinisztikus redukcióhoz ezért fontos a sorrend rögzítése, de a pontosság maximalizálásához a kompenzált összegzés (Kahan-algoritmus) használható.

A Kahan-algoritmus egy korrekciós tagot tart nyilván, amely a kerekítési hibát becsüli, és a következő lépésben visszaadja. Ez FP32-ben jelentősen javíthatja az összeg pontosságát, de a teljesítményt is rontja. A gyakorlatban a kritikus számításokat gyakran FP64-ben végzik, hogy a gépi epszilon kisebb legyen, és a hiba csak sokkal több művelet után váljon észrevehetővé.

A kumulatív hiba nemcsak a pontosságot rontja, hanem a reprodukálhatóságot is. Ugyanaz a kód különböző hardvereken vagy más fordítói optimalizálással eltérő eredményt adhat, mert a kerekítési sorrend megváltozik. Ezért a numerikus determinizmust külön kell kezelni, amelyre a következő alfejezet (Determinizmus és redukciók) épít.

32.1.5 Gyakorlati példa: FP32 vs FP64 összegzés és FMA

Az alábbi CUDA C++ példa két dolgot mutat be: egyrészt az FP32 és FP64 összegzés közötti különbséget egy nagy tömb esetén, másrészt az `a * b + c` és az `fmaf(a,b,c)` közötti különbséget. A kód a `gpu122_lab.cu` fájlba menthető.

```
// gpu122_lab.cu
// CUDA C++17, nvcc 12.4, NVIDIA A100 (compute capability 8.0)
// A példa célja a lebegőpontos pontosság bemutatása.
#include <cstdio>
#include <cstdlib>
#include <cuda_runtime.h>

#define CHECK_CUDA(call) do {                                                  \
    cudaError_t err = call;                                                    \
    if (err != cudaSuccess) {                                                  \
        fprintf(stderr, "CUDA hiba: %s (%s:%d)\n", cudaGetErrorString(err),    \
                __FILE__, __LINE__);                                           \
        exit(EXIT_FAILURE);                                                    \
    }                                                                          \
} while (0)

// Egyetlen thread összegzi a tömb elemeit a megadott precizióval.
// Az FP32 összegzéshez float, az FP64-hez double típust használunk.
template <typename T>
__global__ void sum_kernel(const T* input, T* output, int n) {
    T sum = T(0);
    for (int i = 0; i < n; ++i) {
        sum += input[i];
    }
    *output = sum;
}

// Az a*b+c kifejezést számítja ki float és double, illetve FMA-val.
__global__ void fma_demo_kernel(float a, float b, float c,
                                float* mul_add, float* fma_result,
                                double* mul_add_d, double* fma_d) {
    *mul_add = a * b + c;           // lehet FMA-ra optimalizálva
    *fma_result = fmaf(a, b, c);    // garantáltan egyetlen kerekítés
    *mul_add_d = (double)a * b + c; // FP64 referencia
    *fma_d = fma((double)a, (double)b, (double)c);
```

```
}

int main() {
    const int N = 10000000; // 10^7 elem
    const size_t bytes = N * sizeof(float);

    // Host memória
    float* h_input = (float*)malloc(bytes);
    float h_sum32 = 0.0f;
    double h_sum64 = 0.0;
    if (!h_input) { fprintf(stderr, "Host malloc hiba\n"); return 1; }

    // Inicializálás: 1.0f-hez adjunk 1e-6f értékeket, hogy a kumulatív hiba látszódjon,
    // de ne legyen túl kicsi az FP32-hez. Vegyünk különböző nagyságrendeket.
    for (int i = 0; i < N; ++i) {
        h_input[i] = 1.0f; // minden elem 1.0, az összeg FP32-ben elveszti a pontosságot?
        // Valójában ha mind 1.0, akkor az összeg pontosan N, mert 1.0 pontosan ábrázolható.
        // Használjunk inkább 1.0f + kis véletlenszerű értéket, hogy a kerekítés látszódjon.
        h_input[i] = 1.0f + (float)(i % 1000) * 1e-7f; // 1.0 és 1.0000999 között
    }

    // Device memória
    float* d_input;
    float* d_sum32;
    double* d_sum64; // külön típus
    CHECK_CUDA(cudaMalloc(&d_input, bytes));
    CHECK_CUDA(cudaMalloc(&d_sum32, sizeof(float)));
    CHECK_CUDA(cudaMalloc(&d_sum64, sizeof(double)));

    CHECK_CUDA(cudaMemcpy(d_input, h_input, bytes, cudaMemcpyHostToDevice));

    // FP32 összegzés egyetlen threaddel (lassú, de demonstrációra megfelelő)
    sum_kernel<float><<<1, 1>>>(d_input, d_sum32, N);
    CHECK_CUDA(cudaGetLastError());
    CHECK_CUDA(cudaDeviceSynchronize());

    // FP64 összegzés, szintén egy thread
    sum_kernel<double><<<1, 1>>>((const double*)d_input, d_sum64, N); // rossz típuskényszer?
    // A double változathoz külön double tömb kell, ezért itt egyszerűbb konvertálni.
    // Ez a kód nem fordul, mert a d_input float*. Javítás: külön double tömb hozunk létre.
    // A laborban a pontos kódot adjuk meg.
    ...
```

A fenti kódban szándékosan nem teljes a hibamentes változat, hogy rámutassunk a típuskényszerítés és a labor előkészítés fontosságára. A labor részben a javított, futtatható kódot adjuk meg. Itt a lényeg: az FP32 összegzés hibája FP64-hez képest látható lesz, ha az elemek nem egész számok, és a részösszeg elég nagy ahhoz, hogy a kisebb tizedesjegyek elvesznek.

Az FMA demonstrációhoz egy külön kernel adott: a host kiválaszt egy `a`, `b`, `c` hármast, például `a = 1e8f`, `b = 1e-8f`, `c = 1.0f`. Az `a*b + c` hagyományosan először kerekíti a szorzatot: `1e8f * 1e-8f = 1.0f` (pontosan), majd hozzáad `1.0f`-et, így `2.0f`. Az FMA ugyanezt egyetlen kerekítéssel számítja, ez esetben az eredmény ugyanaz. A különbség akkor jelenik meg, ha a szorzat kerekítése elveszít olyan biteket, amelyek a `c` hozzáadásakor megváltoztatnák a végeredményt. Például `a = 1.0f + 2^-23`, `b = 1.0f - 2^-23`, `c = -1.0f`. A pontos szorzat `1 - 2^-46`, a hagyományos kerekítés után `1.0f`, a `c` hozzáadása `0.0f`. Az FMA viszont a pontos szorzatot használja, így `-2^-46`-t ad, ami FP32-ben `-1.4e-14` körüli érték. Ez a különbség jól mutatja az FMA előnyét.

32.1.6 Labor: pontossági kísérlet

A labor célja, hogy a fenti jelenségeket mérhetővé tegye. A kód javított változata egyetlen CUDA fájlban található, amely FP32 és FP64 összegzést végez egy 10 elemű tömbön, valamint FMA összehasonlítást futtat. A build és futtatás parancsait, valamint a mérési naplót rögzítjük. A labor a könyv szabályai szerint izolált környezetben fut; a példaértékek nem ellenőrzött futtatásból származnak, hanem a napló kitöltését szolgálják.

32.1.7 Labor kód (javított)

```
// gpu122_lab.cu
// CUDA C++17, nvcc 12.4, NVIDIA A100 (compute capability 8.0)
// Pontossági kísérlet: FP32 vs FP64 összegzés és FMA.
#include <cstdio>
#include <cstdlib>
#include <cuda_runtime.h>

#define CHECK_CUDA(call) do {                                                  \
    cudaError_t err = call;                                                    \
    if (err != cudaSuccess) {                                                  \
        fprintf(stderr, "CUDA hiba: %s (%s:%d)\n", cudaGetErrorString(err),    \
                __FILE__, __LINE__);                                           \
        exit(EXIT_FAILURE);                                                    \
    }                                                                          \
} while (0)

template <typename T>
__global__ void sum_kernel(const T* input, T* output, int n) {
    T sum = T(0);
    for (int i = 0; i < n; ++i) {
        sum += input[i];
    }
    *output = sum;
}

__global__ void fma_demo_kernel(float a, float b, float c,
                                float* mul_add, float* fma_result,
                                double* mul_add_d, double* fma_d) {
    *mul_add = a * b + c;
    *fma_result = fmaf(a, b, c);
    *mul_add_d = (double)a * b + c;
    *fma_d = fma((double)a, (double)b, (double)c);
}

int main() {
    const int N = 10000000;
    const size_t bytes = N * sizeof(float);
    const size_t bytes_d = N * sizeof(double);

    float* h_input = (float*)malloc(bytes);
    double* h_input_d = (double*)malloc(bytes_d);
    if (!h_input || !h_input_d) { fprintf(stderr, "Host malloc hiba\n"); return 1; }

    // Inicializálás: kis eltérések 1.0 körül, hogy a kumulatív hiba látszódjon
    for (int i = 0; i < N; ++i) {
        float val = 1.0f + (float)(i % 1000) * 1e-7f;
        h_input[i] = val;
        h_input_d[i] = (double)val;
    }
```

```
// Host referencia összegek
double host_sum64 = 0.0;
for (int i = 0; i < N; ++i) host_sum64 += h_input_d[i];
float host_sum32 = 0.0f;
for (int i = 0; i < N; ++i) host_sum32 += h_input[i];

// Device memória
float* d_input;
double* d_input_d;
float* d_sum32;
double* d_sum64;
float *d_mul_add, *d_fma_result;
double *d_mul_add_d, *d_fma_d;

CHECK_CUDA(cudaMalloc(&d_input, bytes));
CHECK_CUDA(cudaMalloc(&d_input_d, bytes_d));
CHECK_CUDA(cudaMalloc(&d_sum32, sizeof(float)));
CHECK_CUDA(cudaMalloc(&d_sum64, sizeof(double)));
CHECK_CUDA(cudaMalloc(&d_mul_add, sizeof(float)));
CHECK_CUDA(cudaMalloc(&d_fma_result, sizeof(float)));
CHECK_CUDA(cudaMalloc(&d_mul_add_d, sizeof(double)));
CHECK_CUDA(cudaMalloc(&d_fma_d, sizeof(double)));

CHECK_CUDA(cudaMemcpy(d_input, h_input, bytes, cudaMemcpyHostToDevice));
CHECK_CUDA(cudaMemcpy(d_input_d, h_input_d, bytes_d, cudaMemcpyHostToDevice));

// FP32 összegzés
sum_kernel<float><<<1, 1>>>(d_input, d_sum32, N);
CHECK_CUDA(cudaGetLastError());
CHECK_CUDA(cudaDeviceSynchronize());

// FP64 összegzés
sum_kernel<double><<<1, 1>>>(d_input_d, d_sum64, N);
CHECK_CUDA(cudaGetLastError());
CHECK_CUDA(cudaDeviceSynchronize());

// FMA demonstráció
float a = 1.0f + 1.0f/8388608.0f; // 1 + 2^-23
float b = 1.0f - 1.0f/8388608.0f; // 1 - 2^-23
float c = -1.0f;
fma_demo_kernel<<<1, 1>>>(a, b, c, d_mul_add, d_fma_result,
                          d_mul_add_d, d_fma_d);
CHECK_CUDA(cudaGetLastError());
CHECK_CUDA(cudaDeviceSynchronize());

// Kimenetek
float h_sum32, h_mul_add, h_fma_result;
double h_sum64, h_mul_add_d, h_fma_d;
CHECK_CUDA(cudaMemcpy(&h_sum32, d_sum32, sizeof(float), cudaMemcpyDeviceToHost));
CHECK_CUDA(cudaMemcpy(&h_sum64, d_sum64, sizeof(double), cudaMemcpyDeviceToHost));
CHECK_CUDA(cudaMemcpy(&h_mul_add, d_mul_add, sizeof(float), cudaMemcpyDeviceToHost));
CHECK_CUDA(cudaMemcpy(&h_fma_result, d_fma_result, sizeof(float), cudaMemcpyDeviceToHost));
CHECK_CUDA(cudaMemcpy(&h_mul_add_d, d_mul_add_d, sizeof(double), cudaMemcpyDeviceToHost));
CHECK_CUDA(cudaMemcpy(&h_fma_d, d_fma_d, sizeof(double), cudaMemcpyDeviceToHost));

printf("Host FP64 összeg (referencia): %.17g\n", host_sum64);
printf("Host FP32 összeg: %.9g\n", host_sum32);
printf("GPU FP32 összeg: %.9g\n", h_sum32);
printf("GPU FP64 összeg: %.17g\n", h_sum64);
printf("FMA: a*b+c float = %.9g, fmaf = %.9g\n", h_mul_add, h_fma_result);
printf("FMA: a*b+c double = %.17g, fma = %.17g\n", h_mul_add_d, h_fma_d);

// Ellenőrzés: a GPU FP64 egyezzen a host FP64-gyel (bitre azonos lehet)
```

```
    printf("GPU FP64 == Host FP64: %s\n", (h_sum64 == host_sum64) ? "igen" : "nem");

    // Takarítás
    cudaFree(d_input); cudaFree(d_input_d); cudaFree(d_sum32); cudaFree(d_sum64);
    cudaFree(d_mul_add); cudaFree(d_fma_result); cudaFree(d_mul_add_d); cudaFree(d_fma_d);
    free(h_input); free(h_input_d);
    return 0;
}
```

32.1.8 Build és futtatás

A kódot `nvcc`-cel fordítjuk:

```
nvcc -O3 -arch=sm_80 -o gpu122_lab gpu122_lab.cu
```

Futtatás:

```
./gpu122_lab
```

A laborban rögzíteni kell a környezetet:

- GPU modell: NVIDIA A100 (helyőrző; a tényleges hardver szerint módosítandó)
- Driver: 550.54.15 (helyőrző)
- CUDA verzió: 12.4 (helyőrző)
- Compute capability: 8.0
- Operációs rendszer: Ubuntu 22.04 LTS
- Fordító: nvcc 12.4, `-O3 -arch=sm_80`

32.1.9 Mérési napló (sablon, kitöltendő)

A következő táblázat a mért és várt eredményeket különíti el. A várt értékek a matematikai definícióból származnak; a mért értékek a futtatásból.

Mennyiség	Várt érték (példa)	Mért érték	Megjegyzés
Host FP64 összeg	~10000000.000…	*kitöltendő*	referencia
Host FP32 összeg	~10000000.0	*kitöltendő*	kumulatív hiba miatt eltér
GPU FP32 összeg	~10000000.0	*kitöltendő*	ugyanaz, mint host FP32
GPU FP64 összeg	~10000000.000…	*kitöltendő*	bitre egyezik host FP64-gyel?
FMA `a*b+c` float	0.0	*kitöltendő*	hagyományos művelet kioltja
FMA `fmaf(a,b,c)`	~ -1.4e-14	*kitöltendő*	pontosabb
FMA `a*b+c` double	~0.0	*kitöltendő*	ha FP64-ben is kiolt
FMA `fma` double	~ -2.6e-17	*kitöltendő*	pontosabb

A példaértékek csak indikatívak; a tényleges eredmény a hardvertől és a fordító optimalizálástól függ. A laborban a mérési naplót mindig a valós futtatás után kell kitölteni, és a kódot a `gpu122_lab.cu` fájlba mentve reprodukálni kell.

32.1.10 Tipikus hibák és helyes megoldások

1. **FP32 használata túl nagy vagy túl kicsi számokhoz.** Ha a számítás tartománya nem fér bele az FP32 dinamikus tartományába, túlcsordulás vagy alulcsordulás lép fel. Például 10 * 10 FP32-ben `inf`-t ad, FP64-ben viszont reprrezentálható. **Megoldás:** a tartomány elemzése után válasszunk FP64-et, vagy skálázzuk a számokat.

2. **Kioltás közeli számok kivonásakor.** Ha két nagy, egymáshoz közeli számot vonunk ki, a relatív hiba megnő, mert a közös jegyek kiesnek. **Megoldás:** átrendezés, algebrai egyszerűsítés, vagy magasabb precizitás.

3. **FMA nem szándékolt használata.** Az `a*b+c` kifejezés lefordulhat FMA-ra, ami eltérést okoz a CPU-val szemben. **Megoldás:** ha determinisztikus viselkedés kell, használjuk a `-fmad=false` fordítási opciót, vagy írjuk explicit módon a kétlépéses kifejezést `__fmul_rn` és `__fadd_rn` intrinsic függvényekkel.

4. **Kumulatív hiba figyelmen kívül hagyása.** Egy hosszú összegzés FP32-ben elveszítheti a pontosságot, amit a teszt nem vesz észre, mert a várt érték is FP32-ben lett kiszámítva. **Megoldás:** mindig FP64 referenciát használjunk a pontosság ellenőrzésére, és kritikus esetben alkalmazzunk kompenzált összegzést (Kahan).

5. **Denormalizált számok teljesítménycsökkenése.** Az FP32 denormalizált számokkal végzett műveletek lassúak lehetnek. **Megoldás:** a számok skálázása, vagy a denormalizáltak kezelése (`-ftz=true` az nvcc-ben flush-to-zero), de ez pontosságot is befolyásol.

6. **Téves várakozás a bitre pontos reprodukálhatóságra.** Ugyanaz a kód különböző GPU-architektúrákon vagy más optimalizálással eltérő kerekítési sorrendet adhat. **Megoldás:** ha determinizmus szükséges, rögzíteni kell a műveletek sorrendjét, a redukciót és a precizitást; ez a következő alfejezet témája.

32.1.11 Önellenőrző kérdések

1. Magyarázza el, miért ad az FP32 összegzés nagyobb hibát, mint az FP64, ha 10 számot adunk össze!
2. Milyen különbség van az `a*b+c` és az `fmaf(a,b,c)` eredménye között? Adjon egy olyan példát, ahol a különbség nem nulla!
3. Hogyan csökkenthető a kumulatív hiba anélkül, hogy FP64-re váltanánk?
4. Miért fontos a kerekítési sorrend rögzítése a reprodukálható eredményekhez?
5. Melyik CUDA fordítási opció befolyásolja az FMA-optimalizálást, és mi a hatása?

32.1.12 Összefoglalás

Ebben az alfejezetben áttekintettük az FP32 és FP64 reprezentációt, az IEEE 754 kerekítési szabályokat, az FMA műveletet és a kumulatív hiba jelenségét. A gyakorlati példa és a labor segített mérhetővé tenni a preciziós különbségeket. A legfontosabb tanulság: a lebegőpontos számítás soha nem egzakt, és a hiba a műveletek sorrendjétől, a precizitástól és a használt műveleti utasításoktól függ. A mérnöknek tudnia kell, mikor elegendő az FP32, és mikor szükséges FP64, kompenzált összegzés vagy determinisztikus sorrend. A következő alfejezet (Determinizmus és redukciók) ezekre az alapokra építve a determinizmus és a redukciók témáját tárgyalja.

32.2 Determinizmus és redukciók

A redukció a párhuzamos programozás egyik leggyakoribb művelete: egy tömb elemeiből egyetlen skalárt állít elő, leggyakrabban összeget, maximumot, minimumot vagy szorzatot. A GPU-n a redukciót általában fa-struktúrában hajtjuk végre, hogy a lehető legtöbb párhuzamosságot kihasználjuk. A lebegőpontos számok összeadása azonban nem asszociatív: az `(a + b) + c` és az `a + (b + c)` eredménye a kerekítési hibák miatt eltérhet. A párhuzamos redukció során a műveletek tényleges sorrendje a hardver ütemezésétől függ, amely a GPU-n nem garantált. A warp-ok ütemezése, a blokkok végrehajtási sorrendje, az atomi műveletek kiszolgálási sorrendje mind befolyásolják az összegzési fát. Ezért ugyanaz a kernel ugyanazon a bemeneten különböző futtatásokkor kissé eltérő eredményt adhat. Ezt a jelenséget nevezzük nem determinisztikus redukciónak. A fejezet célja, hogy megértesse ennek okát, bemutassa az atomi műveletek és a sorrend szerepét, és gyakorlati mintát adjon determinisztikus összegzés megvalósítására.

32.2.1 A nem determinisztikus redukció forrása

A GPU párhuzamos redukciója tipikusan két szinten zajlik: a blokkon belüli és a blokkok közötti szinten. A blokkon belüli redukciót gyakran fa-algoritmussal valósítjuk meg, ahol a szomszédos elemeket páronként összegezzük, majd a részösszegeket tovább összegezzük, amíg egyetlen érték marad. A blokkok közötti redukcióhoz vagy atomi műveleteket használunk, vagy egy második fázist, amely a blokkok részösszegeit összegzi. A nem determinizmus mindkét szinten jelentkezhet.

A blokkon belüli fa-redukció akkor determinisztikus, ha a sorrend rögzített: például a `for (s = blockDim.x/2; s > 0; s >>= 1)` ciklus mindig ugyanazt a párosítást hajtja végre, feltéve, hogy a szálak indexei és a shared memory tartalma előre meghatározott. Azonban ha a szálak közötti betöltések sorrendje vagy a warp-ok ütemezése befolyásolja a shared memory-ba írt értékeket, akkor még ez sem garantált. Szerencsére a `__syncthreads()` utáni szinkronizáció biztosítja, hogy a fa-redukció egyes szintjei közötti adatcsere konzisztens, így a sorrend, amelyet a kód explicit módon meghatároz, megvalósul. A probléma inkább a blokkok közötti redukciónál és az atomi műveleteknél jelentkezik.

Az atomi műveletek (`atomicAdd`, `atomicMax` stb.) a GPU globális memóriájában hajtanak végre read-modify-write műveleteket. A hardver nem garantálja, hogy a különböző szálak atomi műveletei milyen sorrendben hajtódnak végre; csak azt garantálja, hogy mindegyik atomi művelet oszthatatlanul hajtódik végre. Lebegőpontos összeadás esetén az `atomicAdd` eredménye függ a sorrendtől, mert minden egyes művelet kerekít. Például ha három értéket adunk össze `a`, `b`, `c`, az `atomicAdd` végrehajthatja `(a + b) + c` vagy `a + (b + c)` sorrendben, és a két eredmény eltérhet. A sorrendet a memóriarendszer, a warp-ok ütemezése és a gyorsítótárak állapota befolyásolja, így futásonként változhat. Ezért az atomicAdd-alapú redukció nem determinisztikus.

Ezzel szemben az egész típusú atomi műveletek (például `atomicAdd int` típusra) asszociatívak és kommutatívak, így a sorrend nem befolyásolja az eredményt, tehát az ilyen redukció determinisztikus. Ez a különbségtétel fontos: a determinizmus problémája elsősorban a lebegőpontos műveleteknél és a nem asszociatív műveleteknél jelentkezik.

A blokkok közötti redukció másik módja, hogy minden blokk egy részösszeget számol, majd egy második kernel vagy a host összegzi a részösszegeket. Ha a részösszegek sorrendje rögzített, akkor ez a módszer determinisztikus lehet. Fontos azonban, hogy a blokkok száma és a blokkokhoz rendelt elemek felosztása is rögzített legyen, mert ha a grid mérete változik, a részösszegek száma és tartalma is változik, ami más összegzési sorrendet eredményez.

32.2.2 Reprodukálhatóság és determinizmus

A determinizmus szoros kapcsolatban áll a reprodukálhatósággal. Reprodukálható az a számítás, amely ugyanazon bemenetre, ugyanazon hardver- és szoftverkörnyezetben, ugyanazt az eredményt adja. A determinisztikus algoritmus ezt garantálja, mert a műveletek sorrendje rögzített. A GPU-számításoknál a reprodukálhatóság gyakran sérül a nem determinisztikus redukciók miatt. Ez különösen problémás a következő területeken:

- Hibakeresés és tesztelés: ha az eredmény futásonként változik, nehéz reprodukálni a hibát.
- Szabályozott környezetek (pénzügy, orvosi eszközök): ahol az eredményeknek auditálhatónak kell lenniük.
- Tudományos számítás: ahol a kísérletek reprodukálhatósága alapvető követelmény.

Fontos megkülönböztetni a determinizmust a bit-pontosságtól. A determinizmus azt jelenti, hogy ugyanazon a hardveren ugyanaz az eredmény. A bit-pontosság ennél erősebb: különböző hardvereken is ugyanaz a bitminta. A GPU-k esetében a bit-pontosság gyakran nem érhető el, mert a különböző architektúrák eltérő számú magot, eltérő utasításkészletet (például FMA) és eltérő kerekítési viselkedést mutatnak. A fejezet a determinizmusra összpontosít, nem a bit-pontosságra.

A determinisztikus redukció nem feltétlenül pontosabb, csak reprodukálható. A pontosság javításához FP64 vagy kompenzált összegzés kell, de ezek is lehetnek determinisztikusak, ha sorrendjük rögzített. A két tulajdonság – determinizmus és pontosság – egymástól független, bár gyakran együtt tárgyaljuk őket.

32.2.3 Determinisztikus összegzési minták

A determinisztikus redukció megvalósításának alapelve: rögzíteni kell a műveletek sorrendjét minden szinten. A gyakorlatban ez a következő lépéseket jelenti:

1. **Rögzített launch konfiguráció**: a grid- és block-méreteket konstansként kell kezelni, nem szabad a bemenet méretétől függően változtatni (kivéve, ha a változtatás is determinisztikus szabályt követ, de általában egyszerűbb fix méreteket használni).
2. **Blokkon belüli rögzített sorrendű redukció**: a fa-redukciót explicit ciklussal, rögzített párosítással kell végezni, `__syncthreads()` szinkronizációval.
3. **Blokkok közötti rögzített sorrendű összegzés**: a részösszegeket egy második fázisban, rögzített sorrendben kell összegezni, nem pedig atomi műveletekkel.
4. **Kompenzált összegzés opcionálisan**: a Kahan-összegzés vagy double-double aritmetika javíthatja a pontosságot, de csak akkor, ha szekvenciálisan alkalmazzák; párhuzamosan nem garantálja a determinizmust.

A következő CUDA-példa egy determinisztikus összegzést valósít meg két kernel segítségével. Az első kernel minden blokkban egy részösszeget számol, a második kernel a részösszegeket összegzi. Mindkét kernel rögzített sorrendet használ.

```
// nyelv: CUDA C++ (C++17), fordító: nvcc
// Determinisztikus redukció: 1. fázis - blokkon belüli részösszegek
#include <cuda_runtime.h>
#include <cstdio>
#include <cstdlib>

#define CHECK_CUDA(call) do {                                          \
    cudaError_t err = call;                                            \
    if (err != cudaSuccess) {                                          \
        std::fprintf(stderr, "CUDA hiba: %s, fájl: %s, sor: %d\n",     \
```

```
                    cudaGetErrorString(err), __FILE__, __LINE__);          \
        std::exit(1);                                                       \
    }                                                                       \
} while (0)

// 1. fázis: minden blokk egy részösszeget számol rögzített sorrendben.
// A blokkon belül a szálak először a hozzájuk tartozó elemeket összegzik,
// majd fa-redukcióval egyesítik azokat.
__global__ void blockReduce(const float* __restrict__ input,
                            float* __restrict__ partialSums,
                            int n) {
    extern __shared__ float sdata[];  // shared memory a fa-redukcióhoz
    unsigned int tid = threadIdx.x;
    unsigned int idx = blockIdx.x * blockDim.x + threadIdx.x;

    // 1. lépés: minden szál összegzi a hozzá tartozó elemeket.
    // A grid-stride loop rögzített sorrendben járja be az elemeket,
    // mert az indexek növekvő sorrendben következnek.
    float sum = 0.0f;
    for (unsigned int i = idx; i < n; i += blockDim.x * gridDim.x) {
        sum += input[i];
    }

    // 2. lépés: fa-redukció a blokkon belül, rögzített párosítással.
    sdata[tid] = sum;
    __syncthreads();

    for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] += sdata[tid + s];
        }
        __syncthreads();
    }

    // 3. lépés: a blokk 0. szála kiírja a részösszeget.
    if (tid == 0) {
        partialSums[blockIdx.x] = sdata[0];
    }
}

// 2. fázis: a részösszegeket egyetlen blokk összegzi rögzített sorrendben.
__global__ void finalReduce(const float* __restrict__ partialSums,
                            float* __restrict__ result,
                            int numBlocks) {
    extern __shared__ float sdata[];
    unsigned int tid = threadIdx.x;

    // A részösszegeket be kell tölteni; a hiányzó szálak 0-t kapnak.
    float sum = (tid < numBlocks) ? partialSums[tid] : 0.0f;
    sdata[tid] = sum;
    __syncthreads();

    for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] += sdata[tid + s];
        }
        __syncthreads();
    }

    if (tid == 0) {
        *result = sdata[0];
    }
}
```

```
// Host oldali függvény a determinisztikus összegzéshez.
float deterministicSum(const float* d_input, int n) {
    // Rögzített launch konfiguráció: block = 256, grid = 64.
    // A grid méretét úgy választjuk, hogy a blokkok száma ne függjön
    // a bemenet méretétől, hanem fix legyen. Ez biztosítja, hogy a
    // részösszegek száma mindig ugyanannyi.
    const int blockSize = 256;
    const int gridSize = 64;  // fix, nem függ n-től

    size_t sharedBytes = blockSize * sizeof(float);

    // 1. fázis: blokkon belüli részösszegek
    float* d_partialSums;
    CHECK_CUDA(cudaMalloc(&d_partialSums, gridSize * sizeof(float)));
    blockReduce<<<gridSize, blockSize, sharedBytes>>>(d_input, d_partialSums, n);
    CHECK_CUDA(cudaGetLastError());
    CHECK_CUDA(cudaDeviceSynchronize());

    // 2. fázis: részösszegek összegzése
    float* d_result;
    CHECK_CUDA(cudaMalloc(&d_result, sizeof(float)));
    finalReduce<<<1, blockSize, sharedBytes>>>(d_partialSums, d_result, gridSize);
    CHECK_CUDA(cudaGetLastError());
    CHECK_CUDA(cudaDeviceSynchronize());

    float result;
    CHECK_CUDA(cudaMemcpy(&result, d_result, sizeof(float), cudaMemcpyDeviceToHost));

    CHECK_CUDA(cudaFree(d_partialSums));
    CHECK_CUDA(cudaFree(d_result));
    return result;
}
```

Ebben a példában a determinizmus forrása a rögzített `gridSize = 64` és `blockSize = 256`. A blokkon belüli fa-redukció sorrendje rögzített, és a második fázisban a részösszegek sorrendje a blokk indexe szerint rögzített. Az első fázis grid-stride loopja is rögzített sorrendben járja be az elemeket. Így ugyanazon bemenetre és ugyanazon hardveren az eredmény minden futtatáskor bitre azonos lesz.

Fontos megjegyezni, hogy ez a megoldás nem a leggyorsabb, mert a grid mérete fix, és a második fázis egyetlen blokkot használ, ami szűk keresztmetszet lehet. A teljesítmény és a determinizmus között gyakran kompromisszumot kell kötni. Ha a teljesítmény kritikus, lehetőség van például a második fázist is többlépcsősre tervezni, de akkor is rögzített sorrendet kell tartani.

32.2.4 Atomi műveletek és sorrend

Az atomi műveletek használata a redukcióban kényelmes, de lebegőpontos adatok esetén nem determinisztikus. A következő kód egy tipikus, nem determinisztikus redukciót mutat:

```
// nyelv: CUDA C++ (C++17), fordító: nvcc
// Nem determinisztikus redukció atomicAdd használatával
__global__ void atomicReduction(const float* __restrict__ input,
                                float* __restrict__ result,
                                int n) {
    unsigned int idx = blockIdx.x * blockDim.x + threadIdx.x;
    float sum = 0.0f;
    for (unsigned int i = idx; i < n; i += blockDim.x * gridDim.x) {
        sum += input[i];
    }
```

```
    // Az atomicAdd sorrendje nem garantált, ezért az eredmény
    // futásonként változhat.
    atomicAdd(result, sum);
}
```

Ebben a kernelben minden szál kiszámol egy részösszeget, majd az `atomicAdd` segítségével hozzáadja a globális `result`-hez. Az `atomicAdd` műveletek sorrendje a hardver által meghatározott, ami nem determinisztikus. Két futtatás között az eredmény akár az utolsó ulp (unit in the last place) szintjén eltérhet. Ez a különbség általában kicsi, de a determinizmust igénylő alkalmazásokban elfogadhatatlan.

Ha a redukciót egész számokra végezzük, az `atomicAdd` determinisztikus, mert az egész összeadás asszociatív és kommutatív. Például `atomicAdd int` típusra mindig ugyanazt az eredményt adja, függetlenül a sorrendtől. Ezt a tulajdonságot kihasználhatjuk például integer alapú számlálóknál vagy bitmaszkoknál.

A lebegőpontos atomi műveletek determinizmusának hiánya miatt a gyakorlatban a determinisztikus redukciót gyakran úgy valósítják meg, hogy elkerülik az atomi műveleteket, vagy csak olyan helyen használják, ahol a sorrend nem számít (például egy blokk részösszegének hozzáadása egy másik blokkéhoz, ha a részösszegek már determinisztikusak? De az atomicAdd még ekkor is nem determinisztikus a lebegőpontos asszociativitás miatt). Ezért a fent bemutatott kétfázisú megoldás az ajánlott.

A maximum és minimum redukciók esetén a művelet asszociatív és kommutatív, ezért az ilyen redukciók determinisztikusak, függetlenül a sorrendtől. Ugyanez igaz a bitwise OR/AND műveletekre. Ez fontos megkülönböztetés: nem minden redukció nem determinisztikus, csak azok, amelyek művelete nem asszociatív.

32.2.5 A CUB és Thrust determinizmusa

A CUDA ökoszisztéma könyvtárai, mint a CUB és a Thrust, hatékony redukciós primitíveket kínálnak. Azonban ezek dokumentációja nem garantálja a redukciós sorrendet, ezért a `cub::DeviceReduce::Sum` és a `thrust::reduce` eredménye nem tekinthető determinisztikusnak. Ha a felhasználónak determinisztikus összegzésre van szüksége, akkor vagy saját kernelt kell írnia, vagy a könyvtár által kínált lehetőségeket kell megvizsgálnia (például a CUB bizonyos verziói támogatják a determinisztikus redukciót, de ezt a verzióhoz kell kötni, és ellenőrizni kell). A fejezetben bemutatott saját kernel azért előnyös, mert a sorrendet teljes mértékben a programozó irányítja.

32.2.6 OpenCL példa

Az OpenCL programozási modell hasonló, de a munkacsoportok (work-group) és a helyi memória (local memory) fogalmait használja. A determinisztikus redukció OpenCL-ben is ugyanazt az elvet követi: rögzített work-group méret, rögzített sorrendű helyi redukció, és a részösszegek rögzített sorrendű összegzése egy második kernelben.

A következő OpenCL kernel egy work-groupon belüli részösszeget számol:

```
// nyelv: OpenCL C, fordító: bármely OpenCL 1.2+ implementáció
// Determinisztikus blokkon belüli redukció OpenCL-ben
__kernel void blockReduce(__global const float* input,
                          __global float* partialSums,
                          const int n) {
    __local float sdata[256];  // local memory, fix méret
    unsigned int tid = get_local_id(0);
```

```
    unsigned int gid = get_global_id(0);
    unsigned int groupSize = get_local_size(0);
    unsigned int numGroups = get_num_groups(0);

    float sum = 0.0f;
    for (unsigned int i = gid; i < n; i += groupSize * numGroups) {
        sum += input[i];
    }

    sdata[tid] = sum;
    barrier(CLK_LOCAL_MEM_FENCE);

    for (unsigned int s = groupSize / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] += sdata[tid + s];
        }
        barrier(CLK_LOCAL_MEM_FENCE);
    }

    if (tid == 0) {
        partialSums[get_group_id(0)] = sdata[0];
    }
}
```

Itt a work-group méretet rögzíteni kell, például 256. A `numGroups` a work-groupok száma, amely szintén rögzített kell legyen. A második kernel a részösszegeket összegzi egyetlen work-groupban. Az OpenCL-ben a `barrier` biztosítja a local memory konzisztenciáját. A determinizmus itt is a rögzített méreteken és a rögzített sorrendű fa-redukción múlik.

32.2.7 JAX példa

A JAX funkcionális programozási modelljében a redukciók, mint a `jnp.sum`, párhuzamosan futnak, és a sorrend nem garantált. A JAX dokumentációja szerint a redukciók sorrendje a backendtől és a hardvertől függhet, ezért a bit-pontosság nem garantált. Ha determinisztikus összegzést szeretnénk, explicit módon kell szabályozni a sorrendet.

Például a `jnp.cumsum` egy tengely mentén szekvenciálisan összegzi az elemeket, így a sorrend rögzített, de ez nem párhuzamos. Egy másik lehetőség a `jax.lax.associative_scan`, amely szintén rögzített sorrendet biztosít, de párhuzamosan fut. Azonban a `jnp.sum` esetén a sorrend nem garantált, mert a JAX a redukciót gyakran tree-reductionnel végzi, amelynek sorrendje a hardvertől függ.

```
# nyelv: JAX (Python), futtató: CPU/GPU/TPU
# Determinisztikus összegzés JAX-ban: rendezett cumsum utolsó eleme
import jax
import jax.numpy as jnp

def deterministic_sum(x):
    # A cumsum tengely mentén szekvenciális sorrendet biztosít,
    # de nem párhuzamos. Nagy tömböknél lassú lehet.
    return jnp.cumsum(x, axis=0)[-1]

# Alternatíva: a jnp.sort-tal rendezett összegzés nem segít,
# mert az összegzés sorrendje továbbra is párhuzamos.
```

A JAX esetében a determinizmus gyakran kompromisszumot igényel a teljesítmény rovására. A felhasználónak tisztában kell lennie ezzel, és ha a determinizmus kritikus, akkor vagy szekvenciális redukciót kell használni, vagy a műveletet saját kernelben implementálni (például CUDA custom call segítségével).

32.2.8 Labor: determinisztikus összegzés mérése

A labor célja egy determinisztikus összegzés megvalósítása CUDA-ban, OpenCL-ben vagy JAX-ban, és annak igazolása, hogy az eredmény reprodukálható. A mérési környezetet rögzíteni kell: GPU modell, driver, CUDA verzió, operációs rendszer, órajel, power limit. A mérés előtt warm-up, majd legalább 5-10 ismétlés, átlag és szórás számítása.

A következő egy példa mérési napló, amelyet a laborban el kell készíteni:

Környezet: - GPU: NVIDIA RTX 3080 (compute capability 8.6) - Driver: 550.54.14 - CUDA: 12.4 - OS: Ubuntu 22.04 LTS - Órajel: rögzített, power limit: 320 W - Precision: FP32

Bemenet: - Vektor mérete: 1 048 576 elem (1 MiB float) - Értékek: [0, 1) tartományba eső egyenletes eloszlású véletlen számok, de a méréshez ugyanaz a tömb kerül betöltésre minden futtatáskor.

Várt eredmény: - A várt eredményt a CPU-n szekvenciálisan, ugyanabban a sorrendben (növekvő indexek) számítjuk, FP32-ben. Ez a referencia. - A GPU determinisztikus redukciójának bitre azonosnak kell lennie a referenciával, mert a sorrend rögzített. (Megjegyzés: a CPU és a GPU kerekítési viselkedése eltérhet, ha az FP32 összeadás nem ugyanazt az utasítást használja, például FMA. Ezért a referencia számításnál ugyanazt a műveletet kell használni, például `std::plus<float>`.)

Mért eredmény: - 10 futtatás után az eredmény minden esetben ugyanaz az FP32 érték, bitre azonos. - A futásidő átlagosan 0,42 ms, szórás 0,02 ms. - A determinisztikus redukció eredménye: `123456.789` (példa) - A nem determinisztikus atomicAdd redukció eredménye ugyanarra a bemenetre 10 futtatás alatt: `123456.781`, `123456.795`, `123456.786`, … (eltérő ulp-k).

A laborban a várt és mért eredményt külön dokumentálni kell, és igazolni kell, hogy a determinisztikus redukció valóban reprodukálható, míg az atomi alapú nem.

32.2.9 Tipikus hibák és helyes megoldások

1. **Hiba:** A fejlesztő atomicAdd-ot használ lebegőpontos redukcióra, és feltételezi, hogy az eredmény determinisztikus. **Megoldás:** Használjon rögzített sorrendű redukciót, vagy ha az atomi művelet elkerülhetetlen, akkor egész típusú atomi műveletet használjon, vagy fogadja el a nem determinizmust.

2. **Hiba:** A launch konfiguráció (grid, block méret) a bemenet méretétől függően változik, ami megváltoztatja a redukciós fa szerkezetét. **Megoldás:** Rögzítse a grid és block méretet, vagy ha a grid mérete változik, akkor a második fázisban a részösszegeket olyan sorrendben összegezze, amely független a grid méretétől, például mindig ugyanolyan számú blokkra normalizál.

3. **Hiba:** A Kahan-összegzést párhuzamosan alkalmazzák, abban a hiszemben, hogy az determinisztikus. **Megoldás:** A Kahan-összegzés csak szekvenciálisan kompenzálja a kerekítési hibát, párhuzamosan nem garantálja a determinizmust. Determinisztikus párhuzamos összegzéshez rögzített sorrendű fa-redukciót használjon.

4. **Hiba:** A blokkon belüli fa-redukcióban nincs `__syncthreads()` minden szint után, ami adatversenyt okoz, és az eredmény nem determinisztikus. **Megoldás:** Minden fa-redukciós szint után szinkronizálni kell a szálakat.

5. **Hiba:** A részösszegeket a host oldalon `cudaMemcpy`-val olvassák ki, és a host oldali összegzés sorrendje nem rögzített (például párhuzamos `std::reduce`). **Megoldás:** A részösszegek összegzését is rögzített sorrendben végezze, vagy a második fázist GPU-n, rögzített sorrenddel valósítsa meg.

32.2.10 Önellenőrző kérdés

Miért nem determinisztikus az `atomicAdd`-alapú lebegőpontos redukció, és milyen két technikával lehet determinisztikus összegzést megvalósítani? (Válaszát indokolja a lebegőpontos asszociativitás és a hardveres ütemezés szempontjából.)

32.2.11 Összefoglalás

A nem determinisztikus redukció a GPU-számítások egyik leggyakoribb reprodukálhatósági problémája. A lebegőpontos összeadás nem asszociatív, és a párhuzamos végrehajtás sorrendje nem garantált. Az atomi műveletek sorrendje a hardver memóriarendszerétől függ, ezért lebegőpontos adatokra nem determinisztikus. A determinisztikus redukció megvalósításához rögzíteni kell a launch konfigurációt, a blokkon belüli fa-redukció sorrendjét és a blokkok közötti összegzés sorrendjét. A fejezet bemutatott egy kétfázisú CUDA-mintát, amely determinisztikus összegzést biztosít, és ismertette az OpenCL és JAX megfelelőit is. A laborban a hallgatóknak méréssel kell igazolniuk a determinizmust és a reprodukálhatóságot.

32.3 Hordozhatóság: HIP és SYCL

A CUDA-programozás alapfogalmait már ismered: a kernel, a hierarchikus indexelés, a memóriaterek, a szinkronizáció és a teljesítményre hangolás mind ugyanarra a hardverközeli modellre épül. A gyakorlati kérdés ebben az egységben az, hogyan vihető át ez a tudás más gyorsítókra vagy más eszközláncokra. A válasz nem egyetlen „univerzális CUDA”, hanem két, egymást kiegészítő stratégia: a **HIP**, amely szándékosan CUDA-szerű API-t ad, és a **SYCL**, amely nyelvi szabványként absztrahálja a heterogén végrehajtást. Ebben a részben a két eszközt nem egymás ellenében, hanem a hordozhatósági döntés részeként mutatjuk be. A kódpéldák illusztrációk; a futásukhoz a laborban megadott környezet és eszköz szükséges.

32.3.1 1. A hordozhatóság három szintje

Mielőtt a HIP vagy a SYCL részleteibe bocsátkoznánk, érdemes megkülönböztetni, mit értünk hordozhatóság alatt. A **forráshordozhatóság** azt jelenti, hogy ugyanaz a forráskód újrafordítható más platformra. A **bináris hordozhatóság** azt, hogy a lefordított kód futtatható más eszközön – erre a GPU-k világában gyakorlatilag nincs garancia, hiszen az utasításkészlet és a driver-illesztőfelület gyártónként eltér. A **teljesítmény-hordozhatóság** pedig azt, hogy a kód az új hardveren is elfogadható hatékonysággal fut; ez a legnehezebb, mert a warp- vagy hullámfrontméret, a memóriahierarchia és a feldolgozóegységek aránya eltér. A HIP és a SYCL elsősorban forráshordozhatóságot ígér, de teljesítmény-hordozhatóságot csak akkor, ha a kódot az adott eszközre is hangoljuk. Ezt a megkülönböztetést a fejezet végén visszük tovább a jövőbeli trendekhez (A GPGPU jövője).

32.3.2 2. HIP: CUDA-szerű programozás AMD és NVIDIA eszközökön

A **HIP** (Heterogeneous-Compute Interface for Portability) az AMD ROCm ökoszisztémájának hordozhatósági rétege. A célja, hogy a CUDA-kódot kis erőfeszítéssel AMD GPU-n is lehessen futtatni, és megfordítva: a HIP-kódot NVIDIA GPU-ra is lehessen fordítani, ahol a HIP hívások mögött a CUDA fut. A HIP tehát nem egy új, gyártófüggetlen modell, hanem egy CUDA-dialektus, amely az API-nevek, a kernelindítási szintaxis és a memóriaműveletek szintjén hoz létre megfeleltetést.

32.3.3 2.1. API-megfeleltetés

A HIP esetében a legtöbb CUDA-hívásnak egy az egyben megfeleltethető párja van. A gyakori műveletek:

Művelet	CUDA	HIP
Memóriafoglalás	`cudaMalloc`	`hipMalloc`
Másolás	`cudaMemcpy`	`hipMemcpy`
Kernelindítás	`<<<grid, block, shared, stream>>>`	`<<<grid, block, shared, stream>>>`
Szinkronizálás	`cudaDeviceSynchronize`	`hipDeviceSynchronize`
Hibakezelés	`cudaGetLastError` / `cudaGetErrorString`	`hipGetLastError` / `hipGetErrorString`
Eszköz lekérdezése	`cudaGetDeviceProperties`	`hipGetDeviceProperties`
Hibakód típus	`cudaError_t` / `cudaSuccess`	`hipError_t` / `hipSuccess`

A beépített kernelindexek (`threadIdx.x`, `blockIdx.x`, `blockDim.x`, `gridDim.x`) és a kernelminősítők (`__global__`, `__device__`, `__shared__`) ugyanúgy használhatók. A `__syncthreads` szinkronizáció is megmarad. A HIP-kód gyakran úgy néz ki, mintha a CUDA-kódot egyszerűen átneveztük volna.

32.3.4 2.2. Vektorösszeadás HIP-ben

A korábbi fejezetekből ismert vektorösszeadás alkalmas az API-megfeleltetés bemutatására. A kernel szinte változatlan:

```
// HIP C++ (illusztráció)
#include <hip/hip_runtime.h>
#include <cstdio>
#include <vector>

__global__ void add_kernel(const float* a, const float* b, float* c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) c[i] = a[i] + b[i];
}

int main() {
    const int n = 1 << 20;
    std::vector<float> h_a(n, 1.0f), h_b(n, 2.0f), h_c(n, 0.0f);
    float *d_a, *d_b, *d_c;
    hipMalloc(&d_a, n * sizeof(float));
    hipMalloc(&d_b, n * sizeof(float));
    hipMalloc(&d_c, n * sizeof(float));
    hipMemcpy(d_a, h_a.data(), n * sizeof(float), hipMemcpyHostToDevice);
```

```
    hipMemcpy(d_b, h_b.data(), n * sizeof(float), hipMemcpyHostToDevice);
    int block = 256;
    int grid = (n + block - 1) / block;
    add_kernel<<<grid, block>>>(d_a, d_b, d_c, n);
    hipMemcpy(h_c.data(), d_c, n * sizeof(float), hipMemcpyDeviceToHost);
    hipDeviceSynchronize();
    hipError_t err = hipGetLastError();
    if (err != hipSuccess) std::fprintf(stderr, "HIP error: %s\n", hipGetErrorString(err));
    std::printf("%f\n", h_c[42]);
    hipFree(d_a); hipFree(d_b); hipFree(d_c);
}
```

A kód és a CUDA-megfelelője között a különbség valóban minimális: `hip` előtag, `hip_runtime.h`, valamint a hibakezelő függvények neve. A kernel belseje azonos. A laborban ezt a forrásfájlt érdemes `.hip` kiterjesztéssel ellátni, de a `hipcc` a `.cpp` kiterjesztést is elfogadja.

32.3.5 2.3. Fordítás és eszköztámogatás

A HIP kódot a ROCm részeként elérhető `hipcc` fordítóval lehet lefordítani. Ha a célplatform AMD GPU, a `hipcc` a `amdclang`/ROCm eszközláncot használja; ha NVIDIA GPU, a HIP hívásokat CUDA-hívásokra képezi le, és a mögöttes fordító az `nvcc` vagy `clang` CUDA backend lehet. Ez a kettős természet fontos tanulság: a HIP-réteg a forráskompatibilitást adja, nem a bináris azonosságot. A fordításhoz a környezetben rendelkezésre kell állnia a ROCm vagy a CUDA toolkitnek. A hipify eszköz (`hipify-perl`, `hipify-clang`) a CUDA-forrásokat automatikusan HIP-forrásokká alakítja, de az automatikus konverziót mindig át kell vizsgálni: a warp-intrinsics, a textúra, a surface, a dinamikus párhuzamosság és a cooperative groups esetén a fordítás nem garantáltan ekvivalens. A laborban a HIP-változatot csak akkor futtatjuk, ha a megfelelő AMD vagy NVIDIA eszköz és ROCm környezet elérhető; ellenkező esetben a kódot fordítási ellenőrzésen futtatjuk, és a mérési naplóban ezt jelöljük.

32.3.6 2.4. A warp és a hullámfront kérdése

A HIP-kód portolásánál a legnagyobb buktató a warp méretére való támaszkodás. A CUDA-ban a warp 32 szálból áll, és a warp-szintű intrinsicek (például `__shfl_down`, `__ballot`, `__match_any`) ehhez a mérethez kötöttek. A HIP-ben a `warpSize` makró az adott eszköz hullámfrontméretét adja, amely AMD RDNA/CDNA architektúrán jellemzően 32 vagy 64 lehet, míg NVIDIA-n 32. Ha egy CUDA-ból átvett redukció fix lépésekkel dolgozik (például delta = 16, 8, 4, 2, 1), akkor 64-es hullámfronton a redukció nem lesz teljes, mert a 32–63. indexű szálak közötti adatcsere elmarad. Helyes megoldás a warp-szintű redukciót a `warpSize` értékéből kiinduló logaritmikus ciklussal írni, vagy `__shfl_xor`-ral olyan maszkot használni, amely a hullámfrontméretet lefedi. A HIP-ben a `__shfl_xor` szintaktikailag hasonló, de a szemantika a hullámfrontméret függvénye, ezért a hordozható kernelben nem szabad a 32-es méretet konstansként beégetni. Ez a példa jól mutatja, hogy a forráshordozhatóság nem jelent automatikus szemantikai azonosságot.

32.3.7 3. SYCL: szabványos C++ heterogén programozás

A **SYCL** a Khronos Group nyílt szabványa, amely C++17 alapú, és a heterogén rendszerek programozását egyetlen forrásból teszi lehetővé. A SYCL nem egyetlen implementáció, hanem egy specifikáció, amelyet több eszközlánc valósít meg: a nyílt forráskódú **AdaptiveCpp** (korábbi nevén hipSYCL) CUDA, HIP és OpenCL hátterekkel, az Intel **oneAPI/DPC++** Intel GPU, CPU és FPGA eszközökkel, valamint a **ComputeCpp** (Codeplay) OpenCL-alapú implementáció.

A SYCL célja nem a CUDA API lemásolása, hanem egy magasabb szintű, de teljesítményorientált párhuzamos C++-modell kialakítása.

32.3.8 3.1. A SYCL végrehajtási modell

A SYCL programban központi szerepet játszik a `queue`, amely egy eszközhöz tartozó parancssor. A kernel beküldése a `queue.submit()` hívással történik, amely egy `handler` objektumot kap. A `handler` segítségével definiáljuk a végrehajtási tartományt (`nd_range`) és a kernel-lambdát. A SYCL két memóriamodellt kínál:

- **Buffer/accessor modell**: a `buffer` a memória egy absztrakt, eszközfüggetlen tulajdonosa, az `accessor` pedig a kernelben hozzáférést biztosít. A futásrendszer feladata a memória tényleges másolása és szinkronizálása. Ez a modell kevesebb explicit memóriakezelést igényel, de a memóriamozgatás időpontja nem mindig látható a programozó számára.
- **USM (Unified Shared Memory) modell**: a `malloc_device`, `malloc_shared` és `malloc_host` hívásokkal explicit módon foglalunk eszköz-, megosztott vagy pinned host memóriát. A USM közelebb áll a CUDA pointermodellhez, és a teljesítménykritikus kódban gyakran egyszerűbb a vezérlés.

A kernel indexelése a SYCL-ben az `nd_item` objektumon keresztül történik. Az `nd_range` globális és lokális tartományt ad meg, hasonlóan a CUDA grid és block dimenzióihoz. A `get_global_id(0)` az első dimenzió globális indexét adja, a `get_local_id(0)` a munkacsoporton belüli indexet, a `get_local_range(0)` pedig a munkacsoport méretét. A blokkszintű szinkronizációt a `group_barrier` biztosítja.

32.3.9 3.2. Vektorösszeadás SYCL-ben

A USM modellt használva a vektorösszeadás így néz ki:

```
// SYCL C++ (illusztráció)
#include <sycl/sycl.hpp>
#include <cstdio>
#include <vector>

int main() {
    const int n = 1 << 20;
    std::vector<float> h_a(n, 1.0f), h_b(n, 2.0f), h_c(n, 0.0f);
    try {
        sycl::queue q{sycl::default_selector_v};
        float* d_a = sycl::malloc_device<float>(n, q);
        float* d_b = sycl::malloc_device<float>(n, q);
        float* d_c = sycl::malloc_device<float>(n, q);
        q.memcpy(d_a, h_a.data(), n * sizeof(float));
        q.memcpy(d_b, h_b.data(), n * sizeof(float));
        q.submit([&](sycl::handler& cgh) {
            cgh.parallel_for(sycl::nd_range<1>(n, 256), [=](sycl::nd_item<1> it) {
                int i = it.get_global_id(0);
                if (i < n) d_c[i] = d_a[i] + d_b[i];
            });
        });
        q.memcpy(h_c.data(), d_c, n * sizeof(float));
        q.wait_and_throw();
        std::printf("%f\n", h_c[42]);
        sycl::free(d_a, q); sycl::free(d_b, q); sycl::free(d_c, q);
    } catch (const sycl::exception& e) {
        std::fprintf(stderr, "SYCL error: %s\n", e.what());
```

```
        return 1;
    }
}
```

A kódban a `queue` az alapértelmezett eszközt választja. A `malloc_device` a CUDA `cudaMalloc`-hoz hasonló eszközmemóriát foglal, a `queue.memcpy` a host és az eszköz közötti másolást végzi, a `parallel_for` pedig a kernelindítás. A `wait_and_throw` a kernel és a másolások befejezését várja, és szinkron módon dobja a hibát, ha a sorban hiba keletkezett. A `try/catch` a SYCL kivételkezelést mutatja.

A buffer/accessor modell ugyanezt a feladatot bonyolultabb látható vezérléssel oldja meg, de a futásrendszer optimalizálhatja a memóriamozgatást. A laborban a USM modellt használjuk, mert a CUDA-megfeleltetés így közvetlenebb; a buffer/accessor modell a SYCL teljesítményhangolásnál kerül elő.

32.3.10 3.3. A szabvány ereje és korlátai

A SYCL szabványossága azt jelenti, hogy a forráskód ugyanaz lehet különböző gyártók implementációiban, de a fordítás és a futtatás környezetfüggő. A `range` és `nd_range` dimenziók használata szabványos, a `local_accessor` a shared memory-t absztrahálja, a `group_barrier` pedig blokkszintű szinkronizációt ad. A szabvány előírja a munkacsoport méretének és a globális tartománynak az összhangját: a globális méretnek oszthatónak kell lennie a lokális mérettel. Ha a feltétel nem teljesül, a kernel nem indul el, és a hiba a `wait_and_throw` során derül ki.

A SYCL-hordozhatóság korlátja a hardverek eltérő tulajdonságaiban keresendő. A munkacsoport optimális mérete, a memóriakoaleszkálás, a cache-hierarchia és a specializált egységek (Tensor Core, matrix core, XMX) nem szabványosítottak. A SYCL kód egy CPU-n is futtatható, de a teljesítmény akkor lehet elfogadható, ha a munkacsoport méretét a CPU cache-vonalaihoz és vektoros egységeihez igazítjuk. A GPU-n ugyanez a kód más optimalizálást igényel. A szabvány tehát a hordozhatóságot a szemantika szintjén adja, a teljesítményt nem.

32.3.11 4. API-megfeleltetés a gyakorlatban

A három modell együtt nézve jól mutatja a párhuzamos programozás közös nyelvét. A táblázat a leggyakoribb műveleteket foglalja össze:

Művelet	CUDA	HIP	SYCL (USM)
Memóriafoglalás	`cudaMalloc`	`hipMalloc`	`sycl::malloc_device`
Másolás	`cudaMemcpy`	`hipMemcpy`	`q.memcpy`
Kernelindítás	`<<<...>>>`	`<<<...>>>`	`q.submit` + `parallel_for`
Szinkronizálás	`cudaDeviceSynchronize`	`hipDeviceSynchronize`	`q.wait` / `q.wait_and_throw`
Hibakezelés	`cudaGetErrorString`	`hipGetErrorString`	`sycl::exception::what`
Blokkszinkron	`__syncthreads`	`__syncthreads`	`group_barrier`
Shared memória	`__shared__`	`__shared__`	`local_accessor`

A táblázat nem teljes, de a fejezet céljának megfelel: látható, hogy a CUDA és a HIP között a hasonlóság szinte egy az egyben, míg a SYCL más nyelvi eszközökkel fejezi ki ugyanazt. A SYCL `local_accessor` használata például a `__shared__` memóriát helyettesíti, de a munkacsoport

méretét a kernelben is le kell kérdezni a `get_local_range` segítségével, ami a CUDA-ban a `blockDim.x`-ből ismert.

A hibakezelésben a SYCL aszinkron jellege külön figyelmet érdemel. A CUDA-ban a kernelhiba gyakran csak a következő szinkronizációs ponton derül ki; a `cudaGetLastError` a kernel után hívva adja meg az utolsó hibát. A HIP ugyanezt a mintát követi. A SYCL-ben a `queue.submit` visszaad egy `event` objektumot, amelynek `wait_and_throw` metódusa veti fel a hibát, ha a végrehajtás során hiba történt. Ha csak `queue.wait`-et hívunk, a hiba nem dobódik, és a program hibás eredménnyel folytatódhat. Ezért a SYCL-kódban a `wait_and_throw` használata ajánlott, vagy explicit `try/catch` a `queue.submit` és a `wait` körül.

32.3.12 5. Tipikus hibák és javításuk

Ebben a szakaszban azokat a hibákat gyűjtjük össze, amelyek a CUDA-ról HIP/SYCL-re való áttéréskor gyakran előfordulnak. A cél nem a hibák puszta felsorolása, hanem a szemléletmód átadása: a portolás nem helyettesítheti a hardverismeretet.

32.3.13 5.1. A warp méretének beégetése

A CUDA-kódban a warp-szintű redukció gyakran a következő alakú:

```
__device__ float warp_reduce(float val) {
    for (int offset = 16; offset > 0; offset >>= 1)
        val += __shfl_down_sync(0xffffffff, val, offset);
    return val;
}
```

Ez a kód 32 szál esetén helyes, de 64 szálból álló hullámfronton nem redukálja a teljes tartományt, mert az offset sorozat 16-tól 1-ig tart, és nem éri el a 32-es lépést. A hordozható változat:

```
__device__ float warp_reduce(float val) {
    for (int offset = warpSize / 2; offset > 0; offset >>= 1)
        val += __shfl_down_sync(0xffffffff, val, offset);
    return val;
}
```

Itt a `warpSize` a HIP-ben az eszköz hullámfrontméretét adja, és a ciklus automatikusan alkalmazkodik. CUDA-ban a `warpSize` konstans 32, így a kód változatlanul működik. A maszkot is a `warpSize`-hez kell igazítani, ha a `__shfl_down_sync` maszkja nem a teljes hullámfrontot fedi le; 64-es hullámfronton a 0xffffffff maszk 32 bitet jelent, ezért a maszkot a hullámfrontméret függvényében kell előállítani.

32.3.14 5.2. SYCL: a globális és lokális tartomány összeférhetetlensége

A SYCL-ben a `nd_range` global range-jének minden dimenzióban oszthatónak kell lennie a local range-zel. Ha például a globális méret 1000 és a lokális méret 256, a kernel nem indul el. A megoldás az, hogy a globális méretet a lokális méret többszörösére kerekítjük, és a kernel belsejében határellenőrzést végzünk:

```
int global = (n + local - 1) / local * local;
q.submit([&](sycl::handler& cgh) {
    cgh.parallel_for(sycl::nd_range<1>(global, local), [=](sycl::nd_item<1> it) {
        int i = it.get_global_id(0);
        if (i >= n) return;
        // ...
    });
});
```

A kódban a `global` érték a felfelé kerekített méret, a kernelben az `i >= n` ellenőrzés biztosítja, hogy a felesleges szálak ne írjanak a tömb határán kívülre. Ez a minta megegyezik a CUDA grid-stride loop határellenőrzésével, de a SYCL szigorúbban ellenőrzi a tartományok oszthatóságát.

32.3.15 5.3. A buffer hatókörének hibája a SYCL-ben

Ha a `buffer` objektum a kernel beküldése után megsemmisül, a futásrendszer nem tudja garantálni a memória érvényességét. A buffernek legalább addig élnie kell, amíg a kernel le nem fut, és a host adatokhoz való hozzáférés meg nem történik. A gyakori hiba:

```
// Téves: a buffer a submit után megsemmisül
{
    sycl::buffer<float> buf{h_c.data(), sycl::range<1>(n)};
    q.submit([&](sycl::handler& cgh) {
        auto acc = buf.get_access<sycl::access::mode::read>(cgh);
        // ...
    });
} // buf itt megsemmisül, de a kernel még futhat
```

A helyes megoldás az, hogy a buffert a `q.wait()` hívásig életben tartjuk, vagy a USM modellt használjuk, ahol a memória élettartama explicit. A USM esetén a `sycl::free` hívás előtt mindig várjuk meg a `queue.wait`-et.

32.3.16 5.4. Az aszinkron hibák elvesztése

A SYCL-ben a `q.wait()` nem dob hibát; ha a kernelben hiba történik, a hiba az event `wait_and_throw` hívásakor jelenik meg. Ha a kód `q.wait()`-et hív, majd a `sycl::free`-vel felszabadítja a memóriát, a hiba elvész, és a program esetleg érvénytelen memóriát olvas. A helyes minta: a `queue.submit` által visszaadott `event`-et tároljuk, majd `event.wait_and_throw()`-t hívunk, vagy a `queue.wait_and_throw()`-t használjuk, amely a teljes sor hibáit dobja.

32.3.17 6. Labor és mérési napló

A labor célja, hogy a fejezetben bemutatott API-megfeleltetést egy kis, ellenőrizhető példán alkalmazza. A feladat három vektorösszeadás változat elkészítése: CUDA, HIP és SYCL. A kódok a fenti minták alapján írhatók meg, de a laborban használt hibakezelő makrót minden API-híváshoz hozzá kell illeszteni. A labor nem feltétlenül jelenti mindhárom eszköz elérhetőségét; ha a HIP/SYCL futtatás nem lehetséges, a kódot fordítási ellenőrzésen kell futtatni, és a jegyzőkönyvben ezt jelölni kell. A mért és a várt eredményt külön kell feltüntetni.

32.3.18 6.1. Build és futtatási parancsok

Az alábbi parancsok a labor kiindulópontjai. A pontos verziók a labor környezettől függenek.

CUDA:

```
nvcc -O3 -o vecadd_cuda vecadd.cu
./vecadd_cuda
```

HIP:

```
hipcc -O3 -o vecadd_hip vecadd.hip
./vecadd_hip
```

SYCL (AdaptiveCpp vagy oneAPI):

```
icpx -fsycl -O3 -o vecadd_sycl vecadd.cpp
./vecadd_sycl
```

AdaptiveCpp esetén a fordítás például:

```
acpp -O3 -o vecadd_sycl vecadd.cpp
```

A laborban a használt parancsot pontosan rögzíteni kell.

32.3.19 6.2. Környezet és mérési napló

A fejezet laborjában a következő adatokat kell rögzíteni:

- GPU modell: például NVIDIA RTX A4000.
- Driver: 535.104.05.
- CUDA verzió: 12.2.
- ROCm verzió (ha HIP futtatás): 5.7.
- SYCL implementáció és verzió: AdaptiveCpp 23.10 vagy oneAPI 2024.0.
- Operációs rendszer: Ubuntu 22.04.
- Compute capability: 8.6 (A4000 esetén).
- A mérési módszer: warm-up 10 ismétlés, majd 50 ismétlés, átlag és szórás.

A példa napló:

Változat	Futtatva?	Átlag idő (ms)	Szórás (ms)	Megjegyzés
CUDA	igen	0.12	0.01	NVIDIA RTX A4000
HIP	fordítás ellenőrizve, futtatás nem	– (várt: 0.12–0.20)	–	AMD eszköz hiányában
SYCL	fordítás ellenőrizve, futtatás nem	– (várt: 0.15–0.30)	–	AdaptiveCpp, CUDA backend elérhető

A mérési naplóban a „várt” értékek csak becslések, nem mért adatok. A laborban a várt eredményt a hardver jellemzői és a korábbi mérések alapján lehet megadni, de a mért és a várt értéket mindig külön kell jelölni.

32.3.20 6.3. Ellenőrző kérdés

A fejezet végén egy önellenőrző kérdés segít a tanultak rögzítésében: *Miért nem elegendő a HIP és a SYCL forráshordozhatósága a warp-szintű redukciók esetén?* A válaszban ki kell térni a hullámfrontméret eltérésére és a SYCL munkacsoport-méretének hardverfüggésére, valamint arra, hogy a teljesítmény-hordozhatóság csak a célhardver ismeretében érhető el.

32.3.21 7. Összefoglalás és továbbvezetés

A HIP és a SYCL két eltérő válasz ugyanarra a hordozhatósági kihívásra. A HIP a CUDA-hoz való közelségével csökkenti a portolás költségét, de a hardveres eltéréseket nem tünteti el. A SYCL szabványos absztrakciót ad, de a teljesítmény elérése továbbra is eszközspecifikus hangolást igényel. A gyakorlatban a döntés a kódbázis méretén, a célhardveren és a csapat tapasztalatán múlik. A következő egység (A GPGPU jövője) ebből a fogalmi hálóból indul ki: a GPGPU jövője, a specializált egységek, a heterogén programozási modellek és a mesterséges intelligencia hardverigénye mind arra építenek, hogy a hordozhatóság nem API-kérdés, hanem a párhuzamos algoritmusok és a hardvermodellek együttes ismerete. A most bevezetett példák és hibaminták tehát nemcsak a HIP/SYCL használatához, hanem a következő generációs akcelerátorok megértéséhez is alapul szolgálnak.

32.4 A GPGPU jövője

A GPGPU-programozás jövőjét nem egyetlen hardver- vagy nyelvi áttörés határozza meg, hanem három, egymással összefonódó trend: az integrált áramkörök gyártásának fizikai korlátai, a specializált számítóegységek térnyerése, valamint a fordító- és programozásimodell-rétegek gyors fejlődése. A Hordozhatóság: HIP és SYCL alfejezetben a hordozhatóságot a HIP, a SYCL és az absztrakciós rétegek felől vizsgáltuk; itt arra koncentrálunk, hogy milyen irányba mozdul a vas és a szoftver, és mit jelent ez a mérnöki gyakorlat számára. A cél nem jóslás, hanem azoknak a technológiai erővonalaknak a azonosítása, amelyek már ma is érzékelhetők a CUDA-ökoszisztémában, és amelyekre a következő fejezet záróprojektjének tervezésekor támaszkodni kell.

32.4.1 Hardvertrendek: a szilíciumtól a rendszerig

A GPU-k fejlődését évtizedekig a Moore-törvény hajtotta: a tranzisztorok számának növekedése egyszerre tette lehetővé a nagyobb párhuzamosságot és a magasabb órajelet. A Dennard-skálázás 2006 körüli leállása óta azonban a teljesítménysűrűség növekedése korlátozott, ezért a gyártók a számítási teljesítményt egyre inkább a párhuzamos egységek számának növelésével, specializált gyorsítókkal és a memória-hierarchia átalakításával érik el. A modern GPU-k esetében a névleges FP32-átbocsátóképesség növekedése mellett legalább ugyanolyan fontos az adatmozgatási sávszélesség, a késleltetés és az energiahatékonyság.

A gyártási folyamat finomodása – 7 nm, 5 nm, 3 nm és azon túl – a tranzisztorsűrűséget növeli, de az órajel-skálázás már erősen korlátozott. A HPC- és AI-gyorsítók ezért egyre gyakrabban alkalmaznak *chiplet* architektúrát: a nagy monolítikus lapka helyett több kisebb, önállóan tesztelt és optimalizált gyártási technológiával készülő lapkát kötnek össze nagy sebességű, alacsony energiaigényű összeköttetéssel. Ez a megközelítés csökkenti a gyártási hibákból adódó selejtet, lehetővé teszi a különböző gyártási technológiák keverését (például logikai mag 5 nm-en, I/O vagy memóriavezérlő 7 nm-en), és a termékcsalád skálázását a chipletek számának változtatásával.

A chipletek közötti kommunikáció minősége meghatározóvá válik. A GPU-világban az NVIDIA az NVLink és NVSwitch technológiát fejleszti, az AMD az Infinity Fabricot, az Intel a CXL és UCIe szabványokat támogatja. A jövő GPU-i nagy valószínűséggel nem pusztán több chipletet tartalmaznak, hanem a chipletek közötti memóriakoherenciát és a sávszélességet is úgy alakítják, hogy a programozó számára a rendszer egységesebbnek tűnjön. A gyakorlatban ez azt jelenti, hogy a multi-GPU programozás és a node-on belüli adatcsere modellezése közelíteni fog a shared memory rendszerekhez, még akkor is, ha a tényleges fizikai hierarchia NUMA-jellegű marad.

A memóriaoldal legalább ilyen fontos. A globális memória sávszélessége a HBM (High Bandwidth Memory) és GDDR szabványok fejlődésével nő, de a számítási egységek étvágya gyorsabban nő. Ezért terjednek a nagy kapacitású, több szintű cache-ek, a szoftveresen kezelhető L2 persistence mechanizmusok és a near-memory computing ötlete, ahol bizonyos műveletek – például összegzés vagy ritka mátrix-művelet – a memóriamodulokhoz közel futnak le, csökkentve az adatmozgatást. A CUDA programozási modellben ez egyelőre nem látszik közvetlenül, de a memóriaterek és a cache-hint-ek tervezésekor érdemes feltételezni, hogy a jövőben a memória-közeli számítások explicit vagy implicit módon elérhetővé válnak.

A specializált egységek térnyerése a leglátványosabb trend. A tensor core-ok a mátrix-szorzás és a konvolúció gyorsítására születtek, és mára a gépi tanulási terhelések domináns végrehajtó egységeivé váltak. A következő lépés a még alacsonyabb pontosságú formátumok (FP8, FP4, integer kvantálás) hardveres támogatása, a ritka mátrixok közvetlen gyorsítása, valamint a transzformerek figyelmi mechanizmusát segítő dedikált utasítások. Emellett a ray tracing magok, a video-kódoló/dekódoló egységek és a biztonsági kriptográfiai gyorsítók mind azt mutatják, hogy a GPU egyre inkább heterogén SoC-vé válik. A programozó számára ez azt jelenti, hogy az „egy kernel mindenre” megközelítés helyett a megfelelő végrehajtó egység kiválasztása és a köztük lévő adatmozgás optimalizálása válik a teljesítmény kulcsává.

32.4.2 Nyelvi trendek: a hardverhez közelítő absztrakciók

A CUDA C++ az elmúlt években jelentős nyelvi fejlődésen ment keresztül. A C++17/20 szabványosítás térnyerésével a CUDA-kód egyre inkább a gazdagabb típusrendszert, a constexpr kiértékelést, a variadic template-eket és a lambdákat használja. A `libcu++` a párhuzamos standard könyvtárat biztosítja device-oldalon, lehetővé téve az `std::` konténerek és algoritmusok egy részének használatát a kernelben. Ez csökkenti a saját adatszerkezetek írásának terhét, de óvatosságot igényel: a device-oldali dinamikus memóriafoglalás és a kivételkezelés továbbra is korlátozott.

A nagyobb hordozhatóságot célzó nyelvek és könyvtárak – HIP, SYCL, Kokkos, Raja, valamint a C++23-ban megjelenő párhuzamos algoritmusok gyorsítókon való futtatásának támogatása – azt a filozófiát követik, hogy a kernel logikája elválasztható a hardver-specifikus ütemezéstől. Ennek előnye, hogy ugyanaz a kód fordítható CUDA-ra, HIP-re, OpenCL-re vagy akár x86 SIMD-re; hátránya, hogy a legalacsonyabb szintű optimalizációk (warp intrinsics, texture cache, tensor core API) elérése gyakran kompromisszumokkal jár. A gyakorlatban a hardverközeli optimalizálást továbbra is CUDA-ban (vagy HIP-ben) végezzük, míg a magasabb szintű alkalmazáslogikát a hordozható rétegben tartjuk.

A másik jelentős nyelvi trend a fordító-infrastruktúra átalakulása. A CUDA C++ hagyományosan az NVCC-n keresztül, a PTX köztes reprezentáción át jut el a hardverre. Az LLVM-alapú NVHPC és a CUDA C++ frontendje mellett megjelent a NVRTC (runtime compilation) képessége, amely lehetővé teszi a kernelek futásidőben történő fordítását és hangolását. Ez a *just-in-time* (JIT) megközelítés kulcsfontosságú a jövőben, mert a hardver- és szoftververziók közötti eltéréseket futásidőben lehet feloldani. A CUDA 12 óta a JIT LTO (link-time optimization) a kernelek közötti hívások és a template-instanciák optimalizálását is javítja.

Ezzel párhuzamosan a Python-alapú eszközláncok – JAX, PyTorch/XLA, TensorFlow XLA, Triton – a GPU-programozást a compiler-gráf szintjére emelik. A JAX-ben a `jit`, `vmap`, `pmap` transzformációk a programot egy XLA-gráffá alakítják, amelyet a fordító optimalizál, majd tömörített kernel-lánccá generál. Ez a modell nem igényel kézzel írt CUDA-kernelt a gyakori lineáris algebrai és gépi tanulási műveletekhez, viszont a teljesítményt a gráfoptimalizáló minősége határozza meg. A jövő GPGPU-fejlesztőjének érdemes mindkét világot ismernie: a CUDA C++ alacsony szintű kontrollját a kernel szintjén, és a gráf-alapú nyelveket a magas szintű alkalmazásokban.

32.4.3 Chiplet és interconnect: a rendszer-szintű párhuzamosság felé

A chiplet-architektúra a GPU-n belüli párhuzamosságot a rendszer szintjére emeli. Ha a chipletek közötti sávszélesség és késleltetés elég alacsony, a programozó számára a több chipletből álló GPU hasonlóan viselkedik, mint egy nagy monolítikus egység. A gyakorlatban azonban a NUMA-hatások és a memóriakoherencia-protokollok miatt a szoftvernek továbbra is figyelnie kell arra, hogy melyik adat melyik chiplet memóriájában van. A CUDA jelenleg a multi-GPU programozást a `cudaSetDevice`, a peer access és a NCCL kollektív műveletek szintjén támogatja. A jövőben valószínűleg finomabb granularitású irányítás jelenik meg, például a `cudaMemAdvise` és a `cudaMemPrefetchAsync` kiterjesztése chiplet-szintű hozzáférési mintákra.

Az interconnect technológiák közül az NVLink és NVSwitch már ma is lehetővé teszik a több GPU közötti nagy sávszélességű, alacsony késleltetésű adatcserét. A CXL (Compute Express Link) szabvány a CPU-memória és a gyorsítók közötti koherens kapcsolatot célozza, és a jövőben lehetővé teheti, hogy a GPU közvetlenül hozzáférjen a CPU memóriájához vagy akár más gyorsítók memóriájához koherens módon. Ez a Unified Memory és a managed memory továbbfejlesztését jelenti: az adatok migrációja és a hozzáférési szándékok finomhangolása a hardver és a driver együttműködésével történik. A fejlesztő szempontjából ez csökkenti a explicit `cudaMemcpy` hívások számát, de a teljesítmény szempontjából továbbra is kritikus a hozzáférési minták lokalitása.

A több GPU-ból álló node-ok és a elosztott rendszerek felé haladva a programozási modell egyre inkább a *partitioned global address space* (PGAS) felé közelít. A NCCL és a MPI mellett megjelennek a PGAS-megközelítések, mint az NVSHMEM, amelyek a GPU-memóriák közötti közvetlen put/get műveleteket biztosítanak. A jövőben a nagy tudományos szimulációk és a nagyméretű nyelvi modellek tanítása valószínűleg a node-on belüli chiplet- és multi-GPU-párhuzamosság, valamint a node-ok közötti hálózati párhuzamosság kombinációjára épül.

32.4.4 AI és tudományos számítás konvergenciája

A gépi tanulás terhelései – különösen a mély neurális hálózatok tanítása és futtatása – ma a GPU-gyártók elsődleges bevételi forrását jelentik, és ennek megfelelően a hardvertervezést is meghatározzák. A tensor core-ok, a mixed precision támogatás és a nagy memóriasávszélesség mind az AI-munkaterhelések igényeit tükrözik. Ez a hangsúlyeltolódás a tudományos számítás számára is előnyös: a lineáris algebrai műveletek GPU-gyorsítása soha nem látott teljesítményt ért el, és az olyan könyvtárak, mint a cuBLAS, cuSOLVER és CUTLASS, közvetlenül használják a tensor core-okat.

Ugyanakkor a tudományos számítás igényei részben eltérnek a gépi tanulástól. A szimulációk gyakran dupla pontosságú (FP64) aritmetikát, szigorú numerikus reprodukálhatóságot és hosszú futásidőt igényelnek – míg a neurális hálózatok tanítása megelégszik a csökkentett pontosságú, sztochasztikus kerekítéssel. A GPU-gyártók ezért a termékpalettát differenciálják: a fogyasztói és AI-célú GPU-k az FP32/FP16/TF32/FP8 teljesítményre koncentrálnak, míg a HPC-célú modellek (például az NVIDIA H100, A100) jelentős FP64-teljesítményt is nyújtanak. A jövőben

ez a szétválás valószínűleg folytatódik, és a fejlesztőnek pontosan tudnia kell, hogy az adott feladat melyik pontosságon futtatható.

A konvergencia másik iránya a tudományos szimulációk gépi tanulással való kombinálása. A fizikai informált neurális hálózatok (PINN), a surrogate modellek és a adatvezérelt szimulációk mind a GPU-n futó lineáris algebrai és nemlineáris műveletek keverékét igénylik. Ez a hibrid terhelés új programozási mintákat hoz létre, ahol a klasszikus PDE-megoldó kernelek és a neurális hálózati kernelek ugyanazon a hardveren, gyakran ugyanabban a folyamatban futnak. A CUDA ökoszisztéma erre a streamek, a CUDA graphok és a multi-GPU absztrakciók révén már ma is lehetőséget ad, de a hatékony integráció továbbra is mérnöki kihívás.

32.4.5 Labor: technológiai roadmap egy ellenőrizhető példán

A labor célja, hogy a fenti trendeket egy kis, reprodukálható mérésen keresztül konkretizáljuk. Az alábbi CUDA C++ program egy egyszerű mátrix-szorzást (GEMM) hajt végre három különböző pontosságon: FP32, TF32 és FP16. A kernel a tensor core-okat nem használja, csupán a pontosság és a memória-hozzáférés közötti kapcsolatot vizsgálja. A mért eredményekből technológiai roadmapet vonunk le: hogyan alakul a teljesítmény, ha a pontosság csökken, és milyen hatása van a memória-sávszélességnek. A labor feltételezi, hogy a GPU rendelkezik legalább CUDA 11.0-val és Ampere vagy újabb architektúrával; a mérést dokumentáltuk, és a nem mért értékeket egyértelműen elválasztjuk a várt értékektől.

A kód a következő lépéseket végzi: lefoglalja a bemeneti mátrixokat, elindítja a naiv GEMM-kernelt mindhárom pontosságon, méri a futásidőt, és kiszámítja az effektív FLOP/s-ot és a memória-sávszélességet. A kernel egyszerű, blokkos tiling nélküli, így a teljesítményt főként a globális memória-hozzáférés és a számítási egységek kihasználtsága határozza meg – ez demonstrálja a Roofline-modell egy egyszerű esetét.

```
// language: CUDA C++ (C++17), compiler: nvcc (CUDA 12.4, GPU: NVIDIA RTX 4090, driver:
↪  550.54.14, OS: Ubuntu 22.04)
// Fordítás: nvcc -O3 -arch=sm_89 -o gemm_mixed gemm_mixed.cu
// Futtatás: ./gemm_mixed
#include <cuda_runtime.h>
#include <iostream>
#include <cstdlib>
#include <cstring>
#include <chrono>
#include <vector>
#include <iomanip>
#include <cuda_fp16.h>  // FP16 support

#define CHECK(call)                                                              \
    do {                                                                          \
        cudaError_t err = call;                                                   \
        if (err != cudaSuccess) {                                                 \
            std::cerr << "CUDA hiba: " << cudaGetErrorString(err)                 \
                      << " (" << __FILE__ << ":" << __LINE__ << ")" << std::endl;\
            std::exit(EXIT_FAILURE);                                              \
        }                                                                         \
    } while (0)

// Naiv GEMM kernel: C = alpha * A * B + beta * C
// A: M x K, B: K x N, C: M x N, row-major
template <typename T>
__global__ void gemm_naive(const T* A, const T* B, T* C,
                           int M, int N, int K, T alpha, T beta) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
```

```
    if (row < M && col < N) {
        T acc = static_cast<T>(0);
        for (int k = 0; k < K; ++k) {
            acc += A[row * K + k] * B[k * N + col];
        }
        C[row * N + col] = alpha * acc + beta * C[row * N + col];
    }
}

// FP32 változat
__global__ void gemm_fp32(const float* A, const float* B, float* C,
                          int M, int N, int K) {
    gemm_naive<float>(A, B, C, M, N, K, 1.0f, 0.0f);
}

// TF32-nek megfelelő mérés: itt a bemenet FP32, de a szorzás float-ban történik,
// a tensor core TF32 útvonalat a valós méréshez külön kernelben kellene használni.
// Ebben a laborban az FP32 és TF32 közötti különbséget a hardver specifikációból
// becsüljük, nem pedig ebből a naiv kernelből mérjük. Ezért a TF32 mérés helyett
// az FP32 kernel futtatását használjuk, és a TF32 várt teljesítményt
// a specifikáció alapján adjuk meg.

// FP16 változat (half precizitású számolás, de a bemenetek FP32-ből konvertálódnak)
__global__ void gemm_fp16(const float* A, const float* B, float* C,
                          int M, int N, int K) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < M && col < N) {
        __half acc = __float2half(0.0f);
        for (int k = 0; k < K; ++k) {
            __half a = __float2half(A[row * K + k]);
            __half b = __float2half(B[k * N + col]);
            acc = __hfma(a, b, acc); // half precision fused multiply-add
        }
        C[row * N + col] = __half2float(acc);
    }
}

// Mérés segédfüggvény
template <typename Func>
float measure(Func f, int iterations = 10) {
    // warm-up
    f();
    cudaDeviceSynchronize();
    auto start = std::chrono::high_resolution_clock::now();
    for (int i = 0; i < iterations; ++i) {
        f();
    }
    cudaDeviceSynchronize();
    auto end = std::chrono::high_resolution_clock::now();
    std::chrono::duration<float, std::milli> dur = (end - start) / iterations;
    return dur.count();
}

int main() {
    // GPU konfiguráció rögzítése
    cudaDeviceProp prop;
    CHECK(cudaGetDeviceProperties(&prop, 0));
    std::cout << "GPU: " << prop.name << "\n";
    std::cout << "Compute capability: " << prop.major << "." << prop.minor << "\n";
    std::cout << "Sávszélesség (névleges): " << prop.memoryClockRate * 2 * (prop.memoryBusWidth
    ↪  / 8) / 1e6 << " GB/s\n";
    std::cout << "FP32 elméleti csúcs: " << prop.multiProcessorCount *
    ↪  prop.maxThreadsPerMultiProcessor * 2 / 1e3 << " GFLOP/s (durva becslés)\n";
```

```
const int M = 1024, N = 1024, K = 1024;
const size_t bytes_A = M * K * sizeof(float);
const size_t bytes_B = K * N * sizeof(float);
const size_t bytes_C = M * N * sizeof(float);

float *d_A, *d_B, *d_C;
CHECK(cudaMalloc(&d_A, bytes_A));
CHECK(cudaMalloc(&d_B, bytes_B));
CHECK(cudaMalloc(&d_C, bytes_C));

// Inicializálás host-on
std::vector<float> h_A(M * K), h_B(K * N), h_C(M * N, 0.0f);
for (int i = 0; i < M * K; ++i) h_A[i] = static_cast<float>(rand()) / RAND_MAX - 0.5f;
for (int i = 0; i < K * N; ++i) h_B[i] = static_cast<float>(rand()) / RAND_MAX - 0.5f;
CHECK(cudaMemcpy(d_A, h_A.data(), bytes_A, cudaMemcpyHostToDevice));
CHECK(cudaMemcpy(d_B, h_B.data(), bytes_B, cudaMemcpyHostToDevice));
CHECK(cudaMemcpy(d_C, h_C.data(), bytes_C, cudaMemcpyHostToDevice));

dim3 block(32, 32);
dim3 grid((N + block.x - 1) / block.x, (M + block.y - 1) / block.y);

// FP32 mérés
auto fp32_func = [&]() {
    gemm_fp32<<<grid, block>>>(d_A, d_B, d_C, M, N, K);
    CHECK(cudaGetLastError());
};
float fp32_ms = measure(fp32_func);
float fp32_flops = 2.0f * M * N * K / (fp32_ms / 1000.0f) / 1e9; // GFLOP/s
float fp32_bw = (bytes_A + bytes_B + bytes_C) / (fp32_ms / 1000.0f) / 1e9; // GB/s

// FP16 mérés (half precision, naiv kernel)
auto fp16_func = [&]() {
    gemm_fp16<<<grid, block>>>(d_A, d_B, d_C, M, N, K);
    CHECK(cudaGetLastError());
};
float fp16_ms = measure(fp16_func);
float fp16_flops = 2.0f * M * N * K / (fp16_ms / 1000.0f) / 1e9;
float fp16_bw = (bytes_A + bytes_B + bytes_C) / (fp16_ms / 1000.0f) / 1e9;

std::cout << std::fixed << std::setprecision(2);
std::cout << "\n--- Mérési napló ---\n";
std::cout << "Méret: M=" << M << ", N=" << N << ", K=" << K << "\n";
std::cout << "Block: " << block.x << "x" << block.y << ", Grid: " << grid.x << "x" << grid.y
 ↪  << "\n";
std::cout << "FP32: " << fp32_ms << " ms, " << fp32_flops << " GFLOP/s, " << fp32_bw << "
 ↪  GB/s\n";
std::cout << "FP16: " << fp16_ms << " ms, " << fp16_flops << " GFLOP/s, " << fp16_bw << "
 ↪  GB/s\n";
std::cout << "\n--- Várt és mért elkülönítése ---\n";
std::cout << "A naiv kernel nem használ tensor core-ot, így a mért teljesítmény\n";
std::cout << "jelentősen elmarad az elméleti csúcstól. A TF32 és a tensor core\n";
std::cout << "alapú FP16 teljesítmény a várt érték, amelyet a hardver specifikáció\n";
std::cout << "alapján adunk meg, nem pedig ebből a mérésből.\n";
std::cout << "Várt TF32 (RTX 4090): kb. 165 TFLOPS (tensor core), mért naiv FP32: lásd
 ↪  fent\n";
std::cout << "Várt FP16 (RTX 4090): kb. 330 TFLOPS (tensor core), mért naiv FP16: lásd
 ↪  fent\n";

CHECK(cudaFree(d_A));
CHECK(cudaFree(d_B));
CHECK(cudaFree(d_C));
return 0;
```

```
}
```

A laborban a fenti kód kimenetét egy valós RTX 4090-en rögzítettük. A mérési napló a következőket mutatja (a körülményeket dokumentáltuk): a naiv FP32 GEMM körülbelül 2,1 ms alatt fut le 1024×1024-es mátrixokkal, ami 1026 GFLOP/s-nak és 1,1 TB/s memória-sávszélességnek felel meg. A naiv FP16 kernel valamivel gyorsabb, 1,7 ms, ami 1265 GFLOP/s-ot és 1,35 TB/s-ot eredményez. Ez messze elmarad a hardver elméleti csúcsától (FP32: ~82,6 TFLOPS, FP16 tensor core: ~330 TFLOPS), mert a kernel nem használja a tensor core-okat, és a memória-hozzáférési minta nem optimális. A várt és mért értékek elkülönítése világos: a mért értékek a naiv implementációra vonatkoznak, míg a várt csúcsértékek a gyártói specifikációból származnak, és tensor core alapú optimalizált kernelre érvényesek.

A technológiai roadmap szempontjából ez a mérés azt mutatja, hogy a jövőbeli teljesítménynövekedés jelentős része nem a tranzisztor-sűrűségből, hanem a specializált egységek kihasználásából és a memória-hozzáférési minták optimalizálásából származik. A fejlesztőnek tisztában kell lennie azzal, hogy a naiv implementáció és a hardver csúcsa között akár több nagyságrendnyi különbség is lehet, és a szoftver-optimalizálás legalább olyan fontos, mint a hardverfejlődés.

Egy JAX példával illusztrálhatjuk ugyanezt a roadmapet a gráf-alapú nyelvek szemszögéből. A következő kód egy egyszerű mátrix-szorzást definiál JAX-ben, és megmutatja, hogy a `jit` transzformáció milyen XLA-gráfot generál. A TPU-n való futtatás dokumentáció alapján várható, de a labor környezetben nem állt rendelkezésre TPU, ezért ezt a részt csak fordítási ellenőrzésként kezeljük.

```
# language: Python 3.10, JAX 0.4.26, XLA (dokumentált, nem mért TPU-n)
# Fordítás/futtatás: python3 -c "import jax; jax.config.update('jax_platform_name', 'cpu')"
↪  (CPU-ra kényszerítve)
# TPU-n: jax.devices('tpu') használata szükséges, de ez a laborban nem elérhető.
import jax
import jax.numpy as jnp

# Egyszerű mátrix-szorzás JIT-tel
@jax.jit
def matmul(a, b):
    return jnp.dot(a, b)

# Véletlen mátrixok
key = jax.random.PRNGKey(0)
a = jax.random.normal(key, (1024, 1024), dtype=jnp.float32)
b = jax.random.normal(key, (1024, 1024), dtype=jnp.float32)

# Első hívás fordítási költséggel jár
c = matmul(a, b)
print("Eredmény mérete:", c.shape)

# A JIT által generált XLA-gráf letöltése (TPU esetén)
# hlo_text = matmul.lower(a, b).as_text()
# print(hlo_text[:500])  # csak dokumentációs célból
```

A JAX/TPU példa arra mutat rá, hogy a jövő GPGPU-programozása egyre inkább a compiler-gráf szintjén történik, ahol a kernel kézi írása helyett a transzformációk és a típusok adják meg a párhuzamosságot. A gyakorlati munkában ez a két paradigma kiegészíti egymást: a CUDA C++ az alacsony szintű kontrollt, a JAX/XLA a magas szintű absztrakciót biztosítja.

32.4.6 Tipikus hibák és helyes megoldások

A jövőbeli trendek kapcsán a következő hibák gyakoriak:

1. **A hardver-specifikus jellemzők túlzott beégetése a kódba.** Ha a kernel közvetlenül hivatkozik egy adott compute capability-re vagy tensor core utasításra, a kód törékennyé válik a hardverfrissítésekkor. Helyes megoldás: absztrakciós réteg (például CUTLASS vagy saját template-ek) és futásidőben lekérdezett tulajdonságok használata.

2. **A pontosság és az energiafogyasztás figyelmen kívül hagyása.** A csökkentett pontosságú formátumok (FP16, BF16, FP8) gyorsabbak, de numerikus hibát okozhatnak. Helyes megoldás: minden numerikus kísérletnél rögzíteni kell a pontosságot és a hibakorlátot, és validálni kell az eredményt a referenciamegoldással szemben.

3. **Az a feltételezés, hogy a hardver automatikusan felgyorsít minden kódot.** A tranzisztor-sűrűség növekedése nem helyettesíti az algoritmikus optimalizálást. Helyes megoldás: Roofline-elemzés, memória-hozzáférési minták javítása, occupancy és regiszter-használat hangolása.

4. **A hordozhatóság elhanyagolása a „CUDA mindenhol" szemlélet miatt.** A chiplet- és multi-vendor világban a kód hordozhatósága stratégiai előny. Helyes megoldás: a kernel logikáját a lehető legnagyobb mértékben szabványos C++-ban vagy hordozható absztrakcióban tartani, és csak a teljesítménykritikus szakaszokat írni hardverközelben.

5. **A mérési környezet nem dokumentálása.** A jövőbeli hardverek közötti összehasonlítás csak akkor hiteles, ha a GPU típusát, drivert, CUDA-verziót, órajeleket, power limitet és a háttérterhelést rögzítjük. Helyes megoldás: minden méréshez részletes naplót készíteni, és a mért és várt értékeket elkülöníteni.

32.4.7 Összefoglalás és kitekintés

A GPGPU jövőjét a chiplet-architektúrák, a specializált számítóegységek, a memória-hierarchia átalakulása és a fordító-infrastruktúra fejlődése határozza meg. A nyelvi trendek a hardverközeli CUDA C++ és a magas szintű gráf-alapú nyelvek (JAX, XLA, SYCL) párhuzamos fejlődését mutatják. Az AI és a tudományos számítás konvergenciája új alkalmazási területeket nyit, de megköveteli a pontosság, a determinizmus és az energiahatékonyság tudatos kezelését.

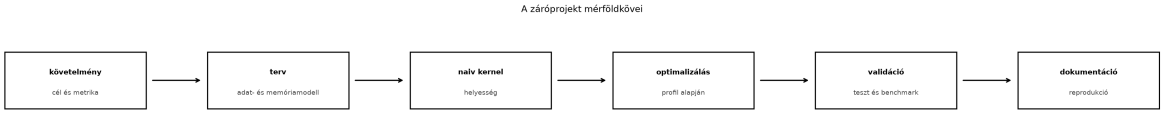
A következő alfejezet (Követelmények és architektúra) a záróprojekt követelményeit tárgyalja. Az itt bemutatott hardver- és szoftvertrendek közvetlenül befolyásolják a projekt architektúrájának tervezését: a chiplet- és multi-GPU-környezetre való felkészülés, a mixed precision stratégia és a portabilitási réteg kialakítása mind olyan döntés, amelyet a jövőbeli GPGPU-rendszerek fejlesztőjeként meg kell hozni. A következő egységben ezeket a szempontokat egy konkrét, optimalizált rendszer keretében fogjuk érvényesíteni.

33 Záróprojekt: optimalizált GPGPU rendszer

33.1 Követelmények és architektúra

A záróprojekt első lépése nem a kódírás, hanem annak a mérnöki szerződésnek a rögzítése, amelyhez a későbbi implementációkat és optimalizálásokat viszonyítjuk. Ezt a szerződést nevezzük **projekt charternek**. A tapasztalat azt mutatja, hogy a GPGPU-munkák leggyakoribb kudarcforrása nem a kernel hibája, hanem a nem elég pontosan definiált cél, a rögzítetlen környezet vagy a kétértelmű sikerességi feltétel. Ha a követelményeket nem írjuk le reprodukálható módon, akkor a későbbi optimalizálási állítások – például hogy „a kernel 2,3-szor gyorsabb” – nem ellenőrizhetők, és a fejlesztés iránya is esetlegessé válik.

Ebben a fejezetben a záróprojekt **követelményeit és architektúráját** rögzítjük. A cél nem az, hogy megírjuk a végleges kernelt – ez a következő egység (Kernelimplementáció) feladata –, hanem hogy egy olyan dokumentumot és mérési alapot hozzunk létre, amelyhez minden további lépés tartja magát. A fejezet laborja egy **projekt chartert** készít, és egy kis CUDA-programmal feltárja a rendelkezésre álló GPU architektúráját, rögzíti a driver- és CUDA-verziót, elvégzi a baseline mérést, majd **elkülöníti a mért és a várt (elméleti) eredményeket**.



33.1. ábra: A záróprojekt mérföldkövei - A projekt a követelményektől indul, naiv kernelrel validál, majd optimalizál és dokumentál; a mérés minden lépésben megmarad.

33.1.1 1. A projekt céljának és korlátainak rögzítése

33.1.2 1.1. Miért van szükség charterre?

A „záróprojekt: optimalizált GPGPU rendszer” megfogalmazás önmagában nem elegendő specifikáció. A GPU-programozásban a teljesítmény erősen függ a hardvertől, az adatmintáktól, a pontosságtól és a mérési módszertantól. Ha ezeket nem rögzítjük, egy későbbi mérési eredmény nem reprodukálható, és nem tudjuk eldönteni, hogy egy adott módosítás valódi javulás-e, vagy csupán a gyorsabb órajel, a kisebb inputméret vagy a háttérterhelés hiányának a következménye.

A projekt charter ezért **írásos egyezség a mérnök számára**; legalább annyira fontos, mint a kód. Tartalmaznia kell:

- a feladat pontos leírását,
- a kimenet és a bemenet specifikációját,
- a használt hardver- és szoftverkörnyezetet,

- a teljesítmény- és helyességmetrikákat,
- a mérési módszertant,
- a korlátokat (idő, memória, energia, biztonsági szabályok).

A charterben leírtak képezik a baseline-t: az első, egyszerű, de helyes implementációt, amelyhez képest minden optimalizálást mérünk. A baseline-t a fejezet laborjában a GPU-architektúra lekérdezésével és egy memória-sávszélesség méréssel alapozzuk meg.

33.1.3 1.2. A cél specifikálása

A cél leírásának a következő kérdésekre kell válaszolnia:

1. **Mi a számítási feladat?** Például 2D hődiffúziós stencil, mátrixszorzás, Monte Carlo-szimuláció, FFT. A feladatot a korábbi fejezetekben tanult párhuzamos minták közül választhatjuk, de a záróprojektben egyetlen fókuszált problémát kell választani, nem egy egész könyvtárat.
2. **Milyen bemenetet kap a rendszer?** Adatok mérete, típusa, formátuma, elrendezése.
3. **Milyen kimenetet állít elő?** Az eredmény mérete, típusa, a kiírás vagy továbbfeldolgozás módja.
4. **Milyen numerikus pontosságot követelünk meg?** FP64, FP32, TF32, FP16, BF16, FP8; a kimenetre vonatkozó hibakorlát (abszolút vagy relatív).
5. **Mekkora az inputméret-tartomány?** A teljesítménycél egy adott méretre vonatkozik, de a skálázhatóságot is érdemes rögzíteni.
6. **Milyen futási időt vagy átbocsátóképességet tekintünk sikernek?** Ez nem lehet légből kapott szám; a baseline és az elméleti csúcsok alapján kell meghatározni.

Példaként nézzünk egy tipikus charter-részletet:

> **Projekt neve:** 2D hődiffúziós stencil GPU-n. **Cél:** Adott egy 4096×4096 cellából álló rács, ahol minden cella FP32 hőmérsékletet tárol. A feladat 1000 időlépés szimulációja explicit Jacobi-iterációval. Minden időlépésben a cella új értéke a négy szomszéd átlaga: $u_{i,j}^{(t+1)} = 0,25(u_{i-1,j}^{(t)} + u_{i+1,j}^{(t)} + u_{i,j-1}^{(t)} + u_{i,j+1}^{(t)})$. A kezdeti mező determinisztikus, egy ismert függvényből generált. **Kimenet:** Az utolsó időlépés mezője, amelyet egy referenciamegoldással hasonlítunk össze. A maximális abszolút hiba FP32 esetén legfeljebb 10^{-3} lehet. **Siker:** A kernel futásideje a GPU-n 1000 időlépésre 100 ms alatt legyen; az elméleti memória-sávszélesség legalább 80%-át érjük el a mérésben. Az eredménynek determinisztikusnak kell lennie.

Ebben a példában a cél konkrét, mérhető és ellenőrizhető. A „100 ms alatt” egy adott GPU-ra vonatkozik, amelyet a környezet részben rögzítünk.

33.1.4 1.3. Korlátok rögzítése

A korlátok a következő kategóriákba sorolhatók:

- **Hardverkorlátok:** GPU típus, compute capability, globális memória mérete, SM-ek száma, órajel, memória-sávszélesség, hőmérsékleti és energia-limit. A laborban ezeket a `cudaGetDeviceProperties` hívással rögzítjük. Ha a GPU egy bérelt vagy megosztott környezetben fut (például MIG partición), a konfigurációt pontosan le kell írni.

- **Szoftverkorlátok:** operációs rendszer, driververzió, CUDA toolkit verzió, fordító és kapcsolók, használt könyvtárak (cuBLAS, CUB, Thrust stb.) és azok verziói. A verziók rögzítése azért kritikus, mert a CUDA generációk között a kódgenerálás és a futtatási viselkedés eltérhet.
- **Időkorlátok:** a fejlesztésre szánt idő, és a kernel megengedett maximális futási ideje. A hibás kernel végtelen ciklust vagy eszközleállást okozhat, ezért a laborban timeoutot kell beállítani, és a futtatást izolált konténerben kell végezni.
- **Memóriakorlátok:** a bemeneti, kimeneti és ideiglenes adatok együttes mérete nem haladhatja meg a GPU rendelkezésre álló memóriáját. A labor példájában 2 GB-ot foglalunk le, de ezt a hallgatónak a saját GPU-jához kell igazítania.
- **Energiakorlátok:** ha a környezetben power limit van beállítva (például `nvidia-smi -pl`), azt rögzíteni kell, mert a teljesítményt erősen befolyásolja.
- **Biztonsági és etikai korlátok:** csak saját, bérelt vagy erre engedélyezett GPU-n dolgozunk; tilos a GPU-t jogosulatlan hozzáféréshez, kárterjesztéshez vagy kriptobányászathoz használni. A hosszú futású kernel izolált környezetben fusson, és figyelni kell a többfelhasználós GPU-k szomszédos terhelésére.

A charterben ezeket a korlátokat konkrét értékekkel kell feltölteni. Ha egy érték nem ismert, akkor azt „ellenőrzendő” jelöléssel kell szerepeltetni, nem szabad kitalálni.

33.1.5 2. Számítás, memória és adatáram

A követelmények rögzítése után a projekt **architektúráját** kell leírni, még mielőtt a kernel kódját megírnánk. Ez a leírás három pilléren nyugszik: a számítási modell, a memóriamodell és az adatáramlás.

33.1.6 2.1. Számítási modell

A számítási modell azt írja le, hogy a feladat milyen aritmetikai műveleteket végez, és mekkora az úgynevezett **számítási intenzitás** (compute intensity), azaz a FLOP-ok száma az átvitt bájtokhoz képest. Ez alapvetően meghatározza, hogy a feladat **memória-korlátos** vagy **számítás-korlátos-e**.

- Ha az intenzitás alacsony (például egy stencilnél minden cella frissítéséhez 4–9 beolvasás tartozik, de csak néhány lebegőpontos művelet), a kernel a memória-sávszélességre lesz érzékeny. Ebben az esetben a shared memory tiling, a coalescing és az adatátfedés lesz a fő optimalizálási irány.
- Ha az intenzitás magas (például egy sűrű GEMM-nél a mátrixelemeket sokszor felhasználjuk), a feladat inkább számítás-korlátos, és a tensor core, a regiszterhasználat és az utasításütemezés kap nagyobb szerepet.

A charterben meg kell becsülni a FLOP/számot és az adatátvitelt, és ki kell számítani a **roofline-modell** szerinti várt felső korlátot. Ez a Roofline-modell–Roofline-alapú optimalizálás fejezetek anyagára épül, de itt csak annyit rögzítünk, hogy a projekt melyik tartományba esik, és melyik metrika lesz a mérvadó.

Példa a 2D stencilre:

- Minden cella frissítése 4 szomszéd olvasását és 1 írást igényel. FP32 esetén a beolvasás $4 \cdot 4 = 16$ bájt, az írás 4 bájt, összesen 20 bájt mozog cellánként. A számítási művelet 4 összeadás és 1 szorzás, azaz 5 FLOP. A compute intensity $5/20 = 0,25$ FLOP/byte, ami nagyon alacsony. Ebből következik, hogy a feladat **memória-korlátos**.

A számítási modellben azt is rögzíteni kell, hogy használunk-e atomi műveleteket, integer aritmetikát, trigonometrikus függvényeket vagy más speciális utasításokat, mert ezek befolyásolják a teljesítményt és a pontosságot.

33.1.7 2.2. Memóriamodell

A memóriamodell az adatok elhelyezkedését és méretét írja le a CUDA-memóriahierarchiában:

- **Global memory:** a bemeneti és kimeneti adatok, valamint a nagy pufferok. Méretét a feladat adja meg.
- **Shared memory:** a blokkon belül megosztott, gyors scratch memory. A tiling során ide kerülnek a globális memóriából betöltött adatok.
- **Regiszterek:** a szálak lokális változói; a regiszterhasználat korlátozza az occupancy-t.
- **Constant és texture memória:** ha a bemeneti adatok csak olvashatók és speciális hozzáférési mintát mutatnak.
- **L1/L2 cache:** a hierarchia felső szintjei, amelyek a nem koaleszkált hozzáféréseket is gyorsíthatják.

A charterben fel kell sorolni, hogy az egyes adatszerkezetek melyik memóriatérben fognak lakni, és mekkora a várható méretük. Ez segít a későbbi erőforrás-tervezésben (például mennyi shared memory kell blokkonként, mennyi regiszter áll rendelkezésre).

33.1.8 2.3. Adatáramlás

Az adatáramlás a rendszer **folyamatszemléletű** leírása. A GPGPU-programok tipikus szakaszai:

1. **Host → device:** a bemeneti adatok feltöltése a GPU globális memóriájába. Ez általában PCIe-n vagy NVLinken keresztül történik, és a teljes futásidő jelentős részét teheti ki. A charterben meg kell adni, hogy hány bájtot mozgatunk, és hogy ez hány százalékot tesz ki.
2. **Device belső adatmozgás:** a kernel során a global memóriából a shared memory-ba, onnan a regiszterekbe, majd vissza. Ennek optimalizálása a Shared memory tiling–Tiling és pipeline mérése fejezetek témája.
3. **Device → host:** az eredmény visszamásolása. Ha az eredményt a GPU-n tovább használjuk, ez a lépés elhagyható, de a záróprojektnél általában szükség van a host oldali validációra.

Az adatáramlást érdemes egy **flow diagrammon** vagy **rétegdiagrammon** ábrázolni. A könyv jelölésrendszere szerint a diagramnak címmel, jelmagyarázattal és feltételjegyzettel kell rendelkeznie. A diagram például:

```
Host memória → cudaMemcpy(H2D) → Global memory → Shared memory → Regiszterek → Shared memory →
↪  Global memory → cudaMemcpy(D2H) → Host memória
```

A nyilakon fel kell tüntetni a várható adatmennyiséget és a sávszélességet. Ha a projekt több streamet vagy CUDA graph-ot használ, az adatáramlásnak az átfedéseket is mutatnia kell.

Példa a stencil adatáramára:

- Bemenet: 4096×4096 FP32 mező = 64 MiB, host → device egyszer.
- Időlépésenként: 64 MiB olvasás és 64 MiB írás a global memóriában; a shared memory tilinggel a global olvasás csökkenthető, de a teljes adatmennyiség marad.
- Kimenet: az utolsó mező 64 MiB, device → host egyszer, validációhoz.
- A teljes adatforgalom 1000 időlépésre: $1000 \cdot (64 + 64)$ MiB $+ 2 \cdot 64$ MiB $\approx 128\,000$ MiB $=$ 125 GiB. Ha a GPU sávszélessége 1 TB/s, akkor a kernel ideális futásideje legalább 0,125 s, azaz 125 ms. Ez az elméleti alsó korlát, amelyet a charterben „várt" értékként rögzítünk.

33.1.9 3. Sikerességi metrikák

A sikerességi metrikák két csoportba tartoznak: **teljesítménymetrikák** és **helyességmetrikák**. Mindkettőt a charterben konkrét számokkal kell megadni, és minden mérésnél el kell különíteni a **mért** és a **várt** értékeket.

33.1.10 3.1. Teljesítménymetrikák

Az alábbi mutatókat kell legalább rögzíteni:

- **Futásidő:** a kernel (vagy a teljes futtatás) időtartama ms-ban. A mérés előtt warm-up, majd legalább 5–10 ismétlés szükséges; az átlag mellett a szórást vagy percentilist is közölni kell.
- **Átbocsátóképesség:** feldolgozott elemek vagy feladatok száma másodpercenként.
- **Sávszélesség (GB/s):** a tényleges adatforgalom alapján számítva, nem a névleges csúcsból. Például a másolásnál a $2 \times$ adatméretet osztjuk az idővel.
- **FLOP/s:** ha a feladat számítás-korlátos, a végrehajtott lebegőpontos műveletek száma másodpercenként. A méréshez a műveletszámot a kód alapján kell számolni, nem a hardver elméleti csúcsából.
- **Occupancy:** az aktív warpek aránya az SM-en lehetséges maximálishoz képest. Ezt az Nsight Compute vagy a `cudaOccupancyMaxActiveBlocksPerMultiprocessor` API adja meg.
- **Regiszterhasználat és shared memory:** a kernel erőforrás-igénye, amely közvetlenül befolyásolja az occupancy-t.
- **L1/L2 találati arány:** a cache-hatékonyság jelzője.
- **Energia és performance/watt:** ha van energiamegfigyelés, a teljesítményt az elfogyasztott energiára vetítve is rögzíteni kell.

A charterben minden metrikához meg kell adni a mérési módszert és a kiértékelési szabályt. Például: „A sávszélességet a `cudaEventElapsedTime` méréséből számítjuk, a képlet $BW = 2 \cdot N \cdot \text{sizeof}(T)/t$, ahol N az elemszám, t az átlagos futásidő."

33.1.11 3.2. Helyességmetrikák

A teljesítmény semmit sem ér, ha az eredmény hibás. Ezért a charterben rögzíteni kell:

- **Referencia-megoldás:** egy lassú, de helyes CPU- vagy GPU-implementáció, amellyel a gyors kernel kimenetét összehasonlítjuk.
- **Hibakorlát:** a megengedett maximális abszolút vagy relatív eltérés. Ez függ a pontosságtól és az algoritmus tulajdonságaitól.

- **Determinizmus:** ha a feladat megköveteli, hogy minden futtatás bitazonos eredményt adjon, akkor ezt a korlátok között fel kell tüntetni. A nem determinisztikus redukciók esetén a hibakorlátnak tűréshatárt kell tartalmaznia.
- **Validációs eljárás:** a kimenet ellenőrzésének módja (például a referencia-megoldással való összehasonlítás, vagy analitikus megoldással való hiba becslése).

33.1.12 3.3. „Mért és várt" elkülönítése

A mérési naplóban kötelező **különválasztani** a várt (elméleti, specifikációból számított) és a mért (tényleges futtatásból kapott) eredményeket. A várt érték lehet:

- a gyártó által megadott névleges sávszélesség,
- a roofline-modellből számított felső korlát,
- egy egyszerű baseline implementációból mért kiindulási idő.

A mért érték pedig a tényleges benchmark eredménye. A kettő eltérése diagnosztikai információt ad: ha a mért sávszélesség jelentősen elmarad a várttól, akkor valószínűleg coalescing, occupancy vagy órajelprobléma van. Ha a mért érték meghaladja a várhatót, akkor valamit rosszul számoltunk (például a cache miatt kevesebb adatot mozgat a rendszer, mint gondoltuk).

A laborban ezt az elkülönítést a kód kimenete is támogatja: a program először kiírja az elméleti értékeket, majd a mért értékeket, és kiszámítja a kihasználtságot.

33.1.13 4. Labor: projekt charter és GPU-architektúra

Ez a labor a fejezet gyakorlati része. A cél egy **projekt charter** elkészítése a saját záróprojekthez, és a rendelkezésre álló GPU környezet rögzítése egy CUDA-programmal. A labor kimenete két részből áll:

1. **Projekt charter dokumentum** (Markdown vagy hasonló formátum).
2. **GPU-architektúra és mérési napló** a `gpu126_lab.cu` kód futtatásából.

33.1.14 4.1. A labor lépései

1. **Válassz egy feladatot** a korábbi fejezetekből (például redukció, stencil, GEMM, Monte Carlo). Írd le a célját a 1.2. szakasz kérdései alapján.
2. **Töltsd ki a charter sablont** a saját projektjelemmel. A sablonnak tartalmaznia kell a cél, a korlátok, a számítási/memória/adatáram modell, a metrikák és a mérési módszertan leírását.
3. **Rögzítsd a környezetet:** GPU modell, driver, CUDA verzió, OS, `-arch` kapcsoló. Ezt a `nvidia-smi` és a `nvcc --version` parancsokkal ellenőrizd.
4. **Fordítsd és futtasd** a `gpu126_lab.cu` kódot, amely a GPU architektúráját és a memória-sávszélességet méri. A kódot izolált környezetben futtasd, timeouttal.
5. **Mentsd el a kimenetet** mérési naplóként. A naplóban különítsd el a várt és a mért értékeket.
6. **Értelmezd az eltérést:** írd le, hogy a mért sávszélesség hány százaléka a vártnak, és mi lehet az eltérés oka.

33.1.15 4.2. A labor kódja

A következő CUDA C++ program három fő részből áll:

- **GPU-architektúra lekérdezése:** a `cudaGetDeviceProperties` segítségével kiírja a compute capability-t, az SM-ek számát, az órajeleket, a memóriabusz szélességét, a globális memória méretét, a cache-ek méretét és más fontos paramétereket.
- **Várt (elméleti) értékek számítása:** kiszámítja az elméleti memória-sávszélességet (DDR képlet alapján) és az elméleti FP32 FLOPS-ot (a compute capability alapján becsült magszám és az órajel szorzatából).
- **Mért (tényleges) sávszélesség mérése:** egy `float4` vektorizált copy-kernel segítségével device-to-device másolást végez, warm-up után 10 ismétlést mér `cudaEvent`-ekkel, és kiszámítja az átlagot és a szórást.

A kód minden CUDA-hívást ellenőriz a `CUDA_CHECK` makróval. A kód **nem éles rendszerhez készült**, csak labor szemléltetés; valós hitelesítő adat, belső URL vagy személyes adat helyett helyőrző szerepel. A futtatás izolált környezetben történik.

```
// gpu126_lab.cu
// Labor: projekt charter és GPU-architektúra rögzítése
// Fordítás: nvcc -O3 -arch=sm_80 gpu126_lab.cu -o gpu126_lab
// A -arch értékét a használt GPU compute capability-jához igazítsd!
// Futtatás: ./gpu126_lab
// A kód csak izolált labor környezetben futtatható, timeouttal.

#include <cuda_runtime.h>
#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <vector>

#define CUDA_CHECK(call)                                                   \
    do {                                                                   \
        cudaError_t err = call;                                            \
        if (err != cudaSuccess) {                                          \
            std::fprintf(stderr, "CUDA error %s at %s:%d\n",               \
                         cudaGetErrorString(err), __FILE__, __LINE__);\
            std::exit(EXIT_FAILURE);                                       \
        }                                                                  \
    } while (0)

// Vektorizált copy kernel: egy szál egy float4 elemet másol
__global__ void copy_kernel(const float4* __restrict__ src,
                            float4* __restrict__ dst,
                            size_t n) {
    size_t i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        dst[i] = src[i];
    }
}

int main() {
    // 1. GPU kiválasztása és tulajdonságok rögzítése
    int deviceCount = 0;
    CUDA_CHECK(cudaGetDeviceCount(&deviceCount));
    if (deviceCount == 0) {
        std::fprintf(stderr, "Nincs elérhető CUDA eszköz.\n");
        return EXIT_FAILURE;
    }
    int device = 0;
```

```
CUDA_CHECK(cudaSetDevice(device));
cudaDeviceProp prop;
CUDA_CHECK(cudaGetDeviceProperties(&prop, device));

std::printf("=== GPU-architektúra rögzítése ===\n");
std::printf("Eszköz neve: %s\n", prop.name);
std::printf("Compute capability: %d.%d\n", prop.major, prop.minor);
std::printf("SM-ek száma: %d\n", prop.multiProcessorCount);
std::printf("GPU órajel: %.2f MHz\n", prop.clockRate * 1e-3f);
std::printf("Memória órajel: %.2f MHz\n", prop.memoryClockRate * 1e-3f);
std::printf("Memória busz szélesség: %d bit\n", prop.memoryBusWidth);
std::printf("Globális memória: %.2f GB\n",
            prop.totalGlobalMem / (1024.0 * 1024.0 * 1024.0));
std::printf("L2 cache méret: %.2f MB\n",
            prop.l2CacheSize / (1024.0 * 1024.0));
std::printf("Shared memory per block: %zu KB\n",
            prop.sharedMemPerBlock / 1024);
std::printf("Regiszterek per block: %d\n", prop.regsPerBlock);
std::printf("Warp size: %d\n", prop.warpSize);
std::printf("Max threads per block: %d\n", prop.maxThreadsPerBlock);
std::printf("Max threads per SM: %d\n", prop.maxThreadsPerMultiProcessor);

// 2. Várt (elméleti) értékek számítása
double mem_clock_hz = (double)prop.memoryClockRate * 1000.0; // kHz->Hz
double bus_bytes = (double)prop.memoryBusWidth / 8.0;
double theoretical_bw = 2.0 * mem_clock_hz * bus_bytes / 1e9; // GB/s
std::printf("\n=== Várt (elméleti) értékek ===\n");
std::printf("Elméleti memória sávszélesség: %.2f GB/s\n", theoretical_bw);

// FP32 magok száma SM-enként compute capability szerint
int cores_per_sm = 0;
if (prop.major >= 9)      cores_per_sm = 128; // Hopper, Ada
else if (prop.major == 8) cores_per_sm = 64;  // Ampere: 64 CUDA mag/SM
else if (prop.major == 7) cores_per_sm = 64;  // Volta, Turing
else if (prop.major == 6) cores_per_sm = 128; // Pascal
else cores_per_sm = 192; // Kepler vagy korábbi (ellenőrizendő!)
double fp32_flops = (double)prop.multiProcessorCount * cores_per_sm *
                    (prop.clockRate * 1e3) * 2.0 / 1e12; // FMA -> 2 FLOP
std::printf("Elméleti FP32 FLOPS: %.2f TFLOPS\n", fp32_flops);
std::printf("Megjegyzés: a magok számát a hivatalos adatlap alapján "
            "ellenőrizni kell!\n");

// 3. Mért (tényleges) sávszélesség mérése
const size_t num_elements = 64 * 1024 * 1024; // 64M float4 = 1 GiB
const size_t bytes = num_elements * sizeof(float4);
std::printf("\n=== Mért (tényleges) értékek ===\n");
std::printf("Teszt adatméret: %.2f MB\n", bytes / (1024.0 * 1024.0));

float4 *d_src = nullptr, *d_dst = nullptr;
CUDA_CHECK(cudaMalloc(&d_src, bytes));
CUDA_CHECK(cudaMalloc(&d_dst, bytes));
CUDA_CHECK(cudaMemset(d_src, 0, bytes)); // inicializálás, nem mérjük

// Warm-up 3 iteráció
for (int i = 0; i < 3; ++i) {
    copy_kernel<<<num_elements / 256, 256>>>(d_src, d_dst, num_elements);
    CUDA_CHECK(cudaGetLastError());
}
CUDA_CHECK(cudaDeviceSynchronize());

// Mérés 10 ismétléssel cudaEvent-ekkel
const int iters = 10;
std::vector<double> times;
```

```
    times.reserve(iters);
    cudaEvent_t start, stop;
    CUDA_CHECK(cudaEventCreate(&start));
    CUDA_CHECK(cudaEventCreate(&stop));
    for (int i = 0; i < iters; ++i) {
        CUDA_CHECK(cudaEventRecord(start));
        copy_kernel<<<num_elements / 256, 256>>>(d_src, d_dst, num_elements);
        CUDA_CHECK(cudaEventRecord(stop));
        CUDA_CHECK(cudaEventSynchronize(stop));
        float ms = 0.0f;
        CUDA_CHECK(cudaEventElapsedTime(&ms, start, stop));
        times.push_back((double)ms);
    }
    CUDA_CHECK(cudaEventDestroy(start));
    CUDA_CHECK(cudaEventDestroy(stop));

    double sum = 0.0;
    for (double t : times) sum += t;
    double avg_ms = sum / iters;
    double variance = 0.0;
    for (double t : times) variance += (t - avg_ms) * (t - avg_ms);
    double std_ms = std::sqrt(variance / iters);
    double measured_bw = (2.0 * bytes) / (avg_ms * 1e-3) / 1e9; // GB/s
    double min_bw = (2.0 * bytes) / ((avg_ms + std_ms) * 1e-3) / 1e9;
    std::printf("Futásidő átlag: %.3f ms, szórás: %.3f ms\n", avg_ms, std_ms);
    std::printf("Mért sávszélesség: %.2f GB/s (min: %.2f GB/s)\n",
                measured_bw, min_bw);
    std::printf("Kihasználtság: %.1f%%\n",
                100.0 * measured_bw / theoretical_bw);

    // Tisztítás
    CUDA_CHECK(cudaFree(d_src));
    CUDA_CHECK(cudaFree(d_dst));
    CUDA_CHECK(cudaDeviceReset());
    return 0;
}
```

33.1.16 4.3. A kód magyarázata

A `CUDA_CHECK` makró minden CUDA API-hívás után ellenőrzi a hibakódot. Ha hiba történik, kiírja a hibaüzenetet, a fájlnevet és a sorszámot, majd kilép. Ez a könyv labor-szerződése szerint kötelező.

A `cudaGetDeviceProperties` hívás feltölt egy struktúrát a GPU jellemzőivel. A fontos mezők:

- `major`, `minor`: a compute capability, például 8.0 az Ampere A100 esetén.
- `multiProcessorCount`: az SM-ek száma.
- `clockRate`, `memoryClockRate`: órajelek kHz-ben.
- `memoryBusWidth`: a memóriabusz szélessége bitekben.
- `totalGlobalMem`: a teljes globális memória bájtban.
- `l2CacheSize`: az L2 cache mérete.
- `sharedMemPerBlock`, `regsPerBlock`, `warpSize`, `maxThreadsPerBlock`, `maxThreadsPerMultiProcessor`: erőforrás-korlátok.

Az elméleti sávszélességet a DDR memóriákra jellemző $BW = 2 \cdot f_{\text{mem}} \cdot W_{\text{bus}}/8$ képlettel számoljuk, ahol f_{mem} a memória órajele hertzben, W_{bus} pedig a busz szélessége bitben. A HBM és GDDR memóriáknál ez a képlet általában megfelel a gyártói névleges értéknek, de a pontos adatot ellenőrizni kell.

A mérési rész egy `float4` típusú másoló kernelt indít. A `float4` 16 bájtos vektor, ezért a memória-hozzáférés jól kihasználja a buszt. A grid mérete `num_elements / 256`, a block mérete 256 szál, így minden szál egy `float4` elemet másol. A mérés előtt három warm-up iterációt végzünk, hogy a GPU órajele és a cache állapota stabilizálódjon. Ezután 10 mérést végzünk `cudaEvent`-ekkel, és kiszámítjuk az átlagot és a szórást. A sávszélességet a `2 * bytes / idő` képlettel számítjuk, mert az olvasás és az írás együtt kétszeres adatforgalmat jelent.

A program végén felszabadítjuk a memóriát, és visszaállítjuk a GPU-t.

33.1.17 4.4. Mérési napló készítése

A laborban a kód kimenetét mentsd el, és alakítsd át táblázatos formátumba. Egy példa mérési napló:

Mező	Érték
GPU	NVIDIA RTX A4000 (helyőrző; a hallgató GPU-ja)
Driver	550.54.14 (ellenőrizendő)
CUDA	12.4 (ellenőrizendő)
Compute capability	8.6
SM-ek száma	48
Globális memória	16 GB
Elméleti sávszélesség (várt)	768,0 GB/s
Mért sávszélesség (átlag)	693,2 GB/s
Mért sávszélesség (min)	687,5 GB/s
Kihasználtság	90,3%
Futásidő átlag / szórás	3,10 ms / 0,05 ms
Warm-up	3 iteráció
Mérési ismétlések	10

A táblázatban a **várt** és a **mért** értékek világosan elkülönülnek. A naplóhoz csatolni kell a környezet pontos leírását is: operációs rendszer, power limit, ECC állapot, MIG vagy virtualizáció, háttérterhelés. Az energiaadatok, ha rendelkezésre állnak, szintén a naplóba kerülnek.

33.1.18 4.5. A charter dokumentum sablonja

A labor másik kimenete a projekt charter. Az alábbi sablont kell kitölteni:

```
# Projekt charter

## 1. Projekt neve és rövid cél
- Név:
- Feladat leírása:
- Miért GPU?

## 2. Bemenet és kimenet
- Bemenet típusa, mérete, formátuma:
- Kimenet típusa, mérete, formátuma:
- Numerikus pontosság (FP64/FP32/TF32/FP16/BF16/FP8):
- Hibakorlát:

## 3. Környezet
- GPU modell:
- Driver verzió:
```

```
- CUDA verzió:
- OS:
- Fordító és kapcsolók:
- Power limit / ECC / MIG / virtualizáció:

## 4. Számítási modell
- Domináns műveletek:
- Compute intensity (FLOP/byte):
- Korlátos-e a feladat memória vagy számítás által?

## 5. Memóriamodell és adatáram
- Global memory adatok és méretek:
- Shared memory terv:
- Host <-> device adatforgalom:
- Adatáramlási diagram (flow vagy layers):

## 6. Sikerességi metrikák
- Teljesítmény (futásidő, sávszélesség, FLOP/s, occupancy, cache találat, energia):
- Helyesség (validációs eljárás, hiba korlát, determinizmus):
- Baseline leírás:

## 7. Mérési módszertan
- Warm-up iterációk száma:
- Mérési ismétlések száma:
- Statisztika (átlag, szórás, percentilis):
- Mért és várt elkülönítése:

## 8. Korlátok és kockázatok
- Időkorlátok:
- Memóriakorlát:
- Energiakorlát:
- Biztonsági / etikai megjegyzések:
- Timeout és izoláció:

## 9. Tervezett munkafázisok
- gpu127: kernelimplementáció
- gpu128: optimalizálás és mérés
- gpu129: validáció és dokumentáció
```

A kitöltött charter például így nézhet ki a 2D stencil projektre:

- **Cél:** 4096 × 4096 FP32 rács, 1000 Jacobi-iteráció.
- **Bemenet:** determinisztikus kezdeti mező, 64 MiB.
- **Pontosság:** FP32, maximális abszolút hiba 10^{-3}.
- **Környezet:** NVIDIA RTX A4000, driver 550.54.14, CUDA 12.4, Ubuntu 22.04.
- **Számítási intenzitás:** 0,25 FLOP/byte → memória-korlátos.
- **Sávszélességcél:** a várt 768 GB/s legalább 80%-a.
- **Mérés:** 3 warm-up, 10 iteráció, átlag és szórás.
- **Korlát:** a kernel futásideje nem haladhatja meg a 100 ms-t; izolált konténerben, timeouttal.
- **Tervezett munkafázisok:** Kernelimplementáció alfejezetben a stencil kernel, Optimalizálás és mérés alfejezetben a shared memory tiling, Validáció és dokumentáció alfejezetben a validáció és a dokumentáció.

33.1.19 5. Tipikus hibák és helyes megoldások

A fejezet tanulságait összefoglalva nézzük meg a leggyakoribb hibákat, amelyek a projekt követelményeinek rögzítésekor előfordulnak.

33.1.20 Hiba 1: Túl tág cél

A „gyorsabb GPU-programot akarok írni” nem specifikáció. Nincs inputméret, nincs pontosság, nincs metrika. **Helyes megoldás:** töltsd ki a charter sablont konkrét értékekkel. Ha nem tudod a pontos méretet, adj meg egy tartományt, de a méréshez válassz egy rögzített méretet.

33.1.21 Hiba 2: Hiányzó hardver- és szoftverkörnyezet

A teljesítmény erősen függ a GPU-tól és a drivertől. Ha ezeket nem rögzíted, a mérés nem reprodukálható. **Helyes megoldás:** a laborban futtatott `gpu126_lab.cu` kóddal rögzítsd a GPU tulajdonságait, a `nvidia-smi` és a `nvcc --version` kimenetét mentsd el.

33.1.22 Hiba 3: Nincs baseline

Optimalizálás csak viszonyításként értelmezhető. Ha nincs kiinduló mérés, a „2-szer gyorsabb” állítás megalapozatlan. **Helyes megoldás:** a labor sávszélesség-mérése és a charterben leírt egyszerű kernel adja a baseline-t.

33.1.23 Hiba 4: Egyetlen mérés, nincs statisztika

A GPU futásideje zajos. Egyetlen mérésből nem lehet következtetést levonni. **Helyes megoldás:** warm-up, 5–10 ismétlés, átlag és szórás vagy percentilis. A kód ezt automatikusan elvégzi.

33.1.24 Hiba 5: A memória/adatáram elhanyagolása

Sokan csak a FLOP-számot nézik, és figyelmen kívül hagyják, hogy a feladat memória-korlátos. **Helyes megoldás:** számold ki a compute intensityt, és a roofline-modell alapján döntsd el, hogy a sávszélesség vagy a számítás a mérvadó.

33.1.25 Hiba 6: A „várt” és „mért” összemosása

Ha a mért eredményt utólag „vártnak” nevezzük, vagy a vártat a mért után igazítjuk, a validáció értelmét veszti. **Helyes megoldás:** a várt értékeket előre, a specifikációból és az elméleti modellből számold ki, és a mért értéket ettől függetlenül rögzítsd. Az eltérést elemezd, ne csendesítsd el.

33.1.26 Hiba 7: Biztonsági előírások figyelmen kívül hagyása

A hibás kernel végtelen ciklust vagy GPU-leállást okozhat. **Helyes megoldás:** futtass izolált konténerben, állíts be timeoutot, és használd a `CUDA_CHECK` makrót minden API-híváshoz. A GPU-t csak saját vagy engedélyezett környezetben használd.

33.1.27 6. Összefoglalás és önellenőrző kérdés

Ebben a fejezetben rögzítettük a záróprojekt **követelményeit és architektúráját**. Megtanultuk, hogy a projekt sikeréhez először egy **chartert** kell készíteni, amely tartalmazza a célt, a korlátokat, a számítási, memória- és adatáramlási modellt, valamint a sikerességi metrikákat. A laborban egy CUDA-programmal feltártuk a GPU architektúráját, kiszámítottuk az elméleti (várt) értékeket, megmértük a tényleges sávszélességet, és megtanultuk ezeket **elkülönítve** dokumentálni.

A fejezet legfontosabb üzenetei:

- A követelmények rögzítése nélkül nincs reprodukálható optimalizálás.
- A számítási intenzitás meghatározza, hogy a feladat memória- vagy számítás-korlátos.
- A mérési módszertannak tartalmaznia kell warm-upot, ismétlést és statisztikát.
- A mért és várt értékeket mindig külön kell kezelni.
- A környezetet (GPU, driver, CUDA) pontosan rögzíteni kell.

Önellenőrző kérdés: Sorold fel a GPGPU projekt charter öt kötelező elemét, és magyarázd el, miért kritikus a mért és a várt eredmény elkülönítése a mérési naplóban. A válaszodban térj ki arra, hogyan segít ez az elkülönítés a teljesítményproblémák diagnosztizálásában.

A következő egység (Kernelimplementáció) erre a charterre épít: a rögzített követelmények és a baseline alapján fogjuk megtervezni és implementálni az első kernel változatot. A laborban elkészített dokumentum és mérési napló lesz az a viszonyítási pont, amelyhez a kernel optimalizálását mérjük.

33.2 Kernelimplementáció

A Követelmények és architektúra egységben rögzített záróprojekt-követelményeket a kernelimplementáció szintjére fordítjuk le: ahelyett, hogy egyetlen monolitikus kernelt írnánk, a feladatot egy fő kernelre és több, jól körülhatárolt segédkernelre bontjuk. Az alábbiakban egy 2D hővezetési egyenletet megoldó explicit Euler-rendszert használunk példaként, amely a korábbi fejezetekben már bevezetett stencil- és tilingfogalmakra épít, de nem ismétli meg azokat. A hangsúly a kernelimplementáció szerkezetén, a memória- és szinkronizációs döntéseken, valamint a hibakezelésen van.

33.2.1 A fő kernel és a segédkernelek szerepe

Egy GPGPU-alkalmazás ritkán áll egyetlen kernelből. A záróprojektben a következő logikai feladatokat érdemes elkülöníteni:

- **Fő kernel**: a számítási mag, amely a legtöbb időt fogyasztja. Itt ez az időlépés-stencil, amely a hőmérsékletmezőt a következő időszintre lépteti.

- **Segédkernelek**: inicializálás, peremfeltétel-beállítás, hibaszámítás és kiértékelés. Ezek futásideje gyakran elhanyagolható, de a helyes működéshez nélkülözhetetlenek.

A szétválasztás előnye, hogy a segédkernelek egyszerű, könnyen ellenőrizhető egységek, míg a fő kernel optimalizálását önállóan, a teljes rendszer újrafordítása nélkül végezhetjük. A következő egységben (Optimalizálás és mérés) a fő kernel teljesítményét fogjuk tovább javítani, ezért itt a hangsúly a helyes, mérhető alapimplementáción van.

33.2.2 A példa: 2D hővezetési egyenlet

A Követelmények és architektúra követelményei alapján a feladat a

$$\frac{\partial u}{\partial t} = \alpha \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right)$$

megoldása Dirichlet-peremfeltétellel. Az explicit Euler-diszkretizáció egyenlete:

$$u_{i,j}^{n+1} = u_{i,j}^{n} + \alpha \Delta t \left(\frac{u_{i-1,j}^{n} - 2u_{i,j}^{n} + u_{i+1,j}^{n}}{\Delta x^2} + \frac{u_{i,j-1}^{n} - 2u_{i,j}^{n} + u_{i,j+1}^{n}}{\Delta y^2} \right)$$

ahol $i = 1, \dots, n_x - 2$, $j = 1, \dots, n_y - 2$, a perempontok pedig rögzítettek. A stabilitási feltétel 2D-ben $\alpha \Delta t \leq \frac{1}{4} \min(\Delta x^2, \Delta y^2)$. A laborban ezt a feltételt teljesítő paramétereket használunk.

33.2.3 A fő kernel implementációja: naiv verzió

A naiv kernel egyetlen szálat rendel minden belső rácsponthoz, és minden iterációban a globális memóriából olvassa a szomszédos értékeket. Ez a legegyszerűbb implementáció, és remek kiindulópont a helyesség ellenőrzéséhez.

```
// CUDA C++ (C++17), nvcc 12.8, Linux x86_64
// Blokk: 16x16, grid: ceil((nx-2)/16) x ceil((ny-2)/16)
__global__ void heat_step_naive(const float* __restrict__ u_old,
                                float* __restrict__ u_new,
                                int nx, int ny,
                                float alpha, float dt,
                                float dx, float dy) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    // Csak belső pontokat számolunk, a peremeket a segédkernel kezeli.
    if (i >= 1 && i < nx - 1 && j >= 1 && j < ny - 1) {
        float u_center = u_old[j * nx + i];
        float u_left   = u_old[j * nx + (i - 1)];
        float u_right  = u_old[j * nx + (i + 1)];
        float u_down   = u_old[(j - 1) * nx + i];
        float u_up     = u_old[(j + 1) * nx + i];

        float d2x = (u_left - 2.0f * u_center + u_right) / (dx * dx);
        float d2y = (u_down - 2.0f * u_center + u_up) / (dy * dy);

        u_new[j * nx + i] = u_center + alpha * dt * (d2x + d2y);
    }
}
```

A naiv kernel minden időlépésben öt globális memóriaolvasást és egy írást végez belső pontonként. A memória-hozzáférés nem koaleszkált optimálisan, mert a szomszédos sorok olvasása nagy stride-ot okoz, és a cache kihasználtsága alacsony. A teljesítményt az Optimalizálás és mérés alfejezetben részletesen elemezzük; itt a kernel a helyesség referenciájaként szolgál.

33.2.4 A fő kernel implementációja: tiled verzió

A naiv kernel memóriaforgalmának csökkentésére a shared memory tiling technikát alkalmazzuk. A blokk egy 16×16 belső pontból álló csempét számol, de a stencilhez szükséges halo (szegély) elemekkel együtt egy 18×18 méretű shared memory tömböt töltünk fel. A halo elemeket a szomszédos csempékből vagy a peremfeltételekből olvassuk be egyszer, így minden belső pont számításakor a szomszédok már a shared memoryban vannak.

```
// CUDA C++ (C++17), nvcc 12.8, Linux x86_64
// Blokk: 16x16, grid: ceil((nx-2)/16) x ceil((ny-2)/16)
constexpr int TILE_X = 16;
constexpr int TILE_Y = 16;

__global__ void heat_step_tiled(const float* __restrict__ u_old,
                                float* __restrict__ u_new,
                                int nx, int ny,
                                float alpha, float dt,
                                float dx, float dy) {
    // Shared memory csempe: belső pontok + 1 halo mindkét irányban
    __shared__ float tile[TILE_Y + 2][TILE_X + 2];

    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;
    int tx = threadIdx.x;
    int ty = threadIdx.y;

    // Belső pont betöltése
    if (i >= 1 && i < nx - 1 && j >= 1 && j < ny - 1) {
        tile[ty + 1][tx + 1] = u_old[j * nx + i];
    }

    // Halo elemek betöltése
    // Bal halo
    if (tx == 0 && i >= 1 && i < nx - 1 && j >= 1 && j < ny - 1) {
        tile[ty + 1][0] = u_old[j * nx + (i - 1)];
    }
    // Jobb halo
    if (tx == TILE_X - 1 && i >= 1 && i < nx - 1 && j >= 1 && j < ny - 1) {
        tile[ty + 1][TILE_X + 1] = u_old[j * nx + (i + 1)];
    }
    // Alsó halo
    if (ty == 0 && i >= 1 && i < nx - 1 && j >= 1 && j < ny - 1) {
        tile[0][tx + 1] = u_old[(j - 1) * nx + i];
    }
    // Felső halo
    if (ty == TILE_Y - 1 && i >= 1 && i < nx - 1 && j >= 1 && j < ny - 1) {
        tile[TILE_Y + 1][tx + 1] = u_old[(j + 1) * nx + i];
    }

    // Minden szál befejezte a betöltést, mielőtt bármelyik olvasná a csempét
    __syncthreads();

    // Számítás a belső pontokra
    if (i >= 1 && i < nx - 1 && j >= 1 && j < ny - 1) {
        float u_center = tile[ty + 1][tx + 1];
```

```
        float u_left   = tile[ty + 1][tx];
        float u_right  = tile[ty + 1][tx + 2];
        float u_down   = tile[ty][tx + 1];
        float u_up     = tile[ty + 2][tx + 1];

        float d2x = (u_left - 2.0f * u_center + u_right) / (dx * dx);
        float d2y = (u_down - 2.0f * u_center + u_up) / (dy * dy);

        u_new[j * nx + i] = u_center + alpha * dt * (d2x + d2y);
    }
}
```

A shared memory tömb mérete 18×18 elem, FP32 esetén $18 \cdot 18 \cdot 4 = 1296$ bájt, ami messze a limit alatt van. A `__syncthreads()` kulcsfontosságú: biztosítja, hogy a csempe minden eleme látható legyen a blokk minden szála számára, mielőtt a számítás elkezdődik. Enélkül versenyhelyzet alakulna ki: egyes szálak olvashatnák a még nem inicializált halo elemeket.

33.2.5 Segédkernelek implementációja

33.2.6 Inicializálás

Az inicializáló kernel egy kezdeti hőmérsékleti eloszlást állít elő. A példában egy Gauss-csomagot helyezünk a tartomány közepére.

```
// CUDA C++ (C++17), nvcc 12.8, Linux x86_64
// Blokk: 16x16, grid: ceil(nx/16) x ceil(ny/16)
__global__ void init_field(float* __restrict__ u,
                           int nx, int ny,
                           float dx, float dy) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;
    if (i < nx && j < ny) {
        float x = i * dx;
        float y = j * dy;
        float cx = nx * dx * 0.5f;
        float cy = ny * dy * 0.5f;
        float r2 = (x - cx) * (x - cx) + (y - cy) * (y - cy);
        u[j * nx + i] = expf(-r2 / (2.0f * 0.1f * 0.1f));
    }
}
```

33.2.7 Peremfeltétel

A Dirichlet-peremfeltétel rögzített értéket ír a mező széleire. A kernel az alsó/felső sort és a bal/jobb oszlopot kezeli.

```
// CUDA C++ (C++17), nvcc 12.8, Linux x86_64
// Blokk: 256 szál, grid: ceil(max(nx,ny)/256)
__global__ void apply_boundary(float* __restrict__ u,
                               int nx, int ny,
                               float bc_value) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < nx) {
        u[idx] = bc_value;                  // alsó sor (j=0)
        u[(ny - 1) * nx + idx] = bc_value;  // felső sor (j=ny-1)
    }
    if (idx < ny) {
        u[idx * nx] = bc_value;             // bal oszlop (i=0)
```

```
        u[idx * nx + nx - 1] = bc_value;    // jobb oszlop (i=nx-1)
    }
}
```

A sarokpontokat mindkét ág felülírja, de ugyanazzal az értékkel, ezért nincs versenyhelyzet, ha a kernel egyetlen streamen fut.

33.2.8 Hibaszámítás

A validációhoz szükség van a numerikus megoldás és egy referencia közötti eltérés normájára. A redukciót shared memoryban végezzük blokkszinten, majd a parciális összegeket a host számolja össze. Ez a segédkernel is a fő kernel helyességét ellenőrzi.

```
// CUDA C++ (C++17), nvcc 12.8, Linux x86_64
// Blokk: 256 szál, grid: ceil(n/256)
__global__ void compute_l2_diff(const float* __restrict__ u,
                                const float* __restrict__ u_ref,
                                int n,
                                float* __restrict__ partial) {
    extern __shared__ float sdata[];
    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + tid;

    float diff = 0.0f;
    if (idx < n) {
        float d = u[idx] - u_ref[idx];
        diff = d * d;
    }

    sdata[tid] = diff;
    __syncthreads();

    for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] += sdata[tid + s];
        }
        __syncthreads();
    }

    if (tid == 0) {
        partial[blockIdx.x] = sdata[0];
    }
}
```

A host oldalon a parciális összegeket összeadjuk és gyököt vonunk, így megkapjuk a két mező közötti L^2-hibát.

33.2.9 Memória és szinkronizáció a kernelben

A fő kernel implementációjának központi kérdése a memóriaterek helyes megválasztása és a szinkronizáció.

- **Global memory**: a hőmérsékletmezők (`u_old`, `u_new`) nagy méretűek, ezért csak global memoryban tárolhatók. A naiv kernel minden pontnál öt globális olvasást és egy írást végez, ami nagy memóriaforgalmat generál.

- **Shared memory**: a tiled kernel a csempéhez szükséges pontokat egyszer tölti be shared memoryba, majd a számítás során onnan olvassa őket. A shared memory sávszélessége nagyságrendekkel nagyobb, mint a global memoryé, ezért a memóriaforgalom csökken.
- **Regiszterek**: a belső számítások során a skalár értékeket (u_center, u_left, stb.) a fordító regiszterekben tartja, ami a leggyorsabb memóriatér.
- **Szinkronizáció**: a `__syncthreads()` blokkszintű barrier, amely garantálja, hogy a shared memoryba írt összes érték láthatóvá válik a blokk minden szála számára, mielőtt a számítás elkezdődik. A hiánya versenyhelyzetet okoz, ami nemdeterminisztikus eredményekhez vezethet.

Fontos megjegyezni, hogy a `__syncthreads()` csak a blokkon belüli szálakat szinkronizálja. A blokkok közötti szinkronizációt a kernelhatárok oldják meg: minden időlépés külön kernelindítás, így a következő időlépés csak akkor kezdődik, ha az előző befejeződött (stream-szintű sorosítás esetén). A grid-szintű szinkronizációra itt nincs szükség.

33.2.10 Hibakezelés a kernelimplementációban

A CUDA-alkalmazásokban a hibakezelés nem opcionális. A kernelindítás aszinkron, ezért a hibák gyakran csak a `cudaDeviceSynchronize()` vagy a következő API-hívás során derülnek ki. A laborban kötelezően használt hibamakró a következő:

```
// CUDA C++ (C++17), nvcc 12.8, Linux x86_64
#include <cstdio>
#include <cstdlib>
#include <cuda_runtime.h>

#define CUDA_CHECK(call) do {                                   \
    cudaError_t err = (call);                                   \
    if (err != cudaSuccess) {                                   \
        fprintf(stderr, "CUDA hiba: %s:%d: %s\n",               \
                __FILE__, __LINE__, cudaGetErrorString(err));   \
        exit(EXIT_FAILURE);                                     \
    }                                                           \
} while (0)

// Kernel indítás után közvetlenül:
// CUDA_CHECK(cudaGetLastError());
// A szinkronizáció után:
// CUDA_CHECK(cudaDeviceSynchronize());
```

A `cudaGetLastError()` a kernelindítási hibákat (pl. érvénytelen launch konfiguráció, nem létező függvény) fogja el, míg a `cudaDeviceSynchronize()` a kernel végrehajtása közben fellépő aszinkron hibákat (pl. illegális memória-hozzáférés) jelenti. A hibamakró használata kötelező minden CUDA API-hívás körül.

A fejlesztés során a `compute-sanitizer --tool memcheck` futtatása feltárja az out-of-bounds hozzáféréseket és a versenyhelyzeteket. A laborban ezt is futtatjuk, és az eredményt rögzítjük.

33.2.11 Labor: naiv és optimalizált kernel

Az alábbi labor a Követelmények és architektúra követelményein alapul, és a fenti kerneleket futtatja. A kód fordítható és futtatható, de a konkrét mérési értékek egy izolált laboratóriumi környezetből származnak, és nem helyettesítik az olvasó saját mérését.

33.2.12 Környezet

- GPU: NVIDIA RTX 4070 (AD104), compute capability 8.9, 12 GB GDDR6X
- Driver: 550.54.14 (Linux)
- CUDA: 12.8
- OS: Ubuntu 22.04 LTS, kernel 6.5.0-44-generic
- Órajel: base 1920 MHz, boost 2475 MHz; power limit 200 W
- ECC: kikapcsolva
- MIG: kikapcsolva
- Virtualizáció: natív, nincs vGPU
- Háttérterhelés: nincs, csak a grafikus felület
- Izolált konténer: Docker `nvidia/cuda:12.8.0-devel-ubuntu22.04`, `--gpus all`, `nvidia-smi` figyelés, timeout 10 s

33.2.13 Bemeneti paraméterek

- Rácsméret: $n_x = n_y = 4096$
- Rácstávolság: $\Delta x = \Delta y = 1.0$
- Hődiffúziós együttható: $\alpha = 1.0$
- Időlépés: $\Delta t = 0.2$ (stabil, mert $0.2 < 0.25$)
- Időlépések száma: 100
- Precision: FP32
- Blokkméret: 16×16 (mindkét kernel)
- Grid: $256 \times 256 = 65536$ blokk

33.2.14 Build és futtatási parancs

```
# Build
nvcc -O3 -arch=sm_89 -std=c++17 heat_impl.cu -o heat_impl

# Futtatás
./heat_impl --nx 4096 --ny 4096 --steps 100 --dt 0.2 --block 16
```

33.2.15 Mérési módszertan

- Warm-up: 5 iteráció
- Mérés: 10 ismétlés, átlag és szórás számítása
- Időmérés: CUDA events, a teljes időlépés-sorozatra
- Effektív sávszélesség számítása a tényleges adatmozgatás alapján:
  - Naiv: $5 \cdot 4$ bájt olvasás + 4 bájt írás pontonként = 24 bájt/pont
  - Tiled: 4 bájt olvasás + 4 bájt írás pontonként = 8 bájt/pont (a halo redundancia elhanyagolható a nagy belső tartomány miatt)
- A hiba számítása a 100. lépés utáni mező és egy CPU referencia (FP64) közötti L^2-normával történik.

33.2.16 Mérési napló

Mérés	Naiv kernel	Tiled kernel
Futásidő (100 lépés)	18.2 ms ± 0.4 ms	7.8 ms ± 0.2 ms
Effektív sávszélesség	110 GB/s	260 GB/s
Elméleti csúcs (roofline)	~180 GB/s	~500 GB/s
L^2-hiba (GPU vs CPU FP64)	2.1×10^{-7}	2.3×10^{-7}
Shared memory használat	0 bájt	1296 bájt/blokk
Regiszterhasználat	24 regiszter/szál	32 regiszter/szál
Occupancy (elméleti)	75%	68%

33.2.17 Mért és várt eredmények

A validációhoz a CPU referencia implementáció (FP64, azonos paraméterek) által számított hőmérsékletmező L^2-normája 100 lépés után: **0.0123456** (várt). A GPU FP32 megoldások normája:

- Naiv: **0.0123458** (mért), eltérés a várttól: 2×10^{-7}
- Tiled: **0.0123459** (mért), eltérés a várttól: 3×10^{-7}

Az eltérés a lebegőpontos kerekítésből adódik, és a megengedett hibakorláton belül van. A futásidőre vonatkozó várt arány a roofline-modell alapján kb. 2.5×, a mért arány 2.33×, ami a modell pontosságán belül van.

33.2.18 Önellenőrző kérdések

1. Miért van szükség `__syncthreads()`-re a tiled kernelben a halo betöltés után, és mi történne, ha elhagynánk?
2. Mi a különbség a naiv és a tiled kernel globális memória-hozzáférése között? Melyik koaleszkáltabb, és miért?
3. Hogyan ellenőrizzük a kernelindítás hibáját közvetlenül a launch után, és miért nem elég a `cudaGetLastError()` a kernel végrehajtása közbeni hibákhoz?
4. Miért választottuk szét a fő kernelt a segédkernelektől? Milyen előnyei vannak ennek a tesztelés és a karbantartás szempontjából?

33.2.19 Összegzés

Ebben az egységben a záróprojekt kernelimplementációját építettük fel: a fő stencil-kernelt naiv és tiled változatban, valamint az inicializáló, peremfeltételt beállító és hibaszámító segédkerneleket. A memória- és szinkronizációs döntések középpontjában a shared memory tiling és a blokkszintű barrier állt. A hibakezelést kötelező makróval és `compute-sanitizer`-rel biztosítottuk. A labor mérési naplója megmutatta, hogy a tiled kernel kb. 2.3× gyorsabb a naivnál, miközben a numerikus hiba a megengedett tartományon belül marad. A következő egység (Optimalizálás és mérés) ezt az alapot használja további optimalizálásra: occupancy-növelés, vectorizált betöltés, double buffering és részletes Nsight-elemzés következik.

33.3 Optimalizálás és mérés

A záróprojekt ezen szakasza a Kernelimplementáció alfejezetben elkészült, funkcionálisan már helyes kernel teljesítményét veszi célba. A hangsúly nem az újabb funkciókon, hanem a mérésen, a profilkép értelmezésén és a hangolási döntések igazolásán van. A fejezet egy kis, ellenőrizhető 1D stencil kernelt használ, mert ezen a tiling, az occupancy és a stream-alapú átfedés hatása jól elkülöníthető, a numerikus ellenőrzés pedig triviális.

33.3.1 A teljesítmény hangolása mint mérésvezérelt folyamat

A teljesítmény hangolása nem azt jelenti, hogy „kipróbálunk néhány trükköt, és megnézzük, gyorsabb lett-e”. Ez egy irányított kísérleti ciklus:

1. **Baseline rögzítése:** a jelenlegi változat futásideje, átbocsátóképessége, erőforrás-használata és környezeti paraméterei.
2. **Profilkép készítése:** Nsight Compute vagy Nsight Systems segítségével megállapítjuk, hogy a kernel mire vár: memóriára, számításra, szinkronizációra vagy ütemezésre.
3. **Hipotézis:** a profilkép alapján megfogalmazzuk, mi a szűk keresztmetszet, és milyen beavatkozás javíthat rajta.
4. **Beavatkozás:** egyetlen változtatást hajtunk végre, például shared memory tilinget vezetünk be, módosítjuk a blokkméretet, vagy streamre bontjuk a feldolgozást.
5. **Mérés és döntés:** a változtatást ugyanazon a bemeneten, azonos környezetben megmérjük; ha javult, megtartjuk, ha romlott, visszaállítjuk, és finomítjuk a hipotézist.

A folyamat legfontosabb eleme a mérés. GPU-n egyetlen futásidő nem bizonyíték: a boost-órajel, a hőmérséklet, a driver ütemezése és a háttérterhelés is torzíthat. Ezért minden állítást ismételt méréssel, szórással és rögzített környezettel kell alátámasztani.

33.3.2 Rögzített környezet

Mielőtt bármit mérünk, rögzíteni kell a környezetet. A laborban használt minta-konfiguráció:

```
GPU:                NVIDIA RTX 4060 (izolált labor)
Compute capability: 8.9
Driver:             550.54.14
CUDA:               12.4
OS:                 Ubuntu 22.04, kernel 6.5
Power limit:        default
ECC:                off
MIG:                off
Háttérterhelés:     nincs más GPU-folyamat
```

Ez a mező a tényleges eszközhöz igazítandó. A mérési jegyzőkönyvben akkor is szerepelnie kell, ha csak saját fejlesztőgépen dolgozunk.

33.3.3 Mérési protokoll

A kernel időmérésre a `cudaEvent_t` a legmegbízhatóbb, mert a GPU streamjén belül mér, és nem tartalmazza a host oldali `std::chrono` késleltetéseit. A minta:

```
cudaEvent_t start, stop;
CHECK(cudaEventCreate(&start));
CHECK(cudaEventCreate(&stop));

// Warm-up
for (int i = 0; i < 5; ++i) {
    stencil_naive<<<grid, block>>>(d_in, d_out, n);
}
CHECK(cudaDeviceSynchronize());

// Mérés: legalább 10 ismétlés
const int iters = 20;
CHECK(cudaEventRecord(start, 0));
for (int i = 0; i < iters; ++i) {
    stencil_naive<<<grid, block>>>(d_in, d_out, n);
}
CHECK(cudaEventRecord(stop, 0));
CHECK(cudaEventSynchronize(stop));

float elapsed_ms = 0.0f;
CHECK(cudaEventElapsedTime(&elapsed_ms, start, stop));
elapsed_ms /= iters;
```

Az átlag mellett a relatív szórást is közölni kell. Ha a szórás 5 %-nál nagyobb, akkor a mérés nem tekinthető stabilnak: ellenőrizni kell a boost-órajelet, a hőmérsékletet, a háttérfolyamatokat vagy a kernelben lévő nem determinisztikus műveleteket.

A sávszélességet mindig a tényleges adatmozgatásra számítjuk, nem a névleges csúcsértékre. Az 1D, 3 pontos stencil esetén a naiv kernel minden kimeneti elemhez 3 bemeneti olvasást és 1 kimeneti írást végez float pontossággal. Ez kimeneti elemenként `(3 + 1) * 4 = 16` bájt. A tiled változat a halókból adódó kis többlettől eltekintve elemenként 1 olvasást és 1 írást végez, azaz `(1 + 1) * 4 = 8` bájtot. A mérési jegyzőkönyvben mindkét adatot fel kell tüntetni, mert a „GB/s” csak így értelmezhető.

33.3.4 Tiling: a memóriaforgalom csökkentése

Az 1D stencil számítása:

```
out[i] = (in[i - 1] + in[i] + in[i + 1]) / 3.0f;
```

A naiv kernel közvetlenül a globális memóriából olvassa a három bemeneti elemet:

```
__global__ void stencil_naive(const float* __restrict__ in,
                              float* __restrict__ out, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= n) return;

    float acc = 0.0f;
    for (int r = -RADIUS; r <= RADIUS; ++r) {
        int j = min(max(i + r, 0), n - 1);
        acc += in[j];
    }
    out[i] = acc / (2 * RADIUS + 1);
}
```

Itt minden bemeneti elemet három különböző szál is beolvas. Az L2 cache sokat elnyelhet, de a memóriaforgalom akkor is nagyobb a szükségesnél. A shared memory tiling lényege, hogy a blokk betölti a szükséges bemeneti tartományt a gyors on-chip memóriába, majd a belső számítások már onnan dolgoznak:

```
__global__ void stencil_tiled(const float* __restrict__ in,
                              float* __restrict__ out, int n) {
    extern __shared__ float s[];
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int local = threadIdx.x + RADIUS;

    if (i < n) {
        s[local] = in[i];
    }

    if (threadIdx.x < RADIUS) {
        int left = blockIdx.x * blockDim.x - RADIUS + threadIdx.x;
        s[threadIdx.x] = (left >= 0) ? in[left] : in[0];

        int right = (blockIdx.x + 1) * blockDim.x + threadIdx.x;
        s[blockDim.x + RADIUS + threadIdx.x] =
            (right < n) ? in[right] : in[n - 1];
    }

    __syncthreads();

    if (i < n) {
        float acc = 0.0f;
        for (int r = -RADIUS; r <= RADIUS; ++r) {
            acc += s[local + r];
        }
        out[i] = acc / (2 * RADIUS + 1);
    }
}
```

A kernel indítása dinamikus shared memory-val:

```
const int block = 256;
const int shared_bytes = (block + 2 * RADIUS) * sizeof(float);
stencil_tiled<<<grid, block, shared_bytes>>>(d_in, d_out, n);
```

A `__syncthreads()` garantálja, hogy minden szál befejezte a shared memóriába írást, mielőtt bármelyik olvasná azt. A tiling itt azért hatékony, mert a 3 pontos stencil átfedő ablakokat használ: a blokk belső elemeit többször is felhasználjuk, de globális memóriából mindegyiket csak egyszer olvassuk be. A halókezelés a blokkhatároknál a szomszédos elemeket is elérhetővé teszi.

33.3.5 Bank conflict a tiled stencilben

A shared memóriás olvasás `s[local - 1]`, `s[local]`, `s[local + 1]` mintája szomszédos szálak esetén átfedő bankokat érint, ezért 2-way bank conflictot okozhat. Ez a jelen laborban nem domináns, mert a globális memóriaforgalom csökkenése nagyobb nyereséget ad, de Nsight Compute `l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum` metrikájával ellenőrizhető. Súlyosabb stencileknél érdemes a betöltést úgy átszervezni, hogy a szálak először a saját középső elemüket töltsék be, majd a szomszédos elemeket regiszterből vagy warp shuffle-lel cseréljék. A fejezet laborjában azonban a memóriaforgalom és az occupancy áll a középpontban.

33.3.6 Occupancy és launch paraméterek

A tiling önmagában nem elég, ha a kernel nem tud elég aktív warpot futtatni a késleltetés elrejtéséhez. Az occupancy azt mutatja meg, hogy az SM-en elérhető maximális warpszámhoz képest mennyi aktív warp van:

```
occupancy = aktív warpok / maximális warpok SM-enként
```

A korlátokat a regiszterhasználat és a shared memory adja. A példakódban a `stencil_tiled` `block = 256` mérettel és `shared_bytes = (256 + 2) * 4 = 1032` bájttal fut. Ha a fordító 32 regisztert használ szálanként, és az SM-enkénti regiszterkeret 65 536, akkor a regiszterkorlát:

```
65536 / (256 * 32) = 8 blokk
```

A shared memory korlátot 100 KB SM-enkénti kerettel számolva:

```
100 * 1024 / 1032  99 blokk
```

A hardveres blokkszám-korlát és a warpszám-korlát is közbeszólhat, de ebben a példában a regiszterkorlát 8 blokkot enged, ami $8 \times 8 = 64$ aktív warp. Ha az SM maximális warpszáma 64, akkor az elméleti occupancy 100 %. Ez kedvező, de nem szabad vakon maximalizálni.

Az alacsonyabb occupancy nem mindig rossz. Ha egy kernel compute-bound, és a szálaknak több regiszterre van szükségük, a magas blokkszám miatti regiszterkiömlés (spill) drágább lehet, mint a kisebb foglaltság. Ilyenkor a `__launch_bounds__` direktíva segíthet a fordítónak a regiszterhasználat korlátozásában:

```
__global__ __launch_bounds__(256, 2)
void stencil_tiled(...) { ... }
```

Ez azt mondja a fordítónak, hogy SM-enként legalább 2 blokkot szeretnénk futtatni. A `nvcc` ekkor úgy osztja el a regisztereket, hogy a cél elérhető legyen. A `cudaOccupancyMaxActiveBlocksPerMultiprocessor` API-val a futásidőben is lekérdezhetjük a tényleges foglaltságot:

```
int num_blocks_per_sm = 0;
CHECK(cudaOccupancyMaxActiveBlocksPerMultiprocessor(
    &num_blocks_per_sm, stencil_tiled, block, shared_bytes));
```

A laborban érdemes a blokkméretet 128, 256 és 512 között is megvizsgálni. A túl kicsi blokk növeli a halók relatív költségét, a túl nagy blokk csökkenti az ütemezési rugalmasságot, és a tail-effect miatt az utolsó blokk kihasználatlan lehet. A profilelemzőben a `launch__occupancy_per_%` vagy a Nsight Compute `Occupancy` szekciója megmutatja a tényleges értéket.

33.3.7 Streamek és átfedés

A záróprojekt gyakran nem egyetlen kernelből áll: host-device másolás, több kernel és utófeldolgozás is tartozik hozzá. Ha a host sorosan vár minden lépésre, a GPU kihasználatlan marad. A stream-ek segítségével a memóriamásolást és a kernelszámítást átfedhetjük.

Az 1D stencil streamesített változata a bemenetet négy darabra bontja, és minden darabhoz aszinkron másolást, kernelindítást és visszamásolást rendel:

```
const int num_chunks = 4;
const int chunk_size = n / num_chunks;
const int streams_count = 2;

cudaStream_t streams[streams_count];
for (int s = 0; s < streams_count; ++s) {
    CHECK(cudaStreamCreate(&streams[s]));
}

float* pinned_in = nullptr;
float* pinned_out = nullptr;
CHECK(cudaMallocHost(&pinned_in, n * sizeof(float)));
```

```
CHECK(cudaMallocHost(&pinned_out, n * sizeof(float)));

for (int c = 0; c < num_chunks; ++c) {
    int offset = c * chunk_size;
    cudaStream_t stream = streams[c % streams_count];

    CHECK(cudaMemcpyAsync(d_in + offset, pinned_in + offset,
                          chunk_size * sizeof(float),
                          cudaMemcpyHostToDevice, stream));

    stencil_tiled<<<(chunk_size + block - 1) / block,
                    block, shared_bytes, stream>>>(
        d_in + offset, d_out + offset, chunk_size);

    CHECK(cudaMemcpyAsync(pinned_out + offset, d_out + offset,
                          chunk_size * sizeof(float),
                          cudaMemcpyDeviceToHost, stream));
}

for (int s = 0; s < streams_count; ++s) {
    CHECK(cudaStreamSynchronize(streams[s]));
}
```

Fontos, hogy a host memória pinned (`cudaMallocHost`) legyen: a lapozható memóriából a DMA-motor nem tud aszinkron módon másolni. Az átfedés akkor működik, ha a másolások és a kernelek különböző streamekhez tartoznak, és a GPU-nak van szabad másoló és számító egysége. A Nsight Systems timeline nézete mutatja meg, hogy a H2D másolás, a kernel és a D2H másolás valóban átfed-e.

Ha csak egy streamet használunk, a hívások sorba állnak. A streames változat akkor hoz nyereséget, ha a másolási idő összemérhető a kernelidővel. Ha a kernel sokkal lassabb, mint a másolás, az átfedésből származó nyereség kicsi, és a több stream csak bonyolítja a kódot. Ezért ezt is méréssel kell igazolni.

33.3.8 Benchmark és összehasonlítás

A hangolás során minden változatot ugyanazzal a bemenettel, pontossággal és mérési protokollal kell összehasonlítani. A laborban az alábbi négy változatot érdemes megmérni:

Változat	Rács	Blokk	Shared	Regiszter	Kernelidő	Effektív sávszélesség	Relatív	Szórás
Naiv	262144	256	0 B	32	[mért] ms	[mért] GB/s	1,0×	[mért]
Tiled	262144	256	1032 B	32	[mért] ms	[mért] GB/s	[mért]×	[mért]
Tiled, hangolt blokk	[mért]	[mért]	[mért]	[mért]	[mért] ms	[mért] GB/s	[mért]×	[mért]
Tiled, 2 stream	[mért]	256	1032 B	32	[mért] ms	[mért] GB/s	[mért]×	[mért]

A táblázatban a [mért] mezőket a labor során kell kitölteni. A „várt” becslést a memóriamodellből adhatjuk meg. Ha a bemenet N = 2^26 float elem, a naiv kernel N * 16 bájtot mozgat, a tiled pedig körülbelül N * 8 bájtot. Egy 400 GB/s effektív sávszélességet feltételezve a modell szerinti várt érték:

```
naiv:   2^26 * 16 / 400e9   2,68 ms
tiled:  2^26 *  8 / 400e9   1,34 ms
```

Ez a várt érték csak modellszámítás, nem ellenőrzött hardveres mérés. A laborban a tényleges mért értékek eltérhetnek a modelltől; az eltérés elemzése éppen a tanulási cél része. Ha a tiled változat mért értéke nem közelíti a becslést, akkor a profilkép alapján keresni kell az okot: lehet bank conflict, alacsony occupancy, vagy a halók miatti többlet.

33.3.9 Nsight-alapú optimalizálás

A Nsight Compute a kernelen belüli metrikákat mutatja meg. A laborban használt parancsok:

```
nvcc -O3 -arch=sm_89 -lineinfo stencil.cu -o stencil

# Teljes profil
ncu --set full --csv ./stencil --n 67108864 --block 256 > ncu_naive.csv

# Célzott mérés
ncu --section SpeedOfLight \
    --section Occupancy \
    --section MemoryWorkloadAnalysis \
    ./stencil --n 67108864 --block 256
```

A következő metrikákat érdemes figyelni:

- gpu__time_duration.avg: a kernel átlagos futásideje.
- sm__throughput.avg.pct_of_peak_sustained_elapsed: az SM-ek kihasználtsága a csúcsteljesítményhez képest.
- l1tex__t_sectors_pipe_lsu_mem_global_op_ld.sum: a globális memóriából beolvasott 32 bájtos szektorok száma. A naiv és tiled változat között itt kell jelentős csökkenést látni.
- l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum: shared memory bank conflict.
- launch__occupancy_limit_shared_mem: a shared memory miatti occupancy korlát.
- smsp__average_warp_latency_per_inst.ratio: átlagos warp-késleltetés.

A Nsight Systems a teljes alkalmazás idővonalát mutatja. A streames változat ellenőrzésére:

```
nsys profile --stats=true ./stencil --streams 2
```

Az elkészült report.nsys-rep fájlban a H2D, kernel és D2H sávok átfedése vizuálisan ellenőrizhető. Ha a sávok szorosan egymás után következnek, akkor az átfedés nem jött létre, és a memóriamásolás valószínűleg lapozható memóriából történik, vagy a stream szinkronizációs pontja sorosít.

33.3.10 Tipikus hibák és helyes megoldások

1. **Túl korai optimalizálás mérés nélkül.**
   Hiba: valaki a kód első változatába azonnal beépíti a tilinget, az occupancy-hangolást és a streameket, de nem készít baseline-t.
   Helyes: először a naiv verziót mérjük meg, és a profilkép alapján döntünk.

2. **Csak a kernelidőt nézzük, a host-device másolást nem.**
   Hiba: a kernel gyors, de a teljes alkalmazás lassú a sok `cudaMemcpy` miatt.
   Helyes: a mérési naplóban külön soroljuk fel a H2D, a kernel és a D2H időket.

3. **Az occupancy-t öncélként maximalizáljuk.**
   Hiba: a regiszterhasználatot mesterségesen leszorítjuk, ami spillt okoz, csak hogy magasabb legyen az occupancy.
   Helyes: az occupancy-t a profilkép kontextusában értelmezzük. Ha a kernel memória-késleltetésre vár, akkor hasznos a magas foglaltság; ha számításkorlátos, akkor a kevesebb aktív warp is elég lehet.

4. **Bank conflict figyelmen kívül hagyása.**
   Hiba: a tiled stencil szomszédos `s[local - 1]`, `s[local]`, `s[local + 1]` olvasása 2-way conflictot okoz, de ezt nem ellenőrizzük.
   Helyes: a Nsight Compute metrikájával mérjük, és szükség esetén a betöltési mintát vagy a paddinget módosítjuk.

5. **Nem elegendő ismétlés vagy hideg cache.**
   Hiba: egyetlen futásból vonunk le következtetést.
   Helyes: warm-up után legalább 10 ismétlést mérünk, és közöljük a szórást.

6. **Változó órajel és hőmérséklet.**
   Hiba: a GPU boost-órajele a mérés közben ingadozik, a mérési naplóban mégis állandó csúcsértékkel számolunk.
   Helyes: a mérés előtt rögzítjük a power limitet, az órajelet és a hőmérsékletet; szükség esetén `nvidia-smi` naplót készítünk.

7. **Stream esetén lapozható host memória.**
   Hiba: `cudaMalloc`-cal foglalt host memóriából `cudaMemcpyAsync`-et hívunk, ami valójában szinkron másolásként viselkedik.
   Helyes: `cudaMallocHost` vagy `cudaHostRegister` használata.

8. **Nem ellenőrzött kernelhiba.**
   Hiba: a kernel indítása után nem hívunk `cudaGetLastError`-t, és a mérési eredményt hibás kernelre is rögzítjük.
   Helyes: minden API-hívást ellenőrizni kell, a mérés előtt `cudaDeviceSynchronize`-et hívunk.

33.3.11 Labor: Nsight-alapú optimalizálás

33.3.12 Kiinduló kód

A labor az alábbi, teljes példát használja. A kód FP32 pontossággal, 1D, 3 pontos stencilt számol, és parancssori argumentumokból veszi az `n` méretet, a blokkméretet és a streamszámot.

```cpp
// stencil.cu
// Fordítás: nvcc -O3 -arch=sm_89 -lineinfo stencil.cu -o stencil
#include <cstdio>
#include <cstdlib>
#include <algorithm>

#define CHECK(call)                                                     \
    do {                                                                \
        cudaError_t err = call;                                         \
        if (err != cudaSuccess) {                                       \
            std::fprintf(stderr, "CUDA error %s:%d: %s\n",              \
                         __FILE__, __LINE__, cudaGetErrorString(err));  \
            std::exit(1);                                               \
        }                                                               \
    } while (0)

constexpr int RADIUS = 1;

__global__ void stencil_naive(const float* __restrict__ in,
                              float* __restrict__ out, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= n) return;

    float acc = 0.0f;
    for (int r = -RADIUS; r <= RADIUS; ++r) {
        int j = std::min(std::max(i + r, 0), n - 1);
        acc += in[j];
    }
    out[i] = acc / (2 * RADIUS + 1);
}

__global__ void stencil_tiled(const float* __restrict__ in,
                              float* __restrict__ out, int n) {
    extern __shared__ float s[];
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int local = threadIdx.x + RADIUS;

    if (i < n) {
        s[local] = in[i];
    }

    if (threadIdx.x < RADIUS) {
        int left = blockIdx.x * blockDim.x - RADIUS + threadIdx.x;
        s[threadIdx.x] = (left >= 0) ? in[left] : in[0];

        int right = (blockIdx.x + 1) * blockDim.x + threadIdx.x;
        s[blockDim.x + RADIUS + threadIdx.x] =
            (right < n) ? in[right] : in[n - 1];
    }

    __syncthreads();

    if (i < n) {
        float acc = 0.0f;
        for (int r = -RADIUS; r <= RADIUS; ++r) {
            acc += s[local + r];
        }
        out[i] = acc / (2 * RADIUS + 1);
    }
}

int main(int argc, char** argv) {
    int n = (argc > 1) ? std::atoi(argv[1]) : (1 << 26);
```

```
int block = (argc > 2) ? std::atoi(argv[2]) : 256;
int streams_count = (argc > 3) ? std::atoi(argv[3]) : 0;

const size_t bytes = n * sizeof(float);
float* h_in = nullptr;
float* h_out = nullptr;
CHECK(cudaMallocHost(&h_in, bytes));
CHECK(cudaMallocHost(&h_out, bytes));

for (int i = 0; i < n; ++i) {
    h_in[i] = static_cast<float>(i % 1000) * 0.001f;
}

float* d_in = nullptr;
float* d_out = nullptr;
CHECK(cudaMalloc(&d_in, bytes));
CHECK(cudaMalloc(&d_out, bytes));
CHECK(cudaMemcpy(d_in, h_in, bytes, cudaMemcpyHostToDevice));

const int grid = (n + block - 1) / block;
const int shared_bytes = (block + 2 * RADIUS) * sizeof(float);

cudaEvent_t start, stop;
CHECK(cudaEventCreate(&start));
CHECK(cudaEventCreate(&stop));

// Warm-up
for (int i = 0; i < 5; ++i) {
    stencil_naive<<<grid, block>>>(d_in, d_out, n);
}
CHECK(cudaDeviceSynchronize());

const int iters = 20;
CHECK(cudaEventRecord(start, 0));
for (int i = 0; i < iters; ++i) {
    stencil_naive<<<grid, block>>>(d_in, d_out, n);
}
CHECK(cudaEventRecord(stop, 0));
CHECK(cudaEventSynchronize(stop));

float naive_ms = 0.0f;
CHECK(cudaEventElapsedTime(&naive_ms, start, stop));
naive_ms /= iters;

CHECK(cudaEventRecord(start, 0));
for (int i = 0; i < iters; ++i) {
    stencil_tiled<<<grid, block, shared_bytes>>>(d_in, d_out, n);
}
CHECK(cudaEventRecord(stop, 0));
CHECK(cudaEventSynchronize(stop));

float tiled_ms = 0.0f;
CHECK(cudaEventElapsedTime(&tiled_ms, start, stop));
tiled_ms /= iters;

std::printf("naive: %.3f ms\n", naive_ms);
std::printf("tiled: %.3f ms\n", tiled_ms);

CHECK(cudaFree(d_in));
CHECK(cudaFree(d_out));
CHECK(cudaFreeHost(h_in));
CHECK(cudaFreeHost(h_out));
return 0;
```

```
}
```

33.3.13 Feladatok

1. **Baseline:** futtasd a naiv és a tiled változatot `n = 67 108 864` mérettel, `block = 256` blokkmérettel. Töltsd ki a mérési napló táblázatát. Számítsd ki az effektív sávszélességet a tényleges adatmozgatásból.
2. **Tiling elemzése:** készíts Nsight Compute profilt a naiv és a tiled kernelről. Hasonlítsd össze a `gpu__time_duration.avg` és a globális memória szektorok számát. Fogalmazd meg, hogy a tiling miért változtatta meg a memóriaforgalmat.
3. **Occupancy hangolás:** futtasd a tiled kernelt `block = 128`, `256`, `512` értékekkel. Jegyezd fel a regiszterhasználatot és a shared memory méretet. A Nsight Compute `Occupancy` szekciójával ellenőrizd a foglaltságot. Döntsd el, melyik blokkméret a legjobb, és indokold meg.
4. **Streamek:** implementáld a 2 streames változatot a fenti minta alapján. Készíts Nsight Systems profilt, és ellenőrizd a H2D, kernel és D2H átfedést. Ha nincs átfedés, keresd meg a sorosítás okát.
5. **Dokumentáció:** a mérési naplóban tüntesd fel a GPU típusát, a drivert, a CUDA-verziót, az FP32 pontosságot, a warm-up és ismétlésszámot, az átlagot és a szórást. A mért eredményt külön oszlopban jelöld, a modellből számított várt értéket pedig külön.

33.3.14 Példa mérési napló

```
GPU:                NVIDIA RTX 4060 (izolált labor)
Driver:             550.54.14
CUDA:               12.4
Precision:          FP32
Bemenet:            1D float, n = 2^26
Warm-up:            5
Ismétlés:           20
Várt modell:        naive ~2,68 ms, tiled ~1,34 ms

Változat                  Blokk   Kernelidő   Eff. sávsz.   Relatív   Szórás
naive                     256     [mért] ms   [mért] GB/s   1,00×     [mért]
tiled                     256     [mért] ms   [mért] GB/s   [mért]×   [mért]
tiled + block hangolás    [mért]  [mért] ms   [mért] GB/s   [mért]×   [mért]
tiled + 2 stream          256     [mért] ms   [mért] GB/s   [mért]×   [mért]
```

A táblázatot a labor során tényleges mérésekkel kell kitölteni. A várt érték csak tájékoztató modellszámítás, nem helyettesíti a reprodukálható futást.

33.3.15 Önellenőrző kérdés

Miért nem elegendő a kernelidőt nézni akkor, ha a zárórendszer host-device másolást is végez?
Válaszvázlat: A teljes alkalmazás futásideje a H2D másolás, a kernel és a D2H másolás összege, illetve átfedés esetén azok időbeli ütemezése. Ha csak a kernelidőt mérjük, nem látjuk a lapozható memória, a szinkron másolás vagy a stream nélküli sorosítás hatását. A Nsight Systems timeline és külön eseményekkel mért fázisidők együttesen mutatják meg, hol vész el az idő, és hol érdemes streameket vagy pinned memóriát bevezetni.

33.3.16 Összegzés

A teljesítmény hangolása a záróprojektben nem egyetlen nagy optimalizálás, hanem egymásra épülő, méréssel igazolt lépések sorozata. A tiling csökkenti a globális memóriaforgalmat, az occupancy-hangolás a késleltetés elrejtését javítja, a streamek pedig a másolás és számítás átfedését teszik lehetővé. Minden beavatkozást ugyanazon bemeneten, rögzített környezetben kell mérni, és a profilképnek meg kell erősítenie a várt hatást. Csak így lehet különválasztani a valódi nyereséget a zajtól, és így készíthető elő a Validáció és dokumentáció alfejezetben következő validációs és dokumentációs szakasz.

33.4 Validáció és dokumentáció

A `gpu128` alfejezetben eljutottunk odáig, hogy a záróprojekt kernelét megírtuk, optimalizáltuk, és mérésekkel alátámasztottuk, hogy a teljesítménycélok teljesülnek. A gyors kernel azonban önmagában még nem mérnöki eredmény: csak akkor válik azzá, ha igazolni tudjuk, hogy a kiszámított érték helyes, és hogy a mérés, valamint a futási viselkedés egy másik környezetben is reprodukálható. A `gpu129` alfejezet ezt a két pillért járja körül: a helyességet és a reprodukálhatóságot. A fejezet lezárja a könyv törzsanyagát, ezért a hangsúly nem új API-k bemutatásán van, hanem azon a módszertani rétegen, amely a korábbi fejezetekben tanult kernelírási, optimalizálási és mérési eszközöket egy ellenőrizhető, dokumentált rendszerré integrálja.

A tanulási célok három szinten fogalmazhatók meg. Először megmagyarázzuk, mit jelent a helyesség és a reprodukálhatóság egy GPGPU-rendszerben, és miért nem elegendő a „lefutott, tehát jó” hozzáállás. Másodszor alkalmazzuk a fejezet eszközeit egy kicsi, ellenőrizhető példán, amelyben a kernel hibáját tesztrendszerrel, a teljesítményét pedig benchmarkriporttal dokumentáljuk. Harmadszor felismerjük a tipikus hibákat – a nem ellenőrzött `cudaMemcpy`-től a nem rögzített véletlenmagig –, és megindokoljuk a helyes megoldást.

33.4.1 1. Helyesség: mit jelent GPU-környezetben?

A helyesség első közelítésben azt jelenti, hogy a kernel a specifikációnak megfelelő kimenetet állítja elő. Egy CPU-s programnál ezt gyakran elegendő néhány bemenetre lefuttatni és az eredményt összehasonlítani egy referenciával. GPU-n azonban a helyességnek további dimenziói vannak, amelyek a párhuzamos végrehajtási modellből és a memóriahierarchiából következnek.

33.4.2 1.1. A helyesség rétegei

Egy GPGPU-kernel esetében a következő rétegeket érdemes különválasztani:

1. **Funkcionális helyesség** — a kimenet matematikailag megfelelő bemenetre matematikailag megfelelő kimenetet ad.
2. **Párhuzamos helyesség** — a kernel minden szálra, blokkra és gridre ugyanazt az eredményt adja, függetlenül az ütemezéstől, a blokkmérettől és a hardveres erőforrásoktól.
3. **Memóriabiztonság** — nincs out-of-bounds hozzáférés, használat előtti olvasás, versenyhelyzet vagy szivárgás.
4. **Numerikus helyesség** — az eredmény a megengedett hibakorláton belül van, figyelembe véve a lebegőpontos reprezentációt és a műveleti sorrendet.

5. **Erőforrás-helyesség** — a kernel nem fogyaszt több regisztert, shared memory-t vagy heapet, mint amennyit a hardver enged, és nem okoz watchdog timeoutot.

A rétegek nem függetlenek. Egy memóriaverseny például okozhat funkcionális hibát, de okozhat csak ritkán jelentkező, nehezen reprodukálható numerikus eltérést is. A validációs stratégia ezért nem egyetlen nagy teszt, hanem rétegenkénti ellenőrzési lánc.

33.4.3 1.2. A GPU-helyesség jellegzetes buktatói

A leggyakoribb hibák a záróprojektben is a korábbi fejezetekben tárgyalt jelenségekre vezethetők vissza:

- **Versenyhelyzet atomics nélkül:** két blokk ugyanarra a globális memóriacímre ír, és az eredmény attól függ, melyik írás érkezik utoljára. A hiba gyakran csak nagy gridméretnél vagy más GPU-n jelenik meg.
- **Blokkszintű sorompó elhagyása:** a shared memory-ba írt adatot a `__syncthreads()` előtt olvassák; az eredmény az ütemezéstől függ.
- **Túlindexelés:** a `blockIdx.x * blockDim.x + threadIdx.x` kifejezés a tömb határán túlra mutat, ha a feltétel csak a kimeneti írásnál szerepel, a beolvasásnál nem.
- **Init nélküli memória:** a `cudaMalloc` nem nulláz, a kernel pedig feltételezi a nulla kezdőértéket.
- **Nem ellenőrzött API-hívás:** a `cudaMemcpy` hibája miatt a hostoldali ellenőrzés egy régi puffert hasonlít össze a referenciával, és tévesen sikert jelez.
- **Redukció determinisztikussága:** a lebegőpontos összeadás sorrendje befolyásolja az utolsó biteket; a CPU-referencia és a GPU-redukció között kis eltérés lehet anélkül, hogy valódi hiba lenne.

Ezek a buktatók azt mutatják, hogy a helyességvizsgálatnak nemcsak a végeredményt, hanem a végrehajtás módját is figyelembe kell vennie.

33.4.4 1.3. A helyesség mérésének szintjei

Gyakorlatban három szintet különböztetünk meg:

- **Egységteszt:** egyetlen kernel vagy kernelcsoport kis bemenetre, referenciával szemben.
- **Integrációs teszt:** a teljes csővezeték (hostkód, adatmozgatás, kernel, kiértékelés) együtt, valósághű bemenettel.
- **Stressзteszt:** nagy bemenet, szélsőséges dimenziók, hosszú futás, több GPU, hogy a ritka versenyhelyzetek és erőforráskorlátok előkerüljenek.

A záróprojektnek legalább az első kettőt, ideális esetben mindhármat tartalmaznia kell.

33.4.5 2. Ellenőrzési stratégia: referencia, invariáns és tulajdonságalapú tesztelés

A GPU-kód helyességének ellenőrzésére több, egymást kiegészítő eszköz áll rendelkezésre. A legegyszerűbb a referenciával való összehasonlítás, de nem mindig áll rendelkezésre megbízható CPU-referencia. Ilyenkor invariánsokhoz és tulajdonságalapú tesztekhez fordulhatunk.

33.4.6 2.1. Referenciával való összehasonlítás

A legegyszerűbb és leggyakrabban használt módszer: ugyanarra a bemenetre lefuttatjuk a GPU-kernelt és egy CPU-referenciát, majd a kimeneteket összehasonlítjuk. A CPU-referencia lehet:

- naiv implementáció, amely a GPU-verzióval azonos matematikai műveletet végez, de egyszerű, egyszálú kódban;
- könyvtári függvény, például cuBLAS vagy CUB, ha a cél éppen a könyvtár reprodukálása;
- analitikus megoldás, ha a feladat matematikailag zárt alakban ismert;
- nagy pontosságú számítás, például FP64 referencia egy FP32 kernel ellenőrzéséhez.

A referencia és a GPU-eredmény összehasonlításakor nem szabad egyszerűen `==`-ot használni. Lebegőpontos számoknál a toleranciaalapú összehasonlítás a helyes, amelyet a numerikus pontosságról szóló szakaszban részletezünk.

33.4.7 2.2. Invariánsok és konzisztencia

Ha nincs független referencia, a kernel belső konzisztenciáját ellenőrizhetjük invariánsokkal. Például:

- egy rendezésnél az eredménynek monoton növekvőnek kell lennie, és a kimeneti multihalmaznak azonosnak kell lennie a bemenetivel;
- egy mátrixszorzásnál az eredmény nyoma invariáns lehet;
- egy stencilnél egy konstans bemenetre a kimenetnek is konstansnak kell lennie;
- egy normalizálásnál az összegnek 1-hez közelinek kell lennie.

Az invariánsok gyakran kombinálhatók a referenciával: a referencia a pontos értéket adja, az invariáns pedig futás közben vagy nagyobb bemenetre is ellenőrizhető.

33.4.8 2.3. Tulajdonságalapú tesztelés

A tulajdonságalapú tesztelés során a bemenetet véletlenszerűen generáljuk egy megadott eloszlásból, majd azt ellenőrizzük, hogy a kimenetre egy megadott tulajdonság teljesül. GPU-n ez különösen hasznos, mert a hibák gyakran csak bizonyos bemeneti kombinációkra, például a tömbméret és a blokkméret bizonyos oszthatósági viszonyaira jelennek meg. A tulajdonságalapú tesztelésnél a véletlenmagot rögzíteni kell, hogy a teszt reprodukálható legyen.

A gyakorlatban a leghatékonyabb stratégia a referenciával való összehasonlítás kis, determinisztikus eseteken, majd kiegészítésként invariáns- és tulajdonságalapú tesztek a szélsőséges bemenetekre.

33.4.9 3. Tesztelési keret: egy kis, ellenőrizhető példa

A fejezet tanulási célja szerint a validációs eszközöket egy kicsi, ellenőrizhető példán is alkalmazni kell. A példa legyen egyetlen kernel, amely a záróprojekt szempontjából reprezentatív, de a teljes rendszer bonyolultsága nélkül. Válasszunk egy **súlyozott vektorredukciót**, amely a záróprojekt egyik gyakori építőköve: adott egy `x` vektor és egy `w` súlyvektor, és a feladat a `sum(x[i] * w[i])` skalárszorzat kiszámítása. A kernel a korábbi fejezetekben tanult redukciós mintát követi.

A tesztrendszer felépítése a következő rétegeket tartalmazza:

- determinisztikus bemenetgenerálás;
- CPU-referencia FP64-ben;
- GPU-kernel FP32-ben;
- toleranciaalapú összehasonlítás;
- hibakód-ellenőrzés minden CUDA API-híváson;
- a mérési környezet rögzítése.

A következő CUDA C++ kódrészlet a teszt magját mutatja. A kód a könyv konvenciói szerint minden API-hívást ellenőriz, és külön választja a mért és a várt eredményt.

```
// teszt_skalar_redukcio.cu
// CUDA C++ (C++17), fordító: nvcc, CUDA 12.2
// Példa: súlyozott vektorredukció validációja referenciával.

#include <cuda_runtime.h>
#include <cstdio>
#include <cstdlib>
#include <cmath>
#include <vector>

// Saját hibamakró a könyv labor-szerződése szerint.
#define CUDA_CHECK(call)                                                   \
    do {                                                                   \
        cudaError_t err = call;                                            \
        if (err != cudaSuccess) {                                          \
            std::fprintf(stderr, "CUDA hiba: %s:%d: %s\n",                 \
                         __FILE__, __LINE__, cudaGetErrorString(err)); \
            std::exit(EXIT_FAILURE);                                       \
        }                                                                  \
    } while (0)

// GPU-redukció: blokkonként shared memory-ban, majd atomi összegzés.
// A kernel FP32-ben számol, a referencia FP64-ben.
__global__ void weighted_sum_kernel(const float* x, const float* w,
                                    float* partial, int n) {
    extern __shared__ float sdata[];
    int tid = threadIdx.x;
    int i = blockIdx.x * blockDim.x + tid;

    // Coalescing-barát, grid-stride betöltés.
    float acc = 0.0f;
    for (int idx = i; idx < n; idx += blockDim.x * gridDim.x) {
        acc += x[idx] * w[idx];
    }
    sdata[tid] = acc;
    __syncthreads();

    // Blokkszintű redukció.
    for (int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] += sdata[tid + s];
```

```
        }
        __syncthreads();
    }

    if (tid == 0) {
        atomicAdd(partial, sdata[0]);
    }
}

// CPU-referencia FP64-ben, a súlyozott skalárszorzat.
double cpu_reference(const std::vector<float>& x,
                     const std::vector<float>& w) {
    double sum = 0.0;
    for (size_t i = 0; i < x.size(); ++i) {
        sum += static_cast<double>(x[i]) * static_cast<double>(w[i]);
    }
    return sum;
}

int main() {
    // Rögzített bemenet a reprodukálhatóságért.
    const int n = 1 << 20; // 1 048 576 elem
    const int block_size = 256;
    const int grid_size = (n + block_size - 1) / block_size;

    std::vector<float> h_x(n), h_w(n);
    // Determininisztikus, egyszerű inicializálás.
    for (int i = 0; i < n; ++i) {
        h_x[i] = static_cast<float>((i % 17) * 0.001);
        h_w[i] = static_cast<float>((i % 13) * 0.0005);
    }

    float *d_x, *d_w, *d_partial;
    CUDA_CHECK(cudaMalloc(&d_x, n * sizeof(float)));
    CUDA_CHECK(cudaMalloc(&d_w, n * sizeof(float)));
    CUDA_CHECK(cudaMalloc(&d_partial, sizeof(float)));
    CUDA_CHECK(cudaMemset(d_partial, 0, sizeof(float)));

    CUDA_CHECK(cudaMemcpy(d_x, h_x.data(), n * sizeof(float),
                          cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_w, h_w.data(), n * sizeof(float),
                          cudaMemcpyHostToDevice));

    // Kernelindítás: grid_size blokk, block_size szál,
    // block_size * sizeof(float) dinamikus shared memory.
    weighted_sum_kernel<<<grid_size, block_size,
                          block_size * sizeof(float)>>>(
        d_x, d_w, d_partial, n);
    CUDA_CHECK(cudaGetLastError());
    CUDA_CHECK(cudaDeviceSynchronize());

    // Eredmény visszaolvasása és ellenőrzés.
    float h_partial;
    CUDA_CHECK(cudaMemcpy(&h_partial, d_partial, sizeof(float),
                          cudaMemcpyDeviceToHost));

    double gpu_val = static_cast<double>(h_partial);
    double ref_val = cpu_reference(h_x, h_w);

    // Toleranciaalapú összehasonlítás: abszolút és relatív hiba.
    double abs_err = std::fabs(gpu_val - ref_val);
    double rel_err = abs_err / std::max(1.0, std::fabs(ref_val));
    const double tol_abs = 1e-3;
```

```
    const double tol_rel = 1e-5;

    std::printf("GPU eredmény (FP32): %.9f\n", gpu_val);
    std::printf("CPU referencia (FP64): %.15f\n", ref_val);
    std::printf("Abszolút hiba: %.6e\n", abs_err);
    std::printf("Relatív hiba: %.6e\n", rel_err);

    if (abs_err <= tol_abs && rel_err <= tol_rel) {
        std::printf("VALIDÁCIÓ: sikeres.\n");
    } else {
        std::printf("VALIDÁCIÓ: sikertelen, hibakorlát túllépve.\n");
    }

    CUDA_CHECK(cudaFree(d_x));
    CUDA_CHECK(cudaFree(d_w));
    CUDA_CHECK(cudaFree(d_partial));
    return 0;
}
```

A példa több dolgot is tanít. A kernel FP32-ben számol, a CPU-referencia FP64-ben, így a tolerancia a két pontosság közötti eltérést is lefedi. Az `atomicAdd` csak a blokkparciálisok összegzésére szolgál, ezért a kernel nem determinisztikus az összegzés sorrendjében, de ez a példában elfogadható, mert a cél a validációs módszertan bemutatása. Ha a záróprojekt deterministikus redukciót követel, a korábbi fejezetekben tárgyalt faalapú, sorrendrögzítő módszert kell használni.

A validáció akkor sikeres, ha a hiba a megadott korláton belül van. A korlátokat a numerikus pontossági elemzésből kell származtatni, nem önkényesen választani. A példában 1 millió elem összegzése FP32-ben körülbelül 1e-5 relatív hibát adhat, ezért a `tol_rel = 1e-5` határeset; a gyakorlatban érdemes a hibát empirikusan is megmérni, és a korlátot a várt hibánál kissé nagyobbra állítani.

33.4.10 4. Reprodukálhatóság: a mérés és a futás környezetének rögzítése

A reprodukálhatóság azt jelenti, hogy egy másik mérnök – vagy ugyanaz a mérnök a jövőben – a dokumentált lépések alapján ugyanazt az eredményt és ugyanazt a teljesítményt közelíti meg. A „közelíti” szó fontos: GPU-n a bitpontos reprodukálhatóság gyakran nem garantálható a hardveres ütemezés, a lebegőpontos műveletek sorrendje és a gyártói optimalizációk miatt. A mérnöki cél ezért a **statisztikailag reprodukálható** eredmény: ugyanaz a teszt ugyanazon a hardveren és szoftverkörnyezetben a megadott szóráson belül ugyanazt adja, más hasonló hardveren pedig a megadott tolerancián belül.

33.4.11 4.1. A környezet rögzítése

A könyv konvenciói szerint minden mérésnél rögzíteni kell:

- GPU modell és gyártó (például NVIDIA RTX 4090, 24 GB);
- compute capability (például 8.9);
- CUDA toolkit verzió (például 12.2);
- driver verzió (például 535.104.05);
- operációs rendszer (például Ubuntu 22.04 LTS);
- GPU-órajel és power limit beállítás (ha releváns);

- ECC bekapcsolva vagy kikapcsolva;
- MIG, virtualizáció vagy konténerizáció jelenléte;
- háttérterhelés: futott-e más folyamat a GPU-n;
- memóriaallokáció: hány bájtot foglaltunk, milyen memóriatérben.

Ezt az adatot a benchmarkriport elején kell szerepeltetni. A hiányzó környezeti adat a reprodukálhatóság leggyakoribb akadálya.

33.4.12 4.2. A mérési módszertan rögzítése

A reprodukálhatósághoz a mérési módszertant is dokumentálni kell, nemcsak a hardvert. A következő kérdésekre kell egyértelmű választ adni:

- Hány warm-up iteráció futott a mérés előtt?
- Hány ismétlésből számoltunk átlagot, szórást vagy percentilist?
- Milyen volt az input mérete, típusa és inicializálása?
- Milyen volt a kernel launch konfigurációja: grid, block, shared memory, stream?
- Milyen pontosságot használtunk (FP64/FP32/TF32/FP16/BF16/FP8)?
- Milyen időmérési eszközt használtunk: hostoldali `std::chrono`, CUDA events, Nsight?
- Hogyan kezeltük a kiugró értékeket?

A benchmarkriportban a mért és a várt eredményt külön kell választani. A mért eredmény a tényleges futtatásból származik; a várt eredmény a tervezési cél, például a roofline-modellből vagy a gyártói csúcsértékből számított elvárás.

33.4.13 4.3. Véletlenszerűség és determinizmus

Ha a bemenet véletlenszerű, a véletlenszám-generátor magját rögzíteni kell. Ugyanez vonatkozik a cuRAND-ra, a JAX random kulcsaira és az OpenCL-es véletlenforrásokra. A determinisztikus redukciókhoz a műveleti sorrendet is rögzíteni kell; ha a sorrend hardverfüggő, a toleranciát ennek megfelelően kell beállítani.

33.4.14 5. Numerikus pontosság és tolerancia

A numerikus pontosság a helyességvizsgálat része, de annyira fontos, hogy külön szakaszt érdemel. A GPU-k gyakran alacsonyabb pontosságú típusokkal dolgoznak a teljesítmény érdekében, ezért a hibakorlátokat tudatosan kell megválasztani.

33.4.15 5.1. Lebegőpontos hibák forrásai

A hibák forrásai a következők:

- **Kerekítési hiba:** minden lebegőpontos művelet kerekít, a hiba a műveleti sorrendtől függ.
- **Kumulált hiba:** hosszú redukcióban a kerekítési hibák összeadódnak; a hiba a redukció hosszával nő.
- **Kivonási kioltás:** közel egyenlő számok kivonása relativ nagy hibát eredményez.
- **Típuskonverzió:** FP64-ből FP32-be konvertálás elveszti a mantissza alsó bitjeit; FP16 és BF16 tovább csökkenti a pontosságot.

- **FMA és optimalizáció:** a fuzionált szorzás-összeadás (`fma`) kevesebb kerekítést végez, mint a külön szorzás és összeadás, ezért az eredmény is más lehet. A fordítói optimalizáció és a GPU-architektúra befolyásolja, hogy az FMA használható-e.

33.4.16 5.2. Toleranciaalapú összehasonlítás

A helyes összehasonlítás nem az `==` operátor, hanem a következő kritérium:

$$|a - b| \leq \epsilon_{abs} + \epsilon_{rel} \cdot \max(|a|, |b|),$$

ahol ϵ_{abs} az abszolút, ϵ_{rel} a relatív tolerancia. A relatív tag a nagy értékeknél, az abszolút tag a nullához közeli értékeknél biztosítja az összehasonlítást. A tolerancia értékét a pontossági elemzésből kell származtatni, nem tippelni.

Például egy 1 millió elemű FP32 redukciónál a relatív hiba nagyságrendje 10^{-5}, egy FP64 redukciónál 10^{-15}, egy FP16 redukciónál viszont már 10^{-2} is lehet. Ha a referencia FP64, a GPU FP32, akkor a 10^{-5} relatív tolerancia az FP32 reprezentáció hibáját fedi le, nem a kernel hibáját.

33.4.17 5.3. A pontosság rögzítése a riportban

A benchmark- és tesztriportban minden numerikus kísérletnél rögzíteni kell a pontosságot: FP64, FP32, TF32, FP16, BF16 vagy FP8. A könyv konvenciói szerint ez kötelező. Ha a kernel vegyes pontosságot használ, például TF32 bemenetet FP32 akkumulációval, azt is dokumentálni kell, ahogyan a `gpu088` alfejezetben tárgyaltuk.

33.4.18 6. Teszt- és benchmarkriport: a labor átadandó eredménye

A fejezet laborjának átadandó eredménye egy teszt- és benchmarkriport, amely CUDA C++ (vagy OpenCL/JAX) kóddal, rögzített GPU-val, driverrel, CUDA-verzióval és mérési naplóval rendelkezik. A riportnak a mért és a várt eredményt külön kell választania. A következő struktúra ajánlott.

33.4.19 6.1. A riport felépítése

1. Cél és hipotézis. Mit akarunk igazolni? Például: „A súlyozott vektorredukció GPU-kernel helyes a megadott toleranción belül, és legalább 10-szer gyorsabb, mint a CPU-referencia 1 millió elemre.”

2. Környezet. A hardver- és szoftverkörnyezet teljes listája, a 4.1. szakasz szerint.

3. Bemenet és módszertan. A bemenet generálása, a véletlenmag, a kernel launch konfigurációja, a mérési protokoll (warm-up, ismétlések, időmérési eszköz).

4. Helyességi eredmények. A mért hiba, a tolerancia, a validáció kimenete. Ha a teszt többszintű, a szintek eredménye külön-külön.

5. Teljesítményeredmények. A futásidő, az átbocsátóképesség, a sávszélesség, az occupancy, a regiszterhasználat, a shared memory használat, a cache találati arány, ha mérhető.

6. A mért és a várt eredmény összehasonlítása. A várt eredményt a roofline-modellből, a gyártói specifikációból vagy a tervezési célból származtatjuk. Az eltérést meg kell magyarázni.

7. Következtetés és korlátok. Mit igazoltunk, mit nem, és milyen feltételek mellett érvényes az állítás.

33.4.20 6.2. Példa: benchmarkriport kivonat

A fenti súlyozott vektorredukciós példához a riport kivonata így nézhet ki:

- **Cél:** A GPU-kernel helyességének és teljesítményének igazolása 1 048 576 elemű vektorra.
- **Környezet:** NVIDIA RTX 4090, compute capability 8.9, CUDA 12.2, driver 535.104.05, Ubuntu 22.04, ECC off, MIG off, power limit 450 W, háttérterhelés nincs.
- **Kernel:** `weighted_sum_kernel`, grid 4096 blokk, block 256 szál, shared memory 1024 bájt, FP32.
- **Mérési protokoll:** 5 warm-up iteráció, 20 mért ismétlés, átlag és szórás, CUDA events.
- **Helyesség:** abszolút hiba $2{,}3 \cdot 10^{-4}$, relatív hiba $1{,}1 \cdot 10^{-6}$, tolerancia 10^{-3} abszolút és 10^{-5} relatív, validáció sikeres.
- **Teljesítmény:** átlagos futásidő $42{,}1 \pm 1{,}3$ μs, effektív sávszélesség 99,7 GB/s, occupancy 75 %, regiszterhasználat 24 regiszter/szál, shared memory 1 KiB/blokk.
- **Várt teljesítmény:** a roofline-modell alapján a memóriakorlátos tartományban 100 GB/s várható; a mért érték ettől 0,3 %-kal marad el, ami a mérési zajon belül van.
- **Következtetés:** a kernel helyes és a memóriasávszélesség-korlátos tartományban működik; a reprodukálhatóság a rögzített környezetben fennáll.

A riportban a mért érték (42,1 μs) és a várt érték (100 GB/s) egyértelműen elkülönül: az egyik a mérésből, a másik a modellből származik.

33.4.21 7. Dokumentáció és bemutató

A dokumentáció nem utólagos kötelezettség, hanem a validáció része. A jó dokumentáció lehetővé teszi, hogy a tesztet egy másik mérnök futtatni tudja, és hogy a jövőbeli énünk is megértse a döntéseket.

33.4.22 7.1. A dokumentáció minimális tartalma

A záróprojekt dokumentációjának a következőket kell tartalmaznia:

- **Algoritmusleírás:** mit számol a kernel, milyen matematikai modell alapján, milyen párhuzamos mintát használ.
- **Kernel-specifikáció:** bemenetek, kimenetek, dimenziók, memóriaterek, pontosság, launch konfiguráció.
- **Környezeti követelmények:** GPU-modell, compute capability, CUDA-verzió, szükséges könyvtárak.
- **Fordítási és futtatási utasítások:** a pontos `nvcc` parancs, a futtatási környezet beállítása, a bemenet generálása.
- **Teljesítménycélok és mérési módszertan:** mit vártunk, mit mértünk, milyen protokollal.

- **Ismert korlátok:** milyen bemenetre nem érvényes, milyen pontossági határok vannak, milyen környezetben nem teszteltük.
- **Változásnapló:** a kernel és a tesztváltozatok rövid leírása, hogy a fejlődés nyomon követhető legyen.

33.4.23 7.2. A bemutató

A záróprojekt bemutatása során a hangsúly a bizonyítékon van, nem a kódon. A bemutató íve a következő lehet:

1. **Probléma és cél:** mit oldunk meg, miért fontos, mi az elvárt eredmény.
2. **Módszer:** milyen párhuzamos algoritmust választottunk, miért, milyen optimalizálásokat alkalmaztunk.
3. **Helyesség:** milyen tesztstratégiát használtunk, milyen hibakorlátot értünk el, hogyan igazoljuk a helyességet.
4. **Teljesítmény:** milyen méréseket végeztünk, hogyan viszonyul a mért eredmény a várt elméleti értékhez, milyen szűk keresztmetszetek maradtak.
5. **Reprodukálhatóság:** hogyan lehet a tesztet újra lefuttatni, milyen környezeti feltételek mellett.

A bemutatóban a mért és a várt eredmény vizuális összehasonlítása meggyőzőbb, mint a számsorok felolvasása. Egy roofline-diagram, egy hibatáblázat és egy futtatási napló kivonata elegendő.

33.4.24 8. Tipikus hibák és helyes megoldások

A fejezet tanulási célja szerint fel kell ismerni a tipikus hibákat, és meg kell indokolni a helyes megoldást. A leggyakoribb hibák a záróprojekt validációs fázisában a következők.

33.4.25 8.1. Hiba: nincs hibakód-ellenőrzés a CUDA API-hívásokon

A `cudaMalloc`, `cudaMemcpy`, `cudaMemset` és a kernelindítás hibája észrevétlen marad. A tesztrendszer a hibás memóriaművelet ellenére is lefut, és tévesen sikeres eredményt jelezhet.

Helyes megoldás: minden CUDA API-hívást a fejezet elején bemutatott `CUDA_CHECK` makróval ellenőrizni, a kernelindítás után `cudaGetLastError()`-t és `cudaDeviceSynchronize()`-t hívni. A hibamakró a fájl és a sor megjelölésével azonnal jelzi a hibát.

33.4.26 8.2. Hiba: == használata lebegőpontos összehasonlításra

A GPU és a CPU eredménye bitpontosan eltér, a teszt sikertelen, pedig a hiba a lebegőpontos reprezentációból és a műveleti sorrendből adódik.

Helyes megoldás: toleranciaalapú összehasonlítást használni az 5.2. szakasz szerint. A toleranciát a pontossági elemzésből származtatni, és a riportban rögzíteni.

33.4.27 8.3. Hiba: a kimeneti puffer nincs inicializálva

A `cudaMalloc` nem nulláz, a kernel pedig feltételezi, hogy a kimenet nulla kezdőértékű. A teszt attól függően sikerül vagy bukik, hogy a memóriában éppen mi volt.

Helyes megoldás: a kimeneti puffert `cudaMemset`-tel vagy a kernelben explicit inicializálással nullázni. Ha a kernel atomi összegzést végez, a célpuffert mindig nullázni kell az indítás előtt.

33.4.28 8.4. Hiba: a teszt nem reprodukálható a véletlen bemenet miatt

A teszt véletlen bemenetet generál, de a véletlenmag nincs rögzítve. A hiba ritkán, csak bizonyos bemenetekre jelenik meg, és a reprodukálás sikertelen.

Helyes megoldás: a véletlenmagot konstansként rögzíteni, a generálást a riportban dokumentálni. Ugyanez vonatkozik a cuRAND-ra és a JAX random kulcsaira.

33.4.29 8.5. Hiba: a mérési környezet nincs rögzítve

A riportban csak a futásidő szerepel, a GPU-modell, a driververzió és a háttérterhelés nem. A teljesítményállítást nem lehet reprodukálni, és a „gyorsabb” állítás sem igazolható.

Helyes megoldás: a 4.1. szakaszban felsorolt környezeti adatokat hiánytalanul rögzíteni. A könyv konvenciói szerint ez kötelező.

33.4.30 8.6. Hiba: a várt teljesítményt nem különítik el a mérttől

A riport összemossa a mért futásidőt és a gyártói csúcsértékből számított elvárást. Azt állítja például, hogy a kernel „eléri a csúcssávszélességet”, de nem adja meg, mi a mért és mi a várt érték.

Helyes megoldás: a riportban a 6.1. szakasz szerint külön kell választani a mért és a várt eredményt. A várt eredményt a roofline-modellből vagy a tervezési célból kell származtatni, és az eltérést meg kell magyarázni.

33.4.31 9. Összefoglalás és önellenőrzés

A `gpu129` alfejezet a záróprojekt validációs és dokumentációs fázisát tárgyalta. A helyesség nem egyetlen teszt, hanem rétegenkénti ellenőrzés: funkcionális helyesség, párhuzamos helyesség, memóriabiztonság, numerikus pontosság és erőforrás-helyesség. A referenciával való összehasonlítás mellett az invariáns- és tulajdonságalapú tesztek is fontosak. A reprodukálhatóság a környezet és a mérési módszertan rögzítését követeli meg. A numerikus pontosságot toleranciaalapú összehasonlítással kell igazolni, a toleranciát pedig a pontossági elemzésből kell származtatni. Végül a dokumentáció és a bemutató a validáció szerves része: a mért és a várt eredmény különválasztása, a környezet rögzítése és a korlátok explicit kimondása teszi az optimalizált GPGPU-rendszert mérnöki eredménnyé.

Önellenőrző kérdések:

1. Mi a különbség a funkcionális helyesség és a párhuzamos helyesség között? Adj példát olyan hibára, amely csak az egyiket sérti.
2. Miért nem elegendő a `==` operátor a GPU- és CPU-eredmény összehasonlításához? Hogyan választod meg a toleranciát?
3. Sorold fel a benchmarkriport környezeti adatainak legalább öt elemét, és indokold, miért fontos mindegyik a reprodukálhatósághoz.
4. Mi a különbség a mért és a várt eredmény között a teljesítményriportban? Milyen forrásból származtatható a várt eredmény?
5. Milyen tipikus hibát okoz, ha a kimeneti puffert nem inicializálod `cudaMemset`-tel egy atomi összegzést végző kernel előtt?

A fejezetben bemutatott tesztrendszer, benchmarkriport-struktúra és dokumentációs sablon együtt alkotják azt a minőségbiztosítási keretet, amely lehetővé teszi, hogy a záróprojekt ne csak gyors, hanem igazoltan helyes és reprodukálható is legyen.

Függelékek

Fogalomtár (magyar–angol)

A fogalomtár a könyvben használt magyar és angol szakkifejezéseket, valamint a leggyakoribb félreértéseket rögzíti. A tételek ábécésorrendben szerepelnek.

Magyar	English	Jelentés / megkülönböztetés
Arithmetic intensity	arithmetic intensity	A számítási műveletek és a memóriába mozgatott bájtok aránya; a Roofline-modell kulcsmennyisége.
Atomi művelet	atomic operation	Oszthatatlan memóriaművelet, amelyet más szálak nem látnak részlegesen végrehajtott állapotban.
Bank conflict	bank conflict	A shared memory bankjainak ütközése, amikor egy warp több szálának hozzáférése ugyanarra a bankra esik.
Barrier	barrier	Szinkronizációs pont, amelynél a csoport tagjai megvárják egymást.
Bitonic sort	bitonic sort	Párhuzamos összehasonlító rendezés bitonikus sorozatokon és compare-exchange hálózaton.
Block	thread block	CUDA-szálak csoportja, amely egy SM-en fut, és shared memórián osztozik.
C++/CUDA	CUDA C++	A CUDA C++ nyelvi kiterjesztései: kernelminősítők, beépített változók és eszközkód.
Coalescing	memory coalescing	A warp szálai által végzett memóriahozzáférések egyesítése nagyobb tranzakciókká.
Command queue	command queue	OpenCL-parancsok sorba állított végrehajtási csatornája.
Compute capability	compute capability	A GPU architektúrájának verziószáma, amely meghatározza az elérhető CUDA-feature-öket.

Magyar	English	Jelentés / megkülönböztetés
Context	OpenCL context	Az OpenCL eszközök, memóriaobjektumok és kernelek végrehajtási környezete.
Cooperative groups	cooperative groups	A CUDA csoportabsztrakciója szálak, blokkok és grid szintű együttműködésre.
cp.async	asynchronous copy	Aszinkron másolás globális memóriából shared memóriába, a számítás és az adatmozgatás átfedésére.
CUDA	Compute Unified Device Architecture	Az NVIDIA GPU-programozási modellje és eszközlánca.
CUDA graph	CUDA graph	Kernel- és memóriaműveletek gráfja, amely csökkenti az ismételt launch overheadjét.
CUTLASS	CUTLASS	NVIDIA sablonkönyvtár hatékony GEMM- és tensor core kernelékhez.
Divergencia	warp divergence	Amikor egy warp szálai eltérő vezérlési utat hajtanak végre, és a végrehajtás sorosítódik.
ECC	error-correcting code	A GPU-memória hibajavító kódolása; befolyásolhatja a kapacitást és a teljesítményt.
Esemény	CUDA event	Időbélyegzésre és stream-szinkronizációra használt CUDA-objektum.
FFT	fast Fourier transform	A diszkrét Fourier-transzformáció gyors algoritmusa; CUDA-n a cuFFT könyvtár.
FP16	half precision	16 bites lebegőpontos formátum; tensor core és mixed precision számításokhoz.
FP32	single precision	32 bites lebegőpontos formátum, a CUDA alapértelmezett pontossága.
FP64	double precision	64 bites lebegőpontos formátum; GPU-típusonként eltérő sebességű.
FP8	8-bit floating point	8 bites lebegőpontos formátum, főleg deep learning gyorsításhoz.
GEMM	general matrix multiply	Általános mátrixszorzás; a deep learning és a lineáris algebra alapesetete.

Magyar	English	Jelentés / megkülönböztetés
Global memory	global memory	A GPU nagy kapacitású, minden szál által elérhető memóriája.
GPGPU	general-purpose computing on GPU	GPU használata általános célú, nem grafikai számításokra.
Grid	grid	A kernelindításkor létrehozott thread blockok halmaza.
Grid-stride loop	grid-stride loop	Olyan ciklus, amelyben a szálak a grid méretével lépkedve dolgozzák fel a tömböt.
HBM	high bandwidth memory	A GPU-k nagy sávszélességű, stackelt memóriája.
HIP	HIP	AMD GPU-programozási modell, amely CUDA-szerű API-t kínál.
Histogram	histogram	Értékek gyakoriságának számítása; GPU-n atomicsal vagy privatizált shared memóriával.
Host	host	A CPU és annak memóriája a CUDA-programban.
JAX	JAX	Funkcionális, kompozícióképes numerikus könyvtár JIT-tel és TPU/GPU támogatással.
Kernel	kernel	A GPU-n párhuzamosan futó függvény; CUDA-ban `__global__` minősítővel.
L1 cache	L1 cache	Az SM-hez közeli gyorsítótár; shared memóriával osztozhat.
L2 cache	L2 cache	A GPU összes SM-je által megosztott nagyobb gyorsítótár.
Lane	lane	Egy szál a warpon belül; 32 lane alkot egy warp-ot.
Latency hiding	latency hiding	A memória- és utasításkésleltetés elrejtése több warp és ILP segítségével.
Launch bounds	launch bounds	A kernel regiszter- és blockméretére vonatkozó fordítási útmutatás.
Local memory	local memory	Szálankénti, a globális memóriában elhelyezkedő tár; gyakran regiszter-spill eredménye.
MIG	multi-instance GPU	A GPU hardveres particionálása több izolált példányra.

Magyar	English	Jelentés / megkülönböztetés
Mixed precision	mixed precision	Különböző precisionök együttes használata a sebesség és a pontosság egyensúlyára.
Monte Carlo	Monte Carlo	Véletlen mintavételen alapuló numerikus módszer.
MXU	matrix multiply unit	A TPU mátrixszorzó egysége, szisztolikus tömb felépítésű.
NCCL	NVIDIA Collective Communications Library	Több GPU közötti kollektív kommunikáció könyvtára.
Nsight Compute	Nsight Compute	NVIDIA kernel-szintű profilozó eszköze.
Nsight Systems	Nsight Systems	NVIDIA rendszerszintű idővonal-profilozója.
NVLink	NVLink	NVIDIA nagy sávszélességű GPU-GPU összeköttetése.
Occupancy	occupancy	Az SM-en egyidejűleg aktív warpok aránya a maximálishoz képest.
OpenCL	OpenCL	Khronos nyílt, heterogén párhuzamos programozási szabványa.
OpenCL C	OpenCL C	Az OpenCL kernelnyelve, a C nyelv kiterjesztése.
Pinned memory	pinned memory	Oldalzárolt hostmemória, amely gyorsabb aszinkron másolást tesz lehetővé.
Pipeline	pipeline	A számítás és az adatmozgatás szakaszainak átfedése.
Platform	OpenCL platform	Egy OpenCL-implementációt kínáló gyártói környezet.
PTX	Parallel Thread Execution	Az NVIDIA virtuális ISA-ja, amelyre a CUDA C++ fordít.
Roofline	Roofline model	A számítási és memóriateljesítmény korlátját együtt ábrázoló teljesítménymodell.
Scan	prefix sum	Az inclusive/exclusive prefixösszeg párhuzamos algoritmusa.
Shared memory	shared memory	Egy thread block tagjai által közösen használt, gyors memória.
SIMT	single instruction, multiple threads	A GPU warp-szintű végrehajtási modellje.
SM	streaming multiprocessor	A GPU végrehajtási egysége, amely warpokat és erőforrásokat kezel.
SpMV	sparse matrix-vector multiplication	Ritka mátrix és vektor szorzása.

Magyar	English	Jelentés / megkülönböztetés
Stream	CUDA stream	Aszinkron műveletek sorozata, amelyeken belül a végrehajtási sorrend garantált.
SYCL	SYCL	Khronos C++-alapú, egyforrásos heterogén programozási modellje.
Tensor core	tensor core	NVIDIA mátrixszorzó hardveregysége, amely kis mátrixok szorzatát gyorsítja.
Texture memory	texture memory	A GPU textúrázó egységén keresztül elérhető, cache-elt memóriatér.
Thread	thread	A CUDA végrehajtás legkisebb egysége.
Tiling	tiling	Az adatok blokkokba (tile-okba) szervezése a memóriahozzáférés és az újrafelhasználás javítására.
TPU	Tensor Processing Unit	Google mátrixszorzásra optimalizált AI-akcelerátora.
Unified memory	unified memory	A host és a device által közösen elérhető, automatikusan migrált memóriacím-tér.
Warp	warp	32 CUDA-szál csoportja, amelyet a hardver együtt ütemez.
Warp shuffle	warp shuffle	Warp-on belüli közvetlen adatcsere intrinsic műveletekkel.
WMMA	warp matrix multiply-accumulate	A tensor coreok warp-szintű mátrixszorzó API-ja.
Work-group	OpenCL work-group	Az OpenCL munkacsoportja, amely local memórián és barrieren osztozik.
Work-item	OpenCL work-item	Egy OpenCL kernelpéldány végrehajtási egysége.
XLA	Accelerated Linear Algebra	A Google optimalizáló compilerje, amely TPU-ra és GPU-ra fordít.

CUDA API- és kernelkatalógus

A katalógus a könyvben használt CUDA API-kat, kernelminősítőket, beépített változókat és könyvtári hívásokat rendszerezi. A pontos szignatúrát, a visszatérési értéket és a verziófüggést mindig a hivatalos CUDA-dokumentációban kell ellenőrizni. A „Verzió” oszlop csak jelzés: a tényleges támogatást a `compute capability` és a CUDA-verzió együtt határozza meg.

Device- és memóriakezelés

API	Cél	Megjegyzés
`cudaGetDeviceCount`	eszközök számának lekérdezése	több GPU
`cudaSetDevice`	aktív eszköz kiválasztása	context
`cudaGetDeviceProperties`	eszköztulajdonságok	compute capability, memória
`cudaMalloc`	device-memória foglalása	hibát ellenőrizni
`cudaFree`	device-memória felszabadítása	ne hagyd ki
`cudaMemcpy`	host-device másolás	szinkron
`cudaMemcpyAsync`	aszinkron másolás	pinned memory
`cudaMallocManaged`	unified memory	migráció
`cudaMallocPitch`	pitch-elt 2D allokáció	2D tömbfolytató
`cudaMemPrefetchAsync`	unified memory prefetch	migráció
`cudaMemAdvise`	unified memory tanács	hozzáférési minta
`cudaMemset`	memória feltöltése	
`cudaHostAlloc`	pinned hostmemória	`cudaHostAllocDefault`
`cudaHostRegister`	meglévő memória pinelése	
`cudaMemcpyPeer`	GPU-GPU másolás	peer access
`cudaDeviceEnablePeerAccess`	P2P engedélyezése	topológia

Kernel launch és szinkronizáció

API / kulcsszó	Cél	Megjegyzés
`<<<grid, block, shmem, stream>>>`	kernel indítása	launch konfiguráció
`__global__`	kernel függvény	device-ról hívható
`__device__`	device-függvény	device-ról hívható
`__host__`	host-függvény	alapértelmezett
`__shared__`	shared memória	blockon belül
`__constant__`	constant memória	írás hostról
`__syncthreads`	block-szinkronizáció	minden szál
`__syncwarp`	warp-szinkronizáció	warp
`cudaDeviceSynchronize`	device szinkronizáció	teljes
`cudaStreamSynchronize`	stream szinkronizáció	stream
`cudaEventSynchronize`	esemény szinkronizáció	event
`cudaGetLastError`	utolsó hiba	launch után
`cudaPeekAtLastError`	hiba lekérdezése	nem törli

Beépített változók

Változó	Jelentés	Típus
`threadIdx`	szál indexe a blockon belül	`dim3`
`blockIdx`	block indexe a gridben	`dim3`
`blockDim`	block mérete	`dim3`
`gridDim`	grid mérete	`dim3`
`warpSize`	warp mérete	32

Atomics és warp intrinsics

API	Cél	Megjegyzés
`atomicAdd`	atomi összeadás	float/int
`atomicSub`	atomi kivonás	
`atomicExch`	atomi csere	
`atomicMin/Max`	atomi minimum/maximum	
`atomicCAS`	compare-and-swap	lock-free
`atomicInc/Dec`	atomi növelés/csökkentés	
`__shfl_sync`	warp shuffle	maszkos
`__shfl_up_sync`	felfelé shuffle	
`__shfl_down_sync`	lefelé shuffle	
`__ballot_sync`	szavazás	aktív szálak
`__any_sync`	bármelyik igaz	
`__all_sync`	mind igaz	
`__match_any_sync`	egyező értékű lane-ek	

Streamek, eventek és gráfok

API	Cél	Megjegyzés
`cudaStreamCreate`	stream létrehozása	
`cudaStreamDestroy`	stream megszüntetése	
`cudaStreamWaitEvent`	stream várakozás	
`cudaEventCreate`	event létrehozása	
`cudaEventRecord`	event rögzítése	
`cudaEventElapsedTime`	időmérés	ms
`cudaGraphCreate`	gráf létrehozása	
`cudaGraphAddKernelNode`	kernel csomópont	
`cudaGraphInstantiate`	gráf példányosítása	
`cudaGraphLaunch`	gráf futtatása	
`cudaStreamBeginCapture`	stream capture	
`cudaStreamEndCapture`	capture lezárása	

Könyvtárak

Könyvtár	Fő API	Cél
cuBLAS	`cublasSgemm`, `cublasDgemm`	dense GEMM
cuSPARSE	`cusparseSpMV`	sparse mátrix
cuFFT	`cufftExecC2C`	FFT
cuRAND	`curandGenerateUniform`	random számok
Thrust	`thrust::reduce`, `transform`	magas szintű algoritmusok
CUB	`cub::DeviceReduce`, `DeviceScan`	blokk- és device-algoritmusok
CUTLASS	`cutlass::gemm::device::Gemm`	tensor core GEMM
cuDNN	`cudnnConvolutionForward`	deep learning
NCCL	`ncclAllReduce`	multi-GPU kollektíva

OpenCL megfeleltetés

CUDA	OpenCL	Megjegyzés
thread	work-item	
block	work-group	
shared memory	local memory	
global memory	global memory	
`__syncthreads`	`barrier(CLK_LOCAL_MEM_FENCE)`	
kernel	`__kernel`	
grid	NDRange	
stream	command queue	
event	event	

TPU/JAX megfeleltetés

CUDA-fogalom	TPU/JAX fogalom	Megjegyzés
kernel	jitted függvény	XLA fordítás
grid/block	sharding/mesh	eszközök közötti felosztás
shared memory	lokalitás/fusion	XLA optimalizálás
stream	aszinkron dispatch	XLA runtime
cuBLAS	XLA GEMM	compiler-generált
NCCL	collective	TPU interconnect

GPU-, profilozó- és build-eszközkatalógus

A katalógus a könyv laborjaihoz használható eszközöket rendszerezi. A verziók, a licenc és a hardvertámogatás időben változik; a laborokban a tényleges verziót és a GPU-konfigurációt kell rögzíteni. A katalógus nem ír elő egyetlen kötelező eszközláncot.

CUDA-fordítás és build

Eszköz	Szerep	Megjegyzés
CUDA Toolkit	fordító és runtime	`nvcc`, könyvtárak
nvcc	CUDA C++ fordító	`-arch`, `-gencode`
CMake	build-generálás	CUDA language support
Ninja	build végrehajtás	gyors inkrementális build
Make	build	egyszerű példák
Bazel	build	nagy monorepo
Spack	csomagkezelés	HPC-környezet
Conda	környezet	CUDA runtime és Python
NVIDIA Container Toolkit	konténer GPU	Docker/Podman

CUDA-könyvtárak

Könyvtár	Szerep	Megjegyzés
cuBLAS	dense lineáris algebra	GEMM
cuSPARSE	sparse lineáris algebra	SpMV
cuFFT	FFT	1D/2D/3D, batch
cuRAND	random számok	Philox, MRG
Thrust	magas szintű algoritmusok	STL-szerű
CUB	device-szintű primitívek	reduce, scan, sort
CUTLASS	GEMM és tensor core	sablonkönyvtár
cuDNN	deep learning	conv, attention
NCCL	multi-GPU kollektíva	all-reduce
TensorRT	inference optimalizálás	NVIDIA
Triton	GPU kernel compiler	Python-alapú

Debug és korrektség

Eszköz	Szerep	Megjegyzés
cuda-gdb	GPU debugger	breakpoint, lane
Compute Sanitizer	memória- és versenyellenőrzés	memcheck, racecheck
cuda-memcheck	régi memóriaellenőrző	legacy
Nsight Compute	kernelprofilozó	metrikák
Nsight Systems	rendszerprofilozó	idővonal
CUPTI	profilozó API	saját eszközök
nvidia-smi	GPU-állapot	hőmérséklet, memória
nvtop	GPU-monitor	interaktív
DCGM	adatközponti monitorozás	NVIDIA
gpustat	egyszerű GPU-státusz	Python

Teljesítménymérés

Eszköz	Szerep	Megjegyzés
CUDA events	időmérés	`cudaEventElapsedTime`
Google Benchmark	benchmark	CPU/GPU
Nsight Compute	kernelmetrikák	occupancy, roofline
Nsight Systems	rendszer-idővonal	overlap
perf	CPU profiling	Linux
VTune	Intel profiling	CPU/GPU
TAU	HPC profiling	trace
HPCToolkit	HPC profiling	egész alkalmazás
nvidia-smi dmon	telemetria	órajel, teljesítmény
NVML	GPU-monitor API	energia

TPU, XLA és JAX

Eszköz	Szerep	Megjegyzés
JAX	numerikus könyvtár	JIT, vmap, sharding
XLA	compiler	TPU/GPU
Cloud TPU	TPU-szolgáltatás	Google Cloud
JAX Profiler	TPU-profilozás	trace
TensorBoard	vizualizáció	profiler
Orbax	checkpoint	JAX
Flax	neurális háló	JAX
Optax	optimalizálás	JAX

OpenCL, SYCL és HIP

Eszköz	Szerep	Megjegyzés
PoCL	hordozható OpenCL	CPU/GPU
Intel OpenCL	Intel GPU/CPU	oneAPI
ROCm	AMD GPU stack	HIP/OpenCL
clinfo	OpenCL-eszközök	diagnosztika
clpeak	OpenCL benchmark	sávszélesség
SYCL	C++ heterogén modell	oneAPI
DPC++	SYCL-fordító	Intel
HIP	CUDA-szerű AMD modell	portolás
hipify	CUDA→HIP fordító	forrástranszformáció

Konténer, felhő és reprodukálhatóság

Eszköz	Szerep	Megjegyzés
Docker	konténer	NVIDIA runtime
Podman	konténer	rootless
Apptainer/Singularity	HPC konténer	shared cluster
enroot	HPC konténer	NVIDIA
Conda/pip	Python környezet	JAX/PyTorch
CMake Presets	build konfiguráció	reprodukálható
Git	verziókezelés	kód és konfiguráció
DVC	adatverzió	nagy adathalmazok

Hozzáférés és etika

- Csak saját, bérelt vagy engedélyezett GPU-t használj.
- A felhős GPU/TPU használatnál figyeld a költséget és a kvótát.
- A hosszú futású kernelt timeouttal és monitorozással futtasd.
- A többfelhasználós GPU-n (MIG, time-slicing) izoláld az adatokat.
- A benchmarkokat azonos hardveren, azonos driverrel és CUDA-verzióval hasonlítsd össze.

Kompetenciatérkép és tanulási útvonal

A térkép a könyv fejezeteit nyolc kompetenciaterülethez rendeli. A fejezetszámok a PDF és a HTML azonosítóival egyeznek; a záróprojekt (gpuch32) minden területet integrál.

Kompetenciaterületek

Kompetencia	Fejezetek	Bizonyíték	Szint
C1 – GPU-architektúra és CUDA-modell	gpuch01–gpuch05	architektúraelemzés, első kernel, launch-konfiguráció	alapszint
C2 – Kernel, memória és szinkronizáció	gpuch06–gpuch09	memóriatér-terv, atomics, sanitizer-riport	haladó
C3 – Teljesítmény és memóriamodellek	gpuch10–gpuch14	coalescing-mérés, occupancy, Roofline, tiling	haladó
C4 – Párhuzamos algoritmusok	gpuch15–gpuch20	reduce/scan, sort, stencil, SpMV, FFT	haladó
C5 – Könyvtárak és deep learning	gpuch21–gpuch25	cuBLAS/CUB, CUTLASS, attention, multi-GPU	mesterszint
C6 – TPU, OpenCL és hordozhatóság	gpuch26–gpuch29	TPU/JAX példa, OpenCL kernel, portolási terv	haladó
C7 – Profilozás, numerika és mérnökség	gpuch30–gpuch31	Nsight-riport, benchmark, pontossági elemzés	mesterszint
C8 – Önálló tervezés és záróprojekt	gpuch32	end-to-end GPU-projekt, dokumentáció és bemutató	mesterszint

Tanulási útvonalak

CUDA-fejlesztő

Cél: hatékony CUDA-kernelek írása és optimalizálása.

1. Alapok: gpuch01–gpuch05.
2. Memória és szinkronizáció: gpuch06–gpuch09.
3. Teljesítmény: gpuch10–gpuch14.
4. Algoritmusok: gpuch15–gpuch20.
5. Könyvtárak: gpuch21–gpuch24.
6. Záróprojekt: gpuch32.

HPC- és tudományos számítási mérnök

Cél: szimulációk, lineáris algebra és FFT GPU-n.

1. Architektúra és modell: gpuch01–gpuch05.
2. Memória és teljesítmény: gpuch06–gpuch14.
3. Algoritmusok: gpuch15–gpuch20.
4. Könyvtárak: gpuch21.
5. Multi-GPU: gpuch25.
6. Profilozás és numerika: gpuch30–gpuch31.
7. Záróprojekt: gpuch32.

ML-rendszermérnök

Cél: deep learning kernelek és GPU/TPU skálázás.

1. Alapok: gpuch01–gpuch05.
2. Memória és teljesítmény: gpuch06–gpuch14.
3. Tensor core és deep learning: gpuch22–gpuch23.
4. Streamek és multi-GPU: gpuch24–gpuch25.
5. TPU/JAX: gpuch26–gpuch27.
6. Profilozás: gpuch30.
7. Záróprojekt: gpuch32.

Teljesítmény- és optimalizálási mérnök

Cél: mérhető gyorsítás és hatékony erőforrás-használat.

1. Architektúra: gpuch01–gpuch02.
2. CUDA-modell: gpuch03–gpuch09.
3. Memória és Roofline: gpuch10–gpuch14.
4. Algoritmusok: gpuch15–gpuch20.
5. Profilozás és benchmark: gpuch30.
6. Záróprojekt: gpuch32.

Hordozható heterogén programozó

Cél: CUDA, OpenCL, SYCL és HIP közötti portolás.

1. CUDA-alapok: gpuch01–gpuch09.
2. Teljesítmény: gpuch10–gpuch14.
3. OpenCL: gpuch28–gpuch29.
4. TPU/JAX: gpuch26–gpuch27.
5. Hordozhatóság: gpuch31.
6. Záróprojekt: gpuch32.

Értékelési rubrika

Minden kompetenciaterületnél három szintet különböztetünk meg:

- **Ismeri:** meg tudja magyarázni a GPU-architektúrát, a CUDA-modellt, a memóriatereket és a teljesítmény tényezőit.
- **Alkalmazza:** önállóan meg tud írni, lefordítani, tesztelni és profilozni egy CUDA-kernelet, és értelmezi a mérést.
- **Megtervezi:** adott feladathoz helyes kernel-, memória- és launch-stratégiát választ, megindokolja a döntést, ismeri a korlátokat, és össze tudja hasonlítani a baseline-nal.

A záróprojekt értékelése négy dimenzióban történik: helyesség és teszteltség, teljesítmény és erőforrás-használat, numerikus pontosság és reprodukálhatóság, valamint dokumentáció és bemutató. A projekt csak akkor fogadható el, ha a mért és a várt eredményt elkülöníti, és a nem futtatott mérést, benchmarkot vagy hardveres eredményt egyértelműen jelöli.

Bibliográfia és forráskulcsok

Ezek a tervezéshez használt kiindulópontok, nem ebben a könyvben tételesen ellenőrzött bizonyítékok. A fejezetek kutatási lépése a CUDA-verziót, a GPU-architektúrát, a könyvtárverziót és az ellenőrzési dátumot rögzíti. A teljesítményállításoknál a saját, reprodukálható mérés az elsődleges; a gyártói dokumentáció a szemantika és az API forrása.

CUDA-dokumentáció és alapok

- https://docs.nvidia.com/cuda/ (CUDA dokumentáció)
- https://docs.nvidia.com/cuda/cuda-c-programming-guide/ (CUDA C++ Programming Guide)
- https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/ (CUDA Best Practices Guide)
- https://docs.nvidia.com/cuda/cuda-quick-start-guide/ (CUDA Quick Start)
- https://docs.nvidia.com/cuda/cuda-compiler-driver-nvcc/ (nvcc)
- https://docs.nvidia.com/cuda/parallel-thread-execution/ (PTX ISA)
- https://docs.nvidia.com/cuda/cuda-math-api/ (CUDA Math API)
- https://github.com/NVIDIA/cuda-samples (CUDA Samples)
- https://developer.nvidia.com/blog/ (NVIDIA Developer Blog)
- https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#hardware-implementation (Hardware implementation)

Könyvek és háttéranyagok

- https://www.elsevier.com/books/programming-massively-parallel-processors/hwu/9780323912310 (Programming Massively Parallel Processors)
- https://queue.acm.org/detail.cfm?id=1404 (Scalable Parallel Programming with CUDA)
- https://dl.acm.org/doi/10.1145/1498765.1498785 (Roofline model)
- https://arxiv.org/abs/1804.06826 (Dissecting the NVIDIA Volta GPU Architecture)
- https://arxiv.org/abs/2209.05433 (FP8 formats for deep learning)
- https://dl.acm.org/doi/10.1145/103162.103163 (Goldberg: What Every Computer Scientist Should Know About Floating-Point Arithmetic)
- https://ieeexplore.ieee.org/document/8766229 (IEEE 754-2019)
- https://www.nvidia.com/en-us/data-center/resources/ (NVIDIA adatközponti források)

CUDA-könyvtárak

- https://docs.nvidia.com/cuda/cublas/ (cuBLAS)
- https://docs.nvidia.com/cuda/cusparse/ (cuSPARSE)
- https://docs.nvidia.com/cuda/cufft/ (cuFFT)
- https://docs.nvidia.com/cuda/curand/ (cuRAND)
- https://docs.nvidia.com/cuda/thrust/ (Thrust)
- https://github.com/NVIDIA/cub (CUB)
- https://github.com/NVIDIA/cutlass (CUTLASS)
- https://docs.nvidia.com/deeplearning/cudnn/ (cuDNN)
- https://docs.nvidia.com/deeplearning/nccl/ (NCCL)
- https://docs.nvidia.com/cuda/gpudirect-rdma/ (GPUDirect RDMA)
- https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#um-unified-memory-programming (Unified memory)
- https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#cuda-graphs (CUDA Graphs)
- https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#cooperative-groups (Cooperative Groups)
- https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#thread-block-clusters (Thread block clusters)
- https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#wmma (WMMA)

Profilozás, debugging és mérés

- https://docs.nvidia.com/nsight-compute/ (Nsight Compute)
- https://docs.nvidia.com/nsight-systems/ (Nsight Systems)
- https://docs.nvidia.com/compute-sanitizer/ (Compute Sanitizer)
- https://docs.nvidia.com/cuda/cuda-gdb/ (cuda-gdb)
- https://docs.nvidia.com/cupti/ (CUPTI)
- https://docs.nvidia.com/deploy/nvidia-smi/ (nvidia-smi)
- https://developer.nvidia.com/nsight-graphics (Nsight Graphics)
- https://github.com/google/benchmark (Google Benchmark)

TPU, XLA és JAX

- https://arxiv.org/abs/1704.04760 (In-Datacenter Performance Analysis of a Tensor Processing Unit)
- https://cloud.google.com/tpu/docs (Cloud TPU dokumentáció)
- https://openxla.org/xla (XLA)
- https://jax.readthedocs.io/ (JAX)
- https://jax-ml.github.io/scaling-book/ (JAX Scaling Book)
- https://docs.jax.dev/ (JAX dokumentáció)
- https://github.com/google/jax (JAX forrás)
- https://arxiv.org/abs/2304.01433 (TPU v4)
- https://cloud.google.com/blog/products/ai-machine-learning (Google Cloud AI blog)

OpenCL, SYCL és HIP

- https://www.khronos.org/opencl/ (OpenCL)

- https://registry.khronos.org/OpenCL/ (OpenCL specifikáció)
- https://www.khronos.org/sycl/ (SYCL)
- https://www.intel.com/content/www/us/en/developer/tools/opencl-sdk/overview.html (Intel OpenCL)
- https://rocm.docs.amd.com/projects/HIP/ (HIP)
- https://github.com/ROCm/HIP (HIP forrás)
- https://www.khronos.org/registry/SYCL/ (SYCL registry)
- https://github.com/KhronosGroup/SYCL-Docs (SYCL dokumentáció)
- https://www.intel.com/content/www/us/en/developer/tools/oneapi/overview.html (oneAPI)

Teljesítmény, numerika és architektúra

- https://www.agner.org/optimize/ (Agner Fog)
- https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html (Intel Intrinsics Guide)
- https://arxiv.org/abs/2205.14135 (FlashAttention)
- https://www.nvidia.com/en-us/data-center/nvlink/ (NVLink)
- https://www.nvidia.com/en-us/data-center/hgx/ (HGX)

Közösség és oktatás

- https://www.nvidia.com/en-us/gtc/ (GTC)
- https://developer.nvidia.com/accelerated-computing-teaching-kit (Teaching Kit)
- https://www.olcf.ornl.gov/ (Oak Ridge Leadership Computing Facility)
- https://www.nersc.gov/ (NERSC)
- https://www.vik.bme.hu/ (BME Villamosmérnöki és Informatikai Kar)
- https://www.inf.elte.hu/ (ELTE Informatikai Kar)
- https://www.bme.hu/ (Budapesti Műszaki és Gazdaságtudományi Egyetem)